Kunpeng HPCKit 26.1.RC1

开发指南

文档版本	01
发布日期	2026-06-30



华为技术有限公司

版权所有 © 华为技术有限公司 2026。 保留一切权利。

非经本公司书面许可，任何单位和个人不得擅自摘抄、复制本文档内容的部分或全部，并不得以任何形式传播。

商标声明


和其他华为商标均为华为技术有限公司的商标。
本文档提及的其他所有商标或注册商标，由各自的所有人拥有。

注意

您购买的产品、服务或特性等应受华为公司商业合同和条款的约束，本文档中描述的全部或部分产品、服务或特性可能不在您的购买或使用范围之内。除非合同另有约定，华为公司对本文档内容不做任何明示或暗示的声明或保证。

由于产品版本升级或其他原因，本文档内容会不定期进行更新。除非另有约定，本文档仅作为使用指导，本文档中的所有陈述、信息和建议不构成任何明示或暗示的担保。

安全声明

漏洞处理流程

华为公司对产品漏洞管理的规定以“漏洞处理流程”为准，该流程的详细内容请参见如下网址：
https://www.huawei.com/cn/psirt/vul-response-process
如企业客户须获取漏洞信息，请参见如下网址：
https://securitybulletin.huawei.com/enterprise/cn/security-advisory

 版权所有 © 华为技术有限公司

目 录

1 HPCKit 介绍.....1
2 HPCKit 架构.....3
3 编译器介绍.....4
3.1 毕昇编译器.....4
3.2 GCC for openEuler.....5
4 Hyper MPI 开发指南.....6
4.1 产品描述.....6
4.1.1 产品介绍.....6
4.1.2 软件架构.....9
4.1.3 关键特性.....10
4.2 操作指南.....11
4.2.1 提交 MPI 作业.....11
4.2.2 运行 MPI 作业.....11
4.2.3 集合操作算法选择说明.....28
4.2.3.1 指定算法说明.....28
4.2.3.2 推荐命令.....31
4.2.3.3 特性支持与功能回退说明.....31
4.2.4 KuComm 使用说明.....34
4.2.5 命令说明及示例.....34
4.2.5.1 传输模式选择说明.....34
4.2.5.2 进程分布.....35
4.2.5.3 关闭 OOB 功能.....35
4.2.5.4 多网络平面场景使用说明.....35
4.2.5.5 HUCX RC 多网口容错.....36
4.2.5.6 启动时延优化.....37
4.2.5.7 命令示例.....38
4.2.5.8 SDMA 使用场景说明.....39
4.2.5.9 Stars 使用场景说明.....39
4.2.5.10 MPI 超时日志分析脚本使用说明.....39
4.3 FAQ.....40
4.3.1 故障处理流程.....40
4.3.2 作业进程数过大，报错：There are not enough slots available.....40

 版权所有 © 华为技术有限公司

4.3.3 多节点运行 mpirun 时异常 42
4.3.4 多节点运行 mpirun 时报某个文件无法找到或打开 42
4.3.5 运行 MPI 作业提示网络设备内存无法分配 43
4.3.6 指定网卡名称错误，报错：network device '***' is not available 44
4.3.7 指定网络类型错误，提示指定的网络类型不可用 44
4.3.8 指定主机名错误，报错：Name or service not known 45
4.3.9 通信过程建链报错：ucp_ep_create(...)failed：Destination is unreachable 47
4.3.10 在-x UCX_IB_GID_INDEX 命令中指定 GID 后报错 47
4.3.11 找不到 ARP 表项引起的超时报错：ibv_create_ah...failed: Connection timed out 48
4.3.12 由于网卡配置不当引起的超时报错：connect... failed：Connection timed out 49
4.3.13 由于丢包等网络问题造成的 UCP request timeout 超时报错 50
4.3.14 由于丢包等网络原因引起的 UD 超时报错：UD endpoint ..... unhandled timeout error 51
4.3.15 运行报错 failed to bind memory，但应用仍可以正常结束 52
4.3.16 Allgatherv 算法 4 指定 TCP 传输，np 大规格执行报错 53
4.3.17 Allreduce 算法 8 精度问题导致多次执行结果不一致 53
4.3.18 超线程环境中 rank 进程未绑定至逻辑核上 54
4.3.19 UCX 版本不匹配告警 54
4.3.20 UCP 请求超时 55
4.3.21 运行 osu_alltoall/osu_alltoallv/osu_alltoallw/osu_ialltoall/osu_ialltoallv/osu_ialltoallw 时出现内存耗尽问题 55
4.3.22 运行 Alltoallv 算法 2 报错：gatherv phase don't support displs >= 2^31 bytes 56
4.3.23 环境 open files 值较大时导致 Hyper MPI 启动慢 56
4.3.24 UCX 使用 SDMA 时，报错：UCX ERROR Failed to create sdma_device[0] handle 57
4.3.25 运行 MPI 作业，指定传输协议为 RC 或者 UD 时创建 qp 报错：Invalid argument 58
4.3.26 运行 MPI 作业，指定传输协议为 RC 或者 UD 时创建 qp 报错：No space left on device 58
4.3.27 运行 MPI 作业，并启用 Stars 集合通信卸载，指定 Stars 传输协议为 rc_acc 时报错：ibv_wr_complete fail, ret 12 59
4.3.28 运行 MPI 作业，网卡驱动报错：transport retry counter exceeded 59
4.3.29 管理、计算网卡都使用板载网卡并安装 RDMA 驱动，跑多节点 OSU 集合通信测试不指定网卡报错 60
4.3.30 满核运行集合通信(i)osu_alltoall(v)/(i)osu_scatter(v)/(i)osu_gather(v)节点卡住挂死 60
4.3.31 在 MPI 线程中使用 fork 方法新建线程，导致运行时报错 61
4.4 算法地图 61
4.5 命令参数 66
4.5.1 Hyper MPI 26.1.RC1 及以上版本命令参数 66
4.5.2 Hyper MPI 25.2.1 及以上版本命令参数 67
4.5.3 Hyper MPI 25.1.0.SPC001 及以上版本命令参数 67
4.5.4 Hyper MPI 25.1.RC1 及以上版本命令参数 68
4.5.5 Hyper MPI 25.0.0 及以上版本命令参数 69
4.5.6 Hyper MPI 2.4.3 及以上版本命令参数 81
4.5.7 Hyper MPI 1.3.*及以上版本命令参数 82
4.5.8 Hyper MPI 1.2.*及以下版本命令参数 93
4.6 术语&缩略语 98
5 KUPL 开发指南 99

 版权所有 © 华为技术有限公司

5.1 概念介绍 99
5.2 使用指导 99
5.2.1 使用 KUPL 加速 KML 直接求解法 99
5.2.2 使用 KUPL API 进行并行计算加速 100
5.2.3 使用 KUPL PROF 进行全量的函数时序统计 101
5.3 KUPL 库函数说明 102
5.3.1 函数说明 102
5.3.2 函数定义 102
5.3.2.1 返回值定义 102
5.3.2.2 executor 相关函数 102
5.3.2.2.1 概念说明 102
5.3.2.2.2 环境变量 102
5.3.2.2.3 kupl_get_num_executors 103
5.3.2.2.4 kupl_get_executor_num 103
5.3.2.2.5 kupl_egroup_create 104
5.3.2.2.6 kupl_egroup_destroy 105
5.3.2.2.7 kupl_egroup_borrow 106
5.3.2.2.8 kupl_egroup_return 107
5.3.2.2.9 kupl_egroup_reset 108
5.3.2.2.10 kupl_egroup_barrier 108
5.3.2.2.11 kupl_egroup_fork_barrier 110
5.3.2.2.12 kupl_egroup_join_barrier 111
5.3.2.3 多线程编程函数 112
5.3.2.3.1 概念说明 112
5.3.2.3.2 kupl_parallel_for 112
5.3.2.3.3 kupl::parallel_for 118
5.3.2.3.4 kupl_parallel_for_reduce 125
5.3.2.3.5 kupl_in_parallel 128
5.3.2.4 计算图编程函数 129
5.3.2.4.1 概念说明 129
5.3.2.4.2 kupl_graph_create 129
5.3.2.4.3 kupl_graph_destroy 130
5.3.2.4.4 kupl_graph_submit 130
5.3.2.4.5 kupl::graph_submit(task) 136
5.3.2.4.6 kupl::graph_submit(taskloop) 139
5.3.2.4.7 kupl_graph_wait 141
5.3.2.4.8 kupl_sgraph_create 142
5.3.2.4.9 kupl_sgraph_destroy 143
5.3.2.4.10 kupl_sgraph_add_node 143
5.3.2.4.11 kupl::sgraph_add_node 145
5.3.2.4.12 kupl_sgraph_add_dep 148
5.3.2.5 多队列多流编程函数 149

 版权所有 © 华为技术有限公司

5.3.2.5.1 概念说明 149
5.3.2.5.2 kupl_queue_create 149
5.3.2.5.3 kupl_get_queue_priority_range 150
5.3.2.5.4 kupl_queue_create_with_priority 151
5.3.2.5.5 kupl_queue_destroy 151
5.3.2.5.6 kupl_queue_wait 152
5.3.2.5.7 kupl_queue_wait_event 153
5.3.2.5.8 kupl_queue_submit 154
5.3.2.5.9 kupl::queue_submit(item) 156
5.3.2.5.10 kupl::queue_submit(kernel) 158
5.3.2.5.11 kupl_event_create 160
5.3.2.5.12 kupl_event_destroy 160
5.3.2.5.13 kupl_event_record 161
5.3.2.5.14 kupl_event_wait 162
5.3.2.5.15 kupl_event_query 163
5.3.2.5.16 kupl_queue_acquire 163
5.3.2.5.17 kupl_queue_wait_all 164
5.3.2.6 内存管理函数 165
5.3.2.6.1 概念说明 165
5.3.2.6.2 环境变量 165
5.3.2.6.3 kupl_malloc 165
5.3.2.6.4 kupl_free 167
5.3.2.6.5 kupl_mlock 167
5.3.2.6.6 kupl_munlock 168
5.3.2.6.7 kupl_set_kernel_concurrency 169
5.3.2.6.8 kupl_get_kernel_concurrency 170
5.3.2.6.9 kupl_set_kernel_concurrency_local 171
5.3.2.6.10 kupl_get_kernel_concurrency_local 172
5.3.2.6.11 kupl_memcpy 172
5.3.2.6.12 kupl_memcpy2d 174
5.3.2.6.13 kupl_memcpy_async 176
5.3.2.6.14 kupl_memcpy2d_async 178
5.3.2.6.15 kupl_hbw_malloc 180
5.3.2.6.16 kupl_hbw_free 181
5.3.2.6.17 kupl_hbw_verify 182
5.3.2.6.18 kupl_hbw_check_available 183
5.3.2.6.19 kupl_hbw_get_policy 183
5.3.2.6.20 kupl_hbw_set_policy 184
5.3.2.6.21 kupl_mem_copyin 185
5.3.2.6.22 kupl_mem_copyout 187
5.3.2.6.23 kupl_mem_query 189
5.3.2.6.24 kupl_mem_is_present 190

 版权所有 © 华为技术有限公司

5.3.2.7 共享内存通信函数 191
5.3.2.7.1 概念说明 191
5.3.2.7.2 kupl_shm_comm_create 191
5.3.2.7.3 kupl_shm_comm_destroy 193
5.3.2.7.4 kupl_shm_comm_rank 195
5.3.2.7.5 kupl_shm_comm_size 196
5.3.2.7.6 kupl_shm_win_alloc 197
5.3.2.7.7 kupl_shm_win_query 200
5.3.2.7.8 kupl_shm_win_free 202
5.3.2.7.9 kupl_shm_attach 204
5.3.2.7.10 kupl_shm_detach 206
5.3.2.7.11 kupl_shm_fence 208
5.3.2.7.12 kupl_shm_peer_fence 210
5.3.2.7.13 kupl_shm_allreduce_init 212
5.3.2.7.14 kupl_shm_bcast_init 215
5.3.2.7.15 kupl_shm_alltoall_init 218
5.3.2.7.16 kupl_shm_request_start 221
5.3.2.7.17 kupl_shm_request_wait 223
5.3.2.7.18 kupl_shm_request_free 226
5.3.2.7.19 kupl_shm_allreduce_batch_init 228
5.3.2.7.20 kupl_shm_info_set 231
5.3.2.8 矩阵编程接口函数 232
5.3.2.8.1 概念说明 233
5.3.2.8.2 Tensor 233
5.3.2.8.3 make_coord 235
5.3.2.8.4 make_shape 237
5.3.2.8.5 make_stride 238
5.3.2.8.6 make_layout 239
5.3.2.8.7 make_tensor 241
5.3.2.8.8 make_tiled_mma 242
5.3.2.8.9 make_tiled_store 244
5.3.2.8.10 make_tiled_copy 246
5.3.2.8.11 mma 250
5.3.2.8.12 store 252
5.3.2.8.13 copy(Trans) 254
5.3.2.8.14 copy(Prefetch) 255
5.3.2.9 公共函数 257
5.3.2.9.1 kupl_get_version 257
5.3.2.9.2 kupl_get_wtime 257
6 鲲鹏数学库 开发指南 259
6.1 总述 259
6.2 安装配置环境 262

 版权所有 © 华为技术有限公司

6.2.1 软硬件要求 262
6.2.2 获取软件包 263
6.2.3 如何使用 KML 264
6.2.3.1 安装 KML 264
6.2.3.2 设置环境变量 264
6.2.3.3 使用方式 265
6.2.3.4 卸载 KML 266
6.3 使用说明 266
6.4 基础数学库 267
6.4.1 KML_SPBLAS 库函数说明 267
6.4.1.1 函数说明 267
6.4.1.2 数据类型与数据存储 267
6.4.1.2.1 数据类型 267
6.4.1.2.2 数据存储 270
6.4.1.3 函数定义 273
6.4.1.3.1 Sparse BLAS Level 1 函数 273
6.4.1.3.1.1 函数说明 273
6.4.1.3.1.2 kml_sparse_?axpyi 273
6.4.1.3.1.3 kml_sparse_?doti 275
6.4.1.3.1.4 kml_sparse_?dotci_sub 277
6.4.1.3.1.5 kml_sparse_?dotui_sub 279
6.4.1.3.1.6 kml_sparse_?gthr 281
6.4.1.3.1.7 kml_sparse_?gthrz 283
6.4.1.3.1.8 kml_sparse_?roti 285
6.4.1.3.1.9 kml_sparse_?sctr 286
6.4.1.3.2 Sparse BLAS Level 2 和 Level 3 函数 288
6.4.1.3.2.1 函数说明 288
6.4.1.3.2.2 kml_sparse_?csrgemv 291
6.4.1.3.2.3 kml_sparse_?csrsymv 294
6.4.1.3.2.4 kml_csparse_?csrgemv 296
6.4.1.3.2.5 kml_csparse_?csrsymv 299
6.4.1.3.2.6 kml_sparse_?csrmv 302
6.4.1.3.2.7 kml_sparse_?csrsv 305
6.4.1.3.2.8 kml_sparse_?csrmm 309
6.4.1.3.2.9 kml_sparse_?csradd 314
6.4.1.3.2.10 kml_sparse_?csrmultcsr 319
6.4.1.3.2.11 kml_sparse_?csrmultd 323
6.4.1.3.2.12 kml_sparse_?csrtrsv 326
6.4.1.3.2.13 kml_csparse_?csrtrsv 329
6.4.1.3.2.14 kml_sparse_?cscsv 332
6.4.1.3.2.15 kml_sparse_?cscmv 335
6.4.1.3.2.16 kml_sparse_?cscmm 338

 版权所有 © 华为技术有限公司

6.4.1.3.2.17 kml_sparse_?cscsm..... 343
6.4.1.3.2.18 kml_sparse_?csrsm..... 347
6.4.1.3.3 Support 函数..... 351
6.4.1.3.3.1 kml_set_thread_num..... 351
6.4.1.3.3.2 KSPBLASGetVersion..... 351
6.4.2 KML_VML 库函数说明..... 352
6.4.2.1 函数说明..... 352
6.4.2.2 环境变量..... 356
6.4.2.3 函数定义..... 356
6.4.2.3.1 v?add..... 356
6.4.2.3.2 v?sub..... 359
6.4.2.3.3 v?sqr..... 361
6.4.2.3.4 v?mul..... 363
6.4.2.3.5 v?div..... 365
6.4.2.3.6 v?sqrt..... 368
6.4.2.3.7 v?cbrt..... 371
6.4.2.3.8 v?abs..... 372
6.4.2.3.9 v?inv..... 374
6.4.2.3.10 v?invcbrt..... 375
6.4.2.3.11 v?invsqrt..... 376
6.4.2.3.12 v?linearfrac..... 378
6.4.2.3.13 v?remainder..... 380
6.4.2.3.14 v?pow..... 381
6.4.2.3.15 v?powx..... 383
6.4.2.3.16 v?powr..... 385
6.4.2.3.17 v?pow2o3..... 387
6.4.2.3.18 v?pow3o2..... 388
6.4.2.3.19 v?hypot..... 389
6.4.2.3.20 v?exp..... 391
6.4.2.3.21 v?exp2..... 393
6.4.2.3.22 v?exp10..... 394
6.4.2.3.23 v?expm1..... 396
6.4.2.3.24 v?ln..... 397
6.4.2.3.25 v?log10..... 399
6.4.2.3.26 v?log1p..... 401
6.4.2.3.27 v?log2..... 402
6.4.2.3.28 v?logb..... 404
6.4.2.3.29 v?cos..... 405
6.4.2.3.30 v?cospi..... 407
6.4.2.3.31 v?sin..... 408
6.4.2.3.32 v?sinpi..... 410
6.4.2.3.33 v?sincos..... 411

 版权所有 © 华为技术有限公司

6.4.2.3.34 v?tan....413
6.4.2.3.35 v?atan....415
6.4.2.3.36 v?atanpi....417
6.4.2.3.37 v?atan2....418
6.4.2.3.38 v?atan2pi....420
6.4.2.3.39 v?sinh....422
6.4.2.3.40 v?cosh....424
6.4.2.3.41 v?tanh....426
6.4.2.3.42 v?asinh....428
6.4.2.3.43 v?acosh....430
6.4.2.3.44 v?atanh....432
6.4.2.3.45 v?sind....434
6.4.2.3.46 v?cosd....435
6.4.2.3.47 v?trunc....437
6.4.2.3.48 v?round....438
6.4.2.3.49 v?nearbyint....439
6.4.2.3.50 v?rint....440
6.4.2.3.51 v?frac....442
6.4.2.3.52 v?fmod....443
6.4.2.3.53 v?arg....444
6.4.2.3.54 v?conj....446
6.4.2.3.55 v?mulbyconj....447
6.4.2.3.56 v?cis....449
6.4.2.3.57 v?acos....450
6.4.2.3.58 v?asin....454
6.4.2.3.59 v?tanpi....458
6.4.2.3.60 v?acospi....459
6.4.2.3.61 v?asinpi....461
6.4.2.3.62 v?tand....462
6.4.2.3.63 v?erf....463
6.4.2.3.64 v?erfc....464
6.4.2.3.65 v?cdfnorm....466
6.4.2.3.66 v?erfinv....467
6.4.2.3.67 v?erfcinv....468
6.4.2.3.68 v?cdfnorminv....470
6.4.2.3.69 v?lgamma....471
6.4.2.3.70 v?tgamma....472
6.4.2.3.71 v?expint1....474
6.4.2.3.72 v?floor....475
6.4.2.3.73 v?ceil....476
6.4.2.3.74 v?modf....478
6.4.2.3.75 v?copysign....479

 版权所有 © 华为技术有限公司

6.4.2.3.76 v?nextafter 480
6.4.2.3.77 v?fdim 482
6.4.2.3.78 v?fmax 483
6.4.2.3.79 v?fmin 485
6.4.2.3.80 v?maxmag 487
6.4.2.3.81 v?minmag 488
6.4.2.3.82 vfloatcvtf16 490
6.4.2.3.83 vf16cvtfloat 490
6.4.2.3.84 KVMLGetVersion 491
6.4.2.3.85 KmlVmlSetNumThreads 492
6.4.2.3.86 KmlVmlSetNumThreadsLocal 493
6.4.2.3.87 KmlVmlGetMaxThreads 493
6.4.3 KML_MATH 库函数说明 493
6.4.3.1 函数说明 494
6.4.3.2 函数定义 502
6.4.3.2.1 三角函数 502
6.4.3.2.1.1 sin 502
6.4.3.2.1.2 sind 504
6.4.3.2.1.3 sinpi 505
6.4.3.2.1.4 asin 506
6.4.3.2.1.5 asind 508
6.4.3.2.1.6 asinpi 509
6.4.3.2.1.7 cos 510
6.4.3.2.1.8 cosd 512
6.4.3.2.1.9 cospi 513
6.4.3.2.1.10 acos 514
6.4.3.2.1.11 acosd 516
6.4.3.2.1.12 acospi 517
6.4.3.2.1.13 sincos 518
6.4.3.2.1.14 sincosd 520
6.4.3.2.1.15 tan 522
6.4.3.2.1.16 tanpi 523
6.4.3.2.1.17 tand 524
6.4.3.2.1.18 atan 526
6.4.3.2.1.19 atand 527
6.4.3.2.1.20 atanpi 528
6.4.3.2.1.21 atan2 530
6.4.3.2.1.22 atan2d 532
6.4.3.2.1.23 atan2pi 534
6.4.3.2.1.24 cot 536
6.4.3.2.1.25 cotd 537
6.4.3.2.2 双曲函数 538

 版权所有 © 华为技术有限公司

6.4.3.2.2.1 sinh 539
6.4.3.2.2.2 cosh 540
6.4.3.2.2.3 sinhcosh 542
6.4.3.2.2.4 tanh 543
6.4.3.2.2.5 asinh 545
6.4.3.2.2.6 acosh 546
6.4.3.2.2.7 atanh 547
6.4.3.2.3 指数和对数函数 549
6.4.3.2.3.1 exp 549
6.4.3.2.3.2 exp2 550
6.4.3.2.3.3 exp10 552
6.4.3.2.3.4 expm1 553
6.4.3.2.3.5 frexp 554
6.4.3.2.3.6 ldexp 555
6.4.3.2.3.7 hypot 556
6.4.3.2.3.8 log 558
6.4.3.2.3.9 log2 559
6.4.3.2.3.10 log10 561
6.4.3.2.3.11 log1p 562
6.4.3.2.3.12 logb 564
6.4.3.2.3.13 ilogb 565
6.4.3.2.3.14 scalb 566
6.4.3.2.3.15 scalbln 567
6.4.3.2.3.16 scalbn 568
6.4.3.2.4 幂和根函数 570
6.4.3.2.4.1 pow 570
6.4.3.2.4.2 powr 571
6.4.3.2.4.3 powr?i?/powi?i?/powc?i? 575
6.4.3.2.4.4 pow2o3 576
6.4.3.2.4.5 pow3o2 577
6.4.3.2.4.6 sqrt 578
6.4.3.2.4.7 invsqrt 579
6.4.3.2.4.8 cbrt 580
6.4.3.2.5 杂项函数 582
6.4.3.2.5.1 fabs 582
6.4.3.2.5.2 fdim 583
6.4.3.2.5.3 fmax 584
6.4.3.2.5.4 fmaxmag 585
6.4.3.2.5.5 fmin 586
6.4.3.2.5.6 fminmag 588
6.4.3.2.5.7 nan 589
6.4.3.2.5.8 nextafter 590

 版权所有 © 华为技术有限公司

6.4.3.2.5.9 nexttoward..... 591
6.4.3.2.5.10 significand..... 592
6.4.3.2.5.11 finite..... 593
6.4.3.2.6 特殊函数..... 594
6.4.3.2.6.1 annuity..... 594
6.4.3.2.6.2 compound..... 596
6.4.3.2.6.3 erf..... 598
6.4.3.2.6.4 erfc..... 599
6.4.3.2.6.5 erfcx..... 600
6.4.3.2.6.6 erfinv..... 602
6.4.3.2.6.7 gamma..... 603
6.4.3.2.6.8 gamma_r..... 604
6.4.3.2.6.9 lgamma..... 606
6.4.3.2.6.10 lgamma_r..... 607
6.4.3.2.6.11 tgamma..... 608
6.4.3.2.6.12 j0..... 610
6.4.3.2.6.13 j1..... 611
6.4.3.2.6.14 jn..... 612
6.4.3.2.6.15 y0..... 613
6.4.3.2.6.16 y1..... 614
6.4.3.2.6.17 yn..... 615
6.4.3.2.7 舍入函数..... 616
6.4.3.2.7.1 ceil..... 616
6.4.3.2.7.2 floor..... 617
6.4.3.2.7.3 rint..... 618
6.4.3.2.7.4 lrint..... 619
6.4.3.2.7.5 llrint..... 620
6.4.3.2.7.6 round..... 621
6.4.3.2.7.7 lround..... 623
6.4.3.2.7.8 llround..... 624
6.4.3.2.7.9 modf..... 625
6.4.3.2.7.10 nearbyint..... 626
6.4.3.2.7.11 trunc..... 627
6.4.3.2.8 取余函数..... 628
6.4.3.2.8.1 drem..... 628
6.4.3.2.8.2 fmod..... 629
6.4.3.2.8.3 remainder..... 630
6.4.3.2.8.4 remquo..... 632
6.4.3.2.9 复数函数..... 633
6.4.3.2.9.1 cabs..... 633
6.4.3.2.9.2 carg..... 635
6.4.3.2.9.3 cdiv..... 638

 版权所有 © 华为技术有限公司

6.4.3.2.9.4 cexp....639
6.4.3.2.9.5 cexp2....642
6.4.3.2.9.6 cexp10....646
6.4.3.2.9.7 cpow....649
6.4.3.2.9.8 csqrt....650
6.4.3.2.9.9 clog....652
6.4.3.2.9.10 clog2....655
6.4.3.2.9.11 clog10....659
6.4.3.2.9.12 ccos....663
6.4.3.2.9.13 ccosh....665
6.4.3.2.9.14 cacos....668
6.4.3.2.9.15 cacosh....671
6.4.3.2.9.16 csin....674
6.4.3.2.9.17 csinh....677
6.4.3.2.9.18 casin....679
6.4.3.2.9.19 casinh....683
6.4.3.2.9.20 ctan....686
6.4.3.2.9.21 ctanh....689
6.4.3.2.9.22 catan....691
6.4.3.2.9.23 catanh....694
6.4.3.2.9.24 cis....695
6.4.3.2.9.25 cisd....697
6.4.3.2.9.26 conj....698
6.4.3.2.9.27 cproj....699
6.4.3.2.9.28 creal....700
6.4.3.2.9.29 cimag....701
6.4.3.2.10 Support 函数....702
6.4.3.2.10.1 KMGetVersion....702
6.4.4 KML_FFT 库函数说明....703
6.4.4.1 函数说明....703
6.4.4.2 数据结构....704
6.4.4.3 环境变量....705
6.4.4.4 单进程函数定义....705
6.4.4.4.1 C2C 变换....705
6.4.4.4.1.1 kml_fft(f/h)_plan_dft....705
6.4.4.4.1.2 kml_fft(f/h)_plan_dft_1d....708
6.4.4.4.1.3 kml_fft(f/h)_plan_dft_2d....710
6.4.4.4.1.4 kml_fft(f/h)_plan_dft_3d....713
6.4.4.4.1.5 kml_fft(f/h)_plan_guru_dft....716
6.4.4.4.1.6 kml_fft(f/h)_plan_guru_split_dft....720
6.4.4.4.1.7 kml_fft(f/h)_plan_guru64_dft....724
6.4.4.4.1.8 kml_fft(f/h)_plan_guru64_split_dft....728

 版权所有 © 华为技术有限公司

6.4.4.4.1.9 kml_fft(f/h)_plan_many_dft 732
6.4.4.4.1.10 kml_ffth_plan_dft_scale 735
6.4.4.4.1.11 kml_ffth_plan_dft_scale_1d 737
6.4.4.4.1.12 kml_ffth_plan_dft_scale_2d 738
6.4.4.4.1.13 kml_ffth_plan_many_dft_scale 740
6.4.4.4.2 R2C 变换 742
6.4.4.4.2.1 kml_fft(f/h)_plan_dft_r2c 742
6.4.4.4.2.2 kml_fft(f/h)_plan_dft_r2c_1d 744
6.4.4.4.2.3 kml_fft(f/h)_plan_dft_r2c_2d 747
6.4.4.4.2.4 kml_fft(f/h)_plan_dft_r2c_3d 749
6.4.4.4.2.5 kml_fft(f/h)_plan_guru_dft_r2c 751
6.4.4.4.2.6 kml_fft(f/h)_plan_guru_split_dft_r2c 755
6.4.4.4.2.7 kml_fft(f/h)_plan_guru64_dft_r2c 759
6.4.4.4.2.8 kml_fft(f/h)_plan_guru64_split_dft_r2c 762
6.4.4.4.2.9 kml_fft(f/h)_plan_many_dft_r2c 766
6.4.4.4.2.10 kml_ffth_plan_dft_scale_r2c 769
6.4.4.4.2.11 kml_ffth_plan_dft_scale_r2c_1d 771
6.4.4.4.2.12 kml_ffth_plan_dft_scale_r2c_2d 772
6.4.4.4.2.13 kml_ffth_plan_many_dft_scale_r2c 773
6.4.4.4.3 C2R 变换 775
6.4.4.4.3.1 kml_fft(f/h)_plan_dft_c2r 776
6.4.4.4.3.2 kml_fft(f/h)_plan_dft_c2r_1d 778
6.4.4.4.3.3 kml_fft(f/h)_plan_dft_c2r_2d 780
6.4.4.4.3.4 kml_fft(f/h)_plan_dft_c2r_3d 782
6.4.4.4.3.5 kml_fft(f/h)_plan_guru_dft_c2r 785
6.4.4.4.3.6 kml_fft(f/h)_plan_guru_split_dft_c2r 788
6.4.4.4.3.7 kml_fft(f/h)_plan_guru64_dft_c2r 792
6.4.4.4.3.8 kml_fft(f/h)_plan_guru64_split_dft_c2r 796
6.4.4.4.3.9 kml_fft(f/h)_plan_many_dft_c2r 799
6.4.4.4.3.10 kml_ffth_plan_dft_scale_c2r 803
6.4.4.4.3.11 kml_ffth_plan_dft_scale_c2r_1d 804
6.4.4.4.3.12 kml_ffth_plan_dft_scale_c2r_2d 806
6.4.4.4.3.13 kml_ffth_plan_many_dft_scale_c2r 807
6.4.4.4.4 R2R 变换 809
6.4.4.4.4.1 kml_fft(f/h)_plan_r2r 809
6.4.4.4.4.2 kml_fft(f/h)_plan_r2r_1d 812
6.4.4.4.4.3 kml_fft(f/h)_plan_r2r_2d 814
6.4.4.4.4.4 kml_fft(f/h)_plan_r2r_3d 817
6.4.4.4.4.5 kml_fft(f/h)_plan_guru_r2r 821
1. guru r2r 矩阵转置功能 821
2. kml_fft(f/h)_plan_guru_r2r 详情 822
6.4.4.4.4.6 kml_fft(f/h)_plan_guru64_r2r 826

 版权所有 © 华为技术有限公司

6.4.4.4.4.7 kml_fft(f/h)_plan_many_r2r....830
6.4.4.4.5 变换执行函数....833
6.4.4.4.5.1 kml_fft(f/h)_execute....833
6.4.4.4.5.2 kml_fft(f/h)_execute_dft....835
6.4.4.4.5.3 kml_fft(f/h)_execute_dft_r2c....836
6.4.4.4.5.4 kml_fft(f/h)_execute_dft_c2r....838
6.4.4.4.5.5 kml_fft(f/h)_execute_r2r....839
6.4.4.4.5.6 kml_fft(f/h)_execute_split_dft....840
6.4.4.4.5.7 kml_fft(f/h)_execute_split_dft_r2c....841
6.4.4.4.5.8 kml_fft(f/h)_execute_split_dft_c2r....842
6.4.4.4.5.9 kml_ffth_execute_dft_scale....843
6.4.4.4.5.10 kml_ffth_execute_dft_scale_r2c....844
6.4.4.4.5.11 kml_ffth_execute_dft_scale_c2r....845
6.4.4.4.6 Upsampling 变换....845
6.4.4.4.6.1 kml_fft(f/h)_plan_upsampling_2d....845
6.4.4.5 多进程函数定义....847
6.4.4.5.1 C2C 变换....847
6.4.4.5.1.1 kml_fft(f/h)_mpi_plan_dft....847
6.4.4.5.1.2 kml_fft(f/h)_mpi_plan_dft_1d....850
6.4.4.5.1.3 kml_fft(f/h)_mpi_plan_dft_2d....852
6.4.4.5.1.4 kml_fft(f/h)_mpi_plan_dft_3d....854
6.4.4.5.1.5 kml_fft(f/h)_mpi_plan_many_dft....857
6.4.4.5.1.6 kml_fft(f)_mpi_plan_dft_3d_ext....859
6.4.4.5.1.7 kml_fft(f)_mpi_plan_create....861
6.4.4.5.2 R2C 变换....863
6.4.4.5.2.1 kml_fft(f/h)_mpi_plan_dft_r2c....863
6.4.4.5.2.2 kml_fft(f)_mpi_plan_dft_r2c_2d....866
6.4.4.5.2.3 kml_fft(f)_mpi_plan_dft_r2c_3d....868
6.4.4.5.2.4 kml_fft(f/h)_mpi_plan_many_dft_r2c....870
6.4.4.5.2.5 kml_fft(f)_mpi_plan_dft_r2c_3d_ext....872
6.4.4.5.2.6 kml_fft(f)_mpi_plan_create_r2c....874
6.4.4.5.3 C2R 变换....876
6.4.4.5.3.1 kml_fft(f/h)_mpi_plan_dft_c2r....876
6.4.4.5.3.2 kml_fft(f/h)_mpi_plan_dft_c2r_2d....878
6.4.4.5.3.3 kml_fft(f/h)_mpi_plan_dft_c2r_3d....880
6.4.4.5.3.4 kml_fft(f/h)_mpi_plan_many_dft_c2r....882
6.4.4.5.3.5 kml_fft(f)_mpi_plan_dft_c2r_3d_ext....884
6.4.4.5.3.6 kml_fft(f)_mpi_plan_create_c2r....886
6.4.4.5.4 R2R 变换....889
6.4.4.5.4.1 kml_fft(f/h)_mpi_plan_r2r....889
6.4.4.5.4.2 kml_fft(f/h)_ mpi_plan_r2r_2d....891
6.4.4.5.4.3 kml_fft(f/h)_mpi_plan_r2r_3d....894

 版权所有 © 华为技术有限公司

6.4.4.5.4.4 kml_fft(f/h)_mpi_plan_many_r2r..... 898
6.4.4.5.4.5 kml_fft(f)_mpi_plan_r2r_3d_ext..... 900
6.4.4.5.4.6 kml_fft(f)_mpi_plan_create_r2r..... 904
6.4.4.5.5 变换执行函数..... 907
6.4.4.5.5.1 kml_fft(f/h)_mpi_execute_dft..... 907
6.4.4.5.5.2 kml_fft(f/h)_mpi_execute_dft_r2c..... 908
6.4.4.5.5.3 kml_fft(f/h)_mpi_execute_dft_c2r..... 909
6.4.4.5.5.4 kml_fft(f/h)_mpi_execute_r2r..... 910
6.4.4.5.5.5 kml_fft(f)_mpi_execute_dft_ext..... 911
6.4.4.5.5.6 kml_fft(f)_mpi_execute_dft_r2c_ext..... 912
6.4.4.5.5.7 kml_fft(f)_mpi_execute_dft_c2r_ext..... 913
6.4.4.5.5.8 kml_fft(f)_mpi_execute_r2r_ext..... 914
6.4.4.5.5.9 kml_fft(f)_mpi_forward_c2c..... 915
6.4.4.5.5.10 kml_fft(f)_mpi_backward_c2c..... 916
6.4.4.5.5.11 kml_fft(f)_mpi_forward_r2c..... 917
6.4.4.5.5.12 kml_fft(f)_mpi_backward _c2r..... 918
6.4.4.5.6 进程函数..... 919
6.4.4.5.6.1 kml_fft_mpi_init..... 919
6.4.4.5.6.2 kml_fft_mpi_cleanup..... 920
6.4.4.5.7 MPI Local Size 函数..... 920
6.4.4.5.7.1 kml_fft(f/h)_mpi_local_size..... 920
6.4.4.5.7.2 kml_fft(f/h)_mpi_local_size_transposed..... 921
6.4.4.5.7.3 kml_fft(f/h)_mpi_local_size_1d..... 923
6.4.4.5.7.4 kml_fft(f/h)_mpi_local_size_many_1d..... 924
6.4.4.5.7.5 kml_fft(f/h)_mpi_local_size_2d..... 926
6.4.4.5.7.6 kml_fft(f)_mpi_local_size_2d_transposed..... 926
6.4.4.5.7.7 kml_fft(f)_mpi_local_size_3d..... 928
6.4.4.5.7.8 kml_fft(f)_mpi_local_size_3d_transposed..... 929
6.4.4.5.7.9 kml_fft(f)_mpi_local_size_many..... 931
6.4.4.5.7.10 kml_fft(f)_mpi_local_size_many_transposed..... 932
6.4.4.5.7.11 kml_fft(f)_mpi_local_size_ext..... 933
6.4.4.5.7.12 kml_fft(f)_mpi_local_size_transposed_ext..... 934
6.4.4.5.7.13 kml_fft(f)_mpi_local_size_3d_ext..... 936
6.4.4.5.7.14 kml_fft(f)_mpi_local_size_3d_transposed_ext..... 937
6.4.4.5.7.15 kml_fft(f)_mpi_split_world..... 939
6.4.4.5.8 MPI transposition..... 940
6.4.4.5.8.1 kml_fft_mpi_plan_transpose..... 941
6.4.4.5.8.2 kml_fft_mpi_plan_many_transpose..... 942
6.4.4.5.9 扩展接口环境变量..... 943
6.4.4.5.9.1 SCALFFT_PROC_GRID..... 943
6.4.4.5.9.2 SCALFFT_TILE_COUNT..... 943
6.4.4.5.9.3 SCALFFT_DECOMPOSE_TYPE..... 943

 版权所有 © 华为技术有限公司

6.4.4.5.9.4 SCALFFT_PRE_A2A_ALGO..... 943
6.4.4.5.9.5 SCALFFT_MID_A2A_ALGO..... 944
6.4.4.5.9.6 SCALFFT_POST_A2A_ALGO..... 945
6.4.4.6 公用函数定义..... 945
6.4.4.6.1 内存函数..... 945
6.4.4.6.1.1 调用约束..... 945
6.4.4.6.1.2 kml_fft(f/h)_malloc..... 945
6.4.4.6.1.3 kml_fft(f/h)_free..... 947
6.4.4.6.1.4 kml_fft(f/h)_destroy_plan..... 948
6.4.4.6.2 线程函数..... 950
6.4.4.6.2.1 调用约束..... 950
6.4.4.6.2.2 kml_fft(f/h)_init_threads..... 950
6.4.4.6.2.3 kml_fft(f/h)_plan_with_nthreads..... 952
6.4.4.6.2.4 kml_fft(f/h)_cleanup_threads..... 954
6.4.4.6.3 KFFTGetVersion..... 956
6.4.4.6.4 kml_fft(f/h)_set_timelimit..... 957
6.4.5 KML_SVML 库函数说明..... 958
6.4.5.1 函数说明..... 958
6.4.5.2 函数定义..... 958
6.4.5.2.1 svml128_atan_f?..... 959
6.4.5.2.2 svml128_atan2_f?..... 959
6.4.5.2.3 svml128_cos_f?..... 961
6.4.5.2.4 svml128_sin_f?..... 962
6.4.5.2.5 svml128_tan_f?..... 963
6.4.5.2.6 svml128_sincos_f?..... 963
6.4.5.2.7 svml128_cosh_f?..... 965
6.4.5.2.8 svml128_sinh_f?..... 966
6.4.5.2.9 svml128_tanh_f?..... 966
6.4.5.2.10 svml128_exp_f?..... 967
6.4.5.2.11 svml128_exp2_f?..... 968
6.4.5.2.12 svml128_expm1_f?..... 969
6.4.5.2.13 svml128_log_f?..... 970
6.4.5.2.14 svml128_log10_f?..... 971
6.4.5.2.15 svml128_log1p_f?..... 972
6.4.5.2.16 svml128_pow_f?..... 973
6.4.5.2.17 KSVMLGetVersion..... 975
6.4.5.3 编译器自动向量化..... 975
6.4.5.3.1 介绍..... 976
6.4.5.3.2 GCC 向量化方法..... 978
6.4.5.3.3 毕昇编译器向量化方法..... 980
6.4.6 KML_VSL 库函数说明..... 980
6.4.6.1 函数说明..... 980

 版权所有 © 华为技术有限公司

6.4.6.2 函数定义 981
6.4.6.2.1 直接法函数 981
6.4.6.2.1.1 初始化函数 981
6.4.6.2.1.2 内存释放函数 981
6.4.6.2.1.3 设置随机数算法函数 982
6.4.6.2.1.4 设置随机种子函数 983
6.4.6.2.1.5 均匀分布函数 983
6.4.6.2.1.6 贝塔分布函数 984
6.4.6.2.1.7 柯西分布函数 986
6.4.6.2.1.8 指数分布函数 987
6.4.6.2.1.9 伽马分布函数 988
6.4.6.2.1.10 对数正态分布函数 989
6.4.6.2.1.11 正态分布函数 990
6.4.6.2.1.12 威布尔分布函数 991
6.4.6.2.1.13 二项分布函数 992
6.4.6.2.1.14 超几何分布函数 993
6.4.6.2.1.15 几何分布函数 994
6.4.6.2.1.16 负二项分布函数 995
6.4.6.2.1.17 泊松分布函数 996
6.4.6.2.1.18 版本查询函数 997
6.4.6.2.2 高性能函数 998
6.4.6.2.2.1 创建随机数流函数 998
6.4.6.2.2.2 释放随机数流函数 1000
6.4.6.2.2.3 跳过随机数流元素函数 1000
6.4.6.2.2.4 分配随机数流函数 1001
6.4.6.2.2.5 拷贝随机数流函数 1002
6.4.6.2.2.6 均匀分布函数 1003
6.4.6.2.2.7 柯西分布函数 1006
6.4.6.2.2.8 指数分布函数 1007
6.4.6.2.2.9 伽马分布函数 1009
6.4.6.2.2.10 对数正态分布函数 1011
6.4.6.2.2.11 威布尔分布函数 1013
6.4.6.2.2.12 拉普拉斯分布函数 1014
6.4.6.2.2.13 瑞利分布函数 1016
6.4.6.2.2.14 耿贝尔分布 1018
6.4.6.2.2.15 二项分布函数 1020
6.4.6.2.2.16 几何分布函数 1021
6.4.6.2.2.17 泊松分布函数 1022
6.4.6.2.2.18 伯努利分布函数 1024
6.4.7 KML_IPL 库函数说明 1025
6.4.7.1 函数说明 1025
6.4.7.2 函数定义 1025

 版权所有 © 华为技术有限公司

6.4.7.2.1 辅助函数 1025
6.4.7.2.1.1 interp 1025
1. kml_?_interp_alloc 1025
2. kml_?_interp_init 1026
3. kml_?_interp_name 1027
4. kml_?_interp_min_size 1027
5. kml_?_interp_free 1028
6.4.7.2.1.2 spline 1029
1. kml_?_spline_alloc 1029
2. kml_?_spline_init 1029
3. kml_?_spline_name 1030
4. kml_?_spline_min_size 1031
5. kml_?_spline_free 1031
6.4.7.2.1.3 interp2d 1032
1. kml_?_interp2d_alloc 1032
2. kml_?_interp2d_init 1032
3. kml_?_interp2d_free 1033
4. kml_?_interp2d_set 1034
5. kml_?_interp2d_get 1035
6. kml_?_interp2d_idx 1036
7. kml_?_interp2d_name 1036
8. kml_?_interp2d_min_size 1037
6.4.7.2.1.4 spline2d 1038
1. kml_?_spline2d_alloc 1038
2. kml_?_spline2d_init 1038
3. kml_?_spline2d_free 1039
4. kml_?_spline2d_set 1040
5. kml_?_spline2d_get 1041
6. kml_?_spline2d_name 1042
7. kml_?_spline2d_min_size 1042
6.4.7.2.2 计算函数 1043
6.4.7.2.2.1 interp 1043
1. kml_?_interp_eval 1043
2. kml_?_interp_eval_deriv 1044
3. kml_?_interp_eval_deriv2 1045
4. kml_?_interp_eval_integ 1046
6.4.7.2.2.2 spline 1048
1. kml_?_spline_eval 1048
2. kml_?_spline_eval_deriv 1049
3. kml_?_spline_eval_deriv2 1050
4. kml_?_spline_eval_integ 1051
6.4.7.2.2.3 interp2d 1051

 版权所有 © 华为技术有限公司

1. kml_?_interp2d_eval..... 1052
2. kml_?_interp2d_eval_extrap..... 1053
3. kml_?_interp2d_eval_deriv_x..... 1054
4. kml_?_interp2d_eval_deriv_y..... 1056
5. kml_?_interp2d_eval_deriv_xx..... 1057
6. kml_?_interp2d_eval_deriv_yy..... 1059
7. kml_?_interp2d_eval_deriv_xy..... 1060
6.4.7.2.2.4 spline2d..... 1061
1. kml_?_spline2d_eval..... 1061
2. kml_?_spline2d_eval_extrap..... 1063
3. kml_?_spline2d_eval_deriv_x..... 1064
4. kml_?_spline2d_eval_deriv_y..... 1065
5. kml_?_spline2d_eval_deriv_xx..... 1066
6. kml_?_spline2d_eval_deriv_yy..... 1067
7. kml_?_spline2d_eval_deriv_xy..... 1069
6.4.7.2.2.5 v_interp2d..... 1070
1. kml_v_?_interp2d_eval_e..... 1070
2. kml_v_?_interp2d_eval_extrap_e..... 1071
3. kml_v_?_interp2d_eval_deriv_x_e..... 1071
4. kml_v_?_interp2d_eval_deriv_y_e..... 1072
5. kml_v_?_interp2d_eval_deriv_xx_e..... 1073
6. kml_v_?_interp2d_eval_deriv_yy_e..... 1074
7. kml_v_?_interp2d_eval_deriv_xy_e..... 1075
6.4.7.2.2.6 v_spline2d..... 1076
1. kml_v_?_spline2d_eval_e..... 1076
2. kml_v_?_spline2d_eval_extrap_e..... 1076
3. kml_v_?_spline2d_eval_deriv_x_e..... 1077
4. kml_v_?_spline2d_eval_deriv_y_e..... 1078
5. kml_v_?_spline2d_eval_deriv_xx_e..... 1078
6. kml_v_?_spline2d_eval_deriv_yy_e..... 1079
7. kml_v_?_spline2d_eval_deriv_xy_e..... 1080
6.4.7.2.2.7 basis 样条函数..... 1081
1. KIPL_Bsnak..... 1081
2. KIPL_Bsint..... 1081
3. KIPL_Bsval..... 1082
4. KIPL_Bsder..... 1083
5. 示例..... 1083
6.4.7.2.2.8 三次样条函数..... 1084
1. KIPL_Csint..... 1084
2. KIPL_Cssmh..... 1084
3. KIPL_Csval..... 1085
4. KIPL_Cs1gd..... 1086

 版权所有 © 华为技术有限公司

5. 示例 1086
6.4.7.2.3 加速函数 1088
6.4.7.2.3.1 kml_interp_accel_alloc 1088
6.4.7.2.3.2 kml_interp_accel_reset 1088
6.4.7.2.3.3 kml_interp_accel_free 1089
6.4.8 KML_BLAS 库函数说明 1089
6.4.8.1 函数说明 1089
6.4.8.2 数据结构 1089
6.4.8.3 环境变量 1091
6.4.8.4 函数定义 1094
6.4.8.4.1 KML_BLAS Level 1 函数 1094
6.4.8.4.1.1 函数说明 1094
6.4.8.4.1.2 ?asum 1095
6.4.8.4.1.3 ?axpy 1096
6.4.8.4.1.4 ?copy 1098
6.4.8.4.1.5 ?dot 1100
6.4.8.4.1.6 ?dotc 1101
6.4.8.4.1.7 ?dotu 1103
6.4.8.4.1.8 ?sdot 1104
6.4.8.4.1.9 i?amax 1106
6.4.8.4.1.10 i?amin 1108
6.4.8.4.1.11 ?nrm2 1109
6.4.8.4.1.12 ?rot 1110
6.4.8.4.1.13 ?rotg 1113
6.4.8.4.1.14 ?rotm 1114
6.4.8.4.1.15 ?rotmg 1116
6.4.8.4.1.16 ?scal 1117
6.4.8.4.1.17 ?swap 1119
6.4.8.4.2 KML_BLAS Level 2 函数 1121
6.4.8.4.2.1 函数说明 1121
6.4.8.4.2.2 ?gbmv 1122
6.4.8.4.2.3 ?gemv 1126
6.4.8.4.2.4 ?ger 1129
6.4.8.4.2.5 ?gerc 1132
6.4.8.4.2.6 ?geru 1134
6.4.8.4.2.7 ?hbmv 1136
6.4.8.4.2.8 ?hemv 1138
6.4.8.4.2.9 ?her 1141
6.4.8.4.2.10 ?her2 1143
6.4.8.4.2.11 ?hpmv 1145
6.4.8.4.2.12 ?hpr 1147
6.4.8.4.2.13 ?hpr2 1148

 版权所有 © 华为技术有限公司

6.4.8.4.2.14 ?sbmv 1150
6.4.8.4.2.15 ?spmv 1153
6.4.8.4.2.16 ?spr 1155
6.4.8.4.2.17 ?spr2 1156
6.4.8.4.2.18 ?symv 1158
6.4.8.4.2.19 ?syr 1161
6.4.8.4.2.20 ?syr2 1163
6.4.8.4.2.21 ?tbmv 1165
6.4.8.4.2.22 ?tbsv 1168
6.4.8.4.2.23 ?tpmv 1171
6.4.8.4.2.24 ?tpsv 1173
6.4.8.4.2.25 ?trmv 1176
6.4.8.4.2.26 ?trsv 1179
6.4.8.4.3 KML_BLAS Level 3 函数 1182
6.4.8.4.3.1 函数说明 1182
6.4.8.4.3.2 ?gemm 1182
6.4.8.4.3.3 ?hgemm 1187
6.4.8.4.3.4 ?bgemm 1192
6.4.8.4.3.5 gemm_?8?8s32 1196
6.4.8.4.3.6 ?hemm 1201
6.4.8.4.3.7 ?her2k 1204
6.4.8.4.3.8 ?herk 1206
6.4.8.4.3.9 ?symm 1209
6.4.8.4.3.10 ?syrk 1213
6.4.8.4.3.11 ?syr2k 1216
6.4.8.4.3.12 ?trmm 1219
6.4.8.4.3.13 ?trsm 1223
6.4.8.4.4 KML_BLAS-like extensions 1226
6.4.8.4.4.1 函数说明 1226
6.4.8.4.4.2 ?axpby 1227
6.4.8.4.4.3 ?axpyc 1230
6.4.8.4.4.4 ?gemm3m 1231
6.4.8.4.4.5 ?omatcopy 1235
6.4.8.4.4.6 ?imatcopy 1238
6.4.8.4.4.7 ?omatadd 1241
6.4.8.4.4.8 ?imatadd 1246
6.4.8.4.4.9 ?omatsub 1249
6.4.8.4.4.10 ?imatsub 1254
6.4.8.4.4.11 ?omatmul 1257
6.4.8.4.4.12 ?imatmul 1261
6.4.8.4.4.13 ?omatdiv 1265
6.4.8.4.4.14 ?imatdiv 1269

 版权所有 © 华为技术有限公司

6.4.8.4.4.15 ?gemm_pack_get_size....1273
6.4.8.4.4.16 ?gemm_pack....1274
6.4.8.4.4.17 ?gemm_compute....1277
6.4.8.4.4.18 ?gemm_batch....1282
6.4.8.4.4.19 gemm_batch_?8?8s32....1288
6.4.8.4.4.20 gemm_s8u8s32_pack_get_size....1293
6.4.8.4.4.21 gemm_s8u8s32_pack....1294
6.4.8.4.4.22 gemm_?8?8s32_compute....1296
6.4.8.4.5 Support Functions....1299
6.4.8.4.5.1 BlasSetNumThreads....1299
6.4.8.4.5.2 BlasSetNumThreadsLocal....1300
6.4.8.4.5.3 BlasGetNumThreads....1300
6.4.8.4.5.4 BlasGetNumThreadsLocal....1301
6.4.8.4.5.5 BlasGetNumProcs....1301
6.4.8.4.5.6 BlasGetParallel....1301
6.4.8.4.5.7 lsame....1302
6.4.8.4.5.8 ?cabs1....1302
6.4.8.4.5.9 KBLASGetVersion....1302
6.4.8.4.5.10 cblas_xerbla....1303
6.4.9 KML_CONV 库函数说明....1303
6.4.9.1 函数说明....1303
6.4.9.2 函数定义....1304
6.4.9.2.1 通用卷积接口....1304
6.4.9.2.1.1 函数说明....1304
6.4.9.2.1.2 conv1d_?....1304
6.4.9.2.1.3 conv1d_direct_?....1306
6.4.9.2.1.4 conv1d_fft_?....1308
6.4.9.2.1.5 conv1d_gemm_?....1310
6.4.9.2.1.6 conv1d_winograd_?....1311
6.4.9.2.1.7 conv2d_?....1313
6.4.9.2.1.8 conv2d_direct_?....1315
6.4.9.2.1.9 conv2d_fft_?....1318
6.4.9.2.1.10 conv2d_gemm_?....1320
6.4.9.2.1.11 conv2d_winograd_?....1322
6.4.9.2.1.12 conv2d_separable_?....1324
6.4.9.2.2 上下采样接口....1326
6.4.9.2.2.1 函数说明....1326
6.4.9.2.2.2 usImage2X_cpu....1326
6.4.9.2.2.3 dsImage_cpu....1327
6.4.9.2.3 图像处理接口....1329
6.4.9.2.3.1 使用指南....1329
6.4.9.2.3.2 函数说明....1332

 版权所有 © 华为技术有限公司

6.4.9.2.3.3 kml_init_filter..... 1332
6.4.9.2.3.4 kml_execute_filter2d..... 1334
6.4.9.2.3.5 kml_free_filter..... 1335
6.5 应用数学库..... 1336
6.5.1 KML_LAPACK 库函数说明..... 1336
6.5.1.1 函数说明..... 1336
6.5.1.2 数据结构..... 1337
6.5.1.3 函数定义..... 1337
6.5.1.3.1 优化的 LAPACK 函数..... 1337
6.5.1.3.2 矩阵分解函数..... 1341
6.5.1.3.2.1 ?getrf..... 1341
6.5.1.3.2.2 ?geqrf..... 1343
6.5.1.3.2.3 ?gerqf..... 1346
6.5.1.3.2.4 ?geqlf..... 1350
6.5.1.3.2.5 ?gelqf..... 1353
6.5.1.3.2.6 ?potrf..... 1356
6.5.1.3.2.7 ?pttrf..... 1358
6.5.1.3.2.8 ?gttrf..... 1360
6.5.1.3.2.9 ?sptrf..... 1362
6.5.1.3.2.10 ?hptrf..... 1364
6.5.1.3.2.11 ?pptrf..... 1366
6.5.1.3.3 线性方程组求解函数..... 1368
6.5.1.3.3.1 ?ppsv..... 1368
6.5.1.3.3.2 ?gesv..... 1370
6.5.1.3.3.3 ?ptsv..... 1373
6.5.1.3.3.4 ?gtsv..... 1375
6.5.1.3.3.5 ?posv..... 1377
6.5.1.3.4 矩阵求逆函数..... 1380
6.5.1.3.4.1 ?getri..... 1380
6.5.1.3.4.2 ?potri..... 1383
6.5.1.3.4.3 ?pptri..... 1385
6.5.1.3.4.4 ?trtri..... 1387
6.5.1.3.5 回代求解函数..... 1389
6.5.1.3.5.1 ?pttrs..... 1389
6.5.1.3.5.2 ?ptts2..... 1391
6.5.1.3.5.3 ?gttrs..... 1393
6.5.1.3.5.4 ?gtts2..... 1396
6.5.1.3.5.5 ?trtrs..... 1399
6.5.1.3.5.6 ?getrs..... 1401
6.5.1.3.5.7 ?pptrs..... 1403
6.5.1.3.6 特征值问题函数..... 1405
6.5.1.3.6.1 ?sy(he)evd..... 1405

 版权所有 © 华为技术有限公司

6.5.1.3.6.2 ?sy(he)ev....1410
6.5.1.3.6.3 ?steqr....1413
6.5.1.3.6.4 ?stedc....1416
6.5.1.3.6.5 ?sy(he)evx....1420
6.5.1.3.6.6 ?sy(he)gv....1424
6.5.1.3.6.7 ?geev....1427
6.5.1.3.7 奇异值问题....1432
6.5.1.3.7.1 ?gesvd....1432
6.5.1.3.7.2 ?gebrd....1437
6.5.1.3.7.3 ?gesdd....1441
6.5.1.3.7.4 ?bdsdc....1445
6.5.1.3.7.5 ?bdsqr....1448
6.5.1.3.8 最小二乘问题函数....1454
6.5.1.3.8.1 ?gels....1454
6.5.1.3.8.2 ?gelss....1457
6.5.1.3.8.3 ?gelsd....1462
6.5.1.3.9 其他函数....1466
6.5.1.3.9.1 ?(or,un)glq....1466
6.5.1.3.9.2 ?(or,un)gqr....1469
6.5.1.3.9.3 ?(or,un)grq....1472
6.5.1.3.9.4 ?(or,un)gql....1475
6.5.1.3.9.5 ?(or,un)mlq....1479
6.5.1.3.9.6 ?(or,un)mqr....1484
6.5.1.3.9.7 ?(or,un)mql....1488
6.5.1.3.9.8 ?(or,un)mrq....1493
6.5.1.3.9.9 ?sy(he)trd....1498
6.5.1.3.9.10 ?lasr....1501
6.5.1.3.9.11 ?sy(he)trd_2stage....1504
6.5.1.3.9.12 ?(or,un)mbr....1509
6.5.1.3.9.13 ?(or,un)gtr....1513
6.5.1.3.9.14 Kml?steqr2....1516
6.5.1.3.9.15 KmlBlockBandedMatrixSolve....1518
6.5.1.3.9.16 KmlBlockBandedMatrixSubstitude....1519
6.5.1.3.10 辅助函数....1519
6.5.1.3.10.1 ?laset....1519
6.5.1.3.10.2 ?laswp....1522
6.5.1.3.10.3 ?lascl....1524
6.5.1.3.10.4 ?lange....1527
6.5.1.3.10.5 ?lacpy....1529
6.5.1.3.11 Support 函数....1531
6.5.1.3.11.1 KLAPACKGetVersion....1531
6.5.1.3.12 混合精度函数....1532

 版权所有 © 华为技术有限公司

6.5.1.3.12.1 ?gesv....1532
6.5.1.4 错误处理....1535
6.5.2 KML_SCALAPACK 库函数说明....1536
6.5.2.1 函数说明....1536
6.5.2.2 数据描述....1536
6.5.2.2.1 数据类型....1537
6.5.2.2.2 数据分布....1537
6.5.2.2.3 矩阵描述符....1539
6.5.2.3 函数定义....1539
6.5.2.3.1 优化的 SCALAPACK 函数....1539
6.5.2.3.2 矩阵分解函数....1540
6.5.2.3.2.1 P?GETRF....1540
6.5.2.3.2.2 P?POTRF....1543
6.5.2.3.2.3 P?GEQRF....1546
6.5.2.3.3 线性方程组求解函数....1550
6.5.2.3.3.1 P?GELS....1550
6.5.2.3.3.2 P?GESV....1555
6.5.2.3.4 特征值问题函数....1559
6.5.2.3.4.1 P?(SY/HE)EV....1559
6.5.2.3.4.2 P?(SY/HE)EVD....1565
6.5.2.3.5 矩阵求逆函数....1571
6.5.2.3.5.1 PGETRI....1571
6.5.2.3.5.2 PPOTRI....1575
6.5.2.3.6 Support 函数....1579
6.5.2.3.6.1 KSCALAPACKGetVersion....1579
6.5.3 KML_SOLVER 库函数说明....1580
6.5.3.1 函数说明....1580
6.5.3.2 数据类型和数据存储....1582
6.5.3.2.1 数据类型....1582
6.5.3.2.2 数据存储....1582
6.5.3.3 函数定义（单机版）....1584
6.5.3.3.1 CG....1584
6.5.3.3.1.1 KmlIssCgInit?I....1585
6.5.3.3.1.2 KmlIssCgSetUserPreconditioner?I....1586
6.5.3.3.1.3 KmlIssCgSet?I?....1587
6.5.3.3.1.4 KmlIssCgAnalyze?I....1589
6.5.3.3.1.5 KmlIssCgFactorize?I....1590
6.5.3.3.1.6 KmlIssCgSolve?I....1591
6.5.3.3.1.7 KmlIssCgGet?I?....1593
6.5.3.3.1.8 KmlIssCgClean?I....1595
6.5.3.3.2 GCR....1596
6.5.3.3.2.1 KmlIssGcrInit?I....1596

 版权所有 © 华为技术有限公司

6.5.3.3.2.2 KmlIssGcrSet?I?.....1598
6.5.3.3.2.3 KmlIssGcrAnalyze?I.....1600
6.5.3.3.2.4 KmlIssGcrFactorize?I.....1601
6.5.3.3.2.5 KmlIssGcrSolve?I.....1602
6.5.3.3.2.6 KmlIssGcrGet?I?.....1604
6.5.3.3.2.7 KmlIssGcrClean?I.....1606
6.5.3.3.3 GCR 算子.....1607
6.5.3.3.3.1 KmlIssGcrAxpyP.....1607
6.5.3.3.3.2 KmlIssGcrAxpyAp.....1609
6.5.3.3.3.3 KmlIssGcrAxpyX.....1610
6.5.3.3.3.4 KmlIssGcrAxpyR.....1612
6.5.3.3.3.5 KmlIssGcrDotprodAlpha.....1614
6.5.3.3.3.6 KmlIssGcrDotprodBeta.....1615
6.5.3.3.3.7 KmlIssGcrSpmv.....1617
6.5.3.3.4 KmlIssGetErrorText.....1618
6.5.3.3.5 KmlIssClearError.....1620
6.5.3.3.6 KSOLVERGetVersion.....1621
6.5.3.3.7 PRECOND.....1622
6.5.3.3.7.1 KmlIssPcStructilu0Setup.....1622
6.5.3.3.7.2 KmlIssPcStructilu0Apply.....1623
6.5.3.3.7.3 KmlIssPcStructilu0Clean.....1625
6.5.3.3.8 LDLT (DEPRECATED).....1626
6.5.3.3.8.1 KmlDssSymInit?I.....1626
6.5.3.3.8.2 KmlDssSymAnalyze?I.....1627
6.5.3.3.8.3 KmlDssSymFactorize?I.....1628
6.5.3.3.8.4 KmlDssSymSolve?I.....1630
6.5.3.3.8.5 KmlDssSymGet?IL.....1632
6.5.3.3.8.6 KmlDssSymGet?II.....1634
6.5.3.3.8.7 KmlDssSymGet?I(D/S).....1636
6.5.3.3.8.8 KmlDssSymSet?II.....1638
6.5.3.3.8.9 KmlDssSymSet?I(D/S).....1641
6.5.3.3.8.10 KmlDssSymClean?I.....1643
6.5.3.3.9 LU (DEPRECATED).....1644
6.5.3.3.9.1 KmlDssGenInit?I.....1644
6.5.3.3.9.2 KmlDssGenAnalyze?I.....1646
6.5.3.3.9.3 KmlDssGenFactorize?I.....1647
6.5.3.3.9.4 KmlDssGenSolve?I.....1648
6.5.3.3.9.5 KmlDssGenGet?IL.....1651
6.5.3.3.9.6 KmlDssGenGet?II.....1653
6.5.3.3.9.7 KmlDssGenGet?I(D/S).....1654
6.5.3.3.9.8 KmlDssGenSet?II.....1656
6.5.3.3.9.9 KmlDssGenSet?I(D/S).....1659

6.5.3.3.9.10 KmlDssGenClean?I....1661
6.5.3.3.10 LLT/LLH (DEPRECATED)....1662
6.5.3.3.10.1 KmlDss(Spd/Hpd)Init?I....1662
6.5.3.3.10.2 KmlDss(Spd/Hpd)Analyze?I....1664
6.5.3.3.10.3 KmlDss(Spd/Hpd)Factorize?I....1665
6.5.3.3.10.4 KmlDss(Spd/Hpd)Solve?I....1666
6.5.3.3.10.5 KmlDss(Spd/Hpd)Get?IL....1669
6.5.3.3.10.6 KmlDss(Spd/Hpd)Get?II....1670
6.5.3.3.10.7 KmlDss(Spd/Hpd)Get?I(D/S)....1672
6.5.3.3.10.8 KmlDss(Spd/Hpd)Set?Il....1674
6.5.3.3.10.9 KmlDss(Spd/Hpd)Set?I(D/S)....1677
6.5.3.3.10.10 KmlDss(Spd/Hpd)Clean?I....1679
6.5.3.3.11 DSS....1680
6.5.3.3.11.1 矩阵函数....1680
1. KmlSolverMatrixCreate....1680
2. KmlSolverMatrixSetValue....1684
3. KmlSolverMatrixDestroy....1685
6.5.3.3.11.2 求解器函数....1686
1. KmlDssInit....1686
2. KmlDssAnalyze....1687
3. KmlDssFactorize....1691
4. KmlDssSolve....1693
5. KmlDssClean....1695
6. KmlDssQuery....1696
7. 示例 1....1698
8. 示例 2....1700
9. 示例 3....1703
6.5.3.4 函数定义（集群版）....1705
6.5.3.4.1 SCAISS-CG....1705
6.5.3.4.1.1 KmlScaissCgInitStripes?I....1705
6.5.3.4.1.2 KmlScaissCgInitWithoutMat?I....1707
6.5.3.4.1.3 KmlScaissCgSetUserPreconditioner?I....1708
6.5.3.4.1.4 KmlScaissCgSet?I?....1710
6.5.3.4.1.5 KmlScaissCgSetUserSpmv?I....1713
6.5.3.4.1.6 KmlScaissCgPcSet?II....1714
6.5.3.4.1.7 KmlScaissCgSubPcSet?I?....1716
6.5.3.4.1.8 KmlScaissCgAnalyze?I....1718
6.5.3.4.1.9 KmlScaissCgFactorize?I....1720
6.5.3.4.1.10 KmlScaissCgSolve?I....1722
6.5.3.4.1.11 KmlScaissCgSolveDx?I....1725
6.5.3.4.1.12 KmlScaissCgGet?I?....1727
6.5.3.4.1.13 KmlScaissCgClean?I....1730

 版权所有 © 华为技术有限公司

6.5.3.4.1.14 示例 1733
6.5.3.4.2 SCAISS-BICGSTAB 1737
6.5.3.4.2.1 KmlScaissBicgstabInitStripes?I 1737
6.5.3.4.2.2 KmlScaissBicgstabInitWithoutMat?I 1739
6.5.3.4.2.3 KmlScaissBicgstabSetUserPreconditioner?I 1740
6.5.3.4.2.4 KmlScaissBicgstabSet?I? 1742
6.5.3.4.2.5 KmlScaissBicgstabSetUserSpmv?I 1745
6.5.3.4.2.6 KmlScaissBicgstabPcSet?II 1746
6.5.3.4.2.7 KmlScaissBicgstabSubPcSet?I? 1748
6.5.3.4.2.8 KmlScaissBicgstabAnalyze?I 1750
6.5.3.4.2.9 KmlScaissBicgstabFactorize?I 1752
6.5.3.4.2.10 KmlScaissBicgstabSolve?I 1753
6.5.3.4.2.11 KmlScaissBicgstabSolveDx?I 1756
6.5.3.4.2.12 KmlScaissBicgstabGet?I? 1759
6.5.3.4.2.13 KmlScaissBicgstabClean?I 1762
6.5.3.4.2.14 示例 1764
6.5.3.4.3 SCAISS-GMRES 1769
6.5.3.4.3.1 KmlScaissGmresInitStripes?I 1769
6.5.3.4.3.2 KmlScaissGmresInitWithoutMat?I 1771
6.5.3.4.3.3 KmlScaissGmresSetUserPreconditioner?I 1772
6.5.3.4.3.4 KmlScaissGmresSet?I? 1774
6.5.3.4.3.5 KmlScaissGmresSetUserSpmv? 1777
6.5.3.4.3.6 KmlScaissGmresPcSet?II 1778
6.5.3.4.3.7 KmlScaissGmresSubPcSet?I? 1781
6.5.3.4.3.8 KmlScaissGmresAnalyze?I 1784
6.5.3.4.3.9 KmlScaissGmresFactorize?I 1786
6.5.3.4.3.10 KmlScaissGmresSolve?I 1787
6.5.3.4.3.11 KmlScaissGmresSolveDx?I 1790
6.5.3.4.3.12 KmlScaissGmresGet?I? 1792
6.5.3.4.3.13 KmlScaissGmresClean?I 1795
6.5.3.4.3.14 示例 1798
6.5.3.4.4 SCAISS-Csi 1802
6.5.3.4.4.1 KmlScaissCsiInitStripes?I 1802
6.5.3.4.4.2 KmlScaissCsiInitWithoutMat?I 1804
6.5.3.4.4.3 KmlScaissCsiSetUserPreconditioner?I 1805
6.5.3.4.4.4 KmlScaissCsiSet?I? 1806
6.5.3.4.4.5 KmlScaissCsiSetUserSpmv?I 1808
6.5.3.4.4.6 KmlScaissCsiPcSet?I? 1809
6.5.3.4.4.7 KmlScaissCsiSubPcSet?I? 1810
6.5.3.4.4.8 KmlScaissCsiAnalyze?I 1812
6.5.3.4.4.9 KmlScaissCsiFactorize?I 1813
6.5.3.4.4.10 KmlScaissCsiSolve?I 1814

 版权所有 © 华为技术有限公司

6.5.3.4.4.11 KmlScaissCsiSolveDx?I....1815
6.5.3.4.4.12 KmlScaissCsiGet?I?....1816
6.5.3.4.4.13 KmlScaissCsiClean?I....1818
6.5.3.4.4.14 示例....1818
6.5.3.4.5 HMG....1823
6.5.3.4.5.1 KmlMgMatrixCreate....1823
6.5.3.4.5.2 KmlMgMatrixSetValues....1827
6.5.3.4.5.3 KmlMgMatrixDestroy....1829
6.5.3.4.5.4 KmlMgMatrixQuery....1829
6.5.3.4.5.5 KmlMgSolverCreate....1830
6.5.3.4.5.6 KmlMgSolverSetPreconditioner....1833
6.5.3.4.5.7 KmlMgSolverSetPreconditionerMatrix....1839
6.5.3.4.5.8 KmlMgSolverPrepare....1840
6.5.3.4.5.9 KmlMgSolverApply....1841
6.5.3.4.5.10 KmlMgSolverDestroy....1844
6.5.3.4.5.11 KmlMgSolverQuery....1845
6.5.3.4.5.12 KmlMgStatusStr....1848
6.5.3.4.5.13 示例....1849
6.5.3.4.6 TYAMG....1854
6.5.3.4.6.1 KmlAmgMatrixCreate....1854
6.5.3.4.6.2 KmlAmgMatrixDestroy....1857
6.5.3.4.6.3 KmlAmgMatrixSetValues....1858
6.5.3.4.6.4 KmlAmgPrecondCreate....1860
6.5.3.4.6.5 KmlAmgPrecondDestroy....1861
6.5.3.4.6.6 KmlAmgPrecondSetSetupOptions....1862
6.5.3.4.6.7 KmlAmgPrecondSetup....1866
6.5.3.4.6.8 KmlAmgPrecondSetApplyOptions....1867
6.5.3.4.6.9 KmlAmgPrecondApply....1869
6.5.3.4.6.10 KmlAmgPrecondQuery....1870
6.5.3.4.6.11 KmlAmgSolverCreate....1871
6.5.3.4.6.12 KmlAmgSolverDestroy....1873
6.5.3.4.6.13 KmlAmgSolverSetSetupOptions....1874
6.5.3.4.6.14 KmlAmgSolverSetup....1876
6.5.3.4.6.15 KmlAmgSolverSetApplyOptions....1877
6.5.3.4.6.16 KmlAmgSolverApply....1879
6.5.3.4.6.17 KmlAmgSolverSetPrec....1879
6.5.3.4.6.18 KmlAmgSolverQuery....1880
6.5.3.4.6.19 示例....1882
6.5.3.4.7 SCADSS-LLT/LLH (DEPRECATED)....1886
6.5.3.4.7.1 KmlScadss(SPD/HPD)Init?I....1886
6.5.3.4.7.2 KmlScadss(SPD/HPD)Set?I?....1888
6.5.3.4.7.3 KmlScadss(SPD/HPD)Analyze?I....1890

6.5.3.4.7.4 KmlScadss(SPD/HPD)Factorize?I..... 1891
6.5.3.4.7.5 KmlScadss(SPD/HPD)Solve?I..... 1892
6.5.3.4.7.6 KmlScadss(SPD/HPD)Get?I?..... 1894
6.5.3.4.7.7 KmlScadss(SPD/HPD)Clean?I..... 1896
6.5.3.4.8 SCADSS..... 1898
6.5.3.4.8.1 矩阵函数..... 1898
1. KmlScasolverMatrixCreate..... 1898
2. KmlScasolverMatrixSetValue..... 1900
3. KmlScasolverMatrixDestroy..... 1901
6.5.3.4.8.2 求解器函数..... 1901
1. KmlScadssInit..... 1901
2. KmlScadssAnalyze..... 1903
3. KmlScadssFactorize..... 1905
4. KmlScadssSolve..... 1906
5. KmlScadssClean..... 1907
6. KmlScadssQuery..... 1907
7. 示例..... 1908
6.5.3.4.9 SCADSS-LU (DEPRECATED)..... 1913
6.5.3.4.9.1 KmlScadssGENInit?I..... 1913
6.5.3.4.9.2 KmlScadssGENSet?I?..... 1915
6.5.3.4.9.3 KmlScadssGENAnalyze?I..... 1917
6.5.3.4.9.4 KmlScadssGENFactorize?I..... 1918
6.5.3.4.9.5 KmlScadssGENSolve?I..... 1919
6.5.3.4.9.6 KmlScadssGENGet?I?..... 1920
6.5.3.4.9.7 KmlScadssGENClean?I..... 1922
6.5.3.4.10 SCADSS-LDLT (DEPRECATED)..... 1924
6.5.3.4.10.1 KmlScadssSYMInit?I..... 1924
6.5.3.4.10.2 KmlScadssSYMSet?I?..... 1926
6.5.3.4.10.3 KmlScadssSYMAnalyze?I..... 1928
6.5.3.4.10.4 KmlScadssSYMFactorize?I..... 1929
6.5.3.4.10.5 KmlScadssSYMSolve?I..... 1930
6.5.3.4.10.6 KmlScadssSYMGet?I?..... 1931
6.5.3.4.10.7 KmlScadssSYMClean?I..... 1933
6.5.3.4.11 KSCAKSOLVERGetVersion..... 1935
6.5.4 KML_EIGENSOLVER 库函数说明..... 1936
6.5.4.1 函数说明..... 1936
6.5.4.2 数据类型..... 1936
6.5.4.3 函数定义..... 1936
6.5.4.3.1 函数返回代码..... 1936
6.5.4.3.2 创建内部对象..... 1936
6.5.4.3.2.1 KesMatrixCreate..... 1937
6.5.4.3.2.2 KesVectorCreate..... 1937

 版权所有 © 华为技术有限公司

6.5.4.3.2.3 KesSolverCreate..... 1938
6.5.4.3.2.4 KesMatrixSetDim..... 1939
6.5.4.3.2.5 KesVectorSetDim..... 1940
6.5.4.3.2.6 KesMatrixSetShellMatVec..... 1940
6.5.4.3.2.7 KesVectorDenseSetValues..... 1941
6.5.4.3.2.8 KesVectorSetComm..... 1942
6.5.4.3.2.9 KesVectorReplicate..... 1942
6.5.4.3.3 销毁内部对象..... 1943
6.5.4.3.3.1 KesMatrixDestroy..... 1943
6.5.4.3.3.2 KesVectorDestroy..... 1943
6.5.4.3.3.3 KesSolverDestroy..... 1944
6.5.4.3.4 设置求解参数..... 1944
6.5.4.3.4.1 KesSolverSetProblemType..... 1944
6.5.4.3.4.2 KesSolverSetMaxIter..... 1945
6.5.4.3.4.3 KesSolverSetMaxOrthoIter..... 1945
6.5.4.3.4.4 KesSolverSetThreshold..... 1946
6.5.4.3.4.5 KesSolverSetDenseEigenSolverType..... 1946
6.5.4.3.4.6 KesSolverSetConvergeDetectorType..... 1947
6.5.4.3.4.7 KesSolverSetOrthogonalizerType..... 1947
6.5.4.3.4.8 KesSolverSetOrthoProjectorType..... 1948
6.5.4.3.5 运行求解器..... 1948
6.5.4.3.5.1 KesRunSolver..... 1949
6.5.4.3.6 查询求解信息..... 1949
6.5.4.3.6.1 KesEigResidual..... 1949
6.5.4.3.6.2 KesGetSolverIter..... 1950
6.5.4.3.6.3 KesGetSolverConv..... 1950
6.5.4.3.6.4 KesVectorNorm..... 1951
6.5.4.4 示例..... 1951
6.5.5 环境变量配置..... 1953
6.6 其他..... 1955
6.6.1 KML_JAVA 库函数说明..... 1956
6.6.1.1 函数说明..... 1956
6.6.1.2 函数定义..... 1956
6.6.1.2.1 BLAS 库函数..... 1956
6.6.1.2.2 VML 库函数..... 1958
6.6.2 KML_NUMPY 库函数说明..... 1958
6.6.2.1 函数说明..... 1958
6.6.2.2 数据类型和数据存储..... 1958
6.6.2.3 函数定义..... 1960
6.6.2.3.1 基础运算函数..... 1960
6.6.2.3.1.1 add..... 1960
6.6.2.3.1.2 subtract..... 1961

 版权所有 © 华为技术有限公司

6.6.2.3.1.3 multiply....1962
6.6.2.3.1.4 exp....1964
6.6.2.3.1.5 sqrt....1965
6.6.2.3.1.6 log....1966
6.6.2.3.1.7 sin....1967
6.6.2.3.1.8 cos....1968
6.6.2.3.1.9 tan....1969
6.6.2.3.1.10 arctan....1970
6.6.2.3.1.11 power....1971
6.6.2.3.1.12 fmod....1972
6.6.2.3.1.13 exp2....1973
6.6.2.3.2 傅里叶变换函数....1974
6.6.2.3.2.1 fft.fft....1974
6.6.2.3.2.2 fft.ifft....1975
6.6.2.3.2.3 fft.fft2....1976
6.6.2.3.2.4 fft.ifft2....1978
6.6.2.3.2.5 fft.fftn....1979
6.6.2.3.2.6 fft.ifftn....1981
6.6.2.3.3 统计基础函数....1982
6.6.2.3.3.1 dot....1982
6.6.2.3.3.2 vdot....1983
6.6.2.3.3.3 inner....1984
6.6.2.3.3.4 matmul....1986
6.6.2.3.3.5 einsum....1987
6.6.2.3.4 线性代数函数....1990
6.6.2.3.4.1 linalg.slogdet....1991
6.6.2.3.4.2 linalg.inv....1992
6.6.2.3.4.3 linalg.det....1993
6.6.2.3.4.4 linalg.solve....1994
6.6.2.3.4.5 tensordot....1994
6.6.2.3.4.6 linalg.lstsq....1996
6.6.2.3.4.7 linalg.qr....1997
6.6.2.3.4.8 linalg.svd....1999
6.6.2.3.4.9 linalg.eigh....2000
6.6.2.3.4.10 linalg.eigvalsh....2002
6.6.2.3.4.11 linalg.cholesky....2003
6.6.2.3.4.12 linalg.matrix_power....2004
6.6.2.3.4.13 linalg.tensorsolve....2005
6.6.2.3.4.14 linalg.norm....2006
6.6.2.3.4.15 linalg.cond....2009
6.6.2.3.4.16 linalg.tensorinv....2010
6.6.2.3.5 版本查询函数....2011

6.6.3 KML_RT 库说明 2012
6.6.3.1 KML_RT 库的作用 2012
6.6.3.2 KML_RT 包含的动态库 2012
6.6.3.3 使用说明 2012
6.6.3.3.1 示例 2013
6.6.3.3.2 环境变量说明 2013
6.6.3.3.3 注意事项 2014
7 Hyper IO 库开发指南 2016
7.1 产品描述 2016
7.1.1 软件架构 2016
7.1.2 关键特性 2018
7.1.3 产品介绍 2018
7.2 使用指南 2019
7.2.1 使用 Hyper IO 2019
7.2.2 Hyper IO 函数说明 2025
7.2.2.1 MPI-IO 桥接函数 2026
7.2.2.1.1 函数说明 2026
7.2.2.1.2 数据类型 2026
7.2.2.1.3 文件操作 2028
7.2.2.1.3.1 以聚合模式打开文件 2028
7.2.2.1.3.2 关闭文件 2029
7.2.2.1.3.3 删除文件 2030
7.2.2.1.3.4 使数据访问操作的语义保持一致 2030
7.2.2.1.4 信息设置与访问 2031
7.2.2.1.4.1 以聚合模式设置和访问 info 对象的新值 2031
7.2.2.1.4.2 以聚合模式更改和返回进程对文件中数据的视图 2032
7.2.2.1.4.3 返回与打开的文件关联的访问模式 2033
7.2.2.1.4.4 返回文件的进程组的副本 2034
7.2.2.1.4.5 返回单个文件指针的当前位置 2034
7.2.2.1.4.6 返回文件中数据类型的范围 2035
7.2.2.1.4.7 将视图相对偏移量转换为绝对字节位置 2036
7.2.2.1.4.8 以非聚合模式（noncollective）更新单个文件指针 2036
7.2.2.1.5 句柄转换 2037
7.2.2.1.6 异常处理程序 2038
7.2.2.1.7 文件写入 2039
7.2.2.1.7.1 从指定的位置开始写入文件 2039
7.2.2.1.7.2 以显式指定的偏移量写入文件 2040
7.2.2.1.7.3 以非阻塞方式写入文件 2042
7.2.2.1.8 文件读取 2044
7.2.2.1.8.1 从指定的位置开始读取文件 2044
7.2.2.1.8.2 以显式指定的偏移量读取文件 2046
7.2.2.1.8.3 以非阻塞方式读取文件 2047

 版权所有 © 华为技术有限公司

7.2.2.1.9 终止函数 2049
7.2.2.1.10 功能不支持函数 2049
7.2.2.2 NetCDF 桥接函数 2051
7.2.2.2.1 函数说明 2051
7.2.2.2.2 文件操作 2051
7.2.2.2.3 信息设置 2052
7.2.2.2.3.1 更改填充值模式 2052
7.2.2.2.3.2 定义一个新维度 2053
7.2.2.2.3.3 定义新变量相关函数 2054
7.2.2.2.3.4 定义变量存储方式 2055
7.2.2.2.3.5 设置变量填充值 2056
7.2.2.2.3.6 设置变量访问方式 2057
7.2.2.2.4 信息查询 2057
7.2.2.2.4.1 查询公共信息 2058
7.2.2.2.4.2 查询维度信息 2060
7.2.2.2.4.3 查询变量信息 2061
7.2.2.2.4.4 查询属性信息 2062
7.2.2.2.5 属性信息写入 2063
7.2.2.2.6 属性信息读取 2064
7.2.2.2.7 单元素模式数据写入 2065
7.2.2.2.8 单元素模式数据读取 2066
7.2.2.2.9 array 模式数据写入 2067
7.2.2.2.10 array 模式数据读取 2069
7.2.2.2.11 其他函数 2071
7.2.2.3 PnetCDF 桥接函数 2071
7.2.2.3.1 函数说明 2071
7.2.2.3.2 文件操作 2071
7.2.2.3.3 信息设置 2072
7.2.2.3.4 信息查询 2073
7.2.2.3.4.1 查询文件和维度信息 2073
7.2.2.3.4.2 查询变量和属性信息 2075
7.2.2.3.5 属性信息写入 2076
7.2.2.3.6 属性信息读取 2077
7.2.2.3.7 单元素模式数据写入 2078
7.2.2.3.8 stride 模式数据写入 2079
7.2.2.3.9 array 模式数据写入 2080
7.2.2.3.10 array 模式数据读取 2082
7.2.2.3.11 全变量模式数据写入 2082
7.2.2.3.12 全变量模式数据读取 2083
7.2.2.3.13 其他函数 2084
7.2.2.4 HDF5 桥接函数 2084
7.2.2.4.1 函数说明 2084

 版权所有 © 华为技术有限公司

7.2.2.4.2 文件接口 2084
7.2.2.4.2.1 创建文件 2084
7.2.2.4.2.2 打开文件 2085
7.2.2.4.2.3 关闭文件 2086
7.2.2.4.2.4 刷新缓存区 2086
7.2.2.4.3 属性接口 2086
7.2.2.4.3.1 创建属性 2086
7.2.2.4.3.2 写入属性 2087
7.2.2.4.3.3 读取属性 2088
7.2.2.4.3.4 打开指定属性 2088
7.2.2.4.3.5 关闭指定属性 2089
7.2.2.4.3.6 属性是否存在 2089
7.2.2.4.4 数据集接口 2090
7.2.2.4.4.1 创建新数据集 2090
7.2.2.4.4.2 打开数据集 2091
7.2.2.4.4.3 关闭数据集 2092
7.2.2.4.4.4 获取数据集的数据空间 2092
7.2.2.4.4.5 写入数据集 2092
7.2.2.4.4.6 读取数据集 2093
7.2.2.4.4.7 更改数据集维度 2094
7.2.2.4.5 群组接口 2094
7.2.2.4.5.1 创建群组 2094
7.2.2.4.5.2 打开群组 2095
7.2.2.4.5.3 关闭群组 2096
7.2.2.4.6 属性列表接口 2096
7.2.2.4.6.1 创建属性列表 2096
7.2.2.4.6.2 终止属性列表访问 2097
7.2.2.4.6.3 通信器存储到属性列表 2097
7.2.2.4.6.4 设置数据传输 2097
7.2.2.4.6.5 设置分块大小 2098
7.2.2.4.6.6 设置元数据 I/O 模式 2098
7.2.2.4.6.7 设置元数据写入模式 2099
7.2.2.4.6.8 元数据缓存配置 2099
7.2.2.4.6.9 获取元数据缓存配置 2100
7.2.2.4.7 数据空间接口 2100
7.2.2.4.7.1 创建数据空间 2100
7.2.2.4.7.2 关闭数据空间 2101
7.2.2.4.7.3 获取数据元素切片 2101
7.2.2.4.7.4 创建指定类型数据空间 2102
7.2.2.4.7.5 确定数据空间维度 2102
7.2.2.4.7.6 检索数据空间维度 2103
7.2.2.4.8 数据类型接口 2103

 版权所有 © 华为技术有限公司

7.2.2.4.8.1 创建数组数据类型 2104
7.2.2.4.8.2 复制现有数据类型 2104
7.2.2.4.8.3 设置数据类型大小 2105
7.2.2.4.8.4 获取数据类型大小 2105
7.2.2.4.8.5 设置字符编码集 2105
7.2.2.4.8.6 设置数据类型精度 2106
7.2.2.4.8.7 定义字符串填充类型 2106
7.2.2.4.9 链路接口 2107
7.2.2.4.9.1 链路是否存在 2107
7.2.2.4.9.2 链路创建 2107
7.2.2.4.10 其他函数 2108
7.2.3 Hyper IO 配置项参考 2108
7.3 Hyper IO 测试 2111
7.4 接口支持列表 2113
7.5 术语和缩略语 2113
8 领域加速库开发指南 2115
8.1 KuTACC 开发指南 2115
8.1.1 总述 2115
8.1.2 KuTACC 库函数说明 2115
8.1.2.1 函数说明 2115
8.1.2.2 函数定义 2115
8.1.2.2.1 功能函数 2115
8.1.2.2.1.1 返回值定义 2115
8.1.2.2.1.2 kutacc_get_version 2116
8.1.2.2.2 共性融合算子函数 2117
8.1.2.2.2.1 通信算子 2117
1. kutacc_af2_all_gather 2117
2. kutacc_af2_transpose 2118
3. kutacc_comm_init 2119
4. kutacc_comm_fini 2120
8.1.2.2.2.2 GEMM 算子 2121
1. kutacc_core_bgemm_ex 2122
2. kutacc_core_bgemm_pack 2124
3. kutacc_core_bgemm_pack_get_size 2125
4. kutacc_af2_layernorm 2126
5. kutacc_af2_linear 2127
8.1.2.2.3 个性融合算子函数 2127
8.1.2.2.3.1 AlphaFold2 2128
1. kutacc_af2_gating_attention 2128
2. kutacc_af2_triangle_multiplication_calc_proj 2132
3. kutacc_af2_triangle_multiplication_equation 2133
4. kutacc_af2_triangle_multiplication_gate_and_out_linear 2134

 版权所有 © 华为技术有限公司

5. kutacc_af2_triangle_multiplication_last....2136
6. kutacc_af2_outer_product_mean_calc_left_and_right_mul....2141
7. kutacc_af2_outer_product_mean_chunk....2143
8. kutacc_af2_global_attention....2149
9. kutacc_af2_invariant_point....2153
10. kutacc_af2_rigid_rot_vec_mul....2159
11. kutacc_af2_rigid_rot_matmul....2160
12. kutacc_af2_transition....2160
8.2 KuDNN 开发指南....2164
8.2.1 总述....2164
8.2.2 环境变量....2164
8.2.3 KuDNN 算子库函数说明....2164
8.2.3.1 函数说明....2164
8.2.3.2 函数定义....2165
8.2.3.2.1 GEMM（单线程）&Linear（多线程）....2165
8.2.3.2.1.1 功能描述....2165
8.2.3.2.1.2 使用说明....2166
8.2.3.2.2 Linear+postops（relu，silu，gelu）....2169
8.2.3.2.2.1 功能描述....2169
8.2.3.2.2.2 使用说明....2170
8.2.3.2.3 Linear+postops（resext，resmul，residential）....2173
8.2.3.2.3.1 功能描述....2173
8.2.3.2.3.2 使用说明....2174
8.2.3.2.4 SoftMax....2178
8.2.3.2.4.1 功能描述....2178
8.2.3.2.4.2 使用说明....2179
8.2.3.2.5 Conv2d&Conv3d....2181
8.2.3.2.5.1 功能描述....2181
8.2.3.2.5.2 使用说明....2182
8.2.3.2.6 GroupNorm....2185
8.2.3.2.6.1 功能描述....2185
8.2.3.2.6.2 使用说明....2186
8.2.3.2.7 LayerNorm....2189
8.2.3.2.7.1 功能描述....2189
8.2.3.2.7.2 使用说明....2189
8.2.3.2.8 RMSNorm....2192
8.2.3.2.8.1 功能描述....2192
8.2.3.2.8.2 使用说明....2193
8.2.3.2.9 RoPE....2196
8.2.3.2.9.1 功能描述....2196
8.2.3.2.9.2 使用说明....2196
8.2.3.2.10 Embedding....2198

 版权所有 © 华为技术有限公司

8.2.3.2.10.1 功能描述 2199
8.2.3.2.10.2 使用说明 2199
8.2.4 KuDNN 适配 PyTorch 接口说明 2201
8.2.4.1 算子接口 2201
8.2.4.1.1 接口说明 2201
8.2.4.1.2 Linear 2202
8.2.4.1.3 SoftMax 2202
8.2.4.1.4 Conv 2203
8.2.4.1.5 GroupNorm 2205
8.2.4.1.6 LayerNorm 2206
8.2.4.1.7 RMSNorm 2206
8.2.4.1.8 Embedding 2207
8.2.4.2 使用示例 2208
8.2.4.2.1 Linear 2209
8.2.4.2.2 SoftMax 2209
8.2.4.2.3 Conv 2209
8.2.4.2.4 GroupNorm 2210
8.2.4.2.5 LayerNorm 2210
8.2.4.2.6 RMSNorm 2211
8.2.4.2.7 Embedding 2211
8.3 KuQCD 开发指南 2212
8.3.1 总述 2212
8.3.2 KuQCD 函数说明 2212
8.3.2.1 数据结构 2212
8.3.2.1.1 Lattice 2212
8.3.2.1.2 LattBase 2212
8.3.2.1.3 CommBase 2214
8.3.2.1.4 SpinorField 2214
8.3.2.1.5 GaugeField 2215
8.3.2.2 函数定义 2216
8.3.2.2.1 kuqcd_init 2216
8.3.2.2.2 kuqcd_load_gaugefield 2217
8.3.2.2.3 kuqcd_solve_multishift 2219
8.3.2.2.4 kuqcd_gauge_free 2220
8.3.2.2.5 kuqcd_end 2220
8.3.2.2.6 示例 2221
9 高性能计算开发工具 2223
10 修订记录 2224

 版权所有 © 华为技术有限公司

1 HPCKit 介绍

HPCKit集成鲲鹏数学库（KML）、华为高性能通信库（Hyper MPI）、鲲鹏统一并行加速库（KUPL）、Hyper IO库和领域加速库等用于提高高性能计算软件性能的组件，是一款帮助用户自动化部署高性能计算相关依赖软件的软件包。

领域加速库是针对特定高性能计算场景（如计算光刻、量子色动力学、石油勘探、人工智能等）在鲲鹏平台上深度优化的加速库。它们利用鲲鹏处理器架构特性，通过指令级优化、算法重构和并行加速等手段，显著提升相应领域应用的运行效率。

当前HPCKit包含的组件列表如下。

表 1-1 HPCKit 集成的组件列表

组件名称	说明	是否默认安装
鲲鹏统一并行加速库（KUPL）	鲲鹏统一并行加速库（Kunpeng Unified Parallel Library，以下简称KUPL）提供了基于鲲鹏平台优化的并行加速基础库函数，所有接口由C/C++、汇编语言实现。	是
鲲鹏数学库（KML）	鲲鹏数学库（Kunpeng Math Library，以下简称KML）提供了基于鲲鹏平台优化的高性能数学函数，所有接口由C/C++、汇编语言实现，部分接口兼容Fortran语言调用，部分提供Java语言封装的接口。HPCKit包含了完整的Lapack、ScaLapack和fftw等适配层。	是
华为高性能通信库（HMPI）	华为高性能通信库（Hyper MPI，以下简称HMPI）实现了并行计算的网络通信功能，可以用来支持制造、气象、分子动力学等应用场景。HPCKit包含了完整的MPI通信库。	是
Hyper IO库	Hyper IO（以下简称HIO）是一类提供系统软件和应用软件之间连接的软件，提供科学数据的存储描述格式定义和文件操作，在高性能计算应用以及大量后处理、可视化及分析框架中广泛应用。	是

版权所有 © 华为技术有限公司

组件名称	说明	是否默认安装
KuLitho库	KuLitho是基于鲲鹏平台深度优化的计算光刻加速库，采用C/C++及汇编语言实现，针对图像处理、图形运算及数学计算等核心功能进行指令级优化，显著提升OPC（光学邻近校正）等场景的端到端仿真性能，为EDA提供高性能领域加速库。	否
KuQCD库	KuQCD是基于鲲鹏平台的格点量子色动力学加速库，采用C/C++实现，针对求解器等功能进行优化，并提供相应接口用于应用中。目前KuQCD仅支持在鲲鹏920新型号上使用，依赖KUPL，支持毕昇编译器和gcc编译器。	是
KuRTM库	KuRTM是基于鲲鹏平台的勘探加速库，采用C语言实现。利用鲲鹏920新型号矩阵及向量化能力，针对1D、2D、3D stencil算子进行优化。目前KuRTM仅支持在鲲鹏920新型号上使用，支持毕昇编译器和gcc编译器。	否
KuDNN库	KuDNN是基于鲲鹏平台的深度神经网络算子库，结合鲲鹏CPU微架构特性与软优化手段，优化AI算子性能。通过插件的形式集成PyTorch等AI框架。	是
KuTACC库	KuTACC是基于鲲鹏平台的Transformer模型融合算子库，采用C/C++实现，针对AI算子各层核心算子进行优化，并提供相应接口用于在应用中直接调用，为AI应用提供高效算力支撑。目前KuTACC基于毕昇编译器和libstdc++标准库实现，对gcc编译器暂未支持。	是
毕昇编译器	毕昇编译器是华为编译器实验室针对鲲鹏通用处理器架构构建，引入并增强了多种编译优化技术，致力于打造高性能、高可信及易扩展的编译器工具链。	是
GCC for openEuler	GCC for openEuler是基于开源GCC开发的编译器工具链（包含编译器、汇编器、链接器），在openEuler社区开源发布。	是

版权所有 © 华为技术有限公司

2 HPCKit 架构

HPCKit功能架构如图2-1所示。

图 2-1 HPCKit 架构图



 版权所有 © 华为技术有限公司

3 编译器介绍

毕昇编译器

GCC for openEuler

3.1 毕昇编译器

毕昇编译器是华为编译器实验室针对鲲鹏等通用处理器架构场景，打造的一款高性能、高可信及易扩展的编译器工具链，增强和引入了多种编译优化技术，支持C/C++/Fortran等编程语言及对应编程语言的OpenMP扩展。

详情请参见**《毕昇编译器用户指南》**。

KMP_DISABLE_MEMKIND_INIT 环境变量说明

Bisheng编译器5.0.0.2版本新增KMP_DISABLE_MEMKIND_INIT环境变量，详细说明，请参见**表3-1**。

表 3-1 KMP_DISABLE_MEMKIND_INIT 环境变量说明

环境变量配置	说明
KMP_DISABLE_MEMKIND_INIT=true	将源码中OpenMP高带宽内存相关扩展接口（omp_high_bw_mem_alloc等）替换为DDR版本的实现，相关接口性能会受一定影响。
KMP_DISABLE_MEMKIND_INIT=false	将源码中OpenMP高带宽内存相关扩展接口（omp_high_bw_mem_alloc等）替换为memkind版本的实现，在具备高带宽内存的硬件平台上，相关接口具有更好性能。

Bisheng编译器中该环境变量默认配置为KMP_DISABLE_MEMKIND_INIT=false，维持原版本实现。但为优化OMP相关程序的初始化时间，在HPCKit Bisheng编译器相关环境变量脚本中，已将该变量配置为KMP_DISABLE_MEMKIND_INIT=true。若用户认为该配置影响业务性能，可通过修改Bisheng编译器对应的modulefile文件或setvars.sh文件，将环境变量修改为KMP_DISABLE_MEMKIND_INIT=false。

 版权所有 © 华为技术有限公司

KMP_BLOCKTIME 环境变量说明

KML_BLOCKTIME是LLVM/OpenMP运行时环境变量。该变量用于控制并行区域执行完毕后，工作线程在进入阻塞状态前保持自旋等待的主动轮询时长。

当程序离开一个并行区域时，工作线程不会立即释放CPU资源，而是进入自旋阶段以优化后续并行任务的响应延迟。其具体状态行为取决于KMP_BLOCKTIME的设定值

- 在KMP_BLOCKTIME设定的时间内：线程保持活跃并积极轮询新任务。如果在期间有新的并行区域启动，线程可以迅速做出响应。
- 超过KMP_BLOCKTIME时间后：如果依然没有新任务，线程就会进入睡眠状态，将CPU资源还给操作系统内核。此后如需再次触发，将引入系统内核开销。

保持OpenMP默认时长设置，请执行：

```
export KMP_BLOCKTIME=200ms
```

如有其他配置需要，可参照如下方式：

```
# 设置等待时间为 5 毫秒
export KMP_BLOCKTIME=5ms

# 设置等待时间为 500 微秒
export KMP_BLOCKTIME=500us

# 让工作线程永远保持活跃，不进入睡眠
export KMP_BLOCKTIME=infinite
```

3.2 GCC for openEuler

GCC for openEuler基于开源GCC-12.3.1版本（https://gcc.gnu.org，2023年5月发行）开发，并进行了优化和改进，实现软硬件深度协同优化，挖掘OpenMP、SVE向量化、数学库等领域极致性能，是一种适用于Linux系统的针对鲲鹏920处理器的高性能编译器。GCC for openEuler默认使用场景为鲲鹏920处理器、Arm架构。

详情请参见**《GCC for openEuler用户指南》**。

 版权所有 © 华为技术有限公司

4 Hyper MPI 开发指南

产品描述
操作指南
FAQ
算法地图
命令参数
术语&缩略语

4.1 产品描述

4.1.1 产品介绍

概念介绍

MPI（Message Passing Interface）是一种支持多语言编程的并行计算通信应用接口规范，它定义了一组函数和语法规则，使得多个进程可以在不同的计算机节点上协同工作，并共同完成一个任务。MPI有多种实现，例如Open MPI、Intel MPI、MPICH等。

Hyper MPI（简称HMPI）是在Open MPI的基础上，结合Open UCX（Unified Communication X）框架的点对点通信操作，开发出专为集合通信优化的UCG（Unified Communication Group）框架，该框架中实现了多种集合操作加速算法。Hyper MPI具有高性能、大规模性、可移植性等特点；支持制造、气象和基因测序等场景解决方案，致力于构建以鲲鹏服务器为坚实硬件基础的高性能计算生态。

Hyper MPI代码已开源，请参见**https://gitcode.com/kunpengcompute/hmpi**、**https://gitcode.com/kunpengcompute/hucx**、**https://gitcode.com/kunpengcompute/xucg**。

价值

MPI支持点对点通信和集合通信，可以实现多种通信模式，例如广播、散射、聚集等。其中，仅MPI 3.1标准中定义的集合通信函数就高达30多种。在众多集合通信操作函数中，Allreduce、Bcast、Barrier、Alltoallv、Allgatherv、Gatherv和Scatterv的调用频率较高。

 版权所有 © 华为技术有限公司

Hyper MPI在集合通信模式中，不管是大包数据，还是小包数据，它的通信性能相较于其它MPI的实现，具有以下优势：

- 高性能优化

  Hyper MPI针对Allreduce、Bcast、Barrier、Alltoallv、Allgatherv、Gatherv、Gather、Scatterv、reduce_scatter和reduce_scatter_block集合通信函数实现基于拓扑感知的算法优化（包括非阻塞接口），并支持集合通信异步硬件卸载。Hyper MPI相较于开源OpenMPI，集合通信性能有较高提升。

- 构筑自由生态

  Hyper MPI开源发布，专为以鲲鹏芯片为核心构建的服务器与大规模集群量身打造。鲲鹏服务器凭借单节点核数众多、指令集设计精简、能耗效率卓越以及极具竞争力的价格优势，与Hyper MPI相辅相成，共同构筑起一个高效、经济的高性能计算生态。

功能

- Hyper MPI支持MPI 3.1标准，主要优化的集合通信操作如表4-1所示。

  说明

  Hyper MPI集合通信算法支持最大发送数据包长度为2^{32}字节。

表 4-1 集合通信操作说明

集合操作名称	函数简介	阻塞接口函数原型	非阻塞接口函数原型
Allreduce	Allreduce是MPI组规约函数。 作用：将各个独立进程中的send buffer进行数学（例如加法、乘法）或逻辑（例如与、或）运算，然后将结果同步到通信域内所有进程的receive buffer中。	int MPI_Allreduce(void *sendbuf, void *recvbuf, int count, MPI_Datatype datatype, MPI_Op op, MPI_Comm comm)	int MPI_Iallreduce(const void *sendbuf, void *recvbuf, int count, MPI_Datatype datatype, MPI_Op op, MPI_Comm comm, MPI_Request *request)
Bcast	Bcast是MPI广播操作函数。 作用：root进程将buffer中的信息发送给通信域内其余进程，使得所有进程获取相同的信息。	int MPI_Bcast(void *buffer, int count, MPI_Datatype datatype, int root, MPI_Comm comm)	int MPI_Ibcast(void *buffer, int count, MPI_Datatype datatype, int root, MPI_Comm comm, MPI_Request *request)
Barrier	Barrier是MPI同步函数。 作用：该通信域中全部进程进行同步操作，即保证进程调用函数后具有良好的同步性。	int MPI_Barrier(MPI_Comm comm)	int MPI_Ibarrier(MPI_Comm comm, MPI_Request *request)

版权所有 © 华为技术有限公司

集合操作名称	函数简介	阻塞接口函数原型	非阻塞接口函数原型
Alltoallv	Alltoallv是MPI多对多通信函数。 作用：该通信域中全部进程进行点对点通信操作，发送数据至其他进程，并从其他进程接收数据，且通信中每个进程发送和接收的长度可变。	int MPI_Alltoallv(const void *sendbuf, const int sendcounts[], const int sdispls[], MPI_Datatype sendtype, void *recvbuf, const int recvcounts[], const int rdispls[], MPI_Datatype recvtype, MPI_Comm comm)	int MPI_Ialltoallv(const void *sendbuf, const int sendcounts[], const int sdispls[], MPI_Datatype sendtype, void *recvbuf, const int recvcounts[], const int rdispls[], MPI_Datatype recvtype, MPI_Comm comm, MPI_Request *request)
Allgatherv	Allgatherv是MPI数据收集函数。 作用：收集该通信域中每个进程可变的数据，并将收集的数据传送给该通信域中的所有进程。	int MPI_Allgatherv(const void *sendbuf, int sendcount, MPI_Datatype sendtype, void *recvbuf, const int recvcounts[], const int displs[], MPI_Datatype recvtype, MPI_Comm comm)	int MPI_Iallgatherv(const void *sendbuf, int sendcount, MPI_Datatype sendtype, void *recvbuf, const int recvcounts[], const int displs[], MPI_Datatype recvtype, MPI_Comm comm, MPI_Request *request)
Scatterv	Scatterv是MPI数据分发函数。 作用：该通信域中根进程将不同的数据块分发给该通信域中不同的进程。	int MPI_Scatterv(const void *sendbuf, const int sendcounts[], const int displs[], MPI_Datatype sendtype, void *recvbuf, int recvcount, MPI_Datatype recvtype, int root, MPI_Comm comm)	int MPI_Iscatterv(const void *sendbuf, const int sendcounts[], const int displs[], MPI_Datatype sendtype, void *recvbuf, int recvcount, MPI_Datatype recvtype, int root, MPI_Comm comm, MPI_Request *request)
Gatherv	Gatherv是MPI数据收集函数。 作用：收集该通信域中每个进程可变的数据。	int MPI_Gatherv(const void *sendbuf, int sendcount, MPI_Datatype sendtype, void *recvbuf, const int *recvcounts, const int *displs, MPI_Datatype recvtype, int root, MPI_Comm comm)	int MPI_Igatherv(const void *sendbuf, int sendcount, MPI_Datatype sendtype, void *recvbuf, const int recvcounts[], const int displs[], MPI_Datatype recvtype, int root, MPI_Comm comm, MPI_Request *request)
Gather	Gatherv是MPI数据收集函数。 作用：收集该通信域中每个进程不变的数据。	int MPI_Gather(const void *sendbuf, int sendcount, MPI_Datatype sendtype, void *recvbuf, int recvcount, MPI_Datatype recvtype, int root, MPI_Comm comm)	int MPI_Igather(const void *sendbuf, int sendcount, MPI_Datatype sendtype, void *recvbuf, int recvcount, MPI_Datatype recvtype, int root, MPI_Comm comm, MPI_Request *request)

版权所有 © 华为技术有限公司

集合操作名称	函数简介	阻塞接口函数原型	非阻塞接口函数原型
Reduce_scatter	Reduce_scatter是MPI组规约和分发函数。 作用：将各个独立进程中的send buffer进行数学（例如加法、乘法）或逻辑（例如与、或）运算，然后将不同的数据结果分发给不同的进程，不同进程拿到的数据个数可以不一样。	int MPI_Reduce_scatter(const void *sendbuf, void *recvbuf, const int* recvcounts[], MPI_Datatype datatype, MPI_Op op, MPI_Comm comm)	int MPI_Ireduce_scatter(const void *sendbuf, void *recvbuf, const int* recvcounts[], MPI_Datatype datatype, MPI_Op op, MPI_Comm comm, MPI_Request *request)
Reduce_scatter_block	Reduce_scatter_block是MPI组规约和分发函数。 作用：将各个独立进程中的send buffer进行数学（例如加法、乘法）或逻辑（例如与、或）运算，然后将不同的数据结果分发给不同的进程，不同进程拿到的数据个数一样。	int MPI_Reduce_scatter_block(const void *sendbuf, void *recvbuf, int recvcount, MPI_Datatype datatype, MPI_Op op, MPI_Comm comm)	int MPI_Ireduce_scatter_block(const void *sendbuf, void *recvbuf, int recvcount, MPI_Datatype datatype, MPI_Op op, MPI_Comm comm, MPI_Request *request)

- Hyper MPI支持Singularity容器化，即能够部署在Singularity容器中。

4.1.2 软件架构

Hyper MPI基于Open MPI的MCA（Modular Component Architecture）框架和Open UCX的分层结构，进行自定义组件的新增和优化。

- 与Open MPI相似之处：

  Hyper MPI包含两个软件层：UCX和OMPI层，即HUCX（Hyper Unified Communication X）和Hyper MPI。

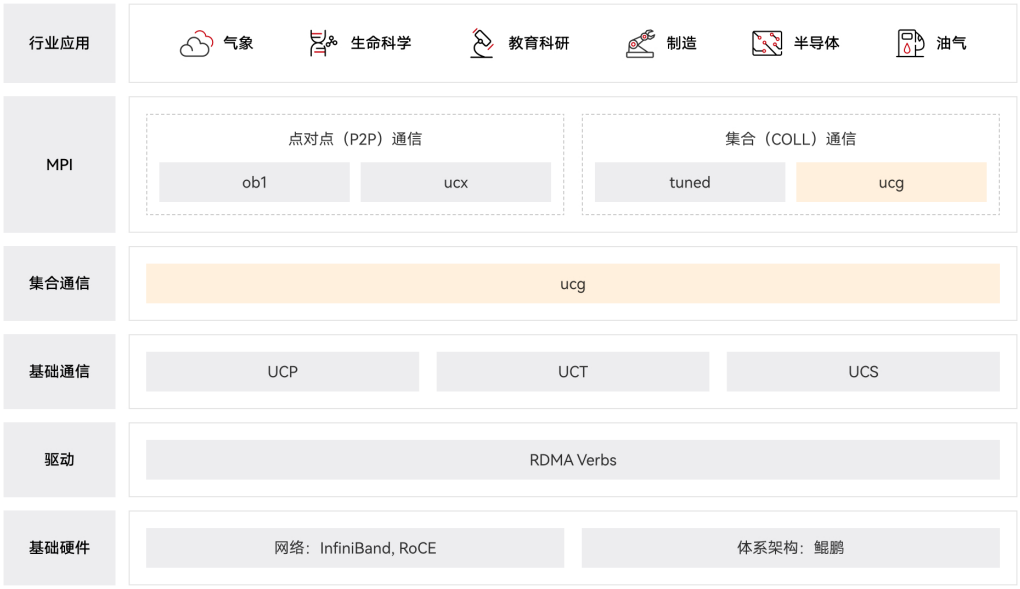
- 与Open MPI不同之处：

  Open MPI采用P2P方式，Hyper MPI在UCX框架基础上增加集合通信模块XUCG（X Unified Communication Group）。该模块南向接口直接依赖于传输层UCT（Unified Communication Transport）和协议层UCP（Unified Communication Protocol），北向接口支持Open MPI的MCA框架中的集合操作组件。

Hyper MPI的软件架构如图4-1所示。

版权所有 © 华为技术有限公司

图 4-1 Hyper MPI 软件架构



Hyper MPI的软件架构中各模块说明如表4-2所示。

表 4-2 Hyper MPI 软件架构中各模块说明

软件层	模块名称	用途
Hyper MPI	P2P	MCA框架中的点对点操作组件。
	COLL	MCA框架中的集合操作组件。
XUCG	UCG	集合通信模块，UCX中的新增集合通信组件，包含UCG集合通信框架和UCG算法加速组件。
HUCX	UCP	UCP协议层，协议层API，基于UCT构建通用协议的实现，包括MPI、OpenSHMEM、PGAS等。
	UCT	通信传输模块，基于硬件的底层API，提供基本的网络收发操作。
	UCS	通用服务与工具，包含一系列数据结构、算法以及通用的系统工具。

4.1.3 关键特性

Hyper MPI针对Allreduce、Bcast、Barrier、Alltoallv、Allgatherv、Gatherv、Gather、Scatterv、reduce_scatter和reduce_scatter_block的阻塞与非阻塞接口这七种集合操作，进行了算法和拓扑感知层面的优化，达到特定集合通信小包情形下的极致性能。相较于其他MPI，Hyper MPI具有以下关键特性：

- 新增UCG模块

  Hyper MPI除了包含Open MPI的所有特性之外，增加了集合通信模块（UCG），该模块中含有大量自研集合通信算法，大幅度提升Allreduce、Bcast、Barrier、

版权所有 © 华为技术有限公司

Alltoallv、Allgatherv、Gatherv、Gather、Scatterv、reduce_scatter和reduce_scatter_block的阻塞与非阻塞接口等集合通信函数的性能。该模块还包含部分集合通信接口的硬件卸载算法，大幅提升计算、通信交叠率。

- 新增KuComm组件

  Hyper MPI在hmpi层集合通信coll框架下新增KuComm分层拓扑组件，提供分层次拓扑框架能力，同时提供Bcast/Alltoall的阻塞集合操作接口，其他集合操作及点对点接口复用OpenMPI 4.x提供的功能。该组件为后续不同层级通信域使用各自最适合的集合通信组件，进行极致性能优化提供了良好基础。

- 新增FP16数据类型

  Hyper MPI在hmpi层新增FP16半精度数据类型支持，支持AI4S场景基础加速库的开发。

  在mpi扩展模块中新增c/c++语言的半精度数据类型：MPIX_SHORT_FLOAT、MPIX_C_SHORT_FLOAT_COMPLEX、MPIX_CXX_SHORT_FLOAT_COMPLEX类型，使用时需要引入<mpi-ext.h>头文件。当前HPCKit中的编译器不支持Fortran的半精度类型real2和complex4，因此hmpi内部也暂不使能Fortran的半精度特性。

- 支持鲲鹏环境

  Hyper MPI可以在以鲲鹏芯片为核心的服务器和集群上发挥作用。

- 支持动态加载闭源组件

  Hyper MPI开源版本安装后，可以将Hyper MPI闭源组件安装到开源Hyper MPI安装目录进行闭源特性使能，无需重新编译安装Hyper MPI开源版本，Hyper MPI闭源组件提供进一步的优化特性。

4.2 操作指南

4.2.1 提交 MPI 作业

Hyper MPI并行作业的调度，通过配置Linux的SSH免密登录直接运行**mpirun**命令进行提交。

4.2.2 运行 MPI 作业

本文介绍运行MPI作业的方法。

背景信息

Hyper MPI支持的集合操作包含Allreduce、Bcast、Barrier、Alltoallv、Allgatherv、Gatherv和Scatterv。Hyper MPI支持如下算法模式。

- 默认算法模式：使用coll-ucg模块运行MPI作业，在运行作业时不需要添加“--mca coll ^ucg”参数。
- 指定算法模式：使用Open MPI原有的集合通信方式运行MPI作业，在运行作业时需要添加“--mca coll ^ucg”参数。

指定算法模式的示例如下：

mpirun -np *16* **-N** *2* **--hostfile** *hf* **--mca coll ^ucg** *test_case*

 版权所有 © 华为技术有限公司

说明

- -np：表示所有节点总进程数，计算方式为单节点的进程数量（N）*节点数量。
- -N：表示每个节点的进程数。
- --hostfile：表示进程运行节点的配置文件。
- test_case：表示mpicc编译后的二进制文件。

Hyper MPI默认对业务进程按照core的策略进行绑核，如果业务进程为多线程程序，为保障线程的正常调度，需要根据业务线程规模手动修改绑核配置，例如：

- 可以通过指定rankfile文件的方式实现细粒度绑核，rankfile文件的编写规则为：

  rank *0*={*ip1*} **slot=***0-7*

  rank *1*={*ip2*} **slot=***0-7*

  上述规则表示将MPI的0号任务绑定在ip1节点的0-7号CPU核上；将MPI的1号任务绑定在ip2节点的0-7号CPU核上。

  运行命令示例如下：

  mpirun --rankfile *rf test_case*

  说明

  --rankfile配置项无法与--hostfile或-N选项同时使用。

- 可以通过配置**--bind-to core**或**--bind-to numa**或**--bind-to socket**实现粗粒度绑核。运行命令示例如下：

  mpirun -np *2* **-N** *1* **--hostfile** *hf* **--bind-to** *numa test_case*

通过默认算法运行 MPI 作业

Hyper MPI在默认情况下根据包长、每个节点的进程数（PPN）和节点数三个参数选择算法，在使用时直接运行**mpirun**命令即可，无需添加额外参数。

- Allreduce、Bcast和Allgatherv集合操作默认算法根据包长、PPN和节点数三个参数进行选择。
- Barrier、Gatherv和Scatterv集合操作默认算法根据PPN和节点数进行选择。
- Alltoallv集合操作默认算法是Ladd。

运行命令示例如下：

mpirun -np *16* **-N** *2* **--hostfile** *hf* **test_case**

说明

Hyper MPI中并非每一个算法都支持所有的场景，在一些不支持场景下，会回退至支持的算法，详情请参见**4.2.3.3 特性支持与功能回退说明**。

通过指定算法运行 MPI 作业

以Allreduce的算法参数**UCG_PLANC_UCX**_*ALLREDUCE*_**ATTR**为例进行介绍。通过指定算法运行MPI作业的命令参数请参见**指定算法支持的命令参数**。

- 方法一：通过命令行参数指定算法

  a. 使用PuTTY工具，以Hyper MPI普通用户（例如“hmpi_user”）登录至作业执行节点。

版权所有 © 华为技术有限公司

b. 在**mpirun**命令中增加如下参数。

-x UCG_PLANC_UCX_*ALLREDUCE*_**ATTR=I:***id*[**S:***score***R:***start-end*]

- 方法二：通过设置环境变量的方式来选择对应的算法

a. 使用PuTTY工具，以Hyper MPI普通用户（例如“hmpi_user”）登录至作业执行节点。

b. 将以下命令写入所有节点的“~/.bashrc”中。

export UCG_PLANC_UCX_*ALLREDUCE*_**ATTR=I:***id*[**S:***score***R:***start-end*]

说明

- id：表示对应算法的序号。对应算法的序号请参见**4.4 算法地图**。如果算法选择的数值不在有效范围内，则会执行默认算法。
- score：表示算法分数，分数越高，算法优先级越高。如果不指定，默认该算法优先级最高。取值为任意自然数。
- start-end：表示算法适用的包大小范围。如果不指定，默认该算法适用于所有包大小。start、end的取值为任意自然数。

c. 执行以下命令，使环境变量生效。

source ~/.bashrc

d. 不再需要该环境变量时，需执行以下操作。

删除“~/.bashrc”中已写入的“**export UCG_PLANC_UCX**_*ALLREDUCE*_**ATTR=I:***id*[**S:***score***R:***start-end*]”命令。

执行以下命令，删除已生效的环境变量。

unset UCG_PLANC_UCX_*ALLREDUCE*_**ATTR**

指定算法支持的命令参数

- Hyper MPI兼容Open MPI的命令参数

具体请参见**https://www.open-mpi.org/doc/current/man1/mpirun.1.php**。

- Hyper MPI新增的命令参数

表4-3中为运行MPI作业时的常用命令参数，更多命令参数请参见**4.5 命令参数**。

版权所有 © 华为技术有限公司

表 4-3 指定算法支持的常用命令参数

Hyper MPI命令参数	取值	参数说明
-x UCG_PLANC_UCX_BCAST_ATTR=	• 取值类型：string，格式为I:*id*[S:*score*R:*start-end*]。 • 取值范围： – id：必填，1~6, 8~14。 – score：选填，任意自然数。 – start：选填，任意自然数。 – end：选填，任意自然数。 • 默认值：无固定值，根据生效规则调节赋值。	可选参数，Bcast的算法参数。 示例： **-x UCG_PLANC_UCX_BCAST_ATTR=I:4**
-x UCG_PLANC_UCX_ALLREDUCE_ATTR=	• 取值类型：string，格式为I:*id*[S:*score*R:*start-end*]。 • 取值范围： – id：必填，1~8,11~14。 – score：选填，任意自然数。 – start：选填，任意自然数。 – end：选填，任意自然数。 • 默认值：无固定值，根据生效规则调节赋值。	可选参数，Allreduce的算法参数。 示例： **-x UCG_PLANC_UCX_ALLREDUCE_ATTR=I:7**

版权所有 © 华为技术有限公司

Hyper MPI命令参数	取值	参数说明
-x UCG_PLANC_UCX_BARRIER_ATTR=	• 取值类型：string，格式为I:*id*[S:*score*R:*start-end*]。 • 取值范围： – id：必填，1~7,10。 – score：选填，任意自然数。 – start：选填，任意自然数。 – end：选填，任意自然数。 • 默认值：无固定值，根据生效规则调节赋值。	可选参数，Barrier的算法参数。 示例： **-x UCG_PLANC_UCX_BARRIER_ATTR=I:1**
-x UCG_PLANC_UCX_ALLTOALLV_ATTR=	• 取值类型：string，格式为I:*id*[S:*score*R:*start-end*]。 • 取值范围： – id：必填，1~5。 – score：选填，任意自然数。 – start：选填，任意自然数。 – end：选填，任意自然数。 • 默认值：无固定值，根据生效规则调节赋值。	可选参数，Alltoallv的算法参数。 示例： **-x UCG_PLANC_UCX_ALLTOALLV_ATTR=I:1**

版权所有 © 华为技术有限公司

Hyper MPI命令参数	取值	参数说明
-x UCG_PLANC_UCX_ALLGATHERV_ATTR=	• 取值类型：string，格式为 I:*id*[S:*score*R:*start-end*]。 • 取值范围： – id：必填，1~6。 – score：选填，任意自然数。 – start：选填，任意自然数。 – end：选填，任意自然数。 • 默认值：无固定值，根据生效规则调节赋值。	可选参数，Allgatherv的算法参数。 示例： **-x UCG_PLANC_UCX_ALLGATHERV_ATTR=I:3**
-x UCG_PLANC_UCX_SCATTERV_ATTR=	• 取值类型：string，格式为 I:*id*[S:*score*R:*start-end*]。 • 取值范围： – id：必填，1~3。 – score：选填，任意自然数。 – start：选填，任意自然数。 – end：选填，任意自然数。 • 默认值：无固定值，根据生效规则调节赋值。	可选参数，Scatterv的算法参数。 示例： **-x UCG_PLANC_UCX_SCATTERV_ATTR=I:1**

版权所有 © 华为技术有限公司

Hyper MPI命令参数	取值	参数说明
-x UCG_PLANC_UCX_GATHERV_ATTR=	• 取值类型：string，格式为 I:*id*[S:*score*R:*start-end*]。 • 取值范围： – id：必填，1~4。 – score：选填，任意自然数。 – start：选填，任意自然数。 – end：选填，任意自然数。 • 默认值：无固定值，根据生效规则调节赋值。	可选参数，Gatherv的算法参数。 示例： **-x UCG_PLANC_UCX_GATHERV_ATTR=I:1**
-x UCG_PLANC_UCX_GATHER_ATTR=	• 取值类型：string，格式为 I:*id*[S:*score*R:*start-end*]。 • 取值范围： – id：必填，1~4。 – score：选填，任意自然数。 – start：选填，任意自然数。 – end：选填，任意自然数。 • 默认值：无固定值，根据生效规则调节赋值。	可选参数，Gather的算法参数。 示例： **-x UCG_PLANC_UCX_GATHER_ATTR=I:1**

版权所有 © 华为技术有限公司

Hyper MPI命令参数	取值	参数说明
-x UCG_PLANC_STARS_BCAST_ATTR=	• 取值类型：string，格式为I:*id*[S:*score*R:*start-end*]。 • 取值范围： – id：必填，1~4。 – score：选填，任意自然数。 – start：选填，任意自然数。 – end：选填，任意自然数。 • 默认值：无固定值，根据生效规则调节赋值。	可选参数，Bcast的卸载算法参数。 示例： **-x UCG_PLANC_STARS_BCAST_ATTR=I:1**
-x UCG_PLANC_STARS_ALLGATHERV_ATTR=	• 取值类型：string，格式为I:*id*[S:*score*R:*start-end*]。 • 取值范围： – id：必填，1。 – score：选填，任意自然数。 – start：选填，任意自然数。 – end：选填，任意自然数。 • 默认值：无固定值，根据生效规则调节赋值。	可选参数，Allgatherv的卸载算法参数。 示例： **-x UCG_PLANC_STARS_ALLGATHERV_ATTR=I:1**

版权所有 © 华为技术有限公司

Hyper MPI命令参数	取值	参数说明
-x UCG_PLANC_STARS_SCATTERV_ATTR=	• 取值类型：string，格式为 I:*id*[S:*score*R:*start-end*]。 • 取值范围： – id：必填，1。 – score：选填，任意自然数。 – start：选填，任意自然数。 – end：选填，任意自然数。 • 默认值：无固定值，根据生效规则调节赋值。	可选参数，Scatterv的卸载算法参数。 示例： **-x UCG_PLANC_STARS_SCATTERV_ATTR=I:1**
-x UCG_PLANC_STARS_ALLTOALLV_ATTR=	• 取值类型：string，格式为 I:*id*[S:*score*R:*start-end*]。 • 取值范围： – id：必填，1。 – score：选填，任意自然数。 – start：选填，任意自然数。 – end：选填，任意自然数。 • 默认值：无固定值，根据生效规则调节赋值。	可选参数，Alltoallv的卸载算法参数。 示例： **-x UCG_PLANC_STARS_ALLTOALLV_ATTR=I:1**

版权所有 © 华为技术有限公司

Hyper MPI命令参数	取值	参数说明
-x UCG_PLANC_STARS_BARRIER_ATTR=	• 取值类型：string，格式为 I:*id*[S:*score*R:*start-end*]。 • 取值范围： – id：必填，1。 – score：选填，任意自然数。 – start：选填，任意自然数。 – end：选填，任意自然数。 • 默认值：无固定值，根据生效规则调节赋值。	可选参数，Barrier的卸载算法参数。 示例： **-x UCG_PLANC_STARS_BARRIER_ATTR=I:1**
-x UCG_PLANC_UCX_IBCAST_ATTR=	• 取值类型：string，格式为 I:*id*[S:*score*R:*start-end*]。 • 取值范围： – id：必填，1~6, 8~14。 – score：选填，任意自然数。 – start：选填，任意自然数。 – end：选填，任意自然数。 • 默认值：无固定值，根据生效规则调节赋值。	可选参数，IBcast的算法参数。 示例： **-x UCG_PLANC_UCX_IBCAST_ATTR=I:4**

版权所有 © 华为技术有限公司

Hyper MPI命令参数	取值	参数说明
-x UCG_PLANC_UCX_IALLREDUCE_ATTR=	• 取值类型：string，格式为I:*id*[S:*score*R:*start-end*]。 • 取值范围： – id：必填，1~8,11~14。 – score：选填，任意自然数。 – start：选填，任意自然数。 – end：选填，任意自然数。 • 默认值：无固定值，根据生效规则调节赋值。	可选参数，IAllreduce的算法参数。 示例： **-x UCG_PLANC_UCX_IALLREDUCE_ATTR=I:7**
-x UCG_PLANC_UCX_IBARRIER_ATTR=	• 取值类型：string，格式为I:*id*[S:*score*R:*start-end*]。 • 取值范围： – id：必填，1~7,10。 – score：选填，任意自然数。 – start：选填，任意自然数。 – end：选填，任意自然数。 • 默认值：无固定值，根据生效规则调节赋值。	可选参数，IBarrier的算法参数。 示例： **-x UCG_PLANC_UCX_IBARRIER_ATTR=I:1**

版权所有 © 华为技术有限公司

Hyper MPI命令参数	取值	参数说明
-x UCG_PLANC_UCX_IALLTOALLV_ATTR=	• 取值类型：string，格式为 I:*id*[S:*score*R:*start-end*]。 • 取值范围： – id：必填，1~5。 – score：选填，任意自然数。 – start：选填，任意自然数。 – end：选填，任意自然数。 • 默认值：无固定值，根据生效规则调节赋值。	可选参数，IAlltoallv的算法参数。 示例： **-x UCG_PLANC_UCX_IALLTOALLV_ATTR=I:1**
-x UCG_PLANC_UCX_IALLGATHERV_ATTR=	• 取值类型：string，格式为 I:*id*[S:*score*R:*start-end*]。 • 取值范围： – id：必填，1~6。 – score：选填，任意自然数。 – start：选填，任意自然数。 – end：选填，任意自然数。 • 默认值：无固定值，根据生效规则调节赋值。	可选参数，IAllgatherv的算法参数。 示例： **-x UCG_PLANC_UCX_IALLGATHERV_ATTR=I:3**

版权所有 © 华为技术有限公司

Hyper MPI命令参数	取值	参数说明
-x UCG_PLANC_UCX_ISCATTERV_ATTR=	• 取值类型：string，格式为 I:*id*[S:*score*R:*start-end*]。 • 取值范围： – id：必填，1~3。 – score：选填，任意自然数。 – start：选填，任意自然数。 – end：选填，任意自然数。 • 默认值：无固定值，根据生效规则调节赋值。	可选参数，IScatterv的算法参数。 示例： **-x UCG_PLANC_UCX_ISCATTERV_ATTR=I:1**
-x UCG_PLANC_UCX_IGATHERV_ATTR=	• 取值类型：string，格式为 I:*id*[S:*score*R:*start-end*]。 • 取值范围： – id：必填，1~4。 – score：选填，任意自然数。 – start：选填，任意自然数。 – end：选填，任意自然数。 • 默认值：无固定值，根据生效规则调节赋值。	可选参数，IGatherv的算法参数。 示例： **-x UCG_PLANC_UCX_IGATHERV_ATTR=I:1**

版权所有 © 华为技术有限公司

Hyper MPI命令参数	取值	参数说明
-x UCG_PLANC_UCX_IGATHER_ATTR=	• 取值类型：string，格式为 I:*id*[S:*score*R:*start-end*]。 • 取值范围： – id：必填，1~4。 – score：选填，任意自然数。 – start：选填，任意自然数。 – end：选填，任意自然数。 • 默认值：无固定值，根据生效规则调节赋值。	可选参数，IGather的算法参数。 示例： **-x UCG_PLANC_UCX_IGATHER_ATTR=I:1**
-x UCG_PLANC_STARS_IBCAST_ATTR=	• 取值类型：string，格式为 I:*id*[S:*score*R:*start-end*]。 • 取值范围： – id：必填，1~4。 – score：选填，任意自然数。 – start：选填，任意自然数。 – end：选填，任意自然数。 • 默认值：无固定值，根据生效规则调节赋值。	可选参数，IBcast的卸载算法参数。 示例： **-x UCG_PLANC_STARS_IBCAST_ATTR=I:1**

版权所有 © 华为技术有限公司

Hyper MPI命令参数	取值	参数说明
-x UCG_PLANC_STARS_IALLGATHERV_ATTR=	• 取值类型：string，格式为I:*id*[S:*score*R:*start-end*]。 • 取值范围： – id：必填，1。 – score：选填，任意自然数。 – start：选填，任意自然数。 – end：选填，任意自然数。 • 默认值：无固定值，根据生效规则调节赋值。	可选参数，IAllgatherv的卸载算法参数。 示例： **-x UCG_PLANC_STARS_IALLGATHERV_ATTR=I:1**
-x UCG_PLANC_STARS_ISCATTERV_ATTR=	• 取值类型：string，格式为I:*id*[S:*score*R:*start-end*]。 • 取值范围： – id：必填，1。 – score：选填，任意自然数。 – start：选填，任意自然数。 – end：选填，任意自然数。 • 默认值：无固定值，根据生效规则调节赋值。	可选参数，IScatterv的卸载算法参数。 示例： **-x UCG_PLANC_STARS_ISCATTERV_ATTR=I:1**

版权所有 © 华为技术有限公司

Hyper MPI命令参数	取值	参数说明
-x UCG_PLANC_STARS_IALLTOALLV_ATTR=	• 取值类型：string，格式为I:*id*[S:*score*R:*start-end*]。 • 取值范围： – id：必填，1。 – score：选填，任意自然数。 – start：选填，任意自然数。 – end：选填，任意自然数。 • 默认值：无固定值，根据生效规则调节赋值。	可选参数，IAlltoallv的卸载算法参数。 示例： **-x UCG_PLANC_STARS_IALLTOALLV_ATTR=I:1**
-x UCG_PLANC_STARS_IBARRIER_ATTR=	• 取值类型：string，格式为I:*id*[S:*score*R:*start-end*]。 • 取值范围： – id：必填，1。 – score：选填，任意自然数。 – start：选填，任意自然数。 – end：选填，任意自然数。 • 默认值：无固定值，根据生效规则调节赋值。	可选参数，IBarrier的卸载算法参数。 示例： **-x UCG_PLANC_STARS_IBARRIER_ATTR=I:1**

版权所有 © 华为技术有限公司

Hyper MPI命令参数	取值	参数说明
-x UCG_PLANC_UCX_REDUCE_SCATTER_BLOCK_ATTR=	• 取值类型：string，格式为I:*id*[S:*score*R:*start-end*]。 • 取值范围： – id：必填，1~2。 – score：选填，任意自然数。 – start：选填，任意自然数。 – end：选填，任意自然数。 • 默认值：无固定值，根据生效规则调节赋值。	可选参数，Reduce_scatter_block的算法参数。 示例： **-x UCG_PLANC_UCX_REDUCE_SCATTER_BLOCK_ATTR=I:1**
-x UCG_PLANC_UCX_IREDUCE_SCATTER_BLOCK_ATTR=	• 取值类型：string，格式为I:*id*[S:*score*R:*start-end*]。 • 取值范围： – id：必填，1~2。 – score：选填，任意自然数。 – start：选填，任意自然数。 – end：选填，任意自然数。 • 默认值：无固定值，根据生效规则调节赋值。	可选参数，IReduce_scatter_block的算法参数。 示例： **-x UCG_PLANC_UCX_IREDUCE_SCATTER_BLOCK_ATTR=I:1**

版权所有 © 华为技术有限公司

Hyper MPI命令参数	取值	参数说明
-x UCG_PLANC_UCX_REDUCE_SCATTER_ATTR=	• 取值类型：string，格式为I:*id*[S:*score*R:*start-end*]。 • 取值范围： – id：必填，1~2。 – score：选填，任意自然数。 – start：选填，任意自然数。 – end：选填，任意自然数。 • 默认值：无固定值，根据生效规则调节赋值。	可选参数，Reduce_scatter的算法参数。 示例： **-x UCG_PLANC_UCX_REDUCE_SCATTER_ATTR=I:1**
-x UCG_PLANC_UCX_IREDUCE_SCATTER_ATTR=	• 取值类型：string，格式为I:*id*[S:*score*R:*start-end*]。 • 取值范围： – id：必填，1~2。 – score：选填，任意自然数。 – start：选填，任意自然数。 – end：选填，任意自然数。 • 默认值：无固定值，根据生效规则调节赋值。	可选参数，IReduce_scatter的算法参数。 示例： **-x UCG_PLANC_UCX_IREDUCE_SCATTER_ATTR=I:1**

4.2.3 集合操作算法选择说明

4.2.3.1 指定算法说明

说明

以下算法序号对应的算法内容请参见**4.4 算法地图**。

关于 Allreduce 指定算法的说明

- 在选用算法13、14时，为保证进程数在Socket上平均分布，需要加上以下选项，命令如下：

  --map-by-socket --rank-by-core

版权所有 © 华为技术有限公司

- 在选用算法5、6时，节点内采用了K-nomial算法，可通过调节K-nomial tree结构的K值参数值，寻找最优参数值，从而获得最佳性能。K值范围为(0,进程数]。示例命令如下：

  -x UCG_PLANC_UCX_ALLREDUCE_FANOUT_INTRA_DEGREE=*3*

  -x UCG_PLANC_UCX_ALLREDUCE_FANIN_INTRA_DEGREE=*8*

- 在选用算法7、8时，节点内与节点间都采用了K-nomial算法，可通过调节K-nomial tree结构的K值参数值，寻找最优参数值，从而获得最佳性能。K值范围为(0,进程数]。示例命令如下：

  -x UCG_PLANC_UCX_ALLREDUCE_FANOUT_INTER_DEGREE=*7* **-x UCG_PLANC_UCX_ALLREDUCE_FANIN_INTER_DEGREE=***7* **-x UCG_PLANC_UCX_ALLREDUCE_FANOUT_INTRA_DEGREE=***3* **-x UCG_PLANC_UCX_ALLREDUCE_FANIN_INTRA_DEGREE=***8*

为提高性能，也可加入以下选项，调整传输模式：

-x UCX_TLS=sm,rc_x

综上，在调用MPI_Allreduce和MPI_Iallreduce时运行命令示例如下（鲲鹏芯片）：

mpirun -np *16* **-N** *2* **--hostfile hf8 -x UCX_TLS=sm,rc_x -x UCG_PLANC_UCX_ALLREDUCE_ATTR=I:***n***S:***200***R:0- test_case**

关于 Bcast 指定算法的说明

- 在选用算法3时，节点间采用了K-nomial算法，可通过调节K值参数值，寻找最优参数值，从而获得最佳性能。K值范围为(0,进程数]。命令示例如下：

  -x UCG_PLANC_UCX_BCAST_NA_KNTREE_INTER_DEGREE=*7*

- 在选用算法4时，节点内与节点间都采用了K-nomial算法，可通过调节K值参数值，寻找最优参数值，从而获得最佳性能。K值范围为(0,进程数]。命令示例如下：

  -x UCG_PLANC_UCX_BCAST_NA_KNTREE_INTER_DEGREE=*7*

  -x UCG_PLANC_UCX_BCAST_NA_KNTREE_INTRA_DEGREE=*3*

- 在选用算法9时，可通过以下参数值调整ESBT算法中划分的块数，寻找最优参数值，从而获得最佳性能。默认值为0，表示auto。命令示例如下：

  -x UCG_PLANC_UCX_BCAST_ESBT_BLOCKS=*3*

  为提高性能，也可加入以下选项（共享内存Shared Memory和基于任意传输的可靠连接），调整传输模式：

-x UCX_TLS=sm,rc_x

综上，在调用MPI_Bcast和MPI_Ibcast时运行命令示例如下（鲲鹏芯片）：

mpirun -np *16* **-N** *2* **--hostfile hf8 -x UCX_TLS=sm,rc_x -x UCG_PLANC_UCX_BCAST_ATTR=I:***n***S:***200***R:0- test_case**

关于 Barrier 指定算法的说明

从**4.4 算法地图**可知，Barrier的算法是Allreduce算法的子集，因此可参考Allreduce指定算法说明，此处不再赘述。

版权所有 © 华为技术有限公司

关于 Alltoallv 指定算法的说明

在选用算法1时，可通过调整以下参数值，调整每次的通信请求次数，请根据实际情况替换。命令示例如下：

-x UCG_PLANC_UCX_ALLTOALLV_LADD_THROTTLED_FACTOR=*2*

在选用算法5时，需要开启以下参数使能共享内存。命令示例如下：

-x UCG_USE_SHM_POOL=y

为提高性能，也可加入以下选项，调整传输模式：

-x UCX_TLS=sm,rc_x

综上，在调用MPI_Alltoallv和MPI_Ialltoallv时运行命令示例如下（鲲鹏芯片）：

mpirun -np *16* **-N** *2* **--hostfile hf8 -x UCX_TLS=sm,rc_x -x UCG_PLANC_UCX_ALLTOALLV_ATTR=I:***n***S:***200***R:0- test_case**

关于 Scatterv 指定算法的说明

- 在选用算法1时，可通过调整以下参数值，寻找最优参数值，从而获得最佳性能。命令示例如下：

  -x UCG_PLANC_UCX_SCATTERV_MIN_SEND_BATCH=*7*

  -x UCG_PLANC_UCX_SCATTERV_MAX_SEND_BATCH=*8*
- 在选用算法2时，采用了K-nomial算法，可通过调整K值参数值，寻找最优参数值，从而获得最佳性能。命令示例如下：

  -x UCG_PLANC_UCX_SCATTERV_KNTREE_DEGREE=*3*
- 在选用算法3时，采用了Node-aware K-nomial tree算法，由于节点间和节点内通信都采用了K-nomial算法，可通过调整节点间和节点内通信的K值参数值，寻找最优参数值，从而获得最佳性能。命令示例如下：

  -x UCG_PLANC_UCX_SCATTERV_NA_KNTREE_INTER_DEGREE=*4*

  -x UCG_PLANC_UCX_SCATTERV_NA_KNTREE_INTRA_DEGREE=*3*

  说明

  - Scatterv操作的算法3在数据量较小的场景下性能较优，数据量大的场景下性能较差。
  - 由于Scatterv操作无法根据数据量的大小自动选择最优算法，建议在数据量较小的场景下使用算法3。

为提高性能，也可加入以下选项，调整传输模式：

-x UCX_TLS=sm,rc_x

综上，在调用MPI_Scatterv和MPI_Iscatterv时运行命令示例如下（鲲鹏芯片）：

mpirun -np *16* **-N** *2* **--hostfile hf8 -x UCX_TLS=sm,rc_x -x UCG_PLANC_UCX_SCATTERV_ATTR=I:***n***S:***200***R:0- test_case**

关于 Allgatherv 指定算法的说明

为提高性能，可加入以下选项，调整传输模式：

-x UCX_TLS=sm,rc_x

综上，在调用MPI_Allgatherv和MPI_Iallgatherv时运行命令示例如下（鲲鹏芯片）：

 版权所有 © 华为技术有限公司

mpirun -np *16* **-N** *2* **--hostfile hf8 -x UCX_TLS=sm,rc_x -x UCG_PLANC_UCX_ALLGATHERV_ATTR=I:***n***S:***200***R:0- test_case**

关于 Gather 指定算法的说明

- 在选用算法3时，采用了K-nomial算法，可通过调整K值参数值，寻找最优参数值，从而获得最佳性能。命令示例如下：

  -x UCG_PLANC_UCX_GATHER_KNTREE_DEGREE=*3*

- 在选用算法4时，采用了Node-aware K-nomial tree算法，由于节点间和节点内通信都采用了K-nomial算法，可通过调整节点间和节点内通信的K值参数值，寻找最优参数值，从而获得最佳性能。命令示例如下：

  -x UCG_PLANC_UCX_GATHER_NA_KNTREE_INTER_DEGREE=*4*

  -x UCG_PLANC_UCX_GATHER_NA_KNTREE_INTRA_DEGREE=*3*

关于 Gatherv 指定算法的说明

- 在选用算法3时，采用了K-nomial算法，可通过调整K值参数值，寻找最优参数值，从而获得最佳性能。命令示例如下：

  -x UCG_PLANC_UCX_GATHERV_KNTREE_DEGREE=*3*

- 在选用算法4时，采用了Node-aware K-nomial tree算法，由于节点间和节点内通信都采用了K-nomial算法，可通过调整节点间和节点内通信的K值参数值，寻找最优参数值，从而获得最佳性能。命令示例如下：

  -x UCG_PLANC_UCX_GATHERV_NA_KNTREE_INTER_DEGREE=*4*

  -x UCG_PLANC_UCX_GATHERV_NA_KNTREE_INTRA_DEGREE=*3*

4.2.3.2 推荐命令

在调用Hyper MPI运行应用时，可能会同时调用Allreduce、Bcast、Barrier、Alltoallv、Allgatherv、Gatherv和Scatterv操作，此时的推荐算法命令示例如下（鲲鹏芯片）：

mpirun -np *16* **-N** *2* **--hostfile hf --map-by socket --rank-by core --bind-to core -x UCX_TLS=sm,rc_x -x UCG_PLANC_UCX_BCAST_ATTR=I:***3* **-x UCG_PLANC_UCX_ALLREDUCE_ATTR=I:***6* **-x UCG_PLANC_UCX_BARRIER_ATTR=I:***5***S:***200***R:***0* **-x UCG_PLANC_UCX_ALLGATHERV_ATTR=I:***5***S:***200***R:***0* **-x UCG_PLANC_UCX_GATHERV_ATTR=I:***2* **-x UCG_PLANC_UCX_SCATTERV_ATTR=I:***1* **-x UCG_PLANC_UCX_ALLTOALLV_ATTR=I:***1* **test_case**

4.2.3.3 特性支持与功能回退说明

Hyper MPI中并非每一个算法都支持所有的场景，以下几种场景属于部分算法支持。

场景一：节点进程不均衡（即 PPN 不均衡）

PPN：Processes Per Node，每节点进程数。

使用**mpirun**命令提交任务不指定PPN或者使用rankfile等情况下，会出现节点进程不均衡的场景，当指定Hyper MPI中Allreduce算法11、13、14，Allgatherv算法6，Scatterv算法3，Barrier算法10，Alltoallv算法2时，会自动回退至支持的算法。

版权所有 © 华为技术有限公司

场景二：Socket 进程不均衡（即 PPS 不均衡）

PPS：Processes Per Socket，每CPU插槽进程数。

使用**mpirun**命令提交任务不指定PPS或者使用rankfile等情况下，会出现Socket进程不均衡的场景，当指定Hyper MPI中Allreduce算法13、14时，会自动回退至支持的算法。

场景三：非对易操作

MPI标准支持用户在Allreduce内进行自定义数学操作，即不局限于求和（MPI_SUM）、乘积（MPI_PROD）等MPI内置的数学操作；用户在自定义数学操作时会出现操作本身不对易的情况，如矩阵乘法。Allreduce算法1支持此类操作。

当用户指定算法不是算法1时，Hyper MPI会根据数据包大小回退成算法1（中小包）。

场景四：指定--bind-to none 或者--bind-to board

当MPI指定--bind-to为none或者board时，Allreduce算法3、6、8、13、14和Barrier算法3、5、7不支持此场景，会自动回退至支持的算法。

场景五：发送的数量 count 小于所有节点总进程数 np 或所有节点进程数减一

总进程数np即通过mpirun -np参数指定的所有MPI进程的总数。其计算逻辑为：np = 节点数 × PPN。

当发送的数量count小于所有节点进程数np时，Allreduce算法4、12、13、14和Bcast算法8、11不支持此场景，会自动回退至支持的算法。

当发送的数量count小于所有节点进程数减一（np-1）时，Bcast算法14和Bcast卸载算法3、4不支持此场景，会自动回退至支持的算法。

场景六：发送的数量 count 对 ppn 取余不等于 0

当发送的数量count对ppn取余不等于0时，Allreduce算法13不支持此场景，会自动回退至支持的算法。

场景七：PPS 为 1

当PPS=1时，Allreduce算法14不支持此场景，会自动回退至支持的算法。

场景八：PPN 为 1

当PPN=1时，Allreduce算法11、13、14，Allgatherv算法6，Scatterv算法3，Barrier算法10，Alltoallv算法2不支持此场景，会自动回退至支持的算法。

场景九：np 不为二的 N 次方

当np不为二的N次方时，Allreduce算法11，Barrier算法10不支持此场景，会自动回退至支持的算法。

版权所有 © 华为技术有限公司

场景十：np 为奇数

当np为奇数时，Hyper MPI中Allgatherv算法1不支持此场景，会自动回退至支持的算法。

场景十一：指定--rank-by node 或者--rank-by board

当MPI指定--rank-by node或者--rank-by board时，Alltoallv算法2和Scatterv算法3不支持此场景，将自动回退至支持的算法。

场景十二：节点数为 1

当节点数为1时，Hyper MPI中Alltoallv算法2、Allgatherv算法6和Scatterv算法3不支持此场景，将自动回退至支持的算法。

场景十三：sendbuf 为 MPI_IN_PLACE

当sendbuf为MPI_IN_PLACE时，Hyper MPI中Alltoallv所有算法和Scatterv算法3不支持此场景，回退到Open MPI。

场景十四：Alltoallv 集合操作 send 或 recv total size 超过 2^32 - 1

此场景下，Hyper MPI中Alltoallv算法2（node-aware plummer）由于是运行时数据，无法在回退时给出判断，会在运行时报错，并给出提示“Alltoallv node-aware plummer don't support datasize >= 2^32 bytes”。

场景十五：Alltoallv 集合操作节点内所有进程 sendcounts 总和超过 2^31

此场景下，Hyper MPI中Alltoallv算法2（node-aware plummer）由于是运行时数据，无法在回退时给出判断，会在运行时报错，并给出提示“Alltoallv node-aware plummer gatherv phase don't support displs >= 2^31 bytes”，sendcounts总和计算公式如下：

$$\sum_{i=0}^{ppn-1}\sum_{j=0}^{p-1} sendcounts[i][j]$$

其中，p为总进程数；ppn为每节点进程数；sendcounts[i]为节点内第i个进程的sendcounts数组。

场景十六：Alltoallv 集合操作各进程 sendcounts 和 recvcounts 大小不一致

此场景下，若命令参数中不指定Alltoallv的具体算法，可能会出现不同进程回退到不同算法的情况，导致程序无法正常运行。在确保各进程sendcounts和recvcounts大小一致的情况下，可以不指定Alltoallv的具体算法，默认选择性能最优算法。

场景十七：进程数小于 3

当进程数小于3时，Bcast卸载算法1不支持此场景，将自动回退至支持的算法。

版权所有 © 华为技术有限公司

场景十八：进程数小于 4

当进程数小于4时，Bcast卸载算法2不支持此场景，将自动回退至支持的算法。

场景十九：Alltoallv plummer-opt 算法没开启 USE_SHM_POOL 特性

当指定plummer-opt算法时，未设置UCG_USE_SHM_POOL为y，将自动回退至支持的算法。

4.2.4 KuComm 使用说明

Hyper MPI在HMPI层集合通信coll框架下新增KuComm分层拓扑组件，提供分层次拓扑框架能力。该框架的核心是根据集群的物理硬件拓扑（节点 - Socket - Core）进行分层，旨在为不同层级（如节点间、节点内、Socket内）的通信域，选择并执行最适合、性能最高的集合通信算法。

当前版本提供Bcast/Alltoall的阻塞集合操作接口，其他集合操作及点对点接口复用Hyper MPI原有的功能。该组件为后续不同层级通信域使用各自最适合的集合通信组件，进行极致性能优化提供了良好基础。

组件使能方式

通过在mpirun命令中添加 --mca coll_kucomm_priority 100 将KuComm组件的优先级设为100（高于其他所有集合通信组件的默认优先级），即可使能KuComm组件。

命令示例

以4节点，每个节点4进程为例

- 使能KuComm前：

  mpirun -np 16 -N 4 --hostfile hf4 -x UCX_TLS=sm,rc osu_bcast
- 使能KuComm后：

  mpirun -np 16 -N 4 --hostfile hf4 -x UCX_TLS=sm,rc **--mca coll_kucomm_priority 100** osu_bcast

特性支持与回退条件

- 当前KuComm组件仅支持Bcast和Alltoall接口，其他接口不会进入KuComm组件。
- 当前KuComm组件仅支持单线程场景调用Bcast和Alltoall接口。
- 当前KuComm组件仅支持多节点进程分布均匀的场景调用Bcast和Alltoall接口，否则会自动回退到Hyper MPI原有的Bcast和Alltoall接口。
- 当前KuComm组件暂不支持单节点Alltoall接口的调用，会自动回退到Hyper MPI原有的Alltoall接口。

4.2.5 命令说明及示例

4.2.5.1 传输模式选择说明

执行以下命令，可查看网络设备及支持的传输模式。

ucx_info -d

 版权所有 © 华为技术有限公司

回显示例：

```
#   Transport: ud_verbs
#      Device: mlx5_0:1
```

其中，Device表示当前网络设备，Transport表示当前网络设备支持的传输模式，可存在多个支持的传输模式。

启动MPI作业时，可通过“-x UCX_TLS”参数设置MPI使用的传输模式。目前常用的参数有ud、rc、shm、mm、rc_x、ud_x、dc_x等。其中shm和mm二选一，ud、rc、rc_x、ud_x、dc_x可以选一个或多个。不设置时默认等同于-x UCX_TLS=all，表示使用当前网络设备支持的所有传输模式。

更多的传输模式以及含义可参见：**https://github.com/openucx/ucx/wiki/UCX-environment-parameters**。

由于默认的参数在不指定场景时会出现不是最优的情况，为达到性能最优，可尝试如下参数组合：

-x UCX_TLS=shm,rc_x,ud_x

-x UCX_TLS=shm,ud_x

-x UCX_TLS=shm,dc_x

建议：Mellanox网卡环境使用ud_x与shm组合传输模式，鲲鹏RoCE网卡环境使用ud和shm组合的传输模式，原因是ud模式下创建QP的数量远低于rc模式且通信速率较快，鲲鹏RoCE网卡环境不支持ud_x模式。

4.2.5.2 进程分布

建议每个MPI任务在不同节点上分布的进程数一致，并增加--bind-to core --map-by socket --rank-by core，使进程在每个节点的Socket上连续均匀的分布，避免算法切换导致性能不达预期。

4.2.5.3 关闭 OOB 功能

建议测试Benchmark（例如OSU、GPCNET等）时，在命令行增加-x UCG_PLANC_UCX_USE_OOB=no，关闭复用OOB（out-of-band）资源功能，以提升Hyper MPI任务的性能。

示例：

mpirun -np *16* **-x UCG_PLANC_UCX_USE_OOB=no osu_barrier**

4.2.5.4 多网络平面场景使用说明

如果mpirun命令的运行环境中存在多个不同的网络平面，即UCX模块使用了多个不同网段的网口，则需要在mpirun命令的启动参数中额外增加-x UCX_RC_VERBS_ROCE_LOCAL_SUBNET=y和-x UCX_UD_VERBS_ROCE_LOCAL_SUBNET=y参数，否则会因网络不通导致下发任务无法继续执行。

示例如下：

mpirun -np *16* **-N** *16* **-x PATH -x LD_LIBRARY_PATH -x UCX_RC_VERBS_ROCE_LOCAL_SUBNET=y -x UCX_UD_VERBS_ROCE_LOCAL_SUBNET=y -x UCX_NET_DEVICES=***mlx5_0:1***,***mlx5_1:1* **osu_init**

版权所有 © 华为技术有限公司

4.2.5.5 HUCX RC 多网口容错

Hyper MPI中HUCX实现了RC（Reliably Connected）多网口容错（rc_failover）功能。该功能使用多个网口，通过自动检测网口异常状态并将业务流量切换至可用网口上实现容错，减少业务因网络组件故障而异常终止的概率。

当前该功能只支持RC连接方式。通过在mpirun命令行中添加如表4-4所示参数来配置该功能。

表 4-4 配置参数

参数	是否必选	说明
-x UCX_TLS=rc_f	是	指定传输模式为“rc_f”。
-x UCX_MAX_EAGER_RAILS=*2*	是	使用EAGER多轨。EAGER通道通常用于处理小包场景，为双边通信。 ● 取值范围为1~4且为整数，默认为1。 ● 若要支持该功能，需配置成大于1。 ● 取值可根据节点网卡故障率决定，取值越高，可靠性越高。推荐值为2。
-x UCX_LOCAL_CONN_SAME_DEV=*y*	是	本节点内是否使用同名网卡建链，可选值为y或n，默认为n。 ● y：本节点内使用同名网卡建链，可缩小故障域。 ● n：本节点内不同进程使用Numa亲和的网卡建链。
-x UCX_MAX_RNDV_RAILS=*2*	否	使用RNDV多轨。RNDV通道通常用于处理大包场景，为单边通信。 ● 取值范围为2~4且为整数，默认为2。 ● 取值可根据节点网卡故障率决定，取值越高，可靠性越高。推荐值为2。
-x UCX_MAX_AUX_RAILS=*2*	否	使用AUX多轨建链。HUCX在使用RC连接方式初始化时，会使用AUX链互相交换地址并进行RC的建链，AUX链通常使用UD（Unreliable Datagram）连接；大规模业务初始化建链时间较长，建议使用AUX多轨建链，并增加AUX多轨数量，以降低在初始化建链过程中网口故障对业务的影响。 ● 取值范围为1~4且为整数，默认为1。 ● 取值可根据节点网卡故障率决定，取值越高，可靠性越高。推荐值为2。

版权所有 © 华为技术有限公司

参数	是否必选	说明
-x UCX_AUX_TIMEOUT=3s	否	设置AUX多轨建链超时时间。默认为20s。格式为： -x UCX_AUX_TIMEOUT=<*number*>[s|us|ms|ns] 其中<number>为大于0的整数。 当集群规模与业务规模较大时，建链时间较长属于正常现象，可根据集群规模与业务规模适当调整该值，以便及时触发容错处理。
-x UCX_NET_DEVICES=*mlx5_0:1,mlx5_1:1*	否	设置网口的选择范围。默认从全部网口中选择。

说明

- 在同一业务进程中，HUCX多网口容错只会在选择的网口范围内进行，不会在未指定的网口中进行切换；当选择的网口数高于设置的多轨数量时，会按照多轨数量进行选择网口（部分网口可能未被选中）。
- 在同一业务进程中，某网口出现故障后，业务流量会被自动切换到其他可用网口；故障网口恢复后，当前业务不会再次选择曾故障网口；新业务进程不受此影响。
- 当业务进行中某网口出现故障，无其他可用网口可供切换时，业务仍然报错退出。

命令示例如下：

mpirun -np *64* **-N** *32* **--hostfile hf -x UCX_TLS=rc_f -x UCX_MAX_EAGER_RAILS=***2* **-x UCX_LOCAL_CONN_SAME_DEV=***y* **-x UCX_MAX_AUX_RAILS=***2* **-x UCX_AUX_TIMEOUT=3s osu_bcast**

4.2.5.6 启动时延优化

通过在mpirun命令行中加入如下参数来优化启动选项：

- **--mca osc ^ucx**：使用osc（单边通信模块）时不指定ucx。由于ucx的初始化在业务启动流程中耗时较长，大部分集合通信业务中使用pml，很少会用到osc，因此该配置项可以节省一次ucx初始化的时间。
- **--mca pml ucx**：指定pml模块只初始化ucx，减少其他模块初始化对启动时延的影响。
- **--mca oob_tcp_if_include** *网卡名称*：为加快业务启动osu_init时的fence流程，建议使用TCP带宽较大、时延更优的网卡，因为fence需要节点间发送TCP数据，带外TCP通信网卡往往会成为业务初始化的性能瓶颈。默认使用环境上获取到的第一张网卡，如有性能更好的其他网卡，可以通过该启动选项，提高通信时延性能。
- **-x UCX_TLS=sm,rc_v**：传输层选择sm + rc_v的组合方式。
  - sm表示节点内使用sm共享内存作为传输层，这加快了节点内进程的通信传输。
  - 由于rc要初始化rc_v、ud_v、rc_mlx和ud_mlx，而rc_v只需初始化rc_v和ud_v，因此节点间选择rc_v，可以加快业务启动时延。

版权所有 © 华为技术有限公司

- **-x UCX_NET_DEVICES**=*网卡名称*：选择的网卡数量会影响ud_v的初始化时延。对于每个网卡设备的初始化，需获取该设备的全部gid_attr，该过程较为耗时，因此建议减少网卡数量的选择。

启动时延优化命令示例：

mpirun --mca osc ^ucx --mca pml ucx -np *32* **-x UCX_TLS=sm,rc_v --hostfile file -x UCX_NET_DEVICES=***mlx5_0:1* **--mca oob_tcp_if_include** *enp125s0f0* **-x PATH -x LD_LIBRARY_PATH osu_init**

4.2.5.7 命令示例

Hyper MPI使用MPI_Allreduce算法6、MPI_Barrier算法5和MPI_Bcast算法3能获得较优的性能指标。

- IB网络环境命令示例

  mpirun -np *384* **-N** *48* **--hostfile hf --bind-to core --map-by socket --rank-by core --mca btl ^vader,tcp,openib -x UCX_TLS=sm,ud_x -x UCX_NET_DEVICES=***mlx5_0:1* **-x UCG_PLANC_UCX_ALLREDUCE_ATTR=I:***6***S:***200***R:***0-* **-x UCG_PLANC_UCX_BARRIER_ATTR=I:***5***S:***200***R:***0* **-x UCG_PLANC_UCX_BCAST_ATTR=I:***3***S:***200***R:***0-* **-x UCG_PLANC_UCX_ALLREDUCE_FANOUT_INTRA_DEGREE=***3* **-x UCG_PLANC_UCX_ALLREDUCE_FANIN_INTRA_DEGREE=***2* **-x UCG_PLANC_UCX_ALLREDUCE_FANOUT_INTER_DEGREE=***7* **-x UCG_PLANC_UCX_ALLREDUCE_FANIN_INTER_DEGREE=***7* **-x UCG_PLANC_UCX_BARRIER_FANOUT_INTRA_DEGREE=***3* **-x UCG_PLANC_UCX_BARRIER_FANIN_INTRA_DEGREE=***2* **-x UCG_PLANC_UCX_BARRIER_FANOUT_INTER_DEGREE=***7* **-x UCG_PLANC_UCX_BARRIER_FANIN_INTER_DEGREE=***7 test_case*

- RoCE网络环境命令示例

  mpirun -np *384* **-N** *48* **--hostfile hf --bind-to core --map-by socket --rank-by core --mca btl ^vader,tcp,openib -x UCX_TLS=sm,ud -x UCX_NET_DEVICES=***mlx5_1:1* **-x UCG_PLANC_UCX_ALLREDUCE_ATTR=I:***6***S:***200***R:***0-* **-x UCG_PLANC_UCX_BARRIER_ATTR=I:***5***S:***200***R:***0* **-x UCG_PLANC_UCX_BCAST_ATTR=I:***3***S:***200***R:***0-* **-x UCG_PLANC_UCX_ALLREDUCE_FANOUT_INTRA_DEGREE=***3* **-x UCG_PLANC_UCX_ALLREDUCE_FANIN_INTRA_DEGREE=***2* **-x UCG_PLANC_UCX_ALLREDUCE_FANOUT_INTER_DEGREE=***7* **-x UCG_PLANC_UCX_ALLREDUCE_FANIN_INTER_DEGREE=***7* **-x UCG_PLANC_UCX_BARRIER_FANOUT_INTRA_DEGREE=***3* **-x UCG_PLANC_UCX_BARRIER_FANIN_INTRA_DEGREE=***2* **-x UCG_PLANC_UCX_BARRIER_FANOUT_INTER_DEGREE=***7* **-x UCG_PLANC_UCX_BARRIER_FANIN_INTER_DEGREE=***7 test_case*

- RoCE网络环境Stars集合通信卸载命令示例

  mpirun -np *384* **-N** *48* **--hostfile hf --bind-to core --map-by socket --rank-by core --mca btl ^vader,tcp,openib -x UCX_TLS=sm,ud -x UCG_PLANC_STARS_TLS=rc_acc -x UCG_PLANC_STARS_RC_ROCE_LOCAL_SUBNET=y -x UCG_PLANC=ucx,stars -x UCX_UD_VERBS_ROCE_LOCAL_SUBNET=y -x UCG_PLANC_STARS_IBCAST_ATTR=I:***1***S:***200***R:***0* **-x UCG_PLANC_STARS_IALLGATHERV_ATTR=I:***1***S:***200***R:***0* **-x**

 版权所有 © 华为技术有限公司

UCG_PLANC_STARS_ISCATTERV_ATTR=I:*1*S:*200*R:*0* -x
UCG_PLANC_STARS_IALLTOALLV_ATTR=I:*1*S:*200*R:*0* -x
UCG_PLANC_STARS_IBARRIER_ATTR=I:*1*S:*200*R:*0* *test_case*

说明

IB和RoCE两种网络环境均使用鲲鹏服务器架构和Mellanox类型的网卡。

4.2.5.8 SDMA 使用场景说明

System Direct memory access（SDMA）是系统中负责数据的高带宽低延迟搬运的模块，是一种分布式分离式的DMA引擎。当前HMPI默认未开启SDMA，若需使用SDMA，需要在mpirun命令的启动参数中显式增加“-x UCX_TLS=sdma”参数。SDMA需要与posix或sysv同时使用，即“-x UCX_TLS=posix,sdma”或“-x UCX_TLS=sysv,sdma”。推荐鲲鹏920 72F系列使用Hyper MPI加SDMA的场景为PPN不大于128。

示例如下：

mpirun --allow-run-as-root -np *2* -x UCX_TLS=posix,sdma osu_bw

4.2.5.9 Stars 使用场景说明

MPI应用使用非阻塞集合通信接口，将通信任务卸载到Stars硬件执行，释放出的CPU资源用于执行其它计算任务，适用于大包通信的场景。推荐鲲鹏920 72F系列使用Hyper MPI加Stars的场景为单通信域进程数不大于1024。编程范式可参考如下示例：

```
MPI_Ibcast(); // 异步集合通信接口,通信任务卸载到Stars硬件执行
Compute();  // MPI应用插入其他计算任务，通信后台执行
MPI_Wait();  // 等待通信完成
```

使用Stars时，需要在mpirun命令的启动参数中指定Stars集合通信算法，比如-x UCG_PLANC_STARS_IBCAST_ATTR=I:{算法编号}S，详细示例说明请参见**RoCE网络环境Stars集合通信卸载命令示例**。

4.2.5.10 MPI 超时日志分析脚本使用说明

HMPI提供了各类卡住问题的超时日志打印来辅助增强定位的能力，通过日志可以区分出发送端和接收端，能够指导进行溯源到根因进程（节点）。超时日志需要通过mpi_logparser.py来解析。

表 4-5 mpi_logparser.py 脚本入参说明

参数	是否必选	说明
-f	-f或-d必须选一个	指定MPI生成的超时日志文件
-d	-f或-d必须选一个	指定MPI生成的超时日志目录
-o	否	分析脚本重定向日志输出
-u	否	最多打印可疑节点的数量
-l	否	分析脚本打印日志级别

版权所有 © 华为技术有限公司

参数	是否必选	说明
-s	否	跳过gid解析

使用示例参考:

```
mpi_logparser.py -f output.log
```

4.3 FAQ

4.3.1 故障处理流程

Hyper MPI提交任务出现异常或者报错时，可通过“回退法”来定位故障原因，并通过以下步骤进行故障处理。

操作步骤

步骤1 切换成默认算法（不指定算法编号）。

- 如无故障，说明指定算法不支持当前场景，可联系华为技术支持获取帮助。
- 如仍有故障，执行**步骤2**。

步骤2 屏蔽coll-ucg模块。

在**mpirun**命令中增加--mca coll ^ucg参数进行提交。如无故障，说明Open MPI支持此场景但Hyper MPI不支持。

如仍有故障，可能性有两个:

- Open MPI本身不支持该场景。
- MPI使用方法不当。可从环境变量设置、MPI安装等多方面进行排查，具体故障可参考故障处理其他章节。

----结束

4.3.2 作业进程数过大，报错：There are not enough slots available...

现象描述

提交MPI作业的进程数大于集群作业执行节点CPU总核数，导致**mpirun**命令运行失败。

运行失败示例如下:

mpirun -np 1025 --hostfile hf8 hmpifile_2021/allreduce/AllReduce

```
--------------------------------------------------------------------------
There are not enough slots available in the system to satisfy the 1025
slots that were requested by the application:

  hmpifile_2021/allreduce/AllReduce
```

 版权所有 © 华为技术有限公司

```
Either request fewer slots for your application, or make more slots
available for use.

A "slot" is the Open MPI term for an allocatable unit where we can
launch a process.  The number of slots available are defined by the
environment in which Open MPI processes are run:

  1. Hostfile, via "slots=N" clauses (N defaults to number of
     processor cores if not provided)
  2. The --host command line parameter, via a ":N" suffix on the
     hostname (N defaults to 1 if not provided)
  3. Resource manager (e.g., SLURM, PBS/Torque, LSF, etc.)
  4. If none of a hostfile, the --host command line parameter, or an
     RM is present, Open MPI defaults to the number of processor cores

In all the above cases, if you want Open MPI to default to the number
of hardware threads instead of the number of processor cores, use the
--use-hwthread-cpus option.

Alternatively, you can use the --oversubscribe option to ignore the
number of available slots when deciding the number of processes to
launch.
--------------------------------------------------------------------------
```

说明

1025：表示提交MPI作业的进程数。

可能原因

运行**mpirun**命令时的进程数大于集群作业执行节点CPU总核数。

恢复步骤

步骤1 使用PuTTY工具，以Hyper MPI普通用户（例如“hmpi_user”）登录至作业执行节点。

步骤2 执行以下命令，查询每个作业执行节点的CPU核数。

lscpu

```
Architecture:          aarch64
Byte Order:            Little Endian
CPU(s):                128
On-line CPU(s) list:   0-127
Thread(s) per core:    1
Core(s) per socket:    64
Socket(s):             2
NUMA node(s):          4
Model:                 0
CPU max MHz:           2600.0000
CPU min MHz:           200.0000
BogoMIPS:              200.00
L1d cache:             64K
L1i cache:             64K
L2 cache:              512K
L3 cache:              65536K
NUMA node0 CPU(s):     0-31
NUMA node1 CPU(s):     32-63
NUMA node2 CPU(s):     64-95
NUMA node3 CPU(s):     96-127
Flags:                 fp asimd evtstrm aes pmull sha1 sha2 crc32 atomics fphp asimdhp cpuid asimdrdm jscvt
fcma dcpop
```

步骤3 计算集群作业执行节点CPU总核数，确保在运行**mpirun**命令时的进程数小于等于集群作业执行节点CPU总核数。以八个节点总核数1024为例，执行以下命令可成功提交MPI作业。

 版权所有 © 华为技术有限公司

mpirun -np *1024* **--hostfile hf8 hmpifile_2021/allreduce/AllReduce**

```
All tests are success
```

----结束

4.3.3 多节点运行 mpirun 时异常

现象描述

- 多节点运行**mpirun**命令时，无任何反应。运行**top**命令时，无**mpirun**进程。
- 多节点运行**mpirun**命令时，出现如下报错信息：

```
[1632387881.405868] [arm-node88:57923:0]       mm_posix.c:194  UCX  ERROR shm_open(file_name=/ucx_shm_posix_23f3f65f flags=0xc2) failed: Permission denied
[1632387881.405910] [arm-node88:57923:0]        uct_mem.c:132  UCX  ERROR failed to allocate 8447 bytes using md posix for mm_recv_fifo: Shared memory error
[1632387881.405917] [arm-node88:57923:0]       mm_iface.c:605  UCX  ERROR mm_iface failed to allocate receive FIFO
[arm-node88:57923] coll_ucx_component.c:360  Warning: Failed to create UCG worker, automatically select other available and highest priority collective component.
[1632387881.411347] [arm-node88:57923:0]       mm_posix.c:194  UCX  ERROR shm_open(file_name=/ucx_shm_posix_6ae5143e flags=0xc2) failed: Permission denied
[1632387881.411359] [arm-node88:57923:0]        uct_mem.c:132  UCX  ERROR failed to allocate 8447 bytes using md posix for mm_recv_fifo: Shared memory error
[1632387881.411366] [arm-node88:57923:0]       mm_iface.c:605  UCX ERROR mm_iface failed to allocate receive FIFO
[arm-node88:57923] pml_ucx.c:274  Error: Failed to create UCP worker
[arm-node88:57923] *** An error occurred in MPI_Allreduce
[arm-node88:57923] *** reported by process [878510081,70368744177671]
[arm-node88:57923] *** on communicator MPI_COMM_WORLD
[arm-node88:57923] *** MPI_ERR_INTERN: internal error
[arm-node88:57923] *** MPI_ERRORS_ARE_FATAL (processes in this communicator will now abort,
[arm-node88:57923] ***    and potentially your MPI job)
```

可能原因

多节点运行**mpirun**命令时，存在节点与节点间不能互相通信。

恢复步骤

步骤1 使用PuTTY工具，以Hyper MPI普通用户（例如“hmpi_user”）登录至作业执行节点。

步骤2 检查所有节点是否都安装Hyper MPI且各节点安装路径是否一致，或者检查Hyper MPI是否安装在已挂载的共享目录上。

步骤3 检查环境变量是否配置正确。详情请参见《Kunpeng HPCKit 26.1.RC1 安装指南》中“安装卸载HPCKit > 设置环境变量”。

----结束

4.3.4 多节点运行 mpirun 时报某个文件无法找到或打开

现象描述

- 多节点运行**mpirun**命令时，报某个.so文件无法找到或打开。

  mpirun --allow-run-as-root -np 4 -N 2 --hostfile /hf2 /AllReduce

```
mpirun: error while loading shared libraries: libopen-rte.so.40: cannot open shared object file: No such file or directory
```


版权所有 © 华为技术有限公司

- 多节点运行**mpirun**命令时，报某个文件无法找到或打开。

  mpirun --allow-run-as-root -np 4 -N 2 --hostfile /hf2 /AllReduce

  ```
  bash: /Hyper-MPI_x.x.x_aarch64_CentOS7.6_GCCx.x.x_MLNX-OFED5.0/ompi/bin/orted: No such file or directory
  ```

可能原因

- 报某个.so文件无法找到或打开。

  “.bashrc”文件未配置“LD_LIBRARY_PATH”环境变量。
- 报某个文件无法找到或打开。

  两个节点Hyper MPI安装路径不同。

恢复步骤

- 报某个.so文件无法找到或打开。
  a. 使用PuTTY工具，以Hyper MPI普通用户（例如“hmpi_user”）登录至作业执行节点。
  b. 检查环境变量是否配置正确，详情请参见**《Kunpeng HPCKit 26.1.RC1 安装指南》**中“安装卸载HPCKit > 设置环境变量”。
- 报某个文件无法找到或打开。
  a. 使用PuTTY工具，以Hyper MPI普通用户（例如“hmpi_user”）登录至作业执行节点。
  b. 检查所有节点是否都安装Hyper MPI且各节点安装路径是否一致，或者检查Hyper MPI是否安装在已挂载的共享目录上。

4.3.5 运行 MPI 作业提示网络设备内存无法分配

现象描述

运行MPI作业时，提示网络设备内存无法分配：

```
ERROR ibv_open_device(mlx5_0) failed: Cannot allocate memory
```

可能原因

MPI作业执行节点内存不足，导致提示网络设备内存无法分配。

恢复步骤

步骤1 使用PuTTY工具，以Hyper MPI普通用户（例如“hmpi_user”）登录至作业执行节点。

步骤2 执行以下命令，查看作业执行节点上正在运行的进程。

top

步骤3 关闭与MPI作业执行无关的进程。

----结束

 版权所有 © 华为技术有限公司

4.3.6 指定网卡名称错误，报错：network device '***' is not available

现象描述

提交MPI作业时指定的网卡名称错误导致**mpirun**命令运行失败。

运行失败示例如下：

mpirun -np 8 -N 1 --hostfile ~/hmpifile_2021/hostfile/hf8 -x UCX_NET_DEVICES=mlx5_0:2 ~/hmpifile_2021/allreduce/AllReduce

```
[1632383945.549496] [arm-node132:2635376:0]    ucp_context.c:732  UCX  WARN  network device 'mlx5_0:2' is not available, please use one or more of: 'enp189s0f0'(tcp), 'enp1s0'(tcp), 'mlx5_0:1'(ib)
```

可能原因

运行**mpirun**命令时指定的网卡资源名称有误。

恢复步骤

步骤1 使用PuTTY工具，以Hyper MPI普通用户（例如“hmpi_user”）登录至作业执行节点。

步骤2 执行以下命令，查询当前作业执行节点所有可用的网卡名称。

ibdev2netdev

```
mlx5_0 port 1 ==> enp1s0 (Up)
```

步骤3 执行以下命令，将网卡名称修改为当前可用网卡名称，如mlx5_0:1。

mpirun -np *16* **-N** *2* **--hostfile** *~/hmpifile_2021/hostfile/hf8* **-x UCX_NET_DEVICES=***mlx5_0:1 ~/hmpifile_2021/allreduce/AllReduce*

说明

- ~/hmpifile_2021/hostfile：表示指定作业运行节点的文件路径。
- hf8：表示指定作业运行节点的文件。
- ~/hmpifile_2021/allreduce：表示指定运行作业的路径。
- AllReduce：表示指定的运行作业。
- mlx5_0:1：表示**步骤2**查询到的网卡名称。

----结束

4.3.7 指定网络类型错误，提示指定的网络类型不可用

现象描述

- 提交MPI作业时指定的网络类型错误导致**mpirun**命令运行失败。

  运行失败示例如下：

  mpirun -np 16 -N 2 --hostfile ~/hmpifile_2021/hostfile/hf8 -x UCX_NET_DEVICES=mlx5_0:1 -x UCX_TLS=tcp ~/hmpifile_2021/allreduce/AllReduce

版权所有 © 华为技术有限公司

```
[1632384264.377328] [arm-node132:2637768:0]    ucp_context.c:1073 UCX  ERROR no usable transports/devices (asked tcp on network:mlx5_0:1 )
[arm-node132:2637768] *** An error occurred in MPI_Allreduce
[arm-node132:2637768] *** reported by process [2539913217,0]
[arm-node132:2637768] *** on communicator MPI_COMM_WORLD
[arm-node132:2637768] *** MPI_ERR_INTERN: internal error
[arm-node132:2637768] *** MPI_ERRORS_ARE_FATAL (processes in this communicator will now abort,
[arm-node132:2637768] ***    and potentially your MPI job)
[arm-node132:2637714] 7 more processes have sent help message help-mpi-errors.txt / mpi_errors_are_fatal
[arm-node132:2637714] Set MCA parameter "orte_base_help_aggregate" to 0 to see all help / error messages
```

- 提交MPI作业时传输模式不支持，报如下错误：

```
UCX  WARN  transport 'rc_x' is not available, please use one or more of: cma, ib, mm, posix, rc, rc_v, rc_verbs, self, shm, sm, sysv, tcp, ud, ud_v, ud_verbs
```

可能原因

运行**mpirun**命令时指定了无效的-x UCX_TLS参数，不同网络设备所支持的传输模式不同。

恢复步骤

步骤1 使用PuTTY工具，以Hyper MPI普通用户（例如“hmpi_user”）登录至作业执行节点。

步骤2 执行以下命令，查询当前作业执行节点所有可用的网卡类型以及相应的网络协议。

ucx_info -d

```
Transport: rc_mlx5
Device: mlx5_0:1
```

步骤3 执行以下命令，将网络协议修改为当前可用网络协议。

mpirun -np *16* **-N** *2* **--hostfile** *~/hmpifile_2021/hostfile/hf8* **-x UCX_NET_DEVICES=***mlx5_0:1* **-x UCX_TLS=***rc_mlx5 ~/hmpifile_2021/allreduce/AllReduce*

说明

- ~/hmpifile_2021/hostfile：表示指定作业运行节点的文件路径。
- hf8：表示指定作业运行节点的文件。
- ~/hmpifile_2021/allreduce：表示指定运行作业的路径。
- AllReduce：表示指定的运行作业。
- mlx5_0:1：表示**步骤2**查询到的网卡类型。
- rc_mlx5：表示**步骤2**查询到的网络协议。

----结束

4.3.8 指定主机名错误，报错：Name or service not known

现象描述

提交MPI作业时指定hostfile中的主机名错误导致**mpirun**命令运行失败。

运行失败示例如下：

版权所有 © 华为技术有限公司

mpirun -np 16 -N 2 --hostfile ~/hmpifile_2021/hostfile/hf8 -x UCX_NET_DEVICES=mlx5_0:1 /home/hmpi_user/hmpifile_2021/allreduce/AllReduce

```
ssh: Could not resolve hostname arm-node056: Name or service not known
--------------------------------------------------------------------------
ORTE was unable to reliably start one or more daemons.
This usually is caused by:

* not finding the required libraries and/or binaries on
  one or more nodes. Please check your PATH and LD_LIBRARY_PATH
  settings, or configure OMPI with --enable-orterun-prefix-by-default

* lack of authority to execute on one or more specified nodes.
  Please verify your allocation and authorities.

* the inability to write startup files into /tmp (--tmpdir/orte_tmpdir_base).
  Please check with your sys admin to determine the correct location to use.

*  compilation of the orted with dynamic libraries when static are required
  (e.g., on Cray). Please check your configure cmd line and consider using
  one of the contrib/platform definitions for your system type.

* an inability to create a connection back to mpirun due to a
  lack of common network interfaces and/or no route found between
  them. Please check network connectivity (including firewalls
  and network routing requirements).
--------------------------------------------------------------------------
--------------------------------------------------------------------------
ORTE does not know how to route a message to the specified daemon
located on the indicated node:

  my node:   arm-node056
  target node:  arm-node011

This is usually an internal programming error that should be
reported to the developers. In the meantime, a workaround may
be to set the MCA param routed=direct on the command line or
in your environment. We apologize for the problem.
--------------------------------------------------------------------------
[arm-node132:2640389] 5 more processes have sent help message help-errmgr-base.txt / no-path
[arm-node132:2640389] Set MCA parameter "orte_base_help_aggregate" to 0 to see all help / error
messages
```

可能原因

运行**mpirun**命令时hf8文件中指定的主机名在局域网中不存在。

恢复步骤

步骤1 使用PuTTY工具，以Hyper MPI普通用户（例如“hmpi_user”）登录至作业执行节点。

步骤2 执行以下命令，修改“hf8”文件。

1. 打开“hf8”文件。

   vi ~*/hmpifile_2021/hostfile/hf8*

   说明

   - ~/hmpifile_2021/hostfile：表示指定作业运行节点的文件路径。
   - hf8：表示指定作业运行节点的文件。

1. 按“i”进入编辑模式，根据错误提示将“arm-node056”改成“arm-node134”（局域网中存在此主机名）。

 版权所有 © 华为技术有限公司

```
arm-node134
```

2. 按“Esc”键，输入**:wq!**，按“Enter”保存并退出编辑。

步骤3 执行以下命令，验证“hf8”文件是否修改成功。

mpirun -np *16* **-N** *2* **--hostfile** *~/hmpifile_2021/hostfile/hf8* **-x UCX_NET_DEVICES=***mlx5_0:1 ~/hmpifile_2021/allreduce/AllReduce*

说明

- ~/hmpifile_2021/allreduce：表示指定运行作业的路径。
- AllReduce：表示指定的运行作业。

出现以下回显信息，表示“hf8”文件修改成功。

```
All tests are success
```

----结束

4.3.9 通信过程建链报错：ucp_ep_create(...)failed：Destination is unreachable

现象描述

MPI进程在通信建链中返回如下错误信息：

```
ucp_ep_create(peer proc=0 peer hostname=6426-node125) failed: Destination is unreachable
```

可能原因

ep在与对端建链时，由于网络问题导致建链请求无法到达对端。

恢复步骤

- 查看除ucp_ep_create报错之外的报错日志，根据错误日志中的信息在本章中检索有无对应的具体章节。
- 若本章中能够检索到报错信息，进入具体章节按照说明进行故障排查。
- 若本章中检索不到报错信息，ucp_ep_create报错中会打印对端节点名，在两端节点上检查网络配置、网卡配置是否有问题。

4.3.10 在-x UCX_IB_GID_INDEX 命令中指定 GID 后报错

现象描述

在-x UCX_IB_GID_INDEX命令中指定无效GID后，出现如下报错：

```
ib_device.c:848 UCX ERROR ibv_query_gid(dev=mlx5_0 port=1 index=10) failed: No such file or directory
```

在-x UCX_IB_GID_INDEX命令中指定GID后，出现如下报错：

```
pml_ucx.c:384  Error: ucp_ep_create(proc=1) failed: Destination is unreachable
```

可能原因

- 命令行指定的GID与服务器GID不一致。

 版权所有 © 华为技术有限公司

- 多台作业执行节点GID不一致。

恢复步骤

- 命令行指定的GID与服务器GID不一致。
  a. 使用PuTTY工具，以Hyper MPI普通用户（例如“hmpi_user”）登录至作业执行节点。
  b. 执行以下命令，查询GID的值。

     show_gids | grep -E "((25[0-5]|2[0-4][0-9]|1[0-9][0-9]|[1-9]?[0-9])\.){3}(25[0-5]|2[0-4][0-9]|1[0-9][0-9]|[1-9]?[0-9])" | grep v2 | awk '{print $3}'

     ```
     5
     ```

  说明

  5：表示查询到的GID值。

1. 执行以下命令，指定GID值为**b**中查询到的GID的值，例如5。

   -x UCX_IB_GID_INDEX=*5*

- 多台作业执行节点GID不一致。
  a. 使用PuTTY工具，以Hyper MPI普通用户（例如“hmpi_user”）登录至作业执行节点。
  b. 执行以下命令，修改“~/.bashrc”文件。

     打开“~/.bashrc”文件。

     vi ~/.bashrc

     按“i”进入编辑模式，添加如下内容。

     ```
     v2_gid=$(show_gids | grep -E "((25[0-5]|2[0-4][0-9]|1[0-9][0-9]|[1-9]?[0-9])\.){3}(25[0-5]|2[0-4][0-9]|1[0-9][0-9]|[1-9]?[0-9])" | grep v2 | awk '{print $3}')
     export UCX_IB_GID_INDEX=$v2_gid
     ```

     按“Esc”键，输入**:wq!**，按“Enter”保存并退出编辑。
  c. 执行以下命令，使配置生效。

     source ~/.bashrc

4.3.11 找不到 ARP 表项引起的超时报错：ibv_create_ah...failed: Connection timed out

现象描述

用户在提交大规模MPI作业时高概率报错建链超时，作业输出日志报错有“Connection timed out”：

```
[agent363:373418:0:373418]    ud iface.c:49   Fatal: iface 0x1ddfcb30: failed to get peer address
=== backtrace (tid: 373418) ====
0 /share/software/mpi/hmpi/1.2.1/bisheng2.5.0/hucx/lib/libucs.so.0(ucs_fatal_error_message+0x38) [0x40012c420128]
1 /share/software/mpi/hmpi/1.2.1/bisheng2.5.0/hucx/lib/libucs.so.0(+0x2025c) [0x40012c42025c]
2 /share/software/mpi/hmpi/1.2.1/bisheng2.5.0/hucx/lib/ucx/libuct_ib.so.0(uct_ud_iface_cep_insert_ep+0) [0x40012c5157e0]
3 /share/software/mpi/hmpi/1.2.1/bisheng2.5.0/hucx/lib/ucx/libuct_ib.so.0(uct_ud_ep_create_connected_common+0xd4) [0x40012c5184c4]
4 /share/software/mpi/hmpi/1.2.1/bisheng2.5.0/hucx/lib/libucp.so.0(ucp_wireup_ep_connect_aux+0xc0) [0x400127f51be0]
5 /share/software/mpi/hmpi/1.2.1/bisheng2.5.0/hucx/lib/libucp.so.0(ucp_wireup_ep_connect+0xe4)
```

 版权所有 © 华为技术有限公司

```
[0x400127f522e4]
6 /share/software/mpi/hmpi/1.2.1/bisheng2.5.0/hucx/lib/libucp.so.0(ucp_wireup_init_lanes+0x8d4)
[0x400127f53e94]
7 /share/software/mpi/hmpi/1.2.1/bisheng2.5.0/hucx/lib/libucp.so.0(ucp_ep_create_to_worker_addr+0x78)
[0x400127f1cf58]
8 /share/software/mpi/hmpi/1.2.1/bisheng2.5.0/hucx/lib/libucp.so.0(ucp_ep_create+0x4b0)
[0x400127f1dbbe]
9 /share/software/mpi/hmpi/1.2.1/bisheng2.5.0/hmpi/lib/openmpi/mca_pml_ucx.so(+0x5940)
[0x400127e55940]
10 /share/software/mpi/hmpi/1.2.1/bisheng2.5.0/hmpi/lib/openmpi/mca_pm1_ucx.so(mca_pml_ucx_send
+0x198) [0x400127e54304]
11 /share/software/mpi/hmpi/1.2.1/bisheng2.5.0/hmpi/lib/libmpi.so.40(ompi_coll_base_sendrecv_actual
+0xbc)[0x4001266ffafc]
12 /share/software/mpi/hmpi/1.2.1/bisheng2.5.0/hmpi/lib/libmpi.so.40(ompi_coll_base_sendrecv_intra_bruck
+0xac)[0x4001266fe710]
13 /share/software/mpi/hmpi/1.2.1/bisheng2.5.0/hmpi/lib/openmpi/mca_coll_ucx.so(+0x5b08)
[0x40012cb25b08]
14 /share/software/mpi/hmpi/1.2.1/bisheng2.5.0/hmpi/lib/libmpi.so.40(mca_coll_base_comm_select+0x880)
[0x4001266f3d64]
15 /share/software/mpi/hmpi/1.2.1/bisheng2.5.0/hmpi/lib/libmpi.so.40(ompi_mpi_init+0xe30)
[0x40012672a7a4]
16 /share/software/mpi/hmpi/1.2.1/bisheng2.5.0/hmpi/lib/libmpi.so.40(MPI_Init+0xa8) [0x4001266d8468]
17  hello_mpi() [0x4008c0]
18 /usr/lib64/libc.so.6(__libc_start_main+0xe0) [0x400126813f40]
19 hello_mpi() [0x4007dc]
================================
```

作业错误日志有“failed to get peer address”：

```
[1693797506.380997] [agent289:3924149:0]         ib_device.c:1252 UCX  ERROR ibv_create_ah(dlid=49152
sl=0 port=1 src_path_bits=0 dgid=::ffff:192.168.1.115 sgid index=5 traffic class=106) on mlx5 0 failed:
Connection timed out
[1693797506.384196] [agent276:1958414:0]         ib_device.c:1252 UCX  ERROR ibv_create_ah(dlid=49152
sl=0 port=1 src_path_bits=0 dgid=::ffff:192.168.1.62 sgid_index=5 traffic_class=106) on mlx5_0 failed:
Connection timed out
```

可能原因

- 报错信息为MPI的UCX模块ibv_create_ah部分报出，原因为未在超时时间内获取对端地址。
- 交换机的ARP防攻击特性，默认配置为限制同一个源IP在1秒内发送的ARP报文数量为30个。Hyper MPI大规模（100+节点）作业启动建链，1秒内会超过30个ARP报文（无ARP缓存情况下），导致ARP丢包，最终导致MPI建链超时。
- 通过Linux本地ARP缓存表，本地缓存对端地址后，可以从本地ARP表中获取对端地址，无需再次发送ARP请求。

恢复步骤

计算节点配置静态ARP表项。

4.3.12 由于网卡配置不当引起的超时报错：connect... failed：Connection timed out

现象描述

多节点运行MPI作业，作业无法正常运行，返回如下报错信息：

```
[1703209660.081479] [node167:2042308:0]          sock.c:272 UCX ERROR connect(fd=37,
dest_addr=66.66.66.168:49703) failed: Connection timed out
[node167:2042308] pml_ucx.c:426  Error: ucp_ep_create(proc=0) failed: Destination is unreachable
```


版权所有 © 华为技术有限公司

可能原因

- 防火墙没有关闭，导致建链失败。
- 如果使用RoCE网络，可能是RoCE网络没有配置，网口没有指定IP地址，通过**show_gids**命令检查发现RoCE网口没有配置IP地址。

```
DEV     PORT    INDEX    GID                                     IPv4          VeR    DEV
---     ----    -----    ---                                     ------------  ---    ---
mlx5_0  1       0        fe80:0000:0000:0000:0ac0:ebff:fe8c:23b8               v1     enplsOfOnp0
mlx5_0  1       1        fe80:0000:0000:0000:0ac0:ebff:fe8c:23b8               v2     enp1s0fOnp0
mlx5_1  1       0        fe80:0000:0000:0000:0ac0:ebff:fe8c:23b9               v1     enp1s0f1npl
mlx5_1    1       1        fe80:0000:0000:0000:0ac0:ebff:fe8c:23b9               v2     enp1s0f1npl
n gids found=4
```

恢复步骤

- 关闭防火墙。
- 配置RoCE网络。

```
DEV     PORT    INDEX    GID                                     IPv4          VeR    DEV
---     ----    -----    ---                                     ------------  ---    ---
mlx5_0  1       0        fe80:0000:0000:0000:0ac0:ebff:fe8c:23b8               v1     enplsOfOnp0
mlx5_0  1       1        fe80:0000:0000:0000:0ac0:ebff:fe8c:23b8               v2     enp1s0fOnp0
mlx5_1  1       0        fe80:0000:0000:0000:0ac0:ebff:fe8c:23b9 xx.xx.xx.xxx  v1     enp1s0f1npl
mlx5_1  1       1        fe80:0000:0000:0000:0ac0:ebff:fe8c:23b9 xx.xx.xx.xxx  v2     enp1s0f1npl
n gids found=4
```

4.3.13 由于丢包等网络问题造成的 UCP request timeout 超时报错

现象描述

MPI运行过程中卡死，卡死超过五分钟后打印超时警告，日志信息如下：

```
UCP request timeout! request tag *** local proc: *** peer proc: *** peer hostname: ***
```

可能原因

- 使用RoCE网络运行，但是没有配置网卡侧和交换机侧的无损网络，导致通信过程中丢包。
- 计算节点间的网络线路出现故障。

恢复步骤

步骤1 在日志信息中获取发生超时的请求中对端进程所在计算节点。

步骤2 如果没有配置网卡侧和交换机侧的无损网络，需要配置后再运行作业。

步骤3 如果无损网络没有问题，排查出错节点间的网络线路是否有问题。

步骤4 如果网络线路没问题，检查网卡侧或交换机侧的丢包记录，确认硬件上是否丢包。

步骤5 如果物理链路和硬件配置上未排查出问题，且使用RC协议，则可以使用UD协议暂时规避问题（RC依赖硬件的重传，而UD软件上会做重传）。

----结束

版权所有 © 华为技术有限公司

4.3.14 由于丢包等网络原因引起的 UD 超时报错：UD endpoint ..... unhandled timeout error

现象描述

运行MPI作业时，报错如下：

```
[arm-129:435170:0:435170]       ud_ep.c:262  Fatal: UD endpoint 0x6c29690 to <no debug data>: unhandled timeout error
==== backtrace (tid: 435170) ====
 0  /workspace/cw/ccsuite/hmpi/install/hucx/lib/libucs.so.0(ucs_handle_error+0x250) [0x4000237b3630]
 1  /workspace/cw/ccsuite/hmpi/install/hucx/lib/libucs.so.0(ucs_fatal_error_message+0xd0) [0x4000237b0940]
 2  /workspace/cw/ccsuite/hmpi/install/hucx/lib/libucs.so.0(+0x1fa08) [0x4000237b0a08]
 3  /workspace/cw/ccsuite/hmpi/install/hucx/lib/ucx/libuct_ib.so.0(+0x475a8) [0x4000238795a8]
 4  /workspace/cw/ccsuite/hmpi/install/hucx/lib/libucs.so.0(+0x18d10) [0x4000237a9d10]
 5  /workspace/cw/ccsuite/hmpi/install/hucx/lib/libucp.so.0(ucp_worker_progress+0x60) [0x4000236f47b0]
 6  /workspace/cw/ccsuite/hmpi/install/hmpi/lib/libopen-pal.so.40(opal_progress+0x38) [0x4000223cbef8]
 7  /workspace/cw/ccsuite/hmpi/install/hmpi/lib/libmpi.so.40(ompi_mpi_init+0xc78) [0x4000220be608]
 8  /workspace/cw/ccsuite/hmpi/install/hmpi/lib/libmpi.so.40(MPI_Init+0x64) [0x400022066404]
 9  /workspace/cw/cwScript/mpijob/bcast_sleep_accurate() [0x400a6c]
10  /usr/lib64/libc.so.6(+0x2afbc) [0x400022146fbc]
11  /usr/lib64/libc.so.6(__libc_start_main+0x94) [0x400022147094]
12  /workspace/cw/cwScript/mpijob/bcast_sleep_accurate() [0x400930]
=================================
[arm-129:435170] *** Process received signal ***
[arm-129:435170] Signal: Aborted (6)
[arm-129:435170] Signal code:  (-6)
[arm-129:435170] [ 0] linux-vdso.so.1(__kernel_rt_sigreturn+0x0)[0x400021fd393c]
[arm-129:435170] [ 1] /usr/lib64/libc.so.6(+0x80e78)[0x40002219ce78]
[arm-129:435170] [ 2] /usr/lib64/libc.so.6(raise+0x1c)[0x400022158cfc]
[arm-129:435170] [ 3] /usr/lib64/libc.so.6(abort+0xe0)[0x400022146d2c]
[arm-129:435170] [ 4] /workspace/cw/ccsuite/hmpi/install/hucx/lib/libucs.so.0(+0x1f944)[0x4000237b0944]
[arm-129:435170] [ 5] /workspace/cw/ccsuite/hmpi/install/hucx/lib/libucs.so.0(+0x1fa08)[0x4000237b0a08]
[arm-129:435170] [ 6] /workspace/cw/ccsuite/hmpi/install/hucx/lib/ucx/libuct_ib.so.0(+0x475a8) [0x4000238795a8]
[arm-129:435170] [ 7] /workspace/cw/ccsuite/hmpi/install/hucx/lib/libucs.so.0(+0x18d10) [0x4000237a9d10]
[arm-129:435170] [ 8] /workspace/cw/ccsuite/hmpi/install/hucx/lib/libucp.so.0(ucp_worker_progress+0x60) [0x4000236f47b0]
[arm-129:435170] [ 9] /workspace/cw/ccsuite/hmpi/install/hmpi/lib/libopen-pal.so.40(opal_progress+0x38) [0x4000223cbef8]
[arm-129:435170] [10] /workspace/cw/ccsuite/hmpi/install/hmpi/lib/libmpi.so.40(ompi_mpi_init+0xc78) [0x4000220be608]
[arm-129:435170] [11] /workspace/cw/ccsuite/hmpi/install/hmpi/lib/libmpi.so.40(MPI_Init+0x64) [0x400022066404]
[arm-129:435170] [12] /workspace/cw/cwScript/mpijob/bcast_sleep_accurate[0x400a6c]
[arm-129:435170] [13] /usr/lib64/libc.so.6(+0x2afbc)[0x400022146fbc]
[arm-129:435170] [14] /usr/lib64/libc.so.6(__libc_start_main+0x94)[0x400022147094]
[arm-129:435170] [15] /workspace/cw/cwScript/mpijob/bcast_sleep_accurate[0x400930]
[arm-129:435170] *** End of error message ***
--------------------------------------------------------------------------
mpirun noticed that process rank 5 with PID 435171 on node arm-129 exited on signal 6 (Aborted).
--------------------------------------------------------------------------
```

可能原因

- 没有使用补丁版本的HUCX源码包，通过调度器指定UCX_TLS=ud，将MPI作业挂起一段时间再恢复，导致超时报错。
- 使用RoCE网络运行，但是没有配置网卡侧和交换机侧的无损网络。
- 计算节点间的网络线路出现了故障。

版权所有 © 华为技术有限公司

恢复步骤

步骤1 在对应版本HUCX的最新补丁包中已经支持该需求，下载对应版本的补丁包，重新编译安装。

步骤2 如果没有配置网卡侧和交换机侧的无损网络，需要配置后再运行作业。

步骤3 如果无损网络没有问题，排查出错节点间的网络线路是否有问题。

步骤4 如果物理链路和硬件配置上未排查出问题，可以设置**-x UCX_UD_TIMER_BACKOFF=1 -x UCX_UD_TIMER_TICK=100ms -x UCX_UD_TIMEOUT=600s**增大超时时间，暂时规避问题。

----结束

4.3.15 运行报错 failed to bind memory，但应用仍可以正常结束

现象描述

在单节点或者多节点运行MPI作业，指定-bind-to core，报错“failed to bind memory”。

```
--------------------------------------------------------------------------
WARNING: Open MPI tried to bind a process but failed.  This is a
warning only; your job will continue, though performance may
be degraded.

  Local host:        node175
  Application name:  /root/tqa/osu-micro-benchmarks-7.1-1/c/mpi/collective/blocking/osu_bcast
  Error message:     failed to bind memory
  Location:          rtc_hwloc.c:447

--------------------------------------------------------------------------
[node175:06049] 63 more processes have sent help message help-orte-odls-default.txt / memory not bound
[node175:06049] Set MCA parameter "orte_base_help_aggregate" to 0 to see all help / error messages
```

可能原因

节点上没有插满内存条，导致部分NUMA node没有绑定内存，可以通过**numactl --hardware**命令检查。

```
available: 4 nodes (0-3)
node 0 cpus: 0 1 2 3 4 5 6 7 8 9 10 11 12 13 14 15 16 17 18 19 20 21 22 23 24 25 26 27 28 29 30 31
node 0 size: 31566 MB
node 0 free: 30548 MB
node 1 cpus: 32 33 34 35 36 37 38 39 40 41 42 43 44 45 46 47 48 49 50 51 52 53 54 55 56 57 58 59 60 61 62 63
node 1 size: 0 MB
node 1 free: 0 MB
node 2 cpus: 64 65 66 67 68 69 70 71 72 73 74 75 76 77 78 79 80 81 82 83 84 85 86 87 88 89 90 91 92 93 94 95
node 2 size: 31049 MB
node 2 free: 29424 MB
node 3 cpus: 96 97 98 99 100 101 102 103 104 105 106 107 108 109 110 111 112 113 114 115 116 117 118 119 120 121 122 123 124 125 126 127
node 3 size: 0 MB
node 3 free: 0 MB
node distances:
node   0   1   2   3
  0:  10  12  35  37
  1:  12  10  37  40
  2:  35  37  10  12
  3:  37  40  12  10
```


版权所有 © 华为技术有限公司

其中node1和node2均没有绑定内存。

恢复步骤

步骤1 进程规模较小的情况下，可以尝试指定rankfile，避免进程绑定在没有内存的NUMA上。

步骤2 插满内存条。

----结束

4.3.16 Allgatherv 算法 4 指定 TCP 传输，np 大规格执行报错

现象描述

集合操作Allgatherv算法4执行传输模式为TCP，多节点满核启动进程报错：

运行失败示例如下：

mpirun --allow-run-as-root --timeout 350 -np 1024 -N 128 --hostfile ~/hmpifile_2021/hostfile/hf8 -x UCX_TLS=tcp -x UCG_PLANC_UCX_ALLGATHERV_ATTR=I:4 ~/hmpifile_2021/allgatherv/allgatherv

```
Authorized users only. All activities may be monitored and reported.
[hmpi03:04566] pml_ucx.c:428  Error: ucp_ep_create(proc=514) failed: Destination is unreachable
[hmpi03:04563] pml_ucx.c:428  Error: ucp_ep_create(proc=515) failed: Destination is unreachable
[1684893155.703980] [hmpi06:4797 :1]         tcp_cm.c:749  UCX  WARN  tcp_iface 0x3c6e6b10: connection establishment for socket fd 755 from <invalid address family> to 192.168.0.1:51311 was unsuccessful
[1684893155.703997] [hmpi06:4797 :1]         tcp_cm.c:749  UCX  WARN  tcp_iface 0x3c6e6b10: connection establishment for socket fd 755 from <invalid address family> to 192.168.0.1:51311 was unsuccessful
[1684893256.847887] [hmpi05:11722:0]         tcp_cm.c:705  UCX  ERROR tcp_ep 0x86843a0: reached maximum number of connection retries (25)
```

可能原因

TCP建链产生的软中断重度负载，导致CPU无法处理，所以TCP建链超时。对于Allgatherv线性算法，为TCP本身的限制，属于正常现象。

恢复步骤

Allgatherv使用TCP传输模式时，优先使用其他算法。

4.3.17 Allreduce 算法 8 精度问题导致多次执行结果不一致

现象描述

应用指定集合操作Allreduce算法8进行多次重复计算，每次浮点计算结果不一致。

可能原因

Allreduce算法8由于自身算法特性，导致规约顺序无法固定，影响浮点计算的精度。

恢复步骤

对于精度要求较高的场景，通过指定Allreduce算法1可以规避，保障应用精度一致。

 版权所有 © 华为技术有限公司

4.3.18 超线程环境中 rank 进程未绑定至逻辑核上

现象描述

超线程环境中rank进程未绑定至逻辑核上。

可能原因

- 若不指定“--bind-to”的值，Hyper MPI默认其值为“core”。这将导致rank进程绑定在物理核上，而不是逻辑核上。

  说明

  “--bind-to”的取值中“core”和“hwthread”定义如下：

  - core：将rank进程绑定至物理核。
  - hwthread：将rank进程绑定至逻辑核。
- 将“--bind-to”的值指定为“core”，导致rank进程绑定在物理核上，而不是逻辑核上。

恢复步骤

将rank进程绑定至逻辑核上，需要将“--bind-to”的值指定为“hwthread”，示例如下：

mpirun -np *4* **--bind-to hwthread** *osu_bcast*

说明

若需要将rank进程按照逻辑核进行分布，则需要将“--bind-to”和“--map-by”的值均指定为“hwthread”，示例如下：

mpirun -np *4* **--map-by hwthread --bind-to hwthread** *osu_bcast*

4.3.19 UCX 版本不匹配告警

现象描述

运行时报错：

```
UCX WARN UCP version is incompatible, required:x.xx, actual:x.xx
```

可能原因

在某个版本的UCX上构建的XUCG和Hyper MPI，但是运行时却警告使用的是另一个版本的UCX，一般是由于系统目录已经存在一个UCX，并且构建XUCG前没有设置正确的UCX环境变量，导致XUCG依赖了错误的UCX版本。

恢复步骤

严格按照用户指南构建Hyper MPI，构建XUCG前正确设置UCX的环境变量。

版权所有 © 华为技术有限公司

4.3.20 UCP 请求超时

现象描述

测试OSU集合通信osu_iallgather和osu_iallgatherv时，当size大于32768，UCP request timeout。

可能原因

尝试使用开源Open MPI 4.1.1+Open UCX 1.15.0的环境运行指令，也会出现相同问题，说明问题发生在开源组件上。可能是由于bcopy进行分包，同时默认的接收队列长度不足，导致数据包未被接收。

恢复步骤

在运行命令中添加参数“-x UCX_RNDV_THRESH=8192”。

4.3.21 运行 osu_alltoall/osu_alltoallv/osu_alltoallw/ osu_ialltoall/osu_ialltoallv/osu_ialltoallw 时出现内存耗尽问题

现象描述

在满核（核数576、内存约500GB）下运行osu_alltoall/osu_alltoallv/osu_alltoallw/ osu_ialltoall/osu_ialltoallv/osu_ialltoallw，当使用默认的max_message_size（1MB）时，会出现内存耗尽问题。

可能原因

通过实测及理论分析，运行osu_alltoall/osu_alltoallv/osu_alltoallw/osu_ialltoall/ osu_ialltoallv/osu_ialltoallw时，会申请较多的内存，内存占用主要包含以下2个方面：

1. 每个进程需要申请sendbuf和recvbuf两块内存，每个大小为：

   bufsize = options.max_message_size * numprocs（需要按PAGESIZE对齐）

   如果指定-c参数，则需要额外申请expected_buffer和temp_buffer两块内存，大小和sendbuf、recvbuf的大小相等。
2. UM通信所需要的内存size_um。

每个节点需要的内存大小：

- 不指定-c参数：

  $$alloc_mem = 2 \times max_message_size \times ppn \times numprocs + size_um$$
- 指定-c参数：

  $$alloc_mem = 4 \times max_message_size \times ppn \times numprocs + size_um$$

osu_alltoall默认的max_message_size为1MB，按单节点2p、进程数575计算，最少需要650GB，远远大于单节点16张32GB满插内存。

版权所有 © 华为技术有限公司

说明

在上述计算中：

- UM通信所需要的内存size_um约为4GB。
- 单节点满插实际可用内存为471GB。

恢复步骤

根据上述公式合理设置max_message_size。例如按最大16节点满核（每个节点的进程数576、内存500GB）计算，在指定-c参数时，建议设置"-m 16384"将max_message_size设置为16KB。

说明

除了通过上述解决方案规避之外，还可以修改用例/应用源码，在调用MPI_Alltoall/MPI_Alltoallv/MPI_Alltoallw/MPI_Ialltoall/MPI_Ialltoallv/MPI_Ialltoallw接口时，将sbuf修改为MPI_IN_PLACE，以降低这些接口使用的内存规模。

4.3.22 运行 Alltoallv 算法 2 报错：gatherv phase don't support displs >= 2^31 bytes

现象描述

运行Alltoallv算法2（node-aware plummer）报错如下：

```
Alltoallv node-aware plummer gatherv phase don't support displs >= 2^31 bytes
```

可能原因

算法不支持节点内所有进程sendcounts总和超过2^31的场景，详见**场景十五：Alltoallv集合操作节点内所有进程sendcounts总和超过2^31**。

例如，运行osu应用时，进程数为128，节点数为2，每节点64进程，节点内所有进程sendcounts总和为128*64*count（count为每个sendcounts的值，即发送数据个数，osu中所有sendcounts都相同），当count达到256K时，sendcounts总和为2G，超过2^31，此场景会报出错误日志并退出作业。

恢复步骤

sendcounts不能超过Alltoallv算法2的限制，或者改用Alltoallv算法1（Ladd）。

4.3.23 环境 open files 值较大时导致 Hyper MPI 启动慢

现象描述

环境ulimit中open files值太大（如1073741816），Hyper MPI在多节点上执行任务，mpirun命令会停滞3分钟。

可能原因

执行SSH之前需要遍历并关闭所有的fd（文件描述符），当环境中使用**ulimit -n**命令设置的open files太大时，Hyper MPI的启动时延会增加。

版权所有 © 华为技术有限公司

恢复步骤

步骤1 排查是否由于fdmax值过大引起Hyper MPI启动时延慢，排查方法如下：

在运行命令中添加**--mca plm_base_verbose 11**参数打开日志开关，在打印内容中查找“plm:rsh: close max fd”字段内容，查看当前fdmax值。通常当fdmax值大于1073741816时，启动时延将大于3min，值越大，时延呈线性增加。

步骤2 若由于fdmax值过大引起Hyper MPI启动时延慢，可以根据需要使用**ulimit -n** *open files值*命令修改当前的open files值，fdmax值会取“当前fd的最大值”与“open files值”的更小者，然后参考**步骤1**查看修改的fdmax值是否生效。

----结束

4.3.24 UCX 使用 SDMA 时，报错：UCX ERROR Failed to create sdma_device[0] handle

现象描述

UCX使用SDMA时，出现如下报错：

```
UCX ERROR Failed to create sdma_device[0] handle
```

可能原因

在开启smmu的环境中，UCX的sm传输层默认会选择SDMA作为共享内存进行数据传输，单IO-Die中SDMA内核驱动默认总通道数为160个，其中包含独立通道数和共享通道数。MPI每启动一个业务进程，UCX会申请该进程所在IO-Die的一个SDMA共享通道。如果SDMA没有申请共享通道，MPI会在初始化阶段报错退出。如果通过命令参数-x UCX_SDMA_SHARED_CHANNEL_MODE=n选择独立通道模式，而SDMA独立通道数量申请不足，MPI同样会在初始化阶段报错退出。

恢复步骤

在使用SDMA运行业务之前，需要检查单IO-Die中SDMA内核共享通道是否大于0；如果采用独立通道模式，则需要根据业务规模手动修改SDMA内核独立通道数。由于单IO-Die中SDMA内核共享通道数和独立通道数默认总共为160个，因此可以通过修改共享通道数来间接修改独立通道数。修改方法如下：

步骤1 查看当前单IO-Die中SDMA内核共享通道数。

cat /sys/module/sdma_dae/parameters/share_chns

```
16
```

步骤2 移除SDMA内核模块。

rmmod sdma_dae

步骤3 重新插入SDMA内核模块，并修改共享通道数。

- **例1**

  modprobe sdma_dae share_chns=*160*

  例如，修改共享通道数为160个，则独立通道数为0个，因此该配置允许MPI在单IO-Die中启动SDMA业务进程。

 版权所有 © 华为技术有限公司

- **例2**

  modprobe sdma_dae share_chns=*8*

  例如，修改共享通道数为8个，则独立通道数为152个，因此该配置允许MPI在使用-x UCX_SDMA_SHARED_CHANNEL_MODE=n时，在单IO-Die中最多启动152个业务进程。

----结束

4.3.25 运行 MPI 作业，指定传输协议为 RC 或者 UD 时创建 qp 报错：Invalid argument

现象描述

运行MPI作业，指定传输协议为RC或者UD时创建qp报错：

```
UCX ERROR Failed to create RC QP TX wr:256 sge:5 in1:64 resp:64 RX wr:0 sge:1 resp:64: Invalid argument
failed: Invalid argument
UCX ERROR Failed to create UD QP TX wr:256 sge:6 inl:64 resp:0 RX wr:4096 sge:1 resp:0: Invalid argument
failed: Invalid argument
```

可能原因

当前BIOS配置不支持较多的sge，因为UCX中RC协议默认使用的sge值为5，UD协议默认使用的sge值为6。通过执行**ibv_devinfo -v**命令可以查看当前网口配置支持的最大值max_sge。

```
max_sge:    2
```

恢复步骤

在执行命令中增加参数**-x UCX_RC_VERBS_TX_MIN_SGE=***2* **-x UCX_UD_VERBS_TX_MIN_SGE=***1*。

其中，**UCX_RC_VERBS_TX_MIN_SGE**配置为**ibv_devinfo -v**命令查询出的max_sge值；**UCX_UD_VERBS_TX_MIN_SGE**配置为max_sge-1。

4.3.26 运行 MPI 作业，指定传输协议为 RC 或者 UD 时创建 qp 报错：No space left on device

现象描述

运行MPI作业，指定传输协议为RC或者UD时创建qp报错：

```
UCX ERROR Failed to create RC QP TX wr:256 sge:5 in1:64 resp:64 RX wr:0 sge:1 resp:64: No space left on
device failed: No space left on device
UCX ERROR Failed to create UD QP TX wr:256 sge:6 inl:64 resp:0 RX wr:4096 sge:1 resp:0: No space left on
device failed: No space left on device
```

可能原因

当MPI作业的建链数过大，所需qp数量超过硬件最大限制而资源不足时，会发生该报错。通过执行**ibv_devinfo -v**命令可以查看网口能够创建的最大qp数量max_qp。

```
max_qp:    1048576
```


版权所有 © 华为技术有限公司

恢复步骤

为了确保MPI作业能够正常运行，可以尝试从以下两个方面进行优化：

- 增加MPI作业使用的网口数量，以减少单一网口所需创建的qp数量。例如，使用两个及以上网口进行通信。
- 减少MPI作业所需qp数量，例如，尝试优化进程排布以减少建链数。

4.3.27 运行 MPI 作业，并启用 Stars 集合通信卸载，指定 Stars 传输协议为 rc_acc 时报错：ibv_wr_complete fail, ret 12

现象描述

运行MPI作业，并使能Stars集合通信卸载，指定Stars传输协议为rc_acc时报错：

```
ibv_wr_complete fail, ret 12
```

可能原因

Stars部分算法（如Bcast算法3、4，Allgatherv算法1）在ibv qp上下发的任务数较多，在作业进程数较大的场景下可能产生qp队列溢出的情况，造成以上报错。

恢复步骤

- 增大Stars RC qp队列深度，可通过“UCG_PLANC_STARS_RC_TX_QUEUE_LEN”参数设置，默认为1024，建议设置为8192。
- 建议使用Stars卸载时单通信域进程不超过1024进程。

4.3.28 运行 MPI 作业，网卡驱动报错：transport retry counter exceeded

现象描述

运行MPI作业，网卡驱动报错：

```
rc_verbs_iface.c:120 send completion with error: transport retry counter exceeded [qpn 0x272 wrid 0x1 vendor_err 0x15]
```

可能原因

HUCX中的传输层RC不会在软件中做重试，而是借助网卡的硬件重试机制；HUCX在初始化传输层时会配置默认的RC超时时间与重试次数，当数据在集群中传输较慢时，如果超出配置的总超时时间，网卡会返回给HUCX超时事件，HUCX获取到对应超时事件后退出应用。

恢复步骤

- 增大RC单次请求超时时间，可通过“UCX_RC_TIMEOUT”参数设置，默认为1s。
- 增大RC超时的重传次数，可通过“UCX_RC_RETRY_COUNT”参数设置，默认为7次。

版权所有 © 华为技术有限公司

说明

需结合网络实际延迟合理设置上述两个参数。

- 若参数设置过大，则会延迟网卡的故障检测，占用系统资源。
- 若参数设置过小，在某些场景（例如网络链路发生拥塞）下会导致网卡超时，应用不稳定，性能下降。
- 若遇到超时错误，建议逐步增大该值并观察结果，合理值需略高于网络最大延迟，避免极端调高。

4.3.29 管理、计算网卡都使用板载网卡并安装 RDMA 驱动，跑多节点 OSU 集合通信测试不指定网卡报错

现象描述

管理、计算网卡都使用板载网卡并安装RDMA驱动，在多节点OSU集合通信测试中，不指定网卡会报错，报错信息中提示流量走到管理口：

```
[node35:1880003:0:1880003]       ud_ep.c:282  Fatal: UD endpoint 0xaaaae8a908c0 to :0: unhandled timeout error with local dev name:roceo1 remote dev gid:[subnet prefix:0 interface id:19004742ffff0000]
```

可能原因

板载网卡驱动将管理网口和计算网口都识别为RDMA网口，HMPI不同进程可能分别选择管理网口和计算网口进行通信，由于管理网络和计算网络不通导致报错。

恢复步骤

在运行MPI应用时只指定计算网口为可用的RDMA网口，排除管理网口。

4.3.30 满核运行集合通信(i)osu_alltoall(v)/(i)osu_scatter(v)/(i)osu_gather(v)节点卡住挂死

现象描述

在满核下运行 (i)osu_alltoall(v)/(i)osu_scatter(v)/(i)osu_gather(v)，当使用默认的max_message_size（1MB）时，节点会卡住挂死。

可能原因

运行集合通信需要的内存超出节点实际内存。下图给出了不同OSU版本部分集合通信占用内存的情况。

图 4-2 不同 OSU 版本节点内占用内存分析

集合通信类型	OSU v7.2以下					OSU v7.2及以上				
	root进程		非root进程		单节点sendbuf、recvbuf内存总和	root进程		非root进程		单节点sendbuf、recvbuf、recvbuf_warmup内存总和
	sendbuf	recvbuf	sendbuf	recvbuf		sendbuf	recvbuf	sendbuf	recvbuf	
(i)gather(v)	max_msg_size	max_msg_size+np	max_msg_size	max_msg_size	max_msg_size+(2+PPN-1+np)	max_msg_size	max_msg_size+np	max_msg_size	max_msg_size+np	(np+1)+max_msg_size+PPN
(i)scatter(v)	max_msg_size+np	max_msg_size	max_msg_size	max_msg_size	max_msg_size+(2+PPN-1+np)	max_msg_size+np	max_msg_size+np	max_msg_size+np	max_msg_size+np	(2+np+1)+max_msg_size+PPN
(i)alltoall(v)	max_msg_size+np	max_msg_size+np	max_msg_size+np	max_msg_size+np	2+np+max_msg_size+PPN	max_msg_size+np	max_msg_size+np	max_msg_size+np	max_msg_size+np	3+np+max_msg_size+PPN

版权所有 © 华为技术有限公司

说明

- PPN -- 每个节点的进程数。
- np -- 总进程数。
- max_msg_size -- 指定集合通信最大包长。

恢复步骤

根据单节点实际内存，代入不同OSU版本的(i)osu_alltoall(v)/(i)osu_scatter(v)/(i)osu_gather(v)内存占用公式中计算max_msg_size，在mpirun命令中通过-m参数指定最大包长的值。

4.3.31 在 MPI 线程中使用 fork 方法新建线程，导致运行时报错

现象描述

在部分服务器上使用MPI运行osu_latency_mp等用例时会有Segment报错，但不影响运行以及功能。报错信息参考：

```
Caught signal 11 (Segmentation fault: address not mapped to object at address 0x3dbaf590)
```

可能原因

根据**OpenMPI官方文档**，在MPI进程中使用fork()、system()、popen()等操作是不安全的。这是因为这些操作依赖于多种因素，包括操作系统、计算机硬件和网络堆栈。开发人员应尽量避免在MPI应用程序中调用此类非安全操作。

规避方法

在部分网卡上，可以通过禁用DC协议来规避。

-x UCX_TLS=^dc

4.4 算法地图

本文介绍Hyper MPI支持算法的算法序号和算法内容的对应关系。

Allreduce 算法地图

表 4-6 Allreduce 算法地图

算法序号	算法内容
1	Recursive doubling
2	Node-aware recursive doubling and binomial tree
3	Socket-aware recursive doubling and binomial tree
4	Ring
5	Node-aware recursive doubling and k-nomial tree

版权所有 © 华为技术有限公司

算法序号	算法内容
6	Socket-aware recursive doubling and k-nomial tree
7	Node-aware k-nomial tree
8	Socket-aware k-nomial tree
11	NAP
12	Rabenseifner
13	Node-aware Rabenseifner
14	Socket-aware Rabenseifner

Bcast 算法地图

表 4-7 Bcast 算法地图

算法序号	算法内容
1	Binomial tree
2	Node-aware binomial tree
3	Node-aware k-nomial tree and binomial tree
4	Node-aware k-nomial tree
6	Ring
8	VanDeGeijn
9	PC-ESBT
10	Knomial tree
11	Long
12	Ring-modified
13	Ring2-modified
14	Long-modified

表 4-8 Bcast 卸载算法地图

算法序号	算法内容
1	Ring-modified
2	Ring2-modified

版权所有 © 华为技术有限公司

算法序号	算法内容
3	Long
4	Long-modified

Barrier 算法地图

表 4-9 Barrier 算法地图

算法序号	算法内容
1	Recursive doubling
2	Node-aware recursive doubling and binomial tree
3	Socket-aware recursive doubling and binomial tree
4	Node-aware recursive doubling and k-nomial tree
5	Socket-aware recursive doubling and k-nomial tree
6	Node-aware k-nomial tree
7	Socket-aware k-nomial tree
10	NAP

表 4-10 Barrier 卸载算法地图

算法序号	算法内容
1	faninfanout

Alltoallv 算法地图

表 4-11 Alltoallv 算法地图

算法序号	算法内容
1	Ladd
2	Node-aware Plummer
3	Linear
4	Pairwise
5	Optimized Node-aware Plummer

版权所有 © 华为技术有限公司

表 4-12 Alltoallv 卸载算法地图

算法序号	算法内容
1	Pairwise

Scatterv 算法地图

表 4-13 Scatterv 算法地图

算法序号	算法内容
1	Linear
2	Knomial tree
3	Node-aware Knomial tree

表 4-14 Scatterv 卸载算法地图

算法序号	算法内容
1	Knomial tree

Allgatherv 算法地图

表 4-15 Allgatherv 算法地图

算法序号	算法内容
1	Neighbor exchange
2	Ring
3	Ring-HPL
4	Linear
5	Bruck
6	Node-aware Rolling

表 4-16 Allgatherv 卸载算法地图

算法序号	算法内容
1	Rolling

版权所有 © 华为技术有限公司

Gatherv 算法地图

表 4-17 Gatherv 算法地图

算法序号	算法内容
1	Linear
2	Node-aware Linear
3	Knomial tree
4	Node-aware Knomial tree

Gather 算法地图

表 4-18 Gather 算法地图

算法序号	算法内容
1	Linear
2	Node-aware Linear
3	Knomial tree
4	Node-aware Knomial tree

Reduce_scatter_block 算法地图

表 4-19 Reduce_scatter_block 算法地图

算法序号	算法内容
1	Linear
2	Ring

Reduce_scatter 算法地图

表 4-20 Reduce_scatter 算法地图

算法序号	算法内容
1	Linear
2	Ring

版权所有 © 华为技术有限公司

4.5 命令参数

本文介绍当前Hyper MPI不同版本间所有的命令参数及其生效规则。

4.5.1 Hyper MPI 26.1.RC1 及以上版本命令参数

表 4-21 HyperMPI 26.1.RC1 及以上版本命令参数

HyperMPI 26.1.RC1及以上版本命令参数	取值	参数说明
-x OMPI_ROMIO_USE_COLL	• 取值类型：int • 取值范围：0/1	1表示MPI-IO里使能集合通信，0表示不使能。默认不使能
--mca ompi_group_union_opt	• 取值类型：bool • 取值范围：true/false	true表示使能优化的group_union操作，false表示不使能。默认不使能
--mca ompi_group_union_hashsize	• 取值类型：uint • 取值范围：0-2^32-1	设置优化的group_union操作的hash table的大小，默认值为1024
-x UCG_USE_SHM_POOL	• 取值类型：bool • 取值范围：y/n	y表示使能共享内存池特性，n表示不使能。默认不使能
--mca coll_tuned_alltoallw_algorithm	• 取值类型：int • 取值范围：1/2	1表示使用alltoallw的linear算法， 2表示使用alltoallw的pairwise算法，使能该参数需要同时添加--mca coll_tuned_use_dynamic_rules true
--mca coll_ucg_enable_coll	• 取值类型：string • 取值范围：集合名称，多个集合名称之间用英文逗号（,）间隔，例如allreduce,bcast。 • 默认值：无	可选参数，运行Hyper MPI作业时，开启的Hyper MPI优化后的集合操作。 示例： **--mca coll_ucg_enable_coll barrier,bcast**

版权所有 © 华为技术有限公司

4.5.2 Hyper MPI 25.2.1 及以上版本命令参数

表 4-22 HyperMPI 25.2.1 及以上版本命令参数

HyperMPI 25.2.1及以上版本命令参数	取值	参数说明
--mca coll_libnbc_ineighbor_alltoall_correct_cyclic_cart --mca coll_libnbc_ineighbor_alltoallv_correct_cyclic_cart --mca coll_libnbc_ineighbor_alltoallw_correct_cyclic_cart	• 取值类型：bool • 取值范围：true/false • 默认值：false	是否修复openmpi多维笛卡尔循环的单维度规模为1或2的情况下ineighbor_alltoall(v|w)的传值顺序正确。输入为true为已修复。

4.5.3 Hyper MPI 25.1.0.SPC001 及以上版本命令参数

表 4-23 HyperMPI 25.1.0.SPC001 及以上版本命令参数

Hyper MPI 25.1.0.SPC001及以上版本命令参数	取值	参数说明
-x UCX_RNDV_THRESH=	• 取值类型：<number>[b|kb|mb|gb]，inf或者auto • 取值范围：任意正整数 • 默认值：auto	指定从eager协议切换至rendezvous协议的阈值大小
--mca coll_tuned_use_dynamic_rules true --mca coll_tuned_dynamic_rules_filename /path/to/file	• 取值类型：string • 取值范围：算法配置文件路径 **说明** 参考配置文件放置于: bisheng编译器版本：/*HPCKit安装部署目录*/HPCKit/latest/hmpi/bisheng/release/hmpi/etc gcc编译器版本：/*HPCKit安装部署目录*/HPCKit/latest/hmpi/gcc/release/hmpi/etc • 默认值：无	针对mca coll tuned模块，指定算法配置文件，通过不同进程数、包大小，选择最优的集合通信算法

版权所有 © 华为技术有限公司

4.5.4 Hyper MPI 25.1.RC1 及以上版本命令参数

本文介绍当前Hyper MPI 25.1.RC1及以上版本命令参数。

说明

Hyper MPI 25.1.RC1及以上版本命令参数为在25.0.0及以上版本命令参数基础上新增的命令参数。

使用Hyper MPI命令参数指定算法时，可以参考**4.4 算法地图**了解具体参数值的含义。

表 4-24 Hyper MPI 25.1.RC1 及以上版本命令参数

Hyper MPI 25.1.RC1及以上版本命令参数	取值	参数说明
-x UCG_LOG_LEVEL=	• 取值类型：string • 取值范围： – fatal：错误，程序会立即终止。 – error：错误信息。 – warn：告警信息。 – info：提示信息。 – debug：调试信息。 – trace：详细调试信息。 **说明** 日志级别fatal>error>warn>info>debug>trace，级别越低，打印的日志信息越详细，例如设置为warn，则会打印fatal、error和warn级别的日志信息。 • 默认值：warn	可选参数，指定UCG日志级别。 示例： **-x UCG_LOG_LEVEL=warn** **说明** 当前release版日志支持到info级别，debug版本日志支持到debug级别，debug日志级别会打印点对点通信的对端进程、包长等信息。可以通过module load命令加载debug版的HMPI。

版权所有 © 华为技术有限公司

Hyper MPI 25.1.RC1及以上版本命令参数	取值	参数说明
-x UCX_LOG_LEVEL=	• 取值类型：string • 取值范围： – fatal：错误，程序会立即终止。 – error：错误信息。 – warn：告警信息。 – info：提示信息。 – debug：调试信息。 – trace：详细调试信息。 **说明** 日志级别fatal>error>warn>info>debug>trace，级别越低，打印的日志信息越详细，例如设置为warn，则会打印fatal、error和warn级别的日志信息。 • 默认值：warn	可选参数，指定UCX日志级别。 示例： **-x UCX_LOG_LEVEL=warn** **说明** 当前release版和日志支持到debug级别，debug版本日支持到trace级别。可以通过module load命令加载debug版的HMPI。

4.5.5 Hyper MPI 25.0.0 及以上版本命令参数

本文介绍当前Hyper MPI 25.0.0及以上版本命令参数。

说明

Hyper MPI 25.0.0及以上版本命令参数为在2.4.3及以上版本命令参数基础上新增的命令参数。

使用Hyper MPI命令参数指定算法时，可以参考**4.4 算法地图**了解具体参数值的含义。

表 4-25 Hyper MPI 25.0.0 及以上版本命令参数

Hyper MPI 25.0.0及以上版本命令参数	取值	参数说明
-x UCG_PLANC_STARS_TLS=	• 取值类型：string • 取值范围： – all – rc_acc – sdma_acc • 默认值：all	可选参数，指定卸载算法所用的传输协议。 示例： **-x UCG_PLANC_STARS_TLS=all**

版权所有 © 华为技术有限公司

Hyper MPI 25.0.0及以上版本命令参数	取值	参数说明
-x UCG_PLANC_STARS_NET_DEVICES=	● 取值类型：string ● 取值范围：当前设备上所有网卡。 ● 默认值：all	可选参数，指定卸载算法所用的网卡。 示例： **-x UCG_PLANC_STARS_NET_DEVICES=all**
-x UCG_PLANC_STARS_SELF_DEVICES=	● 取值类型：string ● 取值范围：memory ● 默认值：memory	可选参数，指定卸载算法所用的环回设备。 示例： **-x UCG_PLANC_STARS_SELF_DEVICES=memory**
-x UCG_PLANC_STARS_MAX_RAILS=	● 取值类型：int ● 取值范围：1~9 ● 默认值：1	可选参数，指定卸载算法最多使用的链路数。 示例： **-x UCG_PLANC_STARS_MAX_RAILS=1**
-x UCG_PLANC_STARS_BCAST_ATTR=	● 取值类型：string，格式为I:*id*[S:*score*R:*start-end*]。 ● 取值范围： – id：必填，1~4。 – score：选填，任意自然数。 – start：选填，任意自然数。 – end：选填，任意自然数。 ● 默认值：无固定值，根据生效规则调节赋值。	可选参数，Bcast的算法参数。 示例： **-x UCG_PLANC_STARS_BCAST_ATTR=I:1**

版权所有 © 华为技术有限公司

Hyper MPI 25.0.0及以上版本命令参数	取值	参数说明
-x UCG_PLANC_STARS_IBCAST_ATTR=	• 取值类型：string，格式为I:*id*[S:*score*R:*start-end*]。 • 取值范围： – id：必填，1~4。 – score：选填，任意自然数。 – start：选填，任意自然数。 – end：选填，任意自然数。 • 默认值：无固定值，根据生效规则调节赋值。	可选参数，IBcast的算法参数。 示例： **-x UCG_PLANC_STARS_IBCAST_ATTR=I:1**
-x UCG_PLANC_STARS_SCATTERV_ATTR=	• 取值类型：string，格式为I:*id*[S:*score*R:*start-end*]。 • 取值范围： – id：必填，1。 – score：选填，任意自然数。 – start：选填，任意自然数。 – end：选填，任意自然数。 • 默认值：无固定值，根据生效规则调节赋值。	可选参数，Scatterv的算法参数。 示例： **-x UCG_PLANC_STARS_SCATTERV_ATTR=I:1**
-x UCG_PLANC_STARS_ISCTTERV_ATTR=	• 取值类型：string，格式为I:*id*[S:*score*R:*start-end*]。 • 取值范围： – id：必填，1。 – score：选填，任意自然数。 – start：选填，任意自然数。 – end：选填，任意自然数。 • 默认值：无固定值，根据生效规则调节赋值。	可选参数，IScatterv的算法参数。 示例： **-x UCG_PLANC_STARS_ISCATTERV_ATTR=I:1**

版权所有 © 华为技术有限公司

Hyper MPI 25.0.0及以上版本命令参数	取值	参数说明
-x UCG_PLANC_STARS_ALLGATHERV_ATTR=	• 取值类型：string，格式为I:*id*[S:*score*R:*start-end*]。 • 取值范围： – id：必填，1。 – score：选填，任意自然数。 – start：选填，任意自然数。 – end：选填，任意自然数。 • 默认值：无固定值，根据生效规则调节赋值。	可选参数，Allgatherv的算法参数。 示例： **-x UCG_PLANC_STARS_ALLGATHERV_ATTR=I:1**
-x UCG_PLANC_STARS_IALLGATHERV_ATTR=	• 取值类型：string，格式为I:*id*[S:*score*R:*start-end*]。 • 取值范围： – id：必填，1。 – score：选填，任意自然数。 – start：选填，任意自然数。 – end：选填，任意自然数。 • 默认值：无固定值，根据生效规则调节赋值。	可选参数，IAllgatherv的算法参数。 示例： **-x UCG_PLANC_STARS_IALLGATHERV_ATTR=I:1**
-x UCG_PLANC_STARS_ALLTOALLV_ATTR=	• 取值类型：string，格式为I:*id*[S:*score*R:*start-end*]。 • 取值范围： – id：必填，1。 – score：选填，任意自然数。 – start：选填，任意自然数。 – end：选填，任意自然数。 • 默认值：无固定值，根据生效规则调节赋值。	可选参数，Alltoallv的算法参数。 示例： **-x UCG_PLANC_STARS_ALLTOALLV_ATTR=I:1**

版权所有 © 华为技术有限公司

Hyper MPI 25.0.0及以上版本命令参数	取值	参数说明
-x UCG_PLANC_STARS_IALLTOALLV_ATTR=	• 取值类型：string，格式为I:*id*[S:*score*R:*start-end*]。 • 取值范围： – id：必填，1。 – score：选填，任意自然数。 – start：选填，任意自然数。 – end：选填，任意自然数。 • 默认值：无固定值，根据生效规则调节赋值。	可选参数，IAlltoallv的算法参数。 示例： **-x UCG_PLANC_STARS_IALLTOALLV_ATTR=I:1**
-x UCG_PLANC_STARS_BARRIER_ATTR=	• 取值类型：string，格式为I:*id*[S:*score*R:*start-end*]。 • 取值范围： – id：必填，1。 – score：选填，任意自然数。 – start：选填，任意自然数。 – end：选填，任意自然数。 • 默认值：无固定值，根据生效规则调节赋值。	可选参数，Barrier的算法参数。 示例： **-x UCG_PLANC_STARS_BARRIER_ATTR=I:1**
-x UCG_PLANC_STARS_IBARRIER_ATTR=	• 取值类型：string，格式为I:*id*[S:*score*R:*start-end*]。 • 取值范围： – id：必填，1。 – score：选填，任意自然数。 – start：选填，任意自然数。 – end：选填，任意自然数。 • 默认值：无固定值，根据生效规则调节赋值。	可选参数，IBarrier的算法参数。 示例： **-x UCG_PLANC_STARS_IBARRIER_ATTR=I:1**

版权所有 © 华为技术有限公司

Hyper MPI 25.0.0及以上版本命令参数	取值	参数说明
-x UCG_PLANC_STARS_SCATTERV_KNTREE_DEGREE=	● 取值类型：int ● 取值范围：任意正整数 **说明** 建议配置为2~*运行作业节点数量*间的正整数，例如2~8。 ● 默认值：2	可选参数，指定Scatterv的算法参数。 示例： **-x UCG_PLANC_STARS_SCATTERV_KNTREE_DEGREE=2**
-x UCG_PLANC_STARS_BARRIER_FANIN_DEGREE=	● 取值类型：int ● 取值范围：任意正整数 **说明** 建议配置为2~*运行作业节点数量*间的正整数，例如2~8。 ● 默认值：4	可选参数，指定Barrier_faninfanout的算法参数。 示例： **-x UCG_PLANC_STARS_BARRIER_FANIN_DEGREE=4**
-x UCG_PLANC_STARS_BARRIER_FANOUT_DEGREE=	● 取值类型：int ● 取值范围：任意正整数 **说明** 建议配置为2~*运行作业节点数量*间的正整数，例如2~8。 ● 默认值：2	可选参数，指定Barrier_faninfanout的算法参数。 示例： **-x UCG_PLANC_STARS_BARRIER_FANOUT_DEGREE=2**
-x UCG_PLANC_STARS_SCATTERV_RUN_HPL=	● 取值类型：string ● 取值范围： – y：使用HPL模式运行。 – n：使用通用模式运行。 ● 默认值：n	可选参数，指定Scatterv的运行模式。 示例： **-x UCG_PLANC_STARS_SCATTERV_RUN_HPL=y**
-x UCG_PLANC_STARS_RC_SEG_SIZE=	● 取值类型：size_t ● 取值范围：0~2^64-1 ● 默认值：8256	可选参数，设置弹性缓冲区大小。 示例： **-x UCG_PLANC_STARS_RC_SEG_SIZE=8256**

版权所有 © 华为技术有限公司

Hyper MPI 25.0.0及以上版本命令参数	取值	参数说明
-x UCG_PLANC_STARS_RC_TX_QUEUE_LEN=	● 取值类型：unsigned ● 取值范围：0~2^32-1 ● 默认值：1024	可选参数，设置QP中发送队列的长度。 示例： **-x UCG_PLANC_STARS_RC_TX_QUEUE_LEN=1024**
-x UCG_PLANC_STARS_RC_TX_MIN_INLINE=	● 取值类型：unsigned ● 取值范围：0~2^32-1 ● 默认值：64	可选参数，设置在发送WQE时为内联数据保留的最小字节。 示例： **-x UCG_PLANC_STARS_RC_TX_MIN_INLINE=64**
-x UCG_PLANC_STARS_RC_TX_INLINE_RESP=	● 取值类型：unsigned ● 取值范围：0~2^32-1 ● 默认值：64	可选参数，设置在发送WQE时为内联响应保留的字节。对于足够小的响应，比如原子操作和small reads，都会被以内联方式接收。 示例： **-x UCG_PLANC_STARS_RC_TX_INLINE_RESP=64**
-x UCG_PLANC_STARS_RC_HOP_LIMIT=	● 取值类型：unsigned ● 取值范围：0~2^32-1 ● 默认值：255	可选参数，设置IB跳数限制/RoCEv2生存时间。应介于0和255之间。 示例： **-x UCG_PLANC_STARS_RC_HOP_LIMIT=255**
-x UCG_PLANC_STARS_RC_PATH_MTU=	● 取值类型：unsigned ● 取值范围： – default – 512 – 1024 – 2048 – 4096 ● 默认值：default	可选参数，设置路径最大传输单元。default表示将为设备选择最佳MTU。 示例： **-x UCG_PLANC_STARS_RC_PATH_MTU=1024**

版权所有 © 华为技术有限公司

Hyper MPI 25.0.0及以上版本命令参数	取值	参数说明
-x UCG_PLANC_STARS_RC_TIMEOUT=	• 取值类型：string，格式为<number>[s|us|ms|ns] • 取值范围：<number>为任意大于0的浮点数。 • 默认值：1ms	可选参数，设置传输超时时间。 示例： **-x UCG_PLANC_STARS_RC_TIMEOUT=1ms**
-x UCG_PLANC_STARS_RC_RETRY_COUNT=	• 取值类型：unsigned • 取值范围：0~2^32-1 • 默认值：7	可选参数，设置传输重试次数。 示例： **-x UCG_PLANC_STARS_RC_RETRY_COUNT=7**
-x UCG_PLANC_STARS_RC_RNR_TIMEOUT=	• 取值类型：string，格式为<number>[s|us|ms|ns] • 取值范围：<number>为任意大于0的浮点数。 • 默认值：1ms	可选参数，设置RNR超时时间。 示例： **-x UCG_PLANC_STARS_RC_RNR_TIMEOUT=1ms**
-x UCG_PLANC_STARS_RC_RNR_RETRY_COUNT=	• 取值类型：unsigned • 取值范围：0~2^32-1 • 默认值：7	可选参数，设置RNR重试次数。 示例： **-x UCG_PLANC_STARS_RC_RNR_RETRY_COUNT=7**
-x UCG_PLANC_STARS_RC_TX_CQ_MODERATION=	• 取值类型：unsigned • 取值范围：0~2^32-1 • 默认值：64	可选参数，在不请求完成的情况下，允许发送WQE的最大数量。 示例： **-x UCG_PLANC_STARS_RC_TX_CQ_MODERATION=64**
-x UCG_PLANC_STARS_RC_TX_CQ_LEN=	• 取值类型：unsigned • 取值范围：0~2^32-1 • 默认值：4096	可选参数，设置发送完成队列的长度。这限制了未完成的信号发送的总数。 示例： **-x UCG_PLANC_STARS_RC_TX_CQ_LEN=4096**

版权所有 © 华为技术有限公司

Hyper MPI 25.0.0及以上版本命令参数	取值	参数说明
-x UCG_PLANC_STARS_RC_TX_MAX_WR=	● 取值类型：unsigned ● 取值范围：0~2^32-1 ● 默认值：-1（即2^32-1）	可选参数，限制未完成的已发布工作请求的数量。实际限制为该值和TX队列长度之间的最小值。-1表示无限制。 示例： **-x UCG_PLANC_STARS_RC_TX_MAX_WR=-1**
-x UCG_PLANC_STARS_IB_RCACHE_MEM_PRIO=	● 取值类型：unsigned ● 取值范围：0~2^32-1 ● 默认值：1000	可选参数，指定注册缓存内存事件优先级。 示例： **-x UCG_PLANC_STARS_IB_RCACHE_MEM_PRIO=1000**
-x UCG_PLANC_STARS_IB_RCACHE_OVERHEAD=	● 取值类型：string，格式为<number>[s|us|ms|ns] ● 取值范围：<number>为任意大于0的浮点数。 ● 默认值：180ns	可选参数，指定注册缓存查找开销。 示例： **-x UCG_PLANC_STARS_IB_RCACHE_OVERHEAD=180ns**
-x UCG_PLANC_STARS_IB_RCACHE_ADDR_ALIGN=	● 取值类型：unsigned ● 取值范围：0~2^32-1 ● 默认值：64	可选参数，指定注册地址对齐缓存。必须为2的幂次方，且介于64和系统页面大小之间。 示例： **-x UCG_PLANC_STARS_IB_RCACHE_ADDR_ALIGN=64**

版权所有 © 华为技术有限公司

Hyper MPI 25.0.0及以上版本命令参数	取值	参数说明
-x UCG_PLANC_STARS_IB_REG_METHODS=	● 取值类型：string ● 取值范围： – rcache：用户空间注册缓存。 – odp：隐式按需分页。 – direct：直接注册。 可以输入多种取值，以英文逗号分割，例如rcache,odp。 ● 默认值：rcache,odp,direct	可选参数，设置不同优先级的注册MR的方法。 示例： **-x UCG_PLANC_STARS_IB_REG_METHODS=rcache,odp,direct**
-x UCG_PLANC_STARS_IB_MEM_REG_OVERHEAD=	● 取值类型：string，格式为<number>[s|us|ms|ns] ● 取值范围：<number>为任意大于0的浮点数。 ● 默认值：16us	可选参数，设置内存注册开销。 示例： **-x UCG_PLANC_STARS_IB_MEM_REG_OVERHEAD=16us**
-x UCG_PLANC_STARS_IB_MEM_REG_GROWTH=	● 取值类型：string，格式为<number>[s|us|ms|ns] ● 取值范围：<number>为任意大于0的浮点数。 ● 默认值：0.06ns	可选参数，设置内存注册增长率。 示例： **-x UCG_PLANC_STARS_IB_MEM_REG_GROWTH=0.1ns**
-x UCG_PLANC_STARS_IB_ETH_PAUSE_ON=	● 取值类型：string ● 取值范围： – y：在以太网上开启Pause帧。 – n：在以太网上关闭Pause帧。 ● 默认值：y	可选参数，是否在以太网上启用Pause帧。Pause帧是一种在以太网家族计算机网络拥塞时暂时停止数据传输的机制。如果将此参数设置为“n”，则将取消IB传输资格，因为在与RoCE一起使用时，在有损结构上可能表现不佳。 示例： **-x UCG_PLANC_STARS_IB_ETH_PAUSE_ON=y**

版权所有 © 华为技术有限公司

Hyper MPI 25.0.0及以上版本命令参数	取值	参数说明
-x UCG_PLANC_STARS_IB_ODP_NUMA_POLICY=	● 取值类型：string ● 取值范围： – default：不改变已有的策略。 – preferred：设置为MPOL_PREFERRED模式。 – bind：设置为MPOL_BIND模式。 ● 默认值：default	可选参数，设置重写ODP区域的NUMA策略，以避免额外的页迁移。 ● MPOL_PREFERRED模式设置优先分配的节点。内核将首先尝试从此节点分配页面，如果首选节点可用内存不足，则回落到其他节点。 ● MPOL_BIND模式指定一个严格的策略，限制内存分配给nodemask指定的节点。 如果nodemask指定了多个节点，则页面分配将来自具有足够空闲内存的节点，并且优先是离分配节点最近的节点。 示例： **-x UCG_PLANC_STARS_IB_ODP_NUMA_POLICY=bind**
-x UCG_PLANC_STARS_IB_ODP_MAX_SIZE=	● 取值类型：string ● 取值范围： – auto：自动检测是否按需分页。 – <number>[b|kb|mb|gb]：设置按需分页的最大内存区域大小为某一具体值。<number>取值范围为0~2^64-1。 ● 默认值：auto	可选参数，设置启用按需分页的最大内存区域大小。 示例： **-x UCG_PLANC_STARS_IB_ODP_MAX_SIZE=16kb**

版权所有 © 华为技术有限公司

Hyper MPI 25.0.0及以上版本命令参数	取值	参数说明
-x UCG_PLANC_STARS_RC_ROCE_LOCAL_SUBNET=	● 取值类型：string ● 取值范围： – y：接口子网内的地址可访问。 – n：所有远程RoCEv2 IP地址可从任何端口访问。 ● 默认值：n	可选参数，使用每个网络设备的本地IP地址和子网掩码，对RoCEv2报文进行路由，用于筛选端口的IB子网前缀，为空表示不进行筛选。 示例： **-x UCG_PLANC_STARS_RC_ROCE_LOCAL_SUBNET=y**
-x UCG_PLANC_STARS_RC_ROCE_LOCAL_SUBNET_PREFIX_LEN=	● 取值类型：string ● 取值范围： – auto：自动从设备地址检测子网前缀长度。 – inf：仅允许同一设备内的连接。 – <num>：为子网前缀指定数字比特位长度值。取值范围为0~32。 ● 默认值：auto	可选参数，使能UCX_IB_ROCE_LOCAL_SUBNET时，用于可达性检查的子网前缀长度，单位为比特。 示例： **-x UCG_PLANC_STARS_RC_ROCE_LOCAL_SUBNET_PREFIX_LEN=8**
-x UCG_PLANC_STARS_SDMA_BW=	● 取值类型：string，格式为<number>[T|G|M|K]B|b[[p|/]s] ● 取值范围：<number>为任意大于0的浮点数。 ● 默认值：16911MBs	可选参数，配置SDMA的带宽，用于数据发送时控制SDMA链路的分包大小，仅在Multi-Rail场景下生效，单链路下配置无意义。 示例： **-x UCG_PLANC_STARS_SDMA_BW=10000MBs**
-x UCX_RC_VERBS_TX_MIN_SGE=	● 取值类型：unsigned ● 取值范围：0~2^32-1 ● 默认值：5	可选参数，在RC协议下发送WQE中要保留的SG条目数。 示例： **-x UCX_RC_VERBS_TX_MIN_SGE=2**

版权所有 © 华为技术有限公司

Hyper MPI 25.0.0及以上版本命令参数	取值	参数说明
-x UCX_UD_VERBS_TX_MIN_SGE=	● 取值类型：unsigned ● 取值范围：0~2^32-1 ● 默认值：5	可选参数，在UD协议下发送WQE中要保留的SG条目数。 示例： **-x UCX_UD_VERBS_TX_MIN_SGE=1**
-x UCX_SDMA_SHARED_CHANNEL_MODE=	● 取值类型：bool ● 取值范围： – y：开启共享通道模式。 – n：使用独立通道模式。 ● 默认值：y	可选参数，开启该功能后，支持SDMA使用共享通道。 示例： **-x UCX_SDMA_SHARED_CHANNEL_MODE=y**
-x UCG_PLANC_STARS_STREAM_DEPTH=	● 取值类型：int ● 取值范围：4096~65535 ● 默认值：4096	可选参数，指定卸载队列深度。 示例： **-x UCG_PLANC_STARS_STREAM_DEPTH=4096**
-x UCG_PLANC_UCX_ALLGATHERV_DEFAULT_POLICY=	● 取值类型：bool ● 取值范围： – y：使用默认算法。 – n：使用优化算法。 ● 默认值：y	可选参数，指定Allgatherv算子使用优化算法（部分环境下可能不适用）。 示例： **-x UCG_PLANC_UCX_ALLGATHERV_DEFAULT_POLICY=y**
-x UCG_PLANC_UCX_ALLTOALLV_DEFAULT_POLICY=	● 取值类型：bool ● 取值范围： – y：使用默认算法。 – n：使用优化算法。 ● 默认值：y	可选参数，指定Alltoallv算子使用优化算法（部分环境下可能不适用）。 示例： **-x UCG_PLANC_UCX_ALLTOALLV_DEFAULT_POLICY=y**

4.5.6 Hyper MPI 2.4.3 及以上版本命令参数

本文介绍当前Hyper MPI 2.4.3及以上版本命令参数。

 版权所有 © 华为技术有限公司

说明

Hyper MPI 2.4.3及以上版本命令参数为在1.3.*及以上版本命令参数基础上新增的命令参数。

使用Hyper MPI命令参数指定算法时，可以参考4.4 算法地图了解具体参数值的含义。

表 4-26 Hyper MPI 2.4.3 及以上版本命令参数

Hyper MPI 2.4.3及以上版本命令参数	取值	参数说明
-x UCG_PLANC_ALLTOALLV_ALGO_SELECT=	• 取值类型：string • 取值范围： – y：开启Alltoallv小包场景下算法自动切换功能。 – n：关闭Alltoallv小包场景下算法自动切换功能。 • 默认值：n	可选参数，开启该功能后，当不指定Alltoallv算法时，在256字节及以下的小包场景下切换为性能更好的Node-aware Plummer算法；关闭该功能后，不进行算法切换。 示例： **-x UCG_PLANC_ALLTOALLV_ALGO_SELECT=y**

4.5.7 Hyper MPI 1.3.*及以上版本命令参数

本文介绍当前Hyper MPI 1.3.*及以上版本命令参数。

使用Hyper MPI命令参数指定算法时，可以参考4.4 算法地图了解具体参数值的含义。

表 4-27 Hyper MPI 1.3.*版本命令参数

Hyper MPI 1.3.*版本命令参数	取值	参数说明
-x UCG_PLANC_UCX_ALLREDUCE_FANOUT_INTER_DEGREE=	• 取值类型：int • 取值范围：任意正整数 **说明** 建议配置为2~*运行作业节点数量*间的正整数，例如2~8。 • 默认值：8	可选参数，Allreduce算法的算法序号为5、6、7和8时的参数。 示例： **-x UCG_PLANC_UCX_ALLREDUCE_FANOUT_INTER_DEGREE=8**

版权所有 © 华为技术有限公司

Hyper MPI 1.3.*版本命令参数	取值	参数说明
-x UCG_PLANC_UCX_ALLREDUCE_FANIN_INTER_DEGREE=	● 取值类型：int ● 取值范围：任意正整数 **说明** 建议配置为2~*运行作业节点数量*间的正整数，例如2~8。 ● 默认值：8	可选参数，Allreduce算法的算法序号为5、6、7和8时的参数。 示例： **-x UCG_PLANC_UCX_ALLREDUCE_FANIN_INTER_DEGREE=8**
-x UCG_PLANC_UCX_ALLREDUCE_FANOUT_INTRA_DEGREE=	● 取值类型：int ● 取值范围：任意正整数 **说明** 建议配置为2~*参与通信进程数量*间的正整数，例如2~8。 ● 默认值：8	可选参数，Allreduce算法的算法序号为7和8时的参数。 示例： **-x UCG_PLANC_UCX_ALLREDUCE_FANOUT_INTRA_DEGREE=2**
-x UCG_PLANC_UCX_ALLREDUCE_FANIN_INTRA_DEGREE=	● 取值类型：int ● 取值范围：任意正整数 **说明** 建议配置为2~*参与通信进程数量*间的正整数，例如2~8。 ● 默认值：4	可选参数，Allreduce算法的算法序号为7和8时的参数。 示例： **-x UCG_PLANC_UCX_ALLREDUCE_FANIN_INTRA_DEGREE=4**
-x UCG_PLANC_UCX_BARRIER_FANOUT_INTER_DEGREE=	● 取值类型：int ● 取值范围：任意正整数 **说明** 建议配置为2~*运行作业节点数量*间的正整数，例如2~8。 ● 默认值：8	可选参数，Barrier算法的算法序号为4、5、6和7时的参数。 示例： **-x UCG_PLANC_UCX_BARRIER_FANOUT_INTER_DEGREE=8**
-x UCG_PLANC_UCX_BARRIER_FANIN_INTER_DEGREE=	● 取值类型：int ● 取值范围：任意正整数 **说明** 建议配置为2~*运行作业节点数量*间的正整数，例如2~8。 ● 默认值：8	可选参数，Barrier算法的算法序号为4、5、6和7时的参数。 示例： **-x UCG_PLANC_UCX_BARRIER_FANIN_INTER_DEGREE=8**

版权所有 © 华为技术有限公司

Hyper MPI 1.3.*版本命令参数	取值	参数说明
-x UCG_PLANC_UCX_BARRIER_FANOUT_INTRA_DEGREE=	● 取值类型：int ● 取值范围：任意正整数 **说明** 建议配置为2~*参与通信进程数量*间的正整数，例如2~8。 ● 默认值：2	可选参数，Barrier算法的算法序号为6和7时的参数。 示例： **-x UCG_PLANC_UCX_BARRIER_FANOUT_INTRA_DEGREE=2**
-x UCG_PLANC_UCX_BARRIER_FANIN_INTRA_DEGREE=	● 取值类型：int ● 取值范围：任意正整数 **说明** 建议配置为2~*参与通信进程数量*间的正整数，例如2~8。 ● 默认值：4	可选参数，Barrier算法的算法序号为6和7时的参数。 示例： **-x UCG_PLANC_UCX_BARRIER_FANIN_INTRA_DEGREE=8**
-x UCG_PLANC_UCX_BCAST_ATTR=	● 取值类型：string，格式为 I:*id*[S:*score*R:*start-end*]。 ● 取值范围： – id：必填，1~14。 – score：选填，任意自然数。 – start：选填，任意自然数。 – end：选填，任意自然数。 ● 默认值：无固定值，根据生效规则调节赋值。	可选参数，Bcast的算法参数。 示例： **-x UCG_PLANC_UCX_BCAST_ATTR=I:4**

版权所有 © 华为技术有限公司

Hyper MPI 1.3.*版本命令参数	取值	参数说明
-x UCG_PLANC_UCX_ALLREDUCE_ATTR=	• 取值类型：string，格式为 I:*id*[S:*score*R:*start-end*]。 • 取值范围： – id：必填，1~14。 – score：选填，任意自然数。 – start：选填，任意自然数。 – end：选填，任意自然数。 • 默认值：无固定值，根据生效规则调节赋值。	可选参数，Allreduce的算法参数。 示例： **-x UCG_PLANC_UCX_ALLREDUCE_ATTR=I:7**
-x UCG_PLANC_UCX_BARRIER_ATTR=	• 取值类型：string，格式为 I:*id*[S:*score*R:*start-end*]。 • 取值范围： – id：必填，1~10。 – score：选填，任意自然数。 – start：选填，任意自然数。 – end：选填，任意自然数。 • 默认值：无固定值，根据生效规则调节赋值。	可选参数，Barrier的算法参数。 示例： **-x UCG_PLANC_UCX_BARRIER_ATTR=I:1**

版权所有 © 华为技术有限公司

Hyper MPI 1.3.*版本命令参数	取值	参数说明
-x UCG_PLANC_UCX_ALLTOALLV_ATTR=	• 取值类型：string，格式为 I:*id*[S:*score*R:*start-end*]。 • 取值范围： – id：必填，1或2。 – score：选填，任意自然数。 – start：选填，任意自然数。 – end：选填，任意自然数。 • 默认值：无固定值，根据生效规则调节赋值。	可选参数，Alltoallv的算法参数。 示例： **-x UCG_PLANC_UCX_ALLTOALLV_ATTR=I:1**
-x UCG_PLANC_UCX_ALLGATHERV_ATTR=	• 取值类型：string，格式为 I:*id*[S:*score*R:*start-end*]。 • 取值范围： – id：必填，1~6。 – score：选填，任意自然数。 – start：选填，任意自然数。 – end：选填，任意自然数。 • 默认值：无固定值，根据生效规则调节赋值。	可选参数，Allgatherv的算法参数。 示例： **-x UCG_PLANC_UCX_ALLGATHERV_ATTR=I:3**

版权所有 © 华为技术有限公司

Hyper MPI 1.3.*版本命令参数	取值	参数说明
-x UCG_PLANC_UCX_SCATTERV_ATTR=	• 取值类型：string，格式为I:*id*[S:*score*R:*start-end*]。 • 取值范围： – id：必填，1~3。 – score：选填，任意自然数。 – start：选填，任意自然数。 – end：选填，任意自然数。 • 默认值：无固定值，根据生效规则调节赋值。	可选参数，Scatterv的算法参数。 示例： **-x UCG_PLANC_UCX_SCATTERV_ATTR=I:1**
-x UCG_PLANC_UCX_GATHERV_ATTR=	• 取值类型：string，格式为I:*id*[S:*score*R:*start-end*]。 • 取值范围： – id：必填，1。 – score：选填，任意自然数。 – start：选填，任意自然数。 – end：选填，任意自然数。 • 默认值：无固定值，根据生效规则调节赋值。	可选参数，Gatherv的算法参数。 示例： **-x UCG_PLANC_UCX_GATHERV_ATTR=I:1**
-x UCG_PLANC_UCX_BCAST_NA_KNTREE_INTER_DEGREE=	• 取值类型：int • 取值范围：任意正整数 **说明** 建议配置为2~*运行作业节点数量*间的正整数，例如2~8。 • 默认值：8	可选参数，Bcast算法的算法序号为3和4时的参数。 示例： **-x UCG_PLANC_UCX_BCAST_NA_KNTREE_INTER_DEGREE=8**

版权所有 © 华为技术有限公司

Hyper MPI 1.3.*版本命令参数	取值	参数说明
-x UCG_PLANC_UCX_BCAST_NA_KNTREE_INTRA_DEGREE=	● 取值类型：int ● 取值范围：任意正整数 **说明** 建议配置为2~*参与通信进程数量*间的正整数，例如2~8。 ● 默认值：2	可选参数，Bcast算法的算法序号为4时的参数。 示例： **-x UCG_PLANC_UCX_BCAST_NA_KNTREE_INTRA_DEGREE=2**
-x UCG_PLANC_UCX_REDUCE_KNTREE_DEGREE=	● 取值类型：int ● 取值范围：任意正整数 **说明** 建议配置为2~*参与通信进程数量*间的正整数，例如2~8。 ● 默认值：2	可选参数，Allreduce算法的算法序号为2、3、5、6、7和8时的参数。 示例： **-x UCG_PLANC_UCX_REDUCE_KNTREE_DEGREE=2**
-x UCG_PLANC_UCX_SCATTERV_KNTREE_DEGREE=	● 取值类型：int ● 取值范围：任意正整数 **说明** 建议配置为2~*参与通信进程数量*间的正整数，例如2~8。 ● 默认值：2	可选参数，Scatterv算法的算法序号为2时的参数。 示例： **-x UCG_PLANC_UCX_SCATTERV_KNTREE_DEGREE=2**
-x UCG_PLANC_UCX_SCATTERV_MIN_SEND_BATCH=	● 取值类型：int ● 取值范围：任意自然数 ● 默认值：无固定值，根据生效规则调节赋值。	可选参数，Scatterv算法的算法序号为1时，发送批处理模式的最小边界值。 示例： **-x UCG_PLANC_UCX_SCATTERV_MIN_SEND_BATCH=1**
-x UCG_PLANC_UCX_SCATTERV_MAX_SEND_BATCH=	● 取值类型：int ● 取值范围：任意自然数 ● 默认值：无固定值，根据生效规则调节赋值。	可选参数，Scatterv算法的算法序号为1时，发送批处理模式的最大边界值。 示例： **-x UCG_PLANC_UCX_SCATTERV_MIN_SEND_BATCH=1**

版权所有 © 华为技术有限公司

Hyper MPI 1.3.*版本命令参数	取值	参数说明
-x UCG_PLANC_UCX_NPOLLS=	● 取值类型：int ● 取值范围：任意自然数 ● 默认值：3	可选参数，P2P请求测试UCP进度轮询周期数。 示例： **-x UCG_PLANC_UCX_NPOLLS=3** **说明** 在涉及原子操作的单边请求（例如osu的benchmark：osu_get_acc_latency/osu_acc_latency）场景中，可通过减小该值（例如设置为0），提高性能。
-x UCG_PLANC_UCX_USE_OOB=	● 取值类型：string ● 取值范围： – y或yes：强制开启 – n或no：关闭 ● 默认值：yes	可选参数，复用ompi pml ucx链路。 示例： **-x UCG_PLANC_UCX_USE_OOB=yes**
-x UCG_USE_MT_MUTEX=	● 取值类型：string ● 取值范围： – y：使用mutex锁 – n：使用spinlock锁 ● 默认值：n	可选参数，在UCG中支持多线程的锁的实现方式。 示例： **-x UCG_USE_MT_MUTEX=n**

版权所有 © 华为技术有限公司

Hyper MPI 1.3.*版本命令参数	取值	参数说明
-x UCG_LOG_LEVEL=	● 取值类型：string ● 取值范围： – fatal：错误，程序会立即终止。 – error：错误信息。 – warn：告警信息。 – info：提示信息。 – debug：调试信息。 – trace：详细调试信息。 **说明** 日志级别fatal>error>warn>info>debug>trace，级别越低，打印的日志信息越详细，例如设置为warn，则会打印fatal、error和warn级别的日志信息。 ● 默认值：warn	可选参数，指定UCG日志级别。 示例： **-x UCG_LOG_LEVEL=warn**
-x UCG_PLANC_UCX_PLANM=	● 取值类型：string ● 取值范围： – all：所有组件包含闭源库 – none：不使用闭源库 – hicoll：使用闭源组件hicoll ● 默认值：all	可选参数，是否加载闭源库。 示例： **-x UCG_PLANC_UCX_PLANM=all**
-x UCG_PLANC_UCX_ALLREDUCE_NUM_NAP_GROUP=	● 取值类型：int ● 取值范围：任意正整数 ● 默认值：1	可选参数，Allreduce算法的算法序号为11时的参数。 示例： **-x UCG_PLANC_UCX_ALLREDUCE_NUM_NAP_GROUP=1**

版权所有 © 华为技术有限公司

Hyper MPI 1.3.*版本命令参数	取值	参数说明
-x UCG_PLANC_UCX_BARRIER_NUM_NAP_GROUP=	● 取值类型：int ● 取值范围：任意正整数 ● 默认值：1	可选参数，Barrier算法的算法序号为10时的参数。 示例： **-x UCG_PLANC_UCX_BARRIER_NUM_NAP_GROUP=1**
-x UCG_PLANC_UCX_BCAST_ESBT_BLOCKS=	● 取值类型：int ● 取值范围：任意自然数 ● 默认值：0	可选参数，Bcast算法的算法序号为9时的参数。 示例： **-x UCG_PLANC_UCX_BCAST_ESBT_BLOCKS=0**
-x UCG_PLANC_UCX_ALLTOALLV_MIN_SEND_BATCH=	● 取值类型：int ● 取值范围：任意自然数 ● 默认值：无固定值，根据生效规则调节赋值。	可选参数，Alltoallv算法的算法序号为1时的参数，发送批处理模式的最小边界值。 示例： **-x UCG_PLANC_UCX_ALLTOALLV_MIN_SEND_BATCH=0**
-x UCG_PLANC_UCX_ALLTOALLV_MAX_SEND_BATCH=	● 取值类型：int ● 取值范围：任意自然数 ● 默认值：无固定值，根据生效规则调节赋值。	可选参数，Alltoallv算法的算法序号为1时的参数，发送批处理模式的最大边界值。 示例： **-x UCG_PLANC_UCX_ALLTOALLV_MAX_SEND_BATCH=1**
-x UCG_PLANC_UCX_REDUCE_CONSISTENCY=	● 取值类型：string ● 取值范围： – y：保证allreduce结果一致性 – n：不保证allreduce结果一致性 ● 默认值：n	可选参数，UCG中Allreduce结果是否保证一致性。 示例： **-x UCG_PLANC_UCX_REDUCE_CONSISTENCY=n**

版权所有 © 华为技术有限公司

Hyper MPI 1.3.*版本命令参数	取值	参数说明
--mca coll_ucg_priority	● 取值类型：int ● 取值范围：任意自然数 ● 默认值：90	可选参数，Hyper MPI调用UCG模块的优先级，值越大，优先级越高。 示例： **--mca coll_ucg_priority 90**
--mca coll_ucg_verbose	● 取值类型：int ● 取值范围：任意自然数 ● 默认值：2	可选参数，UCG组件的日志详细程度。 示例： **--mca coll_ucg_verbose 2**
--mca coll_ucg_max_rcache_size	● 取值类型：int ● 取值范围：任意自然数 ● 默认值：10	可选参数，coll ucg组件的请求cache大小。 示例： **--mca coll_ucg_max_rcache_size 10**
--mca coll_ucg_disable_coll	● 取值类型：string ● 取值范围：集合名称，多个集合名称之间用英文逗号（,）间隔，例如allreduce,bcast。 ● 默认值：无	可选参数，运行Hyper MPI作业时，禁用的Hyper MPI优化后的集合操作。 示例： **--mca coll_ucg_disable_coll barrier,bcast**
--mca topo_tpm_enable	● 取值类型：int ● 取值范围： – 0：关闭 – 1：开启 ● 默认值：0	可选参数，是否使用进程排布功能。 示例： **--mca topo_tpm_enable 0**
--mca pmix_pmix3x_wait_time	● 取值类型：long，单位为s ● 取值范围：除0以外的任意自然数 ● 默认值：5	可选参数，Client端进程等待Server端返回处理结果的时间。 示例： **--mca pmix_pmix3x_wait_time 5**

版权所有 © 华为技术有限公司

Hyper MPI 1.3.*版本命令参数	取值	参数说明
-x UCX_UD_VERBS_MIN_PENDING_TICK	• 取值类型：double单位可为s/ms/us/ns，默认单位为s • 取值范围：任意大于0的实数 • 默认值：3s	可选参数，调度器的最小挂起时间，防止UD VERBS协议下因调度器挂起导致误报超时。 示例： **-x UCX_UD_VERBS_MIN_PENDING_TICK=2s**
-x UCX_UD_MLX5_MIN_PENDING_TICK	• 取值类型：double单位可为s/ms/us/ns，默认单位为s • 取值范围：任意大于0的实数 • 默认值：3s	可选参数，调度器的最小挂起时间，防止UD MLX5协议下因调度器挂起导致误报超时。 示例： **-x UCX_UD_MLX5_MIN_PENDING_TICK=2s**
-x UCX_MIN_PENDING_TIME	• 取值类型：double单位可为s/ms/us/ns，默认单位为s • 取值范围：任意大于0的实数 • 默认值：3s	可选参数，调度器的最小挂起时间，防止ucp request超时检测中因调度器挂起导致误报超时。 示例： **-x UCX_MIN_PENDING_TIME=2s**
-x UCX_REQ_TIMEOUT	• 取值类型：double单位可为s/ms/us/ns，默认单位为s • 取值范围：任意大于0的实数 • 默认值：600s	可选参数，ucp request超时检测的超时阈值，当点对点通信请求未完成时间超过该阈值时给出超时报警。 示例： **-x UCX_REQ_TIMEOUT=100s**

4.5.8 Hyper MPI 1.2.*及以下版本命令参数

Hyper MPI在支持1.3.*版本命令参数的基础上兼容1.2.*及以下版本的命令参数。本文介绍Hyper MPI 1.2.*及以下版本的命令参数，1.2.*及以下版本的命令参数与1.3.*版本命令参数之间的生效规则和转换关系。

生效规则

- 如果不对任何版本的命令参数进行手动赋值，则生效值为1.3.*版本命令参数的默认值。
- 如果手动为Hyper MPI命令参数赋值，生效规则如**表4-28**所示。

版权所有 © 华为技术有限公司

表 4-28 生效规则

1.3.*版本命令参数		1.2.*及以下版本命令参数		生效值
合法	不合法	合法	不合法	
√	-	√	-	1.3.*版本命令参数的值
√	-	-	√	1.3.*版本命令参数的值
-	√	√	-	1.2.*及以下版本命令参数的值
-	√	-	√	1.3.*版本命令参数的默认值
√	-	-	-	1.3.*版本命令参数的值
-	√	-	-	1.3.*版本命令参数的默认值
-	-	√	-	1.2.*及以下版本命令参数的值
-	-	-	√	1.3.*版本命令参数的默认值

- 如果手动为Hyper MPI命令参数赋值，生效规则如下：
  - 同时为1.3.*版本命令参数和1.2.*及以下版本命令参数赋值，且值均合法，则生效值为1.3.*版本命令参数的值。
  - 同时为1.3.*版本命令参数和1.2.*及以下版本命令参数赋值，但1.2.*及以下版本命令参数的值不合法，则生效值为1.3.*版本命令参数的值。
  - 同时为1.3.*版本命令参数和1.2.*及以下版本命令参数赋值，但1.3.*版本命令参数的值不合法，则生效值为1.2.*及以下版本命令参数的值。
  - 同时为1.3.*版本命令参数和1.2.*及以下版本命令参数赋值，且值均不合法，则生效值为1.3.*版本命令参数的默认值。
  - 仅为1.3.*版本命令参数赋值，且值合法，则生效值为1.3.*版本命令参数的值。
  - 仅为1.3.*版本命令参数赋值，但值不合法，则生效值为1.3.*版本命令参数的默认值。
  - 仅为1.2.*及以下版本命令参数赋值，且值合法，则生效值为1.2.*及以下版本命令参数的值。
  - 仅为1.2.*及以下版本命令参数赋值，但值不合法，则生效值为1.3.*版本命令参数的默认值。

使用建议

- 较Hyper MPI 1.2.*版本命令参数，Hyper MPI 1.3.*版本的命令参数可以提供更全面、更优的性能，建议使用Hyper MPI 1.3.*版本的命令参数。
- 使用Hyper MPI命令参数指定算法时，可以参考**4.4 算法地图**了解具体参数值的含义。

版权所有 © 华为技术有限公司

命令参数及转换关系

表 4-29 1.2.*及以下版本命令参数及其与 1.3.*版本命令参数之间转换关系

Hyper MPI 1.2.*及以下版本命令参数	Hyper MPI 1.3.*版本命令参数	取值	参数说明
-x UCX_BUILTIN_DEGREE_INTER_FANOUT=	-x UCG_PLANC_UCX_ALLREDUCE_FANOUT_INTER_DEGREE=	• 取值类型：int • 取值范围：任意正整数 **说明** 建议配置为2~*运行作业节点数量*间的正整数。 • 默认值：8	可选参数，表示算法使用的节点间K项树的扇出值。具体算法如下： • Allreduce算法的算法序号为7和8时的参数。 • Barrier算法的算法序号为6和7时的参数。 • Bcast算法的算法序号为4时的参数。 示例： **-x UCX_BUILTIN_DEGREE_INTER_FANOUT=8**
	-x UCG_PLANC_UCX_BARRIER_FANOUT_INTRA_DEGREE=	• 取值类型：int • 取值范围：任意正整数 **说明** 建议配置为2~*参与通信进程数量*间的正整数。 • 默认值：8	
	-x UCG_PLANC_UCX_BCAST_NA_KNTREE_INTER_DEGREE=	• 取值类型：int • 取值范围：任意正整数 **说明** 建议配置为2~*运行作业节点数量*间的正整数。 • 默认值：8	
-x UCX_BUILTIN_DEGREE_INTER_FANIN=	-x UCG_PLANC_UCX_ALLREDUCE_FANIN_INTER_DEGREE=	• 取值类型：int • 取值范围：任意正整数 **说明** 建议配置为2~*运行作业节点数量*间的正整数。 • 默认值：8	可选参数，表示算法使用的节点间K项树的扇入值。具体算法如下： • Allreduce算法的算法序号为7和8时的参数。 • Barrier算法的算法序号为6和7时的参数。 示例： **-x UCX_BUILTIN_DEGREE_INTER_FANIN=8**
	-x UCG_PLANC_UCX_BARRIER_FANIN_INTER_DEGREE=		

版权所有 © 华为技术有限公司

Hyper MPI 1.2.* 及以下版本命令参数	Hyper MPI 1.3.* 版本命令参数	取值	参数说明
-x UCX_BUILTIN_DEGREE_INTRA_FANOUT=	-x UCG_PLANC_UCX_ALLREDUCE_FANOUT_INTRA_DEGREE= -x UCG_PLANC_UCX_BARRIER_FANOUT_INTRA_DEGREE= -x UCG_PLANC_UCX_BCAST_NA_KNTREE_INTRA_DEGREE=	● 取值类型：int ● 取值范围：任意正整数 **说明** 建议配置为2~*参与通信进程数量*间的正整数。 ● 默认值：2	可选参数，表示算法使用的节点内K项树的扇出值。具体算法如下： ● Allreduce算法的算法序号为5、6、7和8时的参数。 ● Barrier算法的算法序号为4、5、6和7时的参数。 ● Bcast算法的算法序号为3和4时的参数。 示例： **-x UCX_BUILTIN_DEGREE_INTRA_FANOUT=8**
-x UCX_BUILTIN_DEGREE_INTRA_FANIN=	-x UCG_PLANC_UCX_ALLREDUCE_FANIN_INTRA_DEGREE= -x UCG_PLANC_UCX_BARRIER_FANIN_INTRA_DEGREE=	● 取值类型：int ● 取值范围：任意正整数 **说明** 建议配置为2~*参与通信进程数量*间的正整数。 ● 默认值：2	可选参数，表示算法使用的节点内K项树的扇入值。具体算法如下： ● Allreduce算法的算法序号为5、6、7和8时的参数。 ● Barrier算法的算法序号为4、5、6和7时的参数。 示例： **-x UCX_BUILTIN_DEGREE_INTRA_FANIN=2**

版权所有 © 华为技术有限公司

Hyper MPI 1.2.* 及以下版本命令参数	Hyper MPI 1.3.* 版本命令参数	取值	参数说明
-x UCX_BUILTIN_BCAST_ALGORITHM=	-x UCG_PLANC_UCX_BCAST_ATTR=	• 取值类型：int • 取值范围：1~5 • 默认值：无固定值，根据生效规则调节赋值。	可选参数，用于选择Bcast算法序号的参数。 示例： **-x UCX_BUILTIN_BCAST_ALGORITHM=4**
-x UCX_BUILTIN_ALLREDUCE_ALGORITHM=	-x UCG_PLANC_UCX_ALLREDUCE_ATTR=	• 取值类型：int • 取值范围：1~14 • 默认值：无固定值，根据生效规则调节赋值。	可选参数，用于选择Allreduce算法序号的参数。 示例： **-x UCX_BUILTIN_ALLREDUCE_ALGORITHM=7**
-x UCX_BUILTIN_BARRIER_ALGORITHM=	-x UCG_PLANC_UCX_BARRIER_ATTR=	• 取值类型：int • 取值范围：1~10 • 默认值：无固定值，根据生效规则调节赋值。	可选参数，用于选择Barrier算法序号的参数。 示例： **-x UCX_BUILTIN_BARRIER_ALGORITHM=1**
-x UCX_BUILTIN_MAX_MSG_LIST_SIZE=	-x UCG_PLANC_UCX_NPOLLS=	• 取值类型：int • 取值范围：任意正整数 • 默认值：40	可选参数，P2P请求测试UCP进度轮询周期数。 示例： **-x UCX_BUILTIN_MAX_MSG_LIST_SIZE=40**
-x UCX_BUILTIN_REDUCE_CONSISTENCY=	-x UCG_PLANC_UCX_REDUCE_CONSISTENCY=	• 取值类型：string • 取值范围： – y：保证allreduce结果一致性 – n：不保证allreduce结果一致性 • 默认值：n	可选参数，UCG中Allreduce结果是否保证一致性。 示例： **-x UCX_BUILTIN_REDUCE_CONSISTENCY=n**

版权所有 © 华为技术有限公司

4.6 术语&缩略语

名称	说明
CPU	中央处理器（Central Processing Unit）
Hyper MPI	华为高性能通信库（Hyper Message Passing Interface），简称HMPI
集群	将多个服务器集中起来使它们能够像一台服务器一样提供服务的计算机技术。采用集群通常是为了提高系统的稳定性、可靠性、数据处理能力或服务能力。例如，可以减少单点故障、共享存储资源、负荷分担或提高系统性能等。
MCA	模块化组件结构（Modular Component Architecture）
MPI	并行计算通信应用接口（Message Passing Interface）
RoCE	基于融合以太的远程内存直接访问协议（RDMA over Converged Ethernet），是一种允许通过以太网使用远程直接内存访问（RDMA）的网络协议。
许可证	就某一产品特定功能、容量、使用期限等方面进行授权的许可。许可可以是文件或序列号的形式，通常由加密码组成，操作权限随许可的级别而不同。

版权所有 © 华为技术有限公司

5 KUPL 开发指南

概念介绍

使用指导

KUPL库函数说明

5.1 概念介绍

鲲鹏统一并行加速库（Kunpeng Unified Parallel Library，以下简称KUPL）提供了基于鲲鹏平台优化的并行加速基础库函数，所有接口用C/C++、汇编语言实现。本加速库提供包括底层线程管理、任务调度、线程同步、内存申请、共享内存申请、共享内存通信、矩阵编程计算等在内的基础功能，充分发挥鲲鹏处理器的硬件特性，提供高性能的基础接口。

5.2 使用指导

5.2.1 使用 KUPL 加速 KML 直接求解法

当前版本除了通过线程调度、任务分配等优化对数学库直接法求解器进行加速实现加速效果，还直接对外提供KUPL API进行使用从而提供并行加速能力。KML直接求解法ksolver已采用KUPL动态伸缩的能力对矩阵的部分求解过程进行加速优化，只需要在使用ksolver时通过环境变量的配置即可使能KUPL加速能力，具体使用方式如下。

步骤1 节点上安装HPCKit，依据**《Kunpeng HPCKit 26.1.RC1 开发指南》**编写ksolver_testcase，编译ksolver_testcase的时候链接HPCKit中的libksolver.so 。

步骤2 执行ksolver_testcase二进制，设置环境变量KML_DSS_SCHE_MODE=KUPL使能KUPL动态伸缩能力，该环境变量默认值为static，表示走静态调度策略，即不进行负载均衡优化。

命令举例如下：

```
KML_DSS_SCHE_MODE=KUPL OMP_PROC_BIND=close OMP_NUM_THREADS=64 taskset -c 0-63 ksolver_testcase
```

 版权所有 © 华为技术有限公司

说明

如果没有出现如下回显信息，说明命令执行成功。出现如下回显的原因是下载HPCKit后，对so目录架构进行了修改。

```
[warn]: KUPL dynamic mode is not available, fallback to builtin dynamic mode
```

----结束

5.2.2 使用 KUPL API 进行并行计算加速

KUPL当前对外优化特性分为三大模块，具体为众核并行、数据管理和矩阵编程：

- 众核并行模块对外提供了包含egroup、多线程编程、计算图编程、多队列多流编程等KUPL API用以多线程并行计算加速；
- 数据管理模块对外提供了包含内存管理、共享内存通信等KUPL API用以数据拷贝或通信加速；
- 矩阵编程模块对外提供了张量定义、张量计算等KUPL API用以加速矩阵乘计算。

具体上述KUPL特性模块的优化使能方式，可以参考**KUPL sample代码仓**。

KUPL sample代码仓中提供了一组使用KUPL优化特性的示例代码及特定场景KUPL最优实现代码，从而供用户快速上手使能KUPL特性。

KUPL sample的根目录包含4个顶层目录，分别对应众核并行模块、数据管理模块、矩阵编程模块及KUPL综合用例，每个模块的顶层目录再包含模块对应重点特性的用例目录，最终用例目录下挂载具体特性用例的sample代码、makefile及readme说明介绍。具体sample运行方式如下。

步骤1 节点上安装**HPCKit**，HPCKit安装指导请参见**《Kunpeng HPCKit 26.1.RC1 安装指南》**。

步骤2 通过source setvars.sh加载毕昇版本KUPL动态链接库及其他所需环境变量。

步骤3 Git拉取KUPL sample代码仓代码至本地服务器上。

git clone https://gitcode.com/kunpengcompute/hpckit-sample.git

cd kupl

步骤4 编译运行KUPL sample代码仓代码，当前支持两种方式：

- 一键式全量编译安装。
  a. kupl目录下执行sh build.sh，在所有特性的目录下生成对应特性的可执行二进制。
  b. 根目录下执行如cd mt/graph/到KUPL某一具体特性的目录下。
  c. 执行make run运行生成的二进制得到运行结果。
- 按需编译安装。
  a. kupl目录下执行如cd mt/graph/到KUPL某一具体特性的目录下。
  b. 执行make命令令编译生成该特性对应的二进制文件。
  c. 执行make run命令运行生成的二进制得到运行结果。

----结束

版权所有 © 华为技术有限公司

5.2.3 使用 KUPL PROF 进行全量的函数时序统计

KUPL对外提供profile和trace两个工具，来提供运行时的时序统计，通过动态链接到prof对应的动态链接库来使能profile和trace功能。

步骤1 节点上安装**HPCkit**，HPCKit安装指导请参见**《Kunpeng HPCKit 26.1.RC1 安装指南》**

步骤2 通过setvars.sh --kupl_mode=prof链接到prof动态链接库。

步骤3 运行程序时额外配置环境变量KUPL_PROF_LEVEL=statistic来开启特定的函数累计计时，KUPL_PROF_LEVEL=trace来开启全量的函数时序统计。

步骤4 程序运行结束时，如果配置了KUPL_PROF_LEVEL=statistic或KUPL_PROF_LEVEL=trace则会在程序运行打印结尾生成函数累计计时profile报告（报告表头字段详情见**表1 函数累计计时profile报告的表头字段说明**），如果配置了KUPL_PROF_LEVEL=trace还可以在程序运行目录下生成ptrace文件，其中包含各个进程中各个线程运行情况的统计文件（以.json文件保存），该文件可以通过chrome://tracing网页（请使用chrome 94以上版本的Chrome浏览器运行）来观看可视化版本报告（即把想要观看的文件导入进网页）。

表 5-1 函数累计计时 profile 报告的表头字段说明

信息名称	描述
thread_id	执行线程id
msg_id	函数信息号
msg	函数信息名称
sum(ms)	函数总耗时（毫秒）
count	函数执行次数
avg(ns)	函数平均耗时（纳秒）
max(ns)	函数单次最大耗时（纳秒）
min(ns)	函数单次最小耗时（纳秒）

表 5-2 函数累计计时 profile 报告的 msg 字段支持种类含义说明

msg信息名称	函数行为
task_execute	提交给KUPL的任务执行耗时
task_finish	提交给KUPL的任务执行完毕后释放资源耗时
sched_add_taskbase	向KUPL底层Sched中提交任务耗时
sched_get_taskbase	从KUPL底层Sched中获取任务耗时

版权所有 © 华为技术有限公司

说明

通过网页观看可视化版本报告建议使用Chrome浏览器。

----结束

5.3 KUPL 库函数说明

5.3.1 函数说明

KUPL提供了基于鲲鹏平台优化的并行加速基础库函数，所有接口由C/C++、汇编语言实现。其中KUPL库函数包含executor相关函数、多线程编程函数、计算图编程函数、内存拷贝函数、共享内存通信函数、矩阵编程接口函数等，以实现鲲鹏并行计算加速。

5.3.2 函数定义

5.3.2.1 返回值定义

表 5-3 返回值定义

ReturnCode	类型	值	描述
KUPL_OK	int	0	执行成功。
KUPL_ERROR	int	-1	执行过程出现错误，执行失败。

5.3.2.2 executor 相关函数

5.3.2.2.1 概念说明

KUPL中多线程执行操作行为均由执行器executor来运行。该章节介绍了与executor相关的函数操作，包含获取当前执行器编号以及执行器数等行为；除此之外通过使用egroup实现对等线程组概念，从而细粒度控制多线程并发行为。

相关概念定义如下：

- executor，表示执行器，可对应业界多线程编程中的线程概念。
- egroup，表示executor的集合，即执行器的集合。
- kupl_egroup_h，表示egroup数据结构的句柄。

5.3.2.2.2 环境变量

- KUPL_EXECUTOR_BACKEND，表示KUPL采用的具体executor后端模式。
- 当KUPL_EXECUTOR_BACKEND=pthread时，选择后端为pthread模式。
- 下述接口demo均为pthread后端下KUPL实现，因此需显式配置KUPL_EXECUTOR_BACKEND=pthread方可执行。

版权所有 © 华为技术有限公司

- KUPL_EXECUTOR_BACKEND还支持设定为omp，选择后端为omp模式，用于KUPL和OpenMP混合使用的场景，该模式为中间方案，后续会实现KUPL对接OpenMP方案用于替换。

5.3.2.2.3 kupl_get_num_executors

获取kupl executor数量。

接口定义

int kupl_get_num_executors();

参数

无

返回值

返回kupl executor数量。

示例

```
#include <stdio.h>
#include "kupl.h"

int main()
{
    int num = kupl_get_num_executors();
    printf("kupl executor number = %d\n", num);
    return 0;
}
```

运行结果如下。

```
kupl executor number = 128
```

说明

上述示例打印了executor执行器的总数；运行结果以实际为准，上述结果仅供参考。

5.3.2.2.4 kupl_get_executor_num

获取当前执行该函数的kupl executor编号。

接口定义

int kupl_get_executor_num();

参数

无。

返回值

返回当前执行该函数的kupl executor编号。

版权所有 © 华为技术有限公司

示例

```
#include <stdio.h>
#include "kupl.h"

int main()
{
    int num = kupl_get_executor_num();
    printf("current kupl executor id = %d\n", num);
    return 0;
}
```

运行结果如下。

```
current kupl executor id = 0
```

说明

上述示例打印了当前executor执行器的编号。运行结果以实际为准，上述结果仅供参考。

5.3.2.2.5 kupl_egroup_create

创建kupl egroup，即kupl executor的集合。

接口定义

kupl_egroup_h kupl_egroup_create(int *executors, int executors_num);

参数

表 5-4 参数定义

参数名	类型	描述	输入/输出
executors	int *	组成egroup的executor，具体参数传入方式为将需要构成集合的执行器的编号赋值给该数组 约束： 0≤executors[i]≤kupl_get_num_executors()	输入
executors_num	int	egroup中executor数量 约束： 0≤executors_num≤kupl_get_num_executors()	输入

返回值

- 成功：返回创建的egroup

版权所有 © 华为技术有限公司

- 失败：返回nullptr

示例

```
#include <stdio.h>
#include "kupl.h"

int main()
{
    int executor_num = kupl_get_num_executors();
    int executors[executor_num];
    for (int i =0; i < executor_num; i++) {
        executors[i] = i;
    }
    kupl_egroup_h egroup = kupl_egroup_create(executors, executor_num);
    kupl_egroup_destroy(egroup);
    return 0;
}
```

说明

上述示例演示了创建、销毁一个egroup的流程。kupl_egroup_create函数创建了一个包含所有executor执行器的egroup。

5.3.2.2.6 kupl_egroup_destroy

销毁kupl egroup。

接口定义

void kupl_egroup_destroy(kupl_egroup_h group);

参数

表 5-5 参数定义

参数名	类型	描述	输入/输出
group	kupl_egroup_h	需要销毁的egroup	输入

示例

```
#include <stdio.h>
#include "kupl.h"

int main()
{
    int executor_num = kupl_get_num_executors();
    int executors[executor_num];
    for (int i =0; i < executor_num; i++) {
        executors[i] = i;
    }
    kupl_egroup_h egroup = kupl_egroup_create(executors, executor_num);
    kupl_egroup_destroy(egroup);
    return 0;
}
```

说明

上述示例演示了创建、销毁一个egroup的流程。kupl_egroup_destroy函数销毁了一个通过kupl_egroup_create 函数创建的egroup。

版权所有 © 华为技术有限公司

5.3.2.2.7 kupl_egroup_borrow

将src egroup中所有的executor资源移动至dest egroup中。

接口定义

int kupl_egroup_borrow(kupl_egroup_h dest, kupl_egroup_h src);

参数

表 5-6 参数定义

参数名	类型	描述	输入/输出
dest	kupl_egroup_h	需要取用executor的egroup	输入/输出
src	kupl_egroup_h	给出executor的egroup	输入/输出

返回值

- 成功：返回取用executor后的dest egroup大小
- 失败：返回KUPL_ERROR

示例

```
#include <stdio.h>
#include "kupl.h"

int main()
{
    int executor_num = kupl_get_num_executors();
    int n1 = executor_num / 2;
    int n2 = executor_num - executor_num / 2;
    int executors1[n1], executors2[n2];
    for (int i =0; i < n1; i++) {
        executors1[i] = i;
    }
    for (int i =0; i < n2; i++) {
        executors2[i] = i + n1;
    }
    kupl_egroup_h egroup1 = kupl_egroup_create(executors1, n1);
    kupl_egroup_h egroup2 = kupl_egroup_create(executors2, n2);
    printf("egroup1 : %d executors\n", n1);
    n1 = kupl_egroup_borrow(egroup1, egroup2);
    printf("egroup1 : %d executors\n", n1);
    kupl_egroup_destroy(egroup1);
    kupl_egroup_destroy(egroup2);
    return 0;
}
```

运行结果如下。

```
egroup1 : 2 executors
egroup1 : 4 executors
```

 版权所有 © 华为技术有限公司

说明

上述示例演示了创建的egroup1取用egroup2中所有的executor的流程，运行结果打印了取用前与取用后egroup1中的executor数量，由此可见kupl_egroup_borrow函数使egroup1取用了egroup2的所有executors。

5.3.2.2.8 kupl_egroup_return

src egroup向dest egroup归还executors。

接口定义

int kupl_egroup_return(kupl_egroup_h dest, kupl_egroup_h src);

参数

参数名	类型	描述	输入/输出
dest	kupl_egroup_h	被归还executor的egroup	输入/输出
src	kupl_egroup_h	需要归还executor的egroup	输入/输出

返回值

- 成功：返回被归还executor后的dest egroup大小
- 失败：返回KUPL_ERROR

示例

```
#include <stdio.h>
#include "kupl.h"

int main()
{
    int executor_num = kupl_get_num_executors();
    int n1 = executor_num / 2;
    int n2 = executor_num - executor_num / 2;
    int executors1[n1], executors2[n2];
    for (int i =0; i < n1; i++) {
        executors1[i] = i;
    }
    for (int i =0; i < n2; i++) {
        executors2[i] = i + n1;
    }
    kupl_egroup_h egroup1 = kupl_egroup_create(executors1, n1);
    kupl_egroup_h egroup2 = kupl_egroup_create(executors2, n2);
    printf("egroup2 : %d executors\n", n2);
    n1 = kupl_egroup_borrow(egroup1, egroup2);
    n2 = kupl_egroup_return(egroup2, egroup1);
    printf("egroup2 : %d executors\n", n2);
    kupl_egroup_destroy(egroup1);
    kupl_egroup_destroy(egroup2);
    return 0;
}
```

运行结果如下。

 版权所有 © 华为技术有限公司

```
egroup2 : 2 executors
egroup2 : 4 executors
```

说明

上述示例演示了创建的egroup1取用egroup2中executor后，重新向egroup2归还所有executor的流程，运行结果打印了最开始与取用、归还后egroup2中的executor数量，最终kupl_egroup_return函数将egroup1中所有executor都归还给了egroup2。

5.3.2.2.9 kupl_egroup_reset

重置egroup至创建时的状态。

接口定义

void kupl_egroup_reset(kupl_egroup_h group);

参数

参数名	类型	描述	输入/输出
group	kupl_egroup_h	需要重置的egroup对象，egroup为nullptr时，默认barrier全局	输入/输出

示例

```
#include <stdio.h>
#include "kupl.h"

int main()
{
    int executor_num = kupl_get_num_executors();
    int n1 = executor_num/2;
    int n2 = executor_num - executor_num/2;
    int executors1[n1], executors2[n2];
    for (int i =0; i < n1; i++) {
        executors1[i] = i;
    }
    for (int i =0; i < n2; i++) {
        executors2[i] = i + n1;
    }
    kupl_egroup_h egroup1 = kupl_egroup_create(executors1, n1);
    kupl_egroup_h egroup2 = kupl_egroup_create(executors2, n2);
    n1 = kupl_egroup_borrow(egroup1, egroup2);
    kupl_egroup_reset(egroup1);
    kupl_egroup_reset(egroup2);
    kupl_egroup_destroy(egroup1);
    kupl_egroup_destroy(egroup2);
    return 0;
}
```

说明

上述示例演示了创建egroup1与egroup2后，通过kupl_egroup_borrow函数改变egroup1与egroup2，最终通过kupl_egroup_reset函数将egroup1与egroup2都重置为创建时的状态。

5.3.2.2.10 kupl_egroup_barrier

同步egroup中所有executor都到达该位置后，executor行为才继续执行。

版权所有 © 华为技术有限公司

接口定义

void kupl_egroup_barrier(kupl_egroup_h group);

参数

表 5-7 参数定义

参数名	类型	描述	输入/输出
group	kupl_egroup_h	执行barrier操作的egroup对象	输入

示例

```
#include <stdio.h>
#include "kupl.h"

static inline void task_int_loop(kupl_nd_range_t *nd_range, void *args, int tid, int tnum)
{
    printf("before barrier: tid %d \n", tid);
    kupl_egroup_barrier(nullptr);
    printf("after barrier: tid %d \n", tid);
}

int main()
{
    const int num_threads = kupl_get_num_executors();
    kupl_nd_range_t range;
    int lower = 0, upper = num_threads;
    KUPL_1D_RANGE_INIT(range, 0, num_threads);
    int executors[num_threads];
    for (int i = 0; i < num_threads; i++) {
        executors[i] = i;
    }
    kupl_egroup_h eg = kupl_egroup_create(executors, num_threads);
    kupl_parallel_for_desc_t desc = {
        .field_mask = KUPL_PARALLEL_FOR_DESC_FIELD_DEFAULT,
        .range = &range,
        .egroup = eg,
        .concurrency = num_threads,
        .policy = KUPL_LOOP_POLICY_STATIC
    };
    kupl_parallel_for(&desc, task_int_loop, nullptr);
    kupl_egroup_destroy(eg);
    return 0;
}
```

运行结果如下。

```
before barrier: tid 2
before barrier: tid 0
before barrier: tid 1
before barrier: tid 3
after barrier: tid 1
after barrier: tid 0
after barrier: tid 2
after barrier: tid 3
```


版权所有 © 华为技术有限公司

说明

上述示例演示了在omp并行区域中kupl_egroup_barrier函数的作用。根据运行结果，所有线程的before barrier打印完毕后，所有线程的after barrier才开始打印。由此，kupl_egroup_barrier函数此处的作用是同步egroup中所有executor都到达执行完毕before barrier的打印后，代码才继续执行。

5.3.2.2.11 kupl_egroup_fork_barrier

本接口提供一种非全局强制同步的同步语义，仅保证部分同步逻辑，具体表现在：

对于调用该接口的所有线程，主线程不阻塞，直接往下执行；子线程阻塞等待主线程到达后，才继续往下执行。

接口定义

void kupl_egroup_fork_barrier(kupl_egroup_h group);

参数

表 5-8 参数定义

参数名	类型	描述	输入/输出
group	kupl_egroup_h	执行fork barrier操作的egroup对象	输入

示例

```
#include <stdio.h>
#include "kupl.h"

static inline void task_int_loop(kupl_nd_range_t *nd_range, void *args, int tid, int tnum)
{
    printf("before barrier: tid %d \n", tid);
    kupl_egroup_join_barrier(nullptr);
    kupl_egroup_fork_barrier(nullptr);
    printf("after barrier: tid %d \n", tid);
}

int main()
{
    const int num_threads = kupl_get_num_executors();
    int executors[num_threads];
    for (int i = 0; i < num_threads; i++) {
        executors[i] = i;
    }
    kupl_egroup_h eg = kupl_egroup_create(executors, num_threads);
    kupl_parallel_for_desc_t desc = {
        .field_mask = KUPL_PARALLEL_FOR_DESC_FIELD_DEFAULT,
        .range = nullptr,
        .egroup = eg,
        .concurrency = num_threads,
        .policy = KUPL_LOOP_POLICY_STATIC
    };
    kupl_parallel_for(&desc, task_int_loop, nullptr);
    kupl_egroup_destroy(eg);
    return 0;
}
```

运行结果如下。

版权所有 © 华为技术有限公司

```
before barrier: tid 2
before barrier: tid 0
before barrier: tid 1
before barrier: tid 3
after barrier: tid 1
after barrier: tid 0
after barrier: tid 2
after barrier: tid 3
```

说明

上述示例演示了在并行区域中kupl_egroup_fork_barrier函数的作用。根据运行结果，所有线程的before barrier打印完毕后，所有线程的after barrier才开始打印。由此，kupl_egroup_fork_barrier函数的作用是主线程executor到达该位置后，子线程executor行为才继续执行；此处与kupl_egroup_join_barrier函数搭配使用，实现egroup中所有executor的同步。

5.3.2.2.12 kupl_egroup_join_barrier

本接口提供一种非全局强制同步的同步语义，仅保证部分同步逻辑，具体表现在：

对于调用该接口的所有线程，子线程不阻塞，直接往下执行；主线程阻塞等待所有子线程到达后，才继续往下执行。

接口定义

void kupl_egroup_join_barrier(kupl_egroup_h group);

参数

表 5-9 参数定义

参数名	类型	描述	输入/输出
group	kupl_egroup_h	执行join barrier操作的egroup对象	输入

示例

```
#include <stdio.h>
#include "kupl.h"

static inline void task_int_loop(kupl_nd_range_t *nd_range, void *args, int tid, int tnum)
{
    printf("before barrier: tid %d \n", tid);
    kupl_egroup_join_barrier(nullptr);
    kupl_egroup_fork_barrier(nullptr);
    printf("after barrier: tid %d \n", tid);
}

int main()
{
    const int num_threads = kupl_get_num_executors();
    int executors[num_threads];
    for (int i = 0; i < num_threads; i++) {
        executors[i] = i;
    }
    kupl_egroup_h eg = kupl_egroup_create(executors, num_threads);
    kupl_parallel_for_desc_t desc = {
        .field_mask = KUPL_PARALLEL_FOR_DESC_FIELD_DEFAULT,
```


版权所有 © 华为技术有限公司

```
        .range = nullptr,
        .egroup = eg,
        .concurrency = num_threads,
        .policy = KUPL_LOOP_POLICY_STATIC
    };
    kupl_parallel_for(&desc, task_int_loop, nullptr);
    kupl_egroup_destroy(eg);
    return 0;
}
```

运行结果如下。

```
before barrier: tid 2
before barrier: tid 0
before barrier: tid 1
before barrier: tid 3
after barrier: tid 1
after barrier: tid 0
after barrier: tid 2
after barrier: tid 3
```

说明

上述示例演示了在并行区域中kupl_egroup_join_barrier函数的作用。根据运行结果，所有线程的before barrier打印完毕后，所有线程的after barrier才开始打印。由此，kupl_egroup_join_barrier函数的作用是子线程executor都到达该位置后，主线程executor行为才继续执行；此处与kupl_egroup_fork_barrier函数搭配使用，实现egroup中所有executor的同步。

5.3.2.3 多线程编程函数

5.3.2.3.1 概念说明

多线程编程是通过parallel for等多线程编程函数，使得一个进程中可以并发多个线程，每个线程并行执行不同的任务，进而提升性能。KUPL库提供了支持并行的kupl_parallel_for 多线程编程函数。

具体的数据结构将在对应函数中进行详细说明。

5.3.2.3.2 kupl_parallel_for

创建parallel for并行循环。

接口定义

int kupl_parallel_for(kupl_parallel_for_desc_t *desc, kupl_pf_func_t func, void *args);

版权所有 © 华为技术有限公司

参数

表 5-10 参数定义

参数名	类型	描述	输入/输出
desc	kupl_parallel_for_desc_t *	parallel for循环任务的描述，指向kupl_parallel_for_desc_t结构体的指针，具体见下方kupl_parallel_for_desc_t数据结构表	输入
func	kupl_pf_func_t	parallel for循环任务的函数，该函数必须定义为如下形式：void (*kupl_pf_func_t)(kupl_nd_range_t *nd_range, void *args, int tid, int tnum)；其中args为该结构体传入的args参数，nd_range表示经过kupl_parallel_for内部处理后该函数实际执行的循环区域，tid与tnum表示当前函数执行所在线程编号与总线程数	输入
args	void *	func函数需要传入的参数	输入

表 5-11 kupl_parallel_for_desc_t 的数据结构定义

参数名	类型	描述
field_mask	uint64_t	结构体中有效字段的掩码，使用 kupl_parallel_for_desc_field 中的位标识。此掩码中未指定的字段将被忽略。当前所有字段都为必填项。

版权所有 © 华为技术有限公司

参数名	类型	描述
range	kupl_nd_range_t *	parallel for范围，指向kupl_nd_range_t结构体的指针，请参见表5-12
egroup	kupl_egroup_h	执行for循环任务的egroup，即能在哪个egroup中的executor执行器上执行；可设置为空指针，即不指定egroup
concurrency	int	for循环任务的并发度；可设置为 KUPL_CONCURRENCY_DEFAULT ，即不指定并发度
policy	kupl_loop_policy_type_t	parallel for任务遵循的切分策略，当前可设置为KUPL_LOOP_POLICY_STATIC，表示静态切分策略：平均切 KUPL_LOOP_POLICY_DYNAMIC，表示动态切分策略， KUPL_LOOP_POLICY_TASK，表示所有任务会被静态切分但会被以task形式提交用于动态调度

表 5-12 kupl_nd_range_t 的数据结构定义

参数名	类型	描述
dim	int	parallel for范围的维度，最大值为KUPL_MAX_DIM_SIZE，当前为3
nd_range	kupl_range_t[]	每个维度的具体范围，例如nd_range[0]表示维度0的范围；nd_range数组大小为KUPL_MAX_DIM_SIZE；kupl_range_t数据结构说明具体见下表

版权所有 © 华为技术有限公司

表 5-13 kupl_range_t 的数据结构定义

参数名	类型	描述
lower	int64_t	parallel for循环的起始值，即范围的下限
upper	int64_t	parallel for循环的结束值，即范围的上限
step	int64_t	parallel for循环的步长，即每次循环增大的值
blocksize	int64_t	任务拆分后，每次执行一个block，每个block包含的最小迭代次数是blocksize，可以使用KUPL_BLOCKSIZE_DEFAULT表示默认blocksize。当前静态切分策略下blocksize不生效，kupl会按线程数均分数据。只有动态切分和task切分策略下生效，blocksize默认值为1。

由于kupl_nd_range_t的数据结构较为复杂，因此在允许用户自行配置数据结构的同时，也提供了相应的宏，以供用户便捷地配置维度为1的kupl_nd_range_t。具体的宏如**表 kupl_nd_range_t的宏定义**所示。

注：大于一维的任务总数要保证小于int上限，即总blocks < 2^31 - 1

表 5-14 kupl_nd_range_t 的宏定义

宏	描述
KUPL_1D_RANGE_INIT(_range, _col_begin, _col_end)	配置维度为1、步长为1的parallel for范围 具体功能： 将_range的维度设置为1； 将(_range).nd_range[0]的下限，上限，步长，块大小分别设置为_col_begin、_col_end、1、KUPL_BLOCKSIZE_DEFAULT
KUPL_STRIDE_1D_RANGE_INIT(_range, _col_begin, _col_end, _col_step, _col_blocksize)	配置维度为1的parallel for范围 具体功能： 将_range的维度设置为1； 将(_range).nd_range[0]的下限，上限，步长，块大小分别设置为_col_begin、_col_end、_col_step、_col_blocksize

版权所有 © 华为技术有限公司

宏	描述
KUPL_2D_RANGE_INIT(_range, _row_begin, _row_end, _col_begin, _col_end)	配置维度为2、步长为1的parallel for范围 具体功能： 将_range的维度设置为2；将(_range).nd_range[0]的下限、上限、步长、块大小分别设置为_col_begin、_col_end、1、KUPL_BLOCKSIZE_DEFAULT；将(_range).nd_range[1]的上限、下限、步长、块大小分别设置为_row_begin、_row_end、1、KUPL_BLOCKSIZE_DEFAULT
KUPL_STRIDE_2D_RANGE_INIT(_range, _row_begin, _row_end, _row_step, _row_blocksize, _col_begin, _col_end, _col_step, _col_blocksize)	配置维度为2的parallel for范围 具体功能： 将_range的维度设置为2；将(_range).nd_range[0]的下限、上限、步长、块大小分别设置为_col_begin、_col_end、_col_step、_col_blocksize；将(_range).nd_range[1]的上限、下限、步长、块大小分别设置为_row_begin、_row_end、_row_step、_row_blocksize
KUPL_3D_RANGE_INIT(_range, _page_begin, _page_end, _row_begin, _row_end, _col_begin, _col_end)	配置维度为3、步长为1的parallel for范围 具体功能： 将_range的维度设置为3；将(_range).nd_range[0]的下限、上限、步长、块大小分别设置为_col_begin、_col_end、1、KUPL_BLOCKSIZE_DEFAULT；将(_range).nd_range[1]的下限、上限、步长、块大小分别设置为_row_begin、_row_end、1、KUPL_BLOCKSIZE_DEFAULT；将(_range).nd_range[2]的下限、上限、步长、块大小分别设置为_page_begin、_page_end、1、KUPL_BLOCKSIZE_DEFAULT

版权所有 © 华为技术有限公司

宏	描述
KUPL_STRIDE_3D_RANGE_INIT(_range, _page_begin, _page_end, _page_step, _page_blocksize, _row_begin, _row_end, _row_step, _row_blocksize, _col_begin, _col_end, _col_step, _col_blocksize)	配置维度为3的parallel for范围 具体功能： 将_range的维度设置为3；将(_range).nd_range[0]的下限、上限、步长、块大小分别设置为_col_begin、_col_end、_col_step、_col_blocksize; 将(_range).nd_range[1]的下限、上限、步长、块大小分别设置为_row_begin、_row_end、_row_step、_row_blocksize; 将(_range).nd_range[2]的下限、上限、步长、块大小分别设置为_page_begin、_page_end、_page_step、_page_blocksize

返回值

- 成功：返回KUPL_OK
- 失败：返回KUPL_ERROR

示例

一维示例：

```
#include <stdio.h>
#include <pthread.h>
#include "kupl.h"
static inline void task_in_loop(kupl_nd_range_t *nd_range, void *args, int tid, int tnum)
{
    for (int64_t i = nd_range->nd_range[0].lower; i < nd_range->nd_range[0].upper; i += nd_range-
>nd_range[0].step) {
        printf("pthread %lu: task_in_loop exe %d job\n", pthread_self(), i);
    }
}
int main()
{
    const int num_threads = kupl_get_num_executors();
    int count = num_threads;
    kupl_nd_range_t range;
    KUPL_1D_RANGE_INIT(range, 0, count);
    int executors[num_threads];
    for (int i = 0; i < num_threads; i++) {
        executors[i] = i;
    }
    kupl_egroup_h eg = kupl_egroup_create(executors, num_threads);
    kupl_parallel_for_desc_t desc = {
        .field_mask = KUPL_PARALLEL_FOR_DESC_FIELD_DEFAULT,
        .range = &range,
        .egroup = eg,
        .concurrency = num_threads,
        .policy = KUPL_LOOP_POLICY_STATIC
    };
    kupl_parallel_for(&desc, task_in_loop, nullptr);
    kupl_egroup_destroy(eg);
}
```

运行结果如下。

 版权所有 © 华为技术有限公司

```
pthread 281473368858656: task_in_loop exe 0 job
pthread 281473335971872: task_in_loop exe 2 job
pthread 281473344426016: task_in_loop exe 1 job
pthread 281473327517728: task_in_loop exe 3 job
```

三维示例：

```
#include <stdio.h>
#include <pthread.h>
#include "kupl.h"
static inline void task_in_loop(kupl_nd_range_t *nd_range, void *args, int tid, int tnum)
{
    for (int64_t i = nd_range->nd_range[0].lower; i < nd_range->nd_range[0].upper; i += nd_range->nd_range[0].step) {
        for (int64_t j = nd_range->nd_range[1].lower; j < nd_range->nd_range[1].upper; j += nd_range->nd_range[1].step) {
            for (int64_t k = nd_range->nd_range[2].lower; k < nd_range->nd_range[2].upper; k += nd_range->nd_range[2].step) {
                printf("pthread %lu: task_in_loop exe [%d : %d : %d] job\n", pthread_self(), i, j, k);
            }
        }
    }
}
int main()
{
    const int num_threads = kupl_get_num_executors();
    int count = num_threads;
    kupl_nd_range_t range;
    KUPL_3D_RANGE_INIT(range, 0, count, 0, count, 0, count);
    int executors[num_threads];
    for (int i = 0; i < num_threads; i++) {
        executors[i] = i;
    }
    kupl_egroup_h eg = kupl_egroup_create(executors, num_threads);
    kupl_parallel_for_desc_t desc = {
        .field_mask = KUPL_PARALLEL_FOR_DESC_FIELD_DEFAULT,
        .range = &range,
        .egroup = eg,
        .concurrency = num_threads,
        .policy = KUPL_LOOP_POLICY_STATIC
    };
    kupl_parallel_for(&desc, task_in_loop, nullptr);
    kupl_egroup_destroy(eg);
}
```

运行结果如下。

```
pthread 281472966852640: task_in_loop exe [0 : 0 : 0] job
pthread 281472966852640: task_in_loop exe [0 : 0 : 1] job
pthread 281472966852640: task_in_loop exe [0 : 1 : 0] job
pthread 281472966852640: task_in_loop exe [0 : 1 : 1] job
pthread 281472950931488: task_in_loop exe [1 : 0 : 0] job
pthread 281472950931488: task_in_loop exe [1 : 0 : 1] job
pthread 281472950931488: task_in_loop exe [1 : 1 : 0] job
pthread 281472950931488: task_in_loop exe [1 : 1 : 1] job
```

说明

上述示例演示了KUPL执行parallel for循环任务的流程。上述示例中，首先通过KUPL_1D_RANGE_INIT宏配置了parallel for范围，定义了1维的step步长为1，范围从0到count的for循环描述；其次配置了parallel for任务描述，任务的函数为task_int_loop，参数为空，并发度为执行器数量、使用所有执行器；最终通过kupl_parallel_for函数执行for循环。注：以上运行结果以实际为准，上述结果仅供参考。

5.3.2.3.3 kupl::parallel_for

创建parallel for并行循环。相较于5.3.2.3.2 kupl_parallel_for接口而言，该接口通过lambda函数特性捕获parallel for回调函数的入参， 避免用户入参封装行为，提高接口易用性。

接口定义

kupl::parallel_for(kupl_parallel_for_desc_t *desc, const pf_lambda &func);

类型定义

using pf_lambda = std::function<void(const kupl_nd_range_t *nd_range, const int tid, const int tnum)>;

表 5-15 参数定义

参数名	类型	描述	输入/输出
desc	kupl_parallel_for_desc_t *	parallel for循环任务的描述，指向kupl_parallel_for_desc_t结构体的指针，具体见下方kupl_parallel_for_desc_t数据结构表	输入
func	pf_lambda	parallel for循环任务的函数，该函数必须定义为如下形式：std::function<void(const kupl_nd_range_t *nd_range, const int tid, const int tnum)>；其中nd_range表示经过kupl_parallel_for内部处理后该函数实际执行的循环区域，tid与tnum表示当前函数执行所在线程编号与总线程数	输入

版权所有 © 华为技术有限公司

表 5-16 kupl_parallel_for_desc_t 的数据结构定义

参数名	类型	描述
field_mask	uint64_t	结构体中有效字段的掩码，使用kupl_parallel_for_desc_field中的位标识。此掩码中未指定的字段将被忽略。当前所有字段都为必填项，可以使用KUPL_PARALLEL_FOR_DESC_FIELD_DEFAULT表示所有字段都生效。 具体可设置的掩码： ● KUPL_PARALLEL_FOR_DESC_FIELD_RANGE：range生效 ● KUPL_PARALLEL_FOR_DESC_FIELD_EGROUP：egroup生效 ● KUPL_PARALLEL_FOR_DESC_FIELD_CONCURRENCY：concurrency生效 ● KUPL_PARALLEL_FOR_DESC_FIELD_POLICY: policy生效 ● KUPL_PARALLEL_FOR_DESC_FIELD_DEFAULT:上述所有字段都生效
range	kupl_nd_range_t *	parallel for范围，指向kupl_nd_range_t结构体的指针，具体见下方kupl_nd_range_t数据结构表。
egroup	kupl_egroup_h	执行for循环任务的egroup，即能在哪个egroup中的executor执行器上执行；可设置为空指针，即不指定egroup。
concurrency	int	for循环任务的并发度；可设置为KUPL_CONCURRENCY_DEFAULT，即不指定并发度。
policy	kupl_loop_policy_type_t	parallel for任务遵循的切分策略，当前可设置为 ● KUPL_LOOP_POLICY_STATIC，表示静态切分策略：平均切。 ● KUPL_LOOP_POLICY_DYNAMIC，表示动态切分策略。 ● KUPL_LOOP_POLICY_TASK，表示所有任务会被静态切分但会被以task形式提交用于动态调度。

表 5-17 kupl_nd_range_t 的数据结构定义

参数名	类型	描述
dim	int	parallel for范围的维度，最大值为KUPL_MAX_DIM_SIZE，当前为3

版权所有 © 华为技术有限公司

参数名	类型	描述
nd_range	kupl_range_t[]	每个维度的具体范围，例如nd_range[0]表示维度0的范围；nd_range数组大小为KUPL_MAX_DIM_SIZE；kupl_range_t数据结构说明具体见下表

表 5-18 kupl_nd_range_t 的数据结构定义

参数名	类型	描述
dim	int	parallel for范围的维度，最大值为KUPL_MAX_DIM_SIZE，当前为3
nd_range	kupl_range_t[]	每个维度的具体范围，例如nd_range[0]表示维度0的范围；nd_range数组大小为KUPL_MAX_DIM_SIZE；kupl_range_t数据结构说明具体见下表

表 5-19 kupl_range_t 的数据结构定义

参数名	类型	描述
lower	int64_t	parallel for循环的起始值，即范围的下限
upper	int64_t	parallel for循环的结束值，即范围的上限
step	int64_t	parallel for循环的步长，即每次循环增大的值

版权所有 © 华为技术有限公司

参数名	类型	描述
blocksize	int64_t	。任务拆分后，每次执行一个block，每个block包含的最小迭代次数是blocksize，可以使用KUPL_BLOCKSIZE_DEFAULT表示默认blocksize。当前静态切分策略下blocksize不生效，kupl会按线程数均分数据。只有动态切分和task切分策略下生效，blocksize默认值为1。

由于kupl_nd_range_t的数据结构较为复杂，因此在允许用户自行配置数据结构的同时，也提供了相应的宏，以供用户便捷地配置维度为1的kupl_nd_range_t。具体的宏如**表 kupl_nd_range_t的宏定义**所示。

注：大于一维的任务总数要保证小于int上限，即总blocks < 2^31 - 1

表 5-20 kupl_nd_range_t 的宏定义

宏	描述
KUPL_1D_RANGE_INIT(_range, _col_begin, _col_end)	配置维度为1、步长为1的parallel for范围 具体功能： 将_range的维度设置为1； 将(_range).nd_range[0]的下限，上限，步长，块大小分别设置为_col_begin、_col_end、1、KUPL_BLOCKSIZE_DEFAULT
KUPL_STRIDE_1D_RANGE_INIT(_range, _col_begin, _col_end, _col_step, _col_blocksize)	配置维度为1的parallel for范围 具体功能： 将_range的维度设置为1； 将(_range).nd_range[0]的下限，上限，步长，块大小分别设置为_col_begin、_col_end、_col_step、_col_blocksize

版权所有 © 华为技术有限公司

宏	描述
KUPL_2D_RANGE_INIT(_range, _row_begin, _row_end, _col_begin, _col_end)	配置维度为2、步长为1的parallel for范围 具体功能： 将_range的维度设置为2；将(_range).nd_range[0]的下限、上限、步长、块大小分别设置为_col_begin、_col_end、1、KUPL_BLOCKSIZE_DEFAULT；将(_range).nd_range[1]的上限、下限、步长、块大小分别设置为_row_begin、_row_end、1、KUPL_BLOCKSIZE_DEFAULT
KUPL_STRIDE_2D_RANGE_INIT(_range, _row_begin, _row_end, _row_step, _row_blocksize, _col_begin, _col_end, _col_step, _col_blocksize)	配置维度为2的parallel for范围 具体功能： 将_range的维度设置为2；将(_range).nd_range[0]的下限、上限、步长、块大小分别设置为_col_begin、_col_end、_col_step、_col_blocksize；将(_range).nd_range[1]的上限、下限、步长、块大小分别设置为_row_begin、_row_end、_row_step、_row_blocksize
KUPL_3D_RANGE_INIT(_range, _page_begin, _page_end, _row_begin, _row_end, _col_begin, _col_end)	配置维度为3、步长为1的parallel for范围 具体功能： 将_range的维度设置为3；将(_range).nd_range[0]的下限、上限、步长、块大小分别设置为_col_begin、_col_end、1、KUPL_BLOCKSIZE_DEFAULT；将(_range).nd_range[1]的下限、上限、步长、块大小分别设置为_row_begin、_row_end、1、KUPL_BLOCKSIZE_DEFAULT；将(_range).nd_range[2]的下限、上限、步长、块大小分别设置为_page_begin、_page_end、1、KUPL_BLOCKSIZE_DEFAULT

版权所有 © 华为技术有限公司

宏	描述
KUPL_STRIDE_3D_RANGE_INIT(_range, _page_begin, _page_end, _page_step, _page_blocksize, _row_begin, _row_end, _row_step, _row_blocksize, _col_begin, _col_end, _col_step, _col_blocksize)	配置维度为3的parallel for范围 具体功能： 将_range的维度设置为3；将(_range).nd_range[0]的下限、上限、步长、块大小分别设置为_col_begin、_col_end、_col_step、_col_blocksize;将(_range).nd_range[1]的下限、上限、步长、块大小分别设置为_row_begin、_row_end、_row_step、_row_blocksize;将(_range).nd_range[2]的下限、上限、步长、块大小分别设置为_page_begin、_page_end、_page_step、_page_blocksize

返回值

- 成功：返回KUPL_OK
- 失败：返回KUPL_ERROR

示例

```
#include <atomic>
#include <assert.h>
#include "kupl.h"

int main()
{
    int count = 10;
    kupl_nd_range_t range;
    KUPL_1D_RANGE_INIT(range, 0, count);
    kupl_parallel_for_desc_t desc = {
        .field_mask = KUPL_PARALLEL_FOR_DESC_FIELD_DEFAULT,
        .range = &range,
        .egroup = nullptr,
        .concurrency = count,
        .policy = KUPL_LOOP_POLICY_STATIC,
    };

    std::atomic<size_t> sum(0);
    const size_t loop_size = 1000000;
    size_t sum_cal = (1 + loop_size) * loop_size / 2;
    size_t *data = (size_t *)malloc(loop_size * sizeof(loop_size));
    for (size_t i = 0; i < loop_size; i++) {
        data[i] = i + 1;
    }

    int ret = kupl::parallel_for(&desc, [&](const kupl_nd_range_t *nd_range, int tid, int tnum) {
        int start_index = nd_range->nd_range[0].lower * (loop_size / count);
        size_t local_sum = 0;
        for (size_t i = 0; i < loop_size / count; i++) {
            local_sum += data[start_index + i];
        }
        sum += local_sum;
    });
    assert(ret == KUPL_OK);
    assert(sum.load() == sum_cal);
    printf("sum: %lu\n", sum.load());
}
```


版权所有 © 华为技术有限公司

运行结果如下。

```
sum: 500000500000
```

说明

上述示例演示了使用kupl::parallel_for并行计算1-1000000的和。首先通过KUPL_1D_RANGE_INIT宏配置了parallel for范围，定义了1维的step步长为1，范围从0到count的for循环描述；其次配置了parallel for任务描述，并发度为count、使用所有执行器，使用静态切分策略；最终通过kupl_parallel_for函数执行for循环。注：以上运行结果以实际为准，上述结果仅供参考。

5.3.2.3.4 kupl_parallel_for_reduce

执行并行归约

接口定义

int kupl_parallel_for_reduce(kupl_parallel_for_desc_t *desc, kupl_pf_reduce_func_t func, void *args,

kupl_reduce_args_t *rd_args);

参数

表 5-21 参数定义

参数名	类型	描述	输入/输出
desc	kupl_parallel_for_desc_t *	并行迭代任务的描述，指向kupl_parallel_for_desc_t结构体的指针，具体参考kupl_parallel_for章节的定义	输入

版权所有 © 华为技术有限公司

参数名	类型	描述	输入/输出
func	kupl_pf_reduce_func_t	并行规约的函数，该函数必须定义为如下形式：void (*kupl_pf_reduce_func_t) (kupl_nd_range_t *nd_range, void *args, int tid, int tnum, kupl_reduce_args_t *rd_args)；其中nd_range表示经过kupl_parallel_for内部处理后该函数实际执行的循环区域，args为该结构体传入的args参数，tid与tnum表示当前函数执行所在线程编号与总线程数，rd_args 表示规约参数的线程副本，具体定义参考下方 kupl_reduce_args_t的定义	输入
args	void *	func函数需要传入的参数	输入
rd_args	kupl_reduce_args_t *	规约参数，定义了总的规约操作数，具体规约操作的数据类型，规约操作类型，以及规约数据地址。	输入输出

表 5-22 kupl_reduce_args_t 的数据结构定义

参数名	类型	描述
num	int	规约操作的数量
items	kupl_reduce_item_t *	指向具体规约操作定义的指针，详细定义请参见下方

版权所有 © 华为技术有限公司

表 5-23 kupl_reduce_item_t 的数据结构定义

参数名	类型	描述
buffer	void *	规约操作的数据地址
type	kupl_datatype_t	规约操作的数据类型，包括： KUPL_DATATYPE_INT, KUPL_DATATYPE_FLOAT, KUPL_DATATYPE_DOUBLE, KUPL_DATATYPE_FLOAT_COMPLEX, KUPL_DATATYPE_DOUBLE_COMPLEX
op	kupl_reduce_op_t	规约操作的操作类型，包括： KUPL_RD_ADD, KUPL_RD_SUB, KUPL_RD_MAX, KUPL_RD_MIN

返回值

- 成功：返回KUPL_OK
- 失败：返回KUPL_ERROR

示例

一维示例：

```
#include <stdio.h>
#include <pthread.h>
#include "kupl.h"
static inline void task_in_loop_reduce(kupl_nd_range_t *nd_range, void *args, int tid, int tnum,
kupl_reduce_args_t *rd_args)
{
    int *data = (int *)args;
    int localsum = 0;
    for (int i = nd_range->nd_range[0].lower; i < nd_range->nd_range[0].upper; i += nd_range-
>nd_range[0].step) {
        localsum += data[i];
    }
    *(int *)rd_args->items[0].buffer += localsum;
}

int main()
{
    const int n = 100;
    int data[n];
    for (int i = 0; i < n; i++) {
        data[i] = i + 1;
    }
    kupl_nd_range_t range;
    KUPL_1D_RANGE_INIT(range, 0, n);


    kupl_parallel_for_desc_t desc = {
        .field_mask = KUPL_PARALLEL_FOR_DESC_FIELD_DEFAULT,
        .range = &range,
```

 版权所有 © 华为技术有限公司

```
        .egroup = NULL,
        .concurrency = 4,
        .policy = KUPL_LOOP_POLICY_STATIC
    };
    int sum_int = 0;
    kupl_reduce_item_t param_int[1] = {{ .buffer = &sum_int, .type = KUPL_DATATYPE_INT, .op =
KUPL_RD_ADD }};
    kupl_reduce_args_t rd_args_int = { .num = 1, .items = param_int };
    kupl_parallel_for_reduce(&desc, task_in_loop_reduce, data, &rd_args_int);
    printf("sum: %d\n", sum_int);
}
```

运行结果如下。

```
sum: 5050
```

说明

上述示例演示了KUPL执行并行规约的流程，计算了数组 data 中所有数值1-100求和的值。

5.3.2.3.5 kupl_in_parallel

判断当前是否在并行域内。

接口定义

bool kupl_in_parallel();

返回值

- 在并行域内：返回true
- 不在并行域内：返回false

示例

```
#include <stdio.h>
#include <assert.h>
#include "kupl.h"

static void task_int_parallel(kupl_nd_range_t *nd_range, void *args, int tid, int tnum)
{
    bool in_parallel = kupl_in_parallel();
    assert(in_parallel == true);
}

int main()
{
    kupl_parallel_for_desc_t desc = {
        .field_mask = KUPL_PARALLEL_FOR_DESC_FIELD_DEFAULT,
        .range = nullptr,
        .egroup = nullptr,
        .concurrency = 4,
        .policy = KUPL_LOOP_POLICY_STATIC
    };
    kupl_parallel_for(&desc, task_int_parallel, nullptr);
}
```

说明

- 上述示例演示了开启并行域并判断当前是否在并行域内的流程。
- 上述在并行域内调用kupl_in_parallel函数，因此得到true的结果。

 版权所有 © 华为技术有限公司

5.3.2.4 计算图编程函数

5.3.2.4.1 概念说明

使用计算图编程首先需要了解图相关的概念，其中graph，即动态图；task，即添加到动态图中的需要执行的任务；sgraph，即静态图；sgraph node，即静态图节点，通常一个静态图节点对应一个任务，静态图节点之间可以定义依赖关系。

具体地说，动态图和静态图的主要区别在于是否可复用，往动态图中添加task任务，KUPL底层executor会基于sched动态执行task任务实现尽可能的负载均衡效果；而静态图则是一张可复用的图，可以通过向其中添加sgraph node并添加sgraph node之间的依赖关系定义一张静态图，将静态图提交给动态图后静态图中的节点任务才能够被KUPL执行，且执行完后静态图不会被销毁。

另外， kupl_graph_h表示graph数据结构的句柄，kupl_task_h表示task数据结构的句柄，kupl_sgraph_h表示static graph数据结构的句柄，kupl_sgraph_node_h表示sgraph node数据结构的句柄。

5.3.2.4.2 kupl_graph_create

创建一个kupl动态图。通过向kupl动态图中添加task任务，可实现多任务并行执行效果，从而发挥鲲鹏硬件多核能力。

接口定义

kupl_graph_h kupl_graph_create(kupl_egroup_h egroup);

参数

表 5-24 参数定义

参数名	类型	描述	输入/输出
egroup	kupl_egroup_h	该graph使用的executor的集合，可设置为KUPL_ALL_EXECUTORS，即使用全部可用的executors	输入

返回值

- 成功：返回创建的kupl图
- 失败：返回nullptr

示例

```
#include <stdio.h>
#include "kupl.h"

int main()
{
    int executor_num = kupl_get_num_executors();
    int executors[executor_num];
```


版权所有 © 华为技术有限公司

```
    for (int i =0; i < executor_num; i++) {
        executors[i] = i;
    }
    kupl_egroup_h egroup = kupl_egroup_create(executors, executor_num);
    kupl_graph_h graph = kupl_graph_create(egroup);
    kupl_graph_destroy(graph);
    kupl_egroup_destroy(egroup);
    return 0;
}
```

说明

上述示例演示了创建egroup、graph图并最后销毁的流程。上述kupl_graph_create函数创建了使用全部executor的graph图。

5.3.2.4.3 kupl_graph_destroy

销毁一个kupl动态图。

接口定义

void kupl_graph_destroy(kupl_graph_h graph);

参数

表 5-25 参数定义

参数名	类型	描述	输入/输出
graph	kupl_graph_h	需要销毁的kupl graph	输入

示例

```
#include <stdio.h>
#include "kupl.h"

int main()
{
    int executor_num = kupl_get_num_executors();
    int executors[executor_num];
    for (int i =0; i < executor_num; i++) {
        executors[i] = i;
    }
    kupl_egroup_h egroup = kupl_egroup_create(executors, executor_num);
    kupl_graph_h graph = kupl_graph_create(egroup);
    kupl_graph_destroy(graph);
    kupl_egroup_destroy(egroup);
    return 0;
}
```

说明

上述示例演示了创建egroup、graph图并最后销毁的流程。上述kupl_graph_destroy函数销毁了之前创建的graph图。

5.3.2.4.4 kupl_graph_submit

向kupl图中添加task任务，从而供KUPL底层executor获取任务进行执行。

版权所有 © 华为技术有限公司

接口定义

int kupl_graph_submit(kupl_graph_h graph, kupl_task_info_t *info);

参数

表 5-26 参数定义

参数名	类型	描述	输入/输出
graph	kupl_graph_h	需要向其中添加task的kupl graph	输入
info	kupl_task_info_t *	需要添加的task的信息，具体见下表	输入

表 5-27 kupl_task_info_t 的数据结构

参数名	类型	描述
type	kupl_task_type_t	task 的具体类型，可以设置为： ● KUPL_TASK_TYPE_SINGLE：动态图 task ● KUPL_TASK_TYPE_SGRAPH：静态图 task ● KUPL_TASK_TYPE_TASKLOOP: taskloop
desc	void *	task 类型所对应的描述，可以设置为： ● kupl_task_desc_t *：动态图 task 对应的描述，具体见下表 ● kupl_sgraph_task_desc_t *：静态图 task 对应的描述，具体见下表 ● kupl_taskloop_desc_t *: taskloop 对应的描述，具体见下表

版权所有 © 华为技术有限公司

表 5-28 kupl_task_desc_t 的数据结构

参数名	类型	描述
field_mask	uint32_t	掩码，用于指定结构体中哪些值有效，不设置掩码情况下仅func、args参数生效，其他需要设置掩码。 具体可设置的掩码： ● KUPL_TASK_DESC_FIELD_PRIORITY：priority生效 ● KUPL_TASK_DESC_FIELD_DEP：dep生效 ● KUPL_TASK_DESC_FIELD_NAME：name生效 ● KUPL_TASK_DESC_FIELD_FLAG：flag生效
func	void (*kupl_task_func_t)(void *args)	task的函数，task任务需要执行的具体函数
args	void *	func函数需要传入的参数
name	const char *	需要配置掩码KUPL_TASK_DESC_FIELD_NAME后才生效；task任务的名字
priority	int	需要配置掩码KUPL_TASK_DESC_FIELD_PRIORITY后才生效；task任务的优先级，数值越大，优先级越高
ndep	size_t	需要配置掩码KUPL_TASK_DESC_FIELD_DEP后才生效；动态图task中dep_list中的depends数量，即任务传入的用于任务间依赖的参数数量
dep_list	kupl_task_dep_t *	需要配置掩码KUPL_TASK_DESC_FIELD_DEP后才生效；动态图task中的depends 的列表，即任务传入的用于任务间依赖的参数列表

版权所有 © 华为技术有限公司

参数名	类型	描述
flag	uint32_t	需要配置掩码KUPL_TASK_DESC_FIELD_FLAG后才生效；task任务的flag，可设置为KUPL_TASK_FLAG_IMM，表示直接执行task，不需要调度

表 5-29 kupl_task_dep_t 的数据结构

参数名	类型	描述
base_addr	const void *	dep参数的地址
type	kupl_task_dep_type_t	dep的具体类型，可以设置为： KUPL_TASK_DEP_TYPE_IN：作为任务的输入参数 KUPL_TASK_DEP_TYPE_OUT：作为任务的输出参数 KUPL_TASK_DEP_TYPE_INOUT：作为任务的输入输出参数 KUPL_TASK_DEP_TYPE_ALL：该参数依赖之前的所有任务

表 5-30 kupl_sgraph_task_desc_t 的数据结构

参数名	类型	描述
sgraph	kupl_sgraph_h	需要提交的静态图
name	const char *	需要配置掩码KUPL_SGRAPH_TASK_DESC_FIELD_NAME后才能生效；静态图task的名字
priority	int	需要配置掩码KUPL_SGRAPH_TASK_DESC_FIELD_PRIORITY后才能生效；静态图task的优先级，数值越大，优先级越高

版权所有 © 华为技术有限公司

参数名	类型	描述
flag	uint32_t	需要配置掩码KUPL_SGRAPH_TASK_DESC_FIELD_FLAG后才能生效；静态图task的flag，可设置为KUPL_SGRAPH_TASK_FLAG_IMM，表示直接执行静态图任务，不需要调度
field_mask	uint64_t	掩码，用于指定结构体中哪些值有效，不设置掩码情况下仅sgraph参数生效，其他需要设置掩码。 具体可设置的掩码： ● KUPL_SGRAPH_TASK_DESC_FIELD_NAME：name生效 ● KUPL_SGRAPH_TASK_DESC_FIELD_PRIORITY：priority生效 ● KUPL_SGRAPH_TASK_DESC_FIELD_FLAG：flag生效

表 5-31 kupl_taskloop_desc_t 的数据结构

参数名	类型	描述
field_mask	uint64_t	结构体中有效字段的掩码，使用 kupl_taskloop_desc_field 中的位标识。此掩码中未指定的字段将被忽略。当前所有字段都为必填项。 具体可设置的掩码： ● KUPL_TASKLOOP_DESC_FIELD_RANGE：range生效 ● KUPL_TASKLOOP_DESC_FIELD_EGROUP：egroup生效 ● KUPL_TASKLOOP_DESC_FIELD_DEFAULT：上述字段都生效

版权所有 © 华为技术有限公司

参数名	类型	描述
func	void (*kupl_taskloop_func_t)(kupl_nd_range_t *nd_range, void *args)	taskloop 任务需要执行的具体函数
args	void *	func 函数需要传入的参数
range	kupl_nd_range_t *	taskloop 范围，详细使用方式见 kupl_parallel_for 章节
egroup	kupl_egroup_h	执行 taskloop 任务的 egroup，即表示该任务可在指定的 egroup 中的 executor 执行器上执行

返回值

- 成功：返回KUPL_OK
- 失败：返回KUPL_ERROR

示例

```
#include <stdio.h>
#include "kupl.h"
#include <assert.h>

static inline void task_str(void *args)
{
    printf("graph task test.\n");
}

int main()
{
    int executor_num = kupl_get_num_executors();
    int executors[executor_num];
    for (int i = 0; i < executor_num; i++) {
        executors[i] = i;
    }
    kupl_egroup_h egroup = kupl_egroup_create(executors, executor_num);
    kupl_graph_h graph = kupl_graph_create(egroup);
    kupl_task_desc_t task_desc = {
        .field_mask = KUPL_TASK_DESC_FIELD_FLAG,
        .func = task_str,
        .args = NULL,
        .flag = KUPL_TASK_FLAG_IMM,
    };
    kupl_task_info_t info = {
        .type = KUPL_TASK_TYPE_SINGLE,
        .desc = &task_desc,
    };

    int ret = kupl_graph_submit(graph, &info);
    assert(ret == KUPL_OK);

    kupl_graph_wait(graph);
    kupl_graph_destroy(graph);
    kupl_egroup_destroy(egroup);
```


版权所有 © 华为技术有限公司

```
    return 0;
}
```

运行结果如下。

```
graph task test.
```

说明

上述示例演示了向graph图提交task的流程。上述kupl_graph_submit函数提交的task的函数为task_str，函数不需要参数，task设置了flag生效且flag值设置为KUPL_TASK_FLAG_IMM；不需要获取特定的task。

5.3.2.4.5 kupl::graph_submit(task)

向kupl图中添加动态图task，即提交执行task任务。

相较于5.3.2.4.4 kupl_graph_submit接口而言，该接口通过lambda函数特性捕获kupl_graph_submit提交任务回调函数的入参， 避免用户入参封装行为，提高接口易用性。

接口定义

int kupl::graph_submit(kupl_graph_h graph, kupl_task_desc_t *desc, const std::function<void(void)> &func);

参数

表 5-32 参数定义

参数名	类型	描述	输入/输出
graph	kupl_graph_h	需要向其中添加动态图task的kupl graph	输入
desc	kupl_task_desc_t *	需要添加的动态图task的描述，具体见下表	输入
func	std::function<void(void)>	task的函数，task任务需要执行的具体函数	输入

版权所有 © 华为技术有限公司

表 5-33 kupl_task_desc_t 的数据结构

参数名	类型	描述
field_mask	uint64_t	掩码，用于指定结构体中哪些值有效，参数生效需要设置掩码。 具体可设置的掩码： ● KUPL_TASK_DESC_FIELD_NAME：name生效 ● KUPL_TASK_DESC_FIELD_DEP：dep生效 ● KUPL_TASK_DESC_FIELD_PRIORITY：priority生效 ● KUPL_TASK_DESC_FIELD_FLAG：flag生效
name	const char *	需要配置掩码KUPL_TASK_DESC_FIELD_NAME后才生效；task任务的名字
priority	int	需要配置掩码KUPL_TASK_DESC_FIELD_PRIORITY后才生效；task任务的优先级，数值越大，优先级越高
ndep	size_t	需要配置掩码KUPL_TASK_DESC_FIELD_DEP后才生效；动态图task中dep_list中的depends数量，即任务传入的用于任务间依赖的参数数量
dep_list	kupl_task_dep_t *	需要配置掩码KUPL_TASK_DESC_FIELD_DEP后才生效；动态图task中的depends 的列表，即任务传入的用于任务间依赖的参数列表
flag	uint32_t	需要配置掩码KUPL_TASK_DESC_FIELD_FLAG后才生效；task任务的flag，可设置为KUPL_TASK_FLAG_IMM，表示直接执行task，不需要调度

版权所有 © 华为技术有限公司

表 5-34 kupl_task_dep_t 的数据结构

参数名	类型	描述
base_addr	const void *	dep参数的地址
type	kupl_task_dep_type_t	dep的具体类型，可以设置为： KUPL_TASK_DEP_TYPE_IN：作为任务的输入参数 KUPL_TASK_DEP_TYPE_OUT：作为任务的输出参数 KUPL_TASK_DEP_TYPE_INOUT：作为任务的输入输出参数 KUPL_TASK_DEP_TYPE_ALL：该参数依赖之前的所有任务

返回值

- 成功：返回KUPL_OK
- 失败：返回KUPL_ERROR

示例

```
#include <stdio.h>
#include "kupl.h"
#include <assert.h>

int main()
{
    int executor_num = kupl_get_num_executors();
    int executors[executor_num];
    for (int i = 0; i < executor_num; i++) {
        executors[i] = i;
    }
    kupl_egroup_h egroup = kupl_egroup_create(executors, executor_num);
    kupl_graph_h graph = kupl_graph_create(egroup);
    kupl_task_desc_t task_desc = {
        .field_mask = KUPL_TASK_DESC_FIELD_FLAG,
        .flag = KUPL_TASK_FLAG_IMM,
    };
    int ret = kupl::graph_submit(graph, &task_desc, []() {
        printf("graph task test\n");
    });
    assert(ret == KUPL_OK);

    kupl_graph_wait(graph);
    kupl_graph_destroy(graph);
    kupl_egroup_destroy(egroup);
    return 0;
}
```

运行结果如下。

```
graph task test
```

说明

上述示例演示了向graph图提交task的流程。上述kupl::graph_submit函数提交task的函数，打印对应信息。

版权所有 © 华为技术有限公司

5.3.2.4.6 kupl::graph_submit(taskloop)

向kupl图中添加taskloop，即提交执行taskloop任务。

相较于**5.3.2.4.4 kupl_graph_submit**接口而言，该接口通过lambda函数特性捕获kupl_graph_submit提交任务回调函数的入参， 避免用户入参封装行为，提高接口易用性。

相较于**5.3.2.4.5 kupl::graph_submit(task)**接口而言，该接口提交任务为taskloop类型。两者的差异主要在函数接口的入参上，采用C++函数重载的能力进行实现。

接口定义

int kupl::graph_submit(kupl_graph_h graph, kupl_taskloop_desc_t* desc, const std::function<void(const kupl_nd_range_t *)> &func);

参数

表 5-35 参数定义

参数名	类型	描述	输入/输出
graph	kupl_graph_h	需要向其中添加taskloop的kupl graph	输入
desc	kupl_taskloop_desc_t *	需要添加的taskloop的描述，具体见下表	输入
func	std::function<void(const kupl_nd_range_t *)>	taskloop的函数，taskloop任务需要执行的具体函数	输入

版权所有 © 华为技术有限公司

表 5-36 kupl_taskloop_desc_t 的数据结构

参数名	类型	描述
field_mask	uint64_t	结构体中有效字段的掩码，使用 kupl_taskloop_desc_field 中的位标识。此掩码中未指定的字段将被忽略。当前所有字段都为必填项。 具体可设置的掩码： ● KUPL_TASKLOOP_DESC_FIELD_RANGE：range生效 ● KUPL_TASKLOOP_DESC_FIELD_EGROUP：egroup生效 ● KUPL_TASKLOOP_DESC_FIELD_DEFAULT：上述字段都生效
range	kupl_nd_range_t *	taskloop 范围，详细使用方式见 kupl_parallel_for 章节
egroup	kupl_egroup_h	执行 taskloop 任务的 egroup，即能在哪个 egroup 中的 executor 执行器上执行

返回值

- 成功：返回KUPL_OK
- 失败：返回KUPL_ERROR

示例

```
#include <stdio.h>
#include "kupl.h"
#include <assert.h>

int main()
{
    int executor_num = kupl_get_num_executors();
    int executors[executor_num];
    for (int i = 0; i < executor_num; i++) {
        executors[i] = i;
    }
    kupl_egroup_h egroup = kupl_egroup_create(executors, executor_num);
    kupl_graph_h graph = kupl_graph_create(egroup);
    kupl_nd_range_t range;
    KUPL_1D_RANGE_INIT(range, 0, executor_num);
    kupl_taskloop_desc_t taskloop_desc = {
        .field_mask = KUPL_TASKLOOP_DESC_FIELD_DEFAULT,
        .range = &range,
```


版权所有 © 华为技术有限公司

```
    .egroup = egroup
};
int ret = kupl::graph_submit(graph, &taskloop_desc, [](const kupl_nd_range_t *nd_range) {
    printf("graph taskloop test\n");
});
assert(ret == KUPL_OK);

kupl_graph_wait(graph);
kupl_egroup_destroy(egroup);
return 0;
}
```

运行结果如下。

```
graph taskloop test.
graph taskloop test.
graph taskloop test.
graph taskloop test.
```

说明

上述示例演示了向graph图提交taskloop的流程。

5.3.2.4.7 kupl_graph_wait

等待直到kupl图中的所有task都完成。

接口定义

void kupl_graph_wait(kupl_graph_h graph);

参数

表 5-37 参数定义

参数名	类型	描述	输入/输出
graph	kupl_graph_h	需要等待执行其中task的kupl graph	输入

示例

```
#include <stdio.h>
#include "kupl.h"
#include <assert.h>

static inline void task_str(void *args)
{
    printf("graph task test.\n");
}

int main()
{
    int executor_num = kupl_get_num_executors();
    int executors[executor_num];
    for (int i = 0; i < executor_num; i++) {
        executors[i] = i;
    }
    kupl_egroup_h egroup = kupl_egroup_create(executors, executor_num);
    kupl_graph_h graph = kupl_graph_create(egroup);
    kupl_task_desc_t task_desc = {
```


版权所有 © 华为技术有限公司

```
    .field_mask = KUPL_TASK_DESC_FIELD_FLAG,
    .func = task_str,
    .args = NULL,
    .flag = KUPL_TASK_FLAG_IMM,
};
kupl_task_info_t info = {
    .type = KUPL_TASK_TYPE_SINGLE,
    .desc = &task_desc,
};

int ret = kupl_graph_submit(graph, &info);
assert(ret == KUPL_OK);

kupl_graph_wait(graph);
kupl_graph_destroy(graph);
kupl_egroup_destroy(egroup);
return 0;
}
```

运行结果如下。

```
graph task test.
```

说明

上述示例演示了向graph图提交task、并等待直到graph图的全部task执行完成的流程。上述kupl_graph_wait函数等待graph图中所有task都执行完成，才继续往下执行。

5.3.2.4.8 kupl_sgraph_create

创建一个kupl静态图。静态图相较于动态图而言可以被复用，通过往静态图中添加任务节点及给任务节点添加依赖的方式，可使得静态图具备解决一类特定问题的能力。

接口定义

kupl_sgraph_h kupl_sgraph_create();

参数

无

返回值

- 成功：返回创建的静态图
- 失败：返回nullptr

示例

```
#include <stdio.h>
#include "kupl.h"

int main()
{
    kupl_sgraph_h sgraph = kupl_sgraph_create();
    kupl_sgraph_destroy(sgraph);
    return 0;
}
```

说明

上述示例演示了创建静态图并最后销毁的流程。上述 kupl_sgraph_create函数创建了一个静态图。

 版权所有 © 华为技术有限公司

5.3.2.4.9 kupl_sgraph_destroy

销毁一个kupl静态图。

接口定义

void kupl_sgraph_destroy(kupl_sgraph_h sgraph);

参数

表 5-38 参数定义

参数名	类型	描述	输入/输出
sgraph	kupl_sgraph_h	需要销毁的静态图	输入

示例

```
#include <stdio.h>
#include "kupl.h"

int main()
{
    kupl_sgraph_h sgraph = kupl_sgraph_create();
    kupl_sgraph_destroy(sgraph);
    return 0;
}
```

说明

上述示例演示了创建static graph并最后销毁的流程。上述 kupl_sgraph_destroy函数销毁了之前创建的静态图。

5.3.2.4.10 kupl_sgraph_add_node

向kupl静态图中添加 sgraph node节点，即添加任务节点。

接口定义

kupl_sgraph_node_h kupl_sgraph_add_node(kupl_sgraph_h sgraph, kupl_sgraph_node_desc_t *desc);

参数

表 5-39 参数定义

参数名	类型	描述	输入/输出
sgraph	kupl_sgraph_h	需要向其中添加sgraph node的静态图	输入/输出

版权所有 © 华为技术有限公司

参数名	类型	描述	输入/输出
desc	kupl_sgraph_node_desc_t *	需要添加的sgraph node的描述，具体见下表	输入

表 5-40 kupl_sgraph_node_desc_t 的数据结构

参数名	类型	描述
field_mask	uint64_t	掩码，用于指定结构体中哪些值有效，不设置掩码情况下仅func、args参数生效，其他需要设置掩码。 具体可设置的掩码： ● KUPL_SGRAPH_NODE_DESC_FIELD_NAME: name生效 ● KUPL_SGRAPH_NODE_DESC_FIELD_PRIORITY: priority生效 ● KUPL_SGRAPH_NODE_DESC_FIELD_FLAG: flag生效 ● KUPL_SGRAPH_NODE_DESC_FIELD_EGROUP: egroup生效
func	kupl_sgraph_node_func_t	sgraph node节点的函数，该节点提交执行后需要执行的具体函数
args	void *	func函数需要传入的参数
name	const char *	需要配置掩码KUPL_SGRAPH_NODE_DESC_FIELD_NAME后才能生效；该节点的名字
priority	int	需要配置掩码KUPL_SGRAPH_NODE_DESC_FIELD_PRIORITY后才能生效；该节点的优先级，数值越大，优先级越高
flag	uint32_t	需要配置掩码KUPL_SGRAPH_NODE_DESC_FIELD_FLAG后才能生效；该节点的flag，可设置为KUPL_SGRAPH_NODE_FLAG_IMM，表示提交后将直接执行该节点

版权所有 © 华为技术有限公司

参数名	类型	描述
egroup	kupl_egroup_h	指定当前sgraph node的亲和性信息

返回值

- 成功：返回添加的 sgraph node节点
- 失败：返回nullptr

示例

```
#include <stdio.h>
#include "kupl.h"

static inline void task_str(void *args)
{
    printf("static graph task test.\n");
}

int main()
{
    kupl_graph_h graph = kupl_graph_create(nullptr);

    kupl_sgraph_h sgraph = kupl_sgraph_create();
    kupl_sgraph_node_desc_t node_desc = {
        .func = task_str,
        .args = NULL,
    };
    kupl_sgraph_node_h node = kupl_sgraph_add_node(sgraph, &node_desc);

    kupl_sgraph_task_desc_t task_desc = {
        .sgraph = sgraph,
    };
    kupl_task_info_t info = {
        .type = KUPL_TASK_TYPE_SGRAPH,
        .desc = &task_desc,
    };
    kupl_graph_submit(graph, &info);

    kupl_graph_wait(graph);
    kupl_graph_destroy(graph);
    kupl_sgraph_destroy(sgraph);
    return 0;
}
```

运行结果如下。

```
static graph task test.
```

说明

上述示例演示了在静态图中添加gnode，并通过提交静态图执行的流程。上述kupl_sgraph_add_node 函数向静态图sgraph中添加了sgraph node节点，该sgraph node节点的函数为task_str，函数不需要参数。

5.3.2.4.11 kupl::sgraph_add_node

向kupl静态图中添加 sgraph node节点，即添加任务节点。相较于**5.3.2.4.10 kupl_sgraph_add_node**接口而言，该接口通过lambda函数特性捕获kupl_sgraph_add_node提交任务节点回调函数的入参， 避免用户入参封装行为，提高接口易用性。

版权所有 © 华为技术有限公司

接口定义

kupl_sgraph_node_h sgraph_add_node(kupl_sgraph_h sgraph,
kupl_sgraph_node_desc_t *desc, const std::function<void(void)> &func);

参数

表 5-41 参数定义

参数名	类型	描述	输入/输出
sgraph	kupl_sgraph_h	需要向其中添加sgraph node的静态图	输入/输出
desc	kupl_sgraph_node_desc_t *	需要添加的sgraph node的描述，具体见下表	输入
func	std::function<void(void)>	sgraph node节点的函数，该节点提交执行后需要执行的具体函数	输入

表 5-42 kupl_sgraph_gnode_desc_t 的数据结构

参数名	类型	描述
field_mask	uint64_t	掩码，用于指定结构体中哪些值有效。 具体可设置的掩码： ● KUPL_SGRAPH_NODE_DESC_FIELD_NAME: name生效 ● KUPL_SGRAPH_NODE_DESC_FIELD_PRIORITY: priority生效 KUPL_SGRAPH_NODE_DESC_FIELD_FLAG: flag生效 ● KUPL_SGRAPH_NODE_DESC_FIELD_EGROUP: egroup生效
name	const char *	需要配置掩码 KUPL_SGRAPH_NODE_DESC_FIELD_NAME 后才能生效；该节点的名字

版权所有 © 华为技术有限公司

参数名	类型	描述
priority	int	需要配置掩码KUPL_SGRAPH_NODE_DESC_FIELD_PRIORITY 后才能生效；该节点的优先级，数值越大，优先级越高
flag	uint32_t	需要配置掩码KUPL_SGRAPH_NODE_DESC_FIELD_FLAG后才能生效；该节点的flag，可设置为KUPL_SGRAPH_NODE_FLAG_IMM，表示提交后将直接执行该节点
egroup	kupl_egroup_h	指定当前sgraph node的亲和性信息

返回值

- 成功：返回添加的 sgraph node节点
- 失败：返回nullptr

示例

```
#include <stdio.h>
#include "kupl.h"

int main()
{
    kupl_graph_h graph = kupl_graph_create(nullptr);

    kupl_sgraph_h sgraph = kupl_sgraph_create();
    kupl_sgraph_node_desc_t node_desc = {
        .field_mask = KUPL_SGRAPH_NODE_DESC_FIELD_NAME,
        .name = "sgraph_task",
    };
    kupl_sgraph_node_h node = kupl::sgraph_add_node(sgraph, &node_desc, []() {
        printf("sgraph task test\n");
    });

    kupl_sgraph_task_desc_t task_desc = {
        .sgraph = sgraph,
    };
    kupl_task_info_t info = {
        .type = KUPL_TASK_TYPE_SGRAPH,
        .desc = &task_desc,
    };
    kupl_graph_submit(graph, &info);

    kupl_graph_wait(graph);
    kupl_graph_destroy(graph);
    kupl_sgraph_destroy(sgraph);
    return 0;
}
```

运行结果如下。

```
sgraph task test
```


版权所有 © 华为技术有限公司

说明

上述示例演示了在静态图中添加gnode，并通过提交静态图执行的流程。上述kupl::sgraph_add_node函数向静态图sgraph中添加了sgraph node节点，该sgraph node节点的函数不需要参数。

5.3.2.4.12 kupl_sgraph_add_dep

为kupl静态图中的两个节点添加依赖关系，从而描述任务之间的执行先后顺序。

接口定义

int kupl_sgraph_add_dep(kupl_sgraph_node_h precede, kupl_sgraph_node_h succeed);

参数

表 5-43 参数定义

参数名	类型	描述	输入/输出
precede	kupl_sgraph_node_h	添加依赖关系的前继节点	输入
succeed	kupl_sgraph_node_h	添加依赖关系的后继节点	输入

返回值

成功：返回KUPL_OK

失败：返回KUPL_ERROR

示例

```
#include <stdio.h>
#include "kupl.h"

void func1(void *args)
{
    printf("gnode1 task finished\n");
}

void func2(void *args)
{
    printf("gnode2 task finished\n");
}

int main()
{
    kupl_graph_h graph = kupl_graph_create(nullptr);

    kupl_sgraph_h sgraph = kupl_sgraph_create();
    kupl_sgraph_node_desc_t node1_desc = {
        .func = func1,
        .args = NULL,
    };
```


版权所有 © 华为技术有限公司

```
    kupl_sgraph_node_h node1 = kupl_sgraph_add_node(sgraph, &node1_desc);
    kupl_sgraph_node_desc_t node2_desc = {
        .func = func2,
        .args = NULL,
    };
    kupl_sgraph_node_h node2 = kupl_sgraph_add_node(sgraph, &node2_desc);
    kupl_sgraph_add_dep(node1, node2);

    kupl_sgraph_task_desc_t subgraph_task_desc = {
        .sgraph = sgraph,
    };
    kupl_task_info_t info = {
        .type = KUPL_TASK_TYPE_SGRAPH,
        .desc = &subgraph_task_desc,
    };
    kupl_graph_submit(graph, &info);

    kupl_graph_wait(graph);
    kupl_graph_destroy(graph);
    kupl_sgraph_destroy(sgraph);
    return 0;
}
```

运行结果如下。

```
gnode1 task finished
gnode2 task finished
```

说明

上述示例演示了在静态图中添加node1、node2两个节点，并且为node1、node2两个节点添加依赖关系，最终通过提交静态图执行的流程。上述kupl_sgraph_add_dep函数添加了node1到node2的依赖关系，即node1为前继节点、node2为后继节点；根据运行结果得以验证函数功能，node1的任务先执行完毕，再执行node2的任务。

5.3.2.5 多队列多流编程函数

5.3.2.5.1 概念说明

使用多队列多流编程首先需要了解队列和事件相关的概念，其中queue表示队列，event表示事件。事件可以理解为队列中的一个时间戳，算子可以通过kupl_queue_submit接口提交执行。KUPL还提供了对事件和队列分别进行同步的能力。

5.3.2.5.2 kupl_queue_create

创建一个 kupl 队列。

接口定义

kupl_queue_h kupl_queue_create(void);

参数

无

返回值

成功：返回创建的 queue

失败：返回nullptr

 版权所有 © 华为技术有限公司

示例

```
#include <stdio.h>
#include "kupl.h"

int main()
{
    kupl_queue_h queue = kupl_queue_create();
    kupl_queue_destroy(queue);
    return 0;
}
```

说明

上述示例演示了创建、销毁一个 queue 的流程。kupl_queue_create 函数创建了一个可复用的 queue。

5.3.2.5.3 kupl_get_queue_priority_range

获取队列的优先级范围，默认队列优先级最小值为0，队列优先级最大值为3。

接口定义

int kupl_get_queue_priority_range(int *least_priority, int *greatest_priority);

参数

参数名	类型	描述	输入/输出
least_priority	int *	队列优先级的最小值	输出
greatest_priority	int *	队列优先级的最大值	输出

返回值

成功：返回 KUPL_OK

失败：返回 KUPL_ERROR

示例

```
#include <stdio.h>
#include "kupl.h"

int main()
{
    int least_priority;
    int greatest_priority;
    kupl_get_queue_priority_range(&least_priority, &greatest_priority);
    printf("kupl queue priority range: [%d, %d]\n", least_priority, greatest_priority);
    return 0;
}
```

说明

上述示例演示了获取队列的优先级范围的流程。kupl_get_queue_priority_range能够获取队列优先级的最小值与最大值。

版权所有 © 华为技术有限公司

5.3.2.5.4 kupl_queue_create_with_priority

创建一个kupl优先级队列。

接口定义

kupl_queue_h kupl_queue_create_with_priority(int priority);

环境变量

KUPL通过环境变量KUPL_ENABLE_PRIORITY来控制是否启用优先级。

环境变量KUPL_ENABLE_PRIORITY设置为1时，代表开启优先级功能；环境变量KUPL_ENABLE_PRIORITY设置为0时，代表优先级功能未开启。

参数

参数名	类型	描述	输入/输出
priority	int	队列优先级的值	输入

返回值

成功：返回创建的 queue

失败：返回nullptr

示例

```
#include <stdio.h>
#include "kupl.h"

int main()
{
    int least_priority;
    int greatest_priority;
    kupl_get_queue_priority_range(&least_priority, &greatest_priority);
    int priority = least_priority;
    kupl_queue_h queue = kupl_queue_create_with_priority(priority);
    kupl_queue_destroy(queue);
    return 0;
}
```

说明

上述示例演示了创建、销毁一个优先级queue的流程。kupl_queue_create_with_priority函数创建了一个可复用的优先级queue。

5.3.2.5.5 kupl_queue_destroy

销毁一个 kupl 队列。

接口定义

void kupl_queue_destroy(kupl_queue_h queue);

版权所有 © 华为技术有限公司

参数

参数名	类型	描述	输入/输出
queue	kupl_queue_h	需要销毁的 queue 对象	输入

返回值

无

示例

```
#include <stdio.h>
#include "kupl.h"

int main()
{
    kupl_queue_h queue = kupl_queue_create();
    kupl_queue_destroy(queue);
    return 0;
}
```

说明

上述示例演示了创建、销毁一个 queue 的流程。kupl_queue_destroy 函数将 kupl_queue_create 创建的 queue 销毁。

5.3.2.5.6 kupl_queue_wait

同步一个 kupl 队列，等待队列上的 item / kernel 全部执行结束。

接口定义

int kupl_queue_wait(kupl_queue_h queue);

参数

表 5-44 参数定义

参数名	类型	描述	输入/输出
queue	kupl_queue_h	需要同步的 queue	输入

返回值

成功：返回 KUPL_OK

失败：返回 KUPL_ERROR

示例

```
#include <stdio.h>
#include <assert.h>
```


版权所有 © 华为技术有限公司

```
#include "kupl.h"

static inline void kernel_func(void *args)
{
    printf("kernel exec.\n");
}

int main()
{
    kupl_queue_h queue = kupl_queue_create();
    kupl_queue_item_desc_t desc = {
        .field_mask = KUPL_QUEUE_ITEM_DESC_FIELD_NAME,
        .func = kernel_func,
        .args = NULL,
        .name = "kernel_name"
    };
    kupl_queue_submit(queue, &desc);
    int ret = kupl_queue_wait(queue);
    assert(ret == KUPL_OK);
    kupl_queue_destroy(queue);
    return 0;
}
```

运行结果如下。

```
kernel exec.
```

说明

上述示例演示了向 queue 提交 item，并等待直到 queue 上全部 item 执行完成的流程。上述 kupl_queue_wait 函数等待 queue 中所有 item 都执行完成，才继续往下执行。

5.3.2.5.7 kupl_queue_wait_event

同步 queue 与一个 event，常用于两个 queue 之间的同步。

接口定义

int kupl_queue_wait_event(kupl_queue_h queue, kupl_event_h event);

参数

表 5-45 参数定义

参数名	类型	描述	输入/输出
queue	kupl_queue_h	需要同步的 queue	输入
event	kupl_event_h	需要同步的 event	输入

返回值

成功：返回 KUPL_OK

失败：返回 KUPL_ERROR

示例

```
#include <stdio.h>
#include <assert.h>
```

 版权所有 © 华为技术有限公司

```
#include "kupl.h"

static inline void kernel_func(void *args)
{
    printf("kernel exec.\n");
}

int main()
{
    kupl_queue_h q1 = kupl_queue_create();
    kupl_queue_h q2 = kupl_queue_create();
    kupl_event_h event = kupl_event_create();

    kupl_queue_item_desc_t desc = {
        .field_mask = KUPL_QUEUE_ITEM_DESC_FIELD_NAME,
        .func = kernel_func,
        .args = NULL,
        .name = "kernel_name"
    };
    kupl_queue_submit(q1, &desc);
    kupl_event_record(event, q1);
    int ret = kupl_queue_wait_event(q2, event);
    assert(ret == KUPL_OK);

    kupl_event_destroy(event);
    kupl_queue_destroy(q2);
    kupl_queue_destroy(q1);
    return 0;
}
```

运行结果如下。

```
kernel exec.
```

说明

上述示例演示了向 q1 提交 item，并在 q1 和 q2 之间进行同步的流程。上述 kupl_queue_wait_event 函数等待 q1 中的 item 执行完成继续执行。

5.3.2.5.8 kupl_queue_submit

向queue提交需要执行的item。

接口定义

int kupl_queue_submit(kupl_queue_h queue, kupl_queue_item_desc_t *desc);

参数

表 5-46 参数定义

参数名	类型	描述	输入/输出
queue	kupl_queue_h	执行 item 的 queue	输入
desc	kupl_queue_item_desc_t	queue_item 的描述	输入

版权所有 © 华为技术有限公司

表 5-47 kupl_queue_item_desc_t 的数据结构

参数名	类型	描述
field_mask	uint64_t	结构体中有效字段的掩码，使用kupl_queue_item_desc_field中的位标识。此掩码中未指定的字段将被忽略。 具体可设置的掩码： ● KUPL_QUEUE_ITEM_DESC_FIELD_NAME：name生效 ● KUPL_QUEUE_ITEM_DESC_FIELD_EGROUP：egroup生效 ● KUPL_QUEUE_ITEM_DESC_FIELD_ARGS_SIZE：args_size生效
func	kupl_queue_item_func_t	queue_item 的函数指针
args	void *	queue_item 的参数
name	const char *	queue_item 的名字
egroup	kupl_egroup_h	指定当前queue_item的亲和性信息
args_size	size_t	需要拷贝的 args 参数大小

kupl_queue_item_func_t的定义

void (*kupl_queue_item_func_t)(void *args);

返回值

成功：返回 KUPL_OK

失败：返回 KUPL_ERROR

示例

```
#include <stdio.h>
#include <assert.h>
#include "kupl.h"

static inline void kernel_func(void *args)
{
    printf("kernel exec.\n");
}

int main()
{
    kupl_queue_h queue = kupl_queue_create();
    kupl_queue_item_desc_t desc = {
```

 版权所有 © 华为技术有限公司

```
        .field_mask = KUPL_QUEUE_ITEM_DESC_FIELD_NAME,
        .func = kernel_func,
        .args = NULL,
        .name = "kernel_name"
    };
    int ret = kupl_queue_submit(queue, &desc);
    assert(ret == KUPL_OK);
    kupl_queue_wait(queue);
    kupl_queue_destroy(queue);
    return 0;
}
```

运行结果如下。

```
kernel exec.
```

说明

上述示例演示了向 queue 提交 item，并等待直到 queue 上全部 item 执行完成的流程。上述 kupl_queue_submit 函数向 queue 上提交需要执行的 item。

5.3.2.5.9 kupl::queue_submit(item)

向queue提交需要执行的item。相较于5.3.2.5.8 kupl_queue_submit接口而言，该接口通过lambda函数特性捕获kupl_queue_submit提交item回调函数的入参，避免用户入参封装行为，提高接口易用性。

接口定义

int queue_submit(kupl_queue_h queue, kupl_queue_item_desc_t *desc, const std::function<void(void)> &func);

参数

表 5-48 参数定义

参数名	类型	描述	输入/输出
queue	kupl_queue_h	执行 item 的 queue	输入
desc	kupl_queue_item_desc_t	queue_item 的描述	输入
func	const std::function<void(void)>	queue_item 的函数指针	输入

版权所有 © 华为技术有限公司

表 5-49 kupl_queue_item_desc_t 的数据结构

参数名	类型	描述
field_mask	uint64_t	结构体中有效字段的掩码，使用kupl_queue_item_desc_field中的位标识。此掩码中未指定的字段将被忽略。 具体可设置的掩码： • KUPL_QUEUE_ITEM_DESC_FIELD_NAME：name生效 • KUPL_QUEUE_ITEM_DESC_FIELD_EGROUP：egroup生效
func	kupl_queue_item_func_t	queue_item 的函数指针
args	void *	queue_item 的参数
name	const char *	queue_item 的名字
egroup	kupl_egroup_h	指定当前queue_item的亲和性信息

返回值

成功：返回KUPL_OK

失败：返回KUPL_ERROR

示例

```
#include <stdio.h>
#include <assert.h>
#include "kupl.h"

int main()
{
    kupl_queue_h queue = kupl_queue_create();
    kupl_queue_item_desc_t desc = {
        .field_mask = KUPL_QUEUE_ITEM_DESC_FIELD_NAME,
        .name = "func_name"
    };
    int ret = kupl::queue_submit(queue, &desc, []() {
        printf("queue submit test\n");
    });
    assert(ret == KUPL_OK);
    kupl_queue_wait(queue);
    kupl_queue_destroy(queue);
    return 0;
}
```

运行结果如下。

```
queue submit test
```


版权所有 © 华为技术有限公司

说明

上述示例演示了向queue提交item，并等待直到queue上全部任务执行完成的流程。上述kupl_queue_submit函数向queue上提交需要执行的item。

5.3.2.5.10 kupl::queue_submit(kernel)

向 queue 提交需要执行的 kernel。相较于5.3.2.5.9 kupl::queue_submit(item)接口而言，该接口语义为提交kernel至queue上，kernel是指提交至队列执行的并行计算任务。两者的差异主要在函数接口的入参上，采用C++函数重载的能力进行实现。

说明

该接口需要使用 C++ 编译器, range 目前只支持 1d。

接口定义

int kupl::queue_submit(kupl_queue_h queue, kupl_queue_kernel_desc_t *desc, const std::function<void(const kupl_nd_range_t *)> &kernel);

参数

表 5-50 参数定义

参数名	类型	描述	输入/输出
queue	kupl_queue_h	执行 kernel 的 queue	输入
desc	kupl_queue_kernel_desc_t	kernel 的描述	输入
kernel	const std::function<void(const kupl_nd_range_t *)>	kernel 的内容，一般是 lambda 表达式	输入

表 5-51 kupl_queue_kernel_desc_t 的数据结构

参数名	类型	描述
field_mask	uint64_t	kernel 描述的掩码，用于表示哪些参数启用，必选。不开启选项时填 0。 可用选项： KUPL_QUEUE_KERNEL_DESC_FIELD_NAME
range	kupl_nd_range_t *	kernel 的 range 信息，必选

版权所有 © 华为技术有限公司

参数名	类型	描述
egroup	kupl_egroup_h	kernel 可用的 executor 信息，必选
name	const char *	kernel 的名字，可选

返回值

成功：返回 KUPL_OK

失败：返回 KUPL_ERROR

示例

```
#include <atomic>
#include <assert.h>
#include "kupl.h"

int main() {
    auto queue = kupl_queue_create();
    const int num_executors = 10;
    kupl_nd_range_t range;
    KUPL_1D_RANGE_INIT(range, 0, num_executors);
    int exe[num_executors];
    for (int i = 0; i < num_executors; i++) {
        exe[i] = i;
    }
    kupl_egroup_h egroup = kupl_egroup_create(exe, num_executors);
    kupl_queue_kernel_desc_t desc = {
        .range = &range,
        .egroup = egroup,
        .field_mask = 0,
    };

    std::atomic<size_t> sum(0);
    const size_t count = 1000000;
    size_t sum_cal = (1 + count) * count / 2;
    size_t *data = (size_t *)malloc(count * sizeof(count));
    for (size_t i = 0; i < count; i++) {
        data[i] = i + 1;
    }
    int ret = kupl::queue_submit(queue, &desc, [&](const kupl_nd_range_t *nd_range) {
        int start_index = nd_range->nd_range[0].lower * (count / num_executors);
        for (size_t i = 0; i < count / num_executors; i++) {
            sum += data[start_index + i];
        }
    });
    assert(ret == KUPL_OK);
    kupl_queue_wait(queue);
    assert(sum.load() == sum_cal);
    printf("sum: %lu\n", sum.load());
    kupl_egroup_destroy(egroup);
    kupl_queue_destroy(queue);
}
```

运行结果如下。

```
sum: 500000500000
```


版权所有 © 华为技术有限公司

说明

上述示例演示了向 queue 提交 kernel，在 10 个线程上计算一个长度为 1000000 的数组总和，并等待直到 queue 上全部 kernel 执行完成的流程，最终和直接计算的结果相比较，并输出结果。

5.3.2.5.11 kupl_event_create

创建一个可以复用的 kupl 事件。

接口定义

kupl_event_h kupl_event_create(void);

参数

无

返回值

成功：返回创建的 event

失败：返回nullptr

示例

```
#include <stdio.h>
#include "kupl.h"

int main()
{
    kupl_event_h event = kupl_event_create();
    kupl_event_destroy(event);
    return 0;
}
```

说明

上述示例演示了创建、销毁一个 event 的流程。kupl_event_create 函数创建了一个可复用的 event。

5.3.2.5.12 kupl_event_destroy

销毁一个 event 事件。

接口定义

void kupl_event_destroy(kupl_event_h event);

参数

参数名	类型	描述	输入/输出
event	kupl_event_h	需要销毁的 event 对象	输入

版权所有 © 华为技术有限公司

返回值

无

示例

```
#include <stdio.h>
#include "kupl.h"

int main()
{
    kupl_event_h event = kupl_event_create();
    kupl_event_destroy(event);
    return 0;
}
```

说明

上述示例演示了创建、销毁一个 event 的流程。kupl_event_destroy 函数将 kupl_event_create 创建的 event 销毁。

5.3.2.5.13 kupl_event_record

将一个 event 事件记录在 queue 队列上，从而用于后续队列间同步行为。

接口定义

int kupl_event_record(kupl_event_h event, kupl_queue_h queue);

参数

参数名	类型	描述	输入/输出
event	kupl_event_h	需要记录的 event 对象	输入
queue	kupl_queue_h	承载记录的 queue 对象	输入

返回值

成功：返回 KUPL_OK

失败：返回 KUPL_ERROR

示例

```
#include <stdio.h>
#include <assert.h>
#include "kupl.h"

int main()
{
    kupl_event_h event = kupl_event_create();
    kupl_queue_h queue = kupl_queue_create();

    int ret = kupl_event_record(event, queue);
    assert(ret == KUPL_OK);
```


版权所有 © 华为技术有限公司

```
    kupl_queue_destroy(queue);
    kupl_event_destroy(event);
    return 0;
}
```

说明

上述示例演示了记录一个 event 的流程。kupl_event_record 函数将 kupl_event_create 创建的 event 记录在由 kupl_queue_create 创建的 queue 上。

5.3.2.5.14 kupl_event_wait

同步 event，直到 event 之前所有的 kernel 执行完成。

接口定义

int kupl_event_wait(kupl_event_h event);

参数

表 5-52 参数定义

参数名	类型	描述	输入/输出
event	kupl_event_h	需要同步的 event	输入

返回值

成功：返回 KUPL_OK

失败：返回 KUPL_ERROR

示例

```
#include <stdio.h>
#include <assert.h>
#include "kupl.h"

static inline void kernel_func(void *args)
{
    printf("kernel exec.\n");
}

int main()
{
    kupl_event_h event = kupl_event_create();
    kupl_queue_h queue = kupl_queue_create();

    kupl_queue_item_desc_t desc = {
        .func = kernel_func,
        .args = NULL,
        .name = "kernel_name"
    };
    kupl_queue_submit(queue, &desc);
    kupl_event_record(event, queue);
    int ret = kupl_event_wait(event);
    assert(ret == KUPL_OK);

    kupl_queue_destroy(queue);
    kupl_event_destroy(event);
```


版权所有 © 华为技术有限公司

```
    return 0;
}
```

运行结果如下。

```
kernel exec.
```

说明

上述示例演示了向 queue 提交 kernel，并等待直到 kernel 执行完成的流程。上述 kupl_event_wait 函数等待 event 之前所有 kernel 都执行完成，才继续往下执行。

5.3.2.5.15 kupl_event_query

查询 event 状态。

接口定义

int kupl_queue_query(void);

参数

无

返回值

event 被创建：KUPL_EVENT_STATUS_CREATED

event 被提交：KUPL_EVENT_STATUS_SUBMITTED

event 完成：KUPL_EVENT_STATUS_COMPLETE

查询失败：KUPL_ERROR

示例

```
#include <stdio.h>
#include <assert.h>
#include "kupl.h"

int main()
{
    kupl_event_h event = kupl_event_create();

    int ret = kupl_event_query(event);
    assert(ret == KUPL_EVENT_STATUS_CREATED);

    kupl_event_destroy(event);
    return 0;
}
```

说明

上述示例演示了创建一个 event，并查询其状态的流程。上述 kupl_event_query 函数将返回 event 当前的状态。

5.3.2.5.16 kupl_queue_acquire

使用序号获取队列句柄

版权所有 © 华为技术有限公司

接口定义

kupl_queue_h kupl_queue_acquire(int index);

参数

表 5-53 参数定义

参数名	类型	描述	输入/输出
index	int	队列的序号 使用 KUPL_ASYNC_SYNC 表示同步队列	输入

返回值

成功：返回 queue 句柄

失败：返回 nullptr

示例

```
#include <stdio.h>
#include "kupl.h"

int main()
{
    kupl_queue_h queue = kupl_queue_acquire(1);
    kupl_queue_destroy(queue);
    return 0;
}
```

说明

上述示例演示了获取一个序号为 1 的队列句柄，然后销毁的流程。获取队列后可以不销毁，在程序结束后会自动销毁。

5.3.2.5.17 kupl_queue_wait_all

同步所有使用 kupl_queue_acquire 接口创建的队列

接口定义

int kupl_queue_wait_all();

参数

无

返回值

成功：返回 KUPL_OK

失败：返回 KUPL_ERROR

版权所有 © 华为技术有限公司

示例

```
#include <stdio.h>
#include "kupl.h"

static inline void kernel_func(void *args)
{
    printf("kernel exec.\n");
}

int main()
{
    kupl_queue_h q1 = kupl_queue_acquire(1);
    kupl_queue_h q2 = kupl_queue_acquire(2);
    kupl_queue_item_desc_t desc = {
        .field_mask = KUPL_QUEUE_ITEM_DESC_FIELD_NAME,
        .func = kernel_func,
        .args = NULL,
        .name = "kernel_name"
    };
    kupl_queue_submit(q1, &desc);
    kupl_queue_submit(q2, &desc);
    kupl_queue_wait_all();
    return 0;
}
```

说明

上述示例演示了获取序号为 1 和 2 的队列句柄，并分别提交一个 item，最后用 kupl_queue_wait_all 进行同步。

5.3.2.6 内存管理函数

5.3.2.6.1 概念说明

KUPL库提供了内存操作的相关函数，例如内存拷贝的相关函数kupl_memcpy、kupl_memcpy2d等。

5.3.2.6.2 环境变量

- KUPL_ENABLE_HUGEPAGES，1表示KUPL在分配内存时，会优先选择大页内存进行分配，若大页内存不足或不存在，则退回至小页内存分配；0则表示KUPL不使用大页内存。该环境变量默认值为0。
- KUPL_MPOOL_ALIGN_SIZE，表示KUPL分配的内存的字节对齐，最小值为2，最大值为2048。该环境变量默认值为8，代表KUPL分配的地址按照8字节对齐。

5.3.2.6.3 kupl_malloc

使用特定分配策略分配内存，并尽可能地锁住申请的内存。

接口定义

void* kupl_malloc(kupl_mem_kind_t kind, size_t size);

版权所有 © 华为技术有限公司

参数

表 5-54 参数定义

参数名	类型	描述	输入/输出
kind	kupl_mem_kind_t	需要使用的内存分配策略	输入
size	size_t	需要申请的内存大小	输入

表 5-55 kupl_mem_kind_t 的枚举数据结构定义

枚举值	描述
KUPL_MEM_DEFAULT	使用系统默认的分配策略分配内存
KUPL_MEM_LARGE_CAP	从最近的高容量内存节点上申请内存，如果分配失败，则分配返回nullptr并报错
KUPL_MEM_HIGH_BW	从最近的高带宽内存节点上申请内存，如果分配失败，则分配返回nullptr并报错

返回值

- 成功：返回申请得到的内存的指针
- 失败：返回nullptr

示例

```
#include <stdio.h>
#include <stdlib.h>
#include "kupl.h"

int main()
{
    int len = 1024;
    char *data = (char *)kupl_malloc(KUPL_MEM_DEFAULT, len);
    if (data == nullptr) {
        return 0;
    }
    kupl_free(KUPL_MEM_DEFAULT, data);
    return 0;
}
```

说明

- 上述示例演示了申请锁住的内存并释放该内存的流程。
- 上述kupl_malloc函数申请大小为len的内存空间并尽可能锁住，使用的内存分配策略为系统默认的分配策略。
- kupl_malloc函数会在条件允许的情况下锁住申请的内存；其中环境中包含SDMA设备是锁住申请的内存的必要条件。

版权所有 © 华为技术有限公司

5.3.2.6.4 kupl_free

取消锁定内存，并释放内存。

接口定义

void kupl_free(kupl_mem_kind_t kind, void *ptr);

参数

表 5-56 参数定义

参数名	类型	描述	输入/输出
kind	kupl_mem_kind_t	需要使用的内存分配策略	输入
ptr	void*	需要释放的内存的指针	输入

示例

```
#include <stdio.h>
#include <stdlib.h>
#include "kupl.h"

int main()
{
    int len = 1024;
    char *data = (char *)kupl_malloc(KUPL_MEM_DEFAULT, len);
    if (data == nullptr) {
        return 0;
    }
    kupl_free(KUPL_MEM_DEFAULT, data);
    return 0;
}
```

说明

- 上述示例演示了申请锁住的内存并释放该内存的流程。
- 上述kupl_free函数取消data指向的内存锁定并释放内存，使用的内存分配策略为系统默认的分配策略。
- kupl_free函数会在发现内存被锁住时取消锁定，并释放内存；否则直接释放内存。

5.3.2.6.5 kupl_mlock

锁住buffer所在的内存页表。

接口定义

int kupl_mlock(void *buffer, size_t count);

版权所有 © 华为技术有限公司

参数

表 5-57 参数定义

参数名	类型	描述	输入/输出
buffer	void*	需要锁定的内存指针	输入
count	size_t	需要锁定的内存大小	输入

返回值

- 成功：返回KUPL_OK
- 失败：返回KUPL_ERROR

示例

```
#include <stdio.h>
#include <stdlib.h>
#include "kupl.h"

int main()
{
    int len = 1024;
    char *data = (char *)malloc(len);
    if (data == nullptr) {
        return 0;
    }
    kupl_mlock(data, len);
    kupl_munlock(data, len);
    free(data);
    return 0;
}
```

说明

- 上述示例演示了申请内存、锁住内存、取消锁定并最终释放内存的流程。
- 上述kupl_mlock函数用于锁住data指向的内存。
- kupl_mlock函数会锁住内存。其中环境中包含sdma设备是成功锁住内存的必要条件。

5.3.2.6.6 kupl_munlock

取消锁定内存页表。

接口定义

int kupl_munlock(void *buffer, size_t count);

 版权所有 © 华为技术有限公司

参数

表 5-58 参数定义

参数名	类型	描述	输入/输出
buffer	void*	需要取消锁定的内存指针	输入
count	size_t	需要取消锁定的内存大小	输入

返回值

- 成功：返回KUPL_OK
- 失败：返回KUPL_ERROR

示例

```
#include <stdio.h>
#include <stdlib.h>
#include "kupl.h"

int main()
{
    int len = 1024;
    char *data = (char *)malloc(len);
    if (data == nullptr) {
        return 0;
    }
    kupl_mlock(data, len);
    kupl_munlock(data, len);
    free(data);
    return 0;
}
```

说明

- 上述示例演示了申请内存、锁住内存、取消锁定并最终释放内存的流程。
- 上述kupl_munlock函数用于取消锁定data指向的内存。
- kupl_munlock函数会取消锁定内存。其中环境中包含sdma设备是成功取消锁定的必要条件。

5.3.2.6.7 kupl_set_kernel_concurrency

设置全局的memcpy算子并发度。

接口定义

void kupl_set_kernel_concurrency(int num);

版权所有 © 华为技术有限公司

参数

参数名	类型	描述	输入/输出
num	int	需要设置的算子并发度的值 约束： 1≤num≤kupl_get_num_executors()；当输入不在约束范围内时，内部设置num为kupl_get_num_executors()	输入

示例

```
#include <stdio.h>
#include "kupl.h"

int main()
{
    kupl_set_kernel_concurrency(2);
    int num = kupl_get_kernel_concurrency();
    printf("kupl kernel concurrency = %d\n", num);
    return 0;
}
```

运行结果如下。

```
kupl kernel concurrency = 2
```

说明

- 上述示例设置并打印了全局算子并发度的值。上述kupl_set_kernel_concurrency函数将算子并发度设置为2。
- 除了通过kupl_set_kernel_concurrency函数接口设置算子并发度外，还可以通过环境变量KUPL_KERNEL_CONCURRENCY来设置算子并发度。

5.3.2.6.8 kupl_get_kernel_concurrency

获取设置的当前实际并发度的值。

接口定义

int kupl_get_kernel_concurrency();

返回值

返回当前实际并发度的值

当前不在并行域内时将返回1。

示例

```
#include <stdio.h>
#include "kupl.h"
```

 版权所有 © 华为技术有限公司

```
int main()
{
    kupl_set_kernel_concurrency(2);
    int num = kupl_get_kernel_concurrency();
    printf("kupl kernel concurrency = %d\n", num);
    return 0;
}
```

运行结果如下。

```
kupl kernel concurrency = 1
```

说明

上述示例打印了当前并发度的值；上述kupl_get_kernel_concurrency函数获取了当前的实际并发度。

5.3.2.6.9 kupl_set_kernel_concurrency_local

设置当前线程作为主线程用于调用memcpy算子时支持的多线程并发度。

接口定义

void kupl_set_kernel_concurrency_local(int num);

参数

表 5-59 参数定义

参数名	类型	描述	输入/输出
num	int	需要设置的当前线程的算子并发度的值 约束：1≤num≤kupl_get_num_executors()。当输入不在约束范围内时，内部设置num为kupl_get_num_executors()	输入

示例

```
#include <stdio.h>
#include "kupl.h"

int main()
{
    kupl_set_kernel_concurrency_local(2);
    int num = kupl_get_kernel_concurrency_local();
    printf("local kupl kernel concurrency = %d\n", num);
    return 0;
}
```


版权所有 © 华为技术有限公司

运行结果如下。

```
local kupl kernel concurrency = 2
```

说明

- 上述示例设置并打印了当前线程的算子并发度的值。
- 上述kupl_set_kernel_concurrency_local函数将算子并发度设置为2。

5.3.2.6.10 kupl_get_kernel_concurrency_local

获取当前线程作为主线程去调用memcpy算子时支持的多线程并发度的值。

接口定义

int kupl_get_kernel_concurrency_local();

返回值

- 返回当前线程的算子并发度的值
- 算子并发度值未设置时返回kupl_get_kernel_concurrency();

示例

```
#include <stdio.h>
#include "kupl.h"

int main()
{
    kupl_set_kernel_concurrency_local(2);
    int num = kupl_get_kernel_concurrency_local();
    printf("local kupl kernel concurrency = %d\n", num);
    return 0;
}
```

运行结果如下。

```
local kupl kernel concurrency = 2
```

说明

- 上述示例设置并打印了当前线程的算子并发度的值。
- 上述kupl_get_kernel_concurrency_local函数获取了当前线程的算子并发度。

5.3.2.6.11 kupl_memcpy

内存拷贝，将src位置的内存拷贝到dst位置。

接口定义

int kupl_memcpy(void *dst, const void *src, size_t count);

说明

- 当前kupl memcpy支持的最大拷贝长度为UINT32_MAX，即count值不超过UINT32_MAX。
- count 需要小于 src 和 dst 所指向内存的真实大小。

版权所有 © 华为技术有限公司

环境变量

KUPL通过环境变量KUPL_MEMCPY_MT_THRESHOLD与KUPL_SDMA_MEMCPY_THRESHOLD来确认多线程memcpy能力的包大小阈值以及sdma memcpy能力的包大小阈值。多线程memcpy阈值默认为512KB，sdma memcpy阈值默认为2MB。

kupl memcpy共实现3种memcpy方法：单线程glibc memcpy方法、sdma memcpy方法及多线程memcpy方法；当环境中使能sdma时，通过KUPL_SDMA_MEMCPY_THRESHOLD阈值选取memcpy方法，即拷贝的数据包长小于阈值时，进行glibc memcpy，否则进行sdma memcpy；当环境中没有sdma时，通过KUPL_MEMCPY_MT_THRESHOLD阈值选取memcpy方法，即拷贝的数据包长小于阈值时，进行glibc memcpy，否则进行多线程memcpy。

用户可以通过配置KUPL_MEMCPY_MT_THRESHOLD与KUPL_SDMA_MEMCPY_THRESHOLD来设置多线程memcpy与sdma memcpy的阈值；kupl_memcpy2d接口也采用上述环境变量配置方式控制memcpy方法的选取。

参数

表 5-60 参数定义

参数名	类型	描述	输入/输出
dst	void *	指向存储复制内容的目标内存位置的指针	输入/输出
src	const void *	指向要复制的源内存位置的指针	输入
count	size_t	需要复制的内存大小	输入

返回值

- 成功：返回KUPL_OK
- 失败：返回KUPL_ERROR

示例

```
#include <stdio.h>
#include <stdlib.h>
#include <assert.h>
#include "kupl.h"

int main()
{
    int len = 1024;
    char *src = (char *)kupl_malloc(KUPL_MEM_DEFAULT, len);
    char *dest = (char *)kupl_malloc(KUPL_MEM_DEFAULT, len);
    if (src == nullptr || dest == nullptr) {
        goto error;
    }
    for (int i = 0; i < len / sizeof(char); i++) {
        src[i] = i ;
        dest[i] = 0;
```


版权所有 © 华为技术有限公司

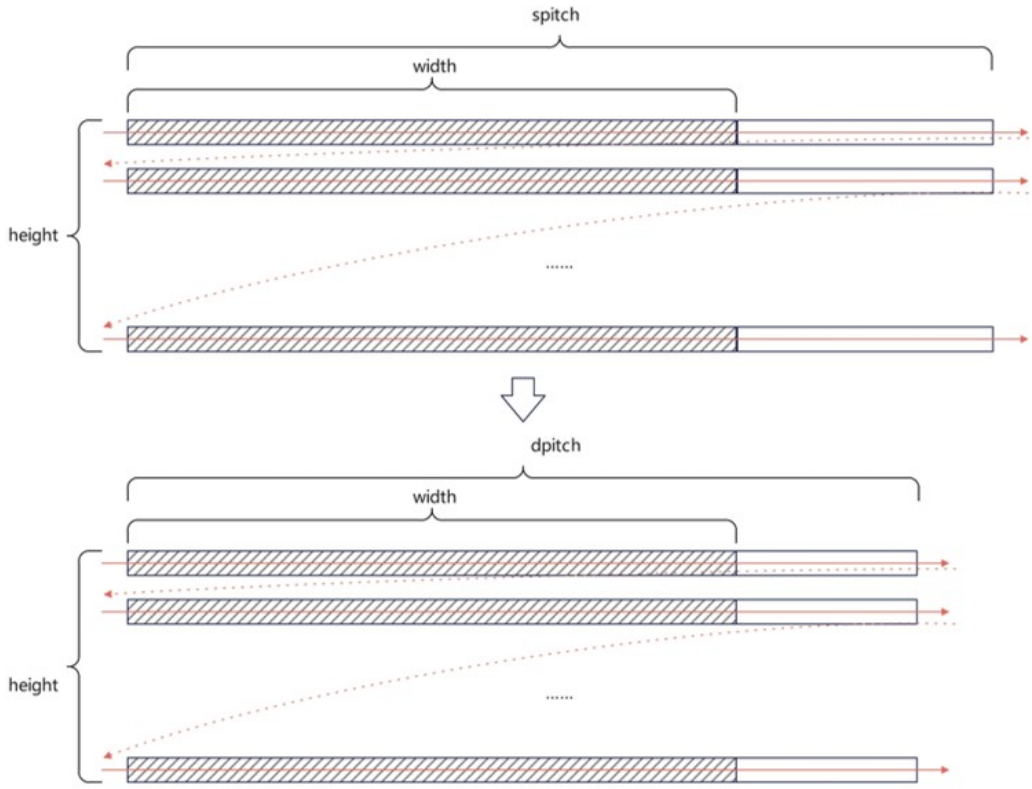
```
    }
    int ret = kupl_memcpy(dest, src, len);
    assert(ret == KUPL_OK);
error:
    kupl_free(KUPL_MEM_DEFAULT, src);
    kupl_free(KUPL_MEM_DEFAULT, dest);
    return 0;
}
```

说明

- 上述示例演示了内存拷贝的流程。
- 上述kupl_memcpy函数将src数组中的内容复制到dest数组，其中复制的内存大小为len。

5.3.2.6.12 kupl_memcpy2d

二维内存拷贝，将src位置的二维内存拷贝到dst位置，具体行为示例如下图所示：



接口定义

int kupl_memcpy2d(void *dst, size_t dpitch, const void *src, size_t spitch, size_t width, size_t height);

说明

- 当前kupl memcpy2d支持的最大宽、高、搬运间隔均不超过UINT32_MAX，即width、spitch-width、dpitch-width、height上限为UINT32_MAX。
- 需分别满足spitch*height小于 src 所指向内存的真实大小，且dpitch*height 小于 dst 所指向内存的真实大小。

版权所有 © 华为技术有限公司

环境变量

KUPL通过环境变量KUPL_MEMCPY_MT_THRESHOLD与KUPL_SDMA_MEMCPY_THRESHOLD来确认多线程memcpy能力的包大小阈值以及sdma memcpy能力的包大小阈值。多线程memcpy阈值默认为512KB，sdma memcpy阈值默认为2MB。

kupl memcpy2d共实现3种memcpy方法：单线程glibc memcpy方法、sdma memcpy方法及多线程memcpy方法；当环境中使能sdma时，通过KUPL_SDMA_MEMCPY_THRESHOLD阈值选取memcpy方法，即拷贝的数据包长小于阈值时，进行glibc memcpy，否则进行sdma memcpy；当环境中没有sdma时，通过KUPL_MEMCPY_MT_THRESHOLD阈值选取memcpy方法，即拷贝的数据包长小于阈值时，进行glibc memcpy，否则进行多线程memcpy。

用户可以通过配置KUPL_MEMCPY_MT_THRESHOLD与KUPL_SDMA_MEMCPY_THRESHOLD来设置多线程memcpy与sdma memcpy的阈值；kupl_memcpy接口也采用上述环境变量配置方式控制memcpy方法的选取。

参数

表 5-61 参数定义

参数名	类型	描述	输入/输出
dst	void *	指向存储复制内容的目标内存位置的指针	输入/输出
dpitch	size_t	目标内存的间距，即每存储width数据的偏移量	输入
src	const void *	指向要复制的源内存位置的指针	输入
spitch	size_t	源内存的间距，即每取出width数据的偏移量	输入
width	size_t	需要拷贝的内存的宽	输入
height	size_t	需要拷贝的内存的高	输入

返回值

- 成功：返回KUPL_OK
- 失败：返回KUPL_ERROR

示例

```
#include <stdio.h>
#include <stdlib.h>
```


版权所有 © 华为技术有限公司

```
#include <assert.h>
#include "kupl.h"

int main()
{
    int len = 65536;
    double *src = (double *)kupl_malloc(KUPL_MEM_DEFAULT, len);
    double *dest = (double *)kupl_malloc(KUPL_MEM_DEFAULT, len);
    if (src == nullptr || dest == nullptr) {
        goto error;
    }
    for (int i = 0; i < len / sizeof(double); i++) {
        src[i] = i ;
        dest[i] = 0;
    }
    int height = 2, width = 200;
    int spitch = 300, dpitch = 400;
    int ret = kupl_memcpy2d(dest, dpitch, src, spitch, width, height);
    assert(ret == KUPL_OK);
error:
    kupl_free(KUPL_MEM_DEFAULT, src);
    kupl_free(KUPL_MEM_DEFAULT, dest);
    return 0;
}
```

说明

- 上述示例演示了二维内存拷贝的流程。
- 上述kupl_memcpy2d函数将src数组中的内容复制到dest数组，其中src位置的数据每取出width数据的偏移量为300。
- dest位置的数据每存储width数据的偏移量为400。
- 拷贝数据的宽和高分别为200和2。

5.3.2.6.13 kupl_memcpy_async

异步内存拷贝，异步地将src位置的内存拷贝到dst位置。

接口定义

int kupl_memcpy_async(void *dst, const void *src, size_t count, kupl_queue_h queue, kupl_event_h event);

说明

- 当前kupl memcpy_async支持的最大拷贝长度为UINT32_MAX，即count值不超过UINT32_MAX。
- count 需要小于 src 和 dst 所指向内存的真实大小。

环境变量

KUPL通过环境变量KUPL_MEMCPY_MT_THRESHOLD与KUPL_SDMA_MEMCPY_THRESHOLD来确认多线程memcpy能力的包大小阈值以及sdma memcpy能力的包大小阈值。多线程memcpy阈值默认为512KB，sdma memcpy阈值默认为2MB。

kupl异步memcpy共实现3种memcpy方法：单线程glibc memcpy方法、sdma memcpy方法及多线程memcpy方法；当环境中使能sdma时，默认选取sdma memcpy方法；当环境中没有sdma时，通过KUPL_MEMCPY_MT_THRESHOLD阈值选取memcpy方法，即拷贝的数据包长小于阈值时，进行glibc memcpy，否则进行多线程memcpy。

 版权所有 © 华为技术有限公司

用户可以通过配置KUPL_MEMCPY_MT_THRESHOLD来设置多线程memcpy的阈值；kupl_memcpy2d_async接口也采用上述环境变量配置方式控制memcpy方法的选取。

参数

表 5-62 参数定义

参数名	类型	描述	输入/输出
dst	void *	指向存储复制内容的目标内存位置的指针	输入/输出
src	const void *	指向要复制的源内存位置的指针	输入
count	size_t	需要复制的内存大小	输入
queue	kupl_queue_h	需要将memcpy任务提交到的队列	输入
event	kupl_event_h	传递event任务的参数，用于后续同步	输入/输出

返回值

- 成功：返回KUPL_OK
- 失败：返回KUPL_ERROR

示例

```
#include <stdio.h>
#include <stdlib.h>
#include <assert.h>
#include "kupl.h"

int main()
{
    int len = 1024;
    char *src = (char *)kupl_malloc(KUPL_MEM_DEFAULT, len);
    char *dest = (char *)kupl_malloc(KUPL_MEM_DEFAULT, len);
    if (src == nullptr || dest == nullptr) {
        goto malloc_error;
    }
    for (int i = 0; i < len / sizeof(char); i++) {
        src[i] = i ;
        dest[i] = 0;
    }
    kupl_queue_h queue = kupl_queue_create();
    kupl_event_h event = kupl_event_create();
    if (queue == nullptr || event == nullptr) {
        goto queue_event_error;
    }
    int ret = kupl_memcpy_async(dest, src, len, queue, event);
    assert(ret == KUPL_OK);
    kupl_event_wait(event);
queue_event_error:
```


版权所有 © 华为技术有限公司

```
    kupl_event_destroy(event);
    kupl_queue_destroy(queue);
malloc_error:
    kupl_free(KUPL_MEM_DEFAULT, src);
    kupl_free(KUPL_MEM_DEFAULT, dest);
    return 0;
}
```

说明

- 上述示例演示了异步内存拷贝的流程。
- 上述kupl_memcpy_async函数将src数组中的内容异步地复制到dest数组，其中复制的内存大小为len，其中event用于异步拷贝、后续的同步。
- 由于实际上只有sdma memcpy实现了真正的异步memcpy，因此在执行其余memcpy方法时，将打印warning信息。

5.3.2.6.14 kupl_memcpy2d_async

二维异步内存拷贝，异步地将src位置的内存拷贝到dst位置。

接口定义

int kupl_memcpy2d_async(void *dst, size_t dpitch, const void *src, size_t spitch, size_t width, size_t height, kupl_queue_h queue, kupl_event_h event);

说明

- 当前kupl memcpy2d_async支持的最大宽、高、搬运间隔均不超过UINT32_MAX，即width、spitch-width、dpitch-width、height上限为UINT32_MAX。。
- spitch*height 与dpitch*height 需要分别小于 src 和 dst 所指向内存的真实大小。

环境变量

KUPL通过环境变量KUPL_MEMCPY_MT_THRESHOLD与KUPL_SDMA_MEMCPY_THRESHOLD来确认多线程memcpy能力的包大小阈值以及sdma memcpy能力的包大小阈值。多线程memcpy阈值默认为512KB，sdma memcpy阈值默认为2MB。

kupl二维异步memcpy共实现3种memcpy方法：单线程glibc memcpy方法、sdma memcpy方法及多线程memcpy方法；当环境中使能sdma时，默认选取sdma memcpy方法；当环境中没有sdma时，通过KUPL_MEMCPY_MT_THRESHOLD阈值选取memcpy方法，即拷贝的数据包长小于阈值时，进行glibc memcpy，否则进行多线程memcpy。

用户可以通过配置KUPL_MEMCPY_MT_THRESHOLD来设置多线程memcpy的阈值；kupl_memcpy_async接口也采用上述环境变量配置方式控制memcpy方法的选取。

版权所有 © 华为技术有限公司

参数

表 5-63 参数定义

参数名	类型	描述	输入/输出
dst	void *	指向存储复制内容的目标内存位置的指针	输入/输出
dpitch	size_t	目标内存的间距，即每存储width数据的偏移量	输入
src	const void *	指向要复制的源内存位置的指针	输入
spitch	size_t	源内存的间距，即每取出width数据的偏移量	输入
width	size_t	需要拷贝的内存的宽	输入
height	size_t	需要拷贝的内存的高	输入
queue	kupl_queue_h	需要将二维memcpy任务提交到的队列	输入
event	kupl_event_h	传递event任务的参数，用于后续同步	输入/输出

返回值

- 成功：返回KUPL_OK
- 失败：返回KUPL_ERROR

示例

```
#include <stdio.h>
#include <stdlib.h>
#include <assert.h>
#include "kupl.h"

int main()
{
    int len = 65536;
    double *src = (double *)kupl_malloc(KUPL_MEM_DEFAULT, len);
    double *dest = (double *)kupl_malloc(KUPL_MEM_DEFAULT, len);
    if (src == nullptr || dest == nullptr) {
        goto malloc_error;
    }
    for (int i = 0; i < len / sizeof(double); i++) {
        src[i] = i ;
```


版权所有 © 华为技术有限公司

```
        dest[i] = 0;
    }
    int height = 2, width = 200;
    int spitch = 300, dpitch = 400;
    kupl_queue_h queue = kupl_queue_create();
    kupl_event_h event = kupl_event_create();
    if (queue == nullptr || event == nullptr) {
        goto queue_event_error;
    }
    int ret = kupl_memcpy2d_async(dest, dpitch, src, spitch, width, height, queue, event);
    assert(ret == KUPL_OK);
    kupl_event_wait(event);
queue_event_error:
    kupl_event_destroy(event);
    kupl_queue_destroy(queue);
malloc_error:
    kupl_free(KUPL_MEM_DEFAULT, src);
    kupl_free(KUPL_MEM_DEFAULT, dest);
    return 0;
}
```

说明

- 上述示例演示了异步二维内存拷贝的流程。
- 上述kupl_memcpy2d_async函数异步地将src数组中的内容复制到dest数组，其中src位置的数据每取出width数据的偏移量为300。
- dest位置的数据每存储width数据的偏移量为400。
- 拷贝数据的宽和高分别为200和2。
- 上述kupl_memcpy_async函数将src数组中的内容异步地复制到dest数组，其中复制的内存大小为len，其中event用于异步拷贝、后续的同步。
- 由于实际上只有sdma memcpy实现了真正的异步memcpy，因此在执行其余memcpy方法时，将打印warning信息。

5.3.2.6.15 kupl_hbw_malloc

申请OPM内存，并尽可能锁住内存。

接口定义

void* kupl_hbw_malloc(size_t size);

参数

表 5-64 参数定义

参数名	类型	描述	输入/输出
size	size_t	需要申请的内存大小	输入

返回值

- 成功：返回申请得到的内存指针
- 失败：返回nullptr

 版权所有 © 华为技术有限公司

示例

```
#include <stdio.h>
#include <stdlib.h>
#include "kupl.h"

int main()
{
    int len = 1024;
    char *data = (char *)kupl_hbw_malloc(len);
    if (data == nullptr) {
        return 0;
    }
    kupl_hbw_free(data);
    return 0;
}
```

说明

- 上述示例演示了申请锁住的OPM内存并释放该内存的流程。
- 上述kupl_hbw_malloc函数申请大小为len的内存空间并尽可能锁住。
- kupl_hbw_malloc函数会在条件允许的情况下通过内存锁定机制防止被交换到磁盘。其中环境中包含sdma设备是锁定申请的内存的必要条件。
- kupl_hbw_malloc在无法申请到指定的OPM内存时，会返回nullptr。

5.3.2.6.16 kupl_hbw_free

取消锁定OPM内存，并释放OPM内存。

接口定义

void kupl_hbw_free(void *ptr);

参数

表 5-65 参数定义

参数名	类型	描述	输入/输出
ptr	void*	需要释放的OPM内存的指针	输入

示例

```
#include <stdio.h>
#include <stdlib.h>
#include "kupl.h"

int main()
{
    int len = 1024;
    char *data = (char *)kupl_hbw_malloc(len);
    if (data == nullptr) {
        return 0;
    }
    kupl_hbw_free(data);
    return 0;
}
```


版权所有 © 华为技术有限公司

说明

- 上述示例演示了申请锁住的OPM内存并释放该内存的流程。
- 上述kupl_hbw_free函数取消data指向的内存锁定并释放内存。
- kupl_hbw_free函数会在发现内存被内存锁定机制锁定时取消锁定，并释放内存；否则直接释放内存。

5.3.2.6.17 kupl_hbw_verify

判断内存空间从地址addr开始到addr+size为止是否全部在OPM内。

接口定义

int kupl_hbw_verify(void *addr, size_t size, int flags);

参数

表 5-66 参数定义

参数名	类型	描述	输入/输出
addr	void*	需要判断的内存的起始地址	输入
size	size_t	需要判断的内存大小	输入
flags	int	额外的flag信息，可能会影响函数行为，当前可以设置为KUPL_HBW_TOUCH_PAGES或为0（0代表不做额外处理）	输入

KUPL_HBW_TOUCH_PAGES：表示在验证前会按照操作系统的分页，依次读写所需验证内存空间中的每一页。这项操作会强制操作系统为验证的地址实际分配内存空间并建立页的映射。在用户不能确定所需验证的虚拟内存是否已存在物理页的映射时需要指定该flags。

返回值

- 若整块内存都在OPM中，返回KUPL_IS_HBW_MEMORY。
- 若给定的内存空间中有不属于OPM的部分则返回KUPL_IS_NOT_HBW_MEMORY。
- 若处理过程中出错返回KUPL_HBW_VERIFY_ERROR。

示例

```
#include <stdio.h>
#include <stdlib.h>
#include <assert.h>
#include "kupl.h"

int main()
{
    int len = 1024;
    char *data = (char *)kupl_hbw_malloc(len);
```


版权所有 © 华为技术有限公司

```
    if (data == nullptr) {
        return 0;
    }

    int ret = kupl_hbw_verify(data, 1024, KUPL_HBW_TOUCH_PAGES);
    assert(ret == KUPL_IS_HBW_MEMORY);
    kupl_hbw_free(data);
    return 0;
}
```

说明

- 上述示例演示了申请OPM内存并验证申请的内存是否全部在OPM上的流程。
- 上述kupl_hbw_verify验证的指针的起始地址为kupl_hbw_malloc申请得到的地址的起始地址，验证的大小也与申请大小保持一致。
- 在设置了HBW_TOUCH_PAGES后，kupl_hbw_verify在验证前会按照操作系统的分页，依次读写所需验证内存空间中的每一页。

5.3.2.6.18 kupl_hbw_check_available

判断系统中是否存在OPM。

接口定义

int kupl_hbw_check_available();

返回值

- 系统中存在OPM：返回1
- 系统中不存在OPM：返回0

示例

```
#include <stdio.h>
#include "kupl.h"

int main()
{
    if (kupl_hbw_check_available()) {
        printf("High Band-Width Memory is available in the system.\n");
    } else {
        printf("Cannot detect High Band-Width Memory in the system.\n");
    }
    return 0;
}
```

说明

- 上述示例演示了判断系统中是否存在OPM的流程。
- 上述示例中，若系统中存在OPM，则会打印"High Band-Width Memory is available in the system."; 若不存在OPM， 则会打印"Cannot detect High Band-Width Memory in the system."。

5.3.2.6.19 kupl_hbw_get_policy

获取当前的OPM内存分配策略。

接口定义

kupl_hbw_policy_t kupl_hbw_get_policy();

返回值

- 返回当前的OPM内存分配策略。具体的策略描述见下面的表格。

表 5-67 kupl_hbw_policy_t 的数据结构定义

值	描述
KUPL_HBW_POLICY_BIND	从最近的NUMA OPM Node上申请内存，如果OPM内存不足，则分配返回nullptr并报错

示例

```
#include <stdio.h>
#include <assert.h>
#include "kupl.h"

int main()
{
    int ret = kupl_hbw_set_policy(KUPL_HBW_POLICY_BIND);
    assert(ret ==  KUPL_OK);
    kupl_hbw_policy_t policy = kupl_hbw_get_policy();
    assert(policy ==  KUPL_HBW_POLICY_BIND);
    return 0;
}
```

说明

- 上述示例演示了设置当前的OPM内存分配策略并获取当前的OPM内存分配策略的流程。
- 设置和获取的KUPL_HBW_POLICY_BIND表示当前的策略为从最近的NUMA OPM Node上申请内存，如果OPM内存不足，则分配返回nullptr并报错。

5.3.2.6.20 kupl_hbw_set_policy

设置当前的OPM内存分配策略为指定值

接口定义

int kupl_hbw_set_policy(kupl_hbw_policy_t policy);

参数

表 5-68 参数定义

参数名	类型	描述	输入/输出
policy	kupl_hbw_policy_t	需要设置的OPM内存分配策略	输入

版权所有 © 华为技术有限公司

返回值

- 成功：返回KUPL_OK
- 失败：返回KUPL_ERROR

示例

```
#include <stdio.h>
#include <assert.h>
#include "kupl.h"

int main()
{
    int ret = kupl_hbw_set_policy(KUPL_HBW_POLICY_BIND);
    assert(ret ==  KUPL_OK);
    kupl_hbw_policy_t policy = kupl_hbw_get_policy();
    assert(policy ==  KUPL_HBW_POLICY_BIND);
    return 0;
}
```

说明

- 上述示例演示了设置当前的OPM内存分配策略并获取当前的OPM内存分配策略的流程。
- 设置和获取的KUPL_HBW_POLICY_BIND表示当前的策略为从最近的NUMA OPM Node上申请内存，如果OPM内存不足，则分配返回nullptr并报错。

5.3.2.6.21 kupl_mem_copyin

内存拷贝，将内存从 ddr 拷贝至 hbw

接口定义

int kupl_mem_copyin(void *ddr_addr, size_t size, kupl_mem_copyin_flag_t flag, kupl_queue_h queue);

参数

表 5-69 参数定义

参数名	类型	描述	输入/输出
ddr_addr	void *	需要拷贝的 ddr 地址	输入
size	size_t	需要拷贝的内存字节大小	输入

版权所有 © 华为技术有限公司

参数名	类型	描述	输入/输出
flag	kupl_mem_copyin_flag_t	进行换入操作的类型，包括： KUPL_MEM_CREATE 如果ddr_addr 不是 present 状态则创建 ddr 至 hbw 的映射（此时状态变成 present），否则引用计数加一 KUPL_MEM_IN 如果ddr_addr 不是 present 状态则创建 ddr 至 hbw 的映射，并将 ddr 上的内容拷贝至 hbw（此时状态变成 present），否则引用计数加一 KUPL_MEM_PUSH 将 ddr 上的内容拷贝至 hbw	输入
queue	kupl_queue_h	进行操作的队列	输入

返回值

- 成功：返回 KUPL_OK
- 失败：返回 KUPL_ERROR

示例

```
#include "kupl.h"
#include <cassert>

static int a = 1;

void query_and_modify(void *args)
{
    void *hbw = kupl_mem_query(&a);
    int ret = kupl_hbw_verify(hbw, sizeof(int), 0);
    *(int *)hbw = 2;
}

int main()
{
    auto queue = kupl_queue_acquire(1);
    kupl_mem_copyin(&a, sizeof(int), KUPL_MEM_IN, queue);
    auto present = kupl_mem_is_present(&a);
    assert(present == true);
    kupl_queue_item_desc_t desc {
        .field_mask = KUPL_QUEUE_ITEM_DESC_FIELD_NAME,
        .func = query_and_modify,
        .args = nullptr,
        .name = "query_and_modify",
```


版权所有 © 华为技术有限公司

```
    };
    kupl_queue_submit(queue, &desc);
    kupl_mem_copyout(&a, sizeof(int), KUPL_MEM_OUT, queue);
    kupl_queue_wait(queue);
    present = kupl_mem_is_present(&a);
    assert(present == false);
    printf("a: %d\n", a);
}
```

运行结果如下。

```
a: 2
```

说明

- 上述示例演示了 ddr 内存和 hbw 内存完整的换入换出流程。首先使用 kupl_mem_copyin 将内存从 ddr 拷贝至 hbw，然后使用 kupl_mem_is_present 判断内存的可访问性，接着提交一个异步任务使用 kupl_mem_query 查询 ddr 对应的 hbw 的地址然后修改，最后使用 kupl_mem_copyout 将 hbw 上的地址再拷贝回 ddr。

5.3.2.6.22 kupl_mem_copyout

内存拷贝，将内存从 hbw 拷贝至 ddr

接口定义

int kupl_mem_copyout(void *ddr_addr, size_t size, kupl_mem_copyout_flag_t flag, kupl_queue_h queue);

参数

表 5-70 参数定义

参数名	类型	描述	输入/输出
ddr_addr	void *	需要拷贝的 ddr 地址	输入
size	size_t	需要拷贝的内存字节大小	输入

版权所有 © 华为技术有限公司

参数名	类型	描述	输入/输出
flag	kupl_mem_copyout_flag_t	进行换入操作的类型，包括： KUPL_MEM_DELETE 如果ddr_addr是 present 状态则引用计数减一，当引用计数归 0 则销毁 ddr 至 hbw 的映射（此时状态变成非 present） KUPL_MEM_OUT 如果ddr_addr 是 present 状态则引用计数减一，当引用计数归 0 则将内存从 hbw 拷贝至 ddr，销毁 ddr 至 hbw 的映射（此时状态变成非 present） KUPL_MEM_PULL 将 hbw上的内容拷贝至 ddr KUPL_MEM_DELETE_FINALIZE 销毁映射 KUPL_MEM_OUT_FINALIZE 将内存从 hbw 拷贝至 ddr，并销毁映射	输入
queue	kupl_queue_h	进行操作的队列	输入

返回值

- 成功：返回 KUPL_OK
- 失败：返回 KUPL_ERROR

示例

```
#include "kupl.h"
#include <cassert>

static int a = 1;

void query_and_modify(void *args)
{
    void *hbw = kupl_mem_query(&a);
    int ret = kupl_hbw_verify(hbw, sizeof(int), 0);
    *(int *)hbw = 2;
}
```


版权所有 © 华为技术有限公司

```
int main()
{
    auto queue = kupl_queue_acquire(1);
    kupl_mem_copyin(&a, sizeof(int), KUPL_MEM_IN, queue);
    auto present = kupl_mem_is_present(&a);
    assert(present == true);
    kupl_queue_item_desc_t desc {
        .field_mask = KUPL_QUEUE_ITEM_DESC_FIELD_NAME,
        .func = query_and_modify,
        .args = nullptr,
        .name = "query_and_modify",
    };
    kupl_queue_submit(queue, &desc);
    kupl_mem_copyout(&a, sizeof(int), KUPL_MEM_OUT, queue);
    kupl_queue_wait(queue);
    present = kupl_mem_is_present(&a);
    assert(present == false);
    printf("a: %d\n", a);
}
```

运行结果如下。

```
a: 2
```

说明

- 上述示例演示了 ddr 内存和 hbw 内存完整的换入换出流程。首先使用 kupl_mem_copyin 将内存从 ddr 拷贝至 hbw，然后使用 kupl_mem_is_present 判断内存的可访问性，接着提交一个异步任务使用 kupl_mem_query 查询 ddr 对应的 hbw 的地址然后修改，最后使用 kupl_mem_copyout 将 hbw 上的地址再拷贝回 ddr。

5.3.2.6.23 kupl_mem_query

查询 ddr 对应的 hbw 地址

接口定义

void *kupl_mem_query(void *ddr_addr);

参数

表 5-71 参数定义

参数名	类型	描述	输入/输出
ddr_addr	void *	需要查询的 ddr 地址	输入

返回值

- 成功：返回 hbw 的地址
- 失败：返回空指针

示例

```
#include "kupl.h"
#include <cassert>

static int a = 1;
```

 版权所有 © 华为技术有限公司

```
void query_and_modify(void *args)
{
    void *hbw = kupl_mem_query(&a);
    int ret = kupl_hbw_verify(hbw, sizeof(int), 0);
    *(int *)hbw = 2;
}

int main()
{
    auto queue = kupl_queue_acquire(1);
    kupl_mem_copyin(&a, sizeof(int), KUPL_MEM_IN, queue);
    auto present = kupl_mem_is_present(&a);
    assert(present == true);
    kupl_queue_item_desc_t desc {
        .field_mask = KUPL_QUEUE_ITEM_DESC_FIELD_NAME,
        .func = query_and_modify,
        .args = nullptr,
        .name = "query_and_modify",
    };
    kupl_queue_submit(queue, &desc);
    kupl_mem_copyout(&a, sizeof(int), KUPL_MEM_OUT, queue);
    kupl_queue_wait(queue);
    present = kupl_mem_is_present(&a);
    assert(present == false);
    printf("a: %d\n", a);
}
```

运行结果如下。

```
a: 2
```

说明

- 上述示例演示了 ddr 内存和 hbw 内存完整的换入换出流程。首先使用 kupl_mem_copyin 将内存从 ddr 拷贝至 hbw，然后使用 kupl_mem_is_present 判断内存的可访问性，接着提交一个异步任务使用 kupl_mem_query 查询 ddr 对应的 hbw 的地址然后修改，最后使用 kupl_mem_copyout 将 hbw 上的地址再拷贝回 ddr。

5.3.2.6.24 kupl_mem_is_present

查询 ddr 地址在 hbw 上的可访问性

接口定义

bool kupl_mem_is_present(void *ddr_addr);

参数

表 5-72 参数定义

参数名	类型	描述	输入/输出
ddr_addr	void *	需要查询的 ddr 地址	输入

返回值

- 成功：返回 true

 版权所有 © 华为技术有限公司

- 失败：返回 false

示例

```
#include "kupl.h"
#include <cassert>

static int a = 1;

void query_and_modify(void *args)
{
    void *hbw = kupl_mem_query(&a);
    int ret = kupl_hbw_verify(hbw, sizeof(int), 0);
    *(int *)hbw = 2;
}

int main()
{
    auto queue = kupl_queue_acquire(1);
    kupl_mem_copyin(&a, sizeof(int), KUPL_MEM_IN, queue);
    auto present = kupl_mem_is_present(&a);
    assert(present == true);
    kupl_queue_item_desc_t desc {
        .field_mask = KUPL_QUEUE_ITEM_DESC_FIELD_NAME,
        .func = query_and_modify,
        .args = nullptr,
        .name = "query_and_modify",
    };
    kupl_queue_submit(queue, &desc);
    kupl_mem_copyout(&a, sizeof(int), KUPL_MEM_OUT, queue);
    kupl_queue_wait(queue);
    present = kupl_mem_is_present(&a);
    assert(present == false);
    printf("a: %d\n", a);
}
```

运行结果如下。

```
a: 2
```

说明

- 上述示例演示了 ddr 内存和 hbw 内存完整的换入换出流程。首先使用 kupl_mem_copyin 将内存从 ddr 拷贝至 hbw，然后使用 kupl_mem_is_present 判断内存的可访问性，接着提交一个异步任务使用 kupl_mem_query 查询 ddr 对应的 hbw 的地址然后修改，最后使用 kupl_mem_copyout 将 hbw 上的地址再拷贝回 ddr。

5.3.2.7 共享内存通信函数

5.3.2.7.1 概念说明

KUPL提供了共享内存底层通信接口，以及基于这些接口的集合通信函数实现。

5.3.2.7.2 kupl_shm_comm_create

创建kupl comm。

接口定义

int kupl_shm_comm_create(int size, int rank, int pid, kupl_shm_oob_cb_h oob_cbs, void *group, kupl_shm_comm_h *comm);

版权所有 © 华为技术有限公司

参数

表 5-73 参数定义

参数名	类型	描述	输入/输出
size	int	需要创建的kupl通信域大小	输入
rank	int	当前进程的进程编号（从0开始的相对编码）	输入
pid	int	当前进程的pid	输入
oob_cbs	kupl_shm_oob_cb_h	带外函数指针，详见表2 shm带外函数的结构体定义	输入
group	void *	带外函数指针依赖的参数	输入
comm	kupl_shm_comm_h	需要创建的kupl通信域	输出

表 5-74 shm 带外函数的结构体定义

参数名	类型	描述
oob_allgather	int (*kupl_shm_oob_allgather_cb_t)(const void*, void*, int, void*, kupl_shm_datatype_t)	带外allgather函数指针，用于kupl_comm创建时进程相关信息的收集
oob_barrier	int (*kupl_shm_oob_barrier_cb_t)(void *)	带外barrier函数指针，确保kupl_comm创建的正确性

返回值

- 成功：返回KUPL_OK
- 失败：返回KUPL_ERROR

示例

```
#include <stdio.h>
#include <mpi.h>
#include <unistd.h>
#include "kupl.h"
```


版权所有 © 华为技术有限公司

```
// 创建 kupl 通信域需要的回调函数 1
static int oob_barrier_callback(void *group)
{
    return MPI_Barrier((MPI_Comm)group);
}
// 创建 kupl 通信域需要的回调函数 2
static int oob_allgather_callback(const void *sendbuf, void *recvbuf, int size, void *group,
                                  kupl_shm_datatype_t datatype)
{
    switch (datatype) {
        case KUPL_SHM_DATATYPE_CHAR:
            return MPI_Allgather(sendbuf, size, MPI_CHAR, recvbuf, size, MPI_CHAR, (MPI_Comm)group);
        default:
            fprintf(stderr, "not support datatype");
            return KUPL_ERROR;
    }
}
int main(int argc, char *argv[])
{
    // 初始化 MPI 环境
    MPI_Init(&argc, &argv);
    MPI_Comm comm = MPI_COMM_WORLD;
    // 获取 MPI 通信域大小
    int world_size;
    MPI_Comm_size(comm, &world_size);
    // 获取进程 rank 号
    int world_rank;
    MPI_Comm_rank(comm, &world_rank);
    // 获取进程 pid 号
    int pid = getpid();
    // 创建 kupl 通信域
    kupl_shm_oob_cb_t oob_cbs;
    kupl_shm_oob_cb_h oob_cbs_h = &oob_cbs;
    oob_cbs_h->oob_allgather = oob_allgather_callback;
    oob_cbs_h->oob_barrier = oob_barrier_callback;
    kupl_shm_comm_h kupl_comm;
    int ret = kupl_shm_comm_create(world_size, world_rank, pid, oob_cbs_h, (void *)comm,
&kupl_comm);
    if (ret != KUPL_OK || kupl_comm == NULL) {
        fprintf(stderr, "kupl shm comm create failed: %d\n", ret);
        return -1;
    }
    kupl_shm_comm_destroy(kupl_comm);
    MPI_Finalize();
    return 0;
}
```

说明

上述示例演示了创建、清理一个kupl comm的流程。kupl_shm_comm_create函数创建了一个kupl comm。

5.3.2.7.3 kupl_shm_comm_destroy

清理kupl comm。

接口定义

int kupl_shm_comm_destroy(kupl_shm_comm_h comm);

 版权所有 © 华为技术有限公司

参数

表 5-75 参数定义

参数名	类型	描述	输入/输出
comm	kupl_shm_comm_h	需要清理的kupl通信域	输入

返回值

- 成功：返回KUPL_OK
- 失败：返回KUPL_ERROR

示例

```
#include <stdio.h>
#include <mpi.h>
#include <unistd.h>
#include "kupl.h"

// 创建 kupl 通信域需要的回调函数 1
static int oob_barrier_callback(void *group)
{
    return MPI_Barrier((MPI_Comm)group);
}
// 创建 kupl 通信域需要的回调函数 2
static int oob_allgather_callback(const void *sendbuf, void *recvbuf, int size, void *group,
                                  kupl_shm_datatype_t datatype)
{
    switch (datatype) {
        case KUPL_SHM_DATATYPE_CHAR:
            return MPI_Allgather(sendbuf, size, MPI_CHAR, recvbuf, size, MPI_CHAR, (MPI_Comm)group);
        default:
            fprintf(stderr, "not support datatype");
            return KUPL_ERROR;
    }
}
int main(int argc, char *argv[])
{
    // 初始化 MPI 环境
    MPI_Init(&argc, &argv);
    MPI_Comm comm = MPI_COMM_WORLD;
    // 获取 MPI 通信域大小
    int world_size;
    MPI_Comm_size(comm, &world_size);
    // 获取进程 rank 号
    int world_rank;
    MPI_Comm_rank(comm, &world_rank);
    // 获取进程 pid 号
    int pid = getpid();
    // 创建 kupl 通信域
    kupl_shm_oob_cb_t oob_cbs;
    kupl_shm_oob_cb_h oob_cbs_h = &oob_cbs;
    oob_cbs_h->oob_allgather = oob_allgather_callback;
    oob_cbs_h->oob_barrier = oob_barrier_callback;
    kupl_shm_comm_h kupl_comm;
    int ret = kupl_shm_comm_create(world_size, world_rank, pid, oob_cbs_h, (void *)comm, &kupl_comm);
    if (ret != KUPL_OK || kupl_comm == NULL) {
        fprintf(stderr, "kupl shm comm create failed: %d\n", ret);
        return -1;
    }
    kupl_shm_comm_destroy(kupl_comm);
```


版权所有 © 华为技术有限公司

```
    MPI_Finalize();
    return 0;
}
```

说明

上述示例演示了创建、清理一个kupl comm的流程。kupl_shm_comm_destroy函数将kupl_shm_comm_create创建的kupl comm清理。

5.3.2.7.4 kupl_shm_comm_rank

获取当前进程的rank。

接口定义

int kupl_shm_comm_rank(kupl_shm_comm_h comm, int *rank);

参数

表 5-76 参数定义

参数名	类型	描述	输入/输出
comm	kupl_shm_comm_h	kupl通信域	输入
rank	int *	需要获取的rank	输出

返回值

成功：返回KUPL_OK

失败：返回KUPL_ERROR

示例

```
#include <stdio.h>
#include <mpi.h>
#include <unistd.h>
#include "kupl.h"

// 创建 kupl 通信域需要的回调函数 1
static int oob_barrier_callback(void *group)
{
    return MPI_Barrier((MPI_Comm)group);
}
// 创建 kupl 通信域需要的回调函数 2
static int oob_allgather_callback(const void *sendbuf, void *recvbuf, int size, void *group,
                                  kupl_shm_datatype_t datatype)
{
    switch (datatype) {
        case KUPL_SHM_DATATYPE_CHAR:
            return MPI_Allgather(sendbuf, size, MPI_CHAR, recvbuf, size, MPI_CHAR, (MPI_Comm)group);
        default:
            fprintf(stderr, "not support datatype");
            return KUPL_ERROR;
    }
}
int main(int argc, char *argv[])
{
```


版权所有 © 华为技术有限公司

```
    // 初始化 MPI 环境
    MPI_Init(&argc, &argv);
    MPI_Comm comm = MPI_COMM_WORLD;
    // 获取 MPI 通信域大小
    int world_size;
    MPI_Comm_size(comm, &world_size);
    // 获取进程 rank 号
    int world_rank;
    MPI_Comm_rank(comm, &world_rank);
    // 获取进程 pid 号
    int pid = getpid();
    // 创建 kupl 通信域
    kupl_shm_oob_cb_t oob_cbs;
    kupl_shm_oob_cb_h oob_cbs_h = &oob_cbs;
    oob_cbs_h->oob_allgather = oob_allgather_callback;
    oob_cbs_h->oob_barrier = oob_barrier_callback;
    kupl_shm_comm_h kupl_comm;
    int ret = kupl_shm_comm_create(world_size, world_rank, pid, oob_cbs_h, (void *)comm, &kupl_comm);
    if (ret != KUPL_OK || kupl_comm == NULL) {
        fprintf(stderr, "kupl shm comm create failed: %d\n", ret);
        return -1;
    }
    int kupl_rank;
    int kupl_size;
    kupl_shm_comm_rank(kupl_comm, &kupl_rank);
    kupl_shm_comm_size(kupl_comm, &kupl_size);
    kupl_shm_comm_destroy(kupl_comm);
    MPI_Finalize();
    return 0;
}
```

说明

上述示例演示了创建、清理一个kupl comm的流程。kupl_shm_comm_rank函数获取当前进程的rank。

5.3.2.7.5 kupl_shm_comm_size

获取kupl comm通信域大小。

接口定义

int kupl_shm_comm_size(kupl_shm_comm_h comm, int *size);

参数

表 5-77 参数定义

参数名	类型	描述	输入/输出
comm	kupl_shm_comm_h	kupl通信域	输入
size	int *	需要获取的kupl通信域大小	输出

返回值

- 成功：返回KUPL_OK

 版权所有 © 华为技术有限公司

- 失败：返回KUPL_ERROR

示例

```
#include <stdio.h>
#include <mpi.h>
#include <unistd.h>
#include "kupl.h"

// 创建 kupl 通信域需要的回调函数 1
static int oob_barrier_callback(void *group)
{
    return MPI_Barrier((MPI_Comm)group);
}
// 创建 kupl 通信域需要的回调函数 2
static int oob_allgather_callback(const void *sendbuf, void *recvbuf, int size, void *group,
                                  kupl_shm_datatype_t datatype)
{
    switch (datatype) {
        case KUPL_SHM_DATATYPE_CHAR:
            return MPI_Allgather(sendbuf, size, MPI_CHAR, recvbuf, size, MPI_CHAR, (MPI_Comm)group);
        default:
            fprintf(stderr, "not support datatype");
            return KUPL_ERROR;
    }
}
int main(int argc, char *argv[])
{
    // 初始化 MPI 环境
    MPI_Init(&argc, &argv);
    MPI_Comm comm = MPI_COMM_WORLD;
    // 获取 MPI 通信域大小
    int world_size;
    MPI_Comm_size(comm, &world_size);
    // 获取进程 rank 号
    int world_rank;
    MPI_Comm_rank(comm, &world_rank);
    // 获取进程 pid 号
    int pid = getpid();
    // 创建 kupl 通信域
    kupl_shm_oob_cb_t oob_cbs;
    kupl_shm_oob_cb_h oob_cbs_h = &oob_cbs;
    oob_cbs_h->oob_allgather = oob_allgather_callback;
    oob_cbs_h->oob_barrier = oob_barrier_callback;
    kupl_shm_comm_h kupl_comm;
    int ret = kupl_shm_comm_create(world_size, world_rank, pid, oob_cbs_h, (void *)comm, &kupl_comm);
    if (ret != KUPL_OK || kupl_comm == NULL) {
        fprintf(stderr, "kupl shm comm create failed: %d\n", ret);
        return -1;
    }
    int kupl_rank;
    int kupl_size;
    kupl_shm_comm_rank(kupl_comm, &kupl_rank);
    kupl_shm_comm_size(kupl_comm, &kupl_size);
    kupl_shm_comm_destroy(kupl_comm);
    MPI_Finalize();
    return 0;
}
```

说明

上述示例演示了创建、清理一个kupl comm的流程。kupl_shm_comm_size函数获取kupl comm的size。

5.3.2.7.6 kupl_shm_win_alloc

为指定kupl comm的每个进程分配一块等长共享内存空间，可通过kupl win访问。

 版权所有 © 华为技术有限公司

接口定义

int kupl_shm_win_alloc(size_t size, kupl_shm_comm_h comm, void **baseptr, kupl_shm_win_h *win)

环境变量

使用环境变量 KUPL_SHM_TYPE 控制共享内存底层实现方式。

可选值：

- posix（默认）
- sls

当共享内存底层实现为sls的情况下，可以使用环境变量KUPL_SHM_ENABLE_HUGEPAGE和KUPL_SHM_ON_PACKAGE来控制是否使能共享内存申请大页和是否使能共享内存申请高带宽内存。

KUPL_SHM_ENABLE_HUGEPAGE，默认值为n，代表不启用内存大页分配；y则代表在启用sls作为共享内存底层实现方式的情况下，启用共享内存大页分配。

KUPL_SHM_ON_PACKAGE，默认值为n，代表不启用高带宽内存分配；y则代表在启用sls作为共享内存底层实现方式的情况下，启用高带宽内存分配。

参数

表 5-78 参数定义

参数名	类型	描述	输入/输出
size	size_t	共享内存空间字节大小	输入
comm	kupl_shm_comm_h	kupl通信域	输入
baseptr	void **	进程共享内存块的首地址	输出
win	kupl_shm_win_h *	需要创建的window对象	输出

返回值

- 成功：返回KUPL_OK
- 失败：返回KUPL_ERROR

示例

```
#include <stdio.h>
#include <mpi.h>
#include <string.h>
#include <unistd.h>
#include "kupl.h"
```


版权所有 © 华为技术有限公司

```
// 创建 kupl 通信域需要的回调函数 1
static int oob_barrier_callback(void *group)
{
    return MPI_Barrier((MPI_Comm)group);
}
// 创建 kupl 通信域需要的回调函数 2
static int oob_allgather_callback(const void *sendbuf, void *recvbuf, int size, void *group,
                                  kupl_shm_datatype_t datatype)
{
    switch (datatype) {
        case KUPL_SHM_DATATYPE_CHAR:
            return MPI_Allgather(sendbuf, size, MPI_CHAR, recvbuf, size, MPI_CHAR, (MPI_Comm)group);
        default:
            fprintf(stderr, "not support datatype");
            return KUPL_ERROR;
    }
}
int main(int argc, char *argv[])
{
    // 初始化 MPI 环境
    MPI_Init(&argc, &argv);
    MPI_Comm comm = MPI_COMM_WORLD;
    // 获取 MPI 通信域大小
    int world_size;
    MPI_Comm_size(comm, &world_size);
    // 获取进程 rank 号
    int world_rank;
    MPI_Comm_rank(comm, &world_rank);
    // 获取进程 pid 号
    int pid = getpid();
    // 创建 kupl 通信域
    kupl_shm_oob_cb_t oob_cbs;
    kupl_shm_oob_cb_h oob_cbs_h = &oob_cbs;
    oob_cbs_h->oob_allgather = oob_allgather_callback;
    oob_cbs_h->oob_barrier = oob_barrier_callback;
    kupl_shm_comm_h kupl_comm;
    int ret = kupl_shm_comm_create(world_size, world_rank, pid, oob_cbs_h, (void *)comm, &kupl_comm);
    if (ret != KUPL_OK || kupl_comm == NULL) {
        fprintf(stderr, "kupl shm comm create failed: %d\n", ret);
        return -1;
    }
    // 申请共享内存
    kupl_shm_win_h win;
    void *local_buffer;
    int count = 4;
    size_t buf_size = count * sizeof(int);
    ret = kupl_shm_win_alloc(buf_size, kupl_comm, &local_buffer, &win);
    if (ret != KUPL_OK) {
        fprintf(stderr, "kupl shm win alloc failed: %d\n", ret);
        return -1;
    }
    // 进程 0 对本进程 buffer 赋值
    if (world_rank == 0) {
        for (int i = 0; i < count; i++) {
            ((int *)local_buffer)[i] = i;
        }
    }
    // 所有其他进程获取进程 0 的 buffer，进行 memcpy
    if (world_rank != 0) {
        void *remote_buffer;
        int remote_rank = 0;
        kupl_shm_win_query(win, remote_rank, &remote_buffer);
        memcpy(local_buffer, remote_buffer, buf_size);
        // 检查是否成功 memcpy
        bool check = true;
        for (int i = 0; i < count; i++) {
            if (((int *)local_buffer)[i] != i) {
                check = false;
                break;
```

 版权所有 © 华为技术有限公司

```
            }
        }
        if (check) {
            printf("check success\n");
        } else {
            printf("check fail\n");
        }
    }
    // 销毁共享内存
    kupl_shm_win_free(win);
    // 销毁 kupl 通信域
    kupl_shm_comm_destroy(kupl_comm);
    MPI_Finalize();
    return 0;
}
```

运行结果如下。

```
check success
```

说明

上述示例演示了创建、查询、清理一个kupl win的流程。kupl_shm_win_alloc函数创建一个kupl win。

5.3.2.7.7 kupl_shm_win_query

获取对端进程的共享内存首地址。

接口定义

int kupl_shm_win_query(kupl_shm_win_h win, int remote_rank, void **baseptr);

参数

表 5-79 参数定义

参数名	类型	描述	输入/输出
win	kupl_shm_win_h	用于查询的window对象	输入
remote_rank	int	对端进程的进程号	输入
baseptr	void **	对端进程共享内存块的首地址	输出

返回值

成功：返回KUPL_OK

失败：返回KUPL_ERROR

示例

```
#include <stdio.h>
#include <mpi.h>
#include <unistd.h>
```


版权所有 © 华为技术有限公司

```
#include <string.h>
#include "kupl.h"

// 创建 kupl 通信域需要的回调函数 1
static int oob_barrier_callback(void *group)
{
    return MPI_Barrier((MPI_Comm)group);
}
// 创建 kupl 通信域需要的回调函数 2
static int oob_allgather_callback(const void *sendbuf, void *recvbuf, int size, void *group,
                                  kupl_shm_datatype_t datatype)
{
    switch (datatype) {
        case KUPL_SHM_DATATYPE_CHAR:
            return MPI_Allgather(sendbuf, size, MPI_CHAR, recvbuf, size, MPI_CHAR, (MPI_Comm)group);
        default:
            fprintf(stderr, "not support datatype");
            return KUPL_ERROR;
    }
}
int main(int argc, char *argv[])
{
    // 初始化 MPI 环境
    MPI_Init(&argc, &argv);
    MPI_Comm comm = MPI_COMM_WORLD;
    // 获取 MPI 通信域大小
    int world_size;
    MPI_Comm_size(comm, &world_size);
    // 获取进程 rank 号
    int world_rank;
    MPI_Comm_rank(comm, &world_rank);
    // 获取进程 pid 号
    int pid = getpid();
    // 创建 kupl 通信域
    kupl_shm_oob_cb_t oob_cbs;
    kupl_shm_oob_cb_h oob_cbs_h = &oob_cbs;
    oob_cbs_h->oob_allgather = oob_allgather_callback;
    oob_cbs_h->oob_barrier = oob_barrier_callback;
    kupl_shm_comm_h kupl_comm;
    int ret = kupl_shm_comm_create(world_size, world_rank, pid, oob_cbs_h, (void *)comm, &kupl_comm);
    if (ret != KUPL_OK || kupl_comm == NULL) {
        fprintf(stderr, "kupl shm comm create failed: %d\n", ret);
        return -1;
    }
    // 申请共享内存
    kupl_shm_win_h win;
    void *local_buffer;
    int count = 4;
    size_t buf_size = count * sizeof(int);
    ret = kupl_shm_win_alloc(buf_size, kupl_comm, &local_buffer, &win);
    if (ret != KUPL_OK) {
        fprintf(stderr, "kupl shm win alloc failed: %d\n", ret);
        return -1;
    }
    // 进程 0 对本进程 buffer 赋值
    if (world_rank == 0) {
        for (int i = 0; i < count; i++) {
            ((int *)local_buffer)[i] = i;
        }
    }
    // 所有其他进程获取进程 0 的 buffer，进行 memcpy
    if (world_rank != 0) {
        void *remote_buffer;
        int remote_rank = 0;
        kupl_shm_win_query(win, remote_rank, &remote_buffer);
        memcpy(local_buffer, remote_buffer, buf_size);
        // 检查是否成功 memcpy
        bool check = true;
        for (int i = 0; i < count; i++) {
```

 版权所有 © 华为技术有限公司

```
            if (((int *)local_buffer)[i] != i) {
                check = false;
                break;
            }
        }
        if (check) {
            printf("check success\n");
        } else {
            printf("check fail\n");
        }
    }
    // 销毁共享内存
    kupl_shm_win_free(win);
    // 销毁 kupl 通信域
    kupl_shm_comm_destroy(kupl_comm);
    MPI_Finalize();
    return 0;
}
```

运行结果如下。

```
check success
```

说明

上述示例演示了创建、查询、清理一个kupl win的流程。kupl_shm_win_query函数用于获取对端进程的共享内存首地址。

5.3.2.7.8 kupl_shm_win_free

清理win。

接口定义

int kupl_shm_win_free(kupl_shm_win_h win)

参数

表 5-80 参数定义

参数名	类型	描述	输入/输出
win	kupl_shm_win_h	需要清理的window对象	输入

返回值

- 成功：返回KUPL_OK
- 失败：返回KUPL_ERROR

示例

```
#include <stdio.h>
#include <mpi.h>
#include <unistd.h>
#include <string.h>
#include "kupl.h"

// 创建 kupl 通信域需要的回调函数 1
```

 版权所有 © 华为技术有限公司

```
static int oob_barrier_callback(void *group)
{
    return MPI_Barrier((MPI_Comm)group);
}
// 创建 kupl 通信域需要的回调函数 2
static int oob_allgather_callback(const void *sendbuf, void *recvbuf, int size, void *group,
                                  kupl_shm_datatype_t datatype)
{
    switch (datatype) {
        case KUPL_SHM_DATATYPE_CHAR:
            return MPI_Allgather(sendbuf, size, MPI_CHAR, recvbuf, size, MPI_CHAR, (MPI_Comm)group);
        default:
            fprintf(stderr, "not support datatype");
            return KUPL_ERROR;
    }
}
int main(int argc, char *argv[])
{
    // 初始化 MPI 环境
    MPI_Init(&argc, &argv);
    MPI_Comm comm = MPI_COMM_WORLD;
    // 获取 MPI 通信域大小
    int world_size;
    MPI_Comm_size(comm, &world_size);
    // 获取进程 rank 号
    int world_rank;
    MPI_Comm_rank(comm, &world_rank);
    // 获取进程 pid 号
    int pid = getpid();
    // 创建 kupl 通信域
    kupl_shm_oob_cb_t oob_cbs;
    kupl_shm_oob_cb_h oob_cbs_h = &oob_cbs;
    oob_cbs_h->oob_allgather = oob_allgather_callback;
    oob_cbs_h->oob_barrier = oob_barrier_callback;
    kupl_shm_comm_h kupl_comm;
    int ret = kupl_shm_comm_create(world_size, world_rank, pid, oob_cbs_h, (void *)comm, &kupl_comm);
    if (ret != KUPL_OK || kupl_comm == NULL) {
        fprintf(stderr, "kupl shm comm create failed: %d\n", ret);
        return -1;
    }
    // 申请共享内存
    kupl_shm_win_h win;
    void *local_buffer;
    int count = 4;
    size_t buf_size = count * sizeof(int);
    ret = kupl_shm_win_alloc(buf_size, kupl_comm, &local_buffer, &win);
    if (ret != KUPL_OK) {
        fprintf(stderr, "kupl shm win alloc failed: %d\n", ret);
        return -1;
    }
    // 进程 0 对本进程 buffer 赋值
    if (world_rank == 0) {
        for (int i = 0; i < count; i++) {
            ((int *)local_buffer)[i] = i;
        }
    }
    // 所有其他进程获取进程 0 的 buffer，进行 memcpy
    if (world_rank != 0) {
        void *remote_buffer;
        int remote_rank = 0;
        kupl_shm_win_query(win, remote_rank, &remote_buffer);
        memcpy(local_buffer, remote_buffer, buf_size);
        // 检查是否成功 memcpy
        bool check = true;
        for (int i = 0; i < count; i++) {
            if (((int *)local_buffer)[i] != i) {
                check = false;
                break;
            }
```

 版权所有 © 华为技术有限公司

```
        }
        if (check) {
            printf("check success\n");
        } else {
            printf("check fail\n");
        }
    }
    // 销毁共享内存
    kupl_shm_win_free(win);
    // 销毁 kupl 通信域
    kupl_shm_comm_destroy(kupl_comm);
    MPI_Finalize();
    return 0;
}
```

运行结果如下。

```
check success
```

说明

上述示例演示了创建、查询、清理一个kupl win的流程。kupl_shm_win_free函数清理kupl_shm_win_alloc创建的kupl win。

5.3.2.7.9 kupl_shm_attach

申请内存，并将申请的内存与另一进程的内存进行映射，实现共享内存的目的。

接口定义

void* kupl_shm_attach(struct kupl_shm_addr_t addr, size_t size)

参数

表 5-81 参数定义

参数名	类型	描述	输入/输出
addr	struct kupl_shm_addr_t	需要映射的内存信息	输入
size	size_t	需要映射的内存大小	输入

表 5-82 映射内存信息结构体 kupl_shm_addr_t 定义

参数名	类型	描述	输入/输出
src_addr	void*	需要映射的内存的虚拟地址	输入
src_pid	int	需要映射的内存所在的进程的进程号	输入
dst_pid	int	本进程的进程号	输入

版权所有 © 华为技术有限公司

返回值

- 成功：返回本进程申请并映射成功的内存指针ptr
- 失败：返回nullptr

示例

```
进程A
#include <stdio.h>
#include <stdlib.h>
#include <unistd.h>
#include <sys/mman.h>
#include <sys/stat.h>
#include <fcntl.h>
#include <string.h>
#include "kupl.h"
#define SHM_NAME "/pid_shm_example"
typedef struct {
    pid_t pid;
    int ready;
    void* addr;
} shared_data_t;
int main() {
    // 创建共享内存对象、填入数据
    int shm_fd = shm_open(SHM_NAME, O_CREAT | O_RDWR, 0666);
    ftruncate(shm_fd, sizeof(shared_data_t));
    shared_data_t *shared_data = mmap(NULL, sizeof(shared_data_t),
                                      PROT_READ | PROT_WRITE, MAP_SHARED, shm_fd, 0);
    // 写入PID到共享内存
    shared_data->pid = getpid();
    printf("Process A: PID %d\n", shared_data->pid);
    shared_data->ready = 1;
    shared_data->addr = kupl_malloc(KUPL_MEM_DEFAULT, sizeof(shared_data_t));
    memset(shared_data->addr, 0, sizeof(shared_data_t));
    data_addr->pid = getpid();
    // 保持运行，等待进程B读取
    printf("Press Enter to exit...\n");
    getchar();
    // 清理
    kupl_free(KUPL_MEM_DEFAULT, shared_data->addr);
    munmap(shared_data, sizeof(shared_data_t));
    shm_unlink(SHM_NAME);
    return 0;
}

进程B
#include <stdio.h>
#include <stdlib.h>
#include <unistd.h>
#include <sys/mman.h>
#include <sys/stat.h>
#include <fcntl.h>
#include <string.h>
#include "kupl.h"
#define SHM_NAME "/pid_shm_example"
typedef struct {
    pid_t pid;
    int ready;
    void* addr;
} shared_data_t;
int main() {
    // 打开共享内存对象
    int shm_fd = shm_open(SHM_NAME, O_RDONLY, 0666);
    shared_data_t *shared_data = mmap(NULL, sizeof(shared_data_t),
                                      PROT_READ, MAP_SHARED, shm_fd, 0);
    shared_data_t *attach_data = (shared_data_t*)calloc(1, sizeof(shared_data_t));
    // 等待数据就绪
    while (shared_data->ready == 0) {
```

 版权所有 © 华为技术有限公司

```
    usleep(100000); // 等待100ms
  }
  // 读取信息
  kupl_shm_addr_t kupl_shm_addr_info;
  kupl_shm_addr_info.src_addr = shared_data->addr;
  kupl_shm_addr_info.src_pid  = shared_data->pid;
  kupl_shm_addr_info.dst_pid  = getpid();
  // 执行映射
  attach_data->addr = kupl_shm_attach(kupl_shm_addr_info, sizeof(shared_data_t));
  if (attach_data->addr == NULL) {
    printf("attach wrong!\n");
  } else {
    printf("attach OK!");
  }
  // 等待数据就绪
  shared_data_t* data_addr = (shared_data_t*)attach_data->addr;
  while (data_addr->ready != 3) {
    usleep(100000); // 等待100ms
  }
  printf("Process B: Read PID %d from shared memory\n", data_addr->pid);
  // 清理
  munmap(shared_data, sizeof(shared_data_t));
  kupl_shm_detach(attach_data->addr);
  free(attach_data);
  close(shm_fd);
  return 0;
}
```

运行结果如下。

```
进程A
Process A:  PID xxxx
Press Enter to exit...

进程B
attach OK!
Process B: Read PID xxxx from shared memory //应与上文进程A打印的PID一致
```

说明

上述示例演示了使用kupl_shm_attach的申请内存并将申请的内存与另一进程的内存进行映射，之后解除由kupl_shm_attach接口建立的映射，并释放由kupl_shm_attach申请的内存的流程。在先后运行进程A和进程B的程序后，A进程后申请内存并写入数据。写入完毕后，会保持运行等待B进程读取完毕并在输入任意键后退出；运行B进程后则会调用kupl_shm_attach尝试进行内存映射。内存映射成功后，会开始尝试读取A进程写入的内容，读取到的内容应为A进程的pid。读取完毕后，B进程会调用detach解除绑定并退出。

5.3.2.7.10 kupl_shm_detach

解除由kupl_shm_attach接口建立的映射，并释放由kupl_shm_attach申请的内存。

接口定义

int kupl_shm_detach (void *ptr)

参数

表 5-83 参数定义

参数名	类型	描述	输入/输出
ptr	void*	需要解除映射并释放的内存指针	输入

 版权所有 © 华为技术有限公司

返回值

- 成功：返回KUPL_OK
- 失败：返回KUPL_ERROR

示例

```
进程A
#include <stdio.h>
#include <stdlib.h>
#include <unistd.h>
#include <sys/mman.h>
#include <sys/stat.h>
#include <fcntl.h>
#include <string.h>
#include "kupl.h"
#define SHM_NAME "/pid_shm_example"
typedef struct {
    pid_t pid;
    int ready;
    void* addr;
} shared_data_t;
int main() {
    // 创建共享内存对象、填入数据
    int shm_fd = shm_open(SHM_NAME, O_CREAT | O_RDWR, 0666);
    ftruncate(shm_fd, sizeof(shared_data_t);
    shared_data_t *shared_data = mmap(NULL, sizeof(shared_data_t),
                                      PROT_READ | PROT_WRITE, MAP_SHARED, shm_fd, 0);
    // 写入PID到共享内存
    shared_data->pid = getpid();
    printf("Process A: PID %d\n", shared_data->pid);
    shared_data->ready = 1;
    shared_data->addr = kupl_malloc(KUPL_MEM_DEFAULT, sizeof(shared_data_t));
    memset(shared_data->addr, 0, sizeof(shared_data_t));
    data_addr->pid = getpid();
    // 保持运行，等待进程B读取
    printf("Press Enter to exit...\n");
    getchar();
    // 清理


(KUPL_MEM_DEFAULT, shared_data->addr);
    munmap(shared_data, sizeof(shared_data_t));
    shm_unlink(SHM_NAME);
    return 0;
}

进程B
#include <stdio.h>
#include <stdlib.h>
#include <unistd.h>
#include <sys/mman.h>
#include <sys/stat.h>
#include <fcntl.h>
#include <string.h>
#include "kupl.h"
#define SHM_NAME "/pid_shm_example"
typedef struct {
    pid_t pid;
    int ready;
    void* addr;
} shared_data_t;
int main() {
    // 打开共享内存对象
    int shm_fd = shm_open(SHM_NAME, O_RDONLY, 0666);
```


版权所有 © 华为技术有限公司

```
    shared_data_t *shared_data = mmap(NULL, sizeof(shared_data_t),
                                      PROT_READ, MAP_SHARED, shm_fd, 0);
    shared_data_t *attach_data = (shared_data_t*)calloc(1, sizeof(shared_data_t));
    // 等待数据就绪
    while (shared_data->ready == 0) {
        usleep(100000); // 等待100ms
    }
    // 读取信息
    kupl_shm_addr_t kupl_shm_addr_info;
    kupl_shm_addr_info.src_addr = shared_data->addr;
    kupl_shm_addr_info.src_pid  = shared_data->pid;
    kupl_shm_addr_info.dst_pid  = getpid();
    // 执行映射
    attach_data->addr = kupl_shm_attach(kupl_shm_addr_info, sizeof(shared_data_t));
    if (attach_data->addr == NULL) {
        printf("attach wrong!\n");
    } else {
        printf("attach OK\n");
    }
    // 等待数据就绪
    shared_data_t* data_addr = (shared_data_t*)attach_data->addr;
    while (data_addr->ready != 3) {
        usleep(100000); // 等待100ms
    }
    printf("Process B: Read PID %d from shared memory\n", data_addr->pid);
    // 清理
    munmap(shared_data, sizeof(shared_data_t));
    kupl_shm_detach(attach_data->addr);
    free(attach_data);
    close(shm_fd);
    return 0;
}
```

运行结果如下。

```
进程A
Process A： PID xxxx
Press Enter to exit...

进程B
attach OK!
Process B: Read PID xxxx from shared memory //应与上文进程A打印的PID一致
```

说明

上述示例演示了使用kupl_shm_attach的申请内存并将申请的内存与另一进程的内存进行映射，之后解除由kupl_shm_attach接口建立的映射，并释放由kupl_shm_attach申请的内存的流程。在先后运行进程A和进程B的程序后，A进程后申请内存并写入数据。写入完毕后，会保持运行等待B进程读取完毕并在输入任意键后退出；运行B进程后则会调用kupl_shm_attach尝试进行内存映射。内存映射成功后，会开始尝试读取A进程写入的内容，读取到的内容应为A进程的pid。读取完毕后，B进程会调用detach解除绑定并退出。

5.3.2.7.11 kupl_shm_fence

win上的进程间同步。

接口定义

int kupl_shm_fence(kupl_shm_win_h win)

环境变量

使用环境变量 KUPL_SHM_FENCE_ALGORITHM控制fence的底层实现。

可选值：

 版权所有 © 华为技术有限公司

- 1：代表linear
- 2：代表rd（默认）

参数

表 5-84 参数定义

参数名	类型	描述	输入/输出
win	kupl_shm_win_h	需要同步的window对象	输入

返回值

- 成功：返回KUPL_OK
- 失败：返回KUPL_ERROR

示例

```
#include <stdio.h>
#include <mpi.h>
#include <unistd.h>
#include "kupl.h"

#define ITERS 5
// 创建 kupl 通信域需要的回调函数 1
static int oob_barrier_callback(void *group)
{
    return MPI_Barrier((MPI_Comm)group);
}
// 创建 kupl 通信域需要的回调函数 2
static int oob_allgather_callback(const void *sendbuf, void *recvbuf, int size, void *group,
                                  kupl_shm_datatype_t datatype)
{
    switch (datatype) {
        case KUPL_SHM_DATATYPE_CHAR:
            return MPI_Allgather(sendbuf, size, MPI_CHAR, recvbuf, size, MPI_CHAR, (MPI_Comm)group);
        default:
            fprintf(stderr, "not support datatype");
            return KUPL_ERROR;
    }
}
int main(int argc, char *argv[])
{
    // 初始化 MPI 环境
    MPI_Init(&argc, &argv);
    MPI_Comm comm = MPI_COMM_WORLD;
    // 获取 MPI 通信域大小
    int world_size;
    MPI_Comm_size(comm, &world_size);
    // 获取进程 rank 号
    int world_rank;
    MPI_Comm_rank(comm, &world_rank);
    // 获取进程 pid 号
    int pid = getpid();
    // 创建 kupl 通信域
    kupl_shm_oob_cb_t oob_cbs;
    kupl_shm_oob_cb_h oob_cbs_h = &oob_cbs;
    oob_cbs_h->oob_allgather = oob_allgather_callback;
    oob_cbs_h->oob_barrier = oob_barrier_callback;
    kupl_shm_comm_h kupl_comm;
```


版权所有 © 华为技术有限公司

```
    int ret = kupl_shm_comm_create(world_size, world_rank, pid, oob_cbs_h, (void *)comm, &kupl_comm);
    if (ret != KUPL_OK || kupl_comm == NULL) {
        fprintf(stderr, "kupl shm comm create failed: %d\n", ret);
        return -1;
    }
    kupl_shm_win_h win;
    void *baseptr;
    size_t bufsize = sizeof(int);
    ret = kupl_shm_win_alloc(bufsize, kupl_comm, &baseptr, &win);
    if (ret != KUPL_OK) {
        fprintf(stderr, "kupl baseptr alloc failed: %d\n", ret);
        return -1;
    }
    ((int *)baseptr)[0] = 0;
    for (int i = 0; i < ITERS; i++) {
        ((int *)baseptr)[0]++;
        kupl_shm_fence(win);
        if ((world_rank & 1) == 1) {
            usleep(100);
        } else {
            usleep(10000);
        }
    }
    kupl_shm_win_free(win);
    kupl_shm_comm_destroy(kupl_comm);
    MPI_Finalize();
    return 0;
}
```

说明

上述示例演示了使用kupl fence的流程。kupl_shm_fence函数用于win中所有进程间的同步。

5.3.2.7.12 kupl_shm_peer_fence

win上两个进程间的同步。

接口定义

int kupl_shm_peer_fence(kupl_shm_win_h win, int remote_rank);

参数

表 5-85 参数定义

参数名	类型	描述	输入/输出
win	kupl_shm_win_h	需要同步的window对象	输入
remote_rank	int	需要同步的对端进程号	输入

返回值

- 成功：返回KUPL_OK
- 失败：返回KUPL_ERROR

 版权所有 © 华为技术有限公司

示例

该示例需要使用两个进程运行

```
#include <stdio.h>
#include <mpi.h>
#include <unistd.h>
#include "kupl.h"

#define ITERS 5
// 创建 kupl 通信域需要的回调函数 1
static int oob_barrier_callback(void *group)
{
    return MPI_Barrier((MPI_Comm)group);
}
// 创建 kupl 通信域需要的回调函数 2
static int oob_allgather_callback(const void *sendbuf, void *recvbuf, int size, void *group,
                                  kupl_shm_datatype_t datatype)
{
    switch (datatype) {
        case KUPL_SHM_DATATYPE_CHAR:
            return MPI_Allgather(sendbuf, size, MPI_CHAR, recvbuf, size, MPI_CHAR, (MPI_Comm)group);
        default:
            fprintf(stderr, "not support datatype");
            return KUPL_ERROR;
    }
}
int main(int argc, char *argv[])
{
    // 初始化 MPI 环境
    MPI_Init(&argc, &argv);
    MPI_Comm comm = MPI_COMM_WORLD;
    // 获取 MPI 通信域大小
    int world_size;
    MPI_Comm_size(comm, &world_size);
    // peer fence 演示，使用2进程运行
    if (world_size != 2) {
        fprintf(stderr, "This demo only support two procs\n");
        MPI_Abort(comm, -1);
    }
    // 获取进程 rank 号
    int world_rank;
    MPI_Comm_rank(comm, &world_rank);
    // 获取进程 pid 号
    int pid = getpid();
    // 创建 kupl 通信域
    kupl_shm_oob_cb_t oob_cbs;
    kupl_shm_oob_cb_h oob_cbs_h = &oob_cbs;
    oob_cbs_h->oob_allgather = oob_allgather_callback;
    oob_cbs_h->oob_barrier = oob_barrier_callback;
    kupl_shm_comm_h kupl_comm;
    int ret = kupl_shm_comm_create(world_size, world_rank, pid, oob_cbs_h, (void *)comm, &kupl_comm);
    if (ret != KUPL_OK || kupl_comm == NULL) {
        fprintf(stderr, "kupl shm comm create failed: %d\n", ret);
        return -1;
    }
    kupl_shm_win_h win;
    void *baseptr;
    size_t bufsize = sizeof(int);
    ret = kupl_shm_win_alloc(bufsize, kupl_comm, &baseptr, &win);
    if (ret != KUPL_OK) {
        fprintf(stderr, "kupl baseptr alloc failed: %d\n", ret);
        return -1;
    }
    ((int *)baseptr)[0] = 0;
    // 进程0和进程1做peer fence
    for (int i = 0; i < ITERS; i++) {
        ((int *)baseptr)[0]++;
        if (world_rank == 0) {
```

 版权所有 © 华为技术有限公司

```
            kupl_shm_peer_fence(win, 1);
            usleep(100);
        } else {
            kupl_shm_peer_fence(win, 0);
            usleep(10000);
        }
    }
    kupl_shm_win_free(win);
    kupl_shm_comm_destroy(kupl_comm);
    MPI_Finalize();
    return 0;
}
```

说明

上述示例演示了使用kupl peer fence的流程。kupl_shm_peer_fence函数用于win中一对进程间的同步。

5.3.2.7.13 kupl_shm_allreduce_init

进程间allreduce初始化。

接口定义

int kupl_shm_allreduce_init(const void *sendbuf, void *recvbuf, int count, kupl_shm_datatype_t datatype, kupl_shm_reduce_op_t op, kupl_shm_comm_h comm, kupl_shm_request_t *request);

环境变量

allreduce 可以使用环境变量指定算法。

环境变量名称为KUPL_SHM_ALLREDUCE_ALGORITHM，可选值：

- 0：代表auto_tuning（默认）
- 1：代表linear算法
- 2：代表 rb（rh_rd）算法

参数

表 5-86 参数定义

参数名	类型	描述	输入/输出
sendbuf	const void *	需要的send buffer	输入
recvbuf	void *	需要的recv buffer	输入
count	int	需要的send count	输入

版权所有 © 华为技术有限公司

参数名	类型	描述	输入/输出
datatype	kupl_shm_datatype_t	需要的数据类型，可设置为KUPL_SHM_DATATYPE_CHAR, KUPL_SHM_DATATYPE_INT, KUPL_SHM_DATATYPE_LONG, KUPL_SHM_DATATYPE_FLOAT, KUPL_SHM_DATATYPE_DOUBLE	输入
op	kupl_shm_reduce_op_t	需要的归约操作，可设置为KUPL_SHM_REDUCE_OP_SUM	输入
comm	kupl_shm_comm_h	需要的kupl comm	输入
request	kupl_shm_request_t *	生成的持久化request	输出

返回值

- 成功：返回KUPL_OK
- 失败：返回KUPL_ERROR

示例

```
#include <stdio.h>
#include <mpi.h>
#include <unistd.h>
#include "kupl.h"

// 创建 kupl 通信域需要的回调函数 1
static int oob_barrier_callback(void *group)
{
    return MPI_Barrier((MPI_Comm)group);
}
// 创建 kupl 通信域需要的回调函数 2
static int oob_allgather_callback(const void *sendbuf, void *recvbuf, int size, void *group,
                                  kupl_shm_datatype_t datatype)
{
    switch (datatype) {
        case KUPL_SHM_DATATYPE_CHAR:
            return MPI_Allgather(sendbuf, size, MPI_CHAR, recvbuf, size, MPI_CHAR, (MPI_Comm)group);
        default:
            fprintf(stderr, "not support datatype");
            return KUPL_ERROR;
    }
}
int main(int argc, char *argv[])
{
    // 初始化 MPI 环境
```


版权所有 © 华为技术有限公司

```
MPI_Init(&argc, &argv);
MPI_Comm comm = MPI_COMM_WORLD;
// 获取 MPI 通信域大小
int world_size;
MPI_Comm_size(comm, &world_size);
// 获取进程 rank 号
int world_rank;
MPI_Comm_rank(comm, &world_rank);
// 获取进程 pid 号
int pid = getpid();
// 创建 kupl 通信域
kupl_shm_oob_cb_t oob_cbs;
kupl_shm_oob_cb_h oob_cbs_h = &oob_cbs;
oob_cbs_h->oob_allgather = oob_allgather_callback;
oob_cbs_h->oob_barrier = oob_barrier_callback;
kupl_shm_comm_h kupl_comm;
int ret = kupl_shm_comm_create(world_size, world_rank, pid, oob_cbs_h, (void *)comm, &kupl_comm);
if (ret != KUPL_OK || kupl_comm == NULL) {
    fprintf(stderr, "kupl shm comm create failed: %d\n", ret);
    return -1;
}
kupl_shm_win_h win_send;
kupl_shm_win_h win_recv;
kupl_shm_win_h ompi_win_recv;
void *sendbuf = NULL;
void *recvbuf = NULL;
void *ompibuf = NULL;
int count = 16000;
size_t buf_size = sizeof(int) * count;
ret = kupl_shm_win_alloc(buf_size, kupl_comm, &sendbuf, &win_send);
if (ret != KUPL_OK) {
    fprintf(stderr, "kupl sendbuf alloc failed: %d\n", ret);
    return -1;
}
ret = kupl_shm_win_alloc(buf_size, kupl_comm, &recvbuf, &win_recv);
if (ret != KUPL_OK) {
    fprintf(stderr, "kupl recvbuf alloc failed: %d\n", ret);
    kupl_shm_win_free(win_send);
    win_send = NULL;
    return -1;
}
ret = kupl_shm_win_alloc(buf_size, kupl_comm, &ompibuf, &ompi_win_recv);
if (ret != KUPL_OK) {
    fprintf(stderr, "kupl ompibuf alloc failed: %d\n", ret);
    kupl_shm_win_free(win_send);
    kupl_shm_win_free(win_recv);
    win_send = NULL;
    win_recv = NULL;
    return -1;
}
auto *t_sendbuf = static_cast<int *>(sendbuf);
auto *t_recvbuf = static_cast<int *>(recvbuf);
auto *t_ompibuf = static_cast<int *>(ompibuf);
for (int i = 0; i < count; i++) {
    t_sendbuf[i] = world_rank + i;
}
kupl_shm_request_h request;
MPI_Barrier(comm);
kupl_shm_allreduce_init(sendbuf, recvbuf, count, KUPL_SHM_DATATYPE_INT,
KUPL_SHM_REDUCE_OP_SUM, kupl_comm,
                        &request);
kupl_shm_request_start(request);
kupl_shm_request_wait(request);
kupl_shm_request_free(request);
MPI_Barrier(comm);
MPI_Allreduce(sendbuf, ompibuf, count, MPI_INT, MPI_SUM, comm);
int check = 0;
for (int i = 0; i < count; i++) {
    if (t_recvbuf[i] != t_ompibuf[i]) {
```


版权所有 © 华为技术有限公司

```
            check = 1;
            break;
        }
    }
    int result = 0;
    MPI_Reduce(&check, &result, 1, MPI_INT, MPI_SUM, 0, comm);
    if (result == 0) {
        printf("check success\n");
    } else {
        printf("check failed\n");
    }
    kupl_shm_win_free(win_send);
    kupl_shm_win_free(win_recv);
    kupl_shm_win_free(ompi_win_recv);
    win_send = NULL;
    win_recv = NULL;
    ompi_win_recv = NULL;
    kupl_shm_comm_destroy(kupl_comm);
    MPI_Finalize();
    return 0;
}
```

运行结果如下。

```
check success
```

说明

上述示例演示了kupl allreduce的流程。kupl_shm_allreduce_init函数初始化allreduce的request。

5.3.2.7.14 kupl_shm_bcast_init

进程间bcast初始化。

接口定义

int kupl_shm_bcast_init(void *buffer, int count, kupl_shm_datatype_t datatype, int root, kupl_shm_comm_h comm, kupl_shm_request_h *request);

环境变量

bcast 可以使用环境变量指定算法。

环境变量名称为KUPL_SHM_BCAST_ALGORITHM，可选值：

- 0：代表auto_tuning（默认）
- 1：代表linear算法
- 2：代表linear_opt算法
- 3：代表topo_aware_linear算法
- 4：代表topo_aware_linear_opt算法
- 5：代表ring_pipeline算法
- 6：代表linear_scatter_linear_allgather算法

版权所有 © 华为技术有限公司

参数

表 5-87 参数定义

参数名	类型	描述	输入/输出
buffer	void *	需要的buffer	输入
count	int	操作的元素个数	输入
datatype	kupl_shm_datatype_t	需要的数据类型，可设置为KUPL_SHM_DATATYPE_CHAR, KUPL_SHM_DATATYPE_INT, KUPL_SHM_DATATYPE_LONG, KUPL_SHM_DATATYPE_FLOAT, KUPL_SHM_DATATYPE_DOUBLE	输入
root	int	广播操作的根进程rank号	输入
comm	kupl_shm_comm_h	需要的kupl comm	输入
request	kupl_shm_request_t *	生成的持久化request	输出

返回值

- 成功：返回KUPL_OK
- 失败：返回KUPL_ERROR

示例

```
#include <stdio.h>
#include <mpi.h>
#include <unistd.h>
#include "kupl.h"

// 创建 kupl 通信域需要的回调函数 1
static int oob_barrier_callback(void *group)
{
    return MPI_Barrier((MPI_Comm)group);
}
// 创建 kupl 通信域需要的回调函数 2
static int oob_allgather_callback(const void *sendbuf, void *recvbuf, int size, void *group,
                                  kupl_shm_datatype_t datatype)
{
    switch (datatype) {
        case KUPL_SHM_DATATYPE_CHAR:
            return MPI_Allgather(sendbuf, size, MPI_CHAR, recvbuf, size, MPI_CHAR, (MPI_Comm)group);
        default:
            fprintf(stderr, "not support datatype");
```


版权所有 © 华为技术有限公司

```
        return KUPL_ERROR;
    }
}
int main(int argc, char *argv[])
{
    // 初始化 MPI 环境
    MPI_Init(&argc, &argv);
    MPI_Comm comm = MPI_COMM_WORLD;
    // 获取 MPI 通信域大小
    int world_size;
    MPI_Comm_size(comm, &world_size);
    // 获取进程 rank 号
    int world_rank;
    MPI_Comm_rank(comm, &world_rank);
    // 获取进程 pid 号
    int pid = getpid();
    // 创建 kupl 通信域
    kupl_shm_oob_cb_t oob_cbs;
    kupl_shm_oob_cb_h oob_cbs_h = &oob_cbs;
    oob_cbs_h->oob_allgather = oob_allgather_callback;
    oob_cbs_h->oob_barrier = oob_barrier_callback;
    kupl_shm_comm_h kupl_comm;
    int ret = kupl_shm_comm_create(world_size, world_rank, pid, oob_cbs_h, (void *)comm, &kupl_comm);
    if (ret != KUPL_OK || kupl_comm == NULL) {
        fprintf(stderr, "kupl shm comm create failed: %d\n", ret);
        return -1;
    }
    kupl_shm_win_h kupl_win;
    kupl_shm_win_h ompi_win;
    void *kuplbuf = NULL;
    void *ompibuf = NULL;
    int count = 16000;
    int broadcast_root = 0;
    size_t buf_size = sizeof(char) * count;
    ret = kupl_shm_win_alloc(buf_size, kupl_comm, &kuplbuf, &kupl_win);
    if (ret != KUPL_OK) {
        fprintf(stderr, "kupl kuplbuf alloc failed: %d\n", ret);
        return -1;
    }
    ret = kupl_shm_win_alloc(buf_size, kupl_comm, &ompibuf, &ompi_win);
    if (ret != KUPL_OK) {
        fprintf(stderr, "kupl ompibuf alloc failed: %d\n", ret);
        kupl_shm_win_free(kupl_win);
        return -1;
    }
    auto *t_kuplbuf = static_cast<char *>(kuplbuf);
    auto *t_ompibuf = static_cast<char *>(ompibuf);
    if (world_rank == broadcast_root) {
        for (int i = 0; i < count; i++) {
            t_kuplbuf[i] = i;
            t_ompibuf[i] = i;
        }
    }
    kupl_shm_request_h request;
    MPI_Barrier(comm);
    kupl_shm_bcast_init(kuplbuf, count, KUPL_SHM_DATATYPE_CHAR, broadcast_root, kupl_comm,
&request);
    kupl_shm_request_start(request);
    kupl_shm_request_wait(request);
    kupl_shm_request_free(request);
    MPI_Barrier(comm);
    MPI_Bcast(ompibuf, count, MPI_CHAR, broadcast_root, comm);
    int check = 0;
    for (int i = 0; i < count; i++) {
        if (t_kuplbuf[i] != t_ompibuf[i]) {
            check = 1;
            break;
        }
    }
```

 版权所有 © 华为技术有限公司

```
    int result = 0;
    MPI_Reduce(&check, &result, 1, MPI_INT, MPI_SUM, 0, comm);
    if (result == 0) {
        printf("check success\n");
    } else {
        printf("check failed\n");
    }
    kupl_shm_win_free(kupl_win);
    kupl_shm_win_free(ompi_win);
    kuplbuf = NULL;
    ompibuf = NULL;
    kupl_shm_comm_destroy(kupl_comm);
    MPI_Finalize();
    return 0;
}
```

运行结果如下。

```
check success
```

说明

上述示例演示了kupl bcast的流程。kupl_shm_bcast_init函数初始化bcast的request。

5.3.2.7.15 kupl_shm_alltoall_init

进程间alltoall初始化。

接口定义

int kupl_shm_alltoall_init(const void *sendbuf, int sendcount, kupl_shm_datatype_t sendtype, void *recvbuf, int recvcount, kupl_shm_datatype_t recvtype, kupl_shm_comm_h comm, kupl_shm_request_h *request);

环境变量

alltoall 可以使用环境变量指定算法。

环境变量名称为KUPL_SHM_ALLTOALL_ALGORITHM，可选值：

- 0：代表auto_tuning（默认）
- 1：代表linear_read算法
- 2：代表linear_write算法

参数

表 5-88 参数定义

参数名	类型	描述	输入/输出
sendbuf	const void *	需要的发送buffer	输入
sendcount	int	需要发送的元素个数	输入

版权所有 © 华为技术有限公司

参数名	类型	描述	输入/输出
sendtype	kupl_shm_datatype_t	需要的发送数据类型，可设置为KUPL_SHM_DATATYPE_CHAR, KUPL_SHM_DATATYPE_INT, KUPL_SHM_DATATYPE_LONG, KUPL_SHM_DATATYPE_FLOAT, KUPL_SHM_DATATYPE_DOUBLE	输入
recvbuf	void *	需要的接收buffer	输入
recvcount	int	需要接收的元素个数	输入
recvtype	kupl_shm_datatype_t	需要的接收数据类型，可设置为KUPL_SHM_DATATYPE_CHAR, KUPL_SHM_DATATYPE_INT, KUPL_SHM_DATATYPE_LONG, KUPL_SHM_DATATYPE_FLOAT, KUPL_SHM_DATATYPE_DOUBLE	输入
comm	kupl_shm_comm_h	需要的kupl comm	输入
request	kupl_shm_request_t *	生成的持久化request	输出

返回值

- 成功：返回KUPL_OK
- 失败：返回KUPL_ERROR

示例

```
#include <stdio.h>
#include <mpi.h>
#include <unistd.h>
#include "kupl.h"

// 创建 kupl 通信域需要的回调函数 1
static int oob_barrier_callback(void *group)
{
    return MPI_Barrier((MPI_Comm)group);
```


版权所有 © 华为技术有限公司

```
}
// 创建 kupl 通信域需要的回调函数 2
static int oob_allgather_callback(const void *sendbuf, void *recvbuf, int size, void *group,
                                  kupl_shm_datatype_t datatype)
{
    switch (datatype) {
        case KUPL_SHM_DATATYPE_CHAR:
            return MPI_Allgather(sendbuf, size, MPI_CHAR, recvbuf, size, MPI_CHAR, (MPI_Comm)group);
        default:
            fprintf(stderr, "not support datatype");
            return KUPL_ERROR;
    }
}
int main(int argc, char *argv[])
{
    // 初始化 MPI 环境
    MPI_Init(&argc, &argv);
    MPI_Comm comm = MPI_COMM_WORLD;
    // 获取 MPI 通信域大小
    int world_size;
    MPI_Comm_size(comm, &world_size);
    // 获取进程 rank 号
    int world_rank;
    MPI_Comm_rank(comm, &world_rank);
    // 获取进程 pid 号
    int pid = getpid();
    // 创建 kupl 通信域
    kupl_shm_oob_cb_t oob_cbs;
    kupl_shm_oob_cb_h oob_cbs_h = &oob_cbs;
    oob_cbs_h->oob_allgather = oob_allgather_callback;
    oob_cbs_h->oob_barrier = oob_barrier_callback;
    kupl_shm_comm_h kupl_comm;
    int ret = kupl_shm_comm_create(world_size, world_rank, pid, oob_cbs_h, (void *)comm, &kupl_comm);
    if (ret != KUPL_OK || kupl_comm == NULL) {
        fprintf(stderr, "kupl shm comm create failed: %d\n", ret);
        return -1;
    }
    kupl_shm_win_h win_send;
    kupl_shm_win_h win_recv;
    kupl_shm_win_h ompi_win_recv;
    void *sendbuf = NULL;
    void *recvbuf = NULL;
    void *ompibuf = NULL;
    int count = 1600;
    size_t buf_size = sizeof(char) * count * world_size;
    ret = kupl_shm_win_alloc(buf_size, kupl_comm, &sendbuf, &win_send);
    if (ret != KUPL_OK) {
        fprintf(stderr, "kupl sendbuf alloc failed: %d\n", ret);
        return -1;
    }
    ret = kupl_shm_win_alloc(buf_size, kupl_comm, &recvbuf, &win_recv);
    if (ret != KUPL_OK) {
        fprintf(stderr, "kupl recvbuf alloc failed: %d\n", ret);
        kupl_shm_win_free(win_send);
        win_send = NULL;
        return -1;
    }
    ret = kupl_shm_win_alloc(buf_size, kupl_comm, &ompibuf, &ompi_win_recv);
    if (ret != KUPL_OK) {
        fprintf(stderr, "kupl ompibuf alloc failed: %d\n", ret);
        kupl_shm_win_free(win_send);
        kupl_shm_win_free(win_recv);
        win_send = NULL;
        win_recv = NULL;
        return -1;
    }
    auto *t_sendbuf = static_cast<char *>(sendbuf);
    auto *t_recvbuf = static_cast<char *>(recvbuf);
    auto *t_ompibuf = static_cast<char *>(ompibuf);
```

 版权所有 © 华为技术有限公司

```
    for (int i = 0; i < count; i++) {
        t_sendbuf[i] = world_rank + i;
    }
    kupl_shm_request_h request;
    MPI_Barrier(comm);
    kupl_shm_alltoall_init(sendbuf, count, KUPL_SHM_DATATYPE_CHAR, recvbuf, count,
KUPL_SHM_DATATYPE_CHAR, kupl_comm,
                           &request);
    kupl_shm_request_start(request);
    kupl_shm_request_wait(request);
    kupl_shm_request_free(request);
    MPI_Barrier(comm);
    MPI_Alltoall(sendbuf, count, MPI_CHAR, ompibuf, count, MPI_CHAR, comm);
    int check = 0;
    for (int i = 0; i < count * world_size; i++) {
        if (t_recvbuf[i] != t_ompibuf[i]) {
            check = 1;
            break;
        }
    }
    int result = 0;
    MPI_Reduce(&check, &result, 1, MPI_INT, MPI_SUM, 0, comm);
    if (result == 0) {
        printf("check success\n");
    } else {
        printf("check failed\n");
    }
    kupl_shm_win_free(win_send);
    kupl_shm_win_free(win_recv);
    kupl_shm_win_free(ompi_win_recv);
    win_send = NULL;
    win_recv = NULL;
    ompi_win_recv = NULL;
    kupl_shm_comm_destroy(kupl_comm);
    MPI_Finalize();
    return 0;
}
```

运行结果如下。

```
check success
```

说明

上述示例演示了kupl alltoall的流程。kupl_shm_alltoall_init函数初始化alltoall的request。

5.3.2.7.16 kupl_shm_request_start

执行request中的集合通信操作。

接口定义

int kupl_shm_request_start(kupl_shm_request_h request);

参数

表 5-89 参数定义

参数名	类型	描述	输入/输出
request	kupl_shm_request_h	需要执行集合通信操作的request	输入

 版权所有 © 华为技术有限公司

返回值

- 成功：返回KUPL_OK
- 失败：返回KUPL_ERROR

示例

```
#include <stdio.h>
#include <mpi.h>
#include <unistd.h>
#include "kupl.h"

// 创建 kupl 通信域需要的回调函数 1
static int oob_barrier_callback(void *group)
{
    return MPI_Barrier((MPI_Comm)group);
}
// 创建 kupl 通信域需要的回调函数 2
static int oob_allgather_callback(const void *sendbuf, void *recvbuf, int size, void *group,
                                  kupl_shm_datatype_t datatype)
{
    switch (datatype) {
        case KUPL_SHM_DATATYPE_CHAR:
            return MPI_Allgather(sendbuf, size, MPI_CHAR, recvbuf, size, MPI_CHAR, (MPI_Comm)group);
        default:
            fprintf(stderr, "not support datatype");
            return KUPL_ERROR;
    }
}
int main(int argc, char *argv[])
{
    // 初始化 MPI 环境
    MPI_Init(&argc, &argv);
    MPI_Comm comm = MPI_COMM_WORLD;
    // 获取 MPI 通信域大小
    int world_size;
    MPI_Comm_size(comm, &world_size);
    // 获取进程 rank 号
    int world_rank;
    MPI_Comm_rank(comm, &world_rank);
    // 获取进程 pid 号
    int pid = getpid();
    // 创建 kupl 通信域
    kupl_shm_oob_cb_t oob_cbs;
    kupl_shm_oob_cb_h oob_cbs_h = &oob_cbs;
    oob_cbs_h->oob_allgather = oob_allgather_callback;
    oob_cbs_h->oob_barrier = oob_barrier_callback;
    kupl_shm_comm_h kupl_comm;
    int ret = kupl_shm_comm_create(world_size, world_rank, pid, oob_cbs_h, (void *)comm, &kupl_comm);
    if (ret != KUPL_OK || kupl_comm == NULL) {
        fprintf(stderr, "kupl shm comm create failed: %d\n", ret);
        return -1;
    }
    kupl_shm_win_h win_send;
    kupl_shm_win_h win_recv;
    kupl_shm_win_h ompi_win_recv;
    void *sendbuf = NULL;
    void *recvbuf = NULL;
    void *ompibuf = NULL;
    int count = 16000;
    size_t buf_size = sizeof(int) * count;
    ret = kupl_shm_win_alloc(buf_size, kupl_comm, &sendbuf, &win_send);
    if (ret != KUPL_OK) {
        fprintf(stderr, "kupl sendbuf alloc failed: %d\n", ret);
        return -1;
    }
    ret = kupl_shm_win_alloc(buf_size, kupl_comm, &recvbuf, &win_recv);
    if (ret != KUPL_OK) {
```

 版权所有 © 华为技术有限公司

```
        fprintf(stderr, "kupl recvbuf alloc failed: %d\n", ret);
        kupl_shm_win_free(win_send);
        win_send = NULL;
        return -1;
    }
    ret = kupl_shm_win_alloc(buf_size, kupl_comm, &ompibuf, &ompi_win_recv);
    if (ret != KUPL_OK) {
        fprintf(stderr, "kupl ompibuf alloc failed: %d\n", ret);
        kupl_shm_win_free(win_send);
        kupl_shm_win_free(win_recv);
        win_send = NULL;
        win_recv = NULL;
        return -1;
    }
    auto *t_sendbuf = static_cast<int *>(sendbuf);
    auto *t_recvbuf = static_cast<int *>(recvbuf);
    auto *t_ompibuf = static_cast<int *>(ompibuf);
    for (int i = 0; i < count; i++) {
        t_sendbuf[i] = world_rank + i;
    }
    kupl_shm_request_h request;
    MPI_Barrier(comm);
    kupl_shm_allreduce_init(sendbuf, recvbuf, count, KUPL_SHM_DATATYPE_INT,
KUPL_SHM_REDUCE_OP_SUM, kupl_comm,
                            &request);
    kupl_shm_request_start(request);
    kupl_shm_request_wait(request);
    kupl_shm_request_free(request);
    MPI_Barrier(comm);
    MPI_Allreduce(sendbuf, ompibuf, count, MPI_INT, MPI_SUM, comm);
    int check = 0;
    for (int i = 0; i < count; i++) {
        if (t_recvbuf[i] != t_ompibuf[i]) {
            check = 1;
            break;
        }
    }
    int result = 0;
    MPI_Reduce(&check, &result, 1, MPI_INT, MPI_SUM, 0, comm);
    if (result == 0) {
        printf("check success\n");
    } else {
        printf("check failed\n");
    }
    kupl_shm_win_free(win_send);
    kupl_shm_win_free(win_recv);
    kupl_shm_win_free(ompi_win_recv);
    win_send = NULL;
    win_recv = NULL;
    ompi_win_recv = NULL;
    kupl_shm_comm_destroy(kupl_comm);
    MPI_Finalize();
    return 0;
}
```

运行结果如下。

```
check success
```

说明

上述示例演示了kupl allreduce的流程。kupl_shm_request_start函数执行request。

5.3.2.7.17 kupl_shm_request_wait

阻塞等待request中的集合通信操作执行结束。

 版权所有 © 华为技术有限公司

接口定义

int kupl_shm_request_wait(kupl_shm_request_h request);

参数

表 5-90 参数定义

参数名	类型	描述	输入/输出
request	kupl_shm_request_h	需要阻塞等待完成的request	输入

返回值

- 成功：返回KUPL_OK
- 失败：返回KUPL_ERROR

示例

```
#include <stdio.h>
#include <mpi.h>
#include <unistd.h>
#include "kupl.h"

// 创建 kupl 通信域需要的回调函数 1
static int oob_barrier_callback(void *group)
{
    return MPI_Barrier((MPI_Comm)group);
}
// 创建 kupl 通信域需要的回调函数 2
static int oob_allgather_callback(const void *sendbuf, void *recvbuf, int size, void *group,
                                  kupl_shm_datatype_t datatype)
{
    switch (datatype) {
        case KUPL_SHM_DATATYPE_CHAR:
            return MPI_Allgather(sendbuf, size, MPI_CHAR, recvbuf, size, MPI_CHAR, (MPI_Comm)group);
        default:
            fprintf(stderr, "not support datatype");
            return KUPL_ERROR;
    }
}
int main(int argc, char *argv[])
{
    // 初始化 MPI 环境
    MPI_Init(&argc, &argv);
    MPI_Comm comm = MPI_COMM_WORLD;
    // 获取 MPI 通信域大小
    int world_size;
    MPI_Comm_size(comm, &world_size);
    // 获取进程 rank 号
    int world_rank;
    MPI_Comm_rank(comm, &world_rank);
    // 获取进程 pid 号
    int pid = getpid();
    // 创建 kupl 通信域
    kupl_shm_oob_cb_t oob_cbs;
    kupl_shm_oob_cb_h oob_cbs_h = &oob_cbs;
    oob_cbs_h->oob_allgather = oob_allgather_callback;
    oob_cbs_h->oob_barrier = oob_barrier_callback;
    kupl_shm_comm_h kupl_comm;
```


版权所有 © 华为技术有限公司

```
    int ret = kupl_shm_comm_create(world_size, world_rank, pid, oob_cbs_h, (void *)comm, &kupl_comm);
    if (ret != KUPL_OK || kupl_comm == NULL) {
        fprintf(stderr, "kupl shm comm create failed: %d\n", ret);
        return -1;
    }
    kupl_shm_win_h win_send;
    kupl_shm_win_h win_recv;
    kupl_shm_win_h ompi_win_recv;
    void *sendbuf = NULL;
    void *recvbuf = NULL;
    void *ompibuf = NULL;
    int count = 16000;
    size_t buf_size = sizeof(int) * count;
    ret = kupl_shm_win_alloc(buf_size, kupl_comm, &sendbuf, &win_send);
    if (ret != KUPL_OK) {
        fprintf(stderr, "kupl sendbuf alloc failed: %d\n", ret);
        return -1;
    }
    ret = kupl_shm_win_alloc(buf_size, kupl_comm, &recvbuf, &win_recv);
    if (ret != KUPL_OK) {
        fprintf(stderr, "kupl recvbuf alloc failed: %d\n", ret);
        kupl_shm_win_free(win_send);
        win_send = NULL;
        return -1;
    }
    ret = kupl_shm_win_alloc(buf_size, kupl_comm, &ompibuf, &ompi_win_recv);
    if (ret != KUPL_OK) {
        fprintf(stderr, "kupl ompibuf alloc failed: %d\n", ret);
        kupl_shm_win_free(win_send);
        kupl_shm_win_free(win_recv);
        win_send = NULL;
        win_recv = NULL;
        return -1;
    }
    auto *t_sendbuf = static_cast<int *>(sendbuf);
    auto *t_recvbuf = static_cast<int *>(recvbuf);
    auto *t_ompibuf = static_cast<int *>(ompibuf);
    for (int i = 0; i < count; i++) {
        t_sendbuf[i] = world_rank + i;
    }
    kupl_shm_request_h request;
    MPI_Barrier(comm);
    kupl_shm_allreduce_init(sendbuf, recvbuf, count, KUPL_SHM_DATATYPE_INT,
KUPL_SHM_REDUCE_OP_SUM, kupl_comm,
                            &request);
    kupl_shm_request_start(request);
    kupl_shm_request_wait(request);
    kupl_shm_request_free(request);
    MPI_Barrier(comm);
    MPI_Allreduce(sendbuf, ompibuf, count, MPI_INT, MPI_SUM, comm);
    int check = 0;
    for (int i = 0; i < count; i++) {
        if (t_recvbuf[i] != t_ompibuf[i]) {
            check = 1;
            break;
        }
    }
    int result = 0;
    MPI_Reduce(&check, &result, 1, MPI_INT, MPI_SUM, 0, comm);
    if (result == 0) {
        printf("check success\n");
    } else {
        printf("check failed\n");
    }
    kupl_shm_win_free(win_send);
    kupl_shm_win_free(win_recv);
    kupl_shm_win_free(ompi_win_recv);
    win_send = NULL;
    win_recv = NULL;
```

 版权所有 © 华为技术有限公司

```
    ompi_win_recv = NULL;
    kupl_shm_comm_destroy(kupl_comm);
    MPI_Finalize();
    return 0;
}
```

运行结果如下。

```
check success
```

说明

上述示例演示了kupl allreduce的流程。kupl_shm_request_wait函数等待request完成。

5.3.2.7.18 kupl_shm_request_free

清理request。

接口定义

int kupl_shm_request_free(kupl_shm_request_h request);

参数

表 5-91 参数定义

参数名	类型	描述	输入/输出
request	kupl_shm_request_h	需要清理的request	输入

返回值

- 成功：返回KUPL_OK
- 失败：返回KUPL_ERROR

示例

```
#include <stdio.h>
#include <mpi.h>
#include <unistd.h>
#include "kupl.h"

// 创建 kupl 通信域需要的回调函数 1
static int oob_barrier_callback(void *group)
{
    return MPI_Barrier((MPI_Comm)group);
}
// 创建 kupl 通信域需要的回调函数 2
static int oob_allgather_callback(const void *sendbuf, void *recvbuf, int size, void *group,
                                  kupl_shm_datatype_t datatype)
{
    switch (datatype) {
        case KUPL_SHM_DATATYPE_CHAR:
            return MPI_Allgather(sendbuf, size, MPI_CHAR, recvbuf, size, MPI_CHAR, (MPI_Comm)group);
        default:
            fprintf(stderr, "not support datatype");
            return KUPL_ERROR;
    }
}
```


版权所有 © 华为技术有限公司

```
int main(int argc, char *argv[])
{
    // 初始化 MPI 环境
    MPI_Init(&argc, &argv);
    MPI_Comm comm = MPI_COMM_WORLD;
    // 获取 MPI 通信域大小
    int world_size;
    MPI_Comm_size(comm, &world_size);
    // 获取进程 rank 号
    int world_rank;
    MPI_Comm_rank(comm, &world_rank);
    // 获取进程 pid 号
    int pid = getpid();
    // 创建 kupl 通信域
    kupl_shm_oob_cb_t oob_cbs;
    kupl_shm_oob_cb_h oob_cbs_h = &oob_cbs;
    oob_cbs_h->oob_allgather = oob_allgather_callback;
    oob_cbs_h->oob_barrier = oob_barrier_callback;
    kupl_shm_comm_h kupl_comm;
    int ret = kupl_shm_comm_create(world_size, world_rank, pid, oob_cbs_h, (void *)comm, &kupl_comm);
    if (ret != KUPL_OK || kupl_comm == NULL) {
        fprintf(stderr, "kupl shm comm create failed: %d\n", ret);
        return -1;
    }
    kupl_shm_win_h win_send;
    kupl_shm_win_h win_recv;
    kupl_shm_win_h ompi_win_recv;
    void *sendbuf = NULL;
    void *recvbuf = NULL;
    void *ompibuf = NULL;
    int count = 16000;
    size_t buf_size = sizeof(int) * count;
    ret = kupl_shm_win_alloc(buf_size, kupl_comm, &sendbuf, &win_send);
    if (ret != KUPL_OK) {
        fprintf(stderr, "kupl sendbuf alloc failed: %d\n", ret);
        return -1;
    }
    ret = kupl_shm_win_alloc(buf_size, kupl_comm, &recvbuf, &win_recv);
    if (ret != KUPL_OK) {
        fprintf(stderr, "kupl recvbuf alloc failed: %d\n", ret);
        kupl_shm_win_free(win_send);
        win_send = NULL;
        return -1;
    }
    ret = kupl_shm_win_alloc(buf_size, kupl_comm, &ompibuf, &ompi_win_recv);
    if (ret != KUPL_OK) {
        fprintf(stderr, "kupl ompibuf alloc failed: %d\n", ret);
        kupl_shm_win_free(win_send);
        kupl_shm_win_free(win_recv);
        win_send = NULL;
        win_recv = NULL;
        return -1;
    }
    auto *t_sendbuf = static_cast<int *>(sendbuf);
    auto *t_recvbuf = static_cast<int *>(recvbuf);
    auto *t_ompibuf = static_cast<int *>(ompibuf);
    for (int i = 0; i < count; i++) {
        t_sendbuf[i] = world_rank + i;
    }
    kupl_shm_request_h request;
    MPI_Barrier(comm);
    kupl_shm_allreduce_init(sendbuf, recvbuf, count, KUPL_SHM_DATATYPE_INT,
KUPL_SHM_REDUCE_OP_SUM, kupl_comm,
                            &request);
    kupl_shm_request_start(request);
    kupl_shm_request_wait(request);
    kupl_shm_request_free(request);
    MPI_Barrier(comm);
    MPI_Allreduce(sendbuf, ompibuf, count, MPI_INT, MPI_SUM, comm);
```

 版权所有 © 华为技术有限公司

```
    int check = 0;
    for (int i = 0; i < count; i++) {
        if (t_recvbuf[i] != t_ompibuf[i]) {
            check = 1;
            break;
        }
    }
    int result = 0;
    MPI_Reduce(&check, &result, 1, MPI_INT, MPI_SUM, 0, comm);
    if (result == 0) {
        printf("check success\n");
    } else {
        printf("check failed\n");
    }
    kupl_shm_win_free(win_send);
    kupl_shm_win_free(win_recv);
    kupl_shm_win_free(ompi_win_recv);
    win_send = NULL;
    win_recv = NULL;
    ompi_win_recv = NULL;
    kupl_shm_comm_destroy(kupl_comm);
    MPI_Finalize();
    return 0;
}
```

运行结果如下。

```
check success
```

说明

上述示例演示了kupl allreduce的流程。kupl_shm_request_free函数清理request。

5.3.2.7.19 kupl_shm_allreduce_batch_init

批量生成request。

接口定义

kupl_shm_allreduce_batch_init(void **sendbuf, void **recvbuf, int *count, kupl_shm_datatype *datatype, kupl_shm_reduce_op_t *op, kupl_shm_comm_h comm, kupl_shm_request_h *request, int request_num);

参数

表 5-92 参数定义

参数名	类型	描述	输入/输出
sendbuf	const void **	需要的send buffer数组	输入
recvbuf	void **	需要的recv buffer数组	输入
count	int *	需要的send count数组	输入

版权所有 © 华为技术有限公司

参数名	类型	描述	输入/输出
datatype	kupl_shm_datatype_t *	需要的数据类型数组，可设置为KUPL_SHM_DATATYPE_CHAR, KUPL_SHM_DATATYPE_INT, KUPL_SHM_DATATYPE_LONG, KUPL_SHM_DATATYPE_FLOAT, KUPL_SHM_DATATYPE_DOUBLE	输入
op	kupl_shm_reduce_op_t *	需要的归约操作数组，可设置为KUPL_SHM_REDUCE_OP_MAX, KUPL_SHM_REDUCE_OP_MIN, KUPL_SHM_REDUCE_OP_SUM	输入
comm	kupl_shm_comm_h	需要的kupl comm	输入
request	kupl_shm_request_t *	生成的持久化request数组	输出
request_num	kupl_shm_request_h *	需要批量生成request的数量	输入

返回值

- 成功：返回KUPL_OK
- 失败：返回KUPL_ERROR

示例

```
#include <stdio.h>
#include <mpi.h>
#include <unistd.h>
#include "kupl.h"

// 创建 kupl 通信域需要的回调函数 1
static int oob_barrier_callback(void *group)
{
    return MPI_Barrier((MPI_Comm)group);
}
// 创建 kupl 通信域需要的回调函数 2
static int oob_allgather_callback(const void *sendbuf, void *recvbuf, int size, void *group,
                                  kupl_shm_datatype_t datatype)
{
    switch (datatype) {
        case KUPL_SHM_DATATYPE_CHAR:
            return MPI_Allgather(sendbuf, size, MPI_CHAR, recvbuf, size, MPI_CHAR, (MPI_Comm)group);
```


版权所有 © 华为技术有限公司

```
        default:
            fprintf(stderr, "not support datatype");
            return KUPL_ERROR;
    }
}

int main(int argc, char *argv[])
{
   int check = 1;
    // 初始化 MPI 环境
    MPI_Init(&argc, &argv);
    MPI_Comm comm = MPI_COMM_WORLD;
    // 获取 MPI 通信域大小
    int world_size;
    MPI_Comm_size(comm, &world_size);
    // 获取进程 rank 号
    int world_rank;
    MPI_Comm_rank(comm, &world_rank);
    // 获取进程 pid 号
    int pid = getpid();


    // 创建 kupl 通信域
    kupl_shm_oob_cb_t oob_cbs;
    kupl_shm_oob_cb_h oob_cbs_h = &oob_cbs;
    oob_cbs_h->oob_allgather = oob_allgather_callback;
    oob_cbs_h->oob_barrier = oob_barrier_callback;
    kupl_shm_comm_h kupl_comm;
    int ret = kupl_shm_comm_create(world_size, world_rank, pid, oob_cbs_h, (void *)comm, &kupl_comm);
    if (ret != KUPL_OK || kupl_comm == NULL) {
        fprintf(stderr, "kupl shm comm create failed: %d\n", ret);
        return -1;
    }
     kupl_shm_win_h win_send;
     kupl_shm_win_h win_recv;
     void *sendbuf;
     void *recvbuf;
     int iter = 2;
     int unit_count = 8000;
     int count = unit_count * iter;
     size_t buf_size = sizeof(double) * count;
     size_t dtmp_buf_size = sizeof(double) * unit_count;
     kupl_shm_win_alloc(buf_size, kupl_comm, &sendbuf, &win_send);
     kupl_shm_win_alloc(dtmp_buf_size, kupl_comm, &recvbuf, &win_recv);

     // set sendbuf
     for (int i = 0; i < count; i++) {
         ((double *)sendbuf)[i] = (i + 1) * 1.0;
     }

     kupl_shm_request_h *request = (kupl_shm_request_h *) malloc(sizeof (kupl_shm_request_h) * iter);
     kupl_shm_datatype datatype[iter];
     kupl_shm_reduce_op_t reduce[iter];
     int count_list[iter];
     void **sendbuf_list = (void **) malloc(sizeof (void *) * iter);
     void **recvbuf_list = (void **) malloc(sizeof (void *) * iter);

     for (int i = 0; i < iter; i++) {
         sendbuf_list[i] = static_cast<double *>(sendbuf) + unit_count * i;
         recvbuf_list[i] = static_cast<double *>(recvbuf);
         reduce[i] = KUPL_SHM_REDUCE_OP_SUM;
         count_list[i] = unit_count;
         datatype[i] = KUPL_SHM_DATATYPE_DOUBLE;
     }
     kupl_shm_allreduce_batch_init(sendbuf_list, recvbuf_list, count_list, datatype,
                                   reduce, kupl_comm, request, iter);
     for (int i = 0; i < iter; ++i) {
         kupl_shm_request_start(request[i]);
         kupl_memcpy(sendbuf_list[i], recvbuf, unit_count * sizeof(double));
```


版权所有 © 华为技术有限公司

```
        kupl_shm_request_free(request[i]);
    }

    for (int i = 0; i < count; i++) {
       double expected = (i + 1) * 1.0 * world_size;
        if (expected != ((double *)sendbuf)[i]) {
            check = 0;
        }
    }

    int result;
    MPI_Reduce(&check, &result, 1, MPI_INT, MPI_SUM, 0, comm);
    if (result == 0) {
        printf("check success\n");
    } else {
        printf("check failed\n");
    }

    kupl_shm_win_free(win_send);
    kupl_shm_win_free(win_recv);
   kupl_shm_comm_destroy(kupl_comm);
   MPI_Finalize();
   return 0;
}
```

运行结果如下。

```
check success
```

说明

上述示例演示了kupl allreduce batch_init的流程。 kupl_shm_allreduce_batch_init批量创建request。

5.3.2.7.20 kupl_shm_info_set

设置KUPL shm参数。

接口定义

int kupl_shm_info_set(kupl_info_flag_t info_flag, uint32_t value);

参数

表 5-93 参数定义

参数名	类型	描述	输入/输出
info_flag	kupl_info_flag_t	参数的类型，KUPL_SHM_INFO_IS_CONTIG	输入
value	uint32_t	参数的值， • 0：多进程申请非连续的共享内存 • 1：多进程申请连续的共享内存	输入

版权所有 © 华为技术有限公司

返回值

- 成功：返回KUPL_OK
- 失败：返回KUPL_ERROR

示例

```
#include <stdio.h>
#include <mpi.h>
#include <unistd.h>
#include "kupl.h"

// 创建 kupl 通信域需要的回调函数 1
static int oob_barrier_callback(void *group)
{
    return MPI_Barrier((MPI_Comm)group);
}
// 创建 kupl 通信域需要的回调函数 2
static int oob_allgather_callback(const void *sendbuf, void *recvbuf, int size, void *group,
                                  kupl_shm_datatype_t datatype)
{
    switch (datatype) {
        case KUPL_SHM_DATATYPE_CHAR:
            return MPI_Allgather(sendbuf, size, MPI_CHAR, recvbuf, size, MPI_CHAR, (MPI_Comm)group);
        default:
            fprintf(stderr, "not support datatype");
            return KUPL_ERROR;
    }
}
int main(int argc, char *argv[])
{
    // 初始化 MPI 环境
    MPI_Init(&argc, &argv);
    MPI_Comm comm = MPI_COMM_WORLD;
    // 获取 MPI 通信域大小
    int world_size;
    MPI_Comm_size(comm, &world_size);
    // 获取进程 rank 号
    int world_rank;
    MPI_Comm_rank(comm, &world_rank);
    // 获取进程 pid 号
    int pid = getpid();
    // 创建 kupl 通信域
    kupl_shm_info_set(KUPL_SHM_INFO_IS_CONTIG, 1);
    kupl_shm_oob_cb_t oob_cbs;
    kupl_shm_oob_cb_h oob_cbs_h = &oob_cbs;
    oob_cbs_h->oob_allgather = oob_allgather_callback;
    oob_cbs_h->oob_barrier = oob_barrier_callback;
    kupl_shm_comm_h kupl_comm;
    int ret = kupl_shm_comm_create(world_size, world_rank, pid, oob_cbs_h, (void *)comm, &kupl_comm);
    if (ret != KUPL_OK || kupl_comm == NULL) {
        fprintf(stderr, "kupl shm comm create failed: %d\n", ret);
        return -1;
    }
    int kupl_rank;
    int kupl_size;
    kupl_shm_comm_rank(kupl_comm, &kupl_rank);
    kupl_shm_comm_size(kupl_comm, &kupl_size);
    kupl_shm_comm_destroy(kupl_comm);
    MPI_Finalize();
    return 0;
}
```

5.3.2.8 矩阵编程接口函数

版权所有 © 华为技术有限公司

5.3.2.8.1 概念说明

KUPL矩阵编程模块对外提供了Tensor概念及基于Tensor对象的一系列操作行为，从而便于用户在鲲鹏硬件平台上快速使能发挥Matrix computation能力加速矩阵乘相关算子。具体操作行为包含针对Tensor对象的取下标、相加、标量乘等operator操作，除此之外还有基于Tensor对象的KUPL MMA和Copy操作，其中MMA实现了D=A*B+C形式的矩阵乘kernel能力，Copy实现了不同内存空间排布的矩阵之间快速转置或者预取的能力。

KUPL矩阵编程模块相关概念如下：

- Int，整数包装类型，用于传递编译期整型常量。
- Ops，枚举方法包装类型，用于传递编译期枚举常量。
- Coord，表示矩阵下标，用于Tensor对象的取下标操作。
- Shape，表示矩阵尺寸或计算尺寸。
- Stride，表示矩阵元素行/列跨度。
- Layout，表示矩阵内存布局。
- Tensor，表示矩阵对象，包含源数据与布局。

使用KUPL矩阵编程模块的接口，需在支持Matrix computation指令的环境下安装HPCKit。

下述编程接口说明中的示例均为基于Bisheng版本KUPL矩阵编程接口实现；GCC版本受限于Matrix computation指令的支持，实现上存在差异。

5.3.2.8.2 Tensor

KUPL矩阵编程模块中的Tensor类用以描述矩阵对象，具体包含源数据与布局。用户通过定义创建KUPL Tensor从而让KUPL感知到用户的矩阵相关信息，用于后续Tensor operator、MMA、Copy等操作行为。

类定义

template <typename dtype, typename Layout>

class Tensor {};

类成员变量

表 5-94 类成员变量定义

成员变量名	成员变量类型	描述
ptr_	dtype*	存储用户矩阵对象的原始内存空间，从而供矩阵编程模块后续操作感知；其中dtype为类模板参数，表示用户矩阵对象的精度类型。
layout_	Layout	描述用户矩阵对象的内存排布信息，从而确保矩阵编程模块后续操作的正确性。

版权所有 © 华为技术有限公司

类成员函数

一、Tensor对象取下标及切片能力：Tensor(coord)

表 5-95 类成员函数 Tensor()输入输出定义

参数名	类型	描述	输入/输出
coord	Coord	表示需要获取的具体坐标；当输入Coord类型中带有Underscore类型时表示Tensor切片能力	输入
value/tensor	dtype/Tensor	Tensor对象取下标及切片能力的返回值信息，当坐标具体到特定点时返回具体坐标的元素值，当输入Coord类型中带有Underscore类型时返回Tensor切片后的子Tensor对象	输出

二、Tensor对象逐元素相加能力：TensorC = TensorA + TensorB

表 5-96 类成员函数 Tensor operator+输入输出定义

参数名	类型	描述	输入/输出
tensorA	Tensor	Tensor逐元素相加操作的A矩阵	输入
tensorB	Tensor	Tensor逐元素相加操作的B矩阵	输入
tensorC	Tensor	Tensor逐元素相加操作的结果输出C矩阵，其中A/B/C矩阵拥有相同的Layout排布	输出

三、Tensor对象标量乘能力：TensorC = A * TensorB / TensorC = TensorB * A

表 5-97 类成员函数 Tensor operator*输入输出定义

参数名	类型	描述	输入/输出
A	dtype	Tensor标量乘操作的标量元素	输入
tensorB	Tensor	Tensor标量乘操作的被乘矩阵	输入

版权所有 © 华为技术有限公司

参数名	类型	描述	输入/输出
tensorC	Tensor	Tensor标量乘操作的相乘结果输出C矩阵，其中B/C矩阵拥有相同的Layout排布	输出

示例

```
#include "stdlib.h"
#include "kupl_mma.h"
using namespace kupl::tensor;

int main()
{
    constexpr int MATRIX_M  = 32;
    constexpr int MATRIX_N  = 16;
    double *data_a = (double *)malloc(sizeof(double) * MATRIX_M * MATRIX_N);
    double *data_b = (double *)malloc(sizeof(double) * MATRIX_M * MATRIX_N);
    double *data_c = (double *)malloc(sizeof(double) * MATRIX_M * MATRIX_N);

    auto shape = make_shape(Int<32>{}, Int<16>{});
    auto stride = make_stride(Int<16>{}, Int<1>{});
    auto layout = make_layout(shape, stride);
    atuo tensor_a = make_tensor(data_a, layout);
    auto tensor_b = make_tensor(data_b, layout);
    auto tensor_c = make_tensor(data_c, layout);

    // Tensor对象取下标及切片操作如下所述
    // 获取下标为（2，2）的Tensor元素
    auto coord1 = make_coord(Int<2>{}, Int<2>{});
    auto ret1 = tensor(coord1);
    // 通过Underscore类型变量实现Tensor切片效果，coord2表示获取第2行所有Tensor元素
    auto coord2 = make_coord(Int<2>{}, Underscore{});
    auto ret2 = tensor(coord2);
    auto coord3 = make_coord(Int<2>{});
    auto ret3 = ret2(coord3);

    // Tensor对象逐元素相加操作如下所述
    tensor_c = tensor_a + tensor_b;

    // Tensor对象标量乘操作如下所述
    tensor_c = tensor_b * 2.0;
    tensor_c = 2.0 * tensor_a;

    free(data_c);
    free(data_b);
    free(data_a);
    return 0;
}
```

说明

上述示例演示了基于32*16*512_F64F64F64矩阵形状的mma流程，其中通过make_tensor创建矩阵对象tensor，作为后续mma/store接口参数。

5.3.2.8.3 make_coord

创建Coord，用于描述矩阵下标，从而用于Tensor对象的取下标操作。

接口定义

template<typename... Args>

版权所有 © 华为技术有限公司

Coord<Args...> make_coord(Args ... args);

参数

表 5-98 参数定义

参数名	类型	描述	输入/输出
args...	Args...，支持任意Int<>、Underscore或int类型对象及任意Coord<>类型对象作为输入。	用于定义矩阵下标。对于非峰形矩阵只需输入Int<>、Underscore或int类型对象，其中Int<>和Underscore类型变量表示编译期可确认变量，Underscore具体表示该维度所有下标；int类型变量表示运行期可确定变量。对于峰形矩阵需要输入Shape<>类型对象进行嵌套描述。	输入

返回值

返回Coord<Args...>对象。

示例

```
#include "stdlib.h"
#include "kupl_mma.h"
using namespace kupl::tensor;

int main()
{
    constexpr int MATRIX_M  = 32;
    constexpr int MATRIX_N  = 16;
    double *data = (double *)malloc(sizeof(double) * MATRIX_M * MATRIX_N);

    auto shape = make_shape(Int<32>{}, Int<16>{});
    auto stride = make_stride(Int<16>{}, Int<1>{});
    auto layout = make_layout(shape, stride);
    atuo tensor = make_tensor(data, layout);

    // 获取下标为（2，2）的Tensor元素
    auto coord1 = make_coord(Int<2>{}, Int<2>{});
    auto ret1 = tensor(coord1);
    // 通过Underscore类型变量实现Tensor切片效果，coord2表示获取第2行所有Tensor元素
    auto coord2 = make_coord(Int<2>{}, Underscore{});
    auto ret2 = tensor(coord2);
    auto coord3 = make_coord(Int<2>{});
    auto ret3 = ret2(coord3);

    free(data);
    return 0;
}
```

说明

上述示例演示了针对32*16的Tensor对象的取下标和切片操作，其中make_coord表示如何生成Coord下标对象。

版权所有 © 华为技术有限公司

5.3.2.8.4 make_shape

创建Shape，用于规定矩阵形状或计算尺寸。

接口定义

template<typename... Args>

Shape<Args...> make_shape(Args ... args);

参数

表 5-99 参数定义

参数名	类型	描述	输入/输出
args...	Args...，支持任意Int<>类型对象及任意Shape<>类型对象作为输入。	用于定义矩阵形状或计算尺寸。对于非峰形矩阵只需输入Int<>类型对象，对于峰形矩阵需要输入Shape<>类型对象进行嵌套描述。	输入

返回值

返回Shape<Args...>对象。

示例

```
#include "stdlib.h"
#include "kupl_mma.h"
using namespace kupl::tensor;

int main()
{
    constexpr int MATRIX_M  = 32;
    constexpr int MATRIX_N  = 16;
    constexpr int MATRIX_K = 512;
    double *data_a = (double *)malloc(sizeof(double) * MATRIX_M * MATRIX_K);
    double *data_b = (double *)malloc(sizeof(double) * MATRIX_K * MATRIX_N);
    double *data_c = (double *)malloc(sizeof(double) * MATRIX_M * MATRIX_N);

    auto shape_a = make_shape(Int<32>{}, Int<512>{});
    auto shape_b = make_shape(Int<512>{}, Int<16>{});
    auto shape_c = make_shape(Int<32>{}, Int<16>{});

    auto stride_a = make_stride(Int<1>{}, Int<32>{});
    auto stride_b = make_stride(Int<16>{}, Int<1>{});
    auto stride_c = make_stride(Int<16>{}, Int<1>{});

    auto layout_a = make_layout(shape_a, stride_a);
    auto layout_b = make_layout(shape_b, stride_b);
    auto layout_c = make_layout(shape_c, stride_c);

    auto mma_atom_shape = make_shape(Int<1>{}, Int<1>{}, Int<1>{});
    auto tiled_mma = make_tiled_mma(Ops<KP36_32x16x512_F64F64F64>{}, mma_atom_shape);
    auto store_atom_shape = make_shape(Int<1>{}, Int<1>{});
    auto tiled_store = make_tiled_store(Ops<KP36_32x16_F64_STORE>{}, store_atom_shape);
```


版权所有 © 华为技术有限公司

```
    auto tensor_a = make_tensor(data_a, layout_a);
    auto tensor_b = make_tensor(data_b, layout_b);
    auto tensor_c = make_tensor(data_c, layout_c);

    mma(tiled_mma, tensor_c, tensor_a, tensor_b, tensor_c);
    store(tiled_store, tensor_c);

    free(data_a);
    free(data_b);
    free(data_c);
    return 0;
}
```

说明

上述示例演示了基于32*16*512_F64F64F64矩阵形状的mma流程，其中通过make_shape创建shape，用于规定矩阵形状或计算尺寸。

5.3.2.8.5 make_stride

创建Stride，用于规定矩阵行/列方向元素跨度。

接口定义

template<typename... Args>

Stride<Args...> make_stride(Args ... args);

参数

表 5-100 参数定义

参数名	类型	描述	输入/输出
args...	Args...，支持任意Int<>类型对象及任意Stride<>类型对象作为输入。	用于定义矩阵行/列方向元素跨度。对于非峰形矩阵只需输入Int<>类型对象，对于峰形矩阵需要输入Stride<>类型对象进行嵌套描述。	输入

返回值

返回Stride<Args...>对象

示例

```
#include "stdlib.h"
#include "kupl_mma.h"
using namespace kupl::tensor;

int main()
{
    constexpr int MATRIX_M  = 32;
    constexpr int MATRIX_N  = 16;
    constexpr int MATRIX_K = 512;
    double *data_a = (double *)malloc(sizeof(double) * MATRIX_M * MATRIX_K);
```


版权所有 © 华为技术有限公司

```
    double *data_b = (double *)malloc(sizeof(double) * MATRIX_K * MATRIX_N);
    double *data_c = (double *)malloc(sizeof(double) * MATRIX_M * MATRIX_N);

    auto shape_a = make_shape(Int<32>{}, Int<512>{});
    auto shape_b = make_shape(Int<512>{}, Int<16>{});
    auto shape_c = make_shape(Int<32>{}, Int<16>{});

    auto stride_a = make_stride(Int<1>{}, Int<32>{});
    auto stride_b = make_stride(Int<16>{}, Int<1>{});
    auto stride_c = make_stride(Int<16>{}, Int<1>{});

    auto layout_a = make_layout(shape_a, stride_a);
    auto layout_b = make_layout(shape_b, stride_b);
    auto layout_c = make_layout(shape_c, stride_c);

    auto mma_atom_shape = make_shape(Int<1>{}, Int<1>{}, Int<1>{});
    auto tiled_mma = make_tiled_mma(Ops<KP36_32x16x512_F64F64F64>{}, mma_atom_shape);
    auto store_atom_shape = make_shape(Int<1>{}, Int<1>{});
    auto tile_store = make_tiled_store(Ops<KP36_32x16_F64_STORE>{}, store_atom_shape);

    auto tensor_a = make_tensor(data_a, layout_a);
    auto tensor_b = make_tensor(data_b, layout_b);
    auto tensor_c = make_tensor(data_c, layout_c);

    tensor_tiled_mma(tiled_mma, tensor_c, tensor_a, tensor_b, tensor_c);
    tensor_tiled_store(tile_store, tensor_c);

    free(data_a);
    free(data_b);
    free(data_c);
    return 0;
}
```

说明

上述示例演示了基于32*16*512_F64F64F64矩阵形状的mma流程，其中通过make_stride创建stride，用于规定矩阵行/列方向的元素跨度。

5.3.2.8.6 make_layout

创建Layout，规定矩阵内存布局。

接口定义

template <typename Shape, typename Stride>

Layout<Shape,Stride> make_layout(Shape shape, Stride stride);

模板参数

表 5-101 模板参数定义

参数名	类型	描述
Shape	typename	矩阵形状类型。
Stride	typename	矩阵元素跨度类型。

版权所有 © 华为技术有限公司

参数

表 5-102 参数定义

参数名	类型	描述	输入/输出
shape	Shape	矩阵形状。	输入
stride	Stride	矩阵元素跨度。	输入

返回值

返回Layout<Shape,Stride>对象

示例

```
#include "stdlib.h"
#include "kupl_mma.h"
using namespace kupl::tensor;

int main()
{
    constexpr int MATRIX_M  = 32;
    constexpr int MATRIX_N  = 16;
    constexpr int MATRIX_K = 512;
    double *data_a = (double *)malloc(sizeof(double) * MATRIX_M * MATRIX_K);
    double *data_b = (double *)malloc(sizeof(double) * MATRIX_K * MATRIX_N);
    double *data_c = (double *)malloc(sizeof(double) * MATRIX_M * MATRIX_N);

    auto shape_a = make_shape(Int<32>{}, Int<512>{});
    auto shape_b = make_shape(Int<512>{}, Int<16>{});
    auto shape_c = make_shape(Int<32>{}, Int<16>{});

    auto stride_a = make_stride(Int<1>{}, Int<32>{});
    auto stride_b = make_stride(Int<16>{}, Int<1>{});
    auto stride_c = make_stride(Int<16>{}, Int<1>{});

    auto layout_a = make_layout(shape_a, stride_a);
    auto layout_b = make_layout(shape_b, stride_b);
    auto layout_c = make_layout(shape_c, stride_c);

    auto mma_atom_shape = make_shape(Int<1>{}, Int<1>{}, Int<1>{});
    auto tiled_mma = make_tiled_mma(Ops<KP36_32x16x512_F64F64F64>{}, mma_atom_shape);
    auto store_atom_shape = make_shape(Int<1>{}, Int<1>{});
    auto tiled_store = make_tiled_store(Ops<KP36_32x16_F64_STORE>{}, store_atom_shape);

    auto tensor_a = make_tensor(data_a, layout_a);
    auto tensor_b = make_tensor(data_b, layout_b);
    auto tensor_c = make_tensor(data_c, layout_c);

    mma(tiled_mma, tensor_c, tensor_a, tensor_b, tensor_c);
    store(tiled_store, tensor_c);

    free(data_a);
    free(data_b);
    free(data_c);
    return 0;
}
```

说明

上述示例演示了基于32*16*512_F64F64F64矩阵形状的mma流程，其中通过make_layout创建矩阵布局。

版权所有 © 华为技术有限公司

5.3.2.8.7 make_tensor

创建Tensor对象，包含源数据及内存布局。

接口定义

template<typename dtype, typename Layout>

Tensor<dtype, Layout> make_tensor(dtype *ptr, Layout layout);

模板参数

表 5-103 模板参数定义

参数名	类型	描述
dtype	typename	精度类型。
Layout	typename	布局类型。

参数

表 5-104 参数定义

参数名	类型	描述	输入/输出
ptr	dtype *	矩阵源数据指针。	输入
layout	Layout	矩阵内存布局。	输入

返回值

返回Tensor<dtype, Layout>对象

示例

```
#include "stdlib.h"
#include "kupl_mma.h"
using namespace kupl::tensor;

int main()
{
    constexpr int MATRIX_M  = 32;
    constexpr int MATRIX_N  = 16;
    constexpr int MATRIX_K = 512;
    double *data_a = (double *)malloc(sizeof(double) * MATRIX_M * MATRIX_K);
    double *data_b = (double *)malloc(sizeof(double) * MATRIX_K * MATRIX_N);
    double *data_c = (double *)malloc(sizeof(double) * MATRIX_M * MATRIX_N);

    auto shape_a = make_shape(Int<32>{}, Int<512>{});
    auto shape_b = make_shape(Int<512>{}, Int<16>{});
    auto shape_c = make_shape(Int<32>{}, Int<16>{});

    auto stride_a = make_stride(Int<1>{}, Int<32>{});
    auto stride_b = make_stride(Int<16>{}, Int<1>{});
```


版权所有 © 华为技术有限公司

```
    auto stride_c = make_stride(Int<16>{}, Int<1>{});

    auto layout_a = make_layout(shape_a, stride_a);
    auto layout_b = make_layout(shape_b, stride_b);
    auto layout_c = make_layout(shape_c, stride_c);

    auto mma_atom_shape = make_shape(Int<1>{}, Int<1>{}, Int<1>{});
    auto tiled_mma = make_tiled_mma(Ops<KP36_32x16x512_F64F64F64>{}, mma_atom_shape);
    auto store_atom_shape = make_shape(Int<1>{}, Int<1>{});
    auto tiled_store = make_tiled_store(Ops<KP36_32x16_F64_STORE>{}, store_atom_shape);

    auto tensor_a = make_tensor(data_a, layout_a);
    auto tensor_b = make_tensor(data_b, layout_b);
    auto tensor_c = make_tensor(data_c, layout_c);

    mma(tiled_mma, tensor_c, tensor_a, tensor_b, tensor_c);
    store(tiled_store, tensor_c);

    free(data_a);
    free(data_b);
    free(data_c);
    return 0;
}
```

说明

上述示例演示了基于32*16*512_F64F64F64矩阵形状的mma流程，其中通过make_tensor创建矩阵对象tensor，作为后续mma/store接口参数。

5.3.2.8.8 make_tiled_mma

创建tiled_mma策略，用于后续矩阵乘加操作。

涉及mma_atom原子矩阵乘加方法和atom_shape原子方法扩展shape输入，其中mma_atom计算矩阵大小与atom_shape相应m/n/k相乘所得乘积结果表示实际tiled_mma方法计算矩阵大小。

当前atom_shape中m/n不支持任意扩展，只能设置为1，k支持任意大小。

接口定义

template<typename MmaAtom, typename Shape>

TiledMma<MmaAtom, Shape> make_tiled_mma(MmaAtom mma_atom, Shape atom_shape);

模板参数

表 5-105 模板参数定义

参数名	类型	描述
MmaAtom	typename	mma原子策略类型。
Shape	typename	形状类型。

版权所有 © 华为技术有限公司

参数

表 5-106 参数定义

参数名	类型	描述	输入/输出
mma_atom	MmaAtom，Ops<枚举 mma_atom_t>	mma原子策略，当前mma_atom_t可设置为： ● KP36_32x16x1_F64F64F64：表示计算尺寸为32*16*1，其中A矩阵为列主序、计算步长为Stride<1, 32>，B矩阵为行主序、计算步长为Stride<16, 1>，C矩阵为行主序、计算步长为Stride<16, 1>，计算精度为float64 * float64 = float64的mma原子行为 ● KP36_32x16x512_F64F64F64：表示计算尺寸为32*16*512，其中A矩阵为列主序、计算步长为Stride<1, 32>，B矩阵为行主序、计算步长为Stride<16, 1>，C矩阵为行主序、计算步长为Stride<16, 1>，计算精度为float64 * float64 = float64的mma原子行为 ● KP36_16x64x2_BF16BF16F32：表示计算尺寸为16*64*2，其中A矩阵为ZZ峰形、计算步长为Stride<2, Stride<1, 32>>，B矩阵为NN峰形、计算步长为Stride<Stride<1, 128>, 2>，C矩阵为行主序、计算步长为Stride<64, 1>，计算精度为bfloat16 * bfloat16 = float32的mma原子行为 ● KP36_16x64x1_BF16BF16F32：表示计算尺寸为16*64*1，其中A矩阵为列主序、计算步长为Stride<1, 16>，B矩阵为行主序、计算步长为Stride<64, 1>，C矩阵为行主序、计算步长为Stride<64, 1>，计算精度为bfloat16 * bfloat16 = float32的mma原子行为 ● KP36_16x64x4_INT8INT8INT32：表示计算尺寸为16*64*4，其中A矩阵为ZZ峰形、计算步长为Stride<4, Stride<1, 64>>，B矩阵为NN峰形、计算步长为Stride<Stride<1, 256>, 4>，C矩阵为行主序、计算步长为Stride<64, 1>，计算精度为int8 * int8 = int32的mma原子行为 ● KP36_32x32x4_INT8INT8INT32：表示计算尺寸为32*32*4，其中A矩阵为ZZ峰形、计算步长为Stride<4, Stride<1, 128>>，B矩阵为NN峰形、计算步长为Stride<Stride<1, 128>, 4>，C矩阵为行主序、计算步长为Stride<32, 1>，计算精度为int8 * int8 = int32的mma原子行为	输入
atom_shape	Shape	原子策略在各维度执行次数，包含m、n和k三个数值。	输入

版权所有 © 华为技术有限公司

返回值

返回TiledMma<MmaAtom, Shape>对象。

示例

```
#include "stdlib.h"
#include "kupl_mma.h"
using namespace kupl::tensor;

int main()
{
    constexpr int MATRIX_M  = 32;
    constexpr int MATRIX_N  = 16;
    constexpr int MATRIX_K = 512;
    double *data_a = (double *)malloc(sizeof(double) * MATRIX_M * MATRIX_K);
    double *data_b = (double *)malloc(sizeof(double) * MATRIX_K * MATRIX_N);
    double *data_c = (double *)malloc(sizeof(double) * MATRIX_M * MATRIX_N);

    auto shape_a = make_shape(Int<32>{}, Int<512>{});
    auto shape_b = make_shape(Int<512>{}, Int<16>{});
    auto shape_c = make_shape(Int<32>{}, Int<16>{});

    auto stride_a = make_stride(Int<1>{}, Int<32>{});
    auto stride_b = make_stride(Int<16>{}, Int<1>{});
    auto stride_c = make_stride(Int<16>{}, Int<1>{});

    auto layout_a = make_layout(shape_a, stride_a);
    auto layout_b = make_layout(shape_b, stride_b);
    auto layout_c = make_layout(shape_c, stride_c);

    auto mma_atom_shape = make_shape(Int<1>{}, Int<1>{}, Int<1>{});
    auto tiled_mma = make_tiled_mma(Ops<KP36_32x16x512_F64F64F64>{}, mma_atom_shape);
    auto store_atom_shape = make_shape(Int<1>{}, Int<1>{});
    auto tiled_store = make_tiled_store(Ops<KP36_32x16_F64_STORE>{}, store_atom_shape);

    auto tensor_a = make_tensor(data_a, layout_a);
    auto tensor_b = make_tensor(data_b, layout_b);
    auto tensor_c = make_tensor(data_c, layout_c);

    mma(tiled_mma, tensor_c, tensor_a, tensor_b, tensor_c);
    store(tiled_store, tensor_c);

    free(data_a);
    free(data_b);
    free(data_c);
    return 0;
}
```

说明

上述示例演示了基于32*16*512_F64F64F64矩阵形状的mma流程，其中通过make_tiled_mma创建mma tile策略。

5.3.2.8.9 make_tiled_store

创建tiled_store策略，用于后续矩阵写回操作。

涉及store_atom原子矩阵写回方法和atom_shape原子方法扩展shape输入，其中store_atom写回矩阵大小与atom_shape相应m/n相乘所得乘积结果表示实际tiled_store方法写回矩阵大小。

当前atom_shape中m/n不支持任意扩展，只能设置为1。

版权所有 © 华为技术有限公司

接口定义

template<typename StoreAtom, typename Shape>

TiledStore<StoreAtom, Shape> make_tiled_store(StoreAtom store_atom, Shape atom_shape);

模板参数

表 5-107 模板参数定义

参数名	类型	描述
StoreAtom	typename	store原子策略类型。
Shape	typename	形状类型。

参数

表 5-108 参数定义

参数名	类型	描述	输入/输出
store_atom	StoreAtom，Ops<枚举store_atom_t>	store原子策略，当前store_atom_t可设置为： ● KP36_32x16_F64_STORE，表示写回尺寸为32*16，其中写回矩阵为行主序、计算步长为Stride<16, 1>，写回精度为float64的store原子行为 ● KP36_16x64_F32_STORE，表示写回尺寸为16*64，其中写回矩阵为行主序、计算步长为Stride<64, 1>，写回精度为float32的store原子行为 ● KP36_16x64_INT32_STORE，表示写回尺寸为16*64，其中写回矩阵为行主序、计算步长为Stride<64, 1>，写回精度为int32的store原子行为 ● KP36_32x32_INT32_STORE，表示写回尺寸为32*32，其中写回矩阵为行主序、计算步长为Stride<32, 1>，写回精度为int32的store原子行为	输入
atom_shape	Shape	原子策略在各维度执行次数，包含m和n两个数值。	输入

版权所有 © 华为技术有限公司

返回值

返回TiledStore<StoreAtom, Shape>对象

示例

```
#include "stdlib.h"
#include "kupl_mma.h"
using namespace kupl::tensor;

int main()
{
    constexpr int MATRIX_M  = 32;
    constexpr int MATRIX_N  = 16;
    constexpr int MATRIX_K = 512;
    double *data_a = (double *)malloc(sizeof(double) * MATRIX_M * MATRIX_K);
    double *data_b = (double *)malloc(sizeof(double) * MATRIX_K * MATRIX_N);
    double *data_c = (double *)malloc(sizeof(double) * MATRIX_M * MATRIX_N);

    auto shape_a = make_shape(Int<32>{}, Int<512>{});
    auto shape_b = make_shape(Int<512>{}, Int<16>{});
    auto shape_c = make_shape(Int<32>{}, Int<16>{});

    auto stride_a = make_stride(Int<1>{}, Int<32>{});
    auto stride_b = make_stride(Int<16>{}, Int<1>{});
    auto stride_c = make_stride(Int<16>{}, Int<1>{});

    auto layout_a = make_layout(shape_a, stride_a);
    auto layout_b = make_layout(shape_b, stride_b);
    auto layout_c = make_layout(shape_c, stride_c);

    auto mma_atom_shape = make_shape(Int<1>{}, Int<1>{}, Int<1>{});
    auto tiled_mma = make_tiled_mma(Ops<KP36_32x16x512_F64F64F64>{}, mma_atom_shape);
    auto store_atom_shape = make_shape(Int<1>{}, Int<1>{});
    auto tiled_store = make_tiled_store(Ops<KP36_32x16_F64_STORE>{}, store_atom_shape);

    auto tensor_a = make_tensor(data_a, layout_a);
    auto tensor_b = make_tensor(data_b, layout_b);
    auto tensor_c = make_tensor(data_c, layout_c);

    mma(tiled_mma, tensor_c, tensor_a, tensor_b, tensor_c);
    store(tiled_store, tensor_c);

    free(data_a);
    free(data_b);
    free(data_c);
    return 0;
}
```

说明

上述示例演示了基于32*16*512_F64F64F64矩阵形状的mma流程，其中通过make_tiled_store创建store tile策略。

5.3.2.8.10 make_tiled_copy

创建tiled_copy策略。其中对于转置Copy操作而言，CopyAtom Ops转置矩阵大小与atom_shape相应m和n相乘所得乘积表示实际转置矩阵大小；对于预取Copy操作而言，CopyAtom Ops预取矩阵大小与atom_shape相应m相乘所得乘积表示实际预取矩阵大小。

当前atom_shape中m/n不支持任意扩展，对于不同的CopyAtom Ops存在不同的限制，具体见下表2参数定义中描述。

版权所有 © 华为技术有限公司

接口定义

template<typename CopyAtom, typename Shape>

TiledCopy<CopyAtom, Shape> make_tiled_copy(CopyAtom copy_atom, Shape atom_shape);

模板参数

表 5-109 模板参数定义

参数名	类型	描述
CopyAtom	typename	copy原子策略类型。
Shape	typename	形状类型。

版权所有 © 华为技术有限公司

参数

表 5-110 参数定义

参数名	类型	描述	输入/输出
copy_atom	CopyAtom，Ops<枚举copy_atom_t>	copy原子策略，当前copy_atom_t可设置为： ● KP36_32x1_F64_TRANS_RM2CM，表示转置原子尺寸为32 * 1、转置精度为float64的copy原子行为，其中源矩阵的排布为行主序，步幅可以描述为Stride<shape_n，1>；目的矩阵的排布为列主序，步幅可以描述为Stride<1，32>。约束：对于该原子方法，shape_n % 8 = 0 ● KP36_1x16_F64_TRANS_CM2RM，表示转置原子尺寸为1 * 16、转置精度为float64的copy原子行为，其中源矩阵的排布为列主序，步幅可以描述为Stride<1，shape_m>；目的矩阵的排布为行主序，步幅可以描述为Stride<16，1>。约束：对于该原子方法，shape_m % 8 = 0 ● KP36_16x2_BF16_TRANS_RM2ZZ，表示转置原子尺寸为16 * 2、转置精度为bfloat16的copy原子行为，其中源矩阵的排布为行主序，步幅可以描述为Stride<2 * shape_n，1>；目的矩阵的排布为ZZ峰形，步幅可以描述为Stride<2，Stride<1, 32>>。约束：对于该原子方法，shape_n % 16 = 0 ● KP36_2x64_BF16_TRANS_CM2NN，表示转置原子尺寸为2 * 64、转置精度为bfloat16的copy原子行为，其中源矩阵的排布为列主序，步幅可以描述为Stride<1，2 * shape_m>；目的矩阵的排布为NN峰形，步幅可以描述为Stride<Stride<1, 128>, 2>。约束：对于该原子方法，shape_m % 16 = 0 ● KP36_16x1_BF16_TRANS_RM2CM，表示转置原子尺寸为16 * 1、转置精度为bfloat16的copy原子行为，其中源矩阵的排布为行主序，步幅可以描述为Stride<shape_n，1>；目的矩阵的排布为列主序，步幅可以描述为Stride<1，16> ● KP36_1x64_BF16_TRANS_CM2RM，表示转置原子尺寸为1 * 64、转置精度为bfloat16的copy原子行为，其中源矩阵的排布为列主序，步幅可以描述为Stride<1，shape_m>；目的矩阵的排布为行主序，步幅可以描述为Stride<64，1> ● KP36_16x4_INT8_TRANS_RM2ZZ，表示转置原子尺寸为16 * 4、转置精度为int8的copy原子行为，其中源矩阵的排布为行主序，步幅可以描述为Stride<4 * shape_n，1>；目的矩阵的排布为ZZ峰形，步幅可以描述为Stride<4，Stride<1, 64>>。约束：对于该原子方法，shape_n % 16 = 0	输入

版权所有 © 华为技术有限公司

参数名	类型	描述	输入/输出
		• KP36_4x64_INT8_TRANS_CM2NN，表示转置原子尺寸为4 * 64、转置精度为int8的copy原子行为，其中源矩阵的排布为列主序，步幅可以描述为Stride<1，4 * shape_m>；目的矩阵的排布为NN峰形，步幅可以描述为Stride<Stride<1, 256>, 4>。约束：对于该原子方法，shape_m % 16 = 0 • KP36_32x4_INT8_TRANS_RM2ZZ，表示转置原子尺寸为32 * 4、转置精度为int8的copy原子行为，其中源矩阵的排布为行主序，步幅可以描述为Stride<4 * shape_n，1>；目的矩阵的排布为ZZ峰形，步幅可以描述为Stride<4，Stride<1, 128>>。约束：对于该原子方法，shape_n % 16 = 0 • KP36_4x32_INT8_TRANS_CM2NN，表示转置原子尺寸为4 * 32、转置精度为int8的copy原子行为，其中源矩阵的排布为列主序，步幅可以描述为Stride<1，4 * shape_m>；目的矩阵的排布为NN峰形，步幅可以描述为Stride<Stride<1, 128>, 4>。约束：对于该原子方法，shape_m % 16 = 0 • KP36_PREFETCH_L1，表示针对tensor对象的L1 cache预取操作。鲲鹏CPU中一次预取指令会处理64字节的内存，因此对于不同的精度tensor对象，KP36_PREFETCH_L1处理不同的shape大小，例如针对double精度而言，处理形状为Shape<8>，处理步幅为Stride<1>；而针对bfloat16精度而言，处理形状为Shape<32>，处理步幅为Stride<1>。约束：对于该原子方式，atom shape_m无法扩展，始终为1 • KP36_PREFETCH_L2，表示针对tensor对象的L2 cache预取操作。鲲鹏CPU中一次预取指令会处理64字节的内存，因此对于不同的精度tensor对象，KP36_PREFETCH_L2处理不同的shape大小，例如针对double精度而言，处理形状为Shape<8>，处理步幅为Stride<1>；而针对bfloat16精度而言，处理形状为Shape<32>，处理步幅为Stride<1>。约束：对于该原子方式，atom shape_m无法扩展，始终为1	
atom_shape	Shape	原子策略在各维度执行次数，针对转置原子方法而言包含m和n两个数值，针对预取原子方法而言包含m一个数值	输入

返回值

返回TiledCopy<StoreCopy, Shape>对象

示例

```
#include <stdio.h>
#include <stdlib.h>
```


版权所有 © 华为技术有限公司

```
#include "kupl_mma.h"
using namespace kupl::tensor;

int main()
{
    constexpr int MATRIX_M = 32;
    constexpr int MATRIX_N = 512;
    double *dst = (double*)malloc(sizeof(double) * MATRIX_M * MATRIX_N);
    double *src = (double*)malloc(sizeof(double) * MATRIX_M * MATRIX_N);
    for (int i = 0; i < MATRIX_M; i++) {
        for (int j = 0; j < MATRIX_N; j++) {
            dst[i * MATRIX_N + j] = 0.0;
            src[i * MATRIX_N + j] = i * MATRIX_N + j;
        }
    }

    auto shape_d = make_shape(Int<MATRIX_M>{}, Int<MATRIX_N>{});
    auto shape_s = make_shape(Int<MATRIX_M>{}, Int<MATRIX_N>{});

    auto stride_d = make_stride(Int<1>{}, Int<MATRIX_M>{});
    auto stride_s = make_stride(Int<MATRIX_N>{}, Int<1>{});

    auto layout_d = make_layout(shape_d, stride_d);
    auto layout_s = make_layout(shape_s, stride_s);

    auto atom_copy_shape = make_shape(Int<1>{}, Int<MATRIX_N>{});
    auto tiled_copy = make_tiled_copy(Ops<KP36_32x1_F64_TRANS_RM2CM>{}, atom_copy_shape);

    auto tensor_d = make_tensor(dst, layout_d);
    auto tensor_s = make_tensor(src, layout_s);
    copy(tiled_copy, tensor_d, tensor_s);

    free(src);
    free(dst);
    return 0;
}
```

说明

上述示例演示了基于32*512_F64矩阵形状从行主序到列主序的copy流程，其中通过make_tiled_copy创建copy tile策略，具体选择的原子方法为COPY_32x1_F64_RM2CM，atom_shape为shape<1, 512>。

5.3.2.8.11 mma

矩阵乘法累加接口，计算A*B+C。

涉及TiledMma策略和Tensor D/A/B/C矩阵输入，其中Tensor D/A/B/C的形状Shape和跨度步长Stride必须与TiledMma策略定义一致。

接口定义

template<typename TiledMma,

typename dtypeD, typename LayoutD,

typename dtypeA, typename LayoutA,

typename dtypeB, typename LayoutB,

typename dtypeC, typename LayoutC>

void mma(TiledMma tiled_mma, Tensor<dtypeD, LayoutD> D, Tensor<dtypeA, LayoutA> A, Tensor<dtypeB, LayoutB> B, Tensor<dtypeC, LayoutC> C);

 版权所有 © 华为技术有限公司

模板参数

表 5-111 模板参数定义

参数名	类型	描述
TiledMma	typename	mma tile策略类型。
dtypeD, dtypeA, dtypeB, dtypeC	typename	D,A,B,C精度类型。
LayoutD, LayoutA, LayoutB, LayoutC	typename	D,A,B,C布局类型。

参数

表 5-112 参数定义

参数名	类型	描述	输入/输出
tiled_mma	TiledMma	mma tile策略。	输入
D	Tensor<dtypeD, LayoutD>	矩阵对象D。	输入
A	Tensor<dtypeA, LayoutA>	矩阵对象A。	输入
B	Tensor<dtypeB, LayoutB>	矩阵对象B。	输入
C	Tensor<dtypeC, LayoutC>	矩阵对象C。	输入

返回值

void

示例

```
#include "stdlib.h"
#include "kupl_mma.h"
using namespace kupl::tensor;

int main()
{
    constexpr int MATRIX_M  = 32;
    constexpr int MATRIX_N  = 16;
    constexpr int MATRIX_K = 512;
    double *data_a = (double *)malloc(sizeof(double) * MATRIX_M * MATRIX_K);
    double *data_b = (double *)malloc(sizeof(double) * MATRIX_K * MATRIX_N);
    double *data_c = (double *)malloc(sizeof(double) * MATRIX_M * MATRIX_N);

    auto shape_a = make_shape(Int<32>{}, Int<512>{});
    auto shape_b = make_shape(Int<512>{}, Int<16>{});
```


版权所有 © 华为技术有限公司

```
    auto shape_c = make_shape(Int<32>{}, Int<16>{});

    auto stride_a = make_stride(Int<1>{}, Int<32>{});
    auto stride_b = make_stride(Int<16>{}, Int<1>{});
    auto stride_c = make_stride(Int<16>{}, Int<1>{});

    auto layout_a = make_layout(shape_a, stride_a);
    auto layout_b = make_layout(shape_b, stride_b);
    auto layout_c = make_layout(shape_c, stride_c);

    auto mma_atom_shape = make_shape(Int<1>{}, Int<1>{}, Int<1>{});
    auto tiled_mma = make_tiled_mma(Ops<KP36_32x16x512_F64F64F64>{}, mma_atom_shape);
    auto store_atom_shape = make_shape(Int<1>{}, Int<1>{});
    auto tiled_store = make_tiled_store(Ops<KP36_32x16_F64_STORE>{}, store_atom_shape);

    auto tensor_a = make_tensor(data_a, layout_a);
    auto tensor_b = make_tensor(data_b, layout_b);
    auto tensor_c = make_tensor(data_c, layout_c);

    mma(tiled_mma, tensor_c, tensor_a, tensor_b, tensor_c);
    store(tiled_store, tensor_c);

    free(data_a);
    free(data_b);
    free(data_c);
    return 0;
}
```

说明

上述示例演示了基于32*16*512_F64F64F64矩阵形状的mma流程，其中通过mma接收Tensor对象进行矩阵乘法计算。

5.3.2.8.12 store

矩阵数据写回。

涉及TiledStore策略和Tensor D矩阵输入，其中Tensor D的形状Shape和跨度步长Stride必须与TiledStore策略定义一致。

接口定义

template<typename TiledStore, typename dtype, typename Layout>

void store(TiledStore tiled_store, Tensor<dtype, Layout> tensor);

模板参数

表 5-113 模板参数定义

参数名	类型	描述
TiledStore	typename	store tile策略类型。
dtype	typename	精度类型。
Layout	typename	布局类型。

版权所有 © 华为技术有限公司

参数

表 5-114 参数定义

参数名	类型	描述	输入/输出
tiled_store	TiledStore	store tile策略。	输入
tensor	Tensor<dtype, Layout>	数据输出矩阵。	输出

返回值

void

示例

```
#include "stdlib.h"
#include "kupl_mma.h"
using namespace kupl::tensor;

int main()
{
    constexpr int MATRIX_M  = 32;
    constexpr int MATRIX_N  = 16;
    constexpr int MATRIX_K = 512;
    double *data_a = (double *)malloc(sizeof(double) * MATRIX_M * MATRIX_K);
    double *data_b = (double *)malloc(sizeof(double) * MATRIX_K * MATRIX_N);
    double *data_c = (double *)malloc(sizeof(double) * MATRIX_M * MATRIX_N);

    auto shape_a = make_shape(Int<32>{}, Int<512>{});
    auto shape_b = make_shape(Int<512>{}, Int<16>{});
    auto shape_c = make_shape(Int<32>{}, Int<16>{});

    auto stride_a = make_stride(Int<1>{}, Int<32>{});
    auto stride_b = make_stride(Int<16>{}, Int<1>{});
    auto stride_c = make_stride(Int<16>{}, Int<1>{});

    auto layout_a = make_layout(shape_a, stride_a);
    auto layout_b = make_layout(shape_b, stride_b);
    auto layout_c = make_layout(shape_c, stride_c);

    auto mma_atom_shape = make_shape(Int<1>{}, Int<1>{}, Int<1>{});
    auto tiled_mma = make_tiled_mma(Ops<KP36_32x16x512_F64F64F64>{}, mma_atom_shape);
    auto store_atom_shape = make_shape(Int<1>{}, Int<1>{});
    auto tiled_store = make_tiled_store(Ops<KP36_32x16_F64_STORE>{}, store_atom_shape);

    auto tensor_a = make_tensor(data_a, layout_a);
    auto tensor_b = make_tensor(data_b, layout_b);
    auto tensor_c = make_tensor(data_c, layout_c);

    mma(tiled_mma, tensor_c, tensor_a, tensor_b, tensor_c);
    store(tiled_store, tensor_c);

    free(data_a);
    free(data_b);
    free(data_c);
    return 0;
}
```


版权所有 © 华为技术有限公司

说明

上述示例演示了基于32*16*512_F64F64F64矩阵形状的mma流程，其中通过store将矩阵运算结果写回内存。

5.3.2.8.13 copy(Trans)

Trans类型的copy接口表示不同排布矩阵之间的转置Copy操作。

涉及TiledCopy策略和Tensor dst/src矩阵输入，其中Tensor dst/src的形状Shape和跨度步长Stride必须与TiledCopy策略定义一致。

接口定义

template<typename TiledCopy,

typename dtypeD, typename LayoutD,

typename dtypeS, typename LayoutS>

void copy(TiledCopy tiled_copy, Tensor<dtypeD, LayoutD> dst, Tensor<dtypeS, LayoutS> src);

模板参数

表 5-115 模板参数定义

参数名	类型	描述
TiledCopy	typename	copy tile策略类型。
dtypeD, dtypeS	typename	dst,src精度类型。
LayoutD, LayoutS	typename	dst,src布局类型。

参数

表 5-116 参数定义

参数名	类型	描述	输入/输出
tiled_copy	TiledCopy	copy tile策略。	输入
dst	Tensor<dtypeD, LayoutD>	转置目标矩阵。	输出
src	Tensor<dtypeS, LayoutS>	转置源矩阵。	输入

返回值

void

 版权所有 © 华为技术有限公司

示例

```
#include <stdio.h>
#include <stdlib.h>
#include "kupl_mma.h"
using namespace kupl::tensor;

int main()
{
    constexpr int MATRIX_M = 32;
    constexpr int MATRIX_N = 512;
    double *dst = (double*)malloc(sizeof(double) * MATRIX_M * MATRIX_N);
    double *src = (double*)malloc(sizeof(double) * MATRIX_M * MATRIX_N);
    for (int i = 0; i < MATRIX_M; i++) {
        for (int j = 0; j < MATRIX_N; j++) {
            dst[i * MATRIX_N + j] = 0.0;
            src[i * MATRIX_N + j] = i * MATRIX_N + j;
        }
    }

    auto shape_d = make_shape(Int<MATRIX_M>{}, Int<MATRIX_N>{});
    auto shape_s = make_shape(Int<MATRIX_M>{}, Int<MATRIX_N>{});

    auto stride_d = make_stride(Int<1>{}, Int<MATRIX_M>{});
    auto stride_s = make_stride(Int<MATRIX_N>{}, Int<1>{});

    auto layout_d = make_layout(shape_d, stride_d);
    auto layout_s = make_layout(shape_s, stride_s);

    auto atom_copy_shape = make_shape(Int<1>{}, Int<MATRIX_N>{});
    auto tiled_copy = make_tiled_copy(Ops<KP36_32x1_F64_TRANS_RM2CM>{}, atom_copy_shape);

    auto tensor_d = make_tensor(dst, layout_d);
    auto tensor_s = make_tensor(src, layout_s);
    copy(tiled_copy, tensor_d, tensor_s);

    free(src);
    free(dst);
    return 0;
}
```

说明

上述示例演示了基于32*512_F64矩阵形状从行主序到列主序的copy流程，其中通过copy根据创建的copy tile策略执行具体的copy转置操作。

5.3.2.8.14 copy(Prefetch)

Prefetch类型的copy接口表示针对矩阵的预取Copy操作，与上述Trans类型的copy接口相比主要差异在于入参上，采用了C++函数重载的特性进行实现。

涉及TiledCopy策略和Tensor src矩阵输入，其中Tensor src的形状Shape和跨度步长Stride必须与TiledCopy策略定义一致。

接口定义

template<typename TiledCopy,

typename dtypeS, typename LayoutS>

void copy(TiledCopy tiled_copy, Tensor<dtypeS, LayoutS> src);

版权所有 © 华为技术有限公司

模板参数

表 5-117 模板参数定义

参数名	类型	描述
TiledStore	typename	copy tile策略类型。
dtypeS	typename	src精度类型。
LayoutS	typename	src布局类型。

参数

表 5-118 参数定义

参数名	类型	描述	输入/输出
tiled_copy	TiledCopy	copy tile策略。	输入
src	Tensor<dtypeS, LayoutS>	转置源矩阵。	输入

返回值

void

示例

```
#include <stdio.h>
#include <stdlib.h>
#include "kupl_mma.h"
using namespace kupl::tensor;

int main()
{
    double *src = (double*)malloc(sizeof(double) * 8);
    auto shape_s = make_shape(Int<8>{});
    auto stride_s = make_stride(Int<1>{});
    auto layout_s = make_layout(shape_s, stride_s);
    auto tensor_s = make_tensor(src, layout_s);

    auto atom_copy_shape = make_shape(Int<1>{});
    auto tiled_copy = make_tiled_copy(Ops<KP36_PREFETCH_L1>, atom_copy_shape);

    copy(tiled_copy, tensor_s);

    free(src);
    return 0;
}
```

说明

上述示例演示了double精度buffer对象的L1预取操作行为，其中一次预取操作可以处理8个double数据。

版权所有 © 华为技术有限公司

5.3.2.9 公共函数

5.3.2.9.1 kupl_get_version

获取KUPL产品版本信息。

接口定义

int kupl_get_version(kupl_version_t *version)

参数

参数名	描述	取值范围	输入/输出
version	产品信息结构体。	非空	输出

返回值

- 成功：返回KUPL_OK
- 失败：返回KUPL_ERROR

示例

```
#include <stdio.h>
#include "kupl.h"

int main()
{
    kupl_version_t version;
    int status = kupl_get_version(&version);
    printf("product_name = %s\n", version.product_name);
    printf("product_version = %s\n", version.product_version);
    printf("component_name = %s\n", version.component_name);
    printf("component_abi_version = %s\n", version.component_abi_version);
    printf("component_version = %s\n", version.component_version);
    printf("component_appendinfo = %s\n", version.component_appendinfo);
    return 0;
}
```

运行结果：

```
product_name = Kunpeng HPCKit
product_version = 26.1.RC1
component_name = KUPL
component_abi_version = 1
component_version = 26.1.RC1
component_appendinfo = gcc
```

说明

以上版本号和编译时间以实际运行结果为准，上述结果仅供参考。

5.3.2.9.2 kupl_get_wtime

获取当前时间（秒）。

版权所有 © 华为技术有限公司

接口定义

int kupl_get_wtime();

返回值

- 返回当前的时间（秒）

示例

```
#include <stdio.h>
#include <assert.h>
#include <unistd.h>
#include "kupl.h"

int main()
{
    double start = kupl_get_wtime();
    usleep(1000001);
    double end = kupl_get_wtime();
    assert(end - start >= 1);
    return 0;
}
```

说明

- 上述示例演示了获取当前时间（秒）的流程。
- 上述kupl_get_wtime函数获取当前的时间，通过两次调用该函数的返回值的差值得到时间间隔。

版权所有 © 华为技术有限公司

6 鲲鹏数学库 开发指南

总述
鲲鹏数学库KML（Kunpeng Math Library）是华为提供的基于鲲鹏平台优化的高性能数学函数库，主要完成标量，矢量，矩阵上的数学计算，包括基本的四则运算、三角函数、对数函数、指数函数、线性代数计算等。本文档提供了KML的安装指南、详细接口定义与示例代码，旨在帮助用户快速熟悉KML的使用。

安装配置环境

使用说明

基础数学库

应用数学库

其他

6.1 总述

鲲鹏数学库KML（Kunpeng Math Library）是华为提供的基于鲲鹏平台优化的高性能数学函数库，主要完成标量，矢量，矩阵上的数学计算，包括基本的四则运算、三角函数、对数函数、指数函数、线性代数计算等。本文档提供了KML的安装指南、详细接口定义与示例代码，旨在帮助用户快速熟悉KML的使用。

数学库概述

KML提供了基于鲲鹏平台优化的高性能数学函数，所有接口由C/C++、汇编语言实现，部分接口兼容Fortran语言调用，部分提供Java语言封装的接口。KML的组成如**表6-1**所示。

 版权所有 © 华为技术有限公司

表 6-1 KML 组成部分

序号	库名	描述	适用场景
1	KML_BLAS	基础线性代数运算数学库（Basic Linear Algebra Subprograms），基于鲲鹏架构，通过向量化、数据预取、编译优化、数据重排等手段，对BLAS的计算效率进行了深度挖掘，使得BLAS接口函数的性能逼近理论峰值。兼容Fortran语言调用。	向量-向量运算 向量-矩阵运算 矩阵-矩阵运算
2	KML_SPBLAS	稀疏基础线性代数运算库（Sparse Basic Linear Algebra Subprograms），基于鲲鹏架构为压缩格式的稀疏矩阵提供了高性能向量、矩阵运算。充分利用鲲鹏的指令集和架构特点，开发了高性能稀疏矩阵运算库，提升高性能计算和大数据解决方案业务的性能。兼容Fortran语言调用。	稀疏的向量-向量运算 稀疏的向量-矩阵运算 稀疏的矩阵-矩阵运算
3	KML_VML	向量数学库（Vector Mathematical Library），通过NEON指令优化、内联汇编等方法，对输入数据进行向量化处理，充分利用了鲲鹏架构下的寄存器特点，实现了在鲲鹏服务器上的性能提升。兼容Fortran语言调用。	向量的数学运算，包括幂函数、三角函数、指数函数、双曲函数、对数函数等。
4	KML_MATH	基础运算数学库，通过周期函数规约、算法改进等手段，提供了基于鲲鹏处理器性能提升较大的函数实现。兼容Fortran语言调用。	基础数学运算，包括三角函数、对数函数、指数函数、幂函数运算。
5	KML_FFT	快速傅里叶（Fast Fourier Transform）运算库，是快速计算序列的离散傅里叶变换（DFT）或其逆变换的方法，广泛应用于工程、科学和数学领域，将傅里叶变换计算需要的复杂度从O(n2)降到了O(n log n)，被IEEE科学与工程计算期刊列入20世纪十大算法。兼容Fortran语言调用。	FFT的C2C、C2R、R2C、R2R变换。
6	KML_LAPACK	线性代数运算库（Linear Algebra PACKage），通过分块、求解算法组合、多线程、BLAS接口优化等手段，基于鲲鹏架构对LAPACK的计算效率进行了优化。兼容Fortran语言调用。	线性方程组运算，包括求解、特征值和奇异值问题求解等。

 版权所有 © 华为技术有限公司

序号	库名	描述	适用场景
7	KML_SVML	短向量数学库（Short Vector Mathematical Library），通过NEON指令优化、内联汇编等方法，对输入向量进行批量处理，充分利用了鲲鹏架构下的寄存器特点，实现了在鲲鹏服务器上的性能提升。兼容Fortran语言调用。	向量的数学运算，包括幂函数、三角函数、指数函数、双曲函数、对数函数等。
8	KML_SOLVER	稀疏求解器库（Sparse Solvers），提供直接求解器和迭代求解器，支持单节点多线程、集群MPI多进程。	单机版：稀疏对称正定矩阵、稀疏一般矩阵线性方程组迭代求解。 集群版：稀疏对称正定矩阵线性方程组迭代求解，正定埃尔米特矩阵线性方程组直接求解。
9	KML_JAVA	基于JNI技术，用Java语言封装的KML数学库。	BLAS、VML。
10	KML_SCALAPACK	分布式的线性代数运算库（Scalable Linear Algebra PACKage），通过分块、求解算法组合、多进程优化等手段，基于鲲鹏架构对SCALAPACK的计算效率进行了优化。兼容Fortran语言调用。	线性方程组运算，包括求解、特征值和奇异值问题求解等。
11	KML_VSL	向量统计库（Vector Statistical Library），包含多种随机数生成函数，可以生成服从不同分布的随机数。	服从不同分布的随机数生成。
12	KML_NUMPY	使用KML封装的Python Numpy模块。 KML_NUMPY库提供了40个适配KML或自行优化后的接口，优化的数据类型为float和double，涵盖了基础运算、傅里叶变换、统计和线性代数四个领域。	基础运算、傅里叶变换、统计和线性代数。
13	KML_EIGENSOLVER	特征值求解库，支持求解大型对称/共轭矩阵部分特征值与特征向量，支持MPI多节点。	实对称/复共轭矩阵的特征值求解。
14	KML_IPL	插值库（Interpolate Library），包含多种常用的插值函数，用于数值分析。	离散数据的拟合。

 版权所有 © 华为技术有限公司

序号	库名	描述	适用场景
15	KML_RT	运行时动态库，该库包含了运行时所需的函数，使得应用程序能够直接链接KML_RT库以实现对KML中数学运算函数的调用而无需链接对应的数学库，可避免用户在编译应用程序时需要链接多个KML数学库的繁琐操作，以提升应用程序开发和编译效率	——

KML仅适用于鲲鹏处理器。

KML支持NEON指令（128位宽）和SVE指令（256及以上位宽）。

为获得最优性能，KML接口内部不做完整入参校验，入参合法性由调用方业务来保证。

应用场景

KML主要在以下场景中使用：

- 大数据：机器学习算法。
- 高性能计算：科学计算、气象、基因、仿真。
- AI（人工智能）：语音识别、图像识别。

6.2 安装配置环境

须知

- 如果您的使用场景是通过HPCKit软件包集成安装鲲鹏数学库，请您跳过此章节，直接从**6.3 使用说明**开始阅读。
- 如果您需要独立安装并使用鲲鹏数学库，请参考本章节进行安装使用。

6.2.1 软硬件要求

为保证您可以顺利安全的使用KML，请检查您的环境，并确保所使用的环境信息在已验证环境范围内。

表 6-2 KML 已验证环境

操作系统	CPU类型	编译器
openEuler 22.03 LTS SP2	华为鲲鹏920系列处理器	GCC 10.3.1
	华为鲲鹏920系列处理器	毕昇编译器 4.1.0

版权所有 © 华为技术有限公司

操作系统	CPU类型	编译器
openEuler 22.03 LTS SP1	华为鲲鹏920系列处理器	GCC 10.3.1
	华为鲲鹏920系列处理器	毕昇编译器 4.1.0
openEuler 20.03 LTS SP1	华为鲲鹏920系列处理器	GCC 7.3.0
	华为鲲鹏920系列处理器	毕昇编译器 4.1.0
麒麟V10 SP3	华为鲲鹏920系列处理器	毕昇编译器 4.1.0
银河麒麟高级服务器操作系统V10（HPC版）AArch64	华为鲲鹏920系列处理器	GCC 12.3.1
	华为鲲鹏920系列处理器	毕昇编译器 4.1.0
麒麟信安服务器操作系统V3.5.2 AArch64	华为鲲鹏920系列处理器	GCC 12.3.1
	华为鲲鹏920系列处理器	毕昇编译器 4.1.0

须知

- KML目前只支持鲲鹏920处理器，其他处理器使用KML时会异常退出。如果以动态的方式加载KML库，程序会在链接KML时退出，造成未定义的结果。
- KML不支持GCC 4.8.5。
- KML为底层原语库，计算流程涉及内存读写、分配，不提供和发布操作系统，操作系统须用户自行安装，不承担操作系统的安全责任，用户需要结合自身应用对操作系统安全加固，包括不安装或者剔除不必要的应用等。
- 为阻止缓冲区溢出攻击，建议使用ASLR（Address space layout randomization）技术，通过对堆、栈、共享库映射等线性区布局的随机化，增加攻击者预测目的地址的难度，防止攻击者直接定位攻击代码位置。该技术可作用于堆、栈、内存映射区（mmap基址、shared libraries、vdso页）。

  开启方式：**echo 2 >/proc/sys/kernel/randomize_va_space**
- 后续KML将不再支持EOS的OS版本。

6.2.2 获取软件包

安装KML之前请先从官网地址获取软件包，再进行软件包校验，确保与网站上的原始软件包一致。如已通过HPCKit软件包方式安装KML组件，可跳过此节。

校验方法：

步骤1 从鲲鹏社区获取对应的软件数字证书和软件安装包。

表 6-3 KML 软件包获取列表

名称	包名	获取地址	验证签名
KML软件包（GCC版本）	KML_26.1.RC1.zip[a]	GCC版本链接	人工验证签名下载 自动验证签名下载

版权所有 © 华为技术有限公司

名称	包名	获取地址	验证签名
KML软件包（毕昇编译器版本）	KML_26.1.RC1_bisheng.zip[a]	毕昇版本链接	人工验证签名下载 自动验证签名下载
a：使用软件包前请先阅读《鲲鹏HPCKit用户许可协议》，如确认继续使用，则默认同意协议的条款和条件。			

步骤2 从华为企业业务网站获取校验工具和校验方法。

步骤3 参见步骤2中下载的《OpenPGP签名验证指南》进行软件包完整性检查。

----结束

6.2.3 如何使用 KML

6.2.3.1 安装 KML

本文提供 **.tar.gz** 包方式单独安装KML，请参照本节内容对KML进行安装和安装后验证。如已通过HPCKit软件包方式安装KML组件，可跳过此节。

安装步骤

步骤1 按照6.2.2 获取软件包获取KML软件包，解压后得到.tar.gz包。

步骤2 执行以下命令，解压tar包。

```
tar -xvf kml-xxxx.aarch64.tar.gz
```

上述命令中涉及的***xxxx***代表版本号。

步骤3 解压之后生成kml文件夹，该目录下的include文件夹包含子库的头文件，lib文件夹包含数学库的动态库文件，则表示安装成功。

----结束

6.2.3.2 设置环境变量

步骤1 系统通过 LD_LIBRARY_PATH 寻找动态库，需手动设置该环境变量链接到KML库，示例如下：

```
export LD_LIBRARY_PATH=/path/kml/lib/neon:/path/kml/lib/noarch:/path/kml/lib/noarch/kvml/single:/path/kml/lib/neon/kspblas/multi:/path/kml/lib/neon/kblas/multi:/path/kml/lib:${LD_LIBRARY_PATH}
```

命令中的/path路径请替换成实际kml的安装目录。

请根据实际环境，选择对应指令集下的库；根据实际需要，选择子库中的单/多线程以及是否加锁版本。

若需永久生效，请将上述 export 语句添加到您的 ~/.bashrc 文件中。

步骤2 执行如下命令检查是否设置成功。

```
echo $LD_LIBRARY_PATH
```

回显信息示例：

 版权所有 © 华为技术有限公司

```
/path/kml/lib/neon:/path/kml/lib/noarch:/path/kml/lib/noarch/kvml/single:/path/kml/lib/neon/
kspblas/multi:/path/kml/lib/neon/kblas/multi:/path/kml/lib:
```

----结束

6.2.3.3 使用方式

使用时，请在GCC编译选项中添加动态库所在路径，链接需要使用的动态库文件，添加编译选项后用**ldd**命令检查程序依赖库是否准确链接。

- 若需要使用KML_BLAS请添加：
  - 单线程不加锁版本：-L /path/kml/lib/neon/kblas/nolocking -lkblas
  - 单线程加锁版本：-L /path/kml/lib/neon/kblas/locking -lkblas
  - 多线程版本：-L /path/kml/lib/neon/kblas/multi -lkblas
- 若需要使用KML_VML请添加：
  - 单线程版本：-L /path/kml/lib/noarch -L /path/kml/lib/noarch/kvml/single -lkvml -lkm
  - 多线程版本：-L /path/kml/lib/noarch -L /path/kml/lib/noarch/kvml/multi -lkvm -lkm
- 若需要使用KML_SPBLAS请添加：
  - 单线程版本：-L /path/kml/lib/neon/kspblas/single -lkspblas
  - 多线程版本：-L /path/kml/lib/neon/kspblas/multi -lkspblas
- 若需要使用KML_FFT请添加：
  - 单精度版本：-L /path/kml/lib/noen -lkfftf
  - 双精度版本：-L /path/kml/lib/neon -lkfft
- 若需要使用KML_MATH请添加：
  - 高性能版本：-L /path/kml/lib/noarch -lkm -lm
  - 高精度版本：-L /path/kml/lib/noarch -lkm_l9 -lm
- 若需要使用KML_SVML请添加：-L /path/kml/lib/neon -lksvml -lm
- 若需要使用KML_VSL请添加：-L /path/kml/lib/noarch -lkvsl
- 若需要使用KML_LAPACK请添加-L /path/kml/lib/neon -L /path/kml/lib/neon/kblas/multi -lklapack -lkblas。
- 若需要使用KML_SOLVER请添加：
  - 单机版：

    毕昇：-L /path/kml/lib/neon -L /path/kml/lib/neon/kblas/multi -lksolver -lklapack_full -lkservice -lkblas -lc++

    GCC：-L /path/kml/lib/neon -L /path/kml/lib/neon/kblas/multi -lksolver -lklapack_full -lkservice -lkblas -lstdc++
  - 集群版：

    毕昇：-L /path/kml/lib/neon -L /path/kml/lib/neon/kblas/multi -lkscasolver -lklapack_full -lkservice -lkblas -lc++

    GCC：-L /path/kml/lib/neon -L /path/kml/lib/neon/kblas/multi -lkscasolver -lklapack_full -lkservice -lkblas -lstdc++
- 若需要使用KML_IPL请添加：-L /path/kml/lib/neon -lkipl -lklapack_full -L /path/kml/lib/neon/kblas/multi -lkblas -lm

 版权所有 © 华为技术有限公司

- 若需要使用KML_SCALAPACK请添加：-L /path/kml/lib/neon -L /path/kml/lib/neon/kblas/multi -lkscalapack -lklapack -lkservice -lkblas
- 若需要使用KML_EIGENSOLVER请添加：

  毕昇：-L /path/kml/lib/neon -L /path/kml/lib/neon/kblas/multi -lkes -lklapack_full -lkscalapack_full -lkservice -lkblas -lc++

  GCC: -L /path/kml/lib/neon -L /path/kml/lib/neon/kblas/multi -lkes -lklapack_full -lkscalapack_full -lkservice -lkblas -lstdc++

6.2.3.4 卸载 KML

若不再需要使用KML，可卸载KML。卸载KML将会影响您正在执行的业务流，建议先停止正在执行的业务流再进行卸载操作。

操作步骤

步骤1 执行以下命令，进入KML的安装目录。

```
cd /path
```

/path：KML的安装目录，请根据实际情况替换。

步骤2 执行以下命令，删除生成的KML组件文件。

```
rm -rf kml
```

----结束

6.3 使用说明

本节将介绍使用各个数学库需要链接的动态库。

使用时，请在编译选项中添加动态库所在路径，链接需要使用的动态库文件，添加编译选项后用**ldd**命令检查程序依赖库是否准确链接。

- 若需要使用KML_BLAS请添加：-lkblas
- 若需要使用KML_VML请添加：-lkvml -lkm -lm
- 若需要使用KML_SPBLAS请添加：-lkspblas
- 若需要使用KML_FFT请添加：
  - 单精度版本：-lkfftf
  - 双精度版本：-lkfft
  - 半精度版本：-lkffth
- 若需要使用KML_MATH请添加：
  - 高性能版本： -lkm -lm
  - 高精度版本： -lkm_l9 -lm
- 若需要使用KML_SVML请添加： -lksvml -lm
- 若需要使用KML_VSL请添加： -lkvsl
- 若需要使用KML_LAPACK请添加：-lklapack -lkblas -lkservice
- 若需要使用KML_SOLVER请添加：
  - 单机版：-lksolver -lklapack -lkservice -lkblas

 版权所有 © 华为技术有限公司

  - 集群版：-lkscasolver -lklapack -lkservice -lkblas
- 若需要使用KML_IPL请添加：-lkipl -lklapack -lkblas -lm
- 若需要使用KML_SCALAPACK请添加：-lkscalapack -lklapack -lkservice -lkblas
- 若需要使用KML_EIGENSOLVER请添加：-lkes -lklapack -lkscalapack -lkservice -lkblas

6.4 基础数学库

基础数学库是底层的原语数学库，解决底层的数学问题，是HPCKit数学库的基础组件，包含KML_FFT傅里叶变换库、KML_BLAS基础线性代数库、KML_SPBLAS稀疏基础线性代数库、KML_MATH基础数学函数库、KML_VML向量运算库、KML_SVML短向量运算库、KML_VSL向量统计库、KML_IPL插值库和KML_CONV卷积库。

6.4.1 KML_SPBLAS 库函数说明

6.4.1.1 函数说明

KML_SPBLAS库是稀疏矩阵（Sparse Matrix）的基础代数运算库，稀疏矩阵是指大部分矩阵元素为零的矩阵。

该库包含三个层级的函数：

- Sparse BLAS Level 1函数：向量与向量间的运算。
- Sparse BLAS Level 2函数：矩阵与向量间的运算。
- Sparse BLAS Level 3函数：矩阵与矩阵间的运算。

6.4.1.2 数据类型与数据存储

6.4.1.2.1 数据类型

提供KML_SPBLAS库函数的数据类型说明。

基本数据类型

整型数据：

```
typedef int KML_INT;  /*32位整型数*/
#define MAX_KML_INT 0x7FFFFFFD          /* 32位KML_INT的最大值减2，避免rows+2申请内存时溢出 */
```

复数数据类型：

单精度复数

```
typedef struct {
    float real;
    float imag;
} KML_Complex8;
```

双精度复数

```
typedef struct {
    double real;
    double imag;
} KML_Complex16;
```

 版权所有 © 华为技术有限公司

枚举类型

表6-4描述了KML_SPBLAS的枚举类型。

表 6-4 枚举类型及定义

枚举类型	说明	定义
kml_sparse_status_t	表示函数执行状态。	typedef enum{ KML_SPARSE_STATUS_SUCCESS = 0, /* the operation was successful */ KML_SPARSE_STATUS_NOT_INITIALIZED = 1, /* empty handle or matrix arrays */ KML_SPARSE_STATUS_ALLOC_FAILED = 2, /* internal error: memory allocation failed */ KML_SPARSE_STATUS_INVALID_VALUE = 3, /* invalid input value */ KML_SPARSE_STATUS_EXECUTION_FAILED = 4, /* e.g. 0-diagonal element for triangular solver, etc. */ KML_SPARSE_STATUS_INTERNAL_ERROR = 5, /* internal error */ KML_SPARSE_STATUS_NOT_SUPPORTED = 6 /* e.g. operation for double precision doesn't support other types */ } kml_sparse_status_t;
kml_sparse_operation_t	表示矩阵的操作（无变化、转置、共轭转置）。	typedef enum { KML_SPARSE_OPERATION_NON_TRANSPOSE = 0, KML_SPARSE_OPERATION_TRANSPOSE = 1, KML_SPARSE_OPERATION_CONJUGATE_TRANSPOSE = 2, KML_SPARSE_OPERATION_NUM } kml_sparse_operation_t;

版权所有 © 华为技术有限公司

枚举类型	说明	定义
kml_sparse_matrix_type_t	表示稀疏矩阵的类型。	typedef enum{ KML_SPARSE_MATRIX_TYPE_GENERAL = 0, KML_SPARSE_MATRIX_TYPE_SYMMETRIC = 1, KML_SPARSE_MATRIX_TYPE_HERMITIAN = 2, KML_SPARSE_MATRIX_TYPE_TRIANGULAR = 3, KML_SPARSE_MATRIX_TYPE_DIAGONAL = 4, KML_SPARSE_MATRIX_TYPE_BLOCK_TRIANGULAR = 5, KML_SPARSE_MATRIX_TYPE_BLOCK_DIAGONAL = 6, KML_SPARSE_MATRIX_TYPE_NUM } kml_sparse_matrix_type_t;
kml_sparse_format_t	表示稀疏矩阵的存储格式。	typedef enum{ KML_SPARSE_FORMAT_COO = 0, KML_SPARSE_FORMAT_CSR = 1, KML_SPARSE_FORMAT_CSC = 2, KML_SPARSE_FORMAT_BSR = 3, KML_SPARSE_FORMAT_SKY = 4, KML_SPARSE_FORMAT_DIA = 5 } kml_sparse_format_t;
kml_sparse_index_num_t	表示稀疏矩阵的indx从0开始或1开始。	typedef enum{ KML_SPARSE_INDEX_BASE_ZERO = 0, KML_SPARSE_INDEX_BASE_ONE = 1 } kml_sparse_index_num_t;
kml_sparse_fill_mode_t	表示稀疏矩阵是上三角或下三角矩阵。	typedef enum{ KML_SPARSE_FILL_MODE_LOWER = 0, KML_SPARSE_FILL_MODE_UPPER = 1 } kml_sparse_fill_mode_t;

版权所有 © 华为技术有限公司

枚举类型	说明	定义
kml_sparse_diag_type_t	表示稀疏矩阵的主对角元是否为单位元1。	typedef enum{ KML_SPARSE_DIAG_NON_UNIT = 0, KML_SPARSE_DIAG_UNIT = 1 } kml_sparse_diag_type_t;
kml_sparse_layout_t	表示稀疏矩阵是行主序或列主序。	typedef enum{ KML_SPARSE_LAYOUT_ROW_MAJOR = 0, KML_SPARSE_LAYOUT_COLUMN_MAJOR = 1 } kml_sparse_layout_t;
kml_sparse_order_t	表示稀疏矩阵是否排序。	typedef enum{ KML_SPARSE_NON_ORDER = 0, KML_SPARSE_ORDER = 1 } kml_sparse_order_t;
kml_sparse_datatype_t	表示数据类型。	typedef enum{ KML_SPARSE_DATATYPE_FLOAT = 0, KML_SPARSE_DATATYPE_DOUBLE = 1, KML_SPARSE_DATATYPE_FLOAT_COMPLEX = 2, KML_SPARSE_DATATYPE_DOUBLE_COMPLEX = 3 } kml_sparse_datatype_t;

6.4.1.2.2 数据存储

提供KML_SPBLAS库函数的数据存储格式说明。

稀疏向量压缩存储格式

仅存储稀疏矩阵中的非零元素。假设有稀疏矩阵a，a中有nz个非零元素：$a(k_1)$, $a(k_2)$, ..., $a(k_{nz})$。

存储时，用两个数组来存储矩阵a的信息：

$x(1)=a(k_1)$, $x(2)=a(k_2)$, ..., $x(nz)=a(k_{nz})$

$indx(1)=k_1$, $indx(2)=k_2$, ..., $indx(nz)=k_{nz}$

压缩存储格式可以节省计算机内存和运行时间。

向量全存储格式

存储稀疏矩阵中所有的元素。

 版权所有 © 华为技术有限公司

稀疏矩阵存储格式

CSR（Compressed Sparse Row，行序稀疏矩阵压缩）格式包含四项要素：值（value）、列（column）、索引B（pointerB）和索引E（pointerE）。

表6-5描述了稀疏矩阵A中四项要素的含义。

表 6-5 CSR 稀疏矩阵 A 中四项要素说明

参数名	描述
值（value）	• A矩阵中的非零元素，存储方式为行主序。 • 数组长度与A中非零元素个数相等。
列（column）	• value数组中第i个元素value[i]在矩阵A中的列序号。 • 数组长度与A中非零元素个数相等。
索引B（pointerB）	• 在基1索引中，pointerB(j)-pointerB(1)+1表示A矩阵第j行第一个非零元素在value数组中的索引。 • 在基0索引中，pointerB(j)-pointerB(0)表示A矩阵第j行第一个非零元素在value数组中的索引。 • 数组长度与A的行数相等。
索引E（pointerE）	• 在基1索引中，pointerE(j)-pointerB(1)表示A矩阵第j行最后一个非零元素在value数组中的索引。 • 在基0索引中，pointerE(j)-pointerB(0)-1表示A矩阵第j行最后一个非零元素在value数组中的索引。 • 数组长度与A的行数相等。

假设矩阵A为：$A=\begin{bmatrix}2 & -3 & 0 & 7\\ 0 & 0 & 1 & -6\\ 8 & 0 & -4 & 5\\ 9 & 0 & 0 & 0\end{bmatrix}$，则A的CSR格式如表6-6所示。

表 6-6 矩阵 A 的 CSR 格式

矩阵	索引方式	CSR格式
$A=\begin{bmatrix}2 & -3 & 0 & 7\\ 0 & 0 & 1 & -6\\ 8 & 0 & -4 & 5\\ 9 & 0 & 0 & 0\end{bmatrix}$	基1索引	value = [2, -3, 7, 1, -6, 8, -4, 5, 9] column = [1, 2, 4, 3, 4, 1, 3, 4, 1] pointerB = [1, 4, 6, 9] pointerE = [4, 6, 9, 10]
	基0索引	value = [2, -3, 7, 1, -6, 8, -4, 5, 9] column = [0, 1, 3, 2, 3, 0, 2, 3, 0] pointerB = [0, 3, 5, 8] pointerE = [3, 5, 8, 9]

版权所有 © 华为技术有限公司

CSC（Compressed Sparse Column，列序稀疏矩阵压缩）格式包含四项要素：值（value）、列（column）、索引B（pointerB）和索引E（pointerE）。

表6-7描述了稀疏矩阵A中四项要素的含义。

表 6-7 CSC 格式稀疏矩阵 A 中四项要素说明

参数名	描述
值（value）	• A矩阵中的非零元素，存储方式为列主序。 • 数组长度与A中非零元素个数相等。
列（column）	• value数组中第i个元素value[i]在矩阵A中的行序号。 • 数组长度与A中非零元素个数相等。
索引B（pointerB）	• 在基1索引中，pointerB(j)-pointerB(1)+1表示A矩阵第j列第一个非零元素在value数组中的索引。 • 在基0索引中，pointerB(j)-pointerB(0)表示A矩阵第j列第一个非零元素在value数组中的索引。 • 数组长度与A的列数相等。
索引E（pointerE）	• 在基1索引中，pointerE(j)-pointerB(1)表示A矩阵第j列最后一个非零元素在value数组中的索引。 • 在基0索引中，pointerE(j)-pointerB(0)-1表示A矩阵第j列最后一个非零元素在value数组中的索引。 • 数组长度与A的列数相等。

假设矩阵A为：$A=\begin{bmatrix}2 & -3 & 0 & 7\\ 0 & 0 & 1 & -6\\ 8 & 0 & -4 & 5\\ 9 & 0 & 0 & 0\end{bmatrix}$，则A的CSC格式如表6-8所示。

表 6-8 矩阵 A 的 CSC 格式

矩阵	索引方式	CSC格式
$A=\begin{bmatrix}2 & -3 & 0 & 7\\ 0 & 0 & 1 & -6\\ 8 & 0 & -4 & 5\\ 9 & 0 & 0 & 0\end{bmatrix}$	基1索引	value = [2, 8, 9, -3, 1, -4, 7, -6, 5] column = [1, 3, 4, 1, 2, 3, 1, 2, 3] pointerB = [1, 4, 5, 7] pointerE = [4, 5, 7, 10]
	基0索引	value = [2, 8, 9, -3, 1, -4, 7, -6, 5] column = [0, 2, 3, 0, 1, 2, 0, 1, 2] pointerB = [0, 3, 4, 6] pointerE = [3, 4, 6, 9]

版权所有 © 华为技术有限公司

6.4.1.3 函数定义

6.4.1.3.1 Sparse BLAS Level 1 函数

6.4.1.3.1.1 函数说明

对Sparse BLAS Level 1所有函数族进行描述。

Sparse BLAS Level1函数的命名规则是：kml_sparse_<character> <operation>

其中，<character>指示数据类型：s（单精度实数浮点型），d（双精度实数浮点型），c（单精度复数），z（双精度复数）。

<operation>指示向量的运算操作。

函数族	数据类型	描述
kml_sparse_?axpyi	s，d，c，z	向量与标量相乘，并将结果加到另一向量上。
kml_sparse_?doti	s，d	向量点积。
kml_sparse_?dotci_sub	c，z	向量共轭点积。
kml_sparse_?dotui_sub	c，z	复数域向量非共轭点积。
kml_sparse_?gthr	s，d，c，z	将全存储向量中的指定元素加载到压缩格式的向量中。
kml_sparse_?gthrz	s，d，c，z	将全存储向量中的指定元素加载到压缩格式的向量中，并将全存储向量元素清零。
kml_sparse_?roti	s，d	对两个实数类型向量进行旋转。
kml_sparse_?sctr	s，d，c，z	将压缩格式的向量写入全存储向量的指定位置。

6.4.1.3.1.2 kml_sparse_?axpyi

标量与向量相乘，并加到另一向量上。

即：y = a * x + y。a是缩放系数，x是稀疏向量，y是稠密向量。

接口定义

C interface：

kml_sparse_status_t kml_sparse_saxpyi(const KML_INT nz, const float a, const float *x, const KML_INT *indx, float *y);

kml_sparse_status_t kml_sparse_daxpyi(const KML_INT nz, const double a, const double *x, const KML_INT *indx, double *y);

版权所有 © 华为技术有限公司

kml_sparse_status_t kml_sparse_caxpyi(const KML_INT nz, const KML_Complex8 a, const KML_Complex8 *x, const KML_INT *indx, KML_Complex8 *y);

kml_sparse_status_t kml_sparse_zaxpyi(const KML_INT nz, const KML_Complex16 a, const KML_Complex16 *x, const KML_INT *indx, KML_Complex16 *y);

Fortran interface：

RES = KML_SPARSE_SAXPYI(NZ, A, X, INDX, Y);

RES = KML_SPARSE_DAXPYI(NZ, A, X, INDX, Y);

RES = KML_SPARSE_CAXPYI(NZ, A, X, INDX, Y);

RES = KML_SPARSE_ZAXPYI(NZ, A, X, INDX, Y);

参数

参数名	类型	描述	输入/输出
nz	整型数	x及indx数组中元素的个数。	输入
a	• 在saxpyi中，a是单精度浮点类型。 • 在daxpyi中，a是双精度浮点类型。 • 在caxpyi中，a是单精度复数。 • 在zaxpyi中，a是双精度复数。	标量a。	输入
x	• 在saxpyi中，x是单精度浮点类型数组。 • 在daxpyi中，x是双精度浮点类型数组。 • 在caxpyi中，x是单精度复数数组。 • 在zaxpyi中，x是双精度复数数组。	存储非零元素的数组x，长度等于nz。	输入
indx	整型数组	indx[i]表示x数组中第i个元素在稠密向量中的序列号，长度等于nz。	输入
y	• 在saxpyi中，y是单精度浮点类型数组。 • 在daxpyi中，y是双精度浮点类型数组。 • 在caxpyi中，y是单精度复数数组。 • 在zaxpyi中，y是双精度复数数组。	数组y，大于或等于max(indx[i])。	输入/输出

 版权所有 © 华为技术有限公司

返回值

函数执行状态，枚举类型kml_sparse_status_t。

依赖

C: "kspblas.h"

Fortran: "kspblas.f03"

示例

C interface：

```
KML_INT nz = 2;
float alpha = 1;
KML_INT indx[2] = {1, 2};
float x[2] = {1, 3};
float y[4] = {-1, 1, 5, 3};
kml_sparse_status_t status = kml_sparse_saxpyi(nz, alpha, x, indx, y);

/*
 *  Output Y:
 *     -1.000000     2.000000     8.000000     3.000000
 *
 */
```

Fortran interface：

```
INTEGER(C_INT) :: NZ = 2
REAL(C_FLOAT) :: ALPHA = 1
INTEGER(C_INT) :: INDX(2)
REAL(C_FLOAT) :: X(2), Y(4)
DATA INDX/1, 2/
DATA X/1, 3/
DATA Y/-1, 1, 5, 3/
INTEGER :: STATUS
STATUS = KML_SPARSE_SAXPYI(NZ, ALPHA, X, INDX, Y)
!
!  OUTPUT Y:
!     -1.000000     2.000000     8.000000     3.000000
!
!
```

6.4.1.3.1.3 kml_sparse_?doti

实数域的点积（欧几里得空间）。

res = x[0] * y[indx[0]] + x[1] * y[indx[1]] + ... + x[nz-1] * y[indx[nz-1]]

接口定义

C interface：

kml_sparse_status_t kml_sparse_sdoti(const KML_INT nz, const float *x,const KML_INT *indx, const float *y, float *doti);

kml_sparse_status_t kml_sparse_ddoti(const KML_INT nz, const double *x,const KML_INT *indx, const double *y, double *doti);

Fortran interface：

 版权所有 © 华为技术有限公司

RES = KML_SPARSE_SDOTI(NZ, X, INDX, Y, DOTI);

RES = KML_SPARSE_DDOTI(NZ, X, INDX, Y, DOTI);

参数

参数名	类型	描述	输入/输出
nz	整型数	x及indx数组中元素的个数。	输入
x	• 在sdoti中，x是单精度浮点类型数组。 • 在ddoti中，x是双精度浮点类型数组。	存储非零元素的数组x，长度等于nz。	输入
indx	整型数组	indx[i]表示x数组中第i个元素在稠密向量中的序列号，数组长度等于nz。	输入
y	• 在sdoti中，y是单精度浮点类型数组。 • 在ddoti中，y是双精度浮点类型数组。	数组y，大于或等于max(indx[i])。	输入
doti	• 在sdoti中，doti是单精度浮点类型。 • 在ddoti中，doti是双精度浮点类型。	x与y的点积。	输出

返回值

函数执行状态，枚举类型kml_sparse_status_t。

依赖

C: "kspblas.h"

Fortran: "kspblas.f03"

示例

C interface：

```
KML_INT nz = 2;
float doti = 0;
KML_INT indx[2] = {1, 2};
float x[2] = {1, 3};
float y[4] = {-1, 1, 5, 3};
kml_sparse_status_t status = kml_sparse_sdoti(nz, x, indx, y, &doti);

/*
 *  Output doti:
 *    16.000000
 */
```

 版权所有 © 华为技术有限公司

Fortran interface：

```
INTEGER(C_INT) :: NZ = 2
REAL(C_FLOAT), TARGET :: DOTI
TYPE(C_PTR) :: PDOTI
INTEGER(C_INT) :: STATUS
INTEGER(C_INT) INDX(2)
REAL(C_FLOAT) X(2)
REAL(C_FLOAT) Y(4)
DATA INDX/1, 2/
DATA X/1, 3/
DATA Y/-1, 1, 5, 3/
PDOTI = C_LOC(DOTI)
STATUS = KML_SPARSE_SDOTI(NZ, X, INDX, Y, PDOTI)
/*
 *  OUTPUT DOTI:
 *    16.000000
 */
```

6.4.1.3.1.4 kml_sparse_?dotci_sub

共轭点积（希尔伯特空间）。

x · y = xˆH * y，即x向量中的元素取其共轭复数后与y向量中对应元素相乘然后累加（复数a + bi的共轭为a - bi），计算公式表示为：conjg(x[0]) * y[indx[0]] + ... + conjg(x[nz-1]) * y[indx[nz-1]]。

接口定义

C interface：

kml_sparse_status_t kml_sparse_cdotci_sub(const KML_INT nz, const KML_Complex8 *x, const KML_INT *indx, const KML_Complex8 *y, KML_Complex8 *dotci);

kml_sparse_status_t kml_sparse_zdotci_sub(const KML_INT nz, const KML_Complex16 *x, const KML_INT *indx, const KML_Complex16 *y, KML_Complex16 *dotci);

Fortran interface：

RES = KML_SPARSE_CDOTCI_SUB(NZ, X, INDX, Y, DOTCI);

RES = KML_SPARSE_ZDOTCI_SUB(NZ, X, INDX, Y, DOTCI);

参数

参数名	类型	描述	输入/输出
nz	整型数	x及indx数组中元素的个数。	输入
x	• 在cdotci中，x是单精度浮点类型数组。 • 在zdotci中，x是双精度浮点类型数组。	存储非零元素的数组x，长度等于nz。	输入

版权所有 © 华为技术有限公司

参数名	类型	描述	输入/输出
indx	整型数组	indx[i]表示x数组中第i个元素在稠密向量中的序列号，长度等于nz。	输入
y	• 在cdotci中，y是单精度浮点类型数组。 • 在zdotci中，y是双精度浮点类型数组。	数组y，大于或等于max(indx[i])。	输入
dotci	• 在cdotci中，dotci是单精度浮点类型。 • 在zdotci中，dotci是双精度浮点类型。	返回共轭x与y的点积。	输出

返回值

函数执行状态，枚举类型kml_sparse_status_t。

依赖

C: "kspblas.h"

Fortran: "kspblas.f03"

示例

C interface：

```
KML_INT nz = 2;
KML_Complex8 dotci = {0, 0};
KML_INT indx[2] = {1, 2};
KML_Complex8 x[2] = {{1, 2}, {3, 4}};
KML_Complex8 y[4] = {{-1, 1}, {5, 3}, {-2, -3}, {4, 1}};
kml_sparse_status_t status = kml_sparse_cdotci_sub(nz, x, indx, y, &dotci);

/*
 *  Output dotci:
 *     {-7.000000, -8.000000}
 */
```

Fortran interface：

```
INTEGER(C_INT) :: NZ = 2
INTEGER(C_INT) :: STATUS
INTEGER(C_INT) :: I
TYPE(KML_COMPLEX8), TARGET :: DOTCI = KML_COMPLEX8(0, 0)
INTEGER(C_INT) :: INDX(2)
TYPE(KML_COMPLEX8) :: CX(2), CY(4)
REAL(C_FLOAT) :: X(4), Y(8)
TYPE(C_PTR) :: PDOTCI
PDOTCI = C_LOC(DOTCI)
DATA INDX/1, 2/
DATA X/1, 2, 3, 4/
DATA Y /-1, 1, 5, 3, -2, -3, 4, 1/
I = 1
```

 版权所有 © 华为技术有限公司

```
DO WHILE(I <= 2)
   CX(I)%REAL = X(2 * I - 1)
   CX(I)%IMAG = X(2 * I)
END DO
I = 1
DO WHILE(I <= 4)
   CY(I)%REAL = Y(2 * I - 1)
   CY(I)%IMAG = Y(2 * I)
END DO
STATUS = KML_SPARSE_CDOTCI_SUB(NZ, CX, INDX, CY, PDOTCI)
!
!  OUTPUT DOTCI:
!    {-7.000000, -8.000000}
!
```

6.4.1.3.1.5 kml_sparse_?dotui_sub

复数域的非共轭点积（欧几里得空间）

x · y = xˆT * y，即对应元素相乘然后累加。

res = x[0] * y[indx[0]] + x[1] * y(indx[1]) + ... + x[nz - 1] * y[indx[nz - 1]]

接口定义

C interface：

kml_sparse_status_t kml_sparse_cdotui_sub(const KML_INT nz, const KML_Complex8 *x, const KML_INT *indx, const KML_Complex8 *y, KML_Complex8 *dotui);

kml_sparse_status_t kml_sparse_zdotui_sub(const KML_INT nz, const KML_Complex16 *x, const KML_INT *indx,const KML_Complex16 *y, KML_Complex16 *dotui);

Fortran interface：

RES = KML_SPARSE_CDOTUI_SUB(NZ, X, INDX, Y, DOTUI);

RES = KML_SPARSE_ZDOTUI_SUB(NZ, X, INDX, Y, DOTUI);

参数

参数名	类型	描述	输入/输出
nz	整型数	x及indx数组中元素的个数。	输入
x	• 在cdotui中，x是单精度浮点类型数组。 • 在zdotui中，x是双精度浮点类型数组。	存储非零元素的数组x，长度等于nz。	输入
indx	整型数组	indx[i]表示x数组中第i个元素在稠密向量中的序列号，数组长度等于nz。	输入

版权所有 © 华为技术有限公司

参数名	类型	描述	输入/输出
y	• 在cdotui中，y是单精度浮点类型数组。 • 在zdotui中，y是双精度浮点类型数组。	数组y，大于或等于max(indx[i])。	输入
dotui	• 在cdotui中，dotui是单精度浮点类型。 • 在zdotui中，dotui是双精度浮点类型。	x与y的点积。	输入/输出

返回值

函数执行状态，枚举类型kml_sparse_status_t。

依赖

C: "kspblas.h"

Fortran: "kspblas.f03"

示例

C interface：

```
KML_INT nz = 2;
KML_Complex8 dotui = {0,0};
KML_INT indx[2] = {1, 2};
KML_Complex8 x[2] = {{1, 2}, {3, 4}};
KML_Complex8 y[4] = {{-1, 1}, {5, 3}, {-2, -3}, {4, 1}};
kml_sparse_status_t status = kml_sparse_cdotui_sub(nz, x, indx, y, &dotui);

/*
 *  Output dotui:
 *     {5.000000, -4.000000}
 */
```

Fortran interface：

```
INTEGER(C_INT) :: NZ = 2
INTEGER(C_INT) :: STATUS
INTEGER(C_INT) :: I
TYPE(KML_COMPLEX8), TARGET :: DOTUI = KML_COMPLEX8(0, 0)
INTEGER(C_INT) :: INDX(2)
TYPE(KML_COMPLEX8) :: CX(2), Y(4)
REAL(C_FLOAT) :: X(4), Y(8)
TYPE(C_PTR) :: PDOTUI
PDOTUI = C_LOC(DOTUI)
DATA INDX/1, 2/
DATA X/1, 2, 3, 4/
DATA Y /-1, 1, 5, 3, -2, -3, 4, 1/
I = 1
DO WHILE(I <= 2)
   CX(I)%REAL = X(2 * I - 1)
   CX(I)%IMAG = X(2 * I)
END DO
I = 1
```

 版权所有 © 华为技术有限公司

```
DO WHILE(I <= 4)
   CY(I)%REAL = Y(2 * I - 1)
   CY(I)%IMAG = Y(2 * I)
END DO
STATUS = KML_SPARSE_CDOTUI_SUB(NZ, CX, INDX, CY, PDOTUI)
!
!  OUTPUT DOTUI:
!    {5.000000, -4.000000}
!
```

6.4.1.3.1.6 kml_sparse_?gthr

将full-storage格式的稀疏向量y中指定的元素加载到compressed格式的向量中。

即x[i]=y[indx[i]]，i=0,1,…nz-1。

接口定义

C interface：

kml_sparse_status_t kml_sparse_sgthr(const KML_INT nz, const float *y, float *x, const KML_INT *indx);

kml_sparse_status_t kml_sparse_dgthr(const KML_INT nz, const double *y, double *x,const KML_INT *indx);

kml_sparse_status_t kml_sparse_cgthr(const KML_INT nz, const KML_Complex8 *y, KML_Complex16 *x, const KML_INT *indx);

kml_sparse_status_t kml_sparse_zgthr(const KML_INT nz, const KML_Complex16 *y, KML_Complex16 *x, const KML_INT *indx);

Fortran interface：

RES = KML_SPARSE_SGTHR(NZ, Y, X, INDX);

RES = KML_SPARSE_DGTHR(NZ, Y, X, INDX);

RES = KML_SPARSE_CGTHR(NZ, Y, X, INDX);

RES = KML_SPARSE_ZGTHR(NZ, Y, X, INDX);

参数

参数名	类型	描述	输入/输出
nz	整型数	x及indx数组中元素的个数。	输入
y	• 在sgthr中，y是单精度浮点类型数组。 • 在dgthr中，y是双精度浮点类型数组。 • 在cgthr中，y是单精度复数数组。 • 在zgthr中，y是双精度复数数组。	数组y，大于或等于max(indx[i])。	输入

版权所有 © 华为技术有限公司

参数名	类型	描述	输入/输出
x	• 在sgthr中，x是单精度浮点类型数组。 • 在dgthr中，x是双精度浮点类型数组。 • 在cgthr中，x是单精度复数数组。 • 在zgthr中，x是双精度复数数组。	存储非零元素的数组x，长度等于nz。	输入/输出
indx	整型数组	indx[i]表示x数组中第i个元素在稠密向量中的序列号，数组长度等于nz。	输入

返回值

函数执行状态，枚举类型kml_sparse_status_t。

依赖

C: "kspblas.h"

Fortran: "kspblas.f03"

示例

C interface：

```
KML_INT nz = 2;
KML_INT indx[2] = {1,2};
float x[2] = {0,0};
float y[4] = {-1, 5, -2 ,3};
kml_sparse_status_t status = kml_sparse_sgthr(nz,y,x,indx);

/*
 *  Output x:
 *     5.000000    -2.000000
 */
```

Fortran interface：

```
INTEGER(C_INT) :: NZ = 2
INTEGER(C_INT) :: INDX(2)
INTEGER(C_INT) :: STATUS
REAL(C_FLOAT) :: X(2), Y(4)
DATA INDX/1,2/
DATA X/0,0/
DATA Y/-1, 5, -2 ,3/
STATUS = KML_SPARSE_SGTHR(NZ,Y,X,INDX);
!
!  OUTPUT X:
!     5.000000    -2.000000
!
```

 版权所有 © 华为技术有限公司

6.4.1.3.1.7 kml_sparse_?gthrz

将full-storage格式的稀疏向量y中指定的元素加载到compressed格式的向量中。

即x[i]=y[indx[i]]，i=0,1,…nz-1。同时将y中对应元素清零。

接口定义

C interface：

kml_sparse_status_t kml_sparse_sgthrz(const KML_INT nz, float *y, float *x, const KML_INT *indx);

kml_sparse_status_t kml_sparse_dgthrz(const KML_INT nz, double *y, double *x, const KML_INT *indx);

kml_sparse_status_t kml_sparse_cgthrz(const KML_INT nz, KML_Complex8 *y, KML_Complex8 *x, const KML_INT *indx);

kml_sparse_status_t kml_sparse_zgthrz(const KML_INT nz, KML_Complex16 *y, KML_Complex16 *x, const KML_INT *indx);

Fortran interface：

RES = KML_SPARSE_SGTHRZ(NZ, Y, X, INDX);

RES = KML_SPARSE_DGTHRZ(NZ, Y, X, INDX);

RES = KML_SPARSE_CGTHRZ(NZ, Y, X, INDX);

RES = KML_SPARSE_ZGTHRZ(NZ, Y, X, INDX);

参数

参数名	类型	描述	输入/输出
nz	整型数	x及indx数组中元素的个数。	输入
y	• 在sgthrz中，y是单精度浮点类型数组。 • 在dgthrz中，y是双精度浮点类型数组。 • 在cgthrz中，y是单精度复数数组。 • 在zgthrz中，y是双精度复数数组。	数组y，大于或等于max(indx[i])。	输入/输出

版权所有 © 华为技术有限公司

参数名	类型	描述	输入/输出
x	• 在sgthrz中，x是单精度浮点类型数组。 • 在dgthrz中，x是双精度浮点类型数组。 • 在cgthrz中，x是单精度复数数组。 • 在zgthrz中，x是双精度复数数组。	存储非零元素的数组x，长度等于nz。	输入/输出
indx	整型数组	indx[i]表示x数组中第i个元素在稠密向量中的序列号，数组长度等于nz。	输入

返回值

函数执行状态，枚举类型kml_sparse_status_t。

依赖

C: "kspblas.h"

Fortran: "kspblas.f03"

示例

C interface：

```
KML_INT nz = 2;
KML_INT indx[2] = {1, 2};
float x[2] = {0, 0};
float y[4] = {-1, 5, -2 ,3};
kml_sparse_status_t status = kml_sparse_sgthrz(nz, y, x, indx);

/*
 *  Output x:
 *     5.000000    -2.000000
 *  Output y:
 *     -1.000000      0.000000      0.000000      3.000000
 */
```

Fortran interface：

```
INTEGER(C_INT) :: NZ = 2
INTEGER(C_INT) :: INDX(2)
INTEGER(C_INT) :: STATUS
REAL(C_FLOAT) :: X(2), Y(4)
DATA INDX/1,2/
DATA X/0,0/
DATA Y/-1, 5, -2 ,3/
STATUS = KML_SPARSE_SGTHRZ(NZ,Y,X,INDX);
!
!  OUTPUT X:
!     5.000000    -2.000000
!  OUTPUT Y:
```

 版权所有 © 华为技术有限公司

```
!    -1.000000     0.000000     0.000000     3.000000
!
```

6.4.1.3.1.8 kml_sparse_?roti

对2个实数类型的向量进行旋转操作。

x[i] = c*x[i] + s*y[indx[i]]

y[indx[i]] = c * y[indx[i]] - s * x[i]

接口定义

C interface：

kml_sparse_status_t kml_sparse_sroti(const KML_INT nz, float *x, const KML_INT *indx, float *y, const float c, const float s);

kml_sparse_status_t kml_sparse_droti(const KML_INT nz, double *x, const KML_INT *indx, double *y, const double c, const double s);

Fortran interface：

RES = KML_SPARSE_SROTI(NZ, X, INDX, Y, C, S);

RES = KML_SPARSE_DROTI(NZ, X, INDX, Y, C, S);

参数

参数名	类型	描述	输入/输出
nz	整型数	x及indx数组中元素的个数。	输入
x	• 在sroti中，x是单精度浮点类型数组。 • 在droti中，x是双精度浮点类型数组。	compressed格式的向量x，长度等于nz。	输入/输出
indx	整型数组	存储数组x中非零元素下标索引，长度等于nz。	输入
y	• 在sroti中，y是单精度浮点类型数组。 • 在droti中，y是双精度浮点类型数组。	full-storage格式的向量y，大于或等于max(indx[i])。	输入/输出
c	• 在sroti中，c是单精度浮点类型标量。 • 在droti中，c是双精度浮点类型标量。	标量。	输入
s	• 在sroti中，s是单精度浮点类型标量。 • 在droti中，s是双精度浮点类型标量。	标量。	输入

版权所有 © 华为技术有限公司

返回值

函数执行状态，枚举类型kml_sparse_status_t。

依赖

C: "kspblas.h"

Fortran: "kspblas.f03"

示例

C interface：

```
KML_INT nz = 2;
KML_INT indx[2] = {1, 2};
float x[2] = {1, 3};
float y[4] = {-1, 5, -2, 4};
float c = 1;
float s = 2;
kml_sparse_status_t status = kml_sparse_sroti(nz, x, indx, y, c, s);

/*
 *  Output x:
 *     11.000000    -1.000000
 *  Output y:
 *     -1.000000     3.000000    -8.000000     4.000000
 */
```

Fortran interface：

```
INTEGER(C_INT) :: NZ = 2
INTEGER(C_INT) :: INDX(2)
REAL(C_FLOAT) :: X(2), Y(4)
REAL(C_FLOAT) :: C = 1
REAL(C_FLOAT) :: S = 2
INTEGER(C_INT) :: STATUS
DATA INDX/1, 2/
DATA X/1, 3/
DATA Y/-1, 5, -2, 4/
STATUS = KML_SPARSE_SROTI(NZ, X, INDX, Y, C, S)
!
!  OUTPUT X:
!    11.000000    -1.000000
!  OUTPUT Y:
!     -1.000000     3.000000    -8.000000     4.000000
!
```

6.4.1.3.1.9 kml_sparse_?sctr

将compressed格式的向量写入full-storage格式的稀疏向量y指定位置。

即y[indx[i]] = x[i]，i=0,1,…nz-1。

接口定义

C interface：

kml_sparse_status_t kml_sparse_ssctr(const KML_INT nz, const float *x, const KML_INT *indx, float *y);

 版权所有 © 华为技术有限公司

kml_sparse_status_t kml_sparse_dsctr(const KML_INT nz, const double *x, const KML_INT *indx, double *y);

kml_sparse_status_t kml_sparse_csctr(const KML_INT nz, const KML_Complex8 *x, const KML_INT *indx, KML_Complex8 *y);

kml_sparse_status_t kml_sparse_zsctr(const KML_INT nz, const KML_Complex16 *x, const KML_INT *indx, KML_Complex16 *y);

Fortran interface：

RES = KML_SPARSE_SSCTR(NZ, X, INDX, Y);

RES = KML_SPARSE_DSCTR(NZ, X, INDX, Y);

RES = KML_SPARSE_CSCTR(NZ, X, INDX, Y);

RES = KML_SPARSE_ZSCTR(NZ, X, INDX, Y);

参数

参数名	类型	描述	输入/输出
nz	整型数	x及indx数组中元素的个数。	输入
y	• 在ssctr中，y是单精度浮点类型数组。 • 在dsctr中，y是双精度浮点类型数组。 • 在csctr中，y是单精度复数数组。 • 在zsctr中，y是双精度复数数组。	数组y，大于或等于max(indx[i])。	输入/输出
x	• 在ssctr中，x是单精度浮点类型数组。 • 在dsctr中，x是双精度浮点类型数组。 • 在csctr中，x是单精度复数数组。 • 在zsctr中，x是双精度复数数组。	存储非零元素的数组x，长度等于nz。	输入
indx	整型数组	indx[i]表示x数组中第i个元素在稠密向量中的序列号，数组长度等于nz。	输入

返回值

函数执行状态，枚举类型kml_sparse_status_t。

 版权所有 © 华为技术有限公司

依赖

C: "kspblas.h"

Fortran: "kspblas.f03"

示例

C interface：

```
KML_INT nz = 2;
KML_INT indx[2] = {1, 2};
float x[2] = {1, 1};
float y[4] = {-1, 5, -2 ,3};
kml_sparse_status_t status = kml_sparse_ssctr(nz, x, indx, y);

/*
 *  Output sctr:
 *     -1.000000     1.000000     1.000000     3.000000
 */
```

Fortran interface：

```
INTEGER(C_INT) :: NZ = 2
INTEGER(C_INT) :: INDX(2)
REAL(C_FLOAT) :: X(2), Y(4)
INTEGER(C_INT) :: STATUS
DATA INDX/1, 2/
DATA X/1, 1/
DATA Y/-1, 5, -2 ,3/
STATUS = KML_SPARSE_SSCTR(NZ, X, INDX, Y)
!
!  OUTPUT SCTR:
!     -1.000000     1.000000     1.000000     3.000000
!
```

6.4.1.3.2 Sparse BLAS Level 2 和 Level 3 函数

6.4.1.3.2.1 函数说明

Sparse BLAS Level2/3（CSR格式）函数的命名规则是：

1. kml_sparse_<character><data><operation>
2. kml_sparse_<character><data><mtype><operation>，仅支持基1索引的矩阵
3. kml_csparse_<character><data><mtype><operation>，仅支持基0索引的矩阵

其中，<character>指示数据类型：s（单精度实数浮点型），d（双精度实数浮点型），c（单精度复数），z（双精度复数）

<data>指示存储类型：coo，csr，dia，sky，csc，bsr

<mtype>指示矩阵类型：ge（通用矩阵），sy（对称矩阵），tr（三角矩阵）

<operation>指示矩阵的运算操作

 版权所有 © 华为技术有限公司

表 6-9 简易接口（仅支持基 1 索引）

函数族	数据类型	描述
kml_sparse_?csrgemv	s，d，c，z	矩阵与向量乘积，矩阵是CSR格式（三数组）基1索引的稀疏一般矩阵。
kml_sparse_?csrsymv	s，d，c，z	矩阵与向量乘积，矩阵是CSR格式（三数组）基1索引的稀疏对称矩阵。
kml_sparse_?csrtrsv	s，d，c，z	求解稀疏三角矩阵方程组，矩阵是CSR格式（三数组）基1索引的稀疏三角矩阵。

表 6-10 简易接口（仅支持基 0 索引）

函数族	数据类型	描述
kml_csparse_?csrgemv	s，d，c，z	矩阵与向量乘积，矩阵是CSR格式（三数组）基0索引的稀疏一般矩阵。
kml_csparse_?csrsymv	s，d，c，z	矩阵与向量乘积，矩阵是CSR格式（三数组）基0索引的稀疏对称矩阵。
kml_csparse_?csrtrsv	s，d，c，z	求解稀疏三角线性方程组，矩阵是CSR格式（三数组）基0索引的稀疏三角矩阵。

表 6-11 通用接口，支持基 1 索引和基 0 索引

函数族	数据类型	描述
kml_sparse_?csrmv	s，d，c，z	矩阵与向量乘积，矩阵是CSR格式的稀疏矩阵。
kml_sparse_?csrsv	s，d，c，z	求解稀疏线性方程组，稀疏矩阵存储格式为CSR。
kml_sparse_?csrmm	s，d，c，z	计算CSR格式的稀疏矩阵和稠密矩阵的积。
kml_sparse_?cscmv	s，d，c，z	矩阵与向量乘积，矩阵是CSC格式的稀疏矩阵。
kml_sparse_?cscsv	s，d，c，z	求解稀疏线性方程组，稀疏矩阵存储格式为CSC。
kml_sparse_?cscmm	s，d，c，z	计算CSC格式的稀疏矩阵和稠密矩阵的积。
kml_sparse_?csrsm	s，d，c，z	求解多个右端项的稀疏线性方程组，矩阵是CSR格式。
kml_sparse_?cscsm	s，d，c，z	求解多个右端项的稀疏线性方程组，矩阵是CSC格式。

 版权所有 © 华为技术有限公司

表 6-12 稀疏矩阵间运算接口

函数族	数据类型	描述
kml_sparse_?csradd	s，d，c，z	矩阵与矩阵相加，矩阵是CSR格式（三数组）基1索引的稀疏矩阵，结果存储在稀疏矩阵。
kml_sparse_?csrmultcsr	s，d，c，z	矩阵与矩阵相乘，矩阵是CSR格式（三数组）基1索引的稀疏矩阵，结果存储在稀疏矩阵。
kml_sparse_?csrmultd	s，d，c，z	矩阵与矩阵相乘，矩阵是CSR格式（三数组）基1索引的稀疏矩阵，结果存储在稠密矩阵。

matdescra 参数取值

matdescra是char指针类型的，包含6个元素（仅使用前4个）。

参数	取值
参数1	• G：一般矩阵 • S：对称矩阵（A=A'） • H：埃尔米特矩阵（A=conjg(A')） • T：三角矩阵 • D：对角矩阵
参数2	• L：下三角 • U：上三角
参数3	• N：非单元矩阵 • U：单元矩阵
参数4	• F：基1索引（索引值从1计数） • C：基0索引（索引值从0计数）

当前支持的matdescra可能取值如表6-13所示。

表 6-13 支持的 matdescra 可能取值

参数1	参数2	参数3	参数4
G	0（忽略）	0（忽略）	F或C
S或H	L	N	F或C
S或H	L	U	F或C
S或H	U	N	F或C
S或H	U	U	F或C

版权所有 © 华为技术有限公司

参数1	参数2	参数3	参数4
T	L	N	F或C
T	L	U	F或C
T	U	N	F或C
T	U	U	F或C
D	0（忽略）	N	F或C
D	0（忽略）	U	F或C

6.4.1.3.2.2 kml_sparse_?csrgemv

矩阵与向量计算，稀疏矩阵采用CSR格式存储，具体执行操作如下：

$y = A * x$或$y = A^T * x$或$y = A^H * x$，

其中，x和y为向量，A是稀疏矩阵，采用CSR格式3数组存储。仅支持矩阵indx从1开始。

接口定义

C interface：

kml_sparse_status_t kml_sparse_scsrgemv(const kml_sparse_operation_t opt, const KML_INT m, const float *a, const KML_INT *ia, const KML_INT *ja, const float *x, float *y);

kml_sparse_status_t kml_sparse_dcsrgemv(const kml_sparse_operation_t opt, const KML_INT m, const double *a, const KML_INT *ia, const KML_INT *ja, const double *x, double *y);

kml_sparse_status_t kml_sparse_ccsrgemv(const kml_sparse_operation_t opt, const KML_INT m, const KML_Complex8 *a, const KML_INT *ia, const KML_INT *ja, const KML_Complex8 *x, KML_Complex8 *y);

kml_sparse_status_t kml_sparse_zcsrgemv(const kml_sparse_operation_t opt, const KML_INT m, const KML_Complex16 *a ,const KML_INT *ia, const KML_INT *ja, const KML_Complex16 *x, KML_Complex16 *y);

Fortran interface：

RES = KML_SPARSE_SCSRGEMV(OPT, M, A, IA, JA, X, Y);

RES = KML_SPARSE_DCSRGEMV(OPT, M, A, IA, JA, X, Y);

RES = KML_SPARSE_CCSRGEMV(OPT, M, A, IA, JA, X, Y);

RES = KML_SPARSE_ZCSRGEMV(OPT, M, A, IA, JA, X, Y);

 版权所有 © 华为技术有限公司

参数

参数名	类型	描述	输入/输出
opt	枚举类型 kml_sparse_operation_t	表示矩阵A的操作， • 如果opt=KML_SPARSE_OPERATION_NON_TRANSPOSE，则$y = A * x$。 • 如果opt=KML_SPARSE_OPERATION_TRANSPOSE，则$y = A^T * x$。 • 如果opt=KML_SPARSE_OPERATION_CONJUGATE_TRANSPOSE，则$y = A^H * x$。	输入
m	整型数	表示A矩阵的行数，取值范围为[1, MAX_KML_INT]。	输入
a	• 在scsrgemv中，a是单精度浮点数组。 • 在dcsrgemv中，a是双精度浮点数组。 • 在ccsrgemv中，a是单精度复数数组。 • 在zcsrgemv中，a是双精度复数数组。	A矩阵中的values数组，存储非零元素。	输入
ia	整型数组	长度为m+1的向量。 ia[i]-1表示A矩阵第i行第一个非零元素在a数组中的索引。	输入
ja	整型数组	ja[i]表示a数组中第i个元素在矩阵A中的列序号。	输入
x	• 在scsrgemv中，x是单精度浮点数组。 • 在dcsrgemv中，x是双精度浮点数组。 • 在ccsrgemv中，x是单精度复数数组。 • 在zcsrgemv中，x是双精度复数数组。	向量x的数组。	输入

版权所有 © 华为技术有限公司

参数名	类型	描述	输入/输出
y	• 在scsrgemv中，y是单精度浮点数组。 • 在dcsrgemv中，y是双精度浮点数组。 • 在ccsrgemv中，y是单精度复数数组。 • 在zcsrgemv中，y是双精度复数数组。	向量y的数组，更新后输出。	输入/输出

⚠ 注意

函数内部不对参数做完整校验，接口调用者需保证ia和ja内部元素不超出矩阵最大索引值。

返回值

函数执行状态，枚举类型kml_sparse_status_t。

依赖

C: "kspblas.h"

Fortran: "kspblas.f03"

示例

C interface：

```
kml_sparse_operation_t opt = KML_SPARSE_OPERATION_NON_TRANSPOSE;
KML_INT m = 4;
float a[9] = {2, -3, 7, 1, -6, 8, -4, 5, 9};
KML_INT ja[9] = {1, 2, 4, 3, 4, 1, 3, 4, 1};
KML_INT ia[5] = {1, 4, 6, 9, 10};
float x[4] = {1, 3, -2, 5};
float y[4] = {-1, 1, 5, 3};
kml_sparse_status_t status = kml_sparse_scsrgemv(opt, m, a, ia, ja, x, y);

/*
 *  Output Y:
 *     28.000000   -32.000000    41.000000     9.000000
 *
 */
```

Fortran interface：

```
INTEGER(C_INT) :: OPT = KML_SPARSE_OPERATION_NON_TRANSPOSE
INTEGER(C_INT) :: M = 4
REAL(C_FLOAT) :: A(9), X(4), Y(4)
INTEGER(C_INT) :: JA(9), IA(5)
INTEGER(C_INT) :: STATUS
DATA A/2, -3, 7, 1, -6, 8, -4, 5, 9/
DATA JA/1, 2, 4, 3, 4, 1, 3, 4, 1/
```

 版权所有 © 华为技术有限公司

```
DATA IA/1, 4, 6, 9, 10/
DATA X/1, 3, -2, 5/
DATA Y/-1, 1, 5, 3/
STATUS = KML_SPARSE_SCSRGEMV(OPT, M, A, IA, JA, X, Y)
!
!  OUTPUT Y:
!     28.000000   -32.000000    41.000000     9.000000
!
```

6.4.1.3.2.3 kml_sparse_?csrsymv

矩阵与向量计算，稀疏矩阵采用CSR格式存储，具体执行操作如下：

y = A * x

其中，x和y为向量，A是上三角或者下三角的对称矩阵，采用CSR格式3数组存储。仅支持矩阵indx从1开始。

接口定义

C interface：

kml_sparse_status_t kml_sparse_scsrsymv(const kml_sparse_fill_mode_t uplo, const KML_INT m, const float *a, const KML_INT *ia, const KML_INT *ja, const float *x, float *y);

kml_sparse_status_t kml_sparse_dcsrsymv(const kml_sparse_fill_mode_t uplo, const KML_INT m, const double *a, const KML_INT *ia, const KML_INT *ja, const double *x, double *y);

kml_sparse_status_t kml_sparse_ccsrsymv(const kml_sparse_fill_mode_t uplo, const KML_INT m, const KML_Complex8 *a, const KML_INT *ia, const KML_INT *ja, const KML_Complex8 *x, KML_Complex8 *y);

kml_sparse_status_t kml_sparse_zcsrsymv(const kml_sparse_fill_mode_t uplo, const KML_INT m, const KML_Complex16 *a, const KML_INT *ia, const KML_INT *ja, const KML_Complex16 *x, KML_Complex16 *y);

Fortran interface：

RES = KML_SPARSE_SCSRSYMV(UPLO, M, A, IA, JA, X, Y);

RES = KML_SPARSE_DCSRSYMV(UPLO, M, A, IA, JA, X, Y);

RES = KML_SPARSE_CCSRSYMV(UPLO, M, A, IA, JA, X, Y);

RES = KML_SPARSE_ZCSRSYMV(UPLO, M, A, IA, JA, X, Y);

参数

参数名	类型	描述	输入/输出
uplo	枚举类型 kml_sparse_fill_mode_t uplo	指示矩阵A的上三角或者下三角被使用： ● U：上三角 ● L：下三角	输入

版权所有 © 华为技术有限公司

参数名	类型	描述	输入/输出
m	整型数	表示A矩阵的行数，取值范围为[1, MAX_KML_INT]。	输入
a	• 在scsrsymv中，a是单精度浮点数组。 • 在dcsrsymv中，a是双精度浮点数组。 • 在ccsrsymv中，a是单精度复数数组。 • 在zcsrsymv中，a是双精度复数数组。	A矩阵中的非零元素。	输入
ia	整型数组	长度为m+1的向量。 ia[i]-1表示A矩阵第i行第一个非零元素在a数组中的索引。	输入
ja	整型数组	ja[i]表示a数组中第i个元素在矩阵A中的列序号。	输入
x	• 在scsrsymv中，x是单精度浮点数组。 • 在dcsrsymv中，x是双精度浮点数组。 • 在ccsrsymv中，x是单精度复数数组。 • 在zcsrsymv中，x是双精度复数数组。	向量x。	输入
y	• 在scsrsymv中，y是单精度浮点数组。 • 在dcsrsymv中，y是双精度浮点数组。 • 在ccsrsymv中，y是单精度复数数组。 • 在zcsrsymv中，y是双精度复数数组。	向量y，更新后输出。	输入/输出

⚠ 注意

函数内部不对参数做完整校验，接口调用者需保证ia和ja内部元素不超出矩阵最大索引值。

返回值

函数执行状态，枚举类型kml_sparse_status_t。

版权所有 © 华为技术有限公司

依赖

C: "kspblas.h"

Fortran: "kspblas.f03"

示例

C interface：

```
kml_sparse_fill_mode_t uplo = KML_SPARSE_FILL_MODE_LOWER;
KML_INT m = 4;
float a[9] = {2, -3, 7, 1, -6, 8, -4, 5, 9};
KML_INT ja[9] = {1, 2, 4, 3, 4, 1, 3, 4, 1};
KML_INT ia[5] = {1, 4, 6, 9, 10};
float x[4] = {1, 3, -2, 5};
float y[4] = {-1, 1, 5, 3};
kml_sparse_status_t status = kml_sparse_scsrsymv(uplo, m, a, ia, ja, x, y);

/*
 *  Output Y:
 *     31.000000     0.000000     16.000000     9.000000
 *
 */
```

Fortran interface：

```
INTEGER(C_INT) :: UPLO = KML_SPARSE_FILL_MODE_LOWER
INTEGER(C_INT) :: M = 4
REAL(C_FLOAT) :: A(9), X(4), Y(4)
INTEGER(C_INT) :: JA(9), IA(5)
INTEGER(C_INT) :: STATUS
DATA A/2, -3, 7, 1, -6, 8, -4, 5, 9/
DATA JA/1, 2, 4, 3, 4, 1, 3, 4, 1/
DATA IA/1, 4, 6, 9, 10/
DATA X/1, 3, -2, 5/
DATA Y/-1, 1, 5, 3/
STATUS = KML_SPARSE_SCSRSYMV(UPLO, M, A, IA, JA, X, Y)
!
!  OUTPUT Y:
!     31.000000     0.000000     16.000000     9.000000
!
!
```

6.4.1.3.2.4 kml_csparse_?csrgemv

矩阵与向量计算，稀疏矩阵采用CSR格式存储，具体执行操作如下：

y = A * x或y = A^T * x或y = A^H * x，

其中，x和y为向量，A是稀疏矩阵，采用CSR格式3数组存储。仅支持矩阵index从0开始。

接口定义

C interface：

kml_sparse_status_t kml_csparse_scsrgemv(const kml_sparse_operation_t opt, const KML_INT m, const float *a, const KML_INT *ia, const KML_INT *ja, const float *x, float *y);

kml_sparse_status_t kml_csparse_dcsrgemv(const kml_sparse_operation_t opt, const KML_INT m, const double *a, const KML_INT *ia, const KML_INT *ja, const double *x, double *y);

 版权所有 © 华为技术有限公司

kml_sparse_status_t kml_csparse_ccsrgemv(const kml_sparse_operation_t opt, const KML_INT m, const KML_Complex8 *a, const KML_INT *ia, const KML_INT *ja,

const KML_Complex8 *x, KML_Complex8 *y);

kml_sparse_status_t kml_csparse_zcsrgemv(const kml_sparse_operation_t opt, const KML_INT m, const KML_Complex16 *a ,const KML_INT *ia, const KML_INT *ja, const KML_Complex16 *x, KML_Complex16 *y);

Fortran interface：

RES = KML_CSPARSE_SCSRGEMV(OPT, M, A, IA, JA, X, Y);

RES = KML_CSPARSE_DCSRGEMV(OPT, M, A, IA, JA, X, Y);

RES = KML_CSPARSE_CCSRGEMV(OPT, M, A, IA, JA, X, Y);

RES = KML_CSPARSE_ZCSRGEMV(OPT, M, A, IA, JA, X, Y);

参数

参数名	类型	描述	输入/输出
opt	枚举类型 kml_sparse_operation_t	表示矩阵A的操作， • 如果opt=KML_SPARSE_OPERATION_NON_TRANSPOSE，则$y = A * x$。 • 如果opt=KML_SPARSE_OPERATION_TRANSPOSE，则$y = A^T * x$。 • 如果opt=KML_SPARSE_OPERATION_CONJUGATE_TRANSPOSE，则$y = A^H * x$。	输入
m	整型数	表示A矩阵的行数，取值范围为[1, MAX_KML_INT]。	输入
a	• 在c_scsrgemv中，a是单精度浮点数组。 • 在c_dcsrgemv中，a是双精度浮点数组。 • 在c_ccsrgemv中，a是单精度复数数组。 • 在c_zcsrgemv中，a是双精度复数数组。	A矩阵中的非零元素。	输入

 版权所有 © 华为技术有限公司

参数名	类型	描述	输入/输出
ia	整型数组	长度为m+1的向量。 ia[i]表示A矩阵第i行第一个非零元素在a数组中的索引。	输入
ja	整型数组	ja[i]表示a数组中第i个元素在矩阵A中的列序号。	输入
x	• 在c_scsrgemv中，x是单精度浮点数组。 • 在c_dcsrgemv中，x是双精度浮点数组。 • 在c_ccsrgemv中，x是单精度复数数组。 • 在c_zcsrgemv中，x是双精度复数数组。	向量x。	输入
y	• 在c_scsrgemv中，y是单精度浮点数组。 • 在c_dcsrgemv中，y是双精度浮点数组。 • 在c_ccsrgemv中，y是单精度复数数组。 • 在c_zcsrgemv中，y是双精度复数数组。	向量y，更新后输出。	输入/输出

⚠ 注意

函数内部不对参数做完整校验，接口调用者需保证ia和ja内部元素不超出矩阵最大索引值。

返回值

函数执行状态，枚举类型kml_sparse_status_t。

依赖

C: "kspblas.h"

Fortran: "kspblas.f03"

示例

C interface：

```
kml_sparse_operation_t opt = KML_SPARSE_OPERATION_NON_TRANSPOSE;
KML_INT m = 4;
```

 版权所有 © 华为技术有限公司

```
float a[9] = {2, -3, 7, 1, -6, 8, -4, 5, 9};
KML_INT ja[9] = {0, 1, 3, 2, 3, 0, 2, 3, 0};
KML_INT ia[5] = {0, 3, 5, 8, 9};
float x[4] = {1, 3, -2, 5};
float y[4] = {-1, 1, 5, 3};
kml_sparse_status_t status = kml_csparse_scsrgemv(opt, m, a, ia, ja, x, y);

/*
 *  Output Y:
 *     28.000000   -32.000000    41.000000     9.000000
 *
 */
```

Fortran interface：

```
INTEGER(C_INT) :: OPT = KML_SPARSE_OPERATION_NON_TRANSPOSE
  INTEGER(C_INT) :: M = 4
  REAL(C_FLOAT) :: A(9), X(4), Y(4)
  INTEGER(C_INT) :: JA(9), IA(5)
  INTEGER(C_INT) :: STATUS
  DATA A/2, -3, 7, 1, -6, 8, -4, 5, 9/
  DATA JA/0, 1, 3, 2, 3, 0, 2, 3, 0/
  DATA IA/0, 3, 5, 8, 9/
  DATA X/1, 3, -2, 5/
  DATA Y/-1, 1, 5, 3/
  STATUS = KML_CSPARSE_SCSRGEMV(OPT, M, A, IA, JA, X, Y)
  !
  !  OUTPUT Y:
  !     28.000000   -32.000000    41.000000     9.000000
  !
```

6.4.1.3.2.5 kml_csparse_?csrsymv

矩阵与向量计算，稀疏矩阵采用CSR格式存储，具体执行操作如下：

y = A * x

其中，x和y为向量，A是上三角或者下三角的对称矩阵，采用CSR格式3数组存储。仅支持矩阵index从0开始。

接口定义

C interface：

kml_sparse_status_t kml_csparse_scsrsymv(const kml_sparse_fill_mode_t uplo, const KML_INT m, const float *a, const KML_INT *ia, const KML_INT *ja, const float *x, float *y);

kml_sparse_status_t kml_csparse_dcsrsymv(const kml_sparse_fill_mode_t uplo, const KML_INT m, const double *a, const KML_INT *ia, const KML_INT *ja, const double *x, double *y);

kml_sparse_status_t kml_csparse_ccsrsymv(const kml_sparse_fill_mode_t uplo, const KML_INT m, const KML_Complex8 *a, const KML_INT *ia, const KML_INT *ja, const KML_Complex8 *x, KML_Complex8 *y);

kml_sparse_status_t kml_csparse_zcsrsymv(const kml_sparse_fill_mode_t uplo, const KML_INT m, const KML_Complex16 *a, const KML_INT *ia, const KML_INT *ja, const KML_Complex16 *x, KML_Complex16 *y);

Fortran interface：

RES = KML_CSPARSE_SCSRSYMV(UPLO, M, A, IA, JA, X, Y);

 版权所有 © 华为技术有限公司

RES = KML_CSPARSE_DCSRSYMV(UPLO, M, A, IA, JA, X, Y);

RES = KML_CSPARSE_CCSRSYMV(UPLO, M, A, IA, JA, X, Y);

RES = KML_CSPARSE_ZCSRSYMV(UPLO, M, A, IA, JA, X, Y);

参数

参数名	类型	描述	输入/输出
uplo	枚举类型 kml_sparse_fill_mode_t uplo	指示矩阵A的上三角或者下三角被使用： ● U：上三角 ● L：下三角	输入
m	整型数	表示A矩阵的行数，取值范围为[1, MAX_KML_INT]。	输入
a	● 在c_scsrsymv中，a是单精度浮点数组。 ● 在c_dcsrsymv中，a是双精度浮点数组。 ● 在c_ccsrsymv中，a是单精度复数数组。 ● 在c_zcsrsymv中，a是双精度复数数组。	A矩阵中的非零元素。	输入
ia	整型数组	长度为m+1的向量。 ia[i]表示A矩阵第i行第一个非零元素在a数组中的索引。	输入
ja	整型数组	ja[i]表示a数组中第i个元素在矩阵A中的列序号。	输入
x	● 在c_scsrsymv中，x是单精度浮点数组。 ● 在c_dcsrsymv中，x是双精度浮点数组。 ● 在c_ccsrsymv中，x是单精度复数数组。 ● 在c_zcsrsymv中，x是双精度复数数组。	向量x。	输入

 版权所有 © 华为技术有限公司

参数名	类型	描述	输入/输出
y	• 在c_scsrsymv中，y是单精度浮点数组。 • 在c_dcsrsymv中，y是双精度浮点数组。 • 在c_ccsrsymv中，y是单精度复数数组。 • 在c_zcsrsymv中，y是双精度复数数组。	向量y，更新后输出。	输入/输出

⚠ 注意

函数内部不对参数做完整校验，接口调用者需保证ia和ja内部元素不超出矩阵最大索引值。

返回值

函数执行状态，枚举类型kml_sparse_status_t。

依赖

C: "kspblas.h"

Fortran: "kspblas.f03"

示例

C interface：

```
kml_sparse_fill_mode_t uplo = KML_SPARSE_FILL_MODE_LOWER;
KML_INT m = 4;
float a[9] = {2, -3, 7, 1, -6, 8, -4, 5, 9};
KML_INT ja[9] = {0, 1, 3, 2, 3, 0, 2, 3, 0};
KML_INT ia[5] = {0, 3, 5, 8, 9};
float x[4] = {1, 3, -2, 5};
float y[4] = {-1, 1, 5, 3};
kml_sparse_status_t status = kml_csparse_scsrsymv(uplo, m, a, ia, ja, x, y);

/*
 *  Output Y:
 *     31.000000     0.000000    16.000000     9.000000
 *
 */
```

Fortran interface：

```
INTEGER(C_INT) :: UPLO = KML_SPARSE_FILL_MODE_LOWER
INTEGER(C_INT) :: M = 4
REAL(C_FLOAT) :: A(9), X(4), Y(4)
INTEGER(C_INT) :: JA(9), IA(5)
INTEGER(C_INT) :: STATUS
DATA A/2, -3, 7, 1, -6, 8, -4, 5, 9/
DATA JA/0, 1, 3, 2, 3, 0, 2, 3, 0/
```

 版权所有 © 华为技术有限公司

```
DATA IA/0, 3, 5, 8, 9/
DATA X/1, 3, -2, 5/
DATA Y/-1, 1, 5, 3/
STATUS = KML_CSPARSE_SCSRSYMV(UPLO, M, A, IA, JA, X, Y)
!
!  OUTPUT Y:
!    31.000000     0.000000    16.000000     9.000000
!
!
```

6.4.1.3.2.6 kml_sparse_?csrmv

矩阵与向量乘积，矩阵是CSR格式的稀疏矩阵。

y = alpha * A * x + beta * y或y = alpha * A^T * x + beta * y或y = alpha * A^H * x + beta * y

其中，alpha和beta是缩放系数，x和y是向量，A是CSR格式的稀疏矩阵。

接口定义

C interface：

kml_sparse_status_t kml_sparse_scsrmv(const kml_sparse_operation_t opt, const KML_INT m, const KML_INT k, const float alpha, const char *matdescra, const float *val, const KML_INT *indx, const KML_INT *pntrb, const KML_INT *pntre, const float *x, const float beta, float *y);

kml_sparse_status_t kml_sparse_dcsrmv(const kml_sparse_operation_t opt, const KML_INT m, const KML_INT k, const double alpha, const char *matdescra, const double *val, const KML_INT *indx, const KML_INT *pntrb, const KML_INT *pntre, const double *x, const double beta, double *y);

kml_sparse_status_t kml_sparse_ccsrmv(const kml_sparse_operation_t opt, const KML_INT m, const KML_INT k, const KML_Complex8 alpha, const char *matdescra, const KML_Complex8 *val, const KML_INT *indx, const KML_INT *pntrb, const KML_INT *pntre, const KML_Complex8 *x, const KML_Complex8 beta, KML_Complex8 *y);

kml_sparse_status_t kml_sparse_zcsrmv(const kml_sparse_operation_t opt, const KML_INT m, const KML_INT k, const KML_Complex16 alpha, const char *matdescra, const KML_Complex16 *val, const KML_INT *indx, const KML_INT *pntrb, const KML_INT *pntre, const KML_Complex16 *x, const KML_Complex16 beta, KML_Complex16 *y);

Fortran interface：

RES = KML_SPARSE_SCSRMV(OPT, M, K, ALPHA, MATDESCRA, VAL, INDX, PNTRB, PNTRE, X, BETA, Y);

RES = KML_SPARSE_DCSRMV(OPT, M, K, ALPHA, MATDESCRA, VAL, INDX, PNTRB, PNTRE, X, BETA, Y);

RES = KML_SPARSE_CCSRMV(OPT, M, K, ALPHA, MATDESCRA, VAL, INDX, PNTRB, PNTRE, X, BETA, Y);

RES = KML_SPARSE_ZCSRMV(OPT, M, K, ALPHA, MATDESCRA, VAL, INDX, PNTRB, PNTRE, X, BETA, Y);

 版权所有 © 华为技术有限公司

参数

参数名	类型	描述	输入/输出
opt	枚举类型 kml_sparse_operation_t	表示矩阵A的操作。 • 如果opt=KML_SPARSE_OPERATION_NON_TRANSPOSE，则y = alpha * A * x + beta * y。 • 如果opt=KML_SPARSE_OPERATION_TRANSPOSE，则y = alpha * A^T * x + beta * y。 • 如果opt=KML_SPARSE_OPERATION_CONJUGATE_TRANSPOSE，则y = alpha * A^H * x + beta * y。	输入
m	整型数	表示A矩阵的行数，取值范围为[1, MAX_KML_INT]。	输入
k	整型数	表示A矩阵的列数，取值范围为[1, MAX_KML_INT]。	输入
alpha	• 在scsrmv中，alpha是单精度浮点类型。 • 在dcsrmv中，alpha是双精度浮点类型。 • 在ccsrmv中，alpha是单精度复数。 • 在zcsrmv中，alpha是双精度复数。	表示系数。	输入
matdescra	char指针	矩阵的操作属性，具体参考**表5 支持的matdescra可能取值**说明。	输入
val	• 在scsrmv中，val是单精度浮点数组。 • 在dcsrmv中，val是双精度浮点数组。 • 在ccsrmv中，val是单精度复数数组。 • 在zcsrmv中，val是双精度复数数组。	A矩阵中的非零元素。	输入
indx	整型数组	indx[i]表示val数组中第i个元素在矩阵A中的列序号。	输入

版权所有 © 华为技术有限公司

参数名	类型	描述	输入/输出
pntrb	整型数组	长度为m的数组，包含矩阵A的行索引，pntrb[i] - pntrb[0]表示第i行第一个非零元素在val和indx数组内的下标。	输入
pntre	整型数组	长度为m的数组，包含矩阵A的行索引，pntre[i] - pntrb[0]-1表示第i行最后一个非零元素在val和indx数组内的下标。	输入
x	• 在scsrmv中，x是单精度浮点数组。 • 在dcsrmv中，x是双精度浮点数组。 • 在ccsrmv中，x是单精度复数数组。 • 在zcsrmv中，x是双精度复数数组。	向量x。	输入
beta	• 在scsrmv中，beta是单精度浮点类型。 • 在dcsrmv中，beta是双精度浮点类型。 • 在ccsrmv中，beta是单精度复数。 • 在zcsrmv中，beta是双精度复数。	表示系数。	输入
y	• 在scsrmv中，y是单精度浮点数组。 • 在dcsrmv中，y是双精度浮点数组。 • 在ccsrmv中，y是单精度复数数组。 • 在zcsrmv中，y是双精度复数数组。	向量y，更新后输出。	输入/输出

⚠ 注意

函数内部不对参数做完整校验，接口调用者需保证pntrb和pntre内部元素不超出矩阵最大索引值。

 版权所有 © 华为技术有限公司

返回值

函数执行状态，枚举类型kml_sparse_status_t。

依赖

C: "kspblas.h"

Fortran: "kspblas.f03"

示例

C interface：

```
kml_sparse_operation_t opt = KML_SPARSE_OPERATION_NON_TRANSPOSE;
KML_INT m = 4;
KML_INT k = 4;
float alpha = 0.5;
float beta = 1.2;
char *matdescra = "G00F"; //一般矩阵，基1索引
float val[9] = {2, -3, 7, 1, -6, 8, -4, 5, 9};
KML_INT indx[9] = {1, 2, 4, 3, 4, 1, 3, 4, 1};
KML_INT pntrb[4] = {1, 4, 6, 9};
KML_INT pntre[4] = {4, 6, 9, 10};
float x[4] = {1, 3, -2, 5};
float y[4] = {-1, 1, 5, 3};
kml_sparse_status_t status = kml_sparse_scsrmv(opt, m, k, alpha, matdescra, val, indx, pntrb, pntre, x,
beta, y);
/*
 *  Output Y:
 *     12.80  -14.80  26.50   8.10
 *
 */
```

Fortran interface：

```
INTEGER(C_INT) :: OPT = KML_SPARSE_OPERATION_NON_TRANSPOSE
INTEGER(C_INT) :: M = 4
INTEGER(C_INT) :: K = 4
REAL(C_FLOAT) :: ALPHA = 0.5
REAL(C_FLOAT) :: BETA = 1.2
CHARACTER(KIND=C_CHAR, LEN=4) :: MATDESCRA = "G00F" !一般矩阵，基1索引
REAL(C_FLOAT) :: VAL(9), X(4), Y(4)
INTEGER(C_INT) INDX(9), PNTRB(4), PNTRE(4)
INTEGER(C_INT) :: STATUS
DATA INDX/1, 2, 4, 3, 4, 1, 3, 4, 1/
DATA PNTRB/1, 4, 6, 9/
DATA PNTRE/4, 6, 9, 10/
DATA VAL /2, -3, 7, 1, -6, 8, -4, 5, 9/
DATA X/1, 3, -2, 5/
DATA Y/-1, 1, 5, 3/
STATUS = KML_SPARSE_SCSRMV(OPT, M, K, ALPHA, MATDESCRA, VAL, INDX, PNTRB, PNTRE, X, BETA, Y)
!
!  OUTPUT Y:
!     12.80  -14.80  26.50   8.10
!
```

6.4.1.3.2.7 kml_sparse_?csrsv

求解三角矩阵方程组计算，稀疏矩阵采用CSR格式存储，具体执行操作如下：

A * y = alpha * x或A^T * y = alpha * x或A^H * y = alpha * x

其中，x和y为向量，A是m * m的稀疏矩阵，采用CSR格式存储。矩阵A中的给定行的非零元素的存储顺序必须与行中显示的顺序相同（从左到右）。

 版权所有 © 华为技术有限公司

接口定义

C interface：

kml_sparse_status_t kml_sparse_scsrsv(const kml_sparse_operation_t opt, const KML_INT m, const float alpha, const char *matdescra, const float *val, const KML_INT *indx, const KML_INT *pntrb, const KML_INT *pntre, const float *x, float *y);

kml_sparse_status_t kml_sparse_dcsrsv(const kml_sparse_operation_t opt, const KML_INT m, const double alpha, const char *matdescra, const double *val, const KML_INT *indx, const KML_INT *pntrb, const KML_INT *pntre, const double *x, double *y);

kml_sparse_status_t kml_sparse_ccsrsv(const kml_sparse_operation_t opt, const KML_INT m, const KML_Complex8 alpha, const char *matdescra, const KML_Complex8 *val, const KML_INT *indx, const KML_INT *pntrb, const KML_INT *pntre, const KML_Complex8 *x, KML_Complex8 *y);

kml_sparse_status_t kml_sparse_zcsrsv(const kml_sparse_operation_t opt, const KML_INT m, const KML_Complex16 alpha, const char *matdescra, const KML_Complex16 *val, const KML_INT *indx, const KML_INT *pntrb, const KML_INT *pntre, const KML_Complex16 *x, KML_Complex16 *y);

Fortran interface：

RES = KML_SPARSE_SCSRSV(OPT, M, ALPHA, MATDESCRA, VAL, INDX, PNTRB, PNTRE, X, Y);

RES = KML_SPARSE_DCSRSV(OPT, M, ALPHA, MATDESCRA, VAL, INDX, PNTRB, PNTRE, X, Y);

RES = KML_SPARSE_CCSRSV(OPT, M, ALPHA, MATDESCRA, VAL, INDX, PNTRB, PNTRE, X, Y);

RES = KML_SPARSE_ZCSRSV(OPT, M, ALPHA, MATDESCRA, VAL, INDX, PNTRB, PNTRE, X, Y);

 版权所有 © 华为技术有限公司

参数

参数名	类型	描述	输入/输出
opt	枚举类型 kml_sparse_operation_t	表示矩阵A的操作。 • 如果opt=KML_SPARSE_OPERATION_NON_TRANSPOSE，则A * y = alpha * x。 • 如果opt=KML_SPARSE_OPERATION_TRANSPOSE，则A^T * y = alpha * x。 • 如果opt=KML_SPARSE_OPERATION_CONJUGATE_TRANSPOSE，则A^H * y = alpha * x。	输入
m	整型数	表示A矩阵的行数，取值范围为[1, MAX_KML_INT]。	输入
alpha	• 在scsrsv中，alpha是单精度浮点类型。 • 在dcsrsv中，alpha是双精度浮点类型。 • 在ccsrsv中，alpha是单精度复数。 • 在zcsrsv中，alpha是双精度复数。	表示系数。	输入
matdescra	char指针	矩阵的操作属性，具体参考**表6-13**说明。	输入
val	• 在scsrsv中，val是单精度浮点数组。 • 在dcsrsv中，val是双精度浮点数组。 • 在ccsrsv中，val是单精度复数数组。 • 在zcsrsv中，val是双精度复数数组。	A矩阵中的非零元素。	输入
indx	整型数组	indx[i]表示val数组中第i个元素在矩阵A中的列序号。	输入

版权所有 © 华为技术有限公司

参数名	类型	描述	输入/输出
pntrb	整型数组	长度为m的数组，包含矩阵A的行索引，pntrb[i] - pntrb[0]表示第i行第一个非零元素在val和indx数组内的下标。	输入
pntre	整型数组	长度为m的数组，包含矩阵A的行索引，pntre[i] - pntrb[0]-1表示第i行最后一个非零元素在val和indx数组内的下标。	输入
x	• 在scsrsv中，x是单精度浮点数组。 • 在dcsrsv中，x是双精度浮点数组。 • 在ccsrsv中，x是单精度复数数组。 • 在zcsrsv中，x是双精度复数数组。	向量x。	输入
y	• 在scsrsv中，y是单精度浮点数组。 • 在dcsrsv中，y是双精度浮点数组。 • 在ccsrsv中，y是单精度复数数组。 • 在zcsrsv中，y是双精度复数数组。	向量y， 更新后输出。	输入/输出

⚠ 注意

函数内部不对参数做完整校验，接口调用者需保证pntrb和pntre内部元素不超出矩阵最大索引值。

返回值

函数执行状态，枚举类型kml_sparse_status_t。

依赖

C: "kspblas.h"

Fortran: "kspblas.f03"

 版权所有 © 华为技术有限公司

示例

C interface：

```
kml_sparse_operation_t opt = KML_SPARSE_OPERATION_NON_TRANSPOSE;
KML_INT m = 4;
float alpha = 1.0;
char *matdescra = "TLNF"; //一般矩阵，基1索引
float val[9] = {2, -3, 7, 1, -6, 8, -4, 5, 9};
KML_INT indx[9] = {1, 2, 4, 2, 4, 1, 3, 1, 4};
KML_INT pntrb[4] = {1, 4, 6, 8};
KML_INT pntre[4] = {4, 6, 8, 10};
float x[4] = {1, 3, -2, 5};
float y[4] = {0, 0, 0, 0};
kml_sparse_status_t status = kml_sparse_scsrsv(opt, m, alpha, matdescra, val, indx, pntrb, pntre, x, y);
/*
 *  Output Y:
 *     0.500000    3.000000    1.500000    0.277778
 *
 */
```

Fortran interface：

```
INTEGER(C_INT) :: OPT = KML_SPARSE_OPERATION_NON_TRANSPOSE
INTEGER(C_INT) :: M = 4
REAL(C_FLOAT) :: ALPHA = 1.0
CHARACTER(KIND=C_CHAR, LEN=4) :: MATDESCRA = "TLNF" !一般矩阵，基1索引
REAL(C_FLOAT) :: VAL(9), X(4), Y(4)
INTEGER(C_INT) :: INDX(9), PNTRB(4),PNTRE(4)
INTEGER(C_INT) :: STATUS
DATA VAL /2, -3, 7, 1, -6, 8, -4, 5, 9/
DATA INDX /1, 2, 4, 2, 4, 1, 3, 1, 4/
DATA PNTRB /1, 4, 6, 8/
DATA PNTRE/4, 6, 8, 10/
DATA X/1, 3, -2, 5/
DATA Y/0, 0, 0, 0/
STATUS = KML_SPARSE_SCSRSV(OPT, M, ALPHA, MATDESCRA, VAL, INDX, PNTRB, PNTRE, X, Y);
!
!  OUTPUT Y:
!     0.500000    3.000000    1.500000    0.277778
!
```

6.4.1.3.2.8 kml_sparse_?csrmm

矩阵与矩阵计算，其中一个稀疏矩阵采用CSR格式存储，具体执行操作如下：

- C = alpha * A*B + beta * C
- C = alpha * A^T * B + beta * C
- C = alpha * A^H * B + beta * C

其中，B和C为稠密矩阵，A是采用CSR格式存储的mxk的稀疏矩阵。

layout与sparse matrix indexing的关系如表4-11所示。

表 6-14 layout 与 sparse matrix indexing 关系

Sparse matrix indexing	Dense matrix layout
KML_SPARSE_INDEX_BASE_ZERO	KML_SPARSE_LAYOUT_ROW_MAJOR
KML_SPARSE_INDEX_BASE_ONE	KML_SPARSE_LAYOUT_COLUMN_MAJOR

版权所有 © 华为技术有限公司

接口定义

C interface：

kml_sparse_status_t kml_sparse_scsrmm(const kml_sparse_operation_t opt, const KML_INT m, const KML_INT n, const KML_INT k, const float alpha, const char *matdescra, const float *val, const KML_INT *indx, const KML_INT *pntrb, const KML_INT *pntre, const float *b, const KML_INT ldb, const float beta , float *c , const KML_INT ldc);

kml_sparse_status_t kml_sparse_dcsrmm(const kml_sparse_operation_t opt, const KML_INT m, const KML_INT n, const KML_INT k, const double alpha, const char *matdescra, const double *val, const KML_INT *indx, const KML_INT *pntrb, const KML_INT *pntre, const double *b, const KML_INT ldb, const double beta , double *c , const KML_INT ldc);

kml_sparse_status_t kml_sparse_ccsrmm(const kml_sparse_operation_t opt, const KML_INT m, const KML_INT n, const KML_INT k, const KML_Complex8 alpha, const char *matdescra, const KML_Complex8 *val, const KML_INT *indx, const KML_INT *pntrb, const KML_INT *pntre, const KML_Complex8 *b, const KML_INT ldb, const KML_Complex8 beta , KML_Complex8 *c , const KML_INT ldc);

kml_sparse_status_t kml_sparse_zcsrmm(const kml_sparse_operation_t opt, const KML_INT m, const KML_INT n, const KML_INT k, const KML_Complex16 alpha, const char *matdescra, const KML_Complex16 *val, const KML_INT *indx, const KML_INT *pntrb, const KML_INT *pntre, const KML_Complex16 *b, const KML_INT ldb, const KML_Complex16 beta , KML_Complex16 *c , const KML_INT ldc);

Fortran interface：

RES = KML_SPARSE_SCSRMM(OPT, M, N, K, ALPHA, MATDESCRA, VAL, INDX, PNTRB, PNTRE, B, LDB, BETA, C, LDC);

RES = KML_SPARSE_DCSRMM(OPT, M, N, K, ALPHA, MATDESCRA, VAL, INDX, PNTRB, PNTRE, B, LDB, BETA, C, LDC);

RES = KML_SPARSE_CCSRMM(OPT, M, N, K, ALPHA, MATDESCRA, VAL, INDX, PNTRB, PNTRE, B, LDB, BETA, C, LDC);

RES = KML_SPARSE_ZCSRMM(OPT, M, N, K, ALPHA, MATDESCRA, VAL, INDX, PNTRB, PNTRE, B, LDB, BETA, C, LDC);

 版权所有 © 华为技术有限公司

参数

参数名	类型	描述	输入/输出
opt	枚举类型 kml_sparse_operation_t	表示是否转置： • KML_SPARSE_OPERATION_NON_TRANSPOSE，C := alpha*A*B + beta*C。 • KML_SPARSE_OPERATION_TRANSPOSE，C := alpha*A^T*B + beta*C。 • KML_SPARSE_OPERATION_CONJUGATE_TRANSPOSE，C := alpha*A^H*B + beta*C。	输入
m	整型数	矩阵A的行数，取值范围为[1, MAX_KML_INT]。	输入
n	整型数	矩阵C的列数，取值范围为[1, MAX_KML_INT]。	输入
k	整型数	矩阵A的列数，取值范围为[1, MAX_KML_INT]。	输入
alpha	• 在scsrmm中，alpha是单精度浮点类型。 • 在dcsrmm中，alpha是双精度浮点类型。 • 在ccsrmm中，alpha是单精度复数。 • 在zcsrmm中，alpha是双精度复数。	标量alpha。	输入
matdescra	char指针	矩阵的操作属性，具体参考**表5 支持的matdescra可能取值**说明。	输入
val	• 在scsrmm中，val是单精度浮点数组。 • 在dcsrmm中，val是双精度浮点数组。 • 在ccsrmm中，val是单精度复数数组。 • 在zcsrmm中，val是双精度复数数组。	CSR格式中values数组，存储矩阵A的非零元素，长度为pntre[m-1] - pntrb[0]。	输入
indx	整型数组	CSR格式中columns数组，用于表示矩阵A中非零元素所在的列索引。	输入

版权所有 © 华为技术有限公司

参数名	类型	描述	输入/输出
pntrb	整型数组	长度为m的数组，包含矩阵A的行索引，pntrb[i] - pntrb[0]表示第i行第一个非零元素在val和indx数组内的下标。	输入
pntre	整型数组	长度为m的数组，包含矩阵A的行索引，pntre[i] - pntrb[0]-1表示第i行最后一个非零元素在val和indx数组内的下标。	输入
b	• 在scsrmm中，b是单精度浮点数组。 • 在dcsrmm中，b是双精度浮点数组。 • 在ccsrmm中，b是单精度复数数组。 • 在zcsrmm中，b是双精度复数数组。	矩阵B的value数组。	输入
ldb	整型数	• 矩阵indx从1开始时，矩阵B的主维度大小。 • 矩阵indx从0开始时，矩阵B的第二维度大小。	输入
beta	• 在scsrmm中，beta是单精度浮点类型。 • 在dcsrmm中，beta是双精度浮点类型。 • 在ccsrmm中，beta是单精度复数。 • 在zcsrmm中，beta是双精度复数。	标量beta。	输入
c	• 在scsrmm中，c是单精度浮点数组。 • 在dcsrmm中，c是双精度浮点数组。 • 在ccsrmm中，c是单精度复数数组。 • 在zcsrmm中，c是双精度复数数组。	矩阵C的value数组。	输入/输出

版权所有 © 华为技术有限公司

参数名	类型	描述	输入/输出
ldc	整型数	• 矩阵indx从1开始时，矩阵C的主维度大小。 • 矩阵indx从0开始时，矩阵C的第二维度大小。	输入

B矩阵参数约束如表6-15所示。

表 6-15 B 矩阵参数约束

opt	B矩阵规模	B矩阵数据排布	参数范围
op(A) = A	k * n	行主序	k * ldb ⩽ MAX_KML_INT
op(A) = A	k * n	列主序	ldb * n ⩽ MAX_KML_INT
op(A) = A^T或A^H	m * n	行主序	m * ldb ⩽ MAX_KML_INT
op(A) = A^T或A^H	m * n	列主序	ldb * n ⩽ MAX_KML_INT

C矩阵参数约束如表6-16所示。

表 6-16 C 矩阵参数约束

opt	C矩阵规模	C矩阵数据排布	参数范围
op(A) = A	m * n	行主序	m * ldc ⩽ MAX_KML_INT
op(A) = A	m * n	列主序	ldc * n ⩽ MAX_KML_INT
op(A) = A^T或A^H	k * n	行主序	k * ldc ⩽ MAX_KML_INT
op(A) = A^T或A^H	k * n	列主序	ldc * n ⩽ MAX_KML_INT

⚠ 注意

函数内部不对参数做完整校验，接口调用者需保证pntrb和pntre内部元素不超出矩阵最大索引值。

版权所有 © 华为技术有限公司

返回值

函数执行状态，枚举类型kml_sparse_status_t。

依赖

C: "kspblas.h"

Fortran: "kspblas.f03"

示例

C interface：

```
kml_sparse_operation_t opt = KML_SPARSE_OPERATION_NON_TRANSPOSE;
KML_INT m = 3;
KML_INT n = 3;
KML_INT k = 3;
float alpha = 1.0;
float beta = 1.0;
char *matdescra = "G00C"; //一般矩阵，基0索引
float val[4] = {9, 8, 5, 2};
KML_INT indx[4] = {2, 1, 2, 2};
KML_INT pntrb[3] = {0, 2, 3};
KML_INT pntre[3] = {2, 3, 4};
float b[9] = {1, 5, 7, 4, 7, 7, 3, 3, 7};
float c[9] = {0, 8, 8, 0, 2, 8, 3, 5, 6};
KML_INT ldb = 3;
KML_INT ldc = 3;
kml_sparse_status_t status = kml_sparse_scsrmm(opt, m, n, k, alpha, matdescra, val, indx, pntrb, pntre, b, ldb, beta, c, ldc);
/*
 *  Output c:
 *     59.00  91.00  127.00  15.00  17.00  43.00  9.00  11.00  20.00
 *
 */
```

Fortran interface：

```
INTEGER(C_INT) :: OPT = KML_SPARSE_OPERATION_NON_TRANSPOSE
INTEGER(C_INT) :: M = 3
INTEGER(C_INT) :: N = 3
INTEGER(C_INT) :: K = 3
REAL(C_FLOAT) :: ALPHA = 1.0
REAL(C_FLOAT) :: BETA = 1.0
INTEGER(C_INT) :: LDB = 3
INTEGER(C_INT) :: LDC = 3
INTEGER(C_INT) :: STATUS
CHARACTER(KIND=C_CHAR, LEN=4) :: MATDESCRA  = "G00C" !一般矩阵，基0索引
REAL(C_FLOAT) :: VAL(4), B(9), C(9)
INTEGER(C_INT) :: INDX(4), PNTRB(4), PNTRE(3)
DATA VAL/9, 8, 5, 2/
DATA B /1, 5, 7, 4, 7, 7, 3, 3, 7/
DATA C /0, 8, 8, 0, 2, 8, 3, 5, 6/
DATA INDX/2, 1, 2, 2/
DATA PNTRB/0, 2, 3/
DATA PNTRE/2, 3, 4/
STATUS = KML_SPARSE_SCSRMM(OPT, M, N, K, ALPHA, MATDESCRA, VAL, INDX, PNTRB, PNTRE, B, LDB, BETA, C, LDC)
!
!  OUTPUT C:
!     59.00  91.00  127.00  15.00  17.00  43.00  9.00  11.00  20.00
!
```

6.4.1.3.2.9 kml_sparse_?csradd

计算2个CSR格式的稀疏矩阵之和。C = A + beta * op(B)

 版权所有 © 华为技术有限公司

A、B、C为3数组CSR格式稀疏矩阵，仅支持基1索引。

op(B)可能的操作为：

- op(B) = B
- op(B) = B^T
- op(B) = B^H

接口定义

C interface：

kml_sparse_status_t kml_sparse_scsradd(const kml_sparse_operation_t opt, const KML_INT request, const KML_INT sort, const KML_INT m, const KML_INT n, const float *a, const KML_INT *ja, KML_INT *ia, const float beta, const float *b, const KML_INT *jb, const KML_INT *ib, float *c, KML_INT *jc, KML_INT *ic, const KML_INT nzmax);

kml_sparse_status_t kml_sparse_dcsradd(const kml_sparse_operation_t opt, const KML_INT request, const KML_INT sort, const KML_INT m, const KML_INT n, const double *a, const KML_INT *ja, const KML_INT *ia, const double beta, const double *b, const KML_INT *jb, const KML_INT *ib, double *c, KML_INT *jc, KML_INT *ic, const KML_INT nzmax);

kml_sparse_status_t kml_sparse_ccsradd(const kml_sparse_operation_t opt, const KML_INT request, const KML_INT sort, const KML_INT m, const KML_INT n, const KML_Complex8 *a, const KML_INT *ja, const KML_INT *ia, const KML_Complex8 beta, const KML_Complex8 *b, const KML_INT *jb, const KML_INT *ib, KML_Complex8 *c, KML_INT *jc, KML_INT *ic, const KML_INT nzmax);

kml_sparse_status_t kml_sparse_zcsradd(const kml_sparse_operation_t opt, const KML_INT request, const KML_INT sort, const KML_INT m, const KML_INT n, const KML_Complex16 *a, const KML_INT *ja, const KML_INT *ia, const KML_Complex16 beta, const KML_Complex16 *b, const KML_INT *jb, const KML_INT *ib, KML_Complex16 *c, KML_INT *jc, KML_INT *ic, const KML_INT nzmax);

Fortran interface：

RES = KML_SPARSE_SCSRADD(OPT, REQUEST, SORT, M, N, A, JA, IA, BETA, B, JB, IB, C, JC, IC, NZMAX);

RES = KML_SPARSE_DCSRADD(OPT, REQUEST, SORT, M, N, A, JA, IA, BETA, B, JB, IB, C, JC, IC, NZMAX);

RES = KML_SPARSE_CCSRADD(OPT, REQUEST, SORT, M, N, A, JA, IA, BETA, B, JB, IB, C, JC, IC, NZMAX);

RES = KML_SPARSE_ZCSRADD(OPT, REQUEST, SORT, M, N, A, JA, IA, BETA, B, JB, IB, C, JC, IC, NZMAX);

 版权所有 © 华为技术有限公司

参数

参数名	类型	描述	输入/输出
opt	枚举类型 kml_sparse_operation_t	表示是否转置： • KML_SPARSE_OPERATION_NON_TRANSPOSE，C := A+beta*B。 • KML_SPARSE_OPERATION_TRANSPOSE，C := A +beta*B^T。 • KML_SPARSE_OPERATION_CONJUGATE_TRANSPOSE，C := A+beta*B^H。	输入
request	整型数	• 0：执行加法操作，ic，jc，c的内存需要提前申请好。 • 1：仅仅计算ic数组的值，长度为m+1，ic的内存需要提前申请好。 • 2：仅计算jc、c的值，长度为ic[m]-1，为request=1的后续步骤，需要在前一步骤计算好ic。	输入
sort	整型数	• 1：需要对ja内的列索引排序。 • 2：需要对jb内的列索引排序。 • 3：需要对ja和jb内的列索引排序。 • 其它值：不对ja和jb内的列索引排序。此时默认ja和jb内的列索引是有序的。	输入
m	整型数	矩阵A的行数，取值范围为[1, MAX_KML_INT]。	输入
n	整型数	矩阵A的列数。	输入
a	• 在scsradd中，a是单精度浮点数组。 • 在dcsradd中，a是双精度浮点数组。 • 在ccsradd中，a是单精度复数数组。 • 在zcsradd中，a是双精度复数数组。	矩阵A的非零元素数组。	输入

版权所有 © 华为技术有限公司

参数名	类型	描述	输入/输出
ja	整型数组	CSR格式的列索引数组。	输入
ia	整型数组	长度为m+1的数组，包含矩阵A的行索引，ia[i] - ia[0]表示第i行第一个非零元素在a和ja数组内的下标。	输入
beta	• 在scsradd中，beta是单精度浮点类型。 • 在dcsradd中，beta是双精度浮点类型。 • 在ccsradd中，beta是单精度复数。 • 在zcsradd中，beta是双精度复数。	标量beta。	输入
b	• 在scsradd中，b是单精度浮点数组。 • 在dcsradd中，b是双精度浮点数组。 • 在ccsradd中，b是单精度复数数组。 • 在zcsradd中，b是双精度复数数组。	矩阵B的非零元素数组。	输入
jb	整型数组	CSR格式的列索引数组。	输入
ib	整型数组	CSR格式稀疏矩阵B的三数组形式的行索引数组，非转置时大小为m+1，转置时大小为n+1，ib[i] - ib[0]表示第i行第一个非零元素在b和jb数组内的下标。	输入
c	• 在scsradd中，c是单精度浮点数组 • 在dcsradd中，c是双精度浮点数组 • 在ccsradd中，c是单精度复数数组 • 在zcsradd中，c是双精度复数数组	矩阵C的非零元素数组。	输入/输出
jc	整型数组	CSR格式的列索引数组。	输入/输出

 版权所有 © 华为技术有限公司

参数名	类型	描述	输入/输出
ic	整型数组	长度为m+1的数组，包含矩阵C的行索引，ic[i] - ic[0]表示第i行第一个非零元素在c和jc数组内的下标。	输入/输出
nzmax	整型数	矩阵C内的非零元素最大个数，当超过nzmax时，计算过程停止。该参数只有request=0或2时有效。	输入

⚠ 注意

函数内部不对参数做完整校验，接口调用者需保证ia和ja内部元素不超出矩阵最大索引值。

返回值

函数执行状态，枚举类型kml_sparse_status_t。

依赖

C: "kspblas.h"

Fortran: "kspblas.f03"

示例

C interface：

```
kml_sparse_operation_t opt = KML_SPARSE_OPERATION_NON_TRANSPOSE;
KML_INT m = 3;
KML_INT n = 3;
KML_INT request = 0; //同时计算c，jc，ic三个数组
KML_INT sort = 0; //a,b每行元素的列索引输入前已排序
float beta = 1.0;
float a[4] = {9, 4, 2, 2};
KML_INT ja[4] = {1, 3, 2, 2};
KML_INT ia[4] = {1, 3, 4, 5};
float b[4] = {2, 5, 9, 8};
KML_INT jb[4] = {2, 3, 2, 1};
KML_INT ib[4] = {1, 3, 4, 5};
float c[9] = {0, 0, 0, 0, 0, 0, 0, 0, 0};
KML_INT jc[9] = {0, 0, 0, 0, 0, 0, 0, 0, 0};
KML_INT ic[4] = {0, 0, 0, 0};
KML_INT nzmax = 9;
kml_sparse_status_t status = kml_sparse_scsradd(opt, request, sort, m, n, a, ja, ia, beta, b, jb, ib, c, jc, ic,
nzmax);
/*
 *  Output c:
 *    9.00  2.00  9.00  11.00  8.00  2.00
 *  Output jc:
 *    1  2  3  2  1  2
 *  Output ic:
```

 版权所有 © 华为技术有限公司

```
*    1  4  5  7
*
*/
```

Fortran interface：

```
INTEGER(C_INT) :: OPT = KML_SPARSE_OPERATION_NON_TRANSPOSE
INTEGER(C_INT) :: M = 3
INTEGER(C_INT) :: N = 3
INTEGER(C_INT) :: REQUEST = 0 !同时计算C，JC，IC三个数组
INTEGER(C_INT) :: SORT = 0    !A,B每行元素的列索引输入前已排序
REAL(C_FLOAT) :: BETA = 1.0
REAL(C_FLOAT) :: A(4), B(4), C(6)
INTEGER(C_INT) :: JA(4), IA(4), JB(4), IB(4), JC(6), IC(4)
INTEGER(C_INT) :: NZMAX = 9
INTEGER(C_INT) :: STATUS
DATA A/9, 4, 2, 2/
DATA B/2, 5, 9, 8/
DATA C/0, 0, 0, 0, 0, 0/
DATA JA/1, 3, 2, 2/
DATA IA/1, 3, 4, 5/
DATA JB/2, 3, 2, 1/
DATA IB/1, 3, 4, 5/
DATA JC/0, 0, 0, 0, 0, 0/
DATA IC/0, 0, 0, 0/
STATUS = KML_SPARSE_SCSRADD(OPT, REQUEST, SORT, M, N, A, JA, IA, BETA, B, JB, IB, C, JC, IC, NZMAX)
!
!  OUTPUT C:
!    9.00  2.00  9.00  11.00  8.00  2.00
!  OUTPUT JC:
!    1  2  3  2  1  2
!  OUTPUT IC:
!    1  4  5  7
!
```

6.4.1.3.2.10 kml_sparse_?csrmultcsr

矩阵与矩阵计算，其中3个稀疏矩阵采用3-arrays CSR格式存储，为基1索引，具体执行操作为：C = op(A) * B。

其中，A、B、C是采用CSR格式存储的稀疏矩阵。

op(A)根据参数是以下3种情况之一：

- op(A) = A
- op(A) = A^T
- op(A) = A^H

接口定义

C interface：

kml_sparse_status_t kml_sparse_scsrmultcsr(const kml_sparse_operation_t opt, const KML_INT request, const KML_INT sort, const KML_INT m, const KML_INT n, const KML_INT k, const float *a, const KML_INT *ja, const KML_INT *ia, const float *b, const KML_INT *jb, const KML_INT *ib, float *c, KML_INT *jc, KML_INT *ic, const KML_INT nzmax);

kml_sparse_status_t kml_sparse_dcsrmultcsr(const kml_sparse_operation_t opt, const KML_INT request, const KML_INT sort, const KML_INT m, const KML_INT n, const KML_INT k, const double *a, const KML_INT *ja, const KML_INT *ia, const double *b, const KML_INT *jb, const KML_INT *ib, double *c, KML_INT *jc, KML_INT *ic, const KML_INT nzmax);

 版权所有 © 华为技术有限公司

kml_sparse_status_t kml_sparse_ccsrmultcsr(const kml_sparse_operation_t opt, const KML_INT request, const KML_INT sort, const KML_INT m, const KML_INT n, const KML_INT k, const KML_Complex8 *a, const KML_INT *ja, const KML_INT *ia, const KML_Complex8 *b, const KML_INT *jb, const KML_INT *ib, KML_Complex8 *c, KML_INT *jc, KML_INT *ic, const KML_INT nzmax);

kml_sparse_status_t kml_sparse_zcsrmultcsr(const kml_sparse_operation_t opt, const KML_INT request, const KML_INT sort, const KML_INT m, const KML_INT n, const KML_INT k, const KML_Complex16 *a, const KML_INT *ja, const KML_INT *ia, const KML_Complex16 *b, const KML_INT *jb, const KML_INT *ib, KML_Complex16 *c, KML_INT *jc, KML_INT *ic, const KML_INT nzmax);

Fortran interface：

RES = KML_SPARSE_SCSRMULTCSR(OPT, REQUEST, SORT, M, N, K, A, JA, IA, B, JB, IB, C, JC, IC, NZMAX);

RES = KML_SPARSE_DCSRMULTCSR(OPT, REQUEST, SORT, M, N, K, A, JA, IA, B, JB, IB, C, JC, IC, NZMAX);

RES = KML_SPARSE_CCSRMULTCSR(OPT, REQUEST, SORT, M, N, K, A, JA, IA, B, JB, IB, C, JC, IC, NZMAX);

RES = KML_SPARSE_ZCSRMULTCSR(OPT, REQUEST, SORT, M, N, K, A, JA, IA, B, JB, IB, C, JC, IC, NZMAX);

参数

参数名	类型	描述	输入/输出
opt	枚举类型 kml_sparse_operation_t	表示是否转置： • opt = 0，C = A * B。 • opt = 1，C = A^T * B。 • opt = 2，C = A^H * B。	输入
request	整型数	• request=0，ic、jc、c的内存由调用者申请。 • request=1，仅更新ic，ic内存由调用者申请。 • request=2，结合request=1使用，更新jc、c。	输入
sort	整型数	当前sort参数未定义。	输入
m	整型数	矩阵A的行数，取值范围为[1, MAX_KML_INT]。	输入
n	整型数	矩阵A的列数，取值范围为[1, MAX_KML_INT]。	输入
k	整型数	矩阵B的列数，取值范围为[1, MAX_KML_INT]。	输入

 版权所有 © 华为技术有限公司

参数名	类型	描述	输入/输出
a	• 在scsrmultcsr中，a是单精度浮点数组。 • 在dcsrmultcsr中，a是双精度浮点数组。 • 在ccsrmultcsr中，a是单精度复数数组。 • 在zcsrmultcsr中，a是双精度复数数组。	CSR格式中values数组，存储矩阵A的非零元素。	输入
ja	整型数组	CSR格式中columns数组，用于表示矩阵A中非零元素所在的列索引。	输入
ia	整型数组	长度为m+1的数组，包含矩阵A的行索引，ia[i] - ia[0]表示第i行第一个非零元素在a和ja数组内的下标。	输入
b	• 在scsrmultcsr中，b是单精度浮点数组。 • 在dcsrmultcsr中，b是双精度浮点数组。 • 在ccsrmultcsr中，b是单精度复数数组。 • 在zcsrmultcsr中，b是双精度复数数组。	CSR格式中values数组，存储矩阵B非零元素。	输入
jb	整型数组	CSR格式中columns数组，用于表示矩阵B非零元素所在的列索引。	输入
ib	整型数组	长度为m+1的数组，包含矩阵B的行索引，ib[i] – ib[0]表示第i行第一个非零元素在b和jb数组内的下标。	输入
c	• 在scsrmultcsr中，c是单精度浮点数组。 • 在dcsrmultcsr中，c是双精度浮点数组。 • 在ccsrmultcsr中，c是单精度复数数组。 • 在zcsrmultcsr中，c是双精度复数数组。	CSR格式中values数组，存储矩阵C非零元素。	输入/输出
jc	整型数组	CSR格式中columns数组，用于表示矩阵C非零元素所在的列索引。	输入/输出

版权所有 © 华为技术有限公司

参数名	类型	描述	输入/输出
ic	整型数组	长度为m+1的数组，包含矩阵C的行索引，ic[i] – ic[0]表示第i行第一个非零元素在val和indx数组内的下标。	输入/输出
nzmax	整型数	在request=0、2时使用，如果C需要的内存超过nzmax对应内存大小时停止计算。	输入

⚠ 注意

函数内部不对参数做完整校验，接口调用者需保证ia、ja、ib、jb、ic、jc内部元素不超出矩阵最大索引值。

返回值

函数执行状态，枚举类型kml_sparse_status_t。

依赖

C: "kspblas.h"

Fortran: "kspblas.f03"

示例

C interface：

```
   kml_sparse_operation_t opt = KML_SPARSE_OPERATION_NON_TRANSPOSE;
   KML_INT m = 3;
   KML_INT n = 3;
   KML_INT k = 3;
   KML_INT request = 0; //同时计算c，jc，ic三个数组
   KML_INT sort = 3; //a,b每行元素的列索引输入前未排序
   float a[4] = {4, 5, 4, 5};
   KML_INT ja[4] = {3, 2, 1, 3};
   KML_INT ia[4] = {1, 2, 4, 5};
   float b[4] = {7, 7, 6, 2};
   KML_INT jb[4] = {3, 2, 3, 2};
   KML_INT ib[4] = {1, 2, 3, 5};
   float c[9] = {0, 0, 0, 0, 0, 0, 0, 0, 0};
   KML_INT jc[9] = {0, 0, 0, 0, 0, 0, 0, 0, 0};
   KML_INT ic[4] = {0, 0, 0, 0};
   KML_INT nzmax = 9;
   kml_sparse_status_t status = kml_sparse_scsrmultcsr(opt, request, sort, m, n, k, a, ja, ia, b, jb, ib, c, jc, ic,
nzmax);
   /*
    *  Output c:
    *    8.00  24.00  35.00  28.00  10.00  30.00
    *  Output jc:
    *    2  3  2  3  2  3
    *  Output ic:
    *    1  3  5  7
```

 版权所有 © 华为技术有限公司

```
 *
 */
```

Fortran interface：

```
  INTEGER(C_INT) :: OPT = KML_SPARSE_OPERATION_NON_TRANSPOSE
  INTEGER(C_INT) :: M = 3
  INTEGER(C_INT) :: N = 3
  INTEGER(C_INT) :: K = 3
  INTEGER(C_INT) :: REQUEST = 0 !同时计算C，JC，IC三个数组
  INTEGER(C_INT) :: SORT = 3 !A,B每行元素的列索引输入前未排序
  REAL(C_FLOAT) :: A(4), B(4), C(6)
  INTEGER(C_INT) :: JA(4), IA(4), JB(4), IB(4), JC(6), IC(4)
  INTEGER(C_INT) :: NZMAX = 9
  INTEGER(C_INT) :: STATUS
  DATA A/4, 5, 4, 5/
  DATA B/7, 7, 6, 2/
  DATA C/0, 0, 0, 0, 0, 0/
  DATA JA/3, 2, 1, 3/
  DATA IA/1, 2, 4, 5/
  DATA JB/3, 2, 3, 2/
  DATA IB/1, 2, 3, 5/
  DATA JC/0, 0, 0, 0, 0, 0/
  DATA IC/0, 0, 0, 0/
  STATUS = KML_SPARSE_SCSRMULTCSR(OPT, REQUEST, SORT, M, N, K, A, JA, IA, B, JB, IB, C, JC, IC,
NZMAX)
  !
  ! OUTPUT C:
  !   8.00  24.00  35.00  28.00  10.00  30.00
  ! OUTPUT JC:
  !   2  3  2  3  2  3
  ! OUTPUT IC:
  !   1  3  5  7
  !
```

6.4.1.3.2.11 kml_sparse_?csrmultd

矩阵与矩阵计算，其中2个稀疏矩阵采用CSR格式存储，为基1索引，具体执行操作为：C := op(A) * B。

其中，C为稠密矩阵，A和B是采用CSR格式存储的稀疏矩阵。

op(A)根据参数是以下3种情况之一：

- op(A) = A
- op(A) = A^T
- op(A) = A^H

接口定义

C interface：

kml_sparse_status_t kml_sparse_scsrmultd(const kml_sparse_operation_t opt, const KML_INT m, const KML_INT n, const KML_INT k, const float *a, const KML_INT *ja, const KML_INT *ia, const float *b, const KML_INT *jb, const KML_INT *ib, float *c, const KML_INT ldc);

kml_sparse_status_t kml_sparse_dcsrmultd(const kml_sparse_operation_t opt, const KML_INT m, const KML_INT n, const KML_INT k, const double *a, const KML_INT *ja, const KML_INT *ia, const double *b, const KML_INT *jb, const KML_INT *ib, double *c, const KML_INT ldc);

kml_sparse_status_t kml_sparse_ccsrmultd(const kml_sparse_operation_t opt, const KML_INT m, const KML_INT n, const KML_INT k, const KML_Complex8 *a,

 版权所有 © 华为技术有限公司

const KML_INT *ja, const KML_INT *ia, const KML_Complex8 *b, const KML_INT *jb, const KML_INT *ib, KML_Complex8 *c, const KML_INT ldc);

kml_sparse_status_t kml_sparse_zcsrmultd(const kml_sparse_operation_t opt, const KML_INT m, const KML_INT n, const KML_INT k, const KML_Complex16 *a, const KML_INT *ja, const KML_INT *ia, const KML_Complex16 *b, const KML_INT *jb, const KML_INT *ib, KML_Complex16 *c, const KML_INT ldc);

Fortran interface：

RES = KML_SPARSE_SCSRMULTD(OPT, M, N, K, A, JA, IA, B, JB, IB, C, LDC);

RES = KML_SPARSE_DCSRMULTD(OPT, M, N, K, A, JA, IA, B, JB, IB, C, LDC);

RES = KML_SPARSE_CCSRMULTD(OPT, M, N, K, A, JA, IA, B, JB, IB, C, LDC);

RES = KML_SPARSE_ZCSRMULTD(OPT, M, N, K, A, JA, IA, B, JB, IB, C, LDC);

参数

参数名	类型	描述	输入/输出
opt	枚举类型 kml_sparse_operation_t	表示是否转置： ● opt = 0，C := A*B。 ● opt = 1，C := A^T*B。 ● opt = 2，C := A^H*B。	输入
m	整型数	矩阵A的行数，取值范围为[1, MAX_KML_INT]。	输入
n	整型数	矩阵A的列数，取值范围为[1, MAX_KML_INT]。	输入
k	整型数	矩阵B的列数，取值范围为[1, MAX_KML_INT]。	输入
a	● 在scsrmultd中，a是单精度浮点数组。 ● 在dcsrmultd中，a是双精度浮点数组。 ● 在ccsrmultd中，a是单精度复数数组。 ● 在zcsrmultd中，a是双精度复数数组。	CSR格式中values数组，存储矩阵A的非零元素。	输入
ja	整型数组	CSR格式中columns数组，用于表示矩阵A中非零元素所在的列索引。	输入
ia	整型数组	长度为m+1的数组，包含矩阵A的行索引，ia[i] - ia[0]表示第i行第一个非零元素在a和ja数组内的下标。	输入

版权所有 © 华为技术有限公司

参数名	类型	描述	输入/输出
b	• 在scsrmultd中，b是单精度浮点数组。 • 在dcsrmultd中，b是双精度浮点数组。 • 在ccsrmultd中，b是单精度复数数组。 • 在zcsrmultd中，b是双精度复数数组。	CSR格式中values数组，存储矩阵B非零元素。	输入
jb	整型数组	CSR格式中columns数组，用于表示矩阵B非零元素所在的列索引。	输入
ib	整型数组	矩阵B行索引，ib[i] – ib[0]表示第i行第一个非零元素在b和jb数组内的下标。	输入
c	• 在scsrmultd中，c是单精度浮点数组。 • 在dcsrmultd中，c是双精度浮点数组。 • 在ccsrmultd中，c是单精度复数数组。 • 在zcsrmultd中，c是双精度复数数组。	矩阵C的数组。	输出
ldc	整型数	稠密矩阵C的主维度，ldc*k ⩽ MAX_KML_INT。	输入

⚠ 注意

函数内部不对参数做完整校验，接口调用者需保证ia、ja、ib、jb内部元素不超出矩阵最大索引值。

返回值

函数执行状态，枚举类型kml_sparse_status_t。

依赖

C: "kspblas.h"

Fortran: "kspblas.f03"

 版权所有 © 华为技术有限公司

示例

C interface：

```
kml_sparse_operation_t opt = KML_SPARSE_OPERATION_NON_TRANSPOSE;
KML_INT m = 3;
KML_INT n = 3;
KML_INT k = 3;
float a[4] = {1, 5, 4, 5};
KML_INT ja[4] = {3, 1, 1, 1};
KML_INT ia[4] = {1, 3, 4, 5};
float b[4] = {9, 4, 2, 8};
KML_INT jb[4] = {2, 3, 1, 1};
KML_INT ib[4] = {1, 3, 4, 5};
float c[9] = {0, 0, 0, 0, 0, 0, 0, 0, 0};
KML_INT ldc = 3;
kml_sparse_status_t status = kml_sparse_scsrmultd(opt, m, n, k, a, ja, ia, b, jb, ib, c, ldc);
/*
 *  Output c:
 *     8.00  0.00  0.00  45.00  36.00  45.00  20.00  16.00  20.00
 *
 */
```

Fortran interface：

```
INTEGER(C_INT) :: OPT = KML_SPARSE_OPERATION_NON_TRANSPOSE
INTEGER(C_INT) :: M = 3
INTEGER(C_INT) :: N = 3
INTEGER(C_INT) :: K = 3
REAL(C_FLOAT) :: A(4), B(4), C(9)
INTEGER(C_INT) :: JA(4), IA(4), JB(4), IB(4)
INTEGER(C_INT) :: LDC = 3
INTEGER(C_INT) :: STATUS
DATA A/1, 5, 4, 5/
DATA B/9, 4, 2, 8/
DATA C/0, 0, 0, 0, 0, 0, 0, 0, 0/
DATA JA/3, 1, 1, 1/
DATA IA/1, 3, 4, 5/
DATA JB /2, 3, 1, 1/
DATA IB /1, 3, 4, 5/
STATUS = KML_SPARSE_SCSRMULTD(OPT, M, N, K, A, JA, IA, B, JB, IB, C, LDC)
!
!  OUTPUT C:
!     8.00  0.00  0.00  45.00  36.00  45.00  20.00  16.00  20.00
!
```

6.4.1.3.2.12 kml_sparse_?csrtrsv

求解稀疏三角线性方程组计算，稀疏矩阵采用CSR格式存储，具体执行操作如下：

- x = A * y
- x = A^T * y
- x = A^H * y

A为3数组CSR格式稀疏矩阵，x、y为向量，仅支持One-based indexing。矩阵A中的给定行的非零元素的存储顺序必须与行中显示的顺序相同（从左到右）。

接口定义

C interface：

kml_sparse_status_t kml_sparse_scsrtrsv(const kml_sparse_fill_mode_t uplo, const kml_sparse_operation_t opt, const kml_sparse_diag_type_t diag, const KML_INT m, const float *a, const KML_INT *ia, const KML_INT *ja, const float *x, float *y);

 版权所有 © 华为技术有限公司

kml_sparse_status_t kml_sparse_dcsrtrsv(const kml_sparse_fill_mode_t uplo, const kml_sparse_operation_t opt, const kml_sparse_diag_type_t diag, const KML_INT m, const double *a, const KML_INT *ia, const KML_INT *ja, const double *x, double *y);

kml_sparse_status_t kml_sparse_ccsrtrsv(const kml_sparse_fill_mode_t uplo, const kml_sparse_operation_t opt, const kml_sparse_diag_type_t diag, const KML_INT m, const KML_Complex8 *a, const KML_INT *ia, const KML_INT *ja, const KML_Complex8 *x, KML_Complex8 *y);

kml_sparse_status_t kml_sparse_zcsrtrsv(const kml_sparse_fill_mode_t uplo, const kml_sparse_operation_t opt, const kml_sparse_diag_type_t diag, const KML_INT m, const KML_Complex16 *a, const KML_INT *ia, const KML_INT *ja, const KML_Complex16 *x, KML_Complex16 *y);

参数

参数名	类型	描述	输入/输出
uplo	枚举类型 kml_sparse_fill_mode_t uplo	指示矩阵A的上三角或者下三角被使用： ● U：上三角 ● L：下三角	输入
opt	枚举类型 kml_sparse_operation_t	表示矩阵A的操作， ● 如果opt=KML_SPARSE_OPERATION_NON_TRANSPOSE，则$x = A * y$。 ● 如果opt=KML_SPARSE_OPERATION_TRANSPOSE，则$x = A^T * y$。 ● 如果opt=KML_SPARSE_OPERATION_CONJUGATE_TRANSPOSE，则$x = A^H * y$。	输入
diag	枚举类型 kml_sparse_diag_type_t	对角元素是否是单位元素。 U：单位元素 N：非单位元素	输入
m	整型数	表示A矩阵的行数和列数，取值范围为[1, MAX_KML_INT]。	输入

版权所有 © 华为技术有限公司

参数名	类型	描述	输入/输出
a	• 在scsrtrsv中，a是单精度浮点数组。 • 在dcsrtrsv中，a是双精度浮点数组。 • 在ccsrtrsv中，a是单精度复数数组。 • 在zcsrtrsv中，a是双精度复数数组。	A矩阵中的非零元素。	输入
ia	整型数组	长度为m+1的向量， ia[i]-1表示A矩阵第i行第一个非零元素在a数组中的索引。	输入
ja	整型数组	ja[i]表示a数组中第i个元素在矩阵A中的列序号。	输入
x	• 在scsrtrsv中，x是单精度浮点数组。 • 在dcsrtrsv中，x是双精度浮点数组。 • 在ccsrtrsv中，x是单精度复数数组。 • 在zcsrtrsv中，x是双精度复数数组。	向量x。	输入
y	• 在scsrtrsv中，y是单精度浮点数组。 • 在dcsrtrsv中，y是双精度浮点数组。 • 在ccsrtrsv中，y是单精度复数数组。 • 在zcsrtrsv中，y是双精度复数数组。	向量y， 更新后输出。	输入/输出

⚠ 注意

函数内部不对参数做完整校验，接口调用者需保证ia和ja内部元素不超出矩阵最大索引值。

返回值

函数执行状态，枚举类型kml_sparse_status_t。

版权所有 © 华为技术有限公司

依赖

C: "kspblas.h"

Fortran: "kspblas.f03"

示例

C interface：

```
kml_sparse_fill_mode_t uplo = KML_SPARSE_FILL_MODE_LOWER;
kml_sparse_diag_type_t diag = KML_SPARSE_DIAG_NON_UNIT;
kml_sparse_operation_t opt = KML_SPARSE_OPERATION_NON_TRANSPOSE;
KML_INT m = 4;
float val[9] = {2, -2, 1, 5, 3, 8, 6, 2, 2};
KML_INT ja[9] = {4, 1, 4, 2, 3, 3, 4, 4, 2};
KML_INT ia[5] = {1, 3, 6, 8, 10};
float x[4] = {6, 6, 4, 6};
float y[4] = {1, 1, 6, 1};
kml_sparse_status_t status = kml_sparse_scsrtrsv(uplo, opt, diag, m, val, ia, ja, x, y);
/*
*  Output Y:
*     -3.000000 1.200000 0.500000 1.800000
*
* */
```

6.4.1.3.2.13 kml_csparse_?csrtrsv

求解稀疏三角线性方程组计算，稀疏矩阵采用CSR格式存储，具体执行操作如下：

- x = A * y
- x = A^T * y
- x = A^H * y

A为3数组CSR格式稀疏矩阵，x、y为向量，仅支持zero-based indexing。矩阵A中的给定行的非零元素的存储顺序必须与行中显示的顺序相同（从左到右）。

接口定义

C interface：

kml_sparse_status_t kml_csparse_scsrtrsv(const char uplo, const kml_sparse_operation_t opt, const kml_sparse_diag_type_t diag, const KML_INT m, const float *a, const KML_INT *ia, const KML_INT *ja, const float *x, float *y);

kml_sparse_status_t kml_csparse_dcsrtrsv(const char uplo, const kml_sparse_operation_t opt, const kml_sparse_diag_type_t diag, const KML_INT m, const double *a, const KML_INT *ia, const KML_INT *ja, const double *x, double *y);

kml_sparse_status_t kml_csparse_ccsrtrsv(const char uplo, const kml_sparse_operation_t opt, const kml_sparse_diag_type_t diag, const KML_INT m, const KML_Complex8 *a, const KML_INT *ia, const KML_INT *ja, const KML_Complex8 *x, KML_Complex8 *y);

kml_sparse_status_t kml_csparse_zcsrtrsv(const char uplo, const kml_sparse_operation_t opt, const kml_sparse_diag_type_t diag, const KML_INT m, const KML_Complex16 *a, const KML_INT *ia, const KML_INT *ja, const KML_Complex16 *x, KML_Complex16 *y);

 版权所有 © 华为技术有限公司

参数

参数名	类型	描述	输入/输出
uplo	枚举类型 kml_sparse_fill_mode_t uplo	指示矩阵A的上三角或者下三角被使用： ● U：上三角 ● L：下三角	输入
opt	枚举类型 kml_sparse_operation_t	表示矩阵A的操作， ● 如果opt=KML_SPARSE_OPERATION_NON_TRANSPOSE，则$x = A * y$。 ● 如果opt=KML_SPARSE_OPERATION_TRANSPOSE，则$x = A^T * y$。 ● 如果opt=KML_SPARSE_OPERATION_CONJUGATE_TRANSPOSE，则$x = A^H * y$。	输入
diag	枚举类型 kml_sparse_diag_type_t	对角元素是否是单位元素。 ● U：单位元素 ● N：非单位元素	输入
m	整型数	表示A矩阵的行数和列数，取值范围为[1, MAX_KML_INT]。	输入
a	● 在c_scsrtrsv中，a是单精度浮点数组。 ● 在c_dcsrtrsv中，a是双精度浮点数组。 ● 在c_ccsrtrsv中，a是单精度复数数组。 ● 在c_zcsrtrsv中，a是双精度复数数组。	A矩阵中的非零元素。	输入
ia	整型数组	长度为m+1的向量。 ia[i]表示A矩阵第i行第一个非零元素在a数组中的索引。	输入
ja	整型数组	ja[i]表示a数组中第i个元素在矩阵A中的列序号。	输入

版权所有 © 华为技术有限公司

参数名	类型	描述	输入/输出
x	• 在c_scsrtrsv中，x是单精度浮点数组。 • 在c_dcsrtrsv中，x是双精度浮点数组。 • 在c_ccsrtrsv中，x是单精度复数数组。 • 在c_zcsrtrsv中，x是双精度复数数组。	向量x。	输入
y	• 在c_scsrtrsv中，y是单精度浮点数组。 • 在c_dcsrtrsv中，y是双精度浮点数组。 • 在c_ccsrtrsv中，y是单精度复数数组。 • 在c_zcsrtrsv中，y是双精度复数数组。	向量y， 更新后输出。	输入/输出

⚠ 注意

函数内部不对参数做完整校验，接口调用者需保证ia和ja内部元素不超出矩阵最大索引值。

返回值

函数执行状态，枚举类型kml_sparse_status_t。

依赖

C: "kspblas.h"

Fortran: "kspblas.f03"

示例

C interface：

```
kml_sparse_fill_mode_t uplo = KML_SPARSE_FILL_MODE_LOWER;
kml_sparse_diag_type_t diag = KML_SPARSE_DIAG_NON_UNIT;
kml_sparse_operation_t opt = KML_SPARSE_OPERATION_NON_TRANSPOSE;
KML_INT m = 4;
float val[9] = {2, -2, 1, 5, 3, 8, 6, 2, 2};
KML_INT ja[9] = {3, 0, 3, 1, 2, 2, 3, 3, 1};
KML_INT ia[5] = {0, 2, 5, 7, 9};
float x[4] = {6, 6, 4, 6};
float y[4] = {1, 1, 6, 1};
kml_sparse_status_t status = kml_csparse_scsrtrsv(uplo, opt, diag, m, val, ia, ja, x, y);
/*
```

 版权所有 © 华为技术有限公司

```
*  Output Y:
*     -3.000000 1.200000 0.500000 1.800000
*
* */
```

6.4.1.3.2.14 kml_sparse_?cscsv

求解三角矩阵方程组计算，稀疏矩阵采用CSC格式存储，具体执行操作如下：

- A * y = alpha * x
- A^T * y = alpha * x
- A^H * y = alpha * x

其中，x和y为向量，A是m * m的稀疏矩阵，采用CSC格式存储。矩阵A中的给定列的非零元素的存储顺序必须与列中显示的顺序相同（从上到下）。

接口定义

C interface：

kml_sparse_status_t kml_sparse_scscsv(const kml_sparse_operation_t opt, const KML_INT m, const float alpha, const char *matdescra, const float *val, const KML_INT *indx, const KML_INT *pntrb, const KML_INT *pntre, const float *x, float *y);

kml_sparse_status_t kml_sparse_dcscsv(const kml_sparse_operation_t opt, const KML_INT m, const double alpha, const char *matdescra, const double *val, const KML_INT *indx, const KML_INT *pntrb, const KML_INT *pntre, const double *x, double *y);

kml_sparse_status_t kml_sparse_ccscsv(const kml_sparse_operation_t opt, const KML_INT m, const KML_Complex8 alpha, const char *matdescra, const KML_Complex8 *val, const KML_INT *indx, const KML_INT *pntrb, const KML_INT *pntre, const KML_Complex8 *x, KML_Complex8 *y);

kml_sparse_status_t kml_sparse_zcscsv(const kml_sparse_operation_t opt, const KML_INT m, const KML_Complex16 alpha, const char *matdescra, const KML_Complex16 *val, const KML_INT *indx, const KML_INT *pntrb, const KML_INT *pntre, const KML_Complex16 *x, KML_Complex16 *y);

 版权所有 © 华为技术有限公司

参数

参数名	类型	描述	输入/输出
opt	枚举类型 kml_sparse_operation_t	表示矩阵A的操作。 • 如果opt=KML_SPARSE_OPERATION_NON_TRANSPOSE，则A * y = alpha * x。 • 如果opt=KML_SPARSE_OPERATION_TRANSPOSE，则A^T * y = alpha * x 。 • 如果opt=KML_SPARSE_OPERATION_CONJUGATE_TRANSPOSE，则A^H * y = alpha * x。	输入
m	整型数	表示A矩阵的行数，取值范围为[1, MAX_KML_INT]。	输入
alpha	• 在scscsv中，alpha是单精度浮点类型。 • 在dcscsv中，alpha是双精度浮点类型。 • 在ccscsv中，alpha是单精度复数。 • 在zcscsv中，alpha是双精度复数。	表示系数。	输入
matdescra	char指针	矩阵的操作属性，具体参考**表5 支持的matdescra可能取值**说明。	输入
val	• 在scscsv中，val是单精度浮点数组。 • 在dcscsv中，val是双精度浮点数组。 • 在ccscsv中，val是单精度复数数组。 • 在zcscsv中，val是双精度复数数组。	A矩阵中的非零元素。	输入
indx	整型数组	indx[i]表示val数组中第i个元素在矩阵A中的行序号。	输入

版权所有 © 华为技术有限公司

参数名	类型	描述	输入/输出
pntrb	整型数组	长度为m的数组，包含矩阵A的列索引，pntrb[i] - pntrb[0]表示第i列第一个非零元素在val和indx数组内的下标。	输入
pntre	整型数组	长度为m的数组，包含矩阵A的列索引，pntre[i] - pntrb[0]-1表示第i列最后一个非零元素在val和indx数组内的下标。	输入
x	• 在scscsv中，x是单精度浮点数组。 • 在dcscsv中，x是双精度浮点数组。 • 在ccscsv中，x是单精度复数数组。 • 在zcscsv中，x是双精度复数数组。	向量x。	输入
y	• 在scscsv中，y是单精度浮点数组。 • 在dcscsv中，y是双精度浮点数组。 • 在ccscsv中，y是单精度复数数组。 • 在zcscsv中，y是双精度复数数组。	向量y， 更新后输出。	输入/输出

⚠ 注意

函数内部不对参数做完整校验，接口调用者需保证pntrb和pntre内部元素不超出矩阵最大索引值。

返回值

函数执行状态，枚举类型kml_sparse_status_t。

依赖

C: "kspblas.h"

Fortran: "kspblas.f03"

 版权所有 © 华为技术有限公司

示例

C interface：

```
kml_sparse_operation_t opt = KML_SPARSE_OPERATION_NON_TRANSPOSE;
char *matdescra = "TLNC";
KML_INT m = 4;
float alpha = 1.0;
float val[9] = {2, 3, 1, 5, 7, 8, 3, 6, 7};
KML_INT indx[9] = {0, 1, 2, 1, 1, 2, 3, 0, 3};
KML_INT pntrb[4] = {0, 3, 4, 7};
KML_INT pntre[4] = {3, 4, 7, 9};
float x[4] = {6, 6, 4, 6};
float y[4] = {1, 1, 6, 1};
kml_sparse_status_t status = kml_sparse_scscsv(opt, m, alpha, matdescra, val, indx, pntrb, pntre, x, y);
/*
*  Output Y:
*     3.000000 -0.600000 0.125000 0.803571
*
* */
```

6.4.1.3.2.15 kml_sparse_?cscmv

矩阵与向量乘积，矩阵是CSC格式的稀疏矩阵。

- y = alpha * A * x + beta * y
- y = alpha * A^T * x + beta * y
- y = alpha * A^H * x + beta * y

其中，alpha和beta是缩放系数，x和y是向量，A是CSC格式的稀疏向量。

接口定义

C interface：

kml_sparse_status_t kml_sparse_scscmv(const kml_sparse_operation_t opt, const KML_INT m, const KML_INT k, const float alpha, const char *matdescra, const float *val, const KML_INT *indx, const KML_INT *pntrb, const KML_INT *pntre, const float *x, const float beta, float *y);

kml_sparse_status_t kml_sparse_dcscmv(const kml_sparse_operation_t opt, const KML_INT m, const KML_INT k, const double alpha, const char *matdescra, const double *val, const KML_INT *indx, const KML_INT *pntrb, const KML_INT *pntre, const double *x, const double beta, double *y);

kml_sparse_status_t kml_sparse_ccscmv(const kml_sparse_operation_t opt, const KML_INT m, const KML_INT k, const KML_Complex8 alpha, const char *matdescra, const KML_Complex8 *val, const KML_INT *indx, const KML_INT *pntrb, const KML_INT *pntre, const KML_Complex8 *x, const KML_Complex8 beta, KML_Complex8 *y);

kml_sparse_status_t kml_sparse_zcscmv(const kml_sparse_operation_t opt, const KML_INT m, const KML_INT k, const KML_Complex16 alpha, const char *matdescra, const KML_Complex16 *val, const KML_INT *indx, const KML_INT *pntrb, const KML_INT *pntre, const KML_Complex16 *x, const KML_Complex16 beta, KML_Complex16 *y);

版权所有 © 华为技术有限公司

参数

参数名	类型	描述	输入/输出
opt	枚举类型 kml_sparse_operation_t	表示矩阵A的操作。 • 如果opt=KML_SPARSE_OPERATION_NON_TRANSPOSE，则y = alpha * A * x + beta * y。 • 如果opt=KML_SPARSE_OPERATION_TRANSPOSE，则y = alpha * A^T * x + beta * y。 • 如果opt=KML_SPARSE_OPERATION_CONJUGATE_TRANSPOSE，则y = alpha * A^H * x + beta * y。	输入
m	整型数	表示A矩阵的行数，取值范围为[1, MAX_KML_INT]。	输入
k	整型数	表示A矩阵的列数，取值范围为[1, MAX_KML_INT]。	输入
alpha	• 在scscmv中，alpha是单精度浮点类型。 • 在dcscmv中，alpha是双精度浮点类型。 • 在ccscmv中，alpha是单精度复数。 • 在zcscmv中，alpha是双精度复数。	表示系数。	输入
matdescra	char指针	矩阵的操作属性，具体参考**表5 支持的matdescra可能取值**说明。	输入
val	• 在scscmv中，val是单精度浮点数组。 • 在dcscmv中，val是双精度浮点数组。 • 在ccscmv中，val是单精度复数数组。 • 在zcscmv中，val是双精度复数数组。	A矩阵中的非零元素。	输入
indx	整型数组	indx[i]表示val数组中第i个元素在矩阵A中的行序号。	输入

版权所有 © 华为技术有限公司

参数名	类型	描述	输入/输出
pntrb	整型数组	长度为m的数组，包含矩阵A的列索引，pntrb[i] - pntrb[0]表示第i列第一个非零元素在val和indx数组内的下标。	输入
pntre	整型数组	长度为m的数组，包含矩阵A的列索引，pntre[i] - pntrb[0]-1表示第i列最后一个非零元素在val和indx数组内的下标。	输入
x	• 在scscmv中，x是单精度浮点数组。 • 在dcscmv中，x是双精度浮点数组。 • 在ccscmv中，x是单精度复数数组。 • 在zcscmv中，x是双精度复数数组。	向量x。	输入
beta	• 在scscmv中，beta是单精度浮点类型。 • 在dcscmv中，beta是双精度浮点类型。 • 在ccscmv中，beta是单精度复数。 • 在zcscmv中，beta是双精度复数。	表示系数。	输入
y	• 在scscmv中，y是单精度浮点数组。 • 在dcscmv中，y是双精度浮点数组。 • 在ccscmv中，y是单精度复数数组。 • 在zcscmv中，y是双精度复数数组。	向量y， 更新后输出。	输入/输出

⚠ 注意

函数内部不对参数做完整校验，接口调用者需保证pntrb和pntre内部元素不超出矩阵最大索引值。

 版权所有 © 华为技术有限公司

返回值

函数执行状态，枚举类型kml_sparse_status_t。

依赖

C: "kspblas.h"

Fortran: "kspblas.f03"

示例

C interface：

```
kml_sparse_operation_t opt = KML_SPARSE_OPERATION_NON_TRANSPOSE;
char *matdescra = "G00C";
KML_INT m = 4;
KML_INT k = 4;
float alpha = 1.0;
float beta = 1.0;
float val[9] = {2, 3, 1, 5, 7, 8, 3, 6, 7};
KML_INT indx[9] = {0, 1, 2, 1, 1, 2, 3, 0, 3};
KML_INT pntrb[4] = {0, 3, 4, 7};
KML_INT pntre[4] = {3, 4, 7, 9};
float x[4] = {6, 6, 4, 6};
float y[4] = {1, 1, 6, 1};
kml_sparse_status_t status = kml_sparse_scscmv(opt, m, k, alpha, matdescra, val, indx, pntrb, pntre, x, beta, y);

/*
 * Output Y:
 *    49.000000 77.000000 44.000000 55.000000
 *
 * */
```

6.4.1.3.2.16 kml_sparse_?cscmm

矩阵与矩阵计算，其中一个稀疏矩阵采用CSC格式存储。具体执行操作如下：

- C = alpha * A*B + beta * C
- C = alpha * A^T * B + beta * C
- C = alpha * A^H * B + beta * C

其中，B和C为稠密矩阵，A是采用CSC格式存储的mxk的稀疏矩阵。

layout与sparse matrix indexing的关系如表6-17所示。

表 6-17 layout 与 sparse matrix indexing 关系

Sparse matrix indexing	Dense matrix layout
KML_SPARSE_INDEX_BASE_ZERO	KML_SPARSE_LAYOUT_ROW_MAJOR
KML_SPARSE_INDEX_BASE_ONE	KML_SPARSE_LAYOUT_COLUMN_MAJOR

接口定义

C interface：

 版权所有 © 华为技术有限公司

kml_sparse_status_t kml_sparse_scscmm(const kml_sparse_operation_t opt, const KML_INT m, const KML_INT n, const KML_INT k, const float alpha, const char *matdescra, const float *val, const KML_INT *indx, const KML_INT *pntrb, const KML_INT *pntre, const float *b, const KML_INT ldb, const float beta , float *c , const KML_INT ldc);

kml_sparse_status_t kml_sparse_dcscmm(const kml_sparse_operation_t opt, const KML_INT m, const KML_INT n, const KML_INT k, const double alpha, const char *matdescra, const double *val, const KML_INT *indx, const KML_INT *pntrb, const KML_INT *pntre, const double *b, const KML_INT ldb, const double beta , double *c , const KML_INT ldc);

kml_sparse_status_t kml_sparse_ccscmm(const kml_sparse_operation_t opt, const KML_INT m, const KML_INT n, const KML_INT k, const KML_Complex8 alpha, const char *matdescra, const KML_Complex8 *val, const KML_INT *indx, const KML_INT *pntrb, const KML_INT *pntre, const KML_Complex8 *b, const KML_INT ldb, const KML_Complex8 beta , KML_Complex8 *c , const KML_INT ldc);

kml_sparse_status_t kml_sparse_zcscmm(const kml_sparse_operation_t opt, const KML_INT m, const KML_INT n, const KML_INT k, const KML_Complex16 alpha, const char *matdescra, const KML_Complex16 *val, const KML_INT *indx, const KML_INT *pntrb, const KML_INT *pntre, const KML_Complex16 *b, const KML_INT ldb, const KML_Complex16 beta , KML_Complex16 *c , const KML_INT ldc);

参数

参数名	类型	描述	输入/输出
opt	枚举类型 kml_sparse_operation_t	表示是否转置： • KML_SPARSE_OPERATION_NON_TRANSPOSE，C := alpha*A*B + beta*C。 • KML_SPARSE_OPERATION_TRANSPOSE，C := alpha*A^T*B + beta*C。 • KML_SPARSE_OPERATION_CONJUGATE_TRANSPOSE，C := alpha*A^H*B + beta*C。	输入
m	整型数	矩阵A的行数，取值范围为[1, MAX_KML_INT]。	输入
n	整型数	矩阵C的列数，取值范围为[1, MAX_KML_INT]。	输入
k	整型数	矩阵A的列数，取值范围为[1, MAX_KML_INT]。	输入

版权所有 © 华为技术有限公司

参数名	类型	描述	输入/输出
alpha	• 在scscmm中，alpha是单精度浮点类型。 • 在dcscmm中，alpha是双精度浮点类型。 • 在ccscmm中，alpha是单精度复数。 • 在zcscmm中，alpha是双精度复数。	标量alpha。	输入
matdescra	char指针	矩阵的操作属性，具体参考**表5 支持的matdescra可能取值**说明。	输入
val	• 在scscmm中，val是单精度浮点数组。 • 在dcscmm中，val是双精度浮点数组。 • 在ccscmm中，val是单精度复数数组。 • 在zcscmm中，val是双精度复数数组。	CSC格式中values数组，存储矩阵A的非零元素，长度为pntre[m-1] - pntrb[0]。	输入
indx	整型数组	CSC格式中columns数组，用于表示矩阵A中非零元素所在的行索引。	输入
pntrb	整型数组	长度为m的数组，包含矩阵A的列索引，pntrb[i] - pntrb[0]表示第i列第一个非零元素在val和indx数组内的下标。	输入
pntre	整型数组	长度为m的数组，包含矩阵A的列索引，pntre[i] - pntrb[0]-1表示第i列最后一个非零元素在val和indx数组内的下标。	输入
b	• 在scscmm中，b是单精度浮点数组。 • 在dcscmm中，b是双精度浮点数组。 • 在ccscmm中，b是单精度复数数组。 • 在zcscmm中，b是双精度复数数组。	矩阵B的value数组。	输入

 版权所有 © 华为技术有限公司

参数名	类型	描述	输入/输出
ldb	整型数	• 矩阵indx从1开始时，矩阵B的主维度大小。 • 矩阵indx从0开始时，矩阵B的第二维度大小。	输入
beta	• 在scscmm中，beta是单精度浮点类型。 • 在dcscmm中，beta是双精度浮点类型。 • 在ccscmm中，beta是单精度复数。 • 在zcscmm中，beta是双精度复数。	标量beta。	输入
c	• 在scscmm中，c是单精度浮点数组。 • 在dcscmm中，c是双精度浮点数组。 • 在ccscmm中，c是单精度复数数组。 • 在zcscmm中，c是双精度复数数组。	矩阵C的value数组。	输入/输出
ldc	整型数	• 矩阵indx从1开始时，矩阵C的主维度大小。 • 矩阵indx从0开始时，矩阵C的第二维度大小。	输入

B矩阵参数约束如表6-18所示。

表 6-18 B 矩阵参数约束

opt	B矩阵规模	B矩阵数据排布	参数范围
op(A) = A	k * n	行主序	k * ldb ⩽ MAX_KML_INT
op(A) = A	k * n	列主序	ldb * n ⩽ MAX_KML_INT
op(A) = A^T或A^H	m * n	行主序	m * ldb ⩽ MAX_KML_INT
op(A) = A^T或A^H	m * n	列主序	ldb * n ⩽ MAX_KML_INT

版权所有 © 华为技术有限公司

C矩阵参数约束如表6-19所示。

表 6-19 C 矩阵参数约束

opt	C矩阵规模	C矩阵数据排布	参数范围
op(A) = A	m * n	行主序	m * ldc ⩽ MAX_KML_INT
op(A) = A	m * n	列主序	ldc * n ⩽ MAX_KML_INT
op(A) = A^T或A^H	k * n	行主序	k * ldc ⩽ MAX_KML_INT
op(A) = A^T或A^H	k * n	列主序	ldc * n ⩽ MAX_KML_INT

⚠ 注意

函数内部不对参数做完整校验，接口调用者需保证pntrb和pntre内部元素不超出矩阵最大索引值。

返回值

函数执行状态，枚举类型kml_sparse_status_t。

依赖

C: "kspblas.h"

Fortran: "kspblas.f03"

示例

C interface：

```
kml_sparse_operation_t opt = KML_SPARSE_OPERATION_NON_TRANSPOSE;
KML_INT m = 3;
KML_INT n = 3;
KML_INT k = 3;
float alpha = 1.0;
float beta = 1.0;
char *matdescra = "G00C"; //一般矩阵，基0索引
float val[4] = {8, 5, 2, 9};
KML_INT indx[4] = {0, 1, 2, 0};
KML_INT pntrb[3] = {0, 0, 1};
KML_INT pntre[3] = {0, 1, 4};
float b[9] = {1, 5, 7, 4, 7, 7, 3, 3, 7};
float c[9] = {0, 8, 8, 0, 2, 8, 3, 5, 6};
KML_INT ldb = 3;
KML_INT ldc = 3;
kml_sparse_status_t status = kml_sparse_scscmm(opt, m, n, k, alpha, matdescra, val, indx, pntrb, pntre,
b, ldb, beta, c, ldc);

/*
```

 版权所有 © 华为技术有限公司

```
    *  Output c:
    *    59.00  91.00  127.00
15.00  17.00  43.00
9.00  11.00  20.00
    *
    * */
```

6.4.1.3.2.17 kml_sparse_?cscsm

求解三角矩阵方程组计算，其中一个稀疏矩阵采用CSC格式存储，具体执行操作如下：

- A * y = alpha * x
- A^T * y = alpha * x
- A^H * y = alpha * x

其中，x和y为m*n的稠密矩阵，A是采用CSC格式存储的m*m的稀疏矩阵。矩阵A中的给定列的非零元素的存储顺序必须与列中显示的顺序相同（从上到下）。

layout与sparse matrix indexing的关系如表6-20所示。

表 6-20 layout 与 sparse matrix indexing 关系

Sparse matrix indexing	Dense matrix layout
KML_SPARSE_INDEX_BASE_ZERO	KML_SPARSE_LAYOUT_ROW_MAJOR
KML_SPARSE_INDEX_BASE_ONE	KML_SPARSE_LAYOUT_COLUMN_MAJOR

接口定义

C interface：

kml_sparse_status_t kml_sparse_scscsm(const kml_sparse_operation_t opt, const KML_INT m, const KML_INT n, const float alpha, const char *matdescra, const float *val, const KML_INT *indx, const KML_INT *pntrb, const KML_INT *pntre, const float *x, const KML_INT ldx, float *y , const KML_INT ldy);

kml_sparse_status_t kml_sparse_dcscsm(const kml_sparse_operation_t opt, const KML_INT m, const KML_INT n, const double alpha, const char *matdescra, const double *val, const KML_INT *indx, const KML_INT *pntrb, const KML_INT *pntre,const double *x, const KML_INT ldx, double *y , const KML_INT ldy);

kml_sparse_status_t kml_sparse_ccscsm(const kml_sparse_operation_t opt, const KML_INT m, const KML_INT n, const KML_Complex8 alpha, const char *matdescra, const KML_Complex8 *val, const KML_INT *indx, const KML_INT *pntrb, const KML_INT *pntre, const KML_Complex8 *x, const KML_INT ldx, KML_Complex8 *y, const KML_INT ldy);

kml_sparse_status_t kml_sparse_zcscsm(const kml_sparse_operation_t opt, const KML_INT m, const KML_INT n, const KML_Complex16 alpha, const char *matdescra, const KML_Complex16 *val, const KML_INT *indx, const KML_INT *pntrb, const KML_INT *pntre, const KML_Complex16 *x, const KML_INT ldx, KML_Complex16 *y , const KML_INT ldy);

 版权所有 © 华为技术有限公司

参数

参数名	类型	描述	输入/输出
opt	枚举类型 kml_sparse_operation_t	表示是否转置： • KML_SPARSE_OPERATION_NON_TRANSPOSE，A * y = alpha * x。 • KML_SPARSE_OPERATION_TRANSPOSE，A^T * y = alpha * x 。 • KML_SPARSE_OPERATION_CONJUGATE_TRANSPOSE，A^H * y = alpha * x。	输入
m	整型数	矩阵A的行数，取值范围为[1, MAX_KML_INT]。	输入
n	整型数	矩阵x的列数，取值范围为[1, MAX_KML_INT]。	输入
alpha	• 在scscsm中，alpha是单精度浮点类型。 • 在dcscsm中，alpha是双精度浮点类型。 • 在ccscsm中，alpha是单精度复数。 • 在zcscsm中，alpha是双精度复数。	标量alpha。	输入
matdescra	char指针	矩阵的操作属性，具体参考**表5 支持的matdescra可能取值**说明。	输入
val	• 在scscsm中，val是单精度浮点数组。 • 在dcscsm中，val是双精度浮点数组。 • 在ccscsm中，val是单精度复数数组。 • 在zcscsm中，val是双精度复数数组。	CSC格式中values数组，存储矩阵A的非零元素，长度为pntre[m-1] - pntrb[0]。	输入
indx	整型数组	CSC格式中columns数组，用于表示矩阵A中非零元素所在的行索引。	输入

版权所有 © 华为技术有限公司

参数名	类型	描述	输入/输出
pntrb	整型数组	长度为m的数组，包含矩阵A的列索引，pntrb[i] - pntrb[0]表示第i列第一个非零元素在val和indx数组内的下标。	输入
pntre	整型数组	长度为m的数组，包含矩阵A的列索引，pntre[i] - pntrb[0]-1表示第i列最后一个非零元素在val和indx数组内的下标。	输入
x	• 在scscsm中，x是单精度浮点数组。 • 在dcscsm中，x是双精度浮点数组。 • 在ccscsm中，x是单精度复数数组。 • 在zcscsm中，x是双精度复数数组。	矩阵x的value数组。	输入
ldx	整型数	• 矩阵indx从1开始时，矩阵x的主维度大小。 • 矩阵indx从0开始时，矩阵x的第二维度大小。	输入
y	• 在scscsm中，y是单精度浮点数组。 • 在dcscsm中，y是双精度浮点数组。 • 在ccscsm中，y是单精度复数数组。 • 在zcscsm中，y是双精度复数数组。	矩阵y的value数组。	输入/输出
ldy	整型数	• 矩阵indx从1开始时，矩阵y的主维度大小。 • 矩阵indx从0开始时，矩阵y的第二维度大小。	输入

x矩阵参数约束如**表6-21**所示。

 版权所有 © 华为技术有限公司

表 6-21 x 矩阵参数约束

opt	x矩阵规模	x矩阵数据排布	参数范围
op(A) = A或A^T或A^H	m * n	行主序	m * ldb ⩽ MAX_KML_INT
op(A) = A或A^T或A^H	m * n	列主序	ldb * n ⩽ MAX_KML_INT

y矩阵参数约束如表6-22所示。

表 6-22 y 矩阵参数约束

opt	y矩阵规模	y矩阵数据排布	参数范围
op(A) = A或A^T或A^H	m * n	行主序	m * ldc ⩽ MAX_KML_INT
op(A) = A或A^T或A^H	m * n	列主序	ldc * n ⩽ MAX_KML_INT

⚠ 注意

函数内部不对参数做完整校验，接口调用者需保证pntrb和pntre内部元素不超出矩阵最大索引值。

返回值

函数执行状态，枚举类型kml_sparse_status_t。

依赖

C: "kspblas.h"

Fortran: "kspblas.f03"

示例

C interface：

```
kml_sparse_operation_t opt = KML_SPARSE_OPERATION_NON_TRANSPOSE;
KML_INT m = 4;
KML_INT n = 2;
float alpha = 1.0;
char *matdescra = "TLNC"; //三角矩阵，基0索引
float val[9] = {2, 3, 1, 5, 7, 8, 3, 6, 7};
KML_INT indx[9] = {0, 1, 2, 1, 1, 2, 3, 0, 3};
KML_INT pntrb[4] = {0, 3, 4, 7};
KML_INT pntre[4] = {3, 4, 7, 9};
```

 版权所有 © 华为技术有限公司

```
    float x[8] = {6, 7, 6, -3, 4, 6, 6, 8};
    float y[8] = {0, 0, 0, 0, 0, 0, 0, 0};
    KML_INT ldx = 2;
    KML_INT ldy = 2;
    kml_sparse_status_t status = kml_sparse_scscsm(opt, m, n, alpha, matdescra, val, indx, pntrb, pntre, x,
ldx, y, ldy);
    /*
     *  Output Y:
     *      3.000000   3.500000
-0.600000   -2.700000
0.125000   0.312500
0.803571   1.008929
     *
     * */
```

6.4.1.3.2.18 kml_sparse_?csrsm

求解三角矩阵方程组计算，其中一个稀疏矩阵采用CSR格式存储，具体执行操作如下：

- A * y = alpha * x
- A^T * y = alpha * x
- A^H * y = alpha * x

其中，x和y为mxn的稠密矩阵，A是采用CSR格式存储的mxm的稀疏矩阵。矩阵A中的给定行的非零元素的存储顺序必须与行中显示的顺序相同（从左到右）。

layout与sparse matrix indexing的关系如表6-23所示。

表 6-23 layout 与 sparse matrix indexing 关系

Sparse matrix indexing	Dense matrix layout
KML_SPARSE_INDEX_BASE_ZERO	KML_SPARSE_LAYOUT_ROW_MAJOR
KML_SPARSE_INDEX_BASE_ONE	KML_SPARSE_LAYOUT_COLUMN_MAJOR

接口定义

C interface：

kml_sparse_status_t kml_sparse_scsrsm(const kml_sparse_operation_t opt, const KML_INT m, const KML_INT n, const float alpha, const char *matdescra, const float *val, const KML_INT *indx, const KML_INT *pntrb, const KML_INT *pntre, const float *x, const KML_INT ldx, float *y , const KML_INT ldy);

kml_sparse_status_t kml_sparse_dcsrsm(const kml_sparse_operation_t opt, const KML_INT m, const KML_INT n, const double alpha, const char *matdescra, const double *val, const KML_INT *indx, const KML_INT *pntrb, const KML_INT *pntre,const double *x, const KML_INT ldx, double *y , const KML_INT ldy);

kml_sparse_status_t kml_sparse_ccsrsm(const kml_sparse_operation_t opt, const KML_INT m, const KML_INT n, const KML_Complex8 alpha, const char *matdescra, const KML_Complex8 *val, const KML_INT *indx, const KML_INT *pntrb, const KML_INT *pntre, const KML_Complex8 *x, const KML_INT ldx, KML_Complex8 *y, const KML_INT ldy);

kml_sparse_status_t kml_sparse_zcsrsm(const kml_sparse_operation_t opt, const KML_INT m, const KML_INT n, const KML_Complex16 alpha, const char

 版权所有 © 华为技术有限公司

*matdescra, const KML_Complex16 *val, const KML_INT *indx, const KML_INT *pntrb, const KML_INT *pntre, const KML_Complex16 *x, const KML_INT ldx, KML_Complex16 *y , const KML_INT ldy);

参数

参数名	类型	描述	输入/输出
opt	枚举类型 kml_sparse_operation_t	表示是否转置： • KML_SPARSE_OPERATION_NON_TRANSPOSE，A * y = alpha * x。 • KML_SPARSE_OPERATION_TRANSPOSE，A^T * y = alpha * x 。 • KML_SPARSE_OPERATION_CONJUGATE_TRANSPOSE，A^H * y = alpha * x。	输入
m	整型数	矩阵A的行数，取值范围为[1, MAX_KML_INT]。	输入
n	整型数	矩阵x的列数，取值范围为[1, MAX_KML_INT]。	输入
alpha	• 在scsrsm中，alpha是单精度浮点类型。 • 在dcsrsm中，alpha是双精度浮点类型。 • 在ccsrsm中，alpha是单精度复数。 • 在zcsrsm中，alpha是双精度复数。	标量alpha。	输入
matdescra	char指针	矩阵的操作属性，具体参考**表5 支持的matdescra可能取值**说明。	输入
val	• 在scsrsm中，val是单精度浮点数组。 • 在dcsrsm中，val是双精度浮点数组。 • 在ccsrsm中，val是单精度复数数组。 • 在zcsrsm中，val是双精度复数数组。	CSR格式中values数组，存储矩阵A的非零元素，长度为pntre[m-1] - pntrb[0]。	输入
indx	整型数组	CSR格式中columns数组，用于表示矩阵A中非零元素所在的列索引。	输入

版权所有 © 华为技术有限公司

参数名	类型	描述	输入/输出
pntrb	整型数组	长度为m的数组，包含矩阵A的行索引，pntrb[i] - pntrb[0]表示第i行第一个非零元素在val和indx数组内的下标。	输入
pntre	整型数组	长度为m的数组，包含矩阵A的行索引，pntre[i] - pntrb[0]-1表示第i行最后一个非零元素在val和indx数组内的下标。	输入
x	• 在scsrsm中，x是单精度浮点数组。 • 在dcsrsm中，x是双精度浮点数组。 • 在ccsrsm中，x是单精度复数数组。 • 在zcsrsm中，x是双精度复数数组。	矩阵x的value数组。	输入
ldx	整型数组	• 矩阵indx从1开始时，矩阵x的主维度大小。 • 矩阵indx从0开始时，矩阵x的第二维度大小。	输入
y	• 在scsrsm中，y是单精度浮点数组。 • 在dcsrsm中，y是双精度浮点数组。 • 在ccsrsm中，y是单精度复数数组。 • 在zcsrsm中，y是双精度复数数组。	矩阵y的value数组。	输入/输出
ldy	整型数组	• 矩阵indx从1开始时，矩阵y的主维度大小。 • 矩阵indx从0开始时，矩阵y的第二维度大小。	输入

x矩阵参数约束如**表6-24**所示。

 版权所有 © 华为技术有限公司

表 6-24 x 矩阵参数约束

opt	x矩阵规模	x矩阵数据排布	参数范围
op(A) = A或A^T或A^H	m * n	行主序	m * ldb ⩽ MAX_KML_INT
op(A) = A或A^T或A^H	m * n	列主序	ldb * n ⩽ MAX_KML_INT

y矩阵参数约束如表6-25所示。

表 6-25 y 矩阵参数约束

opt	y矩阵规模	y矩阵数据排布	参数范围
op(A) = A或A^T或A^H	m * n	行主序	m * ldc ⩽ MAX_KML_INT
op(A) = A或A^T或A^H	m * n	列主序	ldc * n ⩽ MAX_KML_INT

⚠ 注意

函数内部不对参数进行完整校验，接口调用者需保证pntrb和pntre内部元素不超出矩阵最大索引值。

返回值

函数执行状态，枚举类型kml_sparse_status_t。

依赖

C: "kspblas.h"

Fortran: "kspblas.f03"

示例

C interface：

```
kml_sparse_operation_t opt = KML_SPARSE_OPERATION_NON_TRANSPOSE;
KML_INT m = 4;
KML_INT n = 2;
float alpha = 1.0;
char *matdescra = "TLNF"; //三角矩阵，基1索引
float val[9] = {2, -3, 7, 1, -6, 8, -4, 5, 9};
KML_INT indx[9] = {1, 2, 4, 2, 4, 1, 3, 1, 4};
KML_INT pntrb[4] = {1, 4, 6, 8};
KML_INT pntre[4] = {4, 6, 8, 10};
float x[8] = {1, 3, -2, 5, 7, -3, 6, 8};
```


版权所有 © 华为技术有限公司

```
    float y[8] = {0, 0, 0, 0, 0, 0, 0, 0};
    KML_INT ldx = 4;
    KML_INT ldy = 4;
    kml_sparse_status_t status = kml_sparse_scsrsm(opt, m, n, alpha, matdescra, val, indx, pntrb, pntre, x,
ldx, y, ldy);
    /*
     * Output Y:
     *     0.500000  3.000000  1.500000  0.277778
3.500000  -3.000000  5.500000  -1.055556
     *
     * */
```

6.4.1.3.3 Support 函数

6.4.1.3.3.1 kml_set_thread_num

设置Sparse BLAS库接口函数执行的线程数（此函数不可重入）。

接口定义

C interface：

kml_sparse_status_t kml_set_thread_num(const int thread_num);

Fortran interface：

RES = KML_SET_THREAD_NUM(THREAD_NUM);

参数

参数名	类型	描述	输入/输出
thread_num	整型	线程数，取值范围为1~256。	输入

返回值

函数执行状态，枚举类型kml_sparse_status_t。

6.4.1.3.3.2 KSPBLASGetVersion

获取SPBLAS产品版本信息：

int KSPBLASGetVersion(KSPBLASVersion *ver)

参数

参数名	描述	取值范围	输入/输出
ver	产品信息结构体。	非空	输出

版权所有 © 华为技术有限公司

返回值

- 成功：返回1。
- 失败：返回0。

示例

```
#include <stdio.h>
#include "kspblas.h"

int main()
{
    KSPBLASVersion* ver = (KSPBLASVersion*)malloc(sizeof(KSPBLASVersion));
    int status = KSPBLASGetVersion(ver);
    printf("Product Name            : %s\n", ver->productName);
    printf("Product Version         : %s\n", ver->productVersion);
    printf("Component name          : %s\n", ver->componentName);
    printf("Component Version       : %s\n", ver->componentVersion);
    printf("Component AbiVersion    : %s\n", ver->componentAbiVersion);
    printf("Component AppendInfo    : %s\n", ver->componentAppendInfo);
    printf("Software Name           : %s\n", ver->softwareName);
    printf("Software Version        : %s\n", ver->softwareVersion);
    free(ver);
    return 0;
}
```

运行结果：

```
Product Name            : Kunpeng HPCKit
Product Version         : 24.0.RC1
Component name          : HPCKit-kml
Component Version       : 2.3.0
Component AbiVersion    : 1
Component AppendInfo    : gcc
Software Name           : hpckit-kml-spblas
Software Version        : 2.3.0
```

说明

以上版本号和编译时间以实际运行结果为准，上述结果仅供参考。

6.4.2 KML_VML 库函数说明

6.4.2.1 函数说明

矢量数学库（Vector Math Library）借助计算密集型核心数学函数（幂函数、三角函数、指数函数、双曲函数、对数函数等）的矢量实现显著提升应用速度。

函数命名规则

v<character> <name>( )

其中，<character>取值说明如表6-26所示。<name>取值说明如表6-27所示。

表 6-26 <character>取值说明

<character>	描述
s	实数，单精度浮点型
c	复数，单精度浮点型

版权所有 © 华为技术有限公司

<character>	描述
d	实数，双精度浮点型
z	复数，双精度浮点型
h	实数，半精度浮点型
ch	复数，半精度浮点型

表 6-27 <name>取值说明

<name>	数据类型	描述
add	s，d，c，z，h，ch	加法函数。
sub	s，d，c，z，h，ch	减法函数。
sqr	s，d	平方函数。
mul	s，d，c，z，h，ch	乘法函数。
div	s，d，c，z，h，ch	除法函数。
sqrt	s，d，c，z	开平方函数。
cbrt	s，d	开立方函数。
abs	s，d，c，z	计算实数的绝对值/计算复数的模。
hypot	s，d	计算直角三角形斜边。
inv	s，d	计算倒数。
invcbrt	s，d	计算立方根倒数。
invsqrt	s，d	计算平方根倒数。
linearfrac	s，d	线性分式变换函数。
remainder	s，d	计算余数。
pow	s，d，c，z	幂函数a^b。
powx	s，d，c，z	幂函数a^b（b为标量）。
powr	s，d	幂函数a^b（a >= 0）。
pow2o3	s，d	计算x的2/3次方。
pow3o2	s，d	计算x的3/2次方。
exp	s，d，c，z	以自然对数为底的指数函数。

 版权所有 © 华为技术有限公司

<name>	数据类型	描述
exp2	s，d	以2为底的指数函数。
exp10	s，d	以10为底的指数函数。
expm1	s，d	以自然对数为底的指数函数再减1。
ln	s，d，c，z	以自然对数为底的对数函数。
log10	s，d，c，z	以10为底的对数函数。
log1p	s，d	以自然对数为底的对数函数再加1。
log2	s，d	以2为底的对数函数。
logb	s，d	以2为底的对数函数值取整。
cos	s，d，c，z	余弦函数。
sin	s，d，c，z	正弦函数。
tan	s，d，c，z	正切函数。
atan	s，d，c，z	反正切函数。
atanpi	s，d	计算反正切值除以π的值。
atan2	s，d	两个变量四个象限下的反正切函数。
atan2pi	s，d	计算两个变量四个象限下的反正切值除以PI的值。
sincos	s，d	正弦函数与余弦函数。
sinh	s，d，c，z	双曲正弦函数。
cosh	s，d，c，z	双曲余弦函数。
tanh	s，d，c，z	双曲正切函数。
asinh	s，d，c，z	双曲反正弦函数。
acosh	s，d，c，z	双曲反余弦函数。
atanh	s，d，c，z	双曲反正切函数。
sind	s，d	入参为角度的正弦函数。
cosd	s，d	入参为角度的余弦函数。
trunc	s，d	计算x的趋向于0的取整舍入值。
round	s，d	计算x的四舍五入的取整舍入值。
nearbyint	s，d	计算x的基于当前舍入模式的取整舍入值。
rint	s，d	计算x的基于当前舍入模式的取整舍入值。
frac	s，d	计算x的小数部分。

版权所有 © 华为技术有限公司

<name>	数据类型	描述
fmod	s，d	计算模数。
arg	c，z	计算幅角。
conj	c，z	计算共轭值。
mulbyconj	c，z	第一个向量中的元素乘以第二个向量中的共轭值。
cis	c，z	矢量计算以入参实数x为输入，返回 $e^{ix} = (\cos x + i \cdot \sin x)$ 的复数结果。
acos	s，d，c，z	反余弦函数。
asin	s，d，c，z	反正弦函数。
cospi	s，d	矢量计算实数*π的余弦值。
sinpi	s，d	矢量计算实数*π的正弦值。
tanpi	s，d	矢量计算实数*π的正切值。
acospi	s，d	矢量计算实数的反余弦值/π。
asinpi	s，d	矢量计算实数的反正弦值/π。
tand	s，d	入参为角度的正切函数。
erf	s，d	矢量计算x的误差函数值。
erfc	s，d	矢量计算x的互补误差函数。
cdfnorm	s，d	矢量计算x的正态分布累积函数值。
erfinv	s，d	矢量计算x的误差函数逆函数值。
erfcinv	s，d	矢量计算x的互补误差函数逆函数值。
cdfnorminv	s，d	矢量计算x的正态分布累积函数的逆函数值。
lgamma	s，d	矢量计算x的伽马值绝对值的自然对数值。
tgamma	s，d	矢量计算x的伽马值。
expint1	s，d	矢量计算指数积分值。
floor	s，d	矢量计算不大于x的最大整数值。
ceil	s，d	矢量计算不小于x的最小整数值。
modf	s，d	矢量计算输入值的整数和小数部分。
copysign	s，d	矢量计算将入参2的数据符号位拷贝至入参1中。
nextafter	s，d	在y方向上，以指定格式返回x之后的下一个可表示值。
fdim	s，d	如果x>y，矢量计算x的y差值，否则返回0。

版权所有 © 华为技术有限公司

<name>	数据类型	描述
fmax	s，d，h	矢量计算x和y中更大的值。
fmin	s，d，h	矢量计算x和y中更小的值。
fmaxmag	s，d	矢量计算x和y中幅度更大的值。
fminmag	s，d	矢量计算x和y中幅度更小的值。
vfloatcvtf16	s	将float数据类型转换为fp16。
vf16cvtfloat	s	将fp16数据类型转换为float。

6.4.2.2 环境变量

以下环境变量会被KML_VML使用：

环境变量名	变量意义	有效值	默认值
OMP_NUM_THREADS	OMP内置线程数，用于控制KML_VML函数并行执行线程数，不支持线程嵌套的场景	数字	0
KML_VML_NUM_THREADS	VML内置线程数，用于控制KML_VML函数并行执行线程数，优先级高于‘OMP_NUM_THREADS’，不支持线程嵌套的场景	数字	0

6.4.2.3 函数定义

6.4.2.3.1 v?add

计算被加数和加数的和，被加数、加数与相加的和均为向量。

接口定义

C interface：

void vsadd(const int len, const float* src1, const float* src2, float* dst);

void vdadd(const int len, const double* src1, const double* src2, double* dst);

void vcadd(const int len, const float complex *src1, const float complex *src2, float complex *dst);

void vzadd(const int len, const double complex *src1, const double complex *src2, double complex *dst);

 版权所有 © 华为技术有限公司

void vhadd(const int len, const __fp16 *src1, const __fp16 *src2, __fp16 *dst);

void vchadd(const int len, const complex_fp16 *src1, const complex_fp16 *src2, complex_fp16 *dst);

Fortran interface：

CALL VSADD(LEN, SRC1, SRC2, DST);

CALL VDADD(LEN, SRC1, SRC2, DST);

CALL VZADD(LEN, SRC1, SRC2, DST);

参数

参数名	类型	描述	输入/输出
len	整型数	表示输入向量的元素个数。 len≤0时会提示参数len无效并返回。	输入
src1	• 在vsadd中是单精度浮点类型。 • 在vdadd中是双精度浮点类型。 • 在vcadd中是复数单精度浮点类型。 • 在vzadd中是复数双精度浮点类型。 • 在vhadd中是半精度浮点数。 • 在vchadd中是复数半精度浮点数。	被加数向量src1，向量长度为len。 若为空指针，会提示空指针错误并返回。	输入
src2	• 在vsadd中是单精度浮点类型。 • 在vdadd中是双精度浮点类型。 • 在vcadd中是复数单精度浮点类型。 • 在vzadd中是复数双精度浮点类型。 • 在vhadd中是半精度浮点数。 • 在vchadd中是复数半精度浮点数。	加数向量src2，向量长度为len。 若为空指针，会提示空指针错误并返回。	输入

版权所有 © 华为技术有限公司

参数名	类型	描述	输入/输出
dst	• 在vsadd中是单精度浮点类型。 • 在vdadd中是双精度浮点类型。 • 在vcadd中是复数单精度浮点类型。 • 在vzadd中是复数双精度浮点类型。 • 在vhadd中是半精度浮点数。 • 在vchadd中是复数半精度浮点数。	向量和dst，向量长度为len。 若为空指针，会提示空指针错误并返回。	输出

输出结果

- 每一个运算值返回两个数的和，取值范围为(-INF, +INF)。
- 对于vsadd和vdadd，有以下输出情况。

输入值（src1）	输入值（src2）	输出值（dst）
+inf	-inf	nan
-inf	-inf	-inf
+inf	+inf	+inf
-inf	+inf	nan
nan	nan	nan

- 对于vcadd和vzadd，请在社区页面文档本章节下载附件查看特殊输出结果。

依赖

C: "kvml.h"

示例

C interface：

```
int i, len = 4;
float src1[len] = {0.0f, 0.5f, 1.5f, 5.5f};
float src2[len] = {0.3f, 0.7f, 2.5f, 10.0f};
float* dst = (float*)malloc(sizeof(float) * len);
if (dst == NULL) {
    printf("Malloc Failed!\n");
    return 0;
}
vsadd(len, src1, src2, dst);
/**
```

 版权所有 © 华为技术有限公司

```
 *  Output dst:
 *     0.3  1.2  4.0  15.5
 *
 */
```

Fortran interface：

```
INTEGER :: LEN = 4
REAL(4) SRC1(4)
REAL(4) SRC2(4)
REAL(4) DST(4)
DATA SRC1 /1.2, 2.5, -2.4, 3.6/
DATA SRC2 /8.6, -2.5, -12.4, 32.6/
CALL VSADD(LEN, SRC1, SRC2, DST)
!
! OUTPUT DST
!     9.800000191, 0.000000000, -14.799999237, 36.199996948
!
```

6.4.2.3.2 v?sub

计算被减数和减数的差，被减数、减数和结果均为向量。

接口定义

C interface：

void vssub(const int len, const float* src1, const float* src2, float* dst);

void vdsub(const int len, const double* src1, const double* src2, double* dst);

void vcsub(const int len, const float complex *src1, const float complex *src2, float complex *dst);

void vzsub(const int len, const double complex *src1, const double complex *src2, double complex *dst);

void vhsub(const int len, const __fp16 *src1, const __fp16 *src2, __fp16 *dst);

void vchsub(const int len, const complex_fp16 *src1, const complex_fp16 *src2, complex_fp16 *dst);

Fortran interface：

CALL VSSUB(LEN, SRC1, SRC2, DST);

CALL VDSUB(LEN, SRC1, SRC2, DST);

参数

参数名	类型	描述	输入/输出
len	整型数	表示输入向量的元素个数。 len≤0时会提示参数len无效并返回。	输入

 版权所有 © 华为技术有限公司

参数名	类型	描述	输入/输出
src1	• 在vssub中是单精度浮点类型。 • 在vdsub中是双精度浮点类型。 • 在vcsub中是复数单精度浮点类型。 • 在vzsub中是复数双精度浮点类型。 • 在vhsub中是半精度浮点数。 • 在vchsub中是复数半精度浮点数。	被减数向量src1，向量长度为len。 若为空指针，会提示空指针错误并返回。	输入
src2	• 在vssub中是单精度浮点类型。 • 在vdsub中是双精度浮点类型。 • 在vcsub中是复数单精度浮点类型。 • 在vzsub中是复数双精度浮点类型。 • 在vhsub中是半精度浮点数。 • 在vchsub中是复数半精度浮点数。	减数向量src2，向量长度为len。 若为空指针，会提示空指针错误并返回。	输入
dst	• 在vssub中是单精度浮点类型。 • 在vdsub中是双精度浮点类型。 • 在vcsub中是复数单精度浮点类型。 • 在vzsub中是复数双精度浮点类型。 • 在vhsub中是半精度浮点数。 • 在vchsub中是复数半精度浮点数。	输出向量dst，向量长度为len。 若为空指针，会提示空指针错误并返回。	输出

输出结果

- 每一个运算值返回两个数的差，取值范围为(-INF, +INF)。

 版权所有 © 华为技术有限公司

- 对于vssub和vdsub，有以下输出情况。

输入值（src1）	输入值（src2）	输出值（dst）
+inf	-inf	+inf
-inf	-inf	nan
+inf	+inf	nan
-inf	+inf	-inf

- 对于vcsub和vzsub，请在社区页面文档本章节下载附件查看特殊输出结果。

依赖

C: "kvml.h"

示例

C interface：

```
int i, len = 4;
float src1[len] = {0.3f, 0.7f, 2.5f, 10.0f};
float src2[len] = {0.0f, 0.5f, 1.5f, 5.5f};
float* dst = (float*)malloc(sizeof(float) * len);
if (dst == NULL) {
    printf("Malloc Failed!\n");
    return 0;
}
vssub(len, src1, src2, dst);
/**
 *  Output dst:
 *     0.3  0.2  1.0  4.5
 *
 */
```

Fortran interface：

```
INTEGER :: LEN = 4
REAL(4) SRC1(4)
REAL(4) SRC2(4)
REAL(4) DST(4)
DATA SRC1 /1.2, 2.5, -2.4, 3.6/
DATA SRC2 /8.6, -2.5, -12.4, 32.6/
CALL VSSUB(LEN, SRC1, SRC2, DST)
!
! OUTPUT DST
!     -7.400000572, 5.000000000, 10.000000000, -28.999998093
!
```

6.4.2.3.3 v?sqr

计算输入的平方值，输入为向量。

接口定义

C interface：

void vssqr(const int len, const float* src, float* dst);

void vdsqr(const int len, const double* src, double* dst);

 版权所有 © 华为技术有限公司

Fortran interface：

CALL VSSQR(LEN, SRC, DST);

CALL VDSQR(LEN, SRC, DST);

参数

参数名	类型	描述	输入/输出
len	整型数	表示输入向量的元素个数。 len≤0时会提示参数len无效并返回。	输入
src	• 在vssqr中是单精度浮点类型。 • 在vdsqr中是双精度浮点类型。	输入向量src，向量长度为len。 若为空指针，会提示空指针错误并返回。	输入
dst	• 在vssqr中是单精度浮点类型。 • 在vdsqr中是双精度浮点类型。	输出dst，向量长度为len。 若为空指针，会提示空指针错误并返回。	输出

输出结果

每一个运算值返回数的平方，返回值范围为[0, +INF)。

输入值（src）	输出值（dst）
+inf	+inf
-inf	+inf
nan	nan

依赖

C: "kvml.h"

示例

C interface：

```
int i, len = 4;
float src[len] ={0.0f, 0.5f, 1.0f, 5.0f};
float* dst = (float*)malloc(sizeof(float) * len);
if (dst == NULL) {
    printf("Malloc Failed!\n");
    return 0;
}
vssqr(len, src, dst);
```

 版权所有 © 华为技术有限公司

```
/**
 *  Output dst:
 *     0.0  0.25  1.0  25.0
 *
 */
```

Fortran interface：

```
INTEGER :: LEN = 4
REAL(4) SRC(4)
REAL(4) DST(4)
DATA SRC /1.1, 2.6, 3.2, 4.8/
CALL VSSQR(LEN, SRC, DST)
!
! OUTPUT DST:
!     1.210000038, 6.759999275, 10.240000725, 23.040000916
!
```

6.4.2.3.4 v?mul

计算被乘数和乘数的积，被乘数、乘数和积均为向量。

接口定义

C interface：

void vsmul(const int len, const float* src1, const float* src2, float* dst);

void vdmul(const int len, const double* src1, const double* src2, double* dst);

void vcmul(const int len, const float complex *src1, const float complex *src2, float complex *dst);

void vzmul(const int len, const double complex* src1, const double complex* src2, double complex* dst);

void vhmul(const int len, const __fp16 *src1, const __fp16 *src2, __fp16 *dst);

void vchmul(const int len, const complex_fp16 *src1, const complex_fp16 *src2, complex_fp16 *dst);

Fortran interface：

CALL VSMUL(LEN, SRC1, SRC2, DST);

CALL VDMUL(LEN, SRC1, SRC2, DST);

CALL VZMUL(LEN, SRC1, SRC2, DST);

参数

参数名	类型	描述	输入/输出
len	整型数	表示输入向量的元素个数。 len≤0时会提示参数len无效并返回。	输入

 版权所有 © 华为技术有限公司

参数名	类型	描述	输入/输出
src1	• 在vsmul中是单精度浮点类型。 • 在vdmul中是双精度浮点类型。 • 在vcmul中是单精度复数浮点类型。 • 在vzmul中是双精度复数浮点类型。 • 在vhmul中是半精度浮点数。 • 在vchmul中是复数半精度浮点数。	被乘数向量src1，向量长度为len。 若为空指针，会提示空指针错误并返回。	输入
src2	• 在vsmul中是单精度浮点类型。 • 在vdmul中是双精度浮点类型。 • 在vcmul中是单精度复数浮点类型。 • 在vzmul中是双精度复数浮点类型。 • 在vhmul中是半精度浮点数。 • 在vchmul中是复数半精度浮点数。	乘数向量src2，向量长度为len。 若为空指针，会提示空指针错误并返回。	输入
dst	• 在vsmul中是单精度浮点类型。 • 在vdmul中是双精度浮点类型。 • 在vcmul中是单精度复数浮点类型。 • 在vzmul中是双精度复数浮点类型。 • 在vhmul中是半精度浮点数。 • 在vchmul中是复数半精度浮点数。	积向量dst，向量长度为len。 若为空指针，会提示空指针错误并返回。	输出

输出结果

- 每一个运算值返回两个数的积，取值范围为(-INF, +INF)。

 版权所有 © 华为技术有限公司

- 对于vsmul和vdmul，有以下输出情况。

输入值（src1）	输入值（src2）	输出值（dst）
+inf	-inf	-inf
-inf	-inf	+inf
+inf	+inf	+inf
-inf	+inf	-inf
nan	nan	nan

- 对于复数乘法vcmul和vzmul，计算结果和特殊值根据以下公式来定义。

  ```
  Mul(x1+i*y1,x2+i*y2) = (x1*x2-y1*y2) + i*(x1*y2+y1*x2).
  ```

  特殊输出结果请在社区页面文档本章节下载附件查看特殊输出结果。

依赖

C: "kvml.h"

示例

C interface：

```
int i, len = 4;
float src1[len] = {0.0f, 0.5f, 1.5f, 5.5f};
float src2[len] = {0.3f, 0.7f, 2.5f, 10.0f};
float* dst = (float*)malloc(sizeof(float) * len);
if (dst == NULL) {
    printf("Malloc Failed!\n");
    return 0;
}
vsmul(len, src1, src2, dst);
/**
 *  Output dst:
 *     0.0  0.35  3.75  55.0
 *
 */
```

Fortran interface：

```
INTEGER :: LEN = 4
REAL(4) SRC1(4)
REAL(4) SRC2(4)
REAL(4) DST(4)
DATA SRC1 /1.2, 2.5, -2.4, 3.6/
DATA SRC2 /8.6, -2.5, -12.4, 32.6/
CALL VSMUL(LEN, SRC1, SRC2, DST)
!
! OUTPUT DST:
!     10.320000648, -6.250000000, 29.760000229, 117.359992981
!
```

6.4.2.3.5 v?div

计算被除数和除数的商，被除数、除数和商均为向量。

 版权所有 © 华为技术有限公司

接口定义

C interface：

void vsdiv(const int len, const float* src1, const float* src2, float* dst);

void vddiv(const int len, const double* src1, const double* src2, double* dst);

void vcdiv(const int len, const float complex *src1, const float complex *src2, float complex *dst);

void vzdiv(const int len, const double complex* src1, const double complex* src2, double complex* dst);

void vhdiv(const int len, const __fp16 *src1, const __fp16 *src2, __fp16 *dst);

void vchdiv(const int len, const complex_fp16 *src1, const complex_fp16 *src2, complex_fp16 *dst);

Fortran interface：

CALL VSDIV(LEN, SRC1, SRC2, DST);

CALL VDDIV(LEN, SRC1, SRC2, DST);

CALL VZDIV(LEN, SRC1, SRC2, DST);

参数

参数名	类型	描述	输入/输出
len	整型数	表示输入向量的元素个数。 len≤0时会提示参数len无效并返回。	输入
src1	• 在vsdiv中是单精度浮点类型。 • 在vddiv中是双精度浮点类型。 • 在vcdiv中是单精度复数浮点类型。 • 在vzdiv中是双精度复数浮点类型。 • 在vhdiv中是半精度浮点数。 • 在vchdiv中是复数半精度浮点数。	被除数向量src1，向量长度为len。 若为空指针，会提示空指针错误并返回。	输入

 版权所有 © 华为技术有限公司

参数名	类型	描述	输入/输出
src2	• 在vsdiv中是单精度浮点类型。 • 在vddiv中是双精度浮点类型。 • 在vcdiv中是单精度复数浮点类型。 • 在vzdiv中是双精度复数浮点类型。 • 在vhdiv中是半精度浮点数。 • 在vchdiv中是复数半精度浮点数。	除数向量src2，向量长度为len。 若为空指针，会提示空指针错误并返回。	输入
dst	• 在vsdiv中是单精度浮点类型。 • 在vddiv中是双精度浮点类型。 • 在vcdiv中是单精度复数浮点类型。 • 在vzdiv中是双精度复数浮点类型。 • 在vhdiv中是半精度浮点数。 • 在vchdiv中是复数半精度浮点数。	输出向量dst，向量长度为len。 若为空指针，会提示空指针错误并返回。	输出

输出结果

- 每一个运算值返回两个数的商，取值范围为(-INF, +INF)。
- 对于vsdiv和vddiv，有以下输出情况。

输入值（src1）	输入值（src2）	输出值（dst）
+inf	-inf	nan
-inf	-inf	nan
+inf	+inf	nan
-inf	+inf	nan
nan	nan	nan

- 对于复数除法vcdiv和vzdiv，计算结果和特殊值根据以下公式来定义。

```
Div(x1+i*y1,x2+i*y2) = (x1+i*y1)*(x2-i*y2)/(x2*x2+y2*y2).
```

 版权所有 © 华为技术有限公司

特殊输出结果请在社区页面文档本章节下载附件查看特殊输出结果。

依赖

C: "kvml.h"

示例

C interface：

```
int i, len = 4;
float src1[len] = {0.0f, 0.5f, 1.5f, 5.5f};
float src2[len] = {0.3f, 0.1f, 3.0f, 10.0f};
float* dst = (float*)malloc(sizeof(float) * len);
if (dst == NULL) {
    printf("Malloc Failed!\n");
    return 0;
}
vsdiv(len, src1, src2, dst);
/**
 *  Output dst:
 *     0.0  5.0  0.5  0.55
 *
 */
```

Fortran interface：

```
INTEGER :: LEN = 4
REAL(4) SRC1(4)
REAL(4) SRC2(4)
REAL(4) DST(4)
DATA SRC1 /1.2, 2.5, -2.4, 3.6/
DATA SRC2 /8.6, -2.5, -12.4, 32.6/
CALL VSDIV(LEN, SRC1, SRC2, DST)
!
! OUTPUT DST:
!     0.139534876, -1.000000000, 0.193548396, 0.110429451
!
```

6.4.2.3.6 v?sqrt

计算输入的平方根，输入为向量。

接口定义

C interface：

void vssqrt(const int len, const float* src, float* dst);

void vdsqrt(const int len, const double* src, double* dst);

void vcsqrt(const int len, const float complex *src, float complex *dst);

void vzsqrt(const int len, const double complex *src, double complex *dst);

Fortran interface：

CALL VSSQRT(LEN, SRC, DST);

CALL VDSQRT(LEN, SRC, DST);

 版权所有 © 华为技术有限公司

参数

参数名	类型	描述	输入/输出
len	整型数	表示输入向量的元素个数。 len≤0时会提示参数len无效并返回。	输入
src	• 在vssqrt中是单精度浮点类型。 • 在vdsqrt中是双精度浮点类型。 • 在vcsqrt中是单精度复数浮点类型。 • 在vzsqrt中是双精度复数浮点类型。	输入向量src，向量长度为len。 若为空指针，会提示空指针错误并返回。	输入
dst	• 在vssqrt中是单精度浮点类型。 • 在vdsqrt中是双精度浮点类型。 • 在vcsqrt中是单精度复数浮点类型。 • 在vzsqrt中是双精度复数浮点类型	输出dst，向量长度为len。 若为空指针，会提示空指针错误并返回。	输出

输出结果

- 每一个运算值返回数的平方根，取值范围为[0, +INF)。
- 对于vssqrt和vdsqrt，有以下输出情况。

输入值（src）	输出值（dst）
±0	±0
+inf	+inf
< 0	nan
nan	nan

- 对于vcsqrt和vzsqrt，使用欧拉公式和三角函数二倍角公式计算：

$$\sqrt{a+bi}=\sqrt{rcos\theta+i*sin\theta}=\sqrt{re^{i\theta}}=\sqrt{r}*e^{i\frac{\theta}{2}}=\sqrt{r}cos\frac{\theta}{2}+i*\sqrt{r}sin\frac{\theta}{2}$$

，其中 $r=\sqrt{a^2+b^2}$ 。

版权所有 © 华为技术有限公司

特殊取值如下表所示。

RE(z) i * IM(z)	-∞	-X	-0	+0	+X	+∞	NAN
+i * ∞	+∞+i*∞	+∞+i*∞	+∞+i*∞	+∞+i*∞	+∞+i*∞	+∞+i*∞	+∞+i*∞
+i * Y	+0+i*∞	-	-	-	-	+∞+i*0	QNAN +i*QNAN
+i * 0	+0+i*∞	-	+0+i*0	+0+i*0	-	+∞+i*0	QNAN +i*QNAN
-i * 0	+0-i*∞	-	+0+i*0	+0+i*0	-	+∞-i*0	QNAN +i*QNAN
-i * Y	+0-i*∞	-	-	-	-	+∞-i*0	QNAN +i*QNAN
-i * ∞	+∞-i*∞	+∞-i*∞	+∞-i*∞	+∞-i*∞	+∞-i*∞	+∞-i*∞	+∞-i*∞
+i * NAN	QNAN +i*∞	QNAN +i*QNAN	QNAN +i*QNAN	QNAN +i*QNAN	QNAN +i*QNAN	+∞ +i*QNAN	QNAN +i*QNAN

依赖

C: "kvml.h"

示例

C interface：

```
int i, len = 4;
float src[len] = {0.0f, 0.25f, 16.0f, 1.0f};
float* dst = (float*)malloc(sizeof(float) * len);
if (dst == NULL) {
    printf("Malloc Failed!\n");
    return 0;
}
vssqrt(len, src, dst);
/**
 *  Output dst:
 *     0.0  0.5  4.0  1.0
 *
 */
```

Fortran interface：

```
INTEGER :: LEN = 4
REAL(4) SRC(4)
REAL(4) DST(4)
DATA SRC /1, 2, 3, 4/
```

 版权所有 © 华为技术有限公司

```
CALL VSSQRT(LEN, SRC, DST)
!
! OUTPUT DST:
!     1.000000000, 1.414213538, 1.732050776, 2.000000000
!
```

6.4.2.3.7 v?cbrt

计算输入向量的立方根。

接口定义

C interface：

void vscbrt(const int len, const float *src, float *dst);

void vdcbrt(const int len, const double *src, double *dst);

参数

参数名	类型	描述	输入/输出
len	整型数	表示输入向量的元素个数。 len≤0时会提示参数len无效并返回。	输入
src	• 在vscbrt中是单精度浮点类型。 • 在vdcbrt中是双精度浮点类型。	输入向量src，向量长度为len。 若为空指针，会提示空指针错误并返回。	输入
dst	• 在vscbrt中是单精度浮点类型。 • 在vdcbrt中是双精度浮点类型。	输出dst，向量长度为len。 若为空指针，会提示空指针错误并返回。	输出

输出结果

输出每一个输入值的立方根。

输入值（src）	输出值（dst）
+0	+0
-0	-0
+inf	+inf
-inf	-inf
nan	nan

版权所有 © 华为技术有限公司

依赖

C: "kvml.h"

示例

C interface：

```
#define N 4
const float src[N] = {-1.0f, 27.0f, -INFINITY, NAN};
float dst[N] = {0};

vscbrt(N, src, dst);


/**
 *  input :  -1.000   27.000      -inf      nan
 *  output:  -1.000    3.000      -inf      nan
 *
 */
```

6.4.2.3.8 v?abs

计算输入向量的绝对值或模。

接口定义

C interface：

void vsabs(const int len, const float *src, float *dst);

void vdabs(const int len, const double *src, double *dst);

void vcabs(const int len, const float complex *src, float *dst);

void vzabs(const int len, const double complex *src, double *dst);

参数

参数名	类型	描述	输入/输出
len	整型数	表示输入向量的元素个数。 len≤0时会提示参数len无效并返回。	输入
src	• 在vsabs中是单精度浮点类型。 • 在vdabs中是双精度浮点类型。 • 在vcabs中是复数单精度浮点类型。 • 在vzabs中是复数双精度浮点类型。	输入向量src，向量长度为len。 若为空指针，会提示空指针错误并返回。	输入

 版权所有 © 华为技术有限公司

参数名	类型	描述	输入/输出
dst	• 在vsabs中是单精度浮点类型。 • 在vdabs中是双精度浮点类型。 • 在vcabs中是单精度浮点类型。 • 在vzabs中是双精度浮点类型。	输出dst，向量长度为len。 若为空指针，会提示空指针错误并返回。	输出

输出结果

- 对于vsabs、vdabs，返回实数的绝对值。对于vcabs、vzabs，返回复数的模。即对于 $x = a + b * i$ ，$z = a^2 + b^2$ ，求 $\sqrt{z}$ 。
- 对于vsabs和vdabs，有以下输出情况。

输入值（src）	输出值（dst）
+inf	+inf
-inf	+inf
nan	nan

- 对于vcabs和vzabs，有以下输出情况。

输入值实部（src->real）	输入值虚部（src->imag）	输出值（dst）
snan	any	nan
any	snan	nan
±inf	!=snan	inf
!=snan	±inf	inf
qnan	|srcImag| < inf	nan
|srcReal| < inf	qnan	nan
$\sqrt{srcReal^2+srcImag^2} > inf$		inf

依赖

C: "kvml.h"

 版权所有 © 华为技术有限公司

示例

C interface：

```
#define N 4
const float src[N] = {-1.4f, 27.8f, -INFINITY, NAN};
float dst[N] = {0};

vsabs(N, src, dst);
printFArr("input :", N, src);
printFArr("output:", N, dst);

/**
 *  input :  -1.400   27.800     -inf      nan
 *  output:   1.400   27.800      inf      nan
 *
 */
```

6.4.2.3.9 v?inv

计算输入向量的倒数。

接口定义

C interface：

void vsinv(const int len, const float *src, float *dst);

void vdinv(const int len, const double *src, double *dst);

参数

参数名	类型	描述	输入/输出
len	整型数	表示输入向量的元素个数。 len≤0时会提示参数len无效并返回。	输入
src	• 在vsinv中是单精度浮点类型。 • 在vdinv中是双精度浮点类型。	输入向量src，向量长度为len。 若为空指针，会提示空指针错误并返回。	输入
dst	• 在vsinv中是单精度浮点类型。 • 在vdinv中是双精度浮点类型。	输出dst，向量长度为len。 若为空指针，会提示空指针错误并返回。	输出

输出结果

每一个运算值的倒数。

输入值（src）	输出值（dst）
+0	+inf

版权所有 © 华为技术有限公司

输入值（src）	输出值（dst）
-0	-inf
+inf	+0
-inf	-0
nan	nan

依赖

C: "kvml.h"

示例

C interface：

```
#define N 4
const float src[N] = {-2.0f, 3.0f, -INFINITY, +0.0f};
float dst[N] = {0};

vsinv(N, src, dst);
printFArr("input :", N, src);
printFArr("output:", N, dst);

/**
 *  input :  -2.000     3.000      -inf     0.000
 *  output:  -0.500     0.333    -0.000       inf
 *
 */
```

6.4.2.3.10 v?invcbrt

计算输入向量立方根的倒数。

接口定义

C interface：

void vsinvcbrt(const int len, const float *src, float *dst);

void vdinvcbrt(const int len, const double *src, double *dst);

参数

参数名	类型	描述	输入/输出
len	整型数	表示输入向量的元素个数。 len≤0时会提示参数len无效并返回。	输入

版权所有 © 华为技术有限公司

参数名	类型	描述	输入/输出
src	• 在vsinvcbrt中是单精度浮点类型。 • 在vdinvcbrt中是双精度浮点类型。	输入向量src，向量长度为len。 若为空指针，会提示空指针错误并返回。	输入
dst	• 在vsinvcbrt中是单精度浮点类型。 • 在vdinvcbrt中是双精度浮点类型。	输出dst，向量长度为len。 若为空指针，会提示空指针错误并返回。	输出

输出结果

每一个输入值立方根的倒数。

输入值（src）	输出值（dst）
+0	+inf
-0	-inf
+inf	+0
-inf	-0
nan	nan

依赖

C: "kvml.h"

示例

C interface：

```
#define N 4
const float src[N] = {-8.0f, 125.0f, -INFINITY, +0.0f};
float dst[N] = {0};

vsinvcbrt(N, src, dst);
printFArr("input :", N, src);
printFArr("output:", N, dst);

/**
 *  input :  -8.000  125.000     -inf    0.000
 *  output:  -0.500    0.200   -0.000      inf
 *
 */
```

6.4.2.3.11 v?invsqrt

计算输入向量平方根的倒数。

 版权所有 © 华为技术有限公司

接口定义

C interface：

void vsinvsqrt(const int len, const float *src, float *dst);

void vdinvsqrt(const int len, const double *src, double *dst);

参数

参数名	类型	描述	输入/输出
len	整型数	表示输入向量的元素个数。 len≤0时会提示参数len无效并返回。	输入
src	• 在vsinvsqrt中是单精度浮点类型。 • 在vdinvsqrt中是双精度浮点类型。	输入向量src，向量长度为len。 若为空指针，会提示空指针错误并返回。	输入
dst	• 在vsinvsqrt中是单精度浮点类型。 • 在vdinvsqrt中是双精度浮点类型。	输出dst，向量长度为len。 若为空指针，会提示空指针错误并返回。	输出

输出结果

输出每一个输入值平方根的倒数。

输入值（src）	输出值（dst）
+0	+inf
-0	-inf
+inf	+0
<0	nan
nan	nan

依赖

C: "kvml.h"

示例

C interface：

```
#define N 4
const float src[N] = {-8.0f, 25.0f, +INFINITY, -0.0f};
```

 版权所有 © 华为技术有限公司

```
float dst[N] = {0};

vsinvsqrt(N, src, dst);
printFArr("input :", N, src);
printFArr("output:", N, dst);

/**
 *  input :  -8.000   25.000      inf   -0.000
 *  output:     nan    0.200    0.000     -inf
 *
 */
```

6.4.2.3.12 v?linearfrac

使用标量scale，shift对向量src1，src2执行线性分式变换，即

$$dst[i]=(scalea*src1[i]+shifta)\ /\ (scaleb*src2[i]+shiftb)$$

接口定义

C interface：

void vslinearfrac(const int len, const float *src1, const float *src2, const float scalea, const float shifta, const float scaleb, const float shiftb, float *dst);

void vdlinearfrac(const int len, const double *src1, const double *src2, const double scalea, const double shifta, const double scaleb, const double shiftb, double *dst);

参数

参数名	类型	描述	输入/输出
len	整型数	表示输入向量的元素个数。 len≤0时会提示参数len无效并返回。	输入
src1	• 在vslinearfrac中是单精度浮点类型。 • 在vdlinearfrac中是双精度浮点类型。	输入向量src1，向量长度为len。 若为空指针，会提示空指针错误并返回。	输入
src2	• 在vslinearfrac中是单精度浮点类型。 • 在vdlinearfrac中是双精度浮点类型。	输入向量src2，向量长度为len。 若为空指针，会提示空指针错误并返回。	输入
scalea	• 在vslinearfrac中是单精度浮点类型。 • 在vdlinearfrac中是双精度浮点类型。	src1的缩放常量。	输入

版权所有 © 华为技术有限公司

参数名	类型	描述	输入/输出
shifta	• 在vslinearfrac中是单精度浮点类型。 • 在vdlinearfrac中是双精度浮点类型。	src1的偏移常量。	输入
scaleb	• 在vslinearfrac中是单精度浮点类型。 • 在vdlinearfrac中是双精度浮点类型。	src2的缩放常量。	输入
shiftb	• 在vslinearfrac中是单精度浮点类型。 • 在vdlinearfrac中是双精度浮点类型。	src2的偏移常量。	输入
dst	• 在vslinearfrac中是单精度浮点类型。 • 在vdlinearfrac中是双精度浮点类型。	输出dst，向量长度为len。 若为空指针，会提示空指针错误并返回。	输出

输出结果

输出线性分式变换

$$dst[i]=(scalea*src1[i]+shifta)\ /\ (scaleb*src2[i]+shiftb)$$

的结果。

依赖

C: "kvml.h"

示例

C interface：

```
#define N 4
const float src1[N] = {-8.0f, 25.0f, +INFINITY, -0.0f};
const float src2[N] = {6.2f, -4.3f, -192.4, -3.1f};
const float scalea = 0.2f;
const float shifta = 1.0f;
const float scaleb = -0.2f;
const float shiftb = -1.1f;
float dst[N] = {0};

vslinearfrac(N, src1, src2, scalea, shifta, scaleb, shiftb, dst);
printFArr("input1:", N, src1);
printFArr("input2:", N, src2);
printf("scalea : %.3f, shifta : %.3f, scaleb : %.3f, shiftb : %.3f\n", scalea, shifta, scaleb, shiftb);
printFArr("output:", N, dst);

/**
 *  input1:  -8.000   25.000      inf   -0.000
```

 版权所有 © 华为技术有限公司

```
 *  input2:   6.200   -4.300 -192.400   -3.100
 *  scalea : 0.200, shifta : 1.000, scaleb : -0.200, shiftb : -1.100
 *  output:   0.256  -25.000       inf   -2.083
 */
```

6.4.2.3.13 v?remainder

计算两个输入向量相除的余数。

接口定义

C interface：

void vsremainder(const int len, const float* src1, const float* src2, float* dst);

void vdremainder(const int len, const double* src1, const double* src2, double* dst);

参数

参数名	类型	描述	输入/输出
len	整型数	表示输入向量的元素个数。 len≤0时会提示参数len无效并返回。	输入
src1	• 在vsremainder中是单精度浮点类型。 • 在vdremainder中是双精度浮点类型。	输入向量src1，向量长度为len。 若为空指针，会提示空指针错误并返回。	输入
src2	• 在vsremainder中是单精度浮点类型。 • 在vdremainder中是双精度浮点类型。	输入向量src2，向量长度为len。 若为空指针，会提示空指针错误并返回。	输入
dst	• 在vsremainder中是单精度浮点类型。 • 在vdremainder中是双精度浮点类型。	输出dst，向量长度为len。 若为空指针，会提示空指针错误并返回。	输出

输出结果

每一组输入值相除的余数。

输入值（src1）	输入值（src2）	输出值（dst）
any	±nan	nan
any	±0	nan
±inf	any	nan

版权所有 © 华为技术有限公司

输入值（src1）	输入值（src2）	输出值（dst）
±nan	any	nan

依赖

C: "kvml.h"

示例

C interface：

```
#define N 4
const float src1[N] = {-8.0f, 25.0f, +INFINITY, -0.0f};
const float src2[N] = {6.2f, -4.3f, -192.4, -3.1f};
float dst[N] = {0};

vsremainder(N, src1, src2, dst);
printFArr("input1:", N, src1);
printFArr("input2:", N, src2);
printFArr("output:", N, dst);

/**
 *  input1: -8.000   25.000      inf   -0.000
 *  input2:  6.200   -4.300 -192.400   -3.100
 *  output: -1.800   -0.800      nan   -0.000
 */
```

6.4.2.3.14 v?pow

计算变量的幂函数，变量和幂次以及结果均为向量。

接口定义

C interface：

void vspow(const int len, const float* src1, const float* src2, float* dst);

void vdpow(const int len, const double* src1, const double* src2, double* dst);

void vcpow(const int len, const float complex *src1, const float complex *src2, float complex *dst);

void vzpow(const int len, const double complex *src1, const double complex *src2, double complex *dst);

Fortran interface：

CALL VSPOW(LEN, SRC1, SRC2, DST);

CALL VDPOW(LEN, SRC1, SRC2, DST);

 版权所有 © 华为技术有限公司

参数

参数名	类型	描述	输入/输出
len	整型数	表示输入向量的元素个数。 len≤0时会提示参数len无效并返回。	输入
src1	• 在vspow中是单精度浮点类型。 • 在vdpow中是双精度浮点类型。 • 在vcpow中是单精度复数浮点类型。 • 在vzpow中是双精度复数浮点类型。	输入向量src1，向量长度为len。 若为空指针，会提示空指针错误并返回。	输入
src2	• 在vspow中是单精度浮点类型。 • 在vdpow中是双精度浮点类型。 • 在vcpow中是单精度复数浮点类型。 • 在vzpow中是双精度复数浮点类型。	输入向量src2，向量长度为len。 若为空指针，会提示空指针错误并返回。	输入
dst	• 在vspow中是单精度浮点类型。 • 在vdpow中是双精度浮点类型。 • 在vcpow中是单精度复数浮点类型。 • 在vzpow中是双精度复数浮点类型。	输出向量dst，向量长度为len。 若为空指针，会提示空指针错误并返回。	输出

输出结果

- 对于vspow和vdpow，有以下输出情况。
  - src1的值大于0且src2的值为小数，返回幂指数的结果，取值范围dst ∈ (-INF, +INF)。
  - src1的值为-0，src2的值为负的奇数，dst的值则返回-∞；其他src1的值为±0，src2的值为负数，dst的值返回+∞。
  - src1的值为±0，src2的值为正的奇数，dst的值则返回±0；其他src1的值为±0，src2的值为正数，dst的值返回+0。
  - src1的值为±1，src2的值为任意值，dst的值则返回+1。
  - src1的值为任意值，src2的值为±0，dst的值则返回+1。

 版权所有 © 华为技术有限公司

  - src1的值小于+0，src2的值为小数，dst的值则返回nan。
  - |src1的值|小于1，src2的值为-∞，dst的值则返回+∞；|src1的值|大于1，src2的值为-∞，dst的值则返回+0。
  - |src1的值|小于1，src2的值为+∞，dst的值则返回+0；|src1的值|大于1，src2的值为+∞，dst的值则返回+∞。
  - src1的值为-∞，src2的值为负奇数，dst的值则返回-0；其他src1的值为-∞，src2的值为负数，dst的值则返回+0。
  - src1的值为-∞，src2的值为正奇数，dst的值则返回-∞；其他src1的值为-∞，src2的值为正数，dst的值则返回+∞。
  - src1的值为+∞，src2的值小于+0，dst的值则返回+0；其他src1的值为+∞，dst的值则返回+∞。
  - src1的值为大的有限数，且src2的值为大的有限数，dst的值则返回±∞。
  - src1的值为nan，且src2的值为nan，dst的值则返回nan。
- 对于vcpow和vzpow，请在社区页面文档本章节下载附件查看特殊输出结果。

依赖

C: "kvml.h"

示例

C interface：

```
#define N 4
const float src1[N] = {-2.2f, 1.0f, +INFINITY, -0.0f};
const float src2[N] = {+2.0f, 111.0f, +1.1f, -6.9f};
float dst[N] = {0};

vspow(N, src1, src2, dst);

/**
 *  input1:  -2.200    1.000      inf   -0.000
 *  input2:   2.000  111.000    1.100   -6.900
 *  output:   4.840    1.000      inf      inf
 */
```

Fortran interface：

```
INTEGER :: LEN = 4
REAL(4) SRC1(4)
REAL(4) SRC2(4)
REAL(4) DST(4)
DATA SRC1 /1.2, 2.5, 2.4, 3.6/
DATA SRC2 /8.6, -2.5, 1.7, 1.3/
CALL VSPOW(LEN, SRC1, SRC2, DST)
!
! OUTPUT DST:
!     4.79688072, 0.101192884, 4.42954540, 5.28680420
!
```

6.4.2.3.15 v?powx

计算变量的幂函数，变量和结果均为向量，幂次为标量。

接口定义

C interface：

 版权所有 © 华为技术有限公司

void vspowx(const int len, const float* src1, const float src2, float* dst);

void vdpowx(const int len, const double* src1, const double src2, double* dst);

void vcpowx(const int len, const float complex *src1, const float complex src2, float complex *dst);

void vzpowx(const int len, const double complex *src1, const double complex src2, double complex *dst);

参数

参数名	类型	描述	输入/输出
len	整型数	表示输入向量的元素个数。 len≤0时会提示参数len无效并返回。	输入
src1	• 在vspowx中是单精度浮点类型。 • 在vdpowx中是双精度浮点类型。 • 在vcpowx中是单精度复数浮点类型。 • 在vzpowx中是双精度复数浮点类型。	输入向量src1，向量长度为len。 若为空指针，会提示空指针错误并返回。	输入
src2	• 在vspowx中是单精度浮点类型。 • 在vdpowx中是双精度浮点类型。 • 在vcpowx中是单精度复数浮点类型。 • 在vzpowx中是双精度复数浮点类型。	表示幂次值，为标量类型。	输入
dst	• 在vspowx中是单精度浮点类型。 • 在vdpowx中是双精度浮点类型。 • 在vcpowx中是单精度复数浮点类型。 • 在vzpowx中是双精度复数浮点类型。	输出向量dst，向量长度为len。 若为空指针，会提示空指针错误并返回。	输出

输出结果

请参见**6.4.2.3.14 v?pow**的输出结果。

版权所有 © 华为技术有限公司

依赖

C: "kvml.h"

示例

C interface：

```
#define N 4
const float src1[N] = {-2.2f, 1.0f, +INFINITY, -0.0f};
const float src2 = +2.0f;
float dst[N] = {0};

vspowx(N, src1, src2, dst);
printFArr("input1:", N, src1);
printf("src2 = %.3f\n", src2);
printFArr("output:", N, dst);

/**
 *  input1:  -2.200    1.000      inf   -0.000
 *  src2 = 2.000
 *  output:   4.840    1.000      inf    0.000
 */
```

6.4.2.3.16 v?powr

计算变量的幂函数。变量，幂次和结果均为向量，变量≥0。

接口定义

C interface：

void vspowr(const int len, const float* src1, const float* src2, float* dst);

void vdpowr(const int len, const double* src1, const double* src2, double* dst);

参数

参数名	类型	描述	输入/输出
len	整型数	表示输入向量的元素个数。 len≤0时会提示参数len无效并返回。	输入
src1	• 在vspowr中是单精度浮点类型。 • 在vdpowr中是双精度浮点类型。	输入向量src1，向量长度为len。 若为空指针，会提示空指针错误并返回。	输入
src2	• 在vspowr中是单精度浮点类型。 • 在vdpowr中是双精度浮点类型。	输入向量src2，向量长度为len。 若为空指针，会提示空指针错误并返回。	输入

版权所有 © 华为技术有限公司

参数名	类型	描述	输入/输出
dst	• 在vspowr中是单精度浮点类型。 • 在vdpowr中是双精度浮点类型。	输出向量dst，向量长度为len。 若为空指针，会提示空指针错误并返回。	输出

输出结果

返回a^b的计算结果。其他特殊值参考以下说明。

输入值（src1）	输入值（src2）	输出值（dst）
x < 0	any value	nan
0 < x < inf	±0	1
±0	-inf < y < 0	+inf
±0	-inf	+inf
±0	y > 0	+0
1	-inf < y < +inf	1
±0	±0	1
+inf	±0	1
1	+inf	1
x >= 0	nan	nan
nan	any value	nan
0 < x < 1	-inf	+inf
x > 1	-inf	+0
0 <= x < 1	+inf	+0
x > 1	+inf	+inf
+inf	x < 0	+0
+inf	x > 0	+inf
any value	nan	nan

其他特殊值请参见**6.4.2.3.14 v?pow**。

版权所有 © 华为技术有限公司

依赖

C: "kvml.h"

示例

C interface：

```
int i, len = 4;
float src1[len] = {-1.0f, 0.0f, 16.0f, 2.0f};
float src2[len] = {0.0f, 0.0f, 0.5f, 3.0f};
float* dst = (float*)malloc(sizeof(float) * len);
if (dst == NULL) {
    printf("Malloc Failed!\n");
    return 0;
}
vspowr(len, src1, src2, dst);
/**
 *  Output dst:
 *     nan  1.0  4.0  8.0
 *
 */
```

6.4.2.3.17 v?pow2o3

计算输入的2/3次方的值，输入为向量。

接口定义

C interface：

void vspow2o3(const int len, const float* src, float* dst);

void vdpow2o3(const int len, const double* src, double* dst);

参数

参数名	类型	描述	输入/输出
len	整型数	表示输入向量的元素个数。 len≤0时会提示参数len无效并返回。	输入
src	• 在vspow2o3中是单精度浮点类型。 • 在vdpow2o3中是双精度浮点类型。	输入向量src，向量长度为len。 若为空指针，会提示空指针错误并返回。	输入
dst	• 在vspow2o3中是单精度浮点类型。 • 在vdpow2o3中是双精度浮点类型。	输出dst，向量长度为len。 若为空指针，会提示空指针错误并返回。	输出

 版权所有 © 华为技术有限公司

输出结果

输入值（src）	输出值（dst）
+0	+0
-0	+0
+inf	+inf
-inf	+inf
nan	nan

依赖

C: "kvml.h"

示例

C interface：

```
int i, len = 4;
float src[len] = {0.0f, 2.0f, INFINITY, NAN};
float* dst = (float*)malloc(sizeof(float) * len);
if (dst == NULL) {
    printf("Malloc Failed!\n");
    return 0;
}
vspow2o3(len, src, dst);
/**
 *  Output dst:
 *     0.0 1.587401 inf  nan
 *
 */
```

6.4.2.3.18 v?pow3o2

计算输入的3/2次方的值，输入为向量。

接口定义

C interface：

void vspow3o2(const int len, const float* src, float* dst);

void vdpow3o2(const int len, const double* src, double* dst);

参数

参数名	类型	描述	输入/输出
len	整型数	表示输入向量的元素个数。 len≤0时会提示参数len无效并返回。	输入

版权所有 © 华为技术有限公司

参数名	类型	描述	输入/输出
src	• 在vspow3o2中是单精度浮点类型。 • 在vdpow3o2中是双精度浮点类型。	输入向量src，向量长度为len。 若为空指针，会提示空指针错误并返回。	输入
dst	• 在vspow3o2中是单精度浮点类型。 • 在vdpow3o2中是双精度浮点类型。	输出dst，向量长度为len。 若为空指针，会提示空指针错误并返回。	输出

输出结果

输入值（src）	输出值（dst）
<+0	nan
+0	+0
-0	-0
-inf	nan
+inf	+inf
nan	nan

依赖

C: "kvml.h"

示例

C interface：

```
int i, len = 4;
float src[len] = {0.0f, 2.0f, INFINITY, NAN};
float* dst = (float*)malloc(sizeof(float) * len);
if (dst == NULL) {
    printf("Malloc Failed!\n");
    return 0;
}
vspow3o2(len, src, dst);
/**
 *  Output dst:
 *     0.0  2.828427  nan  nan
 *
 */
```

6.4.2.3.19 v?hypot

计算src1，src2的平方之和再开平方的值，src1和src2以及结果均为向量。

版权所有 © 华为技术有限公司

接口定义

C interface：

void vshypot(const int len, const float* src1, const float* src2, float* dst);

void vdhypot(const int len, const double* src1, const double* src2, double* dst);

参数

参数名	类型	描述	输入/输出
len	整型数	表示输入向量的元素个数。 len≤0时会提示参数len无效并返回。	输入
src1	• 在vshypot中是单精度浮点类型。 • 在vdhypot中是双精度浮点类型。	输入向量src1，向量长度为len。 若为空指针，会提示空指针错误并返回。	输入
src2	• 在vshypot中是单精度浮点类型。 • 在vdhypot中是双精度浮点类型。	输入向量src2，向量长度为len。 若为空指针，会提示空指针错误并返回。	输入
dst	• 在vshypot中是单精度浮点类型。 • 在vdhypot中是双精度浮点类型。	输出向量dst，向量长度为len。 若为空指针，会提示空指针错误并返回。	输出

输出结果

计算$\sqrt{x^2+y^2}$的值，取值范围为|(-inf, inf)|。

输入值（src1）	输入值（src2）	输出值（dst）
snan	any	nan
any	snan	nan
±inf	!=snan	inf
!=snan	±inf	inf
qnan	(-inf, inf)	nan
(-inf, inf)	qnan	nan
$\sqrt{x^2+y^2} > inf$		inf

 版权所有 © 华为技术有限公司

依赖

C: "kvml.h"

示例

C interface：

```
#define N 4
const float src1[N] = {3.0f, 5.0f, 8.44f, NAN};
const float src2[N] = {-4.0f, 12.0f, -INFINITY, 1.0f};
float dst[N] = {0};

vshypot(N, src1, src2, dst);
printFArr("input1:", N, src1);
printFArr("input2:", N, src2);
printFArr("output:", N, dst);

/**
 *  input1:   3.000    5.000    8.440      nan
 *  input2:  -4.000   12.000     -inf    1.000
 *  output:   5.000   13.000      inf      nan
 */
```

6.4.2.3.20 v?exp

计算输入以自然对数为底的指数，输入为向量。

接口定义

C interface：

void vsexp(const int len, const float* src, float* dst);

void vdexp(const int len, const double* src, double* dst);

void vcexp(const int len, const float complex *src, float complex *dst);

void vzexp(const int len, const double complex *src, double complex *dst);

Fortran interface：

CALL VSEXP(LEN, SRC, DST);

CALL VDEXP(LEN, SRC, DST);

参数

参数名	类型	描述	输入/输出
len	整型数	表示输入向量的元素个数。 len≤0时会提示参数len无效并返回。	输入

 版权所有 © 华为技术有限公司

参数名	类型	描述	输入/输出
src	• 在vsexp中是单精度浮点类型。 • 在vdexp中是双精度浮点类型。 • 在vcexp中是复数单精度浮点类型。 • 在vzexp中是复数双精度浮点类型。	输入向量src，向量长度为len。 若为空指针，会提示空指针错误并返回。	输入
dst	• 在vsexp中是单精度浮点类型。 • 在vdexp中是双精度浮点类型。 • 在vcexp中是复数单精度浮点类型。 • 在vzexp中是复数双精度浮点类型。	输出dst，向量长度为len。 若为空指针，会提示空指针错误并返回。	输出

输出结果

输出 e^x 的值。其他特殊值参考如下说明。

输入值（src）	输出值（dst）
+0	+1
-0	+1
-inf	+0
+inf	+inf
nan	nan

依赖

C: "kvml.h"

示例

C interface：

```
int i, len = 4;
float src[len] = {0.0f, INFINITY, NAN, 2.0f};
float* dst = (float*)malloc(sizeof(float) * len);
```

 版权所有 © 华为技术有限公司

```
if (dst == NULL) {
    printf("Malloc Failed!\n");
    return 0;
}
vsexp(len, src, dst);
/**
 *  Output dst:
 *     0.0  inf  nan  7.389056205749512
 *
 */
```

Fortran interface：

```
INTEGER :: LEN = 2
REAL(4) SRC(2)
REAL(4) DST(2)
DATA SRC /1.0, 2.0/
CALL VSEXP(LEN, SRC, DST)
!
! OUTPUT DST:
!     0.0 7.389056205749512
!
```

6.4.2.3.21 v?exp2

计算输入以2为底的指数，输入为向量。

接口定义

C interface：

void vsexp2(const int len, const float* src, float* dst);

void vdexp2(const int len, const double* src, double* dst);

参数

参数名	类型	描述	输入/输出
len	整型数	表示输入向量的元素个数。 len≤0时会提示参数len无效并返回。	输入
src	• 在vsexp2中是单精度浮点类型。 • 在vdexp2中是双精度浮点类型。	输入向量src，向量长度为len。 若为空指针，会提示空指针错误并返回。	输入
dst	• 在vsexp2中是单精度浮点类型。 • 在vdexp2中是双精度浮点类型。	输出dst，向量长度为len。 若为空指针，会提示空指针错误并返回。	输出

输出结果

输出 2^x 的值。其他特殊值参考如下说明。

 版权所有 © 华为技术有限公司

输入值（src）	输出值（dst）
+0	+1
-0	+1
inf	inf
-inf	+0
nan	nan

依赖

C: "kvml.h"

示例

C interface：

```
#define N 4
const float src[N] = {-2.2f, 1.0f, +INFINITY, -0.0f};
float dst[N] = {0};

vsexp2(N, src, dst);
printFArr("input :", N, src);
printFArr("output:", N, dst);

/**
 *  input :  -2.200    1.000      inf   -0.000
 *  output:   0.218    2.000       inf    1.000
 *
 */
```

6.4.2.3.22 v?exp10

计算输入以10为底的指数，输入为向量。

接口定义

C interface：

void vsexp10(const int len, const float* src, float* dst);

void vdexp10(const int len, const double* src, double* dst);

参数

参数名	类型	描述	输入/输出
len	整型数	表示输入向量的元素个数。 len≤0时会提示参数len无效并返回。	输入

版权所有 © 华为技术有限公司

参数名	类型	描述	输入/输出
src	• 在vsexp10中是单精度浮点类型。 • 在vdexp10中是双精度浮点类型。	输入向量src，向量长度为len。 若为空指针，会提示空指针错误并返回。	输入
dst	• 在vsexp10中是单精度浮点类型。 • 在vdexp10中是双精度浮点类型。	输出dst，向量长度为len。 若为空指针，会提示空指针错误并返回。	输出

输出结果

输出 10^x 的值。其他特殊值参考如下说明。

输入值（src）	输出值（dst）
+0	+1
-0	+1
>overflow	+inf
<underflow	+0
inf	inf
-inf	+0
nan	nan

依赖

C: "kvml.h"

示例

C interface：

```
#define N 4
const float src[N] = {-2.2f, 1.0f, +INFINITY, -0.0f};
float dst[N] = {0};

vsexp10(N, src, dst);
printFArr("input :", N, src);
printFArr("output:", N, dst);

/**
 *  input :  -2.200    1.000      inf   -0.000
 *  output:   0.006   10.000      inf    1.000
```

 版权所有 © 华为技术有限公司

```
 *
 */
```

6.4.2.3.23 v?expm1

计算输入以自然对数为底的指数值减1，输入为向量。

接口定义

C interface：

void vsexpm1(const int len, const float* src, float* dst);

void vdexpm1(const int len, const double* src, double* dst);

参数

参数名	类型	描述	输入/输出
len	整型数	表示输入向量的元素个数。 len≤0时会提示参数len无效并返回。	输入
src	• 在vsexpm1中是单精度浮点类型。 • 在vdexpm1中是双精度浮点类型。	输入向量src，向量长度为len。 若为空指针，会提示空指针错误并返回。	输入
dst	• 在vsexpm1中是单精度浮点类型。 • 在vdexpm1中是双精度浮点类型。	输出dst，向量长度为len。 若为空指针，会提示空指针错误并返回。	输出

输出结果

输出 $e^x - 1$ 的值。其他特殊值参考如下说明。

输入值（src）	输出值（dst）
+0	+0
-0	-0
>overflow	+inf
inf	inf
-inf	-1
nan	nan

版权所有 © 华为技术有限公司

依赖

C: "kvml.h"

示例

C interface：

```
int i, len = 4;
float src[len] = {0.0f, INFINITY, NAN, 2.0f};
float* dst = (float*)malloc(sizeof(float) * len);
if (dst == NULL) {
    printf("Malloc Failed!\n");
    return 0;
}
vsexpm1(len, src, dst);
/**
 *  Output dst:
 *     -1.0  inf  nan  6.38905609893065
 *
 */
```

6.4.2.3.24 v?ln

计算输入以自然对数e为底的对数值，输入为向量。

接口定义

C interface：

void vsln(const int len, const float* src, float* dst);

void vdln(const int len, const double* src, double* dst);

void vcln(const int len, const float complex *src, float complex *dst);

void vzln(const int len, const double complex *src, double complex *dst);

Fortran interface：

CALL VSLN(LEN, SRC, DST);

CALL VDLN(LEN, SRC, DST);

参数

参数名	类型	描述	输入/输出
len	整型数	表示输入向量的元素个数。 len≤0时会提示参数len无效并返回。	输入

版权所有 © 华为技术有限公司

参数名	类型	描述	输入/输出
src	• 在vsln中是单精度浮点类型。 • 在vdln中是双精度浮点类型。 • 在vcln中是复数单精度浮点类型。 • 在vzln中是复数双精度浮点类型。	输入向量src，向量长度为len。 若为空指针，会提示空指针错误并返回。	输入
dst	• 在vsln中是单精度浮点类型。 • 在vdln中是双精度浮点类型。 • 在vcln中是复数单精度浮点类型。 • 在vzln中是复数双精度浮点类型。	输出dst，向量长度为len。 若为空指针，会提示空指针错误并返回。	输出

输出结果

- 输出 lnx 的值。
- 对vsln和vdln，有以下输出情况。

输入值（src）	输出值（dst）
±0	-inf
<0	nan
+inf	+inf
nan	nan

- 对vcln和vzln，请在社区页面文档本章节下载附件查看特殊输出结果。

依赖

C: "kvml.h"

示例

C interface：

```
int i, len = 4;
float src[len] = {0.0f, 1.0f, INFINITY, NAN};
float* dst = (float*)malloc(sizeof(float) * len);
```

 版权所有 © 华为技术有限公司

```
if (dst == NULL) {
    printf("Malloc Failed!\n");
    return 0;
}
vsln(len, src, dst);
/**
 *  Output dst:
 *     -inf  0  inf  nan
 *
 */
```

Fortran interface：

```
INTEGER :: LEN = 4
REAL(4) SRC(4)
REAL(4) DST(4)
DATA SRC /1, 2, 3, 4/
CALL VSLN(LEN, SRC, DST)
!
! OUTPUT DST:
!     0.00000000, 0.693147182, 1.09861231, 1.38629436
!
```

6.4.2.3.25 v?log10

计算输入以10为底的对数，输入为向量。

接口定义

C interface：

void vslog10(const int len, const float* src, float* dst);

void vdlog10(const int len, const double* src, double* dst);

void vclog10(const int len, const float complex *src, float complex *dst);

void vzlog10(const int len, const double complex *src, double complex *dst);

Fortran interface：

CALL VSLOG10(LEN, SRC, DST);

CALL VDLOG10(LEN, SRC, DST);

参数

参数名	类型	描述	输入/输出
len	整型数	表示输入向量的元素个数。 len≤0时会提示参数len无效并返回。	输入

版权所有 © 华为技术有限公司

参数名	类型	描述	输入/输出
src	• 在vslog10中是单精度浮点类型。 • 在vdlog10中是双精度浮点类型。 • 在vclog10中是复数单精度浮点类型。 • 在vzlog10中是复数双精度浮点类型。	输入向量src，向量长度为len。 若为空指针，会提示空指针错误并返回。	输入
dst	• 在vslog10中是单精度浮点类型。 • 在vdlog10中是双精度浮点类型。 • 在vclog10中是复数单精度浮点类型。 • 在vzlog10中是复数双精度浮点类型。	输出dst，向量长度为len。 若为空指针，会提示空指针错误并返回。	输出

输出结果

- 输出 $log_{10}x$ 的值。
- 对于vslog10和vdlog10，有以下输出情况。

输入值（src）	输出值（dst）
±0	-inf
<0	nan
+inf	+inf
nan	nan

- 对于vclog10和vzlog10，参考libm clog10的特殊值说明。

依赖

C: "kvml.h"

示例

C interface：

```
int i, len = 4;
float src[len] = {0.0f, 1.0f, INFINITY, NAN};
float* dst = (float*)malloc(sizeof(float) * len);
```

 版权所有 © 华为技术有限公司

```
if (dst == NULL) {
    printf("Malloc Failed!\n");
    return 0;
}
vslog10(len, src, dst);
/**
 *  Output dst:
 *      -inf  0  inf  nan
 *
 */
```

Fortran interface：

```
INTEGER :: LEN = 4
REAL(4) SRC(4)
REAL(4) DST(4)
DATA SRC /1, 2, 3, 4/
CALL VSLOG10(LEN, SRC, DST)
!
! OUTPUT DST:
!     0.00000000, 0.301030010, 0.477121234, 0.602060020
!
```

6.4.2.3.26 v?log1p

矢量计算以e为底(1+x)为输入的对数值。

接口定义

C interface：

void vslog1p(const int len, const float* src, float* dst);

void vdlog1p(const int len, const double* src, double* dst);

参数

参数名	类型	描述	输入/输出
len	整型数	表示输入向量的元素个数。 len≤0时会提示参数len无效并返回。	输入
src	• 在vslog1p中是单精度浮点类型。 • 在vdlog1p中是双精度浮点类型。	输入向量src，向量长度为len。 若为空指针，会提示空指针错误并返回。	输入
dst	• 在vslog1p中是单精度浮点类型。 • 在vdlog1p中是双精度浮点类型。	输出dst，向量长度为len。 若为空指针，会提示空指针错误并返回。	输出

输出结果

输出 $ln(x+1)$ 的值。其他特殊值参考如下说明。

 版权所有 © 华为技术有限公司

输入值（src）	输出值（dst）
-1	-inf
<-1	nan
+0	+0
-0	-0
-inf	nan
+inf	+inf
nan	nan

依赖

C: "kvml.h"

示例

C interface：

```
#define N 4
const float src[N] = {-2.2f, 1.0f, +INFINITY, -0.0f};
float dst[N] = {0};

vslog1p(N, src, dst);
printFArr("input1:", N, src);
printFArr("output:", N, dst);

/**
 *  input1:  -2.200    1.000      inf   -0.000
 *  output:      nan    0.693      inf   -0.000
 *
 */
```

6.4.2.3.27 v?log2

矢量计算以2为底的对数值。

接口定义

C interface：

void vslog2(const int len, const float* src, float* dst);

void vdlog2(const int len, const double* src, double* dst);

版权所有 © 华为技术有限公司

参数

参数名	类型	描述	输入/输出
len	整型数	表示输入向量的元素个数。 len≤0时会提示参数len无效并返回。	输入
src	• 在vslog2中是单精度浮点类型。 • 在vdlog2中是双精度浮点类型。	输入向量src，向量长度为len。 若为空指针，会提示空指针错误并返回。	输入
dst	• 在vslog2中是单精度浮点类型。 • 在vdlog2中是双精度浮点类型。	输出dst，向量长度为len。 若为空指针，会提示空指针错误并返回。	输出

输出结果

输出 $log_2 x$ 的值。其他特殊值参考如下说明。

输入值（src）	输出值（dst）
±0	-inf
<0	nan
+inf	+inf
nan	nan

依赖

C: "kvml.h"

示例

C interface：

```
#define N 4
const float src[N] = {2.2f, 1.0f, +INFINITY, -3.0f};
float dst[N] = {0};

vslog2(N, src, dst);
printFArr("input1:", N, src);
printFArr("output:", N, dst);

/**
 *  input1:   2.200    1.000      inf   -3.000
 *  output:   1.138    0.000      inf      nan
```

 版权所有 © 华为技术有限公司

```
 *
 */
```

6.4.2.3.28 v?logb

矢量计算输入x的指数值，以浮点形式返回。

接口定义

C interface：

void vslogb(const int len, const float* src, float* dst);

void vdlogb(const int len, const double* src, double* dst);

参数

参数名	类型	描述	输入/输出
len	整型数	表示输入向量的元素个数。 len≤0时会提示参数len无效并返回。	输入
src	• 在vslogb中是单精度浮点类型。 • 在vdlogb中是双精度浮点类型。	输入向量src，向量长度为len。 若为空指针，会提示空指针错误并返回。	输入
dst	• 在vslogb中是单精度浮点类型。 • 在vdlogb中是双精度浮点类型。	输出dst，向量长度为len。 若为空指针，会提示空指针错误并返回。	输出

输出结果

- 每一个运算值返回其浮点指数值y，对于vslogb，y ∈ [-149,127]；对于vdlogb，y ∈ [-1074,1023]。
- 其他特殊值参考如下说明。

输入值（src）	输出值（dst）
±inf	+inf
±0	-inf
nan	nan

依赖

C: "kvml.h"

版权所有 © 华为技术有限公司

示例

C interface：

```
#define N 4
const float src[N] = {73.2f, 8.0f, +INFINITY, +0.0f};
float dst[N] = {0};

vslogb(N, src, dst);
printFArr("input1:", N, src);
printFArr("output:", N, dst);

/**
 *  input1:  73.200    8.000      inf    0.000
 *  output:   6.000    3.000      inf     -inf
 *
 */
```

6.4.2.3.29 v?cos

计算输入弧度的余弦值，输入为向量。

接口定义

C interface：

void vscos(const int len, const float* src, float* dst);

void vdcos(const int len, const double* src, double* dst);

void vccos(const int len, const float complex *src, float complex *dst);

void vzcos(const int len, const double complex *src, double complex *dst);

Fortran interface：

CALL VSCOS(LEN, SRC, DST);

CALL VDCOS(LEN, SRC, DST);

参数

参数名	类型	描述	输入/输出
len	整型数	表示输入向量的元素个数。 len≤0时会提示参数len无效并返回。	输入
src	• 在vscos中是单精度浮点类型。 • 在vdcos中是双精度浮点类型。 • 在vccos中是复数单精度浮点类型。 • 在vzcos中是复数双精度浮点类型。	输入向量src，向量长度为len。 若为空指针，会提示空指针错误并返回。	输入

版权所有 © 华为技术有限公司

参数名	类型	描述	输入/输出
dst	• 在vscos中是单精度浮点类型。 • 在vdcos中是双精度浮点类型。 • 在vccos中是复数单精度浮点类型。 • 在vzcos中是复数双精度浮点类型。	输出dst，向量长度为len。 若为空指针，会提示空指针错误并返回。	输出

输出结果

- 每一个运算值返回弧度角x的余弦函数值y=cos(x)，y ∈ [-1, +1]。
- 其他特殊值参考如下说明。

输入值（src）	输出值（dst）
±inf	nan
±0	+1
nan	nan

依赖

C: "kvml.h"

示例

C interface：

```
int i, len = 4;
float src[len] = {0.0f, 2.0f, INFINITY, NAN};
float* dst = (float*)malloc(sizeof(float) * len);
if (dst == NULL) {
    printf("Malloc Failed!\n");
    return 0;
}
vscos(len, src, dst);
/**
 *  Output dst:
 *     1.0  -0.416146814823151  nan  nan
 *
 */
```

Fortran interface：

```
INTEGER :: LEN = 4
REAL(4) SRC(4)
REAL(4) DST(4)
DATA SRC /1, 2, 3, 4/
CALL VSCOS(LEN, SRC, DST)
!
```

 版权所有 © 华为技术有限公司

```
! OUTPUT DST:
!     0.540302277, -0.416146815, -0.989992499, -0.653643608
!
```

6.4.2.3.30 v?cospi

计算输入值乘以π的余弦值，输入为向量。

接口定义

C interface：

void vscospi(const int len, const float* src, float* dst);

void vdcospi(const int len, const double* src, double* dst);

参数

参数名	类型	描述	输入/输出
len	整型数	表示输入向量的元素个数。 len≤0时会提示参数len无效并返回。	输入
src	• 在vscospi中是单精度浮点类型。 • 在vdcospi中是双精度浮点类型。	输入向量src，向量长度为len。 若为空指针，会提示空指针错误并返回。	输入
dst	• 在vscospi中是单精度浮点类型。 • 在vdcospi中是双精度浮点类型。	输出dst，向量长度为len。 若为空指针，会提示空指针错误并返回。	输出

输出结果

- 每一个运算值返回余弦函数值y=cos(x * π)，y ∈ [-1, +1]。
- 其他特殊值参考如下说明。

输入值（src）	输出值（dst）
|src| >= 0x1p24 (float)	+1
|src| >= 0x1p53 (double)	+1
±inf	nan
nan	nan

依赖

C: "kvml.h"

 版权所有 © 华为技术有限公司

示例

C interface：

```
int i, len = 4;
float src[len] = {0.0f, 0.3f, INFINITY, NAN};
float* dst = (float*)malloc(sizeof(float) * len);
if (dst == NULL) {
    printf("Malloc Failed!\n");
    return 0;
}
vscospi(len, src, dst);
/**
 *  Output dst:
 *     1.0 0.587785 nan nan
 *
 */
```

6.4.2.3.31 v?sin

计算输入弧度的正弦值，输入为向量。

接口定义

C interface：

void vssin(const int len, const float* src, float* dst);

void vdsin(const int len, const double* src, double* dst);

void vcsin(const int len, const float complex *src, float complex *dst);

void vzsin(const int len, const double complex *src, double complex *dst);

Fortran interface：

CALL VSSIN(LEN, SRC, DST);

CALL VDSIN(LEN, SRC, DST);

参数

参数名	类型	描述	输入/输出
len	整型数	表示输入向量的元素个数。 len≤0时会提示参数len无效并返回。	输入
src	• 在vssin中是单精度浮点类型。 • 在vdsin中是双精度浮点类型。 • 在vcsin中是复数单精度浮点类型。 • 在vzsin中是复数双精度浮点类型。	输入向量src，向量长度为len。 若为空指针，会提示空指针错误并返回。	输入

版权所有 © 华为技术有限公司

参数名	类型	描述	输入/输出
dst	• 在vssin中是单精度浮点类型。 • 在vdsin中是双精度浮点类型。 • 在vcsin中是复数单精度浮点类型。 • 在vzsin中是复数双精度浮点类型。	输出dst，向量长度为len。 若为空指针，会提示空指针错误并返回。	输出

输出结果

- 每一个运算值返回弧度角x的正弦函数值y=sin(x)，y ∈ [-1, +1]。
- 其他特殊值参考如下说明。

输入值（src）	输出值（dst）
+0	+0
-0	-0
nan	nan
±inf	nan

依赖

C: "kvml.h"

示例

C interface：

```
int i, len = 4;
float src[len] = {0.0f, 2.0f, INFINITY, NAN};
float* dst = (float*)malloc(sizeof(float) * len);
if (dst == NULL) {
    printf("Malloc Failed!\n");
    return 0;
}
vssin(len, src, dst);
/**
 *  Output dst:
 *     0.0  0.909297406673431  nan  nan
 *
 */
```

Fortran interface：

```
INTEGER :: LEN = 4
REAL(4) :: SRC(4)
REAL(4) :: DST(4)
DATA SRC/1, 2, 3, 4/
```

 版权所有 © 华为技术有限公司

```
CALL VSSIN(LEN, SRC, DST)
!
! OUTPUT DST:
!     0.841470957, 0.909297407, 0.141120002, -0.756802440
!
```

6.4.2.3.32 v?sinpi

计算输入值乘以π的正弦值，输入为向量。

接口定义

C interface：

void vssinpi(const int len, const float* src, float* dst);

void vdsinpi(const int len, const double* src, double* dst);

参数

参数名	类型	描述	输入/输出
len	整型数	表示输入向量的元素个数。 len≤0时会提示参数len无效并返回。	输入
src	• 在vssinpi中是单精度浮点类型。 • 在vdsinpi中是双精度浮点类型。	输入向量src，向量长度为len。 若为空指针，会提示空指针错误并返回。	输入
dst	• 在vssinpi中是单精度浮点类型。 • 在vdsinpi中是双精度浮点类型。	输出dst，向量长度为len。 若为空指针，会提示空指针错误并返回。	输出

输出结果

- 每一个运算值返回正弦函数值y=sin(x * π)，y ∈ [-1, +1]。
- 其他特殊值参考如下说明。

输入值（src）	输出值（dst）
src >= 0x1p24 (float)	+0
src <= -0x1p24 (float)	-0
src >= 0x1p53 (double)	+0
src <= -0x1p53 (double)	-0
±inf	nan

 版权所有 © 华为技术有限公司

输入值（src）	输出值（dst）
nan	nan

依赖

C: "kvml.h"

示例

C interface：

```
int i, len = 4;
float src[len] = {0.0f, 0.3f, INFINITY, NAN};
float* dst = (float*)malloc(sizeof(float) * len);
if (dst == NULL) {
    printf("Malloc Failed!\n");
    return 0;
}
vssinpi(len, src, dst);
/**
 *  Output dst:
 *     0.0 0.809017 nan nan
 *
 */
```

6.4.2.3.33 v?sincos

计算输入弧度的正弦值和余弦值，输入为向量。

接口定义

C interface：

void vssincos(const int len, const float* src, float* sindst, float* cosdst);

void vdsincos(const int len, const double* src, double* sindst, double* cosdst);

Fortran interface：

CALL VSSINCOS(LEN, SRC, SINDST, COSDST);

CALL VDSINCOS(LEN, SRC, SINDST, COSDST);

参数

参数名	类型	描述	输入/输出
len	整型数	表示输入向量的元素个数。 len≤0时会提示参数len无效并返回。	输入

 版权所有 © 华为技术有限公司

参数名	类型	描述	输入/输出
src	• 在vssincos中是单精度浮点类型。 • 在vdsincos中是双精度浮点类型。	输入向量src，向量长度为len。 若为空指针，会提示空指针错误并返回。	输入
sindst	• 在vssincos中是单精度浮点类型。 • 在vdsincos中是双精度浮点类型。	输出sindst，向量长度为len。 若为空指针，会提示空指针错误并返回。	输出
cosdst	• 在vssincos中是单精度浮点类型。 • 在vdsincos中是双精度浮点类型。	输出cosdst，向量长度为len。 若为空指针，会提示空指针错误并返回。	输出

输出结果

- 每一个运算值返回弧度角x的正弦值sinx和余弦值cosx，sinx ∈ [-1, +1]，cosx ∈ [-1, +1]。
- 其他特殊值参考如下说明。

输入弧度（src）	输出正弦值（sindst）	输出余弦值（cosdst）
+0	+0	+1
-0	-0	+1
±inf	nan	nan
nan	nan	nan

依赖

C: "kvml.h"

示例

C interface：

```
int i, len = 4;
float src[len] = {0.0f, 2.0f, INFINITY, NAN};
float* sindst = (float*)malloc(sizeof(float) * len);
float* cosdst = (float*)malloc(sizeof(float) * len);
if (sindst == NULL || cosdst == NULL) {
    printf("Malloc Failed!\n");
    return 0;
}
vssincos(len, src, sindst, cosdst);
/**
 *  Output sindst:
```

 版权所有 © 华为技术有限公司

```
 *     0.0  0.909297406673431  nan  nan
 *  Output cosdst:
 *     1.0  -0.416146814823151  nan  nan
 *
 */
```

Fortran interface：

```
INTEGER :: LEN = 4
REAL(4) SRC(4)
REAL(4) COSDST(4)
REAL(4) SINDST(4)
DATA SRC /1, 2, 3, 4/
CALL VSSINCOS(LEN, SRC, SINDST, COSDST)
!
! OUTPUT SINDST:
!     0.841470957, 0.909297407, 0.141120002, -0.756802440
! OUTPUT COSDST:
!     0.540302277, -0.416146815, -0.989992499, -0.653643608
!
```

6.4.2.3.34 v?tan

计算输入弧度的正切值，输入为向量。

接口定义

C interface：

void vstan(const int len, const float* src, float* dst);

void vdtan(const int len, const double* src, double* dst);

void vctan(const int len, const float complex *src, float complex *dst);

void vztan(const int len, const double complex *src, double complex *dst);

Fortran interface：

CALL VSTAN(LEN, SRC, DST);

CALL VDTAN(LEN, SRC, DST);

参数

参数名	类型	描述	输入/输出
len	整型数	表示输入向量的元素个数。 len≤0时会提示参数len无效并返回。	输入

版权所有 © 华为技术有限公司

参数名	类型	描述	输入/输出
src	• 在vstan中是单精度浮点类型。 • 在vdtan中是双精度浮点类型。 • 在vctan中是复数单精度浮点类型。 • 在vztan中是复数双精度浮点类型。	输入向量src，向量长度为len。 若为空指针，会提示空指针错误并返回。	输入
dst	• 在vstan中是单精度浮点类型。 • 在vdtan中是双精度浮点类型。 • 在vctan中是复数单精度浮点类型。 • 在vztan中是复数双精度浮点类型。	输出dst，向量长度为len。 若为空指针，会提示空指针错误并返回。	输出

输出结果

- 每个运算值返回弧度角x的正切函数值y=tan(x)，y ∈ (-INF, +INF)。
- 该函数在π(1/2 + n)处具有数学极点；没有通用的浮点表示能够精确地表示极点处的值，因此极点处会返回值，但不能认为是正确的。
- vstan和vdtan的特殊值取值如下说明。

输入值（src）	输出值（dst）
+0	+0
-0	-0
±inf	nan
nan	nan

依赖

C: "kvml.h"

示例

C interface：

```
int i, len = 4;
float src[len] = {0.0f, 2.0f, INFINITY, NAN};
float* dst = (float*)malloc(sizeof(float) * len);
if (dst == NULL) {
```

 版权所有 © 华为技术有限公司

```
    printf("Malloc Failed!\n");
    return 0;
}
vstan(len, src, dst);
/**
 *  Output dst:
 *     0.0  -2.185039997100830  nan  nan
 *
 */
```

Fortran interface：

```
INTEGER :: LEN = 4
REAL(4) SRC(4)
REAL(4) DST(4)
DATA SRC /1, 2, 3, 4/
CALL VSTAN(LEN, SRC, DST)
!
! OUTPUT DST:
!     1.55740786, -2.18504000, -0.142546535, 1.15782118
!
```

6.4.2.3.35 v?atan

计算输入向量的反正切值，输入为向量。

接口定义

C interface：

void vsatan(const int len, const float* src, float* dst);

void vdatan(const int len, const double* src, double* dst);

void vcatan(const int len, const float complex *src, float complex *dst);

void vzatan(const int len, const double complex *src, double complex *dst);

Fortran interface：

CALL VSATAN(LEN, SRC, DST);

CALL VDATAN(LEN, SRC, DST);

参数

参数名	类型	描述	输入/输出
len	整型数	表示输入向量的元素个数。 len≤0时会提示参数len无效并返回。	输入

版权所有 © 华为技术有限公司

参数名	类型	描述	输入/输出
src	• 在vsatan中是单精度浮点类型。 • 在vdatan中是双精度浮点类型。 • 在vcatan中是复数单精度浮点类型。 • 在vzatan中是复数双精度浮点类型。	输入向量src，向量长度为len。 若为空指针，会提示空指针错误并返回。	输入
dst	• 在vsatan中是单精度浮点类型。 • 在vdatan中是双精度浮点类型。 • 在vcatan中是复数单精度浮点类型。 • 在vzatan中是复数双精度浮点类型。	输出dst，向量长度为len。 若为空指针，会提示空指针错误并返回。	输出

输出结果

- 每个运算值返回输入x对应的反正切值y=arctan(x)，y ∈(-π/2, +π/2)。
- 其他特殊值参考如下说明。

输入值（src）	输出值（dst）
+0	+0
-0	-0
±inf	±π/2
nan	nan

依赖

C: "kvml.h"

示例

C interface：

```
int i, len = 4;
float src[len] = {0.0f, 2.0f, INFINITY, NAN};
float* dst = (float*)malloc(sizeof(float) * len);
if (dst == NULL) {
    printf("Malloc Failed!\n");
    return 0;
}
```

 版权所有 © 华为技术有限公司

```
vsatan(len, src, dst);
/**
 *  Output dst:
 *     0.0  1.107148766517639  1.570796370506287  nan
 *
 */
```

Fortran interface：

```
INTEGER :: LEN = 4
REAL(4) SRC(4)
REAL(4) DST(4)
DATA SRC /1, 2, 3, 4/
CALL VSATAN(LEN, SRC, DST)
!
! OUTPUT DST:
!     0.785398185, 1.10714877, 1.24904573, 1.32581770
!
```

6.4.2.3.36 v?atanpi

计算输入向量的反正切值除以π的值，输入为向量。

接口定义

C interface：

void vsatanpi(const int len, const float* src, float* dst);

void vdatanpi(const int len, const double* src, double* dst);

参数

参数名	类型	描述	输入/输出
len	整型数	表示输入向量的元素个数。 len≤0时会提示参数len无效并返回。	输入
src	• 在vsatanpi中是单精度浮点类型。 • 在vdatanpi中是双精度浮点类型。	输入向量src，向量长度为len。 若为空指针，会提示空指针错误并返回。	输入
dst	• 在vsatanpi中是单精度浮点类型。 • 在vdatanpi中是双精度浮点类型。	输出dst，向量长度为len。 若为空指针，会提示空指针错误并返回。	输出

输出结果

- 计算输入x的反正切值除以π的值y=arctan(x)/π，y ∈(-1/2, +1/2)。
- 其他特殊值参考如下说明。

 版权所有 © 华为技术有限公司

输入值（src）	输出值（dst）
+0	+0
-0	-0
±inf	±0.5
nan	nan

依赖

C: "kvml.h"

示例

C interface：

```
int i, len = 4;
float src[len] = {0.0f, 2.0f, INFINITY, NAN};
float* dst = (float*)malloc(sizeof(float) * len);
if (dst == NULL) {
    printf("Malloc Failed!\n");
    return 0;
}
vsatanpi(len, src, dst);
/**
 *  Output dst:
 *     0.0  0.3524164  0.5  nan
 *
 */
```

6.4.2.3.37 v?atan2

计算src1/src2的反正切值，src1和src2以及结果均为向量。

接口定义

C interface：

void vsatan2(const int len, const float* src1, const float* src2, float* dst);

void vdatan2(const int len, const double* src1, const double* src2, double* dst);

Fortran interface：

CALL VSATAN2(LEN, SRC1, SRC2, DST);

CALL VDATAN2(LEN, SRC1, SRC2, DST);

 版权所有 © 华为技术有限公司

参数

参数名	类型	描述	输入/输出
len	整型数	表示输入向量的元素个数。 len≤0时会提示参数len无效并返回。	输入
src1	• 在vsatan2中是单精度浮点类型。 • 在vdatan2中是双精度浮点类型。	输入向量src1，向量长度为len。 若为空指针，会提示空指针错误并返回。	输入
src2	• 在vsatan2中是单精度浮点类型。 • 在vdatan2中是双精度浮点类型。	输入向量src2，向量长度为len。 若为空指针，会提示空指针错误并返回。	输入
dst	• 在vsatan2中是单精度浮点类型。 • 在vdatan2中是双精度浮点类型。	输出向量dst，向量长度为len。 若为空指针，会提示空指针错误并返回。	输出

输出结果

- 每个运算值atan2(y, x)返回的是原点至点(x, y)的方位角，即与x轴的夹角。返回值的单位为弧度，取值范围为(-π, +π]。
- 其他特殊值参考如下说明。

输入值1（src1）	输入值2（src2）	输出值（dst）
±0	-0	±π
±0	+0	±0
±∞	有限值	±π/2
±∞	-∞	±3π/4
±∞	+∞	±π/4
<0	±0	-π/2
>0	±0	-π/2
有限正值	-inf	+π
有限负值	-inf	-π
有限正值	+inf	+0
有限负值	+inf	-0

版权所有 © 华为技术有限公司

输入值1（src1）	输入值2（src2）	输出值（dst）
nan	any	nan
any	nan	nan

依赖

C: "kvml.h"

示例

C interface：

```
int i, len = 4;
float src1[len] = {0.0f, 1.0f, -inf, inf};
float src2[len] = {0.0f, 2.0f, INFINITY, NAN};
float* dst = (float*)malloc(sizeof(float) * len);
if (dst == NULL) {
    printf("Malloc Failed!\n");
    return 0;
}
vsatan2(len, src1, src2, dst);
/**
 *  Output dst:
 *     0.0  0.463647603988647  -0.785398185253143  nan
 *
 */
```

Fortran interface：

```
INTEGER :: LEN = 4
REAL(4) SRC1(4)
REAL(4) SRC2(4)
REAL(4) DST(4)
DATA SRC1 /1.2, 2.5, -2.4, 3.6/
DATA SRC2 /8.6, -2.5, -12.4, 32.6/
CALL VSATAN2(LEN, SRC1, SRC2, DST)
!
! OUTPUT DST:
!     0.138639733, 2.35619450, -2.95040822, 0.109983824
!
```

6.4.2.3.38 v?atan2pi

计算src1/src2的反正切值除以π的值，src1和src2以及结果均为向量。

接口定义

C interface：

void vsatan2pi(const int len, const float* src1, const float* src2, float* dst);

void vdatan2pi(const int len, const double* src1, const double* src2, double* dst);

 版权所有 © 华为技术有限公司

参数

参数名	类型	描述	输入/输出
len	整型数	表示输入向量的元素个数。 len≤0时会提示参数len无效并返回。	输入
src1	• 在vsatan2pi中是单精度浮点类型。 • 在vdatan2pi中是双精度浮点类型。	输入向量src1，向量长度为len。 若为空指针，会提示空指针错误并返回。	输入
src2	• 在vsatan2pi中是单精度浮点类型。 • 在vdatan2pi中是双精度浮点类型。	输入向量src2，向量长度为len。 若为空指针，会提示空指针错误并返回。	输入
dst	• 在vsatan2pi中是单精度浮点类型。 • 在vdatan2pi中是双精度浮点类型。	输出向量dst，向量长度为len。 若为空指针，会提示空指针错误并返回。	输出

输出结果

- 计算每个运算值的反正切值除以π的值，取值范围为[-1, +1]。
- 其他特殊值参考如下说明。

输入值1（src1）	输入值2（src2）	输出值（dst）
±0	-0	±1
±0	+0	±0
±∞	有限值	±1/2
±∞	-∞	±3/4
±∞	+∞	±1/4
<0	±0	-1/2
>0	±0	-1/2
有限正值	-inf	+1
有限负值	-inf	-1
有限正值	+inf	+0
有限负值	+inf	-0
nan	any	nan

版权所有 © 华为技术有限公司

输入值1（src1）	输入值2（src2）	输出值（dst）
any	nan	nan

依赖

C: "kvml.h"

示例

C interface：

```
int i, len = 4;
float src1[len] = {0.0f, 1.0f, -inf, inf};
float src2[len] = {0.0f, 2.0f, INFINITY, NAN};
float* dst = (float*)malloc(sizeof(float) * len);
if (dst == NULL) {
    printf("Malloc Failed!\n");
    return 0;
}
vsatan2pi(len, src1, src2, dst);
/**
 *  Output dst:
 *     0.0  0.14758362  -0.25  nan
 *
 */
```

6.4.2.3.39 v?sinh

计算输入弧度的双曲正弦值，输入为向量。

接口定义

C interface：

void vssinh(const int len, const float* src, float* dst);

void vdsinh(const int len, const double* src, double* dst);

void vcsinh(const int len, const float complex *src, float complex *dst);

void vzsinh(const int len, const double complex *src, double complex *dst);

Fortran interface：

CALL VSSINH(LEN, SRC, DST);

CALL VDSINH(LEN, SRC, DST);

参数

参数名	类型	描述	输入/输出
len	整型数	表示输入向量的元素个数。 len≤0时会提示参数len无效并返回。	输入

 版权所有 © 华为技术有限公司

参数名	类型	描述	输入/输出
src	• 在vssinh中是单精度浮点类型。 • 在vdsinh中是双精度浮点类型。 • 在vcsinh中是复数单精度浮点类型。 • 在vzsinh中是复数双精度浮点类型。	输入向量src，向量长度为len。 若为空指针，会提示空指针错误并返回。	输入
dst	• 在vssinh中是单精度浮点类型。 • 在vdsinh中是双精度浮点类型。 • 在vcsinh中是复数单精度浮点类型。 • 在vzsinh中是复数双精度浮点类型。	输出dst，向量长度为len。 若为空指针，会提示空指针错误并返回。	输出

输出结果

- 每一个运算值返回弧度角x的双曲正弦函数值y=sinh(x)，y ∈ [-∞, +∞]。
- 其他特殊值参考如下说明。

输入弧度（src）	输出双曲正弦值（dst）
+0	+0
-0	-0
+inf	+inf
-inf	-inf
nan	nan
x > 0x1.65a9f8p+6 (float)	+inf
x < -0x1.65a9f8p+6 (float)	-inf
x > 0x1.633ce8fb9f87dp9 (double)	+inf
x < -0x1.633ce8fb9f87dp9 (double)	-inf

依赖

C: "kvml.h"

 版权所有 © 华为技术有限公司

示例

C interface：

```
int i, len = 4;
float src[len] = {0.0f, 1.0f, INFINITY, NAN};
float* dst = (float*)malloc(sizeof(float) * len);
if (dst == NULL) {
    printf("Malloc Failed!\n");
    return 0;
}
vssinh(len, src, dst);
/**
 *  Output dst:
 *     0.0  1.1752011936  inf  nan
 *
 */
```

Fortran interface：

```
INTEGER :: LEN = 4
REAL(4) :: SRC(4)
REAL(4) :: DST(4)
DATA SRC/1, 2, 3, 4/
CALL VSSINH(LEN, SRC, DST)
!
! OUTPUT DST:
!     1.17520119, 3.62686041, 10.0178749, 27.2899172
!
```

6.4.2.3.40 v?cosh

计算输入弧度的双曲余弦值，输入为向量。

接口定义

C interface：

void vscosh(const int len, const float* src, float* dst);

void vdcosh(const int len, const double* src, double* dst);

void vccosh(const int len, const float complex *src, float complex *dst);

void vzcosh(const int len, const double complex *src, double complex *dst);

Fortran interface：

CALL VSCOSH(LEN, SRC, DST);

CALL VDCOSH(LEN, SRC, DST);

参数

参数名	类型	描述	输入/输出
len	整型数	表示输入向量的元素个数。 len≤0时会提示参数len无效并返回。	输入

版权所有 © 华为技术有限公司

参数名	类型	描述	输入/输出
src	• 在vscosh中是单精度浮点类型。 • 在vdcosh中是双精度浮点类型。 • 在vccosh中是复数单精度浮点类型。 • 在vzcosh中是复数双精度浮点类型。	输入向量src，向量长度为len。 若为空指针，会提示空指针错误并返回。	输入
dst	• 在vscosh中是单精度浮点类型。 • 在vdcosh中是双精度浮点类型。 • 在vccosh中是复数单精度浮点类型。 • 在vzcosh中是复数双精度浮点类型。	输出dst，向量长度为len。 若为空指针，会提示空指针错误并返回。	输出

输出结果

- 每一个运算值返回弧度角x的双曲余弦函数值y=cosh(x)，y ∈ [1, +∞]。
- 其他特殊值参考如下说明。

输入弧度（src）	输出双曲余弦值（dst）
+0	+1
-0	+1
+inf	+inf
-inf	+inf
nan	nan
|x| > 0x1.65a9f8p+6 (float)	+inf
|x| > 0x1.633ce8fb9f87dp9 (double)	+inf

依赖

C: "kvml.h"

示例

C interface：

 版权所有 © 华为技术有限公司

```
int i, len = 4;
float src[len] = {0.0f, 2.0f, INFINITY, NAN};
float* dst = (float*)malloc(sizeof(float) * len);
if (dst == NULL) {
    printf("Malloc Failed!\n");
    return 0;
}
vscosh(len, src, dst);
/**
 *  Output dst:
 *     1.0  3.762195691  inf  nan
 *
 */
```

Fortran interface：

```
INTEGER :: LEN = 4
REAL(4) SRC(4)
REAL(4) DST(4)
DATA SRC /1, 2, 3, 4/
CALL VSCOSH(LEN, SRC, DST)
!
! OUTPUT DST:
!     1.543080635, 3.762195691, 10.067661996, 27.308232836
!
```

6.4.2.3.41 v?tanh

计算输入弧度的双曲正切值，输入为向量。

接口定义

C interface：

void vstanh(const int len, const float* src, float* dst);

void vdtanh(const int len, const double* src, double* dst);

void vctanh(const int len, const float complex *src, float complex *dst);

void vztanh(const int len, const double complex *src, double complex *dst);

Fortran interface：

CALL VSTANH(LEN, SRC, DST);

CALL VDTANH(LEN, SRC, DST);

参数

参数名	类型	描述	输入/输出
len	整型数	表示输入向量的元素个数。 len≤0时会提示参数len无效并返回。	输入

版权所有 © 华为技术有限公司

参数名	类型	描述	输入/输出
src	• 在vstanh中是单精度浮点类型。 • 在vdtanh中是双精度浮点类型。 • 在vctanh中是复数单精度浮点类型。 • 在vztanh中是复数双精度浮点类型。	输入向量src，向量长度为len。 若为空指针，会提示空指针错误并返回。	输入
dst	• 在vstanh中是单精度浮点类型。 • 在vdtanh中是双精度浮点类型。 • 在vctanh中是复数单精度浮点类型。 • 在vztanh中是复数双精度浮点类型。	输出dst，向量长度为len。 若为空指针，会提示空指针错误并返回。	输出

输出结果

- 每个运算值返回弧度角x的双曲正切函数值y=tanh(x)=sinh(x)/cosh(x)，y ∈ [-1, +1]。
- 其他特殊值参考如下说明。

输入弧度（src）	输出双曲正切值（dst）
+0	+0
-0	-0
+inf	+1
-inf	-1
nan	nan

依赖

C: "kvml.h"

示例

C interface：

```
int i, len = 4;
float src[len] = {0.0f, 2.0f, INFINITY, NAN};
float* dst = (float*)malloc(sizeof(float) * len);
```

 版权所有 © 华为技术有限公司

```
if (dst == NULL) {
    printf("Malloc Failed!\n");
    return 0;
}
vstanh(len, src, dst);
/**
 *  Output dst:
 *     0.0  0.964027580  1.0  nan
 *
 */
```

Fortran interface：

```
INTEGER :: LEN = 4
REAL(4) SRC(4)
REAL(4) DST(4)
DATA SRC /1, 2, 3, 4/
CALL VSTANH(LEN, SRC, DST)
!
! OUTPUT DST:
!     0.761594156, 0.964027580, 0.995054754, 0.9993292997
!
```

6.4.2.3.42 v?asinh

计算输入弧度的双曲反正弦值，输入为向量。

接口定义

C interface：

void vsasinh(const int len, const float* src, float* dst);

void vdasinh(const int len, const double* src, double* dst);

void vcasinh(const int len, const float complex *src, float complex *dst);

void vzasinh(const int len, const double complex *src, double complex *dst);

Fortran interface：

CALL VSASINH(LEN, SRC, DST);

CALL VDASINH(LEN, SRC, DST);

参数

参数名	类型	描述	输入/输出
len	整型数	表示输入向量的元素个数。 len≤0时会提示参数len无效并返回。	输入

版权所有 © 华为技术有限公司

参数名	类型	描述	输入/输出
src	• 在vsasinh中是单精度浮点类型。 • 在vdasinh中是双精度浮点类型。 • 在vcasinh中是复数单精度浮点类型。 • 在vzasinh中是复数双精度浮点类型。	输入向量src，向量长度为len。 若为空指针，会提示空指针错误并返回。	输入
dst	• 在vsasinh中是单精度浮点类型。 • 在vdasinh中是双精度浮点类型。 • 在vcasinh中是复数单精度浮点类型。 • 在vzasinh中是复数双精度浮点类型。	输出dst，向量长度为len。 若为空指针，会提示空指针错误并返回。	输出

输出结果

- 每一个运算值返回弧度角x的双曲反正弦函数值y=asinh(x)。
- 其他特殊值参考如下说明。

输入弧度（src）	输出双曲反正弦值（dst）
+0	+0
-0	-0
+inf	+inf
-inf	-inf
nan	nan

依赖

C: "kvml.h"

示例

C interface：

```
int i, len = 4;
float src[len] = {0.0f, 1.0f, INFINITY, NAN};
float* dst = (float*)malloc(sizeof(float) * len);
if (dst == NULL) {
    printf("Malloc Failed!\n");
```

 版权所有 © 华为技术有限公司

```
    return 0;
}
vsasinh(len, src, dst);
/**
 *  Output dst:
 *     0.0  0.881373587019543  inf  nan
 *
 */
```

Fortran interface：

```
INTEGER :: LEN = 4
REAL(4) :: SRC(4)
REAL(4) :: DST(4)
DATA SRC/1, 2, 3, 4/
CALL VSASINH(LEN, SRC, DST)
!
! OUTPUT DST:
!     0.881373587, 1.443635475, 1.818446459, 2.094712547
!
```

6.4.2.3.43 v?acosh

计算输入弧度的反双曲余弦值，输入为向量。

接口定义

C interface：

void vsacosh(const int len, const float* src, float* dst);

void vdacosh(const int len, const double* src, double* dst);

void vcacosh(const int len, const float complex *src, float complex *dst);

void vzacosh(const int len, const double complex *src, double complex *dst);

Fortran interface：

CALL VSACOSH(LEN, SRC, DST);

CALL VDACOSH(LEN, SRC, DST);

参数

参数名	类型	描述	输入/输出
len	整型数	表示输入向量的元素个数。 len≤0时会提示参数len无效并返回。	输入

版权所有 © 华为技术有限公司

参数名	类型	描述	输入/输出
src	• 在vsacosh中是单精度浮点类型。 • 在vdacosh中是双精度浮点类型。 • 在vcacosh中是复数单精度浮点类型。 • 在vzacosh中是复数双精度浮点类型。	输入向量src，向量长度为len。 若为空指针，会提示空指针错误并返回。	输入
dst	• 在vsacosh中是单精度浮点类型。 • 在vdacosh中是双精度浮点类型。 • 在vcacosh中是复数单精度浮点类型。 • 在vzacosh中是复数双精度浮点类型。	输出dst，向量长度为len。 若为空指针，会提示空指针错误并返回。	输出

输出结果

- 返回每一个弧度角x的双曲反余弦函数值y=acosh(x)，x ∈ [1, +∞)。
- 其他特殊值参考如下说明。

输入弧度（src）	输出双曲反余弦值（dst）
+1	+0
<1	nan
+inf	+inf
-inf	nan
nan	nan

依赖

C: "kvml.h"

示例

C interface：

```
int i, len = 4;
float src[len] = {0.0f, 2.0f, INFINITY, NAN};
float* dst = (float*)malloc(sizeof(float) * len);
if (dst == NULL) {
    printf("Malloc Failed!\n");
```

 版权所有 © 华为技术有限公司

```
    return 0;
}
vsacosh(len, src, dst);
/**
 *  Output dst:
 *     nan  1.316957896924817  inf  nan
 *
 */
```

Fortran interface：

```
INTEGER :: LEN = 4
REAL(4) SRC(4)
REAL(4) DST(4)
DATA SRC /1, 2, 3, 4/
CALL VSACOSH(LEN, SRC, DST)
!
! OUTPUT DST:
!     0, 1.316957897, 1.762747174, 2.063437069
!
```

6.4.2.3.44 v?atanh

计算输入弧度的双曲反正切值，输入为向量。

接口定义

C interface：

void vsatanh(const int len, const float* src, float* dst);

void vdatanh(const int len, const double* src, double* dst);

void vcatanh(const int len, const float complex *src, float complex *dst);

void vzatanh(const int len, const double complex *src, double complex *dst);

Fortran interface：

CALL VSATANH(LEN, SRC, DST);

CALL VDATANH(LEN, SRC, DST);

参数

参数名	类型	描述	输入/输出
len	整型数	表示输入向量的元素个数。 len≤0时会提示参数len无效并返回。	输入

版权所有 © 华为技术有限公司

参数名	类型	描述	输入/输出
src	• 在vsatanh中是单精度浮点类型。 • 在vdatanh中是双精度浮点类型。 • 在vcatanh中是复数单精度浮点类型。 • 在vzatanh中是复数双精度浮点类型。	输入向量src，向量长度为len。 若为空指针，会提示空指针错误并返回。	输入
dst	• 在vsatanh中是单精度浮点类型。 • 在vdatanh中是双精度浮点类型。 • 在vcatanh中是复数单精度浮点类型。 • 在vzatanh中是复数双精度浮点类型。	输出dst，向量长度为len。 若为空指针，会提示空指针错误并返回。	输出

输出结果

- 每个运算值返回弧度角x的双曲反正切函数值y=atanh(x)，x ∈ (-1, +1)。
- 其他特殊值参考如下说明。

输入弧度（src）	输出双曲反正切值（dst）
+1	+inf
-1	-inf
|x| > 1	nan
+inf	nan
-inf	nan
nan	nan

依赖

C: "kvml.h"

示例

C interface：

```
int i, len = 4;
float src[len] = {0.0f, 0.2f, INFINITY, NAN};
```

 版权所有 © 华为技术有限公司

```
float* dst = (float*)malloc(sizeof(float) * len);
if (dst == NULL) {
    printf("Malloc Failed!\n");
    return 0;
}
vsatanh(len, src, dst);
/**
 *  Output dst:
 *     0.0  0.202732554  nan  nan
 *
 */
```

Fortran interface：

```
INTEGER :: LEN = 4
REAL(4) SRC(4)
REAL(4) DST(4)
DATA SRC /0.1, 0.2, 0.3, 0.4/
CALL VSATANH(LEN, SRC, DST)
!
! OUTPUT DST:
!     0.100335348, 0.202732554, 0.309519604, 0.423648930
!
```

6.4.2.3.45 v?sind

计算输入的正弦值，输入为角度向量。

接口定义

C interface：

void vssind(const int len, const float* src, float* dst);

void vdsind(const int len, const double* src, double* dst);

参数

参数名	类型	描述	输入/输出
len	整型数	表示输入向量的元素个数。 len≤0时会提示参数len无效并返回。	输入
src	• 在vssind中是单精度浮点类型。 • 在vdsind中是双精度浮点类型。	输入向量src，向量长度为len。 若为空指针，会提示空指针错误并返回。	输入
dst	• 在vssind中是单精度浮点类型。 • 在vdsind中是双精度浮点类型。	输出dst，向量长度为len。 若为空指针，会提示空指针错误并返回。	输出

 版权所有 © 华为技术有限公司

输出结果

- 每一个运算值返回角度值x的正弦函数值y=sind(x)=sin(x * π / 180)，y ∈ [-1, +1]。
- 其他特殊值参考如下说明。

输入角度值（src）	输出正弦值（dst）
+0	+0
-0	-0
+inf	nan
-inf	nan
nan	nan

依赖

C: "kvml.h"

示例

C interface：

```
float src[LEN] = {0.0f, 2.0f, INFINITY, NAN};
float* dst = (float*)malloc(sizeof(float) * LEN);
if (dst == NULL) {
    printf("Malloc Failed!\n");
    return;
}
vssind(LEN, src, dst);
/**
 *  Output dst:
 *     0.0  0x1.1de58cp-5  nan  nan
 *
 */
```

6.4.2.3.46 v?cosd

计算输入的余弦值，输入为角度向量。

接口定义

C interface：

void vscosd(const int len, const float* src, float* dst);

void vdcosd(const int len, const double* src, double* dst);

 版权所有 © 华为技术有限公司

参数

参数名	类型	描述	输入/输出
len	整型数	表示输入向量的元素个数。 len≤0时会提示参数len无效并返回。	输入
src	• 在vscosd中是单精度浮点类型。 • 在vdcosd中是双精度浮点类型。	输入向量src，向量长度为len。 若为空指针，会提示空指针错误并返回。	输入
dst	• 在vscosd中是单精度浮点类型。 • 在vdcosd中是双精度浮点类型。	输出dst，向量长度为len。 若为空指针，会提示空指针错误并返回。	输出

输出结果

- 每一个运算值返回角度值x的余弦函数值y=cosd(x)=cos(x * π / 180)，y ∈ [-1, +1]。
- 其他特殊值参考如下说明。

输入角度值（src）	输出余弦值（dst）
+0	+1
-0	+1
+inf	nan
-inf	nan
nan	nan

依赖

C: "kvml.h"

示例

C interface：

```
float src[LEN] = {0.0f, 2.0f, INFINITY, NAN};
float* dst = (float*)malloc(sizeof(float) * LEN);
if (dst == NULL) {
    printf("Malloc Failed!\n");
    return;
}
vscosd(LEN, src, dst);
/**
 *  Output dst:
```

 版权所有 © 华为技术有限公司

```
*    1.0  0x1.ffb026p-1  nan  nan
*
*/
```

6.4.2.3.47 v?trunc

矢量计算输入向0舍入的整数值，以浮点数返回。

接口定义

C interface：

void vstrunc(const int len, const float* src, float* dst);

void vdtrunc(const int len, const double* src, double* dst);

参数

参数名	类型	描述	输入/输出
len	整型数	表示输入向量的元素个数。 len≤0时会提示参数len无效并返回。	输入
src	• 在vstrunc中是单精度浮点类型。 • 在vdtrunc中是双精度浮点类型。	输入向量src，向量长度为len。 若为空指针，会提示空指针错误并返回。	输入
dst	• 在vstrunc中是单精度浮点类型。 • 在vdtrunc中是双精度浮点类型。	输出dst，向量长度为len。 若为空指针，会提示空指针错误并返回。	输出

输出结果

- 每一个运算值返回x的向0取整的值。
- 其他特殊值参考如下说明。

输入值（src）	输出值（dst）
+0	+0
-0	+0
+inf	+inf
-inf	-inf
nan	nan

版权所有 © 华为技术有限公司

依赖

C: "kvml.h"

示例

C interface：

```
float src[LEN] = {1.1f, -2.1f, INFINITY, NAN};
float* dst = (float*)malloc(sizeof(float) * LEN);
if (dst == NULL) {
    printf("Malloc Failed!\n");
    return;
}
vstrunc(LEN, src, dst);
/**
 *  Output dst:
 *     1.0  -2.0  INFINITY  nan
 *
 */
```

6.4.2.3.48 v?round

矢量计算输入的四舍五入的整数值，以浮点数返回。

接口定义

C interface：

void vsround(const int len, const float* src, float* dst);

void vdround(const int len, const double* src, double* dst);

参数

参数名	类型	描述	输入/输出
len	整型数	表示输入向量的元素个数。 len≤0时会提示参数len无效并返回。	输入
src	• 在vsround中是单精度浮点类型。 • 在vdround中是双精度浮点类型。	输入向量src，向量长度为len。 若为空指针，会提示空指针错误并返回。	输入
dst	• 在vsround中是单精度浮点类型。 • 在vdround中是双精度浮点类型。	输出dst，向量长度为len。 若为空指针，会提示空指针错误并返回。	输出

输出结果

- 每一个运算值返回x的四舍五入取整的值。

 版权所有 © 华为技术有限公司

- 其他特殊值参考如下说明。

输入值（src）	输出值（dst）
+0	+0
-0	+0
+inf	+inf
-inf	-inf
nan	nan

依赖

C: "kvml.h"

示例

C interface：

```
float src[LEN] = {1.1f, -2.6f, INFINITY, NAN};
float* dst = (float*)malloc(sizeof(float) * LEN);
if (dst == NULL) {
    printf("Malloc Failed!\n");
    return;
}
vsround(LEN, src, dst);
/**
 *  Output dst:
 *     1.0  -3.0  INFINITY  nan
 *
 */
```

6.4.2.3.49 v?nearbyint

矢量计算输入的基于当前舍入模式取整舍入的整数值，以浮点数返回。

接口定义

C interface：

void vsnearbyint(const int len, const float* src, float* dst);

void vdnearbyint(const int len, const double* src, double* dst);

参数

参数名	类型	描述	输入/输出
len	整型数	表示输入向量的元素个数。 len≤0时会提示参数len无效并返回。	输入

 版权所有 © 华为技术有限公司

参数名	类型	描述	输入/输出
src	• 在vsnearbyint中是单精度浮点类型。 • 在vdnearbyint中是双精度浮点类型。	输入向量src，向量长度为len。 若为空指针，会提示空指针错误并返回。	输入
dst	• 在vsnearbyint中是单精度浮点类型。 • 在vdnearbyint中是双精度浮点类型。	输出dst，向量长度为len。 若为空指针，会提示空指针错误并返回。	输出

输出结果

- 每一个运算值返回x的基于当前舍入模式的取整的值。
- 其他特殊值参考如下说明。

输入值（src）	输出值（dst）
+0	+0
-0	+0
+inf	+inf
-inf	-inf
nan	nan

依赖

C: "kvml.h"

示例

C interface：

```
float src[LEN] = {1.1f, -2.6f, INFINITY, NAN};
float* dst = (float*)malloc(sizeof(float) * LEN);
if (dst == NULL) {
    printf("Malloc Failed!\n");
    return;
}
vsnearbyint(LEN, src, dst);
/**
 *  Output dst:
 *     1.0  -3.0  INFINITY  nan
 *
 */
```

6.4.2.3.50 v?rint

矢量计算输入的基于当前舍入模式取整舍入的整数值，以浮点数返回。

 版权所有 © 华为技术有限公司

接口定义

C interface：

void vsrint(const int len, const float* src, float* dst);

void vdrint(const int len, const double* src, double* dst);

参数

参数名	类型	描述	输入/输出
len	整型数	表示输入向量的元素个数。 len≤0时会提示参数len无效并返回。	输入
src	• 在vsrint中是单精度浮点类型。 • 在vdrint中是双精度浮点类型。	输入向量src，向量长度为len。 若为空指针，会提示空指针错误并返回。	输入
dst	• 在vsrint中是单精度浮点类型。 • 在vdrint中是双精度浮点类型。	输出dst，向量长度为len。 若为空指针，会提示空指针错误并返回。	输出

输出结果

- 每一个运算值返回x的基于当前舍入模式的取整的值。
- 其他特殊值参考如下说明。

输入值（src）	输出值（dst）
+0	+0
-0	+0
+inf	+inf
-inf	-inf
nan	nan

依赖

C: "kvml.h"

示例

C interface：

```
float src[LEN] = {1.1f, -2.6f, INFINITY, NAN};
float* dst = (float*)malloc(sizeof(float) * LEN);
```

 版权所有 © 华为技术有限公司

```
if (dst == NULL) {
    printf("Malloc Failed!\n");
    return;
}
vsrint(LEN, src, dst);
/**
 *  Output dst:
 *     1.0  -3.0  INFINITY  nan
 *
 */
```

6.4.2.3.51 v?frac

矢量计算输入的小数部分。

接口定义

C interface：

void vsfrac(const int len, const float* src, float* dst);

void vdfrac(const int len, const double* src, double* dst);

参数

参数名	类型	描述	输入/输出
len	整型数	表示输入向量的元素个数。 len≤0时会提示参数len无效并返回。	输入
src	• 在vsfrac中是单精度浮点类型。 • 在vdfrac中是双精度浮点类型。	输入向量src，向量长度为len。 若为空指针，会提示空指针错误并返回。	输入
dst	• 在vsfrac中是单精度浮点类型。 • 在vdfrac中是双精度浮点类型。	输出dst，向量长度为len。 若为空指针，会提示空指针错误并返回。	输出

输出结果

- 每一个运算值返回x的小数部分。
- 其他特殊值参考如下说明。

输入值（src）	输出值（dst）
+0	+0
-0	-0
+inf	+0

版权所有 © 华为技术有限公司

输入值（src）	输出值（dst）
-inf	-0
nan	nan

依赖

C: "kvml.h"

示例

C interface：

```
float src[LEN] = {1.1f, -2.6f, INFINITY, NAN};
float* dst = (float*)malloc(sizeof(float) * LEN);
if (dst == NULL) {
    printf("Malloc Failed!\n");
    return;
}
vsfrac(LEN, src, dst);
/**
 *  Output dst:
 *     0.1   -0.6  0.0  nan
 *
 */
```

6.4.2.3.52 v?fmod

矢量计算两个输入向量的浮点余数。

接口定义

C interface：

void vsfmod(const int len, const float* src1, const float* src2, float* dst);

void vdfmod(const int len, const double* src1, const double* src2, double* dst);

参数

参数名	类型	描述	输入/输出
len	整型数	表示输入向量的元素个数。 len≤0时会提示参数len无效并返回。	输入
src1	• 在vsfmod中是单精度浮点类型。 • 在vdfmod中是双精度浮点类型。	输入向量src1，向量长度为len。 若为空指针，会提示空指针错误并返回。	输入

 版权所有 © 华为技术有限公司

参数名	类型	描述	输入/输出
src2	• 在vsfmod中是单精度浮点类型。 • 在vdfmod中是双精度浮点类型。	输入向量src2，向量长度为len。 若为空指针，会提示空指针错误并返回。	输入
dst	• 在vsfmod中是单精度浮点类型。 • 在vdfmod中是双精度浮点类型。	输出dst，向量长度为len。 若为空指针，会提示空指针错误并返回。	输出

输出结果

- 每一个运算值返回向量的浮点余数，计算公式为fmod(x, y) = x - trunc(x/y) * y。
- 其他特殊值参考如下说明。

输入值（src1）	输入值（src2）	输出值（dst）
any	±nan	nan
any	±0	nan
±inf	any	nan
±nan	any	nan

依赖

C: "kvml.h"

示例

C interface：

```
#define N 4
const float src1[N] = {-8.0f, 25.0f, +INFINITY, -0.0f};
const float src2[N] = {6.2f, -4.3f, -192.4, -3.1f};
float dst[N] = {0};

vsfmod(N, src1, src2, dst);
printFArr("input1:", N, src1);
printFArr("input2:", N, src2);
printFArr("output:", N, dst);

/**
 *  input1: -8.000   25.000      inf   -0.000
 *  input2:  6.200   -4.300 -192.400   -3.100
 *  output: -1.800    3.500      nan   -0.000
 */
```

6.4.2.3.53 v?arg

矢量计算输入复数的幅角，即虚部与实部比值的反正切值。

 版权所有 © 华为技术有限公司

接口定义

C interface：

void vcarg(const int len, const float complex* src, float* dst);

void vzarg(const int len, const double complex* src, double* dst);

参数

参数名	类型	描述	输入/输出
len	整型数	表示输入向量的元素个数。 len≤0时会提示参数len无效并返回。	输入
src	• 在vcarg中是复数单精度浮点类型。 • 在vzarg中是复数双精度浮点类型。	输入向量src，向量长度为len。 若为空指针，会提示空指针错误并返回。	输入
dst	• 在vcarg中是单精度浮点类型。 • 在vzarg中是双精度浮点类型。	输出dst，向量长度为len。 若为空指针，会提示空指针错误并返回。	输出

输出结果

- 每一个运算值返回输入复数的幅角，计算公式为arg(x + yi) = arctan(y/x)。
- 其他特殊值参考如下说明。

RE(z) i * IM(z)	-∞	-X	-0	+0	+X	+∞	NAN
+i * ∞	+∞+i* ∞	+π/2	+π/2	+π/2	+π/2	+π/4	NAN
+i * Y	+π	-	+π/2	+π/2	-	+0	NAN
+i * 0	+π	+π	+π	+0	+0	+0	NAN
-i * 0	-π	-π	-π	-0	-0	-0	NAN
-i * Y	-π	-	-π/2	-π/2	-	-0	NAN
-i * ∞	-3*π/4	-π/2	-π/2	-π/2	-π/2	-π/4	NAN
+i * NAN	NAN	NAN	NAN	NAN	NAN	NAN	NAN

版权所有 © 华为技术有限公司

依赖

C: "kvml.h"

示例

C interface：

```
#define N 4
const complex float src[N] = {
    __builtin_complex(-8.0f, 25.0f),
    __builtin_complex(+INFINITY, -2.1f),
    __builtin_complex(2.6f, 25.0f),
    __builtin_complex(-0.0f, +0.1f)
};
float dst[N] = {0};

vcarg(N, src, dst);
printFArr("output:", N, dst);

/**
 *  input: (-8.000,25.000) (  inf,-2.100) (2.600,25.000)  (-0.000,0.100)
 *  output: 1.880    -0.000     1.467     1.571
 */
```

6.4.2.3.54 v?conj

矢量计算输入复数的共轭值。

接口定义

C interface：

void vcconj(const int len, const float complex* src, float complex* dst);

void vzconj(const int len, const double complex* src, double complex* dst);

参数

参数名	类型	描述	输入/输出
len	整型数	表示输入向量的元素个数。 len≤0时会提示参数len无效并返回。	输入
src	• 在vcconj中是复数单精度浮点类型。 • 在vzconj中是复数双精度浮点类型。	输入向量src，向量长度为len。 若为空指针，会提示空指针错误并返回。	输入
dst	• 在vcconj中是复数单精度浮点类型。 • 在vzconj中是复数双精度浮点类型。	输出dst，向量长度为len。 若为空指针，会提示空指针错误并返回。	输出

版权所有 © 华为技术有限公司

输出结果

- 每一个运算结果返回输入复数的共轭值，计算公式为conj(x + yi) = (x - yi)。
- 其他特殊值参考如下说明。

输入实部（src->real）	输入虚部（src->imag）	输出实部（dst->real）	输出虚部（dst->imag）
任意值xR	nan	xR	nan
任意值xR	0	xR	-0
任意值xR	-0	xR	+0
任意值xR	-inf	xR	inf
任意值xR	+inf	xR	-inf

依赖

C: "kvml.h"

示例

C interface：

```
#define N 4
const complex float src[N] = {
    __builtin_complex(-8.0f, 25.0f),
    __builtin_complex(-2.1f, +INFINITY),
    __builtin_complex(2.6f, 25.0f),
    __builtin_complex(+0.1f, NAN)
};
float complex dst[N] = {0};

vcconj(N, src, dst);
printFCArr("input: ", N, src);
printFCArr("output:", N, dst);

/**
 *  input: (-8.000,25.000) (-2.100,  inf) (2.600,25.000) (0.100,  nan)
 *  output:(-8.000,-25.000) (-2.100, -inf) (2.600,-25.000) (0.100, -nan)
 */
```

6.4.2.3.55 v?mulbyconj

计算第一个输入向量乘以第二个向量的共轭值。

接口定义

C interface：

void vcmulbyconj(const int len, const float complex* src1, const float complex* src2, float complex* dst);

void vzmulbyconj(const int len, const double complex* src1, const double complex* src2, double complex* dst);

 版权所有 © 华为技术有限公司

参数

参数名	类型	描述	输入/输出
len	整型数	表示输入向量的元素个数。 len≤0时会提示参数len无效并返回。	输入
src1	• 在vcmulbyconj中是复数单精度浮点类型。 • 在vzmulbyconj中是复数双精度浮点类型。	输入向量src1，向量长度为len。 若为空指针，会提示空指针错误并返回。	输入
src2	• 在vcmulbyconj中是复数单精度浮点类型。 • 在vzmulbyconj中是复数双精度浮点类型。	输入向量src2，向量长度为len。 若为空指针，会提示空指针错误并返回。	输入
dst	• 在vcmulbyconj中是复数单精度浮点类型。 • 在vzmulbyconj中是复数双精度浮点类型。	输出dst，向量长度为len。 若为空指针，会提示空指针错误并返回。	输出

输出结果

- 返回对src2取共轭值后，与src1相乘的结果。
- 特殊值请在社区页面文档本章节下载附件查看特殊输出结果。

依赖

C: "kvml.h"

示例

C interface：

```
#define N 4
const complex float src1[N] = {
    __builtin_complex(-8.0f, 25.0f),
    __builtin_complex(-2.1f, +INFINITY),
    __builtin_complex(2.6f, 25.0f),
    __builtin_complex(+0.1f, NAN)
};
const complex float src2[N] = {
    __builtin_complex(-3.0f, 5.2f),
    __builtin_complex(NAN, -0.0f),
    __builtin_complex(-9.5f, 6.0f),
    __builtin_complex(-1.1f, 1.1f)
};
float complex dst[N] = {0};

vcmulbyconj(N, src1, src2, dst);
printFCArr("input1:", N, src1);
printFCArr("input2:", N, src2);
printFCArr("output:", N, dst);
```

 版权所有 © 华为技术有限公司

```
/**
 *  input1: (-8.000,25.000) (-2.100,  inf) (2.600,25.000) (0.100,  nan)
 *  input2: (-3.000,5.200) (  nan,-0.000) (-9.500,6.000) (-1.100,1.100)
 *  output: (154.000,-33.400) (  nan,  nan) (125.300,-253.100) ( -nan,  nan)
 */
```

6.4.2.3.56 v?cis

矢量计算以入参实数x为输入，返回 $e^{ix} = (\cos x + i \cdot \sin x)$ 的复数结果。

接口定义

C interface：

void vccis(const int len, const float *src, float complex *dst);

void vzcis(const int len, const double *src, double complex *dst);

参数

参数名	类型	描述	输入/输出
len	整型数	表示输入向量的元素个数。 len≤0时会提示参数len无效并返回。	输入
src	• 在vccis中是单精度浮点类型。 • 在vzcis中是双精度浮点类型。	输入向量src，向量长度为len。 若为空指针，会提示空指针错误并返回。	输入
dst	• 在vccis中是复数单精度浮点类型。 • 在vzcis中是复数双精度浮点类型。	输出dst，向量长度为len。 若为空指针，会提示空指针错误并返回。	输出

输出结果

- 每一个运算值返回一个复数，实部为cos，虚部为sin。
- 其他特殊值参考如下说明。

输入值（src）	输出值实部（y.real）	输出值虚部（y.imag）
+0	1	+0
-0	1	-0
±inf	nan	nan
nan	nan	nan

版权所有 © 华为技术有限公司

依赖

C: "kvml.h"

示例

C interface：

```
int i, len = 4;
float src[4] = {0.0f, 2.0f, INFINITY, NAN};
float complex* dst = (float complex*)malloc(sizeof(float complex) * len);
if (dst == NULL) {
    printf("Malloc Failed!\n");
    return 0;
}
vccis(len, src, dst);
/**
 *  Output dst:
 *     1.000000+0.000000*I -0.416147+0.909297*I nan+nan*I nan+nan*I
 */
```

6.4.2.3.57 v?acos

计算输入弧度的反余弦值，输入为向量。

接口定义

C interface：

void vsacos(const int len, const float* src, float* dst);

void vdacos(const int len, const double* src, double* dst);

void vcacos(const int len, const float complex *src, float complex *dst);

void vzacos(const int len, const double complex *src, double complex *dst);

参数

参数名	类型	描述	输入/输出
len	整型数	表示输入向量的元素个数。 len≤0时会提示参数len无效并返回。	输入
src	• 在vsacos中是单精度浮点类型。 • 在vdacos中是双精度浮点类型。 • 在vcacos中是复数单精度浮点类型。 • 在vzacos中是复数双精度浮点类型。	输入向量src，向量长度为len。 若为空指针，会提示空指针错误并返回。	输入

版权所有 © 华为技术有限公司

参数名	类型	描述	输入/输出
dst	• 在vsacos中是单精度浮点类型。 • 在vdacos中是双精度浮点类型。 • 在vcacos中是复数单精度浮点类型。 • 在vzacos中是复数双精度浮点类型。	输出dst，向量长度为len。 若为空指针，会提示空指针错误并返回。	输出

输出结果

- 返回余弦值x的反余弦函数值y，x ∈ [-1, +1]。
- vsacos和vdacos其他特殊值参考如下说明。

输入值（src）	输出值（dst）
±inf	nan
nan	nan
+1	+0
-1	+pi
|x|>1	nan

- vcacos和vzacos其他特殊值参考如下说明。

输入值实部（src.real）	输入值虚部（src.imag）	输出值实部（dst.real）	输出值虚部（dst.imag）
0	0	pi/2	0
0	0	pi/2	0
0	inf	pi/2	-inf
0	-inf	pi/2	inf
0	snan	pi/2	nan
0	-snan	pi/2	nan
0	qnan	pi/2	nan
0	-qnan	pi/2	nan
0	0	pi/2	0
0	0	pi/2	0

版权所有 © 华为技术有限公司

输入值实部（src.real）	输入值虚部（src.imag）	输出值实部（dst.real）	输出值虚部（dst.imag）
0	inf	pi/2	-inf
0	-inf	pi/2	inf
0	snan	pi/2	nan
0	-snan	pi/2	nan
0	qnan	pi/2	nan
0	-qnan	pi/2	nan
inf	0	0	-inf
inf	0	0	inf
inf	inf	pi/4	-inf
inf	-inf	pi/4	inf
inf	snan	nan	-inf
inf	-snan	nan	-inf
inf	qnan	nan	-inf
inf	-qnan	nan	-inf
-inf	0	pi	-inf
-inf	0	pi	inf
-inf	inf	3*pi/4	-inf
-inf	-inf	3*pi/4	inf
-inf	snan	nan	-inf
-inf	-snan	nan	-inf
-inf	qnan	nan	-inf
-inf	-qnan	nan	-inf
snan	0	nan	nan
snan	0	nan	nan
snan	inf	nan	-inf
snan	-inf	nan	inf
snan	snan	nan	nan
snan	-snan	nan	nan
snan	qnan	nan	nan

 版权所有 © 华为技术有限公司

输入值实部（src.real）	输入值虚部（src.imag）	输出值实部（dst.real）	输出值虚部（dst.imag）
snan	-qnan	nan	nan
-snan	0	nan	nan
-snan	0	nan	nan
-snan	inf	nan	-inf
-snan	-inf	nan	inf
-snan	snan	nan	nan
-snan	-snan	nan	nan
-snan	qnan	nan	nan
-snan	-qnan	nan	nan
qnan	0	nan	nan
qnan	0	nan	nan
qnan	inf	nan	-inf
qnan	-inf	nan	inf
qnan	snan	nan	nan
qnan	-snan	nan	nan
qnan	qnan	nan	nan
qnan	-qnan	nan	nan
-qnan	0	nan	nan
-qnan	0	nan	nan
-qnan	inf	nan	-inf
-qnan	-inf	nan	inf
-qnan	snan	nan	nan
-qnan	-snan	nan	nan
-qnan	qnan	nan	nan
-qnan	-qnan	nan	nan

依赖

C: "kvml.h"

版权所有 © 华为技术有限公司

示例

C interface：

```
int i, len = 4;
float src[4] = {0.0f, 2.0f, INFINITY, NAN};
float* dst = (float*)malloc(sizeof(float) * len);
if (dst == NULL) {
    printf("Malloc Failed!\n");
    return 0;
}
vsacos(len, src, dst);

/**
 *  Output dst:
 *     1.570796 nan nan nan
 */
```

6.4.2.3.58 v?asin

计算输入弧度的反正弦值，输入为向量。

接口定义

C interface：

void vsasin(const int len, const float* src, float* dst);

void vdasin(const int len, const double* src, double* dst);

void vcasin(const int len, const float complex *src, float complex *dst);

void vzasin(const int len, const double complex *src, double complex *dst);

参数

参数名	类型	描述	输入/输出
len	整型数	表示输入向量的元素个数。 len≤0时会提示参数len无效并返回。	输入
src	• 在vsasin中是单精度浮点类型。 • 在vdasin中是双精度浮点类型。 • 在vcasin中是复数单精度浮点类型。 • 在vzasin中是复数双精度浮点类型。	输入向量src，向量长度为len。 若为空指针，会提示空指针错误并返回。	输入

版权所有 © 华为技术有限公司

参数名	类型	描述	输入/输出
dst	• 在vsasin中是单精度浮点类型。 • 在vdasin中是双精度浮点类型。 • 在vcasin中是复数单精度浮点类型。 • 在vzasin中是复数双精度浮点类型。	输出dst，向量长度为len。 若为空指针，会提示空指针错误并返回。	输出

输出结果

- 返回正弦值x的反正弦函数值y，x ∈ [-1, +1]。
- vsasin和vdasin其他特殊值参考如下说明。

输入值（src）	输出值（dst）
+0	+0
-0	-0
±1	±pi/2
±inf	nan
nan	nan
|x|>1	nan

- vcasin和vzasin其他特殊值参考如下说明。

输入值实部（x.real）	输入值虚部（x.imag）	输出值实部（y.real）	输出值虚部（y.imag）
0	0	0	0
0	0	0	0
0	inf	0	inf
0	-inf	0	-inf
0	snan	0	nan
0	-snan	0	nan
0	qnan	0	nan
0	-qnan	0	nan
0	0	0	0

版权所有 © 华为技术有限公司

输入值实部（x.real）	输入值虚部（x.imag）	输出值实部（y.real）	输出值虚部（y.imag）
0	0	0	0
0	inf	0	inf
0	-inf	0	-inf
0	snan	0	nan
0	-snan	0	nan
0	qnan	0	nan
0	-qnan	0	nan
inf	0	pi/2	inf
inf	0	pi/2	-inf
inf	inf	pi/4	inf
inf	-inf	pi/4	-inf
inf	snan	nan	inf
inf	-snan	nan	inf
inf	qnan	nan	inf
inf	-qnan	nan	inf
-inf	0	-pi/2	inf
-inf	0	-pi/2	-inf
-inf	inf	-pi/4	inf
-inf	-inf	-pi/4	-inf
-inf	snan	nan	inf
-inf	-snan	nan	inf
-inf	qnan	nan	inf
-inf	-qnan	nan	inf
snan	0	nan	nan
snan	0	nan	nan
snan	inf	nan	inf
snan	-inf	nan	-inf
snan	snan	nan	nan
snan	-snan	nan	nan

版权所有 © 华为技术有限公司

输入值实部（x.real）	输入值虚部（x.imag）	输出值实部（y.real）	输出值虚部（y.imag）
snan	qnan	nan	nan
snan	-qnan	nan	nan
-snan	0	nan	nan
-snan	0	nan	nan
-snan	inf	nan	inf
-snan	-inf	nan	-inf
-snan	snan	nan	nan
-snan	-snan	nan	nan
-snan	qnan	nan	nan
-snan	-qnan	nan	nan
qnan	0	nan	nan
qnan	0	nan	nan
qnan	inf	nan	inf
qnan	-inf	nan	-inf
qnan	snan	nan	nan
qnan	-snan	nan	nan
qnan	qnan	nan	nan
qnan	-qnan	nan	nan
-qnan	0	nan	nan
-qnan	0	nan	nan
-qnan	inf	nan	inf
-qnan	-inf	nan	-inf
-qnan	snan	nan	nan
-qnan	-snan	nan	nan
-qnan	qnan	nan	nan
-qnan	-qnan	nan	nan

依赖

C: "kvml.h"

 版权所有 © 华为技术有限公司

示例

C interface：

```
int i, len = 4;
float src[4] = {0.0f, 2.0f, INFINITY, NAN};
float* dst = (float*)malloc(sizeof(float) * len);
if (dst == NULL) {
    printf("Malloc Failed!\n");
    return 0;
}
vsasin(len, src, dst);

/**
 *  Output dst:
 *     0.000000 nan nan nan
 */
```

6.4.2.3.59 v?tanpi

计算输入值乘以π的正切值，输入为向量。

接口定义

C interface：

void vstanpi(const int len, const float* src, float* dst);

void vdtanpi(const int len, const double* src, double* dst);

参数

参数名	类型	描述	输入/输出
len	整型数	表示输入向量的元素个数。 len≤0时会提示参数len无效并返回。	输入
src	• 在vstanpi中是单精度浮点类型。 • 在vdtanpi中是双精度浮点类型。	输入向量src，向量长度为len。 若为空指针，会提示空指针错误并返回。	输入
dst	• 在vstanpi中是单精度浮点类型。 • 在vdtanpi中是双精度浮点类型。	输出dst，向量长度为len。 若为空指针，会提示空指针错误并返回。	输出

输出结果

- 每一个运算值返回x乘以π的正切函数值y=tan(x * π), y ∈ (-INF, +INF)。
- 其他特殊值参考如下说明。

 版权所有 © 华为技术有限公司

输入值（src）	输出值（dst）
+0	+0
-0	-0
±inf	nan
nan	nan

依赖

C: "kvml.h"

示例

C interface：

```
int i, len = 4;
float src[4] = {0.0f, -0.0f, INFINITY, NAN};
float* dst = (float*)malloc(sizeof(float) * len);
if (dst == NULL) {
    printf("Malloc Failed!\n");
    return 0;
}
vstanpi(len, src, dst);

/**
 *  Output dst:
 *     0.000000 -0.000000 -nan -nan
 */
```

6.4.2.3.60 v?acospi

矢量计算输入值的反余弦值除以π。

接口定义

C interface：

void vsacospi(const int len, const float* src, float* dst);

void vdacospi(const int len, const double* src, double* dst);

参数

参数名	类型	描述	输入/输出
len	整型数	表示输入向量的元素个数。 len≤0时会提示参数len无效并返回。	输入

版权所有 © 华为技术有限公司

参数名	类型	描述	输入/输出
src	• 在vsacospi中是单精度浮点类型。 • 在vdacospi中是双精度浮点类型。	输入向量src，向量长度为len。 若为空指针，会提示空指针错误并返回。	输入
dst	• 在vsacospi中是单精度浮点类型。 • 在vdacospi中是双精度浮点类型。	输出dst，向量长度为len。 若为空指针，会提示空指针错误并返回。	输出

输出结果

- 每一个运算值返回x的反余弦函数值除以π的值，y ∈ [0, 1]。
- 其他特殊值参考如下说明。

输入值（src）	输出值（dst）
1	0
-1	1.0
|x| > 1	nan
±inf	nan
nan	nan

依赖

C: "kvml.h"

示例

C interface：

```
int i, len = 4;
float src[4] = {0.0f, 2.0f, INFINITY, NAN};
float* dst = (float*)malloc(sizeof(float) * len);
if (dst == NULL) {
    printf("Malloc Failed!\n");
    return 0;
}
vsacospi(len, src, dst);

printf("\n");
/**
 *  Output dst:
 *     0.500000 nan nan -nan
 */
```

 版权所有 © 华为技术有限公司

6.4.2.3.61 v?asinpi

矢量计算输入值的反正弦值除以π。

接口定义

C interface：

void vsasinpi(const int len, const float* src, float* dst);

void vdasinpi(const int len, const double* src, double* dst);

参数

参数名	类型	描述	输入/输出
len	整型数	表示输入向量的元素个数。 len≤0时会提示参数len无效并返回。	输入
src	• 在vsasinpi中是单精度浮点类型。 • 在vdasinpi中是双精度浮点类型。	输入向量src，向量长度为len。 若为空指针，会提示空指针错误并返回。	输入
dst	• 在vsasinpi中是单精度浮点类型。 • 在vdasinpi中是双精度浮点类型。	输出dst，向量长度为len。 若为空指针，会提示空指针错误并返回。	输出

输出结果

- 每一个运算值返回x的反正弦函数值除以π的值，y ∈ [-0.5, +0.5]。
- 其他特殊值参考如下说明。

输入值（src）	输出值（dst）
1	0.5
-1	-0.5
|x| > 1	nan
±inf	nan
nan	nan

依赖

C: "kvml.h"

 版权所有 © 华为技术有限公司

示例

C interface：

```
int i, len = 8;
float src[8] = {-1.0f, -0.5f, 0.0f, 0.5f, 1.0f, 2.0f, INFINITY, NAN};
float* dst = (float*)malloc(sizeof(float) * len);
if (dst == NULL) {
    printf("Malloc Failed!\n");
    return 0;
}
vsasinpi(len, src, dst);

/**
 *  Output dst:
 *     -0.500000 -0.166667 0.000000 0.166667 0.500000 nan nan nan
 */
```

6.4.2.3.62 v?tand

矢量计算输入角度的正切值，输入为向量。

接口定义

C interface：

void vstand(const int len, const float* src, float* dst);

void vdtand(const int len, const double* src, double* dst);

参数

参数名	类型	描述	输入/输出
len	整型数	表示输入向量的元素个数。 len≤0时会提示参数len无效并返回。	输入
src	• 在vstand中是单精度浮点类型。 • 在vdtand中是双精度浮点类型。	输入向量src，向量长度为len。 若为空指针，会提示空指针错误并返回。	输入
dst	• 在vstand中是单精度浮点类型。 • 在vdtand中是双精度浮点类型。	输出dst，向量长度为len。 若为空指针，会提示空指针错误并返回。	输出

输出结果

- 每个运算值返回角度x的正切函数值y，y ∈ (-INF, +INF)。
- 其他特殊值参考如下说明。

 版权所有 © 华为技术有限公司

输入值（src）	输出值（dst）
+0	+0
-0	-0
±inf	nan
nan	nan

依赖

C: "kvml.h"

示例

C interface：

```
int i, len = 8;
float src[8] = {-100.0f, -1.0f, -0.0f, 0.0f, 1.0f, 100.0f, INFINITY, NAN};
float* dst = (float*)malloc(sizeof(float) * len);
if (dst == NULL) {
    printf("Malloc Failed!\n");
    return 0;
}
vstand(len, src, dst);

/**
 *  Output dst:
 *     5.671282 -0.017455 -0.000000 0.000000 0.017455 -5.671282 nan nan
 */
```

6.4.2.3.63 v?erf

矢量计算输入值的误差函数值，输入为向量。

接口定义

C interface：

void vserf(const int len, const float* src, float* dst);

void vderf(const int len, const double* src, double* dst);

参数

参数名	类型	描述	输入/输出
len	整型数	表示输入向量的元素个数。 len≤0时会提示参数len无效并返回。	输入

版权所有 © 华为技术有限公司

参数名	类型	描述	输入/输出
src	• 在vserf中是单精度浮点类型。 • 在vderf中是双精度浮点类型。	输入向量src，向量长度为len。 若为空指针，会提示空指针错误并返回。	输入
dst	• 在vserf中是单精度浮点类型。 • 在vderf中是双精度浮点类型。	输出dst，向量长度为len。 若为空指针，会提示空指针错误并返回。	输出

输出结果

- 每个运算值返回x的误差函数值。
- 其他特殊值参考如下说明。

输入值（src）	输出值（dst）
x >= 0x1.ef31fap1(float)	+1.0
x <= -0x1.ef31fap1(float)	-1.0
+inf	+1.0
-inf	-1.0
nan	nan

依赖

C: "kvml.h"

示例

C interface：

```
int i, len = 8;
float src[8] = {-100.0f, -1.0f, -0.0f, 0.0f, 1.0f, 100.0f, INFINITY, NAN};
float* dst = (float*)malloc(sizeof(float) * len);
if (dst == NULL) {
    printf("Malloc Failed!\n");
    return 0;
}
vserf(len, src, dst);

/**
 *  Output dst:
 *     -1.000000 -0.842701 -0.000000 0.000000 0.842701 1.000000 1.000000 nan
 */
```

6.4.2.3.64 v?erfc

矢量计算输入值的互补误差函数值，输入为向量。

 版权所有 © 华为技术有限公司

接口定义

C interface：

void vserfc(const int len, const float* src, float* dst);

void vderfc(const int len, const double* src, double* dst);

参数

参数名	类型	描述	输入/输出
len	整型数	表示输入向量的元素个数。 len≤0时会提示参数len无效并返回。	输入
src	• 在vserfc中是单精度浮点类型。 • 在vderfc中是双精度浮点类型。	输入向量src，向量长度为len。 若为空指针，会提示空指针错误并返回。	输入
dst	• 在vserfc中是单精度浮点类型。 • 在vderfc中是双精度浮点类型。	输出dst，向量长度为len。 若为空指针，会提示空指针错误并返回。	输出

输出结果

- 每个输出值返回x的互补误差函数值。
- 其他特殊值参考如下说明。

输入值（src）	输出值（dst）
x > 0x1.42p3 (float)	+0
x > 0x1.b3ep4 (double)	+0
x < -0x1.42p3 (float)	+2
x < -0x1.b3ep4 (double)	+2
+inf	+0
-inf	+2
nan	nan

依赖

C: "kvml.h"

 版权所有 © 华为技术有限公司

示例

C interface：

```
int i, len = 9;
float src[9] = {-100.0f, -1.0f, -0.0f, 0.0f, 1.0f, 100.0f, -INFINITY, INFINITY, NAN};
float* dst = (float*)malloc(sizeof(float) * len);
if (dst == NULL) {
    printf("Malloc Failed!\n");
    return 0;
}
vserfc(len, src, dst);

/**
 *  Output dst:
 *     2.000000 1.842701 1.000000 1.000000 0.157299 0.000000 2.000000 0.000000 nan
 */
```

6.4.2.3.65 v?cdfnorm

矢量计算输入值的正态分布累积函数值，输入为向量。

接口定义

C interface：

void vscdfnorm(const int len, const float* src, float* dst);

void vdcdfnorm(const int len, const double* src, double* dst);

参数

参数名	类型	描述	输入/输出
len	整型数	表示输入向量的元素个数。 len≤0时会提示参数len无效并返回。	输入
src	• 在vscdfnorm中是单精度浮点类型。 • 在vdcdfnorm中是双精度浮点类型。	输入向量src，向量长度为len。 若为空指针，会提示空指针错误并返回。	输入
dst	• 在vscdfnorm中是单精度浮点类型。 • 在vdcdfnorm中是双精度浮点类型。	输出dst，向量长度为len。 若为空指针，会提示空指针错误并返回。	输出

输出结果

- 每个运算值返回x的正态分布累积函数值。
- 其他特殊值参考如下说明。

 版权所有 © 华为技术有限公司

输入值（src）	输出值（dst）
±0	+0.5
+inf	+1
-inf	+0
nan	nan

依赖

C: "kvml.h"

示例

C interface：

```
int i, len = 9;
float src[9] = {-100.0f, -1.0f, -0.0f, 0.0f, 1.0f, 100.0f, -INFINITY, INFINITY, NAN};
float* dst = (float*)malloc(sizeof(float) * len);
if (dst == NULL) {
    printf("Malloc Failed!\n");
    return 0;
}
vscdfnorm(len, src, dst);

/**
 *  Output dst:
 *     0.000000 0.158655 0.500000 0.500000 0.841345 1.000000 0.000000 1.000000 nan
 */
```

6.4.2.3.66 v?erfinv

矢量计算输入值的误差函数逆函数值，输入为向量。

接口定义

C interface：

void vserfinv(const int len, const float* src, float* dst);

void vderfinv(const int len, const double* src, double* dst);

参数

参数名	类型	描述	输入/输出
len	整型数	表示输入向量的元素个数。 len≤0时会提示参数len无效并返回。	输入

版权所有 © 华为技术有限公司

参数名	类型	描述	输入/输出
src	• 在vserfinv中是单精度浮点类型。 • 在vderfinv中是双精度浮点类型。	输入向量src，向量长度为len。 若为空指针，会提示空指针错误并返回。	输入
dst	• 在vserfinv中是单精度浮点类型。 • 在vderfinv中是双精度浮点类型。	输出dst，向量长度为len。 若为空指针，会提示空指针错误并返回。	输出

输出结果

- 每个运算值返回x的误差函数逆函数值。
- 其他特殊值参考如下说明。

输入值（src）	输出值（dst）
+0	+0
-0	-0
+1	+inf
-1	-inf
|x| > 1	nan

依赖

C: "kvml.h"

示例

C interface：

```
int i, len = 9;
float src[9] = {-100.0f, -1.0f, -0.0f, 0.0f, 1.0f, 100.0f, -INFINITY, INFINITY, NAN};
float* dst = (float*)malloc(sizeof(float) * len);
if (dst == NULL) {
    printf("Malloc Failed!\n");
    return 0;
}
vserfinv(len, src, dst);

/**
 *  Output dst:
 *     nan -inf -0.000000 0.000000 inf nan nan nan nan
 */
```

6.4.2.3.67 v?erfcinv

矢量计算输入值的互补误差函数逆函数值，输入为向量。

 版权所有 © 华为技术有限公司

接口定义

C interface：

void vserfcinv(const int len, const float* src, float* dst);

void vderfcinv(const int len, const double* src, double* dst);

参数

参数名	类型	描述	输入/输出
len	整型数	表示输入向量的元素个数。 len≤0时会提示参数len无效并返回。	输入
src	• 在vserfcinv中是单精度浮点类型。 • 在vderfcinv中是双精度浮点类型。	输入向量src，向量长度为len。 若为空指针，会提示空指针错误并返回。	输入
dst	• 在vserfcinv中是单精度浮点类型。 • 在vderfcinv中是双精度浮点类型。	输出dst，向量长度为len。 若为空指针，会提示空指针错误并返回。	输出

输出结果

- 每个运算值返回x的互补误差函数逆函数值。
- 其他特殊值参考如下说明。

输入值（src）	输出值（dst）
2	-inf
>2	nan
-0	+inf
0 <= x < 0x1p-52	+inf
< 0	nan
±inf	nan
nan	nan

依赖

C: "kvml.h"

 版权所有 © 华为技术有限公司

示例

C interface：

```
int i, len = 9;
float src[9] = {-100.0f, -1.0f, -0.0f, 0.0f, 1.0f, 100.0f, -INFINITY, INFINITY, NAN};
float* dst = (float*)malloc(sizeof(float) * len);
if (dst == NULL) {
    printf("Malloc Failed!\n");
    return 0;
}
vserfcinv(len, src, dst);

/**
 *  Output dst:
 *     nan nan inf inf 0.000000 nan nan nan nan
 */
```

6.4.2.3.68 v?cdfnorminv

矢量计算输入值的正态分布累积函数的逆函数值，输入为向量。

接口定义

C interface：

void vscdfnorminv(const int len, const float* src, float* dst);

void vdcdfnorminv(const int len, const double* src, double* dst);

参数

参数名	类型	描述	输入/输出
len	整型数	表示输入向量的元素个数。 len≤0时会提示参数len无效并返回。	输入
src	• 在vscdfnorminv中是单精度浮点类型。 • 在vdcdfnorminv中是双精度浮点类型。	输入向量src，向量长度为len。 若为空指针，会提示空指针错误并返回。	输入
dst	• 在vscdfnorminv中是单精度浮点类型。 • 在vdcdfnorminv中是双精度浮点类型。	输出dst，向量长度为len。 若为空指针，会提示空指针错误并返回。	输出

输出结果

- 每个运算值返回x的正态分布累积函数的逆函数值。
- 其他特殊值参考如下说明。

 版权所有 © 华为技术有限公司

输入值（src）	输出值（dst）
+1	+inf
+0.5	+0
±0	-inf
±inf	nan
nan	nan

依赖

C: "kvml.h"

示例

C interface：

```
int i, len = 11;
float src[11] = {-100.0f, -1.0f, -0.0f, 0.0f, 0.3f, 0.5f, 1.0f, 100.0f, -INFINITY, INFINITY, NAN};
float* dst = (float*)malloc(sizeof(float) * len);
if (dst == NULL) {
    printf("Malloc Failed!\n");
    return 0;
}
vscdfnorminv(len, src, dst);

/**
 *  Output dst:
 *     nan nan -inf -inf -0.524400 0.000000 inf nan nan nan nan
 */
```

6.4.2.3.69 v?lgamma

矢量计算输入值的伽马值绝对值的自然对数值，输入为向量。

接口定义

C interface：

void vslgamma(const int len, const float* src, float* dst);

void vdlgamma(const int len, const double* src, double* dst);

参数

参数名	类型	描述	输入/输出
len	整型数	表示输入向量的元素个数。 len≤0时会提示参数len无效并返回。	输入

版权所有 © 华为技术有限公司

参数名	类型	描述	输入/输出
src	• 在vslgamma中是单精度浮点类型。 • 在vdlgamma中是双精度浮点类型。	输入向量src，向量长度为len。 若为空指针，会提示空指针错误并返回。	输入
dst	• 在vslgamma中是单精度浮点类型。 • 在vdlgamma中是双精度浮点类型。	输出dst，向量长度为len。 若为空指针，会提示空指针错误并返回。	输出

输出结果

- 每个运算值返回x的伽马值绝对值的自然对数值。
- 其他特殊值参考如下说明。

输入值（src）	输出值（dst）
±0	+inf
±inf	+inf
nan	nan

依赖

C: "kvml.h"

示例

C interface：

```
int i, len = 11;
float src[11] = {-100.0f, -1.0f, -0.0f, 0.0f, 0.3f, 0.5f, 1.0f, 100.0f, -INFINITY, INFINITY, NAN};
float* dst = (float*)malloc(sizeof(float) * len);
if (dst == NULL) {
    printf("Malloc Failed!\n");
    return 0;
}
vslgamma(len, src, dst);

/**
 *  Output dst:
 *     inf inf inf inf 1.095798 0.572365 0.000000 359.134216 inf inf nan
 */
```

6.4.2.3.70 v?tgamma

矢量计算输入数据的伽马值，输入为向量。

版权所有 © 华为技术有限公司

接口定义

C interface：

void vstgamma(const int len, const float* src, float* dst);

void vdtgamma(const int len, const double* src, double* dst);

参数

参数名	类型	描述	输入/输出
len	整型数	表示输入向量的元素个数。 len≤0时会提示参数len无效并返回。	输入
src	• 在vstgamma中是单精度浮点类型。 • 在vdtgamma中是双精度浮点类型。	输入向量src，向量长度为len。 若为空指针，会提示空指针错误并返回。	输入
dst	• 在vstgamma中是单精度浮点类型。 • 在vdtgamma中是双精度浮点类型。	输出dst，向量长度为len。 若为空指针，会提示空指针错误并返回。	输出

输出结果

- 每个运算值返回x的伽马值。
- 其他特殊值参考如下说明。

输入值（src）	输出值（dst）
±0	±inf
+inf	+inf
-inf	nan
nan	nan

依赖

C: "kvml.h"

示例

C interface：

```
int i, len = 11;
float src[11] = {-100.0f, -1.0f, -0.0f, 0.0f, 0.3f, 0.5f, 1.0f, 100.0f, -INFINITY, INFINITY, NAN};
```

 版权所有 © 华为技术有限公司

```
float* dst = (float*)malloc(sizeof(float) * len);
if (dst == NULL) {
    printf("Malloc Failed!\n");
    return 0;
}
vstgamma(len, src, dst);

/**
 *  Output dst:
 *     nan nan -inf inf 2.991569 1.772454 1.000000 inf nan inf nan
 */
```

6.4.2.3.71 v?expint1

矢量计算指数积分值，输入为向量。

接口定义

C interface：

void vsexpint1(const int len, const float* src, float* dst);

void vdexpint1(const int len, const double* src, double* dst);

参数

参数名	类型	描述	输入/输出
len	整型数	表示输入向量的元素个数。 len≤0时会提示参数len无效并返回。	输入
src	• 在vsexpint1中是单精度浮点类型。 • 在vdexpint1中是双精度浮点类型。	输入向量src，向量长度为len。 若为空指针，会提示空指针错误并返回。	输入
dst	• 在vsexpint1中是单精度浮点类型。 • 在vdexpint1中是双精度浮点类型。	输出dst，向量长度为len。 若为空指针，会提示空指针错误并返回。	输出

输出结果

- 每个运算值返回x的指数积分值。
- 其他特殊值参考如下说明。

输入值（src）	输出值（dst）
±0	inf
>=0x1p13	+0
<0	nan

版权所有 © 华为技术有限公司

输入值（src）	输出值（dst）
nan	nan

依赖

C: "kvml.h"

示例

C interface：

```
int i, len = 11;
float src[11] = {-100.0f, -1.0f, -0.0f, 0.0f, 0.3f, 0.5f, 1.0f, 100.0f, -INFINITY, INFINITY, NAN};
float* dst = (float*)malloc(sizeof(float) * len);
if (dst == NULL) {
    printf("Malloc Failed!\n");
    return 0;
}
vsexpint1(len, src, dst);

/**
 *  Output dst:
 *     nan nan inf inf 0.905677 0.559774 0.219384 0.000000 nan 0.000000 nan
 */
```

6.4.2.3.72 v?floor

矢量计算不大于输入值的最大整数值，输入为向量。

接口定义

C interface：

void vsfloor(const int len, const float* src, float* dst);

void vdfloor(const int len, const double* src, double* dst);

参数

参数名	类型	描述	输入/输出
len	整型数	表示输入向量的元素个数。 len≤0时会提示参数len无效并返回。	输入
src	• 在vsfloor中是单精度浮点类型。 • 在vdfloor中是双精度浮点类型。	输入向量src，向量长度为len。 若为空指针，会提示空指针错误并返回。	输入

版权所有 © 华为技术有限公司

参数名	类型	描述	输入/输出
dst	• 在vsfloor中是单精度浮点类型。 • 在vdfloor中是双精度浮点类型。	输出dst，向量长度为len。 若为空指针，会提示空指针错误并返回。	输出

输出结果

- 每个运算值返回x的不大于输入值的最大整数值。
- 其他特殊值参考如下说明。

输入值（src）	输出值（dst）
±0	±0
±inf	±inf
nan	nan

依赖

C: "kvml.h"

示例

C interface：

```
int i, len = 11;
float src[11] = {-100.0f, -1.0f, -0.0f, 0.0f, 0.3f, 0.5f, 1.0f, 100.0f, -INFINITY, INFINITY, NAN};
float* dst = (float*)malloc(sizeof(float) * len);
if (dst == NULL) {
    printf("Malloc Failed!\n");
    return 0;
}
vsfloor(len, src, dst);

/**
 *  Output dst:
 *     -100.000000 -1.000000 -0.000000 0.000000 0.000000 0.000000 1.000000 100.000000 -inf inf nan
 */
```

6.4.2.3.73 v?ceil

矢量计算不小于输入值的最小整数值，输入为向量。

接口定义

C interface：

void vsceil(const int len, const float* src, float* dst);

void vdceil(const int len, const double* src, double* dst);

 版权所有 © 华为技术有限公司

参数

参数名	类型	描述	输入/输出
len	整型数	表示输入向量的元素个数。 len≤0时会提示参数len无效并返回。	输入
src	• 在vsceil中是单精度浮点类型。 • 在vdceil中是双精度浮点类型。	输入向量src，向量长度为len。 若为空指针，会提示空指针错误并返回。	输入
dst	• 在vsceil中是单精度浮点类型。 • 在vdceil中是双精度浮点类型。	输出dst，向量长度为len。 若为空指针，会提示空指针错误并返回。	输出

输出结果

- 每个运算值返回x的不小于输入值的最小整数值。
- 其他特殊值参考如下说明。

输入值（src）	输出值（dst）
±0	±0
±inf	±inf
nan	nan

依赖

C: "kvml.h"

示例

C interface：

```
int i, len = 11;
float src[11] = {-100.0f, -1.0f, -0.0f, 0.0f, 0.3f, 0.5f, 1.0f, 100.0f, -INFINITY, INFINITY, NAN};
float* dst = (float*)malloc(sizeof(float) * len);
if (dst == NULL) {
    printf("Malloc Failed!\n");
    return 0;
}
vsceil(len, src, dst);

/**
 *  Output dst:
 *     -100.000000 -1.000000 -0.000000 0.000000 1.000000 1.000000 1.000000 100.000000 -inf inf nan
 */
```

 版权所有 © 华为技术有限公司

6.4.2.3.74 v?modf

矢量计算输入值的整数和小数部分，输入为向量。

接口定义

C interface：

void vsmodf(const int len, const float* src, float* dst1, float* dst2);

void vdmodf(const int len, const double* src, double* dst1, double* dst2);

参数

参数名	类型	描述	输入/输出
len	整型数	表示输入向量的元素个数。 len≤0时会提示参数len无效并返回。	输入
src	• 在vsmodf中是单精度浮点类型。 • 在vdmodf中是双精度浮点类型。	输入向量src，向量长度为len。 若为空指针，会提示空指针错误并返回。	输入
dst1	• 在vsmodf中是单精度浮点类型。 • 在vdmodf中是双精度浮点类型。	输出dst1，向量长度为len。 若为空指针，会提示空指针错误并返回。	输出
dst2	• 在vsmodf中是单精度浮点类型。 • 在vdmodf中是双精度浮点类型。	输出dst2，向量长度为len。 若为空指针，会提示空指针错误并返回。	输出

输出结果

- 每个运算值返回x的整数和小数部分，分别保存在dst1和dst2指向的内存中。
- 其他特殊值参考如下说明。

输入值（src）	输出值（dst1）	输出值（dst2）
±0	±0	±0
+inf	+inf	+0
-inf	-inf	-0
nan	nan	nan

 版权所有 © 华为技术有限公司

依赖

C: "kvml.h"

示例

C interface：

```
int i, len = 8;
float src[8] = {-1.5f, -0.0f, 0.0f, 1.0f, 1.5f, -INFINITY, INFINITY, NAN};
float* dst1 = (float*)malloc(sizeof(float) * len);
if (dst1 == NULL) {
    printf("Malloc Failed!\n");
    return 0;
}
float* dst2 = (float*)malloc(sizeof(float) * len);
if (dst2 == NULL) {
    printf("Malloc Failed!\n");
    return 0;
}
vsmodf(len, src, dst1, dst2);

/**
 *  Output dst:
 *     (-1.000000 -0.500000)(-0.000000 -0.000000)(0.000000 0.000000)(1.000000 0.000000)(1.000000
0.500000)(-inf -0.000000)(inf 0.000000)(nan nan)
 */
```

6.4.2.3.75 v?copysign

符号拷贝函数，将src2中的符号位数据与src1中的非符号位数据结合，存储到dst，输入为向量。

接口定义

C interface：

void vscopysign(const int len, const float* src1, const float* src2, float* dst);

void vdcopysign(const int len, const double* src1, const double* src2, double* dst);

参数

参数名	类型	描述	输入/输出
len	整型数	表示输入向量的元素个数。 len≤0时会提示参数len无效并返回。	输入
src1	• 在vscopysign中是单精度浮点类型。 • 在vdcopysign中是双精度浮点类型。	输入向量src1，向量长度为len。 若为空指针，会提示空指针错误并返回。	输入

版权所有 © 华为技术有限公司

参数名	类型	描述	输入/输出
src2	• 在vscopysign中是单精度浮点类型。 • 在vdcopysign中是双精度浮点类型。	输入向量src2，向量长度为len。 若为空指针，会提示空指针错误并返回。	输入
dst	• 在vscopysign中是单精度浮点类型。 • 在vdcopysign中是双精度浮点类型。	输出dst，向量长度为len。 若为空指针，会提示空指针错误并返回。	输出

输出结果

- 每个运算值返回x2的符号位拷贝至x1得到的结果。
- 其他特殊值参考如下说明。

输入值（src1）	输入值（src2）的符号位	输出值（dst）
±inf	0	+inf
±inf	1	-inf
nan	0	nan
nan	1	-nan

依赖

C: "kvml.h"

示例

C interface：

```
int i, len = 8;
float src1[8] = {-1.5f, -0.0f, 0.0f, 1.0f, 1.5f, -INFINITY, INFINITY, NAN};
float src2[8] = {1.5f, 0.0f, +0.0f, -1.0f, -1.5f, +INFINITY, -INFINITY, -NAN};
float* dst = (float*)malloc(sizeof(float) * len);
if (dst == NULL) {
    printf("Malloc Failed!\n");
    return 0;
}
vscopysign(len, src1, src2, dst);

/**
 *  Output dst:
 *     1.500000 0.000000 0.000000 -1.000000 -1.500000 inf -inf -nan
 */
```

6.4.2.3.76 v?nextafter

在y方向上，返回x之后的下一个可表示值，输入为向量。

 版权所有 © 华为技术有限公司

接口定义

C interface：

void vsnextafter(const int len, const float* src1, const float* src2, float* dst);

void vdnextafter(const int len, const double* src1, const double* src2, double* dst);

参数

参数名	类型	描述	输入/输出
len	整型数	表示输入向量的元素个数。 len≤0时会提示参数len无效并返回。	输入
src1	• 在vsnextafter中是单精度浮点类型。 • 在vdnextafter中是双精度浮点类型。	输入向量src1，向量长度为len。 若为空指针，会提示空指针错误并返回。	输入
src2	• 在vsnextafter中是单精度浮点类型。 • 在vdnextafter中是双精度浮点类型。	输入向量src2，向量长度为len。 若为空指针，会提示空指针错误并返回。	输入
dst	• 在vsnextafter中是单精度浮点类型。 • 在vdnextafter中是双精度浮点类型。	输出dst，向量长度为len。 若为空指针，会提示空指针错误并返回。	输出

输出结果

- 每个运算值等于在y方向上返回x之后的下一个可表示值。
- 其他特殊值参考如下说明。

输入值（src1）	输入值（src2）	输出值（dst）
nan	!= nan	nan
!= nan	nan	nan
nan	nan	nan
+inf	-inf	0x7f7fffff(float)
+inf	-inf	0x7fefffffffffffff(double)
-inf	+inf	0xff7fffff(float)
-inf	+inf	0xffefffffffffffff(double)

版权所有 © 华为技术有限公司

输入值（src1）	输入值（src2）	输出值（dst）
+inf	+inf	+inf
-inf	-inf	-inf

依赖

C: "kvml.h"

示例

C interface：

```
int i, len = 8;
float src1[8] = {-1.5f, -0.0f, 0.0f, 1.0f, 1.5f, -INFINITY, INFINITY, NAN};
float src2[8] = {-1.1f, 0.0f, +0.0f, -1.0f, -1.5f, +INFINITY, -INFINITY, -NAN};
float* dst = (float*)malloc(sizeof(float) * len);
if (dst == NULL) {
    printf("Malloc Failed!\n");
    return 0;
}
vsnextafter(len, src1, src2, dst);

/**
 *  Output dst:
 *     -1.500000 0.000000 0.000000 1.000000 1.500000
-340282346638528859811704183484516925440.000000
340282346638528859811704183484516925440.000000 nan
 */
```

6.4.2.3.77 v?fdim

如果x>y，矢量计算x和y差值，否则返回0，输入为向量。

接口定义

C interface：

void vsfdim(const int len, const float* src1, const float* src2, float* dst);

void vdfdim(const int len, const double* src1, const double* src2, double* dst);

参数

参数名	类型	描述	输入/输出
len	整型数	表示输入向量的元素个数。 len≤0时会提示参数len无效并返回。	输入
src1	• 在vsfdim中是单精度浮点类型。 • 在vdfdim中是双精度浮点类型。	输入向量src1，向量长度为len。 若为空指针，会提示空指针错误并返回。	输入

版权所有 © 华为技术有限公司

参数名	类型	描述	输入/输出
src2	• 在vsfdim中是单精度浮点类型。 • 在vdfdim中是双精度浮点类型。	输入向量src2，向量长度为len。 若为空指针，会提示空指针错误并返回。	输入
dst	• 在vsfdim中是单精度浮点类型。 • 在vdfdim中是双精度浮点类型。	输出dst，向量长度为len。 若为空指针，会提示空指针错误并返回。	输出

输出结果

- 每个运算值返回：x > y，返回x - y的结果，否则返回0。
- 其他特殊值参考如下说明。

输入值（src1）	输入值（src2）	输出值（dst）
any	nan	nan
nan	any	nan

依赖

C: "kvml.h"

示例

C interface：

```
int i, len = 8;
float src1[8] = {-1.5f, -0.0f, 0.0f, 1.0f, 1.5f, -INFINITY, INFINITY, NAN};
float src2[8] = {-1.1f, 0.0f, +0.0f, -1.0f, -1.5f, +INFINITY, -INFINITY, -NAN};
float* dst = (float*)malloc(sizeof(float) * len);
if (dst == NULL) {
    printf("Malloc Failed!\n");
    return 0;
}
vsfdim(len, src1, src2, dst);

/**
 *  Output dst:
 *     0.000000 0.000000 0.000000 2.000000 3.000000 0.000000 inf nan
 */
```

6.4.2.3.78 v?fmax

矢量计算最大值函数，输入为向量。

接口定义

C interface：

 版权所有 © 华为技术有限公司

```
void vsfmax(const int len, const float* src1, const float* src2, float* dst);
void vdfmax(const int len, const double* src1, const double* src2, double* dst);
void vhfmax(const int len, const __fp16 *src1, const __fp16 *src2, __fp16 *dst);
```

参数

参数名	类型	描述	输入/输出
len	整型数	表示输入向量的元素个数。 len≤0时会提示参数len无效并返回。	输入
src1	• 在vsfmax中是单精度浮点类型。 • 在vdfmax中是双精度浮点类型。 • vhfmax中是半精度浮点类型。	输入向量src1，向量长度为len。 若为空指针，会提示空指针错误并返回。	输入
src2	• 在vsfmax中是单精度浮点类型。 • 在vdfmax中是双精度浮点类型。 • vhfmax中是半精度浮点类型。	输入向量src2，向量长度为len。 若为空指针，会提示空指针错误并返回。	输入
dst	• 在vsfmax中是单精度浮点类型。 • 在vdfmax中是双精度浮点类型。 • vhfmax中是半精度浮点类型。	输出dst，向量长度为len。 若为空指针，会提示空指针错误并返回。	输出

输出结果

- 每个运算值返回：x > y，返回x，否则返回y。
- 其他特殊值参考如下说明。

输入值（src1）	输入值（src2）	输出值（dst）
any	qnan	x
qnan	any	y
any	snan	nan
snan	any	nan

 版权所有 © 华为技术有限公司

依赖

C: "kvml.h"

示例

C interface：

```
int i, len = 8;
float src1[8] = {-1.5f, -0.0f, 0.0f, 1.0f, 1.5f, -INFINITY, INFINITY, NAN};
float src2[8] = {-1.1f, 0.0f, +0.0f, -1.0f, -1.5f, +INFINITY, -INFINITY, -NAN};
float* dst = (float*)malloc(sizeof(float) * len);
if (dst == NULL) {
    printf("Malloc Failed!\n");
    return 0;
}
vsfmax(len, src1, src2, dst);

/**
 *  Output dst:
 *     -1.100000 0.000000 0.000000 1.000000 1.500000 inf inf nan
 */
```

6.4.2.3.79 v?fmin

矢量计算最小值函数，输入为向量。

接口定义

C interface：

void vsfmin(const int len, const float* src1, const float* src2, float* dst);

void vdfmin(const int len, const double* src1, const double* src2, double* dst);

void vhfmin(const int len, const __fp16 *src1, const __fp16 *src2, __fp16 *dst);

参数

参数名	类型	描述	输入/输出
len	整型数	表示输入向量的元素个数。 len≤0时会提示参数len无效并返回。	输入
src1	• 在vsfmin中是单精度浮点类型。 • 在vdfmin中是双精度浮点类型。 • 在vhfmin中是半精度浮点类型。	输入向量src1，向量长度为len。 若为空指针，会提示空指针错误并返回。	输入

版权所有 © 华为技术有限公司

参数名	类型	描述	输入/输出
src2	• 在vsfmin中是单精度浮点类型。 • 在vdfmin中是双精度浮点类型。 • 在vhfmin中是半精度浮点类型。	输入向量src2，向量长度为len。 若为空指针，会提示空指针错误并返回。	输入
dst	• 在vsfmin中是单精度浮点类型。 • 在vdfmin中是双精度浮点类型。 • 在vhfmin中是半精度浮点类型。	输出dst，向量长度为len。 若为空指针，会提示空指针错误并返回。	输出

输出结果

- 每个运算值返回：若x > y，则返回y，否则返回x。
- 其他特殊值参考如下说明。

输入值（src1）	输入值（src2）	输出值（dst）
any	qnan	x
qnan	any	y
any	snan	nan
snan	any	nan

依赖

C: "kvml.h"

示例

C interface：

```
int i, len = 8;
float src1[8] = {-1.5f, -0.0f, 0.0f, 1.0f, 1.5f, -INFINITY, INFINITY, NAN};
float src2[8] = {-1.1f, 0.0f, +0.0f, -1.0f, -1.5f, +INFINITY, -INFINITY, -NAN};
float* dst = (float*)malloc(sizeof(float) * len);
if (dst == NULL) {
    printf("Malloc Failed!\n");
    return 0;
}
vsfmin(len, src1, src2, dst);

/**
 *  Output dst:
```

 版权所有 © 华为技术有限公司

```
*     -1.500000 -0.000000 0.000000 -1.000000 -1.500000 -inf -inf nan
*/
```

6.4.2.3.80 v?maxmag

矢量计算x和y中幅度更大的值，其中x、y为矢量数组。

接口定义

C interface：

void vsmaxmag(const int len, const float* src1, const float* src2, float* dst);

void vdmaxmag(const int len, const double* src1, const double* src2, double* dst);

参数

参数名	类型	描述	输入/输出
len	整型数	表示输入向量的元素个数。 len≤0时会提示参数len无效并返回。	输入
src1	• 在vsmaxmag中是单精度浮点类型。 • 在vdmaxmag中是双精度浮点类型。	输入向量src1，向量长度为len。 若为空指针，会提示空指针错误并返回。	输入
src2	• 在vsmaxmag中是单精度浮点类型。 • 在vdmaxmag中是双精度浮点类型。	输入向量src2，向量长度为len。 若为空指针，会提示空指针错误并返回。	输入
dst	• 在vsmaxmag中是单精度浮点类型。 • 在vdmaxmag中是双精度浮点类型。	输出dst，向量长度为len。 若为空指针，会提示空指针错误并返回。	输出

输出结果

- 每个运算值返回结果如下：
  如果|x| > |y|，返回x，否则返回y。
  如果|x| < |y|，返回y，否则返回x。
  否则返回fmax(x, y)。
- 其他特殊值参考如下说明。

输入值（src1）	输入值（src2）	输出值（dst）
!= nan	nan	x

版权所有 © 华为技术有限公司

输入值（src1）	输入值（src2）	输出值（dst）
nan	!= nan	y
nan	nan	nan

依赖

C: "kvml.h"

示例

C interface：

```
int i, len = 8;
float src1[8] = {-1.5f, -0.0f, 0.0f, 1.0f, 1.5f, -INFINITY, INFINITY, NAN};
float src2[8] = {-1.1f, 0.0f, +0.0f, -1.0f, -1.5f, +INFINITY, -INFINITY, -NAN};
float* dst = (float*)malloc(sizeof(float) * len);
if (dst == NULL) {
    printf("Malloc Failed!\n");
    return 0;
}
vsmaxmag(len, src1, src2, dst);

/**
 *  Output dst:
 *     -1.500000 0.000000 0.000000 1.000000 1.500000 inf inf nan
 */
```

6.4.2.3.81 v?minmag

矢量计算x和y中幅度更小的值，其中x、y为矢量数组。

接口定义

C interface：

void vsminmag(const int len, const float* src1, const float* src2, float* dst);

void vdminmag(const int len, const double* src1, const double* src2, double* dst);

参数

参数名	类型	描述	输入/输出
len	整型数	表示输入向量的元素个数。 len≤0时会提示参数len无效并返回。	输入
src1	• 在vsminmag中是单精度浮点类型。 • 在vdminmag中是双精度浮点类型。	输入向量src1，向量长度为len。 若为空指针，会提示空指针错误并返回。	输入

版权所有 © 华为技术有限公司

参数名	类型	描述	输入/输出
src2	• 在vsminmag中是单精度浮点类型。 • 在vdminmag中是双精度浮点类型。	输入向量src2，向量长度为len。 若为空指针，会提示空指针错误并返回。	输入
dst	• 在vsminmag中是单精度浮点类型。 • 在vdminmag中是双精度浮点类型。	输出dst，向量长度为len。 若为空指针，会提示空指针错误并返回。	输出

输出结果

- 每个运算值返回如下：
  如果|x| < |y|，返回x。
  如果|x| > |y|，返回y。
  如果|x| = |y|，返回fmin(x, y)。
- 其他特殊值参考如下说明。

输入值（src1）	输入值（src2）	输出值（dst）
!= nan	nan	x
nan	!= nan	y
nan	nan	nan

依赖

C: "kvml.h"

示例

C interface：

```
int i, len = 8;
float src1[8] = {-1.5f, -0.0f, 0.0f, 1.0f, 1.5f, -INFINITY, INFINITY, NAN};
float src2[8] = {-1.1f, 0.0f, +0.0f, -1.0f, -1.5f, +INFINITY, -INFINITY, -NAN};
float* dst = (float*)malloc(sizeof(float) * len);
if (dst == NULL) {
    printf("Malloc Failed!\n");
    return 0;
}
vsminmag(len, src1, src2, dst);

/**
 *  Output dst:
 *     -1.100000 -0.000000 0.000000 -1.000000 -1.500000 -inf -inf nan
 */
```

 版权所有 © 华为技术有限公司

6.4.2.3.82 vfloatcvtf16

将float数据类型转换为fp16。

接口定义

C interface：

void vfloatcvtf16(const int len, const float *src, __fp16 *dst);

参数

参数名	类型	描述	输入/输出
len	整型数	表示输入向量的元素个数。 len≤0时会提示参数len无效并返回。	输入
src	单精度浮点数	输入向量src，向量长度为len。 若为空指针，会提示空指针错误并返回。	输入
dst	fp16类型	输出dst，向量长度为len。 若为空指针，会提示空指针错误并返回。	输出

依赖

C: "kvml.h"

示例

C interface：

```
const int len = 8;
float src[8] = {1.0f, 2.0f, 3.0f, 4.0f, 5.0f, 6.0f, 7.0f, 8.0f};
__fp16 dst[8];

vfloatcvtf16(len, src, dst);


/**
 *  Output dst:
 *    1.000000 2.000000 3.000000 4.000000 5.000000 6.000000 7.000000 8.000000
 */
```

6.4.2.3.83 vf16cvtfloat

将fp16数据类型转换为float。

接口定义

C interface：

 版权所有 © 华为技术有限公司

void vf16cvtfloat(const int len, const __fp16 *src, float *dst);

参数

参数名	类型	描述	输入/输出
len	整型数	表示输入向量的元素个数。 len≤0时会提示参数len无效并返回。	输入
src	fp16类型	输入向量src，向量长度为len。 若为空指针，会提示空指针错误并返回。	输入
dst	单精度浮点数	输出dst，向量长度为len。 若为空指针，会提示空指针错误并返回。	输出

依赖

C: "kvml.h"

示例

C interface：

```
const int len = 8;
__fp16 src[8] = {1.0, 2.0, 3.0, 4.0, 5.0, 6.0, 7.0, 8.0}; // __fp16 数组
float dst[8]; // 用于存储转换后的 float 数组

vf16cvtfloat(len, src, dst);

/**
 *  Output dst:
 *     1.000000 2.000000 3.000000 4.000000 5.000000 6.000000 7.000000 8.000000
 */
```

6.4.2.3.84 KVMLGetVersion

获取VML产品版本信息。

接口定义

int KVMLGetVersion(KVMLVersion *ver)

参数

参数名	描述	取值范围	输入/输出
ver	产品信息结构体。	非空	输出

版权所有 © 华为技术有限公司

返回值

- 成功：返回1。
- 失败：返回0。

示例

```
#include <stdio.h>
#include "kvml.h"

int main()
{
    KVMLVersion vmlver;
    int status = KVMLGetVersion(&vmlver);
    if (status == 0) {
        return 0;
    }
    printf("Product Name: %s\n", vmlver.productName);
    printf("Product Version: %s\n", vmlver.productVersion);
    printf("Component Name: %s\n", vmlver.componentName);
    printf("Component Version: %s\n", vmlver.componentVersion);
    printf("Component AppendInfo: %s\n", vmlver.componentAppendInfo);
    printf("Software Name: %s\n", vmlver.softwareName);
    printf("Software Version: %s\n", vmlver.softwareVersion);
    return 0;
}
```

运行结果：

```
Product Name: Kunpeng HPCKit
Product Version: 24.0.RC1
Component Name: HPCKit-kml
Component Version: 2.3.0
Component AppendInfo: gcc
Software Name: hpckit-kml-vml
Software Version: 2.3.0
```

说明

以上版本号和编译时间以实际运行结果为准，上述结果仅供参考。

6.4.2.3.85 KmlVmlSetNumThreads

支持运行时设置线程数。

接口定义

void KmlVmlSetNumThreads(int nThreads)

参数

参数名	类型	描述	输入/输出
nThreads	整型数	设置的线程数，要求nThreads>0。	输入

依赖

#include "kvml.h"

 版权所有 © 华为技术有限公司

示例

```
KmlVmlSetNumThreads(4); // 设置运行线程数为4
```

6.4.2.3.86 KmlVmlSetNumThreadsLocal

支持运行时设置local线程数。

接口定义

int KmlVmlSetNumThreadsLocal(int nThreads)

参数

参数名	类型	描述	输入/输出
nThreads	整型数	设置的线程数，要求nThreads>0。	输入

返回值

使用此接口设置前的local线程数。

依赖

#include "kvml.h"

示例

```
KmlVmlSetNumThreadsLocal(4); // 设置运行线程数为4
```

6.4.2.3.87 KmlVmlGetMaxThreads

获取运行时的线程数。

接口定义

int KmlVmlGetMaxThreads(void)

返回值

返回运行时的线程数，最大值为CPU核数。

依赖

#include "kvml.h"

示例

```
int nthreads = KmlVmlGetMaxThreads(); // 获取运行时的线程数
```

6.4.3 KML_MATH 库函数说明

 版权所有 © 华为技术有限公司

6.4.3.1 函数说明

KML_MATH是C语言实现的基本数学函数库。

函数类	函数名	描述	高性能版本	高精度版本
三角函数（Trigonometric functions）	sin	正弦函数。	实数单精度/双精度	实数单精度/双精度/长双精度
	sind	正弦函数（入参为角度）。	实数单精度/双精度	实数单精度/双精度/长双精度
	sinpi	正弦函数（求入参乘以π的正弦值）。	实数单精度/双精度	无
	cos	余弦函数。	实数单精度/双精度	实数单精度/双精度/长双精度
	cosd	余弦函数（入参为角度）。	实数单精度/双精度	实数单精度/双精度/长双精度
	cospi	余弦函数（求入参乘以π的余弦值）。	实数单精度/双精度	无
	sincos	正弦、余弦函数。	实数单精度/双精度	实数单精度/双精度/长双精度
	sincosd	正弦余弦函数（入参为角度）。	实数单精度/双精度	实数单精度/双精度/长双精度
	tan	正切函数。	实数单精度/双精度	实数单精度/双精度/长双精度
	tand	正切函数（入参为角度）。	实数单精度/双精度	实数单精度/双精度/长双精度
	tanpi	正切函数（求入参乘以π的正切值）。	实数单精度/双精度	无
	asin	反正弦函数。	实数单精度/双精度	实数单精度/双精度/长双精度
	asind	反正弦函数（入参为角度）。	实数单精度/双精度	实数单精度/双精度/长双精度
	asinpi	反正弦函数/π。	实数单精度/双精度	无

版权所有 © 华为技术有限公司

函数类	函数名	描述	高性能版本	高精度版本
	acos	反余弦函数。	实数单精度/双精度	实数单精度/双精度/长双精度
	acosd	反余弦函数（入参为角度）。	实数单精度/双精度	实数单精度/双精度/长双精度
	acospi	反余弦函数/π。	实数单精度/双精度	无
	atan	反正切函数。	实数单精度/双精度	实数单精度/双精度/长双精度
	atand	反正切函数（出参为角度）。	实数单精度/双精度	无
	atanpi	反正切函数/π。	实数单精度/双精度	无
	atan2	反正切函数。	实数单精度/双精度	实数单精度/双精度/长双精度
	atan2d	反正切函数（入参为角度）。	实数单精度/双精度	实数单精度/双精度/长双精度
	atan2pi	反正切函数/π。	实数单精度/双精度	无
	cot	余切函数（入参为弧度）。	实数单精度/双精度	实数单精度/双精度/长双精度
	cotd	余切函数（入参为角度）。	实数单精度/双精度	实数单精度/双精度/长双精度
双曲函数（Hyperbolic functions）	sinh	双曲正弦函数。	实数单精度/双精度	实数单精度/双精度/长双精度
	cosh	双曲余弦函数。	实数单精度/双精度	实数单精度/双精度/长双精度
	sinhcosh	双曲正弦、余弦函数。	实数单精度/双精度	实数单精度/双精度
	tanh	双曲正切函数。	实数单精度/双精度	实数单精度/双精度/长双精度

版权所有 © 华为技术有限公司

函数类	函数名	描述	高性能版本	高精度版本
	asinh	反双曲正弦函数。	实数单精度/双精度	实数单精度/双精度/长双精度
	acosh	反双曲余弦函数。	实数单精度/双精度	实数单精度/双精度
	atanh	反双曲正切函数。	实数单精度/双精度	实数单精度/双精度/长双精度
指数和对数函数（Exponential and logarithmic functions）	exp	指数函数（base e）。	实数单精度/双精度	实数单精度/双精度/长双精度
	exp2	指数函数（base 2）。	实数单精度/双精度	实数单精度/双精度/长双精度
	exp10	指数函数（base 10）。	实数单精度/双精度	实数单精度/双精度/长双精度
	expm1	指数函数（e^x - 1）。	实数单精度/双精度	实数单精度/双精度/长双精度
	frexp	将x转换为归一化[1/2, 1)的有符号数乘以2的积分幂。	无	实数单精度/双精度/长双精度
	ldexp	指数函数（x * 2^exp）。	无	实数单精度/双精度/长双精度
	log	对数函数（base e）。	实数单精度/双精度	实数单精度/双精度/长双精度
	log2	对数函数（base 2）。	实数单精度/双精度	实数单精度/双精度/长双精度
	log10	对数函数（base 10）。	实数单精度/双精度	实数单精度/双精度/长双精度
	log1p	对数函数，即ln(1+x)的值。	实数单精度/双精度	实数单精度/双精度/长双精度
	logb	对数函数，求x的指数。	实数单精度/双精度	实数单精度/双精度/长双精度

版权所有 © 华为技术有限公司

函数类	函数名	描述	高性能版本	高精度版本
	ilogb	计算x的指数，返回整型结果。	实数单精度/双精度	实数单精度/双精度/长双精度
	scalb	求x*(2^y)，y为浮点类型。	实数单精度/双精度	实数单精度/双精度/长双精度
	scalbln	求x*(2^y)，y为长整型。	无	实数单精度/双精度/长双精度
	scalbn	求x*(2^y)，y为整型。	无	实数单精度/双精度/长双精度
	hypot	求(x^2 + y^2)的开方。	实数单精度/双精度	实数单精度/双精度/长双精度
幂和根函数（Power and root function）	pow	幂函数。	实数单精度/双精度	实数单精度/双精度/长双精度
	powr?i?	幂函数。	无	实数单精度/双精度/长双精度
	powi?i?	幂函数。	无	实数整型/长整型
	pow2o3	幂函数（幂值2/3）。	实数单精度/双精度	无
	pow3o2	幂函数（幂值3/2）。	实数单精度/双精度	无
	powr	幂函数（x>0）。	实数单精度/双精度	无
	sqrt	平方根函数。	实数单精度/双精度/长双精度	实数单精度/双精度/长双精度
	invsqrt	求平方根的倒数（$\frac{1}{\sqrt{x}}$）。	实数单精度/双精度	实数单精度/双精度/长双精度
	cbrt	立方根函数。	实数单精度/双精度	实数单精度/双精度/长双精度

版权所有 © 华为技术有限公司

函数类	函数名	描述	高性能版本	高精度版本
杂项函数（Miscellaneous Functions）	fabs	计算浮点数绝对值。	实数单精度/双精度	无
	fdim	如果x<=y，则fdim函数返回零值；否则返回x-y的值。	实数单精度/双精度	无
	fmax	返回入参的最大值。	实数单精度/双精度	无
	fmaxmag	返回绝对值相对较大的入参，否则行为类似于fmax(x,y)。	实数单精度/双精度	无
	fmin	返回入参的最小值。	实数单精度/双精度	无
	fminmag	返回绝对值相对较小的入参，否则行为类似于fmin(x,y)。	实数单精度/双精度	无
	nan	返回输入要求的浮点nan。	实数单精度/双精度	无
	nextafter	在y方向上，以指定格式返回x之后的下一个可表示值。	实数单精度/双精度	无
	nexttoward	返回x在y方向的下个浮点数。	实数单精度/双精度	无
	significand	求x的有效数字。	实数单精度/双精度	无
	finite	判断x是否为有限数。	实数单精度/双精度	无
特殊函数（Special Function）	annuity	求年金现值系数 $\frac{1-(1+x)^{-y}}{x}$ （ ）。	实数单精度/双精度	无
	compound	计算复利终值系数 $(1+x)^{y}$ 。	实数单精度/双精度	无
	erf	误差函数。	实数单精度/双精度	实数单精度/双精度/长双精度
	erfc	互补误差函数。	实数单精度/双精度	无

版权所有 © 华为技术有限公司

函数类	函数名	描述	高性能版本	高精度版本
	erfcx	互补误差函数与 e^{x^2} 的乘积。	实数单精度/双精度	无
	erfinv	逆误差函数。	实数单精度/双精度	无
	gamma	计算x的伽马值绝对值的自然对数。	实数单精度/双精度	实数单精度/双精度
	gamma_r	计算x的伽马值绝对值的自然对数，返回伽马值的符号。	实数单精度/双精度	实数双精度
	lgamma	计算x的伽马值绝对值的自然对数。	实数单精度/双精度	无
	lgamma_r	计算x的伽马值绝对值的自然对数，返回伽马值的符号。	实数单精度/双精度	无
	tgamma	计算x的伽马值。	实数单精度/双精度	实数单精度/双精度
	j0	Bessel 0阶第一类函数。	实数单精度/双精度	无
	j1	Bessel 1阶第一类函数。	实数单精度/双精度	无
	jn	Bessel n阶第一类函数。	实数单精度/双精度	无
	y0	Bessel 0阶第二类函数，x > 0。	实数单精度/双精度	无
	y1	Bessel 1阶第二类函数，x > 0。	实数单精度/双精度	无
	yn	Bessel n阶第二类函数，x > 0。	实数单精度/双精度	无
舍入函数（Nearest Integer Functions）	ceil	向上舍入。	实数单精度/双精度	实数单精度/双精度/长双精度
	floor	向下舍入。	实数单精度/双精度	实数单精度/双精度/长双精度
	llrint	基于当前舍入策略四舍五入。	实数单精度/双精度	实数单精度/双精度/长双精度

版权所有 © 华为技术有限公司

函数类	函数名	描述	高性能版本	高精度版本
	llround	四舍五入。	实数单精度/双精度	实数单精度/双精度/长双精度
	lrint	基于当前舍入策略四舍五入。	实数单精度/双精度	实数单精度/双精度/长双精度
	lround	四舍五入。	实数单精度/双精度	实数单精度/双精度/长双精度
	modf	获取浮点数的整数和小数部分。	无	实数单精度/双精度/长双精度
	nearbyint	四舍五入取偶。	实数单精度/双精度	实数单精度/双精度
	rint	基于当前舍入策略四舍五入。	实数单精度/双精度	实数单精度/双精度/长双精度
	round	四舍五入。	实数单精度/双精度	实数单精度/双精度/长双精度
	trunc	向0取整。	实数单精度/双精度	实数单精度/双精度/长双精度
取余函数（Remainder Functions）	drem	计算x / y的余数。	实数单精度/双精度	无
	fmod	求余数(x-ny)。	实数单精度/双精度	实数单精度/双精度/长双精度
	remainder	求余数（IEEE标准）。	实数单精度/双精度	实数单精度/双精度/长双精度
	remquo	求商和余数。	实数单精度/双精度	实数单精度/双精度/长双精度
复数函数（Complex Functions）	cabs	复数的绝对值。	实数单精度/双精度	复数单精度/双精度/长双精度
	cacos	复数形式的反余弦函数。	复数单精度/双精度	复数单精度/双精度
	cacosh	复数形式的反双曲余弦函数。	复数单精度/双精度	无

版权所有 © 华为技术有限公司

函数类	函数名	描述	高性能版本	高精度版本
	carg	返回复数的极坐标形式的角度值。	复数单精度/双精度	复数单精度/双精度/长双精度
	casin	复数形式的反正弦函数。	复数单精度/双精度	复数单精度/双精度
	casinh	复数形式的反双曲正弦函数。	复数单精度/双精度	复数单精度/双精度
	catan	复数形式的反正切函数。	复数单精度/双精度	复数单精度/双精度
	catanh	计算复数的双曲正切值。	复数单精度/双精度	复数单精度/双精度
	ccos	复数形式的余弦函数。	复数单精度/双精度	复数单精度/双精度
	ccosh	复数形式的双曲余弦函数。	复数单精度/双精度	复数单精度/双精度
	cdiv	复数除法。	无	复数单精度/双精度
	cexp	复数形式的指数函数（base e）。	复数单精度/双精度	复数单精度/双精度/长双精度
	cexp2	复数形式的指数函数（base 2）。	复数单精度/双精度	复数单精度/双精度/长双精度
	cexp10	复数形式的指数函数（base 10）。	复数单精度/双精度	无
	cimag	复数的虚部。	复数单精度/双精度	复数单精度/双精度/长双精度
	cis	复数的形式返回x的sin/cos值。	复数单精度/双精度	无
	cisd	复数的形式返回x的sin/cos值（入参为角度）。	复数单精度/双精度	无
	clog	复数形式的对数函数（base e）。	复数单精度/双精度	复数双精度/长双精度
	clog2	复数形式的对数函数（base 2）。	复数单精度/双精度	无
	clog10	复数形式的对数函数（base 10）。	复数单精度/双精度	复数单精度/双精度/长双精度

版权所有 © 华为技术有限公司

函数类	函数名	描述	高性能版本	高精度版本
	conj	复数z的共轭。	复数单精度/双精度	复数单精度/双精度/长双精度
	cpow	复数形式的幂函数。	复数单精度/双精度	复数单精度/双精度/长双精度
	cproj	复数在黎曼球面的投影。	复数单精度/双精度	复数单精度/双精度/长双精度
	creal	复数的实部。	复数单精度/双精度	复数单精度/双精度/长双精度
	csin	复数形式的正弦函数。	复数单精度/双精度	复数单精度/双精度
	csinh	复数形式的双曲正弦函数。	复数单精度/双精度	复数单精度/双精度
	csqrt	复数形式的平方根函数。	复数单精度/双精度	复数单精度/双精度/长双精度
	ctan	复数形式的正切函数。	复数单精度/双精度	复数单精度/双精度
	ctanh	复数形式的双曲正切函数。	复数单精度/双精度	无

6.4.3.2 函数定义

6.4.3.2.1 三角函数

6.4.3.2.1.1 sin

计算输入弧度的正弦值。

接口定义

C interface：

float sinf(float x);

double sin(double x);

long double sinl(long double x);

long double sinl_18(long double x);

版权所有 © 华为技术有限公司

说明

sinl_18仅在高精度版本中提供。

Fortran interface：

RES = SINF(X);

RES = SIN(X);

参数

参数名	类型	描述	输入/输出
x	• 在sinf中，x是单精度浮点类型。 • 在sin中，x是双精度浮点类型。 • 在sinl和sinl_18中，x是长双精度浮点类型。	表示弧度角的浮点值。	输入

返回值

- 返回弧度角x的正弦函数值y，y ∈ [-1, +1]。
- 输入+0，返回+0。
- 输入-0，返回-0。
- 输入±∞，返回nan。
- 输入nan，返回nan。

依赖

C: "km.h"

示例

C interface：

```
double pi = acos(-1);
// typical usage
double a = pi/6, b = 1.0, c = -3*pi/4, d = pi/3;
// special handling
double e = INFINITY, f = -INFINITY, g = NAN;
double complex k = 0 + 0 * I;
// print result
printf("sin(pi/6) = %.15f\n", sin(a));
printf("sin(1.0) = %.15f\n", sin(b));
printf("sin(-3*pi/4) = %.15f\n", sin(c));
printf("sin(pi/3) = %.15f\n", sin(d));
printf("sin(INFINITY) = %.15f\n", sin(e));
printf("sin(-INFINITY) = %.15f\n", sin(f));
printf("sin(NAN) = %.15f\n", sin(g));
 /*
 *  sin(pi/6) = 0.500000000000000
 *  sin(1.0) = 0.841470984807897
 *  sin(-3*pi/4) = -0.707106781186548
 *  sin(pi/3) = 0.866025403784438
```

 版权所有 © 华为技术有限公司

```
 *  sin(INFINITY) = nan
 *  sin(-INFINITY) = nan
 *  sin(NAN) = nan
 *
 * */
```

Fortran interface：

```
REAL(8) :: X = 1.0
PRINT*,  SIN(X)
!
! OUTPUT
!     0.841470984807897
!
```

6.4.3.2.1.2 sind

计算输入角度的正弦值。

接口定义

C interface：

float sindf(float x);

double sind(double x);

double sind_18(double x);

long double sindl(long double x);

说明

sind_18仅在高精度版本中提供。

参数

参数名	类型	描述	输入/输出
x	• 在sindf中，x是单精度浮点类型。 • 在sind和sind_18中，x是双精度浮点类型。 • 在sindl中，x是长双精度浮点类型。	表示角度的浮点值。	输入

返回值

- 返回角度x的正弦函数值y，y ∈ [-1, +1]。
- 输入+0，返回+0。
- 输入-0，返回-0。
- 输入±∞，返回nan。
- 输入nan，返回nan。

版权所有 © 华为技术有限公司

依赖

C: "km.h"

示例

C interface：

```
// typical usage
double x1 = 90.0, x2 = -90.0, x3 = 45.0, x4 = -45.0, x5 = 30.0, x6 = -30.0;
double a = 0.0, b = INFINITY, c = -INFINITY, d = NAN;
// print result
printf("sind(90.0) = %.15f\n", sind(x1));
printf("sind(-90.0) = %.15f\n", sind(x2));
printf("sind(45.0) = %.15f\n", sind(x3));
printf("sind(-45.0) = %.15f\n", sind(x4));
printf("sind(30.0) = %.15f\n", sind(x5));
printf("sind(-30.0) = %.15f\n", sind(x6));
printf("sind(0.0) = %.15f\n", sind(a));
printf("sind(-0.0) = %.15f\n", sind(-a));
printf("sind(INFINITY) = %.15f\n", sind(b));
printf("sind(-INFINITY) = %.15f\n", sind(c));
printf("sind(NAN) = %.15f\n", sind(d));
/*
 * sind(90.0) = 1.000000000000000
 * sind(-90.0) = -1.000000000000000
 * sind(45.0) = 0.707106781186548
 * sind(-45.0) = -0.707106781186548
 * sind(30.0) = 0.500000000000000
 * sind(-30.0) = -0.500000000000000
 * sind(0.0) = 0.000000000000000
 * sind(-0.0) = -0.000000000000000
 * sind(INFINITY) = nan
 * sind(-INFINITY) = nan
 * sind(NAN) = nan
 *
 * */
```

6.4.3.2.1.3 sinpi

计算输入实数与π相乘的正弦值。

接口定义

C interface：

float sinpif(float x);

double sinpi(double x);

参数

参数名	类型	描述	输入/输出
x	• 在sinpif中，x是单精度浮点类型。 • 在sinpi中，x是双精度浮点类型。	表示输入数据的浮点值。	输入

 版权所有 © 华为技术有限公司

返回值

- 返回x乘以π的正弦函数值y=sin(x * π)，y ∈ [-1, +1]。
- 输入+0，返回+0。
- 输入-0，返回-0。
- 输入±∞，返回nan。
- 输入nan，返回nan。

依赖

C: "km.h"

示例

C interface：

```
// print result
printf("sinpi(0.0) = %.15f\n", sinpi(0.0));
printf("sinpi(-0.0) = %.15f\n", sinpi(-0.0));
printf("sinpi(0.25) = %.15f\n", sinpi(0.25));
printf("sinpi(-0.25) = %.15f\n", sinpi(-0.25));
printf("sinpi(0.5) = %.15f\n", sinpi(0.5));
printf("sinpi(-0.5) = %.15f\n", sinpi(-0.5));
printf("sinpi(1.0) = %.15f\n", sinpi(1.0));
printf("sinpi(-1.0) = %.15f\n", sinpi(-1.0));
printf("sinpi(2.0) = %.15f\n", sinpi(2.0));
printf("sinpi(-2.0) = %.15f\n", sinpi(-2.0));
printf("sinpi(INFINITY) = %.15f\n", sinpi(INFINITY));
printf("sinpi(-INFINITY) = %.15f\n", sinpi(-INFINITY));
printf("sinpi(NAN) = %.15f\n", sinpi(NAN));
printf("sinpi(-NAN) = %.15f\n", sinpi(-NAN));
/*
 *
 * sinpi(0.0) = 0.000000000000000
 * sinpi(-0.0) = -0.000000000000000
 * sinpi(0.25) = 0.707106781186548
 * sinpi(-0.25) = -0.707106781186548
 * sinpi(0.5) = 1.000000000000000
 * sinpi(-0.5) = -1.000000000000000
 * sinpi(1.0) = 0.000000000000000
 * sinpi(-1.0) = -0.000000000000000
 * sinpi(2.0) = 0.000000000000000
 * sinpi(-2.0) = -0.000000000000000
 * sinpi(INFINITY) = nan
 * sinpi(-INFINITY) = nan
 * sinpi(NAN) = nan
 * sinpi(-NAN) = -nan
 *
 * */
```

6.4.3.2.1.4 asin

计算输入正弦值的反正弦值。

接口定义

C interface：

float asinf(float x);

double asin(double x);

long double asinl(long double x);

 版权所有 © 华为技术有限公司

Fortran interface：

RES = ASINF(X);

RES = ASIN(X);

参数

参数名	类型	描述	输入/输出
x	• 在asinf中，x是单精度浮点类型。 • 在asin中，x是双精度浮点类型。 • 在asinl中，x是长双精度浮点类型。	表示正弦值的浮点值。	输入

返回值

- 返回正弦值x的反正弦函数值y，x ∈ [-1, +1]。
- 输入+0，返回+0。
- 输入-0，返回-0。
- 输入x为±1，返回±π/2。
- 输入|x| > 1，返回nan。
- 输入±∞，返回nan。
- 输入nan，返回nan。

依赖

C: "km.h"

示例

C interface：

```
double pi = acos(-1);
// typical usage
double a = pi/6, b = 1.0, c = -3*pi/4, d = pi/3;
// special handling
double e = INFINITY, f = -INFINITY, g = NAN;
// print result
printf("asin(pi/6) = %.15f\n", asin(a));
printf("asin(1.0) = %.15f\n", asin(b));
printf("asin(-3*pi/4) = %.15f\n", asin(c));
printf("asin(pi/3) = %.15f\n", asin(d));
printf("asin(INFINITY) = %.15f\n", asin(e));
printf("asin(-INFINITY) = %.15f\n", asin(f));
printf("asin(NAN) = %.15f\n", asin(g));
/*
 *  asin(pi/6) = 0.551069583099446
 *  asin(1.0) = 1.570796326794897
 *  asin(-3*pi/4) = nan
 *  asin(pi/3) = nan
 *  asin(INFINITY) = nan
```


版权所有 © 华为技术有限公司

```
 *  asin(-INFINITY) = nan
 *  asin(NAN) = nan
 *
 * */
```

Fortran interface：

```
REAL(8) :: X = 1.0
PRINT*,  ASIN(X)
!
! OUTPUT
!     1.570796326794897
!
```

6.4.3.2.1.5 asind

计算输入实数的反正弦值对应的角度。

接口定义

C interface：

float asindf(float x);

double asind(double x);

double asind_18(double x);

long double asindl(long double x);

说明

asind_18仅在高精度版本中提供。

参数

参数名	类型	描述	输入/输出
x	在asindf中，x是单精度浮点类型。 在asind和asind_18中，x是双精度浮点类型。 在asindl中，x是长双精度浮点类型。	表示输入数据的浮点值。	输入

返回值

返回输入x的反正弦函数值对应的角度y=asin(x) * 180 / π，y ∈ [-90, +90]。

- 输入+0，返回+0。
- 输入-0，返回-0。
- 输入x为±1，返回±90。
- 输入|x| > 1，返回nan。
- 输入±∞，返回nan。

 版权所有 © 华为技术有限公司

- 输入nan，返回nan。

依赖

C: "km.h"

示例

C interface：

```
// typical usage
float x1 = -1.0, x2 = 1.0, x3 = 0.5, x4 = -0.5;
float a = 0.0, b = INFINITY, c = -INFINITY, d = NAN;
// print result
printf("asindf(1.0) = %.15f\n", asindf(x1));
printf("asindf(-1.0) = %.15f\n", asindf(x2));
printf("asindf(0.5) = %.15f\n", asindf(x3));
printf("asindf(-0.5) = %.15f\n", asindf(x4));
printf("asindf(0.0) = %.15f\n", asindf(a));
printf("asindf(-0.0) = %.15f\n", asindf(-a));
printf("asindf(INFINITY) = %.15f\n", asindf(b));
printf("asindf(-INFINITY) = %.15f\n", asindf(c));
printf("asindf(NAN) = %.15f\n", asindf(d));
/*
 * asindf(1.0) = -90.000000000000000
 * asindf(-1.0) = 90.000000000000000
 * asindf(0.5) = 30.000000000000000
 * asindf(-0.5) = -30.000000000000000
 * asindf(0.0) = 0.000000000000000
 * asindf(-0.0) = -0.000000000000000
 * asindf(INFINITY) = nan
 * asindf(-INFINITY) = nan
 * asindf(NAN) = nan
 *
 * */
```

6.4.3.2.1.6 asinpi

计算输入实数的反正弦值除以π。

接口定义

C interface：

float asinpif(float x);

double asinpi(double x);

参数

参数名	类型	描述	输入/输出
x	在asinpif中，x是单精度浮点类型。 在asinpi中，x是双精度浮点类型。	表示正弦值的浮点值。	输入

 版权所有 © 华为技术有限公司

返回值

- 返回输入实数x的反正弦函数值除以π的值y=asin(x) / π，y ∈ [-0.5, +0.5]。
- 输入+0，返回+0。
- 输入-0，返回-0。
- 输入x为±1，返回±0.5。
- 输入|x|>1，返回nan。
- 输入±∞，返回nan。
- 输入nan，返回nan。

依赖

C: "km.h"

示例

C interface：

```
// print result
printf("asinpi(0.0) = %.15f\n", asinpi(0.0));
printf("asinpi(-0.0) = %.15f\n", asinpi(-0.0));
printf("asinpi(0.25) = %.15f\n", asinpi(0.25));
printf("asinpi(-0.25) = %.15f\n", asinpi(-0.25));
printf("asinpi(0.5) = %.15f\n", asinpi(0.5));
printf("asinpi(-0.5) = %.15f\n", asinpi(-0.5));
printf("asinpi(1.0) = %.15f\n", asinpi(1.0));
printf("asinpi(-1.0) = %.15f\n", asinpi(-1.0));
printf("asinpi(INFINITY) = %.15f\n", asinpi(INFINITY));
printf("asinpi(-INFINITY) = %.15f\n", asinpi(-INFINITY));
printf("asinpi(NAN) = %.15f\n", asinpi(NAN));
printf("asinpi(-NAN) = %.15f\n", asinpi(-NAN));
/*
 * asinpi(0.0) = 0.000000000000000
 * asinpi(-0.0) = -0.000000000000000
 * asinpi(0.25) = 0.080430623255166
 * asinpi(-0.25) = -0.080430623255166
 * asinpi(0.5) = 0.166666666666667
 * asinpi(-0.5) = -0.166666666666667
 * asinpi(1.0) = 0.500000000000000
 * asinpi(-1.0) = -0.500000000000000
 * asinpi(INFINITY) = nan
 * asinpi(-INFINITY) = nan
 * asinpi(NAN) = nan
 * asinpi(-NAN) = -nan
 *
 * */
```

6.4.3.2.1.7 cos

计算输入弧度的余弦值。

接口定义

C interface：

float cosf(float x);

double cos(double x);

long double cosl(long double x);

 版权所有 © 华为技术有限公司

long double cosl_18(long double x);

说明

cosl_18仅在高精度版本中提供。

Fortran interface：

RES = COSF(X);

RES = COS(X);

参数

参数名	类型	描述	输入/输出
x	• 在cosf中，x是单精度浮点类型。 • 在cos中，x是双精度浮点类型。 • 在cosl和cosl_18中，x是长双精度浮点类型。	表示弧度角的浮点值。	输入

返回值

- 返回弧度角x的余弦函数值y，y ∈ [-1, +1]。
- 输入+0，返回+1。
- 输入-0，返回+1。
- 输入±∞，返回nan。
- 输入nan，返回nan。

依赖

C: "km.h"

示例

C interface：

```
double pi = acos(-1);
 // typical usage
 double a = pi/3, b = pi/2, c = -3*pi/4, d = 0.0;
 // special handling
 double e = INFINITY, f = -INFINITY, g = NAN;
 double complex k = 0 + 0 * I;
 // print result
 printf("cos(pi/3) = %.15f\n", cos(a));
 printf("cos(pi/2) = %.15f\n", cos(b));
 printf("cos(-3*pi/4) = %.15f\n", cos(c));
 printf("cos(0.0) = %.15f\n", cos(d));
 printf("cos(INFINITY) = %.15f\n", cos(e));
 printf("cos(-INFINITY) = %.15f\n", cos(f));
 printf("cos(NAN) = %.15f\n", cos(g));
 /*
  *  cos(pi/3) = 0.500000000000000
  *  cos(pi/2) = 0.000000000000000
```

 版权所有 © 华为技术有限公司

```
 *  cos(-3*pi/4) = -0.707106781186547
 *  cos(0.0) = 1.000000000000000
 *  cos(INFINITY) = nan
 *  cos(-INFINITY) = nan
 *  cos(NAN) = nan
 *
 * */
```

Fortran interface：

```
REAL(8) :: X = 0.0
PRINT*,  COS(X)
!
! OUTPUT
!     1.000000000000000
!
```

6.4.3.2.1.8 cosd

计算输入角度的余弦值。

接口定义

C interface：

float cosdf(float x);

double cosd(double x);

double cosd_18(double x);

long double cosdl(long double x);

说明

cosd_18仅在高精度版本中提供。

参数

参数名	类型	描述	输入/输出
x	• 在cosdf中，x是单精度浮点类型。 • 在cosd和cosd_18中，x是双精度浮点类型。 • 在cosdl中，x是长双精度浮点类型。	表示角度的浮点值。	输入

返回值

- 返回角度x的余弦函数值y，y ∈ [-1, +1]。
- 输入+0，返回1。
- 输入-0，返回1。
- 输入±∞，返回nan。
- 输入±nan，返回±nan。

 版权所有 © 华为技术有限公司

依赖

C: "km.h"

示例

C interface：

```
// typical usage
float x1 = 90.0, x2 = -90.0, x3 = 45.0, x4 = -45.0, x5 = 30.0, x6 = -30.0;
float a = 0.0, b = INFINITY, c = -INFINITY, d = NAN;
// print result
printf("cosd(90.0) = %.15f\n", cosd(x1));
printf("cosd(-90.0) = %.15f\n", cosd(x2));
printf("cosd(45.0) = %.15f\n", cosd(x3));
printf("cosd(-45.0) = %.15f\n", cosd(x4));
printf("cosd(30.0) = %.15f\n", cosd(x5));
printf("cosd(-30.0) = %.15f\n", cosd(x6));
printf("cosd(0.0) = %.15f\n", cosd(a));
printf("cosd(-0.0) = %.15f\n", cosd(-a));
printf("cosd(INFINITY) = %.15f\n", cosd(b));
printf("cosd(-INFINITY) = %.15f\n", cosd(c));
printf("cosd(NAN) = %.15f\n", cosd(d));
/*
 * cosd(90.0) = 0.000000000000000
 * cosd(-90.0) = 0.000000000000000
 * cosd(45.0) = 0.707106781186548
 * cosd(-45.0) = 0.707106781186548
 * cosd(30.0) = 0.866025403784439
 * cosd(-30.0) = 0.866025403784439
 * cosd(0.0) = 1.000000000000000
 * cosd(-0.0) = 1.000000000000000
 * cosd(INFINITY) = nan
 * cosd(-INFINITY) = nan
 * cosd(NAN) = nan
 *
 * */
```

6.4.3.2.1.9 cospi

计算输入实数与π相乘的余弦值。

接口定义

C interface：

float cospif(float x);

double cospi(double x);

参数

参数名	类型	描述	输入/输出
x	• 在cospif中，x是单精度浮点类型。 • 在cospi中，x是双精度浮点类型。	表示输入数据的浮点值。	输入

版权所有 © 华为技术有限公司

返回值

- 返回x乘以π的余弦函数值y=cos(x * π)，y ∈ [-1, +1]。
- 输入+0，返回1。
- 输入-0，返回1。
- 输入±∞，返回nan。
- 输入nan，返回nan。

依赖

C: "km.h"

示例

C interface：

```
// print result
printf("cospi(0.0) = %.15f\n", cospi(0.0));
printf("cospi(-0.0) = %.15f\n", cospi(-0.0));
printf("cospi(0.25) = %.15f\n", cospi(0.25));
printf("cospi(-0.25) = %.15f\n", cospi(-0.25));
printf("cospi(0.5) = %.15f\n", cospi(0.5));
printf("cospi(-0.5) = %.15f\n", cospi(-0.5));
printf("cospi(1.0) = %.15f\n", cospi(1.0));
printf("cospi(-1.0) = %.15f\n", cospi(-1.0));
printf("cospi(2.0) = %.15f\n", cospi(2.0));
printf("cospi(-2.0) = %.15f\n", cospi(-2.0));
printf("cospi(INFINITY) = %.15f\n", cospi(INFINITY));
printf("cospi(-INFINITY) = %.15f\n", cospi(-INFINITY));
printf("cospi(NAN) = %.15f\n", cospi(NAN));
printf("cospi(-NAN) = %.15f\n", cospi(-NAN));
/*
 * cospi(0.0) = 1.000000000000000
 * cospi(-0.0) = 1.000000000000000
 * cospi(0.25) = 0.707106781186548
 * cospi(-0.25) = 0.707106781186548
 * cospi(0.5) = 0.000000000000000
 * cospi(-0.5) = 0.000000000000000
 * cospi(1.0) = -1.000000000000000
 * cospi(-1.0) = -1.000000000000000
 * cospi(2.0) = 1.000000000000000
 * cospi(-2.0) = 1.000000000000000
 * cospi(INFINITY) = nan
 * cospi(-INFINITY) = nan
 * cospi(NAN) = nan
 * cospi(-NAN) = -nan
 *
 * */
```

6.4.3.2.1.10 acos

计算输入余弦值的反余弦值。

接口定义

C interface：

float acosf(float x);

double acos(double x);

long double acosl(long double x);

 版权所有 © 华为技术有限公司

long double acosl_18(long double x);

说明

acosl_18仅在高精度版本中提供。

Fortran interface：

RES = ACOSF(X);

RES = ACOS(X);

参数

参数名	类型	描述	输入/输出
x	• 在acosf中，x是单精度浮点类型。 • 在acos中，x是双精度浮点类型。 • 在acosl和acosl_18中，x是长双精度浮点类型。	表示余弦值的浮点值。	输入

返回值

- 返回余弦值x的反余弦函数值y，x ∈ [-1, +1]。
- 输入±0，返回+π/2。
- 输入+1，返回+0。
- 输入-1，返回+π。
- 输入|x| > 1，返回nan。
- 输入±∞，返回nan。
- 输入nan，返回nan。

依赖

C: "km.h"

示例

C interface：

```
double pi = acos(-1);
 // typical usage
 double a = pi/6, b = pi/9, c = -3*pi/4, d = 0.0;
 // special handling
 double e = INFINITY, f = -INFINITY, g = NAN;
 // print result
 printf("acos(pi/6) = %.15f\n", acos(a));
 printf("acos(pi/9) = %.15f\n", acos(b));
 printf("acos(-3*pi/4) = %.15f\n", acos(c));
 printf("acos(0.0) = %.15f\n", acos(d));
 printf("acos(INFINITY) = %.15f\n", acos(e));
 printf("acos(-INFINITY) = %.15f\n", acos(f));
```

 版权所有 © 华为技术有限公司

```
printf("acos(NAN) = %.15f\n", acos(g));
/*
 *  acos(pi/6) = 1.019726743695450
 *  acos(pi/9) = 1.214222261943226
 *  acos(-3*pi/4) = nan
 *  acos(0.0) = 1.570796326794897
 *  acos(INFINITY) = nan
 *  acos(-INFINITY) = nan
 *  acos(NAN) = nan
 *
 * */
```

Fortran interface：

```
REAL(8) :: X = 0.0
PRINT*,  ACOS(X)
!
! OUTPUT
!     1.570796326794897
!
```

6.4.3.2.1.11 acosd

计算输入实数的反余弦值对应的角度。

接口定义

C interface：

float acosdf(float x);

double acosd(double x);

long double acosdl(long double x);

参数

参数名	类型	描述	输入/输出
x	• 在acosdf中，x是单精度浮点类型。 • 在acosd中，x是双精度浮点类型。 • 在acosdl中，x是长双精度浮点类型。	表示输入数据的浮点值。	输入

返回值

- 返回输入x的反余弦函数值对应的角度y=acos(x) * 180 / π，y ∈ [0, 180]。
- 输入+0，返回90。
- 输入-0，返回90。
- 输入x为1，返回+0。
- 输入x为-1，返回180。
- 输入|x|>1，返回nan。

 版权所有 © 华为技术有限公司

- 输入±∞，返回nan。
- 输入nan，返回nan。

依赖

C： "km.h"

示例

C interface：

```
// typical usage
float x1 = -1.0, x2 = 1.0, x3 = 0.5, x4 = -0.5;
float a = 0.0, b = INFINITY, c = -INFINITY, d = NAN;
// print result
printf("acosdf(1.0) = %.15f\n", acosdf(x1));
printf("acosdf(-1.0) = %.15f\n", acosdf(x2));
printf("acosdf(0.5) = %.15f\n", acosdf(x3));
printf("acosdf(-0.5) = %.15f\n", acosdf(x4));
printf("acosdf(0.0) = %.15f\n", acosdf(a));
printf("acosdf(-0.0) = %.15f\n", acosdf(-a));
printf("acosdf(INFINITY) = %.15f\n", acosdf(b));
printf("acosdf(-INFINITY) = %.15f\n", acosdf(c));
printf("acosdf(NAN) = %.15f\n", acosdf(d));
/*
 * acosdf(1.0) = 180.000000000000000
 * acosdf(-1.0) = 0.000000000000000
 * acosdf(0.5) = 60.000000000000000
 * acosdf(-0.5) = 120.000000000000000
 * acosdf(0.0) = 90.000000000000000
 * acosdf(-0.0) = 90.000000000000000
 * acosdf(INFINITY) = nan
 * acosdf(-INFINITY) = nan
 * acosdf(NAN) = nan
 *
 * */
```

6.4.3.2.1.12 acospi

计算输入实数的反余弦值除以π。

接口定义

C interface：

float acospif(float x);

double acospi(double x);

参数

参数名	类型	描述	输入/输出
x	• 在acospif中，x是单精度浮点类型。 • 在acospi中，x是双精度浮点类型。	表示余弦值的浮点值。	输入

 版权所有 © 华为技术有限公司

返回值

- 返回输入实数x的反余弦函数值除以π的值y=acos(x) / π，y ∈ [0, 1]。
- 输入±0，返回0.5。
- 输入x为1，返回+0。
- 输入x为-1，返回1。
- 输入|x|>1，返回nan。
- 输入±∞，返回nan。
- 输入±nan，返回±nan。

依赖

C: "km.h"

示例

C interface：

```
// print result
printf("acospi(0.0) = %.15f\n", acospi(0.0));
printf("acospi(-0.0) = %.15f\n", acospi(-0.0));
printf("acospi(0.25) = %.15f\n", acospi(0.25));
printf("acospi(-0.25) = %.15f\n", acospi(-0.25));
printf("acospi(0.5) = %.15f\n", acospi(0.5));
printf("acospi(-0.5) = %.15f\n", acospi(-0.5));
printf("acospi(1.0) = %.15f\n", acospi(1.0));
printf("acospi(-1.0) = %.15f\n", acospi(-1.0));
printf("acospi(INFINITY) = %.15f\n", acospi(INFINITY));
printf("acospi(-INFINITY) = %.15f\n", acospi(-INFINITY));
printf("acospi(NAN) = %.15f\n", acospi(NAN));
printf("acospi(-NAN) = %.15f\n", acospi(-NAN));
/*
 * acospi(0.0) = 0.500000000000000
 * acospi(-0.0) = 0.500000000000000
 * acospi(0.25) = 0.419569376744834
 * acospi(-0.25) = 0.580430623255166
 * acospi(0.5) = 0.333333333333333
 * acospi(-0.5) = 0.666666666666667
 * acospi(1.0) = 0.000000000000000
 * acospi(-1.0) = 1.000000000000000
 * acospi(INFINITY) = nan
 * acospi(-INFINITY) = nan
 * acospi(NAN) = nan
 * acospi(-NAN) = -nan
 *
 * */
```

6.4.3.2.1.13 sincos

同时计算输入弧度的正弦值和余弦值。

接口定义

C interface：

void sincosf(float x, float* sinx, float* cosx);

void sincosf_18(float x, float* sinx, float* cosx);

void sincos(double x, double* sinx, double* cosx);

 版权所有 © 华为技术有限公司

void sincosl(long double x, long double* sinx, long double* cosx);

说明

sincosf_18仅在高精度版本中提供。

Fortran interface：

CALL SINCOSF(X, SINX, COSX);

CALL SINCOS(X, SINX, COSX);

参数

参数名	类型	描述	输入/输出
x	• 在sincosf和sincosf_18中，参数是单精度浮点类型。 • 在sincos中，参数是双精度浮点类型。 • 在sincosl中，参数是长双精度浮点类型。	表示弧度角的浮点值。	输入
sinx	• 在sincosf和sincosf_18中，参数是单精度浮点指针类型。 • 在sincos中，参数是双精度浮点指针类型。 • 在sincosl中，参数是长双精度浮点指针类型。	输出参数sinx指向的值。 若为空指针，会出现未定义的行为，接口不会对输出指针进行校验。	输出
cosx	• 在sincosf和sincosf_18中，参数是单精度浮点指针类型。 • 在sincos中，参数是双精度浮点指针类型。 • 在sincosl中，参数是长双精度浮点指针类型。	输出参数cosx指向的值。 若为空指针，会出现未定义的行为，接口不会对输出指针进行校验。	输出

返回值

- 返回弧度角x的正弦数值sinx和余弦函数值cosx，sinx ∈ [-1, +1]，cosx ∈ [-1, +1]。
- 输入+0，返回正弦值+0和余弦值+1。
- 输入-0，返回正弦值-0和余弦值+1。
- 输入±∞，均返回nan。
- 输入nan，均返回nan。

 版权所有 © 华为技术有限公司

依赖

#include "km.h"

示例

C interface：

```
double pi = acos(-1);
// typical usage
double a = pi/6, b = 1.0, c = -3*pi/4, d = pi/3;
// special handling
double e = INFINITY, f = -INFINITY, g = NAN;
// print result
double sinx, cosx;
sincos(a, &sinx, &cosx);
printf("sin(pi/6) = %.15f, cos(pi/6) = %.15f\n", sinx, cosx);
sincos(b, &sinx,&cosx);
printf("sin(1.0) = %.15f, cos(1.0) = %.15f\n", sinx, cosx);
sincos(c, &sinx,&cosx);
printf("sin(-3*pi/4) = %.15f, cos(-3*pi/4) = %.15f\n", sinx, cosx);
sincos(d, &sinx,&cosx);
printf("sin(pi/3) = %.15f, cos(pi/3) = %.15f\n", sinx, cosx);
sincos(e, &sinx,&cosx);
printf("sin(INFINITY) = %.15f, cos(INFINITY) = %.15f\n", sinx, cosx);
sincos(f, &sinx,&cosx);
printf("sin(-INFINITY) = %.15f, cos(-INFINITY) =  %.15f\n", sinx, cosx);
sincos(g, &sinx,&cosx);
printf("sin(NAN) = %.15f, cos(NAN) = %.15f\n", sinx, cosx);
/*
 *  sin(pi/6) = 0.500000000000000, cos(pi/6) = 0.866025403784439
 *  sin(1.0) = 0.841470984807897, cos(1.0) = 0.540302305868140
 *  sin(-3*pi/4) = -0.707106781186548, cos(-3*pi/4) = -0.707106781186547
 *  sin(pi/3) = 0.866025403784439, cos(pi/3) = 0.500000000000000
 *  sin(INFINITY) = -nan, cos(INFINITY) = nan
 *  sin(-INFINITY) = -nan, cos(-INFINITY) =  nan
 *  sin(NAN) = nan, cos(NAN) = -nan
 *
 * */
```

Fortran interface：

```
REAL(8) :: X = 1.0
REAL(8), TARGET :: SINX, COSX
TYPE(C_PTR) :: PSINX, PCOSX
PSINX = C_LOC(SINX)
PCOSX = C_LOC(COSX)
CALL SINCOS(X, PSINX, PCOSX)
PRINT*,  SINX, COSX
!
! OUTPUT
!     0.841470984807897 0.540302305868140
!
```

6.4.3.2.1.14 sincosd

同时计算输入角度的正弦值和余弦值。

接口定义

C interface：

void sincosdf(float x, float* sinx, float* cosx);

void sincosd(double x, double* sinx, double* cosx);

 版权所有 © 华为技术有限公司

void sincosdl(long double x, long double* sinx, long double* cosx);

参数

参数名	类型	描述	输入/输出
x	• 在sincosdf中，参数是单精度浮点类型。 • 在sincosd中，参数是双精度浮点类型。 • 在sincosdl中，参数是长双精度浮点类型。	表示角度的浮点值。	输入
sinx	• 在sincosdf中，参数是单精度浮点指针类型。 • 在sincosd中，参数是双精度浮点指针类型。 • 在sincosdl中，参数是长双精度浮点指针类型。	输出参数sinx指向的值。 若为空指针，会出现未定义的行为，接口不会对输出指针进行校验。	输出
cosx	• 在sincosdf中，参数是单精度浮点指针类型。 • 在sincosd中，参数是双精度浮点指针类型。 • 在sincosdl中，参数是长双精度浮点指针类型。	输出参数cosx指向的值。 若为空指针，会出现未定义的行为，接口不会对输出指针进行校验。	输出

返回值

- 返回角度x的正弦数值sinx和余弦函数值cosx，sinx ∈ [-1, +1]，cosx ∈ [-1, +1]。
- 输入+0，返回正弦值+0和余弦值+1。
- 输入-0，返回正弦值-0和余弦值+1。
- 输入±∞，均返回nan。
- 输入nan，均返回nan。

依赖

#include "km.h"

示例

C interface：

```
// typical usage
double x1 = INFINITY, sinx1, cosx1;
double x2 = 2.0, sinx2, cosx2;
double x3 = -2.5, sinx3, cosx3;
double x4 = NAN, sinx4, cosx4;
double x5 = 0.0, sinx5, cosx5;
```

 版权所有 © 华为技术有限公司

```
sincosd(x1, &sinx1, &cosx1);
sincosd(x2, &sinx2, &cosx2);
sincosd(x3, &sinx3, &cosx3);
sincosd(x4, &sinx4, &cosx4);
sincosd(x5, &sinx5, &cosx5);
// print result
printf("/*\n");
printf(" * sincosd(%.2f), sinval = %.6f cosval= %.6f\n", x1, sinx1, cosx1);
printf(" * sincosd(%.2f), sinval = %.6f cosval= %.6f\n", x2, sinx2, cosx2);
printf(" * sincosd(%.2f), sinval = %.6f cosval= %.6f\n", x3, sinx3, cosx3);
printf(" * sincosd(%.2f), sinval = %.6f cosval= %.6f\n", x4, sinx4, cosx4);
printf(" * sincosd(%.2f), sinval = %.6f cosval= %.6f\n", x5, sinx5, cosx5);
 /*
 * sincosd(inf), sinval = nan cosval= nan
 * sincosd(2.00), sinval = 0.034899 cosval= 0.999391
 * sincosd(-2.50), sinval = -0.043619 cosval= 0.999048
 * sincosd(nan), sinval = nan cosval= nan
 * sincosd(0.00), sinval = 0.000000 cosval= 1.000000
```

6.4.3.2.1.15 tan

计算输入弧度的正切值。

接口定义

C interface：

float tanf(float x);

double tan(double x);

long double tanl(long double x);

Fortran interface：

RES = TANF(X);

RES = TAN(X);

参数

参数名	类型	描述	输入/输出
x	• 在tanf中，x是单精度浮点类型。 • 在tan中，x是双精度浮点类型。 • 在tanl中，x是长双精度浮点类型。	表示弧度角的浮点值。	输入

返回值

- 返回弧度角x的正切函数值y，y ∈ (-INF, +INF)。
- 输入+0，返回+0。
- 输入-0，返回-0。

 版权所有 © 华为技术有限公司

- 输入±∞，返回nan。
- 输入nan，返回nan。

该函数在π(1/2 + n)处具有数学极点；没有通用的浮点表示能够精确地表示极点处的值，因此极点处会返回值，但不能认为是正确的。

依赖

C: "km.h"

示例

C interface：

```
double pi = acos(-1);
// typical usage
double a = pi/3, c = -3*pi/4, d = 0.0;
// special handling
double b = pi/2, e = INFINITY, f = -INFINITY, g = NAN;
double complex k = 0 + 0 * I;
// print result
printf("tan(pi/3) = %.15f\n", tan(a));
printf("tan(-3*pi/4) = %.15f\n", tan(c));
printf("tan(0.0) = %.15f\n", tan(d));
// mathematical poles
printf("tan(pi/2) = %.15f\n", tan(b));
printf("tan(INFINITY) = %.15f\n", tan(e));
printf("tan(-INFINITY) = %.15f\n", tan(f));
printf("tan(NAN) = %.15f\n", tan(g));
/*
 *  tan(pi/3) = 1.732050807568877
 *  tan(-3*pi/4) = 1.000000000000000
 *  tan(0.0) = 0.000000000000000
 *  tan(pi/2) = 16331239353195370.000000000000000
 *  tan(INFINITY) = nan
 *  tan(-INFINITY) = nan
 *  tan(NAN) = nan
 *
 * */
```

Fortran interface：

```
REAL(8) :: X = 0.0
PRINT*,  TAN(X)
!
! OUTPUT
!     0.000000000000000
!
```

6.4.3.2.1.16 tanpi

计算输入实数与π相乘的正切值。

接口定义

C interface：

float tanpif(float x);

double tanpi(double x);

 版权所有 © 华为技术有限公司

参数

参数名	类型	描述	输入/输出
x	• 在tanpif中，x是单精度浮点类型。 • 在tanpi中，x是双精度浮点类型。	表示输入数据的浮点值。	输入

返回值

- 返回x乘以π的正切函数值y=tan(x * π)。
- 输入+0，返回+0。
- 输入-0，返回-0。
- 输入±∞，返回nan。
- 输入nan，返回nan。

依赖

C: "km.h"

示例

C interface：

```
// typical usage
float x1 = -0.2, x2 = 0.2, x3 = 0.4, x4 = -0.4;
// special handling
float a = 0.0, b = INFINITY, c = -INFINITY, d = NAN;
// print result
printf("tanpi(0.2) = %.15f\n", tanpi(x1));
printf("tanpi(-0.2) = %.15f\n", tanpi(x2));
printf("tanpi(0.4) = %.15f\n", tanpi(x3));
printf("tanpi(-0.4) = %.15f\n", tanpi(x4));
printf("tanpi(0.0) = %.15f\n", tanpi(a));
printf("tanpi(-0.0) = %.15f\n", tanpi(-a));
printf("tanpi(INFINITY) = %.15f\n", tanpi(b));
printf("tanpi(-INFINITY) = %.15f\n", tanpi(c));
printf("tanpi(NAN) = %.15f\n", tanpi(d));
/*
 * tanpi(0.2) = -0.726542542310257
 * tanpi(-0.2) = 0.726542542310257
 * tanpi(0.4) = 3.077683733269691
 * tanpi(-0.4) = -3.077683733269691
 * tanpi(0.0) = 0.000000000000000
 * tanpi(-0.0) = -0.000000000000000
 * tanpi(INFINITY) = nan
 * tanpi(-INFINITY) = nan
 * tanpi(NAN) = nan
 * */
```

6.4.3.2.1.17 tand

计算输入角度的正切值。

 版权所有 © 华为技术有限公司

接口定义

C interface：

float tandf(float x);

double tand(double x);

long double tandl(long double x);

参数

参数名	类型	描述	输入/输出
x	• 在tandf中，x是单精度浮点类型。 • 在tand中，x是双精度浮点类型。 • 在tandl中，x是长双精度浮点类型。	表示角度的浮点值。	输入

返回值

- 正常区间：返回角度值x的正切值。
- 输入x = 180 * k，（k为整数），输出为±0。
- 输入x = 180 * k + 90，（k为整数），输出为±inf。
- 输入|x| = inf or nan，输出为nan。

依赖

C: "km.h"

示例

C interface：

```
// typical usage
float x1 = -1.0, x2 = 1.0, x3 = 0.5, x4 = -0.5;
// special handling
float a = 0.0, b = INFINITY, c = -INFINITY, d = NAN;
// print result
printf("tand(1.0) = %.15f\n", tand(x1));
printf("tand(-1.0) = %.15f\n", tand(x2));
printf("tand(0.5) = %.15f\n", tand(x3));
printf("tand(-0.5) = %.15f\n", tand(x4));
printf("tand(0.0) = %.15f\n", tand(a));
printf("tand(-0.0) = %.15f\n", tand(-a));
printf("tand(INFINITY) = %.15f\n", tand(b));
printf("tand(-INFINITY) = %.15f\n", tand(c));
printf("tand(NAN) = %.15f\n", tand(d));
/*
 *  tand(1.0) = 0.017455064928218
 *  tand(-1.0) = -0.017455064928218
 *  tand(0.5) = 0.008726867790759
 *  tand(-0.5) = -0.008726867790759
 *  tand(0.0) = 0.000000000000000
 *  tand(-0.0) = -0.000000000000000
```


版权所有 © 华为技术有限公司

```
*  tand(INFINITY) = nan
*  tand(-INFINITY) = nan
*  tand(NAN) = nan
*
* */
```

6.4.3.2.1.18 atan

计算输入正切值的反正切值。

接口定义

C interface：

float atanf(float x);

double atan(double x);

long double atanl(long double x);

Fortran interface：

RES = ATANF(X)

RES = ATAN(X)

参数

参数名	类型	描述	输入/输出
x	• 在atanf中，x是单精度浮点类型。 • 在atan中，x是双精度浮点类型。 • 在atanl中，x是长双精度浮点类型。	表示正切值的浮点值。	输入

返回值

- 返回正切值x的反正切函数值y，y的取值范围为(-π/2, +π/2)。
- 输入+0，返回+0。
- 输入-0，返回-0。
- 输入+∞，返回+π/2。
- 输入-∞，返回-π/2。
- 输入nan，返回nan。

依赖

C: "km.h"

示例

C interface：

 版权所有 © 华为技术有限公司

```
double pi = acos(-1);
// typical usage
double a = pi/3, c = -3*pi/4, b = pi/2;
// special handling
double d = +0.0, h = -0.0, e = INFINITY, f = -INFINITY, g = NAN;
double complex k = 0 + 0 * I;
// print result
printf("atan(pi/3) = %.15f\n", atan(a));
printf("atan(-3*pi/4) = %.15f\n", atan(c));
printf("atan(pi/2) = %.15f\n", atan(b));
printf("atan(+0.0) = %+.15f\n", atan(d));
printf("atan(-0.0) = %+.15f\n", atan(h));
printf("atan(INFINITY) = %+.15f\n", atan(e));
printf("atan(-INFINITY) = %+.15f\n", atan(f));
printf("atan(NAN) = %.15f\n", atan(g));
/*
 *  atan(pi/3) = 0.808448792630022
 *  atan(-3*pi/4) = -1.169422824815756
 *  atan(pi/2) = 1.003884821853887
 *  atan(+0.0) = +0.000000000000000
 *  atan(-0.0) = -0.000000000000000
 *  atan(INFINITY) = +1.570796326794897
 *  atan(-INFINITY) = -1.570796326794897
 *  atan(NAN) = nan
 * */
```

Fortran interface：

```
REAL(8) :: X = 0.0
PRINT*,  ATAN(X)
!
! OUTPUT
!     0.000000000000000
!
```

6.4.3.2.1.19 atand

计算输入实数的反正切值对应的角度。

接口定义

C interface：

float atandf(float x);

double atand(double x);

参数

参数名	类型	描述	输入/输出
x	• 在atandf中，x是单精度浮点类型。 • 在atand中，x是双精度浮点类型。	表示输入数据的浮点值。	输入

返回值

- 正常区间：返回输入x的反正切函数值对应的角度y=atan(x) * 180 / π，y ∈ [-90, +90]。

 版权所有 © 华为技术有限公司

- 特殊区间：如下所示。

输入值（x）	输出值（y）
+0	+0
-0	-0
+inf	90
-inf	-90
nan	nan

依赖

C: "km.h"

示例

C interface：

```
// typical usage
float x1 = -1.0, x2 = 1.0, x3 = 0.5, x4 = -0.5;
// special handling
float a = 0.0, b = INFINITY, c = -INFINITY, d = NAN;
// print result
printf("atand(1.0) = %.15f\n", atand(x1));
printf("atand(-1.0) = %.15f\n", atand(x2));
printf("atand(0.5) = %.15f\n", atand(x3));
printf("atand(-0.5) = %.15f\n", atand(x4));
printf("atand(0.0) = %.15f\n", atand(a));
printf("atand(-0.0) = %.15f\n", atand(-a));
printf("atand(INFINITY) = %.15f\n", atand(b));
printf("atand(-INFINITY) = %.15f\n", atand(c));
printf("atand(NAN) = %.15f\n", atand(d));
/*
 *  atand(1.0) = -45.000000000000000
 *  atand(-1.0) = 45.000000000000000
 *  atand(0.5) = 26.565051177077990
 *  atand(-0.5) = -26.565051177077990
 *  atand(0.0) = 0.000000000000000
 *  atand(-0.0) = -0.000000000000000
 *  atand(INFINITY) = 90.000000000000000
 *  atand(-INFINITY) = -90.000000000000000
 *  atand(NAN) = nan
 *
 * */
```

6.4.3.2.1.20 atanpi

计算输入实数的反正切值除以π。

接口定义

C interface：

float atanpif(float x);

double atanpi(double x);

 版权所有 © 华为技术有限公司

参数

参数名	类型	描述	输入/输出
x	• 在atanpif中，x是单精度浮点类型。 • 在atanpi中，x是双精度浮点类型。	表示输入数据的浮点值。	输入

返回值

- 正常区间：返回输入实数x的反正切函数值除以π的值y=atan(x) / π，y ∈ [-0.5, +0.5]。
- 特殊区间：如下所示。

输入值（x）	输出值（y）
+0	+0
-0	-0
+inf	0.5
-inf	-0.5
nan	nan

依赖

C: "km.h"

示例

C interface：

```
// typical usage
float x1 = -1.0, x2 = 1.0, x3 = 0.5, x4 = -0.5;
// special handling
float a = 0.0, b = INFINITY, c = -INFINITY, d = NAN;
// print result
printf("atanpi(1.0) = %.15f\n", atanpi(x1));
printf("atanpi(-1.0) = %.15f\n", atanpi(x2));
printf("atanpi(0.5) = %.15f\n", atanpi(x3));
printf("atanpi(-0.5) = %.15f\n", atanpi(x4));
printf("atanpi(0.0) = %.15f\n", atanpi(a));
printf("atanpi(-0.0) = %.15f\n", atanpi(-a));
printf("atanpi(INFINITY) = %.15f\n", atanpi(b));
printf("atanpi(-INFINITY) = %.15f\n", atanpi(c));
printf("atanpi(NAN) = %.15f\n", atanpi(d));
/*
 *  atanpi(1.0) = -0.250000000000000
 *  atanpi(-1.0) = 0.250000000000000
 *  atanpi(0.5) = 0.147583617650433
 *  atanpi(-0.5) = -0.147583617650433
 *  atanpi(0.0) = 0.000000000000000
 *  atanpi(-0.0) = -0.000000000000000
 *  atanpi(INFINITY) = 0.500000000000000
 *  atanpi(-INFINITY) = -0.500000000000000
```

 版权所有 © 华为技术有限公司

```
*  atanpi(NAN) = nan
*
* */
```

6.4.3.2.1.21 atan2

计算y/x的反正切的弧度值。

接口定义

C interface：

float atan2f(float y, float x);

double atan2(double y, double x);

long double atan2l(long double y, long double x);

Fortran interface：

RES = ATAN2F(Y,X);

RES = ATAN2(Y,X);

参数

参数名	类型	描述	输入/输出
y	● 在atan2f中，y是单精度浮点类型。 ● 在atan2中，y是双精度浮点类型。 ● 在atan2l中，y是长双精度浮点类型。	代表y轴坐标的浮点值。	输入
x	● 在atan2f中，x是单精度浮点类型。 ● 在atan2中，x是双精度浮点类型。 ● 在atan2l中，x是长双精度浮点类型。	代表x轴坐标的浮点值。	输入

返回值

对于atan2(y, x)来说，返回的是原点至点(x, y)的方位角，即与x轴的夹角。返回值的单位为弧度，取值范围为(-π, +π]。

- 输入y为±0，x为负或-0，则返回±π。
- 输入y为±0，x为正或+0，则返回±0。
- 输入y为±∞，x为有限，则返回±π/2。
- 输入y为±∞，x为-∞，则返回±3π/4。
- 输入y为±∞，x为+∞，则返回±π/4。

版权所有 © 华为技术有限公司

- 输入x为±0，y为负，则返回-π/2。
- 输入x为±0，y为正，则返回+π/2。
- 输入x是-∞，y是有限且正的，则返回+π。
- 输入x为-∞，y为有限负，则返回-π。
- 输入x为+∞，且y为有限且为正，则返回+0。
- 输入x为+∞，且y为有限且为负，则返回-0。
- 输入x为nan，或y为nan，则返回nan。

依赖

C: "km.h"

示例

C interface：

```
double pi = acos(-1);
// typical usage
double x1 = 1.0, y1 = pi/2, x2 = -3*pi/4, y2 = 2.0;
double x3 = 4.0, y3 = -4.0, x4 = -3.0, y4 = 3.0;
// special handling
double a = 0.0, b = INFINITY, c = -INFINITY, d = NAN;
// print result
printf("atan2(pi/2, 1.0) = %.15f\n", atan2(y1, x1));
printf("atan2(2.0, -3*pi/4) = %.15f\n", atan2(y2, x2));
printf("atan2(-4.0, 4.0) = %.15f\n", atan2(y3, x3));
printf("atan2(3.0, -3.0) = %.15f\n", atan2(y4, x4));
printf("atan2(+0.0, +0.0) = %+.15f\n", atan2(a, a));
printf("atan2(+0.0, -0.0) = %+.15f\n", atan2(a, -a));
printf("atan2(-0.0, +0.0) = %+.15f\n", atan2(-a, a));
printf("atan2(-0.0, -0.0) = %+.15f\n", atan2(-a, -a));
printf("atan2(+0.0, -4.0) = %+.15f\n", atan2(a, y3));
printf("atan2(-0.0, +3.0) = %+.15f\n", atan2(-a, y4));
printf("atan2(-4.0, +0.0) = %+.15f\n", atan2(y3, a));
printf("atan2(+3.0, -0.0) = %+.15f\n", atan2(y4, -a));
printf("atan2(INFINITY, 1.0) = %+.15f\n", atan2(b, x1));
printf("atan2(-INFINITY, -1.0) = %+.15f\n", atan2(c, -x1));
printf("atan2(1.0, -INFINITY) = %+.15f\n", atan2(x1, c));
printf("atan2(-3.0, -INFINITY) = %+.15f\n", atan2(x4, c));
printf("atan2(1.0, INFINITY) = %+.15f\n", atan2(x1, b));
printf("atan2(-3.0, INFINITY) = %+.15f\n", atan2(x4, b));
printf("atan2(1.0, NAN) = %+.15f\n", atan2(x1, d));
printf("atan2(NAN, -1.0) = %+.15f\n", atan2(d, -x1));
/*
 *  atan2(pi/2, 1.0) = 1.003884821853887
 *  atan2(2.0, -3*pi/4) = 2.437780340738381
 *  atan2(-4.0, 4.0) = -0.785398163397448
 *  atan2(3.0, -3.0) = 2.356194490192345
 *  atan2(+0.0, +0.0) = +1.570796326794897
 *  atan2(+0.0, -0.0) = +1.570796326794897
 *  atan2(-0.0, +0.0) = +1.570796326794897
 *  atan2(-0.0, -0.0) = +1.570796326794897
 *  atan2(+0.0, -4.0) = +3.141592653589793
 *  atan2(-0.0, +3.0) = +0.000000000000000
 *  atan2(-4.0, +0.0) = -1.570796326794897
 *  atan2(+3.0, -0.0) = +1.570796326794897
 *  atan2(INFINITY, 1.0) = +1.570796326794897
 *  atan2(INFINITY, -1.0) = -1.570796326794897
 *  atan2(1.0, -INFINITY) = +3.141592653589793
 *  atan2(-3.0, -INFINITY) = -3.141592653589793
 *  atan2(1.0, INFINITY) = +0.000000000000000
 *  atan2(-3.0, INFINITY) = -0.000000000000000
```

 版权所有 © 华为技术有限公司

```
 *  atan2(1.0, NAN) = +nan
 *  atan2(NAN, -1.0) = +nan
 *
 * */
```

Fortran interface：

```
REAL(8) :: X = 3.0
REAL(8) :: Y = -3.0
PRINT*,  ATAN2(Y, X)
!
! OUTPUT
!     2.356194490192345
!
```

6.4.3.2.1.22 atan2d

计算y/x的反正切的角度值。

接口定义

C interface：

float atan2df(float y, float x);

double atan2d(double y, double x);

long double atan2dl(long double y, long double x);

参数

参数名	类型	描述	输入/输出
x	• 在atan2df中，x是单精度浮点类型。 • 在atan2d中，x是双精度浮点类型。 • 在atan2dl中，x是长双精度浮点类型。	表示输入数据的浮点值。	输入
y	• 在atan2df中，y是单精度浮点类型。 • 在atan2d中，y是双精度浮点类型。 • 在atan2dl中，y是长双精度浮点类型。	表示输入数据的浮点值。	输入

返回值

- 正常区间：返回y/x的反正切的角度值，即atan2d(y, x) = atan2(y, x) * 180 / π。
- 特殊区间：如下所示。

 版权所有 © 华为技术有限公司

输入值（y）	输入值（x）	输出值（result）
+0	+0	+0
+0	-0	180
+0	inf	+0
+0	-inf	180
+0	nan	nan
-0	+0	-0
-0	-0	-180
-0	inf	-0
-0	-inf	-180
-0	nan	nan
inf	+0	90
inf	-0	90
inf	inf	45
inf	-inf	135
inf	nan	nan
-inf	+0	-90
-inf	-0	-90
-inf	inf	-45
-inf	-inf	-135
-inf	nan	nan
nan	+0	nan
nan	-0	nan
nan	inf	nan
nan	-inf	nan
nan	nan	nan

依赖

C: "km.h"

示例

C interface：

 版权所有 © 华为技术有限公司

```
// typical usage
float x1 = -1.0, x2 = 1.0, x3 = 0.5, x4 = -0.5;
// special handling
float a = 0.0, b = INFINITY, c = -INFINITY, d = NAN;
// print result
printf("atan2d(1.0, 1.0) = %.15f\n", atan2d(x1, x1));
printf("atan2d(-1.0, -1.0) = %.15f\n", atan2d(x2, x2));
printf("atan2d(0.5, 0.5) = %.15f\n", atan2d(x3, x3));
printf("atan2d(-0.5, -0.5) = %.15f\n", atan2d(x4, x4));
printf("atan2d(0.0, 0.0) = %.15f\n", atan2d(a, a));
printf("atan2d(-0.0, -0.0) = %.15f\n", atan2d(-a, -a));
printf("atan2d(INFINITY, INFINITY) = %.15f\n", atan2d(b, b));
printf("atan2d(-INFINITY, -INFINITY) = %.15f\n", atan2d(c, c));
printf("atan2d(NAN, NAN) = %.15f\n", atan2d(d, d));
/*
 *  atan2d(1.0, 1.0) = -135.000000000000000
 *  atan2d(-1.0, -1.0) = 45.000000000000000
 *  atan2d(0.5, 0.5) = 45.000000000000000
 *  atan2d(-0.5, -0.5) = -135.000000000000000
 *  atan2d(0.0, 0.0) = 0.000000000000000
 *  atan2d(-0.0, -0.0) = -180.000000000000000
 *  atan2d(INFINITY, INFINITY) = 45.000000000000000
 *  atan2d(-INFINITY, -INFINITY) = -135.000000000000000
 *  atan2d(NAN, NAN) = nan
 *
 * */
```

6.4.3.2.1.23 atan2pi

计算y/x的反正切值，并将结果再除以π。

接口定义

C interface：

float atan2pif(float y, float x);

double atan2pi(double y, double x);

参数

参数名	类型	描述	输入/输出
x	• 在atan2pif中，x是单精度浮点类型。 • 在atan2pi中，x是双精度浮点类型。	表示输入数据的浮点值。	输入
y	• 在atan2pif中，y是单精度浮点类型。 • 在atan2pi中，y是双精度浮点类型。	表示输入数据的浮点值。	输入

返回值

- 正常区间：计算y/x的反正切值，并将结果再除以π，即result = atan(y/x) / π，res∈[-1, +1]。

 版权所有 © 华为技术有限公司

- 特殊区间：如下所示。

输入值（y）	输入值（x）	输出值（result）
+0	+0	+0
+0	-0	+1
+0	inf	+0
+0	-inf	+1
+0	nan	nan
-0	+0	-0
-0	-0	-1
-0	inf	-0
-0	-inf	-1
-0	nan	nan
inf	+0	0.5
inf	-0	0.5
inf	inf	0.25
inf	-inf	0.75
inf	nan	nan
-inf	+0	-0.5
-inf	-0	-0.5
-inf	inf	-0.25
-inf	-inf	-0.75
-inf	nan	nan
nan	+0	nan
nan	-0	nan
nan	inf	nan
nan	-inf	nan
nan	nan	nan

依赖

C: "km.h"

 版权所有 © 华为技术有限公司

示例

C interface：

```
// typical usage
float x1 = -1.0, x2 = 1.0, x3 = 0.5, x4 = -0.5;
// special handling
float a = 0.0, b = INFINITY, c = -INFINITY, d = NAN;
// print result
printf("atan2pi(1.0, 1.0) = %.15f\n", atan2pi(x1, x1));
printf("atan2pi(-1.0, -1.0) = %.15f\n", atan2pi(x2, x2));
printf("atan2pi(0.5, 0.5) = %.15f\n", atan2pi(x3, x3));
printf("atan2pi(-0.5, -0.5) = %.15f\n", atan2pi(x4, x4));
printf("atan2pi(0.0, 0.0) = %.15f\n", atan2pi(a, a));
printf("atan2pi(-0.0, -0.0) = %.15f\n", atan2pi(-a, -a));
printf("atan2pi(INFINITY, INFINITY) = %.15f\n", atan2pi(b, b));
printf("atan2pi(-INFINITY, -INFINITY) = %.15f\n", atan2pi(c, c));
printf("atan2pi(NAN, NAN) = %.15f\n", atan2pi(d, d));
/*
 *  atan2pi(1.0, 1.0) = -0.750000000000000
 *  atan2pi(-1.0, -1.0) = 0.250000000000000
 *  atan2pi(0.5, 0.5) = 0.250000000000000
 *  atan2pi(-0.5, -0.5) = -0.750000000000000
 *  atan2pi(0.0, 0.0) = 0.000000000000000
 *  atan2pi(-0.0, -0.0) = -1.000000000000000
 *  atan2pi(INFINITY, INFINITY) = 0.250000000000000
 *  atan2pi(-INFINITY, -INFINITY) = -0.750000000000000
 *  atan2pi(NAN, NAN) = nan
 *
 * */
```

6.4.3.2.1.24 cot

计算输入弧度的三角余切值，输入参数为弧度。

接口定义

C interface：

float cotf(float x);

double cot(double x);

long double cotl(long double x);

参数

参数名	类型	描述	输入/输出
x	• 在cotf中，x是单精度浮点类型。 • 在cot中，x是双精度浮点类型。 • 在cotl中，x是长双精度浮点类型。	表示弧度角的浮点值。	输入

返回值

- 正常区间：返回弧度角x的余切函数值y = cos(x) / sin(x)，y∈[-∞, +∞]。

 版权所有 © 华为技术有限公司

- 输入x = π * k，（k为整数），输出为±inf。
- 输入x = π * k + π/2，（k为整数），输出为±0。
- 输入|x| = inf or nan，输出为nan。

依赖

C: "km.h"

示例

C interface：

```
// typical usage
float x1 = -1.0, x2 = 1.0, x3 = 0.5, x4 = -0.5;
// special handling
float a = 0.0, b = INFINITY, c = -INFINITY, d = NAN;
// print result
printf("cot(1.0) = %.15f\n", cot(x1));
printf("cot(-1.0) = %.15f\n", cot(x2));
printf("cot(0.5) = %.15f\n", cot(x3));
printf("cot(-0.5) = %.15f\n", cot(x4));
printf("cot(0.0) = %.15f\n", cot(a));
printf("cot(-0.0) = %.15f\n", cot(-a));
printf("cot(INFINITY) = %.15f\n", cot(b));
printf("cot(-INFINITY) = %.15f\n", cot(c));
printf("cot(NAN) = %.15f\n", cot(d));
/*
 *  cot(1.0) = -0.642092615934331
 *  cot(-1.0) = 0.642092615934331
 *  cot(0.5) = 1.830487721712452
 *  cot(-0.5) = -1.830487721712452
 *  cot(0.0) = inf
 *  cot(-0.0) = -inf
 *  cot(INFINITY) = nan
 *  cot(-INFINITY) = nan
 *  cot(NAN) = nan
 *
 * */
```

6.4.3.2.1.25 cotd

计算输入角度的三角余切值。

接口定义

C interface：

float cotdf(float x);

double cotd(double x);

long double cotdl(long double x);

 版权所有 © 华为技术有限公司

参数

参数名	类型	描述	输入/输出
x	• 在cotdf中，x是单精度浮点类型。 • 在cotd中，x是双精度浮点类型。 • 在cotdl中，x是长双精度浮点类型。	表示角度的浮点值。	输入

返回值

- 正常区间：返回角度x的余切函数值y = cot(x * π / 180)，y ∈ [-∞, +∞]。
- 输入x = 180 * k，（k为整数），输出为±inf。
- 输入x = 180 * k + 90，（k为整数），输出为±0。
- 输入|x| = inf or nan，输出为nan。

依赖

C: "km.h"

示例

C interface：

```
// typical usage
float x1 = -1.0, x2 = 1.0, x3 = 0.5, x4 = -0.5;
// special handling
float a = 0.0, b = INFINITY, c = -INFINITY, d = NAN;
// print result
printf("cotd(1.0) = %.15f\n", cotd(x1));
printf("cotd(-1.0) = %.15f\n", cotd(x2));
printf("cotd(0.5) = %.15f\n", cotd(x3));
printf("cotd(-0.5) = %.15f\n", cotd(x4));
printf("cotd(0.0) = %.15f\n", cotd(a));
printf("cotd(-0.0) = %.15f\n", cotd(-a));
printf("cotd(INFINITY) = %.15f\n", cotd(b));
printf("cotd(-INFINITY) = %.15f\n", cotd(c));
printf("cotd(NAN) = %.15f\n", cotd(d));
/*
 *  cotd(1.0) = -57.289961630759421
 *  cotd(-1.0) = 57.289961630759421
 *  cotd(0.5) = 114.588650129309613
 *  cotd(-0.5) = -114.588650129309613
 *  cotd(0.0) = inf
 *  cotd(-0.0) = -inf
 *  cotd(INFINITY) = nan
 *  cotd(-INFINITY) = nan
 *  cotd(NAN) = nan
 *
 * */
```

6.4.3.2.2 双曲函数

 版权所有 © 华为技术有限公司

6.4.3.2.2.1 sinh

计算输入弧度的双曲正弦值。

接口定义

C interface：

float sinhf(float x);

double sinh(double x);

long double sinhl(long double x);

Fortran interface：

RES = SINHF(X);

RES = SINH(X);

参数

参数名	类型	描述	输入/输出
x	• 在sinhf中，x是单精度浮点类型。 • 在sinh中，x是双精度浮点类型。 • 在sinhl中，x是长双精度浮点类型	表示弧度角的浮点值。	输入

返回值

- 返回弧度角x的双曲正弦函数值y。
- 输入±0，返回±0。
- sinhf输入大于等于0x1.8p6（96），返回+∞；sinh输入大于等于0x1.8p9（768），返回+∞。
- sinhf输入小于等于-0x1.8p6（-96），返回-∞；sinh输入小于等于-0x1.8p9（-768），返回-∞。
- sinhf输入小于0x1p-13，返回x；sinh输入小于0x1p-26，返回x。
- 输入±∞，返回±∞。
- 输入nan，返回nan。

依赖

C: "km.h"

示例

C interface：

 版权所有 © 华为技术有限公司

```
double pi = acos(-1);
// typical usage
double a = pi/6, b = 1.0, c = -3*pi/4, d = pi/3;
// special handling
double e = INFINITY, f = -INFINITY, g = NAN;
// print result
printf("sinh(pi/6) = %.15f\n", sinh(a));
printf("sinh(1.0) = %.15f\n", sinh(b));
printf("sinh(-3*pi/4) = %.15f\n", sinh(c));
printf("sinh(pi/3) = %.15f\n", sinh(d));
printf("sinh(INFINITY) = %.15f\n", sinh(e));
printf("sinh(-INFINITY) = %.15f\n", sinh(f));
printf("sinh(NAN) = %.15f\n", sinh(g));
/*
 *  sinh(pi/6) = 0.547853473888040
 *  sinh(1.0) = 1.175201193643801
 *  sinh(-3*pi/4) = -5.227971924677803
 *  sinh(pi/3) = 1.249367050523975
 *  sinh(INFINITY) = inf
 *  sinh(-INFINITY) = -inf
 *  sinh(NAN) = nan
 *
 * */
```

Fortran interface：

```
REAL(8) :: X = 1.0
PRINT*,  SINH(X)
!
! OUTPUT
!     1.175201193643801
!
```

6.4.3.2.2.2 cosh

计算输入弧度的双曲余弦值。

接口定义

C interface：

float coshf(float x);

double cosh(double x);

long double coshl(long double x);

Fortran interface：

RES = COSHF(X);

RES = COSH(X);

 版权所有 © 华为技术有限公司

参数

参数名	类型	描述	输入/输出
x	• 在coshf中，x是单精度浮点类型。 • 在cosh中，x是双精度浮点类型。 • 在coshl中，x是长双精度浮点类型。	表示弧度角的浮点值。	输入

返回值

- 返回弧度角x的双曲余弦函数值y，y ∈ [1, +∞]。
- 输入±0，返回+1。
- coshf，当|x| >= 0x1.8p6（96）时返回+∞，当|x| <= 0x1p-13时返回1.0。
- cosh，当|x| >= 0x1.8p9（768）时返回+∞，当|x| <= 0x1p-26时返回1.0。
- 输入±∞，返回+∞。
- 输入nan，返回nan。

依赖

C: "km.h"

示例

C interface：

```
double pi = acos(-1);
 // typical usage
 double a = pi/3, b = pi/2, c = -3*pi/4, d = 0.0;
 // special handling
 double e = INFINITY, f = -INFINITY, g = NAN;
 // print result
 printf("cosh(pi/3) = %.15f\n", cosh(a));
 printf("cosh(pi/2) = %.15f\n", cosh(b));
 printf("cosh(-3*pi/4) = %.15f\n", cosh(c));
 printf("cosh(0.0) = %.15f\n", cosh(d));
 printf("cosh(INFINITY) = %.15f\n", cosh(e));
 printf("cosh(-INFINITY) = %.15f\n", cosh(f));
 printf("cosh(NAN) = %.15f\n", cosh(g));
 /*
  *  cosh(pi/3) = 1.600286857702386
  *  cosh(pi/2) = 2.509178478658057
  *  cosh(-3*pi/4) = 5.322752149519959
  *  cosh(0.0) = 1.000000000000000
  *  cosh(INFINITY) = inf
  *  cosh(-INFINITY) = inf
  *  cosh(NAN) = nan
  *
  * */
```

Fortran interface：

```
REAL(8) :: X = 0.0
PRINT*,  COSH(X)
```

 版权所有 © 华为技术有限公司

```
!
! OUTPUT
!    1.000000000000000
!
```

6.4.3.2.2.3 sinhcosh

分别计算输入弧度的双曲正弦、余弦值。

接口定义

C interface：

void sinhcoshf(float x, float *sinh, float *cosh);

void sinhcosh(double x, double *sinh, double *cosh);

参数

参数名	类型	描述	输入/输出
x	• 在sinhcoshf中，x是单精度浮点类型。 • 在sinhcosh中，x是双精度浮点类型。	表示弧度角的浮点值。	输入
sinh	• 在sinhcoshf中，sinh是单精度浮点指针类型。 • 在sinhcosh中，sinh是双精度浮点指针类型。	表示计算的双曲正弦。 若为空指针，会导致未定义的行为，接口不会对输出指针进行校验。	输出
cosh	• 在sinhcoshf中，cosh是单精度浮点指针类型。 • 在sinhcosh中，cosh是双精度浮点指针类型。	表示计算的双曲余弦。 若为空指针，会出现未定义的行为，接口不会对输出指针进行校验。	输出

返回值

- 返回弧度角x的双曲正弦数值sinh和双曲余弦函数值cosh，sinh ∈ [-∞, +∞]，cosh∈ [1, +∞]。
- 输入+0，返回双曲正弦值+0和双曲余弦值+1。
- 输入-0，返回双曲正弦值-0和双曲余弦值+1。
- sinhcoshf输入大于0x1.8p6f，均返回+∞；sinhcosh输入大于0x1.8p9，均返回+∞。
- sinhcoshf输入小于-0x1.8p6f，双曲正弦值返回-∞，双曲余弦值返回+∞；sinhcosh输入小于-0x1.8p9，双曲正弦值返回-∞，双曲余弦值返回+∞。

版权所有 © 华为技术有限公司

- sinhcoshf输入绝对值小于0x1.p-13，双曲正弦值返回x，双曲余弦值返回1.0；sinhcosh输入绝对值小于0x1.p-26，双曲正弦值返回x，双曲余弦值返回1.0。
- 输入±∞，返回双曲正弦值±∞和双曲余弦值+∞。
- 输入nan，均返回nan。

依赖

C: "km.h"

示例

C interface：

```
// typical usage
double a = 1.0, b = -1.0, c = 0.0, d = -0.0;
// special handling
double e = INFINITY, f = -INFINITY, g = NAN;
// print result
double sinh, cosh;
sinhcosh(a, &sinh, &cosh);
printf("sinh(1.0) = %.15f, cosh(1.0) = %.15f\n", sinh, cosh);
sinhcosh(b, &sinh,&cosh);
printf("sinh(-1.0) = %.15f, cosh(-1.0) = %.15f\n", sinh, cosh);
sinhcosh(c, &sinh,&cosh);
printf("sinh(0.0) = %.15f, cosh(0.0) = %.15f\n", sinh, cosh);
sinhcosh(d, &sinh,&cosh);
printf("sinh(-0.0) = %.15f, cosh(-0.0) = %.15f\n", sinh, cosh);
sinhcosh(e, &sinh,&cosh);
printf("sinh(INFINITY) = %.15f, cosh(INFINITY) = %.15f\n", sinh, cosh);
sinhcosh(f, &sinh,&cosh);
printf("sinh(-INFINITY) = %.15f, cosh(-INFINITY) =  %.15f\n", sinh, cosh);
sinhcosh(g, &sinh,&cosh);
printf("sinh(NAN) = %.15f, cosh(NAN) = %.15f\n", sinh, cosh);
/*
 * sinh(1.0) = 1.175201193643801, cosh(1.0) = 1.543080634815244
 * sinh(-1.0) = -1.175201193643801, cosh(-1.0) = 1.543080634815244
 * sinh(0.0) = 0.000000000000000, cosh(0.0) = 1.000000000000000
 * sinh(-0.0) = -0.000000000000000, cosh(-0.0) = 1.000000000000000
 * sinh(INFINITY) = inf, cosh(INFINITY) = inf
 * sinh(-INFINITY) = -inf, cosh(-INFINITY) =  inf
 * sinh(NAN) = nan, cosh(NAN) = nan
 *
 * */
```

6.4.3.2.2.4 tanh

计算输入弧度的双曲正切值。

接口定义

C interface：

float tanhf(float x);

double tanh(double x);

long double tanhl(long double x);

Fortran interface：

RES = TANHF(X)

RES = TANH(X)

 版权所有 © 华为技术有限公司

参数

参数名	类型	描述	输入/输出
x	• 在tanhf中，x是单精度浮点类型。 • 在tanh中，x是双精度浮点类型。 • 在tanhl中，x是长双精度浮点类型。	表示弧度角的浮点值。	输入

返回值

- 返回弧度角x的双曲正切函数值y = sinh(x) / cosh(x)，y ∈ [-1, +1]。
- 输入+0，返回+0。
- 输入-0，返回-0。
- 输入+∞，返回+1。
- 输入-∞，返回-1。
- 输入nan，返回nan。

依赖

C: "km.h"

示例

C interface：

```
double pi = acos(-1);
// typical usage
double a = pi/3, c = -3*pi/4, d = 0.0;
// special handling
double b = pi/2, e = INFINITY, f = -INFINITY, g = NAN;
// print result
printf("tanh(pi/3) = %.15f\n", tanh(a));
printf("tanh(-3*pi/4) = %.15f\n", tanh(c));
printf("tanh(0.0) = %.15f\n", tanh(d));
// mathematical poles
printf("tanh(pi/2) = %.15f\n", tanh(b));
printf("tanh(INFINITY) = %.15f\n", tanh(e));
printf("tanh(-INFINITY) = %.15f\n", tanh(f));
printf("tanh(NAN) = %.15f\n", tanh(g));
/*
 *  tanh(pi/3) = 0.780714435359268
 *  tanh(-3*pi/4) = -0.982193380007239
 *  tanh(0.0) = 0.000000000000000
 *  tanh(pi/2) = 0.917152335667274
 *  tanh(INFINITY) = 1.000000000000000
 *  tanh(-INFINITY) = -1.000000000000000
 *  tanh(NAN) = nan
 *
 * */
```

Fortran interface：

```
REAL(8) :: X = 0.0
PRINT*,  TANH(X)
```

 版权所有 © 华为技术有限公司

```
!
! OUTPUT
!     0.000000000000000
!
```

6.4.3.2.2.5 asinh

计算输入实数的双曲反正弦值。

接口定义

C interface：

float asinhf(float x);

double asinh(double x);

long double asinhl(long double x);

Fortran interface：

RES = ASINHF(X);

RES = ASINH(X);

参数

参数名	类型	描述	输入/输出
x	• 在asinhf中，x是单精度浮点类型。 • 在asinh中，x是双精度浮点类型。 • 在asinhl中，x是长双精度浮点类型。	表示输入数据的浮点值。	输入

返回值

- 返回弧度角x的双曲反正弦函数值y。
- 输入±0，返回±0。
- 输入±∞，返回±∞。
- 输入nan，返回nan。

依赖

C: "km.h"

示例

C interface：

```
double pi = acos(-1);
// typical usage
double a = pi/6, b = 1.0, c = -3*pi/4, d = pi/3;
```

 版权所有 © 华为技术有限公司

```
// special handling
double e = INFINITY, f = -INFINITY, g = NAN;
// print result
printf("asinh(pi/6) = %.15f\n", asinh(a));
printf("asinh(1.0) = %.15f\n", asinh(b));
printf("asinh(-3*pi/4) = %.15f\n", asinh(c));
printf("asinh(pi/3) = %.15f\n", asinh(d));
printf("asinh(INFINITY) = %.15f\n", asinh(e));
printf("asinh(-INFINITY) = %.15f\n", asinh(f));
printf("asinh(NAN) = %.15f\n", asinh(g));
/*
 *  asinh(pi/6) = 0.502218985034611
 *  asinh(1.0) = 0.881373587019543
 *  asinh(-3*pi/4) = -1.592457372858543
 *  asinh(pi/3) = 0.914356655392886
 *  asinh(INFINITY) = inf
 *  asinh(-INFINITY) = -inf
 *  asinh(NAN) = nan
 *
 * */
```

Fortran interface：

```
REAL(8) :: X = 1.0
PRINT*,  ASINH(X)
!
! OUTPUT
!     0.881373587019543
!
```

6.4.3.2.2.6 acosh

计算输入实数的双曲反余弦值。

接口定义

C interface：

float acoshf(float x);

double acosh(double x);

Fortran interface：

RES = ACOSHF(X);

RES = ACOSH(X);

参数

参数名	类型	描述	输入/输出
x	• 在acoshf中，x是单精度浮点类型。 • 在acosh中，x是双精度浮点类型。	表示输入数据的浮点值。	输入

返回值

- 返回弧度角x的双曲反余弦函数值y，$x \in [1, +\infty)$。

版权所有 © 华为技术有限公司

- 输入+1，返回+0。
- 输入x < +1，返回nan。
- 输入+∞，返回+∞。
- 输入-∞，返回nan。
- 输入nan，返回nan。

依赖

C: "km.h"

示例

C interface：

```
double pi = acos(-1);
// typical usage
double a = pi/3, b = pi/2, c = -3*pi/4, d = 0.0;
// special handling
double e = INFINITY, f = -INFINITY, g = NAN;
// print result
printf("acosh(pi/3) = %.15f\n", acosh(a));
printf("acosh(pi/2) = %.15f\n", acosh(b));
printf("acosh(-3*pi/4) = %.15f\n", acosh(c));
printf("acosh(0.0) = %.15f\n", acosh(d));
printf("acosh(INFINITY) = %.15f\n", acosh(e));
printf("acosh(-INFINITY) = %.15f\n", acosh(f));
printf("acosh(NAN) = %.15f\n", acosh(g));
/*
 *  acosh(pi/3) = 0.306042108613265
 *  acosh(pi/2) = 1.023227478547551
 *  acosh(-3*pi/4) = nan
 *  acosh(0.0) = nan
 *  acosh(INFINITY) = inf
 *  acosh(-INFINITY) = nan
 *  acosh(NAN) = nan
 *
 * */
```

Fortran interface：

```
REAL(8) :: X = 1.0
PRINT*,  ACOSH(X)
!
! OUTPUT
!     0.000000000000000
!
```

6.4.3.2.2.7 atanh

计算输入实数的双曲反正切值。

接口定义

C interface：

float atanhf(float x);

double atanh(double x);

long double atanhl(long double x);

Fortran interface：

 版权所有 © 华为技术有限公司

RES = ATANHF(X)

RES = ATANH(X)

参数

参数名	类型	描述	输入/输出
x	• 在atanhf中，x是单精度浮点类型。 • 在atanh中，x是双精度浮点类型。 • 在atanhl中，x是长双精度浮点类型。	表示输入数据的浮点值。	输入

返回值

- 返回弧度角x的双曲反正切函数值y，x ∈ (-1, +1)。
- 输入+1，返回+∞。
- 输入-1，返回-∞。
- 输入|x| > 1，返回nan。
- 输入±∞，返回nan。
- 输入nan，返回nan。

依赖

C: "km.h"

示例

C interface：

```
double pi = acos(-1);
// typical usage
double a = pi/5, c = -3*pi/4, b = pi/8;
// special handling
double d = +0.0, h = -0.0, e = INFINITY, f = -INFINITY, g = NAN;
// print result
printf("atanh(pi/5) = %.15f\n", atanh(a));
printf("atanh(-3*pi/4) = %.15f\n", atanh(c));
printf("atanh(pi/8) = %.15f\n", atanh(b));
printf("atanh(+0.0) = %+.15f\n", atanh(d));
printf("atanh(-0.0) = %+.15f\n", atanh(h));
printf("atanh(INFINITY) = %+.15f\n", atanh(e));
printf("atanh(-INFINITY) = %+.15f\n", atanh(f));
printf("atanh(NAN) = %.15f\n", atanh(g));
/*
 *  atanh(pi/3) = 0.738632981507743
 *  atanh(-3*pi/4) = nan
 *  atanh(pi/2) = 0.414987256841989
 *  atanh(+0.0) = +0.000000000000000
 *  atanh(-0.0) = -0.000000000000000
 *  atanh(INFINITY) = +nan
 *  atanh(-INFINITY) = +nan
 *  atanh(NAN) = nan
```

 版权所有 © 华为技术有限公司

```
 *
 * */
```

Fortran interface：

```
REAL(8) :: X = 0.0
PRINT*,  ATANH(X)
!
! OUTPUT
!     0.000000000000000
!
```

6.4.3.2.3 指数和对数函数

6.4.3.2.3.1 exp

计算以e为底数的x次幂值。

接口定义

C interface：

float expf(float x);

double exp(double x);

long double expl(long double x);

Fortran interface：

RES = EXPF(X);

RES = EXP(X);

参数

参数名	类型	描述	输入/输出
x	• 在expf中，x是单精度浮点类型。 • 在exp中，x是双精度浮点类型。 • 在expl中，x是长双精度浮点类型。	表示输入数据的浮点值。	输入

返回值

- 返回以e为底数的x次幂值y，y ∈ (0, +INF)。
- 输入+0，返回+1。
- 输入-0，返回-1。
- expf输入大于0x1.62e42ep6（约88.72），返回+∞；exp输入大于等于0x1p10（1024），返回+∞。
- expf输入小于-0x1.9fe368p6（约-103.97），返回+0；exp输入小于等于-0x1p10（-1024），返回+0。

 版权所有 © 华为技术有限公司

- 输入+∞，返回+∞。
- 输入-∞，返回+0。
- 输入nan，返回nan。

依赖

C: "km.h"

示例

C interface：

```
double pi = acos(-1);
// typical usage
double x1 = pi/2, x2 = 2.0;
// special handling
double a = 0.0, b = INFINITY, c = -INFINITY, d = NAN, e = 710.0;
// print result
printf("exp(pi/2) = %.15f\n", exp(x1));
printf("exp(2.0) = %.15f\n", exp(x2));
printf("exp(710.0) = %.15f\n", exp(e));
printf("exp(0.0) = %.15f\n", exp(a));
printf("exp(INFINITY) = %.15f\n", exp(b));
printf("exp(-INFINITY) = %.15f\n", exp(c));
printf("exp(NAN) = %.15f\n", exp(d));
/*
 *  exp(pi/2) = 4.810477380965351
 *  exp(2.0) = 7.389056098930650
 *  exp(710.0) = inf
 *  exp(0.0) = 1.000000000000000
 *  exp(INFINITY) = inf
 *  exp(-INFINITY) = 0.000000000000000
 *  exp(NAN) = nan
 *
 * */
```

Fortran interface：

```
REAL(8) :: X = 2
PRINT*,  EXP(X)
!
! OUTPUT
!     7.389056098930650
!
```

6.4.3.2.3.2 exp2

计算以2为底数的x次幂值。

接口定义

C interface：

float exp2f(float x);

double exp2(double x);

long double exp2l(long double x);

Fortran interface：

RES = EXP2F(X);

 版权所有 © 华为技术有限公司

RES = EXP2(X);

参数

参数名	类型	描述	输入/输出
x	• 在exp2f中，x是单精度浮点类型。 • 在exp2中，x是双精度浮点类型。 • 在exp2l中，x是长双精度浮点类型。	表示输入数据的浮点值。	输入

返回值

- 返回以2为底x的指数函数值y，y ∈ (0, +INF)。
- 输入+0，返回+1。
- 输入-0，返回+1。
- exp2f输入大于128，返回+∞；exp2输入大于等于1024，返回+∞。
- exp2f输入小于-150，返回+0；exp2输入小于等于-1075，返回+0。
- 输入+∞，返回+∞。
- 输入-∞，返回+0。
- 输入nan，返回nan。

依赖

C: "km.h"

示例

C interface：

```
double pi = acos(-1);
// typical usage
double x1 = pi/2, x2 = 2.0;
// special handling
double a = 0.0, b = INFINITY, c = -INFINITY, d = NAN, e = 1024.0;
// print result
    printf("exp2(pi/2) = %.15f\n", exp2(x1));
    printf("exp2(2.0) = %.15f\n", exp2(x2));
    printf("exp2(1024.0) = %.15f\n", exp2(e));
    printf("exp2(0.0) = %.15f\n", exp2(a));
    printf("exp2(INFINITY) = %.15f\n", exp2(b));
    printf("exp2(-INFINITY) = %.15f\n", exp2(c));
    printf("exp2(NAN) = %.15f\n", exp2(d));
/*
 *  exp2(pi/2) = 2.970686423552019
 *  exp2(2.0) = 4.000000000000000
 *  exp2(1024.0) = inf
 *  exp2(0.0) = 1.000000000000000
 *  exp2(INFINITY) = inf
 *  exp2(-INFINITY) = 0.000000000000000
 *  exp2(NAN) = nan
```

 版权所有 © 华为技术有限公司

```
 *
 * */
```

Fortran interface：

```
REAL(8) :: X = 2
PRINT*,  EXP2(X)
!
! OUTPUT
!     4.000000000000000
!
```

6.4.3.2.3.3 exp10

计算以10为底数的x次幂值。

接口定义

C interface：

float exp10f(float x);

double exp10(double x);

long double exp10l(long double x);

参数

参数名	类型	描述	输入/输出
x	● 在exp10f中，x是单精度浮点类型。 ● 在exp10中，x是双精度浮点类型。 ● 在exp10l中，x是长双精度浮点类型。	表示输入数据的浮点值。	输入

返回值

- 返回以10为底x的指数函数值y，y ∈ (0, +INF)。
- 输入+0，返回+1。
- 输入-0，返回+1。
- exp10f输入大于0x1.344135p5（约38.53），返回+∞；exp10输入大于等于0x1p9（512），返回+∞。
- exp10f输入小于-0x1.693c6ap5（约-45.15），返回+0；exp10输入小于等于-0x1p9（-512），返回+0。
- 输入+∞，返回+∞。
- 输入-∞，返回+0。
- 输入nan，返回nan。

依赖

C: "km.h"

 版权所有 © 华为技术有限公司

示例

C interface：

```
// typical usage
double x1 = 1.0, x2 = 2.0;
double a = 0.0, b = INFINITY, c = -INFINITY, d = NAN, e = 1024.0;
// print result
printf("exp10(1.0) = %.15f\n", exp10(x1));
printf("exp10(2.0) = %.15f\n", exp10(x2));
printf("exp10(1024.0) = %.15f\n", exp10(e));
printf("exp10(0.0) = %.15f\n", exp10(a));
printf("exp10(INFINITY) = %.15f\n", exp10(b));
printf("exp10(-INFINITY) = %.15f\n", exp10(c));
printf("exp10(NAN) = %.15f\n", exp10(d));
/*
 *  exp10(1.0) = 10.000000000000000
 *  exp10(2.0) = 100.000000000000000
 *  exp10(1024.0) = inf
 *  exp10(0.0) = 1.000000000000000
 *  exp10(INFINITY) = inf
 *  exp10(-INFINITY) = 0.000000000000000
 *  exp10(NAN) = nan
 *
 * */
```

6.4.3.2.3.4 expm1

计算以e为底数的x次幂值，结果再减一。

接口定义

C interface：

float expm1f(float x);

double expm1(double x);

long double expm1l(long double x);

参数

参数名	类型	描述	输入/输出
x	• 在expm1f中，x是单精度浮点类型。 • 在expm1中，x是双精度浮点类型。 • 在expm1l中，x是长双精度浮点类型。	表示输入数据的浮点值。	输入

返回值

- 返回以e为底x的指数函数值减1，y ∈ (-1, +INF)。
- 输入+0，返回+0。
- 输入-0，返回+0。

版权所有 © 华为技术有限公司

- expm1f输入大于0x1.62e42ep6（约88.72），返回+∞；expm1输入大于0x1.8p9（768），返回+∞。
- expm1f输入小于0x1.9fe368p6（约-103.97），返回-1.0；expm1输入小于-0x1.4cb5fp5（约-41.58），返回-1.0。
- 输入+∞，返回+∞。
- 输入-∞，返回-1.0。
- 输入nan，返回nan。

依赖

C: "km.h"

示例

C interface：

```
// typical usage
double x1 = 0.0, x2 = -0.0;
double a = 1.0, b = INFINITY, c = -INFINITY, d = NAN, e = 1024.0;
// print result
printf("expm1(0.0) = %.15f\n", expm1(x1));
printf("expm1(-0.0) = %.15f\n", expm1(x2));
printf("expm1(1024.0) = %.15f\n", expm1(e));
printf("expm1(1.0) = %.15f\n", expm1(a));
printf("expm1(INFINITY) = %.15f\n", expm1(b));
printf("expm1(-INFINITY) = %.15f\n", expm1(c));
printf("expm1(NAN) = %.15f\n", expm1(d));
/*
 * expm1(0.0) = 0.000000000000000
 * expm1(-0.0) = -0.000000000000000
 * expm1(1024.0) = inf
 * expm1(1.0) = 1.718281828459045
 * expm1(INFINITY) = inf
 * expm1(-INFINITY) = -1.000000000000000
 * expm1(NAN) = nan
 *
 * */
```

6.4.3.2.3.5 frexp

将x转换为归一化[1/2, 1)的有符号数乘以2的积分幂。返回归一化的有符号数，指数存储在exp中。

接口定义

C interface：

float frexpf(float x, int *exp);

double frexp(double x, int *exp);

long double frexpl(long double x, int *exp);

 版权所有 © 华为技术有限公司

参数

参数名	类型	描述	输入/输出
x	• 在frexpf中，x是单精度浮点类型。 • 在frexp中，x是双精度浮点类型。 • 在frexpl中，x是长双精度浮点类型。	表示输入数据的浮点值。	输入
exp	exp是整数类型指针。	指数。	输出

返回值

- 将x转换为归一化[1/2, 1)的有符号数乘以2的积分幂。返回归一化的有符号数，指数存储在exp中。
- 输入x为inf，返回inf，exp值无意义。
- 输入x为-inf，返回-inf，exp值无意义。
- 输入x为nan，返回nan，exp值无意义。

依赖

C: "km.h"

示例

C interface：

```
    // print result
    int32_t exp;
    printf("frexp(3.4, &exp) = %.15f, exp = %d\n", frexp(3.4, &exp), exp);
    printf("frexp(-4.8, &exp) = %.15f, exp = %d\n", frexp(-4.8, &exp), exp);
    printf("frexp(NAN, &exp) = %.15f, exp = %d\n", frexp(NAN, &exp), exp);
    printf("frexp(-NAN, &exp) = %.15f, exp = %d\n", frexp(-NAN, &exp), exp);
    printf("frexp(INFINITY, &exp) = %.15f, exp = %d\n", frexp(INFINITY, &exp), exp);
    printf("frexp(-INFINITY, &exp) = %.15f, exp = %d\n", frexp(-INFINITY, &exp), exp);

result
    /*
     * frexp(3.4, &exp) = 0.850000000000000, exp = 2
     * frexp(-4.8, &exp) = -0.600000000000000, exp = 3
     * frexp(NAN, &exp) = nan, exp = 3
     * frexp(-NAN, &exp) = -nan, exp = 3
     * frexp(INFINITY, &exp) = inf, exp = 3
     * frexp(-INFINITY, &exp) = -inf, exp = 3
     * */
```

6.4.3.2.3.6 ldexp

计算x乘以2的exp次幂的值。

接口定义

C interface：

 版权所有 © 华为技术有限公司

float ldexpf(float x, int exp);

double ldexp(double x, int exp);

long double ldexpl(long double x, int exp);

参数

参数名	类型	描述	输入/输出
x	• 在ldexpf中，x是单精度浮点类型。 • 在ldexp中，x是双精度浮点类型。 • 在ldexpl中，x是长双精度浮点类型。	表示输入数据的浮点值。	输入
exp	exp是整数类型。	指数。	输入

返回值

返回x * 2^exp。

依赖

C: "km.h"

示例

C interface：

```
// print result
printf("ldexp(0.0, 4) = %.15f\n", ldexp(0.0, 4));
printf("ldexp(-0.0, 4) = %.15f\n", ldexp(-0.0, 4));
printf("ldexp(3.0, 4) = %.15f\n", ldexp(3.0, 4));
printf("ldexp(1.73, 3) = %.15f\n", ldexp(1.73, 3));
printf("ldexp(INFINITY, 4) = %.15f\n", ldexp(INFINITY, 4));
printf("ldexp(-INFINITY, 4) = %.15f\n", ldexp(-INFINITY, 4));
printf("ldexp(NAN, 4) = %.15f\n", ldexp(NAN, 4));
/*
 * ldexp(0.0, 4) = 0.000000000000000
 * ldexp(-0.0, 4) = -0.000000000000000
 * ldexp(3.0, 4) = 48.000000000000000
 * ldexp(1.73, 3) = 13.840000000000000
 * ldexp(INFINITY, 4) = inf
 * ldexp(-INFINITY, 4) = -inf
 * ldexp(NAN, 4) = nan
 * */
```

6.4.3.2.3.7 hypot

计算实数x^2 + y^2的平方根。

接口定义

C interface：

float hypotf(float x, float y);

 版权所有 © 华为技术有限公司

double hypot(double x, double y);

long double hypotl(long double x, long double y);

参数

参数名	类型	描述	输入/输出
x	• 在hypotf中，x是单精度浮点类型。 • 在hypot中，x是双精度浮点类型。 • 在hypotl中，x是长双精度浮点类型。	表示输入数据的浮点值。	输入
y	• 在hypotf中，y是单精度浮点类型。 • 在hypot中，y是双精度浮点类型。 • 在hypotl中，y是长双精度浮点类型。	表示输入数据的浮点值。	输入

返回值

- 返回x^2 + y^2的平方根，x ∈ (-INF, +INF)，y ∈ (-INF, +INF)。
- 输入x为±inf或者y为±inf，返回inf。
- 输入x为nan或者y为nan，返回nan。
- x^2 + y^2的平方根超过浮点数最大表示范围时，返回inf（Intel在部分情况会返回nan）。

依赖

C: "km.h"

示例

C interface：

```
// typical usage
float x1 = 3.0, x2 = 0x1.fp127;
float y1 = 4.0, y2 = -0x1p70;
// special handling
float e = INFINITY, f = -INFINITY, g = NAN;
// print result
printf("hypotf(3.0, 4.0) = %.15f\n", hypotf(x1, y1));
printf("hypotf(0x1.fp127, 0x1.fp127) = %.15f\n", hypotf(x2, x2));
printf("hypotf(-0x1p70, -0x1p70) = %.15f\n", hypotf(y2, y2));
printf("hypotf(INFINITY, 0) = %.15f\n", hypotf(e, 0.0f));
printf("hypotf(-INFINITY, 3.0) = %.15f\n", hypotf(f, x1));
printf("hypotf(NAN, 3.0) = %.15f\n", hypotf(g, x1));
printf("hypotf(NAN, INFINITY) = %.15f\n", hypotf(g, e));
/*
 * hypotf(3.0, 4.0) = 5.000000000000000
 * hypotf(0x1.fp127, 0x1.fp127) = inf
```


版权所有 © 华为技术有限公司

```
 * hypotf(-0x1p70, -0x1p70) = 166960865306846062182 4.000000000000000
 * hypotf(INFINITY, 0) = inf
 * hypotf(-INFINITY, 3.0) = inf
 * hypotf(NAN, 3.0) = nan
 * hypotf(NAN, INFINITY) = inf
 *
 * */
```

6.4.3.2.3.8 log

计算以自然对数e为底的对数值。

接口定义

C interface：

float logf(float x);

double log(double x);

double log_18(double x);

long double logl(long double x);

说明

log_18仅在高精度版本中提供。

Fortran interface：

RES = LOGF(X);

RES = LOG(X);

参数

参数名	类型	描述	输入/输出
x	• 在logf中，x是单精度浮点类型。 • 在log和log_18中，x是双精度浮点类型。 • 在logl中，x是长双精度浮点类型。	表示输入数据的浮点值。	输入

返回值

- 返回以e为底x的对数函数值y，y ∈ (-INF, +INF)。
- 输入x为±0，返回-∞。
- 输入x为1，返回+0。
- 输入x为负数，返回nan。
- 输入x为-∞，返回nan。
- 输入x为+∞，返回+∞。

版权所有 © 华为技术有限公司

- 输入x为nan，返回nan。

依赖

C: "km.h"

示例

C interface：

```
double pi = acos(-1);
// typical usage
double x1 = pi/2, x2 = 2.0, x3 = 125, x4 = 5;
// special handling
double a = 0.0, b = INFINITY, c = -INFINITY, d = NAN, e = 1.0;
// print result
printf("base-5 logarithm of 125 = %.15f\n", log(x3)/log(x4));
printf("log(1.0) = %.15f\n", log(e));
printf("log(pi/2) = %.15f\n", log(x1));
printf("log(2.0) = %.15f\n", log(x2));
printf("log(0.0) = %.15f\n", log(a));
printf("log(INFINITY) = %.15f\n", log(b));
printf("log(-INFINITY) = %.15f\n", log(c));
printf("log(NAN) = %.15f\n", log(d));
/*
 *  base-5 logarithm of 125 = 3.000000000000000
 *  log(1.0) = 0.000000000000000
 *  log(pi/2) = 0.451582705289455
 *  log(2.0) = 0.693147180559945
 *  log(+0.0) = -inf
 *  log(INFINITY) = inf
 *  log(-INFINITY) = nan
 *  log(NAN) = nan
 *
 * */
```

Fortran interface：

```
REAL(8) :: X = 2
PRINT*,  LOG(X)
!
! OUTPUT
!     0.693147180559945
!
```

6.4.3.2.3.9 log2

计算以2为底的对数值。

接口定义

C interface：

float log2f(float x);

double log2(double x);

long double log2l(long double x);

Fortran interface：

RES = LOG2F(X);

RES = LOG2(X);

 版权所有 © 华为技术有限公司

参数

参数名	类型	描述	输入/输出
x	• 在log2f中，x是单精度浮点类型。 • 在log2中，x是双精度浮点类型。 • 在log2l中，x是长双精度浮点类型。	表示输入数据的浮点值。	输入

返回值

- 返回以2为底x的对数函数值y，y ∈ (-INF, +INF)。
- 输入x为±0，返回-∞。
- 输入x为1，返回0。
- 输入x为负数，返回nan。
- 输入x为-∞，返回nan。
- 输入x为+∞，返回+∞。
- 输入x为nan，返回nan。

依赖

C: "km.h"

示例

C interface：

```
// typical usage
double x1 = 2.0, x2 = 0.5, x3 = 125, x4 = 5;
// special handling
double a = 0.0, b = INFINITY, c = -INFINITY, d = NAN, e = 1.0;
// print result
    printf("base-5 logarithm of 125 = %.15f\n", log2(x3)/log2(x4));
    printf("log2(1.0) = %.15f\n", log2(e));
    printf("log2(2.0) = %.15f\n", log2(x1));
    printf("log2(0.5) = %.15f\n", log2(x2));
    printf("log2(0.0) = %.15f\n", log2(a));
    printf("log2(INFINITY) = %.15f\n", log2(b));
    printf("log2(-INFINITY) = %.15f\n", log2(c));
    printf("log2(NAN) = %.15f\n", log2(d));
/*
 *  base-5 logarithm of 125 = 3.000000000000000
 *  log2(1.0) = 0.000000000000000
 *  log2(2.0) = 1.000000000000000
 *  log2(0.5) = -1.000000000000000
 *  log2(0.0) = -inf
 *  log2(INFINITY) = inf
 *  log2(-INFINITY) = nan
 *  log2(NAN) = nan
 *
 * */
```

Fortran interface：

 版权所有 © 华为技术有限公司

```
REAL(8) :: X = 0.5
PRINT*,  LOG2(X)
!
! OUTPUT
!     -1.000000000000000
!
```

6.4.3.2.3.10 log10

计算以10为底的对数值。

接口定义

C interface：

float log10f(float x);

double log10(double x);

double log10_18(double x);

long double log10l(long double x);

说明

log10_18仅在高精度版本中提供。

Fortran interface：

RES = LOG10F(X);

RES = LOG10(X);

参数

参数名	类型	描述	输入/输出
x	• 在log10f中，x是单精度浮点类型。 • 在log10和log10_18中，x是双精度浮点类型。 • 在log10l中，x是长双精度浮点类型。	表示输入数据的浮点值。	输入

返回值

- 返回以10为底x的对数函数值y，y ∈ (-INF, +INF)。
- 输入x为±0，返回-∞。
- 输入x为1，返回0。
- 输入x为负数，返回nan。
- 输入x为-∞，返回nan。
- 输入x为+∞，返回+∞。
- 输入x为nan，返回nan。

版权所有 © 华为技术有限公司

依赖

C: "km.h"

示例

C interface：

```
double pi = acos(-1);
// typical usage
double x1 = 10.0, x2 = 2.0, x3 = 125, x4 = 5;
// special handling
double a = 0.0, b = INFINITY, c = -INFINITY, d = NAN, e = 1.0;
// print result
printf("base-5 logarithm of 125 = %.15f\n", log10(x3)/log10(x4));
printf("log10(1.0) = %.15f\n", log10(e));
printf("log10(10.0) = %.15f\n", log10(x1));
printf("log10(2.0) = %.15f\n", log10(x2));
printf("log10(0.0) = %.15f\n", log10(a));
printf("log10(INFINITY) = %.15f\n", log10(b));
printf("log10(-INFINITY) = %.15f\n", log10(c));
printf("log10(NAN) = %.15f\n", log10(d));
/*
 *  base-5 logarithm of 125 = 3.000000000000000
 *  log10(1.0) = 0.000000000000000
 *  log10(10.0) = 1.000000000000000
 *  log10(2.0) = 0.301029995663981
 *  log10(0.0) = -inf
 *  log10(INFINITY) = inf
 *  log10(-INFINITY) = nan
 *  log10(NAN) = nan
 *
 * */
```

Fortran interface：

```
REAL(8) :: X = 2
PRINT*,  LOG10(X)
!
! OUTPUT
!     0.301029995663981
!
```

6.4.3.2.3.11 log1p

返回以e为底1+x的对数函数值，即计算ln(1+x)的值。

接口定义

C interface：

float log1pf(float x);

double log1p(double x);

long double log1pl(long double x);

 版权所有 © 华为技术有限公司

参数

参数名	类型	描述	输入/输出
x	• 在log1pf中，x是单精度浮点类型。 • 在log1p中，x是双精度浮点类型。 • 在log1pl中，x是长双精度浮点类型。	表示输入数据的浮点值。	输入

返回值

- 返回以e为底1+x的对数函数值y，y ∈ (-INF, +INF)。
- 输入x为+0，返回+0。
- 输入x为-0，返回-0。
- 输入x为-1，返回-∞。
- 输入x为+∞，返回+∞。
- 输入x<-1，返回nan。
- 输入x为nan，返回nan。

依赖

C: "km.h"

示例

C interface:

```
// typical usage
double x1 = 0.0, x2 = -0.0, x3 = -1, x4 = -2;
// special handling
double a = INFINITY, b = -INFINITY, c = NAN;
// print result
printf("log1p(0.0) = %.15f\n", log1p(x1));
printf("log1p(-0.0) = %.15f\n", log1p(x2));
printf("log1p(-1) = %.15f\n", log1p(x3));
printf("log1p(-2) = %.15f\n", log1p(x4));
printf("log1p(INFINITY) = %.15f\n", log1p(a));
printf("log1p(-INFINITY) = %.15f\n", log1p(b));
printf("log1p(NAN) = %.15f\n", log1p(c));

/*
 * log1p(0.0) = 0.000000000000000
 * log1p(-0.0) = -0.000000000000000
 * log1p(-1) = -inf
 * log1p(-2) = nan
 * log1p(INFINITY) = inf
 * log1p(-INFINITY) = nan
 * log1p(NAN) = nan
 *
 * */
```

 版权所有 © 华为技术有限公司

6.4.3.2.3.12 logb

计算x的指数，以浮点形式返回。

接口定义

C interface：

float logbf(float x);

double logb(double x);

long double logbl(long double x);

参数

参数名	类型	描述	输入/输出
x	• 在logbf中，x是单精度浮点类型。 • 在logb中，x是双精度浮点类型。 • 在logbl中，x是长双精度浮点类型。	表示输入数据的浮点值。	输入

返回值

- 返回x的指数部分，单精度y ∈ [-150, 127]，双精度y ∈ [-1074, 1024]。
- 输入x为±∞，返回+∞。
- 输入x为±0，返回-∞。
- 输入x为nan，返回nan。

依赖

C: "km.h"

示例

C interface：

```
// typical usage
double x1 = 0.0, x2 = -0.0, x3 = 0x1p-1074, x4 = 0x1p1024;
float x5 = 0x1p-149f;
// special handling
double a = INFINITY, b = -INFINITY, c = NAN;
// print result
printf("logb(0.0) = %.15f\n", logb(x1));
printf("logb(-0.0) = %.15f\n", logb(x2));
printf("logb(0x1p-1074) = %.15f\n", logb(x3));
printf("logb(0x1p1024) = %.15f\n", logb(x4));
printf("logbf(0x1p-150) = %.15f\n", logbf(x5));
printf("logb(INFINITY) = %.15f\n", logb(a));
printf("logb(-INFINITY) = %.15f\n", logb(b));
printf("logb(NAN) = %.15f\n", logb(c));
/*
 * logb(0.0) = -inf
```


版权所有 © 华为技术有限公司

```
 * logb(-0.0) = -inf
 * logb(0x1p-1074) = -1074.000000000000000
 * logb(0x1p1024) = inf
 * logbf(0x1p-150) = -149.000000000000000
 * logb(INFINITY) = inf
 * logb(-INFINITY) = inf
 * logb(NAN) = nan
 *
 * */
```

6.4.3.2.3.13 ilogb

计算x的指数，返回整型结果。

接口定义

C interface：

int ilogbf(float x);

int ilogb(double x);

int ilogbl(long double x);

参数

参数名	类型	描述	输入/输出
x	• 在ilogbf中，x是单精度浮点类型。 • 在ilogb中，x是双精度浮点类型。 • 在ilogbl中，x是长双精度浮点类型。	表示输入数据的浮点值。	输入

返回值

- 输入x为±∞，返回INT_MAX。
- 输入x为±0，返回INT_MIN。
- 输入x为nan，返回INT_MAX。

依赖

C: "km.h"

示例

C interface：

```
// typical usage
double x1 = 0.0, x2 = -0.0, x3 = 0x1p-1074, x4 = 0x1p1024;
float x5 = 0x1p-149f;
// special handling
double a = INFINITY, b = -INFINITY, c = NAN;
// print result
printf("ilogb(0.0) = %d\n", ilogb(x1));
```

 版权所有 © 华为技术有限公司

```
printf("ilogb(-0.0) = %d\n", ilogb(x2));
printf("ilogb(0x1p-1074) = %d\n", ilogb(x3));
printf("ilogb(0x1p1024) = %d\n", ilogb(x4));
printf("ilogbf(0x1p-149) = %d\n", ilogbf(x5));
printf("ilogb(INFINITY) = %d\n", ilogb(a));
printf("ilogb(-INFINITY) = %d\n", ilogb(b));
printf("ilogb(NAN) = %d\n", ilogb(c));
/*
 * ilogb(0.0) = INT_MIN
 * ilogb(-0.0) = INT_MIN
 * ilogb(0x1p-1074) = -1074
 * ilogb(0x1p1024) = INT_MAX
 * ilogbf(0x1p-149) = -149
 * ilogb(INFINITY) = INT_MAX
 * ilogb(-INFINITY) = INT_MAX
 * ilogb(NAN) = INT_MAX
 *
 * */
```

6.4.3.2.3.14 scalb

计算x * 2^y的值。

接口定义

C interface：

float scalbf(float x, float y);

double scalb(double x, double y);

long double scalbl(long double x, long double y);

参数

参数名	类型	描述	输入/输出
x	• 在scalbf中，x是单精度浮点类型。 • 在scalb中，x是双精度浮点类型。 • 在scalbl中，x是长双精度浮点类型。	表示输入数据的浮点值。	输入
y	• 在scalbf中，y是单精度浮点类型。 • 在scalb中，y是双精度浮点类型。 • 在scalbl中，y是长双精度浮点类型。	表示输入数据的浮点值。	输入

返回值

- 输入x为nan，输入y为任意值，返回x*y。
- 输入y为nan或+∞，输入x为任意值，返回x*y。

 版权所有 © 华为技术有限公司

- 输入x为±0，y为-∞，返回x。
- 输入y为-∞，x非0，返回±0（符号与x相同）。
- 输入y为非整数，输入x为任意值，返回nan。

依赖

C: "km.h"

示例

C interface：

```
// typical usage
double x1 = 0.0, x2 = -0.0, x3 = 2.0, x4 = 1100.0;
double y1 = -0.0, y2 = 0.0, y3 = 3.0, y4 = 2.1;
// special handling
double a = INFINITY, b = -INFINITY, c = NAN;
printf("scalb(0.0, -0.0) = %.15f\n", scalb(x1, y1));
printf("scalb(-0.0, 0.0) = %.15f\n", scalb(x2, y2));
printf("scalb(2.0, 3.0) = %.15f\n", scalb(x3, y3));
printf("scalb(2.0, 2.1) = %.15f\n", scalb(x3, y4));
printf("scalb(0.0, -INFINITY) = %.15f\n", scalb(x1, b));
printf("scalb(-0.0, -INFINITY) = %.15f\n", scalb(y1, b));
printf("scalb(2.0, -INFINITY) = %.15f\n", scalb(x3, b));
printf("scalb(2.0, 1100.0) = %.15f\n", scalb(x3, x4));
printf("scalb(INFINITY, 2.0) = %.15f\n", scalb(a, x2));
printf("scalb(-INFINITY, 2.0) = %.15f\n", scalb(b, x2));
printf("scalb(NAN, 2.0) = %.15f\n", scalb(c, x3));
printf("scalb(NAN, INFINITY) = %.15f\n", scalb(c, a));
printf("scalb(NAN, -INFINITY) = %.15f\n", scalb(c, b));
printf("scalb(INFINITY, -INFINITY) = %.15f\n", scalb(a, b));
/*
 * scalb(0.0, -0.0) = 0.000000000000000
 * scalb(-0.0, 0.0) = -0.000000000000000
 * scalb(2.0, 3.0) = 16.000000000000000
 * scalb(2.0, 2.1) = nan
 * scalb(0.0, -INFINITY) = 0.000000000000000
 * scalb(-0.0, -INFINITY) = -0.000000000000000
 * scalb(2.0, -INFINITY) = 0.000000000000000
 * scalb(2.0, 1100.0) = inf
 * scalb(INFINITY, 2.0) = inf
 * scalb(-INFINITY, 2.0) = -inf
 * scalb(NAN, 2.0) = nan
 * scalb(NAN, INFINITY) = nan
 * scalb(NAN, -INFINITY) = nan
 * scalb(INFINITY, -INFINITY) = nan
 *
 * */
```

6.4.3.2.3.15 scalbln

计算x * 2^y的值。

接口定义

C interface：

float scalblnf(float x, long int y);

double scalbln(double x, long int y);

long double scalblnl(long double x, long int y);

 版权所有 © 华为技术有限公司

参数

参数名	类型	描述	输入/输出
x	• 在scalblnf中，x是单精度浮点类型。 • 在scalbln中，x是双精度浮点类型。 • 在scalblnl中，x是长双精度浮点类型。	表示输入数据的浮点值。	输入
y	• 在scalblnf中，y是长整数类型。 • 在scalbln中，y是长整数类型。 • 在scalblnl中，y是长整数类型。	表示输入数据的长整数值。	输入

返回值

- 返回x * 2^y的值，发生溢出时，返回±∞（符号与x保持一致）。

依赖

C: "km_l9.h"

示例

C interface：

```
// typical usage
double x1 = 0.0, x2 = -0.0, x3 = 2.0;
long int y1 = 2, y2 = 3, y3 = 4, y4 = 5, y5 = 1100;
// special handling
double a = INFINITY, b = -INFINITY, c = NAN;
printf("scalbln(0.0, 2) = %.15f\n", scalbln(x1, y1));
printf("scalbln(-0.0, 3) = %.15f\n", scalbln(x2, y2));
printf("scalbln(2.0, 4) = %.15f\n", scalbln(x3, y3));
printf("scalbln(2.0, 5) = %.15f\n", scalbln(x3, y4));
printf("scalbln(2.0, 1100) = %.15f\n", scalbln(x3, y5));
printf("scalbln(INFINITY, 2) = %.15f\n", scalbln(a, y1));
printf("scalbln(-INFINITY, 2) = %.15f\n", scalbln(b, y1));
printf("scalbln(NAN, 2) = %.15f\n", scalbln(c, y1));
/*
 * scalbln(0.0, 2) = 0.000000000000000
 * scalbln(-0.0, 3) = -0.000000000000000
 * scalbln(2.0, 4) = 32.000000000000000
 * scalbln(2.0, 5) = 64.000000000000000
 * scalbln(2.0, 1100) = inf
 * scalbln(INFINITY, 2) = inf
 * scalbln(-INFINITY, 2) = -inf
 * scalbln(NAN, 2) = nan
 */
```

6.4.3.2.3.16 scalbn

计算x * 2^y的值。

 版权所有 © 华为技术有限公司

接口定义

C interface：

float scalbnf(float x, int y);

double scalbn(double x, int y);

long double scalbnl(long double x, int y);

参数

参数名	类型	描述	输入/输出
x	• 在scalbnf中，x是单精度浮点类型。 • 在scalbn中，x是双精度浮点类型。 • 在scalbnl中，x是长双精度浮点类型。	表示输入数据的浮点值。	输入
y	• 在scalbnf中，y是整数类型。 • 在scalbn中，y是整数类型。 • 在scalbnl中，y是整数类型。	表示输入数据的整数值。	输入

返回值

- 返回x * 2^y的值，发生溢出时，返回±∞（符号与x保持一致）。

依赖

C: "km_l9.h"

示例

C interface：

```
// typical usage
double x1 = 0.0, x2 = -0.0, x3 = 2.0;
long int y1 = 2, y2 = 3, y3 = 4, y4 = 5, y5 = 1100;
// special handling
double a = INFINITY, b = -INFINITY, c = NAN;
printf("scalbn(0.0, 2) = %.15f\n", scalbn(x1, y1));
printf("scalbn(-0.0, 3) = %.15f\n", scalbn(x2, y2));
printf("scalbn(2.0, 4) = %.15f\n", scalbn(x3, y3));
printf("scalbn(2.0, 5) = %.15f\n", scalbn(x3, y4));
printf("scalbn(2.0, 1100) = %.15f\n", scalbn(x3, y5));
printf("scalbn(INFINITY, 2) = %.15f\n", scalbn(a, y1));
printf("scalbn(-INFINITY, 2) = %.15f\n", scalbn(b, y1));
printf("scalbn(NAN, 2) = %.15f\n", scalbn(c, y1));
/*
 * scalbn(0.0, 2) = 0.000000000000000
 * scalbn(-0.0, 3) = -0.000000000000000
 * scalbn(2.0, 4) = 32.000000000000000
 * scalbn(2.0, 5) = 64.000000000000000
 * scalbn(2.0, 1100) = inf
 * scalbn(INFINITY, 2) = inf
```


版权所有 © 华为技术有限公司

```
 * scalbn(-INFINITY, 2) = -inf
 * scalbn(NAN, 2) = nan
 */
```

6.4.3.2.4 幂和根函数

6.4.3.2.4.1 pow

计算x的y次幂。

接口定义

C interface：

float powf(float x, float y);

double pow(double x, double y);

long double powl(long double x, long double y);

Fortran interface：

RES = POWF(X, Y);

RES = POW(X, Y);

参数

参数名	类型	描述	输入/输出
x	• 在powf中，x是单精度浮点类型。 • 在pow中，x是双精度浮点类型。 • 在powl中，x是长双精度浮点类型。	表示输入数据的浮点值。	输入
y	• 在powf中，y是单精度浮点类型。 • 在pow中，y是双精度浮点类型。 • 在powl中，y是长双精度浮点类型。	表示输入数据的浮点值。	输入

返回值

- x大于0且y为小数，返回幂指数的结果，取值范围(-INF, +INF)。
- 输入x为-0，y为负的奇数，返回-∞；其他x为±0，y为负数，返回+∞。
- 输入x为±0，y为正的奇数，返回±0；其他x为±0，y为正数，返回+0。
- 输入x为±1，y为任意值，则返回+1。
- 输入x为任意值，y为±0，则返回+1。

 版权所有 © 华为技术有限公司

- 输入x小于+0，y为小数，则返回nan。
- 输入|x|小于1，y为-∞，则返回+∞；输入|x|大于1，y为-∞，则返回+0。
- 输入|x|小于1，y为+∞，则返回+0；输入|x|大于1，y为+∞，则返回+∞。
- 输入x为-∞，y为负奇数，则返回-0；其他x为-∞，y为负数，则返回+0。
- 输入x为-∞，y为正奇数，则返回-∞；其他x为-∞，y为正数，则返回+∞。
- 输入x为+∞，y小于+0，则返回+0；其他x为+∞，则返回+∞。
- 输入x为大的有限数，且y为大的有限数，则返回±∞。
- 输入x为nan，且y为nan，则返回nan。

依赖

C: "km.h"

示例

C interface：

```
double pi = acos(-1);
// typical usage
double x1 = 1.0, x2 = 2.0, x3 = 3.0, x4 = 4.0;
double y1 = 4.0, y2 = -4.0, y3 = -3.0, y4 = 3.0;
// special handling
double a = 0.0, b = INFINITY, c = -INFINITY, d = NAN, e = 1.0;
// print result
printf("pow(1.0, 4.0) = %.15f\n", pow(x1, y1));
printf("pow(2.0, -4.0) = %.15f\n", pow(x2, y2));
printf("pow(3.0, -3.0) = %.15f\n", pow(x3, y3));
printf("pow(4.0, 3.0) = %.15f\n", pow(x4, y4));
printf("pow(1.0, 0.0) = %+.15f\n", pow(x1, a));
printf("pow(INFINITY, 1.0) = %+.15f\n", pow(b, x1));
printf("pow(-INFINITY, -1.0) = %+.15f\n", pow(c, -x1));
printf("pow(1.0, -INFINITY) = %+.15f\n", pow(x1, c));
printf("pow(1.0, NAN) = %+.15f\n", pow(x1, d));
printf("pow(NAN, -1.0) = %+.15f\n", pow(d, -x1));
/*
 *  pow(1.0, 4.0) = 1.000000000000000
 *  pow(2.0, -4.0) = 0.062500000000000
 *  pow(3.0, -3.0) = 0.037037037037037
 *  pow(4.0, 3.0) = 64.000000000000000
 *  pow(1.0, 0.0) = +1.000000000000000
 *  pow(INFINITY, 1.0) = +inf
 *  pow(-INFINITY, -1.0) = -0.000000000000000
 *  pow(1.0, -INFINITY) = +1.000000000000000
 *  pow(1.0, NAN) = +1.000000000000000
 *  pow(NAN, -1.0) = +nan
 *
 * */
```

Fortran interface：

```
REAL(8) :: X = 3.0
REAL(8) :: Y = -3.0
PRINT*,  POW(X, Y)
!
! OUTPUT
!     0.037037037037037
!
```

6.4.3.2.4.2 powr

计算x的y次幂，其中x≥0。

 版权所有 © 华为技术有限公司

接口定义

C interface：

float powrf(float x, float y);

double powr(double x, double y);

参数

参数名	类型	描述	输入/输出
x	• 在powrf中，x是单精度浮点类型。 • 在powr中，x是双精度浮点类型。	表示输入数据的浮点值。	输入
y	• 在powrf中，y是单精度浮点类型。 • 在powr中，y是双精度浮点类型。	表示输入数据的浮点值。	输入

返回值

- x大于等于0且y为正常值，返回幂指数的结果，取值范围(0, +INF)。
- x小于0，返回nan。
- 其他特殊值组合如下所示。

输入值（x）	输入值（y）	输出值（result）
+0	+0	+1
+0	-0	+1
+0	inf	+0
+0	-inf	inf
+0	snan	nan
+0	-snan	nan
+0	qnan	nan
+0	-qnan	nan
-0	+0	+1
-0	-0	+1
-0	inf	+0
-0	-inf	inf
-0	snan	nan

 版权所有 © 华为技术有限公司

输入值（x）	输入值（y）	输出值（result）
-0	-snan	nan
-0	qnan	nan
-0	-qnan	nan
inf	+0	+1
inf	-0	+1
inf	inf	inf
inf	-inf	+0
inf	snan	nan
inf	-snan	nan
inf	qnan	nan
inf	-qnan	nan
-inf	+0	nan
-inf	-0	nan
-inf	inf	nan
-inf	-inf	nan
-inf	snan	nan
-inf	-snan	nan
-inf	qnan	nan
-inf	-qnan	nan
snan	+0	+1
snan	-0	+1
snan	inf	nan
snan	-inf	nan
snan	snan	nan
snan	-snan	nan
snan	qnan	nan
snan	-qnan	nan
-snan	+0	+1
-snan	-0	+1
-snan	inf	nan

版权所有 © 华为技术有限公司

输入值（x）	输入值（y）	输出值（result）
-snan	-inf	nan
-snan	snan	nan
-snan	-snan	nan
-snan	qnan	nan
-snan	-qnan	nan
qnan	+0	+1
qnan	-0	+1
qnan	inf	nan
qnan	-inf	nan
qnan	snan	nan
qnan	-snan	nan
qnan	qnan	nan
qnan	-qnan	nan
-qnan	+0	+1
-qnan	-0	+1
-qnan	inf	nan
-qnan	-inf	nan
-qnan	snan	nan
-qnan	-snan	nan
-qnan	qnan	nan
-qnan	-qnan	nan

依赖

C: "km.h"

示例

C interface：

```
// typical usage
double x1 = 0.0, x2 = -0.0, x3 = 1.5, x4 = -2.5;
// special handling
double a = INFINITY, b = -INFINITY, c = NAN;
// print result
printf("powr(0.0, -0.0) = %.15f\n", powr(x1, x2));
printf("powr(-0.0, 0.0) = %.15f\n", powr(x2, x1));
printf("powr(1.5, 1.5) = %.15f\n", powr(x3, x3));
```

 版权所有 © 华为技术有限公司

```
printf("powr(-2.5, 1.5) = %.15f\n", powr(x4, x3));
printf("powr(INFINITY, 1.5) = %.15f\n", powr(a, x3));
printf("powr(-INFINITY, 1.5) = %.15f\n", powr(b, x3));
printf("powr(NAN, NAN) = %.15f\n", powr(c, c));
return 0;

/*
 * powr(0.0, -0.0) = 1.000000000000000
 * powr(-0.0, 0.0) = 1.000000000000000
 * powr(1.5, 1.5) = 1.837117307087384
 * powr(-2.5, 1.5) = nan
 * powr(INFINITY, 1.5) = inf
 * powr(-INFINITY, 1.5) = nan
 * powr(NAN, NAN) = nan
 */
```

6.4.3.2.4.3 powr?i?/powi?i?/powc?i?

计算不同类别的参数的幂函数值。

接口定义

C interface：

complex double powc16i4(complex double x, int y);

complex double powc16i4_18(complex double x, int y);

int powi4i4(int x, int y);

int powi4i4_18(int x, int y);

int64_t powi8i4(int64_t x, int y);

int64_t powi8i4_18(int64_t x, int y);

int64_t powi8i8(int64_t x, int64_t y);

float powr4i4(float x, int y);

double powr8i4(double x, int y);

long double powr16i4(long double x, int y);

说明

powc16i4_18、powi4i4_18仅在高精度版本提供。

依赖

C: "km.h"

示例

C interface：

```
// typical usage
 printf("powi4i4(1, 2) = %d\n", powi4i4(1, 2));
 printf("powi4i4(3, 4) = %d\n", powi4i4(3, 4));
 printf("powr4i4(1.0, 2) = %.15f\n", powr4i4(1.0, 2));
 printf("powr4i4(3.0, 4) = %.15f\n", powr4i4(3.0, 4));
 printf("powr4i4(INFINITY, 2) = %.15f\n", powr4i4(INFINITY, 2));
 printf("powr4i4(INFINITY, 3) = %.15f\n", powr4i4(INFINITY, 3));
```

 版权所有 © 华为技术有限公司

```
printf("powr4i4(-INFINITY, 3) = %.15f\n", powr4i4(-INFINITY, 3));

/*
 * powi4i4(1, 2) = 1
 * powi4i4(3, 4) = 81
 * powr4i4(1.0, 2) = 1.000000000000000
 * powr4i4(3.0, 4) = 81.000000000000000
 * powr4i4(INFINITY, 2) = inf
 * powr4i4(INFINITY, 3) = inf
 * powr4i4(-INFINITY, 3) = -inf
 *
 * */
```

6.4.3.2.4.4 pow2o3

计算x的2/3次幂。

接口定义

C interface：

float pow2o3f(float x);

double pow2o3(double x);

参数

参数名	类型	描述	输入/输出
x	• 在pow2o3f中，x是单精度浮点类型。 • 在pow2o3中，x是双精度浮点类型。	表示输入数据的浮点值。	输入

返回值

- 输入x为±0，返回+0。
- 输入x为+∞，返回+∞。
- 输入x为-∞，返回+∞。
- 输入x为nan，返回nan。

依赖

C: "km.h"

示例

C interface：

```
// typical usage
double x1 = 0.0, x2 = -0.0, x3 = 1.5, x4 = -2.5;
// special handling
double a = INFINITY, b = -INFINITY, c = NAN;
// print result
printf("pow2o3(0.0) = %.15f\n", pow2o3(x1));
printf("pow2o3(-0.0) = %.15f\n", pow2o3(x2));
printf("pow2o3(1.5) = %.15f\n", pow2o3(x3));
```


版权所有 © 华为技术有限公司

```
printf("pow2o3(-2.5) = %.15f\n", pow2o3(x4));
printf("pow2o3(INFINITY) = %.15f\n", pow2o3(a));
printf("pow2o3(-INFINITY) = %.15f\n", pow2o3(b));
printf("pow2o3(NAN) = %.15f\n", pow2o3(c));
/*
 *  pow2o3(0.0) = 0.000000000000000
 *  pow2o3(-0.0) = 0.000000000000000
 *  pow2o3(1.5) = 1.310370697104448
 *  pow2o3(-2.5) = 1.842015749320193
 *  pow2o3(INFINITY) = inf
 *  pow2o3(-INFINITY) = inf
 *  pow2o3(NAN) = nan
 * */
```

6.4.3.2.4.5 pow3o2

计算x的3/2次幂。

接口定义

C interface：

float pow3o2f(float x);

double pow3o2(double x);

参数

参数名	类型	描述	输入/输出
x	• 在pow3o2f中，x是单精度浮点类型。 • 在pow3o2中，x是双精度浮点类型。	表示输入数据的浮点值。	输入

返回值

- 输入x为+0，返回+0。
- 输入x为-0，返回-0。
- 输入x为+∞，返回+∞。
- 输入x为-∞，返回nan。
- 输入x为nan，返回nan。

依赖

C: "km.h"

示例

C interface：

```
// typical usage
double x1 = 0.0, x2 = -0.0, x3 = 1.5, x4 = -2.5;
// special handling
double a = INFINITY, b = -INFINITY, c = NAN;
// print result
```

 版权所有 © 华为技术有限公司

```
printf("pow3o2(0.0) = %.15f\n", pow3o2(x1));
printf("pow3o2(-0.0) = %.15f\n", pow3o2(x2));
printf("pow3o2(1.5) = %.15f\n", pow3o2(x3));
printf("pow3o2(-2.5) = %.15f\n", pow3o2(x4));
printf("pow3o2(INFINITY) = %.15f\n", pow3o2(a));
printf("pow3o2(-INFINITY) = %.15f\n", pow3o2(b));
printf("pow3o2(NAN) = %.15f\n", pow3o2(c));
/*
 *  pow3o2(0.0) = 0.000000000000000
 *  pow3o2(-0.0) = -0.000000000000000
 *  pow3o2(1.5) = 1.837117307087384
 *  pow3o2(-2.5) = nan
 *  pow3o2(INFINITY) = inf
 *  pow3o2(-INFINITY) = nan
 *  pow3o2(NAN) = nan
 * */
```

6.4.3.2.4.6 sqrt

计算x的平方根。

接口定义

C interface：

float sqrtf(float x);

double sqrt(double x);

long double sqrtl(long double x);

Fortran interface：

RES = SQRTF(X);

RES = SQRT(X);

参数

参数名	类型	描述	输入/输出
x	• 在sqrtf中，x是单精度浮点类型。 • 在sqrt中，x是双精度浮点类型。 • 在sqrtl中，x是长双精度浮点类型。	表示输入数据的浮点值。	输入

返回值

- 输入x为±0，返回±0。
- 输入x为+∞，返回+∞。
- 输入x为-∞，返回nan。
- 输入x为nan，返回nan。

版权所有 © 华为技术有限公司

依赖

C: "km.h"

示例

C interface：

```
// typical usage
double x1 = 1.0, x2 = 2.0, x3 = 3.0, x4 = 4.0;
// special handling
double a = 0.0, b = INFINITY, c = -INFINITY, d = NAN;
// print result
    printf("sqrt(1.0) = %.15f\n", sqrt(x1));
    printf("sqrt(2.0) = %.15f\n", sqrt(x2));
    printf("sqrt(3.0) = %.15f\n", sqrt(x3));
    printf("sqrt(4.0) = %.15f\n", sqrt(x4));
    printf("sqrt(0.0) = %.15f\n", sqrt(a));
    printf("sqrt(INFINITY) = %.15f\n", sqrt(b));
    printf("sqrt(-INFINITY) = %.15f\n", sqrt(c));
    printf("sqrt(NAN) = %.15f\n", sqrt(d));
/*
 *  sqrt(1.0) = 1.000000000000000
 *  sqrt(2.0) = 1.414213562373095
 *  sqrt(3.0) = 1.732050807568877
 *  sqrt(4.0) = 2.000000000000000
 *  sqrt(0.0) = 0.000000000000000
 *  sqrt(INFINITY) = inf
 *  sqrt(-INFINITY) = -inf
 *  sqrt(NAN) = nan
 * */
```

Fortran interface：

```
REAL(8) :: X = 3.0
PRINT*,  SQRT(X)
!
! OUTPUT
!     1.732050807568877
!
```

6.4.3.2.4.7 invsqrt

计算x的平方根的倒数。

接口定义

C interface：

float invsqrtf(float x);

double invsqrt(double x);

long double invsqrtl(long double x);

Fortran interface：

RES = SQRTF(X);

RES = SQRT(X);

 版权所有 © 华为技术有限公司

参数

参数名	类型	描述	输入/输出
x	• 在invsqrtf中，x是单精度浮点类型。 • 在invsqrt中，x是双精度浮点类型。 • 在invsqrtl中，x是扩展精度浮点类型。	表示输入数据的浮点值。	输入

返回值

- 输入x为+0，返回+∞。
- 输入x为-0，返回-∞。
- 输入x<0，返回nan。
- 输入x为+∞，返回0.0。
- 输入x为nan，返回nan。

依赖

C: "km.h"

示例

C interface：

```
// typical usage
double x1 = 1.0, x2 = 2.0, x3 = 3.0, x4 = 4.0;
// special handling
double a = 0.0, b = INFINITY, c = -INFINITY, d = NAN;
// print result
    printf("invsqrt(1.0) = %.15f\n", invsqrt(x1));
    printf("invsqrt(2.0) = %.15f\n", invsqrt(x2));
    printf("invsqrt(3.0) = %.15f\n", invsqrt(x3));
    printf("invsqrt(4.0) = %.15f\n", invsqrt(x4));
    printf("invsqrt(0.0) = %.15f\n", invsqrt(a));
    printf("invsqrt(INFINITY) = %.15f\n", invsqrt(b));
    printf("invsqrt(-INFINITY) = %.15f\n", invsqrt(c));
    printf("invsqrt(NAN) = %.15f\n", invsqrt(d));
/*
 *  invsqrt(1.0) = 1.000000000000000
 *  invsqrt(2.0) = 0.707106781186548
 *  invsqrt(3.0) = 0.577350269189626
 *  invsqrt(4.0) = 0.500000000000000
 *  invsqrt(0.0) = inf
 *  invsqrt(INFINITY) = 0.000000000000000
 *  invsqrt(-INFINITY) = nan
 *  invsqrt(NAN) = nan
 * */
```

6.4.3.2.4.8 cbrt

计算x的立方根。

 版权所有 © 华为技术有限公司

接口定义

C interface：

float cbrtf(float x);

double cbrt(double x);

double cbrt_18(double x);

long double cbrtl(long double x);

说明

cbrt_18仅在高精度版本提供。

Fortran interface：

RES = CBRTF(X);

RES = CBRT(X);

参数

参数名	类型	描述	输入/输出
x	• 在cbrtf中，x是单精度浮点类型。 • 在cbrt和cbrt_18中，x是双精度浮点类型。 • 在cbrtl中，x是长双精度浮点类型。	表示输入数据的浮点值。	输入

返回值

- 输入x为±0，返回±0。
- 输入x为±∞，返回±∞。
- 输入x为nan，返回nan。

依赖

C: "km.h"

示例

C interface：

```
// typical usage
double x1 = 1.0, x2 = 2.0, x3 = 3.0, x4 = 4.0;
// special handling
double a = 0.0, b = INFINITY, c = -INFINITY, d = NAN;
// print result
    printf("cbrt(1.0) = %.15f\n", cbrt(x1));
    printf("cbrt(2.0) = %.15f\n", cbrt(x2));
    printf("cbrt(3.0) = %.15f\n", cbrt(x3));
    printf("cbrt(4.0) = %.15f\n", cbrt(x4));
```


版权所有 © 华为技术有限公司

```
    printf("cbrt(0.0) = %.15f\n", cbrt(a));
    printf("cbrt(INFINITY) = %.15f\n", cbrt(b));
    printf("cbrt(-INFINITY) = %.15f\n", cbrt(c));
    printf("cbrt(NAN) = %.15f\n", cbrt(d));
/*
 *
 *  cbrt(1.0) = 1.000000000000000
 *  cbrt(2.0) = 1.259921049894873
 *  cbrt(3.0) = 1.442249570307408
 *  cbrt(4.0) = 1.587401051968200
 *  cbrt(0.0) = 0.000000000000000
 *  cbrt(INFINITY) = inf
 *  cbrt(-INFINITY) = -inf
 *  cbrt(NAN) = nan

 * */
```

Fortran interface：

```
REAL(8) :: X = 3.0
PRINT*,  CBRT(X)
!
! OUTPUT
!     1.442249570307408
!
```

6.4.3.2.5 杂项函数

6.4.3.2.5.1 fabs

求浮点数入参的绝对值。

接口定义

C interface：

float fabsf(float x);

double fabs(double x);

参数

参数名	类型	描述	输入/输出
x	• 在fabsf中，x是单精度浮点类型。 • 在fabs中，x是双精度浮点类型。	表示输入数据的浮点值。	输入

返回值

- 输入x为±0，返回+0。
- 输入x为±∞，返回+∞。
- 输入x为±nan，则返回nan。

 版权所有 © 华为技术有限公司

依赖

C: "km.h"

示例

C interface：

```
  // typical usage
  printf("fabs(1.0) = %.15f\n", fabs(1.0));
  printf("fabs(-3.0) = %.15f\n", fabs(-3.0));
  // special handling
  printf("fabs(NAN) = %.15f\n", fabs(NAN));
  printf("fabs(-NAN) = %.15f\n", fabs(-NAN));
  printf("fabs(INFINITY) = %.15f\n", fabs(INFINITY));
  printf("fabs(-INFINITY) = %.15f\n", fabs(-INFINITY));

result
 /*
  * fabs(1.0) = 1.000000000000000
  * fabs(-3.0) = 3.000000000000000
  * fabs(NAN) = nan
  * fabs(-NAN) = nan
  * fabs(INFINITY) = inf
  * fabs(-INFINITY) = inf
  * */
```

6.4.3.2.5.2 fdim

如果x≤y，则fdim函数返回零值，否则返回x - y的值。

接口定义

C interface：

float fdimf(float x, float y);

double fdim(double x, double y);

参数

参数名	类型	描述	输入/输出
x	• 在fdimf中，x是单精度浮点类型。 • 在fdim中，x是双精度浮点类型。	表示输入数据的浮点值。	输入
y	• 在fdimf中，y是单精度浮点类型。 • 在fdim中，y是双精度浮点类型。	表示输入数据的浮点值。	输入

返回值

- errno：返回值为inf时，置ERANGE。
- 输入x<y时，返回0。

 版权所有 © 华为技术有限公司

- 输入x>=y时，返回x-y的值。
- 输入x或y为nan时，返回nan。

依赖

C: "km.h"

示例

C interface：

```
// typical usage
printf("fdim(1.0, 2.0) = %.15f\n", fdim(1.0, 2.0));
printf("fdim(3.0, 2.0) = %.15f\n", fdim(3.0, 2.0));
// special handling
printf("fdim(INFINITY, INFINITY) = %.15f\n", fdim(INFINITY, INFINITY));
printf("fdim(INFINITY, -INFINITY) = %.15f\n", fdim(INFINITY, -INFINITY));
printf("fdim(-INFINITY, INFINITY) = %.15f\n", fdim(-INFINITY, INFINITY));
printf("fdim(-INFINITY, -INFINITY) = %.15f\n", fdim(-INFINITY, -INFINITY));
printf("fdim(NAN, -INFINITY) = %.15f\n", fdim(NAN, -INFINITY));

result
 /*
  * fdim(1.0, 2.0) = 0.000000000000000
  * fdim(3.0, 2.0) = 1.000000000000000
  * fdim(INFINITY, INFINITY) = 0.000000000000000
  * fdim(INFINITY, -INFINITY) = inf
  * fdim(-INFINITY, INFINITY) = 0.000000000000000
  * fdim(-INFINITY, -INFINITY) = 0.000000000000000
  * fdim(NAN, -INFINITY) = nan
  * */
```

6.4.3.2.5.3 fmax

返回入参的最大值。

接口定义

C interface：

float fmaxf(float x, float y);

double fmax(double x, double y);

参数

参数名	类型	描述	输入/输出
x	• 在fmaxf中，x是单精度浮点类型。 • 在fmax中，x是双精度浮点类型。	表示输入数据的浮点值。	输入
y	• 在fmaxf中，y是单精度浮点类型。 • 在fmax中，y是双精度浮点类型。	表示输入数据的浮点值。	输入

版权所有 © 华为技术有限公司

返回值

- 返回x，y中的较大值。
- 其他特殊值参考如下说明。

输入值（x）	输入值（y）	返回值
any	qnan	x
qnan	any	y
any	snan	nan
snan	any	nan

依赖

C: "km.h"

示例

C interface：

```
// typical usage
double x1 = 0.0, x2 = -0.0, x3 = 1.4, x4 = -2.7;
double y1 = -0.0, y2 = 0.0, y3 = 1.5, y4 = -2.1;
// special handling
double a = INFINITY, b = -INFINITY, c = NAN;
printf("fmax(0.0, -0.0) = %.15f\n", fmax(x1, y1));
printf("fmax(-0.0, 0.0) = %.15f\n", fmax(x2, y2));
printf("fmax(1.4, 1.5) = %.15f\n", fmax(x3, y3));
printf("fmax(-2.7, -2.1) = %.15f\n", fmax(x4, y4));
printf("fmax(INFINITY, 1.4) = %.15f\n", fmax(a, x3));
printf("fmax(-INFINITY, -2.7) = %.15f\n", fmax(b, x4));
printf("fmax(NAN, 1.4) = %.15f\n", fmax(c, x3));
printf("fmax(NAN, INFINITY) = %.15f\n", fmax(c, a));
printf("fmax(NAN, -INFINITY) = %.15f\n", fmax(c, b));
printf("fmax(INFINITY, -INFINITY) = %.15f\n", fmax(a, b));
/*
 * fmax(0.0, -0.0) = 0.000000000000000
 * fmax(-0.0, 0.0) = 0.000000000000000
 * fmax(1.4, 1.5) = 1.500000000000000
 * fmax(-2.7, -2.1) = -2.100000000000000
 * fmax(INFINITY, 1.4) = inf
 * fmax(-INFINITY, -2.7) = -2.700000000000000
 * fmax(NAN, 1.4) = 1.400000000000000
 * fmax(NAN, INFINITY) = inf
 * fmax(NAN, -INFINITY) = -inf
 * fmax(INFINITY, -INFINITY) = inf
 * */
```

6.4.3.2.5.4 fmaxmag

返回绝对值相对较大的入参，否则行为类似于fmax(x, y)。

接口定义

C interface：

float fmaxmagf(float x, float y);

double fmaxmag(double x, double y);

 版权所有 © 华为技术有限公司

参数

参数名	类型	描述	输入/输出
x	• 在fmaxmagf中，x是单精度浮点类型。 • 在fmaxmag中，x是双精度浮点类型。	表示输入数据的浮点值。	输入
y	• 在fmaxmagf中，y是单精度浮点类型。 • 在fmaxmag中，y是双精度浮点类型。	表示输入数据的浮点值。	输入

返回值

从其两个参数x和y中返回较大幅度的值。如果|x| > |y|它返回x；如果|y| > |x|它返回y；否则它的行为类似于fmax(x,y)。

- 输入x，y均为nan，返回+nan。
- 输入x为nan，y为任意非nan值，返回y。
- 输入x为任意非nan值，y为nan，则返回x。

依赖

C: "km.h"

示例

C interface：

```
// typical usage
printf("fmaxmag(1.0, 2.0) = %.15f\n", fmaxmag(1.0, 2.0));
printf("fmaxmag(3.0, 2.0) = %.15f\n", fmaxmag(3.0, 2.0));
// special handling
printf("fmaxmag(NAN, -NAN) = %.15f\n", fmaxmag(NAN, -NAN));
printf("fmaxmag(NAN, -INFINITY) = %.15f\n", fmaxmag(NAN, -INFINITY));

result
/*
 * fmaxmag(1.0, 2.0) = 2.000000000000000
 * fmaxmag(3.0, 2.0) = 3.000000000000000
 * fmaxmag(NAN, -NAN) = nan
 * fmaxmag(NAN, -INFINITY) = -inf
 * */
```

6.4.3.2.5.5 fmin

返回入参的最小值。

接口定义

C interface：

float fminf(float x, float y);

 版权所有 © 华为技术有限公司

double fmin(double x, double y);

参数

参数名	类型	描述	输入/输出
x	• 在fminf中，x是单精度浮点类型。 • 在fmin中，x是双精度浮点类型。	表示输入数据的浮点值。	输入
y	• 在fminf中，y是单精度浮点类型。 • 在fmin中，y是双精度浮点类型。	表示输入数据的浮点值。	输入

返回值

- 返回x，y中的较小值。
- 其他特殊值参考如下说明。

输入值（x）	输入值（y）	返回值
any	qnan	x
qnan	any	y
any	snan	nan
snan	any	nan

依赖

C: "km.h"

示例

C interface：

```
// typical usage
double x1 = 0.0, x2 = -0.0, x3 = 1.4, x4 = -2.7;
double y1 = -0.0, y2 = 0.0, y3 = 1.5, y4 = -2.1;
// special handling
double a = INFINITY, b = -INFINITY, c = NAN;
printf("fmin(0.0, -0.0) = %.15f\n", fmin(x1, y1));
printf("fmin(-0.0, 0.0) = %.15f\n", fmin(x2, y2));
printf("fmin(1.4, 1.5) = %.15f\n", fmin(x3, y3));
printf("fmin(-2.7, -2.1) = %.15f\n", fmin(x4, y4));
printf("fmin(INFINITY, 1.4) = %.15f\n", fmin(a, x3));
printf("fmin(-INFINITY, -2.7) = %.15f\n", fmin(b, x4));
```

 版权所有 © 华为技术有限公司

```
printf("fmin(NAN, 1.4) = %.15f\n", fmin(c, x3));
printf("fmin(NAN, INFINITY) = %.15f\n", fmin(c, a));
printf("fmin(NAN, -INFINITY) = %.15f\n", fmin(c, b));
printf("fmin(INFINITY, -INFINITY) = %.15f\n", fmin(a, b));
/*
 *fmin(0.0, -0.0) = -0.000000000000000
 *fmin(-0.0, 0.0) = -0.000000000000000
 *fmin(1.4, 1.5) = 1.400000000000000
 *fmin(-2.7, -2.1) = -2.700000000000000
 *fmin(INFINITY, 1.4) = 1.400000000000000
 *fmin(-INFINITY, -2.7) = -inf
 *fmin(NAN, 1.4) = 1.400000000000000
 *fmin(NAN, INFINITY) = inf
 *fmin(NAN, -INFINITY) = -inf
 *fmin(INFINITY, -INFINITY) = -inf
 * */
```

6.4.3.2.5.6 fminmag

返回绝对值相对较小的参数，否则行为类似于fmin(x, y)。

接口定义

C interface：

float fminmagf(float x, float y);

double fminmag(double x, double y);

参数

参数名	类型	描述	输入/输出
x	• 在fminmagf中，x是单精度浮点类型。 • 在fminmag中，x是双精度浮点类型。	表示输入数据的浮点值。	输入
y	• 在fminmagf中，y是单精度浮点类型。 • 在fminmag中，y是双精度浮点类型。	表示输入数据的浮点值。	输入

返回值

从其两个参数x和y中返回较小幅度的值。如果|x| > |y|函数返回y；如果|y| > |x|函数返回x；否则函数的行为类似于fmin(x,y)。

- 输入x，y均为nan，返回+nan。
- 输入x为nan，y为任意非nan值，返回y。
- 输入x为任意非nan值，y为nan，则返回x。

依赖

C: "km.h"

版权所有 © 华为技术有限公司

示例

C interface：

```
// typical usage
printf("fminmag(1.0, 2.0) = %.15f\n", fminmag(1.0, 2.0));
printf("fminmag(3.0, 2.0) = %.15f\n", fminmag(3.0, 2.0));
// special handling
printf("fminmag(NAN, -NAN) = %.15f\n", fminmag(NAN, -NAN));
printf("fminmag(NAN, -INFINITY) = %.15f\n", fminmag(NAN, -INFINITY));

result
/*
 * fminmag(1.0, 2.0) = 2.000000000000000
 * fminmag(3.0, 2.0) = 3.000000000000000
 * fminmag(NAN, -NAN) = nan
 * fminmag(NAN, -INFINITY) = -inf
 * */
```

6.4.3.2.5.7 nan

返回输入要求的浮点nan。

接口定义

C interface：

float nanf(const char *tagp);

double nan(const char *tagp);

参数

参数名	类型	描述	输入/输出
tagp	• 在nanf中，tagp是字符串指针类型。 • 在nan中，tagp是字符串指针类型。	表示输入数据。	输入

返回值

当tagp不为NULL时，返回浮点NAN，如果为NULL，则出现未定义行为，调用者需保证入参不为NULL。

依赖

C: "km.h"

示例

C interface：

```
// typical usage
const char *a = "0x7ff8000000000000";
const char *b = "0x781";
const char *c = "0x781asdf";
printf("/*\n");
```

 版权所有 © 华为技术有限公司

```
printf(" * nan(%s)=0x%lx\n", a, Asuint64(nan(a)));
printf(" * nan(%s)=0x%lx\n", b, Asuint64(nan(b)));
printf(" * nan(%s)=0x%lx\n", c, Asuint64(nan(c)));
printf(" **/\n");
/*
 * nan(0x7ff8000000000000)=0x7ff8000000000000
 * nan(0x781)=0x7ff8000000000781
 * nan(0x781asdf)=0x7ff8000000000000
 **/
```

6.4.3.2.5.8 nextafter

在y方向上，以指定格式返回x之后的下一个可表示值。

接口定义

C interface：

float nextafterf(float x, float y);

double nextafter(double x, double y);

参数

参数名	类型	描述	输入/输出
x	• 在nextafterf中，x是单精度浮点类型。 • 在nextafter中，x是双精度浮点类型。	表示输入数据的浮点值。	输入
y	• 在nextafterf中，y是单精度浮点类型。 • 在nextafter中，y是双精度浮点类型。	表示输入数据的浮点值。	输入

返回值

- 在y方向上，以指定格式返回x之后的下一个可表示值。
- errno：对于溢出场景，置ERANGE。
- 特殊值说明如下所示。

输入值（x）	输入值（y）	输出值（result）
非nan数值	nan	nan
nan	非nan数值	nan
nan	nan	nan
inf	inf	inf
-inf	-inf	-inf

版权所有 © 华为技术有限公司

输入值（x）	输入值（y）	输出值（result）
inf	-inf	0x7feffffffffffff(double) 0x7f7fffff(float)
-inf	inf	0xffeffffffffffff(double) 0xff7fffff(float)

依赖

C: "km.h"

示例

C interface：

```
// typical usage
double x1 = 1.0, y1 = 2.0, x2 = 3.0, y2 = 4.0;
// print result
printf("nextafter(x1, y1) = %.15f\n", nextafter(x1, y1));
printf("nextafter(x2, y2) = %.15f\n", nextafter(x2, y2));
/*
 * nextafter(x1, y1) = 1.000000000000000
 * nextafter(x2, y2) = 3.000000000000000
 */
```

6.4.3.2.5.9 nexttoward

计算x在y方向的下个浮点数。

接口定义

C interface：

float nexttowardf(float x, long double y);

double nexttoward(double x, long double y);

参数

参数名	类型	描述	输入/输出
x	• 在nexttowardf中，x是单精度浮点类型。 • 在nexttoward中，x是双精度浮点类型。	表示输入数据的浮点值。	输入
y	• 在nexttowardf中，y是long double浮点类型。 • 在nexttoward中，y是long double浮点类型。	表示输入数据的浮点值。	输入

版权所有 © 华为技术有限公司

返回值

返回x在y方向上的下一个浮点值。

- 输入x=y，返回x。
- 输入x或者y等于nan，返回nan。

依赖

C: "km.h"

示例

C interface：

```
// typical usage
double x1 = 1.0, x2 = 1.0;
double y1 = 1.0, y2 = 1.1;
double z1 = 1.0, z2 = 0.9;
// special handling
double e = INFINITY, f = -INFINITY, g = NAN;
// print result
printf("\t/*\n");
printf("\t * nexttoward(1.0, 1.0) = %a\n", nexttoward(x1, (long double)y1));
printf("\t * nexttoward(1.0, 1.1) = %a\n", nexttoward(y1, (long double)y2));
printf("\t * nexttoward(1.0, 0.9) = %a\n", nexttoward(z1, (long double)z2));
printf("\t * nexttoward(INFINITY, 0) = %a\n", nexttoward(e, 0.0));
printf("\t * nexttoward(-INFINITY, 1.0) = %a\n", nexttoward(f, x1));
printf("\t * nexttoward(NAN, 1.0) = %a\n", nexttoward(g, x1));
printf("\t * nexttoward(NAN, INFINITY) = %a\n", nexttoward(g, e));
printf("\t */\n");
return 0;
/*
 * nexttoward(1.0, 1.0) = 0x1p+0
 * nexttoward(1.0, 1.1) = 0x1.0000000000001p+0
 * nexttoward(1.0, 0.9) = 0x1.fffffffffffffp-1
 * nexttoward(INFINITY, 0) = 0x1.fffffffffffffp+1023
 * nexttoward(-INFINITY, 1.0) = -0x1.fffffffffffffp+1023
 * nexttoward(NAN, 1.0) = nan
 * nexttoward(NAN, INFINITY) = nan
 */
```

6.4.3.2.5.10 significand

以浮点形式返回x的有效数字。

接口定义

C interface：

float significandf(float x);

double significand(double x);

 版权所有 © 华为技术有限公司

参数

参数名	类型	描述	输入/输出
x	• 在significandf中，x是单精度浮点类型。 • 在significand中，x是双精度浮点类型。	表示输入数据的浮点值。	输入

返回值

- 返回x的有效数字，|y| ∈ [1, 2)。
- 输入+0，返回+0。
- 输入-0，返回-0。
- 输入±∞，返回±∞。
- 输入nan，返回nan。

依赖

C: "km.h"

示例

C interface：

```
// typical usage
double x1 = 0.0, x2 = -0.0, x3 = 1.5, x4 = -2.5, x5 = 0x1p-1024;
// special handling
double a = INFINITY, b = -INFINITY, c = NAN;
// print result
printf("significand(0.0) = %.15f\n", significand(x1));
printf("significand(-0.0) = %.15f\n", significand(x2));
printf("significand(1.5) = %.15f\n", significand(x3));
printf("significand(-2.5) = %.15f\n", significand(x4));
printf("significand(0x1p-1024) = %.15f\n", significand(x5));
printf("significand(INFINITY) = %.15f\n", significand(a));
printf("significand(-INFINITY) = %.15f\n", significand(b));
printf("significand(NAN) = %.15f\n", significand(c));
/*
 * significand(0.0) = 0.000000000000000
 * significand(-0.0) = -0.000000000000000
 * significand(1.5) = 1.500000000000000
 * significand(-2.5) = -1.250000000000000
 * significand(0x1p-1024) = 1.000000000000000
 * significand(INFINITY) = inf
 * significand(-INFINITY) = -inf
 * significand(NAN) = nan
 *
 * */
```

6.4.3.2.5.11 finite

判断输入值x是否为有限数，返回值类型为整型数。

接口定义

C interface：

 版权所有 © 华为技术有限公司

int finitef(float x);

int finite(double x);

参数

参数名	类型	描述	输入/输出
x	• 在finitef中，x是单精度浮点类型。 • 在finite中，x是双精度浮点类型。	表示输入数据的浮点值。	输入

返回值

- 返回判断x是否为有限数。
- 输入浮点数，返回1。
- 输入±∞，返回0。
- 输入nan，返回0。

依赖

C: "km.h"

示例

C interface：

```
// typical usage
double x1 = 0.0, x2 = -0.0, x3 = 1.5, x4 = -2.5;
// special handling
double a = INFINITY, b = -INFINITY, c = NAN;
// print result
printf("finite(0.0) = %d\n", finite(x1));
printf("finite(-0.0) = %d\n", finite(x2));
printf("finite(1.5) = %d\n", finite(x3));
printf("finite(-2.5) = %d\n", finite(x4));
printf("finite(INFINITY) = %d\n", finite(a));
printf("finite(-INFINITY) = %d\n", finite(b));
printf("finite(NAN) = %d\n", finite(c));
/*
 * finite(0.0) = 1
 * finite(-0.0) = 1
 * finite(1.5) = 1
 * finite(-2.5) = 1
 * finite(INFINITY) = 0
 * finite(-INFINITY) = 0
 * finite(NAN) = 0
 *
 * */
```

6.4.3.2.6 特殊函数

6.4.3.2.6.1 annuity

计算年金现值系数，输入参数为实数。

 版权所有 © 华为技术有限公司

接口定义

C interface：

float annuityf(float x, float y);

double annuity(double x, double y);

参数

参数名	类型	描述	输入/输出
x	• 在annuityf中，x是单精度浮点类型。 • 在annuity中，x是双精度浮点类型。	表示输入数据的浮点值。	输入
y	• 在annuityf中，y是单精度浮点类型。 • 在annuity中，y是双精度浮点类型。	表示输入数据的浮点值。	输入

返回值

- x大于-1且y ∈(-∞, +∞)，返回计算结果。
- 输入x为任意值，y为±0，返回0。
- 输入x为nan，y为任意值，返回nan。
- 输入x为任意值，y为nan，返回nan。
- 输入x为+∞，y为-1，则返回-1；y < -1，则返回-∞；y ∈(-1, 0)，则返回-0。
- 输入x为0，y != 0，则返回y。
- 输入x为-1，y < 0，则返回-1；y > 0，则返回+∞。
- 输入x < -1，y != 0，则返回nan。
- 输入y为-∞，x ∈(-1， 0)，则返回1.0 / x；x > 0，则返回-∞。
- 输入y为+∞，x ∈(-1， 0)，则返回+∞；x > 0，则返回1.0 / x。

依赖

C: "km.h"

示例

C interface：

```
// print result
printf("annuity(1.0, 10.0) = %.15f\n", annuity(1.0, 10.0));
printf("annuity(0.5, -3.0) = %.15f\n", annuity(0.5, -3.0));
printf("annuity(0.3, -10.0) = %.15f\n", annuity(0.3, -10.0));
printf("annuity(4, 2.0) = %.15f\n", annuity(4, 2.0));
printf("annuity(INFINITY, 0) = %.15f\n", annuity(INFINITY, 0));
printf("annuity(INFINITY, 0) = %.15f\n", annuity(INFINITY, 0));
printf("annuity(1.0, NAN) = %.15f\n", annuity(1.0, NAN));
printf("annuity(NAN, 1.0) = %.15f\n", annuity(NAN, 1.0));
```

 版权所有 © 华为技术有限公司

```
printf("annuity(INFINITY, -1.0) = %.15f\n", annuity(INFINITY, -1.0));
printf("annuity(INFINITY, -10.0) = %.15f\n", annuity(INFINITY, -10.0));
printf("annuity(INFINITY, 10.0) = %.15f\n", annuity(INFINITY, 10.0));
printf("annuity(0, 1.0) = %.15f\n", annuity(0, 1.0));
printf("annuity(-1.0, -1.0) = %.15f\n", annuity(-1.0, -1.0));
printf("annuity(-1.0, 1.0) = %.15f\n", annuity(-1.0, 1.0));
printf("annuity(-10.0, 1.0) = %.15f\n", annuity(-10.0, 1.0));
printf("annuity(-0.5, -INFINITY) = %.15f\n", annuity(-0.5, -INFINITY));
printf("annuity(0.5, -INFINITY) = %.15f\n", annuity(0.5, -INFINITY));
printf("annuity(-0.5, INFINITY) = %.15f\n", annuity(-0.5, INFINITY));
printf("annuity(0.5, INFINITY) = %.15f\n", annuity(0.5, INFINITY));
/*
 * annuity(1.0, 10.0) = 0.999023437500000
 * annuity(0.5, -3.0) = -4.750000000000000
 * annuity(0.3, -10.0) = 0.999023437500000
 * annuity(4, 2.0) = -4.750000000000000
 * annuity(INFINITY, 0) = 0.000000000000000
 * annuity(INFINITY, 0) = 0.000000000000000
 * annuity(1.0, NAN) = nan
 * annuity(NAN, 1.0) = nan
 * annuity(INFINITY, -1.0) = -1.000000000000000
 * annuity(INFINITY, -10.0) = -inf
 * annuity(INFINITY, 10.0) = 0.000000000000000
 * annuity(0, 1.0) = 1.000000000000000
 * annuity(-1.0, -1.0) = -1.000000000000000
 * annuity(-1.0, 1.0) = inf
 * annuity(-10.0, 1.0) = nan
 * annuity(-0.5, -INFINITY) = -2.000000000000000
 * annuity(0.5, -INFINITY) = -inf
 * annuity(-0.5, INFINITY) = inf
 * annuity(0.5, INFINITY) = 2.000000000000000
 *
 * */
```

6.4.3.2.6.2 compound

计算复利终值系数，入参为实数。

接口定义

C interface：

float compoundf(float x, float y);

double compound(double x, double y);

参数

参数名	类型	描述	输入/输出
x	• 在compoundf中，x是单精度浮点类型。 • 在compound中，x是双精度浮点类型。	表示输入数据的浮点值。	输入
y	• 在compoundf中，y是单精度浮点类型。 • 在compound中，y是双精度浮点类型。	表示输入数据的浮点值。	输入

版权所有 © 华为技术有限公司

返回值

- x大于-1且y ∈(-∞, +∞)，返回计算结果。
- 输入x为任意值，y为±0，返回0。
- 输入x为nan，y为任意值，返回nan。
- 输入x为任意值，y为nan，返回nan。
- 输入x为+∞，y为-1，则返回-1；y < -1，则返回-∞；y ∈(-1, 0)，则返回-0。
- 输入x为0，y != 0，则返回y。
- 输入x为-1，y < 0，则返回-1；y > 0，则返回+∞。
- 输入x < -1，y != 0，则返回nan。
- 输入y为-∞，x ∈(-1， 0)，则返回1.0 / x；x > 0，则返回-∞。
- 输入y为+∞，x ∈(-1， 0)，则返回+∞；x > 0，则返回1.0 / x。

依赖

C: "km.h"

示例

C interface：

```
printf("compound(1.0, 10.0) = %.15f\n", compound(1.0, 10.0));
printf("compound(0.5, -3.0) = %.15f\n", compound(0.5, -3.0));
printf("compound(0.3, -10.0) = %.15f\n", compound(0.3, -10.0));
printf("compound(4, 2.0) = %.15f\n", compound(4, 2.0));
printf("compound(INFINITY, 0) = %.15f\n", compound(INFINITY, 0));
printf("compound(INFINITY, 0) = %.15f\n", compound(INFINITY, 0));
printf("compound(1.0, NAN) = %.15f\n", compound(1.0, NAN));
printf("compound(NAN, 1.0) = %.15f\n", compound(NAN, 1.0));
printf("compound(INFINITY, -1.0) = %.15f\n", compound(INFINITY, -1.0));
printf("compound(INFINITY, -10.0) = %.15f\n", compound(INFINITY, -10.0));
printf("compound(INFINITY, 10.0) = %.15f\n", compound(INFINITY, 10.0));
printf("compound(0, 1.0) = %.15f\n", compound(0, 1.0));
printf("compound(-1.0, -1.0) = %.15f\n", compound(-1.0, -1.0));
printf("compound(-1.0, 1.0) = %.15f\n", compound(-1.0, 1.0));
printf("compound(-10.0, 1.0) = %.15f\n", compound(-10.0, 1.0));
printf("compound(-0.5, -INFINITY) = %.15f\n", compound(-0.5, -INFINITY));
printf("compound(0.5, -INFINITY) = %.15f\n", compound(0.5, -INFINITY));
printf("compound(-0.5, INFINITY) = %.15f\n", compound(-0.5, INFINITY));
printf("compound(0.5, INFINITY) = %.15f\n", compound(0.5, INFINITY));
/**
 * compound(1.0, 10.0) = 1024.000000000000000
 * compound(0.5, -3.0) = 0.296296296296296
 * compound(0.3, -10.0) = 1024.000000000000000
 * compound(4, 2.0) = 0.296296296296296
 * compound(INFINITY, 0) = 1.000000000000000
 * compound(INFINITY, 0) = 1.000000000000000
 * compound(1.0, NAN) = nan
 * compound(NAN, 1.0) = nan
 * compound(INFINITY, -1.0) = 0.000000000000000
 * compound(INFINITY, -10.0) = 0.000000000000000
 * compound(INFINITY, 10.0) = inf
 * compound(0, 1.0) = 1.000000000000000
 * compound(-1.0, -1.0) = inf
 * compound(-1.0, 1.0) = 0.000000000000000
 * compound(-10.0, 1.0) = nan
 * compound(-0.5, -INFINITY) = inf
 * compound(0.5, -INFINITY) = 0.000000000000000
 * compound(-0.5, INFINITY) = 0.000000000000000
 * compound(0.5, INFINITY) = inf
 */
```

 版权所有 © 华为技术有限公司

6.4.3.2.6.3 erf

误差函数，输入参数为实数。

接口定义

C interface：

float erff(float x);

double erf(double x);

double erf_18(double x);

long double erfl(long double x);

说明

erf_18仅在高精度版本中提供。

参数

参数名	类型	描述	输入/输出
x	• 在erff中，x是单精度浮点类型。 • 在erf和erf_18中，x是双精度浮点类型。 • 在erfl中，x是长双精度浮点类型。	表示输入数据的浮点值。	输入

返回值

- 返回入参x的误差函数结果y，y ∈ [-1, +1]。
- 输入nan，返回nan。
- 输入+∞，返回1。
- 输入-∞，返回-1。
- float接口，当输入x >= 0x1.F5AAp1时，返回1。
- float接口，当输入x <= -0x1.F5AAp1时，返回-1。
- double接口，当输入x >= 0x1.7afb48dc96628p+2时，返回1。
- double接口，当输入x <= -0x1.7afb48dc96628p+2时，返回-1。

依赖

C: "km.h"

示例

C interface：

```
// print result
printf("erf(0.0) = %.15f\n", erf(0.0));
```

 版权所有 © 华为技术有限公司

```
printf("erf(-0.0) = %.15f\n", erf(-0.0));
printf("erf(0.25) = %.15f\n", erf(0.25));
printf("erf(-0.25) = %.15f\n", erf(-0.25));
printf("erf(0.5) = %.15f\n", erf(0.5));
printf("erf(-0.5) = %.15f\n", erf(-0.5));
printf("erf(1.0) = %.15f\n", erf(1.0));
printf("erf(-1.0) = %.15f\n", erf(-1.0));
printf("erf(2.0) = %.15f\n", erf(2.0));
printf("erf(-2.0) = %.15f\n", erf(-2.0));
printf("erf(INFINITY) = %.15f\n", erf(INFINITY));
printf("erf(-INFINITY) = %.15f\n", erf(-INFINITY));
printf("erf(NAN) = %.15f\n", erf(NAN));
printf("erf(-NAN) = %.15f\n", erf(-NAN));
/*
 * erf(0.0) = 0.000000000000000
 * erf(-0.0) = -0.000000000000000
 * erf(0.25) = 0.276326390168237
 * erf(-0.25) = -0.276326390168237
 * erf(0.5) = 0.520499877813047
 * erf(-0.5) = -0.520499877813047
 * erf(1.0) = 0.842700792949715
 * erf(-1.0) = -0.842700792949715
 * erf(2.0) = 0.995322265018953
 * erf(-2.0) = -0.995322265018953
 * erf(INFINITY) = 1.000000000000000
 * erf(-INFINITY) = -1.000000000000000
 * erf(NAN) = nan
 * erf(-NAN) = -nan
 *
 * */
```

6.4.3.2.6.4 erfc

互补误差函数，输入参数为实数。erfc(x) = 1 - erf(x)。

接口定义

C interface：

float erfcf(float x);

double erfc(double x);

参数

参数名	类型	描述	输入/输出
x	• 在erfcf中，x是单精度浮点类型。 • 在erfc中，x是双精度浮点类型。	表示输入数据的浮点值。	输入

返回值

- 返回入参x的互补误差函数结果y，y ∈ [0, +2]。
- 输入nan，返回nan。
- 输入+∞，返回0。
- 输入-∞，返回2。

版权所有 © 华为技术有限公司

- float接口，当输入|x| <= 0x1p-26时，返回1。
- float接口，当输入x >= 0x1.466666p+3时，返回0。
- float接口，当输入x <= -0x1.f5a88ap+1时，返回2。
- double接口，当输入|x| <= 0x1p-70时，返回1。
- double接口，当输入x >= 0x1.b39dc41e48bfdp+4时，返回0。
- double接口，当输入x <= -0x1.7744f8f74e94ap+2时，返回2。

依赖

C: "km.h"

示例

C interface：

```
// print result
printf("erfc(0.0) = %.15f\n",erfc(0.0));
printf("erfc(-0.0) = %.15f\n",erfc(-0.0));
printf("erfc(0.25) = %.15f\n",erfc(0.25));
printf("erfc(-0.25) = %.15f\n",erfc(-0.25));
printf("erfc(0.5) = %.15f\n",erfc(0.5));
printf("erfc(-0.5) = %.15f\n",erfc(-0.5));
printf("erfc(1.0) = %.15f\n",erfc(1.0));
printf("erfc(-1.0) = %.15f\n",erfc(-1.0));
printf("erfc(2.0) = %.15f\n",erfc(2.0));
printf("erfc(-2.0) = %.15f\n",erfc(-2.0));
printf("erfc(INFINITY) = %.15f\n",erfc(INFINITY));
printf("erfc(-INFINITY) = %.15f\n",erfc(-INFINITY));
printf("erfc(NAN) = %.15f\n",erfc(NAN));
printf("erfc(-NAN) = %.15f\n",erfc(-NAN));
/*
 * erfc(0.0) = 1.000000000000000
 * erfc(-0.0) = 1.000000000000000
 * erfc(0.25) = 0.723673609831763
 * erfc(-0.25) = 1.276326390168237
 * erfc(0.5) = 0.479500122186953
 * erfc(-0.5) = 1.520499877813047
 * erfc(1.0) = 0.157299207050285
 * erfc(-1.0) = 1.842700792949715
 * erfc(2.0) = 0.004677734981047
 * erfc(-2.0) = 1.995322265018953
 * erfc(INFINITY) = 0.000000000000000
 * erfc(-INFINITY) = 2.000000000000000
 * erfc(NAN) = nan
 * erfc(-NAN) = -nan
 *
 * */
```

6.4.3.2.6.5 erfcx

入参为实数。$erfcx(x) = e^{x^2} erfc(x)$ 。

接口定义

C interface：

float erfcxf(float x);

double erfcx(double x);

 版权所有 © 华为技术有限公司

参数

参数名	类型	描述	输入/输出
x	• 在erfcxf中，x是单精度浮点类型。 • 在erfcx中，x是双精度浮点类型。	表示输入数据的浮点值。	输入

返回值

- 返回入参x的误差函数结果y，y ∈ [-1, +1]。
- 输入nan，返回nan。
- 输入+∞，返回0。
- 输入-∞，返回inf。
- float接口，输入x <= -0x1.2c3cbep3，返回inf。
- double接口，输入x <= -0x1.aa0f4d2e063cfp4，返回inf。

依赖

C: "km.h"

示例

C interface：

```
// print result
printf("erfcx(0.0) = %.15f\n",erfcx(0.0));
printf("erfcx(-0.0) = %.15f\n",erfcx(-0.0));
printf("erfcx(0.25) = %.15f\n",erfcx(0.25));
printf("erfcx(-0.25) = %.15f\n",erfcx(-0.25));
printf("erfcx(0.5) = %.15f\n",erfcx(0.5));
printf("erfcx(-0.5) = %.15f\n",erfcx(-0.5));
printf("erfcx(1.0) = %.15f\n",erfcx(1.0));
printf("erfcx(-1.0) = %.15f\n",erfcx(-1.0));
printf("erfcx(2.0) = %.15f\n",erfcx(2.0));
printf("erfcx(-2.0) = %.15f\n",erfcx(-2.0));
printf("erfcx(INFINITY) = %.15f\n",erfcx(INFINITY));
printf("erfcx(-INFINITY) = %.15f\n",erfcx(-INFINITY));
printf("erfcx(NAN) = %.15f\n",erfcx(NAN));
printf("erfcx(-NAN) = %.15f\n",erfcx(-NAN));
/*
 * erfcx(0.0) = 1.000000000000000
 * erfcx(-0.0) = 1.000000000000000
 * erfcx(0.25) = 0.770346547730997
 * erfcx(-0.25) = 1.358642370104722
 * erfcx(0.5) = 0.615690344192926
 * erfcx(-0.5) = 1.952360489182557
 * erfcx(1.0) = 0.427583576155807
 * erfcx(-1.0) = 5.008980080762283
 * erfcx(2.0) = 0.255395676310506
 * erfcx(-2.0) = 108.940904389977973
 * erfcx(INFINITY) = 0.000000000000000
 * erfcx(-INFINITY) = inf
 * erfcx(NAN) = nan
 * erfcx(-NAN) = -nan
 *
 * */
```

 版权所有 © 华为技术有限公司

6.4.3.2.6.6 erfinv

逆误差函数，输入参数为实数。

接口定义

C interface：

float erfinvf(float x);

double erfinv(double x);

参数

参数名	类型	描述	输入/输出
x	• 在erfinvf中，x是单精度浮点类型。 • 在erfinv中，x是双精度浮点类型。	表示输入数据的浮点值。	输入

返回值

- 返回入参x的逆误差函数结果y，y ∈ [-inf, +inf]。
- 输入nan，返回nan。
- 输入+∞，返回nan。
- 输入-∞，返回nan。
- 输入1，返回+inf。
- 输入-1，返回-inf。
- 输入x > 1，返回nan。
- 输入x < -1，返回nan。

依赖

C: "km.h"

示例

C interface：

```
// print result
printf("erfinv(0.0) = %.15f\n",erfinv(0.0));
printf("erfinv(-0.0) = %.15f\n",erfinv(-0.0));
printf("erfinv(0.25) = %.15f\n",erfinv(0.25));
printf("erfinv(-0.25) = %.15f\n",erfinv(-0.25));
printf("erfinv(0.5) = %.15f\n",erfinv(0.5));
printf("erfinv(-0.5) = %.15f\n",erfinv(-0.5));
printf("erfinv(1.0) = %.15f\n",erfinv(1.0));
printf("erfinv(-1.0) = %.15f\n",erfinv(-1.0));
printf("erfinv(2.0) = %.15f\n",erfinv(2.0));
printf("erfinv(-2.0) = %.15f\n",erfinv(-2.0));
printf("erfinv(INFINITY) = %.15f\n",erfinv(INFINITY));
printf("erfinv(-INFINITY) = %.15f\n",erfinv(-INFINITY));
printf("erfinv(NAN) = %.15f\n",erfinv(NAN));
```

 版权所有 © 华为技术有限公司

```
printf("erfinv(-NAN) = %.15f\n",erfinv(-NAN));
/*
 * erfinv(0.0) = 0.000000000000000
 * erfinv(-0.0) = -0.000000000000000
 * erfinv(0.25) = 0.225312055012178
 * erfinv(-0.25) = -0.225312055012178
 * erfinv(0.5) = 0.476936276204470
 * erfinv(-0.5) = -0.476936276204470
 * erfinv(1.0) = inf
 * erfinv(-1.0) = -inf
 * erfinv(2.0) = nan
 * erfinv(-2.0) = nan
 * erfinv(INFINITY) = nan
 * erfinv(-INFINITY) = nan
 * erfinv(NAN) = nan
 * erfinv(-NAN) = -nan
 *
 * */
```

6.4.3.2.6.7 gamma

计算x的伽马值绝对值的自然对数，输入参数为实数。

接口定义

C interface：

float gammaf(float x);

double gamma(double x);

参数

参数名	类型	描述	输入/输出
x	• 在gammaf中，x是单精度浮点类型。 • 在gamma中，x是双精度浮点类型。	表示输入数据的浮点值。	输入

返回值

- 返回入参x的伽马值绝对值的自然对数。
- 输入nan，返回nan。
- 输入+∞，返回+∞。
- 输入-∞，返回+∞。
- 输入+0，返回+∞。
- 输入-0，返回+∞。

依赖

C: "km.h"

 版权所有 © 华为技术有限公司

示例

C interface：

```
// print result
printf("gamma(0.0) = %.15f\n", gamma(0.0));
printf("gamma(-0.0) = %.15f\n", gamma(-0.0));
printf("gamma(0.25) = %.15f\n", gamma(0.25));
printf("gamma(-0.25) = %.15f\n", gamma(-0.25));
printf("gamma(0.5) = %.15f\n", gamma(0.5));
printf("gamma(-0.5) = %.15f\n", gamma(-0.5));
printf("gamma(1.0) = %.15f\n", gamma(1.0));
printf("gamma(-1.0) = %.15f\n", gamma(-1.0));
printf("gamma(2.0) = %.15f\n", gamma(2.0));
printf("gamma(-2.0) = %.15f\n", gamma(-2.0));
printf("gamma(INFINITY) = %.15f\n", gamma(INFINITY));
printf("gamma(-INFINITY) = %.15f\n", gamma(-INFINITY));
printf("gamma(NAN) = %.15f\n", gamma(NAN));
printf("gamma(-NAN) = %.15f\n", gamma(-NAN));
/*
 * gamma(0.0) = inf
 * gamma(-0.0) = inf
 * gamma(0.25) = 1.288022524698077
 * gamma(-0.25) = 1.589575312551186
 * gamma(0.5) = 0.572364942924700
 * gamma(-0.5) = 1.265512123484645
 * gamma(1.0) = 0.000000000000000
 * gamma(-1.0) = inf
 * gamma(2.0) = 0.000000000000000
 * gamma(-2.0) = inf
 * gamma(INFINITY) = inf
 * gamma(-INFINITY) = inf
 * gamma(NAN) = nan
 * gamma(-NAN) = -nan
 *
 * */
```

6.4.3.2.6.8 gamma_r

计算x的伽马值绝对值的自然对数，返回伽马值的符号，输入参数为实数。

接口定义

C interface：

float gammaf_r(float x, int *signgam);

double gamma_r(double x, int *signgam);

参数

参数名	类型	描述	输入/输出
x	在gammaf_r中，x是单精度浮点类型。 在gamma_r中，x是双精度浮点类型。	表示输入数据的浮点值。	输入
signgam	在gammaf_r中，signgam是整数类型指针。 在gamma_r中，signgam是整数类型指针。	存储x的伽马值的符号	输出

版权所有 © 华为技术有限公司

返回值

- 返回入参x的伽马值绝对值的自然对数、伽马值的符号。
- 输入nan，返回nan，符号为1。
- 输入+∞，返回+∞，符号为1。
- 输入-∞，返回+∞，符号为1。
- 输入+0，返回+∞，符号为1。
- 输入-0，返回+∞，符号为-1。

依赖

C: "km.h"

示例

C interface：

```
// print result
int sign=0;
double res=0.0;
res=gamma_r(0.0, &sign);
printf("gamma_r(0.0, &sign) = %.15f, sign = %d\n", res, sign);
res=gamma_r(-0.0, &sign);
printf("gamma_r(-0.0, &sign) = %.15f, sign = %d\n", res, sign);
res=gamma_r(0.25, &sign);
printf("gamma_r(0.25, &sign) = %.15f, sign = %d\n", res, sign);
res=gamma_r(-0.25, &sign);
printf("gamma_r(-0.25, &sign) = %.15f, sign = %d\n", res, sign);
res=gamma_r(0.5, &sign);
printf("gamma_r(0.5, &sign) = %.15f, sign = %d\n", res, sign);
res=gamma_r(-0.5, &sign);
printf("gamma_r(-0.5, &sign) = %.15f, sign = %d\n", res, sign);
res=gamma_r(1.0, &sign);
printf("gamma_r(1.0, &sign) = %.15f, sign = %d\n", res, sign);
res=gamma_r(-1.0, &sign);
printf("gamma_r(-1.0, &sign) = %.15f, sign = %d\n", res, sign);
res=gamma_r(2.0, &sign);
printf("gamma_r(2.0, &sign) = %.15f, sign = %d\n", res, sign);
res=gamma_r(-2.0, &sign);
printf("gamma_r(-2.0, &sign) = %.15f, sign = %d\n", res, sign);
res=gamma_r(INFINITY, &sign);
printf("gamma_r(INFINITY, &sign) = %.15f, sign = %d\n", res, sign);
res=gamma_r(-INFINITY, &sign);
printf("gamma_r(-INFINITY, &sign) = %.15f, sign = %d\n", res, sign);
res=gamma_r(NAN, &sign);
printf("gamma_r(NAN, &sign) = %.15f, sign = %d\n", res, sign);
res=gamma_r(-NAN, &sign);
printf("gamma_r(-NAN, &sign) = %.15f, sign = %d\n", res, sign);
/*
 * gamma_r(0.0, &sign) = inf, sign = 1
 * gamma_r(-0.0, &sign) = inf, sign = -1
 * gamma_r(0.25, &sign) = 1.288022524698077, sign = 1
 * gamma_r(-0.25, &sign) = 1.589575312551186, sign = -1
 * gamma_r(0.5, &sign) = 0.572364942924700, sign = 1
 * gamma_r(-0.5, &sign) = 1.265512123484645, sign = -1
 * gamma_r(1.0, &sign) = 0.000000000000000, sign = 1
 * gamma_r(-1.0, &sign) = inf, sign = 1
 * gamma_r(2.0, &sign) = 0.000000000000000, sign = 1
 * gamma_r(-2.0, &sign) = inf, sign = 1
 * gamma_r(INFINITY, &sign) = inf, sign = 1
 * gamma_r(-INFINITY, &sign) = inf, sign = 1
```

 版权所有 © 华为技术有限公司

```
 * gamma_r(NAN, &sign) = nan, sign = 1
 * gamma_r(-NAN, &sign) = -nan, sign = 1
 *
 * */
```

6.4.3.2.6.9 lgamma

计算x的伽马值绝对值的自然对数，输入参数为实数。

接口定义

C interface：

float lgammaf(float x);

double lgamma(double x);

参数

参数名	类型	描述	输入/输出
x	• 在lgammaf中，x是单精度浮点类型。 • 在lgamma中，x是双精度浮点类型。	表示输入数据的浮点值。	输入

返回值

- 返回入参x的伽马值绝对值的自然对数。
- 输入nan，返回nan。
- 输入+∞，返回+∞。
- 输入-∞，返回+∞。
- 输入+0，返回+∞。
- 输入-0，返回+∞。

依赖

C: "km.h"

示例

C interface：

```
// print result
printf("lgamma(0.0) = %.15f\n", lgamma(0.0));
printf("lgamma(-0.0) = %.15f\n", lgamma(-0.0));
printf("lgamma(0.25) = %.15f\n", lgamma(0.25));
printf("lgamma(-0.25) = %.15f\n", lgamma(-0.25));
printf("lgamma(0.5) = %.15f\n", lgamma(0.5));
printf("lgamma(-0.5) = %.15f\n", lgamma(-0.5));
printf("lgamma(1.0) = %.15f\n", lgamma(1.0));
printf("lgamma(-1.0) = %.15f\n", lgamma(-1.0));
printf("lgamma(2.0) = %.15f\n", lgamma(2.0));
printf("lgamma(-2.0) = %.15f\n", lgamma(-2.0));
printf("lgamma(INFINITY) = %.15f\n", lgamma(INFINITY));
```

 版权所有 © 华为技术有限公司

```
printf("lgamma(-INFINITY) = %.15f\n", lgamma(-INFINITY));
printf("lgamma(NAN) = %.15f\n", lgamma(NAN));
printf("lgamma(-NAN) = %.15f\n", lgamma(-NAN));
/*
 * lgamma(0.0) = inf
 * lgamma(-0.0) = inf
 * lgamma(0.25) = 1.288022524698077
 * lgamma(-0.25) = 1.589575312551186
 * lgamma(0.5) = 0.572364942924700
 * lgamma(-0.5) = 1.265512123484645
 * lgamma(1.0) = 0.000000000000000
 * lgamma(-1.0) = inf
 * lgamma(2.0) = 0.000000000000000
 * lgamma(-2.0) = inf
 * lgamma(INFINITY) = inf
 * lgamma(-INFINITY) = inf
 * lgamma(NAN) = nan
 * lgamma(-NAN) = -nan
 *
 * */
```

6.4.3.2.6.10 lgamma_r

计算x的伽马值绝对值的自然对数，返回伽马值的符号，输入参数为实数。

接口定义

C interface：

float lgammaf_r(float x);

double lgamma_r(double x);

参数

参数名	类型	描述	输入/输出
x	• 在lgammaf_r中，x是单精度浮点类型。 • 在lgamma_r中，x是双精度浮点类型。	表示输入数据的浮点值。	输入

返回值

- 返回入参x的伽马值绝对值的自然对数、伽马值的符号。
- 输入nan，返回nan，符号为1。
- 输入+∞，返回+∞，符号为1。
- 输入-∞，返回+∞，符号为1。
- 输入+0，返回+∞，符号为1。
- 输入-0，返回+∞，符号为-1。

依赖

C: "km.h"

 版权所有 © 华为技术有限公司

示例

C interface：

```
// print result
int sign=0;
double res=0.0;
res=lgamma_r(0.0, &sign);
printf("lgamma_r(0.0, &sign) = %.15f, sign = %d\n", res, sign);
res=lgamma_r(-0.0, &sign);
printf("lgamma_r(-0.0, &sign) = %.15f, sign = %d\n", res, sign);
res=lgamma_r(0.25, &sign);
printf("lgamma_r(0.25, &sign) = %.15f, sign = %d\n", res, sign);
res=lgamma_r(-0.25, &sign);
printf("lgamma_r(-0.25, &sign) = %.15f, sign = %d\n", res, sign);
res=lgamma_r(0.5, &sign);
printf("lgamma_r(0.5, &sign) = %.15f, sign = %d\n", res, sign);
res=lgamma_r(-0.5, &sign);
printf("lgamma_r(-0.5, &sign) = %.15f, sign = %d\n", res, sign);
res=lgamma_r(1.0, &sign);
printf("lgamma_r(1.0, &sign) = %.15f, sign = %d\n", res, sign);
res=lgamma_r(-1.0, &sign);
printf("lgamma_r(-1.0, &sign) = %.15f, sign = %d\n", res, sign);
res=lgamma_r(2.0, &sign);
printf("lgamma_r(2.0, &sign) = %.15f, sign = %d\n", res, sign);
res=lgamma_r(-2.0, &sign);
printf("lgamma_r(-2.0, &sign) = %.15f, sign = %d\n", res, sign);
res=lgamma_r(INFINITY, &sign);
printf("lgamma_r(INFINITY, &sign) = %.15f, sign = %d\n", res, sign);
res=lgamma_r(-INFINITY, &sign);
printf("lgamma_r(-INFINITY, &sign) = %.15f, sign = %d\n", res, sign);
res=lgamma_r(NAN, &sign);
printf("lgamma_r(NAN, &sign) = %.15f, sign = %d\n", res, sign);
res=lgamma_r(-NAN, &sign);
printf("lgamma_r(-NAN, &sign) = %.15f, sign = %d\n", res, sign);
/*
 * lgamma_r(0.0, &sign) = inf, sign = 1
 * lgamma_r(-0.0, &sign) = inf, sign = -1
 * lgamma_r(0.25, &sign) = 1.288022524698077, sign = 1
 * lgamma_r(-0.25, &sign) = 1.589575312551186, sign = -1
 * lgamma_r(0.5, &sign) = 0.572364942924700, sign = 1
 * lgamma_r(-0.5, &sign) = 1.265512123484645, sign = -1
 * lgamma_r(1.0, &sign) = 0.000000000000000, sign = 1
 * lgamma_r(-1.0, &sign) = inf, sign = 1
 * lgamma_r(2.0, &sign) = 0.000000000000000, sign = 1
 * lgamma_r(-2.0, &sign) = inf, sign = 1
 * lgamma_r(INFINITY, &sign) = inf, sign = 1
 * lgamma_r(-INFINITY, &sign) = inf, sign = 1
 * lgamma_r(NAN, &sign) = nan, sign = 1
 * lgamma_r(-NAN, &sign) = -nan, sign = 1
 *
 * */
```

6.4.3.2.6.11 tgamma

返回入参x的伽马函数值$\int_0^\infty e^{-t}t^{x-1}dt$。

接口定义

C interface：

float tgammaf(float x);

float tgammaf_18(float x);double tgamma(double x);

double tgamma(double x);

 版权所有 © 华为技术有限公司

说明

tgammaf_18仅在高精度版本提供。

参数

参数名	类型	描述	输入/输出
x	• 在tgammaf和tgammaf_18中，x是单精度浮点类型。 • 在tgamma中，x是双精度浮点类型。	表示输入数据的浮点值。	输入

返回值

- 返回x的伽马函数值y，y ∈ (-INF, +INF)。
- 输入x为±0，返回±inf。
- 输入x为负整数，返回-nan。
- 输入x为nan，返回nan。
- 输入x为+inf，返回+inf。
- 输入x为-inf，返回-nan。

依赖

C: "km.h"

示例

C interface：

```
// typical usage
   float x1 = -2.5, x2 = -1.5, x3 = 1.5, x4 = 2.5;
   // special handling
   float a = -2.0, b = -0.0, c = INFINITY, d = NAN;
   // print result
      printf("tgammaf(-2.5) = %.15f\n", tgammaf(x1));
      printf("tgammaf(-1.5) = %.15f\n", tgammaf(x2));
      printf("tgammaf(1.5) = %.15f\n", tgammaf(x3));
      printf("tgammaf(2.5) = %.15f\n", tgammaf(x4));
      printf("tgammaf(-2.0) = %.15f\n", tgammaf(a));
      printf("tgammaf(-0.0) = %.15f\n", tgammaf(b));
      printf("tgammaf(INFINITY) = %.15f\n", tgammaf(c));
      printf("tgammaf(NAN) = %.15f\n", tgammaf(d));
   /*
    *
    *  tgammaf(-2.5) = -0.945308744907379
    *  tgammaf(-1.5) = 2.363271713256836
    *  tgammaf(1.5) = 0.886226952075958
    *  tgammaf(2.5) = 1.329340338706970
    *  tgammaf(-2.0) = -nan
    *  tgammaf(-0.0) = -inf
    *  tgammaf(INFINITY) = inf
    *  tgammaf(NAN) = nan
    *
    * */
```

 版权所有 © 华为技术有限公司

6.4.3.2.6.12 j0

Bessel 0阶第一类函数。

接口定义

C interface：

float j0f(float x);

double j0(double x);

参数

参数名	类型	描述	输入/输出
x	• 在j0f中，x是单精度浮点类型。 • 在j0中，x是双精度浮点类型。	表示输入数据的浮点值。	输入

返回值

- 返回0阶第一类贝塞尔函数值y，y ∈ (-1, +1]，该函数为偶函数。
- 输入x为±0，返回1。
- 输入x为±∞，返回0。
- 输入x为nan，返回nan。

依赖

C: "km.h"

示例

C interface：

```
double x1 = 0.0, x2 = -0.0, x3 = 1.5, x4 = -2.5;
// special handling
double a = INFINITY, b = -INFINITY, c = NAN;
// print result
printf("j0(0.0) = %.15f\n", j0(x1));
printf("j0(-0.0) = %.15f\n", j0(x2));
printf("j0(1.5) = %.15f\n", j0(x3));
printf("j0(-2.5) = %.15f\n", j0(x4));
printf("j0(INFINITY) = %.15f\n", j0(a));
printf("j0(-INFINITY) = %.15f\n", j0(b));
printf("j0(NAN) = %.15f\n", j0(c));

/*
 * j0(0.0) = 1.000000000000000
 * j0(-0.0) = 1.000000000000000
 * j0(1.5) = 0.511827671735918
 * j0(-2.5) = -0.048383776468198
 * j0(INFINITY) = 0.000000000000000
 * j0(-INFINITY) = 0.000000000000000
 * j0(NAN) = nan
```

 版权所有 © 华为技术有限公司

```
 *
 * */
```

6.4.3.2.6.13 j1

Bessel 1阶第一类函数。

接口定义

C interface：

float j1f(float x);

double j1(double x);

参数

参数名	类型	描述	输入/输出
x	• 在j1f中，x是单精度浮点类型。 • 在j1中，x是双精度浮点类型。	表示输入数据的浮点值。	输入

返回值

- 返回1阶第一类贝塞尔函数值y，y ∈ (-1, +1)，该函数为奇函数。
- 输入x为+0，返回+0。
- 输入x为-0，返回-0。
- 输入x为±∞，返回0。
- 输入x为nan，返回nan。

依赖

C: "km.h"

示例

C interface：

```
double x1 = 0.0, x2 = -0.0, x3 = 1.5, x4 = -2.5;
// special handling
double a = INFINITY, b = -INFINITY, c = NAN;
// print result
printf("j1(0.0) = %.15f\n", j1(x1));
printf("j1(-0.0) = %.15f\n", j1(x2));
printf("j1(1.5) = %.15f\n", j1(x3));
printf("j1(-2.5) = %.15f\n", j1(x4));
printf("j1(INFINITY) = %.15f\n", j1(a));
printf("j1(-INFINITY) = %.15f\n", j1(b));
printf("j1(NAN) = %.15f\n", j1(c));

/*
 * j1(0.0) = 0.000000000000000
 * j1(-0.0) = -0.000000000000000
```


版权所有 © 华为技术有限公司

```
 * j1(1.5) = 0.557936507910100
 * j1(-2.5) = -0.497094102464274
 * j1(INFINITY) = 0.000000000000000
 * j1(-INFINITY) = 0.000000000000000
 * j1(NAN) = nan
 *
 * */
```

6.4.3.2.6.14 jn

计算入参的Bessel n阶第一类函数，入参为实数。

接口定义

C interface：

float jnf(int n, float x);

double jn(int n, double x);

参数

参数名	类型	描述	输入/输出
n	• 在jnf中，n是整型类型。 • 在jn中，n是整型类型。	表示计算阶数。	输入
x	• 在jnf中，x是单精度浮点类型。 • 在jn中，x是双精度浮点类型。	表示输入数据的浮点值。	输入

返回值

rSign = (n & (nSign ^ xSign)) * (-1)，其中nSign = n > 0?0:1，xSign = x > 0?0:1。

- 输入x为±0，返回rSign * 0。
- 输入x为±∞，返回0。
- 输入x为nan，返回nan。

依赖

C: "km.h"

示例

C interface：

```
printf("jn(1, 0.0) = %.15f\n", jn(1, 0.0));
printf("jn(2, -0.0) = %.15f\n", jn(2, -0.0));
printf("jn(3, 1.5) = %.15f\n", jn(3, 1.5));
printf("jn(4, -2.5) = %.15f\n", jn(4, -2.5));
printf("jn(5, INFINITY) = %.15f\n", jn(5, INFINITY));
printf("jn(6, -INFINITY) = %.15f\n", jn(6, -INFINITY));
printf("jn(7, NAN) = %.15f\n", jn(7, NAN));
```

 版权所有 © 华为技术有限公司

```
/**
 * jn(1, 0.0) = 0.000000000000000
 * jn(2, -0.0) = 0.000000000000000
 * jn(3, 1.5) = 0.060963951141140
 * jn(4, -2.5) = 0.073781880054255
 * jn(5, INFINITY) = 0.000000000000000
 * jn(6, -INFINITY) = 0.000000000000000
 * jn(7, NAN) = nan
 */
```

6.4.3.2.6.15 y0

Bessel 0阶第二类函数，x > 0。

接口定义

C interface：

float y0f(float x);

double y0(double x);

参数

参数名	类型	描述	输入/输出
x	• 在y0f中，x是单精度浮点类型。 • 在y0中，x是双精度浮点类型。	表示输入数据的浮点值。	输入

返回值

- 返回0阶第二类贝塞尔函数值y，y ∈ (-∞, +1)。
- 输入x为±0，返回-∞。
- 输入x<0，返回nan。
- 输入x为+∞，返回0。
- 输入x为nan，返回nan。

依赖

C: "km.h"

示例

C interface：

```
double x1 = 0.0, x2 = -0.0, x3 = 1.5, x4 = -2.5;
// special handling
double a = INFINITY, b = -INFINITY, c = NAN;
// print result
printf("y0(0.0) = %.15f\n", y0(x1));
printf("y0(-0.0) = %.15f\n", y0(x2));
printf("y0(1.5) = %.15f\n", y0(x3));
printf("y0(-2.5) = %.15f\n", y0(x4));
```

 版权所有 © 华为技术有限公司

```
printf("y0(INFINITY) = %.15f\n", y0(a));
printf("y0(-INFINITY) = %.15f\n", y0(b));
printf("y0(NAN) = %.15f\n", y0(c));

/*
 * y0(0.0) = -inf
 * y0(-0.0) = -inf
 * y0(1.5) = 0.382448923797759
 * y0(-2.5) = nan
 * y0(INFINITY) = 0.000000000000000
 * y0(-INFINITY) = nan
 * y0(NAN) = nan
 *
 * */
```

6.4.3.2.6.16 y1

Bessel 1阶第二类函数，x > 0。

接口定义

C interface：

float y1f(float x);

double y1(double x);

参数

参数名	类型	描述	输入/输出
x	• 在y1f中，x是单精度浮点类型。 • 在y1中，x是双精度浮点类型。	表示输入数据的浮点值。	输入

返回值

- 返回1阶第二类贝塞尔函数值y，y ∈ (-∞, +1)。
- 输入x为±0，返回-∞。
- 输入x<0，返回nan。
- 输入x为+∞，返回0。
- 输入x为nan，返回nan。

依赖

C: "km.h"

示例

C interface：

```
double x1 = 0.0, x2 = -0.0, x3 = 1.5, x4 = -2.5;
// special handling
double a = INFINITY, b = -INFINITY, c = NAN;
```

 版权所有 © 华为技术有限公司

```
// print result
printf("y1(0.0) = %.15f\n", y1(x1));
printf("y1(-0.0) = %.15f\n", y1(x2));
printf("y1(1.5) = %.15f\n", y1(x3));
printf("y1(-2.5) = %.15f\n", y1(x4));
printf("y1(INFINITY) = %.15f\n", y1(a));
printf("y1(-INFINITY) = %.15f\n", y1(b));
printf("y1(NAN) = %.15f\n", y1(c));

/*
 * y1(0.0) = -inf
 * y1(-0.0) = -inf
 * y1(1.5) = -0.412308626973911
 * y1(-2.5) = nan
 * y1(INFINITY) = 0.000000000000000
 * y1(-INFINITY) = nan
 * y1(NAN) = nan
 *
 * */
```

6.4.3.2.6.17 yn

计算入参的Bessel n阶第二类函数，x > 0，入参为实数。

接口定义

C interface：

float ynf(int n, float x);

double yn(int n, double x);

参数

参数名	类型	描述	输入/输出
n	• 在ynf中，n是整型类型。 • 在yn中，n是整型类型。	表示计算的阶数。	输入
x	• 在ynf中，x是单精度浮点类型。 • 在yn中，x是双精度浮点类型。	表示输入数据的浮点值。	输入

返回值

计算rSign = ((n & 1) & nSign) * (-1)，其中nSign = n > 0?0:1。

- 当输入x<0，返回nan。
- 输入x为0，返回-inf*rSign。
- 输入x为inf，返回0。
- 输入x为nan，返回nan。

依赖

C: "km.h"

 版权所有 © 华为技术有限公司

示例

C interface：

```
printf("yn(1, 0.0) = %.15f\n", yn(1, 0.0));
printf("yn(2, -0.0) = %.15f\n", yn(2, -0.0));
printf("yn(3, 1.5) = %.15f\n", yn(3, 1.5));
printf("yn(4, -2.5) = %.15f\n", yn(4, -2.5));
printf("yn(5, INFINITY) = %.15f\n", yn(5, INFINITY));
printf("yn(6, -INFINITY) = %.15f\n", yn(6, -INFINITY));
printf("yn(7, NAN) = %.15f\n", yn(7, NAN));
/**
 * yn(1, 0.0) = -inf
 * yn(2, -0.0) = -inf
 * yn(3, 1.5) = -2.073541399060686
 * yn(4, -2.5) = nan
 * yn(5, INFINITY) = 0.000000000000000
 * yn(6, -INFINITY) = nan
 * yn(7, NAN) = nan
 */
```

6.4.3.2.7 舍入函数

6.4.3.2.7.1 ceil

返回不小于x的最小整数值的浮点形式。

接口定义

C interface：

float ceilf(float x);

double ceil(double x);

long double ceill(long double x);

参数

参数名	类型	描述	输入/输出
x	• 在ceilf中，x是单精度浮点类型。 • 在ceil中，x是双精度浮点类型。 • 在ceill中，x是长双精度浮点类型。	表示输入数据的浮点值。	输入

返回值

- 返回x的舍入值，x ∈ (-inf, inf)。
- 输入+0，返回+0。
- 输入-0，返回-0。
- 输入±∞，返回±∞。

 版权所有 © 华为技术有限公司

- 输入nan，返回nan。

依赖

C: "km.h"

示例

C interface：

```
// typical usage
double x1 = 0.0, x2 = -0.0, x3 = 1.5, x4 = -2.5;
// special handling
double a = INFINITY, b = -INFINITY, c = NAN;
// print result
printf("ceil(0.0) = %.15f\n", ceil(x1));
printf("ceil(-0.0) = %.15f\n", ceil(x2));
printf("ceil(1.5) = %.15f\n", ceil(x3));
printf("ceil(-2.5) = %.15f\n", ceil(x4));
printf("ceil(INFINITY) = %.15f\n", ceil(a));
printf("ceil(-INFINITY) = %.15f\n", ceil(b));
printf("ceil(NAN) = %.15f\n", ceil(c));
/*
 * ceil(0.0) = 0.000000000000000
 * ceil(-0.0) = -0.000000000000000
 * ceil(1.5) = 2.000000000000000
 * ceil(-2.5) = -2.000000000000000
 * ceil(INFINITY) = inf
 * ceil(-INFINITY) = -inf
 * ceil(NAN) = nan
 * */
```

6.4.3.2.7.2 floor

返回不大于x的最大整数值。

接口定义

C interface：

float floorf(float x);

double floor(double x);

long double floorl(long double x);

参数

参数名	类型	描述	输入/输出
x	• 在floorf中，x是单精度浮点类型。 • 在floor中，x是双精度浮点类型。 • 在floorl中，x是长双精度浮点类型。	表示输入数据的浮点值。	输入

 版权所有 © 华为技术有限公司

返回值

- 返回x的向下取整后的值，x ∈ (-inf, inf)。
- 输入+0，返回+0。
- 输入-0，返回-0。
- 输入±∞，返回±∞。
- 输入nan，返回nan。

依赖

C: "km.h"

示例

C interface：

```
// typical usage
double x1 = 0.0, x2 = -0.0, x3 = 1.5, x4 = -2.5;
// special handling
double a = INFINITY, b = -INFINITY, c = NAN;
// print result
printf("floor(0.0) = %.15f\n", floor(x1));
printf("floor(-0.0) = %.15f\n", floor(x2));
printf("floor(1.5) = %.15f\n", floor(x3));
printf("floor(-2.5) = %.15f\n", floor(x4));
printf("floor(INFINITY) = %.15f\n", floor(a));
printf("floor(-INFINITY) = %.15f\n", floor(b));
printf("floor(NAN) = %.15f\n", floor(c));
/*
 * floor(0.0) = 0.000000000000000
 * floor(-0.0) = -0.000000000000000
 * floor(1.5) = 1.000000000000000
 * floor(-2.5) = -3.000000000000000
 * floor(INFINITY) = inf
 * floor(-INFINITY) = -inf
 * floor(NAN) = nan
 * */
```

6.4.3.2.7.3 rint

基于当前的舍入方向，返回x的舍入值。

接口定义

C interface：

float rintf(float x);

double rint(double x);

long double rintl(long double x);

 版权所有 © 华为技术有限公司

参数

参数名	类型	描述	输入/输出
x	• 在rintf中，x是单精度浮点类型。 • 在rint中，x是双精度浮点类型。 • 在rintl中，x是长双精度浮点类型。	表示输入数据的浮点值。	输入

返回值

- 返回x的舍入值，x ∈ (-inf, inf)。
- 输入+0，返回+0。
- 输入-0，返回-0。
- 输入±∞，返回±∞。
- 输入nan，返回nan。

依赖

C: "km.h"

示例

C interface：

```
// typical usage
double x1 = 0.0, x2 = -0.0, x3 = 1.5, x4 = -2.5;
// special handling
double a = INFINITY, b = -INFINITY, c = NAN;
// print result
printf("rint(0.0) = %.15f\n", rint(x1));
printf("rint(-0.0) = %.15f\n", rint(x2));
printf("rint(1.5) = %.15f\n", rint(x3));
printf("rint(-2.5) = %.15f\n", rint(x4));
printf("rint(INFINITY) = %.15f\n", rint(a));
printf("rint(-INFINITY) = %.15f\n", rint(b));
printf("rint(NAN) = %.15f\n", rint(c));

/*
 * rint(0.0) = 0.000000000000000
 * rint(-0.0) = -0.000000000000000
 * rint(1.5) = 2.000000000000000
 * rint(-2.5) = -2.000000000000000
 * rint(INFINITY) = inf
 * rint(-INFINITY) = -inf
 * rint(NAN) = nan
 * */
```

6.4.3.2.7.4 lrint

基于当前舍入策略，以long int形式返回舍入值。

 版权所有 © 华为技术有限公司

接口定义

C interface：

long int lrintf(float x);

long int lrint(double x);

long int lrintl(long double x);

参数

参数名	类型	描述	输入/输出
x	• 在lrintf中，x是单精度浮点类型。 • 在lrint中，x是双精度浮点类型。 • 在lrintl中，x是长双精度浮点类型。	表示输入数据的浮点值。	输入

返回值

返回x的舍入值，x ∈ (-inf, inf)。

依赖

C: "km.h"

示例

C interface：

```
   // typical usage
   printf("lrint(0.0) = %ld\n", lrint(0.0));
   printf("lrint(-0.0) = %ld\n", lrint(-0.0));
   printf("lrint(1.0) = %ld\n", lrint(1.0));
   printf("lrint(1.4) = %ld\n", lrint(1.4));
   printf("lrint(1.5) = %ld\n", lrint(1.5));
   printf("lrint(-3.0) = %ld\n", lrint(-3.0));
   printf("lrint(-3.4) = %ld\n", lrint(-3.4));
   printf("lrint(-3.5) = %ld\n", lrint(-3.5));

result
   /*
    * lrint(0.0) = 0
    * lrint(-0.0) = 0
    * lrint(1.0) = 1
    * lrint(1.4) = 1
    * lrint(1.5) = 2
    * lrint(-3.0) = -3
    * lrint(-3.4) = -3
    * lrint(-3.5) = -4
    * */
```

6.4.3.2.7.5 llrint

基于当前舍入策略，以long long int形式返回舍入值。

 版权所有 © 华为技术有限公司

接口定义

C interface：

long long int llrintf(float x);

long long int llrint(double x);

long long int llrintl(long double x);

参数

参数名	类型	描述	输入/输出
x	• 在llrintf中，x是单精度浮点类型。 • 在llrint中，x是双精度浮点类型。 • 在llrintl中，x是长双精度浮点类型。	表示输入数据的浮点值。	输入

返回值

返回x的舍入值，x ∈ (-inf, inf)。

依赖

C: "km.h"

示例

C interface：

```
// typical usage
printf("llrint(0.0) = %lld\n", llrint(0.0));
printf("llrint(-0.0) = %lld\n", llrint(-0.0));
printf("llrint(1.0) = %lld\n", llrint(1.0));
printf("llrint(1.4) = %lld\n", llrint(1.4));
printf("llrint(1.5) = %lld\n", llrint(1.5));
printf("llrint(-3.0) = %lld\n", llrint(-3.0));
printf("llrint(-3.4) = %lld\n", llrint(-3.4));
printf("llrint(-3.5) = %lld\n", llrint(-3.5));

result
/*
 * llrint(0.0) = 0
 * llrint(-0.0) = 0
 * llrint(1.0) = 1
 * llrint(1.4) = 1
 * llrint(1.5) = 2
 * llrint(-3.0) = -3
 * llrint(-3.4) = -3
 * llrint(-3.5) = -4
 * */
```

6.4.3.2.7.6 round

返回x的四舍五入的舍入值。

 版权所有 © 华为技术有限公司

接口定义

C interface：

float roundf(float x);

double round(double x);

long double roundl(long double x);

参数

参数名	类型	描述	输入/输出
x	• 在roundf中，x是单精度浮点类型。 • 在round中，x是双精度浮点类型。 • 在roundl中，x是长双精度浮点类型。	表示输入数据的浮点值。	输入

返回值

- 返回x的舍入值，x ∈ (-inf, inf)。
- 输入+0，返回+0。
- 输入-0，返回-0。
- 输入±∞，返回±∞。
- 输入nan，返回nan。

依赖

C: "km.h"

示例

C interface：

```
// typical usage
double x1 = 0.0, x2 = -0.0, x3 = 1.5, x4 = -2.5;
// special handling
double a = INFINITY, b = -INFINITY, c = NAN;
// print result
printf("round(0.0) = %.15f\n", round(x1));
printf("round(-0.0) = %.15f\n", round(x2));
printf("round(1.5) = %.15f\n", round(x3));
printf("round(-2.5) = %.15f\n", round(x4));
printf("round(INFINITY) = %.15f\n", round(a));
printf("round(-INFINITY) = %.15f\n", round(b));
printf("round(NAN) = %.15f\n", round(c));
/*
 * round(0.0) = 0.000000000000000
 * round(-0.0) = -0.000000000000000
 * round(1.5) = 2.000000000000000
 * round(-2.5) = -3.000000000000000
 * round(INFINITY) = inf
 * round(-INFINITY) = -inf
```

 版权所有 © 华为技术有限公司

```
 * round(NAN) = nan
 * */
```

6.4.3.2.7.7 lround

基于当前舍入策略，以long int形式返回舍入值。

接口定义

C interface：

long int lroundf(float x);

long int lround(double x);

long int lroundl(long double x);

参数

参数名	类型	描述	输入/输出
x	• 在lroundf中，x是单精度浮点类型。 • 在lround中，x是双精度浮点类型。 • 在lroundl中，x是长双精度浮点类型。	表示输入数据的浮点值。	输入

返回值

返回x的舍入值，x ∈ (-inf, inf)。

依赖

C: "km.h"

示例

C interface：

```
// typical usage
printf("lround(0.0) = %ld\n", lround(0.0));
printf("lround(-0.0) = %ld\n", lround(-0.0));
printf("lround(1.0) = %ld\n", lround(1.0));
printf("lround(1.4) = %ld\n", lround(1.4));
printf("lround(1.5) = %ld\n", lround(1.5));
printf("lround(-3.0) = %ld\n", lround(-3.0));
printf("lround(-3.4) = %ld\n", lround(-3.4));
printf("lround(-3.5) = %ld\n", lround(-3.5));

result
/*
 * lround(0.0) = 0
 * lround(-0.0) = 0
 * lround(1.0) = 1
 * lround(1.4) = 1
 * lround(1.5) = 2
```

 版权所有 © 华为技术有限公司

```
 * lround(-3.0) = -3
 * lround(-3.4) = -3
 * lround(-3.5) = -4
 * */
```

6.4.3.2.7.8 llround

基于当前舍入策略，以long long int形式返回舍入值。

接口定义

C interface：

long long int llroundf(float x);

long long int llround(double x);

long long int llroundl(long double x);

参数

参数名	类型	描述	输入/输出
x	• 在llroundf中，x是单精度浮点类型。 • 在llround中，x是双精度浮点类型。 • 在llroundl中，x是长双精度浮点类型。	表示输入数据的浮点值。	输入

返回值

返回x的舍入值，x ∈ (-inf, inf)。

依赖

C: "km.h"

示例

C interface：

```
// typical usage
printf("llround(0.0) = %lld\n", llround(0.0));
printf("llround(-0.0) = %lld\n", llround(-0.0));
printf("llround(1.0) = %lld\n", llround(1.0));
printf("llround(1.4) = %lld\n", llround(1.4));
printf("llround(1.5) = %lld\n", llround(1.5));
printf("llround(-3.0) = %lld\n", llround(-3.0));
printf("llround(-3.4) = %lld\n", llround(-3.4));
printf("llround(-3.5) = %lld\n", llround(-3.5));

result
/*
 * llround(0.0) = 0
 * llround(-0.0) = 0
 * llround(1.0) = 1
```


版权所有 © 华为技术有限公司

```
 * llround(1.4) = 1
 * llround(1.5) = 2
 * llround(-3.0) = -3
 * llround(-3.4) = -3
 * llround(-3.5) = -4
 * */
```

6.4.3.2.7.9 modf

返回值为小数部分（小数点后的部分），并把整数部分存储在iptr中。

接口定义

C interface：

float modff(float x, float *iptr);

double modf(double x, double *iptr);

long double modfl(long double x, long double *iptr);

参数

参数名	类型	描述	输入/输出
x	• 在modff中，x是单精度浮点类型。 • 在modf中，x是双精度浮点类型。 • 在modfl中，x是长双精度浮点类型。	表示输入数据的浮点值。	输入
iptr	• 在modff中，iptr是单精度浮点类型指针。 • 在modf中，iptr是双精度浮点类型指针。 • 在modfl中，iptr是长双精度浮点类型指针。	存储入参浮点数整数部分	输出

返回值

返回值为小数部分（小数点后的部分），并把整数部分存储在iptr中。

依赖

C: "km.h"

示例

C interface：

```
// typical usage
double iptr;
printf("modf(3.4, &iptr) = %.15f, iptr = %.15f\n", modf(3.4, &iptr), iptr);
printf("modf(-4.8, &iptr) = %.15f, iptr = %.15f\n", modf(-4.8, &iptr), iptr);
```

 版权所有 © 华为技术有限公司

```
result
    /*
     * modf(3.4, &iptr) = 0.400000000000000, iptr = 3.000000000000000
     * modf(-4.8, &iptr) = -0.800000000000000, iptr = -4.000000000000000
     * */
```

6.4.3.2.7.10 nearbyint

基于当前的舍入方向，返回x的舍入值。

接口定义

C interface：

float nearbyintf(float x);

double nearbyint(double x);

参数

参数名	类型	描述	输入/输出
x	• 在nearbyintf中，x是单精度浮点类型。 • 在nearbyint中，x是双精度浮点类型。	表示输入数据的浮点值。	输入

返回值

- 返回x的舍入值，x ∈ (-inf, inf)。
- 输入+0，返回+0。
- 输入-0，返回-0。
- 输入±∞，返回±∞。
- 输入nan，返回nan。

依赖

C: "km.h"

示例

C interface：

```
// typical usage
double x1 = 0.0, x2 = -0.0, x3 = 1.5, x4 = -2.5;
// special handling
double a = INFINITY, b = -INFINITY, c = NAN;
// nearbyint result
printf("nearbyint(0.0) = %.15f\n", nearbyint(x1));
printf("nearbyint(-0.0) = %.15f\n", nearbyint(x2));
printf("nearbyint(1.5) = %.15f\n", nearbyint(x3));
printf("nearbyint(-2.5) = %.15f\n", nearbyint(x4));
printf("nearbyint(INFINITY) = %.15f\n", nearbyint(a));
printf("nearbyint(-INFINITY) = %.15f\n", nearbyint(b));
```

 版权所有 © 华为技术有限公司

```
printf("nearbyint(NAN) = %.15f\n", nearbyint(c));
/*
 * nearbyint(0.0) = 0.000000000000000
 * nearbyint(-0.0) = -0.000000000000000
 * nearbyint(1.5) = 2.000000000000000
 * nearbyint(-2.5) = -2.000000000000000
 * nearbyint(INFINITY) = inf
 * nearbyint(-INFINITY) = -inf
 * nearbyint(NAN) = nan
 * */
```

6.4.3.2.7.11 trunc

向0的方向，返回x的舍入值。

接口定义

C interface：

float truncf(float x);

double trunc(double x);

long double truncl(long double x);

参数

参数名	类型	描述	输入/输出
x	• 在truncf中，x是单精度浮点类型。 • 在trunc中，x是双精度浮点类型。 • 在truncl中，x是长双精度浮点类型。	表示输入数据的浮点值。	输入

返回值

- 返回x的舍入值，x ∈ (-inf, inf)。
- 输入+0，返回+0。
- 输入-0，返回-0。
- 输入±∞，返回±∞。
- 输入nan，返回nan。

依赖

C: "km.h"

示例

C interface：

```
// typical usage
double x1 = 0.0, x2 = -0.0, x3 = 1.4, x4 = -2.7;
// special handling
```

 版权所有 © 华为技术有限公司

```
double a = INFINITY, b = -INFINITY, c = NAN;
printf("trunc(0.0) = %.15f\n", trunc(x1));
printf("trunc(-0.0) = %.15f\n", trunc(x2));
printf("trunc(1.4) = %.15f\n", trunc(x3));
printf("trunc(-2.7) = %.15f\n", trunc(x4));
printf("trunc(INFINITY) = %.15f\n", trunc(a));
printf("trunc(-INFINITY) = %.15f\n", trunc(b));
printf("trunc(NAN) = %.15f\n", trunc(c));
/*
 * trunc(0.0) = 0.000000000000000
 * trunc(-0.0) = -0.000000000000000
 * trunc(1.4) = 1.000000000000000
 * trunc(-2.7) = -2.000000000000000
 * trunc(INFINITY) = inf
 * trunc(-INFINITY) = -inf
 * trunc(NAN) = nan
 * */
```

6.4.3.2.8 取余函数

6.4.3.2.8.1 drem

同时计算浮点x/y的余数。

即x = n * y + d，其中n为整数，|d| < y / 2；当|d| = |y| / 2时，要保证n为偶数。

接口定义

C interface：

float dremf(float x, float y);

double drem(double x, double y);

参数

参数名	类型	描述	输入/输出
x	• 在dremf中，x是单精度浮点类型。 • 在drem中，x是双精度浮点类型。	表示输入数据的浮点值。	输入
y	• 在dremf中，y是单精度浮点类型。 • 在drem中，y是双精度浮点类型。	表示输入数据的浮点值。	输入

返回值

- 正常区间：返回x/y的余数。
- 特殊区间：
  - 输入|x|或者|y|为0或nan，输出nan。
  - 输入|x|为0，|y|不为0或者nan，输出±0。

 版权所有 © 华为技术有限公司

依赖

C: "km.h"

示例

C interface：

```
// typical usage
double x1 = 3.0, x2 = 3.5;
double y1 = 4.0, y2 = 4.5;
// special handling
double e = INFINITY, f = -INFINITY, g = NAN;
// print result
printf("/*\n");
printf(" * drem(3.0, 4.0) = %.15f\n", drem(x1, y1));
printf(" * drem(3.5, 3.5) = %.15f\n", drem(x2, x2));
printf(" * drem(4.5, 4.5) = %.15f\n", drem(y2, y2));
printf(" * drem(INFINITY, 0) = %.15f\n", drem(e, 0.0f));
printf(" * drem(-INFINITY, 3.0) = %.15f\n", drem(f, x1));
printf(" * drem(NAN, 3.0) = %.15f\n", drem(g, x1));
printf(" * drem(NAN, INFINITY) = %.15f\n", drem(g, e));
printf(" **/\n");
/*
 * drem(3.0, 4.0) = -1.000000000000000
 * drem(3.5, 4.5) = 0.000000000000000
 * drem(4.5, 4.5) = 0.000000000000000
 * drem(INFINITY, 0) = nan
 * drem(-INFINITY, 3.0) = nan
 * drem(NAN, 3.0) = nan
 * drem(NAN, INFINITY) = nan
 **/
```

6.4.3.2.8.2 fmod

浮点数取模，即x模y。

接口定义

C interface：

float fmodf(float x, float y);

double fmod(double x, double y);

long double fmodl(long double x, long double y);

参数

参数名	类型	描述	输入/输出
x	• 在fmodf中，x是单精度浮点类型。 • 在fmod中，x是双精度浮点类型。 • 在fmodl中，x是长双精度浮点类型。	表示输入数据的浮点值。	输入

 版权所有 © 华为技术有限公司

参数名	类型	描述	输入/输出
y	• 在fmodf中，y是单精度浮点类型。 • 在fmod中，y是双精度浮点类型。 • 在fmodl中，y是长双精度浮点类型。	表示输入数据的浮点值。	输入

返回值

- 返回x模y的结果。
- 输入x为任意值，y为±0或nan，返回nan。
- 输入x为±0，y不为±0且y不是nan，返回±0。
- 输入x为±∞或者nan，y为任意值，返回nan。

依赖

C: "km.h"

示例

C interface：

```
// typical usage
float x1 = -1.0, x2 = 2.0, x3 = 3.0, x4 = 4.0;
float y1 = 4.0, y2 = -4.0, y3 = -3.0, y4 = 3.0;
// special handling
float a = 0.0, b = INFINITY, c = -INFINITY, d = NAN;
// print result
printf("fmodf(-1.0, 4.0) = %.6f\n", fmodf(x1, y1));
printf("fmodf(2.0, -4.0) = %.6f\n", fmodf(x2, y2));
printf("fmodf(3.0, -3.0) = %.6f\n", fmodf(x3, y3));
printf("fmodf(4.0, 3.0) = %.6f\n", fmodf(x4, y4));
printf("fmodf(1.0, 0.0) = %+.6f\n", fmodf(x1, a));
printf("fmodf(INFINITY, 1.0) = %+.6f\n", fmodf(b, x1));
printf("fmodf(-INFINITY, -1.0) = %+.6f\n", fmodf(c, -x1));
printf("fmodf(1.0, -INFINITY) = %+.6f\n", fmodf(x1, c));
printf("fmodf(1.0, NAN) = %+.6f\n", fmodf(x1, d));
printf("fmodf(NAN, -1.0) = %+.6f\n", fmodf(d, -x1));

/*
 *  fmodf(-1.0, 4.0) = -1.000000
 *  fmodf(2.0, -4.0) = 2.000000
 *  fmodf(3.0, -3.0) = 0.000000
 *  fmodf(4.0, 3.0) = 1.000000
 *  fmodf(1.0, 0.0) = -nan
 *  fmodf(INFINITY, 1.0) = -nan
 *  fmodf(-INFINITY, -1.0) = -nan
 *  fmodf(1.0, -INFINITY) = +1.000000
 *  fmodf(1.0, NAN) = +nan
 *  fmodf(NAN, -1.0) = +nan
 * */
```

6.4.3.2.8.3 remainder

计算x/y的余数。

 版权所有 © 华为技术有限公司

接口定义

C interface：

float remainderf(float x, float y);

double remainder(double x, double y);

long double remainderl(long double x, long double y);

参数

参数名	类型	描述	输入/输出
x	• 在remainderf中，x是单精度浮点类型。 • 在remainder中，x是双精度浮点类型。 • 在remainderl中，x是长双精度浮点类型。	表示输入数据的浮点值。	输入
y	• 在remainderf中，y是单精度浮点类型。 • 在remainder中，y是双精度浮点类型。 • 在remainderl中，y是长双精度浮点类型。	表示输入数据的浮点值。	输入

返回值

- 返回x模y的结果。
- 输入x为任意值，y为±0或nan，返回nan。
- 输入x为±0，y不为±0和nan，返回±0。
- 输入x为±∞或者nan，y为任意值，返回nan。

依赖

C: "km.h"

示例

C interface：

```
// typical usage
float x1 = 3.0, x2 = 3.5;
float y1 = 4.0, y2 = 4.5;
// special handling
float e = INFINITY, f = -INFINITY, g = NAN;
// print result
printf("remainder(3.0, 4.0) = %.15f\n", remainder(x1, y1));
printf("remainder(3.5, 4.5) = %.15f\n", remainder(x2, y2));
printf("remainder(4.5, 4.5) = %.15f\n", remainder(y2, y2));
printf("remainder(INFINITY, 0) = %.15f\n", remainder(e, 0.0f));
```

 版权所有 © 华为技术有限公司

```
printf("remainder(-INFINITY, 3.0) = %.15f\n", remainder(f, x1));
printf("remainder(NAN, 3.0) = %.15f\n", remainder(g, x1));
printf("remainder(NAN, INFINITY) = %.15f\n", remainder(g, e));

/*
 *  remainder(3.0, 4.0) = -1.000000000000000
 *  remainder(3.5, 4.5) = 0.000000000000000
 *  remainder(4.5, 4.5) = 0.000000000000000
 *  remainder(INFINITY, 0) = nan
 *  remainder(-INFINITY, 3.0) = nan
 *  remainder(NAN, 3.0) = nan
 *  remainder(NAN, INFINITY) = nan
 * */
```

6.4.3.2.8.4 remquo

同时计算浮点x/y的商和余数，其中商对INT_MAX取余，余数的计算和remainder保持一致。

接口定义

C interface：

float remquof(float x, float y, int *quo);

double remquo(double x, double y, int *quo);

long double remquol(long double x, long double y, int *quo);

参数

参数名	类型	描述	输入/输出
x	• 在remquof中，x是单精度浮点类型。 • 在remquo中，x是双精度浮点类型。 • 在remquol中，x是长双精度浮点类型。	表示输入数据的浮点值。	输入
y	• 在remquof中，y是单精度浮点类型。 • 在remquo中，y是双精度浮点类型。 • 在remquol中，y是长双精度浮点类型。	表示输入数据的浮点值。	输入
quo	• 在remquof中，quo是int*整型。 • 在remquo中，quo是int*整型。 • 在remquol中，quo是int*整型。	表示输出数据的值。	输出

版权所有 © 华为技术有限公司

返回值

- 正常情况：返回x/y的商和余数。
- 特殊情况：
  - 输入|y|为0或nan，或者|x|为nan或inf，输出quo=0，返回nan。
  - 输入|y|为0，或者|y|为inf，输出quo=0，返回x。

依赖

C: "km.h"

示例

C interface：

```
// typical usage
double x1 = 3.0, x2 = 3.5;
double y1 = 4.0, y2 = 4.5;
// special handling
double e = INFINITY, f = -INFINITY, g = NAN;
// print result
double rem;
int quo;
printf("/*\n");
rem = remquo(x1, y1, &quo);
printf(" * remquo(3.0, 4.0) = %.15f, %d\n", rem, quo);
rem = remquo(x2, x2, &quo);
printf(" * remquo(3.5, 4.5) = %.15f, %d\n", rem, quo);
rem = remquo(y2, y2, &quo);
printf(" * remquo(4.5, 4.5) = %.15f, %d\n", rem, quo);
rem = remquo(e, 0.0f, &quo);
printf(" * remquo(INFINITY, 0) = %.15f, %d\n", rem, quo);
rem = remquo(f, x1, &quo);
printf(" * remquo(-INFINITY, 3.0) = %.15f, %d\n", rem, quo);
rem = remquo(g, x1, &quo);
printf(" * remquo(NAN, 3.0) = %.15f, %d\n", rem, quo);
rem = remquo(g, e, &quo);
printf(" * remquo(NAN, INFINITY) = %.15f, %d\n", rem, quo);
printf(" **/\n");
/*
 * remquo(3.0, 4.0) = -1.000000000000000, 1
 * remquo(3.5, 4.5) = 0.000000000000000, 1
 * remquo(4.5, 4.5) = 0.000000000000000, 1
 * remquo(INFINITY, 0) = nan, 0
 * remquo(-INFINITY, 3.0) = nan, 0
 * remquo(NAN, 3.0) = nan, 0
 * remquo(NAN, INFINITY) = nan, 0
 **/
```

6.4.3.2.9 复数函数

6.4.3.2.9.1 cabs

返回复数的绝对值。

接口定义

C interface：

float cabsf(float complex x);

 版权所有 © 华为技术有限公司

double cabs(double complex x);

double cabs_18(double complex x);

long double cabsl(long double complex x);

说明

cabs_18仅在高精度版本提供。

参数

参数名	类型	描述	输入/输出
x	• 在cabsf中，x是复数单精度浮点类型。 • 在cabs和cabs_18中，x是复数双精度浮点类型。 • 在cabsl中，x是复数长双精度浮点类型。	表示输入数据的浮点值。	输入

返回值

- 返回x的绝对值结果。
- 输入x.real或者x.imag为snan，返回nan。
- 输入x.real或者x.imag为±∞，返回+∞。
- 输入x.real或者x.imag为qnan，返回nan。
- 输入x.real为±0且x.imag为±0，返回+0。
- $\sqrt{x.real^2 + x.imag^2} \geq +\infty$，返回+∞。

依赖

C: "km.h"

示例

C interface：

```
// typical usage
double x1 = INFINITY, y1 = INFINITY;
double x2 = 2.0, y2 = 3.0;
double x3 = -2.5, y3 = -3.4;
double x4 = NAN, y4 = NAN;
double x5 = 0, y5 = 0;
// print result
printf("cabs(%.2f + %.2f*I) = %.6f\n", x1, y1, cabs(__builtin_complex(x1, y1)));
printf("cabs(%.2f + %.2f*I) = %.6f\n", x2, y2, cabs(__builtin_complex(x2, y2)));
printf("cabs(%.2f + %.2f*I) = %.6f\n", x3, y3, cabs(__builtin_complex(x3, y3)));
printf("cabs(%.2f + %.2f*I) = %.6f\n", x4, y4, cabs(__builtin_complex(x4, y4)));
printf("cabs(%.2f + %.2f*I) = %.6f\n", x5, y5, cabs(__builtin_complex(x5, y5)));

/*
 * cabs(inf + inf*I) = inf
 * cabs(2.00 + 3.00*I) = 3.605551
 * cabs(-2.50 + -3.40*I) = 4.220190
```

 版权所有 © 华为技术有限公司

```
 * cabs(nan + nan*I) = nan
 * cabs(0.00 + 0.00*I) = 0.000000
 **/
```

6.4.3.2.9.2 carg

返回复数极坐标形式的角度值。

接口定义

C interface：

float cargf(float complex x);

double carg(double complex x);

long double cargl(long double complex x);

参数

参数名	类型	描述	输入/输出
x	• 在cargf中，x是复数单精度浮点类型。 • 在carg中，x是复数双精度浮点类型。 • 在cargl中，x是复数长双精度浮点类型。	表示输入数据的浮点值。	输入

返回值

- 返回复数极坐标形式的角度值y，y ∈ [-pi, +pi]。
- 特殊值说明如下所示。

输入值实部（real）	输入值虚部（imag）	输出值（y）
+0	+0	+0
+0	-0	-0
+0	inf	pi/2
+0	-inf	-pi/2
+0	snan	nan
+0	-snan	nan
+0	qnan	nan
+0	-qnan	nan
-0	+0	pi
-0	-0	-pi

版权所有 © 华为技术有限公司

输入值实部（real）	输入值虚部（imag）	输出值（y）
-0	inf	pi/2
-0	-inf	-pi/2
-0	snan	nan
-0	-snan	nan
-0	qnan	nan
-0	-qnan	nan
inf	+0	+0
inf	-0	-0
inf	inf	pi/4
inf	-inf	-pi/4
inf	snan	nan
inf	-snan	nan
inf	qnan	nan
inf	-qnan	nan
-inf	+0	pi
-inf	-0	-pi
-inf	inf	3*pi/4
-inf	-inf	-3*pi/4
-inf	snan	nan
-inf	-snan	nan
-inf	qnan	nan
-inf	-qnan	nan
snan	+0	nan
snan	-0	nan
snan	inf	nan
snan	-inf	nan
snan	snan	nan
snan	-snan	nan
snan	qnan	nan
snan	-qnan	nan

 版权所有 © 华为技术有限公司

输入值实部（real）	输入值虚部（imag）	输出值（y）
-snan	+0	nan
-snan	-0	nan
-snan	inf	nan
-snan	-inf	nan
-snan	snan	nan
-snan	-snan	nan
-snan	qnan	nan
-snan	-qnan	nan
qnan	+0	nan
qnan	-0	nan
qnan	inf	nan
qnan	-inf	nan
qnan	snan	nan
qnan	-snan	nan
qnan	qnan	nan
qnan	-qnan	nan
-qnan	+0	nan
-qnan	-0	nan
-qnan	inf	nan
-qnan	-inf	nan
-qnan	snan	nan
-qnan	-snan	nan
-qnan	qnan	nan
-qnan	-qnan	nan

依赖

C: "km.h"

示例

C interface：

```
// typical usage
double x1 = INFINITY, y1 = INFINITY;
```

 版权所有 © 华为技术有限公司

```
double x2 = -INFINITY, y2 = 0;
double x3 = 0, y3 = -INFINITY;
double x4 = NAN, y4 = NAN;
double x5 = 0, y5 = 0;
// print result
printf("/*\n");
printf(" * carg(%.2f + %.2f*I) = %.6f\n", x1, y1, carg(__builtin_complex(x1, y1)));
printf(" * carg(%.2f + %.2f*I) = %.6f\n", x2, y2, carg(__builtin_complex(x2, y2)));
printf(" * carg(%.2f + %.2f*I) = %.6f\n", x3, y3, carg(__builtin_complex(x3, y3)));
printf(" * carg(%.2f + %.2f*I) = %.6f\n", x4, y4, carg(__builtin_complex(x4, y4)));
printf(" * carg(%.2f + %.2f*I) = %.6f\n", x5, y5, carg(__builtin_complex(x5, y5)));

/*
 * carg(inf + inf*I) = 0.785398
 * carg(-inf + 0.00*I) = 3.141593
 * carg(0.00 + -inf*I) = -1.570796
 * carg(nan + nan*I) = nan
 * carg(0.00 + 0.00*I) = 0.000000
 **/
```

6.4.3.2.9.3 cdiv

复数除法运算。

接口定义

C interface：

float complex cdivf(float complex x, float complex y);

double complex cdiv(double complex x, double complex y);

double complex cdiv_18(double complex x, double complex y);

说明

cdiv_18仅在高精度版本中提供。

参数

参数名	类型	描述	输入/输出
x	• 在cdivf中，x是复数单精度浮点类型。 • 在cdiv和cdiv_18中，x是复数双精度浮点类型。	表示输入数据的浮点值。	输入
y	• 在cdivf中，y是复数单精度浮点类型。 • 在cdiv和cdiv_18中，y是复数双精度浮点类型。	表示输入数据的浮点值。	输入

返回值

返回复数除法的运算结果。

 版权所有 © 华为技术有限公司

依赖

C: "km.h"

示例

C interface：

```
double x1 = 1.0, x2 = 1.0, x3 = 1.0, x4 = 1.0;
double complex cx1;
double complex cx2;
// print result
double ry;
double iy;
cx1 = __builtin_complex(x1, x2);
cx2 = __builtin_complex(x3, x4);
ry = __real__ cdiv(cx1, cx2);
iy = __imag__ cdiv(cx1, cx2);
printf("cdiv(cx1, cx2) = %.15f, %.15f\n", ry, iy);

/*
 * cdiv(cx1, cx2) = 1.000000000000000, 0.000000000000000
 */
```

6.4.3.2.9.4 cexp

计算复数以e为底数的指数值。

接口定义

C interface：

float complex cexpf(float complex x);

double complex cexp(double complex x);

double complex cexp_18(double complex x);

long double complex cexpl(long double complex x);

说明

cexp_18仅在高精度版本提供。

参数

参数名	类型	描述	输入/输出
x	• 在cexpf中，x是复数单精度浮点类型。 • 在cexp和cexp_18中，x是复数双精度浮点类型。 • 在cexpl中，x是复数长双精度浮点类型。	表示输入数据的浮点值。	输入

版权所有 © 华为技术有限公司

返回值

- 返回x的以e为底的指数值结果y。
- 特殊值说明如下所示。

输入值实部（x.real）	输入值虚部（x.imag）	输出值实部（y.real）	输出值虚部（y.imag）
+0	+0	+1	+0
+0	-0	+1	-0
+0	inf	nan	nan
+0	-inf	nan	nan
+0	snan	nan	nan
+0	-snan	nan	nan
+0	qnan	nan	nan
+0	-qnan	nan	nan
-0	+0	+1	+0
-0	-0	+1	-0
-0	inf	nan	nan
-0	-inf	nan	nan
-0	snan	nan	nan
-0	-snan	nan	nan
-0	qnan	nan	nan
-0	-qnan	nan	nan
inf	+0	inf	+0
inf	-0	inf	-0
inf	inf	inf	nan
inf	-inf	inf	nan
inf	snan	inf	nan
inf	-snan	inf	nan
inf	qnan	inf	nan
inf	-qnan	inf	nan
-inf	+0	+0	+0
-inf	-0	+0	-0
-inf	inf	+0	+0

 版权所有 © 华为技术有限公司

输入值实部（x.real）	输入值虚部（x.imag）	输出值实部（y.real）	输出值虚部（y.imag）
-inf	-inf	+0	+0
-inf	snan	+0	+0
-inf	-snan	+0	+0
-inf	qnan	+0	+0
-inf	-qnan	+0	+0
snan	+0	nan	+0
snan	-0	nan	-0
snan	inf	nan	nan
snan	-inf	nan	nan
snan	snan	nan	nan
snan	-snan	nan	nan
snan	qnan	nan	nan
snan	-qnan	nan	nan
-snan	+0	nan	+0
-snan	-0	nan	-0
-snan	inf	nan	nan
-snan	-inf	nan	nan
-snan	snan	nan	nan
-snan	-snan	nan	nan
-snan	qnan	nan	nan
-snan	-qnan	nan	nan
qnan	+0	nan	+0
qnan	-0	nan	-0
qnan	inf	nan	nan
qnan	-inf	nan	nan
qnan	snan	nan	nan
qnan	-snan	nan	nan
qnan	qnan	nan	nan
qnan	-qnan	nan	nan

版权所有 © 华为技术有限公司

输入值实部（x.real）	输入值虚部（x.imag）	输出值实部（y.real）	输出值虚部（y.imag）
-qnan	+0	nan	+0
-qnan	-0	nan	-0
-qnan	inf	nan	nan
-qnan	-inf	nan	nan
-qnan	snan	nan	nan
-qnan	-snan	nan	nan
-qnan	qnan	nan	nan
-qnan	-qnan	nan	nan

依赖

C: "kc.h"

示例

C interface：

```
// typical usage
double x1 = INFINITY, y1 = INFINITY;
double x2 = 2.0, y2 = 3.0;
double x3 = -2.5, y3 = -3.4;
double x4 = NAN, y4 = NAN;
double x5 = 0, y5 = 0;
double complex z1 = cexp(__builtin_complex(x1, y1));
double complex z2 = cexp(__builtin_complex(x2, y2));
double complex z3 = cexp(__builtin_complex(x3, y3));
double complex z4 = cexp(__builtin_complex(x4, y4));
double complex z5 = cexp(__builtin_complex(x5, y5));
// print result
printf("/*\n");
printf(" * cexp(%.2f + %.2f*I) = %.6f + %.6f*I\n", x1, y1, __real__(z1), __imag__(z1));
printf(" * cexp(%.2f + %.2f*I) = %.6f + %.6f*I\n", x2, y2, __real__(z2), __imag__(z2));
printf(" * cexp(%.2f + %.2f*I) = %.6f + %.6f*I\n", x3, y3, __real__(z3), __imag__(z3));
printf(" * cexp(%.2f + %.2f*I) = %.6f + %.6f*I\n", x4, y4, __real__(z4), __imag__(z4));
printf(" * cexp(%.2f + %.2f*I) = %.6f + %.6f*I\n", x5, y5, __real__(z5), __imag__(z5));
printf(" **/\n");

/*
 * cexp(inf + inf*I) = inf + nan*I
 * cexp(2.00 + 3.00*I) = -7.315110 + 1.042744*I
 * cexp(-2.50 + -3.40*I) = -0.079360 + 0.020976*I
 * cexp(nan + nan*I) = nan + nan*I
 * cexp(0.00 + 0.00*I) = 1.000000 + 0.000000*I
 **/
```

6.4.3.2.9.5 cexp2

计算复数以2为底数的指数值。

接口定义

C interface：

 版权所有 © 华为技术有限公司

float complex cexp2f(float complex x);

double complex cexp2(double complex x);

long double complex cexp2l(long double complex x);

参数

参数名	类型	描述	输入/输出
x	• 在cexp2f中，x是复数单精度浮点类型。 • 在cexp2中，x是复数双精度浮点类型。 • 在cexp2l中，x是复数长双精度浮点类型。	表示输入数据的浮点值。	输入

返回值

- 返回x的以2为底的指数值结果y。
- 特殊值说明如下所示。

输入值实部（x.real）	输入值虚部（x.imag）	输出值实部（y.real）	输出值虚部（y.imag）
+0	+0	+1	+0
+0	-0	+1	-0
+0	inf	nan	nan
+0	-inf	nan	nan
+0	snan	nan	nan
+0	-snan	nan	nan
+0	qnan	nan	nan
+0	-qnan	nan	nan
-0	+0	+1	+0
-0	-0	+1	-0
-0	inf	nan	nan
-0	-inf	nan	nan
-0	snan	nan	nan
-0	-snan	nan	nan
-0	qnan	nan	nan
-0	-qnan	nan	nan

版权所有 © 华为技术有限公司

输入值实部（x.real）	输入值虚部（x.imag）	输出值实部（y.real）	输出值虚部（y.imag）
inf	+0	inf	+0
inf	-0	inf	-0
inf	inf	inf	nan
inf	-inf	inf	nan
inf	snan	inf	nan
inf	-snan	inf	nan
inf	qnan	inf	nan
inf	-qnan	inf	nan
-inf	+0	+0	+0
-inf	-0	+0	-0
-inf	inf	+0	+0
-inf	-inf	+0	+0
-inf	snan	+0	+0
-inf	-snan	+0	+0
-inf	qnan	+0	+0
-inf	-qnan	+0	+0
snan	+0	nan	+0
snan	-0	nan	-0
snan	inf	nan	nan
snan	-inf	nan	nan
snan	snan	nan	nan
snan	-snan	nan	nan
snan	qnan	nan	nan
snan	-qnan	nan	nan
-snan	+0	nan	+0
-snan	-0	nan	-0
-snan	inf	nan	nan
-snan	-inf	nan	nan
-snan	snan	nan	nan

版权所有 © 华为技术有限公司

输入值实部（x.real）	输入值虚部（x.imag）	输出值实部（y.real）	输出值虚部（y.imag）
-snan	-snan	nan	nan
-snan	qnan	nan	nan
-snan	-qnan	nan	nan
qnan	+0	nan	+0
qnan	-0	nan	-0
qnan	inf	nan	nan
qnan	-inf	nan	nan
qnan	snan	nan	nan
qnan	-snan	nan	nan
qnan	qnan	nan	nan
qnan	-qnan	nan	nan
-qnan	+0	nan	+0
-qnan	-0	nan	-0
-qnan	inf	nan	nan
-qnan	-inf	nan	nan
-qnan	snan	nan	nan
-qnan	-snan	nan	nan
-qnan	qnan	nan	nan
-qnan	-qnan	nan	nan

依赖

C: "kc.h"

示例

C interface：

```
// typical usage
double x1 = INFINITY, y1 = INFINITY;
double x2 = 2.0, y2 = 3.0;
double x3 = -2.5, y3 = -3.4;
double x4 = NAN, y4 = NAN;
double x5 = 0, y5 = 0;
double complex z1 = cexp2(__builtin_complex(x1, y1));
double complex z2 = cexp2(__builtin_complex(x2, y2));
double complex z3 = cexp2(__builtin_complex(x3, y3));
double complex z4 = cexp2(__builtin_complex(x4, y4));
double complex z5 = cexp2(__builtin_complex(x5, y5));
// print result
```

 版权所有 © 华为技术有限公司

```
  printf("/*\n");
  printf(" * cexp2(%.2f + %.2f*I) = %.6f + %.6f*I\n", x1, y1, __real__(z1), __imag__(z1));
  printf(" * cexp2(%.2f + %.2f*I) = %.6f + %.6f*I\n", x2, y2, __real__(z2), __imag__(z2));
  printf(" * cexp2(%.2f + %.2f*I) = %.6f + %.6f*I\n", x3, y3, __real__(z3), __imag__(z3));
  printf(" * cexp2(%.2f + %.2f*I) = %.6f + %.6f*I\n", x4, y4, __real__(z4), __imag__(z4));
  printf(" * cexp2(%.2f + %.2f*I) = %.6f + %.6f*I\n", x5, y5, __real__(z5), __imag__(z5));
  printf(" **/\n");

 /*
  * cexp2(inf + inf*I) = inf + nan*I
  * cexp2(2.00 + 3.00*I) = -1.947978 + 3.493620*I
  * cexp2(-2.50 + -3.40*I) = -0.125063 + -0.124937*I
  * cexp2(nan + nan*I) = nan + nan*I
  * cexp2(0.00 + 0.00*I) = 1.000000 + 0.000000*I
  **/
```

6.4.3.2.9.6 cexp10

计算复数以10为底数的指数值。

接口定义

C interface：

float complex cexp10f(float complex x);

double complex cexp10(double complex x);

参数

参数名	类型	描述	输入/输出
x	• 在cexp10f中，x是复数单精度浮点类型。 • 在cexp10中，x是复数双精度浮点类型。	表示输入数据的浮点值。	输入

返回值

- 返回x的以10为底的指数值结果y。
- 特殊值说明如下。

输入值实部（x.real）	输入值虚部（x.imag）	输出值实部（y.real）	输出值虚部（y.imag）
+0	+0	+1	+0
+0	-0	+1	-0
+0	inf	nan	nan
+0	-inf	nan	nan
+0	snan	nan	nan
+0	-snan	nan	nan

版权所有 © 华为技术有限公司

输入值实部（x.real）	输入值虚部（x.imag）	输出值实部（y.real）	输出值虚部（y.imag）
+0	qnan	nan	nan
+0	-qnan	nan	nan
-0	+0	+1	+0
-0	-0	+1	-0
-0	inf	nan	nan
-0	-inf	nan	nan
-0	snan	nan	nan
-0	-snan	nan	nan
-0	qnan	nan	nan
-0	-qnan	nan	nan
inf	+0	inf	+0
inf	-0	inf	-0
inf	inf	inf	nan
inf	-inf	inf	nan
inf	snan	inf	nan
inf	-snan	inf	nan
inf	qnan	inf	nan
inf	-qnan	inf	nan
-inf	+0	+0	+0
-inf	-0	+0	-0
-inf	inf	+0	+0
-inf	-inf	+0	+0
-inf	snan	+0	+0
-inf	-snan	+0	+0
-inf	qnan	+0	+0
-inf	-qnan	+0	+0
snan	+0	nan	+0
snan	-0	nan	-0
snan	inf	nan	nan

版权所有 © 华为技术有限公司

输入值实部（x.real）	输入值虚部（x.imag）	输出值实部（y.real）	输出值虚部（y.imag）
snan	-inf	nan	nan
snan	snan	nan	nan
snan	-snan	nan	nan
snan	qnan	nan	nan
snan	-qnan	nan	nan
-snan	+0	nan	+0
-snan	-0	nan	-0
-snan	inf	nan	nan
-snan	-inf	nan	nan
-snan	snan	nan	nan
-snan	-snan	nan	nan
-snan	qnan	nan	nan
-snan	-qnan	nan	nan
qnan	+0	nan	+0
qnan	-0	nan	-0
qnan	inf	nan	nan
qnan	-inf	nan	nan
qnan	snan	nan	nan
qnan	-snan	nan	nan
qnan	qnan	nan	nan
qnan	-qnan	nan	nan
-qnan	+0	nan	+0
-qnan	-0	nan	-0
-qnan	inf	nan	nan
-qnan	-inf	nan	nan
-qnan	snan	nan	nan
-qnan	-snan	nan	nan
-qnan	qnan	nan	nan
-qnan	-qnan	nan	nan

版权所有 © 华为技术有限公司

依赖

C: "kc.h"

示例

C interface：

```
// typical usage
double x1 = INFINITY, y1 = INFINITY;
double x2 = 2.0, y2 = 3.0;
double x3 = -2.5, y3 = -3.4;
double x4 = NAN, y4 = NAN;
double x5 = 0, y5 = 0;
double complex z1 = cexp10(__builtin_complex(x1, y1));
double complex z2 = cexp10(__builtin_complex(x2, y2));
double complex z3 = cexp10(__builtin_complex(x3, y3));
double complex z4 = cexp10(__builtin_complex(x4, y4));
double complex z5 = cexp10(__builtin_complex(x5, y5));
// print result
printf("/*\n");
printf(" * cexp10(%.2f + %.2f*I) = %.6f + %.6f*I\n", x1, y1, __real__(z1), __imag__(z1));
printf(" * cexp10(%.2f + %.2f*I) = %.6f + %.6f*I\n", x2, y2, __real__(z2), __imag__(z2));
printf(" * cexp10(%.2f + %.2f*I) = %.6f + %.6f*I\n", x3, y3, __real__(z3), __imag__(z3));
printf(" * cexp10(%.2f + %.2f*I) = %.6f + %.6f*I\n", x4, y4, __real__(z4), __imag__(z4));
printf(" * cexp10(%.2f + %.2f*I) = %.6f + %.6f*I\n", x5, y5, __real__(z5), __imag__(z5));

/*
 * cexp10(inf + inf*I) = inf + nan*I
 * cexp10(2.00 + 3.00*I) = 81.121465 + 58.474848*I
 * cexp10(-2.50 + -3.40*I) = 0.000080 + -0.003161*I
 * cexp10(nan + nan*I) = nan + nan*I
 * cexp10(0.00 + 0.00*I) = 1.000000 + 0.000000*I
 **/
```

6.4.3.2.9.7 cpow

函数pow的复数接口。

接口定义

C interface：

float complex cpowf(float complex x, float complex y);

double complex cpow(double complex x, double complex y);

long double complex cpowl(long double complex x, long double complex y);

Fortran interface：

RES = POWF(X, Y)

RES = POW(X, Y)

版权所有 © 华为技术有限公司

参数

参数名	类型	描述	输入/输出
x	• 在cpowf中，x是复数单精度浮点类型。 • 在cpow中，x是复数双精度浮点类型。 • 在cpowl中，x是复数长双精度浮点类型。	表示输入数据的浮点值。	输入
y	• 在cpowf中，y是复数单精度浮点类型。 • 在cpow中，y是复数双精度浮点类型。 • 在cpowl中，y是复数长双精度浮点类型。	表示输入数据的浮点值。	输入

返回值

特殊值说明请在社区页面下载附件并查看。

依赖

C: "km.h"

示例

C interface：

```
double x1 = 1.0, x2 = 2.0, x3 = 3.0, x4 = 4.0;
double complex cx1 = __builtin_complex(x1, x2);
double complex cx2 = __builtin_complex(x3, x4);

// print result
double ry;
double iy;
ry = __real__ cpow(cx1, cx2);
iy = __imag__ cpow(cx1, cx2);
printf("cpow(cx1, cx2) = %.15f, %.15f\n", ry, iy);
/*
 * cpow(cx1, cx2) = 0.129009594074467, 0.033924092905170
 * */
```

Fortran interface：

```
REAL(8) :: X = 3.0
REAL(8) :: Y = -3.0
PRINT*,  POW(X, Y)
!
! OUTPUT
!     0.037037037037037
!
```

6.4.3.2.9.8 csqrt

计算复数x的平方根。

 版权所有 © 华为技术有限公司

接口定义

C interface：

float complex csqrtf(float complex x);

double complex csqrt(double complex x);

long double complex csqrtl(long double complex x);

Fortran interface：

RES = SQRTF(X);

RES = SQRT(X);

参数

参数名	类型	描述	输入/输出
x	• 在csqrtf中，x是复数单精度浮点类型。 • 在csqrt中，x是复数双精度浮点类型。 • 在csqrtl中，x是复数长双精度浮点类型。	表示输入数据的浮点值。	输入

返回值

输入值实部（x.real）	输入值虚部（x.imag）	输出值实部（y.real）	输出值虚部（y.imag）
±0	+0	+0	+0
±0	-0	+0	-0
±0	+inf	+inf	+inf
±0	-inf	+inf	-inf
+inf	+0	+inf	+0
+inf	-0	+inf	-0
-inf	+0	+0	+inf
-inf	-0	+0	-inf
±inf	+inf	+inf	+inf
±inf	-inf	+inf	-inf
任意数	nan	nan	nan
nan	任意数	nan	nan

 版权所有 © 华为技术有限公司

依赖

C: "km.h"

示例

C interface：

```
// typical usage
double x1 = 1.0, x2 = 2.0, x3 = 3.0, x4 = 4.0;
double complex cx1 = __builtin_complex(x1, x2);
double complex cx2 = __builtin_complex(x3, x4);

// print result
double ry;
double iy;
ry = __real__ csqrt(cx1);
iy = __imag__ csqrt(cx1);
printf("csqrt(cx1) = %.15f, %.15f\n", ry, iy);
ry = __real__ csqrt(cx2);
iy = __imag__ csqrt(cx2);
printf("csqrt(cx2) = %.15f, %.15f\n", ry, iy);
/*
 * csqrt(cx1) = 1.272019649514069, 0.786151377757423
 * csqrt(cx2) = 2.000000000000000, 1.000000000000000
 * */
```

Fortran interface：

```
REAL(8) :: X = 3.0
PRINT*,  SQRT(X)
!
! OUTPUT
!     1.732050807568877
!
```

6.4.3.2.9.9 clog

计算复数以e为底数的对数值。

接口定义

C interface：

float complex clogf(float complex x);

double complex clog(double complex x);

long double complex clogl(long double complex x);

参数

参数名	类型	描述	输入/输出
x	• 在clogf中，x是复数单精度浮点类型。 • 在clog中，x是复数双精度浮点类型。 • 在clogl中，x是复数长双精度浮点类型。	表示输入数据的浮点值。	输入

版权所有 © 华为技术有限公司

返回值

- 返回x的以e为底的对数值结果y。
- 特殊值说明如下所示。

输入值实部（x.real）	输入值虚部（x.imag）	输出值实部（y.real）	输出值虚部（y.imag）
+0	+0	-inf	+0
+0	-0	-inf	-0
+0	inf	inf	pi/2
+0	-inf	inf	-pi/2
+0	snan	nan	nan
+0	-snan	nan	nan
+0	qnan	nan	nan
+0	-qnan	nan	nan
-0	+0	-inf	pi
-0	-0	-inf	-pi
-0	inf	inf	pi/2
-0	-inf	inf	-pi/2
-0	snan	nan	nan
-0	-snan	nan	nan
-0	qnan	nan	nan
-0	-qnan	nan	nan
inf	+0	inf	+0
inf	-0	inf	-0
inf	inf	inf	pi/4
inf	-inf	inf	-pi/4
inf	snan	inf	nan
inf	-snan	inf	nan
inf	qnan	inf	nan
inf	-qnan	inf	nan
-inf	+0	inf	pi
-inf	-0	inf	-pi
-inf	inf	inf	3*pi/4

 版权所有 © 华为技术有限公司

输入值实部（x.real）	输入值虚部（x.imag）	输出值实部（y.real）	输出值虚部（y.imag）
-inf	-inf	inf	-3*pi/4
-inf	snan	inf	nan
-inf	-snan	inf	nan
-inf	qnan	inf	nan
-inf	-qnan	inf	nan
snan	+0	nan	nan
snan	-0	nan	nan
snan	inf	inf	nan
snan	-inf	inf	nan
snan	snan	nan	nan
snan	-snan	nan	nan
snan	qnan	nan	nan
snan	-qnan	nan	nan
-snan	+0	nan	nan
-snan	-0	nan	nan
-snan	inf	inf	nan
-snan	-inf	inf	nan
-snan	snan	nan	nan
-snan	-snan	nan	nan
-snan	qnan	nan	nan
-snan	-qnan	nan	nan
qnan	+0	nan	nan
qnan	-0	nan	nan
qnan	inf	inf	nan
qnan	-inf	inf	nan
qnan	snan	nan	nan
qnan	-snan	nan	nan
qnan	qnan	nan	nan
qnan	-qnan	nan	nan

版权所有 © 华为技术有限公司

输入值实部（x.real）	输入值虚部（x.imag）	输出值实部（y.real）	输出值虚部（y.imag）
-qnan	+0	nan	nan
-qnan	-0	nan	nan
-qnan	inf	inf	nan
-qnan	-inf	inf	nan
-qnan	snan	nan	nan
-qnan	-snan	nan	nan
-qnan	qnan	nan	nan
-qnan	-qnan	nan	nan

依赖

C: "kc.h"

示例

C interface：

```
double x1 = INFINITY, y1 = INFINITY;
double x2 = 2.0, y2 = 3.0;
double x3 = -2.5, y3 = -3.4;
double x4 = NAN, y4 = NAN;
double x5 = 0, y5 = 0;
double complex z1 = clog(__builtin_complex(x1, y1));
double complex z2 = clog(__builtin_complex(x2, y2));
double complex z3 = clog(__builtin_complex(x3, y3));
double complex z4 = clog(__builtin_complex(x4, y4));
double complex z5 = clog(__builtin_complex(x5, y5));
// print result
printf("/*\n");
printf(" * clog(%.2f + %.2f*I) = %.6f + %.6f*I\n", x1, y1, __real__(z1), __imag__(z1));
printf(" * clog(%.2f + %.2f*I) = %.6f + %.6f*I\n", x2, y2, __real__(z2), __imag__(z2));
printf(" * clog(%.2f + %.2f*I) = %.6f + %.6f*I\n", x3, y3, __real__(z3), __imag__(z3));
printf(" * clog(%.2f + %.2f*I) = %.6f + %.6f*I\n", x4, y4, __real__(z4), __imag__(z4));
printf(" * clog(%.2f + %.2f*I) = %.6f + %.6f*I\n", x5, y5, __real__(z5), __imag__(z5));

/*
 * clog(inf + inf*I) = inf + 0.785398*I
 * clog(2.00 + 3.00*I) = 1.282475 + 0.982794*I
 * clog(-2.50 + -3.40*I) = 1.439880 + -2.204819*I
 * clog(nan + nan*I) = nan + nan*I
 * clog(0.00 + 0.00*I) = -inf + 0.000000*I
 **/
```

6.4.3.2.9.10 clog2

计算复数以2为底数的对数值。

接口定义

C interface：

float complex clog2f(float complex x);

 版权所有 © 华为技术有限公司

double complex clog2(double complex x);

参数

参数名	类型	描述	输入/输出
x	• 在clog2f中，x是复数单精度浮点类型。 • 在clog2中，x是复数双精度浮点类型。	表示输入数据的浮点值。	输入

返回值

- 返回x的以2为底的对数值结果y。
- 特殊值说明如下所示。

输入值实部（x.real）	输入值虚部（x.imag）	输出值实部（y.real）	输出值虚部（y.imag）
+0	+0	-inf	+0
+0	-0	-inf	-0
+0	inf	inf	0x40022123045b5dec(pi/2ln2)
+0	-inf	inf	0xc0022123045b5dec(-pi/2ln2)
+0	snan	nan	nan
+0	-snan	nan	nan
+0	qnan	nan	nan
+0	-qnan	nan	nan
-0	+0	-inf	0x40122123045b5dec(pi/ln2)
-0	-0	-inf	0xc0122123045b5dec(-pi/ln2)
-0	inf	inf	0x40022123045b5dec(pi/2ln2)
-0	-inf	inf	0xc0022123045b5dec(-pi/2ln2)
-0	snan	nan	nan
-0	-snan	nan	nan
-0	qnan	nan	nan

版权所有 © 华为技术有限公司

输入值实部（x.real）	输入值虚部（x.imag）	输出值实部（y.real）	输出值虚部（y.imag）
-0	-qnan	nan	nan
inf	+0	inf	+0
inf	-0	inf	-0
inf	inf	inf	0x3ff22123045b5dec(pi/4ln2)
inf	-inf	inf	0xbff22123045b5dec(-pi/4ln2)
inf	snan	inf	nan
inf	-snan	inf	nan
inf	qnan	inf	nan
inf	-qnan	inf	nan
-inf	+0	inf	0x40122123045b5dec(pi/ln2)
-inf	-0	inf	0xc0122123045b5dec(-pi/ln2)
-inf	inf	inf	0x400b31b486890ce1(3*pi/4ln2)
-inf	-inf	inf	0xc00b31b486890ce1(-3*pi/4ln2)
-inf	snan	inf	nan
-inf	-snan	inf	nan
-inf	qnan	inf	nan
-inf	-qnan	inf	nan
snan	+0	nan	nan
snan	-0	nan	nan
snan	inf	inf	nan
snan	-inf	inf	nan
snan	snan	nan	nan
snan	-snan	nan	nan
snan	qnan	nan	nan
snan	-qnan	nan	nan
-snan	+0	nan	nan

版权所有 © 华为技术有限公司

输入值实部（x.real）	输入值虚部（x.imag）	输出值实部（y.real）	输出值虚部（y.imag）
-snan	-0	nan	nan
-snan	inf	inf	nan
-snan	-inf	inf	nan
-snan	snan	nan	nan
-snan	-snan	nan	nan
-snan	qnan	nan	nan
-snan	-qnan	nan	nan
qnan	+0	nan	nan
qnan	-0	nan	nan
qnan	inf	inf	nan
qnan	-inf	inf	nan
qnan	snan	nan	nan
qnan	-snan	nan	nan
qnan	qnan	nan	nan
qnan	-qnan	nan	nan
-qnan	+0	nan	nan
-qnan	-0	nan	nan
-qnan	inf	inf	nan
-qnan	-inf	inf	nan
-qnan	snan	nan	nan
-qnan	-snan	nan	nan
-qnan	qnan	nan	nan
-qnan	-qnan	nan	nan

依赖

C: "kc.h"

示例

C interface：

```
// typical usage
double x1 = INFINITY, y1 = INFINITY;
double x2 = 2.0, y2 = 3.0;
```

 版权所有 © 华为技术有限公司

```
double x3 = -2.5, y3 = -3.4;
double x4 = NAN, y4 = NAN;
double x5 = 0, y5 = 0;
double complex z1 = clog2(__builtin_complex(x1, y1));
double complex z2 = clog2(__builtin_complex(x2, y2));
double complex z3 = clog2(__builtin_complex(x3, y3));
double complex z4 = clog2(__builtin_complex(x4, y4));
double complex z5 = clog2(__builtin_complex(x5, y5));
// print result
printf("/*\n");
printf(" * clog2(%.2f + %.2f*I) = %.6f + %.6f*I\n", x1, y1, __real__(z1), __imag__(z1));
printf(" * clog2(%.2f + %.2f*I) = %.6f + %.6f*I\n", x2, y2, __real__(z2), __imag__(z2));
printf(" * clog2(%.2f + %.2f*I) = %.6f + %.6f*I\n", x3, y3, __real__(z3), __imag__(z3));
printf(" * clog2(%.2f + %.2f*I) = %.6f + %.6f*I\n", x4, y4, __real__(z4), __imag__(z4));
printf(" * clog2(%.2f + %.2f*I) = %.6f + %.6f*I\n", x5, y5, __real__(z5), __imag__(z5));
printf(" **/\n");
/*
 * clog2(inf + inf*I) = inf + 1.133090*I
 * clog2(2.00 + 3.00*I) = 1.850220 + 1.417872*I
 * clog2(-2.50 + -3.40*I) = 2.077308 + -3.180882*I
 * clog2(nan + nan*I) = nan + nan*I
 * clog2(0.00 + 0.00*I) = -inf + 0.000000*I
 **/
```

6.4.3.2.9.11 clog10

计算复数以10为底数的对数值。

接口定义

C interface：

float complex clog10f(float complex x);

double complex clog10(double complex x);

long double complex clog10l(long double complex x);

参数

参数名	类型	描述	输入/输出
x	• 在clog10f中，x是复数单精度浮点类型。 • 在clog10中，x是复数双精度浮点类型。 • 在clog10l中，x是复数长双精度浮点类型。	表示输入数据的浮点值。	输入

返回值

- 返回x的以10为底的对数值y。
- 特殊值说明如下所示。

 版权所有 © 华为技术有限公司

输入值实部（x.real）	输入值虚部（x.imag）	输出值实部（y.real）	输出值虚部（y.imag）
+0	+0	-inf	+0
+0	-0	-inf	-0
+0	inf	inf	0x3fe5d47c4cb2fba1(pi/(2ln10))
+0	-inf	inf	0xbfe5d47c4cb2fba1(-pi/(2ln10))
+0	snan	nan	nan
+0	-snan	nan	nan
+0	qnan	nan	nan
+0	-qnan	nan	nan
-0	+0	-inf	0x3ff5d47c4cb2fba1(pi/ln10)
-0	-0	-inf	0xbff5d47c4cb2fba1(-pi/ln10)
-0	inf	inf	0x3fe5d47c4cb2fba1(pi/(2ln10))
-0	-inf	inf	0xbfe5d47c4cb2fba1(-pi/(2ln10))
-0	snan	nan	nan
-0	-snan	nan	nan
-0	qnan	nan	nan
-0	-qnan	nan	nan
inf	+0	inf	+0
inf	-0	inf	-0
inf	inf	inf	0x3fd5d47c4cb2fba1(pi/(4ln10))
inf	-inf	inf	0xbfd5d47c4cb2fba1(-pi/(4ln10))
inf	snan	inf	nan
inf	-snan	inf	nan
inf	qnan	inf	nan
inf	-qnan	inf	nan

版权所有 © 华为技术有限公司

输入值实部（x.real）	输入值虚部（x.imag）	输出值实部（y.real）	输出值虚部（y.imag）
-inf	+0	inf	0x3ff5d47c4cb2fba1(pi/ln10)
-inf	-0	inf	0xbff5d47c4cb2fba1(-pi/ln10)
-inf	inf	inf	0x3ff05f5d39863cb9(3*pi/(4ln10))
-inf	-inf	inf	0xbff05f5d39863cb9(-3*pi/(4ln10))
-inf	snan	inf	nan
-inf	-snan	inf	nan
-inf	qnan	inf	nan
-inf	-qnan	inf	nan
snan	+0	nan	nan
snan	-0	nan	nan
snan	inf	inf	nan
snan	-inf	inf	nan
snan	snan	nan	nan
snan	-snan	nan	nan
snan	qnan	nan	nan
snan	-qnan	nan	nan
-snan	+0	nan	nan
-snan	-0	nan	nan
-snan	inf	inf	nan
-snan	-inf	inf	nan
-snan	snan	nan	nan
-snan	-snan	nan	nan
-snan	qnan	nan	nan
-snan	-qnan	nan	nan
qnan	+0	nan	nan
qnan	-0	nan	nan

 版权所有 © 华为技术有限公司

输入值实部（x.real）	输入值虚部（x.imag）	输出值实部（y.real）	输出值虚部（y.imag）
qnan	inf	inf	nan
qnan	-inf	inf	nan
qnan	snan	nan	nan
qnan	-snan	nan	nan
qnan	qnan	nan	nan
qnan	-qnan	nan	nan
-qnan	+0	nan	nan
-qnan	-0	nan	nan
-qnan	inf	inf	nan
-qnan	-inf	inf	nan
-qnan	snan	nan	nan
-qnan	-snan	nan	nan
-qnan	qnan	nan	nan
-qnan	-qnan	nan	nan

依赖

C: "kc.h"

示例

C interface：

```
// typical usage
double x1 = INFINITY, y1 = INFINITY;
double x2 = 2.0, y2 = 3.0;
double x3 = -2.5, y3 = -3.4;
double x4 = NAN, y4 = NAN;
double x5 = 0, y5 = 0;
double complex z1 = clog10(__builtin_complex(x1, y1));
double complex z2 = clog10(__builtin_complex(x2, y2));
double complex z3 = clog10(__builtin_complex(x3, y3));
double complex z4 = clog10(__builtin_complex(x4, y4));
double complex z5 = clog10(__builtin_complex(x5, y5));
// print result
printf("/*\n");
printf(" * clog10(%.2f + %.2f*I) = %.6f + %.6f*I\n", x1, y1, __real__(z1), __imag__(z1));
printf(" * clog10(%.2f + %.2f*I) = %.6f + %.6f*I\n", x2, y2, __real__(z2), __imag__(z2));
printf(" * clog10(%.2f + %.2f*I) = %.6f + %.6f*I\n", x3, y3, __real__(z3), __imag__(z3));
printf(" * clog10(%.2f + %.2f*I) = %.6f + %.6f*I\n", x4, y4, __real__(z4), __imag__(z4));
printf(" * clog10(%.2f + %.2f*I) = %.6f + %.6f*I\n", x5, y5, __real__(z5), __imag__(z5));
printf(" **/\n");
/*
 * clog10(inf + inf*I) = inf + 0.341094*I
 * clog10(2.00 + 3.00*I) = 0.556972 + 0.426822*I
 * clog10(-2.50 + -3.40*I) = 0.625332 + -0.957541*I
```

 版权所有 © 华为技术有限公司

```
 * clog10(nan + nan*I) = nan + nan*I
 * clog10(0.00 + 0.00*I) = -inf + 0.000000*I
 **/
```

6.4.3.2.9.12 ccos

计算复数的余弦值。

接口定义

C interface：

float complex ccosf(float complex x);

double complex ccos(double complex x);

参数

参数名	类型	描述	输入/输出
x	• 在ccosf中，x是复数单精度浮点类型。 • 在ccos中，x是复数双精度浮点类型。	表示输入数据的浮点值。	输入

返回值

- 正常区间：返回复数x的余弦值。
- 特殊区间：如下所示。

输入值实部（x.real）	输入值虚部（x.imag）	输出值实部（y.real）	输出值虚部（y.imag）
-inf	-inf	-inf	nan
-inf	< +0	nan	nan
-inf	-0	nan	-0
-inf	+0	nan	+0
-inf	> +0	nan	nan
-inf	inf	inf	nan
-inf	-nan	nan	nan
-inf	nan	nan	nan
< +0	-nan	nan	nan
< +0	nan	nan	nan
-0	-inf	inf	-0
-0	-0	1	-0

版权所有 © 华为技术有限公司

输入值实部（x.real）	输入值虚部（x.imag）	输出值实部（y.real）	输出值虚部（y.imag）
-0	+0	1	+0
-0	inf	inf	+0
-0	-nan	nan	-0
-0	nan	nan	-0
+0	-inf	inf	+0
+0	-0	1	+0
+0	+0	1	-0
+0	inf	inf	-0
+0	-nan	nan	+0
+0	nan	nan	+0
> +0	-nan	nan	nan
> +0	nan	nan	nan
inf	-inf	inf	nan
inf	< +0	nan	nan
inf	-0	nan	+0
inf	+0	nan	-0
inf	> +0	nan	nan
inf	inf	inf	nan
inf	-nan	nan	nan
inf	nan	nan	nan
-nan	-inf	inf	nan
-nan	< +0	nan	nan
-nan	-0	nan	-0
-nan	+0	nan	+0
-nan	> +0	nan	nan
-nan	inf	inf	nan
-nan	-nan	nan	nan
-nan	nan	nan	nan
nan	-inf	inf	nan

版权所有 © 华为技术有限公司

输入值实部（x.real）	输入值虚部（x.imag）	输出值实部（y.real）	输出值虚部（y.imag）
nan	< +0	nan	nan
nan	-0	nan	-0
nan	+0	nan	+0
nan	> +0	nan	nan
nan	inf	inf	nan
nan	-nan	nan	nan
nan	nan	nan	nan

依赖

C: "kc.h"

示例

C interface：

```
   // typical usage
   double x1 = 1.0, x2 = 2.0, x3 = 3.0, x4 = -4.0;
   double complex cx1 = __builtin_complex(x1, x2);
   double complex cx2 = __builtin_complex(x3, x4);
   double complex cx3 = __builtin_complex(INFINITY, -INFINITY);

   // print result
   double ry;
   double iy;
   ry = __real__ ccos(cx1);
   iy = __imag__ ccos(cx1);
   printf("ccos(cx1) = %.15f, %.15f\n", ry, iy);
   ry = __real__ ccos(cx2);
   iy = __imag__ ccos(cx2);
   printf("ccos(cx2) = %.15f, %.15f\n", ry, iy);
   ry = __real__ ccos(cx3);
   iy = __imag__ ccos(cx3);
   printf("ccos(cx3) = %.15f, %.15f\n", ry, iy);

result
   /*
    * ccos(cx1) = 2.032723007019666, -3.051897799151800
    * ccos(cx2) = -27.034945603074224, 3.851153334811777
    * ccos(cx3) = inf, nan
    * */
```

6.4.3.2.9.13 ccosh

计算复数的双曲余弦值。

接口定义

C interface：

float complex ccoshf(float complex x);

double complex ccosh(double complex x);

 版权所有 © 华为技术有限公司

参数

参数名	类型	描述	输入/输出
x	• 在ccoshf中，x是复数单精度浮点类型。 • 在ccosh中，x是复数双精度浮点类型。	表示输入数据的浮点值。	输入

返回值

- 返回x的双曲余弦结果y。
- 特殊值说明如下所示。

输入值实部（x.real）	输入值虚部（x.imag）	输出值实部（y.real）	输出值虚部（y.imag）
0.0	0.0	0.0	0.0
0.0	-0.0	0.0	-0.0
0.0	inf	0.0	nan
0.0	-inf	0.0	nan
0.0	nan	0.0	nan
0.0	-nan	0.0	-nan
-0.0	0.0	-0.0	0.0
-0.0	-0.0	-0.0	-0.0
-0.0	inf	-0.0	nan
-0.0	-inf	-0.0	nan
-0.0	nan	-0.0	nan
-0.0	-nan	-0.0	-nan
inf	0.0	inf	0.0
inf	-0.0	inf	-0.0
inf	inf	inf	nan
inf	-inf	inf	nan
inf	nan	inf	nan
inf	-nan	inf	-nan
-inf	0.0	-inf	0.0
-inf	-0.0	-inf	-0.0

版权所有 © 华为技术有限公司

输入值实部（x.real）	输入值虚部（x.imag）	输出值实部（y.real）	输出值虚部（y.imag）
-inf	inf	-inf	nan
-inf	-inf	-inf	nan
-inf	nan	-inf	nan
-inf	-nan	-inf	-nan
nan	0.0	nan	0.0
nan	-0.0	nan	-0.0
nan	inf	nan	nan
nan	-inf	nan	nan
nan	nan	nan	nan
nan	-nan	nan	-nan
-nan	0.0	-nan	0.0
-nan	-0.0	-nan	-0.0
-nan	inf	-nan	nan
-nan	-inf	-nan	nan
-nan	nan	-nan	nan
-nan	-nan	-nan	-nan

依赖

C: "kc.h"

示例

C interface：

```
// typical usage
double x1 = INFINITY, y1 = INFINITY;
double x2 = 2.0, y2 = 3.0;
double x3 = -2.5, y3 = -3.4;
double x4 = NAN, y4 = NAN;
double x5 = 0, y5 = 0;
double complex z1 = ccosh(__builtin_complex(x1, y1));
double complex z2 = ccosh(__builtin_complex(x2, y2));
double complex z3 = ccosh(__builtin_complex(x3, y3));
double complex z4 = ccosh(__builtin_complex(x4, y4));
double complex z5 = ccosh(__builtin_complex(x5, y5));
// print result
printf("/*\n");
printf(" * ccosh(%.2f + %.2f*I) = %.6f + %.6f*I\n", x1, y1, __real__(z1), __imag__(z1));
printf(" * ccosh(%.2f + %.2f*I) = %.6f + %.6f*I\n", x2, y2, __real__(z2), __imag__(z2));
printf(" * ccosh(%.2f + %.2f*I) = %.6f + %.6f*I\n", x3, y3, __real__(z3), __imag__(z3));
printf(" * ccosh(%.2f + %.2f*I) = %.6f + %.6f*I\n", x4, y4, __real__(z4), __imag__(z4));
printf(" * ccosh(%.2f + %.2f*I) = %.6f + %.6f*I\n", x5, y5, __real__(z5), __imag__(z5));
```


版权所有 © 华为技术有限公司

```
  printf(" **/\n");

/*
 * ccosh(inf + inf*I) = inf + nan*I
 * ccosh(2.00 + 3.00*I) = -3.724546 + 0.511823*I
 * ccosh(-2.50 + -3.40*I) = -5.928686 + -1.546076*I
 * ccosh(nan + nan*I) = nan + nan*I
 * ccosh(0.00 + 0.00*I) = 1.000000 + 0.000000*I
 **/
```

6.4.3.2.9.14 cacos

计算复数反余弦值。

接口定义

C interface：

float complex cacosf(float complex x);

double complex cacos(double complex x);

参数

参数名	类型	描述	输入/输出
x	• 在cacosf中，x是复数单精度浮点类型。 • 在cacos中，x是复数双精度浮点类型。	表示输入数据的浮点值。	输入

返回值

- 正常区间：返回x复数的反余弦值。
- 特殊区间：如下所示。

输入值实部（x.real）	输入值虚部（x.imag）	输出值实部（y.real）	输出值虚部（y.imag）
0	0	pi/2	0
0	0	pi/2	0
0	inf	pi/2	-inf
0	-inf	pi/2	inf
0	snan	pi/2	nan
0	-snan	pi/2	nan
0	qnan	pi/2	nan
0	-qnan	pi/2	nan
0	0	pi/2	0

版权所有 © 华为技术有限公司

输入值实部（x.real）	输入值虚部（x.imag）	输出值实部（y.real）	输出值虚部（y.imag）
0	0	pi/2	0
0	inf	pi/2	-inf
0	-inf	pi/2	inf
0	snan	pi/2	nan
0	-snan	pi/2	nan
0	qnan	pi/2	nan
0	-qnan	pi/2	nan
inf	0	0	-inf
inf	0	0	inf
inf	inf	pi/4	-inf
inf	-inf	pi/4	inf
inf	snan	nan	-inf
inf	-snan	nan	-inf
inf	qnan	nan	-inf
inf	-qnan	nan	-inf
-inf	0	pi	-inf
-inf	0	pi	inf
-inf	inf	3*pi/4	-inf
-inf	-inf	3*pi/4	inf
-inf	snan	nan	-inf
-inf	-snan	nan	-inf
-inf	qnan	nan	-inf
-inf	-qnan	nan	-inf
snan	0	nan	nan
snan	0	nan	nan
snan	inf	nan	-inf
snan	-inf	nan	inf
snan	snan	nan	nan
snan	-snan	nan	nan

版权所有 © 华为技术有限公司

输入值实部（x.real）	输入值虚部（x.imag）	输出值实部（y.real）	输出值虚部（y.imag）
snan	qnan	nan	nan
snan	-qnan	nan	nan
-snan	0	nan	nan
-snan	0	nan	nan
-snan	inf	nan	-inf
-snan	-inf	nan	inf
-snan	snan	nan	nan
-snan	-snan	nan	nan
-snan	qnan	nan	nan
-snan	-qnan	nan	nan
qnan	0	nan	nan
qnan	0	nan	nan
qnan	inf	nan	-inf
qnan	-inf	nan	inf
qnan	snan	nan	nan
qnan	-snan	nan	nan
qnan	qnan	nan	nan
qnan	-qnan	nan	nan
-qnan	0	nan	nan
-qnan	0	nan	nan
-qnan	inf	nan	-inf
-qnan	-inf	nan	inf
-qnan	snan	nan	nan
-qnan	-snan	nan	nan
-qnan	qnan	nan	nan
-qnan	-qnan	nan	nan

依赖

C: "kc.h"

 版权所有 © 华为技术有限公司

示例

C interface：

```
// typical usage
double x1 = INFINITY, y1 = INFINITY;
double x2 = 2.0, y2 = 3.0;
double x3 = -2.5, y3 = -3.4;
double x4 = NAN, y4 = NAN;
double x5 = 0, y5 = 0;
// print result
printf("/*\n");
printf(" * cacos(%.2f + %.2f*I) = %.6f\n", x1, y1, cacos(__builtin_complex(x1, y1)));
printf(" * cacos(%.2f + %.2f*I) = %.6f\n", x2, y2, cacos(__builtin_complex(x2, y2)));
printf(" * cacos(%.2f + %.2f*I) = %.6f\n", x3, y3, cacos(__builtin_complex(x3, y3)));
printf(" * cacos(%.2f + %.2f*I) = %.6f\n", x4, y4, cacos(__builtin_complex(x4, y4)));
printf(" * cacos(%.2f + %.2f*I) = %.6f\n", x5, y5, cacos(__builtin_complex(x5, y5)));
printf(" **/\n");
/*
 * cacos(inf + inf*I) = 0.785398
 * cacos(2.00 + 3.00*I) = 1.000144
 * cacos(-2.50 + -3.40*I) = 2.191594
 * cacos(nan + nan*I) = nan
 * cacos(0.00 + 0.00*I) = 1.570796
 **/
```

6.4.3.2.9.15 cacosh

计算复数反双曲余弦值。

接口定义

C interface：

float complex cacoshf(float complex x);

double complex cacosh(double complex x);

参数

参数名	类型	描述	输入/输出
x	• 在cacoshf中，x是复数单精度浮点类型。 • 在cacosh中，x是复数双精度浮点类型。	表示输入数据的浮点值。	输入

返回值

- 正常区间：返回x复数的反双曲余弦值。
- 特殊区间：如下所示。

输入值实部（x.real）	输入值虚部（x.imag）	输出值实部（y.real）	输出值虚部（y.imag）
0	0	0	pi/2
0	0	0	-pi/2

版权所有 © 华为技术有限公司

输入值实部（x.real）	输入值虚部（x.imag）	输出值实部（y.real）	输出值虚部（y.imag）
0	inf	inf	pi/2
0	-inf	inf	-pi/2
0	snan	nan	nan
0	-snan	nan	nan
0	qnan	nan	nan
0	-qnan	nan	nan
0	0	0	pi/2
0	0	0	-pi/2
0	inf	inf	pi/2
0	-inf	inf	-pi/2
0	snan	nan	nan
0	-snan	nan	nan
0	qnan	nan	nan
0	-qnan	nan	nan
inf	0	inf	0
inf	0	inf	0
inf	inf	inf	pi/4
inf	-inf	inf	-pi/4
inf	snan	inf	nan
inf	-snan	inf	nan
inf	qnan	inf	nan
inf	-qnan	inf	nan
-inf	0	inf	pi
-inf	0	inf	-pi
-inf	inf	inf	3*pi/4
-inf	-inf	inf	-3*pi/4
-inf	snan	inf	nan
-inf	-snan	inf	nan
-inf	qnan	inf	nan

版权所有 © 华为技术有限公司

输入值实部（x.real）	输入值虚部（x.imag）	输出值实部（y.real）	输出值虚部（y.imag）
-inf	-qnan	inf	nan
snan	0	nan	nan
snan	0	nan	nan
snan	inf	inf	nan
snan	-inf	inf	nan
snan	snan	nan	nan
snan	-snan	nan	nan
snan	qnan	nan	nan
snan	-qnan	nan	nan
-snan	0	nan	nan
-snan	0	nan	nan
-snan	inf	inf	nan
-snan	-inf	inf	nan
-snan	snan	nan	nan
-snan	-snan	nan	nan
-snan	qnan	nan	nan
-snan	-qnan	nan	nan
qnan	0	nan	nan
qnan	0	nan	nan
qnan	inf	inf	nan
qnan	-inf	inf	nan
qnan	snan	nan	nan
qnan	-snan	nan	nan
qnan	qnan	nan	nan
qnan	-qnan	nan	nan
-qnan	0	nan	nan
-qnan	0	nan	nan
-qnan	inf	inf	nan
-qnan	-inf	inf	nan

版权所有 © 华为技术有限公司

输入值实部（x.real）	输入值虚部（x.imag）	输出值实部（y.real）	输出值虚部（y.imag）
-qnan	snan	nan	nan
-qnan	-snan	nan	nan
-qnan	qnan	nan	nan
-qnan	-qnan	nan	nan

依赖

C: "kc.h"

示例

C interface：

```
// typical usage
double x1 = INFINITY, y1 = INFINITY;
double x2 = 2.0, y2 = 3.0;
double x3 = -2.5, y3 = -3.4;
double x4 = NAN, y4 = NAN;
double x5 = 0, y5 = 0;
// print result
printf("/*\n");
printf(" * cacosh(%.2f + %.2f*I) = %.6f\n", x1, y1, cacosh(__builtin_complex(x1, y1)));
printf(" * cacosh(%.2f + %.2f*I) = %.6f\n", x2, y2, cacosh(__builtin_complex(x2, y2)));
printf(" * cacosh(%.2f + %.2f*I) = %.6f\n", x3, y3, cacosh(__builtin_complex(x3, y3)));
printf(" * cacosh(%.2f + %.2f*I) = %.6f\n", x4, y4, cacosh(__builtin_complex(x4, y4)));
printf(" * cacosh(%.2f + %.2f*I) = %.6f\n", x5, y5, cacosh(__builtin_complex(x5, y5)));
printf(" **/\n");
/*
 * cacosh(inf + inf*I) = inf
 * cacosh(2.00 + 3.00*I) = 1.983387
 * cacosh(-2.50 + -3.40*I) = 2.137448
 * cacosh(nan + nan*I) = nan
 * cacosh(0.00 + 0.00*I) = 0.000000
 **/
```

6.4.3.2.9.16 csin

计算复数的正弦值。

接口定义

C interface：

float complex csinf(float complex x);

double complex csin(double complex x);

版权所有 © 华为技术有限公司

参数

参数名	类型	描述	输入/输出
x	• 在csinf中，x是复数单精度浮点类型。 • 在csin中，x是复数双精度浮点类型。	表示输入数据的浮点值。	输入

返回值

- 正常区间：返回复数x的正弦值。
- 特殊区间：如下所示。

输入值实部（x.real）	输入值虚部（x.imag）	输出值实部（y.real）	输出值虚部（y.imag）
-inf	-inf	nan	-inf
-inf	< +0	nan	nan
-inf	-0	nan	-0
-inf	+0	nan	+0
-inf	> +0	nan	nan
-inf	inf	nan	inf
-inf	-nan	nan	nan
-inf	nan	nan	nan
< +0	-nan	nan	nan
< +0	nan	nan	nan
-0	-inf	-0	-inf
-0	-0	-0	-0
-0	+0	-0	+0
-0	inf	-0	inf
-0	-nan	-0	nan
-0	nan	-0	nan
+0	-inf	0	-inf
+0	-0	0	-0
+0	+0	0	+0
+0	inf	0	inf

版权所有 © 华为技术有限公司

输入值实部（x.real）	输入值虚部（x.imag）	输出值实部（y.real）	输出值虚部（y.imag）
+0	-nan	0	nan
+0	nan	0	nan
> +0	-nan	nan	nan
> +0	nan	nan	nan
inf	-inf	nan	-inf
inf	< +0	nan	nan
inf	-0	nan	-0
inf	+0	nan	+0
inf	> +0	nan	nan
inf	inf	nan	inf
inf	-nan	nan	nan
inf	nan	nan	nan
-nan	-inf	nan	-inf
-nan	< +0	nan	nan
-nan	-0	nan	-0
-nan	+0	nan	+0
-nan	> +0	nan	nan
-nan	inf	nan	inf
-nan	-nan	nan	nan
-nan	nan	nan	nan
nan	-inf	nan	-inf
nan	< +0	nan	nan
nan	-0	nan	-0
nan	+0	nan	+0
nan	> +0	nan	nan
nan	inf	nan	inf
nan	-nan	nan	nan
nan	nan	nan	nan

版权所有 © 华为技术有限公司

依赖

C: "kc.h"

示例

C interface：

```
// typical usage
double x1 = 1.0, x2 = 2.0, x3 = 3.0, x4 = -4.0;
double complex cx1 = __builtin_complex(x1, x2);
double complex cx2 = __builtin_complex(x3, x4);
double complex cx3 = __builtin_complex(INFINITY, -INFINITY);

// print result
double ry;
double iy;
ry = __real__ csin(cx1);
iy = __imag__ csin(cx1);
printf("csin(cx1) = %.15f, %.15f\n", ry, iy);
ry = __real__ csin(cx2);
iy = __imag__ csin(cx2);
printf("csin(cx2) = %.15f, %.15f\n", ry, iy);
ry = __real__ csin(cx3);
iy = __imag__ csin(cx3);
printf("csin(cx3) = %.15f, %.15f\n", ry, iy);


result
/*
 * csin(cx1) = 3.165778513216168, 1.959601041421606
 * csin(cx2) = 3.853738037919377, 27.016813258003936
 * csin(cx3) = nan, -inf
 * */
```

6.4.3.2.9.17 csinh

计算复数的双曲正弦值。

接口定义

C interface：

float complex csinhf(float complex x);

double complex csinh(double complex x);

参数

参数名	类型	描述	输入/输出
x	• 在csinhf中，x是复数单精度浮点类型。 • 在csinh中，x是复数双精度浮点类型。	表示输入数据的浮点值。	输入

返回值

- 返回x的双曲正弦结果y。

 版权所有 © 华为技术有限公司

- 特殊值说明如下所示。

输入值实部（x.real）	输入值虚部（x.imag）	输出值实部（y.real）	输出值虚部（y.imag）
0.0	0.0	1.0	0.0
0.0	-0.0	1.0	-0.0
0.0	inf	nan	0.0
0.0	-inf	nan	-0.0
0.0	nan	nan	0.0
0.0	-nan	-nan	0.0
-0.0	0.0	1.0	-0.0
-0.0	-0.0	1.0	0.0
-0.0	inf	nan	-0.0
-0.0	-inf	nan	0.0
-0.0	nan	nan	0.0
-0.0	-nan	-nan	0.0
inf	0.0	inf	0.0
inf	-0.0	inf	-0.0
inf	inf	inf	nan
inf	-inf	inf	nan
inf	nan	inf	nan
inf	-nan	inf	-nan
-inf	0.0	inf	-0.0
-inf	-0.0	inf	0.0
-inf	inf	inf	nan
-inf	-inf	inf	nan
-inf	nan	inf	nan
-inf	-nan	inf	-nan
nan	0.0	nan	0.0
nan	-0.0	nan	-0.0
nan	inf	nan	nan
nan	-inf	nan	nan
nan	nan	nan	nan

 版权所有 © 华为技术有限公司

输入值实部（x.real）	输入值虚部（x.imag）	输出值实部（y.real）	输出值虚部（y.imag）
nan	-nan	nan	-nan
-nan	0.0	-nan	-0.0
-nan	-0.0	-nan	0.0
-nan	inf	-nan	nan
-nan	-inf	-nan	nan
-nan	nan	-nan	nan
-nan	-nan	-nan	-nan

依赖

C: "kc.h"

示例

C interface：

```
// typical usage
double x1 = INFINITY, y1 = INFINITY;
double x2 = 2.0, y2 = 3.0;
double x3 = -2.5, y3 = -3.4;
double x4 = NAN, y4 = NAN;
double x5 = 0, y5 = 0;
double complex z1 = csinh(__builtin_complex(x1, y1));
double complex z2 = csinh(__builtin_complex(x2, y2));
double complex z3 = csinh(__builtin_complex(x3, y3));
double complex z4 = csinh(__builtin_complex(x4, y4));
double complex z5 = csinh(__builtin_complex(x5, y5));
// print result
printf("/*\n");
printf(" * csinh(%.2f + %.2f*I) = %.6f + %.6f*I\n", x1, y1, __real__(z1), __imag__(z1));
printf(" * csinh(%.2f + %.2f*I) = %.6f + %.6f*I\n", x2, y2, __real__(z2), __imag__(z2));
printf(" * csinh(%.2f + %.2f*I) = %.6f + %.6f*I\n", x3, y3, __real__(z3), __imag__(z3));
printf(" * csinh(%.2f + %.2f*I) = %.6f + %.6f*I\n", x4, y4, __real__(z4), __imag__(z4));
printf(" * csinh(%.2f + %.2f*I) = %.6f + %.6f*I\n", x5, y5, __real__(z5), __imag__(z5));
/*
 * csinh(inf + inf*I) = inf + nan*I
 * csinh(2.00 + 3.00*I) = -3.590565 + 0.530921*I
 * csinh(-2.50 + -3.40*I) = 5.849327 + 1.567052*I
 * csinh(nan + nan*I) = nan + nan*I
 * csinh(0.00 + 0.00*I) = 0.000000 + 0.000000*I
 **/
```

6.4.3.2.9.18 casin

计算复数反正弦值。

接口定义

C interface：

float complex casinf(float complex x);

double complex casin(double complex x);

 版权所有 © 华为技术有限公司

参数

参数名	类型	描述	输入/输出
x	• 在casinf中，x是复数单精度浮点类型。 • 在casin中，x是复数双精度浮点类型。	表示输入数据的浮点值。	输入

返回值

- 正常区间：返回x复数的反正弦值。
- 特殊区间：如下所示。

输入值实部（x.real）	输入值虚部（x.imag）	输出值实部（y.real）	输出值虚部（y.imag）
0	0	0	0
0	0	0	0
0	inf	0	inf
0	-inf	0	-inf
0	snan	0	nan
0	-snan	0	nan
0	qnan	0	nan
0	-qnan	0	nan
0	0	0	0
0	0	0	0
0	inf	0	inf
0	-inf	0	-inf
0	snan	0	nan
0	-snan	0	nan
0	qnan	0	nan
0	-qnan	0	nan
inf	0	pi/2	inf
inf	0	pi/2	-inf
inf	inf	pi/4	inf
inf	-inf	pi/4	-inf

版权所有 © 华为技术有限公司

输入值实部（x.real）	输入值虚部（x.imag）	输出值实部（y.real）	输出值虚部（y.imag）
inf	snan	nan	inf
inf	-snan	nan	inf
inf	qnan	nan	inf
inf	-qnan	nan	inf
-inf	0	-pi/2	inf
-inf	0	-pi/2	-inf
-inf	inf	-pi/4	inf
-inf	-inf	-pi/4	-inf
-inf	snan	nan	inf
-inf	-snan	nan	inf
-inf	qnan	nan	inf
-inf	-qnan	nan	inf
snan	0	nan	nan
snan	0	nan	nan
snan	inf	nan	inf
snan	-inf	nan	-inf
snan	snan	nan	nan
snan	-snan	nan	nan
snan	qnan	nan	nan
snan	-qnan	nan	nan
-snan	0	nan	nan
-snan	0	nan	nan
-snan	inf	nan	inf
-snan	-inf	nan	-inf
-snan	snan	nan	nan
-snan	-snan	nan	nan
-snan	qnan	nan	nan
-snan	-qnan	nan	nan
qnan	0	nan	nan

 版权所有 © 华为技术有限公司

输入值实部（x.real）	输入值虚部（x.imag）	输出值实部（y.real）	输出值虚部（y.imag）
qnan	0	nan	nan
qnan	inf	nan	inf
qnan	-inf	nan	-inf
qnan	snan	nan	nan
qnan	-snan	nan	nan
qnan	qnan	nan	nan
qnan	-qnan	nan	nan
-qnan	0	nan	nan
-qnan	0	nan	nan
-qnan	inf	nan	inf
-qnan	-inf	nan	-inf
-qnan	snan	nan	nan
-qnan	-snan	nan	nan
-qnan	qnan	nan	nan
-qnan	-qnan	nan	nan

依赖

C: "kc.h"

示例

C interface：

```
// typical usage
double x1 = INFINITY, y1 = INFINITY;
double x2 = 2.0, y2 = 3.0;
double x3 = -2.5, y3 = -3.4;
double x4 = NAN, y4 = NAN;
double x5 = 0, y5 = 0;
// print result
printf("/*\n");
printf(" * casin(%.2f + %.2f*I) = %.6f\n", x1, y1, casin(__builtin_complex(x1, y1)));
printf(" * casin(%.2f + %.2f*I) = %.6f\n", x2, y2, casin(__builtin_complex(x2, y2)));
printf(" * casin(%.2f + %.2f*I) = %.6f\n", x3, y3, casin(__builtin_complex(x3, y3)));
printf(" * casin(%.2f + %.2f*I) = %.6f\n", x4, y4, casin(__builtin_complex(x4, y4)));
printf(" * casin(%.2f + %.2f*I) = %.6f\n", x5, y5, casin(__builtin_complex(x5, y5)));
printf(" **/\n");
/*
 * casin(inf + inf*I) = 0.785398
 * casin(2.00 + 3.00*I) = 0.570653
 * casin(-2.50 + -3.40*I) = -0.620798
 * casin(nan + nan*I) = nan
 * casin(0.00 + 0.00*I) = 0.000000
 **/
```


版权所有 © 华为技术有限公司

6.4.3.2.9.19 casinh

计算复数反双曲正弦值。

接口定义

C interface：

float complex casinhf(float complex x);

double complex casinh(double complex x);

参数

参数名	类型	描述	输入/输出
x	• 在casinhf中，x是复数单精度浮点类型。 • 在casinh中，x是复数双精度浮点类型。	表示输入数据的浮点值。	输入

返回值

- 正常区间：返回x复数的反双曲正弦值。
- 特殊区间：如下所示。

输入值实部（x.real）	输入值虚部（x.imag）	输出值实部（y.real）	输出值虚部（y.imag）
0	0	0	0
0	0	0	0
0	inf	inf	pi/2
0	-inf	inf	-pi/2
0	snan	nan	nan
0	-snan	nan	nan
0	qnan	nan	nan
0	-qnan	nan	nan
0	0	0	0
0	0	0	0
0	inf	-inf	pi/2
0	-inf	-inf	-pi/2
0	snan	nan	nan
0	-snan	nan	nan

版权所有 © 华为技术有限公司

输入值实部（x.real）	输入值虚部（x.imag）	输出值实部（y.real）	输出值虚部（y.imag）
0	qnan	nan	nan
0	-qnan	nan	nan
inf	0	inf	0
inf	0	inf	0
inf	inf	inf	pi/4
inf	-inf	inf	-pi/4
inf	snan	inf	nan
inf	-snan	inf	nan
inf	qnan	inf	nan
inf	-qnan	inf	nan
-inf	0	-inf	0
-inf	0	-inf	0
-inf	inf	-inf	pi/4
-inf	-inf	-inf	-pi/4
-inf	snan	-inf	nan
-inf	-snan	-inf	nan
-inf	qnan	-inf	nan
-inf	-qnan	-inf	nan
snan	0	nan	0
snan	0	nan	0
snan	inf	inf	nan
snan	-inf	inf	nan
snan	snan	nan	nan
snan	-snan	nan	nan
snan	qnan	nan	nan
snan	-qnan	nan	nan
-snan	0	nan	0
-snan	0	nan	0
-snan	inf	inf	nan

版权所有 © 华为技术有限公司

输入值实部（x.real）	输入值虚部（x.imag）	输出值实部（y.real）	输出值虚部（y.imag）
-snan	-inf	inf	nan
-snan	snan	nan	nan
-snan	-snan	nan	nan
-snan	qnan	nan	nan
-snan	-qnan	nan	nan
qnan	0	nan	0
qnan	0	nan	0
qnan	inf	inf	nan
qnan	-inf	inf	nan
qnan	snan	nan	nan
qnan	-snan	nan	nan
qnan	qnan	nan	nan
qnan	-qnan	nan	nan
-qnan	0	nan	0
-qnan	0	nan	0
-qnan	inf	inf	nan
-qnan	-inf	inf	nan
-qnan	snan	nan	nan
-qnan	-snan	nan	nan
-qnan	qnan	nan	nan
-qnan	-qnan	nan	nan

依赖

C: "kc.h"

示例

C interface：

```
// typical usage
double x1 = INFINITY, y1 = INFINITY;
double x2 = 2.0, y2 = 3.0;
double x3 = -2.5, y3 = -3.4;
double x4 = NAN, y4 = NAN;
double x5 = 0, y5 = 0;
// print result
```

 版权所有 © 华为技术有限公司

```
printf("/*\n");
printf(" * casinh(%.2f + %.2f*I) = %.6f\n", x1, y1, casinh(__builtin_complex(x1, y1)));
printf(" * casinh(%.2f + %.2f*I) = %.6f\n", x2, y2, casinh(__builtin_complex(x2, y2)));
printf(" * casinh(%.2f + %.2f*I) = %.6f\n", x3, y3, casinh(__builtin_complex(x3, y3)));
printf(" * casinh(%.2f + %.2f*I) = %.6f\n", x4, y4, casinh(__builtin_complex(x4, y4)));
printf(" * casinh(%.2f + %.2f*I) = %.6f\n", x5, y5, casinh(__builtin_complex(x5, y5)));
printf(" **/\n");
/*
 * casinh(inf + inf*I) = inf
 * casinh(2.00 + 3.00*I) = 1.968638
 * casinh(-2.50 + -3.40*I) = -2.129092
 * casinh(nan + nan*I) = nan
 * casinh(0.00 + 0.00*I) = 0.000000
 **/
```

6.4.3.2.9.20 ctan

计算复数的正切值。

接口定义

C interface：

float complex ctanf(float complex x);

double complex ctan(double complex x);

参数

参数名	类型	描述	输入/输出
x	• 在ctanf中，x是复数单精度浮点类型。	表示输入数据的浮点值。	输入
x	• 在ctan中，x是复数双精度浮点类型。	表示输入数据的浮点值。	输入

返回值

- 正常区间：返回复数x的正切值。
- 特殊区间：如下所示。

输入值实部（x.real）	输入值虚部（x.imag）	输出值实部（y.real）	输出值虚部（y.imag）
-inf	-inf	-0	-1
-inf	< +0	nan	nan
-inf	-0	nan	nan
-inf	+0	nan	nan
-inf	> +0	nan	nan

版权所有 © 华为技术有限公司

输入值实部（x.real）	输入值虚部（x.imag）	输出值实部（y.real）	输出值虚部（y.imag）
-inf	inf	-0	+1
-inf	-nan	nan	nan
-inf	nan	nan	nan
< +0	-nan	nan	nan
< +0	nan	nan	nan
-0	-inf	-0	-1
-0	-0	-0	-0
-0	+0	-0	+0
-0	inf	-0	1
-0	-nan	-0	nan
-0	nan	-0	nan
+0	-inf	0	-1
+0	-0	0	-0
+0	+0	0	+0
+0	inf	0	1
+0	-nan	0	nan
+0	nan	0	nan
> +0	-inf	0	-1
> +0	inf	0	1
> +0	-nan	nan	nan
> +0	nan	nan	nan
inf	-inf	0	-1
inf	< +0	nan	nan
inf	-0	nan	nan
inf	+0	nan	nan
inf	> +0	nan	nan
inf	inf	0	1
inf	-nan	nan	nan
inf	nan	nan	nan

版权所有 © 华为技术有限公司

输入值实部（x.real）	输入值虚部（x.imag）	输出值实部（y.real）	输出值虚部（y.imag）
-nan	-inf	0	-1
-nan	< +0	nan	nan
-nan	-0	nan	nan
-nan	+0	nan	nan
-nan	> +0	nan	nan
-nan	inf	0	1
-nan	-nan	nan	nan
-nan	nan	nan	nan
nan	-inf	0	-1
nan	< +0	nan	nan
nan	-0	nan	nan
nan	+0	nan	nan
nan	> +0	nan	nan
nan	inf	0	1
nan	-nan	nan	nan
nan	nan	nan	nan

依赖

C: "km.h"

示例

C interface：

```
// typical usage
double x1 = 1.0, x2 = 2.0, x3 = 3.0, x4 = -4.0;
double complex cx1 = __builtin_complex(x1, x2);
double complex cx2 = __builtin_complex(x3, x4);
double complex cx3 = __builtin_complex(INFINITY, -INFINITY);

// print result
double ry;
double iy;
ry = __real__ ctan(cx1);
iy = __imag__ ctan(cx1);
printf("ctan(cx1) = %.15f, %.15f\n", ry, iy);
ry = __real__ ctan(cx2);
iy = __imag__ ctan(cx2);
printf("ctan(cx2) = %.15f, %.15f\n", ry, iy);
ry = __real__ ctan(cx3);
iy = __imag__ ctan(cx3);
printf("ctan(cx3) = %.15f, %.15f\n", ry, iy);
```

 版权所有 © 华为技术有限公司

```
result
    /*
     * ctan(cx1) = 0.033812826079897, 1.014793616146634
     * ctan(cx2) = -0.000187346204629, -0.999355987381473
     * ctan(cx3) = 0.000000000000000, -1.000000000000000
     * */
```

6.4.3.2.9.21 ctanh

计算复数的双曲正切值。

接口定义

C interface：

float complex ctanhf(float complex x);

double complex ctanh(double complex x);

参数

参数名	类型	描述	输入/输出
x	• 在ctanhf中，x是复数单精度浮点类型。	表示输入数据的浮点值。	输入
x	• 在ctanh中，x是复数双精度浮点类型。	表示输入数据的浮点值。	输入

返回值

- 返回x的双曲正切结果y。
- 特殊值说明如下所示。

输入值实部（x.real）	输入值虚部（x.imag）	输出值实部（y.real）	输出值虚部（y.imag）
0.0	0.0	0.0	0.0
0.0	-0.0	0.0	-0.0
0.0	inf	0.0	nan
0.0	-inf	0.0	nan
0.0	nan	0.0	nan
0.0	-nan	0.0	nan
-0.0	0.0	-0.0	0.0
-0.0	-0.0	-0.0	-0.0

版权所有 © 华为技术有限公司

输入值实部（x.real）	输入值虚部（x.imag）	输出值实部（y.real）	输出值虚部（y.imag）
-0.0	inf	-0.0	nan
-0.0	-inf	-0.0	nan
-0.0	nan	-0.0	nan
-0.0	-nan	-0.0	nan
inf	0.0	1.0	0.0
inf	-0.0	1.0	-0.0
inf	inf	1.0	0.0
inf	-inf	1.0	-0.0
inf	nan	1.0	0.0
inf	-nan	1.0	-0.0
-inf	0.0	-1.0	0.0
-inf	-0.0	-1.0	-0.0
-inf	inf	-1.0	0.0
-inf	-inf	-1.0	-0.0
-inf	nan	-1.0	0.0
-inf	-nan	-1.0	-0.0
nan	0.0	nan	0.0
nan	-0.0	nan	-0.0
nan	inf	nan	nan
nan	-inf	nan	nan
nan	nan	nan	nan
nan	-nan	nan	nan
-nan	0.0	-nan	0.0
-nan	-0.0	-nan	-0.0
-nan	inf	nan	nan
-nan	-inf	nan	nan
-nan	nan	nan	nan
-nan	-nan	nan	nan

版权所有 © 华为技术有限公司

依赖

C: "kc.h"

示例

C interface：

```
// typical usage
double x1 = INFINITY, y1 = INFINITY;
double x2 = 2.0, y2 = 3.0;
double x3 = -2.5, y3 = -3.4;
double x4 = NAN, y4 = NAN;
double x5 = 0, y5 = 0;
double complex z1 = ctanh(__builtin_complex(x1, y1));
double complex z2 = ctanh(__builtin_complex(x2, y2));
double complex z3 = ctanh(__builtin_complex(x3, y3));
double complex z4 = ctanh(__builtin_complex(x4, y4));
double complex z5 = ctanh(__builtin_complex(x5, y5));
// print result
printf("/*\n");
printf(" * ctanh(%.2f + %.2f*I) = %.6f + %.6f*I\n", x1, y1, __real__(z1), __imag__(z1));
printf(" * ctanh(%.2f + %.2f*I) = %.6f + %.6f*I\n", x2, y2, __real__(z2), __imag__(z2));
printf(" * ctanh(%.2f + %.2f*I) = %.6f + %.6f*I\n", x3, y3, __real__(z3), __imag__(z3));
printf(" * ctanh(%.2f + %.2f*I) = %.6f + %.6f*I\n", x4, y4, __real__(z4), __imag__(z4));
printf(" * ctanh(%.2f + %.2f*I) = %.6f + %.6f*I\n", x5, y5, __real__(z5), __imag__(z5));
printf(" **/\n");

/*
 * ctanh(inf + inf*I) = 1.000000 + 0.000000*I
 * ctanh(2.00 + 3.00*I) = 0.965386 + -0.009884*I
 * ctanh(-2.50 + -3.40*I) = -0.988331 + -0.006581*I
 * ctanh(nan + nan*I) = nan + nan*I
 * ctanh(0.00 + 0.00*I) = 0.000000 + 0.000000*I
 **/
```

6.4.3.2.9.22 catan

计算复数x的反正切值。

接口定义

C interface：

float complex catanf(float complex x);

double complex catan(double complex x);

long double complex catanl(long double complex x);

参数

参数名	类型	描述	输入/输出
x	• 在catanf中，x是复数单精度浮点类型。 • 在catan中，x是复数双精度浮点类型。 • 在catanl中，x是复数长双精度浮点类型。	表示输入数据的浮点值。	输入

版权所有 © 华为技术有限公司

返回值

- 返回x的反正切结果y。
- 特殊值说明如下所示。

输入值实部（x.real）	输入值虚部（x.imag）	输出值实部（y.real）	输出值虚部（y.imag）
-inf	-inf	-pi/2	-0.0
-inf	< +0.0	-pi/2	-0.0
-inf	-0.0	-pi/2	-0.0
-inf	+0.0	-pi/2	+0.0
-inf	> +0.0	-pi/2	+0.0
-inf	inf	-pi/2	+0.0
-inf	-nan	-pi/2	+0.0
-inf	nan	-pi/2	+0.0
< +0.0	-inf	-pi/2	-0.0
< +0.0	inf	-pi/2	+0.0
< +0.0	-nan	nan	nan
< +0.0	nan	nan	nan
-0.0	-inf	-pi/2	-0.0
-0.0	-0.0	-0.0	-0.0
-0.0	+0.0	-0.0	0.0
-0.0	inf	-pi/2	+0.0
-0.0	-nan	nan	nan
-0.0	nan	nan	nan
+0.0	-inf	pi/2	-0.0
+0.0	-0.0	+0.0	-0.0
+0.0	+0.0	+0.0	+0.0
+0.0	inf	pi/2	+0.0
+0.0	-nan	nan	nan
+0.0	nan	nan	nan
> +0.0	-inf	pi/2	-0.0
> +0.0	inf	pi/2	+0.0
> +0.0	-nan	nan	nan

版权所有 © 华为技术有限公司

输入值实部（x.real）	输入值虚部（x.imag）	输出值实部（y.real）	输出值虚部（y.imag）
> +0.0	nan	nan	nan
inf	-inf	pi/2	-0.0
inf	< +0.0	pi/2	-0.0
inf	-0.0	pi/2	-0.0
inf	+0.0	pi/2	+0.0
inf	> +0.0	pi/2	+0.0
inf	inf	pi/2	+0.0
inf	-nan	pi/2	+0.0
inf	nan	pi/2	+0.0
-nan	-inf	nan	-0.0
-nan	< +0.0	nan	nan
-nan	-0.0	nan	-0.0
-nan	+0.0	nan	+0.0
-nan	> +0.0	nan	nan
-nan	inf	nan	+0.0
-nan	-nan	nan	nan
-nan	nan	nan	nan
nan	-inf	nan	-0.0
nan	< +0.0	nan	nan
nan	-0.0	nan	-0.0
nan	+0.0	nan	+0.0
nan	> +0.0	nan	nan
nan	inf	nan	+0.0
nan	-nan	nan	nan
nan	nan	nan	nan

依赖

C: "km.h"

 版权所有 © 华为技术有限公司

示例

C interface：

```
  // typical usage
  double x1 = 1.0, x2 = 2.0, x3 = 3.0, x4 = 4.0;
  double complex cx1 = __builtin_complex(x1, x2);
  double complex cx2 = __builtin_complex(x3, x4);
  double complex cx3 = __builtin_complex(NAN, NAN);
  double complex cx4 = __builtin_complex(INFINITY, -INFINITY);
  double ry;
  double iy;
  ry = __real__ catan(cx1);
  iy = __imag__ catan(cx1);
  printf("catan(cx1) = %.15f, %.15f\n", ry, iy);
  ry = __real__ catan(cx2);
  iy = __imag__ catan(cx2);
  printf("catan(cx2) = %.15f, %.15f\n", ry, iy);
  ry = __real__ catan(cx3);
  iy = __imag__ catan(cx3);
  printf("catan(cx3) = %.15f, %.15f\n", ry, iy);
  ry = __real__ catan(cx4);
  iy = __imag__ catan(cx4);
  printf("catan(cx4) = %.15f, %.15f\n", ry, iy);

result:
  /*
  * catan(cx1) = 1.338972522294493, 0.402359478108525
  * catan(cx2) = 1.448306995231464, 0.158997191679999
  * catan(cx3) = nan, nan
  * catan(cx4) = 1.570796326794897, -0.000000000000000
  */
```

6.4.3.2.9.23 catanh

计算复数的双曲反正切值。

接口定义

C interface：

float complex catanhf(float complex x);

double complex catanh(double complex x);

参数

参数名	类型	描述	输入/输出
x	• 在catanhf中，x是复数单精度浮点类型。 • 在catanh中，x是复数双精度浮点类型。	表示输入数据的浮点值。	输入

返回值

- 返回x的双曲反正切结果y。
- 特殊值说明如下所示。

 版权所有 © 华为技术有限公司

输入值实部（x.real）	输入值虚部（x.imag）	输出值实部（y.real）	输出值虚部（y.imag）
inf	/	+0.0	/
-inf	/	-0.0	/
+0.0	nan	+0.0	nan
nan	±inf	+0.0	/
nan	非±inf	nan	nan

依赖

C: "km.h"

示例

C interface：

```
// typical usage
double x1 = 1.0, x2 = 2.0, x3 = 3.0, x4 = 4.0;
double complex cx1 = __builtin_complex(x1, x2);
double complex cx2 = __builtin_complex(x3, x4);
double complex cx3 = __builtin_complex(NAN, NAN);
double complex cx4 = __builtin_complex(INFINITY, -INFINITY);

// print result
double ry;
double iy;
ry = __real__ catanh(cx1);
iy = __imag__ catanh(cx1);
printf("catanh(cx1) = %.15f, %.15f\n", ry, iy);
ry = __real__ catanh(cx2);
iy = __imag__ catanh(cx2);
printf("catanh(cx2) = %.15f, %.15f\n", ry, iy);
ry = __real__ catanh(cx3);
iy = __imag__ catanh(cx3);
printf("catanh(cx3) = %.15f, %.15f\n", ry, iy);
ry = __real__ catanh(cx4);
iy = __imag__ catanh(cx4);
printf("catanh(cx4) = %.15f, %.15f\n", ry, iy);

result:
/*
 * catanh(cx1) = 0.173286795139986, 1.178097245096172
 * catanh(cx2) = 0.117500907311434, 1.409921049596575
 * catanh(cx3) = nan, nan
 * catanh(cx4) = 0.000000000000000, -1.570796326794897
 */
```

6.4.3.2.9.24 cis

同时计算x的cos和sin值，以复数的形式返回。

接口定义

C interface：

float complex cisf(float x);

 版权所有 © 华为技术有限公司

double complex cis(double x);

参数

参数名	类型	描述	输入/输出
x	• 在cisf中，x是单精度浮点类型。 • 在cis中，x是双精度浮点类型。	表示输入数据的浮点值。	输入

返回值

计算x的cos和sin，并组成复数结果返回y，y.real ∈ [-1, +1]，y.imag∈ [-1, +1]。

- 输入x为+0，返回y.real为+0，y.imag为+1。
- 输入x为-0，返回y.real为-0，y.imag为+1。
- 输入x为±∞，返回y.real为nan，y.imag为nan。
- 输入x为nan，返回y.real为nan，y.imag为nan。

依赖

C: "kc.h"

示例

C interface：

```
// typical usage
double x1 = INFINITY;
double x2 = 2.0;
double x3 = -2.5;
double x4 = NAN;
double x5 = 0.0;
double complex z1 = cis(x1);
double complex z2 = cis(x2);
double complex z3 = cis(x3);
double complex z4 = cis(x4);
double complex z5 = cis(x5);
// print result
printf("/*\n");
printf(" * cis(%.2f) = %.6f + %.6f*I\n", x1, __real__(z1), __imag__(z1));
printf(" * cis(%.2f) = %.6f + %.6f*I\n", x2, __real__(z2), __imag__(z2));
printf(" * cis(%.2f) = %.6f + %.6f*I\n", x3, __real__(z3), __imag__(z3));
printf(" * cis(%.2f) = %.6f + %.6f*I\n", x4, __real__(z4), __imag__(z4));
printf(" * cis(%.2f) = %.6f + %.6f*I\n", x5, __real__(z5), __imag__(z5));
printf(" **/\n");

/*
 * cis(inf) = nan + nan*I
 * cis(2.00) = 0.909297 + -0.416147*I
 * cis(-2.50) = -0.598472 + -0.801144*I
 * cis(nan) = nan + nan*I
 * cis(0.00) = 0.000000 + 1.000000*I
 **/
```

 版权所有 © 华为技术有限公司

6.4.3.2.9.25 cisd

同时计算x的cos和sin值（输入参数为角度），结果以复数的形式返回。

接口定义

C interface：

float complex cisdf(float x);

double complex cisd(double x);

参数

参数名	类型	描述	输入/输出
x	• 在cisdf中，x是单精度浮点类型。 • 在cisd中，x是双精度浮点类型。	表示输入数据的浮点值。	输入

返回值

计算x的cos和sin，并组成复数结果返回y，y.real ∈ [-1, +1]，y.imag∈ [-1, +1]。

- 输入x为+0，返回y.real为+0，y.imag为+1。
- 输入x为-0，返回y.real为-0，y.imag为+1。
- 输入x为±∞，返回y.real为nan，y.imag为nan。
- 输入x为nan，返回y.real为nan，y.imag为nan。

依赖

C: "kc.h"

示例

C interface：

```
// typical usage
double x1 = INFINITY;
double x2 = M_PI;
double x3 = M_PI_2;
double x4 = NAN;
double x5 = M_PI_4;
double complex z1 = cisd(x1);
double complex z2 = cisd(x2);
double complex z3 = cisd(x3);
double complex z4 = cisd(x4);
double complex z5 = cisd(x5);
// print result
printf("/*\n");
printf(" * cisd(%.2f) = %.6f + %.6f*I\n", x1, __real__(z1), __imag__(z1));
printf(" * cisd(%.2f) = %.6f + %.6f*I\n", x2, __real__(z2), __imag__(z2));
printf(" * cisd(%.2f) = %.6f + %.6f*I\n", x3, __real__(z3), __imag__(z3));
printf(" * cisd(%.2f) = %.6f + %.6f*I\n", x4, __real__(z4), __imag__(z4));
printf(" * cisd(%.2f) = %.6f + %.6f*I\n", x5, __real__(z5), __imag__(z5));
```


版权所有 © 华为技术有限公司

```
/*
 * cisd(inf) = nan + nan*I
 * cisd(3.14) = 0.998497 + 0.054804*I
 * cisd(1.57) = 0.999624 + 0.027412*I
 * cisd(nan) = nan + nan*I
 * cisd(0.79) = 0.999906 + 0.013707*I
 **/
```

6.4.3.2.9.26 conj

计算复数的共轭。

接口定义

C interface：

float complex conjf(float complex x);

double complex conj(double complex x);

long double complex conjl(long double complex x);

参数

参数名	类型	描述	输入/输出
x	• 在conjf中，x是复数单精度浮点类型。 • 在conj中，x是复数双精度浮点类型。 • 在conjl中，x是复数长双精度浮点类型。	表示输入数据的浮点值。	输入

返回值

- 返回x的共轭复数y，y.real ∈ (-INF, +INF)，y.imag∈ (-INF, +INF)。
- 输入x，返回y.real=x.real, y.imag=-x.imag。

依赖

C: "kc.h"

示例

C interface：

```
// typical usage
double x1 = INFINITY, y1 = INFINITY;
double x2 = 2.0, y2 = 3.0;
double x3 = -2.5, y3 = -3.4;
double x4 = NAN, y4 = NAN;
double x5 = 0, y5 = 0;
double complex z1 = conj(__builtin_complex(x1, y1));
double complex z2 = conj(__builtin_complex(x2, y2));
double complex z3 = conj(__builtin_complex(x3, y3));
double complex z4 = conj(__builtin_complex(x4, y4));
```

 版权所有 © 华为技术有限公司

```
double complex z5 = conj(__builtin_complex(x5, y5));
// print result
printf("/*\n");
printf(" * conj(%.2f + %.2f*I) = %.6f + %.6f*I\n", x1, y1, __real__(z1), __imag__(z1));
printf(" * conj(%.2f + %.2f*I) = %.6f + %.6f*I\n", x2, y2, __real__(z2), __imag__(z2));
printf(" * conj(%.2f + %.2f*I) = %.6f + %.6f*I\n", x3, y3, __real__(z3), __imag__(z3));
printf(" * conj(%.2f + %.2f*I) = %.6f + %.6f*I\n", x4, y4, __real__(z4), __imag__(z4));
printf(" * conj(%.2f + %.2f*I) = %.6f + %.6f*I\n", x5, y5, __real__(z5), __imag__(z5));
printf(" **/\n");

/*
 * conj(inf + inf*I) = inf + -inf*I
 * conj(2.00 + 3.00*I) = 2.000000 + -3.000000*I
 * conj(-2.50 + -3.40*I) = -2.500000 + 3.400000*I
 * conj(nan + nan*I) = nan + -nan*I
 * conj(0.00 + 0.00*I) = 0.000000 + -0.000000*I
 **/
```

6.4.3.2.9.27 cproj

计算复数在黎曼球面的投影。

接口定义

C interface：

float complex cprojf(float complex x);

double complex cproj(double complex x);

long double complex cprojl(long double complex x);

参数

参数名	类型	描述	输入/输出
x	• 在cprojf中，x是复数单精度浮点类型。 • 在cproj中，x是复数双精度浮点类型。 • 在cprojl中，x是复数长双精度浮点类型。	表示输入数据的浮点值。	输入

返回值

- 返回x在黎曼球面的投影y。
- 输入x.real或者x.imag为±∞，返回y.real为+∞，y.imag为0（y.imag符号和x.imag一致）。
- 输入x.real和x.imag均不为±∞，y=x。

依赖

C: "kc.h"

版权所有 © 华为技术有限公司

示例

C interface：

```
// typical usage
double x1 = INFINITY, y1 = INFINITY;
double x2 = 2.0, y2 = 3.0;
double x3 = -2.5, y3 = -3.4;
double x4 = NAN, y4 = NAN;
double x5 = 0, y5 = 0;
double complex z1 = cproj(__builtin_complex(x1, y1));
double complex z2 = cproj(__builtin_complex(x2, y2));
double complex z3 = cproj(__builtin_complex(x3, y3));
double complex z4 = cproj(__builtin_complex(x4, y4));
double complex z5 = cproj(__builtin_complex(x5, y5));
// print result
printf("/*\n");
printf(" * cproj(%.2f + %.2f*I) = %.6f + %.6f*I\n", x1, y1, __real__(z1), __imag__(z1));
printf(" * cproj(%.2f + %.2f*I) = %.6f + %.6f*I\n", x2, y2, __real__(z2), __imag__(z2));
printf(" * cproj(%.2f + %.2f*I) = %.6f + %.6f*I\n", x3, y3, __real__(z3), __imag__(z3));
printf(" * cproj(%.2f + %.2f*I) = %.6f + %.6f*I\n", x4, y4, __real__(z4), __imag__(z4));
printf(" * cproj(%.2f + %.2f*I) = %.6f + %.6f*I\n", x5, y5, __real__(z5), __imag__(z5));
printf(" **/\n");

/*
 * cproj(inf + inf*I) = inf + 0.000000*I
 * cproj(2.00 + 3.00*I) = 2.000000 + 3.000000*I
 * cproj(-2.50 + -3.40*I) = -2.500000 + -3.400000*I
 * cproj(nan + nan*I) = nan + nan*I
 * cproj(0.00 + 0.00*I) = 0.000000 + 0.000000*I
 **/
```

6.4.3.2.9.28 creal

返回复数的实数部分。

接口定义

C interface：

float crealf(float complex x);

double creal(double complex x);

long double creall(long double complex x);

参数

参数名	类型	描述	输入/输出
x	• 在crealf中，x是复数单精度浮点类型。 • 在creal中，x是复数双精度浮点类型。 • 在creall中，x是复数长双精度浮点类型。	表示输入数据的浮点值。	输入

返回值

返回x的实部的值y，y ∈ (-INF, +INF)。

 版权所有 © 华为技术有限公司

依赖

C: "km.h"

示例

C interface：

```
// typical usage
double x1 = INFINITY, y1 = INFINITY;
double x2 = 2.0, y2 = 3.0;
double x3 = -2.5, y3 = -3.4;
double x4 = NAN, y4 = NAN;
double x5 = 0, y5 = 0;
// print result
printf("/*\n");
printf(" * creal(%.2f + %.2f*I) = %.6f\n", x1, y1, creal(__builtin_complex(x1, y1)));
printf(" * creal(%.2f + %.2f*I) = %.6f\n", x2, y2, creal(__builtin_complex(x2, y2)));
printf(" * creal(%.2f + %.2f*I) = %.6f\n", x3, y3, creal(__builtin_complex(x3, y3)));
printf(" * creal(%.2f + %.2f*I) = %.6f\n", x4, y4, creal(__builtin_complex(x4, y4)));
printf(" * creal(%.2f + %.2f*I) = %.6f\n", x5, y5, creal(__builtin_complex(x5, y5)));
printf(" **/\n");

/*
 * creal(inf + inf*I) = inf
 * creal(2.00 + 3.00*I) = 2.000000
 * creal(-2.50 + -3.40*I) = -2.500000
 * creal(nan + nan*I) = nan
 * creal(0.00 + 0.00*I) = 0.000000
 **/
```

6.4.3.2.9.29 cimag

返回复数的虚部的值。

接口定义

C interface：

float cimagf(float complex x);

double cimag(double complex x);

long double cimagl(long double complex x);

参数

参数名	类型	描述	输入/输出
x	• 在cimagf中，x是复数单精度浮点类型。 • 在cimag中，x是复数双精度浮点类型。 • 在cimagl中，x是复数长双精度浮点类型。	表示输入数据的浮点值。	输入

返回值

返回x的虚部的值y，y ∈ (-INF, +INF)。

 版权所有 © 华为技术有限公司

依赖

C: "km.h"

示例

C interface：

```
// typical usage
double x1 = INFINITY, y1 = INFINITY;
double x2 = 2.0, y2 = 3.0;
double x3 = -2.5, y3 = -3.4;
double x4 = NAN, y4 = NAN;
double x5 = 0, y5 = 0;
// print result
printf("/*\n");
printf(" * cimag(%.2f + %.2f*I) = %.6f\n", x1, y1, cimag(__builtin_complex(x1, y1)));
printf(" * cimag(%.2f + %.2f*I) = %.6f\n", x2, y2, cimag(__builtin_complex(x2, y2)));
printf(" * cimag(%.2f + %.2f*I) = %.6f\n", x3, y3, cimag(__builtin_complex(x3, y3)));
printf(" * cimag(%.2f + %.2f*I) = %.6f\n", x4, y4, cimag(__builtin_complex(x4, y4)));
printf(" * cimag(%.2f + %.2f*I) = %.6f\n", x5, y5, cimag(__builtin_complex(x5, y5)));
printf(" **/\n");

/*
 * cimag(inf + inf*I) = inf
 * cimag(2.00 + 3.00*I) = 3.000000
 * cimag(-2.50 + -3.40*I) = -3.400000
 * cimag(nan + nan*I) = nan
 * cimag(0.00 + 0.00*I) = 0.000000
 **/
```

6.4.3.2.10 Support 函数

6.4.3.2.10.1 KMGetVersion

获取MATH产品版本信息：

int KMGetVersion(KMVersion *ver)

参数

参数名	描述	取值范围	输入/输出
ver	产品信息结构体。	非空	输出

返回值

- 成功：返回1。
- 失败：返回0。

示例

```
#include <stdio.h>
#include "km.h"

int main()
{
    KMVersion mver;
```

 版权所有 © 华为技术有限公司

```
    int status = KMGetVersion(&mver);
    if (status == 0) {
        return 0;
    }
    printf("Product Name: %s\n", mver.productName);
    printf("Product Version: %s\n", mver.productVersion);
    printf("Component Name: %s\n", mver.componentName);
    printf("Component Version: %s\n", mver.componentVersion);
    printf("Component AbiVersion: %s\n", mver.componentAbiVersion);
    printf("Component AppendInfo: %s\n", mver.componentAppendInfo);
    printf("Software Name: %s\n", mver.softwareName);
    printf("Software Version: %s\n", mver.softwareVersion);
    return 0;
}
```

运行结果：

```
Product Name: Kunpeng HPCKit
Product Version: 24.0.RC1
Component Name: HPCKit-kml
Component Version: 2.3.0
Component AbiVersion: 1
Component AppendInfo: gcc
Software Name: hpckit-kml-math
Software Version: 2.3.0
```

说明

以上版本号和编译时间以实际运行结果为准，上述结果仅供参考。

6.4.4 KML_FFT 库函数说明

6.4.4.1 函数说明

快速傅里叶（Fast Fourier Transform）运算库，是快速计算序列的离散傅里叶变换（DFT）或其逆变换的方法，广泛的应用于工程、科学和数学领域，将傅里叶变换计算需要的复杂度从O(n2)降到了O(n log n)，被IEEE科学与工程计算期刊列入20世纪十大算法。

FFT（Fast Fourier Transform）库包含一系列快速傅里叶变换函数的接口标准，KML_FFT库包含以下六大类函数：

- plan函数：用于建立FFT变换的plan，plan使用完毕后需要通过调用 **kml_fft(f/h)_destroy_plan**函数来释放。
  - **6.4.4.5.1 C2C变换**：复数形式到复数形式的FFT变换
  - **6.4.4.5.2 R2C变换**：实数形式到复数形式的FFT变换
  - **6.4.4.5.3 C2R变换**：复数形式到实数形式的FFT变换
  - **6.4.4.5.4 R2R变换**：实数形式到实数形式的FFT变换
- 变换执行函数：用于根据plan执行对应的FFT变换。
- 内存函数：内存分配、释放函数。
- 线程函数：线程初始化、资源释放相关函数。
- MPI Local Size函数：在进程间划分数据。
- MPI transposition函数：转置变换。

对于上述变换，KML_FFT库均支持：

- in-place变换（输入in与输出out是同一个指针），以及out-place变换。

版权所有 © 华为技术有限公司

- 单精度以及双精度：单精度函数以kml_fftf_为函数前缀，双精度函数以kml_fft_为函数前缀。其中，6.4.4.4 单进程函数定义和6.4.4.5 多进程函数定义的函数说明以双精度为例进行说明。
- planning选项：对于plan函数，参数flags支持选择ESTIMATE模式或PATIENT模式
  - ESTIMATE模式：不对不同算法进行实际计算评估，快速选择一个变换方案，可能不是最优的。
  - PATIENT模式：实际评估计算多个变换方案，并测量其执行时间，期望得到一个更优的变换方案，代价是plan函数耗时会增加。

说明

以下接口仅适用于Taishan 200 1280 V2和Taishan 200 2280 V2服务器。

- C Interface以“kml_ffth”开头的接口
- Fortran Interface以“RES = KML_FFTH”的接口

6.4.4.2 数据结构

提供KML_FFT库函数数据结构定义说明。

复数的结构体定义：

- 双精度复数：
```
typedef double kml_fft_complex[2];
```
- 单精度复数：
```
typedef float kml_fftf_complex[2];
```

kml_fft_plan_s结构体及其指针kml_fft_plan的定义：

- 双精度：
```
struct kml_fft_plan_s {
    problem_s problem;
    solver_s solver;
    KML_UINT num_threads;
};
typedef struct kml_fft_plan_s *kml_fft_plan;
```
- 单精度：
```
struct kml_fftf_plan_s {
    problem_s problem;
    solver_s solver;
    KML_UINT num_threads;
};
typedef struct kml_fftf_plan_s *kml_fftf_plan;
```

说明

- problem_s为待变换FFT问题的结构体，包含转换类型，输入输出等参数。
- solver_s为FFT选用求解器的参数，包含求解所用函数，转换为子问题的kml_fft_plan_s等参数。
- num_threads为线程数。

半复数形式：

n0xn1数组A，若满足$A(i,j)=\overline{A(n0-i,n1-j)}$，其中A(n0, j)为A(0, j)，A(i, n1)为A(i, 0)，则采用半复数形式n0x[(n1/2)+1]数组B来保存数组A。

版权所有 © 华为技术有限公司

$$A=\begin{bmatrix} 120 & 8+8i & 0 & 8-8i \\ 16i & 16i & -8+8i & -40-8i \\ -8 & -8+8i & -16 & -8+8i \\ -16i & -40+8i & -8-8i & -16i \end{bmatrix}$$

$$B=\begin{bmatrix} 120 & 8+8i & 0 \\ 16i & 16i & -8+8i \\ -8 & -8+8i & -16 \\ -16i & -40+8i & -8-8i \end{bmatrix}$$

6.4.4.3 环境变量

以下参数会被KML_FFT使用。

环境变量名	变量意义	有效值	默认值
KML_FFT_AFFINITY	多线程核亲和性策略	SPREAD\CLOSE\NO	若不设置，将根据输入规模及线程数自动判断
KML_FFT_THREAD_TYPE	多线程管理后端，用于用户指定后端类型，不支持线程嵌套场景	PTHREAD\OMP\KUPL	PTHREAD

6.4.4.4 单进程函数定义

6.4.4.4.1 C2C 变换

6.4.4.4.1.1 kml_fft(f/h)_plan_dft

建立单个连续数据序列n维C2C变换的plan。

接口定义

C interface：

kml_fft_plan kml_fft_plan_dft(int rank, const int *n, kml_fft_complex *in, kml_fft_complex *out, int sign, unsigned flags);

kml_fftf_plan kml_fftf_plan_dft(int rank, const int *n, kml_fftf_complex *in, kml_fftf_complex *out, int sign, unsigned flags);

kml_ffth_plan kml_ffth_plan_dft(int rank, const int *n, kml_ffth_complex *in, kml_ffth_complex *out, int sign, unsigned flags);

Fortran interface：

RES = KML_FFT_PLAN_DFT(RANK, N, IN, OUT, SIGN, FLAGS);

 版权所有 © 华为技术有限公司

RES = KML_FFTF_PLAN_DFT(RANK, N, IN, OUT, SIGN, FLAGS);

RES = KML_FFTH_PLAN_DFT(RANK, N, IN, OUT, SIGN, FLAGS);

返回值

函数返回一个kml_fft(f/h)_plan类型的结构体指针。将该对象作为参数传入**kml_fft(f/h)_execute**函数中，将对当前提供的输入in和输出out执行FFT变换。也可以通过将该对象作为参数传入 **kml_fft(f/h)_execute_dft**函数中以对新的输入in和输出out执行FFT变换。

如果函数返回非空指针，则表示plan创建成功，否则表示创建失败。

参数

参数名	数据类型	描述	输入/输出
rank	int	FFT变换的维度是rank，约束：1≤rank≤4。	输入
n	const int*	n是一个维度为rank的数组，包含FFT序列每一维度的大小，约束：n[i]≥1, for i in 0 to rank - 1。	输入
in	• 双精度：kml_fft_complex* • 单精度：kml_fftf_complex* • 半精度：kml_ffth_complex*	输入需要变换的数据。	输入
out	• 双精度：kml_fft_complex* • 单精度：kml_fftf_complex* • 半精度：kml_ffth_complex*	输出快速傅里叶变换后的数据。	输出
sign	int	描述正向变换或反向变换。 • -1(KML_FFT_FORWARD)：forward • +1(KML_FFT_BACKWARD)：backward	输入

 版权所有 © 华为技术有限公司

参数名	数据类型	描述	输入/输出
flags	unsigned int	planning选项，可选择ESTIMATE模式或PATIENT模式。 KML_FFT_ESTIMATE：ESTIMATE模式 KML_FFT_PATIENT：PATIENT模式	输入

依赖

C: "kfft.h"

示例

C interface：

```
int rank = 2;
int *n;
n = (int*)kml_fft_malloc(sizeof(int) * rank);
n[0] = 2;
n[1] = 3;
double init[6][2] = {{120, 0}, {8, 8}, {0, 0}, {0, 16}, {0, 16}, {-8, 8}};
kml_fft_complex *in;
in = (kml_fft_complex*)kml_fft_malloc(sizeof(kml_fft_complex) * n[0] * n[1]);
for (int i = 0; i < n[0] * n[1]; i++) {
    in[i][0] = init[i][0];
    in[i][1] = init[i][1];
}
kml_fft_complex *out;
out = (kml_fft_complex*)kml_fft_malloc(sizeof(kml_fft_complex) * n[0] * n[1]);
kml_fft_plan plan;
plan = kml_fft_plan_dft(rank, n, in, out, KML_FFT_FORWARD, KML_FFT_ESTIMATE);
kml_fft_execute_dft(plan, in, out);

kml_fft_destroy_plan(plan);
kml_fft_free(n);
kml_fft_free(in);
kml_fft_free(out);

/*
 * out = {{1.200000e+02, 4.800000e+01}, {1.338564e+02, -1.385641e+01},
 *        {1.061436e+02, 1.385641e+01}, {1.360000e+02, -3.200000e+01},
 *        {1.120000e+02, -8.000000e+00}, {1.120000e+02, -8.000000e+00}}
 */
```

Fortran interface：

```
INTEGER(C_INT) :: RANK = 2
INTEGER(C_INT), DIMENSION(*) :: N
N = KML_FFT_MALLOC(4 * RANK)
N(1) = 2
N(2) = 3
REAL(C_DOUBLE) :: INIT(6, 2)
TYPE(C_PTR) :: PLAN
TYPE(KML_FFT_COMPLEX), POINTER :: IN, OUT
TYPE(C_PTR) :: PIN, POUT
TYPE(C_SIZE_T) :: SIZE
SIZE = 16 * N(1) * N(2)
IN = KML_FFT_MALLOC(SIZE)
OUT = KML_FFT_MALLOC(SIZE)
CALL C_F_POINTER(PIN, IN, SHAPE=[N(1) * N(2)])
```


版权所有 © 华为技术有限公司

```
CALL C_F_POINTER(POUT, OUT, SHAPE=[N(1) * N(2)])
DATA INIT/120, 8, 0, 0, 0, -8, 0, 8, 0, 16, 16, 8/
INTEGER :: I = 1
DO WHILE(I <= N(1) * N(2))
   IN(I)%R = INIT(I, 1)
   IN(I)%I = INIT(I, 2)
   I = I + 1
END DO
PLAN = KML_FFT_PLAN_DFT(RANK, N, IN, OUT, KML_FFT_FORWARD, KML_FFT_ESTIMATE)
CALL KML_FFT_EXECUTE_DFT(PLAN, IN, OUT)

CALL KML_FFT_DESTROY_PLAN(PLAN)
CALL KML_FFT_FREE(N)
CALL KML_FFT_FREE(PIN)
CALL KML_FFT_FREE(POUT)

!
! OUT = /120.00000000000000D0, 133.85640646055103D0,
!       106.14359353944897D0, 136.00000000000000D0,
!       112.00000000000000D0, 112.00000000000000D0,
!       48.000000000000000D0, -13.856406460551021D0,
!       13.856406460551021D0, -32.000000000000000D0,
!       -8.0000000000000000D0, -8.0000000000000000D0/
!
```

6.4.4.4.1.2 kml_fft(f/h)_plan_dft_1d

建立单个连续数据序列1维C2C变换的plan。

接口定义

C interface:

kml_fft_plan kml_fft_plan_dft_1d(int n, kml_fft_complex *in, kml_fft_complex *out, int sign, unsigned flags);

kml_fftf_plan kml_fftf_plan_dft_1d(int n, kml_fftf_complex *in, kml_fftf_complex *out, int sign, unsigned flags);

kml_ffth_plan kml_ffth_plan_dft_1d(int n, kml_ffth_complex *in, kml_ffth_complex *out, int sign, unsigned flags);

Fortran interface:

RES = KML_FFT_PLAN_DFT_1D(N, IN, OUT, SIGN, FLAGS);

RES = KML_FFTF_PLAN_DFT_1D(N, IN, OUT, SIGN, FLAGS);

RES = KML_FFTH_PLAN_DFT_1D(N, IN, OUT, SIGN, FLAGS);

返回值

函数返回一个kml_fft(f/h)_plan类型的结构体指针。将该对象作为参数传入**kml_fft(f/h)_execute**函数中，将对当前提供的输入in和输出out执行FFT变换。也可以通过将该对象作为参数传入 **kml_fft(f/h)_execute_dft**函数中以对新的输入in和输出out执行FFT变换。

如果函数返回非空指针，则表示plan创建成功，否则表示创建失败。

 版权所有 © 华为技术有限公司

参数

参数名	数据类型	描述	输入/输出
n	int	输入FFT序列的大小，约束：n≥1。	输入
in	• 双精度：kml_fft_complex* • 单精度：kml_fftf_complex* • 半精度：kml_ffth_complex*	输入需要变换的数据。	输入
out	• 双精度：kml_fft_complex* • 单精度：kml_fftf_complex* • 半精度：kml_ffth_complex*	输出快速傅里叶变换后的数据。	输出
sign	int	描述正向变换或反向变换。 • -1(KML_FFT_FORWARD)：forward • +1(KML_FFT_BACKWARD)：backward	输入
flags	unsigned int	planning选项，描述ESTIMATE模式或PATIENT模式。 KML_FFT_ESTIMATE：ESTIMATE模式 KML_FFT_PATIENT：PATIENT模式	输入

依赖

C: "kfft.h"

示例

C interface：

```
int N = 16;
kml_fft_complex *in;
kml_fft_complex *out;
kml_fft_plan plan;
```

 版权所有 © 华为技术有限公司

```
in = (kml_fft_complex*)kml_fft_malloc(sizeof(kml_fft_complex) * N);
out = (kml_fft_complex*)kml_fft_malloc(sizeof(kml_fft_complex) * N);
for (int i = 0; i < N; i++){
    in[i][0] = (double)i;
    in[i][1] = (double)i;
}
plan = kml_fft_plan_dft_1d(N, in , out, KML_FFT_FORWARD, KML_FFT_ESTIMATE);
kml_fft_execute_dft(plan, in, out);

kml_fft_destroy_plan(plan);
kml_fft_free(in);
kml_fft_free(out);

/*
 * out = {{1.200000e+02, 1.200000e+02}, {-4.821872e+01, 3.221872e+01},
 *        {-2.731371e+01, 1.131371e+01}, {-1.997285e+01, 3.972846e+00},
 *        {-1.600000e+01, 0.000000e+00}, {-1.334543e+01, -2.654571e+00},
 *        {-1.131371e+01, -4.686292e+00}, {-9.591299e+00, -6.408701e+00},
 *        {-8.000000e+00, -8.000000e+00}, {-6.408701e+00, -9.591299e+00},
 *        {-4.686292e+00, -1.131371e+01}, {-2.654571e+00, -1.334543e+01},
 *        {0.000000e+00, -1.600000e+01}, {3.972846e+00, -1.997285e+01},
 *        {1.131371e+01, -2.731371e+01}, {3.221872e+01, -4.821872e+01}}
 */
```

Fortran interface:

```
INTEGER(C_INT) :: N = 16
TYPE(KML_FFT_COMPLEX), POINTER :: IN, OUT
TYPE(C_PTR) :: PIN, POUT
TYPE(C_PTR) :: PLAN
INTEGER(C_SIZE_T) :: SIZE
SIZE = 16 * N
PIN = KML_FFT_MALLOC(SIZE)
POUT = KML_FFT_MALLOC(SIZE)
CALL C_F_POINTER(PIN, IN, SHAPE=[N])
CALL C_F_POINTER(POUT, OUT, SHAPE=[N])
INTEGER :: I = 1
DO WHILE(I <= N)
   IN%R = I
   IN%I = I
END DO
PLAN = KML_FFT_PLAN_DFT_1D(N, IN , OUT, KML_FFT_FORWARD, KML_FFT_ESTIMATE)
CALL KML_FFT_EXECUTE_DFT(PLAN, IN, OUT)

CALL KML_FFT_DESTROY_PLAN(PLAN)
CALL KML_FFT_FREE(PIN)
CALL KML_FFT_FREE(POUT)

!
! OUT = \1.200000E+02, 1.200000E+0, -4.821872E+01, 3.221872E+01,
!       -2.731371E+01, 1.131371E+01, -1.997285E+01, 3.972846E+00,
!       -1.600000E+01, 0.000000E+00, -1.334543E+01, -2.654571E+00,
!       -1.131371E+01, -4.686292E+00, -9.591299E+00, -6.408701E+00,
!       -8.000000E+00, -8.000000E+00, -6.408701E+00, -9.591299E+00,
!       -4.686292E+00, -1.131371E+01, -2.654571E+00, -1.334543E+01,
!       0.000000E+00, -1.600000E+01, 3.972846E+00, -1.997285E+01,
!       1.131371E+01, -2.731371E+01, 3.221872E+01, -4.821872E+01/
!
```

6.4.4.4.1.3 kml_fft(f/h)_plan_dft_2d

建立单个连续数据序列2维C2C变换的plan。

接口定义

C interface:

 版权所有 © 华为技术有限公司

kml_fft_plan kml_fft_plan_dft_2d(int n0, int n1, kml_fft_complex *in, kml_fft_complex *out, int sign, unsigned flags);

kml_fftf_plan kml_fftf_plan_dft_2d(int n0, int n1, kml_fftf_complex *in, kml_fftf_complex *out, int sign, unsigned flags);

kml_ffth_plan kml_ffth_plan_dft_2d(int n0, int n1, kml_ffth_complex *in, kml_ffth_complex *out, int sign, unsigned flags);

Fortran interface:

RES = KML_FFT_PLAN_DFT_2D(N0, N1, IN, OUT, SIGN, FLAGS);

RES = KML_FFTF_PLAN_DFT_2D(N0, N1, IN, OUT, SIGN, FLAGS);

RES = KML_FFTH_PLAN_DFT_2D(N0, N1, IN, OUT, SIGN, FLAGS);

返回值

函数返回一个kml_fft(f/h)_plan类型的结构体指针。将该对象作为参数传入**kml_fft(f/h)_execute**函数中，将对当前提供的输入in和输出out执行FFT变换。也可以通过将该对象作为参数传入 **kml_fft(f/h)_execute_dft**函数中以对新的输入in和输出out执行FFT变换。

如果函数返回非空指针，则表示plan创建成功，否则表示创建失败。

参数

参数名	数据类型	描述	输入/输出
n0	int	输入FFT序列第一维的大小，约束：n0≥1。	输入
n1	int	输入FFT序列第二维的大小，约束：n1≥1。	输入
in	• 双精度：kml_fft_complex* • 单精度：kml_fftf_complex* • 半精度：kml_ffth_complex*	输入需要变换的数据。	输入

 版权所有 © 华为技术有限公司

参数名	数据类型	描述	输入/输出
out	• 双精度：kml_fft_complex* • 单精度：kml_fftf_complex* • 半精度：kml_ffth_complex*	输出快速傅里叶变换后的数据。	输出
sign	int	描述正向变换或反向变换。 • -1(KML_FFT_FORWARD)：forward • +1(KML_FFT_BACKWARD)：backward	输入
flags	unsigned int	planning选项，描述ESTIMATE模式或PATIENT模式。 KML_FFT_ESTIMATE：ESTIMATE模式 KML_FFT_PATIENT：PATIENT模式	输入

依赖

C: "kfft.h"

示例

C interface：

```
int n0 = 2;
int n1 = 3;
double init[6][2] = {{120, 0}, {8, 8}, {0, 0}, {0, 16}, {0, 16}, {-8, 8}};
kml_fft_complex *in;
in = (kml_fft_complex*)kml_fft_malloc(sizeof(kml_fft_complex) * n0 * n1);
for (int i = 0; i < n0 * n1; i++) {
    in[i][0] = init[i][0];
    in[i][1] = init[i][1];
}
kml_fft_complex *out;
out = (kml_fft_complex*)kml_fft_malloc(sizeof(kml_fft_complex) * n0 * n1);
kml_fft_plan plan;
plan = kml_fft_plan_dft_2d(n0, n1, in, out, KML_FFT_FORWARD, KML_FFT_ESTIMATE);
kml_fft_execute_dft(plan, in, out);

kml_fft_destroy_plan(plan);
kml_fft_free(in);
kml_fft_free(out);

/*
 * out = {{1.200000e+02, 4.800000e+01}, {1.338564e+02, -1.385641e+01},
 *        {1.061436e+02, 1.385641e+01}, {1.360000e+02, -3.200000e+01},
```

 版权所有 © 华为技术有限公司

```
 *       {1.120000e+02, -8.000000e+00}, {1.120000e+02, -8.000000e+00}}
 */
```

Fortran interface：

```
INTEGER(C_INT) :: N0 = 2
INTEGER(C_INT) :: N1 = 3
REAL(C_DOUBLE), DIMENSION(6, 2) :: INIT
TYPE(KML_FFT_COMPLEX), POINTER :: IN, OUT
TYPE(C_PTR) :: PIN, POUT
TYPE(C_SIZE_T) :: SIZE
SIZE = 16 * N0 * N1
PIN = KML_FFT_MALLOC(SIZE)
POUT = KML_FFT_MALLOC(SIZE)
CALL C_F_POINTER(PIN, IN, SHAPE=[N0 * N1])
CALL C_F_POINTER(POUT, OUT, SHAPE=[N0 * N1])
DATA INIT/120, 8, 0, 0, 0, -8, 0, 8, 0, 16, 16, 8/
INTEGER :: I = 1
DO WHILE(I <= N0 * N1)
   IN(I)%R = INIT(I, 0)
   IN(I)%I = INIT(I, 1)
   I = I + 1
END DO

TYPE(C_PTR) :: PLAN
PLAN = KML_FFT_PLAN_DFT_2D(N0, N1, IN, OUT, KML_FFT_FORWARD, KML_FFT_ESTIMATE)
CALL KML_FFT_EXECUTE_DFT(PLAN, IN, OUT)

CALL KML_FFT_DESTROY_PLAN(PLAN)
CALL KML_FFT_FREE(PIN)
CALL KML_FFT_FREE(POUT)

!
! OUT = /1.200000E+02, 4.800000E+01, 1.338564E+02, -1.385641E+01,
!       1.061436E+02, 1.385641E+01, 1.360000E+02, -3.200000E+01,
!       1.120000E+02, -8.000000E+00, 1.120000E+02, -8.000000E+00/
!
```

6.4.4.4.1.4 kml_fft(f/h)_plan_dft_3d

建立单个连续数据序列3维C2C变换的plan。

接口定义

C interface：

kml_fft_plan kml_fft_plan_dft_3d(int n0, int n1, int n2, kml_fft_complex *in, kml_fft_complex *out, int sign, unsigned flags);

kml_fftf_plan kml_fftf_plan_dft_3d(int n0, int n1, int n2, kml_fftf_complex *in, kml_fftf_complex *out, int sign, unsigned flags);

kml_ffth_plan kml_ffth_plan_dft_3d(int n0, int n1, int n2, kml_ffth_complex *in, kml_ffth_complex *out, int sign, unsigned flags);

Fortran interface：

RES = KML_FFT_PLAN_DFT_3D(N0, N1, N2, IN, OUT, SIGN, FLAGS);

RES = KML_FFTF_PLAN_DFT_3D(N0, N1, N2, IN, OUT, SIGN, FLAGS);

RES = KML_FFTH_PLAN_DFT_3D(N0, N1, N2, IN, OUT, SIGN, FLAGS);

返回值

函数返回一个kml_fft(f/h)_plan类型的结构体指针。将该对象作为参数传入**kml_fft(f/h)_execute**函数中，将对当前提供的输入in和输出out执行FFT变换。也可

 版权所有 © 华为技术有限公司

以通过将该对象作为参数传入 **kml_fft(f/h)_execute_dft**函数中以对新的输入in和输出out执行FFT变换。

如果函数返回非空指针，则表示plan创建成功，否则表示创建失败。

参数

参数名	数据类型	描述	输入/输出
n0	int	输入FFT序列第一维的大小，约束：n0≥1。	输入
n1	int	输入FFT序列第二维的大小，约束：n1≥1。	输入
n2	int	输入FFT序列第三维的大小，约束：n2≥1。	输入
in	• 双精度：kml_fft_complex* • 单精度：kml_fftf_complex* • 半精度：kml_ffth_complex*	输入需要变换的数据。	输入
out	• 双精度：kml_fft_complex* • 单精度：kml_fftf_complex* • 半精度：kml_ffth_complex*	输出快速傅里叶变换后的数据。	输出
sign	int	描述正向变换或反向变换。 • -1(KML_FFT_FORWARD)：forward • +1(KML_FFT_BACKWARD)：backward	输入
flags	unsigned int	planning选项，描述ESTIMATE模式或PATIENT模式。 KML_FFT_ESTIMATE：ESTIMATE模式 KML_FFT_PATIENT：PATIENT模式	输入

版权所有 © 华为技术有限公司

依赖

C: "kfft.h"

示例

C interface：

```
int n0 = 2;
int n1 = 3;
int n2 = 2;
double init[12][2] = {{120, 0}, {8, 8}, {0, 0}, {0, 16}, {0, 16}, {-8, 8}, {-8, 0}, {-8, 8}, {-16, 0}, {0, -16}, {-40,
8}, {-8, -8}};
kml_fft_complex *in;
in = (kml_fft_complex*)kml_fft_malloc(sizeof(kml_fft_complex) * n0 * n1 * n2);
for (int i = 0; i < n0 * n1 * n2; i++) {
    in[i][0] = init[i][0];
    in[i][1] = init[i][1];
}
kml_fft_complex *out;
out = (kml_fft_complex*)kml_fft_malloc(sizeof(kml_fft_complex) * n0 * n1 * n2);
kml_fft_plan plan;
plan = kml_fft_plan_dft_3d(n0, n1, n2, in, out, KML_FFT_FORWARD, KML_FFT_ESTIMATE);
kml_fft_execute_dft(plan, in, out);

kml_fft_destroy_plan(plan);
kml_fft_free(in);
kml_fft_free(out);

/*
 * out = {{4.000000e+01, 4.000000e+01}, {7.200000e+01, 8.000000e+00},
 *        {1.272154e+02, -3.064102e+01}, {1.112154e+02, -3.492820e+01},
 *        {1.687846e+02, 3.864102e+01}, {1.527846e+02, -2.107180e+01},
 *        {2.000000e+02, 5.600000e+01}, {1.680000e+02, -4.000000e+01},
 *        {1.229282e+02, -7.215390e+00}, {6.321539e+01, 4.078461e+01},
 *        {1.090718e+02, -4.878461e+01}, {1.047846e+02, -7.846097e-01}}
 */
```

Fortran interface：

```
INTEGER(C_INT) :: N0 = 2
INTEGER(C_INT) :: N1 = 3
INTEGER(C_INT) :: N2 = 2
REAL(C_DOUBLE), DIMENSION(12, 2) :: INIT
TYPE(KML_FFT_COMPLEX), POINTER :: IN(:), OUT(:)
TYPE(C_PTR) :: PIN, POUT
TYPE(C_SIZE_T) :: SIZE
SIZE = 16 * N0 * N1 * N2
PIN = KML_FFT_MALLOC(SIZE)
POUT = KML_FFT_MALLOC(SIZE)
CALL C_F_POINTER(PIN, IN, SHAPE=[N0 * N1 * N2])
CALL C_F_POINTER(POUT, OUT, SHAPE=[N0 * N1 * N2])
DATA INIT/120, 8, 0, 0, 0, -8, -8, -8, -16, 0, -40, -8, 0, 8, 0, 16, 16, 8, 0, 8, 0, -16, 8, -8/
INTEGER :: I = 1
DO WHILE(I <= N0 * N1 * N2)
    IN(I)%R = INIT(I, 0)
    IN(I)%I = INIT(I, 1)
    I = I + 1
END DO
TYPE(C_PTR) :: PLAN
PLAN = KML_FFT_PLAN_DFT_3D(N0, N1, N2, IN, OUT, KML_FFT_FORWARD, KML_FFT_ESTIMATE)
CALL KML_FFT_EXECUTE_DFT(PLAN, IN, OUT)

CALL KML_FFT_DESTROY_PLAN(PLAN)
CALL KML_FFT_FREE(PIN)
CALL KML_FFT_FREE(POUT)
!
! OUT = /4.000000E+01, 4.000000E+01, 7.200000E+01, 8.000000E+00,
```

 版权所有 © 华为技术有限公司

```
!      1.272154E+02, -3.064102E+01, 1.112154E+02, -3.492820E+01,
!      1.687846E+02, 3.864102E+01, 1.527846E+02, -2.107180E+01,
!      2.000000E+02, 5.600000E+01, 1.680000E+02, -4.000000E+01,
!      1.229282E+02, -7.215390E+00, 6.321539E+01, 4.078461E+01,
!      1.090718E+02, -4.878461E+01, 1.047846E+02, -7.846097E-01/
!
```

6.4.4.4.1.5 kml_fft(f/h)_plan_guru_dft

建立多组数据序列n维C2C变换的plan。其中，const kml_fft_iodim *单个FFT的数据序列不需要是连续的，可以以跨步的形式提供。

接口定义

C interface：

kml_fft_plan kml_fft_plan_guru_dft(int rank, const kml_fft_iodim *dims, int howmany_rank, const kml_fft_iodim *howmany_dims, kml_fft_complex *in, kml_fft_complex *out, int sign, unsigned flags);

kml_fftf_plan kml_fftf_plan_guru_dft(int rank, const kml_fftf_iodim *dims, int howmany_rank, const kml_fftf_iodim *howmany_dims, kml_fftf_complex *in, kml_fftf_complex *out, int sign, unsigned flags);

kml_ffth_plan kml_ffth_plan_guru_dft(int rank, const kml_ffth_iodim *dims, int howmany_rank, const kml_ffth_iodim *howmany_dims, kml_ffth_complex *in, kml_ffth_complex *out, int sign, unsigned flags);

Fortran interface：

RES = KML_FFT_PLAN_GURU_DFT(RANK, DIMS, HOWMANY_RANK, HOWMANY_DIMS, IN, OUT, SIGN, FLAGS);

RES = KML_FFTF_PLAN_GURU_DFT(RANK, DIMS, HOWMANY_RANK, HOWMANY_DIMS, IN, OUT, SIGN, FLAGS);

RES = KML_FFTF_PLAN_GURU_DFT(RANK, DIMS, HOWMANY_RANK, HOWMANY_DIMS, IN, OUT, SIGN, FLAGS);

返回值

函数返回一个kml_fft(f/h)_plan类型的结构体指针。将该对象作为参数传入**kml_fft(f/h)_execute**函数中，将对当前提供的输入in和输出out执行FFT变换。也可以通过将该对象作为参数传入 **kml_fft(f/h)_execute_dft**函数中以对新的输入in和输出out执行FFT变换。

如果函数返回非空指针，则表示plan创建成功，否则表示创建失败。

参数

参数名	数据类型	描述	输入/输出
rank	int	单个FFT序列的维度，约束：1≤rank≤4。	输入

版权所有 © 华为技术有限公司

参数名	数据类型	描述	输入/输出
dims	• 双精度：const kml_fft_iodim* • 单精度：const kml_fftf_iodim* • 半精度：const kml_ffth_iodim*	dims是大小为rank的结构体数组，dims[i]包含以下成员： • int n - 第i维FFT的长度。 • int is - 第i维FFT输入序列的相继元素之间的间隔。 • int os - 第i维FFT输出序列的相继元素之间的间隔。 约束：dims[i].n≥1, for i in 0 to rank - 1。	输入
howmany_rank	int	多个rank维FFT之间的内存排布用howmany_rank维的howmany_dims数组来描述，howmany_rank表示每个要计算的rank维FFT变换的起始地址的内存访问模式所需的维数。 约束：0≤howmany_rank≤4	输入
howmany_dims	• 双精度：const kml_fft_iodim* • 单精度：const kml_fftf_iodim* • 半精度：const kml_ffth_iodim*	howmany_dims是大小为howmany_rank的结构体数组，howmany_dims[i]包含以下成员： • int n - howmany_rank维空间的第i维的待变换FFT的个数。 • int is - 第i维的相继FFT输入序列之间的间隔。 • int os - 第i维的相继FFT输出序列之间的间隔。	输入
in	• 双精度：kml_fft_complex* • 单精度：kml_fftf_complex* • 单精度：kml_ffth_complex*	输入需要变换的数据。	输入

版权所有 © 华为技术有限公司

参数名	数据类型	描述	输入/输出
out	• 双精度：kml_fft_complex* • 单精度：kml_fftf_complex* • 单精度：kml_ffth_complex*	输出快速傅里叶变换后的数据。	输出
sign	int	描述正向变换或反向变换。 • -1(KML_FFT_FORWARD)：forward • +1(KML_FFT_BACKWARD)：backward	输入
flags	unsigned int	planning选项，描述ESTIMATE模式或PATIENT模式。 KML_FFT_ESTIMATE：ESTIMATE模式 KML_FFT_PATIENT：PATIENT模式	输入

依赖

C: "kfft.h"

示例

C interface：

```
    int rank = 2;
    kml_fft_iodim *dims;
    dims = (kml_fft_iodim*)kml_fft_malloc(sizeof(kml_fft_iodim) * rank);
    dims[0].n = 2;
    dims[0].is = 3;
    dims[0].os = 3;
    dims[1].n = 3;
    dims[1].is = 1;
    dims[1].os = 1;
    int howmany_rank = 1;
    kml_fft_iodim *howmany_dims;
    howmany_dims = (kml_fft_iodim*)kml_fft_malloc(sizeof(kml_fft_iodim) * howmany_rank);
    howmany_dims[0].n = 2;
    howmany_dims[0].is = 2 * 3;
    howmany_dims[0].os = 2 * 3;
    double init[12][2] = {{120, 0}, {8, 8}, {0, 0}, {0, 16}, {0, 16}, {-8, 8}, {-8, 0}, {-8, 8}, {-16, 0}, {0, -16}, {-40, 8}, {-8, -8}};
    kml_fft_complex *in;
    in = (kml_fft_complex*)kml_fft_malloc(sizeof(kml_fft_complex) * 12);
    for (int i = 0; i < 12; i++) {
        in[i][0] = init[i][0];
        in[i][1] = init[i][1];
```


版权所有 © 华为技术有限公司

```
    }
    kml_fft_complex *out;
    out = (kml_fft_complex*)kml_fft_malloc(sizeof(kml_fft_complex) * 12);
    kml_fft_plan plan;
    plan = kml_fft_plan_guru_dft(rank, dims, howmany_rank, howmany_dims, in, out, KML_FFT_FORWARD,
KML_FFT_ESTIMATE);
    kml_fft_execute_dft(plan, in, out);

    kml_fft_destroy_plan(plan);
    kml_fft_free(howmany_dims);
    kml_fft_free(dims);
    kml_fft_free(in);
    kml_fft_free(out);

    /*
     * out = {{1.200000e+02, 4.800000e+01}, {1.338564e+02, -1.385641e+01},
     *        {1.061436e+02, 1.385641e+01}, {1.360000e+02, -3.200000e+01},
     *        {1.120000e+02, -8.000000e+00}, {1.120000e+02, -8.000000e+00},
     *        {-8.000000e+01, -8.000000e+00}, {4.878461e+01, 7.846097e-01},
     *        {7.215390e+00, -4.078461e+01}, {1.600000e+01, 2.400000e+01},
     *        {-2.692820e+01, -2.264102e+01}, {-1.307180e+01, 4.664102e+01}}
     */
```

Fortran interface：

```
    INTEGER(C_INT) :: RANK = 2
    INTEGER(C_INT) :: HOWMANY_RANK = 1
    TYPE(KML_FFT_IODIM), POINTER :: DIMS(:), HOWMANY_DIMS(:)
    REAL(C_DOUBLE), DIMENSION(12, 2) :: INIT
    TYPE(KML_FFT_COMPLEX), POINTER :: IN(:), OUT(:)
    TYPE(C_PTR) :: PIN, POUT, PDIMS, PHOWMANY_DIMS
    INTEGER(C_SIZE_T) :: SIZE1, SIZE2, SIZE3
    SIZE1 = 16 * 12
    SIZE2 = 24 * RANK
    SIZE3 = 24 * HOWMANY_RANK
    PDIMS = KML_FFT_MALLOC(SIZE2)
    PHOWMANY_DIMS = KML_FFT_MALLOC(SIZE3)
    PIN = KML_FFT_MALLOC(SIZE1)
    POUT = KML_FFT_MALLOC(SIZE1)
    CALL C_F_POINTER(PIN, IN, SHAPE=[12])
    CALL C_F_POINTER(POUT, OUT, SHAPE=[12])
    CALL C_F_POINTER(PDIMS, DIMS, SHAPE=[RANK])
    CALL C_F_POINTER(PHOWMANY_DIMS, HOWMANY_DIMS, SHAPE=[HOWMANY_RANK])

    DIMS(0)%N = 2
    DIMS(0)%IS = 3
    DIMS(0)%OS = 3
    DIMS(1)%N = 3
    DIMS(1)%IS = 1
    DIMS(1)%OS = 1
    HOWMANY_DIMS(0)%N = 2
    HOWMANY_DIMS(0)%IS = 2 * 3
    HOWMANY_DIMS(0)%OS = 2 * 3
    DATA INIT/120, 8, 0, 0, 0, -8, -8, -8, -16, 0, -40, -8, 0, 8, 0, 16, 16, 8, 0, 8, 0, -16, 8, -8/

    INTEGER :: I = 1
    DO WHILE(I <= 12)
        IN(I)%R = INIT(I, 0)
        IN(I)%I = INIT(I, 1)
        I = I + 1
    END DO

    TYPE(C_PTR) :: PLAN
    PLAN = KML_FFT_PLAN_GURU_DFT(RANK, DIMS, HOWMANY_RANK, HOWMANY_DIMS, IN, OUT,
KML_FFT_FORWARD, KML_FFT_ESTIMATE)
    CALL KML_FFT_EXECUTE_DFT(PLAN, IN, OUT)

    CALL KML_FFT_DESTROY_PLAN(PLAN)
    CALL KML_FFT_FREE(PHOWMANY_DIMS)
    CALL KML_FFT_FREE(PDIMS)
```

 版权所有 © 华为技术有限公司

```
CALL KML_FFT_FREE(PIN)
CALL KML_FFT_FREE(POUT)
!
! OUT = /1.200000E+02, 4.800000E+01, 1.338564E+02, -1.385641E+01,
!       1.061436E+02, 1.385641E+01, 1.360000E+02, -3.200000E+01,
!       1.120000E+02, -8.000000E+00, 1.120000E+02, -8.000000E+00,
!       -8.000000E+01, -8.000000E+00, 4.878461E+01, 7.846097E-01,
!       7.215390E+00, -4.078461E+01, 1.600000E+01, 2.400000E+01,
!       -2.692820E+01, -2.264102E+01, -1.307180E+01, 4.664102E+01/
!
```

6.4.4.4.1.6 kml_fft(f/h)_plan_guru_split_dft

建立多组数据序列n维C2C变换的plan。其中，单个FFT的数据序列不需要是连续的，可以以跨步的形式提供。split类接口的输入和输出分别存储在实部和虚部数组中。

接口定义

C interface：

kml_fft_plan kml_fft_plan_guru_split_dft(int rank, const kml_fft_iodim *dims, int howmany_rank, const kml_fft_iodim *howmany_dims, double *ri, double *ii, double *ro, double *io, unsigned flags);

kml_fftf_plan kml_fftf_plan_guru_split_dft(int rank, const kml_fftf_iodim *dims, int howmany_rank, const kml_fftf_iodim *howmany_dims, float *ri, float *ii, float *ro, float *io, unsigned flags);

kml_ffth_plan kml_ffth_plan_guru_split_dft(int rank, const kml_ffth_iodim *dims, int howmany_rank, const kml_ffth_iodim *howmany_dims, __fp16 *ri, __fp16*ii, __fp16 *ro, __fp16 *io, unsigned flags);

Fortran interface：

RES = KML_FFT_PLAN_GURU_SPLIT_DFT(RANK, DIMS, HOWMANY_RANK, HOWMANY_DIMS, RI, II, RO, IO, FLAGS);

RES = KML_FFTF_PLAN_GURU_SPLIT_DFT(RANK, DIMS, HOWMANY_RANK, HOWMANY_DIMS, RI, II, RO, IO, FLAGS);

RES = KML_FFTH_PLAN_GURU_SPLIT_DFT(RANK, DIMS, HOWMANY_RANK, HOWMANY_DIMS, RI, II, RO, IO, FLAGS);

返回值

函数返回一个kml_fft(f/h)_plan类型的结构体指针。将该对象作为参数传入**kml_fft(f/h)_execute**函数中，将对当前提供的输入ri，ii和输出ro，io执行FFT变换。也可以通过将该对象作为参数传入 **kml_fft(f/h)_execute_split_dft**函数中以对新的输入ri，ii和输出ro，io执行FFT变换。

如果函数返回非空指针，则表示plan创建成功，否则表示创建失败。

版权所有 © 华为技术有限公司

参数

参数名	数据类型	描述	输入/输出
rank	int	单个FFT序列的维度。 约束：1≤rank≤4	输入
dims	• 双精度：const kml_fft_iodim* • 单精度：const kml_fftf_iodim* • 半精度：const kml_ffth_iodim*	dims是大小为rank的结构体数组，dims[i]包含以下成员： • int n - 第i维FFT的长度。 • int is - 第i维FFT输入序列的相继元素之间的间隔。 • int os - 第i维FFT输出序列的相继元素之间的间隔。 约束：dims[i].n≥1, for i in 0 to rank - 1	输入
howmany_rank	int	• 多个rank维FFT之间的内存排布用howmany_rank维的howmany_dims数组来描述，howmany_rank表示每个要计算的rank维FFT变换的起始地址的内存访问模式所需的维数。 • 约束：0≤howmany_rank≤4。	输入
howmany_dims	• 双精度：const kml_fft_iodim* • 单精度：const kml_fftf_iodim* • 半精度：const kml_ffth_iodim*	howmany_dims是大小为howmany_rank的结构体数组，howmany_dims[i]包含以下成员： • int n - howmany_rank维空间的第i维的待变换FFT的个数。 • int is - 第i维的相继FFT输入序列之间的间隔。 • int os - 第i维的相继FFT输出序列之间的间隔。	输入
ri	• 双精度：double* • 单精度：float* • 半精度：__fp16*	输入待变换数据的实部。	输入
ii	• 双精度：double* • 单精度：float* • 半精度：__fp16*	输入待变换数据的虚部。	输入

版权所有 © 华为技术有限公司

参数名	数据类型	描述	输入/输出
ro	• 双精度：double* • 单精度：float* • 半精度：__fp16*	输出待变换数据的实部。	输出
io	• 双精度：double* • 单精度：float* • 半精度：__fp16*	输出待变换数据的虚部。	输出
flags	unsigned int	planning选项，描述ESTIMATE模式或PATIENT模式。 KML_FFT_ESTIMATE：ESTIMATE模式 KML_FFT_PATIENT：PATIENT模式	输入

依赖

C: "kfft.h"

示例

C interface：

```
int rank = 2;
kml_fft_iodim *dims;
dims = (kml_fft_iodim*)kml_fft_malloc(sizeof(kml_fft_iodim) * rank);
dims[0].n = 2;
dims[0].is = 3;
dims[0].os = 3;
dims[1].n = 3;
dims[1].is = 1;
dims[1].os = 1;
int howmany_rank = 1;
kml_fft_iodim *howmany_dims;
howmany_dims = (kml_fft_iodim*)kml_fft_malloc(sizeof(kml_fft_iodim) * howmany_rank);
howmany_dims[0].n = 2;
howmany_dims[0].is = 2 * 3;
howmany_dims[0].os = 2 * 3;
double init[12][2] = {{120, 0}, {8, 8}, {0, 0}, {0, 16}, {0, 16}, {-8, 8}, {-8, 0}, {-8, 8}, {-16, 0}, {0, -16}, {-40,
8}, {-8, -8}};
double *ri;
ri = (double*)kml_fft_malloc(sizeof(double) * 12);
double *ii;
ii = (double*)kml_fft_malloc(sizeof(double) * 12);
for (int i = 0; i < 12; i++) {
    ri[i] = init[i][0];
    ii[i] = init[i][1];
}
double *ro;
ro = (double*)kml_fft_malloc(sizeof(double) * 12);
double *io;
io = (double*)kml_fft_malloc(sizeof(double) * 12);
kml_fft_plan plan;
```


版权所有 © 华为技术有限公司

```
    plan = kml_fft_plan_guru_split_dft(rank, dims, howmany_rank, howmany_dims, ri, ii, ro, io,
KML_FFT_ESTIMATE);
    kml_fft_execute_split_dft(plan, ri, ii, ro, io);

    kml_fft_destroy_plan(plan);
    kml_fft_free(howmany_dims);
    kml_fft_free(dims);
    kml_fft_free(ri);
    kml_fft_free(ii);
    kml_fft_free(ro);
    kml_fft_free(io);

    /*
     * ro = {1.200000e+02, 1.338564e+02, 1.061436e+02, 1.360000e+02,
     *       1.120000e+02, 1.120000e+02, -8.000000e+01, 4.878461e+01,
     *       7.215390e+00, 1.600000e+01, -2.692820e+01, -1.307180e+01}
     */

    /*
     * io = {4.800000e+01, -1.385641e+01, 1.385641e+01, -3.200000e+01,
     *       -8.000000e+00, -8.000000e+00, -8.000000e+00, 7.846097e-01,
     *       -4.078461e+01, 2.400000e+01, -2.264102e+01, 4.664102e+01}
     */
```

Fortran interface：

```
    INTEGER(C_INT) :: RANK = 2
    INTEGER(C_INT) :: HOWMANY_RANK = 1
    TYPE(KML_FFT_IODIM), POINTER :: DIMS(:), HOWMANY_DIMS(:)
    REAL(C_DOUBLE), DIMENSION(12, 2) :: INIT
    TYPE(C_DOUBLE), POINTER :: RI(:), II(:), RO(:), IO(:)
    TYPE(C_PTR) :: PRI, PII, PRO, PIO, PDIMS, PHOWMANY_DIMS
    INTEGER(C_SIZE_T) :: SIZE1, SIZE2, SIZE3
    SIZE1 = 8 * 12
    SIZE2 = 24 * RANK
    SIZE3 = 24 * HOWMANY_RANK
    PDIMS = KML_FFT_MALLOC(SIZE2)
    PHOWMANY_DIMS = KML_FFT_MALLOC(SIZE3)
    PRI = KML_FFT_MALLOC(SIZE1)
    PII = KML_FFT_MALLOC(SIZE1)
    PRO = KML_FFT_MALLOC(SIZE1)
    PIO = KML_FFT_MALLOC(SIZE1)
    CALL C_F_POINTER(PRI, RI, SHAPE=[12])
    CALL C_F_POINTER(PII, II, SHAPE=[12])
    CALL C_F_POINTER(PRO, RO, SHAPE=[12])
    CALL C_F_POINTER(PIO, IO, SHAPE=[12])
    CALL C_F_POINTER(PDIMS, DIMS, SHAPE=[RANK])
    CALL C_F_POINTER(PHOWMANY_DIMS, HOWMANY_DIMS, SHAPE=[HOWMANY_RANK])

    DIMS(0)%N = 2
    DIMS(0)%IS = 3
    DIMS(0)%OS = 3
    DIMS(1)%N = 3
    DIMS(1)%IS = 1
    DIMS(1)%OS = 1
    HOWMANY_DIMS(0)%N = 2
    HOWMANY_DIMS(0)%IS = 2 * 3
    HOWMANY_DIMS(0)%OS = 2 * 3
    DATA INIT/120, 8, 0, 0, 0, -8, -8, -8, -16, 0, -40, -8, 0, 8, 0, 16, 16, 8, 0, 8, 0, -16, 8, -8/
    INTEGER :: I = 1
    DO WHILE(I <= 12)
        RI(I) = INIT(I, 0)
        II(I) = INIT(I, 1)
        I = I + 1
    END DO

    TYPE(C_PTR) :: PLAN
    PLAN = KML_FFT_PLAN_GURU_SPLIT_DFT(RANK, DIMS, HOWMANY_RANK, HOWMANY_DIMS, RI, II, RO,
IO, KML_FFT_ESTIMATE)
    CALL KML_FFT_EXECUTE_SPLIT_DFT(PLAN, RI, II, RO, IO)
```

 版权所有 © 华为技术有限公司

```
CALL KML_FFT_DESTROY_PLAN(PLAN)
CALL KML_FFT_FREE(PHOWMANY_DIMS)
CALL KML_FFT_FREE(PDIMS)
CALL KML_FFT_FREE(PRI)
CALL KML_FFT_FREE(PII)
CALL KML_FFT_FREE(PRO)
CALL KML_FFT_FREE(PIO)

!
! RO = /1.200000E+02, 1.338564E+02, 1.061436E+02, 1.360000E+02,
!      1.120000E+02, 1.120000E+02, -8.000000E+01, 4.878461E+01,
!      7.215390E+00, 1.600000E+01, -2.692820E+01, -1.307180E+01/
!

!
! IO = /4.800000E+01, -1.385641E+01, 1.385641E+01, -3.200000E+01,
!      -8.000000E+00, -8.000000E+00, -8.000000E+00, 7.846097E-01,
!      -4.078461E+01, 2.400000E+01, -2.264102E+01, 4.664102E+01/
!
```

⚠ 注意

kml_fft(f)_plan_guru_split_dft返回的plan可以使用kml_fft(f)_execute计算plan时的输入，也可以使用kml_fft(f)_execute_split_dft计算不同的输入。plan时的输入不需要初始化，但是输入必须是合法的（不能为NULL）。

6.4.4.4.1.7 kml_fft(f/h)_plan_guru64_dft

建立多组数据序列n维C2C变换的plan。其中，单个FFT的数据序列不需要是连续的，可以以跨步的形式提供。与kml_fft(f)_plan_guru_dft不同的是，kml_fft(f)_plan_guru64_dft允许部分参数是64位的整型。

接口定义

C interface：

kml_fft_plan kml_fft_plan_guru64_dft(int rank, const kml_fft_iodim64 *dims, int howmany_rank, const kml_fft_iodim64 *howmany_dims, kml_fft_complex *in, kml_fft_complex *out, int sign, unsigned flags);

kml_fftf_plan kml_fftf_plan_guru64_dft(int rank, const kml_fftf_iodim64 *dims, int howmany_rank, const kml_fftf_iodim64 *howmany_dims, kml_fftf_complex *in, kml_fftf_complex *out, int sign, unsigned flags);

kml_ffth_plan kml_ffth_plan_guru64_dft(int rank, const kml_ffth_iodim64 *dims, int howmany_rank, const kml_ffth_iodim64 *howmany_dims, kml_ffth_complex *in, kml_ffth_complex *out, int sign, unsigned flags);

Fortran interface：

RES = KML_FFT_PLAN_GURU64_DFT(RANK, DIMS, HOWMANY_RANK, HOWMANY_DIMS, IN, OUT, SIGN, FLAGS);

RES = KML_FFTF_PLAN_GURU64_DFT(RANK, DIMS, HOWMANY_RANK, HOWMANY_DIMS, IN, OUT, SIGN, FLAGS);

RES = KML_FFTH_PLAN_GURU64_DFT(RANK, DIMS, HOWMANY_RANK, HOWMANY_DIMS, IN, OUT, SIGN, FLAGS);

 版权所有 © 华为技术有限公司

返回值

函数返回一个kml_fft(f/h)_plan类型的结构体指针。将该对象作为参数传入**kml_fft(f/h)_execute**函数中，将对当前提供的输入in和输出out执行FFT变换。也可以通过将该对象作为参数传入 **kml_fft(f/h)_execute_dft**函数中以对新的输入in和输出out执行FFT变换。

如果函数返回非空指针，则表示plan创建成功，否则表示创建失败。

参数

参数名	数据类型	描述	输入/输出
rank	int	单个FFT序列的维度。 约束：1≤rank≤4	输入
dims	• 双精度：const kml_fft_iodim64 * • 单精度：const kml_fftf_iodim64* • 半精度：const kml_ffth_iodim64*	dims是大小为rank的结构体数组，dims[i]包含以下成员： • ptrdiff_t n - 第i维FFT的长度。 • ptrdiff_t is - 第i维FFT输入序列的相继元素之间的间隔。 • ptrdiff_t os - 第i维FFT输出序列的相继元素之间的间隔。 约束：dims[i].n≥1, for i in 0 to rank - 1	输入
howmany_rank	int	多个rank维FFT之间的内存排布用howmany_rank维的howmany_dims数组来描述，howmany_rank表示每个要计算的rank维FFT变换的起始地址的内存访问模式所需的维数。 约束：0≤howmany_rank≤4	输入
howmany_dims	• 双精度：const kml_fft_iodim64 * • 单精度：const kml_fftf_iodim64* • 半精度：const kml_ffth_iodim64*	howmany_dims是大小为howmany_rank的结构体数组，howmany_dims[i]包含以下成员： • ptrdiff_t n - howmany_rank维空间的第i维的待变换FFT的个数。 • ptrdiff_t is - 第i维的相继FFT输入序列之间的间隔。 • ptrdiff_t os - 第i维的相继FFT输出序列之间的间隔。	输入

版权所有 © 华为技术有限公司

参数名	数据类型	描述	输入/输出
in	• 双精度：kml_fft_complex* • 单精度：kml_fftf_complex* • 半精度：kml_ffth_complex*	输入需要变换的数据。	输入
out	• 双精度：kml_fft_complex* • 单精度：kml_fftf_complex* • 半精度：kml_ffth_complex*	输出快速傅里叶变换后的数据。	输出
sign	int	描述正向变换或反向变换。 • -1(KML_FFT_FORWARD)：forward • +1(KML_FFT_BACKWARD)：backward	输入
flags	unsigned int	planning选项，描述ESTIMATE模式或PATIENT模式。 KML_FFT_ESTIMATE：ESTIMATE模式 KML_FFT_PATIENT：PATIENT模式	输入

依赖

C: "kfft.h"

示例

C interface：

```
int rank = 2;
kml_fft_iodim64 *dims;
dims = (kml_fft_iodim64*)kml_fft_malloc(sizeof(kml_fft_iodim64) * rank);
dims[0].n = 2;
dims[0].is = 3;
dims[0].os = 3;
dims[1].n = 3;
dims[1].is = 1;
dims[1].os = 1;
```

 版权所有 © 华为技术有限公司

```
    int howmany_rank = 1;
    kml_fft_iodim64 *howmany_dims;
    howmany_dims = (kml_fft_iodim64*)kml_fft_malloc(sizeof(kml_fft_iodim64) * howmany_rank);
    howmany_dims[0].n = 2;
    howmany_dims[0].is = 2 * 3;
    howmany_dims[0].os = 2 * 3;
    double init[12][2] = {{120, 0}, {8, 8}, {0, 0}, {0, 16}, {0, 16}, {-8, 8}, {-8, 0}, {-8, 8}, {-16, 0}, {0, -16}, {-40,
8}, {-8, -8}};
    kml_fft_complex *in;
    in = (kml_fft_complex*)kml_fft_malloc(sizeof(kml_fft_complex) * 12);
    for (int i = 0; i < 12; i++) {
        in[i][0] = init[i][0];
        in[i][1] = init[i][1];
    }
    kml_fft_complex *out;
    out = (kml_fft_complex*)kml_fft_malloc(sizeof(kml_fft_complex) * 12);
    kml_fft_plan plan;
    plan = kml_fft_plan_guru64_dft(rank, dims, howmany_rank, howmany_dims, in, out,
KML_FFT_FORWARD, KML_FFT_ESTIMATE);
    kml_fft_execute_dft(plan, in, out);

    kml_fft_destroy_plan(plan);
    kml_fft_free(howmany_dims);
    kml_fft_free(dims);
    kml_fft_free(in);
    kml_fft_free(out);

    /*
     * out = {{1.200000e+02, 4.800000e+01}, {1.338564e+02, -1.385641e+01},
     *        {1.061436e+02, 1.385641e+01}, {1.360000e+02, -3.200000e+01},
     *        {1.120000e+02, -8.000000e+00}, {1.120000e+02, -8.000000e+00},
     *        {-8.000000e+01, -8.000000e+00}, {4.878461e+01, 7.846097e-01},
     *        {7.215390e+00, -4.078461e+01}, {1.600000e+01, 2.400000e+01},
     *        {-2.692820e+01, -2.264102e+01}, {-1.307180e+01, 4.664102e+01}}
     */
```

Fortran interface:

```
    INTEGER(C_INT) :: RANK = 2
    INTEGER(C_INT) :: HOWMANY_RANK = 1
    TYPE(KML_FFT_IODIM64), POINTER :: DIMS(:), HOWMANY_DIMS(:)
    REAL(C_DOUBLE), DIMENSION(12, 2) :: INIT
    TYPE(KML_FFT_COMPLEX), POINTER :: IN(:), OUT(:)
    TYPE(C_PTR) :: PIN, POUT, PDIMS, PHOWMANY_DIMS
    INTEGER(C_SIZE_T) :: SIZE1, SIZE2, SIZE3
    SIZE1 = 16 * 12
    SIZE2 = 24 * RANK
    SIZE3 = 24 * HOWMANY_RANK
    PDIMS = KML_FFT_MALLOC(SIZE2)
    PHOWMANY_DIMS = KML_FFT_MALLOC(SIZE3)
    PIN = KML_FFT_MALLOC(SIZE1)
    POUT = KML_FFT_MALLOC(SIZE1)
    CALL C_F_POINTER(PIN, IN, SHAPE=[12])
    CALL C_F_POINTER(POUT, OUT, SHAPE=[12])
    CALL C_F_POINTER(PDIMS, DIMS, SHAPE=[RANK])
    CALL C_F_POINTER(PHOWMANY_DIMS, HOWMANY_DIMS, SHAPE=[HOWMANY_RANK])

    DIMS(0)%N = 2
    DIMS(0)%IS = 3
    DIMS(0)%OS = 3
    DIMS(1)%N = 3
    DIMS(1)%IS = 1
    DIMS(1)%OS = 1
    HOWMANY_DIMS(0)%N = 2
    HOWMANY_DIMS(0)%IS = 2 * 3
    HOWMANY_DIMS(0)%OS = 2 * 3
    DATA INIT/120, 8, 0, 0, 0, -8, -8, -8, -16, 0, -40, -8, 0, 8, 0, 16, 16, 8, 0, 8, 0, -16, 8, -8/

    INTEGER :: I = 1
    DO WHILE(I <= 12)
```

 版权所有 © 华为技术有限公司

```
    IN%R = INIT(I, 0)
    IN%I = INIT(I, 1)
    I = I + 1
  END DO

  TYPE(C_PTR) :: PLAN
  PLAN = KML_FFT_PLAN_GURU64_DFT(RANK, DIMS, HOWMANY_RANK, HOWMANY_DIMS, IN, OUT,
KML_FFT_FORWARD, KML_FFT_ESTIMATE)
  CALL KML_FFT_EXECUTE_DFT(PLAN, IN, OUT)

  CALL KML_FFT_DESTROY_PLAN(PLAN)
  CALL KML_FFT_FREE(PHOWMANY_DIMS)
  CALL KML_FFT_FREE(PDIMS)
  CALL KML_FFT_FREE(PIN)
  CALL KML_FFT_FREE(POUT)
  !
  ! OUT = /1.200000E+02, 4.800000E+01, 1.338564E+02, -1.385641E+01,
  !       1.061436E+02, 1.385641E+01, 1.360000E+02, -3.200000E+01,
  !       1.120000E+02, -8.000000E+00, 1.120000E+02, -8.000000E+00,
  !       -8.000000E+01, -8.000000E+00, 4.878461E+01, 7.846097E-01,
  !       7.215390E+00, -4.078461E+01, 1.600000E+01, 2.400000E+01,
  !       -2.692820E+01, -2.264102E+01, -1.307180E+01, 4.664102E+01/
  !
```

6.4.4.4.1.8 kml_fft(f/h)_plan_guru64_split_dft

建立多组数据序列n维C2C变换的plan。其中，单个FFT的数据序列不需要是连续的，可以以跨步的形式提供。split类接口的输入和输出分别存储在实部和虚部数组中。与kml_fft(f)_plan_guru_split_dft不同的是，kml_fft(f)_plan_guru64_split_dft允许部分参数是64位的整型。

接口定义

C interface：

kml_fft_plan kml_fft_plan_guru64_split_dft(int rank, const kml_fft_iodim64 *dims, int howmany_rank, const kml_fft_iodim64 *howmany_dims, double *ri, double *ii, double *ro, double *io, unsigned flags);

kml_fftf_plan kml_fftf_plan_guru64_split_dft(int rank, const kml_fftf_iodim64 *dims, int howmany_rank, const kml_fftf_iodim64 *howmany_dims, float *ri, float *ii, float *ro, float *io, unsigned flags);

kml_ffth_plan kml_ffth_plan_guru64_split_dft(int rank, const kml_ffth_iodim64 *dims, int howmany_rank, const kml_ffth_iodim64 *howmany_dims, __fp16 *ri, __fp16 *ii, __fp16 *ro, __fp16 *io, unsigned flags);

Fortran interface：

RES = KML_FFT_PLAN_GURU64_SPLIT_DFT(RANK, DIMS, HOWMANY_RANK, HOWMANY_DIMS, RI, II, RO, IO, FLAGS);

RES = KML_FFTF_PLAN_GURU64_SPLIT_DFT(RANK, DIMS, HOWMANY_RANK, HOWMANY_DIMS, RI, II, RO, IO, FLAGS);

RES = KML_FFTH_PLAN_GURU64_SPLIT_DFT(RANK, DIMS, HOWMANY_RANK, HOWMANY_DIMS, RI, II, RO, IO, FLAGS);

返回值

函数返回一个kml_fft(f/h)_plan类型的结构体指针。将该对象作为参数传入**kml_fft(f/h)_execute**函数中，将对当前提供的输入ri，ii和输出ro，io执行FFT变换。

版权所有 © 华为技术有限公司

也可以通过将该对象作为参数传入 **kml_fft(f/h)_execute_split_dft**函数中以对新的输入ri，ii和输出ro，io执行FFT变换。

如果函数返回非空指针，则表示plan创建成功，否则表示创建失败。

参数

参数名	数据类型	描述	输入/输出
rank	int	单个FFT序列的维度。 约束：1≤rank≤4	输入
dims	• 双精度：const kml_fft_iodim64 * • 单精度：const kml_fftf_iodim64* • 半精度：const kml_ffth_iodim64*	dims是大小为rank的结构体数组，dims[i]包含以下成员： • ptrdiff_t n - 第i维FFT的长度。 • ptrdiff_t is - 第i维FFT输入序列的相继元素之间的间隔。 • ptrdiff_t os - 第i维FFT输出序列的相继元素之间的间隔。 约束：dims[i].n≥1, for i in 0 to rank - 1	输入
howmany_rank	int	多个rank维FFT之间的内存排布用howmany_rank维的howmany_dims数组来描述，howmany_rank表示每个要计算的rank维FFT变换的起始地址的内存访问模式所需的维数。 约束：0≤howmany_rank≤4	输入
howmany_dims	• 双精度：const kml_fft_iodim64 * • 单精度：const kml_fftf_iodim64* • 半精度：const kml_ffth_iodim64*	howmany_dims是大小为howmany_rank的结构体数组，howmany_dims[i]包含以下成员： • ptrdiff_t n - howmany_rank维空间的第i维的待变换FFT的个数。 • ptrdiff_t is - 第i维的相继FFT输入序列之间的间隔。 • ptrdiff_t os - 第i维的相继FFT输出序列之间的间隔。	输入
ri	• 双精度：double* • 单精度：float* • 半精度：__fp16*	输入待变换数据的实部。	输入
ii	• 双精度：double* • 单精度：float* • 半精度：__fp16*	输入待变换数据的虚部。	输入

 版权所有 © 华为技术有限公司

参数名	数据类型	描述	输入/输出
ro	• 双精度：double* • 单精度：float* • 半精度：__fp16*	输出待变换数据的实部。	输出
io	• 双精度：double* • 单精度：float* • 半精度：__fp16*	输出待变换数据的虚部。	输出
flags	unsigned int	planning选项，描述ESTIMATE模式或PATIENT模式。 KML_FFT_ESTIMATE：ESTIMATE模式 KML_FFT_PATIENT：PATIENT模式	输入

依赖

C: "kfft.h"

示例

C interface：

```
int rank = 2;
kml_fft_iodim64 *dims;
dims = (kml_fft_iodim64*)kml_fft_malloc(sizeof(kml_fft_iodim64) * rank);
dims[0].n = 2;
dims[0].is = 3;
dims[0].os = 3;
dims[1].n = 3;
dims[1].is = 1;
dims[1].os = 1;
int howmany_rank = 1;
kml_fft_iodim64 *howmany_dims;
howmany_dims = (kml_fft_iodim64*)kml_fft_malloc(sizeof(kml_fft_iodim64) * howmany_rank);
howmany_dims[0].n = 2;
howmany_dims[0].is = 2 * 3;
howmany_dims[0].os = 2 * 3;
double init[12][2] = {{120, 0}, {8, 8}, {0, 0}, {0, 16}, {0, 16}, {-8, 8}, {-8, 0}, {-8, 8}, {-16, 0}, {0, -16}, {-40, 8}, {-8, -8}};
double *ri;
ri = (double*)kml_fft_malloc(sizeof(double) * 12);
double *ii;
ii = (double*)kml_fft_malloc(sizeof(double) * 12);
for (int i = 0; i < 12; i++) {
    ri[i] = init[i][0];
    ii[i] = init[i][1];
}
double *ro;
ro = (double*)kml_fft_malloc(sizeof(double) * 12);
double *io;
io = (double*)kml_fft_malloc(sizeof(double) * 12);
kml_fft_plan plan;
```

 版权所有 © 华为技术有限公司

```
    plan = kml_fft_plan_guru64_split_dft(rank, dims, howmany_rank, howmany_dims, ri, ii, ro, io,
KML_FFT_ESTIMATE);
    kml_fft_execute_split_dft(plan, ri, ii, ro, io);

    kml_fft_destroy_plan(plan);
    kml_fft_free(howmany_dims);
    kml_fft_free(dims);
    kml_fft_free(ri);
    kml_fft_free(ii);
    kml_fft_free(ro);
    kml_fft_free(io);

    /*
     * ro = {1.200000e+02, 1.338564e+02, 1.061436e+02, 1.360000e+02,
     *       1.120000e+02, 1.120000e+02, -8.000000e+01, 4.878461e+01,
     *       7.215390e+00, 1.600000e+01, -2.692820e+01, -1.307180e+01}
     */

    /*
     * io = {4.800000e+01, -1.385641e+01, 1.385641e+01, -3.200000e+01,
     *       -8.000000e+00, -8.000000e+00, -8.000000e+00, 7.846097e-01,
     *       -4.078461e+01, 2.400000e+01, -2.264102e+01, 4.664102e+01}
     */
```

Fortran interface：

```
    INTEGER(C_INT) :: RANK = 2
    INTEGER(C_INT) :: HOWMANY_RANK = 1
    TYPE(KML_FFT_IODIM64), POINTER :: DIMS(:), HOWMANY_DIMS(:)
    REAL(C_DOUBLE), DIMENSION(12, 2) :: INIT
    TYPE(C_DOUBLE), POINTER :: RI(:), II(:), RO(:), IO(:)
    TYPE(C_PTR) :: PRI, PII, PRO, PIO, PDIMS, PHOWMANY_DIMS
    INTEGER(C_SIZE_T) :: SIZE1, SIZE2, SIZE3
    SIZE1 = 8 * 12
    SIZE2 = 24 * RANK
    SIZE3 = 24 * HOWMANY_RANK
    PDIMS = KML_FFT_MALLOC(SIZE2)
    PHOWMANY_DIMS = KML_FFT_MALLOC(SIZE3)
    PRI = KML_FFT_MALLOC(SIZE1)
    PII = KML_FFT_MALLOC(SIZE1)
    PRO = KML_FFT_MALLOC(SIZE1)
    PIO = KML_FFT_MALLOC(SIZE1)
    CALL C_F_POINTER(PRI, RI, SHAPE=[12])
    CALL C_F_POINTER(PII, II, SHAPE=[12])
    CALL C_F_POINTER(PRO, RO, SHAPE=[12])
    CALL C_F_POINTER(PIO, IO, SHAPE=[12])
    CALL C_F_POINTER(PDIMS, DIMS, SHAPE=[RANK])
    CALL C_F_POINTER(PHOWMANY_DIMS, HOWMANY_DIMS, SHAPE=[HOWMANY_RANK])

    DIMS(0)%N = 2
    DIMS(0)%IS = 3
    DIMS(0)%OS = 3
    DIMS(1)%N = 3
    DIMS(1)%IS = 1
    DIMS(1)%OS = 1
    HOWMANY_DIMS(0)%N = 2
    HOWMANY_DIMS(0)%IS = 2 * 3
    HOWMANY_DIMS(0)%OS = 2 * 3
    DATA INIT/120, 8, 0, 0, 0, -8, -8, -8, -16, 0, -40, -8, 0, 8, 0, 16, 16, 8, 0, 8, 0, -16, 8, -8/
    INTEGER :: I = 1
    DO WHILE(I <= 12)
        RI(I) = INIT(I, 0)
        II(I) = INIT(I, 1)
        I = I + 1
    END DO

    TYPE(C_PTR) :: PLAN
    PLAN = KML_FFT_PLAN_GURU64_SPLIT_DFT(RANK, DIMS, HOWMANY_RANK, HOWMANY_DIMS, RI, II,
RO, IO, KML_FFT_ESTIMATE)
    CALL KML_FFT_EXECUTE_SPLIT_DFT(PLAN, RI, II, RO, IO)
```

 版权所有 © 华为技术有限公司

```
CALL KML_FFT_DESTROY_PLAN(PLAN)
CALL KML_FFT_FREE(PHOWMANY_DIMS)
CALL KML_FFT_FREE(PDIMS)
CALL KML_FFT_FREE(PRI)
CALL KML_FFT_FREE(PII)
CALL KML_FFT_FREE(PRO)
CALL KML_FFT_FREE(PIO)

!
! RO = /1.200000E+02, 1.338564E+02, 1.061436E+02, 1.360000E+02,
!      1.120000E+02, 1.120000E+02, -8.000000E+01, 4.878461E+01,
!      7.215390E+00, 1.600000E+01, -2.692820E+01, -1.307180E+01/
!

!
! IO = /4.800000E+01, -1.385641E+01, 1.385641E+01, -3.200000E+01,
!      -8.000000E+00, -8.000000E+00, -8.000000E+00, 7.846097E-01,
!      -4.078461E+01, 2.400000E+01, -2.264102E+01, 4.664102E+01/
!
```

6.4.4.4.1.9 kml_fft(f/h)_plan_many_dft

建立howmany组数据序列n维C2C变换的plan。其中，单个FFT的数据序列不需要是连续的，可以以跨步的形式提供。

接口定义

C interface：

kml_fft_plan kml_fft_plan_many_dft(int rank, const int *n, int howmany, kml_fft_complex *in, const int *inembed, int istride, int idist, kml_fft_complex *out, const int *onembed, int ostride, int odist, int sign, unsigned flags);

kml_fftf_plan kml_fftf_plan_many_dft(int rank, const int *n, int howmany, kml_fftf_complex *in, const int *inembed, int istride, int idist, kml_fftf_complex *out, const int *onembed, int ostride, int odist, int sign, unsigned flags);

kml_ffth_plan kml_ffth_plan_many_dft(int rank, const int *n, int howmany, kml_ffth_complex *in, const int *inembed, int istride, int idist, kml_ffth_complex *out, const int *onembed, int ostride, int odist, int sign, unsigned flags);

Fortran interface：

RES = KML_FFT_PLAN_MANY_DFT(RANK, N, HOWMANY, IN, INEMBED, ISTRIDE, IDIST, OUT, ONEMBED, OSTRIDE, ODIST, SIGN, FLAGS);

RES = KML_FFTF_PLAN_MANY_DFT(RANK, N, HOWMANY, IN, INEMBED, ISTRIDE, IDIST, OUT, ONEMBED, OSTRIDE, ODIST, SIGN, FLAGS);

RES = KML_FFTH_PLAN_MANY_DFT(RANK, N, HOWMANY, IN, INEMBED, ISTRIDE, IDIST, OUT, ONEMBED, OSTRIDE, ODIST, SIGN, FLAGS);

返回值

函数返回一个kml_fft(f/h)_plan类型的结构体指针。将该对象作为参数传入**kml_fft(f/h)_execute**函数中，将对当前提供的输入in和输出out执行FFT变换。也可以通过将该对象作为参数传入 **kml_fft(f/h)_execute_dft**函数中以对新的输入in和输出out执行FFT变换。

如果函数返回非空指针，则表示plan创建成功，否则表示创建失败。

版权所有 © 华为技术有限公司

参数

参数名	数据类型	描述	输入/输出
rank	int	FFT变换的维度是rank，约束：1≤rank≤4。	输入
n	const int*	n为维度为rank的数组，包含FFT序列每一维度的大小，约束：n[i] ≥1, for i in 0, rank - 1。	输入
howmany	int	howmany表示指定多少个多维FFT变换。	输入
in	• 双精度：kml_fft_complex* • 单精度：kml_fftf_complex* • 半精度：kml_ffth_complex*	输入需要变换的数据。	输入
inembed	const int*	inembed是大小为rank的数组或NULL，该数组表示FFT数据in存储的更大空间的每一维度的大小。 约束：inembed[i]≥n[i] for i in 0, rank-1或者如果inembed == NULL，则可以认为inembed与n相等。	输入
istride	int	FFT输入序列的相继元素之间的间隔。	输入
idist	int	idist表示输入的每个FFT序列的间隔。	输入
out	• 双精度：kml_fft_complex* • 单精度：kml_fftf_complex* • 半精度：kml_ffth_complex*	输出快速傅里叶变换后的数据。	输出
onembed	const int*	onembed是大小为rank的数组或者NULL，该数组表示FFT数据out存储的更大空间的每一维度的大小。 约束：onembed[i]≥n[i] for i in 0, rank-1或者如果onembed == NULL，则可以认为onembed与n相等。	输入

版权所有 © 华为技术有限公司

参数名	数据类型	描述	输入/输出
ostride	int	FFT输出序列的相继元素之间的间隔。	输入
odist	int	odist表示输出的每个FFT序列的间隔。	输入
sign	int	描述正向变换或反向变换。 ● -1(KML_FFT_FORWARD)：forward ● +1(KML_FFT_BACKWARD)：backward	输入
flags	unsigned int	planning选项，描述ESTIMATE模式或PATIENT模式。 KML_FFT_ESTIMATE：ESTIMATE模式 KML_FFT_PATIENT：PATIENT模式	输入

依赖

C: "kfft.h"

示例

C interface：

```
   int rank = 1;
   int *n;
   n = (int*)kml_fft_malloc(sizeof(int) * rank);
   n[0] = 2;
   int howmany = 3;
   int istride = 1;
   int ostride = 1;
   int idist = 2;
   int odist = 2;
   double init[6][2] = {{120, 0}, {8, 8}, {0, 0}, {0, 16}, {0, 16}, {-8, 8}};
   kml_fft_complex *in;
   in = (kml_fft_complex*)kml_fft_malloc(sizeof(kml_fft_complex) * 6);
   for (int i = 0; i < 6; i++) {
       in[i][0] = init[i][0];
       in[i][1] = init[i][1];
   }
   kml_fft_complex *out;
   out = (kml_fft_complex*)kml_fft_malloc(sizeof(kml_fft_complex) * 6);
   kml_fft_plan plan;
   plan = kml_fft_plan_many_dft(rank, n, howmany, in, NULL, istride, idist, out, NULL, ostride, odist,
KML_FFT_FORWARD, KML_FFT_ESTIMATE);
   kml_fft_execute_dft(plan, in, out);

   kml_fft_destroy_plan(plan);
   kml_fft_free(n);
   kml_fft_free(in);
   kml_fft_free(out);
```

 版权所有 © 华为技术有限公司

```
/*
 * out = {{1.280000e+02, 8.000000e+00}, {1.120000e+02, -8.000000e+00},
 *        {0.000000e+00, 1.600000e+01}, {0.000000e+00, -1.600000e+01},
 *        {-8.000000e+00, 2.400000e+01}, {8.000000e+00, 8.000000e+00}}
 */
```

Fortran interface:

```
INTEGER(C_INT) :: RANK = 1
INTEGER(C_INT) :: N(1)
N(1) = 2
REAL(C_DOUBLE), DIMENSION(6, 2) :: INIT
TYPE(KML_FFT_COMPLEX), POINTER :: IN(:), OUT(:)
INTEGER, POINTER :: INEMBED(:), ONEMBED(:)
TYPE(C_PTR) :: PIN, POUT
INTEGER(C_INT) :: HOWMANY = 3
INTEGER(C_INT) :: ISTRIDE = 1
INTEGER(C_INT) :: OSTRIDE = 1
INTEGER(C_INT) :: IDIST = 2
INTEGER(C_INT) :: ODIST = 2
INTEGER(C_SIZE_T) :: SIZE = 16 * 6
PIN = KML_FFT_MALLOC(SIZE)
POUT = KML_FFT_MALLOC(SIZE)
CALL C_F_POINTER(PIN, IN, SHAPE=[6])
CALL C_F_POINTER(POUT, OUT, SHAPE=[6])
CALL C_F_POINTER(C_NULL_PTR, INEMBED, SHAPE=[0])
CALL C_F_POINTER(C_NULL_PTR, ONEMBED, SHAPE=[0])

DATA INIT/120, 8, 0, 0, 0, -8, 0, 8, 0, 16, 16, 8/
INTEGER(C_INT) :: I = 1
DO WHILE(I <= 6)
   IN(I)%R = INIT(I, 0)
   IN(I)%I = INIT(I, 1)
   I = I + 1
END DO
TYPE(C_PTR) :: PLAN
PLAN = KML_FFT_PLAN_MANY_DFT(RANK, N, HOWMANY, IN, INEMBED, ISTRIDE, IDIST, OUT,
ONEMBED, OSTRIDE, ODIST, KML_FFT_FORWARD, KML_FFT_ESTIMATE);
CALL KML_FFT_EXECUTE_DFT(PLAN, IN, OUT)

CALL KML_FFT_DESTROY_PLAN(PLAN)
CALL KML_FFT_FREE(PIN)
CALL KML_FFT_FREE(POUT)
!
! OUT = /1.280000E+02, 8.000000E+00, 1.120000E+02, -8.000000E+00,
!       0.000000E+00, 1.600000E+01, 0.000000E+00, -1.600000E+01,
!       -8.000000E+00, 2.400000E+01, 8.000000E+00, 8.000000E+00/
!
```

6.4.4.4.1.10 kml_ffth_plan_dft_scale

建立单个连续数据序列n维C2C变换的plan。

接口定义

C interface:

kml_ffth_plan kml_ffth_plan_dft_scale(int rank, const int *n, kml_ffth_complex *in, kml_ffth_float_complex *out, int sign, unsigned flags);

返回值

函数返回一个kml_ffth_plan类型的结构体指针。将该对象作为参数传入 **kml_fft(f/h)_execute**函数中，将对当前提供的输入in和输出out执行FFT变换。也可以通过将该对象作为参数传入 **kml_ffth_execute_dft_scale**函数中以对新的输入in和输出out执行FFT变换。

 版权所有 © 华为技术有限公司

如果函数返回非空指针，则表示plan创建成功，否则表示创建失败。

参数

参数名	数据类型	描述	输入/输出
rank	int	FFT变换的维度是rank，约束：1≤rank≤4。	输入
n	const int*	n为维度为rank的数组，包含FFT序列每一维度的大小，约束：n[i]≥1, for i in 0 to rank - 1。	输入
in	• kml_ffth_complex*	输入需要变换的数据。	输入
out	• kml_ffth_float_complex	输出快速傅里叶变换后的数据。	输出
sign	int	描述正向变换或反向变换。 • -1(KML_FFT_FORWARD)：forward • +1(KML_FFT_BACKWARD)：backward	输入
flags	unsigned int	planning选项，描述ESTIMATE模式或PATIENT模式。 KML_FFT_ESTIMATE：ESTIMATE模式 KML_FFT_PATIENT：PATIENT模式	输入

依赖

C: "kfft.h"

示例

C interface：

```
   int rank = 2;
   int *n;
   n = (int*)kml_ffth_malloc(sizeof(int) * rank);
   n[0] = 2;
   n[1] = 3;
   __fp16 init[6][2] = {{120, 0}, {8, 8}, {0, 0}, {0, 16}, {0, 16}, {-8, 8}};
   kml_ffth_complex *in;
   in = (kml_ffth_complex*)kml_ffth_malloc(sizeof(kml_ffth_complex) * n[0] * n[1]);
   for (int i = 0; i < n[0] * n[1]; i++) {
       in[i][0] = init[i][0];
       in[i][1] = init[i][1];
   }

   kml_ffth_float_complex  *out;
   out = (kml_ffth_float_complex *)kml_ffth_malloc(sizeof(
kml_ffth_float_complex) * n[0] * n[1]);
```


版权所有 © 华为技术有限公司

```
kml_ffth_plan plan;
plan = kml_ffth_plan_dft_scale(rank, n, in, out, KML_FFT_FORWARD, KML_FFT_ESTIMATE);
kml_ffth_dft_execute_scale(plan, in, out);

kml_ffth_destroy_plan(plan);
kml_ffth_free(n);
kml_ffth_free(in);
kml_ffth_free(out);

/*
 * out = {{1.200000e+02, 4.800000e+01}, {1.338564e+02, -1.385641e+01},
 *        {1.061436e+02, 1.385641e+01}, {1.360000e+02, -3.200000e+01},
 *        {1.120000e+02, -8.000000e+00}, {1.120000e+02, -8.000000e+00}}
 */
```

6.4.4.4.1.11 kml_ffth_plan_dft_scale_1d

建立单个连续数据序列1维C2C变换的plan。

接口定义

C interface:

kml_ffth_plan kml_ffth_plan_dft_scale_1d(int n, kml_ffth_complex *in, kml_ffth_float_complex *out, int sign, unsigned flags);

返回值

函数返回一个kml_ffth_plan类型的结构体指针。将该对象作为参数传入 **kml_fft(f/h)_execute**函数中，将对当前提供的输入in和输出out执行FFT变换。也可以通过将该对象作为参数传入 **kml_ffth_execute_dft_scale**函数中以对新的输入in和输出out执行FFT变换。

如果函数返回非空指针，则表示plan创建成功，否则表示创建失败。

参数

参数名	数据类型	描述	输入/输出
n	int	输入FFT序列的大小，约束：n≥1。	输入
in	• kml_ffth_complex*	输入需要变换的数据。	输入
out	• kml_ffth_float_complex*	输出快速傅里叶变换后的数据。	输出
sign	int	描述正向变换或反向变换。 • -1(KML_FFT_FORWARD):forward • +1(KML_FFT_BACKWARD):backward	输入

版权所有 © 华为技术有限公司

参数名	数据类型	描述	输入/输出
flags	unsigned int	planning选项，描述ESTIMATE模式或PATIENT模式。 KML_FFT_ESTIMATE：ESTIMATE模式 KML_FFT_PATIENT：PATIENT模式	输入

依赖

C: "kfft.h"

示例

C interface：

```
int N = 16;
kml_ffth_complex *in;
kml_ffth_complex *out;
kml_ffth_plan plan;
in = (kml_ffth_complex*)kml_ffth_malloc(sizeof(kml_ffth_complex) * N);
out = (kml_ffth_float_complex*)kml_ffth_malloc(sizeof(kml_ffth_float_complex) * N);
for (int i = 0; i < N; i++){
    in[i][0] = (__fp16)i;
    in[i][1] = (__fp16)i;
}
plan = kml_ffth_plan_dft_scale_1d(N, in , out, KML_FFT_FORWARD, KML_FFT_ESTIMATE);
kml_ffth_execute_dft_scale(plan, in, out);

kml_ffth_destroy_plan(plan);
kml_ffth_free(in);
kml_ffth_free(out);

/*
 * out = {{1.200000e+02, 1.200000e+02}, {-4.821872e+01, 3.221872e+01},
 *        {-2.731371e+01, 1.131371e+01}, {-1.997285e+01, 3.972846e+00},
 *        {-1.600000e+01, 0.000000e+00}, {-1.334543e+01, -2.654571e+00},
 *        {-1.131371e+01, -4.686292e+00}, {-9.591299e+00, -6.408701e+00},
 *        {-8.000000e+00, -8.000000e+00}, {-6.408701e+00, -9.591299e+00},
 *        {-4.686292e+00, -1.131371e+01}, {-2.654571e+00, -1.334543e+01},
 *        {0.000000e+00, -1.600000e+01}, {3.972846e+00, -1.997285e+01},
 *        {1.131371e+01, -2.731371e+01}, {3.221872e+01, -4.821872e+01}}
 */
```

6.4.4.4.1.12 kml_ffth_plan_dft_scale_2d

建立单个连续数据序列2维C2C变换的plan。

接口定义

C interface：

kml_ffth_plan kml_ffth_plan_dft_scale_2d(int n0, int n1, kml_ffth_complex *in, kml_ffth_float_complex *out, int sign, unsigned flags);

 版权所有 © 华为技术有限公司

返回值

函数返回一个kml_ffth_plan类型的结构体指针。将该对象作为参数传入 **kml_fft(f/h)_execute**函数中，将对当前提供的输入in和输出out执行FFT变换。也可以通过将该对象作为参数传入 **kml_ffth_execute_dft_scale**函数中以对新的输入in和输出out执行FFT变换。

如果函数返回非空指针，则表示plan创建成功，否则表示创建失败。

参数

参数名	数据类型	描述	输入/输出
n0	int	输入FFT序列的大小，约束：n0≥1。	输入
n1	int	输入FFT序列的大小，约束：n1≥1。	输入
in	• kml_ffth_complex*	输入需要变换的数据。	输入
out	• kml_ffth_float_complex *	输出快速傅里叶变换后的数据。	输出
sign	int	描述正向变换或反向变换。 • -1(KML_FFT_FORWARD)：forward • +1(KML_FFT_BACKWARD)：backward	输入
flags	unsigned int	planning选项，描述ESTIMATE模式或PATIENT模式。 KML_FFT_ESTIMATE：ESTIMATE模式 KML_FFT_PATIENT：PATIENT模式	输入

依赖

C: "kfft.h"

示例

C interface：

```
int n0 = 2;
int n1 = 3;
kml_ffth_complex *in;
kml_ffth_complex *out;
kml_ffth_plan plan;
in = (kml_ffth_complex*)kml_ffth_malloc(sizeof(kml_ffth_complex) * n0 * n1);
out = (kml_ffth_float_complex*)kml_ffth_malloc(sizeof(kml_ffth_float_complex) * n0 * n1);
for (int i = 0; i < n0 * n1; i++){
    in[i][0] = (__fp16)i;
    in[i][1] = (__fp16)i;
}
```

 版权所有 © 华为技术有限公司

```
plan = kml_ffth_plan_dft_scale_2d(n0 , n1, in , out, KML_FFT_FORWARD, KML_FFT_ESTIMATE);
kml_ffth_execute_dft_scale(plan, in, out);

kml_ffth_destroy_plan(plan);
kml_ffth_free(in);
kml_ffth_free(out);

/*
 * out = {{1.200000e+02, 4.800000e+01}, {1.338564e+02, -1.385641e+01},
 *        {1.061436e+02, 1.385641e+01}, {1.360000e+02, -3.200000e+01},
 *        {1.120000e+02, -8.000000e+00}, {1.120000e+02, -8.000000e+00}}
 */
```

6.4.4.4.1.13 kml_ffth_plan_many_dft_scale

建立howmany组数据序列n维C2C变换的plan。其中，单个FFT的数据序列不需要是连续的，可以以跨步的形式提供。

接口定义

C interface：

kml_ffth_plan kml_ffth_plan_many_dft_scale(int rank, const int *n, int howmany, kml_ffth_complex *in, const int *inembed, int istride, int idist, kml_ffth_float_complex *out, const int *onembed, int ostride, int odist, int sign, unsigned flags);

返回值

函数返回一个kml_ffth_plan类型的结构体指针。将该对象作为参数传入 **kml_fft(f/h)_execute**函数中，将对当前提供的输入in和输出out执行FFT变换。也可以通过将该对象作为参数传入 **kml_ffth_execute_dft_scale**函数中以对新的输入in和输出out执行FFT变换。

如果函数返回非空指针，则表示plan创建成功，否则表示创建失败。

参数

参数名	数据类型	描述	输入/输出
rank	int	FFT变换的维度是rank，约束：1≤rank≤4。	输入
n	const int*	n为维度为rank的数组，包含FFT序列每一维度的大小，约束：n[i] ≥1, for i in 0, rank - 1。	输入
howmany	int	howmany表示指定多少个多维FFT变换。	输入
in	• kml_ffth_complex*	输入需要变换的数据。	输入

版权所有 © 华为技术有限公司

参数名	数据类型	描述	输入/输出
inembed	const int*	inembed是大小为rank的数组或NULL，该数组表示FFT数据in存储的更大空间的每一维度的大小。 约束：inembed[i]≥n[i] for i in 0, rank-1或者如果inembed == NULL，则可以认为inembed与n相等。	输入
istride	int	FFT输入序列的相继元素之间的间隔。	输入
idist	int	idist表示输入的每个FFT序列的间隔。	输入
out	• kml_ffth_float_complex*	输出快速傅里叶变换后的数据。	输出
onembed	const int*	onembed是大小为rank的数组或者NULL，该数组表示FFT数据out存储的更大空间的每一维度的大小。 约束：onembed[i]≥n[i] for i in 0, rank-1或者如果onembed == NULL，则可以认为onembed与n相等。	输入
ostride	int	FFT输出序列的相继元素之间的间隔。	输入
odist	int	odist表示输出的每个FFT序列的间隔。	输入
sign	int	描述正向变换或反向变换。 • -1(KML_FFT_FORWARD)：forward • +1(KML_FFT_BACKWARD)：backward	输入
flags	unsigned int	planning选项，描述ESTIMATE模式或PATIENT模式。 KML_FFT_ESTIMATE：ESTIMATE模式 KML_FFT_PATIENT：PATIENT模式	输入

依赖

C: "kfft.h"

 版权所有 © 华为技术有限公司

示例

C interface：

```
int rank = 1;
int *n;
n = (int*)kml_ffth_malloc(sizeof(int) * rank);
n[0] = 2;
int howmany = 3;
int istride = 1;
int ostride = 1;
int idist = 2;
int odist = 2;
__fp16 init[6][2] = {{120, 0}, {8, 8}, {0, 0}, {0, 16}, {0, 16}, {-8, 8}};
kml_ffth_complex *in;
in = (kml_ffth_complex*)kml_ffth_malloc(sizeof(kml_ffth_complex) * 6);
for (int i = 0; i < 6; i++) {
    in[i][0] = init[i][0];
    in[i][1] = init[i][1];
}
kml_ffth_float_complex *out;
out = (kml_ffth_float_complex*)kml_ffth_malloc(sizeof(kml_ffth_float_complex) * 6);
kml_ffth_plan plan;
plan = kml_ffth_plan_many_dft_scale(rank, n, howmany, in, NULL, istride, idist, out, NULL, ostride, odist,
KML_FFT_FORWARD, KML_FFT_ESTIMATE);
kml_ffth_execute_dft_scale(plan, in, out);

kml_ffth_destroy_plan(plan);
kml_ffth_free(n);
kml_ffth_free(in);
kml_ffth_free(out);

/*
 * out = {{1.280000e+02, 8.000000e+00}, {1.120000e+02, -8.000000e+00},
 *        {0.000000e+00, 1.600000e+01}, {0.000000e+00, -1.600000e+01},
 *        {-8.000000e+00, 2.400000e+01}, {8.000000e+00, 8.000000e+00}}
 */
```

6.4.4.4.2 R2C 变换

6.4.4.4.2.1 kml_fft(f/h)_plan_dft_r2c

建立单个连续数据序列n维R2C变换的plan。

接口定义

C interface：

kml_fft_plan kml_fft_plan_dft_r2c(int rank, const int *n, double *in, kml_fft_complex *out, unsigned flags);

kml_fftf_plan kml_fftf_plan_dft_r2c(int rank, const int *n, float *in, kml_fftf_complex *out, unsigned flags);

kml_ffth_plan kml_ffth_plan_dft_r2c(int rank, const int *n, __fp16 *in, kml_ffth_complex *out, unsigned flags);

Fortran interface：

RES = KML_FFT_PLAN_DFT_R2C(RANK, N, IN, OUT, FLAGS);

RES = KML_FFTF_PLAN_DFT_R2C(RANK, N, IN, OUT, FLAGS);

RES = KML_FFTH_PLAN_DFT_R2C(RANK, N, IN, OUT, FLAGS);

 版权所有 © 华为技术有限公司

返回值

函数返回一个kml_fft(f/h)_plan类型的结构体指针。将该对象作为参数传入**kml_fft(f/h)_execute**函数中，将对当前提供的输入in和输出out执行FFT变换。也可以通过将该对象作为参数传入 **kml_fft(f/h)_execute_dft_r2c**函数中以对新的输入in和输出out执行FFT变换。

如果函数返回非空指针，则表示plan创建成功，否则表示创建失败。

参数

参数名	数据类型	描述	输入/输出
rank	int	FFT变换的维度是rank，约束：1 <= rank <= 4。	输入
n	const int*	n为维度为rank的数组，包含FFT序列每一维度的大小，约束：n[i] >= 1, for i in 0 to rank - 1。	输入
in	• 双精度：double* • 单精度：float* • 半精度：__fp16*	输入需要变换的数据。	输入
out	• 双精度：kml_fft_complex* • 单精度：kml_fftf_complex* • 半精度：kml_ffth_complex*	输出快速傅里叶变换后的数据。	输出
flags	unsigned int	planning选项，描述ESTIMATE模式或PATIENT模式。 KML_FFT_ESTIMATE：ESTIMATE模式 KML_FFT_PATIENT：PATIENT模式	输入

依赖

C: "kfft.h"

示例

C interface：

```
int rank = 2;
int *n;
```


版权所有 © 华为技术有限公司

```
n = (int*)kml_fft_malloc(sizeof(int) * rank);
n[0] = 2;
n[1] = 4;
double init[8] = {1, 2, 3, 4, 5, 6, 7, 8};
double *in;
in = (double*)kml_fft_malloc(sizeof(double) * n[0] * n[1]);
for (int i = 0; i < n[0] * n[1]; i++) {
    in[i] = init[i];
}
kml_fft_complex *out;
out = (kml_fft_complex*)kml_fft_malloc(sizeof(kml_fft_complex) * n[0] * (n[1] / 2 + 1));
kml_fft_plan plan;
plan = kml_fft_plan_dft_r2c(rank, n, in, out, KML_FFT_ESTIMATE);
kml_fft_execute_dft_r2c(plan, in, out);

kml_fft_destroy_plan(plan);
kml_fft_free(n);
kml_fft_free(in);
kml_fft_free(out);

/*
 * out = {{3.600000e+01, 0.000000e+00}, {-4.000000e+00, 4.000000e+00},
 *        {-4.000000e+00, 0.000000e+00}, {-1.600000e+01, 0.000000e+00},
 *        {0.000000e+00, 0.000000e+00}, {0.000000e+00, 0.000000e+00}}
 */
```

Fortran interface：

```
INTEGER(C_INT) :: RANK = 2
INTEGER(C_INT) :: N(2)
REAL(C_DOUBLE) :: INIT(8)
REAL(C_DOUBLE), POINTER :: IN
TYPE(KML_FFT_COMPLEX), POINTER :: OUT
TYPE(C_PTR) :: PIN, POUT
INTEGER(C_SIZE_T) :: RSIZE, CSIZE
RSIZE = N(1) * N(2)
CSIZE = N(1) * (N(2) / 2 + 1)
PIN = KML_FFT_MALLOC(8 * RSIZE)
POUT = KML_FFT_MALLOC(16 * CSIZE)
CALL C_F_POINTER(PIN, IN, SHAPE=[RSIZE])
CALL C_F_POINTER(POUT, OUT, SHAPE=[CSIZE])
DATA N/2, 4/
DATA INIT/1, 2, 3, 4, 5, 6, 7, 8/
INTEGER :: I = 1
DO WHILE(I <= RSIZE)
   IN(I) = INIT(I)
   I = I + 1
END DO

TYPE(C_PTR) :: PLAN
PLAN = KML_FFT_PLAN_DFT_R2C(RANK, N, IN, OUT, KML_FFT_ESTIMATE)
CALL KML_FFT_EXECUTE_DFT_R2C(PLAN, IN, OUT)

CALL KML_FFT_DESTROY_PLAN(PLAN)
CALL KML_FFT_FREE(PIN)
CALL KML_FFT_FREE(POUT)

!
! OUT = /3.600000E+01, 0.000000E+00, -4.000000E+00, 4.000000E+00,
!       -4.000000E+00, 0.000000E+00, -1.600000E+01, 0.000000E+00,
!       0.000000E+00, 0.000000E+00, 0.000000E+00, 0.000000E+00/
!
```

6.4.4.4.2.2 kml_fft(f/h)_plan_dft_r2c_1d

建立单个连续数据序列1维R2C变换的plan。

 版权所有 © 华为技术有限公司

接口定义

C interface：

kml_fft_plan kml_fft_plan_dft_r2c_1d(int n, double *in, kml_fft_complex *out, unsigned flags);

kml_fftf_plan kml_fftf_plan_dft_r2c_1d(int n, float *in, kml_fftf_complex *out, unsigned flags);

kml_ffth_plan kml_ffth_plan_dft_r2c_1d(int n, __fp16 *in, kml_ffth_complex *out, unsigned flags);

Fortran interface：

RES = KML_FFT_PLAN_DFT_R2C_1D(N, IN, OUT, FLAGS);

RES = KML_FFTF_PLAN_DFT_R2C_1D(N, IN, OUT, FLAGS);

RES = KML_FFTH_PLAN_DFT_R2C_1D(N, IN, OUT, FLAGS);

返回值

函数返回一个kml_fft(f/h)_plan类型的结构体指针。将该对象作为参数传入**kml_fft(f/h)_execute**函数中，将对当前提供的输入in和输出out执行FFT变换。也可以通过将该对象作为参数传入 **kml_fft(f/h)_execute_dft_r2c**函数中以对新的输入in和输出out执行FFT变换。

如果函数返回非空指针，则表示plan创建成功，否则表示创建失败。

参数

参数名	数据类型	描述	输入/输出
n	int	输入FFT序列的大小，约束：n ≥ 1。	输入
in	• 双精度：double* • 单精度：float • 半精度：__fp16*	输入需要变换的数据。	输入
out	• 双精度：kml_fft_complex* • 单精度：kml_fftf_complex* • 半精度：kml_ffth_complex*	输出快速傅里叶变换后的数据。	输出

 版权所有 © 华为技术有限公司

参数名	数据类型	描述	输入/输出
flags	unsigned int	planning选项，描述ESTIMATE模式或PATIENT模式。 KML_FFT_ESTIMATE：ESTIMATE模式 KML_FFT_PATIENT：PATIENT模式	输入

依赖

C: "kfft.h"

Fortran: "kfft.h"

示例

C interface：

```
int n0 = 4;
double *in;
double init[4] = {1, 2, 3, 4};
in = (double*)kml_fft_malloc(sizeof(double) * n0);
for (int i = 0; i < n0; i++) {
    in[i] = init[i];
}
kml_fft_complex *out;
out = (kml_fft_complex*)kml_fft_malloc(sizeof(kml_fft_complex) * (n0 / 2 + 1));
kml_fft_plan plan;
plan = kml_fft_plan_dft_r2c_1d(n0, in, out, KML_FFT_ESTIMATE);
kml_fft_execute_dft_r2c(plan, in, out);

kml_fft_destroy_plan(plan);
kml_fft_free(in);
kml_fft_free(out);

/*
 * out = {{1.000000e+01, 0.000000e+00}, {-2.000000e+00, 2.000000e+00},
 *        {-2.000000e+00, 0.000000e+00}}
 */
```

Fortran interface：

```
INTEGER(C_INT) :: N0 = 4
REAL(C_DOUBLE) :: INIT(4)
REAL(C_DOUBLE), POINTER :: IN(:)
TYPE(KML_FFT_COMPLEX), POINTER :: OUT(:)
TYPE(C_PTR) :: PIN, POUT
INTEGER(C_SIZE_T) :: RSIZE, CSIZE
RSIZE = N0
CSIZE = N0 / 2 + 1
IN = KML_FFT_MALLOC(8 * RSIZE)
OUT = KML_FFT_MALLOC(16 * CSIZE)
CALL C_F_POINTER(PIN, IN, SHAPE=[RSIZE])
CALL C_F_POINTER(POUT, OUT, SHAPE=[CSIZE])
DATA INIT/1, 2, 3, 4/
INTEGER :: I = 1
DO WHILE(I <= RSIZE)
    IN(I) = INIT(I)
    I = I + 1
END DO
```


版权所有 © 华为技术有限公司

```
TYPE(C_PTR) :: PLAN
PLAN = KML_FFT_PLAN_DFT_R2C_1D(N0, IN, OUT, KML_FFT_ESTIMATE)
CALL KML_FFT_EXECUTE_DFT_R2C(PLAN, IN, OUT)

CALL KML_FFT_DESTROY_PLAN(PLAN)
CALL KML_FFT_FREE(PIN)
CALL KML_FFT_FREE(POUT)

!
! OUT =/1.000000E+01, 0.000000E+00, -2.000000E+00, 2.000000E+00,
!       -2.000000E+00, 0.000000E+00/
!
```

6.4.4.4.2.3 kml_fft(f/h)_plan_dft_r2c_2d

建立单个连续数据序列2维R2C变换的plan。

接口定义

C interface：

kml_fft_plan kml_fft_plan_dft_r2c_2d(int n0, int n1, double *in, kml_fft_complex *out, unsigned flags);

kml_fftf_plan kml_fftf_plan_dft_r2c_2d(int n0, int n1, float *in, kml_fftf_complex *out, unsigned flags);

kml_ffth_plan kml_ffth_plan_dft_r2c_2d(int n0, int n1, __fp16 *in, kml_ffth_complex *out, unsigned flags);

Fortran interface：

RES = KML_FFT_PLAN_DFT_R2C_2D(N0, N1, IN, OUT, FLAGS);

RES = KML_FFTF_PLAN_DFT_R2C_2D(N0, N1, IN, OUT, FLAGS);

RES = KML_FFTH_PLAN_DFT_R2C_2D(N0, N1, IN, OUT, FLAGS);

返回值

函数返回一个kml_fft(f/h)_plan类型的结构体指针。将该对象作为参数传入**kml_fft(f/h)_execute**函数中，将对当前提供的输入in和输出out执行FFT变换。也可以通过将该对象作为参数传入 **kml_fft(f/h)_execute_dft_r2c**函数中以对新的输入in和输出out执行FFT变换。

如果函数返回非空指针，则表示plan创建成功，否则表示创建失败。

参数

参数名	数据类型	描述	输入/输出
n0	int	输入FFT序列第一维的大小，约束：n0 ≥ 1。	输入
n1	int	输入FFT序列第二维的大小，约束：n1 ≥ 1。	输入

版权所有 © 华为技术有限公司

参数名	数据类型	描述	输入/输出
in	• 双精度：double* • 单精度：float* • 半精度：__fp16*	输入需要变换的数据。	输入
out	• 双精度：kml_fft_complex* • 单精度：kml_fftf_complex* • 半精度：kml_ffth_complex*	输出快速傅里叶变换后的数据。	输出
flags	unsigned int	planning选项，描述ESTIMATE模式或PATIENT模式。 KML_FFT_ESTIMATE：ESTIMATE模式 KML_FFT_PATIENT：PATIENT模式	输入

依赖

C: "kfft.h"

示例

C interface：

```
int n0 = 2;
int n1 = 4;
double init[8] = {1, 2, 3, 4, 5, 6, 7, 8};
double *in;
in = (double*)kml_fft_malloc(sizeof(double) * n0 * n1);
for (int i = 0; i < n0 * n1; i++) {
    in[i] = init[i];
}
kml_fft_complex *out;
out = (kml_fft_complex*)kml_fft_malloc(sizeof(kml_fft_complex) * n0 * (n1 / 2 + 1));
kml_fft_plan plan;
plan = kml_fft_plan_dft_r2c_2d(n0, n1, in, out, KML_FFT_ESTIMATE);
kml_fft_execute_dft_r2c(plan, in, out);

kml_fft_destroy_plan(plan);
kml_fft_free(in);
kml_fft_free(out);

/*
 * out = {{3.600000e+01, 0.000000e+00}, {-4.000000e+00, 4.000000e+00},
 *        {-4.000000e+00, 0.000000e+00}, {-1.600000e+01, 0.000000e+00},
 *        {0.000000e+00, 0.000000e+00}, {0.000000e+00, 0.000000e+00}}
 */
```

 版权所有 © 华为技术有限公司

Fortran interface：

```
INTEGER(C_INT) :: N0 = 2
INTEGER(C_INT) :: N1 = 4
REAL(C_DOUBLE) :: INIT(8)
REAL(C_DOUBLE), POINTER :: IN
TYPE(KML_FFT_COMPLEX), POINTER :: OUT
TYPE(C_PTR) :: PIN, POUT
INTEGER(C_SIZE_T) :: RSIZE, CSIZE
RSIZE = N0 * N1
CSIZE = N0 * (N1 / 2 + 1)
PIN = KML_FFT_MALLOC(8 * RSIZE)
POUT = KML_FFT_MALLOC(16 * CSIZE)
CALL C_F_POINTER(PIN, IN, SHAPE=[RSIZE])
CALL C_F_POINTER(POUT, OUT, SHAPE=[CSIZE])
DATA N/2, 4/
DATA INIT/1, 2, 3, 4, 5, 6, 7, 8/
INTEGER :: I = 1
DO WHILE(I <= RSIZE)
   IN(I) = INIT(I)
   I = I + 1
END DO

TYPE(C_PTR) :: PLAN
PLAN = KML_FFT_PLAN_DFT_R2C_2D(N0, N1, IN, OUT, KML_FFT_ESTIMATE)
CALL KML_FFT_EXECUTE_DFT_R2C(PLAN, IN, OUT)

CALL KML_FFT_DESTROY_PLAN(PLAN)
CALL KML_FFT_FREE(PIN)
CALL KML_FFT_FREE(POUT)

!
! OUT = /3.600000E+01, 0.000000E+00, -4.000000E+00, 4.000000E+00,
!       -4.000000E+00, 0.000000E+00, -1.600000E+01, 0.000000E+00,
!       0.000000E+00, 0.000000E+00, 0.000000E+00, 0.000000E+00/
!
```

6.4.4.4.2.4 kml_fft(f/h)_plan_dft_r2c_3d

建立单个连续数据序列3维R2C变换的plan。

接口定义

C interface：

kml_fft_plan kml_fft_plan_dft_r2c_3d(int n0, int n1, int n2, double *in, kml_fft_complex *out, unsigned flags);

kml_fftf_plan kml_fftf_plan_dft_r2c_3d(int n0, int n1, int n2, float *in, kml_fftf_complex *out, unsigned flags);

kml_ffth_plan kml_ffth_plan_dft_r2c_3d(int n0, int n1, int n2, __fp16 *in, kml_ffth_complex *out, unsigned flags);

Fortran interface：

RES = KML_FFT_PLAN_DFT_R2C_3D(N0, N1, N2, IN, OUT, FLAGS);

RES = KML_FFTF_PLAN_DFT_R2C_3D(N0, N1, N2, IN, OUT, FLAGS);

RES = KML_FFTH_PLAN_DFT_R2C_3D(N0, N1, N2, IN, OUT, FLAGS);

返回值

函数返回一个kml_fft(f/h)_plan类型的结构体指针。将该对象作为参数传入**kml_fft(f/h)_execute**函数中，将对当前提供的输入in和输出out执行FFT变换。也可

 版权所有 © 华为技术有限公司

以通过将该对象作为参数传入 **kml_fft(f/h)_execute_dft_r2c**函数中以对新的输入in和输出out执行FFT变换。

如果函数返回非空指针，则表示plan创建成功，否则表示创建失败。

参数

参数名	数据类型	描述	输入/输出
n0	int	输入FFT序列第一维的大小，约束：n0 ≥ 1。	输入
n1	int	输入FFT序列第二维的大小，约束：n1 ≥ 1。	输入
n2	int	输入FFT序列第三维的大小，约束：n2 ≥ 1。	输入
in	• 双精度：double* • 单精度：float* • 半精度：__fp16*	输入需要变换的数据。	输入
out	• 双精度：kml_fft_complex* • 单精度：kml_fftf_complex* • 半精度：kml_ffth_complex*	输出快速傅里叶变换后的数据。	输出
flags	unsigned int	planning选项，描述ESTIMATE模式或PATIENT模式。 KML_FFT_ESTIMATE：ESTIMATE模式 KML_FFT_PATIENT：PATIENT模式	输入

依赖

C: "kfft.h"

示例

C interface：

```
int n0 = 2;
int n1 = 3;
int n2 = 2;
double init[12] = {120, 8, 0, 0, 0, -8, -8, -8, -16, 0, -40, -8};
double *in;
```

 版权所有 © 华为技术有限公司

```
in = (double*)kml_fft_malloc(sizeof(double) * n0 * n1 * n2);
for (int i = 0; i < n0 * n1 * n2; i++) {
    in[i] = init[i];
}
kml_fft_complex *out;
out = (kml_fft_complex*)kml_fft_malloc(sizeof(kml_fft_complex) * n0 * n1 * (n2 / 2 + 1));
kml_fft_plan plan;
plan = kml_fft_plan_dft_r2c_3d(n0, n1, n2, in, out, KML_FFT_ESTIMATE);
kml_fft_execute_dft_r2c(plan, in, out);

kml_fft_destroy_plan(plan);
kml_fft_free(in);
kml_fft_free(out);

/*
 * out = {{4.000000e+01, 0.000000e+00}, {7.200000e+01, 0.000000e+00},
 *        {1.480000e+02, -3.464102e+01}, {1.320000e+02, -6.928203e+00},
 *        {1.480000e+02, 3.464102e+01}, {1.320000e+02, 6.928203e+00},
 *        {2.000000e+02, 0.000000e+00}, {1.680000e+02, 0.000000e+00},
 *        {1.160000e+02, 2.078461e+01}, {8.400000e+01, 2.078461e+01},
 *        {1.160000e+02, -2.078461e+01}, {8.400000e+01, -2.078461e+01}}
 */
```

Fortran interface：

```
INTEGER(C_INT) :: N0 = 2
INTEGER(C_INT) :: N1 = 3
INTEGER(C_INT) :: N2 = 2
REAL(C_DOUBLE) :: INIT(12)
REAL(C_DOUBLE), POINTER :: IN
TYPE(KML_FFT_COMPLEX), POINTER :: OUT
TYPE(C_PTR) :: PIN, POUT
INTEGER(C_SIZE_T) :: RSIZE, CSIZE
RSIZE = N0 * N1 * N2
CSIZE = N0 * N1 * (N2 / 2 + 1)
PIN = KML_FFT_MALLOC(8 * RSIZE)
POUT = KML_FFT_MALLOC(16 * CSIZE)
CALL C_F_POINTER(PIN, IN, SHAPE=[RSIZE])
CALL C_F_POINTER(POUT, OUT, SHAPE=[CSIZE])
DATA INIT/120, 8, 0, 0, 0, -8, -8, -8, -16, 0, -40, -8/
INTEGER :: I = 1
DO WHILE(I <= RSIZE)
   IN(I) = INIT(I)
   I = I + 1
END DO

TYPE(C_PTR) :: PLAN
PLAN = KML_FFT_PLAN_DFT_R2C_3D(N0, N1, N2, IN, OUT, KML_FFT_ESTIMATE)
CALL KML_FFT_EXECUTE_DFT_R2C(PLAN, IN, OUT)

CALL KML_FFT_DESTROY_PLAN(PLAN)
CALL KML_FFT_FREE(PIN)
CALL KML_FFT_FREE(POUT)
!
! OUT = /4.000000E+01, 0.000000E+00, 7.200000E+01, 0.000000E+00,
!       1.480000E+02, -3.464102E+01, 1.320000E+02, -6.928203E+00,
!       1.480000E+02, 3.464102E+01, 1.320000E+02, 6.928203E+00,
!       2.000000E+02, 0.000000E+00, 1.680000E+02, 0.000000E+00,
!       1.160000E+02, 2.078461E+01, 8.400000E+01, 2.078461E+01,
!       1.160000E+02, -2.078461E+01, 8.400000E+01, -2.078461E+01/
!
```

6.4.4.4.2.5 kml_fft(f/h)_plan_guru_dft_r2c

建立多组数据序列n维R2C变换的plan。

其中，单个FFT的数据序列不需要是连续的，可以以跨步的形式提供。

 版权所有 © 华为技术有限公司

接口定义

C interface：

kml_fft_plan kml_fft_plan_guru_dft_r2c(int rank, const kml_fft_iodim *dims, int howmany_rank, const kml_fft_iodim *howmany_dims, double *in, kml_fft_complex *out, unsigned flags);

kml_fftf_plan kml_fftf_plan_guru_dft_r2c(int rank, const kml_fftf_iodim *dims, int howmany_rank, const kml_fftf_iodim *howmany_dims, float *in, kml_fftf_complex *out, unsigned flags);

kml_ffth_plan kml_ffth_plan_guru_dft_r2c(int rank, const kml_ffth_iodim *dims, int howmany_rank, const kml_ffth_iodim *howmany_dims, __fp16 *in, kml_ffth_complex *out, unsigned flags);

Fortran interface：

RES = KML_FFT_PLAN_GURU_DFT_R2C(RANK, DIMS, HOWMANY_RANK, HOWMANY_DIMS, IN, OUT, FLAGS);

RES = KML_FFTF_PLAN_GURU_DFT_R2C(RANK, DIMS, HOWMANY_RANK, HOWMANY_DIMS, IN, OUT, FLAGS);

RES = KML_FFTH_PLAN_GURU_DFT_R2C(RANK, DIMS, HOWMANY_RANK, HOWMANY_DIMS, IN, OUT, FLAGS);

返回值

函数返回一个kml_fft(f/h)_plan类型的结构体指针。将该对象作为参数传入**kml_fft(f/h)_execute**函数中，将对当前提供的输入in和输出out执行FFT变换。也可以通过将该对象作为参数传入 **kml_fft(f/h)_execute_dft_r2c**函数中以对新的输入in和输出out执行FFT变换。

如果函数返回非空指针，则表示plan创建成功，否则表示创建失败。

参数

参数名	数据类型	描述	输入/输出
rank	int	单个FFT序列的维度 约束：1 ≤ rank ≤ 4	输入
dims	• 双精度：const kml_fft_iodim* • 单精度：const kml_fftf_iodim* • 半精度：const kml_ffth_iodim*	dims是大小为rank的结构体数组，dims[i]包含以下成员： • int n - 第i维FFT的长度。 • int is - 第i维FFT输入序列的相继元素之间的间隔。 • int os - 第i维FFT输出序列的相继元素之间的间隔。 约束：dims[i].n ≥ 1, for i in 0 to rank - 1。	输入

 版权所有 © 华为技术有限公司

参数名	数据类型	描述	输入/输出
howmany_rank	int	多个rank维FFT之间的内存排布用howmany_rank维的howmany_dims数组来描述，howmany_rank表示每个要计算的rank维FFT变换的起始地址的内存访问模式所需的维数。 约束：0 ≤ howmany_rank ≤ 4。	输入
howmany_dims	• 双精度：const kml_fft_iodim* • 单精度：const kml_fftf_iodim* • 半精度：const kml_ffth_iodim*	howmany_dims是大小为howmany_rank的结构体数组，howmany_dims[i]包含以下成员： • int n - howmany_rank维空间的第i维的待变换FFT的个数。 • int is - 第i维的相继FFT输入序列之间的间隔。 • int os - 第i维的相继FFT输出序列之间的间隔。	输入
in	• 双精度：double* • 单精度：float* • 半精度：__fp16*	输入需要变换的数据。	输入
out	• 双精度：kml_fft_complex* • 单精度：kml_fftf_complex* • 半精度：kml_ffth_complex*	输出快速傅里叶变换后的数据。	输出
flags	unsigned int	planning选项，描述ESTIMATE模式或PATIENT模式。 KML_FFT_ESTIMATE：ESTIMATE模式 KML_FFT_PATIENT：PATIENT模式	输入

依赖

C: "kfft.h"

示例

C interface：

 版权所有 © 华为技术有限公司

```
int rank = 2;
kml_fft_iodim *dims;
dims = (kml_fft_iodim*)kml_fft_malloc(sizeof(kml_fft_iodim) * rank);
dims[0].n = 2;
dims[0].is = 3;
dims[0].os = 2;
dims[1].n = 3;
dims[1].is = 1;
dims[1].os = 1;
int howmany_rank = 1;
kml_fft_iodim *howmany_dims;
howmany_dims = (kml_fft_iodim*)kml_fft_malloc(sizeof(kml_fft_iodim) * howmany_rank);
howmany_dims[0].n = 2;
howmany_dims[0].is = 2 * 3;
howmany_dims[0].os = 2 * 2;
double init[12] = {120, 8, 0, 0, 0, -8, -8, -8, -16, 0, -40, -8};
double *in;
in = (double*)kml_fft_malloc(sizeof(double) * 12);
for (int i = 0; i < 12; i++) {
    in[i] = init[i];
}
kml_fft_complex *out;
out = (kml_fft_complex*)kml_fft_malloc(sizeof(kml_fft_complex) * 8);
kml_fft_plan plan;
plan = kml_fft_plan_guru_dft_r2c(rank, dims, howmany_rank, howmany_dims, in, out,
KML_FFT_ESTIMATE);
kml_fft_execute_dft_r2c(plan, in, out);

kml_fft_destroy_plan(plan);
kml_fft_free(howmany_dims);
kml_fft_free(dims);
kml_fft_free(in);
kml_fft_free(out);

/*
 * out = {{1.200000e+02, 0.000000e+00}, {1.200000e+02, -1.385641e+01},
 *        {1.360000e+02, 0.000000e+00}, {1.120000e+02, 0.000000e+00},
 *        {-8.000000e+01, 0.000000e+00}, {2.800000e+01, 2.078461e+01},
 *        {1.600000e+01, 0.000000e+00}, {-2.000000e+01, -3.464102e+01}}
 */
```

Fortran interface:

```
INTEGER(C_INT) :: RANK = 2
INTEGER(C_INT) :: HOWMANY_RANK = 1
TYPE(KML_FFT_IODIM), POINTER :: DIMS(:), HOWMANY_DIMS(:)
REAL(C_DOUBLE), DIMENSION(12) :: INIT
TYPE(C_DOUBLE), POINTER :: IN(:)
TYPE(KML_FFT_COMPLEX), POINTER :: OUT(:)
TYPE(C_PTR) :: PIN, POUT, PDIMS, PHOWMANY_DIMS
INTEGER(C_SIZE_T) :: SIZE1, SIZE2, SIZE3, SIZE4
SIZE1 = 8 * 12
SIZE2 = 16 * 8
SIZE3 = 12 * RANK
SIZE4 = 12 * HOWMANY_RANK
PDIMS = KML_FFT_MALLOC(SIZE3)
PHOWMANY_DIMS = KML_FFT_MALLOC(SIZE4)
PIN = KML_FFT_MALLOC(SIZE1)
POUT = KML_FFT_MALLOC(SIZE2)
CALL C_F_POINTER(PIN, IN, SHAPE=[12])
CALL C_F_POINTER(POUT, OUT, SHAPE=[8])
CALL C_F_POINTER(PDIMS, DIMS, SHAPE=[RANK])
CALL C_F_POINTER(PHOWMANY_DIMS, HOWMANY_DIMS, SHAPE=[HOWMANY_RANK])

DIMS(0)%N = 2
DIMS(0)%IS = 3
DIMS(0)%OS = 2
DIMS(1)%N = 3
DIMS(1)%IS = 1
DIMS(1)%OS = 1
```

 版权所有 © 华为技术有限公司

```
  HOWMANY_DIMS(0)%N = 2
  HOWMANY_DIMS(0)%IS = 2 * 3
  HOWMANY_DIMS(0)%OS = 2 * 3
  DATA INIT/120, 8, 0, 0, 0, -8, -8, -8, -16, 0, -40, -8/
  INTEGER :: I = 1
  DO WHILE(I <= 12)
     IN(I) = INIT(I)
     I = I + 1
  END DO
  TYPE(C_PTR) :: PLAN
  PLAN = KML_FFT_PLAN_GURU_DFT_R2C(RANK, DIMS, HOWMANY_RANK, HOWMANY_DIMS, IN, OUT,
KML_FFT_ESTIMATE)
  CALL KML_FFT_EXECUTE_DFT_R2C(PLAN, IN, OUT)

  CALL KML_FFT_DESTROY_PLAN(PLAN)
  CALL KML_FFT_FREE(PHOWMANY_DIMS)
  CALL KML_FFT_FREE(PDIMS)
  CALL KML_FFT_FREE(PIN)
  CALL KML_FFT_FREE(POUT)
  !
  ! OUT = /1.200000E+02, 0.000000E+00, 1.200000E+02, -1.385641E+01,
  !       1.360000E+02, 0.000000E+00, 1.120000E+02, 0.000000E+00,
  !       -8.000000E+01, 0.000000E+00, 2.800000E+01, 2.078461E+01,
  !       1.600000E+01, 0.000000E+00, -2.000000E+01, -3.464102E+01/
  !
```

6.4.4.4.2.6 kml_fft(f/h)_plan_guru_split_dft_r2c

建立多组数据序列n维R2C变换的plan。

其中，单个FFT的数据序列不需要是连续的，可以以跨步的形式提供。split类接口的输入和输出分别存储在实部和虚部数组中。

接口定义

C interface：

kml_fft_plan kml_fft_plan_guru_split_dft_r2c(int rank, const kml_fft_iodim *dims, int howmany_rank, const kml_fft_iodim *howmany_dims, double *in, double *ro, double *io, unsigned flags);

kml_fftf_plan kml_fftf_plan_guru_split_dft_r2c(int rank, const kml_fftf_iodim *dims, int howmany_rank, const kml_fftf_iodim *howmany_dims, float *in, float *ro, float *io, unsigned flags);

kml_ffth_plan kml_ffth_plan_guru_split_dft_r2c(int rank, const kml_ffth_iodim *dims, int howmany_rank, const kml_ffth_iodim *howmany_dims, __fp16 *in, __fp16 *ro, __fp16 *io, unsigned flags);

Fortran interface：

RES = KML_FFT_PLAN_GURU_SPLIT_DFT_R2C(RANK, DIMS, HOWMANY_RANK, HOWMANY_DIMS, IN, RO, IO, FLAGS);

RES = KML_FFTF_PLAN_GURU_SPLIT_DFT_R2C(RANK, DIMS, HOWMANY_RANK, HOWMANY_DIMS, IN, RO, IO, FLAGS);

RES = KML_FFTH_PLAN_GURU_SPLIT_DFT_R2C(RANK, DIMS, HOWMANY_RANK, HOWMANY_DIMS, IN, RO, IO, FLAGS);

返回值

函数返回一个kml_fft(f/h)_plan类型的结构体指针。将该对象作为参数传入**kml_fft(f/h)_execute**函数中，将对当前提供的输入in和输出ro，io执行FFT变换。也

 版权所有 © 华为技术有限公司

可以通过将该对象作为参数传入 **kml_fft(f/h)_execute_split_dft_r2c**函数中以对新的输入in和输出ro，io执行FFT变换。

如果函数返回非空指针，则表示plan创建成功，否则表示创建失败。

参数

参数名	数据类型	描述	输入/输出
rank	int	单个FFT序列的维度。 约束：1 ≤ rank ≤ 4。	输入
dims	• 双精度：const kml_fft_iodim* • 单精度：const kml_fftf_iodim* • 半精度：const kml_ffth_iodim*	dims是大小为rank的结构体数组，dims[i]包含以下成员： • int n - 第i维FFT的长度。 • int is - 第i维FFT输入序列的相继元素之间的间隔。 • int os - 第i维FFT输出序列的相继元素之间的间隔。 约束：dims[i].n ≥ 1, for i in 0 to rank - 1。	输入
howmany_rank	int	多个rank维FFT之间的内存排布用howmany_rank维的howmany_dims数组来描述，howmany_rank表示每个要计算的rank维FFT变换的起始地址的内存访问模式所需的维数。 约束：0 ≤ howmany_rank ≤ 4。	输入
howmany_dims	• 双精度：const kml_fft_iodim* • 单精度：const kml_fftf_iodim* • 半精度：const kml_ffth_iodim*	howmany_dims是大小为howmany_rank的结构体数组，howmany_dims[i]包含以下成员： • int n - howmany_rank维空间的第i维的待变换FFT的个数。 • int is - 第i维的相继FFT输入序列之间的间隔。 • int os - 第i维的相继FFT输出序列之间的间隔。	输入
in	• 双精度：double* • 单精度：float* • 半精度：__fp16*	输入待变换数据。	输入
ro	• 双精度：double* • 单精度：float* • 半精度：__fp16*	输出待变换数据的实部。	输入

版权所有 © 华为技术有限公司

参数名	数据类型	描述	输入/输出
io	• 双精度：double* • 单精度：float* • 半精度：__fp16*	输出待变换数据的虚部。	输入
flags	unsigned int	planning选项，描述ESTIMATE模式或PATIENT模式。 KML_FFT_ESTIMATE：ESTIMATE模式 KML_FFT_PATIENT：PATIENT模式	输入

依赖

C: "kfft.h"

示例

C interface：

```
int rank = 2;
kml_fft_iodim *dims;
dims = (kml_fft_iodim*)kml_fft_malloc(sizeof(kml_fft_iodim) * rank);
dims[0].n = 2;
dims[0].is = 3;
dims[0].os = 2;
dims[1].n = 3;
dims[1].is = 1;
dims[1].os = 1;
int howmany_rank = 1;
kml_fft_iodim *howmany_dims;
howmany_dims = (kml_fft_iodim*)kml_fft_malloc(sizeof(kml_fft_iodim) * howmany_rank);
howmany_dims[0].n = 2;
howmany_dims[0].is = 2 * 3;
howmany_dims[0].os = 2 * 2;
double init[12] = {120, 8, 0, 0, 0, -8, -8, -8, -16, 0, -40, -8};
double *in;
in = (double*)kml_fft_malloc(sizeof(double) * 12);
for (int i = 0; i < 12; i++) {
    in[i] = init[i];
}
kml_fft_plan plan;
double *ro;
double *io;
ro = (double*)kml_fft_malloc(sizeof(double) * 8);
io = (double*)kml_fft_malloc(sizeof(double) * 8);
plan = kml_fft_plan_guru_split_dft_r2c(rank, dims, howmany_rank, howmany_dims, in, ro, io,
KML_FFT_ESTIMATE);
kml_fft_execute_split_dft_r2c(plan, in, ro, io);

kml_fft_destroy_plan(plan);
kml_fft_free(howmany_dims);
kml_fft_free(dims);
kml_fft_free(in);
kml_fft_free(ro);
kml_fft_free(io);
```

 版权所有 © 华为技术有限公司

```
/*
 * ro = {1.200000e+02, 1.200000e+02, 1.360000e+02, 1.120000e+02,
 *       -8.000000e+01, 2.800000e+01, 1.600000e+01, -2.000000e+01}
 */

/*
 * io = {0.000000e+00, -1.385641e+01, 0.000000e+00, 0.000000e+00,
 *       0.000000e+00, 2.078461e+01, 0.000000e+00, -3.464102e+01}
 */
```

Fortran interface:

```
INTEGER(C_INT) :: RANK = 2
INTEGER(C_INT) :: HOWMANY_RANK = 1
TYPE(KML_FFT_IODIM), POINTER :: DIMS(:), HOWMANY_DIMS(:)
REAL(C_DOUBLE), DIMENSION(12) :: INIT
TYPE(C_DOUBLE), POINTER :: IN(:), RO(:), IO(:)
TYPE(C_PTR) :: PIN, PRO, PIO, PDIMS, PHOWMANY_DIMS
INTEGER(C_SIZE_T) :: SIZE1, SIZE2, SIZE3, SIZE4
SIZE1 = 8 * 12
SIZE2 = 8 * 8
SIZE3 = 12 * RANK
SIZE4 = 12 * HOWMANY_RANK
PDIMS = KML_FFT_MALLOC(SIZE3)
PHOWMANY_DIMS = KML_FFT_MALLOC(SIZE4)
PIN = KML_FFT_MALLOC(SIZE1)
PRO = KML_FFT_MALLOC(SIZE2)
PRI = KML_FFT_MALLOC(SIZE2)
CALL C_F_POINTER(PIN, IN, SHAPE=[12])
CALL C_F_POINTER(PRO, RO, SHAPE=[8])
CALL C_F_POINTER(PIO, IO, SHAPE=[8])
CALL C_F_POINTER(PDIMS, DIMS, SHAPE=[RANK])
CALL C_F_POINTER(PHOWMANY_DIMS, HOWMANY_DIMS, SHAPE=[HOWMANY_RANK])

DIMS(0)%N = 2
DIMS(0)%IS = 3
DIMS(0)%OS = 2
DIMS(1)%N = 3
DIMS(1)%IS = 1
DIMS(1)%OS = 1
HOWMANY_DIMS(0)%N = 2
HOWMANY_DIMS(0)%IS = 2 * 3
HOWMANY_DIMS(0)%OS = 2 * 3
DATA INIT/120, 8, 0, 0, 0, -8, -8, -8, -16, 0, -40, -8/
INTEGER :: I
DO WHILE(I <= 12)
   IN(I) = INIT(I)
   I = I + 1
END DO
TYPE(C_PTR) :: PLAN
PLAN = KML_FFT_PLAN_GURU_SPLIT_DFT_R2C(RANK, DIMS, HOWMANY_RANK, HOWMANY_DIMS, IN,
RO, IO, KML_FFT_ESTIMATE)
CALL KML_FFT_EXECUTE_SPLIT_DFT_R2C(PLAN, IN, RO, IO)

CALL KML_FFT_DESTROY_PLAN(PLAN)
CALL KML_FFT_FREE(PHOWMANY_DIMS)
CALL KML_FFT_FREE(PDIMS)
CALL KML_FFT_FREE(PIN)
CALL KML_FFT_FREE(PRO)
CALL KML_FFT_FREE(PIO)
!
! RO = /1.200000E+02, 1.200000E+02, 1.360000E+02, 1.120000E+02,
!       -8.000000E+01, 2.800000E+01, 1.600000E+01, -2.000000E+01/
!

!
! IO = /0.000000E+00, -1.385641E+01, 0.000000E+00, 0.000000E+00,
!       0.000000E+00, 2.078461E+01, 0.000000E+00, -3.464102E+01/
!
```

 版权所有 © 华为技术有限公司

⚠ 注意

kml_fft(f)_plan_guru_split_dft_r2c返回的plan可以使用kml_fft(f)_execute计算plan时的输入，也可以使用kml_fft(f)_execute_split_dft_r2c计算不同的输入。plan时的输入不需要初始化，但是输入必须是合法的（不能为NULL）。

6.4.4.4.2.7 kml_fft(f/h)_plan_guru64_dft_r2c

建立多组数据序列n维R2C变换的plan。其中，单个FFT的数据序列不需要是连续的，可以以跨步的形式提供。与kml_fft(f)_plan_guru_dft不同的是，kml_fft(f)_plan_guru64_dft允许部分参数是64位的整型。

接口定义

C interface：

kml_fft_plan kml_fft_plan_guru64_dft_r2c(int rank, const kml_fft_iodim64 *dims, int howmany_rank, const kml_fft_iodim64 *howmany_dims, double *in, kml_fft_complex *out, unsigned flags);

kml_fftf_plan kml_fftf_plan_guru64_dft_r2c(int rank, const kml_fftf_iodim64 *dims, int howmany_rank, const kml_fftf_iodim64 *howmany_dims, float *in, kml_fftf_complex *out, unsigned flags);

kml_ffth_plan kml_ffth_plan_guru64_dft_r2c(int rank, const kml_ffth_iodim64 *dims, int howmany_rank, const kml_ffth_iodim64 *howmany_dims, __fp16 *in, kml_ffth_complex *out, unsigned flags);

Fortran interface：

RES = KML_FFT_PLAN_GURU64_SPLIT_DFT_R2C(RANK, DIMS, HOWMANY_RANK, HOWMANY_DIMS, IN, RO, IO, FLAGS);

RES = KML_FFTF_PLAN_GURU64_SPLIT_DFT_R2C(RANK, DIMS, HOWMANY_RANK, HOWMANY_DIMS, IN, RO, IO, FLAGS);

RES = KML_FFTH_PLAN_GURU64_SPLIT_DFT_R2C(RANK, DIMS, HOWMANY_RANK, HOWMANY_DIMS, IN, RO, IO, FLAGS);

返回值

函数返回一个kml_fft(f/h)_plan类型的结构体指针。将该对象作为参数传入**kml_fft(f/h)_execute**函数中，将对当前提供的输入in和输出out执行FFT变换。也可以通过将该对象作为参数传入 **kml_fft(f/h)_execute_dft_r2c**函数中以对新的输入in和输出out执行FFT变换。

如果函数返回非空指针，则表示plan创建成功，否则表示创建失败。

 版权所有 © 华为技术有限公司

参数

参数名	数据类型	描述	输入/输出
rank	int	单个FFT序列的维度。 约束：1 ≤ rank ≤ 4。	输入
dims	• 双精度：const kml_fft_iodim64 * • 单精度：const kml_fftf_iodim64* • 半精度：const kml_ffth_iodim64*	dims是大小为rank的结构体数组，dims[i]包含以下成员： • ptrdiff_t n - 第i维FFT的长度。 • ptrdiff_t is - 第i维FFT输入序列的相继元素之间的间隔。 • ptrdiff_t os - 第i维FFT输出序列的相继元素之间的间隔。 约束：dims[i].n ≥ 1, for i in 0 to rank - 1。	输入
howmany_rank	int	多个rank维FFT之间的内存排布用howmany_rank维的howmany_dims数组来描述，howmany_rank表示每个要计算的rank维FFT变换的起始地址的内存访问模式所需的维数。 约束：0 ≤ howmany_rank ≤ 4。	输入
howmany_dims	• 双精度：const kml_fft_iodim64 * • 单精度：const kml_fftf_iodim64* • 半精度：const kml_ffth_iodim64*	howmany_dims是大小为howmany_rank的结构体数组，howmany_dims[i]包含以下成员： • ptrdiff_t n - howmany_rank维空间的第i维的待变换FFT的个数。 • ptrdiff_t is - 第i维的相继FFT输入序列之间的间隔。 • ptrdiff_t os - 第i维的相继FFT输出序列之间的间隔。	输入
in	• 双精度：double* • 单精度：float* • 半精度：__fp16*	输入需要变换的数据。	输入
out	• 双精度：kml_fft_complex* • 单精度：kml_fftf_complex* • 半精度：kml_ffth_complex*	输出快速傅里叶变换后的数据。	输出

版权所有 © 华为技术有限公司

参数名	数据类型	描述	输入/输出
flags	unsigned int	planning选项，描述ESTIMATE模式或PATIENT模式。 KML_FFT_ESTIMATE：ESTIMATE模式 KML_FFT_PATIENT：PATIENT模式	输入

依赖

C: "kfft.h"

示例

C interface：

```
   int rank = 2;
   kml_fft_iodim64 *dims;
   dims = (kml_fft_iodim64*)kml_fft_malloc(sizeof(kml_fft_iodim64) * rank);
   dims[0].n = 2;
   dims[0].is = 3;
   dims[0].os = 2;
   dims[1].n = 3;
   dims[1].is = 1;
   dims[1].os = 1;
   int howmany_rank = 1;
   kml_fft_iodim64 *howmany_dims;
   howmany_dims = (kml_fft_iodim64*)kml_fft_malloc(sizeof(kml_fft_iodim64) * howmany_rank);
   howmany_dims[0].n = 2;
   howmany_dims[0].is = 2 * 3;
   howmany_dims[0].os = 2 * 2;
   double init[12] = {120, 8, 0, 0, 0, -8, -8, -8, -16, 0, -40, -8};
   double *in;
   in = (double*)kml_fft_malloc(sizeof(double) * 12);
   for (int i = 0; i < 12; i++) {
       in[i] = init[i];
   }
   kml_fft_complex *out;
   out = (kml_fft_complex*)kml_fft_malloc(sizeof(kml_fft_complex) * 8);
   kml_fft_plan plan;
   plan = kml_fft_plan_guru64_dft_r2c(rank, dims, howmany_rank, howmany_dims, in, out,
KML_FFT_ESTIMATE);
   kml_fft_execute_dft_r2c(plan, in, out);

   kml_fft_destroy_plan(plan);
   kml_fft_free(howmany_dims);
   kml_fft_free(dims);
   kml_fft_free(in);
   kml_fft_free(out);

   /*
    * out = {{1.200000e+02, 0.000000e+00}, {1.200000e+02, -1.385641e+01},
    *        {1.360000e+02, 0.000000e+00}, {1.120000e+02, 0.000000e+00},
    *        {-8.000000e+01, 0.000000e+00}, {2.800000e+01, 2.078461e+01},
    *        {1.600000e+01, 0.000000e+00}, {-2.000000e+01, -3.464102e+01}}
    */
```

Fortran interface：

```
INTEGER(C_INT) :: RANK = 2
   INTEGER(C_INT) :: HOWMANY_RANK = 1
```

 版权所有 © 华为技术有限公司

```
  TYPE(KML_FFT_IODIM64), POINTER :: DIMS(:), HOWMANY_DIMS(:)
  REAL(C_DOUBLE), DIMENSION(12) :: INIT
  TYPE(C_DOUBLE), POINTER :: IN(:)
  TYPE(KML_FFT_COMPLEX), POINTER :: OUT(:)
  TYPE(C_PTR) :: PIN, POUT, PDIMS, PHOWMANY_DIMS
  INTEGER(C_SIZE_T) :: SIZE1, SIZE2, SIZE3, SIZE4
  SIZE1 = 8 * 12
  SIZE2 = 16 * 8
  SIZE3 = 24 * RANK
  SIZE4 = 24 * HOWMANY_RANK
  PDIMS = KML_FFT_MALLOC(SIZE3)
  PHOWMANY_DIMS = KML_FFT_MALLOC(SIZE4)
  PIN = KML_FFT_MALLOC(SIZE1)
  POUT = KML_FFT_MALLOC(SIZE2)
  CALL C_F_POINTER(PIN, IN, SHAPE=[12])
  CALL C_F_POINTER(POUT, OUT, SHAPE=[8])
  CALL C_F_POINTER(PDIMS, DIMS, SHAPE=[RANK])
  CALL C_F_POINTER(PHOWMANY_DIMS, HOWMANY_DIMS, SHAPE=[HOWMANY_RANK])

  DIMS(0)%N = 2
  DIMS(0)%IS = 3
  DIMS(0)%OS = 2
  DIMS(1)%N = 3
  DIMS(1)%IS = 1
  DIMS(1)%OS = 1
  HOWMANY_DIMS(0)%N = 2
  HOWMANY_DIMS(0)%IS = 2 * 3
  HOWMANY_DIMS(0)%OS = 2 * 3
  DATA INIT/120, 8, 0, 0, 0, -8, -8, -8, -16, 0, -40, -8/
  INTEGER :: I
  DO WHILE(I <= 12)
     IN(I) = INIT(I)
     I = I + 1
  END DO
  TYPE(C_PTR) :: PLAN
  PLAN = KML_FFT_PLAN_GURU64_DFT_R2C(RANK, DIMS, HOWMANY_RANK, HOWMANY_DIMS, IN,
OUT, KML_FFT_ESTIMATE)
  CALL KML_FFT_EXECUTE_DFT_R2C(PLAN, IN, OUT)

  CALL KML_FFT_DESTROY_PLAN(PLAN)
  CALL KML_FFT_FREE(PHOWMANY_DIMS)
  CALL KML_FFT_FREE(PDIMS)
  CALL KML_FFT_FREE(PIN)
  CALL KML_FFT_FREE(POUT)
  !
  ! OUT = /1.200000E+02, 0.000000E+00, 1.200000E+02, -1.385641E+01,
  !       1.360000E+02, 0.000000E+00, 1.120000E+02, 0.000000E+00,
  !       -8.000000E+01, 0.000000E+00, 2.800000E+01, 2.078461E+01,
  !       1.600000E+01, 0.000000E+00, -2.000000E+01, -3.464102E+01/
  !
```

6.4.4.4.2.8 kml_fft(f/h)_plan_guru64_split_dft_r2c

建立多组数据序列n维R2C变换的plan。其中，单个FFT的数据序列不需要是连续的，可以以跨步的形式提供。split类接口的输入和输出分别存储在实部和虚部数组中。与kml_fft_plan_guru_split_dft_r2c不同的是，kml_fft_plan_guru64_split_dft_r2c允许部分参数是64位的整型。

接口定义

C interface：

kml_fft_plan kml_fft_plan_guru64_split_dft_r2c(int rank, const kml_fft_iodim64 *dims, int howmany_rank, const kml_fft_iodim64 *howmany_dims, double *in, double *ro, double *io, unsigned flags);

 版权所有 © 华为技术有限公司

kml_fftf_plan kml_fftf_plan_guru64_split_dft_r2c(int rank, const kml_fftf_iodim64 *dims, int howmany_rank, const kml_fftf_iodim64 *howmany_dims, float *in, float *ro, float *io, unsigned flags);

kml_ffth_plan kml_ffth_plan_guru64_split_dft_r2c(int rank, const kml_ffth_iodim64 *dims, int howmany_rank, const kml_ffth_iodim64 *howmany_dims, __fp16*in, __fp16 *ro, __fp16 *io, unsigned flags);

Fortran interface：

RES = KML_FFT_PLAN_GURU64_SPLIT_DFT_R2C(RANK, DIMS, HOWMANY_RANK, HOWMANY_DIMS, IN, RO, IO, FLAGS);

RES = KML_FFTF_PLAN_GURU64_SPLIT_DFT_R2C(RANK, DIMS, HOWMANY_RANK, HOWMANY_DIMS, IN, RO, IO, FLAGS);

RES = KML_FFTH_PLAN_GURU64_SPLIT_DFT_R2C(RANK, DIMS, HOWMANY_RANK, HOWMANY_DIMS, IN, RO, IO, FLAGS);

返回值

函数返回一个kml_fft(f/h)_plan类型的结构体指针。将该对象作为参数传入**kml_fft(f/h)_execute**函数中，将对当前提供的输入in和输出ro，io执行FFT变换。也可以通过将该对象作为参数传入 **kml_fft(f/h)_execute_split_dft_r2c**函数中以对新的输入in和输出ro，io执行FFT变换。

如果函数返回非空指针，则表示plan创建成功，否则表示创建失败。

参数

参数名	数据类型	描述	输入/输出
rank	int	单个FFT序列的维度。 约束：1 ≤ rank ≤ 4。	输入
dims	• 单精度：const kml_fftf_iodim64 * • 双精度：const kml_fft_iodim64 * • 半精度：const kml_ffth_iodim64*	dims是大小为rank的结构体数组，dims[i]包含以下成员： • ptrdiff_t n - 第i维FFT的长度。 • ptrdiff_t is - 第i维FFT输入序列的相继元素之间的间隔。 • ptrdiff_t os - 第i维FFT输出序列的相继元素之间的间隔。 约束：dims[i].n ≥ 1, for i in 0 to rank - 1。	输入
howmany_rank	int	多个rank维FFT之间的内存排布用howmany_rank维的howmany_dims数组来描述，howmany_rank表示每个要计算的rank维FFT变换的起始地址的内存访问模式所需的维数。 约束：0 ≤ howmany_rank ≤ 4。	输入

版权所有 © 华为技术有限公司

参数名	数据类型	描述	输入/输出
howmany_dims	• 单精度：const kml_fftf_iodim64 * • 双精度：const kml_fft_iodim64 * • 半精度：const kml_ffth_iodim64*	howmany_dims是大小为howmany_rank的结构体数组，howmany_dims[i]包含以下成员： • ptrdiff_t n - howmany_rank维空间的第i维的待变换FFT的个数。 • ptrdiff_t is - 第i维的相继FFT输入序列之间的间隔。 • ptrdiff_t os - 第i维的相继FFT输出序列之间的间隔。	输入
in	• 双精度：double* • 单精度：float* • 半精度：__fp16*	输入待变换数据。	输入
ro	• 双精度：double* • 单精度：float* • 半精度：__fp16*	输出待变换数据的实部。	输入
io	• 双精度：double* • 单精度：float* • 半精度：__fp16*	输出待变换数据的虚部。	输入
flags	unsigned int	planning选项，描述ESTIMATE模式或PATIENT模式。 KML_FFT_ESTIMATE：ESTIMATE模式 KML_FFT_PATIENT：PATIENT模式	输入

依赖

C: "kfft.h"

示例

C interface：

```
int rank = 2;
kml_fft_iodim64 *dims;
dims = (kml_fft_iodim64*)kml_fft_malloc(sizeof(kml_fft_iodim64) * rank);
dims[0].n = 2;
dims[0].is = 3;
dims[0].os = 2;
dims[1].n = 3;
dims[1].is = 1;
dims[1].os = 1;
```

 版权所有 © 华为技术有限公司

```
int howmany_rank = 1;
kml_fft_iodim64 *howmany_dims;
howmany_dims = (kml_fft_iodim64*)kml_fft_malloc(sizeof(kml_fft_iodim64) * howmany_rank);
howmany_dims[0].n = 2;
howmany_dims[0].is = 2 * 3;
howmany_dims[0].os = 2 * 2;
double init[12] = {120, 8, 0, 0, 0, -8, -8, -8, -16, 0, -40, -8};
double *in;
in = (double*)kml_fft_malloc(sizeof(double) * 12);
for (int i = 0; i < 12; i++) {
    in[i] = init[i];
}
kml_fft_plan plan;
double *ro;
double *io;
ro = (double*)kml_fft_malloc(sizeof(double) * 8);
io = (double*)kml_fft_malloc(sizeof(double) * 8);
plan = kml_fft_plan_guru64_split_dft_r2c(rank, dims, howmany_rank, howmany_dims, in, ro, io,
KML_FFT_ESTIMATE);
kml_fft_execute_split_dft_r2c(plan, in, ro, io);

kml_fft_destroy_plan(plan);
kml_fft_free(howmany_dims);
kml_fft_free(dims);
kml_fft_free(in);
kml_fft_free(ro);
kml_fft_free(io);

/*
 * ro = {1.200000e+02, 1.200000e+02, 1.360000e+02, 1.120000e+02,
 *       -8.000000e+01, 2.800000e+01, 1.600000e+01, -2.000000e+01}
 */

/*
 * io = {0.000000e+00, -1.385641e+01, 0.000000e+00, 0.000000e+00,
 *       0.000000e+00, 2.078461e+01, 0.000000e+00, -3.464102e+01}
 */
```

Fortran interface：

```
INTEGER(C_INT) :: RANK = 2
INTEGER(C_INT) :: HOWMANY_RANK = 1
TYPE(KML_FFT_IODIM64), POINTER :: DIMS(:), HOWMANY_DIMS(:)
REAL(C_DOUBLE), DIMENSION(12) :: INIT
TYPE(C_DOUBLE), POINTER :: IN(:), RO(:), IO(:)
TYPE(C_PTR) :: PIN, PRO, PIO, PDIMS, PHOWMANY_DIMS
INTEGER(C_SIZE_T) :: SIZE1, SIZE2, SIZE3, SIZE4
SIZE1 = 8 * 12
SIZE2 = 8 * 8
SIZE3 = 24 * RANK
SIZE4 = 24 * HOWMANY_RANK
PDIMS = KML_FFT_MALLOC(SIZE3)
PHOWMANY_DIMS = KML_FFT_MALLOC(SIZE4)
PIN = KML_FFT_MALLOC(SIZE1)
PRO = KML_FFT_MALLOC(SIZE2)
PRI = KML_FFT_MALLOC(SIZE2)
CALL C_F_POINTER(PIN, IN, SHAPE=[12])
CALL C_F_POINTER(PRO, RO, SHAPE=[8])
CALL C_F_POINTER(PIO, IO, SHAPE=[8])
CALL C_F_POINTER(PDIMS, DIMS, SHAPE=[RANK])
CALL C_F_POINTER(PHOWMANY_DIMS, HOWMANY_DIMS, SHAPE=[HOWMANY_RANK])

DIMS(0)%N = 2
DIMS(0)%IS = 3
DIMS(0)%OS = 2
DIMS(1)%N = 3
DIMS(1)%IS = 1
DIMS(1)%OS = 1
HOWMANY_DIMS(0)%N = 2
HOWMANY_DIMS(0)%IS = 2 * 3
```

 版权所有 © 华为技术有限公司

```
  HOWMANY_DIMS(0)%OS = 2 * 3
  DATA INIT/120, 8, 0, 0, 0, -8, -8, -8, -16, 0, -40, -8/
  INTEGER :: I
  DO WHILE(I <= 12)
     IN(I) = INIT(I)
     I = I + 1
  END DO
  TYPE(C_PTR) :: PLAN
  PLAN = KML_FFT_PLAN_GURU64_SPLIT_DFT_R2C(RANK, DIMS, HOWMANY_RANK, HOWMANY_DIMS,
IN, RO, IO, KML_FFT_ESTIMATE)
  CALL KML_FFT_EXECUTE_SPLIT_DFT_R2C(PLAN, IN, RO, IO)

  CALL KML_FFT_DESTROY_PLAN(PLAN)
  CALL KML_FFT_FREE(PHOWMANY_DIMS)
  CALL KML_FFT_FREE(PDIMS)
  CALL KML_FFT_FREE(PIN)
  CALL KML_FFT_FREE(PRO)
  CALL KML_FFT_FREE(PIO)
  !
  ! RO = /1.200000E+02, 1.200000E+02, 1.360000E+02, 1.120000E+02,
  !       -8.000000E+01, 2.800000E+01, 1.600000E+01, -2.000000E+01/
  !

  !
  ! IO = /0.000000E+00, -1.385641E+01, 0.000000E+00, 0.000000E+00,
  !       0.000000E+00, 2.078461E+01, 0.000000E+00, -3.464102E+01/
  !
```

6.4.4.4.2.9 kml_fft(f/h)_plan_many_dft_r2c

建立howmany组数据序列n维R2C变换的plan。其中，单个FFT的数据序列不需要是连续的，可以以跨步的形式提供。

接口定义

C interface：

kml_fft_plan kml_fft_plan_many_dft_r2c(int rank, const int *n, int howmany, double *in, const int *inembed, int istride, int idist, kml_fft_complex *out, const int *onembed, int ostride, int odist, unsigned flags);

kml_fftf_plan kml_fftf_plan_many_dft_r2c(int rank, const int *n, int howmany, float *in, const int *inembed, int istride, int idist, kml_fftf_complex *out, const int *onembed, int ostride, int odist, unsigned flags);

kml_ffth_plan kml_ffth_plan_many_dft_r2c(int rank, const int *n, int howmany, __fp16 *in, const int *inembed, int istride, int idist, kml_ffth_complex *out, const int *onembed, int ostride, int odist, unsigned flags);

Fortran interface：

RES = KML_FFT_PLAN_MANY_DFT_R2C(RANK, N, HOWMANY, IN, INEMBED, ISTRIDE, IDIST, OUT, ONEMBED, OSTRIDE, ODIST, FLAGS);

RES = KML_FFTF_PLAN_MANY_DFT_R2C(RANK, N, HOWMANY, IN, INEMBED, ISTRIDE, IDIST, OUT, ONEMBED, OSTRIDE, ODIST, FLAGS);

RES = KML_FFTH_PLAN_MANY_DFT_R2C(RANK, N, HOWMANY, IN, INEMBED, ISTRIDE, IDIST, OUT, ONEMBED, OSTRIDE, ODIST, FLAGS);

返回值

函数返回一个kml_fft(f/h)_plan类型的结构体指针。将该对象作为参数传入**kml_fft(f/h)_execute**函数中，将对当前提供的输入in和输出out执行FFT变换。也可

 版权所有 © 华为技术有限公司

以通过将该对象作为参数传入 **kml_fft(f/h)_execute_dft_r2c**函数中以对新的输入in和输出out执行FFT变换。

如果函数返回非空指针，则表示plan创建成功，否则表示创建失败。

参数

参数名	数据类型	描述	输入/输出
rank	int	FFT变换的维度是rank，约束：1 ≤ rank ≤ 4。	输入
n	const int*	n为维度为rank的数组，包含FFT序列每一维度的大小，约束：n[i] ≥ 1, for i in 0 to rank - 1。	输入
howmany	int	howmany表示指定多少个多维FFT变换。	输入
in	• 双精度：double* • 单精度：float* • 半精度：__fp16*	输入需要变换的数据。	输入
inembed	const int*	inembed是大小为rank的数组或NULL，该数组表示FFT数据in存储的更大空间的每一维度的大小。 约束：inembed[i] ≥ n[i] for i in 0, rank-1 或者：如果inembed == NULL，则可以认为inembed与n相等	输入
istride	int	FFT输入序列的相继元素之间的间隔。	输入
idist	int	idist表示输入的每个FFT序列的间隔。	输入
out	• 双精度：kml_fft_complex* • 单精度：kml_fftf_complex* • 半精度：kml_ffth_complex*	输出快速傅里叶变换后的数据。	输出

版权所有 © 华为技术有限公司

参数名	数据类型	描述	输入/输出
onembed	const int*	onembed是大小为rank的数组或者NULL，该数组表示FFT数据out存储的更大空间的每一维度的大小。 约束：onembed[i] ≥ n[i] for i in 0, rank-1 或者：如果onembed == NULL，则可以认为onembed与n相等	输入
ostride	int	FFT输出序列的相继元素之间的间隔。	输入
odist	int	odist表示输出的每个FFT序列的间隔。	输入
flags	unsigned int	planning选项，描述ESTIMATE模式或PATIENT模式。 KML_FFT_ESTIMATE：ESTIMATE模式 KML_FFT_PATIENT：PATIENT模式	输入

依赖

C: "kfft.h"

示例

C interface：

```
   int rank = 1;
   int *n;
   n = (int*)kml_fft_malloc(sizeof(int) * rank);
   n[0] = 4;
   int howmany = 3;
   int istride = 1;
   int ostride = 1;
   int idist = 4;
   int odist = n[0] / 2 + 1;
   double init[12] = {120, 8, 0, 0, 0, -8, -8, -8, -16, 0, -40, -8};
   double *in;
   in = (double*)kml_fft_malloc(sizeof(double) * 12);
   for (int i = 0; i < 12; i++) {
       in[i] = init[i];
   }
   kml_fft_complex *out;
   out = (kml_fft_complex*)kml_fft_malloc(sizeof(kml_fft_complex) * (n[0] / 2 + 1) * howmany);
   kml_fft_plan plan;
   plan = kml_fft_plan_many_dft_r2c(rank, n, howmany, in, NULL, istride, idist, out, NULL, ostride, odist,
KML_FFT_ESTIMATE);
   kml_fft_execute_dft_r2c(plan, in, out);

   kml_fft_destroy_plan(plan);
   kml_fft_free(n);
   kml_fft_free(in);
   kml_fft_free(out);
```

 版权所有 © 华为技术有限公司

```
/*
 * out = {{1.280000e+02, 0.000000e+00}, {1.200000e+02, -8.000000e+00},
 *        {1.120000e+02, 0.000000e+00}, {-2.400000e+01, 0.000000e+00},
 *        {8.000000e+00, -0.000000e+00}, {8.000000e+00, 0.000000e+00},
 *        {-6.400000e+01, 0.000000e+00}, {2.400000e+01, -8.000000e+00},
 *        {-4.800000e+01, 0.000000e+00}}
 */
```

Fortran interface：

```
INTEGER(C_INT) :: RANK = 1
INTEGER(C_INT) :: N(1)
REAL(C_DOUBLE), DIMENSION(12) :: INIT
TYPE(C_DOUBLE), POINTER :: IN(:)
TYPE(KML_FFT_COMPLEX), POINTER :: OUT(:)
INTEGER, POINTER :: INEMBED(:), ONEMBED(:)
TYPE(C_PTR) :: PIN, POUT
INTEGER(C_INT) :: HOWMANY = 3
INTEGER(C_INT) :: ISTRIDE = 1
INTEGER(C_INT) :: OSTRIDE = 1
INTEGER(C_INT) :: IDIST = 4
INTEGER(C_INT) :: ODIST = 3
INTEGER(C_SIZE_T) :: RSIZE, CSIZE
RSIZE = 8 * 12
CSIZE = 16 * 9
PIN = KML_FFT_MALLOC(RSIZE)
POUT = KML_FFT_MALLOC(CSIZE)
CALL C_F_POINTER(PIN, IN, SHAPE=[12])
CALL C_F_POINTER(POUT, OUT, SHAPE=[9])
CALL C_F_POINTER(C_NULL_PTR, INEMBED, SHAPE=[0])
CALL C_F_POINTER(C_NULL_PTR, ONEMBED, SHAPE=[0])
N(1) = 4
DATA INIT/120, 8, 0, 0, 0, -8, -8, -8, -16, 0, -40, -8/

INTEGER :: I
DO WHILE(I <= 12)
   IN(I) = INIT(I)
   I = I + 1
END DO

TYPE(C_PTR) :: PLAN
PLAN = KML_FFT_PLAN_MANY_DFT_R2C(RANK, N, HOWMANY, IN, INEMBED, ISTRIDE, IDIST, OUT,
ONEMBED, OSTRIDE, ODIST, KML_FFT_ESTIMATE);
CALL KML_FFT_EXECUTE_DFT_R2C(PLAN, IN, OUT);

CALL KML_FFT_DESTROY_PLAN(PLAN)
CALL KML_FFT_FREE(PIN)
CALL KML_FFT_FREE(POUT)
!
! OUT = /1.280000E+02, 0.000000E+00, 1.200000E+02, -8.000000E+00,
!       1.120000E+02, 0.000000E+00, -2.400000E+01, 0.000000E+00,
!       8.000000E+00, -0.000000E+00, 8.000000E+00, 0.000000E+00,
!       -6.400000E+01, 0.000000E+00, 2.400000E+01, -8.000000E+00,
!       -4.800000E+01, 0.000000E+00/
!
```

6.4.4.4.2.10 kml_ffth_plan_dft_scale_r2c

建立单个连续数据序列n维R2C变换的plan。

接口定义

C interface：

kml_ffth_plan kml_ffth_plan_dft_scale_r2c(int rank, const int *n, __fp16 *in, kml_ffth_float_complex *out, unsigned flags);

 版权所有 © 华为技术有限公司

返回值

函数返回一个kml_ffth_plan类型的结构体指针。将该对象作为参数传入 **kml_fft(f/h)_execute**函数中，将对当前提供的输入in和输出out执行FFT变换。也可以通过将该对象作为参数传入 **kml_ffth_execute_dft_scale_r2c**函数中以对新的输入in和输出out执行FFT变换。

如果函数返回非空指针，则表示plan创建成功，否则表示创建失败。

参数

参数名	数据类型	描述	输入/输出
rank	int	FFT变换的维度是rank，约束：1 <= rank <= 4。	输入
n	const int*	n为维度为rank的数组，包含FFT序列每一维度的大小，约束：n[i] >= 1, for i in 0 to rank - 1。	输入
in	• __fp16*	输入需要变换的数据。	输入
out	• kml_ffth_float_complex*	输出快速傅里叶变换后的数据。	输出
flags	unsigned int	planning选项，描述ESTIMATE模式或PATIENT模式。 KML_FFT_ESTIMATE：ESTIMATE模式 KML_FFT_PATIENT：PATIENT模式	输入

依赖

C: "kfft.h"

示例

C interface：

```
int rank = 2;
int *n;
n = (int*)kml_ffth_malloc(sizeof(int) * rank);
n[0] = 2;
n[1] = 4;
double init[8] = {1, 2, 3, 4, 5, 6, 7, 8};
double *in;
in = (__fp16*)kml_ffth_malloc(sizeof(__fp16) * n[0] * n[1]);
for (int i = 0; i < n[0] * n[1]; i++) {
    in[i] = init[i];
}
kml_ffth_float_complex *out;
out = (kml_ffth_float_complex*)kml_ffth_malloc(sizeof(kml_ffth_float_complex) * n[0] * (n[1] / 2 + 1));
kml_ffth_plan plan;
plan = kml_ffth_plan_dft_scale_r2c(rank, n, in, out, KML_FFT_ESTIMATE);
kml_ffth_execute_dft_scale_r2c(plan, in, out);
```

 版权所有 © 华为技术有限公司

```
kml_ffth_destroy_plan(plan);
kml_ffth_free(n);
kml_ffth_free(in);
kml_ffth_free(out);

/*
 * out = {{3.600000e+01, 0.000000e+00}, {-4.000000e+00, 4.000000e+00},
 *        {-4.000000e+00, 0.000000e+00}, {-1.600000e+01, 0.000000e+00},
 *        {0.000000e+00, 0.000000e+00}, {0.000000e+00, 0.000000e+00}}
 */
```

6.4.4.4.2.11 kml_ffth_plan_dft_scale_r2c_1d

建立单个连续数据序列1维R2C变换的plan。

接口定义

C interface：

kml_ffth_plan kml_ffth_plan_dft_scale_r2c_1d(int n, __fp16 *in, kml_ffth_complex *out, unsigned flags);

返回值

函数返回一个kml_ffth_plan类型的结构体指针。将该对象作为参数传入 **kml_fft(f/h)_execute**函数中，将对当前提供的输入in和输出out执行FFT变换。此外，也可以通过将该对象作为参数传入 **kml_ffth_execute_dft_scale_r2c**函数中以对新的输入in和输出out执行FFT变换。

如果函数返回非空指针，则表示plan创建成功，否则表示创建失败。

参数

参数名	数据类型	描述	输入/输出
n	int	输入FFT序列的大小，约束：n ≥ 1。	输入
in	• __fp16*	输入需要变换的数据。	输入
out	• kml_ffth_float_complex*	输出快速傅里叶变换后的数据。	输出
flags	unsigned int	planning选项，描述ESTIMATE模式或PATIENT模式。 KML_FFT_ESTIMATE：ESTIMATE模式 KML_FFT_PATIENT：PATIENT模式	输入

依赖

C: "kfft.h"

 版权所有 © 华为技术有限公司

示例

C interface：

```
int n0 = 4;
double *in;
double init[4] = {1, 2, 3, 4};
in = (__fp16*)kml_ffth_malloc(sizeof(__fp16) * n0);
for (int i = 0; i < n0; i++) {
    in[i] = init[i];
}
kml_ffth_float_complex *out;
out = (kml_ffth_float_complex*)kml_ffth_malloc(sizeof(kml_ffth_float_complex) * (n0 / 2 + 1));
kml_ffth_plan plan;
plan = kml_ffth_plan_dft_scale_r2c_1d(n0, in, out, KML_FFT_ESTIMATE);
kml_ffth_execute_dft_scale_r2c(plan, in, out);

kml_ffth_destroy_plan(plan);
kml_ffth_free(in);
kml_ffth_free(out);

/*
 * out = {{1.000000e+01, 0.000000e+00}, {-2.000000e+00, 2.000000e+00},
 *        {-2.000000e+00, 0.000000e+00}}
 */
```

6.4.4.4.2.12 kml_ffth_plan_dft_scale_r2c_2d

建立单个连续数据序列2维R2C变换的plan。

接口定义

C interface：

kml_ffth_plan kml_ffth_plan_dft_scale_r2c_2d(int n0, int n1, __fp16 *in, kml_ffth_float_complex *out, unsigned flags);

返回值

函数返回一个kml_ffth_plan类型的结构体指针。将该对象作为参数传入 **kml_fft(f/h)_execute**函数中，将对当前提供的输入in和输出out执行FFT变换。也可以通过将该对象作为参数传入 **kml_ffth_execute_dft_scale_r2c**函数中以对新的输入in和输出out执行FFT变换。

如果函数返回非空指针，则表示plan创建成功，否则表示创建失败。

参数

参数名	数据类型	描述	输入/输出
n0	int	输入FFT序列第一维的大小，约束：n0 ≥ 1。	输入
n1	int	输入FFT序列第二维的大小，约束：n1 ≥ 1。	输入
in	• __fp16*	输入需要变换的数据。	输入

版权所有 © 华为技术有限公司

参数名	数据类型	描述	输入/输出
out	• kml_ffth_float_complex*	输出快速傅里叶变换后的数据。	输出
flags	unsigned int	planning选项，描述ESTIMATE模式或PATIENT模式。 KML_FFT_ESTIMATE：ESTIMATE模式 KML_FFT_PATIENT：PATIENT模式	输入

依赖

C: "kfft.h"

示例

C interface：

```
int n0 = 2;
int n1 = 4;
__fp16 init[8] = {1, 2, 3, 4, 5, 6, 7, 8};
__fp16*in;
in = (__fp16*)kml_ffth_malloc(sizeof(__fp16) * n0 * n1);
for (int i = 0; i < n0 * n1; i++) {
    in[i] = init[i];
}
kml_ffth_float_complex *out;
out = (kml_ffth_float_complex*)kml_ffth_malloc(sizeof(kml_ffth_float_complex) * n0 * (n1 / 2 + 1));
kml_ffth_plan plan;
plan = kml_ffth_plan_dft_scale_r2c_2d(n0, n1, in, out, KML_FFT_ESTIMATE);
kml_ffth_execute_dft_scale_r2c(plan, in, out);

kml_ffth_destroy_plan(plan);
kml_ffth_free(in);
kml_ffth_free(out);

/*
 * out = {{3.600000e+01, 0.000000e+00}, {-4.000000e+00, 4.000000e+00},
 *        {-4.000000e+00, 0.000000e+00}, {-1.600000e+01, 0.000000e+00},
 *        {0.000000e+00, 0.000000e+00}, {0.000000e+00, 0.000000e+00}}
 */
```

6.4.4.4.2.13 kml_ffth_plan_many_dft_scale_r2c

建立howmany组数据序列n维R2C变换的plan。其中，单个FFT的数据序列不需要是连续的，可以以跨步的形式提供。

接口定义

C interface：

kml_ffth_plan kml_ffth_plan_many_dft_scale_r2c(int rank, const int *n, int howmany, __fp16 *in, const int *inembed, int istride, int idist, kml_ffth_float_complex *out, const int *onembed, int ostride, int odist, unsigned flags);

 版权所有 © 华为技术有限公司

返回值

函数返回一个kml_ffth_plan类型的结构体指针。将该对象作为参数传入 **kml_fft(f/h)_execute**函数中，将对当前提供的输入in和输出out执行FFT变换。也可以通过将该对象作为参数传入 **kml_ffth_execute_dft_scale_r2c**函数中以对新的输入in和输出out执行FFT变换。

如果函数返回非空指针，则表示plan创建成功，否则表示创建失败。

参数

参数名	数据类型	描述	输入/输出
rank	int	FFT变换的维度是rank，约束：1 ≤ rank ≤ 4。	输入
n	const int*	n为维度为rank的数组，包含FFT序列每一维度的大小，约束：n[i] ≥ 1, for i in 0 to rank - 1。	输入
howmany	int	howmany表示指定多少个多维FFT变换。	输入
in	• __fp16*	输入需要变换的数据。	输入
inembed	const int*	inembed是大小为rank的数组或NULL，该数组表示FFT数据in存储的更大空间的每一维度的大小。 约束：inembed[i] ≥ n[i] for i in 0, rank-1 或者：如果inembed == NULL，则可以认为inembed与n相等	输入
istride	int	FFT输入序列的相继元素之间的间隔。	输入
idist	int	idist表示输入的每个FFT序列的间隔。	输入
out	• kml_ffth_float_complex*	输出快速傅里叶变换后的数据。	输出
onembed	const int*	onembed是大小为rank的数组或者NULL，该数组表示FFT数据out存储的更大空间的每一维度的大小。 约束：onembed[i] ≥ n[i] for i in 0, rank-1 或者：如果onembed == NULL，则可以认为onembed与n相等	输入
ostride	int	FFT输出序列的相继元素之间的间隔。	输入

版权所有 © 华为技术有限公司

参数名	数据类型	描述	输入/输出
odist	int	odist表示输出的每个FFT序列的间隔。	输入
flags	unsigned int	planning选项，描述ESTIMATE模式或PATIENT模式。 KML_FFT_ESTIMATE：ESTIMATE模式 KML_FFT_PATIENT：PATIENT模式	输入

依赖

C: "kfft.h"

示例

C interface：

```
   int rank = 1;
   int *n;
   n = (int*)kml_ffth_malloc(sizeof(int) * rank);
   n[0] = 4;
   int howmany = 3;
   int istride = 1;
   int ostride = 1;
   int idist = 4;
   int odist = n[0] / 2 + 1;
   __fp16 init[12] = {120, 8, 0, 0, 0, -8, -8, -8, -16, 0, -40, -8};
   __fp16 *in;
   in = (__fp16*)kml_ffth_malloc(sizeof(__fp16) * 12);
   for (int i = 0; i < 12; i++) {
       in[i] = init[i];
   }
   kml_ffth_float_complex *out;
   out = (kml_ffth_float_complex*)kml_ffth_malloc(sizeof(kml_ffth_float_complex) * (n[0] / 2 + 1) *
howmany);
   kml_ffth_plan plan;
   plan = kml_ffth_plan_many_dft_scale_r2c(rank, n, howmany, in, NULL, istride, idist, out, NULL, ostride,
odist, KML_FFT_ESTIMATE);
   kml_ffth_execute_dft_scale_r2c(plan, in, out);

   kml_ffth_destroy_plan(plan);
   kml_ffth_free(n);
   kml_ffth_free(in);
   kml_ffth_free(out);

   /*
    * out = {{1.280000e+02, 0.000000e+00}, {1.200000e+02, -8.000000e+00},
    *        {1.120000e+02, 0.000000e+00}, {-2.400000e+01, 0.000000e+00},
    *        {8.000000e+00, -0.000000e+00}, {8.000000e+00, 0.000000e+00},
    *        {-6.400000e+01, 0.000000e+00}, {2.400000e+01, -8.000000e+00},
    *        {-4.800000e+01, 0.000000e+00}}
    */
```

6.4.4.4.3 C2R 变换

 版权所有 © 华为技术有限公司

6.4.4.4.3.1 kml_fft(f/h)_plan_dft_c2r

建立单个连续数据序列n维C2R变换的plan。

接口定义

C interface：

kml_fft_plan kml_fft_plan_dft_c2r(int rank, const int *n, kml_fft_complex *in, double *out, unsigned flags);

kml_fftf_plan kml_fftf_plan_dft_c2r(int rank, const int *n, kml_fftf_complex *in, float *out, unsigned flags);

kml_ffth_plan kml_ffth_plan_dft_c2r(int rank, const int *n, kml_ffth_complex *in, __fp16 *out, unsigned flags);

Fortran interface：

RES = KML_FFT_PLAN_DFT_C2R(RANK, N, IN, OUT, FLAGS);

RES = KML_FFTF_PLAN_DFT_C2R(RANK, N, IN, OUT, FLAGS);

RES = KML_FFTH_PLAN_DFT_C2R(RANK, N, IN, OUT, FLAGS);

返回值

函数返回一个kml_fft(f/h)_plan类型的结构体指针。将该对象作为参数传入**kml_fft(f/h)_execute**函数中，将对当前提供的输入in和输出out执行FFT变换。也可以通过将该对象作为参数传入 **kml_fft(f/h)_execute_dft_c2r**函数中以对新的输入in和输出out执行FFT变换。

如果函数返回非空指针，则表示plan创建成功，否则表示创建失败。

参数

参数名	数据类型	描述	输入/输出
rank	int	FFT变换的维度是rank，约束：1 ≤ rank ≤ 4。	输入
n	const int*	n为维度为rank的数组，包含FFT序列每一维度的大小，约束：n[i] ≥ 1, for i in 0 to rank - 1。	输入
in	• 双精度：kml_fft_complex* • 单精度：kml_fftf_complex* • 半精度：kml_ffth_complex*	输入需要变换的数据。	输入

版权所有 © 华为技术有限公司

参数名	数据类型	描述	输入/输出
out	• 双精度：double* • 单精度：float* • 半精度：__fp16*	输出快速傅里叶变换后的数据。	输出
flags	unsigned int	planning选项，描述ESTIMATE模式或PATIENT模式。 KML_FFT_ESTIMATE：ESTIMATE模式 KML_FFT_PATIENT：PATIENT模式	输入

依赖

C: "kfft.h"

示例

C interface：

```
int rank = 2;
int *n;
n = (int*)kml_fft_malloc(sizeof(int) * rank);
n[0] = 2;
n[1] = 4;
double init[6][2] = {{120, 0}, {8, 8}, {0, 0}, {0, 16}, {0, 16}, {-8, 8}};
kml_fft_complex *in;
in = (kml_fft_complex*)kml_fft_malloc(sizeof(kml_fft_complex) * n[0] * (n[1] / 2 + 1));
for (int i = 0; i < n[0] * (n[1] / 2 + 1); i++) {
    in[i][0] = init[i][0];
    in[i][1] = init[i][1];
}
double *out;
out = (double*)kml_fft_malloc(sizeof(double) * n[0] * n[1]);
kml_fft_plan plan;
plan = kml_fft_plan_dft_c2r(rank, n, in, out, KML_FFT_ESTIMATE);
kml_fft_execute_dft_c2r(plan, in, out);

kml_fft_destroy_plan(plan);
kml_fft_free(n);
kml_fft_free(in);
kml_fft_free(out);

/*
 * out = {1.280000e+02, 8.000000e+01, 9.600000e+01, 1.760000e+02,
 *        1.440000e+02, 1.280000e+02, 1.120000e+02, 9.600000e+01}
 */
```

Fortran interface：

```
INTEGER(C_INT) :: RANK = 2
INTEGER(C_INT) :: N(2)
REAL(C_DOUBLE), DIMENSION(6, 2) :: INIT
DATA N/2, 3/
 TYPE(KML_FFT_COMPLEX), POINTER :: IN
REAL(C_DOUBLE), POINTER :: OUT
TYPE(C_PTR) :: PIN, POUT
```


版权所有 © 华为技术有限公司

```
INTEGER(C_SIZE_T) :: CSIZE, RSIZE
CSIZE = N(1) * (N(2) / 2 + 1)
RSIZE = N(1) * N(2)
PIN = KML_FFT_MALLOC(SIZEOF(16 * CSIZE))
POUT = KML_FFT_MALLOC(8 * RSIZE)
CALL C_F_POINTER(PIN, IN, SHAPE=[CSIZE])
CALL C_F_POINTER(POUT, OUT, SHAPE=[RSIZE])
DATA INIT/120, 8, 0, 0, 0, -8, 0, 8, 0, 16, 16, 8/
INTEGER :: I = 1
DO WHILE(I <= CSIZE)
   IN(I)%R = INIT(I, 0)
   IN(I)%I = INIT(I, 1)
END DO

TYPE(C_PTR) :: PLAN
PLAN = KML_FFT_PLAN_DFT_C2R(RANK, N, IN, OUT, KML_FFT_ESTIMATE)
CALL KML_FFT_EXECUTE_DFT_C2R(PLAN, IN, OUT)

CALL KML_FFT_DESTROY_PLAN(PLAN)
CALL KML_FFT_FREE(PIN)
CALL KML_FFT_FREE(POUT)

!
! OUT = /1.280000E+02, 8.000000E+01, 9.600000E+01, 1.760000E+02,
!       1.440000E+02, 1.280000E+02, 1.120000E+02, 9.600000E+01/
!
```

6.4.4.4.3.2 kml_fft(f/h)_plan_dft_c2r_1d

建立单个连续数据序列1维C2R变换的plan。

接口定义

C interface：

kml_fft_plan kml_fft_plan_dft_c2r_1d(int n, kml_fft_complex *in, double *out, unsigned flags);

kml_fftf_plan kml_fftf_plan_dft_c2r_1d(int n, kml_fftf_complex *in, float *out, unsigned flags);

kml_ffth_plan kml_ffth_plan_dft_c2r_1d(int n, kml_ffth_complex *in, __fp16 *out, unsigned flags);

Fortran interface：

RES = KML_FFT_PLAN_DFT_C2R_1D(N, IN, OUT, FLAGS);

RES = KML_FFTF_PLAN_DFT_C2R_1D(N, IN, OUT, FLAGS);

RES = KML_FFTH_PLAN_DFT_C2R_1D(N, IN, OUT, FLAGS);

返回值

函数返回一个kml_fft(f/h)_plan类型的结构体指针。将该对象作为参数传入**kml_fft(f/h)_execute**函数中，将对当前提供的输入in和输出out执行FFT变换。也可以通过将该对象作为参数传入 **kml_fft(f/h)_execute_dft_c2r**函数中以对新的输入in和输出out执行FFT变换。

如果函数返回非空指针，则表示plan创建成功，否则表示创建失败。

版权所有 © 华为技术有限公司

参数

参数名	数据类型	描述	输入/输出
n	int	输入FFT序列的大小，约束：n ≥ 1。	输入
in	• 双精度：kml_fft_complex* • 单精度：kml_fftf_complex* • 半精度：kml_ffth_complex*	输入需要变换的数据。	输入
out	• 双精度：double* • 单精度：float* • 半精度：__fp16*	输出快速傅里叶变换后的数据。	输出
flags	unsigned int	planning选项，描述ESTIMATE模式或PATIENT模式。 KML_FFT_ESTIMATE：ESTIMATE模式 KML_FFT_PATIENT：PATIENT模式	输入

依赖

C: "kfft.h"

示例

C interface：

```
int n0 = 4;
double init[3][2] = {{120, 0}, {8, 8}, {0, 0}};
kml_fft_complex *in;
in = (kml_fft_complex*)kml_fft_malloc(sizeof(kml_fft_complex) * (n0 / 2 + 1));
for (int i = 0; i < (n0 / 2 + 1); i++) {
    in[i][0] = init[i][0];
    in[i][1] = init[i][1];
}
double *out;
out = (double*)kml_fft_malloc(sizeof(double) * n0);
kml_fft_plan plan;
plan = kml_fft_plan_dft_c2r_1d(n0, in, out, KML_FFT_ESTIMATE);
kml_fft_execute_dft_c2r(plan, in, out);

kml_fft_destroy_plan(plan);
kml_fft_free(in);
kml_fft_free(out);

/*
```

 版权所有 © 华为技术有限公司

```
 * out = {1.360000e+02, 1.040000e+02, 1.040000e+02, 1.360000e+02}
 */
```

Fortran interface：

```
INTEGER(C_INT) :: N0 = 4
REAL(C_DOUBLE), DIMENSION(3, 2) :: INIT
TYPE(KML_FFT_COMPLEX), POINTER :: IN
REAL(C_DOUBLE), POINTER :: OUT
TYPE(C_PTR) :: PIN, POUT
INTEGER(C_SIZE_T) :: CSIZE, RSIZE
CSIZE = (N0 / 2 + 1)
RSIZE = N0
PIN = KML_FFT_MALLOC(SIZEOF(16 * CSIZE))
POUT = KML_FFT_MALLOC(8 * RSIZE)
CALL C_F_POINTER(PIN, IN, SHAPE=[CSIZE])
CALL C_F_POINTER(POUT, OUT, SHAPE=[RSIZE])
DATA INIT/120, 8, 0, 0, 8, 0/
INTEGER :: I = 1
DO WHILE(I <= CSIZE)
   IN(I)%R = INIT(I, 0)
   IN(I)%I = INIT(I, 1)
   I = I + 1
END DO
TYPE(C_PTR) :: PLAN
PLAN = KML_FFT_PLAN_DFT_C2R_1D(N0, IN, OUT, KML_FFT_ESTIMATE)
CALL KML_FFT_EXECUTE_DFT_C2R(PLAN, IN, OUT)

CALL KML_FFT_DESTROY_PLAN(PLAN)
CALL KML_FFT_FREE(PIN)
CALL KML_FFT_FREE(POUT)

!
! OUT = /1.360000e+02, 1.040000e+02, 1.040000e+02, 1.360000e+02/
!
```

6.4.4.4.3.3 kml_fft(f/h)_plan_dft_c2r_2d

建立单个连续数据序列2维C2R变换的plan。

接口定义

C interface：

kml_fft_plan kml_fft_plan_dft_c2r_2d(int n0, int n1, kml_fft_complex *in, double *out, unsigned flags);

kml_fftf_plan kml_fftf_plan_dft_c2r_2d(int n0, int n1, kml_fftf_complex *in, float *out, unsigned flags);

kml_ffth_plan kml_ffth_plan_dft_c2r_2d(int n0, int n1, kml_ffth_complex *in, __fp16 *out, unsigned flags);

Fortran interface：

RES = KML_FFT_PLAN_DFT_C2R_2D(N0, N1, IN, OUT, FLAGS);

RES = KML_FFTF_PLAN_DFT_C2R_2D(N0, N1, IN, OUT, FLAGS);

RES = KML_FFTH_PLAN_DFT_C2R_2D(N0, N1, IN, OUT, FLAGS);

返回值

函数返回一个kml_fft(f/h)_plan类型的结构体指针。将该对象作为参数传入**kml_fft(f/h)_execute**函数中，将对当前提供的输入in和输出out执行FFT变换。也可

 版权所有 © 华为技术有限公司

以通过将该对象作为参数传入 **kml_fft(f/h)_execute_dft_c2r**函数中以对新的输入in和输出out执行FFT变换。

如果函数返回非空指针，则表示plan创建成功，否则表示创建失败。

参数

参数名	数据类型	描述	输入/输出
n0	int	输入FFT序列第一维的大小，约束：n0 ≥ 1。	输入
n1	int	输入FFT序列第二维的大小，约束：n1 ≥ 1。	输入
in	• 双精度：kml_fft_complex* • 单精度：kml_fftf_complex* • 半精度：kml_ffth_complex*	输入需要变换的数据。	输入
out	• 双精度：double* • 单精度：float* • 半精度：__fp16*	输出快速傅里叶变换后的数据。	输出
flags	unsigned int	planning选项，描述ESTIMATE模式或PATIENT模式。 KML_FFT_ESTIMATE：ESTIMATE模式 KML_FFT_PATIENT：PATIENT模式	输入

依赖

C: "kfft.h"

示例

C interface：

```
    int n0 = 4;
    int n1 = 4;
    double init[12][2] = {{120, 0}, {8, 8}, {0, 0}, {0, 16}, {0, 16}, {-8, 8}, {-8, 0}, {-8, 8}, {-16, 0}, {0, -16}, {-40, 8}, {-8, -8}};
    kml_fft_complex *in;
    in = (kml_fft_complex*)kml_fft_malloc(sizeof(kml_fft_complex) * n0 * (n1 / 2 + 1));
    for (int i = 0; i < n0 * (n1 / 2 + 1); i++){
        in[i][0] = init[i][0];
```

 版权所有 © 华为技术有限公司

```
    in[i][1] = init[i][1];
}
double *out;
out = (double*)kml_fft_malloc(sizeof(double) * n0 * n1);
kml_fft_plan plan;
plan = kml_fft_plan_dft_c2r_2d(n0, n1, in , out , KML_FFT_ESTIMATE);
kml_fft_execute_dft_c2r(plan, in, out);

kml_fft_destroy_plan(plan);
kml_fft_free(in);
kml_fft_free(out);

/*
 * out = {0.000000e+00, 6.400000e+01, 1.600000e+02, 2.240000e+02,
 *        1.120000e+02, 1.600000e+01, 8.000000e+01, 1.760000e+02,
 *        1.920000e+02, 1.280000e+02, 3.200000e+01, 9.600000e+01,
 *        2.400000e+02, 2.080000e+02, 1.440000e+02, 4.800000e+01}
 */
```

Fortran interface：

```
INTEGER(C_INT) :: N0 = 4
INTEGER(C_INT) :: N1 = 4
REAL(C_DOUBLE), DIMENSION(12, 2) :: INIT
TYPE(KML_FFT_COMPLEX), POINTER :: IN
REAL(C_DOUBLE), POINTER :: OUT
TYPE(C_PTR) :: PIN, POUT
INTEGER(C_SIZE_T) :: CSIZE, RSIZE
CSIZE = N0 * (N1 / 2 + 1)
RSIZE = N0 * N1
PIN = KML_FFT_MALLOC(SIZEOF(16 * CSIZE))
POUT = KML_FFT_MALLOC(8 * RSIZE)
CALL C_F_POINTER(PIN, IN, SHAPE=[CSIZE])
CALL C_F_POINTER(POUT, OUT, SHAPE=[RSIZE])
DATA INIT/120, 8, 0, 0, 0, -8, -8, -8, -16, 0, -40, -8, 0, 8, 0, 16, 16, 8, 0, 8, 0, 16, 8, -8/
INTEGER :: I = 1
DO WHILE(I <= CSIZE)
   IN(I)%R = INIT(I, 0)
   IN(I)%I = INIT(I, 1)
   I = I + 1
END DO
TYPE(C_PTR) :: PLAN
PLAN = KML_FFT_PLAN_DFT_C2R_2D(N0, N1, IN , OUT , KML_FFT_ESTIMATE);
CALL KML_FFT_EXECUTE_DFT_C2R(PLAN, IN, OUT);

CALL KML_FFT_DESTROY_PLAN(PLAN)
CALL KML_FFT_FREE(PIN)
CALL KML_FFT_FREE(POUT)

!
! OUT = /0.000000e+00, 6.400000e+01, 1.600000e+02, 2.240000e+02,
!       1.120000e+02, 1.600000e+01, 8.000000e+01, 1.760000e+02,
!       1.920000e+02, 1.280000e+02, 3.200000e+01, 9.600000e+01,
!       2.400000e+02, 2.080000e+02, 1.440000e+02, 4.800000e+01/
!
```

6.4.4.4.3.4 kml_fft(f/h)_plan_dft_c2r_3d

建立单个连续数据序列3维C2R变换的plan。

接口定义

C interface：

kml_fft_plan kml_fft_plan_dft_c2r_3d(int n0, int n1, int n2, kml_fft_complex *in, double *out, unsigned flags);

 版权所有 © 华为技术有限公司

kml_fftf_plan kml_fftf_plan_dft_c2r_3d(int n0, int n1, int n2, kml_fftf_complex *in, float *out, unsigned flags);

kml_ffth_plan kml_ffth_plan_dft_c2r_3d(int n0, int n1, int n2, kml_ffth_complex *in, __fp16 *out, unsigned flags);

Fortran interface：

RES = KML_FFT_PLAN_DFT_C2R_3D(N0, N1, N2, IN, OUT, FLAGS);

RES = KML_FFTF_PLAN_DFT_C2R_3D(N0, N1, N2, IN, OUT, FLAGS);

RES = KML_FFTH_PLAN_DFT_C2R_3D(N0, N1, N2, IN, OUT, FLAGS);

返回值

函数返回一个kml_fft(f/h)_plan类型的结构体指针。将该对象作为参数传入**kml_fft(f/h)_execute**函数中，将对当前提供的输入in和输出out执行FFT变换。也可以通过将该对象作为参数传入 **kml_fft(f/h)_execute_dft_c2r**函数中以对新的输入in和输出out执行FFT变换。

如果函数返回非空指针，则表示plan创建成功，否则表示创建失败。

参数

参数名	数据类型	描述	输入/输出
n0	int	输入FFT序列第一维的大小，约束：n0 ≥ 1。	输入
n1	int	输入FFT序列第二维的大小，约束：n1 ≥ 1。	输入
n2	int	输入FFT序列第三维的大小，约束：n2 ≥ 1。	输入
in	• 双精度：kml_fft_complex* • 单精度：kml_fftf_complex* • 半精度：kml_ffth_complex*	输入需要变换的数据。	输入
out	• 双精度：double* • 单精度：float* • 半精度：__fp16*	输出快速傅里叶变换后的数据。	输出

 版权所有 © 华为技术有限公司

参数名	数据类型	描述	输入/输出
flags	unsigned int	planning选项，描述ESTIMATE模式或PATIENT模式。 KML_FFT_ESTIMATE：ESTIMATE模式 KML_FFT_PATIENT：PATIENT模式	输入

依赖

#include "kfft.h"

示例

C interface：

```
int n0 = 2;
int n1 = 3;
int n2 = 4;
double init[18][2] = {{120, 0}, {8, 8}, {0, 0}, {0, 16}, {0, 16}, {-8, 8}, {-8, 0}, {-8, 8}, {-16, 0}, {0, -16}, {-40, 8}, {-8, -8},
    {120, 0}, {8, 8}, {0, 0}, {0, 16}, {0, 16}, {-8, 8}};
kml_fft_complex *in;
in = (kml_fft_complex*)kml_fft_malloc(sizeof(kml_fft_complex) * n0 * n1 * (n2 / 2 + 1));
for (int i = 0; i < n0 * n1 * (n2 / 2 + 1); i++) {
    in[i][0] = init[i][0];
    in[i][1] = init[i][1];
}
double *out;
out = (double*)kml_fft_malloc(sizeof(double) * n0 * n1 * n2);
kml_fft_plan plan;
plan = kml_fft_plan_dft_c2r_3d(n0, n1, n2, in, out, KML_FFT_ESTIMATE);
kml_fft_execute_dft_c2r(plan, in, out);

kml_fft_free(in);
kml_fft_free(out);

/*
 * out = {1.280000e+02, 1.440000e+02, 2.560000e+02, 4.000000e+02,
 *        8.000000e+00, 4.428719e+01, 1.360000e+02, 6.771281e+01,
 *        8.000000e+00, 9.971281e+01, 1.360000e+02, 1.228719e+01,
 *        4.800000e+01, 0.000000e+00, -8.000000e+01, 0.000000e+00,
 *        2.427180e+02, 1.541436e+02, 7.414359e+01, 1.541436e+02,
 *        3.812820e+02, 1.818564e+02, 1.018564e+02, 1.818564e+02}
 */
```

Fortran interface：

```
INTEGER(C_INT) :: N0 = 2
INTEGER(C_INT) :: N1 = 3
INTEGER(C_INT) :: N2 = 4
REAL(C_DOUBLE), DIMENSION(18, 2) :: INIT
TYPE(KML_FFT_COMPLEX), POINTER :: IN
REAL(C_DOUBLE), POINTER :: OUT
TYPE(C_PTR) :: PIN, POUT
INTEGER(C_SIZE_T) :: CSIZE, RSIZE
CSIZE = N0 * N1 * (N2 / 2 + 1)
RSIZE = N0 * N1 * N2
PIN = KML_FFT_MALLOC(SIZEOF(16 * CSIZE))
POUT = KML_FFT_MALLOC(8 * RSIZE)
CALL C_F_POINTER(PIN, IN, SHAPE=[CSIZE])
```


版权所有 © 华为技术有限公司

```
   CALL C_F_POINTER(POUT, OUT, SHAPE=[RSIZE])
   DATA INIT/120, 8, 0, 0, 0, -8, -8, -8, -16, 0, -40, -8, 120, 8, 0, 0, 0, -8,0, 8, 0, 16, 16, 8, 0, 8, 0, 16, 8, -8, 0,
8, 0, 16, 16, 8/
   INTEGER :: I = 1
   DO WHILE(I <= CSIZE)
      IN(I)%R = INIT(I, 0)
      IN(I)%I = INIT(I, 1)
      I = I + 1
   END DO
   TYPE(C_PTR) :: PLAN
   PLAN = KML_FFT_PLAN_DFT_C2R_3D(N0, N1, N2, IN , OUT , KML_FFT_ESTIMATE);
   CALL KML_FFT_EXECUTE_DFT_C2R(PLAN, IN, OUT);

   CALL KML_FFT_DESTROY_PLAN(PLAN)
   CALL KML_FFT_FREE(PIN)
   CALL KML_FFT_FREE(POUT)
   !
   ! OUT = /1.280000E+02, 1.440000E+02, 2.560000E+02, 4.000000E+02,
   !       8.000000E+00, 4.428719E+01, 1.360000E+02, 6.771281E+01,
   !       8.000000E+00, 9.971281E+01, 1.360000E+02, 1.228719E+01,
   !       4.800000E+01, 0.000000E+00, -8.000000E+01, 0.000000E+00,
   !       2.427180E+02, 1.541436E+02, 7.414359E+01, 1.541436E+02,
   !       3.812820E+02, 1.818564E+02, 1.018564E+02, 1.818564E+02/
   !
```

6.4.4.4.3.5 kml_fft(f/h)_plan_guru_dft_c2r

建立多组数据序列n维C2R变换的plan。其中，单个FFT的数据序列不需要是连续的，可以以跨步的形式提供。

接口定义

C interface：

kml_fft_plan kml_fft_plan_guru_dft_c2r(int rank, const kml_fft_iodim *dims, int howmany_rank, const kml_fft_iodim *howmany_dims, kml_fft_complex *in, double *out, unsigned flags);

kml_fftf_plan kml_fftf_plan_guru_dft_c2r(int rank, const kml_fftf_iodim *dims, int howmany_rank, const kml_fftf_iodim *howmany_dims, kml_fftg_complex *in, float *out, unsigned flags);

kml_ffth_plan kml_ffth_plan_guru_dft_c2r(int rank, const kml_ffth_iodim *dims, int howmany_rank, const kml_ffth_iodim *howmany_dims, kml_ffth_complex *in, __fp16 *out, unsigned flags);

Fortran interface：

RES = KML_FFT_PLAN_GURU_DFT_C2R(RANK, DIMS, HOWMANY_RANK, HOWMANY_DIMS, IN, OUT, FLAGS);

RES = KML_FFTF_PLAN_GURU_DFT_C2R(RANK, DIMS, HOWMANY_RANK, HOWMANY_DIMS, IN, OUT, FLAGS);

RES = KML_FFTH_PLAN_GURU_DFT_C2R(RANK, DIMS, HOWMANY_RANK, HOWMANY_DIMS, IN, OUT, FLAGS);

返回值

函数返回一个kml_fft(f/h)_plan类型的结构体指针。将该对象作为参数传入**kml_fft(f/h)_execute**函数中，将对当前提供的输入in和输出out执行FFT变换。也可

 版权所有 © 华为技术有限公司

以通过将该对象作为参数传入 **kml_fft(f/h)_execute_dft_c2r**函数中以对新的输入in和输出out执行FFT变换。

如果函数返回非空指针，则表示plan创建成功，否则表示创建失败。

参数

参数名	数据类型	描述	输入/输出
rank	int	单个FFT序列的维度。 约束：1 ≤ rank ≤ 4。	输入
dims	• 双精度：const kml_fft_iodim* • 单精度：const kml_fftf_iodim* • 半精度：const kml_ffth_iodim*	dims是大小为rank的结构体数组，dims[i]包含以下成员： • int n - 第i维FFT的长度。 • int is - 第i维FFT输入序列的相继元素之间的间隔。 • int os - 第i维FFT输出序列的相继元素之间的间隔。 约束：dims[i].n ≥ 1, for i in 0 to rank - 1。	输入
howmany_rank	int	多个rank维FFT之间的内存排布用howmany_rank维的howmany_dims数组来描述，howmany_rank表示每个要计算的rank维FFT变换的起始地址的内存访问模式所需的维数。 约束：0 ≤ howmany_rank ≤ 3。	输入
howmany_dims	• 双精度：const kml_fft_iodim* • 单精度：const kml_fftf_iodim* • 半精度：const kml_ffth_iodim*	howmany_dims是大小为howmany_rank的结构体数组，howmany_dims[i]包含以下成员： • int n - howmany_rank维空间的第i维的待变换FFT的个数。 • int is - 第i维的相继FFT输入序列之间的间隔。 • int os - 第i维的相继FFT输出序列之间的间隔。	输入
in	• 双精度：kml_fft_complex* • 单精度：kml_fftf_complex* • 半精度：kml_ffth_complex*	输入需要变换的数据。	输入

 版权所有 © 华为技术有限公司

参数名	数据类型	描述	输入/输出
out	• 双精度：double* • 单精度：float* • 半精度：__fp16*	输出快速傅里叶变换后的数据。	输出
flags	unsigned int	planning选项，描述ESTIMATE模式或PATIENT模式。 KML_FFT_ESTIMATE：ESTIMATE模式 KML_FFT_PATIENT：PATIENT模式	输入

依赖

C: "kfft.h"

示例

C interface：

```
int rank = 2;
kml_fft_iodim *dims;
dims = (kml_fft_iodim*)kml_fft_malloc(sizeof(kml_fft_iodim) * rank);
dims[0].n = 2;
dims[0].is = 2;
dims[0].os = 3;
dims[1].n = 3;
dims[1].is = 1;
dims[1].os = 1;
int howmany_rank = 1;
kml_fft_iodim *howmany_dims;
howmany_dims = (kml_fft_iodim*)kml_fft_malloc(sizeof(kml_fft_iodim) * howmany_rank);
howmany_dims[0].n = 2;
howmany_dims[0].is = 2 * 2;
howmany_dims[0].os = 2 * 3;
double init[8][2] = {{120, 0}, {8, 8}, {0, 0}, {0, 16}, {0, 16}, {-8, 8}, {-8, 0}, {-8, 8}};
kml_fft_complex *in;
in = (kml_fft_complex*)kml_fft_malloc(sizeof(kml_fft_complex) * 8);
for (int i = 0; i < 8; i++) {
    in[i][0] = init[i][0];
    in[i][1] = init[i][1];
}
double *out;
out = (double*)kml_fft_malloc(sizeof(double) * 12);
kml_fft_plan plan;
plan = kml_fft_plan_guru_dft_c2r(rank, dims, howmany_rank, howmany_dims, in, out,
KML_FFT_ESTIMATE);
kml_fft_execute_dft_c2r(plan, in, out);

kml_fft_destroy_plan(plan);
kml_fft_free(howmany_dims);
kml_fft_free(dims);
kml_fft_free(in);
kml_fft_free(out);

/*
 * out = {1.360000e+02, 7.043078e+01, 1.535692e+02, 1.360000e+02,
```

 版权所有 © 华为技术有限公司

```
 *       1.258564e+02, 9.814359e+01, -4.000000e+01, -1.971281e+01,
 *       3.571281e+01, 8.000000e+00, 8.000000e+00, 8.000000e+00}
 */
```

Fortran interface：

```
  INTEGER(C_INT) :: RANK = 2
  INTEGER(C_INT) :: HOWMANY_RANK = 1
  REAL(C_DOUBLE), DIMENSION(8, 2) :: INIT
  TYPE(KML_FFT_IODIM), POINTER :: DIMS(:), HOWMANY_DIMS(:)
  TYPE(KML_FFT_COMPLEX), POINTER :: IN(:)
  REAL(8), POINTER :: OUT(:)
  TYPE(C_PTR) :: PIN, POUT, PDIMS, PHOWMANY_DIMS
  INTEGER(C_SIZE_T) :: CSIZE, RSIZE
  CSIZE = 16 * 8
  RSIZE = 8 * 12
  PIN = KML_FFT_MALLOC(CSIZE)
  POUT = KML_FFT_MALLOC(RSIZE)
  CALL C_F_POINTER(PIN, IN, SHAPE=[8])
  CALL C_F_POINTER(POUT, OUT, SHAPE=[12])
  CSIZE = 12 * 2
  RSIZE = 12 * 1
  PIODIMS = KML_FFT_MALLOC(CSIZE)
  PHOWMANY_DIMS = KML_FFT_MALLOC(RSIZE)
  CALL C_F_POINTER(PIODIMS, DIMS, SHAPE=[2])
  CALL C_F_POINTER(PHOWMANY_DIMS, HOWMANY_DIMS, SHAPE=[1])
  DIMS(0)%N = 2
  DIMS(0)%IS = 2
  DIMS(0)%OS = 3
  DIMS(1)%N = 3
  DIMS(1)%IS = 1
  DIMS(1)%OS = 1
  HOWMANY_DIMS(0)%N = 2
  HOWMANY_DIMS(0)%IS = 2 * 2
  HOWMANY_DIMS(0)%OS = 2 * 3
  DATA INIT/120, 8, 0, 0, 0, -8, -8, -8,  0, 8, 0, 16, 16, 8, 0, 8/
  INTEGER :: I = 1
  DO WHILE(I <= 8)
     IN(I)%R = INIT(I, 0)
     IN(I)%I = INIT(I, 1)
     I = I + 1
  END DO
  TYPE(C_PTR) :: PLAN
  PLAN = KML_FFT_PLAN_GURU_DFT_C2R(RANK, DIMS, HOWMANY_RANK, HOWMANY_DIMS, IN, OUT,
KML_FFT_ESTIMATE)
  CALL KML_FFT_EXECUTE_DFT_C2R(PLAN, IN, OUT);

  CALL KML_FFT_DESTROY_PLAN(PLAN)
  CALL KML_FFT_FREE(PHOWMANY_DIMS)
  CALL KML_FFT_FREE(PDIMS)
  CALL KML_FFT_FREE(PIN)
  CALL KML_FFT_FREE(POUT)
  !
  ! OUT = /1.360000E+02, 7.043078E+01, 1.535692E+02, 1.360000E+02,
  !       1.258564E+02, 9.814359E+01, -4.000000E+01, -1.971281E+01,
  !       3.571281E+01, 8.000000E+00, 8.000000E+00, 8.000000E+00/
  !
```

6.4.4.4.3.6 kml_fft(f/h)_plan_guru_split_dft_c2r

建立多组数据序列n维C2R变换的plan。其中，单个FFT的数据序列不需要是连续的，可以以跨步的形式提供。split类接口的输入和输出分别存储在实部和虚部数组中。

接口定义

C interface：

 版权所有 © 华为技术有限公司

kml_fft_plan kml_fft_plan_guru_split_dft_c2r(int rank, const kml_fft_iodim *dims, int howmany_rank, const kml_fft_iodim *howmany_dims, double *ri, double *ii, double *out, unsigned flags);

kml_fftf_plan kml_fftf_plan_guru_split_dft_c2r(int rank, const kml_fftf_iodim *dims, int howmany_rank, const kml_fftf_iodim *howmany_dims, float *ri, float *ii, float *out, unsigned flags);

kml_ffth_plan kml_ffth_plan_guru_split_dft_c2r(int rank, const kml_ffth_iodim *dims, int howmany_rank, const kml_ffth_iodim *howmany_dims, __fp16 *ri, __fp16 *ii, __fp16 *out, unsigned flags);

Fortran interface:

RES = KML_FFT_PLAN_GURU_SPLIT_DFT_C2R(RANK, DIMS, HOWMANY_RANK, HOWMANY_DIMS, RI, II, OUT, FLAGS);

RES = KML_FFTF_PLAN_GURU_SPLIT_DFT_C2R(RANK, DIMS, HOWMANY_RANK, HOWMANY_DIMS, RI, II, OUT, FLAGS);

RES = KML_FFTH_PLAN_GURU_SPLIT_DFT_C2R(RANK, DIMS, HOWMANY_RANK, HOWMANY_DIMS, RI, II, OUT, FLAGS);

返回值

函数返回一个kml_fft(f/h)_plan类型的结构体指针。将该对象作为参数传入**kml_fft(f/h)_execute**函数中，将对当前提供的输入ri，ii和输出out执行FFT变换。也可以通过将该对象作为参数传入 **kml_fft(f/h)_execute_split_dft_c2r**函数中以对新的输入ri，ii和输出out执行FFT变换。

如果函数返回非空指针，则表示plan创建成功，否则表示创建失败。

参数

参数名	数据类型	描述	输入/输出
rank	int	单个FFT序列的维度。 约束：1 ≤ rank ≤ 4。	输入
dims	• 双精度：const kml_fft_iodim* • 单精度：const kml_fftf_iodim* • 半精度：const kml_ffth_iodim*	• dims是大小为rank的结构体数组，dims[i]包含以下成员： • int n - 第i维FFT的长度。 • int is - 第i维FFT输入序列的相继元素之间的间隔。 • int os - 第i维FFT输出序列的相继元素之间的间隔。 约束：dims[i].n ≥ 1, for i in 0 to rank - 1。	输入

版权所有 © 华为技术有限公司

参数名	数据类型	描述	输入/输出
howmany_rank	int	多个rank维FFT之间的内存排布用howmany_rank维的howmany_dims数组来描述，howmany_rank表示每个要计算的rank维FFT变换的起始地址的内存访问模式所需的维数。 约束：0 ≤ howmany_rank ≤ 4。	输入
howmany_dims	• 双精度：const kml_fft_iodim* • 单精度：const kml_fftf_iodim* • 半精度：const kml_ffth_iodim*	howmany_dims是大小为howmany_rank的结构体数组，howmany_dims[i]包含以下成员： • int n - howmany_rank维空间的第i维的待变换FFT的个数。 • int is - 第i维的相继FFT输入序列之间的间隔。 • int os - 第i维的相继FFT输出序列之间的间隔。	输入
ri	• 双精度：double* • 单精度：float* • 半精度：__fp16*	输入待变换数据的实部。	输入
ii	• 双精度：double* • 单精度：float* • 半精度：__fp16*	输入待变换数据的虚部。	输入
out	• 双精度：double* • 单精度：float* • 半精度：__fp16*	输出快速傅里叶变换后的数据。	输出
flags	unsigned int	planning选项，描述ESTIMATE模式或PATIENT模式。 KML_FFT_ESTIMATE：ESTIMATE模式 KML_FFT_PATIENT：PATIENT模式	输入

依赖

C: "kfft.h"

示例

C interface：

 版权所有 © 华为技术有限公司

```
int rank = 2;
kml_fft_iodim *dims;
dims = (kml_fft_iodim*)kml_fft_malloc(sizeof(kml_fft_iodim) * rank);
dims[0].n = 2;
dims[0].is = 2;
dims[0].os = 3;
dims[1].n = 3;
dims[1].is = 1;
dims[1].os = 1;
int howmany_rank = 1;
kml_fft_iodim *howmany_dims;
howmany_dims = (kml_fft_iodim*)kml_fft_malloc(sizeof(kml_fft_iodim) * howmany_rank);
howmany_dims[0].n = 2;
howmany_dims[0].is = 2 * 2;
howmany_dims[0].os = 2 * 3;
double init[8][2] = {{120, 0}, {8, 8}, {0, 0}, {0, 16}, {0, 16}, {-8, 8}, {-8, 0}, {-8, 8}};
double *ri;
ri = (double*)kml_fft_malloc(sizeof(double) * 8);
double *ii;
ii = (double*)kml_fft_malloc(sizeof(double) * 8);
for (int i = 0; i < 8; i++) {
    ri[i] = init[i][0];
    ii[i] = init[i][1];
}
double *out;
out = (double*)kml_fft_malloc(sizeof(double) * 12);
kml_fft_plan plan;
plan = kml_fft_plan_guru_split_dft_c2r(rank, dims, howmany_rank, howmany_dims, ri, ii, out,
KML_FFT_ESTIMATE);
kml_fft_execute_split_dft_c2r(plan, ri, ii, out);

kml_fft_destroy_plan(plan);
kml_fft_free(howmany_dims);
kml_fft_free(dims);
kml_fft_free(ri);
kml_fft_free(ii);
kml_fft_free(out);

/*
 * out = {1.360000e+02, 7.043078e+01, 1.535692e+02, 1.360000e+02,
 *        1.258564e+02, 9.814359e+01, -4.000000e+01, -1.971281e+01,
 *        3.571281e+01, 8.000000e+00, 8.000000e+00, 8.000000e+00}
 */
```

Fortran interface:

```
INTEGER(C_INT) :: RANK = 2
INTEGER(C_INT) :: HOWMANY_RANK = 1
REAL(C_DOUBLE), DIMENSION(8, 2) :: INIT
TYPE(KML_FFT_IODIM), POINTER :: DIMS(:), HOWMANY_DIMS(:)
REAL(8), POINTER :: RI(:), II(:), OUT(:)
TYPE(C_PTR) :: PRI,PII, POUT, PDIMS, PHOWMANY_DIMS
INTEGER(C_SIZE_T) :: CSIZE, RSIZE
CSIZE = 8 * 8
RSIZE = 8 * 12
PRI = KML_FFT_MALLOC(CSIZE)
PII = KML_FFT_MALLOC(CSIZE)
POUT = KML_FFT_MALLOC(RSIZE)
CALL C_F_POINTER(PRI, IN, SHAPE=[8])
CALL C_F_POINTER(PII, IN, SHAPE=[8])
CALL C_F_POINTER(POUT, OUT, SHAPE=[12])
CSIZE = 12 * 2
RSIZE = 12 * 1
PIODIMS = KML_FFT_MALLOC(CSIZE)
PHOWMANY_DIMS = KML_FFT_MALLOC(RSIZE)
CALL C_F_POINTER(PIODIMS, DIMS, SHAPE=[2])
CALL C_F_POINTER(PHOWMANY_DIMS, HOWMANY_DIMS, SHAPE=[1])
DIMS(0)%N = 2
DIMS(0)%IS = 2
DIMS(0)%OS = 3
```

 版权所有 © 华为技术有限公司

```
   DIMS(1)%N = 3
   DIMS(1)%IS = 1
   DIMS(1)%OS = 1
   HOWMANY_DIMS(0)%N = 2
   HOWMANY_DIMS(0)%IS = 2 * 2
   HOWMANY_DIMS(0)%OS = 2 * 3
   DATA INIT/120, 8, 0, 0, 0, -8, -8, -8,  0, 8, 0, 16, 16, 8, 0, 8/
   INTEGER :: I = 1
   DO WHILE(I <= 8)
      RI(I) = INIT(I, 0)
      II(I) = INIT(I, 1)
      I = I + 1
   END DO
   TYPE(C_PTR) :: PLAN
   PLAN = KML_FFT_PLAN_GURU_SPLIT_DFT_C2R(RANK, DIMS, HOWMANY_RANK, HOWMANY_DIMS, RI,
II, OUT, KML_FFT_ESTIMATE)
   CALL KML_FFT_EXECUTE_SPLIT_DFT_C2R(PLAN, RI, II, OUT);

   CALL KML_FFT_DESTROY_PLAN(PLAN)
   CALL KML_FFT_FREE(PHOWMANY_DIMS)
   CALL KML_FFT_FREE(PDIMS)
   CALL KML_FFT_FREE(PRI)
   CALL KML_FFT_FREE(PII)
   CALL KML_FFT_FREE(POUT)
   !
   ! OUT = /1.360000E+02, 7.043078E+01, 1.535692E+02, 1.360000E+02,
   !       1.258564E+02, 9.814359E+01, -4.000000E+01, -1.971281E+01,
   !       3.571281E+01, 8.000000E+00, 8.000000E+00, 8.000000E+00/
   !
```

⚠ 注意

kml_fft(f)_plan_guru_split_dft_c2r返回的plan可以使用kml_fft(f)_execute计算plan时的输入，也可以使用kml_fft(f)_execute_split_dft_c2r计算不同的输入。plan时的输入不需要初始化，但是输入必须是合法的（不能为NULL）。

6.4.4.4.3.7 kml_fft(f/h)_plan_guru64_dft_c2r

建立多组数据序列n维C2R变换的plan。其中，单个FFT的数据序列不需要是连续的，可以以跨步的形式提供。与kml_fft(f)_plan_guru_dft不同的是，kml_fft(f)_plan_guru64_dft允许部分参数是64位的整型。

接口定义

C interface：

kml_fft_plan kml_fft_plan_guru64_dft_c2r(int rank, const kml_fft_iodim64 *dims, int howmany_rank, const kml_fft_iodim64 *howmany_dims, kml_fft_complex *in, double *out, unsigned flags);

kml_fftf_plan kml_fftf_plan_guru64_dft_c2r(int rank, const kml_fftf_iodim64 *dims, int howmany_rank, const kml_fftf_iodim64 *howmany_dims, kml_fftf_complex *in, float *out, unsigned flags);

kml_ffth_plan kml_ffth_plan_guru64_dft_c2r(int rank, const kml_ffth_iodim64 *dims, int howmany_rank, const kml_ffth_iodim64 *howmany_dims, kml_ffth_complex *in, __fp16 *out, unsigned flags);

Fortran interface：

 版权所有 © 华为技术有限公司

RES = KML_FFT_PLAN_GURU64_DFT_C2R(RANK, DIMS, HOWMANY_RANK, HOWMANY_DIMS, IN, OUT, FLAGS);

RES = KML_FFTF_PLAN_GURU64_DFT_C2R(RANK, DIMS, HOWMANY_RANK, HOWMANY_DIMS, IN, OUT, FLAGS);

RES = KML_FFTH_PLAN_GURU64_DFT_C2R(RANK, DIMS, HOWMANY_RANK, HOWMANY_DIMS, IN, OUT, FLAGS);

返回值

函数返回一个kml_fft(f/h)_plan类型的结构体指针。将该对象作为参数传入**kml_fft(f/h)_execute**函数中，将对当前提供的输入in和输出out执行FFT变换。也可以通过将该对象作为参数传入 **kml_fft(f/h)_execute_dft_c2r**函数中以对新的输入in和输出out执行FFT变换。

如果函数返回非空指针，则表示plan创建成功，否则表示创建失败。

参数

参数名	数据类型	描述	输入/输出
rank	int	单个FFT序列的维度。 约束：1 ≤ rank ≤ 3。	输入
dims	• 双精度：const kml_fft_iodim64 * • 单精度：const kml_fftf_iodim64 * • 半精度：const kml_ffth_iodim64*	dims是大小为rank的结构体数组，dims[i]包含以下成员： • ptrdiff_t n - 第i维FFT的长度。 • ptrdiff_t is - 第i维FFT输入序列的相继元素之间的间隔。 • ptrdiff_t os - 第i维FFT输出序列的相继元素之间的间隔。 约束：dims[i].n ≥ 1, for i in 0 to rank - 1。	输入
howmany_rank	int	多个rank维FFT之间的内存排布用howmany_rank维的howmany_dims数组来描述，howmany_rank表示每个要计算的rank维FFT变换的起始地址的内存访问模式所需的维数。 约束：0 ≤ howmany_rank ≤ 3。	输入

 版权所有 © 华为技术有限公司

参数名	数据类型	描述	输入/输出
howmany_dims	• 双精度：const kml_fft_iodim64 * • 单精度：const kml_fftf_iodim64 * • 半精度：const kml_ffth_iodim64*	howmany_dims是大小为howmany_rank的结构体数组，howmany_dims[i]包含以下成员： • ptrdiff_t n - howmany_rank维空间的第i维的待变换FFT的个数。 • ptrdiff_t is - 第i维的相继FFT输入序列之间的间隔。 • ptrdiff_t os - 第i维的相继FFT输出序列之间的间隔。	输入
in	• 双精度：kml_fft_complex* • 单精度：kml_fftf_complex* • 半精度：kml_ffth_complex*	输入需要变换的数据。	输入
out	• 双精度：double* • 单精度：float* • 半精度：__fp16*	输出快速傅里叶变换后的数据。	输出
flags	unsigned int	planning选项，描述ESTIMATE模式或PATIENT模式。 KML_FFT_ESTIMATE：ESTIMATE模式 KML_FFT_PATIENT：PATIENT模式	输入

依赖

C: "kfft.h"

示例

C interface：

```
int rank = 2;
kml_fft_iodim64 *dims;
dims = (kml_fft_iodim64*)kml_fft_malloc(sizeof(kml_fft_iodim64) * rank);
dims[0].n = 2;
dims[0].is = 2;
dims[0].os = 3;
dims[1].n = 3;
dims[1].is = 1;
dims[1].os = 1;
```

 版权所有 © 华为技术有限公司

```
int howmany_rank = 1;
kml_fft_iodim64 *howmany_dims;
howmany_dims = (kml_fft_iodim64*)kml_fft_malloc(sizeof(kml_fft_iodim64) * howmany_rank);
howmany_dims[0].n = 2;
howmany_dims[0].is = 2 * 2;
howmany_dims[0].os = 2 * 3;
double init[8][2] = {{120, 0}, {8, 8}, {0, 0}, {0, 16}, {0, 16}, {-8, 8}, {-8, 0}, {-8, 8}};
kml_fft_complex *in;
in = (kml_fft_complex*)kml_fft_malloc(sizeof(kml_fft_complex) * 8);
for (int i = 0; i < 8; i++) {
    in[i][0] = init[i][0];
    in[i][1] = init[i][1];
}
double *out;
out = (double*)kml_fft_malloc(sizeof(double) * 12);
kml_fft_plan plan;
plan = kml_fft_plan_guru64_dft_c2r(rank, dims, howmany_rank, howmany_dims, in, out,
KML_FFT_ESTIMATE);
kml_fft_execute_dft_c2r(plan, in, out);

kml_fft_destroy_plan(plan);
kml_fft_free(howmany_dims);
kml_fft_free(dims);
kml_fft_free(in);
kml_fft_free(out);

/*
 * out = {1.360000e+02, 7.043078e+01, 1.535692e+02, 1.360000e+02,
 *        1.258564e+02, 9.814359e+01, -4.000000e+01, -1.971281e+01,
 *        3.571281e+01, 8.000000e+00, 8.000000e+00, 8.000000e+00}
 */
```

Fortran interface：

```
INTEGER(C_INT) :: RANK = 2
INTEGER(C_INT) :: HOWMANY_RANK = 1
REAL(C_DOUBLE), DIMENSION(8, 2) :: INIT
TYPE(KML_FFT_IODIM64), POINTER :: DIMS(:), HOWMANY_DIMS(:)
TYPE(KML_FFT_COMPLEX), POINTER :: IN(:)
REAL(8), POINTER :: OUT(:)
TYPE(C_PTR) :: PIN, POUT, PDIMS, PHOWMANY_DIMS
INTEGER(C_SIZE_T) :: CSIZE, RSIZE
CSIZE = 16 * 8
RSIZE = 8 * 12
PIN = KML_FFT_MALLOC(CSIZE)
POUT = KML_FFT_MALLOC(RSIZE)
CALL C_F_POINTER(PIN, IN, SHAPE=[8])
CALL C_F_POINTER(POUT, OUT, SHAPE=[12])
CSIZE = 24 * 2
RSIZE = 24 * 1
PIODIMS = KML_FFT_MALLOC(CSIZE)
PHOWMANY_DIMS = KML_FFT_MALLOC(RSIZE)
CALL C_F_POINTER(PIODIMS, DIMS, SHAPE=[2])
CALL C_F_POINTER(PHOWMANY_DIMS, HOWMANY_DIMS, SHAPE=[1])
DIMS(0)%N = 2
DIMS(0)%IS = 2
DIMS(0)%OS = 3
DIMS(1)%N = 3
DIMS(1)%IS = 1
DIMS(1)%OS = 1
HOWMANY_DIMS(0)%N = 2
HOWMANY_DIMS(0)%IS = 2 * 2
HOWMANY_DIMS(0)%OS = 2 * 3
DATA INIT/120, 8, 0, 0, 0, -8, -8, -8,  0, 8, 0, 16, 16, 8, 0, 8/
INTEGER :: I = 1
DO WHILE(I <= 8)
    IN(I)%R = INIT(I, 0)
    IN(I)%I = INIT(I, 1)
    I = I + 1
END DO
```

 版权所有 © 华为技术有限公司

```
    TYPE(C_PTR) :: PLAN
    PLAN = KML_FFT_PLAN_GURU64_DFT_C2R(RANK, DIMS, HOWMANY_RANK, HOWMANY_DIMS, IN,
OUT, KML_FFT_ESTIMATE)
    CALL KML_FFT_EXECUTE_DFT_C2R(PLAN, IN, OUT);

    CALL KML_FFT_DESTROY_PLAN(PLAN)
    CALL KML_FFT_FREE(PHOWMANY_DIMS)
    CALL KML_FFT_FREE(PDIMS)
    CALL KML_FFT_FREE(PIN)
    CALL KML_FFT_FREE(POUT)
    !
    ! OUT = /1.360000E+02, 7.043078E+01, 1.535692E+02, 1.360000E+02,
    !       1.258564E+02, 9.814359E+01, -4.000000E+01, -1.971281E+01,
    !       3.571281E+01, 8.000000E+00, 8.000000E+00, 8.000000E+00/
    !
```

6.4.4.4.3.8 kml_fft(f/h)_plan_guru64_split_dft_c2r

建立多组数据序列n维C2R变换的plan。其中，单个FFT的数据序列不需要是连续的，可以以跨步的形式提供。split类接口的输入和输出分别存储在实部和虚部数组中。与kml_fft(f)_plan_guru_split_dft_r2c不同的是，kml_fft(f)_plan_guru64_split_dft_r2c允许部分参数是64位的整型。

接口定义

C interface：

kml_fft_plan kml_fft_plan_guru64_split_dft_c2r(int rank, const kml_fft_iodim64 *dims, int howmany_rank, const kml_fft_iodim64 *howmany_dims, double *ri, double *ii, double *out, unsigned flags);

kml_fftf_plan kml_fftf_plan_guru64_split_dft_c2r(int rank, const kml_fftf_iodim64 *dims, int howmany_rank, const kml_fftf_iodim64 *howmany_dims, float *ri, float *ii, float *out, unsigned flags);

kml_ffth_plan kml_ffth_plan_guru64_split_dft_c2r(int rank, const kml_ffth_iodim64 *dims, int howmany_rank, const kml_ffth_iodim64 *howmany_dims, __fp16 *ri, __fp16 *ii, __fp16 *out, unsigned flags);

Fortran interface：

RES = KML_FFT_PLAN_GURU64_SPLIT_DFT_C2R(RANK, DIMS, HOWMANY_RANK, HOWMANY_DIMS, RI, II, OUT, FLAGS);

RES = KML_FFTF_PLAN_GURU64_SPLIT_DFT_C2R(RANK, DIMS, HOWMANY_RANK, HOWMANY_DIMS, RI, II, OUT, FLAGS);

RES = KML_FFTH_PLAN_GURU64_SPLIT_DFT_C2R(RANK, DIMS, HOWMANY_RANK, HOWMANY_DIMS, RI, II, OUT, FLAGS);

返回值

函数返回一个kml_fft(f/h)_plan类型的结构体指针。将该对象作为参数传入**kml_fft(f/h)_execute**函数中，将对当前提供的输入ri，ii和输出out执行FFT变换。也可以通过将该对象作为参数传入 **kml_fft(f/h)_execute_split_dft_c2r**函数中以对新的输入ri，ii和输出out执行FFT变换。

如果函数返回非空指针，则表示plan创建成功，否则表示创建失败。

 版权所有 © 华为技术有限公司

参数

参数名	数据类型	描述	输入/输出
rank	int	单个FFT序列的维度。 约束：1 ≤ rank ≤ 3。	输入
dims	• 双精度：const kml_fft_iodim64 * • 单精度：const kml_fftf_iodim64 * • 半精度：const kml_ffth_iodim64*	dims是大小为rank的结构体数组，dims[i]包含以下成员： • ptrdiff_t n - 第i维FFT的长度。 • ptrdiff_t is - 第i维FFT输入序列的相继元素之间的间隔。 • ptrdiff_t os - 第i维FFT输出序列的相继元素之间的间隔。 约束：dims[i].n ≥ 1, for i in 0 to rank - 1。	输入
howmany_rank	int	多个rank维FFT之间的内存排布用howmany_rank维的howmany_dims数组来描述，howmany_rank表示每个要计算的rank维FFT变换的起始地址的内存访问模式所需的维数。 约束：0 ≤ howmany_rank ≤ 3。	输入
howmany_dims	• 双精度：const kml_fft_iodim64 * • 单精度：const kml_fftf_iodim64 * • 半精度：const kml_ffth_iodim64*	howmany_dims是大小为howmany_rank的结构体数组，howmany_dims[i]包含以下成员： • ptrdiff_t n - howmany_rank维空间的第i维的待变换FFT的个数。 • ptrdiff_t is - 第i维的相继FFT输入序列之间的间隔。 • ptrdiff_t os - 第i维的相继FFT输出序列之间的间隔。	输入
ri	• 双精度：double* • 单精度：float* • 半精度：__fp16*	输入待变换数据的实部。	输入
ii	• 双精度：double* • 单精度：float* • 半精度：__fp16*	输入待变换数据的虚部。	输入

版权所有 © 华为技术有限公司

参数名	数据类型	描述	输入/输出
out	• 双精度：double* • 单精度：float* • 半精度：__fp16*	输出快速傅里叶变换后的数据。	输出
flags	unsigned int	planning选项，描述ESTIMATE模式或PATIENT模式。 KML_FFT_ESTIMATE：ESTIMATE模式 KML_FFT_PATIENT：PATIENT模式	输入

依赖

C: "kfft.h"

示例

C interface：

```
    int rank = 2;
    kml_fft_iodim64 *dims;
    dims = (kml_fft_iodim64*)kml_fft_malloc(sizeof(kml_fft_iodim64) * rank);
    dims[0].n = 2;
    dims[0].is = 2;
    dims[0].os = 3;
    dims[1].n = 3;
    dims[1].is = 1;
    dims[1].os = 1;
    int howmany_rank = 1;
    kml_fft_iodim64 *howmany_dims;
    howmany_dims = (kml_fft_iodim64*)kml_fft_malloc(sizeof(kml_fft_iodim64) * howmany_rank);
    howmany_dims[0].n = 2;
    howmany_dims[0].is = 2 * 2;
    howmany_dims[0].os = 2 * 3;
    double init[8][2] = {{120, 0}, {8, 8}, {0, 0}, {0, 16}, {0, 16}, {-8, 8}, {-8, 0}, {-8, 8}};
    double *ri;
    ri = (double*)kml_fft_malloc(sizeof(double) * 8);
    double *ii;
    ii = (double*)kml_fft_malloc(sizeof(double) * 8);
    for (int i = 0; i < 8; i++) {
        ri[i] = init[i][0];
        ii[i] = init[i][1];
    }
    double *out;
    out = (double*)kml_fft_malloc(sizeof(double) * 12);
    kml_fft_plan plan;
    plan = kml_fft_plan_guru64_split_dft_c2r(rank, dims, howmany_rank, howmany_dims, ri, ii, out,
KML_FFT_ESTIMATE);
    kml_fft_execute_split_dft_c2r(plan, ri, ii, out);

    kml_fft_destroy_plan(plan);
    kml_fft_free(howmany_dims);
    kml_fft_free(dims);
    kml_fft_free(ri);
    kml_fft_free(ii);
    kml_fft_free(out);
```

 版权所有 © 华为技术有限公司

```
/*
 * out = {1.360000e+02, 7.043078e+01, 1.535692e+02, 1.360000e+02,
 *        1.258564e+02, 9.814359e+01, -4.000000e+01, -1.971281e+01,
 *        3.571281e+01, 8.000000e+00, 8.000000e+00, 8.000000e+00}
 */
```

Fortran interface：

```
  INTEGER(C_INT) :: RANK = 2
  INTEGER(C_INT) :: HOWMANY_RANK = 1
  REAL(C_DOUBLE), DIMENSION(8, 2) :: INIT
  TYPE(KML_FFT_IODIM64), POINTER :: DIMS(:), HOWMANY_DIMS(:)
  REAL(8), POINTER :: RI(:), II(:), OUT(:)
  TYPE(C_PTR) :: PRI,PII, POUT, PDIMS, PHOWMANY_DIMS
  INTEGER(C_SIZE_T) :: CSIZE, RSIZE
  CSIZE = 8 * 8
  RSIZE = 8 * 12
  PRI = KML_FFT_MALLOC(CSIZE)
  PII = KML_FFT_MALLOC(CSIZE)
  POUT = KML_FFT_MALLOC(RSIZE)
  CALL C_F_POINTER(PRI, IN, SHAPE=[8])
  CALL C_F_POINTER(PII, IN, SHAPE=[8])
  CALL C_F_POINTER(POUT, OUT, SHAPE=[12])
  CSIZE = 24 * 2
  RSIZE = 24 * 1
  PIODIMS = KML_FFT_MALLOC(CSIZE)
  PHOWMANY_DIMS = KML_FFT_MALLOC(RSIZE)
  CALL C_F_POINTER(PIODIMS, DIMS, SHAPE=[2])
  CALL C_F_POINTER(PHOWMANY_DIMS, HOWMANY_DIMS, SHAPE=[1])
  DIMS(0)%N = 2
  DIMS(0)%IS = 2
  DIMS(0)%OS = 3
  DIMS(1)%N = 3
  DIMS(1)%IS = 1
  DIMS(1)%OS = 1
  HOWMANY_DIMS(0)%N = 2
  HOWMANY_DIMS(0)%IS = 2 * 2
  HOWMANY_DIMS(0)%OS = 2 * 3
  DATA INIT/120, 8, 0, 0, 0, -8, -8, -8,  0, 8, 0, 16, 16, 8, 0, 8/
  INTEGER :: I = 1
  DO WHILE(I <= 8)
     RI(I) = INIT(I, 0)
     II(I) = INIT(I, 1)
     I = I + 1
  END DO
  TYPE(C_PTR) :: PLAN
  PLAN = KML_FFT_PLAN_GURU64_SPLIT_DFT_C2R(RANK, DIMS, HOWMANY_RANK, HOWMANY_DIMS,
RI, II, OUT, KML_FFT_ESTIMATE)
  CALL KML_FFT_EXECUTE_SPLIT_DFT_C2R(PLAN, RI, II, OUT);

  CALL KML_FFT_DESTROY_PLAN(PLAN)
  CALL KML_FFT_FREE(PHOWMANY_DIMS)
  CALL KML_FFT_FREE(PDIMS)
  CALL KML_FFT_FREE(PRI)
  CALL KML_FFT_FREE(PII)
  CALL KML_FFT_FREE(POUT)
  !
  ! OUT = /1.360000E+02, 7.043078E+01, 1.535692E+02, 1.360000E+02,
  !       1.258564E+02, 9.814359E+01, -4.000000E+01, -1.971281E+01,
  !       3.571281E+01, 8.000000E+00, 8.000000E+00, 8.000000E+00/
  !
```

6.4.4.4.3.9 kml_fft(f/h)_plan_many_dft_c2r

创建howmany组数据序列n维C2R变换的plan。其中，单个FFT的数据序列不需要是连续的，可以以跨步的形式提供。

 版权所有 © 华为技术有限公司

接口定义

C interface：

kml_fft_plan kml_fft_plan_many_dft_c2r(int rank, const int *n, int howmany, kml_fft_complex *in, const int *inembed, int istride, int idist, double *out, const int *onembed, int ostride, int odist, unsigned flags);

kml_fftf_plan kml_fftf_plan_many_dft_c2r(int rank, const int *n, int howmany, kml_fftf_complex *in, const int *inembed, int istride, int idist, float *out, const int *onembed, int ostride, int odist, unsigned flags);

kml_ffth_plan kml_ffth_plan_many_dft_c2r(int rank, const int *n, int howmany, kml_ffth_complex *in, const int *inembed, int istride, int idist, __fp16 *out, const int *onembed, int ostride, int odist, unsigned flags);

Fortran interface：

RES = KML_FFT_PLAN_MANY_DFT_C2R(RANK, N, HOWMANY, IN, INEMBED, ISTRIDE, IDIST, OUT, ONEMBED, OSTRIDE, ODIST, FLAGS);

RES = KML_FFTF_PLAN_MANY_DFT_C2R(RANK, N, HOWMANY, IN, INEMBED, ISTRIDE, IDIST, OUT, ONEMBED, OSTRIDE, ODIST, FLAGS);

RES = KML_FFTH_PLAN_MANY_DFT_C2R(RANK, N, HOWMANY, IN, INEMBED, ISTRIDE, IDIST, OUT, ONEMBED, OSTRIDE, ODIST, FLAGS)

返回值

函数返回一个kml_fft(f/h)_plan类型的结构体指针。将该对象作为参数传入**kml_fft(f/h)_execute**函数中，将对当前提供的输入in和输出out执行FFT变换。也可以通过将该对象作为参数传入 **kml_fft(f/h)_execute_dft_c2r**函数中以对新的输入in和输出out执行FFT变换。

如果函数返回非空指针，则表示plan创建成功，否则表示创建失败。

参数

参数名	数据类型	描述	输入/输出
rank	int	FFT变换的维度是rank，约束：1 ≤ rank ≤ 3。	输入
n	const int*	n为维度为rank的数组，包含FFT序列每一维度的大小，约束：n[i] ≥ 1, for i in 0 to rank - 1。	输入
howmany	int	howmany表示指定多少个多维FFT变换。	输入

版权所有 © 华为技术有限公司

参数名	数据类型	描述	输入/输出
in	• 双精度：kml_fft_complex* • 单精度：kml_fftf_complex* • 半精度：kml_ffth_complex*	输入需要变换的数据。	输入
inembed	const int*	inembed是大小为rank的数组或NULL，该数组表示FFT数据in存储的更大空间的每一维度的大小。 约束：inembed[i] ≥ n[i] for i in 0, rank-1 或者：如果inembed == NULL，则可以认为inembed与n相等	输入
istride	int	FFT输入序列的相继元素之间的间隔。	输入
idist	int	idist表示输入的每个FFT序列的间隔。	输入
out	• 双精度：double* • 单精度：float* 半精度：__fp16*	输出快速傅里叶变换后的数据。	输出
onembed	const int*	onembed是大小为rank的数组或者NULL，该数组表示FFT数据out存储的更大空间的每一维度的大小。 约束：onembed[i] ≥ n[i] for i in 0, rank-1 或者：如果onembed == NULL，则可以认为onembed与n相等	输入
ostride	int	FFT输出序列的相继元素之间的间隔。	输入
odist	int	odist表示输出的每个FFT序列的间隔。	输入
flags	unsigned int	planning选项，描述ESTIMATE模式或PATIENT模式。 KML_FFT_ESTIMATE：ESTIMATE模式 KML_FFT_PATIENT：PATIENT模式	输入

版权所有 © 华为技术有限公司

依赖

C: "kfft.h"

示例

C interface：

```
int rank = 1;
int *n;
n = (int*)kml_fft_malloc(sizeof(int) * rank);
n[0] = 4;
int howmany = 3;
int istride = 1;
int ostride = 1;
int idist = n[0] / 2 + 1;
int odist = 4;
double init[9][2] = {{120, 0}, {8, 8}, {0, 0}, {0, 16}, {0, 16}, {-8, 8}, {-8, 0}, {-8, 8}, {-16, 0}};
kml_fft_complex *in;
in = (kml_fft_complex*)kml_fft_malloc(sizeof(kml_fft_complex) * (n[0] / 2 + 1) * howmany);
for (int i = 0; i < (n[0] / 2 + 1) * howmany; i++){
    in[i][0] = init[i][0];
    in[i][1] = init[i][1];
}
double *out;
out = (double*)kml_fft_malloc(sizeof(double) * n[0] * howmany);
kml_fft_plan plan;
plan = kml_fft_plan_many_dft_c2r(rank, n, howmany, in, NULL, istride, idist, out, NULL, ostride, odist,
KML_FFT_ESTIMATE);
kml_fft_execute_dft_c2r(plan, in, out);

kml_fft_destroy_plan(plan);
kml_fft_free(n);
kml_fft_free(in);
kml_fft_free(out);

/*
 * out = {1.360000e+02, 1.040000e+02, 1.040000e+02, 1.360000e+02,
 *        -8.000000e+00, -2.400000e+01, -8.000000e+00, 4.000000e+01,
 *        -4.000000e+01, -8.000000e+00, -8.000000e+00, 2.400000e+01}
 */
```

Fortran interface：

```
INTEGER(C_INT) :: RANK = 1
INTEGER(C_INT) :: N(1)
N(1) = 4
REAL(C_DOUBLE), DIMENSION(9, 2) :: INIT
TYPE(KML_FFT_COMPLEX), POINTER :: IN(:),
REAL(C_DOUBLE), POINTER :: OUT(:)
INTEGER, POINTER :: INEMBED(:), ONEMBED(:)
TYPE(C_PTR) :: PIN, POUT
INTEGER(C_INT) :: HOWMANY = 3
INTEGER(C_INT) :: ISTRIDE = 1
INTEGER(C_INT) :: OSTRIDE = 1
INTEGER(C_INT) :: IDIST = 3
INTEGER(C_INT) :: ODIST = 4
INTEGER(C_SIZE_T) :: CSIZE, RSIZE
CSIZE = 16 * 9
RSIZE = 8 * 12
PIN = KML_FFT_MALLOC(SIZE)
POUT = KML_FFT_MALLOC(SIZE)
CALL C_F_POINTER(PIN, IN, SHAPE=[9])
CALL C_F_POINTER(POUT, OUT, SHAPE=[12])
CALL C_F_POINTER(C_NULL_PTR, INEMBED, SHAPE=[0])
CALL C_F_POINTER(C_NULL_PTR, ONEMBED, SHAPE=[0])

DATA INIT/120, 8, 0, 0, 0, -8, -8, -8, -16, 0, 8, 0, 16, 16, 8, 0, 0, 0/
```


版权所有 © 华为技术有限公司

```
  INTEGER I = 1
  DO WHILE(I <= 9)
     IN(I)%R = INIT(I, 0)
     IN(I)%I = INIT(I, 1)
     I = I + 1
  END DO
  TYPE(C_PTR) :: PLAN
  PLAN = KML_FFT_PLAN_MANY_DFT_C2R(RANK, N, HOWMANY, IN, INEMBED, ISTRIDE, IDIST, OUT,
ONEMBED, OSTRIDE, ODIST, KML_FFT_ESTIMATE)
  CALL KML_FFT_EXECUTE_DFT_C2R(PLAN, IN, OUT)

  CALL KML_FFT_DESTROY_PLAN(PLAN)
  CALL KML_FFT_FREE(PIN)
  CALL KML_FFT_FREE(POUT)
  !
  ! OUT = /1.360000E+02, 1.040000E+02, 1.040000E+02, 1.360000E+02,
  !       -8.000000E+00, -2.400000E+01, -8.000000E+00, 4.000000E+01,
  !       -4.000000E+01, -8.000000E+00, -8.000000E+00, 2.400000E+01/
  !
```

6.4.4.4.3.10 kml_ffth_plan_dft_scale_c2r

建立单个连续数据序列n维C2R变换的plan。

接口定义

C interface：

kml_ffth_plan kml_ffth_plan_dft_scale_c2r(int rank, const int *n, kml_ffth_complex *in, float *out, unsigned flags);

返回值

函数返回一个kml_ffth_plan类型的结构体指针。将该对象作为参数传入 **kml_fft(f/h)_execute**函数中，将对当前提供的输入in和输出out执行FFT变换。也可以通过将该对象作为参数传入 **kml_ffth_execute_dft_scale_c2r**函数中以对新的输入in和输出out执行FFT变换。

如果函数返回非空指针，则表示plan创建成功，否则表示创建失败。

参数

参数名	数据类型	描述	输入/输出
rank	int	FFT变换的维度是rank，约束：1 ≤ rank ≤ 3。	输入
n	const int*	n为维度为rank的数组，包含FFT序列每一维度的大小，约束：n[i] ≥ 1, for i in 0 to rank - 1。	输入
in	• kml_ffth_complex*	输入需要变换的数据。	输入
out	• float*	输出快速傅里叶变换后的数据。	输出

版权所有 © 华为技术有限公司

参数名	数据类型	描述	输入/输出
flags	unsigned int	planning选项，描述ESTIMATE模式或PATIENT模式。 KML_FFT_ESTIMATE：ESTIMATE模式 KML_FFT_PATIENT：PATIENT模式	输入

依赖

C: "kfft.h"

示例

C interface：

```
int rank = 2;
int *n;
n = (int*)kml_ffth_malloc(sizeof(int) * rank);
n[0] = 2;
n[1] = 4;
_fp16 init[6][2] = {{120, 0}, {8, 8}, {0, 0}, {0, 16}, {0, 16}, {-8, 8}};
kml_ffth_complex *in;
in = (kml_ffth_complex*)kml_ffth_malloc(sizeof(kml_ffth_complex) * n[0] * (n[1] / 2 + 1));
for (int i = 0; i < n[0] * (n[1] / 2 + 1); i++) {
    in[i][0] = init[i][0];
    in[i][1] = init[i][1];
}
float *out;
out = (float*)kml_fft_malloc(sizeof(float) * n[0] * n[1]);
kml_ffth_plan plan;
plan = kml_ffth_plan_dft_scale_c2r(rank, n, in, out, KML_FFT_ESTIMATE);
kml_ffth_execute_dft_scale_c2r(plan, in, out);

kml_ffth_destroy_plan(plan);
kml_ffth_free(n);
kml_ffth_free(in);
kml_ffth_free(out);

/*
 * out = {1.280000e+02, 8.000000e+01, 9.600000e+01, 1.760000e+02,
 *        1.440000e+02, 1.280000e+02, 1.120000e+02, 9.600000e+01}
 */
```

6.4.4.4.3.11 kml_ffth_plan_dft_scale_c2r_1d

建立单个连续数据序列1维C2R变换的plan。

接口定义

C interface：

kml_ffth_plan kml_ffth_plan_dft_scale_c2r_1d(int n, kml_ffth_complex *in, float *out, unsigned flags);

 版权所有 © 华为技术有限公司

返回值

函数返回一个kml_ffth_plan类型的结构体指针。将该对象作为参数传入 **kml_fft(f/h)_execute**函数中，将对当前提供的输入in和输出out执行FFT变换。也可以通过将该对象作为参数传入 **kml_ffth_execute_dft_scale_c2r**函数中以对新的输入in和输出out执行FFT变换。

如果函数返回非空指针，则表示plan创建成功，否则表示创建失败。

参数

参数名	数据类型	描述	输入/输出
n	int	输入FFT序列的大小，约束：n ≥ 1。	输入
in	• kml_ffth_complex*	输入需要变换的数据。	输入
out	• float*	输出快速傅里叶变换后的数据。	输出
flags	unsigned int	planning选项，描述ESTIMATE模式或PATIENT模式。 KML_FFT_ESTIMATE：ESTIMATE模式 KML_FFT_PATIENT：PATIENT模式	输入

依赖

C: "kfft.h"

示例

C interface：

```
int n0 = 4;
__fp16 init[3][2] = {{120, 0}, {8, 8}, {0, 0}};
kml_ffth_complex *in;
in = (kml_ffth_complex*)kml_ffth_malloc(sizeof(kml_ffth_complex) * (n0 / 2 + 1));
for (int i = 0; i < (n0 / 2 + 1); i++) {
    in[i][0] = init[i][0];
    in[i][1] = init[i][1];
}
float *out;
out = (float*)kml_ffth_malloc(sizeof(float) * n0);
kml_ffth_plan plan;
plan = kml_ffth_plan_dft_scale_c2r_1d(n0, in, out, KML_FFT_ESTIMATE);
kml_ffth_execute_dft_scale_c2r(plan, in, out);

kml_ffth_destroy_plan(plan);
kml_ffth_free(in);
kml_ffth_free(out);

/*
 * out = {1.360000e+02, 1.040000e+02, 1.040000e+02, 1.360000e+02}
 */
```

 版权所有 © 华为技术有限公司

6.4.4.4.3.12 kml_ffth_plan_dft_scale_c2r_2d

建立单个连续数据序列2维C2R变换的plan。

接口定义

C interface：

kml_ffth_plan kml_ffth_plan_dft_scale_c2r_2d(int n0, int n1, kml_ffth_complex *in, float *out, unsigned flags);

返回值

函数返回一个kml_ffth_plan类型的结构体指针。将该对象作为参数传入 **kml_fft(f/h)_execute**函数中，将对当前提供的输入in和输出out执行FFT变换。也可以通过将该对象作为参数传入 **kml_ffth_execute_dft_scale_c2r**函数中以对新的输入in和输出out执行FFT变换。

如果函数返回非空指针，则表示plan创建成功，否则表示创建失败。

参数

参数名	数据类型	描述	输入/输出
n0	int	输入FFT序列第一维的大小，约束：n0 ≥ 1。	输入
n1	int	输入FFT序列第二维的大小，约束：n1 ≥ 1。	输入
in	kml_ffth_complex*	输入需要变换的数据。	输入
out	float*	输出快速傅里叶变换后的数据。	输出
flags	unsigned int	planning选项，描述ESTIMATE模式或PATIENT模式。 KML_FFT_ESTIMATE：ESTIMATE模式 KML_FFT_PATIENT：PATIENT模式	输入

依赖

C: "kfft.h"

示例

C interface：

```
    int n0 = 4;
    int n1 = 4;
    __fp16 init[12][2] = {{120, 0}, {8, 8}, {0, 0}, {0, 16}, {0, 16}, {-8, 8}, {-8, 0}, {-8, 8}, {-16, 0}, {0, -16}, {-40,
8}, {-8, -8}};
    kml_ffth_complex *in;
    in = (kml_ffth_complex*)kml_ffth_malloc(sizeof(kml_ffth_complex) * n0 * (n1 / 2 + 1));
```

 版权所有 © 华为技术有限公司

```
for (int i = 0; i < n0 * (n1 / 2 + 1); i++){
    in[i][0] = init[i][0];
    in[i][1] = init[i][1];
}
float *out;
out = (float*)kml_ffth_malloc(sizeof(float) * n0 * n1);
kml_ffth_plan plan;
plan = kml_ffth_plan_dft_scale_c2r_2d(n0, n1, in , out , KML_FFT_ESTIMATE);
kml_ffth_execute_dft_scale_c2r(plan, in, out);

kml_ffth_destroy_plan(plan);
kml_ffth_free(in);
kml_ffth_free(out);

/*
 * out = {0.000000e+00, 6.400000e+01, 1.600000e+02, 2.240000e+02,
 *        1.120000e+02, 1.600000e+01, 8.000000e+01, 1.760000e+02,
 *        1.920000e+02, 1.280000e+02, 3.200000e+01, 9.600000e+01,
 *        2.400000e+02, 2.080000e+02, 1.440000e+02, 4.800000e+01}
 */
```

6.4.4.4.3.13 kml_ffth_plan_many_dft_scale_c2r

创建howmany组数据序列n维C2R变换的plan。其中，单个FFT的数据序列不需要是连续的，可以以跨步的形式提供。

接口定义

C interface：

kml_ffth_plan kml_ffth_plan_many_dft_scale_c2r(int rank, const int *n, int howmany, kml_ffth_complex *in, const int *inembed, int istride, int idist, float *out, const int *onembed, int ostride, int odist, unsigned flags);

返回值

函数返回一个kml_ffth_plan类型的结构体指针。将该对象作为参数传入 **kml_fft(f/h)_execute**函数中，将对当前提供的输入in和输出out执行FFT变换。也可以通过将该对象作为参数传入 **kml_ffth_execute_dft_scale_c2r**函数中以对新的输入in和输出out执行FFT变换。

如果函数返回非空指针，则表示plan创建成功，否则表示创建失败。

参数

参数名	数据类型	描述	输入/输出
rank	int	FFT变换的维度是rank，约束：1 ≤ rank ≤ 3。	输入
n	const int*	n为维度为rank的数组，包含FFT序列每一维度的大小，约束：n[i] ≥ 1, for i in 0 to rank - 1。	输入
howmany	int	howmany表示指定多少个多维FFT变换。	输入

版权所有 © 华为技术有限公司

参数名	数据类型	描述	输入/输出
in	• kml_ffth_complex*	输入需要变换的数据。	输入
inembed	const int*	inembed是大小为rank的数组或NULL，该数组表示FFT数据in存储的更大空间的每一维度的大小。 约束：inembed[i] ≥ n[i] for i in 0, rank-1 或者：如果inembed == NULL，则可以认为inembed与n相等	输入
istride	int	FFT输入序列的相继元素之间的间隔。	输入
idist	int	idist表示输入的每个FFT序列的间隔。	输入
out	• float*	输出快速傅里叶变换后的数据。	输出
onembed	const int*	onembed是大小为rank的数组或者NULL，该数组表示FFT数据out存储的更大空间的每一维度的大小。 约束：onembed[i] ≥ n[i] for i in 0, rank-1 或者：如果onembed == NULL，则可以认为onembed与n相等	输入
ostride	int	FFT输出序列的相继元素之间的间隔。	输入
odist	int	odist表示输出的每个FFT序列的间隔。	输入
flags	unsigned int	planning选项，描述ESTIMATE模式或PATIENT模式。 KML_FFT_ESTIMATE：ESTIMATE模式 KML_FFT_PATIENT：PATIENT模式	输入

依赖

C: "kfft.h"

示例

C interface：

```
int rank = 1;
int *n;
n = (int*)kml_ffth_malloc(sizeof(int) * rank);
n[0] = 4;
```

 版权所有 © 华为技术有限公司

```
    int howmany = 3;
    int istride = 1;
    int ostride = 1;
    int idist = n[0] / 2 + 1;
    int odist = 4;
    __fp16 init[9][2] = {{120, 0}, {8, 8}, {0, 0}, {0, 16}, {0, 16}, {-8, 8}, {-8, 0}, {-8, 8}, {-16, 0}};
    kml_ffth_complex *in;
    in = (kml_ffth_complex*)kml_ffth_malloc(sizeof(kml_ffth_complex) * (n[0] / 2 + 1) * howmany);
    for (int i = 0; i < (n[0] / 2 + 1) * howmany; i++){
        in[i][0] = init[i][0];
        in[i][1] = init[i][1];
    }
    float *out;
    out = (float*)kml_ffth_malloc(sizeof(float) * n[0] * howmany);
    kml_ffth_plan plan;
    plan = kml_ffth_plan_many_dft_scale_c2r(rank, n, howmany, in, NULL, istride, idist, out, NULL, ostride,
odist, KML_FFT_ESTIMATE);
    kml_ffth_execute_dft_scale_c2r(plan, in, out);

    kml_ffth_destroy_plan(plan);
    kml_ffth_free(n);
    kml_ffth_free(in);
    kml_ffth_free(out);

    /*
     * out = {1.360000e+02, 1.040000e+02, 1.040000e+02, 1.360000e+02,
     *        -8.000000e+00, -2.400000e+01, -8.000000e+00, 4.000000e+01,
     *        -4.000000e+01, -8.000000e+00, -8.000000e+00, 2.400000e+01}
     */
```

6.4.4.4.4 R2R 变换

6.4.4.4.4.1 kml_fft(f/h)_plan_r2r

建立单个连续数据序列n维R2R变换的plan。

接口定义

C interface：

kml_fft_plan kml_fft_plan_r2r(int rank, const int *n, double *in, double *out, const kml_fft_r2r_kind *kind, unsigned flags);

kml_fftf_plan kml_fftf_plan_r2r(int rank, const int *n, float *in, float *out, const kml_fftf_r2r_kind *kind, unsigned flags);

kml_ffth_plan kml_ffth_plan_r2r(int rank, const int *n, __fp16 *in, __fp16 *out, const kml_ffth_r2r_kind *kind, unsigned flags);

Fortran interface：

RES = KML_FFT_PLAN_R2R(RANK, N, IN, OUT, KIND, FLAGS);

RES = KML_FFTF_PLAN_R2R(RANK, N, IN, OUT, KIND, FLAGS);

RES = KML_FFTH_PLAN_R2R(RANK, N, IN, OUT, KIND, FLAGS);

KML_FFT_REDFT11和KML_FFT_RODFT11只支持长度为4的整数倍大小的序列，其他变换类型只支持长度为2的整数倍大小的序列。

返回值

函数返回一个kml_fft(f/h)_plan类型的结构体指针。将该对象作为参数传入**kml_fft(f/h)_execute**函数中，将对当前提供的输入in和输出out执行FFT变换。也可

 版权所有 © 华为技术有限公司

以通过将该对象作为参数传入 **kml_fft(f/h)_execute_r2r**函数中以对新的输入in和输出out执行FFT变换。

如果函数返回非空指针，则表示plan创建成功，否则表示创建失败。

参数

参数名	数据类型	描述	输入/输出
rank	int	FFT变换的维度是rank，约束：1 ≤ rank ≤ 3。	输入
n	const int*	n为维度为rank的数组，包含FFT序列每一维度的大小，约束：n[i] ≥ 1, for i in 0 to rank - 1。	输入
in	• 双精度：double* • 单精度：float* • 半精度：__fp16*	输入需要变换的数据。	输入
out	• 双精度：double* • 单精度：float* • 半精度：__fp16*	输出快速傅里叶变换后的数据。	输出
kind	• 双精度：const kml_fft_r2r_kind * • 单精度：const kml_fftf_r2r_kind* • 半精度：const kml_ffth_r2r_kind*	kind是大小为rank的数组，包含FFT序列每一维度的R2R变换类型，kind[i] (for i in 0 to rank - 1)有以下可选值： • KML_FFT_R2HC • KML_FFT_HC2R • KML_FFT_DHT • KML_FFT_REDFT00 • KML_FFT_REDFT01 • KML_FFT_REDFT10 • KML_FFT_REDFT11 • KML_FFT_RODFT00 • KML_FFT_RODFT01 • KML_FFT_RODFT10 • KML_FFT_RODFT11	输入
flags	unsigned int	planning选项，描述ESTIMATE模式或PATIENT模式。 KML_FFT_ESTIMATE：ESTIMATE模式 KML_FFT_PATIENT：PATIENT模式	输入

 版权所有 © 华为技术有限公司

依赖

C: "kfft.h"

示例

C interface：

```
int rank = 2;
int *n;
n = (int*)kml_fft_malloc(sizeof(int) * rank);
n[0] = 2;
n[1] = 4;
double init[8] = {1, 2, 3, 4, 5, 6, 7, 8};
double *in;
in = (double*)kml_fft_malloc(sizeof(double) * n[0] * n[1]);
for (int i = 0; i < n[0] * n[1]; i++) {
    in[i] = init[i];
}
double *out;
out = (double*)kml_fft_malloc(sizeof(double) * n[0] * n[1]);
kml_fft_r2r_kind *kind;
kind = (kml_fft_r2r_kind*)kml_fft_malloc(sizeof(kml_fft_r2r_kind) * rank);
kind[0] = KML_FFT_DHT;
kind[1] = KML_FFT_DHT;
kml_fft_plan plan;
plan = kml_fft_plan_r2r(rank, n, in, out, kind, KML_FFT_ESTIMATE);
kml_fft_execute_r2r(plan, in, out);

kml_fft_destroy_plan(plan);
kml_fft_free(n);
kml_fft_free(kind);
kml_fft_free(in);
kml_fft_free(out);

/*
 * out = {3.600000e+01, -8.000000e+00, -4.000000e+00, 0.000000e+00,
 *        -1.600000e+01, 0.000000e+00, 0.000000e+00, 0.000000e+00}
 */
```

Fortran interface：

```
INTEGER(C_INT) :: RANK = 2
INTEGER(C_INT) :: N(2)
INTEGER(C_INT) :: KIND(2)
REAL(C_DOUBLE) :: INIT(8)
REAL(C_DOUBLE), POINTER :: IN(:), OUT(:)
TYPE(C_PTR) :: PIN, POUT
INTEGER(C_SIZE_T) :: SIZE
SIZE = N(1) * N(2)
DATA N/2, 4/
DATA KIND/KML_FFT_DHT, KML_FFT_DHT/
PIN = KML_FFT_MALLOC(8 * SIZE)
POUT = KML_FFT_MALLOC(8 * SIZE)
CALL C_F_POINTER(PIN, IN, SHAPE=[SIZE])
CALL C_F_POINTER(POUT, OUT, SHAPE=[SIZE])
DATA INIT/1, 2, 3, 4, 5, 6, 7, 8/
INTEGER :: I = 1
DO WHILE(I <=  SIZE)
   IN(I) = INIT(I)
   I = I + 1
END DO

TYPE(C_PTR) :: PLAN
PLAN = KML_FFT_PLAN_R2R(RANK, N, IN, OUT, KIND, KML_FFT_ESTIMATE)
```

 版权所有 © 华为技术有限公司

```
CALL KML_FFT_EXECUTE_R2R(PLAN, IN, OUT)

CALL KML_FFT_DESTROY_PLAN(PLAN)
CALL KML_FFT_FREE(PIN)
CALL KML_FFT_FREE(POUT)

!
! OUT = /3.600000E+01, -8.000000E+00, -4.000000E+00, 0.000000E+00,
!       -1.600000E+01, 0.000000E+00, 0.000000E+00, 0.000000E+00/
!
```

6.4.4.4.4.2 kml_fft(f/h)_plan_r2r_1d

建立单个连续数据序列1维R2R变换的plan。

接口定义

C interface：

kml_fft_plan kml_fft_plan_r2r_1d(int n, double *in, double *out, kml_fft_r2r_kind kind, unsigned flags);

kml_fftf_plan kml_fftf_plan_r2r_1d(int n, float *in, float *out, kml_fftf_r2r_kind kind, unsigned flags);

kml_ffth_plan kml_ffth_plan_r2r_1d(int n, __fp16 *in, __fp16 *out, kml_ffth_r2r_kind kind, unsigned flags);

Fortran interface：

RES = KML_FFT_PLAN_R2R_ID(N, IN, OUT, KIND, FLAGS);

RES = KML_FFTF_PLAN_R2R_ID(N, IN, OUT, KIND, FLAGS);

RES = KML_FFTH_PLAN_R2R_ID(N, IN, OUT, KIND, FLAGS);

KML_FFT_REDFT11和KML_FFT_RODFT11只支持长度为4的整数倍大小的序列，其他变换类型只支持长度为2的整数倍大小的序列。

返回值

函数返回一个kml_fft(f/h)_plan类型的结构体指针。将该对象作为参数传入**kml_fft(f/h)_execute**函数中，将对当前提供的输入in和输出out执行FFT变换。也可以通过将该对象作为参数传入 **kml_fft(f/h)_execute_r2r**函数中以对新的输入in和输出out执行FFT变换。

如果函数返回非空指针，则表示plan创建成功，否则表示创建失败。

参数

参数名	数据类型	描述	输入/输出
n	int	输入FFT序列的大小，约束：n ≥ 1。	输入

版权所有 © 华为技术有限公司

参数名	数据类型	描述	输入/输出
in	• 双精度：double* • 单精度：float* • 半精度：__fp16*	输入需要变换的数据。	输入
out	• 双精度：double* • 单精度：float* • 半精度：__fp16*	输出快速傅里叶变换后的数据。	输出
kind	• 双精度：kml_fft_r2r_kind • 单精度：kml_fftf_r2r_kind • 半精度：const kml_ffth_r2r_kind*	kind表示FFT序列的R2R变换类型，有以下可选值： • KML_FFT_R2HC • KML_FFT_HC2R • KML_FFT_DHT • KML_FFT_REDFT00 • KML_FFT_REDFT01 • KML_FFT_REDFT10 • KML_FFT_REDFT11 • KML_FFT_RODFT00 • KML_FFT_RODFT01 • KML_FFT_RODFT10 • KML_FFT_RODFT11	输入
flags	unsigned int	planning选项，描述ESTIMATE模式或PATIENT模式。 KML_FFT_ESTIMATE：ESTIMATE模式 KML_FFT_PATIENT：PATIENT模式	输入

依赖

C: "kfft.h"

示例

C interface：

```
int N = 16;
double *in;
double *out;
kml_fft_plan plan;
in = (double*)kml_fft_malloc(sizeof(double) * N);
out = (double*)kml_fft_malloc(sizeof(double) * N);
for (int i = 0; i < N; i++){
```


版权所有 © 华为技术有限公司

```
    in[i] = (double)i;
}

plan = kml_fft_plan_r2r_1d(N, in , out, KML_FFT_REDFT10, KML_FFT_ESTIMATE);
kml_fft_execute_r2r(plan, in, out);

kml_fft_destroy_plan(plan);
kml_fft_free(in);
kml_fft_free(out);

/*
 * out = {2.400000e+02, -1.035857e+02, 0.000000e+00, -1.135629e+01,
 *        0.000000e+00, -3.968777e+00, 0.000000e+00, -1.920738e+00,
 *        0.000000e+00, -1.061666e+00, 0.000000e+00, -6.060758e-01,
 *        0.000000e+00, -3.169964e-01, 0.000000e+00, -9.896796e-02}
 */
```

Fortran interface：

```
INTEGER(C_INT) :: N = 16
REAL(C_DOUBLE) :: INIT(16)
REAL(C_DOUBLE), POINTER :: IN(:), OUT(:)
TYPE(C_PTR) :: PIN, POUT
INTEGER(C_SIZE_T) :: SIZE
SIZE = N
DATA KIND/KML_FFT_DHT, KML_FFT_DHT/
PIN = KML_FFT_MALLOC(8 * SIZE)
POUT = KML_FFT_MALLOC(8 * SIZE)
CALL C_F_POINTER(PIN, IN, SHAPE=[SIZE])
CALL C_F_POINTER(POUT, OUT, SHAPE=[SIZE])
DATA INIT/1, 2, 3, 4, 5, 6, 7, 8/
INTEGER :: I = 1
DO WHILE(I <=  SIZE)
   IN(I) = INIT(I)
   I = I + 1
END DO
TYPE(C_PTR) :: PLAN
PLAN = KML_FFT_PLAN_R2R_1D(N, IN , OUT, KML_FFT_REDFT10, KML_FFT_ESTIMATE)
CALL KML_FFT_EXECUTE_R2R(PLAN, IN, OUT)

CALL KML_FFT_DESTROY_PLAN(PLAN)
CALL KML_FFT_FREE(PIN)
CALL KML_FFT_FREE(POUT)
!
! OUT = /2.400000E+02, -1.035857E+02, 0.000000E+00, -1.135629E+01,
!       0.000000E+00, -3.968777E+00, 0.000000E+00, -1.920738E+00,
!       0.000000E+00, -1.061666E+00, 0.000000E+00, -6.060758E-01,
!       0.000000E+00, -3.169964E-01, 0.000000E+00, -9.896796E-02/
!
```

6.4.4.4.4.3 kml_fft(f/h)_plan_r2r_2d

建立单个连续数据序列2维R2R变换的plan。

接口定义

C interface：

kml_fft_plan kml_fft_plan_r2r_2d(int n0, int n1, double *in, double *out, kml_fft_r2r_kind kind0, kml_fft_r2r_kind kind1, unsigned flags);

kml_fftf_plan kml_fftf_plan_r2r_2d(int n0, int n1, float *in, float *out, kml_fftf_r2r_kind kind0, kml_fftf_r2r_kind kind1, unsigned flags);

kml_ffth_plan kml_ffth_plan_r2r_2d(int n0, int n1, __fp16 *in, __fp16 *out, kml_ffth_r2r_kind kind0, kml_ffth_r2r_kind kind1, unsigned flags);

 版权所有 © 华为技术有限公司

Fortran interface：

RES = KML_FFT_PLAN_R2R_2D(N0, N1, IN, OUT, KIND0, KIND1, FLAGS);

RES = KML_FFTF_PLAN_R2R_2D(N0, N1, IN, OUT, KIND0, KIND1, FLAGS);

RES = KML_FFTH_PLAN_R2R_2D(N0, N1, IN, OUT, KIND0, KIND1, FLAGS);

KML_FFT_REDFT11和KML_FFT_RODFT11只支持长度为4的整数倍大小的序列，其他变换类型只支持长度为2的整数倍大小的序列。

返回值

函数返回一个kml_fft(f/h)_plan类型的结构体指针。将该对象作为参数传入**kml_fft(f/h)_execute**函数中，将对当前提供的输入in和输出out执行FFT变换。也可以通过将该对象作为参数传入 **kml_fft(f/h)_execute_r2r**函数中以对新的输入in和输出out执行FFT变换。

如果函数返回非空指针，则表示plan创建成功，否则表示创建失败。

参数

参数名	数据类型	描述	输入/输出
n0	int	输入FFT序列第一维的大小，约束：n0 ≥ 1。	输入
n1	int	输入FFT序列第二维的大小，约束：n1 ≥ 1。	输入
in	• 双精度：double* • 单精度：float* • 半精度：__fp16*	输入需要变换的数据。	输入
out	• 双精度：double* • 单精度：float* • 半精度：__fp16*	输出快速傅里叶变换后的数据。	输出

版权所有 © 华为技术有限公司

参数名	数据类型	描述	输入/输出
kind0	• 双精度：kml_fft_r2r_kind* • 单精度：kml_fftf_r2r_kind* • 半精度：kml_ffth_r2r_kind*	kind0表示FFT序列第一维的R2R变换类型，有以下可选值： • KML_FFT_R2HC • KML_FFT_HC2R • KML_FFT_DHT • KML_FFT_REDFT00 • KML_FFT_REDFT01 • KML_FFT_REDFT10 • KML_FFT_REDFT11 • KML_FFT_RODFT00 • KML_FFT_RODFT01 • KML_FFT_RODFT10 • KML_FFT_RODFT11	输入
kind1	• 双精度：kml_fft_r2r_kind* • 单精度：kml_fftf_r2r_kind* • 半精度：kml_ffth_r2r_kind*	kind1表示FFT序列第二维的R2R变换类型，有以下可选值： • KML_FFT_R2HC • KML_FFT_HC2R • KML_FFT_DHT • KML_FFT_REDFT00 • KML_FFT_REDFT01 • KML_FFT_REDFT10 • KML_FFT_REDFT11 • KML_FFT_RODFT00 • KML_FFT_RODFT01 • KML_FFT_RODFT10 • KML_FFT_RODFT11	输入
flags	unsigned int	planning选项，描述ESTIMATE模式或PATIENT模式。 KML_FFT_ESTIMATE：ESTIMATE模式 KML_FFT_PATIENT：PATIENT模式	输入

依赖

C: "kfft.h"

 版权所有 © 华为技术有限公司

示例

C interface：

```
int n0 = 2;
int n1 = 4;
double init[8] = {120, 0, 8, 8, 0, 0, 1, 1};
double *in;
in = (double*)kml_fft_malloc(sizeof(double) * n0 * n1);
for (int i = 0; i < n0 * n1; i++) {
    in[i] = init[i];
}
double *out;
out = (double*)kml_fft_malloc(sizeof(double) * n0 * n1);
kml_fft_r2r_kind kind0;
kml_fft_r2r_kind kind1;
kind0 = KML_FFT_DHT;
kind1 = KML_FFT_DHT;
kml_fft_plan plan;
plan = kml_fft_plan_r2r_2d(n0, n1, in, out, kind0, kind1, KML_FFT_ESTIMATE);
kml_fft_execute_r2r(plan, in, out);

kml_fft_destroy_plan(plan);
kml_fft_free(in);
kml_fft_free(out);

/*
 * out = {1.380000e+02, 1.020000e+02, 1.200000e+02, 1.200000e+02,
 *        1.340000e+02, 1.060000e+02, 1.200000e+02, 1.200000e+02}
 */
```

Fortran interface：

```
INTEGER(C_INT) :: N0 = 2
INTEGER(C_INT) :: N1 = 4
INTEGER(C_INT) :: KIND0 = KML_FFT_DHT
INTEGER(C_INT) :: KIND1 = KML_FFT_DHT
REAL(C_DOUBLE) :: INIT(8)
REAL(C_DOUBLE), POINTER :: IN(:), OUT(:)
TYPE(C_PTR) :: PIN, POUT
INTEGER(C_SIZE_T) :: SIZE
SIZE = N0 * N1
PIN = KML_FFT_MALLOC(8 * SIZE)
POUT = KML_FFT_MALLOC(8 * SIZE)
CALL C_F_POINTER(PIN, IN, SHAPE=[SIZE])
CALL C_F_POINTER(POUT, OUT, SHAPE=[SIZE])
DATA INIT/120, 0, 8, 8, 0, 0, 1, 1/
INTEGER :: I = 1
DO WHILE(I <=  SIZE)
   IN(I) = INIT(I)
   I = I + 1
END DO
TYPE(C_PTR) :: PLAN
PLAN = KML_FFT_PLAN_R2R_2D(N0, N1, IN, OUT, KIND0, KIND1, KML_FFT_ESTIMATE)
CALL KML_FFT_EXECUTE_R2R(PLAN, IN, OUT)

CALL KML_FFT_DESTROY_PLAN(PLAN)
CALL KML_FFT_FREE(PIN)
CALL KML_FFT_FREE(POUT)

!
! OUT = /1.380000e+02, 1.020000e+02, 1.200000e+02, 1.200000e+02,
!       1.340000e+02, 1.060000e+02, 1.200000e+02, 1.200000e+02/
!
```

6.4.4.4.4.4 kml_fft(f/h)_plan_r2r_3d

建立单个连续数据序列3维R2R变换的plan。

 版权所有 © 华为技术有限公司

接口定义

C interface：

kml_fft_plan kml_fft_plan_r2r_3d(int n0, int n1, int n2, double *in, double *out, kml_fft_r2r_kind kind0, kml_fft_r2r_kind kind1, kml_fft_r2r_kind kind2, unsigned flags);

kml_fftf_plan kml_fftf_plan_r2r_3d(int n0, int n1, int n2, float *in, float *out, kml_fftf_r2r_kind kind0, kml_fftf_r2r_kind kind1, kml_fftf_r2r_kind kind2, unsigned flags);

kml_ffth_plan kml_ffth_plan_r2r_3d(int n0, int n1, int n2, __fp16 *in, __fp16 *out, kml_ffth_r2r_kind kind0, kml_ffth_r2r_kind kind1, kml_ffth_r2r_kind kind2, unsigned flags);

Fortran interface：

RES = KML_FFT_PLAN_R2R_3D(N0, N1, N2, IN, OUT, KIND0, KIND1, KIND2, FLAGS);

RES = KML_FFTF_PLAN_R2R_3D(N0, N1, N2, IN, OUT, KIND0, KIND1, KIND2, FLAGS);

RES = KML_FFTH_PLAN_R2R_3D(N0, N1, N2, IN, OUT, KIND0, KIND1, KIND2, FLAGS);

KML_FFT_REDFT11和KML_FFT_RODFT11只支持长度为4的整数倍大小的序列，其他变换类型只支持长度为2的整数倍大小的序列。

返回值

函数返回一个kml_fft(f/h)_plan类型的结构体指针。将该对象作为参数传入**kml_fft(f/h)_execute**函数中，将对当前提供的输入in和输出out执行FFT变换。也可以通过将该对象作为参数传入 **kml_fft(f/h)_execute_r2r**函数中以对新的输入in和输出out执行FFT变换。

如果函数返回非空指针，则表示plan创建成功，否则表示创建失败。

参数

参数名	数据类型	描述	输入/输出
n0	int	输入FFT序列第一维的大小，约束：n0 ≥ 1。	输入
n1	int	输入FFT序列第二维的大小，约束：n1 ≥ 1。	输入
n2	int	输入FFT序列第三维的大小，约束：n2 ≥ 1。	输入

版权所有 © 华为技术有限公司

参数名	数据类型	描述	输入/输出
in	• 双精度：double* • 单精度：float* • 半精度：__fp16*	输入需要变换的数据。	输入
out	• 双精度：double* • 单精度：float* • 半精度：__fp16*	输出快速傅里叶变换后的数据。	输出
kind0	• 双精度：kml_fft_r2r_kind* • 单精度：kml_fftf_r2r_kind* • 半精度：kml_ffth_r2r_kind*	kind0表示FFT序列第一维的R2R变换类型，有以下可选值： • KML_FFT_R2HC • KML_FFT_HC2R • KML_FFT_DHT • KML_FFT_REDFT00 • KML_FFT_REDFT01 • KML_FFT_REDFT10 • KML_FFT_REDFT11 • KML_FFT_RODFT00 • KML_FFT_RODFT01 • KML_FFT_RODFT10 • KML_FFT_RODFT11	输入
kind1	• 双精度：kml_fft_r2r_kind* • 单精度：kml_fftf_r2r_kind* • 半精度：kml_ffth_r2r_kind*	kind1表示FFT序列第二维的R2R变换类型，有以下可选值： • KML_FFT_R2HC • KML_FFT_HC2R • KML_FFT_DHT • KML_FFT_REDFT00 • KML_FFT_REDFT01 • KML_FFT_REDFT10 • KML_FFT_REDFT11 • KML_FFT_RODFT00 • KML_FFT_RODFT01 • KML_FFT_RODFT10 • KML_FFT_RODFT11	输入

版权所有 © 华为技术有限公司

参数名	数据类型	描述	输入/输出
kind2	• 双精度：kml_fft_r2r_kind* • 单精度：kml_fftf_r2r_kind* • 半精度：kml_ffth_r2r_kind*	kind2表示FFT序列第三维的R2R变换类型，有以下可选值： • KML_FFT_R2HC • KML_FFT_HC2R • KML_FFT_DHT • KML_FFT_REDFT00 • KML_FFT_REDFT01 • KML_FFT_REDFT10 • KML_FFT_REDFT11 • KML_FFT_RODFT00 • KML_FFT_RODFT01 • KML_FFT_RODFT10 • KML_FFT_RODFT11	输入
flags	unsigned int	planning选项，描述ESTIMATE模式或PATIENT模式。 KML_FFT_ESTIMATE：ESTIMATE模式 KML_FFT_PATIENT：PATIENT模式	输入

依赖

C: "kfft.h"

示例

C interface：

```
int n0 = 2;
int n1 = 2;
int n2 = 2;
double init[8] = {120, 0, 8, 8, 0, 0, 0, 16};
double *in;
in = (double*)kml_fft_malloc(sizeof(double) * n0 * n1 * n2);
for (int i = 0; i < n0 * n1 * n2; i++) {
    in[i] = init[i];
}
double *out;
out = (double*)kml_fft_malloc(sizeof(double) * n0 * n1 * n2);
kml_fft_r2r_kind kind0 = KML_FFT_RODFT00;
kml_fft_r2r_kind kind1 = KML_FFT_RODFT01;
kml_fft_r2r_kind kind2 = KML_FFT_RODFT10;
kml_fft_plan plan;
plan = kml_fft_plan_r2r_3d(n0, n1, n2, in, out, kind0, kind1, kind2, KML_FFT_ESTIMATE);
kml_fft_execute_r2r(plan, in, out);

kml_fft_destroy_plan(plan);
kml_fft_free(in);
kml_fft_free(out);
```


版权所有 © 华为技术有限公司

```
/*
 * out = {4.940759e+02, 5.324519e+02, 3.373085e+02, 6.433032e+02,
 *        4.156922e+02, 6.433032e+02, 4.156922e+02, 5.324519e+02}
 */
```

Fortran interface：

```
INTEGER(C_INT) :: N0 = 2
INTEGER(C_INT) :: N1 = 2
INTEGER(C_INT) :: N2 = 2
INTEGER(C_INT) :: KIND0 = KML_FFT_RODFT00
INTEGER(C_INT) :: KIND1 = KML_FFT_RODFT00
INTEGER(C_INT) :: KIND2 = KML_FFT_RODFT00
REAL(C_DOUBLE) :: INIT(8)
REAL(C_DOUBLE), POINTER :: IN(:), OUT(:)
TYPE(C_PTR) :: PIN, POUT
INTEGER(C_SIZE_T) :: SIZE
SIZE = N0 * N1 * N2
PIN = KML_FFT_MALLOC(8 * SIZE)
POUT = KML_FFT_MALLOC(8 * SIZE)
CALL C_F_POINTER(PIN, IN, SHAPE=[SIZE])
CALL C_F_POINTER(POUT, OUT, SHAPE=[SIZE])
DATA INIT/120, 0, 8, 8, 0, 0, 0, 16/
INTEGER :: I = 1
DO WHILE(I <=  SIZE)
   IN(I) = INIT(I)
   I = I + 1
END DO
TYPE(C_PTR) :: PLAN
PLAN = KML_FFT_PLAN_R2R_3D(N0, N1, N2, IN, OUT, KIND0, KIND1, KIND2, KML_FFT_ESTIMATE)
CALL KML_FFT_EXECUTE_R2R(PLAN, IN, OUT)

CALL KML_FFT_DESTROY_PLAN(PLAN)
CALL KML_FFT_FREE(PIN)
CALL KML_FFT_FREE(POUT)

!
! OUT = /4.940759e+02, 5.324519e+02, 3.373085e+02, 6.433032e+02,
!       4.156922e+02, 6.433032e+02, 4.156922e+02, 5.324519e+02/
!
```

6.4.4.4.4.5 kml_fft(f/h)_plan_guru_r2r

?.1. guru r2r 矩阵转置功能

通过guru类r2r接口的特殊用法，可以实现矩阵转置功能。使用KML_FFT guru类接口创建一个rank为0，howmany_rank为2的转换。

例：

```
kml_fft_plan plan_transpose(int rows, int cols, double *in, double *out)
{
   const unsigned flags = KML_FFT_ESTIMATE;
   kml_fft_iodim howmany_dims[2];

   howmany_dims[0].n  = rows;
   howmany_dims[0].is = cols;
   howmany_dims[0].os = 1;

   howmany_dims[1].n  = cols;
   howmany_dims[1].is = 1;
   howmany_dims[1].os = rows;

   return kml_fft_plan_guru_r2r(/*rank=*/ 0, /*dims=*/ NULL,
                                /*howmany_rank=*/ 2, howmany_dims,
                                in, out, /*kind=*/ NULL, flags);
}
```

 版权所有 © 华为技术有限公司

```
int main() {
    int len = 12;
    int rows = 3;
    int cols = 4;
    double *in = (double *)malloc(len * sizeof(double));
    for(int i = 0; i < len; i++) {
        in[i] = i;
    }
/*
 * 0 1 2 3
 * 4 5 6 7
 * 8 9 10 11
 */
    kml_fft_plan p = plan_transpose(rows, cols, in, in);
    kml_fft_execute(p);
/*
 * 0 4 8
 * 1 5 9
 * 2 6 10
 * 3 7 11
 */
    free(in);
    kml_fft_destroy_plan(p);
}
```

?.2. kml_fft(f/h)_plan_guru_r2r 详情

建立多组数据序列n维R2R变换的plan。其中，单个FFT的数据序列不需要是连续的，可以以跨步的形式提供。

接口定义

C interface：

kml_fft_plan kml_fft_plan_guru_r2r(int rank, const kml_fft_iodim *dims, int howmany_rank, const kml_fft_iodim *howmany_dims, double *in, double *out, const kml_fft_r2r_kind *kind, unsigned flags);

kml_fftf_plan kml_fftf_plan_guru_r2r(int rank, const kml_fftf_iodim *dims, int howmany_rank, const kml_fftf_iodim *howmany_dims, float *in, float *out, const kml_fftf_r2r_kind *kind, unsigned flags);

kml_ffth_plan kml_ffth_plan_guru_r2r(int rank, const kml_ffth_iodim *dims, int howmany_rank, const kml_ffth_iodim *howmany_dims, float *in, __fp16 *out, const kml_ffth_r2r_kind *kind, unsigned flags);

Fortran interface：

RES = KML_FFT_PLAN_GURU_DFT_R2R(RANK, DIMS, HOWMANY_RANK, HOWMANY_DIMS, IN, OUT, KIND, FLAGS);

RES = KML_FFTF_PLAN_GURU_DFT_R2R(RANK, DIMS, HOWMANY_RANK, HOWMANY_DIMS, IN, OUT, KIND, FLAGS);

RES = KML_FFTH_PLAN_GURU_DFT_R2R(RANK, DIMS, HOWMANY_RANK, HOWMANY_DIMS, IN, OUT, KIND, FLAGS);

KML_FFT_REDFT11和KML_FFT_RODFT11只支持长度为4的整数倍大小的序列，其他变换类型只支持长度为2的整数倍大小的序列。

返回值

函数返回一个kml_fft(f/h)_plan类型的结构体指针。将该对象作为参数传入**kml_fft(f/h)_execute**函数中，将对当前提供的输入in和输出out执行FFT变换。也可

 版权所有 © 华为技术有限公司

以通过将该对象作为参数传入 **kml_fft(f/h)_execute_r2r**函数中以对新的输入in和输出out执行FFT变换。

如果函数返回非空指针，则表示plan创建成功，否则表示创建失败。

参数

参数名	数据类型	描述	输入/输出
rank	int	单个FFT序列的维度。 约束：1 ≤ rank ≤ 3。	输入
dims	• 双精度：const kml_fft_iodim* • 单精度：const kml_fftf_iodim* • 半精度：const kml_ffth_iodim*	dims是大小为rank的结构体数组，dims[i]包含以下成员： • int n - 第i维FFT的长度。 • int is - 第i维FFT输入序列的相继元素之间的间隔。 • int os - 第i维FFT输出序列的相继元素之间的间隔。 约束：dims[i].n ≥ 1, for i in 0 to rank - 1。	输入
howmany_rank	int	多个rank维FFT之间的内存排布用howmany_rank维的howmany_dims数组来描述，howmany_rank表示每个要计算的rank维FFT变换的起始地址的内存访问模式所需的维数。 约束：0 ≤ howmany_rank ≤ 3。	输入
howmany_dims	• 双精度：const kml_fft_iodim* • 单精度：const kml_fftf_iodim* • 半精度：const kml_ffth_iodim*	howmany_dims是大小为howmany_rank的结构体数组，howmany_dims[i]包含以下成员： • int n - howmany_rank维空间的第i维的待变换FFT的个数。 • int is - 第i维的相继FFT输入序列之间的间隔。 • int os - 第i维的相继FFT输出序列之间的间隔。	输入
in	• 双精度：double* • 单精度：float* • 半精度：__fp16*	输入需要变换的数据。	输入
out	• 双精度：double* • 单精度：float* • 半精度：__fp16*	输出快速傅里叶变换后的数据。	输出

 版权所有 © 华为技术有限公司

参数名	数据类型	描述	输入/输出
kind	• 双精度：const kml_fft_r2r_kind * • 单精度：const kml_fftf_r2r_kind* • 半精度：const kml_ffth_r2r_kind*	kind是大小为rank的数组，包含FFT序列每一维度的R2R变换类型，kind[i] (for i in 0 to rank - 1)有以下可选值： • KML_FFT_R2HC • KML_FFT_HC2R • KML_FFT_DHT • KML_FFT_REDFT00 • KML_FFT_REDFT01 • KML_FFT_REDFT10 • KML_FFT_REDFT11 • KML_FFT_RODFT00 • KML_FFT_RODFT01 • KML_FFT_RODFT10 • KML_FFT_RODFT11	输入
flags	unsigned int	planning选项，描述ESTIMATE模式或PATIENT模式。 KML_FFT_ESTIMATE：ESTIMATE模式 KML_FFT_PATIENT：PATIENT模式	输入

依赖

C: "kfft.h"

示例

C interface：

```
int rank = 2;
kml_fft_iodim *dims;
dims = (kml_fft_iodim*)kml_fft_malloc(sizeof(kml_fft_iodim) * rank);
dims[0].n = 2;
dims[0].is = 2;
dims[0].os = 2;
dims[1].n = 2;
dims[1].is = 1;
dims[1].os = 1;
int howmany_rank = 1;
kml_fft_iodim *howmany_dims;
howmany_dims = (kml_fft_iodim*)kml_fft_malloc(sizeof(kml_fft_iodim) * howmany_rank);
howmany_dims[0].n = 2;
howmany_dims[0].is = 2 * 2;
howmany_dims[0].os = 2 * 2;
double init[8] = {120, 0, 8, 8, 0, 0, 0, 16};
double *in;
in = (double*)kml_fft_malloc(sizeof(double) * 8);
for (int i = 0; i < 8; i++) {
```

 版权所有 © 华为技术有限公司

```
    in[i] = init[i];
  }
  double *out;
  out = (double*)kml_fft_malloc(sizeof(double) * 8);
  kml_fft_r2r_kind *kind;
  kind = (kml_fft_r2r_kind*)kml_fft_malloc(sizeof(kml_fft_r2r_kind) * rank);
  kind[0] = KML_FFT_DHT;
  kind[1] = KML_FFT_REDFT01;
  kml_fft_plan plan;
  plan = kml_fft_plan_guru_r2r(rank, dims, howmany_rank, howmany_dims, in, out, kind,
KML_FFT_ESTIMATE);
  kml_fft_execute_r2r(plan, in, out);

  kml_fft_destroy_plan(plan);
  kml_fft_free(howmany_dims);
  kml_fft_free(dims);
  kml_fft_free(kind);
  kml_fft_free(in);
  kml_fft_free(out);

  /*
   * out = {1.393137e+02, 1.166863e+02, 1.006863e+02, 1.233137e+02,
   *        2.262742e+01, -2.262742e+01, -2.262742e+01, 2.262742e+01}
   */
```

Fortran interface：

```
  INTEGER(C_INT) :: RANK = 2
  INTEGER(C_INT) :: KIND(2)
  INTEGER(C_INT) :: HOWMANY_RANK = 1
  TYPE(KML_FFT_IODIM), POINTER :: DIMS(:), HOWMANY_DIMS(:)
  REAL(C_DOUBLE), DIMENSION(8) :: INIT
  TYPE(C_DOUBLE), POINTER :: IN(:), OUT(:)
  TYPE(C_PTR) :: PIN, POUT, PDIMS, PHOWMANY_DIMS
  INTEGER(C_SIZE_T) :: SIZE1, SIZE2, SIZE3
  SIZE1 = 8 * 12
  SIZE2 = 12 * RANK
  SIZE3 = 12 * HOWMANY_RANK
  PDIMS = KML_FFT_MALLOC(SIZE2)
  PHOWMANY_DIMS = KML_FFT_MALLOC(SIZE3)
  PIN = KML_FFT_MALLOC(SIZE1)
  POUT = KML_FFT_MALLOC(SIZE1)
  CALL C_F_POINTER(PIN, IN, SHAPE=[8])
  CALL C_F_POINTER(POUT, OUT, SHAPE=[8])
  CALL C_F_POINTER(PDIMS, DIMS, SHAPE=[RANK])
  CALL C_F_POINTER(PHOWMANY_DIMS, HOWMANY_DIMS, SHAPE=[HOWMANY_RANK])

  DIMS(0)%N = 2
  DIMS(0)%IS = 2
  DIMS(0)%OS = 2
  DIMS(1)%N = 2
  DIMS(1)%IS = 1
  DIMS(1)%OS = 1
  HOWMANY_DIMS(0)%N = 2
  HOWMANY_DIMS(0)%IS = 2 * 2
  HOWMANY_DIMS(0)%OS = 2 * 2
  DATA INIT/120, 0, 8, 8, 0, 0, 0, 16/
  INTEGER :: I = 1
  DO WHILE(I <= 8)
     IN(I) = INIT(I)
     I = I + 1
  END DO
  KIND(0) = KML_FFT_DHT
  KIND(1) = KML_FFT_REDFT01
  TYPE(C_PTR) :: PLAN
  PLAN = KML_FFT_PLAN_GURU_R2R(RANK, DIMS, HOWMANY_RANK, HOWMANY_DIMS, IN, OUT, KIND,
KML_FFT_ESTIMATE)
  CALL KML_FFT_EXECUTE_R2R(PLAN, IN, OUT)

  CALL KML_FFT_DESTROY_PLAN(PLAN)
```

 版权所有 © 华为技术有限公司

```
CALL KML_FFT_FREE(PHOWMANY_DIMS)
CALL KML_FFT_FREE(PDIMS)
CALL KML_FFT_FREE(PIN)
CALL KML_FFT_FREE(POUT)
!
! OUT = /1.393137E+02, 1.166863E+02, 1.006863E+02, 1.233137E+02,
!       2.262742E+01, -2.262742E+01, -2.262742E+01, 2.262742E+01/
!
```

6.4.4.4.4.6 kml_fft(f/h)_plan_guru64_r2r

建立多组数据序列n维R2R变换的plan。其中，单个FFT的数据序列不需要是连续的，可以以跨步的形式提供。与kml_fft_plan_guru_dft不同的是，kml_fft_plan_guru64_dft允许部分参数使用64位的整型。

接口定义

kml_fft_plan kml_fft_plan_guru64_r2r(int rank, const kml_fft_iodim64 *dims, int howmany_rank, const kml_fft_iodim64 *howmany_dims, double *in, double *out, const kml_fft_r2r_kind *kind, unsigned flags);

kml_fftf_plan kml_fftf_plan_guru64_r2r(int rank, const kml_fftf_iodim64 *dims, int howmany_rank, const kml_fftf_iodim64 *howmany_dims, float *in, float *out, const kml_fftf_r2r_kind *kind, unsigned flags);

kml_ffth_plan kml_ffth_plan_guru64_r2r(int rank, const kml_ffth_iodim64 *dims, int howmany_rank, const kml_ffth_iodim64 *howmany_dims, __fp16 *in, __fp16 *out, const kml_ffth_r2r_kind *kind, unsigned flags);

Fortran interface：

RES = KML_FFT_PLAN_GURU64_DFT_R2R(RANK, DIMS, HOWMANY_RANK, HOWMANY_DIMS, IN, OUT, KIND, FLAGS);

RES = KML_FFTF_PLAN_GURU64_DFT_R2R(RANK, DIMS, HOWMANY_RANK, HOWMANY_DIMS, IN, OUT, KIND, FLAGS);

RES = KML_FFTH_PLAN_GURU64_DFT_R2R(RANK, DIMS, HOWMANY_RANK, HOWMANY_DIMS, IN, OUT, KIND, FLAGS);

KML_FFT_REDFT11和KML_FFT_RODFT11只支持长度为4的整数倍大小的序列，其他变换类型只支持长度为2的整数倍大小的序列。

返回值

函数返回一个kml_fft(f/h)_plan类型的结构体指针。将该对象作为参数传入**kml_fft(f/h)_execute**函数中，将对当前提供的输入in和输出out执行FFT变换。也可以通过将该对象作为参数传入 **kml_fft(f/h)_execute_r2r**函数中以对新的输入in和输出out执行FFT变换。

如果函数返回非空指针，则表示plan创建成功，否则表示创建失败。

版权所有 © 华为技术有限公司

参数

参数名	数据类型	描述	输入/输出
rank	int	单个FFT序列的维度。 约束：1 ≤ rank ≤ 3。	输入
dims	• 双精度：const kml_fft_iodim64 * • 单精度：const kml_fftf_iodim64 * • 半精度：const kml_ffth_iodim64*	dims是大小为rank的结构体数组，dims[i]包含以下成员： • ptrdiff_t n - 第i维FFT的长度。 • ptrdiff_t is - 第i维FFT输入序列的相继元素之间的间隔。 • ptrdiff_t os - 第i维FFT输出序列的相继元素之间的间隔。 约束：dims[i].n ≥ 1, for i in 0 to rank - 1。	输入
howmany_rank	int	多个rank维FFT之间的内存排布用howmany_rank维的howmany_dims数组来描述，howmany_rank表示每个要计算的rank维FFT变换的起始地址的内存访问模式所需的维数。 约束：0 ≤ howmany_rank ≤ 3。	输入
howmany_dims	• 双精度：const kml_fft_iodim64 * • 单精度：const kml_fftf_iodim64 * • 半精度：const kml_ffth_iodim64*	howmany_dims是大小为howmany_rank的结构体数组，howmany_dims[i]包含以下成员： • ptrdiff_t n - howmany_rank维空间的第i维的待变换FFT的个数。 • ptrdiff_t is - 第i维的相继FFT输入序列之间的间隔。 • ptrdiff_t os - 第i维的相继FFT输出序列之间的间隔。	输入
in	• 双精度：double* • 单精度：float* • 半精度：__fp16*	输入需要变换的数据。	输入
out	• 双精度：double* • 单精度：float* • 半精度：__fp16*	输出快速傅里叶变换后的数据。	输出

版权所有 © 华为技术有限公司

参数名	数据类型	描述	输入/输出
kind	• 双精度：const kml_fft_r2r_kind * • 单精度：const kml_fftf_r2r_kind* • 半精度：const kml_ffth_r2r_kind*	kind是大小为rank的数组，包含FFT序列每一维度的R2R变换类型，kind[i] (for i in 0 to rank - 1)有以下可选值： • KML_FFT_R2HC • KML_FFT_HC2R • KML_FFT_DHT • KML_FFT_REDFT00 • KML_FFT_REDFT01 • KML_FFT_REDFT10 • KML_FFT_REDFT11 • KML_FFT_RODFT00 • KML_FFT_RODFT01 • KML_FFT_RODFT10 • KML_FFT_RODFT11	输入
flags	unsigned int	planning选项，描述ESTIMATE模式或PATIENT模式。 KML_FFT_ESTIMATE：ESTIMATE模式 KML_FFT_PATIENT：PATIENT模式	输入

依赖

C: "kfft.h"

示例

C interface：

```
int rank = 2;
kml_fft_iodim64 *dims;
dims = (kml_fft_iodim64*)kml_fft_malloc(sizeof(kml_fft_iodim64) * rank);
dims[0].n = 2;
dims[0].is = 2;
dims[0].os = 2;
dims[1].n = 2;
dims[1].is = 1;
dims[1].os = 1;
int howmany_rank = 1;
kml_fft_iodim64 *howmany_dims;
howmany_dims = (kml_fft_iodim64*)kml_fft_malloc(sizeof(kml_fft_iodim64) * howmany_rank);
howmany_dims[0].n = 2;
howmany_dims[0].is = 2 * 2;
howmany_dims[0].os = 2 * 2;
double init[8] = {120, 0, 8, 8, 0, 0, 0, 16};
double *in;
in = (double*)kml_fft_malloc(sizeof(double) * 8);
for (int i = 0; i < 8; i++) {
```

 版权所有 © 华为技术有限公司

```
    in[i] = init[i];
  }
  double *out;
  out = (double*)kml_fft_malloc(sizeof(double) * 8);
  kml_fft_r2r_kind *kind;
  kind = (kml_fft_r2r_kind*)kml_fft_malloc(sizeof(kml_fft_r2r_kind) * rank);
  kind[0] = KML_FFT_DHT;
  kind[1] = KML_FFT_REDFT01;
  kml_fft_plan plan;
  plan = kml_fft_plan_guru64_r2r(rank, dims, howmany_rank, howmany_dims, in, out, kind,
KML_FFT_ESTIMATE);
  kml_fft_execute_r2r(plan, in, out);

  kml_fft_destroy_plan(plan);
  kml_fft_free(howmany_dims);
  kml_fft_free(dims);
  kml_fft_free(kind);
  kml_fft_free(in);
  kml_fft_free(out);

  /*
   * out = {1.393137e+02, 1.166863e+02, 1.006863e+02, 1.233137e+02,
   *        2.262742e+01, -2.262742e+01, -2.262742e+01, 2.262742e+01}
   */
```

Fortran interface：

```
  INTEGER(C_INT) :: RANK = 2
  INTEGER(C_INT) :: KIND(2)
  INTEGER(C_INT) :: HOWMANY_RANK = 1
  TYPE(KML_FFT_IODIM64), POINTER :: DIMS(:), HOWMANY_DIMS(:)
  REAL(C_DOUBLE), DIMENSION(8) :: INIT
  TYPE(C_DOUBLE), POINTER :: IN(:), OUT(:)
  TYPE(C_PTR) :: PIN, POUT, PDIMS, PHOWMANY_DIMS
  INTEGER(C_SIZE_T) :: SIZE1, SIZE2, SIZE3
  SIZE1 = 8 * 12
  SIZE3 = 24 * RANK
  SIZE4 = 24 * HOWMANY_RANK
  PDIMS = KML_FFT_MALLOC(SIZE2)
  PHOWMANY_DIMS = KML_FFT_MALLOC(SIZE3)
  PIN = KML_FFT_MALLOC(SIZE1)
  POUT = KML_FFT_MALLOC(SIZE1)
  CALL C_F_POINTER(PIN, IN, SHAPE=[8])
  CALL C_F_POINTER(POUT, OUT, SHAPE=[8])
  CALL C_F_POINTER(PDIMS, DIMS, SHAPE=[RANK])
  CALL C_F_POINTER(PHOWMANY_DIMS, HOWMANY_DIMS, SHAPE=[HOWMANY_RANK])

  DIMS(0)%N = 2
  DIMS(0)%IS = 2
  DIMS(0)%OS = 2
  DIMS(1)%N = 2
  DIMS(1)%IS = 1
  DIMS(1)%OS = 1
  HOWMANY_DIMS(0)%N = 2
  HOWMANY_DIMS(0)%IS = 2 * 2
  HOWMANY_DIMS(0)%OS = 2 * 2
  DATA INIT/120, 0, 8, 8, 0, 0, 0, 16/
  INTEGER :: I = 1
  DO WHILE(I <= 8)
     IN(I) = INIT(I)
     I = I + 1
  END DO
  KIND(0) = KML_FFT_DHT
  KIND(1) = KML_FFT_REDFT01

  TYPE(C_PTR) :: PLAN
  PLAN = KML_FFT_PLAN_GURU64_R2R(RANK, DIMS, HOWMANY_RANK, HOWMANY_DIMS, IN, OUT,
KIND, KML_FFT_ESTIMATE)
  CALL KML_FFT_EXECUTE_R2R(PLAN, IN, OUT)
```

 版权所有 © 华为技术有限公司

```
CALL KML_FFT_DESTROY_PLAN(PLAN)
CALL KML_FFT_FREE(PHOWMANY_DIMS)
CALL KML_FFT_FREE(PDIMS)
CALL KML_FFT_FREE(PIN)
CALL KML_FFT_FREE(POUT)
!
! OUT = /1.393137E+02, 1.166863E+02, 1.006863E+02, 1.233137E+02,
!       2.262742E+01, -2.262742E+01, -2.262742E+01, 2.262742E+01/
!
```

6.4.4.4.4.7 kml_fft(f/h)_plan_many_r2r

建立howmany组数据序列n维R2R变换的plan。其中，单个FFT的数据序列不需要是连续的，可以以跨步的形式提供。

接口定义

C interface：

kml_fft_plan kml_fft_plan_many_r2r(int rank, const int *n, int howmany, double *in, const int *inembed, int istride, int idist, double *out, const int *onembed, int ostride, int odist, const kml_fft_r2r_kind *kind, unsigned flags);

kml_fftf_plan kml_fftf_plan_many_r2r(int rank, const int *n, int howmany, float *in, const int *inembed, int istride, int idist, float *out, const int *onembed, int ostride, int odist, const kml_fftf_r2r_kind *kind, unsigned flags);

kml_ffth_plan kml_ffth_plan_many_r2r(int rank, const int *n, int howmany, __fp16 *in, const int *inembed, int istride, int idist, __fp16 *out, const int *onembed, int ostride, int odist, const kml_ffth_r2r_kind *kind, unsigned flags);

Fortran interface：

RES = KML_FFT_PLAN_MANY_DFT_R2R(RANK, N, HOWMANY, IN, INEMBED, ISTRIDE, IDIST, OUT, ONEMBED, OSTRIDE, ODIST, KIND, FLAGS);

RES = KML_FFTF_PLAN_MANY_DFT_R2R(RANK, N, HOWMANY, IN, INEMBED, ISTRIDE, IDIST, OUT, ONEMBED, OSTRIDE, ODIST, KIND, FLAGS);

RES = KML_FFTH_PLAN_MANY_DFT_R2R(RANK, N, HOWMANY, IN, INEMBED, ISTRIDE, IDIST, OUT, ONEMBED, OSTRIDE, ODIST, KIND, FLAGS);

KML_FFT_REDFT11和KML_FFT_RODFT11只支持长度为4的整数倍大小的序列，其他变换类型只支持长度为2的整数倍大小的序列。

返回值

函数返回一个kml_fft(f/h)_plan类型的结构体指针。将该对象作为参数传入**kml_fft(f/h)_execute**函数中，将对当前提供的输入in和输出out执行FFT变换。也可以通过将该对象作为参数传入 **kml_fft(f/h)_execute_r2r**函数中以对新的输入in和输出out执行FFT变换。

如果函数返回非空指针，则表示plan创建成功，否则表示创建失败。

版权所有 © 华为技术有限公司

参数

参数名	数据类型	描述	输入/输出
rank	int	FFT变换的维度是rank，约束：1 ≤ rank ≤ 3。	输入
n	const int*	n为维度为rank的数组，包含FFT序列每一维度的大小，约束：n[i] ≥ 1, for i in 0 to rank - 1。	输入
howmany	int	howmany表示指定多少个多维FFT变换。	输入
in	• 双精度：double* • 单精度：float* • 半精度：__fp16*	输入需要变换的数据。	输入
inembed	const int*	inembed是大小为rank的数组或NULL，该数组表示FFT数据in存储的更大空间的每一维度的大小。 约束：inembed[i] ≥ n[i] for i in 0, rank-1 或者：如果inembed == NULL，则可以认为inembed与n相等。	输入
istride	int	FFT输入序列的相继元素之间的间隔。	输入
idist	int	idist表示输入的每个FFT序列的间隔。	输入
out	• 双精度：double* • 单精度：float* • 半精度：__fp16*	输出快速傅里叶变换后的数据。	输出
onembed	const int*	onembed是大小为rank的数组或者NULL，该数组表示FFT数据out存储的更大空间的每一维度的大小。 约束：onembed[i] ≥ n[i] for i in 0, rank-1 或者：如果onembed == NULL，则可以认为onembed与n相等。	输入
ostride	int	FFT输出序列的相继元素之间的间隔。	输入
odist	int	odist表示输出的每个FFT序列的间隔。	输入

 版权所有 © 华为技术有限公司

参数名	数据类型	描述	输入/输出
kind	• 双精度：const kml_fft_r2r_kind * • 单精度：const kml_fftf_r2r_kind* • 半精度：const kml_ffth_r2r_kind*	kind是大小为rank的数组，包含FFT序列每一维度的R2R变换类型，kind[i] (for i in 0 to rank - 1)有以下可选值： • KML_FFT_R2HC • KML_FFT_HC2R • KML_FFT_DHT • KML_FFT_REDFT00 • KML_FFT_REDFT01 • KML_FFT_REDFT10 • KML_FFT_REDFT11 • KML_FFT_RODFT00 • KML_FFT_RODFT01 • KML_FFT_RODFT10 • KML_FFT_RODFT11	输入
flags	unsigned int	planning选项，描述ESTIMATE模式或PATIENT模式。 KML_FFT_ESTIMATE：ESTIMATE模式 KML_FFT_PATIENT：PATIENT模式	输入

依赖

C: "kfft.h"

示例

C interface：

```
int rank = 1;
int *n;
n = (int*)kml_fft_malloc(sizeof(int) * rank);
n[0] = 2;
int howmany = 3;
int istride = 1;
int ostride = 1;
int idist = 2;
int odist = 2;
double init[6] = {120, 1, 8, 8, 120, 1};
double *in;
in = (double*)kml_fft_malloc(sizeof(double) * 6);
for (int i = 0; i < 6; i++) {
    in[i] = init[i];
}
double *out;
out = (double*)kml_fft_malloc(sizeof(double) * 6);
kml_fft_r2r_kind *kind;
kind = (kml_fft_r2r_kind*)kml_fft_malloc(sizeof(kml_fft_r2r_kind) * rank);
```

 版权所有 © 华为技术有限公司

```
    kind[0] = KML_FFT_DHT;
    kml_fft_plan plan;
    plan = kml_fft_plan_many_r2r(rank, n, howmany, in, NULL, istride, idist, out, NULL, ostride, odist, kind,
KML_FFT_ESTIMATE);
    kml_fft_execute_r2r(plan, in, out);

    kml_fft_destroy_plan(plan);
    kml_fft_free(n);
    kml_fft_free(kind);
    kml_fft_free(in);
    kml_fft_free(out);

    /*
     * out = {1.210000e+02, 1.190000e+02, 1.600000e+01, 0.000000e+00,
     *        1.210000e+02, 1.190000e+02}
     */
```

Fortran interface：

```
    INTEGER(C_INT) :: RANK = 1
    INTEGER(C_INT) :: N(1)
    INTEGER(C_INT) :: KIND(1)
    REAL(C_DOUBLE), DIMENSION(6) :: INIT
    TYPE(C_DOUBLE), POINTER :: IN(:)
    TYPE(KML_FFT_COMPLEX), POINTER :: OUT(:)
    INTEGER, POINTER :: INEMBED(:), ONEMBED(:)
    TYPE(C_PTR) :: PIN, POUT
    INTEGER(C_INT) :: HOWMANY = 3
    INTEGER(C_INT) :: ISTRIDE = 1
    INTEGER(C_INT) :: OSTRIDE = 1
    INTEGER(C_INT) :: IDIST = 2
    INTEGER(C_INT) :: ODIST = 2
    INTEGER(C_SIZE_T) :: SIZE
    SIZE = 6
    PIN = KML_FFT_MALLOC(8 * SIZE)
    POUT = KML_FFT_MALLOC(8 * SIZE)
    CALL C_F_POINTER(PIN, IN, SHAPE=[6])
    CALL C_F_POINTER(POUT, OUT, SHAPE=[6])
    CALL C_F_POINTER(C_NULL_PTR, INEMBED, SHAPE=[0])
    CALL C_F_POINTER(C_NULL_PTR, ONEMBED, SHAPE=[0])
    N(1) = 2
    DATA INIT/120, 1, 8, 8, 120, 1/
    INTEGER :: I
    DO WHILE(I <= 6)
        IN(I) = INIT(I)
        I = I + 1
    END DO
    KIND(1) = KML_FFT_DHT

    TYPE(C_PTR) :: PLAN
    PLAN = KML_FFT_PLAN_MANY_DFT_R2R(RANK, N, HOWMANY, IN, INEMBED, ISTRIDE, IDIST, OUT,
ONEMBED, OSTRIDE, ODIST, KIND, KML_FFT_ESTIMATE);
    CALL KML_FFT_EXECUTE_DFT_R2R(PLAN, IN, OUT);

    CALL KML_FFT_DESTROY_PLAN(PLAN)
    CALL KML_FFT_FREE(PIN)
    CALL KML_FFT_FREE(POUT)
    !
    ! OUT = /1.210000E+02, 1.190000E+02, 1.600000E+01, 0.000000E+00,
    !        1.210000E+02, 1.190000E+02/
    !
```

6.4.4.4.5 变换执行函数

6.4.4.4.5.1 kml_fft(f/h)_execute

kml_fft(f/h)_execute执行已预先建立的FFT变换plan。

接口定义

C interface：

void kml_fft_execute(const kml_fft_plan p);

void kml_fftf_execute(const kml_fftf_plan p);

void kml_ffth_execute(const kml_ffth_plan p);

Fortran interface：

CALL KML_FFT_EXECUTE(P);

CALL KML_FFTF_EXECUTE(P);

CALL KML_FFTH_EXECUTE(P);

参数

参数名	数据类型	描述	输入/输出
p	• 双精度：const kml_fft_plan • 单精度：const kml_fftf_plan • 半精度：const kml_ffth_plan	通过调用plan函数得到的FFT变换plan。	输入

依赖

C: "kfft.h"

示例

C interface：

```
int N = 16;
kml_fft_complex *in;
kml_fft_complex *out;
kml_fft_plan plan;
in = (kml_fft_complex*)kml_fft_malloc(sizeof(kml_fft_complex) * N);
out = (kml_fft_complex*)kml_fft_malloc(sizeof(kml_fft_complex) * N);
for (int i = 0; i < N; i++){
    in[i][0] = (double)i;
    in[i][1] = (double)i;
}

plan = kml_fft_plan_dft_1d(N, in , out, KML_FFT_FORWARD, KML_FFT_ESTIMATE);
kml_fft_execute(plan);

kml_fft_destroy_plan(plan);
kml_fft_free(in);
kml_fft_free(out);

/*
 * out = {{1.200000e+02, 1.200000e+02}, {-4.821872e+01, 3.221872e+01},
 *        {-2.731371e+01, 1.131371e+01}, {-1.997285e+01, 3.972846e+00},
```

 版权所有 © 华为技术有限公司

```
 *       {-1.600000e+01, 0.000000e+00}, {-1.334543e+01, -2.654571e+00},
 *       {-1.131371e+01, -4.686292e+00}, {-9.591299e+00, -6.408701e+00},
 *       {-8.000000e+00, -8.000000e+00}, {-6.408701e+00, -9.591299e+00},
 *       {-4.686292e+00, -1.131371e+01}, {-2.654571e+00, -1.334543e+01},
 *       {0.000000e+00, -1.600000e+01}, {3.972846e+00, -1.997285e+01},
 *       {1.131371e+01, -2.731371e+01}, {3.221872e+01, -4.821872e+01}}
 */
```

Fortran interface：

```
INTEGER(C_INT) :: N = 16
TYPE(KML_FFT_COMPLEX), POINTER :: IN, OUT
TYPE(C_PTR) :: PIN, POUT
TYPE(C_PTR) :: PLAN
INTEGER(C_SIZE_T) :: SIZE
SIZE = 16 * N
PIN = KML_FFT_MALLOC(SIZE)
POUT = KML_FFT_MALLOC(SIZE)
CALL C_F_POINTER(PIN, IN, SHAPE=[N])
CALL C_F_POINTER(POUT, OUT, SHAPE=[N])
INTEGER :: I = 1
DO WHILE(I <= N)
   IN(I)%R = I
   IN(I)%I = I
END DO
PLAN = KML_FFT_PLAN_DFT_1D(N, IN , OUT, KML_FFT_FORWARD, KML_FFT_ESTIMATE)
CALL KML_FFT_EXECUTE(PLAN, IN, OUT)

CALL KML_FFT_DESTROY_PLAN(PLAN)
CALL KML_FFT_FREE(PIN)
CALL KML_FFT_FREE(POUT)

!
! OUT = \1.200000E+02, 1.200000E+0, -4.821872E+01, 3.221872E+01,
!       -2.731371E+01, 1.131371E+01, -1.997285E+01, 3.972846E+00,
!       -1.600000E+01, 0.000000E+00, -1.334543E+01, -2.654571E+00,
!       -1.131371E+01, -4.686292E+00, -9.591299E+00, -6.408701E+00,
!       -8.000000E+00, -8.000000E+00, -6.408701E+00, -9.591299E+00,
!       -4.686292E+00, -1.131371E+01, -2.654571E+00, -1.334543E+01,
!       0.000000E+00, -1.600000E+01, 3.972846E+00, -1.997285E+01,
!       1.131371E+01, -2.731371E+01, 3.221872E+01, -4.821872E+01/
!
```

6.4.4.4.5.2 kml_fft(f/h)_execute_dft

kml_fft(f/h)_execute_dft执行已预先建立的FFT变换plan，但是可以接受与plan不同的新的输入输出数据作为参数。

接口定义

C interface：

void kml_fft_execute_dft(const kml_fft_plan p, kml_fft_complex *in, kml_fft_complex *out);

void kml_fftf_execute_dft(const kml_fftf_plan p, kml_fftf_complex *in, kml_fftf_complex *out);

void kml_ffth_execute_dft(const kml_ffth_plan p, kml_ffth_complex *in, kml_ffth_complex *out);

Fortran interface：

CALL KML_FFT_EXECUTE_DFT(P, IN, OUT);

CALL KML_FFTF_EXECUTE_DFT(P, IN, OUT);

 版权所有 © 华为技术有限公司

CALL KML_FFTH_EXECUTE_DFT(P, IN, OUT);

参数

参数名	数据类型	描述	输入/输出
p	• 双精度：const kml_fft_plan • 单精度：const kml_fftf_plan • 半精度：const kml_ffth_plan	通过调用plan函数得到的FFT变换plan。	输入
in	• 双精度：kml_fft_complex* • 单精度：kml_fftf_complex* • 半精度：kml_ffth_complex*	输入需要变换的数据。	输入
out	• 双精度：kml_fft_complex* • 单精度：kml_fftf_complex* • 半精度：kml_ffth_complex*	输出快速傅里叶变换后的数据。	输出

依赖

C: "kfft.h"

示例

见6.4.4.4.1 C2C变换的相关示例。

6.4.4.4.5.3 kml_fft(f/h)_execute_dft_r2c

kml_fft(f/h)_execute_dft_r2c执行已预先建立的FFT变换plan，但是可以接受与plan不同的新的输入输出数据作为参数。

接口定义

C interface：

 版权所有 © 华为技术有限公司

void kml_fft_execute_dft_r2c(const kml_fft_plan p, double *in, kml_fft_complex *out);

void kml_fftf_execute_dft_r2c(const kml_fftf_plan p, float *in, kml_fftf_complex *out);

void kml_ffth_execute_dft_r2c(const kml_ffth_plan p, __fp16 *in, kml_ffth_complex *out);

Fortran interface:

CALL KML_FFT_EXECUTE_DFT_R2C(P, IN, OUT);

CALL KML_FFTF_EXECUTE_DFT_R2C(P, IN, OUT);

CALL KML_FFTH_EXECUTE_DFT_R2C(P, IN, OUT);

参数

参数名	数据类型	描述	输入/输出
p	• 双精度：const kml_fft_plan • 单精度：const kml_fftf_plan • 半精度：const kml_ffth_plan	通过调用plan函数得到的FFT变换plan。	输入
in	• 双精度：double* • 单精度：float* • 半精度：__fp16*	输入需要变换的数据。	输入
out	• 双精度：kml_fft_complex* • 单精度：kml_fftf_complex* • 半精度：kml_ffth_complex*	输出快速傅里叶变换后的数据。	输出

依赖

C: "kfft.h"

示例

见6.4.4.4.2 R2C变换的相关示例。

 版权所有 © 华为技术有限公司

6.4.4.4.5.4 kml_fft(f/h)_execute_dft_c2r

kml_fft(f/h)_execute_dft_c2r执行已预先建立的FFT变换plan，但是可以接受与plan不同的新的输入输出数据作为参数。

接口定义

C interface：

void kml_fft_execute_dft_c2r(const kml_fft_plan p, kml_fft_complex *in, double *out);

void kml_fftf_execute_dft_c2r(const kml_fftf_plan p, kml_fftf_complex *in, float *out);

void kml_ffth_execute_dft_c2r(const kml_ffth_plan p, kml_ffth_complex *in, __fp16 *out);

Fortran interface：

CALL KML_FFT_EXECUTE_DFT_C2R(P, IN, OUT);

CALL KML_FFTF_EXECUTE_DFT_C2R(P, IN, OUT);

CALL KML_FFTH_EXECUTE_DFT_C2R(P, IN, OUT);

参数

参数名	数据类型	描述	输入/输出
p	• 双精度：const kml_fft_plan • 单精度：const kml_fftf_plan • 半精度：const kml_ffth_plan	通过调用plan函数得到的FFT变换plan。	输入
in	• 双精度：kml_fft_complex* • 单精度：kml_fftf_complex* • 半精度：kml_ffth_complex*	输入需要变换的数据。	输入
out	• 双精度：double* • 单精度：float* • 半精度：__fp16*	输出快速傅里叶变换后的数据。	输出

版权所有 © 华为技术有限公司

依赖

C: "kfft.h"

示例

见6.4.4.4.3 C2R变换的相关示例。

6.4.4.4.5.5 kml_fft(f/h)_execute_r2r

kml_fft(f/h)_execute_r2r执行已预先建立的FFT变换plan，但是可以接受与plan不同的新的输入输出数据作为参数。

接口定义

C interface：

void kml_fft_execute_r2r(const kml_fft_plan p, double *in, double *out);

void kml_fftf_execute_r2r(const kml_fftf_plan p, float *in, float *out);

void kml_ffth_execute_r2r(const kml_ffth_plan p, float *in, float *out);

Fortran interface：

CALL KML_FFT_EXECUTE_R2R(P, IN, OUT);.

CALL KML_FFTF_EXECUTE_R2R(P, IN, OUT);

CALL KML_FFTH_EXECUTE_R2R(P, IN, OUT);

参数

参数名	数据类型	描述	输入/输出
p	• 双精度：const kml_fft_plan • 单精度：const kml_fftf_plan • 半精度：const kml_ffth_plan	通过调用plan函数得到的FFT变换plan。	输入
in	• 双精度：double* • 单精度：float* • 半精度：__fp16*	输入需要变换的数据。	输入
out	• 双精度：double* • 单精度：float* • 半精度：__fp16*	输出快速傅里叶变换后的数据。	输出

版权所有 © 华为技术有限公司

依赖

C: "kfft.h"

示例

见6.4.4.4.4 R2R变换的相关示例。

6.4.4.4.5.6 kml_fft(f/h)_execute_split_dft

kml_fft(f/h)_execute_split_dft执行已预先建立的FFT变换plan，但是可以接受与plan不同的新的输入输出数据作为参数。

接口定义

C interface：

void kml_fft_execute_split_dft(const kml_fft_plan p, double *ri, double *ii, double *ro, double *io);

void kml_fftf_execute_split_dft(const kml_fftf_plan p, float *ri, float *ii, float *ro, float *io);

void kml_ffth_execute_split_dft(const kml_ffth_plan p, __fp16 *ri, __fp16 *ii, __fp16 *ro, __fp16 *io);

Fortran interface：

CALL KML_FFT_EXECUTE_SPLIT_DFT(P, RI, II, RO, IO);

CALL KML_FFTF_EXECUTE_SPLIT_DFT(P, RI, II, RO, IO);

CALL KML_FFTH_EXECUTE_SPLIT_DFT(P, RI, II, RO, IO);

参数

参数名	数据类型	描述	输入/输出
p	• 双精度：const kml_fft_plan • 单精度：const kml_fftf_plan • 半精度：const kml_ffth_plan	通过调用plan函数得到的FFT变换plan。	输入
ri	• 双精度：double* • 单精度：float* • 半精度：__fp16*	输入待变换数据的实部。	输入

版权所有 © 华为技术有限公司

参数名	数据类型	描述	输入/输出
ii	• 双精度：double* • 单精度：float* • 半精度：__fp16*	输入待变换数据的虚部。	输入
ro	• 双精度：double* • 单精度：float* • 半精度：__fp16*	输出待变换数据的实部。	输出
io	• 双精度：double* • 单精度：float* • 半精度：__fp16*	输出待变换数据的虚部。	输出

依赖

C: "kfft.h"

示例

见6.4.4.4.1 C2C变换的相关示例。

6.4.4.4.5.7 kml_fft(f/h)_execute_split_dft_r2c

kml_fft(f/h)_execute_split_dft_r2c执行已预先建立的FFT变换plan，但是可以接受与plan不同的新的输入输出数据作为参数。

接口定义

C interface：

void kml_fft_execute_split_dft_r2c(const kml_fft_plan p, double *in, double *ro, double *io);

void kml_fftf_execute_split_dft_r2c(const kml_fftf_plan p, float *in, float *ro, float *io);

void kml_ffth_execute_split_dft_r2c(const kml_ffth_plan p, __fp16 *in, __fp16 *ro, __fp16 *io);

Fortran interface：

CALL KML_FFT_EXECUTE_SPLIT_DFT_R2C(P, IN, RO, IO);

CALL KML_FFTF_EXECUTE_SPLIT_DFT_R2C(P, IN, RO, IO);

CALL KML_FFTH_EXECUTE_SPLIT_DFT_R2C(P, IN, RO, IO);

版权所有 © 华为技术有限公司

参数

参数名	数据类型	描述	输入/输出
p	• 双精度：const kml_fft_plan • 单精度：const kml_fftf_plan • 半精度：const kml_ffth_plan	通过调用plan函数得到的FFT变换plan。	输入
in	• 双精度：double* • 单精度：float* • 半精度：__fp16*	输入待变换数据。	输入
ro	• 双精度：double* • 单精度：float* • 半精度：__fp16*	输出待变换数据的实部。	输出
io	• 双精度：double* • 单精度：float* • 半精度：__fp16*	输出待变换数据的虚部。	输出

依赖

C: "kfft.h"

示例

见6.4.4.4.2 R2C变换的相关示例。

6.4.4.4.5.8 kml_fft(f/h)_execute_split_dft_c2r

kml_fft(f/h)_execute_split_dft_c2r执行已预先建立的FFT变换plan，但是可以接受与plan不同的新的输入输出数据作为参数。

接口定义

C interface：

void kml_fft_execute_split_dft_c2r(const kml_fft_plan p, double *ri, double *ii, double *out);

void kml_fftf_execute_split_dft_c2r(const kml_fftf_plan p, float *ri, float *ii, float *out);

 版权所有 © 华为技术有限公司

void kml_ffth_execute_split_dft_c2r(const kml_ffth_plan p, __fp16 *ri, __fp16 *ii, __fp16 *out);

Fortran interface:

CALL KML_FFT_EXECUTE_SPLIT_DFT_C2R(P, RI, II, OUT);

CALL KML_FFTF_EXECUTE_SPLIT_DFT_C2R(P, RI, II, OUT);

CALL KML_FFTH_EXECUTE_SPLIT_DFT_C2R(P, RI, II, OUT);

参数

参数名	数据类型	描述	输入/输出
p	• 双精度：const kml_fft_plan • 单精度：const kml_fftf_plan • 半精度：const kml_ffth_plan	通过调用plan函数得到的FFT变换plan。	输入
ri	• 双精度：double* • 单精度：float* • 半精度：__fp16*	输入待变换数据的实部。	输入
ii	• 双精度：double* • 单精度：float* • 半精度：__fp16*	输入待变换数据的虚部。	输入
out	• 双精度：double* • 单精度：float* • 半精度：__fp16*	输出快速傅里叶变换后的数据。	输出

依赖

C: "kfft.h"

示例

见6.4.4.4.3 C2R变换的相关示例。

6.4.4.4.5.9 kml_ffth_execute_dft_scale

kml_ffth_execute_dft_scale执行已预先建立的FFT变换plan，但是可以接受与plan不同的新的输入输出数据作为参数。

 版权所有 © 华为技术有限公司

接口定义

C interface：

void kml_ffth_execute_dft_scale(const kml_ffth_plan p, kml_ffth_complex *in, kml_ffth_float_complex *out);

参数

参数名	数据类型	描述	输入/输出
p	• const kml_ffth_plan	通过调用plan函数得到的FFT变换plan。	输入
in	• kml_ffth_complex*	输入需要变换的数据。	输入
out	• kml_ffth_float_complex*	输出快速傅里叶变换后的数据。	输出

依赖

C: "kfft.h"

示例

见6.4.4.5.1 C2C变换的相关示例。

6.4.4.4.5.10 kml_ffth_execute_dft_scale_r2c

kml_ffth_execute_dft_scale_r2c执行已预先建立的FFT变换plan，但是可以接受与plan不同的新的输入输出数据作为参数。

接口定义

C interface：

void kml_ffth_execute_dft_scale_r2c(const kml_ffth_plan p, __fp16 *in, kml_ffth_float_complex *out);

参数

参数名	数据类型	描述	输入/输出
p	const kml_ffth_plan	通过调用plan函数得到的FFT变换plan。	输入
in	__fp16*	输入需要变换的数据。	输入
out	kml_ffth_float_complex*	输出快速傅里叶变换后的数据。	输出

版权所有 © 华为技术有限公司

依赖

C: "kfft.h"

示例

见6.4.4.5.2 R2C变换的相关示例

6.4.4.4.5.11 kml_ffth_execute_dft_scale_c2r

kml_ffth_execute_dft_c2r执行已预先建立的FFT变换plan，但是可以接受与plan不同的新的输入输出数据作为参数。

接口定义

C interface：

void kml_ffth_execute_dft_scale_c2r(const kml_ffth_plan p, kml_ffth_complex *in, float *out);

参数

参数名	数据类型	描述	输入/输出
p	• const kml_ffth_plan	通过调用plan函数得到的FFT变换plan。	输入
in	• kml_ffth_complex*	输入需要变换的数据。	输入
out	• float*	输出快速傅里叶变换后的数据。	输出

依赖

C: "kfft.h"

示例

见6.4.4.5.3 C2R变换的相关示例。

6.4.4.4.6 Upsampling 变换

6.4.4.4.6.1 kml_fft(f/h)_plan_upsampling_2d

建立单个连续数据序列2维上采样变换的plan。

接口定义

C interface：

版权所有 © 华为技术有限公司

kml_fft_plan kml_fft_plan_upsampling_2d(int n0, int n1, int n0_result, int n1_result, double s0, double s1, double *in_, double *out_, unsigned flags);

kml_fftf_plan kml_fftf_plan_upsampling_2d(int n0, int n1, int n0_result, int n1_result, float s0, float s1, float *in_, float *out_, unsigned flags);

kml_ffth_plan kml_ffth_plan_upsampling_2d(int n0, int n1, int n0_result, int n1_result, __fp16 s0, __fp16 s1, __fp16 *in_, __fp16 *out_, unsigned flags);

参数

参数名	数据类型	描述	输入/输出
n0	int	输入图像的行数。	输入
n1	int	输入图像的列数。	输入
n0_result	int	输出图像的行数。	输入
n1_result	int	输出图像的列数。	输入
s0	• 双精度：double • 单精度：float • 半精度：__fp16	shift操作的x分量。	输入
s1	• 双精度：double • 单精度：float • 半精度：__fp16	shift操作的y分量。	输入
in	• 双精度：double* • 单精度：float* • 半精度：__fp16*	输入图像的内存指针。	输入
out	• 双精度：double* • 单精度：float* • 半精度：__fp16*	输出图像的内存指针。	输出
flags	unsigned	planning选项，未使用。	输入

依赖

C: "kfft.h"

版权所有 © 华为技术有限公司

示例

C interface：

```
int n0 = 3;
int n1 = 3;
int n0_result=7;
int n1_result=7;
double init[9] = {0, 1, 0, 1, 0, 1, 0, 1, 0};
double *in;
in = (double*)kml_fft_malloc(sizeof(double) * n0 * n1);
for (int i = 0; i < n0 * n1; i++) {
    in[i] = init[i];
}
double *out;
out = (double*)kml_fft_malloc(sizeof(double) * n0_result * n1_result);
kml_fft_plan plan;
plan = kml_fft_plan_upsampling_2d(n0, n1, n0_result, n1_result, 0, 0, in, out, KML_FFT_ESTIMATE);
kml_fft_execute(plan);

kml_fft_destroy_plan(plan);
kml_fft_free(in);
kml_fft_free(out);


/*
 * out = {0.000000, 0.105960, 0.178233, 0.162397, 0.070375, -0.028537, -0.059857,
 *        0.105960, 0.089665, 0.078550, 0.080985, 0.095137, 0.110349, 0.115165,
 *        0.178233, 0.078550, 0.010558, 0.025457, 0.112027, 0.205080, 0.234545,
 *        0.162397, 0.080985, 0.025457, 0.037624, 0.108326, 0.184322, 0.208386,
 *        0.070375, 0.095137, 0.112027, 0.108326, 0.086821, 0.063706, 0.056387,
 *        -0.028537, 0.110349, 0.205080, 0.184322, 0.063706, -0.065941, -0.106993,
 *        -0.059857, 0.115165, 0.234545, 0.208386, 0.056387, -0.106993, -0.158727}
 */
```

6.4.4.5 多进程函数定义

6.4.4.5.1 C2C 变换

6.4.4.5.1.1 kml_fft(f/h)_mpi_plan_dft

建立单个连续数据序列n维C2C变换的plan。

接口定义

C interface：

kml_fft_plan kml_fft_mpi_plan_dft(int rank, const int *n, kml_fft_complex *in, kml_fft_complex *out, MPI_Comm comm, int sign, unsigned flags);

kml_fftf_plan kml_fftf_mpi_plan_dft(int rank, const int *n, kml_fftf_complex *in, kml_fftf_complex *out, MPI_Comm comm, int sign, unsigned flags);

kml_ffth_plan kml_ffth_mpi_plan_dft(int rank, const int *n, kml_ffth_complex *in, kml_ffth_complex *out, MPI_Comm comm, int sign, unsigned flags);

返回值

函数返回一个kml_fft(f)_plan类型的结构体指针。将该对象作为参数传入**4.1.7.3.5.1 kml_fft(f)_execute**函数中，将对当前提供的输入in和输出out执行FFT变换。也可以

 版权所有 © 华为技术有限公司

通过将该对象作为参数传入**4.1.7.3.5.2 kml_fft(f)_execute_dft**函数中以对新的输入in和输出out执行FFT变换。

如果函数返回非空指针，则表示plan创建成功，否则表示创建失败。

参数

参数名	数据类型	描述	输入/输出
rank	int	FFT变换的维度是rank。约束：1≤rank≤3。	输入
n	const int *	n为维度为rank的数组，包含FFT序列每一维度的大小。约束：n[i]≥1, for i in 0 to rank - 1。	输入
in	• 双精度：kml_fft_complex* • 单精度：kml_fftf_complex* • 半精度：kml_ffth_complex*	输入需要变换的数据。	输入
out	• 双精度：kml_fft_complex* • 单精度：kml_fftf_complex* • 半精度：kml_ffth_complex*	输出快速傅里叶变换后的数据。	输出
comm	MPI_Comm	MPI通信器的句柄。	输入
sign	int	描述正向变换或反向变换。 • -1(KML_FFT_FORWARD)：forward • +1(KML_FFT_BACKWARD)：backward	输入

版权所有 © 华为技术有限公司

参数名	数据类型	描述	输入/输出
flags	unsigned int	planning选项，可设置为如下选项： KML_FFT_ESTIMATE：ESTIMATE模式 KML_FFT_PATIENT：PATIENT模式 KML_FFT_MPI_SCRAMBLED_[IN|OUT]：表示输入/输出按照scrambled格式，仅对一维变换生效 KML_FFT_MPI_TRANSPOSED_[IN|OUT]：表示输入/输出由$n_0 \times n_1 \times n_2 \times \cdots \times n_{d-1}$转化为$n_1 \times n_0 \times n_2 \times \cdots \times n_{d-1}$	输入

依赖

C: "kfft-mpi.h"

示例

```
kml_fft_plan plan;
ptrdiff_t alloc_local, local_n0, local_0_start;

MPI_Init(&argc, &argv);
MPI_Comm comm = MPI_COMM_WORLD;
kml_fft_mpi_init();
kml_fft_complex *in = NULL;
kml_fft_complex *out = NULL;
ptrdiff_t rank = 2;
ptrdiff_t *n = (ptrdiff_t *)kml_fft_malloc(sizeof(ptrdiff_t) * rank);

n[0] = 3;
n[1] = 4;

/* obtain local data size and allocate */
alloc_local = kml_fft_mpi_local_size(rank, n, comm, &local_n0, &local_0_start);
if (alloc_local == -1) {
    printf("[%s][%d] allocate size fail.\n", __func__, __LINE__);
}
in = (kml_fft_complex *)kml_fft_malloc(sizeof(kml_fft_complex) * alloc_local);
if (in == NULL) {
    printf("[%s][%d] malloc memory fail.\n", __func__, __LINE__);
}
out = (kml_fft_complex *)kml_fft_malloc(sizeof(kml_fft_complex) * alloc_local);
if (out == NULL) {
    printf("[%s][%d] malloc memory fail.\n", __func__, __LINE__);
}

/* create plan for in-place forward DFT */
plan = kml_fft_mpi_plan_dft(rank, n, in, out, comm,
                            KML_FFT_FORWARD, KML_FFT_MPI_TRANSPOSED_IN);

/* compute transforms, in-place, as many times as desired */
kml_fft_execute(plan);

kml_fft_destroy_plan(plan);
```


版权所有 © 华为技术有限公司

```
kml_fft_mpi_cleanup();
MPI_Finalize();
```

6.4.4.5.1.2 kml_fft(f/h)_mpi_plan_dft_1d

建立单个连续数据序列1维C2C变换的plan。

接口定义

C interface:

kml_fft_plan kml_fft_mpi_plan_dft_1d(int n, kml_fft_complex *in, kml_fft_complex *out, MPI_Comm comm, int sign, unsigned flags);

kml_fftf_plan kml_fftf_mpi_plan_dft_1d(int n, kml_fftf_complex *in, kml_fftf_complex *out, MPI_Comm comm, int sign, unsigned flags);

kml_ffth_plan kml_ffth_mpi_plan_dft_1d(int n, kml_ffth_complex *in, kml_ffth_complex *out, MPI_Comm comm, int sign, unsigned flags);

返回值

函数返回一个kml_fft(f)_plan类型的结构体指针。将该对象作为参数传入 **kml_fft(f/h)_execute**函数中，将对当前提供的输入in和输出out执行FFT变换。也可以通过将该对象作为参数传入 **kml_fft(f/h)_execute_dft**函数中以对新的输入in和输出out执行FFT变换。

如果函数返回非空指针，则表示plan创建成功，否则表示创建失败。

参数

参数名	数据类型	描述	输入/输出
n	int	输入FFT序列的大小，约束：n≥1。	输入
in	• 双精度：kml_fft_complex* • 单精度：kml_fftf_complex* • 半精度：kml_ffth_complex*	输入需要变换的数据。	输入

版权所有 © 华为技术有限公司

参数名	数据类型	描述	输入/输出
out	• 双精度：kml_fft_complex* • 单精度：kml_fftf_complex* • 半精度：kml_ffth_complex*	输出快速傅里叶变换后的数据。	输出
comm	MPI_Comm	MPI通信器句柄。	输入
sign	int	描述正向变换或反向变换。 • -1(KML_FFT_FORWARD)：forward • +1(KML_FFT_BACKWARD)：backward	输入
flags	unsigned int	planning选项，可设置为如下选项： KML_FFT_ESTIMATE：ESTIMATE模式 KML_FFT_PATIENT：PATIENT模式 KML_FFT_MPI_SCRAMBLED_[IN|OUT]：表示输入/输出按照scrambled格式，仅对一维变换生效 KML_FFT_MPI_TRANSPOSED_[IN|OUT]：表示输入/输出由$n_0 \times n_1 \times n_2 \times \cdots \times n_{d-1}$转化为$n_1 \times n_0 \times n_2 \times \cdots \times n_{d-1}$	输入

依赖

C: "kfft-mpi.h"

示例

C interface：

```
const ptrdiff_t N0 = 4;
kml_fft_plan plan;
ptrdiff_t alloc_local, local_ni, local_i_start, local_no, local_o_start;

MPI_Init(&argc, &argv);
MPI_Comm comm = MPI_COMM_WORLD;
kml_fft_mpi_init();
kml_fft_complex *in = NULL;
kml_fft_complex *out = NULL;
```

 版权所有 © 华为技术有限公司

```
/* get local data size and allocate */
alloc_local = kml_fft_mpi_local_size_1d(N0, comm, KML_FFT_FORWARD, KML_FFT_MPI_TRANSPOSED_IN,
                                        &local_ni, &local_i_start, &local_no, &local_o_start);
if (alloc_local == -1) {
    printf("[%s][%d] allocate size fail.\n", __func__, __LINE__);
}
in = (kml_fft_complex *)kml_fft_malloc(sizeof(kml_fft_complex) * alloc_local);
if (in == NULL) {
    printf("[%s][%d] malloc memory fail.\n", __func__, __LINE__);
}
out = (kml_fft_complex *)kml_fft_malloc(sizeof(kml_fft_complex) * alloc_local);
if (out == NULL) {
    printf("[%s][%d] malloc memory fail.\n", __func__, __LINE__);
}

/* create plan for in-place forward DFT */
plan = kml_fft_mpi_plan_dft_1d(N0, in, out, comm,
                               KML_FFT_FORWARD, KML_FFT_MPI_TRANSPOSED_IN);

/* compute transforms, in-place, as many times as desired */
kml_fft_execute_dft(plan, in, out);

kml_fft_destroy_plan(plan);

MPI_Finalize();
return 0;
```

6.4.4.5.1.3 kml_fft(f/h)_mpi_plan_dft_2d

建立单个连续数据序列2维C2C变换的plan。

接口定义

C interface：

kml_fft_plan kml_mpi_fft_plan_dft_2d(int n0, int n1, kml_fft_complex *in, kml_fft_complex *out, MPI_Comm comm, int sign, unsigned flags);

kml_fftf_plan kml_mpi_fftf_plan_dft_2d(int n0, int n1, kml_fftf_complex *in, kml_fftf_complex *out, MPI_Comm comm, int sign, unsigned flags);

kml_ffth_plan kml_mpi_ffth_plan_dft_2d(int n0, int n1, kml_ffth_complex *in, kml_ffth_complex *out, MPI_Comm comm, int sign, unsigned flags);

返回值

函数返回一个kml_fft(f)_plan类型的结构体指针。将该对象作为参数传入 **kml_fft(f/h)_execute**函数中，将对当前提供的输入in和输出out执行FFT变换。也可以通过将该对象作为参数传入 **kml_fft(f/h)_execute_dft**函数中以对新的输入in和输出out执行FFT变换。

如果函数返回非空指针，则表示plan创建成功，否则表示创建失败。

参数

参数名	数据类型	描述	输入/输出
n0	int	输入FFT序列第一维的大小，约束：n0≥1。	输入

版权所有 © 华为技术有限公司

参数名	数据类型	描述	输入/输出
n1	int	输入FFT序列第二维的大小，约束：n1≥1。	输入
in	• 双精度：kml_fft_complex* • 单精度：kml_fftf_complex* • 半精度：kml_ffth_complex*	输入需要变换的数据。	输入
out	• 双精度：kml_fft_complex* • 单精度：kml_fftf_complex* • 半精度：kml_ffth_complex*	输出快速傅里叶变换后的数据。	输出
comm	MPI_Comm	MPI通信器句柄。	输入
sign	int	描述正向变换或反向变换。 • -1(KML_FFT_FORWARD)：forward • +1(KML_FFT_BACKWARD)：backward	输入
flags	unsigned int	planning选项，可设置为如下选项： KML_FFT_ESTIMATE：ESTIMATE模式 KML_FFT_PATIENT：PATIENT模式 KML_FFT_MPI_SCRAMBLED_[IN|OUT]：表示输入/输出按照scrambled格式，仅对一维变换生效 KML_FFT_MPI_TRANSPOSED_[IN|OUT]：表示输入/输出由$n_0 \times n_1 \times n_2 \times \cdots \times n_{d-1}$转化为$n_1 \times n_0 \times n_2 \times \cdots \times n_{d-1}$	输入

版权所有 © 华为技术有限公司

依赖

C: "kfft-mpi.h"

示例

C interface：

```
const ptrdiff_t N0 = 4, N1 = 4;
    kml_fft_plan plan;
    ptrdiff_t alloc_local, local_n0, local_0_start, i, j;

    MPI_Init(&argc, &argv);
    MPI_Comm comm = MPI_COMM_WORLD;
    kml_fft_mpi_init();
    kml_fft_complex *in = NULL;
    kml_fft_complex *out = NULL;

    /* get local data size and allocate */
    alloc_local = kml_fft_mpi_local_size_2d(N0, N1, comm,
                                        &local_n0, &local_0_start);
    if (alloc_local == -1) {
            printf("[%s][%d] allocate size fail.\n", __func__, __LINE__);
    }
    in = (kml_fft_complex *)kml_fft_malloc(sizeof(kml_fft_complex) * alloc_local);
    if (in == NULL) {
        printf("[%s][%d] malloc memory fail.\n", __func__, __LINE__);
    }
    out = (kml_fft_complex *)kml_fft_malloc(sizeof(kml_fft_complex) * alloc_local);
    if (out == NULL) {
        printf("[%s][%d] malloc memory fail.\n", __func__, __LINE__);
    }

    /* create plan for in-place forward DFT */
    plan = kml_fft_mpi_plan_dft_2d(N0, N1, in, out, comm,
                                KML_FFT_FORWARD, KML_FFT_MPI_TRANSPOSED_IN);

    /* compute transforms, in-place, as many times as desired */
    kml_fft_execute(plan);

    kml_fft_destroy_plan(plan);

    MPI_Finalize();
```

6.4.4.5.1.4 kml_fft(f/h)_mpi_plan_dft_3d

建立单个连续数据序列3维C2C变换的plan。

接口定义

C interface：

kml_fft_plan kml_fft_mpi_plan_dft_3d(int n0, int n1, int n2, kml_fft_complex *in, kml_fft_complex *out, MPI_Comm comm, int sign, unsigned flags);

kml_fftf_plan kml_fftf_mpi_plan_dft_3d(int n0, int n1, int n2, kml_fftf_complex *in, kml_fftf_complex *out, MPI_Comm comm, int sign, unsigned flags);

kml_ffth_plan kml_ffth_mpi_plan_dft_3d(int n0, int n1, int n2, kml_ffth_complex *in, kml_ffth_complex *out, MPI_Comm comm, int sign, unsigned flags);

返回值

函数返回一个kml_fft(f)_plan类型的结构体指针。将该对象作为参数传入 **kml_fft(f/h)_execute**函数中，将对当前提供的输入in和输出out执行FFT变换。也可以通过将该

 版权所有 © 华为技术有限公司

对象作为参数传入 **kml_fft(f/h)_execute_dft**函数中以对新的输入in和输出out执行FFT变换。

如果函数返回非空指针，则表示plan创建成功，否则表示创建失败。

参数

参数名	数据类型	描述	输入/输出
n0	int	输入FFT序列第一维的大小，约束：n0≥1。	输入
n1	int	输入FFT序列第二维的大小，约束：n1≥1。	输入
n2	int	输入FFT序列第三维的大小，约束：n2≥1。	输入
in	• 双精度：kml_fft_complex* • 单精度：kml_fftf_complex* • 半精度：kml_ffth_complex*	输入需要变换的数据。	输入
out	• 双精度：kml_fft_complex* • 单精度：kml_fftf_complex* • 半精度：kml_ffth_complex*	输出快速傅里叶变换后的数据。	输出
comm	MPI_Comm	MPI通信器句柄	输入
sign	int	描述正向变换或反向变换。 • -1(KML_FFT_FORWARD)：forward • +1(KML_FFT_BACKWARD)：backward	输入

版权所有 © 华为技术有限公司

参数名	数据类型	描述	输入/输出
flags	unsigned int	planning选项，可设置为如下选项： KML_FFT_ESTIMATE：ESTIMATE模式 KML_FFT_PATIENT：PATIENT模式 KML_FFT_MPI_SCRAMBLED_[IN|OUT]：表示输入/输出按照scrambled格式，仅对一维变换生效 KML_FFT_MPI_TRANSPOSED_[IN|OUT]：表示输入/输出由$n_0 \times n_1 \times n_2 \times \cdots \times n_{d-1}$转化为$n_1 \times n_0 \times n_2 \times \cdots \times n_{d-1}$	输入

依赖

C: "kfft-mpi.h"

示例

C interface：

```
const ptrdiff_t N0 = 4, N1 = 4, N2 = 4;
kml_fft_plan plan;
ptrdiff_t alloc_local, local_n0, local_0_start;

MPI_Init(&argc, &argv);
MPI_Comm comm = MPI_COMM_WORLD;
kml_fft_mpi_init();
kml_fft_complex *in = NULL;
kml_fft_complex *out = NULL;

/* get local data size and allocate */
alloc_local = kml_fft_mpi_local_size_3d(N0, N1, N2, comm,
                                        &local_n0, &local_0_start);
if (alloc_local == -1) {
    printf("[%s][%d] allocate size fail.\n", __func__, __LINE__);
}
in = (kml_fft_complex *)kml_fft_malloc(sizeof(kml_fft_complex) * alloc_local);
if (in == NULL) {
    printf("[%s][%d] malloc memory fail.\n", __func__, __LINE__);
}
out = (kml_fft_complex *)kml_fft_malloc(sizeof(kml_fft_complex) * alloc_local);
if (out == NULL) {
    printf("[%s][%d] malloc memory fail.\n", __func__, __LINE__);
}

/* create plan for in-place forward DFT */
plan = kml_fft_mpi_plan_dft_3d(N0, N1, N2, in, out, comm,
                               KML_FFT_FORWARD, KML_FFT_MPI_TRANSPOSED_IN);

/* compute transforms, in-place, as many times as desired */
kml_fft_execute(plan);

kml_fft_destroy_plan(plan);
```


版权所有 © 华为技术有限公司

```
MPI_Finalize();
```

6.4.4.5.1.5 kml_fft(f/h)_mpi_plan_many_dft

建立howmany组数据序列n维C2C变换的plan。其中单个FFT的数据序列不需要是连续的，可以以跨步的形式提供。

接口定义

C interface：

kml_fft_plan kml_fft_mpi_plan_dft_3d(int rank, const int *n, int howmany, ptrdiff_t iblock, ptrdiff_t oblock, kml_fft_complex *in, kml_fft_complex *out, MPI_Comm comm, int sign, unsigned flags);

kml_fftf_plan kml_fftf_mpi_plan_dft_3d(int rank, const int *n, int howmany, ptrdiff_t iblock, ptrdiff_t oblock, kml_fftf_complex *in, kml_fftf_complex *out, MPI_Comm comm, int sign, unsigned flags);

kml_ffth_plan kml_ffth_mpi_plan_dft_3d(int rank, const int *n, int howmany, ptrdiff_t iblock, ptrdiff_t oblock, kml_ffth_complex *in, kml_ffth_complex *out, MPI_Comm comm, int sign, unsigned flags);

返回值

函数返回一个kml_fft(f)_plan类型的结构体指针。将该对象作为参数传入 **kml_fft(f/h)_execute**函数中，将对当前提供的输入in和输出out执行FFT变换。也可以通过将该对象作为参数传入 **kml_fft(f/h)_execute_dft**函数中以对新的输入in和输出out执行FFT变换。

如果函数返回非空指针，则表示plan创建成功，否则表示创建失败。

参数

参数名	数据类型	描述	输入/输出
rank	int	FFT变换的维度是rank，约束：1≤rank≤3。	输入
n	const int*	n为维度为rank的数组，包含FFT序列每一维度的大小，约束：n[i]≥1, for i in 0, rank - 1。	输入
howmany	int	howmany表示指定多少个多维FFT变换。	输入
iblock	ptrdiff_t	n0维度上的block大小	输入
oblock	ptrdiff_t	n1维度上的block大小	输入

版权所有 © 华为技术有限公司

参数名	数据类型	描述	输入/输出
in	• 双精度：kml_fft_complex* • 单精度：kml_fftf_complex* • 半精度：kml_ffth_complex*	输入需要变换的数据。	输入
out	• 双精度：kml_fft_complex* • 单精度：kml_fftf_complex* • 半精度：kml_ffth_complex*	输出快速傅里叶变换后的数据。	输出
comm	MPI_Comm	MPI通信器句柄	输入
sign	int	描述正向变换或反向变换。 • -1(KML_FFT_FORWARD)：forward • +1(KML_FFT_BACKWARD)：backward	输入
flags	unsigned int	planning选项，可设置为如下选项： KML_FFT_ESTIMATE：ESTIMATE模式 KML_FFT_PATIENT：PATIENT模式 KML_FFT_MPI_SCRAMBLED_[IN|OUT]：表示输入/输出按照scrambled格式，仅对一维变换生效 KML_FFT_MPI_TRANSPOSED_[IN|OUT]：表示输入/输出由$n_0 \times n_1 \times n_2 \times \cdots \times n_{d-1}$转化为$n_1 \times n_0 \times n_2 \times \cdots \times n_{d-1}$	输入

依赖

C: "kfft-mpi.h"

 版权所有 © 华为技术有限公司

示例

C interface：

```
const ptrdiff_t N0 = 4, N1 = 4;
kml_fft_plan plan;
ptrdiff_t alloc_local, local_n0, local_0_start, i, j;

MPI_Init(&argc, &argv);
MPI_Comm comm = MPI_COMM_WORLD;
kml_fft_mpi_init();
kml_fft_complex *in = NULL;
kml_fft_complex *out = NULL;
ptrdiff_t block0 = 0;
ptrdiff_t block1 = 0;
ptrdiff_t howmany = 1;
ptrdiff_t rank = 2;
ptrdiff_t *n = (ptrdiff_t *)kml_fft_malloc(sizeof(ptrdiff_t) * rank);

n[0] = N0;
n[1] = N1;

/* get local data size and allocate */
alloc_local = kml_fft_mpi_local_size_many(rank, n, howmany, block0, comm,
                                &local_n0, &local_0_start);
if (alloc_local == -1) {
       printf("[%s][%d] allocate size fail.\n", __func__, __LINE__);
}
in = (kml_fft_complex *)kml_fft_malloc(sizeof(kml_fft_complex) * alloc_local);
if (in == NULL) {
    printf("[%s][%d] malloc memory fail.\n", __func__, __LINE__);
}
out = (kml_fft_complex *)kml_fft_malloc(sizeof(kml_fft_complex) * alloc_local);
if (out == NULL) {
    printf("[%s][%d] malloc memory fail.\n", __func__, __LINE__);
}

/* create plan for in-place forward DFT */
plan = kml_fft_mpi_plan_many_dft(rank, n, howmany, block0, block1, in, out, comm,
                            KML_FFT_FORWARD, KML_FFT_MPI_TRANSPOSED_IN);

/* compute transforms, in-place, as many times as desired */
kml_fft_execute_dft(plan, in, out);

kml_fft_destroy_plan(plan);

MPI_Finalize();
```

6.4.4.5.1.6 kml_fft(f)_mpi_plan_dft_3d_ext

建立单个连续数据序列3维C2C变换的plan。

接口定义

C interface：

kml_fft_plan kml_fft_mpi_plan_dft_3d_ext(int n0, int n1, int n2, const int *inbox_order, const int *outbox_order, MPI_Comm comm, const kml_fft_mpi_options options);

kml_fftf_plan kml_fftf_mpi_plan_dft_3d_ext(int n0, int n1, int n2, const int *inbox_order, const int *outbox_order, MPI_Comm comm, const kml_fftf_mpi_options options);

 版权所有 © 华为技术有限公司

返回值

函数返回一个kml_fft(f)_plan类型的结构体指针。将该对象作为参数传入**kml_fft(f)_mpi_execute_dft_ext函数**、**kml_fft(f)_mpi_forward_c2c函数**、**kml_fft(f)_mpi_backward_c2c函数**中使用，支持forward及backward方向复用，不需要在plan创建阶段指定in、out buffer。

如果函数返回非空指针，则表示plan创建成功，否则表示创建失败。

参数

参数名	数据类型	描述	输入/输出
n0	int	输入FFT序列第一维的大小，约束：n0≥1。	输入
n1	int	输入FFT序列第二维的大小，约束：n1≥1。	输入
n2	int	输入FFT序列第三维的大小，约束：n2≥1。	输入
inbox_order	const int *	是长度为3的一维数组，输入数据维度的顺序，取值为0、1、2的任意顺序组合，输入NULL，默认order为0、1、2。	输入
outbox_order	const int *	是长度为3的一维数组，输出数据维度的顺序，取值为0、1、2的任意顺序组合，输入NULL，默认order为0、1、2。	输入
comm	MPI_Comm	通信域。	输入
options	• 双精度：const kml_fft_mpi_options • 单精度： const kml_fftf_mpi_options	3DFFT分解算法、通信算法flag。	输入

依赖

C: "kfft-mpi.h"

示例

C interface：

```
const int n0 = 4, n1 = 4, n2 = 4;
kml_fft_plan plan;
int provided;
MPI_Init_thread(&argc, &argv, MPI_THREAD_MULTIPLE, &provided);
MPI_Comm comm = MPI_COMM_WORLD;
```

 版权所有 © 华为技术有限公司

```
    kml_fft_complex *in = NULL;
    kml_fft_complex *out = NULL;

    /* get local data size and allocate */
    ptrdiff_t low[3] = {0};
    ptrdiff_t high[3] = {0};
    ptrdiff_t alloc_local = kml_fft_mpi_local_size_3d_ext(n0, n1, n2, comm,
SCALFFT_DECOMPOSE_TYPE_PENCIL, low, high);
    if (alloc_local == -1) {
        printf("[%s][%d] allocate size fail.\n", __func__, __LINE__);
    }
    in = (kml_fft_complex *)kml_fft_malloc(sizeof(kml_fft_complex) * alloc_local);
    if (in == NULL) {
        printf("[%s][%d] malloc memory fail.\n", __func__, __LINE__);
    }
    out = (kml_fft_complex *)kml_fft_malloc(sizeof(kml_fft_complex) * alloc_local);
    if (out == NULL) {
        printf("[%s][%d] malloc memory fail.\n", __func__, __LINE__);
    }

    /* create plan */
    kml_fft_mpi_options options = {
        .a2a_algo    = A2A_ALGO_AUTO_TUNING,
        .decomp_type = SCALFFT_DECOMPOSE_TYPE_PENCIL
    };
    plan = kml_fft_mpi_plan_dft_3d_ext(n0, n1, n2, NULL, NULL, comm, options);

    /* execute plan */
    int scale = 0;
    kml_fft_mpi_execute_dft_ext(plan, in, out, scale, KML_FFT_FORWARD);
    kml_fft_mpi_execute_dft_ext(plan, out, in, scale, KML_FFT_BACKWARD);

    kml_fft_destroy_plan_ext(plan);

    kml_fft_free(in);
    kml_fft_free(out);
    MPI_Finalize();
```

6.4.4.5.1.7 kml_fft(f)_mpi_plan_create

建立单个连续数据序列3维C2C变换的plan。

接口定义

C interface：

kml_fft_plan kml_fft_mpi_plan_create(int backend, const int *inbox_low, const int *inbox_high, const int *inbox_order, const int *outbox_low, const int *outbox_high, const int *outbox_order, MPI_Comm comm, const kml_fft_mpi_options options);

kml_fftf_plan kml_fftf_mpi_plan_create(int backend, const int *inbox_low, const int *inbox_high, const int *inbox_order, const int *outbox_low, const int *outbox_high, const int *outbox_order, MPI_Comm comm, const kml_fftf_mpi_options options);

返回值

函数返回一个kml_fft(f)_plan类型的结构体指针。将该对象作为参数传入**kml_fft(f)_mpi_execute_dft_ext函数**、**kml_fft(f)_mpi_forward_c2c函数**、**kml_fft(f)_mpi_backward_c2c函数**中使用，支持forward及backward方向复用，且不需要在plan创建阶段指定in、out buffer。

如果函数返回非空指针，则表示plan创建成功，否则表示创建失败。

 版权所有 © 华为技术有限公司

参数

参数名	数据类型	描述	输入/输出
backend	int	底层库类型，当前仅支持BACKEND_KFFT。	输入
inbox_low	const int *	是长度为3的一维数组，输入数据在三个维度的切分范围下限，-1≤inbox_low[i]≤n[i]-1 for i in 0、1、2。	输入
inbox_high	const int *	是长度为3的一维数组，输入数据在三个维度的切分范围上限，-1≤inbox_high[i]≤n[i]-1 for i in 0、1、2，且0≤（inbox_high[i]-inbox_low[i]+1）≤n[i] for i in 0、1、2。	输入
inbox_order	const int *	是长度为3的一维数组，输入数据维度的顺序，取值为0、1、2的任意顺序组合，输入NULL，默认order为0、1、2。	输入
outbox_low	const int *	是长度为3的一维数组，输出数据在三个维度的切分范围下限，-1≤outbox_low[i]≤n[i]-1 for i in 0、1、2。	输入
outbox_high	const int *	是长度为3的一维数组，输出数据在三个维度的切分范围上限，-1≤outbox_high[i]≤n[i]-1 for i in 0、1、 2，且 0≤(outbox_high[i] - outbox_low[i] + 1)≤n[i] for i in 0、1、2。	输入
outbox_order	const int *	是长度为3的一维数组，输出数据维度的顺序，取值为0、1、2的任意顺序组合，输入NULL，默认order为0、1、2。	输入
comm	MPI_Comm	通信域。	输入
options	• 双精度：const kml_fft_mpi_options • 单精度： const kml_fftf_mpi_options	3DFFT分解算法、通信算法flag。	输入

依赖

C: "kfft-mpi.h"

 版权所有 © 华为技术有限公司

示例

C interface：

```
    const int n0 = 4, n1 = 4, n2 = 4;
    kml_fft_plan plan;
    int provided;
    MPI_Init_thread(&argc, &argv, MPI_THREAD_MULTIPLE, &provided);
    MPI_Comm comm = MPI_COMM_WORLD;
    kml_fft_complex *in = NULL;
    kml_fft_complex *out = NULL;

    /* get local data size and allocate */
    ptrdiff_t low[3] = {0};
    ptrdiff_t high[3] = {0};
    ptrdiff_t n[3] = {n0, n1, n2};
    ptrdiff_t alloc_local = kml_fft_mpi_local_size_ext(3, n, comm, SCALFFT_DECOMPOSE_TYPE_PENCIL, low,
high);
    if (alloc_local == -1) {
        printf("[%s][%d] allocate size fail.\n", __func__, __LINE__);
    }
    in = (kml_fft_complex *)kml_fft_malloc(sizeof(kml_fft_complex) * alloc_local);
    if (in == NULL) {
        printf("[%s][%d] malloc memory fail.\n", __func__, __LINE__);
    }
    out = (kml_fft_complex *)kml_fft_malloc(sizeof(kml_fft_complex) * alloc_local);
    if (out == NULL) {
        printf("[%s][%d] malloc memory fail.\n", __func__, __LINE__);
    }

    /* create plan */
    int low_int[3] = {low[0], low[1], low[2]};
    int high_int[3] = {high[0], high[1], high[2]};
    kml_fft_mpi_options options = {
        .a2a_algo    = A2A_ALGO_AUTO_TUNING,
        .decomp_type = SCALFFT_DECOMPOSE_TYPE_PENCIL
    };
    plan = kml_fft_mpi_plan_create(BACKEND_KFFT, low_int, high_int, NULL, low_int, high_int, NULL, comm,
options);

    /* execute plan */
    int scale = 0;
    kml_fft_mpi_execute_dft_ext(plan, in, out, scale, KML_FFT_FORWARD);
    kml_fft_mpi_execute_dft_ext(plan, out, in, scale, KML_FFT_BACKWARD);
    // kml_fft_mpi_forward_c2c(plan, in, out, scale);
    // kml_fft_mpi_backward_c2c(plan, out, in, scale);

    kml_fft_destroy_plan_ext(plan);

    kml_fft_free(in);
    kml_fft_free(out);
    MPI_Finalize();
```

6.4.4.5.2 R2C 变换

6.4.4.5.2.1 kml_fft(f/h)_mpi_plan_dft_r2c

建立单个连续数据序列n维R2C变换的plan。

接口定义

C interface：

kml_fft_plan kml_fft_mpi_plan_dft_r2c(int rank, const int *n, double *in, kml_fft_complex *out, MPI_Comm comm, unsigned flags);

 版权所有 © 华为技术有限公司

kml_fftf_plan kml_fftf_mpi_plan_dft_r2c(int rank, const int *n, float *in, kml_fftf_complex *out, MPI_Comm comm, unsigned flags);

kml_ffth_plan kml_ffth_mpi_plan_dft_r2c(int rank, const int *n, __fp16 *in, kml_ffth_complex *out, MPI_Comm comm, unsigned flags);

返回值

函数返回一个kml_fft(f)_plan类型的结构体指针。将该对象作为参数传入 **kml_fft(f/h)_execute**函数中，将对当前提供的输入in和输出out执行FFT变换。也可以通过将该对象作为参数传入 **kml_fft(f/h)_execute_dft_r2c**函数中以对新的输入in和输出out执行FFT变换。

如果函数返回非空指针，则表示plan创建成功，否则表示创建失败。

参数

参数名	数据类型	描述	输入/输出
rank	int	FFT变换的维度是rank，约束：1 <= rank <= 3。	输入
n	const int*	n为维度为rank的数组，包含FFT序列每一维度的大小，约束：n[i] >= 1, for i in 0 to rank - 1。	输入
in	• 双精度：double* • 单精度：float* • 半精度：__fp16*	输入需要变换的数据。	输入
out	• 双精度：kml_fft_complex* • 单精度：kml_fftf_complex* • 半精度：kml_ffth_complex*	输出快速傅里叶变换后的数据。	输出
comm	MPI_Comm	MPI句柄通信器	输入

版权所有 © 华为技术有限公司

参数名	数据类型	描述	输入/输出
flags	unsigned int	planning选项，可设置为如下选项： KML_FFT_ESTIMATE：ESTIMATE模式 KML_FFT_PATIENT：PATIENT模式 KML_FFT_MPI_SCRAMBLED_[IN|OUT]：表示输入/输出按照scrambled格式，仅对一维变换生效 KML_FFT_MPI_TRANSPOSED_[IN|OUT]：表示输入/输出由$n_0 \times n_1 \times n_2 \times \cdots \times n_{d-1}$转化为$n_1 \times n_0 \times n_2 \times \cdots \times n_{d-1}$	输入

依赖

C: "kfft-mpi.h"

示例

C interface：

```
const ptrdiff_t N0 = 4, N1 = 4;
kml_fft_plan plan;
ptrdiff_t alloc_local, local_n0, local_0_start;

MPI_Init(&argc, &argv);
MPI_Comm comm = MPI_COMM_WORLD;
kml_fft_mpi_init();
double *in = NULL;
kml_fft_complex *out = NULL;
ptrdiff_t rank = 2;
ptrdiff_t *n = (ptrdiff_t *)kml_fft_malloc(sizeof(ptrdiff_t) * rank);

n[0] = N0;
n[1] = N1;

/* get local data size and allocate */
alloc_local = kml_fft_mpi_local_size(rank, n, comm, &local_n0, &local_0_start);
if (alloc_local == -1) {
    printf("[%s][%d] allocate size fail.\n", __func__, __LINE__);
}
in = (double *)kml_fft_malloc(sizeof(double) * alloc_local * 2);
if (in == NULL) {
    printf("[%s][%d] malloc memory fail.\n", __func__, __LINE__);
}
out = (kml_fft_complex *)kml_fft_malloc(sizeof(kml_fft_complex) * alloc_local);
if (out == NULL) {
    printf("[%s][%d] malloc memory fail.\n", __func__, __LINE__);
}

/* create plan for in-place forward DFT */
plan = kml_fft_mpi_plan_dft_r2c(rank, n, in, out, comm, KML_FFT_ESTIMATE);

/* compute transforms, in-place, as many times as desired */
kml_fft_execute_dft_r2c(plan, in, out);
```

 版权所有 © 华为技术有限公司

```
kml_fft_destroy_plan(plan);
kml_fft_mpi_cleanup();
MPI_Finalize();
```

6.4.4.5.2.2 kml_fft(f)_mpi_plan_dft_r2c_2d

建立单个连续数据序列2维R2C变换的plan。

接口定义

C interface：

kml_fft_plan kml_fft_mpi_plan_dft_r2c_2d(int n0, int n1, double *in, kml_fft_complex *out, MPI_Comm comm, unsigned flags);

kml_fftf_plan kml_fftf_mpi_plan_dft_r2c_2d(int n0, int n1, float *in, kml_fftf_complex *out, MPI_Comm comm, unsigned flags);

返回值

函数返回一个kml_fft(f)_plan类型的结构体指针。将该对象作为参数传入 **kml_fft(f/h)_execute**函数中，将对当前提供的输入in和输出out执行FFT变换。也可以通过将该对象作为参数传入 **kml_fft(f/h)_execute_dft_r2c**函数中以对新的输入in和输出out执行FFT变换。

如果函数返回非空指针，则表示plan创建成功，否则表示创建失败。

参数

参数名	数据类型	描述	输入/输出
n0	int	输入FFT序列第一维的大小，约束：n0 ≥ 1。	输入
n1	int	输入FFT序列第二维的大小，约束：n1 ≥ 1。	输入
in	• 双精度：double* • 单精度：float*	输入需要变换的数据。	输入
out	• 双精度：kml_fft_complex* • 单精度：kml_fftf_complex*	输出快速傅里叶变换后的数据。	输出
comm	MPI_Comm	MPI句柄通信器	输入

版权所有 © 华为技术有限公司

参数名	数据类型	描述	输入/输出
flags	unsigned int	planning选项，可设置为如下选项： KML_FFT_ESTIMATE：ESTIMATE模式 KML_FFT_PATIENT：PATIENT模式 KML_FFT_MPI_SCRAMBLED_[IN|OUT]：表示输入/输出按照scrambled格式，仅对一维变换生效 KML_FFT_MPI_TRANSPOSED_[IN|OUT]：表示输入/输出由$n_0 \times n_1 \times n_2 \times \cdots \times n_{d-1}$转化为$n_1 \times n_0 \times n_2 \times \cdots \times n_{d-1}$	输入

依赖

C: "kfft-mpi.h"

示例

C interface：

```
const ptrdiff_t N0 = 4, N1 = 4;
kml_fft_plan plan;
ptrdiff_t alloc_local, local_n0, local_0_start;

MPI_Init(&argc, &argv);
MPI_Comm comm = MPI_COMM_WORLD;
kml_fft_mpi_init();
double *in = NULL;
kml_fft_complex *out = NULL;

/* get local data size and allocate */
alloc_local = kml_fft_mpi_local_size_2d(N0, N1, comm, &local_n0, &local_0_start);
if (alloc_local == -1) {
    printf("[%s][%d] allocate size fail.\n", __func__, __LINE__);
}
in = (double *)kml_fft_malloc(sizeof(double) * alloc_local * 2);
if (in == NULL) {
    printf("[%s][%d] malloc memory fail.\n", __func__, __LINE__);
}
out = (kml_fft_complex *)kml_fft_malloc(sizeof(kml_fft_complex) * alloc_local);
if (out == NULL) {
    printf("[%s][%d] malloc memory fail.\n", __func__, __LINE__);
}

/* create plan for in-place forward DFT */
plan = kml_fft_mpi_plan_dft_r2c_2d(N0, N1, in, out, comm, KML_FFT_MPI_TRANSPOSED_OUT);

/* compute transforms, in-place, as many times as desired */
kml_fft_execute_dft_r2c(plan, in, out);

kml_fft_destroy_plan(plan);
kml_fft_mpi_cleanup();

MPI_Finalize();
```

 版权所有 © 华为技术有限公司

6.4.4.5.2.3 kml_fft(f)_mpi_plan_dft_r2c_3d

建立单个连续数据序列3维R2C变换的plan。

接口定义

C interface：

kml_fft_plan kml_fft_mpi_plan_dft_r2c_3d(int n0, int n1, int n2, double *in, kml_fft_complex *out, MPI_Comm comm, unsigned flags);

kml_fftf_plan kml_fftf_mpi_plan_dft_r2c_3d(int n0, int n1, int n2, float *in, kml_fftf_complex *out, MPI_Comm comm, unsigned flags);

返回值

函数返回一个kml_fft(f)_plan类型的结构体指针。将该对象作为参数传入 **kml_fft(f/h)_execute**函数中，将对当前提供的输入in和输出out执行FFT变换。也可以通过将该对象作为参数传入 **kml_fft(f/h)_execute_dft_r2c**函数中以对新的输入in和输出out执行FFT变换。

如果函数返回非空指针，则表示plan创建成功，否则表示创建失败。

参数

参数名	数据类型	描述	输入/输出
n0	int	输入FFT序列第一维的大小，约束：n0 ≥ 1。	输入
n1	int	输入FFT序列第二维的大小，约束：n1 ≥ 1。	输入
n2	int	输入FFT序列第三维的大小，约束：n2 ≥ 1。	输入
in	• 双精度：double* • 单精度：float*	输入需要变换的数据。	输入
out	• 双精度：kml_fft_complex* • 单精度：kml_fftf_complex*	输出快速傅里叶变换后的数据。	输出
comm	MPI_Comm	MPI句柄通信器	输入

版权所有 © 华为技术有限公司

参数名	数据类型	描述	输入/输出
flags	unsigned int	planning选项，可设置为如下选项： KML_FFT_ESTIMATE：ESTIMATE模式 KML_FFT_PATIENT：PATIENT模式 KML_FFT_MPI_SCRAMBLED_[IN|OUT]：表示输入/输出按照scrambled格式，仅对一维变换生效 KML_FFT_MPI_TRANSPOSED_[IN|OUT]：表示输入/输出由$n_0 \times n_1 \times n_2 \times \cdots \times n_{d-1}$转化为$n_1 \times n_0 \times n_2 \times \cdots \times n_{d-1}$	输入

依赖

C: "kfft-mpi.h"

示例

C interface：

```
const ptrdiff_t N0 = 4, N1 = 4, N2 = 4;
kml_fft_plan plan;
ptrdiff_t alloc_local, local_n0, local_0_start;

MPI_Init(&argc, &argv);
MPI_Comm comm = MPI_COMM_WORLD;
kml_fft_mpi_init();
double *in = NULL;
kml_fft_complex *out = NULL;

/* get local data size and allocate */
alloc_local = kml_fft_mpi_local_size_3d(N0, N1, N2, comm, &local_n0, &local_0_start);
if (alloc_local == -1) {
    printf("[%s][%d] allocate size fail.\n", __func__, __LINE__);
}
in = (double *)kml_fft_malloc(sizeof(double) * alloc_local * 2);
if (in == NULL) {
    printf("[%s][%d] malloc memory fail.\n", __func__, __LINE__);
}
out = (kml_fft_complex *)kml_fft_malloc(sizeof(kml_fft_complex) * alloc_local);
if (out == NULL) {
    printf("[%s][%d] malloc memory fail.\n", __func__, __LINE__);
}

/* create plan for in-place forward DFT */
plan = kml_fft_mpi_plan_dft_r2c_3d(N0, N1, N2, in, out, comm, KML_FFT_ESTIMATE);

/* compute transforms, in-place, as many times as desired */
kml_fft_execute(plan);

kml_fft_destroy_plan(plan);
kml_fft_mpi_cleanup();

MPI_Finalize();
```

 版权所有 © 华为技术有限公司

6.4.4.5.2.4 kml_fft(f/h)_mpi_plan_many_dft_r2c

建立howmany组数据序列n维R2C变换的plan。其中，单个FFT的数据序列不需要是连续的，可以以跨步的形式提供。

接口定义

C interface：

kml_fft_plan kml_fft_mpi_plan_many_dft_r2c(int rank, const int *n, int howmany, ptrdiff_t iblock, ptrdiff_t oblock, double *in, kml_fft_complex *out, MPI_Comm comm, unsigned flags);

kml_fftf_plan kml_fftf_mpi_plan_many_dft_r2c(int rank, const int *n, int howmany, ptrdiff_t iblock, ptrdiff_t oblock, float *in, kml_fftf_complex *out, MPI_Comm comm, unsigned flags);

kml_ffth_plan kml_ffth_mpi_plan_many_dft_r2c(int rank, const int *n, int howmany, ptrdiff_t iblock, ptrdiff_t oblock, __fp16 *in, kml_ffth_complex *out, MPI_Comm comm, unsigned flags);

返回值

函数返回一个kml_fft(f)_plan类型的结构体指针。将该对象作为参数传入 **kml_fft(f/h)_execute**函数中，将对当前提供的输入in和输出out执行FFT变换。也可以通过将该对象作为参数传入 **kml_fft(f/h)_execute_dft_r2c**函数中以对新的输入in和输出out执行FFT变换。

如果函数返回非空指针，则表示plan创建成功，否则表示创建失败。

参数

参数名	数据类型	描述	输入/输出
rank	int	FFT变换的维度是rank，约束：1 ≤ rank ≤ 3。	输入
n	const int*	n为维度为rank的数组，包含FFT序列每一维度的大小，约束：n[i] ≥ 1, for i in 0 to rank - 1。	输入
howmany	int	howmany表示指定多少个多维FFT变换。	输入
iblock	ptrdiff_t	n0维度上的block大小	输入
oblock	ptrdiff_t	n1维度上的block大小	输入
in	• 双精度：double* • 单精度：float* • 半精度：__fp16*	输入需要变换的数据。	输入

版权所有 © 华为技术有限公司

参数名	数据类型	描述	输入/输出
out	• 双精度：kml_fft_complex* • 单精度：kml_fftf_complex* • 半精度：kml_ffth_complex*	输出快速傅里叶变换后的数据。	输出
comm	MPI_Comm	MPI句柄通信器	输入
flags	unsigned int	planning选项，可设置为如下选项： KML_FFT_ESTIMATE：ESTIMATE模式 KML_FFT_PATIENT：PATIENT模式 KML_FFT_MPI_SCRAMBLED_[IN|OUT]：表示输入/输出按照scrambled格式，仅对一维变换生效 KML_FFT_MPI_TRANSPOSED_[IN|OUT]：表示输入/输出由$n_0 \times n_1 \times n_2 \times \cdots \times n_{d-1}$转化为$n_1 \times n_0 \times n_2 \times \cdots \times n_{d-1}$	输入

依赖

C: "kfft-mpi.h"

示例

C interface：

```
const ptrdiff_t N0 = 4, N1 = 4;
kml_fft_plan plan;
ptrdiff_t alloc_local, local_n0, local_0_start;

MPI_Init(&argc, &argv);
MPI_Comm comm = MPI_COMM_WORLD;
kml_fft_mpi_init();
ptrdiff_t block0 = 0;
ptrdiff_t block1 = 0;
ptrdiff_t howmany = 1;
double *in = NULL;
kml_fft_complex *out = NULL;
ptrdiff_t rank = 2;
ptrdiff_t *n = (ptrdiff_t *)kml_fft_malloc(sizeof(ptrdiff_t) * rank);

n[0] = N0;
n[1] = N1;
```

 版权所有 © 华为技术有限公司

```
    /* get local data size and allocate */
    alloc_local = kml_fft_mpi_local_size_many(rank, n, howmany, block0, comm, &local_n0, &local_0_start);
    if (alloc_local == -1) {
        printf("[%s][%d] allocate size fail.\n", __func__, __LINE__);
    }
    in = (double *)kml_fft_malloc(sizeof(double) * alloc_local * 2);
    if (in == NULL) {
        printf("[%s][%d] malloc memory fail.\n", __func__, __LINE__);
    }
    out = (kml_fft_complex *)kml_fft_malloc(sizeof(kml_fft_complex) * alloc_local);
    if (out == NULL) {
        printf("[%s][%d] malloc memory fail.\n", __func__, __LINE__);
    }

    /* create plan for in-place forward DFT */
    plan = kml_fft_mpi_plan_many_dft_r2c(rank, n, howmany, block0, block1, in, out, comm,
KML_FFT_ESTIMATE);

    /* compute transforms, in-place, as many times as desired */
    kml_fft_execute(plan);

    kml_fft_destroy_plan(plan);
    kml_fft_mpi_cleanup();

    MPI_Finalize();
```

6.4.4.5.2.5 kml_fft(f)_mpi_plan_dft_r2c_3d_ext

建立单个连续数据序列3维R2C变换的plan。

接口定义

C interface：

kml_fft_plan kml_fft_mpi_plan_dft_r2c_3d_ext(int n0, int n1, int n2, const int *inbox_order, const int *outbox_order, MPI_Comm comm, const kml_fft_mpi_options options);

kml_fftf_plan kml_fftf_mpi_plan_dft_r2c_3d_ext(int n0, int n1, int n2, const int *inbox_order, const int *outbox_order, MPI_Comm comm, const kml_fftf_mpi_options options);

返回值

函数返回一个kml_fft(f)_plan类型的结构体指针。将该对象作为参数传入**kml_fft(f)_mpi_execute_dft_r2c_ext函数**、 **kml_fft(f)_mpi_forward_r2c**函数中，且不需要在plan创建阶段指定in、out buffer。

如果函数返回非空指针，则表示plan创建成功，否则表示创建失败。

参数

参数名	数据类型	描述	输入/输出
n0	int	输入FFT序列第一维的大小，约束：n0≥1。	输入
n1	int	输入FFT序列第二维的大小，约束：n1≥1。	输入

版权所有 © 华为技术有限公司

参数名	数据类型	描述	输入/输出
n2	int	输入FFT序列第三维的大小，约束：n2≥1。	输入
inbox_order	const int *	是长度为3的一维数组，输入数据维度的顺序，取值为0、1、2的任意顺序组合，输入NULL，默认order为0、1、2。	输入
outbox_order	const int *	是长度为3的一维数组，输出数据维度的顺序，取值为0、1、2的任意顺序组合，输入NULL，默认order为0、1、2。	输入
comm	MPI_Comm	通信域	输入
options	• 双精度：const kml_fft_mpi_options • 单精度： const kml_fftf_mpi_options	3DFFT分解算法、通信算法flag	输入

依赖

C: "kfft-mpi.h"

示例

C interface：

```
    const int n0 = 4, n1 = 4, n2 = 4;
    kml_fft_plan plan;
    int provided;
    MPI_Init_thread(&argc, &argv, MPI_THREAD_MULTIPLE, &provided);
    MPI_Comm comm = MPI_COMM_WORLD;
    double *in = NULL;
    kml_fft_complex *out = NULL;

    /* get local data size and allocate */
    ptrdiff_t low[3] = {0};
    ptrdiff_t high[3] = {0};
    ptrdiff_t alloc_local = kml_fft_mpi_local_size_3d_ext(n0, n1, n2, comm,
SCALFFT_DECOMPOSE_TYPE_PENCIL, low, high);
    if (alloc_local == -1) {
        printf("[%s][%d] allocate size fail.\n", __func__, __LINE__);
    }
    in = (double *)kml_fft_malloc(sizeof(double) * alloc_local * 2);
    if (in == NULL) {
        printf("[%s][%d] malloc memory fail.\n", __func__, __LINE__);
    }
    out = (kml_fft_complex *)kml_fft_malloc(sizeof(kml_fft_complex) * alloc_local);
    if (out == NULL) {
        printf("[%s][%d] malloc memory fail.\n", __func__, __LINE__);
    }
```


版权所有 © 华为技术有限公司

```
/* create plan */
 kml_fft_mpi_options options = {
    .a2a_algo    = A2A_ALGO_AUTO_TUNING,
    .decomp_type = SCALFFT_DECOMPOSE_TYPE_PENCIL
 };
 plan = kml_fft_mpi_plan_dft_r2c_3d_ext(n0, n1, n2, NULL, NULL, comm, options);

 /* execute plan */
 int scale = 0;
 kml_fft_mpi_execute_dft_r2c_ext(plan, in, out, scale);

 kml_fft_destroy_plan_ext(plan);

 kml_fft_free(in);
 kml_fft_free(out);
 MPI_Finalize();
```

6.4.4.5.2.6 kml_fft(f)_mpi_plan_create_r2c

建立单个连续数据序列3维R2C变换的plan。

接口定义

C interface：

kml_fft_plan kml_fft_mpi_plan_create_r2c(int backend, const int *inbox_low, const int *inbox_high, const int *inbox_order, const int *outbox_low, const int *outbox_high, const int *outbox_order, MPI_Comm comm, const kml_fft_mpi_options options);

kml_fftf_plan kml_fftf_mpi_plan_create_r2c(int backend, const int *inbox_low, const int *inbox_high, const int *inbox_order, const int *outbox_low, const int *outbox_high, const int *outbox_order, MPI_Comm comm, const kml_fftf_mpi_options options);

返回值

函数返回一个kml_fft(f)_plan类型的结构体指针。将该对象作为参数传入**kml_fft(f)_mpi_execute_dft_r2c_ext函数**、**kml_fft(f)_mpi_forward_r2c函数**中使用，且不需要在plan创建阶段指定in、out buffer。

如果函数返回非空指针，则表示plan创建成功，否则表示创建失败。

参数

参数名	数据类型	描述	输入/输出
backend	int	底层库类型，当前仅支持BACKEND_KFFT。	输入
inbox_low	const int *	是长度为3的一维数组，输入数据在三个维度的切分范围下限，-1≤inbox_low[i]≤n[i]-1 for i in 0、1、2。	输入

版权所有 © 华为技术有限公司

参数名	数据类型	描述	输入/输出
inbox_high	const int *	是长度为3的一维数组，输入数据在三个维度的切分范围上限，-1≤inbox_high[i]≤n[i]-1 for i in 0、1、2，且0≤（inbox_high[i]-inbox_low[i]+1）≤n[i] for i in 0、1、2。	输入
inbox_order	const int *	是长度为3的一维数组，输入数据维度的顺序，取值为0、1、2的任意顺序组合，输入NULL，默认order为0、1、2。	输入
outbox_low	const int *	是长度为3的一维数组，输出数据在三个维度的切分范围下限，-1≤outbox_low[i]≤n[i]-1 for i in 0、1、2。	输入
outbox_high	const int *	是长度为3的一维数组，输出数据在三个维度的切分范围上限，-1≤outbox_high[i]≤n[i]-1 for i in 0、1、2，且 0≤(outbox_high[i] - outbox_low[i] + 1)≤n[i] for i in 0、1、2。	输入
outbox_order	const int *	是长度为3的一维数组，输出数据维度的顺序，取值为0、1、2的任意顺序组合，输入NULL，默认order为0、1、2。	输入
comm	MPI_Comm	通信域。	输入
options	• 双精度：const kml_fft_mpi_options • 单精度： const kml_fftf_mpi_options	3DFFT分解算法、通信算法flag。	输入

依赖

C: "kfft-mpi.h"

示例

C interface：

```
const int n0 = 4, n1 = 4, n2 = 4;
kml_fft_plan plan;
int provided;
MPI_Init_thread(&argc, &argv, MPI_THREAD_MULTIPLE, &provided);
MPI_Comm comm = MPI_COMM_WORLD;
```

 版权所有 © 华为技术有限公司

```
    double *in = NULL;
    kml_fft_complex *out = NULL;

    /* get local data size and allocate */
    ptrdiff_t low[3] = {0};
    ptrdiff_t high[3] = {0};
    ptrdiff_t alloc_local = kml_fft_mpi_local_size_3d_ext(n0, n1, n2, comm,
SCALFFT_DECOMPOSE_TYPE_PENCIL, low, high);
    if (alloc_local == -1) {
        printf("[%s][%d] allocate size fail.\n", __func__, __LINE__);
    }
    in = (double*)kml_fft_malloc(sizeof(double) * alloc_local * 2);
    if (in == NULL) {
        printf("[%s][%d] malloc memory fail.\n", __func__, __LINE__);
    }
    out = (kml_fft_complex *)kml_fft_malloc(sizeof(kml_fft_complex) * alloc_local);
    if (out == NULL) {
        printf("[%s][%d] malloc memory fail.\n", __func__, __LINE__);
    }

    /* create plan */
    int low_int[3] = {low[0], low[1], low[2]};
    int high_int[3] = {high[0], high[1], high[2]};
    kml_fft_mpi_options options = {
    .a2a_algo    = A2A_ALGO_AUTO_TUNING,
    .decomp_type = SCALFFT_DECOMPOSE_TYPE_PENCIL
    };
    plan = kml_fft_mpi_plan_create_r2c(BACKEND_KFFT, low_int, high_int, NULL, low_int, high_int, NULL,
comm, options);

    /* execute plan */
    int scale = 0;
    kml_fft_mpi_execute_dft_r2c_ext(plan, in, out, scale);
    // kml_fft_mpi_forward_r2c(plan, in, out, scale);

    kml_fft_destroy_plan_ext(plan);

    kml_fft_free(in);
    kml_fft_free(out);
    MPI_Finalize();
```

6.4.4.5.3 C2R 变换

6.4.4.5.3.1 kml_fft(f/h)_mpi_plan_dft_c2r

建立单个连续数据序列n维C2R变换的plan。

接口定义

C interface：

kml_fft_plan kml_fft_mpi_plan_dft_c2r(int rank, const int *n, kml_fft_complex *in, double *out, MPI_Comm comm, unsigned flags);

kml_fftf_plan kml_fftf_mpi_plan_dft_c2r(int rank, const int *n, kml_fftf_complex *in, float *out, MPI_Comm comm, unsigned flags);

kml_fftf_plan kml_ffth_mpi_plan_dft_c2r(int rank, const int *n, kml_fftf_complex *in, float *out, MPI_Comm comm, unsigned flags);

返回值

函数返回一个kml_fft(f)_plan类型的结构体指针。将该对象作为参数传入 **kml_fft(f/h)_execute**函数中，将对当前提供的输入in和输出out执行FFT变换。也可以通过将该

 版权所有 © 华为技术有限公司

对象作为参数传入 **kml_fft(f/h)_execute_dft_c2r**函数中以对新的输入in和输出out执行FFT变换。

如果函数返回非空指针，则表示plan创建成功，否则表示创建失败。

参数

参数名	数据类型	描述	输入/输出
rank	int	FFT变换的维度是rank，约束：1 ≤ rank ≤ 3。	输入
n	const int*	n为维度为rank的数组，包含FFT序列每一维度的大小，约束：n[i] ≥ 1, for i in 0 to rank - 1。	输入
in	• 双精度：kml_fft_complex* • 单精度：kml_fftf_complex*	输入需要变换的数据。	输入
out	• 双精度：double* • 单精度：float*	输出快速傅里叶变换后的数据。	输出
comm	MPI_Comm	MPI通信器句柄	输入
flags	unsigned int	planning选项，可设置为如下选项： KML_FFT_ESTIMATE：ESTIMATE模式 KML_FFT_PATIENT：PATIENT模式 KML_FFT_MPI_SCRAMBLED_[IN|OUT]：表示输入/输出按照scrambled格式，仅对一维变换生效 KML_FFT_MPI_TRANSPOSED_[IN|OUT]：表示输入/输出由$n_0 \times n_1 \times n_2 \times \cdots \times n_{d-1}$转化为$n_1 \times n_0 \times n_2 \times \cdots \times n_{d-1}$	输入

依赖

C: "kfft-mpi.h"

示例

C interface：

```
const ptrdiff_t N0 = 4, N1 = 4;
kml_fft_plan plan;
```

 版权所有 © 华为技术有限公司

```
ptrdiff_t alloc_local, local_n0, local_0_start;

MPI_Init(&argc, &argv);
MPI_Comm comm = MPI_COMM_WORLD;
kml_fft_mpi_init();
kml_fft_complex *in = NULL;
double *out = NULL;
ptrdiff_t rank = 2;
ptrdiff_t *n = (ptrdiff_t *)kml_fft_malloc(sizeof(ptrdiff_t) * rank);

n[0] = N0;
n[1] = N1;

/* get local data size and allocate */
alloc_local = kml_fft_mpi_local_size(rank, n, comm, &local_n0, &local_0_start);
if (alloc_local == -1) {
    printf("[%s][%d] allocate size fail.\n", __func__, __LINE__);
}
in = (kml_fft_complex *)kml_fft_malloc(sizeof(kml_fft_complex) * alloc_local);
if (in == NULL) {
    printf("[%s][%d] malloc memory fail.\n", __func__, __LINE__);
}
out = (double *)kml_fft_malloc(sizeof(double) * alloc_local * 2);
if (out == NULL) {
    printf("[%s][%d] malloc memory fail.\n", __func__, __LINE__);
}

/* create plan for in-place forward DFT */
plan = kml_fft_mpi_plan_dft_c2r(rank, n, in, out, comm, KML_FFT_ESTIMATE);

/* compute transforms, in-place, as many times as desired */
kml_fft_execute_dft_c2r(plan, in, out);

kml_fft_destroy_plan(plan);
kml_fft_mpi_cleanup();
MPI_Finalize();
```

6.4.4.5.3.2 kml_fft(f/h)_mpi_plan_dft_c2r_2d

建立单个连续数据序列2维C2R变换的plan。

接口定义

C interface：

kml_fft_plan kml_fft_mpi_plan_dft_c2r_2d(int n0, int n1, kml_fft_complex *in, double *out, MPI_Comm comm, unsigned flags);

kml_fftf_plan kml_fftf_mpi_plan_dft_c2r_2d(int n0, int n1, kml_fftf_complex *in, float *out, MPI_Comm comm, unsigned flags);

kml_ffth_plan kml_ffth_mpi_plan_dft_c2r_2d(int n0, int n1, kml_ffth_complex *in, __fp16 *out, MPI_Comm comm, unsigned flags);

返回值

函数返回一个kml_fft(f)_plan类型的结构体指针。将该对象作为参数传入 **kml_fft(f/h)_execute**函数中，将对当前提供的输入in和输出out执行FFT变换。也可以通过将该对象作为参数传入 **kml_fft(f/h)_execute_dft_c2r**函数中以对新的输入in和输出out执行FFT变换。

如果函数返回非空指针，则表示plan创建成功，否则表示创建失败。

 版权所有 © 华为技术有限公司

参数

参数名	数据类型	描述	输入/输出
n0	int	输入FFT序列第一维的大小，约束：n0 ≥ 1。	输入
n1	int	输入FFT序列第二维的大小，约束：n1 ≥ 1。	输入
in	• 双精度：kml_fft_complex* • 单精度：kml_fftf_complex* • 半精度：kml_ffth_complex*	输入需要变换的数据。	输入
out	• 双精度：double* • 单精度：float* • 半精度：__fp16*	输出快速傅里叶变换后的数据。	输出
comm	MPI_Comm	MPI通信器句柄	输入
flags	unsigned int	planning选项，可设置为如下选项： KML_FFT_ESTIMATE：ESTIMATE模式 KML_FFT_PATIENT：PATIENT模式 KML_FFT_MPI_SCRAMBLED_[IN|OUT]：表示输入/输出按照scrambled格式，仅对一维变换生效 KML_FFT_MPI_TRANSPOSED_[IN|OUT]：表示输入/输出由$n_0 \times n_1 \times n_2 \times \cdots \times n_{d-1}$转化为$n_1 \times n_0 \times n_2 \times \cdots \times n_{d-1}$	输入

依赖

C: "kfft-mpi.h"

示例

C interface：

```
const ptrdiff_t N0 = 4, N1 = 4;
kml_fft_plan plan;
```

 版权所有 © 华为技术有限公司

```
ptrdiff_t alloc_local, local_n0, local_0_start;

MPI_Init(&argc, &argv);
MPI_Comm comm = MPI_COMM_WORLD;
kml_fft_mpi_init();
kml_fft_complex *in = NULL;
double *out = NULL;

/* get local data size and allocate */
alloc_local = kml_fft_mpi_local_size_2d(N0, N1, comm, &local_n0, &local_0_start);
if (alloc_local == -1) {
    printf("[%s][%d] allocate size fail.\n", __func__, __LINE__);
}
in = (kml_fft_complex *)kml_fft_malloc(sizeof(kml_fft_complex) * alloc_local);
if (in == NULL) {
    printf("[%s][%d] malloc memory fail.\n", __func__, __LINE__);
}
out = (double *)kml_fft_malloc(sizeof(double) * alloc_local * 2);
if (out == NULL) {
    printf("[%s][%d] malloc memory fail.\n", __func__, __LINE__);
}

/* create plan for in-place forward DFT */
plan = kml_fft_mpi_plan_dft_c2r_2d(N0, N1, in, out, comm, KML_FFT_MPI_TRANSPOSED_IN);

/* compute transforms, in-place, as many times as desired */
kml_fft_execute_dft_c2r(plan, in, out);

kml_fft_destroy_plan(plan);
kml_fft_mpi_cleanup();

MPI_Finalize();
```

6.4.4.5.3.3 kml_fft(f/h)_mpi_plan_dft_c2r_3d

建立单个连续数据序列3维C2R变换的plan。

接口定义

C interface：

kml_fft_plan kml_fft_mpi_plan_dft_c2r_3d(int n0, int n1, int n2, kml_fft_complex *in, double *out, MPI_Comm comm, unsigned flags);

kml_fftf_plan kml_fftf_mpi_plan_dft_c2r_3d(int n0, int n1, int n2, kml_fftf_complex *in, float *out, MPI_Comm comm, unsigned flags);

kml_ffth_plan kml_ffth_mpi_plan_dft_c2r_3d(int n0, int n1, int n2, kml_ffth_complex *in, __fp16 *out, MPI_Comm comm, unsigned flags);

返回值

函数返回一个kml_fft(f)_plan类型的结构体指针。将该对象作为参数传入 **kml_fft(f/h)_execute**函数中，将对当前提供的输入in和输出out执行FFT变换。也可以通过将该对象作为参数传入 **kml_fft(f/h)_execute_dft_c2r**函数中以对新的输入in和输出out执行FFT变换。

如果函数返回非空指针，则表示plan创建成功，否则表示创建失败。

 版权所有 © 华为技术有限公司

参数

参数名	数据类型	描述	输入/输出
n0	int	输入FFT序列第一维的大小，约束：n0 ≥ 1。	输入
n1	int	输入FFT序列第二维的大小，约束：n1 ≥ 1。	输入
n2	int	输入FFT序列第三维的大小，约束：n2 ≥ 1。	输入
in	• 双精度：kml_fft_complex* • 单精度：kml_fftf_complex* • 半精度：kml_ffth_complex*	输入需要变换的数据。	输入
out	• 双精度：double* • 单精度：float* • 半精度：__fp16*	输出快速傅里叶变换后的数据。	输出
comm	MPI_Comm	MPI通信器句柄	输入
flags	unsigned int	planning选项，可设置为如下选项： KML_FFT_ESTIMATE：ESTIMATE模式 KML_FFT_PATIENT：PATIENT模式 KML_FFT_MPI_SCRAMBLED_[IN|OUT]：表示输入/输出按照scrambled格式，仅对一维变换生效 KML_FFT_MPI_TRANSPOSED_[IN|OUT]：表示输入/输出由$n_0 \times n_1 \times n_2 \times \cdots \times n_{d-1}$转化为$n_1 \times n_0 \times n_2 \times \cdots \times n_{d-1}$	输入

依赖

C: "kfft-mpi.h"

示例

C interface：

 版权所有 © 华为技术有限公司

```
const ptrdiff_t N0 = 4, N1 = 4, N2 = 4;
kml_fft_plan plan;
ptrdiff_t alloc_local, local_n0, local_0_start;

MPI_Init(&argc, &argv);
MPI_Comm comm = MPI_COMM_WORLD;
kml_fft_mpi_init();
kml_fft_complex *in = NULL;
double *out = NULL;

/* get local data size and allocate */
alloc_local = kml_fft_mpi_local_size_3d(N0, N1, N2, comm, &local_n0, &local_0_start);
if (alloc_local == -1) {
    printf("[%s][%d] allocate size fail.\n", __func__, __LINE__);
}
in = (kml_fft_complex *)kml_fft_malloc(sizeof(kml_fft_complex) * alloc_local);
if (in == NULL) {
    printf("[%s][%d] malloc memory fail.\n", __func__, __LINE__);
}
out = (double *)kml_fft_malloc(sizeof(double) * alloc_local * 2);
if (out == NULL) {
    printf("[%s][%d] malloc memory fail.\n", __func__, __LINE__);
}

/* create plan for in-place forward DFT */
plan = kml_fft_mpi_plan_dft_c2r_3d(N0, N1, N2, in, out, comm, KML_FFT_ESTIMATE);

/* compute transforms, in-place, as many times as desired */
kml_fft_execute_dft_c2r(plan, in, out);

kml_fft_destroy_plan(plan);
kml_fft_mpi_cleanup();

MPI_Finalize();
```

6.4.4.5.3.4 kml_fft(f/h)_mpi_plan_many_dft_c2r

创建howmany组数据序列n维C2R变换的plan。其中单个FFT的数据序列不需要是连续的，可以以跨步的形式提供。

接口定义

C interface：

kml_fft_plan kml_fft_mpi_plan_many_dft_c2r(int rank, const int *n, int howmany, ptrdiff_t iblock, ptrdiff_t oblock, kml_fft_complex *in, double *out, MPI_Comm comm, unsigned flags);

kml_fftf_plan kml_fftf_mpi_plan_many_dft_c2r(int rank, const int *n, int howmany, ptrdiff_t iblock, ptrdiff_t oblock, kml_fftf_complex *in, float *out, MPI_Comm comm, unsigned flags);

kml_ffth_plan kml_ffth_mpi_plan_many_dft_c2r(int rank, const int *n, int howmany, ptrdiff_t iblock, ptrdiff_t oblock, kml_ffth_complex *in, __fp16 *out, MPI_Comm comm, unsigned flags);

返回值

函数返回一个kml_fft(f)_plan类型的结构体指针。将该对象作为参数传入 **kml_fft(f/h)_execute**函数中，将对当前提供的输入in和输出out执行FFT变换。也可以通过将该对象作为参数传入 **kml_fft(f/h)_execute_dft_c2r**函数中以对新的输入in和输出out执行FFT变换。

 版权所有 © 华为技术有限公司

如果函数返回非空指针，则表示plan创建成功，否则表示创建失败。

参数

参数名	数据类型	描述	输入/输出
rank	int	FFT变换的维度是rank，约束：1 ≤ rank ≤ 3。	输入
n	const int*	n为维度为rank的数组，包含FFT序列每一维度的大小，约束：n[i] ≥ 1, for i in 0 to rank - 1。	输入
howmany	int	howmany表示指定多少个多维FFT变换。	输入
iblock	ptrdiff_t	n0维度上的block大小	输入
oblock	ptrdiff_t	n1维度上的block大小	输入
in	• 双精度：kml_fft_complex* • 单精度：kml_fftf_complex* • 半精度：kml_ffth_complex*	输入需要变换的数据。	输入
out	• 双精度：double* • 单精度：float* • 半精度：__fp16*	输出快速傅里叶变换后的数据。	输出
comm	MPI_Comm	MPI通信器句柄	输入
flags	unsigned int	planning选项，可设置为如下选项： KML_FFT_ESTIMATE：ESTIMATE模式 KML_FFT_PATIENT：PATIENT模式 KML_FFT_MPI_SCRAMBLED_[IN|OUT]：表示输入/输出按照scrambled格式，仅对一维变换生效 KML_FFT_MPI_TRANSPOSED_[IN|OUT]：表示输入/输出由$n_0 \times n_1 \times n_2 \times \cdots \times n_{d-1}$转化为$n_1 \times n_0 \times n_2 \times \cdots \times n_{d-1}$	输入

版权所有 © 华为技术有限公司

依赖

C: "kfft-mpi.h"

示例

C interface：

```
const ptrdiff_t N0 = 4, N1 = 4;
kml_fft_plan plan;
ptrdiff_t alloc_local, local_n0, local_0_start;

MPI_Init(&argc, &argv);
MPI_Comm comm = MPI_COMM_WORLD;
kml_fft_mpi_init();
ptrdiff_t block0 = 0;
ptrdiff_t block1 = 0;
ptrdiff_t howmany = 1;
kml_fft_complex *in = NULL;
double *out = NULL;
ptrdiff_t rank = 2;
ptrdiff_t *n = (ptrdiff_t *)kml_fft_malloc(sizeof(ptrdiff_t) * rank);

n[0] = N0;
n[1] = N1;

/* get local data size and allocate */
alloc_local = kml_fft_mpi_local_size_many(rank, n, howmany, block0, comm, &local_n0, &local_0_start);
if (alloc_local == -1) {
    printf("[%s][%d] allocate size fail.\n", __func__, __LINE__);
}
in = (kml_fft_complex *)kml_fft_malloc(sizeof(kml_fft_complex) * alloc_local);
if (in == NULL) {
    printf("[%s][%d] malloc memory fail.\n", __func__, __LINE__);
}
out = (double *)kml_fft_malloc(sizeof(double) * alloc_local * 2);
if (out == NULL) {
    printf("[%s][%d] malloc memory fail.\n", __func__, __LINE__);
}

/* create plan for in-place forward DFT */
plan = kml_fft_mpi_plan_many_dft_c2r(rank, n, howmany, block0, block1, in, out, comm,
KML_FFT_ESTIMATE);

/* compute transforms, in-place, as many times as desired */
kml_fft_execute(plan);

kml_fft_destroy_plan(plan);
kml_fft_mpi_cleanup();

MPI_Finalize();
```

6.4.4.5.3.5 kml_fft(f)_mpi_plan_dft_c2r_3d_ext

建立单个连续数据序列3维C2R变换的plan。

接口定义

C interface：

kml_fft_plan kml_fft_mpi_plan_dft_c2r_3d_ext(int n0, int n1, int n2, const int *inbox_order, const int *outbox_order, MPI_Comm comm, const kml_fft_mpi_options options);

 版权所有 © 华为技术有限公司

kml_fftf_plan kml_fftf_mpi_plan_dft_c2r_3d_ext(int n0, int n1, int n2, const int *inbox_order, const int *outbox_order, MPI_Comm comm, const kml_fftf_mpi_options options);

返回值

函数返回一个kml_fft(f)_plan类型的结构体指针。将该对象作为参数传入**kml_fft(f)_mpi_execute_dft_c2r_ext函数**、**kml_fft(f)_mpi_backward _c2r函数**中使用，且不需要在plan创建阶段指定in、out buffer。

如果函数返回非空指针，则表示plan创建成功，否则表示创建失败。

参数

参数名	数据类型	描述	输入/输出
n0	int	输入FFT序列第一维的大小，约束：n0≥1。	输入
n1	int	输入FFT序列第二维的大小，约束：n1≥1。	输入
n2	int	输入FFT序列第三维的大小，约束：n2≥1。	输入
inbox_order	const int *	是长度为3的一维数组，输入数据维度的顺序，取值为0、1、2的任意顺序组合，输入NULL，默认order为0、1、2。	输入
outbox_order	const int *	是长度为3的一维数组，输出数据维度的顺序，取值为0、1、2的任意顺序组合，输入NULL，默认order为0、1、2。	输入
comm	MPI_Comm	通信域。	输入
options	• 双精度：const kml_fft_mpi_options • 单精度： const kml_fftf_mpi_options	3DFFT分解算法、通信算法flag。	输入

依赖

C: "kfft-mpi.h"

示例

C interface：

 版权所有 © 华为技术有限公司

```
    const int n0 = 4, n1 = 4, n2 = 4;
    kml_fft_plan plan;
    int provided;
    MPI_Init_thread(&argc, &argv, MPI_THREAD_MULTIPLE, &provided);
    MPI_Comm comm = MPI_COMM_WORLD;
    kml_fft_complex *in = NULL;
    double *out = NULL;

    /* get local data size and allocate */
    ptrdiff_t low[3] = {0};
    ptrdiff_t high[3] = {0};
    ptrdiff_t alloc_local = kml_fft_mpi_local_size_3d_ext(n0, n1, n2, comm,
SCALFFT_DECOMPOSE_TYPE_PENCIL, low, high);
    if (alloc_local == -1) {
        printf("[%s][%d] allocate size fail.\n", __func__, __LINE__);
    }
    in = (kml_fft_complex *)kml_fft_malloc(sizeof(kml_fft_complex) * alloc_local * 2);
    if (in == NULL) {
        printf("[%s][%d] malloc memory fail.\n", __func__, __LINE__);
    }
    out = (double *)kml_fft_malloc(sizeof(double) * alloc_local * 2);
    if (out == NULL) {
        printf("[%s][%d] malloc memory fail.\n", __func__, __LINE__);
    }

    /* create plan */
    kml_fft_mpi_options options = {
        .a2a_algo    = A2A_ALGO_AUTO_TUNING,
        .decomp_type = SCALFFT_DECOMPOSE_TYPE_PENCIL
    };
    plan = kml_fft_mpi_plan_dft_c2r_3d_ext(n0, n1, n2, NULL, NULL, comm, options);

    /* execute plan */
    int scale = 0;
    kml_fft_mpi_execute_dft_c2r_ext(plan, in, out, scale);

    kml_fft_destroy_plan_ext(plan);

    kml_fft_free(in);
    kml_fft_free(out);
    MPI_Finalize();
```

6.4.4.5.3.6 kml_fft(f)_mpi_plan_create_c2r

建立单个连续数据序列3维C2R变换的plan。

接口定义

C interface：

kml_fft_plan kml_fft_mpi_plan_create_c2r(int backend, const int *inbox_low, const int *inbox_high, const int *inbox_order, const int *outbox_low, const int *outbox_high, const int *outbox_order, MPI_Comm comm, const kml_fft_mpi_options options);

kml_fftf_plan kml_fftf_mpi_plan_create_c2r(int backend, const int *inbox_low, const int *inbox_high, const int *inbox_order, const int *outbox_low, const int *outbox_high, const int *outbox_order, MPI_Comm comm, const kml_fftf_mpi_options options);

 版权所有 © 华为技术有限公司

返回值

函数返回一个kml_fft(f)_plan类型的结构体指针。将该对象作为参数传入**kml_fft(f)_mpi_execute_dft_c2r_ext函数**、**kml_fft(f)_mpi_backward _c2r函数**中使用，且不需要在plan创建阶段指定in、out buffer。

如果函数返回非空指针，则表示plan创建成功，否则表示创建失败。

参数

参数名	数据类型	描述	输入/输出
backend	int	底层库类型，当前仅支持BACKEND_KFFT。	输入
inbox_low	const int *	是长度为3的一维数组，输入数据在三个维度的切分范围下限，-1≤inbox_low[i]≤n[i]-1 for i in 0、1、2。	输入
inbox_high	const int *	是长度为3的一维数组，输入数据在三个维度的切分范围上限，-1≤inbox_high[i]≤n[i]-1 for i in 0、1、 2，且0≤（inbox_high[i]-inbox_low[i]+1）≤n[i] for i in 0、1、2。	输入
inbox_order	const int *	是长度为3的一维数组，输入数据维度的顺序，取值为0、1、2的任意顺序组合，输入NULL，默认order为0、1、2。	输入
outbox_low	const int *	是长度为3的一维数组，输出数据在三个维度的切分范围下限，-1≤outbox_low[i]≤n[i]-1 for i in 0、1、2。	输入
outbox_high	const int *	是长度为3的一维数组，输出数据在三个维度的切分范围上限，-1≤outbox_high[i]≤n[i]-1 for i in 0、1、2，且 0≤(outbox_high[i] - outbox_low[i] + 1)≤n[i] for i in 0、1、2。	输入
outbox_order	const int *	是长度为3的一维数组，输出数据维度的顺序，取值为0、1、2的任意顺序组合，输入NULL，默认order为0、1、2。	输入
comm	MPI_Comm	通信域。	输入

版权所有 © 华为技术有限公司

参数名	数据类型	描述	输入/输出
options	• 双精度：const kml_fft_mpi_options • 单精度： const kml_fftf_mpi_options	3DFFT分解算法、通信算法flag。	输入

依赖

C: "kfft-mpi.h"

示例

C interface：

```
const int n0 = 4, n1 = 4, n2 = 4;
    kml_fft_plan plan;
    int provided;
    MPI_Init_thread(&argc, &argv, MPI_THREAD_MULTIPLE, &provided);
    MPI_Comm comm = MPI_COMM_WORLD;
    kml_fft_complex *in = NULL;
    double *out = NULL;

    /* get local data size and allocate */
    ptrdiff_t low[3] = {0};
    ptrdiff_t high[3] = {0};
    ptrdiff_t alloc_local = kml_fft_mpi_local_size_3d_ext(n0, n1, n2, comm,
SCALFFT_DECOMPOSE_TYPE_PENCIL, low, high);
    if (alloc_local == -1) {
        printf("[%s][%d] allocate size fail.\n", __func__, __LINE__);
    }
    in = (kml_fft_complex *)kml_fft_malloc(sizeof(kml_fft_complex) * alloc_local * 2);
    if (in == NULL) {
        printf("[%s][%d] malloc memory fail.\n", __func__, __LINE__);
    }
    out = (double *)kml_fft_malloc(sizeof(double) * alloc_local * 2);
    if (out == NULL) {
        printf("[%s][%d] malloc memory fail.\n", __func__, __LINE__);
    }

    /* create plan */
    int low_int[3] = {low[0], low[1], low[2]};
    int high_int[3] = {high[0], high[1], high[2]};
    kml_fft_mpi_options options = {
        .a2a_algo    = A2A_ALGO_AUTO_TUNING,
        .decomp_type = SCALFFT_DECOMPOSE_TYPE_PENCIL
    };
    plan = kml_fft_mpi_plan_create_c2r(BACKEND_KFFT, low_int, high_int, NULL, low_int, high_int, NULL,
comm, options);

    /* execute plan */
    int scale = 0;
    kml_fft_mpi_execute_dft_c2r_ext(plan, in, out, scale);
    // kml_fft_mpi_backward_c2r(plan, in, out, scale);

    kml_fft_destroy_plan_ext(plan);

    kml_fft_free(in);
    kml_fft_free(out);
```

 版权所有 © 华为技术有限公司

```
MPI_Finalize();
```

6.4.4.5.4 R2R 变换

6.4.4.5.4.1 kml_fft(f/h)_mpi_plan_r2r

建立单个连续数据序列n维R2R变换的plan。

接口定义

C interface：

kml_fft_plan kml_fft_mpi_plan_r2r(int rank, const ptrdiff_t *n, double *in, double *out, const kml_fft_r2r_kind *kind, MPI_Comm comm, unsigned flags);

kml_fftf_plan kml_fftf_mpi_plan_r2r(int rank, const ptrdiff_t *n, float *in, float *out, const kml_fftf_r2r_kind *kind, MPI_Comm comm, unsigned flags);

kml_ffth_plan kml_ffth_mpi_plan_r2r(int rank, const ptrdiff_t *n, __fp16 *in, __fp16 *out, const kml_ffth_r2r_kind *kind, MPI_Comm comm, unsigned flags);

返回值

函数返回一个kml_fft(f)_plan类型的结构体指针。将该对象作为参数传入**kml_fft(f)_execute函数**中调用，将对当前提供的输入in和输出out执行FFT变换。也可以通过将该对象作为参数传入**kml_fft(f)_execute_r2r函数**中以对新的输入in和输出out执行FFT变换。

如果函数返回非空指针，则表示plan创建成功，否则表示创建失败。

参数

参数名	数据类型	描述	输入/输出
rank	int	FFT变换的维度是rank，约束：1 ≤ rank ≤ 3。	输入
n	const ptrdiff_t *	n为维度为rank的数组，包含FFT序列每一维度的大小，约束：n[i] ≥ 1, for i in 0 to rank - 1。	输入
in	• 双精度：double* • 单精度：float* • 半精度：__fp16*	输入需要变换的数据。	输入
out	• 双精度：double* • 单精度：float* • 半精度：__fp16*	输出快速傅里叶变换后的数据。	输出

版权所有 © 华为技术有限公司

参数名	数据类型	描述	输入/输出
kind	• 双精度：const kml_fft_r2r_kind * • 单精度：const kml_fftf_r2r_kind* • 半精度：const kml_ffth_r2r_kind*	kind是大小为rank的数组，包含FFT序列每一维度的R2R变换类型，kind[i] (for i in 0 to rank - 1)有以下可选值： • KML_FFT_R2HC • KML_FFT_HC2R • KML_FFT_DHT • KML_FFT_REDFT00 • KML_FFT_REDFT01 • KML_FFT_REDFT10 • KML_FFT_REDFT11 • KML_FFT_RODFT00 • KML_FFT_RODFT01 • KML_FFT_RODFT10 • KML_FFT_RODFT11	输入
comm	MPI_Comm	MPI句柄通信器。	输入
flags	unsigned int	planning选项，可设置为如下选项： KML_FFT_ESTIMATE：ESTIMATE模式 KML_FFT_PATIENT：PATIENT模式 KML_FFT_MPI_SCRAMBLED_[IN|OUT]：表示输入/输出按照scrambled格式，仅对一维变换生效 KML_FFT_MPI_TRANSPOSED_[IN|OUT]：表示输入/输出由$n_0 \times n_1 \times n_2 \times \cdots \times n_{d-1}$转化为$n_1 \times n_0 \times n_2 \times \cdots \times n_{d-1}$	输入

依赖

C: "kfft-mpi.h"

示例

C interface：

```
const ptrdiff_t N0 = 4, N1 = 4;
kml_fft_plan plan;
ptrdiff_t alloc_local, local_n0, local_0_start;

MPI_Init(&argc, &argv);
MPI_Comm comm = MPI_COMM_WORLD;
```

 版权所有 © 华为技术有限公司

```
kml_fft_mpi_init();
double *in = NULL;
double *out = NULL;
ptrdiff_t rank = 2;
kml_fft_r2r_kind kind0 = KML_FFT_R2HC;
kml_fft_r2r_kind kind1 = KML_FFT_R2HC;
ptrdiff_t *n = (ptrdiff_t *)kml_fft_malloc(sizeof(ptrdiff_t) * rank);
kml_fft_r2r_kind *kind = (kml_fft_r2r_kind *)kml_fft_malloc(sizeof(kml_fft_r2r_kind) * rank);

n[0] = N0;
n[1] = N1;
kind[0] = kind0;
kind[1] = kind1;

/* get local data size and allocate */
alloc_local = kml_fft_mpi_local_size(rank, n, comm, &local_n0, &local_0_start);
if (alloc_local == -1) {
    printf("[%s][%d] allocate size fail.\n", __func__, __LINE__);
}
in = (double *)kml_fft_malloc(sizeof(double) * alloc_local);
if (in == NULL) {
    printf("[%s][%d] malloc memory fail.\n", __func__, __LINE__);
}
out = (double *)kml_fft_malloc(sizeof(double) * alloc_local);
if (out == NULL) {
    printf("[%s][%d] malloc memory fail.\n", __func__, __LINE__);
}

/* create plan for in-place forward DFT */
plan = kml_fft_mpi_plan_r2r(rank, n, in, out, kind, comm, KML_FFT_ESTIMATE);

/* compute transforms, in-place, as many times as desired */
kml_fft_execute_r2r(plan, in, out);

kml_fft_destroy_plan(plan);
kml_fft_mpi_cleanup();
MPI_Finalize();
```

6.4.4.5.4.2 kml_fft(f/h)_ mpi_plan_r2r_2d

建立单个连续数据序列2维R2R变换的plan。

接口定义

C interface：

kml_fft_plan kml_fft_mpi_plan_r2r_2d(ptrdiff_t n0, ptrdiff_t n1, double *in, double *out, kml_fft_r2r_kind kind0, kml_fft_r2r_kind kind1, MPI_Comm comm, unsigned flags);

kml_fftf_plan kml_fftf_mpi_plan_r2r_2d(ptrdiff_t n0, ptrdiff_t n1, float *in, float *out, kml_fftf_r2r_kind kind0, kml_fftf_r2r_kind kind1, MPI_Comm comm, unsigned flags);

kml_ffth_plan kml_ffth_mpi_plan_r2r_2d(ptrdiff_t n0, ptrdiff_t n1, __fp16 *in, __fp16 *out, kml_ffth_r2r_kind kind0, kml_ffth_r2r_kind kind1, MPI_Comm comm, unsigned flags);

返回值

函数返回一个kml_fft(f)_plan类型的结构体指针。将该对象作为参数传入**kml_fft(f)_execute函数**中使用，将对当前提供的输入in和输出out执行FFT变换。也可以通过将该对象作为参数传入**kml_fft(f)_execute_r2r函数**中以对新的输入in和输出out执行FFT变换。

 版权所有 © 华为技术有限公司

如果函数返回非空指针，则表示plan创建成功，否则表示创建失败。

参数

参数名	数据类型	描述	输入/输出
n0	ptrdiff_t	输入FFT序列第一维的大小，约束：n0 ≥ 1。	输入
n1	ptrdiff_t	输入FFT序列第二维的大小，约束：n1 ≥ 1。	输入
in	• 双精度：double* • 单精度：float* • 半精度：__fp16*	输入需要变换的数据。	输入
out	• 双精度：double* • 单精度：float* • 半精度：__fp16*	输出快速傅里叶变换后的数据。	输出
kind0	• 双精度：kml_fft_r2r_kind* • 单精度：kml_fftf_r2r_kind* • 半精度：kml_ffth_r2r_kind*	kind0表示FFT序列第一维的R2R变换类型，有以下可选值： • KML_FFT_R2HC • KML_FFT_HC2R • KML_FFT_DHT • KML_FFT_REDFT00 • KML_FFT_REDFT01 • KML_FFT_REDFT10 • KML_FFT_REDFT11 • KML_FFT_RODFT00 • KML_FFT_RODFT01 • KML_FFT_RODFT10 • KML_FFT_RODFT11	输入

版权所有 © 华为技术有限公司

参数名	数据类型	描述	输入/输出
kind1	• 双精度：kml_fft_r2r_kind* • 单精度：kml_fftf_r2r_kind*	kind1表示FFT序列第二维的R2R变换类型，有以下可选值： • KML_FFT_R2HC • KML_FFT_HC2R • KML_FFT_DHT • KML_FFT_REDFT00 • KML_FFT_REDFT01 • KML_FFT_REDFT10 • KML_FFT_REDFT11 • KML_FFT_RODFT00 • KML_FFT_RODFT01 • KML_FFT_RODFT10 • KML_FFT_RODFT11	输入
comm	MPI_Comm	MPI句柄通信器。	输入
flags	unsigned int	planning选项，可设置为如下选项： KML_FFT_ESTIMATE：ESTIMATE模式 KML_FFT_PATIENT：PATIENT模式 KML_FFT_MPI_SCRAMBLED_[IN|OUT]：表示输入/输出按照scrambled格式，仅对一维变换生效 KML_FFT_MPI_TRANSPOSED_[IN|OUT]：表示输入/输出由$n_0 \times n_1 \times n_2 \times \cdots \times n_{d-1}$转化为$n_1 \times n_0 \times n_2 \times \cdots \times n_{d-1}$	输入

依赖

C: "kfft-mpi.h"

示例

C interface：

```
const ptrdiff_t N0 = 4, N1 = 4;
kml_fft_plan plan;
ptrdiff_t alloc_local, local_n0, local_0_start;

MPI_Init(&argc, &argv);
MPI_Comm comm = MPI_COMM_WORLD;
kml_fft_mpi_init();
kml_fft_r2r_kind kind0 = KML_FFT_R2HC;
kml_fft_r2r_kind kind1 = KML_FFT_R2HC;
double *in = NULL;
```

 版权所有 © 华为技术有限公司

```
double *out = NULL;

/* get local data size and allocate */
alloc_local = kml_fft_mpi_local_size_2d(N0, N1, comm, &local_n0, &local_0_start);
if (alloc_local == -1) {
    printf("[%s][%d] allocate size fail.\n", __func__, __LINE__);
}
in = (double *)kml_fft_malloc(sizeof(double) * alloc_local);
if (in == NULL) {
    printf("[%s][%d] malloc memory fail.\n", __func__, __LINE__);
}
out = (double *)kml_fft_malloc(sizeof(double) * alloc_local);
if (out == NULL) {
    printf("[%s][%d] malloc memory fail.\n", __func__, __LINE__);
}

/* create plan for in-place forward DFT */
plan = kml_fft_mpi_plan_r2r_2d(N0, N1, in, out, comm, kind0, kind1, KML_FFT_MPI_TRANSPOSED_IN);

/* compute transforms, in-place, as many times as desired */
kml_fft_execute_r2r(plan, in, out);

kml_fft_destroy_plan(plan);
kml_fft_mpi_cleanup();

MPI_Finalize();
```

6.4.4.5.4.3 kml_fft(f/h)_mpi_plan_r2r_3d

建立单个连续数据序列3维R2R变换的plan。

接口定义

C interface：

kml_fft_plan kml_fft_mpi_plan_r2r_3d(ptrdiff_t n0, ptrdiff_t n1, ptrdiff_t n2, double *in, double *out, kml_fft_r2r_kind kind0, kml_fft_r2r_kind kind1, kml_fft_r2r_kind kind2, MPI_Comm comm, unsigned flags);

kml_fftf_plan kml_fftf_mpi_plan_r2r_3d(ptrdiff_t n0, ptrdiff_t n1, ptrdiff_t n2, float *in, float *out, kml_fftf_r2r_kind kind0, kml_fftf_r2r_kind kind1, kml_fftf_r2r_kind kind2, MPI_Comm comm, unsigned flags);

kml_ffth_plan kml_ffth_mpi_plan_r2r_3d(ptrdiff_t n0, ptrdiff_t n1, ptrdiff_t n2, __fp16 *in, __fp16 *out, kml_ffth_r2r_kind kind0, kml_ffth_r2r_kind kind1, kml_ffth_r2r_kind kind2, MPI_Comm comm, unsigned flags);

返回值

函数返回一个kml_fft(f)_plan类型的结构体指针。将该对象作为参数传入**kml_fft(f)_execute函数**中使用，将对当前提供的输入in和输出out执行FFT变换。也可以通过将该对象作为参数传入**kml_fft(f)_execute_r2r函数**中以对新的输入in和输出out执行FFT变换。

如果函数返回非空指针，则表示plan创建成功，否则表示创建失败。

版权所有 © 华为技术有限公司

参数

参数名	数据类型	描述	输入/输出
n0	ptrdiff_t	输入FFT序列第一维的大小，约束：n0 ≥ 1。	输入
n1	ptrdiff_t	输入FFT序列第二维的大小，约束：n1 ≥ 1。	输入
n2	ptrdiff_t	输入FFT序列第三维的大小，约束：n2 ≥ 1。	输入
in	• 双精度：double* • 单精度：float* • 半精度：__fp16*	输入需要变换的数据。	输入
out	• 双精度：double* • 单精度：float* • 半精度：__fp16*	输出快速傅里叶变换后的数据。	输出
kind0	• 双精度：kml_fft_r2r_kind* • 单精度：kml_fftf_r2r_kind* • 半精度：kml_ffth_r2r_kind*	kind0表示FFT序列第一维的R2R变换类型，有以下可选值： • KML_FFT_R2HC • KML_FFT_HC2R • KML_FFT_DHT • KML_FFT_REDFT00 • KML_FFT_REDFT01 • KML_FFT_REDFT10 • KML_FFT_REDFT11 • KML_FFT_RODFT00 • KML_FFT_RODFT01 • KML_FFT_RODFT10 • KML_FFT_RODFT11	输入

版权所有 © 华为技术有限公司

参数名	数据类型	描述	输入/输出
kind1	• 双精度：kml_fft_r2r_kind* • 单精度：kml_fftf_r2r_kind* • 半精度：kml_ffth_r2r_kind*	kind1表示FFT序列第二维的R2R变换类型，有以下可选值： • KML_FFT_R2HC • KML_FFT_HC2R • KML_FFT_DHT • KML_FFT_REDFT00 • KML_FFT_REDFT01 • KML_FFT_REDFT10 • KML_FFT_REDFT11 • KML_FFT_RODFT00 • KML_FFT_RODFT01 • KML_FFT_RODFT10 • KML_FFT_RODFT11	输入
kind2	• 双精度：kml_fft_r2r_kind* • 单精度：kml_fftf_r2r_kind* • 半精度：kml_ffth_r2r_kind*	kind2表示FFT序列第三维的R2R变换类型，有以下可选值： • KML_FFT_R2HC • KML_FFT_HC2R • KML_FFT_DHT • KML_FFT_REDFT00 • KML_FFT_REDFT01 • KML_FFT_REDFT10 • KML_FFT_REDFT11 • KML_FFT_RODFT00 • KML_FFT_RODFT01 • KML_FFT_RODFT10 • KML_FFT_RODFT11	输入
comm	MPI_Comm	MPI句柄通信器	输入

版权所有 © 华为技术有限公司

参数名	数据类型	描述	输入/输出
flags	unsigned int	planning选项，可设置为如下选项： KML_FFT_ESTIMATE：ESTIMATE模式 KML_FFT_PATIENT：PATIENT模式 KML_FFT_MPI_SCRAMBLED_[IN|OUT]：表示输入/输出按照scrambled格式，仅对一维变换生效 KML_FFT_MPI_TRANSPOSED_[IN|OUT]：表示输入/输出由$n_0 \times n_1 \times n_2 \times \cdots \times n_{d-1}$转化为$n_1 \times n_0 \times n_2 \times \cdots \times n_{d-1}$	输入

依赖

C: "kfft-mpi.h"

示例

C interface：

```
const ptrdiff_t N0 = 4, N1 = 4, N2 = 4;
kml_fft_plan plan;
ptrdiff_t alloc_local, local_n0, local_0_start;

MPI_Init(&argc, &argv);
MPI_Comm comm = MPI_COMM_WORLD;
kml_fft_mpi_init();
kml_fft_r2r_kind kind0 = KML_FFT_R2HC;
kml_fft_r2r_kind kind1 = KML_FFT_R2HC;
kml_fft_r2r_kind kind2 = KML_FFT_R2HC;
double *in = NULL;
double *out = NULL;

/* get local data size and allocate */
alloc_local = kml_fft_mpi_local_size_3d(N0, N1, N2, comm, &local_n0, &local_0_start);
if (alloc_local == -1) {
    printf("[%s][%d] allocate size fail.\n", __func__, __LINE__);
}
in = (double *)kml_fft_malloc(sizeof(double) * alloc_local);
if (in == NULL) {
    printf("[%s][%d] malloc memory fail.\n", __func__, __LINE__);
}
out = (double *)kml_fft_malloc(sizeof(double) * alloc_local);
if (out == NULL) {
    printf("[%s][%d] malloc memory fail.\n", __func__, __LINE__);
}

/* create plan for in-place forward DFT */
plan = kml_fft_mpi_plan_r2r_3d(N0, N1, N2, in, out, kind0, kind1, kind2, comm,
KML_FFT_MPI_TRANSPOSED_IN);

/* compute transforms, in-place, as many times as desired */
kml_fft_execute(plan);

kml_fft_destroy_plan(plan);
```

 版权所有 © 华为技术有限公司

```
kml_fft_mpi_cleanup();

MPI_Finalize();
```

6.4.4.5.4.4 kml_fft(f/h)_mpi_plan_many_r2r

建立howmany组数据序列n维R2R变换的plan。其中，单个FFT的数据序列不需要是连续的，可以以跨步的形式提供。

接口定义

C interface：

kml_fft_plan kml_fft_mpi_plan_many_r2r(int rank, const ptrdiff_t *n, ptrdiff_t howmany, ptrdiff_t iblock, ptrdiff_t oblock, double *in, double *out, const kml_fft_r2r_kind *kind, MPI_Comm comm, unsigned flags);

kml_fftf_plan kml_fftf_mpi_plan_many_r2r(int rank, const ptrdiff_t *n, ptrdiff_t howmany, ptrdiff_t iblock, ptrdiff_t oblock, float *in, float *out, const kml_fftf_r2r_kind *kind, MPI_Comm comm, unsigned flags);

kml_ffth_plan kml_ffth_mpi_plan_many_r2r(int rank, const ptrdiff_t *n, ptrdiff_t howmany, ptrdiff_t iblock, ptrdiff_t oblock, __fp16 *in, __fp16 *out, const kml_fftf_r2r_kind *kind, MPI_Comm comm, unsigned flags);

返回值

函数返回一个kml_fftf_plan类型的结构体指针。将该对象作为参数传入**kml_fft(f)_execute**函数中，将对当前提供的输入in和输出out执行FFT变换。也可以通过将该对象作为参数传入**kml_fft(f)_execute_r2r**函数中以对新的输入in和输出out执行FFT变换。

如果函数返回非空指针，则表示plan创建成功，否则表示创建失败。

参数

参数名	数据类型	描述	输入/输出
rank	int	FFT变换的维度是rank，约束：1 ≤ rank ≤ 3。	输入
n	const int*	n为维度为rank的数组，包含FFT序列每一维度的大小，约束：n[i] ≥ 1, for i in 0 to rank - 1。	输入
howmany	int	howmany表示指定多少个多维FFT变换。	输入
iblock	ptrdiff_t	n0维度上的block大小。	输入
oblock	ptrdiff_t	n1维度上的block大小。	输入

版权所有 © 华为技术有限公司

参数名	数据类型	描述	输入/输出
in	• 双精度：double* • 单精度：float* • 半精度：__fp16*	输入需要变换的数据。	输入
out	• 双精度：double* • 单精度：float* • 半精度：__fp16*	输出快速傅里叶变换后的数据。	输出
kind	• 双精度：const kml_fft_r2r_kind* • 单精度：const kml_fftf_r2r_kind* • 半精度：const kml_ffth_r2r_kind*	kind是大小为rank的数组，包含FFT序列每一维度的R2R变换类型，kind[i] (for i in 0 to rank - 1)有以下可选值： • KML_FFT_R2HC • KML_FFT_HC2R • KML_FFT_DHT • KML_FFT_REDFT00 • KML_FFT_REDFT01 • KML_FFT_REDFT10 • KML_FFT_REDFT11 • KML_FFT_RODFT00 • KML_FFT_RODFT01 • KML_FFT_RODFT10 • KML_FFT_RODFT11	输入
comm	MPI_Comm	MPI通信器句柄	输入
flags	unsigned int	planning选项，可设置为如下选项： KML_FFT_ESTIMATE：ESTIMATE模式 KML_FFT_PATIENT：PATIENT模式 KML_FFT_MPI_SCRAMBLED_[IN|OUT]：表示输入/输出按照scrambled格式，仅对一维变换生效 KML_FFT_MPI_TRANSPOSED_[IN|OUT]：表示输入/输出由$n_0 \times n_1 \times n_2 \times \cdots \times n_{d-1}$转化为$n_1 \times n_0 \times n_2 \times \cdots \times n_{d-1}$	输入

版权所有 © 华为技术有限公司

依赖

C: "kfft-mpi.h"

示例

C interface：

```
const ptrdiff_t N0 = 4, N1 = 4;
kml_fft_plan plan;
ptrdiff_t alloc_local, local_n0, local_0_start;

MPI_Init(&argc, &argv);
MPI_Comm comm = MPI_COMM_WORLD;
kml_fft_mpi_init();
kml_fft_r2r_kind kind0 = KML_FFT_R2HC;
kml_fft_r2r_kind kind1 = KML_FFT_R2HC;
ptrdiff_t block0 = 0;
ptrdiff_t block1 = 0;
ptrdiff_t howmany = 1;
double *in = NULL;
double *out = NULL;
ptrdiff_t rank = 2;
ptrdiff_t *n = (ptrdiff_t *)kml_fft_malloc(sizeof(ptrdiff_t) * rank);
kml_fft_r2r_kind *kind = (kml_fft_r2r_kind*)kml_fft_malloc(sizeof(kml_fft_r2r_kind)*rank);

n[0] = N0;
n[1] = N1;
kind[0] = kind0;
kind[1] = kind1;

/* get local data size and allocate */
alloc_local = kml_fft_mpi_local_size_many(rank, n, howmany, block0, comm, &local_n0, &local_0_start);
if (alloc_local == -1) {
    printf("[%s][%d] allocate size fail.\n", __func__, __LINE__);
}
in = (double *)kml_fft_malloc(sizeof(double) * alloc_local);
if (in == NULL) {
    printf("[%s][%d] malloc memory fail.\n", __func__, __LINE__);
}
out = (double *)kml_fft_malloc(sizeof(double) * alloc_local);
if (out == NULL) {
    printf("[%s][%d] malloc memory fail.\n", __func__, __LINE__);
}

/* create plan for in-place forward DFT */
plan = kml_fft_mpi_plan_many_r2r(rank, n, howmany, block0, block1, in, out, kind, comm,
KML_FFT_ESTIMATE);

/* compute transforms, in-place, as many times as desired */
kml_fft_execute_r2r(plan, in, out);

kml_fft_destroy_plan(plan);
kml_fft_mpi_cleanup();

MPI_Finalize();
```

6.4.4.5.4.5 kml_fft(f)_mpi_plan_r2r_3d_ext

建立单个连续数据序列3维R2R变换的plan。

接口定义

C interface：

 版权所有 © 华为技术有限公司

kml_fft_plan kml_fft_mpi_plan_r2r_3d_ext(int n0, int n1, int n2, const int *inbox_order, const int *outbox_order, const kml_fft_r2r_kind kind0, const kml_fft_r2r_kind kind1, const kml_fft_r2r_kind kind2, MPI_Comm comm, const kml_fft_mpi_options options);

kml_fftf_plan kml_fftf_mpi_plan_r2r_3d_ext(int n0, int n1, int n2, const int *inbox_order, const int *outbox_order, const kml_fftf_r2r_kind kind0, const kml_fftf_r2r_kind kind1, const kml_fftf_r2r_kind kind2, MPI_Comm comm, const kml_fftf_mpi_options options);

返回值

函数返回一个kml_fft(f)_plan类型的结构体指针。将该对象作为参数传入**kml_fft(f)_mpi_execute_r2r_ext函数**中使用，且不需要在plan创建阶段指定in、out buffer。

如果函数返回非空指针，则表示plan创建成功，否则表示创建失败。

参数

参数名	数据类型	描述	输入/输出
n0	int	输入FFT序列第一维的大小，约束：n0≥1。	输入
n1	int	输入FFT序列第二维的大小，约束：n1≥1。	输入
n2	int	输入FFT序列第三维的大小，约束：n2≥1。	输入
inbox_order	const int *	是长度为3的一维数组，输入数据维度的顺序，取值为0、1、2的任意顺序组合，输入NULL，默认order为0、1、2。	输入
outbox_order	const int *	是长度为3的一维数组，输出数据维度的顺序，取值为0、1、2的任意顺序组合，输入NULL，默认order为0、1、2。	输入

 版权所有 © 华为技术有限公司

参数名	数据类型	描述	输入/输出
kind0	• 双精度：const kml_fft_r2r_kind • 单精度： const kml_fftf_r2r_kind	kind0表示FFT序列第一维的R2R变换类型，有以下可选值： • KML_FFT_R2HC • KML_FFT_HC2R • KML_FFT_DHT • KML_FFT_REDFT00 • KML_FFT_REDFT01 • KML_FFT_REDFT10 • KML_FFT_REDFT11 • KML_FFT_RODFT00 • KML_FFT_RODFT01 • KML_FFT_RODFT10 • KML_FFT_RODFT11	输入
kind1	• 双精度：const kml_fft_r2r_kind • 单精度： const kml_fftf_r2r_kind	kind1表示FFT序列第二维的R2R变换类型，有以下可选值： • KML_FFT_R2HC • KML_FFT_HC2R • KML_FFT_DHT • KML_FFT_REDFT00 • KML_FFT_REDFT01 • KML_FFT_REDFT10 • KML_FFT_REDFT11 • KML_FFT_RODFT00 • KML_FFT_RODFT01 • KML_FFT_RODFT10 • KML_FFT_RODFT11	输入

 版权所有 © 华为技术有限公司

参数名	数据类型	描述	输入/输出
kind2	• 双精度：const kml_fft_r2r_kind • 单精度： const kml_fftf_r2r_kind	kind2表示FFT序列第三维的R2R变换类型，有以下可选值： • KML_FFT_R2HC • KML_FFT_HC2R • KML_FFT_DHT • KML_FFT_REDFT00 • KML_FFT_REDFT01 • KML_FFT_REDFT10 • KML_FFT_REDFT11 • KML_FFT_RODFT00 • KML_FFT_RODFT01 • KML_FFT_RODFT10 • KML_FFT_RODFT11	输入
comm	MPI_Comm	通信域。	输入
options	• 双精度：const kml_fft_mpi_options • 单精度： const kml_fftf_mpi_options	3DFFT分解算法、通信算法flag。	输入

依赖

C: "kfft-mpi.h"

示例

C interface：

```
   const int n0 = 4, n1 = 4, n2 = 4;
   kml_fft_plan plan;
   int provided;
   MPI_Init_thread(&argc, &argv, MPI_THREAD_MULTIPLE, &provided);
   MPI_Comm comm = MPI_COMM_WORLD;
   double *in = NULL;
   double *out = NULL;

   /* get local data size and allocate */
   ptrdiff_t low[3] = {0};
   ptrdiff_t high[3] = {0};
   ptrdiff_t alloc_local = kml_fft_mpi_local_size_3d_ext(n0, n1, n2, comm,
SCALFFT_DECOMPOSE_TYPE_PENCIL, low, high);
   if (alloc_local == -1) {
       printf("[%s][%d] allocate size fail.\n", __func__, __LINE__);
   }
   in = (double *)kml_fft_malloc(sizeof(double) * alloc_local);
   if (in == NULL) {
```

 版权所有 © 华为技术有限公司

```
    printf("[%s][%d] malloc memory fail.\n", __func__, __LINE__);
}
out = (double *)kml_fft_malloc(sizeof(double) * alloc_local);
if (out == NULL) {
    printf("[%s][%d] malloc memory fail.\n", __func__, __LINE__);
}

/* create plan */
kml_fft_mpi_options options = {
    .a2a_algo    = A2A_ALGO_AUTO_TUNING,
    .decomp_type = SCALFFT_DECOMPOSE_TYPE_PENCIL
};
kml_fft_r2r_kind kind0 = KML_FFT_DHT;
kml_fft_r2r_kind kind1 = KML_FFT_DHT;
kml_fft_r2r_kind kind2 = KML_FFT_DHT;
plan = kml_fft_mpi_plan_r2r_3d_ext(n0, n1, n2, NULL, NULL, kind0, kind1, kind2,  comm, options);

/* execute plan */
int scale = 0;
kml_fft_mpi_execute_r2r_ext(plan, in, out, scale);

kml_fft_destroy_plan_ext(plan);

kml_fft_free(in);
kml_fft_free(out);
MPI_Finalize();
```

6.4.4.5.4.6 kml_fft(f)_mpi_plan_create_r2r

建立单个连续数据序列3维R2R变换的plan。

接口定义

C interface：

kml_fft_plan kml_fft_mpi_plan_create_r2r(int backend, const int *inbox_low, const int *inbox_high, const int *inbox_order, const int *outbox_low, const int *outbox_high, const int *outbox_order, const kml_fft_r2r_kind *kind, MPI_Comm comm, const kml_fft_mpi_options options);

kml_fftf_plan kml_fftf_mpi_plan_create_r2r(int backend, const int *inbox_low, const int *inbox_high, const int *inbox_order, const int *outbox_low, const int *outbox_high, const int *outbox_order, const kml_fftf_r2r_kind *kind, MPI_Comm comm, const kml_fftf_mpi_options options);

返回值

函数返回一个kml_fft(f)_plan类型的结构体指针。将该对象作为参数**kml_fft(f)_mpi_execute_r2r_ext函数**中使用，且不需要在plan创建阶段指定in、out buffer。

如果函数返回非空指针，则表示plan创建成功，否则表示创建失败。

参数

参数名	数据类型	描述	输入/输出
backend	int	底层库类型，当前仅支持BACKEND_KFFT	输入

 版权所有 © 华为技术有限公司

参数名	数据类型	描述	输入/输出
inbox_low	const int *	是长度为3的一维数组，输入数据在三个维度的切分范围下限，-1≤inbox_low[i]≤n[i]-1 for i in 0、1、 2	输入
inbox_high	const int *	是长度为3的一维数组，输入数据在三个维度的切分范围上限，-1≤inbox_high[i]≤n[i]-1 for i in 0、1、2，且0≤（inbox_high[i]-inbox_low[i]+1）≤n[i] for i in 0、1、2	输入
inbox_order	const int *	是长度为3的一维数组，输入数据维度的顺序，取值为0、1、2的任意顺序组合，输入NULL，默认order为0、1、2。	输入
outbox_low	const int *	是长度为3的一维数组，输出数据在三个维度的切分范围下限，-1≤outbox_low[i]≤n[i]-1 for i in 0、1、 2	输入
outbox_high	const int *	是长度为3的一维数组，输出数据在三个维度的切分范围上限，-1≤outbox_high[i]≤n[i]-1 for i in 0、1、2，且 0≤(outbox_high[i] - outbox_low[i] + 1)≤n[i] for i in 0、1、2	输入
outbox_order	const int *	是长度为3的一维数组，输出数据维度的顺序，取值为0、1、2的任意顺序组合，输入NULL，默认order为0、1、2。	输入

版权所有 © 华为技术有限公司

参数名	数据类型	描述	输入/输出
kind	• 双精度：const kml_fft_r2r_kind* • 单精度：const kml_fftf_r2r_kind*	kind是大小为3的数组，包含FFT序列每一维度的R2R变换类型，kind[i] (for i in 0、1、2)有以下可选值： • KML_FFT_R2HC • KML_FFT_HC2R • KML_FFT_DHT • KML_FFT_REDFT00 • KML_FFT_REDFT01 • KML_FFT_REDFT10 • KML_FFT_REDFT11 • KML_FFT_RODFT00 • KML_FFT_RODFT01 • KML_FFT_RODFT10 • KML_FFT_RODFT11	输入
comm	MPI_Comm	通信域	输入
options	• 双精度：const kml_fft_mpi_options • 单精度： const kml_fftf_mpi_options	3DFFT分解算法、通信算法flag	输入

依赖

C: "kfft-mpi.h"

示例

C interface：

```
  const int n0 = 4, n1 = 4, n2 = 4;
   kml_fft_plan plan;
   int provided;
   MPI_Init_thread(&argc, &argv, MPI_THREAD_MULTIPLE, &provided);
   MPI_Comm comm = MPI_COMM_WORLD;
   double *in = NULL;
   double *out = NULL;

   /* get local data size and allocate */
   ptrdiff_t low[3] = {0};
   ptrdiff_t high[3] = {0};
   ptrdiff_t alloc_local = kml_fft_mpi_local_size_3d_ext(n0, n1, n2, comm,
SCALFFT_DECOMPOSE_TYPE_PENCIL, low, high);
   if (alloc_local == -1) {
       printf("[%s][%d] allocate size fail.\n", __func__, __LINE__);
   }
   in = (double *)kml_fft_malloc(sizeof(double) * alloc_local);
```

 版权所有 © 华为技术有限公司

```
    if (in == NULL) {
        printf("[%s][%d] malloc memory fail.\n", __func__, __LINE__);
    }
    out = (double *)kml_fft_malloc(sizeof(double) * alloc_local);
    if (out == NULL) {
        printf("[%s][%d] malloc memory fail.\n", __func__, __LINE__);
    }

    /* create plan */
    int low_int[3] = {low[0], low[1], low[2]};
    int high_int[3] = {high[0], high[1], high[2]};
    kml_fft_mpi_options options = {
        .a2a_algo    = A2A_ALGO_AUTO_TUNING,
        .decomp_type = SCALFFT_DECOMPOSE_TYPE_PENCIL
    };
    kml_fft_r2r_kind kind[3] = {KML_FFT_DHT, KML_FFT_DHT, KML_FFT_DHT};
    plan = kml_fft_mpi_plan_create_r2r(BACKEND_KFFT, low_int, high_int, NULL, low_int, high_int, NULL,
kind, comm, options);

    /* execute plan */
    int scale = 0;
    kml_fft_mpi_execute_r2r_ext(plan, in, out, scale);

    kml_fft_destroy_plan_ext(plan);

    kml_fft_free(in);
    kml_fft_free(out);
    MPI_Finalize();
```

6.4.4.5.5 变换执行函数

6.4.4.5.5.1 kml_fft(f/h)_mpi_execute_dft

kml_fft(f/h)_mpi_execute_dft执行已预先建立的FFT变换plan，但是可以接受与plan不同的新的输入输出数据作为参数。

接口定义

C interface：

void kml_fft_mpi_execute_dft(const kml_fft_plan p, kml_fft_complex *in, kml_fft_complex *out);

void kml_fftf_mpi_execute_dft(const kml_fftf_plan p, kml_fftf_complex *in, kml_fftf_complex *out);

void kml_ffth_mpi_execute_dft(const kml_ffth_plan p, kml_ffth_complex *in, kml_ffth_complex *out);

 版权所有 © 华为技术有限公司

参数

参数名	数据类型	描述	输入/输出
p	• 双精度：const kml_fft_plan • 单精度：const kml_fftf_plan • 半精度：const kml_ffth_plan	通过调用plan函数得到的FFT变换plan。	输入
in	• 双精度：kml_fft_complex* • 单精度：kml_fftf_complex* • 半精度：kml_ffth_complex*	输入需要变换的数据。	输入
out	• 双精度：kml_fft_complex* • 单精度：kml_fftf_complex* • 半精度：kml_ffth_complex*	输出快速傅里叶变换后的数据。	输出

依赖

C: "kfft-mpi.h"

示例

见6.4.4.5.1 C2C变换相关用例。

6.4.4.5.5.2 kml_fft(f/h)_mpi_execute_dft_r2c

kml_fft(f/h)_mpi_execute_dft_r2c执行已预先建立的FFT变换plan，但是可以接受与plan不同的新的输入输出数据作为参数。

接口定义

C interface：

```
void kml_fft_mpi_execute_dft_r2c(const kml_fft_plan p, double *in,
kml_fft_complex *out);
```


版权所有 © 华为技术有限公司

void kml_fftf_mpi_execute_dft_r2c(const kml_fftf_plan p, float *in, kml_fftf_complex *out);

void kml_ffth_mpi_execute_dft_r2c(const kml_ffth_plan p, __fp16 *in, kml_ffth_complex *out);

参数

参数名	数据类型	描述	输入/输出
p	• 双精度：const kml_fft_plan • 单精度：const kml_fftf_plan • 半精度：const kml_ffth_plan	通过调用plan函数得到的FFT变换plan。	输入
in	• 双精度：double* • 单精度：float* • 半精度：__fp16*	输入需要变换的数据。	输入
out	• 双精度：kml_fft_complex* • 单精度：kml_fftf_complex* • 半精度：kml_ffth_complex*	输出快速傅里叶变换后的数据。	输出

依赖

C: "kfft-mpi.h"

示例

见6.4.4.5.2 R2C变换的相关用例。

6.4.4.5.5.3 kml_fft(f/h)_mpi_execute_dft_c2r

kml_fft(f/h)_mpi_execute_dft_c2r执行已预先建立的FFT变换plan，但是可以接受与plan不同的新的输入输出数据作为参数。

接口定义

C interface：

 版权所有 © 华为技术有限公司

void kml_fft_mpi_execute_dft_c2r(const kml_fft_plan p, kml_fft_complex *in, double *out);

void kml_fftf_mpi_execute_dft_c2r(const kml_fftf_plan p, kml_fftf_complex *in, float *out);

void kml_ffth_mpi_execute_dft_c2r(const kml_ffth_plan p, kml_ffth_complex *in, __fp16 *out);

参数

参数名	数据类型	描述	输入/输出
p	• 双精度：const kml_fft_plan • 单精度：const kml_fftf_plan • 半精度：const kml_ffth_plan	通过调用plan函数得到的FFT变换plan。	输入
in	• 双精度：kml_fft_complex* • 单精度：kml_fftf_complex* • 半精度：kml_ffth_complex*	输入需要变换的数据。	输入
out	• 双精度：double* • 单精度：float* • 半精度：__fp16*	输出快速傅里叶变换后的数据。	输出

依赖

C: "kfft-mpi.h"

示例

见**6.4.4.5.3 C2R变换**的相关用例。

6.4.4.5.5.4 kml_fft(f/h)_mpi_execute_r2r

kml_fft(f/h)_mpi_execute_r2r执行已预先建立的FFT变换plan，但是可以接受与plan不同的新的输入输出数据作为参数。

版权所有 © 华为技术有限公司

接口定义

C interface：

void kml_fft_mpi_execute_r2r(const kml_fft_plan p, double *in, double *out);

void kml_fftf_mpi_execute_r2r(const kml_fftf_plan p, float *in, float *out);

void kml_ffth_mpi_execute_r2r(const kml_ffth_plan p, __fp16 *in, __fp16 *out);

参数

参数名	数据类型	描述	输入/输出
p	• 双精度：const kml_fft_plan • 单精度：const kml_fftf_plan • 半精度：const kml_ffth_plan	通过调用plan函数得到的FFT变换plan。	输入
in	• 双精度：double* • 单精度：float* • 半精度：__fp16*	输入需要变换的数据。	输入
out	• 双精度：double* • 单精度：float* • 半精度：__fp16*	输出快速傅里叶变换后的数据。	输出

依赖

C: "kfft-mpi.h"

示例

见6.4.4.5.4 R2R变换的相关用例。

6.4.4.5.5.5 kml_fft(f)_mpi_execute_dft_ext

kml_fft(f)_ mpi_execute_dft_ext执行已预先建立的FFT变换plan，forward与backward可复用同一plan。

接口定义

C interface：

void kml_fft_mpi_execute_dft_ext(const kml_fft_plan p, kml_fft_complex *in, kml_fft_complex *out, int scale, int sign_flag);

版权所有 © 华为技术有限公司

void kml_fftf_mpi_execute_dft_ext(const kml_fftf_plan p, kml_fftf_complex *in, kml_fftf_complex *out, int scale, int sign_flag);

参数

参数名	数据类型	描述	输入/输出
p	• 双精度：const kml_fft_plan • 单精度：const kml_fftf_plan	通过调用plan函数得到的FFT变换plan。	输入
in	• 双精度：kml_fft_complex* • 单精度：kml_fftf_complex*	输入需要变换的数据。	输入
out	• 双精度：kml_fft_complex* • 单精度：kml_fftf_complex*	输出快速傅里叶变换后的数据。	输出
scale	int	是否对输出结果进行缩放，scale设置为1表示对输出结果进行缩放，scale设置为0表示对输出结果不做处理。	输入
sign_flag	int	3DFFT计算方向，KML_FFT_FORWARD/KML_FFT_BACKWARD。	输入

依赖

C: "kfft-mpi.h"

示例

见6.4.4.5.1 C2C变换的相关示例。

6.4.4.5.5.6 kml_fft(f)_mpi_execute_dft_r2c_ext

kml_fft(f)_ mpi_execute_dft_r2c_ext执行已预先建立的FFT变换plan。

接口定义

C interface：

 版权所有 © 华为技术有限公司

void kml_fft_mpi_execute_dft_r2c_ext(const kml_fft_plan p, double *in, kml_fft_complex *out, int scale);

void kml_fftf_mpi_execute_dft_r2c_ext(const kml_fftf_plan p, float *in, kml_fftf_complex *out, int scale);

参数

参数名	数据类型	描述	输入/输出
p	• 双精度：const kml_fft_plan • 单精度：const kml_fftf_plan	通过调用plan函数得到的FFT变换plan。	输入
in	• 双精度：double* • 单精度：float*	输入需要变换的数据。	输入
out	• 双精度：kml_fft_complex* • 单精度：kml_fftf_complex*	输出快速傅里叶变换后的数据。	输出
scale	int	是否对输出结果进行缩放，scale设置为1表示对输出结果进行缩放，scale设置为0表示对输出结果不做处理。	输入

依赖

C: "kfft-mpi.h"

示例

见6.4.4.5.2 R2C变换的相关示例。

6.4.4.5.5.7 kml_fft(f)_mpi_execute_dft_c2r_ext

kml_fft(f)_ mpi_execute_dft_c2r_ext执行已预先建立的FFT变换plan。

接口定义

C interface：

void kml_fft_mpi_execute_dft_c2r_ext(const kml_fft_plan p, kml_fft_complex *in, double *out, int scale);

void kml_fftf_mpi_execute_dft_c2r_ext(const kml_fftf_plan p, kml_fftf_complex *in, float *out, int scale);

 版权所有 © 华为技术有限公司

参数

参数名	数据类型	描述	输入/输出
p	• 双精度：const kml_fft_plan • 单精度：const kml_fftf_plan	通过调用plan函数得到的FFT变换plan。	输入
in	• 双精度：kml_fft_complex* • 单精度：kml_fftf_complex*	输入需要变换的数据。	输入
out	• 双精度：double* • 单精度：float*	输出快速傅里叶变换后的数据。	输出
scale	int	是否对输出结果进行缩放，scale设置为1表示对输出结果进行缩放，scale设置为0表示对输出结果不做处理。	输入

依赖

C: "kfft-mpi.h"

示例

见6.4.4.5.3 C2R变换的相关示例。

6.4.4.5.5.8 kml_fft(f)_mpi_execute_r2r_ext

kml_fft(f)_ mpi_execute_r2r_ext执行已预先建立的FFT变换plan。

接口定义

C interface：

void kml_fft_mpi_execute_r2r_ext(const kml_fft_plan p, double *in, double *out, int scale);

void kml_fftf_mpi_execute_r2r_ext(const kml_fftf_plan p, float *in, float *out, int scale);

版权所有 © 华为技术有限公司

参数

参数名	数据类型	描述	输入/输出
p	• 双精度：const kml_fft_plan • 单精度：const kml_fftf_plan	通过调用plan函数得到的FFT变换plan。	输入
in	• 双精度：double* • 单精度：float*	输入需要变换的数据。	输入
out	• 双精度：double* • 单精度：float*	输出快速傅里叶变换后的数据。	输出
scale	int	是否对输出结果进行缩放，scale设置为1表示对输出结果进行缩放，scale设置为0表示对输出结果不做处理	输入

依赖

C: "kfft-mpi.h"

示例

见6.4.4.5.4 R2R变换的相关示例。

6.4.4.5.5.9 kml_fft(f)_mpi_forward_c2c

kml_fft(f)_mpi_forward_c2c按照forward方向执行已预先建立的FFT变换plan，forward与backward可复用同一plan。

接口定义

C interface：

void kml_fft_mpi_forward_c2c(const kml_fft_plan p, kml_fft_complex *in, kml_fft_complex *out, int scale);

void kml_fftf_mpi_forward_c2c(const kml_fftf_plan p, kml_fftf_complex *in, kml_fftf_complex *out, int scale);

 版权所有 © 华为技术有限公司

参数

参数名	数据类型	描述	输入/输出
p	• 双精度：const kml_fft_plan • 单精度：const kml_fftf_plan	通过调用plan函数得到的FFT变换plan。	输入
in	• 双精度：kml_fft_complex* • 单精度：kml_fftf_complex*	输入需要变换的数据。	输入
out	• 双精度：kml_fft_complex* • 单精度：kml_fftf_complex*	输出快速傅里叶变换后的数据。	输出
scale	int	是否对输出结果进行缩放，scale设置为1表示对输出结果进行缩放，scale设置为0表示对输出结果不做处理；	输入

依赖

C: "kfft-mpi.h"

示例

见 **kml_fft(f)_mpi_forward_c2c** 的相关示例。

6.4.4.5.5.10 kml_fft(f)_mpi_backward_c2c

kml_fft(f)_mpi_backward_c2c按照backward方向执行已预先建立的FFT变换plan，forward与backward可复用同一plan。

接口定义

C interface：

void kml_fft_mpi_backward_c2c(const kml_fft_plan p, kml_fft_complex *in, kml_fft_complex *out, int scale);

void kml_fftf_mpi_backward_c2c(const kml_fftf_plan p, kml_fftf_complex *in, kml_fftf_complex *out, int scale);

 版权所有 © 华为技术有限公司

参数

参数名	数据类型	描述	输入/输出
p	• 双精度：const kml_fft_plan • 单精度：const kml_fftf_plan	通过调用plan函数得到的FFT变换plan。	输入
in	• 双精度：kml_fft_complex* • 单精度：kml_fftf_complex*	输入需要变换的数据。	输入
out	• 双精度：kml_fft_complex* • 单精度：kml_fftf_complex*	输出快速傅里叶变换后的数据。	输出
scale	int	是否对输出结果进行缩放，scale设置为1表示对输出结果进行缩放，scale设置为0表示对输出结果不做处理。	输入

依赖

C: "kfft-mpi.h"

示例

见 **kml_fft(f)_mpi_plan_create** C2C变换的相关示例。

6.4.4.5.5.11 kml_fft(f)_mpi_forward_r2c

kml_fft(f)_mpi_forward_r2c按照forward方向执行已预先建立的FFT变换plan。

接口定义

C interface：

void kml_fft_mpi_forward_r2c(const kml_fft_plan p, double *in, kml_fft_complex *out, int scale);

void kml_fftf_mpi_forward_r2c(const kml_fftf_plan p, float *in, kml_fftf_complex *out, int scale);

 版权所有 © 华为技术有限公司

参数

参数名	数据类型	描述	输入/输出
p	• 双精度：const kml_fft_plan • 单精度：const kml_fftf_plan	通过调用plan函数得到的FFT变换plan。	输入
in	• 双精度：double* • 单精度：float*	输入需要变换的数据。	输入
out	• 双精度：kml_fft_complex* • 单精度：kml_fftf_complex*	输出快速傅里叶变换后的数据。	输出
scale	int	是否对输出结果进行缩放，scale设置为1表示对输出结果进行缩放，scale设置为0表示对输出结果不做处理。	输入

依赖

C: "kfft-mpi.h"

示例

见 **kml_fft(f)_mpi_plan_create_r2c** R2C变换的相关示例。

6.4.4.5.5.12 kml_fft(f)_mpi_backward _c2r

kml_fft(f)_ mpi_backward _c2r按照backward方向执行已预先建立的FFT变换plan。

接口定义

C interface：

void kml_fft_mpi_backward_c2r(const kml_fft_plan p, kml_fft_complex *in, double *out, int scale);

void kml_fftf_mpi_backward_c2r(const kml_fftf_plan p, kml_fftf_complex *in, float *out, int scale);

 版权所有 © 华为技术有限公司

参数

参数名	数据类型	描述	输入/输出
p	• 双精度：const kml_fft_plan • 单精度：const kml_fftf_plan	通过调用plan函数得到的FFT变换plan。	输入
in	• 双精度：kml_fft_complex* • 单精度：kml_fftf_complex*	输入需要变换的数据。	输入
out	• 双精度：double* • 单精度：float*	输出快速傅里叶变换后的数据。	输出
scale	int	是否对输出结果进行缩放，scale设置为1表示对输出结果进行缩放，scale设置为0表示对输出结果不做处理	输入

依赖

C: "kfft-mpi.h"

示例

见 **kml_fft(f)_mpi_plan_create_c2r** C2R变换的相关示例。

6.4.4.5.6 进程函数

mpi环境下的kml-fft资源初始化和释放。

6.4.4.5.6.1 kml_fft_mpi_init

mpi环境下的kml-fft资源初始化。

接口定义

C interface：

int kml_fft_mpi_init(void)

int kml_fftf_mpi_init(void)

返回值

kml_fft_mpi_init函数执行成功返回1，否则返回0。

 版权所有 © 华为技术有限公司

参数

无

依赖

C: "kfft-mpi.h"

示例

见 **kml_fft(f/h)_mpi_plan_dft**的相关示例。

6.4.4.5.6.2 kml_fft_mpi_cleanup

用于释放mpi环境下的kml-fft资源。

接口定义

C interface：

int kml_fft_mpi_cleanup(void)

int kml_fftf_mpi_cleanup(void)

返回值

kml_fft_mpi_cleanup函数执行成功返回1，否则返回0。

参数

无

依赖

C: "kfft-mpi.h"

示例

见 **kml_fft(f/h)_mpi_plan_dft**的相关示例。

6.4.4.5.7 MPI Local Size 函数

Cluster FFT的local size函数用于获取本地进程所需分配的空间大小、数据起始位置、数据大小等信息。

6.4.4.5.7.1 kml_fft(f/h)_mpi_local_size

描述了本地进程数据的大小和位置，以及计算需要分配的空间。

接口定义

C interface：

ptrdiff_t kml_fft_mpi_local_size(int rank, const ptrdiff_t *n, MPI_Comm comm, ptrdiff_t *local_n0, ptrdiff_t *local_0_start);

 版权所有 © 华为技术有限公司

ptrdiff_t kml_fftf_mpi_local_size(int rank, const ptrdiff_t *n, MPI_Comm comm, ptrdiff_t *local_n0, ptrdiff_t *local_0_start);

ptrdiff_t kml_ffth_mpi_local_size(int rank, const ptrdiff_t *n, MPI_Comm comm, ptrdiff_t *local_n0, ptrdiff_t *local_0_start);

返回值

函数返回一个ptrdiff_t类型的值，表示要分配的元素的数量。

参数

参数名	数据类型	描述	输入/输出
rank	• ptrdiff_t	待处理数据维度数量，约束：rank ≥ 1。	输入
n	• const ptrdiff_t *	n为维度为rank的数组，包含数据序列每一维度的大小，约束：n[i] ≥ 1, for i in 0 to rank - 1。	输入
comm	• MPI_Comm	MPI通信器的句柄。	输入
local_n0	• ptrdiff_t*	本地进程数据大小。	输出
local_0_start	• ptrdiff_t*	本地进程数据相对全局数据起点偏移量。	输出

依赖

C: "kfft-mpi.h"

示例

见 **kml_fft(f/h)_mpi_plan_dft**的相关示例。

6.4.4.5.7.2 kml_fft(f/h)_mpi_local_size_transposed

描述了本地进程数据的大小和位置，以及计算需要分配的空间。

接口定义

C interface：

ptrdiff_t kml_fft_mpi_local_size_transposed(int rank, const ptrdiff_t *n, MPI_Comm comm, ptrdiff_t *local_n0, ptrdiff_t *local_0_start, ptrdiff_t *local_n1, ptrdiff_t *local_1_start);

ptrdiff_t kml_fftf_mpi_local_size_transposed(int rank, const ptrdiff_t *n, MPI_Comm comm, ptrdiff_t *local_n0, ptrdiff_t *local_0_start, ptrdiff_t *local_n1, ptrdiff_t *local_1_start);

 版权所有 © 华为技术有限公司

ptrdiff_t kml_ffth_mpi_local_size_transposed(int rank, const ptrdiff_t *n, MPI_Comm comm, ptrdiff_t *local_n0, ptrdiff_t *local_0_start, ptrdiff_t *local_n1, ptrdiff_t *local_1_start);

返回值

函数返回一个ptrdiff_t类型的值，表示要分配的元素的数量。

参数

参数名	数据类型	描述	输入/输出
rank	• int	待处理数据维度数量，约束：rank ≥ 1。	输入
n	• n	n为维度为rank的数组，包含数据序列每一维度的大小，约束：n[i] ≥ 1, for i in 0 to rank - 1。	输入
comm	• MPI_Comm	MPI通信器的句柄。	输入
local_n0	• ptrdiff_t*	本地进程数据大小。	输出
local_0_start	• ptrdiff_t*	本地进程数据相对全局数据起点偏移量。	输出
local_n1	• ptrdiff_t*	本地进程输出数据大小。	输出
local_1_start	• ptrdiff_t*	本地进程输出数据相对全局数据起点偏移量。	输出

依赖

C: "kfft-mpi.h"

示例

```
const ptrdiff_t N0 = 4, N1 = 4;
kml_fft_plan plan;
ptrdiff_t alloc_local, local_n0, local_0_start, local_n1, local_1_start;
ptrdiff_t rank = 2;

MPI_Init(&argc, &argv);
MPI_Comm comm = MPI_COMM_WORLD;
kml_fft_mpi_init();
ptrdiff_t *n = (ptrdiff_t *)kml_fft_malloc(sizeof(ptrdiff_t) * rank);
if (n == NULL) {
    printf("[%s][%d] malloc memory fail.\n", __func__, __LINE__);
}

n[0] = N0;
n[1] = N1;
double *in = NULL;
double *out = NULL;

/* get local data size and allocate */
alloc_local = kml_fft_mpi_local_size_transposed(rank, n, comm,
```


版权所有 © 华为技术有限公司

```
                                        &local_n0, &local_0_start,
                                        &local_n1, &local_1_start);
if (alloc_local == -1) {
    printf("[%s][%d] allocate size fail.\n", __func__, __LINE__);
}
in = (double *)kml_fft_malloc(sizeof(double) * alloc_local * 2);
if (in == NULL) {
    printf("[%s][%d] malloc memory fail.\n", __func__, __LINE__);
}
out = (double *)kml_fft_malloc(sizeof(double) * alloc_local * 2);
if (out == NULL) {
    printf("[%s][%d] malloc memory fail.\n", __func__, __LINE__);
}

/* create plan for in-place forward DFT */
plan = kml_fft_mpi_plan_transpose(n[0], n[1], in, out, comm, KML_FFT_ESTIMATE);

/* compute transforms, in-place, as many times as desired */
kml_fft_execute(plan);

kml_fft_destroy_plan(plan);
kml_fft_mpi_cleanup();

MPI_Finalize();
```

6.4.4.5.7.3 kml_fft(f/h)_mpi_local_size_1d

描述了本地进程数据的大小和位置，以及计算需要分配的空间。

接口定义

C interface：

ptrdiff_t kml_fft_mpi_local_size_1d(ptrdiff_t n0, MPI_Comm comm, int sign, unsigned flags, ptrdiff_t *local_ni, ptrdiff_t *local_i_start, ptrdiff_t *local_no, ptrdiff_t *local_o_start);

ptrdiff_t kml_fftf_mpi_local_size_1d(ptrdiff_t n0, MPI_Comm comm, int sign, unsigned flags, ptrdiff_t *local_ni, ptrdiff_t *local_i_start, ptrdiff_t *local_no, ptrdiff_t *local_o_start);

ptrdiff_t kml_ffth_mpi_local_size_1d(ptrdiff_t n0, MPI_Comm comm, int sign, unsigned flags, ptrdiff_t *local_ni, ptrdiff_t *local_i_start, ptrdiff_t *local_no, ptrdiff_t *local_o_start);

返回值

函数返回一个ptrdiff_t类型的值，表示要分配的元素的数量；local_ni和local_i_start参数返回存储在此过程中的转换输入的1d数组的一部分（local_i_start~local_i_start+local_ni-1）；local_no和local_o_start参数返回存储在此过程中的转换输出的1d数组的一部分（local_o_start~local_o_start+local_no-1）。

参数

参数名	数据类型	描述	输入/输出
n0	ptrdiff_t	1D FFT待处理数据大小，约束：n0 ≥ 1。	输入

 版权所有 © 华为技术有限公司

参数名	数据类型	描述	输入/输出
comm	MPI_Comm	MPI通信器的句柄。	输入
sign	int	与创建plan时传入的参数匹配。	输入
flags	unsigned int	与创建plan时传入的参数匹配。	输入
local_ni	ptrdiff_t*	本地进程输入数据大小。	输出
local_i_start	ptrdiff_t*	本地进程输入数据相对全局输入数据起点偏移量。	输出
local_no	ptrdiff_t*	本地进程输出数据大小。	输出
local_o_start	ptrdiff_t*	本地进程输出数据相对全局输出数据起点偏移量。	输出

依赖

C: "kfft-mpi.h"

示例

见 **kml_fft(f/h)_mpi_plan_dft_1d**相关的示例。

6.4.4.5.7.4 kml_fft(f/h)_mpi_local_size_many_1d

描述了本地进程数据的大小和位置，以及计算需要分配的空间。

接口定义

C interface：

ptrdiff_t kml_fft_mpi_local_size_many_1d(ptrdiff_t n0, ptrdiff_t howmany, MPI_Comm comm, int sign, unsigned flags, ptrdiff_t *local_ni, ptrdiff_t *local_i_start, ptrdiff_t *local_no, ptrdiff_t *local_o_start);

ptrdiff_t kml_fftf_mpi_local_size_many_1d(ptrdiff_t n0, ptrdiff_t howmany, MPI_Comm comm, int sign, unsigned flags, ptrdiff_t *local_ni, ptrdiff_t *local_i_start, ptrdiff_t *local_no, ptrdiff_t *local_o_start);

ptrdiff_t kml_ffth_mpi_local_size_many_1d(ptrdiff_t n0, ptrdiff_t howmany, MPI_Comm comm, int sign, unsigned flags, ptrdiff_t *local_ni, ptrdiff_t *local_i_start, ptrdiff_t *local_no, ptrdiff_t *local_o_start);

返回值

函数返回一个ptrdiff_t类型的值，表示要分配的元素的数量（复数，对于复数DFT）；local_ni和local_i_start参数返回存储在此过程中的转换输入的1d数组的一部分（local_i_start~local_i_start+local_ni-1）；local_no和local_o_start参数返回存储在此过程中的转换输出的1d数组的一部分（local_o_start~local_o_start+local_no-1）。

 版权所有 © 华为技术有限公司

参数

参数名	数据类型	描述	输入/输出
n0	ptrdiff_t	1D FFT待处理数据大小，约束：n0 ≥ 1。	输入
howmany	ptrdiff_t	howmany表示变换的数量，与创建plan时传入参数匹配。	输入
comm	MPI_Comm	MPI通信器的句柄。	输入
sign	int	与创建plan时传入的参数匹配。	输入
flags	unsigned int	与创建plan时传入的参数匹配。	输入
local_ni	ptrdiff_t*	本地进程输入数据大小。	输出
local_i_start	ptrdiff_t*	本地进程输入数据相对全局输入数据起点偏移量。	输出
local_no	ptrdiff_t*	本地进程输出数据大小。	输出
local_o_start	ptrdiff_t*	本地进程输出数据相对全局输出数据起点偏移量。	输出

依赖

C: "kfft-mpi.h"

示例

```
    const ptrdiff_t N0 = 4;
    kml_fft_plan plan;
    ptrdiff_t alloc_local, local_ni, local_i_start, local_no, local_o_start;
    MPI_Init(&argc, &argv);
    MPI_Comm comm = MPI_COMM_WORLD;
    kml_fft_mpi_init();
    kml_fft_complex *in = NULL;
    kml_fft_complex *out = NULL;
    ptrdiff_t howmany = 1;
    /* get local data size and allocate */
    alloc_local = kml_fft_mpi_local_size_many_1d(N0, howmany, comm, KML_FFT_FORWARD,
KML_FFT_MPI_TRANSPOSED_IN,
                                         &local_ni, &local_i_start, &local_no, &local_o_start);
    if (alloc_local == -1) {
        printf("[%s][%d] allocate size fail.\n", __func__, __LINE__);
    }
    in = (kml_fft_complex *)kml_fft_malloc(sizeof(kml_fft_complex) * alloc_local);
    if (in == NULL) {
        printf("[%s][%d] malloc memory fail.\n", __func__, __LINE__);
    }
    out = (kml_fft_complex *)kml_fft_malloc(sizeof(kml_fft_complex) * alloc_local);
    if (out == NULL) {
        printf("[%s][%d] malloc memory fail.\n", __func__, __LINE__);
    }
    /* create plan for in-place forward DFT */
    plan = kml_fft_mpi_plan_dft_1d(N0, in, out, comm,
                                KML_FFT_FORWARD, KML_FFT_MPI_TRANSPOSED_IN);
```


版权所有 © 华为技术有限公司

```
/* compute transforms, in-place, as many times as desired */
kml_fft_execute_dft(plan, int, out);
kml_fft_destroy_plan(plan);
MPI_Finalize();
return 0;
```

6.4.4.5.7.5 kml_fft(f/h)_mpi_local_size_2d

描述了本地进程数据的大小和位置，以及计算需要分配的空间。

接口定义

C interface：

ptrdiff_t kml_fft_mpi_local_size_2d(ptrdiff_t n0, ptrdiff_t n1, MPI_Comm comm, ptrdiff_t *local_n0, ptrdiff_t *local_0_start);

ptrdiff_t kml_fftf_mpi_local_size_2d(ptrdiff_t n0, ptrdiff_t n1, MPI_Comm comm, ptrdiff_t *local_n0, ptrdiff_t *local_0_start);

ptrdiff_t kml_ffth_mpi_local_size_2d(ptrdiff_t n0, ptrdiff_t n1, MPI_Comm comm, ptrdiff_t *local_n0, ptrdiff_t *local_0_start);

返回值

函数返回一个ptrdiff_t类型的值，表示要分配的元素的数量。

参数

参数名	数据类型	描述	输入/输出
n0	• ptrdiff_t	2D待处理数据第1维度大小，约束：n0 ≥ 1。	输入
n1	• ptrdiff_t	2D待处理数据第2维度大小，约束：n1 ≥ 1。	输入
comm	• MPI_Comm	MPI通信器的句柄。	输入
local_n0	• ptrdiff_t*	本地进程数据大小。	输出
local_0_start	• ptrdiff_t*	本地进程数据相对全局数据起点偏移量。	输出

依赖

C: "kfft-mpi.h"

示例

见 **kml_fft(f/h)_mpi_plan_dft_2d**的相关示例。

6.4.4.5.7.6 kml_fft(f)_mpi_local_size_2d_transposed

描述了本地进程数据的大小和位置，以及计算需要分配的空间。

 版权所有 © 华为技术有限公司

接口定义

C interface：

ptrdiff_t kml_fft_mpi_local_size_2d_transposed(ptrdiff_t n0, ptrdiff_t n1, MPI_Comm comm, ptrdiff_t *local_n0, ptrdiff_t *local_0_start, ptrdiff_t *local_n1, ptrdiff_t *local_1_start);

ptrdiff_t kml_fftf_mpi_local_size_2d_transposed(ptrdiff_t n0, ptrdiff_t n1, MPI_Comm comm, ptrdiff_t *local_n0, ptrdiff_t *local_0_start, ptrdiff_t *local_n1, ptrdiff_t *local_1_start);

ptrdiff_t kml_ffth_mpi_local_size_2d_transposed(ptrdiff_t n0, ptrdiff_t n1, MPI_Comm comm, ptrdiff_t *local_n0, ptrdiff_t *local_0_start, ptrdiff_t *local_n1, ptrdiff_t *local_1_start);

返回值

函数返回一个ptrdiff_t类型的值，表示要分配的元素的数量。

参数

参数名	数据类型	描述	输入/输出
n0	• ptrdiff_t	待处理数据第1维度大小，约束：n0 ≥ 1。	输入
n1	• ptrdiff_t	待处理数据第2维度大小，约束：n1 ≥ 1。	输入
comm	• MPI_Comm	MPI通信器的句柄。	输入
local_n0	• ptrdiff_t*	本地进程数据大小。	输出
local_0_start	• ptrdiff_t*	本地进程数据相对全局数据起点偏移量。	输出
local_n1	• ptrdiff_t*	本地进程输出数据大小。	输出
local_1_start	• ptrdiff_t*	本地进程输出数据相对全局数据起点偏移量。	输出

依赖

C: "kfft-mpi.h"

示例

```
const ptrdiff_t N0 = 4, N1 = 4;
kml_fft_plan plan;
ptrdiff_t alloc_local, local_n0, local_0_start, local_n1, local_1_start;

MPI_Init(&argc, &argv);
MPI_Comm comm = MPI_COMM_WORLD;
kml_fft_mpi_init();
```


版权所有 © 华为技术有限公司

```
double *in = NULL;
double *out = NULL;

/* get local data size and allocate */
alloc_local = kml_fft_mpi_local_size_2d_transposed(N0, N1, comm, &local_n0, &local_0_start,
                                                   &local_n1, &local_1_start);
if (alloc_local == -1) {
    printf("[%s][%d] allocate size fail.\n", __func__, __LINE__);
}
in = (double *)kml_fft_malloc(sizeof(double) * alloc_local * 2);
if (in == NULL) {
    printf("[%s][%d] malloc memory fail.\n", __func__, __LINE__);
}
out = (double *)kml_fft_malloc(sizeof(double) * alloc_local * 2);
if (out == NULL) {
    printf("[%s][%d] malloc memory fail.\n", __func__, __LINE__);
}

/* create plan for in-place forward DFT */
plan = kml_fft_mpi_plan_transpose(N0, N1, in, out, comm, KML_FFT_ESTIMATE);

/* compute transforms, in-place, as many times as desired */
kml_fft_execute(plan);

kml_fft_destroy_plan(plan);
kml_fft_mpi_cleanup();

MPI_Finalize();
```

6.4.4.5.7.7 kml_fft(f)_mpi_local_size_3d

描述了本地进程数据的大小和位置，以及计算需要分配的空间。

接口定义

C interface：

ptrdiff_t kml_fft_mpi_local_size_3d(ptrdiff_t n0, ptrdiff_t n1, ptrdiff_t n2, MPI_Comm comm, ptrdiff_t *local_n0, ptrdiff_t *local_0_start);

ptrdiff_t kml_fftf_mpi_local_size_3d(ptrdiff_t n0, ptrdiff_t n1, ptrdiff_t n2, MPI_Comm comm, ptrdiff_t *local_n0, ptrdiff_t *local_0_start);

ptrdiff_t kml_ffth_mpi_local_size_3d(ptrdiff_t n0, ptrdiff_t n1, ptrdiff_t n2, MPI_Comm comm, ptrdiff_t *local_n0, ptrdiff_t *local_0_start);

返回值

函数返回一个ptrdiff_t类型的值，表示要分配的元素的数量（复数，对于复数DFT）。

参数

参数名	数据类型	描述	输入/输出
n0	• ptrdiff_t	待处理数据第1个维度大小，约束：n0 ≥ 1	输入
n1	• ptrdiff_t	待处理数据第2个维度大小，约束：n1 ≥ 1	输入

版权所有 © 华为技术有限公司

参数名	数据类型	描述	输入/输出
n2	• ptrdiff_t	待处理数据第3个维度大小，约束：n2 ≥ 1	输入
comm	• MPI_Comm	MPI通信器的句柄。	输入
local_n0	• ptrdiff_t*	本地进程数据大小。	输出
local_0_start	• ptrdiff_t*	本地进程数据相对全局数据起点偏移量。	输出

依赖

C: "kfft-mpi.h"

示例

见 **kml_fft(f/h)_mpi_plan_dft_3d**的相关示例。

6.4.4.5.7.8 kml_fft(f)_mpi_local_size_3d_transposed

描述了本地进程数据的大小和位置，以及计算需要分配的空间。

接口定义

C interface：

ptrdiff_t kml_fft_mpi_local_size_3d_transposed(ptrdiff_t n0, ptrdiff_t n1, ptrdiff_t n2, MPI_Comm comm, ptrdiff_t *local_n0, ptrdiff_t *local_0_start, ptrdiff_t *local_n1, ptrdiff_t *local_1_start);

ptrdiff_t kml_fftf_mpi_local_size_3d_transposed(ptrdiff_t n0, ptrdiff_t n1, ptrdiff_t n2, MPI_Comm comm, ptrdiff_t *local_n0, ptrdiff_t *local_0_start, ptrdiff_t *local_n1, ptrdiff_t *local_1_start);

ptrdiff_t kml_ffth_mpi_local_size_3d_transposed(ptrdiff_t n0, ptrdiff_t n1, ptrdiff_t n2, MPI_Comm comm, ptrdiff_t *local_n0, ptrdiff_t *local_0_start, ptrdiff_t *local_n1, ptrdiff_t *local_1_start);

返回值

函数返回一个ptrdiff_t类型的值，表示要分配的元素的数量（复数，对于复数DFT）。当前进程应从索引local_0_start开始，存储整个输入数据集的切片的local_n0。也就是说，当前进程应存储local_0_start到local_0_start+local_n0-1的输入数据。当前进程应从索引local_1_start开始，存储整个输出数据集的切片的local_n1。也就是说，当前进程应存储local_1_start到local_1_start+local_n1-1的输出数据。

 版权所有 © 华为技术有限公司

参数

参数名	数据类型	描述	输入/输出
n0	ptrdiff_t	待处理数据第1维度大小，约束：n0 ≥ 1。	输入
n1	ptrdiff_t	待处理数据第2维度大小，约束：n1 ≥ 1。	输入
n2	ptrdiff_t	待处理数据第3维度大小，约束：n2 ≥ 1。	输入
comm	MPI_Comm	MPI通信器的句柄。	输入
local_n0	ptrdiff_t*	本地进程数据大小。	输出
local_0_start	ptrdiff_t*	本地进程数据相对全局数据起点偏移量。	输出
local_n1	ptrdiff_t*	本地进程输出数据大小。	输出
local_1_start	ptrdiff_t*	本地进程输出数据相对全局数据起点偏移量。	输出

依赖

C: "kfft-mpi.h"

示例

```
const ptrdiff_t N0 = 4, N1 = 4, N2 = 4;
kml_fft_plan plan;
ptrdiff_t alloc_local, local_n0, local_0_start, local_n1, local_1_start;

MPI_Init(&argc, &argv);
MPI_Comm comm = MPI_COMM_WORLD;
kml_fft_mpi_init();
double *in = NULL;
double *out = NULL;

/* get local data size and allocate */
alloc_local = kml_fft_mpi_local_size_3d_transposed(N0, N1, N2, comm, &local_n0, &local_0_start,
                                                   &local_n1, &local_1_start);
if (alloc_local == -1) {
    printf("[%s][%d] allocate size fail.\n", __func__, __LINE__);
}
in = (double *)kml_fft_malloc(sizeof(double) * alloc_local * 2);
if (in == NULL) {
    printf("[%s][%d] malloc memory fail.\n", __func__, __LINE__);
}
out = (double *)kml_fft_malloc(sizeof(double) * alloc_local * 2);
if (out == NULL) {
    printf("[%s][%d] malloc memory fail.\n", __func__, __LINE__);
}

/* create plan for in-place forward DFT */
plan = kml_fft_mpi_plan_transpose(N0, N1, in, out, comm, KML_FFT_ESTIMATE);

/* compute transforms, in-place, as many times as desired */
```


版权所有 © 华为技术有限公司

```
kml_fft_execute(plan);

kml_fft_destroy_plan(plan);
kml_fft_mpi_cleanup();

MPI_Finalize();
```

6.4.4.5.7.9 kml_fft(f)_mpi_local_size_many

描述了本地进程数据的大小和位置，以及计算需要分配的空间。

接口定义

C interface：

ptrdiff_t kml_fft_mpi_local_size_many(int rank, const ptrdiff_t *n, ptrdiff_t howmany, ptrdiff_t block0, MPI_Comm comm, ptrdiff_t *local_n0, ptrdiff_t *local_0_start);

ptrdiff_t kml_fftf_mpi_local_size_many(int rank, const ptrdiff_t *n, ptrdiff_t howmany, ptrdiff_t block0, MPI_Comm comm, ptrdiff_t *local_n0, ptrdiff_t *local_0_start);

ptrdiff_t kml_ffth_mpi_local_size_many(int rank, const ptrdiff_t *n, ptrdiff_t howmany, ptrdiff_t block0, MPI_Comm comm, ptrdiff_t *local_n0, ptrdiff_t *local_0_start);

返回值

函数返回一个ptrdiff_t类型的值，表示要分配的元素的数量。

参数

参数名	数据类型	描述	输入/输出
rank	• ptrdiff_t	待处理数据维度数量。	输入
n	• const ptrdiff_t *	n为维度为rank的数组，包含数据序列每一维度的大小，约束：n[i] ≥ 1, for i in 0 to rank - 1。	输入
howmany	• ptrdiff_t	howmany表示指定多少个变换。	输入
block0	• ptrdiff_t	指定block的大小。	输入
comm	• MPI_Comm	MPI通信器的句柄。	输入
sign	• int	与创建plan时传入的参数匹配。	输入
flags	• unsigned int	与创建plan时传入的参数匹配。	输入
local_n0	• ptrdiff_t*	本地进程数据大小。	输出
local_0_start	• ptrdiff_t*	本地进程数据相对全局数据起点偏移量。	输出

版权所有 © 华为技术有限公司

依赖

C: "kfft-mpi.h"

示例

见 **kml_fft(f/h)_mpi_plan_many_dft**的相关示例。

6.4.4.5.7.10 kml_fft(f)_mpi_local_size_many_transposed

描述了本地进程数据的大小和位置，以及计算需要分配的空间。

接口定义

C interface：

ptrdiff_t kml_fft_mpi_local_size_many_transposed(int rank, const ptrdiff_t *n, ptrdiff_t howmany, MPI_Comm comm, ptrdiff_t *local_n0, ptrdiff_t *local_0_start, ptrdiff_t *local_n1, ptrdiff_t *local_1_start);

ptrdiff_t kml_fftf_mpi_local_size_many_transposed(int rank, const ptrdiff_t *n, ptrdiff_t howmany, MPI_Comm comm, ptrdiff_t *local_n0, ptrdiff_t *local_0_start, ptrdiff_t *local_n1, ptrdiff_t *local_1_start);

返回值

函数返回一个ptrdiff_t类型的值，表示要分配的元素的数量。

参数

参数名	数据类型	描述	输入/输出
rank	• int	待处理数据维度数量，约束：rank ≥ 1	输入
n	• n	n为维度为rank的数组，包含数据序列每一维度的大小，约束：n[i] ≥ 1, for i in 0 to rank - 1。	输入
howmany	• ptrdiff_t	howmany表示指定多少个变换。	输入
comm	• MPI_Comm	MPI通信器的句柄。	输入
local_n0	• ptrdiff_t*	本地进程数据大小。	输出
local_0_start	• ptrdiff_t*	本地进程数据相对全局数据起点偏移量。	输出
local_n1	• ptrdiff_t*	本地进程输出数据大小。	输出
local_1_start	• ptrdiff_t*	本地进程输出数据相对全局数据起点偏移量。	输出

版权所有 © 华为技术有限公司

依赖

C: "kfft-mpi.h"

示例

```
const ptrdiff_t N0 = 4, N1 = 4;
kml_fft_plan plan;
ptrdiff_t alloc_local, local_n0, local_0_start, local_n1, local_1_start;

MPI_Init(&argc, &argv);
MPI_Comm comm = MPI_COMM_WORLD;
kml_fft_mpi_init();
ptrdiff_t *n = (ptrdiff_t *)kml_fft_malloc(sizeof(ptrdiff_t) * 2);
if (n == NULL) {
    printf("[%s][%d] malloc memory fail.\n", __func__, __LINE__);
}

n[0] = N0;
n[1] = N1;
ptrdiff_t howmany = 1;
ptrdiff_t block0 = 0;
ptrdiff_t block1 = 0;
double *in = NULL;
double *out = NULL;

/* get local data size and allocate */
alloc_local = kml_fft_mpi_local_size_many_transposed(2, n, howmany, comm,
                                        &local_n0, &local_0_start,
                                        &local_n1, &local_1_start);
if (alloc_local == -1) {
    printf("[%s][%d] allocate size fail.\n", __func__, __LINE__);
}
in = (double *)kml_fft_malloc(sizeof(double) * alloc_local * 2);
if (in == NULL) {
    printf("[%s][%d] malloc memory fail.\n", __func__, __LINE__);
}
out = (double *)kml_fft_malloc(sizeof(double) * alloc_local * 2);
if (out == NULL) {
    printf("[%s][%d] malloc memory fail.\n", __func__, __LINE__);
}

/* create plan for in-place forward DFT */
plan = kml_fft_mpi_plan_many_transpose(n[0], n[1], howmany, block0, block1, in, out, comm,
KML_FFT_ESTIMATE);

/* compute transforms, in-place, as many times as desired */
kml_fft_execute(plan);

kml_fft_destroy_plan(plan);
kml_fft_mpi_cleanup();

MPI_Finalize();
```

6.4.4.5.7.11 kml_fft(f)_mpi_local_size_ext

描述了本地进程数据的大小和位置，以及计算所需分配的空间。

接口定义

C interface：

ptrdiff_t kml_fft_mpi_local_size_ext(int rnk, const ptrdiff_t *n, MPI_Comm comm, enum SCALFFT_DECOMPOSE_TYPE_E decomp_type, ptrdiff_t *low, ptrdiff_t *high);

 版权所有 © 华为技术有限公司

ptrdiff_t kml_fftf_mpi_local_size_ext(int rnk, const ptrdiff_t *n, MPI_Comm comm, enum SCALFFT_DECOMPOSE_TYPE_E decomp_type, ptrdiff_t *low, ptrdiff_t *high);

返回值

函数返回一个ptrdiff_t类型的值，表示要分配的buffer的元素个数，成功则返回值大于等于0，失败返回-1。

参数

参数名	数据类型	描述	输入/输出
rnk	int	待处理数据维度数量	输入
n	const ptrdiff_t *	n是维度为rnk的数组，包含数据序列每一维度的大小，约束：n[i] ≥ 1, for i in 0 to rnk - 1	输入
comm	MPI_Comm	通信域	输入
decomp_type	enum SCALFFT_DECOMPOSE_TYPE_E	分解算法，SCALFFT_DECOMPOSE_TYPE_SLAB，SCALFFT_DECOMPOSE_TYPE_PENCIL，SCALFFT_DECOMPOSE_TYPE_BRICK	输入
low	ptrdiff_t *	本地数据起点	输出
high	ptrdiff_t *	本地数据终点	输出

依赖

C: "kfft-mpi.h"

示例

C interface：见 **kml_fft(f)_mpi_plan_create**。

6.4.4.5.7.12 kml_fft(f)_mpi_local_size_transposed_ext

描述了使能box_order后，本地进程数据的大小和位置，以及计算所需分配的空间。

接口定义

C interface：

ptrdiff_t kml_fft_mpi_local_size_transposed_ext(int rank, const ptrdiff_t *n, MPI_Comm comm, enum SCALFFT_DECOMPOSE_TYPE_E decomp_type, const int *order, ptrdiff_t *low, ptrdiff_t *high);

版权所有 © 华为技术有限公司

ptrdiff_t kml_fftf_mpi_local_size_transposed_ext(int rank, const ptrdiff_t *n, MPI_Comm comm, enum SCALFFT_DECOMPOSE_TYPE_E decomp_type, const int *order, ptrdiff_t *low, ptrdiff_t *high);

返回值

函数返回一个ptrdiff_t类型的值，表示要分配的buffer元素个数，成功则返回值大于等于0，失败返回-1。

参数

表 6-28 参数定义

参数名	数据类型	描述	输入/输出
rank	int	进程号	输入
n	ptrdiff_t *	待处理数据第1个维度大小，约束：n0 ≥ 1	输入
comm	MPI_Comm	通信域	输入
decomp_type	enum SCALFFT_DECOMPOSE_TYPE_E	分解算法，SCALFFT_DECOMPOSE_TYPE_SLAB，SCALFFT_DECOMPOSE_TYPE_PENCIL，SCALFFT_DECOMPOSE_TYPE_BRICK	输入
order	int *	是长度为3的一维数组，输入数据维度的顺序，取值为0、1、2的任意顺序组合，输入NULL，默认order为0、1、2。	输入
low	ptrdiff_t *	本地数据起点	输出
high	ptrdiff_t *	本地数据终点	输出

依赖

C: "kfft-mpi.h"

示例

C interface：

```
const int n0 = 4, n1 = 4, n2 = 4;
kml_fft_plan plan;
int provided;
MPI_Init_thread(&argc, &argv, MPI_THREAD_MULTIPLE, &provided);
MPI_Comm comm = MPI_COMM_WORLD;
kml_fft_complex *in = NULL;
kml_fft_complex *out = NULL;
```

 版权所有 © 华为技术有限公司

```
    /* get local data size and allocate */
    ptrdiff_t in_low[3] = {0};
    ptrdiff_t in_high[3] = {0};
    ptrdiff_t out_low[3] = {0};
    ptrdiff_t out_high[3] = {0};
    ptrdiff_t n[3] = {n0, n1, n2};
    int in_order[3] = {0, 1, 2};
    int out_order[3] = {1, 2, 0};
    ptrdiff_t in_alloc_local = kml_fft_mpi_local_size_transposed_ext(3, n, comm,
SCALFFT_DECOMPOSE_TYPE_PENCIL, in_order, in_low, in_high);
    if (in_alloc_local == -1) {
        printf("[%s][%d] allocate size fail.\n", __func__, __LINE__);
    }
    ptrdiff_t out_alloc_local = kml_fft_mpi_local_size_transposed_ext(3, n, comm,
SCALFFT_DECOMPOSE_TYPE_PENCIL, out_order, out_low, out_high);
    if (out_alloc_local == -1) {
        printf("[%s][%d] allocate size fail.\n", __func__, __LINE__);
    }
    in = (kml_fft_complex *)kml_fft_malloc(sizeof(kml_fft_complex) * in_alloc_local);
    if (in == NULL) {
        printf("[%s][%d] malloc memory fail.\n", __func__, __LINE__);
    }
    out = (kml_fft_complex *)kml_fft_malloc(sizeof(kml_fft_complex) * out_alloc_local);
    if (out == NULL) {
        printf("[%s][%d] malloc memory fail.\n", __func__, __LINE__);
    }

    /* create plan */
    int in_low_int[3] = {in_low[0], in_low[1], in_low[2]};
    int in_high_int[3] = {in_high[0], in_high[1], in_high[2]};
    int out_low_int[3] = {out_low[0], out_low[1], out_low[2]};
    int out_high_int[3] = {out_high[0], out_high[1], out_high[2]};
    kml_fft_mpi_options options = {
            .a2a_algo    = A2A_ALGO_AUTO_TUNING,
            .decomp_type = SCALFFT_DECOMPOSE_TYPE_PENCIL
    };
    plan = kml_fft_mpi_plan_create(BACKEND_KFFT, in_low_int, in_high_int, in_order, out_low_int,
out_high_int, out_order, comm, options);

    /* execute plan */
    int scale = 0;
    kml_fft_mpi_execute_dft_ext(plan, in, out, scale, KML_FFT_FORWARD);
    kml_fft_mpi_execute_dft_ext(plan, out, in, scale, KML_FFT_BACKWARD);
    // kml_fft_mpi_forward_c2c(plan, in, out, scale);
    // kml_fft_mpi_backward_c2c(plan, out, in, scale);

    kml_fft_destroy_plan_ext(plan);

    kml_fft_free(in);
    kml_fft_free(out);
    MPI_Finalize();
```

6.4.4.5.7.13 kml_fft(f)_mpi_local_size_3d_ext

描述了本地进程数据的大小和位置，以及计算所需分配的空间。

接口定义

C interface：

ptrdiff_t kml_fft_mpi_local_size_3d_ext(ptrdiff_t n0, ptrdiff_t n1, ptrdiff_t n2, MPI_Comm comm, enum SCALFFT_DECOMPOSE_TYPE_E decomp_type, ptrdiff_t *low, ptrdiff_t *high);

版权所有 © 华为技术有限公司

ptrdiff_t kml_fftf_mpi_local_size_3d_ext(ptrdiff_t n0, ptrdiff_t n1, ptrdiff_t n2, MPI_Comm comm, enum SCALFFT_DECOMPOSE_TYPE_E decomp_type, ptrdiff_t *low, ptrdiff_t *high);

返回值

函数返回一个ptrdiff_t类型的值，表示要分配的buffer元素个数，成功则返回值大于等于0，失败返回-1。

参数

参数名	数据类型	描述	输入/输出
n0	ptrdiff_t *	待处理数据第1个维度大小，约束：n0 ≥ 1	输入
n1	ptrdiff_t *	待处理数据第2个维度大小，约束：n1 ≥ 1	输入
n2	ptrdiff_t *	待处理数据第3个维度大小，约束：n2 ≥ 1	输入
comm	MPI_Comm	通信域	输入
decomp_type	enum SCALFFT_DECOMPOSE_TYPE_E	分解算法，SCALFFT_DECOMPOSE_TYPE_SLAB，SCALFFT_DECOMPOSE_TYPE_PENCIL，SCALFFT_DECOMPOSE_TYPE_BRICK	输入
low	ptrdiff_t *	本地数据起点	输出
high	ptrdiff_t *	本地数据终点	输出

依赖

C: "kfft-mpi.h"

示例

C interface：见 **kml_fft(f)_mpi_plan_dft_3d_ext**。

6.4.4.5.7.14 kml_fft(f)_mpi_local_size_3d_transposed_ext

描述了使能box_order后，本地进程数据的大小和位置，以及计算所需分配的空间。

接口定义

C interface：

 版权所有 © 华为技术有限公司

ptrdiff_t kml_fft_mpi_local_size_3d_transposed_ext(ptrdiff_t n0, ptrdiff_t n1, ptrdiff_t n2, MPI_Comm comm, enum SCALFFT_DECOMPOSE_TYPE_E decomp_type, const int *order, ptrdiff_t *low, ptrdiff_t *high);

ptrdiff_t kml_fftf_mpi_local_size_3d_transposed_ext(ptrdiff_t n0, ptrdiff_t n1, ptrdiff_t n2, MPI_Comm comm, enum SCALFFT_DECOMPOSE_TYPE_E decomp_type, const int *order, ptrdiff_t *low, ptrdiff_t *high);

返回值

函数返回一个ptrdiff_t类型的值，表示要分配的buffer元素个数，成功则返回值大于等于0，失败返回-1。

参数

参数名	数据类型	描述	输入/输出
n0	ptrdiff_t *	待处理数据第1个维度大小，约束：n0 ≥ 1	输入
n1	ptrdiff_t *	待处理数据第2个维度大小，约束：n1 ≥ 1	输入
n2	ptrdiff_t *	待处理数据第3个维度大小，约束：n2 ≥ 1	输入
comm	MPI_Comm	通信域	输入
decomp_type	enum SCALFFT_DECOMPOSE_TYPE_E	分解算法，SCALFFT_DECOMPOSE_TYPE_SLAB，SCALFFT_DECOMPOSE_TYPE_PENCIL，SCALFFT_DECOMPOSE_TYPE_BRICK	输入
order	int *	是长度为3的一维数组，输入数据维度的顺序，取值为0、1、2的任意顺序组合，输入NULL，默认order为0、1、2。	输入
low	ptrdiff_t *	本地数据起点	输出
high	ptrdiff_t *	本地数据终点	输出

依赖

C: "kfft-mpi.h"

示例

C interface：

 版权所有 © 华为技术有限公司

```
const int n0 = 4, n1 = 4, n2 = 4;
    kml_fft_plan plan;
    int provided;
    MPI_Init_thread(&argc, &argv, MPI_THREAD_MULTIPLE, &provided);
    MPI_Comm comm = MPI_COMM_WORLD;
    kml_fft_complex *in = NULL;
    kml_fft_complex *out = NULL;

    /* get local data size and allocate */
    ptrdiff_t in_low[3] = {0};
    ptrdiff_t in_high[3] = {0};
    ptrdiff_t out_low[3] = {0};
    ptrdiff_t out_high[3] = {0};
    ptrdiff_t n[3] = {n0, n1, n2};
    int in_order[3] = {0, 1, 2};
    int out_order[3] = {1, 2, 0};
    ptrdiff_t in_alloc_local = kml_fft_mpi_local_size_3d_transposed_ext(n[0], n[1], n[2], comm,
SCALFFT_DECOMPOSE_TYPE_PENCIL, in_order, in_low, in_high);
    if (in_alloc_local == -1) {
        printf("[%s][%d] allocate size fail.\n", __func__, __LINE__);
    }
    ptrdiff_t out_alloc_local = kml_fft_mpi_local_size_3d_transposed_ext(n[0], n[1], n[2], comm,
SCALFFT_DECOMPOSE_TYPE_PENCIL, out_order, out_low, out_high);
    if (out_alloc_local == -1) {
        printf("[%s][%d] allocate size fail.\n", __func__, __LINE__);
    }
    in = (kml_fft_complex *)kml_fft_malloc(sizeof(kml_fft_complex) * in_alloc_local);
    if (in == NULL) {
        printf("[%s][%d] malloc memory fail.\n", __func__, __LINE__);
    }
    out = (kml_fft_complex *)kml_fft_malloc(sizeof(kml_fft_complex) * out_alloc_local);
    if (out == NULL) {
        printf("[%s][%d] malloc memory fail.\n", __func__, __LINE__);
    }

    /* create plan */
    int in_low_int[3] = {in_low[0], in_low[1], in_low[2]};
    int in_high_int[3] = {in_high[0], in_high[1], in_high[2]};
    int out_low_int[3] = {out_low[0], out_low[1], out_low[2]};
    int out_high_int[3] = {out_high[0], out_high[1], out_high[2]};
    kml_fft_mpi_options options = {
            .a2a_algo    = A2A_ALGO_AUTO_TUNING,
            .decomp_type = SCALFFT_DECOMPOSE_TYPE_PENCIL
    };
plan = kml_fft_mpi_plan_create(BACKEND_KFFT, in_low_int, in_high_int, in_order, out_low_int, out_high_int,
out_order, comm, options);

    /* execute plan */
    int scale = 0;
    kml_fft_mpi_execute_dft_ext(plan, in, out, scale, KML_FFT_FORWARD);
    kml_fft_mpi_execute_dft_ext(plan, out, in, scale, KML_FFT_BACKWARD);
    // kml_fft_mpi_forward_c2c(plan, in, out, scale);
    // kml_fft_mpi_backward_c2c(plan, out, in, scale);

    kml_fft_destroy_plan_ext(plan);

    kml_fft_free(in);
    kml_fft_free(out);
    MPI_Finalize();[l1] [l2]
```

6.4.4.5.7.15 kml_fft(f)_mpi_split_world

描述了所有进程数据的大小和位置。

接口定义

C interface：

 版权所有 © 华为技术有限公司

void kml_fft_mpi_split_world(const int *world, const int *proc_grid, int(*box_low)[3], int(*box_high)[3]);

void kml_fftf_mpi_split_world(const int *world, const int *proc_grid, int(*box_low)[3], int(*box_high)[3]);

返回值

无返回值。

参数

参数名	数据类型	描述	输入/输出
world	const int*	n是长度为3的数组，包含各维度的大小；	输入
proc_grid	const int*	proc_grid是长度为3的数组，包含各维度进程分布数量	输入
box_low	int**	所有进程三个维度切分后的起点	输出
box_high	int**	所有进程三个维度切分后的终点	输出

依赖

C: "kfft-mpi.h"

示例

C interface:

```
const int n0 = 32, n1 = 32, n2 = 32;
kml_fft_plan plan;
int provided;
MPI_Init_thread(&argc, &argv, MPI_THREAD_MULTIPLE, &provided);
MPI_Comm comm = MPI_COMM_WORLD;
int size;
MPI_Comm_size(comm, &size);

/* get local data size and allocate */
int box_low[size][3];
int box_high[size][3];
int world[3] = {n0, n1, n2};
int proc_grid[3] = {1, 1, size};
kml_fft_mpi_split_world(world, proc_grid, box_low, box_high);
MPI_Finalize();
```

6.4.4.5.8 MPI transposition

KML-FFT提供了一些例程，用于规划一个分布式n0×n1实数数组的转置，或者一个具有指定block大小的howmany-tuples实数数组 。

版权所有 © 华为技术有限公司

6.4.4.5.8.1 kml_fft_mpi_plan_transpose

假设有一个n0×n1的数组，以行为主顺序，在n0维上分布块。将其转换为在n1维上块分布的n1×n0数组 。

接口定义

C interface：

kml_fft_plan kml_fft_mpi_plan_transpose (ptrdiff_t n0, ptrdiff_t n1, double *in, double *out, MPI_Comm comm, unsigned flags);

kml_fft_plan kml_fftf_mpi_plan_transpose(ptrdiff_t n0, ptrdiff_t n1, float *in, float *out, MPI_Comm comm, unsigned flags);

返回值

函数返回一个kml_fft(f)_plan类型的结构体指针。将该对象作为参数传入 **kml_fft(f/h)_execute**函数，将对当前提供的输入in和输出out执行FFT变换。

如果函数返回非空指针，则表示plan创建成功，否则表示创建失败。

参数

参数名	数据类型	描述	输入/输出
n0	ptrdiff_t	输入FFT序列第一维的大小，约束：n0 ≥ 1。	输入
n1	ptrdiff_t	输入FFT序列第二维的大小，约束：n1 ≥ 1。	输入
in	• 双精度：double* • 单精度：float*	输入需要变换的数据。	输入
out	• 双精度：double* • 单精度：float*	输出快速傅里叶变换后的数据。	输出
comm	MPI_Comm	MPI通信器的句柄。	输入
flags	unsigned int	对于转置plan，这些标志所指示的分别是输出和/或输入被局部转置。	输入

依赖

C: "kfft-mpi.h"

示例

见 **kml_fft(f/h)_mpi_local_size_transposed**的相关用例。

版权所有 © 华为技术有限公司

6.4.4.5.8.2 kml_fft_mpi_plan_many_transpose

假设有一个n0×n1的数组，以行为主顺序，在n0维上分布块。将其转换为在n1维上块分布的n1×n0数组 。更一般地说，可以执行数字元组的转置，输入和输出的块大小由用户指定。

接口定义

C interface：

kml_fft_plan kml_fft_mpi_plan_many_transpose(ptrdiff_t n0, ptrdiff_t n1, ptrdiff_t howmany，ptrdiff_t block0, ptrdiff_t block1, double *in, double *out, MPI_Comm comm, unsigned flags);

kml_fft_plan kml_fftf_mpi_plan_many_transpose(ptrdiff_t n0, ptrdiff_t n1, ptrdiff_t howmany，ptrdiff_t block0, ptrdiff_t block1, float *in, float *out, MPI_Comm comm, unsigned flags);

返回值

函数返回一个kml_fft(f)_plan类型的结构体指针。将该对象作为参数传入 **kml_fft(f/h)_execute**函数中，将对当前提供的输入in和输出out执行FFT变换。

如果函数返回非空指针，则表示plan创建成功，否则表示创建失败。

参数

参数名	数据类型	描述	输入/输出
n0	ptrdiff_t	输入FFT序列第一维的大小，约束：n0 ≥ 1。	输入
n1	ptrdiff_t	输入FFT序列第二维的大小，约束：n1 ≥ 1。	输入
howmany	int	howmany表示指定多少个变换。	输入
block0	ptrdiff_t	n0维度上的block大小。	输入
block1	ptrdiff_t	n1维度上的block大小。	输入
in	• 双精度：double* • 单精度：float*	输入需要变换的数据。	输入
out	• 双精度：double* • 单精度：float*	输出快速傅里叶变换后的数据。	输出
comm	MPI_Comm	MPI通信器的句柄。	输入
flags	unsigned int	对于转置plan，这些标志所指示的分别是输出和/或输入被局部转置。	输入

版权所有 © 华为技术有限公司

依赖

C: "kfft-mpi.h"

示例

见 **kml_fft(f)_mpi_local_size_many_transposed**的相关用例。

6.4.4.5.9 扩展接口环境变量

6.4.4.5.9.1 SCALFFT_PROC_GRID

设置三维进程排布。

示例

当总进程数为15时，可以通过设置环境变量“-x SCALFFT_PROC_GRID=1,3,5”，指定扩展接口计算3D FFT时的三维进程排布为1*3*5。

6.4.4.5.9.2 SCALFFT_TILE_COUNT

设置多线程计算3D FFT时矩阵的最大切分数。

示例

通过设置环境变量“-x SCALFFT_TILE_COUNT=5”，指定扩展接口计算3D FFT时最多将矩阵切为5份。

6.4.4.5.9.3 SCALFFT_DECOMPOSE_TYPE

计算3D FFT时，设置三维矩阵的分解方式。

可选值

表 6-29 参数定义

参数名	数值	描述
SCALFFT_DECOMPOSE_TYPE_SLAB	0	Slab分解。
SCALFFT_DECOMPOSE_TYPE_PENCIL	1	Pencil分解。
SCALFFT_DECOMPOSE_TYPE_BRICK	2	Brick分解，暂不支持。

示例

通过设置环境变量“-x SCALFFT_DECOMPOSE_TYPE=1”，指定扩展接口计算3D FFT时的分解方式为Pencil分解。

6.4.4.5.9.4 SCALFFT_PRE_A2A_ALGO

设置计算3D FFT时pre阶段的转置算法。

版权所有 © 华为技术有限公司

可选值

表 6-30 参数定义

参数名	数值	描述
A2A_ALGO_AUTO_TUNING	0	库根据进程数和输入矩阵的大小自动选择最优转置算法。
A2A_ALGO_COLL_TASK_ALLTOALLV	1	使用MPI A2A接口的task化转置算法。
A2A_ALGO_P2P_TASK_LINEAR	2	使用linear A2A的task化转置算法。
A2A_ALGO_P2P_TASK_PAIRWISE	3	使用pairwise A2A的task化转置算法。
A2A_ALGO_P2P_PIPELINE_LINEAR	4	pipeline转置算法。
A2A_ALGO_P2P_LAMMPS	5	Lammps原生集合通信转置算法。
A2A_ALGO_P2P_PAIRWISE_WO_PACK	6	使用pairwise A2A，无pack流程的转置算法，当前仅支持slab分解。
A2A_ALGO_P2P_LINEAR_WO_PACK	7	使用linear A2A，无pack流程的转置算法，当前仅支持slab分解。
A2A_ALGO_P2P_PIPELINE_INTRA_NODE	8	单节点，仅使用pack、unpack的pipeline转置算法。

示例

通过设置环境变量“-x SCALFFT_PRE_A2A_ALGO=2”，指定扩展接口计算3D FFT时pre阶段的转置算法为linear算法。

6.4.4.5.9.5 SCALFFT_MID_A2A_ALGO

设置计算3D FFT时mid阶段的转置算法。

可选值

请参见 **SCALFFT_PRE_A2A_ALGO**可选值。

示例

- 通过设置环境变量“-x SCALFFT_MID_A2A_ALGO=2”,指定扩展接口计算3D FFT时所有mid阶段的转置均采用linear算法。
- 通过设置环境变量“-x SCALFFT_MID_A2A_ALGO=2,4”，指定扩展接口计算3D FFT时mid阶段的两次转置分别采用linear和pipeline算法。
- 通过设置环境变量“-x SCALFFT_MID_A2A_ALGO=2,3,4”,指定扩展接口计算3D FFT时mid阶段的三次转置分别为linear、pairwise和pipeline算法。

 版权所有 © 华为技术有限公司

6.4.4.5.9.6 SCALFFT_POST_A2A_ALGO

设置计算3D FFT时post阶段的转置算法。

可选值

请参见 **SCALFFT_PRE_A2A_ALGO**可选值。

示例

通过设置环境变量“-x SCALFFT_POST_A2A_ALGO=2”，指定扩展接口计算3D FFT时post阶段的转置算法为linear算法。

6.4.4.6 公用函数定义

6.4.4.6.1 内存函数

6.4.4.6.1.1 调用约束

内存函数的调用约束：plan使用完毕后需要通过调用 **kml_fft(f/h)_destroy_plan**函数来释放； **kml_fft(f/h)_malloc**函数所申请的内存空间只能通过调用 **kml_fft(f/h)_free**函数释放。

6.4.4.6.1.2 kml_fft(f/h)_malloc

kml_fft(f/h)_malloc用于分配所需的内存空间。所申请的内存空间只能通过调用kml_fft(f/h)_free函数释放。

接口定义

C interface：

void* kml_fft_malloc(size_t n);

void* kml_fftf_malloc(size_t n);

void* kml_ffth_malloc(size_t n);

Fortran interface：

RES = KML_FFT_MALLOC(N);

RES = KML_FFTF_MALLOC(N);

RES = KML_FFTH_MALLOC(N);

返回值

函数返回一个void*类型的指针，指向已分配的内存块

 版权所有 © 华为技术有限公司

参数

参数名	数据类型	描述	输入/输出
n	size_t	需要申请的内存大小（以bytes为单位）。	输入

依赖

C: "kfft.h"

示例

C interface：

```
int N = 16;
kml_fft_complex *in;
kml_fft_complex *out;
kml_fft_plan plan;
in = (kml_fft_complex*) kml_fft_malloc(sizeof(kml_fft_complex) * N);
out = (kml_fft_complex*) kml_fft_malloc(sizeof(kml_fft_complex) * N);
for (int i = 0;i < N; i++){
    in[i][0] = (double) i;
    in[i][1] = (double) i;
}

plan = kml_fft_plan_dft_1d(N, in , out, KML_FFT_FORWARD, KML_FFT_ESTIMATE);
kml_fft_execute_dft(plan, in, out);

kml_fft_destroy_plan(plan);
kml_fft_free(in);
kml_fft_free(out);

/*
 * out = {{1.200000e+02, 1.200000e+02}, {-4.821872e+01, 3.221872e+01},
 *        {-2.731371e+01, 1.131371e+01}, {-1.997285e+01, 3.972846e+00},
 *        {-1.600000e+01, 0.000000e+00}, {-1.334543e+01, -2.654571e+00},
 *        {-1.131371e+01, -4.686292e+00}, {-9.591299e+00, -6.408701e+00},
 *        {-8.000000e+00, -8.000000e+00}, {-6.408701e+00, -9.591299e+00},
 *        {-4.686292e+00, -1.131371e+01}, {-2.654571e+00, -1.334543e+01},
 *        {0.000000e+00, -1.600000e+01}, {3.972846e+00, -1.997285e+01},
 *        {1.131371e+01, -2.731371e+01}, {3.221872e+01, -4.821872e+01}}
 */
```

Fortran interface：

```
INTEGER(C_INT) :: N = 16
TYPE(KML_FFT_COMPLEX), POINTER :: IN, OUT
TYPE(C_PTR) :: PIN, POUT
TYPE(C_PTR) :: PLAN
INTEGER(C_SIZE_T) :: SIZE
SIZE = 16 * N
PIN = KML_FFT_MALLOC(SIZE)
POUT = KML_FFT_MALLOC(SIZE)
CALL C_F_POINTER(PIN, IN, SHAPE=[N])
CALL C_F_POINTER(POUT, OUT, SHAPE=[N])
INTEGER :: I = 1
DO WHILE(I <= N)
    IN(I)%R = I
    IN(I)%I = I
    I = I + 1
END DO
```

 版权所有 © 华为技术有限公司

```
PLAN = KML_FFT_PLAN_DFT_1D(N, IN , OUT, KML_FFT_FORWARD, KML_FFT_ESTIMATE)
CALL KML_FFT_EXECUTE(PLAN, IN, OUT)

CALL KML_FFT_DESTROY_PLAN(PLAN)
CALL KML_FFT_FREE(PIN)
CALL KML_FFT_FREE(POUT)

!
! OUT = \1.200000E+02, 1.200000E+0, -4.821872E+01, 3.221872E+01,
!       -2.731371E+01, 1.131371E+01, -1.997285E+01, 3.972846E+00,
!       -1.600000E+01, 0.000000E+00, -1.334543E+01, -2.654571E+00,
!       -1.131371E+01, -4.686292E+00, -9.591299E+00, -6.408701E+00,
!       -8.000000E+00, -8.000000E+00, -6.408701E+00, -9.591299E+00,
!       -4.686292E+00, -1.131371E+01, -2.654571E+00, -1.334543E+01,
!       0.000000E+00, -1.600000E+01, 3.972846E+00, -1.997285E+01,
!       1.131371E+01, -2.731371E+01, 3.221872E+01, -4.821872E+01/
!
```

6.4.4.6.1.3 kml_fft(f/h)_free

kml_fft(f/h)_free释放由kml_fft(f/h)_malloc函数申请的内存。

接口定义

C interface：

void kml_fft_free(void *p);

void kml_fftf_free(void *p);

void kml_ffth_free(void *p);

Fortran interface：

CALL KML_FFT_FREE(P);.

CALL KML_FFTF_FREE(P);

CALL KML_FFTH_FREE(P);

参数

参数名	数据类型	描述	输入/输出
p	void*	p是一个指针，指向之前申请的内存块。	输入

依赖

C: "kfft.h"

示例

C interface：

```
int N = 16;
kml_fft_complex *in;
kml_fft_complex *out;
```

 版权所有 © 华为技术有限公司

```
kml_fft_plan plan;
in = (kml_fft_complex*)kml_fft_malloc(sizeof(kml_fft_complex) * N);
out = (kml_fft_complex*)kml_fft_malloc(sizeof(kml_fft_complex) * N);
for (int i = 0; i < N; i++){
    in[i][0] = (double)i;
    in[i][1] = (double)i;
}

plan = kml_fft_plan_dft_1d(N, in , out, KML_FFT_FORWARD, KML_FFT_ESTIMATE);
kml_fft_execute(plan);

kml_fft_destroy_plan(plan);
kml_fft_free(in);
kml_fft_free(out);

/*
 * out = {{1.200000e+02, 1.200000e+02}, {-4.821872e+01, 3.221872e+01},
 *        {-2.731371e+01, 1.131371e+01}, {-1.997285e+01, 3.972846e+00},
 *        {-1.600000e+01, 0.000000e+00}, {-1.334543e+01, -2.654571e+00},
 *        {-1.131371e+01, -4.686292e+00}, {-9.591299e+00, -6.408701e+00},
 *        {-8.000000e+00, -8.000000e+00}, {-6.408701e+00, -9.591299e+00},
 *        {-4.686292e+00, -1.131371e+01}, {-2.654571e+00, -1.334543e+01},
 *        {0.000000e+00, -1.600000e+01}, {3.972846e+00, -1.997285e+01},
 *        {1.131371e+01, -2.731371e+01}, {3.221872e+01, -4.821872e+01}}
 */
```

Fortran interface:

```
INTEGER(C_INT) :: N = 16
TYPE(KML_FFT_COMPLEX), POINTER :: IN, OUT
TYPE(C_PTR) :: PIN, POUT
TYPE(C_PTR) :: PLAN
INTEGER(C_SIZE_T) :: SIZE
SIZE = 16 * N
PIN = KML_FFT_MALLOC(SIZE)
POUT = KML_FFT_MALLOC(SIZE)
CALL C_F_POINTER(PIN, IN, SHAPE=[N])
CALL C_F_POINTER(POUT, OUT, SHAPE=[N])
INTEGER :: I = 1
DO WHILE(I <= N)
    IN(I)%R = I
    IN(I)%I = I
    I = I + 1
END DO
PLAN = KML_FFT_PLAN_DFT_1D(N, IN , OUT, KML_FFT_FORWARD, KML_FFT_ESTIMATE)
CALL KML_FFT_EXECUTE(PLAN, IN, OUT)

CALL KML_FFT_DESTROY_PLAN(PLAN)
CALL KML_FFT_FREE(PIN)
CALL KML_FFT_FREE(POUT)

!
! OUT = \1.200000E+02, 1.200000E+0, -4.821872E+01, 3.221872E+01,
!       -2.731371E+01, 1.131371E+01, -1.997285E+01, 3.972846E+00,
!       -1.600000E+01, 0.000000E+00, -1.334543E+01, -2.654571E+00,
!       -1.131371E+01, -4.686292E+00, -9.591299E+00, -6.408701E+00,
!       -8.000000E+00, -8.000000E+00, -6.408701E+00, -9.591299E+00,
!       -4.686292E+00, -1.131371E+01, -2.654571E+00, -1.334543E+01,
!       0.000000E+00, -1.600000E+01, 3.972846E+00, -1.997285E+01,
!       1.131371E+01, -2.731371E+01, 3.221872E+01, -4.821872E+01/
!
```

6.4.4.6.1.4 kml_fft(f/h)_destroy_plan

kml_fft(f/h)_destroy_plan释放FFT变换plan的所有内存。调用此函数之后，此plan不再可用。

 版权所有 © 华为技术有限公司

接口定义

C interface：

void kml_fft_destroy_plan(kml_fft_plan p);

void kml_fftf_destroy_plan(kml_fftf_plan p);

void kml_ffth_destroy_plan(kml_ffth_plan p);

Fortran interface：

CALL KML_FFT_DESTROY_PLAN(P);

CALL KML_FFTF_DESTROY_PLAN(P);

CALL KML_FFTH_DESTROY_PLAN(P);

参数

参数名	数据类型	描述	输入/输出
p	• 双精度：const kml_fft_plan • 单精度：const kml_fftf_plan • 半精度：const kml_ffth_plan	待释放内存的FFT变换plan。	输入

依赖

C: "kfft.h"

示例

C interface：

```
int N = 16;
kml_fft_complex *in;
kml_fft_complex *out;
kml_fft_plan plan;
in = (kml_fft_complex*)kml_fft_malloc(sizeof(kml_fft_complex) * N);
out = (kml_fft_complex*)kml_fft_malloc(sizeof(kml_fft_complex) * N);
for (int i = 0; i < N; i++){
    in[i][0] = (double)i;
    in[i][1] = (double)i;
}

plan = kml_fft_plan_dft_1d(N, in , out, KML_FFT_FORWARD, KML_FFT_ESTIMATE);
kml_fft_execute(plan);

kml_fft_destroy_plan(plan);
kml_fft_free(in);
kml_fft_free(out);

/*
 * out = {{1.200000e+02, 1.200000e+02}, {-4.821872e+01, 3.221872e+01},
 *        {-2.731371e+01, 1.131371e+01}, {-1.997285e+01, 3.972846e+00},
```


版权所有 © 华为技术有限公司

```
 *       {-1.600000e+01, 0.000000e+00}, {-1.334543e+01, -2.654571e+00},
 *       {-1.131371e+01, -4.686292e+00}, {-9.591299e+00, -6.408701e+00},
 *       {-8.000000e+00, -8.000000e+00}, {-6.408701e+00, -9.591299e+00},
 *       {-4.686292e+00, -1.131371e+01}, {-2.654571e+00, -1.334543e+01},
 *       {0.000000e+00, -1.600000e+01}, {3.972846e+00, -1.997285e+01},
 *       {1.131371e+01, -2.731371e+01}, {3.221872e+01, -4.821872e+01}}
 */
```

Fortran interface:

```
INTEGER(C_INT) :: N = 16
   TYPE(KML_FFT_COMPLEX), POINTER :: IN, OUT
   TYPE(C_PTR) :: PIN, POUT
   TYPE(C_PTR) :: PLAN
   INTEGER(C_SIZE_T) :: SIZE
   SIZE = 16 * N
   PIN = KML_FFT_MALLOC(SIZE)
   POUT = KML_FFT_MALLOC(SIZE)
   CALL C_F_POINTER(PIN, IN, SHAPE=[N])
   CALL C_F_POINTER(POUT, OUT, SHAPE=[N])
   INTEGER :: I = 1
   DO WHILE(I <= N)
      IN(I)%R = I
      IN(I)%I = I
      I = I + 1
   END DO
   PLAN = KML_FFT_PLAN_DFT_1D(N, IN , OUT, KML_FFT_FORWARD, KML_FFT_ESTIMATE)
   CALL KML_FFT_EXECUTE(PLAN, IN, OUT)

   CALL KML_FFT_DESTROY_PLAN(PLAN)
   CALL KML_FFT_FREE(PIN)
   CALL KML_FFT_FREE(POUT)

   !
   ! OUT = \1.200000E+02, 1.200000E+0, -4.821872E+01, 3.221872E+01,
   !       -2.731371E+01, 1.131371E+01, -1.997285E+01, 3.972846E+00,
   !       -1.600000E+01, 0.000000E+00, -1.334543E+01, -2.654571E+00,
   !       -1.131371E+01, -4.686292E+00, -9.591299E+00, -6.408701E+00,
   !       -8.000000E+00, -8.000000E+00, -6.408701E+00, -9.591299E+00,
   !       -4.686292E+00, -1.131371E+01, -2.654571E+00, -1.334543E+01,
   !       0.000000E+00, -1.600000E+01, 3.972846E+00, -1.997285E+01,
   !       1.131371E+01, -2.731371E+01, 3.221872E+01, -4.821872E+01/
   !
```

6.4.4.6.2 线程函数

6.4.4.6.2.1 调用约束

线程函数的调用顺序约束：

1. 首先调用 **kml_fft(f/h)_init_threads**初始化线程的使用。
2. 然后调用 **kml_fft(f/h)_plan_with_nthreads**指定线程数。
3. 最后调用 **kml_fft(f/h)_cleanup_threads**释放多线程框架相关的资源。

6.4.4.6.2.2 kml_fft(f/h)_init_threads

kml_fft(f/h)_init_threads用于初始化线程的使用。（此函数不可重入。）

接口定义

C interface:

int kml_fft_init_threads(void);

 版权所有 © 华为技术有限公司

int kml_fftf_init_threads(void);

int kml_ffth_init_threads(void);

Fortran interface：

RES = KML_FFT_INIT_THREADS();

RES = KML_FFTF_INIT_THREADS();

RES = KML_FFTH_INIT_THREADS();

返回值

kml_fft(f)_init_threads函数执行成功返回1，否则返回0。

参数

无

依赖

C: "kfft.h"

示例

C interface：

```
kml_fft_init_threads();
int nthreads = 4;
kml_fft_plan_with_nthreads(nthreads);
int N = 16;
kml_fft_complex *in;
kml_fft_complex *out;
kml_fft_plan plan;
in = (kml_fft_complex*)kml_fft_malloc(sizeof(kml_fft_complex) * N);
out = (kml_fft_complex*)kml_fft_malloc(sizeof(kml_fft_complex) * N);
for (int i = 0; i < N; i++){
    in[i][0] = (double)i;
    in[i][1] = (double)i;
}

plan = kml_fft_plan_dft_1d(N, in , out, KML_FFT_FORWARD, KML_FFT_ESTIMATE);
kml_fft_execute(plan);
kml_fft_destroy_plan(plan);
kml_fft_cleanup_threads();
kml_fft_free(in);
kml_fft_free(out);

/*
 * out = {{1.200000e+02, 1.200000e+02}, {-4.821872e+01, 3.221872e+01},
 *        {-2.731371e+01, 1.131371e+01}, {-1.997285e+01, 3.972846e+00},
 *        {-1.600000e+01, 0.000000e+00}, {-1.334543e+01, -2.654571e+00},
 *        {-1.131371e+01, -4.686292e+00}, {-9.591299e+00, -6.408701e+00},
 *        {-8.000000e+00, -8.000000e+00}, {-6.408701e+00, -9.591299e+00},
 *        {-4.686292e+00, -1.131371e+01}, {-2.654571e+00, -1.334543e+01},
 *        {0.000000e+00, -1.600000e+01}, {3.972846e+00, -1.997285e+01},
 *        {1.131371e+01, -2.731371e+01}, {3.221872e+01, -4.821872e+01}}
 */
```

Fortran interface：

```
INTEGER(C_INT) :: STATUS
STATUS = KML_FFT_INIT_THREADS()
```

 版权所有 © 华为技术有限公司

```
INTEGER(C_INT) :: NTHREADS = 4
CALL KML_FFT_PLAN_WITH_NTHREADS(NTHREADS)
INTEGER(C_INT) :: N = 16
TYPE(KML_FFT_COMPLEX), POINTER :: IN, OUT
TYPE(C_PTR) :: PIN, POUT
TYPE(C_PTR) :: PLAN
INTEGER(C_SIZE_T) :: SIZE
SIZE = 16 * N
PIN = KML_FFT_MALLOC(SIZE)
POUT = KML_FFT_MALLOC(SIZE)
CALL C_F_POINTER(PIN, IN, SHAPE=[N])
CALL C_F_POINTER(POUT, OUT, SHAPE=[N])
INTEGER :: I = 1
DO WHILE(I <= N)
   IN(I)%R = I
   IN(I)%I = I
   I = I + 1
END DO
PLAN = KML_FFT_PLAN_DFT_1D(N, IN , OUT, KML_FFT_FORWARD, KML_FFT_ESTIMATE)
CALL KML_FFT_EXECUTE(PLAN, IN, OUT)

CALL KML_FFT_DESTROY_PLAN(PLAN)
CALL KML_FFT_FREE(PIN)
CALL KML_FFT_FREE(POUT)

!
! OUT = \1.200000E+02, 1.200000E+0, -4.821872E+01, 3.221872E+01,
!       -2.731371E+01, 1.131371E+01, -1.997285E+01, 3.972846E+00,
!       -1.600000E+01, 0.000000E+00, -1.334543E+01, -2.654571E+00,
!       -1.131371E+01, -4.686292E+00, -9.591299E+00, -6.408701E+00,
!       -8.000000E+00, -8.000000E+00, -6.408701E+00, -9.591299E+00,
!       -4.686292E+00, -1.131371E+01, -2.654571E+00, -1.334543E+01,
!       0.000000E+00, -1.600000E+01, 3.972846E+00, -1.997285E+01,
!       1.131371E+01, -2.731371E+01, 3.221872E+01, -4.821872E+01/
!
```

6.4.4.6.2.3 kml_fft(f/h)_plan_with_nthreads

kml_fft(f/h)_plan_with_nthreads用于指定FFT库接口函数执行的线程数。（此函数不可重入。）

接口定义

C interface：

void kml_fft_plan_with_nthreads(int nthreads);

void kml_fftf_plan_with_nthreads(int nthreads);

void kml_ffth_plan_with_nthreads(int nthreads);

Fortran interface：

CALL KML_FFT_PLAN_WITH_NTHREADS(NTHREADS);

CALL KML_FFTF_PLAN_WITH_NTHREADS(NTHREADS);

CALL KML_FFTH_PLAN_WITH_NTHREADS(NTHREADS);

 版权所有 © 华为技术有限公司

参数

参数名	数据类型	描述	输入/输出
nthreads	int	执行后续FFT变换plan所使用的线程数。	输入

依赖

C: "kfft.h"

示例

C interface：

```
kml_fft_init_threads();
int nthreads = 4;
kml_fft_plan_with_nthreads(nthreads);
int N = 16;
kml_fft_complex *in;
kml_fft_complex *out;
kml_fft_plan plan;
in = (kml_fft_complex*)kml_fft_malloc(sizeof(kml_fft_complex) * N);
out = (kml_fft_complex*)kml_fft_malloc(sizeof(kml_fft_complex) * N);
for (int i = 0; i < N; i++){
    in[i][0] = (double)i;
    in[i][1] = (double)i;
}

plan = kml_fft_plan_dft_1d(N, in , out, KML_FFT_FORWARD, KML_FFT_ESTIMATE);
kml_fft_execute(plan);
kml_fft_destroy_plan(plan);
kml_fft_cleanup_threads();
kml_fft_free(in);
kml_fft_free(out);

/*
 * out = {{1.200000e+02, 1.200000e+02}, {-4.821872e+01, 3.221872e+01},
 *        {-2.731371e+01, 1.131371e+01}, {-1.997285e+01, 3.972846e+00},
 *        {-1.600000e+01, 0.000000e+00}, {-1.334543e+01, -2.654571e+00},
 *        {-1.131371e+01, -4.686292e+00}, {-9.591299e+00, -6.408701e+00},
 *        {-8.000000e+00, -8.000000e+00}, {-6.408701e+00, -9.591299e+00},
 *        {-4.686292e+00, -1.131371e+01}, {-2.654571e+00, -1.334543e+01},
 *        {0.000000e+00, -1.600000e+01}, {3.972846e+00, -1.997285e+01},
 *        {1.131371e+01, -2.731371e+01}, {3.221872e+01, -4.821872e+01}}
 */
```

Fortran interface：

```
INTEGER(C_INT) :: STATUS
STATUS = KML_FFT_INIT_THREADS()
INTEGER(C_INT) :: NTHREADS = 4
CALL KML_FFT_PLAN_WITH_NTHREADS(NTHREADS)
INTEGER(C_INT) :: N = 16
TYPE(KML_FFT_COMPLEX), POINTER :: IN, OUT
TYPE(C_PTR) :: PIN, POUT
TYPE(C_PTR) :: PLAN
INTEGER(C_SIZE_T) :: SIZE
SIZE = 16 * N
PIN = KML_FFT_MALLOC(SIZE)
POUT = KML_FFT_MALLOC(SIZE)
CALL C_F_POINTER(PIN, IN, SHAPE=[N])
```


版权所有 © 华为技术有限公司

```
CALL C_F_POINTER(POUT, OUT, SHAPE=[N])
INTEGER :: I = 1
DO WHILE(I <= N)
   IN(I)%R = I
   IN(I)%I = I
   I = I + 1
END DO
PLAN = KML_FFT_PLAN_DFT_1D(N, IN , OUT, KML_FFT_FORWARD, KML_FFT_ESTIMATE)
CALL KML_FFT_EXECUTE(PLAN, IN, OUT)

CALL KML_FFT_DESTROY_PLAN(PLAN)
CALL KML_FFT_FREE(PIN)
CALL KML_FFT_FREE(POUT)

!
! OUT = \1.200000E+02, 1.200000E+0, -4.821872E+01, 3.221872E+01,
!       -2.731371E+01, 1.131371E+01, -1.997285E+01, 3.972846E+00,
!       -1.600000E+01, 0.000000E+00, -1.334543E+01, -2.654571E+00,
!       -1.131371E+01, -4.686292E+00, -9.591299E+00, -6.408701E+00,
!       -8.000000E+00, -8.000000E+00, -6.408701E+00, -9.591299E+00,
!       -4.686292E+00, -1.131371E+01, -2.654571E+00, -1.334543E+01,
!       0.000000E+00, -1.600000E+01, 3.972846E+00, -1.997285E+01,
!       1.131371E+01, -2.731371E+01, 3.221872E+01, -4.821872E+01/
!
```

6.4.4.6.2.4 kml_fft(f/h)_cleanup_threads

kml_fft(f/h)_cleanup_threads用于释放多线程框架相关的资源，与kml_fft(f/h)_init_threads相对应，在整个系统退出时调用。调用后不能再使用多线程功能。（此函数不可重入。）

接口定义

C interface：

void kml_fft_cleanup_threads(void);

void kml_fftf_cleanup_threads(void);

void kml_ffth_cleanup_threads(void);

Fortran interface：

CALL KML_FFT_CLEANUP_THREADS();

CALL KML_FFTF_CLEANUP_THREADS();

CALL KML_FFTH_CLEANUP_THREADS();

参数

无

依赖

C: "kfft.h"

示例

C interface：

```
kml_fft_init_threads();
int nthreads = 4;
```

 版权所有 © 华为技术有限公司

```
kml_fft_plan_with_nthreads(nthreads);
int N = 16;
kml_fft_complex *in;
kml_fft_complex *out;
kml_fft_plan plan;
in = (kml_fft_complex*)kml_fft_malloc(sizeof(kml_fft_complex) * N);
out = (kml_fft_complex*)kml_fft_malloc(sizeof(kml_fft_complex) * N);
for (int i = 0; i < N; i++){
    in[i][0] = (double)i;
    in[i][1] = (double)i;
}

plan = kml_fft_plan_dft_1d(N, in , out, KML_FFT_FORWARD, KML_FFT_ESTIMATE);
kml_fft_execute(plan);
kml_fft_destroy_plan(plan);
kml_fft_cleanup_threads();
kml_fft_free(in);
kml_fft_free(out);

/*
 * out = {{1.200000e+02, 1.200000e+02}, {-4.821872e+01, 3.221872e+01},
 *        {-2.731371e+01, 1.131371e+01}, {-1.997285e+01, 3.972846e+00},
 *        {-1.600000e+01, 0.000000e+00}, {-1.334543e+01, -2.654571e+00},
 *        {-1.131371e+01, -4.686292e+00}, {-9.591299e+00, -6.408701e+00},
 *        {-8.000000e+00, -8.000000e+00}, {-6.408701e+00, -9.591299e+00},
 *        {-4.686292e+00, -1.131371e+01}, {-2.654571e+00, -1.334543e+01},
 *        {0.000000e+00, -1.600000e+01}, {3.972846e+00, -1.997285e+01},
 *        {1.131371e+01, -2.731371e+01}, {3.221872e+01, -4.821872e+01}}
 */
```

Fortran interface：

```
INTEGER(C_INT) :: STATUS
STATUS = KML_FFT_INIT_THREADS()
INTEGER(C_INT) :: NTHREADS = 4
CALL KML_FFT_PLAN_WITH_NTHREADS(NTHREADS)
INTEGER(C_INT) :: N = 16
TYPE(KML_FFT_COMPLEX), POINTER :: IN, OUT
TYPE(C_PTR) :: PIN, POUT
TYPE(C_PTR) :: PLAN
INTEGER(C_SIZE_T) :: SIZE
SIZE = 16 * N
PIN = KML_FFT_MALLOC(SIZE)
POUT = KML_FFT_MALLOC(SIZE)
CALL C_F_POINTER(PIN, IN, SHAPE=[N])
CALL C_F_POINTER(POUT, OUT, SHAPE=[N])
INTEGER :: I = 1
DO WHILE(I <= N)
   IN(I)%R = I
   IN(I)%I = I
   I = I + 1
END DO
PLAN = KML_FFT_PLAN_DFT_1D(N, IN , OUT, KML_FFT_FORWARD, KML_FFT_ESTIMATE)
CALL KML_FFT_EXECUTE(PLAN, IN, OUT)

CALL KML_FFT_DESTROY_PLAN(PLAN)
CALL KML_FFT_FREE(PIN)
CALL KML_FFT_FREE(POUT)

!
! OUT = \1.200000E+02, 1.200000E+0, -4.821872E+01, 3.221872E+01,
!       -2.731371E+01, 1.131371E+01, -1.997285E+01, 3.972846E+00,
!       -1.600000E+01, 0.000000E+00, -1.334543E+01, -2.654571E+00,
!       -1.131371E+01, -4.686292E+00, -9.591299E+00, -6.408701E+00,
!       -8.000000E+00, -8.000000E+00, -6.408701E+00, -9.591299E+00,
!       -4.686292E+00, -1.131371E+01, -2.654571E+00, -1.334543E+01,
!       0.000000E+00, -1.600000E+01, 3.972846E+00, -1.997285E+01,
!       1.131371E+01, -2.731371E+01, 3.221872E+01, -4.821872E+01/
!
```

 版权所有 © 华为技术有限公司

6.4.4.6.3 KFFTGetVersion

获取FFT产品版本信息。

接口定义

int KFFTGetVersion(KFFTVersion*ver)

参数

参数名	描述	取值范围	输入/输出
ver	产品信息结构体	非空	输出

返回值

- 成功：返回1。
- 失败：返回0。

示例

```
#include <stdio.h>
#include "kfft.h"

int main()
{
    KFFTVersion ver;
    int ret = KFFTGetVersion(&ver);
    if (ret == 1) {
        printf("\nFFT\n");
        printf("Product Name:         %s\n", ver.productName);
        printf("Product Version:      %s\n", ver.productVersion);
        printf("Component Name:       %s\n", ver.componentName);
        printf("Component Version:    %s\n", ver.componentVersion);
        printf("Component AbiVersion: %s\n", ver.componentAbiVersion);
        printf("Component AppendInfo: %s\n", ver.componentAppendInfo);
        printf("Software Name:        %s\n", ver.softwareName);
        printf("Software Version:     %s\n", ver.softwareVersion);
    }

    return 0;
}
```

运行结果：

```
FFT
Product Name:         Kunpeng HPCKit
Product Version:      24.0.RC1
Component Name:       HPCKit-kml
Component Version:    2.3.0
Component AbiVersion: 1
Component AppendInfo: gcc
Software Name:        hpckit-kml-fft
Software Version:     2.3.0
```

说明

以上版本号和编译时间以实际运行结果为准，上述结果仅供参考。

版权所有 © 华为技术有限公司

6.4.4.6.4 kml_fft(f/h)_set_timelimit

设置FFT PATIENT模式超时时间

接口定义

void kml_fft_set_timelimit(double tlim)

void kml_fftf_set_timelimit(double tlim)

void kml_ffth_set_timelimit(double tlim)

参数

参数名	描述	取值范围	输入/输出
tlim	设置FFT PATIENT模式超时时间，单位为秒。若不调用该接口，表现为无时间限制。	double 负数、0意味着无时间限制，plan接口将进行完整的PATIENT模式搜索 正数值将设定指定秒数的超时时间，内部搜索循环会在每次评估后进行超时判断，若超时，则输出已评估列表中的最优解。	输入

依赖

C: "kfft.h"

示例

C interface：

```
int N = 16;
kml_fft_complex *in;
kml_fft_complex *out;
kml_fft_plan plan;
in = (kml_fft_complex*)kml_fft_malloc(sizeof(kml_fft_complex) * N);
out = (kml_fft_complex*)kml_fft_malloc(sizeof(kml_fft_complex) * N);
for (int i = 0; i < N; i++){
    in[i][0] = (double)i;
    in[i][1] = (double)i;
}
kml_fft_set_timelimit(10);
plan = kml_fft_plan_dft_1d(N, in , out, KML_FFT_FORWARD, KML_FFT_PATIENT);
kml_fft_execute_dft(plan, in, out);

kml_fft_destroy_plan(plan);
kml_fft_free(in);
kml_fft_free(out);

/*
```

 版权所有 © 华为技术有限公司

```
 * out = {{1.200000e+02, 1.200000e+02}, {-4.821872e+01, 3.221872e+01},
 *        {-2.731371e+01, 1.131371e+01}, {-1.997285e+01, 3.972846e+00},
 *        {-1.600000e+01, 0.000000e+00}, {-1.334543e+01, -2.654571e+00},
 *        {-1.131371e+01, -4.686292e+00}, {-9.591299e+00, -6.408701e+00},
 *        {-8.000000e+00, -8.000000e+00}, {-6.408701e+00, -9.591299e+00},
 *        {-4.686292e+00, -1.131371e+01}, {-2.654571e+00, -1.334543e+01},
 *        {0.000000e+00, -1.600000e+01}, {3.972846e+00, -1.997285e+01},
 *        {1.131371e+01, -2.731371e+01}, {3.221872e+01, -4.821872e+01}}
 */
```

6.4.5 KML_SVML 库函数说明

6.4.5.1 函数说明

KML_SVML是短向量（short vector math library）运算库。包括三角函数、双曲函数、指数函数、对数函数和幂函数等。SVML是基础运算的SIMD代码向量化实现方式。

函数类	函数名	数据类型	描述
三角函数（Trigonometric functions）	sin	f32，f64	正弦函数
	cos	f32，f64	余弦函数
	sincos	f32，f64	正弦、余弦函数
	tan	f32，f64	正切函数
	atan	f32，f64	反正切函数
	atan2	f32，f64	反正切函数
双曲函数（hyperbolic functions）	sinh	f32，f64	双曲正弦函数
	cosh	f32，f64	双曲余弦函数
	tanh	f32，f64	双曲正切函数
指数和对数函数（Exponential and logarithmic functions）	exp	f32，f64	指数函数（base e）
	exp2	f32，f64	指数函数（base 2）
	expm1	f32，f64	指数函数（base e）
	log	f32，f64	对数函数（base e）
	log10	f32，f64	对数函数（base 10）
	log1p	f32，f64	对数函数（base e）
幂函数（Power function）	pow	f32，f64	幂函数

6.4.5.2 函数定义

 版权所有 © 华为技术有限公司

6.4.5.2.1 svml128_atan_f?

计算输入向量的反正切值。

接口定义

C interface：

float32x4_t svml128_atan_f32(float32x4_t src);

float64x2_t svml128_atan_f64(float64x2_t src);

参数

参数名	类型	描述	输入/输出
src	• 在svml128_atan_f32中，src是单精度浮点类型。 • 在svml128_atan_f64中，src是双精度浮点类型。	表示输入向量的浮点值。	输入

输出结果

- 返回 (-π/2,π/2) 上正切值等于x的那个唯一确定的角，取值范围为(-π/2, +π/2)。
- 输入+0，返回+0。
- 输入-0，返回-0。
- 输入+∞，返回+π/2。
- 输入-∞，返回-π/2。
- 输入nan，返回nan。

依赖

C: "ksvml.h"

示例

C interface：

```
float32x4_t src = {0.0f, 2.0f, INFINITY, NAN};
float32x4_t dst = svml128_atan_f32(src);
printf("%.15g  %.15g  %.15g  %.15g\n", dst[0], dst[1], dst[2], dst[3]);
/**
 *  Output dst:
 *     0  1.10714871779409  1.5707963267949  -nan
 *
 */
```

6.4.5.2.2 svml128_atan2_f?

计算输入向量的反正切值，输入src1以及src2均为向量。

 版权所有 © 华为技术有限公司

接口定义

C interface：

float32x4_t svml128_atan2_f32(float32x4_t src1, float32x4_t src2);

float64x2_t svml128_atan2_f64(float64x2_t src1, float64x2_t src2);

参数

参数名	类型	描述	输入/输出
src1	• 在svml128_atan2_f32中，src1是单精度浮点类型。 • 在svml128_atan2_f64中，src1是双精度浮点类型。	表示输入向量的浮点值。	输入
src2	• 在svml128_atan2_f32中，src2是单精度浮点类型。 • 在svml128_atan2_f64中，src2是双精度浮点类型。	表示输入向量的浮点值。	输入

输出结果

对于atan2(y, x)来说：

- 返回的是原点至点(x, y)的方位角，即与x轴的夹角。返回值的单位为弧度，取值范围为(-π, +π]。
- 输入y为±0，x为负或-0，则返回±π。
- 输入y为±0，x为正或+0，则返回±0。
- 输入y为±∞，x为有限，则返回±π/2。
- 输入y为±∞，x为-∞，则返回±3π/4。
- 输入y为±∞，x为+∞，则返回±π/4。
- 输入x为±0，y为负，则返回-π/2。
- 输入x为±0，y为正，则返回+π/2。
- 输入x是-∞，y是有限且正的，则返回+π。
- 输入x为-∞，y为有限负，则返回-π。
- 输入x为+∞，且y为有限且为正，则返回+0。
- 输入x为+∞，且y为有限且为负，则返回-0。
- 输入x为nan，或y为nan，则返回nan。

依赖

C: "ksvml.h"

版权所有 © 华为技术有限公司

示例

C interface：

```
float32x4_t src = {0.0f, 2.0f, INFINITY, NAN};
float32x4_t src2 = {0.0f, 2.0f, INFINITY, NAN};
float32x4_t dst = svml128_atan2_f32(src, src2);
printf("%.15g  %.15g  %.15g  %.15g\n", dst[0], dst[1], dst[2], dst[3]);
/**
 *  Output dst:
 *  0  0.785398163397448  0.785398163397448  -nan
 */
```

6.4.5.2.3 svml128_cos_f?

计算输入向量的余弦值。

接口定义

C interface：

float32x4_t svml128_cos_f32(float32x4_t src);

float64x2_t svml128_cos_f64(float64x2_t src);

参数

参数名	类型	描述	输入/输出
src	● 在svml128_cos_f32中，src是单精度浮点类型。 ● 在svml128_cos_f64中，src是双精度浮点类型。	表示输入向量的浮点值。	输入

输出结果

- 返回弧度角x的余弦函数值y，y ∈ [-1, +1]。
- 输入+0，返回+1。
- 输入-0，返回+1。
- 输入±∞，返回nan。
- 输入nan，返回nan。

依赖

C: "ksvml.h"

示例

C interface：

```
float32x4_t src = {0.0f, 2.0f, INFINITY, NAN};
float32x4_t dst = svml128_cos_f32(src);
printf("%.15g  %.15g  %.15g  %.15g\n", dst[0], dst[1], dst[2], dst[3]);
```


版权所有 © 华为技术有限公司

```
/**
 *  Output dst:
 *     1  -0.416146836547142  nan  -nan
 *
 */
```

6.4.5.2.4 svml128_sin_f?

计算输入向量的正弦值。

接口定义

C interface：

float32x4_t svml128_sin_f32(float32x4_t src);

float64x2_t svml128_sin_f64(float64x2_t src);

参数

参数名	类型	描述	输入/输出
src	● 在svml128_sin_f32中，src是单精度浮点类型。 ● 在svml128_sin_f64中，src是双精度浮点类型。	表示输入向量的浮点值。	输入

输出结果

- 返回弧度角x的正弦函数值y，y ∈ [-1, +1]。
- 输入+0，返回+0。
- 输入-0，返回-0。
- 输入±∞，返回nan。
- 输入nan，返回nan。

依赖

C: "ksvml.h"

示例

C interface：

```
float32x4_t src = {0.0f, 2.0f, INFINITY, NAN};
float32x4_t dst = svml128_sin_f32(src);
printf("%.15g  %.15g  %.15g  %.15g\n", dst[0], dst[1], dst[2], dst[3]);
/**
 *  Output dst:
 *     0  0.909297426825682  nan  -nan
 *
 */
```


版权所有 © 华为技术有限公司

6.4.5.2.5 svml128_tan_f?

计算输入向量的正切值。

接口定义

C interface：

float32x4_t svml128_tan_f32(float32x4_t src);

float64x2_t svml128_tan_f64(float64x2_t src);

参数

参数名	类型	描述	输入/输出
src	• 在svml128_tan_f32中，src是单精度浮点类型。 • 在svml128_tan_f64中，src是双精度浮点类型。	表示输入向量的浮点值。	输入

输出结果

- 返回弧度角x的正切函数值y，y ∈ (-INF, +INF)。
- 输入+0，返回+0。
- 输入-0，返回-0。
- 输入±∞，返回nan。
- 输入nan，返回nan。

该函数在π(1/2 + n)处具有数学极点；没有通用的浮点表示能够精确地表示极点处的值，因此极点处会返回值，但不能认为是正确的。

依赖

C: "ksvml.h"

示例

C interface：

```
float32x4_t src = {0.0f, 2.0f, INFINITY, NAN};
float32x4_t dst = svml128_tan_f32(src);
printf("%.15g  %.15g  %.15g  %.15g\n", dst[0], dst[1], dst[2], dst[3]);
/**
 *  Output dst:
 *     0  -2.18503986326152  nan  -nan
 *
 */
```

6.4.5.2.6 svml128_sincos_f?

计算输入向量的正弦值与余弦值。

 版权所有 © 华为技术有限公司

接口定义

C interface：

void svml128_sincos_f32(float32x4_t src, float32x4_t *sindst, float32x4_t *cosdst);

void svml128_sincos_f64(float64x2_t src, float64x2_t *sindst ,float64x2_t *cosdst);

参数

参数名	类型	描述	输入/输出
src	• 在svml128_sincos_f32中，src是单精度浮点类型。 • 在svml128_sincos_f64中，src是双精度浮点类型。	表示输入向量的浮点值。	输入
sindst	• 在svml128_sincos_f32中，sindst是单精度浮点类型。 • 在svml128_sincos_f64中，sindst是双精度浮点类型。	表示输出向量的浮点值。 若为空指针，会提示空指针错误并返回。	输出
cosdst	• 在svml128_sincos_f32中，cosdst是单精度浮点类型。 • 在svml128_sincos_f64中，cosdst是双精度浮点类型。	表示输出向量的浮点值。 若为空指针，会提示空指针错误并返回。	输出

输出结果

- 每一个运算值返回弧度角x的正弦数值sinx和余弦函数值cosx，sinx ∈ [-1, +1]，cosx ∈ [-1, +1]。
- 输入+0，返回正弦值+0和余弦值+1。
- 输入-0，返回正弦值-0和余弦值+1。
- 输入±∞，均返回nan。
- 输入nan，均返回nan。

依赖

C: "ksvml.h"

示例

C interface：

 版权所有 © 华为技术有限公司

```
float32x4_t src = {0.0f, 2.0f, INFINITY, NAN};
float32x4_t dst = (float32x4_t*)malloc(4*sizeof(float32x4_t));
float32x4_t dst2 = (float32x4_t*)malloc(4*sizeof(float32x4_t));
vatan_neon_f32(src, dst, dst2);
printf("%.15g  %.15g  %.15g  %.15g\n", dst[0], dst[1], dst[2], dst[3]);
printf("%.15g  %.15g  %.15g  %.15g\n", dst2[0], dst2[1], dst2[2], dst2[3]);
/**
 *  Output dst:
 *     0  0.909297426825682  nan  -nan
 *     1  -0.416146836547142  nan  -nan
 *
 */
```

6.4.5.2.7 svml128_cosh_f?

计算输入向量的双曲余弦值。

接口定义

C interface：

float32x4_t svml128_cosh_f32(float32x4_t src);

float64x2_t svml128_cosh_f64(float64x2_t src);

参数

参数名	类型	描述	输入/输出
src	• 在svml128_cosh_f32中，src是单精度浮点类型。 • 在svml128_cosh_f64中，src是双精度浮点类型。	表示输入向量的浮点值。	输入

输出结果

- 返回弧度角x的双曲余弦函数值y，y ∈ [+1, +INF]。
- 输入+0，返回+1。
- 输入-0，返回+1。
- 输入±∞，返回INF。
- 输入nan，返回nan。

依赖

C: "ksvml.h"

示例

C interface：

```
float32x4_t src = {0.0f, 2.0f, INFINITY, NAN};
float32x4_t dst = svml128_cosh_f32(src);
printf("%.15g  %.15g  %.15g  %.15g\n", dst[0], dst[1], dst[2], dst[3]);
/**
```


版权所有 © 华为技术有限公司

```
 *  Output dst:
 *     1  3.76219569108363  inf  -nan
 *
 */
```

6.4.5.2.8 svml128_sinh_f?

计算输入向量的双曲正弦值。

接口定义

C interface：

float32x4_t svml128_sinh_f32(float32x4_t src);

float64x2_t svml128_sinh_f64(float64x2_t src);

参数

参数名	类型	描述	输入/输出
src	• 在svml128_sinh_f32中，src是单精度浮点类型。 • 在svml128_sinh_f64中，src是双精度浮点类型。	表示输入向量的浮点值。	输入

输出结果

- 返回弧度角x的双曲正弦函数值y，y ∈ [-INF, +INF]。
- 输入+0，返回+0。
- 输入-0，返回-0。
- 输入±∞，返回±INF。
- 输入nan，返回nan。

依赖

C: "ksvml.h"

示例

C interface：

```
float32x4_t src = {0.0f, 2.0f, INFINITY, NAN};
float32x4_t dst = svml128_sinh_f32(src);
printf("%.15g  %.15g  %.15g  %.15g\n", dst[0], dst[1], dst[2], dst[3]);
/**
 *  Output dst:
 *     0  0.909297426825682  nan  -nan
 *
 */
```

6.4.5.2.9 svml128_tanh_f?

计算输入向量的双曲正切值。

 版权所有 © 华为技术有限公司

接口定义

C interface：

float32x4_t svml128_tanh_f32(float32x4_t src);

float64x2_t svml128_tanh_f64(float64x2_t src);

参数

参数名	类型	描述	输入/输出
src	• 在svml128_tanh_f32中，src是单精度浮点类型。 • 在svml128_tanh_f64中，src是双精度浮点类型。	表示输入向量的浮点值。	输入

输出结果

- 返回输入值x的双曲正切函数值y，y ∈ [-1, +1]。
- 输入+0，返回+0。
- 输入-0，返回-0。
- 输入±∞，返回±1。
- 输入nan，返回nan。

依赖

C: "ksvml.h"

示例

C interface：

```
float32x4_t src = {0.0f, 2.0f, INFINITY, NAN};
float32x4_t dst = svml128_tanh_f32(src);
printf("%.15g  %.15g  %.15g  %.15g\n", dst[0], dst[1], dst[2], dst[3]);
/**
 *  Output dst:
 *     0  0.964027580075817  1  -nan
 *
 */
```

6.4.5.2.10 svml128_exp_f?

计算输入向量以e为底数的指数值。

接口定义

C interface：

float32x4_t svml128_exp_f32(float32x4_t src);

float64x2_t svml128_exp_f64(float64x2_t src);

 版权所有 © 华为技术有限公司

参数

参数名	类型	描述	输入/输出
src	• 在svml128_exp_f32中，src是单精度浮点类型。 • 在svml128_exp_f64中，src是双精度浮点类型。	表示输入向量的浮点值。	输入

输出结果

- 返回以e为底x的指数函数值y，y ∈ (0, +INF)。
- 输入+0，返回+1。
- 输入-0，返回-1。
- svml128_exp_f32输入大于0x1.62e42ep6f（约88），返回+∞；svml128_exp_f64输入大于0x1.62p9（约708），返回+∞。
- svml128_exp_f32输入小于-0x1.9fe368p6f（约-104），返回+0；svml128_exp_f64输入小于-0x1.62p9（约-708），返回+0。
- 输入+∞，返回+∞。
- 输入-∞，返回+0。
- 输入nan，返回nan。

依赖

C: "ksvml.h"

示例

C interface：

```
float32x4_t src = {0.0f, 2.0f, INFINITY, NAN};
float32x4_t dst = svml128_exp_f32(src);
printf("%.15g  %.15g  %.15g  %.15g\n", dst[0], dst[1], dst[2], dst[3]);
/**
 *  Output dst:
 *     1  7.38905609893065  inf  -nan
 *
 */
```

6.4.5.2.11 svml128_exp2_f?

计算输入向量以2为底数的指数值。

接口定义

C interface：

float32x4_t svml128_exp2_f32(float32x4_t src);

float64x2_t svml128_exp2_f64(float64x2_t src);

 版权所有 © 华为技术有限公司

参数

参数名	类型	描述	输入/输出
src	• 在svml128_exp2_f32中，src是单精度浮点类型。 • 在svml128_exp2_f64中，src是双精度浮点类型。	表示输入向量的浮点值。	输入

输出结果

- 返回以2为底x的指数函数值y，y ∈ (0, +INF)。
- 输入+0，返回+1。
- 输入-0，返回-1。
- expf输入值大于128，返回+∞；exp输入值大于1024，返回+∞。
- expf输入值小于-150，返回+0；exp输入值小于-1075，返回+0。
- 输入+∞，返回+∞。
- 输入-∞，返回+0。
- 输入nan，返回nan。

依赖

C: "ksvml.h"

示例

C interface：

```
float32x4_t src = {0.0f, 2.0f, INFINITY, NAN};
float32x4_t dst = svml128_exp2_f32(src);
printf("%.15g  %.15g  %.15g  %.15g\n", dst[0], dst[1], dst[2], dst[3]);
/**
 *  Output dst:
 *     1  4  inf  -nan
 *
 */
```

6.4.5.2.12 svml128_expm1_f?

计算输入向量以e为底数的指数值减1的结果。

接口定义

C interface：

float32x4_t svml128_expm1_f32(float32x4_t src);

float64x2_t svml128_expm1_f64(float64x2_t src);

 版权所有 © 华为技术有限公司

参数

参数名	类型	描述	输入/输出
src	• 在svml128_expm1_f32中，src是单精度浮点类型。 • 在svml128_expm1_f64中，src是双精度浮点类型。	表示输入向量的浮点值。	输入

输出结果

- 返回以e为底x的指数函数值y，y ∈ (0, +INF)。
- 输入+0，返回+0。
- 输入-0，返回-0。
- svml128_expm1_f32输入大于0x1.62e42ep6f（约88），返回+∞；svml128_expm1_f64输入大于0x1.62p9（约708），返回+∞。
- svml128_expm1_f32输入小于-0x1.9fe368p6f（约-104），返回+0；svml128_expm1_f64输入小于-0x1.62p9（约-708），返回+0。
- 输入+∞，返回+∞。
- 输入-∞，返回+0。
- 输入nan，返回nan。

依赖

C: "ksvml.h"

示例

C interface：

```
float32x4_t src = {0.0f, 2.0f, INFINITY, NAN};
float32x4_t dst = svml128_expm1_f32(src);
printf("%.15g  %.15g  %.15g  %.15g\n", dst[0], dst[1], dst[2], dst[3]);
/**
 *  Output dst:
 *     0  6.38905609893065  inf  -nan
 *
 */
```

6.4.5.2.13 svml128_log_f?

计算输入向量以自然对数e为底的对数值。

接口定义

C interface：

float32x4_t svml128_log_f32(float32x4_t src);

版权所有 © 华为技术有限公司

float64x2_t svml128_log_f64(float64x2_t src);

参数

参数名	类型	描述	输入/输出
src	• 在svml128_log_f32中，src是单精度浮点类型。 • 在svml128_log_f64中，src是双精度浮点类型。	表示输入向量的浮点值。	输入

输出结果

- 返回以e为底x的对数函数值y，y ∈ (-INF, +INF)。
- 输入x为±0，返回-∞。
- 输入x为1，返回0。
- 输入x为负数，返回nan。
- 输入x为+∞，返回+∞。
- 输入x为nan，返回nan。

依赖

C: "ksvml.h"

示例

C interface：

```
float32x4_t src = {0.0f, 2.0f, INFINITY, NAN};
float32x4_t dst = svml128_log_f32(src);
printf("%.15g  %.15g  %.15g  %.15g\n", dst[0], dst[1], dst[2], dst[3]);
/**
 *  Output dst:
 *     -inf  0.693147180559945  inf  -nan
 *
 */
```

6.4.5.2.14 svml128_log10_f?

计算输入向量以10为底的对数值。

接口定义

C interface：

float32x4_t svml128_log10_f32(float32x4_t src);

float64x2_t svml128_log10_f64(float64x2_t src);

 版权所有 © 华为技术有限公司

参数

参数名	类型	描述	输入/输出
src	• 在svml128_log10_f32中，src是单精度浮点类型。 • 在svml128_log10_f64中，src是双精度浮点类型。	表示输入向量的浮点值。	输入

输出结果

- 返回以10为底x的对数函数值y，y ∈ (-INF, +INF)。
- 输入x为±0，返回-∞。
- 输入x为1，返回0。
- 输入x为负数，返回nan。
- 输入x为+∞，返回+∞。
- 输入x为nan，返回nan。

依赖

C: "ksvml.h"

示例

C interface：

```
float32x4_t src = {0.0f, 2.0f, INFINITY, NAN};
float32x4_t dst = svml128_log10_f32(src);
printf("%.15g  %.15g  %.15g  %.15g\n", dst[0], dst[1], dst[2], dst[3]);
/**
 *  Output dst:
 *     -inf  0.301029995663981  inf  -nan
 *
 */
```

6.4.5.2.15 svml128_log1p_f?

计算输入向量以自然对数e为底，输入值加1的对数值。

接口定义

C interface：

float32x4_t svml128_log1p_f32(float32x4_t src);

float64x2_t svml128_log1p_f64(float64x2_t src);

 版权所有 © 华为技术有限公司

参数

参数名	类型	描述	输入/输出
src	• 在svml128_log1p_f32中，src是单精度浮点类型。 • 在svml128_log1p_f64中，src是双精度浮点类型。	表示输入向量的浮点值。	输入

输出结果

- 返回以e为底x+1的对数函数值y，y ∈ (-INF, +INF)。
- 输入x为±0，返回±0。
- 输入x为-1，返回-INF。
- 输入x为负数，返回nan。
- 输入x为+∞，返回+∞。
- 输入x为nan，返回nan。

依赖

C: "ksvml.h"

示例

C interface：

```
float32x4_t src = {0.0f, 2.0f, INFINITY, NAN};
float32x4_t dst = svml128_log1p_f32(src);
printf("%.15g  %.15g  %.15g  %.15g\n", dst[0], dst[1], dst[2], dst[3]);
/**
 *  Output dst:
 *     0  1.09861228866811  inf  -nan
 *
 */
```

6.4.5.2.16 svml128_pow_f?

计算x的y次幂，x与y均为向量。

接口定义

C interface：

float32x4_t svml128_pow_f32(float32x4_t src1, float32x4_t src2);

float64x2_t svml128_pow_f64(float64x2_t src1, float64x2_t src2);

 版权所有 © 华为技术有限公司

参数

参数名	类型	描述	输入/输出
src1	• 在svml128_pow_f32中，src1是单精度浮点类型。 • 在svml128_pow_f64中，src1是双精度浮点类型。	表示输入向量的浮点值。	输入
src2	• 在svml128_pow_f32中，src2是单精度浮点类型。 • 在svml128_pow_f64中，src2是双精度浮点类型。	表示输入向量的浮点值。	输入

输出结果

- x大于0且y为小数，返回幂指数的结果，取值范围r ∈ (-INF, +INF)。
- 输入x为-0，y为负的奇数，则返回-∞；其他x为±0，y为负数，返回+∞。
- 输入x为±0，y为正的奇数，则返回±0；其他x为±0，y为正数，返回+0。
- 输入x为±1，y为任意值，则返回+1。
- 输入x为任意值，y为±0，则返回+1。
- 输入x小于+0，y为小数，则返回nan。
- 输入|x|小于1，y为-∞，则返回+∞；输入|x|大于1，y为-∞，则返回+0。
- 输入|x|小于1，y为+∞，则返回+0；输入|x|大于1，y为+∞，则返回+∞。
- 输入x为-∞，y为负奇数，则返回-0；其他x为-∞，y为负数，则返回+0。
- 输入x为-∞，y为正奇数，则返回-∞；其他x为-∞，y为正数，则返回+∞。
- 输入x为+∞，y小于+0，则返回+0；其他x为+∞，则返回+∞。
- 输入x为大的有限数，且y为大的有限数，则返回±∞。
- 输入x为nan，且y为nan，则返回nan。

依赖

C: "ksvml.h"

示例

C interface：

```
float32x4_t src = {0.0f, 2.0f, INFINITY, NAN};
float32x4_t src2 = {0.0f, 2.0f, INFINITY, NAN};
float32x4_t dst = svml128_pow_f32(src, src2);
printf("%.15g  %.15g  %.15g  %.15g\n", dst[0], dst[1], dst[2], dst[3]);
/**
 *  Output dst:
 *    1  4  inf  nan
 *
 */
```


版权所有 © 华为技术有限公司

6.4.5.2.17 KSVMLGetVersion

获取SVML产品版本信息。

接口定义

int KSVMLGetVersion(KSVMLVersion *ver)

参数

参数名	描述	取值范围	输入/输出
ver	产品信息结构体	非空	输出

返回值

- 成功：返回1。
- 失败：返回0。

示例

```
#include <stdio.h>
#include "ksvml.h"

int main()
{
    KSVMLVersion svmlver;
    int status = KSVMLGetVersion(&svmlver);
    if (status == 0) {
        return 0;
    }
    printf("Product Name: %s\n", svmlver.productName);
    printf("Product Version: %s\n", svmlver.productVersion);
    printf("Component Name: %s\n", svmlver.componentName);
    printf("Component Version: %s\n", svmlver.componentVersion);
    printf("Component AbiVersion: %s\n", svmlver.componentAbiVersion);
    printf("Component AppendInfo: %s\n", svmlver.componentAppendInfo);
    printf("Software Name: %s\n", svmlver.softwareName);
    printf("Software Version: %s\n", svmlver.softwareVersion);
    return 0;
}
```

运行结果：

```
Product Name: Kunpeng HPCKit
Product Version: 24.0.RC1
Component Name: HPCKit-kml
Component Version: 2.3.0
Component AbiVersion: 1
Component AppendInfo: gcc
Software Name: hpckit-kml-svml
Software Version: 2.3.0
```

说明

版本号和编译时间以实际运行结果为准，上述结果仅供参考。

6.4.5.3 编译器自动向量化

 版权所有 © 华为技术有限公司

6.4.5.3.1 介绍

编译器的自动向量化（automatic vectorization）将一次处理一对标量运算转换为一次并行处理多对向量运算，可以显著加速一些带有循环的程序运算。除了上述接口外，KML_SVML同时为编译器提供了向量化接口，见表1 编译器向量化接口。在编译过程中，针对不同的编译器添加对应的编译选项，使编译器自动调用对应的向量化接口，请参见9.3.2-GCC向量化方法和9.3.3-毕昇编译器向量化方法。

表 6-31 编译器向量化接口

SVML接口	MATH接口	描述
_ZGVnN2v_sin _ZGVsNxv_sin _ZGVnN4v_sinf _ZGVsNxv_sinf	sin sinf	正弦函数
_ZGVnN2v_cos _ZGVsNxv_cos _ZGVnN4v_cosf _ZGVsNxv_cosf	cos cosf	余弦函数
_ZGVnN2v_tan _ZGVsNxv_tan _ZGVnN4v_tanf _ZGVsNxv_tanf	tan tanf	正切函数
_ZGVnN2v_atan _ZGVsNxv_atan _ZGVnN4v_atanf _ZGVsNxv_atanf	atan atanf	反正切函数
_ZGVnN2vv_atan2 _ZGVsNxvv_atan2 _ZGVnN4vv_atan2f _ZGVsNxvv_atan2f	atan2 atan2f	反正切函数
_ZGVnN2v_sinh _ZGVsNxv_sinh _ZGVnN4v_sinhf _ZGVsNxv_sinhf	sinh sinhf	双曲正弦函数
_ZGVnN2v_cosh _ZGVsNxv_cosh _ZGVnN4v_coshf _ZGVsNxv_coshf	cosh coshf	双曲余弦函数

版权所有 © 华为技术有限公司

SVML接口	MATH接口	描述
_ZGVnN2v_tanh _ZGVsNxv_tanh _ZGVnN4v_tanhf _ZGVsNxv_tanhf	tanh tanhf	双曲正切函数
_ZGVnN2v_exp _ZGVsNxv_exp _ZGVnN4v_expf _ZGVsNxv_expf	exp expf	指数函数（base e）
_ZGVnN2v_exp2 _ZGVsNxv_exp2 _ZGVnN4v_exp2f _ZGVsNxv_exp2f	exp2 exp2f	指数函数（base 2）
_ZGVnN2v_expm1 _ZGVsNxv_expm1 _ZGVnN4v_expm1f _ZGVsNxv_expm1f	expm1 expm1f	指数函数（base e-1）
_ZGVnN2v_log _ZGVsNxv_log _ZGVsNxv_logf _ZGVnN4v_logf	log logf	对数函数（base e）
_ZGVnN2v_log10 _ZGVsNxv_log10 _ZGVnN4v_log10f _ZGVsNxv_log10f	log10 log10f	对数函数（base 10）
_ZGVnN2v_log1p _ZGVsNxv_log1p _ZGVnN4v_log1pf _ZGVsNxv_log1pf	log1p log1pf	对数函数（base e）
_ZGVnN2vv_pow _ZGVsNxvv_pow _ZGVnN4vv_powf _ZGVsNxvv_powf	pow powf	幂函数

版权所有 © 华为技术有限公司

须知

以上接口为提供给编译器调用的接口，不推荐用户直接调用。

6.4.5.3.2 GCC 向量化方法

若要使用GCC自动向量化，必须使用GCC 9.3.0及以上版本进行编译。

使用方法如下：

1. 头文件km.h中
   - 单精度接口定义前添加#pragma omp declare simd simdlen(4) notinbranch。
   - 在双精度接口定义前添加#pragma omp declare simd simdlen(2) notinbranch。
2. 在链接选项中同时加上-lkm -lksvml -lm以提供向量接口，在编译选项中加上-fopenmp-simd -fno-math-errno -O3，其中-O3是为了开启编译器向量化优化，这是向量化数学库起效的前提。

示例 1

km.h：

```
...
#pragma omp declare simd simdlen(4) notinbranch
float expf(float)
...
```

test.c：

```
#include <km.h>
#include <stdio.h>
#include <stdlib.h>
int main()
{
	long loop = 1e7;
	int len = 8192;
	float *a = (float*)malloc(sizeof(float) * len);
	float *b = (float*)malloc(sizeof(float) * len);
	float *d = (float*)malloc(sizeof(float) * len);
	for (int i = 0; i < len; i++) {
		a[i] = rand() * 7.7680f - 6.3840f;
		b[i] = rand() * 8.7680f - 6.3840f;
		d[i] = 0;
	}
	for (int j = 0; j < len; j++) {
		d[j] = expf(a[j]);
	}
	printf("%f\n", d[0]);
	return 1;
}
```

编译指令：

```
gcc test.c -lkm -lksvml -lm -fopenmp-simd -fno-math-errno -O3
```

使用nm命令查看调用接口：

```
nm -D a.out
```


版权所有 © 华为技术有限公司

```
U abort
w __gmon_start__
w _ITM_deregisterTMCloneTable
w _ITM_registerTMCloneTable
U __libc_start_main
U malloc
U printf
U rand
U _ZGVnN4v_expf
```

出现_ZGVnN4v_前缀接口表示调用成功。

示例 2

若代码中出现深层嵌套，如test.c：

```
#include <km.h>
#include <stdio.h>
#include <stdlib.h>
int main()
{
      long loop = 1e7;
      int len = 8192;
      float *a = (float*)malloc(sizeof(float) * len);
      float *b = (float*)malloc(sizeof(float) * len);
      float *d = (float*)malloc(sizeof(float) * len);
      for (int i = 0; i < len; i++) {
            a[i] = rand() * 7.7680f - 6.3840f;
            b[i] = rand() * 8.7680f - 6.3840f;
            d[i] = 0;
      }
      for (int i = 0; i < loop; i++) {
            for (int j = 0; j < len; j++) {
                  d[j] = expf(a[j]);
            }
      }
      printf("%f\n", d[0]);
      return 1;
}
```

则需要添加额外编译选项提示编译器不将外层循环和内层循环合并，编译指令：

```
gcc test.c -lkm -lksvml -lm -fopenmp-simd -fno-math-errno -O3 -fno-tree-loop-ivcanon -fno-loop-interchange
```

使用nm命令查看调用接口：

```
nm -D a.out
```

```
U abort
w __gmon_start__
w _ITM_deregisterTMCloneTable
w _ITM_registerTMCloneTable
U __libc_start_main
U malloc
U printf
U rand
U _ZGVnN4v_expf
```

出现_ZGVnN4v_前缀接口表示调用成功。

 版权所有 © 华为技术有限公司

6.4.5.3.3 毕昇编译器向量化方法

使用毕昇编译器时在链接选项中同时加上-lksvml -fveclib=MATHLIB -lkm -lm，并添加-O3或-O2选项以开启向量接口。其中-O3/-O2是为了开启向量化优化，这是向量化数学库起效的前提。

示例

test.c：

```
#include <km.h>
#include <stdio.h>
#include <stdlib.h>
int main()
{
	long loop = 1e7;
	int len = 8192;
	float *a = (float*)malloc(sizeof(float) * len);
	float *b = (float*)malloc(sizeof(float) * len);
	float *d = (float*)malloc(sizeof(float) * len);
	for (int i = 0; i < len; i++) {
		a[i] = rand() * 7.7680f - 6.3840f;
		b[i] = rand() * 8.7680f - 6.3840f;
		d[i] = 0;
	}
	for (int i = 0; i < loop; i++) {
		for (int j = 0; j < len; j++) {
			d[j] = expf(a[j]);
		}
	}
	printf("%f\n", d[0]);
	return 1;
}
```

编译指令：

```
clang test.c -lksvml -fveclib=MATHLIB -lkm -lm -O3
```

使用nm命令查看调用接口：

```
nm -D a.out
```

```
U abort
w __gmon_start__
w _ITM_deregisterTMCloneTable
w _ITM_registerTMCloneTable
U __libc_start_main
U malloc
U printf
U rand
U _ZGVnN4v_expf
```

出现_ZGVnN4v_前缀接口表示调用成功。

6.4.6 KML_VSL 库函数说明

6.4.6.1 函数说明

KML_VSL是C语言实现的统计库。KML_VSL使用优化后的基础随机数生成算法，提供多种常用的连续、离散分布的伪随机数生成函数。

 版权所有 © 华为技术有限公司

6.4.6.2 函数定义

6.4.6.2.1 直接法函数

6.4.6.2.1.1 初始化函数

初始化policy结构体并设置随机数种子。

接口定义

KmlVslResult kml_vsl_init(VslPolicy **policy, RNGtype type, uint32_t seed);

参数

参数名	类型	描述	输入/输出
policy	VslPolicy类型的二级指针	指向随机数生成器结构体的二级指针。	输出
type	RNGtype	随机数生成的算法。	输入
seed	uint32_t	随机数种子。	输入

依赖

#include "krng.h"

示例

```
#include <time.h>
#include <stdlib.h>
#include "krng.h"
int main()
{
    VslPolicy *policy;
    kml_vsl_init(&policy, LECUYER_CMRG, time(NULL));
    kml_vsl_destroy(policy);
}
```

6.4.6.2.1.2 内存释放函数

释放policy结构体内存，调用此函数后此policy不能再被使用。

接口定义

KmlVslResult kml_vsl_destroy(VslPolicy *policy);

版权所有 © 华为技术有限公司

参数

参数名	类型	描述	输入/输出
policy	VslPolicy类型的指针	指向随机数生成器结构体指针。	输入

依赖

#include "krng.h"

示例

```
#include <time.h>
#include <stdlib.h>
#include "krng.h"
int main()
{
    VslPolicy *policy;
    kml_vsl_init(&policy, LECUYER_CMRG, time(NULL));
    kml_vsl_destroy(policy);
}
```

6.4.6.2.1.3 设置随机数算法函数

改变policy中的随机数算法并重新随机设置种子。

接口定义

KmlVslResult kml_vsl_setRNGKind(VslPolicy *policy, RNGtype newkind);

参数

参数名	类型	描述	输入/输出
policy	VslPolicy类型的指针	指向随机数生成器结构体的指针。	输出
newkind	RNGtype	随机数生成的算法。	输入

依赖

#include "krng.h"

示例

```
#include <time.h>
#include <stdlib.h>
#include "krng.h"
int main()
{
```


版权所有 © 华为技术有限公司

```
    VslPolicy *policy;
    kml_vsl_init(&policy, LECUYER_CMRG, time(NULL));
    kml_vsl_setRNGKind(policy, WICHMANN_HILL);
    kml_vsl_destroy(policy);
}
```

6.4.6.2.1.4 设置随机种子函数

重新设置policy中的随机种子。

接口定义

KmlVslResult kml_vsl_setSeed(VslPolicy *policy, uint32_t seed);

参数

参数名	类型	描述	输入/输出
policy	VslPolicy类型的指针	指向随机数生成器结构体的指针。	输出
seed	uint32_t	随机数种子。	输入

依赖

#include "krng.h"

示例

```
#include <stdio.h>
#include <time.h>
#include <stdlib.h>
#include "krng.h"
int main()
{
    VslPolicy *policy;
    kml_vsl_init(&policy, LECUYER_CMRG, time(NULL));
    kml_vsl_setRNGKind(policy, WICHMANN_HILL);
    kml_vsl_setSeed(policy, 0);
    kml_vsl_destroy(policy);
}
```

6.4.6.2.1.5 均匀分布函数

生成符合均匀分布（Uniform distribution）的随机数向量。

接口定义

KmlVslResult kml_vsl_runif(VslPolicy *policy, N01type normal_method, int len, double *dst, double min, double max)

版权所有 © 华为技术有限公司

概率密度函数

$$f(x)=\begin{cases}\dfrac{1}{b-a}, a<x<b\\ 0, else\end{cases}$$

参数

参数名	类型	描述	输入/输出
len	int	输出向量的长度。	输入
min	double	均匀分布的min。	输入
max	double	均匀分布的max。	输入
dst	double类型指针	指向结果的指针。	输出
policy	VslPolicy类型指针	指向随机数生成器结构体指针。	输入
normal_method	N01type类型枚举体	生成正态分布的算法。	输入

依赖

#include "krng.h"

示例

```
#include <stdio.h>
#include <time.h>
#include <stdlib.h>
#include "krng.h"
int main()
{
    VslPolicy *policy;
    kml_vsl_init(&policy, LECUYER_CMRG, time(NULL));
    int len = 10;
    double *dst = (double *)malloc(sizeof(double) * len);
    int a = kml_vsl_runif(policy, BOX_MULLER, len, dst, 0, 1);
    for (int i = 0; i < len; i++) {
        printf("%f ", dst[i]);
    }
    printf("\n");
    kml_vsl_destroy(policy);
}
```

6.4.6.2.1.6 贝塔分布函数

生成符合贝塔分布（Beta distribution）的随机数向量。

版权所有 © 华为技术有限公司

接口定义

KmlVslResult kml_vsl_rbeta(VslPolicy *policy, N01type normal_method, int len, double *dst, double shape1, double shape2)

概率密度函数

$$f(x)=\frac{1}{B(\alpha,\beta)}x^{\alpha-1}(1-x)^{\beta-1}$$

参数

参数名	类型	描述	输入/输出
len	int	输出向量的长度。	输入
shape1	double	Shape1参数。	输入
shape2	double	Shape2参数。	输入
dst	double类型指针	指向结果的指针。	输出
policy	VslPolicy类型指针	指向随机数生成器结构体指针。	输入
normal_method	N01type类型枚举体	生成正态分布的算法。	输入

依赖

#include "krng.h"

示例

```
#include <stdio.h>
#include <time.h>
#include <stdlib.h>
#include "krng.h"
int main()
{
    VslPolicy *policy;
    kml_vsl_init(&policy, LECUYER_CMRG, time(NULL));
    int len = 10;
    double *dst = (double *)malloc(sizeof(double) * len);
    int a = kml_vsl_rbeta(policy, BOX_MULLER, len, dst, 1, 1);
    for (int i = 0; i < len; i++) {
        printf("%f ", dst[i]);
    }
    printf("\n");
    kml_vsl_destroy(policy);
}
```

 版权所有 © 华为技术有限公司

6.4.6.2.1.7 柯西分布函数

生成符合柯西分布（Cauchy distribution）的随机数向量。

接口定义

KmlVslResult kml_vsl_rcauchy(VslPolicy *policy, N01type normal_method, int len, double *dst, double location, double scale)

概率密度函数

$$f(x)=\frac{1}{\pi y(1+(\frac{x-x_0}{\gamma})^2)}$$

参数

参数名	类型	描述	输入/输出
len	int	输出向量的长度。	输入
location	double	Location参数。	输入
scale	double	Scale参数。	输入
dst	double类型指针	指向结果的指针。	输出
policy	VslPolicy类型指针	指向随机数生成器结构体指针。	输入
normal_method	N01type类型枚举体	生成正态分布的算法。	输入

依赖

#include "krng.h"

示例

```
#include <stdio.h>
#include <time.h>
#include <stdlib.h>
#include "krng.h"
int main()
{
    VslPolicy *policy;
    kml_vsl_init(&policy, LECUYER_CMRG, time(NULL));
    int len = 10;
    double *dst = (double *)malloc(sizeof(double) * len);
    int a = kml_vsl_rcauchy(policy, BOX_MULLER, len, dst, 0, 1);
```

 版权所有 © 华为技术有限公司

```
    for (int i = 0; i < len; i++) {
        printf("%f ", dst[i]);
    }
    printf("\n");
    kml_vsl_destroy(policy);
}
```

6.4.6.2.1.8 指数分布函数

生成符合指数分布（Exponential distribution）的随机数向量。

接口定义

KmlVslResult kml_vsl_rexp(VslPolicy *policy, N01type normal_method, int len, double *dst, double rate)

概率密度函数

$$f(x)=\lambda e^{\lambda x}$$

参数

参数名	类型	描述	输入/输出
len	int	输出向量的长度。	输入
rate	double	lambda参数。	输入
dst	double类型指针	指向结果的指针。	输出
policy	VslPolicy类型指针	指向随机数生成器结构体指针。	输入
normal_method	N01type类型枚举体	生成正态分布的算法。	输入

依赖

#include "krng.h"

示例

```
#include <stdio.h>
#include <time.h>
#include <stdlib.h>
#include "krng.h"
int main()
{
    VslPolicy *policy;
    kml_vsl_init(&policy, LECUYER_CMRG, time(NULL));
    int len = 10;
    double *dst = (double *)malloc(sizeof(double) * len);
    int a = kml_vsl_rexp(policy, BOX_MULLER, len, dst, 1);
    for (int i = 0; i < len; i++) {
```

 版权所有 © 华为技术有限公司

```
        printf("%f ", dst[i]);
    }
    printf("\n");
    kml_vsl_destroy(policy);
}
```

6.4.6.2.1.9 伽马分布函数

生成符合伽马分布（Gamma distribution）的随机数向量。

接口定义

KmlVslResult kml_vsl_rgamma(VslPolicy *policy, N01type normal_method, int len, double *dst, double location, double scale)

概率密度函数

$$f(x)=\frac{1}{\Gamma(k)\theta^{k}}x^{k-1}e^{-\frac{x}{\theta}}$$

参数

参数名	类型	描述	输入/输出
len	int	输出向量的长度。	输入
location	double	location参数。	输入
scale	double	scale参数。	输入
dst	double类型指针	指向结果的指针。	输出
policy	VslPolicy类型指针	指向随机数生成器结构体指针。	输入
normal_method	N01type类型枚举体	生成正态分布的算法。	输入

依赖

#include "krng.h"

示例

```
#include <stdio.h>
#include <time.h>
#include <stdlib.h>
#include "krng.h"
int main()
{
    VslPolicy *policy;
```

 版权所有 © 华为技术有限公司

```
    kml_vsl_init(&policy, LECUYER_CMRG, time(NULL));
    int len = 10;
    double *dst = (double *)malloc(sizeof(double) * len);
    int a = kml_vsl_rgamma(policy, BOX_MULLER, len, dst, 1, 1);
    for (int i = 0; i < len; i++) {
        printf("%f ", dst[i]);
    }
    printf("\n");
    kml_vsl_destroy(policy);
}
```

6.4.6.2.1.10 对数正态分布函数

生成符合对数正态分布（Log-normal distribution）的随机数向量。

接口定义

KmlVslResult kml_vsl_rlnorm(VslPolicy *policy, N01type normal_method, int len, double *dst, double meanlog, double sdlog)

概率密度函数

$$f(x)=\Phi(\frac{ln(x)-\mu}{\sigma})$$

参数

参数名	类型	描述	输入/输出
len	int	输出向量的长度。	输入
meanlog	double	Log mean参数。	输入
sdlog	double	Log SD参数。	输入
dst	double类型指针	指向结果的指针。	输出
policy	VslPolicy类型指针	指向随机数生成器结构体指针。	输入
normal_method	N01type类型枚举体	生成正态分布的算法。	输入

依赖

#include "krng.h"

示例

```
#include <stdio.h>
#include <time.h>
```


版权所有 © 华为技术有限公司

```
#include <stdlib.h>
#include "krng.h"
int main()
{
    VslPolicy *policy;
    kml_vsl_init(&policy, LECUYER_CMRG, time(NULL));
    int len = 10;
    double *dst = (double *)malloc(sizeof(double) * len);
    int a = kml_vsl_rlnorm(policy, BOX_MULLER, len, dst, 0, 1);
    for (int i = 0; i < len; i++) {
        printf("%f ", dst[i]);
    }
    printf("\n");
    kml_vsl_destroy(policy);
}
```

6.4.6.2.1.11 正态分布函数

生成符合正态分布（Normal distribution）的随机数向量。

接口定义

KmlVslResult kml_vsl_rnorm(VslPolicy *policy, N01type normal_method, int len, double *dst, double mean, double sd)

概率密度函数

$$f(x)=\frac{1}{\sqrt{2\pi}\sigma}exp(-\frac{(x-\mu)^2}{2\sigma^2})$$

参数

参数名	类型	描述	输入/输出
len	int	输出向量的长度。	输入
mean	double	mean参数。	输入
sd	double	SD参数。	输入
dst	double类型指针	指向结果的指针。	输出
policy	VslPolicy类型指针	指向随机数生成器结构体指针。	输入
normal_method	N01type类型枚举体	生成正态分布的算法。	输入

依赖

#include "krng.h"

 版权所有 © 华为技术有限公司

示例

```
#include <stdio.h>
#include <time.h>
#include <stdlib.h>
#include "krng.h"
int main()
{
    VslPolicy *policy;
    kml_vsl_init(&policy, LECUYER_CMRG, time(NULL));
    int len = 10;
    double *dst = (double *)malloc(sizeof(double) * len);
    int a = kml_vsl_rnorm(policy, BOX_MULLER, len, dst, 0, 1);
    for (int i = 0; i < len; i++) {
        printf("%f ", dst[i]);
    }
    printf("\n");
    kml_vsl_destroy(policy);
}
```

6.4.6.2.1.12 威布尔分布函数

生成符合威布尔分布（Weibull distribution）的随机数向量。

接口定义

KmlVslResult kml_vsl_rweibull(VslPolicy *policy, N01type normal_method, int len, double *dst, double shape, double scale)

概率密度函数

$$f(x)=\begin{cases}\frac{k}{\lambda}(\frac{x}{\lambda})^{k-1}e^{-(x/\lambda)^k}, x\geq 0\\ 0, x<0\end{cases}$$

参数

参数名	类型	描述	输入/输出
len	int	输出向量的长度。	输入
shape	double	shape参数。	输入
scale	double	scale参数。	输入
dst	double类型指针	指向结果的指针。	输出
policy	VslPolicy类型指针	指向随机数生成器结构体指针。	输入
normal_method	N01type类型枚举体	生成正态分布的算法。	输入

版权所有 © 华为技术有限公司

依赖

#include "krng.h"

示例

```
#include <stdio.h>
#include <time.h>
#include <stdlib.h>
#include "krng.h"
int main()
{
    VslPolicy *policy;
    kml_vsl_init(&policy, LECUYER_CMRG, time(NULL));
    int len = 10;
    double *dst = (double *)malloc(sizeof(double) * len);
    int a = kml_vsl_rweibull(policy, BOX_MULLER, len, dst, 0, 1);
    for (int i = 0; i < len; i++) {
        printf("%f ", dst[i]);
    }
    printf("\n");
    kml_vsl_destroy(policy);
}
```

6.4.6.2.1.13 二项分布函数

生成符合二项分布（Binomial distribution）的随机数向量。

接口定义

KmlVslResult kml_vsl_rbinom(VslPolicy *policy, N01type normal_method, int len, int *dst, int size, double prob)

概率密度函数

$$f(X=k)=\binom{n}{k}p^{k}(1-p)^{n-k}$$

参数

参数名	类型	描述	输入/输出
len	int	输出向量的长度。	输入
size	int	二项分布试验次数。	输入
prob	double	二项分布成功的概率。	输入
dst	int类型指针	指向结果的指针。	输出
policy	VslPolicy类型指针	指向随机数生成器结构体指针。	输入

版权所有 © 华为技术有限公司

参数名	类型	描述	输入/输出
normal_method	N01type类型枚举体	生成正态分布的算法。	输入

依赖

#include "krng.h"

示例

```
#include <stdio.h>
#include <time.h>
#include <stdlib.h>
#include "krng.h"
int main()
{
    VslPolicy *policy;
    kml_vsl_init(&policy, LECUYER_CMRG, time(NULL));
    int len = 10;
    int *dst = (int*)malloc(sizeof(int) * len);
    int a = kml_vsl_rbinom(policy, BOX_MULLER, len, dst, 10, 0.1);
    for (int i = 0; i < len; i++) {
        printf("%d ", dst[i]);
    }
    printf("\n");
    kml_vsl_destroy(policy);
}
```

6.4.6.2.1.14 超几何分布函数

生成符合超几何分布（Hypergeometric distribution）的随机数向量。

接口定义

KmlVslResult kml_vsl_rhyper(VslPolicy *policy, N01type normal_method, int len, double *dst, int m, int n, int k)

概率密度函数

$$f(X=k)=\frac{\binom{M}{k}\binom{N-M}{n-k}}{\binom{N}{n}}$$

版权所有 © 华为技术有限公司

参数

参数名	类型	描述	输入/输出
len	int	输出向量的长度。	输入
m	int	M参数。	输入
n	int	N参数。	输入
k	int	K参数。	输入
dst	int类型指针	指向结果的指针。	输出
policy	VslPolicy类型指针	指向随机数生成器结构体指针。	输入
normal_method	N01type类型枚举体	生成正态分布的算法。	输入

依赖

#include "krng.h"

示例

```
#include <stdio.h>
#include <time.h>
#include <stdlib.h>
#include "krng.h"
int main()
{
    VslPolicy *policy;
    kml_vsl_init(&policy, LECUYER_CMRG, time(NULL));
    int len = 10;
    int *dst = (int *)malloc(sizeof(int) * len);
    int a = kml_vsl_rhyper(policy, BOX_MULLER, len, dst, 10, 2, 3);
    for (int i = 0; i < len; i++) {
        printf("%d ", dst[i]);
    }
    printf("\n");
    kml_vsl_destroy(policy);
}
```

6.4.6.2.1.15 几何分布函数

生成符合几何分布（Geometric distribution）的随机数向量。

接口定义

KmlVslResult kml_vsl_rgeom(VslPolicy *policy, N01type normal_method, int len, int *dst, double prob)

概率密度函数

$$f(X=k)=(1-p)^{k-1}p$$

版权所有 © 华为技术有限公司

参数

参数名	类型	描述	输入/输出
len	int	输出向量的长度。	输入
prob	double	概率参数。	输入
dst	int类型指针	指向结果的指针。	输出
policy	VslPolicy类型指针	指向随机数生成器结构体指针。	输入
normal_method	N01type类型枚举体	生成正态分布的算法。	输入

依赖

#include "krng.h"

示例

```
#include <stdio.h>
#include <time.h>
#include <stdlib.h>
#include "krng.h"
int main()
{
    VslPolicy *policy;
    kml_vsl_init(&policy, LECUYER_CMRG, time(NULL));
    int len = 10;
    int *dst = (int *)malloc(sizeof(int) * len);
    int a = kml_vsl_rgeom(policy, BOX_MULLER, len, dst, 0.1);
    for (int i = 0; i < len; i++) {
        printf("%d ", dst[i]);
    }
    printf("\n");
    kml_vsl_destroy(policy);
}
```

6.4.6.2.1.16 负二项分布函数

生成符合负二项分布（Negative binomial distribution）的随机数向量。

接口定义

KmlVslResult kml_vsl_rnbinom(VslPolicy *policy, N01type normal_method, int len, int *dst, double size, double mu)

概率密度函数

$$f\left(X=k \right)=\left(\begin{matrix} k+r-1 \\ r-1 \end{matrix} \right)\left(1-p \right)^{k}p^{r}$$

版权所有 © 华为技术有限公司

参数

参数名	类型	描述	输入/输出
len	int	输出向量的长度。	输入
size	int	size参数。	输入
mu	double	Mu参数。	输入
dst	int类型指针	指向结果的指针。	输出
policy	VslPolicy类型指针	指向随机数生成器结构体指针。	输入
normal_method	N01type类型枚举体	生成正态分布的算法。	输入

依赖

#include "krng.h"

示例

```
#include <stdio.h>
#include <time.h>
#include <stdlib.h>
#include "krng.h"
int main()
{
    VslPolicy *policy;
    kml_vsl_init(&policy, LECUYER_CMRG, time(NULL));
    int len = 10;
    int *dst = (int *)malloc(sizeof(int) * len);
    int a = kml_vsl_rnbinom(policy, BOX_MULLER, len, dst, 10, 0.1);
    for (int i = 0; i < len; i++) {
        printf("%d ", dst[i]);
    }
    printf("\n");
    kml_vsl_destroy(policy);
}
```

6.4.6.2.1.17 泊松分布函数

生成符合泊松分布（Poisson distribution）的随机数向量。

接口定义

KmlVslResult kml_vsl_rpois(VslPolicy *policy, N01type normal_method, int len, int *dst, double lambda)

概率密度函数

$$f(X=k)=\frac{\lambda^{k}}{k!}e^{-\lambda}$$

版权所有 © 华为技术有限公司

参数

参数名	类型	描述	输入/输出
len	int	输出向量的长度。	输入
lambda	double	lambda参数。	输入
dst	int类型指针	指向结果的指针。	输出
policy	VslPolicy类型指针	指向随机数生成器结构体指针。	输入
normal_method	N01type类型枚举体	生成正态分布的算法。	输入

依赖

#include "krng.h"

示例

```
#include <stdio.h>
#include <time.h>
#include <stdlib.h>
#include "krng.h"
int main()
{
    VslPolicy *policy;
    kml_vsl_init(&policy, LECUYER_CMRG, time(NULL));
    int len = 10;
    int *dst = (int *)malloc(sizeof(int) * len);
    int a = kml_vsl_rpois(policy, BOX_MULLER, len, dst, 10);
    for (int i = 0; i < len; i++) {
        printf("%d ", dst[i]);
    }
    printf("\n");
    kml_vsl_destroy(policy);
}
```

6.4.6.2.1.18 版本查询函数

获取VSL产品版本信息。

接口定义

KmlVslResult kml_vsl_getVersion(KVSLVersion *ver);

参数

参数名	描述	取值范围	输入/输出
ver	产品信息结构体。	非空	输出

版权所有 © 华为技术有限公司

返回值

- 成功：返回1。
- 失败：返回0。

示例

```
#include <stdio.h>
#include "krng.h"

int main(void)
{
    KVSLVersion ver;
    KmlVslResult ret = kml_vsl_getVersion(&ver);
    if (ret == 1) {
        printf("Vsl\n");
        printf("Product Name:          %s\n", ver.productName);
        printf("Product Version:       %s\n", ver.productVersion);
        printf("Component Name:        %s\n", ver.componentName);
        printf("Component Version:     %s\n", ver.componentVersion);
        printf("Component AbiVersion: %s\n", ver.componentAbiVersion);
        printf("Component AppendInfo: %s\n", ver.componentAppendInfo);
        printf("Software Name:         %s\n", ver.softwareName);
        printf("Software Version:      %s\n", ver.softwareVersion);
    }
}
```

运行结果：

```
Vsl
Product Name:          Kunpeng HPCKit
Product Version:       24.0.RC1
Component Name:        HPCKit-kml
Component Version:     2.3.0
Component AbiVersion: 1
Component AppendInfo: gcc
Software Name:         hpckit-kml-vsl
Software Version:      2.3.0
```

说明

版本号和编译时间以实际运行结果为准，上述结果仅供参考。

6.4.6.2.2 高性能函数

6.4.6.2.2.1 创建随机数流函数

初始化stream结构体并设置随机数种子。

接口定义

int vslNewStream(VSLStreamStatePtr *stream, const int brng, const unsigned seed);

版权所有 © 华为技术有限公司

参数

参数名	类型	描述	输入/输出
stream	VSLStreamStatePtr类型指针	指向随机数流结构体的指针。	输出
brng	int	随机数生成器类型。 ● VSL_BRNG_MCG31：31位乘法同余生成器 ● VSL_BRNG_MRG32K3A：带两个3阶组件的组合式多种递归生成器 ● VSL_BRNG_MCG59：59位乘法同余生成器 ● VSL_BRNG_SOBOL：Sobol低差异序列生成器 ● VSL_BRNG_MT19937：梅森旋转算法19937生成器 ● VSL_BRNG_PHILOX4X32X10：PHILOX4X32X10生成器 ● VSL_BRNG_RNDR：RNDR生成器 ● VSL_BRNG_RNDRRS：RNDRRS生成器 ● VSL_BRNG_ARS5：ARS5生成器 ● VSL_BRNG_MT2203：梅森旋转算法2203生成器	输入
seed	unsigned	随机数种子，为非负整数。	输入

依赖

#include "krng.h"

示例

```
#include <stdio.h>
#include <stdlib.h>
#include "krng.h"
int main()
{
   /* initialize stream with given BRNG type and seed */
    VSLStreamStatePtr stream;
    unsigned seed = 42;
    int errcode = vslNewStream(&stream, VSL_BRNG_MCG59, seed);
    if (errcode != VSL_STATUS_OK) {
        fprintf(stderr, "Failure in newstream\n");
        return 0;
    }
```

 版权所有 © 华为技术有限公司

```
    /* deinitialize the stream */
    errcode = vslDeleteStream(&stream);
    if (errcode != VSL_STATUS_OK) {
        fprintf(stderr, "Failure in deleting stream\n");
        return 0;
    }
    return 0;
}
```

6.4.6.2.2.2 释放随机数流函数

释放stream结构体内存，调用此函数后此stream不能再被使用。

接口定义

int vslDeleteStream(VSLStreamStatePtr *stream);

参数

参数名	类型	描述	输入/输出
stream	VSLStreamStatePtr类型指针	指向随机数流结构体的指针。	输入

依赖

#include "krng.h"

示例

```
#include <stdio.h>
#include <stdlib.h>
#include "krng.h"
int main()
{
   /* initialize stream with given BRNG type and seed */
    VSLStreamStatePtr stream;
    unsigned seed = 42;
    int errcode = vslNewStream(&stream, VSL_BRNG_MCG59, seed);
    if (errcode != VSL_STATUS_OK) {
        fprintf(stderr, "Failure in newstream\n");
        return 0;
    }

    /* deinitialize the stream */
    errcode = vslDeleteStream(&stream);
    if (errcode != VSL_STATUS_OK) {
        fprintf(stderr, "Failure in deleting stream\n");
        return 0;
    }
    return 0;
}
```

6.4.6.2.2.3 跳过随机数流元素函数

跳过随机数流stream中的元素。

 版权所有 © 华为技术有限公司

接口定义

int vslSkipAheadStream(VSLStreamStatePtr stream, const SizeType nskip);

参数

参数名	类型	描述	输入/输出
stream	VSLStreamStatePtr类型指针	指向随机数流结构体的指针。	输入、输出
nskip	SizeType	跳过的元素个数，非负整数。	输入

依赖

#include "krng.h"

示例

```
#include <stdio.h>
#include <stdlib.h>
#include "krng.h"
int main()
{
   /* initialize stream with given BRNG type and seed */
    VSLStreamStatePtr stream;
    unsigned seed = 42;
    int errcode = vslNewStream(&stream, VSL_BRNG_MCG59, seed);
    if (errcode != VSL_STATUS_OK) {
        fprintf(stderr, "Failure in newstream\n");
        return 0;
    }

    int forward_skip = 10;
    if (vslSkipAheadStream(stream, forward_skip)) {
        fprintf(stderr, "Failure in SkipAhead, skip = %d\n", forward_skip);
        return 0;
    }

    /* deinitialize the stream */
    errcode = vslDeleteStream(&stream);
    if (errcode != VSL_STATUS_OK) {
        fprintf(stderr, "Failure in deleting stream\n");
        return 0;
    }
    return 0;
}
```

6.4.6.2.2.4 分配随机数流函数

分配随机数流stream。

接口定义

int vslLeapfrogStream(VSLStreamStatePtr stream, const SizeType k, const SizeType nstreams);

版权所有 © 华为技术有限公司

参数

参数名	类型	描述	输入/输出
stream	VSLStreamStatePtr类型指针	指向随机数流结构体的指针。	输入、输出
k	SizeType	节点下标。	输入
nstreams	SizeType	节点最大数量。	输入

依赖

#include "krng.h"

示例

```
#include <stdio.h>
#include <stdlib.h>
#include "krng.h"
int main()
{
   /* initialize stream with given BRNG type and seed */
    VSLStreamStatePtr stream;
    unsigned seed = 42;
    int errcode = vslNewStream(&stream, VSL_BRNG_MCG59, seed);
    if (errcode != VSL_STATUS_OK) {
        fprintf(stderr, "Failure in newstream\n");
        return 0;
    }

    int n = 10;
    int nstreams = 10;
    if (vslLeapfrogStream(stream, n, nstreams)) {
        fprintf(stderr, "Failure in vslLeapfrogStream, skip = %d\n", forward_skip);
        return 0;
    }

    /* deinitialize the stream */
    errcode = vslDeleteStream(&stream);
    if (errcode != VSL_STATUS_OK) {
        fprintf(stderr, "Failure in deleting stream\n");
        return 0;
    }
    return 0;
}
```

6.4.6.2.2.5 拷贝随机数流函数

拷贝随机数流stream。

接口定义

int vslCopyStream(VSLStreamStatePtr *newStream, VSLStreamStatePtr stream);

版权所有 © 华为技术有限公司

参数

参数名	类型	描述	输入/输出
newStream	VSLStreamStatePtr类型	新随机数流。	输出
stream	VSLStreamStatePtr类型指针	指向被拷贝随机数流结构体的指针。	输入

依赖

#include "krng.h"

示例

```
#include <stdio.h>
#include <stdlib.h>
#include "krng.h"
int main()
{
   /* initialize stream with given BRNG type and seed */
    VSLStreamStatePtr stream;
    unsigned seed = 42;
    int errcode = vslNewStream(&stream, VSL_BRNG_MCG59, seed);
    if (errcode != VSL_STATUS_OK) {
        fprintf(stderr, "Failure in newstream\n");
        return 0;
    }

    VSLStreamStatePtr stream2;
    if (vslCopyStream(&stream2, stream)) {
        fprintf(stderr, "Failure in vslCopyStream\n");
        return 0;
    }

    /* deinitialize the stream */
    errcode = vslDeleteStream(&stream);
    if (errcode != VSL_STATUS_OK) {
        fprintf(stderr, "Failure in deleting stream\n");
        return 0;
    }
    return 0;
}
```

6.4.6.2.2.6 均匀分布函数

生成符合均匀分布（Uniform distribution）的随机数向量。

接口定义

int vsRngUniform (const int method, VSLStreamStatePtr stream, const SizeType n, float *r, const float a, const float b);

int vdRngUniform (const int method, VSLStreamStatePtr stream, const SizeType n, double *r, const double a, const double b);

int viRngUniform(const int method, VSLStreamStatePtr stream, const SizeType n, int *r, const int a, const int b);

 版权所有 © 华为技术有限公司

int viRngUniformBits32(const int method, VSLStreamStatePtr stream, const SizeType n, uint32_t *r);

int viRngUniformBits64(const int method, VSLStreamStatePtr stream, const SizeType n, uint64_t *r);

int viRngUniformBits(const int method, VSLStreamStatePtr stream, const SizeType n, unsigned int *r);

概率密度函数

$$f(x)=\begin{cases}\dfrac{1}{b-a}, a<x<b\\ 0, else\end{cases}$$

参数

参数名	类型	描述	输入/输出
method	int	随机数生成模式。 ● VSL_RNG_METHOD_UNIFORM_STD：标准模式 ● VSL_RNG_METHOD_UNIFORM_STD_ACCURATE：精确模式	输入
stream	VSLStreamStatePtr类型	随机数流。	输入
n	SizeType	输出向量的长度。	输入
r	● 在vsRngUniform中是单精度浮点类型指针。 ● 在vdRngUniform中是双精度浮点类型指针。 ● 在viRngUniform中是整型指针。 ● 在viRngUniformBits32中是32位无符号整型指针。 ● 在viRngUniformBits64中是64位无符号整型指针。 ● 在viRngUniformBits中是无符号整型指针。	指向结果的指针。	输出

版权所有 © 华为技术有限公司

参数名	类型	描述	输入/输出
a	• 在vsRngUniform中是单精度浮点类型。 • 在vdRngUniform中是双精度浮点类型。 • 在viRngUniform中是整型。	均匀分布的min。	输入
b	• 在vsRngUniform中是单精度浮点类型。 • 在vdRngUniform中是双精度浮点类型。 • 在viRngUniform中是整型。	均匀分布的max。	输入

依赖

#include "krng.h"

示例

```
#include <stdio.h>
#include <stdlib.h>
#include "krng.h"
int main()
{
   /* initialize stream with given BRNG type and seed */
    VSLStreamStatePtr stream;
    unsigned seed = 42;
    int errcode = vslNewStream(&stream, VSL_BRNG_MCG59, seed);
    if (errcode != VSL_STATUS_OK) {
        fprintf(stderr, "Failure in newstream\n");
        return 0;
    }

    SizeType n = 10;
    float *r = (float *)malloc(sizeof(float) * n);
    if (r == NULL) {
        fprintf(stderr, "Failure in malloc\n");
        return 0;
    }

    if (vsRngUniform(VSL_RNG_METHOD_UNIFORM_STD, stream, n, r, 0.0, 1.0)) {
        fprintf(stderr, "Failure in vsRngUniform\n");
        goto out;
    }

    /* deinitialize the stream */
    errcode = vslDeleteStream(&stream);
    if (errcode != VSL_STATUS_OK) {
        fprintf(stderr, "Failure in deleting stream\n");
        goto out;
    }

out:
    free(r);
```


版权所有 © 华为技术有限公司

```
    return 0;
}
```

6.4.6.2.2.7 柯西分布函数

生成符合柯西分布（Cauchy distribution）的随机数向量。

接口定义

int vsRngCauchy (const int method, VSLStreamStatePtr stream, const SizeType n, float *r, const float a, const float beta);

int vdRngCauchy (const int method, VSLStreamStatePtr stream, const SizeType n, double *r, const double a, const double beta);

概率密度函数

$$f(x)=\frac{1}{\pi y(1+(\frac{x-x_0}{\gamma})^2)}$$

参数

参数名	类型	描述	输入/输出
method	int	随机数生成模式。 ● VSL_RNG_METHOD_CAUCHY_ICDF：逆累积分布函数法模式	输入
stream	VSLStreamStatePtr类型	随机数流。	输入
n	SizeType	输出向量的长度。	输入
r	● 在vsRngCauchy中是单精度浮点类型指针。 ● 在vdRngCauchy中是双精度浮点类型指针。	指向结果的指针。	输出
a	● 在vsRngCauchy中是单精度浮点类型。 ● 在vdRngCauchy中是双精度浮点类型。	柯西分布的location参数。	输入
beta	● 在vsRngCauchy中是单精度浮点类型。 ● 在vdRngCauchy中是双精度浮点类型。	柯西分布的scale参数。	输入

版权所有 © 华为技术有限公司

依赖

#include "krng.h"

示例

```
#include <stdio.h>
#include <stdlib.h>
#include "krng.h"
int main()
{
   /* initialize stream with given BRNG type and seed */
    VSLStreamStatePtr stream;
    unsigned seed = 42;
    int errcode = vslNewStream(&stream, VSL_BRNG_MCG59, seed);
    if (errcode != VSL_STATUS_OK) {
        fprintf(stderr, "Failure in newstream\n");
        return 0;
    }

    SizeType n = 10;
    float *r = (float *)malloc(sizeof(float) * n);
    if (r == NULL) {
        fprintf(stderr, "Failure in malloc\n");
        return 0;
    }

    if (vsRngCauchy(VSL_RNG_METHOD_CAUCHY_ICDF, stream, n, r, 0.0, 1.0)) {
        fprintf(stderr, "Failure in vsRngCauchy\n");
        goto out;
    }

    /* deinitialize the stream */
    errcode = vslDeleteStream(&stream);
    if (errcode != VSL_STATUS_OK) {
        fprintf(stderr, "Failure in deleting stream\n");
        goto out;
    }

out:
    free(r);
    return 0;
}
```

6.4.6.2.2.8 指数分布函数

生成符合指数分布（Exponential distribution）的随机数向量。

接口定义

int vsRngExponential (const int method, VSLStreamStatePtr stream, const SizeType n, float *r, const float a, const float beta);

int vdRngExponential (const int method, VSLStreamStatePtr stream, const SizeType n, double *r, const double a, const double beta);

概率密度函数

$$f(x)=\lambda e^{\lambda x}$$

版权所有 © 华为技术有限公司

参数

参数名	类型	描述	输入/输出
method	int	随机数生成模式。 • VSL_RNG_METHOD_EXPONENTIAL_ICDF：逆累积分布函数法模式 • VSL_RNG_METHOD_EXPONENTIAL_ICDF_ACCURATE：逆累积分布函数法精确模式	输入
stream	VSLStreamStatePtr类型	随机数流。	输入
n	SizeType	输出向量的长度。	输入
r	• 在vsRngExponential中是单精度浮点类型指针。 • 在vdRngExponential中是双精度浮点类型指针。	指向结果的指针。	输出
a	• 在vsRngExponential中是单精度浮点类型。 • 在vdRngExponential中是双精度浮点类型。	对数运算的结果加数。	输入
beta	• 在vsRngExponential中是单精度浮点类型。 • 在vdRngExponential中是双精度浮点类型。	对数运算的结果乘数。	输入

依赖

#include "krng.h"

示例

```
#include <stdio.h>
#include <stdlib.h>
#include "krng.h"
int main()
{
   /* initialize stream with given BRNG type and seed */
    VSLStreamStatePtr stream;
    unsigned seed = 42;
    int errcode = vslNewStream(&stream, VSL_BRNG_MCG59, seed);
    if (errcode != VSL_STATUS_OK) {
        fprintf(stderr, "Failure in newstream\n");
        return 0;
    }

    SizeType n = 10;
    float *r = (float *)malloc(sizeof(float) * n);
```


版权所有 © 华为技术有限公司

```
    if (r == NULL) {
        fprintf(stderr, "Failure in malloc\n");
        return 0;
    }

    if (vsRngExponential(VSL_RNG_METHOD_EXPONENTIAL_ICDF, stream, n, r, 0.0, 1.0)) {
        fprintf(stderr, "Failure in vsRngExponential\n");
        goto out;
    }

    /* deinitialize the stream */
    errcode = vslDeleteStream(&stream);
    if (errcode != VSL_STATUS_OK) {
        fprintf(stderr, "Failure in deleting stream\n");
        goto out;
    }

out:
    free(r);
    return 0;
}
```

6.4.6.2.2.9 伽马分布函数

生成符合伽马分布（Gamma distribution）的随机数向量。

接口定义

int vsRngGamma (const int method, VSLStreamStatePtr stream, const SizeType n, float *r, const float alpha, const float a, const float beta);

int vdRngGamma (const int method, VSLStreamStatePtr stream, const SizeType n, double *r, const double alpha, const double a, const double beta);

概率密度函数

$$f(x)=\frac{1}{\Gamma(k)\theta^{k}}x^{k-1}e^{-\frac{x}{\theta}}$$

参数

参数名	类型	描述	输入/输出
method	int	随机数生成模式。 • VSL_RNG_METHOD_GAMMA_FAST：快速模式	输入
stream	VSLStreamStatePtr类型	随机数流。	输入
n	SizeType	输出向量的长度。	输入
r	• 在vsRngGamma中是单精度浮点类型指针。 • 在vdRngGamma中是双精度浮点类型指针。	指向结果的指针。	输出

版权所有 © 华为技术有限公司

参数名	类型	描述	输入/输出
alpha	• 在vsRngGamma中是单精度浮点类型。 • 在vdRngGamma中是双精度浮点类型。	alpha参数，不同的值对应不同的算法。	输入
a	• 在vsRngGamma中是单精度浮点类型。 • 在vdRngGamma中是双精度浮点类型。	运算的结果加数。	输入
beta	• 在vsRngGamma中是单精度浮点类型。 • 在vdRngGamma中是双精度浮点类型。	运算的结果乘数。	输入

依赖

#include "krng.h"

示例

```
#include <stdio.h>
#include <stdlib.h>
#include "krng.h"
int main()
{
   /* initialize stream with given BRNG type and seed */
    VSLStreamStatePtr stream;
    unsigned seed = 42;
    int errcode = vslNewStream(&stream, VSL_BRNG_MCG59, seed);
    if (errcode != VSL_STATUS_OK) {
        fprintf(stderr, "Failure in newstream\n");
        return 0;
    }

    SizeType n = 10;
    float *r = (float *)malloc(sizeof(float) * n);
    if (r == NULL) {
        fprintf(stderr, "Failure in malloc\n");
        return 0;
    }

    if (vsRngGamma(VSL_RNG_METHOD_GAMMA_FAST, stream, n, r, 0.1, 3.1, 2.3)) {
        fprintf(stderr, "Failure in vsRngGamma\n");
        goto out;
    }

    /* deinitialize the stream */
    errcode = vslDeleteStream(&stream);
    if (errcode != VSL_STATUS_OK) {
        fprintf(stderr, "Failure in deleting stream\n");
        goto out;
    }

out:
    free(r);
```


版权所有 © 华为技术有限公司

```
    return 0;
}
```

6.4.6.2.2.10 对数正态分布函数

生成符合对数正态分布（Log-normal distribution）的随机数向量。

接口定义

int vsRngLognormal (const int method, VSLStreamStatePtr stream, const SizeType n, float *r, const float a, const float sigma, const float b, const float beta);

int vdRngLognormal (const int method, VSLStreamStatePtr stream, const SizeType n, double *r, const double a, const double sigma, const double b, const double beta);

概率密度函数

$$f(x)=\Phi(\frac{ln(x)-\mu}{\sigma})$$

参数

参数名	类型	描述	输入/输出
method	int	随机数生成模式。 ● VSL_RNG_METHOD_LOGNORMAL_ICDF：逆累积分布函数法模式 ● VSL_RNG_METHOD_LOGNORMAL_ICDF_ACCURATE：逆累积分布函数法精确模式	输入
stream	VSLStreamStatePtr类型	随机数流。	输入
n	SizeType	输出向量的长度。	输入
r	● 在vsRngLognormal中是单精度浮点类型指针。 ● 在vdRngLognormal中是双精度浮点类型指针。	指向结果的指针。	输出
a	● 在vsRngLognormal中是单精度浮点类型。 ● 在vdRngLognormal中是双精度浮点类型。	a参数，用于高斯分布逆累积分布函数（ICDF）生成。	输入

版权所有 © 华为技术有限公司

参数名	类型	描述	输入/输出
sigma	• 在vsRngLognormal中是单精度浮点类型。 • 在vdRngLognormal中是双精度浮点类型。	sigma参数，用于高斯分布逆累积分布函数（ICDF）生成。	输入
b	• 在vsRngLognormal中是单精度浮点类型。 • 在vdRngLognormal中是双精度浮点类型。	运算的结果加数。	输入
beta	• 在vsRngLognormal中是单精度浮点类型。 • 在vdRngLognormal中是双精度浮点类型。	运算的结果乘数。	输入

依赖

#include "krng.h"

示例

```
#include <stdio.h>
#include <stdlib.h>
#include "krng.h"
int main()
{
   /* initialize stream with given BRNG type and seed */
    VSLStreamStatePtr stream;
    unsigned seed = 42;
    int errcode = vslNewStream(&stream, VSL_BRNG_MCG59, seed);
    if (errcode != VSL_STATUS_OK) {
        fprintf(stderr, "Failure in newstream\n");
        return 0;
    }

    SizeType n = 10;
    float *r = (float *)malloc(sizeof(float) * n);
    if (r == NULL) {
        fprintf(stderr, "Failure in malloc\n");
        return 0;
    }

    if (vsRngLognormal(VSL_RNG_METHOD_LOGNORMAL_ICDF, stream, n, r, 5.0, 1.0, 6.0, 5.0)) {
        fprintf(stderr, "Failure in vsRngLognormal\n");
        goto out;
    }

    /* deinitialize the stream */
    errcode = vslDeleteStream(&stream);
    if (errcode != VSL_STATUS_OK) {
        fprintf(stderr, "Failure in deleting stream\n");
        goto out;
    }

out:
    free(r);
```

 版权所有 © 华为技术有限公司

```
    return 0;
}
```

6.4.6.2.2.11 威布尔分布函数

生成符合威布尔分布（Weibull distribution）的随机数向量。

接口定义

int vsRngWeibull (const int method, VSLStreamStatePtr stream, const SizeType n, float *r, const float alpha, const float a, const float beta);

int vdRngWeibull (const int method, VSLStreamStatePtr stream, const SizeType n, double *r, const double alpha, const double a, const double beta);

概率密度函数

$$f(x)=\begin{cases}\frac{k}{\lambda}(\frac{x}{\lambda})^{k-1}e^{-(x/\lambda)^k}, x\geq 0\\ 0, x<0\end{cases}$$

参数

参数名	类型	描述	输入/输出
method	int	随机数生成模式。 ● VSL_RNG_METHOD_WEIBULL_ICDF：逆累积分布函数法模式 ● VSL_RNG_METHOD_WEIBULL_ICDF_ACCURATE：逆累积分布函数法精确模式	输入
stream	VSLStreamStatePtr类型	随机数流。	输入
n	SizeType	输出向量的长度。	输入
r	● 在vsRngLognormal中是单精度浮点类型指针。 ● 在vdRngLognormal中是双精度浮点类型指针。	指向结果的指针。	输出
alpha	● 在vsRngLognormal中是单精度浮点类型。 ● 在vdRngLognormal中是双精度浮点类型。	alpha参数，倒数作为指数用于结果运算中的指数运算。	输入

版权所有 © 华为技术有限公司

参数名	类型	描述	输入/输出
a	• 在vsRngLognormal中是单精度浮点类型。 • 在vdRngLognormal中是双精度浮点类型。	运算的结果加数。	输入
beta	• 在vsRngLognormal中是单精度浮点类型。 • 在vdRngLognormal中是双精度浮点类型。	运算的结果乘数。	输入

依赖

#include "krng.h"

示例

```
#include <stdio.h>
#include <stdlib.h>
#include "krng.h"
int main()
{
   /* initialize stream with given BRNG type and seed */
    VSLStreamStatePtr stream;
    unsigned seed = 42;
    int errcode = vslNewStream(&stream, VSL_BRNG_MCG59, seed);
    if (errcode != VSL_STATUS_OK) {
        fprintf(stderr, "Failure in newstream\n");
        return 0;
    }

    SizeType n = 10;
    float *r = (float *)malloc(sizeof(float) * n);
    if (r == NULL) {
        fprintf(stderr, "Failure in malloc\n");
        return 0;
    }

    if (vsRngWeibull(VSL_RNG_METHOD_WEIBULL_ICDF, stream, n, r, 0.6, 0.5, 0.7)) {
        fprintf(stderr, "Failure in vsRngWeibull\n");
        goto out;
    }

    /* deinitialize the stream */
    errcode = vslDeleteStream(&stream);
    if (errcode != VSL_STATUS_OK) {
        fprintf(stderr, "Failure in deleting stream\n");
        goto out;
    }

out:
    free(r);
    return 0;
}
```

6.4.6.2.2.12 拉普拉斯分布函数

生成符合拉普拉斯分布（Laplace distribution）的随机数向量。

 版权所有 © 华为技术有限公司

接口定义

int vsRngLaplace (const int method, VSLStreamStatePtr stream, const SizeType n, float *r, const float a, const float beta);

int vdRngLaplace (const int method, VSLStreamStatePtr stream, const SizeType n, double *r, const double a, const double beta);

概率密度函数

$$f(x)=\frac{1}{2}e^{\frac{|x-\lambda|}{\lambda}}$$

参数

参数名	类型	描述	输入/输出
method	int	随机数生成模式。 • VSL_RNG_METHOD_LAPLACE_ICDF：逆累积分布函数法模式	输入
stream	VSLStreamStatePtr类型	随机数流。	输入
n	SizeType	输出向量的长度。	输入
r	• 在vsRngLaplace中是单精度浮点类型指针。 • 在vdRngLaplace中是双精度浮点类型指针。	指向结果的指针。	输出
a	• 在vsRngLaplace中是单精度浮点类型。 • 在vdRngLaplace中是双精度浮点类型。	运算的结果加数。	输入
beta	• 在vsRngLaplace中是单精度浮点类型。 • 在vdRngLaplace中是双精度浮点类型。	运算的结果乘数。	输入

依赖

#include "krng.h"

示例

```
#include <stdio.h>
#include <stdlib.h>
```

 版权所有 © 华为技术有限公司

```
#include "krng.h"
int main()
{
   /* initialize stream with given BRNG type and seed */
    VSLStreamStatePtr stream;
    unsigned seed = 42;
    int errcode = vslNewStream(&stream, VSL_BRNG_MCG59, seed);
    if (errcode != VSL_STATUS_OK) {
        fprintf(stderr, "Failure in newstream\n");
        return 0;
    }

    SizeType n = 10;
    float *r = (float *)malloc(sizeof(float) * n);
    if (r == NULL) {
        fprintf(stderr, "Failure in malloc\n");
        return 0;
    }

    if (vsRngLaplace(VSL_RNG_METHOD_LAPLACE_ICDF, stream, n, r, 0.0, 1.0)) {
        fprintf(stderr, "Failure in vsRngLaplace\n");
        goto out;
    }

    /* deinitialize the stream */
    errcode = vslDeleteStream(&stream);
    if (errcode != VSL_STATUS_OK) {
        fprintf(stderr, "Failure in deleting stream\n");
        goto out;
    }

out:
    free(r);
    return 0;
}
```

6.4.6.2.2.13 瑞利分布函数

生成符合瑞利分布（Rayleigh distribution）的随机数向量。

接口定义

int vsRngRayleigh (const int method, VSLStreamStatePtr stream, const SizeType n, float *r, const float a, const float beta);

int vdRngRayleigh (const int method, VSLStreamStatePtr stream, const SizeType n, double *r, const double a, const double beta);

概率密度函数

$$f(x)=\frac{x}{\sigma^2}e^{\frac{-x^2}{2\sigma^2}}$$

 版权所有 © 华为技术有限公司

参数

参数名	类型	描述	输入/输出
method	int	随机数生成模式。 • VSL_RNG_METHOD_RAYLEIGH_ICDF：逆累积分布函数法模式 • VSL_RNG_METHOD_RAYLEIGH_ICDF_ACCURATE：逆累积分布函数法精确模式	输入
stream	VSLStreamStatePtr类型	随机数流。	输入
n	SizeType	输出向量的长度。	输入
r	• 在vsRngLaplace中是单精度浮点类型指针。 • 在vdRngLaplace中是双精度浮点类型指针。	指向结果的指针。	输出
a	• 在vsRngLaplace中是单精度浮点类型。 • 在vdRngLaplace中是双精度浮点类型。	运算的结果加数。	输入
beta	• 在vsRngLaplace中是单精度浮点类型。 • 在vdRngLaplace中是双精度浮点类型。	运算的结果乘数。	输入

依赖

#include "krng.h"

示例

```
#include <stdio.h>
#include <stdlib.h>
#include "krng.h"
int main()
{
   /* initialize stream with given BRNG type and seed */
    VSLStreamStatePtr stream;
    unsigned seed = 42;
    int errcode = vslNewStream(&stream, VSL_BRNG_MCG59, seed);
    if (errcode != VSL_STATUS_OK) {
        fprintf(stderr, "Failure in newstream\n");
        return 0;
    }

    SizeType n = 10;
    float *r = (float *)malloc(sizeof(float) * n);
```

 版权所有 © 华为技术有限公司

```
    if (r == NULL) {
        fprintf(stderr, "Failure in malloc\n");
        return 0;
    }

    if (vsRngRayleigh(VSL_RNG_METHOD_RAYLEIGH_ICDF, stream, n, r, 0.0, 1.0)) {
        fprintf(stderr, "Failure in vsRngRayleigh\n");
        goto out;
    }

    /* deinitialize the stream */
    errcode = vslDeleteStream(&stream);
    if (errcode != VSL_STATUS_OK) {
        fprintf(stderr, "Failure in deleting stream\n");
        goto out;
    }

out:
    free(r);
    return 0;
}
```

6.4.6.2.2.14 耿贝尔分布

生成符合耿贝尔分布（Gumbel distribution）的随机数向量。

接口定义

int vsRngGumbel (const int method, VSLStreamStatePtr stream, const SizeType n, float *r, const float a, const float beta);

int vdRngGumbel (const int method, VSLStreamStatePtr stream, const SizeType n, double *r, const double a, const double beta);

概率密度函数

$$f(x) = e^{-e^{-(x-\mu)/\beta}}$$

参数

参数名	类型	描述	输入/输出
method	int	随机数生成模式。 • VSL_RNG_METHOD_GUMBEL_ICDF：逆累积分布函数法模式	输入
stream	VSLStreamStatePtr类型	随机数流。	输入
n	SizeType	输出向量的长度。	输入

版权所有 © 华为技术有限公司

参数名	类型	描述	输入/输出
r	• 在vsRngGumbel中是单精度浮点类型指针。 • 在vdRngGumbel中是双精度浮点类型指针。	指向结果的指针。	输出
a	• 在vsRngGumbel中是单精度浮点类型。 • 在vdRngGumbel中是双精度浮点类型。	运算的结果加数。	输入
beta	• 在vsRngGumbel中是单精度浮点类型。 • 在vdRngGumbel中是双精度浮点类型。	运算的结果乘数。	输入

依赖

#include "krng.h"

示例

```
#include <stdio.h>
#include <stdlib.h>
#include "krng.h"
int main()
{
   /* initialize stream with given BRNG type and seed */
    VSLStreamStatePtr stream;
    unsigned seed = 42;
    int errcode = vslNewStream(&stream, VSL_BRNG_MCG59, seed);
    if (errcode != VSL_STATUS_OK) {
        fprintf(stderr, "Failure in newstream\n");
        return 0;
    }

    SizeType n = 10;
    float *r = (float *)malloc(sizeof(float) * n);
    if (r == NULL) {
        fprintf(stderr, "Failure in malloc\n");
        return 0;
    }

    if (vsRngGumbel(VSL_RNG_METHOD_GUMBEL_ICDF, stream, n, r, 0.0, 1.0)) {
        fprintf(stderr, "Failure in vsRngGumbel\n");
        goto out;
    }

    /* deinitialize the stream */
    errcode = vslDeleteStream(&stream);
    if (errcode != VSL_STATUS_OK) {
        fprintf(stderr, "Failure in deleting stream\n");
        goto out;
    }

out:
    free(r);
```


版权所有 © 华为技术有限公司

```
    return 0;
}
```

6.4.6.2.2.15 二项分布函数

生成符合二项分布（Binomial distribution）的随机数向量。

接口定义

int viRngBinomial (const int method, VSLStreamStatePtr stream, const SizeType n, int *r, const int m, const double p);

概率密度函数

$$f(X=k)=\binom{n}{k}p^{k}(1-p)^{n-k}$$

参数

参数名	类型	描述	输入/输出
method	int	随机数生成模式。 • VSL_RNG_METHOD_BINOMIAL_FAST：快速模式	输入
stream	VSLStreamStatePtr类型	随机数流。	输入
n	SizeType	输出向量的长度。	输入
r	整型指针	指向结果的指针。	输出
m	int	二项分布的试验次数。	输入
p	double	二项分布的成功概率。	输入

依赖

#include "krng.h"

示例

```
#include <stdio.h>
#include <stdlib.h>
#include "krng.h"
int main()
{
  /* initialize stream with given BRNG type and seed */
   VSLStreamStatePtr stream;
   unsigned seed = 42;
   int errcode = vslNewStream(&stream, VSL_BRNG_MCG59, seed);
   if (errcode != VSL_STATUS_OK) {
       fprintf(stderr, "Failure in newstream\n");
```

 版权所有 © 华为技术有限公司

```
    return 0;
  }

  SizeType n = 10;
  int *r = (int *)malloc(sizeof(int) * n);
  if (r == NULL) {
    fprintf(stderr, "Failure in malloc\n");
    return 0;
  }

  if (viRngBinomial(VSL_RNG_METHOD_BINOMIAL_FAST, stream, n, r, 100, 0.5)) {
    fprintf(stderr, "Failure in viRngBinomial\n");
    goto out;
  }

  /* deinitialize the stream */
  errcode = vslDeleteStream(&stream);
  if (errcode != VSL_STATUS_OK) {
    fprintf(stderr, "Failure in deleting stream\n");
    goto out;
  }

out:
  free(r);
  return 0;
}
```

6.4.6.2.2.16 几何分布函数

生成符合几何分布（Geometric distribution）的随机数向量。

接口定义

int viRngGeometric (const int method, VSLStreamStatePtr stream, const SizeType n, int *r, const double p);

概率密度函数

$$f(X=k)=(1-p)^{k-1}p$$

参数

参数名	类型	描述	输入/输出
method	int	随机数生成模式。 • VSL_RNG_METHOD_GEOMETRIC_ICDF：逆累积分布函数法模式	输入
stream	VSLStreamStatePtr类型	随机数流。	输入
n	SizeType	输出向量的长度。	输入
r	整型指针	指向结果的指针。	输出
p	double	几何分布的成功概率	输入

版权所有 © 华为技术有限公司

依赖

#include "krng.h"

示例

```
#include <stdio.h>
#include <stdlib.h>
#include "krng.h"
int main()
{
   /* initialize stream with given BRNG type and seed */
    VSLStreamStatePtr stream;
    unsigned seed = 42;
    int errcode = vslNewStream(&stream, VSL_BRNG_MCG59, seed);
    if (errcode != VSL_STATUS_OK) {
        fprintf(stderr, "Failure in newstream\n");
        return 0;
    }

    SizeType n = 10;
    int *r = (int *)malloc(sizeof(int) * n);
    if (r == NULL) {
        fprintf(stderr, "Failure in malloc\n");
        return 0;
    }

    if (viRngGeometric(VSL_RNG_METHOD_GEOMETRIC_ICDF, stream, n, r, 0.5)) {
        fprintf(stderr, "Failure in viRngGeometric\n");
        goto out;
    }

    /* deinitialize the stream */
    errcode = vslDeleteStream(&stream);
    if (errcode != VSL_STATUS_OK) {
        fprintf(stderr, "Failure in deleting stream\n");
        goto out;
    }

out:
    free(r);
    return 0;
}
```

6.4.6.2.2.17 泊松分布函数

生成符合泊松分布（Poisson distribution）的随机数向量。

接口定义

int viRngPoisson (const int method, VSLStreamStatePtr stream, const SizeType n, int *r, const double lambda);

概率密度函数

$$f(X=k)=\frac{\lambda^k}{k!}e^{-\lambda}$$

版权所有 © 华为技术有限公司

参数

参数名	类型	描述	输入/输出
method	int	随机数生成模式。 • VSL_RNG_METHOD_POISSON_FAST：快速模式 • VSL_RNG_METHOD_POISSON_ACCURATE：精确模式	输入
stream	VSLStreamStatePtr类型	随机数流。	输入
n	SizeType	输出向量的长度。	输入
r	整型指针	指向结果的指针。	输出
lambda	double	泊松分布的lambda参数	输入

依赖

#include "krng.h"

示例

```
#include <stdio.h>
#include <stdlib.h>
#include "krng.h"
int main()
{
   /* initialize stream with given BRNG type and seed */
    VSLStreamStatePtr stream;
    unsigned seed = 42;
    int errcode = vslNewStream(&stream, VSL_BRNG_MCG59, seed);
    if (errcode != VSL_STATUS_OK) {
        fprintf(stderr, "Failure in newstream\n");
        return 0;
    }

    SizeType n = 10;
    int *r = (int *)malloc(sizeof(int) * n);
    if (r == NULL) {
        fprintf(stderr, "Failure in malloc\n");
        return 0;
    }

    if (viRngPoisson(VSL_RNG_METHOD_POISSON_ACCURATE, stream, n, r, 2.0)) {
        fprintf(stderr, "Failure in viRngPoisson\n");
        goto out;
    }

    /* deinitialize the stream */
    errcode = vslDeleteStream(&stream);
    if (errcode != VSL_STATUS_OK) {
        fprintf(stderr, "Failure in deleting stream\n");
        goto out;
    }

out:
```


版权所有 © 华为技术有限公司

```
    free(r);
    return 0;
}
```

6.4.6.2.2.18 伯努利分布函数

生成符合伯努利分布（Bernoulli distribution）的随机数向量。

接口定义

int viRngBernoulli (const int method, VSLStreamStatePtr stream, const SizeType n, int *r, const double p);

概率密度函数

$$f\left(k \right)=\begin{cases} p, & k=1 \\ 1\text{-}p, & k=0 \end{cases}$$

参数

参数名	类型	描述	输入/输出
method	int	随机数生成模式。 • VSL_RNG_METHOD_BERNOULLI_ICDF：逆累积分布函数法模式	输入
stream	VSLStreamStatePtr类型	随机数流。	输入
n	SizeType	输出向量的长度。	输入
r	整型指针	指向结果的指针。	输出
p	double	伯努利分布的成功概率	输入

依赖

#include "krng.h"

示例

```
#include <stdio.h>
#include <stdlib.h>
#include "krng.h"
int main()
{
   /* initialize stream with given BRNG type and seed */
    VSLStreamStatePtr stream;
    unsigned seed = 42;
    int errcode = vslNewStream(&stream, VSL_BRNG_MCG59, seed);
    if (errcode != VSL_STATUS_OK) {
        fprintf(stderr, "Failure in newstream\n");
```

 版权所有 © 华为技术有限公司

```
    return 0;
  }

  SizeType n = 10;
  int *r = (int *)malloc(sizeof(int) * n);
  if (r == NULL) {
    fprintf(stderr, "Failure in malloc\n");
    return 0;
  }

  if (viRngBernoulli(VSL_RNG_METHOD_BERNOULLI_ICDF, stream, n, r, 0.5)) {
    fprintf(stderr, "Failure in viRngBernoulli\n");
    goto out;
  }

  /* deinitialize the stream */
  errcode = vslDeleteStream(&stream);
  if (errcode != VSL_STATUS_OK) {
    fprintf(stderr, "Failure in deleting stream\n");
    goto out;
  }

out:
  free(r);
  return 0;
}
```

6.4.7 KML_IPL 库函数说明

6.4.7.1 函数说明

KML_IPL库是使用C语言实现的插值库，用于数学的数值分析领域，可以使用插值来填充缺失的数据、对现有数据进行平滑处理以及进行预测等。该库包含了多种常用的插值函数，如B样条插值和三次样条插值。

6.4.7.2 函数定义

6.4.7.2.1 辅助函数

6.4.7.2.1.1 interp

?.1. kml_?_interp_alloc

这个函数将分配一个新的类型T对象，新对象在x方向长度为size。

接口定义

kml_float_interp *kml_float_interp_alloc(const kml_float_interp_type *T, size_t size)

kml_double_interp *kml_double_interp_alloc(const kml_double_interp_type *T, size_t size);

 版权所有 © 华为技术有限公司

参数

参数名	类型	描述	输入/输出
T	kml_float_interp_type* / kml_double_interp_type*	指向类型对象的指针。	输入
size	size_t	x方向网格长度。	输入

依赖

#include "kipl.h"

示例

```
const size_t xSize = 4;
kml_float_interp *interp = kml_float_interp_alloc(kml_float_interp_cspline, xSize);
```

?.2. kml_?_interp_init

初始化插值对象interp。数据(xarr, yarr)中，xarr是大小为xsize的数组，yarr是函数结果数组，大小为xsize。插值对象（kml_float_interp，kml_double_interp）不保存xarr，yarr的数据数组，它只存有从数据中计算出的静态状态。xarr数据数组必须是严格升序排列的。

接口定义

int kml_float_interp_init(kml_float_interp *interp, const float xarr[], const float yarr[], const size_t size)

int kml_double_interp_init(kml_double_interp *interp, const double xarr[], const double yarr[], size_t size);

参数

参数名	类型	描述	输入/输出
interp	kml_float_interp* / kml_double_interp*	指向插值对象的指针。	输出
xarr	float类型数组 / double类型数组	存有x值的数组。	输入
yarr	float类型数组 / double类型数组	存有y值的数组。	输入
size	size_t	x数组长度。	输入

 版权所有 © 华为技术有限公司

依赖

#include "kipl.h"

示例

```
const size_t xSize = 4;
float xArr[4] = {1,1.33,1.67,2};
float* yArr = (float*)malloc(xSize * sizeof(float));
kml_float_interp *interp = kml_float_interp_alloc(kml_float_interp_cspline, xSize);
kml_float_interp_init(interp, xArr, yArr, xSize);
```

?.3. kml_?_interp_name

返回interp使用的插值类型的名称。

接口定义

const char *kml_float_interp_name(const kml_float_interp *interp)

const char *kml_double_interp_name(const kml_double_interp *interp);

参数

参数名	类型	描述	输入/输出
interp	kml_float_interp* / kml_double_interp*	指向插值对象的指针。	输入

依赖

#include "kipl.h"

示例

```
const size_t xSize = 4;
float xArr[4] = {1,1.33,1.67,2};
float* yArr = (float*)malloc(xSize * sizeof(float));
float x = 1;
float y = 0;
kml_float_interp *interp = kml_float_interp_alloc(kml_float_interp_cspline, xSize);
const char *name = kml_float_interp_name(interp);
```

?.4. kml_?_interp_min_size

返回插值对象interp或插值类型T所需要的最小的点的数量。例如，三次插值最小需要3个点。

接口定义

unsigned int kml_float_interp_min_size(const kml_float_interp *interp)

unsigned int kml_float_interp_type_min_size(const kml_float_interp_type *T)

unsigned int kml_double_interp_min_size(const kml_double_interp *interp);

 版权所有 © 华为技术有限公司

unsigned int kml_double_interp_type_min_size(const kml_double_interp_type *T)

参数

参数名	类型	描述	输入/输出
interp	const kml_float_interp* / const kml_double_interp*	指向插值对象的指针。	输入

依赖

#include "kipl.h"

示例

```
const size_t xSize = 4;
float xArr[4] = {1,1.33,1.67,2};
float* yArr = (float*)malloc(xSize * sizeof(float));
float x = 1;
float y = 0;
kml_float_interp *interp = kml_float_interp_alloc(kml_float_interp_cspline, xSize);
size_t min_size = kml_float_interp_min_size(interp);
```

?.5. kml_?_interp_free

释放插值对象interp。

接口定义

void kml_float_interp_free(kml_float_interp *interp)

void kml_double_interp_free(kml_double_interp *interp);

参数

参数名	类型	描述	输入/输出
interp	kml_float_interp* / kml_double_interp*	指向插值对象的指针。	输入

依赖

#include "kipl.h"

示例

```
const size_t xSize = 4;
float xArr[4] = {1,1.33,1.67,2};
float* yArr = (float*)malloc(xSize * sizeof(float));
kml_float_interp *interp = kml_float_interp_alloc(kml_float_interp_cspline, xSize);
kml_float_interp_init(interp, xSize, xArr, yArr);
kml_float_interp_free(interp);
```

 版权所有 © 华为技术有限公司

6.4.7.2.1.2 spline

?.1. kml_?_spline_alloc

这个函数将分配一个新的类型T对象，新对象在x方向长度为size。

接口定义

kml_float_spline *kml_float_spline_alloc(const kml_float_interp_type *T, size_t size)

kml_double_spline *kml_double_spline_alloc(const kml_double_interp_type *T, size_t size);

参数

参数名	类型	描述	输入/输出
T	const kml_float_interp_type* / const kml_double_interp_type*	指向类型对象的指针。	输入
size	size_t	x方向网格长度。	输入

依赖

#include "kipl.h"

示例

```
const size_t xSize = 4;
kml_float_spline *spline = kml_float_spline_alloc(kml_float_interp_cspline, xSize);
```

?.2. kml_?_spline_init

初始化插值对象spline。数据(xarr, yarr)中，xarr是大小为xsize的数组，yarr是函数结果数组，大小为xsize。插值对象（kml_float_spline，kml_double_spline）不保存xarr，yarr的数据数组，它只存有从数据中计算出的静态状态。xarr数据数组必须是严格升序排列的。

接口定义

int kml_float_spline_init(kml_float_spline *spline, const float xarr[], const float yarr[], const size_t size)

int kml_double_spline_init(kml_double_spline *spline, const double xarr[], const double yarr[], size_t size);

 版权所有 © 华为技术有限公司

参数

参数名	类型	描述	输入/输出
spline	kml_float_spline* / kml_double_spline*	指向插值对象的指针。	输出
xarr	float类型数组 / double类型数组	存有x值的数组。	输入
yarr	float类型数组 / double类型数组	存有y值的数组。	输入
size	size_t	x数组长度。	输入

依赖

#include "kipl.h"

示例

```
const size_t xSize = 4;
float xArr[4] = {1,1.33,1.67,2};
float* yArr = (float*)malloc(xSize * sizeof(float));
kml_float_spline *spline = kml_float_spline_alloc(kml_float_interp_cspline, xSize);
kml_float_spline_init(spline, xArr, yArr, xSize);
```

?.3. kml_?_spline_name

返回spline使用的插值类型的名称。

接口定义

const char *kml_float_spline_name(const kml_float_spline *spline)

const char *kml_double_spline_name(const kml_double_spline *spline);

参数

参数名	类型	描述	输入/输出
spline	const kml_float_spline* / const kml_double_spline*	指向插值对象的指针。	输入

依赖

#include "kipl.h"

示例

```
const size_t xSize = 4;
float xArr[4] = {1,1.33,1.67,2};
```

 版权所有 © 华为技术有限公司

```
float* yArr = (float*)malloc(xSize * sizeof(float));
float x = 1;
float y = 0;
kml_float_spline *spline = kml_float_spline_alloc(kml_float_interp_cspline, xSize);
const char *name = kml_float_spline_name(spline);
```

?.4. kml_?_spline_min_size

返回插值对象spline所需要的最小的点的数量。例如，三次插值最小需要3个点。

接口定义

unsigned int kml_float_spline_min_size(const kml_float_spline *spline)

unsigned int kml_double_spline_min_size(const kml_double_spline *spline);

参数

参数名	类型	描述	输入/输出
spline	const kml_float_spline* / const kml_double_spline*	指向插值对象的指针。	输入

依赖

#include "kipl.h"

示例

```
const size_t xSize = 4;
float xArr[4] = {1,1.33,1.67,2};
float* yArr = (float*)malloc(xSize * sizeof(float));
float x = 1;
float y = 0;
kml_float_spline *spline = kml_float_spline_alloc(kml_float_interp_cspline, xSize);
size_t min_size = kml_float_spline_min_size(spline);
```

?.5. kml_?_spline_free

释放插值对象spline。

接口定义

void kml_float_spline_free(kml_float_spline *spline)

void kml_double_spline_free(kml_double_spline *spline);

参数

参数名	类型	描述	输入/输出
spline	kml_float_spline* / kml_double_spline*	指向插值对象的指针。	输入

 版权所有 © 华为技术有限公司

依赖

#include "kipl.h"

示例

```
const size_t xSize = 4;
float xArr[4] = {1,1.33,1.67,2};
float* yArr = (float*)malloc(xSize * sizeof(float));
kml_float_spline *spline = kml_float_spline_alloc(kml_float_interp_cspline, xSize);
kml_float_spline_init(spline, xSize, xArr, yArr);
kml_float_spline_free(spline);
```

6.4.7.2.1.3 interp2d

?.1. kml_?_interp2d_alloc

这个函数将分配一个新的类型T对象，新对象在x方向有长度为xsize的网格，在y方向有长度为ysize的网格。

接口定义

kml_float_interp2d *kml_float_interp2d_alloc(const kml_float_interp2d_type *T, const size_t xsize, const size_t ysize)

kml_double_interp2d *kml_double_interp2d_alloc(const kml_double_interp2d_type *T, const size_t xsize, const size_t ysize);

参数

参数名	类型	描述	输入/输出
T	const kml_float_interp2d_type* / const kml_double_interp2d_type*	指向类型对象的指针。	输入
xsize	size_t	x方向网格长度。	输入
ysize	size_t	y方向网格长度。	输入

依赖

#include "kipl.h"

示例

```
const size_t xSize = 4;
const size_t ySize = 4;
kml_float_interp2d *interp = kml_float_interp2d_alloc(kml_float_interp2d_bicubic, xSize, ySize);
```

?.2. kml_?_interp2d_init

初始化插值对象interp。数据(xarr, yarr, zarr)中，xarr和yarr分别是大小为xsize和ysize的数组，zarr是函数结果数组，大小为xsize * ysize。插值对象（ kml_float_interp2d，

 版权所有 © 华为技术有限公司

kml_double_interp2d ）不保存xarr，yarr，zarr的数据数组，它只存有从数据中计算出的静态状态。xarr和yarr数据数组必须是严格升序排列的。

接口定义

int kml_float_interp2d_init(kml_float_interp2d *interp, const float xarr[], const float yarr[], const float zarr[], const size_t xsize, const size_t ysize)

int kml_double_interp2d_init(kml_double_interp2d *interp, const double xarr[], const double yarr[], const double zarr[],

const size_t xsize, const size_t ysize);

参数

参数名	类型	描述	输入/输出
interp	kml_float_interp2d* / kml_double_interp2d*	指向插值对象的指针。	输出
xarr	float类型数组 / double类型数组	存有x值的数组。	输入
yarr	float类型数组 / double类型数组	存有y值的数组。	输入
zarr	float类型数组 / double类型数组	存有z值的数组。	输入
xsize	size_t	x数组长度。	输入
ysize	size_t	y数组长度。	输入

依赖

#include "kipl.h"

示例

```
const size_t xSize = 4;
const size_t ySize = 4;
float xArr[4] = {1,1.33,1.67,2};
float yArr[4] = {1,1.33,1.67,2};
float* zArr = (float*)malloc(xSize * ySize * sizeof(float));
kml_float_interp2d *interp = kml_float_interp2d_alloc(kml_float_interp2d_bicubic, xSize, ySize);
kml_float_interp2d_init(interp, xArr, yArr, zArr, xSize, ySize);
```

?.3. kml_?_interp2d_free

释放插值对象interp。

接口定义

void kml_float_interp2d_free(kml_float_interp2d *interp)

void kml_double_interp2d_free(kml_double_interp2d *interp);

版权所有 © 华为技术有限公司

参数

参数名	类型	描述	输入/输出
interp	kml_float_interp2d* / kml_double_interp2d*	指向插值对象的指针。	输入

依赖

#include "kipl.h"

示例

```
const size_t xSize = 4;
const size_t ySize = 4;
float xArr[4] = {1,1.33,1.67,2};
float yArr[4] = {1,1.33,1.67,2};
float* zArr = (float*)malloc(xSize * ySize * sizeof(float));
kml_float_interp2d *interp = kml_float_interp2d_alloc(kml_float_interp2d_bicubic, xSize, ySize);
kml_float_interp2d_init(interp, xArr, yArr, zArr, xSize, ySize);
kml_float_interp2d_free(interp);
```

?.4. kml_?_interp2d_set

设zarr在网格点(i,j)处为z。

接口定义

int kml_float_interp2d_set(const kml_float_interp2d *interp, float zarr[], const size_t i, const size_t j, const float z)

int kml_double_interp2d_set(const kml_double_interp2d *interp, double zarr[], const size_t i, const size_t j,

const double z);

参数

参数名	类型	描述	输入/输出
interp	kml_float_interp2d*, kml_double_interp2d*	插值对象指针。	输入
zarr	float, double	存有z值的数组。	输出
i	size_t	x方向的index。	输入
j	size_t	y方向的index。	输入
z	float	要改变的z值。	输入

版权所有 © 华为技术有限公司

依赖

#include "kipl.h"

示例

```
const size_t xSize = 4;
const size_t ySize = 4;
float xArr[4] = {1,1.33,1.67,2};
float yArr[4] = {1,1.33,1.67,2};
float* zArr = (float*)malloc(xSize * ySize * sizeof(float));
size_t i = 1;
size_t j = 1;
float z = 0;
kml_float_interp2d *interp = kml_float_interp2d_alloc(kml_float_interp2d_bicubic, xSize, ySize);
kml_float_interp2d_init(interp, xArr, yArr, zArr, xSize, ySize);
kml_float_interp2d_set(interp, zArr, i, j, z);
```

?.5. kml_?_interp2d_get

返回数组zarr在网格(i, j)的值zij。

接口定义

float kml_float_interp2d_get(const kml_float_interp2d *interp, const float zarr[], const size_t i, const size_t j)

double kml_double_interp2d_get(const kml_double_interp2d *interp, const double zarr[], const size_t i, const size_t j);

参数

参数名	类型	描述	输入/输出
interp	const kml_float_interp2d* / const kml_double_interp2d*	指向插值对象的指针。	输入
zarr	float类型数组 / double类型数组	存有z值的数组。	输入
i	size_t	x方向的index。	输入
j	size_t	y方向的index。	输入

依赖

#include "kipl.h"

示例

```
const size_t xSize = 4;
const size_t ySize = 4;
float xArr[4] = {1,1.33,1.67,2};
float yArr[4] = {1,1.33,1.67,2};
float* zArr = (float*)malloc(xSize * ySize * sizeof(size_t));
float x = 1;
float y = 1;
```

 版权所有 © 华为技术有限公司

```
float z = 0;
kml_float_interp2d *interp = kml_float_interp2d_alloc(kml_float_interp2d_bicubic, xSize, ySize);
kml_float_interp2d_init(interp, xArr, yArr, zArr, xSize, ySize);
z = kml_float_interp2d_get(interp, zArr, x, y);
```

?.6. kml_?_interp2d_idx

返回网格(i, j)点的index，index的值为j * xsize + i。

接口定义

size_t kml_float_interp2d_idx(const kml_float_interp2d *interp, const size_t i, const size_t j)

size_t kml_double_interp2d_idx(const kml_double_interp2d *interp, const size_t i, const size_t j);

参数

参数名	类型	描述	输入/输出
interp	const kml_float_interp2d* / const kml_double_interp2d*	指向插值对象的指针。	输入
i	size_t	x方向的index。	输入
j	size_t	y方向的index。	输入

依赖

#include "kipl.h"

示例

```
const size_t xSize = 4;
const size_t ySize = 4;
float xArr[4] = {1,1.33,1.67,2};
float yArr[4] = {1,1.33,1.67,2};
float* zArr = (float*)malloc(xSize * ySize * sizeof(float));
size_t x = 1;
size_t y = 1;
kml_float_interp2d *interp = kml_float_interp2d_alloc(kml_float_interp2d_bicubic, xSize, ySize);
kml_float_interp2d_init(interp, xArr, yArr, zArr, xSize, ySize);
size_t idx = kml_float_interp2d_idx(interp, x, y);
```

?.7. kml_?_interp2d_name

返回interp使用的插值类型的名称。

接口定义

const char *kml_float_interp2d_name(const kml_float_interp2d *interp)

const char *kml_double_interp2d_name(const kml_double_interp2d *interp);

 版权所有 © 华为技术有限公司

参数

参数名	类型	描述	输入/输出
interp	const kml_float_interp2d* / const kml_double_interp2d*	指向插值对象的指针。	输入

依赖

#include "kipl.h"

示例

```
const size_t xSize = 4;
const size_t ySize = 4;
float xArr[4] = {1,1.33,1.67,2};
float yArr[4] = {1,1.33,1.67,2};
float* zArr = (float*)malloc(xSize * ySize * sizeof(float));
float x = 1;
float y = 1;
float z = 0;
kml_float_interp2d *interp = kml_float_interp2d_alloc(kml_float_interp2d_bicubic, xSize, ySize);
const char *name = kml_float_interp2d_name(interp);
```

?.8. kml_?_interp2d_min_size

返回插值对象interp或插值类型T所需要的最小的点的数量。例如，双三次插值最小需要4个点。

接口定义

size_t kml_float_interp2d_min_size(const kml_float_interp2d *interp)

size_t kml_float_interp2d_type_min_size(const kml_float_interp2d_type *T)

size_t kml_double_interp2d_min_size(const kml_double_interp2d *interp);

size_t kml_double_interp2d_type_min_size(const kml_double_interp2d_type *T);

参数

参数名	类型	描述	输入/输出
interp	const kml_float_interp2d* / const kml_double_interp2d*	指向插值对象的指针。	输入

依赖

#include "kipl.h"

 版权所有 © 华为技术有限公司

示例

```
const size_t xSize = 4;
const size_t ySize = 4;
float xArr[4] = {1,1.33,1.67,2};
float yArr[4] = {1,1.33,1.67,2};
float* zArr = (float*)malloc(xSize * ySize * sizeof(float));
float x = 1;
float y = 1;
float z = 0;
kml_float_interp2d *interp = kml_float_interp2d_alloc(kml_float_interp2d_bicubic, xSize, ySize);
size_t min_size = kml_float_interp2d_min_size(interp);
```

6.4.7.2.1.4 spline2d

?.1. kml_?_spline2d_alloc

该函数将分配一个新的类型T对象，新对象在x方向有长度为xsize的网格，在y方向有长度为ysize的网格。

接口定义

kml_float_spline2d *kml_float_spline2d_alloc(const kml_float_interp2d_type *T, size_t xsize, size_t ysize)

kml_double_spline2d *kml_double_spline2d_alloc(const kml_double_interp2d_type *T, size_t xsize, size_t ysize);

参数

参数名	类型	描述	输入/输出
T	const kml_float_interp2d_type* / const kml_double_interp2d_type*	指向类型对象的指针。	输入
xsize	size_t	x方向网格长度。	输入
ysize	size_t	y方向网格长度。	输入

依赖

#include "kipl.h"

示例

```
const size_t xSize = 4;
const size_t ySize = 4;
kml_float_spline2d *spline = kml_float_spline2d_alloc(kml_float_interp2d_bicubic, xSize, ySize);
```

?.2. kml_?_spline2d_init

对于插值对象spline，这个函数将做初始化操作。数据(xarr, yarr, zarr)中，xarr和yarr分别是大小为xsize和ysize的数组，zarr是函数结果数组，大小为xsize * ysize。插值对

 版权所有 © 华为技术有限公司

象（kml_float_spline2d，kml_double_spline2d）不保存xarr，yarr，zarr的数据数组，它只存有从数据中计算出的静态状态。xarr和yarr数据数组必须是严格升序排列的。

接口定义

int kml_float_spline2d_init(kml_float_spline2d *spline, const float xarr[], const float yarr[], const float zarr[], size_t xsize, size_t ysize)

int kml_double_spline2d_init(kml_double_spline2d *spline, const double xarr[], const double yarr[], const double zarr[],

size_t xsize, size_t ysize);

参数

参数名	类型	描述	输入/输出
spline	kml_float_spline2d* / kml_double_spline2d*	指向插值对象的指针。	输出
xarr	float类型数组 / double类型数组	存有x值的数组。	输入
yarr	float类型数组 / double类型数组	存有y值的数组。	输入
zarr	float类型数组 / double类型数组	存有z值的数组。	输入
xsize	size_t	x数组长度。	输入
ysize	size_t	y数组长度。	输入

依赖

#include "kipl.h"

示例

```
const size_t xSize = 4;
const size_t ySize = 4;
float xArr[4] = {1,1.33,1.67,2};
float yArr[4] = {1,1.33,1.67,2};
float* zArr = (float*)malloc(xSize * ySize * sizeof(float));
kml_float_spline2d *spline = kml_float_spline2d_alloc(kml_float_interp2d_bicubic, xSize, ySize);
kml_float_spline2d_init(spline, xArr, yArr, zArr, xSize, ySize);
```

?.3. kml_?_spline2d_free

释放插值对象spline。

接口定义

void kml_float_spline2d_free(kml_float_spline2d *spline)

 版权所有 © 华为技术有限公司

void kml_double_spline2d_free(kml_double_spline2d *spline);

参数

参数名	类型	描述	输入/输出
spline	kml_float_spline2d*, kml_double_spline2d*	插值对象指针。	输入

依赖

#include "kipl.h"

示例

```
const size_t xSize = 4;
const size_t ySize = 4;
float xArr[4] = {1,1.33,1.67,2};
float yArr[4] = {1,1.33,1.67,2};
float* zArr = (float*)malloc(xSize * ySize * sizeof(float));
kml_float_spline2d *spline = kml_float_spline2d_alloc(kml_float_interp2d_bicubic, xSize, ySize);
kml_float_spline2d_init(spline, xArr, yArr, zArr, xSize, ySize);
kml_float_spline2d_free(spline);
```

?.4. kml_?_spline2d_set

设zarr在网格点(i,j)处为z。

接口定义

int kml_float_spline2d_set(const kml_float_spline2d *spline, float zarr[], const size_t i, const size_t j, const float z)

int kml_double_spline2d_set(const kml_double_spline2d *spline, double zarr[], const size_t i, const size_t j,

const double z);

参数

参数名	类型	描述	输入/输出
spline	kml_float_spline2d*, kml_double_spline2d*	插值对象指针。	输入
zarr	float, double	存有z值的数组。	输出
i	size_t	x方向的index。	输入
j	size_t	y方向的index。	输入
z	float	要改变的z值。	输入

版权所有 © 华为技术有限公司

依赖

#include "kipl.h"

示例

```
const size_t xSize = 4;
const size_t ySize = 4;
float xArr[4] = {1,1.33,1.67,2};
float yArr[4] = {1,1.33,1.67,2};
float* zArr = (float*)malloc(xSize * ySize * sizeof(float));
float x = 1;
float y = 1;
float z = 0;
kml_float_spline2d *spline = kml_float_spline2d_alloc(kml_float_interp2d_bicubic, xSize, ySize);
kml_float_spline2d_init(spline, xArr, yArr, zArr, xSize, ySize);
kml_float_spline2d_eval(spline, xArr, yArr, zArr, x, y, xacc, yacc, &z);
kml_float_spline2d_set(spline, zArr, x, y, z);
```

?.5. kml_?_spline2d_get

返回数组zarr在网格(i, j)的值zij。

接口定义

float kml_float_spline2d_get(const kml_float_spline2d *spline, const float zarr[], const size_t i, const size_t j)

double kml_double_spline2d_get(const kml_double_spline2d *spline, const double zarr[], const size_t i, const size_t j);

参数

参数名	类型	描述	输入/输出
spline	const kml_float_spline2d* / const kml_double_spline2d*	指向插值对象的指针。	输入
zarr	float类型数组 / double类型数组	存有z值的数组。	输入
i	size_t	x方向的index。	输入
j	size_t	y方向的index。	输入

依赖

#include "kipl.h"

示例

```
const size_t xSize = 4;
const size_t ySize = 4;
float xArr[4] = {1,1.33,1.67,2};
float yArr[4] = {1,1.33,1.67,2};
float* zArr = (float*)malloc(xSize * ySize * sizeof(float));
size_t x = 1;
```

 版权所有 © 华为技术有限公司

```
size_t y = 1;
kml_float_spline2d *spline = kml_float_spline2d_alloc(kml_float_interp2d_bicubic, xSize, ySize);
kml_float_spline2d_init(spline, xArr, yArr, zArr, xSize, ySize);
z = kml_float_spline2d_get(spline, zArr, x, y);
```

?.6. kml_?_spline2d_name

返回spline使用的插值类型的名称。

接口定义

const char *kml_float_spline2d_name(const kml_float_spline2d *spline)

const char *kml_double_spline2d_name(const kml_double_spline2d *spline);

参数

参数名	类型	描述	输入/输出
spline	const kml_float_spline2d* / const kml_double_spline2d*	指向插值对象的指针。	输入

依赖

#include "kipl.h"

示例

```
const size_t xSize = 4;
const size_t ySize = 4;
float xArr[4] = {1,1.33,1.67,2};
float yArr[4] = {1,1.33,1.67,2};
float* zArr = (float*)malloc(xSize * ySize * sizeof(float));
float x = 1;
float y = 1;
float z = 0;
kml_float_spline2d *spline = kml_float_spline2d_alloc(kml_float_interp2d_bicubic, xSize, ySize);
const char *name = kml_float_spline2d_name(spline);
```

?.7. kml_?_spline2d_min_size

返回插值对象spline所需要的最小的点的数量。例如，双三次插值最小需要4个点。

接口定义

size_t kml_float_spline2d_min_size(const kml_float_spline2d *spline)

size_t kml_double_spline2d_min_size(const kml_double_spline2d *spline);

版权所有 © 华为技术有限公司

参数

参数名	类型	描述	输入/输出
spline	const kml_float_spline2d* / const kml_double_spline2d*	指向插值对象的指针。	输入

依赖

#include "kipl.h"

示例

```
const size_t xSize = 4;
const size_t ySize = 4;
float xArr[4] = {1,1.33,1.67,2};
float yArr[4] = {1,1.33,1.67,2};
float* zArr = (float*)malloc(xSize * ySize * sizeof(float));
float x = 1;
float y = 1;
float z = 0;
kml_float_spline2d *spline = kml_float_spline2d_alloc(kml_float_interp2d_bicubic, xSize, ySize);
size_t min_size = kml_float_spline2d_min_size(spline);
```

6.4.7.2.2 计算函数

6.4.7.2.2.1 interp

?.1. kml_?_interp_eval

这个函数返回给定点(x)的插值结果y。计算中使用插值对象interp，数据数组xarr，yarr以及加速器xacc。对于x在xarr的区间外，将返回报错信息。

接口定义

int kml_float_interp_eval_e(const kml_float_interp *interp, const float xa[],

const float ya[], float x, kml_interp_accel *a, float *y)

float kml_float_interp_eval(const kml_float_interp *interp, const float xa[], const float ya[], float x,

kml_interp_accel *a)

int kml_double_interp_eval_e(const kml_double_interp *interp, const double xa[], const double ya[], double x,

kml_interp_accel *a, double *y);

double kml_double_interp_eval(const kml_double_interp *interp, const double xa[], const double ya[], double x,

kml_interp_accel *a);

 版权所有 © 华为技术有限公司

参数

参数名	类型	描述	输入/输出
interp	kml_float_interp*, kml_double_interp*	插值对象指针	输入
xa	float, double	存有x值的数组	输入
ya	float, double	存有y值的数组	输入
x	float, double	求解点x的值	输入
y	float, double	求解点y的值	输入
a	kml_interp_accel *	加速器对象指针	输入

依赖

#include "kipl.h"

示例

```
const size_t xMin = 4;
const size_t xMax = 2048;
const size_t xSize = 4;
const float xValLo = 1;
const float xValHi = 2;
float xArr[4] = {1,1.33,1.67,2};
float* yArr = (float*)malloc(xSize * sizeof(float));
float x = 1;
float y = 0;
kml_float_interp *interp = kml_float_interp_alloc(kml_float_interp_cspline, xSize);
kml_interp_accel *xacc = kml_interp_accel_alloc();
kml_float_interp_init(interp, xSize, xArr, yArr);
y = kml_float_interp_eval(interp, xArr, yArr, x, xacc);
kml_float_interp_free(interp);
kml_interp_accel_free(xacc);
```

?.2. kml_?_interp_eval_deriv

这个函数返回插值函数对于给定点(x)的微分。计算中使用插值对象interp，数据数组xarr，yarr以及加速器xacc。对于x在xarr的区间外，将返回报错信息。

接口定义

int kml_float_interp_eval_deriv_e(const kml_float_interp *interp, const float xa[], const float ya[], float x,

kml_interp_accel *a, float *dydx)

float kml_float_interp_eval_deriv(const kml_float_interp *interp, const float xa[], const float ya[], float x,

kml_interp_accel *a)

int kml_double_interp_eval_deriv_e(const kml_double_interp *interp, const double xa[], const double ya[], double x,

 版权所有 © 华为技术有限公司

kml_interp_accel *a, double *dydx);

double kml_double_interp_eval_deriv(const kml_double_interp *interp, const double xa[], const double ya[], double x,

kml_interp_accel *a);

参数

参数名	类型	描述	输入/输出
interp	kml_float_interp*, kml_double_interp*	插值对象指针	输入
xa	float, double	存有x值的数组	输入
ya	float, double	存有y值的数组	输入
x	float, double	求解点x的值	输入
a	kml_interp_accel *	加速器对象指针	输入
dydx	float, double	定点x处的二阶微分	输入

依赖

#include "kipl.h"

示例

```
const size_t xMin = 4;
const size_t xMax = 2048;
const size_t xSize = 4;
const float xValLo = 1;
const float xValHi = 2;
float xArr[4] = {1,1.33,1.67,2};
float* yArr = (float*)malloc(xSize * sizeof(float));
float x = 1;
float y = 0;
kml_float_interp *interp = kml_float_interp_alloc(kml_float_interp_cspline, xSize);
kml_interp_accel *xacc = kml_interp_accel_alloc();
kml_float_interp_init(interp, xSize, xArr, yArr);
y = kml_float_interp_eval_deriv(interp, xArr, yArr, x, xacc);
kml_float_interp_free(interp);
kml_interp_accel_free(xacc);
```

?.3. kml_?_interp_eval_deriv2

这个函数返回插值函数对于给定点(x)的二阶微分。计算中使用插值对象interp，数据数组xarr，yarr以及加速器xacc。对于x在xarr的区间外，将返回报错信息。

接口定义

int kml_float_interp_eval_deriv2_e(const kml_float_interp *interp, const float xa[], const float ya[], float x,

kml_interp_accel *a, float *d2)

 版权所有 © 华为技术有限公司

float kml_float_interp_eval_deriv2(const kml_float_interp *interp, const float xa[],

const float ya[], float x, kml_interp_accel *a)

int kml_double_interp_eval_deriv2_e(const kml_double_interp *interp, const double xa[], const double ya[], double x,

kml_interp_accel *a, double *d2);

double kml_double_interp_eval_deriv2(const kml_double_interp *interp, const double xa[], const double ya[], double x,

kml_interp_accel *a);

参数

参数名	类型	描述	输入/输出
interp	kml_float_interp*, kml_double_interp*	插值对象指针	输入
xa	float, double	存有x值的数组	输入
ya	float, double	存有y值的数组	输入
x	float, double	求解点x的值	输入
a	kml_interp_accel *	加速器对象指针	输入
dydx	float, double	定点x处的二阶微分	输入

依赖

#include "kipl.h"

示例

```
const size_t xMin = 4;
const size_t xMax = 2048;
const size_t xSize = 4;
const float xValLo = 1;
const float xValHi = 2;
float xArr[4] = {1,1.33,1.67,2};
float* yArr = (float*)malloc(xSize * sizeof(float));
float x = 1;
float y = 0;
kml_float_interp *interp = kml_float_interp_alloc(kml_float_interp_cspline, xSize);
kml_interp_accel *xacc = kml_interp_accel_alloc();
kml_float_interp_init(interp, xSize, xArr, yArr);
y = kml_float_interp_eval_deriv2(interp, xArr, yArr, x, xacc);
kml_float_interp_free(interp);
kml_interp_accel_free(xacc);
```

?.4. kml_?_interp_eval_integ

这个函数返回插值函数对于给定点(x)的积分。计算中使用插值对象interp，数据数组xarr，yarr以及加速器xacc。对于x在xarr的区间外，将返回报错信息。

 版权所有 © 华为技术有限公司

接口定义

int kml_float_interp_eval_integ_e(const kml_float_interp *interp, const float xa[], const float ya[],

float a, float b, kml_interp_accel *acc, float *result)

float kml_float_interp_eval_integ(const kml_float_interp *interp, const float xa[], const float ya[],

float a, float b, kml_interp_accel *acc)

int kml_double_interp_eval_integ_e(const kml_double_interp *interp, const double xa[], const double ya[], double a,

double b, kml_interp_accel *acc, double *result);

double kml_double_interp_eval_integ(const kml_double_interp *interp, const double xa[], const double ya[], double a,

double b, kml_interp_accel *acc);

参数

参数名	类型	描述	输入/输出
interp	kml_float_interp*, kml_double_interp*	插值对象指针	输入
xa	float, double	存有x值的数组	输入
ya	float, double	存有y值的数组	输入
x	float, double	求解点x的值	输入
a	kml_interp_accel *	加速器对象指针	输入
dydx	float, double	定点x处的二阶微分	输入

依赖

#include "kipl.h"

示例

```
const size_t xMin = 4;
const size_t xMax = 2048;
const size_t xSize = 4;
const float xValLo = 1;
const float xValHi = 2;
float xArr[4] = {1,1.33,1.67,2};
float* yArr = (float*)malloc(xSize * sizeof(float));
float x = 1;
float y = 0;
kml_float_interp *interp = kml_float_interp_alloc(kml_float_interp_cspline, xSize);
kml_interp_accel *xacc = kml_interp_accel_alloc();
kml_float_interp_init(interp, xSize, xArr, yArr);
y = kml_float_interp_eval_integ(interp, xArr, yArr, x, xacc);
```


版权所有 © 华为技术有限公司

```
kml_float_interp_free(interp);
kml_interp_accel_free(xacc);
```

6.4.7.2.2.2 spline

?.1. kml_?_spline_eval

这个函数返回给定点(x)的插值结果y。计算中使用插值对象spline，加速器xacc。对于x在xarr的区间外，将返回报错信息。

接口定义

int kml_float_spline_eval_e(const kml_float_spline *spline,

float x, kml_spline_accel *a, float *y)

float kml_float_spline_eval(const kml_float_spline *spline, float x,

kml_spline_accel *a)

int kml_double_spline_eval_e(const kml_double_spline *spline, double x, kml_interp_accel *a, double *y);

double kml_double_spline_eval(const kml_double_spline *spline, double x, kml_interp_accel *a);

参数

参数名	类型	描述	输入/输出
spline	kml_float_spline*, kml_double_spline*	插值对象指针	输入
x	float, double	求解点x的值	输入
y	float, double	求解点y的值	输入
a	kml_spline_accel *	加速器对象指针	输入

依赖

#include "kipl.h"

示例

```
const size_t xMin = 4;
const size_t xMax = 2048;
const size_t xSize = 4;
const float xValLo = 1;
const float xValHi = 2;
float xArr[4] = {1,1.33,1.67,2};
float* yArr = (float*)malloc(xSize * sizeof(float));
float x = 1;
float y = 0;
kml_float_spline *spline = kml_float_spline_alloc(kml_float_interp_cspline, xSize);
kml_interp_accel *xacc = kml_interp_accel_alloc();
```

 版权所有 © 华为技术有限公司

```
kml_float_spline_init(spline, xSize, xArr, yArr);
y = kml_float_spline_eval(spline, xArr, x, xacc);
kml_float_spline_free(spline);
kml_interp_accel_free(xacc);
```

?.2. kml_?_spline_eval_deriv

这个函数返回插值函数对于给定点(x)的微分。计算中使用插值对象spline，加速器xacc。对于x在xarr的区间外，将返回报错信息。

接口定义

int kml_float_spline_eval_deriv_e(const kml_float_spline *spline, float x,

kml_spline_accel *a, float *dydx)

float kml_float_spline_eval_deriv(const kml_float_spline *spline, float x,

kml_spline_accel *a)

int kml_double_spline_eval_deriv_e(const kml_double_spline *spline, double x, kml_interp_accel *a, double *dydx);

double kml_double_spline_eval_deriv(const kml_double_spline *spline, double x, kml_interp_accel *a);

参数

参数名	类型	描述	输入/输出
spline	kml_float_spline*, kml_double_spline*	插值对象指针	输入
x	float, double	求解点x的值	输入
y	float, double	求解点y的值	输入
a	kml_spline_accel *	加速器对象指针	输入

依赖

#include "kipl.h"

示例

```
const size_t xMin = 4;
const size_t xMax = 2048;
const size_t xSize = 4;
const float xValLo = 1;
const float xValHi = 2;
float xArr[4] = {1,1.33,1.67,2};
float* yArr = (float*)malloc(xSize * sizeof(float));
float x = 1;
float y = 0;
kml_float_spline *spline = kml_float_spline_alloc(kml_float_interp_cspline, xSize);
kml_interp_accel *xacc = kml_interp_accel_alloc();
kml_float_spline_init(spline, xSize, xArr, yArr);
y = kml_float_spline_eval_deriv(spline, x, xacc);
```

 版权所有 © 华为技术有限公司

```
kml_float_spline_free(spline);
kml_interp_accel_free(xacc);
```

?.3. kml_?_spline_eval_deriv2

这个函数返回插值函数对于给定点(x)的二阶微分。计算中使用插值对象spline，加速器xacc。对于x在xarr的区间外，将返回报错信息。

接口定义

int kml_float_spline_eval_deriv2_e(const kml_float_spline *spline, float x,

kml_spline_accel *a, float *d2)

float kml_float_spline_eval_deriv2(const kml_float_spline *spline, float x, kml_spline_accel *a)

int kml_double_spline_eval_deriv2_e(const kml_double_spline *spline, double x, kml_interp_accel *a, double *d2);

double kml_double_spline_eval_deriv2(const kml_double_spline *spline, double x, kml_interp_accel *a);

参数

参数名	类型	描述	输入/输出
spline	kml_float_spline*, kml_double_spline*	插值对象指针	输入
x	float, double	求解点x的值	输入
y	float, double	求解点y的值	输入
a	kml_spline_accel *	加速器对象指针	输入

依赖

#include "kipl.h"

示例

```
const size_t xMin = 4;
const size_t xMax = 2048;
const size_t xSize = 4;
const float xValLo = 1;
const float xValHi = 2;
float xArr[4] = {1,1.33,1.67,2};
float* yArr = (float*)malloc(xSize * sizeof(float));
float x = 1;
float y = 0;
kml_float_spline *spline = kml_float_spline_alloc(kml_float_interp_cspline, xSize);
kml_interp_accel *xacc = kml_interp_accel_alloc();
kml_float_spline_init(spline, xSize, xArr, yArr);
y = kml_float_spline_eval_deriv2(spline, xArr, x, xacc);
kml_float_spline_free(spline);
kml_interp_accel_free(xacc);
```

 版权所有 © 华为技术有限公司

?.4. kml_?_spline_eval_integ

这个函数返回插值函数对于给定点(x)的积分。计算中使用插值对象spline，加速器xacc。对于x在xarr的区间外，将返回报错信息。

接口定义

int kml_float_interp_eval_integ_e(const kml_float_spline *spline, float a, float b, kml_spline_accel *acc, float *result)

float kml_float_spline_eval_integ(const kml_float_spline *spline, float a, float b, kml_spline_accel *acc)

int kml_double_spline_eval_integ_e(const kml_double_spline *spline, double a, double b, kml_interp_accel *acc, double *result);

double kml_double_spline_eval_integ(const kml_double_spline *spline, double a, double b, kml_interp_accel *acc);

参数

参数名	类型	描述	输入/输出
spline	kml_float_spline*, kml_double_spline*	插值对象指针	输入
x	float, double	求解点x的值	输入
y	float, double	求解点y的值	输入
a	kml_spline_accel *	加速器对象指针	输入

依赖

#include "kipl.h"

示例

```
const size_t xMin = 4;
const size_t xMax = 2048;
const size_t xSize = 4;
const float xValLo = 1;
const float xValHi = 2;
float xArr[4] = {1,1.33,1.67,2};
float* yArr = (float*)malloc(xSize * sizeof(float));
float x = 1;
float y = 0;
kml_float_spline *spline = kml_float_spline_alloc(kml_float_interp_cspline, xSize);
kml_interp_accel *xacc = kml_interp_accel_alloc();
kml_float_spline_init(spline, xSize, xArr, yArr);
y = kml_float_spline_eval_integ(spline, xArr, x, xacc);
kml_float_spline_free(spline);
kml_interp_accel_free(xacc);
```

6.4.7.2.2.3 interp2d

版权所有 © 华为技术有限公司

?.1. kml_?_interp2d_eval

该函数返回给定点(x, y)的插值结果z。计算中使用插值对象interp，数据数组xarr，yarr和zarr以及加速器xacc和yacc。对于x在xarr的区间外或y在yarr的区间外，将返回报错信息。

接口定义

float kml_float_interp2d_eval(const kml_float_interp2d *interp, const float xarr[], const float yarr[], const float zarr[], const float x, const float y, kml_interp_accel *xa, kml_interp_accel *ya)

int kml_float_interp2d_eval_e(const kml_float_interp2d *interp, const float xarr[], const float yarr[], const float zarr[], const float x, const float y, kml_interp_accel *xa, kml_interp_accel *ya, float *z)

double kml_double_interp2d_eval(const kml_double_interp2d *interp, const double xarr[], const double yarr[],

const double zarr[], const double x, const double y, kml_interp_accel *xa,

kml_interp_accel *ya);

int kml_double_interp2d_eval_e(const kml_double_interp2d *interp, const double xarr[], const double yarr[],

const double zarr[], const double x, const double y, kml_interp_accel *xa,

kml_interp_accel *ya, double *z);

参数

参数名	类型	描述	输入/输出
interp	kml_float_interp2d*, kml_double_interp2d*	插值对象指针	输入
xarr	float, double	存有x值的数组	输入
yarr	float, double	存有y值的数组	输入
zarr	float, double	存有z值的数组	输入
x	float, double	求解点x的值	输入
y	float, double	求解点y的值	输入
xa	kml_interp_accel *	加速器对象指针	输入
ya	kml_interp_accel *	加速器对象指针	输入
z	float *, double *	插值点计算结果	输出

依赖

#include "kipl.h"

 版权所有 © 华为技术有限公司

示例

```
const size_t xMin = 4;
const size_t xMax = 2048;
const size_t xSize = 4;
const size_t yMin = 4;
const size_t yMax = 2048;
const size_t ySize = 4;
const float xValLo = 1;
const float xValHi = 2;
const float yValLo = 1;
const float yValHi = 2;
float xArr[4] = {1,1.33,1.67,2};
float yArr[4] = {1,1.33,1.67,2};
float* zArr = (float*)malloc(xSize * ySize * sizeof(float));
float x = 1;
float y = 1;
float z = 0;
kml_float_interp2d *interp = kml_float_interp2d_alloc(kml_float_interp2d_bicubic, xSize, ySize);
kml_interp_accel *xacc = kml_interp_accel_alloc();
kml_interp_accel *yacc = kml_interp_accel_alloc();
kml_float_interp2d_init(interp, xArr, yArr, zArr, xSize, ySize);
z = kml_float_interp2d_eval(interp, xArr, yArr, zArr, x, y, xacc, yacc);
```

?.2. kml_?_interp2d_eval_extrap

该函数返回给定点(x, y)的插值结果z。计算中使用插值对象interp，数据数组xarr，yarr和zarr以及加速器xacc和yacc。函数不检查最大值最小值越界，因此当x在xarr的区间外或y在yarr的区间外时，外推法被使用。

接口定义

float kml_float_interp2d_eval_extrap(const kml_float_interp2d *interp, const float xarr[], const float yarr[], const float zarr[], const float x, const float y, kml_interp_accel *xa, kml_interp_accel *ya)

int kml_float_interp2d_eval_extrap_e(const kml_float_interp2d *interp, const float xarr[], const float yarr[], const float zarr[], const float x, const float y, kml_interp_accel *xa, kml_interp_accel *ya, float *z)

double kml_double_interp2d_eval_extrap(const kml_double_interp2d *interp, const double xarr[], const double yarr[],

const double zarr[], const double x, const double y, kml_interp_accel *xa,

kml_interp_accel *ya);

int kml_double_interp2d_eval_extrap_e(const kml_double_interp2d *interp, const double xarr[], const double yarr[],

const double zarr[], const double x, const double y, kml_interp_accel *xa,

kml_interp_accel *ya, double *z);

参数

参数名	类型	描述	输入/输出
interp	kml_float_interp2d*, kml_double_interp2d*	插值对象指针	输入

版权所有 © 华为技术有限公司

参数名	类型	描述	输入/输出
xarr	float, double	存有x值的数组	输入
yarr	float, double	存有y值的数组	输入
zarr	float, double	存有z值的数组	输入
x	float, double	求解点x的值	输入
y	float, double	求解点y的值	输入
xa	kml_interp_accel *	加速器对象指针	输入
ya	kml_interp_accel *	加速器对象指针	输入
z	float *, double *	插值点计算结果	输出

依赖

#include "kipl.h"

示例

```
const size_t xMin = 4;
const size_t xMax = 2048;
const size_t xSize = 4;
const size_t yMin = 4;
const size_t yMax = 2048;
const size_t ySize = 4;
const float xValLo = 1;
const float xValHi = 2;
const float yValLo = 1;
const float yValHi = 2;
float xArr[4] = {1,1.33,1.67,2};
float yArr[4] = {1,1.33,1.67,2};
float* zArr = (float*)malloc(xSize * ySize * sizeof(float));
float x = 1;
float y = 1;
float z = 0;
kml_float_interp2d *interp = kml_float_interp2d_alloc(kml_float_interp2d_bicubic, xSize, ySize);
kml_interp_accel *xacc = kml_interp_accel_alloc();
kml_interp_accel *yacc = kml_interp_accel_alloc();
kml_float_interp2d_init(interp, xArr, yArr, zArr, xSize, ySize);
z = kml_float_interp2d_eval_extrap(interp, xArr, yArr, zArr, x, y, xacc, yacc);
```

?.3. kml_?_interp2d_eval_deriv_x

该函数返回插值函数对于给定点(x, y)的偏导。计算中使用插值对象interp，数据数组xarr，yarr和zarr以及加速器xacc和yacc。对于x在xarr的区间外或y在yarr的区间外，将返回报错信息。

接口定义

float kml_float_interp2d_eval_deriv_x(const kml_float_interp2d *interp, const float xarr[], const float yarr[], const float zarr[], const float x, const float y, kml_interp_accel *xa, kml_interp_accel *ya)

 版权所有 © 华为技术有限公司

int kml_float_interp2d_eval_deriv_x_e(const kml_float_interp2d *interp, const float xarr[], const float yarr[], const float zarr[], const float x, const float y, kml_interp_accel *xa, kml_interp_accel *ya, float *z)

double kml_double_interp2d_eval_deriv_x(const kml_double_interp2d *interp, const double xarr[], const double yarr[],

const double zarr[], const double x, const double y, kml_interp_accel *xa,

kml_interp_accel *ya);

int kml_double_interp2d_eval_deriv_x_e(const kml_double_interp2d *interp, const double xarr[], const double yarr[],

const double zarr[], const double x, const double y, kml_interp_accel *xa,

kml_interp_accel *ya, double *z);

参数

参数名	类型	描述	输入/输出
interp	kml_float_interp2d*, kml_double_interp2d*	插值对象指针	输入
xarr	float, double	存有x值的数组	输入
yarr	float, double	存有y值的数组	输入
zarr	float, double	存有z值的数组	输入
x	float, double	求解点x的值	输入
y	float, double	求解点y的值	输入
xa	kml_interp_accel *	加速器对象指针	输入
ya	kml_interp_accel *	加速器对象指针	输入
z	float *, double *	插值点偏导值∂z/∂x计算结果	输出

依赖

#include "kipl.h"

示例

```
const size_t xMin = 4;
const size_t xMax = 2048;
const size_t xSize = 4;
const size_t yMin = 4;
const size_t yMax = 2048;
const size_t ySize = 4;
const float xValLo = 1;
const float xValHi = 2;
const float yValLo = 1;
const float yValHi = 2;
float xArr[4] = {1,1.33,1.67,2};
```

 版权所有 © 华为技术有限公司

```
float yArr[4] = {1,1.33,1.67,2};
float* zArr = (float*)malloc(xSize * ySize * sizeof(float));
float x = 1;
float y = 1;
float z = 0;
kml_float_interp2d *interp = kml_float_interp2d_alloc(kml_float_interp2d_bicubic, xSize, ySize);
kml_interp_accel *xacc = kml_interp_accel_alloc();
kml_interp_accel *yacc = kml_interp_accel_alloc();
kml_float_interp2d_init(interp, xArr, yArr, zArr, xSize, ySize);
z = kml_float_interp2d_eval_deriv_x(interp, xArr, yArr, zArr, x, y, xacc, yacc);
```

?.4. kml_?_interp2d_eval_deriv_y

该函数返回插值函数对于给定点(x, y)的偏导。计算中使用插值对象interp，数据数组xarr，yarr和zarr以及加速器xacc和yacc。对于x在xarr的区间外或y在yarr的区间外，将返回报错信息。

接口定义

float kml_float_interp2d_eval_deriv_y(const kml_float_interp2d *interp, const float xarr[], const float yarr[], const float zarr[], const float x, const float y, kml_interp_accel *xa, kml_interp_accel *ya)

int kml_float_interp2d_eval_deriv_y_e(const kml_float_interp2d *interp, const float xarr[], const float yarr[], const float zarr[], const float x, const float y, kml_interp_accel *xa, kml_interp_accel *ya, float *z)

double kml_double_interp2d_eval_deriv_y(const kml_double_interp2d *interp, const double xarr[], const double yarr[],

const double zarr[], const double x, const double y, kml_interp_accel *xa,

kml_interp_accel *ya);

int kml_double_interp2d_eval_deriv_y_e(const kml_double_interp2d *interp, const double xarr[], const double yarr[],

const double zarr[], const double x, const double y, kml_interp_accel *xa,

kml_interp_accel *ya, double *z);

参数

参数名	类型	描述	输入/输出
interp	kml_float_interp2d*, kml_double_interp2d*	插值对象指针	输入
xarr	float, double	存有x值的数组	输入
yarr	float, double	存有y值的数组	输入
zarr	float, double	存有z值的数组	输入
x	float, double	求解点x的值	输入
y	float, double	求解点y的值	输入
xa	kml_interp_accel *	加速器对象指针	输入

 版权所有 © 华为技术有限公司

参数名	类型	描述	输入/输出
ya	kml_interp_accel *	加速器对象指针	输入
z	float *, double *	插值点偏导值∂z/∂y计算结果	输出

依赖

#include "kipl.h"

示例

```
const size_t xMin = 4;
const size_t xMax = 2048;
const size_t xSize = 4;
const size_t yMin = 4;
const size_t yMax = 2048;
const size_t ySize = 4;
const float xValLo = 1;
const float xValHi = 2;
const float yValLo = 1;
const float yValHi = 2;
float xArr[4] = {1,1.33,1.67,2};
float yArr[4] = {1,1.33,1.67,2};
float* zArr = (float*)malloc(xSize * ySize * sizeof(float));
float x = 1;
float y = 1;
float z = 0;
kml_float_interp2d *interp = kml_float_interp2d_alloc(kml_float_interp2d_bicubic, xSize, ySize);
kml_interp_accel *xacc = kml_interp_accel_alloc();
kml_interp_accel *yacc = kml_interp_accel_alloc();
kml_float_interp2d_init(interp, xArr, yArr, zArr, xSize, ySize);
z = kml_float_interp2d_eval_deriv_y(interp, xArr, yArr, zArr, x, y, xacc, yacc);
```

?.5. kml_?_interp2d_eval_deriv_xx

该函数返回插值函数对于给定点(x, y)的偏导。计算中使用插值对象interp，数据数组xarr，yarr和zarr以及加速器xacc和yacc。对于x在xarr的区间外或y在yarr的区间外，将返回报错信息。

接口定义

float kml_float_interp2d_eval_deriv_xx(const kml_float_interp2d *interp, const float xarr[], const float yarr[], const float zarr[], const float x, const float y, kml_interp_accel *xa, kml_interp_accel *ya)

int kml_float_interp2d_eval_deriv_xx_e(const kml_float_interp2d *interp, const float xarr[], const float yarr[], const float zarr[], const float x, const float y, kml_interp_accel *xa, kml_interp_accel *ya, float *z)

double kml_double_interp2d_eval_deriv_xx(const kml_double_interp2d *interp, const double xarr[], const double yarr[],

const double zarr[], const double x, const double y, kml_interp_accel *xa,

kml_interp_accel *ya);

 版权所有 © 华为技术有限公司

int kml_double_interp2d_eval_deriv_xx_e(const kml_double_interp2d *interp, const double xarr[], const double yarr[],

const double zarr[], const double x, const double y, kml_interp_accel *xa,

kml_interp_accel *ya, double *z);

参数

参数名	类型	描述	输入/输出
interp	kml_float_interp2d*, kml_double_interp2d*	插值对象指针	输入
xarr	float, double	存有x值的数组	输入
yarr	float, double	存有y值的数组	输入
zarr	float, double	存有z值的数组	输入
x	float, double	求解点x的值	输入
y	float, double	求解点y的值	输入
xa	kml_interp_accel *	加速器对象指针	输入
ya	kml_interp_accel *	加速器对象指针	输入
z	float *, double *	插值点二阶偏导值计算结果	输出

依赖

#include "kipl.h"

示例

```
const size_t xMin = 4;
const size_t xMax = 2048;
const size_t xSize = 4;
const size_t yMin = 4;
const size_t yMax = 2048;
const size_t ySize = 4;
const float xValLo = 1;
const float xValHi = 2;
const float yValLo = 1;
const float yValHi = 2;
float xArr[4] = {1,1.33,1.67,2};
float yArr[4] = {1,1.33,1.67,2};
float* zArr = (float*)malloc(xSize * ySize * sizeof(float));
float x = 1;
float y = 1;
float z = 0;
kml_float_interp2d *interp = kml_float_interp2d_alloc(kml_float_interp2d_bicubic, xSize, ySize);
kml_interp_accel *xacc = kml_interp_accel_alloc();
kml_interp_accel *yacc = kml_interp_accel_alloc();
kml_float_interp2d_init(interp, xArr, yArr, zArr, xSize, ySize);
z = kml_float_interp2d_eval_deriv_xx(interp, xArr, yArr, zArr, x, y, xacc, yacc);
```

 版权所有 © 华为技术有限公司

?.6. kml_?_interp2d_eval_deriv_yy

该函数返回插值函数对于给定点(x, y)的偏导。计算中使用插值对象interp，数据数组xarr，yarr和zarr以及加速器xacc和yacc。对于x在xarr的区间外或y在yarr的区间外，将返回报错信息。

接口定义

float kml_float_interp2d_eval_deriv_yy(const kml_float_interp2d *interp, const float xarr[], const float yarr[], const float zarr[], const float x, const float y, kml_interp_accel *xa, kml_interp_accel *ya)

int kml_float_interp2d_eval_deriv_yy_e(const kml_float_interp2d *interp, const float xarr[], const float yarr[], const float zarr[], const float x, const float y, kml_interp_accel *xa, kml_interp_accel *ya, float *z)

double kml_double_interp2d_eval_deriv_yy(const kml_double_interp2d *interp, const double xarr[], const double yarr[],

const double zarr[], const double x, const double y, kml_interp_accel *xa,

kml_interp_accel *ya);

int kml_double_interp2d_eval_deriv_yy_e(const kml_double_interp2d *interp, const double xarr[], const double yarr[],

const double zarr[], const double x, const double y, kml_interp_accel *xa,

kml_interp_accel *ya, double *z);

参数

参数名	类型	描述	输入/输出
interp	kml_float_interp2d*, kml_double_interp2d*	插值对象指针	输入
xarr	float, double	存有x值的数组	输入
yarr	float, double	存有y值的数组	输入
zarr	float, double	存有z值的数组	输入
x	float, double	求解点x的值	输入
y	float, double	求解点y的值	输入
xa	kml_interp_accel *	加速器对象指针	输入
ya	kml_interp_accel *	加速器对象指针	输入
z	float *, double *	插值点二阶偏导值计算结果	输出

依赖

#include "kipl.h"

 版权所有 © 华为技术有限公司

示例

```
const size_t xMin = 4;
const size_t xMax = 2048;
const size_t xSize = 4;
const size_t yMin = 4;
const size_t yMax = 2048;
const size_t ySize = 4;
const float xValLo = 1;
const float xValHi = 2;
const float yValLo = 1;
const float yValHi = 2;
float xArr[4] = {1,1.33,1.67,2};
float yArr[4] = {1,1.33,1.67,2};
float* zArr = (float*)malloc(xSize * ySize * sizeof(float));
float x = 1;
float y = 1;
float z = 0;
kml_float_interp2d *interp = kml_float_interp2d_alloc(kml_float_interp2d_bicubic, xSize, ySize);
kml_interp_accel *xacc = kml_interp_accel_alloc();
kml_interp_accel *yacc = kml_interp_accel_alloc();
kml_float_interp2d_init(interp, xArr, yArr, zArr, xSize, ySize);
z = kml_float_interp2d_eval_deriv_yy(interp, xArr, yArr, zArr, x, y, xacc, yacc);
```

?.7. kml_?_interp2d_eval_deriv_xy

该函数返回插值函数对于给定点(x, y)的偏导。计算中使用插值对象interp，数据数组xarr，yarr和zarr以及加速器xacc和yacc。对于x在xarr的区间外或y在yarr的区间外，将返回报错信息。

接口定义

float kml_float_interp2d_eval_deriv_xy(const kml_float_interp2d *interp, const float xarr[], const float yarr[], const float zarr[], const float x, const float y, kml_interp_accel *xa, kml_interp_accel *ya)

int kml_float_interp2d_eval_deriv_xy_e(const kml_float_interp2d *interp, const float xarr[], const float yarr[], const float zarr[], const float x, const float y, kml_interp_accel *xa, kml_interp_accel *ya, float *z)

double kml_double_interp2d_eval_deriv_xy(const kml_double_interp2d *interp, const double xarr[], const double yarr[],

const double zarr[], const double x, const double y, kml_interp_accel *xa,

kml_interp_accel *ya);

int kml_double_interp2d_eval_deriv_xy_e(const kml_double_interp2d *interp, const double xarr[], const double yarr[],

const double zarr[], const double x, const double y, kml_interp_accel *xa,

kml_interp_accel *ya, double *z);

参数

参数名	类型	描述	输入/输出
interp	kml_float_interp2d*, kml_double_interp2d*	插值对象指针	输入

 版权所有 © 华为技术有限公司

参数名	类型	描述	输入/输出
xarr	float, double	存有x值的数组	输入
yarr	float, double	存有y值的数组	输入
zarr	float, double	存有z值的数组	输入
x	float, double	求解点x的值	输入
y	float, double	求解点y的值	输入
xa	kml_interp_accel *	加速器对象指针	输入
ya	kml_interp_accel *	加速器对象指针	输入
z	float *, double *	插值点二阶偏导值计算结果	输出

依赖

#include "kipl.h"

示例

```
const size_t xMin = 4;
const size_t xMax = 2048;
const size_t xSize = 4;
const size_t yMin = 4;
const size_t yMax = 2048;
const size_t ySize = 4;
const float xValLo = 1;
const float xValHi = 2;
const float yValLo = 1;
const float yValHi = 2;
float xArr[4] = {1,1.33,1.67,2};
float yArr[4] = {1,1.33,1.67,2};
float* zArr = (float*)malloc(xSize * ySize * sizeof(float));
float x = 1;
float y = 1;
float z = 0;
kml_float_interp2d *interp = kml_float_interp2d_alloc(kml_float_interp2d_bicubic, xSize, ySize);
kml_interp_accel *xacc = kml_interp_accel_alloc();
kml_interp_accel *yacc = kml_interp_accel_alloc();
kml_float_interp2d_init(interp, xArr, yArr, zArr, xSize, ySize);
z = kml_float_interp2d_eval_deriv_xy(interp, xArr, yArr, zArr, x, y, xacc, yacc);
```

6.4.7.2.2.4 spline2d

?.1. kml_?_spline2d_eval

这个函数返回给定点(x, y)的插值结果z。计算中使用插值对象spline，数据数组xarr，yarr和zarr以及加速器xacc和yacc。对于x在xarr的区间外或y在yarr的区间外，将返回报错信息。

接口定义

float kml_float_spline2d_eval(const kml_float_spline2d *spline, const float x, const float y, kml_interp_accel *xa, kml_interp_accel *ya)

版权所有 © 华为技术有限公司

int kml_float_spline2d_eval_e(const kml_float_spline2d *spline, const float x, const float y, kml_interp_accel *xa, kml_interp_accel *ya, float *z)

double kml_double_spline2d_eval(const kml_double_spline2d *spline, const double x, const double y, kml_interp_accel *xa,

kml_interp_accel *ya);

int kml_double_spline2d_eval_e(const kml_double_spline2d *spline, const double x, const double y, kml_interp_accel *xa,

kml_interp_accel *ya, double *z);

参数

参数名	类型	描述	输入/输出
spline	kml_float_spline2d*, kml_double_spline2d*	插值对象指针	输入
x	float, double	求解点x的值	输入
y	float, double	求解点y的值	输入
xa	kml_interp_accel *	加速器对象指针	输入
ya	kml_interp_accel *	加速器对象指针	输入
z	float *, double *	插值点计算结果	输出

依赖

#include "kipl.h"

示例

```
const size_t xMin = 4;
const size_t xMax = 2048;
const size_t xSize = 4;
const size_t yMin = 4;
const size_t yMax = 2048;
const size_t ySize = 4;
const float xValLo = 1;
const float xValHi = 2;
const float yValLo = 1;
const float yValHi = 2;
float xArr[4] = {1,1.33,1.67,2};
float yArr[4] = {1,1.33,1.67,2};
float* zArr = (float*)malloc(xSize * ySize * sizeof(float));
float x = 1;
float y = 1;
float z = 0;
kml_float_spline2d *spline = kml_float_spline2d_alloc(kml_float_interp2d_bicubic, xSize, ySize);
kml_interp_accel *xacc = kml_interp_accel_alloc();
kml_interp_accel *yacc = kml_interp_accel_alloc();
kml_float_spline2d_init(spline, xArr, yArr, zArr, xSize, ySize);
z = kml_float_spline2d_eval(spline, x, y, xacc, yacc);
```

 版权所有 © 华为技术有限公司

?.2. kml_?_spline2d_eval_extrap

这个函数返回给定点(x, y)的插值结果z。计算中使用插值对象spline，数据数组xarr，yarr和zarr以及加速器xacc和yacc。函数不检查最大值最小值越界，因此当x在xarr的区间外或y在yarr的区间外时，外推法被使用。

接口定义

float kml_float_spline2d_eval_extrap(const kml_float_spline2d *spline, const float x, const float y, kml_interp_accel *xa, kml_interp_accel *ya)

int kml_float_spline2d_eval_extrap_e(const kml_float_spline2d *spline, const float x, const float y, kml_interp_accel *xa, kml_interp_accel *ya, float *z)

double kml_double_spline2d_eval_extrap(const kml_double_spline2d *spline, const double x, const double y,

kml_interp_accel *xa, kml_interp_accel *ya);

int kml_double_spline2d_eval_extrap_e(const kml_double_spline2d *spline, const double x, const double y,

kml_interp_accel *xa, kml_interp_accel *ya, double *z);

参数

参数名	类型	描述	输入/输出
spline	kml_float_spline2d*, kml_double_spline2d*	插值对象指针	输入
x	float, double	求解点x的值	输入
y	float, double	求解点y的值	输入
xa	kml_interp_accel *	加速器对象指针	输入
ya	kml_interp_accel *	加速器对象指针	输入
z	float *, double *	插值点计算结果	输出

依赖

#include "kipl.h"

示例

```
const size_t xMin = 4;
const size_t xMax = 2048;
const size_t xSize = 4;
const size_t yMin = 4;
const size_t yMax = 2048;
const size_t ySize = 4;
const float xValLo = 1;
const float xValHi = 2;
const float yValLo = 1;
const float yValHi = 2;
```

 版权所有 © 华为技术有限公司

```
float xArr[4] = {1,1.33,1.67,2};
float yArr[4] = {1,1.33,1.67,2};
float* zArr = (float*)malloc(xSize * ySize * sizeof(float));
float x = 1;
float y = 1;
float z = 0;
kml_float_spline2d *spline = kml_float_spline2d_alloc(kml_float_interp2d_bicubic, xSize, ySize);
kml_interp_accel *xacc = kml_interp_accel_alloc();
kml_interp_accel *yacc = kml_interp_accel_alloc();
kml_float_spline2d_init(spline, xArr, yArr, zArr, xSize, ySize);
z = kml_float_spline2d_eval_extrap(spline, x, y, xacc, yacc);
```

?.3. kml_?_spline2d_eval_deriv_x

这个函数返回插值函数对于给定点(x, y)的偏导。计算中使用插值对象spline，数据数组xarr，yarr和zarr以及加速器xacc和yacc。对于x在xarr的区间外或y在yarr的区间外，将返回报错信息。

接口定义

float kml_float_spline2d_eval_deriv_x(const kml_float_spline2d *spline, const float x, const float y, kml_interp_accel *xa, kml_interp_accel *ya)

int kml_float_spline2d_eval_deriv_x_e(const kml_float_spline2d *spline, const float x, const float y, kml_interp_accel *xa, kml_interp_accel *ya, float *z)

double kml_double_spline2d_eval_deriv_x(const kml_double_spline2d *spline, const double x, const double y,

kml_interp_accel *xa, kml_interp_accel *ya);

int kml_double_spline2d_eval_deriv_x_e(const kml_double_spline2d *spline, const double x, const double y,

kml_interp_accel *xa, kml_interp_accel *ya, double *z);

参数

参数名	类型	描述	输入/输出
spline	kml_float_spline2d*, kml_double_spline2d*	插值对象指针	输入
x	float, double	求解点x的值	输入
y	float, double	求解点y的值	输入
xa	kml_interp_accel *	加速器对象指针	输入
ya	kml_interp_accel *	加速器对象指针	输入
z	float *, double *	插值点偏导值∂z/∂x计算结果	输出

依赖

#include "kipl.h"

 版权所有 © 华为技术有限公司

示例

```
const size_t xMin = 4;
const size_t xMax = 2048;
const size_t xSize = 4;
const size_t yMin = 4;
const size_t yMax = 2048;
const size_t ySize = 4;
const float xValLo = 1;
const float xValHi = 2;
const float yValLo = 1;
const float yValHi = 2;
float xArr[4] = {1,1.33,1.67,2};
float yArr[4] = {1,1.33,1.67,2};
float* zArr = (float*)malloc(xSize * ySize * sizeof(float));
float x = 1;
float y = 1;
float z = 0;
kml_float_spline2d *spline = kml_float_spline2d_alloc(kml_float_interp2d_bicubic, xSize, ySize);
kml_interp_accel *xacc = kml_interp_accel_alloc();
kml_interp_accel *yacc = kml_interp_accel_alloc();
kml_float_spline2d_init(spline, xArr, yArr, zArr, xSize, ySize);
z = kml_float_spline2d_eval_deriv_x(spline, x, y, xacc, yacc);
```

?.4. kml_?_spline2d_eval_deriv_y

这个函数返回插值函数对于给定点(x, y)的偏导。计算中使用插值对象spline，数据数组xarr，yarr和zarr以及加速器xacc和yacc。对于x在xarr的区间外或y在yarr的区间外，将返回报错信息。

接口定义

float kml_float_spline2d_eval_deriv_y(const kml_float_spline2d *spline, const float x, const float y, kml_interp_accel *xa, kml_interp_accel *ya)

int kml_float_spline2d_eval_deriv_y_e(const kml_float_spline2d *spline, const float x, const float y, kml_interp_accel *xa, kml_interp_accel *ya, float *z)

double kml_double_spline2d_eval_deriv_y(const kml_double_spline2d *spline, const double x, const double y,

kml_interp_accel *xa, kml_interp_accel *ya);

int kml_double_spline2d_eval_deriv_y_e(const kml_double_spline2d *spline, const double x, const double y,

kml_interp_accel *xa, kml_interp_accel *ya, double *z);

参数

参数名	类型	描述	输入/输出
spline	kml_float_spline2d*, kml_double_spline2d*	插值对象指针	输入
x	float, double	求解点x的值	输入
y	float, double	求解点y的值	输入
xa	kml_interp_accel *	加速器对象指针	输入

 版权所有 © 华为技术有限公司

参数名	类型	描述	输入/输出
ya	kml_interp_accel *	加速器对象指针	输入
z	float *, double *	插值点偏导值∂z/∂y计算结果	输出

依赖

#include "kipl.h"

示例

```
const size_t xMin = 4;
const size_t xMax = 2048;
const size_t xSize = 4;
const size_t yMin = 4;
const size_t yMax = 2048;
const size_t ySize = 4;
const float xValLo = 1;
const float xValHi = 2;
const float yValLo = 1;
const float yValHi = 2;
float xArr[4] = {1,1.33,1.67,2};
float yArr[4] = {1,1.33,1.67,2};
float* zArr = (float*)malloc(xSize * ySize * sizeof(float));
float x = 1;
float y = 1;
float z = 0;
kml_float_spline2d *spline = kml_float_spline2d_alloc(kml_float_interp2d_bicubic, xSize, ySize);
kml_interp_accel *xacc = kml_interp_accel_alloc();
kml_interp_accel *yacc = kml_interp_accel_alloc();
kml_float_spline2d_init(spline, xArr, yArr, zArr, xSize, ySize);
z = kml_float_spline2d_eval_deriv_y(spline, x, y, xacc, yacc);
```

?.5. kml_?_spline2d_eval_deriv_xx

这个函数返回插值函数对于给定点(x, y)的偏导。计算中使用插值对象spline，数据数组xarr，yarr和zarr以及加速器xacc和yacc。对于x在xarr的区间外或y在yarr的区间外，将返回报错信息。

接口定义

float kml_float_spline2d_eval_deriv_xx(const kml_float_spline2d *spline, const float x, const float y, kml_interp_accel *xa, kml_interp_accel *ya)

int kml_float_spline2d_eval_deriv_xx_e(const kml_float_spline2d *spline, const float x, const float y, kml_interp_accel *xa, kml_interp_accel *ya, float *z)

double kml_double_spline2d_eval_deriv_xx(const kml_double_spline2d *spline, const double x, const double y,

kml_interp_accel *xa, kml_interp_accel *ya);

int kml_double_spline2d_eval_deriv_xx_e(const kml_double_spline2d *spline, const double x, const double y,

kml_interp_accel *xa, kml_interp_accel *ya, double *z);

 版权所有 © 华为技术有限公司

参数

参数名	类型	描述	输入/输出
spline	kml_float_spline2d*, kml_double_spline2d*	插值对象指针	输入
x	float, double	求解点x的值	输入
y	float, double	求解点y的值	输入
xa	kml_interp_accel *	加速器对象指针	输入
ya	kml_interp_accel *	加速器对象指针	输入
z	float *, double *	插值点二阶偏导值计算结果	输出

依赖

#include "kipl.h"

示例

```
const size_t xMin = 4;
const size_t xMax = 2048;
const size_t xSize = 4;
const size_t yMin = 4;
const size_t yMax = 2048;
const size_t ySize = 4;
const float xValLo = 1;
const float xValHi = 2;
const float yValLo = 1;
const float yValHi = 2;
float xArr[4] = {1,1.33,1.67,2};
float yArr[4] = {1,1.33,1.67,2};
float* zArr = (float*)malloc(xSize * ySize * sizeof(float));
float x = 1;
float y = 1;
float z = 0;
kml_float_spline2d *spline = kml_float_spline2d_alloc(kml_float_interp2d_bicubic, xSize, ySize);
kml_interp_accel *xacc = kml_interp_accel_alloc();
kml_interp_accel *yacc = kml_interp_accel_alloc();
kml_float_spline2d_init(spline, xArr, yArr, zArr, xSize, ySize);
z = kml_float_spline2d_eval_deriv_xx(spline, x, y, xacc, yacc);
```

?.6. kml_?_spline2d_eval_deriv_yy

这个函数返回插值函数对于给定点(x, y)的偏导。计算中使用插值对象spline，数据数组xarr，yarr和zarr以及加速器xacc和yacc。对于x在xarr的区间外或y在yarr的区间外，将返回报错信息。

接口定义

float kml_float_spline2d_eval_deriv_yy(const kml_float_spline2d *spline, const float x, const float y, kml_interp_accel *xa, kml_interp_accel *ya)

int kml_float_spline2d_eval_deriv_yy_e(const kml_float_spline2d *spline, const float x, const float y, kml_interp_accel *xa, kml_interp_accel *ya, float *z)

 版权所有 © 华为技术有限公司

double kml_double_spline2d_eval_deriv_yy(const kml_double_spline2d *spline, const double x, const double y,

kml_interp_accel *xa, kml_interp_accel *ya);

int kml_double_spline2d_eval_deriv_yy_e(const kml_double_spline2d *spline, const double x, const double y,

kml_interp_accel *xa, kml_interp_accel *ya, double *z);

参数

参数名	类型	描述	输入/输出
spline	kml_float_spline2d*, kml_double_spline2d*	插值对象指针	输入
x	float, double	求解点x的值	输入
y	float, double	求解点y的值	输入
xa	kml_interp_accel *	加速器对象指针	输入
ya	kml_interp_accel *	加速器对象指针	输入
z	float *, double *	插值点二阶偏导值计算结果	输出

依赖

#include "kipl.h"

示例

```
const size_t xMin = 4;
const size_t xMax = 2048;
const size_t xSize = 4;
const size_t yMin = 4;
const size_t yMax = 2048;
const size_t ySize = 4;
const float xValLo = 1;
const float xValHi = 2;
const float yValLo = 1;
const float yValHi = 2;
float xArr[4] = {1,1.33,1.67,2};
float yArr[4] = {1,1.33,1.67,2};
float* zArr = (float*)malloc(xSize * ySize * sizeof(float));
float x = 1;
float y = 1;
float z = 0;
kml_float_spline2d *spline = kml_float_spline2d_alloc(kml_float_interp2d_bicubic, xSize, ySize);
kml_interp_accel *xacc = kml_interp_accel_alloc();
kml_interp_accel *yacc = kml_interp_accel_alloc();
kml_float_spline2d_init(spline, xArr, yArr, zArr, xSize, ySize);
z = kml_float_spline2d_eval_deriv_yy(spline, x, y, xacc, yacc);
```

 版权所有 © 华为技术有限公司

?.7. kml_?_spline2d_eval_deriv_xy

这个函数返回插值函数对于给定点(x, y)的偏导。计算中使用插值对象spline，数据数组xarr，yarr和zarr以及加速器xacc和yacc。对于x在xarr的区间外或y在yarr的区间外，将返回报错信息。

接口定义

float kml_float_spline2d_eval_deriv_xy(const kml_float_spline2d *spline, const float x, const float y, kml_interp_accel *xa, kml_interp_accel *ya)

int kml_float_spline2d_eval_deriv_xy_e(const kml_float_spline2d *spline, const float x, const float y, kml_interp_accel *xa, kml_interp_accel *ya, float *z)

double kml_double_spline2d_eval_deriv_xy(const kml_double_spline2d *spline, const double x, const double y,

kml_interp_accel *xa, kml_interp_accel *ya);

int kml_double_spline2d_eval_deriv_xy_e(const kml_double_spline2d *spline, const double x, const double y,

kml_interp_accel *xa, kml_interp_accel *ya, double *z);

参数

参数名	类型	描述	输入/输出
spline	kml_float_spline2d*, kml_double_spline2d*	插值对象指针	输入
x	float, double	求解点x的值	输入
y	float, double	求解点y的值	输入
xa	kml_interp_accel *	加速器对象指针	输入
ya	kml_interp_accel *	加速器对象指针	输入
z	float *, double *	插值点二阶偏导值计算结果	输出

依赖

#include "kipl.h"

示例

```
const size_t xMin = 4;
const size_t xMax = 2048;
const size_t xSize = 4;
const size_t yMin = 4;
const size_t yMax = 2048;
const size_t ySize = 4;
const float xValLo = 1;
const float xValHi = 2;
const float yValLo = 1;
const float yValHi = 2;
```

 版权所有 © 华为技术有限公司

```
float xArr[4] = {1,1.33,1.67,2};
float yArr[4] = {1,1.33,1.67,2};
float* zArr = (float*)malloc(xSize * ySize * sizeof(float));
float x = 1;
float y = 1;
float z = 0;
kml_float_spline2d *spline = kml_float_spline2d_alloc(kml_float_interp2d_bicubic, xSize, ySize);
kml_interp_accel *xacc = kml_interp_accel_alloc();
kml_interp_accel *yacc = kml_interp_accel_alloc();
kml_float_spline2d_init(spline, xArr, yArr, zArr, xSize, ySize);
z = kml_float_spline2d_eval_deriv_xy(spline, x, y, xacc, yacc);
```

6.4.7.2.2.5 v_interp2d

?.1. kml_v_?_interp2d_eval_e

函数返回给定点(x, y)的插值结果z。计算中使用插值对象interp，数据数组xarr，yarr和zarr。需保证x数组在xarr的区间内或y数组在yarr的区间内。

接口定义

C interface:

int kml_v_float_interp2d_eval_e(const kml_double_interp2d *interp, const float xarr[], const float yarr[], const float zarr[], const float x[], const float y[], size_t xinSize, size_t yinSize, float *z, size_t zinSize);

int kml_v_double_interp2d_eval_e(const kml_double_interp2d *interp, const double xarr[], const double yarr[], const double zarr[], const double x[], const double y[], size_t xinSize, size_t yinSize, double *z, size_t zinSize);

参数

参数名	类型	描述	输入/输出
interp	kml_float_interp2d*, kml_double_interp2d*	插值对象指针	输入
xarr	float, double	存有x值的数组	输入
yarr	float, double	存有y值的数组	输入
zarr	float, double	存有z值的数组	输入
x	float, double	求解点x的值数组	输入
y	float, double	求解点y的值数组	输入
xinSize	size_t	x数组长度	输入
yinSize	size_t	y数组长度	输入
z	float *, double *	插值点计算结果数组指针	输出
zinSize	size_t	计算结果数组长度	输入

版权所有 © 华为技术有限公司

?.2. kml_v_?_interp2d_eval_extrap_e

函数返回给定点(x, y)的插值结果z。计算中使用插值对象interp，数据数组xarr，yarr和zarr。无需保证x数组在xarr的区间内或y数组在yarr的区间内。

接口定义

C interface:

int kml_v_float_interp2d_eval_extrap_e(const kml_float_interp2d *interp, const float xarr[], const float yarr[], const float zarr[], const float x[], const float y[], size_t xin_size, size_t yin_size, float *z, size_t zin_size);

int kml_v_double_interp2d_eval_extrap_e(const kml_double_interp2d *interp, const double xarr[], const double yarr[], const double zarr[], const double x[], const double y[], size_t xinSize, size_t yinSize, double *z, size_t zinSize);

参数

表 6-32 参数说明

参数名	类型	描述	输入/输出
interp	kml_float_interp2d*, kml_double_interp2d*	插值对象指针	输入
xarr	float, double	存有x值的数组	输入
yarr	float, double	存有y值的数组	输入
zarr	float, double	存有z值的数组	输入
x	float, double	求解点x的值数组	输入
y	float, double	求解点y的值数组	输入
xinSize	size_t	x数组长度	输入
yinSize	size_t	y数组长度	输入
z	float *, double *	插值点计算结果数组指针	输出
zinSize	size_t	计算结果数组长度	输入

?.3. kml_v_?_interp2d_eval_deriv_x_e

这些函数返回插值函数对于给定点(x, y)的偏导$d = \frac{\partial z}{\partial x}$。计算中使用插值对象interp，数据数组xarr，yarr和zarr。需保证x数组在xarr的区间内或y数组在yarr的区间内。

接口定义

C interface:

 版权所有 © 华为技术有限公司

int kml_v_float_interp2d_eval_deriv_x_e(const kml_float_interp2d *interp, const float xarr[], const float yarr[], const float zarr[], const float x[], const float y[], size_t xin_size, size_t yin_size, float *z, size_t zin_size);

int kml_v_double_interp2d_eval_deriv_x_e(const kml_double_interp2d *interp, const double xarr[], const double yarr[], const double zarr[], const double x[], const double y[], size_t xinSize, size_t yinSize, double *z, size_t zinSize);

参数

参数名	类型	描述	输入/输出
interp	kml_float_interp2d*, kml_double_interp2d*	插值对象指针	输入
xarr	float, double	存有x值的数组	输入
yarr	float, double	存有y值的数组	输入
zarr	float, double	存有z值的数组	输入
x	float, double	求解点x的值数组	输入
y	float, double	求解点y的值数组	输入
xinSize	size_t	x数组长度	输入
yinSize	size_t	y数组长度	输入
z	float *, double *	插值点偏导值∂z/∂x计算结果数组指针	输出
zinSize	size_t	计算结果数组长度	输入

?.4. kml_v_?_interp2d_eval_deriv_y_e

这些函数返回插值函数对于给定点(x, y)的偏导$d = \frac{\partial z}{\partial y}$。计算中使用插值对象interp，数据数组xarr，yarr和zarr。需保证x数组在xarr的区间内或y数组在yarr的区间内。

接口定义

C interface:

int kml_v_float_interp2d_eval_deriv_y_e(const kml_float_interp2d *interp, const float xarr[], const float yarr[], const float zarr[], const float x[], const float y[], size_t xin_size, size_t yin_size, float *z, size_t zin_size);

int kml_v_double_interp2d_eval_deriv_y_e(const kml_double_interp2d *interp, const double xarr[], const double yarr[], const double zarr[], const double x[], const double y[], size_t xinSize, size_t yinSize, double *z, size_t zinSize);

版权所有 © 华为技术有限公司

参数

参数名	类型	描述	输入/输出
interp	kml_float_interp2d*, kml_double_interp2d*	插值对象指针	输入
xarr	float, double	存有x值的数组	输入
yarr	float, double	存有y值的数组	输入
zarr	float, double	存有z值的数组	输入
x	float, double	求解点x的值数组	输入
y	float, double	求解点y的值数组	输入
xinSize	size_t	x数组长度	输入
yinSize	size_t	y数组长度	输入
z	float *, double *	插值点偏导值∂z/∂y计算结果数组指针	输出
zinSize	size_t	计算结果数组长度	输入

?.5. kml_v_?_interp2d_eval_deriv_xx_e

函数返回插值函数对于给定点(x, y)的偏导$d = \frac{\partial^2 z}{\partial x^2}$。计算中使用插值对象interp，数据数组xarr，yarr和zarr。需保证x数组在xarr的区间内或y数组在yarr的区间内。

接口定义

C interface:

int kml_v_float_interp2d_eval_deriv_xx_e(const kml_float_interp2d *interp, const float xarr[], const float yarr[], const float zarr[], const float x[], const float y[], size_t xin_size, size_t yin_size, float *z, size_t zin_size);

int kml_v_double_interp2d_eval_deriv_xx_e(const kpl_double_interp2d *interp, const double xarr[], const double yarr[], const double zarr[], const double x[], const double y[], size_t xinSize, size_t yinSize, double *z, size_t zinSize);

参数

参数名	类型	描述	输入/输出
interp	kml_float_interp2d*, kml_double_interp2d*	插值对象指针	输入
xarr	float, double	存有x值的数组	输入
yarr	float, double	存有y值的数组	输入

版权所有 © 华为技术有限公司

参数名	类型	描述	输入/输出
zarr	float, double	存有z值的数组	输入
x	float, double	求解点x的值数组	输入
y	float, double	求解点y的值数组	输入
xinSize	size_t	x数组长度	输入
yinSize	size_t	y数组长度	输入
z	float *, double *	$d = \frac{\partial^2 z}{\partial x^2}$ 插值点二阶偏导值 计算结果数组指针	输出
zinSize	size_t	计算结果数组长度	输入

?.6. kml_v_?_interp2d_eval_deriv_yy_e

函数返回插值函数对于给定点(x, y)的偏导 $d = \frac{\partial^2 z}{\partial y^2}$。计算中使用插值对象interp，数据数组xarr，yarr和zarr。需保证x数组在xarr的区间内或y数组在yarr的区间内。

接口定义

C interface:

int kml_v_float_interp2d_eval_deriv_yy_e(const kml_float_interp2d *interp, const float xarr[], const float yarr[], const float zarr[], const float x[], const float y[], size_t xin_size, size_t yin_size, float *z, size_t zin_size);

int kml_v_double_interp2d_eval_deriv_yy_e(const kml_double_interp2d *interp, const double xarr[], const double yarr[], const double zarr[], const double x[], const double y[], size_t xinSize, size_t yinSize, double *z, size_t zinSize);

参数

参数名	类型	描述	输入/输出
interp	kml_float_interp2d*, kml_double_interp2d*	插值对象指针	输入
xarr	float, double	存有x值的数组	输入
yarr	float, double	存有y值的数组	输入
zarr	float, double	存有z值的数组	输入
x	float, double	求解点x的值数组	输入
y	float, double	求解点y的值数组	输入
xinSize	size_t	x数组长度	输入

 版权所有 © 华为技术有限公司

参数名	类型	描述	输入/输出
yinSize	size_t	y数组长度	输入
z	float *, double *	$d = \frac{\partial^2 z}{\partial y^2}$ 插值点二阶偏导值 计算结果数组指针	输出
zinSize	size_t	计算结果数组长度	输入

?.7. kml_v_?_interp2d_eval_deriv_xy_e

函数返回插值函数对于给定点(x, y)的偏导 $d = \frac{\partial^2 z}{\partial x \partial y}$ 。计算中使用插值对象interp，数据数组xarr，yarr和zarr。需保证x数组在xarr的区间内或y数组在yarr的区间内。

接口定义

C interface:

int kml_v_float_interp2d_eval_deriv_xy_e(const kml_float_interp2d *interp, const float xarr[], const float yarr[], const float zarr[], const float x[], const float y[], size_t xin_size, size_t yin_size, float *z, size_t zin_size);

int kml_v_double_interp2d_eval_deriv_xy_e(const kml_double_interp2d *interp, const double xarr[], const double yarr[], const double zarr[], const double x[], const double y[], size_t xinSize, size_t yinSize, kml_interp_accel *xa, kml_interp_accel *ya, double *z, size_t zinSize);

参数

参数名	类型	描述	输入/输出
interp	kml_float_interp2d*, kml_double_interp2d*	插值对象指针	输入
xarr	float, double	存有x值的数组	输入
yarr	double	存有y值的数组	输入
zarr	float, double	存有z值的数组	输入
x	float, double	求解点x的值数组	输入
y	float, double	求解点y的值数组	输入
xinSize	size_t	x数组长度	输入
yinSize	size_t	y数组长度	输入

 版权所有 © 华为技术有限公司

参数名	类型	描述	输入/输出
z	float *, double *	插值点二阶偏导值 $d = \frac{\partial^2 z}{\partial x \partial y}$ 计算结果数组指针	输出
zinSize	size_t	计算结果数组长度	输入

6.4.7.2.2.6 v_spline2d

?.1. kml_v_?_spline2d_eval_e

函数返回给定点(x, y)的插值结果z。计算中使用插值对象spline，数据数组xarr，yarr和zarr。需保证x数组在xarr的区间内或y数组在yarr的区间内。

接口定义

C interface:

int kml_v_float_spline2d_eval_e(const kml_float_spline2d *interp, const float x[], const float y[], size_t xin_size, size_t yin_size, float *z, size_t zin_size);

int kml_v_double_spline2d_eval_e(const kml_double_spline2d *spline, const double x[], const double y[], size_t xinSize, size_t yinSize, double *z, size_t zinSize);

参数

参数名	类型	描述	输入/输出
spline	kml_float_spline2d *, kml_double_spline2d *	插值对象指针	输入
x	float, double	求解点x的值数组	输入
y	float, double	求解点y的值数组	输入
xinSize	size_t	x数组长度	输入
yinSize	size_t	y数组长度	输入
z	float *, double *	插值点计算结果数组指针	输出
zinSize	size_t	z数组长度	输入

?.2. kml_v_?_spline2d_eval_extrap_e

函数返回给定点(x, y)的插值结果z。计算中使用插值对象spline，数据数组xarr，yarr和zarr。无需保证x数组在xarr的区间内或y数组在yarr的区间内。

版权所有 © 华为技术有限公司

接口定义

C interface:

int kml_v_float_spline2d_eval_extrap_e(const kml_float_spline2d *interp, const float x[], const float y[], size_t xin_size, size_t yin_size, float *z, size_t zin_size);

int kml_v_double_spline2d_eval_extrap_e(const kml_double_spline2d *spline, const double x[], const double y[], size_t xinSize, size_t yinSize, double *z, size_t zinSize);

参数

参数名	类型	描述	输入/输出
spline	kml_float_spline2d *, kml_double_spline2d *	插值对象指针	输入
x	float, double	求解点x的值数组	输入
y	float, double	求解点y的值数组	输入
xinSize	size_t	x数组长度	输入
yinSize	size_t	y数组长度	输入
z	float *, double *	插值点计算结果数组指针	输出
zinSize	size_t	z数组长度	输入

?.3. kml_v_?_spline2d_eval_deriv_x_e

这些函数返回插值函数对于给定点(x, y)的偏导$d = \frac{\partial z}{\partial x}$。计算中使用插值对象spline，数据数组xarr，yarr和zarr。需保证x数组在xarr的区间内或y数组在yarr的区间内。

接口定义

C interface:

int kml_v_float_spline2d_eval_deriv_x_e(const kml_float_spline2d *interp, const float x[], const float y[], size_t xin_size, size_t yin_size, float *z, size_t zin_size);

int kml_v_double_spline2d_eval_deriv_x_e(const kml_double_spline2d *spline, const double x, const double y, size_t xinSize, size_t yinSize, double *z, size_t zinSize);

参数

参数名	类型	描述	输入/输出
spline	kml_float_spline2d *, kml_double_spline2d *	插值对象指针	输入
x	float, double	求解点x的值数组	输入
y	float, double	求解点y的值数组	输入

版权所有 © 华为技术有限公司

参数名	类型	描述	输入/输出
xinSize	size_t	x数组长度	输入
yinSize	size_t	y数组长度	输入
z	float *, double *	插值点偏导值∂z/∂x计算结果数组指针	输出
zinSize	size_t	z数组长度	输入

?.4. kml_v_?_spline2d_eval_deriv_y_e

这些函数返回插值函数对于给定点(x, y)的偏导$d = \frac{\partial z}{\partial y}$。计算中使用插值对象spline，数据数组xarr，yarr和zarr。需保证x数组在xarr的区间内或y数组在yarr的区间内。

接口定义

C interface:

int kml_v_float_spline2d_eval_deriv_y_e(const kml_float_spline2d *interp, const float x[], const float y[], size_t xin_size, size_t yin_size, float *z, size_t zin_size);

int kml_v_double_spline2d_eval_deriv_y_e(const kml_double_spline2d *spline, const double x, const double y, size_t xinSize, size_t yinSize, double *z, size_t zinSize);

参数

参数名	类型	描述	输入/输出
spline	kml_float_spline2d *, kml_double_spline2d *	插值对象指针	输入
x	float, double	求解点x的值数组	输入
y	float, double	求解点y的值数组	输入
xinSize	size_t	x数组长度	输入
yinSize	size_t	y数组长度	输入
z	float *, double *	插值点偏导值∂z/∂y计算结果数组指针	输出
zinSize	size_t	z数组长度	输入

?.5. kml_v_?_spline2d_eval_deriv_xx_e

函数返回插值函数对于给定点(x, y)的偏导$d = \frac{\partial^2 z}{\partial x^2}$。计算中使用插值对象spline，数据数组xarr，yarr和zarr。需保证x数组在xarr的区间内或y数组在yarr的区间内。

 版权所有 © 华为技术有限公司

接口定义

C interface:

int kml_v_float_spline2d_eval_deriv_xx_e(const kml_float_spline2d *interp, const float x[], const float y[], size_t xin_size, size_t yin_size, float *z, size_t zin_size);

int kml_v_double_spline2d_eval_deriv_xx_e(const kml_double_spline2d *spline, const double x, const double y, size_t xinSize, size_t yinSize, double *z, size_t zinSize);

参数

参数名	类型	描述	输入/输出
spline	kml_float_spline2d *, kml_double_spline2d *	插值对象指针	输入
x	float, double	求解点x的值数组	输入
y	float, double	求解点y的值数组	输入
xinSize	size_t	x数组长度	输入
yinSize	size_t	y数组长度	输入
z	float *, double *	插值点二阶偏导值计算结果数组指针 $d = \frac{\partial^2 z}{\partial x^2}$	输出
zinSize	size_t	z数组长度	输入

?.6. kml_v_?_spline2d_eval_deriv_yy_e

函数返回插值函数对于给定点(x, y)的偏导 $d = \frac{\partial^2 z}{\partial y^2}$。计算中使用插值对象spline，数据数组xarr，yarr和zarr。需保证x数组在xarr的区间内或y数组在yarr的区间内。

接口定义

C interface:

int kml_v_float_spline2d_eval_deriv_yy_e(const kml_float_spline2d *interp, const float x[], const float y[], size_t xin_size, size_t yin_size, float *z, size_t zin_size);

int kml_v_double_spline2d_eval_deriv_yy_e(const kml_double_spline2d *spline, const double x, const double y, size_t xinSize, size_t yinSize, double *z, size_t zinSize);

 版权所有 © 华为技术有限公司

参数

参数名	类型	描述	输入/输出
spline	kml_float_spline2d *, kml_double_spline2d *	插值对象指针	输入
x	float, double	求解点x的值数组	输入
y	float, double	求解点y的值数组	输入
xinSize	size_t	x数组长度	输入
yinSize	size_t	y数组长度	输入
z	float *, double *	$d = \frac{\partial^2 z}{\partial y^2}$ 插值点二阶偏导值计算结果数组指针	输出
zinSize	size_t	z数组长度	输入

?.7. kml_v_?_spline2d_eval_deriv_xy_e

函数返回插值函数对于给定点(x, y)的偏导 $d = \frac{\partial^2 z}{\partial x \partial y}$。计算中使用插值对象spline，数据数组xarr，yarr和zarr。需保证x数组在xarr的区间内或y数组在yarr的区间内。

接口定义

C interface:

int kml_v_float_spline2d_eval_deriv_xy_e(const kml_float_spline2d *interp, const float x[], const float y[], size_t xin_size, size_t yin_size, float *z, size_t zin_size);

int kml_v_double_spline2d_eval_deriv_xy_e(const kml_double_spline2d *spline, const double x, const double y, size_t xinSize, size_t yinSize, double *z, size_t zinSize);

参数

参数名	类型	描述	输入/输出
spline	kml_float_spline2d *, kml_double_spline2d *	插值对象指针	输入
x	float, double	求解点x的值数组	输入
y	float, double	求解点y的值数组	输入
xinSize	size_t	x数组长度	输入
yinSize	size_t	y数组长度	输入

版权所有 © 华为技术有限公司

参数名	类型	描述	输入/输出
z	float *, double *	插值点二阶偏导值 $d = \frac{\partial^2 z}{\partial x \partial y}$ 计算结果数组指针	输出
zinSize	size_t	z数组长度	输入

6.4.7.2.2.7 basis 样条函数

?.1. KIPL_Bsnak

计算“非节点”样条线节点序列。

接口定义

void KIPL_Bsnak(int nData, float *xData, int k, float *xKnot)

参数

参数名	类型	描述	输入/输出
nData	整型数	表示xData数组元素个数。	输入
xData	单精度浮点数组	长度为nData的数组，表示数据点的横坐标。	输入
k	整型数	表示样条的阶数。	输入
xKnot	单精度浮点数组	长度为nData+k的数组，表示生成的节点序列。	输出

依赖

#include "kipl.h"

?.2. KIPL_Bsint

计算basis样条插值的系数。

接口定义

void KIPL_Bsint(int nData, float *xData, float *fData, int k, float *xKnot, float *coef)

版权所有 © 华为技术有限公司

参数

参数名	类型	描述	输入/输出
nData	整型数	表示xData数组元素个数。	输入
xData	单精度浮点数组	长度为nData的数组，表示数据点的横坐标。	输入
fData	单精度浮点数组	长度为nData的数组，表示数据点的纵坐标。	输入
k	整型数	表示样条的阶数。	输入
xKnot	单精度浮点数组	长度为nData+k的数组，表示KIPL_Bsnak生成的节点序列。	输入
coef	单精度浮点数组	长度为nData的数组，表示生成的系数序列。	输出

依赖

#include "kipl.h"

?.3. KIPL_Bsval

计算basis样条拟合函数在位置x处的值。

接口定义

float KIPL_Bsval(float xi, int k, float *xKnot, int ncoef, float *coef)

参数

参数名	类型	描述	输入/输出
xi	单精度浮点数	需要计算拟合值的位置。	输入
ncoef	整型数	表示coef数组长度，与nData相等。	输入
k	整型数	表示样条的阶数。	输入
xKnot	单精度浮点数组	长度为nData+k的数组，表示KIPL_Bsnak生成的节点序列。	输入
coef	单精度浮点数组	长度为nData的数组，表示生成的系数序列。	输入
返回值	单精度浮点数	basis样条拟合函数在位置x处的值。	输出

 版权所有 © 华为技术有限公司

依赖

#include "kipl.h"

?.4. KIPL_Bsder

计算basis样条拟合函数在位置x处的导数值。

接口定义

float KIPL_Bsder(int ideriv, float xi, int k, float *xKnot, int ncoef, float *coef)

参数

参数名	类型	描述	输入/输出
ideriv	整型数	需要计算的导数阶数。	输入
xi	单精度浮点数	需要计算导数值的位置。	输入
ncoef	整型数	表示coef数组长度，与nData相等。	输入
k	整型数	表示样条的阶数。	输入
xKnot	单精度浮点数组	长度为nData+k的数组，表示KIPL_Bsnak生成的节点序列。	输入
coef	单精度浮点数组	长度为nData的数组，表示生成的系数序列。	输入
返回值	单精度浮点数	basis样条拟合函数在位置x处的导数值。	输出

依赖

#include "kipl.h"

?.5. 示例

```
#include <stdio.h>
#include <math.h>
#include "kipl.h"

int main()
{
    const int data_n = 11;
    float *data_x = (float *)malloc(data_n * sizeof(float));
    float *data_y = (float *)malloc(data_n * sizeof(float));
    for (int i = 0; i < data_n; i++) {
        data_x[i] = (float)(i) / (data_n - 1);
        data_y[i] = sin(15.0 * data_x[i]);
    }
    int test_n = 2 * data_n - 1;
    float *test_x = （ float * ）malloc(test_n * sizeof(float));
```

 版权所有 © 华为技术有限公司

```
    float *test_y = （ float * ）malloc(test_n * sizeof(float));
    for (int i = 0; i < test_n; i++) {
        test_x[i] = (float)(i) / (test_n - 1);
        test_y[i] = sin(15.0 * test_x[i]);
    }
    const int k = 3;
    float *bscoef = (float *)malloc(data_n * sizeof(float));
    float *xknot = (float *)malloc((data_n + k) * sizeof(float));
    int ideriv = 0;

    KIPL_Bsnak(data_n, data_x, k, xknot);
    KIPL_Bsint(data_n, data_x, data_y, k, xknot, bscoef);

    for (int i = 0; i < test_n; i++) {
        float y, deriv;
        float x = test_x[i];
        y = KIPL_Bsval(x, k, xknot, data_n, bscoef);
        deriv = KIPL_Bsder(ideriv, x, k, xknot, data_n, bscoef);
    }
    free(data_x);
    free(data_y);
    return 0;
}
```

6.4.7.2.2.8 三次样条函数

?.1. KIPL_Csint

计算三次样条插值的系数。

接口定义

void KIPL_Csint(int nData, float *xData, float *fData, float *coef)

参数

参数名	类型	描述	输入/输出
nData	整型数	表示xData数组元素个数。	输入
xData	单精度浮点数组	长度为nData的数组，表示数据点的横坐标。	输入
fData	单精度浮点数组	长度为nData的数组，表示数据点的纵坐标。	输入
coef	单精度浮点数组	长度为4*nData的数组，表示生成的系数序列。	输出

依赖

#include "kipl.h"

?.2. KIPL_Cssmh

计算平滑的三次样条系数逼近噪声数据。

 版权所有 © 华为技术有限公司

接口定义

void KIPL_Cssmh(int nData, float *xData, float *fData, float *weight, float smpar, float *coef)

参数

参数名	类型	描述	输入/输出
nData	整型数	表示xData数组元素个数。	输入
xData	单精度浮点数组	长度为nData的数组，表示数据点的横坐标。	输入
fData	单精度浮点数组	长度为nData的数组，表示数据点的纵坐标。	输入
weight	单精度浮点数组	包含fData标准偏差估计值的长度为nData的数组	输入
smpar	单精度浮点数	控制平滑度的非负数。	输入
coef	单精度浮点数组	长度为4*nData的数组，表示生成的系数序列。	输出

依赖

#include "kipl.h"

?.3. KIPL_Csval

计算三次样条拟合函数在位置x的值。

接口定义

float KIPL_Csval(float xi, int initv, float *xData, float *coef)

参数

参数名	类型	描述	输入/输出
xi	单精度浮点数	需要计算三次样条拟合值的位置。	输入
initv	整型数	表示拟合得到三次多项式个数，等于nData-1。	输入
xData	单精度浮点数组	长度为initv+1的数组，表示数据点的横坐标。	输入

 版权所有 © 华为技术有限公司

参数名	类型	描述	输入/输出
coef	单精度浮点数组	长度为4*(initv+1)的数组，表示生成的系数序列。	输出

依赖

#include "kipl.h"

?.4. KIPL_Cs1gd

计算三次样条拟合函数的导数值。

接口定义

void KIPL_Cs1gd(int ideriv, int nVec, float *xVec, int initv, float *xData, float *coef, float *derValue)

参数

参数名	类型	描述	输入/输出
ideriv	整型数	需要计算的导数阶数。	输入
nVec	整型数	数组xVec的长度。	输入
xVec	单精度浮点数组	长度为nVec的数组，包含要计算三次样条的点。	输入
initv	整型数	表示拟合得到三次多项式个数，等于nData-1。	输入
xData	单精度浮点数组	长度为initv+1的数组，表示数据点的横坐标。	输入
coef	单精度浮点数组	长度为4*(initv+1)的数组，表示生成的系数序列。	输入
derValue	单精度浮点数组	长度为nVec的数组，包含三次样条在xVec各点的ideriv阶导数的值。	输出

依赖

#include "kipl.h"

?.5. 示例

```
#include <stdio.h>
#include <math.h>
```

 版权所有 © 华为技术有限公司

```
#include "kipl.h"

int main()
{
    int data_n = 11;
    float *data_x = (float *)malloc(data_n * sizeof(float));
    float *data_y = (float *)malloc(data_n * sizeof(float));

    for (int i = 0; i < data_n; i++) {
        data_x[i] = (float)(i) / (data_n - 1);
        data_y[i] = sin(15.0 * data_x[i]);
    }
    int test_n = 2 * data_n - 1;
    float test_x[test_n];
    float test_y[test_n];
    for (int i = 0; i < test_n; i++) {
        test_x[i] = (float)(i) / (test_n - 1);
        test_y[i] = sin(15.0 * test_x[i]);
    }
    float b[data_n], cscoef[4 * data_n];
    KIPL_Csint(data_n, data_x, data_y, cscoef);

    int nintv = data_n - 1;
    for (int i = 0; i < test_n; i++) {
        float y;
        float x = test_x[i];
        y = KIPL_Csval(x, nintv, data_x, cscoef);
        printf("%f  %f\n", x, y);
    }
    int nintv = data_n - 1;
    int ideriv = 0;
    float derValue[test_n];
    KIPL_Cs1gd(ideriv, test_n, test_x, nintv, data_x, cscoef, derValue);

    free(data_x);
    free(data_y);
    return 0;
}
#include <stdio.h>
#include <math.h>
#include "kipl.h"

int main()
{
    int data_n = 11;
    float *data_x = (float *)malloc(data_n * sizeof(float));
    float *data_y = (float *)malloc(data_n * sizeof(float));
    for (int i = 0; i < data_n; i++) {
        data_x[i] = (float)(i) / (data_n - 1);
        data_y[i] = sin(15.0 * data_x[i]);
    }
    int test_n = 2 * data_n - 1;
    float test_x[test_n];
    float test_y[test_n];
    for (int i = 0; i < test_n; i++) {
        test_x[i] = (float)(i) / (test_n - 1);
        test_y[i] = sin(15.0 * test_x[i]);
    }
    float b[data_n], cscoef[4 * data_n];
    float weight[data_n];
    float smpar = data_n;
    for (int i = 0; i < data_n; i++) {
        weight[i] = 0.1;
    }
    KIPL_Cssmh(data_n, data_x, data_y, weight, smpar, cscoef);
    int nintv = data_n - 1;
    for (int i = 0; i < test_n; i++) {
        float y;
        float x = test_x[i];
```

 版权所有 © 华为技术有限公司

```
        y = KIPL_Csval(x, nintv, data_x, cscoef);
        printf("%f  %f\n", x, y);
    }
    free(data_x);
    free(data_y);
    return 0;
}
```

6.4.7.2.3 加速函数

6.4.7.2.3.1 kml_interp_accel_alloc

该函数返回一个加速器对象的指针，作为插值查找的迭代器。当多个插值使用时，同一个加速器对象可以被同一个区间的数据使用，不同的区间的数据需要使用不同的加速器。

接口定义

kml_interp_accel *kml_interp_accel_alloc(void)

参数

参数名	类型	描述	输入/输出
NA	NA	NA	NA

依赖

#include "kml_interp.h"

示例

```
kml_interp_accel *acc = kml_interp_accel_alloc();
```

6.4.7.2.3.2 kml_interp_accel_reset

该函数重置加速器对象a。它需要在信息不再适用时被使用，例如，切换到一个新的数据数组。

接口定义

int kml_interp_accel_reset(kml_interp_accel *a)

参数

参数名	类型	描述	输入/输出
a	kml_interp_accel *	加速器指针	输出

版权所有 © 华为技术有限公司

依赖

#include "kml_interp.h"

示例

```
kml_interp_accel *acc = kml_interp_accel_alloc();
kml_interp_accel_reset(acc);
```

6.4.7.2.3.3 kml_interp_accel_free

该函数释放加速器对象a。

接口定义

int kml_interp_accel_free(kml_interp_accel *a)

参数

参数名	类型	描述	输入/输出
a	kml_interp_accel *	加速器指针	输入

依赖

#include "kml_interp.h"

示例

```
kml_interp_accel *acc = kml_interp_accel_alloc();
kml_interp_accel_free(acc);
```

6.4.8 KML_BLAS 库函数说明

6.4.8.1 函数说明

BLAS（Basic Linear Algebra Subprograms）提供了一系列基本线性代数运算函数的标准接口，包括矢量线性组合、矩阵乘以矢量、矩阵乘以矩阵等功能。BLAS已被广泛应用于工业界和科学计算，成为业界标准。KML_BLAS库提供BLAS函数的C语言接口。

说明

鲲鹏920专业版硬件环境下使用KML_BLAS共享库时，需要安装libnuma共享库，否则将无法正常使用。

6.4.8.2 数据结构

提供KML_BLAS库函数的数据类型说明。

 版权所有 © 华为技术有限公司

数据类型

数据类型	类型描述
s	单精度浮点类型
d	双精度浮点类型
c	单精度复数类型
z	双精度复数类型
b	半精度浮点类型
h	半精度浮点类型

宏定义

```
#define CBLAS_INDEX size_t
```

枚举类型

表6-33描述了KML_BLAS的枚举类型。

表 6-33 枚举类型及定义

枚举类型	说明	定义
CBLAS_ORDER	定义矩阵存储为行主序还是列主序。	enum CBLAS_ORDER { CblasRowMajor=101, CblasColMajor=102 }CBLAS_ORDER;
CBLAS_TRANSPOSE	定义常规矩阵或共轭矩阵是否进行转置。	enum CBLAS_TRANSPOSE { CblasNoTrans=111, CblasTrans=112, CblasConjTrans=113, CblasConjNoTrans=114 } CBLAS_TRANSPOSE;
CBLAS_UPLO	定义使用矩阵的上三角还是下三角部分。	enum CBLAS_UPLO { CblasUpper=121, CblasLower=122 } CBLAS_UPLO;
CBLAS_DIAG	定义使用矩阵对角线是否为单位元。	enum CBLAS_DIAG { CblasNonUnit=131, CblasUnit=132 } CBLAS_DIAG;

版权所有 © 华为技术有限公司

枚举类型	说明	定义
CBLAS_SIDE	定义埃尔米特矩阵为左操作还是右操作元素。	enum CBLAS_SIDE { CblasLeft=141, CblasRight=142 } CBLAS_SIDE;
CBLAS_IDENTIFIER	定义执行pack的是A还是B矩阵。	typedef enum CBLAS_IDENTIFIER { CblasA = 151, CblasB = 152 } CBLAS_IDENTIFIER;

6.4.8.3 环境变量

以下环境变量会被KML_BLAS使用。

表 6-34 变量名说明

环境变量名	变量意义	有效值	默认值
BLAS_VERBOSE	告警等级，如果该值为1，则会输出info信息（组件版本、接口名称等），如果该值大于1，则会输出debug信息（组件版本、接口名称、接口参数、执行耗时等）	整数	0
BLAS_THREAD_TIMEOUT	循环等待时间，设置用于PTHREAD多线程后端模式下线程间数据同步等待的最大时间	4~30	0
BLAS_NUM_THREADS	BLAS内置线程数，用于设置BLAS的并行执行线程数，优先级高于'OMP_NUM_THREADS'	整数	0
OMP_NUM_THREADS	OMP内置线程数，用于设置BLAS的并行执行线程数	整数	0

版权所有 © 华为技术有限公司

环境变量名	变量意义	有效值	默认值
BLAS_JC_NT	用于设置多线程场景下矩阵操作时n方向切分块策略，其中'BLAS_JC_NT'与'BLAS_IC_NT'的乘积必须等于设置的线程数，否则将使用默认的分块策略	整数	0
BLAS_IC_NT	于设置多线程场景下矩阵操作时m方向切分块策略 其中'BLAS_JC_NT'与'BLAS_IC_NT'的乘积必须等于设置的线程数，否则将使用默认的分块策略	整数	0
BLAS_NUMA_THREADS	单numa的核数设置，用于在PTHREAD多线程后端模式下线程的numa亲和性	正整数	0
KML_BLAS_THREAD_TYPE	线程管理后端方式	PTHREAD\OMP\KUPL	OMP
SGEMM_P	单精度浮点类型gemm的m方向切分块大小，只在鲲鹏920 7270、7280上有效	正整数，鲲鹏9207270、7280上必须为48的整倍数，SGEMM_P*SGEMM_Q*4*2 ≤ 37748736	128
SGEMM_Q	单精度浮点类型gemm的k方向切分块大小，只在鲲鹏920 7270、7280上有效	正整数，鲲鹏9207270、7280上必须为48的整倍数，SGEMM_P*SGEMM_Q*8*2 ≤ 37748736	512
SGEMM_R	单精度浮点类型gemm的n方向切分块大小，只在鲲鹏920 7270、7280上有效	正整数，鲲鹏9207270、7280上必须为48的整倍数，SGEMM_Q*SGEMM_R*4*2 ≤ 37748736	4096

版权所有 © 华为技术有限公司

环境变量名	变量意义	有效值	默认值
DGEMM_P	双精度浮点类型gemm的m方向切分块大小，只在鲲鹏920 7270、7280上有效	正整数，鲲鹏9207270、7280上必须为24的整倍数，DGEMM_P*DGEMM_Q*8*2 ≤ 37748736	128
DGEMM_Q	双精度浮点类型gemm的k方向切分块大小，只在鲲鹏920 7270、7280上有效	正整数，鲲鹏9207270、7280上必须为24的整倍数，DGEMM_Q*DGEMM_R*8*2 ≤ 37748736	512
DGEMM_R	双精度浮点类型gemm的n方向切分块大小，只在鲲鹏920 7270、7280上有效	正整数，鲲鹏9207270、7280上必须为24的整倍数，DGEMM_Q*DGEMM_R*8*2 ≤ 37748736	4096
CGEMM_P	单精度复数类型gemm的m方向切分块大小，只在鲲鹏920 7270、7280上有效	正整数，鲲鹏9207270、7280上必须为8的整倍数，CGEMM_P*CGEMM_Q*8*2 ≤ 37748736	64
CGEMM_Q	单精度复数类型gemm的k方向切分块大小，只在鲲鹏920 7270、7280上有效	正整数，鲲鹏9207270、7280上必须为8的整倍数，CGEMM_Q*CGEMM_R*8*2 ≤ 37748736	512
CGEMM_R	单精度复数类型gemm的n方向切分块大小，只在鲲鹏920 7270、7280上有效	正整数，鲲鹏9207270、7280上必须为8的整倍数，CGEMM_Q*CGEMM_R*8*2 ≤ 37748736	4096
ZGEMM_P	双精度复数类型gemm的m方向切分块大小，只在鲲鹏920 7270、7280上有效	正整数，鲲鹏9207270、7280上必须为8的整倍数，ZGEMM_P*ZGEMM_Q*16*2 ≤ 37748736	64

版权所有 © 华为技术有限公司

环境变量名	变量意义	有效值	默认值
ZGEMM_Q	双精度复数类型gemm的k方向切分块大小，只在鲲鹏920 7270、7280上有效	正整数，鲲鹏920 7270、7280上必须为8的整倍数，ZGEMM_Q*ZGEMM_R*16*2 ≤ 37748736	320
ZGEMM_R	双精度复数类型gemm的n方向切分块大小，只在鲲鹏920 7270、7280上有效	正整数，鲲鹏920 7270、7280上必须为8的整倍数，ZGEMM_Q*ZGEMM_R*16*2 ≤ 37748736	2048

6.4.8.4 函数定义

6.4.8.4.1 KML_BLAS Level 1 函数

6.4.8.4.1.1 函数说明

对KML_BLAS Level1所有函数族进行描述。

函数族	数据类型	描述
?asum	s，d，sc，dz	向量元素绝对值求和。
?axpy	s，d，c，z	向量缩放与加和。
?copy	s，d，c，z	向量拷贝。
?dot	s，d	向量点积。
?dotc	c，z	共轭向量与另一向量的点积。
?dotu	c，z	复数向量点积。
?sdot	d，sd	扩展精度点积。
i?amax	s，d，c，z	向量中最大绝对值的索引。
i?amin	s，d，c，z	向量中最小绝对值的索引。
?nrm2	s，d，sc，dz	计算欧几里得范数。
?rot	s，d，sc，dz	基于平面上点旋转。
?rotg	s，d	构造旋转平面。
?rotm	s，d	修改旋转平面。
?rotmg	s，d	构造修改的转换平面。

版权所有 © 华为技术有限公司

函数族	数据类型	描述
?scal	s，d，c，z，cs，zd	向量缩放。
?swap	s，d，c，z	两个向量元素交换。

6.4.8.4.1.2 ?asum

向量元素绝对值求和（对于复数，则实部和虚部分别取绝对值后再求和）。

接口定义

C interface：

float cblas_sasum(const BLASINT n, const float *x, const BLASINT incx);

double cblas_dasum(const BLASINT n, const double *x, const BLASINT incx);

float cblas_scasum(const BLASINT n, const void *x, const BLASINT incx);

double cblas_dzasum(const BLASINT n, const void *x, const BLASINT incx);

Fortran interface：

RES = SASUM(N, X, INCX)

RES = DASUM(N, X, INCX)

RES = SCASUM(N, X, INCX)

RES = DZASUM(N, X, INCX)

参数

参数名	类型	描述	输入/输出
n	整型数	表示x向量元素个数。	输入
x	• 在dasum中是双精度浮点类型。 • 在sasum中是单精度浮点类型。 • 在scasum中是单精度复数类型。 • 在zasum中是双精度复数类型。	向量x，向量规模至少是(1+(n-1)*abs(incx))。	输入
incx	整型数	表示x向量增长步长。	输入

版权所有 © 华为技术有限公司

依赖

#include "kblas.h"

示例

C interface：

```
int n = 5, incx = 1, incy = 1;
/*
 *  Input X:  0.340188, -0.105617, 0.283099, 0.298440, 0.411647
 */
float x[5] = {0.340188, -0.105617, 0.283099, 0.298440, 0.411647};

float res = cblas_sasum(n, x, incx);
/*
 *  Output : 1.438991
 */
```

Fortran interface：

```
    INTEGER :: N=5
    INTEGER :: INCX=1
    REAL(4) X(5)
    DATA X /0.340188, -0.105617, 0.283099, 0.298440, 0.411647/
    RES=SASUM(N,X,INCX)

*     Output : 1.438991
```

6.4.8.4.1.3 ?axpy

向量缩放后的乘积与另一向量的加和。

即$y = \alpha * x + y$。alpha是乘法系数，x和y是长度为n的向量。

接口定义

C interface：

void cblas_saxpy(const BLASINT n, const float alpha, const float *x, const BLASINT incx, float *y, const BLASINT incy);

void cblas_daxpy(const BLASINT n, const double alpha, const double *x, const BLASINT incx, double *y, const BLASINT incy);

void cblas_caxpy(const BLASINT n, const void *alpha, const void *x, const BLASINT incx, void *y, const BLASINT incy);

void cblas_zaxpy(const BLASINT n, const void *alpha, const void *x, const BLASINT incx, void *y, const BLASINT incy);

Fortran interface：

CALL SAXPY(N, ALPHA, X, INCX, Y, INCY)

CALL DAXPY(N, ALPHA, X, INCX, Y, INCY)

CALL CAXPY(N, ALPHA, X, INCX, Y, INCY)

CALL ZAXPY(N, ALPHA, X, INCX, Y, INCY)

 版权所有 © 华为技术有限公司

参数

参数名	类型	描述	输入/输出
n	整型数	表示x向量的元素个数。	输入
alpha	• 在daxpy中是双精度浮点类型。 • 在saxpy中是单精度浮点类型。 • 在caxpy中是单精度复数类型。 • 在zaxpy中是双精度复数类型。	表示乘法系数。	输入
x	• 在daxpy中是双精度浮点类型。 • 在saxpy中是单精度浮点类型。 • 在caxpy中是单精度复数类型。 • 在zaxpy中是双精度复数类型。	向量x，向量规模至少是(1+(n-1)*abs(incx))。	输入
incx	整型数	表示x向量增长步长。	输入
y	• 在daxpy中是双精度浮点类型。 • 在saxpy中是单精度浮点类型。 • 在caxpy中是单精度复数类型。 • 在zaxpy中是双精度复数类型。	向量y，向量规模至少是(1+(n-1)*abs(incy))。	输入/输出
incy	整型数	表示y向量增长步长。	输入

依赖

#include "kblas.h"

示例

C interface：

```
float alpha = 2.0;
int n = 5, incx = 1, incy = 1;
/**
 *    X:f 0.340188, -0.105617, 0.283099, 0.298440, 0.411647
 *    Y: -0.302449, -0.164777, 0.268230, -0.222225, 0.053970
```

 版权所有 © 华为技术有限公司

```
 */
float x[5] = {0.340188, -0.105617, 0.283099, 0.298440, 0.411647};
float y[5] = {-0.302449, -0.164777, 0.268230, -0.222225, 0.053970};

cblas_saxpy(n, alpha, x, incx, y, incy);
/**
 *    Output Y: 0.377927, -0.376011, 0.834428, 0.374655, 0.877264
 */
```

Fortran interface：

```
    REAL(4) :: ALPHA=2.0
    INTEGER :: N=5
    INTEGER :: INCX=1
    INTEGER :: INCY=1
    REAL(4) :: X(5)
    DATA X /0.340188, -0.105617, 0.283099, 0.298440, 0.411647/
    REAL(4) :: Y(5)
    DATA Y /-0.302449, -0.164777, 0.268230, -0.222225, 0.053970/
    CALL SAXPY(N, ALPHA, X, INCX, Y, INCY)

*     Output Y : 0.377927, -0.376011, 0.834428, 0.374655, 0.877264
```

6.4.8.4.1.4 ?copy

向量拷贝。

即$Y = X$。x是源向量，y是目标向量。

接口定义

C interface：

void cblas_scopy(const BLASINT n, const float *x, const BLASINT incx, float *y, const BLASINT incy);

void cblas_dcopy(const BLASINT n, const double *x, const BLASINT incx, double *y, const BLASINT incy);

void cblas_ccopy(const BLASINT n, const void *x, const BLASINT incx, void *y, const BLASINT incy);

void cblas_zcopy(const BLASINT n, const void *x, const BLASINT incx, void *y, const BLASINT incy);

Fortran interface：

CALL SCOPY(N, X, INCX, Y, INCY)

CALL DCOPY(N, X, INCX, Y, INCY)

CALL CCOPY(N, X, INCX, Y, INCY)

CALL ZCOPY(N, X, INCX, Y, INCY)

参数

参数名	类型	描述	输入/输出
n	整型数	表示向量x和y的元素个数。	输入

版权所有 © 华为技术有限公司

参数名	类型	描述	输入/输出
x	• 在scopy中是单精度浮点类型。 • 在dcopy中是双精度浮点类型。 • 在ccopy中是单精度复数类型。 • 在zcopy中是双精度复数类型。	向量x，向量规模至少是(1+(n-1)*abs(incx))。	输入
incx	整型数	表示x向量增长步长。	输入
y	• 在scopy中是单精度浮点类型。 • 在dcopy中是双精度浮点类型。 • 在ccopy中是单精度复数类型。 • 在zcopy中是双精度复数类型。	向量y，向量规模至少是(1+(n-1)*abs(incy))。	输出
incy	整型数	表示y向量增长步长。	输入

依赖

#include "kblas.h"

示例

C interface：

```
double y[5];
int n = 5, incx = 1, incy = 1;
/*
 *   x: 1, 2, 3, 4, 5
 */
double x[5] = {1, 2, 3, 4, 5};

cblas_dcopy(n, x, incx, y, incy);
/*
 * Output  y: 1, 2, 3, 4, 5
 */
```

Fortran interface：

```
    REAL(8) :: Y(5)
    INTEGER :: N=5
    INTEGER :: INCX=1
    INTEGER :: INCY=1
    REAL(8) :: X(5)
    DATA X /1, 2, 3, 4, 5/
    CALL DCOPY(N, X, INCX, Y, INCY)

*     Output Y : 1, 2, 3, 4, 5
```


版权所有 © 华为技术有限公司

6.4.8.4.1.5 ?dot

计算向量与向量的点积。

即$res = \sum_{i=1}^{n} x_i * y_i$。x和y是含有n个元素的向量。

接口定义

C interface：

float cblas_sdot(const BLASINT n, const float *x, const BLASINT incx, const float *y, const BLASINT incy);

double cblas_ddot(const BLASINT n, const double *x, const BLASINT incx, const double *y, const BLASINT incy);

Fortran interface：

RES = SDOT(N, X, INCX, Y, INCY)

RES = DDOT(N, X, INCX, Y, INCY)

参数

参数名	类型	描述	输入/输出
n	整型数	表示x向量的元素个数。	输入
x	• 在ddot中是双精度浮点类型。 • 在sdot中是单精度浮点类型。	向量x，向量规模至少是(1+(n-1)*abs(incx))。	输入
incx	整型数。	表示x向量增长步长。	输入
y	• 在ddot中是双精度浮点类型。 • 在sdot中是单精度浮点类型。	向量y，向量规模至少是(1+(n-1)*abs(incy))。	输入
incy	整型数	表示y向量增长步长。	输入

返回值

向量点积结果。

- sdot是单精度浮点类型。
- ddot是双精度浮点类型。

依赖

#include "kblas.h"

 版权所有 © 华为技术有限公司

示例

C interface：

```
float res;
int n = 5, incx = 1, incy = 1;
/**
 *   X: 0.340188, -0.105617, 0.283099, 0.298440, 0.411647
 *   Y: -0.302449, -0.164777, 0.268230, -0.222225, 0.053970
 */
float x[5] = {0.340188, -0.105617, 0.283099, 0.298440, 0.411647};
float y[5] = {-0.302449, -0.164777, 0.268230, -0.222225, 0.053970};

res = cblas_sdot(n, x, incx, y, incy);
/**
 *   Output:
 *     -0.053655
 */
```

Fortran interface：

```
    REAL(4) :: RES
    INTEGER :: N=5
    INTEGER :: INCX=1
    INTEGER :: INCY=1
    REAL(4) :: X(5)
    DATA X /0.340188, -0.105617, 0.283099, 0.298440, 0.411647/
    REAL(4) :: Y(5)
    DATA Y /-0.302449, -0.164777, 0.268230, -0.222225, 0.053970/
    RES=SDOT(N, X, INCX, Y, INCY)

*     Output :  -0.053655
```

6.4.8.4.1.6 ?dotc

计算共轭向量与另一向量的点积。

即$res = \sum_{i=1}^{n} conjg(x_i) * y_i$。x和y是长度为n的复数向量。

接口定义

C interface：

float _Complex cblas_cdotc(const BLASINT n, const void *x, const BLASINT incx, const void *y, const BLASINT incy);

void cblas_cdotc_sub(const BLASINT n, const void *x, const BLASINT incx, const void *y, const BLASINT incy, void *ret);

double _Complex cblas_zdotc(const BLASINT n, const void *x, const BLASINT incx, const void *y, const BLASINT incy);

void cblas_zdotc_sub(const BLASINT n, const void *x, const BLASINT incx, const void *y, const BLASINT incy, void *ret);

Fortran interface：

RES = CDOTC(N, CX, INCX, CY, INCY)

RES = ZDOTC(N, CX, INCX, CY, INCY)

 版权所有 © 华为技术有限公司

参数

参数名	类型	描述	输入/输出
n	整型数	表示x、y向量的元素个数。	输入
x	• 在cdotc中是单精度复数类型。 • 在zdotc中是双精度复数类型。	向量x，向量规模至少是(1+(n-1)*abs(incx))。	输入
incx	整型数。	表示x向量增长步长。	输入
y	• 在cdotc中是单精度复数类型。 • 在zdotc中是双精度复数类型。	向量y，向量规模至少是(1+(n-1)*abs(incy))。	输入
incy	整型数	表示y向量增长步长。	输入
ret	• 在cdotc中是单精度复数类型。 • 在zdotc中是双精度复数类型。	计算结果。	输出

依赖

#include "kblas.h"

示例

C interface：

```
int n = 5, incx = 1, incy = 1;
/**
 *   X:
 *     0.340188, -0.105617, 0.283099, 0.298440, 0.411647,
 *     -0.302449, -0.164777, 0.268230, -0.222225, 0.053970
 *   Y:
 *     -0.022603, 0.128871, -0.135216, 0.013401, 0.452230,
 *     0.416195, 0.135712, 0.217297, -0.358397, 0.106969
 */
float x[10] = {0.340188, -0.105617, 0.283099, 0.298440, 0.411647,
               -0.302449, -0.164777, 0.268230, -0.222225, 0.053970};
float y[10] = {-0.022603, 0.128871, -0.135216, 0.013401, 0.452230,
                0.416195, 0.135712, 0.217297, -0.358397, 0.106969};

float _Complex res = cblas_cdotc(n, x, incx, y, incy);
/**
 *   Output:  0.126042, 0.317067
 *
 */
```

Fortran interface：

```
  COMPLEX(4) :: RES
  INTEGER :: N=5
```

 版权所有 © 华为技术有限公司

```
      INTEGER :: INCX=1
      INTEGER :: INCY=1
      COMPLEX(4) :: X(5)
      DATA X/(0.340188, -0.105617), (0.283099, 0.298440),
     $       (0.411647, -0.302449), (-0.164777, 0.268230),
     $       (-0.222225, 0.053970)/
      COMPLEX(4) :: Y(5)
      DATA Y/(-0.022603, 0.128871), (-0.135216, 0.013401),
     $       (0.452230, 0.416195), (0.135712, 0.217297),
     $       (-0.358397, 0.106969)/
      EXTERNAL CDOTC
      RES=CDOTC(N,X,INCX,Y,INCY)

*     Output : (0.126043, 0.317066)
```

6.4.8.4.1.7 ?dotu

计算两个复数向量的点积。

接口定义

C interface：

float _Complex cblas_cdotu(const BLASINT n, const void *x, const BLASINT incx, const void *y, const BLASINT incy);

void cblas_cdotu_sub(const BLASINT n, const void *x, const BLASINT incx, const void *y, const BLASINT incy, void *ret);

double _Complex cblas_zdotu(const BLASINT n, const void *x, const BLASINT incx, const void *y, const BLASINT incy);

void cblas_zdotu_sub(const BLASINT n, const void *x, const BLASINT incx, const void *y, const BLASINT incy, void *ret);

Fortran interface：

RES = CDOTU(N, CX, INCX, CY, INCY)

RES = ZDOTU(N, CX, INCX, CY, INCY)

参数

参数名	类型	描述	输入/输出
n	整型数。	表示x、y向量的元素个数。	输入
x	• 在cdotu中是单精度复数类型。 • 在zdotu中是双精度复数类型。	向量x，向量规模至少是(1+(n-1)*abs(incX))。	输入
incx	整型数。	表示x中向量元素的增量。	输入
y	• 在cdotu中是单精度复数类型。 • 在zdotu中是双精度复数类型。	向量y，向量规模至少是(1+(n-1)*abs(incy))。	输入

版权所有 © 华为技术有限公司

参数名	类型	描述	输入/输出
incy	整型数	表示y向量增长步长。	输入
ret	• 在cdotc中是单精度复数类型。 • 在zdotc中是双精度复数类型。	计算结果。	输出

依赖

#include "kblas.h"

示例

C interface：

```
int n = 5, incx = 1, incy = 1;
/**
 *    X:
 *      0.340188, -0.105617, 0.283099, 0.298440, 0.411647,
 *      -0.302449, -0.164777, 0.268230, -0.222225, 0.053970
 *    Y:
 *      -0.022603, 0.128871, -0.135216, 0.013401, 0.452230,
 *      0.416195, 0.135712, 0.217297, -0.358397, 0.106969
 */
float x[10] = {0.340188, -0.105617, 0.283099, 0.298440, 0.411647,
               -0.302449, -0.164777, 0.268230, -0.222225, 0.053970};
float y[10] = {-0.022603, 0.128871, -0.135216, 0.013401, 0.452230,
                0.416195, 0.135712, 0.217297, -0.358397, 0.106969};

float _Complex res = cblas_cdotu(n, x, incx, y, incy);
/**
 *    Output:  0.268904 0.001699
 *
 */
```

Fortran interface：

```
    COMPLEX(4) :: RES
    INTEGER :: N=5
    INTEGER :: INCX=1
    INTEGER :: INCY=1
    COMPLEX(4) :: X(5)
    DATA CX/(0.340188, -0.105617), (0.283099, 0.298440),
   $        (0.411647, -0.302449), (-0.164777, 0.268230),
   $        (-0.222225, 0.053970)/
    COMPLEX(4) :: Y(5)
    DATA CY/(-0.022603, 0.128871), (-0.135216, 0.013401),
   $        (0.452230, 0.416195), (0.135712, 0.217297),
   $        (-0.358397, 0.106969)/
    EXTERNAL CDOTU
    RES=CDOTU(N, CX, INCX, CY, INCY)

*     Output : (0.268904 0.001699)
```

6.4.8.4.1.8 ?sdot

扩展精度点积，输入为单精度，以双精度计算，sdsdot输出单精度，dsdot输出双精度。

 版权所有 © 华为技术有限公司

接口定义

C interface：

float cblas_sdsdot(const BLASINT n, const float alpha, const float *x, const BLASINT incx, const float *y, const BLASINT incy);

double cblas_dsdot(const BLASINT n, const float *x, const BLASINT incx, const float *y, const BLASINT incy);

Fortran interface：

RES = SDSDOT(N, ALPHA, SX, INCX, SY, INCY)

RES = DSDOT(N, SX, INCX, SY, INCY)

参数

参数名	类型	描述	输入/输出
n	整型数	表示x、y向量的元素个数。	输入
alpha	在sdsdot中是单精度类型。	单精度标量，加到点积中返回。	输入
x	单精度复数类型	向量x，向量规模至少是(1+(n-1)*abs(incX))。	输入
incx	整型数	表示x中向量元素的增量。	输入
y	单精度复数类型	向量y，向量规模至少是(1+(n-1)*abs(incy))。	输入
incy	整型数	表示y向量增长步长。	输入

返回值

向量点积的结果。

- 在sdsdot中，返回值是单精度类型。
- 在dsdot中，返回值是双精度复数类型。

依赖

#include "kblas.h"

示例

C interface：

```
int n = 10, incx = 1, incy = 1;
/**
 *    X:
 *      0.340188, -0.105617, 0.283099, 0.298440, 0.411647,
 *      -0.302449, -0.164777, 0.268230, -0.222225, 0.053970
```

 版权所有 © 华为技术有限公司

```
 *    Y:
 *      -0.022603, 0.128871, -0.135216, 0.013401, 0.452230,
 *      0.416195, 0.135712, 0.217297, -0.358397, 0.106969
 */
float x[10] = {0.340188, -0.105617, 0.283099, 0.298440, 0.411647,
               -0.302449, -0.164777, 0.268230, -0.222225, 0.053970};
float y[10] = {-0.022603, 0.128871, -0.135216, 0.013401, 0.452230,
                0.416195, 0.135712, 0.217297, -0.358397, 0.106969};

double res = cblas_dsdot(n, x, incx, y, incy);
/**
 *    Output:  0.126042
 */
```

Fortran interface：

```
    REAL(8) :: RES
    INTEGER :: N=10
    INTEGER :: INCX=1
    INTEGER :: INCY=1
    REAL(4) :: X(10)
    DATA X /0.340188, -0.105617, 0.283099, 0.298440, 0.411647,
   $        -0.302449, -0.164777, 0.268230, -0.222225, 0.053970/
    REAL(4) :: Y(10)
    DATA Y /-0.022603, 0.128871, -0.135216, 0.013401, 0.452230,
   $        0.416195, 0.135712, 0.217297, -0.358397, 0.106969/
    EXTERNAL DSDOT
    RES=DSDOT(N,X,INCX,Y,INCY)

*    Output :  0.126042
```

6.4.8.4.1.9 i?amax

实数向量返回最大绝对值的索引值，复数向量返回实部绝对值和虚部绝对值之和的最大值索引值。

接口定义

C interface：

CBLAS_INDEX cblas_isamax(const BLASINT n, const float *x, const BLASINT incx);

CBLAS_INDEX cblas_idamax(const BLASINT n, const double *x, const BLASINT incx);

CBLAS_INDEX cblas_icamax(const BLASINT n, const void *x, const BLASINT incx);

CBLAS_INDEX cblas_izamax(const BLASINT n, const void *x, const BLASINT incx);

Fortran interface：

INDEX = ISAMAX(N, X, INCX)

INDEX = IDAMAX(N, X, INCX)

INDEX = ICAMAX(N, X, INCX)

INDEX = IZAMAX(N, X, INCX)

 版权所有 © 华为技术有限公司

参数

参数名	类型	描述	输入/输出
n	整型数	表示x向量的元素个数。	输入
x	• 在isamax中是双精度浮点类型。 • 在idamax中是单精度浮点类型。 • 在icamax中是单精度复数类型。 • 在izamax中是双精度复数类型。	向量x，向量规模至少是(1+(n-1)*abs(incX))。	输入
incx	整型数	表示x向量增长步长。	输入

返回值

实数向量返回最大绝对值的索引值，复数向量返回实部绝对值和虚部绝对值之和的最大值索引值。CBLAS_INDEX类型。

依赖

#include "kblas.h"

示例

C interface：

```
int n = 5, incx = 1;
int res;
/**
 *   X:  0.340188, -0.105617, 0.283099, 0.298440, 0.411647
 */
float x[5] = {0.340188, -0.105617, 0.283099, 0.298440, 0.411647};

res = cblas_isamax(n, x, incx);
/**
 * Output:  4
 */
```

Fortran interface：

```
    INTEGER :: N=5
    INTEGER :: INCX=1
    INTEGER :: INDEX
    REAL(4) :: X(5)
    DATA X/0.340188, -0.105617, 0.283099, 0.298440, 0.411647/
    EXTERNAL ISAMAX
    INDEX=ISAMAX(N, X, INCX)

*     Output : 5
```


版权所有 © 华为技术有限公司

说明

在C语言中，index是从0开始，在Fortran语言中，index是从1开始，因此在该Fortran用例中最大绝对值是第五个元素，所以会返回最大绝对值的索引值为5。

6.4.8.4.1.10 i?amin

实数向量返回最小绝对值的索引值，复数向量返回实部绝对值和虚部绝对值之和的最小值索引值。

接口定义

C interface：

CBLAS_INDEX cblas_isamin(const BLASINT n, const float *x, const BLASINT incx);

CBLAS_INDEX cblas_idamin(const BLASINT n, const double *x, const BLASINT incx);

CBLAS_INDEX cblas_icamin(const BLASINT n, const void *x, const BLASINT incx);

CBLAS_INDEX cblas_izamin(const BLASINT n, const void *x, const BLASINT incx);

Fortran interface：

INDEX = ISAMIN(N, X, INCX)

INDEX = IDAMIN(N, X, INCX)

INDEX = ICAMIN(N, X, INCX)

INDEX = IZAMIN(N, X, INCX)

参数

参数名	类型	描述	输入/输出
n	整型数	表示x向量的元素个数。	输入
x	• 在isamin中是双精度浮点类型。 • 在idamin中是单精度浮点类型。 • 在icamin中是单精度复数类型。 • 在izamin中是双精度复数类型。	向量x，向量规模至少是(1+(n-1)*abs(incX))。	输入
incx	整型数	表示x向量增长步长。	输入

返回值

实数向量返回最小绝对值的索引值，复数向量返回实部绝对值和虚部绝对值之和的最小值索引值。CBLAS_INDEX类型。

 版权所有 © 华为技术有限公司

依赖

#include "kblas.h"

示例

C interface：

```
int n = 5, incx = 1;
int res;
/**
 *   X:  0.340188, -0.105617, 0.283099, 0.298440, 0.411647
 */
float x[5] = {0.340188, -0.105617, 0.283099, 0.298440, 0.411647};

res = cblas_isamin(n, x, incx);
/**
 * Output:  1
 */
```

Fortran interface：

```
    INTEGER :: N=5
    INTEGER :: INCX=1
    INTEGER :: INDEX
    REAL(4) :: X(5)
    DATA X/0.340188, -0.105617, 0.283099, 0.298440, 0.411647/
    EXTERNAL ISAMIN
    INDEX=ISAMIN(N, X, INCX)

*     Output : 1
```

6.4.8.4.1.11 ?nrm2

计算向量的欧几里得范数。

即$res = ||x||$。x是含有n个元素的向量，||x||相当于$\sqrt{|x_1|^2 + |x_2|^2 + ... + |x_n|^2}$。

接口定义

C interface：

float cblas_snrm2(const BLASINT N, const float *X, const BLASINT incX);

double cblas_dnrm2(const BLASINT N, const double *X, const BLASINT incX);

float cblas_scnrm2(const BLASINT N, const float *X, const BLASINT incX);

double cblas_dznrm2(const BLASINT N, const double *X, const BLASINT incX);

Fortran interface：

RES = SNRM2(N, X, INCX)

RES = DNRM2(N, X, INCX)

RES = SCNRM2(N, X, INCX)

RES = DZNRM2(N, X, INCX)

 版权所有 © 华为技术有限公司

参数

参数名	类型	描述	输入/输出
N	整型数	表示X向量的元素个数。	输入
X	• 在snrm2中是单精度浮点类型。 • 在dnrm2中是双精度浮点类型。 • 在scnrm2中是单精度复数类型。 • 在dznrm2中是双精度复数类型。	向量X，向量规模至少是(1+(n-1)*abs(incX))。	输入/输出
incX	整型数	表示X向量增长步长。	输入

返回值

向量的范数。float或双精度浮点类型。

依赖

#include "kblas.h"

示例

C interface：

```
int n = 5, incx = 1;
float res;
/**
 *    X: 0.340188, -0.105617, 0.283099, 0.298440, 0.411647
 */
float x[5] = {0.340188, -0.105617, 0.283099, 0.298440, 0.411647};

res = cblas_snrm2(n, x, incx);
/**
 *    Output: 0.682310
 */
```

Fortran interface：

```
    INTEGER :: N=5
    INTEGER :: INCX=1
    REAL(4) :: RES
    REAL(4) :: X(5)
    DATA X/0.340188, -0.105617, 0.283099, 0.298440, 0.411647/
    EXTERNAL SNRM2
    RES=SNRM2(N, X, INCX)

*     Output : 0.682310
```

6.4.8.4.1.12 ?rot

基于平面上的点旋转。

 版权所有 © 华为技术有限公司

即：

$$x_i = c * x_i + s y_i$$

$$y_i = c * y_i - s x_i$$

x和y是含有n个元素的向量，c是旋转角的余弦，s是旋转角的正弦。

接口定义

C interface：

void cblas_srot(const BLASINT N, float *X, const BLASINT incX, float *Y, const BLASINT incY, const float c, const float s);

void cblas_drot(const BLASINT N, double *X, const BLASINT incX, double *Y, const BLASINT incY, const double c, const double s);

void cblas_csrot(const BLASINT N, float *X, const BLASINT incX, float *Y, const BLASINT incY, const float c, const float s);

void cblas_zdrot(const BLASINT N, double *X, const BLASINT incX, double *Y, const BLASINT incY, const double c, const double s);

Fortran interface：

CALL SROT(N, SX, INCX, SY, INCY, SC, SS)

CALL DROT(N, SX, INCX, SY, INCY, SC, SS)

CALL CSROT(N, SX, INCX, SY, INCY, SC, SS)

CALL ZDROT(N, SX, INCX, SY, INCY, SC, SS)

参数

参数名	类型	描述	输入/输出
N	整型数	表示X向量的元素个数。	输入
X	• 在srot中是单精度浮点类型。 • 在drot中是双精度浮点类型。 • 在csrot中是单精度复数类型。 • 在zdrot中是双精度复数类型。	向量X，向量规模至少是(1+(n-1)*abs(incx))。	输入/输出
incX	整型数	表示X向量增长步长。	输入

版权所有 © 华为技术有限公司

参数名	类型	描述	输入/输出
Y	• 在srot中是单精度浮点类型。 • 在drot中是双精度浮点类型。 • 在csrot中是单精度复数类型。 • 在zdrot中是双精度复数类型。	向量Y，向量规模至少是(1+(n-1)*abs(incy))。	输入/输出
incY	整型数	表示Y向量增长步长。	输入
c	• 在srot，csrot中是单精度浮点类型。 • 在drot，zdrot中是双精度浮点类型。	旋转角的余弦。	输入
s	• 在srot，csrot中是单精度浮点类型。 • 在drot，zdrot中是双精度浮点类型。	旋转角的正弦。	输入

依赖

#include "kblas.h"

示例

C interface：

```
int n = 5, incx = 1, incy=1;
float c = 1, s = 1;
/*
 * C = 1, S = 1
 * Input X: 0.340188, -0.105617, 0.283099, 0.298440, 0.411647
 * Input Y: -0.302449, -0.164777, 0.268230, -0.222225, 0.053970
 **/
float x[5] = {0.340188, -0.105617, 0.283099, 0.298440, 0.411647};
float y[5] = {-0.302449, -0.164777, 0.268230, -0.222225, 0.053970};

cblas_srot(n, x, incx, y, incy, c, s);
/*
 * Output X: 0.037739, -0.270394, 0.551329, 0.076215, 0.465617
 * Output Y: -0.642636, -0.059160, -0.014870, -0.520665, -0.357677
 **/
```

Fortran interface：

```
  INTEGER :: N=5
  INTEGER :: INCX=1
  INTEGER :: INCY=1
  REAL(4) :: C=1
  REAL(4) :: S=1
  REAL(4) :: X(5)
```

 版权所有 © 华为技术有限公司

```
      DATA X /0.340188, -0.105617, 0.283099, 0.298440, 0.411647/
      REAL(4) :: Y(5)
      DATA Y /-0.302449, -0.164777, 0.268230, -0.222225, 0.053970/
      EXTERNAL SROT
      CALL SROT(N, X, INCX, Y, INCY, C, S)

*     Output X: 0.037739, -0.270394, 0.551329, 0.076215, 0.465617
*     Output Y: -0.642636, -0.059160, -0.014870, -0.520665, -0.357677
```

6.4.8.4.1.13 ?rotg

构造旋转平面。

接口定义

C interface：

void cblas_srotg(float *a, float *b, float *c, float *s);

void cblas_drotg(double *a, double *b, double *c, double *s);

Fortran interface：

CALL SROTG(SA, SB, SC, SS)

CALL DROTG(SA, SB, SC, SS)

参数

参数名	类型	描述	输入/输出
a	• 在srotg中是单精度浮点类型。 • 在drotg中是双精度浮点类型。	笛卡尔坐标系x坐标。	输入/输出
b	• 在srotg中是单精度浮点类型。 • 在drotg中是双精度浮点类型。	笛卡尔坐标系y坐标。	输入/输出
c	• 在srotg中是单精度浮点类型。 • 在drotg中是双精度浮点类型。	吉文斯旋转参数c。	输出
s	• 在srotg中是单精度浮点类型。 • 在drotg中是双精度浮点类型。	吉文斯旋转参数s。	输出

依赖

#include "kblas.h"

 版权所有 © 华为技术有限公司

示例

C interface：

```
float a = 6.0;
float b = -8.0;
float c, s;

cblas_srotg(&a, &b, &c, &s);
/*
 * Output a:-10.000001, b:-1.666667, c:-0.600000, s:0.800000
 */
```

Fortran interface：

```
    REAL(4) :: A=6.0
    REAL(4) :: B=-8.0
    REAL(4) :: C
    REAL(4) :: S
    EXTERNAL SROTG
    CALL SROTG(A, B, C, S)

*     Output A:-10.000001, B:-1.666667, C:-0.600000, S:0.800000
```

6.4.8.4.1.14 ?rotm

修改旋转平面。

即：

$$x_i = h_{11}x_i + h_{12}y_i$$

$$y_i = h_{21}x_i + h_{22}y_i$$

x和y是含有n个元素的向量，h是转换矩阵元素。

接口定义

C interface：

void cblas_srotm(const BLASINT N, float *X, const BLASINT incX, float *Y, const BLASINT incY, const float *H);

void cblas_drotm(const BLASINT N, double *X, const BLASINT incX, double *Y, const BLASINT incY, const double *H);

Fortran interface：

CALL SROTM(N, SX, INCX, SY, INCY, DTEMP)

CALL DROTM(N, SX, INCX, SY, INCY, DTEMP)

参数

参数名	类型	描述	输入/输出
N	整型数	表示X向量的元素个数。	输入

版权所有 © 华为技术有限公司

参数名	类型	描述	输入/输出
X	• 在srotm中是单精度浮点类型。 • 在drotm中是双精度浮点类型。	向量X，向量规模至少是(1+(n-1)*abs(incx))。	输入/输出
incX	整型数	表示X向量增长步长。	输入
Y	• 在srotm中是单精度浮点类型。 • 在drotm中是双精度浮点类型。	向量Y，向量规模至少是(1+(n-1)*abs(incy))。	输入/输出
incY	整型数	表示Y向量增长步长。	输入
H	• 在srotm中是单精度浮点类型。 • 在drotm中是双精度浮点类型。	转换矩阵H。	输入

依赖

#include "kblas.h"

示例

C interface：

```
double d1 = 10.0, d2 = 2.0;
double x1 = 2.0, x2 = 1.0;
double p[5];
int n = 5, incx = 1, incy = 1;
double x[5]={1.0, 2.0, 1.0, 3.0, 1.0};
double y[5]={3.0, 1.0, 1.0, 5.0, 1.0};

cblas_drotmg(&d1, &d2, &x1, x2, p);
cblas_drotm(n, x, incx, y, incy, p);
/*
 * Output x:  1.300000,  2.100000, 1.100000,  3.500000, 1.100000
 * Output y:  2.500000,  0.000000, 0.500000,  3.500000, 0.500000
 */
```

Fortran interface：

```
  REAL(8) :: D1=10.0
  REAL(8) :: D2=2.0
  REAL(8) :: X1=2.0
  REAL(8) :: X2=1.0
  REAL(8) :: P(5)
  INTEGER :: N=5
  INTEGER :: INCX=1
  INTEGER :: INCY=1
  REAL(8) :: X(5)
  DATA X/1.0, 2.0, 1.0, 3.0, 1.0/
  REAL(8) :: Y(5)
  DATA Y/3.0, 1.0, 1.0, 5.0, 1.0/
```

 版权所有 © 华为技术有限公司

```
      EXTERNAL DROTMG
      CALL DROTMG(D1, D2, X1, X2, P)
      EXTERNAL DROTM
      CALL DROTM(N, X, INCX, Y, INCY, P)

*     Output X:  1.300000,  2.100000, 1.100000,  3.500000, 1.100000
*     Output Y:  2.500000,  0.000000, 0.500000,  3.500000, 0.500000
```

6.4.8.4.1.15 ?rotmg

构造修改的转换平面。

接口定义

C interface：

void cblas_srotmg(float *d1, float *d2, float *b1, const float b2, float *H);

void cblas_drotmg(double *d1, double *d2, double *b1, const double b2, double *H);

Fortran interface：

CALL SROTMG(D1, D2, X1, X2, PARAM)

CALL DROTMG(D1, D2, X1, X2, PARAM)

参数

参数名	类型	描述	输入/输出
d1	• 在srotmg中是单精度浮点类型。 • 在drotmg中是双精度浮点类型。	d1取值	输入/输出
d2	• 在srotmg中是单精度浮点类型。 • 在drotmg中是双精度浮点类型。	d2取值	输入/输出
b1	• 在srotmg中是单精度浮点类型。 • 在drotmg中是双精度浮点类型。	b1取值	输入/输出
b2	• 在srotmg中是单精度浮点类型。 • 在drotmg中是双精度浮点类型。	b2取值	输入/输出

版权所有 © 华为技术有限公司

参数名	类型	描述	输入/输出
H	• 在srotmg中是单精度浮点类型。 • 在drotmg中是双精度浮点类型。	转换矩阵H	输出

依赖

#include "kblas.h"

示例

C interface：

```
double d1 = 10.0, d2 = 1.0, x1 = -2.0, x2 = 4.0;
double p[5];

cblas_drotmg(&d1, &d2, &x1, x2, p);
/**
 * d1:7.142857, d2:0.714286, x1:-2.800000, x2:4.000000
 * param:0.000000, 0.000000, 2.000000, -0.200000, 0.000000
 */
```

Fortran interface：

```
    REAL(8) :: D1=10.0
    REAL(8) :: D2=1.0
    REAL(8) :: X1=-2.0
    REAL(8) :: X2=4.0
    REAL(8) :: PARAM(5)
    EXTERNAL DROTMG
    CALL DROTMG(D1, D2, X1, X2, P)

*     D1:7.142857, D2:0.714286, X1:-2.800000, X2:4.000000
*     PARAM:0.000000, 0.000000, 2.000000, -0.200000, 0.000000
```

6.4.8.4.1.16 ?scal

向量缩放。

即$x = \mathrm{alpha} * x$。alpha是乘法系数，x是含有n个元素的向量。

接口定义

C interface：

void cblas_dscal(const BLASINT N, const double alpha, double *X, const BLASINT incX);

void cblas_sscal(const BLASINT N, const float alpha, float *X, const BLASINT incX);

void cblas_cscal(const BLASINT N, const void *alpha, void *X, const BLASINT incX);

void cblas_zscal(const BLASINT N, const void *alpha, void *X, const BLASINT incX);

void cblas_csscal(const BLASINT N, const float alpha, void *X, const BLASINT incX);

 版权所有 © 华为技术有限公司

void cblas_zdscal(const BLASINT N, const double alpha, void *X, const BLASINT incX);

Fortran interface：

CALL SSCAL(N, ALPHA, X, INCX)

CALL DSCAL(N, ALPHA, X, INCX)

CALL CSCAL(N, ALPHA, X, INCX)

CALL ZSCAL(N, ALPHA, X, INCX)

CALL CSSCAL(N, ALPHA, X, INCX)

CALL ZDSCAL(N, ALPHA, X, INCX)

参数

参数名	类型	描述	输入/输出
N	整型数	表示X向量的元素个数。	输入
alpha	• 在dscal中是双精度浮点类型。 • 在sscal中是单精度浮点类型。 • 在cscal中是单精度复数类型。 • 在zscal中是双精度复数类型。 • 在csscal中是单精度浮点类型。 • 在zdscal中是双精度浮点类型。	表示系数。	输入
X	• 在dscal中是双精度浮点类型。 • 在sscal中是单精度浮点类型。 • 在cscal中是单精度复数类型。 • 在zscal中是双精度复数类型。 • 在csscal中是单精度复数类型。 • 在zdscal中是双精度复数类型。	向量X，向量规模至少是(1+(n-1)*abs(incX))。	输入/输出
incX	整型数	表示X向量增长步长。	输入

版权所有 © 华为技术有限公司

依赖

#include "kblas.h"

示例

C interface：

```
float alpha = 2.0;
int n = 5, incx = 1;
/**
 * X: 0.340188, -0.105617, 0.283099, 0.298440, 0.411647
 */
float x[5] = {0.340188, -0.105617, 0.283099, 0.298440, 0.411647};

cblas_sscal(n, alpha, x, incx);
/**
 * Output X: 0.680375, -0.211234, 0.566198, 0.596880, 0.823295
 */
```

Fortran interface：

```
    REAL(4) :: ALPHA=2.0
    INTEGER :: N=5
    INTEGER :: INCX=1
    REAL(4) :: X(5)
    DATA X /0.340188, -0.105617, 0.283099, 0.298440, 0.411647/
    EXTERNAL SSCAL
    CALL SSCAL(N, ALPHA, X, INCX)

*   Output X: 0.680375, -0.211234, 0.566198, 0.596880, 0.823295
```

6.4.8.4.1.17 ?swap

两个向量元素交换。

接口定义

C interface：

void cblas_sswap(const BLASINT n, float *x, const BLASINT incx, float *y, const BLASINT incy);

void cblas_dswap(const BLASINT n, double *x, const BLASINT incx, double *y, const BLASINT incy);

void cblas_cswap(const BLASINT n, void *x, const BLASINT incx, void *y, const BLASINT incy);

void cblas_zswap(const BLASINT n, void *x, const BLASINT incx, void *y, const BLASINT incy);

Fortran interface：

CALL SSWAP(N, SX, INCX, SY, INCY)

CALL DSWAP(N, SX, INCX, SY, INCY)

CALL CSWAP(N, SX, INCX, SY, INCY)

CALL ZSWAP(N, SX, INCX, SY, INCY)

 版权所有 © 华为技术有限公司

参数

参数名	类型	描述	输入/输出
n	整型数	表示x、y向量的元素个数。	输入
x	• 在sswap中是单精度浮点类型。 • 在dswap中是双精度浮点类型。 • 在cswap中是单精度复数类型。 • 在zswap中是双精度复数类型。	向量x，向量规模至少是(1+(n-1)*abs(incx))。	输入/输出
incx	整型数	表示x向量增长步长。	输入
y	• 在sswap中是单精度浮点类型。 • 在dswap中是双精度浮点类型。 • 在cswap中是单精度复数类型。 • 在zswap中是双精度复数类型。	向量y，向量规模至少是(1+(n-1)*abs(incy))。	输入/输出
incy	整型数	表示y中向量元素增长步长。	输入

依赖

#include "kblas.h"

示例

C interface：

```
int n = 5, incx = 1, incy = 1;
/*
 *  Input X:  0.340188, -0.105617, 0.283099, 0.298440, 0.411647
 *  Input Y:  -0.302449, -0.164777, 0.268230, -0.222225, 0.053970
 * */
float x[5] = {0.340188, -0.105617, 0.283099, 0.298440, 0.411647};
float y[5] = {-0.302449, -0.164777, 0.268230, -0.222225, 0.053970};

cblas_sswap(n, x, incx, y, incy);
/*
 *  Output X: -0.302449, -0.164777, 0.268230, -0.222225, 0.053970
 *  Output Y: 0.340188, -0.105617, 0.283099, 0.298440, 0.411647
 * */
```

Fortran interface：

```
  INTEGER :: N=5
  INTEGER :: INCX=1
```

 版权所有 © 华为技术有限公司

```
    INTEGER :: INCY=1
    REAL(4) :: X(5)
    DATA X /0.340188, -0.105617, 0.283099, 0.298440, 0.411647/
    REAL(4) :: Y(5)
    DATA Y /-0.302449, -0.164777, 0.268230, -0.222225, 0.053970/
    EXTERNAL SSWAP
    CALL SSWAP(N, X, INCX, Y, INCY)

*   Output X: -0.302449, -0.164777, 0.268230, -0.222225, 0.053970
*   Output Y: 0.340188, -0.105617, 0.283099, 0.298440, 0.411647
```

6.4.8.4.2 KML_BLAS Level 2 函数

6.4.8.4.2.1 函数说明

对KML_BLAS Level2所有函数族进行描述。

函数族	数据类型	描述
?gbmv	s，d，c，z	向量与带状矩阵乘积。
?gemv	s，d，c，z	向量与矩阵乘积。
?ger	s，d	一般矩阵秩1更新。
?gerc	c，z	一般复数矩阵秩1更新。
?geru	c，z	一般共轭矩阵秩1更新。
?hbmv	c，z	向量与埃尔米特带状矩阵乘积。
?hemv	c，z	向量与埃尔米特矩阵乘积。
?her	c，z	复数埃尔米特矩阵秩1更新。
?her2	c，z	复数埃尔米特矩阵秩2更新。
?hpmv	c，z	向量与三角展开的埃尔米特矩阵乘积。
?hpr	c，z	三角展开的埃尔米特矩阵秩1更新。
?hpr2	c，z	三角展开的埃尔米特矩阵秩2更新。
?sbmv	s，d	向量与对称带状矩阵乘积。
?spmv	s，d	向量与压缩的对称矩阵乘积。
?spr	s，d	对称展开矩阵秩1更新。
?spr2	s，d	对称展开矩阵秩2更新。
?symv	s，d	向量与对称矩阵乘积。
?syr	s，d	对称矩阵秩1更新。
?syr2	s，d	对称矩阵秩2更新。
?tbmv	s，d，c，z	向量与三角带状矩阵乘积。
?tbsv	s，d，c，z	线性方程组求解，系数矩阵为三角带状矩阵。

版权所有 © 华为技术有限公司

函数族	数据类型	描述
?tpmv	s，d，c，z	向量与压缩的三角矩阵乘积。
?tpsv	s，d，c，z	线性方程组求解，系数矩阵是压缩的三角矩阵。
?trmv	s，d，c，z	向量与三角矩阵乘积。
?trsv	s，d，c，z	线性方程组求解，系数矩阵是三角矩阵。

6.4.8.4.2.2 ?gbmv

向量与带状矩阵乘积。

即：$y=alpha*op(A)*x+beta*y$

对于Fortran接口另有：$y=alpha*op(A)*conj(x)+beta*y$

alpha和beta是乘法系数，x和y是向量，A是m*n的带状矩阵。

接口定义

C interface：

void cblas_sgbmv(const enum CBLAS_ORDER order, const enum CBLAS_TRANSPOSE TransA, const BLASINT M, const BLASINT N, const BLASINT KL, const BLASINT KU, const float alpha, const float *A, const BLASINT lda, const float *X, const BLASINT incX, const float beta, float *Y, const BLASINT incY);

void cblas_dgbmv(const enum CBLAS_ORDER order, const enum CBLAS_TRANSPOSE TransA, const BLASINT M, const BLASINT N, const BLASINT KL, const BLASINT KU, const double alpha, const double *A, const BLASINT lda, const double *X, const BLASINT incX, const double beta, double *Y, const BLASINT incY);

void cblas_cgbmv(const enum CBLAS_ORDER order, const enum CBLAS_TRANSPOSE TransA, const BLASINT M, const BLASINT N, const BLASINT KL, const BLASINT KU, const void *alpha, const void *A, const BLASINT lda, const void *X, const BLASINT incX, const void *beta, void *Y, const BLASINT incY);

void cblas_zgbmv(const enum CBLAS_ORDER order, const enum CBLAS_TRANSPOSE TransA, const BLASINT M, const BLASINT N, const BLASINT KL, const BLASINT KU, const void *alpha, const void *A, const BLASINT lda, const void *X, const BLASINT incX, const void *beta, void *Y, const BLASINT incY);

Fortran interface：

CALL SGBMV(TRANS, M, N, KL, KU, ALPHA, A, LDA, X, INCX, BETA, Y, INCY)

CALL DGBMV(TRANS, M, N, KL, KU, ALPHA, A, LDA, X, INCX, BETA, Y, INCY)

CALL CGBMV(TRANS, M, N, KL, KU, ALPHA, A, LDA, X, INCX, BETA, Y, INCY)

 版权所有 © 华为技术有限公司

CALL ZGBMV(TRANS, M, N, KL, KU, ALPHA, A, LDA, X, INCX, BETA, Y, INCY)

参数

参数名	类型	描述	输入/输出
order	枚举类型CBLAS_ORDER	表示矩阵是行主序或列主序。	输入
TransA	枚举类型CBLAS_TRANSPOSE	矩阵A为常规矩阵，转置矩阵或共轭矩阵。 ● 如果TransA = CblasNoTrans，$op(A) = A$。 ● 如果TransA = CblasTrans，$op(A) = A'$。 ● 如果TransA = CblasConjTrans，$op(A)=conjg(A)$。 ● 如果TransA = CblasConjTrans，$op(A) = conjg(A')$。 对于Fortran接口另有： ● 如果TransA = N，op(A) = A。 ● 如果TransA = T，op(A) = A'。 ● 如果TransA = R，op(A) = conjg(A)。 ● 如果TransA = C，op(A) = conjg(A')。	输入
M	整型数	表示矩阵A的行数，要求M大于或等于零。	输入
N	整型数	表示矩阵A的列数，要求N大于或等于零。	输入
KL	整型数	表示A的次对角阶数，要求KL大于或等于零。	输入
KU	整型数	表示A的超对角阶数，要求KU大于或等于零。	输入

 版权所有 © 华为技术有限公司

参数名	类型	描述	输入/输出
alpha	• 在sgbmv中是单精度浮点类型。 • 在dgbmv中是双精度浮点类型。 • 在cgbmv中是单精度复数类型。 • 在zgbmv中是双精度复数类型。	表示乘法系数。	输入
A	• 在sgbmv中是单精度浮点类型。 • 在dgbmv中是双精度浮点类型。 • 在cgbmv中是单精度复数类型。 • 在zgbmv中是双精度复数类型。	带状矩阵，矩阵规模是lda*N。	输入
lda	整型数	矩阵A中主维度长度，要求lda大于或等于(KL+KU+1)。	输入
X	• 在sgbmv中是单精度浮点类型。 • 在dgbmv中是双精度浮点类型。 • 在cgbmv中是单精度复数类型。 • 在zgbmv中是双精度复数类型。	向量X。 如果TransA=CblasNoTrans，则向量规模至少是(1+(N-1)*abs(incX))。 否则，向量规模至少是(1+(M-1)*abs(incX))。	输入
incX	整型数	表示X中向量增长步长，不能为零。	输入
beta	• 在sgbmv中是单精度浮点类型。 • 在dgbmv中是双精度浮点类型。 • 在cgbmv中是单精度复数类型。 • 在zgbmv中是双精度复数类型。	乘法系数。	输入

版权所有 © 华为技术有限公司

参数名	类型	描述	输入/输出
Y	• 在sgbmv中是单精度浮点类型。 • 在dgbmv中是双精度浮点类型。 • 在cgbmv中是单精度复数类型。 • 在zgbmv中是双精度复数类型。	向量Y。 如果TransA=CblasNoTrans，则向量规模至少是(1+(M-1)*abs(incY))。 否则，向量规模至少是(1+(N-1)*abs(incY))。	输入/输出
incY	整型数	表示Y中向量增长步长，不能为零。	输入

依赖

#include "kblas.h"

示例

C interface：

```
int m = 5, n = 4;
 float alpha = 2.0;
 /**
  *       |  .    .   2.0  2.0 |
  *       |  .   1.0  2.0  3.0 |
  *       | 1.0  2.0  2.0  4.0 |
  *  A  =  | 1.0  3.0  2.0  5.0 |
  *       | 1.0  4.0  2.0   .  |
  *       | 1.0  5.0   .    .  |
  *       |  .    .    .    .  |
  *       |  .    .    .    .  |
  */
 float a[32] = {0, 0, 1.0, 1.0, 1.0, 1.0, 0, 0,
                0, 1.0, 2.0, 3.0, 4.0, 5.0, 0, 0,
                2.0, 2.0, 2.0, 2.0, 2.0, 0, 0, 0,
                2.0, 3.0, 4.0, 5.0, 0, 0, 0, 0};
 float x[4] = {2.0, 2.0, 3.0, 4.0};
 float y[5] = {1.0, 1.0, 13.0, 4.0, 5.0};

 cblas_sgbmv(CblasColMajor,CblasNoTrans, m, n, 3, 2, alpha, a, 8, x, 1, 10.0, y, 1);
 /**
  *  Output y: 30.0, 50.0, 182.0, 104.0, 122.0
  */
```

Fortran interface：

```
  INTEGER :: M=5
  INTEGER :: N=4
  REAL(4) :: ALPHA=2.0
  REAL(4) A(8, 4)
  DATA A /0, 0, 1, 1, 1, 1, 0, 0, 0, 1, 2, 3, 4, 5, 0, 0,
         2, 2, 2, 2, 2, 0, 0, 0, 2, 3, 4, 5, 0, 0, 0, 0/
  REAL(4) X(4)
  DATA X /2, 2, 3, 4/
  REAL(4) Y(5)
  DATA Y /1, 1, 13, 4, 5/
```

 版权所有 © 华为技术有限公司

```
      CALL SGBMV('N', M, N, 3, 2, ALPHA, A, 8, X, 1, 10, Y, 1)
*     Output y: 20.0      40.0      52.0      64.0      72.0
```

6.4.8.4.2.3 ?gemv

向量与矩阵乘积。

即：$y=alpha*op(A)*x+beta*y$

对于Fortran接口另有：$y=alpha*op(A)*conj(x)+beta*y$

alpha和beta是乘法系数，x和y是向量，A是m*n的一般矩阵。

接口定义

C interface：

void cblas_sgemv(const enum CBLAS_ORDER order, const enum CBLAS_TRANSPOSE trans, const BLASINT m, const BLASINT n, const float alpha, const float *a, const BLASINT lda, const float *x, const BLASINT incx, const float beta, float *y, const BLASINT incy);

void cblas_dgemv(const enum CBLAS_ORDER order, const enum CBLAS_TRANSPOSE trans, const BLASINT m, const BLASINT n, const double alpha, const double *a, const BLASINT lda, const double *x, const BLASINT incx, const double beta, double *y, const BLASINT incy);

void cblas_cgemv(const enum CBLAS_ORDER order, const enum CBLAS_TRANSPOSE trans, const BLASINT m, const BLASINT n, const void *alpha, const void *a, const BLASINT lda, const void *x, const BLASINT incx, const void *beta, void *y, const BLASINT incy);

void cblas_zgemv(const enum CBLAS_ORDER order, const enum CBLAS_TRANSPOSE trans, const BLASINT m, const BLASINT n, const void *alpha, const void *a, const BLASINT lda, const void *x, const BLASINT incx, const void *beta, void *y, const BLASINT incy);

Fortran interface：

CALL SGEMV(TRANS, M, N, ALPHA, A, LDA, X, INCX, BETA, Y, INCY)

CALL DGEMV(TRANS, M, N, ALPHA, A, LDA, X, INCX, BETA, Y, INCY)

CALL CGEMV(TRANS, M, N, ALPHA, A, LDA, X, INCX, BETA, Y, INCY)

CALL ZGEMV(TRANS, M, N, ALPHA, A, LDA, X, INCX, BETA, Y, INCY)

参数

参数名	类型	描述	输入/输出
order	枚举类型CBLAS_ORDER	表示矩阵是行主序或列主序。	输入

版权所有 © 华为技术有限公司

参数名	类型	描述	输入/输出
trans	枚举类型CBLAS_TRANSPOSE	表示矩阵A的操作。 ● 如果TransA = CblasNoTrans，$op(A) = A$。 ● 如果TransA = CblasTrans，$op(A) = A'$。 ● 如果TransA = CblasConjTrans，$op(A)=conjg(A)$。 ● 如果TransA = CblasConjTrans，$op(A) = conjg(A')$。 对于Fortran接口另有： ● 如果TransA = N，op(A) = A。 ● 如果TransA = T，op(A) = A'。 ● 如果TransA = R，op(A) = conjg(A)。 ● 如果TransA = C，op(A) = conjg(A')。	输入
m	整型数	表示矩阵A的行数，要求m大于或等于零。	输入
n	整型数	表示矩阵A的列数，要求n大于或等于零。	输入
alpha	● 在sgemv中是单精度浮点类型。 ● 在dgemv中是双精度浮点类型。 ● 在cgemv中是单精度复数类型。 ● 在zgemv中是双精度复数类型。	表示乘法系数。	输入

版权所有 © 华为技术有限公司

参数名	类型	描述	输入/输出
a	• 在sgemv中是单精度浮点类型。 • 在dgemv中是双精度浮点类型。 • 在cgemv中是单精度复数类型。 • 在zgemv中是双精度复数类型。	矩阵，矩阵规模是lda*n。	输入
lda	整型数	矩阵A中主维度长度，如果矩阵A为列存，要求lda大于或等于max(1, m)，否则要求大于或等于max(1, n)。	输入
x	• 在sgemv中是单精度浮点类型。 • 在dgemv中是双精度浮点类型。 • 在cgemv中是单精度复数类型。 • 在zgemv中是双精度复数类型。	向量x。 如果trans=CblasNoTrans，则向量规模至少是(1+(n-1)*abs(incx))。 否则，向量规模至少是(1+(m-1)*abs(incy))。	输入
incx	整型数	表示x中向量增长步长，不能为零。	输入
beta	• 在sgemv中是单精度浮点类型。 • 在dgemv中是双精度浮点类型。 • 在cgemv中是单精度复数类型。 • 在zgemv中是双精度复数类型。	乘法系数。	输入
y	• 在sgemv中是单精度浮点类型。 • 在dgemv中是双精度浮点类型。 • 在cgemv中是单精度复数类型。 • 在zgemv中是双精度复数类型。	向量y。 如果trans=CblasNoTrans，则向量规模至少是(1+(m-1)*abs(incy))。 否则，向量规模至少是(1+(n-1)*abs(incy))。	输入/输出
incy	整型数	表示y中向量增长步长，不能为零。	输入

版权所有 © 华为技术有限公司

依赖

#include "kblas.h"

示例

C interface：

```
int m = 3, n = 3, lda = 3, incx = 1, incy = 1;
float alpha = 1.0, beta = 2.0;
/**
 *  A:
 *     0.340188, 0.298440, -0.164777
 *     -0.105617, 0.411647, 0.268230
 *     0.283099, -0.302449, -0.222225
 *  X:
 *     0.053970, -0.022603, 0.128871
 *  Y:
 *     -0.135216, 0.013401, 0.452230
 */
float a[9] = {0.340188, 0.298440, -0.164777,
              -0.105617, 0.411647, 0.268230,
              0.283099, -0.302449, -0.222225};
float x[3] = {0.053970, -0.022603, 0.128871};
float y[3] = {-0.135216, 0.013401, 0.452230};

cblas_sgemv(CblasColMajor,CblasNoTrans, m, n, alpha, a, lda, x, incx, beta, y, incy);
/**
 *  Output Y: -0.213202, -0.005373, 0.860866
 *
 */
```

Fortran interface：

```
      INTEGER :: M=3
      INTEGER :: N=3
      INTEGER :: LDA=3
      INTEGER :: INCX=1
      INTEGER :: INCY=1
      REAL(4) :: ALPHA=1.0
      REAL(4) :: BETA=2.0
      REAL(4) :: A(3, 3)
      DATA A/0.340188, 0.298440, -0.164777,
     $       -0.105617, 0.411647, 0.268230,
     $       0.283099, -0.302449, -0.222225/
      REAL(4) :: X(3)
      DATA X/0.053970, -0.022603, 0.128871/
      REAL(4) :: Y(3)
      DATA Y/-0.135216, 0.013401, 0.452230/
      EXTERNAL SGEMV
      CALL SGEMV('N', M, N, ALPHA, A, LDA, X, INCX, BETA, Y, INCY)

*     Output Y: -0.213201538, -5.37255174E-03, 0.860865831
```

6.4.8.4.2.4 ?ger

一般矩阵秩1更新。

即：$A = A + \text{alpha} * x * y'$。alpha是乘法系数，A是m*n的一般矩阵，x是含有m个元素的向量，y是含有n个元素的向量。

版权所有 © 华为技术有限公司

接口定义

C interface：

void cblas_sger(const enum CBLAS_ORDER order, const BLASINT M, const BLASINT N, const float alpha, const float *X, const BLASINT incX, const float *Y, const BLASINT incY, float *A, const BLASINT lda);

void cblas_dger(const enum CBLAS_ORDER order, const BLASINT M, const BLASINT N, const double alpha, const double *X, const BLASINT incX, const double *Y, const BLASINT incY, double *A, const BLASINT lda);

Fortran interface：

CALL SGER(M, N, ALPHA, X, INCX, Y, INCY, A, LDA)

CALL DGER(M, N, ALPHA, X, INCX, Y, INCY, A, LDA)

参数

参数名	类型	描述	输入/输出
order	枚举类型CBLAS_ORDER	表示矩阵是行主序或列主序。	输入
M	整型数	表示矩阵A的行数，要求M大于或等于零。	输入
N	整型数	表示矩阵A的列数，要求N大于或等于零。	输入
alpha	• 在sger中是单精度浮点类型。 • 在dger中是双精度浮点类型。	乘法系数。	输入
X	• 在sger中是单精度浮点类型。 • 在dger中是双精度浮点类型。	矩阵X，向量规模至少是(1+(M-1)*abs(incx))。	输入
incX	整型数	向量X的增长步长，不能为零。	输入
Y	• 在sger中是单精度浮点类型。 • 在dger中是双精度浮点类型。	矩阵Y，向量规模至少是(1+(N-1)*abs(incy))。	输入
incY	整型数	向量Y的增长步长，不能为零。	输入

版权所有 © 华为技术有限公司

参数名	类型	描述	输入/输出
A	• 在sger中是单精度浮点类型。 • 在dger中是双精度浮点类型。	矩阵A(lda, n)。	输出
lda	整型数	矩阵A中主维度长度，如果矩阵A为列存，要求lda大于或等于max(1, m)，否则要求大于或等于max(1, n)。	输入

依赖

#include "kblas.h"

示例

C interface：

```
int m = 4, n = 4, lda =4;
float alpha = 1.0;
int incx = 1, incy = 1;
/**
 * X:
 *    0.340188, -0.105617, 0.283099, 0.298440
 * Y:
 *    0.411647, -0.302449, -0.164777, 0.268230
 *
 * Input A:
 *    -0.222225       -0.135216       0.135712        -0.483699
 *    0.053970        0.013401        0.217297        -0.257113
 *    -0.022603       0.452230        -0.358397       -0.362768
 *    0.128871        0.416195        0.106969        0.304177
 */
float x[4] = {0.340188, -0.105617, 0.283099, 0.298440};
float y[4] = {0.411647, -0.302449, -0.164777, 0.268230};
float a[16] = {-0.222225, -0.135216, 0.135712, -0.483699,
               0.053970 , 0.013401, 0.217297 , -0.257113,
               -0.022603, 0.452230, -0.358397, -0.362768,
               0.128871 , 0.416195, 0.106969, 0.304177};

cblas_sger(CblasColMajor, m, n, alpha, x, incx, y, incy, a, lda);
/**
 * Output A:
 *    -0.082188       -0.048920       -0.078658       0.220120
 *    -0.178693       0.045345        0.469633        0.387865
 *    0.252249        0.131674        -0.405045       0.182905
 *    -0.360847       -0.347376       -0.411944       0.384228
 */
```

Fortran interface：

```
  INTEGER :: M=4
  INTEGER :: N=4
  INTEGER :: LDA=4
  INTEGER :: INCX=1
  INTEGER :: INCY=1
  REAL(4) :: ALPHA=1.0
  REAL(4) :: A(4, 4)
```

 版权所有 © 华为技术有限公司

```
      DATA A/-0.222225, -0.135216, 0.135712, -0.483699,
     $       0.053970 , 0.013401, 0.217297 , -0.257113,
     $       -0.022603, 0.452230, -0.358397, -0.362768,
     $       0.128871 , 0.416195, 0.106969, 0.304177/
      REAL(4) :: X(4)
      DATA X/0.340188, -0.105617, 0.283099, 0.298440/
      REAL(4) :: Y(4)
      DATA Y/0.411647, -0.302449, -0.164777, 0.268230/
      EXTERNAL SGER
      CALL SGER(M, N, ALPHA, X, INCX, Y, INCY, A, LDA)
*     Output A:
*     -0.082188       -0.048920       -0.078658        0.220120
*     -0.178693        0.045345        0.469633        0.387865
*      0.252249        0.131674       -0.405045        0.182905
*     -0.360847       -0.347376       -0.411944        0.384228
```

6.4.8.4.2.5 ?gerc

一般复数矩阵秩1更新。

即：$A = A + \mathrm{alpha} * x * y'$。alpha是乘法系数，A是m*n的一般矩阵，x是含有m个元素的向量，y是含有n个元素的向量。

接口定义

C interface：

void cblas_cgerc(const enum CBLAS_ORDER order, const BLASINT M, const BLASINT N, const void *alpha, const void *X, const BLASINT incX, const void *Y, const BLASINT incY, void *A, const BLASINT lda);

void cblas_zgerc(const enum CBLAS_ORDER order, const BLASINT M, const BLASINT N, const void *alpha, const void *X, const BLASINT incX, const void *Y, const BLASINT incY, void *A, const BLASINT lda);

Fortran interface：

CALL CGERC(M, N, ALPHA, X, INCX, Y, INCY, A, LDA)

CALL ZGERC(M, N, ALPHA, X, INCX, Y, INCY, A, LDA)

参数

参数名	类型	描述	输入/输出
order	枚举类型CBLAS_ORDER	表示矩阵是行主序或列主序。	输入
M	整型数	矩阵A的行数。	输入
N	整型数	矩阵A的列数。	输入
alpha	• 在cgerc中是单精度复数类型。 • 在zgerc中是双精度复数类型。	乘法系数。	输入

版权所有 © 华为技术有限公司

参数名	类型	描述	输入/输出
X	• 在cgerc中是单精度复数类型。 • 在zgerc中是双精度复数类型。	矩阵X，向量规模至少1+(m-1)*abs(incX)。	输入
incX	整型数	向量X的增长步长，不能为零。	输入
Y	• 在cgerc中是单精度复数类型。 • 在zgerc中是双精度复数类型。	矩阵Y，向量规模至少1+(n-1)*abs(incY)。	输入
incY	整型数	向量Y的增长步长，不能为零。	输入
A	• 在cgerc中是单精度复数类型。 • 在zgerc中是双精度复数类型。	矩阵A(lda, n)。	输出
lda	整型数	矩阵A中主维度长度，如果矩阵A为列存，要求lda大于或等于max(1, m)，否则要求大于或等于max(1,n)。	输入

依赖

#include "kblas.h"

示例

C interface：

```
int m = 2, n = 2, lda = 2;
float alpha[2] = {1.0, 2.0};
int incx = 1, incy = 1;
float x[4] = {1.0, 2.0, 3.0, 4.0};
float y[4] = {2.0, 3.0, 4.0, 2.0};
float a[8] = {-1.0, 2.0, 2.0, 2.0, 3.0, 4.0, 5.0, 6.0};

cblas_cgerc(CblasColMajor, m, n, alpha, x, incx, y, incy, a, lda);
/**
 *  Output A:
 *           (5.000000, 19.000000), (-1.000000, 26.000000)
 *           (22.000000, 37.000000), (5.000000, 56.000000)
 */
```

Fortran interface：

```
  INTEGER :: M=2
  INTEGER :: N=2
  INTEGER :: LDA=2
```

 版权所有 © 华为技术有限公司

```
      INTEGER :: INCX=1
      INTEGER :: INCY=1
      COMPLEX(4) :: ALPHA=(1.0, 2.0)
      COMPLEX(4) :: A(2, 2)
      DATA A/(-1.0, 2.0), (2.0, 2.0),
     $       (3.0, 4.0), (5.0, 6.0)/
      COMPLEX(4) :: X(2)
      DATA X/(1.0, 2.0), (3.0, 4.0)/
      COMPLEX(4) :: Y(2)
      DATA Y/(2.0, 3.0), (4.0, 2.0)/
      EXTERNAL CGERC
      CALL CGERC(M, N, ALPHA, X, INCX, Y, INCY, A, LDA)

*     Output A:
*           (5.000000, 19.000000), (-1.000000, 26.000000)
*           (22.000000, 37.000000), (5.000000, 56.000000)
```

6.4.8.4.2.6 ?geru

一般共轭矩阵秩1更新。

即：$A = A + \text{alpha} * x * \text{conj}(y')$。alpha是乘法系数，A是m*n的一般矩阵，x是含有m个元素的向量，y是含有n个元素的向量。

接口定义

C interface：

void cblas_cgeru(const enum CBLAS_ORDER order, const BLASINT M, const BLASINT N, const void *alpha, const void *X, const BLASINT incX, const void *Y, const BLASINT incY, void *A, const BLASINT lda);

void cblas_zgeru(const enum CBLAS_ORDER order, const BLASINT M, const BLASINT N, const void *alpha, const void *X, const BLASINT incX, const void *Y, const BLASINT incY, void *A, const BLASINT lda);

Fortran interface：

CALL CGERU(M, N, ALPHA, X, INCX, Y, INCY, A, LDA)

CALL ZGERU(M, N, ALPHA, X, INCX, Y, INCY, A, LDA)

参数

参数名	类型	描述	输入/输出
order	枚举类型CBLAS_ORDER	表示矩阵是行主序或列主序。	输入
M	整型数	矩阵A的行数。	输入
N	整型数	矩阵A的列数。	输入
alpha	• 在cgeru中是单精度复数类型。 • 在zgeru中是双精度复数类型。	乘法系数。	输入

 版权所有 © 华为技术有限公司

参数名	类型	描述	输入/输出
X	• 在cgeru中是单精度复数类型。 • 在zgeru中是双精度复数类型。	矩阵X，向量规模至少是(1+(m-1)*abs(incX))。	输入
incX	整型数	向量X的增长步长，不能为零。	输入
Y	• 在cgeru中是单精度复数类型。 • 在zgeru中是双精度复数类型。	矩阵Y，向量规模至少是(1+(n-1)*abs(incY))。	输入
incY	整型数	向量Y的增长步长，不能为零。	输入
A	• 在cgeru中是单精度复数类型。 • 在zgeru中是双精度复数类型。	矩阵A(lda, n)。	输出
lda	整型数	矩阵A中主维度长度，如果矩阵A为列存，要求lda大于或等于max(1, m)，否则要求大于或等于max(1,n)。	输入

依赖

#include "kblas.h"

示例

C interface：

```
int m = 4, n = 4, lda = 4;
float alpha[2] = {1.0, 2.0};
int incx = 1, incy = 1;
/**
 * X:
 *    0.340188, -0.105617, 0.283099, 0.298440, 0.411647, -0.302449, -0.164777, 0.268230
 * Y:
 *   -0.222225, 0.053970, -0.022603, 0.128871, -0.135216, 0.013401, 0.452230, 0.416195
 * A:
 *     0.135712, 0.217297, -0.343321, -0.099056, 0.112640, -0.203968, 0.026745,    0.269914
 *    -0.358397, 0.106969, -0.370210, -0.391191, 0.137552, 0.024287, -0.099771, 0.391529
 *    -0.483699, -0.257113, 0.498924, -0.281743, -0.006417, 0.472775, -0.216685, -0.147542
 *    -0.362768, 0.304177, 0.012932, 0.339112, -0.207483, 0.271358, 0.307725, 0.419026
 */
float x[8] = {0.340188, -0.105617, 0.283099, 0.298440, 0.411647, -0.302449, -0.164777, 0.268230};
float y[8] = {-0.222225, 0.053970, -0.022603, 0.128871, -0.135216, 0.013401, 0.452230, 0.416195};
float a[32] = {0.135712, 0.217297, -0.343321, -0.099056, 0.112640, -0.203968, 0.026745, 0.269914,
               -0.358397, 0.106969, -0.370210, -0.391191, 0.137552, 0.024287, -0.099771, 0.391529,
               -0.483699, -0.257113, 0.498924, -0.281743, -0.006417, 0.472775, -0.216685, -0.147542,
               -0.362768, 0.304177, 0.012932, 0.339112, -0.207483, 0.271358, 0.307725, 0.419026};
```

 版权所有 © 华为技术有限公司

```
 cblas_cgeru(CblasColMajor, m, n, alpha, x, incx, y, incy, a, 4);
/**
 * Output A:
 *    -0.017847 0.119331  -0.444931 0.165040  -0.565962 -0.327440 -0.352610 0.793599
 *    -0.320256 -0.308135 -0.474544 -0.451172 0.529765 -0.402861  -0.488827 0.599533
 *    -0.141372 -0.264850 0.047453 0.143518   -0.150850 0.415971  0.035456  0.929981
 *    0.185887  0.245696  -0.076018 0.302546  -0.121045 -0.148647 0.016127  0.099442
 */
```

Fortran interface：

```
    INTEGER :: M=4
    INTEGER :: N=4
    INTEGER :: LDA=4
    INTEGER :: INCX=1
    INTEGER :: INCY=1
    COMPLEX(4) :: ALPHA=(1.0, 2.0)
    COMPLEX(4) :: A(4, 4)
    DATA A/(0.135712, 0.217297), (-0.343321, -0.099056),
   $       (0.112640, -0.203968), (0.026745, 0.269914),
   $       (-0.358397, 0.106969), (-0.370210, -0.391191),
   $       (0.137552, 0.024287), (-0.099771, 0.391529),
   $       (-0.483699, -0.257113), (0.498924, -0.281743),
   $       (-0.006417, 0.472775), (-0.216685, -0.147542),
   $       (-0.362768, 0.304177), (0.012932, 0.339112),
   $       (-0.207483, 0.271358), (0.307725, 0.419026)/
    COMPLEX(4) :: X(4)
    DATA X/(0.340188, -0.105617), (0.283099, 0.298440),
   $       (0.411647, -0.302449), (-0.164777, 0.268230)/
    COMPLEX(4) :: Y(4)
    DATA Y/(-0.222225, 0.053970), (-0.022603, 0.128871),
   $       (-0.135216, 0.013401), (0.452230, 0.416195)/
    EXTERNAL CGERU
    CALL CGERU(M, N, ALPHA, X, INCX, Y, INCY, A, LDA)

*    Output A:
   *    -0.017847 0.119331  -0.444931 0.165040  -0.565962 -0.327440 -0.352610 0.793599
   *    -0.320256 -0.308135 -0.474544 -0.451172 0.529765 -0.402861  -0.488827 0.599533
   *    -0.141372 -0.264850 0.047453 0.143518   -0.150850 0.415971  0.035456  0.929981
   *    0.185887  0.245696  -0.076018 0.302546  -0.121045 -0.148647 0.016127  0.099442
```

6.4.8.4.2.7 ?hbmv

向量与埃尔米特带状矩阵乘积。

即：$y = \mathrm{alpha} * A * x + \mathrm{beta} * y$。alpha和beta是乘法系数，x和y是含有n个元素的向量，A是n阶埃尔米特带状矩阵。

接口定义

C interface：

void cblas_chbmv(const enum CBLAS_ORDER order, const enum CBLAS_UPLO Uplo, const BLASINT N, const BLASINT K, const void *alpha, const void *A, const BLASINT lda, const void *X, const BLASINT incX, const void *beta, void *Y, const BLASINT incY);

void cblas_zhbmv(const enum CBLAS_ORDER order, const enum CBLAS_UPLO Uplo, const BLASINT N, const BLASINT K, const void *alpha, const void *A, const BLASINT lda, const void *X, const BLASINT incX, const void *beta, void *Y, const BLASINT incY);

Fortran interface：

 版权所有 © 华为技术有限公司

CALL CHBMV(UPLO, N, K, ALPHA, A, LDA, X, INCX, BETA, Y, INCY)

CALL ZHBMV(UPLO, N, K, ALPHA, A, LDA, X, INCX, BETA, Y, INCY)

参数

参数名	类型	描述	输入/输出
order	枚举类型CBLAS_ORDER	表示矩阵是行主序或列主序。	输入
Uplo	枚举类型CBLAS_UPLO	表示使用矩阵A展开方式（上三角或下三角） ● 如果Uplo=CblasUpper，则使用A的上三角展开。 ● 如果Uplo=CblasLower，则使用A的下三角展开。	输入
N	整型数	表示矩阵A的阶数，要求N大于或等于零。	输入
K	整型数	表示矩阵A的超对角阶数，要求K大于或等于零。	输入
alpha	● 在chbmv中是单精度复数类型。 ● 在zhbmv中是双精度复数类型。	表示系数。	输入
A	● 在chbmv中是单精度复数类型。 ● 在zhbmv中是双精度复数类型。	埃尔米特带状矩阵A(lda, n)。	输入
lda	整型数	矩阵A中主维度长度，要求lda大于或等于(k+1)。	输入
X	● 在chbmv中是单精度复数类型。 ● 在zhbmv中是双精度复数类型。	向量X，向量规模至少是(1+(N-1)*abs(incX))。	输入
incX	整型数	表示X中向量增长步长，不能为零。	输入
beta	● 在chbmv中是单精度复数类型。 ● 在zhbmv中是双精度复数类型。	乘法系数。	输入

 版权所有 © 华为技术有限公司

参数名	类型	描述	输入/输出
Y	• 在chbmv中是单精度复数类型。 • 在zhbmv中是双精度复数类型。	向量Y，向量规模至少是(1+(N-1)*abs(incY))。	输入/输出
incY	整型数	表示Y中向量增长步长，不能为零。	输入

依赖

#include "kblas.h"

示例

C interface：

```
int n = 3, k = 1, lda = 3;
float alpha[2] = {1.0, 0}, beta[2] = {1.0, 0};
int incx = 1, incy = 1;
/*
 *           | 1.0, 0.0  7.0, 0.0, 6.0, 0 |
 *      ap =  | 3.0,-5.0  4.0, 8.0     *   |
 *           |    *         *         *   |
 */
float ap[18] = {1.0, 0, 3.0, -5.0, 0, 0, 7.0, 0, 4.0, 8.0, 0.0, 0.0, 6.0, 0, 0, 0, 0, 0};
float x[6] = {3.0, 0.0, 2.0, 0.0, 1.0, 0.0};
float y[6] = {1.0, 0.0, 2.0, 0.0, 3.0, 0.0};

cblas_chbmv(CblasColMajor,CblasLower, n, k, alpha, ap, lda, x, incx, beta, y, incy);
/*
 * Output y = | 10.0, 10.0, 29.0, -23.0, 17.0, 16.0|
 */
```

Fortran interface：

```
  INTEGER :: N=3
  INTEGER :: K=1
  INTEGER :: LDA=3
  COMPLEX(4) :: ALPHA=(1.0, 0.0)
  COMPLEX(4) :: BETA=(1.0, 0.0)
  INTEGER :: INCX=1
  INTEGER :: INCY=1
  COMPLEX(4) :: A(3, 3)
  DATA A/(1.0, 0), (3.0, -5.0), (0, 0),
 $       (7.0, 0), (4.0, 8.0), (0.0, 0.0),
 $       (6.0, 0), (0, 0), (0, 0)/
  COMPLEX(4) :: X(3), Y(3)
  DATA X/(3.0, 0.0), (2.0, 0.0), (1.0, 0.0)/
  DATA Y/(1.0, 0.0), (2.0, 0.0), (3.0, 0.0)/
  EXTERNAL CHBMV
  CALL CHBMV('L', N, K, ALPHA, A, LDA, X, INCX, BETA, Y, INCY)

*     Output Y = | (10.0000000, 10.0000000) (29.0000000, -23.0000000) (17.0000000, 16.0000000)|
```

6.4.8.4.2.8 ?hemv

向量与埃尔米特矩阵乘积。

 版权所有 © 华为技术有限公司

即：$y = \text{alpha} * A * x + \text{beta} * y$。alpha和beta是乘法系数，x和y是含有n个元素的向量，A是n阶埃尔米特矩阵。

接口定义

C interface：

void cblas_chemv(const enum CBLAS_ORDER order, const enum CBLAS_UPLO Uplo, const BLASINT N, const void *alpha, const void *A, const BLASINT lda, const void *X, const BLASINT incX, const void *beta, void *Y, const BLASINT incY);

void cblas_zhemv(const enum CBLAS_ORDER order, const enum CBLAS_UPLO Uplo, const BLASINT N, const void *alpha, const void *A, const BLASINT lda, const void *X, const BLASINT incX, const void *beta, void *Y, const BLASINT incY);

Fortran interface：

CALL CHEMV(UPLO, N, ALPHA, A, LDA, X, INCX, BETA, Y, INCY)

CALL ZHEMV(UPLO, N, ALPHA, A, LDA, X, INCX, BETA, Y, INCY)

参数

参数名	类型	描述	输入/输出
order	枚举类型CBLAS_ORDER	表示矩阵是行主序或列主序。	输入
Uplo	枚举类型CBLAS_UPLO	表示使用矩阵A的上三角或下三角， ● 如果Uplo=CblasUpper，则使用A的上三角部分。 ● 如果Uplo=CblasLower，则使用A的下三角部分。	输入
N	整型数	表示矩阵A的阶数，要求N大于或等于零。	输入
alpha	● 在chemv中是单精度复数类型。 ● 在zhemv中是双精度复数类型。	表示系数。	输入
A	● 在chemv中是单精度复数类型。 ● 在zhemv中是双精度复数类型。	埃尔米特矩阵A(lda, n)。	输入
lda	整型数	矩阵A中主维度长度，要求lda大于或等于(k+1)。	输入

版权所有 © 华为技术有限公司

参数名	类型	描述	输入/输出
X	• 在chemv中是单精度复数类型。 • 在zhemv中是双精度复数类型。	向量X，向量规模至少是(1+(N-1)*abs(incX))。	输入
incX	整型数	表示X中向量增长步长，不能为零。	输入
beta	• 在chemv中是单精度复数类型。 • 在zhemv中是双精度复数类型。	乘法系数。	输入
Y	• 在chemv中是单精度复数类型。 • 在zhemv中是双精度复数类型。	向量Y，向量规模至少是(1+(N-1)*abs(incY))。	输入/输出
incY	整型数	表示Y中向量增长步长，不能为零。	输入

依赖

#include "kblas.h"

示例

C interface：

```
int n = 3, lda = 3;
float alpha[2] = {1.0, 0}, beta[2] = {1.0, 0};
int incx = 1, incy = 1;
/*            |  (21.0,  * )      *           *      |
 *      A =   | (1.0, -1.0)  (3.0,  * )       *      |
 *            |  (1.0, 3.0)  (2.0, 2.0)    (2.0, *  ) |
 */
float a[18] = {21.0, 0, 1.0, -1.0, 1.0, 3.0, 0, 0, 3.0, 0, 2.0, 2.0, 0, 0, 0, 0, 2.0, 0};
float x[6] = {3.0, 2.0, 1.0, 2.0, -1.0, 1.0};
float y[6] = {1.0, 2.0, 3.0, 2.0, -1.0, 1.0};

cblas_chemv(CblasColMajor,CblasLower, n, alpha, a, lda, x, incx, beta, y, incy);
/*
 * Output y = | 65.0, 51.0, 11.0, 11.0, -8.0, 20.0|
 */
```

Fortran interface：

```
  INTEGER :: N=3
  INTEGER :: LDA=3
  COMPLEX(4) :: ALPHA=(1.0, 0)
  COMPLEX(4) :: BETA=(1.0, 0)
  INTEGER :: INCX=1
  INTEGER :: INCY=1
  COMPLEX(4) :: A(3, 3)
  DATA A/(21.0, 0), (1.0, -1.0), (1.0, 3.0),
```

 版权所有 © 华为技术有限公司

```
     $       (0, 0), (3.0, 0), (2.0, 2.0),
     $       (0, 0), (0, 0), (2.0, 0)/

      COMPLEX(4) :: X(3)
      DATA X/(3.0, 2.0), (1.0, 2.0), (-1.0, 1.0)/
      COMPLEX(4) :: Y(3)
      DATA Y/(1.0, 2.0), (3.0, 2.0), (-1.0, 1.0)/
      EXTERNAL CHEMV
      CALL CHEMV('L', N, ALPHA, A, LDA, X, INCX, BETA, Y, INCY)

*     Output Y : (65.0, 51.0), (11.0, 11.0), (-8.0, 20.0)
```

6.4.8.4.2.9 ?her

复数埃尔米特矩阵秩1更新。

即：$A = A + \text{alpha} * x * \text{conj}(x')$。A是n*n的埃尔米特矩阵，alpha是乘法系数，x是有n个元素的向量。

接口定义

C interface：

void cblas_cher(const enum CBLAS_ORDER order, const enum CBLAS_UPLO Uplo, const BLASINT N, const float alpha, const void *X, const BLASINT incX, void *A, const BLASINT lda);

void cblas_zher(const enum CBLAS_ORDER order, const enum CBLAS_UPLO Uplo, const BLASINT N, const double alpha, const void *X, const BLASINT incX, void *A, const BLASINT lda);

Fortran interface：

CALL CHER(UPLO, N, ALPHA, X, INCX, A, LDA)

CALL ZHER(UPLO, N, ALPHA, X, INCX, A, LDA)

参数

参数名	类型	描述	输入/输出
order	枚举类型CBLAS_ORDER	表示矩阵是行主序或列主序。	输入
Uplo	枚举类型CBLAS_UPLO	埃尔米特矩阵存储方式（上三角或下三角）。 ● 如果Uplo=CblasUpper，则使用A的上三角展开。 ● 如果Uplo=CblasLower，则使用A的下三角展开。	输入
N	整型数	向量X的元素个数。	输入
alpha	● 在cher中是单精度浮点类型。 ● 在zher中是双精度浮点类型。	乘法系数。	输入

版权所有 © 华为技术有限公司

参数名	类型	描述	输入/输出
X	• 在cher中是单精度复数类型。 • 在zher中是双精度复数类型。	矩阵X，长度至少1+(n-1)*abs(incX)。	输入
incX	整型数	向量X的增长步长，不能为零。	输入
A	• 在cher中是单精度复数类型。 • 在zher中是双精度复数类型。	矩阵A(lda, n)。	输出
lda	整型数	矩阵A的主维度长度，要求lda大于或等于max(1, n)。	输入

依赖

#include "kblas.h"

示例

C interface：

```
int n = 3;
float alpha = 2.9;
int incx = 1;
/**
 *            |  (1.0, 0.0)   (13.0, 5.0)   (12.0, 0) |
 *        A =  | (13.0, -5.0)   (2.0, 0.0)   (4.0, -18.0) |
 *            |  (12.0, 0)   (4.0, 18.0)    (3.0, 0.0) |
 */
float ap[18] = {1.0, 0, 13.0, -5.0, 12.0, 0, 13.0, 5.0, 2.0, 0, 4.0, 18.0, 12.0, 0, 4.0, -18.0, 3.0, 0};
float x[6] = {2.0, -2.0, 14.0, 1, 3.0, 4.0};

cblas_cher(CblasColMajor, CblasLower, n, alpha, x, incx, ap, 3);
/**
 *            | (24.200001, 0.000000)   (13.000000    5.000000)     (12.000000   0.000000)  |
 * Output  A = | (88.400002, 82.000000) (573.300049   0.000000)     (4.000000    -18.000000)|
 *            | (6.200001, 40.600002)   (137.400009   171.700012)    (75.500000    0.000000) |

 */
```

Fortran interface：

```
    INTEGER :: N=3
    REAL(4) :: ALPHA=2.9
    INTEGER :: INCX=1
    COMPLEX(4) :: AP(3, 3)
    DATA AP/(1.0, 0), (13.0, -5.0), (12.0, 0), (13.0, 5.0), (2.0, 0), (4.0, 18.0), (12.0, 0), (4.0, -18.0), (3.0, 0)/
    COMPLEX(4) :: X(3)
    DATA X/(2.0, -2.0), (14.0, 1), (3.0, 4.0)/
    EXTERNAL CHER
    CALL CHER('L', N, ALPHA, X, INCX, AP, LDA)

*               | (24.200001, 0.000000)   (13.000000    5.000000)     (12.000000   0.000000)  |
*     Output  A = | (88.400002, 82.000000) (573.300049   0.000000)     (4.000000    -18.000000)|
*               | (6.200001, 40.600002)   (137.400009   171.700012)    (75.500000    0.000000) |
```


版权所有 © 华为技术有限公司

6.4.8.4.2.10 ?her2

复数埃尔米特矩阵秩2更新。

即：$A = A + \text{alpha} * x * \text{conj}(y') + \text{conj}(\text{alpha}) * y * \text{conj}(x')$。A是n*n的埃尔米特矩阵，alpha是乘法系数，x和y是有n个元素的向量。

接口定义

C interface：

void cblas_cher2(const enum CBLAS_ORDER order, const enum CBLAS_UPLO Uplo, const BLASINT N, const void *alpha, const void *X, const BLASINT incX, const void *Y, const BLASINT incY, void *A, const BLASINT lda);

void cblas_zher2(const enum CBLAS_ORDER order, const enum CBLAS_UPLO Uplo, const BLASINT N, const void *alpha, const void *X, const BLASINT incX, const void *Y, const BLASINT incY, void *A, const BLASINT lda);

Fortran interface：

CALL CHER2(UPLO, N, ALPHA, X, INCX, Y, INCY, A, LDA)

CALL ZHER2(UPLO, N, ALPHA, X, INCX, Y, INCY, A, LDA)

参数

参数名	类型	描述	输入/输出
order	枚举类型CBLAS_ORDER	表示矩阵是行主序或列主序。	输入
Uplo	枚举类型CBLAS_UPLO	埃尔米特矩阵存储方式（上三角或下三角）。 • 如果Uplo=CblasUpper，则使用A的上三角。 • 如果Uplo=CblasLower，则使用A的下三角。	输入
N	整型数	向量X的元素个数。	输入
alpha	• 在cher2中是单精度复数类型。 • 在zher2中是双精度复数类型。	乘法系数。	输入
X	• 在cher2中是单精度复数类型。 • 在zher2中是双精度复数类型。	矩阵X，长度至少 1+(n-1)*abs(incX)。	输入
incX	整型数	向量X的增长步长，不能为零。	输入

版权所有 © 华为技术有限公司

参数名	类型	描述	输入/输出
Y	• 在cher2中是单精度复数类型。 • 在zher2中是双精度复数类型。	矩阵Y，长度至少 1+(n-1)*abs(incY)。	输入
incY	整型数	向量Y的增长步长，不能为零。	输入
A	• 在cher2中是单精度复数类型。 • 在zher2中是双精度复数类型。	矩阵A(lda, n)。	输出
lda	整型数	矩阵A的主维度长度，要求lda大于或等于max(1, n)。	输入

依赖

#include "kblas.h"

示例

C interface：

```
int n = 3;
float alpha[2] = {1.0, 2.9};
int incx = 1, incy = 1;
/**
 *             |  (1.0, 0.0)   (13.0, 5.0)   (12.0, 0) |
 *         A =  | (13.0, -5.0)   (2.0, 0.0)   (4.0, -18.0) |
 *             |  (12.0, 0)   (4.0, 18.0)    (3.0, 0.0) |
 */
float ap[18] = {1.0, 0, 13.0, -5.0, 12.0, 0, 13.0, 5.0, 2.0, 0, 4.0, 18.0, 12.0, 0, 4.0, -18.0, 3.0, 0};
float x[6] = {2.0, -2.0, 14.0, 1, 3.0, 4.0};
float y[6] = {2.0, -2.0, 14.0, 1, 3.0, 4.0};

cblas_cher2(CblasColMajor, CblasLower, n, alpha, x, incx, y, incy, ap, 3);
/**
 *            | 17.000000 0.000000       13.000000 5.000000       12.000000 0.000000 |
 *  Output A = | 65.000000 55.000000      396.000000 0.000000      4.000000 -18.000000|
 *            | 8.000000 28.000000       96.000000 124.000000     53.000000 0.000000 |
 */
```

Fortran interface：

```
  INTEGER :: N=3
  COMPLEX(4) :: ALPHA=(1.0, 2.9)
  INTEGER :: INCX=1
  INTEGER :: INCY=1
  COMPLEX(4) :: AP(3, 3)
  DATA AP/(1.0, 0), (13.0, -5.0), (12.0, 0), (13.0, 5.0), (2.0, 0),
 $        (4.0, 18.0), (12.0, 0), (4.0, -18.0), (3.0, 0)/
  COMPLEX(4) :: X(3)
  DATA X/(2.0, -2.0), (14.0, 1), (3.0, 4.0)/
  COMPLEX(4) :: Y(3)
  DATA Y/(2.0, -2.0), (14.0, 1), (3.0, 4.0)/
```

 版权所有 © 华为技术有限公司

```
    EXTERNAL CHER2
    CALL CHER2('L', N, ALPHA, X, INCX, Y, INCY, A, LDA)

*               | 17.000000 0.000000      13.000000 5.000000      12.000000 0.000000 |
*    Output A = | 65.000000 55.000000     396.000000 0.000000     4.000000 -18.000000|
*               | 8.000000 28.000000      96.000000 124.000000    53.000000 0.000000 |
```

6.4.8.4.2.11 ?hpmv

向量与三角展开的埃尔米特矩阵乘积。

即$y = \mathrm{alpha} * A * x + \mathrm{beta} * y$。alpha和beta是缩放系数，x和y是含有n个元素的向量，A是三角展开的埃尔米特矩阵。

接口定义

C interface：

void cblas_chpmv(const enum CBLAS_ORDER order, const enum CBLAS_UPLO Uplo, const BLASINT N, const void *alpha, const void *Ap, const void *X, const BLASINT incX, const void *beta, void *Y, const BLASINT incY);

void cblas_zhpmv(const enum CBLAS_ORDER order, const enum CBLAS_UPLO Uplo, const BLASINT N, const void *alpha, const void *Ap, const void *X, const BLASINT incX, const void *beta, void *Y, const BLASINT incY);

Fortran interface：

CALL CHPMV(UPLO, N, ALPHA, AP, X, INCX, BETA, Y, INCY)

CALL ZHPMV(UPLO, N, ALPHA, AP, X, INCX, BETA, Y, INCY)

参数

参数名	类型	描述	输入/输出
order	枚举类型CBLAS_ORDER	表示矩阵是行主序或列主序。	输入
Uplo	枚举类型CBLAS_UPLO	表示矩阵A的存储展开方式（上三角或下三角）。 ● 如果Uplo=CblasUpper，则使用A的上三角展开。 ● 如果Uplo=CblasLower，则使用A的下三角展开。	输入
N	整型数	表示矩阵A的阶数，要求N大于或等于零。	输入
alpha	● 在chpmv中是单精度复数类型。 ● 在zhpmv中是双精度复数类型。	表示系数。	输入

 版权所有 © 华为技术有限公司

参数名	类型	描述	输入/输出
Ap	• 在chpmv中是单精度复数类型。 • 在zhpmv中是双精度复数类型。	压缩的埃尔米特矩阵，矩阵规模至少是(n*(n+1)/2)。	输入
X	• 在chpmv中是单精度复数类型。 • 在zhpmv中是双精度复数类型。	向量X，向量规模至少是(1+(N-1)*abs(incX))。	输入
incX	整型数	表示X中向量增长步长，不能为零。	输入
beta	• 在chpmv中是单精度复数类型。 • 在zhpmv中是双精度复数类型。	乘法系数。	输入
Y	• 在chpmv中是单精度复数类型。 • 在zhpmv中是双精度复数类型。	向量Y，向量规模至少是(1+(N-1)*abs(incY))。	输入/输出
incY	整型数	表示Y中向量增长步长，不能为零。	输入

依赖

#include "kblas.h"

示例

C interface：

```
int n = 3;
float alpha[2] = {1.0, 0}, beta[2] = {1.0, 0};
int incx = 1, incy = 1;
/*            |  (1.0, 0)   (3.0, 3.0)   (3.0, -3.0) |
*       A =   | (3.0, -3.0)   (2.0, 0.0)   (3.0, -1.0) |
*             |  (3.0, 3.0)   (3.0, 1.0)    (3.0, 0.0) |
*/
float ap[12] = {1.0, 0, 3.0, -3.0, 3.0, 3.0, 2.0, 0, 3.0, 1.0, 3.0, 0};
float x[6] = {3.0, 2.0, 1.0, 2.0, -1.0, 1.0};
float y[6] = {1.0, 2.0, 3.0, 2.0, -1.0, 1.0};

cblas_chpmv(CblasColMajor,CblasLower, n, alpha, ap, x, incx, beta, y, incy);
/*
* Output y = |1.000000, 19.000000, 18.000000, 7.000000,  0.000000, 26.000000|
*/
```

Fortran interface：

```
INTEGER :: N=3
COMPLEX(4) :: ALPHA=(1.0, 0)
```

 版权所有 © 华为技术有限公司

```
    COMPLEX(4) :: BETA=(1.0, 0)
    INTEGER :: INCX=1
    INTEGER :: INCY=1
    COMPLEX(4) :: AP(6)
    DATA AP/(1.0, 0), (3.0, -3.0), (3.0, 3.0), (2.0, 0), (3.0, 1.0), (3.0, 0)/
    COMPLEX(4) :: X(3), Y(3)
    DATA X/(3.0, 2.0), (1.0, 2.0), (-1.0, 1.0)/
    DATA Y/(1.0, 2.0), (3.0, 2.0), (-1.0, 1.0)/
    EXTERNAL CHPMV
    CALL CHPMV('L', N, ALPHA, AP, X, INCX, BETA, Y, INCY)

*   Output Y = |(1.000000, 19.000000), (18.000000, 7.000000),  (0.000000, 26.000000)|
```

6.4.8.4.2.12 ?hpr

三角展开的埃尔米特矩阵秩1更新。

即：$A = A + \text{alpha} * x * \text{conj}(x')$。alpha是缩放系数，x是含有n个元素的向量，A是三角展开的埃尔米特矩阵。

接口定义

C interface：

void cblas_chpr(const enum CBLAS_ORDER order, const enum CBLAS_UPLO Uplo, const BLASINT N, const float alpha, const void *X, const BLASINT incX, void *A);

void cblas_zhpr(const enum CBLAS_ORDER order, const enum CBLAS_UPLO Uplo, const BLASINT N, const double alpha, const void *X,const BLASINT incX, void *A);

Fortran interface：

CALL CHPR(UPLO, N, ALPHA, X, INCX, AP)

CALL ZHPR(UPLO, N, ALPHA, X, INCX, AP)

参数

参数名	类型	描述	输入/输出
order	枚举类型CBLAS_ORDER	表示矩阵是行主序或列主序。	输入
Uplo	枚举类型CBLAS_UPLO	埃尔米特矩阵展开存储方式（上三角或下三角展开）。 ● 如果Uplo=CblasUpper，则使用A的上三角展开。 ● 如果Uplo=CblasLower，则使用A的下三角展开。	输入
N	整型数	向量X的元素个数。	输入
alpha	● 在chpr中是单精度复数类型。 ● 在zhpr中是双精度复数类型。	乘法系数。	输入

 版权所有 © 华为技术有限公司

参数名	类型	描述	输入/输出
X	• 在chpr中是单精度复数类型。 • 在zhpr中是双精度复数类型。	矩阵X，长度至少 1+(n-1)*abs(incX)。	输入
incX	整型数	向量X的增长步长，不能为零。	输入
A	• 在chpr中是单精度复数类型。 • 在zhpr中是双精度复数类型。	矩阵A，矩阵规模至少是(n*(n+1)/2)。	输入/输出

依赖

#include "kblas.h"

示例

C interface：

```
int n = 3;
float alpha = 1.0;
int incx = 1;
/**
 *             |  (2.0, 0.0)   (13.0, 5.0)   (12.0, -13.0) |
 *         A =  | (13.0, -5.0)   (3.0, 0.0)   (4.0, -18.0)  |
 *             |  (12.0, 13.0)   (4.0, 18.0)    (2.0, 0.0) |
 */
float ap[12] = {2.0, 0, 13.0, -5.0, 12.0, 13.0, 3.0, 0, 4.0, 18.0, 2.0, 0};
float x[6] = {2.0, -2.0, 14.0, 1, 3.0, 4.0};

cblas_chpr(CblasColMajor, CblasLower, n, alpha, x, incx, ap);
/**
 * Output ap = |(10.0, 0.0), (39.0, 25.0), (10.0, 27.0), (200.0, 0.0), (50.0, 71.0), (27.0, 0.0) |
 */
```

Fortran interface：

```
    INTEGER :: N=3
    REAL(4) :: ALPHA=1.0
    INTEGER :: INCX=1
    COMPLEX(4) :: AP(6), X(3)
    DATA AP/(2.0, 0), (13.0, -5.0), (12.0, 13.0), (3.0, 0),
   $        (4.0, 18.0), (2.0, 0)/
    DATA X/(2.0, -2.0), (14.0, 1), (3.0, 4.0)/
    EXTERNAL CHPR
    CALL CHPR('L', N, ALPHA, X, INCX, AP)

*     Output AP = | (10.0, 0.0), (39.0, 25.0), (10.0, 27.0), (200.0, 0.0), (50.0, 71.0), (27.0, 0.0) |
```

6.4.8.4.2.13 ?hpr2

三角展开的埃尔米特矩阵秩2更新。

 版权所有 © 华为技术有限公司

即：$A = A + \text{alpha} * x * \text{conj}(y') + \text{conj}(\text{alpha}) * y * \text{conj}(x')$。alpha是乘法系数，x和y是含有n个元素的向量，A是n*n的埃尔米特矩阵。

接口定义

C interface：

void cblas_chpr2(const enum CBLAS_ORDER order, const enum CBLAS_UPLO Uplo, const BLASINT N, const void *alpha, const void *X, const BLASINT incX, const void *Y, const BLASINT incY, void *Ap);

void cblas_zhpr2(const enum CBLAS_ORDER order, const enum CBLAS_UPLO Uplo, const BLASINT N, const void *alpha, const void *X, const BLASINT incX, const void *Y, const BLASINT incY, void *Ap);

Fortran interface：

CALL CHPR2(UPLO, N, ALPHA, X, INCX, Y, INCY, AP)

CALL ZHPR2(UPLO, N, ALPHA, X, INCX, Y, INCY, AP)

参数

参数名	类型	描述	输入/输出
order	枚举类型CBLAS_ORDER	表示矩阵是行主序或列主序。	输入
Uplo	枚举类型CBLAS_UPLO	埃尔米特矩阵展开存储方式（上三角或下三角展开） • 如果Uplo=CblasUpper，则使用A的上三角展开。 • 如果Uplo=CblasLower，则使用A的下三角展开。	输入
N	整型数	向量X的元素个数。	输入
alpha	• 在chpr2中是单精度复数类型。 • 在zhpr2中是双精度复数类型。	乘法系数。	输入
X	• 在chpr2中是单精度复数类型。 • 在zhpr2中是双精度复数类型。	矩阵X，长度至少 1+(n-1)*abs(incX)。	输入
incX	整型数	向量X的增长步长，不能为零。	输入
Y	• 在cher2中是单精度复数类型。 • 在zher2中是双精度复数类型。	矩阵Y，长度至少1+(n-1)*abs(incY)。	输入

版权所有 © 华为技术有限公司

参数名	类型	描述	输入/输出
incY	整型数	向量Y的增长步长，不能为零。	输入
Ap	• 在chpr2中是单精度复数类型。 • 在zhpr2中是双精度复数类型。	矩阵A三角存储，矩阵规模至少是(n*(n+1)/2)。	输入/输出

依赖

#include "kblas.h"

示例

C interface：

```
int n = 3;
float alpha[2] = {1.0, 2.9};
int incx = 1, incy = 1;
/**
 *             |  (2.0, 0.0)   (13.0, 5.0)   (12.0, -13.0) |
 *         A =  | (13.0, -5.0)   (2.0, 0.0)   (4.0, -18.0)  |
 *             |  (12.0, 13.0)   (4.0, 18.0)    (1.0, 0.0) |
 */
float ap[12] = {2.0, 0, 13.0, -5.0, 12.0, 13.0, 2.0, 0, 4.0, 18.0, 1.0, 0};
float x[6] = {2.0, -2.0, 14.0, 1, 3.0, 4.0};
float y[6] = {2.0, -2.0, 14.0, 1, 3.0, 4.0};

cblas_chpr2(CblasColMajor, CblasLower, n, alpha, x, incx, y, incy, ap);
/**
 * Output ap = |(18.0, 0.0), (65.0, 55.0), (8.0, 41.0), (396.0, 0.0), (96.0, 124.0), (51.0, 0.0)|
 */
```

Fortran interface：

```
 INTEGER :: N=3
  COMPLEX(4) :: ALPHA=(1.0, 2.9)
  INTEGER :: INCX=1, INCY=1
  COMPLEX(4) :: AP(6), X(3), Y(3)
  DATA AP/(2.0, 0), (13.0, -5.0), (12.0, 13.0), (2.0, 0),
 $        (4.0, 18.0), (1.0, 0)/
  DATA X/(2.0, -2.0), (14.0, 1), (3.0, 4.0)/
  DATA Y/(2.0, -2.0), (14.0, 1), (3.0, 4.0)/
  EXTERNAL CHPR2
  CALL CHPR2('L', N, ALPHA, X, INCX, Y, INCY, AP)

*     Output AP = |(18.0, 0.0), (65.0, 55.0), (8.0, 41.0), (396.0, 0.0), (96.0, 124.0), (51.0, 0.0)|
```

6.4.8.4.2.14 ?sbmv

向量与对称带状矩阵乘积。

即$y = \mathrm{alpha} * A * x + \mathrm{beta} * y$。alpha和beta是乘法系数，x和y是含有n个元素的向量，A是n*n的对称带状矩阵。

 版权所有 © 华为技术有限公司

接口定义

C interface：

void cblas_ssbmv(const enum CBLAS_ORDER order, const enum CBLAS_UPLO Uplo, const BLASINT N, const BLASINT K, const float alpha, const float *A, const BLASINT lda, const float *X, const BLASINT incX, const float beta, float *Y, const BLASINT incY);

void cblas_dsbmv(const enum CBLAS_ORDER order, const enum CBLAS_UPLO Uplo, const BLASINT N, const BLASINT K, const double alpha, const double *A, const BLASINT lda, const double *X, const BLASINT incX, const double beta, double *Y, const BLASINT incY);

Fortran interface：

CALL SSBMV(UPLO, N, K, ALPHA, A, LDA, X, INCX, BETA, Y, INCY)

CALL DSBMV(UPLO, N, K, ALPHA, A, LDA, X, INCX, BETA, Y, INCY)

参数

参数名	类型	描述	输入/输出
order	枚举类型CBLAS_ORDER	表示矩阵是行主序或列主序。	输入
Uplo	枚举类型CBLAS_UPLO	表示使用矩阵A的上三角或下三角。 ● 如果Uplo=CblasUpper，则使用A的上三角部分。 ● 如果Uplo=CblasLower，则使用A的下三角部分。	输入
N	整型数	表示矩阵A的阶数，要求N大于或等于零。	输入
K	整型数	表示矩阵A的超对角阶数，要求K大于或等于零。	输入
alpha	● 在ssbmv中是单精度浮点类型。 ● 在dsbmv中是双精度浮点类型。	表示乘法系数。	输入
A	● 在ssbmv中是单精度浮点类型。 ● 在dsbmv中是双精度浮点类型。	对称带状矩阵，矩阵规模是lda*n。	输入
lda	整型数	矩阵A中每列存储的地址距离，要求lda大于或等于(k+1)。	输入

版权所有 © 华为技术有限公司

参数名	类型	描述	输入/输出
X	• 在ssbmv中是单精度浮点类型。 • 在dsbmv中是双精度浮点类型。	向量X，向量规模至少是(1+(N-1)*abs(incX))。	输入
incX	整型数	表示X中向量增长步长，不能为零。	输入
beta	• 在ssbmv中是单精度浮点类型。 • 在dsbmv中是双精度浮点类型。	乘法系数。	输入
Y	• 在ssbmv中是单精度浮点类型。 • 在dsbmv中是双精度浮点类型。	向量Y，向量规模至少是(1+(N-1)*abs(incY))。	输入/输出
incY	整型数	表示Y中向量增长步长，不能为零。	输入

依赖

#include "kblas.h"

示例

C interface：

```
int n = 4, k = 2;
float alpha = 2.0;
/**
 *       |  .    .   2.0  2.0 |
 *       |  .   1.0  2.0  3.0 |
 *       | 1.0  2.0  2.0  4.0 |
 *  A  =  | 1.0  3.0  2.0  5.0 |
 *       | 1.0  4.0  2.0  6.0 |
 *       | 1.0  5.0  2.0  2.0 |
 *       |  .    .    .    .  |
 *       |  .    .    .    .  |
 */
float a[32] = {0, 0, 1.0, 1.0, 1.0, 1.0, 0, 0,
               0, 1.0, 2.0, 3.0, 4.0, 5.0, 0, 0,
               2.0, 2.0, 2.0, 2.0, 2.0, 2.0, 0, 0,
               2.0, 3.0, 4.0, 5.0, 6.0, 2.0, 0, 0};
float x[4] = {2.0, 2.0, 3.0, 4.0};
float y[4] = {1.0, 1.0, 13.0, 4.0};

cblas_ssbmv(CblasColMajor,CblasUpper, n, k, alpha, a, 8, x, 1, 1.0, y, 1);
/**
 * Output y = |21.0, 41.0, 65.0, 62.0|
 */
```

Fortran interface：

版权所有 © 华为技术有限公司

```
      INTEGER :: N=4, K=2
      REAL(4) :: ALPHA=2.0
      REAL(4) :: BETA=1.0
      REAL(4) :: A(32)
      DATA A/0, 0, 1.0, 1.0, 1.0, 1.0, 0, 0,
     $       0, 1.0, 2.0, 3.0, 4.0, 5.0, 0, 0,
     $       2.0, 2.0, 2.0, 2.0, 2.0, 2.0, 0, 0,
     $       2.0, 3.0, 4.0, 5.0, 6.0, 2.0, 0, 0/
      REAL(4) :: X(4), Y(4)
      DATA X/2.0, 2.0, 3.0, 4.0/
      DATA Y/1.0, 1.0, 13.0, 4.0/
      EXTERNAL SSBMV
      CALL SSBMV('L', N, K, ALPHA, A, 8, X, 1, BETA, Y, 1)

*     Output Y = |21.0, 41.0, 65.0, 62.0|
```

6.4.8.4.2.15 ?spmv

向量与压缩的对称矩阵乘积

即$y = \mathrm{alpha} * A * x + \mathrm{beta} * y$。alpha和beta是缩放系数，x和y是含有n个元素的向量，A是n阶压缩的对称矩阵。

接口定义

C interface：

void cblas_sspmv(const enum CBLAS_ORDER order, const enum CBLAS_UPLO Uplo, const BLASINT N, const float alpha, const float *Ap, const float *X, const BLASINT incX, const float beta, float *Y, const BLASINT incY);

void cblas_dspmv(const enum CBLAS_ORDER order, const enum CBLAS_UPLO Uplo, const BLASINT N, const double alpha, const double *Ap, const double *X, const BLASINT incX, const double beta, double *Y, const BLASINT incY);

Fortran interface：

CALL SSPMV(UPLO, N, ALPHA, AP, X, INCX, BETA, Y, INCY)

CALL DSPMV(UPLO, N, ALPHA, AP, X, INCX, BETA, Y, INCY)

参数

参数名	类型	描述	输入/输出
order	枚举类型CBLAS_ORDER	表示矩阵是行主序或列主序。	输入
Uplo	枚举类型CBLAS_UPLO	表示使用矩阵A存储展开方式（上三角或下三角）。 ● 如果Uplo=CblasUpper，则使用A的上三角展开。 ● 如果Uplo=CblasLower，则使用A的下三角展开。	输入
N	整型数	表示矩阵A的阶数，要求N大于或等于零。	输入

 版权所有 © 华为技术有限公司

参数名	类型	描述	输入/输出
alpha	• 在sspmv中是单精度复数类型。 • 在dspmv中是双精度复数类型。	表示乘法系数。	输入
Ap	• 在sspmv中是单精度复数类型。 • 在dspmv中是双精度复数类型。	压缩的对称矩阵，矩阵规模至少是(n*(n+1)/2)。	输入
X	• 在sspmv中是单精度复数类型。 • 在dspmv中是双精度复数类型。	向量X，向量规模至少是(1+(N-1)*abs(incX))。	输入
incX	整型数	表示X中向量增长步长，不能为零。	输入
beta	• 在sspmv中是单精度复数类型。 • 在dspmv中是双精度复数类型。	乘法系数。	输入
Y	• 在sspmv中是单精度复数类型。 • 在dspmv中是双精度复数类型。	向量Y，向量规模至少是(1+(N-1)*abs(incY))。	输入/输出
incY	整型数	表示Y中向量增长步长，不能为零。	输入

依赖

#include "kblas.h"

示例

C interface：

```
int n = 3;
float alpha = 1.0, beta = 1.0;
int incx = 1, incy = 1;
/*
 * A =  | 2.0  1.0  3.0 |
 *      | 1.0  6.0  9.0 |
 *      | 3.0  9.0  2.0 |
 */
float a[6] = {2.0, 1.0, 3.0, 6.0, 9.0, 2.0};
float x[3] = {1.0, 1.0, 1.0};
float y[3] = {3.0, 2.0, 2.0};
```

 版权所有 © 华为技术有限公司

```
cblas_sspmv(CblasColMajor,CblasLower, n, alpha, a, x, incx, beta, y, incy);
/*
 * Output y = |9.0, 18.0, 16.0|
 */
```

Fortran interface：

```
    INTEGER :: N=3
    REAL(4) :: ALPHA=2.0
    REAL(4) :: BETA=1.0
    REAL(4) :: A(6), X(3), Y(3)
    DATA A/2.0, 1.0, 3.0, 6.0, 9.0, 2.0/
    DATA X/1.0, 1.0, 1.0/
    DATA Y/3.0, 2.0, 2.0/
    EXTERNAL SSPMV
    CALL SSPMV('L', N, ALPHA, A, X, INCX, BETA, Y, INCY)

*     Output Y = |9.0, 18.0, 16.0|
```

6.4.8.4.2.16 ?spr

对称展开矩阵秩1更新。

即：$A = A + \text{alpha} * x * x'$。alpha为乘法系数，x为包含n个元素的向量，A为n*n的三角展开对称矩阵。

接口定义

C interface：

void cblas_sspr(const enum CBLAS_ORDER order, const enum CBLAS_UPLO Uplo, const BLASINT N, const float alpha, const float *X, const BLASINT incX, float *Ap);

void cblas_dspr(const enum CBLAS_ORDER order, const enum CBLAS_UPLO Uplo, const BLASINT N, const double alpha, const double *X, const BLASINT incX, double *Ap);

Fortran interface：

CALL SSPR(UPLO, N, ALPHA, X, INCX, AP)

CALL DSPR(UPLO, N, ALPHA, X, INCX, AP)

参数

参数名	类型	描述	输入/输出
order	枚举类型CBLAS_ORDER	表示矩阵是行主序或列主序。	输入
Uplo	枚举类型CBLAS_UPLO	对称矩阵展开存储方式（上三角或下三角展开）。 ● 如果Uplo=CblasUpper，则使用A的上三角展开。 ● 如果Uplo=CblasLower，则使用A的下三角展开。	输入
N	整型数	向量X的元素个数。	输入

 版权所有 © 华为技术有限公司

参数名	类型	描述	输入/输出
alpha	• 在sspr中是单精度浮点类型。 • 在dspr中是双精度浮点类型。	乘法系数。	输入
X	• 在sspr中是单精度浮点类型。 • 在dspr中是双精度浮点类型。	矩阵X，长度至少1+(n-1)*abs(incX)。	输入
incX	整型数	向量X的增长步长，不能为零。	输入
Ap	在sspr中是单精度浮点类型。 在dspr中是双精度浮点类型。	矩阵A。	输出

依赖

#include "kblas.h"

示例

C interface：

```
int n = 3;
float alpha = 1.0;
int incx = 1;
float x[3] = {2.0, 2.0, 1.0};
/**
 *            | 3.0  1.0  2.0 |
 *        A = | 1.0  6.0  3.0 |
 *            | 2.0  3.0  3.0 |
 */
float a[6] = {3.0, 1.0, 2.0, 6.0, 3.0, 3.0};

cblas_sspr(CblasColMajor,CblasLower, n, alpha, x, incx, a);
/**
 *  Output a = |7.0, 5.0, 4.0, 10.0, 5.0, 4.0|
 */
```

Fortran interface：

```
  INTEGER :: N=3, INCX=1
  REAL(4) :: ALPHA=2.0
  REAL(4) :: X(3), A(6)
  DATA X/2.0, 2.0, 1.0/
  DATA A/3.0, 1.0, 2.0, 6.0, 3.0, 3.0/
  EXTERNAL SSPR
  CALL SSPR('L', N, ALPHA, X, INCX, A)

*    Output A = |7.0, 5.0, 4.0, 10.0, 5.0, 4.0|
```

6.4.8.4.2.17 ?spr2

对称展开矩阵秩2更新。

 版权所有 © 华为技术有限公司

即：$A = A + \text{alpha} * x * y' + \text{alpha} * y * x'$。alpha为乘法系数，x和y为包含n个元素的向量，A为n*n的三角展开对称矩阵。

接口定义

C interface：

void cblas_sspr2(const enum CBLAS_ORDER order, const enum CBLAS_UPLO Uplo, const BLASINT N, const float alpha, const float *X, const BLASINT incX, const float *Y, const BLASINT incY, float *A);

void cblas_dspr2(const enum CBLAS_ORDER order, const enum CBLAS_UPLO Uplo, const BLASINT N, const double alpha, const double *X, const BLASINT incX, const double *Y, const BLASINT incY, double *A);

Fortran interface：

CALL SSPR2(UPLO, N, ALPHA, X, INCX, Y, INCY, AP)

CALL DSPR2(UPLO, N, ALPHA, X, INCX, Y, INCY, AP)

参数

参数名	类型	描述	输入/输出
order	枚举类型CBLAS_ORDER	表示矩阵是行主序或列主序。	输入
Uplo	枚举类型CBLAS_UPLO	对称矩阵展开存储方式（上三角或下三角展开）。 • 如果Uplo=CblasUpper，则使用A的上三角展开。 • 如果Uplo=CblasLower，则使用A的下三角展开。	输入
N	整型数	向量X的元素个数。	输入
alpha	• 在sspr2中是单精度复数类型。 • 在dspr2中是双精度复数类型。	乘法系数。	输入
X	• 在sspr2中是单精度复数类型。 • 在dspr2中是双精度复数类型。	矩阵X，长度至少 1+(n-1)*abs(incX)。	输入
incX	整型数	向量X的增长步长，不能为零。	输入

版权所有 © 华为技术有限公司

参数名	类型	描述	输入/输出
Y	• 在sspr2中是单精度复数类型。 • 在dspr2中是双精度复数类型。	矩阵Y，长度至少 1+(n-1)*abs(incY)。	输入
incY	整型数	向量Y的增长步长，不能为零。	输入
A	• 在sspr2中是单精度复数类型。 • 在dspr2中是双精度复数类型。	矩阵A三角存储。	输入/输出

依赖

#include "kblas.h"

示例

C interface：

```
float alpha = 1.0;
int n = 3, incx = 1, incy = 1;
float x[3] = {2.0, 2.0, 1.0};
float y[3] = {1.0, 2.0, 1.0};
/**
 *            | 3.0  1.0  2.0 |
 *        A = | 1.0  6.0  3.0 |
 *            | 2.0  3.0  3.0 |
 */
float a[6] = {3.0, 1.0, 2.0, 6.0, 3.0, 3.0};

cblas_sspr2(CblasColMajor,CblasLower, n, alpha, x, incx, y, incy, a);
/**
 *  Output a = |7.0, 7.0, 5.0, 14.0, 7.0, 5.0|
 */
```

Fortran interface：

```
  REAL(4) :: ALPHA=1.0
  INTEGER :: N=3
  INTEGER :: INCX=1
  INTEGER :: INCY=1
  REAL(4) :: X(3), Y(3), A(6)
  DATA X/2.0, 2.0, 1.0/
  DATA Y/1.0, 2.0, 1.0/
  DATA A/3.0, 1.0, 2.0, 6.0, 3.0, 3.0/
  EXTERNAL SSPR2
  CALL SSPR2('L', N, ALPHA, X, INCX, Y, INCY, A)

*   Output A = |7.0, 7.0, 5.0, 14.0, 7.0, 5.0|
```

6.4.8.4.2.18 ?symv

向量与对称矩阵乘积

 版权所有 © 华为技术有限公司

即$y = \mathrm{alpha} * A * x + \mathrm{beta} * y$。alpha和beta是乘法系数，x和y为包含n个元素向量，A为n*n的对称矩阵。

接口定义

C interface：

void cblas_ssymv(const enum CBLAS_ORDER order, const enum CBLAS_UPLO Uplo, const BLASINT N, const float alpha, const float *A, const BLASINT lda, const float *X, const BLASINT incX, const float beta, float *Y, const BLASINT incY);

void cblas_dsymv(const enum CBLAS_ORDER order, const enum CBLAS_UPLO Uplo, const BLASINT N, const double alpha, const double *A, const BLASINT lda, const double *X, const BLASINT incX, const double beta, double *Y, const BLASINT incY);

Fortran interface：

CALL SSYMV(UPLO, N, ALPHA, A, LDA, X, INCX, BETA, Y, INCY)

CALL DSYMV(UPLO, N, ALPHA, A, LDA, X, INCX, BETA, Y, INCY)

参数

参数名	类型	描述	输入/输出
order	枚举类型CBLAS_ORDER	表示矩阵是行主序或列主序。	输入
Uplo	枚举类型CBLAS_UPLO	表示使用矩阵A的上三角或下三角。 • 如果Uplo=CblasUpper，则使用A的上三角部分。 • 如果Uplo=CblasLower，则使用A的下三角部分。	输入
N	整型数	表示矩阵A的阶数，要求N大于或等于零。	输入
alpha	• 在ssymv中是单精度浮点类型。 • 在dsymv中是双精度浮点类型。	表示系数。	输入
A	• 在ssymv中是单精度浮点类型。 • 在dsymv中是双精度浮点类型。	对称矩阵A(lda, n)。	输入

版权所有 © 华为技术有限公司

参数名	类型	描述	输入/输出
lda	整型数	矩阵A中每列存储的地址距离，要求lda大于或等于max(1, n)。	输入
X	• 在ssymv中是单精度浮点类型。 • 在dsymv中是双精度浮点类型。	向量X，向量规模至少是(1+(N-1)*abs(incX))。	输入
incX	整型数	表示X中向量增长步长，不能为零。	输入
beta	• 在ssymv中是单精度浮点类型。 • 在dsymv中是双精度浮点类型。	乘法系数。	输入
Y	• 在ssymv中是单精度浮点类型。 • 在dsymv中是双精度浮点类型。	向量Y，向量规模至少是(1+(N-1)*abs(incY))。	输入/输出
incY	整型数	表示Y中向量增长步长，不能为零。	输入

依赖

#include "kblas.h"

示例

C interface：

```
int n = 3, lda = 3;
float alpha = 1.0, beta = 1.0;
int incx = 1, incy = 1;
/*
 * A =  | 0.0   *    *  |
 *      | 2.0  3.0   *  |
 *      | 0.0  4.0  1.0 |
 */
float a[9] = {0, 2.0, 0, 0, 3.0, 4.0, 0, 0, 1.0};
float x[3] = {5.0, 2.0, 1.0};
float y[3] = {1.0, 1.0, 3.0};

cblas_ssymv(CblasColMajor,CblasLower, n, alpha, a, lda, x, incx, beta, y, incy);
/*
 * Output y: 5.000000, 21.000000, 12.000000
 */
```

Fortran interface：

```
INTEGER :: N=3, LDA=3
REAL(4) :: ALPHA=1.0, BETA=1.0
```


版权所有 © 华为技术有限公司

```
      INTEGER :: INCX=1, INCY=1
      REAL(4) :: A(9), X(3), Y(3)
      DATA A/0, 2.0, 0, 0, 3.0, 4.0, 0, 0, 1.0/
      DATA X/5.0, 2.0, 1.0/
      DATA Y/1.0, 1.0, 3.0/
      EXTERNAL SSYMV
      CALL SSYMV('L', N, ALPHA, A, LDA, X, INCX, BETA, Y, INCY)

*     Output Y: 5.000000, 21.000000, 12.000000
```

6.4.8.4.2.19 ?syr

对称矩阵秩1更新

即：$A = A + \text{alpha} * x * x'$。alpha为乘法系数，x为包含n个元素的向量，A为n*n的对称矩阵。

接口定义

C interface：

void cblas_ssyr(const enum CBLAS_ORDER order, const enum CBLAS_UPLO Uplo, const BLASINT N, const float alpha, const float *X, const BLASINT incX, float *A, const BLASINT lda);

void cblas_dsyr(const enum CBLAS_ORDER order, const enum CBLAS_UPLO Uplo, const BLASINT N, const double alpha, const double *X, const BLASINT incX, double *A, const BLASINT lda);

Fortran interface：

CALL SSYR(UPLO, N, ALPHA, X, INCX, A, LDA)

CALL DSYR(UPLO, N, ALPHA, X, INCX, A, LDA)

参数

参数名	类型	描述	输入/输出
order	枚举类型CBLAS_ORDER	表示矩阵是行主序或列主序。	输入
Uplo	枚举类型CBLAS_UPLO	使用矩阵A的上三角或下三角。 ● 如果Uplo=CblasUpper，则使用A的上三角。 ● 如果Uplo=CblasLower，则使用A的下三角。	输入
N	整型数	向量X的元素个数。	输入
alpha	● 在ssyr中是单精度浮点类型。 ● 在dsyr中是双精度浮点类型。	乘法系数。	输入

 版权所有 © 华为技术有限公司

参数名	类型	描述	输入/输出
X	• 在ssyr中是单精度浮点类型。 • 在dsyr中是双精度浮点类型。	矩阵X，长度至少1+(n-1)*abs(incX)。	输入
incX	整型数	向量X的增长步长，不能为零。	输入
A	• 在ssyr中是单精度浮点类型。 • 在dsyr中是双精度浮点类型。	矩阵A(lda, n)。	输入/输出
lda	整型数	矩阵A的主维度长度，要求lda大于或等于max(1, n)。	输入

依赖

#include "kblas.h"

示例

C interface：

```
int n = 3, lda = 4, incx = 1;
float alpha = 1.0;
float x[3] = {1.0, 3.0, 2.0};
/**
 *              | 3.0  14.0  12.0 |
 *        A   = |  .   16.0  17.0 |
 *              |  .    .    13.0 |
 *              |  .    .     .   |
 */
float a[12] = {3.0, 0, 0, 0, 14.0, 16.0, 0, 0, 12, 17.0, 13.0, 0};

cblas_ssyr(CblasColMajor,CblasUpper, n, alpha, x, incx, a, lda);
/**
 *            | 4.0  17.0  14.0 |
 *  Output A = |  .   25.0  23.0 |
 *            |  .    .    17.0 |
 *            |  .    .     .   |
 */
```

Fortran interface：

```
    INTEGER :: N=3, LDA=4
    INTEGER :: INCX=1
    REAL(4) :: ALPHA=1.0
    REAL(4) :: X(3), A(12)
    DATA X/1.0, 3.0, 2.0/
    DATA A/3.0, 0, 0, 0, 14.0, 16.0, 0, 0, 12, 17.0, 13.0, 0/
    EXTERNAL SSYR
    CALL SSYR('U', N, ALPHA, X, INCX, A, LDA)

*              | 4.0  17.0  14.0 |
*    Output A = |  .   25.0  23.0 |
```

 版权所有 © 华为技术有限公司

```
*           | .   .   17.0 |
*           | .   .    .   |
```

6.4.8.4.2.20 ?syr2

对称矩阵秩2更新

即：$A = A + \text{alpha} * x * y' + \text{alpha} * y * x'$。alpha为乘法系数，x和y为包含n个元素的向量，A为n*n的对称矩阵。

接口定义

C interface：

void cblas_ssyr2(const enum CBLAS_ORDER order, const enum CBLAS_UPLO Uplo,const BLASINT N, const float alpha, const float *X, const BLASINT incX, const float *Y, const BLASINT incY, float *A, const BLASINT lda);

void cblas_dsyr2(const enum CBLAS_ORDER order, const enum CBLAS_UPLO Uplo, const BLASINT N, const double alpha, const double *X, const BLASINT incX, const double *Y, const BLASINT incY, double *A, const BLASINT lda);

Fortran interface：

CALL SSYR2(UPLO, N, ALPHA, X, INCX, Y, INCY, A, LDA)

CALL DSYR2(UPLO, N, ALPHA, X, INCX, Y, INCY, A, LDA)

参数

参数名	类型	描述	输入/输出
order	枚举类型CBLAS_ORDER	表示矩阵是行主序或列主序。	输入
Uplo	枚举类型CBLAS_UPLO	使用矩阵A的上三角或下三角。 • 如果Uplo=CblasUpper，则使用A的上三角。 • 如果Uplo=CblasLower，则使用A的下三角。	输入
N	整型数	向量X的元素个数。	输入
alpha	• 在ssyr2中是单精度复数类型。 • 在dsyr2中是双精度复数类型。	乘法系数。	输入
X	• 在ssyr2中是单精度复数类型。 • 在dsyr2中是双精度复数类型。	矩阵X，长度至少1+(n-1)*abs(incX)。	输入

 版权所有 © 华为技术有限公司

参数名	类型	描述	输入/输出
incX	整型数	向量X的增长步长，不能为零。	输入
Y	• 在ssyr2中是单精度复数类型。 • 在dsyr2中是双精度复数类型。	矩阵Y，长度至少1+(n-1)*abs(incY)。	输入
incY	整型数	向量Y的增长步长，不能为零。	输入
A	• 在ssyr2中是单精度复数类型。 • 在dsyr2中是双精度复数类型。	矩阵A(lda, n)。	输入/输出
lda	整型数	矩阵A的主维度长度，要求lda大于或等于max(1, n)。	输入

依赖

#include "kblas.h"

示例

C interface：

```
int n = 3, lda = 4;
float alpha = 1.0;
int incx = 1, incy = 1;
float x[3] = {1.0, 3.0, 2.0};
float y[3] = {2.0, 3.0, 2.0};
/**
 *          | 3.0  1.0   1.0  |
 *    A   = |  .   16.0  10.0 |
 *          |  .    .    1.0  |
 *          |  .    .     .   |
 */
float a[12] = {3.0, 0, 0, 0, 1.0, 16.0, 0, 0, 1.0, 10.0, 1.0, 0};

cblas_ssyr2(CblasColMajor,CblasUpper, n, alpha, x, incx, y, incy, a, lda);
/***
 *          | 7.0  10.  7.0 |
 *          |  .   34.  22.0|
 * Output A = |  .    .   9.0 |
 *          |  .    .    .  |
 */
```

Fortran interface：

```
  INTEGER :: N=3, LDA=4
  INTEGER :: INCX=1, INCY=1
  REAL(4) :: ALPHA=1.0
  REAL(4) :: X(3), Y(3), A(12)
  DATA X/1.0, 3.0, 2.0/
  DATA Y/2.0, 3.0, 2.0/
```

 版权所有 © 华为技术有限公司

```
      DATA A/3.0, 0, 0, 0, 1.0, 16.0, 0, 0, 1.0, 10.0, 1.0, 0/
      EXTERNAL SSYR2
      CALL SSYR2('U', N, ALPHA, X, INCX, Y, INCY, A, LDA)

*               | 7.0  10.  7.0 |
*               |  .   34.  22.0|
*     Output A = |  .    .   9.0 |
*               |  .    .    .  |
```

6.4.8.4.2.21 ?tbmv

向量与三角带状矩阵乘积。

即：$x=op(A)*x$ 。x为包含n个元素的向量，A为n*n的单元或非单元的三角带状矩阵。

接口定义

C interface：

void cblas_stbmv(const enum CBLAS_ORDER order, const enum CBLAS_UPLO Uplo, const enum CBLAS_TRANSPOSE TransA, const enum CBLAS_DIAG Diag, const BLASINT N, const BLASINT K, const float *A, const BLASINT lda, float *X, const BLASINT incX);

void cblas_dtbmv(const enum CBLAS_ORDER order, const enum CBLAS_UPLO Uplo, const enum CBLAS_TRANSPOSE TransA, const enum CBLAS_DIAG Diag, const BLASINT N, const BLASINT K, const double *A, const BLASINT lda, double *X, const BLASINT incX);

void cblas_ctbmv(const enum CBLAS_ORDER order, const enum CBLAS_UPLO Uplo, const enum CBLAS_TRANSPOSE TransA, const enum CBLAS_DIAG Diag, const BLASINT N, const BLASINT K, const void *A, const BLASINT lda, void *X, const BLASINT incX);

void cblas_ztbmv(const enum CBLAS_ORDER order, const enum CBLAS_UPLO Uplo, const enum CBLAS_TRANSPOSE TransA, const enum CBLAS_DIAG Diag, const BLASINT N, const BLASINT K, const void *A, const BLASINT lda, void *X, const BLASINT incX);

Fortran interface：

CALL STBMV(UPLO, TRANS, DIAG, N, K, A, LDA, X, INCX)

CALL DTBMV(UPLO, TRANS, DIAG, N, K, A, LDA, X, INCX)

CALL CTBMV(UPLO, TRANS, DIAG, N, K, A, LDA, X, INCX)

CALL ZTBMV(UPLO, TRANS, DIAG, N, K, A, LDA, X, INCX)

参数

参数名	类型	描述	输入/输出
order	枚举类型CBLAS_ORDER	表示矩阵是行主序或列主序。	输入

 版权所有 © 华为技术有限公司

参数名	类型	描述	输入/输出
Uplo	枚举类型CBLAS_UPLO	表示矩阵A是上三角或下三角矩阵。 ● 如果Uplo=CblasUpper，则A是上三角矩阵。 ● 如果Uplo=CblasLower，则A是下三角矩阵。	输入
TransA	枚举类型CBLAS_TRANSPOSE	矩阵A为常规矩阵，转置矩阵或共轭矩阵。 ● 如果TransA = CblasNoTrans，$op(A) = A$。 ● 如果TransA = CblasTrans，$op(A) = A'$。 ● 如果TransA = CblasConjTrans，$op(A)=conjg(A)$。 ● 如果TransA = CblasConjTrans，$op(A) = conjg(A')$。	输入
Diag	枚举类型CBLAS_DIAG	表示A是否是单元三角矩阵。 ● 如果Diag=CblasUnit，则A是单元三角矩阵。 ● 如果Diag=CblasNonUnit，则A不是单元三角矩阵。	输入
N	整型数	表示矩阵A的阶数，要求N大于或等于零。	输入
K	整型数	表示矩阵A的超对角阶数，要求K大于或等于零。	输入
A	● 在stbmv中是单精度浮点类型。 ● 在dtbmv中是双精度浮点类型。 ● 在ctbmv中是单精度复数类型。 ● 在ztbmv中是双精度复数类型。	三角带状矩阵A(lda, n)。	输入

版权所有 © 华为技术有限公司

参数名	类型	描述	输入/输出
lda	整型数	矩阵A中每列存储的地址距离，要求lda大于或等于(k+1)。	输入
X	• 在stbmv中是单精度浮点类型。 • 在dtbmv中是双精度浮点类型。 • 在ctbmv中是单精度复数类型。 • 在ztbmv中是双精度复数类型。	向量X，向量规模至少是(1+(N-1)*abs(incX))。	输入/输出
incX	整型数	表示X中向量增长步长，不能为零。	输入

依赖

#include "kblas.h"

示例

C interface：

```
int n = 7, k = 3, lda = 5, incx = 1;
/**
 *       | .   .   .   2.0  2.0  3.0 1.0 |
 *       | .   .   2.0  1.0  2.0  4.0 0.0 |
 *       | .   2.0  2.0  3.0  4.0  2.0 1.0 |
 *  A  =  | 1.0  3.0  3.0  3.0  3.0  3.0 3.0 |
 *       | .   .   .   .   .   .   . |
 */
float a[35] = {0, 0, 0, 1.0, 0,
               0, 0, 2.0, 3.0, 0,
               0, 2.0, 2.0, 3.0, 0,
               2.0, 1.0, 3.0, 3.0, 0,
               2.0, 2.0, 4.0, 3.0, 0,
               3.0, 4.0, 2.0, 3.0, 0,
               1.0, 0.0, 1.0, 3.0, 0};
float x[7] = {2.0, 2.0, 1.0, 2.0, 1.0, 3.0, 4.0};

cblas_stbmv(CblasColMajor,CblasUpper, CblasNoTrans, CblasNonUnit, n, k, a, lda, x, incx);
/**
 * Output x = |12.0, 12.0, 20.0, 26.0, 9.0, 13.0, 12.0|
 */
```

Fortran interface：

```
  INTEGER :: N=7, K=3
  INTEGER :: LDA=5, INCX=1
  REAL(4) :: A(35), X(7)
  DATA A/0, 0, 0, 1.0, 0,
 $       0, 0, 2.0, 3.0, 0,
 $       0, 2.0, 2.0, 3.0, 0,
 $       2.0, 1.0, 3.0, 3.0, 0,
 $       2.0, 2.0, 4.0, 3.0, 0,
```

 版权所有 © 华为技术有限公司

```
     $       3.0, 4.0, 2.0, 3.0, 0,
     $       1.0, 0.0, 1.0, 3.0, 0/
      DATA X/2.0, 2.0, 1.0, 2.0, 1.0, 3.0, 4.0/
      EXTERNAL STBMV
      CALL STBMV('U', 'N', 'N', N, K, A, LDA, X, INCX)

*     Output X = |12.0, 12.0, 20.0, 26.0, 9.0, 13.0, 12.0|
```

6.4.8.4.2.22 ?tbsv

线性方程组求解，系数矩阵为三角带状矩阵。

即：$op(A)*x=b$。b和x为包含n个元素的向量，A为n*n的单元或非单元、上三角或下三角的带状矩阵。

接口定义

C interface：

void cblas_stbsv(const enum CBLAS_ORDER order, const enum CBLAS_UPLO Uplo, const enum CBLAS_TRANSPOSE TransA, const enum CBLAS_DIAG Diag, const blasint N, const blasint K, const float *A, const blasint lda, float *X, const blasint incX);

void cblas_dtbsv(const enum CBLAS_ORDER order, const enum CBLAS_UPLO Uplo, const enum CBLAS_TRANSPOSE TransA, const enum CBLAS_DIAG Diag, const blasint N, const blasint K, const double *A, const blasint lda, double *X, const blasint incX);

void cblas_ctbsv(const enum CBLAS_ORDER order, const enum CBLAS_UPLO Uplo, const enum CBLAS_TRANSPOSE TransA, const enum CBLAS_DIAG Diag, const blasint N, const blasint K, const void *A, const blasint lda, void *X, const blasint incX);

void cblas_ztbsv(const enum CBLAS_ORDER order, const enum CBLAS_UPLOUplo, const enum CBLAS_TRANSPOSE TransA, const enum CBLAS_DIAG Diag, const blasint N, const blasint K, const void* A, const blasint lda, void *X, const blasint incX);

Fortran interface：

CALL STBSV(UPLO, TRANS, DIAG, N, K, A, LDA, X, INCX)

CALL DTBSV(UPLO, TRANS, DIAG, N, K, A, LDA, X, INCX)

CALL CTBSV(UPLO, TRANS, DIAG, N, K, A, LDA, X, INCX)

CALL ZTBSV(UPLO, TRANS, DIAG, N, K, A, LDA, X, INCX)

参数

参数名	类型	描述	输入/输出
order	枚举类型CBLAS_ORDER	表示矩阵是行主序或列主序。	输入

 版权所有 © 华为技术有限公司

参数名	类型	描述	输入/输出
Uplo	枚举类型CBLAS_UPLO	表示矩阵A是上三角或下三角矩阵。 ● 如果Uplo=CblasUpper，则A是上三角矩阵。 ● 如果Uplo=CblasLower，则A是下三角矩阵。	输入
TransA	枚举类型CBLAS_TRANSPOSE	矩阵A为常规矩阵，转置矩阵或共轭矩阵。 ● 如果TransA = CblasNoTrans，$op(A) = A$。 ● 如果TransA = CblasTrans，$op(A) = A'$。 ● 如果TransA = CblasConjTrans，$op(A)=conjg(A)$。 ● 如果TransA = CblasConjTrans，$op(A) = conjg(A')$。	输入
Diag	枚举类型CBLAS_DIAG	表示A是否是单元三角矩阵。 ● 如果Diag=CblasUnit，则A是单元三角矩阵。 ● 如果Diag=CblasNonUnit，则A不是单元三角矩阵。	输入
N	整型数	表示矩阵A的阶数，要求N大于或等于零。	输入
K	整型数	表示矩阵A的超对角阶数，要求K大于或等于零。	输入
A	● 在stbsv中是单精度浮点类型。 ● 在dtbsv中是双精度浮点类型。 ● 在ctbsv中是单精度复数类型。 ● 在ztbsv中是双精度复数类型。	三角带状矩阵A(lda, n)。	输入

版权所有 © 华为技术有限公司

参数名	类型	描述	输入/输出
lda	整型数	矩阵A中每列存储的地址距离，要求lda大于或等于(k+1)。	输入
X	• 在stbsv中是单精度浮点类型。 • 在dtbsv中是双精度浮点类型。 • 在ctbsv中是单精度复数类型。 • 在ztbsv中是双精度复数类型。	向量X，向量规模至少是(1+(N-1)*abs(incX))。	输入/输出
incX	整型数	表示X中向量增长步长，不能为零。	输入

依赖

#include "kblas.h"

示例

C interface：

```
int n = 7, k = 3, lda = 5, incx = 1;
/**
 *       | .   .   .   2.0  2.0  3.0 1.0 |
 *       | .   .   2.0  1.0  2.0  4.0 0.0 |
 *       | .   2.0  2.0  3.0  4.0  2.0 1.0 |
 *  A  =  | 1.0  3.0  3.0  3.0  3.0  3.0 3.0 |
 *       | .   .   .   .   .   .   . |
 */
float a[35] = {0, 0, 0, 1.0, 0,
               0, 0, 2.0, 3.0, 0,
               0, 2.0, 2.0, 3.0, 0,
               2.0, 1.0, 3.0, 3.0, 0,
               2.0, 2.0, 4.0, 3.0, 0,
               3.0, 4.0, 2.0, 3.0, 0,
               1.0, 0.0, 1.0, 3.0, 0};
float x[7] = {12.0, 12.0, 20.0, 26.0, 9.0, 13.0, 12.0};

cblas_stbsv(CblasColMajor, CblasUpper, CblasNoTrans, CblasNonUnit, n, k, a, lda, x, incx);
/**
 * Output x = |2.000000 2.000000 1.000000 2.000000 1.000000 3.000000 4.000000|
 */
```

Fortran interface：

```
  PROGRAM TBMV
  INTEGER :: N=7, K=3
  INTEGER :: LDA=5, INCX=1
  REAL(4) :: A(35), X(7)
  DATA A/0, 0, 0, 1.0, 0,
 $       0, 0, 2.0, 3.0, 0,
 $       0, 2.0, 2.0, 3.0, 0,
 $       2.0, 1.0, 3.0, 3.0, 0,
```

 版权所有 © 华为技术有限公司

```
      $       2.0, 2.0, 4.0, 3.0, 0,
      $       3.0, 4.0, 2.0, 3.0, 0,
      $       1.0, 0.0, 1.0, 3.0, 0/
       DATA X/12.0, 12.0, 20.0, 26.0, 9.0, 13.0, 12.0/
       EXTERNAL STBMV
       CALL STBSV('U', 'N', 'N', N, K, A, LDA, X, INCX)
       WRITE(*,*) X
       END

*      Output X = |2.00000000, 2.0000000, 1.00000000, 2.00000000, 1.00000000, 3.00000000, 4.00000000|
```

6.4.8.4.2.23 ?tpmv

向量与压缩的三角展开矩阵乘积。

即：$x=op(A)*x$ 。x包含n个元素的向量，A为n*n的单元或非单元按照三角展开的矩阵。

接口定义

C interface：

void cblas_stpmv(const enum CBLAS_ORDER order, const enum CBLAS_UPLO Uplo, const enum CBLAS_TRANSPOSE TransA, const enum CBLAS_DIAG Diag, const BLASINT N, const float *Ap, float *X, const BLASINT incX);

void cblas_dtpmv(const enum CBLAS_ORDER order, const enum CBLAS_UPLO Uplo, const enum CBLAS_TRANSPOSE TransA, const enum CBLAS_DIAG Diag, const BLASINT N, const double *Ap, double *X, const BLASINT incX);

void cblas_ctpmv(const enum CBLAS_ORDER order, const enum CBLAS_UPLO Uplo, const enum CBLAS_TRANSPOSE TransA, const enum CBLAS_DIAG Diag, const BLASINT N, const void *Ap, void *X, const BLASINT incX);

void cblas_ztpmv(const enum CBLAS_ORDER order, const enum CBLAS_UPLO Uplo, const enum CBLAS_TRANSPOSE TransA, const enum CBLAS_DIAG Diag, const BLASINT N, const void *Ap, void *X, const BLASINT incX);

Fortran interface：

CALL STPMV(UPLO, TRANS, DIAG, N, AP, X, INCX)

CALL DTPMV(UPLO, TRANS, DIAG, N, AP, X, INCX)

CALL CTPMV(UPLO, TRANS, DIAG, N, AP, X, INCX)

CALL ZTPMV(UPLO, TRANS, DIAG, N, AP, X, INCX)

参数

参数名	类型	描述	输入/输出
order	枚举类型CBLAS_ORDER	表示矩阵是行主序或列主序。	输入

 版权所有 © 华为技术有限公司

参数名	类型	描述	输入/输出
Uplo	枚举类型CBLAS_UPLO	表示使用矩阵A的上三角或下三角。 ● 如果Uplo=CblasUpper，则使用A的上三角部分。 ● 如果Uplo=CblasLower，则使用A的下三角部分。	输入
TransA	枚举类型CBLAS_TRANSPOSE	矩阵A为常规矩阵，转置矩阵或共轭矩阵。 ● 如果TransA = CblasNoTrans，$op(A) = A$。 ● 如果TransA = CblasTrans，$op(A) = A'$。 ● 如果TransA = CblasConjTrans，$op(A)=conjg(A)$。 ● 如果TransA = CblasConjTrans，$op(A) = conjg(A')$。	输入
Diag	● 枚举类型CBLAS_DIAG	表示A是否是单元三角矩阵。 ● 如果Diag=CblasUnit，则A是单元三角矩阵。 ● 如果Diag=CblasNonUnit，则A不是单元三角矩阵。	输入
N	整型数	表示矩阵A的阶数，要求N大于或等于零。	输入
Ap	● 在stpmv中是单精度浮点类型。 ● 在dtpmv中是双精度浮点类型。 ● 在ctpmv中是单精度复数类型。 ● 在ztpmv中是双精度复数类型。	压缩的三角矩阵，矩阵规模至少是(N*(N+1)/2)。	输入

版权所有 © 华为技术有限公司

参数名	类型	描述	输入/输出
X	• 在stpmv中是单精度浮点类型。 • 在dtpmv中是双精度浮点类型。 • 在ctpmv中是单精度复数类型。 • 在ztpmv中是双精度复数类型。	向量X，向量规模至少是(1+(N-1)*abs(incX))。	输入/输出
incX	整型数	表示X中向量增长步长，不能为零。	输入

依赖

#include "kblas.h"

示例

C interface：

```
int  n = 4, incx = 1;
float x[4] = {4.0, 3.0, 2.0, 1.0};
/**
 *        |  1.0 1.0  1.0  3.0 |
 * A   =  |  .   3.0  3.0  2.0 |
 *        |  .    .   2.0  3.0 |
 *        |  .    .    .   1.0 |
 */
float ap[10] = {1.0, 1.0, 3.0, 1.0, 3.0, 2.0, 3.0, 2.0, 3.0, 1.0};

cblas_stpmv(CblasColMajor, CblasUpper, CblasTrans, CblasNonUnit, n, ap, x, incx);
/**
 * Output x = |4.0, 13.0, 17.0, 25.0|
 */
```

Fortran interface：

```
    INTEGER :: N=4, INCX=1
    REAL(4) :: X(4), AP(10)
    DATA X/4.0, 3.0, 2.0, 1.0/
    DATA AP/1.0, 1.0, 3.0, 1.0, 3.0, 2.0, 3.0, 2.0, 3.0, 1.0/
    EXTERNAL STPMV
    CALL STPMV('U', 'T', 'N', N, AP, X, INCX)

*     Output X = |4.0, 13.0, 17.0, 25.0|
```

6.4.8.4.2.24 ?tpsv

线性方程组求解，系数矩阵是压缩的三角矩阵。

即：$op(A)*x=b$ 。b和x为包含n个元素的向量，A为n*n的单元或非单元按照三角展开的矩阵。

 版权所有 © 华为技术有限公司

接口定义

C interface：

void cblas_stpsv(const enum CBLAS_ORDER order, const enum CBLAS_UPLO Uplo, const enum CBLAS_TRANSPOSE TransA, const enum CBLAS_DIAG Diag, const BLASINT N, const float *Ap, float *X, const BLASINT incX);

void cblas_dtpsv(const enum CBLAS_ORDER order, const enum CBLAS_UPLO Uplo, const enum CBLAS_TRANSPOSE TransA, const enum CBLAS_DIAG Diag, const BLASINT N, const double *Ap, double *X, const BLASINT incX);

void cblas_ctpsv(const enum CBLAS_ORDER order, const enum CBLAS_UPLO Uplo, const enum CBLAS_TRANSPOSE TransA, const enum CBLAS_DIAG Diag, const BLASINT N, const void *Ap, void *X, const BLASINT incX);

void cblas_ztpsv(const enum CBLAS_ORDER order, const enum CBLAS_UPLO Uplo, const enum CBLAS_TRANSPOSE TransA, const enum CBLAS_DIAG Diag, const BLASINT N, const void *Ap, void *X, const BLASINT incX);

Fortran interface：

CALL STPSV(UPLO, TRANS, DIAG, N, AP, X, INCX)

CALL DTPSV(UPLO, TRANS, DIAG, N, AP, X, INCX)

CALL CTPSV(UPLO, TRANS, DIAG, N, AP, X, INCX)

CALL ZTPSV(UPLO, TRANS, DIAG, N, AP, X, INCX)

参数

参数名	类型	描述	输入/输出
order	枚举类型CBLAS_ORDER	表示矩阵是行主序或列主序。	输入
Uplo	枚举类型CBLAS_UPLO	表示使用矩阵A存储展开方式上三角或下三角。 • 如果Uplo=CblasUpper，则使用A的上三角部分展开。 • 如果Uplo=CblasLower，则使用A的下三角部分展开。	输入

版权所有 © 华为技术有限公司

参数名	类型	描述	输入/输出
TransA	枚举类型CBLAS_TRANSPOSE	矩阵A为常规矩阵，转置矩阵或共轭矩阵。 ● 如果TransA = CblasNoTrans，$op(A) = A$。 ● 如果TransA = CblasTrans，$op(A) = A'$。 ● 如果TransA = CblasConjTrans，$op(A)=conjg(A)$。 ● 如果TransA = CblasConjTrans，$op(A) = conjg(A')$。	输入
Diag	枚举类型CBLAS_DIAG	表示A是否是单元三角矩阵。 ● 如果Diag=CblasUnit，则A是单元三角矩阵。 ● 如果Diag=CblasNonUnit，则A不是单元三角矩阵。	输入
N	整型数	表示矩阵A的阶数，要求N大于或等于零。	输入
Ap	● 在stpsv中是单精度浮点类型。 ● 在dtpsv中是双精度浮点类型。 ● 在ctpsv中是单精度复数类型。 ● 在ztpsv中是双精度复数类型。	压缩的三角矩阵，矩阵规模至少是(N*(N+1)/2)。	输入
X	● 在stpsv中是单精度浮点类型。 ● 在dtpsv中是双精度浮点类型。 ● 在ctpsv中是单精度复数类型。 ● 在ztpsv中是双精度复数类型。	向量X，向量规模至少是(1+(N-1)*abs(incX))。 获得方程解后更新输出。	输入/输出

 版权所有 © 华为技术有限公司

参数名	类型	描述	输入/输出
incX	整型数	表示X中向量增长步长，不能为零。	输入

依赖

#include "kblas.h"

示例

C interface：

```
int  n = 4,incx = 1;
float x[4] = {4.0, 3.0, 2.0, 1.0};
/**
 *        |  1.0 1.0  1.0  3.0 |
 * A   =  |  .   3.0  3.0  2.0 |
 *        |  .    .   2.0  3.0 |
 *        |  .    .    .   1.0 |
 */
float ap[10] = {1.0, 1.0, 3.0, 1.0, 3.0, 2.0, 3.0, 2.0, 3.0, 1.0};

cblas_stpsv(CblasColMajor, CblasUpper, CblasTrans, CblasNonUnit, n, ap, x, incx);
/**
 * Output x = |4.000000, -0.333333, -0.500000, -8.833333|
 */
```

Fortran interface：

```
    INTEGER :: N=4, INCX=1
    REAL(4) :: X(4), AP(10)
    DATA X/4.0, 3.0, 2.0, 1.0/
    DATA AP/1.0, 1.0, 3.0, 1.0, 3.0, 2.0, 3.0, 2.0, 3.0, 1.0/
    EXTERNAL STPSV
    CALL STPSV('U', 'T', 'N', N, AP, X, INCX)

*     Output X = |4.000000, -0.333333, -0.500000, -8.833333|
```

6.4.8.4.2.25 ?trmv

向量与三角矩阵乘积。

即：$x=op(A)*x$ 。x是n维包含n个元素的向量，A是n*n的单元或非单元三角矩阵。

接口定义

C interface：

void cblas_strmv(const enum CBLAS_ORDER order, const enum CBLAS_UPLO Uplo, const enum CBLAS_TRANSPOSE TransA, const enum CBLAS_DIAG Diag, const BLASINT N, const float *A, const BLASINT lda, float *X, const BLASINT incX);

void cblas_dtrmv(const enum CBLAS_ORDER order, const enum CBLAS_UPLO Uplo, const enum CBLAS_TRANSPOSE TransA, const enum CBLAS_DIAG Diag, const BLASINT N, const double *A, const BLASINT lda, double *X, const BLASINT incX);

 版权所有 © 华为技术有限公司

void cblas_ctrmv(const enum CBLAS_ORDER order, const enum CBLAS_UPLO Uplo, const enum CBLAS_TRANSPOSE TransA, const enum CBLAS_DIAG Diag, const BLASINT N, const void *A, const BLASINT lda, void *X, const BLASINT incX);

void cblas_ztrmv(const enum CBLAS_ORDER order, const enum CBLAS_UPLO Uplo, const enum CBLAS_TRANSPOSE TransA, const enum CBLAS_DIAG Diag, const BLASINT N, const void *A, const BLASINT lda, void *X, const BLASINT incX);

Fortran interface：

CALL STRMV(UPLO, TRANS, DIAG, N, A, LDA, X, INCX)

CALL DTRMV(UPLO, TRANS, DIAG, N, A, LDA, X, INCX)

CALL CTRMV(UPLO, TRANS, DIAG, N, A, LDA, X, INCX)

CALL ZTRMV(UPLO, TRANS, DIAG, N, A, LDA, X, INCX)

参数

参数名	类型	描述	输入/输出
order	枚举类型CBLAS_ORDER	表示矩阵是行主序或列主序。	输入
Uplo	枚举类型CBLAS_UPLO	表示使用矩阵A的上三角或下三角。 ● 如果Uplo=CblasUpper，则使用A的上三角部分。 ● 如果Uplo=CblasLower，则使用A的下三角部分。	输入
TransA	枚举类型CBLAS_TRANSPOSE	矩阵A为常规矩阵，转置矩阵或共轭矩阵。 ● 如果TransA = CblasNoTrans，$op(A) = A$。 ● 如果TransA = CblasTrans，$op(A) = A'$。 ● 如果TransA = CblasConjTrans，$op(A)=conjg(A)$。 ● 如果TransA = CblasConjTrans，$op(A) = conjg(A')$。	输入

版权所有 © 华为技术有限公司

参数名	类型	描述	输入/输出
Diag	枚举类型CBLAS_DIAG	表示A是否是单元三角矩阵。 ● 如果Diag=CblasUnit，则A是单元三角矩阵。 ● 如果Diag=CblasNonUnit，则A不是单元三角矩阵。	输入
N	整型数	表示矩阵A的阶数，要求N大于或等于零。	输入
A	● 在strmv中是单精度浮点类型。 ● 在dtrmv中是双精度浮点类型。 ● 在ctrmv中是单精度复数类型。 ● 在ztrmv中是双精度复数类型。	三角矩阵，矩阵规模至少是lda*N。	输入
lda	整型数	矩阵A中每列存储的地址距离，要求lda大于或等于max(1, N)。	输入
X	● 在strmv中是单精度浮点类型。 ● 在dtrmv中是双精度浮点类型。 ● 在ctrmv中是单精度复数类型。 ● 在ztrmv中是双精度复数类型。	向量X，向量规模至少是(1+(N-1)*abs(incX))。	输入/输出
incX	整型数	表示X中向量增长步长，不能为零。	输入

依赖

#include "kblas.h"

示例

C interface：

```
int  n = 4, incx = 1;
float x[4] = {4.0, 3.0, 2.0, 1.0};
/**
*          | .   2.0  1.0  1.0 |
*   A   =  | .    .   1.0  2.0 |
```

 版权所有 © 华为技术有限公司

```
*         | .    .    .   1.0 |
*         | .    .    .    . |
**/
float a[16] = {0, 0, 0, 0,
               2.0, 0, 0, 0,
               1.0, 1.0, 0, 0,
               1.0, 2.0, 1.0, 0};

cblas_strmv(CblasColMajor, CblasUpper, CblasTrans, CblasUnit, n, a, 4, x, incx);
/**
* Output x = |4.0, 11.0, 9.0, 13.0|
**/
```

Fortran interface：

```
    INTEGER :: N=4, INCX=1
    REAL(4) :: X(4), AP(16)
    DATA X/4.0, 3.0, 2.0, 1.0/
    DATA A/0, 0, 0, 0,
   $       2.0, 0, 0, 0,
   $       1.0, 1.0, 0, 0,
   $       1.0, 2.0, 1.0, 0/
    EXTERNAL STRMV
    CALL STRMV('U', 'T', 'U', N, A, 4, X, INCX)

*     Output X = |4.0, 11.0, 9.0, 13.0|
```

6.4.8.4.2.26 ?trsv

线性方程组求解，系数矩阵是三角矩阵。

即：$op(A)*x=b$ 。x和b为包含n个元素的向量，A是n*n的单元或非单元的三角矩阵。

接口定义

C interface：

void cblas_strsv(const enum CBLAS_ORDER order, const enum CBLAS_UPLO Uplo, const enum CBLAS_TRANSPOSE TransA, const enum CBLAS_DIAG Diag, const BLASINT N, const float *A, const BLASINT lda, float *X, const BLASINT incX);

void cblas_dtrsv(const enum CBLAS_ORDER order, const enum CBLAS_UPLO Uplo, const enum CBLAS_TRANSPOSE TransA, const enum CBLAS_DIAG Diag, const BLASINT N, const double *A, const BLASINT lda, double *X, const BLASINT incX);

void cblas_ctrsv(const enum CBLAS_ORDER order, const enum CBLAS_UPLO Uplo, const enum CBLAS_TRANSPOSE TransA, const enum CBLAS_DIAG Diag, const BLASINT N, const void *A, const BLASINT lda, void *X, const BLASINT incX);

void cblas_ztrsv(const enum CBLAS_ORDER order, const enum CBLAS_UPLO Uplo, const enum CBLAS_TRANSPOSE TransA, const enum CBLAS_DIAG Diag, const BLASINT N, const void *A, const BLASINT lda, void *X, const BLASINT incX);

Fortran interface：

CALL STRSV(UPLO, TRANS, DIAG, N, A, LDA, X, INCX)

CALL DTRSV(UPLO, TRANS, DIAG, N, A, LDA, X, INCX)

CALL CTRSV(UPLO, TRANS, DIAG, N, A, LDA, X, INCX)

 版权所有 © 华为技术有限公司

CALL ZTRSV(UPLO, TRANS, DIAG, N, A, LDA, X, INCX)

参数

参数名	类型	描述	输入/输出
order	枚举类型CBLAS_ORDER	表示矩阵是行主序或列主序。	输入
Uplo	枚举类型CBLAS_UPLO	表示使用矩阵A的上三角或下三角。 ● 如果Uplo=CblasUpper，则使用A的上三角部分。 ● 如果Uplo=CblasLower，则使用A的下三角部分。	输入
TransA	枚举类型CBLAS_TRANSPOSE	矩阵A为常规矩阵，转置矩阵或共轭矩阵。 ● 如果TransA = CblasNoTrans，$op(A) = A$。 ● 如果TransA = CblasTrans，$op(A) = A'$。 ● 如果TransA = CblasConjTrans，$op(A)=conjg(A)$。 ● 如果TransA = CblasConjTrans，$op(A) = conjg(A')$。	输入
Diag	枚举类型CBLAS_DIAG	表示A是否是单元三角矩阵。 ● 如果Diag=CblasUnit，则A是单元三角矩阵。 ● 如果Diag=CblasNonUnit，则A不是单元三角矩阵。	输入
N	整型数	表示矩阵A的阶数，要求N大于或等于零。	输入

版权所有 © 华为技术有限公司

参数名	类型	描述	输入/输出
A	• 在strsv中是单精度浮点类型。 • 在dtrsv中是双精度浮点类型。 • 在ctrsv中是单精度复数类型。 • 在ztrsv中是双精度复数类型。	三角矩阵，矩阵规模至少是lda*N。	输入
lda	整型数	矩阵A中每列存储的地址距离，要求lda大于或等于max(1, N)。	输入
X	• 在strsv中是单精度浮点类型。 • 在dtrsv中是双精度浮点类型。 • 在ctrsv中是单精度复数类型。 • 在ztrsv中是双精度复数类型。	向量X，向量规模至少是(1+(N-1)*abs(incX))。	输入/输出
incX	整型数	表示X中向量增长步长，不能为零。	输入

依赖

#include "kblas.h"

示例

C interface：

```
int  n = 4, lda = 4,incx = 1;
float x[4] = {4.0, 3.0, 2.0, 1.0};
/**
 *        |  1.0 1.0  1.0  3.0 |
 * A   =  |  .   3.0  3.0  2.0 |
 *        |  .    .   2.0  3.0 |
 *        |  .    .    .   1.0 |
 */
float ap[16] = {1.0, 0, 0, 0, 1.0, 3.0, 0, 0, 1.0, 3.0, 2.0, 0, 3.0, 2.0, 3.0, 1.0};

cblas_strsv(CblasColMajor, CblasUpper, CblasTrans, CblasNonUnit, n, ap, lda, x, incx);
/**
 * Output x = |4.000000, -0.333333, -0.500000, -8.833333|
 */
```

Fortran interface：

```
  INTEGER :: N=4, INCX=1
  INTEGER :: LDA=4
```

 版权所有 © 华为技术有限公司

```
      REAL(4) :: X(4), AP(16)
      DATA X/4.0, 3.0, 2.0, 1.0/
      DATA AP/1.0, 0, 0, 0, 1.0, 3.0, 0, 0, 1.0, 3.0, 2.0, 0, 3.0,
     $        2.0, 3.0, 1.0/
      EXTERNAL STRSV
      CALL STRSV('U', 'T', 'N', N, A, LDA, X, INCX)

*     Output X = |4.000000, -0.333333, -0.500000, -8.833333|
```

6.4.8.4.3 KML_BLAS Level 3 函数

6.4.8.4.3.1 函数说明

对KML_BLAS Level3所有函数族进行描述。

函数族	数据类型	描述
?gemm	s，d，c，z	一般矩阵乘矩阵。
?hgemm	h、sh、ch、csh	一般矩阵乘矩阵。
?bgemm	b、sb、cb、csb	一般矩阵乘矩阵。
gemm_?8?8s32	s、u	一般矩阵乘矩阵。
?hemm	c，z	复数埃尔米特矩阵乘矩阵。
?her2k	c，z	复数矩阵乘矩阵秩2更新，矩阵C为复数埃尔米特矩阵。
?herk	c，z	复数矩阵乘矩阵秩k更新，矩阵C为复数埃尔米特矩阵。
?symm	s，d，c，z	实数对称矩阵乘矩阵。
?syrk	s，d，c，z	矩阵乘矩阵秩k更新，矩阵C为对称矩阵。
?syr2k	s，d，c，z	矩阵乘矩阵秩2更新，矩阵C为对称矩阵。
?trmm	s，d，c，z	矩阵乘矩阵，其中一个矩阵为三角矩阵。
?trsm	s，d，c，z	三角矩阵方程求解。

6.4.8.4.3.2 ?gemm

一般矩阵乘矩阵。

即：$C = \text{alpha} * \text{op}(A) * \text{op}(B) + \text{beta} * C$。

op(X)可取值：

$$op(X)=X, op(X)=X^{'}, op(X)=conjg(X), op(X)=conjg(X^{'})$$

，alpha，beta为乘法系数，op(A)为m*k矩阵，op(B)为k*n矩阵，C为m*n矩阵。

版权所有 © 华为技术有限公司

接口定义

C interface：

void cblas_sgemm(const enum CBLAS_ORDER Order, const enum CBLAS_TRANSPOSE TransA, const enum CBLAS_TRANSPOSE TransB, const BLASINT M, const BLASINT N, const BLASINT K, const float alpha, const float *A, const BLASINT lda, const float *B, const BLASINT ldb, const float beta, float *C, const BLASINT ldc);

void cblas_dgemm(const enum CBLAS_ORDER Order, const enum CBLAS_TRANSPOSE TransA, const enum CBLAS_TRANSPOSE TransB, const BLASINT M, const BLASINT N, const BLASINT K, const double alpha, const double *A, const BLASINT lda, const double *B, const BLASINT ldb, const double beta, double *C, const BLASINT ldc);

void cblas_cgemm(const enum CBLAS_ORDER Order, const enum CBLAS_TRANSPOSE TransA, const enum CBLAS_TRANSPOSE TransB, const BLASINT M, const BLASINT N, const BLASINT K, const void *alpha, const void *A, const BLASINT lda, const void *B, const BLASINT ldb, const void *beta, void *C, const BLASINT ldc);

void cblas_zgemm(const enum CBLAS_ORDER Order, const enum CBLAS_TRANSPOSE TransA, const enum CBLAS_TRANSPOSE TransB, const BLASINT M, const BLASINT N, const BLASINT K, const void *alpha, const void *A, const BLASINT lda, const void *B, const BLASINT ldb, const void *beta, void *C, const BLASINT ldc);

Fortran interface：

CALL SGEMM(TRANSA, TRANSB, M, N, K, ALPHA, A, LDA, B, LDB, BETA, C, LDC)

CALL DGEMM(TRANSA, TRANSB, M, N, K, ALPHA, A, LDA, B, LDB, BETA, C, LDC)

CALL CGEMM(TRANSA, TRANSB, M, N, K, ALPHA, A, LDA, B, LDB, BETA, C, LDC)

CALL ZGEMM(TRANSA, TRANSB, M, N, K, ALPHA, A, LDA, B, LDB, BETA, C, LDC)

参数

参数名	类型	描述	输入/输出
order	枚举类型CBLAS_ORDER	表示矩阵是行主序或列主序。	输入

版权所有 © 华为技术有限公司

参数名	类型	描述	输入/输出
TransA	枚举类型CBLAS_TRANSPOSE	矩阵A为常规矩阵，转置矩阵或共轭矩阵。 ● 如果TransA = CblasNoTrans，$op(A) = A$。 ● 如果TransA = CblasTrans，$op(A) = A'$。 ● 如果TransA = CblasConjTrans，$op(A)=conjg(A)$。 ● 如果TransA = CblasConjTrans，$op(A) = conjg(A')$。	输入
TransB	枚举类型CBLAS_TRANSPOSE	矩阵B为常规矩阵，转置矩阵或共轭矩阵。 ● 如果TransB = CblasNoTrans，$op(B) = B$。 ● 如果TransB = CblasTrans，$op(B) = B'$。 ● 如果TransB = CblasConjTrans，$op(B)=conjg(B)$。 ● 如果TransB = CblasConjTrans，$op(B) = conjg(B')$。	输入
M	整型数	矩阵op(A)和矩阵C的行。	输入
N	整型数	矩阵op(B)和矩阵C的列。	输入
K	整型数	矩阵op(A)的列和矩阵op(B)的行。	输入

版权所有 © 华为技术有限公司

参数名	类型	描述	输入/输出
alpha	• 在sgemm中是单精度浮点类型。 • 在dgemm中是双精度浮点类型。 • 在cgemm中是单精度复数类型。 • 在zgemm中是双精度复数类型。	乘法系数。	输入
A	• 在sgemm中是单精度浮点类型。 • 在dgemm中是双精度浮点类型。 • 在cgemm中是单精度复数类型。 • 在zgemm中是双精度复数类型。	矩阵A。	输入
lda	整型数	• 矩阵为列存，TransA = CblasNoTrans，lda至少max(1, m)，否则max(1, k)。 • 矩阵为行存，TransA = CblasNoTrans，lda至少max(1, k)，否则max(1, m)。	输入
B	• 在sgemm中是单精度浮点类型。 • 在dgemm中是双精度浮点类型。 • 在cgemm中是单精度复数类型。 • 在zgemm中是双精度复数类型。	矩阵B。	输入
ldb	整型数	• 矩阵为列存，TransB = CblasNoTrans，ldb至少max(1, k)，否则max(1, n)。 • 矩阵为行存，TransB = CblasNoTrans，ldb至少max(1, n)，否则max(1, k)。	输入

 版权所有 © 华为技术有限公司

参数名	类型	描述	输入/输出
beta	• 在sgemm中是单精度浮点类型。 • 在dgemm中是双精度浮点类型。 • 在cgemm中是单精度复数类型。 • 在zgemm中是双精度复数类型。	乘法系数。	输入
C	• 在sgemm中是单精度浮点类型。 • 在dgemm中是双精度浮点类型。 • 在cgemm中是单精度复数类型。 • 在zgemm中是双精度复数类型。	矩阵C。	输入/输出
ldc	整型数	矩阵为列存，ldc至少max(1, m)，否则max(1, n)。	输入

依赖

#include "kblas.h"

示例

C interface：

```
int m = 4, k = 3, n = 4, lda = 4, ldb = 3, ldc = 4;
float alpha = 1.0, beta = 2.0;
 /*
 * A:
 *     0.340188,       0.411647,       -0.222225,
 *     -0.105617,      -0.302449,      0.053970,
 *     0.283099,       -0.164777,      -0.022603,
 *     0.298440,       0.268230,       0.128871,
 * B:
 *     -0.135216,      0.416195,       -0.358397,      -0.257113,
 *     0.013401,       0.135712,       0.106969,       -0.362768,
 *     0.452230,       0.217297,       -0.483699,      0.304177,
 * C:
 *     -0.343321,      0.498924,       0.112640,       -0.006417,
 *     -0.099056,      -0.281743,      -0.203968,      0.472775,
 *     -0.370210,      0.012932,       0.137552,       -0.207483,
 *     -0.391191,      0.339112,       0.024287,       0.271358,
 */
float a[12] = {0.340188, -0.105617, 0.283099,
               0.298440, 0.411647, -0.302449,
               -0.164777, 0.268230, -0.222225,
               0.053970, -0.022603, 0.128871};
float b[12] = {-0.135216, 0.013401, 0.452230, 0.416195,
```


版权所有 © 华为技术有限公司

```
                0.135712, 0.217297, -0.358397, 0.106969,
                -0.483699, -0.257113, -0.362768, 0.304177};
float c[16] = {-0.343321, -0.099056, -0.370210, -0.391191,
                0.498924, -0.281743, 0.012932, 0.339112,
                0.112640, -0.203968, 0.137552, 0.024287,
                -0.006417, 0.472775, -0.207483, 0.271358};

cblas_sgemm(CblasColMajor,CblasNoTrans,CblasNoTrans, m, n, k, alpha, a, lda, b, ldb, beta, c, ldc);
/*
 * Output C:
 *     -0.827621       1.147010        0.254881        -0.317229
 *     -0.163476       -0.636762       -0.428542       1.098841
 *     -0.791128       0.116416        0.166949        -0.434854
 *     -0.760862       0.866839        -0.092028       0.407877
 *
 */
```

Fortran interface：

```
      INTEGER :: M=4, K=3, N=4
      INTEGER :: LDA=4, LDB=3, LDC=4
      REAL(4) :: ALPHA=1.0, BETA=2.0
      REAL(4) :: A(12), B(12), C(16)
      DATA A/0.340188, -0.105617, 0.283099,
     $       0.298440, 0.411647, -0.302449,
     $       -0.164777, 0.268230, -0.222225,
     $       0.053970, -0.022603, 0.128871/
      DATA B/-0.135216, 0.013401, 0.452230, 0.416195,
     $       0.135712, 0.217297, -0.358397, 0.106969,
     $       -0.483699, -0.257113, -0.362768, 0.304177/
      DATA C/-0.343321, -0.099056, -0.370210, -0.391191,
     $       0.498924, -0.281743, 0.012932, 0.339112,
     $       0.112640, -0.203968, 0.137552, 0.024287,
     $       -0.006417, 0.472775, -0.207483, 0.271358/
      EXTERNAL SGEMM
      CALL SGEMM('N', 'N', M, N, K, ALPHA, A, LDA, B, LDB, BETA, C,
     $           LDC)
*     Output C:
*         -0.827621       1.147010        0.254881        -0.317229
*         -0.163476       -0.636762       -0.428542       1.098841
*         -0.791128       0.116416        0.166949        -0.434854
*         -0.760862       0.866839        -0.092028       0.407877
```

6.4.8.4.3.3 ?hgemm

一般矩阵乘矩阵。

即：$C = \text{alpha} * \text{op}(A) * \text{op}(B) + \text{beta} * C$。

op(X)可取值：

$$op(X)=X, op(X)=X^{T}, op(X)=conjg(X), op(X)=conjg(X^{T})$$

，alpha，beta为乘法系数，op(A)为m*k矩阵，op(B)为k*n矩阵，C为m*n矩阵。

接口定义

C interface：

void cblas_hgemm(const enum CBLAS_ORDER order, const enum CBLAS_TRANSPOSE transA, const enum CBLAS_TRANSPOSE transB, const BLASINT M, const BLASINT N, const BLASINT K, const __fp16 alpha, const __fp16 *A, const BLASINT lda, const __fp16 *B, const BLASINT ldb, const __fp16 beta, __fp16 *C, const BLASINT ldc);

 版权所有 © 华为技术有限公司

void cblas_shgemm(const enum CBLAS_ORDER order, const enum CBLAS_TRANSPOSE transA, const enum CBLAS_TRANSPOSE transB, const BLASINT M, const BLASINT N, const BLASINT K, const float alpha, const __fp16 *A, const BLASINT lda, const __fp16 *B, const BLASINT ldb, const float beta, float *C, const BLASINT ldc);

void cblas_chgemm(const enum CBLAS_ORDER order, const enum CBLAS_TRANSPOSE transA, const enum CBLAS_TRANSPOSE transB, const BLASINT M, const BLASINT N, const BLASINT K, const void *alpha, const void *A, const BLASINT lda, const void *B, const BLASINT ldb, const void *beta, void *C, const BLASINT ldc);

void cblas_cshgemm(const enum CBLAS_ORDER order, const enum CBLAS_TRANSPOSE transA, const enum CBLAS_TRANSPOSE transB, const BLASINT M, const BLASINT N, const BLASINT K, const void *alpha, const void *A, const BLASINT lda, const void *B, const BLASINT ldb, const void *beta, void *C, const BLASINT ldc);

Fortran interface：

CALL HGEMM(TRANSA, TRANSB, M, N, K, ALPHA, A, LDA, B, LDB, BETA, C, LDC)

CALL SHGEMM(TRANSA, TRANSB, M, N, K, ALPHA, A, LDA, B, LDB, BETA, C, LDC)

CALL CHGEMM(TRANSA, TRANSB, M, N, K, ALPHA, A, LDA, B, LDB, BETA, C, LDC)

CALL CSHGEMM(TRANSA, TRANSB, M, N, K, ALPHA, A, LDA, B, LDB, BETA, C, LDC)

参数

参数名	类型	描述	输入/输出
order	枚举类型CBLAS_ORDER	表示矩阵是行主序或列主序。	输入
TransA	枚举类型CBLAS_TRANSPOSE	矩阵A为常规矩阵，转置矩阵或共轭矩阵。 ● 如果TransA = CblasNoTrans，$op(A) = A$。 ● 如果TransA = CblasTrans，$op(A) = A'$。 ● 如果TransA = CblasConjTrans，$op(A)=conjg(A)$。 ● 如果TransA = CblasConjTrans，$op(A) = conjg(A')$。	输入

 版权所有 © 华为技术有限公司

参数名	类型	描述	输入/输出
TransB	枚举类型CBLAS_TRANSPOSE	矩阵B为常规矩阵，转置矩阵或共轭矩阵。 ● 如果TransB = CblasNoTrans，$op(B) = B$。 ● 如果TransB = CblasTrans，$op(B) = B'$。 ● 如果TransB = CblasConjTrans，$op(B)=conjg(B)$。 ● 如果TransB = CblasConjTrans，$op(B) = conjg(B')$。	输入
M	整型数	矩阵op(A)和矩阵C的行。	输入
N	整型数	矩阵op(B)和矩阵C的列。	输入
K	整型数	矩阵op(A)的列和矩阵op(B)的行。	输入
alpha	● 在hgemm中是半精度浮点类型。 ● 在shgemm中是单精度浮点类型。 ● 在chgemm中是半精度复数类型。 ● 在cshgemm中是单精度复数类型。	乘法系数。	输入
A	● 在hgemm中是半精度浮点类型。 ● 在shgemm中是半精度浮点类型。 ● 在chgemm中是半精度复数类型。 ● 在cshgemm中是半精度复数类型。	矩阵A。	输入

版权所有 © 华为技术有限公司

参数名	类型	描述	输入/输出
lda	整型数	• 矩阵为列存，TransA = CblasNoTrans，lda至少max(1, m)，否则max(1, k)。 • 矩阵为行存，TransA = CblasNoTrans，lda至少max(1, k)，否则max(1, m)。	输入
B	• 在hgemm中是半精度浮点类型。 • 在shgemm中是半精度浮点类型。 • 在chgemm中是半精度复数类型。 • 在cshgemm中是半精度复数类型。	矩阵B。	输入
ldb	整型数	• 矩阵为列存，TransB = CblasNoTrans，ldb至少max(1, k)，否则max(1, n)。 • 矩阵为行存，TransB = CblasNoTrans，ldb至少max(1, n)，否则max(1, k)。	输入
beta	• 在hgemm中是半精度浮点类型。 • 在shgemm中是单精度浮点类型。 • 在chgemm中是半精度复数类型。 • 在cshgemm中是单精度复数类型。	乘法系数。	输入
C	• 在hgemm中是半精度浮点类型。 • 在shgemm中是单精度浮点类型。 • 在chgemm中是半精度复数类型。 • 在cshgemm中是单精度复数类型。	矩阵C。	输入/输出
ldc	整型数	矩阵为列存，ldc至少max(1, m)，否则max(1, n)。	输入

 版权所有 © 华为技术有限公司

依赖

#include "kblas.h"

示例

C interface：

```
int m = 4, k = 3, n = 4, lda = 4, ldb = 3, ldc = 4;
__fp16 alpha = 1.0, beta = 2.0;
 /*
 * A:
 *     0.340188,       0.411647,       -0.222225,
 *     -0.105617,      -0.302449,      0.053970,
 *     0.283099,       -0.164777,      -0.022603,
 *     0.298440,       0.268230,       0.128871,
 * B:
 *     -0.135216,      0.416195,       -0.358397,      -0.257113,
 *     0.013401,       0.135712,       0.106969,       -0.362768,
 *     0.452230,       0.217297,       -0.483699,      0.304177,
 * C:
 *     -0.343321,      0.498924,       0.112640,       -0.006417,
 *     -0.099056,      -0.281743,      -0.203968,      0.472775,
 *     -0.370210,      0.012932,       0.137552,       -0.207483,
 *     -0.391191,      0.339112,       0.024287,       0.271358,
 */
__fp16 a[12] = {0.340188, -0.105617, 0.283099,
                0.298440, 0.411647, -0.302449,
                -0.164777, 0.268230, -0.222225,
                0.053970, -0.022603, 0.128871};
__fp16 b[12] = {-0.135216, 0.013401, 0.452230, 0.416195,
                0.135712, 0.217297, -0.358397, 0.106969,
                -0.483699, -0.257113, -0.362768, 0.304177};
__fp16 c[16] = {-0.343321, -0.099056, -0.370210, -0.391191,
                0.498924, -0.281743, 0.012932, 0.339112,
                0.112640, -0.203968, 0.137552, 0.024287,
                -0.006417, 0.472775, -0.207483, 0.271358};

cblas_hgemm(CblasColMajor,CblasNoTrans,CblasNoTrans, m, n, k, alpha, a, lda, b, ldb, beta, c, ldc);
/*
 * Output C:
 *     -0.827621       1.147010        0.254881        -0.317229
 *     -0.163476       -0.636762       -0.428542       1.098841
 *     -0.791128       0.116416        0.166949        -0.434854
 *     -0.760862       0.866839        -0.092028       0.407877
 *
 */
```

Fortran interface：

```
  INTEGER :: M=4, K=3, N=4
  INTEGER :: LDA=4, LDB=3, LDC=4
  REAL(4) :: ALPHA=1.0, BETA=2.0
  REAL(4) :: A(12), B(12), C(16)
  DATA A/0.340188, -0.105617, 0.283099,
 $       0.298440, 0.411647, -0.302449,
 $       -0.164777, 0.268230, -0.222225,
 $       0.053970, -0.022603, 0.128871/
  DATA B/-0.135216, 0.013401, 0.452230, 0.416195,
 $       0.135712, 0.217297, -0.358397, 0.106969,
 $       -0.483699, -0.257113, -0.362768, 0.304177/
  DATA C/-0.343321, -0.099056, -0.370210, -0.391191,
 $       0.498924, -0.281743, 0.012932, 0.339112,
 $       0.112640, -0.203968, 0.137552, 0.024287,
 $       -0.006417, 0.472775, -0.207483, 0.271358/
```

 版权所有 © 华为技术有限公司

```
      EXTERNAL HGEMM
      CALL HGEMM('N', 'N', M, N, K, ALPHA, A, LDA, B, LDB, BETA, C,
     $           LDC)
*     Output C:
*          -0.827621       1.147010        0.254881       -0.317229
*          -0.163476      -0.636762       -0.428542        1.098841
*          -0.791128       0.116416        0.166949       -0.434854
*          -0.760862       0.866839       -0.092028        0.407877
```

6.4.8.4.3.4 ?bgemm

一般矩阵乘矩阵。

即：$C = \text{alpha} * \text{op}(A) * \text{op}(B) + \text{beta} * C$。

op(X)可取值：

$$op(X)=X, op(X)=X^{T}, op(X)=conjg(X), op(X)=conjg(X^{T})$$

，alpha，beta为乘法系数，op(A)为m*k矩阵，op(B)为k*n矩阵，C为m*n矩阵。

接口定义

C interface：

void cblas_bgemm(const enum CBLAS_ORDER order, const enum CBLAS_TRANSPOSE transA, const enum CBLAS_TRANSPOSE transB, const BLASINT M, const BLASINT N, const BLASINT K, const __bf16 alpha, const __bf16 *A, const BLASINT lda, const __bf16 *B, const BLASINT ldb, const __bf16 beta, __bf16 *C, const BLASINT ldc);

void cblas_sbgemm(const enum CBLAS_ORDER order, const enum CBLAS_TRANSPOSE transA, const enum CBLAS_TRANSPOSE transB, const BLASINT M, const BLASINT N, const BLASINT K, const float alpha, const __bf16 *A, const BLASINT lda, const __bf16 *B, const BLASINT ldb, const float beta, float *C, const BLASINT ldc);

void cblas_cbgemm(const enum CBLAS_ORDER order, const enum CBLAS_TRANSPOSE transA, const enum CBLAS_TRANSPOSE transB, const BLASINT M, const BLASINT N, const BLASINT K, const void *alpha, const void *A, const BLASINT lda, const void *B, const BLASINT ldb, const void *beta, void *C, const BLASINT ldc);

void cblas_csbgemm(const enum CBLAS_ORDER order, const enum CBLAS_TRANSPOSE transA, const enum CBLAS_TRANSPOSE transB, const BLASINT M, const BLASINT N, const BLASINT K, const void *alpha, const void *A, const BLASINT lda, const void *B, const BLASINT ldb, const void *beta, void *C, const BLASINT ldc);

Fortran interface：

CALL BGEMM(TRANSA, TRANSB, M, N, K, ALPHA, A, LDA, B, LDB, BETA, C, LDC)

CALL SBGEMM(TRANSA, TRANSB, M, N, K, ALPHA, A, LDA, B, LDB, BETA, C, LDC)

CALL CBGEMM(TRANSA, TRANSB, M, N, K, ALPHA, A, LDA, B, LDB, BETA, C, LDC)

CALL CSBGEMM(TRANSA, TRANSB, M, N, K, ALPHA, A, LDA, B, LDB, BETA, C, LDC)

 版权所有 © 华为技术有限公司

参数

参数名	类型	描述	输入/输出
order	枚举类型CBLAS_ORDER	表示矩阵是行主序或列主序。	输入
TransA	枚举类型CBLAS_TRANSPOSE	矩阵A为常规矩阵，转置矩阵或共轭矩阵。 ● 如果TransA = CblasNoTrans，$op(A) = A$。 ● 如果TransA = CblasTrans，$op(A) = A'$。 ● 如果TransA = CblasConjTrans，$op(A)=conjg(A)$。 ● 如果TransA = CblasConjTrans，$op(A) = conjg(A')$。	输入
TransB	枚举类型CBLAS_TRANSPOSE	矩阵B为常规矩阵，转置矩阵或共轭矩阵。 ● 如果TransB = CblasNoTrans，$op(B) = B$。 ● 如果TransB = CblasTrans，$op(B) = B'$。 ● 如果TransB = CblasConjTrans，$op(B)=conjg(B)$。 ● 如果TransB = CblasConjTrans，$op(B) = conjg(B')$。	输入
M	整型数	矩阵op(A)和矩阵C的行。	输入
N	整型数	矩阵op(B)和矩阵C的列。	输入
K	整型数	矩阵op(A)的列和矩阵op(B)的行。	输入

版权所有 © 华为技术有限公司

参数名	类型	描述	输入/输出
alpha	• 在bgemm中是半精度浮点类型。 • 在sbgemm中是单精度浮点类型。 • 在cbgemm中是半精度复数类型。 • 在csbgemm中是单精度复数类型。	乘法系数。	输入
A	• 在bgemm中是半精度浮点类型。 • 在sbgemm中是半精度浮点类型。 • 在cbgemm中是半精度复数类型。 • 在csbgemm中是半精度复数类型。	矩阵A。	输入
lda	整型数	• 矩阵为列存，TransA = CblasNoTrans，lda至少max(1, m)，否则max(1, k)。 • 矩阵为行存，TransA = CblasNoTrans，lda至少max(1, k)，否则max(1, m)。	输入
B	• 在bgemm中是半精度浮点类型。 • 在sbgemm中是半精度浮点类型。 • 在cbgemm中是半精度复数类型。 • 在csbgemm中是半精度复数类型。	矩阵B。	输入
ldb	整型数	• 矩阵为列存，TransB = CblasNoTrans，ldb至少max(1, k)，否则max(1, n)。 • 矩阵为行存，TransB = CblasNoTrans，ldb至少max(1, n)，否则max(1, k)。	输入

版权所有 © 华为技术有限公司

参数名	类型	描述	输入/输出
beta	• 在bgemm中是半精度浮点类型。 • 在sbgemm中是单精度浮点类型。 • 在cbgemm中是半精度复数类型。 • 在csbgemm中是单精度复数类型。	乘法系数。	输入
C	• 在bgemm中是半精度浮点类型。 • 在sbgemm中是单精度浮点类型。 • 在cbgemm中是半精度复数类型。 • 在csbgemm中是单精度复数类型。	矩阵C。	输入/输出
ldc	整型数	矩阵为列存，ldc至少max(1, m)，否则max(1, n)。	输入

依赖

#include "kblas.h"

示例

C interface：

```
int m = 4, k = 3, n = 4, lda = 4, ldb = 3, ldc = 4;
__bf16 alpha = 1.0, beta = 2.0;
 /*
 * A:
 *     0.340188,       0.411647,       -0.222225,
 *     -0.105617,      -0.302449,      0.053970,
 *     0.283099,       -0.164777,      -0.022603,
 *     0.298440,       0.268230,       0.128871,
 * B:
 *     -0.135216,      0.416195,       -0.358397,      -0.257113,
 *     0.013401,       0.135712,       0.106969,       -0.362768,
 *     0.452230,       0.217297,       -0.483699,      0.304177,
 * C:
 *     -0.343321,      0.498924,       0.112640,       -0.006417,
 *     -0.099056,      -0.281743,      -0.203968,      0.472775,
 *     -0.370210,      0.012932,       0.137552,       -0.207483,
 *     -0.391191,      0.339112,       0.024287,       0.271358,
 */
__bf16 a[12] = {0.340188, -0.105617, 0.283099,
                0.298440, 0.411647, -0.302449,
                -0.164777, 0.268230, -0.222225,
                0.053970, -0.022603, 0.128871};
__bf16 b[12] = {-0.135216, 0.013401, 0.452230, 0.416195,
```


版权所有 © 华为技术有限公司

```
                0.135712, 0.217297, -0.358397, 0.106969,
                -0.483699, -0.257113, -0.362768, 0.304177};
__bf16 c[16] = {-0.343321, -0.099056, -0.370210, -0.391191,
                0.498924, -0.281743, 0.012932, 0.339112,
                0.112640, -0.203968, 0.137552, 0.024287,
                -0.006417, 0.472775, -0.207483, 0.271358};

cblas_bgemm(CblasColMajor,CblasNoTrans,CblasNoTrans, m, n, k, alpha, a, lda, b, ldb, beta, c, ldc);
/*
 * Output C:
 *     -0.827621       1.147010        0.254881        -0.317229
 *     -0.163476       -0.636762       -0.428542       1.098841
 *     -0.791128       0.116416        0.166949        -0.434854
 *     -0.760862       0.866839        -0.092028       0.407877
 *
 */
```

Fortran interface：

```
      INTEGER :: M=4, K=3, N=4
      INTEGER :: LDA=4, LDB=3, LDC=4
      REAL(4) :: ALPHA=1.0, BETA=2.0
      REAL(4) :: A(12), B(12), C(16)
      DATA A/0.340188, -0.105617, 0.283099,
     $       0.298440, 0.411647, -0.302449,
     $       -0.164777, 0.268230, -0.222225,
     $       0.053970, -0.022603, 0.128871/
      DATA B/-0.135216, 0.013401, 0.452230, 0.416195,
     $       0.135712, 0.217297, -0.358397, 0.106969,
     $       -0.483699, -0.257113, -0.362768, 0.304177/
      DATA C/-0.343321, -0.099056, -0.370210, -0.391191,
     $       0.498924, -0.281743, 0.012932, 0.339112,
     $       0.112640, -0.203968, 0.137552, 0.024287,
     $       -0.006417, 0.472775, -0.207483, 0.271358/
      EXTERNAL BGEMM
      CALL BGEMM('N', 'N', M, N, K, ALPHA, A, LDA, B, LDB, BETA, C,
     $           LDC)
*     Output C:
*         -0.827621       1.147010        0.254881        -0.317229
*         -0.163476       -0.636762       -0.428542       1.098841
*         -0.791128       0.116416        0.166949        -0.434854
*         -0.760862       0.866839        -0.092028       0.407877
```

6.4.8.4.3.5 gemm_?8?8s32

一般矩阵乘矩阵。

即：

$$C=alpha*(op(A)+A_offset)*(op(B)+B_offset)+beta*C+C_offset$$

。

op(X)可取值：$op(X)=X,op(X)=X^T$，alpha，beta为乘法系数，op(A)为m*k矩阵，op(B)为k*n矩阵，C为m*n矩阵。

A_offset为一个标量，类型固定为int8_t，op(A)中所有元素加上A_offset；

B_offset为一个标量，类型固定为int8_t，op(B)中所有元素加上B_offset；

C_offset有三种定义（元素类型均为int32_t）：

- 一个标量，C矩阵的每一个元素都加上C_offset
- 一个行向量，C矩阵的每一行都加上这个行向量C_offset

 版权所有 © 华为技术有限公司

- 一个列向量，C矩阵的每一列都加上这个列向量C_offset

接口定义

C interface：

void cblas_gemm_s8s8s32(

const CBLAS_LAYOUT layout, const CBLAS_TRANSPOSE transa, const CBLAS_TRANSPOSE transb, const CBLAS_OFFSET offsetc,

const BLASINT m, const BLASINT n, const BLASINT k, const float alpha,

const BLASINT8 *a, const BLASINT lda, const BLASINT8 oa,

const BLASINT8 *b, const BLASINT ldb, const BLASINT8 ob,

const float beta, int32_t *c, const BLASINT ldc, const int32_t *oc);

void cblas_gemm_u8u8s32(

const CBLAS_LAYOUT layout, const CBLAS_TRANSPOSE transa, const CBLAS_TRANSPOSE transb, const CBLAS_OFFSET offsetc,

const BLASINT m, const BLASINT n, const BLASINT k, const float alpha,

const BLASUINT8 *a, const BLASINT lda, const BLASINT8 oa,

const BLASUINT8 *b, const BLASINT ldb, const BLASINT8 ob,

const float beta, int32_t *c, const BLASINT ldc, const int32_t *oc);

void cblas_gemm_s8u8s32(

const CBLAS_LAYOUT layout, const CBLAS_TRANSPOSE transa, const CBLAS_TRANSPOSE transb, const CBLAS_OFFSET offsetc,

const BLASINT m, const BLASINT n, const BLASINT k, const float alpha,

const BLASINT8 *a, const BLASINT lda, const BLASINT8 oa,

const BLASUINT8 *b, const BLASINT ldb, const BLASINT8 ob,

const float beta, int32_t *c, const BLASINT ldc, const int32_t *oc);

void cblas_gemm_u8s8s32(

const CBLAS_LAYOUT layout, const CBLAS_TRANSPOSE transa, const CBLAS_TRANSPOSE transb, const CBLAS_OFFSET offsetc,

const BLASINT m, const BLASINT n, const BLASINT k, const float alpha,

const BLASUINT8 *a, const BLASINT lda, const BLASINT8 oa,

const BLASINT8 *b, const BLASINT ldb, const BLASINT8 ob,

const float beta, int32_t *c, const BLASINT ldc, const int32_t *oc);

Fortran interface：

int8 gemm无Fortran接口。

 版权所有 © 华为技术有限公司

参数

参数名	类型	描述	输入/输出
layout	枚举类型CBLAS_LAYOUT	表示矩阵是行主序或列主序。 ● 如果layout = CblasRowMajor，则代表行主序。 ● 如果layout = CblasColMajor，则代表列主序。	输入
transa	枚举类型CBLAS_TRANSPOSE	矩阵A为常规矩阵，转置矩阵。 ● 如果TransA = CblasNoTrans，$op(A) = A$。 ● 如果TransA = CblasTrans，$op(A) = A'$。	输入
transb	枚举类型CBLAS_TRANSPOSE	矩阵B为常规矩阵，转置矩阵。 ● 如果TransB = CblasNoTrans，$op(B) = B$。 ● 如果TransB = CblasTrans，$op(B) = B'$。	输入
offsetc	枚举类型CBLAS_OFFSET	表示参数oc（GEMM定义中的C_offset）为行向量/列向量/标量。 ● 如果offsetc = CblasRowOffset，则代表C_offset是一个行向量。 ● 如果offsetc = CblasColOffset，则代表C_offset是一个列向量。 ● 如果offsetc = CblasFixOffset，则代表C_offset是一个标量。	输入
M	整型数	矩阵op(A)和矩阵C的行。	输入
N	整型数	矩阵op(B)和矩阵C的列。	输入
K	整型数	矩阵op(A)的列和矩阵op(B)的行。	输入

版权所有 © 华为技术有限公司

参数名	类型	描述	输入/输出
alpha	单精度浮点类型。	乘法系数。	输入
A	• 在gemm_s8s8s32中是int8_t类型。 • 在gemm_u8u8s32中是uint8_t类型。 • 在gemm_s8u8s32中是int8_t类型。 • 在gemm_u8s8s32中是uint8_t类型。	矩阵A。	输入
lda	整型数	• 矩阵为列存，TransA = CblasNoTrans，lda至少max(1, m)，否则max(1, k)。 • 矩阵为行存，TransA = CblasNoTrans，lda至少max(1, k)，否则max(1, m)。	输入
oa	int8_t类型	GEMM定义中的A_offset的值	输入
B	• 在gemm_s8s8s32中是int8_t类型。 • 在gemm_u8u8s32中是uint8_t类型。 • 在gemm_s8u8s32中是uint8_t类型。 • 在gemm_u8s8s32中是int8_t类型。	矩阵B。	输入
ldb	整型数	• 矩阵为列存，TransB = CblasNoTrans，ldb至少max(1, k)，否则max(1, n)。 • 矩阵为行存，TransB = CblasNoTrans，ldb至少max(1, n)，否则max(1, k)。	输入
ob	int8_t类型	GEMM定义中的B_offset的值	输入
beta	单精度浮点类型	乘法系数。	输入
C	int32_t类型	矩阵C。	输入/输出
ldc	整型数	矩阵为列存，ldc至少max(1, m)，否则max(1, n)。	输入

版权所有 © 华为技术有限公司

参数名	类型	描述	输入/输出
oc	int32_t类型	GEMM定义中的C_offset的值 ● 当offsetc的值为CblasRowOffset时，此输入代表行向量。 ● 当offsetc的值为CblasColOffset时，此输入代表列向量。 ● 当offsetc的值为CblasFixOffset时，此输入代表标量。	输入

依赖

#include "kblas.h"

示例

C interface：

```
int m = 4, k = 3, n = 4, lda = 4, ldb = 3, ldc = 4;
float alpha = 1.0, beta = 2.0;

int8_t oa=3, ob=5;
CBLAS_OFFSET offsetc = CblasRowOffset;
int32_t oc[] = {1, 2, 3, 4};

 /*
 * A:
 *    35,     39,     27,
 *    38,     55,     40,
 *    46,     35,     41,
 *    54,     55,     24,
 * B:
 *    35,      54,      35,      40,
 *    38,      39,      55,      41,
 *    46,      55,      27,      24,
 * C:
 *    7,     7,     7,     7,
 *    7,     7,     7,     7,
 *    7,     7,     7,     7,
 *    7,     7,     7,     7,
 */
int8_t a[12] = {35, 38, 46, 54,
                39, 55, 35, 55,
                27, 40, 41, 24};
int8_t b[12] = {35, 38, 46,
                54, 39, 55,
                35, 55, 27,
                40, 41, 24};
int32_t c[16] = {7, 7, 7, 7,
                 7, 7, 7, 7,
                 7, 7, 7, 7,
                 7, 7, 7, 7};

cblas_gemm_s8s8s32(CblasColMajor, CblasNoTrans, CblasNoTrans, offsetc,
    m, n, k, alpha,
    a, lda, oa,
```

 版权所有 © 华为技术有限公司

```
    b, ldb, ob,
    beta, c, ldc, oc);
/*
 * Output C:
 *     4871 5906 5017 4530
 *     6342 7567 6513 5778
 *     5853 7219 5665 5247
 *     6166 7551 6641 6034
 *
 */
```

6.4.8.4.3.6 ?hemm

复数埃尔米特矩阵乘矩阵。

即：$C = \text{alpha} * A * B + \text{beta} * C$ 或 $C = alpha * B * A + \text{beta} * C$。

alpha，beta为乘法系数，A为埃尔米特矩阵，B和C为m*n一般矩阵。

接口定义

C interface：

void cblas_chemm(const enum CBLAS_ORDER Order, const enum CBLAS_SIDE Side, const enum CBLAS_UPLO Uplo, const BLASINT M, const BLASINT N, const void *alpha, const void *A, const BLASINT lda, const void *B, const BLASINT ldb, const void *beta, void *C, const BLASINT ldc);

void cblas_zhemm(const enum CBLAS_ORDER Order, const enum CBLAS_SIDE Side, const enum CBLAS_UPLO Uplo, const BLASINT M, const BLASINT N, const void *alpha, const void *A, const BLASINT lda, const void *B, const BLASINT ldb, const void *beta, void *C, const BLASINT ldc);

Fortran interface：

CALL CHEMM(SIDE, UPLO, M, N, ALPHA, A, LDA, B, LDB, BETA, C, LDC)

CALL ZHEMM(SIDE, UPLO, M, N, ALPHA, A, LDA, B, LDB, BETA, C, LDC)

参数

参数名	类型	描述	输入/输出
order	枚举类型CBLAS_ORDER	表示矩阵是行主序或列主序。	输入
Side	枚举类型CBLAS_SIDE	埃尔米特矩阵A在矩阵B的左边还是右边。	输入
Uplo	枚举类型CBLAS_UPLO	表示使用矩阵A的上三角或下三角。 ● 如果Uplo=CblasUpper，则使用A的上三角部分。 ● 如果Uplo=CblasLower，则使用A的下三角部分。	输入

版权所有 © 华为技术有限公司

参数名	类型	描述	输入/输出
M	整型数	矩阵C的行。 ● A*B为矩阵A的行。 ● B*A为矩阵B的行。	输入
N	整型数	矩阵C的列。 ● A*B为矩阵B的列。 ● B*A为矩阵A的列。	输入
alpha	● 在chemm中是单精度复数类型。 ● 在zhemm中是双精度复数类型。	乘法系数。	输入
A	● 在chemm中是单精度复数类型。 ● 在zhemm中是双精度复数类型。	埃尔米特矩阵A(lda, ka)。 如果Side = CblasLeft，ka = m，否则ka = n。	输入
lda	整型数	如果Side = CblasLeft，lda至少max(1, m)，否则至少max(1, n)。	输入
B	● 在chemm中是单精度复数类型。 ● 在zhemm中是双精度复数类型。	矩阵B(ldb, n)。	输入
ldb	整型数	● 矩阵为列存，ldb至少max(1, m)。 ● 矩阵为行存，ldb至少max(1, n)。	输入
beta	● 在chemm中是单精度复数类型。 ● 在zhemm中是双精度复数类型。	乘法系数。	输入
C	● 在chemm中是单精度复数类型。 ● 在zhemm中是双精度复数类型。	矩阵C(ldc, n)。	输入/输出
ldc	整型数	● 矩阵为列存，ldc至少max(1, m)。 ● 矩阵为行存，ldc至少max(1, n)。	输入

 版权所有 © 华为技术有限公司

依赖

#include "kblas.h"

示例

C interface：

```
int m = 2, n = 3, lda = 4, ldb =3, ldc = 5;
float alpha[2] = {2.0, 3.0}, beta[2] = {1.0, 6.0};
/**
 *
 *              |  (2.0,  . )        .          .      |
 *       A    =  |  (13.0, -2.0)  (2.0,  . )     .      |
 *              | (-2.0, -6.0)   (2.0, 14.0)   (2.0,  . ) |
 *              |      .             .          .      |
 *
 *              | (2.0, 2.0)   (-13.0, 12.0)    (3.0, 3.0)  |
 *       B    =  | (2.0, 6.0)    (4.0, -5.0)   (-11.0, -4.0) |
 *              |     .              .          .      |
 *
 *              |  (3.0, 6.0)     (-1.0, 6.0)   (1.0, 7.0)  |
 *              | (-11.0, 8.0)    (1.0, 1.0)   (-14.0, 2.0) |
 *       C    =  |       .             .          .      |
 *              |       .             .          .      |
 *              |       .             .          .      |
 *
 */
float a[24] = {2.0, 0, 13.0, -2.0, -2.0, -6.0, 0, 0,
              0, 0, 2.0, 0, 2.0, 14.0, 0, 0,
              0, 0, 0, 0,2.0, 0, 0, 0};
float b[18] = {2.0, 2.0, 2.0, 6.0, 0, 0,
              -13.0, 12.0, 4.0, -5.0, 0, 0,
              3.0, 3.0, -11.0, -4.0, 0, 0};
float c[30] = {3.0,6.0,-11.0,8.0,0,0,0,0,0,0,
              -1.0,6.0,1.0,1.0,0,0,0,0,0,0,
              1.0,7.0,-14.0,2.0,0,0,0,0,0,0};

cblas_chemm(CblasColMajor,CblasRight,CblasLower, m, n, alpha, a, lda, b, ldb, beta, c, ldc);
/**
 * Output C
 *              | (-777.0,  -39.0)   (-423.0, 84.0)   (-437.0, 849.0) |
 *              | (-10.0, 100.0)    (337.0,  -5.0)   (-52.0, -602.0)  |
 *       C    =  |       .               .               .       |
 *              |       .               .               .       |
 *              |       .               .               .       |
 *
 */
```

Fortran interface：

```
    INTEGER :: M=2, N=3
    INTEGER :: LDA=4, LDB=3, LDC=5
    COMPLEX(4) :: ALPHA=(2.0, 3.0), BETA=(1.0, 6.0)
    COMPLEX(4) :: A(4, 3), B(3, 3), C(5, 3)
    DATA A/(2.0, 0), (13.0, -2.0), (-2.0, -6.0), (0, 0),
   $       (0, 0), (2.0, 0), (2.0, 14.0), (0, 0),
   $       (0, 0), (0, 0), (2.0, 0), (0, 0)/
    DATA B/(2.0, 2.0), (2.0, 6.0), (0, 0),
   $       (-13.0, 12.0), (4.0, -5.0), (0, 0),
   $       (3.0, 3.0), (-11.0, -4.0), (0, 0)/
    DATA C/(3.0, 6.0), (-11.0, 8.0), (0, 0), (0, 0), (0, 0),
   $       (-1.0, 6.0), (1.0, 1.0), (0, 0), (0, 0), (0, 0),
   $       (1.0, 7.0), (-14.0, 2.0), (0, 0), (0, 0), (0, 0)/
    EXTERNAL CHEMM
    CALL CHEMM('R', 'L', M, N, ALPHA, A, LDA, B, LDB, BETA, C, LDC)

*     Output C
```

 版权所有 © 华为技术有限公司

```
*               | (-777.0,  -39.0)   (-423.0, 84.0)   (-437.0, 849.0) |
*               | (-10.0, 100.0)     (337.0,  -5.0)   (-52.0, -602.0)  |
*          C    = |       .              .              .       |
*               |       .              .              .       |
*               |       .              .              .       |
```

6.4.8.4.3.7 ?her2k

复数矩阵乘矩阵秩2更新，矩阵C为复数埃尔米特矩阵。

即：$C = \mathrm{alpha} * A * B^H + \mathrm{conjg}(\mathrm{alpha}) * B * A^H + \mathrm{beta} * C$ 或 $C = \mathrm{alpha} * A^H * B + \mathrm{conjg}(\mathrm{alpha}) * B^H * A + \mathrm{beta} * C$。

alpha，beta为乘法系数，C为n*n的埃尔米特矩阵；第一种情况矩阵A，B为n*k，第二种情况矩阵A，B为k*n。

接口定义

C interface：

void cblas_cher2k(const enum CBLAS_ORDER Order, const enum CBLAS_UPLO Uplo, const enum CBLAS_TRANSPOSE Trans, const BLASINT N, const BLASINT K, const void *alpha, const void *A, const BLASINT lda, const void *B, const BLASINT ldb, const float beta, void *C, const BLASINT ldc);

void cblas_zher2k(const enum CBLAS_ORDER Order, const enum CBLAS_UPLO Uplo, const enum CBLAS_TRANSPOSE Trans, const BLASINT N, const BLASINT K, const void *alpha, const void *A, const BLASINT lda, const void *B, const BLASINT ldb, const double beta, void *C, const BLASINT ldc);

Fortran interface：

CALL CHER2K(UPLO, TRANS, N, K, ALPHA, A, LDA, B, LDB, BETA, C, LDC)

CALL ZHER2K(UPLO, TRANS, N, K, ALPHA, A, LDA, B, LDB, BETA, C, LDC)

参数

参数名	类型	描述	输入/输出
order	枚举类型CBLAS_ORDER	表示矩阵是行主序或列主序。	输入
Uplo	枚举类型CBLAS_UPLO	矩阵C存储方式。	输入
Trans	枚举类型CBLAS_TRANSPOSE	矩阵A和B为常规矩阵，转置矩阵或共轭矩阵。 ● 当Trans = CblasNoTrans，$C = \mathrm{alpha} * A * B^H + \mathrm{conj}(\mathrm{alpha}) * B * A^H + \mathrm{beta} * C$。 ● 当Trans = CblasConjTrans，$C = \mathrm{alpha} * A^H * B + \mathrm{conj}(\mathrm{alpha}) * B^H * A + \mathrm{beta} * C$。	输入
N	整型数	矩阵C的行。	输入

版权所有 © 华为技术有限公司

参数名	类型	描述	输入/输出
K	整型数	当Trans = CblasNoTrans，表示矩阵A、B的列。 当Trans = CblasConjTrans，表示矩阵A、B的行。	输入
alpha	• 在cher2k中是单精度复数类型。 • 在zher2k中是双精度复数类型。	乘法系数。	输入
A	• 在cher2k中是单精度复数类型。 • 在zher2k中是双精度复数类型。	矩阵A(lda, ka)。 如果Trans = CblasNoTrans，ka = k，否则ka = n。	输入
lda	整型数	• 矩阵为列存，如果Trans = CblasNoTrans，lda至少max(1, n), 否则至少max(1, k)。 • 矩阵为列存，如果Trans = CblasNoTrans，lda至少max(1, k), 否则至少max(1, n)。	输入
B	• 在cher2k中是单精度复数类型。 • 在zher2k中是双精度复数类型。	矩阵B(ldb, kb)。 Trans = CblasNoTrans，kb = k，否则kb = n。	输入
ldb	整型数	• 矩阵为列存，如果Trans = CblasNoTrans，ldb至少max(1, n)，否则至少max(1, k)。 • 矩阵为列存，如果Trans = CblasNoTrans，ldb至少max(1, k)，否则至少max(1, n)。	输入
beta	• 在cher2k中是单精度复数类型。 • 在zher2k中是双精度复数类型。	乘法系数。	输入
C	• 在cher2k中是单精度复数类型。 • 在zher2k中是双精度复数类型。	矩阵C(ldc, n)。	输入/输出

版权所有 © 华为技术有限公司

参数名	类型	描述	输入/输出
ldc	整型数	ldc至少max(1, n)。	输入

依赖

#include "kblas.h"

示例

C interface：

```
int n = 2, lda = 2, k = 2, ldb =3, ldc = 3;
float alpha[2] = {2.0, 1.0}, beta = 2.0;
/**
 *                  | 4.0 1.0   1.0 2.0 |
 *     A(2 * 2)    =  | 3.0 2.0   4.0 1.0 |
 *
 *     B(2 * 2)    =  | 2.0 1.0   3.0  2.0 |
 *                  | 11.0 2.0 -2.0  1.0 |
 *
 *                  |  3.0 1.0  2.0  1.0 |
 *     C(3 * 2)    =  | -1.0 1.0  -6.0 2.0 |
 *                  |  .           .    |
 */
float a[8] = {4.0, 1.0, 3.0, 2.0, 1.0, 2.0, 4.0, 1.0};
float b[8] = {2.0, 1.0, 11.0, 2.0, 3.0, 2.0, -2.0, 1.0};
float c[12] = {3.0, 1.0, -1.0, 1.0, 0, 0, 2.0, 1.0, -6.0, 2.0, 0, 0};

cblas_cher2k(CblasColMajor, CblasUpper, CblasConjTrans, n, k, alpha, a, lda, b, ldb, beta, c, ldc);
/**
 * Output C
 *                 | (218.0, 0.0)   (123.0, -27.0) |
 *    C(3 * 2)    =  | (-1.0 1.0)     (46.0, 0.0)    |
 *                 |  .                .        |
 *
 */
```

Fortran interface：

```
      INTEGER :: N=2, K=2
      INTEGER :: LDA=2, LDB=3, LDC=3
      COMPLEX(4) :: ALPHA=(2.0, 1.0)
      REAL(4) :: BETA=2.0
      COMPLEX(4) :: A(2, 2), B(2, 2), C(3, 2)
      DATA A/(4.0, 1.0), (3.0, 2.0), (1.0, 2.0), (4.0, 1.0)/
      DATA B/(2.0, 1.0), (11.0, 2.0), (3.0, 2.0), (-2.0, 1.0)/
      DATA C/(3.0, 1.0), (-1.0, 1.0), (0, 0), (2.0, 1.0), (-6.0, 2.0),
     $       (0, 0)/
      EXTERNAL CHER2K
      CALL CHER2K('U', 'C', N, K, ALPHA, A, LDA, B, LDB, BETA, C, LDC)

*     Output C
*                    | (218.0, 0.0)   (123.0, -27.0) |
*        C(3 * 2)    =  | (-1.0 1.0)     (46.0, 0.0)    |
*                    |  .                .        |
```

6.4.8.4.3.8 ?herk

复数矩阵乘矩阵秩k更新，矩阵C为复数埃尔米特矩阵。

即：$C = \text{alpha} * A * A^H + \text{beta} * C$ 或 $C = \text{alpha} * A^H * A + \text{beta} * C$。

 版权所有 © 华为技术有限公司

alpha，beta为实乘法系数，C为n*n的埃尔米特矩阵；第一种情况矩阵A为n*k，第二种情况矩阵A为k*n。

接口定义

C interface：

void cblas_cherk(const enum CBLAS_ORDER Order, const enum CBLAS_UPLO Uplo, const enum CBLAS_TRANSPOSE Trans, const BLASINT N, const BLASINT K, const float alpha, const void *A, const BLASINT lda, const float beta, void *C, const BLASINT ldc);

void cblas_zherk(const enum CBLAS_ORDER Order, const enum CBLAS_UPLO Uplo, const enum CBLAS_TRANSPOSE Trans, const BLASINT N, const BLASINT K, const double alpha, const void *A, const BLASINT lda, const double beta, void *C, const BLASINT ldc);

Fortran interface：

CALL CHERK(UPLO, TRANS, N, K, ALPHA, A, LDA, BETA, C, LDC)

CALL ZHERK(UPLO, TRANS, N, K, ALPHA, A, LDA, BETA, C, LDC)

参数

参数名	类型	描述	输入/输出
order	枚举类型CBLAS_ORDER	表示矩阵是行主序或列主序。	输入
Uplo	枚举类型CBLAS_UPLO	矩阵C存储方式。	输入
Trans	枚举类型CBLAS_TRANSPOSE	矩阵A为常规矩阵或共轭矩阵。 ● 当Trans = CblasNoTrans，$C = \text{alpha} * A * A^H + \text{beta} * C$。 ● 当Trans = CblasConjTrans，$C = \text{alpha} * A^H * A + \text{beta} * C$。	输入
N	整型数	矩阵C的阶数。	输入
K	整型数	● 当Trans = CblasNoTrans，矩阵A的列。 ● 当Trans = CblasConjTrans，矩阵A的行。	输入
alpha	● 在cherk中是单精度实数类型。 ● 在zherk中是双精度实数类型。	乘法系数。	输入

 版权所有 © 华为技术有限公司

参数名	类型	描述	输入/输出
A	• 在cherk中是单精度复数类型。 • 在zherk中是双精度复数类型。	矩阵A(lda, ka)。 如果Trans = CblasNoTrans，ka = k，否则ka = n。	输入
lda	整型数	• 矩阵为列存，Trans = CblasNoTrans，lda至少max(1, n)，否则至少max(1, k)。 • 矩阵为行存，Trans = CblasNoTrans，lda至少max(1, k)，否则至少max(1, n)。	输入
beta	• 在cherk中是单精度实数类型。 • 在zherk中是双精度类型。	乘法系数。	输入
C	• 在cherk中是单精度复数类型。 • 在zherk中是双精度复数类型。	矩阵C(ldc, n)。	输入/输出
ldc	整型数	ldc至少max(1, n)。	输入

依赖

#include "kblas.h"

示例

C interface：

```
int n = 2, lda = 2, k = 2, ldc = 2;
float alpha = 2.0, beta = 2.0;
/**
 *                  | 4.0 1.0   1.0 2.0 |
 *     A(2 * 2)    = | 3.0 2.0   4.0 1.0 |
 *
 *                  |  3.0 1.0  2.0  1.0 |
 *     C(2 * 2)    = | -1.0 1.0  -6.0 2.0 |
 */
float a[8] = {4.0, 1.0, 3.0, 2.0, 1.0, 2.0, 4.0, 1.0};
float c[8] = {3.0, 1.0, -1.0, 1.0, 2.0, 1.0, -6.0, 2.0};

cblas_cherk(CblasColMajor, CblasUpper, CblasConjTrans, n, k, alpha, a, lda, beta, c, ldc);
/**
 * Output C
 *                 | (66.0, 0.0)   (44.0, 6.0) |
 *    C(2 * 2)    = | (-1.0, 1.0)   (32.0, 0.0) |
 *
 */
```


版权所有 © 华为技术有限公司

Fortran interface：

```
    INTEGER :: N=2, K=2
    INTEGER :: LDA=2, LDC=2
    REAL(4) :: ALPHA=2.0
    REAL(4) :: BETA=2.0
    COMPLEX(4) :: A(2, 2), C(2, 2)
    DATA A/(4.0, 1.0), (3.0, 2.0), (1.0, 2.0), (4.0, 1.0)/
    DATA B/(2.0, 1.0), (11.0, 2.0), (3.0, 2.0), (-2.0, 1.0)/
    DATA C/(3.0, 1.0), (-1.0, 1.0), (2.0, 1.0), (-6.0, 2.0)/
    EXTERNAL CHERK
    CALL CHERK('U', 'C', N, K, ALPHA, A, LDA, BETA, C, LDC)

*    Output C
*                | (66.0, 0.0)   (44.0, 6.0) |
*    C(2 * 2)    =  | (-1.0, 1.0)   (32.0, 0.0) |
*
```

6.4.8.4.3.9 ?symm

实数对称矩阵乘矩阵。

即：$C = \text{alpha} * A * B + \text{beta} * C$ 或 $C = \text{alpha} * B * A + \text{beta} * C$。

alpha，beta为乘法系数，A为对称矩阵，B，C为m*n一般矩阵。

接口定义

C interface：

void cblas_ssymm(const enum CBLAS_ORDER Order, const enum CBLAS_SIDE Side, const enum CBLAS_UPLO Uplo, const BLASINT M, const BLASINT N, const float alpha, const float *A, const BLASINT lda, const float *B, const BLASINT ldb, const float beta, float *C, const BLASINT ldc);

void cblas_dsymm(const enum CBLAS_ORDER Order, const enum CBLAS_SIDE Side, const enum CBLAS_UPLO Uplo, const BLASINT M, const BLASINT N, const double alpha, const double *A, const BLASINT lda, const double *B, const BLASINT ldb, const double beta, double *C, const BLASINT ldc);

void cblas_csymm(const enum CBLAS_ORDER Order, const enum CBLAS_SIDE Side, const enum CBLAS_UPLO Uplo, const BLASINT M, const BLASINT N, const void *alpha, const void *A, const BLASINT lda, const void *B, const BLASINT ldb, const void *beta, void *C, const BLASINT ldc);

void cblas_zsymm(const enum CBLAS_ORDER Order, const enum CBLAS_SIDE Side, const enum CBLAS_UPLO Uplo, const BLASINT M, const BLASINT N, const void *alpha, const void *A, const BLASINT lda, const void *B, const BLASINT ldb, const void *beta, void *C, const BLASINT ldc);

Fortran interface：

CALL SSYMM(SIDE, UPLO, M, N, ALPHA, A, LDA, B, LDB, BETA, C, LDC)

CALL DSYMM(SIDE, UPLO, M, N, ALPHA, A, LDA, B, LDB, BETA, C, LDC)

CALL CSYMM(SIDE, UPLO, M, N, ALPHA, A, LDA, B, LDB, BETA, C, LDC)

CALL ZSYMM(SIDE, UPLO, M, N, ALPHA, A, LDA, B, LDB, BETA, C, LDC)

 版权所有 © 华为技术有限公司

参数

参数名	类型	描述	输入/输出
order	枚举类型CBLAS_ORDER	表示矩阵是行主序或列主序。	输入
Side	枚举类型CBLAS_SIDE	对称矩阵A在矩阵B的左边还是右边。 ● Side = CblasLeft，$C = alpha * A * B + beta * C$。 ● Side = CblasRight，$C = alpha * B * A + beta * C$。	输入
Uplo	枚举类型CBLAS_UPLO	表示使用矩阵A的上三角或下三角。 ● 如果Uplo=CblasUpper，则使用A的上三角部分。 ● 如果Uplo=CblasLower，则使用A的下三角部分。	输入
M	整型数	矩阵C的行 ● A*B为矩阵A的行。 ● B*A为矩阵B的行。	输入
N	整型数	矩阵C的列 ● A*B为矩阵B的列。 ● B*A为矩阵A的列。	输入
alpha	● 在ssymm中是单精度浮点类型。 ● 在dsymm中是双精度浮点类型。 ● 在csymm中是单精度复数类型。 ● 在zsymm中是双精度复数类型。	乘法系数。	输入
A	● 在ssymm中是单精度浮点类型。 ● 在dsymm中是双精度浮点类型。 ● 在csymm中是单精度复数类型。 ● 在zsymm中是双精度复数类型。	矩阵A(lda, ka)。 如果Side = CblasLeft，ka = m，否则ka = n。	输入

版权所有 © 华为技术有限公司

参数名	类型	描述	输入/输出
lda	整型数	如果Side = CblasLeft，lda至少max(1, m)，否则至少max(1, n)。	输入
B	• 在ssymm中是单精度浮点类型。 • 在dsymm中是双精度浮点类型。 • 在csymm中是单精度复数类型。 • 在zsymm中是双精度复数类型。	矩阵B(ldb, n)。	输入
ldb	整型数	• 矩阵为列存，ldb至少max(1, m)。 • 矩阵为行存，ldb至少max(1, n)。	输入
beta	• 在ssymm中是单精度浮点类型。 • 在dsymm中是双精度浮点类型。 • 在csymm中是单精度复数类型。 • 在zsymm中是双精度复数类型。	乘法系数。	输入
C	• 在ssymm中是单精度浮点类型。 • 在dsymm中是双精度浮点类型。 • 在csymm中是单精度复数类型。 • 在zsymm中是双精度复数类型。	矩阵C。	输入/输出
ldc	整型数	• 矩阵为列存，ldc至少max(1, m)。 • 矩阵为行存，ldc至少max(1, n)。	输入

依赖

#include "kblas.h"

C interface：

 版权所有 © 华为技术有限公司

```
int m = 3, n = 6, lda = 4, ldb =3, ldc = 5;
float alpha = 2.0, beta = 2.0;
/**
 *                | 4.0    .     . |
 *    A(4 * 3)    =  | 3.0   4.0    . |
 *                | 6.0  -7.0  -1.0 |
 *                |  .     .     . |
 *
 *                 | -1.0  -3.0   2.0   2.0  -1.0    8.0 |
 *     B(3 * 6)    =  | 2.0   3.0   0.0   4.0   2.0    -2.0 |
 *                 | 11.0  -2.0  -21.0  -1.0  -1.0   5.0 |
 *
 *                 |  3.0   2.0  -1.0  3.0   -1.0  -11.0|
 *                 |  9.0  13.0  5.0   5.0   3.0   -5.0 |
 *     C(5 * 6)    =  | -1.0  -6.0  -3.0  3.0   -1.0   32.0|
 *                 |  .     .     .     .     .      . |
 *                 |  .     .     .     .     .      . |
 */
float a[12] = {4.0, 3.0, 6.0, 0,
               3.0, 4.0, -7.0, 0,
               6.0, -7.0, -1.0, 0};
float b[18] = {-1.0, 2.0, 11.0,
               -3.0, 3.0, -2.0,
               2.0, 0, -21.0,
               2.0, 4.0, -1.0,
               -1.0, 2.0, -1.0,
               8.0, -2.0, 5.0};
float c[30] = {6.0, 9.0, -1.0, 0, 0,
               2.0, 13.0, -6.0, 0, 0,
               -1.0, 5.0, -3.0, 0, 0,
               3.0, 5.0, 3.0, 0, 0,
               -1.0, 3.0, -1.0, 0, 0,
               -11.0, -5.0, 32.0, 0, 0};

cblas_ssymm(CblasColMajor,CblasLeft,CblasLower, m, n, alpha, a, lda, b, ldb, beta, c, ldc);
/**
 * Output C
 *                |  148.0  -26.0 -238.0  34.0  -10.0  90.0 |
 *                | -126.0   60.0  316.0  68.0  30.0  -48.0 |
 *    C(5 * 6)    =  | -64.0   -86.0  60.0  -24.0  -40.0  178.0|
 *                |  .      .      .      .      .      .   |
 *                |  .      .      .      .      .      .   |
 *
 */
```

Fortran interface:

```
      INTEGER :: M=3, N=6
      INTEGER :: LDA=4, LDB=3, LDC=5
      REAL(4) :: ALPHA=2.0, BETA=2.0
      REAL(4) :: A(4, 3), B(3,6), C(5, 6)
      DATA A/4.0, 3.0, 6.0, 0,
     $       3.0, 4.0, -7.0, 0,
     $       6.0, -7.0, -1.0, 0/
      DATA B/-1.0, 2.0, 11.0,
     $       -3.0, 3.0, -2.0,
     $       2.0, 0, -21.0,
     $       2.0, 4.0, -1.0,
     $       -1.0, 2.0, -1.0,
     $       8.0, -2.0, 5.0/
      DATA C/6.0, 9.0, -1.0, 0, 0,
     $       2.0, 13.0, -6.0, 0, 0,
     $       -1.0, 5.0, -3.0, 0, 0,
     $       3.0, 5.0, 3.0, 0, 0,
     $       -1.0, 3.0, -1.0, 0, 0,
     $       -11.0, -5.0, 32.0, 0, 0/
      EXTERNAL SSYMM
      CALL SSYMM('L', 'L', M, N, ALPHA, A, LDA, B, LDB, BETA, C, LDC)

*     Output C
```

 版权所有 © 华为技术有限公司

```
*                  |  148.0  -26.0 -238.0  34.0  -10.0  90.0 |
*                  | -126.0   60.0  316.0  68.0  30.0  -48.0 |
*       C(5 * 6)   =  | -64.0   -86.0  60.0  -24.0  -40.0  178.0|
*                  |  .     .     .     .     .     .  |
*                  |  .     .     .     .     .     .  |
```

6.4.8.4.3.10 ?syrk

矩阵乘矩阵秩k更新，矩阵C为对称矩阵。

即：$C = alpha*A*A' + beta*C$ 或 $C = alpha*A'*A + beta*C$。

alpha，beta为乘法系数，C为n*n的对称矩阵；第一种情况矩阵A为n*k，第二种情况矩阵A为k*n一般矩阵。

接口定义

C interface：

void cblas_ssyrk(const enum CBLAS_ORDER Order, const enum CBLAS_UPLO Uplo, const enum CBLAS_TRANSPOSE Trans, const BLASINT N, const BLASINT K, const float alpha, const float *A, const BLASINT lda, const float beta, float *C, const BLASINT ldc);

void cblas_dsyrk(const enum CBLAS_ORDER Order, const enum CBLAS_UPLO Uplo, const enum CBLAS_TRANSPOSE Trans, const BLASINT N, const BLASINT K, const double alpha, const double *A, const BLASINT lda, const double beta, double *C, const BLASINT ldc);

void cblas_csyrk(const enum CBLAS_ORDER Order, const enum CBLAS_UPLO Uplo, const enum CBLAS_TRANSPOSE Trans, const BLASINT N, const BLASINT K, const void *alpha, const void *A, const BLASINT lda, const void *beta, void *C, const BLASINT ldc);

void cblas_zsyrk(const enum CBLAS_ORDER Order, const enum CBLAS_UPLO Uplo, const enum CBLAS_TRANSPOSE Trans, const BLASINT N, const BLASINT K, const void *alpha, const void *A, const BLASINT lda, const void *beta, void *C, const BLASINT ldc);

Fortran interface：

CALL SSYRK(UPLO, TRANS, N, K, ALPHA, A, LDA, BETA, C, LDC)

CALL DSYRK(UPLO, TRANS, N, K, ALPHA, A, LDA, BETA, C, LDC)

CALL CSYRK(UPLO, TRANS, N, K, ALPHA, A, LDA, BETA, C, LDC)

CALL ZSYRK(UPLO, TRANS, N, K, ALPHA, A, LDA, BETA, C, LDC)

参数

参数名	类型	描述	输入/输出
order	枚举类型CBLAS_ORDER	表示矩阵是行主序或列主序。	输入
Uplo	枚举类型CBLAS_UPLO	矩阵C存储方式。	输入

 版权所有 © 华为技术有限公司

参数名	类型	描述	输入/输出
Trans	枚举类型CBLAS_TRANSPOSE	矩阵A和C为常规矩阵、转置矩阵或共轭矩阵。 ● 当Trans = CblasNoTrans，$C = \alpha * A * A^T + \alpha * A * A^T + \beta * C$。 ● 当Trans = CblasTrans/CblasConjTrans，$C = \alpha * A^T * A + \alpha * A^T * A + \beta * C$。	输入
N	整型数	矩阵C的主序长度。	输入
K	整型数	当Trans = CblasNoTrans，K为矩阵A的列，否则为矩阵A的行。	输入
alpha	● 在ssyrk中是单精度浮点类型。 ● 在dsyrk中是双精度浮点类型。 ● 在csyrk中是单精度复数类型。 ● 在zsyrk中是双精度复数类型。	乘法系数。	输入
A	● 在ssyrk中是单精度浮点类型。 ● 在dsyrk中是双精度浮点类型。 ● 在csyrk中是单精度复数类型。 ● 在zsyrk中是双精度复数类型。	矩阵A(lda, ka)。 当Trans = CblasNoTrans, ka = k，否则ka = n。	输入
lda	整型数	● 矩阵为列存，Trans = CblasNoTrans，lda至少max(1, n)，否则至少max(1, k)。 ● 矩阵为行存，Trans = CblasNoTrans，lda至少max(1, k)，否则至少max(1, n)。	输入

版权所有 © 华为技术有限公司

参数名	类型	描述	输入/输出
beta	• 在ssyrk中是单精度浮点类型。 • 在dsyrk中是双精度浮点类型。 • 在csyrk中是单精度复数类型。 • 在zsyrk中是双精度复数类型。	乘法系数。	输入
C	• 在ssyrk中是单精度浮点类型。 • 在dsyrk中是双精度浮点类型。 • 在csyrk中是单精度复数类型。 • 在zsyrk中是双精度复数类型。	对称矩阵C。	输入/输出
ldc	整型数	ldc至少max(1, n)。	输入

依赖

#include "kblas.h"

示例

C interface：

```
int n = 3, lda = 3, k = 2,  ldc = 3;
float alpha = 2.0, beta = 2.0;
/**
 *                | 4.0   1.0  |
 *    A(3 * 2)    =  | 3.0   4.0  |
 *                | .     .   |
 *
 *                 |  3.0   2.0  .|
 *     C(3 * 3)    =  | -1.0  -6.0  .|
 *                 |  .     .   .|
 */
float a[6] = {4.0, 3.0, 0, 1.0, 4.0, 0};
float c[9] = {3.0, -1.0, 0, 2.0, -6.0, 0, 0, 0, 0};

cblas_ssyrk(CblasColMajor, CblasUpper, CblasNoTrans, n, k, alpha, a, lda, beta, c, ldc);
/**
 * Output C
 *                |  40.0  36.0  .|
 *    C(3 * 3)    =  |  -1.0  38.0  .|
 *                |  .      .   .|
 *
 */
```

Fortran interface：

 版权所有 © 华为技术有限公司

```
      INTEGER :: N=3, K=2
      INTEGER :: LDA=3, LDC=3
      REAL(4) :: ALPHA=2.0, BETA=2.0
      REAL(4) :: A(3, 2), C(3, 3)
      DATA A/4.0, 3.0, 0, 1.0, 4.0, 0/
      DATA C/3.0, -1.0, 0, 2.0, -6.0, 0/
      EXTERNAL SSYRK
      CALL SSYRK('U', 'N', N, K, ALPHA, A, LDA, BETA, C, LDC)

*     Output C
*                   |  40.0  36.0  .|
*       C(3 * 3)    =  |  30.0  38.0  .|
*                   |   .     .   .|
```

6.4.8.4.3.11 ?syr2k

矩阵乘矩阵秩2更新，矩阵C为对称矩阵。

即：$C = \mathrm{alpha} * A * B' + \mathrm{alpha} * B * A' + \mathrm{beta} * C$ 或 $C = alpha * A' * B + alpha * B' * A + \mathrm{beta} * C$。

alpha，beta为乘法系数，C为n*n的对称矩阵；第一种情况矩阵A，B为n*k，第二种情况矩阵A，B为k*n一般矩阵。

接口定义

C interface：

void cblas_ssyr2k(const enum CBLAS_ORDER Order, const enum CBLAS_UPLO Uplo, const enum CBLAS_TRANSPOSE Trans, const BLASINT N, const BLASINT K, const float alpha, const float *A, const BLASINT lda, const float *B, const BLASINT ldb, const float beta, float *C, const BLASINT ldc);

void cblas_dsyr2k(const enum CBLAS_ORDER Order, const enum CBLAS_UPLO Uplo, const enum CBLAS_TRANSPOSE Trans, const BLASINT N, const BLASINT K, const double alpha, const double *A, const BLASINT lda, const double *B, const BLASINT ldb, const double beta, double *C, const BLASINT ldc);

void cblas_csyr2k(const enum CBLAS_ORDER Order, const enum CBLAS_UPLO Uplo, const enum CBLAS_TRANSPOSE Trans, const BLASINT N, const BLASINT K, const void *alpha, const void *A, const BLASINT lda, const void *B, const BLASINT ldb, const void *beta, void *C, const BLASINT ldc);

void cblas_zsyr2k(const enum CBLAS_ORDER Order, const enum CBLAS_UPLO Uplo, const enum CBLAS_TRANSPOSE Trans, const BLASINT N, const BLASINT K, const void *alpha, const void *A, const BLASINT lda, const void *B, const BLASINT ldb, const void *beta, void *C, const BLASINT ldc);

Fortran interface：

CALL SSYR2K(UPLO, TRANS, N, K, ALPHA, A, LDA, B, LDB, BETA, C, LDC)

CALL DSYR2K(UPLO, TRANS, N, K, ALPHA, A, LDA, B, LDB, BETA, C, LDC)

CALL CSYR2K(UPLO, TRANS, N, K, ALPHA, A, LDA, B, LDB, BETA, C, LDC)

CALL ZSYR2K(UPLO, TRANS, N, K, ALPHA, A, LDA, B, LDB, BETA, C, LDC)

 版权所有 © 华为技术有限公司

参数

参数名	类型	描述	输入/输出
order	枚举类型CBLAS_ORDER	表示矩阵是行主序或列主序。	输入
Uplo	枚举类型CBLAS_UPLO	矩阵C存储方式。	输入
Trans	枚举类型CBLAS_TRANSPOSE	矩阵A和B为常规矩阵，转置矩阵或共轭矩阵。 ● 当Trans = CblasNoTrans，$C = \alpha * A * B^T + \alpha * B * A^T + \beta * C$。 ● 当Trans = CblasTrans/CblasConjTrans，$C = \alpha * A^T * B + \alpha * B^T * A + \beta * C$。	输入
N	整型数	矩阵C的主序长度。	输入
K	整型数	当Trans = CblasNoTrans，K为矩阵A和B的列，否则为矩阵A和B的行。	输入
alpha	● 在ssyr2k中是单精度浮点类型。 ● 在dsyr2k中是双精度浮点类型。 ● 在csyr2k中是单精度复数类型。 ● 在zsyr2k中是双精度复数类型。	乘法系数。	输入
A	● 在ssyr2k中是单精度浮点类型。 ● 在dsyr2k中是双精度浮点类型。 ● 在csyr2k中是单精度复数类型。 ● 在zsyr2k中是双精度复数类型。	矩阵A(lda, ka)。 当Trans = CblasNoTrans，ka = k，否则ka = n。	输入
lda	整型数	● 矩阵为列存，Trans = CblasNoTrans，lda至少max(1, n)，否则至少max(1, k)。 ● 矩阵为行存，Trans = CblasNoTrans，lda至少max(1, k)，否则至少max(1, n)。	输入

版权所有 © 华为技术有限公司

参数名	类型	描述	输入/输出
B	• 在ssyr2k中是单精度浮点类型。 • 在dsyr2k中是双精度浮点类型。 • 在csyr2k中是单精度复数类型。 • 在zsyr2k中是双精度复数类型。	矩阵B(ldb, kb)。 当Trans = CblasNoTrans，kb = k，否则kb = n。	输入
ldb	整型数	• 矩阵为列存，Trans = CblasNoTrans，ldb至少max(1, n)，否则至少max(1, k)。 • 矩阵为行存，Trans = CblasNoTrans，ldb至少max(1, k)，否则至少max(1, n)。	输入
beta	• 在ssyr2k中是单精度浮点类型。 • 在dsyr2k中是双精度浮点类型。 • 在csyr2k中是单精度复数类型。 • 在zsyr2k中是双精度复数类型。	乘法系数。	输入
C	• 在ssyr2k中是单精度浮点类型。 • 在dsyr2k中是双精度浮点类型。 • 在csyr2k中是单精度复数类型。 • 在zsyr2k中是双精度复数类型。	对称矩阵C。	输入/输出
ldc	整型数	ldc至少max(1, n)。	输入

依赖

#include "kblas.h"

示例

C interface：

 版权所有 © 华为技术有限公司

```
int n = 2, lda = 3, k = 2, ldb =3, ldc = 3;
float alpha = 2.0, beta = 2.0;
/**
 *                | 4.0   1.0  |
 *    A(3 * 2)    =  | 3.0   4.0  |
 *                | .     .   |
 *
 *     B(2 * 2)    =  | 2.0   3.0  |
 *                  | 11.0  -2.0 |
 *
 *                  |  3.0   2.0 |
 *     C(3 * 2)    =  | -1.0  -6.0 |
 *                  |  .     .  |
 */
float a[6] = {4.0, 3.0, 0, 1.0, 4.0, 0};
float b[4] = {2.0, 11.0, 3.0, -2.0};
float c[6] = {3.0, -1.0, 0, 2.0, -6.0, 0};

cblas_ssyr2k(CblasColMajor, CblasUpper, CblasNoTrans, n, k, alpha, a, lda, b, ldb, beta, c, ldc);
/**
 * Output C
 *                |  30.0  96.0 |
 *    C(3 * 2)    =  | -1.0   184.0 |
 *                |  .      .   |
 *
 */
```

Fortran interface：

```
    INTEGER :: N=2, K=2
    INTEGER :: LDA=3, LDB=3, LDC=3
    REAL(4) :: ALPHA=2.0, BETA=2.0
    REAL(4) :: A(3, 2), B(2, 2), C(3, 3)
    DATA A/4.0, 3.0, 0, 1.0, 4.0, 0/
    DATA C/3.0, -1.0, 0, 2.0, -6.0, 0/
    EXTERNAL SSYR2K
    CALL SSYR2K('U', 'N', N, K, ALPHA, A, LDA, B, LDB, BETA, C, LDC)

*    Output C
*                  |  30.0  96.0 |
*       C(3 * 2)    =  | -1.0   184.0 |
*                  |  .      .   |
```

6.4.8.4.3.12 ?trmm

矩阵乘矩阵，其中一个矩阵为三角矩阵。

即：$B = \mathrm{alpha} * \mathrm{op}(A) * B$ 或 $B = \mathrm{alpha} * B * \mathrm{op}(A)$。

alpha为乘法系数，B为m*n的一般矩阵，A为三角矩阵。

接口定义

C interface：

void cblas_strmm(const enum CBLAS_ORDER Order, const enum CBLAS_SIDE Side, const enum CBLAS_UPLO Uplo, const enum CBLAS_TRANSPOSE TransA, const enum CBLAS_DIAG Diag, const BLASINT M, const BLASINT N, const float alpha, const float *A, const BLASINT lda, float *B, const BLASINT ldb);

void cblas_dtrmm(const enum CBLAS_ORDER Order, const enum CBLAS_SIDE Side, const enum CBLAS_UPLO Uplo, const enum CBLAS_TRANSPOSE TransA, const enum CBLAS_DIAG Diag, const BLASINT M, const BLASINT N, const double alpha, const double *A, const BLASINT lda, double *B, const BLASINT ldb);

 版权所有 © 华为技术有限公司

void cblas_ctrmm(const enum CBLAS_ORDER Order, const enum CBLAS_SIDE Side, const enum CBLAS_UPLO Uplo, const enum CBLAS_TRANSPOSE TransA, const enum CBLAS_DIAG Diag, const BLASINT M, const BLASINT N, const void *alpha, const void *A, const BLASINT lda, void *B, const BLASINT ldb);

void cblas_ztrmm(const enum CBLAS_ORDER Order, const enum CBLAS_SIDE Side, const enum CBLAS_UPLO Uplo, const enum CBLAS_TRANSPOSE TransA, const enum CBLAS_DIAG Diag, const BLASINT M, const BLASINT N, const void *alpha, const void *A, const BLASINT lda, void *B, const BLASINT ldb);

Fortran interface：

CALL STRMM(SIDE, UPLO, TRANSA, DIAG, M, N, ALPHA, A, LDA, B, LDB)

CALL DTRMM(SIDE, UPLO, TRANSA, DIAG, M, N, ALPHA, A, LDA, B, LDB)

CALL CTRMM(SIDE, UPLO, TRANSA, DIAG, M, N, ALPHA, A, LDA, B, LDB)

CALL ZTRMM(SIDE, UPLO, TRANSA, DIAG, M, N, ALPHA, A, LDA, B, LDB)

参数

参数名	类型	描述	输入/输出
order	枚举类型CBLAS_ORDER	表示矩阵是行主序或列主序。	输入
Side	枚举类型CBLAS_SIDE	矩阵A在矩阵B的左边还是右边。 ● 当Side = CblasLeft，$B = \text{alpha} * \text{op}(A) * B$。 ● 当Side = CblasRight，$B = \text{alpha} * B * \text{op}(A)$。	输入
Uplo	枚举类型CBLAS_UPLO	表示使用矩阵A的上三角或下三角。 ● 如果Uplo=CblasUpper，则使用A的上三角部分。 ● 如果Uplo=CblasLower，则使用A的下三角部分。	输入

版权所有 © 华为技术有限公司

参数名	类型	描述	输入/输出
TransA	枚举类型CBLAS_TRANSPOSE	矩阵A是否为常规矩阵，转置矩阵或共轭矩阵。 ● 当TransA = CblasNoTrans，$op(A) = A$。 ● 当TransA = CblasTrans，$op(A) = A'$。 ● 当TransA = CblasConjTrans，$op(A)=conjg(A)$。 ● 当TransA = CblasConjTrans，$op(A) = conjg(A')$。	输入
Diag	枚举类型CBLAS_DIAG	是否为单位三角矩阵。	输入
M	整型数	矩阵B的行。	输入
N	整型数	矩阵B的列。	输入
alpha	● 在strmm中是单精度浮点类型。 ● 在dtrmm中是双精度浮点类型。 ● 在ctrmm中是单精度复数类型。 ● 在ztrmm中是双精度复数类型。	乘法系数。	输入
A	● 在strmm中是单精度浮点类型。 ● 在dtrmm中是双精度浮点类型。 ● 在ctrmm中是单精度复数类型。 ● 在ztrmm中是双精度复数类型。	矩阵(lda, k)。 ● 当Side = CblasLeft，k = m。 ● 当Side = CblasRight，k = n。	输入

版权所有 © 华为技术有限公司

参数名	类型	描述	输入/输出
lda	整型数	• 矩阵为列存，side = CblasLeft，lda至少max(1, m)，否则至少max(1, n)。 • 矩阵为行存，Trans = CblasNoTrans，lda至少max(1, n)，否则至少max(1, m)。	输入
B	• 在strmm中是单精度浮点类型。 • 在dtrmm中是双精度浮点类型。 • 在ctrmm中是单精度复数类型。 • 在ztrmm中是双精度复数类型。	矩阵B。	输入/输出
ldb	整型数	矩阵为列存，ldb至少max(1, m)，否则至少max(1, n)。	输入

依赖

#include "kblas.h"

示例

C interface：

```
int m = 3, n = 3, lda = 3, ldb = 3;
float alpha = 1.0;
/**
 *           | 4.0   1.0  2.0 |
 *    A    = |  .    1.0  3.0 |
 *           |  .     .   1.0 |
 *
 *     B    =  | 2.0   3.0  1.0|
 *             | 11.0  -2.0 2.0|
 *             | 1.0   1.0  6.0|
 */
float a[9] = {4.0, 0, 0, 1.0, 1.0, 0, 2.0, 3.0, 1.0};
float b[9] = {2.0, 11.0, 1.0, 3.0, -2.0, 1.0, 1.0, 2.0, 6.0};

cblas_strmm(CblasColMajor, CblasRight, CblasUpper, CblasNoTrans, CblasNonUnit, m, n, alpha, a, lda, b, ldb);
/**
 * Output B
 *        | 8.000000        5.000000        14.000000|
 *    B = | 44.000000       9.000000        18.000000|
 *        | 4.000000        2.000000        11.000000|
 */
```

Fortran interface：

```
INTEGER :: M=3, N=3
INTEGER :: LDA=3, LDB=3
```

 版权所有 © 华为技术有限公司

```
      REAL(4) :: ALPHA=1.0
      REAL(4) :: A(3, 3), B(3, 3)
      DATA A/4.0, 0, 0, 1.0, 1.0, 0, 2.0, 3.0, 1.0/
      DATA B/2.0, 11.0, 1.0, 3.0, -2.0, 1.0, 1.0, 2.0, 6.0/
      EXTERNAL STRMM
      CALL STRMM('R', 'U', 'N', 'N', M, N, ALPHA, A, LDA, B, LDB)

*     Output B
*           | 8.000000        5.000000        14.000000|
*        B = | 44.000000       9.000000        18.000000|
*           | 4.000000        2.000000        11.000000|
```

6.4.8.4.3.13 ?trsm

三角矩阵方程求解。

方程是以下一种形式：

$op(A)*X=alpha*B$，或$X*op(A)=alpha*B$。

$op(A)=A$，或$op(A)=A'$，或 $op(A)=conjg(A)$ ，或 $op(A)=conjg(A')$。

alpha是乘法系数，X和B为m*n一般矩阵，A是三角矩阵。

接口定义

C interface：

void cblas_strsm(const enum CBLAS_ORDER Order, const enum CBLAS_SIDE Side, const enum CBLAS_UPLO Uplo, const enum CBLAS_TRANSPOSE TransA, const enum CBLAS_DIAG Diag, const BLASINT M, const BLASINT N, const float alpha, const float *A, const BLASINT lda, float *B, const BLASINT ldb);

void cblas_dtrsm(const enum CBLAS_ORDER Order, const enum CBLAS_SIDE Side, const enum CBLAS_UPLO Uplo, const enum CBLAS_TRANSPOSE TransA, const enum CBLAS_DIAG Diag, const BLASINT M, const BLASINT N, const double alpha, const double *A, const BLASINT lda, double *B, const BLASINT ldb);

void cblas_ctrsm(const enum CBLAS_ORDER Order, const enum CBLAS_SIDE Side, const enum CBLAS_UPLO Uplo, const enum CBLAS_TRANSPOSE TransA, const enum CBLAS_DIAG Diag, const BLASINT M, const BLASINT N, const void *alpha, const void *A, const BLASINT lda, void *B, const BLASINT ldb);

void cblas_ztrsm(const enum CBLAS_ORDER Order, const enum CBLAS_SIDE Side, const enum CBLAS_UPLO Uplo, const enum CBLAS_TRANSPOSE TransA, const enum CBLAS_DIAG Diag, const BLASINT M, const BLASINT N, const void *alpha, const void *A, const BLASINT lda, void *B, const BLASINT ldb);

Fortran interface：

CALL STRSM(SIDE, UPLO, TRANSA, DIAG, M, N, ALPHA, A, LDA, B, LDB)

CALL DTRSM(SIDE, UPLO, TRANSA, DIAG, M, N, ALPHA, A, LDA, B, LDB)

CALL CTRSM(SIDE, UPLO, TRANSA, DIAG, M, N, ALPHA, A, LDA, B, LDB)

CALL ZTRSM(SIDE, UPLO, TRANSA, DIAG, M, N, ALPHA, A, LDA, B, LDB)

 版权所有 © 华为技术有限公司

参数

参数名	类型	描述	输入/输出
Order	枚举类型CBLAS_ORDER	表示矩阵是行主序或列主序。	输入
Side	枚举类型CBLAS_SIDE	表示op(A)在X的左边或右边。 ● 如果Side=CblasLeft，则 $op(A)*X=alpha*B$。 ● 如果Side=CblasRight，则 $X*op(A)=alpha*B$。	输入
Uplo	枚举类型CBLAS_UPLO	表示使用矩阵A的上三角或下三角。 ● 如果Uplo=CblasUpper，则使用A的上三角部分。 ● 如果Uplo=CblasLower，则使用A的下三角部分。	输入
TransA	枚举类型CBLAS_TRANSPOSE	矩阵A为常规矩阵，转置矩阵或共轭矩阵。 ● 如果trans=CblasNoTrans，则 $op(A)=A$。 ● 如果trans=CblasTrans，则 $op(A)=A'$。 ● 如果trans=CblasConjTrans，则 $op(A)=conjg(A)$。 ● 如果trans=CblasConjTrans，则 $op(A)=conjg(A')$。	输入
Diag	枚举类型CBLAS_DIAG	表示A是否是单元三角矩阵。 ● 如果Diag=CblasUnit，则A是单元三角矩阵。 ● 如果Diag=CblasNonUnit，则A不是单元三角矩阵。	输入
M	整型数	矩阵B的行。	输入
N	整型数	矩阵B的列。	输入

版权所有 © 华为技术有限公司

参数名	类型	描述	输入/输出
alpha	• 在strsm中是单精度浮点类型。 • 在dtrsm中是双精度浮点类型。 • 在ctrsm中是单精度复数类型。 • 在ztrsm中是双精度复数类型。	乘法系数。	输入
A	• 在strsm中是单精度浮点类型。 • 在dtrsm中是双精度浮点类型。 • 在ctrsm中是单精度复数类型。 • 在ztrsm中是双精度复数类型。	三角矩阵A(lda, N)。	输入
lda	整型数	矩阵A中每列存储的地址距离，要求lda大于或等于max(1, N)。	输入
B	• 在strsm中是单精度浮点类型。 • 在dtrsm中是双精度浮点类型。 • 在ctrsm中是单精度复数类型。 • 在ztrsm中是双精度复数类型。	矩阵B。	输入/输出
ldb	整型数	矩阵为列存，ldb至少max(1, m)，否则至少max(1, n)。	输入

依赖

#include "kblas.h"

示例

C interface：

```
int m = 3, n = 3, lda = 3, ldb = 3;
float alpha = 1.0;
/**
 *            | 4.0   1.0  2.0 |
 *     A    = | .     1.0  3.0 |
 *            | .     .    1.0 |
```

 版权所有 © 华为技术有限公司

```
    *
    *     B    =  | 2.0   3.0  1.0|
    *             | 11.0  -2.0 2.0|
    *             | 1.0   1.0  6.0|
    */
   float a[9] = {4.0, 0, 0, 1.0, 1.0, 0, 2.0, 3.0, 1.0};
   float b[9] = {2.0, 11.0, 1.0, 3.0, -2.0, 1.0, 1.0, 2.0, 6.0};

   cblas_strsm(CblasColMajor, CblasRight, CblasUpper, CblasNoTrans, CblasNonUnit, m, n, alpha, a, lda, b,
ldb);
   /**
    * Output B
    *         | 0.500000        2.500000        -7.500000|
    *    B =  | 2.750000        -4.750000       10.750000|
    *         | 0.250000        0.750000        3.250000 |
    */
```

Fortran interface：

```
      INTEGER :: M=3, N=3
      INTEGER :: LDA=3, LDB=3
      REAL(4) :: ALPHA=1.0
      REAL(4) :: A(3, 3), B(3, 3)
      DATA A/4.0, 0, 0, 1.0, 1.0, 0, 2.0, 3.0, 1.0/
      DATA B/2.0, 11.0, 1.0, 3.0, -2.0, 1.0, 1.0, 2.0, 6.0/
      EXTERNAL STRSM
      CALL STRSM('R', 'U', 'N', 'N', M, N, ALPHA, A, LDA, B, LDB)

*     Output B
*           | 0.500000        2.500000        -7.500000|
*        B =  | 2.750000        -4.750000       10.750000|
*           | 0.250000        0.750000        3.250000 |
```

6.4.8.4.4 KML_BLAS-like extensions

6.4.8.4.4.1 函数说明

对KML_BLAS-like extensions所有函数族进行描述。

函数族	数据类型	描述
?axpby	s，d，c，z	向量分别缩放与加和。
?axpyc	c，z	对x向量取共轭，再进行向量缩放与加和。
?gemm3m	c，z	一般复数矩阵乘矩阵。
?omatcopy	s，d，c，z	矩阵out-place转置或复制的缩放，结果存储到另一地址空间。
?imatcopy	s，d，c，z	矩阵in-place转置或复制的缩放，结果存储到原地址空间。
?omatadd	s，d，c，z	矩阵缩放后out-place加计算。

版权所有 © 华为技术有限公司

函数族	数据类型	描述
?imatadd	s，d，c，z	矩阵缩放后in-place加计算。
?omatsub	s，d，c，z	矩阵缩放后out-place减计算。
?imatsub	s，d，c，z	矩阵缩放后in-place减计算。
?omatmul	s，d，c，z	矩阵缩放后out-place乘计算。
?imatmul	s，d，c，z	矩阵缩放后in-place乘计算。
?omatdiv	s，d，c，z	矩阵缩放后out-place除计算。
?imatdiv	s，d，c，z	矩阵缩放后in-place除计算。
?gemm_pack_get_size	s，d，c，z，b	返回存储pack后的矩阵所需的字节数。
?gemm_pack	s，d，c，z，b	对矩阵进行pack操作并将其存储到已分配的缓冲区中。
?gemm_compute	s，d，c，z，b	对pack后的A\B矩阵进行一般矩阵乘。
?gemm_batch	s，d，c，z，h，b	批量一般矩阵乘矩阵。
gemm_batch_?8?8s32	int8	批量一般矩阵乘矩阵。

6.4.8.4.4.2 ?axpby

向量缩放后的乘积与另一向量缩放后的乘积的加和。

即$y = \alpha * x + \beta * y$。alpha，beta是乘法系数，x和y是长度为n的向量。

接口定义

C interface：

 版权所有 © 华为技术有限公司

void cblas_saxpby(const BLASINT n, const float alpha, const float *x, const BLASINT incx, const float beta, float *y, const BLASINT incy);

void cblas_daxpby(const BLASINT n, const double alpha, const double *x, const BLASINT incx, const double beta, double *y, const BLASINT incy);

void cblas_caxpby(const BLASINT n, const void *alpha, const void *x, const BLASINT incx, const void *beta, void *y, const BLASINT incy);

void cblas_zaxpby(const BLASINT n, const void *alpha, const void *x, const BLASINT incx, const void *beta, void *y, const BLASINT incy);

Fortran interface：

CALL SAXPBY(N, ALPHA, X, INCX, BETA, Y, INCY)

CALL DAXPBY(N, ALPHA, X, INCX, BETA, Y, INCY)

CALL CAXPBY(N, ALPHA, X, INCX, BETA, Y, INCY)

CALL ZAXPBY(N, ALPHA, X, INCX, BETA, Y, INCY)

参数

参数名	类型	描述	输入/输出
n	整型数	表示x向量的元素个数。	输入
alpha	• 在daxpby中是双精度浮点类型。 • 在saxpby中是单精度浮点类型。 • 在caxpby中是单精度复数类型。 • 在zaxpby中是双精度复数类型。	表示乘法系数。	输入
x	• 在daxpby中是双精度浮点类型。 • 在saxpby中是单精度浮点类型。 • 在caxpby中是单精度复数类型。 • 在zaxpby中是双精度复数类型。	向量x，向量规模至少是(1+(n-1)*abs(incx))。	输入
incx	整型数	表示x向量增长步长。	输入

 版权所有 © 华为技术有限公司

参数名	类型	描述	输入/输出
beta	• 在daxpby中是双精度浮点类型。 • 在saxpby中是单精度浮点类型。 • 在caxpby中是单精度复数类型。 • 在zaxpby中是双精度复数类型。	表示乘法系数。	输入
y	• 在daxpby中是双精度浮点类型。 • 在saxpby中是单精度浮点类型。 • 在caxpby中是单精度复数类型。 • 在zaxpby中是双精度复数类型。	向量y，向量规模至少是(1+(n-1)*abs(incy))。	输入/输出
incy	整型数	表示y向量增长步长。	输入

依赖

#include "kblas.h"

示例

C interface：

```
float alpha = 2.0;
float beta = 3.0;
int n = 5, incx = 1, incy = 1;
/**
 *    X:f 0.340188, -0.105617, 0.283099, 0.298440, 0.411647
 *    Y: -0.302449, -0.164777, 0.268230, -0.222225, 0.053970
 */
float x[5] = {0.340188, -0.105617, 0.283099, 0.298440, 0.411647};
float y[5] = {-0.302449, -0.164777, 0.268230, -0.222225, 0.053970};

cblas_saxpby(n, alpha, x, incx, beta, y, incy);
/**
 *    Output Y: -0.226971, -0.705565, 1.370888, -0.069795, 0.985204
 */
```

Fortran interface：

```
  REAL(4) :: ALPHA=2.0
  REAL(4) :: BETA=3.0
  INTEGER :: N=5
  INTEGER :: INCX=1
  INTEGER :: INCY=1
  REAL(4) :: X(5)
  DATA X /0.340188, -0.105617, 0.283099, 0.298440, 0.411647/
  REAL(4) :: Y(5)
```

 版权所有 © 华为技术有限公司

```
      DATA Y /-0.302449, -0.164777, 0.268230, -0.222225, 0.053970/
      CALL SAXPBY(N, ALPHA, X, INCX, BETA, Y, INCY)

*     Output Y : -0.226971, -0.705565, 1.370888, -0.069795, 0.985204
```

6.4.8.4.4.3 ?axpyc

先对x向量进行共轭处理，再将缩放后的乘积与另一向量的加和。

即$y = \alpha * \mathrm{conj}(x) + y$。alpha是乘法系数，x和y是长度为n的向量。

接口定义

C interface：

void cblas_caxpyc(const BLASINT n, const void *alpha, const void *x, const BLASINT incx, void *y, const BLASINT incy);

void cblas_zaxpyc(const BLASINT n, const void *alpha, const void *x, const BLASINT incx, void *y, const BLASINT incy);

Fortran interface：

CALL CAXPYC(N, ALPHA, X, INCX, Y, INCY)

CALL ZAXPYC(N, ALPHA, X, INCX, Y, INCY)

参数

参数名	类型	描述	输入/输出
n	整型数	表示x向量的元素个数。	输入
alpha	• 在caxpyc中是单精度复数类型。 • 在zaxpyc中是双精度复数类型。	表示乘法系数。	输入
x	• 在caxpyc中是单精度复数类型。 • 在zaxpyc中是双精度复数类型。	向量x，向量规模至少是(1+(n-1)*abs(incx))。	输入
incx	整型数	表示x向量增长步长。	输入
y	• 在caxpyc中是单精度复数类型。 • 在zaxpyc中是双精度复数类型。	向量y，向量规模至少是(1+(n-1)*abs(incy))。	输入/输出
incy	整型数	表示y向量增长步长。	输入

 版权所有 © 华为技术有限公司

依赖

#include "kblas.h"

示例

C interface：

```
float alpha[2] = {0.25, 0.5};
int n = 2, incx = 1, incy = 1;
float x[4] = {1.0, 3.0, 5.0, 7.0};
float y[4] = {2.0, 4.0, 6.0, 8.0};

cblas_caxpyc(n, alpha, x, incx, y, incy);
/**
 *    Output Y: 3.75, 3.75, 10.75, 8.75
 */
```

Fortran interface：

```
    REAL(4) :: ALPHA(2)
    DATA ALPHA /0.25, 0.5
    INTEGER :: N=5
    INTEGER :: INCX=1
    INTEGER :: INCY=1
    REAL(4) :: X(4)
    DATA X /1.0, 3.0, 5.0, 7.0/
    REAL(4) :: Y(4)
    DATA Y /2.0, 4.0, 6.0, 8.0/
    CALL CAXPYC(N, ALPHA, X, INCX, Y, INCY)

*     Output Y : 3.75, 3.75, 10.75, 8.75
```

6.4.8.4.4.4 ?gemm3m

复数矩阵乘以矩阵。

即：$C = alpha*op(A)*op(B)+beta*C$。

op(X)可取值：

$op(X)=X, op(X)=X', op(X)=conjg(X), op(X)=conjg(X')$

，alpha，beta为乘法系数，op(A)为m*k矩阵，op(B)为k*n矩阵，C为m*n矩阵。

接口定义

C interface:

void cblas_cgemm3m(const enum CBLAS_ORDER Order, const enum CBLAS_TRANSPOSE TransA, const enum CBLAS_TRANSPOSE TransB, const BLASINT M, const BLASINT N, const BLASINT K, const void *alpha, const void *A, const BLASINT lda, const void *B, const BLASINT ldb, const void *beta, void *C, const BLASINT ldc);

void cblas_zgemm3m(const enum CBLAS_ORDER Order, const enum CBLAS_TRANSPOSE TransA, const enum CBLAS_TRANSPOSE TransB, const BLASINT M, const BLASINT N, const BLASINT K, const void *alpha, const void *A, const BLASINT lda, const void *B, const BLASINT ldb, const void *beta, void *C, const BLASINT ldc);

Fortran interface:

 版权所有 © 华为技术有限公司

CALL CGEMM3M(TRANSA, TRANSB, M, N, K, ALPHA, A, LDA, B, LDB, BETA, C, LDC)

CALL ZGEMM3M(TRANSA, TRANSB, M, N, K, ALPHA, A, LDA, B, LDB, BETA, C, LDC)

参数

参数名	类型	描述	输入/输出
order	枚举类型CBLAS_ORDER	表示矩阵是行主序或列主序。	输入
TransA	枚举类型CBLAS_TRANSPOSE	矩阵A为常规矩阵，转置矩阵或共轭矩阵。 ● 如果TransA = CblasNoTrans，$op(A) = A$。 ● 如果TransA = CblasTrans，$op(A) = A'$。 ● 如果TransA = CblasConjTrans，$op(A)=conjg(A)$。 ● 如果TransA = CblasConjTrans，$op(A) = conjg(A')$。	输入
TransB	枚举类型CBLAS_TRANSPOSE	矩阵B为常规矩阵，转置矩阵或共轭矩阵。 ● 如果TransB = CblasNoTrans，$op(B) = B$。 ● 如果TransB = CblasTrans，$op(B) = B'$。 ● 如果TransB = CblasConjTrans，$op(B)=conjg(B)$。 ● 如果TransB = CblasConjTrans，$op(B) = conjg(B')$。	输入
M	整型数	矩阵op(A)和矩阵C的行。	输入

 版权所有 © 华为技术有限公司

参数名	类型	描述	输入/输出
N	整型数	矩阵op(B)和矩阵C的列。	输入
K	整型数	矩阵op(A)的列和矩阵op(B)的行。	输入
alpha	• 在cgemm3m中是单精度复数类型。 • 在zgemm3m中是双精度复数类型。	乘法系数。	输入
A	• 在cgemm3m中是单精度复数类型。 • 在zgemm3m中是双精度复数类型。	矩阵A。	输入
lda	整型数	• 矩阵为列存，TransA = CblasNoTrans，lda至少max(1, m)，否则max(1, k)。 • 矩阵A为行存，TransA = CblasNoTrans，lda至少max(1, k)，否则max(1, m)。	输入
B	• 在cgemm3m中是单精度复数类型。 • 在zgemm3m中是双精度复数类型。	矩阵B。	输入
ldb	整型数	• 矩阵为列存，TransA = CblasNoTrans，ldb至少max(1, k)，否则max(1, n)。 • 矩阵A为行存，TransA = CblasNoTrans，lda至少max(1, n)，否则max(1, k)。	输入
beta	• 在cgemm3m中是单精度复数类型。 • 在zgemm3m中是双精度复数类型。	乘法系数。	输入
C	• 在cgemm3m中是单精度复数类型。 • 在zgemm3m中是双精度复数类型。	矩阵C。	输入/输出

 版权所有 © 华为技术有限公司

参数名	类型	描述	输入/输出
ldc	整型数	矩阵为列存，ldc至少max(1, m)，否则至少max(1,n)。	输入

依赖

#include "kblas.h"

示例

C interface：

```
int m = 4, k = 3, n = 4, lda = 4, ldb = 3, ldc = 4;
float alpha[2] = {1.0, 2.0}, beta[2] = {2.0, -1.0};
/**
 *    A:
 *        0.340188, -0.105617,  -0.222225, 0.053970,  0.135712, 0.217297
 *        0.283099, 0.298440,  -0.022603, 0.128871,  -0.358397, 0.106969
 *        0.411647, -0.302449,  -0.135216, 0.013401,  -0.483699, -0.257113
 *        -0.164777, 0.268230,  0.452230, 0.416195,  -0.362768, 0.304177
 *    B:
 *        -0.343321, -0.099056,  0.012932, 0.339112,  -0.006417, 0.472775,  -0.099771, 0.391529
 *        -0.370210, -0.391191,  0.112640, -0.203968,  -0.207483, 0.271358,  -0.216685, -0.147542
 *        0.498924, -0.281743,  0.137552, 0.024287,  0.026745, 0.269914,  0.307725, 0.419026
 *    C:
 *        -0.430245, 0.449327,  -0.435829, -0.479977,  -0.233334, 0.039760,  -0.062362, 0.431835
 *        0.025995, -0.413944,  -0.042298, -0.436904,  -0.124793, 0.260249,  0.430810, 0.220952
 *        -0.307786, 0.163227,  -0.261720, 0.470634,  0.012535, 0.167724,  -0.215707, 0.238534
 *        0.390233, -0.151107,  0.402208, 0.350920,  0.031606, -0.460720,  0.139979, -0.145951
 *
 */
float a[24] = {0.340188, -0.105617, 0.283099, 0.298440, 0.411647, -0.302449, -0.164777, 0.268230,
               -0.222225, 0.053970, -0.022603, 0.128871, -0.135216, 0.013401, 0.452230, 0.416195,
               0.135712, 0.217297, -0.358397, 0.109, -0.483699, -0.257113, -0.362768, 0.304177};
float b[24] = {-0.343321, -0.099056, -0.370210, -0.391191, 0.498924, -0.281743,
               0.012932, 0.339112, 0.112640, -0.203968, 0.137552, 0.024287,
               -0.006417, 0.472775, -0.207483, 0.271358, 0.026745, 0.269914,
               -0.099771, 0.529, -0.216685, -0.147542, 0.307725, 0.419026};
float c[32] = {-0.430245, 0.449327, 0.025995, -0.413944, -0.307786, 0.163227, 0.390233, -0.151107,
               -0.435829, -0.479977, -0.042298, -0.436904, -0.261720, 0.470634, 0.402208, 0.350920,
               -0.233334, 0.039760, -0.124793, 0.260249, 0.012535, 0.167724, 0.031606, -0.460720,
               -0.062362, 0.431835, 0.430810, 0.220952, -0.215707, 0.238534, 0.139979, -0.145951};

cblas_cgemm3m(CblasColMajor,CblasNoTrans,CblasNoTrans, m, n, k, alpha, a, lda, b, ldb, beta, c, ldc);
/**
 * Output C:
 *        -0.585492, 1.678710,  -1.709228, -0.246373,  -0.667621, 0.493688,  -0.334783, 1.318867
 *        -0.490883, -1.181802,  -0.897803, -0.957732,  -0.212003, 0.226129,  0.806885, -0.646487
 *        -1.120088, -0.106275,  -0.253022, 1.390848,  0.374721, 0.785023,  -0.050149, 0.894140
 *        0.918170, -0.878755,  1.270267, 0.205796,  -0.555751, -1.934741,  0.391359, -1.563382
 */
```

Fortran interface:

```
  PROGRAM GEMM3M
  INTEGER :: M=4, K=3, N=4
  INTEGER :: LDA=4, LDB=3, LDC=4
  COMPLEX(4) :: ALPHA=(1.0, 2.0), BETA=(2.0, -1.0)
  COMPLEX(4) :: A(4, 3), B(3, 4), C(4, 4)
  DATA A/(0.340188, -0.105617), (0.283099, 0.298440),
 $       (0.411647, -0.302449), (-0.164777, 0.268230),
```

 版权所有 © 华为技术有限公司

```
     $       (-0.222225, 0.053970), (-0.022603, 0.128871),
     $       (-0.135216, 0.013401), (0.452230, 0.416195),
     $       (0.135712, 0.217297), (-0.358397, 0.109),
     $       (-0.483699, -0.257113), (-0.362768, 0.304177)/
      DATA B/(-0.343321, -0.099056), (-0.370210, -0.391191),
     $       (0.498924, -0.281743), (0.012932, 0.339112),
     $       (0.112640, -0.203968), (0.137552, 0.024287),
     $       (-0.006417, 0.472775), (-0.207483, 0.271358),
     $       (0.026745, 0.269914), (-0.099771, 0.529),
     $       (-0.216685, -0.147542), (0.307725, 0.419026)/
      DATA C/(-0.430245, 0.449327), (0.025995, -0.413944),
     $       (-0.307786, 0.163227), (0.390233, -0.151107),
     $       (-0.435829, -0.479977), (-0.042298, -0.436904),
     $       (-0.261720, 0.470634), (0.402208, 0.350920),
     $       (-0.233334, 0.039760), (-0.124793, 0.260249),
     $        (0.012535, 0.167724), (0.031606, -0.460720),
     $       (-0.062362, 0.431835), (0.430810, 0.220952),
     $       (-0.215707, 0.238534), (0.139979, -0.145951)/
      EXTERNAL CGEMM3M
      CALL CGEMM3M('N', 'N', M, N, K, ALPHA, A, LDA, B, LDB,
     $            BETA, C, LDC)
      END
*     Output C:
*           (-0.585492, 1.678710), (-1.709228, -0.246373),  (-0.667621, 0.493688),  (-0.334783, 1.318867)
*           (-0.490883, -1.181802), (-0.897803, -0.957732),  (-0.212003, 0.226129),  (0.806885, -0.646487)
*           (-1.120088, -0.106275), (-0.253022, 1.390848),  (0.374721, 0.785023),  (-0.050149, 0.894140)
*           (0.918170, -0.878755),  (1.270267, 0.205796),  (-0.555751, -1.934741),  (0.391359, -1.563382)
```

6.4.8.4.4.5 ?omatcopy

矩阵out-place转置或复制的缩放，结果存储到另一地址空间。

即：$B = alpha * op(A)$。

op(X)可取值：$op(X) = X$, $op(X) = X'$, $op(X) = conjg(X')$, $op(X) = conjg(X)$, alpha为乘法系数，A为m*n矩阵。

接口定义

C interface：

void cblas_somatcopy(const enum CBLAS_ORDER Order, const enum CBLAS_TRANSPOSE Trans, const BLASINT M, const BLASINT N, const float alpha, const float *A, const BLASINT lda, float *B, const BLASINT ldb);

void cblas_domatcopy(const enum CBLAS_ORDER Order, const enum CBLAS_TRANSPOSE Trans, const BLASINT M, const BLASINT N, const double alpha, const double *A, const BLASINT lda, double *B, const BLASINT ldb);

void cblas_comatcopy(const enum CBLAS_ORDER Order, const enum CBLAS_TRANSPOSE Trans, const BLASINT M, const BLASINT N, const float *alpha, const float *A, const BLASINT lda, float *B, const BLASINT ldb);

void cblas_zomatcopy(const enum CBLAS_ORDER Order, const enum CBLAS_TRANSPOSE Trans, const BLASINT M, const BLASINT N, const double *alpha, const double *A, const BLASINT lda, double *B, const BLASINT ldb);

Fortran interface：

CALL SOMATCOPY(ORDER, TRANS, M, N, ALPHA, A, LDA, B, LDB)

CALL DOMATCOPY(ORDER, TRANS, M, N, ALPHA, A, LDA, B, LDB)

 版权所有 © 华为技术有限公司

CALL COMATCOPY(ORDER, TRANS, M, N, ALPHA, A, LDA, B, LDB)

CALL ZOMATCOPY(ORDER, TRANS, M, N, ALPHA, A, LDA, B, LDB)

参数

参数名	类型	描述	输入/输出
order	枚举类型CBLAS_ORDER	表示矩阵是行主序或列主序。	输入
Trans	枚举类型CBLAS_TRANSPOSE	矩阵A为常规矩阵，转置矩阵或共轭矩阵。 ● 如果Trans = CblasNoTrans，$op(A) = A$。 ● 如果Trans = CblasTrans，$op(A) = A'$。 ● 如果Trans = CblasConjTrans，$op(A) = conjg(A')$。 ● 如果Trans = CblasConjNoTrans，$op(A) = conjg(A)$。	输入
M	整型数	矩阵A的行。	输入
N	整型数	矩阵A的列。	输入
alpha	● 在somatcopy中是单精度浮点类型。 ● 在domatcopy中是双精度浮点类型。 ● 在comatcopy中是单精度复数类型。 ● 在zomatcopy中是双精度复数类型。	乘法系数。	输入
A	● 在somatcopy中是单精度浮点类型。 ● 在domatcopy中是双精度浮点类型。 ● 在comatcopy中是单精度复数类型。 ● 在zomatcopy中是双精度复数类型。	矩阵A。	输入

 版权所有 © 华为技术有限公司

参数名	类型	描述	输入/输出
lda	整型数	• 矩阵为列存，TransA = CblasNoTrans，lda至少max(1, m)，否则max(1, n)。 • 矩阵A为行存，TransA = CblasNoTrans，lda至少max(1, n)，否则max(1, m)。	输入
B	• 在somatcopy中是单精度浮点类型。 • 在domatcopy中是双精度浮点类型。 • 在comatcopy中是单精度复数类型。 • 在zomatcopy中是双精度复数类型。	矩阵B。	输出
ldb	整型数	• 矩阵为列存，TransA = CblasNoTrans，ldb至少max(1, m)，否则max(1, n)。 • 矩阵A为行存，TransA = CblasNoTrans，lda至少max(1, n)，否则max(1, m)。	输入

依赖

#include "kblas.h"

示例

C interface：

```
int m = 4;
int k = 3;
int lda = 4;
int ldb = 4;
float alpha[2] = {1.0, 2.0};
/**
 *    A:
 *        0.340188, -0.105617,  -0.222225, 0.053970,  0.135712, 0.217297
 *        0.283099, 0.298440,  -0.022603, 0.128871,  -0.358397, 0.106969
 *        0.411647, -0.302449,  -0.135216, 0.013401,  -0.483699, -0.257113
 *        -0.164777, 0.268230,  0.452230, 0.416195,  -0.362768, 0.304177
 */
float a[24] = {0.340188, -0.105617, 0.283099, 0.298440, 0.411647, -0.302449, -0.164777, 0.268230,
               -0.222225, 0.053970, -0.022603, 0.128871, -0.135216, 0.013401, 0.452230, 0.416195,
               0.135712, 0.217297, -0.358397, 0.109, -0.483699, -0.257113, -0.362768, 0.304177};
float b[24] = {0};
```


版权所有 © 华为技术有限公司

```
cblas_comatcopy(CblasColMajor, CblasNoTrans, m, k, alpha, a, lda, b, ldb);
/**
 * Output B:
 *       0.551422, 0.574759, -0.313781, 0.864638, 1.016545, 0.520845
 *        -0.701237, -0.061324, -0.330165, -0.390480, -0.280345, 0.083665
 *        -0.162018, -0.257031, -0.380160, 1.320655, -0.298882, 0.488721
 *        -0.576397, -0.607794, 0.030527, -1.224511, -0.971122, -0.421359
 */
```

Fortran interface：

```
    PROGRAM OMATCOPY
    INTEGER :: M=4, K=3
    INTEGER :: LDA=4, LDB=4
    COMPLEX(4) :: ALPHA=(1.0, 2.0)
    COMPLEX(4) :: A(4, 3), B(4, 3)
    DATA A/(0.340188, -0.105617), (0.283099, 0.298440),
   $       (0.411647, -0.302449), (-0.164777, 0.268230),
   $       (-0.222225, 0.053970), (-0.022603, 0.128871),
   $       (-0.135216, 0.013401), (0.452230, 0.416195),
   $       (0.135712, 0.217297), (-0.358397, 0.109),
   $       (-0.483699, -0.257113), (-0.362768, 0.304177)/
    EXTERNAL COMATCOPY
    CALL COMATCOPY('C', 'N', M, K, ALPHA, A, LDA, B, LDB)
    END

*     Output B:
*           (0.551422000,0.574759007)          (-0.313781023,0.864637971)          (1.01654494,0.520844996)
*           (-0.701236963,-6.132400036E-02)    (-0.330164999,-0.390479982)
(-0.280344993,8.366499841E-02)
*           (-0.162018001,-0.257030994)        (-0.380160004,1.32065499)
(-0.298882008,0.488721013)
*           (-0.576397002,-0.607794046)        (3.052702546E-02,-1.22451103)
(-0.971121967,-0.421359003)
```

6.4.8.4.4.6 ?imatcopy

矩阵in-place转置或复制的缩放，结果存储到原地址空间。

即：$AB = alpha * op(AB)$。

op(X)可取值：$op(X) = X$, $op(X) = X'$, $op(X) = conjg(X')$, $op(X) = conjg(X)$，alpha为乘法系数，AB为m*n矩阵。

接口定义

C interface：

void cblas_simatcopy(const enum CBLAS_ORDER cblasOrder, const enum CBLAS_TRANSPOSE Trans, const BLASINT M, const BLASINT N, const float alpha, float *AB, const BLASINT lda, const BLASINT ldb);

void cblas_dimatcopy(const enum CBLAS_ORDER cblasOrder, const enum CBLAS_TRANSPOSE Trans, const BLASINT M, const BLASINT N, const double alpha, double *AB, const BLASINT lda, const BLASINT ldb);

void cblas_cimatcopy(const enum CBLAS_ORDER cblasOrder, const enum CBLAS_TRANSPOSE Trans, const BLASINT M, const BLASINT N, const float *alpha, float *AB, const BLASINT lda, const BLASINT ldb);

 版权所有 © 华为技术有限公司

void cblas_zimatcopy(const enum CBLAS_ORDER cblasOrder, const enum CBLAS_TRANSPOSE Trans, const BLASINT M, const BLASINT N, const double *alpha, double *AB, const BLASINT lda, const BLASINT ldb);

Fortran interface：

CALL SIMATCOPY(ORDER, TRANS, M, N, ALPHA, AB, LDA, LDB)

CALL DIMATCOPY(ORDER, TRANS, M, N, ALPHA, AB, LDA, LDB)

CALL CIMATCOPY(ORDER, TRANS, M, N, ALPHA, AB, LDA, LDB)

CALL ZIMATCOPY(ORDER, TRANS, M, N, ALPHA, AB, LDA, LDB)

参数

参数名	类型	描述	输入/输出
order	枚举类型CBLAS_ORDER	表示矩阵是行主序或列主序。	输入
Trans	枚举类型CBLAS_TRANSPOSE	矩阵A为常规矩阵，转置矩阵或共轭矩阵。 ● 如果Trans = CblasNoTrans，$op(A) = A$。 ● 如果Trans = CblasTrans，$op(A) = A'$。 ● 如果Trans = CblasConjTrans，$op(A) = conjg(A')$。 ● 如果Trans = CblasConjNoTrans，$op(A) = conjg(A)$。	输入
M	整型数	矩阵AB的行。	输入
N	整型数	矩阵AB的列。	输入
alpha	● 在simatcopy中是单精度浮点类型。 ● 在dimatcopy中是双精度浮点类型。 ● 在cimatcopy中是单精度复数类型。 ● 在zimatcopy中是双精度复数类型。	乘法系数。	输入

 版权所有 © 华为技术有限公司

参数名	类型	描述	输入/输出
AB	• 在simatcopy中是单精度浮点类型。 • 在dimatcopy中是双精度浮点类型。 • 在cimatcopy中是单精度复数类型。 • 在zimatcopy中是双精度复数类型。	矩阵AB。	输入/输出
lda	整型数	矩阵AB的leading dimension。 • 矩阵AB为列存，lda至少max(1, m)。 • 矩阵AB为行存，lda至少max(1, n)。	输入
ldb	整型数	输出结果的leading dimension。 • 矩阵AB为列存，Trans = CblasNoTrans，ldb至少max(1, m)，否则max(1, n)。 • 矩阵AB为行存，Trans = CblasNoTrans，lda至少max(1, n)，否则max(1, m)。	输入

依赖

#include "kblas.h"

示例

C interface：

```
int m = 4;
int k = 3;
int lda = 4;
int ldb = 4;
float alpha[2] = {1.0, 2.0};
/**
 *    AB:
 *        0.340188, -0.105617,  -0.222225, 0.053970,  0.135712, 0.217297
 *        0.283099, 0.298440,  -0.022603, 0.128871,  -0.358397, 0.106969
 *        0.411647, -0.302449,  -0.135216, 0.013401,  -0.483699, -0.257113
 *        -0.164777, 0.268230,  0.452230, 0.416195,  -0.362768, 0.304177
 */
float ab[24] = {0.340188, -0.105617, 0.283099, 0.298440, 0.411647, -0.302449, -0.164777, 0.268230,
                -0.222225, 0.053970, -0.022603, 0.128871, -0.135216, 0.013401, 0.452230, 0.416195,
                0.135712, 0.217297, -0.358397, 0.109, -0.483699, -0.257113, -0.362768, 0.304177};
```

 版权所有 © 华为技术有限公司

```
cblas_cimatcopy(CblasColMajor, CblasNoTrans, m, k, alpha, ab, lda, ldb);
/**
 * Output AB:
 *       0.551422, 0.574759, -0.313781, 0.864638, 1.016545, 0.520845
 *        -0.701237, -0.061324, -0.330165, -0.390480, -0.280345, 0.083665
 *        -0.162018, -0.257031, -0.380160, 1.320655, -0.298882, 0.488721
 *        -0.576397, -0.607794, 0.030527, -1.224511, -0.971122, -0.421359
 */
```

Fortran interface:

```
    PROGRAM OMATCOPY
    INTEGER :: M=4, K=3
    INTEGER :: LDA=4, LDB=4
    COMPLEX(4) :: ALPHA=(1.0, 2.0)
    COMPLEX(4) :: AB(4, 3)
    DATA AB/(0.340188, -0.105617), (0.283099, 0.298440),
   $        (0.411647, -0.302449), (-0.164777, 0.268230),
   $        (-0.222225, 0.053970), (-0.022603, 0.128871),
   $        (-0.135216, 0.013401), (0.452230, 0.416195),
   $        (0.135712, 0.217297), (-0.358397, 0.109),
   $        (-0.483699, -0.257113), (-0.362768, 0.304177)/
    EXTERNAL CIMATCOPY
    CALL CIMATCOPY('C', 'N', M, K, ALPHA, AB, LDA, LDB)
    END

*     Output AB:
*            (0.551422000,0.574759007)          (-0.313781023,0.864637971)          (1.01654494,0.520844996)
*            (-0.701236963,-6.132400036E-02)    (-0.330164999,-0.390479982)
(-0.280344993,8.366499841E-02)
*            (-0.162018001,-0.257030994)        (-0.380160004,1.32065499)
(-0.298882008,0.488721013)
*            (-0.576397002,-0.607794046)        (3.052702546E-02,-1.22451103)
(-0.971121967,-0.421359003)
```

6.4.8.4.4.7 ?omatadd

矩阵缩放后out-place加计算。

即 $C := alpha * op(A) + beta * op(B)$ 。

op可取值： $op(A) = A, A^T, conj(A), conj(A^T)$ 。

接口定义

C interface:

void cblas_somatadd(const enum CBLAS_ORDER ordering, const enum CBLAS_TRANSPOSE TransA, const enum CBLAS_TRANSPOSE TransB, const BLASINT m, const BLASINT n, const float alpha, float *A, const BLASINT lda, const float beta, float *B, const BLASINT ldb, float *C, const BLASINT ldc);

void cblas_domatadd(const enum CBLAS_ORDER ordering, const enum CBLAS_TRANSPOSE TransA, const enum CBLAS_TRANSPOSE TransB, const BLASINT m, const BLASINT n, const double alpha, double *A, const BLASINT lda, const double beta, double *B, const BLASINT ldb, double *C, const BLASINT ldc);

void cblas_comatadd(const enum CBLAS_ORDER ordering, const enum CBLAS_TRANSPOSE TransA, const enum CBLAS_TRANSPOSE TransB, const

 版权所有 © 华为技术有限公司

BLASINT m, const BLASINT n, const float *alpha, float *A, const BLASINT lda, const float *beta, float *B, const BLASINT ldb, float *C, const BLASINT ldc);

void cblas_zomatadd(const enum CBLAS_ORDER ordering, const enum CBLAS_TRANSPOSE TransA, const enum CBLAS_TRANSPOSE TransB, const BLASINT m, const BLASINT n, const double *alpha, double *A, const BLASINT lda, const double *beta, double *B, const BLASINT ldb, double *C, const BLASINT ldc);

Fortran interface：

CALL SOMATADD(ORDER, TRANSA, TRANSB, M, N, ALPHA, A, LDA, BETA, B, LDB, C, LDC)

CALL DOMATADD(ORDER, TRANSA, TRANSB, M, N, ALPHA, A, LDA, BETA, B, LDB, C, LDC)

CALL COMATADD(ORDER, TRANSA, TRANSB, M, N, ALPHA, A, LDA, BETA, B, LDB, C, LDC)

CALL ZOMATADD(ORDER, TRANSA, TRANSB, M, N, ALPHA, A, LDA, BETA, B, LDB, C, LDC)

参数

参数名	类型	描述	输入/输出
order	枚举类型CBLAS_ORDER	表示矩阵是行主序或列主序。	输入
TransA	枚举类型CBLAS_TRANSPOSE	矩阵A为常规矩阵、转置矩阵、共轭矩阵或转置共轭矩阵。 • 如果TransA = CblasNoTrans，op(A) = A。 • 如果TransA = CblasTrans，op(A) = A'。 • 如果TransA = CblasConjNoTrans，op(A) = conjg(A)。 • 如果TransA = CblasConjTrans，op(A) = conjg(A')。	输入

版权所有 © 华为技术有限公司

参数名	类型	描述	输入/输出
TransB	枚举类型CBLAS_TRANSPOSE	矩阵B为常规矩阵、转置矩阵、共轭矩阵或转置共轭矩阵。 ● 如果TransB = CblasNoTrans，op(B) = B。 ● 如果TransB = CblasTrans，op(B) = B'。 ● 如果TransB = CblasConjNoTrans，op(B) = conjg(B)。 ● 如果TransB = CblasConjTrans，op(B) = conjg(B')。	输入
M	整型数	矩阵op(A)、op(B)、C的行数。	输入
N	整型数	矩阵op(A)、op(B)、C的列数。	输入
alpha	● 在somatadd中是单精度浮点类型。 ● 在domatadd中是双精度浮点类型。 ● 在comatadd中是单精度复数类型。 ● 在zomatadd中是双精度复数类型。	表示乘法系数。	输入
A	● 在somatadd中是单精度浮点类型。 ● 在domatadd中是双精度浮点类型。 ● 在comatadd中是单精度复数类型。 ● 在zomatadd中是双精度复数类型。	矩阵A。	输入
lda	整型数	● 矩阵A为列存，TransA = CblasNoTrans，lda至少max(1, m)，否则max(1, n)。 ● 矩阵A为行存，TransA = CblasNoTrans，lda至少max(1, n)，否则max(1, m)。	输入

 版权所有 © 华为技术有限公司

参数名	类型	描述	输入/输出
beta	• 在somatadd中是单精度浮点类型。 • 在domatadd中是双精度浮点类型。 • 在comatadd中是单精度复数类型。 • 在zomatadd中是双精度复数类型。	表示乘法系数。	输入
B	• 在somatadd中是单精度浮点类型。 • 在domatadd中是双精度浮点类型。 • 在comatadd中是单精度复数类型。 • 在zomatadd中是双精度复数类型。	矩阵B。	输入
ldb	整型数	• 矩阵为列存，TransB = CblasNoTrans，ldb至少max(1, m)，否则max(1, n)。 • 矩阵为行存，TransB = CblasNoTrans，ldb至少max(1, n)，否则max(1, m)。	输入
C	• 在somatadd中是单精度浮点类型。 • 在domatadd中是双精度浮点类型。 • 在comatadd中是单精度复数类型。 • 在zomatadd中是双精度复数类型。	矩阵C。	输出
ldc	整型数	矩阵为列存，ldc至少max(1, m)，否则max(1, n)。	输入

依赖

#include "kblas.h"

示例

C interface：

 版权所有 © 华为技术有限公司

```
int m = 4;
int n = 3;
int lda = 4;
int ldb = 4;
int ldc = 4;
float alpha = 2.0;
float beta = 1.0;

/**
 *    A:
 *        0.558402, 0.691866, 0.533786,
 *        0.670904, 0.655601, 0.542604,
 *        0.743670, 0.965085, 0.993501,
 *        0.623749, 0.587617, 0.966428
 *    B:
 *        0.730575, 0.624083, 0.526782,
 *        0.804889, 0.677061, 0.529442,
 *        0.785305, 0.802311, 0.508981,
 *        0.574068, 0.646785, 0.717143
 *    C:
 *        0, 0, 0
 *        0, 0, 0
 *        0, 0, 0
 *        0, 0, 0
 */

float a[12] = {0.558402, 0.691866, 0.533786,
               0.670904, 0.655601, 0.542604,
               0.743670, 0.965085, 0.993501,
               0.623749, 0.587617, 0.966428
};

float b[12] = {0.730575, 0.624083, 0.526782,
               0.804889, 0.677061, 0.529442,
               0.785305, 0.802311, 0.508981,
               0.574068, 0.646785, 0.717143,
};

float c[12] = {0};

cblas_somatadd(CblasColMajor, CblasNoTrans, CblasNoTrans, m, n, alpha, a, lda, beta, b, ldb, c, ldc);

/**
 * Output C:
 *        1.847379, 2.007814, 1.594354,
 *        2.146696, 1.988263, 1.614650,
 *        2.272644, 2.732482, 2.495982,
 *        1.821566, 1.822019, 2.649999
 */
```

Fortran interface：

```
  PROGRAM OMATADD
  INTEGER :: M=4, N=3
  INTEGER :: LDA=4, LDB=4, LDC=4
  REAL(4) :: ALPHA=2.0
  REAL(4) :: BETA=1.0
  REAL(4) :: A(4, 3), B(4, 3), C(4, 3)
  DATA A/0.558402, 0.691866, 0.533786,
        0.670904, 0.655601, 0.542604,
        0.743670, 0.965085, 0.993501,
        0.623749, 0.587617, 0.966428/
  DATA B/0.730575, 0.624083, 0.526782,
        0.804889, 0.677061, 0.529442,
        0.785305, 0.802311, 0.508981,
        0.574068, 0.646785, 0.717143/
  DATA C/0.0, 0.0, 0.0,
        0.0, 0.0, 0.0,
        0.0, 0.0, 0.0/
  EXTERNAL SOMATADD
```

 版权所有 © 华为技术有限公司

```
    CALL SOMATADD('C', 'N', 'N', M, N, ALPHA, A, LDA, BETA, B, LDB, C, LDC)
    END

  * Output C:
  *       1.847379, 2.007814, 1.594354,
  *       2.146696, 1.988263, 1.614650,
  *       2.272644, 2.732482, 2.495982,
  *       1.821566, 1.822019, 2.649999
```

6.4.8.4.4.8 ?imatadd

矩阵缩放后in-place加计算。

即 $BC := alpha * op(A) + BC$ 。

op可取值： $op(A) = A, A^T, conj(A), conj(A^T)$ 。

接口定义

C interface：

void cblas_simatadd(const enum CBLAS_ORDER ordering, const enum CBLAS_TRANSPOSE TransA, const BLASINT m, const BLASINT n, const float alpha, float *A, const BLASINT lda, float *BC, const BLASINT ldb);

void cblas_dimatadd(const enum CBLAS_ORDER ordering, const enum CBLAS_TRANSPOSE TransA, const BLASINT m, const BLASINT n, const double alpha, double *A, const BLASINT lda, double *BC, const BLASINT ldb);

void cblas_cimatadd(const enum CBLAS_ORDER ordering, const enum CBLAS_TRANSPOSE TransA, const BLASINT m, const BLASINT n, const float *alpha, float *A, const BLASINT lda, float *BC, const BLASINT ldb);

void cblas_zimatadd(const enum CBLAS_ORDER ordering, const enum CBLAS_TRANSPOSE TransA, const BLASINT m, const BLASINT n, const double *alpha, double *A, const BLASINT lda, double *BC, const BLASINT ldb);

Fortran interface：

CALL SIMATADD(ORDER, TRANSA, M, N, ALPHA, A, LDA, BC, LDB)

CALL DIMATADD(ORDER, TRANSA, M, N, ALPHA, A, LDA, BC, LDB)

CALL CIMATADD(ORDER, TRANSA, M, N, ALPHA, A, LDA, BC, LDB)

CALL ZIMATADD(ORDER, TRANSA, M, N, ALPHA, A, LDA, BC, LDB)

参数

参数名	类型	描述	输入/输出
order	枚举类型CBLAS_ORDER	表示矩阵是行主序或列主序。	输入

 版权所有 © 华为技术有限公司

参数名	类型	描述	输入/输出
TransA	枚举类型CBLAS_TRANSPOSE	矩阵A为常规矩阵，转置矩阵，共轭矩阵或转置共轭矩阵。 ● 如果TransA = CblasNoTrans，op(A) = A。 ● 如果TransA = CblasTrans，op(A) = A'。 ● 如果TransA = CblasConjNoTrans，op(A) = conjg(A)。 ● 如果TransA = CblasConjTrans，op(A) = conjg(A')。	输入
M	整型数	矩阵op(A)、BC的行。	输入
N	整型数	矩阵op(A)、BC的列。	输入
alpha	● 在simatadd中是单精度浮点类型。 ● 在dimatadd中是双精度浮点类型。 ● 在cimatadd中是单精度复数类型。 ● 在zimatadd中是双精度复数类型。	表示乘法系数。	输入
A	● 在simatadd中是单精度浮点类型。 ● 在dimatadd中是双精度浮点类型。 ● 在cimatadd中是单精度复数类型。 ● 在zimatadd中是双精度复数类型。	矩阵A。	输入
lda	整型数	● 矩阵A为列存，TransA = CblasNoTrans，lda至少max(1, m)，否则max(1, n)。 ● 矩阵A为行存，TransA = CblasNoTrans，lda至少max(1, n)，否则max(1, m)。	输入

 版权所有 © 华为技术有限公司

参数名	类型	描述	输入/输出
BC	• 在simatadd中是单精度浮点类型。 • 在dimatadd中是双精度浮点类型。 • 在cimatadd中是单精度复数类型。 • 在zimatadd中是双精度复数类型。	矩阵BC。	输入
ldb	整型数	• 矩阵为列存，TransB = CblasNoTrans，ldb至少max(1, m)，否则max(1, n)。 • 矩阵为行存，TransB = CblasNoTrans，ldb至少max(1, n)，否则max(1, m)。	输入

依赖

#include "kblas.h"

示例

C interface：

```
int m = 4;
int n = 3;
int lda = 4;
int ldb = 4;
float alpha = 2.0;

/**
 *    A:
 *        0.558402, 0.691866, 0.533786,
 *        0.670904, 0.655601, 0.542604,
 *        0.743670, 0.965085, 0.993501,
 *        0.623749, 0.587617, 0.966428
 *    BC:
 *        0.730575, 0.624083, 0.526782,
 *        0.804889, 0.677061, 0.529442,
 *        0.785305, 0.802311, 0.508981,
 *        0.574068, 0.646785, 0.717143
 */

float a[12] = {0.558402, 0.691866, 0.533786,
               0.670904, 0.655601, 0.542604,
               0.743670, 0.965085, 0.993501,
               0.623749, 0.587617, 0.966428
};

float bc[12] = {0.730575, 0.624083, 0.526782,
               0.804889, 0.677061, 0.529442,
               0.785305, 0.802311, 0.508981,
```

 版权所有 © 华为技术有限公司

```
            0.574068, 0.646785, 0.717143,
};

cblas_simatadd(CblasColMajor, CblasNoTrans, m, n, alpha, a, lda, bc, ldb);

/**
 * Output BC:
 *       1.847379, 2.007814, 1.594354,
 *       2.146696, 1.988263, 1.614650,
 *       2.272644, 2.732482, 2.495982,
 *       1.821566, 1.822019, 2.649999
 */
```

Fortran interface：

```
  PROGRAM IMATADD
  INTEGER :: M=4, N=3
  INTEGER :: LDA=4, LDB=4
  REAL(4) :: ALPHA=2.0
  REAL(4) :: A(4, 3), BC(4, 3)
  DATA A/0.558402, 0.691866, 0.533786,
        0.670904, 0.655601, 0.542604,
        0.743670, 0.965085, 0.993501,
        0.623749, 0.587617, 0.966428/
  DATA BC/0.730575, 0.624083, 0.526782,
        0.804889, 0.677061, 0.529442,
        0.785305, 0.802311, 0.508981,
        0.574068, 0.646785, 0.717143/
  EXTERNAL SIMATADD
  CALL SIMATADD('C', 'N', M, N, ALPHA, A, LDA, BC, LDB)
  END

 * Output BC:
 *       1.847379, 2.007814, 1.594354,
 *       2.146696, 1.988263, 1.614650,
 *       2.272644, 2.732482, 2.495982,
 *       1.821566, 1.822019, 2.649999
```

6.4.8.4.4.9 ?omatsub

矩阵缩放后out-place减计算。

即 $C := alpha * op(A) - beta * op(B)$ 。

op可取值： $op(A) = A, A^T, conj(A), conj(A^T)$ 。

接口定义

C interface：

void cblas_somatsub(const enum CBLAS_ORDER ordering, const enum CBLAS_TRANSPOSE TransA, const enum CBLAS_TRANSPOSE TransB, const BLASINT m, const BLASINT n, const float alpha, float *A, const BLASINT lda, const float beta, float *B, const BLASINT ldb, float *C, const BLASINT ldc);

void cblas_domatsub(const enum CBLAS_ORDER ordering, const enum CBLAS_TRANSPOSE TransA, const enum CBLAS_TRANSPOSE TransB, const BLASINT m, const BLASINT n, const double alpha, double *A, const BLASINT lda, const double beta, double *B, const BLASINT ldb, double *C, const BLASINT ldc);

void cblas_comatsub(const enum CBLAS_ORDER ordering, const enum CBLAS_TRANSPOSE TransA, const enum CBLAS_TRANSPOSE TransB, const

 版权所有 © 华为技术有限公司

BLASINT m, const BLASINT n, const float *alpha, float *A, const BLASINT lda, const float *beta, float *B, const BLASINT ldb, float *C, const BLASINT ldc);

void cblas_zomatsub(const enum CBLAS_ORDER ordering, const enum CBLAS_TRANSPOSE TransA, const enum CBLAS_TRANSPOSE TransB, const BLASINT m, const BLASINT n, const double *alpha, double *A, const BLASINT lda, const double *beta, double *B, const BLASINT ldb, double *C, const BLASINT ldc);

Fortran interface：

CALL SOMATSUB(ORDER, TRANSA, TRANSB, M, N, ALPHA, A, LDA, BETA, B, LDB, C, LDC)

CALL DOMATSUB(ORDER, TRANSA, TRANSB, M, N, ALPHA, A, LDA, BETA, B, LDB, C, LDC)

CALL COMATSUB(ORDER, TRANSA, TRANSB, M, N, ALPHA, A, LDA, BETA, B, LDB, C, LDC)

CALL ZOMATSUB(ORDER, TRANSA, TRANSB, M, N, ALPHA, A, LDA, BETA, B, LDB, C, LDC)

参数

参数名	类型	描述	输入/输出
order	枚举类型CBLAS_ORDER	表示矩阵是行主序或列主序。	输入
TransA	枚举类型CBLAS_TRANSPOSE	矩阵A为常规矩阵，转置矩阵，共轭矩阵或转置共轭矩阵。 • 如果TransA = CblasNoTrans，op(A) = A。 • 如果TransA = CblasTrans，op(A) = A'。 • 如果TransA = CblasConjNoTrans，op(A) = conjg(A)。 • 如果TransA = CblasConjTrans，op(A) = conjg(A')。	输入

版权所有 © 华为技术有限公司

参数名	类型	描述	输入/输出
TransB	枚举类型CBLAS_TRANSPOSE	矩阵B为常规矩阵，转置矩阵，共轭矩阵或转置共轭矩阵。 ● 如果TransB = CblasNoTrans，op(B) = B。 ● 如果TransB = CblasTrans，op(B) = B'。 ● 如果TransB = CblasConjNoTrans，op(B) = conjg(B)。 ● 如果TransB = CblasConjTrans，op(B) = conjg(B')。	输入
M	整型数	矩阵op(A)、op(B)、C的行数。	输入
N	整型数	矩阵op(A)、op(B)、C的列数。	输入
alpha	● 在somatsub中是单精度浮点类型。 ● 在domatsub中是双精度浮点类型。 ● 在comatsub中是单精度复数类型。 ● 在zomatsub中是双精度复数类型。	表示乘法系数。	输入
A	● 在somatsub中是单精度浮点类型。 ● 在domatsub中是双精度浮点类型。 ● 在comatsub中是单精度复数类型。 ● 在zomatsub中是双精度复数类型。	矩阵A。	输入
lda	整型数	● 矩阵A为列存，TransA = CblasNoTrans，lda至少max(1, m)，否则max(1, n)。 ● 矩阵A为行存，TransA = CblasNoTrans，lda至少max(1, n)，否则max(1, m)。	输入

 版权所有 © 华为技术有限公司

参数名	类型	描述	输入/输出
beta	• 在somatsub中是单精度浮点类型。 • 在domatsub中是双精度浮点类型。 • 在comatsub中是单精度复数类型。 • 在zomatsub中是双精度复数类型。	表示乘法系数。	输入
B	• 在somatsub中是单精度浮点类型。 • 在domatsub中是双精度浮点类型。 • 在comatsub中是单精度复数类型。 • 在zomatsub中是双精度复数类型。	矩阵B。	输入
ldb	整型数	• 矩阵为列存，TransB = CblasNoTrans，ldb至少max(1, m)，否则max(1, n)。 • 矩阵为行存，TransB = CblasNoTrans，ldb至少max(1, n)，否则max(1, m)。	输入
C	• 在somatsub中是单精度浮点类型。 • 在domatsub中是双精度浮点类型。 • 在comatsub中是单精度复数类型。 • 在zomatsub中是双精度复数类型。	矩阵C。	输出
ldc	整型数	矩阵为列存，ldc至少max(1, m)，否则max(1, n)。	输入

依赖

#include "kblas.h"

示例

C interface：

 版权所有 © 华为技术有限公司

```
int m = 4;
int n = 3;
int lda = 4;
int ldb = 4;
int ldc = 4;
float alpha = 2.0;
float beta = 1.0;

/**
 *    A:
 *        0.878666, 0.587949, 0.911689,
 *        0.537057, 0.738171, 0.775693,
 *        0.933167, 0.700325, 0.563810,
 *        0.712020, 0.606780, 0.538903
 *    B:
 *        0.780532, 0.649196, 0.703486,
 *        0.854725, 0.511782, 0.570734,
 *        0.550857, 0.832913, 0.937889,
 *        0.838400, 0.885429, 0.970482
 *    C:
 *        0, 0, 0,
 *        0, 0, 0,
 *        0, 0, 0,
 *        0, 0, 0
 */

float a[12] = {0.878666, 0.587949, 0.911689,
               0.537057, 0.738171, 0.775693,
               0.933167, 0.700325, 0.563810,
               0.712020, 0.606780, 0.538903
};

float b[12] = {0.780532, 0.649196, 0.703486,
               0.854725, 0.511782, 0.570734,
               0.550857, 0.832913, 0.937889,
               0.838400, 0.885429, 0.970482
};

float c[12] = {0};

cblas_somatsub(CblasColMajor, CblasNoTrans, CblasNoTrans, m, n, alpha, a, lda, beta, b, ldb, c, ldc);

/**
 * Output C:
 *        0.976800, 0.526701, 1.119892,
 *        0.219388, 0.964561, 0.980653,
 *        1.315476, 0.567737, 0.189731,
 *        0.585641, 0.328131, 0.107325
 */
```

Fortran interface:

```
  PROGRAM OMATSUB
  INTEGER :: M=4, N=3
  INTEGER :: LDA=4, LDB=4, LDC=4
  REAL(4) :: ALPHA=2.0
  REAL(4) :: BETA=1.0
  REAL(4) :: A(4, 3), B(4, 3), C(4, 3)
  DATA A/0.878666, 0.587949, 0.911689,
        0.537057, 0.738171, 0.775693,
        0.933167, 0.700325, 0.563810,
        0.712020, 0.606780, 0.538903/
  DATA B/0.780532, 0.649196, 0.703486,
        0.854725, 0.511782, 0.570734,
        0.550857, 0.832913, 0.937889,
        0.838400, 0.885429, 0.970482/
  DATA C/0.0, 0.0, 0.0,
        0.0, 0.0, 0.0,
        0.0, 0.0, 0.0/
  EXTERNAL SOMATSUB
```

 版权所有 © 华为技术有限公司

```
    CALL SOMATSUB('C', 'N', 'N', M, N, ALPHA, A, LDA, BETA, B, LDB, C, LDC)
    END

  * Output BC:
  *       0.976800, 0.526701, 1.119892,
  *       0.219388, 0.964561, 0.980653,
  *       1.315476, 0.567737, 0.189731,
  *       0.585641, 0.328131, 0.107325
```

6.4.8.4.4.10 ?imatsub

矩阵缩放后in-place减计算。

即 $BC := BC - alpha * op(A)$ 。

op可取值： $op(A) = A, A^T, conj(A), conj(A^T)$ 。

接口定义

C interface：

void cblas_simatsub(const enum CBLAS_ORDER ordering, const enum CBLAS_TRANSPOSE TransA, const BLASINT m, const BLASINT n, const float alpha, float *A, const BLASINT lda, float *BC, const BLASINT ldb);

void cblas_dimatsub(const enum CBLAS_ORDER ordering, const enum CBLAS_TRANSPOSE TransA, const BLASINT m, const BLASINT n, const double alpha, double *A, const BLASINT lda, double *BC, const BLASINT ldb);

void cblas_cimatsub(const enum CBLAS_ORDER ordering, const enum CBLAS_TRANSPOSE TransA, const BLASINT m, const BLASINT n, const float *alpha, float *A, const BLASINT lda, float *BC, const BLASINT ldb);

void cblas_zimatsub(const enum CBLAS_ORDER ordering, const enum CBLAS_TRANSPOSE TransA, const BLASINT m, const BLASINT n, const double *alpha, double *A, const BLASINT lda, double *BC, const BLASINT ldb);

Fortran interface：

CALL SIMATSUB(ORDER, TRANSA, M, N, ALPHA, A, LDA, BC, LDB)

CALL DIMATSUB(ORDER, TRANSA, M, N, ALPHA, A, LDA, BC, LDB)

CALL CIMATSUB(ORDER, TRANSA, M, N, ALPHA, A, LDA, BC, LDB)

CALL ZIMATSUB(ORDER, TRANSA, M, N, ALPHA, A, LDA, BC, LDB)

参数

参数名	类型	描述	输入/输出
order	枚举类型CBLAS_ORDER	表示矩阵是行主序或列主序。	输入

 版权所有 © 华为技术有限公司

参数名	类型	描述	输入/输出
TransA	枚举类型CBLAS_TRANSPOSE	矩阵A为常规矩阵，转置矩阵，共轭矩阵或转置共轭矩阵。 ● 如果TransA = CblasNoTrans，op(A) = A。 ● 如果TransA = CblasTrans，op(A) = A'。 ● 如果TransA = CblasConjNoTrans，op(A) = conjg(A)。 ● 如果TransA = CblasConjTrans，op(A) = conjg(A')。	输入
M	整型数	矩阵op(A)、BC的行。	输入
N	整型数	矩阵op(A)、BC的列。	输入
alpha	● 在simatsub中是单精度浮点类型。 ● 在dimatsub中是双精度浮点类型。 ● 在cimatsub中是单精度复数类型。 ● 在zimatsub中是双精度复数类型。	表示乘法系数。	输入
A	● 在simatsub中是单精度浮点类型。 ● 在dimatsub中是双精度浮点类型。 ● 在cimatsub中是单精度复数类型。 ● 在zimatsub中是双精度复数类型。	矩阵A。	输入
lda	整型数	● 矩阵A为列存，TransA = CblasNoTrans，lda至少max(1, m)，否则max(1, n)。 ● 矩阵A为行存，TransA = CblasNoTrans，lda至少max(1, n)，否则max(1, m)。	输入

版权所有 © 华为技术有限公司

参数名	类型	描述	输入/输出
BC	• 在simatsub中是单精度浮点类型。 • 在dimatsub中是双精度浮点类型。 • 在cimatsub中是单精度复数类型。 • 在zimatsub中是双精度复数类型。	矩阵BC。	输入
ldb	整型数	• 矩阵为列存，TransB = CblasNoTrans，ldb至少max(1, m)，否则max(1, n)。 • 矩阵为行存，TransB = CblasNoTrans，ldb至少max(1, n)，否则max(1, m)。	输入

依赖

#include "kblas.h"

示例

C interface：

```
int m = 4;
int n = 3;
int lda = 4;
int ldb = 4;
float alpha = 2.0;

/**
 *    A:
 *        0.878666, 0.587949, 0.911689,
 *        0.537057, 0.738171, 0.775693,
 *        0.933167, 0.700325, 0.563810,
 *        0.712020, 0.606780, 0.538903
 *    BC:
 *        0.780532, 0.649196, 0.703486,
 *        0.854725, 0.511782, 0.570734,
 *        0.550857, 0.832913, 0.937889,
 *        0.838400, 0.885429, 0.970482
 */

float a[12] = {0.878666, 0.587949, 0.911689,
               0.537057, 0.738171, 0.775693,
               0.933167, 0.700325, 0.563810,
               0.712020, 0.606780, 0.538903
};

float bc[12] = {0.780532, 0.649196, 0.703486,
               0.854725, 0.511782, 0.570734,
               0.550857, 0.832913, 0.937889,
```

 版权所有 © 华为技术有限公司

```
            0.838400, 0.885429, 0.970482
};

float c[12] = {0};

cblas_simatsub(CblasColMajor, CblasNoTrans, m, n, alpha, a, lda, bc, ldb);

/**
 * Output C:
 *        -0.976800, -0.526701, -1.119892,
 *        -0.219388, -0.964561, -0.980653,
 *        -1.315476, -0.567737, -0.189731,
 *        -0.585641, -0.328131, -0.107325
 */
```

Fortran interface：

```
  PROGRAM IMATSUB
  INTEGER :: M=4, N=3
  INTEGER :: LDA=4, LDB=4
  REAL(4) :: ALPHA=2.0
  REAL(4) :: A(4, 3), BC(4, 3), C(4, 3)
  DATA A/0.878666, 0.587949, 0.911689,
        0.537057, 0.738171, 0.775693,
        0.933167, 0.700325, 0.563810,
        0.712020, 0.606780, 0.538903/
  DATA BC/0.780532, 0.649196, 0.703486,
        0.854725, 0.511782, 0.570734,
        0.550857, 0.832913, 0.937889,
        0.838400, 0.885429, 0.970482/
  EXTERNAL SIMATSUB
  CALL SIMATSUB('C', 'N', M, N, ALPHA, A, LDA, BC, LDB)
  END

 * Output BC:
 *        0.694343, 0.882727, 0.822963,
 *        0.859743, 0.543875, 0.927105,
 *        0.516767, 0.659971, 0.950618,
 *        0.849312, 0.576374, 0.557569
```

6.4.8.4.4.11 ?omatmul

矩阵缩放后out-place乘计算。

即 $C := alpha * op(A) \cdot op(B)$ 。

op可取值： $op(A) = A, A^T, conj(A), conj(A^T)$ 。

接口定义

C interface：

void cblas_somatmul(const enum CBLAS_ORDER ordering, const enum CBLAS_TRANSPOSE TransA, const enum CBLAS_TRANSPOSE TransB, const BLASINT m, const BLASINT n, const float alpha, float *A, const BLASINT lda, float *B, const BLASINT ldb, float *C, const BLASINT ldc);

void cblas_domatmul(const enum CBLAS_ORDER ordering, const enum CBLAS_TRANSPOSE TransA, const enum CBLAS_TRANSPOSE TransB, const BLASINT m, const BLASINT n, const double alpha, double *A, const BLASINT lda, double *B, const BLASINT ldb, double *C, const BLASINT ldc);

 版权所有 © 华为技术有限公司

void cblas_comatmul(const enum CBLAS_ORDER ordering, const enum CBLAS_TRANSPOSE TransA, const enum CBLAS_TRANSPOSE TransB, const BLASINT m, const BLASINT n, const float *alpha, float *A, const BLASINT lda, float *B, const BLASINT ldb, float *C, const BLASINT ldc);

void cblas_zomatmul(const enum CBLAS_ORDER ordering, const enum CBLAS_TRANSPOSE TransA, const enum CBLAS_TRANSPOSE TransB, const BLASINT m, const BLASINT n, const double *alpha, double *A, const BLASINT lda, double *B, const BLASINT ldb, double *C, const BLASINT ldc);

Fortran interface：

CALL SOMATMUL(ORDER, TRANSA, TRANSB, M, N, ALPHA, A, LDA, B, LDB, C, LDC)

CALL DOMATMUL(ORDER, TRANSA, TRANSB, M, N, ALPHA, A, LDA, B, LDB, C, LDC)

CALL COMATMUL(ORDER, TRANSA, TRANSB, M, N, ALPHA, A, LDA, B, LDB, C, LDC)

CALL ZOMATMUL(ORDER, TRANSA, TRANSB, M, N, ALPHA, A, LDA, B, LDB, C, LDC)

参数

参数名	类型	描述	输入/输出
order	枚举类型CBLAS_ORDER	表示矩阵是行主序或列主序。	输入
TransA	枚举类型CBLAS_TRANSPOSE	矩阵A为常规矩阵，转置矩阵或共轭矩阵。 ● 如果TransA = CblasNoTrans，op(A) = A。 ● 如果TransA = CblasTrans，op(A) = A'。 ● 如果TransA = CblasConjNoTrans，op(A) = conjg(A)。 ● 如果TransA = CblasConjTrans，op(A) = conjg(A')。	输入

 版权所有 © 华为技术有限公司

参数名	类型	描述	输入/输出
TransB	枚举类型CBLAS_TRANSPOSE	矩阵B为常规矩阵，转置矩阵或共轭矩阵。 ● 如果TransB = CblasNoTrans，op(B) = B。 ● 如果TransB = CblasTrans，op(B) = B'。 ● 如果TransB = CblasConjNoTrans，op(B) = conjg(B)。 ● 如果TransB = CblasConjTrans，op(B) = conjg(B')。	输入
M	整型数	矩阵op(A)、op(B)、C的行数。	输入
N	整型数	矩阵op(A)、op(B)、C的列数。	输入
alpha	● 在somatmul中是单精度浮点类型。 ● 在domatmul中是双精度浮点类型。 ● 在comatmul中是单精度复数类型。 ● 在zomatmul中是双精度复数类型。	表示乘法系数。	输入
A	● 在somatmul中是单精度浮点类型。 ● 在domatmul中是双精度浮点类型。 ● 在comatmul中是单精度复数类型。 ● 在zomatmul中是双精度复数类型。	矩阵A。	输入
lda	整型数	● 矩阵A为列存，TransA = CblasNoTrans，lda至少max(1, m)，否则max(1, n)。 ● 矩阵A为行存，TransA = CblasNoTrans，lda至少max(1, n)，否则max(1, m)。	输入

 版权所有 © 华为技术有限公司

参数名	类型	描述	输入/输出
B	• 在somatmul中是单精度浮点类型。 • 在domatmul中是双精度浮点类型。 • 在comatmul中是单精度复数类型。 • 在zomatmul中是双精度复数类型。	矩阵B。	输入
ldb	整型数	• 矩阵为列存，TransB = CblasNoTrans，ldb至少max(1, m)，否则max(1, n)。 • 矩阵为行存，TransB = CblasNoTrans，ldb至少max(1, n)，否则max(1, m)。	输入
C	• 在somatmul中是单精度浮点类型。 • 在domatmul中是双精度浮点类型。 • 在comatmul中是单精度复数类型。 • 在zomatmul中是双精度复数类型。	矩阵C。	输出
ldc	整型数	矩阵为列存，ldc至少max(1, m)，否则max(1, n)。	输入

依赖

#include "kblas.h"

示例

C interface：

```
int m = 4;
int n = 3;
int lda = 4;
int ldb = 4;
int ldc = 4;
float alpha = 2.0;

/**
 *    A:
 *        -0.399988, -0.211286, 0.226351,
 *        0.057980, 0.185737, 0.015954,
```

 版权所有 © 华为技术有限公司

```
 *         -0.181192, 0.092167, -0.125647,
 *         0.206140, -0.441963, -0.307887
 *    B:
 *         0.274879, -0.184964, -0.331672,
 *         -0.476422, 0.318246, 0.295214,
 *         -0.067229, 0.314890, -0.072014,
 *         -0.088924, -0.222191, -0.121713,
 *    C:
 *         0, 0, 0,
 *         0, 0, 0,
 *         0, 0, 0,
 *         0, 0, 0
 */

float a[12] = {-0.399988, -0.211286, 0.226351,
               0.057980, 0.185737, 0.015954,
               -0.181192, 0.092167, -0.125647,
               0.206140, -0.441963, -0.307887
};

float b[12] = {0.274879, -0.184964, -0.331672,
               -0.476422, 0.318246, 0.295214,
               -0.067229, 0.314890, -0.072014,
               -0.088924, -0.222191, -0.121713,
};

float c[12] = {0};

cblas_somatmul(CblasColMajor, CblasNoTrans, CblasNoTrans, m, n, alpha, a, lda, b, ldb, c, ldc);

/**
 * Output C:
 *         -0.219897, 0.078161, -0.150148,
 *         -0.055246, 0.118220, 0.009419,
 *         0.024363, 0.058045, 0.018097,
 *         -0.036661, 0.196401, 0.074947
 */
```

Fortran interface：

```
  PROGRAM OMATMUL
  INTEGER :: M=4, N=3
  INTEGER :: LDA=4, LDB=4, LDC=4
  REAL(4) :: ALPHA=2.0
  REAL(4) :: A(4, 3), B(4, 3), C(4, 3)
  DATA A/-0.399988, -0.211286, 0.226351,
         0.057980, 0.185737, 0.015954,
         -0.181192, 0.092167, -0.125647,
         0.206140, -0.441963, -0.307887/
  DATA B/0.274879, -0.184964, -0.331672,
         -0.476422, 0.318246, 0.295214,
         -0.067229, 0.314890, -0.072014,
         -0.088924, -0.222191, -0.121713/
  DATA C/0.0, 0.0, 0.0,
         0.0, 0.0, 0.0,
         0.0, 0.0, 0.0/
  EXTERNAL SOMATMUL
  CALL SOMATMUL('C', 'N', 'N', M, N, ALPHA, A, LDA, B, LDB, C, LDC)
  END

 * Output C:
 *         -0.219897, 0.078161, -0.150148,
 *         -0.055246, 0.118220, 0.009419,
 *         0.024363, 0.058045, 0.018097,
 *         -0.036661, 0.196401, 0.074947
```

6.4.8.4.4.12 ?imatmul

矩阵缩放后in-place乘计算。

 版权所有 © 华为技术有限公司

即 $BC := alpha * BC \cdot op(A)$ 。

op可取值： $op(A) = A, A^T, conj(A), conj(A^T)$ 。

接口定义

C interface：

void cblas_simatmul(const enum CBLAS_ORDER ordering, const enum CBLAS_TRANSPOSE TransA, const BLASINT m, const BLASINT n, const float alpha, float *A, const BLASINT lda, float *BC, const BLASINT ldb);

void cblas_dimatmul(const enum CBLAS_ORDER ordering, const enum CBLAS_TRANSPOSE TransA, const BLASINT m, const BLASINT n, const double alpha, double *A, const BLASINT lda, double *BC, const BLASINT ldb);

void cblas_cimatmul(const enum CBLAS_ORDER ordering, const enum CBLAS_TRANSPOSE TransA, const BLASINT m, const BLASINT n, const float *alpha, float *A, const BLASINT lda, float *BC, const BLASINT ldb);

void cblas_zimatmul(const enum CBLAS_ORDER ordering, const enum CBLAS_TRANSPOSE TransA, const BLASINT m, const BLASINT n, const double *alpha, double *A, const BLASINT lda, double *BC, const BLASINT ldb);

Fortran interface：

CALL SIMATMUL(ORDER, TRANSA, M, N, ALPHA, A, LDA, BC, LDB)

CALL DIMATMUL(ORDER, TRANSA, M, N, ALPHA, A, LDA, BC, LDB)

CALL CIMATMUL(ORDER, TRANSA, M, N, ALPHA, A, LDA, BC, LDB)

CALL ZIMATMUL(ORDER, TRANSA, M, N, ALPHA, A, LDA, BC, LDB)

参数

参数名	类型	描述	输入/输出
order	枚举类型CBLAS_ORDER	表示矩阵是行主序或列主序。	输入

 版权所有 © 华为技术有限公司

参数名	类型	描述	输入/输出
TransA	枚举类型CBLAS_TRANSPOSE	矩阵A为常规矩阵，转置矩阵或共轭矩阵。 • 如果TransA = CblasNoTrans，op(A) = A。 • 如果TransA = CblasTrans，op(A) = A'。 • 如果TransA = CblasConjNoTrans，op(A) = conjg(A)。 • 如果TransA = CblasConjTrans，op(A) = conjg(A')。	输入
M	整型数	矩阵op(A)、BC的行。	输入
N	整型数	矩阵op(A)、BC的列。	输入
alpha	• 在simatmul中是单精度浮点类型。 • 在dimatmul中是双精度浮点类型。 • 在cimatmul中是单精度复数类型。 • 在zimatmul中是双精度复数类型。	表示乘法系数。	输入
A	• 在simatmul中是单精度浮点类型。 • 在dimatmul中是双精度浮点类型。 • 在cimatmul中是单精度复数类型。 • 在zimatmul中是双精度复数类型。	矩阵A。	输入
lda	整型数	• 矩阵A为列存，TransA = CblasNoTrans，lda至少max(1, m)，否则max(1, n)。 • 矩阵A为行存，TransA = CblasNoTrans，lda至少max(1, n)，否则max(1, m)。	输入

版权所有 © 华为技术有限公司

参数名	类型	描述	输入/输出
BC	• 在simatmul中是单精度浮点类型。 • 在dimatmul中是双精度浮点类型。 • 在cimatmul中是单精度复数类型。 • 在zimatmul中是双精度复数类型。	矩阵BC。	输入
ldb	整型数	• 矩阵为列存，TransB = CblasNoTrans，ldb至少max(1, m)，否则max(1, n)。 • 矩阵为行存，TransB = CblasNoTrans，ldb至少max(1, n)，否则max(1, m)。	输入

依赖

#include "kblas.h"

示例

C interface：

```
int m = 4;
int n = 3;
int lda = 4;
int ldb = 4;
float alpha = 2.0;

/**
 *    A:
 *        -0.399988, -0.211286, 0.226351,
 *        0.057980, 0.185737, 0.015954,
 *        -0.181192, 0.092167, -0.125647,
 *        0.206140, -0.441963, -0.307887
 *    BC:
 *        0.274879, -0.184964, -0.331672,
 *        -0.476422, 0.318246, 0.295214,
 *        -0.067229, 0.314890, -0.072014,
 *        -0.088924, -0.222191, -0.121713
 */

float a[12] = {-0.399988, -0.211286, 0.226351,
               0.057980, 0.185737, 0.015954,
               -0.181192, 0.092167, -0.125647,
               0.206140, -0.441963, -0.307887
};

float bc[12] = {0.274879, -0.184964, -0.331672,
               -0.476422, 0.318246, 0.295214,
               -0.067229, 0.314890, -0.072014,
```

 版权所有 © 华为技术有限公司

```
            -0.088924, -0.222191, -0.121713,
};

cblas_simatmul(CblasColMajor, CblasNoTrans, m, n, alpha, a, lda, bc, ldb);

/**
 * Output BC:
 *        -0.219897, 0.078161, -0.150148,
 *        -0.055246, 0.118220, 0.009419,
 *        0.024363, 0.058045, 0.018097,
 *        -0.036661, 0.196401, 0.074947
 */
```

Fortran interface：

```
  PROGRAM IMATMUL
  INTEGER :: M=4, N=3
  INTEGER :: LDA=4, LDB=4
  REAL(4) :: ALPHA=2.0
  REAL(4) :: A(4, 3), BC(4, 3)
  DATA A/-0.399988, -0.211286, 0.226351,
        0.057980, 0.185737, 0.015954,
        -0.181192, 0.092167, -0.125647,
        0.206140, -0.441963, -0.307887/
  DATA BC/0.274879, -0.184964, -0.331672,
        -0.476422, 0.318246, 0.295214,
        -0.067229, 0.314890, -0.072014,
        -0.088924, -0.222191, -0.121713/
  EXTERNAL SIMATMUL
  CALL SIMATMUL('C', 'N', M, N, ALPHA, A, LDA, BC, LDB)
  END

 * Output BC:
 *        -0.219897, 0.078161, -0.150148,
 *        -0.055246, 0.118220, 0.009419,
 *        0.024363, 0.058045, 0.018097,
 *        -0.036661, 0.196401, 0.074947
```

6.4.8.4.4.13 ?omatdiv

矩阵缩放后out-place除计算。

即 $C := alpha * op(A) / op(B)$ 。

op可取值： $op(A) = A, A^T, conj(A), conj(A^T)$ 。

接口定义

C interface：

void cblas_somatdiv(const enum CBLAS_ORDER ordering, const enum CBLAS_TRANSPOSE TransA, const enum CBLAS_TRANSPOSE TransB, const BLASINT m, const BLASINT n, const float alpha, float *A, const BLASINT lda, float *B, const BLASINT ldb, float *C, const BLASINT ldc);

void cblas_domatdiv(const enum CBLAS_ORDER ordering, const enum CBLAS_TRANSPOSE TransA, const enum CBLAS_TRANSPOSE TransB, const BLASINT m, const BLASINT n, const double alpha, double *A, const BLASINT lda, double *B, const BLASINT ldb, double *C, const BLASINT ldc);

void cblas_comatdiv(const enum CBLAS_ORDER ordering, const enum CBLAS_TRANSPOSE TransA, const enum CBLAS_TRANSPOSE TransB, const

 版权所有 © 华为技术有限公司

BLASINT m, const BLASINT n, const float *alpha, float *A, const BLASINT lda, float *B, const BLASINT ldb, float *C, const BLASINT ldc);

void cblas_zomatdiv(const enum CBLAS_ORDER ordering, const enum CBLAS_TRANSPOSE TransA, const enum CBLAS_TRANSPOSE TransB, const BLASINT m, const BLASINT n, const double *alpha, double *A, const BLASINT lda, double *B, const BLASINT ldb, double *C, const BLASINT ldc);

Fortran interface：

CALL SOMATDIV(ORDER, TRANSA, TRANSB, M, N, ALPHA, A, LDA, B, LDB, C, LDC)

CALL DOMATDIV(ORDER, TRANSA, TRANSB, M, N, ALPHA, A, LDA, B, LDB, C, LDC)

CALL COMATDIV(ORDER, TRANSA, TRANSB, M, N, ALPHA, A, LDA, B, LDB, C, LDC)

CALL ZOMATDIV(ORDER, TRANSA, TRANSB, M, N, ALPHA, A, LDA, B, LDB, C, LDC)

参数

参数名	类型	描述	输入/输出
order	枚举类型CBLAS_ORDER	表示矩阵是行主序或列主序。	输入
TransA	枚举类型CBLAS_TRANSPOSE	矩阵A为常规矩阵，转置矩阵或共轭矩阵。 ● 如果TransA = CblasNoTrans，op(A) = A。 ● 如果TransA = CblasTrans，op(A) = A'。 ● 如果TransA = CblasConjNoTrans，op(A) = conjg(A)。 ● 如果TransA = CblasConjTrans，op(A) = conjg(A')。	输入

 版权所有 © 华为技术有限公司

参数名	类型	描述	输入/输出
TransB	枚举类型CBLAS_TRANSPOSE	矩阵B为常规矩阵，转置矩阵或共轭矩阵。 • 如果TransB = CblasNoTrans，op(B) = B。 • 如果TransB = CblasTrans，op(B) = B'。 • 如果TransB = CblasConjNoTrans，op(B) = conjg(B)。 • 如果TransB = CblasConjTrans，op(B) = conjg(B')。	输入
M	整型数	矩阵op(A)、op(B)、C的行数。	输入
N	整型数	矩阵op(A)、op(B)、C的列数。	输入
alpha	• 在somatdiv中是单精度浮点类型。 • 在domatdiv中是双精度浮点类型。 • 在comatdiv中是单精度复数类型。 • 在zomatdiv中是双精度复数类型。	表示乘法系数。	输入
A	• 在somatdiv中是单精度浮点类型。 • 在domatdiv中是双精度浮点类型。 • 在comatdiv中是单精度复数类型。 • 在zomatdiv中是双精度复数类型。	矩阵A。	输入
lda	整型数	• 矩阵A为列存，TransA = CblasNoTrans，lda至少max(1, m)，否则max(1, n)。 • 矩阵A为行存，TransA = CblasNoTrans，lda至少max(1, n)，否则max(1, m)。	输入

版权所有 © 华为技术有限公司

参数名	类型	描述	输入/输出
B	• 在somatdiv中是单精度浮点类型。 • 在domatdiv中是双精度浮点类型。 • 在comatdiv中是单精度复数类型。 • 在zomatdiv中是双精度复数类型。	矩阵B。	输入
ldb	整型数	• 矩阵为列存，TransB = CblasNoTrans，ldb至少max(1, m)，否则max(1, n)。 • 矩阵为行存，TransB = CblasNoTrans，ldb至少max(1, n)，否则max(1, m)。	输入
C	• 在somatdiv中是单精度浮点类型。 • 在domatdiv中是双精度浮点类型。 • 在comatdiv中是单精度复数类型。 • 在zomatdiv中是双精度复数类型。	矩阵C。	输出
ldc	整型数	矩阵为列存，ldc至少max(1, m)，否则max(1, n)。	输入

依赖

#include "kblas.h"

示例

C interface：

```
int m = 4;
int n = 3;
int lda = 4;
int ldb = 4;
int ldc = 4;
float alpha = 2.0;

/**
 *    A:
 *        0.645728, 0.664078, 0.775762,
 *        0.806953, 0.601041, 0.714864,
```

 版权所有 © 华为技术有限公司

```
 *         0.915711, 0.803360, 0.562129,
 *         0.522928, 0.647835, 0.824213
 *    B:
 *         0.718160, 0.604610, 0.991237,
 *         0.676218, 0.818167, 0.722511,
 *         0.539246, 0.880499, 0.814675,
 *         0.543172, 0.862320, 0.731872
 *    C:
 *         0, 0, 0,
 *         0, 0, 0,
 *         0, 0, 0,
 *         0, 0, 0
 */

float a[12] = {0.645728, 0.664078, 0.775762,
               0.806953, 0.601041, 0.714864,
               0.915711, 0.803360, 0.562129,
               0.522928, 0.647835, 0.824213
};

float b[12] = {0.718160, 0.604610, 0.991237,
               0.676218, 0.818167, 0.722511,
               0.539246, 0.880499, 0.814675,
               0.543172, 0.862320, 0.731872
};

float c[12] = {0};

cblas_somatdiv(CblasColMajor, CblasNoTrans, CblasNoTrans, m, n, alpha, a, lda, b, ldb, c, ldc);

/**
 * Output C:
 *         1.798285, 2.196716, 1.565241,
 *         2.386664, 1.469238, 1.978835,
 *         3.396264, 1.824785, 1.380008,
 *         1.925460, 1.502539, 2.252343
 */
```

Fortran interface：

```
  PROGRAM OMATDIV
  INTEGER :: M=4, N=3
  INTEGER :: LDA=4, LDB=4, LDC=4
  REAL(4) :: ALPHA=2.0
  REAL(4) :: A(4, 3), B(4, 3), C(4, 3)
  DATA A/0.645728, 0.664078, 0.775762,
         0.806953, 0.601041, 0.714864,
         0.915711, 0.803360, 0.562129,
         0.522928, 0.647835, 0.824213/
  DATA B/0.718160, 0.604610, 0.991237,
         0.676218, 0.818167, 0.722511,
         0.539246, 0.880499, 0.814675,
         0.543172, 0.862320, 0.731872/
  DATA C/0.0, 0.0, 0.0,
         0.0, 0.0, 0.0,
         0.0, 0.0, 0.0/
  EXTERNAL SOMATDIV
  CALL SOMATDIV('C', 'N', 'N', M, N, ALPHA, A, LDA, B, LDB, C, LDC)
  END

 * Output C:
 *         1.798285, 2.196716, 1.565241,
 *         2.386664, 1.469238, 1.978835,
 *         3.396264, 1.824785, 1.380008,
 *         1.925460, 1.502539, 2.252343
```

6.4.8.4.4.14 ?imatdiv

矩阵缩放后in-place除计算。

 版权所有 © 华为技术有限公司

即 $BC := alpha * BC / op(A)$ 。

op可取值： $op(A) = A, A^T, conj(A), conj(A^T)$ 。

接口定义

C interface：

void cblas_simatdiv(const enum CBLAS_ORDER ordering, const enum CBLAS_TRANSPOSE TransA, const BLASINT m, const BLASINT n, const float alpha, float *A, const BLASINT lda, float *BC, const BLASINT ldb);

void cblas_dimatdiv(const enum CBLAS_ORDER ordering, const enum CBLAS_TRANSPOSE TransA, const BLASINT m, const BLASINT n, const double alpha, double *A, const BLASINT lda, double *BC, const BLASINT ldb);

void cblas_cimatdiv(const enum CBLAS_ORDER ordering, const enum CBLAS_TRANSPOSE TransA, const BLASINT m, const BLASINT n, const float *alpha, float *A, const BLASINT lda, float *BC, const BLASINT ldb);

void cblas_zimatdiv(const enum CBLAS_ORDER ordering, const enum CBLAS_TRANSPOSE TransA, const BLASINT m, const BLASINT n, const double *alpha, double *A, const BLASINT lda, double *BC, const BLASINT ldb);

Fortran interface：

CALL SIMATDIV(ORDER, TRANSA, M, N, ALPHA, A, LDA, BC, LDB)

CALL DIMATDIV(ORDER, TRANSA, M, N, ALPHA, A, LDA, BC, LDB)

CALL CIMATDIV(ORDER, TRANSA, M, N, ALPHA, A, LDA, BC, LDB)

CALL ZIMATDIV(ORDER, TRANSA, M, N, ALPHA, A, LDA, BC, LDB)

参数

参数名	类型	描述	输入/输出
order	枚举类型CBLAS_ORDER	表示矩阵是行主序或列主序。	输入

 版权所有 © 华为技术有限公司

参数名	类型	描述	输入/输出
TransA	枚举类型CBLAS_TRANSPOSE	矩阵A为常规矩阵，转置矩阵或共轭矩阵。 • 如果TransA = CblasNoTrans，op(A) = A。 • 如果TransA = CblasTrans，op(A) = A'。 • 如果TransA = CblasConjNoTrans，op(A) = conjg(A)。 • 如果TransA = CblasConjTrans，op(A) = conjg(A')。	输入
M	整型数	矩阵op(A)、BC的行。	输入
N	整型数	矩阵op(A)、BC的列。	输入
alpha	• 在simatdiv中是单精度浮点类型。 • 在dimatdiv中是双精度浮点类型。 • 在cimatdiv中是单精度复数类型。 • 在zimatdiv中是双精度复数类型。	表示乘法系数。	输入
A	• 在simatdiv中是单精度浮点类型。 • 在dimatdiv中是双精度浮点类型。 • 在cimatdiv中是单精度复数类型。 • 在zimatdiv中是双精度复数类型。	矩阵A。	输入
lda	整型数	• 矩阵A为列存，TransA = CblasNoTrans，lda至少max(1, m)，否则max(1, n)。 • 矩阵A为行存，TransA = CblasNoTrans，lda至少max(1, n)，否则max(1, m)。	输入

版权所有 © 华为技术有限公司

参数名	类型	描述	输入/输出
BC	• 在simatdiv中是单精度浮点类型。 • 在dimatdiv中是双精度浮点类型。 • 在cimatdiv中是单精度复数类型。 • 在zimatdiv中是双精度复数类型。	矩阵BC。	输入
ldb	整型数	• 矩阵为列存，TransB = CblasNoTrans，ldb至少max(1, m)，否则max(1, n)。 • 矩阵为行存，TransB = CblasNoTrans，ldb至少max(1, n)，否则max(1, m)。	输入

依赖

#include "kblas.h"

示例

C interface：

```
int m = 4;
int n = 3;
int lda = 4;
int ldb = 4;
float alpha = 2.0;

/**
 *    A:
 *        0.235257, 0.268674, 0.151790,
 *        -0.204541, 0.142802, 0.036246,
 *        -0.238498, 0.423810, 0.181002,
 *        -0.002402, -0.240821, -0.309177
 *    BC:
 *        -0.284033, -0.407091, 0.487585,
 *        -0.126079, 0.380664, -0.407300,
 *        0.243892, -0.259406, -0.257296,
 *        0.018376, 0.148356, 0.379532,
 */

float a[12] = {0.645728, 0.664078, 0.775762,
               0.806953, 0.601041, 0.714864,
               0.915711, 0.803360, 0.562129,
               0.522928, 0.647835, 0.824213
};

float bc[12] = {-2.414662, -3.030372, 6.424446,
                1.232803, 5.331337, -22.474379,
                -2.045228, -1.224161, -2.843022,
```

 版权所有 © 华为技术有限公司

```
                -15.298911, -1.232085, -2.455110
};

cblas_simatdiv(CblasColMajor, CblasNoTrans, m, n, alpha, a, lda, bc, ldb);

/**
 * Output BC:
 *        -2.414662, -3.030372, 6.424446,
 *        1.232803, 5.331337, -22.474379,
 *        -2.045228, -1.224161, -2.843022,
 *        -15.298911, -1.232085, -2.455110
 */
```

Fortran interface:

```
  PROGRAM IMATDIV
  INTEGER :: M=4, N=3
  INTEGER :: LDA=4, LDB=4
  REAL(4) :: ALPHA=2.0
  REAL(4) :: A(4, 3), BC(4, 3)
  DATA A/0.645728, 0.664078, 0.775762,
         0.806953, 0.601041, 0.714864,
         0.915711, 0.803360, 0.562129,
         0.522928, 0.647835, 0.824213/
  DATA BC/-2.414662, -3.030372, 6.424446,
          1.232803, 5.331337, -22.474379,
          -2.045228, -1.224161, -2.843022,
          -15.298911, -1.232085, -2.455110/

  EXTERNAL SIMATDIV
  CALL SIMATDIV('C', 'N', M, N, ALPHA, A, LDA, BC, LDB)
  END

 * Output BC:
 *        1.798285, 2.196716, 1.565241,
 *        2.386664, 1.469238, 1.978835,
 *        3.396264, 1.824785, 1.380008,
 *        1.925460, 1.502539, 2.252343
```

6.4.8.4.4.15 ?gemm_pack_get_size

返回存储pack后的矩阵所需的字节数。

接口定义

C interface:

size_t cblas_sgemm_pack_get_size(const enum CBLAS_IDENTIFIER identifier, const BLASINT m, const BLASINT n, const BLASINT k);

size_t cblas_dgemm_pack_get_size(const enum CBLAS_IDENTIFIER identifier, const BLASINT m, const BLASINT n, const BLASINT k);

size_t cblas_cgemm_pack_get_size(const enum CBLAS_IDENTIFIER identifier, const BLASINT m, const BLASINT n, const BLASINT k);

size_t cblas_zgemm_pack_get_size(const enum CBLAS_IDENTIFIER identifier, const BLASINT m, const BLASINT n, const BLASINT k);

size_t cblas_bgemm_pack_get_size(const enum CBLAS_IDENTIFIER identifier, const BLASINT m, const BLASINT n, const BLASINT k);

size_t cblas_sbgemm_pack_get_size(const enum CBLAS_IDENTIFIER identifier, const BLASINT m, const BLASINT n, const BLASINT k);

 版权所有 © 华为技术有限公司

参数

参数名	类型	描述	输入/输出
identifier	枚举类型CBLAS_IDENTIFIER	指定要pack的矩阵。 ● 如果identifier= CblasA，返回pack A矩阵所需字节数。 ● 如果identifier= CblasB，返回pack B矩阵所需字节数。	输入
m	整型数	需要pack的矩阵A的行。	输入
n	整型数	需要pack的矩阵B的列。	输入
k	整型数	需要pack的矩阵A的列和B的行。	输入

依赖

#include "kblas.h"

示例

C interface：

```
int m = 4, k = 3, n = 4;

size_t size = cblas_sgemm_pack_get_size(CblasA, m, n, k);
/*
 * Output size: 64
 */
```

6.4.8.4.4.16 ?gemm_pack

对矩阵进行pack操作并将其存储到已分配的缓冲区中。

即：$C = \text{alpha} * \text{op}(A) * \text{op}(B) + \text{beta} * C$。

op(X)可取值：

$$op(X)=X, op(X)=X^{T}, op(X)=conjg(X), op(X)=conjg(X^{T})$$

，alpha，beta为乘法系数，op(A)为m*k矩阵，op(B)为k*n矩阵，C为m*n矩阵。

接口定义

C interface：

void cblas_sgemm_pack(const enum CBLAS_ORDER order, const enum CBLAS_IDENTIFIER identifier, const enum CBLAS_TRANSPOSE trans, const BLASINT m, const BLASINT n, const BLASINT k, const float *src, const BLASINT ld, float *dst);

 版权所有 © 华为技术有限公司

void cblas_dgemm_pack(const enum CBLAS_ORDER order, const enum CBLAS_IDENTIFIER identifier, const enum CBLAS_TRANSPOSE trans, const BLASINT m, const BLASINT n, const BLASINT k, const double *src, const BLASINT ld, double *dst);

void cblas_cgemm_pack(const enum CBLAS_ORDER order, const enum CBLAS_IDENTIFIER identifier, const enum CBLAS_TRANSPOSE trans, const BLASINT m, const BLASINT n, const BLASINT k, const void *src, const BLASINT ld, void *dst);

void cblas_zgemm_pack(const enum CBLAS_ORDER order, const enum CBLAS_IDENTIFIER identifier, const enum CBLAS_TRANSPOSE trans, const BLASINT m, const BLASINT n, const BLASINT k, const void *src, const BLASINT ld, void *dst);

void cblas_bgemm_pack(const enum CBLAS_ORDER order, const enum CBLAS_IDENTIFIER identifier, const enum CBLAS_TRANSPOSE trans, const BLASINT m, const BLASINT n, const BLASINT k, const __bf16 *src, const BLASINT ld, __bf16 *dst);

void cblas_sbgemm_pack(const enum CBLAS_ORDER order, const enum CBLAS_IDENTIFIER identifier, const enum CBLAS_TRANSPOSE trans, const BLASINT m, const BLASINT n, const BLASINT k, const __bf16 *src, const BLASINT ld, __bf16 *dst);

参数

参数名	类型	描述	输入/输出
order	枚举类型CBLAS_ORDER	表示矩阵是行主序或列主序。	输入
identifier	枚举类型CBLAS_IDENTIFIER	指定要pack的矩阵。 ● 如果identifier= CblasA，pack A矩阵。 ● 如果identifier= CblasB，pack B矩阵。	输入
trans	枚举类型CBLAS_TRANSPOSE	矩阵A为常规矩阵，转置矩阵。 ● 如果trans= CblasNoTrans，$op(A) = A$。 ● 如果trans= CblasTrans，$op(A) = A'$。 矩阵B为常规矩阵，转置矩阵。 ● 如果trans= CblasNoTrans，$op(B) = B$。 ● 如果trans= CblasTrans，$op(B) = B'$。	输入

 版权所有 © 华为技术有限公司

参数名	类型	描述	输入/输出
m	整型数	矩阵op(A)和矩阵C的行。	输入
n	整型数	矩阵op(B)和矩阵C的列。	输入
k	整型数	矩阵op(A)的列和矩阵op(B)的行。	输入
src	• 在sgemm中是单精度浮点类型。 • 在dgemm中是双精度浮点类型。 • 在cgemm中是单精度复数类型。 • 在zgemm中是双精度复数类型。 • 在bgemm中是半精度浮点类型。	矩阵A\B。	输入
ld	整型数	• 矩阵为列存，identifier = CblasA, trans = CblasNoTrans，ld至少max(1, m)，否则max(1, k)。 • 矩阵为行存，identifier = CblasA, trans = CblasNoTrans，ld至少max(1, k)，否则max(1, m)。 • 矩阵为列存，identifier = CblasB, trans = CblasNoTrans，ld至少max(1, k)，否则max(1, n)。 • 矩阵为行存，identifier = CblasB, trans = CblasNoTrans，ld至少max(1, n)，否则max(1, k)。	输入

版权所有 © 华为技术有限公司

参数名	类型	描述	输入/输出
dst	• 在sgemm中是单精度浮点类型。 • 在dgemm中是双精度浮点类型。 • 在cgemm中是单精度复数类型。 • 在zgemm中是双精度复数类型。 • 在bgemm中是半精度浮点类型。	pack后的矩阵A\B。	输入/输出

依赖

#include "kblas.h"

示例

C interface：

```
int m = 4, k = 3, n = 4, ld = 4;
 /*
 * src:
 *     0.340188,       0.411647,       -0.222225,
 *     -0.105617,      -0.302449,      0.053970,
 *     0.283099,       -0.164777,      -0.022603,
 *     0.298440,       0.268230,       0.128871,
 */
float src[12] = {0.340188, -0.105617, 0.283099,
                 0.298440, 0.411647, -0.302449,
                 -0.164777, 0.268230, -0.222225,
                 0.053970, -0.022603, 0.128871};
size_t size = cblas_sgemm_pack_get_size(CblasA, m, n, k);
float *dst = (float *)malloc(size);
cblas_sgemm_pack(CblasColMajor, CblasA, CblasTrans, m, n, k, src, ld, dst);
free(dst);
/*
 * Output dst:
 *     0.340188       -0.105617       0.283099        0.298440
 *     0.411647       -0.302449       -0.164777       0.268230
 *    -0.222225       0.053970        -0.022603       0.128871
 */
```

6.4.8.4.4.17 ?gemm_compute

对pack后的A\B矩阵进行一般矩阵乘。

即：$C = \text{alpha} * \text{op}(A) * \text{op}(B) + \text{beta} * C$。

op(X)可取值：

$$op(X)=X, op(X)=X^{T}, op(X)=conjg(X), op(X)=conjg(X^{T})$$

，alpha，beta为乘法系数，op(A)为m*k矩阵，op(B)为k*n矩阵，C为m*n矩阵。

版权所有 © 华为技术有限公司

接口定义

C interface：

void cblas_sgemm_compute(const enum CBLAS_ORDER order, const enum CBLAS_TRANSPOSE transA, const enum CBLAS_TRANSPOSE transB, const BLASINT m, const BLASINT n, const BLASINT k, const float alpha, const float *a, const BLASINT lda, const float *b, const BLASINT ldb, const float beta, float *c, const BLASINT ldc);

void cblas_dgemm_compute(const enum CBLAS_ORDER order, const enum CBLAS_TRANSPOSE transA, const enum CBLAS_TRANSPOSE transB, const BLASINT m, const BLASINT n, const BLASINT k, const double alpha, const double *a, const BLASINT lda, const double *b, const BLASINT ldb, const double beta, double *c, const BLASINT ldc);

void cblas_cgemm_compute(const enum CBLAS_ORDER order, const enum CBLAS_TRANSPOSE TransA, const enum CBLAS_TRANSPOSE TransB, const BLASINT m, const BLASINT n, const BLASINT k, const void *alpha, const void *a, const BLASINT lda, const void *b, const BLASINT ldb, const void *beta, void *c, const BLASINT ldc);

void cblas_zgemm_compute(const enum CBLAS_ORDER order, const enum CBLAS_TRANSPOSE TransA, const enum CBLAS_TRANSPOSE TransB, const BLASINT m, const BLASINT n, const BLASINT k, const void *alpha, const void *a, const BLASINT lda, const void *b, const BLASINT ldb, const void *beta, void *c, const BLASINT ldc);

void cblas_bgemm_compute(const enum CBLAS_ORDER order, const enum CBLAS_TRANSPOSE transA, const enum CBLAS_TRANSPOSE transB, const BLASINT m, const BLASINT n, const BLASINT k, const __bf16 alpha, const __bf16 *a, const BLASINT lda, const __bf16 *b, const BLASINT ldb, const __bf16 beta, __bf16 *c, const BLASINT ldc);

void cblas_sbgemm_compute(const enum CBLAS_ORDER order, const enum CBLAS_TRANSPOSE transA, const enum CBLAS_TRANSPOSE transB, const BLASINT m, const BLASINT n, const BLASINT k, const __bf16 alpha, const __bf16 *a, const BLASINT lda, const __bf16 *b, const BLASINT ldb, const __bf16 beta, float *c, const BLASINT ldc);

参数

参数名	类型	描述	输入/输出
order	枚举类型CBLAS_ORDER	表示矩阵是行主序或列主序。	输入
TransA	枚举类型CBLAS_TRANSPOSE	矩阵A为常规矩阵，转置矩阵或共轭矩阵。 ● 如果trans= CblasNoTrans，$op(A) = A$。 ● 如果trans= CblasTrans，$op(A) = A'$。	输入

版权所有 © 华为技术有限公司

参数名	类型	描述	输入/输出
TransB	枚举类型CBLAS_TRANSPOSE	矩阵B为常规矩阵，转置矩阵或共轭矩阵。 • 如果trans= CblasNoTrans，$op(B) = B$。 • 如果trans= CblasTrans，$op(B) = B'$。	输入
M	整型数	矩阵op(A)和矩阵C的行。	输入
N	整型数	矩阵op(B)和矩阵C的列。	输入
K	整型数	矩阵op(A)的列和矩阵op(B)的行。	输入
alpha	• 在sgemm中是单精度浮点类型。 • 在dgemm中是双精度浮点类型。 • 在cgemm中是单精度复数类型。 • 在zgemm中是双精度复数类型。 • 在bgemm中是半精度浮点类型。 • 在sbgemm中是半精度浮点类型。	乘法系数。	输入
A	• 在sgemm中是单精度浮点类型。 • 在dgemm中是双精度浮点类型。 • 在cgemm中是单精度复数类型。 • 在zgemm中是双精度复数类型。 • 在bgemm中是半精度浮点类型。 • 在sbgemm中是半精度浮点类型。	矩阵A。	输入

 版权所有 © 华为技术有限公司

参数名	类型	描述	输入/输出
lda	整型数	• 矩阵为列存，TransA = CblasNoTrans，lda至少max(1, m)，否则max(1, k)。 • 矩阵为行存，TransA = CblasNoTrans，lda至少max(1, k)，否则max(1, m)。	输入
B	• 在sgemm中是单精度浮点类型。 • 在dgemm中是双精度浮点类型。 • 在cgemm中是单精度复数类型。 • 在zgemm中是双精度复数类型。 • 在bgemm中是半精度浮点类型。 • 在sbgemm中是半精度浮点类型。	矩阵B。	输入
ldb	整型数	• 矩阵为列存，TransB = CblasNoTrans，ldb至少max(1, k)，否则max(1, n)。 • 矩阵为行存，TransB = CblasNoTrans，ldb至少max(1, n)，否则max(1, k)。	输入
beta	• 在sgemm中是单精度浮点类型。 • 在dgemm中是双精度浮点类型。 • 在cgemm中是单精度复数类型。 • 在zgemm中是双精度复数类型。 • 在bgemm中是半精度浮点类型。 • 在sbgemm中是单精度浮点类型。	乘法系数。	输入

版权所有 © 华为技术有限公司

参数名	类型	描述	输入/输出
C	• 在sgemm中是单精度浮点类型。 • 在dgemm中是双精度浮点类型。 • 在cgemm中是单精度复数类型。 • 在zgemm中是双精度复数类型。 • 在bgemm中是半精度浮点类型。 • 在sbgemm中是单精度浮点类型。	矩阵C。	输入/输出
ldc	整型数	矩阵为列存，ldc至少max(1, m)，否则max(1, n)。	输入

依赖

#include "kblas.h"

示例

C interface：

```
int m = 4, k = 3, n = 4, lda = 4, ldb = 3, ldc = 4;
float alpha = 1.0, beta = 2.0;
 /*
 * A:
 *     0.340188,       0.411647,       -0.222225,
 *     -0.105617,      -0.302449,      0.053970,
 *     0.283099,       -0.164777,      -0.022603,
 *     0.298440,       0.268230,       0.128871,
 * B:
 *     -0.135216,      0.416195,       -0.358397,      -0.257113,
 *     0.013401,       0.135712,       0.106969,       -0.362768,
 *     0.452230,       0.217297,       -0.483699,      0.304177,
 * C:
 *     -0.343321,      0.498924,       0.112640,       -0.006417,
 *     -0.099056,      -0.281743,      -0.203968,      0.472775,
 *     -0.370210,      0.012932,       0.137552,       -0.207483,
 *     -0.391191,      0.339112,       0.024287,       0.271358,
 */
float a[12] = {0.340188, -0.105617, 0.283099,
               0.298440, 0.411647, -0.302449,
               -0.164777, 0.268230, -0.222225,
               0.053970, -0.022603, 0.128871};
float b[12] = {-0.135216, 0.013401, 0.452230, 0.416195,
               0.135712, 0.217297, -0.358397, 0.106969,
               -0.483699, -0.257113, -0.362768, 0.304177};
float c[16] = {-0.343321, -0.099056, -0.370210, -0.391191,
               0.498924, -0.281743, 0.012932, 0.339112,
               0.112640, -0.203968, 0.137552, 0.024287,
               -0.006417, 0.472775, -0.207483, 0.271358};
size_t size_a = cblas_sgemm_pack_get_size(CblasA, m, n, k);
```

 版权所有 © 华为技术有限公司

```
    size_t size_b = cblas_sgemm_pack_get_size(CblasB, m, n, k);
    float *sa = (float *)malloc(size_a);
    float *sb = (float *)malloc(size_b);
    cblas_sgemm_pack(CblasColMajor, CblasA, CblasNoTrans, m, n, k, a, ld, sa);
    cblas_sgemm_pack(CblasColMajor, CblasB, CblasNoTrans, m, n, k, b, ld, sb);
    cblas_sgemm_compute(CblasColMajor,CblasNoTrans,CblasNoTrans, m, n, k, alpha, sa, lda, sb, ldb, beta, c,
ldc);
    free(sa);
    free(sb);
    /*
     * Output C:
     *     -0.827621       1.147010        0.254881        -0.317229
     *     -0.163476       -0.636762       -0.428542       1.098841
     *     -0.791128       0.116416        0.166949        -0.434854
     *     -0.760862       0.866839        -0.092028       0.407877
     *
     */
```

6.4.8.4.4.18 ?gemm_batch

批量一般矩阵乘矩阵，此函数描述如下：

$$
\begin{aligned}
&idx = 0\\
&for\ i = 0, group_count - 1\\
&\quad for\ j = 0, group_size[i] - 1\\
&\qquad A, B, and\ C\ matrix\ in\ a_array[idx], b_array[idx], and\ c_array[idx]\\
&\qquad C = alpha_array[i] * op(A) * op(B) + beta_array[i] * C\\
&\quad end\ for\\
&end\ for
\end{aligned}
$$

op(X)可取值：

$$op(X)=X, op(X)=X^{T}, op(X)=conjg(X), op(X)=conjg(X^{T})$$

，alpha，beta为乘法系数，op(A)为m*k矩阵，op(B)为k*n矩阵，C为m*n矩阵。

注：batchgemm适用于矩阵规格小且batch规模大的场景，不适用矩阵大且batch小的场景

接口定义

C interface：

void cblas_sgemm_batch(const enum CBLAS_ORDER order, const enum CBLAS_TRANSPOSE *transA_array,

const enum CBLAS_TRANSPOSE *transB_array, const BLASINT *m_array, const BLASINT *n_array, const BLASINT *k_array,

const float *alpha_array, const float **a_array, const BLASINT *lda_array, const float **b_array,

const BLASINT *ldb_array, const float *beta_array, float **c_array, const BLASINT *ldc_array,

const BLASINT group_count, const BLASINT *group_size);

 版权所有 © 华为技术有限公司

void cblas_dgemm_batch(const enum CBLAS_ORDER order, const enum CBLAS_TRANSPOSE *transA_array,

const enum CBLAS_TRANSPOSE *transB_array, const BLASINT *m_array, const BLASINT *n_array, const BLASINT *k_array,

const double *alpha_array, const double **a_array, const BLASINT *lda_array, const double **b_array,

const BLASINT *ldb_array, const double *beta_array, double **c_array, const BLASINT *ldc_array,

const BLASINT group_count, const BLASINT *group_size);

void cblas_cgemm_batch(const enum CBLAS_ORDER order, const enum CBLAS_TRANSPOSE *transA_array,

const enum CBLAS_TRANSPOSE *transB_array, const BLASINT *m_array, const BLASINT *n_array, const BLASINT *k_array,

const float *alpha_array, const float **a_array, const BLASINT *lda_array, const float **b_array,

const BLASINT *ldb_array, const float *beta_array, float **c_array, const BLASINT *ldc_array,

const BLASINT group_count, const BLASINT *group_size);

void cblas_zgemm_batch(const enum CBLAS_ORDER order, const enum CBLAS_TRANSPOSE *transA_array,

const enum CBLAS_TRANSPOSE *transB_array, const BLASINT *m_array, const BLASINT *n_array, const BLASINT *k_array,

const double *alpha_array, const double **a_array, const BLASINT *lda_array, const double **b_array,

const BLASINT *ldb_array, const double *beta_array, double **c_array, const BLASINT *ldc_array,

const BLASINT group_count, const BLASINT *group_size);

void cblas_hgemm_batch(const enum CBLAS_ORDER order, const enum CBLAS_TRANSPOSE *transA_array,

const enum CBLAS_TRANSPOSE *transB_array, const BLASINT *m_array, const BLASINT *n_array, const BLASINT *k_array,

const __fp16 *alpha_array, const __fp16 **a_array, const BLASINT *lda_array, const __fp16 **b_array,

const BLASINT *ldb_array, const __fp16 *beta_array, __fp16 **c_array, const BLASINT *ldc_array,

const BLASINT group_count, const BLASINT *group_size);

void cblas_bgemm_batch(const enum CBLAS_ORDER order, const enum CBLAS_TRANSPOSE *transA_array,

const enum CBLAS_TRANSPOSE *transB_array, const BLASINT *m_array, const BLASINT *n_array, const BLASINT *k_array,

 版权所有 © 华为技术有限公司

const __bf16 *alpha_array, const __bf16 **a_array, const BLASINT *lda_array, const __bf16 **b_array,

const BLASINT *ldb_array, const __bf16 *beta_array, __bf16 **c_array, const BLASINT *ldc_array,

const BLASINT group_count, const BLASINT *group_size);

参数

参数名	类型	描述	输入/输出
order	枚举类型CBLAS_ORDER	表示矩阵是行主序或列主序。	输入
transA_array	枚举类型CBLAS_TRANSPOSE	表示矩阵A转置情况的序列。 矩阵A为常规矩阵，转置矩阵或共轭矩阵。 ● 如果TransA = CblasNoTrans，$op(A) = A$。 ● 如果TransA = CblasTrans，$op(A) = A'$。 ● 如果TransA = CblasConjTrans，$op(A)=conjg(A)$。 ● 如果TransA = CblasConjTrans，$op(A) = conjg(A')$。	输入

 版权所有 © 华为技术有限公司

参数名	类型	描述	输入/输出
transB_array	枚举类型CBLAS_TRANSPOSE	表示矩阵B转置情况的序列。 矩阵B为常规矩阵，转置矩阵或共轭矩阵。 ● 如果TransB = CblasNoTrans，$op(B) = B$。 ● 如果TransB = CblasTrans，$op(B) = B'$。 ● 如果TransB = CblasConjTrans，$op(B)=conjg(B)$。 ● 如果TransB = CblasConjTrans，$op(B) = conjg(B')$。	输入
m_array	整型数	矩阵op(A)和矩阵C的行数的序列。	输入
n_array	整型数	矩阵op(B)和矩阵C的列数的序列。	输入
k_array	整型数	矩阵op(A)的列和矩阵op(B)的行数的序列。	输入
alpha_array	● 在sgemm中是单精度浮点类型。 ● 在dgemm中是双精度浮点类型。 ● 在hgemm中是fp16类型。 ● 在bgemm中是bf16类型。	乘法系数的序列。	输入
a_array	● 在sgemm中是单精度浮点类型。 ● 在dgemm中是双精度浮点类型。 ● 在hgemm中是fp16类型。 ● 在bgemm中是bf16类型。	矩阵A的序列。	输入

版权所有 © 华为技术有限公司

参数名	类型	描述	输入/输出
lda_array	整型数	表示矩阵A的leading dimension的序列。 ● 矩阵为列存，TransA = CblasNoTrans，lda至少max(1, m)，否则max(1, k)。 ● 矩阵为行存，TransA = CblasNoTrans，lda至少max(1, k)，否则max(1, m)。	输入
b_array	● 在sgemm中是单精度浮点类型。 ● 在dgemm中是双精度浮点类型。 ● 在hgemm中是fp16类型。 ● 在bgemm中是bf16类型。	矩阵B的序列。	输入
ldb_array	整型数	表示矩阵B的leading dimension的序列。 ● 矩阵为列存，TransB = CblasNoTrans，ldb至少max(1, k)，否则max(1, n)。 ● 矩阵为行存，TransB = CblasNoTrans，ldb至少max(1, n)，否则max(1, k)。	输入
beta_array	● 在sgemm中是单精度浮点类型。 ● 在dgemm中是双精度浮点类型。 ● 在hgemm中是fp16类型。 ● 在bgemm中是bf16类型。	乘法系数的序列。	输入
c_array	● 在sgemm中是单精度浮点类型。 ● 在dgemm中是双精度浮点类型。 ● 在hgemm中是fp16类型。 ● 在bgemm中是bf16类型。	矩阵C的序列。	输入/输出

版权所有 © 华为技术有限公司

参数名	类型	描述	输入/输出
ldc_array	整型数	表示矩阵C的leading dimension的序列。 矩阵为列存，ldc至少max(1, m)，否则max(1, n)。	输入
group_count	整型数	指定group的数量。必须至少为0。	输入
group_size	整型数	大小为 group_count 的数组。元素 group_size[i] 指定了第 i 组中的矩阵数量。	输入

依赖

#include "kblas.h"

示例

```
  int m = 2, k = 3, n = 2, lda = 2, ldb = 3, ldc = 2;
  int group_count = 2;
  int group_size[2] = {1, 1};
  int m_array[2] = {2, 2};
  int k_array[2] = {3, 3};
  int n_array[2] = {2, 2};
  int lda_array[2] = {2, 2};
  int ldb_array[2] = {3, 3};
  int ldc_array[2] = {2, 2};
  CBLAS_TRANSPOSE transA_array[2] = {CblasNoTrans, CblasNoTrans};
  CBLAS_TRANSPOSE transB_array[2] = {CblasNoTrans, CblasNoTrans};
  float alpha_array[2] = {1.0, 1.0};
  float beta_array[2] = {2.0, 2.0};
   /*
   * A:
   *     0.340188,       0.411647,       -0.222225,
   *     -0.105617,      -0.302449,       0.053970,
   *     0.283099,       -0.164777,      -0.022603,
   *     0.298440,       0.268230,       0.128871,
   * B:
   *     -0.135216,      0.416195,
   *     0.013401,       0.135712,
   *     0.452230,       0.217297,
   *     -0.358397,      -0.257113,
   *      0.106969,      -0.362768,
   *     -0.483699,      0.304177
   * C:
   *     -0.343321,      0.498924,       0.112640,       -0.006417,
   *     -0.099056,      -0.281743,      -0.203968,      0.472775
   */
  float a_data[12] = { 0.340188,  -0.105617, 0.283099,  0.298440, 0.411647,  -0.302449,
                       -0.164777, 0.268230,  -0.222225, 0.053970, -0.022603, 0.128871 };
  float b_data[12] = { -0.135216, 0.416195,  0.013401, 0.135712,  0.452230,  0.217297,
                       -0.358397, -0.257113, 0.106969, -0.362768, -0.483699, 0.304177 };
  float c_data[8] = { -0.343321, -0.099056, -0.370210, -0.391191, 0.498924, -0.281743, 0.012932,
0.339112 };
  float* a_array[2];
  float* b_array[2];
  float* c_array[2];
  a_array[0] = a_data;
  a_array[1] = a_data + m*k*group_size[0];
```


版权所有 © 华为技术有限公司

```
    b_array[0] = b_data;
    b_array[1] = b_data + k*n*group_size[0];
    c_array[0] = c_data;
    c_array[1] = c_data + m*n*group_size[0];
    cblas_sgemm_batch(CblasColMajor, transA_array, transB_array, m_array, n_array, k_array, alpha_array,
a_array,
                      lda_array, b_array, ldb_array, beta_array, c_array, ldc_array, group_count, group_size);
```

6.4.8.4.4.19 gemm_batch_?8?8s32

批量一般矩阵乘矩阵，此函数描述如下：

$$
\begin{aligned}
&idx = 0\\
&for\ i = 0, group_count - 1\\
&\quad for\ j = 0, group_size[i] - 1\\
&\qquad A, B, and\ C\ matrix\ in\ a_array[idx], b_array[idx], and\ c_array[idx]\\
&\qquad A_offset, B_offset, and\ C_offset\ in\ oa_array[idx], ob_array[idx], and\ oc_array[idx]\\
&\qquad C = alpha_array[i] * op(A+A_offset) * op(B+B_offset) + beta_array[i]*C + C_offset\\
&\quad end\ for\\
&end\ for
\end{aligned}
$$

op(X)可取值：$op(X)=X, op(X)=X^{T}$，alpha，beta为乘法系数，op(A)为m*k矩阵，op(B)为k*n矩阵，C为m*n矩阵。

A_offset为一个标量，类型固定为int8_t，op(A)中所有元素加上A_offset；

B_offset为一个标量，类型固定为int8_t，op(B)中所有元素加上B_offset；

C_offset有三种定义（元素类型均为int32_t）：

- 一个标量，C矩阵的每一个元素都加上C_offset
- 一个行向量，C矩阵的每一行都加上这个行向量C_offset
- 一个列向量，C矩阵的每一列都加上这个列向量C_offset

注：batchgemm适用于**矩阵规格小且batch规模大**的场景，不适用矩阵大且batch小的场景

接口定义

C interface：

void cblas_gemm_batch_s8s8s32(const CBLAS_LAYOUT layout, const CBLAS_TRANSPOSE *transA_array,

const CBLAS_TRANSPOSE *transB_array, const CBLAS_OFFSET *offsetc_array, const BLASINT *m_array,

const BLASINT *n_array, const BLASINT *k_array, const float *alpha_array, const BLASINT8 **a_array,

const BLASINT *lda_array, const BLASINT8 *oa_array, const BLASINT8 **b_array, const BLASINT *ldb_array,

const BLASINT8 *ob_array, const float *beta, int32_t **c_array, const BLASINT *ldc_array, const int32_t **oc_array,

 版权所有 © 华为技术有限公司

const BLASINT group_count, const BLASINT *group_size);

void cblas_gemm_batch_u8u8s32(const CBLAS_LAYOUT layout, const CBLAS_TRANSPOSE *transA_array,

const CBLAS_TRANSPOSE *transB_array, const CBLAS_OFFSET *offsetc_array, const BLASINT *m_array,

const BLASINT *n_array, const BLASINT *k_array, const float *alpha_array, const BLASUINT8 **a_array,

const BLASINT *lda_array, const BLASINT8 *oa_array, const BLASUINT8 **b_array, const BLASINT *ldb_array,

const BLASINT8 *ob_array, const float *beta, int32_t **c_array, const BLASINT *ldc_array, const int32_t **oc_array,

const BLASINT group_count, const BLASINT *group_size);

void cblas_gemm_batch_s8u8s32(const CBLAS_LAYOUT layout, const CBLAS_TRANSPOSE *transA_array,

const CBLAS_TRANSPOSE *transB_array, const CBLAS_OFFSET *offsetc_array, const BLASINT *m_array,

const BLASINT *n_array, const BLASINT *k_array, const float *alpha_array, const BLASINT8 **a_array,

const BLASINT *lda_array, const BLASINT8 *oa_array, const BLASUINT8 **b_array, const BLASINT *ldb_array,

const BLASINT8 *ob_array, const float *beta, int32_t **c_array, const BLASINT *ldc_array, const int32_t **oc_array,

const BLASINT group_count, const BLASINT *group_size);

void cblas_gemm_batch_u8s8s32(const CBLAS_LAYOUT layout, const CBLAS_TRANSPOSE *transA_array,

const CBLAS_TRANSPOSE *transB_array, const CBLAS_OFFSET *offsetc_array, const BLASINT *m_array,

const BLASINT *n_array, const BLASINT *k_array, const float *alpha_array, const BLASUINT8 **a_array,

const BLASINT *lda_array, const BLASINT8 *oa_array, const BLASINT8 **b_array, const BLASINT *ldb_array,

const BLASINT8 *ob_array, const float *beta, int32_t **c_array, const BLASINT *ldc_array, const int32_t **oc_array,

const BLASINT group_count, const BLASINT *group_size);

参数

参数名	类型	描述	输入/输出
layout	枚举类型CBLAS_ORDER	表示矩阵是行主序或列主序。	输入

 版权所有 © 华为技术有限公司

参数名	类型	描述	输入/输出
transA_array	枚举类型CBLAS_TRANSPOSE	表示矩阵A转置情况的序列。 矩阵A为常规矩阵，转置矩阵或共轭矩阵。 ● 如果TransA = CblasNoTrans，$op(A) = A$。 ● 如果TransA = CblasTrans，$op(A) = A'$。 ● 如果TransA = CblasConjTrans，$op(A)=conjg(A)$。 ● 如果TransA = CblasConjTrans，$op(A) = conjg(A')$。	输入
transB_array	枚举类型CBLAS_TRANSPOSE	表示矩阵B转置情况的序列。 矩阵B为常规矩阵，转置矩阵或共轭矩阵。 ● 如果TransB = CblasNoTrans，$op(B) = B$。 ● 如果TransB = CblasTrans，$op(B) = B'$。 ● 如果TransB = CblasConjTrans，$op(B)=conjg(B)$。 ● 如果TransB = CblasConjTrans，$op(B) = conjg(B')$。	输入

版权所有 © 华为技术有限公司

参数名	类型	描述	输入/输出
offsetc_array	枚举类型CBLAS_OFFSET	表示参数offetsc的序列。 ● 如果offsetc = CblasRowOffset，则代表C_offset是一个行向量。 ● 如果offsetc = CblasColOffset，则代表C_offset是一个列向量。 ● 如果offsetc = CblasFixOffset，则代表C_offset是一个标量。	输入
m_array	整型数	矩阵op(A)和矩阵C的行数的序列。	输入
n_array	整型数	矩阵op(B)和矩阵C的列数的序列。	输入
k_array	整型数	矩阵op(A)的列和矩阵op(B)的行数的序列。	输入
alpha_array	单精度浮点类型。	乘法系数的序列。	输入
a_array	● 在*_s8*8s32中是int8_t类型。 ● 在*_u8*8s32中是uint8_t类型。	矩阵A的序列。	输入
lda_array	整型数	表示矩阵A的leading dimension的序列。 ● 矩阵为列存，TransA = CblasNoTrans，lda至少max(1, m)，否则max(1, k)。 ● 矩阵为行存，TransA = CblasNoTrans，lda至少max(1, k)，否则max(1, m)。	输入
oa_array	整型数	GEMM定义中的A_offset的值的序列	输入
b_array	● 在*_*8s8s32中是int8_t类型。 ● 在*_*8u8s32中是uint8_t类型。	矩阵B的序列。	输入

 版权所有 © 华为技术有限公司

参数名	类型	描述	输入/输出
ldb_array	整型数	表示矩阵A的leading dimension的序列。 ● 矩阵为列存，TransB = CblasNoTrans，ldb至少max(1, k)，否则max(1, n)。 ● 矩阵为行存，TransB = CblasNoTrans，ldb至少max(1, n)，否则max(1, k)。	输入
ob_array	整型数	GEMM定义中的B_offset的值的序列	输入
beta_array	● 单精度浮点类型。	乘法系数的序列。	输入
c_array	整型数	矩阵C的序列。	输入/输出
ldc_array	整型数	表示矩阵C的leading dimension的序列。 矩阵为列存，ldc至少max(1, m)，否则max(1, n)。	输入
oc_array	整型数	GEMM定义中的C_offset的值的序列	输入
group_count	整型数	指定group的数量。必须至少为0。	输入
group_size	整型数	大小为 group_count 的数组。元素 group_size[i] 指定了第 i 组中的矩阵数量。	输入

依赖

#include "kblas.h"

示例

```
int m = 2, k = 3, n = 2, lda = 2, ldb = 3, ldc = 2;
    int group_count = 2;
    int group_size[2] = {1, 1};
    int m_array[2] = {2, 2};
    int k_array[2] = {3, 3};
    int n_array[2] = {2, 2};
    int lda_array[2] = {2, 2};
    int ldb_array[2] = {3, 3};
    int ldc_array[2] = {2, 2};
    CBLAS_TRANSPOSE transA_array[2] = {CblasNoTrans, CblasNoTrans};
    CBLAS_TRANSPOSE transB_array[2] = {CblasNoTrans, CblasNoTrans};
```


版权所有 © 华为技术有限公司

```
float alpha_array[2] = {1.0, 1.0};
float beta_array[2] = {1.0, 1.0};
CBLAS_OFFSET offestc_array[2] = {CblasFixOffset, CblasFixOffset};
int32_t oc_array[2] = {0, 0};
int8_t oa_array[2] = {0, 0};
int8_t ob_array[2] = {0, 0};
int8_t a_data[12] = {1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1};
int8_t b_data[12] = {1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1};
int32_t c_data[8] = {1, 1, 1, 1, 1, 1, 1, 1};
int8_t* a_array[2];
int8_t* b_array[2];
int32_t* c_array[2];
a_array[0] = a_data;
a_array[1] = a_data + m*k*group_size[0];
b_array[0] = b_data;
b_array[1] = b_data + k*n*group_size[0];
c_array[0] = c_data;
c_array[1] = c_data + m*n*group_size[0];
cblas_gemm_batch_s8s8s32(CblasColMajor, transA_array, transB_array, offestc_array, m_array, n_array,
k_array, alpha_array, a_array,
                lda_array, oa_array, b_array, ldb_array, ob_array, beta_array, c_array, ldc_array, oc_array,
group_count, group_size);
```

6.4.8.4.4.20 gemm_s8u8s32_pack_get_size

返回存储pack后的矩阵所需的字节数。

接口定义

C interface：

size_t cblas_gemm_s8u8s32_pack_get_size (const enum CBLAS_IDENTIFIER identifier, const BLASINT m, const BLASINT n, const BLASINT k);

参数

参数名	类型	描述	输入/输出
identifier	枚举类型CBLAS_IDENTIFIER	指定要pack的矩阵。 • 如果identifier= CblasA，返回pack A矩阵所需字节数。 • 如果identifier= CblasB，返回pack B矩阵所需字节数。	输入
m	整型数	需要pack的矩阵A的行。	输入
n	整型数	需要pack的矩阵B的列。	输入
k	整型数	需要pack的矩阵A的列和B的行。	输入

依赖

#include "kblas.h"

 版权所有 © 华为技术有限公司

示例

```
int m = 4, k = 3, n = 4;

size_t size = cblas_gemm_s8u8s32_pack_get_size(CblasA, m, n, k);
/*
 * Output size: 64
 */
```

6.4.8.4.4.21 gemm_s8u8s32_pack

对矩阵进行pack操作并将其存储到已分配的缓冲区中。

即：$C = \text{alpha} * \text{op}(A) * \text{op}(B) + \text{beta} * C$。

op(X)可取值：

$$op(X)=X, op(X)=X^{T}, op(X)=conjg(X), op(X)=conjg(X^{T})$$

，alpha，beta为乘法系数，op(A)为m*k矩阵，op(B)为k*n矩阵，C为m*n矩阵。

接口定义

C interface：

void cblas_gemm_s8u8s32_pack(const CBLAS_ORDER order, const enum CBLAS_IDENTIFIER identifier,

const CBLAS_TRANSPOSE trans, const BLASINT m, const BLASINT n, const BLASINT k,

const void *src, const BLASINT ld, void *dst);

参数

参数名	类型	描述	输入/输出
order	枚举类型CBLAS_ORDER	表示矩阵是行主序或列主序。	输入
identifier	枚举类型CBLAS_IDENTIFIER	指定要pack的矩阵。 ● 如果identifier= CblasA，pack A矩阵。 ● 如果identifier= CblasB，pack B矩阵。	输入

版权所有 © 华为技术有限公司

参数名	类型	描述	输入/输出
trans	枚举类型CBLAS_TRANSPOSE	矩阵A为常规矩阵，转置矩阵。 ● 如果trans= CblasNoTrans，$op(A) = A$。 ● 如果trans= CblasTrans，$op(A) = A'$。 矩阵B为常规矩阵，转置矩阵。 ● 如果trans= CblasNoTrans，$op(B) = B$。 ● 如果trans= CblasTrans，$op(B) = B'$。	输入
m	整型数	矩阵op(A)和矩阵C的行。	输入
n	整型数	矩阵op(B)和矩阵C的列。	输入
k	整型数	矩阵op(A)的列和矩阵op(B)的行。	输入
src	● int8类型。	矩阵A\B。	输入
ld	整型数	● 矩阵为列存，identifier = CblasA, trans = CblasNoTrans，ld至少max(1, m)，否则max(1, k)。 ● 矩阵为行存，identifier = CblasA, trans = CblasNoTrans，ld至少max(1, k)，否则max(1, m)。 ● 矩阵为列存，identifier = CblasB, trans = CblasNoTrans，ld至少max(1, k)，否则max(1, n)。 ● 矩阵为行存，identifier = CblasB, trans = CblasNoTrans，ld至少max(1, n)，否则max(1, k)。	输入
dst	● int8类型	pack后的矩阵A\B。	输入/输出

版权所有 © 华为技术有限公司

依赖

#include "kblas.h"

示例

参见 **gemm_?8?8s32_compute**示例

6.4.8.4.4.22 gemm_?8?8s32_compute

对pack后的A\B矩阵进行一般矩阵乘。

即：$C = \text{alpha} * \text{op}(A) * \text{op}(B) + \text{beta} * C$。

op(X)可取值：

$$op(X)=X, op(X)=X^{T}, op(X)=conjg(X), op(X)=conjg(X^{T})$$

，alpha，beta为乘法系数，op(A)为m*k矩阵，op(B)为k*n矩阵，C为m*n矩阵。

接口定义

C interface：

void cblas_gemm_s8s8s32_compute(const CBLAS_LAYOUT layout, const CBLAS_TRANSPOSE transa, const CBLAS_TRANSPOSE transb,

const CBLAS_OFFSET offsetc, const BLASINT m, const BLASINT n, const BLASINT k, const float alpha, const BLASINT8 *a, const BLASINT lda,

const BLASINT8 oa,const BLASINT8 *b, const BLASINT ldb, const BLASINT8 ob, const float beta, int32_t *c,const BLASINT ldc, const int32_t *oc);

void cblas_gemm_s8u8s32_compute(const CBLAS_LAYOUT layout, const CBLAS_TRANSPOSE transa, const CBLAS_TRANSPOSE transb,

const CBLAS_OFFSET offsetc, const BLASINT m, const BLASINT n, const BLASINT k, const float alpha, const BLASINT8 *a, const BLASINT lda,

const BLASINT8 oa, const BLASUINT8 *b, const BLASINT ldb, const BLASINT8 ob, const float beta, int32_t *c, const BLASINT ldc, const int32_t *oc);

void cblas_gemm_u8u8s32_compute(const CBLAS_LAYOUT layout, const CBLAS_TRANSPOSE transa, const CBLAS_TRANSPOSE transb,

const CBLAS_OFFSET offsetc, const BLASINT m, const BLASINT n, const BLASINT k,const float alpha, const BLASUINT8 *a, const BLASINT lda,

const BLASINT8 oa,const BLASUINT8 *b, const BLASINT ldb, const BLASINT8 ob, const float beta, int32_t *c,const BLASINT ldc, const int32_t *oc);

void cblas_gemm_u8s8s32_compute(const CBLAS_LAYOUT layout, const CBLAS_TRANSPOSE transa, const CBLAS_TRANSPOSE transb,

const CBLAS_OFFSET offsetc, const BLASINT m, const BLASINT n, const BLASINT k,const float alpha, const BLASUINT8 *a, const BLASINT lda,

 版权所有 © 华为技术有限公司

const BLASINT8 oa,const BLASINT8 *b, const BLASINT ldb, const BLASINT8 ob, const float beta, int32_t *c,const BLASINT ldc, const int32_t *oc);

参数

参数名	类型	描述	输入/输出
order	枚举类型CBLAS_ORDER	表示矩阵是行主序或列主序。	输入
TransA	枚举类型CBLAS_TRANSPOSE	矩阵A为常规矩阵，转置矩阵或共轭矩阵。 ● 如果trans= CblasNoTrans，$op(A) = A$。 ● 如果trans= CblasTrans，$op(A) = A'$。	输入
TransB	枚举类型CBLAS_TRANSPOSE	矩阵B为常规矩阵，转置矩阵或共轭矩阵。 ● 如果trans= CblasNoTrans，$op(B) = B$。 ● 如果trans= CblasTrans，$op(B) = B'$。	输入
M	整型数	矩阵op(A)和矩阵C的行。	输入
N	整型数	矩阵op(B)和矩阵C的列。	输入
K	整型数	矩阵op(A)的列和矩阵op(B)的行。	输入
alpha	● 单精度浮点类型。	乘法系数。	输入
A	● 在gemm_s8?8s32是int8类型。 ● 在gemm_u8?8s32是uint8类型。	矩阵A。	输入
lda	整型数	● 矩阵为列存，TransA = CblasNoTrans，lda至少max(1, m)，否则max(1, k)。 ● 矩阵为行存，TransA = CblasNoTrans，lda至少max(1, k)，否则max(1, m)。	输入

版权所有 © 华为技术有限公司

参数名	类型	描述	输入/输出
B	• 在gemm_?8s8s32是int8类型。 • 在gemm_?8u8s32是uint8类型。	矩阵B。	输入
ldb	整型数	• 矩阵为列存，TransB = CblasNoTrans，ldb至少max(1, k)，否则max(1, n)。 • 矩阵为行存，TransB = CblasNoTrans，ldb至少max(1, n)，否则max(1, k)。	输入
beta	• 单精度浮点类型。	乘法系数。	输入
C	• 是int类型。	矩阵C。	输入/输出
ldc	整型数	矩阵为列存，ldc至少max(1, m)，否则max(1, n)。	输入

依赖

#include "kblas.h"

示例

```
void test_igemm_compute(const enum CBLAS_ORDER order, const enum CBLAS_TRANSPOSE transA, const enum CBLAS_TRANSPOSE transB, BLASINT row, BLASINT col)
{
    BLASUINT8 *a = (BLASUINT8 *)malloc(sizeof(BLASUINT8) * row * col);
    BLASINT8 *b = (BLASINT8 *)malloc(sizeof(BLASINT8) * col * row);
    int32_t *c = (int32_t *)malloc(sizeof(int32_t) * row * row);
    for (int i = 0; i < row * col; i++) {
        a[i] = i;
        b[i] = i;
    }
    for (int i = 0; i < row * row; i++) {
        c[i] = i;
    }
    BLASINT m = row;
    BLASINT n = row;
    BLASINT k = col;
    BLASINT lda, ldb, ldc = m;
    size_t size_a = cblas_gemm_s8u8s32_pack_get_size(CblasA, m, n, k);
    size_t size_b = cblas_gemm_s8u8s32_pack_get_size(CblasB, m, n, k);
    BLASUINT8 *sa = (BLASUINT8 *)malloc(size_a);
    BLASINT8 *sb = (BLASINT8 *)malloc(size_b);
    if (order == CblasColMajor) {
        if (transA == CblasNoTrans) {
            lda = m;
        } else {
            lda = k;
        }
```


版权所有 © 华为技术有限公司

```
        if (transB == CblasNoTrans) {
            ldb = k;
        } else {
            ldb = n;
        }
    } else { // CblasRowMajor
        if (transA == CblasNoTrans) {
            lda = k;
        } else {
            lda = m;
        }
        if (transB == CblasNoTrans) {
            ldb = n;
        } else {
            ldb = k;
        }
    }
    float alpha = 2.0;
    float beta = 3.0;
    cblas_gemm_s8u8s32_pack(order, CblasA, transA, m, n, k, a, lda, sa);
    cblas_gemm_s8u8s32_pack(order, CblasB, transB, m, n, k, b, ldb, sb);
    int oc = 3;
    cblas_gemm_u8s8s32_compute(order, transA, transB, CblasFixOffset, m, n, k, alpha, sa, lda, 0, sb, ldb, 0,
beta, c, ldc, &oc);

    free(a);
    free(b);
    free(c);
    free(sa);
    free(sb);
}
```

6.4.8.4.5 Support Functions

6.4.8.4.5.1 BlasSetNumThreads

支持运行时设置线程数。

接口定义

void BlasSetNumThreads(int numThreads)

参数

参数名	类型	描述	输入/输出
numThreads	整型数	设置的线程数，要求numThreads>0。	输入

依赖

#include "kblas.h"

示例

```
BlasSetNumThreads(4); // 设置运行线程数为4
```


版权所有 © 华为技术有限公司

说明

支持通过设置环境变量设置线程数，如下所示：

export BLAS_NUM_THREADS=4

export OMP_NUM_THREADS=4

单独使用时，均可以设置运行线程数为4。当两个环境变量同时存在时，BLAS_NUM_THREADS的优先级更高。

6.4.8.4.5.2 BlasSetNumThreadsLocal

支持运行时设置local线程数。

接口定义

void BlasSetNumThreadsLocal(int numThreadsLocal)

参数

参数名	类型	描述	输入/输出
numThreadsLocal	整型数	设置的线程数，要求numThreadsLocal>0。	输入

依赖

#include "kblas.h"

示例

```
BlasSetNumThreadsLocal(4); // 设置运行local线程数为4
```

6.4.8.4.5.3 BlasGetNumThreads

获取运行时的线程数。

接口定义

int BlasGetNumThreads(void)

返回值

返回运行时的线程数，最大值为CPU核数。

依赖

#include "kblas.h"

示例

```
int nthreads = BlasGetNumThreads(); // 获取运行时的线程数
```

 版权所有 © 华为技术有限公司

6.4.8.4.5.4 BlasGetNumThreadsLocal

获取运行时的local线程数。

接口定义

int BlasGetNumThreadsLocal(void)

返回值

返回运行时的local线程数，最大值为CPU核数。

依赖

#include "kblas.h"

示例

```
int nthreadsLocal = BlasGetNumThreadsLocal(); // 获取运行时的local线程数
```

6.4.8.4.5.5 BlasGetNumProcs

获取机器的物理核数量。

接口定义

int BlasGetNumProcs(void)

返回值

单线程时返回1，多线程时返回机器的物理核数量。

依赖

#include "kblas.h"

6.4.8.4.5.6 BlasGetParallel

获取KML_BLAS的并行类型。

接口定义

int BlasGetParallel(void)

返回值

- 返回0表示KML_BLAS为单线程数学库。
- 返回1表示KML_BLAS为普通多线程数学库。
- 返回2表示KML_BLAS为OpenMP多线程数学库。
- 返回3表示KML_BLAS为KUPL多线程数学库。

依赖

#include "kblas.h"

 版权所有 © 华为技术有限公司

6.4.8.4.5.7 lsame

判断两个字符是否相同，不区分字符大小写。

接口定义

Fortran interface：

RES = LSAME(A, B)

返回值

- 返回0表示字符A和字符B不同。
- 返回1表示字符A和字符B相同。

6.4.8.4.5.8 ?cabs1

对复数的实部和虚部分别取绝对值，并相加。

即y = |Re(A)| + |Im(A)|。

接口定义

Fortran interface：

RES = SCABS1(A)

RES = DCABS1(A)

返回值

返回计算结果。

6.4.8.4.5.9 KBLASGetVersion

获取BLAS产品版本信息。

接口定义

int KBLASGetVersion(KBLASVersion *ver)

参数

参数名	描述	取值范围	输入/输出
ver	产品信息结构体。	非空	输出

返回值

- 成功：返回1。
- 失败：返回0。

 版权所有 © 华为技术有限公司

示例

```
#include <stdio.h>
#include "kblas.h"

int main()
{
    KBLASVersion ver;
    int ret = KBLASGetVersion(&ver);
    if (ret == 0 || strlen(ver.productName) == 0 || strlen(ver.productVersion) == 0 ||
        strlen(ver.componentName) == 0 || strlen(ver.componentVersion) == 0 ||
        strlen(ver.componentAppendInfo) == 0 ||
        strlen(ver.softwareName) == 0 || strlen(ver.softwareVersion) == 0) {
            printf("Get information error\n");
            return -1;
    }

    printf("Product Name: %s\n", ver.productName);
    printf("Product Version: %s\n", ver.productVersion);
    printf("Component Name: %s\n", ver.componentName);
    printf("Component Version: %s\n", ver.componentVersion);
    printf("Component AbiVersion: %s\n", ver.componentAbiVersion);
    printf("Component AppendInfo: %s\n", ver.componentAppendInfo);
    printf("Software Name: %s\n", ver.softwareName);
    printf("Software Version: %s\n", ver.softwareVersion);
    return 0;
}
```

运行结果：

```
Product Name: Kunpeng HPCKit
Product Version: 24.0.RC1
Component Name: HPCKit-kml
Component Version: 2.3.0
Component AbiVersion: 1
Component AppendInfo: gcc
Software Name: hpckit-kml-blas
Software Version: 2.3.0
```

说明

以上版本号和编译时间以实际运行结果为准，上述结果仅供参考。

6.4.8.4.5.10 cblas_xerbla

错误处理函数。

接口定义

Fortran interface：

CALL XERBLA(SRNAME, INFO, LENGTH)

6.4.9 KML_CONV 库函数说明

6.4.9.1 函数说明

KML_CONV是鲲鹏CPU上的卷积库，当前支持以下三类计算。

- 通用1D/2D卷积接口
- 上下采样接口
- 图像处理filter2D接口

 版权所有 © 华为技术有限公司

6.4.9.2 函数定义

6.4.9.2.1 通用卷积接口

6.4.9.2.1.1 函数说明

函数族	数据类型	描述
conv1d_?	fp32, fp16	通用1D卷积接口
conv1d_direct_?	fp32, fp16	Direct算法实现的1D卷积接口
conv1d_fft_?	fp32, fp16	FFT算法实现的1D卷积接口
conv1d_gemm_?	fp32, fp16	GEMM算法实现的1D卷积接口
conv1d_winograd_?	fp32, fp16	Winograd算法实现的1D卷积接口
conv2d_?	fp32, fp16	通用2D卷积接口
conv2d_direct_?	fp32, fp16	Direct算法实现的2D卷积接口
conv2d_fft_?	fp32, fp16	FFT算法实现的2D卷积接口
conv2d_gemm_?	fp32, fp16	GEMM算法实现的2D卷积接口
conv2d_winograd_?	fp32, fp16	Winograd算法实现的2D卷积接口

6.4.9.2.1.2 conv1d_?

通用1D卷积接口，根据输入特征，调用高性能算法实现。

接口定义

C interface：

void conv1d_fp32(const float *input, const int batch, const int inputChannels, const int inputLength, const float* kernel, const int kernelLength, const int stride, const int padLength, const int dilation, const float *bias, float *output, const int outputChannels);

void conv1d_fp16(const __fp16 *input, const int batch, const int inputChannels, const int inputLength, const __fp16 *kernel, const int kernelLength, const int stride, const int padLength, const int dilation, const __fp16 *bias, __fp16 *output, const int outputChannels);

 版权所有 © 华为技术有限公司

参数

参数名	类型	描述	输入/输出
input	• conv1d_fp32中是float类型 • conv1d_fp16中是__fp16类型	输入数据	输入
batch	int类型	输入数据的批数	输入
inputChannels	int类型	输入通道数	输入
inputLength	int类型	输入数据的长度	输入
kernel	• conv1d_fp32中是float类型 • conv1d_fp16中是__fp16类型	卷积核	输入
kernelLength	int类型	卷积核长度	输入
stride	int类型	步长	输入
padLength	int类型	在原始input数据两端分别置零的长度	输入
dilation	int类型	膨胀系数	输入
bias	• conv1d_fp32中是float类型 • conv1d_fp16中是__fp16类型	偏置值, 值为NULL时表示无偏置	输入
output	• conv1d_fp32中是float类型 • conv1d_fp16中是__fp16类型	输出结果数据	输出
outputChannels	int类型	输出通道数	输入

依赖

```
#include "conv.h"
```

 版权所有 © 华为技术有限公司

示例

C interface：

```
int batch = 1;
int inputChannels = 1;
int inputLength = 10;
int kernelLength = 3;
int stride = 1;
int padLength = 0;
int dilation = 1;
int outputChannels = 1;
float input[10] = {1.0, 2.0, 3.0, 4.0, 5.0, 6.0, 7.0, 8.0, 9.0, 10.0};
float kernel[3] = {1.0, 2.0, 3.0};
float *bias = NULL;

int outputLength = (inputLength + 2 * padLength - dilation * (kernelLength - 1) - 1) / stride + 1;
/*
 *  outputLength = 8
 */
float output[8] = {0.0};

conv1d_fp32(input, batch, inputChannels, inputLength, kernel, kernelLength, stride, padLength, dilation,
bias, output, outputChannels);
/*
 * output = [14.0 20.0 26.0 32.0 38.0 44.0 50.0 56.0]
 */
```

6.4.9.2.1.3 conv1d_direct_?

Direct算法实现的通用1D卷积接口。

接口定义

C interface：

void conv1d_direct_fp32(const float *input, const int batch, const int inputChannels, const int inputLength, const float* kernel, const int kernelLength, const int stride, const int padLength, const int dilation, const float *bias, float *output, const int outputChannels);

void conv1d_direct_fp16(const __fp16 *input, const int batch, const int inputChannels, const int inputLength, const __fp16 *kernel, const int kernelLength, const int stride, const int padLength, const int dilation, const __fp16 *bias, __fp16 *output, const int outputChannels);

参数

参数名	类型	描述	输入/输出
input	• conv1d_direct_fp32中是float类型 • conv1d_direct_fp16中是__fp16类型	输入数据	输入
batch	int类型	输入数据的批数	输入

版权所有 © 华为技术有限公司

参数名	类型	描述	输入/输出
inputChannels	int类型	输入通道数	输入
inputLength	int类型	输入数据的长度	输入
kernel	• conv1d_direct_fp32中是float类型 • conv1d_direct_fp16中是__fp16类型	卷积核	输入
kernelLength	int类型	卷积核长度	输入
stride	int类型	步长	输入
padLength	int类型	在原始input数据两端分别置零的长度	输入
dilation	int类型	膨胀系数	输入
bias	• conv1d_direct_fp32中是float类型 • conv1d_direct_fp16中是__fp16类型	偏置值, 值为NULL时表示无偏置	输入
output	• conv1d_direct_fp32中是float类型 • conv1d_direct_fp16中是__fp16类型	输出结果数据	输出
outputChannels	int类型	输出通道数	输入

依赖

#include "conv.h"

示例

C interface：

```
int batch = 1;
int inputChannels = 1;
int inputLength = 10;
int kernelLength = 3;
int stride = 1;
```

 版权所有 © 华为技术有限公司

```
int padLength = 0;
int dilation = 1;
int outputChannels = 1;
float input[10] = {1.0, 2.0, 3.0, 4.0, 5.0, 6.0, 7.0, 8.0, 9.0, 10.0};
float kernel[3] = {1.0, 2.0, 3.0};
float *bias = NULL;

int outputLength = (inputLength + 2 * padLength - dilation * (kernelLength - 1) - 1) / stride + 1;
/*
 *  outputLength = 8
 */
float output[8] = {0.0};

conv1d_direct_fp32(input, batch, inputChannels, inputLength, kernel, kernelLength, stride, padLength,
dilation, bias, output, outputChannels);
/*
 * output = [14.0 20.0 26.0 32.0 38.0 44.0 50.0 56.0]
 */
```

6.4.9.2.1.4 conv1d_fft_?

FFT算法实现的1D卷积接口，仅支持膨胀系数dilation=1的非膨胀卷积计算。

接口定义

C interface：

void conv1d_fft_fp32(const float *input, const int batch, const int inputChannels, const int inputLength, const float* kernel, const int kernelLength, const int stride, const int padLength, const float *bias, float *output, const int outputChannels);

void conv1d_fft_fp16(const __fp16 *input, const int batch, const int inputChannels, const int inputLength, const __fp16 *kernel, const int kernelLength, const int stride, const int padLength, const __fp16 *bias, __fp16 *output, const int outputChannels);

参数

参数名	类型	描述	输入/输出
input	• conv1d_fft_fp32中是float类型 • conv1d_fft_fp16中是__fp16类型	输入数据	输入
batch	int类型	输入数据的批数	输入
inputChannels	int类型	输入通道数	输入
inputLength	int类型	输入数据的长度	输入
kernel	• conv1d_fft_fp32中是float类型 • conv1d_fft_fp16中是__fp16类型	卷积核	输入

 版权所有 © 华为技术有限公司

参数名	类型	描述	输入/输出
kernelLength	int类型	卷积核长度	输入
stride	int类型	步长	输入
padLength	int类型	在原始input数据两端分别置零的长度	输入
bias	• conv1d_fft_fp32中是float类型 • conv1d_fft_fp16中是__fp16类型	偏置值, 值为NULL时表示无偏置	输入
output	• conv1d_fft_fp32中是float类型 • conv1d_fft_fp16中是__fp16类型	输出结果数据	输出
outputChannels	int类型	输出通道数	输入

依赖

#include "conv.h"

示例

C interface：

```
  int batch = 1;
  int inputChannels = 1;
  int inputLength = 10;
  int kernelLength = 3;
  int stride = 1;
  int padLength = 0;
  int outputChannels = 1;
  float input[10] = {1.0, 2.0, 3.0, 4.0, 5.0, 6.0, 7.0, 8.0, 9.0, 10.0};
  float kernel[3] = {1.0, 2.0, 3.0};
  float *bias = NULL;

  int outputLength = (inputLength + 2 * padLength - (kernelLength - 1) - 1) / stride + 1;
  /*
   *    outputLength = 8
   */
  float output[8] = {0.0};

  conv1d_fft_fp32(input, batch, inputChannels, inputLength, kernel, kernelLength, stride, padLength, bias,
output, outputChannels);
  /*
   * output = [14.0 20.0 26.0 32.0 38.0 44.0 50.0 56.0]
   */
```


版权所有 © 华为技术有限公司

6.4.9.2.1.5 conv1d_gemm_?

GEMM算法实现的通用1D卷积接口。

接口定义

C interface：

void conv1d_gemm_fp32(const float *input, const int batch, const int inputChannels, const int inputLength, const float* kernel, const int kernelLength, const int stride, const int padLength, const int dilation, const float *bias, float *output, const int outputChannels);

void conv1d_gemm_fp16(const __fp16 *input, const int batch, const int inputChannels, const int inputLength, const __fp16 *kernel, const int kernelLength, const int stride, const int padLength, const int dilation, const __fp16 *bias, __fp16 *output, const int outputChannels);

参数

参数名	类型	描述	输入/输出
input	• conv1d_gemm_fp32中是float类型 • conv1d_gemm_fp16中是__fp16类型	输入数据	输入
batch	int类型	输入数据的批数	输入
inputChannels	int类型	输入通道数	输入
inputLength	int类型	输入数据的长度	输入
kernel	• conv1d_gemm_fp32中是float类型 • conv1d_gemm_fp16中是__fp16类型	卷积核	输入
kernelLength	int类型	卷积核长度	输入
stride	int类型	步长	输入
padLength	int类型	在原始input数据两端分别置零的长度	输入
dilation	int类型	膨胀系数	输入

版权所有 © 华为技术有限公司

参数名	类型	描述	输入/输出
bias	• conv1d_gemm_fp32中是float类型 • conv1d_gemm_fp16中是__fp16类型	偏置值, 值为NULL时表示无偏置	输入
output	• conv1d_gemm_fp32中是float类型 • conv1d_gemm_fp16中是__fp16类型	输出结果数据	输出
outputChannels	int类型	输出通道数	输入

依赖

#include "conv.h"

示例

C interface：

```
  int batch = 1;
  int inputChannels = 1;
  int inputLength = 10;
  int kernelLength = 3;
  int stride = 1;
  int padLength = 0;
  int dilation = 1;
  int outputChannels = 1;
  float input[10] = {1.0, 2.0, 3.0, 4.0, 5.0, 6.0, 7.0, 8.0, 9.0, 10.0};
  float kernel[3] = {1.0, 2.0, 3.0};
  float *bias = NULL;

  int outputLength = (inputLength + 2 * padLength - dilation * (kernelLength - 1) - 1) / stride + 1;
  /*
   *  outputLength = 8
   */
  float output[8] = {0.0};

  conv1d_gemm_fp32(input, batch, inputChannels, inputLength, kernel, kernelLength, stride, padLength,
dilation, bias, output, outputChannels);
  /*
   * output = [14.0 20.0 26.0 32.0 38.0 44.0 50.0 56.0]
   */
```

6.4.9.2.1.6 conv1d_winograd_?

Winograd算法实现的1D卷积接口，仅支持步长stride=1，膨胀系数dilation=1的非膨胀卷积计算。

接口定义

C interface：

 版权所有 © 华为技术有限公司

void conv1d_winograd_fp32(const float *input, const int batch, const int inputChannels, const int inputLength, const float* kernel, const int kernelLength, const int padLength, const float *bias, float *output, const int outputChannels);

void conv1d_winograd_fp16(const __fp16 *input, const int batch, const int inputChannels, const int inputLength, const __fp16 *kernel, const int kernelLength, const int padLength, const __fp16 *bias, __fp16 *output, const int outputChannels);

参数

参数名	类型	描述	输入/输出
input	• conv1d_winograd_fp32中是float类型 • conv1d_winograd_fp16中是__fp16类型	输入数据	输入
batch	int类型	输入数据的批数	输入
inputChannels	int类型	输入通道数	输入
inputLength	int类型	输入数据的长度	输入
kernel	• conv1d_winograd_fp32中是float类型 • conv1d_winograd_fp16中是__fp16类型	卷积核	输入
kernelLength	int类型	卷积核长度	输入
padLength	int类型	在原始input数据两端分别置零的长度	输入
bias	• conv1d_winograd_fp32中是float类型 • conv1d_winograd_fp16中是__fp16类型	偏置值, 值为NULL时表示无偏置	输入
output	• conv1d_winograd_fp32中是float类型 • conv1d_winograd_fp16中是__fp16类型	输出结果数据	输出
outputChannels	int类型	输出通道数	输入

版权所有 © 华为技术有限公司

依赖

#include "conv.h"

示例

C interface：

```
   int batch = 1;
   int inputChannels = 1;
   int inputLength = 10;
   int kernelLength = 3;
   int padLength = 0;
   int outputChannels = 1;
   float input[10] = {1.0, 2.0, 3.0, 4.0, 5.0, 6.0, 7.0, 8.0, 9.0, 10.0};
   float kernel[3] = {1.0, 2.0, 3.0};
   float *bias = NULL;

   int outputLength = (inputLength + 2 * padLength - (kernelLength - 1) - 1) + 1;
   /*
    *   outputLength = 8
    */
   float output[8] = {0.0};

   conv1d_winograd_fp32(input, batch, inputChannels, inputLength, kernel, kernelLength, padLength, bias,
output, outputChannels);
   /*
    * output = [14.0 20.0 26.0 32.0 38.0 44.0 50.0 56.0]
    */
```

6.4.9.2.1.7 conv2d_?

通用2D卷积接口，根据输入特征，调用高性能算法实现。

接口定义

C interface：

void conv2d_fp32(const float *input, const int batch, const int inputChannels, const int inputHeight, const int inputWidth, const float *kernel, const int kernelHeight, const int kernelWidth, const int strideY, const int strideX, const int padHeight, const int padWidth, const int dilationY, const int dilationX, const float *bias, float *output, const int outputChannels);

void conv2d_fp16(const __fp16 *input, const int batch, const int inputChannels, const int inputHeight, const int inputWidth, const __fp16 *kernel, const int kernelHeight, const int kernelWidth, const int strideY, const int strideX, const int padHeight, const int padWidth, const int dilationY, const int dilationX, const __fp16 *bias, __fp16 *output, const int outputChannels);

参数

参数名	类型	描述	输入/输出
input	• conv2d_fp32中是float类型 • conv2d_fp16中是__fp16类型	输入数据	输入

版权所有 © 华为技术有限公司

参数名	类型	描述	输入/输出
batch	int类型	输入数据的批数	输入
inputChannels	int类型	输入通道数	输入
inputHeight	int类型	输入数据的高度	输入
inputWidth	int类型	输入数据的宽度	输入
kernel	• conv2d_fp32中是float类型 • conv2d_fp16中是__fp16类型	卷积核	输入
kernelHeight	int类型	卷积核的高度	输入
kernelWidth	int类型	卷积核的宽度	输入
strideY	int类型	卷积核在高度方向移动的步长	输入
strideX	int类型	卷积核在宽度方向移动的步长	输入
padHeight	int类型	在原始input数据高度方向两端分别置零的长度	输入
padWidth	int类型	在原始input数据宽度方向两端分别置零的长度	输入
dilationY	int类型	高度方向的膨胀系数	输入
dilationX	int类型	宽度方向的膨胀系数	输入
bias	• conv2d_fp32中是float类型 • conv2d_fp16中是__fp16类型	偏置值, 值为NULL时表示无偏置	输入
output	• conv2d_fp32中是float类型 • conv2d_fp16中是__fp16类型	输出结果数据	输出
outputChannels	int类型	输出通道数	输入

 版权所有 © 华为技术有限公司

依赖

#include "conv.h"

示例

C interface：

```
int batch = 1;
int inputChannels = 1;
int inputHeight = 6;
int inputWidth = 6;
int kernelHeight = 3;
int kernelWidth = 3;
int strideY = 1;
int strideX = 1;
int padHeight = 0;
int padWidth = 0;
int dilationY = 1;
int dilationX = 1;
int outputChannels = 1;
float input[36] = {1.0, 2.0, 3.0, 4.0, 5.0, 6.0,
                   7.0, 8.0, 9.0, 10.0, 11.0, 12.0,
                   13.0, 14.0, 15.0, 16.0, 17.0, 18.0,
                   19.0, 20.0, 21.0, 22.0, 23.0, 24.0,
                   25.0, 26.0, 27.0, 28.0, 29.0, 30.0,
                   31.0, 32.0, 33.0, 34.0, 35.0, 36.0};
float kernel[9] = {1.0, 2.0, 3.0,
                   4.0, 5.0, 6.0,
                   7.0, 8.0, 9.0};

float *bias = NULL;
int outputHeight = (inputHeight + 2 * padHeight - dilationY * (kernelHeight - 1) - 1) / strideY + 1;
int outputWidth = (inputWidth + 2 * padWidth - dilationX * (kernelWidth - 1) - 1) / strideX + 1;
/*
 *  outputHeight x outputWidth = 4 x 4
 */

float output[16] = {0.0};
conv2d_fp32(input, batch, inputChannels, inputHeight, inputWidth, kernel, kernelHeight, kernelWidth,
strideY, strideX, padHeight, padWidth, dilationY, dilationX, bias, output, outputChannels);
/*
 * output = [474.0 519.0 564.0 609.0,
 *           744.0 789.0 834.0 879.0,
 *           1014.0 1059.0 1104.0 1149.0,
 *           1284.0 1329.0 1374.0 1419.0]
 */
```

6.4.9.2.1.8 conv2d_direct_?

Direct算法实现的通用2D卷积接口。

接口定义

C interface：

void conv2d_direct_fp32(const float *input, const int batch, const int inputChannels, const int inputHeight, const int inputWidth, const float *kernel, const int kernelHeight, const int kernelWidth, const int strideY, const int strideX, const int padHeight, const int padWidth, const int dilationY, const int dilationX, const float *bias, float *output, const int outputChannels);

 版权所有 © 华为技术有限公司

```
void conv2d_direct_fp16(const __fp16 *input, const int batch, const int inputChannels, const int inputHeight, const int inputWidth, const __fp16 *kernel, const int kernelHeight, const int kernelWidth, const int strideY, const int strideX, const int padHeight, const int padWidth, const int dilationY, const int dilationX, const __fp16 *bias, __fp16 *output, const int outputChannels);
```

参数

参数名	类型	描述	输入/输出
input	• conv2d_direct_fp32中是float类型 • conv2d_direct_fp16中是__fp16类型	输入数据	输入
batch	int类型	输入数据的批数	输入
inputChannels	int类型	输入通道数	输入
inputHeight	int类型	输入数据的高度	输入
inputWidth	int类型	输入数据的宽度	输入
kernel	• conv2d_direct_fp32中是float类型 • conv2d_direct_fp16中是__fp16类型	卷积核	输入
kernelHeight	int类型	卷积核的高度	输入
kernelWidth	int类型	卷积核的宽度	输入
strideY	int类型	卷积核在高度方向移动的步长	输入
strideX	int类型	卷积核在宽度方向移动的步长	输入
padHeight	int类型	在原始input数据高度方向两端分别置零的长度	输入
padWidth	int类型	在原始input数据宽度方向两端分别置零的长度	输入
dilationY	int类型	高度方向的膨胀系数	输入
dilationX	int类型	宽度方向的膨胀系数	输入

 版权所有 © 华为技术有限公司

参数名	类型	描述	输入/输出
bias	• conv2d_direct_fp32中是float类型 • conv2d_direct_fp16中是__fp16类型	偏置值, 值为NULL时表示无偏置	输入
output	• conv2d_direct_fp32中是float类型 • conv2d_direct_fp16中是__fp16类型	输出结果数据	输出
outputChannels	int类型	输出通道数	输入

依赖

#include "conv.h"

示例

C interface：

```
  int batch = 1;
  int inputChannels = 1;
  int inputHeight = 6;
  int inputWidth = 6;
  int kernelHeight = 3;
  int kernelWidth = 3;
  int strideY = 1;
  int strideX = 1;
  int padHeight = 0;
  int padWidth = 0;
  int dilationY = 1;
  int dilationX = 1;
  int outputChannels = 1;
  float input[36] = {1.0, 2.0, 3.0, 4.0, 5.0, 6.0,
                     7.0, 8.0, 9.0, 10.0, 11.0, 12.0,
                     13.0, 14.0, 15.0, 16.0, 17.0, 18.0,
                     19.0, 20.0, 21.0, 22.0, 23.0, 24.0,
                     25.0, 26.0, 27.0, 28.0, 29.0, 30.0,
                     31.0, 32.0, 33.0, 34.0, 35.0, 36.0};
  float kernel[9] = {1.0, 2.0, 3.0,
                     4.0, 5.0, 6.0,
                     7.0, 8.0, 9.0};

  float *bias = NULL;
  int outputHeight = (inputHeight + 2 * padHeight - dilationY * (kernelHeight - 1) - 1) / strideY + 1;
  int outputWidth = (inputWidth + 2 * padWidth - dilationX * (kernelWidth - 1) - 1) / strideX + 1;
  /*
   *  outputHeight x outputWidth = 4 x 4
   */

  float output[16] = {0.0};
  conv2d_direct_fp32(input, batch, inputChannels, inputHeight, inputWidth, kernel, kernelHeight,
kernelWidth, strideY, strideX, padHeight, padWidth, dilationY, dilationX, bias, output,
      outputChannels);
  /*
```


版权所有 © 华为技术有限公司

```
 * output = [474.0 519.0 564.0 609.0,
 *           744.0 789.0 834.0 879.0,
 *           1014.0 1059.0 1104.0 1149.0,
 *           1284.0 1329.0 1374.0 1419.0]
 */
```

6.4.9.2.1.9 conv2d_fft_?

FFT算法实现的2D卷积接口，仅支持膨胀系数为1的非膨胀卷积计算。

接口定义

C interface：

void conv2d_fft_fp32(const float *input, const int batch, const int inputChannels, const int inputHeight, const int inputWidth, const float *kernel, const int kernelHeight, const int kernelWidth, const int strideY, const int strideX, const int padHeight, const int padWidth, const float *bias, float *output, const int outputChannels);

void conv2d_fft_fp16(const __fp16 *input, const int batch, const int inputChannels, const int inputHeight, const int inputWidth, const __fp16 *kernel, const int kernelHeight, const int kernelWidth, const int strideY, const int strideX, const int padHeight, const int padWidth, const __fp16 *bias, __fp16 *output, const int outputChannels);

参数

参数名	类型	描述	输入/输出
input	• conv2d_fft_fp32中是float类型 • conv2d_fft_fp16中是__fp16类型	输入数据	输入
batch	int类型	输入数据的批数	输入
inputChannels	int类型	输入通道数	输入
inputHeight	int类型	输入数据的高度	输入
inputWidth	int类型	输入数据的宽度	输入
kernel	• conv2d_fft_fp32中是float类型 • conv2d_fft_fp16中是__fp16类型	卷积核	输入
kernelHeight	int类型	卷积核的高度	输入

版权所有 © 华为技术有限公司

参数名	类型	描述	输入/输出
kernelWidth	int类型	卷积核的宽度	输入
strideY	int类型	卷积核在高度方向移动的步长	输入
strideX	int类型	卷积核在宽度方向移动的步长	输入
padHeight	int类型	在原始input数据高度方向两端分别置零的长度	输入
padWidth	int类型	在原始input数据宽度方向两端分别置零的长度	输入
bias	• conv2d_fft_fp32中是float类型 • conv2d_fft_fp16中是__fp16类型	偏置值, 值为NULL时表示无偏置	输入
output	• conv2d_fft_fp32中是float类型 • conv2d_fft_fp16中是__fp16类型	输出结果数据	输出
outputChannels	int类型	输出通道数	输入

依赖

#include "conv.h"

示例

C interface：

```
int batch = 1;
int inputChannels = 1;
int inputHeight = 6;
int inputWidth = 6;
int kernelHeight = 3;
int kernelWidth = 3;
int strideY = 1;
int strideX = 1;
int padHeight = 0;
int padWidth = 0;
int outputChannels = 1;
float input[36] = {1.0, 2.0, 3.0, 4.0, 5.0, 6.0,
                   7.0, 8.0, 9.0, 10.0, 11.0, 12.0,
                   13.0, 14.0, 15.0, 16.0, 17.0, 18.0,
                   19.0, 20.0, 21.0, 22.0, 23.0, 24.0,
                   25.0, 26.0, 27.0, 28.0, 29.0, 30.0,
                   31.0, 32.0, 33.0, 34.0, 35.0, 36.0};
float kernel[9] = {1.0, 2.0, 3.0,
                   4.0, 5.0, 6.0,
```


版权所有 © 华为技术有限公司

```
                    7.0, 8.0, 9.0};

    float *bias = NULL;
    int outputHeight = (inputHeight + 2 * padHeight - (kernelHeight - 1) - 1) / strideY + 1;
    int outputWidth = (inputWidth + 2 * padWidth - (kernelWidth - 1) - 1) / strideX + 1;
    /*
     *   outputHeight x outputWidth = 4 x 4
     */

    float output[16] = {0.0};
    conv2d_fft_fp32(input, batch, inputChannels, inputHeight, inputWidth, kernel, kernelHeight, kernelWidth,
strideY, strideX, padHeight, padWidth, bias, output, outputChannels);
    /*
     * output = [474.0 519.0 564.0 609.0,
     *           744.0 789.0 834.0 879.0,
     *           1014.0 1059.0 1104.0 1149.0,
     *           1284.0 1329.0 1374.0 1419.0]
     */
```

6.4.9.2.1.10 conv2d_gemm_?

GEMM算法实现的通用2D卷积接口。

接口定义

C interface：

void conv2d_gemm_fp32(const float *input, const int batch, const int inputChannels, const int inputHeight, const int inputWidth, const float *kernel, const int kernelHeight, const int kernelWidth, const int strideY, const int strideX, const int padHeight, const int padWidth, const int dilationY, const int dilationX, const float *bias, float *output, const int outputChannels);

void conv2d_gemm_fp16(const __fp16 *input, const int batch, const int inputChannels, const int inputHeight, const int inputWidth, const __fp16 *kernel, const int kernelHeight, const int kernelWidth, const int strideY, const int strideX, const int padHeight, const int padWidth, const int dilationY, const int dilationX, const __fp16 *bias, __fp16 *output, const int outputChannels);

参数

参数名	类型	描述	输入/输出
input	• conv2d_gemm_fp32中是float类型 • conv2d_gemm_fp16中是__fp16类型	输入数据	输入
batch	int类型	输入数据的批数	输入
inputChannels	int类型	输入通道数	输入
inputHeight	int类型	输入数据的高度	输入

版权所有 © 华为技术有限公司

参数名	类型	描述	输入/输出
inputWidth	int类型	输入数据的宽度	输入
kernel	• conv2d_gemm_fp32中是float类型 • conv2d_gemm_fp16中是__fp16类型	卷积核	输入
kernelHeight	int类型	卷积核的高度	输入
kernelWidth	int类型	卷积核的宽度	输入
strideY	int类型	卷积核在高度方向移动的步长	输入
strideX	int类型	卷积核在宽度方向移动的步长	输入
padHeight	int类型	在原始input数据高度方向两端分别置零的长度	输入
padWidth	int类型	在原始input数据宽度方向两端分别置零的长度	输入
dilationY	int类型	高度方向的膨胀系数	输入
dilationX	int类型	宽度方向的膨胀系数	输入
bias	• conv2d_gemm_fp32中是float类型 • conv2d_gemm_fp16中是__fp16类型	偏置值, 值为NULL时表示无偏置	输入
output	• conv2d_gemm_fp32中是float类型 • conv2d_gemm_fp16中是__fp16类型	输出结果数据	输出
outputChannels	int类型	输出通道数	输入

依赖

```
#include "conv.h"
```

 版权所有 © 华为技术有限公司

示例

C interface：

```
int batch = 1;
int inputChannels = 1;
int inputHeight = 6;
int inputWidth = 6;
int kernelHeight = 3;
int kernelWidth = 3;
int strideY = 1;
int strideX = 1;
int padHeight = 0;
int padWidth = 0;
int dilationY = 1;
int dilationX = 1;
int outputChannels = 1;
float input[36] = {1.0, 2.0, 3.0, 4.0, 5.0, 6.0,
                   7.0, 8.0, 9.0, 10.0, 11.0, 12.0,
                   13.0, 14.0, 15.0, 16.0, 17.0, 18.0,
                   19.0, 20.0, 21.0, 22.0, 23.0, 24.0,
                   25.0, 26.0, 27.0, 28.0, 29.0, 30.0,
                   31.0, 32.0, 33.0, 34.0, 35.0, 36.0};
float kernel[9] = {1.0, 2.0, 3.0,
                   4.0, 5.0, 6.0,
                   7.0, 8.0, 9.0};

float *bias = NULL;
int outputHeight = (inputHeight + 2 * padHeight - dilationY * (kernelHeight - 1) - 1) / strideY + 1;
int outputWidth = (inputWidth + 2 * padWidth - dilationX * (kernelWidth - 1) - 1) / strideX + 1;
/*
 *  outputHeight x outputWidth = 4 x 4
 */

float output[16] = {0.0};
conv2d_gemm_fp32(input, batch, inputChannels, inputHeight, inputWidth, kernel, kernelHeight,
kernelWidth, strideY, strideX, padHeight, padWidth, dilationY, dilationX, bias, output,
    outputChannels);
/*
 * output = [474.0 519.0 564.0 609.0,
 *           744.0 789.0 834.0 879.0,
 *           1014.0 1059.0 1104.0 1149.0,
 *           1284.0 1329.0 1374.0 1419.0]
 */
```

6.4.9.2.1.11 conv2d_winograd_?

Winograd算法实现的2D卷积接口，仅支持步长为1、膨胀系数为1的非膨胀卷积计算。

接口定义

C interface：

void conv2d_winograd_fp32(const float *input, const int batch, const int inputChannels, const int inputHeight, const int inputWidth, const float *kernel, const int kernelHeight, const int kernelWidth,const int padHeight, const int padWidth, const float *bias, float *output, const int outputChannels);

void conv2d_winograd_fp16(const __fp16 *input, const int batch, const int inputChannels, const int inputHeight, const int inputWidth, const __fp16 *kernel, const int kernelHeight, const int kernelWidth, const int padHeight, const int padWidth, const __fp16 *bias, __fp16 *output, const int outputChannels);

 版权所有 © 华为技术有限公司

参数

参数名	类型	描述	输入/输出
input	• conv2d_winograd_fp32中是float类型 • conv2d_winograd_fp16中是__fp16类型	输入数据	输入
batch	int类型	输入数据的批数	输入
inputChannels	int类型	输入通道数	输入
inputHeight	int类型	输入数据的高度	输入
inputWidth	int类型	输入数据的宽度	输入
kernel	• conv2d_winograd_fp32中是float类型 • conv2d_winograd_fp16中是__fp16类型	卷积核	输入
kernelHeight	int类型	卷积核的高度	输入
kernelWidth	int类型	卷积核的宽度	输入
padHeight	int类型	在原始input数据高度方向两端分别置零的长度	输入
padWidth	int类型	在原始input数据宽度方向两端分别置零的长度	输入
bias	• conv2d_winograd_fp32中是float类型 • conv2d_winograd_fp16中是__fp16类型	偏置值, 值为NULL时表示无偏置	输入
output	• conv2d_winograd_fp32中是float类型 • conv2d_winograd_fp16中是__fp16类型	输出结果数据	输出
outputChannels	int类型	输出通道数	输入

版权所有 © 华为技术有限公司

依赖

#include "conv.h"

示例

C interface：

```
int batch = 1;
int inputChannels = 1;
int inputHeight = 6;
int inputWidth = 6;
int kernelHeight = 3;
int kernelWidth = 3;
int strideY = 1;
int strideX = 1;
int padHeight = 0;
int padWidth = 0;
int dilationY = 1;
int dilationX = 1;
int outputChannels = 1;
float input[36] = {1.0, 2.0, 3.0, 4.0, 5.0, 6.0,
                   7.0, 8.0, 9.0, 10.0, 11.0, 12.0,
                   13.0, 14.0, 15.0, 16.0, 17.0, 18.0,
                   19.0, 20.0, 21.0, 22.0, 23.0, 24.0,
                   25.0, 26.0, 27.0, 28.0, 29.0, 30.0,
                   31.0, 32.0, 33.0, 34.0, 35.0, 36.0};
float kernel[9] = {1.0, 2.0, 3.0,
                   4.0, 5.0, 6.0,
                   7.0, 8.0, 9.0};

float *bias = NULL;
int outputHeight = (inputHeight + 2 * padHeight - (kernelHeight - 1) - 1) + 1;
int outputWidth = (inputWidth + 2 * padWidth - (kernelWidth - 1) - 1) + 1;
/*
 *  outputHeight x outputWidth = 4 x 4
 */

float output[16] = {0.0};
conv2d_winograd_fp32(input, batch, inputChannels, inputHeight, inputWidth, kernel, kernelHeight,
kernelWidth, padHeight, padWidth, bias, output, outputChannels);
/*
 * output = [474.0 519.0 564.0 609.0,
 *           744.0 789.0 834.0 879.0,
 *           1014.0 1059.0 1104.0 1149.0,
 *           1284.0 1329.0 1374.0 1419.0]
 */
```

6.4.9.2.1.12 conv2d_separable_?

Separable算法实现的2D卷积接口，仅支持步长为1、膨胀系数为1的非膨胀卷积计算，并且满足卷积核kernel矩阵等于两个向量外积的情况。

接口定义

C interface：

void conv2d_separable_fp32(const float *input, const int inputHeight, const int inputWidth, const float *fx, const int fxlength, const float *fy, const int fylength, const int padHeight, const int padWidth, const float *bias, float *output);

 版权所有 © 华为技术有限公司

参数

参数名	类型	描述	输入/输出
input	float类型	输入数据	输入
inputHeight	int类型	输入数据的高度	输入
inputWidth	int类型	输入数据的宽度	输入
fx	float类型	卷积核行方向	输入
fxLength	int类型	卷积核行方向大小	输入
fy	float类型	卷积核列方向	输入
fyLength	int类型	卷积核列方向大小	输入
padHeight	int类型	在原始input数据高度方向两端分别置零的长度	输入
padWidth	int类型	在原始input数据宽度方向两端分别置零的长度	输入
bias	float类型	偏置值，暂不支持, 值为NULL时表示无偏置	输入
output	float类型	输出结果数据	输出

依赖

#include "conv.h"

示例

C interface：

```
int inputHeight = 6;
int inputWidth = 6;
int kernelHeight = 3;
int kernelWidth = 3;
int padHeight = 0;
int padWidth = 0;
int outputChannels = 1;
float input[36] = {1.0, 2.0, 3.0, 4.0, 5.0, 6.0,
                   7.0, 8.0, 9.0, 10.0, 11.0, 12.0,
                   13.0, 14.0, 15.0, 16.0, 17.0, 18.0,
                   19.0, 20.0, 21.0, 22.0, 23.0, 24.0,
                   25.0, 26.0, 27.0, 28.0, 29.0, 30.0,
                   31.0, 32.0, 33.0, 34.0, 35.0, 36.0};
float fx[3] = {1.0, 2.0, 3.0};
float fy[3] = {4.0, 5.0, 6.0};

float *bias = NULL;
int outputHeight = (inputHeight + 2 * padHeight - (kernelHeight - 1) - 1) + 1;
```

 版权所有 © 华为技术有限公司

```
    int outputWidth = (inputWidth + 2 * padWidth - (kernelWidth - 1) - 1) + 1;
    /*
     *  outputHeight x outputWidth = 4 x 4
     */

    float output[16] = {0.0};
    conv2d_separable_fp32(input, inputHeight, inputWidth, fx, kernelWidth, fy, kernelHeight, padHeight,
padWidth, bias, output);
    /*
     * output = [822.000000, 912.000000, 1002.000000, 1092.000000,
     *           1362.000000, 1452.000000, 1542.000000, 1632.000000,
     *           1902.000000, 1992.000000, 2082.000000, 2172.000000,
     *           2442.000000, 2532.000000, 2622.000000, 2712.000000]
     */
```

6.4.9.2.2 上下采样接口

6.4.9.2.2.1 函数说明

函数族	数据类型	描述
usImage2X_cpu	fp32	上采样upsampling接口
dsImage_cpu	fp32	下采样downsampling接口

6.4.9.2.2.2 usImage2X_cpu

上采样upsampling接口。

接口定义

C interface：

void usImage2X_cpu(float* outbuf, const float* inimg, const float* filter, int filter_width, int n, int m);

参数

参数名	类型	描述	输入/输出
outbuf	float类型	上采样最终结果输出	输出
inimg	float类型	输入数据	输入
filter	float类型	滤波器	输入
filter_width	int类型	滤波器的长度	输入
n	int类型	输入数据的宽度	输入
m	int类型	输入数据的高度	输入

版权所有 © 华为技术有限公司

依赖

#include "conv.h"

示例

C interface：

```
int n = 6;
int m = 6;
float inimg[36] = {1.0, 2.0, 3.0, 4.0, 5.0, 6.0,
                   7.0, 8.0, 9.0, 10.0, 11.0, 12.0,
                   13.0, 14.0, 15.0, 16.0, 17.0, 18.0,
                   19.0, 20.0, 21.0, 22.0, 23.0, 24.0,
                   25.0, 26.0, 27.0, 28.0, 29.0, 30.0,
                   31.0, 32.0, 33.0, 34.0, 35.0, 36.0};
int filter_width = 3;
float filter[3] = {1.0, 2.0, 3.0};
float outbuf[144] = {0.0};
/*
 * output size is 2m x 2n
 */

usImage2X_cpu(outbuf, inimg, filter, filter_width, n, m);
/*
 *  outbuf = [1.0 8.0 2.0 14.0 3.0 20.0 4.0 26.0 5.0 32.0 6.0 17.0
 *            23.0 130.0 28.0 178.0 33.0 208.0 38.0 238.0 43.0 268.0 48.0 139.0
 *            7.0 38.0 8.0 50.0 9.0 56.0 10.0 62.0 11.0 68.0 12.0 35.0
 *            54.0 288.0 60.0 372.0 66.0 408.0 72.0 444.0 78.0 480.0 84.0 246.0
 *            13.0 68.0 14.0 86.0 15.0 92.0 16.0 98.0 17.0 104.0 18.0 53.0
 *            90.0 468.0 96.0 588.0 102.0 624.0 108.0 660.0 114.0 696.0 120.0 354.0
 *            19.0 98.0 20.0 122.0 21.0 128.0 22.0 134.0 23.0 140.0 24.0 71.0
 *            126.0 648.0 132.0 804.0 138.0 840.0 144.0 876.0 150.0 912.0 156.0 462.0
 *            25.0 128.0 26.0 158.0 27.0 164.0 28.0 170.0 29.0 176.0 30.0 89.0
 *            162.0 828.0 168.0 1020.0 174.0 1056.0 180.0 1092.0 186.0 1128.0 192.0 570.0]
 */
```

6.4.9.2.2.3 dsImage_cpu

下采样downsampling接口。

接口定义

C interface：

void dsImage_cpu(float* out, const float* in, float *tmp, const float* filter, int filter_width, int n, int m);

参数

参数名	类型	描述	输入/输出
out	float类型	下采样最终结果输出	输出
in	float类型	输入数据	输入
tmp	float类型	下采样中间结果输出	输出
filter	float类型	滤波器	输入

版权所有 © 华为技术有限公司

参数名	类型	描述	输入/输出
filter_width	int类型	滤波器的长度	输入
n	int类型	输入数据的宽度	输入
m	int类型	输入数据的高度	输入

依赖

#include "conv.h"

示例

C interface：

```
int n = 12;
int m = 12;
float in[144] = {1.0, 8.0, 2.0, 14.0, 3.0, 20.0, 4.0, 26.0, 5.0, 32.0, 6.0, 17.0,
                 23.0, 130.0, 28.0, 178.0, 33.0, 208.0, 38.0, 238.0, 43.0, 268.0, 48.0, 139.0,
                 7.0, 38.0, 8.0, 50.0, 9.0, 56.0, 10.0, 62.0, 11.0, 68.0, 12.0, 35.0,
                 54.0, 288.0, 60.0, 372.0, 66.0, 408.0, 72.0, 444.0, 78.0, 480.0, 84.0, 246.0,
                 13.0, 68.0, 14.0, 86.0, 15.0, 92.0, 16.0, 98.0, 17.0, 104.0, 18.0, 53.0,
                 90.0, 468.0, 96.0, 588.0, 102.0, 624.0, 108.0, 660.0, 114.0, 696.0, 120.0, 354.0,
                 19.0, 98.0, 20.0, 122.0, 21.0, 128.0, 22.0, 134.0, 23.0, 140.0, 24.0, 71.0,
                 126.0, 648.0, 132.0, 804.0, 138.0, 840.0, 144.0, 876.0, 150.0, 912.0, 156.0, 462.0,
                 25.0, 128.0, 26.0, 158.0, 27.0, 164.0, 28.0, 170.0, 29.0, 176.0, 30.0, 89.0,
                 162.0, 828.0, 168.0, 1020.0, 174.0, 1056.0, 180.0, 1092.0, 186.0, 1128.0, 192.0, 570.0,
                 31.0, 158.0, 32.0, 194.0, 33.0, 200.0, 34.0, 206.0, 35.0, 212.0, 36.0, 107.0,
                 87.0, 444.0, 90.0, 546.0, 93.0, 564.0, 96.0, 582.0, 99.0, 600.0, 102.0, 303.0, };
int filter_width = 3;
float filter[3] = {0.0, 1.0, 0.0};
float tmp[72] = {0.0};
/*
 * tmp size is m x [n/2]
 */
float out[36] = {0.0};
/*
 * output size is [m/2] x [n/2]
 */

dsImage_cpu(out, in, tmp, filter, filter_width, n, m);
/*
 * tmp = [1.0 2.0 3.0 4.0 5.0 6.0
 *        23.0 28.0 33.0 38.0 43.0 48.0
 *        7.0 8.0 9.0 10.0 11.0 12.0
 *        54.0 60.0 66.0 72.0 78.0 84.0
 *        13.0 14.0 15.0 16.0 17.0 18.0
 *        90.0 96.0 102.0 108.0 114.0 120.0
 *        19.0 20.0 21.0 22.0 23.0 24.0
 *        126.0 132.0 138.0 144.0 150.0 156.0
 *        25.0 26.0 27.0 28.0 29.0 30.0
 *        162.0 168.0 174.0 180.0 186.0 192.0]
 *
 * out = [1.0 2.0 3.0 4.0 5.0 6.0
 *        7.0 8.0 9.0 10.0 11.0 12.0
 *        13.0 14.0 15.0 16.0 17.0 18.0
 *        19.0 20.0 21.0 22.0 23.0 24.0
 *        25.0 26.0 27.0 28.0 29.0 30.0]
 */
```

 版权所有 © 华为技术有限公司

6.4.9.2.3 图像处理接口

6.4.9.2.3.1 使用指南

KML_CONV只提供filter2D的核心算子实现，用户需按使用指南将核心算子注册进开源OpenCV，并调用开源OpenCV filter2d接口完成滤波计算。

算子注册

步骤1 获取OpenCV源码。

下载地址：**https://github.com/opencv/opencv/archive/refs/tags/4.9.0.tar.gz**

步骤2 解压OpenCV安装包。

```
tar -xvf opencv-4.9.0.tar.gz
```

步骤3 注册kml_conv核心算子。

1. opencv/3rdparty 目录下新增名为“kconv”的文件夹。
   ```
   cd your_path_to_opencv/opencv-4.9.0/3rdparty
   mkdir -p kconv
   ```
2. 拷贝头文件及相应依赖库至“opencv/3rdparty/kconv”文件夹下。
   - 拷贝conv头文件。
     ```
     cp cv.h your_path_to_opencv/opencv-4.9.0/3rdparty/kconv
     ```
   - 拷贝CONV、BLAS、FFT依赖库。
     ```
     cp libkconv_ext.so your_path_to_opencv/opencv-4.9.0/3rdparty/Kunpeng
     cp libkblas.so your_path_to_opencv/opencv-4.9.0/3rdparty/Kunpeng
     cp libkfftf.so your_path_to_opencv/opencv-4.9.0/3rdparty/Kunpeng
     cp libkffth.so your_path_to_opencv/opencv-4.9.0/3rdparty/Kunpeng
     ```
3. “opencv/3rdparty/Kunpeng”文件夹下新增头文件与源文件。
   - 新增头文件kml_cv.hpp。
     ```
     #ifndef KML_CV_HPP
     #define KML_CV_HPP
     #include <opencv2/core/base.hpp>
     #include "cv.h"
     #endif
     ```
   - 新增源文件kml_cv.cpp。
     ```
     #include "kml_cv.hpp"
     ```
4. “opencv/3rdparty/kconv”文件夹下新增CMakeLists.txt。
   ```
   project(kml_conv_ext CXX)
   add_library(kml_conv_ext STATIC kml_cv.cpp)
   target_link_libraries(kml_conv_ext ${CMAKE_CURRENT_SOURCE_DIR}/libkconv.so $
   {CMAKE_CURRENT_SOURCE_DIR}/libkblas.so ${CMAKE_CURRENT_SOURCE_DIR}/libkfftf.so $
   {CMAKE_CURRENT_SOURCE_DIR}/libkffth.so)
   set(KML_HAL_VERSION "0.0.1" PARENT_SCOPE)
   set(KML_HAL_LIBRARIES "kml_conv_ext" PARENT_SCOPE)
   set(KML_HAL_HEADERS "kml_conv_ext.hpp" PARENT_SCOPE)
   set(KML_HAL_INCLUDE_DIRS "${CMAKE_BINARY_DIR}" PARENT_SCOPE)
   set_target_properties(kml_conv_ext PROPERTIES LINKER_LANGUAGE CXX)
   set_target_properties(kml_conv_ext PROPERTIES ARCHIVE_OUTPUT_DIRECTORY
   ${3P_LIBRARY_OUTPUT_PATH})
   if(NOT BUILD_SHARED_LIBS)
     ocv_install_target(kail_dnn_ext EXPORT OpenCVModules ARCHIVE DESTINATION $
   {OPENCV_3P_LIB_INSTALL_PATH} COMPONENT dev)
   endif()
   set(OPENCV_SRC_DIR "${CMAKE_SOURCE_DIR}")
   target_include_directories(kml_conv_ext PRIVATE ${CMAKE_CURRENT_SOURCE_DIR} $
   {OPENCV_SRC_DIR}/modules/core/include)
   configure_file("${CMAKE_CURRENT_SOURCE_DIR}/kml_cv.hpp" "${CMAKE_BINARY_DIR}/kml_cv.hpp"
   COPYONLY)
   ```

 版权所有 © 华为技术有限公司

```
configure_file("${CMAKE_CURRENT_SOURCE_DIR}/cv.h" "${CMAKE_BINARY_DIR}/cv.h" COPYONLY)
set_target_properties(${KML_LIBRARIES} PROPERTIES
      OUTPUT_NAME ${KML_LIBRARIES}
      DEBUG_POSTFIX "${OPENCV_DEBUG_POSTFIX}"
      COMPILE_PDB_NAME ${KML_LIBRARIES}
      COMPILE_PDB_NAME_DEBUG "${KML_LIBRARIES}${OPENCV_DEBUG_POSTFIX}"
      ARCHIVE_OUTPUT_DIRECTORY ${3P_LIBRARY_OUTPUT_PATH}
    )
```

5. 注册KML_CONV高性能filter2D核心算子。

   修改“opencv-4.9.0/CMakeLists.txt”第965-995行。

```
set(OpenCV_HAL "kml_conv_ext")

foreach(hal ${OpenCV_HAL})
  if(hal STREQUAL "carotene")
    ...
  elseif(hal STREQUAL "openvx")
    add_subdirectory(3rdparty/openvx)
    ocv_hal_register(OPENVX_HAL_LIBRARIES OPENVX_HAL_HEADERS OPENVX_HAL_INCLUDE_DIRS)
    list(APPEND OpenCV_USED_HAL "openvx (ver ${OPENVX_HAL_VERSION})")
  elseif(hal STREQUAL "kml_conv_ext")
    add_subdirectory(3rdparty/kconv)
    ocv_hal_register(KML_HAL_LIBRARIES KML_HEADERS KML_INCLUDE_DIRS)
    list(APPEND OpenCV_USED_HAL "kml_conv_ext (ver ${KML_VERSION})")
  else()
    ocv_debug_message(STATUS "OpenCV HAL: ${hal} ...")
    ocv_clear_vars(OpenCV_HAL_LIBRARIES OpenCV_HAL_HEADERS OpenCV_HAL_INCLUDE_DIRS)
    find_package(${hal} NO_MODULE QUIET)
    if(${hal}_FOUND)
      ocv_hal_register(OpenCV_HAL_LIBRARIES OpenCV_HAL_HEADERS OpenCV_HAL_INCLUDE_DIRS)
      list(APPEND OpenCV_USED_HAL "${hal} (ver ${${hal}_VERSION})")
    endif()
endif()
```

步骤4 编译安装OpenCV。

```
cd your_path_to_opencv/opencv-4.9.0
mkdir -p build
cd build
cmake .. -DWITH_ADE=OFF -DCMAKE_INSTALL_PREFIX=your_path_to_install_opencv # cmake到build文件夹下
make -j
make install # 安装到your_path_to_install_opencv中
```

----结束

OpenCV filter2D 接口定义

完成上述KML_CONV filter2D算子注册后，用户可直接调用OpenCV filter2D接口完成滤波计算。

OpenCV filter2D使用指南：

https://docs.opencv.org/4.9.0/d4/d86/group__imgproc__filter.html

C++ interface：

void cv::filter2D(InputArray src, OutputArray dst, int ddepth, InputArray kernel, Point anchor = Point(-1,-1), double delta = 0, int borderType = BORDER_DEFAULT)

参数列表：

 版权所有 © 华为技术有限公司

参数名	类型	描述	输入/输出
src	InputArray	输入图像	输入
dst	OutputArray	输出图像	输出
int	ddepth	输出图像的深度	输入
kernel	InputArray	卷积核	输入
anchor	Point	卷积核锚点，表示滤波起始点在卷积核中的相对位置 默认值为为Point(-1,-1),表示滤波起始点位于卷积核中心	输入
delta	0	偏置值，默认值为0	输入
int	borderType	边界填充类型，支持以下入参: BORDER_CONSTANT BORDER_REPLICATE BORDER_REFLECT BORDER_REFLECT_101 BORDER_REFLECT101 BORDER_DEFAULT（默认值） BORDER_ISOLATED	输入

完成上述KML_CONV算子注册后的OpenCV 将新增对CV_16F图像深度filter2D计算的功能支持。下表是OpenCV filter2D接口所支持的输入图像深度、输出图像深度组合。

输入图像深度	输出图像深度
CV_8U	-1/CV_16S/CV_32F/CV_64F （输出图像深度值为-1时，表示输出图像深度与输入图像深度相同）
CV_16U/CV_16S	-1/CV_32F/CV_64F
CV_32F	**-1/CV_32F**
CV_64F	-1/CV_64F
CV_16F	**-1/CV_16F**

当输入图像深度与输出图像深度均为CV_32F或CV_16F时，将调用KML_CONV高性能算子实现完成滤波计算。其他情况下，将调用OpenCV自身算子实现完成滤波计算。

 版权所有 © 华为技术有限公司

依赖

#include <opencv2/imgproc/imgproc.hpp>

示例

C++ interface：

```
cv::Mat src = (cv::Mat_<float>(6, 6) << 1, 2, 3, 4, 5, 6,
                                        7, 8, 9, 10, 11, 12,
                                        13, 14, 15, 16, 17, 18,
                                        19, 20, 21, 22, 23, 24,
                                        25, 26, 27, 28, 29, 30,
                                        31, 32, 33, 34, 35, 36);

cv::Mat kernel = (cv::Mat_<float>(3, 3) << 0, -1, 0,
                                           -1, 5, -1,
                                           0, -1, 0);
cv::Mat dst;
cv::filter2D(src, dst, -1, kernel, cv::Point(-1, -1), 0, cv::BORDER_CONSTANT);
/*
 * dst = [-4, -2, 0, 2, 4, 13;
 *        13, 8, 9, 10, 11, 25;
 *        25, 14, 15, 16, 17, 37;
 *        37, 20, 21, 22, 23, 49;
 *        49, 26, 27, 28, 29, 61;
 *        98, 70, 72, 74, 76, 115]
 */
```

6.4.9.2.3.2 函数说明

KML_CONV提供以下3个filter2D核心算子，替换OpenCV相应接口。

函数名	OpenCV所被替换接口	功能描述
kml_init_filter	cv_hal_filterInit	资源初始化
kml_execute_filter2d	cv_hal_filter	执行filter2D核心计算
kml_free_filter	cv_hal_filterFree	资源释放

6.4.9.2.3.3 kml_init_filter

filter2D计算资源初始化。

接口定义

C interface：

int kml_init_filter(cvhalFilter2D **context, unsigned char *kernel_data, size_t kernel_step, int kernel_type, int kernel_width, int kernel_height, int max_width, int max_height, int src_type, int dst_type, int borderType, double delta, int anchor_x, int anchor_y, bool allowSubmatrix, bool allowInplace);

 版权所有 © 华为技术有限公司

参数

参数名	类型	描述	输入/输出
context	cvhalFilter2D **	空结构体指针，完成资源初始化后，指向内部申请的资源结构体空间	输入/输出
kernel_data	unsigned char*	卷积核数据	输出
kernel_step	size_t	卷积核每行数据的字节数	输入
kernel_width	int	卷积核的宽度	输入
kernel_height	int	卷积核的高度	输入
max_width	int	完整图像宽度	输入
max_height	int	完整图像高度	输入
src_type	int	输入图像类型	输入
dst_type	int	输出图像类型	输入
borderType	int	边界填充类型	输入
delta	double	添加在输出图像的偏置值	输入
anchor_x	int	滤波起始点在卷积核中的宽度方向的相对坐标	输入
anchor_y	int	滤波起始点在卷积核中的高度方向的相对坐标	输入
allowSubMatrix	bool	是否支持输入图像是子矩阵，true表示支持，false表示不支持	输入
allowInplace	bool	是否支持inplace运算， true表示支持，false表示不支持	输入

函数接口入参由OpenCV filter2D上层对入参处理传入，kml_init_filter不支持用户显式调用。

 版权所有 © 华为技术有限公司

返回值

场景	返回值
输入图像深度、输出图像深度不为CV_16F或CV_32F	1
卷积核通道数不等于1	1
传入OpenCV filter2D接口的边界填充类型不在以下列表中： BORDER_CONSTANT BORDER_REPLICATE BORDER_REFLECT BORDER_REFLECT_101 BORDER_REFLECT101 BORDER_DEFAULT BORDER_ISOLATED	1
内部资源申请失败	-1
其余场景	0

6.4.9.2.3.4 kml_execute_filter2d

执行filter2D核心滤波计算过程。

接口定义

C interface：

int kml_execute_filter2d(cvhalFilter2D *context, unsigned char *src_data, size_t src_step, unsigned char *dst_data, size_t dst_step, int width, int height, int full_width, int full_height, int offset_x, int offset_y);

参数

参数名	类型	描述	输入/输出
context	cvhalFilter2D*	空结构体指针，完成资源初始化后，指向内部申请的资源结构体空间	输入/输出
src_data	unsigned char*	输入图像数据	输出
src_step	size_t	输入图像每行数据的字节数	输入

版权所有 © 华为技术有限公司

参数名	类型	描述	输入/输出
dst_data	unsigned char*	输出图像数据	输入
dst_step	size_t	输出图像每行数据的字节数	输入
width	int	输入图像感兴趣区域的宽度	输入
height	int	输入图像感兴趣区域的高度	输入
full_width	int	完整图像的宽度	输入
full_height	int	完整图像的高度	输入
offset_x	int	感兴趣区域起始点相对于完整图像起始点在宽度方向的偏移量	输入
offset_y	int	感兴趣区域起始点相对于完整图像起始点在高度方向的偏移量	输入

函数接口入参由OpenCV filter2D上层对入参处理传入，kml_init_filter不支持用户显式调用。

返回值

0

6.4.9.2.3.5 kml_free_filter

filter2D计算资源释放。

接口定义

C interface：

int kml_free_filter(cvhalFilter2D *context);

参数

参数名	类型	描述	输入/输出
context	cvhalFilter2D*	空结构体指针，完成资源初始化后，指向内部申请的资源结构体空间	输入/输出

版权所有 © 华为技术有限公司

函数接口入参由OpenCV filter2D上层对入参处理传入，kml_init_filter不支持用户显式调用。

返回值

场景	返回值
资源释放失败	-1
资源释放成功	0

6.5 应用数学库

应用数学库是上层的算法数学库，利用基础数学库、MPI等其他组件，实现了大型矩阵运算等数学问题，包含KML_LAPACK线性代数运算库、KML_SCALAPACK可扩展线性代数运算库、KML_SOLVER稀疏求解器库、KML_EIGENSOLVER特征值求解库。

6.5.1 KML_LAPACK 库函数说明

6.5.1.1 函数说明

线性代数运算库（Linear Algebra PACKage），通过分块、求解算法组合、多线程、BLAS接口优化等手段，基于鲲鹏架构对LAPACK的计算效率进行了优化。

当前，Netlib LAPACK标准定义了2000多个LAPACK接口规范，包括接口名称、参数列表和功能说明。鲲鹏HPCKit已对其中的200余个接口进行了优化，具体清单请参见**表6-36**。其余接口将在后续版本中通过统一打包为完整的LAPACK函数库发布。目前社区文档中仅包含已优化的200多个接口说明，其余接口的文档请参见**Netlib LAPACK文档**。

LAPACK原本用Fortran语言实现并由Fortran语言调用，也可由C语言直接调用。以下采用C语法说明标准Fortran接口。

由于LAPACK部分接口会在内部分配额外的内存，且这些额外内存主要受矩阵规模及线程数大小的影响，其具体关系如**表6-35**所示（其他接口所需额外内存均小于这三类接口）。因此，在机器内存配置为256GB的典型场景下，为更好的使用优化接口，我们建议最大规模不超过45000（涉及**表6-35**中函数接口除外，其以**表6-35**中建议规模为准）。

表 6-35 LAPACK 额外内存使用及建议规模说明

函数	额外内存占用空间（单位为元素个数）	建议最大规模（256GB机器内存）
?potri	6400 * ldb + 640000	45000
?getri	2000 * ldb + 40000 * 线程数 + 400000	45000
?stedc	(n * n + 8 * n + 10000) * 线程数 + 10000000	18000

版权所有 © 华为技术有限公司

需要注意的是，当矩阵规模超过建议值时，可能调用到性能较低的接口而非优化接口，进而会导致最终使用的性能下降。换言之，若机器内存足够大，能满足额外内存的需求，则可在更大矩阵规模下获得最佳性能。

举例：以dsyevd为例，其实现需要调用dstedc接口。则在典型场景下，假定元素类型为双精度（8字节每元素），矩阵大小为18000x18000，使用96个线程，则所需内存的计算如下：

1. 额外的内存占用：
   ((18000 * 18000 + 18000 * 8 + 10000) * 96 + 10000000) * 8 = 249,030,272,000字节（约231.9 GB）
2. 必须的内存占用包含：
   - 矩阵本身内存占用：18000 * 18000 * 8 = 2,592,000,000字节（约2.4GB）
   - 工作数组work（6n+2n*n + 3+5n）内存占用：(1 * 6 * 18000 + 2 * 18000 * 18000 + 3 + 5 * 18000) * 8=5,185,584,024字节（约4.8GB）

故在此情形下，调用dsyevd时，共需要内存约为231.9 + 2.4 + 4.8 = 239.1GB。

6.5.1.2 数据结构

提供KML_LAPACK库函数的数据类型和存储类型说明。

数据类型

数据类型	类型描述
s	单精度浮点类型
d	双精度浮点类型
c	单精度复数类型
z	双精度复数类型

存储类型

普通矩阵按照Fortran的列优先方式存储（column-major），数据结构中涉及到的索引从1开始（如ipiv中保存的值）。

压缩存储矩阵（packed）：以列优先方式存储，对于实对称、共轭对称、上三角或下三角矩阵，只需一半的元素即可确定整个矩阵，因此可使用压缩格式只保存这一部分数据。对于$n \times n$大小的压缩存储矩阵A，对应元素A_{ij}的存储位置如下：

- 上三角存储（uplo='U'，且$1 \le i \le j \le n$）：$i-1+j \times (j-1)/2$。
- 下三角存储（uplo='L'，且$1 \le j \le i \le n$）：$i-1+(j-1) \times (2n-j)/2$。

6.5.1.3 函数定义

6.5.1.3.1 优化的 LAPACK 函数

对优化的LAPACK的所有函数族进行描述。

 版权所有 © 华为技术有限公司

表 6-36 优化的 LAPACK 函数列表

功能	函数族	数据类型	描述
矩阵分解	?getrf	s，d，c，z	计算矩阵的LU分解，允许行交换。
	?geqrf	s，d，c，z	矩阵QR分解。
	?gerqf	s，d，c，z	矩阵RQ分解。
	?geqlf	s，d，c，z	矩阵QL分解。
	?gelqf	s，d，c，z	矩阵LQ分解。
	?potrf	s，d，c，z	实对称或共轭对称正定矩阵的Cholesky分解。
	?pttrf	s，d，c，z	计算实（共轭）对称正定三对角矩阵A的LDL*或U*DU分解。
	?gttrf	s，d，c，z	计算一般三对角矩阵A的LU分解。
	?sptrf	s，d，c，z	计算压缩对称矩阵的LDL*或U*DU分解。
	?hptrf	c，z	计算压缩Hermite矩阵的LDL*或U*DU分解
	?pptrf	s，d，c，z	计算压缩存储的对称正定矩阵的LLT或UTU分解。
线性方程组求解	?ppsv	s，d，c，z	压缩存储实对称或共轭对称正定矩阵的Cholesky分解。
	?gesv	s，d，c，z	通过LU分解的结果来求解线性方程组。
	?ptsv	s，d，c，z	求解线性方程组，其中系数矩阵A为实（共轭）对称正定三对角矩阵。
	?gtsv	s，d，c，z	求解线性方程组A*X=B，其中系数矩阵A为一般三对角矩阵。
	?posv	s，d，c，z	求解线性方程组A*X=B，其中系数矩阵A为实对称或共轭对称正定矩阵。
矩阵求逆	?getri	s，d，c，z	根据?getrf结果计算逆矩阵。
	?potri	s，d，c，z	根据?potrf结果计算逆矩阵。
	?pptri	s，d，c，z	根据?pptrf结果计算逆矩阵。
	?trtri	s，d，c，z	计算上/下三角矩阵的逆矩阵。
回代求解	?pttrs	s，d，c，z	求解三对角方程AX=B，其中系数矩阵A由?pttrf分解而来。
	?ptts2	s，d，c，z	求解三对角方程AX=B，其中系数矩阵A由?pttrf分解而来。

版权所有 © 华为技术有限公司

功能	函数族	数据类型	描述
	?gttrs	s，d，c，z	求解三对角方程A*X=B，或A**T *X=B，或A**H *X=B，其中系数矩阵A由?gttrf分解而来。
	?gtts2	s，d，c，z	求解三对角方程A*X=B，或A**T *X=B，或A**H *X=B，其中系数矩阵A由?gttrf分解而来。
	?trtrs	s，d，c，z	求解三角方程A * X = B或A**T * X = B。
	?getrs	s，d，c，z	求解一般线性方程AX=B，其中系数矩阵A由?getrf分解而来。
	?pptrs	s，d，c，z	求解压缩存储的对称正定线性方程AX=B，其中系数矩阵A由?pptrf分解而来。
特征值问题	?(sy/he)evd	s，d，c，z	计算实对称（Hermite）矩阵特征值和特征向量，其中特征向量通过分治算法计算。
	?(sy/he)ev	s，d，c，z	计算实对称（Hermite）矩阵特征值和特征向量。
	?steqr	s，d，c，z	使用QL或QR方法求解对称三对角矩阵的特征值和特征向量。
	?stedc	s，d，c，z	对称(Hermite)三对角矩阵的特征值求解，Divide-and-Conquer算法
	?sy(he)evx	s，d，c，z	计算对称（Hermite）矩阵规定范围内的特征值和特征向量（可选）。
	?sy(he)gv	s，d，c，z	计算广义特征值问题的所有特征值和特征向量（可选）。
	?geev	s，d，c，z	计算一般矩阵的部分特征值与特征向量。
奇异值问题	?gesvd	s，d，c，z	一般矩阵SVD分解
	?gebrd	s，d，c，z	一般矩阵变换到两对角矩阵
	?bdsdc	s，d	两对角矩阵的SVD分解，Divide-and-Conquer
	?gesdd	s，d，c，z	一般矩阵SVD分解，Divide-and-Conquer
	?bdsqr	s，d	两对角矩阵的SVD分解
最小二乘问题	?gels	s，d，c，z	使用矩阵的QR或LQ分解来求解超定或欠定线性方程组。
	?gelss	s，d，c，z	使用SVD方法对线性最小二乘问题求解其最小范数解。

 版权所有 © 华为技术有限公司

功能	函数族	数据类型	描述
	?gelsd	s，d，c，z	使用分治法对线性最小二乘问题求解其最小范数解。
其他	?(or,un)glq	s，d，c，z	生成具有正交行的实/复矩阵Q，其中计算Q的H由?gelqf返回的。
	?(or,un)gqr	s，d，c，z	生成具有正交行的实/复矩阵Q，其中计算Q的H由?geqrf返回的。
	?(or,un)grq	s，d，c，z	生成具有正交行的实/复矩阵Q，其中计算Q的H由?gerqf返回的。
	?(or,un)gql	s，d，c，z	生成具有正交列的实/复矩阵Q，其中计算Q的H由?geqlf返回的。
	?(or,un)mlq	s，d，c，z	计算$C = Q^* * C$或$C = C * Q^*$，其中Q是由?gelqf计算得到，Q^*为Q^T或Q^H。
	?(or,un)mqr	s，d，c，z	计算$C = Q^* * C$或$C = C * Q^*$，其中Q是由?geqrf计算得到，Q^*为Q^T或Q^H。
	?(or,un)mql	s，d，c，z	计算$C = Q^* * C$或$C = C * Q^*$，其中Q是由?geqlf计算得到，Q^*为Q^T或Q^H。
	?(or,un)mrq	s，d，c，z	计算$C = Q^* * C$或$C = C * Q^*$，其中Q是由?gerqf计算得到，Q^*为Q^T或Q^H。
	?sy(he)trd	s，d，c，z	将对称矩阵或Hermite矩阵通过相似变换成对称三对角T。
	?sy(he)trd_2stage	s，d，c，z	将对称矩阵A转为对称三对角矩阵T，即A**T*A*Q=T，其中Q为正交矩阵。
	?lasr	s，d，c，z	对矩阵A做平面旋转操作。
	?(or,un)mbr	s，d，c，z	计算C=Q*C或Q**T*C或C*Q或C*Q**T或C=P*C或P**T*C或C*P或C*P**T，其中正交矩阵Q和P**T由函数GEBRD计算得到，即A=Q*B*P**T。
	?(or,un)gtr	s，d，c，z	通过反射因子生成正交矩阵Q，其中反射因子由SYTRD计算得来。
	Kml?steqr2	s，d	使用隐式QL或QR方法计算对称三对角矩阵的所有特征值和可选的特征向量（仅有V case）。
	KmlBlockBandedMatrixSolve	d	求解分块大小为5的块状条带矩阵，同时返回矩阵的分解结果。

 版权所有 © 华为技术有限公司

功能	函数族	数据类型	描述
	KmlBlockBandedMatrixSubstitude	d	对分块大小为5的块状条带矩阵的分解结果进行回代。
辅助函数	?laset	s，d，c，z	初始化m*n的矩阵，且将对角线元素设置为beta，非对角线元素设置为alpha。
	?laswp	s，d，c，z	对矩阵做一系列的行交换操作。
	?lascl	s，d，c，z	对矩阵做scalar操作。
	?lange	s，d，c，z	计算矩阵的范数（包含：1范数，F范数，无穷范数等）。
	?lacpy	s，d，c，z	拷贝矩阵A的所有元素或部分元素到矩阵B。
混合精度线性方程组求解	?gesv	zc，ds	用混合精度法来求解一般线性方程组。

6.5.1.3.2 矩阵分解函数

6.5.1.3.2.1 ?getrf

计算矩阵A的LU分解，允许行交换（partial pivoting）。

分解结果为A=PLU，其中P为置换矩阵、L为下三角矩阵或下阶梯矩阵且对角线为1，U是上三角矩阵或上阶梯矩阵。

接口定义

C Interface：

void sgetrf_(const int *m, const int *n, float *a, const int *lda, int *ipiv, int *info);

void dgetrf_(const int *m, const int *n, double *a, const int *lda, int *ipiv, int *info);

void cgetrf_(const int *m, const int *n, float _Complex *a, const int *lda, int *ipiv, int *info);

void zgetrf_(const int *m, const int *n, double _Complex *a, const int *lda, int *ipiv, int *info);

Fortran Interface：

SGETRF(m, n, a, lda, ipiv, info)

DGETRF(m, n, a, lda, ipiv, info)

CGETRF(m, n, a, lda, ipiv, info)

ZGETRF(m, n, a, lda, ipiv, info)

 版权所有 © 华为技术有限公司

参数

参数名	类型	描述	输入/输出
m	整数型	矩阵A的行数。	输入
n	整数型	矩阵A的列数。	输入
a	• 在sgetrf中为单精度浮点型数组。 • 在dgetrf中为双精度浮点型数组。 • 在cgetrf中为单精度复数型数组。 • 在zgetrf中为双精度复数型数组。	• 调用前保存矩阵A。 • 调用后保存分解结果L和U，不保存L的对角线元素（均为1）。	输入/输出
lda	整数型	A的leading dimension大小，要求lda ≥ max(1, n)。	输入
ipiv	整数型数组	?getrf得到的主元索引，长度为min(m,n)：1 ≤ ipiv ≤ min(m,n)，表示分解中矩阵第i行与第ipiv[i-1]行交换。	输出
info	整数型	执行结果： • 等于0：成功。 • 小于0：第-info个参数值不合法。 • 大于0：矩阵U对角线上第info个元素为0，矩阵分解完成但U是奇异的，会导致求解线性方程组时出现除以零的错误。	输出

依赖

#include "klapack.h"

示例

C Interface：

```
int m = 4;
int n = 4;
int lda = 4;
int ipiv[4];
int info = 0;
/*
 * A (stored in column-major):
 *  1.80  2.88   2.05  -0.89
 *  5.25  -2.95  -0.95  -3.80
 *  1.58  -2.69  -2.90  -1.04
```


版权所有 © 华为技术有限公司

```
 * -1.11  -0.66  -0.59   0.80
 */
double a[] = {1.80,  5.25,  1.58, -1.11,
              2.88, -2.95, -2.69, -0.66,
              2.05, -0.95, -2.90, -0.59,
              -0.89, -3.80, -1.04,  0.80};
dgetrf_(&m, &n, a, &lda, ipiv, &info);
/*
 * Output:
 *  5.2500  -2.9500  -0.9500  -3.8000
 *  0.3429   3.8914   2.3757   0.4129
 *  0.3010  -0.4631  -1.5139   0.2948
 * -0.2114  -0.3299   0.0047   0.1314
 */
```

Fortran Interface：

```
      PARAMETER (m = 4)
      PARAMETER (n = 4)
      PARAMETER (lda = 4)
      INTEGER :: ipiv(4)
      INTEGER :: info = 0
*     A (stored in column-major):
*      1.80  2.88    2.05  -0.89
*      5.25  -2.95  -0.95  -3.80
*      1.58  -2.69  -2.90  -1.04
*     -1.11  -0.66  -0.59   0.80
      REAL(8) :: a(m, n)
      DATA a / 1.80,  5.25,  1.58, -1.11,
     $         2.88, -2.95, -2.69, -0.66,
     $         2.05, -0.95, -2.90, -0.59,
     $         -0.89, -3.80, -1.04,  0.80 /
      EXTERNAL DGETRF
      CALL DGETRF(m, n, a, lda, ipiv, info);
*     Output:
*      5.2500  -2.9500  -0.9500  -3.8000
*      0.3429   3.8914   2.3757   0.4129
*      0.3010  -0.4631  -1.5139   0.2948
*     -0.2114  -0.3299   0.0047   0.1314
```

6.5.1.3.2.2 ?geqrf

计算矩阵的QR分解。即A = Q * R。

接口定义

C Interface：

void dgeqrf_(const int *m, const int *n, double *a, const int *lda, double *tau, double *work, const int *lwork, int *info);

void sgeqrf_(const int *m, const int *n, float *a, const int *lda, float *tau, float *work, const int *lwork, int *info);

void cgeqrf_(const int *m, const int *n, float _Complex *a, const int *lda, float _Complex *tau, float _Complex *work, const int *lwork, int *info);

void zgeqrf_(const int *m, const int *n, double _Complex *a, const int *lda, double _Complex *tau, double _Complex *work, const int *lwork, int *info);

Fortran Interface：

DGEQRF(m, n, a, lda, tau, work, lwork, info);

SGEQRF(m, n, a, lda, tau, work, lwork, info);

 版权所有 © 华为技术有限公司

CGEQRF(m, n, a, lda, tau, work, lwork, info);

ZGEQRF(m, n, a, lda, tau, work, lwork, info);

参数

参数名	类型	描述	输入/输出
m	整数型	矩阵A的行数。	输入
n	整数型	矩阵A的列数。	输入
a	• 在sgeqrf中为单精度浮点型数组。 • 在dgeqrf中为双精度浮点型数组。 • 在cgeqrf中为单精度复数型数组。 • 在zgeqrf中为双精度复数型数组。	• 调用前保存待分解的矩阵A。 • 调用后对角线以及对角线以上的部分保存大小为min(m,n)*n的矩阵R（当m≥n时R为上三角矩阵），对角线以下元素和tau共同表示正交矩阵Q（参见说明）。	输入/输出
lda	整数型	A的leading dimension大小，要求lda ≥ max(1, m)。	输出
tau	• 在sgeqrf中为单精度浮点型数组。 • 在dgeqrf中为双精度浮点型数组。 • 在cgeqrf中为单精度复数型数组。 • 在zgeqrf中为双精度复数型数组。	初等反射的系数，长度为min(m,n)（参见说明）。	输出
work	• 在sgeqrf中为单精度浮点型数组。 • 在dgeqrf中为双精度浮点型数组。 • 在cgeqrf中为单精度复数型数组。 • 在zgeqrf中为双精度复数型数组。	临时存储空间，使用lwork=-1调用后work[0]为最优的lwork值。	输出
lwork	整数型	work数组的长度。 lwork=-1时查询最优work大小，结果保存在work[0]中，否则要求lwork ≥ n。	输入

版权所有 © 华为技术有限公司

参数名	类型	描述	输入/输出
info	整数型	执行结果： ● 等于0：成功。 ● 小于0：第-info个参数值不合法。	输出

说明

分解结果矩阵Q通过一系列初等反射乘积表示：Q=H(1)*H(2)*...*H(k), k=min(m,n)。H(i)=I-tau*v*v'。tau为标量，v为向量且前i-1个元素为0，第i个元素为1，剩余元素保存在a的第i列中（a的下三角部分）。

依赖

#include "klapack.h"

示例

C Interface：

```
int m = 6;
int n = 4;
int lda = 6;
int info = 0;
double tau[4];
double *work = NULL;
double qwork;
int lwork = -1;
/*
 * A (6x4, stored in column-major):
 *  2.229  1.273  0.087  0.035
 *  8.667  4.317  4.091  3.609
 *  0.205  7.810  2.553  6.507
 *  2.758  2.911  8.791  5.051
 *  8.103  1.396  1.317  4.738
 *  8.859  3.161  0.808  5.972
 */
double a[] = {2.229, 8.667, 0.205, 2.758, 8.103, 8.859,
              1.273, 4.317, 7.810, 2.911, 1.396, 3.161,
              0.087, 4.091, 2.553, 8.791, 1.317, 0.808,
              0.035, 3.609, 6.507, 5.051, 4.738, 5.972};
/* Query optimal work size */
dgeqrf_(&m, &n, a, &lda, tau, &qwork, &lwork, &info);
if (info != 0) {
    return ERROR;
}
lwork = (int)qwork;
work = (double *)malloc(sizeof(double) * lwork);
/* Calculate QR */
dgeqrf_(&m, &n, a, &lda, tau, work, &lwork, &info);
free(work);
/*
 * Output:
 * tau
 *   1.1464 1.0946 1.3670 1.7415
 * A output (stored in column-major)
 *  -15.2274 -5.8577  -5.1387  -9.0573
 *   0.4965  -8.2070  -4.5806  -6.5291
```


版权所有 © 华为技术有限公司

```
*   0.0117   0.8600   7.4527   1.3953
*   0.1580   0.1986  -0.6634  -3.5188
*   0.4642  -0.2130  -0.0183   0.2805
*   0.5075  -0.0510   0.1506   0.2642
*/
```

Fortran Interface：

```
      PARAMETER (m = 6)
      PARAMETER (n = 4)
      PARAMETER (lda = 6)
      INTEGER :: info = 0
      REAL(8) :: tau(4)
      REAL(8) :: qwork(1)
      INTEGER :: lwork = -1
      REAL(8), ALLOCATABLE :: work(:)
*
*     A (6x4, stored in column-major):
*       2.229  1.273  0.087  0.035
*       8.667  4.317  4.091  3.609
*       0.205  7.810  2.553  6.507
*       2.758  2.911  8.791  5.051
*       8.103  1.396  1.317  4.738
*       8.859  3.161  0.808  5.972
*
      REAL(8) :: a(m, n)
      DATA a / 2.229, 8.667, 0.205, 2.758, 8.103, 8.859,
     $         1.273, 4.317, 7.810, 2.911, 1.396, 3.161,
     $         0.087, 4.091, 2.553, 8.791, 1.317, 0.808,
     $         0.035, 3.609, 6.507, 5.051, 4.738, 5.972 /

      EXTERNAL DGEQRF
*     Query optimal work size
      CALL DGEQRF(m, n, a, lda, tau, qwork, lwork, info)
      IF (info.NE.0) THEN
         CALL EXIT(1)
      END IF
      lwork = INT(qwork(1))
      ALLOCATE(work(lwork))
*     Calculate QR
      CALL DGEQRF(m, n, a, lda, tau, work, lwork, info)
      DEALLOCATE(work)

*     Output:
*     tau
*       1.1464 1.0946 1.3670 1.7415
*     A output (stored in column-major)
*      -15.2274 -5.8577  -5.1387  -9.0573
*       0.4965  -8.2070  -4.5806  -6.5291
*       0.0117   0.8600   7.4527   1.3953
*       0.1580   0.1986  -0.6634  -3.5188
*       0.4642  -0.2130  -0.0183   0.2805
*       0.5075  -0.0510   0.1506   0.2642
```

6.5.1.3.2.3 ?gerqf

计算矩阵的RQ分解，即A = R * Q。

接口定义

C Interface：

void sgerqf_(const int *M, const int *N, float *A, const int *LDA, float *TAU, float *WORK, const int *LWORK, int *INFO);

void dgerqf_(const int *M, const int *N, double *A, const int *LDA, double *TAU, double *WORK, const int *LWORK, int *INFO);

 版权所有 © 华为技术有限公司

void cgerqf_(const int *M, const int *N, float _Complex *A, const int *LDA, float _Complex *TAU, float _Complex *WORK, const int *LWORK, int *INFO);

void zgerqf_(const int *M, const int *N, double _Complex *A, const int *LDA, double _Complex *TAU, double _Complex *WORK, const int *LWORK, int *INFO);

Fortran Interface：

DGERQF(m, n, a, lda, tau, work, lwork, info);

SGERQF(m, n, a, lda, tau, work, lwork, info);

CGERQF(m, n, a, lda, tau, work, lwork, info);

ZGERQF(m, n, a, lda, tau, work, lwork, info);

参数

参数名	类型	描述	输入/输出
m	整数型	矩阵A的行数。	输入
n	整数型	矩阵A的列数。	输入
a	• 在sgerqf中为单精度浮点型数组。 • 在dgerqf中为双精度浮点型数组。 • 在cgerqf中为单精度复数型数组。 • 在zgerqf中为双精度复数型数组。	• 调用前保存待分解的矩阵A。 • 调用后对角线以及对角线以上的部分保存大小为min(m,n)*n的矩阵R（当m≥n时R为上三角矩阵），对角线以下元素和tau共同表示正交矩阵Q（参见说明）。	输入/输出
lda	整数型	A的leading dimension大小，要求lda≥max(1, m)。	输出
tau	• 在sgerqf中为单精度浮点型数组。 • 在dgerqf中为双精度浮点型数组。 • 在cgerqf中为单精度复数型数组。 • 在zgerqf中为双精度复数型数组。	初等反射的系数，长度为min(m,n)（参见说明）。	输出

版权所有 © 华为技术有限公司

参数名	类型	描述	输入/输出
work	• 在sgerqf中为单精度浮点型数组。 • 在dgerqf中为双精度浮点型数组。 • 在cgerqf中为单精度复数型数组。 • 在zgerqf中为双精度复数型数组。	临时存储空间，使用lwork=-1调用后work[0]为最优的lwork值。	输出
lwork	整数型	work数组的长度。 lwork=-1时查询最优work大小，结果保存在work[0]中，否则要求lwork≥n。	输入
info	整数型	执行结果： • 等于0：成功。 • 小于0：第-info个参数值不合法。	输出

说明

分解结果矩阵Q通过一系列初等反射乘积表示：Q=H(1)*H(2)*...*H(k), k=min(m,n)。H(i)=I-tau*v*v'。tau为标量，v为向量且前i-1个元素为0，第i个元素为1，剩余元素保存在a的第i列中（a的下三角部分）。

依赖

#include "klapack.h"

示例

C Interface：

```
int m = 6;
int n = 4;
int lda = 6;
int info = 0;
double tau[4];
double *work = NULL;
double qwork;
int lwork = -1;
/*
 * A (6x4, stored in column-major):
 *  2.229  1.273  0.087  0.035
 *  8.667  4.317  4.091  3.609
 *  0.205  7.810  2.553  6.507
 *  2.758  2.911  8.791  5.051
 *  8.103  1.396  1.317  4.738
 *  8.859  3.161  0.808  5.972
 */
double a[] = {2.229, 8.667, 0.205, 2.758, 8.103, 8.859,
              1.273, 4.317, 7.810, 2.911, 1.396, 3.161,
```

 版权所有 © 华为技术有限公司

```
                0.087, 4.091, 2.553, 8.791, 1.317, 0.808,
                0.035, 3.609, 6.507, 5.051, 4.738, 5.972};
/* Query optimal work size */
dgerqf_(&m, &n, a, &lda, tau, &qwork, &lwork, &info);
if (info != 0) {
    return ERROR;
}
lwork = (int)qwork;
work = (double *)malloc(sizeof(double) * lwork);
/* Calculate RQ */
dgerqf_(&m, &n, a, &lda, tau, work, &lwork, &info);
free(work);
/*
 * Output:
 * tau
 *   0.000000        1.683018        1.409717        1.534598
 * A output (stored in column-major)
 *  1.359608        2.967627        -1.011896       -0.433983
 *  0.345857        0.516770        0.307017        -2.757397
 *  -5.500889       -8.779283       -0.546907       0.184390
 *  0.141517        -0.775170       6.494784        -1.230625
 *  -1.582355       0.047133        -2.152892       -10.320058
 *  -6.035814       -6.347009       -9.449164       -11.171009
 */
```

Fortran Interface：

```
      PARAMETER (m = 6)
      PARAMETER (n = 4)
      PARAMETER (lda = 6)
      INTEGER :: info = 0
      REAL(8) :: tau(4)
      REAL(8) :: qwork(1)
      INTEGER :: lwork = -1
      REAL(8), ALLOCATABLE :: work(:)
*
*     A (6x4, stored in column-major):
*       2.229  1.273  0.087  0.035
*       8.667  4.317  4.091  3.609
*       0.205  7.810  2.553  6.507
*       2.758  2.911  8.791  5.051
*       8.103  1.396  1.317  4.738
*       8.859  3.161  0.808  5.972
*
      REAL(8) :: a(m, n)
      DATA a / 2.229, 8.667, 0.205, 2.758, 8.103, 8.859,
     $         1.273, 4.317, 7.810, 2.911, 1.396, 3.161,
     $         0.087, 4.091, 2.553, 8.791, 1.317, 0.808,
     $         0.035, 3.609, 6.507, 5.051, 4.738, 5.972 /

      EXTERNAL DGERQF
*     Query optimal work size
      CALL DGERQF(m, n, a, lda, tau, qwork, lwork, info)
      IF (info.NE.0) THEN
         CALL EXIT(1)
      END IF
      lwork = INT(qwork(1))
      ALLOCATE(work(lwork))
*     Calculate RQ
      CALL DGERQF(m, n, a, lda, tau, work, lwork, info)
      DEALLOCATE(work)

*     Output:
*     tau
*       0.000000        1.683018        1.409717        1.534598
*     A output (stored in column-major)
*      1.359608        2.967627        -1.011896       -0.433983
*      0.345857        0.516770        0.307017        -2.757397
*      -5.500889       -8.779283       -0.546907       0.184390
*      0.141517        -0.775170       6.494784        -1.230625
```


版权所有 © 华为技术有限公司

```
*      -1.582355     0.047133     -2.152892     -10.320058
*      -6.035814     -6.347009     -9.449164     -11.171009
```

6.5.1.3.2.4 ?geqlf

计算矩阵的QL分解，即A = Q * L。

接口定义

C Interface：

void sgeqlf_(const int *M, const int *N, float *A, const int *LDA, float *TAU, float *WORK, const int *LWORK, int *INFO);

void dgeqlf_(const int *M, const int *N, double *A, const int *LDA, double *TAU, double *WORK, const int *LWORK, int *INFO);

void cgeqlf_(const int *M, const int *N, float _Complex *A, const int *LDA, float _Complex *TAU, float _Complex *WORK, const int *LWORK, int *INFO);

void zgeqlf_(const int *M, const int *N, double _Complex *A, const int *LDA, double _Complex *TAU, double _Complex *WORK, const int *LWORK, int *INFO);

Fortran Interface：

DGEQLF(m, n, a, lda, tau, work, lwork, info);

SGEQLF(m, n, a, lda, tau, work, lwork, info);

CGEQLF(m, n, a, lda, tau, work, lwork, info);

ZGEQLF(m, n, a, lda, tau, work, lwork, info);

参数

参数名	类型	描述	输入/输出
m	整数型	矩阵A的行数。	输入
n	整数型	矩阵A的列数。	输入
a	• 在sgeqlf中为单精度浮点型数组。 • 在dgeqlf中为双精度浮点型数组。 • 在cgeqlf中为单精度复数型数组。 • 在zgeqlf中为双精度复数型数组。	• 调用前保存待分解的矩阵A。 • 调用后对角线以及对角线以上的部分保存大小为min(m,n)*n的矩阵R（当m≥n时L为下三角矩阵），对角线以下元素和tau共同表示正交矩阵Q（参见说明）。	输入/输出
lda	整数型	A的leading dimension大小，要求lda≥max(1, m)。	输出

版权所有 © 华为技术有限公司

参数名	类型	描述	输入/输出
tau	• 在sgeqlf中为单精度浮点型数组。 • 在dgeqlf中为双精度浮点型数组。 • 在cgeqlf中为单精度复数型数组。 • 在zgeqlf中为双精度复数型数组。	初等反射的系数，长度为min(m,n)（参见说明）。	输出
work	• 在sgeqlf中为单精度浮点型数组。 • 在dgeqlf中为双精度浮点型数组。 • 在cgeqlf中为单精度复数型数组。 • 在zgeqlf中为双精度复数型数组。	临时存储空间，使用lwork=-1调用后work[0]为最优的lwork值。	输出
lwork	整数型	work数组的长度。 lwork=-1时查询最优work大小，结果保存在work[0]中，否则要求lwork≥n。	输入
info	整数型	执行结果： • 等于0：成功。 • 小于0：第-info个参数值不合法。	输出

说明

分解结果矩阵Q通过一系列初等反射乘积表示：Q=H(1)*H(2)*...*H(k), k=min(m,n)。H(i)=I-tau*v*v'。tau为标量，v为向量且前i-1个元素为0，第i个元素为1，剩余元素保存在a的第i列中（a的下三角部分）。

依赖

#include "klapack.h"

示例

C Interface：

```
int m = 6;
int n = 4;
int lda = 6;
int info = 0;
double tau[4];
double *work = NULL;
```

 版权所有 © 华为技术有限公司

```
double qwork;
int lwork = -1;
/*
 * A (6x4, stored in column-major):
 *  2.229  1.273  0.087  0.035
 *  8.667  4.317  4.091  3.609
 *  0.205  7.810  2.553  6.507
 *  2.758  2.911  8.791  5.051
 *  8.103  1.396  1.317  4.738
 *  8.859  3.161  0.808  5.972
 */
double a[] = {2.229, 8.667, 0.205, 2.758, 8.103, 8.859,
              1.273, 4.317, 7.810, 2.911, 1.396, 3.161,
              0.087, 4.091, 2.553, 8.791, 1.317, 0.808,
              0.035, 3.609, 6.507, 5.051, 4.738, 5.972};
/* Query optimal work size */
dgeqlf_(&m, &n, a, &lda, tau, &qwork, &lwork, &info);
if (info != 0) {
    return ERROR;
}
lwork = (int)qwork;
work = (double *)malloc(sizeof(double) * lwork);
/* Calculate QL */
dgeqlf_(&m, &n, a, &lda, tau, work, &lwork, &info);
free(work);
/*
 * Output:
 * tau
 *   1.628275        1.498542        1.123799        1.506551
 * A output (stored in column-major)
 *  -0.199388       -0.434210       8.949191        -3.693237
 *  -1.136543       -11.698489      -0.184444       -0.181795
 *  -0.517264       4.452845        0.152113        -9.045264
 *  -0.009043       -0.309964       0.056353        -0.824829
 *  6.975979        -7.366574       0.001971        0.203192
 *  0.366353        0.284379        0.266756        -11.789537
 */
```

Fortran Interface：

```
      PARAMETER (m = 6)
      PARAMETER (n = 4)
      PARAMETER (lda = 6)
      INTEGER :: info = 0
      REAL(8) :: tau(4)
      REAL(8) :: qwork(1)
      INTEGER :: lwork = -1
      REAL(8), ALLOCATABLE :: work(:)
*
*     A (6x4, stored in column-major):
*       2.229  1.273  0.087  0.035
*       8.667  4.317  4.091  3.609
*       0.205  7.810  2.553  6.507
*       2.758  2.911  8.791  5.051
*       8.103  1.396  1.317  4.738
*       8.859  3.161  0.808  5.972
*
      REAL(8) :: a(m, n)
      DATA a / 2.229, 8.667, 0.205, 2.758, 8.103, 8.859,
     $         1.273, 4.317, 7.810, 2.911, 1.396, 3.161,
     $         0.087, 4.091, 2.553, 8.791, 1.317, 0.808,
     $         0.035, 3.609, 6.507, 5.051, 4.738, 5.972 /

      EXTERNAL DGEQLF
*     Query optimal work size
      CALL DGEQLF(m, n, a, lda, tau, qwork, lwork, info)
      IF (info.NE.0) THEN
         CALL EXIT(1)
      END IF
      lwork = INT(qwork(1))
```

 版权所有 © 华为技术有限公司

```
      ALLOCATE(work(lwork))
*     Calculate QL
      CALL DGEQLF(m, n, a, lda, tau, work, lwork, info)
      DEALLOCATE(work)

*     Output:
*     tau
*       1.628275        1.498542        1.123799        1.506551
*     A output (stored in column-major)
*      -0.199388       -0.434210        8.949191       -3.693237
*      -1.136543       -11.698489       -0.184444       -0.181795
*      -0.517264       4.452845        0.152113        -9.045264
*      -0.009043       -0.309964       0.056353        -0.824829
*      6.975979        -7.366574       0.001971        0.203192
*      0.366353        0.284379        0.266756        -11.789537
```

6.5.1.3.2.5 ?gelqf

计算矩阵的LQ分解，即A = L * Q。

接口定义

C Interface：

void sgelqf_(const int *M, const int *N, float *A, const int *LDA, float *TAU, float *WORK, const int *LWORK, int *INFO);

void dgelqf_(const int *M, const int *N, double *A, const int *LDA, double *TAU, double *WORK, const int *LWORK, int *INFO);

void cgelqf_(const int *M, const int *N, float _Complex *A, const int *LDA, float _Complex *TAU, float _Complex *WORK, const int *LWORK, int *INFO);

void zgelqf_(const int *M, const int *N, double _Complex *A, const int *LDA, double _Complex *TAU, double _Complex *WORK, const int *LWORK, int *INFO);

Fortran Interface：

DGELQF(m, n, a, lda, tau, work, lwork, info);

SGELQF(m, n, a, lda, tau, work, lwork, info);

CGELQF(m, n, a, lda, tau, work, lwork, info);

ZGELQF(m, n, a, lda, tau, work, lwork, info);

参数

参数名	类型	描述	输入/输出
m	整数型	矩阵A的行数。	输入
n	整数型	矩阵A的列数。	输入

版权所有 © 华为技术有限公司

参数名	类型	描述	输入/输出
a	• 在sgelqf中为单精度浮点型数组。 • 在dgelqf中为双精度浮点型数组。 • 在cgelqf中为单精度复数型数组。 • 在zgelqf中为双精度复数型数组。	• 调用前保存待分解的矩阵A。 • 调用后对角线以及对角线以上的部分保存大小为min(m,n)*n的矩阵R（当m≥n时L为下三角矩阵），对角线以下元素和tau共同表示正交矩阵Q（参见说明）。	输入/输出
lda	整数型	A的leading dimension大小，要求lda≥max(1, m)。	输出
tau	• 在sgelqf中为单精度浮点型数组。 • 在dgelqf中为双精度浮点型数组。 • 在cgelqf中为单精度复数型数组。 • 在zgelqf中为双精度复数型数组。	初等反射的系数，长度为min(m,n)（参见说明）。	输出
work	• 在sgelqf中为单精度浮点型数组。 • 在dgelqf中为双精度浮点型数组。 • 在cgelqf中为单精度复数型数组。 • 在zgelqf中为双精度复数型数组。	临时存储空间，使用lwork=-1调用后work[0]为最优的lwork值。	输出
lwork	整数型	work数组的长度。 lwork=-1时查询最优work大小，结果保存在work[0]中，否则要求lwork≥n。	输入
info	整数型	执行结果： • 等于0：成功。 • 小于0：第-info个参数值不合法。	输出

说明

分解结果矩阵Q通过一系列初等反射乘积表示：Q=H(1)*H(2)*...*H(k), k=min(m,n)。H(i)=I-tau*v*v'。tau为标量，v为向量且前i-1个元素为0，第i个元素为1，剩余元素保存在a的第i列中（a的下三角部分）。

版权所有 © 华为技术有限公司

依赖

#include "klapack.h"

示例

C Interface：

```
int m = 6;
int n = 4;
int lda = 6;
int info = 0;
double tau[4];
double *work = NULL;
double qwork;
int lwork = -1;
/*
 * A (6x4, stored in column-major):
 *  2.229  1.273  0.087  0.035
 *  8.667  4.317  4.091  3.609
 *  0.205  7.810  2.553  6.507
 *  2.758  2.911  8.791  5.051
 *  8.103  1.396  1.317  4.738
 *  8.859  3.161  0.808  5.972
 */
double a[] = {2.229, 8.667, 0.205, 2.758, 8.103, 8.859,
              1.273, 4.317, 7.810, 2.911, 1.396, 3.161,
              0.087, 4.091, 2.553, 8.791, 1.317, 0.808,
              0.035, 3.609, 6.507, 5.051, 4.738, 5.972};
/* Query optimal work size */
dgelqf_(&m, &n, a, &lda, tau, &qwork, &lwork, &info);
if (info != 0) {
   return ERROR;
}
lwork = (int)qwork;
work = (double *)malloc(sizeof(double) * lwork);
/* Calculate LQ */
dgelqf_(&m, &n, a, &lda, tau, work, &lwork, &info);
free(work);
/*
 * Output:
 * tau
 *   1.867784        1.115696        1.413422        0.000000
 * A output (stored in column-major)
 *  -2.568611       -9.848324       -4.223656       -4.202616
 *  -7.832678       -9.363028        0.265340        5.150248
 *  5.402586        9.567440        4.194706        4.480431
 *  0.018134        -0.653528       -7.929047       -1.156667
 *  1.133611        -0.463361        0.007295       -0.604570
 *  0.644209        -2.887734        3.399866        4.103192
 */
```

Fortran Interface：

```
      PARAMETER (m = 6)
      PARAMETER (n = 4)
      PARAMETER (lda = 6)
      INTEGER :: info = 0
      REAL(8) :: tau(4)
      REAL(8) :: qwork(1)
      INTEGER :: lwork = -1
      REAL(8), ALLOCATABLE :: work(:)
*
*     A (6x4, stored in column-major):
*       2.229  1.273  0.087  0.035
*       8.667  4.317  4.091  3.609
*       0.205  7.810  2.553  6.507
*       2.758  2.911  8.791  5.051
```


版权所有 © 华为技术有限公司

```
*       8.103  1.396  1.317  4.738
*       8.859  3.161  0.808  5.972
*
      REAL(8) :: a(m, n)
      DATA a / 2.229, 8.667, 0.205, 2.758, 8.103, 8.859,
     $         1.273, 4.317, 7.810, 2.911, 1.396, 3.161,
     $         0.087, 4.091, 2.553, 8.791, 1.317, 0.808,
     $         0.035, 3.609, 6.507, 5.051, 4.738, 5.972 /

      EXTERNAL DGELQF
*     Query optimal work size
      CALL DGELQF(m, n, a, lda, tau, qwork, lwork, info)
      IF (info.NE.0) THEN
         CALL EXIT(1)
      END IF
      lwork = INT(qwork(1))
      ALLOCATE(work(lwork))
*     Calculate LQ
      CALL DGELQF(m, n, a, lda, tau, work, lwork, info)
      DEALLOCATE(work)

*     Output:
*     tau
*       1.867784        1.115696        1.413422        0.000000
*     A output (stored in column-major)
*      -2.568611       -9.848324       -4.223656       -4.202616
*      -7.832678       -9.363028        0.265340        5.150248
*       5.402586        9.567440        4.194706        4.480431
*       0.018134       -0.653528       -7.929047       -1.156667
*       1.133611       -0.463361        0.007295       -0.604570
*       0.644209       -2.887734        3.399866        4.103192
```

6.5.1.3.2.6 ?potrf

计算对称正定矩阵或者Hermite正定矩阵的Cholesky分解。

即$A = LL^T$或者 $A = U^TU$，U上三角矩阵，L下三角矩阵。

接口定义

C Interface：

void spotrf_(const char *uplo, const int *n, float *a, const int *lda, int *info);

void dpotrf_(const char *uplo, const int *n, double *a, const int *lda, int *info);

void cpotrf_(const char *uplo, const int *n, float _Complex *a, const int *lda, int *info);

void zpotrf_(const char *uplo, const int *n, double _Complex *a, const int *lda, int *info);

Fortran Interface：

SPOTRF(uplo, n, a, lda, info);

DPOTRF(uplo, n, a, lda, info);

CPOTRF(uplo, n, a, lda, info);

ZPOTRF(uplo, n, a, lda, info);

 版权所有 © 华为技术有限公司

参数

参数名	类型	描述	输入/输出
uplo	字符型	'U'：保存A的上三角矩阵。 'L'：保存A的下三角矩阵。	输入
n	整数型	对称矩阵A的行数或列数。	输入
a	• 在spotrf中为单精度浮点型数组。 • 在dpotrf中为双精度浮点型数组。 • 在cpotrf中为单精度复数型数组。 • 在zpotrf中为双精度复数型数组。	• 调用前保存待分解的矩阵A。 • 当uplo = 'U'，调用后对角线以及对角线以上的部分保存上三角矩阵U。 • 当uplo = 'L'，调用后对角线以及对角线以下的部分保存下三角矩阵L。	输入/输出
lda	整数型	A的leading dimension大小，要求lda≥max(1, n)。	输入
info	整数型	执行结果： • 等于0：退出成功。 • 小于0：第-info个参数值不合法。 • 大于0：A中info大小的主子式非正定，无法完成分解。	输出

依赖

#include "klapack.h"

示例

C Interface：

```
char uplo = 'L'
int n = 4;
int lda = 4;
int info = 0;
/*
* A (stored in column-major):
*  231.8009    33.9545    9.4153   11.1156
*   33.9454   162.2902   6.6684   53.2023
*   9.4153   6.6684  100.3220  71.5384
*   11.1156   53.2023    71.5384  123.4425
*/
double a[] = {231.8009,   33.9545,   9.4143,   11.1156,
              33.9495,  162.2902,   6.6684,   53.2033,
              9.4143,    6.6684,  100.322,   71.5384,
              11.1156,   53.2033,  71.5384,  106.5638 };
 dpotrf_(&uplo, &n, a, &lda, &info);
/*
* Output:
```


版权所有 © 华为技术有限公司

```
*  15.2250   33.9495   9.4143   11.1156
*   2.2302   12.5426   6.6684   53.2033
*   0.6183    0.4217   9.9881   71.5384
*   0.7301     4.112   6.9436     6.396
*/
```

Fortran Interface：

```
CHARACTER :: uplo = "L"
PARAMETER (n = 4)
PARAMETER (lda = 4)
INTEGER :: info = 0
REAL(8) :: a(n,n)
* A (stored in column-major):
*  231.8009    33.9545    9.4153   11.1156
*   33.9454   162.2902   6.6684   53.2023
*   9.4153   6.6684  100.3220  71.5384
*   11.1156   53.2023    71.5384  123.4425

DATA a / 231.8009,  33.9545,  9.4143, 11.1156,
 $          33.9495,  162.2902, 6.6684, 53.2033,
 $          9.4143,    6.6684,  100.322, 71.5384,
 $          11.1156,   53.2033, 71.5384, 106.5638 /
 EXTERNAL DPOTRF
 CALL DPOTRF (uplo, n, a, lda, info);
* Output:
*  15.2250   33.9495   9.4143   11.1156
*   2.2302   12.5426   6.6684   53.2033
*   0.6183    0.4217   9.9881   71.5384
*   0.7301     4.112   6.9436     6.396
```

6.5.1.3.2.7 ?pttrf

计算实（共轭）对称正定三对角矩阵A的LDL*或U*DU分解，其中D为对角块矩阵，L为下三角矩阵，U为上三角矩阵。

接口定义

C Interface：

void spttrf_(const int *N, float *D, float *E, int *INFO);

void dpttrf_(const int *N, double *D, double *E, int *INFO);

void cpttrf_(const int *N, float *D, float _Complex *E, int *INFO);

void zpttrf_(const int *N, double *D, double _Complex *E, int *INFO);

Fortran Interface：

SPTTRF(N, D, E, INFO);

DPTTRF(N, D, E, INFO);

CPTTRF(N, D, E, INFO);

ZPTTRF(N, D, E, INFO);

 版权所有 © 华为技术有限公司

参数

参数名	类型	描述	输入/输出
N	整数型	矩阵A的维数，N≥0。	输入
D	• 在spttrf/cpttrf中为单精度浮点型数组。 • 在dpttrf/zpttrf中为双精度浮点型数组。	输入时，为三对角矩阵A的对角元素，维度为N。 输出时，为分解后的对角元素D。	输入、输出
E	• 在spttrf中为单精度浮点型数组。 • 在dpttrf中为双精度浮点型数组。 • 在cpttrf中为单精度复数型数组。 • 在zpttrf中为双精度复数型数组。	输入时，为三对角矩阵A的次对角元素，维度为N-1。 输出时，为分解后矩阵A的次对角元素。	输入、输出
INFO	整数型	• 等于0：成功退出。 • 小于0：表示第-INFO个参数不合法。 • 大于0：表示第INFO阶主子式不正定。	输出

依赖

#include "klapack.h"

示例

C Interface：

```
const int n = 4;
double d[] = {103.608696, 103.739130, 103.347826, 100.652174};
double e[] = {0.352941, 0.000000, 0.823529};
int info = 0;
dpttrf_(&n, d, e, &info);
/*
* Output:
*  d: 103.608696      103.737928      103.347826      100.645612
*  e:  0.003406        0.000000        0.007969
*/
```

Fortran Interface：

```
PARAMETER (n = 4)
INTEGER :: info = 0
REAL(8) :: d(n)
REAL(8) :: e(n-1)

DATA d / 103.608696, 103.739130, 103.347826, 100.652174 /
DATA e / 0.352941,0.000000,0.823529/
```

 版权所有 © 华为技术有限公司

```
 EXTERNAL DPTTRF
 CALL DPTTRF (n, d, e, info)
* Output:
*  d: 103.608696      103.737928      103.347826      100.645612
*  e:  0.003406        0.000000        0.007969
```

6.5.1.3.2.8 ?gttrf

计算一般三对角矩阵A的LU分解。

接口定义

C Interface：

void sgttrf(const int *N, float *DL, float *D, float *DU, float *DU2, int *IPIV, int *INFO);

void dgttrf(const int *N, double *DL, double *D, double *DU, double *DU2, int *IPIV, int *INFO);

void cgttrf(const int *N, float _Complex *DL, float _Complex *D, float _Complex *DU, float _Complex *DU2, int *IPIV, int *INFO);

void zgttrf(const int *N, double _Complex *DL, double _Complex *D, double _Complex *DU, double _Complex *DU2, int *IPIV, int *INFO);

Fortran Interface：

SGTTRF(N,DL,D,DU,DU2,IPIV,INFO);

DGTTRF(N,DL,D,DU,DU2,IPIV,INFO);

CGTTRF(N,DL,D,DU,DU2,IPIV,INFO);

ZGTTRF(N,DL,D,DU,DU2,IPIV,INFO);

参数

参数名	类型	描述	输入/输出
N	整数型	矩阵A的维数，N≥0。	输入
DL	• 在sgttrf中为单精度浮点型数组。 • 在dgttrf中为双精度浮点型数组。 • 在cgttrf中为单精度复数型数组。 • 在zgttrf中为双精度复数型数组。	输入时，为三对角矩阵A的次对角元素，维度为N-1。 输出时，为L矩阵。	输入、输出

版权所有 © 华为技术有限公司

参数名	类型	描述	输入/输出
D	• 在sgttrf中为单精度浮点型数组。 • 在dgttrf中为双精度浮点型数组。 • 在cgttrf中为单精度复数型数组。 • 在zgttrf中为双精度复数型数组。	输入时，为三对角矩阵A的对角元素，维度为N。 输出时，为U的对角元素。	输入、输出
DU	• 在sgttrf中为单精度浮点型数组。 • 在dgttrf中为双精度浮点型数组。 • 在cgttrf中为单精度复数型数组。 • 在zgttrf中为双精度复数型数组。	输入时，为三对角矩阵A的超对角元素，维度为N-1。 输出时，为U的第一超对角元素。	输入、输出
DU2	• 在sgttrf中为单精度浮点型数组。 • 在dgttrf中为双精度浮点型数组。 • 在cgttrf中为单精度复数型数组。 • 在zgttrf中为双精度复数型数组。	输出时，为U的第二超对角元素。	输出
IPIV	整数型	置换数组。	输出
INFO	整数型	• 等于0：成功退出。 • 小于0：第-info个参数值不合法。 • 大于0：表示U(INFO, INFO)为零，分解失败。	输出

依赖

#include "klapack.h"

示例

C Interface：

```
const int n = 4;
double dl[] = {0.923077, 0.153846, 0.153846};
double d[] = {0.521739, 0.043478, 0.304348, 0.086957};
```

 版权所有 © 华为技术有限公司

```
double du[] = {0.924528, 0.150943, 0.566038};
double du2[] = {0.986301, 0.397260};
double ipiv[4];
int info = 0;
dgttrf_(&n, dl, d, du, du2, ipiv, &info);
if (info != 0) {
    printf("ERROR, info = %d\n", info);
}

/*
* Output:
*   d:    0.923077        0.899954        0.318932        -0.186088
*   dl:   0.565217        0.170949        0.482378
*   du:   0.043478        -0.085316       0.566038
*   du2:  0.150943        0.000000
*   ipiv: 2       2       3       4
*/
```

Fortran Interface：

```
PARAMETER (n = 4)
INTEGER :: info = 0
REAL(8) :: d(n)
REAL(8) :: dl(n-1)
REAL(8) :: du(n-1)
REAL(8) :: du2(n-2)
INTEGER :: ipiv(n)

DATA d / 0.521739, 0.043478, 0.304348, 0.086957 /
DATA dl / 0.923077, 0.153846, 0.153846 /
DATA du / 0.924528, 0.150943, 0.566038 /
DATA du2 / 0.986301, 0.397260 /
 EXTERNAL DGTTRF
 CALL DGTTRF(n, dl, d, du, du2, ipiv, info);
*
* Output:
*   d:    0.923077        0.899954        0.318932        -0.186088
*   dl:   0.565217        0.170949        0.482378
*   du:   0.043478        -0.085316       0.566038
*   du2:  0.150943        0.000000
*   ipiv: 2       2       3       4
```

6.5.1.3.2.9 ?sptrf

计算压缩对称矩阵的LDL*或U*DU分解。

接口定义

C Interface：

void ssptrf_(const char *uplo, const int *n, float *ap, int *ipiv, int *info);

void dsptrf_(const char *uplo, const int *n, double *ap, int *ipiv, int *info);

void csptrf_(const char *uplo, const int *n, float _Complex *ap, int *ipiv, int *info);

void zsptrf_(const char *uplo, const int *n, double _Complex *ap, int *ipiv, int *info);

Fortran Interface：

SSPTRF(UPLO, N, AP, IPIV, INFO);

DSPTRF(UPLO, N, AP, IPIV, INFO);

CSPTRF(UPLO, N, AP, IPIV, INFO);

ZSPTRF(UPLO, N, AP, IPIV, INFO);

 版权所有 © 华为技术有限公司

参数

参数名	类型	描述	输入/输出
UPLO	字符数	• 'U'：保存A的上三角矩阵。 • 'L'：保存A的下三角矩阵。	输入
N	整数型	矩阵A的维数，N≥0。	输入
AP	• 在ssptrf中为单精度浮点型数组。 • 在dsptrf中为双精度浮点型数组。 • 在csptrf中为单精度复数型数组。 • 在zsptrf中为双精度复数型数组。	• 输入时，为压缩对称矩阵A，大小为N*(N+1)/2。 • 输出时，块对角矩阵D，和用于获得因子U或L的乘法器，存储为上或下三角矩阵覆盖A。	输入、输出
IPIV	整数型	置换数组，大小为N，表示对角矩阵D的块结构和交换细节。 • 若IPIV(k) > 0，第k与IPIV(k)的行列交换，且D(k,k)是1*1的对角线块。 • 若UPLO='U'且IPIV(k)=IPIV(k-1) < 0，则第k-1与-IPIV(k)的行列交换，且D(k-1:k, k-1:k)是2*2的对角线块。 • 若UPLO='L'且IPIV(k)=IPIV(k+1) < 0，则第k+1与-IPIV(k)的行列交换，且D(k:k+1;k:k+1)是2*2的对角线块。	输出
INFO	整数型	• 等于0：成功退出。 • 小于0：第-info个参数值不合法。 • 大于0：表示U(INFO, INFO)为零，分解失败。	输出

依赖

#include "klapack.h"

示例

C Interface：

 版权所有 © 华为技术有限公司

```
const char uplo = 'U';
const int n = 4;
double ap[] = {0.521739, 0.043478, 0.304348, 0.130435, 0.304348, 0.043478, 0.652174, 0.086957, 0.521739,
0.086957};
double ipiv[4];
int info = 0;
dsptrf_(&uplo, &n, ap, ipiv, &info);
if (info != 0) {
    printf("ERROR, info = %d\n", info);
}

/*
* Output:
*   ap: -0.129534   1.024339   0.302185   -0.492537   -0.044777   -0.728261   1.250000   0.083333
0.250000  0.521739
*   ipiv: 1       2       1       1
*/
```

Fortran Interface：

```
CHARACTER::uplo = "U"
PARAMETER (n = 4)
INTEGER :: info = 0
REAL(8) :: ap(n*(n+1)/2)
INTEGER :: ipiv(n)

DATA ap / 0.521739, 0.043478, 0.304348, 0.130435, 0.304348, 0.043478, 0.652174, 0.086957, 0.521739,
0.086957 /

 EXTERNAL DSPTRF
 CALL DSPTRF(uplo, n, ap, ipiv, info)
*
* Output:
*   ap: -0.129534   1.024339   0.302185   -0.492537   -0.044777   -0.728261   1.250000   0.083333
0.250000  0.521739
*   ipiv: 1       2       1       1
```

6.5.1.3.2.10 ?hptrf

计算压缩Hermite矩阵的LDL*或U*DU分解。

接口定义

C Interface：

void chptrf_(const char *uplo, const int *n, float _Complex *ap, int *ipiv, int *info);

void zhptrf_(const char *uplo, const int *n, double _Complex *ap, int *ipiv, int *info);

Fortran Interface：

CHPTRF(UPLO, N, AP, IPIV, INFO);

ZHPTRF(UPLO, N, AP, IPIV, INFO);

参数

参数名	类型	描述	输入/输出
UPLO	字符型	• 'U'：保存A的上三角矩阵。 • 'L'：保存A的下三角矩阵。	输入

 版权所有 © 华为技术有限公司

参数名	类型	描述	输入/输出
N	整数型	矩阵A的维数，N≥0。	输入
AP	• 在chptrf中为单精度复数型数组。 • 在zhptrf中为双精度复数型数组。	• 输入时，为压缩对称矩阵A，大小为N*(N+1)/2。 • 输出时，块对角矩阵D，和用于获得因子U或L的乘法器，存储为上或下三角矩阵覆盖A。	输入、输出
IPIV	整数型	置换数组，大小为N，表示对角矩阵D的块结构和交换细节。 • 若IPIV(k) > 0，第k与IPIV(k)的行列交换，且D(k,k)是1*1的对角线块。 • 若UPLO='U'且IPIV(k)=IPIV(k-1) < 0，则第k-1与-IPIV(k)的行列交换，且D(k-1:k, k-1:k)是2*2的对角线块。 • 若UPLO='L'且IPIV(k)=IPIV(k+1) < 0，则第k+1与-IPIV(k)的行列交换，且D(k:k+1;k:k+1)是2*2的对角线块。	输出
INFO	整数型	• 等于0：成功退出。 • 小于0：第-info个参数值不合法。 • 大于0：表示U(INFO, INFO)为零，分解失败。	输出

依赖

#include "klapack.h"

示例

C Interface：

```
const char uplo = 'U';
const int n = 4;
double ap[] = {0.521739, 0.043478, 0.304348, 0.130435, 0.304348, 0.043478, 0.652174, 0.086957, 0.521739, 0.086957};
double ipiv[4];
int info = 0;
zhptrf_(&uplo, &n, ap, ipiv, &info);
if (info != 0) {
    printf("ERROR, info = %d\n", info);
```

 版权所有 © 华为技术有限公司

```
}

/*
* Output:
*   ap: -0.129534   1.024339   0.302185   -0.492537   -0.044777   -0.728261   1.250000   0.083333
0.250000  0.521739
*   ipiv: 1       2       1       1
*/
```

Fortran Interface：

```
CHARACTER::uplo = "U"
PARAMETER (n = 4)
INTEGER :: info = 0
REAL(8) :: ap(n*(n+1)/2)
INTEGER :: ipiv(n)

DATA ap / 0.521739, 0.043478, 0.304348, 0.130435, 0.304348, 0.043478, 0.652174, 0.086957, 0.521739,
0.086957 /

 EXTERNAL ZHPTRF
 CALL ZHPTRF(uplo, n, ap, ipiv, info);
*
* Output:
*   ap: -0.129534   1.024339   0.302185   -0.492537   -0.044777   -0.728261   1.250000   0.083333
0.250000  0.521739
*   ipiv: 1       2       1       1
```

6.5.1.3.2.11 ?pptrf

计算压缩存储的对称正定矩阵的LLT或UTU分解。

接口定义

C Interface：

void spptrf_(const char *uplo, const int *n, float *ap, int *info);

void dpptrf_(const char *uplo, const int *n, double *ap, int *info);

void cpptrf_(const char *uplo, const int *n, float _Complex *ap, int *info);

void zpptrf_(const char *uplo, const int *n, double _Complex *ap, int *info);

Fortran Interface：

SPPTRF(UPLO, N, AP, INFO);

DPPTRF(UPLO, N, AP, INFO);

CPPTRF(UPLO, N, AP, INFO);

ZPPTRF(UPLO, N, AP, INFO);

参数

参数名	类型	描述	输入/输出
UPLO	字符数	• 'U'：保存A的上三角矩阵。 • 'L'：保存A的下三角矩阵。	输入

版权所有 © 华为技术有限公司

参数名	类型	描述	输入/输出
N	整数型	矩阵A的维数，N≥0。	输入
AP	• 在spptrf中为单精度浮点型数组。 • 在dpptrf中为双精度浮点型数组。 • 在cpptrf中为单精度复数型数组。 • 在zpptrf中为双精度复数型数组。	• 输入时，为矩阵A的上/下三角部分，大小为N*(N+1)/2。 • 输出时，为三角因子U或L，存储为上或下三角矩阵覆盖A。	输入、输出
INFO	整数型	• 等于0：成功退出。 • 小于0：第-info个参数值不合法。 • 大于0：表示顺序主子式不正定，分解失败。	输出

依赖

#include "klapack.h"

示例

C Interface：

```
const char uplo = 'U';
const int n = 4;
double ap[] = {231.8009,33.9545,162.2902,9.4143,6.6684,100.322,11.1156,53.2033,71.5384,106.5638};
int info = 0;
dpptrf_(&uplo, &n, ap, &info);
if (info != 0) {
    printf("ERROR, info = %d\n", info);
}

/*
* Output:
*    ap: 15.225009    2.230179    12.542587    0.618344    0.421714    9.988083    0.730088    4.111996
6.943561    6.396031
*/
```

Fortran Interface：

```
CHARACTER::uplo = "U"
PARAMETER (n = 4)
INTEGER :: info = 0
REAL(8) :: ap(n*(n+1)/2)

DATA ap / 231.8009,33.9545,162.2902,9.4143,6.6684,100.322,11.1156,53.2033,71.5384,106.5638 /

 EXTERNAL DPPTRF
 CALL DPPTRF(uplo, n, ap, info);
*
* Output:
```

 版权所有 © 华为技术有限公司

```
*  ap: 15.225009    2.230179    12.542587    0.618344    0.421714    9.988083    0.730088    4.111996
6.943561    6.396031
```

6.5.1.3.3 线性方程组求解函数

6.5.1.3.3.1 ?ppsv

求解线性方程组$AX = B$。其中A为实对称或共轭对称的$n \times n$正定矩阵，以压缩方式存储；X和B为$n \times nrhs$矩阵。计算矩阵A的Cholesky分解，并通过分解结果求解线性方程组。

接口定义

C Interface：

void dppsv_(const char *uplo, const int *n, const int *nrhs, double *ap, double *b, const int *ldb, int *info);

void sppsv_(const char *uplo, const int *n, const int *nrhs, float *ap, float *b, const int *ldb, int *info);

void cppsv_(const char *uplo, const int *n, const int *nrhs, float _Complex *ap, float _Complex *b, const int *ldb, int *info);

void zppsv_(const char *uplo, const int *n, const int *nrhs, double _Complex *ap, double _Complex *b, const int *ldb, int *info);

Fortran Interface：

DPPSV(uplo, n, nrhs, ap, b, ldb, info);

SPPSV(uplo, n, nrhs, ap, b, ldb, info);

CPPSV(uplo, n, nrhs, ap, b, ldb, info);

ZPPSV(uplo, n, nrhs, ap, b, ldb, info);

参数

参数名	类型	描述	输入/输出
uplo	字符型	● 'U'：A中保存上三角矩阵。 ● 'L'：A中保存下三角矩阵。	输入
n	整数型	对称矩阵A的行数或列数。	输入
nrhs	整数型	右侧项的数量，即矩阵B的列数，要求nrhs≥0。	输入

 版权所有 © 华为技术有限公司

参数名	类型	描述	输入/输出
ap	• 在sppsv中为单精度浮点型数组。 • 在dppsv中为双精度浮点型数组。 • 在cppsv中为单精度复数型数组。 • 在zppsv中为双精度复数型数组。	• 调用前以压缩格式保存对称矩阵A，见数据结构中关于压缩存储矩阵的说明。 • 调用后以压缩格式保存A的Cholesky分解结果（U或L）。	输入/输出
b	• 在sppsv中为单精度浮点型数组。 • 在dppsv中为双精度浮点型数组。 • 在cppsv中为单精度复数型数组。 • 在zppsv中为双精度复数型数组。	• 调用前为右侧矩阵B。 • 调用后为解矩阵X。	输入/输出
ldb	整数型	B的leading dimension大小，要求ldb≥max(1, n)。	输入
info	整数型	执行结果： • 等于0：成功。 • 小于0：第-info个参数值不合法。 • 大于0：A中info大小的顺序主子式非正定，无法完成分解。	输出

依赖

#include "klapack.h"

示例

C Interface：

```
char uplo = 'L';
int n = 5;
int nrhs = 2;
int ldb = 5;
int info = 0;
/*
 * A (stored in column-major):
 *  178.535    33.2929    9.2538   11.1156   21.5482
 *   33.2929  156.5955    5.1031   55.771    41.3063
 *    9.2538    5.1031  100.5198   73.8396   23.04
 *   11.1156   55.771    73.8396  106.5638   56.19
 *   21.5482   41.3063   23.04     56.19    114.5161
```


版权所有 © 华为技术有限公司

```
 */
double ap[] = {178.535,  33.2929,  9.2538, 11.1156, 21.5482,
                      156.5955,  5.1031, 55.771,  41.3063,
                              100.5198, 73.8396, 23.04,
                                      106.5638, 56.19,
                                              114.5161};
double b[] = {0.374, 6.242, 5.593, 2.456, 5.222,
              6.906, 2.31 , 3.948, 6.403, 5.664};

dppsv_(&uplo, &n, &nrhs, ap, b, &ldb, &info);
/*
 * Output X:
 *  -0.0186   0.0373
 *   0.0770  -0.0235
 *   0.1536  -0.0208
 *  -0.1574   0.0726
 *   0.0677   0.0195
 */
```

Fortran Interface：

```
      CHARACTER :: uplo = "L"
      PARAMETER (n = 5)
      PARAMETER (nrhs = 2)
      PARAMETER (ldb = 5)
      INTEGER :: info = 0

*     A (stored in column-major):
*      178.535    33.2929    9.2538   11.1156   21.5482
*       33.2929  156.5955    5.1031   55.771    41.3063
*        9.2538    5.1031  100.5198   73.8396   23.04
*       11.1156   55.771    73.8396  106.5638   56.19
*       21.5482   41.3063   23.04     56.19    114.5161

      REAL(8) :: ap(15)
      DATA ap / 178.535,  33.2929,  9.2538, 11.1156, 21.5482,
     $                   156.5955, 5.1031, 55.771,  41.3063,
     $                           100.5198, 73.8396, 23.04,
     $                                   106.5638, 56.19,
     $                                           114.5161 /
      REAL(8) :: b(n, nrhs)
      DATA b / 0.374, 6.242, 5.593, 2.456, 5.222,
     $         6.906, 2.31 , 3.948, 6.403, 5.664 /

      EXTERNAL DPPSV
      CALL DPPSV(uplo, n, nrhs, ap, b, ldb, info);
*     Output X:
*      -0.0186   0.0373
*       0.0770  -0.0235
*       0.1536  -0.0208
*      -0.1574   0.0726
*       0.0677   0.0195
```

6.5.1.3.3.2 ?gesv

求解线性方程组$A * X = B$，其中A是$n \times n$的矩阵，X和B为$n \times nrhs$的矩阵。求解过程中，对A使用具有部分旋转和行交换的LU分解，将其分解为$A = P * L * U$的形式，其中P是置换矩阵，L是单位下三角矩阵，U是上三角矩阵。并通过LU分解的结果来求解线性方程组。

接口定义

C Interface：

void sgesv_(const int *n,const int *nrhs,float *a,const int *lda,int *ipiv, float *b, const int *ldb,int *info);

 版权所有 © 华为技术有限公司

void dgesv_(const int *n,const int *nrhs,double *a,const int *lda,int *ipiv, double *b, const int *ldb,int *info);

void cgesv_(const int *n,const int *nrhs,float_Complex *a,const int *lda,int *ipiv, float_Complex *b, const int *ldb,int *info);

void zgesv_(const int *n,const int *nrhs, double_Complex *a,const int *lda,int *ipiv, double_Complex *b, const int *ldb,int *info);

Fortran Interface：

SGESV(n, nrhs, a, lda, ipiv, b, ldb, info);

DGESV(n, nrhs, a, lda, ipiv, b, ldb, info);

CGESV(n, nrhs, a, lda, ipiv, b, ldb, info);

ZGESV(n, nrhs, a, lda, ipiv, b, ldb, info);

参数

参数名	类型	描述	输入/输出
n	整数型	矩阵A的阶数，要求n≥0。	输入
nrhs	整数型	右端项的数量，即矩阵B的列数，要求nrhs≥0。	输入
a	• 在sgesv中为单精度浮点型数组。 • 在dgesv中为双精度浮点型数组。 • 在cgesv中为单精度复数型数组。 • 在zgesv中为双精度复数型数组。	矩阵维度为(lda,n)。 • 输入：n*n的系数矩阵A。 • 输出：储存了$A = P * L * U$中的L和U，其中L的单位对角线元素不存储。	输入/输出
lda	整数型	A的leading dimension大小，要求lda≥max(1, n)。	输入
ipiv	整数型数组	数组维度为n。 存放了置换矩阵P的主元索引，即矩阵第i行与第ipiv(i)行交换。	输出
b	• 在sgesv中为单精度浮点型数组。 • 在dgesv中为双精度浮点型数组。 • 在cgesv中为单精度复数型数组。 • 在zgesv中为双精度复数型数组。	矩阵维度为(ldb,nrhs)。 • 输入：$n \times nrhs$的右端矩阵B。 • 输出：在info=0的情况下，为$n \times nrhs$的解矩阵X。	输入/输出

版权所有 © 华为技术有限公司

参数名	类型	描述	输入/输出
ldb	整数型	B的leading dimension大小，要求ldb≥max(1, n)。	输入
info	整数型	执行结果： • 等于0：成功。 • 小于0：第-info个参数值不合法。 • 大于0：$U(info,info)=0$，分解已经完成，但是因为U奇异，所以无法完成求解。	输出

依赖

#include "klapack.h"

示例

C Interface：

```
int n = 5;
int nrhs = 2;
int lda = 5;
int ldb = 5;
int ipiv[5];
int info = 0;
double a[]={72.1673 , 66.1857 , 64.7644 , 28.0199 , 91.4151,
            6.5180 , 62.8483 , 72.4323 , 46.5760 ,  8.6928,
            28.9821 , 42.1828 , 18.6437 , 99.8612 , 35.6972,
            67.9812 ,  5.0880 , 85.5035 ,79.2945 , 54.5920,
            28.6869 , 49.7512 ,  7.5186 ,28.6929 , 84.6041};
double b[]={ 9.4532 ,  1.5204 ,  2.2127 ,  0.9891, 7.1778,
             6.8955 ,   7.2465,  3.5019 ,  8.2268, 3.5287};
dgesv_(&n,&nrhs,a,&lda,ipiv,b,&ldb,&info);
/*
* Output:
* 0.17538061067669766         0.16637572403098155
* -0.11183914210321674        -3.7758100714851153E-002
* 5.5415093265516101E-002     0.15550950667724139
* -1.4901096204948673E-002    -7.3593964815566168E-002
* -0.10693481055391466        -0.15230871441871899
*/
```

Fortran Interface：

```
PARAMETER(n=5)
PARAMETER(nrhs=2)
PARAMETER(lda=5)
PARAMETER(ldb=5)
INTEGER::info =0
INTEGER :: ipiv(n)
REAL(8) :: a(n*n)
      DATA a / 72.1673 , 66.1857 , 64.7644 , 28.0199 , 91.4151,
     $          6.5180 , 62.8483 , 72.4323 , 46.5760 ,  8.6928,
     $          28.9821 , 42.1828 , 18.6437 , 99.8612 , 35.6972,
     $          67.9812 ,  5.0880 , 85.5035 ,79.2945 , 54.5920,
```

 版权所有 © 华为技术有限公司

```
      $          28.6869 , 49.7512 ,  7.5186 ,28.6929 , 84.6041 /
        REAL(8) :: b(n, nrhs)
        DATA b /9.4532 ,  1.5204 ,  2.2127 ,  0.9891, 7.1778,
      $          6.8955 ,   7.2465,  3.5019 ,  8.2268, 3.5287 /
        EXTERNAL DGESV
        CALL DGESV(n, nrhs, a, lda, ipiv, b, ldb, info);
*       Output:
*       0.17538061067669766         0.16637572403098155
*       -0.11183914210321674        -3.7758100714851153E-002
*       5.5415093265516101E-002     0.15550950667724139
*       -1.4901096204948673E-002    -7.3593964815566168E-002
*       -0.10693481055391466        -0.15230871441871899
```

6.5.1.3.3.3 ?ptsv

求解线性方程组，其中系数矩阵A为实（共轭）对称正定三对角矩阵。

接口定义

C Interface：

void sptsv_(const int *N, const int *NRHS, float *D, float *E, float *B, const int *LDB, int *INFO);

void dptsv_(const int *N, const int *NRHS, double *D, double *E, double *B, const int *LDB, int *INFO);

void cptsv_(const int *N, const int *NRHS, float *D, float _Complex *E, float _Complex *B, const int *LDB, int *INFO);

void zptsv_(const int *N, const int *NRHS, double *D, double _Complex *E, double _Complex *B, const int *LDB, int *INFO);

Fortran Interface：

SPTSV(N, NRHS, D, E, B, LDB, INFO);

DPTSV(N, NRHS, D, E, B, LDB, INFO);

CPTSV(N, NRHS, D, E, B, LDB, INFO);

ZPTSV(N, NRHS, D, E, B, LDB, INFO);

参数

参数名	类型	描述	输入/输出
N	整数型	矩阵A的维数，N≥0。	输入
NRHS	整数型	矩阵B的列数，NRHS≥0。	输入
D	• 在sptsv/cptsv中为单精度浮点型数组。 • 在dptsv/zptsv中为双精度浮点型数组。	输入时，为三对角矩阵A的对角元素，维度为N。 输出时，为分解后的对角元素D。	输入、输出

 版权所有 © 华为技术有限公司

参数名	类型	描述	输入/输出
E	• 在sptsv中为单精度浮点型数组。 • 在dptsv中为双精度浮点型数组。 • 在cptsv中为单精度复数型数组。 • 在zptsv中为双精度复数型数组。	输入时，为三对角矩阵A的次对角元素，维度为N-1。 输出时，为分解后矩阵A的次对角元素。	输入、输出
B	• 在sptsv中为单精度浮点型数组。 • 在dptsv中为双精度浮点型数组。 • 在cptsv中为单精度复数型数组。 • 在zptsv中为双精度复数型数组。	输入时，为右端项矩阵，维度为(LDB, NRHS)。 输出时，为结果X，维度为(LDB, NRHS)。	输入、输出
LDB	整数型	矩阵B的主维。	输入
INFO	整数型	• 等于0：成功退出。 • 小于0：第-info个参数值不合法。 • 大于0：表示第INFO阶顺序主子式不正定。	输出

依赖

#include "klapack.h"

示例

C Interface：

```
   const int n = 4;
   const int ldb = n;
   const int nrhs = 1;
   double d[] = {7.608696,7.739130, 7.347826,4.652174};
   double e[] = {0.352941,0.000000,0.823529};
   double b[] = {0.300000,0.100000,0.700000,0.200000};
   int info = 0;

   dptsv_(&n, &nrhs, d, e, b, &ldb, &info);
/*
* Output:
*   d: 7.608696       7.722759       7.347826       4.559874
*   e: 0.046387       0.000000       0.112078
*   b: 0.038912       0.011147       0.092279       0.026655
*/
```

Fortran Interface：

 版权所有 © 华为技术有限公司

```
PARAMETER (n = 4)
PARAMETER (ldb = 4)
PARAMETER (nrhs = 1)
INTEGER :: info = 0
REAL(8) :: d(n)
REAL(8) :: e(n-1)
REAL(8) :: b(ldb, nrhs)

DATA d / 7.608696,7.739130, 7.347826,4.652174 /
DATA e / 0.352941,0.000000,0.823529 /
DATA b / 0.300000,0.100000,0.700000,0.200000 /
 EXTERNAL DPTSV
 CALL DPTSV(n, nrhs, d, e, b, ldb, info);
*
* Output:
*   d: 7.608696       7.722759       7.347826       4.559874
*   e: 0.046387       0.000000       0.112078
*   b: 0.038912       0.011147       0.092279       0.026655
```

6.5.1.3.3.4 ?gtsv

求解线性方程组A*X=B，其中系数矩阵A为一般三对角矩阵。

接口定义

C Interface：

void sgtsv(const int *N, const int *NRHS, float *DL, float *D, float *DU, float *B, const int *LDB, int *INFO);

void dgtsv(const int *N, const int *NRHS, double *DL, double *D, double *DU, double *B, const int *LDB, int *INFO);

void cgtsv(const int *N, const int *NRHS, float _Complex *DL, float _Complex *D, float _Complex *DU, float _Complex *B, const int *LDB, int *INFO);

void zgtsv(const int *N, const int *NRHS, double _Complex *DL, double _Complex *D, double _Complex *DU, double _Complex *B, const int *LDB, int *INFO);

Fortran Interface：

SGTSV(N, NRHS, DL, D, DU, B, LDB, INFO);

DGTSV(N, NRHS, DL, D, DU, B, LDB, INFO);

CGTSV(N, NRHS, DL, D, DU, B, LDB, INFO);

ZGTSV(N, NRHS, DL, D, DU, B, LDB, INFO);

参数

参数名	类型	描述	输入/输出
N	整数型	矩阵A的维数，N≥0。	输入
NRHS	整数型	矩阵B的列数，NRHS≥0。	输入

版权所有 © 华为技术有限公司

参数名	类型	描述	输入/输出
DL	• 在sgtsv中为单精度浮点型数组。 • 在dgtsv中为双精度浮点型数组。 • 在cgtsv中为单精度复数型数组。 • 在zgtsv中为双精度复数型数组。	输入时，为三对角矩阵A的次对角元素，维度为N-1。 输出时，为U的第二超对角元素。	输入、输出
D	• 在sgtsv中为单精度浮点型数组。 • 在dgtsv中为双精度浮点型数组。 • 在cgtsv中为单精度复数型数组。 • 在zgtsv中为双精度复数型数组。	输入时，为三对角矩阵A的对角元素，维度为N。 输出时，为U的对角元素。	输入、输出
DU	• 在sgtsv中为单精度浮点型数组。 • 在dgtsv中为双精度浮点型数组。 • 在cgtsv中为单精度复数型数组。 • 在zgtsv中为双精度复数型数组。	输入时，为三对角矩阵A的超对角元素，维度为N-1。 输出时，为U的第一超对角元素。	输入、输出
B	• 在sgtsv中为单精度浮点型数组。 • 在dgtsv中为双精度浮点型数组。 • 在cgtsv中为单精度复数型数组。 • 在zgtsv中为双精度复数型数组。	右端项矩阵，维度为(LDB, NRHS)。	输入、输出
LDB	整数型	矩阵B的主维。	输入
INFO	整数型	• 等于0：成功退出。 • 小于0：第-info个参数值不合法。 • 大于0：表示U(INFO, INFO)为零。	输出

 版权所有 © 华为技术有限公司

依赖

include "klapack.h"

示例

C Interface：

```
const int n = 4;
const int ldb = n;
const int nrhs = 1;
double dl[] = {0.923077, 0.153846, 0.153846};
double d[] = {0.521739, 0.043478, 0.304348, 0.086957};
double du[] = {0.924528, 0.150943, 0.566038};
double b[] = {0.986301, 0.397260, 0.958904, 0.520548};
int info = 0;
dgtsv_(&n, &nrhs, dl, d, du, b, &ldb, &info);
if (info != 0) {
    printf("ERROR, info = %d\n", info);
}

/*
* Output:
*   d:  0.923077       0.899954       0.318932       -0.186088
*   dl: 0.150943       0.000000       0.153846
*   du: 0.043478        -0.085316       0.566038
*   b:  -0.239541      1.201996       3.750506       -0.649209
*/
```

Fortran Interface：

```
PARAMETER (n = 4)
PARAMETER (ldb = 4)
PARAMETER (nrhs = 1)
INTEGER :: info = 0
REAL(8) :: d(n)
REAL(8) :: dl(n-1)
REAL(8) :: du(n-1)
REAL(8) :: b(ldb, nrhs)

DATA d / 0.521739, 0.043478, 0.304348, 0.086957 /
DATA dl / 0.923077, 0.153846, 0.153846 /
DATA du / 0.924528, 0.150943, 0.566038 /
DATA b / 0.986301, 0.397260, 0.958904, 0.520548 /
 EXTERNAL DGTSV
 CALL DGTSV(n, nrhs, dl, d, du, b, ldb, info);
*
* Output:
*   d:  0.923077       0.899954       0.318932       -0.186088
*   dl: 0.150943       0.000000       0.153846
*   du: 0.043478        -0.085316       0.566038
*   b:  -0.239541      1.201996       3.750506       -0.649209
```

6.5.1.3.3.5 ?posv

求解线性方程组$AX = B$。其中A为实对称或共轭对称的$n \times n$正定矩阵；X和B为$n \times nrhs$矩阵。计算矩阵A的Cholesky分解，并通过分解结果求解线性方程组。

接口定义

C Interface：

void dposv_(const char *uplo, const int *n, const int *nrhs, double *a, const int *lda, double *b, const int *ldb, int *info);

 版权所有 © 华为技术有限公司

void sposv_(const char *uplo, const int *n, const int *nrhs, float *a, const int *lda, float *b, const int *ldb, int *info);

void cposv_(const char *uplo, const int *n, const int *nrhs, float _Complex *a, const int *lda, float _Complex *b, const int *ldb, int *info);

void zposv_(const char *uplo, const int *n, const int *nrhs, double _Complex *a, const int *lda, double _Complex *b, const int *ldb, int *info);

Fortran Interface：

DPOSV(uplo, n, nrhs, a, lda, b, ldb, info);

SPOSV(uplo, n, nrhs, a, lda, b, ldb, info);

CPOSV(uplo, n, nrhs, a, lda, b, ldb, info);

ZPOSV(uplo, n, nrhs, a, lda, b, ldb, info);

参数

参数名	类型	描述	输入/输出
uplo	字符型	• 'U'：A中保存上三角矩阵。 • 'L'：A中保存下三角矩阵。	输入
n	整数型	对称矩阵A的行数或列数。	输入
nrhs	整数型	右侧项的数量，即矩阵B的列数，要求nrhs≥0。	输入
a	• 在sposv中为单精度浮点型数组。 • 在dposv中为双精度浮点型数组。 • 在cposv中为单精度复数型数组。 • 在zposv中为双精度复数型数组。	• 调用前：如果uplo='U'，那么矩阵A将包含上三角部分，下三角部分不会使用；如果uplo='L'，那么矩阵A将包含下三角部分，上三角部分不会使用。 • 调用后：保存A的Cholesky分解结果（U或L）。	输入/输出
lda	整数型	A的leading dimension大小，要求lda≥max(1, n)。	输入
b	• 在sposv中为单精度浮点型数组。 • 在dposv中为双精度浮点型数组。 • 在cposv中为单精度复数型数组。 • 在zposv中为双精度复数型数组。	• 调用前：为右侧矩阵B。 • 调用后：为解矩阵X。	输入/输出

 版权所有 © 华为技术有限公司

参数名	类型	描述	输入/输出
ldb	整数型	B的leading dimension大小，要求ldb≥max(1, n)。	输入
info	整数型	执行结果： ● 等于0：成功。 ● 小于0：第-info个参数值不合法。 ● 大于0：A中info大小的顺序主子式非正定，无法完成分解。	输出

依赖

#include "klapack.h"

示例

C Interface：

```
char uplo = 'L';
int n = 4;
int nrhs = 1;
int lda = 4;
int ldb = 4;
int info = 0;
/*
* A (stored in column-major):
*  231.8009    33.9545    9.4153    11.1156
*   33.9454    162.2902   6.6684    53.2023
*   9.4153     6.6684     100.3220  71.5384
*   11.1156    53.2023    71.5384   106.5638
*/

double a[] = {231.8009,   33.9545,   9.4143,   11.1156,
              33.9495,  162.2902,   6.6684,   53.2033,
              9.4143,    6.6684,  100.322,   71.5384,
              11.1156,   53.2033,  71.5384,  106.5638};
double b[] = {1.0, 2.0, 3.0, 4.0};

dposv_(&uplo, &n, &nrhs, a, &lda, b, &ldb, &info);
/*
 * Output X:  0.002361        0.000904        0.006432        0.032521
 */
```

Fortran Interface：

```
CHARACTER :: uplo = "L"
PARAMETER (n = 4)
PARAMETER (nrhs = 1)
PARAMETER (lda = 4)
PARAMETER (ldb = 4)
INTEGER :: info = 0

*
*  A (stored in column-major):
*  231.8009    33.9545    9.4153    11.1156
*   33.9454    162.2902   6.6684    53.2023
*   9.4153     6.6684     100.3220  71.5384
```


版权所有 © 华为技术有限公司

```
*   11.1156    53.2023    71.5384   106.5638


REAL(8) :: a(lda, n)
DATA a / 231.8009,   33.9545,   9.4143,   11.1156,
 $        33.9495,  162.2902,   6.6684,   53.2033,
 $        9.4143,    6.6684,  100.322,   71.5384,
 $        11.1156,   53.2033,  71.5384,  106.5638  /
REAL(8) :: b(ldb, nrhs)
DATA b / 1.0, 2.0, 3.0, 4.0 /

EXTERNAL DPOSV
CALL DPOSV(uplo, n, nrhs, a, lda, b, ldb, info);
* Output X: 0.002361         0.000904         0.006432         0.032521
```

6.5.1.3.4 矩阵求逆函数

6.5.1.3.4.1 ?getri

根据?getrf得到的LU分解结果计算逆矩阵。

接口定义

C Interface：

void sgetri_(const int *n, float *a, const int *lda, const int *ipiv, float *work, const int *lwork, int *info);

void dgetri_(const int *n, double *a, const int *lda, const int *ipiv, double *work, const int *lwork, int *info);

void cgetri_(const int *n, float _Complex *a, const int *lda, const int *ipiv, float _Complex *work, const int *lwork, int *info);

void zgetri_(const int *n, double _Complex *a, const int *lda, const int *ipiv, double _Complex *work, const int *lwork, int *info);

Fortran Interface：

SGETRI(n, a, lda, ipiv, work, lwork, info)

DGETRI(n, a, lda, ipiv, work, lwork, info)

CGETRI(n, a, lda, ipiv, work, lwork, info)

ZGETRI(n, a, lda, ipiv, work, lwork, info)

参数

参数名	类型	描述	输入/输出
n	整数型	矩阵A的行数或列数。	输入

版权所有 © 华为技术有限公司

参数名	类型	描述	输入/输出
a	• 在sgetri中为单精度浮点型数组。 • 在dgetri中为双精度浮点型数组。 • 在cgetri中为单精度复数型数组。 • 在zgetri中为双精度复数型数组。	• 调用前保存矩阵A的LU分解结果（由?getrf得到）。 • 调用后保存原矩阵A的逆矩阵。	输入/输出
lda	整数型	A的leading dimension大小，要求lda ≥ max(1, n)。	输入
ipiv	整数型数组	?getrf得到的主元索引，长度为n：1 ≤ ipiv ≤ n，表示分解中矩阵第i行与第ipiv[i-1]行交换。	输入
work	• 在sgetri中为单精度浮点型数组。 • 在dgetri中为双精度浮点型数组。 • 在cgetri中为单精度复数型数组。 • 在zgetri中为双精度复数型数组。	临时存储空间，使用lwork=-1调用后work[0]为最优的lwork值。	输出
lwork	整数型	work数组的长度。 lwork=-1时查询最优work大小，结果保存在work[0]中，否则要求lwork≥n。	输入
info	整数型	执行结果： • 等于0：成功。 • 小于0：第-info个参数值不合法。 • 大于0：U的对角线上第info个元素为0，矩阵不可逆。	输出

依赖

#include "klapack.h"

示例

C Interface：

 版权所有 © 华为技术有限公司

```
int n = 4;
int lda = 4;
int info = 0;
double *work = NULL;
double qwork;
int lwork = -1;
/*
 * Origin A:
 *  1.80  2.88    2.05  -0.89
 *  5.25  -2.95  -0.95  -3.80
 *  1.58  -2.69  -2.90  -1.04
 * -1.11  -0.66  -0.59   0.80
 * After LU factorization (via getrf, stored in column-major):
 *  5.2500  -2.9500  -0.9500  -3.8000
 *  0.3429   3.8914   2.3757   0.4129
 *  0.3010  -0.4631  -1.5139   0.2948
 * -0.2114  -0.3299   0.0047   0.1314
 */
double a[] = {5.2500, 0.3429,  0.3010, -0.2114,
              -2.9500, 3.8914, -0.4631, -0.3299,
              -0.9500, 2.3757, -1.5139,  0.0047,
              -3.8000, 0.4129,  0.2948,  0.1314};
int ipiv[] = {2, 2, 3, 4};
/* Query optimal work size */
dgetri_(&n, a, &lda, ipiv, &qwork, &lwork, &info);
if (info != 0) {
   return ERROR;
}
lwork = (int)qwork;
work = (double *)malloc(sizeof(double) * lwork);
/* Calculate inversion */
dgetri_(&n, a, &lda, ipiv, work, &lwork, &info);
free(work);
/*
 * Output:
 *   1.7718  0.5753  0.0844  4.8145
 *  -0.1174 -0.4455  0.4113 -1.7122
 *   0.1798  0.4526 -0.6675  1.4820
 *   2.4941  0.7644 -0.0358  7.6104
 */
```

Fortran Interface:

```
      PARAMETER (n = 4)
      PARAMETER (lda = 4)
      INTEGER :: info = 0
      REAL(8) qwork(1)
      REAL(8), ALLOCATABLE :: work(:)
      INTEGER :: lwork = -1

*     Origin A:
*      1.80  2.88    2.05  -0.89
*      5.25  -2.95  -0.95  -3.80
*      1.58  -2.69  -2.90  -1.04
*     -1.11  -0.66  -0.59   0.80
*     After LU factorization (via getrf, stored in column-major):
*      5.2500  -2.9500  -0.9500  -3.8000
*      0.3429   3.8914   2.3757   0.4129
*      0.3010  -0.4631  -1.5139   0.2948
*     -0.2114  -0.3299   0.0047   0.1314

      REAL(8) :: a(n, n)
      DATA a / 5.2500, 0.3429,  0.3010, -0.2114,
     $         -2.9500, 3.8914, -0.4631, -0.3299,
     $         -0.9500, 2.3757, -1.5139,  0.0047,
     $         -3.8000, 0.4129,  0.2948,  0.1314 /
      INTEGER :: ipiv(n)
      DATA ipiv / 2, 2, 3, 4 /
*     Query optimal work size
      EXTERNAL DGETRI
```

 版权所有 © 华为技术有限公司

```
      CALL DGETRI(n, a, lda, ipiv, qwork, lwork, info)
      IF (info.NE.0) THEN
         CALL EXIT(1)
      END IF
      lwork = INT(qwork(1))
      ALLOCATE(work(lwork))
*     Calculate inversion
      CALL DGETRI(n, a, lda, ipiv, work, lwork, info)
      DEALLOCATE(work)
*     Output:
*       1.7718  0.5753  0.0844  4.8145
*      -0.1174 -0.4455  0.4113 -1.7122
*       0.1798  0.4526 -0.6675  1.4820
*       2.4941  0.7644 -0.0358  7.6104
```

6.5.1.3.4.2 ?potri

计算对称正定矩阵的逆。

接口定义

C interface：

void spotri_(const char *UPLO, const int *N, const float *A, const int *LDA, int *INFO);

void dpotri_(const char *UPLO, const int *N, const double *A, const int *LDA, int *INFO);

void cpotri_(const char *UPLO, const int *N, const float _Complex *A, const int *LDA, int *INFO);

void zpotri_(const char *UPLO, const int *N, const double _Complex *A, const int *LDA, int *INFO);

Fortran Interface：

SPOTRI(UPLO, N, A, LDA, INFO);

DPOTRI(UPLO, N, A, LDA, INFO);

CPOTRI(UPLO, N, A, LDA, INFO);

ZPOTRI(UPLO, N, A, LDA, INFO);

参数

参数名	类型	描述	输入/输出
UPLO	字符型	'U'：保存A的上三角矩阵。 'L'：保存A的下三角矩阵。	输入
N	整数型	矩阵A的维数，n ≥ 0。	输入

版权所有 © 华为技术有限公司

参数名	类型	描述	输入/输出
A	• 在spotri中为单精度浮点型数组。 • 在dpotri中为双精度浮点型数组。 • 在cpotri中为单精度复数型数组。 • 在zpotri中为双精度复数型数组。	• 调用前保存矩阵A的Cholesky分解结果（由?potrf得到）。 • 调用后保存原矩阵A的逆矩阵。	输入、输出
LDA	整数型	A的leading dimension大小，lda ≥ max(1,n)。	输入
INFO	整数型	执行结果： • 等于0：成功。 • 小于0：第-info个参数值不合法。 • 大于0：U的对角线上第info个元素为0，矩阵不可逆。	输出

依赖

#include "klapack.h"

示例

C Interface：

```
char uplo = 'L'
int n = 4;
int lda = n;
int info = 0;
/*
* A: After Cholesky factorization (via potrf, stored in column-major):
*  231.8009    33.9545    9.4153   11.1156
*   33.9454   162.2902   6.6684   53.2023
*   9.4153   6.6684  100.3220  71.5384
*   11.1156   53.2023    71.5384  123.4425
*/
double a[] = {231.8009,   33.9545,   9.4143,   11.1156,
              33.9495,  162.2902,   6.6684,   53.2033,
              9.4143,    6.6684,  100.322,   71.5384,
              11.1156,   53.2033,  71.5384,  106.5638 };
 dpotri_(&uplo, &n, a, &lda, &info);
/*
* Output:
*   0.000020  -0.000006  -0.000005   0.000002
*   33.949500  0.000046   0.000015  -0.000026
*   9.414300   6.668400   0.000144  -0.000063
*   11.115600  53.203300  71.538400  0.000088
*/
```

Fortran Interface：

 版权所有 © 华为技术有限公司

```
CHARACTER :: uplo = "L"
PARAMETER (n = 4)
PARAMETER (lda = 4)
INTEGER :: info = 0
REAL(8) :: a(n,n)
* A: After Cholesky factorization (via potrf, stored in column-major):
*  231.8009    33.9545    9.4153   11.1156
*   33.9454   162.2902   6.6684   53.2023
*   9.4153   6.6684  100.3220  71.5384
*   11.1156   53.2023    71.5384  123.4425

DATA a / 231.8009,  33.9545,  9.4143, 11.1156,
 $          33.9495,  162.2902, 6.6684, 53.2033,
 $          9.4143,    6.6684,  100.322, 71.5384,
 $          11.1156,   53.2033, 71.5384, 106.5638 /
 EXTERNAL DPOTRF
 CALL DPOTRI (uplo, n, a, lda, info);
*
* Output:
*   0.000020  -0.000006  -0.000005   0.000002
*   33.949500  0.000046   0.000015  -0.000026
*   9.414300   6.668400   0.000144  -0.000063
*   11.115600  53.203300  71.538400  0.000088
*
```

6.5.1.3.4.3 ?pptri

计算压缩存储的对称正定矩阵的逆矩阵。

接口定义

C interface：

void spptri_(const char *UPLO, const int *N, const float *AP, int *INFO);

void dpptri_(const char *UPLO, const int *N, const double *AP, int *INFO);

void cpptri_(const char *UPLO, const int *N, const float _Complex *AP, int *INFO);

void zpptri_(const char *UPLO, const int *N, const double _Complex *AP, int *INFO);

Fortran Interface：

SPPTRI(UPLO, N, AP, INFO);

DPPTRI(UPLO, N, AP, INFO);

CPPTRI(UPLO, N, AP, INFO);

ZPPTRI(UPLO, N, AP, INFO);

参数

参数名	类型	描述	输入/输出
UPLO	字符型	• 'U'：保存A的上三角矩阵。 • 'L'：保存A的下三角矩阵。	输入
N	整数型	矩阵A的维数，n ≥ 0。	输入

版权所有 © 华为技术有限公司

参数名	类型	描述	输入/输出
AP	• 在spptri中为单精度浮点型数组。 • 在dpptri中为双精度浮点型数组. • 在cpptri中为单精度复数型数组。 • 在zpptri中为双精度复数型数组。	• 调用前：如果UPLO='U'时，以压缩格式保存矩阵A的Cholesky分解结果的U部分；如果UPLO='L'时，以压缩格式保存矩阵A的Cholesky分解结果的L部分。 • 调用后：保存矩阵A的逆矩阵。	输入、输出
INFO	整数型	执行结果： • 等于0：成功。 • 小于0：第-info个参数值不合法。 • 大于0：U的对角线上第info个元素为0，矩阵不可逆。	输出

依赖

#include "klapack.h"

示例

C Interface：

```
const char uplo = 'U';
const int n = 4;
double ap[] = {231.8009,33.9545,162.2902,9.4143,6.6684,100.322,11.1156,53.2033,71.5384,106.5638};
int info = 0;
dpptrf_(&uplo, &n, ap, &info);
if (info != 0) {
    printf("ERROR, info = %d\n", info);
}

dpptri_(&uplo, &n, ap, &info);
if (info != 0) {
    printf("ERROR, info = %d\n", info);
}

/*
* Output:
*   ap: 0.004478    -0.001104    0.008634    -0.000781    0.004837    0.021837    0.000608    -0.007443
-0.016993    0.024444
*/
```

Fortran Interface：

```
CHARACTER::uplo = "U"
PARAMETER (n = 4)
INTEGER :: info = 0
REAL(8) :: ap(n*(n+1)/2)

DATA ap / 231.8009,33.9545,162.2902,9.4143,6.6684,100.322,11.1156,53.2033,71.5384,106.5638 /
```

 版权所有 © 华为技术有限公司

```
 EXTERNAL DPPTRF,DPPTRI
 CALL DPPTRF(uplo, n, ap, info)
 CALL DPPTRI(uplo, n, ap, info)
*
* Output:
*    ap: 0.004478    -0.001104    0.008634    -0.000781    0.004837    0.021837    0.000608    -0.007443
-0.016993    0.024444
```

6.5.1.3.4.4 ?trtri

计算上/下三角矩阵的逆矩阵。

接口定义

C Interface：

void strtri_(const char *uplo, const char *diag, const int *n, float *a, const int *lda, int *info);

void dtrtri_(const char *uplo, const char *diag, const int *n, double *a, const int *lda, int *info);

void ctrtri_(const char *uplo, const char *diag, const int *n, float _Complex *a, const int *lda, int *info);

void ztrtri_(const char *uplo, const char *diag, const int *n, double _Complex *a, const int *lda, int *info);

Fortran Interface：

STRTRI(UPLO, DIAG, N, A, LDA, INFO)

DTRTRI(UPLO, DIAG, N, A, LDA, INFO)

CTRTRI(UPLO, DIAG, N, A, LDA, INFO)

ZTRTRI(UPLO, DIAG, N, A, LDA, INFO)

参数

参数名	类型	描述	输入/输出
UPLO	字符型	● 'U'：矩阵A的上三角部分。 ● 'L'：矩阵A的下三角部分。	输入
DIAG	字符型	● 'N'：矩阵A为非单位三角矩阵。 ● 'U'：矩阵为单位三角矩阵。	输入/输出
N	整数型	矩阵A的维度，N≥0。	输入

版权所有 © 华为技术有限公司

参数名	类型	描述	输入/输出
A	• 在strtri中为单精度浮点型数组。 • 在dtrtri中为双精度浮点型数组。 • 在ctrtri中为单精度复数型数组。 • 在ztrtri中为双精度复数型数组。	• 调用前：如果UPLO='U'，则矩阵A包含了上三角部分，下三角部分不会使用；如果UPLO='L'，则矩阵A包含了下三角部分，上三角部分不会使用；如果DIAG='U'，则对角元素不会使用。 • 调用后：以相同格式存储矩阵A的逆矩阵。	输出
LDA	整数型	矩阵A的主维，LDA≥max(1,N)。	输入
INFO	整数型	执行结果： • 等于0：成功。 • 小于0：第-info个参数值不合法。 • 大于0：U的对角线上第info个元素为0，矩阵不可逆。	输出

依赖

#include "klapack.h"

示例

C Interface：

```
const char uplo = 'U';
const char diag = 'N';
const int n = 4;
const int lda = n;
/*
* A: After Cholesky factorization (via potrf, stored in column-major):
*  231.8009    33.9545    9.4153   11.1156
*   33.9454   162.2902   6.6684   53.2023
*   9.4153   6.6684  100.3220  71.5384
*   11.1156   53.2023    71.5384  106.5638
*/
double a[] = {231.8009,   33.9545,   9.4143,   11.1156,
              33.9495,  162.2902,   6.6684,   53.2033,
              9.4143,    6.6684,  100.322,   71.5384,
              11.1156,   53.2033,  71.5384,  106.5638 };
int info = 0;
dtrtri_(&uplo, &diag, &n, a, &lda, &info);
if (info != 0) {
    printf("ERROR, info = %d\n", info);
}

/*
* Output:
```

 版权所有 © 华为技术有限公司

```
*   a:
*    0.004314    -0.000902    -0.000345    0.000232
*    33.954500    0.006162    -0.000410    -0.002801
*    9.414300    6.668400    0.009968    -0.006692
*    11.115600    53.203300    71.538400    0.009384
*/
```

Fortran Interface：

```
CHARACTER::uplo = "U"
CHARACTER::diag = "N"
PARAMETER (n = 4)
PARAMETER (lda = 4)
INTEGER :: info = 0
REAL(8) :: a(lda, n)

* A: After Cholesky factorization (via potrf, stored in column-major):
*  231.8009    33.9545    9.4153   11.1156
*   33.9454   162.2902   6.6684   53.2023
*   9.4153   6.6684  100.3220  71.5384
*   11.1156   53.2023    71.5384  106.5638

DATA a / 231.8009,  33.9545,  9.4143, 11.1156,
 $           33.9495,  162.2902, 6.6684, 53.2033,
 $           9.4143,    6.6684,  100.322, 71.5384,
 $           11.1156,   53.2033, 71.5384, 106.5638 /

 EXTERNAL DTRTRI
 CALL DTRTRI(uplo, diag,  n, a, lda, info)

*
* Output:
*   a:
*    0.004314    -0.000902    -0.000345    0.000232
*    33.954500    0.006162    -0.000410    -0.002801
*    9.414300    6.668400    0.009968    -0.006692
*    11.115600    53.203300    71.538400    0.009384
*
```

6.5.1.3.5 回代求解函数

6.5.1.3.5.1 ?pttrs

求解三对角方程AX=B，其中系数矩阵A由?pttrf分解而来。

接口定义

C Interface：

void spttrs_(const int *N, const int *NRHS, float *D, float *E, float *B, const int *LDB, int *INFO);

void dpttrs_(const int *N, const int *NRHS, double *D, double *E, double *B, const int *LDB, int *INFO);

void cpttrs_(const char *UPLO, const int *N, const int *NRHS, float *D, float _Complex *E, float _Complex *B, const int *LDB, int *INFO);

void zpttrs_(const char *UPLO, const int *N, const int *NRHS, double *D, double _Complex *E, double _Complex *B, const int *LDB, int *INFO);

Fortran Interface：

SPTTRS(N, NRHS, D, E, B, LDB, INFO);

 版权所有 © 华为技术有限公司

DPTTRS(N, NRHS, D, E, B, LDB, INFO);

CPTTRS(UPLO, N, NRHS, D, E, B, LDB, INFO);

ZPTTRS(UPLO, N, NRHS, D, E, B, LDB, INFO);

参数

参数名	类型	描述	输入/输出
UPLO（复数类型特有）	字符型	• 'U'： A=U**H*D*U • 'L'： A=L*D*L**H	输入
N	整数型	矩阵A的维数，N ≥ 0。	输入
NRHS	整数型	矩阵B的列数，NRHS ≥ 0。	输入
D	• 在sptrs/cpttrs中为单精度浮点型数组。 • 在dpttrs/zpttrs中为双精度浮点型数组。	三对角矩阵A的对角元素，维度为N。	输入
E	• 在sptrs中为单精度浮点型数组。 • 在dpttrs中为双精度浮点型数组。 • 在cpttrs中为单精度复数型数组。 • 在zpttrs中为双精度复数型数组。	三对角矩阵A的次对角元素，维度为N-1。	输入
B	• 在sptrs中为单精度浮点型数组。 • 在dpttrs中为双精度浮点型数组。 • 在cpttrs中为单精度复数型数组。 • 在zpttrs中为双精度复数型数组。	输入时，为右端项矩阵，维度为(LDB, NRHS)。 输出时，为解X，维度为(LDB, NRHS)。	输入、输出
LDB	整数型	矩阵B的主维。	输入
INFO	整数型	• 等于0：成功退出。 • 小于0：第-info个参数值不合法。	输出

依赖

#include "klapack.h"

 版权所有 © 华为技术有限公司

示例

C Interface：

```
    const int n = 4;
    const int ldb = n;
    const int nrhs = 1;
    double d[] = {7.608696,7.739130, 7.347826,4.652174};
    double e[] = {0.352941,0.000000,0.823529};
    double b[] = {0.300000,0.100000,0.700000,0.200000};
    int info = 0;

    dpttrs_(&n, &nrhs, d, e, b, &ldb, &info);
/*
* Output:
*    b: 0.039697        -0.000760        0.161909        -0.080924
*/
```

Fortran Interface：

```
PARAMETER (n = 4)
PARAMETER (ldb = 4)
PARAMETER (nrhs = 1)
INTEGER :: info = 0
REAL(8) :: d(n)
REAL(8) :: e(n-1)
REAL(8) :: b(ldb, nrhs)

DATA d / 7.608696,7.739130, 7.347826,4.652174 /
DATA e / 0.352941,0.000000,0.823529 /
DATA b / 0.300000,0.100000,0.700000,0.200000 /
 EXTERNAL DPTTRS
 CALL DPTTRS(n, nrhs, d, e, b, ldb, info);
*
* Output:
*   b: 0.039697        -0.000760        0.161909        -0.080924
```

6.5.1.3.5.2 ?ptts2

求解三对角方程AX=B，其中系数矩阵A由?pttrf分解而来。

接口定义

C Interface：

void sptts2_(const int *N, const int *NRHS, float *D, float *E, float *B, const int *LDB);

void dptts2_(const int *N, const int *NRHS, double *D, double *E, double *B, const int *LDB);

void cptts2_(const int *IUPLO, const int *N, const int *NRHS, float *D, float _Complex *E, float _Complex *B, const int *LDB);

void zptts2_(const int *IUPLO, const int *N, const int *NRHS, double *D, double _Complex *E, double _Complex *B, const int *LDB);

Fortran Interface：

SPTTS2(N, NRHS, D, E, B, LDB);

DPTTS2(N, NRHS, D, E, B, LDB);

CPTTS2(IUPLO, N, NRHS, D, E, B, LDB);

版权所有 © 华为技术有限公司

ZPTTS2(IUPLO, N, NRHS, D, E, B, LDB);

参数

参数名	类型	描述	输入/输出
IUPLO（复数类型特有）	整数型	● 1：A=U**H*D*U ● 0：A=L*D*L**H	输入
N	整数型	矩阵A的维数，N ≥ 0。	输入
NRHS	整数型	矩阵B的列数，NRHS ≥ 0。	输入
D	● 在sptts2/cptts2中为单精度浮点型数组。 ● 在dptts2/zptts2中为双精度浮点型数组。	三对角矩阵A的对角元素，维度为N。	输入
E	● 在sptts2中为单精度浮点型数组。 ● 在dptts2中为双精度浮点型数组。 ● 在cptts2中为单精度复数型数组。 ● 在zptts2中为双精度复数型数组。	三对角矩阵A的次对角元素，维度为N-1。	输入
B	● 在sptts2中为单精度浮点型数组。 ● 在dptts2中为双精度浮点型数组。 ● 在cptts2中为单精度复数型数组。 ● 在zptts2中为双精度复数型数组。	输入时，为右端项矩阵，维度为(LDB, NRHS)。 输出时，为解X，维度为(LDB, NRHS)。	输入、输出
LDB	整数型	矩阵B的主维。	输入

依赖

#include "klapack.h"

示例

C Interface：

```
const int n = 4;
const int ldb = n;
```

 版权所有 © 华为技术有限公司

```
    const int nrhs = 1;
    double d[] = {7.608696,7.739130, 7.347826,4.652174};
    double e[] = {0.352941,0.000000,0.823529};
    double b[] = {0.300000,0.100000,0.700000,0.200000};

    dptts2_(&n, &nrhs, d, e, b, &ldb);
/*
* Output:
*    b: 0.039697        -0.000760        0.161909        -0.080924
*/
```

Fortran Interface：

```
PARAMETER (n = 4)
PARAMETER (ldb = 4)
PARAMETER (nrhs = 1)
REAL(8) :: d(n)
REAL(8) :: e(n-1)
REAL(8) :: b(ldb, nrhs)

DATA d / 7.608696,7.739130, 7.347826,4.652174 /
DATA e / 0.352941,0.000000,0.823529 /
DATA b / 0.300000,0.100000,0.700000,0.200000 /
 EXTERNAL DPTTS2
 CALL DPTTS2(n, nrhs, d, e, b, ldb);
*
* Output:
*  b: 0.039697        -0.000760        0.161909        -0.080924
```

6.5.1.3.5.3 ?gttrs

求解三对角方程A*X=B或A**T *X=B或A**H *X=B，其中系数矩阵A由?gttrf分解而来。

接口定义

C Interface：

void sgttrs_(const char *TRANS, const int *N, const int *NRHS, float *DL, float *D, float *DU, float *DU2, int *IPIV, float *B, const int *LDB, int *INFO);

void dgttrs_(const char *TRANS, const int *N, const int *NRHS, double *DL, double *D, double *DU, double *DU2, int *IPIV, double *B, const int *LDB, int *INFO);

void cgttrs_(const char *TRANS, const int *N, const int *NRHS, float _Complex *DL, float _Complex *D, float _Complex *DU, float _Complex *DU2, int *IPIV, float _Complex *B, const int *LDB, int *INFO);

void zgttrs_(const char *TRANS, const int *N, const int *NRHS, double _Complex *DL, double _Complex *D, double _Complex *DU, double _Complex *DU2, int *IPIV, double _Complex *B, const int *LDB, int *INFO);

Fortran Interface：

SGTTRS(TRANS, N, NRHS, DL, D, DU, DU2, IPIV, B, LDB, INFO);

DGTTRS(TRANS, N, NRHS, DL, D, DU, DU2, IPIV, B, LDB, INFO);

CGTTRS(TRANS, N, NRHS, DL, D, DU, DU2, IPIV, B, LDB, INFO);

ZGTTRS(TRANS, N, NRHS, DL, D, DU, DU2, IPIV, B, LDB, INFO);

 版权所有 © 华为技术有限公司

参数

参数名	类型	描述	输入/输出
TRANS	字符型	• 'N'，A*X=B • 'T'，A**T*X=B • 'C'，A**T*X=B（共轭转置）	输入
N	整数型	矩阵A的阶数，N ≥ 0。	输入
NRHS	整数型	矩阵B的列数，NRHS ≥ 0。	输入
DL	• 在sgttrs中为单精度浮点型数组。 • 在dgttrs中为双精度浮点型数组。 • 在cgttrs中为单精度复数型数组。 • 在zgttrs中为双精度复数型数组。	LU分解中的L部分，维度为N-1。	输入
D	• 在sgttrs中为单精度浮点型数组。 • 在dgttrs中为双精度浮点型数组。 • 在cgttrs中为单精度复数型数组。 • 在zgttrs中为双精度复数型数组。	U的对角元素，维度为N。	输入
DU	• 在sgttrs中为单精度浮点型数组。 • 在dgttrs中为双精度浮点型数组。 • 在cgttrs中为单精度复数型数组。 • 在zgttrs中为双精度复数型数组。	为U的第一超对角元素。	输入
DU2	• 在sgttrs中为单精度浮点型数组。 • 在dgttrs中为双精度浮点型数组。 • 在cgttrs中为单精度复数型数组。 • 在zgttrs中为双精度复数型数组。	为U的第二超对角元素。	输入

 版权所有 © 华为技术有限公司

参数名	类型	描述	输入/输出
IPIV	整数型数组	置换数组。	输入
B	• 在sgttrs中为单精度浮点型数组。 • 在dgttrs中为双精度浮点型数组。 • 在cgttrs中为单精度复数型数组。 • 在zgttrs中为双精度复数型数组。	右端项矩阵，维度为(LDB, NRHS)。	输入、输出
LDB	整数型	矩阵B的主维。	输入
INFO	整数型	• 等于0：成功退出。 • 小于0：第-info个参数值不合法。	输出

依赖

include "klapack.h"

示例

C Interface：

```
const char trans = 'N';
const int n = 4;
const int ldb = n;
const int nrhs = 1;
double dl[] = {0.565217, 0.170949, 0.482378};
double d[] = {0.923077, 0.899954, 0.318932, -0.186088};
double du[] = {0.043478, -0.085316, 0.566038};
double du2[] = {0.150943, 0.000000};
double b[] = {0.301370, 0.835616, 0.109589, 0.041096};
int ipiv[] = {2,2,3,4};
int info = 0;
dgttrs_(&trans, &n, &nrhs, dl, d, du, du2, ipiv, b, &ldb, &info);
if (info != 0) {
    printf("ERROR, info = %d\n", info);
}

/*
* Output:
*    b:  0.882521        -0.172061        0.188567        0.138984
*/
```

Fortran Interface：

```
CHARACTER :: trans = "N"
PARAMETER (n = 4)
PARAMETER (ldb = 4)
PARAMETER (nrhs = 1)
INTEGER :: info = 0
REAL(8) :: d(n)
REAL(8) :: dl(n-1)
```

 版权所有 © 华为技术有限公司

```
REAL(8) :: du(n-1)
REAL(8) :: du2(n-2)
REAL(8) :: b(ldb, nrhs)
INTEGER :: ipiv(n)

DATA d / 0.923077, 0.899954, 0.318932, -0.186088 /
DATA dl / 0.565217, 0.170949, 0.482378/
DATA du / 0.043478, -0.085316, 0.566038 /
DATA du2 / 0.150943, 0.000000 /
DATA b / 0.301370, 0.835616, 0.109589, 0.041096 /
DATA ipiv / 2,2,3,4 /
 EXTERNAL DGTTRS
 CALL DGTTRS(trans, n, nrhs, dl, d, du, du2, ipiv, b, ldb, info);
*
* Output:
*   d:  0.882521        -0.172061       0.188567        0.138984
```

6.5.1.3.5.4 ?gtts2

求解三对角方程A*X=B或A**T *X=B或A**H *X=B，其中系数矩阵A由?gttrf分解而来。

接口定义

C Interface：

sgtts2_(const int *trans, const int *n, const int *nrhs, const float *dl, const float *d, const float *du, const float *du2, const int *ipiv, float *b, const int *ldb);

dgtts2_(const int *trans, const int *n, const int *nrhs, const double *dl, const double *d, const double *du, const double *du2, const int *ipiv, double *b, const int *ldb);

cgtts2_(const int *trans, const int *n, const int *nrhs, const float _Complex *dl, const float _Complex *d, const float _Complex *du, const float _Complex *du2, const int *ipiv, float _Complex *b, const int *ldb);

zgtts2_(const int *trans, const int *n, const int *nrhs, const double _Complex *dl, const double _Complex *d, const double _Complex *du, const double _Complex *du2, const int *ipiv,double _Complex *b, const int *ldb);

Fortran Interface：

SGTTS2(TRANS, N, NRHS, DL, D, DU, DU2, IPIV, B, LDB);

DGTTS2(TRANS, N, NRHS, DL, D, DU, DU2, IPIV, B, LDB);

CGTTS2(TRANS, N, NRHS, DL, D, DU, DU2, IPIV, B, LDB);

ZGTTS2(TRANS, N, NRHS, DL, D, DU, DU2, IPIV, B, LDB);

参数

参数名	类型	描述	输入/输出
TRANS	字符型	• 'N'：A*X=B。 • 'T'：A**T*X=B。 • 'C'：A**T*X=B（共轭转置）。	输入

 版权所有 © 华为技术有限公司

参数名	类型	描述	输入/输出
N	整数型	矩阵A的阶数，N ≥ 0。	输入
NRHS	整数型	矩阵B的列数，NRHS ≥ 0。	输入
DL	• 在sgtts2中为单精度浮点型数组。 • 在dgtts2中为双精度浮点型数组。 • 在cgtts2中为单精度复数型数组。 • 在zgtts2中为双精度复数型数组。	LU分解中的L部分，维度为N-1。	输入
D	• 在sgtts2中为单精度浮点型数组。 • 在dgtts2中为双精度浮点型数组。 • 在cgtts2中为单精度复数型数组。 • 在zgtts2中为双精度复数型数组。	U的对角元素，维度为N。	输入
DU	• 在sgtts2中为单精度浮点型数组。 • 在dgtts2中为双精度浮点型数组。 • 在cgtts2中为单精度复数型数组。 • 在zgtts2中为双精度复数型数组。	为U的第一超对角元素。	输入
DU2	• 在sgtts2中为单精度浮点型数组。 • 在dgtts2中为双精度浮点型数组。 • 在cgtts2中为单精度复数型数组。 • 在zgtts2中为双精度复数型数组。	为U的第二超对角元素。	输入
IPIV	整数型数组	置换数组。	输入

版权所有 © 华为技术有限公司

参数名	类型	描述	输入/输出
B	• 在sgtts2中为单精度浮点型数组。 • 在dgtts2中为双精度浮点型数组。 • 在cgtts2中为单精度复数型数组。 • 在zgtts2中为双精度复数型数组。	右端项矩阵，维度为(LDB, NRHS)。	输入、输出
LDB	整数型	矩阵B的主维。	输入

依赖

include "klapack.h"

示例

C Interface：

```
const char trans = 'N';
const int n = 4;
const int ldb = n;
const int nrhs = 1;
double dl[] = {0.565217, 0.170949, 0.482378};
double d[] = {0.923077, 0.899954, 0.318932, -0.186088};
double du[] = {0.043478, -0.085316, 0.566038};
double du2[] = {0.150943, 0.000000};
double b[] = {0.301370, 0.835616, 0.109589, 0.041096};
int ipiv[] = {2,2,3,4};

dgtts2_(&trans, &n, &nrhs, dl, d, du, du2, ipiv, b, &ldb);

/*
* Output:
*    b:  0.882521        -0.172061        0.188567        0.138984
*/
```

Fortran Interface：

```
CHARACTER :: trans = "N"
PARAMETER (n = 4)
PARAMETER (ldb = 4)
PARAMETER (nrhs = 1)
REAL(8) :: d(n)
REAL(8) :: dl(n-1)
REAL(8) :: du(n-1)
REAL(8) :: du2(n-2)
REAL(8) :: b(ldb, nrhs)
INTEGER :: ipiv(n)

DATA d / 0.923077, 0.899954, 0.318932, -0.186088 /
DATA dl / 0.565217, 0.170949, 0.482378/
DATA du / 0.043478, -0.085316, 0.566038 /
DATA du2 / 0.150943, 0.000000 /
DATA b / 0.301370, 0.835616, 0.109589, 0.041096 /
DATA ipiv / 2,2,3,4 /
 EXTERNAL DGTTS2
```

 版权所有 © 华为技术有限公司

```
 CALL DGTTS2(trans, n, nrhs, dl, d, du, du2, ipiv, b, ldb)
*
* Output:
*    d:  0.882521        -0.172061        0.188567        0.138984
```

6.5.1.3.5.5 ?trtrs

求解三角方程A * X = B，或A**T * X = B。

接口定义

C Interface：

void dtrtrs_(const char *uplo, const char *trans, const char *diag, const int *n, const int *nrhs, const double *a, const int *lda, double *b, const int *ldb, int *info);

void strtrs_(const char *uplo, const char *trans, const char *diag, const int *n, const int *nrhs, const float *a, const int *lda, float *b, const int *ldb, int *info);

void ctrtrs_(const char *uplo, const char *trans, const char *diag, const int *n, const int *nrhs, const float _Complex *a, const int *lda, float _Complex *b, const int *ldb, int *info);

void ztrtrs_(const char *uplo, const char *trans, const char *diag, const int *n, const int *nrhs, const double _Complex *a, const int *lda, double _Complex *b, const int *ldb, int *info);

Fortran Interface：

DTRTRS(UPLO, TRANS, DIAG, N, NRHS, A, LDA, B, LDB, INFO);

STRTRS(UPLO, TRANS, DIAG, N, NRHS, A, LDA, B, LDB, INFO);

CTRTRS(UPLO, TRANS, DIAG, N, NRHS, A, LDA, B, LDB, INFO);

ZTRTRS(UPLO, TRANS, DIAG, N, NRHS, A, LDA, B, LDB, INFO);

参数

参数名	类型	描述	输入/输出
UPLO	字符型	• 'U'，A是上三角矩阵。 • 'L'，A是下三角矩阵。	输入
TRANS	字符型	• 'N'，A * X = B • 'T'，A**T * X = B • 'C'，A**H * X = B（共轭转置）	输入
DIAG	字符型	• 'N'，A是非单位三角矩阵。 • 'U'。A是单位三角矩阵。	输入
N	整数型	矩阵A的阶数，N ≥ 0。	输入
NRHS	整数型	矩阵B的列数，NRHS ≥ 0。	输入

版权所有 © 华为技术有限公司

参数名	类型	描述	输入/输出
A	• 在strtrs中为单精度浮点型数组。 • 在dtrtrs中为双精度浮点型数组。 • 在ctrtrs中为单精度复数型数组。 • 在ztrtrs中为双精度复数型数组。	三角矩阵A，大小为(LDA, N)。	输入
LDA	整数型	矩阵A的主维，LDA ≥ max(1, N)。	输入
B	• 在strtrs中为单精度浮点型数组。 • 在dtrtrs中为双精度浮点型数组。 • 在ctrtrs中为单精度复数型数组。 • 在ztrtrs中为双精度复数型数组。	输入时，为右端矩阵B，大小为(LDB, NRHS)。 输出时， 若INFO = 0，为方程的解矩阵X。	输入、输出
LDB	整数型	右端矩阵B的主维， LDB ≥ max(1, N)。	输入
INFO	整数型	• 等于0：成功。 • 小于0：第-INFO个参数值不合法。 • 大于0：矩阵A的第INFO个对角元素为0，为奇异矩阵。	输出

依赖

include "klapack.h"

示例

C Interface：

```
const char uplo = 'L';
const char trans = 'N';
const char diag = 'N';
const int n = 4;
const int lda = n;
const int ldb = n;
const int nrhs = 1;

double *a = (double*)malloc(lda * n * sizeof(double));
double *b = (double*)malloc(ldb * nrhs * sizeof(double));
```

 版权所有 © 华为技术有限公司

```
int info = 0;

// Generate Matrix A.
for (int i = 0; i < n; i++) {
    for (int j = i; j < lda; j++) {
        a[i * lda + j] = (rand() % 53 + 1) / 53.0;
    }
}
// Generate right hand side matrix B.
for (int i = 0; i < ldb * nrhs; i++) {
    b[i] = (rand() % 23 + 1) / 23.0;
}

// Call dtrtrs.
dtrtrs_(&uplo, &trans, &diag, &n, &nrhs, a, &lda, b, &ldb, &info);
if (info != 0) {
    printf("ERROR, info = %d\n", info);
}
```

Fortran Interface：

```
CHARACTER :: uplo = "L"
CHARACTER :: trans = "N"
CHARACTER :: diag = "N"
PARAMETER (n = 4)
PARAMETER (lda = 4)
PARAMETER (ldb = 4)
PARAMETER (nrhs = 1)
INTEGER :: info = 0
REAL(8) :: d(lda, n)
REAL(8) :: b(ldb, nrhs)

 EXTERNAL DTRTRS
 CALL DTRTRS(uplo, trans, diag, n, nrhs, d, lda, b, ldb, info)
```

6.5.1.3.5.6 ?getrs

求解一般线性方程A*X=B或A**T * X=B或A**H * X=B。其中系数矩阵A由?getrf分解而来。

接口定义

C Interface：

void sgetrs_(const char *TRANS, const int *N, const int *NRHS, float *A, const int *LDA, int *IPIV, float *B, const int *LDB, int *INFO);

void dgetrs_(const char *TRANS, const int *N, const int *NRHS, double *A, const int *LDA, int *IPIV, double *B, const int *LDB, int *INFO);

void cgetrs_(const char *TRANS, const int *N, const int *NRHS, float _Complex *A, const int *LDA, int *IPIV, float _Complex *B, const int *LDB, int *INFO);

void zgetrs_(const char *TRANS, const int *N, const int *NRHS, double _Complex *A, const int *LDA, int *IPIV, double _Complex *B, const int *LDB, int *INFO);

Fortran Interface：

SGETRS(TRANS, N, NRHS, A, LDA, IPIV, B, LDB, INFO);

DGETRS(TRANS, N, NRHS, A, LDA, IPIV, B, LDB, INFO);

CGETRS(TRANS, N, NRHS, A, LDA, IPIV, B, LDB, INFO);

ZGETRS(TRANS, N, NRHS, A, LDA, IPIV, B, LDB, INFO);

 版权所有 © 华为技术有限公司

参数

参数名	类型	描述	输入/输出
TRANS	字符型	● 'N'，A*X=B。 ● 'T'，A**T*X=B。 ● 'C'，A**H*X=B（共轭转置）。	输入
N	整数型	矩阵A的阶数，N ≥ 0。	输入
NRHS	整数型	矩阵B的列数，NRHS ≥ 0。	输入
A	● 在sgetrs中为单精度浮点型数组。 ● 在dgetrs中为双精度浮点型数组。 ● 在cgetrs中为单精度复数型数组。 ● 在zgetrs中为双精度复数型数组。	矩阵A的L和U部分，来自GETRF分解而来。大小为LDA*N。	输入
LDA	整数型	矩阵A的主维，LDA ≥ max(1,N)。	输入
IPIV	整数型数组	置换数组。	输入
B	● 在sgetrs中为单精度浮点型数组 ● 在dgetrs中为双精度浮点型数组 ● 在cgetrs中为单精度复数型数组 ● 在zgetrs中为双精度复数型数组	右端项矩阵，大小为LDB*NRHS。 ● 调用前：右端项矩阵B。 ● 调用后：解矩阵X。	输入、输出
LDB	整数型	矩阵B的主维，LDB ≥ max(1, N)。	输入
INFO	整数型	● 等于0：成功退出。 ● 小于0：第-info个参数值不合法。	输出

依赖

include "klapack.h"

示例

C Interface：

 版权所有 © 华为技术有限公司

```
char trans = 'N';
int n = 5;
int nrhs = 1;
int lda = 5;
int ldb = 5;
int ipiv[5];
int info = 0;
double a[]={72.1673 , 66.1857 , 64.7644 , 28.0199 , 91.4151,
            6.5180 , 62.8483 , 72.4323 , 46.5760 ,  8.6928,
            28.9821 , 42.1828 , 18.6437 , 99.8612 , 35.6972,
            67.9812 ,  5.0880 , 85.5035 ,79.2945 , 54.5920,
            28.6869 , 49.7512 ,  7.5186 ,28.6929 , 84.6041};
double b[]={1.0, 2.0, 3.0, 4.0, 5.0};
dgetrf_(&n, &n, a, &lda, ipiv, &info);
dgetrs_(&trans, &n, &nrhs, a, &lda, ipiv, b, &ldb, &info);
/*
* Output: -0.073465    0.038136    -0.033917    0.055903    0.112798
*/
```

Fortran Interface：

```
CHARACTER::trans='N'
PARAMETER(n=5)
PARAMETER(nrhs=1)
PARAMETER(lda=5)
PARAMETER(ldb=5)
INTEGER::info =0
INTEGER :: ipiv(n)
REAL(8) :: a(n*n)
       DATA a / 72.1673 , 66.1857 , 64.7644 , 28.0199 , 91.4151,
     $          6.5180 , 62.8483 , 72.4323 , 46.5760 ,  8.6928,
     $          28.9821 , 42.1828 , 18.6437 , 99.8612 , 35.6972,
     $          67.9812 ,  5.0880 , 85.5035 ,79.2945 , 54.5920,
     $          28.6869 , 49.7512 ,  7.5186 ,28.6929 , 84.6041 /
 REAL(8) :: b(n, nrhs)
 DATA b / 1.0, 2.0, 3.0, 4.0, 5.0 /
 EXTERNAL DGETRF, DGETRS
 CALL DGETRF(n, n, a, lda, ipiv, info);
 CALL DGETRS(trans, n, nrhs, a, lda, ipiv, b, ldb, info);
* Output: -0.073465    0.038136    -0.033917    0.055903    0.112798
```

6.5.1.3.5.7 ?pptrs

求解三对角方程A*X=B或A**T *X=B或A**H *X=B。其中系数矩阵A由?gttrf分解而来。

接口定义

C Interface：

void spptrs_(const char *UPLO, const int *N, const int *NRHS, float *AP, float *B, const int *LDB, int *INFO);

void dpptrs_(const char *UPLO, const int *N, const int *NRHS, double *AP, double *B, const int *LDB, int *INFO);

void cpptrs_(const char *UPLO, const int *N, const int *NRHS, float _Complex *AP, float _Complex *B, const int *LDB, int *INFO);

void zpptrs_(const char *UPLO, const int *N, const int *NRHS, double _Complex *AP, double _Complex *B, const int *LDB, int *INFO);

Fortran Interface：

SPPTRS(UPLO, N, NRHS, AP, B, LDB, INFO);

DPPTRS(UPLO, N, NRHS, AP, B, LDB, INFO);

 版权所有 © 华为技术有限公司

CPPTRS(UPLO, N, NRHS, AP, B, LDB, INFO);

ZPPTRS(UPLO, N, NRHS, AP, B, LDB, INFO);

参数

参数名	类型	描述	输入/输出
UPLO	字符型	• 'U'：存储矩阵A的上三角部分。 • 'T'：存储矩阵A的下三角部分。	输入
N	整数型	矩阵A的阶数，N ≥ 0。	输入
NRHS	整数型	矩阵B的列数，NRHS ≥ 0。	输入
AP	• 在spptrs中为单精度浮点型数组。 • 在dpptrs中为双精度浮点型数组。 • 在cpptrs中为单精度复数型数组。 • 在zpptrs中为双精度复数型数组。	压缩存储矩阵A的Cholesky分解得到的L和U。大小为N*(N+1)/2。	输入
B	• 在spptrs中为单精度浮点型数组。 • 在dpptrs中为双精度浮点型数组。 • 在cpptrs中为单精度复数型数组。 • 在zpptrs中为双精度复数型数组。	右端项矩阵，大小为LDB*NRHS。 • 调用前：右端项矩阵B。 • 调用后：解矩阵X。	输入、输出
LDB	整数型	矩阵B的主维，LDB ≥ max(1, N)。	输入
INFO	整数型	• 等于0：成功退出。 • 小于0：第-info个参数值不合法。	输出

依赖

include "klapack.h"

示例

C Interface：

 版权所有 © 华为技术有限公司

```
const char uplo = 'U';
const int n = 4;
const int nrhs = 1;
const int ldb = n;
double ap[] = {231.8009,33.9545,162.2902,9.4143,6.6684,100.322,11.1156,53.2033,71.5384,106.5638};
int info = 0;
double b[]={1.0, 2.0, 3.0, 4.0};
dpptrf_(&uplo, &n, ap, &info);
dpptrs_(&uplo, &n, &nrhs, ap, b, &ldb, &info);
if (info != 0) {
    printf("ERROR, info = %d\n", info);
}

/*
* Output:
*   ap: 0.002361    0.000904    0.006432    0.032521
*/
```

Fortran Interface：

```
CHARACTER::uplo = "U"
PARAMETER (n = 4)
PARAMETER (nrhs = 4)
PARAMETER (ldb = n)
INTEGER :: info = 0
REAL(8) :: ap(n*(n+1)/2)

DATA ap / 231.8009,33.9545,162.2902,9.4143,6.6684,100.322,11.1156,53.2033,71.5384,106.5638 /
DATA b / 1.0, 2.0, 3.0, 4.0 /
 EXTERNAL DPPTRF,DPPTRS
 CALL DPPTRF(uplo, n, ap, info);
 CALL DPPTRS(uplo, n, nrhs, ap, b, ldb, info);
*
* Output:
*   ap: 0.002361    0.000904    0.006432    0.032521
```

6.5.1.3.6 特征值问题函数

6.5.1.3.6.1 ?sy(he)evd

计算对称（Hermite）矩阵的全部特征值和特征向量（可选）。其中特征向量通过分治算法计算。

即矩阵A分解为：$A = Z\Lambda Z^T$，其中Λ为对角矩阵，对角线元素λ_i为特征值；Z为正交矩阵，其每列向量z_i为对应的特征向量。即$Az_i = \lambda_i z_i$，其中$i = 1,2,...,n$。

接口定义

C Interface：

void dsyevd_(const char *jobz, const char *uplo, const int *n, double *a, const int *lda, double *w, double *work, const int *lwork, int *iwork, const int *liwork, int *info);

void ssyevd_(const char *jobz, const char *uplo, const int *n, float *a, const int *lda, float *w, float *work, const int *lwork, int *iwork, const int *liwork, int *info);

void cheevd_(const char *jobz, const char *uplo, const int *n, float _Complex *a, const int *lda, float *w, float _Complex *work, const int *lwork, float *rwork, const int *lrwork, int *iwork, const int *liwork, int *info);

 版权所有 © 华为技术有限公司

void zheevd_(const char *jobz, const char *uplo, const int *n, double _Complex *a, const int *lda, double *w, double _Complex *work, const int *lwork, double *rwork, const int *lrwork, int *iwork, const int *liwork, int *info);

Fortran Interface：

DSYEVD(JOBZ, UPLO, N, A, LDA, W, WORK, LWORK, IWORK, LIWORK, INFO);

SSYEVD(JOBZ, UPLO, N, A, LDA, W, WORK, LWORK, IWORK, LIWORK, INFO);

CHEEVD(JOBZ, UPLO, N, A, LDA, W, WORK, LWORK, RWORK, LRWORK, IWORK, LIWORK, INFO);

ZHEEVD(JOBZ, UPLO, N, A, LDA, W, WORK, LWORK, RWORK, LRWORK, IWORK, LIWORK, INFO);

参数

参数名	类型	描述	输入/输出
jobz	字符型	● 'N'：只计算特征值。 ● 'V'：同时计算特征向量和特征值。	输入
uplo	字符型	● 'U'：A中保存上三角矩阵。 ● 'L'：A中保存下三角矩阵。	输入
n	整数型	对称矩阵A的行数或列数。	输入
a	● 在ssyevd中为单精度浮点型数组。 ● 在dsyevd中为双精度浮点型数组。 ● 在cheevd中为单精度复数型数组。 ● 在zheevd中为双精度复数型数组。	● 调用前保存待分解的对称矩阵。 ● 调用后保存特征向量。	输入/输出
lda	整数型	A的leading dimension大小，要求lda ≥max(1, n)。	输入
w	● 在ssyevd/cheevd中为单精度浮点型数组。 ● 在dsyevd/zheevd中为双精度浮点型数组。	按升序排列的特征值，长度为n。	输出

 版权所有 © 华为技术有限公司

参数名	类型	描述	输入/输出
work	• 在ssyevd中为单精度浮点型数组。 • 在dsyevd中为双精度浮点型数组。 • 在cheevd中为单精度复数型数组。 • 在zheevd中为双精度复数型数组。	临时存储空间，调用后work[0]为最优的lwork值。	输出
lwork	整数型	work数组的长度。 lwork=-1时查询最优work大小，结果保存在work[0]中，否则： • 当n ≤ 1时，lwork ≥ 1。 • 当jobz='N'且n>1时，lwork ≥ 2*n+1。 • 当jobz='V'且n>1时，lwork ≥ 1+6*n+2*n*n。	输入
rwork（复数类型特有）	• 在cheevd中为单精度实数。 • 在zheevd中为双精度实数。	临时存储空间，调用后work[0]为最优的lwork值。	输出
lrwork（复数类型特有）	• 在cheevd中为单精度复数型数组。 • 在zheevd中为双精度复数型数组。	rwork数组的长度。 lrwork=-1时查询最优work大小，结果保存在rwork[0]中，否则： • 当n ≤ 1时，lwork ≥ 1。 • 当jobz='N'且n>1时，lwork ≥ 2*n+1。 • 当jobz='V'且n>1时，lwork ≥ 1+6*n+2*n*n。	输入
iwork	整数型数组	临时存储空间，使用lwork=-1调用后iwork[0]为最优的liwork值。	输出

版权所有 © 华为技术有限公司

参数名	类型	描述	输入/输出
liwork	整数型	iwork数组的长度。 liwork=-1时查询最优iwork大小，结果保存在iwork[0]中，否则： ● 当jobz='N'或n ≤ 1时，liwork ≥ 1。 ● 当jobz='V'且n>1时，liwork ≥ 3+5*n。	输入
info	整数型	执行结果： ● 等于0：成功。 ● 小于0：第-info个参数值不合法。 ● 大于0：jobz='N'算法无法收敛，无法计算第info/(n+1)个特征值。	输出

依赖

#include "klapack.h"

示例

C Interface：

```
char jobz = 'V';
char uplo = 'L';
int n = 5;
int lda = 5;
int info = 0;
double w[5];
double *work = NULL;
double qwork;
int lwork = -1;
int *iwork = NULL;
int qiwork;
int liwork = -1;
/*
 * Symmetric A (stored in column-major):
 *   7.027  8.710  1.015  6.929  7.584
 *   8.710  0.839  2.469  3.850  0.559
 *   1.015  2.469  1.930  6.761  7.207
 *   6.929  3.850  6.761  4.344  4.804
 *   7.584  0.559  7.207  4.804  6.177
 */
double a[] = {7.027, 8.710, 1.015, 6.929, 7.584,
              8.710, 0.839, 2.469, 3.850, 0.559,
              1.015, 2.469, 1.930, 6.761, 7.207,
              6.929, 3.850, 6.761, 4.344, 4.804,
              7.584, 0.559, 7.207, 4.804, 6.177};
/* Query optimal work size */
dsyevd_(&jobz, &uplo, &n, a, &lda, w, &qwork, &lwork, &qiwork, &liwork, &info);
if (info != 0) {
    return ERROR;
```

 版权所有 © 华为技术有限公司

```
}
lwork = (int)qwork;
work = (double *)malloc(sizeof(double) * lwork);
liwork = (int)qiwork;
iwork = (int *)malloc(sizeof(int) * liwork);
/* Calculate eigenvalues and eigenvectors */
dsyevd_(&jobz, &uplo, &n, a, &lda, w, work, &lwork, iwork, &liwork, &info);
free(work);
free(iwork);
/*
 * Output:
 * Eigenvalues (in w)
 *  -8.8422 -3.3411 1.1888 6.2050 25.1065
 * Eigenvectors (in a, stored in column-major)
 *   0.5405  -0.1616  -0.3059  -0.5235  -0.5604
 *  -0.4913   0.5975   0.2432  -0.4882  -0.3228
 *   0.4886   0.4989   0.3454   0.5213  -0.3482
 *  -0.2410  -0.6062   0.5831   0.1038  -0.4729
 *  -0.4120   0.0219  -0.6228   0.4528  -0.4867
 */
```

Fortran Interface:

```
      CHARACTER :: jobz = "V"
      CHARACTER :: uplo = "L"
      PARAMETER (n = 5)
      PARAMETER (lda = 5)
      INTEGER :: info = 0
      REAL(8) :: w(5);
      REAL(8) :: qwork(1)
      REAL(8), ALLOCATABLE :: work(:)
      INTEGER :: lwork = -1
      INTEGER :: qiwork(1)
      REAL(8), ALLOCATABLE :: iwork(:)
      INTEGER :: liwork = -1

*     Symmetric A (stored in column-major):
*       7.027  8.710  1.015  6.929  7.584
*       8.710  0.839  2.469  3.850  0.559
*       1.015  2.469  1.930  6.761  7.207
*       6.929  3.850  6.761  4.344  4.804
*       7.584  0.559  7.207  4.804  6.177
      REAL(8) :: a(n, n)
      DATA a / 7.027, 8.710, 1.015, 6.929, 7.584,
     $         8.710, 0.839, 2.469, 3.850, 0.559,
     $         1.015, 2.469, 1.930, 6.761, 7.207,
     $         6.929, 3.850, 6.761, 4.344, 4.804,
     $         7.584, 0.559, 7.207, 4.804, 6.177 /
*     Query optimal work size
      EXTERNAL DSYEVD
      CALL DSYEVD(jobz, uplo, n, a, lda, w, qwork, lwork, qiwork,
     $            liwork, info)
      IF (info.NE.0) THEN
         CALL EXIT(1)
      END IF
      lwork = INT(qwork(1))
      liwork = INT(qiwork(1))
      ALLOCATE(work(lwork), iwork(liwork))
*     Calculate eigenvalues and eigenvectors
      CALL DSYEVD(jobz, uplo, n, a, lda, w, work, lwork, iwork,
     $            liwork, info)
      DEALLOCATE(work, iwork);

*     Output:
*     Eigenvalues (in w)
*      -8.8422 -3.3411 1.1888 6.2050 25.1065
*     Eigenvectors (in a, stored in column-major)
*       0.5405  -0.1616  -0.3059  -0.5235  -0.5604
*      -0.4913   0.5975   0.2432  -0.4882  -0.3228
*       0.4886   0.4989   0.3454   0.5213  -0.3482
```


版权所有 © 华为技术有限公司

```
*       -0.2410  -0.6062   0.5831   0.1038  -0.4729
*       -0.4120   0.0219  -0.6228   0.4528  -0.4867
```

6.5.1.3.6.2 ?sy(he)ev

计算对称（Hermite）矩阵的所有特征值与特征向量。

接口定义

C Interface：

ssyev_(const char *jobz, const char *uplo, const int *n, float *A, const int *lda, float *w, float *work, const int *lwork, int * info);

dsyev_(const char *jobz, const char *uplo, const int *n, double *A, const int *lda, double *w, double *work, const int *lwork, int * info);

cheev_(const char *jobz, const char *uplo, const int *n, float _Complex *A, const int *lda, float *w, float _Complex *work, const int *lwork, float* rwork, int * info);

zheev_(const char *jobz, const char *uplo, const int *n, double _Complex *A, const int *lda, double *w, double _Complex *work, const int *lwork, double* rwork, int * info);

Fortran Interface：

SSYEV(jobz, uplo, n, a, lda, w, work, lwork, info);

DSYEV(jobz, uplo, n, a, lda, w, work, lwork, info);

CHEEV(jobz, uplo, n, a, lda, w, work, lwork, rwork, info);

ZHEEV(jobz, uplo, n, a, lda, w, work, lwork, rwork, info);

参数

参数	类型	说明	输入/输出
jobz	字符型	● ‘N’：只计算特征值。 ● ‘V’：计算特征值及特征向量。	输入
uplo	字符型	● ‘U’：存储A的上三角部分。 ● ‘L’：存储A的下三角部分。	输入
n	整数型	矩阵A的维度，大于或等于0。	输入

 版权所有 © 华为技术有限公司

参数	类型	说明	输入/输出
a	• 在ssyev中为单精度浮点型数组。 • 在dsyev中为双精度浮点型数组。 • 在cheev中为单精度复数型数组。 • 在zheev中为双精度复数型数组。	• 调用前保存待分解的对称矩阵。 • 调用后保存特征向量。	输入，输出
lda	整数型	矩阵A的主维，大于或等于max(1, N)。	输入
w	• 在ssyev/cheev中为单精度浮点型数组。 • 在dsyev/zheev中为双精度浮点型数组。	Info=0时，以升序存储特征值，长度为N。	输出
work	• 在ssyev中为单精度浮点型数组。 • 在dsyev中为双精度浮点型数组。 • 在cheev中为单精度复数型数组。 • 在zheev中为双精度复数型数组。	Info=0时，work(0)返回最优lwork大小。	输出
lwork	整数型	work数组的长度。 lwork=-1时查询最优work大小，结果保存在work[0]中。且lwork ≥ max(1, 2*n-1)。	输入
rwork（复数类型特有）	• 在ssyev/cheev中为单精度浮点型数组。 • 在dsyev/zheev中为双精度浮点型数组。	工作数组，取值max(1, 3*n-1)。	输出
info	整数型	状态值： • 等于0：表示成功。 • 小于0：第-info个参数值不合法。 • 大于0：算法出错。	输出

依赖

```
#include "klapack.h"
```

 版权所有 © 华为技术有限公司

示例

C Interface：

```
char jobz = 'V';
char uplo = 'L';
int n = 5;
int lda = 5;
int info = 0;
double w[5];
double *work = NULL;
double qwork;
int lwork = -1;
/*
 * Symmetric A (stored in column-major):
 *   7.027  8.710  1.015  6.929  7.584
 *   8.710  0.839  2.469  3.850  0.559
 *   1.015  2.469  1.930  6.761  7.207
 *   6.929  3.850  6.761  4.344  4.804
 *   7.584  0.559  7.207  4.804  6.177
 */
double a[] = {7.027, 8.710, 1.015, 6.929, 7.584,
              8.710, 0.839, 2.469, 3.850, 0.559,
              1.015, 2.469, 1.930, 6.761, 7.207,
              6.929, 3.850, 6.761, 4.344, 4.804,
              7.584, 0.559, 7.207, 4.804, 6.177};
/* Query optimal work size */
dsyev_(&jobz, &uplo, &n, a, &lda, w, &qwork, &lwork, &info);
if (info != 0) {
    return ERROR;
}
lwork = (int)qwork;
work = (double *)malloc(sizeof(double) * lwork);
/* Calculate eigenvalues and eigenvectors */
dsyev_(&jobz, &uplo, &n, a, &lda, w, work, &lwork, &info);
free(work);
/*
 * Output:
 * Eigenvalues (in w)
 *  -1.770722   -0.845606   -0.203058   -5.712058   3.044889
 * Eigenvectors (in a, stored in column-major)
 *  0.547211        -0.528441       0.339953        -0.172302       -0.531245
 *  0.050685        -16.554919      -5.702155       2.261598        15.521155
 *  -0.456110       0.988485        0.587808        -3.161022       -1.099502
 * -0.443428       -1.398420       0.424293        0.800628        0.016423
 * -0.541596       -1.770722       -0.845606       -0.203058       -5.712058

 */
```

Fortran Interface：

```
      CHARACTER :: jobz = "V"
      CHARACTER :: uplo = "L"
      PARAMETER (n = 5)
      PARAMETER (lda = 5)
      INTEGER :: info = 0
      REAL(8) :: w(5);
      REAL(8) :: qwork(1)
      REAL(8), ALLOCATABLE :: work(:)
      INTEGER :: lwork = -1

*     Symmetric A (stored in column-major):
*       7.027  8.710  1.015  6.929  7.584
*       8.710  0.839  2.469  3.850  0.559
*       1.015  2.469  1.930  6.761  7.207
*       6.929  3.850  6.761  4.344  4.804
*       7.584  0.559  7.207  4.804  6.177
      REAL(8) :: a(n, n)
      DATA a / 7.027, 8.710, 1.015, 6.929, 7.584,
```


版权所有 © 华为技术有限公司

```
     $          8.710, 0.839, 2.469, 3.850, 0.559,
     $          1.015, 2.469, 1.930, 6.761, 7.207,
     $          6.929, 3.850, 6.761, 4.344, 4.804,
     $          7.584, 0.559, 7.207, 4.804, 6.177 /
*       Query optimal work size
        EXTERNAL DSYEVD
        CALL DSYEV(jobz, uplo, n, a, lda, w, qwork, lwork, info)
        IF (info.NE.0) THEN
           CALL EXIT(1)
        END IF
        lwork = INT(qwork(1))
        ALLOCATE(work(lwork))
*       Calculate eigenvalues and eigenvectors
        CALL DSYEV(jobz, uplo, n, a, lda, w, work, lwork, info)
        DEALLOCATE(work);

*       Output:
*       Eigenvalues (in w)
*        -1.770722   -0.845606   -0.203058   -5.712058   3.044889
*       Eigenvectors (in a, stored in column-major)
*         0.547211       -0.528441       0.339953       -0.172302       -0.531245
*         0.050685       -16.554919      -5.702155       2.261598        15.521155
*         -0.456110      0.988485        0.587808       -3.161022       -1.099502
*         -0.443428      -1.398420       0.424293       0.800628        0.016423
*         -0.541596      -1.770722       -0.845606      -0.203058       -5.71205
```

6.5.1.3.6.3 ?steqr

使用QL或QR方法求解对称三对角矩阵的特征值和特征向量。

接口定义

C Interface：

ssteqr_(const char *compz, const int *n, float *d, float *e, float *z, const int *ldz, float *work, int *info);

dsteqr_(const char *compz, const int *n, double *d, double *e, double *z, const int *ldz, double *work, int *info);

csteqr_(const char *compz, const int *n, float *d, float *e, float _Complex *z, const int *ldz, float *work, int *info);

zsteqr_(const char *compz, const int *n, double *d, double *e, double _Complex *z, const int *ldz, double *work, int *info);

Fortran Interface：

SSTEQR(COMPZ, N, D, E, Z, LDZ, WORK, INFO);

DSTEQR(COMPZ, N, D, E, Z, LDZ, WORK, INFO);

CSTEQR(COMPZ, N, D, E, Z, LDZ, WORK, INFO);

ZSTEQR(COMPZ, N, D, E, Z, LDZ, WORK, INFO);

 版权所有 © 华为技术有限公司

参数

参数	类型	说明	输入/输出
compz	字符型	• 'N'：只计算特征值。 • 'V'：计算特征值和特征向量。 • 'I'： 计算特征值和特征向量。	输入
n	整数型	矩阵的维度，n≥0。	输入
d	• 在ssteqr中为单精度浮点型数组。 • 在dsteqr中为双精度浮点型数组。 • 在csteqr中为单精度浮点型数组。 • 在zsteqr中为双精度浮点型数组。	长度为n。 • 进入时：三对角矩阵的对角元素。 • 退出时：若info=0，保存特征值，并以升序排列。	输入，输出
e	• 在ssteqr中为单精度浮点型数组。 • 在dsteqr中为双精度浮点型数组。 • 在csteqr中为单精度浮点型数组。 • 在zsteqr中为双精度浮点型数组。	长度为n-1。 • 进入时：三对角矩阵的次对角元素。 • 退出时：被销毁。	输入，输出
z	• 在ssteqr中为单精度浮点型数组。 • 在dsteqr中为双精度浮点型数组。 • 在csteqr中为单精度复数型数组。 • 在zsteqr中为双精度复数型数组。	大小为ldz*n。 • 进入时：若compz='V'时，则为正交矩阵。 • 退出时： – 若info=0，且compz='V'时，则为原对称矩阵的正交特征向量。 – 若compz='I'时，则为对称三对角矩阵的正交特征向量。 – 若compz='N'时，不使用。	输入，输出
ldz	整数型	矩阵z的主维，ldz ≥ max(1, n)。	输入

版权所有 © 华为技术有限公司

参数	类型	说明	输入/输出
work	• 在ssteqr中为单精度浮点型数组。 • 在dsteqr中为双精度浮点型数组。 • 在csteqr中为单精度浮点型数组。 • 在zsteqr中为双精度浮点型数组。	工作数组，大小为max(1, 2*n-2)。当compz='N'时，不使用。	输出
info	整数型	• 等于0：表示成功。 • 小于0：info=-i，表示第i个参数非法。 • 大于0：算法无法在30*n次的迭代中找到所有特征值。	输出

依赖

#include "klapack.h"

示例

C Interface：

```
const char compz = 'N';
const int n = 4;
const int ldz = 4;

int info = 0;

double d[] = {72.1673, 66.1857, 64.7644, 28.0199, 91.4151};
double e[] = {6.8955, 7.2465, 3.5019, 8.2268, 3.5287};
int *ipiv = (int*)malloc(n * sizeof(int));
double b[] = {9.4532, 1.5204, 2.2127, 0.9891, 7.1778,
              6.8955, 7.2465, 3.5019, 8.2268, 3.5287};

double *z = (double*)malloc(ldz * n * sizeof(double));
double *work = (double*)malloc(n * sizeof(double));

dsteqr_(&compz, &n, d, e, z, &ldz, work, &info);
if (info != 0) {
    printf("ERROR, info = %d\n", info);
}
/* output */
* d
* 27.676252       56.816901       68.501999       78.142148
* e
* 0.000000        0.000000        0.000000
```

Fortran Interface：

```
CHARACTER :: compz = "N"
PARAMETER (n = 4)
PARAMETER (ldz = 4)
INTEGER :: info = 0
```


版权所有 © 华为技术有限公司

```
REAL(8) :: d(n)
REAL(8) :: e(n-1)
REAL(8) :: z(ldz, n)
REAL(8) :: work(n)

DATA d / 72.1673, 66.1857, 64.7644, 28.0199 /
DATA e / 6.8955, 7.2465, 3.5019 /
EXTERNAL DSTEQR
 CALL DSTEQR(compz, n, d, e, z, ldz, work, info);
*
* Output:
* d:
* 27.676252       56.816901       68.501999       78.142148
* e:
* 0.000000        0.000000        0.000000
```

6.5.1.3.6.4 ?stedc

对称三对角矩阵的特征值求解，使用Divide-and-Conquer算法。

接口定义

C Interface：

sstedc_(const char *compz, const int *n, float *d, float *e, float *z, const int *ldz, float *work, const int lwork,

int *iwork, const int *liwork, int *info);

dstedc_(const char *compz, const int *n, double *d, double *e, double *z, const int *ldz, double *work, const int lwork,

int *iwork, const int *liwork, int *info);

cstedc_(const char *compz, const int *n, float *d, float *e, float _Complex *z, const int *ldz, float _Complex *work, const int lwork, float *rwork, const int *lrwork,

int *iwork, const int *liwork, int *info);

zstedc_(const char *compz, const int *n, double *d, double *e, double _Complex *z, const int *ldz, double _Complex *work, const int lwork, double *rwork, const int *lrwork,

int *iwork, const int *liwork, int *info);

Fortran Interface：

SSTEDC(COMPZ, N, D, E, Z, LDZ, WORK, LWORK, RWORK, LRWORK, IWORK, LIWORK, INFO);

DSTEDC(COMPZ, N, D, E, Z, LDZ, WORK, LWORK, RWORK, LRWORK, IWORK, LIWORK, INFO);

CSTEDC(COMPZ, N, D, E, Z, LDZ, WORK, LWORK, RWORK, LRWORK, IWORK, LIWORK, INFO);

ZSTEDC(COMPZ, N, D, E, Z, LDZ, WORK, LWORK, RWORK, LRWORK, IWORK, LIWORK, INFO);

 版权所有 © 华为技术有限公司

参数

参数	类型	说明	输入/输出
compz	字符型	• 'N'：只计算特征值。 • 'V'：计算特征值和原Hermitian矩阵的特征向量。 • 'I'：计算特征值和三对角矩阵的特征向量。	输入
n	整数型	矩阵的维度，n≥0	输入
d	• 在sstedc中为单精度浮点型数组 • 在dstedc中为双精度浮点型数组 • 在cstedc中为单精度浮点型数组 • 在zstedc中为双精度浮点型数组	长度为n • 进入时：三对角矩阵的对角元素。 • 退出时：若info=0，保存特征值，并以升序排列。	输入，输出
e	• 在sstedc中为单精度浮点型数组 • 在dstedc中为双精度浮点型数组 • 在cstedc中为单精度浮点型数组 • 在zstedc中为双精度浮点型数组	长度为n-1 • 进入时：三对角矩阵的次对角元素。 • 退出时：被销毁。	输入，输出
z	• 在sstedc中为单精度浮点型数组 • 在dstedc中为双精度浮点型数组 • 在cstedc中为单精度复数型数组 • 在zstedc中为双精度复数型数组	大小为ldz*n • 进入时：若compz='V'时，则为正交矩阵。 • 退出时： – 若info=0，且compz='V'时，则为原对称矩阵的正交特征向量。 – 若compz='I'时，则为对称三对角矩阵的正交特征向量。 – 若compz='N'时，不使用。	输入，输出
ldz	整数型	矩阵z的主维，ldz >=1；若需要求特征向量，则ldz ≥ max(1, n)。	输入

版权所有 © 华为技术有限公司

参数	类型	说明	输入/输出
work	• 在sstedc中为单精度浮点型数组 • 在dstedc中为双精度浮点型数组 • 在cstedc中为单精度复数型数组 • 在zstedc中为双精度复数型数组	• 工作数组，大小为max(1, lwork)。 • 退出时，若info=0, work(1)返回最优lwork大小。	输出
lwork	整数型	work数组的大小 • 若compz='N'或'I'，或n<=1：lwork>=1。 • 若compz='V'且n>1，lwork >= n*n。 • 若lwork=-1，表明函数将只查询最优的lwork大小。	输入
rwork	• 在sstedc中为单精度浮点型数组 • 在dstedc中为双精度浮点型数组 • 在cstedc中为单精度浮点型数组 • 在zstedc中为双精度浮点型数组	• 工作数组，大小为max(1, lrwork). • 退出时，若info=0，rwork(1)返回最优lrwork大小	输出
lrwork	整数型	rwork数组的大小 • 若compz='N'或n<=1：lrwork>=1。 • 若compz='V'且 n>1，lrwork >= 1+3*n+2*n*lgn+4*n**2，这里的lgn满足2**k>=n的最小值。 • 若compz='I'且 n>1，lrwork>=1+4*n+2*n**2。 • 若lrwork=-1，表明函数将只查询最优的lrwork大小。	输入
iwork	整数型	• 工作数组，大小为max(1, liwork)。 • 退出时，若info=0，iwork(1)返回最优liwork大小。	输出

版权所有 © 华为技术有限公司

参数	类型	说明	输入/输出
liwork	整数型	iwork数组的大小 ● 若compz='N'或n<=1：liwork>=1。 ● 若compz='V'或n>1，liwork >= 6+6*n+5*n*lgn。 ● 若compz='I'或n>1，liwork>=3+5*n。 ● 若liwork=-1，表明函数将只查询最优的liwork大小。	输入
info	整数型	● 等于0：表示成功。 ● 小于0：info=-i，表示第i个参数非法。 ● 大于0：算法在处理位于INFO/(N+1)到mod(INFO,N+1)行、列中的子矩阵时无法计算特征值。	输出

依赖

#include "klapack.h"

示例

C Interface：

```
const char compz = 'N';
const int n = 4;
const int ldz = 4;
double d[] = {72.1673, 66.1857, 64.7644, 28.0199};
double e[] = {6.8955, 7.2465, 3.5019};
double z[] = {0.924528, 0.056604, 0.415094, 0.924528,
              0.622642, 0.905660, 0.150943, 0.471698,
              0.132075, 0.566038, 0.811321, 0.924528,
              0.339623, 0.283019, 0.679245, 0.283019};
int lwork = 1;
int liwork = 1;
double *work = (double*)malloc(lwork * sizeof(double));
int *iwork = (int*)malloc(liwork * sizeof(int));

int info = 0;

dstedc_(&compz, &n, d, e, z, &ldz, work, &lwork, iwork, &liwork, &info);
if (info != 0) {
    printf("ERROR, info = %d\n", info);
}
/* output */
* d
* 27.676252       56.816901       68.501999       78.142148
* e
* 0.000000        0.000000        0.000000
* z
* 0.924528        0.056604        0.415094        0.924528        0.622642        0.905660        0.150943
0.471698        0.132075        0.566038        * 0.811321        0.924528        0.339623        0.283019
0.679245        0.283019
```


版权所有 © 华为技术有限公司

Fortran Interface：

```
CHARACTER :: compz = "N"
PARAMETER (n = 4)
PARAMETER (ldz = 4)
PARAMETER (lwork = 1)
PARAMETER (liwork = -1)
INTEGER :: info = 0
REAL(8) :: d(n)
REAL(8) :: e(n-1)
REAL(8) :: z(ldz, n)
REAL(8) :: work(lwork)
INTEGER :: iwork(liwork)

DATA d / 72.1673, 66.1857, 64.7644, 28.0199 /
DATA e / 6.8955, 7.2465, 3.5019 /
DATA z / 0.924528, 0.056604, 0.415094, 0.924528,
        0.622642, 0.905660, 0.150943, 0.471698,
        0.132075, 0.566038, 0.811321, 0.924528,
        0.339623, 0.283019, 0.679245, 0.283019/
EXTERNAL DSTEDC
 CALL DSTEDC(compz, n, d, e, z, ldz, work, liwork, iwork, liwork, info);
*
* Output:
* d:
* 27.676252       56.816901       68.501999       78.142148
* e:
* 0.000000        0.000000        0.000000
* z:
* 0.924528        0.056604        0.415094        0.924528        0.622642        0.905660        0.150943
0.471698        0.132075        0.566038        * 0.811321        0.924528        0.339623        0.283019
0.679245        0.283019
```

6.5.1.3.6.5 ?sy(he)evx

计算对称（Hermite）矩阵规定范围内的特征值和特征向量（可选）。

接口定义

C Interface:

ssyevx_(const char *jobz, const char *range, const char *uplo, const int *n, float *a, const int *lda, const float *vl, const float *vu, const int *il, const int *iu, const float *abstol, int *m, float *w, float *z,

const int *ldz, float *work, const int *lwork, int *iwork, int *ifail, int *info);

dsyevx_(const char *jobz, const char *range, const char *uplo, const int *n, double *a, const int *lda, const double *vl, const double *vu, const int *il, const int *iu, const double *abstol, int *m, double *w,

double *z, const int *ldz, double *work, const int *lwork, int *iwork, int *ifail, int *info);

cheevx_(const char *jobz, const char *range, const char *uplo, const int *n, kml_complex_float *a, const int *lda, const float *vl, const float *vu, const int *il, const int *iu, const float *abstol, int *m,

float *w, kml_complex_float *z, const int *ldz, kml_complex_float *work, const int *lwork, float *rwork, int *iwork, int *ifail, int *info);

zheevx_(const char *jobz, const char *range, const char *uplo, const int *n, kml_complex_double *a, const int *lda, const double *vl, const double *vu, const int *il, const int *iu, const double *abstol, int *m,

 版权所有 © 华为技术有限公司

double *w, kml_complex_double *z, const int *ldz, kml_complex_double *work, const int *lwork, double *rwork, int *iwork, int *ifail, int *info);

Fortran Interface:

SSYEVX(JOBZ, RANGE, UPLO, N, A, LDA, VL, VU, IL, IU, ABSTOL, M, W, Z, LDZ, WORK, LWORK, IWORK, IFAIL, INFO);

DSYEVX(JOBZ, RANGE, UPLO, N, A, LDA, VL, VU, IL, IU, ABSTOL, M, W, Z, LDZ, WORK, LWORK, IWORK, IFAIL, INFO);

CHEEVX(JOBZ, RANGE, UPLO, N, A, LDA, VL, VU, IL, IU, ABSTOL, M, W, Z, LDZ, WORK, LWORK, RWORK, IWORK, IFAIL, INFO);

ZHEEVX(JOBZ, RANGE, UPLO, N, A, LDA, VL, VU, IL, IU, ABSTOL, M, W, Z, LDZ, WORK, LWORK, RWORK, IWORK, IFAIL, INFO);

参数

参数	类型	说明	输入/输出
jobz	字符型	• N：只计算特征值； • V：计算特征值和特征向量	输入
range	字符型	• A：计算所有特征值； • V：在(vl, vu]范围内的特征值； • I：第il和iu个特征值会被计算	输入
uplo	字符型	• U：矩阵A的上三角部分被存储； • L：矩阵A的下三角部分被存储	输入
n	整数型	矩阵A的维度	输入
a	• 在ssyevx中为单精度浮点型数组。 • 在dsyevx中为双精度浮点型数组。 • 在cheevx中为单精度复数型数组。 • 在zheevx中为双精度复数型数组。	矩阵A	输入/输出
lda	整数型	矩阵A的主维	输入
vl	• 在ssyevx/cheevx中为单精度浮点型数。 • 在dsyevx/zheevx中为双精度浮点型数。	• 当range=V时，表示获取特征值的下界； • 当range=A或I时参数无效	输入
vu	单精度/双精度浮点	• 当range=V时，表示获取特征值的上界； • 当range=A或I时参数无效	输入

 版权所有 © 华为技术有限公司

参数	类型	说明	输入/输出
il	整数型	• 当range=I时，表示会返回第il个特征值； • 当range=A或V时，参数无效	输入
iu	整数型	• 当range=I时，表示会返回第iu个特征值； • 当range=A或V时，参数无效	输入
abstol	单精度/双精度浮点	特征值的绝对误差容忍度	输入
m	整数型	需要计算的特征值个数，若range=A，m=n； 若range=I，m=iu-il+1	输出
w	单/双精度浮点	一个长度为n的浮点数组，前m个存储了计算得到的m个特征值（升序排列）	输出
z	• 在ssyevx中为单精度浮点型数组。 • 在dsyevx中为双精度浮点型数组。 • 在cheevx中为单精度复数型数组。 • 在zheevx中为双精度复数型数组。	• 若jobz=V, 且info=0,前m列存储了对应的特征向量 • 若jobz=N，则此参数无效	输出
ldz	整数型	z的主维	输入
work	• 在ssyevx中为单精度浮点型数组。 • 在dsyevx中为双精度浮点型数组。 • 在cheevx中为单精度复数型数组。 • 在zheevx中为双精度复数型数组。	临时存储空间，调用后work[0]为最优的lwork值。	输出
lwork	整数型	work数组的长度。 当输入lwork=-1时查询最优work大小，结果保存在work[0]中，否则： • 当n ≤ 1时，lwork ≥ 1。 • 当n > 1时，ssyevx与dsyevx中lwork ≥ 8*n；cheevx与zheevx中lwork ≥ 2*n。	输入

版权所有 © 华为技术有限公司

参数	类型	说明	输入/输出
rwork（复数类型特有）	• 在cheevx中为单精度浮点型数组。 • 在zheevx中为双精度浮点型数组。	工作数组，用于临时存储数据，长度为7*n。	输出
iwork	整数型	工作数组，用于临时存储数据，长度为5*n。	输出
ifail	整数型	输出数组，指示哪些特征值未收敛。	输出
info	整数型	状态值： 等于0：表示成功。 小于0：第-info个参数值不合法。 大于0：第info个特征值计算无法收敛。	输出

依赖

#include "klapack.h"

示例

```
  char jobz = 'V';
  char range = 'A';
  char uplo = 'L';
  int n = 5;
  double a[] = {7.027, 8.710, 1.015, 6.929, 7.584,
                8.710, 0.839, 2.469, 3.850, 0.559,
                1.015, 2.469, 1.930, 6.761, 7.207,
                6.929, 3.850, 6.761, 4.344, 4.804,
                7.584, 0.559, 7.207, 4.804, 6.177};
  int lda = 5;
  double vl = 0.0;
  double vu = 1.0;
  int il = 1;
  int iu = 2;
  double abstol = 1e-5;
  int m = 5;
  double w[5];
  double z[25];
  int ldz = 5;
  double work[40];
  int lwork = 40;
  int iwork[25];
  int ifail[5];
  int info = 0;

  dsyevx_(&jobz, &range, &uplo, &n, a, &lda, &vl, &vu, &il, &iu, &abstol, &m, w, z, &ldz, work, lwork, iwork,
ifail, &info);

  /*
   * Output:
   * Eigenvalues (in w)
   *  -8.842215 -3.341090 1.188784 6.204987 25.106533
   * Eigenvectors (in a)
   *   0.540506 -0.491256 0.488567 -0.240992 -0.412003
```

 版权所有 © 华为技术有限公司

```
*   -0.161613 0.597461 0.498950 -0.606212 0.021854
*   -0.305939 0.243208 0.345443 0.583128 -0.622802
*   0.523491 0.488212 -0.521315 -0.103794 -0.452839
*   -0.560440 -0.322810 -0.348211 -0.472883 -0.486653
*/
```

Fortran Interface:

```
CHARACTER :: jobz = "V"
CHARACTER :: range = "A"
CHARACTER :: uplo = "L"
PARAMETER (n = 5)
PARAMETER (lda = 5)
PARAMETER (ldz = 5)
PARAMETER (lwork = 40)
PARAMETER (m = 5)
REAL(8) :: a(lda, n)
REAL(8) :: vl = 0.0
REAL(8) :: vu = 1.0
INTEGER :: il = 1
INTEGER :: iu = 2
REAL(8) :: abstol = 1e-5
REAL(8) :: w(m)
REAL(8) :: z(ldz, n)
REAL(8) :: work(lwork)
INTEGER :: iwork(5*n)
INTEGER :: info = 0

DATA a / 7.027, 8.710, 1.015, 6.929, 7.584,
       8.710, 0.839, 2.469, 3.850, 0.559,
       1.015, 2.469, 1.930, 6.761, 7.207,
       6.929, 3.850, 6.761, 4.344, 4.804,
       7.584, 0.559, 7.207, 4.804, 6.177 /
EXTERNAL DSYEVX
CALL DSYEVX(jobz, range, uplo, n, a, lda, vl, vu, il, iu, abstol, m, w, z, ldz, work, lwork, iwork, ifail, info)
```

6.5.1.3.6.6 ?sy(he)gv

计算广义特征值问题的所有特征值和特征向量（可选）。

矩阵A，B为对称（Hermitian）矩阵，同时B为正定矩阵。

接口定义

C Interface:

dsygv_(const int *itype, const char *jobz, const char *uplo, const int *n, double *a, const int *lda, double *b, const int *ldb, double *w, double *work, const int *lwork, int *info);

ssygv_(const int *itype, const char *jobz, const char *uplo, const int *n, float *a, const int *lda, float *b, const int *ldb, float *w, float *work, const int *lwork, int *info);

chegv_(const int *itype, const char *jobz, const char *uplo, const int *n, kml_complex_float *a, const int *lda, kml_complex_float *b, const int *ldb, float *w, kml_complex_float *work, const int *lwork,

float *rwork, int *info);

zhegv_(const int *itype, const char *jobz, const char *uplo, const int *n, kml_complex_double *a, const int *lda, kml_complex_double *b, const int *ldb, double *w, kml_complex_double *work, const int *lwork,

double *rwork, int *info);

 版权所有 © 华为技术有限公司

Fortran Interface:

DSYGV(ITYPE, JOBZ, UPLO, N, A, LDA, B, LDB, W, WORK, LWORK, INFO);

SSYGV(ITYPE, JOBZ, UPLO, N, A, LDA, B, LDB, W, WORK, LWORK, INFO);

CHEGV(ITYPE, JOBZ, UPLO, N, A, LDA, B, LDB, W, WORK, LWORK, RWORK, INFO);

ZHEGV(ITYPE, JOBZ, UPLO, N, A, LDA, B, LDB, W, WORK, LWORK, RWORK, INFO);

参数

参数	类型	说明	输入/输出
itype	整数型	求解问题类型 ● 1：A*x=lambda*B*x ● 2：A*B*x = (lambda)*x ● 3：B*A*x = (lambda)*x	输入
jobz	字符型	● N：只计算特征值； ● V：计算特征值和特征向量	输入
uplo	字符型	● U：矩阵A的上三角部分被存储； ● L：矩阵A的下三角部分被存储	输入
n	整数型	矩阵A和B的维度	输入
a	● 在ssygv中为单精度浮点型数组。 ● 在dsygv中为双精度浮点型数组。 ● 在chegv中为单精度复数型数组。 ● 在zhegv中为双精度复数型数组。	矩阵A	输入/输出
lda	整数型	矩阵A的主维	输入
b	● 在ssygv中为单精度浮点型数组。 ● 在dsygv中为双精度浮点型数组。 ● 在chegv中为单精度复数型数组。 ● 在zhegv中为双精度复数型数组。	矩阵B	输入/输出

 版权所有 © 华为技术有限公司

参数	类型	说明	输入/输出
ldb	整数型	矩阵B的主维	输入
w	单/双精度	长度为n的数组，存储得到特征值（升序排列）	输出
work	单/双精度	若info=0，work(0)返回最佳lwork	输出
lwork	整数型	work数组的长度。 lwork ≥ max(1, 2*n-1)。	输入
rwork（复数类型特有）	• 在chegv中为单精度复数型数组。 • 在zhegv中为双精度复数型数组。	工作数组，用于临时存储数据。	输出
info	整数型	状态值： 等于0：表示成功。 小于0：第-info个参数值不合法。 大于0：算法出错。	输出

依赖

#include "klapack.h"

示例

C Interface:

```
int itype = 1;
char jobz = 'V';
char uplo = 'L';
int n = 5;
double a[] = {7.027, 8.710, 1.015, 6.929, 7.584,
              8.710, 0.839, 2.469, 3.850, 0.559,
              1.015, 2.469, 1.930, 6.761, 7.207,
              6.929, 3.850, 6.761, 4.344, 4.804,
              7.584, 0.559, 7.207, 4.804, 6.177};
int lda = 5;
double b[] = {2.345678,1.234567,0.987654,1.456789,0.876543,
          1.234567,2.345678,1.123456,0.987654,1.234567,
          0.987654,1.123456,2.345678,1.234567,0.987654,
          1.456789,0.987654,1.234567,2.345678,1.123456,
          0.876543,1.234567,0.987654,1.123456,2.345678};
int ldb = 5;
double w[5];
double work[15];
int lwork = 15;
int info = 0;

dsygv_(&itype, &jobz, &uplo, &n, a, &lda, b, &ldb, w, work, &lwork, &info);
```

 版权所有 © 华为技术有限公司

```
/*
 * Output:
 * Eigenvalues (in w)
 *  -8.726204 -2.319727 1.043130 4.411714 4.668052
 * Eigenvectors (in a)
 *   0.604055 -0.456650 0.512079 -0.505769 -0.177986
 *   0.174314 -0.523003 -0.357093 0.517106 -0.028092
 *   -0.277558 0.232180 0.310527 0.581010 -0.538289
 *   0.541974 0.126215 -0.182883 -0.010722 0.210494
 *   -0.205335 -0.456850 0.389506 0.070554 0.553286
 */
```

Fortran Interface:

```
INTEGER :: itype = 1
CHARACTER :: jobz = "V"
CHARACTER :: uplo = "L"
PARAMETER (n = 5)
PARAMETER (lda = 5)
PARAMETER (ldb = 5)
PARAMETER (lwork = 15)
REAL(8) :: a(lda, n)
REAL(8) :: w(n)
REAL(8) :: work(lwork)
INTEGER :: info = 0

DATA a / 7.027, 8.710, 1.015, 6.929, 7.584,
        8.710, 0.839, 2.469, 3.850, 0.559,
        1.015, 2.469, 1.930, 6.761, 7.207,
        6.929, 3.850, 6.761, 4.344, 4.804,
        7.584, 0.559, 7.207, 4.804, 6.177 /
DATA b / 2.345678,1.234567,0.987654,1.456789,0.876543,
        1.234567,2.345678,1.123456,0.987654,1.234567,
        0.987654,1.123456,2.345678,1.234567,0.987654,
        1.456789,0.987654,1.234567,2.345678,1.123456,
        0.876543,1.234567,0.987654,1.123456,2.345678 /

EXTERNAL DSYGV
CALL DSYGV(itype, jobz, uplo, n, a, lda, b, ldb, w, work, lwork, info)
```

6.5.1.3.6.7 ?geev

计算一般矩阵的部分特征值与特征向量。

接口定义

C Interface：

void sgeev_(const char *jobvl, const char *jobvr, const int *n, float *a, const int *lda, float *wr, float *wi, float *vl, const int *ldvl, float *vr, const int *ldvr, float *work, int *lwork, int *info);

void dgeev_(const char *jobvl, const char *jobvr, const int *n, double *a, const int *lda, double *wr, double *wi, double *vl, const int *ldvl, double *vr, const int *ldvr, double *work, int *lwork, int *info);

void cgeev_(const char *jobvl, const char *jobvr, const int *n, float _Complex *a, const int *lda, float _Complex *w, float _Complex *vl, const int *ldvl, float _Complex *vr, const int *ldvr, float _Complex *work, int *lwork, float *rwork, int *info);

void zgeev_(const char *jobvl, const char *jobvr, const int *n, double _Complex *a, const int *lda, double _Complex *w, double _Complex *vl, const int *ldvl, double _Complex *vr, const int *ldvr, double _Complex *work, int *lwork, double *rwork, int *info);

 版权所有 © 华为技术有限公司

Fortran Interface：

SGEEV(jobvl, jobvr, n, a, lda, wr, wi, vl, ldvl, vr, ldvr, work, lwork, info);

DGEEV(jobvl, jobvr, n, a, lda, wr, wi, vl, ldvl, vr, ldvr, work, lwork, info);

CGEEV(jobvl, jobvr, n, a, lda, w, vl, ldvl, vr, ldvr, work, lwork, rwork, info);

ZGEEV(jobvl, jobvr, n, a, lda, w, vl, ldvl, vr, ldvr, work, lwork, rwork, info);

参数

参数	类型	说明	输入/输出
jobvl	字符型	● ‘N’：左特征向量不计算。 ● ‘V’：左特征向量计算。	输入
jobvr	字符型	● ‘N’：右特征向量不计算。 ● ‘V’：右特征向量计算。	输入
n	整数型	矩阵A的维度，大于或等于0。	输入
a	● 在sgeev中为单精度浮点型数组。 ● 在dgeev中为双精度浮点型数组。 ● 在cgeev中为单精度复数型数组。 ● 在zgeev中为双精度复数型数组。	N*N的矩阵	输入，输出
lda	整数型	矩阵A的主维，大于或等于max(1, N)。	输入
w	● 在cgeev中为单精度复数型数组。 ● 在zgeev中为双精度复数型数组。	包含了所有的特征值，长度为n。	输出
wr	● 在sgeev中为单精度浮点型数组。 ● 在dgeev中为双精度浮点型数组。	特征值的实部，大小为n	输出
wi	● 在sgeev中为单精度浮点型数组。 ● 在dgeev中为双精度浮点型数组。	特征值的虚部，大小为n	输出

 版权所有 © 华为技术有限公司

参数	类型	说明	输入/输出
vl	• 在sgeev中为单精度浮点型数组。 • 在dgeev中为双精度浮点型数组。 • 在cgeev中为单精度复数型数组。 • 在zgeev中为双精度复数型数组。	大小：ldvl*n 若jobvl='v'：存放左特征向量；若jobvl='n'：不使用。	输出
ldvl	整数型	vl的leading dimension, ldvl>=1	输入
vr	• 在sgeev中为单精度浮点型数组。 • 在dgeev中为双精度浮点型数组。 • 在cgeev中为单精度复数型数组。 • 在zgeev中为双精度复数型数组。	大小：ldvr*n 若jobvr='v'：存放右特征向量；若jobvr='n'：不使用。	输出
ldvr	整数型	vr的leading dimension, ldvr>=1	输入
work	• 在sgeev中为单精度浮点型数组。 • 在dgeev中为双精度浮点型数组。 • 在cgeev中为单精度复数型数组。 • 在zgeev中为双精度复数型数组。	大小：max(1, lwork) 临时工作空间；info=0时，work(1)返回最优的lwork大小。	输出
lwork	整数型	work数组的长度。 lwork=-1时查询最优work大小，结果保存在work[0]中。且lwork ≥ max(1, 2*n-1)。	输入
rwork（复数特有）	• 在cgeev中为单精度实数。 • 在zgeev中为双精度实数。	临时存储空间，大小为2*n。	输出
info	整数型	状态值： • 等于0：表示成功。 • 小于0：第-info个参数值不合法。 • 大于0：算法出错。	输出

版权所有 © 华为技术有限公司

依赖

#include "klapack.h"

示例

C Interface：

```
#include <stdio.h>
#include <stdlib.h>
#include "lapack.h"

int main() {
    int i, j;
    int n = 3;  // 矩阵维度
    int lda = n;
    int ldvl = n;
    int ldvr = n;
    int lwork;
    int info;

    // 定义矩阵A (3x3)
    float a[9] = {
        1.0, 2.0, 3.0,
        4.0, 5.0, 6.0,
        7.0, 8.0, 9.0
    };

    // 输出原始矩阵
    printf("Original matrix A:\n");
    for (i = 0; i < n; i++) {
        for (j = 0; j < n; j++) {
            printf("%8.4f ", a[i*n + j]);
        }
        printf("\n");
    }
    printf("\n");

    // 分配内存用于存储特征值和特征向量
    float *wr = (float*)malloc(n * sizeof(float));  // 实部
    float *wi = (float*)malloc(n * sizeof(float));  // 虚部
    float *vl = (float*)malloc(n * n * sizeof(float));  // 左特征向量
    float *vr = (float*)malloc(n * n * sizeof(float));  // 右特征向量

    // 查询最优工作空间大小
    lwork = -1;
    float query_work;
    sgeev_("V", "V", &n, a, &lda, wr, wi, vl, &ldvl, vr, &ldvr, &query_work, &lwork, &info,1,1);

    // 分配工作空间
    lwork = (int)query_work;
    float *work = (float*)malloc(lwork * sizeof(float));

    // 计算特征值和特征向量
    sgeev_("V", "V", &n, a, &lda, wr, wi, vl, &ldvl, vr, &ldvr, work, &lwork, &info,1,1);

    // 检查是否成功
    if (info != 0) {
        printf("SGEEV failed with error code %d\n", info);
        return 1;
    }

    // 输出特征值
    printf("Eigenvalues:\n");
    for (i = 0; i < n; i++) {
```

 版权所有 © 华为技术有限公司

```
        if (wi[i] == 0.0) {
            printf("%8.4f\n", wr[i]);
        } else {
            printf("%8.4f + %8.4fi\n", wr[i], wi[i]);
            printf("%8.4f - %8.4fi\n", wr[i], -wi[i]);
            i++;  // 跳过共轭对
        }
    }
    printf("\n");

    // 输出右特征向量
    printf("Right eigenvectors:\n");
    for (i = 0; i < n; i++) {
        if (wi[i] == 0.0) {
            printf("Eigenvector for eigenvalue %8.4f:\n", wr[i]);
            for (j = 0; j < n; j++) {
                printf("%8.4f ", vr[j*n + i]);
            }
            printf("\n");
        } else {
            printf("Eigenvector pair for eigenvalue %8.4f + %8.4fi:\n", wr[i], wi[i]);
            for (j = 0; j < n; j++) {
                printf("%8.4f + %8.4fi ", vr[j*n + i], vr[j*n + i+1]);
            }
            printf("\n");
            for (j = 0; j < n; j++) {
                printf("%8.4f - %8.4fi ", vr[j*n + i], -vr[j*n + i+1]);
            }
            printf("\n");
            i++;  // 跳过共轭对
        }
    }

    // 释放内存
    free(wr);
    free(wi);
    free(vl);
    free(vr);
    free(work);

    return 0;
}
```

Fortran Interface：

```
 ! 定义常量与变量
integer, parameter :: n = 3          ! 矩阵维度
character(1) :: jobvl = 'N'         ! 不计算左特征向量
character(1) :: jobvr = 'V'         ! 计算右特征向量
integer :: lda = n, ldvl = n, ldvr = n
integer :: lwork, info, i, j,k
double precision :: a(n, n)         ! 输入矩阵（列优先）
double precision :: wr(n), wi(n)    ! 特征值实部和虚部
double precision :: vr(n, n), vl(n, n)        ! 特征向量（列存储）
double precision, allocatable :: work(:) ! 工作空间数组

! 初始化测试矩阵（按列优先存储）
! 矩阵示例：
! [ 1.0  4.0  7.0 ]
! [ 2.0  5.0  8.0 ]
! [ 3.0  6.0  9.0 ]
a = reshape([1.0d0, 2.0d0, 3.0d0, &
             4.0d0, 5.0d0, 6.0d0, &
             7.0d0, 8.0d0, 9.0d0], [n, n])

! === 步骤1：查询最优工作空间大小 ===
lwork = -1
allocate(work(1)) ! 临时分配最小空间
call dgeev(jobvl, jobvr, n, a, lda, wr, wi, vl, ldvl, vr, ldvr, work, lwork, info)
```

 版权所有 © 华为技术有限公司

```
! 检查查询是否成功
if (info /= 0) then
   print *, "工作空间查询失败，错误码：", info
   stop
end if

! 根据查询结果分配工作空间
lwork = int(work(1))
deallocate(work)
allocate(work(lwork))

! === 步骤2：实际计算特征值与特征向量 ===
call dgeev(jobvl, jobvr, n, a, lda, wr, wi, vl, ldvl, vr, ldvr, work, lwork, info)

! 检查计算是否成功
if (info /= 0) then
   print *, "特征值计算失败，错误码：", info
   stop
end if

k = 1
! === 输出结果 ===
print *, "=== 特征值 ==="
do i = 1, n
   if (wi(k) == 0.0d0) then
      ! 实数特征值
      write(*, '(A, I1, A, F12.6)') "λ", k, " = ", wr(k)
   else
      ! 复数共轭对（显示一对）
      write(*, '(A, I1, A, F12.6, A, F12.6, A)') "λ", k, " = ", wr(k), " ± ", wi(k), "i"
      k = k + 1  ! 跳过下一个共轭虚部
   end if
   k = k + 1
end do

print *, ""
print *, "=== 右特征向量（按列存储）==="
do j = 1, n
   write(*, '(A, I1, A)', advance='no') "向量 ", j, ": "
   do i = 1, n
      write(*, '(F12.6, 2X)', advance='no') vr(i, j)
   end do
   print *, ""
end do

! 释放内存
deallocate(work)
```

6.5.1.3.7 奇异值问题

6.5.1.3.7.1 ?gesvd

一般矩阵SVD分解。

接口定义

C Interface：

sgesvd_(const char *jobu, const char *jobvt, const int *m, const int *n, float *a, const int *lda, float *s, float *u, const int *ldu, float *vt,

const int *ldvt, float *work, const int *lwork, int *info);

dgesvd_(const char *jobu, const char *jobvt, const int *m, const int *n, double *a, const int *lda, double *s, double *u, const int *ldu, double *vt,

 版权所有 © 华为技术有限公司

const int *ldvt, double *work, const int *lwork, int *info);

cgesvd_(const char *jobu, const char *jobvt, const int *m, const int *n, float _Complex *a, const int *lda, float *s, float _Complex *u, const int *ldu, float _Complex *vt,

const int *ldvt, float _Complex *work, const int *lwork, float *rwork, int *info);

zgesvd_(const char *jobu, const char *jobvt, const int *m, const int *n, double _Complex *a, const int *lda, double *s, double _Complex *u, const int *ldu, double _Complex *vt,

const int *ldvt, double _Complex *work, const int *lwork, double *rwork, int *info);

Fortran Interface：

SGESVD(JOBU, JOBVT, M, N, A, LDA, S, U, LDU, VT, LDVT, WORK, LWORK, INFO);

DGESVD(JOBU, JOBVT, M, N, A, LDA, S, U, LDU, VT, LDVT, WORK, LWORK, INFO);

CGESVD(JOBU, JOBVT, M, N, A, LDA, S, U, LDU, VT, LDVT, WORK, LWORK, RWORK, INFO);

ZGESVD(JOBU, JOBVT, M, N, A, LDA, S, U, LDU, VT, LDVT, WORK, LWORK, RWORK, INFO);

参数

参数	类型	说明	输入/输出
jobu	字符型	• 'A'：返回酉矩阵U（左奇异值向量）的所有列。 • 'S'：返回酉矩阵U（左奇异值向量）的前min(m,n)列。 • 'O'：返回酉矩阵U（左奇异值向量）的前min(m,n)列，并复写到矩阵a。 • 'N'：酉矩阵U（左奇异值向量）不被计算。	输入
jobvt	字符型	• 'A'：返回V**H（右奇异值向量）的所有列，保存在矩阵vt中。 • 'S'：返回V**H（右奇异值向量）的前min(m,n)列，保存在矩阵vt中。 • 'O'：返回V**H（右奇异值向量）的前min(m,n)列，并复写到矩阵a。 • 'N'：V**H（右奇异值向量）不被计算。	输入
m	整数型	矩阵a的行数，m≥0。	输入
n	整数型	矩阵a的列数，n≥0。	输入

版权所有 © 华为技术有限公司

参数	类型	说明	输入/输出
a	• 在sgesvd中为单精度浮点型数组 • 在dgesvd中为双精度浮点型数组 • 在cgesvd中为单精度复数型数组 • 在zgesvd中为双精度复数型数组	大小为lda*n。 输入时：矩阵a为m*n的矩阵。 输出时： • jobu='O'，矩阵a被酉矩阵U（左奇异值向量）的前min(m,n)列覆盖。 • jobvt='O'，矩阵a被V**H（右奇异值向量, 按行优先存储）的前min(m,n)列覆盖。 • jobu !='O'且jobvt != 'O'，矩阵a的内容被销毁。	输入，输出
lda	• 整数型	矩阵a的主维，lda≥max(1,m)。	输入
s	• 在sgesvd中为单精度浮点型数组 • 在dgesvd中为双精度浮点型数组 • 在cgesvd中为单精度浮点型数组 • 在zgesvd中为双精度浮点型数组	大小为min(m, n)。 输出为矩阵a的奇异值，且按s(i)>=s(i+1)的规则排序。	输出
u	• 在sgesvd中为单精度浮点型数组 • 在dgesvd中为双精度浮点型数组 • 在cgesvd中为单精度复数型数组 • 在zgesvd中为双精度复数型数组	大小为ldu*ucol，其中当jobu='A'时，ucol=m; jobu='S'时，ucol=min(m,n)。 若jobu='A'，其为m*m的酉矩阵u。 若jobu='S'，其包含了U矩阵（左奇异值向量，列优先存储）的前min(m,n)列。 若jobu='N'或'O'，其不被使用。	输出
ldu	整数型	矩阵u的主维，ldu>=1；当jobu='S'或'A'时，ldu>=m。	输入
vt	• 在sgesvd中为单精度浮点型数组 • 在dgesvd中为双精度浮点型数组 • 在cgesvd中为单精度复数型数组 • 在zgesvd中为双精度复数型数组	大小为ldvt*n。 若jobvt='A'，其为n*n的酉矩阵v**H。 若jobvt='S'，其包含矩阵v**H（右奇异值向量，按行优先存储）的前min(m,n)行。 若jobvt='N'或'O'，其不被使用。	输出

 版权所有 © 华为技术有限公司

参数	类型	说明	输入/输出
ldvt	整数型	矩阵vt的主维，ldvt>=1；若jobvt='A'，则ldvt>=n；若jobvt='S'，则ldvt>=min(m,n)。	输入
work	• 在sgesvd中为单精度浮点型数组 • 在dgesvd中为双精度浮点型数组 • 在cgesvd中为单精度复数型数组 • 在zgesvd中为双精度复数型数组	大小为max(1, lwork)。 退出时，若info=0，work(1)返回最优lwork大小。	输出
lwork	整数型	work数组的大小，lwork>=max(1, 2*min(m,n)+max(m,n)) 若lwork=-1，则表明函数只计算最优的work大小。	输入
rwork（复数类型特有）	• 在cgesvd中为单精度浮点型数组 • 在zgesvd中为双精度浮点型数组	大小为5*min(m,n) 退出时，若info>0，rwork(1:min(m,n)-1)包含了上双对角矩阵B的未收敛超对角线元素，而其对角线元素保存在s中。	输出
info	整数型	• 等于0：表示成功。 • 小于0：info=-i，表示第i个参数非法。 • 大于0：若ZBDSQR不收敛，info则表示中间形式的矩阵B中有多少个超对角线不收敛到零。	输出

依赖

#include "klapack.h"

示例

C Interface：

```
const char jobu = 'A';
const char jobvt = 'A';
const int m = 5;
const int n = 5;
const int lda = 5;
const int ldu = 5;
const int ldvt = 5;

double a[] = {72.1673, 66.1857, 64.7644, 28.0199, 91.4151,
              6.5180,  62.8483, 72.4323, 46.5760, 8.6928,
```

 版权所有 © 华为技术有限公司

```
            28.9821, 42.1828, 18.6437, 99.8612, 35.6972,
            67.9812, 5.0880,  85.5035, 79.2945, 54.5920,
            28.6869, 49.7512, 7.5186,  28.6929, 84.6041};

double *s = (double*)malloc(m * sizeof(double));
double *u = (double*)malloc(ldu*m * sizeof(double));
double *vt = (double*)malloc(ldvt*n * sizeof(double));

double qwork;
int lwork = -1;
int info = 0;

dgesvd_(&jobu, &jobvt, &m, &n, a, &lda, s, u, &ldu, vt, &ldvt, &qwork, &lwork, &info);
if (info != 0) {
    printf("Error, info = %d\n", info);
    return info;
}

lwork = (int)qwork;
double *work = (double*)malloc(lwork * sizeof(double));

dgesvd_(&jobu, &jobvt, &m, &n, a, &lda, s, u, &ldu, vt, &ldvt, work, &lwork, &info);
if (info != 0) {
    printf("Error, info = %d\n", info);
    return info;
}
/* output */
* s
* 254.071516      87.547400       67.835073       64.370735       14.897476
* u
* -0.395825       -0.377874       -0.458327       -0.488090       -0.502234       -0.155273       -0.217110
0.348128        0.628260        -0.642534       * -0.312281       0.362918        -0.694714       0.528618
0.093312        -0.399962       0.761762        0.321232        -0.289476       -0.269743
* -0.749482       -0.313183       0.287969        0.061841        0.503430
* vt
* -0.562230       -0.504441       -0.297298       0.148954        -0.564680       -0.340949       0.391046
-0.060652       0.818829        0.238069        * -0.403926       0.372759        0.728616        -0.186514
-0.363629       -0.527964       0.375181        -0.468379       -0.520847       0.299721        * -0.354610
-0.559386       0.397083        -0.035531       0.634351
```

Fortran Interface：

```
CHARACTER :: jobu = "A"
CHARACTER :: jobvt = "A"
PARAMETER (m = 5)
PARAMETER (n = 5)
PARAMETER (lda = 5)
PARAMETER (ldu = 5)
PARAMETER (ldvt = 5)
INTEGER :: info = 0
REAL(8) :: a(lad, n)
REAL(8) :: s(m)
REAL(8) :: u(ldu, m)
REAL(8) :: vt(ldvt, n)
REAL(8), ALLOCATABLE :: work(:)
INTEGER :: lwork = -1
REAL(8) :: qwork

DATA a / 72.1673, 66.1857, 64.7644, 28.0199, 91.4151,
        6.5180,  62.8483, 72.4323, 46.5760, 8.6928,
        28.9821, 42.1828, 18.6437, 99.8612, 35.6972,
        67.9812, 5.0880,  85.5035, 79.2945, 54.5920,
        28.6869, 49.7512, 7.5186,  28.6929, 84.6041/
EXTERNAL DGESVD
CALL DGESVD(jobu, jobvt, m, n, a, lda, s, u, ldu, vt, ldvt, qwork, lwork, info);
lwork = (int)qwork;
work = (double *)malloc(sizeof(double) * lwork);
CALL DGESVD(jobu, jobvt, m, n, a, lda, s, u, ldu, vt, ldvt, work, lwork, info);
*
* Output:
```

 版权所有 © 华为技术有限公司

```
* s
* 254.071516     87.547400      67.835073      64.370735      14.897476
* u
* -0.395825      -0.377874      -0.458327      -0.488090      -0.502234      -0.155273      -0.217110
0.348128       0.628260       -0.642534      * -0.312281      0.362918       -0.694714      0.528618
0.093312       -0.399962      0.761762       0.321232       -0.289476      -0.269743
* -0.749482      -0.313183      0.287969       0.061841       0.503430
* vt
* -0.562230      -0.504441      -0.297298      0.148954       -0.564680      -0.340949      0.391046
-0.060652      0.818829       0.238069       * -0.403926      0.372759       0.728616       -0.186514
-0.363629      -0.527964      0.375181       -0.468379      -0.520847      0.299721       * -0.354610
-0.559386      0.397083       -0.035531      0.634351
```

6.5.1.3.7.2 ?gebrd

一般矩阵变换到两对角矩阵。

接口定义

C Interface：

sgebrd_(const int *m, const int *n, float *a, const int *lda, float *d, float *e, float *tauq, float *taup, float *work, const int lwork, int *info);

dgebrd_(const int *m, const int *n, double *a, const int *lda, double *d, double *e, double *tauq, double *taup, double *work, const int lwork, int *info);

cgebrd_(const int *m, const int *n, float _Complex *a, const int *lda, float *d, float *e, float _Complex *tauq, float _Complex *taup, float _Complex *work,

const int lwork, int *info);

zgebrd_(const int *m, const int *n, double _Complex *a, const int *lda, double *d, double *e, double _Complex *tauq, double _Complex *taup, double _Complex *work,

const int lwork, int *info);

Fortran Interface：

SGEBRD(M, N, A, LDA, D, E, TAUQ, TAUP, WORK, LWORK, INFO);

DGEBRD(M, N, A, LDA, D, E, TAUQ, TAUP, WORK, LWORK, INFO);

CGEBRD(M, N, A, LDA, D, E, TAUQ, TAUP, WORK, LWORK, INFO);

ZGEBRD(M, N, A, LDA, D, E, TAUQ, TAUP, WORK, LWORK, INFO);

参数

参数	类型	说明	输入/输出
m	整数型	矩阵a的行数，m≥0。	输入
n	整数型	矩阵a的列数，n≥0。	输入

版权所有 © 华为技术有限公司

参数	类型	说明	输入/输出
a	• 在sgebrd中为单精度浮点型数组 • 在dgebrd中为双精度浮点型数组 • 在cgebrd中为单精度复数型数组 • 在zgebrd中为双精度复数型数组	大小为lda*n • 进入时：m*n的矩阵。 • 退出时： 若m≥n，其对角线和超对角线位置被上双对角矩阵B，对角线以下的位置保存tauq数组，超对角线上方位置保存taup数组。 若m＜n，其对角线和次对角线位置被下双对角矩阵B，次对角线以下的位置保存tauq数组，对角线上方位置保存taup数组。	输入，输出
lda	整数型	矩阵a的主维，lda≥max(1, m)。	输入
d	• 在sgebrd中为单精度浮点型数组 • 在dgebrd中为双精度浮点型数组 • 在cgebrd中为单精度浮点型数组 • 在zgebrd中为双精度浮点型数组	• 长度为min(m, n)。 • 双对角。	输入，输出
e	• 在sgebrd中为单精度浮点型数组 • 在dgebrd中为双精度浮点型数组 • 在cgebrd中为单精度浮点型数组 • 在zgebrd中为双精度浮点型数组	大小为ldz*n • 进入时：若compz='V'时，则为正交矩阵。 • 退出时： – 若info=0，且compz='V'时，则为原对称矩阵的正交特征向量。 – 若compz='I'时，则为对称三对角矩阵的正交特征向量。 – 若compz='N'时，不使用。	输入，输出
tauq	• 在sgebrd中为单精度浮点型数组 • 在dgebrd中为双精度浮点型数组 • 在cgebrd中为单精度复数型数组 • 在zgebrd中为双精度复数型数组	矩阵z的主维，ldz >=1；若需要求特征向量，则ldz ≥ max(1, n)。	输入

 版权所有 © 华为技术有限公司

参数	类型	说明	输入/输出
taup	• 在sstedc中为单精度浮点型数组 • 在dstedc中为双精度浮点型数组 • 在cstedc中为单精度复数型数组 • 在zstedc中为双精度复数型数组	• 工作数组，大小为max(1, lwork)。 • 退出时，若info=0，work(1)返回最优lwork大小。	输出
work	• 在sgebrd中为单精度浮点型数组 • 在dgebrd中为双精度浮点型数组 • 在cgebrd中为单精度复数型数组 • 在zgebrd中为双精度复数型数组	• 工作数组，大小为max(1, liwork)。 • 退出时，若info=0，iwork(1)返回最优liwork大小。	输出
lwork	整数型	iwork数组的大小 • 若compz='N'或n<=1：liwork>=1。 • 若compz='V'或n>1: liwork >= 6+6*n+5*n*lgn。 • 若compz='I'或n>1: liwork>=3+5*n。 • 若liwork=-1，表明函数将只查询最优的liwork大小。	输入
info	整数型	• 等于0：表示成功。 • 小于0：info=-i，表示第i个参数非法。 • 大于0：算法在处理位于INFO/(N+1)到mod(INFO,N+1)行、列中的子矩阵时无法计算特征值。	输出

依赖

#include "klapack.h"

示例

C Interface：

```
const int m = 5;
const int n = 5;
const int lda = 5;
```

 版权所有 © 华为技术有限公司

```
double a[] = {72.1673, 66.1857, 64.7644, 28.0199, 91.4151,
              6.5180,  62.8483, 72.4323, 46.5760, 8.6928,
              28.9821, 42.1828, 18.6437, 99.8612, 35.6972,
              67.9812, 5.0880,  85.5035, 79.2945, 54.5920,
              28.6869, 49.7512, 7.5186,  28.6929, 84.6041};

double *d = (double*)malloc(m * sizeof(double));
double *e = (double*)malloc((m-1) * sizeof(double));
double *tauq = (double*)malloc(m * sizeof(double));
double *taup = (double*)malloc(m * sizeof(double));

double qwork;
int lwork = -1;
int info = 0;

dgebrd_(&m, &n, a, &lda, d, e, tauq, taup, &qwork, &lwork, &info);
if (info != 0) {
    printf("Error, info = %d\n", info);
    return info;
}

lwork = (int)qwork;
double *work = (double*)malloc(lwork * sizeof(double));

dgebrd_(&m, &n, a, &lda, d, e, tauq, taup, work, &lwork, &info);
if (info != 0) {
    printf("Error, info = %d\n", info);
    return info;
}
/* output */
* d
* -151.409804     -96.158378      30.604367       -49.958984      -65.000480
* e
* 187.838419      -61.829661      47.241856       4.133032
* tauq
* 1.476636        1.056414        1.268028        1.198872        0.000000
* taup
* 1.401567        1.618720        1.104858        0.000000        0.000000
```

Fortran Interface：

```
PARAMETER (m = 5)
PARAMETER (n = 5)
PARAMETER (lda = 5)
INTEGER :: info = 0
REAL(8) :: d(m)
REAL(8) :: e(m-1)
REAL(8) :: tauq(m)
REAL(8) :: taup(m)
REAL(8), ALLOCATABLE :: work(:)
INTEGER :: lwork = -1
REAL(8) :: qwork

DATA a / 72.1673, 66.1857, 64.7644, 28.0199, 91.4151,
        6.5180,  62.8483, 72.4323, 46.5760, 8.6928,
        28.9821, 42.1828, 18.6437, 99.8612, 35.6972,
        67.9812, 5.0880,  85.5035, 79.2945, 54.5920,
        28.6869, 49.7512, 7.5186,  28.6929, 84.6041/
EXTERNAL DGEBRD
CALL DGEBRD(m, n, a, lda, d, e, tauq, taup, qwork, lwork, info);
lwork = (int)qwork;
work = (double *)malloc(sizeof(double) * lwork);
CALL DGEBRD(m, n, a, lda, d, e, tauq, taup, work, lwork, info);
*
* Output:
* d
* -151.409804     -96.158378      30.604367       -49.958984      -65.000480
* e
* 187.838419      -61.829661      47.241856       4.133032
* tauq
```

 版权所有 © 华为技术有限公司

```
* 1.476636     1.056414     1.268028     1.198872     0.000000
* taup
* 1.401567     1.618720     1.104858     0.000000     0.000000
```

6.5.1.3.7.3 ?gesdd

计算矩形矩阵的奇异值分解，可以用来计算左和右奇异向量，使用分治算法。

接口定义

C Interface：

void sgesdd_(const char *jobz, const int *m, const int *n, float *a, const int *lda, float *s, float *u, const int *ldu, float *vt, const int *ldvt, float *work, const int *lwork, int *iwork, int *info);

void dgesdd_(const char *jobz, const int *m, const int *n, double *a, const int *lda, double *s, double *u, const int *ldu, double *vt, const int *ldvt, double *work, const int *lwork, int *iwork, int *info);

void cgesdd_(const char *jobz, const int *m, const int *n, float _Complex *a, const int *lda, float *s, float _Complex *u, const int *ldu, float _Complex *vt, const int *ldvt, float _Complex *work, const int *lwork, float *rwork, int *iwork, int *info);

void zgesdd_(const char *jobz, const int *m, const int *n, double _Complex *a, const int *lda, double *s, double _Complex *u, const int *ldu, double _Complex *vt, const int *ldvt, double _Complex *work, const int *lwork, double *rwork, int *iwork, int *info);

Fortran Interface：

SGESDD(JOBZ, M, N, A, LDA, S, U, LDU, VT, LDVT, WORK, LWORK, IWORK, INFO)

DGESDD(JOBZ, M, N, A, LDA, S, U, LDU, VT, LDVT, WORK, LWORK, IWORK, INFO)

CGESDD(JOBZ, M, N, A, LDA, S, U, LDU, VT, LDVT, WORK, LWORK, RWORK, IWORK, INFO)

ZGESDD(JOBZ, M, N, A, LDA, S, U, LDU, VT, LDVT, WORK, LWORK, RWORK, IWORK, INFO)

参数

参数	类型	说明	输入/输出
jobz	字符型	• 'N'：只计算特征值不返回U和Vt。 • 'S'：返回前min(M,N)列U和前min(M,N)行Vt。 • 'O'：如果M>=N，前N列U重写在矩阵A上，所有Vt行都返回在数组Vt上。 • 'A'：所有M列U和所有N行Vt都返回在数组U和Vt上。	输入
m	整数型	矩阵的维度，m≥0。	输入

版权所有 © 华为技术有限公司

参数	类型	说明	输入/输出
n	整数型	矩阵的维度，n≥0。	输入
a	• 在sgesdd中为单精度浮点型数组 • 在dgesdd中为双精度浮点型数组 • 在cgesdd中为单精度复数型数组 • 在zgesdd中为双精度复数型数组	大小为M*N • 进入时：M*N的矩阵。 • 退出时： 如果jobz='O'，M>=N时，A被U的前N列覆盖（左奇异值向量，按列存储）；如果jobz!='O'，销毁A。	输入，输出
lda	整数型	矩阵A的前导维度，lda>=max(1,M)。	输入
s	• 在sstedc中为单精度浮点型数组 • 在dstedc中为双精度浮点型数组 • 在cstedc中为单精度浮点型数组 • 在zstedc中为双精度浮点型数组	• 大小为min(M,N)。 • 矩阵A的奇异值，降序排列。	输出
u	• 在sgesdd中为单精度浮点型数组 • 在dgesdd中为双精度浮点型数组 • 在cgesdd中为单精度复数型数组 • 在zgesdd中为双精度复数型数组	• 二维数组，大小为LDU*UCOL。 • 如果jobz='A'或'O'并且M<N，UCOL=M，u中存储M*M的正交矩阵。 • 如果jobz='S',UCOL=min(M,N)，uz中存储前min(M,N)列的U，左奇异值向量，按列存储。 • 如果jobz='O'并且M>=N，或者jobz='N', 该参数无意义。	输出
ldu	整数型	• 矩阵u的前导维度，ldu>=1。 • 如果jobz='S'或'A'，或者jobz='O'并且M<N，LDU>=M。	输入
vt	• 在sgesdd中为单精度浮点型数组 • 在dgesdd中为双精度浮点型数组 • 在cgesdd中为单精度复数型数组 • 在zgesdd中为双精度复数型数组	• 二维数组，大小为LDVT*N。 • 如果jobz='A'或'O'并且M<N，UCOL=M，存储M*M的正交矩阵。 • 如果jobz='S', 存储前min(M,N)行的vt，右奇异值向量，按行存储。 • 如果jobz='O'并且M>=N，或者jobz='N', 该参数无意义。	输出

 版权所有 © 华为技术有限公司

参数	类型	说明	输入/输出
ldvt	整数型	● 矩阵vt的前导维度，ldvt>=1。 ● 如果jobz='A'或'O'并且M<N，ldvt>=N。 ● 如果jobz='S', ldvt>=min(M,N)。	输入
work	● 在sgesdd中为单精度浮点型数组 ● 在dgesdd中为双精度浮点型数组 ● 在cgesdd中为单精度复数型数组 ● 在zgesdd中为双精度复数型数组	● 数组维度为max(1,LWORK)。 ● 退出时，如果info=0，第一个元素返回合适的lwork。	输出
lwork	整数型	矩阵work的维度，lwork>=1。如果lwork=1，默认为查询状态，在work第一个元素中返回合适的大小，并且不检查其它work的参数。 ● 如果JOBZ ='N'，则LWORK >= 3*mn + max(mx, 7*mn)。 ● 如果JOBZ ='O'，则LWORK >= 3*mn + max(mx, 5*mn*mn + 4*mn)。 ● 如果JOBZ ='S'，则LWORK >= 4*mn*mn + 7*mn。 ● 如果JOBZ ='A'，则LWORK >= 4*mn*mn + 6*mn + mx。	输入
rwork（复数类型特有）	● 在cgesvd中为单精度浮点型数组 ● 在zgesvd中为双精度浮点型数组	RWORK长度为LRWORK，记mx=max(M, N)，mn=min(M, N)， 如果JOBZ = 'N'，则LRWORK>=5*mn; 其他如果mx>>mn，则LRWORK >= 5*mn*mn + 5*mn; 其他情况，LRWORK >= max(5*mn*mn + 5*mn, 2*mx*mn + 2*mn*mn + mn)。	输出
iwork	整数型数组	数组维度为8*min(M,N)。	输出
info	整数型	● 等于0：表示成功。 ● 小于0：info=-i，表示第i个参数非法。 ● 大于0：计算出错。	输出

 版权所有 © 华为技术有限公司

依赖

#include "klapack.h"

示例

C Interface：

```
const char jobz = 'A';
const int m = 5;
const int n = 5;
const int lda = 5;
const int ldu = 5;
const int ldvt = 5;

double a[] = {72.1673, 66.1857, 64.7644, 28.0199, 91.4151,
              6.5180,  62.8483, 72.4323, 46.5760, 8.6928,
              28.9821, 42.1828, 18.6437, 99.8612, 35.6972,
              67.9812, 5.0880,  85.5035, 79.2945, 54.5920,
              28.6869, 49.7512, 7.5186,  28.6929, 84.6041};

double *s = (double*)malloc(m * sizeof(double));
double *u = (double*)malloc(ldu*m * sizeof(double));
double *vt = (double*)malloc(ldvt*n * sizeof(double));
double *iwork = (double*)malloc(8 * m * sizeof(double));

double qwork;
int lwork = -1;
int info = 0;

dgesdd_(&jobz, &m, &n, a, &lda, s, u, &ldu, vt, &ldvt, &qwork, &lwork, iwork, &info);
if (info != 0) {
    printf("Error, info = %d\n", info);
    return info;
}

lwork = (int)qwork;
double *work = (double*)malloc(lwork * sizeof(double));

dgesdd_(&jobz, &m, &n, a, &lda, s, u, &ldu, vt, &ldvt, work, &lwork, iwork, &info);
if (info != 0) {
    printf("Error, info = %d\n", info);
    return info;
}

* output:
* s:
* 254.071516      87.547400       67.835073       64.370735       14.897476
* u:
* -0.395825       -0.377874       -0.458327       -0.488090       -0.502234       -0.155273       -0.217110
0.348128        0.628260        -0.642534       * -0.312281       0.362918        -0.694714       0.528618
0.093312        -0.399962       0.761762        0.321232        -0.289476       -0.269743       * -0.749482
-0.313183       0.287969        0.061841        0.503430
* vt:
* -0.562230       -0.504441       -0.297298       0.148954        -0.564680       -0.340949       0.391046
-0.060652       0.818829        0.238069        * -0.403926       0.372759        0.728616        -0.186514
-0.363629       -0.527964       0.375181        -0.468379       -0.520847       0.299721        * -0.354610
-0.559386       0.397083        -0.035531       0.634351
```

Fortran Interface：

```
CHARACTER :: jobz = "A"
PARAMETER (m = 5)
PARAMETER (n = 5)
PARAMETER (lda = 5)
PARAMETER (ldu = 5)
PARAMETER (ldvt = 5)
INTEGER :: info = 0
```


版权所有 © 华为技术有限公司

```
REAL(8) :: a(lad, n)
REAL(8) :: s(m)
REAL(8) :: u(ldu, m)
REAL(8) :: vt(ldvt, n)
REAL(8) :: iwork(8, m)
REAL(8), ALLOCATABLE :: work(:)
INTEGER :: lwork = -1
REAL(8) :: qwork

DATA a / 72.1673, 66.1857, 64.7644, 28.0199, 91.4151,
        6.5180,  62.8483, 72.4323, 46.5760, 8.6928,
        28.9821, 42.1828, 18.6437, 99.8612, 35.6972,
        67.9812, 5.0880,  85.5035, 79.2945, 54.5920,
        28.6869, 49.7512, 7.5186,  28.6929, 84.6041/
EXTERNAL DGESDD
CALL DGESDD(jobu, jobvt, m, n, a, lda, s, u, ldu, vt, ldvt, qwork, lwork, iwork, info);
lwork = (int)qwork;
work = (double *)malloc(sizeof(double) * lwork);
CALL DGESDD(jobu, jobvt, m, n, a, lda, s, u, ldu, vt, ldvt, work, lwork, iwork, info);
*
* Output:
* s:
* 254.071516      87.547400       67.835073       64.370735       14.897476
* u:
* -0.395825       -0.377874       -0.458327       -0.488090       -0.502234       -0.155273       -0.217110
0.348128        0.628260        -0.642534       * -0.312281       0.362918        -0.694714       0.528618
0.093312        -0.399962       0.761762        0.321232        -0.289476       -0.269743       * -0.749482
-0.313183       0.287969        0.061841        0.503430
* vt:
* -0.562230       -0.504441       -0.297298       0.148954        -0.564680       -0.340949       0.391046
-0.060652       0.818829        0.238069        * -0.403926       0.372759        0.728616        -0.186514
-0.363629       -0.527964       0.375181        -0.468379       -0.520847       0.299721        * -0.354610
-0.559386       0.397083        -0.035531       0.634351
```

6.5.1.3.7.4 ?bdsdc

计算N*N的上/下双对角矩阵B的奇异值分解，使用分治算法。

接口定义

C Interface：

void sbdsdc_(const char *uplo, const char *compq, const int *n, float *d, float *e, float *u, const int *ldu, float *vt, const int *ldvt, float *q, int *iq,

float *work, int *iwork, int *info);

void dbdsdc_(const char *uplo, const char *compq, const int *n, double *d, double *e, double *u, const int *ldu, double *vt, const int *ldvt, double *q,

int *iq, double *work, int *iwork, int *info);

Fortran Interface：

SBDSDC(UPLO, COMPQ, N, D, E, U, LDU, VT, Q, IQ, WORK, IWORK, INFO)

DBDSDC(UPLO, COMPQ, N, D, E, U, LDU, VT, Q, IQ, WORK, IWORK, INFO)

版权所有 © 华为技术有限公司

参数

参数	类型	说明	输入/输出
uplo	字符型	• ='U'，B为上双对角矩阵 • ='L'，B为下双对角矩阵	输入
compq	字符型	• ='N'，只计算奇异值； • ='P'，计算奇异值，并且以紧凑的形式计算奇异向量 • ='I'，计算奇异值和奇异向量	输入
n	整数型	矩阵B的阶数	输入
d	• sbdsdc中为单精度实数数组 • dbdsdc中为双精度实数数组	• 输入时，存放B的对角元素。 • 输出时，存放B的奇异值。	输入，输出
e	• sbdsdc中为单精度实数数组 • dbdsdc中为双精度实数数组	• 输入，存放B的非对角元素。 • 输出时，e被销毁。	输入，输出
u	• sbdsdc中为单精度实数数组 • dbdsdc中为双精度实数数组	• 维度是LDU*N。 • 如果COMPQ='I'，退出时，U包括左奇异向量。 • 如果COMPQ为其他值，U无意义。	输出
ldu	整数型	• U的前导维度，LDU>=1。 • 如果需要奇异向量，LDU>=max(1,N)。	输入
vt	• sbdsdc中为单精度实数数组 • dbdsdc中为双精度实数数组	• 维度ldvt*n。 • 如果COMPQ='I'，退出时，vt包括右奇异向量。 • 如果COMPQ为其他值，vt无意义。	输出
ldvt	整数型	• vt的前导维度，ldvt>=1。 • 如果需要奇异向量，ldvt>=max(1,N)。	输入
q	• sbdsdc中为单精度实数数组 • dbdsdc中为双精度实数数组	• 如果COMPQ='P'，Q和IQ存放左右特征向量，以紧凑的形式，仅需要NlogN的空间而不是2N^2。 • 如果COMPQ为其它值，Q无意义。	输出

版权所有 © 华为技术有限公司

参数	类型	说明	输入/输出
iq	• sbdsdc中为单精度实数数组 • dbdsdc中为双精度实数数组	• 如果COMPQ='P'，Q和IQ存放左右特征向量，以紧凑的形式，仅需要NlogN的空间而不是2N^2。 • 如果COMPQ为其它值，IQ无意义。	输出
work	• sbdsdc中为单精度实数数组 • dbdsdc中为双精度实数数组	• 维度(MAX(1,LWORK))。 • 如果COMPQ ='N'，则LWORK>=(4*N)。 • 如果COMPQ ='P'，则LWORK>=(6*N)。 • 如果COMPQ ='I'，则LWORK>=(3*N**2+4*N)。	输出
lwork	整型数组	大小为（8*N）。	输出
info	整数型	• 等于0：表示成功。 • 小于0：info=-i，表示第i个参数非法。 • 大于0：计算出错。	输出

依赖

#include "klapack.h"

示例

C Interface

```
const char uplo = 'l';
const char compq = 'i';
const int n = 5;
const int ldu = 5;
const int ldvt = 5;

double d[] = {-151.409804,-96.158378,30.604367,-49.958984,-65.000480};
double e[] = {187.838419,-61.829661,47.241856,4.133032};
double *u = (double*)malloc(ldu * n * sizeof(double));
double *vt = (double*)malloc(ldvt * n * sizeof(double));
double *q = (double*)malloc(sizeof(double));
int *iq = (int*)malloc(sizeof(int));
double *work = (double*)malloc(4 * n * sizeof(double));
double *iwork = (double*)malloc(8 * n * sizeof(double));

int info = 0;

dbdsdc_(&uplo, &compq, &n, d, e, u, &ldu, vt, &ldvt, q, iq, work, iwork, &info);
if (info != 0) {
    printf("Error, info = %d\n", info);
    return info;
}

* output:
```

 版权所有 © 华为技术有限公司

```
* d:
* 254.071515      87.547400       67.835074       64.370735       14.897476
* e:
* 0.000000       0.000000       0.000000       0.000000
* u:
* 0.562230       -0.822915       -0.081874       -0.001979       0.000007       0.504441       0.270668
0.734742       0.363253       -0.021916       * -0.297298       -0.191539       -0.135822       0.802484
-0.460975       -0.148954       -0.098365       -0.044364       0.423397       0.887076       * -0.564680
-0.450761       0.658054       -0.211655       -0.010871
* vt:
* -0.943444       -0.291675       0.133196       0.063327       0.055560       0.331373       -0.816196
0.395311       0.189552       0.178367       * -0.010230       0.452864       0.497591       0.289640
0.680678       0.000389       -0.208325       -0.619098       -0.271648       0.706772       * -0.000002
0.016272       0.441712       -0.895754       0.047431
```

Fortran Interface

```
CHARACTER :: uplo = "L"
CHARACTER :: compq = "i"
PARAMETER (n = 5)
PARAMETER (ldu = 5)
PARAMETER (ldvt = 5)
INTEGER :: info = 0
REAL(8) :: d(n)
REAL(8) :: e(n-1)
REAL(8) :: u(ldu, n)
REAL(8) :: vt(ldvt, n)
REAL(8) :: iwork(8, m)
REAL(8), ALLOCATABLE :: q(:)
INTEGER, ALLOCATABLE :: iq(:)
REAL(8) :: work(4*n)
REAL(8) :: iwork(8*n)

DATA d / -151.409804,-96.158378,30.604367,-49.958984,-65.000480 /
DATA e / 187.838419,-61.829661,47.241856,4.133032 /
EXTERNAL DBDSDC
CALL DBDSDC(uplo, compq, n, d, e, u, ldu, vt, ldvt, q, iq, work, iwork, info);

* output:
* d:
* 254.071515      87.547400       67.835074       64.370735       14.897476
* e:
* 0.000000       0.000000       0.000000       0.000000
* u:
* 0.562230       -0.822915       -0.081874       -0.001979       0.000007       0.504441       0.270668
0.734742       0.363253       -0.021916       * -0.297298       -0.191539       -0.135822       0.802484
-0.460975       -0.148954       -0.098365       -0.044364       0.423397       0.887076       * -0.564680
-0.450761       0.658054       -0.211655       -0.010871
* vt:
* -0.943444       -0.291675       0.133196       0.063327       0.055560       0.331373       -0.816196
0.395311       0.189552       0.178367       * -0.010230       0.452864       0.497591       0.289640
0.680678       0.000389       -0.208325       -0.619098       -0.271648       0.706772       * -0.000002
0.016272       0.441712       -0.895754       0.047431
```

6.5.1.3.7.5 ?bdsqr

使用对双对角矩阵的QR算法的SVD分解来求解奇异值或左右奇异向量。

接口定义

C Interface：

void sbdsqr_(const char *uplo, const int *n, const int *ncvt, const int *nru, const int *ncc, float *d, float *e, float *vt, const int *ldvt, float *u, const int *ldu,

float *c, const int *ldc, float *work, int *info)

 版权所有 © 华为技术有限公司

void dbdsqr_(const char *uplo, const int *n, const int *ncvt, const int *nru, const int *ncc, double *d, double *e, double *vt, const int *ldvt, double *u, const int *ldu,

double *c, const int *ldc, double *work, int *info)

void cbdsqr_(const char *uplo, const int *n, const int *ncvt, const int *nru, const int *ncc, float *d, float *e, float _Complex *vt, const int *ldvt, float _Complex *u, const int *ldu,

float _Complex *c, const int *ldc, float *work, int *info)

void zbdsqr_(const char *uplo, const int *n, const int *ncvt, const int *nru, const int *ncc, double *d, double *e, double _Complex *vt, const int *ldvt, double _Complex *u,

const int *ldu, double _Complex *c, const int *ldc, double *work, int *info)

Fortran Interface：

SBDSQR(UPLO, N, NCVT, NRU, NCC, D, E, VT, LDVT, U, LDU, C, LDC, WORK, INFO)

DBDSQR(UPLO, N, NCVT, NRU, NCC, D, E, VT, LDVT, U, LDU, C, LDC, WORK, INFO)

CBDSQR(UPLO, N, NCVT, NRU, NCC, D, E, VT, LDVT, U, LDU, C, LDC, WORK, INFO)

ZBDSQR(UPLO, N, NCVT, NRU, NCC, D, E, VT, LDVT, U, LDU, C, LDC, WORK, INFO)

参数

参数	类型	说明	输入/输出
uplo	字符型	• 'U'：B是上双对角矩阵。 • 'L'：B是下双对角矩阵。	输入
n	整数型	矩阵B的阶数，n>=0。	输入
nru	整数型	矩阵U的行数，nru≥0。	输入
ncvt	整数型	矩阵VT的列数，ncvt≥0。	输入
ncc	整数型	矩阵C的列数，NCC≥0。	输入

 版权所有 © 华为技术有限公司

参数	类型	说明	输入/输出
d	• 在sbdsqr中为单精度浮点型数组 • 在dbdsqr中为双精度浮点型数组 • 在cbdsqr中为单精度浮点型数组 • 在zbdsqr中为双精度浮点型数组	维度为n。 • 进入时：为双对角矩阵B的n个对角元素 • 退出时： – 若info =0，则B的奇异值按降序排列	输入，输出
e	• 在sbdsqr中为单精度浮点型数组 • 在dbdsqr中为双精度浮点型数组 • 在cbdsqr中为单精度浮点型数组 • 在zbdsqr中为双精度浮点型数组	维度为n-1。 • 进入时：为双对角矩阵B的n-1个非对角元素 • 退出时： – 若info =0，则e被摧毁。 – 若info >0，则D和E会容纳与输入正交化相等的双对角矩阵的对角和超对角元素。	输入 输出
vt	• 在sbdsqr中为单精度浮点型数组 • 在dbdsqr中为双精度浮点型数组 • 在cbdsqr中为单精度复数型数组 • 在zbdsqr中为双精度复数型数组	维度为ldvt*ncvt，nvct=0时无意义。 • 进入时：为N*NCVT大小的矩阵VT。 • 退出时：被P^H*VT覆盖。	输入，输出
ldvt	整数型	vt的前导维度。 • ncvt>0时，ldvt>=max(1,N)。 • ncvt=0时，ldvt>-1。	输入

 版权所有 © 华为技术有限公司

参数	类型	说明	输入/输出
u	• 在sbdsqr中为单精度浮点型数组 • 在dbdsqr中为双精度浮点型数组 • 在cbdsqr中为单精度复数型数组 • 在zbdsqr中为双精度复数型数组	维度为LDU*N，NRU=0时无意义。 • 进入时，NRU*N的矩阵。 • 退出时，U被U*Q覆盖。	输入，输出
ldu	整数型	矩阵U的前导维度，ldu>=max(1,NRU)。	输入
c	• 在sbdsqr中为单精度浮点型数组 • 在dbdsqr中为双精度浮点型数组 • 在cbdsqr中为单精度复数型数组 • 在zbdsqr中为双精度复数型数组	维度为LDC*NCC，NCC=0时无意义。 • 进入时，N*NCC的矩阵C。 • 退出时，被Q^H*C覆盖。	输入，输出
ldc	整数型	C的前导维度。 • 如果NCC>0，LDC>=max(1,N)。 • 如果NCC=0，LDC>=1。	输入
work	• 在sbdsqr中为单精度浮点型数组 • 在dbdsqr中为双精度浮点型数组 • 在cbdsqr中为单精度浮点型数组 • 在zbdsqr中为双精度浮点型数组	维度为4*N。	输出

版权所有 © 华为技术有限公司

参数	类型	说明	输入/输出
info	整数型	• 等于0：表示成功。 • 小于0：info=-i，表示第i个参数非法。 • 大于0：计算出错。	输出

依赖

#include "klapack.h"

示例

C Interface

```
const char uplo = 'l';
const int n = 5;
const int ncvt = 5;
const int nru = 5;
const int ncc = 5;
const int ldu = 5;
const int ldvt = 5;
const int ldc = 5;

double d[] = {-151.409804,-96.158378,30.604367,-49.958984,-65.000480};
double e[] = {187.838419,-61.829661,47.241856,4.133032};
double u[] = {0.562230,-0.822915,-0.081874,-0.001979,0.000007,
              0.504441,0.270668,0.734742,0.363253,-0.021916,
              -0.297298,-0.191539,-0.135822,0.802484,-0.460975,
              -0.148954,-0.098365,-0.044364,0.423397,0.887076,
              -0.564680,-0.450761,0.658054,-0.211655,-0.010871};
double c[] = {72.1673, 66.1857, 64.7644, 28.0199, 91.4151,
              6.5180,  62.8483, 72.4323, 46.5760, 8.6928,
              28.9821, 42.1828, 18.6437, 99.8612, 35.6972,
              67.9812, 5.0880,  85.5035, 79.2945, 54.5920,
              28.6869, 49.7512, 7.5186,  28.6929, 84.6041};
double vt[] = {-0.943444,-0.291675,0.133196,0.063327,0.055560,
               0.331373,-0.816196,0.395311,0.189552,0.178367,
               -0.010230,0.452864,0.497591,0.289640,0.680678,
               0.000389,-0.208325,-0.619098,-0.271648,0.706772,
               -0.000002,0.016272,0.441712,-0.895754,0.047431};
double *work = (double*)malloc(4 * (n-1) * sizeof(double));

int info = 0;

dbdsqr_(&uplo, &n, &ncvt, &nru, &ncc, d, e, vt, &ldvt, u, &ldu, c, &ldc, work, &info);
if (info != 0) {
    printf("Error, info = %d\n", info);
    return info;
}

* output:
* d:
* 254.071515      87.547400       67.835074       64.370735       14.897476
* e:
* 0.000000        0.000000        0.000000        0.000000
* u:
* -0.074378       -0.669531       -0.639450       -0.366581       0.054025        0.159978        -0.508435
```


版权所有 © 华为技术有限公司

```
0.027239        0.845380        -0.022155       * -0.082621       0.347676        -0.436891       0.259353
0.783682        -0.684158       -0.337057       0.510908        -0.079527       0.388547        * -0.702835
0.242353        -0.372103       0.278142        -0.481107

* vt:
* 0.792096        0.561274        -0.189353       -0.143425       0.033614        -0.587068       0.711959
-0.120375       -0.230494       0.284338        * 0.154739        -0.190565       0.546605        -0.459085
0.655902        -0.063174       -0.042355       0.090008        -0.778081       -0.617013       * 0.000527
0.374135        0.801734        0.331865        -0.327278
* c:
* -19.247910      110.078454      -62.583336      72.822482       -34.891161      -54.076617      90.246359
9.555710        17.064964       5.701693        * -20.142043      75.228107       44.453325       64.653600
-44.635543      26.877086       126.100144      5.668549        67.580452       -1.791622       * -25.484162
42.029730       -35.053810      77.698370       -40.669875
```

Fortran Interface

```
CHARACTER :: uplo = "L"
PARAMETER (n = 5)
PARAMETER (ncvt = 5)
PARAMETER (nru = 5)
PARAMETER (ncc = 5)
PARAMETER (ldu = 5)
PARAMETER (ldc = 5)
PARAMETER (ldvt = 5)
INTEGER :: info = 0
REAL(8) :: d(n)
REAL(8) :: e(n-1)
REAL(8) :: u(ldu, n)
REAL(8) :: c(ldc, ncc)
REAL(8) :: vt(ldvt, ncvt)
REAL(8) :: work(4*(n-1))

DATA d / -151.409804,-96.158378,30.604367,-49.958984,-65.000480 /
DATA e / 187.838419,-61.829661,47.241856,4.133032 /
DATA u / 0.562230,-0.822915,-0.081874,-0.001979,0.000007,
         0.504441,0.270668,0.734742,0.363253,-0.021916,
         -0.297298,-0.191539,-0.135822,0.802484,-0.460975,
         -0.148954,-0.098365,-0.044364,0.423397,0.887076,
         -0.564680,-0.450761,0.658054,-0.211655,-0.010871 /
DATA c / 72.1673, 66.1857, 64.7644, 28.0199, 91.4151,
          6.5180,  62.8483, 72.4323, 46.5760, 8.6928,
          28.9821, 42.1828, 18.6437, 99.8612, 35.6972,
          67.9812, 5.0880,  85.5035, 79.2945, 54.5920,
          28.6869, 49.7512, 7.5186,  28.6929, 84.6041 /
DATA vt / -0.943444,-0.291675,0.133196,0.063327,0.055560,
           0.331373,-0.816196,0.395311,0.189552,0.178367,
           -0.010230,0.452864,0.497591,0.289640,0.680678,
           0.000389,-0.208325,-0.619098,-0.271648,0.706772,
           -0.000002,0.016272,0.441712,-0.895754,0.047431 /
EXTERNAL DBDSQR
CALL DBDSQR(uplo, n, ncvt, nru, ncc, d, e, vt, ldvt, u, ldu, c, ldc, work, info);

* output:
* d:
* 254.071515      87.547400       67.835074       64.370735       14.897476
* e:
* 0.000000        0.000000        0.000000        0.000000
* u:
* -0.074378       -0.669531       -0.639450       -0.366581       0.054025        0.159978        -0.508435
0.027239        0.845380        -0.022155       * -0.082621       0.347676        -0.436891       0.259353
0.783682        -0.684158       -0.337057       0.510908        -0.079527       0.388547        * -0.702835
0.242353        -0.372103       0.278142        -0.481107

* vt:
* 0.792096        0.561274        -0.189353       -0.143425       0.033614        -0.587068       0.711959
-0.120375       -0.230494       0.284338        * 0.154739        -0.190565       0.546605        -0.459085
0.655902        -0.063174       -0.042355       0.090008        -0.778081       -0.617013       * 0.000527
0.374135        0.801734        0.331865        -0.327278
* c:
```


版权所有 © 华为技术有限公司

```
* -19.247910      110.078454      -62.583336      72.822482       -34.891161      -54.076617      90.246359
9.555710        17.064964       5.701693        * -20.142043    75.228107       44.453325       64.653600
-44.635543      26.877086       126.100144      5.668549        67.580452       -1.791622       * -25.484162
42.029730       -35.053810      77.698370       -40.669875
```

6.5.1.3.8 最小二乘问题函数

6.5.1.3.8.1 ?gels

使用矩阵的QR或LQ分解来求解超定或欠定线性方程组。

接口定义

C Interface：

sgels_(const char *trans, const int *m, const int *n, const int *nrhs, float *a, const int *lda, float *b, const int *ldb, float *work, const int *lwork, int *info);

dgels_(const char *trans, const int *m, const int *n, const int *nrhs, double *a, const int *lda, double *b, const int *ldb, double *work, const int *lwork, int *info);

cgels_(const char *trans, const int *m, const int *n, const int *nrhs, float _Complex *a, const int *lda, float _Complex *b, const int *ldb, float _Complex *work, const int *lwork, int *info);

zgels_(const char *trans, const int *m, const int *n, const int *nrhs, double _Complex *a, const int *lda, double _Complex *b, const int *ldb, double _Complex *work, const int *lwork, int *info);

Fortran Interface：

SGELS(TRANS, M, N, NRHS, A, LDA, B, LDB, WORK, LWORK, INFO);

DGELS(TRANS, M, N, NRHS, A, LDA, B, LDB, WORK, LWORK, INFO);

CGELS(TRANS, M, N, NRHS, A, LDA, B, LDB, WORK, LWORK, INFO);

ZGELS(TRANS, M, N, NRHS, A, LDA, B, LDB, WORK, LWORK, INFO);

参数

参数	类型	说明	输入/输出
trans	字符型	• 'N'：矩阵A不转置。 • 'C'：矩阵A转置。	输入
m	整数型	矩阵A的行数，m≥0。	输入
n	整数型	矩阵A的列数，n≥0。	输入
nrhs	整数型	右端项列数，nrhs≥0。	输入

版权所有 © 华为技术有限公司

参数	类型	说明	输入/输出
a	• 在sgels中为单精度浮点型数组。 • 在dgels中为双精度浮点型数组。 • 在cgels中为单精度复数型数组。 • 在zgels中为双精度复数型数组。	大小为lda*n。 • 进入时：为m*n的矩阵。 • 退出时： – 若m≥n，则为GEQRF返回的QR分解结果。 – 若m<n，则为GELQF返回的LQ分解。	输入，输出
lda	整数型	矩阵A的主维，lda≥max(1, m)	输入
b	• 在sgels中为单精度浮点型数组。 • 在dgels中为双精度浮点型数组。 • 在cgels中为单精度复数型数组。 • 在zgels中为双精度复数型数组。	右端矩阵，大小为ldb*nrhs。 • 进入时： – 若trans='N'，则为m*nrhs的矩阵。 – 若trans='C'或'T'，则为n*nrhs的矩阵。 • 退出时：若info=0 – 若trans='N'，且m≥n，则矩阵的第1到n行包含了最小二乘向量；同时，每列的残差平方和由该列的第n+1到m个元素给出。 – 若trans='N'，且m<n，则矩阵的第1到n行包含了最小范数解向量。 – 若trans='C'或'T'，且m>=n，则矩阵的第1到n行包含了最小范数解向量。 – 若trans='C'或'T'，且m<n，则矩阵的第1到n行包含了最小二乘向量；同时，每列的残差平方和由该列的第n+1到m个元素给出。	输入，输出
ldb	整数型	矩阵b的主维。	输入

 版权所有 © 华为技术有限公司

参数	类型	说明	输入/输出
work	• 在sgels中为单精度浮点型数组。 • 在dgels中为双精度浮点型数组。 • 在cgels中为单精度复数型数组。 • 在zgels中为双精度复数型数组。	工作数组。Info=0时，work(0)返回最优lwork大小。	输出
lwork	整数型	work的大小。	输入
Info	整数型	状态值： • 等于0：表示成功。 • 小于0：info=-i，表示第i个参数非法。 • 大于0：算法出错。	输出

依赖

#include "klapack.h"

示例

C Interface：

```
const char trans = 'N';
const int m = 4;
const int n = 4;
const int nrhs = 2;
const int lda = 4;
const int ldb = 4;

double a[] = {72.1673, 66.1857, 64.7644, 28.0199, 91.4151,
              6.5180,  62.8483, 72.4323, 46.5760, 8.6928,
              28.9821, 42.1828, 18.6437, 99.8612, 35.6972,
              67.9812, 5.0880,  85.5035, 79.2945, 54.5920,
              28.6869, 49.7512, 7.5186,  28.6929, 84.6041};

double b[] = {9.4532, 1.5204, 2.2127, 0.9891, 7.1778,
              6.8955, 7.2465, 3.5019, 8.2268, 3.5287};

double qwork;
int lwork = -1;
int info = 0;

dgels_(&trans, &m, &n, &nrhs, a, &lda, b, &ldb, &qwork, &lwork, &info);
if (info != 0) {
    printf("Error, info = %d\n", info);
    return info;
}

lwork = (int)qwork;
double *work = (double*)malloc(lwork * sizeof(double));
```

 版权所有 © 华为技术有限公司

```
dgels_(&trans, &m, &n, &nrhs, a, &lda, b, &ldb, work, &lwork, &info);
if (info != 0) {
    printf("Error, info = %d\n", info);
    return info;
}
/* output */
* a
* -120.698833     -108.770377     -57.958881      -100.842591
* 0.343169        75.924679       38.025488       -19.044303
* 0.335800        0.031671        7.685677        67.409117
* 0.145282        -0.313887       -0.556039       33.759113
* b
* 0.153366        0.083995
* -0.538653       0.096517
* 1.080726        -0.174382
* -0.145340       0.022261
```

Fortran Interface：

```
CHARACTER :: trans = "N"
PARAMETER (n = 4)
PARAMETER (m = 4)
PARAMETER (lda = 4)
PARAMETER (ldb = 4)
PARAMETER (nrhs = 2)

INTEGER :: info = 0
REAL(8) :: a(lda, n)
REAL(8) :: b(ldb, nrhs)
REAL(8), ALLOCATABLE :: work(:)
REAL(8) :: qwork(1)
INTEGER :: lwork = -1

DATA a / 72.1673, 66.1857, 64.7644, 28.0199, 91.4151,
&         6.5180,  62.8483, 72.4323, 46.5760, 8.6928,
&        28.9821, 42.1828, 18.6437, 99.8612, 35.6972,
&        67.9812, 5.0880,  85.5035, 79.2945, 54.5920,
&        28.6869, 49.7512, 7.5186,  28.6929, 84.6041 /
DATA b / 9.4532, 1.5204, 2.2127, 0.9891, 7.1778,
&        6.8955, 7.2465, 3.5019, 8.2268, 3.5287 /
EXTERNAL DGELS
CALL DGELS(trans, m, n, nrhs, a, lda, b, ldb, qwork, lwork, info)
IF (info.NE.0) THEN
    CALL EXIT(1)
END IF
lwork = INT(qwork(1))
ALLOCATE(work(lwork))
CALL DGELS(trans, m, n, nrhs, a, lda, b, ldb, work, lwork, info)
IF (info.NE.0) THEN
    CALL EXIT(1)
END IF
DEALLOCATE(work);
*
* Output:
*
a
* -120.698833     -108.770377     -57.958881      -100.842591
* 0.343169        75.924679       38.025488       -19.044303
* 0.335800        0.031671        7.685677        67.409117
* 0.145282        -0.313887       -0.556039       33.759113
* b
* 0.153366        0.083995
* -0.538653       0.096517
* 1.080726        -0.174382
* -0.145340       0.022261
```

6.5.1.3.8.2 ?gelss

使用SVD方法对线性最小二乘问题求解其最小范数解。

 版权所有 © 华为技术有限公司

接口定义

C Interface：

sgelss_(const int *m, const int *n, const int *nrhs, float *a, const int *lda, float *b, const int *ldb, float *s, const float *rcond, int *rank, float *work, const int *lwork, float *rwork, int *info);

dgelss_(const int *m, const int *n, const int *nrhs, double *a, const int *lda, double *b, const int *ldb, double *s, const double *rcond, int *rank, double *work, const int *lwork, double *rwork, int *info);

cgelss_(const int *m, const int *n, const int *nrhs, float _Complex *a, const int *lda, float _Complex *b, const int *ldb, float *s, const float *rcond, int *rank, float _Complex *work, const int *lwork, float *rwork, int *info);

zgelss_(const int *m, const int *n, const int *nrhs, double _Complex *a, const int *lda, double _Complex *b, const int *ldb, double *s, const double *rcond, int *rank, double _Complex *work, const int *lwork, double *rwork, int *info);

Fortran Interface：

SGELSS(M, N, NRHS, A, LDA, B, LDB, S, RCOND, RANK, WORK, LWORK, INFO)

DGELSS(M, N, NRHS, A, LDA, B, LDB, S, RCOND, RANK, WORK, LWORK, INFO)

CGELSS(M, N, NRHS, A, LDA, B, LDB, S, RCOND, RANK, WORK, LWORK, RWORK, INFO)

ZGELSS(M, N, NRHS, A, LDA, B, LDB, S, RCOND, RANK, WORK, LWORK, RWORK, INFO)

参数

参数	类型	说明	输入/输出
m	整数型	矩阵A的行数，m≥0。	输入
n	整数型	矩阵A的列数，n≥0。	输入
nrhs	整数型	右端项列数，nrhs≥0。	输入
a	• 在sgelss中为单精度浮点型数组。 • 在dgelss中为双精度浮点型数组。 • 在cgelss中为单精度复数型数组。 • 在zgelss中为双精度复数型数组。	大小为lda*n。 • 进入时：为m*n的矩阵。 • 退出时，A的前min(m, n)行将被其右奇异向量覆盖，并按行存储。	输入，输出
lda	整数型	矩阵A的主维，lda≥max(1, m)。	输入

版权所有 © 华为技术有限公司

参数	类型	说明	输入/输出
b	• 在sgelss中为单精度浮点型数组。 • 在dgelss中为双精度浮点型数组。 • 在cgelss中为单精度复数型数组。 • 在zgelss中为双精度复数型数组。	右端矩阵，大小为ldb*nrhs。 • 进入时：m*nrhs大小的右端项矩阵B。 • 退出时，B被N*NRHS大小的解矩阵X覆盖。如果m≥n且RANK=n，则第i列中解的剩余平方和由该列中元素n+1:m的平方和给出。	输入，输出
ldb	整数型	矩阵b的主维。	输入
s	• 在sgelss中为单精度浮点型数组。 • 在dgelss中为双精度浮点型数组。 • 在cgelss中为单精度复数型数组。 • 在zgelss中为双精度复数型数组。	A的奇异值按降序排列。 A的条件数二范数= S(1)/ S(min(m,n))。	输出
rcond	• 在sgelss中为单精度浮点型数。 • 在dgelss中为双精度浮点型数。 • 在cgelss中为单精度浮点型数。 • 在zgelss中为双精度浮点型数。	RCOND用于确定A的有效秩。 奇异值S(i)≤RCOND*S(1)被视为零。如果RCOND<0，则使用机器精度。	输入
rank	整数型	A的有效秩，即大于RCOND*S(1)的奇异值的数量。	输出
work	• 在sgelss中为单精度浮点型数组。 • 在dgelss中为双精度浮点型数组。 • 在cgelss中为单精度复数型数组。 • 在zgelss中为双精度复数型数组。	退出时，如果INFO=0，则work(0)返回最佳lwork。	输出
lwork	整数型	work的大小。	输入

 版权所有 © 华为技术有限公司

参数	类型	说明	输入/输出
rwork（复数类型特有）	• 在cgelss中为单精度浮点型数组。 • 在zgelss中为双精度浮点型数组。	大小至少为max(1, 5*min(m, n))。	输出
info	整数型	• 等于0：表示成功。 • 小于0：info=-i，表示第i个参数非法。 • 大于0：算法计算SVD时无法收敛，若info=i，则中间双对角线形式的i个对角线元素没有收敛到零。	输出

依赖

#include "klapack.h"

示例

C Interface：

```
const int m = 4;
const int n = 4;
const int nrhs = 2;
const int lda = 4;
const int ldb = 4;

double a[] = {72.1673, 66.1857, 64.7644, 28.0199, 91.4151,
              6.5180,  62.8483, 72.4323, 46.5760, 8.6928,
              28.9821, 42.1828, 18.6437, 99.8612, 35.6972,
              67.9812, 5.0880,  85.5035, 79.2945, 54.5920,
              28.6869, 49.7512, 7.5186,  28.6929, 84.6041};

double b[] = {9.4532, 1.5204, 2.2127, 0.9891, 7.1778,
              6.8955, 7.2465, 3.5019, 8.2268, 3.5287};

double *s = (double*)malloc(n * sizeof(double));
double rcond = -1.0;
int rank;

double qwork;
int lwork = -1;
int info = 0;

dgelss_(&m, &n, &nrhs, a, &lda, b, &ldb, s, &rcond, &rank, &qwork, &lwork, &info);
if (info != 0) {
    printf("Error, info = %d\n", info);
    return info;
}

lwork = (int)qwork;
double *work = (double*)malloc(lwork * sizeof(double));

dgelss_(&m, &n, &nrhs, a, &lda, b, &ldb, s, &rcond, &rank, work, &lwork, &info);
if (info != 0) {
    printf("Error, info = %d\n", info);
```

 版权所有 © 华为技术有限公司

```
    return info;
}

/* output */
* a
* -0.552769       -0.578412       -0.312159       -0.512292
* -0.073442       0.602089        0.251939        -0.754070
* -0.827844       0.298713        0.251050        0.403012
* -0.061050       0.462277        -0.880941       0.080724
* b
* 0.153366        0.083995
* -0.538653       0.096517
* 1.080726        -0.174382
* -0.145340       0.022261
* rank
* 4
```

Fortran Interface：

```
CHARACTER :: trans = "N"
PARAMETER (n = 4)
PARAMETER (m = 4)
PARAMETER (lda = 4)
PARAMETER (ldb = 4)
PARAMETER (nrhs = 2)

INTEGER :: info = 0
REAL(8) :: a(lda, n)
REAL(8) :: b(ldb, nrhs)
REAL(8) :: s(n)
REAL(8), ALLOCATABLE :: work(:)
REAL(8) :: rcond = -1.0
REAL(8) :: qwork(1)
INTEGER :: lwork = -1
INTEGER :: rank

DATA a / 72.1673, 66.1857, 64.7644, 28.0199, 91.4151,
&         6.5180,  62.8483, 72.4323, 46.5760, 8.6928,
&        28.9821, 42.1828, 18.6437, 99.8612, 35.6972,
&        67.9812, 5.0880,  85.5035, 79.2945, 54.5920,
&        28.6869, 49.7512, 7.5186,  28.6929, 84.6041 /
DATA b / 9.4532, 1.5204, 2.2127, 0.9891, 7.1778,
&        6.8955, 7.2465, 3.5019, 8.2268, 3.5287 /
EXTERNAL DGELSS
CALL DGELSS(m, n, nrhs, a, lda, b, ldb, s, rcond, rank, qwork, lwork, info)
IF (info.NE.0) THEN
   CALL EXIT(1)
END IF
lwork = INT(qwork(1))
ALLOCATE(work(lwork))
CALL DGELSS(m, n, nrhs, a, lda, b, ldb, s, rcond, rank, work, lwork, info)
IF (info.NE.0) THEN
   CALL EXIT(1)
END IF
DEALLOCATE(work);
*
* Output:
* a
* -0.552769       -0.578412       -0.312159       -0.512292
* -0.073442       0.602089        0.251939        -0.754070
* -0.827844       0.298713        0.251050        0.403012
* -0.061050       0.462277        -0.880941       0.080724
* b
* 0.153366        0.083995
* -0.538653       0.096517
* 1.080726        -0.174382
* -0.145340       0.022261
* rank
* 4
```

 版权所有 © 华为技术有限公司

6.5.1.3.8.3 ?gelsd

使用分治法对线性最小二乘问题求解其最小范数解。

接口定义

C Interface：

sgelsd_(const int *m, const int *n, const int *nrhs, float *a, const int *lda, float *b, const int *ldb, float *s, const float *rcond, int *rank, float *work, const int *lwork, float *rwork, int *iwork, int *info);

dgelsd_(const int *m, const int *n, const int *nrhs, double *a, const int *lda, double *b, const int *ldb, double *s, const double *rcond, int *rank, double *work, const int *lwork, double *rwork, int *iwork, int *info);

cgelsd_(const int *m, const int *n, const int *nrhs, float _Complex *a, const int *lda, float _Complex *b, const int *ldb, float *s, const float *rcond, int *rank, float _Complex *work, const int *lwork, float *rwork, int *iwork, int *info);

zgelsd_(const int *m, const int *n, const int *nrhs, double _Complex *a, const int *lda, double _Complex *b, const int *ldb, double *s, const double *rcond, int *rank, double _Complex *work, const int *lwork, double *rwork, int *iwork, int *info);

Fortran Interface：

SGELSD(M, N, NRHS, A, LDA, B, LDB, S, RCOND, RANK, WORK, LWORK, IWORK, INFO)

DGELSD(M, N, NRHS, A, LDA, B, LDB, S, RCOND, RANK, WORK, LWORK, IWORK, INFO)

CGELSD(M, N, NRHS, A, LDA, B, LDB, S, RCOND, RANK, WORK, LWORK, RWORK, IWORK, INFO)

ZGELSD(M, N, NRHS, A, LDA, B, LDB, S, RCOND, RANK, WORK, LWORK, RWORK, IWORK, INFO)

参数

参数	类型	说明	输入/输出
m	整数型	矩阵A的行数，m≥0。	输入
n	整数型	矩阵A的列数，n≥0。	输入
nrhs	整数型	右端项列数，nrhs≥0。	输入

 版权所有 © 华为技术有限公司

参数	类型	说明	输入/输出
a	• 在sgelsd中为单精度浮点型数组。 • 在dgelsd中为双精度浮点型数组。 • 在cgelsd中为单精度复数型数组。 • 在zgelsd中为双精度复数型数组。	大小为lda*n。 • 进入时：为m*n的矩阵。 • 退出时，A被销毁。	输入，输出
lda	整数型	矩阵A的主维，lda≥max(1, m)。	输入
b	• 在sgelsd中为单精度浮点型数组。 • 在dgelsd中为双精度浮点型数组。 • 在cgelsd中为单精度复数型数组。 • 在zgelsd中为双精度复数型数组。	右端矩阵，大小为ldb*nrhs。 • 进入时：m*nrhs大小的右端项矩阵B。 • 退出时，B被N*NRHS大小的解矩阵X覆盖。如果m≥n且RANK=n，则第i列中解的剩余平方和由该列中元素n+1:m的平方和给出。	输入，输出
ldb	整数型	矩阵b的主维。	输入
s	• 在sgelsd中为单精度浮点型数组。 • 在dgelsd中为双精度浮点型数组。 • 在cgelsd中为单精度复数型数组。 • 在zgelsd中为双精度复数型数组。	A的奇异值按降序排列。 A的条件数二范数= S(1)/ S(min(m,n))。	输出
rcond	• 在sgelsd中为单精度浮点型数。 • 在dgelsd中为双精度浮点型数。 • 在cgelsd中为单精度浮点型数。 • 在zgelsd中为双精度浮点型数。	RCOND用于确定A的有效秩。 奇异值S(i)≤RCOND*S(1)被视为零。如果RCOND<0，则使用机器精度。	输入
rank	整数型	A的有效秩，即大于RCOND*S(1)的奇异值的数量。	输出

版权所有 © 华为技术有限公司

参数	类型	说明	输入/输出
work	• 在sgelsd中为单精度浮点型数组。 • 在dgelsd中为双精度浮点型数组。 • 在cgelsd中为单精度复数型数组。 • 在zgelsd中为双精度复数型数组。	退出时，如果INFO=0，则work(0)返回最佳lwork。	输出
lwork	整数型	work的大小。	输入
rwork（复数类型特有）	• 在cgelsd中为单精度浮点型数组。 • 在zgelsd中为双精度浮点型数组。	大小至少为max(1, 5*min(m, n))。	输出
iwork	整数型数组	维度是max(1,liwork), liwork=max(1, 3*minmn*nlvl + 11*minmn), minmn=min(m,n)。退出时，如果info=0,iwork(0)返回最佳liwork。	输出
info	整数型	• =0：表示成功。 • <0：info=-i，表示第i个参数非法。 • >0：算法计算SVD时无法收敛，若info=i，则中间双对角线形式的i个对角线元素没有收敛到零。	输出

依赖

#include "klapack.h"

示例

C Interface：

```
const int m = 4;
const int n = 4;
const int nrhs = 2;
const int lda = 4;
const int ldb = 4;

double a[] = {72.1673, 66.1857, 64.7644, 28.0199, 91.4151,
              6.5180,  62.8483, 72.4323, 46.5760, 8.6928,
              28.9821, 42.1828, 18.6437, 99.8612, 35.6972,
              67.9812, 5.0880,  85.5035, 79.2945, 54.5920,
              28.6869, 49.7512, 7.5186,  28.6929, 84.6041};
```


版权所有 © 华为技术有限公司

```
double b[] = {9.4532, 1.5204, 2.2127, 0.9891, 7.1778,
              6.8955, 7.2465, 3.5019, 8.2268, 3.5287};

double *s = (double*)malloc(n * sizeof(double));
int *iwork = (int*)malloc((3 * n * n + 11 * n) * sizeof(int));
double rcond = -1.0;
int rank;

double qwork;
int lwork = -1;
int info = 0;

dgelsd_(&m, &n, &nrhs, a, &lda, b, &ldb, s, &rcond, &rank, &qwork, &lwork, iwork, &info);
if (info != 0) {
    printf("Error, info = %d\n", info);
    return info;
}

lwork = (int)qwork;
double *work = (double*)malloc(lwork * sizeof(double));

dgelsd_(&m, &n, &nrhs, a, &lda, b, &ldb, s, &rcond, &rank, work, &lwork, iwork, &info);
if (info != 0) {
    printf("Error, info = %d\n", info);
    return info;
}
/* output */
* a
* -120.698833     159.246523      0.216251        0.376255
* 0.343169        -73.504609      -11.813549      -0.661117
* 0.335800        0.090502        84.406748       25.275449
* 0.145282        -0.806922       -0.918078       -3.175151
* b
* 0.153366        0.083995
* -0.538653       0.096517
* 1.080726        -0.174382
* -0.145340       0.022261
* rank
* 4
```

Fortran Interface:

```
CHARACTER :: trans = "N"
PARAMETER (n = 4)
PARAMETER (m = 4)
PARAMETER (lda = 4)
PARAMETER (ldb = 4)
PARAMETER (nrhs = 2)

INTEGER :: info = 0
REAL(8) :: a(lda, n)
REAL(8) :: b(ldb, nrhs)
REAL(8) :: s(n)
INTEGER :: iwork(3*n*n+11*n)
REAL(8), ALLOCATABLE :: work(:)
REAL(8) :: rcond = -1.0
REAL(8) :: qwork(1)
INTEGER :: lwork = -1
INTEGER :: rank

DATA a / 72.1673, 66.1857, 64.7644, 28.0199, 91.4151,
&         6.5180,  62.8483, 72.4323, 46.5760, 8.6928,
&        28.9821, 42.1828, 18.6437, 99.8612, 35.6972,
&        67.9812, 5.0880,  85.5035, 79.2945, 54.5920,
&        28.6869, 49.7512, 7.5186,  28.6929, 84.6041 /
DATA b / 9.4532, 1.5204, 2.2127, 0.9891, 7.1778,
&        6.8955, 7.2465, 3.5019, 8.2268, 3.5287 /
EXTERNAL DGELSD
CALL DGELSD(m, n, nrhs, a, lda, b, ldb, s, rcond, rank, qwork, lwork, iwork, info)
```

 版权所有 © 华为技术有限公司

```
IF (info.NE.0) THEN
   CALL EXIT(1)
END IF
lwork = INT(qwork(1))
ALLOCATE(work(lwork))
CALL DGELSD(m, n, nrhs, a, lda, b, ldb, s, rcond, rank, qwork, lwork, iwork, info)
IF (info.NE.0) THEN
   CALL EXIT(1)
END IF
DEALLOCATE(work);
*
* Output:
* a
* -120.698833     159.246523      0.216251        0.376255
* 0.343169        -73.504609      -11.813549      -0.661117
* 0.335800        0.090502        84.406748       25.275449
* 0.145282        -0.806922       -0.918078       -3.175151
* b
* 0.153366        0.083995
* -0.538653       0.096517
* 1.080726        -0.174382
* -0.145340       0.022261
* rank
* 4
```

6.5.1.3.9 其他函数

6.5.1.3.9.1 ?(or,un)glq

生成具有正交行的实/复矩阵Q，且该矩阵定义为K个N阶基本反射器的乘积的前M行，即：Q=H(k)...H(2)H(1)，其中H为?gelqf返回的。

接口定义

C Interface：

void sorglq_(const int *m, const int *n, const int *k, float *a, const int *lda, const float *tau, float *work, const int *lwork, int *info);

void dorglq_(const int *m, const int *n, const int *k, double *a, const int *lda, const double *tau, double *work, const int *lwork, int *info);

void cunglq_(const int *m, const int *n, const int *k, float_Complex *a, const int *lda, const float_Complex *tau, float_Complex *work, const int *lwork, int *info);

void zunglq_(const int *m, const int *n, const int *k, double_Complex *a, const int *lda, const double_Complex *tau, double_Complex *work, const int *lwork, int *info);

Fortran Interface：

SORGLQ(M, N, K, A, LDA, TAU, WORK, LWORK, INFO);

DORGLQ(M, N, K, A, LDA, TAU, WORK, LWORK, INFO);

CUNGLQ(M, N, K, A, LDA, TAU, WORK, LWORK, INFO);

ZUNGLQ(M, N, K, A, LDA, TAU, WORK, LWORK, INFO);

 版权所有 © 华为技术有限公司

参数

参数名	类型	描述	输入/输出
m	整数型	矩阵Q的行数，m ≥ 0。	输入
n	整数型	矩阵Q的列数，n ≥ m。	输入
k	整数型	构成矩阵Q的基本反射器的数量，m ≥ k ≥0。	输入
a	• 在sorglq中为单精度浮点型数组。 • 在dorglq中为双精度浮点型数组。 • 在cunglq中为单精度复数型数组。 • 在zunglq中为双精度复数型数组。	维度(lda, n)。 • 作为输入时，第i行必须包含基本反射器H(i)向量，其中H(i)由?gelqf返回。 • 作为输出时，其为m*n的矩阵Q。	输入、输出
lda	整数型	A的leading dimension，lda ≥ max(1, m)。	输入
tau	• 在sorglq中为单精度浮点型数组。 • 在dorglq中为双精度浮点型数组。 • 在cunglq中为单精度复数型数组。 • 在zunglq中为双精度复数型数组。	tau(i)必须包含基本反射器H(i)的常量因子，其由?gelqf返回。	输入
work	• 在sorglq中为单精度浮点型数组。 • 在dorglq中为双精度浮点型数组。 • 在cunglq中为单精度复数型数组。 • 在zunglq中为双精度复数型数组。	维数(max(1,lwork))。若info=0，work(1)返回最优lwork值。	输出
lwork	整数型	数组work的维数。lwork ≥ max(1, m)，对于最优性能：lwork ≥ m*nb，其中nb为最优块大小如果lwork=-1，则该例程只计算work数组的最优大小，并以work数组的第一个值返回。	输入

版权所有 © 华为技术有限公司

参数名	类型	描述	输入/输出
info	整数型	• info=0：成功退出。 • info<0：第-info个参数值不合法。	输出

依赖

#include "klapack.h"

示例

C Interface：

```
int m = 4;
int n = 4;
int k = 4;
int lda = 4;
int info = 0;
double tau[4] = {1.003949, 1.125229, 1.978923, 0.000000};
double *work = NULL;
double qwork;
int lwork = -1;
/*
 * tau:
 *   1.003949  1.125229  1.978923  0.000000
 * A (4x4, stored in column-major):
 *   -1.036040  -0.892502   -0.120115  -0.425611
 *   0.654120   0.468127    0.644302   -0.601906
 *   -0.135860  0.502159    1.077637   -0.103204
 *   -0.480109  -1.207226   0.162668   0.227483
 */
double a[] = {-1.036040, 0.654120,  0.135860, -0.480109,
              -0.892502, 0.468127,  0.502159, -1.207226,
              -0.120115, 0.644302,  1.077637,  0.162668,
              -0.425611, -0.601906, -0.103204, 0.227483};
/* Query optimal work size */
dorglq_(&m, &n, &k, a, &lda, tau, &qwork, &lwork, &info);
if (info != 0) {
    return ERROR;
}
lwork = (int)qwork;
work = (double *)malloc(sizeof(double) * lwork);
/* Calculate Q */
dorglq_(&m, &n, &k, a, &lda, tau, work, &lwork, &info);
free(work);
/*
 * Output:
 * A output (stored in column-major)
 *   -0.003949  0.089763  0.576860   0.811886
 *   0.896027  -0.205343  0.333180   -0.209669
 *   0.427292   0.639078  -0.551720  0.323428
 *   0.120589  -0.735769  -0.501826  0.438490
 */
```

Fortran Interface：

```
    PARAMETER (m = 4)
    PARAMETER (n = 4)
    PARAMETER (k = 4)
    PARAMETER (lda = 4)
    INTEGER :: info = 0
```

 版权所有 © 华为技术有限公司

```
      REAL(8) :: tau(4)
      REAL(8) :: qwork(1)
      INTEGER :: lwork = -1
      REAL(8), ALLOCATABLE :: work(:)
*
*     tau:
*       1.003949  1.125229  1.978923  0.000000
*     A (4x4, stored in column-major):
*       -1.036040  -0.892502   -0.120115  -0.425611
*       0.654120   0.468127    0.644302   -0.601906
*       -0.135860  0.502159    1.077637   -0.103204
*       -0.480109  -1.207226   0.162668   0.227483

*
      DATA tau /1.003949, 1.125229, 1.978923, 0.000000/
      REAL(8) :: a(m, n)
      DATA a / -1.036040, 0.654120,  0.135860, -0.480109,
     $          -0.892502, 0.468127,  0.502159, -1.207226,
     $          -0.120115, 0.644302,  1.077637,  0.162668,
     $          -0.425611, -0.601906, -0.103204, 0.227483 /

      EXTERNAL DORGLQ
*     Query optimal work size
      CALL DORGLQ(m, n, k, a, lda, tau, qwork, lwork, info)
      IF (info.NE.0) THEN
         CALL EXIT(1)
      END IF
      lwork = INT(qwork(1))
      ALLOCATE(work(lwork))
*     Calculate Q
      CALL DORGLQ(m, n, k, a, lda, tau, work, lwork, info)
      DEALLOCATE(work)

*     Output:
*     A output (stored in column-major)
*      -0.003949  0.089763  0.576860   0.811886
*      0.896027  -0.205343  0.333180   -0.209669
*      0.427292   0.639078  -0.551720  0.323428
*      0.120589  -0.735769  -0.501826  0.438490
```

6.5.1.3.9.2 ?(or,un)gqr

生成具有正交行的实/复矩阵Q，且该矩阵定义为K个N阶基本反射器的乘积的前M行，即：Q=H(k)...H(2)H(1)，其中H为?geqrf返回的。

接口定义

C Interface：

void sorgqr_(const int *m, const int *n, const int *k, float *a, const int *lda, const float *tau, float *work, const int *lwork, int *info);

void dorgqr_(const int *m, const int *n, const int *k, double *a, const int *lda, const double *tau, double *work, const int *lwork, int *info);

void cungqr_(const int *m, const int *n, const int *k, float_Complex *a, const int *lda, const float_Complex *tau, float_Complex *work, const int *lwork, int *info);

void zungqr_(const int *m, const int *n, const int *k, double_Complex *a, const int *lda, const double_Complex *tau, double_Complex *work, const int *lwork, int *info);

Fortran Interface：

SORGQR(M, N, K, A, LDA, TAU, WORK, LWORK, INFO);

 版权所有 © 华为技术有限公司

DORGQR(M, N, K, A, LDA, TAU, WORK, LWORK, INFO);

CUNGQR(M, N, K, A, LDA, TAU, WORK, LWORK, INFO);

ZUNGQR(M, N, K, A, LDA, TAU, WORK, LWORK, INFO);

参数

参数名	类型	描述	输入/输出
m	整数型	矩阵Q的行数，m ≥ 0。	输入
n	整数型	矩阵Q的列数，m ≥ n ≥ 0。	输入
k	整数型	构成矩阵Q的基本反射器的数量，n ≥ k ≥ 0。	输入
a	• 在sorgqr中为单精度浮点型数组。 • 在dorgqr中为双精度浮点型数组。 • 在cungqr中为单精度复数型数组。 • 在zungqr中为双精度复数型数组。	维度(lda, n)。 • 作为输入时，第i列必须包含基本反射器H(i)向量，其中H(i)由?geqrf返回。 • 作为输出时，其为m*n的矩阵Q。	输入、输出
lda	整数型	A的leading dimension，lda ≥ max(1, m)。	输入
tau	• 在sorgqr中为单精度浮点型数组。 • 在dorgqr中为双精度浮点型数组。 • 在cungqr中为单精度复数型数组。 • 在zungqr中为双精度复数型数组。	tau(i)必须包含基本反射器H(i)的常量因子，其由?gelqf返回。	输入
work	• 在sorgqr中为单精度浮点型数组。 • 在dorgqr中为双精度浮点型数组。 • 在cungqr中为单精度复数型数组。 • 在zungqr中为双精度复数型数组。	维数(max(1,lwork))。 若info=0，work(1)返回最优lwork值。	输出

 版权所有 © 华为技术有限公司

参数名	类型	描述	输入/输出
lwork	整数型	数组work的维数。lwork ≥ max(1, n)，对于最优性能：lwork ≥ n*nb，其中nb为最优块大小如果lwork=-1，则该例程只计算work数组的最优大小，并以work数组的第一个值返回。	输入
info	整数型	• info=0：成功退出。 • info<0：第-info个参数值不合法。	输出

依赖

#include "klapack.h"

示例

C Interface：

```
int m = 3;
int n = 3;
int k = 3;
int lda = 3;
int info = 0;
double tau[3] = {1.421559, 1.259761, 0.000000};
double *work = NULL;
double qwork;
int lwork = -1;
/*
 * tau:
 *   1.421559  1.259761  0.000000
 * A (3x3, stored in column-major):
 *   -1.053312  -0.004846   1.108093
 *   0.372543   -1.483477   0.560874
 *   0.517801   -0.766553   0.127926
 */
double a[] = {-1.053312, 0.372543,  0.517801,
              -0.004846, -1.483477, -0.766553,
              1.108093,  0.560874,  0.127926};
/* Query optimal work size */
dorgqr_(&m, &n, &k, a, &lda, tau, &qwork, &lwork, &info);
if (info != 0) {
    return ERROR;
}
lwork = (int)qwork;
work = (double *)malloc(sizeof(double) * lwork);
/* Calculate Q */
dorgqr_(&m, &n, &k, a, &lda, tau, work, &lwork, &info);
free(work);
/*
 * Output:
 * A output (stored in column-major)
 *   -0.421559  -0.529592  -0.736085
 *   -0.573250  -0.473321  0.668844
 *   -0.702618  0.703918   -0.104055
 */
```

 版权所有 © 华为技术有限公司

Fortran Interface：

```
      PARAMETER (m = 3)
      PARAMETER (n = 3)
      PARAMETER (k = 3)
      PARAMETER (lda = 3)
      INTEGER :: info = 0
      REAL(8) :: tau(3)
      REAL(8) :: qwork(1)
      INTEGER :: lwork = -1
      REAL(8), ALLOCATABLE :: work(:)
*
*     tau:
*       1.421559  1.259761  0.000000
*     A (3x3, stored in column-major):
*       -1.053312  -0.004846   1.108093
*       0.372543   -1.483477   0.560874
*       0.517801   -0.766553   0.127926

*
      DATA tau /1.421559, 1.259761, 0.000000/
      REAL(8) :: a(m, n)
      DATA a / -1.053312, 0.372543,  0.517801,
     $          -0.004846, -1.483477, -0.766553,
     $          1.108093,  0.560874,  0.127926 /

      EXTERNAL DORGQR
*     Query optimal work size
      CALL DORGQR(m, n, k, a, lda, tau, qwork, lwork, info)
      IF (info.NE.0) THEN
         CALL EXIT(1)
      END IF
      lwork = INT(qwork(1))
      ALLOCATE(work(lwork))
*     Calculate Q
      CALL DORGQR(m, n, k, a, lda, tau, work, lwork, info)
      DEALLOCATE(work)

*     Output:
*     A output (stored in column-major)
*      -0.421559  -0.529592  -0.736085
*      -0.573250  -0.473321  0.668844
*      -0.702618  0.703918   -0.104055
```

6.5.1.3.9.3 ?(or,un)grq

生成具有正交行的实/复矩阵Q，且该矩阵定义为K个N阶基本反射器的乘积的前M行，即：Q=H(1)H(2)…H(k)，其中H为?gerqf返回的。

接口定义

C Interface：

void sorgrq_(const int *M, const int *N, const int *K, float *A, const int *lda, float *tau, float *work, const int *lwork, int *info);

void dorgrq_(const int *M, const int *N, const int *K, double *A, const int *lda, double *tau, double *work, const int *lwork, int *info);

void cungrq_(const int *M, const int *N, const int *K, float _Complex *A, const int *lda, float _Complex *tau, float _Complex *work, const int *lwork, int *info);

void zungrq_(const int *M, const int *N, const int *K, double _Complex *A, const int *lda, double _Complex *tau, double _Complex *work, const int *lwork, int *info);

Fortran Interface：

 版权所有 © 华为技术有限公司

SORGRQ(M,N,K,A,LDA,TAU,WORK,LWORK,INFO);

DORGRQ(M,N,K,A,LDA,TAU,WORK,LWORK,INFO);

CUNGRQ(M,N,K,A,LDA,TAU,WORK,LWORK,INFO);

ZUNGRQ(M,N,K,A,LDA,TAU,WORK,LWORK,INFO);

参数

参数名	类型	描述	输入/输出
M	整数型	Q的行数，M ≥ 0。	输入
N	整数型	Q的列数，N ≥ 0。	输入
K	整数型	构成矩阵Q的基本反射器的数量，M ≥ K ≥ 0。	输入
A	• 在sorgrq中为单精度浮点型数组。 • 在dorgrq中为双精度浮点型数组。 • 在cungrq中为单精度复数型数组。 • 在zungrq中为双精度复数型数组。	维度(lda, n)。 • 作为输入时，第i列必须包含基本反射器H(i)向量，其中H(i)由?gerqf返回。 • 作为输出时，其为m*n的矩阵Q。	输入、输出
lda	整数型	矩阵A的主维，lda ≥ max(1,M)。	输入
tau	• 在sorgrq中为单精度浮点型数组。 • 在dorgrq中为双精度浮点型数组。 • 在cungrq中为单精度复数型数组。 • 在zungrq中为双精度复数型数组。	tau(i)必须包含基本反射器H(i)的常量因子，其由?gerqf返回。	输入
work	• 在sorgrq中为单精度浮点型数组。 • 在dorgrq中为双精度浮点型数组。 • 在cungrq中为单精度复数型数组。 • 在zungrq中为双精度复数型数组。	维数(max(1,lwork))。 若info=0，work(1)返回最优lwork值。	输出

 版权所有 © 华为技术有限公司

参数名	类型	描述	输入/输出
lwork	整数型	数组work的维数。 lwork ≥ max(1, M)，对于最优性能：lwork ≥ M*NB，其中NB为最优块大小如果lwork=-1，则该例程只计算work数组的最优大小，并以work数组的第一个值返回。	输入
info	整数型	• info=0：成功退出。 • info<0：第-info个参数值不合法。	输出

依赖

#include "klapack.h"

示例

C Interface：

```
int m = 4;
int n = 4;
int k = 4;
int lda = m;
int info = 0;
double tau[4] = {1.003949, 1.125229, 1.978923, 0.000000};
double *work = NULL;
double qwork;
int lwork = -1;
/*
 * tau:
 *   1.003949  1.125229  1.978923  0.000000
 * A (4x4, stored in column-major):
 *   -1.036040  -0.892502   -0.120115  -0.425611
 *   0.654120   0.468127    0.644302   -0.601906
 *   -0.135860  0.502159    1.077637   -0.103204
 *   -0.480109  -1.207226   0.162668   0.227483

 *
 */
double a[] = {-1.036040, 0.654120,  0.135860, -0.480109,
              -0.892502, 0.468127,  0.502159, -1.207226,
              -0.120115, 0.644302,  1.077637,  0.162668,
              -0.425611, -0.601906, -0.103204, 0.227483};

/* Query optimal work size */
dorgrq_(&m, &n, &k, a, &lda, tau, &qwork, &lwork, &info);
if (info != 0) {
    return ERROR;
}
```

 版权所有 © 华为技术有限公司

```
lwork = (int)qwork;
work = (double *)malloc(sizeof(double) * lwork);
/* Calculate Q */
dorgrq_(&m, &n, &k, a, &lda, tau, work, &lwork, &info);
free(work);
/*
 * Output:
 * A output (stored in column-major)
 *   -0.002365       -0.692243       -0.268856       -0.000000
 *   0.001733        0.036633        -0.993734       -0.000000
 *   -0.002338       0.322332        -0.978923       -0.000000
 *   0.000000        0.000000        0.000000        1.000000
 */
```

Fortran Interface：

```
      PARAMETER (m = 4)
      PARAMETER (n = 4)
      PARAMETER (k = 4)
      PARAMETER (lda = 4)
      INTEGER :: info = 0
      REAL(8) :: tau(4)
      REAL(8) :: qwork(1)
      INTEGER :: lwork = -1
      REAL(8), ALLOCATABLE :: work(:)
*
*     tau:
*       1.003949  1.125229  1.978923  0.000000
*     A (4x4, stored in column-major):
*       -1.036040  -0.892502   -0.120115  -0.425611
*       0.654120   0.468127    0.644302   -0.601906
*       -0.135860  0.502159    1.077637   -0.103204
*       -0.480109  -1.207226   0.162668   0.227483

*
      DATA tau /1.003949, 1.125229, 1.978923, 0.000000/
      REAL(8) :: a(m, n)
      DATA a / -1.036040, 0.654120,  0.135860, -0.480109,
     $          -0.892502, 0.468127,  0.502159, -1.207226,
     $          -0.120115, 0.644302,  1.077637,  0.162668,
     $          -0.425611, -0.601906, -0.103204, 0.227483 /

      EXTERNAL DORGRQ
*     Query optimal work size
      CALL DORGRQ(m, n, k, a, lda, tau, qwork, lwork, info)
      IF (info.NE.0) THEN
         CALL EXIT(1)
      END IF
      lwork = INT(qwork(1))
      ALLOCATE(work(lwork))
*     Calculate Q
      CALL DORGRQ(m, n, k, a, lda, tau, work, lwork, info)
      DEALLOCATE(work)
*
* Output:
* A output (stored in column-major)
*   -0.002365       -0.692243       -0.268856       -0.000000
*   0.001733        0.036633        -0.993734       -0.000000
*   -0.002338       0.322332        -0.978923       -0.000000
*   0.000000        0.000000        0.000000        1.000000
```

6.5.1.3.9.4 ?(or,un)gql

生成具有正交列的实/复矩阵Q，且该矩阵定义为K个M阶基本反射器的乘积的前N列，即：Q=H(k)...H(2)H(1)，其中H为?geqlf返回的。

版权所有 © 华为技术有限公司

接口定义

C Interface：

void sorgql_(const int *m, const int *n, const int *k, float *a, const int *lda, float *tau, float *work, const int *lwork, int *info);

void dorgql_(const int *m, const int *n, const int *k, double *a, const int *lda, double *tau, double *work, const int *lwork, int *info);

void cungql_(const int *m, const int *n, const int *k, float _Complex *a, const int *lda, float _Complex *tau, float _Complex *work, const int *lwork, int *info);

void zungql_(const int *m, const int *n, const int *k, double _Complex *a, const int *lda, double _Complex *tau, double _Complex *work, const int *lwork, int *info);

Fortran Interface：

SORGQL(M,N,K,A,LDA,TAU,WORK,LWORK,INFO);

DORGQL(M,N,K,A,LDA,TAU,WORK,LWORK,INFO);

CUNGQL(M,N,K,A,LDA,TAU,WORK,LWORK,INFO);

ZUNGQL(M,N,K,A,LDA,TAU,WORK,LWORK,INFO);

参数

参数名	类型	描述	输入/输出
m	整数型	Q的行数，m ≥ 0。	输入
n	整数型	Q的列数，m ≥ n ≥ 0。	输入
k	整数型	构成矩阵Q的基本反射器的数量，n ≥ k ≥ 0。	输入
a	• 在sorgql中为单精度浮点型数组。 • 在dorgql中为双精度浮点型数组。 • 在cungql中为单精度复数型数组。 • 在zungql中为双精度复数型数组。	维度(lda, n)。 • 作为输入时，第i列必须包含基本反射器H(i)向量，其中H(i)由?geqlf返回。 • 作为输出时，其为m*n的矩阵Q。	输入、输出
lda	整数型	矩阵A的主维， lda ≥ max(1,M)。	输入

版权所有 © 华为技术有限公司

参数名	类型	描述	输入/输出
tau	• 在sorgql中为单精度浮点型数组。 • 在dorgql中为双精度浮点型数组。 • 在cungql中为单精度复数型数组。 • 在zungql中为双精度复数型数组。	tau(i)必须包含基本反射器H(i)的常量因子，其由?geqlf返回。	输入
work	• 在sorgql中为单精度浮点型数组。 • 在dorgql中为双精度浮点型数组。 • 在cungql中为单精度复数型数组。 • 在zungql中为双精度复数型数组。	维数(max(1,lwork))。 若info=0，work(1)返回最优lwork值。	输出
lwork	整数型	数组work的维数。 lwork ≥ max(1, N)，对于最优性能：lwork ≥ N*NB，其中NB为最优块大小如果lwork=-1，则该例程只计算work数组的最优大小，并以work数组的第一个值返回。	输入
info	整数型	• info=0：成功退出。 • info<0：第-info个参数值不合法。	输出

依赖

#include "klapack.h"

示例

C Interface：

```
int m = 4;
int n = 4;
int k = 4;
int lda = m;
```

 版权所有 © 华为技术有限公司

```
int info = 0;
double tau[4] = {1.003949, 1.125229, 1.978923, 0.000000};
double *work = NULL;
double qwork;
int lwork = -1;
/*
 * tau:
 *   1.003949  1.125229  1.978923  0.000000
 * A (4x4, stored in column-major):
 *   -1.036040  -0.892502   -0.120115  -0.425611
 *   0.654120   0.468127    0.644302   -0.601906
 *   -0.135860  0.502159    1.077637   -0.103204
 *   -0.480109  -1.207226   0.162668   0.227483

 *
 */
double a[] = {-1.036040, 0.654120,  0.135860, -0.480109,
              -0.892502, 0.468127,  0.502159, -1.207226,
              -0.120115, 0.644302,  1.077637,  0.162668,
              -0.425611, -0.601906, -0.103204, 0.227483};

/* Query optimal work size */
dorgql_(&m, &n, &k, a, &lda, tau, &qwork, &lwork, &info);
if (info != 0) {
    return ERROR;
}
lwork = (int)qwork;
work = (double *)malloc(sizeof(double) * lwork);
/* Calculate Q */
dorgql_(&m, &n, &k, a, &lda, tau, work, &lwork, &info);
free(work);
/*
 * Output:
 * A output (stored in column-major)
 *   -0.001005       -0.000771       0.004959        0.000000
 *   0.956417        0.131450        0.398383        0.000000
 *   0.237698        -1.275024       -0.978923       0.000000
 *   -0.000000       -0.000000       -0.000000       1.000000
 */
```

Fortran Interface：

```
      PARAMETER (m = 4)
      PARAMETER (n = 4)
      PARAMETER (k = 4)
      PARAMETER (lda = 4)
      INTEGER :: info = 0
      REAL(8) :: tau(4)
      REAL(8) :: qwork(1)
      INTEGER :: lwork = -1
      REAL(8), ALLOCATABLE :: work(:)
*
*     tau:
*       1.003949  1.125229  1.978923  0.000000
*     A (4x4, stored in column-major):
*       -1.036040  -0.892502   -0.120115  -0.425611
*       0.654120   0.468127    0.644302   -0.601906
*       -0.135860  0.502159    1.077637   -0.103204
*       -0.480109  -1.207226   0.162668   0.227483

*
      DATA tau /1.003949, 1.125229, 1.978923, 0.000000/
      REAL(8) :: a(m, n)
      DATA a / -1.036040, 0.654120,  0.135860, -0.480109,
     $         -0.892502, 0.468127,  0.502159, -1.207226,
     $         -0.120115, 0.644302,  1.077637,  0.162668,
     $         -0.425611, -0.601906, -0.103204, 0.227483 /

      EXTERNAL DORGQL
*     Query optimal work size
```

 版权所有 © 华为技术有限公司

```
      CALL DORGQL(m, n, k, a, lda, tau, qwork, lwork, info)
      IF (info.NE.0) THEN
         CALL EXIT(1)
      END IF
      lwork = INT(qwork(1))
      ALLOCATE(work(lwork))
*     Calculate Q
      CALL DORGQL(m, n, k, a, lda, tau, work, lwork, info)
      DEALLOCATE(work)
*
* Output:
* A output (stored in column-major)
*   -0.001005       -0.000771       0.004959        0.000000
*   0.956417        0.131450        0.398383        0.000000
*   0.237698        -1.275024       -0.978923       0.000000
*   -0.000000       -0.000000       -0.000000       1.000000
```

6.5.1.3.9.5 ?(or,un)mlq

计算$C = Q^* * C$或$C = C * Q^*$，其中Q是由?gelqf计算得到。当Q转置，实数时Q^*表示Q^T，复数时Q^*表示Q^H；当Q不转置，Q^*表示Q。

接口定义

C Interface：

void sormlq_(const char *uplo, const char *trans, const int *m, const int *n, const int *k, float *a, const int *lda, const float *tau, float *c, const int *ldc, float *work, const int *lwork, int *info);

void dormlq_(const char *uplo, const char *trans, const int *m, const int *n, const int *k, double *a, const int *lda, const double *tau, double *c, const int *ldc, double *work, const int *lwork, int *info);

void cunmlq_(const char *uplo, const char *trans, const int *m, const int *n, const int *k, float _Complex *a, const int *lda, const float _Complex *tau, float _Complex *c, const int *ldc, float _Complex *work, const int *lwork, int *info);

void zunmlq_(const char *uplo, const char *trans, const int *m, const int *n, const int *k, double _Complex *a, const int *lda, const double _Complex *tau, double _Complex *c, const int *ldc, double _Complex *work, const int *lwork, int *info);

Fortran Interface：

SORMLQ(side,trans,m,n,k,a,lda,tau,c,ldc,work,lwork,info);

DORMLQ(side,trans,m,n,k,a,lda,tau,c,ldc,work,lwork,info);

CUNMLQ(side,trans,m,n,k,a,lda,tau,c,ldc,work,lwork,info);

ZUNMLQ(side,trans,m,n,k,a,lda,tau,c,ldc,work,lwork,info);

 版权所有 © 华为技术有限公司

参数

参数名	类型	描述	输入/输出
side	字符型	• "L"：Q*在左边。 • "R"：Q*在右边。	输入
trans	字符型	• "N"：Q不转置。 • "T"：Q转置。	输入
M	整数型	C的行数，M ≥ 0。	输入
N	整数型	C的列数，N ≥ 0。	输入
K	整数型	生成Q的元素反射器的个数。 • side="L"时，M ≥ K ≥ 0。 • side="R"时，N ≥ K ≥ 0。	输入
A	• 在sormlq中为单精度浮点型数组。 • 在dormlq中为双精度浮点型数组。 • 在cunmlq中为单精度复数型数组。 • 在zunmlq中为双精度复数型数组。	原始矩阵。 • 当side="L"时，维数为（LDA, M）。 • 当side="R"时，维数为（LDA,N）。	输入
lda	整数型	矩阵A的主维，LDA ≥ max(1,k)。	输入

版权所有 © 华为技术有限公司

参数名	类型	描述	输入/输出
tau	• 在sormlq中为单精度浮点型数组。 • 在dormlq中为双精度浮点型数组。 • 在cunmlq中为单精度复数型数组。 • 在zunmlq中为双精度复数型数组。	tau(i)必须包含基本反射器H(i)的常量因子，其由?gelqf返回。	输入
C	• 在sormlq中为单精度浮点型数组。 • 在dormlq中为双精度浮点型数组。 • 在cunmlq中为单精度复数型数组。 • 在zunmlq中为双精度复数型数组。	待计算矩阵，维数为（ LDC, N ）。	输入、输出
ldc	整数型	矩阵C的主维度，ldc ≥ max(1,M)。	输入
work	• 在sormlq中为单精度浮点型数组。 • 在dormlq中为双精度浮点型数组。 • 在cunmlq中为单精度复数型数组。 • 在zunmlq中为双精度复数型数组。	维数(max(1,lwork))。 若info=0，work(1)返回最优lwork值。	输出

版权所有 © 华为技术有限公司

参数名	类型	描述	输入/输出
lwork	整数型	work的大小。 ● 当side="L"时，lwork ≥ max(1,N)。 ● 当side="R"时，lwork ≥ max(1,M)。 如果lwork=-1，则该例程只计算work数组的最优大小，并以work数组的第一个值返回。	输入
info	整数型	● info=0：成功退出。 ● info<0：第-info个参数值不合法。	输出

依赖

#include "klapack.h"

示例

C Interface：

```
char side = 'L';
char trans = 'N';
int m = 4;
int n = 4;
int k = 4;
int lda = 4;
int ldc = 4;
int info = 0;
double tau[4] = {1.003949, 1.125229, 1.978923, 0.000000};
double *work = NULL;
double qwork;
int lwork = -1;
/*
 * tau:
 *   1.003949  1.125229  1.978923  0.000000
 * A (4x4, stored in column-major):
 *   -1.036040  -0.892502   -0.120115  -0.425611
 *   0.654120   0.468127    0.644302   -0.601906
 *   -0.135860  0.502159    1.077637   -0.103204
 *   -0.480109  -1.207226   0.162668   0.227483

 *
 *C (4x4, stored in column-major):
 *   2.229, 8.103, 7.810, 0.087,
 *   8.667, 8.859, 2.911, 4.091,
 *   0.205, 1.273, 1.396, 2.553,
 *   2.758, 4.317, 3.161, 8.791
 */
```

 版权所有 © 华为技术有限公司

```
double a[] = {-1.036040, 0.654120,  0.135860, -0.480109,
              -0.892502, 0.468127,  0.502159, -1.207226,
              -0.120115, 0.644302,  1.077637,  0.162668,
              -0.425611, -0.601906, -0.103204, 0.227483};
double c[] = {2.229, 8.667, 0.205, 2.758,
              8.103, 8.859, 1.273, 4.317,
              7.810, 2.911, 1.396, 3.161,
              0.087, 4.091, 2.553, 8.791};
/* Query optimal work size */
dormlq_(&side, &trans, &m, &n, &k, a, &lda, tau, c, &ldc, &qwork, &lwork, &info);
if (info != 0) {
   return ERROR;
}
lwork = (int)qwork;
work = (double *)malloc(sizeof(double) * lwork);
/* Calculate Q */
dormlq_(&side, &trans, &m, &n, &k, a, &lda, tau, c, &ldc, work, &lwork, &info);
free(work);
/*
 * Output:
 * C output (stored in column-major)
 *   8.960251        0.032123        2.548981        0.974400
 *   9.904029        0.730485        4.605351        6.675692
 *   4.096503        1.096288        3.030637        7.364972
 *   7.729487        2.907470        -4.718100       3.175610
 */
```

Fortran Interface：

```
      CHARACTER :: side = "L"
      CHARACTER :: trans = "N"
      PARAMETER (m = 4)
      PARAMETER (n = 4)
      PARAMETER (k = 4)
      PARAMETER (lda = 4)
      PARAMETER (ldc = 4)
      INTEGER :: info = 0
      REAL(8) :: tau(4)
      REAL(8) :: qwork(1)
      INTEGER :: lwork = -1
      REAL(8), ALLOCATABLE :: work(:)
*
*     tau:
*       1.003949  1.125229  1.978923  0.000000
*     A (4x4, stored in column-major):
*       -1.036040  -0.892502   -0.120115  -0.425611
*       0.654120   0.468127    0.644302   -0.601906
*       -0.135860  0.502159    1.077637   -0.103204
*       -0.480109  -1.207226   0.162668   0.227483

*     C (4x4, stored in column-major):
*       2.229, 8.103, 7.810, 0.087,
*       8.667, 8.859, 2.911, 4.091,
*       0.205, 1.273, 1.396, 2.553,
*       2.758, 4.317, 3.161, 8.791

*
      DATA tau /1.003949, 1.125229, 1.978923, 0.000000/
      REAL(8) :: a(m, n)
      DATA a / -1.036040, 0.654120,  0.135860, -0.480109,
     $         -0.892502, 0.468127,  0.502159, -1.207226,
     $         -0.120115, 0.644302,  1.077637,  0.162668,
     $         -0.425611, -0.601906, -0.103204, 0.227483 /

      DATA c / 2.229, 8.667, 0.205, 2.758,
     $         8.103, 8.859, 1.273, 4.317,
     $         7.810, 2.911, 1.396, 3.161,
     $         0.087, 4.091, 2.553, 8.791
      EXTERNAL DORMLQ
*     Query optimal work size
```


版权所有 © 华为技术有限公司

```
      CALL DORMLQ(side, trans, m, n, k, a, lda, tau, c, ldc, qwork, lwork, info)
      IF (info.NE.0) THEN
         CALL EXIT(1)
      END IF
      lwork = INT(qwork(1))
      ALLOCATE(work(lwork))
*     Calculate Q
      CALL DORMLQ(side, trans, m, n, k, a, lda, tau, c, ldc, work, lwork, info)
      DEALLOCATE(work)
*
* Output:
* C output (stored in column-major)
*   8.960251       0.032123       2.548981       0.974400
*   9.904029       0.730485       4.605351       6.675692
*   4.096503       1.096288       3.030637       7.364972
*   7.729487       2.907470      -4.718100       3.175610
*
```

6.5.1.3.9.6 ?(or,un)mqr

计算$C = Q^* * C$或$C = C * Q^*$，其中Q是由?geqrf计算得到。当Q转置，实数时Q^*表示Q^T，复数时Q^*表示Q^H；当Q不转置，Q^*表示Q。

接口定义

C Interface：

void sormqr_(const char *side, const char *trans, const int *m, const int *n, const int *k, float *a, const int *lda, float *tau, float *c, const int *ldc, float *work, const int *lwork, int *info);

void dormqr_(const char *side, const char *trans, const int *m, const int *n, const int *k, double *a, const int *lda, double *tau, double *c, const int *ldc, double *work, const int *lwork, int *info);

void cunmqr_(const char *side, const char *trans, const int *m, const int *n, const int *k, float _Complex *a, const int *lda, float _Complex *tau, float _Complex *c, const int *ldc, float _Complex *work, const int *lwork, int *info);

void zunmqr_(const char *side, const char *trans, const int *m, const int *n, const int *k, double _Complex *a, const int *lda, double _Complex *tau, double _Complex *c, const int *ldc, double _Complex *work, const int *lwork, int *info);

Fortran Interface：

SORMQR(SIDE,TRANS,M,N,K,A,LDA,TAU,C,LDC,WORK, LWORK, INFO);

DORMQR(SIDE,TRANS,M,N,K,A,LDA,TAU,C,LDC,WORK, LWORK, INFO);

CUNMQR(SIDE,TRANS,M,N,K,A,LDA,TAU,C,LDC,WORK, LWORK, INFO);

ZUNMQR(SIDE,TRANS,M,N,K,A,LDA,TAU,C,LDC,WORK, LWORK, INFO);

版权所有 © 华为技术有限公司

参数

参数名	类型	描述	输入/输出
side	字符型	• "L"：Q*在左边。 • "R"：Q*在右边。	输入
trans	字符型	• "N"：Q不转置。 • "T"：Q转置。	输入
M	整数型	C的行数，M ≥ 0。	输入
N	整数型	C的列数，N ≥ 0。	输入
K	整数型	生成Q的元素反射器的个数。 • side="L"时，M ≥ K ≥ 0。 • side="R"时，N ≥ K ≥ 0。	输入
A	• 在sormqr中为单精度浮点型数组。 • 在dormqr中为双精度浮点型数组。 • 在cunmqr中为单精度复数型数组。 • 在zunmqr中为双精度复数型数组。	原始矩阵。	输入
lda	整数型	矩阵A的主维度。 • 当side="L"时，lda ≥ max(1,M)。 • 当side="R"时，lda ≥ max(1,N)。	输入
tau	• 在sormqr中为单精度浮点型数组。 • 在dormqr中为双精度浮点型数组。 • 在cunmqr中为单精度复数型数组。 • 在zunmqr中为双精度复数型数组。	tau(i)必须包含基本反射器H(i)的常量因子，其由?geqrf返回。	输入

版权所有 © 华为技术有限公司

参数名	类型	描述	输入/输出
C	• 在sormqr中为单精度浮点型数组。 • 在dormqr中为双精度浮点型数组。 • 在cunmqr中为单精度复数型数组。 • 在zunmqr中为双精度复数型数组。	待计算矩阵，维数为（ldc, N ）。	输入、输出
ldc	整数型	矩阵C的主维度，ldc ≥ max(1,M)。	输入
work	• 在sormqr中为单精度浮点型数组。 • 在dormqr中为双精度浮点型数组。 • 在cunmqr中为单精度复数型数组。 • 在zunmqr中为双精度复数型数组。	维数(max(1,lwork))。 若info=0，work(1)返回最优lwork值。	输出
lwork	整数型	work的大小。 • 当side="L"时，lwork ≥ max(1,N)。 • 当side="R"时，lwork ≥ max(1,M)。 如果lwork=-1，则该例程只计算work数组的最优大小，并以work数组的第一个值返回。	输入
info	整数型	• info=0：成功退出。 • info<0：第-info个参数值不合法。	输出

依赖

#include "klapack.h"

示例

C Interface：

 版权所有 © 华为技术有限公司

```
char side = 'L';
char trans = 'N';
int m = 4;
int n = 4;
int k = 4;
int lda = 4;
int ldc = 4;
int info = 0;
double tau[4] = {1.003949, 1.125229, 1.978923, 0.000000};
double *work = NULL;
double qwork;
int lwork = -1;
/*
 * tau:
 *   1.003949  1.125229  1.978923  0.000000
 * A (4x4, stored in column-major):
 *   -1.036040  -0.892502   -0.120115  -0.425611
 *   0.654120   0.468127    0.644302   -0.601906
 *   -0.135860  0.502159    1.077637   -0.103204
 *   -0.480109  -1.207226   0.162668   0.227483

 *
 *C (4x4, stored in column-major):
 *   2.229, 8.103, 7.810, 0.087,
 *   8.667, 8.859, 2.911, 4.091,
 *   0.205, 1.273, 1.396, 2.553,
 *   2.758, 4.317, 3.161, 8.791

 */
double a[] = {-1.036040, 0.654120,  0.135860, -0.480109,
              -0.892502, 0.468127,  0.502159, -1.207226,
              -0.120115, 0.644302,  1.077637,  0.162668,
              -0.425611, -0.601906, -0.103204, 0.227483};
double c[] = {2.229, 8.667, 0.205, 2.758,
              8.103, 8.859, 1.273, 4.317,
              7.810, 2.911, 1.396, 3.161,
              0.087, 4.091, 2.553, 8.791};
/* Query optimal work size */
dormqr_(&side, &trans, &m, &n, &k, a, &lda, tau, c, &ldc, &qwork, &lwork, &info);
if (info != 0) {
   return ERROR;
}
lwork = (int)qwork;
work = (double *)malloc(sizeof(double) * lwork);
/* Calculate Q */
dormqr_(&side, &trans, &m, &n, &k, a, &lda, tau, c, &ldc, &qwork, &lwork, &info);
free(work);
/*
 * Output:
 * C output (stored in column-major)
 *   3.095833        3.557362        -3.821313       8.984414
 *   0.830861        0.623616        -5.370613       11.371951
 *   -2.390220       -2.230944       -3.001512       5.595986
 *   -9.592006       6.367668        -2.321534       1.764135
 */
```

Fortran Interface:

```
    CHARACTER :: side = "L"
    CHARACTER :: trans = "N"
    PARAMETER (m = 4)
    PARAMETER (n = 4)
    PARAMETER (k = 4)
    PARAMETER (lda = 4)
    PARAMETER (ldc = 4)
    INTEGER :: info = 0
    REAL(8) :: tau(4)
    REAL(8) :: qwork(1)
    INTEGER :: lwork = -1
    REAL(8), ALLOCATABLE :: work(:)
```

 版权所有 © 华为技术有限公司

```
*
*      tau:
*        1.003949  1.125229  1.978923  0.000000
*      A (4x4, stored in column-major):
*        -1.036040  -0.892502   -0.120115  -0.425611
*        0.654120   0.468127    0.644302   -0.601906
*        -0.135860  0.502159    1.077637   -0.103204
*        -0.480109  -1.207226   0.162668   0.227483

*      C (4x4, stored in column-major):
*        2.229, 8.103, 7.810, 0.087,
*        8.667, 8.859, 2.911, 4.091,
*        0.205, 1.273, 1.396, 2.553,
*        2.758, 4.317, 3.161, 8.791

*
       DATA tau /1.003949, 1.125229, 1.978923, 0.000000/
       REAL(8) :: a(m, n)
       DATA a / -1.036040, 0.654120,  0.135860, -0.480109,
     $          -0.892502, 0.468127,  0.502159, -1.207226,
     $          -0.120115, 0.644302,  1.077637,  0.162668,
     $          -0.425611, -0.601906, -0.103204, 0.227483 /

       DATA c / 2.229, 8.667, 0.205, 2.758,
     $          8.103, 8.859, 1.273, 4.317,
     $          7.810, 2.911, 1.396, 3.161,
     $          0.087, 4.091, 2.553, 8.791
       EXTERNAL DORMQR
*      Query optimal work size
       CALL DORMQR(side, trans, m, n, k, a, lda, tau, c, ldc, qwork, lwork, info)
       IF (info.NE.0) THEN
          CALL EXIT(1)
       END IF
       lwork = INT(qwork(1))
       ALLOCATE(work(lwork))
*      Calculate Q
       CALL DORMQR(side, trans, m, n, k, a, lda, tau, c, ldc, work, lwork, info)
       DEALLOCATE(work)
*
* Output:
* C output (stored in column-major)
*   3.095833       3.557362       -3.821313       8.984414
*   0.830861       0.623616       -5.370613       11.371951
*   -2.390220      -2.230944       -3.001512       5.595986
*   -9.592006      6.367668       -2.321534       1.764135
*
```

6.5.1.3.9.7 ?(or,un)mql

计算$C = Q^* * C$或$C = C * Q^*$，其中Q是由?geqlf计算得到。当Q转置，实数时Q^*表示Q^T，复数时Q^*表示Q^H；当Q不转置，Q^*表示Q。

接口定义

C Interface：

void sormql_(const char *uplo, const char *trans, const int *m, const int *n, const int *k, float *a, const int *lda, const float *tau, float *c, const int *ldc, float *work, const int *lwork, int *info);

void dormql_(const char *uplo, const char *trans, const int *m, const int *n, const int *k, double *a, const int *lda, const double *tau, double *c, const int *ldc, double *work, const int *lwork, int *info);

 版权所有 © 华为技术有限公司

void cunmql_(const char *uplo, const char *trans, const int *m, const int *n, const int *k, float _Complex *a, const int *lda, const float _Complex *tau, float _Complex *c, const int *ldc, float _Complex *work, const int *lwork, int *info);

void zunmql_(const char *uplo, const char *trans, const int *m, const int *n, const int *k, double _Complex *a, const int *lda, const double _Complex *tau, double _Complex *c, const int *ldc, double _Complex *work, const int *lwork, int *info);

Fortran Interface：

SORMQL(side,trans,m,n,k,a,lda,tau,c,ldc,work,lwork,info);

DORMQL(side,trans,m,n,k,a,lda,tau,c,ldc,work,lwork,info);

CUNMQL(side,trans,m,n,k,a,lda,tau,c,ldc,work,lwork,info);

ZUNMQL(side,trans,m,n,k,a,lda,tau,c,ldc,work,lwork,info);

参数

参数名	类型	描述	输入/输出
side	字符型	• "L"：Q*在左边。 • "R"：Q*在右边。	输入
trans	字符型	• "N"：Q不转置。 • "T"：Q转置。	输入
M	整数型	C的行数，M ≥ 0。	输入
N	整数型	C的列数，N ≥ 0。	输入
K	整数型	生成Q的元素反射器的个数。 • side="L"时，M ≥ K ≥ 0。 • side="R"时，N ≥ K ≥ 0。	输入

 版权所有 © 华为技术有限公司

参数名	类型	描述	输入/输出
A	• 在sormql中为单精度浮点型数组。 • 在dormql中为双精度浮点型数组。 • 在cunmql中为单精度复数型数组。 • 在zunmql中为双精度复数型数组。	原始矩阵，维数（ LDA,K ）。	输入
lda	整数型	矩阵A的主维度。 • 当side="L"时，lda ≥ max(1,M)。 • 当side="R"时，lda ≥ max(1,N)。	输入
tau	• 在sormql中为单精度浮点型数组。 • 在dormql中为双精度浮点型数组。 • 在cunmql中为单精度复数型数组。 • 在zunmql中为双精度复数型数组。	tau(i)必须包含基本反射器H(i)的常量因子，其由?geqlf返回。	输入
C	• 在sormql中为单精度浮点型数组。 • 在dormql中为双精度浮点型数组。 • 在cunmql中为单精度复数型数组。 • 在zunmql中为双精度复数型数组。	待计算矩阵，维数为（ LDC, N ）。	输入、输出

 版权所有 © 华为技术有限公司

参数名	类型	描述	输入/输出
ldc	整数型	矩阵C的主维度，ldc ≥ max(1,M)。	输入
work	• 在sormql中为单精度浮点型数组。 • 在dormql中为双精度浮点型数组。 • 在cunmql中为单精度复数型数组。 • 在zunmql中为双精度复数型数组。	维数(max(1,lwork))。 若info=0，work(1)返回最优lwork值。	输出
lwork	整数型	work的大小。 • 当side="L"时，lwork ≥ max(1,N)。 • 当side="R"时，lwork ≥ max(1,M)。 如果lwork=-1，则该例程只计算work数组的最优大小，并以work数组的第一个值返回。	输入
info	整数型	• info=0：成功退出。 • info<0：第-info个参数值不合法。	输出

依赖

#include "klapack.h"

示例

C Interface：

```
char side = 'L';
char trans = 'N';
int m = 4;
int n = 4;
int k = 4;
int lda = 4;
```

 版权所有 © 华为技术有限公司

```
int ldc = 4;
int info = 0;
double tau[4] = {1.003949, 1.125229, 1.978923, 0.000000};
double *work = NULL;
double qwork;
int lwork = -1;
/*
 * tau:
 *   1.003949  1.125229  1.978923  0.000000
 * A (4x4, stored in column-major):
 *   -1.036040  -0.892502   -0.120115  -0.425611
 *   0.654120   0.468127    0.644302   -0.601906
 *   -0.135860  0.502159    1.077637   -0.103204
 *   -0.480109  -1.207226   0.162668   0.227483

 *
 *C (4x4, stored in column-major):
 *   2.229, 8.103, 7.810, 0.087,
 *   8.667, 8.859, 2.911, 4.091,
 *   0.205, 1.273, 1.396, 2.553,
 *   2.758, 4.317, 3.161, 8.791

 */
double a[] = {-1.036040, 0.654120,  0.135860, -0.480109,
              -0.892502, 0.468127,  0.502159, -1.207226,
              -0.120115, 0.644302,  1.077637,  0.162668,
              -0.425611, -0.601906, -0.103204, 0.227483};
double c[] = {2.229, 8.667, 0.205, 2.758,
              8.103, 8.859, 1.273, 4.317,
              7.810, 2.911, 1.396, 3.161,
              0.087, 4.091, 2.553, 8.791};
/* Query optimal work size */
dormql_(&side, &trans, &m, &n, &k, a, &lda, tau, c, &ldc, &qwork, &lwork, &info);
if (info != 0) {
    return ERROR;
}
lwork = (int)qwork;
work = (double *)malloc(sizeof(double) * lwork);
/* Calculate Q */
dormql_(&side, &trans, &m, &n, &k, a, &lda, tau, c, &ldc, &qwork, &lwork, &info);
free(work);
/*
 * Output:
 * C output (stored in column-major)
 *   8.335757        0.876180        3.263161        2.758000
 *   8.767347        -0.464834       2.323291        4.317000
 *   3.108108        -1.403301       -0.168152       3.161000
 *   4.519460        -2.717441       -0.868974       8.791000
 */
```

Fortran Interface:

```
      CHARACTER :: side = "L"
      CHARACTER :: trans = "N"
      PARAMETER (m = 4)
      PARAMETER (n = 4)
      PARAMETER (k = 4)
      PARAMETER (lda = 4)
      PARAMETER (ldc = 4)
      INTEGER :: info = 0
      REAL(8) :: tau(4)
      REAL(8) :: qwork(1)
      INTEGER :: lwork = -1
      REAL(8), ALLOCATABLE :: work(:)
*
*       tau:
*         1.003949  1.125229  1.978923  0.000000
*       A (4x4, stored in column-major):
*         -1.036040  -0.892502   -0.120115  -0.425611
*         0.654120   0.468127    0.644302   -0.601906
```


版权所有 © 华为技术有限公司

```
*       -0.135860  0.502159    1.077637   -0.103204
*       -0.480109  -1.207226   0.162668   0.227483

*      C (4x4, stored in column-major):
*        2.229, 8.103, 7.810, 0.087,
*        8.667, 8.859, 2.911, 4.091,
*        0.205, 1.273, 1.396, 2.553,
*        2.758, 4.317, 3.161, 8.791

*
       DATA tau /1.003949, 1.125229, 1.978923, 0.000000/
       REAL(8) :: a(m, n)
       DATA a / -1.036040, 0.654120,  0.135860, -0.480109,
     $          -0.892502, 0.468127,  0.502159, -1.207226,
     $          -0.120115, 0.644302,  1.077637,  0.162668,
     $          -0.425611, -0.601906, -0.103204, 0.227483 /

       DATA c / 2.229, 8.667, 0.205, 2.758,
     $          8.103, 8.859, 1.273, 4.317,
     $          7.810, 2.911, 1.396, 3.161,
     $          0.087, 4.091, 2.553, 8.791
       EXTERNAL DORMQL
*      Query optimal work size
       CALL DORMQL(side, trans, m, n, k, a, lda, tau, c, ldc, qwork, lwork, info)
       IF (info.NE.0) THEN
          CALL EXIT(1)
       END IF
       lwork = INT(qwork(1))
       ALLOCATE(work(lwork))
*      Calculate Q
       CALL DORMQL(side, trans, m, n, k, a, lda, tau, c, ldc, work, lwork, info)
       DEALLOCATE(work)
 *
 * Output:
 * C output (stored in column-major)
 *   8.335757       0.876180       3.263161       2.758000
 *   8.767347       -0.464834      2.323291       4.317000
 *   3.108108       -1.403301      -0.168152      3.161000
 *   4.519460       -2.717441      -0.868974      8.791000
 *
```

6.5.1.3.9.8 ?(or,un)mrq

计算$C = Q^* * C$或$C = C * Q^*$，其中Q是由?gerqf计算得到。当Q转置，实数时Q^*表示Q^T，复数时Q^*表示Q^H；当Q不转置，Q^*表示Q。

接口定义

C Interface：

void sormrq_(const char *side, const char *trans, const int *m, const int *n, const int *k, float *a, const int *lda, float *tau, float *c, const int *ldc, float *work, const int *lwork, int *info);

void dormrq_(const char *side, const char *trans, const int *m, const int *n, const int *k, double *a, const int *lda, double *tau, double *c, const int *ldc, double *work, const int *lwork, int *info);

void cunmrq_(const char *side, const char *trans, const int *m, const int *n, const int *k, float _Complex *a, const int *lda, float _Complex *tau, float _Complex *c, const int *ldc, float _Complex *work, const int *lwork, int *info);

 版权所有 © 华为技术有限公司

void zunmrq_(const char *side, const char *trans, const int *m, const int *n, const int *k, double _Complex *a, const int *lda, double _Complex *tau, double _Complex *c, const int *ldc, double _Complex *work, const int *lwork, int *info);

Fortran Interface：

SORMQR(SIDE,TRANS,M,N,K,A,LDA,TAU,C,LDC,WORK, LWORK, INFO);

DORMQR(SIDE,TRANS,M,N,K,A,LDA,TAU,C,LDC,WORK, LWORK, INFO);

CUNMQR(SIDE,TRANS,M,N,K,A,LDA,TAU,C,LDC,WORK, LWORK, INFO);

ZUNMQR(SIDE,TRANS,M,N,K,A,LDA,TAU,C,LDC,WORK, LWORK, INFO);

参数

参数名	类型	描述	输入/输出
side	字符型	• "L"：Q*在左边。 • "R"：Q*在右边。	输入
trans	字符型	• "N"：Q不转置。 • "T"：Q转置。	输入
M	整数型	C的行数，M ≥ 0。	输入
N	整数型	C的列数，N ≥ 0。	输入
K	整数型	生成Q的元素反射器的个数。 • side="L"时，M ≥ K ≥ 0。 • side="R"时，N ≥ K ≥ 0。	输入
A	• 在sormrq中为单精度浮点型数组。 • 在dormrq中为双精度浮点型数组。 • 在cunmrq中为单精度复数型数组。 • 在zunmrq中为双精度复数型数组。	原始矩阵。 • 当side="L"时，维数为（lda, M）。 • 当side="R"时，维数为（lda,N）。	输入
lda	整数型	矩阵A的主维，lda ≥ max(1,K)。	输入

版权所有 © 华为技术有限公司

参数名	类型	描述	输入/输出
tau	• 在sormrq中为单精度浮点型数组。 • 在dormrq中为双精度浮点型数组。 • 在cunmrq中为单精度复数型数组。 • 在zunmrq中为双精度复数型数组。	tau(i)必须包含基本反射器H(i)的常量因子，其由?gerqf返回。	输入
C	• 在sormrq中为单精度浮点型数组。 • 在dormrq中为双精度浮点型数组。 • 在cunmrq中为单精度复数型数组。 • 在zunmrq中为双精度复数型数组。	待计算矩阵，维数为（ ldc, N ）。	输入、输出
ldc	整数型	矩阵C的主维度，ldc ≥ max(1,M)。	输入
work	• 在sormrq中为单精度浮点型数组。 • 在dormrq中为双精度浮点型数组。 • 在cunmrq中为单精度复数型数组。 • 在zunmrq中为双精度复数型数组。	维数(max(1,lwork))。 若info=0，work(1)返回最优lwork值。	输出
lwork	整数型	work的大小。 • 当side="L"时，lwork ≥ max(1,N)。 • 当side="R"时，lwork ≥ max(1,M)。 如果lwork=-1，则该例程只计算work数组的最优大小，并以work数组的第一个值返回。	输入

版权所有 © 华为技术有限公司

参数名	类型	描述	输入/输出
info	整数型	• info=0：成功退出。 • info<0：第-info个参数值不合法。	输出

依赖

#include "klapack.h"

示例

C Interface：

```
char side = 'L';
char trans = 'N';
int m = 4;
int n = 4;
int k = 4;
int lda = 4;
int ldc = 4;
int info = 0;
double tau[4] = {1.003949, 1.125229, 1.978923, 0.000000};
double *work = NULL;
double qwork;
int lwork = -1;
/*
 * tau:
 *   1.003949  1.125229  1.978923  0.000000
 * A (4x4, stored in column-major):
 *   -1.036040  -0.892502   -0.120115  -0.425611
 *   0.654120   0.468127    0.644302   -0.601906
 *   -0.135860  0.502159    1.077637   -0.103204
 *   -0.480109  -1.207226   0.162668   0.227483

 *
 * C (4x4, stored in column-major):
 *   2.229, 8.103, 7.810, 0.087,
 *   8.667, 8.859, 2.911, 4.091,
 *   0.205, 1.273, 1.396, 2.553,
 *   2.758, 4.317, 3.161, 8.791

 */
double a[] = {-1.036040, 0.654120,  0.135860, -0.480109,
              -0.892502, 0.468127,  0.502159, -1.207226,
              -0.120115, 0.644302,  1.077637,  0.162668,
              -0.425611, -0.601906, -0.103204, 0.227483};
double c[] = {2.229, 8.667, 0.205, 2.758,
              8.103, 8.859, 1.273, 4.317,
              7.810, 2.911, 1.396, 3.161,
              0.087, 4.091, 2.553, 8.791};
/* Query optimal work size */
dormrq_(&side, &trans, &m, &n, &k, a, &lda, tau, c, &ldc, &qwork, &lwork, &info);
if (info != 0) {
    return ERROR;
}
lwork = (int)qwork;
work = (double *)malloc(sizeof(double) * lwork);
/* Calculate Q */
dormrq_(&side, &trans, &m, &n, &k, a, &lda, tau, c, &ldc, &qwork, &lwork, &info);
```

 版权所有 © 华为技术有限公司

```
free(work);
/*
 * Output:
 * C output (stored in column-major)
 *   0.009265       -1.159433       -9.412653       2.758000
 *   -0.006793      -4.874383       -12.228202      4.317000
 *   -0.016693      -4.849802       -6.359105       3.161000
 *   0.000914       0.912554        -6.587947       8.791000
 */
```

Fortran Interface：

```
      CHARACTER :: side = "L"
      CHARACTER :: trans = "N"
      PARAMETER (m = 4)
      PARAMETER (n = 4)
      PARAMETER (k = 4)
      PARAMETER (lda = 4)
      PARAMETER (ldc = 4)
      INTEGER :: info = 0
      REAL(8) :: tau(4)
      REAL(8) :: qwork(1)
      INTEGER :: lwork = -1
      REAL(8), ALLOCATABLE :: work(:)
*
*     tau:
*       1.003949  1.125229  1.978923  0.000000
*     A (4x4, stored in column-major):
*       -1.036040  -0.892502   -0.120115  -0.425611
*       0.654120   0.468127    0.644302   -0.601906
*       -0.135860  0.502159    1.077637   -0.103204
*       -0.480109  -1.207226   0.162668   0.227483

*     C (4x4, stored in column-major):
*       2.229, 8.103, 7.810, 0.087,
*       8.667, 8.859, 2.911, 4.091,
*       0.205, 1.273, 1.396, 2.553,
*       2.758, 4.317, 3.161, 8.791

*
      DATA tau /1.003949, 1.125229, 1.978923, 0.000000/
      REAL(8) :: a(m, n)
      DATA a / -1.036040, 0.654120,  0.135860, -0.480109,
     $          -0.892502, 0.468127,  0.502159, -1.207226,
     $          -0.120115, 0.644302,  1.077637,  0.162668,
     $          -0.425611, -0.601906, -0.103204, 0.227483 /

      DATA c / 2.229, 8.667, 0.205, 2.758,
     $          8.103, 8.859, 1.273, 4.317,
     $          7.810, 2.911, 1.396, 3.161,
     $          0.087, 4.091, 2.553, 8.791
      EXTERNAL DORMRQ
*     Query optimal work size
      CALL DORMRQ(side, trans, m, n, k, a, lda, tau, c, ldc, qwork, lwork, info)
      IF (info.NE.0) THEN
         CALL EXIT(1)
      END IF
      lwork = INT(qwork(1))
      ALLOCATE(work(lwork))
*     Calculate Q
      CALL DORMRQ(side, trans, m, n, k, a, lda, tau, c, ldc, work, lwork, info)
      DEALLOCATE(work)
*
* Output:
* C output (stored in column-major)
*   0.009265       -1.159433       -9.412653       2.758000
*   -0.006793      -4.874383       -12.228202      4.317000
*   -0.016693      -4.849802       -6.359105       3.161000
*   0.000914       0.912554        -6.587947       8.791000
*
```


版权所有 © 华为技术有限公司

6.5.1.3.9.9 ?sy(he)trd

将对称矩阵或Hermite矩阵通过相似变换成对称三对角T，即Q**H*A*Q=T。

接口定义

C Interface：

void ssytrd_(const char *uplo, const int *n, float *a, const int *lda, float *d, float *e, float *tau, float *work, const int *lwork, int *info);

void dsytrd_(const char *uplo, const int *n, double *a, const int *lda, double *d, double *e, double *tau, double *work, const int *lwork, int *info);

void chetrd_(const char *uplo, const int *n, float _Complex *a, const int *lda, float *d, float *e, float _Complex *tau, float _Complex *work, const int *lwork, int *info);

void zhetrd_(const char *uplo, const int *n, double _Complex *a, const int *lda, double *d, double *e, double _Complex *tau, double _Complex *work, const int *lwork, int *info);

Fortran Interface：

SSYTRD(UPLO,N,A,LDA,D,E,TAU,WORK,LWORK,INFO);

DSYTRD(UPLO,N,A,LDA,D,E,TAU,WORK,LWORK,INFO);

CHETRD(UPLO,N,A,LDA,D,E,TAU,WORK,LWORK,INFO);

ZHETRD(UPLO,N,A,LDA,D,E,TAU,WORK,LWORK,INFO);

参数

参数名	类型	描述	输入/输出
uplo	字符型	• 'U'：A中保存上三角矩阵。 • 'L'：A中保存下三角矩阵。	输入
N	整数型	矩阵A的维数，N ≥ 0。	输入
A	• 在ssytrd中为单精度浮点型数组。 • 在dsytrd中为双精度浮点型数组。 • 在chetrd中为单精度复数型数组。 • 在zhetrd中为双精度复数型数组。	• 调用前保存待分解的对称矩阵。 • 调用后保存特征向量。	输入、输出
lda	整数型	矩阵A的主维，LDA ≥ max(1,N)。	输入

版权所有 © 华为技术有限公司

参数名	类型	描述	输入/输出
D	• 在ssytrd/chetrd中为单精度浮点型数组。 • 在dsytrd/zhetrd中为双精度浮点型数组。	三对角矩阵T的对角元素，即D(i)=A(i,i)。	输出
E	• 在ssytrd/chetrd中为单精度浮点型数组。 • 在dsytrd/zhetrd中为双精度浮点型数组。	三对角矩阵T非对角元素。 • 当UPLO="U"时，E(i)=A(i,i+1)。 • 当UPLO="L"时，E(i)=A(i+1,i)。	输出
tau	• 在ssytrd中为单精度浮点型数组。 • 在dsytrd中为双精度浮点型数组。 • 在chetrd中为单精度复数型数组。 • 在zhetrd中为双精度复数型数组。	tau(i)必须包含基本反射器的常量因子，维数为N-1。	输出
work	• 在ssytrd中为单精度浮点型数组。 • 在dsytrd中为双精度浮点型数组。 • 在chetrd中为单精度复数型数组。 • 在zhetrd中为双精度复数型数组。	维数(max(1,lwork))。 若info=0，work(1)返回最优lwork值。	输出
lwork	整数型	数组work的维数。lwork ≥ max(1, N)，对于最优性能：lwork ≥ N*NB，其中NB为最优块大小如果lwork=-1，则该例程只计算work数组的最优大小，并以work数组的第一个值返回。	输入

版权所有 © 华为技术有限公司

参数名	类型	描述	输入/输出
info	整数型	• info=0：成功退出。 • info<0：第-info个参数值不合法。	输出

依赖

#include "klapack.h"

示例

C Interface：

```
char uplo = 'L';
int n = 4;
int lda = n;
int info = 0;
double tau[4] = {1.003949, 1.125229, 1.978923, 0.000000};
double *work = NULL;
double qwork;
int lwork = -1;
double *d = (double*)malloc(n * sizeof(double));
double *e = (double*)malloc((n - 1) * sizeof(double));
/*
 * tau:
 *   1.003949  1.125229  1.978923  0.000000
 * A (4x4, stored in column-major):
 *   7.027  8.710  1.015  6.929
 *   8.710  0.839  2.469  3.850
 *   1.015  2.469  1.930  6.761
 *   6.929  3.850  6.761  4.344
 *
 */
double a[] = {7.027, 8.710, 1.015, 6.929,
              8.710, 0.839, 2.469, 3.850,
              1.015, 2.469, 1.930, 6.761,
              6.929, 3.850, 6.761, 4.344};

/* Query optimal work size */
dsytrd_(&uplo, &n, a, &lda, d, e, tau, &qwork, &lwork, &info);
if (info != 0) {
    return ERROR;
}
lwork = (int)qwork;
work = (double *)malloc(sizeof(double) * lwork);
/* Calculate Q */
dsytrd_(&uplo, &n, a, &lda, d, e, tau, work, &lwork, &info);
free(work);
/*
 * Output:
 * A output (stored in column-major)
 *   7.027000        -11.176107       0.051041        0.348434
 *   8.710000        7.026600        6.375997        0.224161
 *   1.015000        2.469000        3.358516        -1.395021
 *   6.929000        3.850000        6.761000        -3.272116
 *
 * D output (stored in column-major)
 *   7.027000        7.026600        3.358516        -3.272116
 *
```

 版权所有 © 华为技术有限公司

```
   * E output (stored in column-major)
   *   -11.176107      6.375997        -1.395021
   */
```

Fortran Interface：

```
      CHARACTER :: uplo = "L"
      PARAMETER (n = 4)
      PARAMETER (lda = 4)
      INTEGER :: info = 0
      REAL(8) :: tau(4)
      REAL(8) :: qwork(1)
      INTEGER :: lwork = -1
      REAL(8), ALLOCATABLE :: work(:)
      REAL(8), ALLOCATABLE :: d(:)
      REAL(8), ALLOCATABLE :: e(:)
*
*     tau:
*       1.003949  1.125229  1.978923  0.000000
*     A (4x4, stored in column-major):
*       7.027  8.710  1.015  6.929
*       8.710  0.839  2.469  3.850
*       1.015  2.469  1.930  6.761
*       6.929  3.850  6.761  4.344
*
      DATA tau /1.003949, 1.125229, 1.978923, 0.000000/
      REAL(8) :: a(n, n)
      DATA a / 7.027, 8.710, 1.015, 6.929,
     $          8.710, 0.839, 2.469, 3.850,
     $          1.015, 2.469, 1.930, 6.761,
     $          6.929, 3.850, 6.761, 4.344 /

      EXTERNAL DSYTRD
*     Query optimal work size
      CALL DSYTRD(uplo, n, a, lda, d, e, tau, qwork, lwork, info)
      IF (info.NE.0) THEN
         CALL EXIT(1)
      END IF
      lwork = INT(qwork(1))
      ALLOCATE(work(lwork))
*     Calculate Q
      CALL DSYTRD(uplo, n, a, lda, d, e, tau, work, lwork, info)
      DEALLOCATE(work)

*     Output:
*     A output (stored in column-major)
*      7.027000        -11.176107      0.051041        0.348434
*      8.710000        7.026600        6.375997        0.224161
*      1.015000        2.469000        3.358516        -1.395021
*      6.929000        3.850000        6.761000        -3.272116
*
*     D output (stored in column-major)
*      7.027000        7.026600        3.358516        -3.272116
*
*     E output (stored in column-major)
*      -11.176107      6.375997        -1.395021
```

6.5.1.3.9.10 ?lasr

对矩阵A做平面旋转操作。

接口定义

C Interface：

void slasr_(const char *SIDE, const char *PIVOT, const char *DIRECT, const int *M, const int *N, float *C, float *S, float *A, const int *LDA);

 版权所有 © 华为技术有限公司

void dlasr_(const char *SIDE, const char *PIVOT, const char *DIRECT, const int *M, const int *N, double *C, double *S, double *A, const int *LDA);

void clasr_(const char *SIDE, const char *PIVOT, const char *DIRECT, const int *M, const int *N, float *C, float *S, float _Complex *A, const int *LDA);

void zlasr_(const char *SIDE, const char *PIVOT, const char *DIRECT, const int *M, const int *N, double *C, double *S, double _Complex *A, const int *LDA);

Fortran Interface：

SLASR(SIDE, PIVOT, DIRECT, M, N, C, S, A, LDA);

DLASR(SIDE, PIVOT, DIRECT, M, N, C, S, A, LDA);

CLASR(SIDE, PIVOT, DIRECT, M, N, C, S, A, LDA);

ZLASR(SIDE, PIVOT, DIRECT, M, N, C, S, A, LDA);

参数

参数名	类型	描述	输入/输出
SIDE	字符型	• 'L'：A:=P*A。 • 'R'：A:=A*P**T。	输入
PIVOT	字符型	• 'V'：Variable pivot, plane (k, k+1)。 • 'T'：Top pivot, plane(1, k +1)。 • 'B'：Bottom pivot, plane (k, z)。	输入
DIRECT	字符型	• 'F'：Forward, P=P(z-1)*… *P(2)*P(1)。 • 'B'：Backward, P=P(1)*P(2)*… *P(z-1)。	输入
M	整数型	矩阵A的行数。	输入
N	整数型	矩阵A的列数。	输入
C	• 在slasr/clasr中为单精度浮点型数组。 • 在dlasr/zlasr中为双精度浮点型数组。	平面旋转的cosine值。	输入

 版权所有 © 华为技术有限公司

参数名	类型	描述	输入/输出
S	• 在slasr/clasr中为单精度浮点型数组。 • 在dlasr/zlasr中为双精度浮点型数组。	平面旋转的sine值。	输入
A	• 在slasr中为单精度浮点型数组。 • 在dlasr中为双精度浮点型数组。 • 在clasr中为单精度复数型数组。 • 在zlasr中为双精度复数型数组。	矩阵A，维度为（LDA,N）。	输入，输出
LDA	整数型	矩阵A的主维度。	输入

依赖

#include "klapack.h"

示例

C Interface：

```
   const char side = 'L';
   const char pivot = 'V';
   const char direct = 'F';
   int m = 4;
   int n = 4;
   int lda = m;

   double c[] = {0.000000, 0.900000, 1.100000};
   double s[] = {0.181818, 0.545455, 0.000000};
   double a[] = {0.600000,0.200000,0.900000,0.100000,
                 0.200000,0.700000,0.000000,0.900000,
                 0.300000,0.600000,0.000000,0.600000,
                 0.200000,0.600000,0.100000,0.800000};

   dlasr_(&side, &pivot, &direct, &m, &n, c, s, a, &lda);
/*
    * Output:
    * a output (stored in column-major)
    *   0.036364      0.392727      0.956455      0.110000
    *   0.127273     -0.032727      0.021818      0.990000
    *   0.109091     -0.049091      0.032727      0.660000
    *   0.109091      0.021818      0.120818      0.880000
```

Fortran Interface：

```
CHARACTER :: side = "L"
CHARACTER :: pivot = "V"
CHARACTER :: direct = "F"
PARAMETER (m = 4)
PARAMETER (n = 4)
PARAMETER (lda = 4)
REAL(8) :: c(m-1)
```

 版权所有 © 华为技术有限公司

```
REAL(8) :: s(m-1)
REAL(8) :: a(lda, n)

DATA c / 0.000000, 0.900000, 1.100000 /
DATA s / 0.181818, 0.545455, 0.000000 /
DATA a / 0.600000,0.200000,0.900000,0.100000,
$        0.200000,0.700000,0.000000,0.900000,
$        0.300000,0.600000,0.000000,0.600000,
$        0.200000,0.600000,0.100000,0.800000 /
 EXTERNAL DLASR
 CALL DLASR(side, pivot, direct, m, n, c, s, a, lda)
*
* Output:
* a output (stored in column-major)
*   0.036364       0.392727       0.956455       0.110000
*   0.127273       -0.032727      0.021818       0.990000
*   0.109091       -0.049091      0.032727       0.660000
*   0.109091       0.021818       0.120818       0.880000
```

6.5.1.3.9.11 ?sy(he)trd_2stage

将对称矩阵或Hermite矩阵A转为对称或Hermite三对角矩阵T，即A**H*A*Q=T，其中Q为正交矩阵。

接口定义

C Interface：

void ssytrd_2stage(const char *VECT, const char *UPLO, const int *N, float *A, const int *LDA, float *D, float *E, float *TAU, float *HOUS2, int *LHOUS2, float *WORK, const int *LWORK, int *INFO);

void dsytrd_2stage(const char *VECT, const char *UPLO, const int *N, double *A, const int *LDA, double *D, double *E, double *TAU, double *HOUS2, int *LHOUS2, double *WORK, const int *LWORK, int *INFO);

void chetrd_2stage(const char *VECT, const char *UPLO, const int *N, float _Complex *A, const int *LDA, float *D, float *E, float _Complex *TAU, float _Complex *HOUS2, int *LHOUS2, float _Complex *WORK, const int *LWORK, int *INFO);

void zhetrd_2stage(const char *VECT, const char *UPLO, const int *N, double _Complex *A, const int *LDA, double *D, double *E, double _Complex *TAU, double _Complex *HOUS2, int *LHOUS2, double _Complex *WORK, const int *LWORK, int *INFO);

Fortran Interface：

SSYTRD_2STAGE(VECT, UPLO, N, A, LDA, D, E, TAU, HOUS2, LHOUS2, WORK, LWORK, INFO);

DSYTRD_2STAGE(VECT, UPLO, N, A, LDA, D, E, TAU, HOUS2, LHOUS2, WORK, LWORK, INFO);

CHETRD_2STAGE(VECT, UPLO, N, A, LDA, D, E, TAU, HOUS2, LHOUS2, WORK, LWORK, INFO);

ZHETRD_2STAGE(VECT, UPLO, N, A, LDA, D, E, TAU, HOUS2, LHOUS2, WORK, LWORK, INFO);

 版权所有 © 华为技术有限公司

参数

参数名	类型	描述	输入/输出
VECT	字符型	• 'N'：不需要Householder表达式。 • 'V'：需要Householder表达式。	输入
UPLO	字符型	• 'U'：存放A的上三角部分。 • 'L'：存放A的下三角部分。	输入
N	• 在ssytrd_2stage中为单精度浮点型数组。 • 在dsytrd_2stage中为双精度浮点型数组。 • 在chetrd_2stage中为单精度复数型数组。 • 在zhetrd_2stage中为双精度复数型数组。	矩阵A的列数，N ≥ 0。	输入
A	• 在ssytrd_2stage中为单精度浮点型数组。 • 在dsytrd_2stage中为双精度浮点型数组。 • 在chetrd_2stage中为单精度复数型数组。 • 在zhetrd_2stage中为双精度复数型数组。	矩阵A，维度为（LDA, N）。	输入、输出
LDA	整数型	矩阵A的主维，LDA ≥ max(1, N)。	输入
D	• 在ssytrd_2stage/chetrd_2stage中为单精度浮点型数组。 • 在dsytrd_2stage/zhetrd_2stage中为双精度浮点型数组。	对称三对角矩阵T的对角元素，大小为N。	输出
E	• 在ssytrd_2stage/chetrd_2stage中为单精度浮点型数组。 • 在dsytrd_2stage/zhetrd_2stage中为双精度浮点型数组。	对称三对角矩阵T的非对角元素，大小为N-1。	输出

版权所有 © 华为技术有限公司

参数名	类型	描述	输入/输出
TAU	• 在ssytrd_2stage中为单精度浮点型数组。 • 在dsytrd_2stage中为双精度浮点型数组。 • 在chetrd_2stage中为单精度复数型数组。 • 在zhetrd_2stage中为双精度复数型数组。	基本反射器的标量因子。	输出
HOUS2	• 在ssytrd_2stage中为单精度浮点型数组。 • 在dsytrd_2stage中为双精度浮点型数组。 • 在chetrd_2stage中为单精度复数型数组。 • 在zhetrd_2stage中为双精度复数型数组。	存放第二阶段中Householder表达形式。	输出
LHOUS2	整数型	HOUS2的长度。 • VECT='N'时，LHOUS2=max(1,4*n)。 • VECT='V'时，不可用。	输入
WORK	• 在ssytrd_2stage中为单精度浮点型数组。 • 在dsytrd_2stage中为双精度浮点型数组。 • 在chetrd_2stage中为单精度复数型数组。 • 在zhetrd_2stage中为双精度复数型数组。	工作数组，大小为LWORK。	输出

 版权所有 © 华为技术有限公司

参数名	类型	描述	输入/输出
LWORK	整数型	工作数组的长度，LWORK ≥ max(1, dimension)。其中，dimension= max(stage1,stage2) + (KD+1)*N = N*KD + N*max(KD+1,FACTOPTNB) + max(2*KD*KD, KD*NTHREADS) + (KD+1)*N，当LWORK=-1时，返回最优LWORK值。	输入
INFO	整数型	• 等于0：成功退出。 • 小于0： 第-info个参数值不合法。	输出

依赖

include "klapack.h"

示例

C Interface：

```
char vect = 'N';
char uplo = 'L';
int n = 4;
int lda = n;
int info = 0;
double tau = (double*)malloc(n * sizeof(double));
double *work = NULL;
double *hous2 = NULL;
double qwork;
double qhous2;
int lwork = -1;
int lhous2 = -1;
double *d = (double*)malloc(n * sizeof(double));
double *e = (double*)malloc((n - 1) * sizeof(double));

double a[] = {0.521739, 0.043478, 0.304348, 0.130435,
              0.043478, 0.304348, 0.043478, 0.652174,
              0.304348, 0.043478, 0.086957, 0.521739,
              0.130435, 0.652174, 0.521739, 0.086957};

/* Query optimal work size and hous2 size */
dsytrd_2stage_(&vect, &uplo, &n, a, &lda, d, e, tau, &qhous2, &lhous2, &qwork, &lwork, &info)
if (info != 0) {
    return ERROR;
}
lwork = (int)qwork;
```

 版权所有 © 华为技术有限公司

```
work = (double *)malloc(sizeof(double) * lwork);
lhous2 = (int)qhous2;
hous2 = (double*)malloc(lhous2 * sizeof(double));

/* Calculate */
dsytrd_2stage_(&vect, &uplo, &n, a, &lda, d, e, tau, hous2, &lhous2, work, &lwork, &info);
if (info != 0) {
    return ERROR;
}

free(work);
free(hous2);
free(tau);

/*
* Output:
* A output (stored in column-major)
*   0.521739        0.043478        0.304348        0.130435
*   0.043478        0.304348        0.043478        0.652174
*   0.304348        0.043478        0.086957        0.521739
*   0.130435        0.652174        0.521739        0.086957
*
* D output (stored in column-major)
*   0.521739        0.538688        0.325765        -0.386192
*
* E output (stored in column-major)
*   -0.333963       -0.507861       -0.471868
*/
```

Fortran Interface：

```
CHARACTER :: vect = "N"
CHARACTER :: uplo = "L"
PARAMETER (n = 4)
PARAMETER (lda = 4)
INTEGER :: info = 0
REAL(8) :: tau(4)
REAL(8) :: qwork(1)
REAL(8) :: qhous2(1)
INTEGER :: lwork = -1
INTEGER :: lhous2 = -1
REAL(8), ALLOCATABLE :: work(:)
REAL(8), ALLOCATABLE :: hous2(:)
REAL(8), ALLOCATABLE :: d(:)
REAL(8), ALLOCATABLE :: e(:)
*
*       tau:
*         1.003949  1.125229  1.978923  0.000000
*       A (4x4, stored in column-major):
*         0.521739, 0.043478, 0.304348, 0.130435,
*         0.043478, 0.304348, 0.043478, 0.652174,
*         0.304348, 0.043478, 0.086957, 0.521739,
*         0.130435, 0.652174, 0.521739, 0.086957

*
      REAL(8) :: a(m, n)
      DATA a / 0.521739, 0.043478, 0.304348, 0.130435,
     $         0.043478, 0.304348, 0.043478, 0.652174,
     $         0.304348, 0.043478, 0.086957, 0.521739,
     $         0.130435, 0.652174, 0.521739, 0.086957 /

      EXTERNAL DSYTRD_2STAGE
*     Query optimal work size and hous2 size.
      CALL DSYTRD_2STAGE(vect, uplo, n, a, lda, d, e, tau, qhous2, lhous2, qwork, lwork, info)
      IF (info.NE.0) THEN
         CALL EXIT(1)
      END IF
      lwork = INT(qwork(1))
      ALLOCATE(work(lwork))
      lhous2 = INT(qhous2(1))
```

 版权所有 © 华为技术有限公司

```
      ALLOCATE(hous2(lhous2))
*     Calculate
      CALL DSYTRD_2STAGE(vect, uplo, n, a, lda, d, e, tau, hous2, lhous2, work, lwork, info)
      DEALLOCATE(work)
      DEALLOCATE(hous2)

* Output:
* A output (stored in column-major)
*   0.521739       0.043478       0.304348       0.130435
*   0.043478       0.304348       0.043478       0.652174
*   0.304348       0.043478       0.086957       0.521739
*   0.130435       0.652174       0.521739       0.086957
*
* D output (stored in column-major)
*   0.521739       0.538688       0.325765       -0.386192
*
* E output (stored in column-major)
*   -0.333963      -0.507861      -0.471868
```

6.5.1.3.9.12 ?(or,un)mbr

计算C=Q*C或Q**T*C或C*Q或C*Q**T或C=P*C或P**T*C或C*P或C*P**T，其中正交矩阵Q和P**T由函数GEBRD计算得到，即A=Q*B*P**T。

接口定义

C Interface：

sormbr_(const char *vect, const char *side, const char *trans, const int *m, const int *n, const int *k, const float *a, const int *lda, const float *tau, float *c, const int *ldc, float *work, const int *lwork, int *info);

dormbr_(const char *vect, const char *side, const char *trans, const int *m, const int *n, const int *k, const double *a, const int *lda, const double *tau, double *c, const int *ldc, double *work, const int *lwork, int *info);

cunmbr_(const char *vect, const char *side, const char *trans, const int *m, const int *n, const int *k, const float _Complex *a, const int *lda, const float _Complex *tau, float _Complex *c, const int *ldc, float _Complex *work, const int *lwork, int *info);

zunmbr_(const char *vect, const char *side, const char *trans, const int *m, const int *n, const int *k, const double _Complex *a, const int *lda, const double _Complex *tau, double _Complex *c, const int *ldc, double _Complex *work, const int *lwork, int *info);

Fortran Interface：

SORMBR(VECT, SIDE, TRANS, M, N, K, A, LDA, TAU, C, LDC, WORK, LWORK, INFO);

DORMBR(VECT, SIDE, TRANS, M, N, K, A, LDA, TAU, C, LDC, WORK, LWORK, INFO);

CUNMBR(VECT, SIDE, TRANS, M, N, K, A, LDA, TAU, C, LDC, WORK, LWORK, INFO);

ZUNMBR(VECT, SIDE, TRANS, M, N, K, A, LDA, TAU, C, LDC, WORK, LWORK, INFO);

 版权所有 © 华为技术有限公司

参数

参数	类型	说明	输入/输出
vect	字符型	• 'Q'：则使用Q计算。 • 'P'：则使用P计算。	输入
side	字符型	• 'L'：则Q，P左乘C。 • 'R'：则Q，P右乘C。	输入
trans	字符型	• 'N'：Q，P不转置。 • 'T'：Q，P转置。	输入
m	整数型	矩阵C的行数，m≥0。	输入
n	整数型	矩阵C的列数，n≥0。	输入
k	整数型	Q的列数或P的行数，k≥0。	输入
a	• 在sormbr中为单精度浮点型数组。 • 在dormbr中为双精度浮点型数组。 • 在cunmbr中为单精度复数型数组。 • 在zunmbr中为双精度复数型数组。	• 若vect='Q'，大小为lda*min(nq,k)。 • 若vect='P'，大小为lda*nq。 该数组表示了反射器H和G，可通过两者的乘积计算出Q和P。	输入
lda	整数型	矩阵A的主维度。 • 若vect='Q'，则lda≥max(1,nq)。 • 若vect='P'，则lda≥(1, min(nq, k))。	输入
tau	• 在sormbr中为单精度浮点型数组。 • 在dormbr中为双精度浮点型数组。 • 在cunmbr中为单精度复数型数组。 • 在zunmbr中为双精度复数型数组。	大小为min(nq,k)。 该数组表示H和G的标量因子。	输入

版权所有 © 华为技术有限公司

参数	类型	说明	输入/输出
c	• 在sormbr中为单精度浮点型数组。 • 在dormbr中为双精度浮点型数组。 • 在cunmbr中为单精度复数型数组。 • 在zunmbr中为双精度复数型数组。	大小为ldc*n。 • 进入时：为m*n的矩阵。 • 退出时：为Q*C或Q**H*C或C*Q**H或C*Q或P*C或P**H*C或C*P或C*P**H。	输入，输出
ldc	整数型	矩阵c的主维度，ldc≥max(1,m)。	输入
work	• 在sormbr中为单精度浮点型数组。 • 在dormbr中为双精度浮点型数组。 • 在cunmbr中为单精度复数型数组。 • 在zunmbr中为双精度复数型数组。	工作数组，大小为max(1,lwork)。 info=0时，work(0)返回最优lwork大小。	输出
lwork	整数型	work的大小。 • 若side='L'，lwork≥max(1,n)。 • 若side='R'，lwork≥max(1,m)。 • 若lwork=-1，则返回最优的大小。	输入
Info	整数型	• 等于0：表示成功。 • 小于0：info=-i，表示第i个参数非法。	输出

依赖

#include "klapack.h"

示例

C Interface：

```
const char vect = 'Q';
const char side = 'L';
const char trans = 'N';
const int n = 4;
const int m = 4;
const int k = 4;
```

 版权所有 © 华为技术有限公司

```
const int lda = n;
const int ldc = m;

int info = 0;
double *work = NULL;
double qwork;
int lwork = -1;

double *d = (double*)malloc(n * sizeof(double));
double *e = (double*)malloc((n - 1) * sizeof(double));
double *tauq = (double*)malloc(n * sizeof(double));
double *taup = (double*)malloc(n * sizeof(double));

double a[] = {7.027, 8.710, 1.015, 6.929,
              8.710, 0.839, 2.469, 3.850,
              1.015, 2.469, 1.930, 6.761,
              6.929, 3.850, 6.761, 4.344};

double c[] = {7.027, 8.710, 1.015, 6.929,
              8.710, 0.839, 2.469, 3.850,
              1.015, 2.469, 1.930, 6.761,
              6.929, 3.850, 6.761, 4.344};

/* Query optimal work size */
dgebrd_(&m, &n, a, &lda, d, e, tauq, taup, &qwork, &lwork, &info);
if (info != 0) {
    printf("ERROR, info = %d\n", info);
    return info;
}
lwork = (int)qwork;
work = (double *)malloc(sizeof(double) * lwork);
/* Calculate Q */
dgebrd_(&m, &n, a, &lda, d, e, tauq, taup, work, &lwork, &info);
if (info != 0) {
    printf("ERROR, info = %d\n", info);
    return info;
}

dormbr_(&vect, &side, &trans, &m, &n, &k, a, &lda, tauq, c, &ldc, work, &lwork, &info);
if (info != 0) {
    printf("ERROR, info = %d\n", info);
    return info;
}
/* output */
* a
* -13.201670      13.064517       0.286648        0.441152
* 0.430577        -8.159976       2.308820        -0.069390
* 0.050176        -0.414492       5.045717        -1.018082
* 0.342534        -0.018475       0.799162        -3.517170
```

Fortran Interface:

```
CHARACTER :: vect = "Q"
CHARACTER :: side = "L"
CHARACTER :: trans = "N"
PARAMETER (n = 4)
PARAMETER (m = 4)
PARAMETER (lda = 4)
PARAMETER (ldc = 4)
PARAMETER (k = 4)

INTEGER :: info = 0
REAL(8) :: a(lda, n)
REAL(8) :: c(ldc, n)
REAL(8) :: d(n)
REAL(8) :: e(n-1)
REAL(8) :: tauq(n)
REAL(8) :: taup(n)
REAL(8), ALLOCATABLE :: work(:)
REAL(8) :: qwork(1)
```

 版权所有 © 华为技术有限公司

```
INTEGER :: lwork = -1

DATA a / 7.027, 8.710, 1.015, 6.929,
&        8.710, 0.839, 2.469, 3.850,
&        1.015, 2.469, 1.930, 6.761,
&        6.929, 3.850, 6.761, 4.344 /

DATA c / 7.027, 8.710, 1.015, 6.929,
&        8.710, 0.839, 2.469, 3.850,
&        1.015, 2.469, 1.930, 6.761,
&        6.929, 3.850, 6.761, 4.344 /
EXTERNAL DGEBRD,DORMBR
CALL DGEBRD(m, n, a, lda, d, e, tauq, taup, qwork, lwork, info)
IF (info.NE.0) THEN
   CALL EXIT(1)
END IF
lwork = INT(qwork(1))
ALLOCATE(work(lwork))
CALL DGEBRD(m, n, a, lda, d, e, tauq, taup, qwork, lwork, info)
IF (info.NE.0) THEN
   CALL EXIT(1)
END IF
CALL DORMBR(vect, side, trans, m, n, k, a, lda, tauq, c, ldc, work, lwork, info)
DEALLOCATE(work);
*
* Output:
* a
* -13.201670      13.064517       0.286648       0.441152
* 0.430577       -8.159976       2.308820       -0.069390
* 0.050176       -0.414492       5.045717       -1.018082
* 0.342534       -0.018475       0.799162       -3.517170
```

6.5.1.3.9.13 ?(or,un)gtr

通过反射因子生成正交矩阵Q，其中反射因子由SYTRD计算得来。

接口定义

C Interface：

sorgtr_(const char *uplo, const int *n, float *a, const int *lda, const float *tau, float *work, const int *lwork, int *info);

dorgtr_(const char *uplo, const int *n, double *a, const int *lda, const double *tau, double *work, const int *lwork, int *info);

cungtr_(const char *uplo, const int *n, float _Complex *a, const int *lda, const float _Complex *tau, float _Complex *work, const int *lwork, int *info);

zungtr_(const char *uplo, const int *n, double _Complex *a, const int *lda, const double _Complex *tau, double _Complex *work, const int *lwork, int *info);

Fortran Interface：

SORGTR(UPLO, N, A, LDA, TAU, WORK, LWORK, INFO);

DORGTR(UPLO, N, A, LDA, TAU, WORK, LWORK, INFO);

CUNGTR(UPLO, N, A, LDA, TAU, WORK, LWORK, INFO);

ZUNGTR(UPLO, N, A, LDA, TAU, WORK, LWORK, INFO);

 版权所有 © 华为技术有限公司

参数

参数	类型	说明	输入/输出
uplo	字符型	• 'U'：存放A的上三角部分。 • 'L'：存放A的下三角部分。	输入
n	整数型	矩阵A的维度， n≥0。	输入
a	• 在sorgtr中为单精度浮点型数组。 • 在dorgtr中为双精度浮点型数组。 • 在cungtr中为单精度复数型数组。 • 在zungtr中为双精度复数型数组。	大小为lda*n。 • 进入时：表示反射器。 • 退出时：n*n的正交矩阵Q。	输入，输出
lda	整数型	矩阵A的主维，lda≥max(1, n)。	输入
tau	• 在sorgtr中为单精度浮点型数组。 • 在dorgtr中为双精度浮点型数组。 • 在cungtr中为单精度复数型数组。 • 在zungtr中为双精度复数型数组。	大小为n-1。 该数组表示H的标量因子。	输入
work	• 在sorgtr中为单精度浮点型数组。 • 在dorgtr中为双精度浮点型数组。 • 在cungtr中为单精度复数型数组。 • 在zungtr中为双精度复数型数组。	工作数组，大小为max(1, lwork)。 info=0时，work(0)返回最优lwork大小。	输出
lwork	整数型	work的大小。 • 若lwork≥n - 1，为获取最优性能lwork≥(n-1)*nb，nb为最优分块大小。 • 若lwork=-1，work返回最优大小。	输入
Info	整数型	• 等于0：表示成功。 • 小于0：info=-i，表示第i个参数非法。	输出

版权所有 © 华为技术有限公司

依赖

#include "klapack.h"

示例

C Interface：

```
char uplo = 'L';
int n = 4;
int lda = n;
int info = 0;
double tau[4] = {1.003949, 1.125229, 1.978923, 0.000000};
double *work = NULL;
double qwork;
int lwork = -1;
double *d = (double*)malloc(n * sizeof(double));
double *e = (double*)malloc((n - 1) * sizeof(double));
/*
 * tau:
 *   1.003949  1.125229  1.978923  0.000000
 * A (4x4, stored in column-major):
 *   7.027  8.710  1.015  6.929
 *   8.710  0.839  2.469  3.850
 *   1.015  2.469  1.930  6.761
 *   6.929  3.850  6.761  4.344
 *
 */
double a[] = {7.027, 8.710, 1.015, 6.929,
              8.710, 0.839, 2.469, 3.850,
              1.015, 2.469, 1.930, 6.761,
              6.929, 3.850, 6.761, 4.344};

/* Query optimal work size */
dsytrd_(&uplo, &n, a, &lda, d, e, tau, &qwork, &lwork, &info);
if (info != 0) {
    printf("ERROR, info = %d\n", info);
    return 1;
}
lwork = (int)qwork;
work = (double *)malloc(sizeof(double) * lwork);
/* Calculate Q */
dsytrd_(&uplo, &n, a, &lda, d, e, tau, work, &lwork, &info);

dorgtr_(&uplo, &n, a, &lda, tau, work, &lwork, &info);
/* output */
* a
* 1.000000        0.000000        0.000000        0.000000
* 0.000000        -0.779341       0.346782        -0.521890
* 0.000000        -0.090819       -0.886612       -0.453510
* 0.000000        -0.619983       -0.306042       0.722467
```

Fortran Interface：

```
CHARACTER :: uplo = "L"
PARAMETER (n = 4)
PARAMETER (lda = 4)

INTEGER :: info = 0
REAL(8) :: a(lda, n)
REAL(8) :: d(n)
REAL(8) :: e(n-1)
REAL(8) :: tau(n)
REAL(8), ALLOCATABLE :: work(:)
REAL(8) :: qwork(1)
```


版权所有 © 华为技术有限公司

```
INTEGER :: lwork = -1

DATA a / 7.027, 8.710, 1.015, 6.929,
&        8.710, 0.839, 2.469, 3.850,
&        1.015, 2.469, 1.930, 6.761,
&        6.929, 3.850, 6.761, 4.344 /

DATA tau / 1.003949, 1.125229, 1.978923, 0.000000 /
EXTERNAL DSYTRD,DORGTR
CALL DSYTRD(uplo, n, a, lda, d, e, tau, qwork, lwork, info)
IF (info.NE.0) THEN
   CALL EXIT(1)
END IF
lwork = INT(qwork(1))
ALLOCATE(work(lwork))
CALL DSYTRD(uplo, n, a, lda, d, e, tau, qwork, lwork, info)
IF (info.NE.0) THEN
   CALL EXIT(1)
END IF
CALL DORGTR(uplo, n, a, lda, tau, work, lwork, info)
DEALLOCATE(work);
*
* Output:
* a
*
1.000000        0.000000        0.000000        0.000000
* 0.000000        -0.779341       0.346782        -0.521890
* 0.000000        -0.090819       -0.886612       -0.453510
* 0.000000        -0.619983       -0.306042       0.722467
```

6.5.1.3.9.14 Kml?steqr2

LAPACK函数 ?STEQR 的修改版本（仅有V case），使用隐式QL或QR方法计算对称三对角矩阵的所有特征值和可选的特征向量。用于KML ScaLAPACK中，ScaLAPACK各进程可以调用该函数分别对分布式矩阵Q进行更新。仅有C格式接口，无fortran格式接口。

接口定义

C Interface：

int KmlSsteqr2(constint n, float*d, float*e, float*z, constint ldz, constint nr, float *work);

int KmlDsteqr2(const int n, double *d, double *e, double *z, const int ldz, const int nr, double *work);

参数

参数名	类型	描述	输入/输出
N	整数型	矩阵的行数/列数	输入
D	KmlSsteqr2中为单精度浮点型数组 KmlDsteqr2中为双精度浮点型数组	维度为N 输入时，存放三对角矩阵的对角元素。 输出时，如果INFO为0，特征值升序排列	输入，输出

版权所有 © 华为技术有限公司

参数名	类型	描述	输入/输出
E	KmlSsteqr2中为单精度浮点型数组 KmlDsteqr2中为双精度浮点型数组	维度为N-1 输入时，存放三对角矩阵的次对角线元素 输出时，E被销毁	输入
Z	KmlSsteqr2中为单精度浮点型数组 KmlDsteqr2中为双精度浮点型数组	维度为N*N 输入时，存放正交矩阵 输出时，如果INFO为0，存放正交特征向量	输入，输出
LDZ	整数型	矩阵Z的主维，LDZ >= max(1,N)	输入
NR	整数型	NR = MAX(1, NUMROC(N, NB, MYPROW, 0, NPROCS))	输入
WORK	KmlSsteqr2中为单精度浮点型数组 KmlDsteqr2中为双精度浮点型数组	工作数组，维度为max(1,2*N-2)	输入

依赖

#include "klapack.h"

示例

C Interface：

```
const int n = 4;
const int ldz = 4;
const int nr = 2;

double d[] = {14.1234, 15.5678, 16.9012, 17.3456};
double e[] = {18.7890, 19.1234, 20.5678, 21.9012};
double *z = (double *)malloc(ldz * n * sizeof(double));
// 1st column
z[0 * ldz + 0] = 0.5;
z[0 * ldz + 1] = 0.5;
z[0 * ldz + 2] = 0.5;
z[0 * ldz + 3] = 0.5;

// 2nd column
z[1 * ldz + 0] = 0.5;
z[1 * ldz + 1] = -0.5;
z[1 * ldz + 2] = 0.5;
z[1 * ldz + 3] = -0.5;

// 3rd column
z[2 * ldz + 0] = 0.5;
z[2 * ldz + 1] = 0.5;
z[2 * ldz + 2] = -0.5;
z[2 * ldz + 3] = -0.5;

// 4th column
z[3 * ldz + 0] = 0.5;
```


版权所有 © 华为技术有限公司

```
z[3 * ldz + 1] = -0.5;
z[3 * ldz + 2] = -0.5;
z[3 * ldz + 3] = 0.5;

double *work = (double *)malloc((2 * n - 2) * sizeof(double));
KmlDsteqr2(n, d, e, z, ldz, nr, work);

/*
 * Output:
d :
   -1.534857e+01   3.519426e+00   2.804530e+01   4.772184e+01
z :
   -1.449053e-03   2.307024e-01   -8.569433e-02   9.692424e-01
   -9.756888e-01   1.491886e-03   -2.181372e-01   -2.110012e-02
   5.000000e-01   5.000000e-01   -5.000000e-01   -5.000000e-01
   5.000000e-01   -5.000000e-01   -5.000000e-01   5.000000e-01
*/
```

6.5.1.3.9.15 KmlBlockBandedMatrixSolve

KML_LAPACK的自定义接口，用于求解分块大小为5的块状条带矩阵，同时返回矩阵的分解结果。此接口不包含pivoting，因此需要保证矩阵为对角占优。

接口定义

void KmlBlockBandedMatrixSolve_(int *n, double *a, double *x, int *batchSize, int *info);

参数

参数名	类型	描述	输入/输出
n	整数型	矩阵大小，至少为20，且为5的倍数	输入
a	双精度浮点型	矩阵A的元素	输入/输出
x	双精度浮点型	作为输入时为右端项，输出时为解向量。	输入/输出
batchSize	整数型	batch大小，0<batchSize<=8。	输入
info	整数型	• info=0：成功退出。 • info<0：第-info个参数值不合法。	输入/输出

示例

```
int n = 25;
int batchSize = 8;
int blockSize = 25;
int info = 0;
double *a = (double *)malloc(sizeof(double) * blockSize * batchSize * n);
double *x = (double *)malloc(sizeof(double) * n * batchSize);
```

 版权所有 © 华为技术有限公司

```
double *exact_x = (double *)malloc(sizeof(double) * n * batchSize);
/* 填充数据 */
KmlBlockBandedMatrixSolve_(&n, aa, xx, &batchSize, &info);
```

6.5.1.3.9.16 KmlBlockBandedMatrixSubstitude

KML_LAPACK的自定义接口，用于对分块大小为5的块状条带矩阵的分解结果进行回代。此接口不包含pivoting，因此需要保证矩阵为对角占优。

接口定义

void KmlBlockBandedMatrixSubstitude_(int *n, double *a, double *x, int *batchSize, int *info);

参数

参数名	类型	描述	输入/输出
n	整数型	矩阵大小，至少为20，且为5的倍数	输入
a	双精度浮点型	矩阵A的元素	输入/输出
x	双精度浮点型	作为输入时为右端项，输出时为解向量。	输入/输出
batchSize	整数型	batch大小，0<batchSize<=8。	输入
info	整数型	• info=0：成功退出。 • info<0：第-info个参数值不合法。	输入/输出

示例

```
int n = 25;
int batchSize = 8;
int blockSize = 25;
int info = 0;
double *a = (double *)malloc(sizeof(double) * blockSize * batchSize * n);
double *x = (double *)malloc(sizeof(double) * n * batchSize);
double *exact_x = (double *)malloc(sizeof(double) * n * batchSize);
/* 填充数据 */
KmlBlockBandedMatrixSolve_(&n, aa, xx, &batchSize, &info);
double *rhs = (double *)malloc(sizeof(double) * n * batchSize);
/* 填充新的右端项 */
KmlBlockBandedMatrixSubstitude_(&n, aa, rhs, &batchSize, &info);
```

6.5.1.3.10 辅助函数

6.5.1.3.10.1 ?laset

初始化m*n的矩阵，且将对角线元素设置为beta，非对角线元素设置为alpha。

 版权所有 © 华为技术有限公司

接口定义

C Interface：

void slaset_(const char *uplo, const int *m, const int *n, const float *alpha, const float *beta, float *a, const int *lda);

void dlaset_(const char *uplo, const int *m, const int *n, const double *alpha, const double *beta, double *a, const int *lda);

void claset_(const char *uplo, const int *m, const int *n, const float _Complex *alpha, const float _Complex *beta, float _Complex *a, const int *lda);

void zlaset_(const char *uplo, const int *m, const int *n, const double _Complex *alpha, const double _Complex *beta, double _Complex *a, const int *lda);

Fortran Interface：

SLASET(UPLO, M, N, ALPHA, BETA, A, LDA);

DLASET(UPLO, M, N, ALPHA, BETA, A, LDA);

CLASET(UPLO, M, N, ALPHA, BETA, A, LDA);

ZLASET(UPLO, M, N, ALPHA, BETA, A, LDA);

参数

参数名	类型	描述	输入/输出
UPLO	字符型	• 'U'：矩阵A的上三角部分被初始化。 • 'L'：矩阵A的下三角部分被初始化。 • 其他：整个矩阵A都将被初始化。	输入
M	整数型	矩阵A的行数。	输入
N	整数型	矩阵A的列数。	输入
ALPHA	• 在slaset中为单精度浮点型。 • 在dlaset中为双精度浮点型。 • 在claset中为单精度复数型。 • 在zlaset中为双精度复数型。	矩阵A的非对角线元素值。	输入

版权所有 © 华为技术有限公司

参数名	类型	描述	输入/输出
BETA	• 在slaset中为单精度浮点型。 • 在dlaset中为双精度浮点型。 • 在claset中为单精度复数型。 • 在zlaset中为双精度复数型。	矩阵A的对角线元素值。	输入
A	• 在slaset中为单精度浮点型数组。 • 在dlaset中为双精度浮点型数组。 • 在claset中为单精度复数型数组。 • 在zlaset中为双精度复数型数组。	矩阵A，维度为（LDA,N）。	输出
LDA	整数型	矩阵A的主维度。	输入

依赖

#include "klapack.h"

示例

C Interface：

```
const char uplo = 'U';
const int m = 4;
const int n = 4;
const int lda = m;

const double alpha = 2.0;
const double beta = 1.0;
double *a = (double*)malloc(lda * n * sizeof(double));
for (int i = 0; i < lda * n; i++) {
    a[i] = -1;
}

dlaset_(&uplo, &m, &n, &alpha, &beta, a, &lda);
/*
 * Output:
 * a output (stored in column-major)
 *
 *  1.000000        2.000000        2.000000        2.000000
 *  -1.000000       1.000000        2.000000        2.000000
 *  -1.000000       -1.000000       1.000000        2.000000
 *  -1.000000       -1.000000       -1.000000       1.000000
```

Fortran Interface：

```
CHARACTER :: uplo = "U"
PARAMETER (m = 4)
```

 版权所有 © 华为技术有限公司

```
PARAMETER (n = 4)
PARAMETER (lda = 4)
REAL(8) :: alpha
REAL(8) :: beta
REAL(8) :: a(lda, n)

DATA alpha / 2.0 /
DATA beta / 1.0 /
DATA a / -1, -1, -1, -1, -1, -1, -1, -1, -1, -1, -1, -1, -1, -1, -1, -1 /
 EXTERNAL DLASET
 CALL DLASET(uplo, m, n, alpha, beta, a, lda);
*
* Output:
* a output (stored in column-major)
*  1.000000       2.000000       2.000000       2.000000
*  -1.000000      1.000000       2.000000       2.000000
*  -1.000000      -1.000000      1.000000       2.000000
*  -1.000000      -1.000000      -1.000000      1.000000
```

6.5.1.3.10.2 ?laswp

对矩阵做一系列的行交换操作。

接口定义

C Interface：

void slaswp_(const int *n, float *a, const int *lda, const int *k1, const int *k2, const int *ipiv, const int *incx);

void dlaswp_(const int *n, double *a, const int *lda, const int *k1, const int *k2, const int *ipiv, const int *incx);

void claswp_(const int *n, float _Complex *a, const int *lda, const int *k1, const int *k2, const int *ipiv, const int *incx);

void zlaswp_(const int *n, double _Complex *a, const int *lda, const int *k1, const int *k2, const int *ipiv, const int *incx);

Fortran Interface：

SLASWP(N, A, LDA, K1, K2, IPIV, INCX);

DLASWP(N, A, LDA, K1, K2, IPIV, INCX);

CLASWP(N, A, LDA, K1, K2, IPIV, INCX);

ZLASWP(N, A, LDA, K1, K2, IPIV, INCX);

参数

参数名	类型	描述	输入/输出
N	整数型	矩阵A的列数。	输入

版权所有 © 华为技术有限公司

参数名	类型	描述	输入/输出
A	• 在slaswp中为单精度浮点型。 • 在dlaswp中为双精度浮点型。 • 在claswp中为单精度复数型。 • 在zlaswp中为双精度复数型。	矩阵A，大小为LDA*N。 • 调用前：原始矩阵A。 • 调用后：交换后的矩阵。	输入，输出
LDA	整数型	矩阵A的主维。	输入
K1	整数型	IPIV的索引值，用于表示交换的起始行。	输入
K2	整数型	IPIV的索引值，用于表示交换的末尾行，K2>=K1。	输入
IPIV	整数型数组	交换的索引数组，大小为K1+(K2-K1)*abs(INCX)。	输入
INCX	整数型	IPIV的取值步长。	输入

依赖

#include "klapack.h"

示例

C Interface：

```
const int n = 4;
const int lda = n;
const int k1 = 1;
const int k2 = 2;
const int incx = 1;

double a[] = { 1.0, 1.0, 1.0, 1.0,
               2.0, 2.0, 2.0, 2.0,
               8.0, 3.0, 3.0, 3.0,
               4.0, 4.0, 4.0, 4.0};
int ipiv[3] = {3,1};
dlaswp_(&n, a, &lda, &k1, &k2, ipiv, &incx);
/*
 * Output:
 * a output:
 *
 *  1.000000    1.000000    1.000000    1.000000
 *  2.000000    2.000000    2.000000    2.000000
 *  3.000000    3.000000    8.000000    3.000000
 *  4.000000    4.000000    4.000000    4.000000
```

Fortran Interface：

 版权所有 © 华为技术有限公司

```
PARAMETER (n = 4)
PARAMETER (lda = n)
PARAMETER (k1 = 1)
PARAMETER (k2 = 2)
PARAMETER (incx = 1)
REAL(8) :: a(lda, n)
INTEGER :: ipiv(3)

DATA a / 1.0, 1.0, 1.0, 1.0,
$        2.0, 2.0, 2.0, 2.0,
$        8.0, 3.0, 3.0, 3.0,
$        4.0, 4.0, 4.0, 4.0 /
DATA ipiv / 3, 1 /
 EXTERNAL DLASWP
 CALL DLASWP(n, a, lda, k1, k2, ipiv, incx);
*
* Output:
* a output (stored in column-major)
*  1.000000    1.000000    1.000000    1.000000
*  2.000000    2.000000    2.000000    2.000000
*  3.000000    3.000000    8.000000    3.000000
*  4.000000    4.000000    4.000000    4.000000
```

6.5.1.3.10.3 ?lascl

对矩阵做scalar操作。

接口定义

C Interface：

void slascl_(const char *type, const int *kl, const int *ku, const float *cfrom, const float *cto, const int *m, const int *n, float *a, const int *lda, int *info);

void dlascl_(const char *type, const int *kl, const int *ku, const double *cfrom, const double *cto, const int *m, const int *n, double *a, const int *lda, int *info);

void clascl_(const char *type, const int *kl, const int *ku, const float *cfrom, const float *cto, const int *m, const int *n, float _Complex *a, const int *lda, int *info);

void zlascl_(const char *type, const int *kl, const int *ku, const double *cfrom, const double *cto, const int *m, const int *n, double _Complex *a, const int *lda, int *info);

Fortran Interface：

SLASCL(TYPE, KL, KU, CFROM, CTO, M, N, A, LDA, INFO);

DLASCL(TYPE, KL, KU, CFROM, CTO, M, N, A, LDA, INFO);

CLASCL(TYPE, KL, KU, CFROM, CTO, M, N, A, LDA, INFO);

ZLASCL(TYPE, KL, KU, CFROM, CTO, M, N, A, LDA, INFO);

 版权所有 © 华为技术有限公司

参数

参数名	类型	描述	输入/输出
TYPE	字符型	● 'G'：矩阵A是全矩阵。 ● 'L'：矩阵A是下三角矩阵。 ● 'U'：矩阵A是上三角矩阵。 ● 'H'：矩阵A是上Hessenberg矩阵。 ● 'B'：矩阵A是对称带状矩阵，下带宽为KL，上带宽为KU，且只存对角线下部分。 ● 'Q'：矩阵A是对称带状矩阵，下带宽为KL，上带宽为KU，且只存对角线上部分。 ● 'Z'：矩阵A是带状矩阵，下带宽为KL，上带宽为KU。	输入
KL	整数型	矩阵A的下带宽，当TYPE='B'，'Q'或'Z'时，有效。	输入
KU	整数型	矩阵A的上带宽，当TYPE='B'，'Q'或'Z'时，有效。	输入
CFROM	● 在slascl中为单精度浮点型。 ● 在dlascl中为双精度浮点型。 ● 在clascl中为单精度浮点型。 ● 在zlascl中为双精度浮点型。	非零值，为scalar值的分母部分，即：scalar=CTO/CFROM。	输入
CTO	● 在slascl中为单精度浮点型。 ● 在dlascl中为双精度浮点型。 ● 在clascl中为单精度浮点型。 ● 在zlascl中为双精度浮点型。	scalar值的分子部分，即：scalar=CTO/CFROM。	输入
M	整数型	矩阵A的行数，M≥0。	输入
N	整数型	矩阵A的列数，N≥0。	输入

 版权所有 © 华为技术有限公司

参数名	类型	描述	输入/输出
A	• 在slascl中为单精度浮点型数组。 • 在dlascl中为双精度浮点型数组。 • 在clascl中为单精度复数型数组。 • 在zlascl中为双精度复数型数组。	矩阵A，维度为（ LDA, N ）。	输入，输出
LDA	整数型	矩阵A的主维度。 • TYPE='G'，'L'，'U'，'H'时，LDA>=max(1, M)。 • TYPE='B'时，LDA>=KL+1。 • TYPE='Q'时，LDA>=KU+1。 • TYPE='Z'时，LDA>=2*KL+KU+1。	输入
INFO	整数型	• 等于0：运行成功。 • 小于0：第-info个参数值不合法。	输出

依赖

#include "klapack.h"

示例

C Interface：

```
const char type = 'G';
const int n = 4;
const int m = 4;
const int kl = 2;
const int ku = 2;
const double cfrom = 2.0;
const double cto = 1.0;
const int lda = m;
int info = 0;

double a[] = {1.0, 1.0, 1.0, 1.0,
              2.0, 2.0, 2.0, 2.0,
              3.0, 3.0, 3.0, 3.0,
              4.0, 4.0, 4.0, 4.0};

dlascl_(&type, &kl, &ku, &cfrom, &cto, &m, &n, a, &lda, &info);
/*
 * Output:
 * a output
 *
 *  0.500000        0.500000        0.500000        0.500000
 *  1.000000        1.000000        1.000000        1.000000
```

 版权所有 © 华为技术有限公司

```
*  1.500000       1.500000       1.500000       1.500000
*  2.000000       2.000000       2.000000       2.000000
```

Fortran Interface：

```
CHARACTER :: type = "G"
PARAMETER (m = 4)
PARAMETER (n = 4)
PARAMETER (kl = 2)
PARAMETER (ku = 2)
PARAMETER (lda = 4)
REAL(8) :: cfrom
REAL(8) :: cto
REAL(8) :: a(lda, n)

DATA cfrom / 2.0 /
DATA cto / 1.0 /
DATA a / 1.0, 1.0, 1.0, 1.0,
        2.0, 2.0, 2.0, 2.0,
        3.0, 3.0, 3.0, 3.0,
        4.0, 4.0, 4.0, 4.0 /
 EXTERNAL DLASCL
 CALL DLASCL(type, kl, ku, cfrom, cto, m, n, a, lda, info)
*
* Output:
* a output
*
*  0.500000       0.500000       0.500000       0.500000
*  1.000000       1.000000       1.000000       1.000000
*  1.500000       1.500000       1.500000       1.500000
*  2.000000       2.000000       2.000000       2.000000
```

6.5.1.3.10.4 ?lange

计算矩阵的范数，包含：1范数，F范数，无穷范数等。

接口定义

C Interface：

float slange_(const char *norm, const int *m, const int *n, const float *a, const int *lda, float *work);

double dlange_(const char *norm, const int *m, const int *n, const double *a, const int *lda, double *work);

float clange_(const char *norm, const int *m, const int *n, const float _Complex *a, const int *lda, float _Complex *work);

double zlange_(const char *norm, const int *m, const int *n, const double _Complex *a, const int *lda, double _Complex *work);

Fortran Interface：

SLANGE(NORM, M, N, A, LDA, WORK);

DLANGE(NORM, M, N, A, LDA, WORK);

CLANGE(NORM, M, N, A, LDA, WORK);

ZLANGE(NORM, M, N, A, LDA, WORK);

 版权所有 © 华为技术有限公司

参数

参数名	类型	描述	输入/输出
NORM	字符型	• 'M'或'm'：表示max(abs(A(i,j)))。 • '1'或'O'或'o'：表示计算矩阵的1范数。 • 'I'或'i'：表示计算矩阵的无穷范数。 • 'F'或'f'或'E'或'e'：表示计算矩阵的F范数。	输入
M	整数型	矩阵A的行数，M≥0。当M=0时，返回0。	输入
N	整数型	矩阵A的列数，N≥0。当N=0时，返回0。	输入
A	• 在slange中为单精度浮点型数组。 • 在dlange中为双精度浮点型数组。 • 在clange中为单精度复数型数组。 • 在zlange中为双精度复数型数组。	矩阵A，大小为LDA*N。	输入
LDA	整数型	矩阵A的主维，LDA≥max(M,1)。	输入
WORK	• 在slange中为单精度浮点型数组。 • 在dlange中为双精度浮点型数组。 • 在clange中为单精度复数型数组。 • 在zlange中为双精度复数型数组。	工作数组，大小为max(1, LWORK)。 • NORM='I'时，LWORK≥M。 • 其他情况，WORK将不被使用。	输出

依赖

#include "klapack.h"

示例

C Interface：

```
const char norm = 'M';
const int n = 4;
```

 版权所有 © 华为技术有限公司

```
const int m = 4;
const int lda = m;
int info = 0;

double a[] = {1.0, 1.0, 1.0, 1.0,
              2.0, 2.0, 2.0, 2.0,
              3.0, 3.0, 3.0, 3.0,
              4.0, 4.0, 4.0, 4.0};
double work;
double res = dlange_(&norm, &m, &n, a, &lda, &work);
/*
 * Output:
 * res: 4.000000
```

Fortran Interface：

```
CHARACTER :: norm = "M"
PARAMETER (m = 4)
PARAMETER (n = 4)
PARAMETER (lda = 4)
REAL(8) :: work
REAL(8) :: a(lda, n)

DATA a / 1.0, 1.0, 1.0, 1.0,
         2.0, 2.0, 2.0, 2.0,
         3.0, 3.0, 3.0, 3.0,
         4.0, 4.0, 4.0, 4.0 /
 EXTERNAL DLANGE
 CALL DLANGE(norm, m, n, a, lda, work);
*
* Output:
* res: 4.000000
```

6.5.1.3.10.5 ?lacpy

拷贝矩阵A的所有元素或部分元素到矩阵B。

接口定义

C Interface：

slacpy_(const char *uplo, const int *m, const int *n, const float *a, const int *lda, float *b, const int *ldb);

dlacpy_(const char *uplo, const int *m, const int *n, const double *a, const int *lda, double *b, const int *ldb);

clacpy_(const char *uplo, const int *m, const int *n, const float _Complex *a, const int *lda, float _Complex *b, const int *ldb);

zlacpy_(const char *uplo, const int *m, const int *n, const double _Complex *a, const int *lda, double _Complex *b, const int *ldb);

Fortran Interface：

SLACPY(UPLO, M, N, A, LDA, B, LDB);

DLACPY(UPLO, M, N, A, LDA, B, LDB);

CLACPY(UPLO, M, N, A, LDA, B, LDB);

ZLACPY(UPLO, M, N, A, LDA, B, LDB);

 版权所有 © 华为技术有限公司

参数

参数	类型	说明	输入/输出
uplo	字符型	• 'U'，只拷贝矩阵A的上三角部分。 • 'L'，只拷贝矩阵A的下三角部分。 • 其他值，拷贝矩阵A的所有元素。	输入
m	整数型	矩阵A的行数，m≥0。	输入
n	整数型	矩阵A的列数，n≥0。	输入
a	• 在slacpy中为单精度浮点型数组。 • 在dlacpy中为双精度浮点型数组。 • 在clacpy中为单精度复数型数组。 • 在zlacpy中为双精度复数型数组。	大小为lda*n。 • 若uplo='U'时，只有上三角或梯形部分元素可以访问。 • 若uplo='L'时，只有下三角或梯形部分元素可以访问。	输入
lda	整数型	矩阵A的主维，lda≥max(1,m)。	输入
b	• 在slacpy中为单精度浮点型数组。 • 在dlacpy中为双精度浮点型数组。 • 在clacpy中为单精度复数型数组。 • 在zlacpy中为双精度复数型数组。	大小为ldb*n。 退出时，在指定的uplo范围内，矩阵b=矩阵a。	输出
ldb	整数型	矩阵b的主维，ldb≥max(1,m)。	输入

依赖

#include "klapack.h"

示例

C Interface：

```
const char uplo = 'F';
const int m = 4;
const int n = 4;
const int lda = 4;
```


版权所有 © 华为技术有限公司

```
const int ldb = 4;

double a[] = { 1.0, 1.0, 1.0, 1.0,
               2.0, 2.0, 2.0, 2.0,
               8.0, 3.0, 3.0, 3.0,
               4.0, 4.0, 4.0, 4.0};
double *b = (double*)malloc(lda * n * sizeof(double));

dlacpy_(&uplo, &m, &n, a, &lda, b, &ldb);

/* output */
* b
* 1.000000        2.000000        8.000000        4.000000
* 1.000000        2.000000        3.000000        4.000000
* 1.000000        2.000000        3.000000        4.000000
* 1.000000        2.000000        3.000000        4.000000
```

Fortran Interface:

```
CHARACTER :: uplo = "F"
PARAMETER (n = 4)
PARAMETER (m = 4)
PARAMETER (lda = 4)
PARAMETER (ldb = 4)
REAL(8) :: a(lda, n)
REAL(8) :: b(ldb, n)

DATA a / 1.0, 1.0, 1.0, 1.0,
$        2.0, 2.0, 2.0, 2.0,
$        8.0, 3.0, 3.0, 3.0,
$        4.0, 4.0, 4.0, 4.0 /
 EXTERNAL DLACPY
 CALL DLACPY(uplo, m, n, a, lda, b, ldb);
*
* Output:
* a output (stored in column-major)
* 1.000000        2.000000        8.000000        4.000000
* 1.000000        2.000000        3.000000        4.000000
* 1.000000        2.000000        3.000000        4.000000
* 1.000000        2.000000        3.000000        4.000000
```

6.5.1.3.11 Support 函数

6.5.1.3.11.1 KLAPACKGetVersion

获取LAPACK产品版本信息。

接口定义

int KLAPACKGetVersion(KLAPACKVersion *ver);

参数

参数名	类型	描述	输入/输出
ver	KLAPACKVersion结构体指针	存储版本信息，非空。	输入、输出

依赖

#include "kml_service.h"

 版权所有 © 华为技术有限公司

```
#include "klapack.h"
```

示例

```
int main()
{
    KLAPACKVersion *ver = (KLAPACKVersion*)malloc(sizeof(KLAPACKVersion));
    int res = KLAPACKGetVersion(ver);
    if (res != 0) {
        printf("Get version error.\n");
        return 0;
    }
    printf("Product Name: %s\nProduct Version: %s\nComponent Name: %s\nComponent Version: %s
\nComponent AbiVersion: %s\nComponent AppendInfo: %s\nSoftware Name: %s\nSoftware Version: %s\n",
            ver->productName, ver->productVersion, ver->componentName,
            ver->componentVersion, ver->componentAbiVersion, ver->componentAppendInfo,
            ver->softwareName, ver->softwareVersion);
    return 0;
}
```

运行结果：

```
Product Name: Kunpeng HPCKit
Product Version: 24.0.RC1
Component Name: HPCKit-kml
Component Version: 2.3.0
Component AbiVersion: 1
Component AppendInfo: gcc
Software Name: hpckit-kml-lapack
Software Version: 2.3.0
```

说明

版本号以实际运行结果为准，上述结果仅供参考。

6.5.1.3.12 混合精度函数

6.5.1.3.12.1 ?gesv

求解线性方程组$A * X = B$，其中A是$n \times n$的矩阵，X和B为$n \times nrhs$的矩阵。求解过程中，对A使用具有部分旋转和行交换的LU分解，将其分解为$A = P * L * U$的形式，其中P是置换矩阵，L是单位下三角矩阵，U是上三角矩阵。并通过LU分解的结果来求解线性方程组。

接口定义

C Interface：

void dsgesv_(const int *n, const int *nrhs, double *a, const int *lda, int *ipiv, const double *b, const int *ldb, double *x, const int *ldx, double *work, float *swork, int *iter, int *info);

void zcgesv_(const int *n, const int *nrhs, double _Complex *a, const int *lda, int *ipiv, const double _Complex *b, const int *ldb, double _Complex *x, const int *ldx, double _Complex *work, float _Complex *swork, double *rwork, int *iter, int *info);

Fortran Interface：

DSGESV(N, NRHS, A, LDA, IPIV, B, LDB, X, LDX, WORK, SWORK, ITER, info);

版权所有 © 华为技术有限公司

ZCGESV(N, NRHS, A, LDA, IPIV, B, LDB, X, LDX, WORK, SWORK, RWORK, ITER, info);

参数

参数名	类型	描述	输入/输出
n	整数型	矩阵A的阶数，要求n≥0。	输入
nrhs	整数型	右端项的数量，即矩阵B的列数，要求nrhs≥0。	输入
a	• 在dsgesv中为双精度浮点型数组。 • 在zcgesv中为双精度复数型数组。	矩阵大小为(lda,n)。 • 输入：n*n的系数矩阵A。 • 输出：储存了$A = P * L * U$中的L和U，其中L的单位对角线元素不存储。	输入，输出
lda	整数型	A的leading dimension大小，要求lda≥max(1, n)。	输入
ipiv	整数型数组	数组维度为n。 存放了置换矩阵P的主元索引，即矩阵第i行与第ipiv(i)行交换。	输出
b	• 在dsgesv中为双精度浮点型数组。 • 在zcgesv中为双精度复数型数组。	矩阵维度为(ldb,nrhs)。 • 输入：$n \times nrhs$的右端矩阵B。	输入
ldb	整数型	B的主维大小，要求ldb≥max(1, n)。	输入
x	• 在dsgesv中为双精度浮点型数组。 • 在zcgesv中为双精度复数型数组。	大小为ldx*nrhs。 若info=0时，输出为n*nrhs的解矩阵。	输出
ldx	整数型	矩阵x的主维，ldx≥max(1, n)。	输入
work	• 在dsgesv中为双精度浮点型数组。 • 在zcgesv中为双精度复数型数组。	大小为n*nrhs。 用于保存残差向量。	输出
swork	• 在dsgesv中为单精度浮点型数组。 • 在zcgesv中为单精度复数型数组。	大小为n*(n+nrhs)。 用于存储单精度矩阵和单精度中的解。	输出

版权所有 © 华为技术有限公司

参数名	类型	描述	输入/输出
rwork(复数类型特有)	双精度浮点型数组	工作数组，大小为n。	输出
iter	整数型	• 小于0：迭代细化失败，使用双精度完成分解。 • 大于0：迭代细化成功，返回迭代步数。	输出
info	整数型	执行结果： • 等于0：成功。 • 小于0：第-info个参数值不合法。 • 大于0：$U(info,info)=0$，分解已经完成，但是因为U奇异，所以无法完成求解。	输出

依赖

#include "klapack.h"

示例

C Interface：

```
const int n = 4;
const int nrhs = 2;
const int lda = 4;
const int ldb = 4;
const int ldx = 4;

int info = 0;
int iter = 0;

double a[] = {72.1673, 66.1857, 64.7644, 28.0199, 91.4151,
              6.5180,  62.8483, 72.4323, 46.5760, 8.6928,
              28.9821, 42.1828, 18.6437, 99.8612, 35.6972,
              67.9812, 5.0880,  85.5035, 79.2945, 54.5920,
              28.6869, 49.7512, 7.5186,  28.6929, 84.6041};
int *ipiv = (int*)malloc(n * sizeof(int));
double b[] = {9.4532, 1.5204, 2.2127, 0.9891, 7.1778,
              6.8955, 7.2465, 3.5019, 8.2268, 3.5287};

double *x = (double*)malloc(ldx * nrhs * sizeof(double));
double *work = (double*)malloc(n * nrhs * sizeof(double));
float *swork = (float*)malloc(n * (n + nrhs) * sizeof(float));

dsgesv_(&n, &nrhs, a, &lda, ipiv, b, &ldb, x, &ldx, work, swork, &iter, &info);
if (info != 0) {
    printf("ERROR, info = %d\n", info);
}

if (iter < 0) {
```

 版权所有 © 华为技术有限公司

```
    printf("Iterative refinement failed, iter = %d\n", iter);
}
/* output */
* x
* 0.153366        0.083995
* -0.538653       0.096517
* 1.080726        -0.174382
* -0.145340       0.022261
* iter
* 2
```

Fortran Interface:

```
PARAMETER (n = 4)
PARAMETER (nrhs = 2)
PARAMETER (lda = 4)
PARAMETER (ldb = 4)
PARAMETER (ldx = 4)
REAL(8) :: a(lda, n)
REAL(8) :: b(ldb, n)
REAL(8) :: b(ldx, n)
REAL(8) :: work(nrhs, n)
REAL(4) :: swork(n+nrhs, n)
INTERGER::ipiv(n)

INTEGER :: info = 0
INTEGER :: iter = 0

DATA a / 72.1673, 66.1857, 64.7644, 28.0199, 91.4151,
&         6.5180,  62.8483, 72.4323, 46.5760, 8.6928,
&        28.9821, 42.1828, 18.6437, 99.8612, 35.6972,
&        67.9812, 5.0880,  85.5035, 79.2945, 54.5920,
&        28.6869, 49.7512, 7.5186,  28.6929, 84.6041 /
 EXTERNAL DSGESV
 CALL DSGESV(n, nrhs, a, lda, ipiv, b, ldb, x, ldx, work, swork, iter, info);
IF (info.NE.0) THEN
    CALL EXIT(1)
END IF
IF (iter.LT.0) THEN
    CALL EXIT(1)
END IF
*
* Output:
* 0.153366        0.083995
* -0.538653       0.096517
* 1.080726        -0.174382
* -0.145340       0.022261
* iter
* 2
```

6.5.1.4 错误处理

KML_LAPACK会对输入参数进行检查，当出现参数为空指针或数值不合法等错误时会调用LAPACK错误处理函数xerbla_执行错误处理，默认情况下将使用Netlib LAPACK自带的xerbla_实现。

此实现的行为是打印以下错误信息并以错误码0退出进程：

```
** On entry to FUNC parameter number  N had an illegal value
```

其中FUNC为出错的LAPACK函数名称，N为参数序号。

用户可以提供自定义的xerbla_()函数实现，避免在出错时直接退出整个程序，示例实现如下：

```
void xerbla_(const char *func, const int *param, int funcLen)
{
```

 版权所有 © 华为技术有限公司

```
    printf("XERBLA: function %.*s, info %i.\n", funcLen, func, *param);
}
```

使用上述示例函数时会在参数错误时打印一行错误信息，但不会退出进程。此时会返回函数的调用者，并将info参数设为相应的错误参数值。

请注意：

1. 当参数中出现多个错误参数时，info可能设为任一错误参数的序号（取反）。
2. 当输入的矩阵维度过大无法正常计算时，info会设为相关维度参数的序号（取反）。
3. 用户程序在链接时需要首先链接包含自定义xerbla_实现的文件，再链接liblapack_adapt.a，否则会出现重复定义符号错误或仍然使用了Netlib LAPACK的xerbla_实现。
4. 若用户提供自定义的xerbla_实现，所有错误处理（包括KML_LAPACK未优化的函数）都会使用此实现。

6.5.2 KML_SCALAPACK 库函数说明

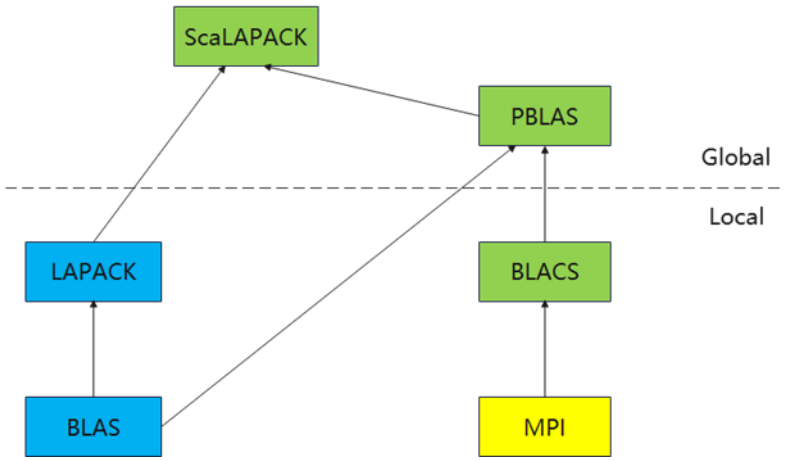
6.5.2.1 函数说明

高性能分布式并行线性代数库(ScaLAPACK)，适用于大规模并行和分布式场景，包括ScaLAPACK、PBLAS和BLACS三个组件，它是LAPACK项目在并行场景下的延伸。

基本线性代数通信库BLACS是一个消息传递库，作为ScaLAPACK的通信组件，它的功能包括同步和接收数据，不同于MPI，它进行通信的对象主要是子矩阵块。

PBLAS分布式基础数学库，它的接口类似于BLAS，是在BLAS和BLACS基础上构建的，目的是提供ScaLAPACK所需要的构建在分布式内存标准上的BLAS函数。

ScaLAPACK和底层通信库，基础数学库关系如下：



当前，Netlib ScaLAPACK标准定义了130多个ScaLAPACK接口规范，包括接口名称、参数列表和功能说明。鲲鹏HPCKit已对其中的部分接口进行了优化，具体清单请参见**表6-37**。其余接口将在后续版本中通过统一打包为完整的ScaLAPACK函数库发布。目前社区文档中仅包含已优化的接口说明，其余接口的文档请参见**Netlib SCALAPACK文档**。

6.5.2.2 数据描述

版权所有 © 华为技术有限公司

6.5.2.2.1 数据类型

提供KML_SCALAPACK库函数的数据类型说明。

数据类型	类型描述
s	单精度浮点类型
d	双精度浮点类型
c	单精度复数类型
z	双精度复数类型

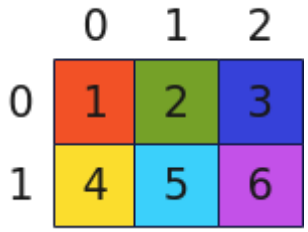
6.5.2.2.2 数据分布

ScaLAPACK是分布式接口，矩阵是分布在各进程上，而进程以进程网格形式排列，一般情况下进程网格是2D的，也存在1D的进程网格。进程网格由用户指定，存放在上下文句柄`ctxt`中，指定时需要通过blacs接口，例如

```
    char layout='R'; // Block cyclic, Row major processor mapping
    int zero=0;
    blacs_get_(&zero, &zero, &ctxt); // -> Create context
    blacs_gridinit_(&ctxt, &layout, &P, &Q); // Context -> Initialize the grid
    blacs_gridinfo_(&ctxt, &nprow, &npcol, &myrow, &mycol); // Context -> Context grid info (# procs row/
col, current procs row/col)
    ...
    blacs_gridexit_(&ctxt);
```

其中，blacs_get_将系统默认上下文到其它blacs函数中，起到创建作用。blacs_gridinit正式指定进程网格的P和Q，这两个数都是整数，使用时必须保证P*Q<=MPI procs。blacs_gridinfo获取当前进程在进程网格中的位置，即当前进程在进程网格中的行数和列数。在scalapack所有函数调用结束后，blacs_gridexit_函数可以释放blacs进程网格信息。

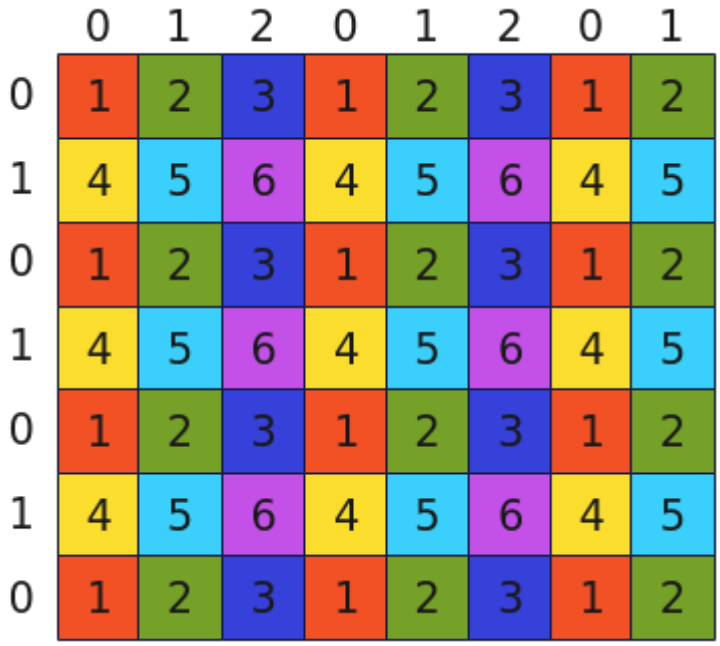
在2D网格中，若进程形成的进程网格拥有P个进程行和Q个进程列，可以用它在进程网格中的坐标表示为（p，q），其中0≤p≤P，0≤q≤Q，下面为6个节点的进程分布。



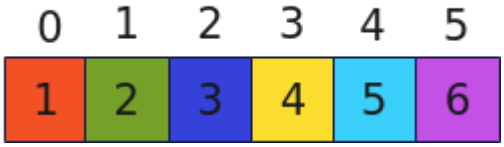
上图中进程是以行优先顺序分布至进程网格上，除此之外还可以使用列优先顺序分布至进程网格上。在默认情况下，使用行优先顺序进行进程网格映射。

当进程网格确定后，矩阵按照预先设置的分块大小m_b x n_b，循环往复地分配至进程网格上。下面是将矩阵分布在进程网格上，各块所在的进程编号情况：

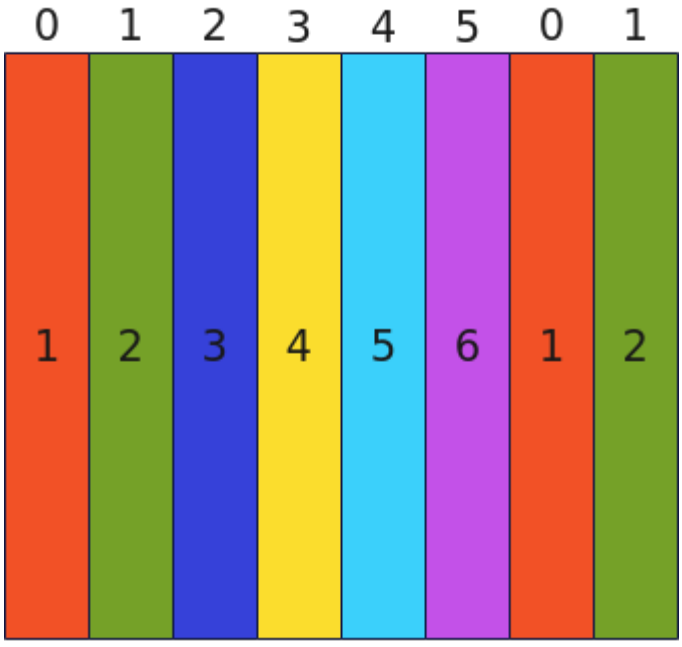
版权所有 © 华为技术有限公司



当P或Q被设置为1时，进程网格被设置为1D，在进程网格中的坐标可以按照进程编号设置为p或者q，其中0≤p≤P或0≤q≤Q。



将相同大小的矩阵分配至该1D进程网格上，矩阵划分及每个列块所在进程编号如下图所示：



版权所有 © 华为技术有限公司

上图为将P置为1的1D进程网格的数据分布情况，在实际应用中，也可以将Q置为1，此时进程网格上的数据块以行为基本划分单元。

6.5.2.2.3 矩阵描述符

矩阵描述符DESCA与每个全局矩阵相关联在分布式矩阵中，用来描述全局矩阵与进程和内存位置之间映射关系，每个分布式矩阵理论上都有单独矩阵描述符，但分布模式相同的矩阵可以共用一个描述符。矩阵描述符条目中使用的符号x_表示全局矩阵的属性。例如，M_表示行数，M_A具体表示全局矩阵A中的行数。根据矩阵类型的不同，描述符类型和矩阵存储方式也会发生变化，下面是一个矩阵的描述符DESCA的内容。

密集矩阵

一个通用的分布矩阵A（M_ × N_）由它的维度、用于其分解的基本MB_ × NB_块的大小、在其本地内存中具有第一个矩阵条目(RSRC_,CSRC_)的进程的坐标和BLACS定义此矩阵的上下文（CTXT_）决定。此外，本地前导维度LLD_与指向用于本地存储该分布式矩阵的数据结构的本地内存地址相关联。

描述符名称	范围	定义
DTYPE_A	全局	表示矩阵的类型。
CTXT_A	全局	BLACS上下文句柄，表示全局矩阵A在进程网格上的分布。
M_A	全局	矩阵A的全局行数。
N_A	全局	矩阵A的全局列数。
MB_A	全局	用于分配行的分块因子。
NB_A	全局	用于分配列的分块因子。
RSRC_A	全局	分布在矩阵A的第一行的进程行。
CSRC_A	全局	分布在矩阵A的第一列的进程列。
LLD_A	本地	本地矩阵的前导维度。$LLD_A \geq \mathrm{MAX}(1, LOC_R(M_A))$

矩阵描述符通过descinit_接口初始化。

```
int descA[9];
descinit_( descA,  &m, &n, &mb, &nb, &rsrc, &csrc, &ctxt, &lddA, &info);
```

其中，m和n表示全局矩阵行和列，mb和nb表示矩阵分块的大小，rsrc和csrc表示全局矩阵第一个元素(0,0)所在进程网格中的位置，因此有0 <= rsrc < nprow，0 <= csrc< npcol。ctxt即为包含进程网格信息的上下文句柄，info是scalapack函数的返回值，用来表示函数调用是否报错及报错原因。

6.5.2.3 函数定义

6.5.2.3.1 优化的 SCALAPACK 函数

对优化的SCALAPACK所有函数族进行描述。

 版权所有 © 华为技术有限公司

表 6-37 SCALAPACK 函数接口说明

功能	函数族	数据类型	描述
函数分解	P?GETRF	s，d，c，z	计算一般矩阵的LU分解，允许行交换。
	P?POTRF	s，d，c，z	实对称或共轭对称正定矩阵的Cholesky分解。
	P?GEQRF	s，d，c，z	计算一般矩阵的QR分解。
线性方程组求解	P?GELS	s，d，c，z	求解满秩矩阵A或其转置的超定或欠定实线性系统，使用A的QR或LQ分解。
	P?GESV	s，d，c，z	使用部分选主元的LU分解算法求解线性方程组
特征值问题函数	P?(SY/HE)EV	s，d，c，z	计算实数对称矩阵/复数厄米特矩阵A的所有特征值以及特征向量（可选）
	P?(SY/HE)EVD	s，d，c，z	计算实数对称矩阵/复数厄米特矩阵A的所有特征值和特征向量
矩阵求逆函数	PGETRI	s，d，c，z	计算分布式矩阵A的逆矩阵，其中A是LU分解后的LU矩阵。
	PPOTRI	s，d，c，z	计算经过Cholesky分解后的矩阵的逆矩阵。

6.5.2.3.2 矩阵分解函数

6.5.2.3.2.1 P?GETRF

计算矩阵A的LU分解，允许行交换（partial pivoting）。

分解结果为A=PLU，其中P为置换矩阵、L为下三角矩阵或下阶梯矩阵且对角线为1，U是上三角矩阵或上阶梯矩阵。

接口定义

C Interface：

void psgetrf_(const int *m, const int *n, float *a, const int *ia, const int *ja, const int *desca, int *ipiv, int *info)

void pdgetrf_(const int *m, const int *n, double*a, const int *ia, const int *ja, const int *desca, int *ipiv, int *info)

void pcgetrf_(const int *m, const int *n, float _Complex *a, const int *ia, const int *ja, const int *desca, int *ipiv, int *info)

void pzgetrf_(const int *m, const int *n, double _Complex*a, const int *ia, const int *ja, const int *desca, int *ipiv, int *info)

 版权所有 © 华为技术有限公司

Fortran Interface：

PSGETRF(m, n, a, ia, ja, desca, ipiv, info)

PDGETRF(m, n, a, ia, ja, desca, ipiv, info)

PCGETRF(m, n, a, ia, ja, desca, ipiv, info)

PZGETRF(m, n, a, ia, ja, desca, ipiv, info)

参数

参数	类型	范围	说明	输入/输出
m	整型	全局	要操作的行数，比如子矩阵的行数。	输入
n	整型	全局	要操作的列数，比如子矩阵的列数。	输入
a	• 在psgetrf中为单精度浮点型数组。 • 在pdgetrf中为双精度浮点型数组。 • 在pcgetrf中为单精度复数型数组。 • 在pzgetrf中为双精度复数型数组。	本地	• 调用前保存分布式矩阵A的本地M*N部分。 • 调用后保存本地部分存放的分解结果L和U，不保存L的对角线元素（均为1）。	输入，输出
ia	整型	全局	子矩阵在全局矩阵中的行索引。	输入
ja	整型	全局	子矩阵在全局矩阵中的列索引。	输入
desca	整型数组	本地，全局	分布式矩阵A的矩阵描述符。	输入
ipiv	整型	本地	包含了主元信息。	输出
info	整型	全局	执行结果： • 等于0：成功。 • 小于0：第-info个参数值不合法。 • 大于0：矩阵U对角线上第info个元素为0，矩阵分解完成但U是奇异的，会导致求解线性方程组时出现除以零的错误。	输出

依赖

#include <kscalapack.h>

 版权所有 © 华为技术有限公司

示例

C Interface：

```
int izero=0;
int ione=1;
int myrank_mpi, nprocs_mpi;
MPI_Init( &argc, &argv);
MPI_Comm_rank(MPI_COMM_WORLD, &myrank_mpi);
MPI_Comm_size(MPI_COMM_WORLD, &nprocs_mpi);

int n = 8;       // (Global) Matrix size
int nprow = 2;   // Number of row procs
int npcol = 2;   // Number of column procs
int nb = 4;      // (Global) Block size
char uplo='L';   // Matrix is lower triangular
char layout='R'; // Block cyclic, Row major processor mapping

printf("Usage: ./test matrix_size block_size nprocs_row nprocs_col\n");

if(argc > 1) {
    n = atoi(argv[1]);
}
if(argc > 2) {
    nb = atoi(argv[2]);
}
if(argc > 3) {
    nprow = atoi(argv[3]);
}
if(argc > 4) {
    npcol = atoi(argv[4]);
}
assert(nprow * npcol == nprocs_mpi);
// Initialize BLACS
int iam, nprocs;
int zero = 0;
int ictxt, myrow, mycol;
blacs_pinfo_(&iam, &nprocs) ; // BLACS rank and world size
blacs_get_(&zero, &zero, &ictxt ); // -> Create context
blacs_gridinit_(&ictxt, &layout, &nprow, &npcol ); // Context -> Initialize the grid
blacs_gridinfo_(&ictxt, &nprow, &npcol, &myrow, &mycol ); // Context -> Context grid info (# procs row/
col, current procs row/col)

// Compute the size of the local matrices
int mpA    = numroc_( &n, &nb, &myrow, &izero, &nprow ); // My proc -> row of local A
int nqA    = numroc_( &n, &nb, &mycol, &izero, &npcol ); // My proc -> col of local A

//allocate and fill the matrix A
double *A;
A = (double *)calloc(mpA*nqA,sizeof(double)) ;
if (A==NULL){ printf("Error of memory allocation A on proc %dx%d\n",myrow,mycol); exit(0); }
int k = 0;
for (int j = 0; j < nqA; j++) { // local col
    int l_j = j / nb; // which block
    int x_j = j % nb; // where within that block
    int J   = (l_j * npcol + mycol) * nb + x_j; // global col
    for (int i = 0; i < mpA; i++) { // local row
        int l_i = i / nb; // which block
        int x_i = i % nb; // where within that block
        int I   = (l_i * nprow + myrow) * nb + x_i; // global row
        assert(I < n);
        assert(J < n);
        if(I == J) {
            A[k] = i + j + (rand())%10;
        } else {
            A[k] = i + j + 5  + (rand())%10;
        }
        //printf("%d %d -> %d %d -> %f\n", i, j, I, J, A[k]);
        k++;
    }
```


版权所有 © 华为技术有限公司

```
}
//create descriptor
int descA[9];
int info=0;
int ipiv[10] = {0};
int lddA = mpA > 1 ? mpA : 1;
descinit_( descA,  &n, &n, &nb, &nb, &izero, &izero, &ictxt, &lddA, &info);
if(info != 0) {
    printf("Error in descinit, info = %d\n", info);
}

//run pdpotrf_ and time
double MPIt1 = MPI_Wtime();
printf("[%dx%d] Starting \n", myrow, mycol);
pdgetrf_(&n, &n, A, &ione, &ione, descA, ipiv,&info);
if (info != 0) {
    printf("Error in calculate, info = %d\n", info);
}
double MPIt2 = MPI_Wtime();
printf("[%dx%d] Done, time %e s.\n", myrow, mycol, MPIt2 - MPIt1);
free(A);

//exit and finanlize
blacs_gridexit_(&ictxt);
MPI_Finalize();
return 0;

/*
 * Origin A:
 *
4.00000 19.0000 25.0000 16.0000 19.0000 19.0000 25.0000 16.0000
22.0000 8.00000 17.0000 25.0000 22.0000 21.0000 17.0000 25.0000
23.0000 22.0000 7.00000 19.0000 23.0000 22.0000 18.0000 19.0000
21.0000 18.0000 23.0000 13.0000 21.0000 18.0000 23.0000 22.0000
19.0000 19.0000 25.0000 16.0000 4.00000 19.0000 25.0000 16.0000
22.0000 21.0000 17.0000 25.0000 22.0000 8.00000 17.0000 25.0000
23.0000 22.0000 18.0000 19.0000 23.0000 22.0000 7.00000 19.0000
21.0000 18.0000 23.0000 22.0000 21.0000 18.0000 23.0000 13.0000
*/

// After PDGETRF:
/*
23.0000 22.0000 7.00000 19.0000 23.0000 22.0000 18.0000 19.0000
0.173913 15.1739 23.7826 12.6957 15.0000 15.1739 21.8696 12.6957
0.956522 -0.859599 30.7479 17.7393 12.8940 13.0000 18.5817 17.7393
0.913043 -0.137536 0.646538 -14.0708 -6.27341 -8.40499 -2.44069 -5.07082
0.826087 0.0544413 0.582891 0.762348 -18.5499 -1.17005 -0.0305972 -6.86113
0.956522 -0.00286533 0.337340 -0.0624197 0.253277 -17.6137 -6.56767 2.29955
1.00000 0.00000 0.357749 0.451018 0.0961398 0.0424351 -16.2651 -3.49711
0.913043 -0.137536 0.646538 0.360379 0.216315 0.290848 -0.0218688 -11.5045
*/
```

6.5.2.3.2.2 P?POTRF

计算对称正定矩阵或者Hermite正定矩阵的Cholesky分解。

即$A = LL^T$或者 $A = U^T U$，U上三角矩阵，L下三角矩阵。

接口定义

C Interface：

void pspotrf_(const char *uplo, const int *n, float *a, const int *ia, const int *ja, const int *desca, int *info)

void pdpotrf_(const char *uplo, const int *n, double *a, const int *ia, const int *ja, const int *desca, int *info)

 版权所有 © 华为技术有限公司

void pcpotrf_(const char *uplo, const int *n, float _Complex *a, const int *ia, const int *ja, const int *desca, int *info)

void pzpotrf_(const char *uplo, const int *n, double _Complex *a, const int *ia, const int *ja, const int *desca, int *info)

Fortran Interface：

PSPOTRF(uplo, n, a, ia, ja, desca, info)

PDPOTRF(uplo, n, a, ia, ja, desca, info)

PCPOTRF(uplo, n, a, ia, ja, desca, info)

PZPOTRF(uplo, n, a, ia, ja, desca, info)

参数

参数	类型	范围	说明	输入/输出
uplo	字符	全局	U为上三角矩阵，L为下三角矩阵。	输入
n	整型	全局	要操作的列数，比如子矩阵的列数。	输入
a	• 在pspotrf中为单精度浮点型数组。 • 在pdpotrf中为双精度浮点型数组。 • 在pcpotrf中为单精度复数型数组。 • 在pzpotrf中为双精度复数型数组。	本地	• 调用前保存待分解的分布式矩阵A的本地部分。 • 当uplo = 'U'，调用后根据在全局矩阵中的位置存放对角线以及对角线以上的部分保存上三角矩阵U。 • 当uplo = 'L'，调用后根据在全局矩阵中的位置存放对角线以及对角线以下的部分保存下三角矩阵L。	输入，输出
ia	整型	全局	子矩阵在全局矩阵中的行索引。	输入
ja	整型	全局	子矩阵在全局矩阵中的列索引。	输入
desca	整型数组	本地，全局	分布式矩阵A的矩阵描述符。	输入
info	整型	全局	执行结果： • 等于0：退出成功。 • 小于0：第-info个参数值不合法。 • 大于0：A中info大小的主子式非正定，无法完成分解。	输出

 版权所有 © 华为技术有限公司

依赖

#include <kscalapack.h>

示例

C Interface：

```
int izero=0;
int ione=1;
int myrank_mpi, nprocs_mpi;
MPI_Init( &argc, &argv);
MPI_Comm_rank(MPI_COMM_WORLD, &myrank_mpi);
MPI_Comm_size(MPI_COMM_WORLD, &nprocs_mpi);

int n = 8;       // (Global) Matrix size
int nprow = 2;   // Number of row procs
int npcol = 2;   // Number of column procs
int nb = 4;      // (Global) Block size
char uplo='L';   // Matrix is lower triangular
char layout='R'; // Block cyclic, Row major processor mapping

printf("Usage: ./test matrix_size block_size nprocs_row nprocs_col\n");

if(argc > 1) {
    n = atoi(argv[1]);
}
if(argc > 2) {
    nb = atoi(argv[2]);
}
if(argc > 3) {
    nprow = atoi(argv[3]);
}
if(argc > 4) {
    npcol = atoi(argv[4]);
}

assert(nprow * npcol == nprocs_mpi);

// Initialize BLACS
int iam, nprocs;
int zero = 0;
int ictxt, myrow, mycol;
blacs_pinfo_(&iam, &nprocs) ; // BLACS rank and world size
blacs_get_(&zero, &zero, &ictxt ); // -> Create context
blacs_gridinit_(&ictxt, &layout, &nprow, &npcol ); // Context -> Initialize the grid
blacs_gridinfo_(&ictxt, &nprow, &npcol, &myrow, &mycol ); // Context -> Context grid info (# procs row/
col, current procs row/col)

// Compute the size of the local matrices
int mpA    = numroc_( &n, &nb, &myrow, &izero, &nprow ); // My proc -> row of local A
int nqA    = numroc_( &n, &nb, &mycol, &izero, &npcol ); // My proc -> col of local A

//allocate and fill the matrices A and B
double *A;
A = (double *)calloc(mpA*nqA,sizeof(double)) ;
if (A==NULL){ printf("Error of memory allocation A on proc %dx%d\n",myrow,mycol); exit(0); }
int k = 0;
for (int j = 0; j < nqA; j++) { // local col
    int l_j = j / nb; // which block
    int x_j = j % nb; // where within that block
    int J   = (l_j * npcol + mycol) * nb + x_j; // global col
    for (int i = 0; i < mpA; i++) { // local row
        int l_i = i / nb; // which block
        int x_i = i % nb; // where within that block
        int I   = (l_i * nprow + myrow) * nb + x_i; // global row
```

 版权所有 © 华为技术有限公司

```
        assert(I < n);
        assert(J < n);
        if(I == J) {
            A[k] = nb * nb;
        } else {
            A[k] = I +  J;
        }
        //printf("%d %d -> %d %d -> %f\n", i, j, I, J, A[k]);
        k++;
    }
}

//create descriptor
int descA[9];
int info=0;
int lddA = mpA > 1 ? mpA : 1;
descinit_( descA,  &n, &n, &nb, &nb, &izero, &izero, &ictxt, &lddA, &info);
if(info != 0) {
    printf("Error in descinit, info = %d\n", info);
}

//run pdpotrf_ and time
double MPIt1 = MPI_Wtime();
printf("[%dx%d] Starting \n", myrow, mycol);
pdpotrf_(&uplo, &n, A, &ione, &ione, descA, &info);
if (info != 0) {
    printf("Error in calculate, info = %d\n", info);
}
double MPIt2 = MPI_Wtime();
//exit and finanlize
blacs_gridexit_(&ictxt);
MPI_Finalize();
return 0;

/*
 * Origin A:
 *
*16 1 2 3 4 5 6 7
*1 16 3 4 5 6 7 8
*2 3 16 5 6 7 8 9
*3 4 5 16 7 8 9 10
*4 5 6 7 16 9 10 11
*5 6 7 8 9 16 11 12
*6 7 8 9 10 11 16 13
*7 8 9 10 11 12 13 16
*/


/*After PDPOTRF:
*4.00000 1.00000 2.00000 3.00000 4.00000 5.00000 6.00000 7.00000
*0.250000 3.99218 3.00000 4.00000 5.00000 6.00000 7.00000 8.00000
*0.500000 0.720158 3.90274 5.00000 6.00000 7.00000 8.00000 9.00000
*0.750000 0.954992 1.00884 3.67529 7.00000 8.00000 9.00000 10.0000
*1.00000 1.18983 1.18971 1.06481 3.32191 9.00000 10.0000 11.0000
*1.25000 1.42466 1.37058 1.17522 0.955149 2.86983 11.0000 12.0000
*1.50000 1.65949 1.55145 1.28562 0.996646 0.756689 2.31741 13.0000
*1.75000 1.89433 1.73232 1.39603 1.03814 0.734271 0.500013 1.59132
*/
```

6.5.2.3.2.3 P?GEQRF

计算矩阵的QR分解，即$A = Q \times R$。

接口定义

C Interface：

 版权所有 © 华为技术有限公司

void psgeqrf_(const int *m, const int *n, float*a, const int *ia, const int *ja, const int *desca, float*tau, float*work, int *lwork, int *info)

void pdgeqrf_(const int *m, const int *n, double *a, const int *ia, const int *ja, const int *desca, double *tau, double *work, int *lwork, int *info)

void pcgeqrf_(const int *m, const int *n, float _Complex *a, const int *ia, const int *ja, const int *desca, float _Complex *tau, float _Complex *work, int *lwork, int *info)

void pzgeqrf_(const int *m, const int *n, double _Complex *a, const int *ia, const int *ja, const int *desca, double _Complex *tau, double _Complex *work, int *lwork, int *info)

Fortran Interface：

PSGEQRF(m, n, a, ia, ja, desca, tau, work, lwork, info)

PDGEQRF(m, n, a, ia, ja, desca, tau, work, lwork, info)

PCGEQRF(m, n, a, ia, ja, desca, tau, work, lwork, info)

PZGEQRF(m, n, a, ia, ja, desca, tau, work, lwork, info)

参数

参数	类型	范围	说明	输入/输出
m	整型	全局	要操作的行数，比如子矩阵的行数。	输入
n	整型	全局	要操作的列数，比如子矩阵的列数。	输入
a	• 在psgeqrf中为单精度浮点型数组。 • 在pdgeqrf中为双精度浮点型数组。 • 在pcgeqrf中为单精度复数型数组。 • 在pzgeqrf中为双精度复数型数组。	本地	• 调用前保存待分解的矩阵A。 • 调用后对角线以及对角线以上的部分保存大小为min(m,n)*n的矩阵R（当m ≥ n时R为上三角矩阵），对角线以下元素和tau共同表示正交矩阵Q。	输入，输出
ia	整型	全局	子矩阵在全局矩阵中的行索引。	输入
ja	整型	全局	子矩阵在全局矩阵中的列索引。	输入
desca	整型数组	本地，全局	分布式矩阵A的矩阵描述符。	输入

 版权所有 © 华为技术有限公司

参数	类型	范围	说明	输入/输出
tau	• 在psgeqrf中为单精度浮点型数组。 • 在pdgeqrf中为双精度浮点型数组。 • 在pcgeqrf中为单精度复数型数组。 • 在pzgeqrf中为双精度复数型数组。	本地	初等反射的系数，长度为min(m,n)。	输出
work	• 在psgeqrf中为单精度浮点型数组。 • 在pdgeqrf中为双精度浮点型数组。 • 在pcgeqrf中为单精度复数型数组。 • 在pzgeqrf中为双精度复数型数组。	本地	临时存储空间，退出时返回最小和最佳的lwork。	输出
lwork	整型	本地，全局	Work数组的维度。	输入
info	整型	全局	执行结果： • 等于0：成功。 • 小于0：第-info个参数值不合法。	输出

依赖

#include <kscalapack.h>

示例

```
int izero=0;
int ione=1;
int myrank_mpi, nprocs_mpi;
MPI_Init( &argc, &argv);
MPI_Comm_rank(MPI_COMM_WORLD, &myrank_mpi);
MPI_Comm_size(MPI_COMM_WORLD, &nprocs_mpi);
```

 版权所有 © 华为技术有限公司

```
int n = 8;       // (Global) Matrix size
int nprow = 2;   // Number of row procs
int npcol = 2;   // Number of column procs
int nb = 4;      // (Global) Block size
char uplo='L';   // Matrix is lower triangular
char layout='R'; // Block cyclic, Row major processor mapping
double tau[8];
int lwork = 48;
double work[48];
printf("Usage: ./test matrix_size block_size nprocs_row nprocs_col\n");

if(argc > 1) {
    n = atoi(argv[1]);
}
if(argc > 2) {
    nb = atoi(argv[2]);
}
if(argc > 3) {
    nprow = atoi(argv[3]);
}
if(argc > 4) {
    npcol = atoi(argv[4]);
}

assert(nprow * npcol == nprocs_mpi);

// Initialize BLACS
int iam, nprocs;
int zero = 0;
int ictxt, myrow, mycol;
blacs_pinfo_(&iam, &nprocs) ; // BLACS rank and world size
blacs_get_(&zero, &zero, &ictxt ); // -> Create context
blacs_gridinit_(&ictxt, &layout, &nprow, &npcol ); // Context -> Initialize the grid
blacs_gridinfo_(&ictxt, &nprow, &npcol, &myrow, &mycol ); // Context -> Context grid info (# procs row/
col, current procs row/col)

// Compute the size of the local matrices
int mpA    = numroc_( &n, &nb, &myrow, &izero, &nprow ); // My proc -> row of local A
int nqA    = numroc_( &n, &nb, &mycol, &izero, &npcol ); // My proc -> col of local A

double *A;
A = (double *)calloc(mpA*nqA,sizeof(double)) ;
if (A==NULL){ printf("Error of memory allocation A on proc %dx%d\n",myrow,mycol); exit(0); }
int k = 0;
for (int j = 0; j < nqA; j++) { // local col
    int l_j = j / nb; // which block
    int x_j = j % nb; // where within that block
    int J   = (l_j * npcol + mycol) * nb + x_j; // global col
    for (int i = 0; i < mpA; i++) { // local row
        int l_i = i / nb; // which block
        int x_i = i % nb; // where within that block
        int I   = (l_i * nprow + myrow) * nb + x_i; // global row
        assert(I < n);
        assert(J < n);
        if(I == J) {
            A[k] = i + j + 1.5  +  (rand())%10;
        } else {
            A[k] = n + rand()% 10;
        }
        //printf("%d %d -> %d %d -> %f\n", i, j, I, J, A[k]);
        k++;
    }
}
int descA[9];
int info=0;
int ipiv[10] = {0};
int lddA = mpA > 1 ? mpA : 1;
descinit_( descA,  &n, &n, &nb, &nb, &izero, &izero, &ictxt, &lddA, &info);
if(info != 0) {
```

 版权所有 © 华为技术有限公司

```
    printf("Error in descinit, info = %d\n", info);
}
double MPIt1 = MPI_Wtime();
printf("[%dx%d] Starting \n", myrow, mycol);
pdgeqrf_(&n, &n, A,  &ione, &ione, descA, tau, work, &lwork, &info);
if (info != 0) {
    printf("Error in calculate, info = %d\n", info);
}
double MPIt2 = MPI_Wtime();
printf("[%dx%d] Done, time %e s.\n", myrow, mycol, MPIt2 - MPIt1);
free(A);
blacs_gridexit_(&ictxt);
MPI_Finalize();
return 0;
/*
 * Origin A:
/*
4.50000 11.0000 17.0000 8.00000 11.0000 11.0000 17.0000 8.00000
14.0000 8.50000 9.00000 17.0000 14.0000 13.0000 9.00000 17.0000
15.0000 14.0000 7.50000 11.0000 15.0000 14.0000 10.0000 11.0000
13.0000 10.0000 15.0000 13.5000 13.0000 10.0000 15.0000 14.0000
11.0000 11.0000 17.0000 8.00000 4.50000 11.0000 17.0000 8.00000
14.0000 13.0000 9.00000 17.0000 14.0000 8.50000 9.00000 17.0000
15.0000 14.0000 10.0000 11.0000 15.0000 14.0000 7.50000 11.0000
13.0000 10.0000 15.0000 14.0000 13.0000 10.0000 15.0000 13.5000
*/
    // After PDGEQRF:
/*
-36.3490 -31.6790 -32.1329 -35.4205 -35.1867 -31.6790 -32.1329 -35.4205
0.342726 8.46706 11.1681 0.285094 4.34884 6.07544 11.1681 0.285094
0.367206 0.114565 13.7477 3.02971 -0.816048 1.94287 13.2930 3.02971
0.318245 0.245466 -0.182222 7.89794 5.36891 -0.658967 0.174397 7.86629
0.269284 0.0337660 -0.199487 0.530335 5.89453 2.63366 -0.221784 0.0288312
0.342726 0.111495 0.258678 -0.396869 0.304197 4.86159 0.325469 -0.0199092
0.367206 0.114565 0.265211 0.293751 -0.0482493 -0.0280641 3.47962 0.00528634
0.318245 0.245466 -0.182222 0.173244 -0.0665625 0.227001 -0.112593 -0.705509
*/
```

6.5.2.3.3 线性方程组求解函数

6.5.2.3.3.1 P?GELS

使用A的QR或LQ分解求解满秩矩阵A或其转置的超定或欠定实线性系统。

- 如果TRANS = 'N'且m ≥ n：求超定系统的最小二乘解，即求解最小二乘问题min||B-AX||。
- 如果TRANS = 'N'且m < n：求欠定系统AX=B的最小范数解。
- 如果TRANS = 'T'且m ≥ n：求未定系统$A^TX=B$的最小范数解。
- 如果TRANS = 'T'且m < n：求超定系统的最小二乘解，即求解最小二乘问题min||B-AX||。

接口定义

C Interface：

void psgels_(const char *trans, const int *m, const int *n, const int *nrhs, float *a, const int *ia, const int *ja,const int *desca, float *b, const int *ib, const int *jb, const int *descb, float *work, int *lwork, int *info)

void pdgels_(const char *trans, const int *m, const int *n, const int *nrhs, double *a, const int *ia, const int *ja,const int *desca, double *b, const int *ib, const int *jb, const int *descb, double *work, int *lwork, int *info)

void pcgels_(const char *trans, const int *m, const int *n, const int *nrhs, float _Complex *a, const int *ia, const int *ja,const int *desca, float _Complex *b, const int *ib, const int *jb, const int *descb, float _Complex*work, int *lwork, int *info)

void pzgels_(const char *trans, const int *m, const int *n, const int *nrhs, double _Complex *a, const int *ia, const int *ja,const int *desca, double _Complex*b, const int *ib, const int *jb, const int *descb, double _Complex*work, int *lwork, int *info)

Fortran Interface：

PSGELS(trans, m, n, nrhs, a, ia, ja, desca, b, ib, jb, descb, work, lwork, info)

PDGELS(trans, m, n, nrhs, a, ia, ja, desca, b, ib, jb, descb, work, lwork, info)

PCGELS(trans, m, n, nrhs, a, ia, ja, desca, b, ib, jb, descb, work, lwork, info)

PZGELS(trans, m, n, nrhs, a, ia, ja, desca, b, ib, jb, descb, work, lwork, info)

参数

参数	类型	范围	说明	输入/输出
trans	字符型	全局	判断矩阵A是否转置。	输入
m	整型	全局	要操作的行数，比如子矩阵的行数。	输入
n	整型	全局	要操作的列数，比如子矩阵的列数。	输入
nrhs	整型	全局	右侧的数量，即分布式子矩阵(B)和X的列数。	输入
a	• 在psgels中为单精度浮点型数组。 • 在pdgels中为双精度浮点型数组。 • 在pcgels中为单精度复数型数组。 • 在pzgels中为双精度复数型数组。	本地	分布式矩阵a的本地部分。 • 当m ≥ n时，矩阵A存储它自己的QR分解结果（本地部分）。 • 当m < n时，矩阵A存储它自己的LQ分解结果（本地部分）。	输入，输出
ia	整型	全局	子矩阵在全局矩阵中的行索引。	输入
ja	整型	全局	子矩阵在全局矩阵中的列索引。	输入
desca	整型数组	本地，全局	分布式矩阵A的矩阵描述符。	输入

版权所有 © 华为技术有限公司

参数	类型	范围	说明	输入/输出
b	• 在psgels中为单精度浮点型数组。 • 在pdgels中为双精度浮点型数组。 • 在pcgels中为单精度复数型数组。 • 在pzgels中为双精度复数型数组。	本地	分布式矩阵B的本地部分。其中： • 当TRANS = 'N'时，B存储全局矩阵B的{IB:IB+M-1, JB:JB +NRHS-1}部分，否则存储B(IB:IB+N-1, JB: JB+NRHS-1)部分。 • 当TRANS = 'N'时，解向量存储在子矩阵B右侧的N×NRHS部分，否则存放在右侧的M×NRHS部分。	输入，输出
ib	整型	全局	子矩阵在全局矩阵中的行索引。	输入
jb	整型	全局	子矩阵在全局矩阵中的列索引。	输入
descb	整型数组	本地，全局	分布式矩阵B的矩阵描述符。	输入
work	实数组	本地	退出时，返回最小和最佳的lwork。	输出
lwork	整型	本地，全局	Work数组的维度。	输入
info	整型	全局	• 等于0：表示成功。 • 小于0：info=-i，表示第i个参数非法。 • 大于0：算法出错。	输出

依赖

#include <kscalapack.h>

示例

```
int izero=0;
int ione=1;
int myrank_mpi, nprocs_mpi;
MPI_Init( &argc, &argv);
MPI_Comm_rank(MPI_COMM_WORLD, &myrank_mpi);
MPI_Comm_size(MPI_COMM_WORLD, &nprocs_mpi);
char trans = 'N';
int n = 8;       // (Global) Matrix size
```

 版权所有 © 华为技术有限公司

```
  int nprow = 2;   // Number of row procs
  int npcol = 2;   // Number of column procs
  int nrhs = 5;
  int nb = 4;      // (Global) Block size
  char layout='R'; // Block cyclic, Row major processor mapping
  int lwork = 52;
  double work[52];
  printf("Usage: ./test matrix_size block_size nprocs_row nprocs_col\n");

  if(argc > 1) {
     n = atoi(argv[1]);
  }
  if(argc > 2) {
     nb = atoi(argv[2]);
  }
  if(argc > 3) {
     nprow = atoi(argv[3]);
  }
  if(argc > 4) {
     npcol = atoi(argv[4]);
  }

  assert(nprow * npcol == nprocs_mpi);

  // Initialize BLACS
  int iam, nprocs;
  int zero = 0;
  int ictxt, myrow, mycol;
  blacs_pinfo_(&iam, &nprocs) ; // BLACS rank and world size
  blacs_get_(&zero, &zero, &ictxt ); // -> Create context
  blacs_gridinit_(&ictxt, &layout, &nprow, &npcol ); // Context -> Initialize the grid
  blacs_gridinfo_(&ictxt, &nprow, &npcol, &myrow, &mycol ); // Context -> Context grid info (# procs row/
col, current procs row/col)

  // Compute the size of the local matrices
  int mpA    = numroc_( &n, &nb, &myrow, &izero, &nprow ); // My proc -> row of local A
  int nqA    = numroc_( &n, &nb, &mycol, &izero, &npcol ); // My proc -> col of local A
  int mpB    = numroc_( &n, &nb, &myrow, &izero, &nprow ); // My proc -> row of local B
  int nqB    = numroc_( &n, &nb, &mycol, &izero, &npcol ); // My proc -> col of local B
  double *A;
  A = (double *)calloc(mpA*nqA,sizeof(double)) ;
  if (A==NULL){ printf("Error of memory allocation A on proc %dx%d\n",myrow,mycol); exit(0); }
  int k = 0;
  for (int j = 0; j < nqA; j++) { // local col
     int l_j = j / nb; // which block
     int x_j = j % nb; // where within that block
     int J   = (l_j * npcol + mycol) * nb + x_j; // global col
     for (int i = 0; i < mpA; i++) { // local row
        int l_i = i / nb; // which block
        int x_i = i % nb; // where within that block
        int I   = (l_i * nprow + myrow) * nb + x_i; // global row
        assert(I < n);
        assert(J < n);
        if(I == J) {
           A[k] = i + j + 1.5  +  (rand())%10;
        } else {
           A[k] = n + rand()% 10;
        }
        //printf("%d %d -> %d %d -> %f\n", i, j, I, J, A[k]);

        k++;
     }
  }

  double *B;
  B = (double *)calloc(mpB*nqB,sizeof(double)) ;
  if (B==NULL){ printf("Error of memory allocation B on proc %dx%d\n",myrow,mycol); exit(0); }
  k = 0;
  for (int j = 0; j < nqB; j++) { // local col
```

 版权所有 © 华为技术有限公司

```
    int l_j = j / nb; // which block
    int x_j = j % nb; // where within that block
    int J   = (l_j * npcol + mycol) * nb + x_j; // global col
    for (int i = 0; i < mpB; i++) { // local row
        int l_i = i / nb; // which block
        int x_i = i % nb; // where within that block
        int I   = (l_i * nprow + myrow) * nb + x_i; // global row
        assert(I < n);
        assert(J < n);
        if(I == J) {
            B[k] = i + j + 1.5  +  (rand())%10;
        } else {
            B[k] = n + rand()% 10;
        }
        //printf("%d %d -> %d %d -> %f\n", i, j, I, J, B[k]);
         k++;
    }
}

int descA[9];
int descB[9];
int info=0;
int lddA = mpA > 1 ? mpA : 1;
int lddB = mpB > 1 ? mpB : 1;
descinit_( descA,  &n, &n, &nb, &nb, &izero, &izero, &ictxt, &lddA, &info);
descinit_( descB,  &n, &n, &nb, &nb, &izero, &izero, &ictxt, &lddB, &info);
if(info != 0) {
    printf("Error in descinit, info = %d\n", info);
}
double MPIt1 = MPI_Wtime();
printf("[%dx%d] Starting \n", myrow, mycol);
pdgels_(&trans, &n, &n, &nrhs, A, &ione, &ione, descA, B, &ione, &ione, descB, work, &lwork, &info);
if (info != 0) {
    printf("Error in calculate, info = %d\n", info);
}
double MPIt2 = MPI_Wtime();
printf("[%dx%d] Done, time %e s.\n", myrow, mycol, MPIt2 - MPIt1);
free(A);
free(B);
blacs_gridexit_(&ictxt);
MPI_Finalize();


//original matrix;
/*
beginA.dat
4.50000 11.0000 17.0000 8.00000 11.0000 11.0000 17.0000 8.00000
14.0000 8.50000 9.00000 17.0000 14.0000 13.0000 9.00000 17.0000
15.0000 14.0000 7.50000 11.0000 15.0000 14.0000 10.0000 11.0000
13.0000 10.0000 15.0000 13.5000 13.0000 10.0000 15.0000 14.0000
11.0000 11.0000 17.0000 8.00000 4.50000 11.0000 17.0000 8.00000
14.0000 13.0000 9.00000 17.0000 14.0000 8.50000 9.00000 17.0000
15.0000 14.0000 10.0000 11.0000 15.0000 14.0000 7.50000 11.0000
13.0000 10.0000 15.0000 14.0000 13.0000 10.0000 15.0000 13.5000
*/
double *B
/*
beginB.dat
3.40000 11.6000 12.6000 17.6000 10.6000 11.6000 12.6000 17.6000
16.6000 13.4000 10.6000 15.6000 16.6000 18.6000 10.6000 15.6000
12.6000 17.6000 9.40000 19.6000 12.6000 17.6000 12.6000 19.6000
16.6000 19.6000 13.6000 11.4000 16.6000 19.6000 13.6000 12.6000
10.6000 11.6000 12.6000 17.6000 3.40000 11.6000 12.6000 17.6000
16.6000 18.6000 10.6000 15.6000 16.6000 13.4000 10.6000 15.6000
12.6000 17.6000 12.6000 19.6000 12.6000 17.6000 9.40000 19.6000
16.6000 19.6000 13.6000 12.6000 16.6000 19.6000 13.6000 11.4000
*/

//print result
```

 版权所有 © 华为技术有限公司

```
/*
endA.dat
-36.3490 -31.6790 -32.1329 -35.4205 -35.1867 -31.6790 -32.1329 -35.4205
0.342726 8.46706 11.1681 0.285094 4.34884 6.07544 11.1681 0.285094
0.367206 0.114565 13.7477 3.02971 -0.816048 1.94287 13.2930 3.02971
0.318245 0.245466 -0.182222 7.89794 5.36891 -0.658967 0.174397 7.86629
0.269284 0.0337660 -0.199487 0.530335 5.89453 2.63366 -0.221784 0.0288312
0.342726 0.111495 0.258678 -0.396869 0.304197 4.86159 0.325469 -0.0199092
0.367206 0.114565 0.265211 0.293751 -0.0482493 -0.0280641 3.47962 0.00528634
0.318245 0.245466 -0.182222 0.173244 -0.0665625 0.227001 -0.112593 -0.705509
*/
/*
endB.dat
1.24237 1.03392 0.212867 -0.604674 0.134677 11.6000 12.6000 17.6000
-0.507703 0.100760 -0.00174929 1.23343 -0.507703 18.6000 10.6000 15.6000
0.141842 0.287368 0.919597 -0.117208 0.141842 17.6000 12.6000 19.6000
0.167171 -0.231371 -0.0104530 1.43888 0.167171 19.6000 13.6000 12.6000
0.134677 1.03392 0.212867 -0.604674 1.24237 11.6000 12.6000 17.6000
-0.507703 -1.05480 -0.00174929 1.23343 -0.507703 13.4000 10.6000 15.6000
0.141842 0.287368 -0.360403 -0.117208 0.141842 17.6000 9.40000 19.6000
0.167171 -0.231371 -0.0104530 -0.961121 0.167171 19.6000 13.6000 11.4000
*/
```

6.5.2.3.3.2 P?GESV

使用部分选主元的LU分解算法求解线性方程组Ax=B，其中A是N*N的分布式子矩阵，B是具有NRHS个向量的右端项矩阵。

接口定义

C Interface：

void psgesv_(const int *n, const int *nrhs, float *a, const int *ia, const int *ja, const int *desca, int *ipiv, float *b, const int *ib, const int *jb, const int *descb, int *info);

void pdgesv_(const int *n, const int *nrhs, double *a, const int *ia, const int *ja, const int *desca, int *ipiv, double *b, const int *ib, const int *jb, const int *descb, int *info);

void pcgesv_(const int *n, const int *nrhs, float _Complex *a, const int *ia, const int *ja, const int *desca, int *ipiv, float _Complex *b, const int *ib, const int *jb, const int *descb, int *info);

void pzgesv_(const int *n, const int *nrhs, double _Complex *a, const int *ia, const int *ja, const int *desca, int *ipiv, double _Complex *b, const int *ib, const int *jb, const int *descb, int *info);

Fortran Interface：

PSGESV(n, nrhs, a, ia, ja, desca, ipiv, b, ib, jb, descb, info)

PDGESV(n, nrhs, a, ia, ja, desca, ipiv, b, ib, jb, descb, info)

PCGESV(n, nrhs, a, ia, ja, desca, ipiv, b, ib, jb, descb, info)

PZGESV(n, nrhs, a, ia, ja, desca, ipiv, b, ib, jb, descb, info)

 版权所有 © 华为技术有限公司

参数

参数	类型	范围	说明	输入/输出
n	整型	全局	矩阵的行数和列数。	输入
nrhs	整型	全局	右侧的数量，即分布式子矩阵(B)和X的列数。	输入
a	• 在psgesv中为单精度浮点型数组。 • 在pdgesv中为双精度浮点型数组。 • 在pcgesv中为单精度复数型数组。 • 在pzgesv中为双精度复数型数组。	本地	• 调用前保存分布式矩阵A的本地M*N部分。 • 调用后保存本地部分存放的分解结果L和U，不保存L的对角线元素（均为1）。	输入，输出
ia	整型	全局	子矩阵A在全局矩阵中的行索引。	输入
ja	整型	全局	子矩阵A在全局矩阵中的列索引。	输入
desca	整型数组	本地，全局	分布式矩阵A的矩阵描述符。	输入
ipiv	整型	本地	包含了主元及交换信息。	输出
b	• 在psgesv中为单精度浮点型数组。 • 在pdgesv中为双精度浮点型数组。 • 在pcgesv中为单精度复数型数组。 • 在pzgesv中为双精度复数型数组。	本地	• 调用前保存右端项 • 调用后保存求解结果	输入，输出
ib	整型	全局	子矩阵B在全局矩阵中的行索引。	输入
jb	整型	全局	子矩阵B在全局矩阵中的列索引。	输入

版权所有 © 华为技术有限公司

参数	类型	范围	说明	输入/输出
descb	整型数组	本地，全局	分布式矩阵B的矩阵描述符。	输入
info	整型	全局	● 等于0：表示成功。 ● 小于0：info=-i，表示第i个参数非法。 ● 大于0：算法出错。	输出

依赖

#include <kscalapack.h>

示例

```
  int izero=0;
  int ione=1;
  int myrank_mpi, nprocs_mpi;
  MPI_Init( &argc, &argv);
  MPI_Comm_rank(MPI_COMM_WORLD, &myrank_mpi);
  MPI_Comm_size(MPI_COMM_WORLD, &nprocs_mpi);

  int n = 8;       // (Global) Matrix size
  int nprow = 2;   // Number of row procs
  int npcol = 2;   // Number of column procs
  int nb = 4;      // (Global) Block size
  char uplo='L';   // Matrix is lower triangular
  char layout='R'; // Block cyclic, Row major processor mapping
  int nrhs = 1;

  printf("Usage: ./test matrix_size block_size nprocs_row nprocs_col\n");

  if(argc > 1) {
     n = atoi(argv[1]);
  }
  if(argc > 2) {
     nb = atoi(argv[2]);
  }
  if(argc > 3) {
     nprow = atoi(argv[3]);
  }
  if(argc > 4) {
     npcol = atoi(argv[4]);
  }
  assert(nprow * npcol == nprocs_mpi);
  // Initialize BLACS
  int iam, nprocs;
  int zero = 0;
  int ictxt, myrow, mycol;
  blacs_pinfo_(&iam, &nprocs) ; // BLACS rank and world size
  blacs_get_(&zero, &zero, &ictxt ); // -> Create context
  blacs_gridinit_(&ictxt, &layout, &nprow, &npcol ); // Context -> Initialize the grid
  blacs_gridinfo_(&ictxt, &nprow, &npcol, &myrow, &mycol ); // Context -> Context grid info (# procs row/
col, current procs row/col)

  // Compute the size of the local matrices
```


版权所有 © 华为技术有限公司

```
int mpA    = numroc_( &n, &nb, &myrow, &izero, &nprow ); // My proc -> row of local A
int nqA    = numroc_( &n, &nb, &mycol, &izero, &npcol ); // My proc -> col of local A
int mpB    = numroc_( &n, &nb, &myrow, &izero, &nprow );
ofstream f1;
string filename = to_string(myrank_mpi)+"Abegin.dat";
f1.open(filename);
double *A;
A = (double *)calloc(mpA*nqA,sizeof(double)) ;
if (A==NULL){ printf("Error of memory allocation A on proc %dx%d\n",myrow,mycol); exit(0); }
int k = 0;
for (int j = 0; j < nqA; j++) { // local col
    int l_j = j / nb; // which block
    int x_j = j % nb; // where within that block
    int J   = (l_j * npcol + mycol) * nb + x_j; // global col
    for (int i = 0; i < mpA; i++) { // local row
        int l_i = i / nb; // which block
        int x_i = i % nb; // where within that block
        int I   = (l_i * nprow + myrow) * nb + x_i; // global row
        assert(I < n);
        assert(J < n);
        if(I == J) {
            A[k] = 2*n + 1.5  +  (rand())%10;
        } else {
            A[k] = i + j + rand()% 10;
        }
        //printf("%d %d -> %d %d -> %f\n", i, j, I, J, A[k]);
        f1 <<I << " "<<J << " " << A[k]<<endl;
        k++;
    }
}
f1.close();

//create descriptor
int descA[9];
int info=0;
int ipiv[10] = {0};
int lddB = mpB > 1 ? mpB : 1;
descinit_( descA,  &n, &n, &nb, &nb, &izero, &izero, &ictxt, &lddB, &info);
if(info != 0) {
    printf("Error in descinit, info = %d\n", info);
}

filename = to_string(myrank_mpi)+"Bbegin.dat";
f1.open(filename);
double *B;
B = (double *)calloc(mpA,sizeof(double)) ;
if (A==NULL){ printf("Error of memory allocation A on proc %dx%d\n",myrow,mycol); exit(0); }
k = 0;
for (int j = 0; j < mpB; j++) { // local col
    int l_i = j / nb; // which block
    int x_i = j % nb; // where within that block
    int I   = (l_i * nprow + myrow) * nb + x_i; // global row
   B[j] = j + 1.5  +  (rand())%10;
    f1 <<I << " " << B[j]<<endl;
}
f1.close();
int descB[9];
int nbrhs=1;
descinit_( descB,  &n, &nrhs, &nb, &nbrhs, &izero, &izero, &ictxt, &lddB, &info); // nbrhs need to be
revised when nrhs!=1



//run pdpotrf_ and time
double MPIt1 = MPI_Wtime();
printf("[%dx%d] Starting \n", myrow, mycol);
pdgesv_(&n, &nrhs, A, &ione, &ione, descA, ipiv, B, &ione, &ione, descB, &info);
if (info != 0) {
    printf("Error in calculate, info = %d\n", info);
```

 版权所有 © 华为技术有限公司

```
    }
    filename = to_string(myrank_mpi)+"Bend.dat";
    f1.open(filename);
    for (int j = 0; j < mpB; j++) {
        int l_i = j / nb; // which block
        int x_i = j % nb; // where within that block
        int I   = (l_i * nprow + myrow) * nb + x_i; // global row
       f1 <<I<< " " << B[j]<<endl;
    }
    f1.close();

    double MPIt2 = MPI_Wtime();
    printf("[%dx%d] Done, time %e s.\n", myrow, mycol, MPIt2 - MPIt1);
    filename = to_string(myrank_mpi)+"end.dat";
    f1.open(filename);
    k = 0;
    for (int j = 0; j < nqA; j++) { // local col
        int l_j = j / nb; // which block
        int x_j = j % nb; // where within that block
        int J   = (l_j * npcol + mycol) * nb + x_j; // global col
        for (int i = 0; i < mpA; i++) { // local row
            int l_i = i / nb; // which block
            int x_i = i % nb; // where within that block
            int I   = (l_i * nprow + myrow) * nb + x_i; // global row
            assert(I < n);
            assert(J < n);
            f1 <<I << " "<<J << " " << A[k]<<endl;
            k++;
        }
    }
    f1.close();
    free(A);
    /*
    origin A:
[[20.500000  4.000000 11.000000  3.000000  3.000000  4.000000 11.000000 3.000000]
 [ 7.000000 22.500000  4.000000 13.000000  7.000000  7.000000  4.000000 13.000000]
 [ 9.000000  9.000000 19.500000  8.000000  9.000000  9.000000  6.000000 8.000000]
 [ 8.000000  6.000000 12.000000 23.500000  8.000000  6.000000 12.000000 12.000000]
 [ 3.000000  4.000000 11.000000  3.000000 20.500000  4.000000 11.000000 3.000000]
 [ 7.000000  7.000000  4.000000 13.000000  7.000000 22.500000  4.000000 13.000000]
 [ 9.000000  9.000000  6.000000  8.000000  9.000000  9.000000 19.500000 8.000000]
 [ 8.000000  6.000000 12.000000 12.000000  8.000000  6.000000 12.000000 23.500000]]
origin B:
[[ 1.500000]
 [ 8.500000]
 [ 5.500000]
 [10.500000]
 [ 1.500000]
 [ 8.500000]
 [ 5.500000]
 [10.500000]]
X:
[[-0.073846]
 [ 0.069101]
 [ 0.047280]
 [ 0.273735]
 [-0.073846]
 [ 0.069101]
 [ 0.047280]
 [ 0.273735]]
    */
```

6.5.2.3.4 特征值问题函数

6.5.2.3.4.1 P?(SY/HE)EV

计算实数对称矩阵/复数厄米特矩阵A的所有特征值以及特征向量（可选），其中A是分布式矩阵。

接口定义

C Interface：

void pssyev_(const char *jobz, const char *uplo, const int *n, float *a, const int *ia, const int *ja, const int *desca, float *w, float *z, const int *iz, const int *jz, const int *descz, float *work, const int *lwork, int *info);

void pdsyev_(const char *jobz, const char *uplo, const int *n, double *a, const int *ia, const int *ja, const int *desca, double *w, double *z, const int *iz, const int *jz, const int *descz, double *work, const int *lwork, int *info);

void pcheev_(const char *jobz, const char *uplo, const int *n, float _Complex *a, const int *ia, const int *ja, const int *desca, float *w, float _Complex *z, const int *iz, const int *jz, const int *descz, float _Complex *work, const int *lwork, float *rwork, const int *lrwork, int *info);

void pzheev_(const char *jobz, const char *uplo, const int *n, double _Complex *a, const int *ia, const int *ja, const int *desca, double *w, double _Complex *z, const int *iz, const int *jz, const int *descz, double _Complex *work, const int *lwork, double *rwork, const int *lrwork, int *info);

Fortran Interface：

PSSYEV(jobz, uplo, n, a, ia, ja, desca, w, z, iz, jz, descz, work, lwork, info)

PDSYEV(jobz, uplo, n, a, ia, ja, desca, w, z, iz, jz, descz, work, lwork, info)

PCHEEV(jobz, uplo, n, a, ia, ja, desca, w, z, iz, jz, descz, work, lwork, rwork, lrwork, info)

PZHEEV(jobz, uplo, n, a, ia, ja, desca, w, z, iz, jz, descz, work, lwork, rwork, lrwork, info)

参数

参数	类型	范围	说明	输入/输出
jobz	字符型	全局	指定任务种类。 ● 'N'：只计算特征值。 ● 'V'：计算特征值和特征向量。	输入
uplo	字符型	全局	指定矩阵类型 ● 'U'：矩阵存储上三角部分。 ● 'L'：矩阵存储下三角部分。	输入
n	整型	全局	矩阵的行数和列数。	输入

 版权所有 © 华为技术有限公司

参数	类型	范围	说明	输入/输出
a	• 在pssyev中为单精度浮点型数组。 • 在pdsyev中为双精度浮点型数组。 • 在pcheev中为单精度复数型数组。 • 在pzheev中为双精度复数型数组。	本地	调用前保存待分解的分布式矩阵A的本地部分。 • 当uplo = 'U'，调用后根据在全局矩阵中的位置存放对角线以及对角线以上的部分保存上三角矩阵U。 • 当uplo = 'L'，调用后根据在全局矩阵中的位置存放对角线以及对角线以下的部分保存下三角矩阵L。	输入，输出
ia	整型	全局	子矩阵A在全局矩阵中的行索引。	输入
ja	整型	全局	子矩阵A在全局矩阵中的列索引。	输入
desca	整型数组	本地，全局	分布式矩阵A的矩阵描述符。	输入
w	• 在pssyev/pcheev中为单精度浮点型数组。 • 在pdsyev/pzheev中为双精度浮点型数组。	全局	如果info=0，特征值升序排列。	输出
z	• 在pssyev中为单精度浮点型数组。 • 在pdsyev中为双精度浮点型数组。 • 在pcheev中为单精度复数型数组。 • 在pzheev中为双精度复数型数组。	本地	存放特征向量的矩阵。全局大小N*N，本地大小LDD_z * LOCc(jz+n-1) • 如果jobz='V'，前M列存放矩阵相应特征值的特征向量。 • 如果jobz='N'，Z参数没有意义。	输出
iz	整型	全局	子矩阵Z在全局矩阵中的行索引	输入

版权所有 © 华为技术有限公司

参数	类型	范围	说明	输入/输出
jz	整型	全局	子矩阵Z在全局矩阵中的列索引。	输入
descz	整型数组	本地，全局	分布式矩阵Z的矩阵描述符。	输入
work	• 在pssyev中为单精度浮点型数组。 • 在pdsyev中为双精度浮点型数组。 • 在pcheev中为单精度复数型数组。 • 在pzheev中为双精度复数型数组。	本地	Info=0时，work(0)返回最优lwork大小。	输出
lwork	整型	本地	work数组需要的空间大小。	输入
rwork（复数类型特有）	• 在cheevd中为单精度复数型数组。 • 在zheevd中为双精度复数型数组。	本地	矩阵需要的空间大小。 • jobz='N'，lwork≥max(nb*(np0 + 1), 3) + 3*n • jobz='V'，lwork≥(np0 + nq0 + nb)*nb + 3*n + n*n	输出
lrwork（复数类型特有）	整型	本地	rwork数组需要的空间大小。	输入
info	整型	全局	• 等于0：表示成功。 • 小于0：info=-i，表示第i个参数非法。 • 大于0：算法出错。	输出

依赖

```
#include <kscalapack.h>
```

 版权所有 © 华为技术有限公司

示例

```
int izero=0;
int ione=1;
int myrank_mpi, nprocs_mpi;
MPI_Init( &argc, &argv);
MPI_Comm_rank(MPI_COMM_WORLD, &myrank_mpi);
MPI_Comm_size(MPI_COMM_WORLD, &nprocs_mpi);

int n = 8;       // (Global) Matrix size
int nprow = 2;   // Number of row procs
int npcol = 2;   // Number of column procs
int nb = 4;      // (Global) Block size
char uplo='L';   // Matrix is lower triangular
char jobs='V'; // Block cyclic, Row major processor mapping
double tau[8];
char layout='R';
double w[8];
int iz;
int jz;
int descz;
int lwork = 160;
double work[160];

if(argc > 1) {
    n = atoi(argv[1]);
}
if(argc > 2) {
    nb = atoi(argv[2]);
}
if(argc > 3) {
    nprow = atoi(argv[3]);
}
if(argc > 4) {
    npcol = atoi(argv[4]);
}

assert(nprow * npcol == nprocs_mpi);

// Initialize BLACS
int iam, nprocs;
int zero = 0;
int ictxt, myrow, mycol;
blacs_pinfo_(&iam, &nprocs) ; // BLACS rank and world size
blacs_get_(&zero, &zero, &ictxt ); // -> Create context
blacs_gridinit_(&ictxt, &layout, &nprow, &npcol ); // Context -> Initialize the grid
blacs_gridinfo_(&ictxt, &nprow, &npcol, &myrow, &mycol ); // Context -> Context grid info (# procs row/
col, current procs row/col)

// Compute the size of the local matrices
int mpA    = numroc_( &n, &nb, &myrow, &izero, &nprow ); // My proc -> row of local A
int nqA    = numroc_( &n, &nb, &mycol, &izero, &npcol ); // My proc -> col of local A
int mpZ    = numroc_( &n, &nb, &myrow, &izero, &nprow ); // My proc -> row of local Z
int nqZ    = numroc_( &n, &nb, &mycol, &izero, &npcol ); // My proc -> col of local Z

ofstream f1;
string filename = to_string(myrank_mpi)+"A.dat";
f1.open(filename);
double *A;
A = (double *)calloc(mpA*nqA,sizeof(double));
double *Z;
Z = (double *)calloc(mpZ*nqZ,sizeof(double));
if (A==NULL){ printf("Error of memory allocation A on proc %dx%d\n",myrow,mycol); exit(0); }
int k = 0;
double globalA[n][n];
for(int j = 0; j < n; j++) {
    for (int i = 0; i <= j; i++){
      if (i == j)    globalA[i][j]= 3 * n  +  (rand())%10;
      else {
          globalA[i][j] = n  +  (rand())%10;
```


版权所有 © 华为技术有限公司

```
            globalA[j][i] = n  +  (rand())%10;
          }
          if (myrank_mpi == 0)
          f1 <<i << " "<<j << " " << globalA[i][j]<<endl;
      }
    }
    f1.close();
    for (int j = 0; j < nqA; j++) { // local col
        int l_j = j / nb; // which block
        int x_j = j % nb; // where within that block
        int J   = (l_j * npcol + mycol) * nb + x_j; // global col
        for (int i = 0; i <= j; i++) { // local row
            int l_i = i / nb; // which block
            int x_i = i % nb; // where within that block
            int I   = (l_i * nprow + myrow) * nb + x_i; // global row
            assert(I < n);
            assert(J < n);
            A[k] = globalA[I][J];
            // printf("%d %d -> %d %d -> %f\n", i, j, I, J, A[k]);
            k++;
        }
    }

    int descA[9];
    int info=0;
    int ipiv[10] = {0};
    int lddA = mpA > 1 ? mpA : 1;
    descinit_( descA,  &n, &n, &nb, &nb, &izero, &izero, &ictxt, &lddA, &info);
    int descZ[9];
    descinit_( descZ,  &n, &n, &nb, &nb, &izero, &izero, &ictxt, &lddA, &info);

    if(info != 0) {
        printf("Error in descinit, info = %d\n", info);
    }
    double MPIt1 = MPI_Wtime();
    printf("[%dx%d] Starting \n", myrow, mycol);
    pdsyev_(&jobs, &uplo, &n, A,  &ione, &ione, descA, w, Z, &ione, &ione, descZ, work, &lwork, &info);
    if (info != 0) {
        printf("Error in calculate, info = %d\n", info);
    }
    double MPIt2 = MPI_Wtime();

    filename = to_string(myrank_mpi)+"Z.dat";
    f1.open(filename);
    k = 0;
    for (int j = 0; j < nqZ; j++) { // local col
        int l_j = j / nb; // which block
        int x_j = j % nb; // where within that block
        int J   = (l_j * npcol + mycol) * nb + x_j; // global col
        for (int i = 0; i < mpZ; i++) { // local row
            int l_i = i / nb; // which block
            int x_i = i % nb; // where within that block
            int I   = (l_i * nprow + myrow) * nb + x_i; // global row
            assert(I < n);
            assert(J < n);
            f1 <<I << " "<<J << " " << Z[k]<<endl;
            k++;
        }
    }
    f1.close();
    for (int i = 0; i < n; i++) {
        printf("%lf ",w[i]);
    }
    printf("\n[%dx%d] Done, time %e s.\n", myrow, mycol, MPIt2 - MPIt1);
    free(A);
    blacs_gridexit_(&ictxt);
    MPI_Finalize();
    return 0;
/*
```

 版权所有 © 华为技术有限公司

```
Origin A:
[[27.  0.  0.  0.  0.  0.  0.  0.]
 [14. 29.  0.  0.  0.  0.  0.  0.]
 [11. 14. 33.  0.  0.  0.  0.  0.]
 [ 9. 15. 17. 30.  0.  0.  0.  0.]
 [ 8. 10.  9. 15. 26.  0.  0.  0.]
 [ 8. 11. 13. 10. 16. 31.  0.  0.]
 [11.  9. 17.  9. 12. 16. 29.  0.]
 [ 8. 14.  8. 11.  8.  9. 13. 31.]]
Z:
[[-3.808840e-01  3.560630e-01  1.508930e-01  4.958660e-02 -2.479740e-01 5.474460e-01 -3.507160e-01
-4.676850e-01]
 [-1.215670e-01  9.875040e-02 -3.955830e-01 -3.017760e-02  3.132180e-01 1.838380e-01  7.044540e-01
-4.357930e-01]
 [ 5.219660e-01 -3.517350e-01 -4.716980e-01 -3.487300e-01 -2.010880e-01 2.340930e-01 -2.884750e-01
-2.850760e-01]
 [ 1.710940e-01 -4.669610e-01  7.019050e-01 -1.670770e-01  1.194220e-01 3.689920e-01  2.620540e-01
-1.140160e-01]
 [-3.644890e-01 -4.411160e-01  5.683900e-02  3.502880e-02 -3.815800e-01 -5.387890e-01  4.586880e-02
-4.796940e-01]
 [-8.826880e-04  3.503620e-02  1.090290e-01 -1.568230e-01  7.541030e-01 -2.801260e-01 -4.334450e-01
-3.567550e-01]
 [ 5.430560e-01  5.514490e-01  3.000940e-01 -1.117970e-01 -2.539910e-01 -3.196940e-01  1.823870e-01
-3.137810e-01]
 [ 3.326610e-01 -1.476200e-01 -1.999970e-02  8.993240e-01  8.318180e-02 6.770530e-02 -7.489810e-02
-2.030950e-01]]
W:
[-35.145752 -11.559735 -6.089042 -0.555364 16.500808 23.793150 25.601339 102.454598]
 */
```

6.5.2.3.4.2 P?(SY/HE)EVD

计算实数对称矩阵/复数厄米特矩阵A的所有特征值和特征向量，其中A是分布式矩阵。

接口定义

C Interface：

void pssyevd_(const char *jobz, const char *uplo, const int *n, float *a, const int *ia, const int *ja, const int *desca, float *w, float *z, const int *iz, const int *jz, const int *descz, float *work, const int *lwork, int *iwork, const int *liwork, int *info);

void pdsyevd_(const char *jobz, const char *uplo, const int *n, double *a, const int *ia, const int *ja, const int *desca, double *w, double *z, const int *iz, const int *jz, const int *descz, double *work, const int *lwork, int *iwork, const int *liwork, int *info);

void pcheevd_(const char *jobz, const char *uplo, const int *n, float _Complex *a, const int *ia, const int *ja, const int *desca, float *w, float _Complex *z, const int *iz, const int *jz, const int *descz, float _Complex *work, const int *lwork, float *rwork, const int *lrwork, int *iwork, const int *liwork, int *info);

void pzheevd_(const char *jobz, const char *uplo, const int *n, double _Complex *a, const int *ia, const int *ja, const int *desca, double *w, double _Complex *z, const int *iz, const int *jz, const int *descz, double _Complex *work, const int *lwork, double *rwork, const int *lrwork, int *iwork, const int *liwork, int *info);

Fortran Interface：

PSSYEVD(jobz, uplo, n, a, ia, ja, desca, w, z, iz, jz, descz, work, lwork, iwork, liwork, info)

 版权所有 © 华为技术有限公司

PDSYEVD(jobz, uplo, n, a, ia, ja, desca, w, z, iz, jz, descz, work, lwork, iwork, liwork, info)

PCHEEVD(jobz, uplo, n, a, ia, ja, desca, w, z, iz, jz, descz, work, lwork, rwork, lrwork, iwork, liwork, info)

PZHEEVD(jobz, uplo, n, a, ia, ja, desca, w, z, iz, jz, descz, work, lwork, rwork, lrwork, iwork, liwork, info)

参数

参数	类型	范围	说明	输入/输出
jobz	字符型	全局	指定任务种类。 ● 'N'：只计算特征值。 ● 'V'：计算特征值和特征向量。	输入
uplo	字符型	全局	指定矩阵类型。 ● 'U'：矩阵存储上三角部分。 ● 'L'：矩阵存储下三角部分。	输入
n	整型	全局	矩阵的行数和列数。	输入
a	● 在pssyevd中为单精度浮点型数组。 ● 在pdsyevd中为双精度浮点型数组。 ● 在pcheevd中为单精度复数型数组。 ● 在pzheevd中为双精度复数型数组。	本地	调用前保存待分解的分布式矩阵A的本地部分。 ● 当uplo = 'U'，调用后根据在全局矩阵中的位置存放对角线以及对角线以上的部分保存上三角矩阵U。 ● 当uplo = 'L'，调用后根据在全局矩阵中的位置存放对角线以及对角线以下的部分保存下三角矩阵L。	输入，输出
ia	整型	全局	子矩阵A在全局矩阵中的行索引。	输入
ja	整型	全局	子矩阵A在全局矩阵中的列索引。	输入
desca	整型数组	本地，全局	分布式矩阵A的矩阵描述符。	输入

版权所有 © 华为技术有限公司

参数	类型	范围	说明	输入/输出
w	• 在pssyevd/pcheevd中为单精度浮点型数组。 • 在pdsyevd/pzheevd中为双精度浮点型数组。	全局	如果info=0，特征值升序排列。	输出
z	• 在pssyevd中为单精度浮点型数组。 • 在pdsyevd中为双精度浮点型数组。 • 在pcheevd中为单精度复数型数组。 • 在pzheevd中为双精度复数型数组。	本地	存放特征向量的矩阵。全局大小N*N，本地大小LDD_z * LOCc(jz+n-1)。 • 如果jobz='V'，前M列存放矩阵相应特征值的特征向量。 • 如果jobz='N'，Z参数没有意义。	输出
iz	整型	全局	子矩阵Z在全局矩阵中的行索引	输入
jz	整型	全局	子矩阵Z在全局矩阵中的列索引。	输入
descz	整型数组	本地，全局	分布式矩阵Z的矩阵描述符。	输入
work	• 在pssyevd中为单精度浮点型数组。 • 在pdsyevd中为双精度浮点型数组。 • 在pcheevd中为单精度复数型数组。 • 在pzheevd中为双精度复数型数组。	本地	Info=0时，work(0)返回最优lwork大小。	输出

版权所有 © 华为技术有限公司

参数	类型	范围	说明	输入/输出
lwork	整型	本地/全局	work数组需要的空间大小。	输入
rwork（复数类型特有）	• 在cheevd中为单精度复数型数组。 • 在zheevd中为双精度复数型数组。	本地	矩阵需要的空间大小。 • jobz='N'，lrwork≥max(nb*(np0 + 1), 3) + 3*n • jobz='V'，lrwork≥(np0 + nq0 + nb)*nb + 3*n + n*n	输出
lrwork（复数类型特有）	整型	本地/全局	rwork数组需要的空间大小。	输入
iwork	整型数组	本地	Info=0时，iwork(0)返回最优liwork大小。	输出
liwork	整型	本地/全局	liwork数组的维度，liwork=7*n +8*npcol+2。	输入
info	整型	全局	• 等于0：表示成功。 • 小于0：info=-i，表示第i个参数非法。 • 大于0：算法出错。	输出

依赖

#include <kscalapack.h>

示例

```
int main(){
    int izero=0;
    int ione=1;
    int myrank_mpi, nprocs_mpi;
    MPI_Init( &argc, &argv);
    MPI_Comm_rank(MPI_COMM_WORLD, &myrank_mpi);
    MPI_Comm_size(MPI_COMM_WORLD, &nprocs_mpi);

    int n = 8;       // (Global) Matrix size
    int nprow = 2;   // Number of row procs
    int npcol = 2;   // Number of column procs
    int nb = 4;      // (Global) Block size
    char uplo='L';   // Matrix is lower triangular
    char jobs='V'; // Block cyclic, Row major processor mapping
```

 版权所有 © 华为技术有限公司

```
double tau[8];
char layout='R';
double w[8];
int iz;
int jz;
int descz;
int lwork = 160;
double work[lwork];
int liwork = 90;
int iwork[liwork];


if(argc > 1) {
    n = atoi(argv[1]);
}
if(argc > 2) {
    nb = atoi(argv[2]);
}
if(argc > 3) {
    nprow = atoi(argv[3]);
}
if(argc > 4) {
    npcol = atoi(argv[4]);
}

assert(nprow * npcol == nprocs_mpi);

// Initialize BLACS
int iam, nprocs;
int zero = 0;
int ictxt, myrow, mycol;
blacs_pinfo_(&iam, &nprocs) ; // BLACS rank and world size
blacs_get_(&zero, &zero, &ictxt ); // -> Create context
blacs_gridinit_(&ictxt, &layout, &nprow, &npcol ); // Context -> Initialize the grid
blacs_gridinfo_(&ictxt, &nprow, &npcol, &myrow, &mycol ); // Context -> Context grid info (# procs row/
col, current procs row/col)

// Compute the size of the local matrices
int mpA    = numroc_( &n, &nb, &myrow, &izero, &nprow ); // My proc -> row of local A
int nqA    = numroc_( &n, &nb, &mycol, &izero, &npcol ); // My proc -> col of local A
int mpZ    = numroc_( &n, &nb, &myrow, &izero, &nprow ); // My proc -> row of local Z
int nqZ    = numroc_( &n, &nb, &mycol, &izero, &npcol ); // My proc -> col of local Z

ofstream f1;
string filename = to_string(myrank_mpi)+"A.dat";
f1.open(filename);
double *A;
A = (double *)calloc(mpA*nqA,sizeof(double));
double *Z;
Z = (double *)calloc(mpZ*nqZ,sizeof(double));
if (A==NULL){ printf("Error of memory allocation A on proc %dx%d\n",myrow,mycol); exit(0); }
int k = 0;
double globalA[n][n];
for(int j = 0; j < n; j++) {
    for (int i = 0; i <= j; i++){
      if (i == j)    globalA[i][j]= 3 * n  +  (rand())%10;
      else {
          globalA[i][j] = n  +  (rand())%10;
        globalA[j][i] = n  +  (rand())%10;
       }
      if (myrank_mpi == 0)
      f1 <<i << " "<<j << " " << globalA[i][j]<<endl;
  }
}
f1.close();
for (int j = 0; j < nqA; j++) { // local col
    int l_j = j / nb; // which block
    int x_j = j % nb; // where within that block
    int J   = (l_j * npcol + mycol) * nb + x_j; // global col
```


版权所有 © 华为技术有限公司

```
        for (int i = 0; i <= j; i++) { // local row
            int l_i = i / nb; // which block
            int x_i = i % nb; // where within that block
            int I   = (l_i * nprow + myrow) * nb + x_i; // global row
            assert(I < n);
            assert(J < n);
            A[k] = globalA[I][J];
            // printf("%d %d -> %d %d -> %f\n", i, j, I, J, A[k]);
            k++;
        }
    }

    int descA[9];
    int info=0;
    int ipiv[10] = {0};
    int lddA = mpA > 1 ? mpA : 1;
    descinit_( descA,  &n, &n, &nb, &nb, &izero, &izero, &ictxt, &lddA, &info);
    int descZ[9];
    descinit_( descZ,  &n, &n, &nb, &nb, &izero, &izero, &ictxt, &lddA, &info);

    if(info != 0) {
        printf("Error in descinit, info = %d\n", info);
    }
    double MPIt1 = MPI_Wtime();
    printf("[%dx%d] Starting \n", myrow, mycol);
    pdsyevd_(&jobs, &uplo, &n, A,  &ione, &ione, descA, w, Z, &ione, &ione, descZ, work, &lwork, iwork,
&liwork, &info);
    if (info != 0) {
        printf("Error in calculate, info = %d\n", info);
    }
    double MPIt2 = MPI_Wtime();

    filename = to_string(myrank_mpi)+"Z.dat";
    f1.open(filename);
    k = 0;
    for (int j = 0; j < nqZ; j++) { // local col
        int l_j = j / nb; // which block
        int x_j = j % nb; // where within that block
        int J   = (l_j * npcol + mycol) * nb + x_j; // global col
        for (int i = 0; i < mpZ; i++) { // local row
            int l_i = i / nb; // which block
            int x_i = i % nb; // where within that block
            int I   = (l_i * nprow + myrow) * nb + x_i; // global row
            assert(I < n);
            assert(J < n);
            f1 <<I << " "<<J << " " << Z[k]<<endl;
            k++;
        }
    }
    f1.close();
    for (int i = 0; i < n; i++) {
        printf("%lf ",w[i]);
    }
    printf("\n[%dx%d] Done, time %e s.\n", myrow, mycol, MPIt2 - MPIt1);
    free(A);
    blacs_gridexit_(&ictxt);
    MPI_Finalize();
    return 0;
}
/*
Origin A:
[[27.  0.  0.  0.  0.  0.  0.  0.]
 [14. 29.  0.  0.  0.  0.  0.  0.]
 [11. 14. 33.  0.  0.  0.  0.  0.]
 [ 9. 15. 17. 30.  0.  0.  0.  0.]
 [ 8. 10.  9. 15. 26.  0.  0.  0.]
 [ 8. 11. 13. 10. 16. 31.  0.  0.]
 [11.  9. 17.  9. 12. 16. 29.  0.]
 [ 8. 14.  8. 11.  8.  9. 13. 31.]]
```


版权所有 © 华为技术有限公司

```
w://特征值
[-35.145752 -11.559735 -6.089042 -0.555364 16.500808 23.793150 25.601339 102.454598]
Z：//特征向量
[[ 3.808840e-01 -3.560630e-01 -1.508930e-01 -4.958660e-02 -2.479740e-01
   5.474460e-01  3.507160e-01 -4.676850e-01]
 [ 1.215670e-01 -9.875040e-02  3.955830e-01  3.017760e-02  3.132180e-01
   1.838380e-01 -7.044540e-01 -4.357930e-01]
 [-5.219660e-01  3.517350e-01  4.716980e-01  3.487300e-01 -2.010880e-01
   2.340930e-01  2.884750e-01 -2.850760e-01]
 [-1.710940e-01  4.669610e-01 -7.019050e-01  1.670770e-01  1.194220e-01
   3.689920e-01 -2.620540e-01 -1.140160e-01]
 [ 3.644890e-01  4.411160e-01 -5.683900e-02 -3.502880e-02 -3.815800e-01
  -5.387890e-01 -4.586880e-02 -4.796940e-01]
 [ 8.826880e-04 -3.503620e-02 -1.090290e-01  1.568230e-01  7.541030e-01
  -2.801260e-01  4.334450e-01 -3.567550e-01]
 [-5.430560e-01 -5.514490e-01 -3.000940e-01  1.117970e-01 -2.539910e-01
  -3.196940e-01 -1.823870e-01 -3.137810e-01]
 [-3.326610e-01  1.476200e-01  1.999970e-02 -8.993240e-01  8.318180e-02
   6.770530e-02  7.489810e-02 -2.030950e-01]]
*/
```

说明

如果调用该接口过程中出现警告"KML INFORM: GEQRF performance in eigensolver is suboptimal due values close to under/overflow"，可能会导致性能下降，建议将KML_FAST_EIGENSOLVER=0添加到环境变量。

6.5.2.3.5 矩阵求逆函数

6.5.2.3.5.1 PGETRI

计算分布式矩阵A的逆矩阵，其中A是LU分解后的LU矩阵。

接口定义

C Interface：

void psgetri_(const int *n, float *a, const int *ia, const int *ja, const int *desca, const int *ipiv, float *work, const int *lwork, int *iwork, const int *liwork, int *info);

void pdgetri_(const int *n, double *a, const int *ia, const int *ja, const int *desca, const int *ipiv, double *work, const int *lwork, int *iwork, const int *liwork, int *info);

void pcgetri_(const int *n, float _Complex *a, const int *ia, const int *ja, const int *desca,const int *ipiv, float _Complex *work, const int *lwork, int *iwork, const int *liwork, int *info);

void pzgetri_(const int *n, double _Complex *a, const int *ia, const int *ja, const int *desca, const int *ipiv, double _Complex *work, const int *lwork, int *iwork, const int *liwork, int *info);

Fortran Interface：

PSGETRI (n, a, ia, ja, desca, ipiv, work, lwork, iwork, liwork, info)

PDGETRI (n, a, ia, ja, desca, ipiv, work, lwork, iwork, liwork, info)

PCGETRI (n, a, ia, ja, desca, ipiv, work, lwork, iwork, liwork, info)

PZGETRI (n, a, ia, ja, desca, ipiv, work, lwork, iwork, liwork, info)

 版权所有 © 华为技术有限公司

参数

参数	类型	范围	说明	输入/输出
n	整型	全局	要操作的行数与列数，全局矩阵的行数。	输入
a	• 在psgetri中为单精度浮点型数组。 • 在pdgetri中为双精度浮点型数组。 • 在pcgetri中为单精度复数型数组。 • 在pzgetri中为双精度复数型数组。	本地	• 调用前保存分布式矩阵A，下三角为L，上三角为U。 • 调用后保存分解后的结果。	输入，输出
ia	整型	全局	子矩阵在全局矩阵中的行索引。	输入
ja	整型	全局	子矩阵在全局矩阵中的列索引。	输入
desca	整型数组	本地，全局	分布式矩阵A的矩阵描述符。	输入
ipiv	整型	本地	包含了主元信息。	输出
work	• 在psgetri中为单精度浮点型数组。 • 在pdgetri中为双精度浮点型数组。 • 在pcgetri中为单精度复数型数组。 • 在pzgetri中为双精度复数型数组。	本地	• lwork=0时，忽略该参数。 • lwork≠0时，为函数提供辅助空间，其中， – 如果lwork≠-1，空间大小至少为lwork； – 如果lwork=-1，空间大小至少为1，work[0]返回最小的可用空间大小。	输入，输出
lwork	整型	本地	数组work的空间大小，lwork≥LOCr（n+mod（ia-1，mb））*nb	输入
iwork	整型数组	本地	用于物理转置pivot的辅助数组	输入，输出
liwork	整型	本地/全局	iwork数组空间的大小	输入

版权所有 © 华为技术有限公司

参数	类型	范围	说明	输入/输出
info	整型	全局	执行结果： ● 等于0：成功。 ● 小于0：第-info个参数值不合法。 ● 大于0：矩阵U对角线上第info个元素为0，矩阵分解完成但U是奇异的，会导致求解线性方程组时出现除以零的错误。	输出

依赖

#include <kscalapack.h>

示例

C Interface：

```
int izero=0;
int ione=1;
int myid, nprocs_mpi;
MPI_Init( &argc, &argv);
MPI_Comm_rank(MPI_COMM_WORLD, &myid);
MPI_Comm_size(MPI_COMM_WORLD, &nprocs_mpi);
int n = 8;       // (Global) Matrix size
int nprow = 2;   // Number of row procs
int npcol = 2;   // Number of column procs
int nb = 4;      // (Global) Block size
char uplo='L';   // Matrix is lower triangular
char layout='R'; // Block cyclic, Row major processor mapping
if(argc > 1) {
    n = atoi(argv[1]);
}
if(argc > 2) {
    nb = atoi(argv[2]);
}
if(argc > 3) {
    nprow = atoi(argv[3]);
}
if(argc > 4) {
    npcol = atoi(argv[4]);
}
assert(nprow * npcol == nprocs_mpi);
// Initialize BLACS
int iam, nprocs;
int zero = 0;
int ictxt, myrow, mycol;
blacs_pinfo_(&iam, &nprocs) ; // BLACS rank and world size
blacs_get_(&zero, &zero, &ictxt ); // -> Create context
blacs_gridinit_(&ictxt, &layout, &nprow, &npcol ); // Context -> Initialize the grid
blacs_gridinfo_(&ictxt, &nprow, &npcol, &myrow, &mycol ); // Context -> Context grid info (# procs row/col, current procs row/col)
// Compute the size of the local matrices
int mpA    = numroc_( &n, &nb, &myrow, &izero, &nprow ); // My proc -> row of local A
int nqA    = numroc_( &n, &nb, &mycol, &izero, &npcol ); // My proc -> col of local A
ofstream f1;
string filename;
double *A;
```


版权所有 © 华为技术有限公司

```
A = (double *)calloc(mpA*nqA,sizeof(double)) ;
if (A==NULL){ printf("Error of memory allocation A on proc %dx%d\n",myrow,mycol); exit(0); }
int k = 0;
for (int j = 0; j < nqA; j++) { // local col
    int l_j = j / nb; // which block
    int x_j = j % nb; // where within that block
    int J   = (l_j * npcol + mycol) * nb + x_j; // global col
    for (int i = 0; i < mpA; i++) { // local row
        int l_i = i / nb; // which block
        int x_i = i % nb; // where within that block
        int I   = (l_i * nprow + myrow) * nb + x_i; // global row
        assert(I < n);
        assert(J < n);
        if(I == J) {
            A[k] = i + j + 1.5  +  (rand())%10;
        } else {
            A[k] = n + rand()% 10;
        }
        //printf("%d %d -> %d %d -> %f\n", i, j, I, J, A[k]);
        k++;
    }
}
if (myid == 0)
    printf("*****************begin test**********************\n");
//create descriptor
int descA[9];
int info=0;
int ipiv[10] = {0};
int lddA = mpA > 1 ? mpA : 1;
descinit_( descA,  &n, &n, &nb, &nb, &izero, &izero, &ictxt, &lddA, &info);
if(info != 0) {
    printf("Error in descinit, info = %d\n", info);
} else {
    if (myid == 0)
        printf("Finish descinit !\n");
}

pdgetrf_(&n, &n, A, &ione, &ione, descA, ipiv, &info);
int lwork = 20, liwork = 20; //
double * work;
int* iwork;
work = (double *)calloc(lwork, sizeof(double));
iwork = (int *)calloc(liwork, sizeof(int));
if (myid == 0)
    printf("Finish preparing\n");
filename = to_string(myid)+"begin.dat";
f1.open(filename);
k = 0;
for (int j = 0; j < nqA; j++) { // local col
    int l_j = j / nb; // which block
    int x_j = j % nb; // where within that block
    int J   = (l_j * npcol + mycol) * nb + x_j; // global col
    for (int i = 0; i < mpA; i++) { // local row
        int l_i = i / nb; // which block
        int x_i = i % nb; // where within that block
        int I   = (l_i * nprow + myrow) * nb + x_i; // global row
        assert(I < n);
        assert(J < n);
        f1 <<I << " "<<J << " " << A[k]<<endl;
        k++;
    }
}
f1.close();
double MPIt1 = MPI_Wtime();
printf("[%dx%d] Starting pdgetri \n", myrow, mycol);
pdgetri_(&n, A, &ione, &ione, descA, ipiv, work, &lwork, iwork, &liwork, &info);
if (info != 0) {
    printf("Error in calculate, info = %d\n", info);
}
```

 版权所有 © 华为技术有限公司

```
    double MPIt2 = MPI_Wtime();
    printf("[%dx%d] Done, time %e s.\n", myrow, mycol, MPIt2 - MPIt1);
    filename = to_string(myid)+"end.dat";
    f1.open(filename);
    k = 0;
    for (int j = 0; j < nqA; j++) { // local col
        int l_j = j / nb; // which block
        int x_j = j % nb; // where within that block
        int J   = (l_j * npcol + mycol) * nb + x_j; // global col
        for (int i = 0; i < mpA; i++) { // local row
            int l_i = i / nb; // which block
            int x_i = i % nb; // where within that block
            int I   = (l_i * nprow + myrow) * nb + x_i; // global row
            assert(I < n);
            assert(J < n);
            f1 <<I << " "<<J << " " << A[k]<<endl;
            k++;
        }
    }
    f1.close();
    free(A);
    //exit and finalize
    blacs_gridexit_(&ictxt);
    MPI_Finalize();
    return 0;

/*输入矩阵A
[[ 1.500000e+01  1.400000e+01  7.500000e+00  1.100000e+01  1.500000e+01
   1.400000e+01  1.000000e+01  1.100000e+01]
 [ 3.000000e-01  6.800000e+00  1.475000e+01  4.700000e+00  6.500000e+00
   6.800000e+00  1.400000e+01  4.700000e+00]
 [ 8.666670e-01 -3.137250e-01  1.312750e+01  5.441180e+00  2.039220e+00
  -1.223420e-16  1.072550e+01  5.941180e+00]
 [ 9.333330e-01 -9.803920e-03  1.633680e-01  5.890500e+00 -2.694170e-01
  -4.500000e+00 -1.948280e+00  5.808810e+00]
 [ 7.333330e-01  1.078430e-01  7.548540e-01 -7.946400e-01 -8.954380e+00
  -3.575880e+00 -1.487500e+00 -4.423370e-01]
 [ 9.333330e-01 -6.715690e-01  9.069270e-01  8.411780e-01 -3.062640e-01
   7.190140e+00  5.246760e-01 -5.202240e-01]
 [ 1.000000e+00  0.000000e+00  1.904410e-01 -1.759140e-01  4.866260e-02
  -8.589560e-02 -4.767850e+00 -1.327490e-01]
 [ 8.666670e-01 -3.137250e-01  1.000000e+00  8.488250e-02 -2.553930e-03
   5.185420e-02 -2.818240e-02 -9.709620e-01]]
*/
/*输出矩阵A
[[-0.130400 -0.056865  0.030892  0.076706  0.023446 -0.056865  0.030892
   0.076706]
 [ 0.043322 -0.078041  0.013146 -0.076609  0.043322  0.144181  0.013146
  -0.076609]
 [ 0.004289 -0.019433 -0.208930  0.028675  0.004289 -0.019433  0.191070
   0.028675]
 [ 0.014375  0.050912 -0.029025 -1.029910  0.014375  0.050912 -0.029025
   0.970093]
 [ 0.023446 -0.056865  0.030892  0.076706 -0.130400 -0.056865  0.030892
   0.076706]
 [ 0.043322  0.144181  0.013146 -0.076609  0.043322 -0.078041  0.013146
  -0.076609]
 [ 0.004289 -0.019433  0.191070  0.028675  0.004289 -0.019433 -0.208930
   0.028675]
 [ 0.014375  0.050912 -0.029025  0.970093  0.014375  0.050912 -0.029025
  -1.029910]]
*/
```

6.5.2.3.5.2 PPOTRI

计算经过Cholesky分解后的矩阵的逆矩阵。

 版权所有 © 华为技术有限公司

接口定义

C Interface：

void pspotri_(const char *uplo, const int *n, float *a, const int *ia, const int *ja, const int *desca, int *info)

void pdpotri_(const char *uplo, const int *n, double *a, const int *ia, const int *ja, const int *desca, int *info)

void pcpotri_(const char *uplo, const int *n, float _Complex *a, const int *ia, const int *ja, const int *desca, int *info)

void pzpotri_(const char *uplo, const int *n, double _Complex *a, const int *ia, const int *ja, const int *desca, int *info)

Fortran Interface：

PSPOTRI(uplo, n, a, ia, ja, desca, info)

PDPOTRI(uplo, n, a, ia, ja, desca, info)

PCPOTRI(uplo, n, a, ia, ja, desca, info)

PZPOTRI(uplo, n, a, ia, ja, desca, info)

参数

参数	类型	范围	说明	输入/输出
uplo	字符	全局	U为上三角矩阵，L为下三角矩阵。	输入
n	整型	全局	要操作的列数，比如子矩阵的列数。	输入
a	• 在pspotrf中为单精度浮点型数组。 • 在pdpotrf中为双精度浮点型数组。 • 在pcpotrf中为单精度复数型数组。 • 在pzpotrf中为双精度复数型数组。	本地	是全局实对称或Hermitian的本地部分，在该接口中，为进行Cholesky分解后的矩阵。 • 当uplo = 'L'，调用后根据在全局矩阵中的位置存放对角线以及对角线以下的部分保存下三角矩阵L。 • 当uplo = 'U'，仅存放上三角矩阵U。	输入，输出
ia	整型	全局	子矩阵在全局矩阵中的行索引。	输入
ja	整型	全局	子矩阵在全局矩阵中的列索引。	输入
desca	整型数组	本地，全局	分布式矩阵A的矩阵描述符。	输入

版权所有 © 华为技术有限公司

参数	类型	范围	说明	输入/输出
info	整型	全局	执行结果： ● 等于0：退出成功。 ● 小于0：第-info个参数值不合法。 ● 大于0：A中info大小的主子式非正定，无法完成分解。	输出

依赖

#include <kscalapack.h>

示例

```
int izero=0;
int ione=1;
int myid, nprocs_mpi;
MPI_Init( &argc, &argv);
MPI_Comm_rank(MPI_COMM_WORLD, &myid);
MPI_Comm_size(MPI_COMM_WORLD, &nprocs_mpi);

int n = 8;       // (Global) Matrix size
int nprow = 2;   // Number of row procs
int npcol = 2;   // Number of column procs
int nb = 4;      // (Global) Block size
char uplo='L';   // Matrix is lower triangular
char layout='R'; // Block cyclic, Row major processor mapping

printf("Usage: ./test matrix_size block_size nprocs_row nprocs_col\n");

if(argc > 1) {
    n = atoi(argv[1]);
}
if(argc > 2) {
    nb = atoi(argv[2]);
}
if(argc > 3) {
    nprow = atoi(argv[3]);
}
if(argc > 4) {
    npcol = atoi(argv[4]);
}

assert(nprow * npcol == nprocs_mpi);

// Initialize BLACS
int iam, nprocs;
int zero = 0;
int ictxt, myrow, mycol;
blacs_pinfo_(&iam, &nprocs) ; // BLACS rank and world size
blacs_get_(&zero, &zero, &ictxt ); // -> Create context
blacs_gridinit_(&ictxt, &layout, &nprow, &npcol ); // Context -> Initialize the grid
blacs_gridinfo_(&ictxt, &nprow, &npcol, &myrow, &mycol ); // Context -> Context grid info (# procs row/col, current procs row/col)

ofstream f1;
string filename;
// Compute the size of the local matrices
int mpA    = numroc_( &n, &nb, &myrow, &izero, &nprow ); // My proc -> row of local A
```


版权所有 © 华为技术有限公司

```
int nqA    = numroc_( &n, &nb, &mycol, &izero, &npcol ); // My proc -> col of local A

//allocate and fill the matrices A and B
double *A;
A = (double *)calloc(mpA*nqA,sizeof(double)) ;
if (A==NULL){ printf("Error of memory allocation A on proc %dx%d\n",myrow,mycol); exit(0); }
int k = 0;
for (int j = 0; j < nqA; j++) { // local col
   int l_j = j / nb; // which block
   int x_j = j % nb; // where within that block
   int J   = (l_j * npcol + mycol) * nb + x_j; // global col
   for (int i = 0; i < mpA; i++) { // local row
      int l_i = i / nb; // which block
      int x_i = i % nb; // where within that block
      int I   = (l_i * nprow + myrow) * nb + x_i; // global row
      assert(I < n);
      assert(J < n);
      if(I == J) {
         A[k] = nb * nb;
      } else {
         A[k] = I +  J;
      }
      //printf("%d %d -> %d %d -> %f\n", i, j, I, J, A[k]);
      k++;
   }
}
//create descriptor
int descA[9];
int info=0;
int lddA = mpA > 1 ? mpA : 1;
descinit_( descA,  &n, &n, &nb, &nb, &izero, &izero, &ictxt, &lddA, &info);
if(info != 0) {
   printf("Error in descinit, info = %d\n", info);
}

pdpotrf_(&uplo, &n, A, &ione, &ione, descA, &info);

filename = to_string(myid)+"begin.dat";
f1.open(filename);
k = 0;
for (int j = 0; j < nqA; j++) { // local col
   int l_j = j / nb; // which block
   int x_j = j % nb; // where within that block
   int J   = (l_j * npcol + mycol) * nb + x_j; // global col
   for (int i = 0; i < mpA; i++) { // local row
      int l_i = i / nb; // which block
      int x_i = i % nb; // where within that block
      int I   = (l_i * nprow + myrow) * nb + x_i; // global row
      assert(I < n);
      assert(J < n);
      f1 <<I << " "<<J << " " << A[k]<<endl;
      k++;
   }
}
f1.close();
//run pdpotri_ and time
double MPIt1 = MPI_Wtime();
printf("[%dx%d] Starting \n", myrow, mycol);
pdpotri_(&uplo, &n, A, &ione, &ione, descA, &info);
if (info != 0) {
   printf("Error in calculate, info = %d\n", info);
}
double MPIt2 = MPI_Wtime();
printf("[%dx%d] Finishing \n", myrow, mycol);
filename = to_string(myid)+"end.dat";
f1.open(filename);
k = 0;
for (int j = 0; j < nqA; j++) { // local col
   int l_j = j / nb; // which block
```

 版权所有 © 华为技术有限公司

```
        int x_j = j % nb; // where within that block
        int J   = (l_j * npcol + mycol) * nb + x_j; // global col
        for (int i = 0; i < mpA; i++) { // local row
            int l_i = i / nb; // which block
            int x_i = i % nb; // where within that block
            int I   = (l_i * nprow + myrow) * nb + x_i; // global row
            assert(I < n);
            assert(J < n);
            f1 <<I << " "<<J << " " << A[k]<<endl;
            k++;
        }
    }
    f1.close();
    free(A);
    //exit and finanlize
    blacs_gridexit_(&ictxt);
    MPI_Finalize();
    return 0;
/*
输入矩阵A
[[ 4.000000  1.000000  2.000000  3.000000  4.000000  5.000000  6.000000  7.000000]
 [ 0.250000  3.992180  3.000000  4.000000  5.000000  6.000000  7.000000  8.000000]
 [ 0.500000  0.720158  3.902740  5.000000  6.000000  7.000000  8.000000  9.000000]
 [ 0.750000  0.954992  1.008840  3.675290  7.000000  8.000000  9.000000 10.000000]
 [ 1.000000  1.189830  1.189710  1.064810  3.321910  9.000000 10.000000 11.000000]
 [ 1.250000  1.424660  1.370580  1.175220  0.955149  2.869830 11.000000 12.000000]
 [ 1.500000  1.659490  1.551450  1.285620  0.996646  0.756689  2.317410 13.000000]
 [ 1.750000  1.894330  1.732320  1.396030  1.038140  0.734271  0.500013  1.591320]]

输出矩阵A
[[ 8.797610e-02  1.000000e+00  2.000000e+00  3.000000e+00  4.000000e+00  5.000000e+00  6.000000e
+00  7.000000e+00]
 [ 2.352190e-02  9.302160e-02  3.000000e+00  4.000000e+00  5.000000e+00  6.000000e+00  7.000000e+00
8.000000e+00]
 [ 2.091620e-02  1.902120e-02  9.982790e-02  5.000000e+00  6.000000e+00  7.000000e+00  8.000000e+00
9.000000e+00]
 [ 1.726830e-02  1.542070e-02  1.295720e-02  1.095080e-01  7.000000e+00  8.000000e+00  9.000000e+00
1.000000e+01]
 [ 1.179650e-02  1.001990e-02  7.651130e-03  4.334850e-03  1.243600e-01  9.000000e+00  1.000000e+01
1.100000e+01]
 [ 2.676710e-03  1.018570e-03 -1.192280e-03 -4.287470e-03 -8.930260e-03  1.499980e-01  1.100000e+01
1.200000e+01]
 [-1.556280e-02 -1.698410e-02 -1.887910e-02 -2.153210e-02 -2.551170e-02 -3.214420e-02  2.045910e-01
1.300000e+01]
 [-7.028140e-02 -7.099200e-02 -7.193950e-02 -7.326610e-02 -7.525580e-02 -7.857210e-02 -8.520470e-02
3.948980e-01]]
*/
```

6.5.2.3.6 Support 函数

6.5.2.3.6.1 KSCALAPACKGetVersion

获取SCALAPACK产品版本信息。

接口定义

int KSCALAPACKGetVersion(KSCALAPACKVersion *ver);

 版权所有 © 华为技术有限公司

参数

参数名	类型	描述	输入/输出
ver	KSCALAPACKVersion结构体指针	存储版本信息，非空。	输入、输出

依赖

#include "kml_service.h"

#include "kscalapack.h"

示例

```
int main()
{
    KSCALAPACKVersion *ver = (KSCALAPACKVersion*)malloc(sizeof(KSCALAPACKVersion));
    int res = KSCALAPACKGetVersion(ver);
    if (res != 0) {
        printf("Get version error.\n");
        return 0;
    }
    printf("Product Name: %s\nProduct Version: %s\nComponent Name: %s\nComponent Version: %s
\nComponent AbiVersion: %s\nComponent AppendInfo: %s\nSoftware Name: %s\nSoftware Version: %s\n",
            ver->productName, ver->productVersion, ver->componentName,
            ver->componentVersion, ver->componentAbiVersion, ver->componentAppendInfo,
            ver->softwareName, ver->softwareVersion);
    return 0;
}
```

运行结果：

```
Product Name: Kunpeng HPCKit
Product Version: 24.0.RC1
Component Name: HPCKit-kml
Component Version: 2.3.0
Component AbiVersion: 1
Component AppendInfo: gcc
Software Name: hpckit-kml-scalapack
Software Version: 2.3.0
```

说明

版本号以实际运行结果为准，上述结果仅供参考。

6.5.3 KML_SOLVER 库函数说明

6.5.3.1 函数说明

稀疏求解器库（Sparse Solvers），提供直接求解器和迭代求解器，支持单节点多线程、集群MPI多进程。

- SOLVER提供单机版稀疏线性方程组求解器，稀疏矩阵是指大部分矩阵元素为零的矩阵。该求解器包含的求解方法有：
  - 迭代求解：CG（Conjugate Gradient，共轭梯度），支持单精度实数和双精度实数数据类型，不支持复数。

版权所有 © 华为技术有限公司

  - 迭代求解：GCR（Generalized Conjugate Residual，广义共轭残差），支持单精度实数和双精度实数数据类型，不支持复数。
  - 直接求解：LDLT分解和求解，支持对称不定矩阵的求解，支持单精度实数、双精度实数、单精度复数、双精度复数四种数据类型。
  - 直接求解：LU分解和求解，支持通用矩阵的求解，支持单精度实数、双精度实数、单精度复数、双精度复数四种数据类型。
  - 直接求解：LLT/LLH分解和求解，支持对称正定矩阵的求解，支持单精度实数、双精度实数、单精度复数、双精度复数四种数据类型。
  - 直接求解：DSS（Direct Sparse Solver，直接法稀疏矩阵求解器），可根据矩阵类型自动选择最佳求解方法。**直接法推荐使用此套接口**。
    - 支持对称正定、对称不定、Hermitian正定、Hermitian不定、结构对称和非对称矩阵（通用矩阵）的求解。
    - 支持单精度实数、双精度实数、单精度复数、双精度复数四种数据类型。
- SOLVER提供单机版预条件子。包含的预条件子有：
  - 结构化ILU0预条件子，支持单精度实数，不支持双精度实数、单精度复数、双精度复数。
  - ILU0预条件，支持单精度实数和双精度实数数据类型，不支持复数。
- SOLVER还提供集群版线性方程组求解器。该求解器包含的求解方法有：
  - 迭代求解：CG算法，支持单精度实数和双精度实数数据类型，不支持复数。
  - 迭代求解：BICGSTAB算法，支持单精度实数和双精度实数数据类型，不支持复数。
  - 迭代求解：GMRES算法，支持单精度实数和双精度实数数据类型，不支持复数。
  - 迭代求解：CSI算法，支持单精度实数和双精度实数数据类型，不支持复数。
  - 直接求解：SCADSS-LDLT分解和求解，支持对称不定矩阵的求解，支持单精度实数、双精度实数、单精度复数、双精度复数四种数据类型。
  - 直接求解：SCADSS-LU分解和求解，支持通用矩阵的求解，支持单精度实数、双精度实数、单精度复数、双精度复数四种数据类型。
  - 直接求解：SCADSS-LLT/LLH分解和求解，支持对称正定矩阵的求解，支持单精度实数、双精度实数、单精度复数、双精度复数四种数据类型。
  - 迭代求解：HMG（Hybrid Multigrid）混合多重网格法，支持结构化矩阵。
  - 迭代求解：TYAMG代数多重网格法，支持CSR格式数据类型，不支持CSC和COO格式。其中粗化类型支持PMIS和HMIS方法。
  - 直接求解：SCADSS（集群版直接法稀疏矩阵求解器），可根据矩阵类型自动选择最佳求解方法。**集群版直接法推荐使用此套接口**。
    - 支持对称正定、对称不定、Hermitian正定、Hermitian不定、结构对称和非对称矩阵（通用矩阵）的求解。
    - 支持单精度实数、双精度实数、单精度复数、双精度复数四种数据类型。
- SOLVER还提供集群版预条件子。包含的预条件子有：
  - ICC预条件，支持单精度实数和双精度实数数据类型，不支持复数。
  - ILU0预条件，支持单精度实数和双精度实数数据类型，不支持复数。

 版权所有 © 华为技术有限公司

- SOR预条件：支持单精度实数和双精度实数数据类型，不支持复数。
- ILUT预条件：支持单精度实数和双精度实数数据类型，不支持复数。
- AMG预条件：支持单精度实数和双精度实数数据类型，不支持复数。

6.5.3.2 数据类型和数据存储

6.5.3.2.1 数据类型

基本数据类型

数据类型	类型描述
int	32位整型
s	单精度实数浮点类型
d	双精度实数浮点类型
c	单精度复数浮点类型
z	双精度复数浮点类型

6.5.3.2.2 数据存储

向量全存储格式

存储向量中所有的元素。

稀疏矩阵存储格式

CSR（Compressed Sparse Row，行序稀疏矩阵压缩）格式包含三项要素：值（value）、列（column）、行索引（pointerB）。对于集群版的LLT和LLH方法，只存储上三角或者下三角部分。CSR稀疏矩阵A中三项要素说明如**表6-38**所示。

表 6-38 CSR 稀疏矩阵 A 中三项要素说明

参数名	描述
值（value）	• A矩阵中的非零元素，存储方式为行主序。 • 数组长度与A中非零元素个数相等。
列（column）	• value数组中第i个元素value[i]在矩阵A中的列序号。 • 数组长度与A中非零元素个数相等。
索引B（pointerB）	• 在基0索引中，pointerB(j)表示A矩阵第j行第一个非零元素在value数组中的索引。 • 数组长度与A的行数+1相等。

版权所有 © 华为技术有限公司

假设矩阵A为：$A=\begin{bmatrix} 2 & -3 & 0 & 7 \\ -3 & 0 & 1 & -6 \\ 0 & 1 & -4 & 5 \\ 7 & -6 & 5 & 0 \end{bmatrix}$，则A的CSR格式如表6-39所示。

当使用集群版的LLT或LLH方法，只需存储上三角矩阵，其格式如表6-40所示。

表 6-39 矩阵 A 的 CSR 格式

矩阵	索引方式	CSR格式
$A=\begin{bmatrix} 2 & -3 & 0 & 7 \\ -3 & 0 & 1 & -6 \\ 0 & 1 & -4 & 5 \\ 7 & -6 & 5 & 0 \end{bmatrix}$	基0索引	value = [2, -3, 7, -3, 1, -6, 1, -4, 5, 7, -6, 5] column = [0, 1, 3, 0, 2, 3, 1, 2, 3, 0, 1, 2] pointerB = [0, 3, 6, 9, 12]

表 6-40 对称矩阵 A 的上三角 CSR 格式

矩阵	索引方式	CSR格式
$A=\begin{bmatrix} 2 & -3 & 0 & 7 \\ -3 & 0 & 1 & -6 \\ 0 & 1 & -4 & 5 \\ 7 & -6 & 5 & 0 \end{bmatrix}$	基0索引	value = [2, -3, 7, 1, -6, -4, 5] column = [0, 1, 3, 2, 3, 2, 3] pointerB = [0, 3, 5, 7, 7]

CSC（Compressed Sparse Column，列序稀疏矩阵压缩），格式包含三项要素：值（value）、行（row）、列索引（pointerB），与**CSR**格式同理，不做赘述。

COO（Coordinate，坐标），格式包含三项要素：值（value）、行坐标（row）、列坐标（column）。COO稀疏矩阵A中三项要素说明如表6-41所示。

表 6-41 COO 稀疏矩阵 A 中三项要素说明

参数名	描述
值（value）	• A矩阵中的非零元素，存储方式为行主序。 • 数组长度与A中非零元素个数相等。
行坐标（row）	• value数组中第i个元素value[i]在矩阵A中的行序号。 • 数组长度与A中非零元素个数相等。
列坐标（column）	• value数组中第i个元素value[i]在矩阵A中的列序号。 • 数组长度与A中非零元素个数相等。

假设矩阵A为：$A=\begin{bmatrix} 2 & -3 & 0 & 7 \\ -3 & 0 & 1 & -6 \\ 0 & 1 & -4 & 5 \\ 7 & -6 & 5 & 0 \end{bmatrix}$，则A的COO格式如表6-42所示。

版权所有 © 华为技术有限公司

表 6-42 矩阵 A 的 COO 格式

矩阵	COO格式
$A=\begin{bmatrix} 2 & -3 & 0 & 7 \\ -3 & 0 & 1 & -6 \\ 0 & 1 & -4 & 5 \\ 7 & -6 & 5 & 0 \end{bmatrix}$	value = [2, -3, 7, -3, 1, -6, 1, -4, 5, 7, -6, 5] row = [0, 1, 3, 0, 2, 3, 1, 2, 3, 0, 1, 2] column = [0, 1, 3, 0, 2, 3, 1, 2, 3, 0, 1, 2]

稠密矩阵存储格式

稠密格式分为行主序和列主序，包含值（value）和主维度（ld，Leading dimension）。若稠密矩阵A为行主序，则ld为保存每一行所需要的数组长度，若为列主序，则ld表示保存每一列所需要的数组长度。稠密矩阵A参数含义如表6-43所示。

表 6-43 Dense 稠密矩阵 A

参数名	描述
值（value）	• A矩阵中的所有元素，存储方式为行（列）主序。 • 数组长度等于行（列）数乘以ld。
主维度（ld）	整数，表示两行（列）之间的元素间隔个数，大于等于列（行）元素个数。

假设矩阵A为：$A=\begin{bmatrix} 2 & -3 & 0 & 7 \\ -1 & 2 & 4 & -6 \\ 2 & 1 & -4 & 3 \\ 3 & 5 & 9 & 8 \end{bmatrix}$，则A的稠密存储格式如表6-44所示。

表 6-44 矩阵 A 的稠密格式

矩阵	Dense格式（行主序）	Dense格式（列主序）
$A=\begin{bmatrix} 2 & -3 & 0 & 7 \\ -1 & 2 & 4 & -6 \\ 2 & 1 & -4 & 3 \\ 3 & 5 & 9 & 8 \end{bmatrix}$	value = [2, -3, 0, 7, -1, 2, 4, -6, 2, 1, -4, 3, 3, 5, 9, 8] ld = 4	value = [2, -1, 2, 3, -3, 2, 1, 5, 0, 4, -4, 9, 7, -6, 3, 8] ld = 4

6.5.3.3 函数定义（单机版）

6.5.3.3.1 CG

版权所有 © 华为技术有限公司

6.5.3.3.1.1 KmlIssCgInit?I

初始化数据结构，并将用户提供的系数矩阵关联到求解系统中。初始化单机版的CG方法时使用完整的CSR格式。

接口定义

C Interface：

int KmlIssCgInitSI(KmlSolverTask **handle, int n, const float *a, const int *ja, const int *ia);

int KmlIssCgInitDI(KmlSolverTask **handle, int n, const double *a, const int *ja, const int *ia);

参数

参数名	类型	描述	输入/输出
handle	KmlSolverTask **	求解器句柄，handle不可为空指针。	输入/输出
n	int	n为矩阵A的维度。	输入
a	• 在KmlIssCgInitSI中为float *。 • 在KmlIssCgInitDI中为double *。	a为维度为非零元素的个数的数组，a[i]为第i个非零元的值。该数组的生命周期与handle相同。	输入
ja	int *	ja为维度为非零元素的个数的数组，ja[i]为第i个非零元的列号。该数组的生命周期与handle相同。	输入
ia	int *	ia为n+1维数组，ia[i]为第i行第一个非零元在所有非零元中的索引。该数组的生命周期与handle相同。	输入

返回值

返回值	类型	描述
KMLSS_NO_ERROR	int	正常执行。
KMLSS_NULL_ARGUMENT	int	{handle, a, ja, ia}中存在空参数。
KMLSS_BAD_N	int	参数n不是正整数。
KMLSS_NO_MEMORY	int	内存不足。

版权所有 © 华为技术有限公司

依赖

#include "kml_iss.h"

示例

C Interface：

```
KmlSolverTask *handle;
int ierr;
int n = 8;

double a[26] = {1.0, 1.0, 2.0, 9.0, 2.0, 1.0, -3.0, 2.0, 3.0, 2.0, 1.0, 1.0, 9.0, -5.0, 2.0, 6.0, 1.0, -3.0, 1.0, 4.0, 1.0,
-5.0, 7.0, 2.0,1.0, 2.0};
int ja[26] = {0, 3, 4, 1, 2, 3, 5, 1, 2, 7, 0, 1, 3, 6, 0, 4, 5, 1, 4, 5, 7, 3, 6, 2, 5, 7};
int ia[9] = {0, 3, 7, 10, 14, 17, 21, 23, 26};

ierr = KmlIssCgInitDI(&handle, n, a, ja, ia);
if (ierr != 0) {
    printf("\nERROR in KmlIssCgInitDI: %d", ierr);
    return 1;
}
```

6.5.3.3.1.2 KmlIssCgSetUserPreconditioner?I

关联用户自定义预条件回调函数。如不使用自定义的预条件，该接口可不调用。

接口定义

C Interface：

int KmlIssCgSetUserPreconditionerSI(KmlSolverTask **handle, void *ustruct, int (*fptr)(void *ustruct, float *x));

int KmlIssCgSetUserPreconditionerDI(KmlSolverTask **handle, void *ustruct, int (*fptr)(void *ustruct, double *x));

参数

参数名	类型	描述	输入/输出
handle	KmlSolverTask **	求解器句柄，传入之前步骤的变量。	输入/输出
ustruct	void *	指向用户数据的指针，每次调用预条件子时作为第一个参数传递。	输入
(*fptr)	int	用户自定义的函数指针，指向回调函数，该回调函数进行预条件子与其第二参数间的计算。	输入

返回值

返回值	类型	描述
KMLSS_NO_ERROR	int	正常执行。

 版权所有 © 华为技术有限公司

返回值	类型	描述
KMLSS_NULL_ARGUMENT	int	{handle, fptr}中存在空参数。

依赖

#include "kml_iss.h"

示例

C Interface：

```
class pc
{
    public:
        int n;
        double *b;
};
int mut(void *unstruct,double *x)
{
        /*
        Compute the product of the precondition matrix and the vector x,
    the precondition matrix is a diagonal matrix
    */
        int n;
        pc *a;
        a = (pc *)unstruct;
        n = a->n;
        for(int i=0; i<n; i++){
            x[i] = x[i] * a->b[i];
        }
        return 0;
    }

KmlSolverTask *handle;
int ierr;
int n = 8;

double a[26] = {1.0, 1.0, 2.0, 9.0, 2.0, 1.0, -3.0, 2.0, 3.0, 2.0, 1.0, 1.0, 9.0, -5.0, 2.0, 6.0, 1.0, -3.0, 1.0, 4.0, 1.0,
-5.0, 7.0, 2.0,1.0, 2.0};
int ja[26] = {0, 3, 4, 1, 2, 3, 5, 1, 2, 7, 0, 1, 3, 6, 0, 4, 5, 1, 4, 5, 7, 3, 6, 2, 5, 7};
int ia[9] = {0, 3, 7, 10, 14, 17, 21, 23, 26};

pc *unstruct = new pc;
unstruct->b = new double [n];
for(int i=0; i<n; i++){
        unstruct->b[i] = 2;
}
ierr = KmlIssCgInitDI(&handle, n, a, ja, ia);
ierr = KmlIssCgSetUserPreconditionerDI(&handle, unstruct, mut);
if (ierr != 0) {
    printf("\nERROR in KmlIssCgSetUserPreconditionerDI: %d", ierr);
    return 1;
    }
```

6.5.3.3.1.3 KmlIssCgSet?I?

设置迭代求解的相关参数。

接口定义

C Interface：

 版权所有 © 华为技术有限公司

int KmlIssCgSetSII(KmlSolverTask **handle, enum KML_SOLVER_PARAM param, const int *data, int nd);

int KmlIssCgSetDII(KmlSolverTask **handle, enum KML_SOLVER_PARAM param, const int *data, int nd);

int KmlIssCgSetSIS(KmlSolverTask **handle, enum KML_SOLVER_PARAM param, const float *data, int nd);

int KmlIssCgSetDID(KmlSolverTask **handle, enum KML_SOLVER_PARAM param, const double *data, int nd);

参数

参数名	类型	描述	输入/输出
handle	KmlSolverTask **	求解器句柄，传入之前步骤的变量。	输入/输出
param	enum KML_SOLVER_PARAM	• KMLSS_THRESHOLD表示迭代残差阈值。 • KMLSS_MAX_ITERATION_COUNT表示最大迭代次数。 • KMLSS_PRECONDITIONER_TYPE表示预条件子类型。 • KMLSS_VECTOR_NORM_TYPE表示迭代残差的计算范数。 • KMLSS_MATRIX_FORMAT表示矩阵稀疏存储格式。	输入
data	• 在KmlIssCgSetSII中为const int *。 • 在KmlIssCgSetDII中为const int *。 • 在KmlIssCgSetSIS中为const float *。 • 在KmlIssCgSetDID中为const double *。	迭代求解相关参数数组（如最大迭代次数）。 当前可选的计算范数： • 1-NORM：KMLSS_L1 • 2-NORM：KMLSS_L2 当前可选的稀疏矩阵存储格式： • CSR格式：KMLSS_CSR • ELLPACK格式：KMLSS_ELL • COO格式：KMLSS_COO • ELLPACK_COO混合格式：KMLSS_SELLCOO 当前可选择的预条件子类型： • ILU预条件子：KMLSS_ILU • ICC预条件子：KMLSS_ICC • SOR预条件子：KMLSS_SOR • ILUT预条件子：KMLSS_ILUT	输入

 版权所有 © 华为技术有限公司

参数名	类型	描述	输入/输出
nd	int	data数组元素个数。	输入

返回值

返回值	类型	描述
KMLSS_NO_ERROR	int	正常执行。
KMLSS_DATA_SIZE	int	参数nd不等于1。
KMLSS_NULL_ARGUMENT	int	{handle, data}中存在空参数。
KMLSS_BAD_SELECTOR	int	param为无效参数。

依赖

#include "kml_iss.h"

示例

C Interface：

```
KmlSolverTask *handle;
int ierr;
int n = 8;

double a[26] = {1.0, 1.0, 2.0, 9.0, 2.0, 1.0, -3.0, 2.0, 3.0, 2.0, 1.0, 1.0, 9.0, -5.0, 2.0, 6.0, 1.0, -3.0, 1.0, 4.0, 1.0,
-5.0, 7.0, 2.0,1.0, 2.0};
int ja[26] = {0, 3, 4, 1, 2, 3, 5, 1, 2, 7, 0, 1, 3, 6, 0, 4, 5, 1, 4, 5, 7, 3, 6, 2, 5, 7};
int ia[9] = {0, 3, 7, 10, 14, 17, 21, 23, 26};

ierr = KmlIssCgInitDI(&handle, n, a, ja, ia);

int nd = 1;
int idata[] = {0};
double ddata[] = {0};
double eps = 1e-4;
ierr = KmlIssCgSetDID(&handle, KMLSS_THRESHOLD, &eps, nd);
if (ierr != 0) {
    printf("\nERROR in KmlIssCgSetDID: %d", ierr);
    return 1;
    }
int max_iters = 2000;
ierr = KmlIssCgSetDII(&handle, KMLSS_MAX_ITERATION_COUNT, &max_iters, nd);
if (ierr != 0) {
    printf("\nERROR in KmlIssCgSetDII: %d", ierr);
    return 1;
    }
```

6.5.3.3.1.4 KmlIssCgAnalyze?I

分析稀疏矩阵并改变存储形式。

 版权所有 © 华为技术有限公司

接口定义

C Interface：

int KmlIssCgAnalyzeSI(KmlSolverTask **handle);

int KmlIssCgAnalyzeDI(KmlSolverTask **handle);

参数

参数名	类型	描述	输入/输出
handle	KmlSolverTask **	求解器句柄，传入之前步骤的变量。	输入/输出

返回值

返回值	类型	描述
KMLSS_NO_ERROR	int	正常执行。
KMLSS_NULL_ARGUMENT	int	handle为空参数。

依赖

#include "kml_iss.h"

示例

C Interface：

```
KmlSolverTask *handle;
int ierr;
int n = 8;

double a[26] = {1.0, 1.0, 2.0, 9.0, 2.0, 1.0, -3.0, 2.0, 3.0, 2.0, 1.0, 1.0, 9.0, -5.0, 2.0, 6.0, 1.0, -3.0, 1.0, 4.0, 1.0,
-5.0, 7.0, 2.0,1.0, 2.0};
int ja[26] = {0, 3, 4, 1, 2, 3, 5, 1, 2, 7, 0, 1, 3, 6, 0, 4, 5, 1, 4, 5, 7, 3, 6, 2, 5, 7};
int ia[9] = {0, 3, 7, 10, 14, 17, 21, 23, 26};

ierr = KmlIssCgInitDI(&handle, n, a, ja, ia);
ierr = KmlIssCgAnalyzeDI(&handle);
if (ierr!= 0) {
   printf("\nERROR in KmlIssCgAnalyzeDI: %d", ierr);
   return 1;
   }
```

6.5.3.3.1.5 KmlIssCgFactorize?I

计算预条件子。

 版权所有 © 华为技术有限公司

接口定义

C Interface：

int KmlIssCgFactorizeSI(KmlSolverTask **handle);

int KmlIssCgFactorizeDI(KmlSolverTask **handle);

参数

参数名	类型	描述	输入/输出
handle	KmlSolverTask **	求解器句柄，传入之前步骤的变量。	输入/输出

返回值

返回值	类型	描述
KMLSS_NO_ERROR	int	正常执行。
KMLSS_NULL_ARGUMENT	int	handle为空参数。

依赖

#include "kml_iss.h"

示例

C Interface：

```
KmlSolverTask *handle;
int ierr;
int n = 8;

double a[26] = {1.0, 1.0, 2.0, 9.0, 2.0, 1.0, -3.0, 2.0, 3.0, 2.0, 1.0, 1.0, 9.0, -5.0, 2.0, 6.0, 1.0, -3.0, 1.0, 4.0, 1.0,
-5.0, 7.0, 2.0,1.0, 2.0};
int ja[26] = {0, 3, 4, 1, 2, 3, 5, 1, 2, 7, 0, 1, 3, 6, 0, 4, 5, 1, 4, 5, 7, 3, 6, 2, 5, 7};
int ia[9] = {0, 3, 7, 10, 14, 17, 21, 23, 26};

ierr = KmlIssCgInitDI(&handle, n, a, ja, ia);
ierr = KmlIssCgAnalyzeDI(&handle);
ierr = KmlIssCgFactorizeDI(&handle);
if (ierr != 0) {
    printf("\nERROR in KmlIssCgFactorizeDI: %d", ierr);
    return 1;
    }
```

6.5.3.3.1.6 KmlIssCgSolve?I

求解线性代数方程组。

版权所有 © 华为技术有限公司

接口定义

C Interface：

int KmlIssCgSolveSI(KmlSolverTask **handle, int nb, float *x, int ldx, const float *b, int ldb);

int KmlIssCgSolveDI(KmlSolverTask **handle, int nb, double *x, int ldx, const double *b, int ldb);

参数

参数名	类型	描述	输入/输出
handle	KmlSolverTask **	求解器句柄，传入之前步骤的变量。	输入/输出
nb	int	右端项矢量的个数，当前nb只能为1。	输入
x	• 在KmlIssCgSolveSI中为float * • 在KmlIssCgSolveDI中为double *	读取并存储求解结果，其元素个数为n。	输入/输出
ldx	int	x的leading dimension大小，当前参数未使用。	输入
b	• 在KmlIssCgSolveSI中为const float * • 在KmlIssCgSolveDI中为const double *	读取右端项，其元素个数为n。	输入
ldb	int	b的leading dimension大小，当前参数未使用。	输入

返回值

返回值	类型	描述
KMLSS_NO_ERROR	int	正常执行。
KMLSS_BAD_NB	int	参数nb不等于1。
KMLSS_NO_MEMORY	int	内存不足。
KMLSS_NULL_ARGUMENT	int	{handle, x, b}中存在空参数。

版权所有 © 华为技术有限公司

依赖

#include "kml_iss.h"

示例

C Interface：

```
KmlSolverTask *handle;
int ierr;
int n = 8;

double a[26] = {1.0, 1.0, 2.0, 9.0, 2.0, 1.0, -3.0, 2.0, 3.0, 2.0, 1.0, 1.0, 9.0, -5.0, 2.0, 6.0, 1.0, -3.0, 1.0, 4.0, 1.0,
-5.0, 7.0, 2.0,1.0, 2.0};
int ja[26] = {0, 3, 4, 1, 2, 3, 5, 1, 2, 7, 0, 1, 3, 6, 0, 4, 5, 1, 4, 5, 7, 3, 6, 2, 5, 7};
int ia[9] = {0, 3, 7, 10, 14, 17, 21, 23, 26};

ierr = KmlIssCgInitDI(&handle, n, a, ja, ia);
ierr = KmlIssCgAnalyzeDI(&handle);
ierr = KmlIssCgFactorizeDI(&handle);

int nd = 1;
int idata[] = {0};
double ddata[] = {0};
double eps = 1e-4;
ierr = KmlIssCgSetDID(&handle, KMLSS_THRESHOLD, &eps, nd);
int max_iters = 2000;
ierr = KmlIssCgSetDII(&handle, KMLSS_MAX_ITERATION_COUNT, &max_iters, nd);

int ldx = n;
int ldb = n;
int nb = 1;
double b[8] = {4.0, 9.0, 7.0, 6.0, 9.0, 3.0, 2.0, 5.0};
double x[8];
ierr = KmlIssCgSolveDI(&handle, nb, x, ldx, b, ldb);
if (ierr != 0) {
    printf("\nERROR in KmlIssCgSolveDI: %d", ierr);
    return 1;
    }
```

6.5.3.3.1.7 KmlIssCgGet?I?

获得迭代求解相关参数。

接口定义

C Interface：

int KmlIssCgGetSII(KmlSolverTask **handle, enum KML_SOLVER_PARAM param, int *data, int nd);

int KmlIssCgGetDII(KmlSolverTask **handle, enum KML_SOLVER_PARAM param, int *data, int nd);

int KmlIssCgGetSIS(KmlSolverTask **handle, enum KML_SOLVER_PARAM param, float *data, int nd);

int KmlIssCgGetDID(KmlSolverTask **handle, enum KML_SOLVER_PARAM param, double *data, int nd);

 版权所有 © 华为技术有限公司

参数

参数名	类型	描述	输入/输出
handle	KmlSolverTask **	求解器句柄，传入之前步骤的变量。	输入/输出
param	enum KML_SOLVER_PARAM	• KMLSS_ITERATION_COUNT表示迭代次数。 • KMLSS_TOLERANCE表示最终的迭代残差。	输入
data	• 在KmlIssCgGetSII中为int *。 • 在KmlIssCgGetDII中为int *。 • 在KmlIssCgGetSIS中为float *。 • 在KmlIssCgGetDID中为double *。	• 在KmlIssCgGetSII中为迭代步数。 • 在KmlIssCgGetDII中为迭代步数。 • 在KmlIssCgGetSIS中为残差。 • 在KmlIssCgGetDID中为残差。	输出
nd	int	data数组元素个数。	输入

返回值

返回值	类型	描述
KMLSS_NO_ERROR	int	正常执行。
KMLSS_DATA_SIZE	int	参数nd不等于1。
KMLSS_NULL_ARGUMENT	int	{handle, data}中存在空参数。
KMLSS_BAD_SELECTOR	int	param为无效参数。

依赖

#include "kml_iss.h"

示例

C Interface：

```
KmlSolverTask *handle;
int ierr;
int n = 8;

double a[26] = {1.0, 1.0, 2.0, 9.0, 2.0, 1.0, -3.0, 2.0, 3.0, 2.0, 1.0, 1.0, 9.0, -5.0, 2.0, 6.0, 1.0, -3.0, 1.0, 4.0, 1.0, -5.0, 7.0, 2.0,1.0, 2.0};
int ja[26] = {0, 3, 4, 1, 2, 3, 5, 1, 2, 7, 0, 1, 3, 6, 0, 4, 5, 1, 4, 5, 7, 3, 6, 2, 5, 7};
```

 版权所有 © 华为技术有限公司

```
int ia[9] = {0, 3, 7, 10, 14, 17, 21, 23, 26};

ierr = KmlIssCgInitDI(&handle, n, a, ja, ia);
ierr = KmlIssCgAnalyzeDI(&handle);
ierr = KmlIssCgFactorizeDI(&handle);

int nd = 1;
int idata[] = {0};
double ddata[] = {0};
double eps = 1e-4;
ierr = KmlIssCgSetDID(&handle, KMLSS_THRESHOLD, &eps, nd);
int max_iters = 2000;
ierr = KmlIssCgSetDII(&handle, KMLSS_MAX_ITERATION_COUNT, &max_iters, nd);

int ldx = n;
int ldb = n;
int nb = 1;
double b[8] = {4.0, 9.0, 7.0, 6.0, 9.0, 3.0, 2.0, 5.0};
double x[8];
ierr = KmlIssCgSolveDI(&handle, nb, x, ldx, b, ldb);
ierr = KmlIssCgGetDII(&handle, KMLSS_ITERATION_COUNT, idata, nd);
if (ierr != 0) {
    printf("\nERROR in KmlIssCgGetDII: %d", ierr);
    return 1;
    }
ierr = KmlIssCgGetDID(&handle, KMLSS_TOLERANCE, ddata, nd);
if (ierr != 0) {
    printf("\nERROR in KmlIssCgGetDID: %d", ierr);
    return 1;
    }
printf("[Solve info] Res= \t%.12g\t, iteration = \t%d\t\n", sqrt(ddata[0]), idata[0]);
```

运行结果

```
[Solve info] Res=       1.25581676326e-12       , iteration =   8
```

6.5.3.3.1.8 KmlIssCgClean?I

释放内部数据结构。

接口定义

C Interface：

int KmlIssCgCleanSI(KmlSolverTask **handle);

int KmlIssCgCleanDI(KmlSolverTask **handle);

参数

参数名	类型	描述	输入/输出
handle	KmlSolverTask **	求解器句柄，传入之前步骤的变量。	输入/输出

版权所有 © 华为技术有限公司

返回值

返回值	类型	描述
KMLSS_NO_ERROR	int	正常执行。
KMLSS_NULL_ARGUMENT	int	handle为空参数。

依赖

#include "kml_iss.h"

示例

C Interface：

```
KmlSolverTask *handle;
int ierr;
int n = 8;

double a[26] = {1.0, 1.0, 2.0, 9.0, 2.0, 1.0, -3.0, 2.0, 3.0, 2.0, 1.0, 1.0, 9.0, -5.0, 2.0, 6.0, 1.0, -3.0, 1.0, 4.0, 1.0,
-5.0, 7.0, 2.0,1.0, 2.0};
int ja[26] = {0, 3, 4, 1, 2, 3, 5, 1, 2, 7, 0, 1, 3, 6, 0, 4, 5, 1, 4, 5, 7, 3, 6, 2, 5, 7};
int ia[9] = {0, 3, 7, 10, 14, 17, 21, 23, 26};

ierr = KmlIssCgInitDI(&handle, n, a, ja, ia);
ierr = KmlIssCgAnalyzeDI(&handle);
ierr = KmlIssCgFactorizeDI(&handle);

int nd = 1;
int idata[] = {0};
double ddata[] = {0};
double eps = 1e-4;
ierr = KmlIssCgSetDID(&handle, KMLSS_THRESHOLD, &eps, nd);
int max_iters = 2000;
ierr = KmlIssCgSetDII(&handle, KMLSS_MAX_ITERATION_COUNT, &max_iters, nd);

int ldx = n;
int ldb = n;
int nb = 1;
double b[8] = {4.0, 9.0, 7.0, 6.0, 9.0, 3.0, 2.0, 5.0};
double x[8];
ierr = KmlIssCgSolveDI(&handle, nb, x, ldx, b, ldb);
ierr = KmlIssCgCleanDI(&handle);
if (ierr != 0) {
    printf("'\nERROR in KmlIssCgCleanDI: %d", ierr);
    return 1;
    }
```

6.5.3.3.2 GCR

6.5.3.3.2.1 KmlIssGcrInit?I

初始化数据结构，并将用户提供的系数矩阵关联到求解系统中，初始化单机版的GCR方法时使用完整的CSR格式。

接口定义

C Interface：

 版权所有 © 华为技术有限公司

int KmlIssGcrInitSI(KmlSolverTask **handle, int n, const float *a, const int *ja, const int *ia);

int KmlIssGcrInitDI(KmlSolverTask **handle, int n, const double *a, const int *ja, const int *ia);

参数

参数名	类型	描述	输入/输出
handle	KmlSolverTask **	求解器句柄，handle不可为空指针。	输入/输出
n	int	n为矩阵A的维度。	输入
a	• 在KmlIssGcrInitSI中为float *。 • 在KmlIssGcrInitDI中为double *。	a为维度为非零元素的个数的数组，a[i]为第i个非零元的值。该数组的生命周期与handle相同。	输入
ja	int *	ja为维度为非零元素的个数的数组，ja[i]为第i个非零元的列号。该数组的生命周期与handle相同。	输入
ia	int *	ia为n+1维数组，ia[i]为第i行第一个非零元在所有非零元中的索引。该数组的生命周期与handle相同。	输入

返回值

返回值	类型	描述
KMLSS_NO_ERROR	int	正常执行。
KMLSS_NULL_ARGUMENT	int	{handle, a, ja, ia}中存在空参数。
KMLSS_BAD_N	int	参数n不是正整数。
KMLSS_NO_MEMORY	int	内存不足。

依赖

#include "kml_iss.h"

示例

C Interface：

```
KmlSolverTask *handle;
int ierr;
```

 版权所有 © 华为技术有限公司

```
int n = 8;

double a[26] = {1.0, 1.0, 2.0, 9.0, 2.0, 1.0, -3.0, 2.0, 3.0, 2.0, 1.0, 1.0, 9.0, -5.0, 2.0, 6.0, 1.0, -3.0, 1.0, 4.0, 1.0,
-5.0, 7.0, 2.0,1.0, 2.0};
int ja[26] = {0, 3, 4, 1, 2, 3, 5, 1, 2, 7, 0, 1, 3, 6, 0, 4, 5, 1, 4, 5, 7, 3, 6, 2, 5, 7};
int ia[9] = {0, 3, 7, 10, 14, 17, 21, 23, 26};

ierr = KmlIssGcrInitDI(&handle, n, a, ja, ia);
if (ierr != 0) {
   printf("\nERROR in KmlIssGcrInitDI: %d", ierr);
   return 1;
   }
```

6.5.3.3.2.2 KmlIssGcrSet?I?

设置迭代求解的相关参数。

接口定义

C Interface：

int KmlIssGcrSetSII(KmlSolverTask **handle, enum KML_SOLVER_PARAM param, const int *data, int nd);

int KmlIssGcrSetDII(KmlSolverTask **handle, enum KML_SOLVER_PARAM param, const int *data, int nd);

int KmlIssGcrSetSIS(KmlSolverTask **handle, enum KML_SOLVER_PARAM param, const float *data, int nd);

int KmlIssGcrSetDID(KmlSolverTask **handle, enum KML_SOLVER_PARAM param, const double *data, int nd);

参数

参数名	类型	描述	输入/输出
handle	KmlSolverTask **	求解器句柄，传入之前步骤的变量。	输入/输出
param	enum KML_SOLVER_PARAM	• KMLSS_THRESHOLD表示迭代残差阈值。 • KMLSS_MAX_ITERATION_COUNT表示最大迭代次数。 • KMLSS_RESTART_PARAM表示重启参数的大小。 • KMLSS_PRECONDITIONER_TYPE表示预条件子类型。 • KMLSS_VECTOR_NORM_TYPE表示迭代残差的计算范数。 • KMLSS_MATRIX_FORMAT表示矩阵稀疏存储格式。	输入

 版权所有 © 华为技术有限公司

参数名	类型	描述	输入/输出
data	• 在KmlIssGcrSetSII中为const int *。 • 在KmlIssGcrSetDII中为const int *。 • 在KmlIssGcrSetSIS中为const float *。 • 在KmlIssGcrSetDID中为const double *。	迭代求解相关参数数组（如最大迭代次数）。 当前可选的计算范数： • 1-NORM：KMLSS_L1 • 2-NORM：KMLSS_L2 当前可选的稀疏矩阵存储格式： • CSR格式：KMLSS_CSR • ELLPACK格式：KMLSS_ELL • COO格式：KMLSS_COO • ELLPACK_COO混合格式：KMLSS_SELLCOO 当前可选择的预条件子类型： • ILU预条件子：KMLSS_ILU • ICC预条件子：KMLSS_ICC • SOR预条件子：KMLSS_SOR • ILUT预条件子：KMLSS_ILUT	输入
nd	int	data数组大小。	输入

返回值

返回值	类型	描述
KMLSS_NO_ERROR	int	正常执行。
KMLSS_DATA_SIZE	int	参数nd不等于1。
KMLSS_NULL_ARGUMENT	int	{handle, data}中存在空参数。
KMLSS_BAD_SELECTOR	int	param为无效参数。

依赖

#include "kml_iss.h"

示例

C Interface：

```
KmlSolverTask *handle;
int ierr;
int n = 8;
```

 版权所有 © 华为技术有限公司

```
double a[26] = {1.0, 1.0, 2.0, 9.0, 2.0, 1.0, -3.0, 2.0, 3.0, 2.0, 1.0, 1.0, 9.0, -5.0, 2.0, 6.0, 1.0, -3.0, 1.0, 4.0, 1.0,
-5.0, 7.0, 2.0,1.0, 2.0};
int ja[26] = {0, 3, 4, 1, 2, 3, 5, 1, 2, 7, 0, 1, 3, 6, 0, 4, 5, 1, 4, 5, 7, 3, 6, 2, 5, 7};
int ia[9] = {0, 3, 7, 10, 14, 17, 21, 23, 26};

ierr = KmlIssGcrInitDI(&handle, n, a, ja, ia);

int nd = 1;
int idata[] = {0};
double ddata[] = {0};
double eps = 1e-4;
ierr = KmlIssGcrSetDID(&handle, KMLSS_THRESHOLD, &eps, nd);
if (ierr != 0) {
    printf("\nERROR in KmlIssGcrSetDID: %d", ierr);
    return 1;
    }
int max_iters = 2000;
ierr = KmlIssGcrSetDII(&handle, KMLSS_MAX_ITERATION_COUNT, &max_iters, nd);
if (ierr != 0) {
    printf("\nERROR in KmlIssGcrSetDII: %d", ierr);
    return 1;
    }
```

6.5.3.3.2.3 KmlIssGcrAnalyze?I

分析稀疏矩阵并改变存储形式。

接口定义

C Interface：

int KmlIssGcrAnalyzeSI(KmlSolverTask **handle);

int KmlIssGcrAnalyzeDI(KmlSolverTask **handle);

参数

参数名	类型	描述	输入/输出
handle	KmlSolverTask **	求解器句柄，传入之前步骤的变量。	输入/输出

返回值

返回值	类型	描述
KMLSS_NO_ERROR	int	正常执行。
KMLSS_NULL_ARGUMENT	int	handle为空参数。

依赖

#include "kml_iss.h"

 版权所有 © 华为技术有限公司

示例

C Interface：

```
KmlSolverTask *handle;
int ierr;
int n = 8;

double a[26] = {1.0, 1.0, 2.0, 9.0, 2.0, 1.0, -3.0, 2.0, 3.0, 2.0, 1.0, 1.0, 9.0, -5.0, 2.0, 6.0, 1.0, -3.0, 1.0, 4.0, 1.0,
-5.0, 7.0, 2.0,1.0, 2.0};
int ja[26] = {0, 3, 4, 1, 2, 3, 5, 1, 2, 7, 0, 1, 3, 6, 0, 4, 5, 1, 4, 5, 7, 3, 6, 2, 5, 7};
int ia[9] = {0, 3, 7, 10, 14, 17, 21, 23, 26};

ierr = KmlIssGcrInitDI(&handle, n, a, ja, ia);
ierr = KmlIssGcrAnalyzeDI(&handle);
if (ierr != 0) {
    printf("\nERROR in KmlIssGcrAnalyzeDI: %d", ierr);
    return 1;
    }
```

6.5.3.3.2.4 KmlIssGcrFactorize?I

计算预条件子。

接口定义

C Interface：

int KmlIssGcrFactorizeSI(KmlSolverTask **handle);

int KmlIssGcrFactorizeDI(KmlSolverTask **handle);

参数

参数名	类型	描述	输入/输出
handle	KmlSolverTask **	求解器句柄，传入之前步骤的变量。	输入/输出

返回值

返回值	类型	描述
KMLSS_NO_ERROR	int	正常执行。
KMLSS_NULL_ARGUMENT	int	handle为空参数。

依赖

#include "kml_iss.h"

 版权所有 © 华为技术有限公司

示例

C Interface：

```
KmlSolverTask *handle;
int ierr;
int n = 8;

double a[26] = {1.0, 1.0, 2.0, 9.0, 2.0, 1.0, -3.0, 2.0, 3.0, 2.0, 1.0, 1.0, 9.0, -5.0, 2.0, 6.0, 1.0, -3.0, 1.0, 4.0, 1.0,
-5.0, 7.0, 2.0,1.0, 2.0};
int ja[26] = {0, 3, 4, 1, 2, 3, 5, 1, 2, 7, 0, 1, 3, 6, 0, 4, 5, 1, 4, 5, 7, 3, 6, 2, 5, 7};
int ia[9] = {0, 3, 7, 10, 14, 17, 21, 23, 26};

ierr = KmlIssGcrInitDI(&handle, n, a, ja, ia);
ierr = KmlIssGcrAnalyzeDI(&handle);
ierr = KmlIssGcrFactorizeDI(&handle);
if (ierr != 0) {
    printf("\nERROR in KmlIssGcrFactorizeDI: %d", ierr);
    return 1;
    }
```

6.5.3.3.2.5 KmlIssGcrSolve?I

求解线性代数方程组。

接口定义

C Interface：

int KmlIssGcrSolveSI(KmlSolverTask **handle, int nb, float *x, int ldx, const float *b, int ldb);

int KmlIssGcrSolveDI(KmlSolverTask **handle, int nb, double *x, int ldx, const double *b, int ldb);

参数

参数名	类型	描述	输入/输出
handle	KmlSolverTask **	求解器句柄，传入之前步骤的变量。	输入/输出
nb	int	右端项矢量的个数，当前nb只能为1。	输入
x	• 在KmlIssGcrSolveSI中为float * • 在KmlIssGcrSolveDI中为double *	读取并存储求解结果，其元素个数为n。	输入/输出
ldx	int	x的leading dimension大小，当前参数未使用。	输入

版权所有 © 华为技术有限公司

参数名	类型	描述	输入/输出
b	• 在KmlIssGcrSolveSI中为float * • 在KmlIssGcrSolveDI中为double *	读取右端项，其元素个数为n。	输入
ldb	int	b的leading dimension大小，当前参数未使用。	输入

返回值

返回值	类型	描述
KMLSS_NO_ERROR	int	正常执行。
KMLSS_BAD_NB	int	参数nb不等于1。
KMLSS_NO_MEMORY	int	内存不足。
KMLSS_NULL_ARGUMENT	int	{handle, x, b}中存在空参数。

依赖

#include "kml_iss.h"

示例

C Interface：

```
KmlSolverTask *handle;
int ierr;
int n = 8;

double a[26] = {1.0, 1.0, 2.0, 9.0, 2.0, 1.0, -3.0, 2.0, 3.0, 2.0, 1.0, 1.0, 9.0, -5.0, 2.0, 6.0, 1.0, -3.0, 1.0, 4.0, 1.0,
-5.0, 7.0, 2.0,1.0, 2.0};
int ja[26] = {0, 3, 4, 1, 2, 3, 5, 1, 2, 7, 0, 1, 3, 6, 0, 4, 5, 1, 4, 5, 7, 3, 6, 2, 5, 7};
int ia[9] = {0, 3, 7, 10, 14, 17, 21, 23, 26};

ierr = KmlIssGcrInitDI(&handle, n, a, ja, ia);
ierr = KmlIssGcrAnalyzeDI(&handle);
ierr = KmlIssGcrFactorizeDI(&handle);

int nd = 1;
int idata[] = {0};
double ddata[] = {0};
double eps = 1e-4;
ierr = KmlIssGcrSetDID(&handle, KMLSS_THRESHOLD, &eps, nd);
int max_iters = 2000;
ierr = KmlIssGcrSetDII(&handle, KMLSS_MAX_ITERATION_COUNT, &max_iters, nd);

int ldx = n;
```

 版权所有 © 华为技术有限公司

```
int ldb = n;
int nb = 1;
double b[8] = {4.0, 9.0, 7.0, 6.0, 9.0, 3.0, 2.0, 5.0};
double x[8];
ierr = KmlIssGcrSolveDI(&handle, nb, x, ldx, b, ldb);
if (ierr != 0) {
    printf("\nERROR in KmlIssGcrSolveDI: %d", ierr);
    return 1;
    }
```

6.5.3.3.2.6 KmlIssGcrGet?I?

获得迭代求解相关参数。

接口定义

C Interface：

int KmlIssGcrGetSII(KmlSolverTask **handle, enum KML_SOLVER_PARAM param, int *data, int nd);

int KmlIssGcrGetDII(KmlSolverTask **handle, enum KML_SOLVER_PARAM param, int *data, int nd);

int KmlIssGcrGetSIS(KmlSolverTask **handle, enum KML_SOLVER_PARAM param, float *data, int nd);

int KmlIssGcrGetDID(KmlSolverTask **handle, enum KML_SOLVER_PARAM param, double *data, int nd);

参数

参数名	类型	描述	输入/输出
handle	KmlSolverTask **	求解器句柄，传入之前步骤的变量。	输入/输出
param	enum KML_SOLVER_PARAM	• KMLSS_ITERATION_COUNT表示迭代次数。 • KMLSS_TOLERANCE表示最终的迭代残差。	输入
data	• 在KmlIssGcrGetSII中为int *。 • 在KmlIssGcrGetDII中为int *。 • 在KmlIssGcrGetSIS中为float *。 • 在KmlIssGcrGetDID中为double *。	• 在KmlIssGcrGetSII中为迭代步数。 • 在KmlIssGcrGetDII中为迭代步数。 • 在KmlIssGcrGetSIS中为残差。 • 在KmlIssGcrGetDID中为残差。	输入
nd	int	data数组元素个数。	输入

版权所有 © 华为技术有限公司

返回值

返回值	类型	描述
KMLSS_NO_ERROR	int	正常执行。
KMLSS_DATA_SIZE	int	参数nd不等于1。
KMLSS_NULL_ARGUMENT	int	{handle, data}中存在空参数。
KMLSS_BAD_SELECTOR	int	param为无效参数。

依赖

#include "kml_iss.h"

示例

C Interface：

```
KmlSolverTask *handle;
int ierr;
int n = 8;

double a[26] = {1.0, 1.0, 2.0, 9.0, 2.0, 1.0, -3.0, 2.0, 3.0, 2.0, 1.0, 1.0, 9.0, -5.0, 2.0, 6.0, 1.0, -3.0, 1.0, 4.0, 1.0,
-5.0, 7.0, 2.0,1.0, 2.0};
int ja[26] = {0, 3, 4, 1, 2, 3, 5, 1, 2, 7, 0, 1, 3, 6, 0, 4, 5, 1, 4, 5, 7, 3, 6, 2, 5, 7};
int ia[9] = {0, 3, 7, 10, 14, 17, 21, 23, 26};

ierr = KmlIssGcrInitDI(&handle, n, a, ja, ia);
ierr = KmlIssGcrAnalyzeDI(&handle);
ierr = KmlIssGcrFactorizeDI(&handle);

int nd = 1;
int idata[] = {0};
double ddata[] = {0};
double eps = 1e-4;
ierr = KmlIssGcrSetDID(&handle, KMLSS_THRESHOLD, &eps, nd);
int max_iters = 2000;
ierr = KmlIssGcrSetDII(&handle, KMLSS_MAX_ITERATION_COUNT, &max_iters, nd);

int ldx = n;
int ldb = n;
int nb = 1;
double b[8] = {4.0, 9.0, 7.0, 6.0, 9.0, 3.0, 2.0, 5.0};
double x[8];
ierr = KmlIssGcrSolveDI(&handle, nb, x, ldx, b, ldb);
ierr = KmlIssGcrGetDII(&handle, KMLSS_ITERATION_COUNT, idata, nd);
if (ierr != 0) {
    printf("\nERROR in KmlIssGcrGetDII: %d", ierr);
    return 1;
    }
ierr = KmlIssGcrGetDID(&handle, KMLSS_TOLERANCE, ddata, nd);
if (ierr != 0) {
    printf("\nERROR in KmlIssGcrGetDID: %d", ierr);
    return 1;
    }
printf("[Solve info] Res= \t%.12g\t, iteration = \t%d\t\n", sqrt(ddata[0]), idata[0]);
```

运行结果

 版权所有 © 华为技术有限公司

```
[Solve info] Res=       5.43730434757e-17       , iteration =   8
```

6.5.3.3.2.7 KmlIssGcrClean?I

释放内部数据结构。

接口定义

C Interface：

int KmlIssGcrCleanSI(KmlSolverTask **handle);

int KmlIssGcrCleanDI(KmlSolverTask **handle);

参数

参数名	类型	描述	输入/输出
handle	KmlSolverTask **	求解器句柄，传入之前步骤的变量。	输入/输出

返回值

返回值	类型	描述
KMLSS_NO_ERROR	int	正常执行。
KMLSS_NULL_ARGUMENT	int	handle为空参数。

依赖

#include "kml_iss.h"

示例

C Interface：

```
KmlSolverTask *handle;
int ierr;
int n = 8;

double a[26] = {1.0, 1.0, 2.0, 9.0, 2.0, 1.0, -3.0, 2.0, 3.0, 2.0, 1.0, 1.0, 9.0, -5.0, 2.0, 6.0, 1.0, -3.0, 1.0, 4.0, 1.0,
-5.0, 7.0, 2.0,1.0, 2.0};
int ja[26] = {0, 3, 4, 1, 2, 3, 5, 1, 2, 7, 0, 1, 3, 6, 0, 4, 5, 1, 4, 5, 7, 3, 6, 2, 5, 7};
int ia[9] = {0, 3, 7, 10, 14, 17, 21, 23, 26};

ierr = KmlIssGcrInitDI(&handle, n, a, ja, ia);
ierr = KmlIssGcrAnalyzeDI(&handle);
ierr = KmlIssGcrFactorizeDI(&handle);

int nd = 1;
int idata[] = {0};
double ddata[] = {0};
```

 版权所有 © 华为技术有限公司

```
double eps = 1e-4;
ierr = KmlIssGcrSetDID(&handle, KMLSS_THRESHOLD, &eps, nd);
int max_iters = 2000;
ierr = KmlIssGcrSetDII(&handle, KMLSS_MAX_ITERATION_COUNT, &max_iters, nd);

int ldx = n;
int ldb = n;
int nb = 1;
double b[8] = {4.0, 9.0, 7.0, 6.0, 9.0, 3.0, 2.0, 5.0};
double x[8];
ierr = KmlIssGcrSolveDI(&handle, nb, x, ldx, b, ldb);
ierr = KmlIssGcrCleanDI(&handle);
if (ierr != 0) {
    printf("\nERROR in KmlIssGcrCleanDI: %d", ierr);
    return 1;
    }
```

6.5.3.3.3 GCR 算子

6.5.3.3.3.1 KmlIssGcrAxpyP

$$p_{j+1}=r_{j+1}+\sum_{i=0}^{j}\beta_{ij}p_i$$

计算GCR算法中的 。

接口定义

C Interface：

int KmlIssGcrAxpyP(float *r, float *p, const double *b, const int *m, const int *mt, const int *xl, const int *xr, const int *yl, const int *yr, const int *zl, const int *zr, const int *ybegin, const int *yend, const int *zbegin, const int *zend);

参数

参数名	类型	描述	输入/输出
r	float *	GCR算法残差向量r，按(z,x,y)方向存储。	输入/输出
p	float *	GCR算法中间变量p，按(z,x,y)方向存储。	输入/输出
b	const double *	GCR算法中间变量beta。	输入
m	const int *	GCR算法中间变量j。	输入
mt	const int *	GCR算法中间变量p存储位置，一般为j+1。	输入
xl	const int *	求解空间x方向起始位置。	输入
xr	const int *	求解空间x方向结束位置。	输入
yl	const int *	求解空间y方向起始位置。	输入

 版权所有 © 华为技术有限公司

参数名	类型	描述	输入/输出
yr	const int *	求解空间y方向结束位置。	输入
zl	const int *	求解空间z方向起始位置。	输入
zr	const int *	求解空间z方向结束位置。	输入
ybegin	const int *	辅助求解空间y方向起始位置。	输入
yend	const int *	辅助求解空间y方向结束位置。	输入
zbegin	const int *	辅助求解空间z方向起始位置。	输入
zend	const int *	辅助求解空间z方向结束位置。	输入

返回值

返回值	类型	描述
KMLSS_NO_ERROR	int	正常执行。
KMLSS_NULL_ARGUMENT	int	输入中存在空指针。
KMLSS_BAD_DATA	int	求解空间相关参数为无效参数。

依赖

#include "kml_gcr_specific.h"

示例

C Interface：

```
int xl = 5;
int yl = 5;
int zl = 5;
int xr = 6;
int yr = 6;
int zr = 6;
int ybegin = yl;
int yend = yr;
int zbegin = zl;
int zend = zr;
int m = 1;
int mt = 2;
int n = 64;
float *r = (float*)malloc(sizeof(float) * 64);
float *p = (float*)malloc(sizeof(float) * 256);
double *b = (double*)malloc(sizeof(double) * 2);
for (int i=0; i<64; i++) {
    r[i] = rand();
}
for (int i=0; i<256; i++) {
    p[i] = rand();
}
```

 版权所有 © 华为技术有限公司

```
for (int i=0; i<2; i++) {
    b[i] = 1.0;
}

KmlIssGcrAxpyP(r, p, b, &m, &mt, &xl, &xr, &yl, &yr, &zl, &zr, &ybegin, &yend, &zbegin, &zend);
```

6.5.3.3.3.2 KmlIssGcrAxpyAp

$$Ap_{j+1}=Ar_{j+1}+\sum_{i=0}^{j}\beta_{ij}Ap_i$$

计算GCR算法中的 。

接口定义

C Interface：

int KmlIssGcrAxpyAp(float *ar, float *ap, const double *b, const int *m, const int *mt, const int *xl, const int *xr, const int *yl, const int *yr, const int *zl, const int *zr, const int *ybegin, const int *yend, const int *zbegin, const int *zend);

参数

参数名	类型	描述	输入/输出
ar	float *	GCR算法中间变量ar，按(z,x,y)方向存储。	输入/输出
ap	float *	GCR算法中间变量ap，按(z,x,y)方向存储。	输入/输出
b	const double *	GCR算法中间变量beta。	输入
m	const int *	GCR算法中间变量j。	输入
mt	const int *	GCR算法中间变量ap存储位置，一般为j+1。	输入
xl	const int *	求解空间x方向起始位置。	输入
xr	const int *	求解空间x方向结束位置。	输入
yl	const int *	求解空间y方向起始位置。	输入
yr	const int *	求解空间y方向结束位置。	输入
zl	const int *	求解空间z方向起始位置。	输入
zr	const int *	求解空间z方向结束位置。	输入
ybegin	const int *	辅助求解空间y方向起始位置。	输入
yend	const int *	辅助求解空间y方向结束位置。	输入
zbegin	const int *	辅助求解空间z方向起始位置。	输入
zend	const int *	辅助求解空间z方向结束位置。	输入

 版权所有 © 华为技术有限公司

返回值

返回值	类型	描述
KMLSS_NO_ERROR	int	正常执行。
KMLSS_NULL_ARGUMENT	int	输入中存在空指针。
KMLSS_BAD_DATA	int	求解空间相关参数为无效参数。

依赖

#include "kml_gcr_specific.h"

示例

C Interface：

```
int xl = 5;
int yl = 5;
int zl = 5;
int xr = 6;
int yr = 6;
int zr = 6;
int ybegin = yl;
int yend = yr;
int zbegin = zl;
int zend = zr;
int m = 1;
int mt = 2;
float *ar = (float*)malloc(sizeof(float) * 9);
float *ap = (float*)malloc(sizeof(float) * 36);
double *b = (double*)malloc(sizeof(double) * 2);
for (int i=0; i<9; i++) {
    ar[i] = rand();
}
for (int i=0; i<36; i++) {
    ap[i] = rand();
}
for (int i=0; i<2; i++) {
    b[i] = 1.0;
}

KmlIssGcrAxpyAp(ar, ap, b, &m, &mt, &xl, &xr, &yl, &yr, &zl, &zr, &ybegin, &yend, &zbegin, &zend);
```

6.5.3.3.3.3 KmlIssGcrAxpyX

计算GCR算法中的 $x_{j+1}=x_j+\alpha_j p_j$ 。

接口定义

C Interface：

 版权所有 © 华为技术有限公司

int KmlIssGcrAxpyX(float *x, const float *p, const double *ac, const int *xl, const int *xr, const int *yl, const int *yr, const int *zl, const int *ylT, const int *yrT, const int *zlT, const int *ybegin, const int *yend, const int *zbegin, const int *zend);

参数

参数名	类型	描述	输入/输出
x	float *	GCR算法中间变量x，按(z,x,y)方向存储。	输入/输出
p	float *	GCR算法中间变量p，按(z,x,y)方向存储。	输入
ac	const double *	GCR算法中间变量alpha。	输入
xl	const int *	求解空间x方向起始位置。	输入
xr	const int *	求解空间x方向结束位置。	输入
yl	const int *	求解空间y方向起始位置。	输入
yr	const int *	求解空间y方向结束位置。	输入
zl	const int *	求解空间z方向起始位置。	输入
ylT	const int *	求解空间yT方向起始位置。	输入
yrT	const int *	求解空间yT方向结束位置。	输入
zlT	const int *	求解空间zT方向起始位置。	输入
ybegin	const int *	辅助求解空间y方向起始位置。	输入
yend	const int *	辅助求解空间y方向结束位置。	输入
zbegin	const int *	辅助求解空间z方向起始位置。	输入
zend	const int *	辅助求解空间z方向结束位置。	输入

返回值

返回值	类型	描述
KMLSS_NO_ERROR	int	正常执行。
KMLSS_NULL_ARGUMENT	int	输入中存在空指针。
KMLSS_BAD_DATA	int	求解空间相关参数为无效参数。

依赖

#include "kml_gcr_specific.h"

 版权所有 © 华为技术有限公司

示例

C Interface：

```
int xl = 5;
int yl = 5;
int zl = 5;
int xr = 6;
int yr = 6;
int zr = 6;
int ybegin = 5;
int yend = 6;
int zbegin = 5;
int zend = 6;
int ylm = 4;
int yrm = 7;
int zlm = 4;
float *x = (float*)malloc(sizeof(float) * 9);
float *p = (float*)malloc(sizeof(float) * 36);
double ac = rand();
for (int i=0; i<9; i++) {
    x[i] = rand();
}
for (int i=0; i<36; i++) {
    p[i] = rand();
}

KmlIssGcrAxpyX(x, p, &ac, &xl, &xr, &ylm, &yrm, &zlm, &yl, &yr, &zl, &ybegin, &yend, &zbegin, &zend);
```

6.5.3.3.3.4 KmlIssGcrAxpyR

计算GCR算法中的 $r_{j+1}=r_j-\alpha_j Ap_j$ 。

接口定义

C Interface：

int KmlIssGcrAxpyR(float *r, const float *ap, double *d, const double *ac, const int *xl, const int *xr, const int *yl, const int *zl, const int *ylT, const int *yrT, const int *zlT, const int *ybegin, const int *yend, const int *zbegin, const int *zend);

参数

参数名	类型	描述	输入/输出
r	float *	GCR算法中间变量r，按(z,x,y)方向存储。	输入/输出
ap	float *	GCR算法中间变量ap，按(z,x,y)方向存储。	输入
d	double *	GCR算法中间变量d。	输入/输出
ac	const double *	GCR算法中间变量alpha。	输入
xl	const int *	求解空间x方向起始位置。	输入
xr	const int *	求解空间x方向结束位置。	输入

版权所有 © 华为技术有限公司

参数名	类型	描述	输入/输出
yl	const int *	求解空间y方向起始位置。	输入
zl	const int *	求解空间z方向起始位置。	输入
ylT	const int *	求解空间yT方向起始位置。	输入
yrT	const int *	求解空间yT方向结束位置。	输入
zlT	const int *	求解空间zT方向起始位置。	输入
ybegin	const int *	辅助求解空间y方向起始位置。	输入
yend	const int *	辅助求解空间y方向结束位置。	输入
zbegin	const int *	辅助求解空间z方向起始位置。	输入
zend	const int *	辅助求解空间z方向结束位置。	输入

返回值

返回值	类型	描述
KMLSS_NO_ERROR	int	正常执行。
KMLSS_NULL_ARGUMENT	int	输入中存在空指针。
KMLSS_BAD_DATA	int	求解空间相关参数为无效参数。

依赖

#include "kml_gcr_specific.h"

示例

C Interface：

```
int xl = 5;
int yl = 5;
int zl = 5;
int xr = 6;
int yr = 6;
int zr = 6;
int ybegin = 5;
int yend = 6;
int zbegin = 5;
int zend = 6;
int ylm = 4;
int yrm = 7;
int zlm = 4;
float *r = (float*)malloc(sizeof(float) * 45);
float *ap = (float*)malloc(sizeof(float) * 45);
double ac = rand();
double d = 0.0;
for (int i=0; i<45; i++) {
```

 版权所有 © 华为技术有限公司

```
    r[i] = rand();
    ap[i] = rand();
}

KmlIssGcrAxpyR(r, ap, &d, &ac, &xl, &xr, &ylm, &zlm, &yl, &yr, &zl, &ybegin, &yend, &zbegin, &zend);
```

6.5.3.3.3.5 KmlIssGcrDotprodAlpha

计算GCR算法中的 $\alpha_j=(r_j, Ap_j)/(Ap_j, Ap_j)$ 。

接口定义

C Interface：

int KmlIssGcrDotprodAlpha(const float *r, const float *ap, double *c, const int *xl, const int *xr, const int *yl, const int *yr, const int *zl, const int *ybegin, const int *yend, const int *zbegin, const int *zend);

参数

参数名	类型	描述	输入/输出
r	float *	GCR算法中间变量r，按(z,x,y)方向存储。	输入
ap	float *	GCR算法中间变量ap，按(z,x,y)方向存储。	输入
c	double *	计算结果，c[0]为分子，c[1]为分母。	输入/输出
xl	const int *	求解空间x方向起始位置。	输入
xr	const int *	求解空间x方向结束位置。	输入
yl	const int *	求解空间y方向起始位置。	输入
zl	const int *	求解空间z方向起始位置。	输入
ybegin	const int *	辅助求解空间y方向起始位置。	输入
yend	const int *	辅助求解空间y方向结束位置。	输入
zbegin	const int *	辅助求解空间z方向起始位置。	输入
zend	const int *	辅助求解空间z方向结束位置。	输入

返回值

返回值	类型	描述
KMLSS_NO_ERROR	int	正常执行。

 版权所有 © 华为技术有限公司

返回值	类型	描述
KMLSS_NULL_ARGUMENT	int	输入中存在空指针。
KMLSS_BAD_DATA	int	求解空间相关参数为无效参数。

依赖

#include "kml_gcr_specific.h"

示例

C Interface：

```
int xl = 5;
int yl = 5;
int zl = 5;
int xr = 6;
int yr = 6;
int zr = 6;
int ybegin = 5;
int yend = 6;
int zbegin = 5;
int zend = 6;
float *r = (float*)malloc(sizeof(float) * 9);
float *ap = (float*)malloc(sizeof(float) * 36);
double *c=(double*)malloc(sizeof(double) * 2);
for (int i=0; i<9; i++) {
    r[i] = rand();
}
for (int i=0; i<36; i++) {
    ap[i] = rand();
}
for (int i=0; i<2; i++) {
    c[i] = 0.0;
}

KmlIssGcrDotprodAlpha(r, ap, c, &xl, &xr, &yl, &yr, &zl, &ybegin, &yend, &zbegin, &zend);
```

6.5.3.3.3.6 KmlIssGcrDotprodBeta

计算GCR算法中的

$$\beta_{ij}=-(Ar_{j+1}, Ap_i)/(Ap_i, Ap_i), for\ i=0,1,...,j$$

。

接口定义

C Interface：

int KmlIssGcrDotprodBeta(const float *ar, const float *ap, double *c, const int *m, const int *xl, const int *xr, const int *yl, const int *yr, const int *zl, const int *zr, const int *ybegin, const int *yend, const int *zbegin, const int *zend);

版权所有 © 华为技术有限公司

参数

参数名	类型	描述	输入/输出
ar	const float *	GCR算法中间变量ar，按(z,x,y)方向存储。	输入
ap	const float *	GCR算法中间变量ap，按(z,x,y)方向存储。	输入
c	double *	计算结果beta。	输入/输出
m	const int *	GCR算法中间变量j。	输入
xl	const int *	求解空间x方向起始位置。	输入
xr	const int *	求解空间x方向结束位置。	输入
yl	const int *	求解空间y方向起始位置。	输入
yr	const int *	求解空间y方向结束位置。	输入
zl	const int *	求解空间z方向起始位置。	输入
zr	const int *	求解空间z方向结束位置。	输入
ybegin	const int *	辅助求解空间y方向起始位置。	输入
yend	const int *	辅助求解空间y方向结束位置。	输入
zbegin	const int *	辅助求解空间z方向起始位置。	输入
zend	const int *	辅助求解空间z方向结束位置。	输入

依赖

#include "kml_gcr_specific.h"

返回值

返回值	类型	描述
KMLSS_NO_ERROR	int	正常执行。
KMLSS_NULL_ARGUMENT	int	输入中存在空指针。
KMLSS_BAD_DATA	int	求解空间相关参数为无效参数。

示例

C Interface：

 版权所有 © 华为技术有限公司

```
int xl = 5;
int yl = 5;
int zl = 5;
int xr = 6;
int yr = 6;
int zr = 6;
int ybegin = 5;
int yend = 6;
int zbegin = 5;
int zend = 6;
int m = 1;
float *r = (float*)malloc(sizeof(float) * 27);
float *ap = (float*)malloc(sizeof(float) * 108);
double *c=(double*)malloc(sizeof(double) * 4);
for (int i=0; i<27; i++) {
    r[i] = rand();
}
for (int i=0; i<108; i++) {
    ap[i] = rand();
}
for (int i=0; i<4; i++) {
    c[i] = 0.0;
}

KmlIssGcrDotprodBeta(r, ap, c, &m, &xl, &xr, &yl, &yr, &zl, &zr, &ybegin, &yend, &zbegin, &zend);
```

6.5.3.3.3.7 KmlIssGcrSpmv

计算GCR算法中的稀疏矩阵向量乘。

接口定义

C Interface：

int KmlIssGcrSpmv(const float *a, const float *x, float *b, const int *xl, const int *xr, const int *yl, const int *yr, const int *zl, const int *ybegin, const int *yend, const int *zbegin, const int *zend);

参数

参数名	类型	描述	输入/输出
a	const float *	矩阵a，按(z,x,y)方向存储。	输入
x	const float *	向量x，按(z,x,y)方向存储。	输入
b	double *	计算结果b。	输入/输出
xl	const int *	求解空间x方向起始位置。	输入
xr	const int *	求解空间x方向结束位置。	输入
yl	const int *	求解空间y方向起始位置。	输入
yr	const int *	求解空间y方向结束位置。	输入
zl	const int *	求解空间z方向起始位置。	输入
ybegin	const int *	辅助求解空间y方向起始位置。	输入

版权所有 © 华为技术有限公司

参数名	类型	描述	输入/输出
yend	const int *	辅助求解空间y方向结束位置。	输入
zbegin	const int *	辅助求解空间z方向起始位置。	输入
zend	const int *	辅助求解空间z方向结束位置。	输入

返回值

返回值	类型	描述
KMLSS_NO_ERROR	int	正常执行。
KMLSS_NULL_ARGUMENT	int	输入中存在空指针。
KMLSS_BAD_DATA	int	求解空间相关参数为无效参数。

依赖

#include "kml_gcr_specific.h"

示例

C Interface：

```
int xl = 5;
int yl = 5;
int zl = 5;
int xr = 6;
int yr = 6;
int zr = 6;
int ybegin = 5;
int yend = 6;
int zbegin = 5;
int zend = 6;
float *a = (float*)malloc(sizeof(float) * 45);
float *x = (float*)malloc(sizeof(float) * 3);
float *b = (float*)malloc(sizeof(float) * 45);
for (int i=0; i<45; i++) {
    a[i] = rand();
    b[i] = rand();
}
for (int i=0; i<3; i++) {
    x[i] = rand();
}

KmlIssGcrSpmv(a, x, b, &xl, &xr, &yl, &yr, &zl, &ybegin, &yend, &zbegin, &zend);
```

6.5.3.3.4 KmlIssGetErrorText

获得用户可读的最后错误描述，但该功能还未实现，返回的text为空。

版权所有 © 华为技术有限公司

接口定义

C Interface：

int KmlIssGetErrorText(KmlSolverTask **handle, char *text, int textSize);

参数

参数名	类型	描述	输入/输出
handle	KmlSolverTask **	求解器句柄，传入之前步骤的变量。	输入/输出
text	char *	存储错误文本信息。	输入/输出
textSize	int	文本大小。	输入

返回值

返回值	类型	描述
NO_ERROR	int	正常执行。
NONZERO_INDEXING	int	非零索引。
NO_DIAGONAL_ELEMENT	int	无对角元素。
ZERO_DIAGONAL_ELEMENT	int	对角线元素为零。
NO_MEMORY	int	内存不足。
BAD_HANDLE	int	handle错误。
OUT_OF_EXECUTION_ORDER	int	错误的执行顺序。
INCONSISTENT_PERMUTATION	int	不一致的排列。
KML_SOLVER_INVALID_ARGUMENT	int	无效的参数。
NOT_IMPLEMENTED	int	未执行。

依赖

#include "kml_iss.h"

 版权所有 © 华为技术有限公司

示例

C Interface：

```
KmlSolverTask *handle
int n = 8;
double a[26] = {1.0, 1.0, 2.0, 9.0, 2.0, 1.0, -3.0, 2.0, 3.0, 2.0, 1.0, 1.0, 9.0, -5.0, 2.0, 6.0, 1.0, -3.0, 1.0, 4.0,
1.0, -5.0, 7.0, 2.0,\
                1.0, 2.0};
int ja[26] = {0, 3, 4, 1, 2, 3, 5, 1, 2, 7, 0, 1, 3, 6, 0, 4, 5, 1, 4, 5, 7, 3, 6, 2, 5, 7};
int ia[9] = {0, 3, 7, 10, 14, 17, 21, 23, 26};

char *text;
int textSize;
int ierr;
ierr = KmlIssGcrInitDI(&handle, n, a, ja, ia);
ierr = KmlIssGetError(&handle,text,textSize);
```

6.5.3.3.5 KmlIssClearError

清除最后的错误码，运行后错误码清零。

接口定义

C Interface：

int KmlIssClearError(KmlSolverTask **handle);

参数

参数名	类型	描述	输入/输出
handle	KmlSolverTask **	求解器句柄，传入之前步骤的变量。	输入/输出

返回值

返回值	类型	描述
NO_ERROR	int	正常执行。
NONZERO_INDEXING	int	非零索引。
NO_DIAGONAL_ELEMENT	int	无对角元素。
ZERO_DIAGONAL_ELEMENT	int	对角线元素为零。
NO_MEMORY	int	内存不足。
BAD_HANDLE	int	handle错误。
OUT_OF_EXECUTION_ORDER	int	错误的执行顺序。

 版权所有 © 华为技术有限公司

返回值	类型	描述
INCONSISTENT_PERMUTATION	int	不一致的排列。
KML_SOLVER_INVALID_ARGUMENT	int	无效的参数。
NOT_IMPLEMENTED	int	未执行。

依赖

#include "kml_iss.h"

示例

C Interface：

```
   KmlSolverTask *handle;
   int n = 8;
   double a[26] = {1.0, 1.0, 2.0, 9.0, 2.0, 1.0, -3.0, 2.0, 3.0, 2.0, 1.0, 1.0, 9.0, -5.0, 2.0, 6.0, 1.0, -3.0, 1.0, 4.0,
1.0, -5.0, 7.0, 2.0,\
                   1.0, 2.0};
   int ja[26] = {0, 3, 4, 1, 2, 3, 5, 1, 2, 7, 0, 1, 3, 6, 0, 4, 5, 1, 4, 5, 7, 3, 6, 2, 5, 7};
   int ia[9] = {0, 3, 7, 10, 14, 17, 21, 23, 26};
   int ierr;
   ierr = KmlIssGCRInitDI(&handle, n, a, ja, ia);
   ierr = KmlIssClearError(&handle);
```

6.5.3.3.6 KSOLVERGetVersion

获得求解器产品版本信息。

接口定义

C Interface：

int KSOLVERGetVersion(KSOLVERVersion *ver);

参数

参数名	类型	描述	输入/输出
ver	KSOLVERVersion结构体指针	存储版本信息，非空。	输入/输出

依赖

#include "kml_solver.h"

#include "kml_service.h"

 版权所有 © 华为技术有限公司

示例

C Interface：

```
int main()
{
    KSOLVERVersion* ver = (KSOLVERVersion*)malloc(sizeof(KSOLVERVersion));
    int status = KSOLVERGetVersion(ver);
    printf("Product Name: %s\nProduct Version: %s\nComponent Name: %s\nComponent Version: %s
\nComponent AppendInfo: %s\nSoftware Name: %s\nSoftware Version: %s\n",
            ver->productName, ver->productVersion,
            ver->componentName, ver->componentVersion, ver->componentAppendInfo,
            ver->softwareName, ver->softwareVersion);
    return 0;
}
```

运行结果：

```
Product Name: Kunpeng HPCKit
Product Version: 24.0.RC1
Component Name: HPCKit-kml
Component Version: 2.3.0
Component AppendInfo: gcc
Software Name: hpckit-kml-solver
Software Version: 2.3.0
```

说明

版本号以实际运行结果为准，上述结果仅供参考。

6.5.3.3.7 PRECOND

6.5.3.3.7.1 KmlIssPcStructilu0Setup

分析并分解矩阵，计算ilu0预条件；创建并返回求解器句柄。

接口定义

C Interface：

int KmlIssPcStructilu0Setup(KmlIssPsStructilu0Handle *gSmoother, float *v, const int *xl, const int *xr, const int *yl, const int *yr, const int *zl, const int *zr);

参数

参数名	类型	描述	输入/输出
gSmoother	KmlIssPsStructilu0Handle *	求解器句柄。	输出
v	float *	待求解的19对角矩阵v，使用(l,k,i,j)格式数组存储；其中，l=1:19为对角线元素，k，i，j为空间位置。	输入
xl	const int *	求解空间x方向起始位置。	输入
xr	const int *	求解空间x方向结束位置。	输入
yl	const int *	求解空间y方向起始位置。	输入

版权所有 © 华为技术有限公司

参数名	类型	描述	输入/输出
yr	const int *	求解空间y方向结束位置。	输入
zl	const int *	求解空间z方向起始位置。	输入
zr	const int *	求解空间z方向结束位置。	输入

返回值

返回值	类型	描述
KMLSS_NO_ERROR	int	正常执行。
KMLSS_NULL_ARGUMENT	int	输入中存在空指针。
KMLSS_BAD_DATA	int	求解空间相关参数为无效参数。

依赖

#include "kml_struct_ilu.h"

示例

C Interface：

```
const int xl = 5;
const int yl = 5;
const int zl = 5;
const int xr = 6;
const int yr = 6;
const int zr = 6;
int ybegin = 5;
int yend = 6;
int zbegin = 5;
int zend = 6;
float *v = (float*)malloc(sizeof(float) * 171);
float *b = (float*)malloc(sizeof(float) * 9);
for (int i=0; i<171; i++) {
    v[i] = rand();
}
for (int i=0; i<9; i++) {
    b[i] = rand();
}
KmlIssPsStructilu0Handle gSmoother;
KmlIssPcStructilu0Setup(&gSmoother, v, &xl, &xr, &yl, &yr, &zl, &zr);
```

6.5.3.3.7.2 KmlIssPcStructilu0Apply

ilu0预条件回代，计算 $r_i^{'}=M^{-1}r_i$

 版权所有 © 华为技术有限公司

接口定义

C Interface：

int KmlIssPcStructilu0Apply(KmlIssPsStructilu0Handle gSmoother, float *bb, float *xx);

参数

参数名	类型	描述	输入/输出
gSmoother	KmlIssPsStructilu0Handle	求解器句柄，handle不可为空指针。	输入/输出
bb	float *	输入向量r。	输入
xx	float *	输出向量r'。	输入/输出

返回值

返回值	类型	描述
KMLSS_NO_ERROR	int	正常执行。
KMLSS_NULL_ARGUMENT	int	输入中存在空指针。

依赖

#include "kml_struct_ilu.h"

示例

C Interface：

```
int xl = 5;
int yl = 5;
int zl = 5;
int xr = 6;
int yr = 6;
int zr = 6;
int ybegin = 5;
int yend = 6;
int zbegin = 5;
int zend = 6;
float *v = (float*)malloc(sizeof(float) * 171);
float *b = (float*)malloc(sizeof(float) * 9);
float *x = (float*)malloc(sizeof(float) * 9);
for (int i=0; i<171; i++) {
    v[i] = rand();
}
for (int i=0; i<9; i++) {
    b[i] = rand();
    x[i] = rand();
}
```


版权所有 © 华为技术有限公司

```
KmlIssPsStructilu0Handle gSmoother;
KmlIssPcStructilu0Setup(&gSmoother, v, &xl, &xr, &yl, &yr, &zl, &zr);
KmlIssPcStructilu0Apply(gSmoother, b, x);
```

6.5.3.3.7.3 KmlIssPcStructilu0Clean

释放内部数据结构。

接口定义

C Interface：

int KmlIssPcStructilu0Clean(KmlIssPsStructilu0Handle gSmoother);

参数

参数名	类型	描述	输入/输出
gSmoother	KmlIssPsStructilu0Handle	求解器句柄，handle不可为空指针。	输入/输出

返回值

返回值	类型	描述
KMLSS_NO_ERROR	int	正常执行。

依赖

#include "kml_struct_ilu.h"

示例

C Interface：

```
int xl = 5;
int yl = 5;
int zl = 5;
int xr = 6;
int yr = 6;
int zr = 6;
int ybegin = 5;
int yend = 6;
int zbegin = 5;
int zend = 6;
float *v = (float*)malloc(sizeof(float) * 171);
float *b = (float*)malloc(sizeof(float) * 9);
float *x = (float*)malloc(sizeof(float) * 9);
for (int i=0; i<171; i++) {
    v[i] = rand();
}
for (int i=0; i<9; i++) {
    b[i] = rand();
    x[i] = rand();
}
KmlIssPsStructilu0Handle gSmoother;
```


版权所有 © 华为技术有限公司

```
KmlIssPcStructilu0Setup(&gSmoother, v, &xl, &xr, &yl, &yr, &zl, &zr);
KmlIssPcStructilu0Apply(gSmoother, b, x);
KmlIssPcStructilu0Clean(gSmoother);
```

6.5.3.3.8 LDLT (DEPRECATED)

直接法推荐使用**DSS**。

6.5.3.3.8.1 KmlDssSymInit?I

初始化数据结构，并将用户提供的系数矩阵关联到求解系统中，初始化单机版的LDLT方法时使用上三角的CSR格式。

接口定义

C Interface：

int KmlDssSymInitSI(KmlSolverTask **pHandle, int n, float *a, int *ja, int *ia)

int KmlDssSymInitDI(KmlSolverTask **pHandle, int n, double *a, int *ja, int *ia)

int KmlDssSymInitCI(KmlSolverTask **pHandle, int n, kml_complex_float *a, int *ja, int *ia)

int KmlDssSymInitZI(KmlSolverTask **pHandle, int n, kml_complex_double *a, int *ja, int *ia)

参数

参数名	类型	描述	输入/输出
pHandle	KmlSolverTask **	求解器句柄，pHandle不可为空指针。	输出
n	int	n为矩阵A的维度。	输入
a	• 在KmlDssSymInitSI中为float *。 • 在KmlDssSymInitDI中为double *。 • 在KmlDssSymInitCI中为kml_complex_float *。 • 在KmlDssSymInitZI中为kml_complex_double *。	a为维度为非零元素个数的数组，a[i]为第i个非零元的值。该数组的生命周期与pHandle相同。	输入
ja	int	ja为维度为非零元素个数的数组，ja[i]为第i个非零元的列号。该数组的生命周期与pHandle相同。	输入

版权所有 © 华为技术有限公司

参数名	类型	描述	输入/输出
ia	int	ia为n+1维数组，ia[i]为第i行第一个非零元在所有非零元中的索引。该数组的生命周期与pHandle相同。	输入

返回值

返回值	类型	描述
KMLSS_NO_ERROR	int	正常执行。
KMLSS_NULL_ARGUMENT	int	pHandle，a，ja，ia中有空指针。
KMLSS_BAD_N	int	矩阵维数输入值小于等于0。
KMLSS_NO_MEMORY	int	内存开辟失败。

依赖

#include "kml_dss.h"或#include "kml_solver.h"

示例

C Interface：

```
int n = 8;
int ia[9] = {0, 2, 4, 6, 7, 8, 9, 10, 11};
int ja[11] = {0, 7, 1, 6, 2, 5, 3, 4, 5, 6, 7};
double a[11] = {1.0, 2.0, -2.0, 3.0, 3.0, 4.0, -4.0, 5.0, -6.0, 7.0, 8.0};

KmlSolverTask *handle;
int ierr = KmlDssSymInitDI(&handle, n, a, ja, ia);
if (ierr != 0) {
	printf("\nERROR in KmlDssSymInitDI: %d", ierr);
}
```

6.5.3.3.8.2 KmlDssSymAnalyze?I

分析数据结构，对稀疏矩阵进行行/列重排和符号分解。

接口定义

C Interface：

int KmlDssSymAnalyzeSI(KmlSolverTask **pHandle);

int KmlDssSymAnalyzeDI(KmlSolverTask **pHandle);

int KmlDssSymAnalyzeCI(KmlSolverTask **pHandle);

int KmlDssSymAnalyzeZI(KmlSolverTask **pHandle);

 版权所有 © 华为技术有限公司

参数

参数名	类型	描述	输入/输出
pHandle	KmlSolverTask **	求解器句柄，传入之前步骤的变量。	输入

返回值

返回值	类型	描述
KMLSS_NO_ERROR	int	正常执行。
KMLSS_NULL_ARGUMENT	int	pHandle为空指针。
KMLSS_BAD_HANDLE	int	pHandle格式存在问题。
KMLSS_NONZERO_INDEXING	int	行索引指针不是从0开始，即ia[0]不等于0。
KMLSS_MISSING_DIAGONAL_ELEMENT	int	稀疏矩阵对角线元素全部为空。
KMLSS_INTERNAL_ERROR	int	稀疏矩阵对角线存在空元素或算法内部执行失败。
KMLSS_REORDERING_PROBLEM	int	排序算法失败。

依赖

#include "kml_dss.h"或#include "kml_solver.h"

示例

C Interface：

```
int n = 8;
int ia[9] = {0, 2, 4, 6, 7, 8, 9, 10, 11};
int ja[11] = {0, 7, 1, 6, 2, 5, 3, 4, 5, 6, 7};
double a[11] = {1.0, 2.0, -2.0, 3.0, 3.0, 4.0, -4.0, 5.0, -6.0, 7.0, 8.0};

KmlSolverTask *handle;
int ierr = KmlDssSymInitDI(&handle, n, a, ja, ia);
if (ierr != 0) {
	printf("\nERROR in KmlDssSymInitDI: %d", ierr);
}
ierr = KmlDssSymAnalyzeDI(&handle);
if (ierr != 0) {
	printf("\nERROR in KmlDssSymAnalyzeDI: %d", ierr);
}
```

6.5.3.3.8.3 KmlDssSymFactorize?I

实现对称不定矩阵的LDLT分解。

 版权所有 © 华为技术有限公司

接口定义

C Interface：

int KmlDssSymFactorizeSI(KmlSolverTask **pHandle);

int KmlDssSymFactorizeDI(KmlSolverTask **pHandle);

int KmlDssSymFactorizeCI(KmlSolverTask **pHandle);

int KmlDssSymFactorizeZI(KmlSolverTask **pHandle);

参数

参数名	类型	描述	输入/输出
pHandle	KmlSolverTask **	求解器句柄，传入之前步骤的变量。	输入

返回值

返回值	类型	描述
KMLSS_NO_ERROR	int	正常执行。
KMLSS_NULL_ARGUMENT	int	pHandle为空指针。
KMLSS_BAD_HANDLE	int	参数无效或格式存在问题。
KMLSS_INVALID_CALL_ORDER	int	错误的调用顺序，之前未进入分解阶段。
KMLSS_NO_MEMORY	int	内存分配失败。
KMLSS_INTERNAL_ERROR	int	算法内部执行失败。

依赖

#include "kml_dss.h"或#include "kml_solver.h"

示例

C Interface：

```
int n = 8;
int ia[9] = {0, 2, 4, 6, 7, 8, 9, 10, 11};
int ja[11] = {0, 7, 1, 6, 2, 5, 3, 4, 5, 6, 7};
double a[11] = {1.0, 2.0, -2.0, 3.0, 3.0, 4.0, -4.0, 5.0, -6.0, 7.0, 8.0};

KmlSolverTask *handle;
```

 版权所有 © 华为技术有限公司

```
int ierr = KmlDssSymInitDI(&handle, n, a, ja, ia);
if (ierr != 0) {
    printf("\nERROR in KmlDssSymInitDI: %d", ierr);
}
ierr = KmlDssSymAnalyzeDI(&handle);
if (ierr != 0) {
    printf("\nERROR in KmlDssSymAnalyzeDI: %d", ierr);
}
ierr = KmlDssSymFactorizeDI(&handle);
if (ierr != 0) {
    printf("\nERROR in KmlDssSymFactorizeDI: %d", ierr);
}
for (int i = 0; i < n; i++) {
    printf("%lf ", x[i]);
}
```

6.5.3.3.8.4 KmlDssSymSolve?I

求解$LDL^{T}x=b$。

接口定义

C Interface：

int KmlDssSymSolveSI(KmlSolverTask **pHandle, int nb, float *x, int ldx, float *b, int ldb);

int KmlDssSymSolveDI(KmlSolverTask **pHandle, int nb, double *x, int ldx, double *b, int ldb);

int KmlDssSymSolveCI(KmlSolverTask **pHandle, int nb, kml_complex_float *x, int ldx, kml_complex_float *b, int ldb);

int KmlDssSymSolveZI(KmlSolverTask **pHandle, int nb, kml_complex_double *x, int ldx, kml_complex_double *b, int ldb);

参数

参数名	类型	描述	输入/输出
pHandle	KmlSolverTask **	求解器句柄，传入之前步骤的变量。	输入
nb	int	右端项矢量的个数。	输入

版权所有 © 华为技术有限公司

参数名	类型	描述	输入/输出
x	• 在KmlDssSymSolveSI中为float *。 • 在KmlDssSymSolveDI中为double *。 • 在KmlDssSymSolveCI中为kml_complex_float *。 • 在KmlDssSymSolveZI中为kml_complex_double *。	用于存储求解结果的稠密向量，其元素个数为ldx*nb，第k个矢量的第i个元素位置为x[i + k * ldx]。	输出
ldx	int	x的leading dimension大小，要求ldx ≥ max(1, n)。	输入
b	• 在KmlDssSymSolveSI中为float *。 • 在KmlDssSymSolveDI中为double *。 • 在KmlDssSymSolveCI中为kml_complex_float *。 • 在KmlDssSymSolveZI中为kml_complex_double *。	稠密向量存储右端项，其元素个数为ldb*nb，第k个矢量的第i个元素位置为b[i + k * ldb]。	输入
ldb	int	b的leading dimension大小，要求ldb ≥ max(1, n)。	输入

返回值

返回值	类型	描述
KMLSS_NO_ERROR	int	正常执行。
KMLSS_NULL_ARGUMENT	int	pHandle，x，b为空。
KMLSS_BAD_NB	int	输入参数nb小于等于0。
KMLSS_BAD_HANDLE	int	pHandle格式存在问题。

版权所有 © 华为技术有限公司

返回值	类型	描述
KMLSS_BAD_LDX	int	ldx取值不在有效范围内。
KMLSS_BAD_LDB	int	ldb取值不在有效范围内。
KMLSS_INVALID_CALL_ORDER	int	函数执行顺序错误，尚未执行分解阶段就提前执行求解函数。
KMLSS_NO_MEMORY	int	内存分配失败。

依赖

#include "kml_dss.h"或#include "kml_solver.h"

示例

C Interface：

```
int n = 8;
int ia[9] = {0, 2, 4, 6, 7, 8, 9, 10, 11};
int ja[11] = {0, 7, 1, 6, 2, 5, 3, 4, 5, 6, 7};
double a[11] = {1.0, 2.0, -2.0, 3.0, 3.0, 4.0, -4.0, 5.0, -6.0, 7.0, 8.0};

double b[8] = {3.0, 1.0, 7.0, -4.0, 5.0, -2.0, 10.0, 10.0};
double x[8];
int nrhs = 1;
int ldx = n, ldb = n;

KmlSolverTask *handle;
int ierr = KmlDssSymInitDI(&handle, n, a, ja, ia);
if (ierr != 0) {
	printf("\nERROR in KmlDssSymInitDI: %d", ierr);
}
ierr = KmlDssSymAnalyzeDI(&handle);
if (ierr != 0) {
	printf("\nERROR in KmlDssSymAnalyzeDI: %d", ierr);
}
ierr = KmlDssSymFactorizeDI(&handle);
if (ierr != 0) {
	printf("\nERROR in KmlDssSymFactorizeDI: %d", ierr);
}
ierr = KmlDssSymSolveDI(&handle, nrhs, x, ldx, b, ldb);
if (ierr != 0) {
	printf("\nERROR in KmlDssSymSolveDI: %d", ierr);
}
for (int i = 0; i < n; i++) {
	printf("%lf ", x[i]);
}
printf("\n");
```

运行结果：

```
1.000000 1.000000 1.000000 1.000000 1.000000 1.000000 1.000000 1.000000
```

6.5.3.3.8.5 KmlDssSymGet?IL

获取与handle关联的长整型值。

接口定义

C Interface：

 版权所有 © 华为技术有限公司

int KmlDssSymGetSIL(KmlSolverTask **pHandle, enum KmlSolverParam param, int64_t *value, int nvalue);

int KmlDssSymGetDIL(KmlSolverTask **pHandle, enum KmlSolverParam param, int64_t *value, int nvalue);

int KmlDssSymGetCIL(KmlSolverTask **pHandle, enum KmlSolverParam param, int64_t *value, int nvalue);

int KmlDssSymGetZIL(KmlSolverTask **pHandle, enum KmlSolverParam param, int64_t *value, int nvalue);

参数

参数名	类型	描述	输入/输出
pHandle	KmlSolverTask **	求解器句柄，pHandle不可为空指针。	输入
param	enum KmlSolverParam	当前仅支持KMLSS_FILL_IN，表示填入数目。	输入
value	int64_t *	直接求解相关参数数组。	输出
nvalue	int	value数组元素个数，目前要求为1。	输入

返回值

返回值	类型	描述
KMLSS_NO_ERROR	int	正常执行。
KMLSS_NULL_ARGUMENT	int	pHandle或data为空指针。
KMLSS_BAD_HANDLE	int	pHandle格式存在问题。
KMLSS_BAD_DATA_SIZE	int	用户设定的value数组长度nvalue不合理。
KMLSS_INTERNAL_ERROR	int	pHandle状态异常，KMLSS_FILL_IN获取失败。
KMLSS_BAD_SELECTOR	int	用户的KML_SOLVER_PARAM输入当前不支持，或在求解过程中不存在。

依赖

#include "kml_dss.h"或#include "kml_solver.h"

 版权所有 © 华为技术有限公司

示例

C Interface：

```
int n = 8;
int ia[9] = {0, 2, 4, 6, 7, 8, 9, 10, 11};
int ja[11] = {0, 7, 1, 6, 2, 5, 3, 4, 5, 6, 7};
double a[11] = {1.0, 2.0, -2.0, 3.0, 3.0, 4.0, -4.0, 5.0, -6.0, 7.0, 8.0};

double b[8] = {3.0, 1.0, 7.0, -4.0, 5.0, -2.0, 10.0, 10.0};
double x[8];
int nrhs = 1;
int ldx = n, ldb = n;

KmlSolverTask *handle;
int ierr = KmlDssSymInitDI(&handle, n, a, ja, ia);
if (ierr != 0) {
	printf("\nERROR in KmlDssSymInitDI: %d", ierr);
}
ierr = KmlDssSymAnalyzeDI(&handle);
if (ierr != 0) {
	printf("\nERROR in KmlDssSymAnalyzeDI: %d", ierr);
}
ierr = KmlDssSymFactorizeDI(&handle);
if (ierr != 0) {
	printf("\nERROR in KmlDssSymFactorizeDI: %d", ierr);
}
ierr = KmlDssSymSolveDI(&handle, nrhs, x, ldx, b, ldb);
if (ierr != 0) {
	printf("\nERROR in KmlDssSymSolveDI: %d", ierr);
}

int64_t nnz;
ierr = KmlDssSymGetDIL(&handle, KMLSS_FILL_IN, &nnz, 1);
if (ierr != 0) {
	printf("\nERROR in KmlDssSymGetDIL: %d", ierr);
}
printf("NNZ = %d\n", nnz);
```

运行结果：

```
NNZ = 32
```

6.5.3.3.8.6 KmlDssSymGet?II

获取与handle关联的整型值。

接口定义

C Interface：

int KmlDssSymGetSII(KmlSolverTask **pHandle, enum KmlSolverParam param, int *value, int nvalue);

int KmlDssSymGetDII(KmlSolverTask **pHandle, enum KmlSolverParam param, int *value, int nvalue);

int KmlDssSymGetCII(KmlSolverTask **pHandle, enum KmlSolverParam param, int *value, int nvalue);

int KmlDssSymGetZII(KmlSolverTask **pHandle, enum KmlSolverParam param, int *value, int nvalue);

 版权所有 © 华为技术有限公司

参数

参数名	类型	描述	输入/输出
pHandle	KmlSolverTask **	求解器句柄，pHandle不可为空指针。	输入
param	enum KmlSolverParam	当前仅支持KMLSS_REFINEMENT_STEPS，表示迭代优化次数。	输入
value	int *	直接求解相关参数数组。	输出
nvalue	int	value数组元素个数，目前要求为1。	输入

返回值

返回值	类型	描述
KMLSS_NO_ERROR	int	正常执行。
KMLSS_NULL_ARGUMENT	int	pHandle或data为空指针。
KMLSS_BAD_HANDLE	int	pHandle格式存在问题。
KMLSS_BAD_DATA_SIZE	int	用户设定的value数组长度nvalue不合理，即不为1。
KMLSS_BAD_SELECTOR	int	用户的KML_SOLVER_PARAM输入当前不支持，或在求解过程中不存在。

依赖

#include "kml_dss.h"或#include "kml_solver.h"

示例

C Interface：

```
int n = 8;
int ia[9] = {0, 2, 4, 6, 7, 8, 9, 10, 11};
int ja[11] = {0, 7, 1, 6, 2, 5, 3, 4, 5, 6, 7};
double a[11] = {1.0, 2.0, -2.0, 3.0, 3.0, 4.0, -4.0, 5.0, -6.0, 7.0, 8.0};

double b[8] = {3.0, 1.0, 7.0, -4.0, 5.0, -2.0, 10.0, 10.0};
double x[8];
int nrhs = 1;
int ldx = n, ldb = n;

KmlSolverTask *handle;
int ierr = KmlDssSymInitDI(&handle, n, a, ja, ia);
if (ierr != 0) {
    printf("\nERROR in KmlDssSymInitDI: %d", ierr);
```

 版权所有 © 华为技术有限公司

```
}
ierr = KmlDssSymAnalyzeDI(&handle);
if (ierr != 0) {
	printf("\nERROR in KmlDssSymAnalyzeDI: %d", ierr);
}
ierr = KmlDssSymFactorizeDI(&handle);
if (ierr != 0) {
	printf("\nERROR in KmlDssSymFactorizeDI: %d", ierr);
}

double tolerance = 1.e-12;
ierr = KmlDssSymSetDID(&handle, KMLSS_REFINEMENT_TOLERANCE_LEVEL, &tolerance, 1);
if (ierr != 0) {
	printf("\nERROR in KmlDssSymSetDID: %d", ierr);
}
int nsteps = 2;
ierr = KmlDssSymSetDII(&handle, KMLSS_REFINEMENT_MAX_STEPS, &nsteps, 1);
if (ierr != 0) {
	printf("\nERROR in KmlDssSymSetDII: %d", ierr);
}

ierr = KmlDssSymSolveDI(&handle, nrhs, x, ldx, b, ldb);
if (ierr != 0) {
	printf("\nERROR in KmlDssSymSolveDI: %d", ierr);
}

printf("\n");
int nstep;
ierr = KmlDssSymGetDII(&handle, KMLSS_REFINEMENT_STEPS, &nstep, 1);
if (ierr != 0) {
	printf("\nERROR in KmlDssSymGetDII: %d", ierr);
}
printf("REFINEMENT_STEP = %d\n", nstep);
```

运行结果：

```
REFINEMENT_STEP = 1
```

6.5.3.3.8.7 KmlDssSymGet?I(D/S)

获取与handle关联的实数型值。

接口定义

C Interface：

int KmlDssSymGetSIS(KmlSolverTask **pHandle, enum KmlSolverParam param, float *value, int nvalue);

int KmlDssSymGetDID(KmlSolverTask **pHandle, enum KmlSolverParam param, double *value, int nvalue);

int KmlDssSymGetCIS(KmlSolverTask **pHandle, enum KmlSolverParam param, float *value, int nvalue);

int KmlDssSymGetZID(KmlSolverTask **pHandle, enum KmlSolverParam param, double *value, int nvalue);

 版权所有 © 华为技术有限公司

参数

参数名	类型	描述	输入/输出
pHandle	KmlSolverTask **	求解器句柄，pHandle不可为空指针。	输入
param	enum KmlSolverParam	当前仅支持KMLSS_REFINEMENT_RESIDUAL，表示迭代优化的残差。	输入
value	double *或float *	直接求解相关参数数组。	输出
nvalue	int	value数组元素个数，要求与右端项数目nb一致。	输入

返回值

返回值	类型	描述
KMLSS_NO_ERROR	int	正常执行。
KMLSS_NULL_ARGUMENT	int	pHandle或data为空指针、或者参数不合理。
KMLSS_BAD_HANDLE	int	pHandle格式存在问题。
KMLSS_BAD_DATA_SIZE	int	用户设定的value数组长度nvalue不合理，即与右端项数目不一致。
KMLSS_BAD_SELECTOR	int	用户的KML_SOLVER_PARAM输入当前不支持，或在求解过程中不存在。

依赖

#include "kml_dss.h"或#include "kml_solver.h"

示例

C Interface：

```
int n = 8;
int ia[9] = {0, 2, 4, 6, 7, 8, 9, 10, 11};
int ja[11] = {0, 7, 1, 6, 2, 5, 3, 4, 5, 6, 7};
double a[11] = {1.0, 2.0, -2.0, 3.0, 3.0, 4.0, -4.0, 5.0, -6.0, 7.0, 8.0};

double b[8] = {3.0, 1.0, 7.0, -4.0, 5.0, -2.0, 10.0, 10.0};
double x[8];
int nrhs = 1;
int ldx = n, ldb = n;

KmlSolverTask *handle;
int ierr = KmlDssSymInitDI(&handle, n, a, ja, ia);
```

 版权所有 © 华为技术有限公司

```
if (ierr != 0) {
	printf("\nERROR in KmlDssSymInitDI: %d", ierr);
}
ierr = KmlDssSymAnalyzeDI(&handle);
if (ierr != 0) {
	printf("\nERROR in KmlDssSymAnalyzeDI: %d", ierr);
}
ierr = KmlDssSymFactorizeDI(&handle);
if (ierr != 0) {
	printf("\nERROR in KmlDssSymFactorizeDI: %d", ierr);
}

double tolerance = 1.e-12;
ierr = KmlDssSymSetDID(&handle, KMLSS_REFINEMENT_TOLERANCE_LEVEL, &tolerance, 1);
if (ierr != 0) {
	printf("\nERROR in KmlDssSymSetDID: %d", ierr);
}
int nsteps = 2;
ierr = KmlDssSymSetDII(&handle, KMLSS_REFINEMENT_MAX_STEPS, &nsteps, 1);
if (ierr != 0) {
	printf("\nERROR in KmlDssSymSetDII: %d", ierr);
}
ierr = KmlDssSymSolveDI(&handle, nrhs, x, ldx, b, ldb);
if (ierr != 0) {
	printf("\nERROR in KmlDssSymSolveDI: %d", ierr);
}

double res;
ierr = KmlDssSymGetDID(&handle, KMLSS_REFINEMENT_RESIDUAL, &res, 1);
if (ierr != 0) {
	printf("\nERROR in KmlDssSymGetDID: %d", ierr);
}
printf("REFINEMENT_RESIDUAL = %f\n", res);
```

运行结果：

```
REFINEMENT_RESIDUAL = 0.000000
```

6.5.3.3.8.8 KmlDssSymSet?II

设置与handle关联的整型值。

接口定义

C Interface：

int KmlDssSymSetSII(KmlSolverTask **pHandle, enum KmlSolverParam param, int *value, int nvalue);

int KmlDssSymSetDII(KmlSolverTask **pHandle, enum KmlSolverParam param, int *value, int nvalue);

int KmlDssSymSetCII(KmlSolverTask **pHandle, enum KmlSolverParam param, int *value, int nvalue);

int KmlDssSymSetZII(KmlSolverTask **pHandle, enum KmlSolverParam param, int *value, int nvalue);

版权所有 © 华为技术有限公司

参数

参数名	类型	描述	输入/输出
pHandle	KmlSolverTask **	求解器句柄，pHandle不可为空指针。	输入
param	enum KmlSolverParam	• KMLSS_PERM表示用户给定的矩阵排序。 • KMLSS_REFINEMENT_MAX_STEPS表示最大的迭代优化次数。 • KMLSS_BOOST_THRESHOLD表示boost阈值。 • KMLSS_SCALING_TYPE表示scaling算法的类型。 • KMLSS_MATCHING_TYPE表示matching算法的类型。 • KMLSS_REFINEMENT_TOLERANCE_LEVEL表示迭代优化算法设置的阈值，取10的负幂次。	输入
value	int *	直接求解相关参数数组。	输入
nvalue	int	value数组元素个数。	输入

对应不同的param参数，value有不同的取值，特别是可以取一些常量。在此一一列举。

param参数	值	功能
KMLSS_PERM	一维数组，数组长度与矩阵维度一致。	用户自定义行/列排序
KMLSS_REFINEMENT_MAX_STEPS	长度为1的正整数数组或正整数的地址。	迭代优化的最大次数
KMLSS_BOOST_THRESHOLD	长度为1的正整数数组或正整数的地址。	控制solver内部的boost参数
KMLSS_SCALING_TYPE	KMLSS_NO_SCALING	无scaling操作
	KMLSS_ROWCOLUMN_SCALING	执行行/列的scaling操作
	KMLSS_MATCHBASED_SCALING	执行与Match算法相关的scaling操作
KMLSS_MATCHING_TYPE	KMLSS_MATCHING_OFF	关闭Match算法
	KMLSS_MATCHING_HUNGARIAN	匈牙利算法

 版权所有 © 华为技术有限公司

param参数	值	功能
KMLSS_REFINEMENT_TOLERANCE_LEVEL	长度为1的正整数数组或正整数的地址	迭代优化算法设置的阈值取10的负幂次

返回值

返回值	类型	描述
KMLSS_NO_ERROR	int	正常执行。
KMLSS_NULL_ARGUMENT	int	handle或data为空指针
KMLSS_BAD_HANDLE	int	pHandle格式存在问题。
KMLSS_BAD_SELECTOR	int	用户的KML_SOLVER_PARAM输入当前不支持，或在求解过程中不存在。
KMLSS_BAD_DATA	int	用户的取值不符合要求。

依赖

#include "kml_dss.h"或#include "kml_solver.h"

示例

C Interface：

```
int n = 8;
int ia[9] = {0, 2, 4, 6, 7, 8, 9, 10, 11};
int ja[11] = {0, 7, 1, 6, 2, 5, 3, 4, 5, 6, 7};
double a[11] = {1.0, 2.0, -2.0, 3.0, 3.0, 4.0, -4.0, 5.0, -6.0, 7.0, 8.0};

double b[8] = {3.0, 1.0, 7.0, -4.0, 5.0, -2.0, 10.0, 10.0};
double x[8];
int nrhs = 1;
int ldx = n, ldb = n;

KmlSolverTask *handle;
int ierr = KmlDssSymInitDI(&handle, n, a, ja, ia);
if (ierr != 0) {
	printf("\nERROR in KmlDssSymInitDI: %d", ierr);
}
ierr = KmlDssSymAnalyzeDI(&handle);
if (ierr != 0) {
	printf("\nERROR in KmlDssSymAnalyzeDI: %d", ierr);
}
ierr = KmlDssSymFactorizeDI(&handle);
if (ierr != 0) {
	printf("\nERROR in KmlDssSymFactorizeDI: %d", ierr);
}

double tolerance = 1.e-12;
ierr = KmlDssSymSetDID(&handle, KMLSS_REFINEMENT_TOLERANCE_LEVEL, &tolerance, 1);
if (ierr != 0) {
	printf("\nERROR in KmlDssSymSetDID: %d", ierr);
```

 版权所有 © 华为技术有限公司

```
}
int nsteps = 2;
ierr = KmlDssSymSetDII(&handle, KMLSS_REFINEMENT_MAX_STEPS, &nsteps, 1);
if (ierr != 0) {
	printf("\nERROR in KmlDssSymSetDII: %d", ierr);
}

ierr = KmlDssSymSolveDI(&handle, nrhs, x, ldx, b, ldb);
if (ierr != 0) {
	printf("\nERROR in KmlDssSymSolveDI: %d", ierr);
}

int nstep;
ierr = KmlDssSymGetDII(&handle, KMLSS_REFINEMENT_STEPS, &nstep, 1);
printf("REFINEMENT_STEP = %d\n", nstep);
```

运行结果：

```
REFINEMENT_STEP = 1
```

6.5.3.3.8.9 KmlDssSymSet?I(D/S)

设置与handle关联的实数型值。

接口定义

C Interface：

int KmlDssSymSetSIS(KmlSolverTask **pHandle, enum KmlSolverParam param, float *value, int nvalue);

int KmlDssSymSetDID(KmlSolverTask **pHandle, enum KmlSolverParam param, double *value, int nvalue);

int KmlDssSymSetCIS(KmlSolverTask **pHandle, enum KmlSolverParam param, float *value, int nvalue);

int KmlDssSymSetZID(KmlSolverTask **pHandle, enum KmlSolverParam param, double *value, int nvalue);

参数

参数名	类型	描述	输入/输出
pHandle	KmlSolverTask **	求解器句柄，pHandle不可为空指针。	输入
param	enum KmlSolverParam	• KMLSS_REFINEMENT_TOLERANCE_LEVEL表示迭代优化算法设置的阈值。 • KMLSS_PIVOTING_THRESHOLD表示迭代pivot算法设置的阈值，对应的value即为取值。	输入
value	double *或float *	直接求解相关参数数组。	输入
nvalue	int	value数组元素个数。	输入

版权所有 © 华为技术有限公司

返回值

返回值	类型	描述
KMLSS_NO_ERROR	int	正常执行。
KMLSS_BAD_SELECTOR	int	用户的KML_SOLVER_PARAM输入当前不支持，或在求解过程中不存在。

依赖

#include "kml_dss.h"或#include "kml_solver.h"

示例

C Interface：

```
int n = 8;
int ia[9] = {0, 2, 4, 6, 7, 8, 9, 10, 11};
int ja[11] = {0, 7, 1, 6, 2, 5, 3, 4, 5, 6, 7};
double a[11] = {1.0, 2.0, -2.0, 3.0, 3.0, 4.0, -4.0, 5.0, -6.0, 7.0, 8.0};

double b[8] = {3.0, 1.0, 7.0, -4.0, 5.0, -2.0, 10.0, 10.0};
double x[8];
int nrhs = 1;
int ldx = n, ldb = n;

KmlSolverTask *handle;
int ierr = KmlDssSymInitDI(&handle, n, a, ja, ia);
if (ierr != 0) {
	printf("\nERROR in KmlDssSymInitDI: %d", ierr);
}
ierr = KmlDssSymAnalyzeDI(&handle);
if (ierr != 0) {
	printf("\nERROR in KmlDssSymAnalyzeDI: %d", ierr);
}
ierr = KmlDssSymFactorizeDI(&handle);
if (ierr != 0) {
	printf("\nERROR in KmlDssSymFactorizeDI: %d", ierr);
}

double tolerance = 1.e-12;
ierr = KmlDssSymSetDID(&handle, KMLSS_REFINEMENT_TOLERANCE_LEVEL, &tolerance, 1);
if (ierr != 0) {
	printf("\nERROR in KmlDssSymSetDID: %d", ierr);
}
int nsteps = 2;
ierr = KmlDssSymSetDII(&handle, KMLSS_REFINEMENT_MAX_STEPS, &nsteps, 1);
if (ierr != 0) {
	printf("\nERROR in KmlDssSymSetDII: %d", ierr);
}

ierr = KmlDssSymSolveDI(&handle, nrhs, x, ldx, b, ldb);
if (ierr != 0) {
	printf("\nERROR in KmlDssSymSolveDI: %d", ierr);
}

int nstep;
ierr = KmlDssSymGetDII(&handle, KMLSS_REFINEMENT_STEPS, &nstep, 1);
printf("REFINEMENT_STEP = %d\n", nstep);
double res;
ierr = KmlDssSymGetDID(&handle, KMLSS_REFINEMENT_RESIDUAL, &res, 1);
printf("REFINEMENT_RESIDUAL = %f\n", res);
```

 版权所有 © 华为技术有限公司

运行结果：

```
REFINEMENT_STEP = 1
REFINEMENT_RESIDUAL = 0.000000
```

6.5.3.3.8.10 KmlDssSymClean?I

释放内部数据结构，将Handle指向空指针。

接口定义

C Interface：

int KmlDssSymCleanSI(KmlSolverTask **pHandle);

int KmlDssSymCleanDI(KmlSolverTask **pHandle);

int KmlDssSymCleanCI(KmlSolverTask **pHandle);

int KmlDssSymCleanZI(KmlSolverTask **pHandle);

参数

参数名	类型	描述	输入/输出
pHandle	KmlSolverTask **	求解器句柄，传入之前步骤的变量。	输入/输出

返回值

返回值	类型	描述
KMLSS_NO_ERROR	int	正常执行。
KMLSS_NULL_ARGUMENT	int	无效的参数。
KMLSS_BAD_HANDLE	int	pHandle格式存在问题。

依赖

#include "kml_dss.h"或#include "kml_solver.h"

示例

C Interface：

```
int n = 8;
int ia[9] = {0, 2, 4, 6, 8, 10, 12, 13, 14};
int ja[14] = {0, 2, 1, 3, 2, 4, 3, 5, 4, 6, 5, 7, 6, 7};
double a[14] = {1.0, 2.0, 3.0, 4.0, 5.0, 6.0, 7.0, 8.0, 9.0, 10.0, 11.0, 12.0, 13.0, 14.0};
double b[8] = {3.0, 7.0, 13.0, 19.0, 25.0, 31.0, 23.0, 26.0};
double x[8];
```


版权所有 © 华为技术有限公司

```
KmlSolverTask* matrix_handle = NULL;
int ierr = KmlDssSymInitDI(&matrix_handle, n, a, ja, ia);
if (ierr != 0) {
    printf("\nERROR in KmlDssSymInitDI: %d", ierr);
}
ierr = KmlDssSymAnalyzeDI(&matrix_handle);
if (ierr != 0) {
    printf("\nERROR in KmlDssSymAnalyzeDI: %d", ierr);
}
ierr = KmlDssSymFactorizeDI(&matrix_handle);
if (ierr != 0) {
    printf("\nERROR in KmlDssSymFactorizeDI: %d", ierr);
}
ierr = KmlDssSymSolveDI(&matrix_handle, nrhs, x, ldx, b, ldb);
if (ierr != 0) {
    printf("\nERROR in KmlDssSymSolveDI: %d", ierr);
}
ierr = KmlDssSymCleanDI(&matrix_handle);
if (ierr != 0) {
    printf("\nERROR in KmlDssSymCleanDI: %d", ierr);
}
```

6.5.3.3.9 LU (DEPRECATED)

直接法推荐使用**DSS**。

6.5.3.3.9.1 KmlDssGenInit?I

初始化数据结构，并将用户提供的系数矩阵关联到求解系统中，初始化LU方法时使用完整矩阵的CSR格式。

接口定义

C Interface：

int KmlDssGenInitSI(KmlSolverTask **pHandle, int n, float *a, int *ja, int *ia)

int KmlDssGenInitDI(KmlSolverTask **pHandle, int n, double *a, int *ja, int *ia)

int KmlDssGenInitCI(KmlSolverTask **pHandle, int n, kml_complex_float *a, int *ja, int *ia)

int KmlDssGenInitZI(KmlSolverTask **pHandle, int n, kml_complex_double *a, int *ja, int *ia)

参数

参数名	类型	描述	输入/输出
pHandle	KmlSolverTask **	求解器句柄，pHandle不可为空指针。	输入/输出
n	int	n为矩阵A的维度。	输入

版权所有 © 华为技术有限公司

参数名	类型	描述	输入/输出
a	• 在KmlDssGenInitSI中为float *。 • 在KmlDssGenInitDI中为double *。 • 在KmlDssGenInitCI中为kml_complex_float *。 • 在KmlDssGenInitZI中为kml_complex_double *。	a是维度为非零元素个数的数组，a[i]为第i个非零元的值。该数组的生命周期与pHandle相同。	输入
ja	int	ja是维度为非零元素个数的数组，ja[i]为第i个非零元的列号。该数组的生命周期与pHandle相同。	输入
ia	int	ia为n+1维数组，ia[i]为第i行第一个非零元在所有非零元中的索引。该数组的生命周期与pHandle相同。	输入

返回值

返回值	类型	描述
KMLSS_NO_ERROR	int	正常执行。
KMLSS_NULL_ARGUMENT	int	pHandle，a，ja，ia中有空指针。
KMLSS_BAD_N	int	矩阵维数输入值小于等于0。
KMLSS_NO_MEMORY	int	内存开辟失败。

依赖

#include "kml_dss.h"或#include "kml_solver.h"

示例

C Interface：

```
int n = 8;
int ia[9] = {0, 2, 4, 6, 7, 8, 10, 12, 14};
int ja[14] = {0, 7, 1, 6, 2, 5, 3, 4, 2, 5, 1, 6, 0, 7};
double a[14] = {1.0, 2.0, -2.0, 3.0, 3.0, 4.0, -4.0, 5.0, 4.0, -6.0, 3.0, 7.0, 2.0, 8.0};

KmlSolverTask *handle;
```

 版权所有 © 华为技术有限公司

```
int ierr = KmlDssGenInitDI(&handle, n, a, ja, ia);
if (ierr != 0) {
    printf("\nERROR in KmlDssGenInitDI: %d", ierr);
}
```

6.5.3.3.9.2 KmlDssGenAnalyze?I

分析数据结构，对稀疏矩阵进行重排和符号分解。

接口定义

C Interface：

int KmlDssGenAnalyzeSI(KmlSolverTask **pHandle);

int KmlDssGenAnalyzeDI(KmlSolverTask **pHandle);

int KmlDssGenAnalyzeCI(KmlSolverTask **pHandle);

int KmlDssGenAnalyzeZI(KmlSolverTask **pHandle);

参数

参数名	类型	描述	输入/输出
pHandle	KmlSolverTask **	求解器句柄，传入之前步骤的变量。	输入

返回值

返回值	类型	描述
KMLSS_NO_ERROR	int	正常执行。
KMLSS_NULL_ARGUMENT	int	pHandle为空指针。
KMLSS_BAD_HANDLE	int	pHandle格式存在问题。
KMLSS_NONZERO_INDEXING	int	行索引指针不是从0开始，即ia[0]不等于0。
KMLSS_MISSING_DIAGONAL_ELEMENT	int	稀疏矩阵对角线元素全部为空。
KMLSS_INTERNAL_ERROR	int	稀疏矩阵对角线存在空元素/算法内部执行失败。
KMLSS_REORDERING_PROBLEM	int	排序算法失败。

依赖

#include "kml_dss.h"或#include "kml_solver.h"

版权所有 © 华为技术有限公司

示例

C Interface：

```
int n = 8;
int ia[9] = {0, 2, 4, 6, 7, 8, 10, 12, 14};
int ja[14] = {0, 7, 1, 6, 2, 5, 3, 4, 2, 5, 1, 6, 0, 7};
double a[14] = {1.0, 2.0, -2.0, 3.0, 3.0, 4.0, -4.0, 5.0, 4.0, -6.0, 3.0, 7.0, 2.0, 8.0};

KmlSolverTask *handle;
int ierr = KmlDssGenInitDI(&handle, n, a, ja, ia);
if (ierr != 0) {
    printf("\nERROR in KmlDssGenInitDI: %d", ierr);
}
ierr = KmlDssGenAnalyzeDI(&handle);
if (ierr != 0) {
    printf("\nERROR in KmlDssGenAnalyzeDI: %d", ierr);
}
```

6.5.3.3.9.3 KmlDssGenFactorize?I

实现通用矩阵的LU分解。

接口定义

C Interface：

int KmlDssGenFactorizeSI(KmlSolverTask **pHandle);

int KmlDssGenFactorizeDI(KmlSolverTask **pHandle);

int KmlDssGenFactorizeCI(KmlSolverTask **pHandle);

int KmlDssGenFactorizeZI(KmlSolverTask **pHandle);

参数

参数名	类型	描述	输入/输出
pHandle	KmlSolverTask **	求解器句柄，传入之前步骤的变量。	输入

返回值

返回值	类型	描述
KMLSS_NO_ERROR	int	正常执行。
KMLSS_NULL_ARGUMENT	int	pHandle为空指针。
KMLSS_BAD_HANDLE	int	参数无效或格式存在问题。
KMLSS_INVALID_CALL_ORDER	int	错误的调用顺序，之前未进入分解阶段。

版权所有 © 华为技术有限公司

返回值	类型	描述
KMLSS_NO_MEMORY	int	内存分配失败。
KMLSS_INTERNAL_ERROR	int	算法内部执行失败。

依赖

#include "kml_dss.h"或#include "kml_solver.h"

示例

C Interface：

```
int n = 8;
int ia[9] = {0, 2, 4, 6, 7, 8, 10, 12, 14};
int ja[14] = {0, 7, 1, 6, 2, 5, 3, 4, 2, 5, 1, 6, 0, 7};
double a[14] = {1.0, 2.0, -2.0, 3.0, 3.0, 4.0, -4.0, 5.0, 4.0, -6.0, 3.0, 7.0, 2.0, 8.0};

KmlSolverTask *handle;
int ierr = KmlDssGenInitDI(&handle, n, a, ja, ia);
if (ierr != 0) {
    printf("\nERROR in KmlDssGenInitDI: %d", ierr);
}
ierr = KmlDssGenAnalyzeDI(&handle);
if (ierr != 0) {
    printf("\nERROR in KmlDssGenAnalyzeDI: %d", ierr);
}
ierr = KmlDssGenFactorizeDI(&handle);
if (ierr != 0) {
    printf("\nERROR in KmlDssGenFactorizeDI: %d", ierr);
}
```

6.5.3.3.9.4 KmlDssGenSolve?I

求解LUx=b。

接口定义

C Interface：

int KmlDssGenSolveSI(KmlSolverTask **pHandle, int nb, float *x, int ldx, float *b, int ldb);

int KmlDssGenSolveDI(KmlSolverTask **pHandle, int nb, double *x, int ldx, double *b, int ldb);

int KmlDssGenSolveCI(KmlSolverTask **pHandle, int nb, kml_complex_float *x, int ldx, kml_complex_float *b, int ldb);

int KmlDssGenSolveZI(KmlSolverTask **pHandle, int nb, kml_complex_double *x, int ldx, kml_complex_double *b, int ldb);

 版权所有 © 华为技术有限公司

参数

参数名	类型	描述	输入/输出
pHandle	KmlSolverTask **	求解器句柄，传入之前步骤的变量。	输入
nb	int	右端项矢量的个数。	输入
x	• 在KmlDssSymSolveSI中为float *。 • 在KmlDssSymSolveDI中为double *。 • 在KmlDssSymSolveCI中为kml_complex_float *。 • 在KmlDssSymSolveZI中为kml_complex_double *。	稠密向量存储求解结果，其元素个数为ldx*nb，第k个矢量的第i个元素位置为x[i + k * ldx]。	输入/输出
ldx	int	x的leading dimension大小，要求ldx ≥ max(1, n)。	输入
b	• 在KmlDssSymSolveSI中为float *。 • 在KmlDssSymSolveDI中为double *。 • 在KmlDssSymSolveCI中为kml_complex_float *。 • 在KmlDssSymSolveZI中为kml_complex_double *。	稠密向量存储右端项，其元素个数为ldb*nb，第k个矢量的第i个元素位置为b[i + k * ldb]。	输入
ldb	int	b的leading dimension大小，要求ldb ≥ max(1, n)。	输入

返回值

返回值	类型	描述
KMLSS_NO_ERROR	int	正常执行。

 版权所有 © 华为技术有限公司

返回值	类型	描述
KMLSS_NULL_ARGUMENT	int	pHandle，x，b为空。
KMLSS_BAD_NB	int	输入参数nb小于等于0。
KMLSS_BAD_HANDLE	int	pHandle格式存在问题。
KMLSS_BAD_LDX	int	ldx取值不在有效范围内。
KMLSS_BAD_LDB	int	ldb取值不在有效范围内。
KMLSS_INVALID_CALL_ORDER	int	函数执行顺序错误，尚未执行分解阶段就提前执行求解函数。
KMLSS_NO_MEMORY	int	内存分配失败。

依赖

#include "kml_dss.h"或#include "kml_solver.h"

示例

C Interface：

```
int n = 8;
int ia[9] = {0, 2, 4, 6, 7, 8, 10, 12, 14};
int ja[14] = {0, 7, 1, 6, 2, 5, 3, 4, 2, 5, 1, 6, 0, 7};
double a[14] = {1.0, 2.0, -2.0, 3.0, 3.0, 4.0, -4.0, 5.0, 4.0, -6.0, 3.0, 7.0, 2.0, 8.0};

double b[8] = {3.0, 1.0, 7.0, -4.0, 5.0, -2.0, 10.0, 10.0};
double x[8];
int nrhs = 1;
int ldx = n, ldb = n;

KmlSolverTask *handle;
int ierr = KmlDssGenInitDI(&handle, n, a, ja, ia);
if (ierr != 0) {
    printf("\nERROR in KmlDssGenInitDI: %d", ierr);
}
ierr = KmlDssGenAnalyzeDI(&handle);
if (ierr != 0) {
    printf("\nERROR in KmlDssGenAnalyzeDI: %d", ierr);
}
ierr = KmlDssGenFactorizeDI(&handle);
if (ierr != 0) {
    printf("\nERROR in KmlDssGenFactorizeDI: %d", ierr);
}
ierr = KmlDssGenSolveDI(&handle, nrhs, x, ldx, b, ldb);
if (ierr != 0) {
    printf("\nERROR in KmlDssGenSolveDI: %d", ierr);
}
for (int i = 0; i < n; i++) {
    printf("%lf ", x[i]);
}
printf("\n");
```

运行结果：

```
1.000000 1.000000 1.000000 1.000000 1.000000 1.000000 1.000000 1.000000
```

 版权所有 © 华为技术有限公司

6.5.3.3.9.5 KmlDssGenGet?IL

获取与handle关联的长整型值。

接口定义

C Interface：

int KmlDssGenGetSIL(KmlSolverTask **pHandle, enum KmlSolverParam param, int64_t *value, int nvalue);

int KmlDssGenGetDIL(KmlSolverTask **pHandle, enum KmlSolverParam param, int64_t *value, int nvalue);

int KmlDssGenGetCIL(KmlSolverTask **pHandle, enum KmlSolverParam param, int64_t *value, int nvalue);

int KmlDssGenGetZIL(KmlSolverTask **pHandle, enum KmlSolverParam param, int64_t *value, int nvalue);

参数

参数名	类型	描述	输入/输出
pHandle	KmlSolverTask **	求解器句柄，pHandle不可为空指针。	输入
param	enum KmlSolverParam	KMLSS_FILL_IN表示填充数量。	输入
value	int64_t *	直接求解相关参数数组。	输出
nvalue	int	value数组元素个数，目前要求为1。	输入

返回值

返回值	类型	描述
KMLSS_NO_ERROR	int	正常执行。
KMLSS_NULL_ARGUMENT	int	pHandle或data为空指针。
KMLSS_BAD_HANDLE	int	pHandle格式存在问题。
KMLSS_BAD_DATA_SIZE	int	用户设定的value数组长度nvalue不合理。
KMLSS_INTERNAL_ERROR	int	pHandle状态异常，KMLSS_FILL_IN获取失败。
KMLSS_BAD_SELECTOR	int	用户的KML_SOLVER_PARAM输入当前不支持，或在求解过程中不存在。

版权所有 © 华为技术有限公司

依赖

#include "kml_dss.h"或#include "kml_solver.h"

示例

C Interface：

```
int n = 8;
int ia[9] = {0, 3, 7, 9, 11, 13, 15, 16, 17};
int ja[17] = {0, 3, 4, 1, 2, 3, 5, 2, 7, 3, 6, 4, 5, 5, 7, 6, 7};
double a[17] = {1.0, 1.0, 2.0, 9.0, 2.0, 1.0, -3.0, 3.0, 2.0, 9.0, -5.0, 6.0, 1.0, 4.0, 1.0, 7.0, 2.0};
double b[8] = {4.0, 9.0, 7.0, 6.0, 9.0, 3.0, 2.0, 5.0};
double x[8];
KmlScasolverTask* matrix_handle = NULL;
int ierr = KmlScadssSpdInitDI(&matrix_handle, n, a, ja, ia, MPI_COMM_WORLD);
ierr = KmlScadssSpdAnalyzeDI(&matrix_handle);

int nrhs = 1;
int ldx=n, ldb=n;
ierr = KmlScadssSpdSolveDI(&matrix_handle, nrhs, x, ldx, b, ldb);
if (rank == 0) {
    int64_t nnz;
    ierr = KmlScadssSpdGetDIL(&matrix_handle, KMLSS_FILL_IN, &nnz, 1);
    printf("NNZ = %lld\n", nnz);
}

int n = 8;
int ia[9] = {0, 2, 4, 6, 7, 8, 10, 12, 14};
int ja[14] = {0, 7, 1, 6, 2, 5, 3, 4, 2, 5, 1, 6, 0, 7};
double a[14] = {1.0, 2.0, -2.0, 3.0, 3.0, 4.0, -4.0, 5.0, 4.0, -6.0, 3.0, 7.0, 2.0, 8.0};

double b[8] = {3.0, 1.0, 7.0, -4.0, 5.0, -2.0, 10.0, 10.0};
double x[8];
int nrhs = 1;
int ldx = n, ldb = n;

KmlSolverTask *handle;
int ierr = KmlDssGenInitDI(&handle, n, a, ja, ia);
if (ierr != 0) {
    printf("\nERROR in KmlDssGenInitDI: %d", ierr);
}
ierr = KmlDssGenAnalyzeDI(&handle);
if (ierr != 0) {
    printf("\nERROR in KmlDssGenAnalyzeDI: %d", ierr);
}
ierr = KmlDssGenFactorizeDI(&handle);
if (ierr != 0) {
    printf("\nERROR in KmlDssGenFactorizeDI: %d", ierr);
}
ierr = KmlDssGenSolveDI(&handle, nrhs, x, ldx, b, ldb);
if (ierr != 0) {
    printf("\nERROR in KmlDssGenSolveDI: %d", ierr);
}

int64_t nnz;
ierr = KmlDssGenGetDIL(&handle, KMLSS_FILL_IN, &nnz, 1);
if (ierr != 0) {
    printf("\nERROR in KmlDssGenGetDIL: %d", ierr);
}
printf("NNZ = %lld\n", nnz);
```

运行结果：

```
NNZ = 32
```

 版权所有 © 华为技术有限公司

6.5.3.3.9.6 KmlDssGenGet?II

获取与handle关联的整型值。

接口定义

C Interface：

int KmlDssGenGetSII(KmlSolverTask **pHandle, enum KmlSolverParam param, int *value, int nvalue);

int KmlDssGenGetDII(KmlSolverTask **pHandle, enum KmlSolverParam param, int *value, int nvalue);

int KmlDssGenGetCII(KmlSolverTask **pHandle, enum KmlSolverParam param, int *value, int nvalue);

int KmlDssGenGetZII(KmlSolverTask **pHandle, enum KmlSolverParam param, int *value, int nvalue);

参数

参数名	类型	描述	输入/输出
pHandle	KmlSolverTask **	求解器句柄，pHandle不可为空指针。	输入
param	enum KmlSolverParam	KMLSS_REFINEMENT_STEPS表示迭代优化次数。	输入
value	int *	直接求解相关参数数组。	输出
nvalue	int	value数组元素个数，目前要求为1。	输入

返回值

返回值	类型	描述
KMLSS_NO_ERROR	int	正常执行。
KMLSS_NULL_ARGUMENT	int	pHandle或data为空指针。
KMLSS_BAD_HANDLE	int	pHandle格式存在问题。
KMLSS_BAD_DATA_SIZE	int	用户设定的value数组长度nvalue不合理，即不为1。
KMLSS_BAD_SELECTOR	int	用户的KML_SOLVER_PARAM输入当前不支持，或在求解过程中不存在。

版权所有 © 华为技术有限公司

依赖

#include "kml_dss.h"或#include "kml_solver.h"

示例

C Interface：

```
int n = 8;
int ia[9] = {0, 2, 4, 6, 7, 8, 10, 12, 14};
int ja[14] = {0, 7, 1, 6, 2, 5, 3, 4, 2, 5, 1, 6, 0, 7};
double a[14] = {1.0, 2.0, -2.0, 3.0, 3.0, 4.0, -4.0, 5.0, 4.0, -6.0, 3.0, 7.0, 2.0, 8.0};

double b[8] = {3.0, 1.0, 7.0, -4.0, 5.0, -2.0, 10.0, 10.0};
double x[8];
int nrhs = 1;
int ldx = n, ldb = n;


KmlSolverTask *handle;
int ierr = KmlDssGenInitDI(&handle, n, a, ja, ia);
if (ierr != 0) {
    printf("\nERROR in KmlDssGenInitDI: %d", ierr);
}
ierr = KmlDssGenAnalyzeDI(&handle);
if (ierr != 0) {
    printf("\nERROR in KmlDssGenAnalyzeDI: %d", ierr);
}
ierr = KmlDssGenFactorizeDI(&handle);
if (ierr != 0) {
    printf("\nERROR in KmlDssGenFactorizeDI: %d", ierr);
}

double tolerance = 1.e-12;
ierr = KmlDssGenSetDID(&handle, KMLSS_REFINEMENT_TOLERANCE_LEVEL, &tolerance, 1);
if (ierr != 0) {
    printf("\nERROR in KmlDssGenSetDID: %d", ierr);
}
int nsteps = 2;
ierr = KmlDssGenSetDII(&handle, KMLSS_REFINEMENT_MAX_STEPS, &nsteps, 1);
if (ierr != 0) {
    printf("\nERROR in KmlDssGenSetDII: %d", ierr);
}
ierr = KmlDssGenSolveDI(&handle, nrhs, x, ldx, b, ldb);
if (ierr != 0) {
    printf("\nERROR in KmlDssGenSolveDI: %d", ierr);
}

int nstep;
ierr = KmlDssGenGetDII(&handle, KMLSS_REFINEMENT_STEPS, &nstep, 1);
if (ierr != 0) {
    printf("\nERROR in KmlDssGenGetDII: %d", ierr);
}
printf("REFINEMENT_STEP = %d\n", nstep);
```

运行结果：

```
REFINEMENT_STEP = 1
```

6.5.3.3.9.7 KmlDssGenGet?I(D/S)

获取与handle关联的实数型值。

接口定义

C Interface：

 版权所有 © 华为技术有限公司

int KmlDssGenGetSIS(KmlSolverTask **pHandle, enum KmlSolverParam param, float *value, int nvalue);

int KmlDssGenGetDID(KmlSolverTask **pHandle, enum KmlSolverParam param, double *value, int nvalue);

int KmlDssGenGetCIS(KmlSolverTask **pHandle, enum KmlSolverParam param, float *value, int nvalue);

int KmlDssGenGetZID(KmlSolverTask **pHandle, enum KmlSolverParam param, double *value, int nvalue);

参数

参数名	类型	描述	输入/输出
pHandle	KmlSolverTask **	求解器句柄，pHandle不可为空指针。	输入
param	enum KmlSolverParam	KMLSS_REFINEMENT_RESIDUAL表示迭代优化的残差。	输入
value	double *或float *	直接求解相关参数数组。	输入
nvalue	int	value数组元素个数。	输入

返回值

返回值	类型	描述
KMLSS_NO_ERROR	int	正常执行。
KMLSS_NULL_ARGUMENT	int	pHandle或data为空指针、或者参数不合理。
KMLSS_BAD_HANDLE	int	pHandle格式存在问题。
KMLSS_BAD_DATA_SIZE	int	用户设定的value数组长度nvalue不合理，即与右端项数目不一致。
KMLSS_BAD_SELECTOR	int	用户的KML_SOLVER_PARAM输入当前不支持，或在求解过程中不存在。

依赖

#include "kml_dss.h"或#include "kml_solver.h"

示例

C Interface：

```
int n = 8;
int ia[9] = {0, 2, 4, 6, 7, 8, 10, 12, 14};
```

 版权所有 © 华为技术有限公司

```
int ja[14] = {0, 7, 1, 6, 2, 5, 3, 4, 2, 5, 1, 6, 0, 7};
double a[14] = {1.0, 2.0, -2.0, 3.0, 3.0, 4.0, -4.0, 5.0, 4.0, -6.0, 3.0, 7.0, 2.0, 8.0};

double b[8] = {3.0, 1.0, 7.0, -4.0, 5.0, -2.0, 10.0, 10.0};
double x[8];
int nrhs = 1;
int ldx = n, ldb = n;

KmlSolverTask *handle;
int ierr = KmlDssGenInitDI(&handle, n, a, ja, ia);
if (ierr != 0) {
    printf("\nERROR in KmlDssGenInitDI: %d", ierr);
}
ierr = KmlDssGenAnalyzeDI(&handle);
if (ierr != 0) {
    printf("\nERROR in KmlDssGenAnalyzeDI: %d", ierr);
}
ierr = KmlDssGenFactorizeDI(&handle);
if (ierr != 0) {
    printf("\nERROR in KmlDssGenFactorizeDI: %d", ierr);
}

double tolerance = 1.e-12;
ierr = KmlDssGenSetDID(&handle, KMLSS_REFINEMENT_TOLERANCE_LEVEL, &tolerance, 1);
if (ierr != 0) {
    printf("\nERROR in KmlDssGenSetDID: %d", ierr);
}
int nsteps = 2;
ierr = KmlDssGenSetDII(&handle, KMLSS_REFINEMENT_MAX_STEPS, &nsteps, 1);
if (ierr != 0) {
    printf("\nERROR in KmlDssGenSetDII: %d", ierr);
}

ierr = KmlDssGenSolveDI(&handle, nrhs, x, ldx, b, ldb);
if (ierr != 0) {
    printf("\nERROR in KmlDssGenSolveDI: %d", ierr);
}
double res;
ierr = KmlDssGenGetDID(&handle, KMLSS_REFINEMENT_RESIDUAL, &res, 1);
if (ierr != 0) {
    printf("\nERROR in KmlDssGenGetDID: %d", ierr);
}
printf("REFINEMENT_RESIDUAL = %f\n", res);
```

运行结果：

```
REFINEMENT_RESIDUAL = 0.000000
```

6.5.3.3.9.8 KmlDssGenSet?II

设置与handle关联的整型值。

接口定义

C Interface：

int KmlDssGenSetSII(KmlSolverTask **pHandle, enum KmlSolverParam param, int *value, int nvalue);

int KmlDssGenSetDII(KmlSolverTask **pHandle, enum KmlSolverParam param, int *value, int nvalue);

int KmlDssGenSetCII(KmlSolverTask **pHandle, enum KmlSolverParam param, int *value, int nvalue);

int KmlDssGenSetZII(KmlSolverTask **pHandle, enum KmlSolverParam param, int *value, int nvalue);

 版权所有 © 华为技术有限公司

参数

参数名	类型	描述	输入/输出
pHandle	KmlSolverTask **	求解器句柄，pHandle不可为空指针。	输入
param	enum KmlSolverParam	• KMLSS_PERM表示用户给定的矩阵排序。 • KMLSS_REFINEMENT_MAX_STEPS表示最大的迭代优化次数。 • KMLSS_BOOST_THRESHOLD表示boost阈值。 • KMLSS_SCALING_TYPE表示scaling算法的类型。 • KMLSS_MATCHING_TYPE表示matching算法的类型。 • KMLSS_REFINEMENT_TOLERANCE_LEVEL表示迭代优化算法设置的阈值，取10的负幂次。	输入
value	int*	直接求解相关参数数组。	输入
nvalue	int	value数组元素个数。	输入

对应不同的param参数，value有不同的取值，特别是可以取一些常量。在此一一列举。

param参数	值	功能
KMLSS_PERM	一维数组，数组长度与矩阵维度一致。	用户自定义行/列排序。
KMLSS_REFINEMENT_MAX_STEPS	长度为1的正整数数组或正整数的地址。	迭代优化的最大次数。
KMLSS_BOOST_THRESHOLD	长度为1的正整数数组或正整数的地址。	控制solver内部的boost参数。
KMLSS_SCALING_TYPE	KMLSS_NO_SCALING	无scaling操作。
	KMLSS_ROWCOLUMN_SCALING	执行行/列的scaling操作。
	KMLSS_MATCHBASED_SCALING	执行与Match算法相关的scaling操作。
KMLSS_MATCHING_TYPE	KMLSS_MATCHING_OFF	关闭Match算法。
	KMLSS_MATCHING_HUNGARIAN	匈牙利算法。

 版权所有 © 华为技术有限公司

param参数	值	功能
KMLSS_REFINEMENT_TOLERANCE_LEVEL	长度为1的正整数数组或正整数的地址。	迭代优化算法设置的阈值取10的负幂次。

返回值

返回值	类型	描述
KMLSS_NO_ERROR	int	正常执行。
KMLSS_NULL_ARGUMENT	int	handle或data为空指针
KMLSS_BAD_HANDLE	int	pHandle格式存在问题。
KMLSS_BAD_SELECTOR	int	用户的KML_SOLVER_PARAM输入当前不支持，或在求解过程中不存在。
KMLSS_BAD_DATA	int	用户的取值不符合要求。

依赖

#include "kml_dss.h"或#include "kml_solver.h"

示例

C Interface：

```
int n = 8;
int ia[9] = {0, 2, 4, 6, 7, 8, 10, 12, 14};
int ja[14] = {0, 7, 1, 6, 2, 5, 3, 4, 2, 5, 1, 6, 0, 7};
double a[14] = {1.0, 2.0, -2.0, 3.0, 3.0, 4.0, -4.0, 5.0, 4.0, -6.0, 3.0, 7.0, 2.0, 8.0};

double b[8] = {3.0, 1.0, 7.0, -4.0, 5.0, -2.0, 10.0, 10.0};
double x[8];
int nrhs = 1;
int ldx = n, ldb = n;

KmlSolverTask *handle;
int ierr = KmlDssGenInitDI(&handle, n, a, ja, ia);
if (ierr != 0) {
    printf("\nERROR in KmlDssGenInitDI: %d", ierr);
}
ierr = KmlDssGenAnalyzeDI(&handle);
if (ierr != 0) {
    printf("\nERROR in KmlDssGenAnalyzeDI: %d", ierr);
}
ierr = KmlDssGenFactorizeDI(&handle);
if (ierr != 0) {
    printf("\nERROR in KmlDssGenFactorizeDI: %d", ierr);
}

double tolerance = 1.e-12;
ierr = KmlDssGenSetDID(&handle, KMLSS_REFINEMENT_TOLERANCE_LEVEL, &tolerance, 1);
if (ierr != 0) {
    printf("\nERROR in KmlDssGenSetDID: %d", ierr);
```

 版权所有 © 华为技术有限公司

```
}
int nsteps = 2;
ierr = KmlDssGenSetDII(&handle, KMLSS_REFINEMENT_MAX_STEPS, &nsteps, 1);
if (ierr != 0) {
    printf("\nERROR in KmlDssGenSetDII: %d", ierr);
}

ierr = KmlDssGenSolveDI(&handle, nrhs, x, ldx, b, ldb);
if (ierr != 0) {
    printf("\nERROR in KmlDssGenSolveDI: %d", ierr);
}
for (int i = 0; i < n; i++) {
    printf("%lf ", x[i]);
}
printf("\n");
```

运行结果：

```
1.000000 1.000000 1.000000 1.000000 1.000000 1.000000 1.000000 1.000000
```

6.5.3.3.9.9 KmlDssGenSet?I(D/S)

设置与handle关联的实数型值。

接口定义

C Interface：

int KmlDssGenSetSIS(KmlSolverTask **pHandle, enum KmlSolverParam param, float *value, int nvalue);

int KmlDssGenSetDID(KmlSolverTask **pHandle, enum KmlSolverParam param, double *value, int nvalue);

int KmlDssGenSetCIS(KmlSolverTask **pHandle, enum KmlSolverParam param, float *value, int nvalue);

int KmlDssGenSetZID(KmlSolverTask **pHandle, enum KmlSolverParam param, double *value, int nvalue);

参数

参数名	类型	描述	输入/输出
pHandle	KmlSolverTask **	求解器句柄，pHandle不可为空指针。	输入
param	enum KmlSolverParam	• KMLSS_REFINEMENT_TOLERANCE_LEVEL表示迭代优化算法设置的阈值。 • KMLSS_PIVOTING_THRESHOLD表示迭代pivot算法设置的阈值，对应的value即为取值。	输入
value	double *或float *	直接求解相关参数数组。	输入
nvalue	int	value数组元素个数。	输入

版权所有 © 华为技术有限公司

返回值

返回值	类型	描述
KMLSS_NO_ERROR	int	正常执行。
KMLSS_BAD_SELECTOR	int	用户的KML_SOLVER_PARAM输入当前不支持，或在求解过程中不存在。

依赖

#include "kml_dss.h"或#include "kml_solver.h"

示例

C Interface：

```
int n = 8;
int ia[9] = {0, 2, 4, 6, 7, 8, 10, 12, 14};
int ja[14] = {0, 7, 1, 6, 2, 5, 3, 4, 2, 5, 1, 6, 0, 7};
double a[14] = {1.0, 2.0, -2.0, 3.0, 3.0, 4.0, -4.0, 5.0, 4.0, -6.0, 3.0, 7.0, 2.0, 8.0};

double b[8] = {3.0, 1.0, 7.0, -4.0, 5.0, -2.0, 10.0, 10.0};
double x[8];
int nrhs = 1;
int ldx = n, ldb = n;

KmlSolverTask *handle;
int ierr = KmlDssGenInitDI(&handle, n, a, ja, ia);
if (ierr != 0) {
    printf("\nERROR in KmlDssGenInitDI: %d", ierr);
}
ierr = KmlDssGenAnalyzeDI(&handle);
if (ierr != 0) {
    printf("\nERROR in KmlDssGenAnalyzeDI: %d", ierr);
}
ierr = KmlDssGenFactorizeDI(&handle);
if (ierr != 0) {
    printf("\nERROR in KmlDssGenFactorizeDI: %d", ierr);
}

double tolerance = 1.e-12;
ierr = KmlDssGenSetDID(&handle, KMLSS_REFINEMENT_TOLERANCE_LEVEL, &tolerance, 1);
if (ierr != 0) {
    printf("\nERROR in KmlDssGenSetDID: %d", ierr);
}
int nsteps = 2;
ierr = KmlDssGenSetDII(&handle, KMLSS_REFINEMENT_MAX_STEPS, &nsteps, 1);
if (ierr != 0) {
    printf("\nERROR in KmlDssGenSetDII: %d", ierr);
}

ierr = KmlDssGenSolveDI(&handle, nrhs, x, ldx, b, ldb);
if (ierr != 0) {
    printf("\nERROR in KmlDssGenSolveDI: %d", ierr);
}
for (int i = 0; i < n; i++) {
    printf("%lf ", x[i]);
}
printf("\n");
```

运行结果：

```
1.000000 1.000000 1.000000 1.000000 1.000000 1.000000 1.000000 1.000000
```

 版权所有 © 华为技术有限公司

6.5.3.3.9.10 KmlDssGenClean?I

释放内部数据结构，将Handle指向空指针。

接口定义

C Interface：

int KmlDssGenCleanSI(KmlSolverTask **pHandle);

int KmlDssGenCleanDI(KmlSolverTask **pHandle);

int KmlDssGenCleanCI(KmlSolverTask **pHandle);

int KmlDssGenCleanZI(KmlSolverTask **pHandle);

参数

参数名	类型	描述	输入/输出
pHandle	KmlSolverTask **	求解器句柄，传入之前步骤的变量。	输入/输出

返回值

返回值	类型	描述
KMLSS_NO_ERROR	int	正常执行。
KMLSS_NULL_ARGUMENT	int	无效的参数。
KMLSS_BAD_HANDLE	int	pHandle格式存在问题。

依赖

#include "kml_dss.h"或#include "kml_solver.h"

示例

C Interface：

```
int n = 8;
int ia[9] = {0, 2, 4, 6, 8, 10, 12, 13, 14};
int ja[14] = {0, 2, 1, 3, 2, 4, 3, 5, 4, 6, 5, 7, 6, 7};
double a[14] = {1.0, 2.0, 3.0, 4.0, 5.0, 6.0, 7.0, 8.0, 9.0, 10.0, 11.0, 12.0, 13.0, 14.0};
double b[8] = {3.0, 7.0, 13.0, 19.0, 25.0, 31.0, 23.0, 26.0};
double x[8];
KmlSolverTask* matrix_handle = NULL;
int ierr = KmlDssGenInitDI(&matrix_handle, n, a, ja, ia);
if (ierr != 0) {
    printf("\nERROR in KmlDssGenInitDI: %d", ierr);
}
```


版权所有 © 华为技术有限公司

```
ierr = KmlDssGenAnalyzeDI(&matrix_handle);
if (ierr != 0) {
    printf("\nERROR in KmlDssGenAnalyzeDI: %d", ierr);
}
ierr = KmlDssGenFactorizeDI(&matrix_handle);
if (ierr != 0) {
    printf("\nERROR in KmlDssGenFactorizeDI: %d", ierr);
}
ierr = KmlDssGenSolveDI(&matrix_handle, nrhs, x, ldx, b, ldb);
if (ierr != 0) {
    printf("\nERROR in KmlDssGenSolveDI: %d", ierr);
}
ierr = KmlDssGenCleanDI(&matrix_handle);
if (ierr != 0) {
    printf("\nERROR in KmlDssGenCleanDI: %d", ierr);
}
```

6.5.3.3.10 LLT/LLH (DEPRECATED)

直接法推荐使用**DSS**。

6.5.3.3.10.1 KmlDss(Spd/Hpd)Init?I

初始化数据结构，并将用户提供的系数矩阵关联到求解系统中，初始化LLT\LLH方法时使用上三角的CSR格式。

接口定义

C Interface：

int KmlDssSpdInitSI(KmlSolverTask **pHandle, int n, float *a, int *ja, int *ia);

int KmlDssSpdInitDI(KmlSolverTask **pHandle, int n, double *a, int *ja, int *ia);

int KmlDssHpdInitCI(KmlSolverTask **pHandle, int n, kml_complex_float *a, int *ja, int *ia);

int KmlDssHpdInitZI(KmlSolverTask **pHandle, int n, kml_complex_double *a, int *ja, int *ia);

参数

参数名	类型	描述	输入/输出
pHandle	KmlSolverTask **	求解器句柄，pHandle不可为空指针。	输出
n	int	n为矩阵A的维度。	输入

版权所有 © 华为技术有限公司

参数名	类型	描述	输入/输出
a	• 在KmlDssSpdInitSI中为float *。 • 在KmlDssSpdInitDI中为double *。 • 在KmlDssHpdInitCI中为kml_complex_float *。 • 在KmlDssHpdInitZI中为kml_complex_double *。	a为维度为非零元素的个数的数组，a[i]为第i个非零元的值。该数组的生命周期与pHandle相同。	输入
ja	int	ja为维度为非零元素的个数的数组，ja[i]为第i个非零元的列号。该数组的生命周期与pHandle相同。	输入
ia	int	ia为n+1维数组，ia[i]为第i行第一个非零元在所有非零元中的索引。该数组的生命周期与pHandle相同。	输入

返回值

返回值	类型	描述
KMLSS_NO_ERROR	int	正常执行。
KMLSS_NULL_ARGUMENT	int	pHandle，a，ja，ia中有空指针。
KMLSS_BAD_N	int	矩阵维数输入值小于等于0。
KMLSS_NO_MEMORY	int	内存开辟失败。

依赖

#include "kml_dss.h"或#include "kml_solver.h"

示例

C Interface：

```
int n = 2;
int ia[3] = {0,2,4};
int ja[4] = {0,1,0,1};
double a[4] = {2,1,1,2};
KmlSolverTask *handle;
```

 版权所有 © 华为技术有限公司

```
int ierr = KmlDssSpdInitDI(&handle, n, a, ja, ia);
if (ierr != 0) {
      printf("\nERROR in KmlDssSpdInitDI: %d", ierr);
}
```

6.5.3.3.10.2 KmlDss(Spd/Hpd)Analyze?I

分析数据结构，对稀疏矩阵进行重排和符号分解。

接口定义

C Interface：

int KmlDssSpdAnalyzeSI(KmlSolverTask **pHandle);

int KmlDssSpdAnalyzeDI(KmlSolverTask **pHandle);

int KmlDssHpdAnalyzeCI(KmlSolverTask **pHandle);

int KmlDssHpdAnalyzeZI(KmlSolverTask **pHandle);

参数

参数名	类型	描述	输入/输出
pHandle	KmlSolverTask **	求解器句柄，传入之前步骤的变量。	输入

返回值

返回值	类型	描述
KMLSS_NO_ERROR	int	正常执行。
KMLSS_NULL_ARGUMENT	int	pHandle为空指针。
KMLSS_BAD_HANDLE	int	pHandle格式存在问题。
KMLSS_NONZERO_INDEXING	int	行索引指针不是从0开始，即ia[0]不等于0。
KMLSS_MISSING_DIAGONAL_ELEMENT	int	稀疏矩阵对角线元素全部为空。
KMLSS_INTERNAL_ERROR	int	稀疏矩阵对角线存在空元素/算法内部执行失败。
KMLSS_REORDERING_PROBLEM	int	排序算法失败。

依赖

#include "kml_dss.h"或#include "kml_solver.h"

版权所有 © 华为技术有限公司

示例

C Interface：

```
int n = 2;
int ia[3] = {0,2,4};
int ja[4] = {0,1,0,1};
double a[4] = {2,1,1,2};

KmlSolverTask *handle;
int ierr = KmlDssSpdInitDI(&handle, n, a, ja, ia);
if (ierr != 0) {
	printf("\nERROR in KmlDssSpdInitDI: %d", ierr);
}
ierr = KmlDssSpdAnalyzeDI(&handle);
if (ierr != 0) {
	printf("\nERROR in KmlDssSpdAnalyzeDI: %d", ierr);
}
```

6.5.3.3.10.3 KmlDss(Spd/Hpd)Factorize?I

实现对称正定矩阵的LLT分解或者Hermitian正定矩阵的LLH分解。

接口定义

C Interface：

int KmlDssSpdFactorizeSI(KmlSolverTask **pHandle);

int KmlDssSpdFactorizeDI(KmlSolverTask **pHandle);

int KmlDssHpdFactorizeCI(KmlSolverTask **pHandle);

int KmlDssHpdFactorizeZI(KmlSolverTask **pHandle);

参数

参数名	类型	描述	输入/输出
pHandle	KmlSolverTask **	求解器句柄，传入之前步骤的变量。	输入

返回值

返回值	类型	描述
KMLSS_NO_ERROR	int	正常执行。
KMLSS_NULL_ARGUMENT	int	pHandle为空指针。
KMLSS_BAD_HANDLE	int	参数无效或格式存在问题。
KMLSS_INVALID_CALL_ORDER	int	错误的调用顺序，之前未进入分解阶段。

版权所有 © 华为技术有限公司

返回值	类型	描述
KMLSS_NO_MEMORY	int	开辟内存失败。
KMLSS_INTERNAL_ERROR	int	算法内部执行失败。

依赖

#include "kml_dss.h"或#include "kml_solver.h"

示例

C Interface：

```
int n = 2;
int ia[3] = {0,2,4};
int ja[4] = {0,1,0,1};
double a[4] = {2,1,1,2};
double x[2];

KmlSolverTask *handle;
int ierr = KmlDssSpdInitDI(&handle, n, a, ja, ia);
if (ierr != 0) {
    printf("\nERROR in KmlDssSpdInitDI: %d", ierr);
}
ierr = KmlDssSpdAnalyzeDI(&handle);
if (ierr != 0) {
    printf("\nERROR in KmlDssSpdAnalyzeDI: %d", ierr);
}
ierr = KmlDssSpdFactorizeDI(&handle);
if (ierr != 0) {
    printf("\nERROR in KmlDssSpdFactorizeDI: %d", ierr);
}
for (int i = 0; i < n; i++) {
    printf("%lf ", x[i]);
}
```

6.5.3.3.10.4 KmlDss(Spd/Hpd)Solve?I

求解$LL^{T}x=b$或者$LL^{H}x=b$。

接口定义

C Interface：

int KmlDssSpdSolveSI(KmlSolverTask **pHandle, int nb, float *x, int ldx, float *b, int ldb);

int KmlDssSpdSolveDI(KmlSolverTask **pHandle, int nb, double *x, int ldx, double *b, int ldb);

int KmlDssHpdSolveCI(KmlSolverTask **pHandle, int nb, kml_complex_float *x, int ldx, kml_complex_float *b, int ldb);

int KmlDssHpdSolveZI(KmlSolverTask **pHandle, int nb, kml_complex_double *x, int ldx, kml_complex_double *b, int ldb);

 版权所有 © 华为技术有限公司

参数

参数名	类型	描述	输入/输出
pHandle	KmlSolverTask **	求解器句柄，传入之前步骤的变量。	输入
nb	int	右端项矢量的个数。	输入
x	• 在KmlDssSpdSolveSI中为float *。 • 在KmlDssSpdSolveDI中为double *。 • 在KmlDssHpdSolveCI中为kml_complex_float *。 • 在KmlDssHpdSolveZI中为kml_complex_double *。	用于存储求解结果的稠密向量，其元素个数为ldx*nb，第k个矢量的第i个元素位置为x[i + k * ldx]。	输出
ldx	int	x的leading dimension大小，要求ldx ≥ max(1, n)。	输入
b	• 在KmlDssSpdSolveSI中为float *。 • 在KmlDssSpdSolveDI中为double *。 • 在KmlDssHpdSolveCI中为kml_complex_float *。 • 在KmlDssHpdSolveZI中为kml_complex_double *。	稠密向量存储右端项，其元素个数为ldb*nb，第k个矢量的第i个元素位置为b[i + k * ldb]。	输入
ldb	int	b的leading dimension大小，要求ldb ≥ max(1, n)。	输入

版权所有 © 华为技术有限公司

返回值

返回值	类型	描述
KMLSS_NO_ERROR	int	正常执行。
KMLSS_NULL_ARGUMENT	int	pHandle，x，b为空。
KMLSS_BAD_NB	int	输入参数nb小于等于0。
KMLSS_BAD_HANDLE	int	pHandle格式存在问题。
KMLSS_BAD_LDX	int	ldx取值不在有效范围内。
KMLSS_BAD_LDB	int	ldb取值不在有效范围内。
KMLSS_INVALID_CALL_ORDER	int	函数执行顺序错误，尚未执行分解阶段就提前执行求解函数。
KMLSS_NO_MEMORY	int	开辟内存失败。

依赖

#include "kml_dss.h"或#include "kml_solver.h"

示例

C Interface：

```
int n = 2;
int ia[3] = {0,2,4};
int ja[4] = {0,1,0,1};
double a[4] = {2,1,1,2};
double b[2] = {3,1};
double x[2];

int nrhs = 1;
int ldx = n, ldb = n;

KmlSolverTask *handle;
int ierr = KmlDssSpdInitDI(&handle, n, a, ja, ia);
if (ierr != 0) {
	printf("\nERROR in KmlDssSpdInitDI: %d", ierr);
}
ierr = KmlDssSpdAnalyzeDI(&handle);
if (ierr != 0) {
	printf("\nERROR in KmlDssSpdAnalyzeDI: %d", ierr);
}
ierr = KmlDssSpdFactorizeDI(&handle);
if (ierr != 0) {
	printf("\nERROR in KmlDssSpdFactorizeDI: %d", ierr);
}
ierr = KmlDssSpdSolveDI(&handle, nrhs, x, ldx, b, ldb);
if (ierr != 0) {
	printf("\nERROR in KmlDssSpdSolveDI: %d", ierr);
}
for (int i = 0; i < n; i++) {
	printf("%lf ", x[i]);
```

 版权所有 © 华为技术有限公司

```
}
printf("\n");
```

运行结果：

```
1.666667 -0.333333
```

6.5.3.3.10.5 KmlDss(Spd/Hpd)Get?IL

获取与handle关联的长整型值。

接口定义

C Interface：

int KmlDssSpdGetSIL(KmlSolverTask **pHandle, enum KmlSolverParam param, int64_t *value, int nvalue);

int KmlDssSpdGetDIL(KmlSolverTask **pHandle, enum KmlSolverParam param, int64_t *value, int nvalue);

int KmlDssHpdGetCIL(KmlSolverTask **pHandle, enum KmlSolverParam param, int64_t *value, int nvalue);

int KmlDssHpdGetZIL(KmlSolverTask **pHandle, enum KmlSolverParam param, int64_t *value, int nvalue);

参数

参数名	类型	描述	输入/输出
pHandle	KmlSolverTask **	求解器句柄，pHandle不可为空指针。	输入
param	enum KmlSolverParam	KMLSS_FILL_IN表示填入数目。	输入
value	int64_t *	直接求解相关参数数组。	输出
nvalue	int	value数组元素个数，目前要求为1。	输入

返回值

返回值	类型	描述
KMLSS_NO_ERROR	int	正常执行。
KMLSS_NULL_ARGUMENT	int	pHandle或data为空指针。
KMLSS_BAD_HANDLE	int	pHandle格式存在问题。
KMLSS_BAD_DATA_SIZE	int	用户设定的value数组长度nvalue不合理。

版权所有 © 华为技术有限公司

返回值	类型	描述
KMLSS_INTERNAL_ERROR	int	pHandle状态异常，KMLSS_FILL_IN获取失败。
KMLSS_BAD_SELECTOR	int	用户的KML_SOLVER_PARAM输入当前不支持，或在求解过程中不存在。

依赖

#include "kml_dss.h"或#include "kml_solver.h"

示例

C Interface：

```
int n = 2;
int ia[3] = {0,2,4};
int ja[4] = {0,1,0,1};
double a[4] = {2,1,1,2};
double b[2] = {3,1};
double x[2];

int nrhs = 1;
int ldx = n, ldb = n;

KmlSolverTask *handle;
int ierr = KmlDssSpdInitDI(&handle, n, a, ja, ia);
if (ierr != 0) {
	printf("\nERROR in KmlDssSpdInitDI: %d", ierr);
}
ierr = KmlDssSpdAnalyzeDI(&handle);
if (ierr != 0) {
	printf("\nERROR in KmlDssSpdAnalyzeDI: %d", ierr);
}
ierr = KmlDssSpdFactorizeDI(&handle);
if (ierr != 0) {
	printf("\nERROR in KmlDssSpdFactorizeDI: %d", ierr);
}
ierr = KmlDssSpdSolveDI(&handle, nrhs, x, ldx, b, ldb);
if (ierr != 0) {
	printf("\nERROR in KmlDssSpdSolveDI: %d", ierr);
}

int64_t nnz;
ierr = KmlDssSpdGetDIL(&handle, KMLSS_FILL_IN, &nnz, 1);
if (ierr != 0) {
	printf("\nERROR in KmlDssSpdGetDIL: %d", ierr);
}
printf("NNZ = %lld\n", nnz);
```

运行结果：

```
NNZ = 4
```

6.5.3.3.10.6 KmlDss(Spd/Hpd)Get?Il

获取与handle关联的整型值。

接口定义

C Interface：

 版权所有 © 华为技术有限公司

int KmlDssSpdGetSII(KmlSolverTask **pHandle, enum KmlSolverParam param, int *value, int nvalue);

int KmlDssSpdGetDII(KmlSolverTask **pHandle, enum KmlSolverParam param, int *value, int nvalue);

int KmlDssHpdGetCII(KmlSolverTask **pHandle, enum KmlSolverParam param, int *value, int nvalue);

int KmlDssHpdGetZII(KmlSolverTask **pHandle, enum KmlSolverParam param, int *value, int nvalue);

参数

参数名	类型	描述	输入/输出
pHandle	KmlSolverTask **	求解器句柄，pHandle不可为空指针。	输入
param	enum KmlSolverParam	当前仅支持KMLSS_REFINEMENT_STEPS，表示迭代优化次数。	输入
value	int *	直接求解相关参数数组。	输出
nvalue	int	value数组元素个数，目前要求为1。	输入

返回值

返回值	类型	描述
KMLSS_NO_ERROR	int	正常执行。
KMLSS_NULL_ARGUMENT	int	pHandle或data为空指针。
KMLSS_BAD_HANDLE	int	pHandle格式存在问题。
KMLSS_BAD_DATA_SIZE	int	用户设定的value数组长度nvalue不合理，即不为1。
KMLSS_BAD_SELECTOR	int	用户的KML_SOLVER_PARAM输入当前不支持，或在求解过程中不存在。

依赖

#include "kml_dss.h"或#include "kml_solver.h"

示例

C Interface：

 版权所有 © 华为技术有限公司

```
int n = 2;
int ia[3] = {0,2,4};
int ja[4] = {0,1,0,1};
double a[4] = {2,1,1,2};
double b[2] = {3,1};
double x[2];

int nrhs = 1;
int ldx = n, ldb = n;

KmlSolverTask *handle;
int ierr = KmlDssSpdInitDI(&handle, n, a, ja, ia);
if (ierr != 0) {
	printf("\nERROR in KmlDssSpdInitDI: %d", ierr);
}
ierr = KmlDssSpdAnalyzeDI(&handle);
if (ierr != 0) {
	printf("\nERROR in KmlDssSpdAnalyzeDI: %d", ierr);
}
ierr = KmlDssSpdFactorizeDI(&handle);
if (ierr != 0) {
	printf("\nERROR in KmlDssSpdFactorizeDI: %d", ierr);
}

double tolerance = 1.e-12;
ierr = KmlDssSpdSetDID(&handle, KMLSS_REFINEMENT_TOLERANCE_LEVEL, &tolerance, 1);
if (ierr != 0) {
	printf("\nERROR in KmlDssSpdSetDID: %d", ierr);
}
int nsteps = 2;
ierr = KmlDssSpdSetDII(&handle, KMLSS_REFINEMENT_MAX_STEPS, &nsteps, 1);
if (ierr != 0) {
	printf("\nERROR in KmlDssSpdSetDII: %d", ierr);
}

ierr = KmlDssSpdSolveDI(&handle, nrhs, x, ldx, b, ldb);
if (ierr != 0) {
	printf("\nERROR in KmlDssSpdSolveDI: %d", ierr);
}

printf("\n");
int nstep;
ierr = KmlDssSpdGetDII(&handle, KMLSS_REFINEMENT_STEPS, &nstep, 1);
if (ierr != 0) {
	printf("\nERROR in KmlDssSpdGetDII: %d", ierr);
}
printf("REFINEMENT_STEP = %d\n", nstep);
```

运行结果：

```
REFINEMENT_STEP = 1
```

6.5.3.3.10.7 KmlDss(Spd/Hpd)Get?I(D/S)

获取与handle关联的实数型值。

接口定义

C Interface：

int KmlDssSpdGetSIS(KmlSolverTask **pHandle, enum KmlSolverParam param, float *value, int nvalue);

int KmlDssSpdGetDID(KmlSolverTask **pHandle, enum KmlSolverParam param, double *value, int nvalue);

int KmlDssHpdGetCIS(KmlSolverTask **pHandle, enum KmlSolverParam param, float *value, int nvalue);

 版权所有 © 华为技术有限公司

int KmlDssHpdGetZID(KmlSolverTask **pHandle, enum KmlSolverParam param, double *value, int nvalue);

参数

参数名	类型	描述	输入/输出
pHandle	KmlSolverTask **	求解器句柄，pHandle不可为空指针。	输入
param	enum KmlSolverParam	当前仅支持KMLSS_REFINEMENT_RESIDUAL，表示迭代优化的残差。	输入
value	double *或float *	直接求解相关参数数组。	输出
nvalue	int	value数组元素个数，要求与右端项数目nb一致。	输入

返回值

返回值	类型	描述
KMLSS_NO_ERROR	int	正常执行。
KMLSS_NULL_ARGUMENT	int	pHandle或data为空指针、或者参数不合理。
KMLSS_BAD_HANDLE	int	pHandle格式存在问题。
KMLSS_BAD_DATA_SIZE	int	用户设定的value数组长度nvalue不合理，即与右端项数目不一致。
KMLSS_BAD_SELECTOR	int	用户的KML_SOLVER_PARAM输入当前不支持，或在求解过程中不存在。

依赖

#include "kml_dss.h"或#include "kml_solver.h"

示例

C Interface：

```
int n = 2;
int ia[3] = {0,2,4};
int ja[4] = {0,1,0,1};
double a[4] = {2,1,1,2};
double b[2] = {3,1};
double x[2];

int nrhs = 1;
```


版权所有 © 华为技术有限公司

```
int ldx = n, ldb = n;

KmlSolverTask *handle;
int ierr = KmlDssSpdInitDI(&handle, n, a, ja, ia);
if (ierr != 0) {
	printf("\nERROR in KmlDssSpdInitDI: %d", ierr);
}
ierr = KmlDssSpdAnalyzeDI(&handle);
if (ierr != 0) {
	printf("\nERROR in KmlDssSpdAnalyzeDI: %d", ierr);
}
ierr = KmlDssSpdFactorizeDI(&handle);
if (ierr != 0) {
	printf("\nERROR in KmlDssSpdFactorizeDI: %d", ierr);
}

double tolerance = 1.e-12;
ierr = KmlDssSpdSetDID(&handle, KMLSS_REFINEMENT_TOLERANCE_LEVEL, &tolerance, 1);
if (ierr != 0) {
	printf("\nERROR in KmlDssSpdSetDID: %d", ierr);
}
int nsteps = 2;
ierr = KmlDssSpdSetDII(&handle, KMLSS_REFINEMENT_MAX_STEPS, &nsteps, 1);
if (ierr != 0) {
	printf("\nERROR in KmlDssSpdSetDII: %d", ierr);
}
ierr = KmlDssSpdSolveDI(&handle, nrhs, x, ldx, b, ldb);
if (ierr != 0) {
	printf("\nERROR in KmlDssSpdSolveDI: %d", ierr);
}

double res;
ierr = KmlDssSpdGetDID(&handle, KMLSS_REFINEMENT_RESIDUAL, &res, 1);
if (ierr != 0) {
	printf("\nERROR in KmlDssSpdGetDID: %d", ierr);
}
printf("REFINEMENT_RESIDUAL = %f\n", res);
```

运行结果：

```
REFINEMENT_RESIDUAL = 0.000000
```

6.5.3.3.10.8 KmlDss(Spd/Hpd)Set?Il

设置与handle关联的值。

C Interface：

int KmlDssSpdSetSII(KmlSolverTask **pHandle, enum KmlSolverParam param, int *value, int nvalue);

int KmlDssSpdSetDII(KmlSolverTask **pHandle, enum KmlSolverParam param, int *value, int nvalue);

int KmlDssHpdSetCII(KmlSolverTask **pHandle, enum KmlSolverParam param, int *value, int nvalue);

int KmlDssHpdSetZII(KmlSolverTask **pHandle, enum KmlSolverParam param, int *value, int nvalue);

 版权所有 © 华为技术有限公司

参数

参数名	类型	描述	输入/输出
pHandle	KmlSolverTask **	求解器句柄，pHandle不可为空指针。	输入
param	enum KmlSolverParam	• KMLSS_PERM表示用户给定的矩阵排序。 • KMLSS_REFINEMENT_MAX_STEPS表示最大的迭代优化次数。 • KMLSS_BOOST_THRESHOLD表示boost阈值。 • KMLSS_SCALING_TYPE表示scaling算法的类型。 • KMLSS_MATCHING_TYPE表示matching算法的类型。 • KMLSS_REFINEMENT_TOLERANCE_LEVEL表示迭代优化算法设置的阈值，取10的负幂次。	输入
value	int *	直接求解相关参数数组。	输入
nvalue	int	value数组元素个数。	输入

对应不同的param参数，value有不同的取值，特别是可以取一些常量。在此一一列举。

param参数	值	功能
KMLSS_PERM	一维数组，数组长度与矩阵维度一致。	用户自定义行/列排序
KMLSS_REFINEMENT_MAX_STEPS	长度为1的正整数数组或正整数的地址。	迭代优化的最大次数
KMLSS_BOOST_THRESHOLD	长度为1的正整数数组或正整数的地址。	控制solver内部的boost参数
KMLSS_SCALING_TYPE	KMLSS_NO_SCALING	无scaling操作
	KMLSS_ROWCOLUMN_SCALING	执行行/列的scaling操作
	KMLSS_MATCHBASED_SCALING	执行与Match算法相关的scaling操作
KMLSS_MATCHING_TYPE	KMLSS_MATCHING_OFF	关闭Match算法
	KMLSS_MATCHING_HUNGARIAN	匈牙利算法

 版权所有 © 华为技术有限公司

param参数	值	功能
KMLSS_REFINEMENT_TOLERANCE_LEVEL	长度为1的正整数数组或正整数的地址	迭代优化算法设置的阈值取10的负幂次

返回值

返回值	类型	描述
KMLSS_NO_ERROR	int	正常执行。
KMLSS_NULL_ARGUMENT	int	handle或data为空指针
KMLSS_BAD_HANDLE	int	pHandle格式存在问题。
KMLSS_BAD_SELECTOR	int	用户的KML_SOLVER_PARAM输入当前不支持，或在求解过程中不存在。
KMLSS_BAD_DATA	int	用户的取值不符合要求。

依赖

#include "kml_dss.h"或#include "kml_solver.h"

示例

C Interface：

```
int n = 2;
int ia[3] = {0,2,4};
int ja[4] = {0,1,0,1};
double a[4] = {2,1,1,2};
double b[2] = {3,1};
double x[2];

int nrhs = 1;
int ldx = n, ldb = n;

KmlSolverTask *handle;
int ierr = KmlDssSpdInitDI(&handle, n, a, ja, ia);
if (ierr != 0) {
	printf("\nERROR in KmlDssSpdInitDI: %d", ierr);
}
ierr = KmlDssSpdAnalyzeDI(&handle);
if (ierr != 0) {
	printf("\nERROR in KmlDssSpdAnalyzeDI: %d", ierr);
}
ierr = KmlDssSpdFactorizeDI(&handle);
if (ierr != 0) {
	printf("\nERROR in KmlDssSpdFactorizeDI: %d", ierr);
}

double tolerance = 1.e-12;
ierr = KmlDssSpdSetDID(&handle, KMLSS_REFINEMENT_TOLERANCE_LEVEL, &tolerance, 1);
if (ierr != 0) {
	printf("\nERROR in KmlDssSpdSetDID: %d", ierr);
}
```

 版权所有 © 华为技术有限公司

```
int nsteps = 2;
ierr = KmlDssSpdSetDII(&handle, KMLSS_REFINEMENT_MAX_STEPS, &nsteps, 1);
if (ierr != 0) {
	printf("\nERROR in KmlDssSpdSetDII: %d", ierr);
}

ierr = KmlDssSpdSolveDI(&handle, nrhs, x, ldx, b, ldb);
if (ierr != 0) {
	printf("\nERROR in KmlDssSpdSolveDI: %d", ierr);
}

int nstep;
ierr = KmlDssSpdGetDII(&handle, KMLSS_REFINEMENT_STEPS, &nstep, 1);
printf("REFINEMENT_STEP = %d\n", nstep);
```

运行结果：

```
REFINEMENT_STEP = 1
```

6.5.3.3.10.9 KmlDss(Spd/Hpd)Set?I(D/S)

设置与handle关联的实数型值。

接口定义

C Interface：

int KmlDssSpdSetDID(KmlSolverTask **pHandle, enum KmlSolverParam param, double *value, int nvalue);

int KmlDssHpdSetZID(KmlSolverTask **pHandle, enum KmlSolverParam param, double *value, int nvalue);

int KmlDssSpdSetSIS(KmlSolverTask **pHandle, enum KmlSolverParam param, float *value, int nvalue);

int KmlDssHpdSetCIS(KmlSolverTask **pHandle, enum KmlSolverParam param, float *value, int nvalue);

参数

参数名	类型	描述	输入/输出
pHandle	KmlSolverTask **	求解器句柄，pHandle不可为空指针。	输入
param	enum KmlSolverParam	• KMLSS_REFINEMENT_TOLERANCE_LEVEL表示迭代优化算法设置的阈值。 • KMLSS_PIVOTING_THRESHOLD表示迭代pivot算法设置的阈值，对应的value即为取值。	输入
value	double *或float *	直接求解相关参数数组。	输入
nvalue	int	value数组元素个数。	输入

版权所有 © 华为技术有限公司

返回值

返回值	类型	描述
KMLSS_NO_ERROR	int	正常执行。
KMLSS_BAD_SELECTOR	int	用户的KML_SOLVER_PARAM输入当前不支持，或在求解过程中不存在。

依赖

#include "kml_dss.h"或#include "kml_solver.h"

示例

C Interface：

```
int n = 2;
int ia[3] = {0,2,4};
int ja[4] = {0,1,0,1};
double a[4] = {2,1,1,2};
double b[2] = {3,1};
double x[2];

int nrhs = 1;
int ldx = n, ldb = n;

KmlSolverTask *handle;
int ierr = KmlDssSpdInitDI(&handle, n, a, ja, ia);
if (ierr != 0) {
	printf("\nERROR in KmlDssSpdInitDI: %d", ierr);
}
ierr = KmlDssSpdAnalyzeDI(&handle);
if (ierr != 0) {
	printf("\nERROR in KmlDssSpdAnalyzeDI: %d", ierr);
}
ierr = KmlDssSpdFactorizeDI(&handle);
if (ierr != 0) {
	printf("\nERROR in KmlDssSpdFactorizeDI: %d", ierr);
}

double tolerance = 1.e-12;
ierr = KmlDssSpdSetDID(&handle, KMLSS_REFINEMENT_TOLERANCE_LEVEL, &tolerance, 1);
if (ierr != 0) {
	printf("\nERROR in KmlDssSpdSetDID: %d", ierr);
}
int nsteps = 2;
ierr = KmlDssSpdSetDII(&handle, KMLSS_REFINEMENT_MAX_STEPS, &nsteps, 1);
if (ierr != 0) {
	printf("\nERROR in KmlDssSpdSetDII: %d", ierr);
}

ierr = KmlDssSpdSolveDI(&handle, nrhs, x, ldx, b, ldb);
if (ierr != 0) {
	printf("\nERROR in KmlDssSpdSolveDI: %d", ierr);
}

int nstep;
ierr = KmlDssSpdGetDII(&handle, KMLSS_REFINEMENT_STEPS, &nstep, 1);
printf("REFINEMENT_STEP = %d\n", nstep);
double res;
ierr = KmlDssSymGetDID(&handle, KMLSS_REFINEMENT_RESIDUAL, &res, 1);
printf("REFINEMENT_RESIDUAL = %f\n", res);
```

 版权所有 © 华为技术有限公司

运行结果：

```
REFINEMENT_STEP = 1
REFINEMENT_RESIDUAL = 0.000000
```

6.5.3.3.10.10 KmlDss(Spd/Hpd)Clean?I

释放内部数据结构，将Handle指向空指针。

接口定义

C Interface：

int KmlDssSpdCleanSI(KmlSolverTask **pHandle);

int KmlDssSpdCleanDI(KmlSolverTask **pHandle);

int KmlDssHpdCleanCI(KmlSolverTask **pHandle);

int KmlDssHpdCleanZI(KmlSolverTask **pHandle);

参数

参数名	类型	描述	输入/输出
pHandle	KmlSolverTask **	求解器句柄，传入之前步骤的变量。	输入/输出

返回值

返回值	类型	描述
KMLSS_NO_ERROR	int	正常执行。
KMLSS_NULL_ARGUMENT	int	无效的参数。
KMLSS_BAD_HANDLE	int	pHandle格式存在问题。

依赖

#include "kml_dss.h"或#include "kml_solver.h"

示例

C Interface：

```
int n = 2;
int ia[3] = {0,2,4};
int ja[4] = {0,1,0,1};
double a[4] = {2,1,1,2};
double b[2] = {3,1};
```


版权所有 © 华为技术有限公司

```
double x[2];
int nrhs = 1;
int ldx = n, ldb = n;
KmlSolverTask* matrix_handle = NULL;
int ierr = KmlDssSpdInitDI(&matrix_handle, n, a, ja, ia);
if (ierr != 0) {
    printf("\nERROR in KmlDssSpdInitDI: %d", ierr);
}
ierr = KmlDssSpdAnalyzeDI(&matrix_handle);
if (ierr != 0) {
    printf("\nERROR in KmlDssSpdAnalyzeDI: %d", ierr);
}
ierr = KmlDssSpdFactorizeDI(&matrix_handle);
if (ierr != 0) {
    printf("\nERROR in KmlDssSpdFactorizeDI: %d", ierr);
}
ierr = KmlDssSpdSolveDI(&matrix_handle, nrhs, x, ldx, b, ldb);
if (ierr != 0) {
    printf("\nERROR in KmlDssSpdSolveDI: %d", ierr);
}
ierr = KmlDssSpdCleanDI(&matrix_handle);
if (ierr != 0) {
    printf("\nERROR in KmlDssSpdCleanDI: %d", ierr);
}
```

6.5.3.3.11 DSS

6.5.3.3.11.1 矩阵函数

?.1. KmlSolverMatrixCreate

创建矩阵操作句柄。

说明

创建矩阵时是浅拷贝，用户需要保证在KmlSolverMatrixDestroy前不释放通过参数store指定的内存。

接口定义

C Interface：

int KmlSolverMatrixCreate(KmlSolverMatrix **pMatrix, const KmlSolverMatrixStore *store, const KmlSolverMatrixOption *matOption);

参数

参数名	类型	描述	输入/输出
pMatrix	KmlSolverMatrix **	矩阵句柄，pMatrix不可为空指针。	输入/输出
store	KmlSolverMatrixStore *	矩阵格式信息结构体，不可为空指针，具体内容如表6-45所示。	输入
matOption	KmlSolverMatrixOption *	矩阵配置选项，不可为空指针，具体内容如表6-46所示。	输入

版权所有 © 华为技术有限公司

表 6-45 KmlSolverMatrixStore 结构体字段描述

字段名	类型	描述
valueType	KmlSolverValueType	矩阵数值类型，有如下值。 • KMLSS_VALUE_FP32表示float。 • KMLSS_VALUE_FP64表示double。 • KMLSS_VALUE_FP32C表示complex float。 • KMLSS_VALUE_FP64C表示complex double。
indexType	KmlSolverIndexType	矩阵坐标索引类型。 • KMLSS_INDEX_INT32表示int32_t。 • KMLSS_INDEX_INT64表示int64_t。
nRow	int64_t	矩阵行数。
nCol	int64_t	矩阵列数。
format	KmlSolverMatrixStoreFormat	矩阵存储格式，有如下值。 • KMLSS_MATRIX_STORE_CSR表示CSR格式。 • KMLSS_MATRIX_STORE_CSR_L表示CSR格式，只有下三角部分。 • KMLSS_MATRIX_STORE_CSR_U表示CSR格式，只有上三角部分。 • KMLSS_MATRIX_STORE_CSC表示CSC格式。 • KMLSS_MATRIX_STORE_CSC_L表示CSC格式，只有下三角部分。 • KMLSS_MATRIX_STORE_CSC_U表示CSC格式，只有上三角部分。 • KMLSS_MATRIX_STORE_COO表示COO格式。 • KMLSS_MATRIX_STORE_COO_L表示COO格式，只有下三角部分。 • KMLSS_MATRIX_STORE_COO_U表示COO格式，只有上三角部分。 • KMLSS_MATRIX_STORE_DENSE_ROW_MAJOR表示行主序稠密存储。 • KMLSS_MATRIX_STORE_DENSE_COL_MAJOR表示列主序稠密存储。
csr	KmlSolverMatrixStoreCSR 该类型字段描述如表6-47所示。	format为KMLSS_MATRIX_STORE_CSR、KMLSS_MATRIX_STORE_CSR_L或KMLSS_MATRIX_STORE_CSR_U时，该字段有效。

版权所有 © 华为技术有限公司

字段名	类型	描述
csc	KmlSolverMatrixStoreCSC 该类型字段描述如表6-48所示。	format为KMLSS_MATRIX_STORE_CSC、KMLSS_MATRIX_STORE_CSC_L或KMLSS_MATRIX_STORE_CSC_U时，该字段有效。
coo	KmlSolverMatrixStoreCOO 该类型字段描述如表6-49所示。	format为KMLSS_MATRIX_STORE_COO、KMLSS_MATRIX_STORE_COO_L或KMLSS_MATRIX_STORE_COO_U时，该字段有效。
dense	KmlSolverMatrixStoreDense 该类型字段描述如表6-50所示。	format为KMLSS_MATRIX_STORE_DENSE_ROW_MAJOR或KMLSS_MATRIX_STORE_DENSE_COL_MAJOR时，该字段有效。

表 6-46 KmlSolverMatrixOption 结构体字段描述

字段名	类型	描述
fieldMask	uint64_t	配置生效的字段掩码。非0的位代表配置对应字段指定的值，0的位代表使用默认值，可配置的掩码位见具体字段的说明。
type	KmlSolverMatrixType	指定矩阵类型，有如下值。 • KMLSS_MATRIX_GEN表示一般矩阵，**默认选项**。 • KMLSS_MATRIX_STRUCT_SYM表示结构对称矩阵。 • KMLSS_MATRIX_SYM表示对称矩阵。 • KMLSS_MATRIX_HER表示hermitian矩阵。 • KMLSS_MATRIX_SPD表示对称正定矩阵。 • KMLSS_MATRIX_HPD表示hermitian正定矩阵。 掩码位：KMLSS_MATRIX_OPTION_TYPE

表 6-47 KmlSolverMatrixStoreCSR 结构体字段描述

字段名	类型	描述
value	void *	非零元数值数组，大小为非零元个数。
rowOffset	void *	行偏移数组，大小为行数加1。

版权所有 © 华为技术有限公司

字段名	类型	描述
colIndex	void *	列坐标数组，大小为非零元个数，需按升序排列。

表 6-48 KmlSolverMatrixStoreCSC 结构体字段描述

字段名	类型	描述
value	void *	非零元数值数组，大小为非零元个数。
rowIndex	void *	行坐标数组，大小为非零元个数，需按升序排列。
colOffset	void *	列偏移数组，大小为行数加1。

表 6-49 KmlSolverMatrixStoreCOO 结构体字段描述

字段名	类型	描述
value	void *	非零元数值数组，大小为非零元个数。
rowId	void *	行坐标数组，大小为非零元个数，需按升序排列。
colId	void *	列坐标数组，大小为非零元个数，同一行的列号需按升序排列。
nnz	int64_t	非零元个数。

表 6-50 KmlSolverMatrixStoreDense 结构体字段描述

字段名	类型	描述
value	void *	矩阵数值数组，大小为行数乘以列数。
ld	int64_t	主维度跨度。若为行主序，则ld为保存每一行所需要的数组长度，若为列主序，则ld表示保存每一列所需要的数组长度。

返回值

返回值	类型	描述
KMLSS_NO_ERROR	int	正常执行。
KMLSS_INTERNAL_ERROR	int	内部错误。

版权所有 © 华为技术有限公司

返回值	类型	描述
KMLSS_NULL_ARGUMENT	int	pMatrix，store，matOption存在空参数。

依赖

#include "kml_solver.h"

?.2. KmlSolverMatrixSetValue

设置或修改矩阵数值。

说明

指定的矩阵数值应当与创建该矩阵句柄时指定的非零元结构对应，该接口应用于矩阵结构不变但值变化的场景。通过该接口更新value数组后，原value数组空间可释放。

接口定义

C Interface：

int KmlSolverMatrixSetValue(KmlSolverMatrix *matrix, void *values);

参数

参数名	类型	描述	输入/输出
matrix	KmlSolverMatrix *	由KmlSolverMatrixCreate函数创建的矩阵句柄。	输入
values	void *	矩阵数值数组，不可为空指针。	输入

返回值

返回值	类型	描述
KMLSS_NO_ERROR	int	正常执行。
KMLSS_INTERNAL_ERROR	int	内部错误。
KMLSS_NULL_ARGUMENT	int	matrix，values存在空参数。

依赖

#include "kml_solver.h"

 版权所有 © 华为技术有限公司

?.3. KmlSolverMatrixDestroy

销毁由KmlSolverMatrixCreate函数创建的矩阵句柄。

接口定义

C Interface：

int KmlSolverMatrixDestroy(KmlSolverMatrix **pMatrix);

参数

参数名	类型	描述	输入/输出
pMatrix	KmlSolverMatrix **	由KmlSolverMatrixCreate函数创建的矩阵句柄。	输入/输出

返回值

返回值	类型	描述
KMLSS_NO_ERROR	int	正常执行。
KMLSS_INTERNAL_ERROR	int	内部错误。
KMLSS_NULL_ARGUMENT	int	pMatrix为空参数。

依赖

#include "kml_solver.h"

示例

C Interface：

```
#include "kml_solver.h"

KmlSolverMatrix *handle;
int ierr;
int n = 8;

int ia[9] = {0, 3, 7, 10, 14, 17, 21, 23, 26};
int ja[26] = {0, 3, 4, 1, 2, 3, 5, 1, 2, 7, 0, 1, 3, 6, 0, 4, 5, 1, 4, 5, 7, 3, 6, 2, 5, 7};
double a[26] = {1.0, 1.0, 2.0, 9.0, 2.0, 1.0, -3.0, 2.0, 3.0, 2.0, 1.0, 1.0, 9.0, -5.0, 2.0, 6.0, 1.0, -3.0, 1.0, 4.0, 1.0,
-5.0, 7.0, 2.0, 1.0, 2.0};

KmlSolverMatrixStore store;
store.indexType = KMLSS_INDEX_INT32;
store.valueType = KMLSS_VALUE_FP64;
store.nRow = n;
store.nCol = n;
store.format = KMLSS_MATRIX_STORE_CSR;
```

 版权所有 © 华为技术有限公司

```
store.csr.rowOffset = ia;
store.csr.colIndex = ja;
store.csr.value = a;

KmlSolverMatrixOption opt;
opt.fieldMask = KMLSS_MATRIX_OPTION_TYPE;
opt.type = KMLSS_MATRIX_GEN;

ierr = KmlSolverMatrixCreate(&handle, &store, &opt);
if (ierr != KMLSS_NO_ERROR) {
    printf("\nERROR in KmlSolverMatrixCreate: %d", ierr);
    return 1;
}

double b[26] = {2.0, 2.0, 3.0, 1.0, 3.0, 5.0, -7.0, 9.0, 2.0, 1.0, 2.0, 2.0, 4.0, -3.0, 3.0, 7.0, 3.0, -8.0, 6.0, 2.0, 3.0,
-7.0, 2.0, 9.0, 2.0, 6.0};
ierr = KmlSolverMatrixSetValue(handle, b);
if (ierr != KMLSS_NO_ERROR) {
    printf("\nERROR in KmlSolverMatrixSetValue: %d", ierr);
    return 1;
}

ierr = KmlSolverMatrixDestroy(&handle);
if (ierr != KMLSS_NO_ERROR) {
    printf("\nERROR in KmlSolverMatrixDestroy: %d", ierr);
    return 1;
}
```

6.5.3.3.11.2 求解器函数

?.1. KmlDssInit

创建求解器操作句柄。

接口定义

C Interface：

int KmlDssInit(KmlDssSolver **pSolver, const KmlDssInitOption *option);

参数

参数名	类型	描述	输入/输出
pSolver	KmlDssSolver **	求解器句柄指针，pSolver不可为空指针。	输入/输出
option	KmlDssInitOption *	求解器配置选项，不可为空指针，具体内容如表6-51所示。	输入

表 6-51 KmlDssInitOption 结构体字段描述

字段名	类型	描述
fieldMask	uint64_t	配置生效的字段掩码。非0的位代表配置对应字段指定的值，0的位代表使用默认值，可配置的掩码位见具体字段的说明。

版权所有 © 华为技术有限公司

字段名	类型	描述
bwrMode	KmlDssBWRMode	二进制可重现（Bit-Wise Reproducibility）控制选项，有如下值。 ● KMLDSS_BWR_OFF表示不保证二进制可重现，**默认选项**。 ● KMLDSS_BWR_FIXED_THREADS表示在相同线程数下多次运行结果二进制可重现。 掩码位：KMLDSS_INIT_OPTION_BWR_MODE
nThreads	int32_t	求解器初始线程数，如果未设置，默认取OpenMP并行域线程数。 掩码位：KMLDSS_INIT_OPTION_NTHREADS

返回值

返回值	类型	描述
KMLSS_NO_ERROR	int	正常执行。
KMLSS_INTERNAL_ERROR	int	内部错误。
KMLSS_NULL_ARGUMENT	int	pSolver，option存在空参数。

依赖

#include "kml_dss.h"

?.2. KmlDssAnalyze

分析输入矩阵，包括Reordering、符号分解等。

接口定义

C Interface：

int KmlDssAnalyze(KmlDssSolver *solver, const KmlSolverMatrix *matrix, const KmlDssAnalyzeOption *option);

参数

参数名	类型	描述	输入/输出
solver	KmlDssSolver *	由KmlDssInit创建的求解器句柄。	输入

版权所有 © 华为技术有限公司

参数名	类型	描述	输入/输出
matrix	KmlSolverMatrix *	矩阵句柄，由KmlSolverMatrixCreate函数创建，不可为空指针。	输入
option	KmlDssAnalyzeOption *	分析阶段配置选项，不可为空指针，具体内容如表6-52所示。	输入

表 6-52 KmlDssAnalyzeOption 结构体字段描述

字段名	类型	描述
fieldMask	uint64_t	配置生效的字段掩码。非0的位代表配置对应字段指定的值，0的位代表使用默认值，可配置的掩码位请参见具体字段的说明。
nThreadsAnalyze	int32_t	分析阶段线程数，默认和KmlDssInit阶段设置的求解器初始线程数相同。 掩码位：KMLDSS_ANALYZE_OPTION_NTHREADS_ANALYZE
nThreadsFactorize	int32_t	分解阶段线程数，默认和KmlDssInit阶段设置的求解器初始线程数相同。 掩码位：KMLDSS_ANALYZE_OPTION_NTHREADS_FACTORIZE
nThreadsSolve	int32_t	回代阶段线程数，默认和KmlDssInit阶段设置的求解器初始线程数相同。 掩码位：KMLDSS_ANALYZE_OPTION_NTHREADS_SOLVE
matchingType	KmlDssMatchingType	matching算法类型，有如下值。 • KMLDSS_MATCHING_OFF表示关闭matching算法，默认选项。 • KMLDSS_MATCHING_MC64表示最大权重匹配算法（Maximum Weighted Matching Algorithm），可以改善数值分解过程中的主元大小，一般用于非对称矩阵。 • KMLDSS_MATCHING_SMC64表示对称最大权重匹配算法（Symmetric Maximum Weighted Matching Algorithm），可以改善数值分解过程中的主元大小，一般用于对称或结构对称矩阵。 掩码位：KMLDSS_ANALYZE_OPTION_MATCHING_TYPE

版权所有 © 华为技术有限公司

字段名	类型	描述
nThreadsMatching	int32_t	matching阶段线程数，默认和分析阶段线程数相同。 掩码位：KMLDSS_ANALYZE_OPTION_NTHREADS_MATCHING
rdrType	KmlDssRdrType	Fill-in reduction算法类型，有如下值。 ● KMLDSS_RDR_KRDR表示KML Reorder做Fill-in reduction，默认选项。 ● KMLDSS_RDR_CUSTOM表示使用自定义Fill-in reduction函数。 掩码位：KMLDSS_ANALYZE_OPTION_RDR_TYPE
nThreadsRdr	int32_t	Fill-in reduction线程数，默认和分析阶段线程数相同。 掩码位：KMLDSS_ANALYZE_OPTION_NTHREADS_RDR
customRdrAlgo	KmlDssCustomRdrAlgo	用户自定义的Fill-in reduction函数，该函数的原型为typedef int (*KmlDssCustomRdrAlgo)(const KmlSolverMatrixStore *store, void *perm, void *iperm, void* arg); ● 如果rdrType未设置为KMLDSS_RDR_CUSTOM时忽略该字段。 ● 如果rdrType设置成了KMLDSS_RDR_CUSTOM但该字段及rdrPerm字段均未设置，则不进行Fill-in reduction，优先级为rdrPerm>customRdrAlgo。 掩码位：KMLDSS_ANALYZE_OPTION_CUSTOM_RDR_ALGO
customRdrArgs	void *	在调用自定义Fill-in reduction函数时会将该字段作为最后一个参数，用户可以通过该字段传入自定义Fill-in reduction函数中需要用到的其它参数。 掩码位：KMLDSS_ANALYZE_OPTION_CUSTOM_RDR_ARGS

版权所有 © 华为技术有限公司

字段名	类型	描述
rdrPerm	void *	传入用户自定义的Fill-in reduction的perm，映射关系为重排后矩阵的第i行/列是原始矩阵中的第rdrPerm[i]行/列。 ● 如果rdrType未设置为KMLDSS_RDR_CUSTOM时忽略该字段。 ● 如果rdrType设置成了KMLDSS_RDR_CUSTOM但该字段及customRdrAlgo字段均未设置，则等价于KMLDSS_FR_NONE，优先级为rdrPerm>customRdrAlgo。 掩码位：KMLDSS_ANALYZE_OPTION_RDR_PERM
pivotingType	KmlDssPivotingType	pivoting算法类型，有如下值。 ● KMLDSS_PIVOTING_OFF表示关闭动态选主元，默认选项。 ● KMLDSS_PIVOTING_LOCAL表示局部动态选主元。 掩码位：KMLDSS_ANALYZE_OPTION_PIVOTING_TYPE
pivotingThreshold	double	pivoting阈值，暂不支持设置。 掩码位：KMLDSS_ANALYZE_OPTION_PIVOTING_THRESHOLD
batchNRHS	int64_t	回代阶段每一批同时求解的最大右端向量个数，该参数为性能优化选项，和实际求解的右端向量个数无关。默认数值为1。 掩码位：KMLDSS_ANALYZE_OPTION_BATCH_NRHS
schurSize	int64_t	部分分解中schur补矩阵的大小。默认数值为0。 掩码位：KMLDSS_ANALYZE_OPTION_SCHUR_SIZE
sparseSupernodeSize	int32_t	性能调优参数，表示合并后的 sparse supernode 的最大列数。在求解器内部，若干小的 supernode 可以合并为一个 sparse supernode，并使用稀疏矩阵格式表示和计算。该数值为 0 时，不开启 sparse supernode。默认数值为0。 掩码位：KMLDSS_ANALYZE_OPTION_SPARSE_SUPERNODE_SIZE

版权所有 © 华为技术有限公司

字段名	类型	描述
sparseSupernodeThreshold	int32_t	性能调优参数，表示可以合并为 sparse supernode 的 supernode 的最大列数，超过该阈值的 supernode 不参与合并。该数值为 0 时，不开启 sparse supernode。默认数值为0 掩码位：KMLDSS_ANALYZE_OPTION_SPARSE_SUPERNODE_THRESHOLD
schurFormat	KmlSolverMatrixStoreFormat	部分分解中schur补矩阵的存储格式，可选矩阵存储格式请参见**KmlSolverMatrixCreate()**的参数说明，默认值为KMLSS_MATRIX_STORE_DENSE_ROW_MAJOR。 掩码位：KMLDSS_ANALYZE_OPTION_SCHUR_FORMAT
matrixTransType	KmlDssMatrixTransType	转置类型，有如下值。 • KMLDSS_MATRIX_NO_TRANS表示求解原始线性系统，默认选项。 • KMLDSS_MATRIX_TRANS表示求解转置后的线性系统。 掩码位：KMLDSS_ANALYZE_OPTION_MATRIX_TRANS_TYPE

返回值

返回值	类型	描述
KMLSS_NO_ERROR	int	正常执行。
KMLSS_INTERNAL_ERROR	int	内部错误。
KMLSS_NULL_ARGUMENT	int	solver，matrix，option存在空参数。

依赖

#include "kml_dss.h"

?.3. KmlDssFactorize

对矩阵做数值分解，需先使用KmlDssAnalyze进行分析。

接口定义

C Interface：

 版权所有 © 华为技术有限公司

int KmlDssFactorize(KmlDssSolver *solver, const KmlSolverMatrix *matrix, const KmlDssFactorizeOption *option);

参数

参数名	类型	描述	输入/输出
solver	KmlDssSolver *	由KmlDssInit创建的求解器句柄。	输入
matrix	KmlSolverMatrix *	矩阵句柄，由KmlSolverMatrixCreate函数创建，不可为空指针。	输入
option	KmlDssFactorizeOption *	分解阶段配置选项，不可为空指针，具体内容如表6-53所示。	输入

表 6-53 KmlDssFactorizeOption 结构体字段描述

字段名	类型	描述
fieldMask	uint64_t	配置生效的字段掩码。非0的位代表配置对应字段指定的值，0的位代表使用默认值，可配置的掩码位见具体字段的说明。
perturbationThreshold	double	扰动阈值，有如下值。 ● 等于0表示不做扰动，默认数值。 ● 大于0表示使用用户指定的扰动阈值。 ● 小于0表示使用求解器自动计算的扰动阈值。 掩码位：KMLDSS_FACTORIZE_OPTION_PERTURBATION_THRESHOLD

返回值

返回值	类型	描述
KMLSS_NO_ERROR	int	正常执行。
KMLSS_INTERNAL_ERROR	int	内部错误。
KMLSS_NULL_ARGUMENT	int	solver，matrix，option存在空参数。

依赖

#include "kml_dss.h"

版权所有 © 华为技术有限公司

?.4. KmlDssSolve

对矩阵做回代求解，需先使用KmlDssFactorize进行分解。

接口定义

C Interface：

int KmlDssSolve(KmlDssSolver *solver, const KmlSolverMatrix *b, KmlSolverMatrix *x, const KmlDssSolveOption *option);

参数

参数名	类型	描述	输入/输出
solver	KmlDssSolver *	由KmlDssInit创建的求解器句柄。	输入
b	KmlSolverMatrix *	右端向量结构体指针，本质是矩阵，由KmlSolverMatrixCreate函数创建，不可为空指针。	输入
x	KmlSolverMatrix *	解空间结构体指针，本质是矩阵，由KmlSolverMatrixCreate函数创建，输入不可为空指针，输出为包含求解出的x的数值数组。	输出
option	KmlDssSolveOption *	分解阶段配置选项，不可为空指针，具体内容如表6-54所示。	输入

表 6-54 KmlDssSolveOption 结构体字段描述

字段名	类型	描述
fieldMask	uint64_t	配置生效的字段掩码。非0的位代表配置对应字段指定的值，0的位代表使用默认值，可配置的掩码位见具体字段的说明。

版权所有 © 华为技术有限公司

字段名	类型	描述
stage	KmlDssSolveStage	设置回代阶段，有如下值。 ● KMLDSS_SOLVE_ALL表示全阶段回代，包括迭代修正，默认选项。 ● KMLDSS_SOLVE_FORWARD表示只做前向回代。 ● KMLDSS_SOLVE_DIAGONAL表示只做对角块求解。 ● KMLDSS_SOLVE_BACKWARD表示只做后向回代。 ● KMLDSS_SOLVE_REFINE表示只做迭代修正。 掩码位：KMLDSS_SOLVE_OPTION_SOLVE_STAGE
refineMethod	KmlDssRefineMethod	迭代修正方法，有如下值。 ● KMLDSS_REFINE_OFF表示关闭迭代修正，默认选项。 ● KMLDSS_REFINE_CLASSICAL表示常规迭代修正。 掩码位：KMLDSS_SOLVE_OPTION_REFINE_METHOD
refineMaxSteps	int32_t	最大迭代修正步数，有如下值。 ● 等于0表示由求解器自动决定，默认数值。 ● 大于0表示用户设置最大迭代修正步数。 掩码位：KMLDSS_SOLVE_OPTION_REFINE_MAX_STEP

版权所有 © 华为技术有限公司

字段名	类型	描述
refineTolerance	double	迭代修正残差阈值，该字段有如下值。 ● 等于0表示由求解器自动决定，默认数值。 ● 大于0表示用户设置迭代修正残差阈值。 掩码位：KMLDSS_SOLVE_OPTION_REFINE_TOLERANCE 迭代满足如下任一条件则立即结束迭代修正。 ● $refineTolerance<BERR$ ● $BERR_{i+1}>BERR_i/2$ ● 达到refineMaxSteps步数 其中，berr的计算方式为 $BERR\equiv\max_i\frac{\lfloor Ax-b\rfloor_i}{(\lvert A\rvert\cdot\lvert x\rvert+\lvert b\rvert)_i}$

返回值

返回值	类型	描述
KMLSS_NO_ERROR	int	正常执行。
KMLSS_INTERNAL_ERROR	int	内部错误。
KMLSS_NULL_ARGUMENT	int	solver，b，x，option存在空参数。

依赖

#include "kml_dss.h"

?.5. KmlDssClean

销毁由KmlDssInit创建的求解器句柄。

接口定义

C Interface：

int KmlDssClean(KmlDssSolver **pSolver);

 版权所有 © 华为技术有限公司

参数

参数名	类型	描述	输入/输出
pSolver	KmlDssSolver **	由KmlDssInit创建的求解器句柄。	输入/输出

返回值

返回值	类型	描述
KMLSS_NO_ERROR	int	正常执行。
KMLSS_NULL_ARGUMENT	int	pSolver为空参数。

依赖

#include "kml_dss.h"

?.6. KmlDssQuery

查询求解器内部信息。

接口定义

C Interface：

int KmlDssQuery(KmlDssSolver *solver, KmlDssInfo *info);

参数

参数名	类型	描述	输入/输出
solver	KmlDssSolver *	由KmlDssInit创建的求解器句柄。	输入
info	KmlDssInfo *	信息查询结构体指针，输入不可为空指针，输出为查询后的结果，具体内容如表6-55所示。	输入/输出

表 6-55 KmlDssInfo 结构体字段描述

字段名	类型	描述
fieldMask	uint64_t	配置生效的字段掩码。非0的位代表配置对应字段指定的值，0的位代表不查询该字段，可配置的掩码位见具体字段的说明。

版权所有 © 华为技术有限公司

字段名	类型	描述
rdrPerm	void *	分析阶段使用的对称排序的perm，映射关系为重排后矩阵的第i行/列是原始矩阵中的第rdrPerm[i]行/列。该数组内存需由用户申请和管理。 掩码位：KMLDSS_INFO_RDR_PERM
schurNnz	int64_t	部分分解中schur补矩阵的非零元个数。 掩码位：KMLDSS_INFO_SCHUR_NNZ
schurMat	KmlSolverMatrixStore	部分分解中schur补矩阵的存储结构，需用户分配存储空间并指定。结构体说明请参见**KmlSolverMatrixCreate()**的参数说明。 掩码位：KMLDSS_INFO_SCHUR_MAT
refineSteps	int32_t	迭代修正的步数。 掩码位：KMLDSS_INFO_REFINE_STEPS
peakMem	int64_t	峰值内存，单位为Byte。 掩码位：KMLDSS_INFO_PEAK_MEM
nFillIn	int64_t	分解后矩阵的总非零元个数。 掩码位：KMLDSS_INFO_FILL_IN
nFillInL	int64_t	分解后L矩阵的总非零元个数，包含对角线元素。 掩码位：KMLDSS_INFO_FILL_IN_L
nFillInU	int64_t	分解后U矩阵的总非零元个数，包含对角线元素。 掩码位：KMLDSS_INFO_FILL_IN_U
pivotGrowth	double	分解阶段的 pivot growth，计算公式为 $$\gamma = \frac{\max_k \left(\max_i \lvert l_{ik} \rvert \cdot \max_j \lvert u_{kj} \rvert \right)}{\max_{i,j} \lvert a_{ij} \rvert}$$ 掩码位：KMLDSS_INFO_PIVOT_GROWTH

返回值

返回值	类型	描述
KMLSS_NO_ERROR	int	正常执行。
KMLSS_INTERNAL_ERROR	int	内部错误。
KMLSS_NULL_ARGUMENT	int	solver，info存在空参数。

版权所有 © 华为技术有限公司

依赖

#include "kml_solver.h"

?.7. 示例 1

C Interface：

```
#include <stdio.h>
#include <stdlib.h>
#include "kml_solver.h"

int main()
{
    int ierr;
    int n = 8;
    int nrhs = 1;
    // Create matrix A
    int ia[9] = {0, 2, 4, 6, 7, 8, 10, 12, 14};
    int ja[14] = {0, 7, 1, 6, 2, 5, 3, 4, 2, 5, 1, 6, 0, 7};
    double a[14] = {1.0, 2.0, -2.0, 3.0, 3.0, 4.0, -4.0, 5.0, 4.0, -6.0, 3.0, 7.0, 2.0, 8.0};
    KmlSolverMatrixStore storeA;
    storeA.indexType = KMLSS_INDEX_INT32;
    storeA.valueType = KMLSS_VALUE_FP64;
    storeA.nRow = n;
    storeA.nCol = n;
    storeA.format = KMLSS_MATRIX_STORE_CSR;
    storeA.csr.rowOffset = ia;
    storeA.csr.colIndex = ja;
    storeA.csr.value = a;
    KmlSolverMatrixOption optA;
    optA.fieldMask = KMLSS_MATRIX_OPTION_TYPE;
    optA.type = KMLSS_MATRIX_GEN;
    KmlSolverMatrix *A;
    ierr = KmlSolverMatrixCreate(&A, &storeA, &optA);
    if (ierr != KMLSS_NO_ERROR) {
        printf("ERROR when create A: %d\n", ierr);
        return 1;
    }
    // Create vector b
    double b[8] = {3.0, 1.0, 7.0, -4.0, 5.0, -2.0, 10.0, 10.0};
    KmlSolverMatrixStore storeB;
    storeB.indexType = KMLSS_INDEX_INT32;
    storeB.valueType = KMLSS_VALUE_FP64;
    storeB.nRow = n;
    storeB.nCol = nrhs;
    storeB.format = KMLSS_MATRIX_STORE_DENSE_COL_MAJOR;
    storeB.dense.value = b;
    storeB.dense.ld = n;
    KmlSolverMatrixOption optB;
    optB.fieldMask = KMLSS_MATRIX_OPTION_TYPE;
    optB.type = KMLSS_MATRIX_GEN;
    KmlSolverMatrix *B;
    ierr = KmlSolverMatrixCreate(&B, &storeB, &optB);
    if (ierr != KMLSS_NO_ERROR) {
        printf("ERROR when create b: %d\n", ierr);
        return 1;
    }
    // Create vector x
    double x[8] = {0};
    KmlSolverMatrixStore storeX;
    storeX.indexType = KMLSS_INDEX_INT32;
    storeX.valueType = KMLSS_VALUE_FP64;
    storeX.nRow = n;
    storeX.nCol = nrhs;
    storeX.format = KMLSS_MATRIX_STORE_DENSE_COL_MAJOR;
    storeX.dense.value = x;
    storeX.dense.ld = n;
    KmlSolverMatrixOption optX;
```

 版权所有 © 华为技术有限公司

```
    optX.fieldMask = KMLSS_MATRIX_OPTION_TYPE;
    optX.type = KMLSS_MATRIX_GEN;
    KmlSolverMatrix *X;
    ierr = KmlSolverMatrixCreate(&X, &storeX, &optX);
    if (ierr != KMLSS_NO_ERROR) {
        printf("ERROR when create x: %d\n", ierr);
        return 1;
    }
    // Init solver
    KmlDssInitOption opt;
    opt.fieldMask = KMLDSS_INIT_OPTION_BWR_MODE | KMLDSS_INIT_OPTION_NTHREADS;
    opt.bwrMode = KMLDSS_BWR_OFF;
    opt.nThreads = 32;
    KmlDssSolver *solver;
    ierr = KmlDssInit(&solver, &opt);
    if (ierr != KMLSS_NO_ERROR) {
        printf("ERROR in KmlDssInit: %d\n", ierr);
        return ierr;
    }
    // Analyze
    KmlDssAnalyzeOption optAnalyze;
    optAnalyze.fieldMask = KMLDSS_ANALYZE_OPTION_MATCHING_TYPE |
                           KMLDSS_ANALYZE_OPTION_RDR_TYPE |
                           KMLDSS_ANALYZE_OPTION_NTHREADS_RDR;
    optAnalyze.matchingType = KMLDSS_MATCHING_OFF;
    optAnalyze.rdrType = KMLDSS_RDR_KRDR;
    optAnalyze.nThreadsRdr = 8;
    ierr = KmlDssAnalyze(solver, A, &optAnalyze);
    if (ierr != KMLSS_NO_ERROR) {
        printf("ERROR in KmlDssAnalyze: %d\n", ierr);
        return ierr;
    }
    // Factorize
    KmlDssFactorizeOption optFact;
    optFact.fieldMask = KMLDSS_FACTORIZE_OPTION_PERTURBATION_THRESHOLD;
    optFact.perturbationThreshold = 1e-8;
    ierr = KmlDssFactorize(solver, A, &optFact);
    if (ierr != KMLSS_NO_ERROR) {
        printf("ERROR in KmlDssFactorize: %d\n", ierr);
        return ierr;
    }
    // Solve
    KmlDssSolveOption optSolve;
    optSolve.fieldMask = KMLDSS_SOLVE_OPTION_SOLVE_STAGE |
KMLDSS_SOLVE_OPTION_REFINE_METHOD;
    optSolve.stage = KMLDSS_SOLVE_ALL;
    optSolve.refineMethod = KMLDSS_REFINE_OFF;
    ierr = KmlDssSolve(solver, B, X, &optSolve);
    if (ierr != KMLSS_NO_ERROR) {
        printf("ERROR in KmlDssSolve: %d\n", ierr);
        return ierr;
    }
    // Output result x
    printf("Result of first factorize and solve:\n");
    for (int i = 0; i < n; i++) {
        printf("%lf ", x[i]);
    }
    printf("\n");
    // Set new values of A
    double a1[14] = {2.0, 3.0, -3.0, 4.0, 4.0, 5.0, -5.0, 6.0, 5.0, -7.0, 4.0, 8.0, 3.0, 9.0};
    KmlSolverMatrixSetValue(A, a1);
    // Factorize with new values
    ierr = KmlDssFactorize(solver, A, &optFact);
    if (ierr != KMLSS_NO_ERROR) {
        printf("ERROR in KmlDssFactorize: %d\n", ierr);
        return ierr;
    }
    // Set new values of B
    double b1[8] = {1.0, 1.0, 1.0, 1.0, 1.0, 1.0, 1.0, 1.0};
```

 版权所有 © 华为技术有限公司

```
    KmlSolverMatrixSetValue(B, b1);
    // Solve with new values
    ierr = KmlDssSolve(solver, B, X, &optSolve);
    if (ierr != KMLSS_NO_ERROR) {
        printf("ERROR in KmlDssSolve: %d\n", ierr);
        return ierr;
    }
    // Output new result x
    printf("Result of second factorize and solve:\n");
    for (int i = 0; i < n; i++) {
        printf("%lf ", x[i]);
    }
    printf("\n");
    // Query
    KmlDssInfo info;
    info.fieldMask = KMLDSS_INFO_PEAK_MEM;
    ierr = KmlDssQuery(solver, &info);
    if (ierr != KMLSS_NO_ERROR) {
        printf("ERROR in KmlDssQuery: %d\n", ierr);
        return ierr;
    }
    printf("Peak memory is %ld Byte\n", info.peakMem);
    // Destroy
    KmlDssClean(&solver);
    KmlSolverMatrixDestroy(&A);
    KmlSolverMatrixDestroy(&B);
    KmlSolverMatrixDestroy(&X);
    return 0;
}
```

运行结果：

```
Result of first factorize and solve:
1.000000 1.000000 1.000000 1.000000 1.000000 1.000000 1.000000 1.000000
Result of second factorize and solve:
0.666667 -0.100000 0.226415 -0.200000 0.166667 0.018868 0.175000 -0.111111
Peak memory is 167775740 Byte
```

?.8. 示例 2

使用schur补对矩阵进行求解：

```
#include <stdio.h>
#include <stdlib.h>
#include "kml_solver.h"
void dgesv_(const int *n, const int *nrhs, double *a, const int *lda, int *ipiv, double *b, const int *ldb,
    int *info);
int main()
{
    int ierr;
    int n = 8;
    int nrhs = 1;
    // Create matrix A
    int ia[9] = {0, 2, 4, 6, 7, 8, 10, 12, 14};
    int ja[14] = {0, 7, 1, 6, 2, 5, 3, 4, 2, 5, 1, 6, 0, 7};
    double a[14] = {1.0, 2.0, -2.0, 3.0, 3.0, 4.0, -4.0, 5.0, 4.0, -6.0, 3.0, 7.0, 2.0, 8.0};
    KmlSolverMatrixStore storeA;
    storeA.indexType = KMLSS_INDEX_INT32;
    storeA.valueType = KMLSS_VALUE_FP64;
    storeA.nRow = n;
    storeA.nCol = n;
    storeA.format = KMLSS_MATRIX_STORE_CSR;
    storeA.csr.rowOffset = ia;
    storeA.csr.colIndex = ja;
    storeA.csr.value = a;
    KmlSolverMatrixOption optA;
    optA.fieldMask = KMLSS_MATRIX_OPTION_TYPE;
    optA.type = KMLSS_MATRIX_GEN;
    KmlSolverMatrix *A;
    ierr = KmlSolverMatrixCreate(&A, &storeA, &optA);
```

 版权所有 © 华为技术有限公司

```
if (ierr != KMLSS_NO_ERROR) {
    printf("ERROR when create A: %d\n", ierr);
    return 1;
}
// Create vector b
double b[8] = {3.0, 1.0, 7.0, -4.0, 5.0, -2.0, 10.0, 10.0};
KmlSolverMatrixStore storeB;
storeB.indexType = KMLSS_INDEX_INT32;
storeB.valueType = KMLSS_VALUE_FP64;
storeB.nRow = n;
storeB.nCol = nrhs;
storeB.format = KMLSS_MATRIX_STORE_DENSE_COL_MAJOR;
storeB.dense.value = b;
storeB.dense.ld = n;
KmlSolverMatrixOption optB;
optB.fieldMask = KMLSS_MATRIX_OPTION_TYPE;
optB.type = KMLSS_MATRIX_GEN;
KmlSolverMatrix *B;
ierr = KmlSolverMatrixCreate(&B, &storeB, &optB);
if (ierr != KMLSS_NO_ERROR) {
    printf("ERROR when create b: %d\n", ierr);
    return 1;
}
// Create vector x
double x[8] = {0};
KmlSolverMatrixStore storeX;
storeX.indexType = KMLSS_INDEX_INT32;
storeX.valueType = KMLSS_VALUE_FP64;
storeX.nRow = n;
storeX.nCol = nrhs;
storeX.format = KMLSS_MATRIX_STORE_DENSE_COL_MAJOR;
storeX.dense.value = x;
storeX.dense.ld = n;
KmlSolverMatrixOption optX;
optX.fieldMask = KMLSS_MATRIX_OPTION_TYPE;
optX.type = KMLSS_MATRIX_GEN;
KmlSolverMatrix *X;
ierr = KmlSolverMatrixCreate(&X, &storeX, &optX);
if (ierr != KMLSS_NO_ERROR) {
    printf("ERROR when create x: %d\n", ierr);
    return 1;
}
// Init solver
KmlDssInitOption opt;
opt.fieldMask = KMLDSS_INIT_OPTION_BWR_MODE | KMLDSS_INIT_OPTION_NTHREADS;
opt.bwrMode = KMLDSS_BWR_OFF;
opt.nThreads = 32;
KmlDssSolver *solver;
ierr = KmlDssInit(&solver, &opt);
if (ierr != KMLSS_NO_ERROR) {
    printf("ERROR in KmlDssInit: %d\n", ierr);
    return ierr;
}
// Analyze
int schurSize = 2;
KmlDssAnalyzeOption optAnalyze;
optAnalyze.fieldMask = KMLDSS_ANALYZE_OPTION_MATCHING_TYPE |
                       KMLDSS_ANALYZE_OPTION_RDR_TYPE |
                       KMLDSS_ANALYZE_OPTION_NTHREADS_RDR |
                       KMLDSS_ANALYZE_OPTION_SCHUR_SIZE |
                       KMLDSS_ANALYZE_OPTION_SCHUR_FORMAT;
optAnalyze.matchingType = KMLDSS_MATCHING_OFF;
optAnalyze.rdrType = KMLDSS_RDR_KRDR;
optAnalyze.nThreadsRdr = 8;
optAnalyze.schurSize = schurSize;
optAnalyze.schurFormat = KMLSS_MATRIX_STORE_CSR;
ierr = KmlDssAnalyze(solver, A, &optAnalyze);
if (ierr != KMLSS_NO_ERROR) {
    printf("ERROR in KmlDssAnalyze: %d\n", ierr);
```

 版权所有 © 华为技术有限公司

```
    return ierr;
}
// Factorize
KmlDssFactorizeOption optFact;
optFact.fieldMask = KMLDSS_FACTORIZE_OPTION_PERTURBATION_THRESHOLD;
optFact.perturbationThreshold = 1e-8;
ierr = KmlDssFactorize(solver, A, &optFact);
if (ierr != KMLSS_NO_ERROR) {
    printf("ERROR in KmlDssFactorize: %d\n", ierr);
    return ierr;
}
// Solve forward
KmlDssSolveOption optSolve;
optSolve.fieldMask = KMLDSS_SOLVE_OPTION_SOLVE_STAGE;
optSolve.stage = KMLDSS_SOLVE_FORWARD;
ierr = KmlDssSolve(solver, B, X, &optSolve);
if (ierr != KMLSS_NO_ERROR) {
    printf("ERROR in KmlDssSolve: %d\n", ierr);
    return ierr;
}
// Solve schur
double xTemp[8] = {0};
KmlDssInfo info;
info.fieldMask = KMLDSS_INFO_SCHUR_NNZ;
ierr = KmlDssQuery(solver, &info);
if (ierr != KMLSS_NO_ERROR) {
    printf("ERROR in KmlDssQuery: %d\n", ierr);
    return ierr;
}
int schurNnz = info.schurNnz;
KmlSolverMatrixStore schurMat;
schurMat.csr.value = malloc(sizeof(double) * schurNnz);
schurMat.csr.rowOffset = malloc(sizeof(int) * (schurSize + 1));
schurMat.csr.colIndex = malloc(sizeof(int) * schurNnz);
info.fieldMask = KMLDSS_INFO_SCHUR_MAT;
info.schurMat = &schurMat;
ierr = KmlDssQuery(solver, &info);
if (ierr != KMLSS_NO_ERROR) {
    printf("ERROR in KmlDssQuery: %d\n", ierr);
    return ierr;
}
double *denseA = (double *)malloc(sizeof(double) * schurSize * schurSize);
int *ja_schur = (int *)(schurMat.csr.colIndex);
int *ia_schur = (int *)(schurMat.csr.rowOffset);
double *value_schur = (double *)(schurMat.csr.value);
for (int i = 0; i < schurSize; i++) {
    for (int id = ia_schur[i]; id < ia_schur[i + 1]; id++) {
        int j = ja_schur[id];
        denseA[i + j * schurSize] = value_schur[id];
    }
}
// use lapack to solve scuhrmat
int *ipiv = (int *)malloc(sizeof(int) * schurSize);
int iinfo = 0;
dgesv_(&schurSize, &nrhs, denseA, &schurSize, ipiv, &xTemp[n - schurSize], &n, &iinfo);
free(denseA);
free(ipiv);
KmlSolverMatrixSetValue(B, xTemp);
// Solve backward
optSolve.stage = KMLDSS_SOLVE_BACKWARD;
ierr = KmlDssSolve(solver, B, X, &optSolve);
if (ierr != KMLSS_NO_ERROR) {
    printf("ERROR in KmlDssSolve: %d\n", ierr);
    return ierr;
}
// Output result x
printf("Result of first factorize and solve:\n");
for (int i = 0; i < n; i++) {
    printf("%lf ", x[i]);
```

 版权所有 © 华为技术有限公司

```
    }
    printf("\n");
    // Destroy
    KmlDssClean(&solver);
    KmlSolverMatrixDestroy(&A);
    KmlSolverMatrixDestroy(&B);
    KmlSolverMatrixDestroy(&X);
    return 0;
}
```

?.9. 示例 3

使用自定义fill-in-reduction函数（调用mtmetis作为示例）：

```
#include <stdio.h>
#include <stdlib.h>
#include "mtmetis.h"
#include "kml_solver.h"
static int userFR(const KmlSolverMatrixStore *store, void *perm, void *iperm, void* arg)
{
    int64_t numRows = store->nRow;
    int *offsets = (int*)store->csr.rowOffset;
    int *indices = (int*)store->csr.colIndex;
    int64_t nnz = offsets[numRows] - offsets[0];
    int xadj[numRows + 1];
    int adjncy[nnz];
    xadj[0] = 0;
    int nzCount = 0;
    for (int i = 0; i < numRows; i++) {
        for (int offset = offsets[i]; offset < offsets[i + 1]; ++offset) {
            int j = indices[offset];
            if (j != i) {
                adjncy[nzCount] = j;
                nzCount++;
            }
        }
        xadj[i + 1] = nzCount;
    }
    int rc = 0;
    double *options = mtmetis_init_options();

    options[MTMETIS_OPTION_NTHREADS] = 1;
    mtmetis_vtx_type n = numRows;
    rc = MTMETIS_NodeND(&n, (mtmetis_adj_type*)xadj, (mtmetis_vtx_type*)adjncy,
                        NULL, options, (mtmetis_pid_type*)perm, (mtmetis_pid_type*)iperm);
    free(options);

    return rc == MTMETIS_SUCCESS ? 0 : -1;
}
int main()
{
    int ierr;
    int n = 8;
    int nrhs = 1;
    // Create matrix A
    int ia[9] = {0, 2, 4, 6, 7, 8, 10, 12, 14};
    int ja[14] = {0, 7, 1, 6, 2, 5, 3, 4, 2, 5, 1, 6, 0, 7};
    double a[14] = {1.0, 2.0, -2.0, 3.0, 3.0, 4.0, -4.0, 5.0, 4.0, -6.0, 3.0, 7.0, 2.0, 8.0};
    KmlSolverMatrixStore storeA;
    storeA.indexType = KMLSS_INDEX_INT32;
    storeA.valueType = KMLSS_VALUE_FP64;
    storeA.nRow = n;
    storeA.nCol = n;
    storeA.format = KMLSS_MATRIX_STORE_CSR;
    storeA.csr.rowOffset = ia;
    storeA.csr.colIndex = ja;
    storeA.csr.value = a;
    KmlSolverMatrixOption optA;
    optA.fieldMask = KMLSS_MATRIX_OPTION_TYPE;
```

 版权所有 © 华为技术有限公司

```
optA.type = KMLSS_MATRIX_GEN;
KmlSolverMatrix *A;
ierr = KmlSolverMatrixCreate(&A, &storeA, &optA);
if (ierr != KMLSS_NO_ERROR) {
    printf("ERROR when create A: %d\n", ierr);
    return 1;
}
// Create vector b
double b[8] = {3.0, 1.0, 7.0, -4.0, 5.0, -2.0, 10.0, 10.0};
KmlSolverMatrixStore storeB;
storeB.indexType = KMLSS_INDEX_INT32;
storeB.valueType = KMLSS_VALUE_FP64;
storeB.nRow = n;
storeB.nCol = nrhs;
storeB.format = KMLSS_MATRIX_STORE_DENSE_COL_MAJOR;
storeB.dense.value = b;
storeB.dense.ld = n;
KmlSolverMatrixOption optB;
optB.fieldMask = KMLSS_MATRIX_OPTION_TYPE;
optB.type = KMLSS_MATRIX_GEN;
KmlSolverMatrix *B;
ierr = KmlSolverMatrixCreate(&B, &storeB, &optB);
if (ierr != KMLSS_NO_ERROR) {
    printf("ERROR when create b: %d\n", ierr);
    return 1;
}
// Create vector x
double x[8] = {0};
KmlSolverMatrixStore storeX;
storeX.indexType = KMLSS_INDEX_INT32;
storeX.valueType = KMLSS_VALUE_FP64;
storeX.nRow = n;
storeX.nCol = nrhs;
storeX.format = KMLSS_MATRIX_STORE_DENSE_COL_MAJOR;
storeX.dense.value = x;
storeX.dense.ld = n;
KmlSolverMatrixOption optX;
optX.fieldMask = KMLSS_MATRIX_OPTION_TYPE;
optX.type = KMLSS_MATRIX_GEN;
KmlSolverMatrix *X;
ierr = KmlSolverMatrixCreate(&X, &storeX, &optX);
if (ierr != KMLSS_NO_ERROR) {
    printf("ERROR when create x: %d\n", ierr);
    return 1;
}
// Init solver
KmlDssInitOption opt;
opt.fieldMask = KMLDSS_INIT_OPTION_BWR_MODE | KMLDSS_INIT_OPTION_NTHREADS;
opt.bwrMode = KMLDSS_BWR_OFF;
opt.nThreads = 32;
KmlDssSolver *solver;
ierr = KmlDssInit(&solver, &opt);
if (ierr != KMLSS_NO_ERROR) {
    printf("ERROR in KmlDssInit: %d\n", ierr);
    return ierr;
}
// Analyze
KmlDssAnalyzeOption optAnalyze;
optAnalyze.fieldMask = KMLDSS_ANALYZE_OPTION_MATCHING_TYPE |
                       KMLDSS_ANALYZE_OPTION_RDR_TYPE |
                       KMLDSS_ANALYZE_OPTION_NTHREADS_RDR |
                       KMLDSS_ANALYZE_OPTION_CUSTOM_RDR_ALGO |
                       KMLDSS_ANALYZE_OPTION_CUSTOM_RDR_ARGS;
optAnalyze.matchingType = KMLDSS_MATCHING_OFF;
optAnalyze.rdrType = KMLDSS_RDR_CUSTOM;
optAnalyze.customRdrAlgo = userFR;
optAnalyze.customRdrArgs = NULL;
optAnalyze.nThreadsRdr = 8;
ierr = KmlDssAnalyze(solver, A, &optAnalyze);
```

 版权所有 © 华为技术有限公司

```
    if (ierr != KMLSS_NO_ERROR) {
        printf("ERROR in KmlDssAnalyze: %d\n", ierr);
        return ierr;
    }
    // Factorize
    KmlDssFactorizeOption optFact;
    optFact.fieldMask = KMLDSS_FACTORIZE_OPTION_PERTURBATION_THRESHOLD;
    optFact.perturbationThreshold = 1e-8;
    ierr = KmlDssFactorize(solver, A, &optFact);
    if (ierr != KMLSS_NO_ERROR) {
        printf("ERROR in KmlDssFactorize: %d\n", ierr);
        return ierr;
    }
    // Solve
    KmlDssSolveOption optSolve;
    optSolve.fieldMask = KMLDSS_SOLVE_OPTION_SOLVE_STAGE |
KMLDSS_SOLVE_OPTION_REFINE_METHOD;
    optSolve.stage = KMLDSS_SOLVE_ALL;
    optSolve.refineMethod = KMLDSS_REFINE_OFF;
    ierr = KmlDssSolve(solver, B, X, &optSolve);
    if (ierr != KMLSS_NO_ERROR) {
        printf("ERROR in KmlDssSolve: %d\n", ierr);
        return ierr;
    }
    // Output result x
    printf("Result of first factorize and solve:\n");
    for (int i = 0; i < n; i++) {
        printf("%lf ", x[i]);
    }
    printf("\n");
    // Destroy
    KmlDssClean(&solver);
    KmlSolverMatrixDestroy(&A);
    KmlSolverMatrixDestroy(&B);
    KmlSolverMatrixDestroy(&X);
}
```

6.5.3.4 函数定义（集群版）

6.5.3.4.1 SCAISS-CG

6.5.3.4.1.1 KmlScaissCgInitStripes?I

初始化数据结构，并将用户提供的系数矩阵关联到求解系统中，初始化集群版的CG方法时，每个进程使用局部的CSR格式矩阵。

接口定义

C Interface：

int KmlScaissCgInitStripesSI(KmlScasolverTask **handle, int n, int nstripes, const int *stripeWidth,const int *stripeRow, const float * const *a, const int * const *ja, const int * const *ia, MPI_Comm comm);

int KmlScaissCgInitStripesDI(KmlScasolverTask **handle, int n, int nstripes, const int *stripeWidth, const int *stripeRow, const double * const *a, const int * const *ja, const int * const *ia, MPI_Comm comm);

 版权所有 © 华为技术有限公司

参数

参数名	类型	描述	输入/输出
handle	KmlScasolverTask **	求解器句柄，handle不可为空指针。	输入/输出
n	int	n为矩阵A的维度。	输入
nstripes	int	nstripes为存储在当前MPI进程中的矩阵数，目前只能取1。	输入
stripeWidth	int *	stripeWidth为当前MPI进程中的矩阵行数。该变量的生命周期与handle相同。	输入
stripeRow	int *	stripeRow为当前MPI进程中的矩阵的全局行索引，即首行在全局矩阵中对应行数。该数组的生命周期与handle相同。	输入
a	• 在KmlScaissCgInitStripesSI中为float *。 • 在KmlScaissCgInitStripesDI中为double *。	a为维度为非零元素的个数的数组，a[i]为第i个非零元的值。该数组的生命周期与handle相同。	输入
ja	int *	ja为维度为非零元素的个数的数组，ja[i]为第i个非零元的列号。该数组的生命周期与handle相同。	输入
ia	int *	ia为stripeWidth+1维数组，ia[i]为第i行第一个非零元在所有非零元中的索引，ia[0]=0。该数组的生命周期与handle相同。	输入
comm	MPI_Comm	为MPI进程通信域。	输入

返回值

返回值	类型	描述
KMLSS_NO_ERROR	int	正常执行。
KMLSS_NULL_ARGUMENT	int	handle，stripeWidth，stripeRow，a，ja，ia中存在空参数。
KMLSS_BAD_N	int	参数n不是正整数。
KMLSS_NOT_IMPLEMENTED	int	参数nstripes不等于1。

版权所有 © 华为技术有限公司

返回值	类型	描述
KMLSS_NO_MEMORY	int	内存不足。

依赖

#include "kml_scaiss.h"

示例

C Interface：

```
MPI_Init(NULL, NULL);
int size, rank;
MPI_Comm_size(MPI_COMM_WORLD, &size);//获取总进程数
MPI_Comm_rank(MPI_COMM_WORLD, &rank);//获取当前进程标识

int mat_size = 8;
int n = mat_size / size;
int n_beg = n * rank;
if (n * size != mat_size && rank == (size - 1)) {
    n = mat_size - n * rank;
}
int ja[26] = { 0, 3, 4, 1, 2, 3, 5, 1, 2, 7, 0, 1, 3, 6, 0, 4, 5, 1, 4, 5, 7, 3, 6, 2, 5, 7 };
double a[26] = { 1.0, 1.0, 2.0, 9.0, 2.0, 1.0, -3.0, 2.0, 3.0, 2.0, 1.0, 1.0, 9.0, -5.0, 2.0, 6.0, 1.0, -3.0, 1.0, 4.0, 1.0,
-5.0, 7.0, 2.0, 1.0, 2.0 };
int ia[9] = { 0, 3, 7, 10, 14, 17, 21, 23, 26 };
int a_beg = ia[n_beg];
for (int i = n_beg; i < (n_beg + n + 1); i++) {
    ia[i] -= a_beg;
}

/* Internal KML_SCAISS structure */
KmlScasolverTask *handle;

/* KML_SCAISS control parameters */
int error;            /* Output error handle */

/* Create data structures */
const double *a_holder = &a[a_beg];
const int *ja_holder = &ja[a_beg];
const int *ia_holder = &ia[n_beg];
error = KmlScaissCgInitStripesDI(&handle, mat_size, 1, &n, &n_beg, &a_holder, &ja_holder, &ia_holder,
MPI_COMM_WORLD);
```

6.5.3.4.1.2 KmlScaissCgInitWithoutMat?I

初始化数据结构，用于无矩阵求解的初始化，需要与用户自定义矩阵向量乘功能一并使用

接口定义

C Interface：

int KmlScaissCgInitWithoutMatSI(KmlScasolverTask **handle, int nstripes, const int *stripeWidth,MPI_Comm comm);

int KmlScaissCgInitWithoutMatDI(KmlScasolverTask **handle, int nstripes, const int *stripeWidth,MPI_Comm comm);

 版权所有 © 华为技术有限公司

参数

参数名	类型	描述	输入/输出
handle	KmlScasolverTask **	求解器句柄，handle不可为空指针。	输入/输出
nstripes	int	nstripes为存储在当前MPI进程中的矩阵数，目前只能取1。	输入
stripeWidth	int *	stripeWidth为当前MPI进程中的矩阵行数。该变量的生命周期与handle相同。	输入
comm	MPI_Comm	为MPI进程通信域。	输入

返回值

返回值	类型	描述
KMLSS_NO_ERROR	int	正常执行。
KMLSS_NULL_ARGUMENT	int	handle，stripeWidth中存在空参数。
KMLSS_NOT_IMPLEMENTED	int	参数nstripes不等于1。
KMLSS_NO_MEMORY	int	内存不足。

6.5.3.4.1.3 KmlScaissCgSetUserPreconditioner?I

关联用户自定义预条件子回调函数，如不使用自定义的预条件子，该接口无需使用。

接口定义

C Interface：

int KmlScaissCgSetUserPreconditionerSI(KmlScasolverTask **handle, void *ustruct, int (*fptr)(void *ustruct, float *x));

int KmlScaissCgSetUserPreconditionerDI(KmlScasolverTask **handle, void *ustruct, int (*fptr)(void *ustruct, double *x));

版权所有 © 华为技术有限公司

参数

参数名	类型	描述	输入/输出
handle	KmlScasolverTask **	求解器句柄，传入之前步骤的变量。	输入/输出
ustruct	void *	指向用户数据的指针，每次调用预条件子时作为第一个参数传递。	输入
(*fptr)	int	用户自定义的函数指针，指向回调函数，该回调函数进行预条件子与其第二参数间的计算。	输入

返回值

返回值	类型	描述
KMLSS_NO_ERROR	int	正常执行。
KMLSS_NULL_ARGUMENT	int	handle，fptr中存在空参数。

依赖

#include "kml_scaiss.h"

示例

C Interface：

```
// USER data struct
typedef struct {
    int n;
    const int *ia;
    const int *ja;
    const double *a;
} duser;

int diagonal_preconditioner(void *usr, double *x)
{
    duser *u = (duser *)usr;
    int i, j;

    /* Apply diagonal preconditioner*/
    for (i = 0; i < u->n; i++) {
        for (j = u->ia[i]; j < u->ia[i + 1]; j++) {
            if (u->ja[j] == i)
                x[i] /= u->a[j];
        }
    }
    return 0;
}

MPI_Init(NULL, NULL);
int size, rank;
MPI_Comm_size(MPI_COMM_WORLD, &size);//获取总进程数
```

 版权所有 © 华为技术有限公司

```
MPI_Comm_rank(MPI_COMM_WORLD, &rank);//获取当前进程标识

int mat_size = 8;
int n = mat_size / size;
int n_beg = n * rank;
if (n * size != mat_size && rank == (size - 1)) {
    n = mat_size - n * rank;
}
int ja[26] = { 0, 3, 4, 1, 2, 3, 5, 1, 2, 7, 0, 1, 3, 6, 0, 4, 5, 1, 4, 5, 7, 3, 6, 2, 5, 7 };
double a[26] = { 1.0, 1.0, 2.0, 9.0, 2.0, 1.0, -3.0, 2.0, 3.0, 2.0, 1.0, 1.0, 9.0, -5.0, 2.0, 6.0, 1.0, -3.0, 1.0, 4.0, 1.0,
-5.0, 7.0, 2.0, 1.0, 2.0 };
int ia[9] = { 0, 3, 7, 10, 14, 17, 21, 23, 26 };
int a_beg = ia[n_beg];
for (int i = n_beg; i < (n_beg + n + 1); i++) {
    ia[i] -= a_beg;
}

/* Internal KML_SCAISS structure */
KmlScasolverTask *handle;

/* KML_SCAISS control parameters */
int error;              /* Output error handle */

/* Create data structures */
const double *a_holder = &a[a_beg];
const int *ja_holder = &ja[a_beg];
const int *ia_holder = &ia[n_beg];
error = KmlScaissCgInitStripesDI(&handle, mat_size, 1, &n, &n_beg, &a_holder, &ja_holder, &ia_holder,
MPI_COMM_WORLD);

duser prea;
prea.n = n;
prea.ia = &ia[n_beg];
prea.ja = &ja[a_beg];
prea.a = &a[a_beg];

error = KmlScaissCgSetUserPreconditionerDI(&handle, &prea, &diagonal_preconditioner);
if (error != 0) {
    printf("\nERROR in KmlScaissCgSetUserPreconditionerDI: %d", error);
    return 1;
}
```

6.5.3.4.1.4 KmlScaissCgSet?I?

设置迭代求解的相关参数。

接口定义

C Interface：

int KmlScaissCgSetSII(KmlScasolverTask **handle, enum KmlSolverParam param, const int *data, int nd);

int KmlScaissCgSetSIS(KmlScasolverTask **handle, enum KmlSolverParam param, const float *data, int nd);

int KmlScaissCgSetDII(KmlScasolverTask **handle, enum KmlSolverParam param, const int *data, int nd);

int KmlScaissCgSetDID(KmlScasolverTask **handle, enum KmlSolverParam param, const double *data, int nd);

 版权所有 © 华为技术有限公司

参数

参数名	类型	描述	输入/输出
handle	KmlScasolverTask **	求解器句柄，传入之前步骤的变量。	输入/输出
param	enum KmlSolverParam	• KMLSS_THRESHOLD表示迭代残差相对阈值。 • KMLSS_ABS_TOLERANCE表示迭代残差绝对阈值。 • KMLSS_MAX_ITERATION_COUNT表示最大迭代次数。 • KMLSS_PRECONDITIONER_TYPE表示预条件子类型。 • KMLSS_VECTOR_NORM_TYPE表示迭代残差的计算范数。 • KMLSS_MATRIX_FORMAT表示矩阵对角块稀疏存储格式。 • KMLSS_OFFDIAG_MATRIX_FORMAT表示矩阵非对角块稀疏存储格式。	输入
data	• 在KmlScaissCgSetSII/KmlScaissCgSetDII中为const int *。 • 在KmlScaissCgSetSIS中为const float *。 • 在KmlScaissCgSetDID中为const double *。	迭代求解相关参数数组（如阈值、最大迭代次数等）。 当前可选的计算范数： • 1-NORM：KMLSS_L1。 • 2-NORM：KMLSS_L2。 当前可选的稀疏矩阵存储格式： • CSR格式：KMLSS_CSR。 • ELLPACK格式：KMLSS_ELL。 • COO格式：KMLSS_COO。 • ELLPACK_COO混合格式：KMLSS_SELLCOO。 当前可选择的预条件子类型： • JACOBI预条件子：KMLSS_JACOBI。 • BLOCK JACOBI预条件子：KMLSS_BJACOBI。 • AMG预条件子：KMLSS_AMG。	输入
nd	int	data数组元素个数。	输入

版权所有 © 华为技术有限公司

返回值

返回值	类型	描述
KMLSS_NO_ERROR	int	正常执行。
KMLSS_DATA_SIZE	int	参数nd不等于1。
KMLSS_NULL_ARGUMENT	int	handle，data中存在空参数。
KMLSS_BAD_SELECTOR	int	param为无效参数。
KMLSS_BAD_PRECONDITIONER	int	选择的预条件子暂未实现。

依赖

#include "kml_scaiss.h"

示例

C Interface：

```
MPI_Init(NULL, NULL);
int size, rank;
MPI_Comm_size(MPI_COMM_WORLD, &size);//获取总进程数
MPI_Comm_rank(MPI_COMM_WORLD, &rank);//获取当前进程标识

int mat_size = 8;
int n = mat_size / size;
int n_beg = n * rank;
if (n * size != mat_size && rank == (size - 1)) {
    n = mat_size - n * rank;
}
int ja[26] = { 0, 3, 4, 1, 2, 3, 5, 1, 2, 7, 0, 1, 3, 6, 0, 4, 5, 1, 4, 5, 7, 3, 6, 2, 5, 7 };
double a[26] = { 1.0, 1.0, 2.0, 9.0, 2.0, 1.0, -3.0, 2.0, 3.0, 2.0, 1.0, 1.0, 9.0, -5.0, 2.0, 6.0, 1.0, -3.0, 1.0, 4.0, 1.0,
-5.0, 7.0, 2.0, 1.0, 2.0 };
int ia[9] = { 0, 3, 7, 10, 14, 17, 21, 23, 26 };
int a_beg = ia[n_beg];
for (int i = n_beg; i < (n_beg + n + 1); i++) {
    ia[i] -= a_beg;
}

/* Internal KML_SCAISS structure */
KmlScasolverTask *handle;

/* KML_SCAISS control parameters */
int error;              /* Output error handle */

/* Create data structures */
const double *a_holder = &a[a_beg];
const int *ja_holder = &ja[a_beg];
const int *ia_holder = &ia[n_beg];
error = KmlScaissCgInitStripesDI(&handle, mat_size, 1, &n, &n_beg, &a_holder, &ja_holder, &ia_holder,
MPI_COMM_WORLD);

double eps = 1e-4;
error = KmlScaissCgSetDID(&handle, KMLSS_THRESHOLD, &eps, 1);
if (error != 0) {
    printf("\nERROR in KmlScaissCgSetDID with KMLSS_THRESHOLD: %d", error);
    return 1;
```

 版权所有 © 华为技术有限公司

```
}
int max_iters = 2000;
error = KmlScaissCgSetDII(&handle, KMLSS_MAX_ITERATION_COUNT, &max_iters, 1);
if (error != 0) {
    printf("\nERROR in KmlScaissCgSetDII with KMLSS_MAX_ITERATION_COUNT: %d", error);
    return 1;
}
int precond = KMLSS_BJACOBI;
error = KmlScaissCgSetDII(&handle, KMLSS_PRECONDITIONER_TYPE, &precond, 1);
if (error != 0) {
    printf("\nERROR in KmlScaissCgSetDII with KMLSS_PRECONDITIONER_TYPE: %d", error);
    return 1;
}
```

6.5.3.4.1.5 KmlScaissCgSetUserSpmv?I

关联用户自定义矩阵向量乘函数，需要与无矩阵初始化接口一起使用。

接口定义

C Interface：

int KmlScaissCgSetUserSpmvSI(KmlScasolverTask **handle, void *ustruct, int (*fptr)(void *ustruct, const float *x, float *y));

int KmlScaissCgSetUserSpmvDI(KmlScasolverTask **handle, void *ustruct, int (*fptr)(void *ustruct, const double *x, double *y));

参数

参数名	类型	描述	输入/输出
handle	KmlScasolverTask **	求解器句柄，传入之前步骤的变量。	输入/输出
ustruct	void *	指向用户数据的指针，每次调用预条件子时作为第一个参数传递。	输入
(*fptr)	int	用户自定义的函数指针，指向矩阵向量乘函数，该矩阵向量乘函数执行矩阵与输入向量x的乘法，并将结果存储在输出向量y中。	输入

返回值

返回值	类型	描述
KMLSS_NO_ERROR	int	正常执行。
KMLSS_DATA_SIZE	int	参数nd不等于1。
KMLSS_NULL_ARGUMENT	int	handle，data中存在空参数。

版权所有 © 华为技术有限公司

6.5.3.4.1.6 KmlScaissCgPcSet?II

设置迭代求解的预条件子相关参数，当前仅支持为block jacobi预条件子设置相关参数。

接口定义

C Interface：

int KmlScaissCgPcSetSII(KmlScasolverTask **handle, enum KmlSolverParam param, const int *data, int nd);

int KmlScaissCgPcSetDII(KmlScasolverTask **handle, enum KmlSolverParam param, const int *data, int nd);

参数

参数名	类型	描述	输入/输出
handle	KmlScasolverTask **	求解器句柄，传入之前步骤的变量。	输入/输出
param	enum KmlSolverParam	• KMLSS_BLOCK_METHOD表示每个分块中的预条件子类型。 • KMLSS_NUM_BLOCKS表示每个进程中的分块数量。 • KMLSS_BLOCK_SIZES表示每个进程中的分块大小。 以下为预条件AMG相关设置项： • KMLSS_AMG_CYCLE表示AMG中CYCLE类型。 • KMLSS_AMG_COARSENING表示粗化算子类型。 • KMLSS_AMG_INTERPOLATION表示插值算子类型。 • KMLSS_AMG_CONNECTION表示粗网格构造方法。	输入
data	const int *	预条件子相关参数（如预条件子类型、分块数量、分块大小）。 当前可选择的预条件子类型： • ILU预条件子：KMLSS_ILU。 • ICC预条件子：KMLSS_ICC。 • SOR预条件子：KMLSS_SOR。 • ILUT预条件子：KMLSS_ILUT。 • 预条件AMG配置项描述具体见**表1 预条件AMG中具体配置项描述**所示。	输入

版权所有 © 华为技术有限公司

参数名	类型	描述	输入/输出
nd	int	data数组元素个数。	输入

表 6-56 预条件 AMG 中具体配置项描述

配置项	可选值
KMLSS_AMG_CYCLE	• KMLSS_AMG_V_CYCLE • KMLSS_AMG_K_CYCLE
KMLSS_AMG_COARSENING	• KMLSS_AMG_NOTAY • KMLSS_AMG_PMIS • KMLSS_AMG_HMIS
KMLSS_AMG_INTERPOLATION	• KMLSS_AMG_AGGREGATION • KMLSS_AMG_DIRECT
KMLSS_AMG_CONNECTION	• KMLSS_AMG_NEGATIVE • KMLSS_AMG_SEPARATE • KMLSS_AMG_ABS

返回值

返回值	类型	描述
KMLSS_NO_ERROR	int	正常执行。
KMLSS_DATA_SIZE	int	参数nd不等于1。
KMLSS_NULL_ARGUMENT	int	handle，data中存在空参数。
KMLSS_BAD_SELECTOR	int	param为无效参数。
KMLSS_BAD_PRECONDITIONER	int	选择的预条件子暂未实现。

依赖

#include "kml_scaiss.h"

示例

C Interface：

```
MPI_Init(NULL, NULL);
int size, rank;
```

 版权所有 © 华为技术有限公司

```
MPI_Comm_size(MPI_COMM_WORLD, &size);//获取总进程数
MPI_Comm_rank(MPI_COMM_WORLD, &rank);//获取当前进程标识

int mat_size = 8;
int n = mat_size / size;
int n_beg = n * rank;
if (n * size != mat_size && rank == (size - 1)) {
    n = mat_size - n * rank;
}
int ja[26] = { 0, 3, 4, 1, 2, 3, 5, 1, 2, 7, 0, 1, 3, 6, 0, 4, 5, 1, 4, 5, 7, 3, 6, 2, 5, 7 };
double a[26] = { 1.0, 1.0, 2.0, 9.0, 2.0, 1.0, -3.0, 2.0, 3.0, 2.0, 1.0, 1.0, 9.0, -5.0, 2.0, 6.0, 1.0, -3.0, 1.0, 4.0, 1.0,
-5.0, 7.0, 2.0, 1.0, 2.0 };
int ia[9] = { 0, 3, 7, 10, 14, 17, 21, 23, 26 };
int a_beg = ia[n_beg];
for (int i = n_beg; i < (n_beg + n + 1); i++) {
    ia[i] -= a_beg;
}

/* Internal KML_SCAISS structure */
KmlScasolverTask *handle;

/* KML_SCAISS control parameters */
int error;              /* Output error handle */

/* Create data structures */
const double *a_holder = &a[a_beg];
const int *ja_holder = &ja[a_beg];
const int *ia_holder = &ia[n_beg];
error = KmlScaissCgInitStripesDI(&handle, mat_size, 1, &n, &n_beg, &a_holder, &ja_holder, &ia_holder,
MPI_COMM_WORLD);

double eps = 1e-4;
error = KmlScaissCgSetDID(&handle, KMLSS_THRESHOLD, &eps, 1);
if (error != 0) {
    printf("\nERROR in KmlScaissCgSetDID with KMLSS_THRESHOLD: %d", error);
    return 1;
}
int max_iters = 2000;
error = KmlScaissCgSetDII(&handle, KMLSS_MAX_ITERATION_COUNT, &max_iters, 1);
if (error != 0) {
    printf("\nERROR in KmlScaissCgSetDII with KMLSS_MAX_ITERATION_COUNT: %d", error);
    return 1;
}
int precond = KMLSS_BJACOBI;
error = KmlScaissCgSetDII(&handle, KMLSS_PRECONDITIONER_TYPE, &precond, 1);
if (error != 0) {
    printf("\nERROR in KmlScaissCgSetDII with KMLSS_PRECONDITIONER_TYPE: %d", error);
    return 1;
}
int num_blocks = 1;
error = KmlScaissCgPcSetDII(&handle, KMLSS_NUM_BLOCKS, &num_blocks, 1);
if (error != 0) {
    printf("ERROR in KmlScaissCgPcSetDII: %d\n", error);
    return 1;
}
int subprecond = KMLSS_ICC;
error = KmlScaissCgPcSetDII(&handle, KMLSS_BLOCK_METHOD, &subprecond, 1);
if (error != 0) {
    printf("ERROR in KmlScaissCgPcSetDII: %d\n", error);
    return 1;
}
```

6.5.3.4.1.7 KmlScaissCgSubPcSet?I?

设置迭代求解的block jacobi中子预条件子相关参数。

 版权所有 © 华为技术有限公司

接口定义

C Interface：

int KmlScaissCgSubPcSetSII(KmlScasolverTask **handle, enum KmlSolverPreconditionerParam param, int subn, const int *data, int nd);

int KmlScaissCgSubPcSetDII(KmlScasolverTask **handle, enum KmlSolverPreconditionerParam param, int subn, const int *data, int nd);

int KmlScaissCgSubPcSetSIS(KmlScasolverTask **handle, enum KmlSolverPreconditionerParam param, int subn, const float *data, int nd);

int KmlScaissCgSubPcSetDID(KmlScasolverTask **handle, enum KmlSolverPreconditionerParam param, int subn, const double *data, int nd);

参数

参数名	类型	描述	输入/输出
handle	KmlScasolverTask **	求解器句柄，传入之前步骤的变量。	输入/输出
param	enum KmlSolverParam	• KML_RELAXATION_FACTOR表示SOR预条件子相关参数。 • KMLSS_DROP_FILL表示ILUT预条件子每行最多保留元素个数。 • KMLSS_DROP_TOL表示ILUT预条件子保留元素的阈值。	输入
subn	在KmlScaissCgSubPcSetSIS、KmlScaissCgSubPcSetSII、KmlScaissCgSubPcSetDII、KmlScaissCgSubPcSetDID中为int。	子预条件子索引。	输入

版权所有 © 华为技术有限公司

参数名	类型	描述	输入/输出
data	• 在KmlScaissCgSubPcSetSII、KmlScaissCgSubPcSetDII中为const int *。 • 在KmlScaissCgSubPcSetSIS中为const float *。 • 在KmlScaissCgSubPcSetDID中为const double *。	预条件子相关参数。	输入
nd	在KmlScaissCgPcSetSII、KmlScaissCgPcSetSIS、KmlScaissCgPcSetDII、KmlScaissCgPcSetDID中为int。	data数组元素个数。	输入

返回值

返回值	类型	描述
KMLSS_NO_ERROR	int	正常执行。
KMLSS_DATA_SIZE	int	参数nd不等于1。
KMLSS_NULL_ARGUMENT	int	handle，data中存在空参数。
KMLSS_BAD_SELECTOR	int	param为无效参数。
KMLSS_BAD_PRECONDITIONER	int	选择的预条件子暂未实现。

6.5.3.4.1.8 KmlScaissCgAnalyze?I

分析稀疏矩阵并确定不同进程之间的通信关系。

版权所有 © 华为技术有限公司

接口定义

C Interface：

int KmlScaissCgAnalyzeSI(KmlScasolverTask **handle);

int KmlScaissCgAnalyzeDI(KmlScasolverTask **handle);

参数

参数名	类型	描述	输入/输出
handle	KmlScasolverTask **	求解器句柄，传入之前步骤的变量。	输入/输出

返回值

返回值	类型	描述
KMLSS_NO_ERROR	int	正常执行。
KMLSS_NULL_ARGUMENT	int	handle为空参数。

依赖

#include "kml_scaiss.h"

示例

C Interface：

```
MPI_Init(NULL, NULL);
int size, rank;
MPI_Comm_size(MPI_COMM_WORLD, &size);//获取总进程数
MPI_Comm_rank(MPI_COMM_WORLD, &rank);//获取当前进程标识

int mat_size = 8;
int n = mat_size / size;
int n_beg = n * rank;
if (n * size != mat_size && rank == (size - 1)) {
    n = mat_size - n * rank;
}
int ja[26] = { 0, 3, 4, 1, 2, 3, 5, 1, 2, 7, 0, 1, 3, 6, 0, 4, 5, 1, 4, 5, 7, 3, 6, 2, 5, 7 };
double a[26] = { 1.0, 1.0, 2.0, 9.0, 2.0, 1.0, -3.0, 2.0, 3.0, 2.0, 1.0, 1.0, 9.0, -5.0, 2.0, 6.0, 1.0, -3.0, 1.0, 4.0, 1.0,
-5.0, 7.0, 2.0, 1.0, 2.0 };
int ia[9] = { 0, 3, 7, 10, 14, 17, 21, 23, 26 };
int a_beg = ia[n_beg];
for (int i = n_beg; i < (n_beg + n + 1); i++) {
    ia[i] -= a_beg;
}

/* Internal KML_SCAISS structure */
KmlScasolverTask *handle;

/* KML_SCAISS control parameters */
```


版权所有 © 华为技术有限公司

```
int error;            /* Output error handle */

/* Create data structures */
const double *a_holder = &a[a_beg];
const int *ja_holder = &ja[a_beg];
const int *ia_holder = &ia[n_beg];
error = KmlScaissCgInitStripesDI(&handle, mat_size, 1, &n, &n_beg, &a_holder, &ja_holder, &ia_holder,
MPI_COMM_WORLD);

double eps = 1e-4;
error = KmlScaissCgSetDID(&handle, KMLSS_THRESHOLD, &eps, 1);
if (error != 0) {
    printf("\nERROR in KmlScaissCgSetDID with KMLSS_THRESHOLD: %d", error);
    return 1;
}
int max_iters = 2000;
error = KmlScaissCgSetDII(&handle, KMLSS_MAX_ITERATION_COUNT, &max_iters, 1);
if (error != 0) {
    printf("\nERROR in KmlScaissCgSetDII with KMLSS_MAX_ITERATION_COUNT: %d", error);
    return 1;
}
int precond = KMLSS_BJACOBI;
error = KmlScaissCgSetDII(&handle, KMLSS_PRECONDITIONER_TYPE, &precond, 1);
if (error != 0) {
    printf("\nERROR in KmlScaissCgSetDII with KMLSS_PRECONDITIONER_TYPE: %d", error);
    return 1;
}
int num_blocks = 1;
error = KmlScaissCgPcSetDII(&handle, KMLSS_NUM_BLOCKS, &num_blocks, 1);
if (error != 0) {
    printf("ERROR in KmlScaissCgPcSetDII: %d\n", error);
    return 1;
}
int subprecond = KMLSS_ICC;
error = KmlScaissCgPcSetDII(&handle, KMLSS_BLOCK_METHOD, &subprecond, 1);
if (error != 0) {
    printf("ERROR in KmlScaissCgPcSetDII: %d\n", error);
    return 1;
}
error = KmlScaissCgAnalyzeDI(&handle);
if (error != 0) {
    printf("ERROR in KmlScaissCgAnalyzeDI: %d\n", error);
    return 1;
}
```

6.5.3.4.1.9 KmlScaissCgFactorize?I

在求解器内部对矩阵进行赋值，并计算预条件子。无论是否使用预条件子，都需要调用该接口。

接口定义

C Interface：

int KmlScaissCgFactorizeSI(KmlScasolverTask **handle);

int KmlScaissCgFactorizeDI(KmlScasolverTask **handle);

 版权所有 © 华为技术有限公司

参数

参数名	类型	描述	输入/输出
handle	KmlScasolverTask **	求解器句柄，传入之前步骤的变量。	输入/输出

返回值

返回值	类型	描述
KMLSS_NO_ERROR	int	正常执行。
KMLSS_NULL_ARGUMENT	int	handle为空参数。
KMLSS_ZERO_DIAGONAL_ELEMENT	int	矩阵对角线存在零元。

依赖

#include "kml_scaiss.h"

示例

C Interface：

```
MPI_Init(NULL, NULL);
int size, rank;
MPI_Comm_size(MPI_COMM_WORLD, &size);//获取总进程数
MPI_Comm_rank(MPI_COMM_WORLD, &rank);//获取当前进程标识

int mat_size = 8;
int n = mat_size / size;
int n_beg = n * rank;
if (n * size != mat_size && rank == (size - 1)) {
    n = mat_size - n * rank;
}
int ja[26] = { 0, 3, 4, 1, 2, 3, 5, 1, 2, 7, 0, 1, 3, 6, 0, 4, 5, 1, 4, 5, 7, 3, 6, 2, 5, 7 };
double a[26] = { 1.0, 1.0, 2.0, 9.0, 2.0, 1.0, -3.0, 2.0, 3.0, 2.0, 1.0, 1.0, 9.0, -5.0, 2.0, 6.0, 1.0, -3.0, 1.0, 4.0, 1.0,
-5.0, 7.0, 2.0, 1.0, 2.0 };
int ia[9] = { 0, 3, 7, 10, 14, 17, 21, 23, 26 };
int a_beg = ia[n_beg];
for (int i = n_beg; i < (n_beg + n + 1); i++) {
    ia[i] -= a_beg;
}

/* Internal KML_SCAISS structure */
KmlScasolverTask *handle;

/* KML_SCAISS control parameters */
int error;              /* Output error handle */

/* Create data structures */
const double *a_holder = &a[a_beg];
const int *ja_holder = &ja[a_beg];
const int *ia_holder = &ia[n_beg];
```

 版权所有 © 华为技术有限公司

```
error = KmlScaissCgInitStripesDI(&handle, mat_size, 1, &n, &n_beg, &a_holder, &ja_holder, &ia_holder,
MPI_COMM_WORLD);

double eps = 1e-4;
error = KmlScaissCgSetDID(&handle, KMLSS_THRESHOLD, &eps, 1);
if (error != 0) {
    printf("\nERROR in KmlScaissCgSetDID with KMLSS_THRESHOLD: %d", error);
    return 1;
}
int max_iters = 2000;
error = KmlScaissCgSetDII(&handle, KMLSS_MAX_ITERATION_COUNT, &max_iters, 1);
if (error != 0) {
    printf("\nERROR in KmlScaissCgSetDII with KMLSS_MAX_ITERATION_COUNT: %d", error);
    return 1;
}
int precond = KMLSS_BJACOBI;
error = KmlScaissCgSetDII(&handle, KMLSS_PRECONDITIONER_TYPE, &precond, 1);
if (error != 0) {
    printf("\nERROR in KmlScaissCgSetDII with KMLSS_PRECONDITIONER_TYPE: %d", error);
    return 1;
}
int num_blocks = 1;
error = KmlScaissCgPcSetDII(&handle, KMLSS_NUM_BLOCKS, &num_blocks, 1);
if (error != 0) {
    printf("ERROR in KmlScaissCgPcSetDII: %d\n", error);
    return 1;
}
int subprecond = KMLSS_ICC;
error = KmlScaissCgPcSetDII(&handle, KMLSS_BLOCK_METHOD, &subprecond, 1);
if (error != 0) {
    printf("ERROR in KmlScaissCgPcSetDII: %d\n", error);
    return 1;
}
error = KmlScaissCgAnalyzeDI(&handle);
if (error != 0) {
    printf("ERROR in KmlScaissCgAnalyzeDI: %d\n", error);
    return 1;
}
error = KmlScaissCgFactorizeDI(&handle);
if (error != 0) {
    printf("ERROR in KmlScaissCgFactorizeDI: %d\n", error);
    return 1;
}
```

6.5.3.4.1.10 KmlScaissCgSolve?I

求解线性代数方程组，每个进程读取并存储全局求解结果。

接口定义

C Interface：

int KmlScaissCgSolveSI(KmlScasolverTask **handle, int nb, float *x, int ldx, const float *b, int ldb);

int KmlScaissCgSolveDI(KmlScasolverTask **handle, int nb, double *x, int ldx, const double *b, int ldb);

 版权所有 © 华为技术有限公司

参数

参数名	类型	描述	输入/输出
handle	KmlScasolverTask **	求解器句柄，传入之前步骤的变量。	输入/输出
nb	int	右端项矢量的个数，当前nb只能为1。	输入
x	• 在KmlScaissCgSolveSI中为float *。 • 在KmlScaissCgSolveDI中为double *。	读取并存储全局求解结果，其元素个数为全局矩阵大小n。	输入/输出
ldx	int	x的leading dimension大小，当前参数未使用。	输入
b	• 在KmlScaissCgSolveSI中为const float *。 • 在KmlScaissCgSolveDI中为const double *。	读取进程对应的局部右端项，其元素个数为当前进程对应的矩阵的行数，即stripeWidth。	输入
ldb	int	b的leading dimension大小，当前参数未使用。	输入

返回值

返回值	类型	描述
KMLSS_NO_ERROR	int	正常执行。
KMLSS_BAD_NB	int	参数nb不等于1。
KMLSS_NO_MEMORY	int	内存不足。
KMLSS_NULL_ARGUMENT	int	handle，x，b中存在空参数。

依赖

```
#include "kml_scaiss.h"
```

 版权所有 © 华为技术有限公司

示例

C Interface：

```
MPI_Init(NULL, NULL);
int size, rank;
MPI_Comm_size(MPI_COMM_WORLD, &size);//获取总进程数
MPI_Comm_rank(MPI_COMM_WORLD, &rank);//获取当前进程标识

int mat_size = 8;
int n = mat_size / size;
int n_beg = n * rank;
if (n * size != mat_size && rank == (size - 1)) {
    n = mat_size - n * rank;
}
int ja[26] = { 0, 3, 4, 1, 2, 3, 5, 1, 2, 7, 0, 1, 3, 6, 0, 4, 5, 1, 4, 5, 7, 3, 6, 2, 5, 7 };
double a[26] = { 1.0, 1.0, 2.0, 9.0, 2.0, 1.0, -3.0, 2.0, 3.0, 2.0, 1.0, 1.0, 9.0, -5.0, 2.0, 6.0, 1.0, -3.0, 1.0, 4.0, 1.0,
-5.0, 7.0, 2.0, 1.0, 2.0 };
int ia[9] = { 0, 3, 7, 10, 14, 17, 21, 23, 26 };
int a_beg = ia[n_beg];
for (int i = n_beg; i < (n_beg + n + 1); i++) {
    ia[i] -= a_beg;
}

/* Internal KML_SCAISS structure */
KmlScasolverTask *handle;

/* KML_SCAISS control parameters */
int error;              /* Output error handle */

/* Create data structures */
const double *a_holder = &a[a_beg];
const int *ja_holder = &ja[a_beg];
const int *ia_holder = &ia[n_beg];
error = KmlScaissCgInitStripesDI(&handle, mat_size, 1, &n, &n_beg, &a_holder, &ja_holder, &ia_holder,
MPI_COMM_WORLD);

double eps = 1e-4;
error = KmlScaissCgSetDID(&handle, KMLSS_THRESHOLD, &eps, 1);
if (error != 0) {
    printf("\nERROR in KmlScaissCgSetDID with KMLSS_THRESHOLD: %d", error);
    return 1;
}
int max_iters = 2000;
error = KmlScaissCgSetDII(&handle, KMLSS_MAX_ITERATION_COUNT, &max_iters, 1);
if (error != 0) {
    printf("\nERROR in KmlScaissCgSetDII with KMLSS_MAX_ITERATION_COUNT: %d", error);
    return 1;
}
int precond = KMLSS_BJACOBI;
error = KmlScaissCgSetDII(&handle, KMLSS_PRECONDITIONER_TYPE, &precond, 1);
if (error != 0) {
    printf("\nERROR in KmlScaissCgSetDII with KMLSS_PRECONDITIONER_TYPE: %d", error);
    return 1;
}
int num_blocks = 1;
error = KmlScaissCgPcSetDII(&handle, KMLSS_NUM_BLOCKS, &num_blocks, 1);
if (error != 0) {
    printf("ERROR in KmlScaissCgPcSetDII: %d\n", error);
    return 1;
}
int subprecond = KMLSS_ICC;
error = KmlScaissCgPcSetDII(&handle, KMLSS_BLOCK_METHOD, &subprecond, 1);
if (error != 0) {
    printf("ERROR in KmlScaissCgPcSetDII: %d\n", error);
    return 1;
}
error = KmlScaissCgAnalyzeDI(&handle);
if (error != 0) {
    printf("ERROR in KmlScaissCgAnalyzeDI: %d\n", error);
```

 版权所有 © 华为技术有限公司

```
    return 1;
}
error = KmlScaissCgFactorizeDI(&handle);
if (error != 0) {
    printf("ERROR in KmlScaissCgFactorizeDI: %d\n", error);
    return 1;
}

int nrhs = 1;
int ldx = mat_size, ldb = n;
double b[8] = { 4.0, 9.0, 7.0, 6.0, 9.0, 3.0, 2.0, 5.0 };
double x[8] = { 0.0, 0.0, 0.0, 0.0, 0.0, 0.0, 0.0, 0.0 };
error = KmlScaissCgSolveDxDI(&handle, nrhs, &x[0], ldx, &b[n_beg], ldb);
if (error != 0) {
    printf("ERROR in KmlScaissCgSolveDI: %d\n", error);
    return 1;
}
```

6.5.3.4.1.11 KmlScaissCgSolveDx?I

求解线性代数方程组，每个进程读取并存储当前进程对应的求解结果。

接口定义

C Interface：

int KmlScaissCgSolveDxSI(KmlScasolverTask **handle, int nb, float *x, int ldx, const float *b, int ldb);

int KmlScaissCgSolveDxDI(KmlScasolverTask **handle, int nb, double *x, int ldx, const double *b, int ldb);

参数

参数名	类型	描述	输入/输出
handle	KmlScasolverTask **	求解器句柄，传入之前步骤的变量。	输入/输出
nb	int	右端项矢量的个数，当前nb只能为1。	输入
x	• 在KmlScaissCgSolveSI中为float *。 • 在KmlScaissCgSolveDI中为double *。	读取并存储当前进程对应的求解结果，其元素个数为当前进程对应的矩阵的行数，即stripeWidth。	输入/输出
ldx	int	x的leading dimension大小，当前参数未使用。	输入

版权所有 © 华为技术有限公司

参数名	类型	描述	输入/输出
b	• 在KmlScaissCgSolveSI中为const float * • 在KmlScaissCgSolveDI中为const double *。	读取进程对应的局部右端项，其元素个数为当前进程对应的矩阵的行数，即stripeWidth。	输入
ldb	int	b的leading dimension大小，当前参数未使用。	输入

返回值

返回值	类型	描述
KMLSS_NO_ERROR	int	正常执行。
KMLSS_BAD_NB	int	参数nb不等于1。
KMLSS_NO_MEMORY	int	内存不足。
KMLSS_NULL_ARGUMENT	int	handle，x，b中存在空参数。

依赖

#include "kml_scaiss.h"

示例

C Interface：

```
MPI_Init(NULL, NULL);
int size, rank;
MPI_Comm_size(MPI_COMM_WORLD, &size);//获取总进程数
MPI_Comm_rank(MPI_COMM_WORLD, &rank);//获取当前进程标识

int mat_size = 8;
int n = mat_size / size;
int n_beg = n * rank;
if (n * size != mat_size && rank == (size - 1)) {
    n = mat_size - n * rank;
}
int ja[26] = { 0, 3, 4, 1, 2, 3, 5, 1, 2, 7, 0, 1, 3, 6, 0, 4, 5, 1, 4, 5, 7, 3, 6, 2, 5, 7 };
double a[26] = { 1.0, 1.0, 2.0, 9.0, 2.0, 1.0, -3.0, 2.0, 3.0, 2.0, 1.0, 1.0, 9.0, -5.0, 2.0, 6.0, 1.0, -3.0, 1.0, 4.0, 1.0,
-5.0, 7.0, 2.0, 1.0, 2.0 };
int ia[9] = { 0, 3, 7, 10, 14, 17, 21, 23, 26 };
int a_beg = ia[n_beg];
for (int i = n_beg; i < (n_beg + n + 1); i++) {
    ia[i] -= a_beg;
}
```


版权所有 © 华为技术有限公司

```
/* Internal KML_SCAISS structure */
KmlScasolverTask *handle;

/* KML_SCAISS control parameters */
int error;            /* Output error handle */

/* Create data structures */
const double *a_holder = &a[a_beg];
const int *ja_holder = &ja[a_beg];
const int *ia_holder = &ia[n_beg];
error = KmlScaissCgInitStripesDI(&handle, mat_size, 1, &n, &n_beg, &a_holder, &ja_holder, &ia_holder,
MPI_COMM_WORLD);

double eps = 1e-4;
error = KmlScaissCgSetDID(&handle, KMLSS_THRESHOLD, &eps, 1);
if (error != 0) {
    printf("\nERROR in KmlScaissCgSetDID with KMLSS_THRESHOLD: %d", error);
    return 1;
}
int max_iters = 2000;
error = KmlScaissCgSetDII(&handle, KMLSS_MAX_ITERATION_COUNT, &max_iters, 1);
if (error != 0) {
    printf("\nERROR in KmlScaissCgSetDII with KMLSS_MAX_ITERATION_COUNT: %d", error);
    return 1;
}
int precond = KMLSS_BJACOBI;
error = KmlScaissCgSetDII(&handle, KMLSS_PRECONDITIONER_TYPE, &precond, 1);
if (error != 0) {
    printf("\nERROR in KmlScaissCgSetDII with KMLSS_PRECONDITIONER_TYPE: %d", error);
    return 1;
}
int num_blocks = 1;
error = KmlScaissCgPcSetDII(&handle, KMLSS_NUM_BLOCKS, &num_blocks, 1);
if (error != 0) {
    printf("ERROR in KmlScaissCgPcSetDII: %d\n", error);
    return 1;
}
int subprecond = KMLSS_ICC;
error = KmlScaissCgPcSetDII(&handle, KMLSS_BLOCK_METHOD, &subprecond, 1);
if (error != 0) {
    printf("ERROR in KmlScaissCgPcSetDII: %d\n", error);
    return 1;
}
error = KmlScaissCgAnalyzeDI(&handle);
if (error != 0) {
    printf("ERROR in KmlScaissCgAnalyzeDI: %d\n", error);
    return 1;
}
error = KmlScaissCgFactorizeDI(&handle);
if (error != 0) {
    printf("ERROR in KmlScaissCgFactorizeDI: %d\n", error);
    return 1;
}

int nrhs = 1;
int ldx = n, ldb = n;
double b[8] = { 4.0, 9.0, 7.0, 6.0, 9.0, 3.0, 2.0, 5.0 };
double x[8] = { 0.0, 0.0, 0.0, 0.0, 0.0, 0.0, 0.0, 0.0 };
error = KmlScaissCgSolveDxDI(&handle, nrhs, &x[n_beg], ldx, &b[n_beg], ldb);
if (error != 0) {
    printf("ERROR in KmlScaissCgSolveDI: %d\n", error);
    return 1;
}
```

6.5.3.4.1.12 KmlScaissCgGet?I?

获得迭代求解相关参数。

 版权所有 © 华为技术有限公司

接口定义

C Interface：

int KmlScaissCgGetSII(KmlScasolverTask **handle, enum KmlSolverParam param, int *data, int nd);

int KmlScaissCgGetSIS(KmlScasolverTask **handle, enum KmlSolverParam param, float *data, int nd);

int KmlScaissCgGetDII(KmlScasolverTask **handle, enum KmlSolverParam param, int *data, int nd);

int KmlScaissCgGetDID(KmlScasolverTask **handle, enum KmlSolverParam param, double *data, int nd);

参数

参数名	类型	描述	输入/输出
handle	KmlScasolverTask **	求解器句柄，传入之前步骤的变量。	输入/输出
param	enum KmlSolverParam	• KMLSS_ITERATION_COUNT表示迭代次数。 • KMLSS_MAX_ITERATION_COUNT表示设置的最大迭代次数。 • KMLSS_TOLERANCE表示最终的迭代相对残差。 • KMLSS_THRESHOLD表示设置的相对残差。	输入
data	• 在KmlScaissCgGetSII/KmlScaissCgGetDII中为int *。 • 在KmlScaissCgGetSIS中为float *。 • 在KmlScaissCgGetDID中为double *。	• 在KmlScaissCgGetSII/KmlScaissCgGetDII中为迭代步数。 • 在KmlScaissCgGetSIS中为残差。 • 在KmlScaissCgGetDID中为残差。	输出
nd	int	data数组元素个数。	输入

返回值

返回值	类型	描述
KMLSS_NO_ERROR	int	正常执行。

版权所有 © 华为技术有限公司

返回值	类型	描述
KMLSS_DATA_SIZE	int	参数nd不等于1。
KMLSS_NULL_ARGUMENT	int	handle，data中存在空参数。
KMLSS_BAD_SELECTOR	int	param为无效参数。

依赖

#include "kml_scaiss.h"

示例

C Interface：

```
MPI_Init(NULL, NULL);
int size, rank;
MPI_Comm_size(MPI_COMM_WORLD, &size);//获取总进程数
MPI_Comm_rank(MPI_COMM_WORLD, &rank);//获取当前进程标识

int mat_size = 8;
int n = mat_size / size;
int n_beg = n * rank;
if (n * size != mat_size && rank == (size - 1)) {
    n = mat_size - n * rank;
}
int ja[26] = { 0, 3, 4, 1, 2, 3, 5, 1, 2, 7, 0, 1, 3, 6, 0, 4, 5, 1, 4, 5, 7, 3, 6, 2, 5, 7 };
double a[26] = { 1.0, 1.0, 2.0, 9.0, 2.0, 1.0, -3.0, 2.0, 3.0, 2.0, 1.0, 1.0, 9.0, -5.0, 2.0, 6.0, 1.0, -3.0, 1.0, 4.0, 1.0,
-5.0, 7.0, 2.0, 1.0, 2.0 };
int ia[9] = { 0, 3, 7, 10, 14, 17, 21, 23, 26 };
int a_beg = ia[n_beg];
for (int i = n_beg; i < (n_beg + n + 1); i++) {
    ia[i] -= a_beg;
}

/* Internal KML_SCAISS structure */
KmlScasolverTask *handle;

/* KML_SCAISS control parameters */
int error;            /* Output error handle */

/* Create data structures */
const double *a_holder = &a[a_beg];
const int *ja_holder = &ja[a_beg];
const int *ia_holder = &ia[n_beg];
error = KmlScaissCgInitStripesDI(&handle, mat_size, 1, &n, &n_beg, &a_holder, &ja_holder, &ia_holder,
MPI_COMM_WORLD);

double eps = 1e-4;
error = KmlScaissCgSetDID(&handle, KMLSS_THRESHOLD, &eps, 1);
if (error != 0) {
    printf("\nERROR in KmlScaissCgSetDID with KMLSS_THRESHOLD: %d", error);
    return 1;
}
int max_iters = 2000;
error = KmlScaissCgSetDII(&handle, KMLSS_MAX_ITERATION_COUNT, &max_iters, 1);
if (error != 0) {
    printf("\nERROR in KmlScaissCgSetDII with KMLSS_MAX_ITERATION_COUNT: %d", error);
    return 1;
}
int precond = KMLSS_BJACOBI;
```

 版权所有 © 华为技术有限公司

```
error = KmlScaissCgSetDII(&handle, KMLSS_PRECONDITIONER_TYPE, &precond, 1);
if (error != 0) {
    printf("\nERROR in KmlScaissCgSetDII with KMLSS_PRECONDITIONER_TYPE: %d", error);
    return 1;
}
int num_blocks = 1;
error = KmlScaissCgPcSetDII(&handle, KMLSS_NUM_BLOCKS, &num_blocks, 1);
if (error != 0) {
    printf("ERROR in KmlScaissCgPcSetDII: %d\n", error);
    return 1;
}
int subprecond = KMLSS_ICC;
error = KmlScaissCgPcSetDII(&handle, KMLSS_BLOCK_METHOD, &subprecond, 1);
if (error != 0) {
    printf("ERROR in KmlScaissCgPcSetDII: %d\n", error);
    return 1;
}
error = KmlScaissCgAnalyzeDI(&handle);
if (error != 0) {
    printf("ERROR in KmlScaissCgAnalyzeDI: %d\n", error);
    return 1;
}
error = KmlScaissCgFactorizeDI(&handle);
if (error != 0) {
    printf("ERROR in KmlScaissCgFactorizeDI: %d\n", error);
    return 1;
}

int nrhs = 1;
int ldx = n, ldb = n;
double b[8] = { 4.0, 9.0, 7.0, 6.0, 9.0, 3.0, 2.0, 5.0 };
double x[8] = { 0.0, 0.0, 0.0, 0.0, 0.0, 0.0, 0.0, 0.0 };
error = KmlScaissCgSolveDxDI(&handle, nrhs, &x[n_beg], ldx, &b[n_beg], ldb);
if (error != 0) {
    printf("ERROR in KmlScaissCgSolveDI: %d\n", error);
    return 1;
}

int idata[] = {0};
error = KmlScaissCgGetDII(&handle, KMLSS_ITERATION_COUNT, idata, 1);
if (error != 0) {
    printf("\nERROR in KmlScaissCgGetDII: %d", error);
    return 1;
    }
double ddata[] = {0.0};
error = KmlScaissCgGetDID(&handle, KMLSS_TOLERANCE, ddata, 1);
if (error != 0) {
    printf("\nERROR in KmlScaissCgGetDID: %d", error);
    return 1;
    }
if (rank == 0) printf("[Solve info] Res= \t%.12g\t, iteration = \t%d\t\n", sqrt(ddata[0]), idata[0]);
```

运行结果（2个进程）：

```
[Solve info] Res=      2.04475505752e-15       , iteration =   8
```

6.5.3.4.1.13 KmlScaissCgClean?I

释放内部数据结构。

接口定义

C Interface：

int KmlScaissCgCleanSI(KmlScaSolverTask **handle);

int KmlScaissCgCleanDI(KmlScaSolverTask **handle);

 版权所有 © 华为技术有限公司

参数

参数名	类型	描述	输入/输出
handle	KmlScaSolverTask **	求解器句柄，传入之前步骤的变量。	输入/输出

返回值

返回值	类型	描述
KMLSS_NO_ERROR	int	正常执行。
KMLSS_NULL_ARGUMENT	int	handle为空参数。

依赖

#include "kml_scaiss.h"

示例

C Interface：

```
MPI_Init(NULL, NULL);
int size, rank;
MPI_Comm_size(MPI_COMM_WORLD, &size);//获取总进程数
MPI_Comm_rank(MPI_COMM_WORLD, &rank);//获取当前进程标识

int mat_size = 8;
int n = mat_size / size;
int n_beg = n * rank;
if (n * size != mat_size && rank == (size - 1)) {
    n = mat_size - n * rank;
}
int ja[26] = { 0, 3, 4, 1, 2, 3, 5, 1, 2, 7, 0, 1, 3, 6, 0, 4, 5, 1, 4, 5, 7, 3, 6, 2, 5, 7 };
double a[26] = { 1.0, 1.0, 2.0, 9.0, 2.0, 1.0, -3.0, 2.0, 3.0, 2.0, 1.0, 1.0, 9.0, -5.0, 2.0, 6.0, 1.0, -3.0, 1.0, 4.0, 1.0,
-5.0, 7.0, 2.0, 1.0, 2.0 };
int ia[9] = { 0, 3, 7, 10, 14, 17, 21, 23, 26 };
int a_beg = ia[n_beg];
for (int i = n_beg; i < (n_beg + n + 1); i++) {
    ia[i] -= a_beg;
}

/* Internal KML_SCAISS structure */
KmlScaSolverTask *handle;

/* KML_SCAISS control parameters */
int error;              /* Output error handle */

/* Create data structures */
const double *a_holder = &a[a_beg];
const int *ja_holder = &ja[a_beg];
const int *ia_holder = &ia[n_beg];
error = KmlScaissCgInitStripesDI(&handle, mat_size, 1, &n, &n_beg, &a_holder, &ja_holder, &ia_holder,
MPI_COMM_WORLD);

double eps = 1e-4;
```

 版权所有 © 华为技术有限公司

```
error = KmlScaissCgSetDID(&handle, KMLSS_THRESHOLD, &eps, 1);
if (error != 0) {
    printf("\nERROR in KmlScaissCgSetDID with KMLSS_THRESHOLD: %d", error);
    return 1;
}
int max_iters = 2000;
error = KmlScaissCgSetDII(&handle, KMLSS_MAX_ITERATION_COUNT, &max_iters, 1);
if (error != 0) {
    printf("\nERROR in KmlScaissCgSetDII with KMLSS_MAX_ITERATION_COUNT: %d", error);
    return 1;
}
int precond = KMLSS_BJACOBI;
error = KmlScaissCgSetDII(&handle, KMLSS_PRECONDITIONER_TYPE, &precond, 1);
if (error != 0) {
    printf("\nERROR in KmlScaissCgSetDII with KMLSS_PRECONDITIONER_TYPE: %d", error);
    return 1;
}
int num_blocks = 1;
error = KmlScaissCgPcSetDII(&handle, KMLSS_NUM_BLOCKS, &num_blocks, 1);
if (error != 0) {
    printf("ERROR in KmlScaissCgPcSetDII: %d\n", error);
    return 1;
}
int subprecond = KMLSS_ICC;
error = KmlScaissCgPcSetDII(&handle, KMLSS_BLOCK_METHOD, &subprecond, 1);
if (error != 0) {
    printf("ERROR in KmlScaissCgPcSetDII: %d\n", error);
    return 1;
}
error = KmlScaissCgAnalyzeDI(&handle);
if (error != 0) {
    printf("ERROR in KmlScaissCgAnalyzeDI: %d\n", error);
    return 1;
}
error = KmlScaissCgFactorizeDI(&handle);
if (error != 0) {
    printf("ERROR in KmlScaissCgFactorizeDI: %d\n", error);
    return 1;
}

int nrhs = 1;
int ldx = n, ldb = n;
double b[8] = { 4.0, 9.0, 7.0, 6.0, 9.0, 3.0, 2.0, 5.0 };
double x[8] = { 0.0, 0.0, 0.0, 0.0, 0.0, 0.0, 0.0, 0.0 };
error = KmlScaissCgSolveDxDI(&handle, nrhs, &x[n_beg], ldx, &b[n_beg], ldb);
if (error != 0) {
    printf("ERROR in KmlScaissCgSolveDI: %d\n", error);
    return 1;
}

int idata[] = {0};
error = KmlScaissCgGetDII(&handle, KMLSS_ITERATION_COUNT, idata, 1);
if (error != 0) {
    printf("\nERROR in KmlScaissCgGetDII: %d", error);
    return 1;
}
double ddata[] = {0.0};
error = KmlScaissCgGetDID(&handle, KMLSS_TOLERANCE, ddata, 1);
if (error != 0) {
    printf("\nERROR in KmlScaissCgGetDID: %d", error);
    return 1;
}
if (rank == 0) printf("[Solve info] Res= \t%.12g\t, iteration = \t%d\t\n", sqrt(ddata[0]), idata[0]);

error = KmlScaissCgCleanDI(&handle);
if (error != 0) {
    printf("\nERROR in KmlScaissCgCleanDI: %d", error);
    return 1;
}
```

 版权所有 © 华为技术有限公司

6.5.3.4.1.14 示例

依赖

#include "kml_scaiss.h"

示例

C Interface：

```
#include <stdio.h>
#include <stdlib.h>
#include <kml_scaiss.h>
#include "mpi.h"
// USER data struct
typedef struct {
    int n;
    const int *ia;
    const int *ja;
    const double *a;
    MPI_Comm comm;
} duser;
int diagonal_preconditioner(void *usr, double *x)
{
    duser *u = (duser *)usr;
    int i, j;
    /* Apply diagonal preconditioner */
    for (i = 0; i < u->n; i++) {
        for (j = u->ia[i]; j < u->ia[i + 1]; j++) {
            if (u->ja[j] == i) {
                x[i] /= u->a[j];
            }
        }
    }
    return 0;
}
int user_spmv(void *usr, const double *x, double *y)
{
    duser *u = (duser *)usr;
    int size;
    int rank;
    MPI_Comm_size(u->comm, &size);
    MPI_Comm_rank(u->comm, &rank);
    int n = 8 / size;
    double fullX[8] = { 0.0 };
    MPI_Allgather(&x[0], n, MPI_DOUBLE, &fullX[0], n, MPI_DOUBLE, u->comm);
    int i, j;
    for (i = 0; i < u->n; i++) {
        double sum = 0.0;
        for (j = u->ia[i]; j < u->ia[i + 1]; j++) {
            int k = u->ja[j];
            double value = u->a[j];
            sum += fullX[k] * value;
        }
        y[i] = sum;
    }
    return 0;
}
int main(void)
{
    /* MPI initialization */
    MPI_Init(NULL, NULL);
    int size, rank;
    MPI_Comm_size(MPI_COMM_WORLD, &size);
    MPI_Comm_rank(MPI_COMM_WORLD, &rank);
    /* Matrix data (CSR, stored full matrix)
       | 1   *   *   1   2   *   *   * |
       | *   9   2   1   *  -3   *   * |
```


版权所有 © 华为技术有限公司

```
    | *   2   3   *   *   *   *   2  |
    | 1   1   *   9   *   *  -5   *  |
    | 2   *   *   *   6   1   *   *  |
    | *  -3   *   *   1   4   *   1  |
    | *   *   *  -5   *   *   7   *  |
    | *   *   2   *   *   1   *   2  |
*/
/* Initialize separations */
int mat_size = 8;
int n = mat_size / size;
int n_beg = n * rank;
if (n * size != 8 && rank == (size - 1)) {
    n = mat_size - n * rank;
}
int ia[9] = { 0, 3, 7, 10, 14, 17, 21, 23, 26 };
int a_beg = ia[n_beg];
for (int i = n_beg; i < (n_beg + n + 1); i++) {
    ia[i] -= a_beg;
}
/* clang-format off */
int ja[26] = { 0,       3, 4,
               1, 2, 3,    5,
               1, 2,          7,
            0, 1,    3,       6,
            0,          4, 5,
               1,       4, 5,    7,
                  3,       6,
                2,       5,    7 };
double a[26] = { 1.0,           1.0, 2.0,
                    9.0, 2.0, 1.0,     -3.0,
                    2.0, 3.0,                2.0,
                 1.0, 1.0,      9.0,          -5.0,
                 2.0,               6.0, 1.0,
                    -3.0,          1.0, 4.0,      1.0,
                              -5.0,          7.0,
                         2.0,          1.0,      2.0 };
/* clang-format on */
/* Right-hand side vector */
double b[8] = { 4.0, 9.0, 7.0, 6.0, 9.0, 3.0, 2.0, 5.0 };
/* Solution vector */
double x[8] = { 0.0, 0.0, 0.0, 0.0, 0.0, 0.0, 0.0, 0.0 };
/* Internal KML_SCAISS structure */
KmlScasolverTask *handle;
/* KML_SCAISS control parameters */
int nrhs = 1;          /* Number of right-hand sides */
int ldx = n, ldb = n; /*!Leading dimension of B and X */
int error;             /* Output error handle */
/* Create data structures */
const double *a_holder = &a[a_beg];
const int *ja_holder = &ja[a_beg];
const int *ia_holder = &ia[n_beg];
error =
    KmlScaissCgInitStripesDI(&handle, mat_size, 1, &n, &n_beg, &a_holder, &ja_holder, &ia_holder,
MPI_COMM_WORLD);
if (error != 0) {
    printf("ERROR in KmlScaissCgInitStripesDI: %d\n", error);
    return 1;
}
error = KmlScaissCgAnalyzeDI(&handle);
if (error != 0) {
    printf("ERROR in KmlScaissCgAnalyzeDI: %d\n", error);
    return 1;
}
error = KmlScaissCgFactorizeDI(&handle);
if (error != 0) {
    printf("ERROR in KmlScaissCgFactorizeDI: %d\n", error);
    return 1;
}
/* solve */
```

 版权所有 © 华为技术有限公司

```
double x7[8] = { 0.0, 0.0, 0.0, 0.0, 0.0, 0.0, 0.0, 0.0 };
error = KmlScaissCgSolveDI(&handle, nrhs, &x7[0], ldx, &b[n_beg], ldb);
if (error != 0) {
    printf("ERROR in KmlScaissCgSolveDI: %d\n", error);
    return 1;
}
/* Print the solution */
if (rank == 0) {
    printf("solve, x:\n");
    for (int i = 0; i < n; i++) {
        printf("%lf\n", x7[i]);
    }
}
/* solveDx */
double x0[8] = { 0.0, 0.0, 0.0, 0.0, 0.0, 0.0, 0.0, 0.0 };
error = KmlScaissCgSolveDxDI(&handle, nrhs, &x0[n_beg], ldx, &b[n_beg], ldb);
if (error != 0) {
    printf("ERROR in KmlScaissCgSolveDI: %d\n", error);
    return 1;
}
/* Print the solution */
if (rank == 0) {
    printf("solveDx, x:\n");
    for (int i = 0; i < n; i++) {
        printf("%lf\n", x0[i]);
    }
}
/* L1 norm */
int NormType = KMLSS_L1;
error = KmlScaissCgSetDII(&handle, KMLSS_VECTOR_NORM_TYPE, &NormType, 1);
if (error != 0) {
    printf("ERROR in KmlScaissCgSetDII: %d\n", error);
    return 1;
}
/* Solve */
error = KmlScaissCgSolveDI(&handle, nrhs, &x[0], ldx, &b[n_beg], ldb);
if (error != 0) {
    printf("ERROR in KmlScaissCgSolveDI: %d\n", error);
    return 1;
}
/* Print the solution */
if (rank == 0) {
    printf("L1 norm, x:\n");
    for (int i = 0; i < mat_size; i++) {
        printf("%lf\n", x[i]);
    }
}
/* SolveDx */
double x6[8] = { 0.0, 0.0, 0.0, 0.0, 0.0, 0.0, 0.0, 0.0 };
error = KmlScaissCgSolveDxDI(&handle, nrhs, &x6[n_beg], ldx, &b[n_beg], ldb);
if (error != 0) {
    printf("ERROR in KmlScaissCgSolveDI: %d\n", error);
    return 1;
}
/* Print the solution */
if (rank == 0) {
    printf("L2 norm, x:\n");
    for (int i = 0; i < n; i++) {
        printf("%lf\n", x6[i]);
    }
}
/* L2 norm */
NormType = KMLSS_L2;
error = KmlScaissCgSetDII(&handle, KMLSS_VECTOR_NORM_TYPE, &NormType, 1);
if (error != 0) {
    printf("ERROR in KmlScaissCgSetDII: %d\n", error);
    return 1;
}
/* Solve */
```

 版权所有 © 华为技术有限公司

```
double x2[8] = { 0.0, 0.0, 0.0, 0.0, 0.0, 0.0, 0.0, 0.0 };
error = KmlScaissCgSolveDI(&handle, nrhs, &x2[0], ldx, &b[n_beg], ldb);
if (error != 0) {
    printf("ERROR in KmlScaissCgSolveDI: %d\n", error);
    return 1;
}
/* Print the solution */
if (rank == 0) {
    printf("L2 norm, x:\n");
    for (int i = 0; i < mat_size; i++) {
        printf("%lf\n", x2[i]);
    }
}
/* abs residual */
double res = 1e-10;
error = KmlScaissCgSetDID(&handle, KMLSS_ABS_TOLERANCE, &res, 1);
if (error != 0) {
    printf("ERROR in KmlScaissCgSetDID: %d\n", error);
    return 1;
}
/* Solve */
double x3[8] = { 0.0, 0.0, 0.0, 0.0, 0.0, 0.0, 0.0, 0.0 };
error = KmlScaissCgSolveDI(&handle, nrhs, &x3[0], ldx, &b[n_beg], ldb);
if (error != 0) {
    printf("ERROR in KmlScaissCgSolveDI: %d\n", error);
    return 1;
}
/* Print the solution */
if (rank == 0) {
    printf("abs residual, x:\n");
    for (int i = 0; i < mat_size; i++) {
        printf("%lf\n", x3[i]);
    }
}
/* relative residual */
res = 1e-10;
error = KmlScaissCgSetDID(&handle, KMLSS_THRESHOLD, &res, 1);
if (error != 0) {
    printf("ERROR in KmlScaissCgSetDID: %d\n", error);
    return 1;
}
/* Solve */
double x4[8] = { 0.0, 0.0, 0.0, 0.0, 0.0, 0.0, 0.0, 0.0 };
error = KmlScaissCgSolveDI(&handle, nrhs, &x4[0], ldx, &b[n_beg], ldb);
if (error != 0) {
    printf("ERROR in KmlScaissCgSolveDI: %d\n", error);
    return 1;
}
/* Print the solution */
if (rank == 0) {
    printf("relative residual, x:\n");
    for (int i = 0; i < mat_size; i++) {
        printf("%lf\n", x4[i]);
    }
}
/* Second call with preconditioner */
double x5[8] = { 0.0, 0.0, 0.0, 0.0, 0.0, 0.0, 0.0, 0.0 };
duser prea;
prea.n = n;
prea.ia = &ia[n_beg];
prea.ja = &ja[a_beg];
prea.a = &a[a_beg];
/* Set user preconditioner */
error = KmlScaissCgSetUserPreconditionerDI(&handle, &prea, &diagonal_preconditioner);
if (error != 0) {
    printf("ERROR in KmlScaissCgSetUserPreconditionerDI: %d\n", error);
    return 1;
}
/* Solve */
```


版权所有 © 华为技术有限公司

```
    error = KmlScaissCgSolveDI(&handle, nrhs, &x5[0], ldx, &b[n_beg], ldb);
    if (error != 0) {
        printf("ERROR in KmlScaissCgSolveDI: %d\n", error);
        return 1;
    }
    /* Finalize and Clean-up */
    error = KmlScaissCgCleanDI(&handle);
    if (error != 0) {
        printf("ERROR in KmlScaissCgCleanDI: %d\n", error);
        return 1;
    }
    /* Print the solution */
    if (rank == 0) {
        printf("user preconditioner, x:\n");
        for (int i = 0; i < mat_size; i++) {
            printf("%lf\n", x5[i]);
        }
    }
    /* Second call with user spmv */
    KmlScasolverTask *handle2;
    double x8[8] = { 0.0, 0.0, 0.0, 0.0, 0.0, 0.0, 0.0, 0.0 };
    duser prea2;
    prea2.n = n;
    prea2.ia = &ia[n_beg];
    prea2.ja = &ja[a_beg];
    prea2.a = &a[a_beg];
    prea2.comm = MPI_COMM_WORLD;
    error = KmlScaissCgInitWithoutMatDI(&handle2, 1, &n, MPI_COMM_WORLD);
    if (error != 0) {
        printf("ERROR in KmlScaissCgInitWithoutMatDI: %d\n", error);
        return 1;
    }
    /* Set user spmv */
    error = KmlScaissCgSetUserSpmvDI(&handle2, &prea2, &user_spmv);
    if (error != 0) {
        printf("ERROR in KmlScaissCgSetUserSpmvDI: %d\n", error);
        return 1;
    }
    /* Solve */
    error = KmlScaissCgSolveDxDI(&handle2, nrhs, &x8[n_beg], ldx, &b[n_beg], ldb);
    if (error != 0) {
        printf("ERROR in KmlScaissCgSolveDI: %d\n", error);
        return 1;
    }
    /* Finalize and Clean-up */
    error = KmlScaissCgCleanDI(&handle2);
    if (error != 0) {
        printf("ERROR in KmlScaissCgCleanDI: %d\n", error);
        return 1;
    }
    /* Print the solution */
    if (rank == 0) {
        printf("user spmv, x:\n");
        for (int i = 0; i < n; i++) {
            printf("%lf\n", x8[i]);
        }
    }
    MPI_Finalize();
    return 0;
}
```

6.5.3.4.2 SCAISS-BICGSTAB

6.5.3.4.2.1 KmlScaissBicgstabInitStripes?I

初始化数据结构，并将用户提供的系数矩阵关联到求解系统中。初始化集群版的BICGSTAB方法时，每个进程使用局部的CSR格式矩阵。

 版权所有 © 华为技术有限公司

接口定义

C Interface：

int KmlScaissBicgstabInitStripesSI(KmlScasolverTask **handle, int n, int nstripes, const int *stripeWidth,const int *stripeRow, const float * const *a, const int * const *ja, const int * const *ia, MPI_Comm comm);

int KmlScaissBicgstabInitStripesDI(KmlScasolverTask **handle, int n, int nstripes, const int *stripeWidth, const int *stripeRow, const double * const *a, const int * const *ja, const int * const *ia, MPI_Comm comm);

参数

参数名	类型	描述	输入/输出
handle	KmlScasolverTask **	求解器句柄，handle不可为空指针。	输入/输出
n	int	n为矩阵A的维度。	输入
nstripes	int	nstripes为存储在当前MPI进程中的矩阵数，目前只能取1。	输入
stripeWidth	int *	stripeWidth为当前MPI进程中的矩阵行数。该变量的生命周期与handle相同。	输入
stripeRow	int *	stripeRow为当前MPI进程中的矩阵的全局行索引，即首行在全局矩阵中对应行数。该数组的生命周期与handle相同。	输入
a	• 在KmlScaissBicgstabInitStripesSI中为float *。 • 在KmlScaissBicgstabInitStripesDI中为double *。	a为维度为非零元素的个数的数组，a[i]为第i个非零元的值。该数组的生命周期与handle相同。	输入
ja	int *	ja为维度为非零元素的个数的数组，ja[i]为第i个非零元的列号。该数组的生命周期与handle相同。	输入
ia	int *	ia为stripeWidth+1维数组，ia[i]为第i行第一个非零元在所有非零元中的索引，ia[0]=0。该数组的生命周期与handle相同。	输入
comm	MPI_Comm	为MPI进程通信域。	输入

版权所有 © 华为技术有限公司

返回值

返回值	类型	描述
KMLSS_NO_ERROR	int	正常执行。
KMLSS_NULL_ARGUMENT	int	handle，stripeWidth，stripeRow，a，ja，ia中存在空参数。
KMLSS_BAD_N	int	参数n不是正整数。
KMLSS_NOT_IMPLEMENTED	int	参数nstripes不等于1。
KMLSS_NO_MEMORY	int	内存不足。

依赖

#include "kml_scaiss.h"

示例

C Interface：

```
MPI_Init(NULL, NULL);
int size, rank;
MPI_Comm_size(MPI_COMM_WORLD, &size);//获取总进程数
MPI_Comm_rank(MPI_COMM_WORLD, &rank);//获取当前进程标识

int mat_size = 8;
int n = mat_size / size;
int n_beg = n * rank;
if (n * size != mat_size && rank == (size - 1)) {
    n = mat_size - n * rank;
}
int ja[26] = { 0, 3, 4, 1, 2, 3, 5, 1, 2, 7, 0, 1, 3, 6, 0, 4, 5, 1, 4, 5, 7, 3, 6, 2, 5, 7 };
double a[26] = { 1.0, 1.0, 2.0, 9.0, 2.0, 1.0, -3.0, 2.0, 3.0, 2.0, 1.0, 1.0, 9.0, -5.0, 2.0, 6.0, 1.0, -3.0, 1.0, 4.0, 1.0,
-5.0, 7.0, 2.0, 1.0, 2.0 };
int ia[9] = { 0, 3, 7, 10, 14, 17, 21, 23, 26 };
int a_beg = ia[n_beg];
for (int i = n_beg; i < (n_beg + n + 1); i++) {
    ia[i] -= a_beg;
}

/* Internal KML_SCAISS structure */
KmlScasolverTask *handle;

/* KML_SCAISS control parameters */
int error;            /* Output error handle */

/* Create data structures */
const double *a_holder = &a[a_beg];
const int *ja_holder = &ja[a_beg];
const int *ia_holder = &ia[n_beg];
error = KmlScaissBicgstabInitStripesDI(&handle, mat_size, 1, &n, &n_beg, &a_holder, &ja_holder, &ia_holder,
MPI_COMM_WORLD);
```

6.5.3.4.2.2 KmlScaissBicgstabInitWithoutMat?I

初始化数据结构，用于无矩阵求解的初始化，需要与用户自定义矩阵向量乘功能一并使用。

版权所有 © 华为技术有限公司

接口定义

C Interface：

int KmlScaissBicgstabInitWithoutMatSI(KmlScasolverTask **handle, int nstripes, const int *stripeWidth, MPI_Comm comm);

int KmlScaissBicgstabInitWithoutMatDI(KmlScasolverTask **handle, int nstripes, const int *stripeWidth, MPI_Comm comm);

参数

参数名	类型	描述	输入/输出
handle	KmlScasolverTask **	求解器句柄，handle不可为空指针。	输入/输出
nstripes	int	nstripes为存储在当前MPI进程中的矩阵数，目前只能取1。	输入
stripeWidth	int *	stripeWidth为当前MPI进程中的矩阵行数。该变量的生命周期与handle相同。	输入
comm	MPI_Comm	为MPI进程通信域。	输入

返回值

返回值	类型	描述
KMLSS_NO_ERROR	int	正常执行。
KMLSS_NULL_ARGUMENT	int	handle，stripeWidth中存在空参数。
KMLSS_NOT_IMPLEMENTED	int	参数nstripes不等于1。
KMLSS_NO_MEMORY	int	内存不足。

6.5.3.4.2.3 KmlScaissBicgstabSetUserPreconditioner?I

关联用户自定义预条件子回调函数，如果不使用自定义预条件子，可以不调用该接口。

接口定义

C Interface：

int KmlScaissBicgstabSetUserPreconditionerSI(KmlScasolverTask **handle, void *ustruct, int (*fptr)(void *ustruct, float *x));

版权所有 © 华为技术有限公司

int KmlScaissBicgstabSetUserPreconditionerDI(KmlScasolverTask **handle, void *ustruct, int (*fptr)(void *ustruct, double *x));

参数

参数名	类型	描述	输入/输出
handle	KmlScasolverTask **	求解器句柄，传入之前步骤的变量。	输入/输出
ustruct	void *	指向用户数据的指针，每次调用预条件子时作为第一个参数传递。	输入
(*fptr)	int	用户自定义的函数指针，指向回调函数，该回调函数进行预条件子与其第二参数间的计算。	输入

返回值

返回值	类型	描述
KMLSS_NO_ERROR	int	正常执行。
KMLSS_NULL_ARGUMENT	int	handle，fptr中存在空参数。

依赖

#include "kml_scaiss.h"

示例

C Interface：

```
// USER data struct
typedef struct {
    int n;
    const int *ia;
    const int *ja;
    const double *a;
} duser;

int diagonal_preconditioner(void *usr, double *x)
{
    duser *u = (duser *)usr;
    int i, j;

    /* Apply diagonal preconditioner*/
    for (i = 0; i < u->n; i++) {
        for (j = u->ia[i]; j < u->ia[i + 1]; j++) {
            if (u->ja[j] == i)
                x[i] /= u->a[j];
        }
    }
    return 0;
}
```

 版权所有 © 华为技术有限公司

```
MPI_Init(NULL, NULL);
int size, rank;
MPI_Comm_size(MPI_COMM_WORLD, &size);//获取总进程数
MPI_Comm_rank(MPI_COMM_WORLD, &rank);//获取当前进程标识

int n = 8 / size;
int n_beg = n * rank;
if (n * size != 8 && rank == (size - 1)) {
    n = 8 - n * rank;
}
int ja[26] = { 0, 3, 4, 1, 2, 3, 5, 1, 2, 7, 0, 1, 3, 6, 0, 4, 5, 1, 4, 5, 7, 3, 6, 2, 5, 7 };
double a[26] = { 1.0, 1.0, 2.0, 9.0, 2.0, 1.0, -3.0, 2.0, 3.0, 2.0, 1.0, 1.0, 9.0, -5.0, 2.0, 6.0, 1.0, -3.0, 1.0, 4.0, 1.0,
-5.0, 7.0, 2.0, 1.0, 2.0 };
int ia[9] = { 0, 3, 7, 10, 14, 17, 21, 23, 26 };
int a_beg = ia[n_beg];
for (int i = n_beg; i < 9; i++) {
    ia[i] -= a_beg;
}

/* Internal KML_SCAISS structure */
KmlScasolverTask *handle;

/* KML_SCAISS control parameters */
int error;              /* Output error handle */

/* Create data structures */
const double *a_holder = &a[a_beg];
const int *ja_holder = &ja[a_beg];
const int *ia_holder = &ia[n_beg];
error = KmlScaissCgInitStripesDI(&handle, 8, 1, &n, &n_beg, &a_holder, &ja_holder, &ia_holder,
MPI_COMM_WORLD);

duser prea;
prea.n = n;
prea.ia = ia;
prea.ja = ja;
prea.a = a;

error = KmlScaissCgSetUserPreconditionerDI(&handle, &prea, &diagonal_preconditioner);
if (error != 0) {
    printf("\nERROR in KmlScaissCgSetUserPreconditionerDI: %d", error);
    return 1;
    }
```

6.5.3.4.2.4 KmlScaissBicgstabSet?I?

设置迭代求解的相关参数。

接口定义

C Interface：

int KmlScaissBicgstabSetSII(KmlScasolverTask **handle, enum KmlSolverParam param, const int *data, int nd);

int KmlScaissBicgstabSetSIS(KmlScasolverTask **handle, enum KmlSolverParam param, const float *data, int nd);

int KmlScaissBicgstabSetDII(KmlScasolverTask **handle, enum KmlSolverParam param, const int *data, int nd);

int KmlScaissBicgstabSetDID(KmlScasolverTask **handle, enum KmlSolverParam param, const double *data, int nd);

 版权所有 © 华为技术有限公司

参数

参数名	类型	描述	输入/输出
handle	KmlScasolverTask **	求解器句柄，传入之前步骤的变量。	输入/输出
param	enum KmlSolverParam	• KMLSS_THRESHOLD表示迭代残差相对阈值。 • KMLSS_ABS_TOLERANCE表示迭代残差绝对阈值。 • KMLSS_MAX_ITERATION_COUNT表示最大迭代次数。 • KMLSS_PRECONDITIONER_TYPE表示预条件子类型。 • KMLSS_VECTOR_NORM_TYPE表示迭代残差的计算范数。 • KMLSS_MATRIX_FORMAT表示矩阵对角块稀疏存储格式。 • KMLSS_OFFDIAG_MATRIX_FORMAT表示矩阵非对角块稀疏存储格式。	输入
data	• 在KmlScaissBicgstabSetSII/KmlScaissBicgstabSetDII中为const int *。 • 在KmlScaissBicgstabSetSIS中为const float *。 • 在KmlScaissBicgstabSetDID中为const double *。	迭代求解相关参数数组（如阈值、最大迭代次数等）。 当前可选的计算范数： • 1-NORM：KMLSS_L1。 • 2-NORM：KMLSS_L2。 当前可选的稀疏矩阵存储格式： • CSR格式：KMLSS_CSR。 • ELLPACK格式：KMLSS_ELL。 • COO格式：KMLSS_COO。 • ELLPACK_COO混合格式：KMLSS_SELLCOO。 当前可选择的预条件子类型： • JACOBI预条件子：KMLSS_JACOBI。 • BLOCK JACOBI预条件子：KMLSS_BJACOBI。 • AMG预条件子：KMLSS_AMG。	输入
nd	int	data数组元素个数。	输入

版权所有 © 华为技术有限公司

返回值

返回值	类型	描述
KMLSS_NO_ERROR	int	正常执行。
KMLSS_DATA_SIZE	int	参数nd不等于1。
KMLSS_NULL_ARGUMENT	int	handle，data中存在空参数。
KMLSS_BAD_SELECTOR	int	param为无效参数。
KMLSS_BAD_PRECONDITIONER	int	选择的预条件暂未实现。

依赖

#include "kml_scaiss.h"

示例

C Interface：

```
MPI_Init(NULL, NULL);
int size, rank;
MPI_Comm_size(MPI_COMM_WORLD, &size);//获取总进程数
MPI_Comm_rank(MPI_COMM_WORLD, &rank);//获取当前进程标识

int mat_size = 8;
int n = mat_size / size;
int n_beg = n * rank;
if (n * size != mat_size && rank == (size - 1)) {
    n = mat_size - n * rank;
}
int ja[26] = { 0, 3, 4, 1, 2, 3, 5, 1, 2, 7, 0, 1, 3, 6, 0, 4, 5, 1, 4, 5, 7, 3, 6, 2, 5, 7 };
double a[26] = { 1.0, 1.0, 2.0, 9.0, 2.0, 1.0, -3.0, 2.0, 3.0, 2.0, 1.0, 1.0, 9.0, -5.0, 2.0, 6.0, 1.0, -3.0, 1.0, 4.0, 1.0,
-5.0, 7.0, 2.0, 1.0, 2.0 };
int ia[9] = { 0, 3, 7, 10, 14, 17, 21, 23, 26 };
int a_beg = ia[n_beg];
for (int i = n_beg; i < (n_beg + n + 1); i++) {
    ia[i] -= a_beg;
}

/* Internal KML_SCAISS structure */
KmlScasolverTask *handle;

/* KML_SCAISS control parameters */
int error;              /* Output error handle */

/* Create data structures */
const double *a_holder = &a[a_beg];
const int *ja_holder = &ja[a_beg];
const int *ia_holder = &ia[n_beg];
error = KmlScaissBicgstabInitStripesDI(&handle, mat_size, 1, &n, &n_beg, &a_holder, &ja_holder, &ia_holder,
MPI_COMM_WORLD);

double eps = 1e-4;
error = KmlScaissBicgstabSetDID(&handle, KMLSS_THRESHOLD, &eps, 1);
if (error != 0) {
    printf("\nERROR in KmlScaissBicgstabSetDID with KMLSS_THRESHOLD: %d", error);
    return 1;
```

 版权所有 © 华为技术有限公司

```
}
int max_iters = 2000;
error = KmlScaissBicgstabSetDII(&handle, KMLSS_MAX_ITERATION_COUNT, &max_iters, 1);
if (error != 0) {
    printf("\nERROR in KmlScaissBicgstabSetDII with KMLSS_MAX_ITERATION_COUNT: %d", error);
    return 1;
}
int precond = KMLSS_BJACOBI;
error = KmlScaissBicgstabSetDII(&handle, KMLSS_PRECONDITIONER_TYPE, &precond, 1);
if (error != 0) {
    printf("\nERROR in KmlScaissBicgstabSetDII with KMLSS_PRECONDITIONER_TYPE: %d", error);
    return 1;
}
```

6.5.3.4.2.5 KmlScaissBicgstabSetUserSpmv?I

关联用户自定义矩阵向量乘函数，需要与无矩阵初始化接口一起使用。

接口定义

C Interface：

int KmlScaissBicgstabSetUserSpmvSI(KmlScasolverTask **handle, void *ustruct, int (*fptr)(void *ustruct, const float *x, float *y));

int KmlScaissBicgstabSetUserSpmvDI(KmlScasolverTask **handle, void *ustruct, int (*fptr)(void *ustruct, const double *x, double *y));

参数

参数名	类型	描述	输入/输出
handle	KmlScasolverTask **	求解器句柄，传入之前步骤的变量。	输入/输出
ustruct	void *	指向用户数据的指针，每次调用预条件子时作为第一个参数传递。	输入
(*fptr)	int	用户自定义的函数指针，指向矩阵向量乘函数，该矩阵向量乘函数进行矩阵与其第二参数间的计算。	输入

返回值

返回值	类型	描述
KMLSS_NO_ERROR	int	正常执行。
KMLSS_DATA_SIZE	int	参数nd不等于1。
KMLSS_NULL_ARGUMENT	int	handle，data中存在空参数。

 版权所有 © 华为技术有限公司

6.5.3.4.2.6 KmlScaissBicgstabPcSet?II

设置迭代求解的预条件子相关参数，当前仅支持为block jacobi预条件子设置相关参数。

接口定义

C Interface：

int KmlScaissBicgstabPcSetSII(KmlScasolverTask **handle, enum KmlSolverParam param, const int *data, int nd);

int KmlScaissBicgstabPcSetDII(KmlScasolverTask **handle, enum KmlSolverParam param, const int *data, int nd);

参数

参数名	类型	描述	输入/输出
handle	KmlScasolverTask **	求解器句柄，传入之前步骤的变量。	输入/输出
param	enum KmlSolverParam	• KMLSS_BLOCK_METHOD表示每个分块中的预条件子类型。 • KMLSS_NUM_BLOCKS表示每个进程中的分块数量。 • KMLSS_BLOCK_SIZES表示每个进程中的分块大小。 以下为预条件AMG相关设置项： • KMLSS_AMG_CYCLE表示AMG中CYCLE类型。 • KMLSS_AMG_COARSENING表示粗化算子类型。 • KMLSS_AMG_INTERPOLATION表示插值算子类型。 • KMLSS_AMG_CONNECTION表示粗网格构造方法。	输入
data	const int *	预条件子相关参数（如预条件子类型、分块数量、分块大小）。 当前可选择的预条件子类型： • ILU预条件子：KMLSS_ILU。 • ICC预条件子：KMLSS_ICC。 • SOR预条件子：KMLSS_SOR。 • ILUT预条件子：KMLSS_ILUT。 • 预条件AMG配置项描述具体见**表1 预条件AMG中具体配置项描述**所示。	输入

版权所有 © 华为技术有限公司

参数名	类型	描述	输入/输出
nd	int	data数组元素个数。	输入

返回值

返回值	类型	描述
KMLSS_NO_ERROR	int	正常执行。
KMLSS_DATA_SIZE	int	参数nd不等于1。
KMLSS_NULL_ARGUMENT	int	handle，data中存在空参数。
KMLSS_BAD_SELECTOR	int	param为无效参数。
KMLSS_BAD_PRECONDITIONER	int	选择的预条件子暂未实现。

依赖

#include "kml_scaiss.h"

示例

C Interface：

```
MPI_Init(NULL, NULL);
int size, rank;
MPI_Comm_size(MPI_COMM_WORLD, &size);//获取总进程数
MPI_Comm_rank(MPI_COMM_WORLD, &rank);//获取当前进程标识

int mat_size = 8;
int n = mat_size / size;
int n_beg = n * rank;
if (n * size != mat_size && rank == (size - 1)) {
    n = mat_size - n * rank;
}
int ja[26] = { 0, 3, 4, 1, 2, 3, 5, 1, 2, 7, 0, 1, 3, 6, 0, 4, 5, 1, 4, 5, 7, 3, 6, 2, 5, 7 };
double a[26] = { 1.0, 1.0, 2.0, 9.0, 2.0, 1.0, -3.0, 2.0, 3.0, 2.0, 1.0, 1.0, 9.0, -5.0, 2.0, 6.0, 1.0, -3.0, 1.0, 4.0, 1.0,
-5.0, 7.0, 2.0, 1.0, 2.0 };
int ia[9] = { 0, 3, 7, 10, 14, 17, 21, 23, 26 };
int a_beg = ia[n_beg];
for (int i = n_beg; i < (n_beg + n + 1); i++) {
    ia[i] -= a_beg;
}

/* Internal KML_SCAISS structure */
KmlScasolverTask *handle;

/* KML_SCAISS control parameters */
int error;            /* Output error handle */

/* Create data structures */
const double *a_holder = &a[a_beg];
const int *ja_holder = &ja[a_beg];
```

 版权所有 © 华为技术有限公司

```
const int *ia_holder = &ia[n_beg];
error = KmlScaissBicgstabInitStripesDI(&handle, mat_size, 1, &n, &n_beg, &a_holder, &ja_holder, &ia_holder,
MPI_COMM_WORLD);

double eps = 1e-4;
error = KmlScaissBicgstabSetDID(&handle, KMLSS_THRESHOLD, &eps, 1);
if (error != 0) {
    printf("\nERROR in KmlScaissBicgstabSetDID with KMLSS_THRESHOLD: %d", error);
    return 1;
}
int max_iters = 2000;
error = KmlScaissBicgstabSetDII(&handle, KMLSS_MAX_ITERATION_COUNT, &max_iters, 1);
if (error != 0) {
    printf("\nERROR in KmlScaissBicgstabSetDII with KMLSS_MAX_ITERATION_COUNT: %d", error);
    return 1;
}
int precond = KMLSS_BJACOBI;
error = KmlScaissBicgstabSetDII(&handle, KMLSS_PRECONDITIONER_TYPE, &precond, 1);
if (error != 0) {
    printf("\nERROR in KmlScaissBicgstabSetDII with KMLSS_PRECONDITIONER_TYPE: %d", error);
    return 1;
}
int num_blocks = 1;
error = KmlScaissBicgstabPcSetDII(&handle, KMLSS_NUM_BLOCKS, &num_blocks, 1);
if (error != 0) {
    printf("ERROR in KmlScaissBicgstabPcSetDII: %d\n", error);
    return 1;
}
int subprecond = KMLSS_ILU;
error = KmlScaissBicgstabPcSetDII(&handle, KMLSS_BLOCK_METHOD, &subprecond, 1);
if (error != 0) {
    printf("ERROR in KmlScaissBicgstabPcSetDII: %d\n", error);
    return 1;
}
```

6.5.3.4.2.7 KmlScaissBicgstabSubPcSet?I?

设置迭代求解的block jacobi中子预条件子相关参数。

接口定义

C Interface：

int KmlScaissBicgstabSubPcSetSII(KmlScasolverTask **handle, enum KmlSolverPreconditionerParam param, int subn, const int *data, int nd);

int KmlScaissBicgstabSubPcSetDII(KmlScasolverTask **handle, enum KmlSolverPreconditionerParam param, int subn, const int *data, int nd);

int KmlScaissBicgstabSubPcSetSIS(KmlScasolverTask **handle, enum KmlSolverPreconditionerParam param, int subn, const float *data, int nd);

int KmlScaissBicgstabSubPcSetDID(KmlScasolverTask **handle, enum KmlSolverPreconditionerParam param, int subn, const double *data, int nd);

参数

参数名	类型	描述	输入/输出
handle	KmlScasolverTask **	求解器句柄，传入之前步骤的变量。	输入/输出

版权所有 © 华为技术有限公司

参数名	类型	描述	输入/输出
param	enum KmlSolverParam	• KML_RELAXATION_FACTOR表示SOR预条件子相关参数。 • KMLSS_DROP_FILL表示ILUT预条件子每行最多保留元素个数。 • KMLSS_DROP_TOL表示ILUT预条件子保留元素的阈值。	输入
subn	在KmlScaissBicgstabSubPcSetSIS、KmlScaissBicgstabSubPcSetSII、KmlScaissBicgstabSubPcSetDII、KmlScaissBicgstabSubPcSetDID中为int。	子预条件子索引。	输入
data	• 在KmlScaissBicgstabSubPcSetSII、KmlScaissBicgstabSubPcSetDII中为const int *。 • 在KmlScaissBicgstabSubPcSetSIS中为const float *。 • 在KmlScaissBicgstabSubPcSetDID中为const double *。	预条件子相关参数。	输入
nd	• 在KmlScaissBicgstabSubPcSetSIS、KmlScaissBicgstabSubPcSetSII、KmlScaissBicgstabSubPcSetDII、KmlScaissBicgstabSubPcSetDID中为int。	data数组元素个数。	输入

版权所有 © 华为技术有限公司

返回值

返回值	类型	描述
KMLSS_NO_ERROR	int	正常执行。
KMLSS_DATA_SIZE	int	参数nd不等于1。
KMLSS_NULL_ARGUMENT	int	handle，data中存在空参数。
KMLSS_BAD_SELECTOR	int	param为无效参数。
KMLSS_BAD_PRECONDITIONER	int	选择的预条件子暂未实现。

6.5.3.4.2.8 KmlScaissBicgstabAnalyze?I

分析稀疏矩阵并确定不同进程之间的通信关系。

接口定义

C Interface：

int KmlScaissBicgstabAnalyzeSI(KmlScasolverTask **handle);

int KmlScaissBicgstabAnalyzeDI(KmlScasolverTask **handle);

参数

参数名	类型	描述	输入/输出
handle	KmlScasolverTask **	求解器句柄，传入之前步骤的变量。	输入/输出

返回值

返回值	类型	描述
KMLSS_NO_ERROR	int	正常执行。
KMLSS_NULL_ARGUMENT	int	handle为空参数。

依赖

#include "kml_scaiss.h"

 版权所有 © 华为技术有限公司

示例

C Interface：

```
MPI_Init(NULL, NULL);
int size, rank;
MPI_Comm_size(MPI_COMM_WORLD, &size);//获取总进程数
MPI_Comm_rank(MPI_COMM_WORLD, &rank);//获取当前进程标识

int mat_size = 8;
int n = mat_size / size;
int n_beg = n * rank;
if (n * size != mat_size && rank == (size - 1)) {
    n = mat_size - n * rank;
}
int ja[26] = { 0, 3, 4, 1, 2, 3, 5, 1, 2, 7, 0, 1, 3, 6, 0, 4, 5, 1, 4, 5, 7, 3, 6, 2, 5, 7 };
double a[26] = { 1.0, 1.0, 2.0, 9.0, 2.0, 1.0, -3.0, 2.0, 3.0, 2.0, 1.0, 1.0, 9.0, -5.0, 2.0, 6.0, 1.0, -3.0, 1.0, 4.0, 1.0,
-5.0, 7.0, 2.0, 1.0, 2.0 };
int ia[9] = { 0, 3, 7, 10, 14, 17, 21, 23, 26 };
int a_beg = ia[n_beg];
for (int i = n_beg; i < (n_beg + n + 1); i++) {
    ia[i] -= a_beg;
}

/* Internal KML_SCAISS structure */
KmlScasolverTask *handle;

/* KML_SCAISS control parameters */
int error;            /* Output error handle */

/* Create data structures */
const double *a_holder = &a[a_beg];
const int *ja_holder = &ja[a_beg];
const int *ia_holder = &ia[n_beg];
error = KmlScaissBicgstabInitStripesDI(&handle, mat_size, 1, &n, &n_beg, &a_holder, &ja_holder, &ia_holder,
MPI_COMM_WORLD);

double eps = 1e-4;
error = KmlScaissBicgstabSetDID(&handle, KMLSS_THRESHOLD, &eps, 1);
if (error != 0) {
    printf("\nERROR in KmlScaissBicgstabSetDID with KMLSS_THRESHOLD: %d", error);
    return 1;
}
int max_iters = 2000;
error = KmlScaissBicgstabSetDII(&handle, KMLSS_MAX_ITERATION_COUNT, &max_iters, 1);
if (error != 0) {
    printf("\nERROR in KmlScaissBicgstabSetDII with KMLSS_MAX_ITERATION_COUNT: %d", error);
    return 1;
}
int precond = KMLSS_BJACOBI;
error = KmlScaissBicgstabSetDII(&handle, KMLSS_PRECONDITIONER_TYPE, &precond, 1);
if (error != 0) {
    printf("\nERROR in KmlScaissBicgstabSetDII with KMLSS_PRECONDITIONER_TYPE: %d", error);
    return 1;
}
int num_blocks = 1;
error = KmlScaissBicgstabPcSetDII(&handle, KMLSS_NUM_BLOCKS, &num_blocks, 1);
if (error != 0) {
    printf("ERROR in KmlScaissBicgstabPcSetDII: %d\n", error);
    return 1;
}
int subprecond = KMLSS_ILU;
error = KmlScaissBicgstabPcSetDII(&handle, KMLSS_BLOCK_METHOD, &subprecond, 1);
if (error != 0) {
    printf("ERROR in KmlScaissBicgstabPcSetDII: %d\n", error);
    return 1;
}
error = KmlScaissBicgstabAnalyzeDI(&handle);
if (error != 0) {
```

 版权所有 © 华为技术有限公司

```
    printf("ERROR in KmlScaissBicgstabAnalyzeDI: %d\n", error);
    return 1;
}
```

6.5.3.4.2.9 KmlScaissBicgstabFactorize?I

在求解器内部对矩阵进行赋值，并计算预条件子。无论是否使用预条件子，都需要调用该接口。

接口定义

C Interface：

int KmlScaissBicgstabFactorizeSI(KmlScasolverTask **handle);

int KmlScaissBicgstabFactorizeDI(KmlScasolverTask **handle);

参数

参数名	类型	描述	输入/输出
handle	KmlScasolverTask **	求解器句柄，传入之前步骤的变量。	输入/输出

返回值

返回值	类型	描述
KMLSS_NO_ERROR	int	正常执行。
KMLSS_NULL_ARGUMENT	int	handle为空参数。
KMLSS_ZERO_DIAGONAL_ELEMENT	int	矩阵对角线存在零元。

依赖

#include "kml_scaiss.h"

示例

C Interface：

```
MPI_Init(NULL, NULL);
int size, rank;
MPI_Comm_size(MPI_COMM_WORLD, &size);//获取总进程数
MPI_Comm_rank(MPI_COMM_WORLD, &rank);//获取当前进程标识

int mat_size = 8;
int n = mat_size / size;
int n_beg = n * rank;
if (n * size != mat_size && rank == (size - 1)) {
    n = mat_size - n * rank;
```

 版权所有 © 华为技术有限公司

```
}
int ja[26] = { 0, 3, 4, 1, 2, 3, 5, 1, 2, 7, 0, 1, 3, 6, 0, 4, 5, 1, 4, 5, 7, 3, 6, 2, 5, 7 };
double a[26] = { 1.0, 1.0, 2.0, 9.0, 2.0, 1.0, -3.0, 2.0, 3.0, 2.0, 1.0, 1.0, 9.0, -5.0, 2.0, 6.0, 1.0, -3.0, 1.0, 4.0, 1.0,
-5.0, 7.0, 2.0, 1.0, 2.0 };
int ia[9] = { 0, 3, 7, 10, 14, 17, 21, 23, 26 };
int a_beg = ia[n_beg];
for (int i = n_beg; i < (n_beg + n + 1); i++) {
    ia[i] -= a_beg;
}

/* Internal KML_SCAISS structure */
KmlScasolverTask *handle;

/* KML_SCAISS control parameters */
int error;            /* Output error handle */

/* Create data structures */
const double *a_holder = &a[a_beg];
const int *ja_holder = &ja[a_beg];
const int *ia_holder = &ia[n_beg];
error = KmlScaissBicgstabInitStripesDI(&handle, mat_size, 1, &n, &n_beg, &a_holder, &ja_holder, &ia_holder,
MPI_COMM_WORLD);

double eps = 1e-4;
error = KmlScaissBicgstabSetDID(&handle, KMLSS_THRESHOLD, &eps, 1);
if (error != 0) {
    printf("\nERROR in KmlScaissBicgstabSetDID with KMLSS_THRESHOLD: %d", error);
    return 1;
}
int max_iters = 2000;
error = KmlScaissBicgstabSetDII(&handle, KMLSS_MAX_ITERATION_COUNT, &max_iters, 1);
if (error != 0) {
    printf("\nERROR in KmlScaissBicgstabSetDII with KMLSS_MAX_ITERATION_COUNT: %d", error);
    return 1;
}
int precond = KMLSS_BJACOBI;
error = KmlScaissBicgstabSetDII(&handle, KMLSS_PRECONDITIONER_TYPE, &precond, 1);
if (error != 0) {
    printf("\nERROR in KmlScaissBicgstabSetDII with KMLSS_PRECONDITIONER_TYPE: %d", error);
    return 1;
}
int num_blocks = 1;
error = KmlScaissBicgstabPcSetDII(&handle, KMLSS_NUM_BLOCKS, &num_blocks, 1);
if (error != 0) {
    printf("ERROR in KmlScaissBicgstabPcSetDII: %d\n", error);
    return 1;
}
int subprecond = KMLSS_ILU;
error = KmlScaissBicgstabPcSetDII(&handle, KMLSS_BLOCK_METHOD, &subprecond, 1);
if (error != 0) {
    printf("ERROR in KmlScaissBicgstabPcSetDII: %d\n", error);
    return 1;
}
error = KmlScaissBicgstabAnalyzeDI(&handle);
if (error != 0) {
    printf("ERROR in KmlScaissBicgstabAnalyzeDI: %d\n", error);
    return 1;
}
error = KmlScaissBicgstabFactorizeDI(&handle);
if (error != 0) {
    printf("ERROR in KmlScaissBicgstabFactorizeDI: %d\n", error);
    return 1;
}
```

6.5.3.4.2.10 KmlScaissBicgstabSolve?I

求解线性代数方程组，每个进程读取并存储全局求解结果。

 版权所有 © 华为技术有限公司

接口定义

C Interface：

int KmlScaissBicgstabSolveSI(KmlScasolverTask **handle, int nb, float *x, int ldx, const float *b, int ldb);

int KmlScaissBicgstabSolveDI(KmlScasolverTask **handle, int nb, double *x, int ldx, const double *b, int ldb);

参数

参数名	类型	描述	输入/输出
handle	KmlScasolverTask **	求解器句柄，传入之前步骤的变量。	输入/输出
nb	int	右端项矢量的个数，当前nb只能为1。	输入
x	• 在KmlScaissBicgstabSolveSI中为float *。 • 在KmlScaissBicgstabSolveDI中为double *。	读取并存储全局求解结果，其元素个数为全局矩阵大小n。	输入/输出
ldx	int	x的leading dimension大小，当前参数未使用。	输入
b	• 在KmlScaissBicgstabSolveSI中为const float *。 • 在KmlScaissBicgstabSolveDI中为const double *。	读取进程对应的局部右端项，其元素个数为当前进程对应的矩阵的行数，即stripeWidth。	输入
ldb	int	b的leading dimension大小，当前参数未使用。	输入

返回值

返回值	类型	描述
KMLSS_NO_ERROR	int	正常执行。
KMLSS_BAD_NB	int	参数nb不等于1。

 版权所有 © 华为技术有限公司

返回值	类型	描述
KMLSS_NO_MEMORY	int	内存不足。
KMLSS_NULL_ARGUMENT	int	handle，x，b中存在空参数。

依赖

#include "kml_scaiss.h"

示例

C Interface：

```
MPI_Init(NULL, NULL);
int size, rank;
MPI_Comm_size(MPI_COMM_WORLD, &size);//获取总进程数
MPI_Comm_rank(MPI_COMM_WORLD, &rank);//获取当前进程标识

int mat_size = 8;
int n = mat_size / size;
int n_beg = n * rank;
if (n * size != mat_size && rank == (size - 1)) {
    n = mat_size - n * rank;
}
int ja[26] = { 0, 3, 4, 1, 2, 3, 5, 1, 2, 7, 0, 1, 3, 6, 0, 4, 5, 1, 4, 5, 7, 3, 6, 2, 5, 7 };
double a[26] = { 1.0, 1.0, 2.0, 9.0, 2.0, 1.0, -3.0, 2.0, 3.0, 2.0, 1.0, 1.0, 9.0, -5.0, 2.0, 6.0, 1.0, -3.0, 1.0, 4.0, 1.0,
-5.0, 7.0, 2.0, 1.0, 2.0 };
int ia[9] = { 0, 3, 7, 10, 14, 17, 21, 23, 26 };
int a_beg = ia[n_beg];
for (int i = n_beg; i < (n_beg + n + 1); i++) {
    ia[i] -= a_beg;
}

/* Internal KML_SCAISS structure */
KmlScasolverTask *handle;

/* KML_SCAISS control parameters */
int error;            /* Output error handle */

/* Create data structures */
const double *a_holder = &a[a_beg];
const int *ja_holder = &ja[a_beg];
const int *ia_holder = &ia[n_beg];
error = KmlScaissBicgstabInitStripesDI(&handle, mat_size, 1, &n, &n_beg, &a_holder, &ja_holder, &ia_holder,
MPI_COMM_WORLD);

double eps = 1e-4;
error = KmlScaissBicgstabSetDID(&handle, KMLSS_THRESHOLD, &eps, 1);
if (error != 0) {
    printf("\nERROR in KmlScaissBicgstabSetDID with KMLSS_THRESHOLD: %d", error);
    return 1;
}
int max_iters = 2000;
error = KmlScaissBicgstabSetDII(&handle, KMLSS_MAX_ITERATION_COUNT, &max_iters, 1);
if (error != 0) {
    printf("\nERROR in KmlScaissBicgstabSetDII with KMLSS_MAX_ITERATION_COUNT: %d", error);
    return 1;
}
int precond = KMLSS_BJACOBI;
error = KmlScaissBicgstabSetDII(&handle, KMLSS_PRECONDITIONER_TYPE, &precond, 1);
if (error != 0) {
```

 版权所有 © 华为技术有限公司

```
    printf("\nERROR in KmlScaissBicgstabSetDII with KMLSS_PRECONDITIONER_TYPE: %d", error);
    return 1;
}
int num_blocks = 1;
error = KmlScaissBicgstabPcSetDII(&handle, KMLSS_NUM_BLOCKS, &num_blocks, 1);
if (error != 0) {
    printf("ERROR in KmlScaissBicgstabPcSetDII: %d\n", error);
    return 1;
}
int subprecond = KMLSS_ILU;
error = KmlScaissBicgstabPcSetDII(&handle, KMLSS_BLOCK_METHOD, &subprecond, 1);
if (error != 0) {
    printf("ERROR in KmlScaissBicgstabPcSetDII: %d\n", error);
    return 1;
}
error = KmlScaissBicgstabAnalyzeDI(&handle);
if (error != 0) {
    printf("ERROR in KmlScaissBicgstabAnalyzeDI: %d\n", error);
    return 1;
}
error = KmlScaissBicgstabFactorizeDI(&handle);
if (error != 0) {
    printf("ERROR in KmlScaissBicgstabFactorizeDI: %d\n", error);
    return 1;
}

int nrhs = 1;
int ldx = mat_size, ldb = n;
double b[8] = { 4.0, 9.0, 7.0, 6.0, 9.0, 3.0, 2.0, 5.0 };
double x[8] = { 0.0, 0.0, 0.0, 0.0, 0.0, 0.0, 0.0, 0.0 };
error = KmlScaissBicgstabSolveDI(&handle, nrhs, &x[0], ldx, &b[n_beg], ldb);
if (error != 0) {
    printf("ERROR in KmlScaissBicgstabSolveDI: %d\n", error);
    return 1;
}
```

6.5.3.4.2.11 KmlScaissBicgstabSolveDx?I

求解线性代数方程组，每个进程读取并存储当前进程对应的求解结果。

接口定义

C Interface：

int KmlScaissBicgstabSolveDxSI(KmlScasolverTask **handle, int nb, float *x, int ldx, const float *b, int ldb);

int KmlScaissBicgstabSolveDxDI(KmlScasolverTask **handle, int nb, double *x, int ldx, const double *b, int ldb);

参数

参数名	类型	描述	输入/输出
handle	KmlScasolverTask **	求解器句柄，传入之前步骤的变量。	输入/输出
nb	int	右端项矢量的个数，当前nb只能为1。	输入

版权所有 © 华为技术有限公司

参数名	类型	描述	输入/输出
x	• 在KmlScaissBicgstabSolveDxSI中为float *。 • 在KmlScaissBicgstabSolveDxDI中为double *。	读取并存储当前进程对应的求解结果，其元素个数为当前进程对应的矩阵的行数，即stripeWidth。	输入/输出
ldx	int	x的leading dimension大小，当前参数未使用。	输入
b	• 在KmlScaissBicgstabSolveDxSI中为const float *。 • 在KmlScaissBicgstabSolveDxDI中为const double *。	读取进程对应的局部右端项，其元素个数为当前进程对应的矩阵的行数，即stripeWidth。	输入
ldb	int	b的leading dimension大小，当前参数未使用。	输入

返回值

返回值	类型	描述
KMLSS_NO_ERROR	int	正常执行。
KMLSS_BAD_NB	int	参数nb不等于1。
KMLSS_NO_MEMORY	int	内存不足。
KMLSS_NULL_ARGUMENT	int	handle，x，b中存在空参数。

依赖

#include "kml_scaiss.h"

示例

C Interface：

```
MPI_Init(NULL, NULL);
int size, rank;
MPI_Comm_size(MPI_COMM_WORLD, &size);//获取总进程数
```

 版权所有 © 华为技术有限公司

```
MPI_Comm_rank(MPI_COMM_WORLD, &rank);//获取当前进程标识

int mat_size = 8;
int n = mat_size / size;
int n_beg = n * rank;
if (n * size != mat_size && rank == (size - 1)) {
    n = mat_size - n * rank;
}
int ja[26] = { 0, 3, 4, 1, 2, 3, 5, 1, 2, 7, 0, 1, 3, 6, 0, 4, 5, 1, 4, 5, 7, 3, 6, 2, 5, 7 };
double a[26] = { 1.0, 1.0, 2.0, 9.0, 2.0, 1.0, -3.0, 2.0, 3.0, 2.0, 1.0, 1.0, 9.0, -5.0, 2.0, 6.0, 1.0, -3.0, 1.0, 4.0, 1.0,
-5.0, 7.0, 2.0, 1.0, 2.0 };
int ia[9] = { 0, 3, 7, 10, 14, 17, 21, 23, 26 };
int a_beg = ia[n_beg];
for (int i = n_beg; i < (n_beg + n + 1); i++) {
    ia[i] -= a_beg;
}

/* Internal KML_SCAISS structure */
KmlScasolverTask *handle;

/* KML_SCAISS control parameters */
int error;            /* Output error handle */

/* Create data structures */
const double *a_holder = &a[a_beg];
const int *ja_holder = &ja[a_beg];
const int *ia_holder = &ia[n_beg];
error = KmlScaissBicgstabInitStripesDI(&handle, mat_size, 1, &n, &n_beg, &a_holder, &ja_holder, &ia_holder,
MPI_COMM_WORLD);

double eps = 1e-4;
error = KmlScaissBicgstabSetDID(&handle, KMLSS_THRESHOLD, &eps, 1);
if (error != 0) {
    printf("\nERROR in KmlScaissBicgstabSetDID with KMLSS_THRESHOLD: %d", error);
    return 1;
}
int max_iters = 2000;
error = KmlScaissBicgstabSetDII(&handle, KMLSS_MAX_ITERATION_COUNT, &max_iters, 1);
if (error != 0) {
    printf("\nERROR in KmlScaissBicgstabSetDII with KMLSS_MAX_ITERATION_COUNT: %d", error);
    return 1;
}
int precond = KMLSS_BJACOBI;
error = KmlScaissBicgstabSetDII(&handle, KMLSS_PRECONDITIONER_TYPE, &precond, 1);
if (error != 0) {
    printf("\nERROR in KmlScaissBicgstabSetDII with KMLSS_PRECONDITIONER_TYPE: %d", error);
    return 1;
}
int num_blocks = 1;
error = KmlScaissBicgstabPcSetDII(&handle, KMLSS_NUM_BLOCKS, &num_blocks, 1);
if (error != 0) {
    printf("ERROR in KmlScaissBicgstabPcSetDII: %d\n", error);
    return 1;
}
int subprecond = KMLSS_ILU;
error = KmlScaissBicgstabPcSetDII(&handle, KMLSS_BLOCK_METHOD, &subprecond, 1);
if (error != 0) {
    printf("ERROR in KmlScaissBicgstabPcSetDII: %d\n", error);
    return 1;
}
error = KmlScaissBicgstabAnalyzeDI(&handle);
if (error != 0) {
    printf("ERROR in KmlScaissBicgstabAnalyzeDI: %d\n", error);
    return 1;
}
error = KmlScaissBicgstabFactorizeDI(&handle);
if (error != 0) {
    printf("ERROR in KmlScaissBicgstabFactorizeDI: %d\n", error);
    return 1;
```

 版权所有 © 华为技术有限公司

```
}

int nrhs = 1;
int ldx = n, ldb = n;
double b[8] = { 4.0, 9.0, 7.0, 6.0, 9.0, 3.0, 2.0, 5.0 };
double x[8] = { 0.0, 0.0, 0.0, 0.0, 0.0, 0.0, 0.0, 0.0 };
error = KmlScaissBicgstabSolveDxDI(&handle, nrhs, &x[n_beg], ldx, &b[n_beg], ldb);
if (error != 0) {
    printf("ERROR in KmlScaissBicgstabSolveDxDI: %d\n", error);
    return 1;
}
```

6.5.3.4.2.12 KmlScaissBicgstabGet?I?

获得迭代求解相关参数。

接口定义

C Interface：

int KmlScaissBicgstabGetSII(KmlScasolverTask **handle, enum KmlSolverParam param, int *data, int nd);

int KmlScaissBicgstabGetSIS(KmlScasolverTask **handle, enum KmlSolverParam param, float *data, int nd);

int KmlScaissBicgstabGetDII(KmlScasolverTask **handle, enum KmlSolverParam param, int *data, int nd);

int KmlScaissBicgstabGetDID(KmlScasolverTask **handle, enum KmlSolverParam param, double *data, int nd);

参数

参数名	类型	描述	输入/输出
handle	KmlScasolverTask **	求解器句柄，传入之前步骤的变量。	输入/输出
param	enum KmlSolverParam	• KMLSS_ITERATION_COUNT表示迭代次数。 • KMLSS_MAX_ITERATION_COUNT表示设置的最大迭代次数。 • KMLSS_TOLERANCE表示最终的迭代相对残差。 • KMLSS_THRESHOLD表示设置的相对残差。	输入

版权所有 © 华为技术有限公司

参数名	类型	描述	输入/输出
data	• 在KmlScaissBicgstabGetSII/KmlScaissBicgstabGetDII中为int *。 • 在KmlScaissBicgstabGetSIS中为float *。 • 在KmlScaissBicgstabGetDID中为double *。	• 在KmlScaissCgGetSII/KmlScaissCgGetDII中为迭代步数。 • 在KmlScaissCgGetSIS中为残差。 • 在KmlScaissCgGetDID中为残差。	输出
nd	int	data数组元素个数。	输入

返回值

返回值	类型	描述
KMLSS_NO_ERROR	int	正常执行。
KMLSS_DATA_SIZE	int	参数nd不等于1。
KMLSS_NULL_ARGUMENT	int	handle，data中存在空参数。
KMLSS_BAD_SELECTOR	int	param为无效参数。

依赖

#include "kml_scaiss.h"

示例

C Interface：

```
MPI_Init(NULL, NULL);
int size, rank;
MPI_Comm_size(MPI_COMM_WORLD, &size);//获取总进程数
MPI_Comm_rank(MPI_COMM_WORLD, &rank);//获取当前进程标识

int mat_size = 8;
int n = mat_size / size;
int n_beg = n * rank;
if (n * size != mat_size && rank == (size - 1)) {
    n = mat_size - n * rank;
}
int ja[26] = { 0, 3, 4, 1, 2, 3, 5, 1, 2, 7, 0, 1, 3, 6, 0, 4, 5, 1, 4, 5, 7, 3, 6, 2, 5, 7 };
double a[26] = { 1.0, 1.0, 2.0, 9.0, 2.0, 1.0, -3.0, 2.0, 3.0, 2.0, 1.0, 1.0, 9.0, -5.0, 2.0, 6.0, 1.0, -3.0, 1.0, 4.0, 1.0,
-5.0, 7.0, 2.0, 1.0, 2.0 };
```

 版权所有 © 华为技术有限公司

```
int ia[9] = { 0, 3, 7, 10, 14, 17, 21, 23, 26 };
int a_beg = ia[n_beg];
for (int i = n_beg; i < (n_beg + n + 1); i++) {
    ia[i] -= a_beg;
}

/* Internal KML_SCAISS structure */
KmlScasolverTask *handle;

/* KML_SCAISS control parameters */
int error;              /* Output error handle */

/* Create data structures */
const double *a_holder = &a[a_beg];
const int *ja_holder = &ja[a_beg];
const int *ia_holder = &ia[n_beg];
error = KmlScaissBicgstabInitStripesDI(&handle, mat_size, 1, &n, &n_beg, &a_holder, &ja_holder, &ia_holder,
MPI_COMM_WORLD);

double eps = 1e-4;
error = KmlScaissBicgstabSetDID(&handle, KMLSS_THRESHOLD, &eps, 1);
if (error != 0) {
    printf("\nERROR in KmlScaissBicgstabSetDID with KMLSS_THRESHOLD: %d", error);
    return 1;
}
int max_iters = 2000;
error = KmlScaissBicgstabSetDII(&handle, KMLSS_MAX_ITERATION_COUNT, &max_iters, 1);
if (error != 0) {
    printf("\nERROR in KmlScaissBicgstabSetDII with KMLSS_MAX_ITERATION_COUNT: %d", error);
    return 1;
}
int precond = KMLSS_BJACOBI;
error = KmlScaissBicgstabSetDII(&handle, KMLSS_PRECONDITIONER_TYPE, &precond, 1);
if (error != 0) {
    printf("\nERROR in KmlScaissBicgstabSetDII with KMLSS_PRECONDITIONER_TYPE: %d", error);
    return 1;
}
int num_blocks = 1;
error = KmlScaissBicgstabPcSetDII(&handle, KMLSS_NUM_BLOCKS, &num_blocks, 1);
if (error != 0) {
    printf("ERROR in KmlScaissBicgstabPcSetDII: %d\n", error);
    return 1;
}
int subprecond = KMLSS_ILU;
error = KmlScaissBicgstabPcSetDII(&handle, KMLSS_BLOCK_METHOD, &subprecond, 1);
if (error != 0) {
    printf("ERROR in KmlScaissBicgstabPcSetDII: %d\n", error);
    return 1;
}
error = KmlScaissBicgstabAnalyzeDI(&handle);
if (error != 0) {
    printf("ERROR in KmlScaissBicgstabAnalyzeDI: %d\n", error);
    return 1;
}
error = KmlScaissBicgstabFactorizeDI(&handle);
if (error != 0) {
    printf("ERROR in KmlScaissBicgstabFactorizeDI: %d\n", error);
    return 1;
}

int nrhs = 1;
int ldx = n, ldb = n;
double b[8] = { 4.0, 9.0, 7.0, 6.0, 9.0, 3.0, 2.0, 5.0 };
double x[8] = { 0.0, 0.0, 0.0, 0.0, 0.0, 0.0, 0.0, 0.0 };
error = KmlScaissBicgstabSolveDxDI(&handle, nrhs, &x[n_beg], ldx, &b[n_beg], ldb);
if (error != 0) {
    printf("ERROR in KmlScaissBicgstabSolveDI: %d\n", error);
    return 1;
}
```

 版权所有 © 华为技术有限公司

```
int idata[] = {0};
error = KmlScaissBicgstabGetDII(&handle, KMLSS_ITERATION_COUNT, idata, 1);
if (error != 0) {
    printf("\nERROR in KmlScaissBicgstabGetDII: %d", error);
    return 1;
}
double ddata[] = {0.0};
error = KmlScaissBicgstabGetDID(&handle, KMLSS_TOLERANCE, ddata, 1);
if (error != 0) {
    printf("\nERROR in KmlScaissBicgstabGetDID: %d", error);
    return 1;
}
if (rank == 0) printf("[Solve info] Res= \t%.12g\t, iteration = \t%d\t\n", sqrt(ddata[0]), idata[0]);
```

运行结果：

```
[Solve info] Res=       1.15345976131e-14       , iteration =   8
```

6.5.3.4.2.13 KmlScaissBicgstabClean?I

释放内部数据结构。

接口定义

C Interface：

int KmlScaissBicgstabCleanSI(KmlScaSolverTask **handle);

int KmlScaissBicgstabCleanDI(KmlScaSolverTask **handle);

参数

参数名	类型	描述	输入/输出
handle	KmlScasolverTask **	求解器句柄，传入之前步骤的变量。	输入/输出

返回值

返回值	类型	描述
KMLSS_NO_ERROR	int	正常执行。
KMLSS_NULL_ARGUMENT	int	handle为空参数。

依赖

#include "kml_scaiss.h"

示例

C Interface：

 版权所有 © 华为技术有限公司

```
MPI_Init(NULL, NULL);
int size, rank;
MPI_Comm_size(MPI_COMM_WORLD, &size);//获取总进程数
MPI_Comm_rank(MPI_COMM_WORLD, &rank);//获取当前进程标识

int mat_size = 8;
int n = mat_size / size;
int n_beg = n * rank;
if (n * size != mat_size && rank == (size - 1)) {
    n = mat_size - n * rank;
}
int ja[26] = { 0, 3, 4, 1, 2, 3, 5, 1, 2, 7, 0, 1, 3, 6, 0, 4, 5, 1, 4, 5, 7, 3, 6, 2, 5, 7 };
double a[26] = { 1.0, 1.0, 2.0, 9.0, 2.0, 1.0, -3.0, 2.0, 3.0, 2.0, 1.0, 1.0, 9.0, -5.0, 2.0, 6.0, 1.0, -3.0, 1.0, 4.0, 1.0,
-5.0, 7.0, 2.0, 1.0, 2.0 };
int ia[9] = { 0, 3, 7, 10, 14, 17, 21, 23, 26 };
int a_beg = ia[n_beg];
for (int i = n_beg; i < (n_beg + n + 1); i++) {
    ia[i] -= a_beg;
}

/* Internal KML_SCAISS structure */
KmlScasolverTask *handle;

/* KML_SCAISS control parameters */
int error;            /* Output error handle */

/* Create data structures */
const double *a_holder = &a[a_beg];
const int *ja_holder = &ja[a_beg];
const int *ia_holder = &ia[n_beg];
error = KmlScaissBicgstabInitStripesDI(&handle, mat_size, 1, &n, &n_beg, &a_holder, &ja_holder, &ia_holder,
MPI_COMM_WORLD);

double eps = 1e-4;
error = KmlScaissBicgstabSetDID(&handle, KMLSS_THRESHOLD, &eps, 1);
if (error != 0) {
    printf("\nERROR in KmlScaissBicgstabSetDID with KMLSS_THRESHOLD: %d", error);
    return 1;
}
int max_iters = 2000;
error = KmlScaissBicgstabSetDII(&handle, KMLSS_MAX_ITERATION_COUNT, &max_iters, 1);
if (error != 0) {
    printf("\nERROR in KmlScaissBicgstabSetDII with KMLSS_MAX_ITERATION_COUNT: %d", error);
    return 1;
}
int precond = KMLSS_BJACOBI;
error = KmlScaissBicgstabSetDII(&handle, KMLSS_PRECONDITIONER_TYPE, &precond, 1);
if (error != 0) {
    printf("\nERROR in KmlScaissBicgstabSetDII with KMLSS_PRECONDITIONER_TYPE: %d", error);
    return 1;
}
int num_blocks = 1;
error = KmlScaissBicgstabPcSetDII(&handle, KMLSS_NUM_BLOCKS, &num_blocks, 1);
if (error != 0) {
    printf("ERROR in KmlScaissBicgstabPcSetDII: %d\n", error);
    return 1;
}
int subprecond = KMLSS_ILU;
error = KmlScaissBicgstabPcSetDII(&handle, KMLSS_BLOCK_METHOD, &subprecond, 1);
if (error != 0) {
    printf("ERROR in KmlScaissBicgstabPcSetDII: %d\n", error);
    return 1;
}
error = KmlScaissBicgstabAnalyzeDI(&handle);
if (error != 0) {
    printf("ERROR in KmlScaissBicgstabAnalyzeDI: %d\n", error);
    return 1;
}
error = KmlScaissBicgstabFactorizeDI(&handle);
```

 版权所有 © 华为技术有限公司

```
if (error != 0) {
    printf("ERROR in KmlScaissBicgstabFactorizeDI: %d\n", error);
    return 1;
}

int nrhs = 1;
int ldx = n, ldb = n;
double b[8] = { 4.0, 9.0, 7.0, 6.0, 9.0, 3.0, 2.0, 5.0 };
double x[8] = { 0.0, 0.0, 0.0, 0.0, 0.0, 0.0, 0.0, 0.0 };
error = KmlScaissBicgstabSolveDxDI(&handle, nrhs, &x[n_beg], ldx, &b[n_beg], ldb);
if (error != 0) {
    printf("ERROR in KmlScaissBicgstabSolveDI: %d\n", error);
    return 1;
}

int idata[] = {0};
error = KmlScaissBicgstabGetDII(&handle, KMLSS_ITERATION_COUNT, idata, 1);
if (error != 0) {
    printf("\nERROR in KmlScaissBicgstabGetDII: %d", error);
    return 1;
}
double ddata[] = {0.0};
error = KmlScaissBicgstabGetDID(&handle, KMLSS_TOLERANCE, ddata, 1);
if (error != 0) {
    printf("\nERROR in KmlScaissBicgstabGetDID: %d", error);
    return 1;
}
if (rank == 0) printf("[Solve info] Res= \t%.12g\t, iteration = \t%d\t\n", sqrt(ddata[0]), idata[0]);

error = KmlScaissBicgstabCleanDI(&handle);
if (error != 0) {
    printf("\nERROR in KmlScaissBicgstabCleanDI: %d", error);
    return 1;
}
```

6.5.3.4.2.14 示例

依赖

#include "kml_scaiss.h"

示例

C Interface：

```
#include <stdio.h>
#include <kml_scaiss.h>
#include "mpi.h"
// USER data struct
typedef struct {
    int n;
    const int *ia;
    const int *ja;
    const double *a;
    MPI_Comm comm;
} duser;
int diagonal_preconditioner(void *usr, double *x)
{
    duser *u = (duser *)usr;
    int i, j;
    /* Apply diagonal preconditioner */
    for (i = 0; i < u->n; i++) {
        for (j = u->ia[i]; j < u->ia[i + 1]; j++) {
            if (u->ja[j] == i) {
                x[i] /= u->a[j];
            }
        }
    }
```

 版权所有 © 华为技术有限公司

```
    return 0;
}
int user_spmv(void *usr, const double *x, double *y)
{
    duser *u = (duser *)usr;
    int size;
    int rank;
    MPI_Comm_size(u->comm, &size);
    MPI_Comm_rank(u->comm, &rank);
    int n = 8 / size;
    double fullX[8] = { 0.0 };
    MPI_Allgather(&x[0], n, MPI_DOUBLE, &fullX[0], n, MPI_DOUBLE, u->comm);
    int i, j;
    for (i = 0; i < u->n; i++) {
        double sum = 0.0;
        for (j = u->ia[i]; j < u->ia[i + 1]; j++) {
            int k = u->ja[j];
            double value = u->a[j];
            sum += fullX[k] * value;
        }
        y[i] = sum;
    }
    return 0;
}
int main(void)
{
    /* MPI initialization */
    MPI_Init(NULL, NULL);
    int size, rank;
    MPI_Comm_size(MPI_COMM_WORLD, &size);
    MPI_Comm_rank(MPI_COMM_WORLD, &rank);
    /* Matrix data (CSR, stored full matrix)
        | 1   *   *   1   2   *   *   * |
        | *   9   2   1   *  -3   *   * |
        | *   2   3   *   *   *   *   2 |
        | 1   1   *   9   *   *  -5   * |
        | 2   *   *   *   6   1   *   * |
        | *  -3   *   *   1   4   *   1 |
        | *   *   *  -5   *   *   7   * |
        | *   *   2   *   *   1   *   2 |
    */
    /* Initialize separations */
    int mat_size = 8;
    int n = mat_size / size;
    int n_beg = n * rank;
    if (n * size != 8 && rank == (size - 1)) {
        n = mat_size - n * rank;
    }
    int ia[9] = { 0, 3, 7, 10, 14, 17, 21, 23, 26 };
    int a_beg = ia[n_beg];
    for (int i = n_beg; i < (n_beg + n + 1); i++) {
        ia[i] -= a_beg;
    }
    /* clang-format off */
    int ja[26] = { 0,       3, 4,
                   1, 2, 3,    5,
                   1, 2,          7,
                0, 1,    3,       6,
                0,          4, 5,
                   1,       4, 5,    7,
                        3,       6,
                     2,       5,    7 };
    double a[26] = { 1.0,           1.0, 2.0,
                        9.0, 2.0, 1.0,     -3.0,
                        2.0, 3.0,                  2.0,
                     1.0, 1.0,      9.0,          -5.0,
                     2.0,               6.0, 1.0,
                         -3.0,          1.0, 4.0,      1.0,
                                   -5.0,          7.0,
```


版权所有 © 华为技术有限公司

```
                                2.0,          1.0,      2.0 };
    /* clang-format on */
    /* Right-hand side vector */
    double b[8] = { 4.0, 9.0, 7.0, 6.0, 9.0, 3.0, 2.0, 5.0 };
    /* Solution vector */
    double x[8] = { 0.0, 0.0, 0.0, 0.0, 0.0, 0.0, 0.0, 0.0 };
    /* Internal KML_SCAISS structure */
    KmlScasolverTask *handle;
    /* KML_SCAISS control parameters */
    int nrhs = 1;          /* Number of right-hand sides */
    int ldx = n, ldb = n; /*!Leading dimension of B and X */
    int error;             /* Output error handle */
    /* Create data structures */
    const double *a_holder = &a[a_beg];
    const int *ja_holder = &ja[a_beg];
    const int *ia_holder = &ia[n_beg];
    error = KmlScaissBicgstabInitStripesDI(&handle, mat_size, 1, &n, &n_beg, &a_holder, &ja_holder,
&ia_holder,
        MPI_COMM_WORLD);
    if (error != 0) {
        printf("ERROR in KmlScaissBicgstabInitStripesDI: %d\n", error);
        return 1;
    }
    error = KmlScaissBicgstabAnalyzeDI(&handle);
    if (error != 0) {
        printf("ERROR in KmlScaissBicgstabAnalyzeDI: %d\n", error);
        return 1;
    }
    error = KmlScaissBicgstabFactorizeDI(&handle);
    if (error != 0) {
        printf("ERROR in KmlScaissBicgstabFactorizeDI: %d\n", error);
        return 1;
    }
    /* solve */
    double x7[8] = { 0.0, 0.0, 0.0, 0.0, 0.0, 0.0, 0.0, 0.0 };
    error = KmlScaissBicgstabSolveDI(&handle, nrhs, &x7[0], ldx, &b[n_beg], ldb);
    if (error != 0) {
        printf("ERROR in KmlScaissBicgstabSolveDI: %d\n", error);
        return 1;
    }
    /* Print the solution */
    if (rank == 0) {
        printf("solve, x:\n");
        for (int i = 0; i < n; i++) {
            printf("%lf\n", x7[i]);
        }
    }
    /* solveDx */
    double x0[8] = { 0.0, 0.0, 0.0, 0.0, 0.0, 0.0, 0.0, 0.0 };
    error = KmlScaissBicgstabSolveDxDI(&handle, nrhs, &x0[n_beg], ldx, &b[n_beg], ldb);
    if (error != 0) {
        printf("ERROR in KmlScaissBicgstabSolveDI: %d\n", error);
        return 1;
    }
    /* Print the solution */
    if (rank == 0) {
        printf("solveDx, x:\n");
        for (int i = 0; i < n; i++) {
            printf("%lf\n", x0[i]);
        }
    }
    /* L1 norm */
    int NormType = KMLSS_L1;
    error = KmlScaissBicgstabSetDII(&handle, KMLSS_VECTOR_NORM_TYPE, &NormType, 1);
    if (error != 0) {
        printf("ERROR in KmlScaissBicgstabSetDII: %d\n", error);
        return 1;
    }
    /* Solve */
```

 版权所有 © 华为技术有限公司

```
error = KmlScaissBicgstabSolveDI(&handle, nrhs, &x[0], ldx, &b[n_beg], ldb);
if (error != 0) {
    printf("ERROR in KmlScaissBicgstabSolveDI: %d\n", error);
    return 1;
}
/* Print the solution */
if (rank == 0) {
    printf("L1 norm, x:\n");
    for (int i = 0; i < mat_size; i++) {
        printf("%lf\n", x[i]);
    }
}
/* SolveDx */
double x6[8] = { 0.0, 0.0, 0.0, 0.0, 0.0, 0.0, 0.0, 0.0 };
error = KmlScaissBicgstabSolveDxDI(&handle, nrhs, &x6[n_beg], ldx, &b[n_beg], ldb);
if (error != 0) {
    printf("ERROR in KmlScaissBicgstabSolveDI: %d\n", error);
    return 1;
}
/* Print the solution */
if (rank == 0) {
    printf("L2 norm, x:\n");
    for (int i = 0; i < n; i++) {
        printf("%lf\n", x6[i]);
    }
}
/* L2 norm */
NormType = KMLSS_L2;
error = KmlScaissBicgstabSetDII(&handle, KMLSS_VECTOR_NORM_TYPE, &NormType, 1);
if (error != 0) {
    printf("ERROR in KmlScaissBicgstabSetDII: %d\n", error);
    return 1;
}
/* Solve */
double x2[8] = { 0.0, 0.0, 0.0, 0.0, 0.0, 0.0, 0.0, 0.0 };
error = KmlScaissBicgstabSolveDI(&handle, nrhs, &x2[0], ldx, &b[n_beg], ldb);
if (error != 0) {
    printf("ERROR in KmlScaissBicgstabSolveDI: %d\n", error);
    return 1;
}
/* Print the solution */
if (rank == 0) {
    printf("L2 norm, x:\n");
    for (int i = 0; i < mat_size; i++) {
        printf("%lf\n", x2[i]);
    }
}
/* abs residual */
double res = 1e-10;
error = KmlScaissBicgstabSetDID(&handle, KMLSS_ABS_TOLERANCE, &res, 1);
if (error != 0) {
    printf("ERROR in KmlScaissBicgstabSetDIA: %d\n", error);
    return 1;
}
/* Solve */
double x3[8] = { 0.0, 0.0, 0.0, 0.0, 0.0, 0.0, 0.0, 0.0 };
error = KmlScaissBicgstabSolveDI(&handle, nrhs, &x3[0], ldx, &b[n_beg], ldb);
if (error != 0) {
    printf("ERROR in KmlScaissBicgstabSolveDI: %d\n", error);
    return 1;
}
/* Print the solution */
if (rank == 0) {
    printf("abs residual, x:\n");
    for (int i = 0; i < mat_size; i++) {
        printf("%lf\n", x3[i]);
    }
}
/* relative residual */
```

 版权所有 © 华为技术有限公司

```
res = 1e-10;
error = KmlScaissBicgstabSetDID(&handle, KMLSS_THRESHOLD, &res, 1);
if (error != 0) {
    printf("ERROR in KmlScaissBicgstabSetDIA: %d\n", error);
    return 1;
}
/* Solve */
double x4[8] = { 0.0, 0.0, 0.0, 0.0, 0.0, 0.0, 0.0, 0.0 };
error = KmlScaissBicgstabSolveDI(&handle, nrhs, &x4[0], ldx, &b[n_beg], ldb);
if (error != 0) {
    printf("ERROR in KmlScaissBicgstabSolveDI: %d\n", error);
    return 1;
}
/* Print the solution */
if (rank == 0) {
    printf("relative residual, x:\n");
    for (int i = 0; i < mat_size; i++) {
        printf("%lf\n", x4[i]);
    }
}
/* Second call with preconditioner */
double x5[8] = { 0.0, 0.0, 0.0, 0.0, 0.0, 0.0, 0.0, 0.0 };
duser prea;
prea.n = n;
prea.ia = &ia[n_beg];
prea.ja = &ja[a_beg];
prea.a = &a[a_beg];
/* Set user preconditioner */
error = KmlScaissBicgstabSetUserPreconditionerDI(&handle, &prea, &diagonal_preconditioner);
if (error != 0) {
    printf("ERROR in KmlScaissBicgstabSetUserPreconditionerDI: %d\n", error);
    return 1;
}
/* Solve */
error = KmlScaissBicgstabSolveDI(&handle, nrhs, &x5[0], ldx, &b[n_beg], ldb);
if (error != 0) {
    printf("ERROR in KmlScaissBicgstabSolveDI: %d\n", error);
    return 1;
}
/* Finalize and Clean-up */
error = KmlScaissBicgstabCleanDI(&handle);
if (error != 0) {
    printf("ERROR in KmlScaissBicgstabCleanDI: %d\n", error);
    return 1;
}
/* Print the solution */
if (rank == 0) {
    printf("user preconditioner, x:\n");
    for (int i = 0; i < mat_size; i++) {
        printf("%lf\n", x5[i]);
    }
}
/* Second call with user spmv */
KmlScasolverTask *handle2;
double x8[8] = { 0.0, 0.0, 0.0, 0.0, 0.0, 0.0, 0.0, 0.0 };
duser prea2;
prea2.n = n;
prea2.ia = &ia[n_beg];
prea2.ja = &ja[a_beg];
prea2.a = &a[a_beg];
prea2.comm = MPI_COMM_WORLD;
error = KmlScaissBicgstabInitWithoutMatDI(&handle2, 1, &n, MPI_COMM_WORLD);
if (error != 0) {
    printf("ERROR in KmlScaissBicgstabInitWithoutMatDI: %d\n", error);
    return 1;
}
/* Set user spmv */
error = KmlScaissBicgstabSetUserSpmvDI(&handle2, &prea2, &user_spmv);
if (error != 0) {
```

 版权所有 © 华为技术有限公司

```
        printf("ERROR in KmlScaissBicgstabSetUserSpmvDI: %d\n", error);
        return 1;
    }
    /* Solve */
    error = KmlScaissBicgstabSolveDxDI(&handle2, nrhs, &x8[n_beg], ldx, &b[n_beg], ldb);
    if (error != 0) {
        printf("ERROR in KmlScaissBicgstabSolveDI: %d\n", error);
        return 1;
    }
    /* Finalize and Clean-up */
    error = KmlScaissBicgstabCleanDI(&handle2);
    if (error != 0) {
        printf("ERROR in KmlScaissBicgstabCleanDI: %d\n", error);
        return 1;
    }
    /* Print the solution */
    if (rank == 0) {
        printf("user spmv, x:\n");
        for (int i = 0; i < n; i++) {
            printf("%lf\n", x8[i]);
        }
    }
    MPI_Finalize();
    return 0;
}
```

6.5.3.4.3 SCAISS-GMRES

6.5.3.4.3.1 KmlScaissGmresInitStripes?I

初始化数据结构，并将用户提供的系数矩阵关联到求解系统中，初始化集群版的GMRES方法时，每个进程使用局部的CSR格式矩阵。

接口定义

C Interface：

int KmlScaissGmresInitStripesSI(KmlScasolverTask **handle, int n, int nstripes, const int *stripeWidth,const int *stripeRow, const float * const *a, const int * const *ja, const int * const *ia, MPI_Comm comm);

int KmlScaissGmresInitStripesDI(KmlScasolverTask **handle, int n, int nstripes, const int *stripeWidth, const int *stripeRow, const double * const *a, const int * const *ja, const int * const *ia, MPI_Comm comm);

参数

参数名	类型	描述	输入/输出
handle	KmlScasolverTask **	求解器句柄，handle不可为空指针。	输入/输出
n	int	n为矩阵A的维度。	输入
nstripes	int	nstripes为存储在当前MPI进程中的矩阵数，目前只能取1。	输入

版权所有 © 华为技术有限公司

参数名	类型	描述	输入/输出
stripeWidth	int *	stripeWidth为当前MPI进程中的矩阵行数。该变量的生命周期与handle相同。	输入
stripeRow	int *	stripeRow为当前MPI进程中的矩阵的全局行索引，即首行在全局矩阵中对应行数。该数组的生命周期与handle相同。	输入
a	• 在KmlScaissGmresInitStripesSI中为float *。 • 在KmlScaissGmresInitStripesDI中为double *。	a为维度为非零元素的个数的数组，a[i]为第i个非零元的值。该数组的生命周期与handle相同。	输入
ja	int *	ja为维度为非零元素的个数的数组，ja[i]为第i个非零元的列号。该数组的生命周期与handle相同。	输入
ia	int *	ia为stripeWidth+1维数组，ia[i]为第i行第一个非零元在所有非零元中的索引，ia[0]=0。该数组的生命周期与handle相同。	输入
comm	MPI_Comm	为MPI进程通信域。	输入

返回值

返回值	类型	描述
KMLSS_NO_ERROR	int	正常执行。
KMLSS_NULL_ARGUMENT	int	handle，stripeWidth，stripeRow，a，ja，ia中存在空参数。
KMLSS_BAD_N	int	参数n不是正整数。
KMLSS_NOT_IMPLEMENTED	int	参数nstripes不等于1。
KMLSS_NO_MEMORY	int	内存不足。

依赖

```
#include "kml_scaiss.h"
```

 版权所有 © 华为技术有限公司

示例

C Interface：

```
MPI_Init(NULL, NULL);
int size, rank;
MPI_Comm_size(MPI_COMM_WORLD, &size);//获取总进程数
MPI_Comm_rank(MPI_COMM_WORLD, &rank);//获取当前进程标识

int mat_size = 8;
int n = mat_size / size;
int n_beg = n * rank;
if (n * size != mat_size && rank == (size - 1)) {
    n = mat_size - n * rank;
}
int ja[26] = { 0, 3, 4, 1, 2, 3, 5, 1, 2, 7, 0, 1, 3, 6, 0, 4, 5, 1, 4, 5, 7, 3, 6, 2, 5, 7 };
double a[26] = { 1.0, 1.0, 2.0, 9.0, 2.0, 1.0, -3.0, 2.0, 3.0, 2.0, 1.0, 1.0, 9.0, -5.0, 2.0, 6.0, 1.0, -3.0, 1.0, 4.0, 1.0,
-5.0, 7.0, 2.0, 1.0, 2.0 };
int ia[9] = { 0, 3, 7, 10, 14, 17, 21, 23, 26 };
int a_beg = ia[n_beg];
for (int i = n_beg; i < (n_beg + n + 1); i++) {
    ia[i] -= a_beg;
}

/* Internal KML_SCAISS structure */
KmlScasolverTask *handle;

/* KML_SCAISS control parameters */
int error;            /* Output error handle */

/* Create data structures */
const double *a_holder = &a[a_beg];
const int *ja_holder = &ja[a_beg];
const int *ia_holder = &ia[n_beg];
error = KmlScaissGmresInitStripesDI(&handle, mat_size, 1, &n, &n_beg, &a_holder, &ja_holder, &ia_holder,
MPI_COMM_WORLD);
```

6.5.3.4.3.2 KmlScaissGmresInitWithoutMat?I

初始化数据结构，用于无矩阵求解的初始化，需要与用户自定义矩阵向量乘功能一并使用。

接口定义

C Interface：

int KmlScaissGmresInitWithoutMatSI(KmlScasolverTask **handle, int nstripes, const int *stripeWidth, MPI_Comm comm);

int KmlScaissGmresInitWithoutMatDI(KmlScasolverTask **handle, int nstripes, const int *stripeWidth, MPI_Comm comm);

参数

参数名	类型	描述	输入/输出
handle	KmlScasolverTask **	求解器句柄，handle不可为空指针。	输入/输出
nstripes	int	nstripes为存储在当前MPI进程中的矩阵数，目前只能取1。	输入

版权所有 © 华为技术有限公司

参数名	类型	描述	输入/输出
stripeWidth	int *	stripeWidth为当前MPI进程中的矩阵行数。该变量的生命周期与handle相同。	输入
comm	MPI_Comm	为MPI进程通信域。	输入

返回值

返回值	类型	描述
KMLSS_NO_ERROR	int	正常执行。
KMLSS_NULL_ARGUMENT	int	handle，stripeWidth中存在空参数。
KMLSS_NOT_IMPLEMENTED	int	参数nstripes不等于1。
KMLSS_NO_MEMORY	int	内存不足。

6.5.3.4.3.3 KmlScaissGmresSetUserPreconditioner?I

关联用户自定义预条件子回调函数，如不使用自定义的预条件子，该接口无需使用。

接口定义

C Interface：

int KmlScaissGmresSetUserPreconditionerSI(KmlScasolverTask **handle, void *ustruct, int (*fptr)(void *ustruct, float *x));

int KmlScaissGmresSetUserPreconditionerDI(KmlScasolverTask **handle, void *ustruct, int (*fptr)(void *ustruct, double *x));

参数

参数名	类型	描述	输入/输出
handle	KmlScasolverTask **	求解器句柄，传入之前步骤的变量。	输入/输出
ustruct	void *	指向用户数据的指针，每次调用预条件子时作为第一个参数传递。	输入

版权所有 © 华为技术有限公司

参数名	类型	描述	输入/输出
(*fptr)	int	用户自定义的函数指针，指向回调函数，该回调函数进行预条件子与其第二参数间的计算。	输入

返回值

返回值	类型	描述
KMLSS_NO_ERROR	int	正常执行。
KMLSS_NULL_ARGUMENT	int	handle，fptr中存在空参数。

依赖

#include "kml_scaiss.h"

示例

C Interface：

```
// USER data struct
typedef struct {
    int n;
    const int *ia;
    const int *ja;
    const double *a;
} duser;

int diagonal_preconditioner(void *usr, double *x)
{
    duser *u = (duser *)usr;
    int i, j;

    /* Apply diagonal preconditioner*/
    for (i = 0; i < u->n; i++) {
        for (j = u->ia[i]; j < u->ia[i + 1]; j++) {
            if (u->ja[j] == i)
                x[i] /= u->a[j];
        }
    }
    return 0;
}

MPI_Init(NULL, NULL);
int size, rank;
MPI_Comm_size(MPI_COMM_WORLD, &size);//获取总进程数
MPI_Comm_rank(MPI_COMM_WORLD, &rank);//获取当前进程标识

int mat_size = 8;
int n = mat_size / size;
int n_beg = n * rank;
if (n * size != mat_size && rank == (size - 1)) {
    n = mat_size - n * rank;
}
int ja[26] = { 0, 3, 4, 1, 2, 3, 5, 1, 2, 7, 0, 1, 3, 6, 0, 4, 5, 1, 4, 5, 7, 3, 6, 2, 5, 7 };
```

 版权所有 © 华为技术有限公司

```
double a[26] = { 1.0, 1.0, 2.0, 9.0, 2.0, 1.0, -3.0, 2.0, 3.0, 2.0, 1.0, 1.0, 9.0, -5.0, 2.0, 6.0, 1.0, -3.0, 1.0, 4.0, 1.0,
-5.0, 7.0, 2.0, 1.0, 2.0 };
int ia[9] = { 0, 3, 7, 10, 14, 17, 21, 23, 26 };
int a_beg = ia[n_beg];
for (int i = n_beg; i < (n_beg + n + 1); i++) {
    ia[i] -= a_beg;
}

/* Internal KML_SCAISS structure */
KmlScasolverTask *handle;

/* KML_SCAISS control parameters */
int error;            /* Output error handle */

/* Create data structures */
const double *a_holder = &a[a_beg];
const int *ja_holder = &ja[a_beg];
const int *ia_holder = &ia[n_beg];
error = KmlScaissGmresInitStripesDI(&handle, mat_size, 1, &n, &n_beg, &a_holder, &ja_holder, &ia_holder,
MPI_COMM_WORLD);

duser prea;
prea.n = n;
prea.ia = &ia[n_beg];
prea.ja = &ja[a_beg];
prea.a = &a[a_beg];

error = KmlScaissGmresSetUserPreconditionerDI(&handle, &prea, &diagonal_preconditioner);
if (error != 0) {
    printf("\nERROR in KmlScaissGmresSetUserPreconditionerDI: %d", error);
    return 1;
}
```

6.5.3.4.3.4 KmlScaissGmresSet?I?

设置迭代求解的相关参数。

接口定义

C Interface：

int KmlScaissGmresSetSII(KmlScasolverTask **handle, enum KmlSolverParam param, const int *data, int nd);

int KmlScaissGmresSetSIS(KmlScasolverTask **handle, enum KmlSolverParam param, const float *data, int nd);

int KmlScaissGmresSetDII(KmlScasolverTask **handle, enum KmlSolverParam param, const int *data, int nd);

int KmlScaissGmresSetDID(KmlScasolverTask **handle, enum KmlSolverParam param, const double *data, int nd);

参数

参数名	类型	描述	输入/输出
handle	KmlScasolverTask **	求解器句柄，传入之前步骤的变量。	输入/输出

 版权所有 © 华为技术有限公司

参数名	类型	描述	输入/输出
param	enum KmlSolverParam	● KMLSS_THRESHOLD表示迭代残差相对阈值。 ● KMLSS_ABS_TOLERANCE表示迭代残差绝对阈值。 ● KMLSS_MAX_ITERATION_COUNT表示最大迭代次数。 ● KMLSS_PRECONDITIONER_TYPE表示预条件子类型。 ● KMLSS_VECTOR_NORM_TYPE表示迭代残差的计算范数 ● KMLSS_MATRIX_FORMAT表示矩阵对角块稀疏存储格式。 ● KMLSS_OFFDIAG_MATRIX_FORMAT表示矩阵非对角块稀疏存储格式。 ● KMLSS_ORTHOGONALIZATION_TYPE表示正交化方法。 ● KMLSS_RESTART_PARAM表示重启参数。	输入

版权所有 © 华为技术有限公司

参数名	类型	描述	输入/输出
data	• 在KmlScaissGmresSetSII/KmlScaissGmresSetDII中为const int *。 • 在KmlScaissGmresSetSIS中为const float *。 • 在KmlScaissGmresSetDID中为const double *。	迭代求解相关参数数组（如阈值、最大迭代次数等）。 当前可选的正交化方法： • Gram-Schmidt正交化：KMLSS_GS • modified Gram-Schmidt正交化：KMLSS_MGS 当前可选的计算范数： • 1-NORM：KMLSS_L1 • 2-NORM：KMLSS_L2 当前可选的稀疏矩阵存储格式： • CSR格式：KMLSS_CSR • ELLPACK格式：KMLSS_ELL • COO格式：KMLSS_COO • ELLPACK_COO混合格式：KMLSS_SELLCOO 当前可选择的预条件子类型： • JACOBI预条件子：KMLSS_JACOBI • BLOCK JACOBI预条件子：KMLSS_BJACOBI • AMG预条件子：KMLSS_AMG	输入
nd	int	data数组元素个数。	输入

返回值

返回值	类型	描述
KMLSS_NO_ERROR	int	正常执行。
KMLSS_DATA_SIZE	int	参数nd不等于1。
KMLSS_NULL_ARGUMENT	int	handle，data中存在空参数。
KMLSS_BAD_SELECTOR	int	param为无效参数。
KMLSS_BAD_PRECONDITIONER	int	选择的预条件子暂未实现。

版权所有 © 华为技术有限公司

依赖

#include "kml_scaiss.h"

示例

C Interface：

```
MPI_Init(NULL, NULL);
int size, rank;
MPI_Comm_size(MPI_COMM_WORLD, &size);//获取总进程数
MPI_Comm_rank(MPI_COMM_WORLD, &rank);//获取当前进程标识

int mat_size = 8;
int n = mat_size / size;
int n_beg = n * rank;
if (n * size != mat_size && rank == (size - 1)) {
    n = mat_size - n * rank;
}
int ja[26] = { 0, 3, 4, 1, 2, 3, 5, 1, 2, 7, 0, 1, 3, 6, 0, 4, 5, 1, 4, 5, 7, 3, 6, 2, 5, 7 };
double a[26] = { 1.0, 1.0, 2.0, 9.0, 2.0, 1.0, -3.0, 2.0, 3.0, 2.0, 1.0, 1.0, 9.0, -5.0, 2.0, 6.0, 1.0, -3.0, 1.0, 4.0, 1.0,
-5.0, 7.0, 2.0, 1.0, 2.0 };
int ia[9] = { 0, 3, 7, 10, 14, 17, 21, 23, 26 };
int a_beg = ia[n_beg];
for (int i = n_beg; i < (n_beg + n + 1); i++) {
    ia[i] -= a_beg;
}

/* Internal KML_SCAISS structure */
KmlScasolverTask *handle;

/* KML_SCAISS control parameters */
int error;              /* Output error handle */

/* Create data structures */
const double *a_holder = &a[a_beg];
const int *ja_holder = &ja[a_beg];
const int *ia_holder = &ia[n_beg];
error = KmlScaissGmresInitStripesDI(&handle, mat_size, 1, &n, &n_beg, &a_holder, &ja_holder, &ia_holder,
MPI_COMM_WORLD);

double eps = 1e-4;
error = KmlScaissGmresSetDID(&handle, KMLSS_THRESHOLD, &eps, 1);
if (error != 0) {
    printf("\nERROR in KmlScaissGmresSetDID with KMLSS_THRESHOLD: %d", error);
    return 1;
}
int max_iters = 2000;
error = KmlScaissGmresSetDII(&handle, KMLSS_MAX_ITERATION_COUNT, &max_iters, 1);
if (error != 0) {
    printf("\nERROR in KmlScaissGmresSetDII with KMLSS_MAX_ITERATION_COUNT: %d", error);
    return 1;
}
int precond = KMLSS_BJACOBI;
error = KmlScaissGmresSetDII(&handle, KMLSS_PRECONDITIONER_TYPE, &precond, 1);
if (error != 0) {
    printf("\nERROR in KmlScaissGmresSetDII with KMLSS_PRECONDITIONER_TYPE: %d", error);
    return 1;
}
```

6.5.3.4.3.5 KmlScaissGmresSetUserSpmv?

关联用户自定义矩阵向量乘函数，需要与无矩阵初始化接口一起使用。

接口定义

C Interface:

 版权所有 © 华为技术有限公司

int KmlScaissGmresSetUserSpmvSI(KmlScasolverTask **handle, void *ustruct, int (*fptr)(void *ustruct, const float *x, float *y));

int KmlScaissGmresSetUserSpmvDI(KmlScasolverTask **handle, void *ustruct, int (*fptr)(void *ustruct, const double *x, double *y));

参数

参数名	类型	描述	输入/输出
handle	KmlScasolverTask **	求解器句柄，传入之前步骤的变量。	输入/输出
ustruct	void *	指向用户数据的指针，每次调用预条件子时作为第一个参数传递。	输入
(*fptr)	int	用户自定义的函数指针，指向矩阵向量乘函数，该矩阵向量乘函数进行矩阵与其第二参数间的计算。	输入

返回值

返回值	类型	描述
KMLSS_NO_ERROR	int	正常执行。
KMLSS_DATA_SIZE	int	参数nd不等于1。
KMLSS_NULL_ARGUMENT	int	handle，data中存在空参数。

6.5.3.4.3.6 KmlScaissGmresPcSet?II

设置迭代求解的预条件子相关参数，当前仅支持为block jacobi预条件子设置相关参数。

接口定义

C Interface：

int KmlScaissGmresPcSetSII(KmlScasolverTask **handle, enum KmlSolverParam param, const int *data, int nd);

int KmlScaissGmresPcSetSIS(KmlScasolverTask **handle, enum KmlSolverParam param, const float *data, int nd);

int KmlScaissGmresPcSetDII(KmlScasolverTask **handle, enum KmlSolverParam param, const int *data, int nd);

int KmlScaissGmresPcSetDID(KmlScasolverTask **handle, enum KmlSolverParam param, const double *data, int nd);

 版权所有 © 华为技术有限公司

参数

参数名	类型	描述	输入/输出
handle	KmlScasolverTask **	求解器句柄，传入之前步骤的变量。	输入/输出
param	enum KmlSolverParam	• KMLSS_BLOCK_METHOD表示每个分块中的预条件子类型。 • KMLSS_NUM_BLOCKS表示每个进程中的分块数量。 • KMLSS_BLOCK_SIZES表示每个进程中的分块大小。 以下为预条件AMG相关设置项： • KMLSS_AMG_CYCLE表示AMG中CYCLE类型。 • KMLSS_AMG_COARSENING表示粗化算子类型。 • KMLSS_AMG_INTERPOLATION表示插值算子类型。 • KMLSS_AMG_CONNECTION表示粗网格构造方法。	输入
data	const int *	预条件子相关参数（如预条件子类型、分块数量、分块大小）。 当前可选择的预条件子类型： • ILU预条件子：KMLSS_ILU。 • ICC预条件子：KMLSS_ICC。 • SOR预条件子：KMLSS_SOR。 • ILUT预条件子：KMLSS_ILUT。 • 预条件AMG配置项描述具体见**表1 预条件AMG中具体配置项描述**所示。	输入
nd	int	data数组元素个数。	输入

返回值

返回值	类型	描述
KMLSS_NO_ERROR	int	正常执行。
KMLSS_DATA_SIZE	int	参数nd不等于1。
KMLSS_NULL_ARGUMENT	int	handle，data中存在空参数。

版权所有 © 华为技术有限公司

返回值	类型	描述
KMLSS_BAD_SELECTOR	int	param为无效参数。
KMLSS_BAD_PRECONDITIONER	int	选择的预条件子暂未实现。

依赖

#include "kml_scaiss.h"

示例

C Interface：

```
MPI_Init(NULL, NULL);
int size, rank;
MPI_Comm_size(MPI_COMM_WORLD, &size);//获取总进程数
MPI_Comm_rank(MPI_COMM_WORLD, &rank);//获取当前进程标识

int mat_size = 8;
int n = mat_size / size;
int n_beg = n * rank;
if (n * size != mat_size && rank == (size - 1)) {
    n = mat_size - n * rank;
}
int ja[26] = { 0, 3, 4, 1, 2, 3, 5, 1, 2, 7, 0, 1, 3, 6, 0, 4, 5, 1, 4, 5, 7, 3, 6, 2, 5, 7 };
double a[26] = { 1.0, 1.0, 2.0, 9.0, 2.0, 1.0, -3.0, 2.0, 3.0, 2.0, 1.0, 1.0, 9.0, -5.0, 2.0, 6.0, 1.0, -3.0, 1.0, 4.0, 1.0,
-5.0, 7.0, 2.0, 1.0, 2.0 };
int ia[9] = { 0, 3, 7, 10, 14, 17, 21, 23, 26 };
int a_beg = ia[n_beg];
for (int i = n_beg; i < (n_beg + n + 1); i++) {
    ia[i] -= a_beg;
}

/* Internal KML_SCAISS structure */
KmlScasolverTask *handle;

/* KML_SCAISS control parameters */
int error;              /* Output error handle */

/* Create data structures */
const double *a_holder = &a[a_beg];
const int *ja_holder = &ja[a_beg];
const int *ia_holder = &ia[n_beg];
error = KmlScaissGmresInitStripesDI(&handle, mat_size, 1, &n, &n_beg, &a_holder, &ja_holder, &ia_holder,
MPI_COMM_WORLD);

double eps = 1e-4;
error = KmlScaissGmresSetDID(&handle, KMLSS_THRESHOLD, &eps, 1);
if (error != 0) {
    printf("\nERROR in KmlScaissGmresSetDID with KMLSS_THRESHOLD: %d", error);
    return 1;
}
int max_iters = 2000;
error = KmlScaissGmresSetDII(&handle, KMLSS_MAX_ITERATION_COUNT, &max_iters, 1);
if (error != 0) {
    printf("\nERROR in KmlScaissGmresSetDII with KMLSS_MAX_ITERATION_COUNT: %d", error);
    return 1;
}
int precond = KMLSS_BJACOBI;
error = KmlScaissGmresSetDII(&handle, KMLSS_PRECONDITIONER_TYPE, &precond, 1);
if (error != 0) {
```

 版权所有 © 华为技术有限公司

```
    printf("\nERROR in KmlScaissGmresSetDII with KMLSS_PRECONDITIONER_TYPE: %d", error);
    return 1;
}
int num_blocks = 1;
error = KmlScaissGmresPcSetDII(&handle, KMLSS_NUM_BLOCKS, &num_blocks, 1);
if (error != 0) {
    printf("ERROR in KmlScaissGmresPcSetDII: %d\n", error);
    return 1;
}
int subprecond = KMLSS_ICC;
error = KmlScaissGmresPcSetDII(&handle, KMLSS_BLOCK_METHOD, &subprecond, 1);
if (error != 0) {
    printf("ERROR in KmlScaissGmresPcSetDII: %d\n", error);
    return 1;
}
```

6.5.3.4.3.7 KmlScaissGmresSubPcSet?I?

设置迭代求解的block jacobi中子预条件子相关参数。

接口定义

C Interface：

int KmlScaissGmresSubPcSetSII(KmlScasolverTask **handle, enum KmlSolverPreconditionerParam param, int subn, const int *data, int nd);

int KmlScaissGmresSubPcSetDII(KmlScasolverTask **handle, enum KmlSolverPreconditionerParam param, int subn, const int *data, int nd);

int KmlScaissGmresSubPcSetSIS(KmlScasolverTask **handle, enum KmlSolverPreconditionerParam param, int subn, const float *data, int nd);

int KmlScaissGmresSubPcSetDID(KmlScasolverTask **handle, enum KmlSolverPreconditionerParam param, int subn, const double *data, int nd);

参数

参数名	类型	描述	输入/输出
handle	KmlScasolverTask **	求解器句柄，传入之前步骤的变量。	输入/输出
param	enum KmlSolverParam	• KML_RELAXATION_FACTOR表示SOR预条件子相关参数。 • KMLSS_DROP_FILL表示ILUT预条件子每行最多保留元素个数。 • KMLSS_DROP_TOL表示ILUT预条件子保留元素的阈值。	输入

版权所有 © 华为技术有限公司

参数名	类型	描述	输入/输出
subn	• 在KmlScaissGmresSubPcSetSIS、KmlScaissGmresSubPcSetSII、KmlScaissGmresSubPcSetDII、KmlScaissGmresSubPcSetDID中为int。	子预条件子索引。	输入
data	• 在KmlScaissGmresSubPcSetSII、KmlScaissGmresSubPcSetDII中为const int *。 • 在KmlScaissGmresSubPcSetSIS中为const float *。 • 在KmlScaissGmresSubPcSetDID中为const double *。	预条件子相关参数。	输入
nd	• 在KmlScaissGmresSubPcSetSII、KmlScaissGmresSubPcSetSIS、KmlScaissGmresSubPcSetDII、KmlScaissGmresSubPcSetDID中为int。	data数组元素个数。	输入

返回值

返回值	类型	描述
KMLSS_NO_ERROR	int	正常执行。
KMLSS_DATA_SIZE	int	参数nd不等于1。
KMLSS_NULL_ARGUMENT	int	handle，data中存在空参数。

版权所有 © 华为技术有限公司

返回值	类型	描述
KMLSS_BAD_SELECTOR	int	param为无效参数。
KMLSS_BAD_PRECONDITIONER	int	选择的预条件子暂未实现。

依赖

#include "kml_scaiss.h"

示例

C Interface：

```
MPI_Init(NULL, NULL);
int size, rank;
MPI_Comm_size(MPI_COMM_WORLD, &size);//获取总进程数
MPI_Comm_rank(MPI_COMM_WORLD, &rank);//获取当前进程标识

int mat_size = 8;
int n = mat_size / size;
int n_beg = n * rank;
if (n * size != mat_size && rank == (size - 1)) {
    n = mat_size - n * rank;
}
int ja[26] = { 0, 3, 4, 1, 2, 3, 5, 1, 2, 7, 0, 1, 3, 6, 0, 4, 5, 1, 4, 5, 7, 3, 6, 2, 5, 7 };
double a[26] = { 1.0, 1.0, 2.0, 9.0, 2.0, 1.0, -3.0, 2.0, 3.0, 2.0, 1.0, 1.0, 9.0, -5.0, 2.0, 6.0, 1.0, -3.0, 1.0, 4.0, 1.0,
-5.0, 7.0, 2.0, 1.0, 2.0 };
int ia[9] = { 0, 3, 7, 10, 14, 17, 21, 23, 26 };
int a_beg = ia[n_beg];
for (int i = n_beg; i < (n_beg + n + 1); i++) {
    ia[i] -= a_beg;
}

/* Internal KML_SCAISS structure */
KmlScasolverTask *handle;

/* KML_SCAISS control parameters */
int error;            /* Output error handle */

/* Create data structures */
const double *a_holder = &a[a_beg];
const int *ja_holder = &ja[a_beg];
const int *ia_holder = &ia[n_beg];
error = KmlScaissGmresInitStripesDI(&handle, mat_size, 1, &n, &n_beg, &a_holder, &ja_holder, &ia_holder,
MPI_COMM_WORLD);

double eps = 1e-4;
error = KmlScaissGmresSetDID(&handle, KMLSS_THRESHOLD, &eps, 1);
if (error != 0) {
    printf("\nERROR in KmlScaissGmresSetDID with KMLSS_THRESHOLD: %d", error);
    return 1;
}
int max_iters = 2000;
error = KmlScaissGmresSetDII(&handle, KMLSS_MAX_ITERATION_COUNT, &max_iters, 1);
if (error != 0) {
    printf("\nERROR in KmlScaissGmresSetDII with KMLSS_MAX_ITERATION_COUNT: %d", error);
    return 1;
}
int precond = KMLSS_BJACOBI;
error = KmlScaissGmresSetDII(&handle, KMLSS_PRECONDITIONER_TYPE, &precond, 1);
if (error != 0) {
```

 版权所有 © 华为技术有限公司

```
    printf("\nERROR in KmlScaissGmresSetDII with KMLSS_PRECONDITIONER_TYPE: %d", error);
    return 1;
}
int num_blocks = 1;
error = KmlScaissGmresPcSetDII(&handle, KMLSS_NUM_BLOCKS, &num_blocks, 1);
if (error != 0) {
    printf("ERROR in KmlScaissGmresPcSetDII: %d\n", error);
    return 1;
}
int subprecond = KMLSS_ICC;
error = KmlScaissGmresPcSetDII(&handle, KMLSS_BLOCK_METHOD, &subprecond, 1);
if (error != 0) {
    printf("ERROR in KmlScaissGmresPcSetDII: %d\n", error);
    return 1;
}
```

6.5.3.4.3.8 KmlScaissGmresAnalyze?I

分析稀疏矩阵并确定不同进程之间的通信关系。

接口定义

C Interface：

int KmlScaissGmresAnalyzeSI(KmlScasolverTask **handle);

int KmlScaissGmresAnalyzeDI(KmlScasolverTask **handle);

参数

参数名	类型	描述	输入/输出
handle	KmlScasolverTask **	求解器句柄，传入之前步骤的变量。	输入/输出

返回值

返回值	类型	描述
KMLSS_NO_ERROR	int	正常执行。
KMLSS_NULL_ARGUMENT	int	handle为空参数。

依赖

#include "kml_scaiss.h"

示例

C Interface：

```
MPI_Init(NULL, NULL);
int size, rank;
```

 版权所有 © 华为技术有限公司

```
MPI_Comm_size(MPI_COMM_WORLD, &size);//获取总进程数
MPI_Comm_rank(MPI_COMM_WORLD, &rank);//获取当前进程标识

int mat_size = 8;
int n = mat_size / size;
int n_beg = n * rank;
if (n * size != mat_size && rank == (size - 1)) {
    n = mat_size - n * rank;
}
int ja[26] = { 0, 3, 4, 1, 2, 3, 5, 1, 2, 7, 0, 1, 3, 6, 0, 4, 5, 1, 4, 5, 7, 3, 6, 2, 5, 7 };
double a[26] = { 1.0, 1.0, 2.0, 9.0, 2.0, 1.0, -3.0, 2.0, 3.0, 2.0, 1.0, 1.0, 9.0, -5.0, 2.0, 6.0, 1.0, -3.0, 1.0, 4.0, 1.0,
-5.0, 7.0, 2.0, 1.0, 2.0 };
int ia[9] = { 0, 3, 7, 10, 14, 17, 21, 23, 26 };
int a_beg = ia[n_beg];
for (int i = n_beg; i < (n_beg + n + 1); i++) {
    ia[i] -= a_beg;
}

/* Internal KML_SCAISS structure */
KmlScasolverTask *handle;

/* KML_SCAISS control parameters */
int error;              /* Output error handle */

/* Create data structures */
const double *a_holder = &a[a_beg];
const int *ja_holder = &ja[a_beg];
const int *ia_holder = &ia[n_beg];
error = KmlScaissGmresInitStripesDI(&handle, mat_size, 1, &n, &n_beg, &a_holder, &ja_holder, &ia_holder,
MPI_COMM_WORLD);

double eps = 1e-4;
error = KmlScaissGmresSetDID(&handle, KMLSS_THRESHOLD, &eps, 1);
if (error != 0) {
    printf("\nERROR in KmlScaissGmresSetDID with KMLSS_THRESHOLD: %d", error);
    return 1;
}
int max_iters = 2000;
error = KmlScaissGmresSetDII(&handle, KMLSS_MAX_ITERATION_COUNT, &max_iters, 1);
if (error != 0) {
    printf("\nERROR in KmlScaissGmresSetDII with KMLSS_MAX_ITERATION_COUNT: %d", error);
    return 1;
}
int precond = KMLSS_BJACOBI;
error = KmlScaissGmresSetDII(&handle, KMLSS_PRECONDITIONER_TYPE, &precond, 1);
if (error != 0) {
    printf("\nERROR in KmlScaissGmresSetDII with KMLSS_PRECONDITIONER_TYPE: %d", error);
    return 1;
}
int num_blocks = 1;
error = KmlScaissGmresPcSetDII(&handle, KMLSS_NUM_BLOCKS, &num_blocks, 1);
if (error != 0) {
    printf("ERROR in KmlScaissGmresPcSetDII: %d\n", error);
    return 1;
}
int subprecond = KMLSS_ICC;
error = KmlScaissGmresPcSetDII(&handle, KMLSS_BLOCK_METHOD, &subprecond, 1);
if (error != 0) {
    printf("ERROR in KmlScaissGmresPcSetDII: %d\n", error);
    return 1;
}
error = KmlScaissGmresAnalyzeDI(&handle);
if (error != 0) {
    printf("ERROR in KmlScaissGmresAnalyzeDI: %d\n", error);
    return 1;
}
```

 版权所有 © 华为技术有限公司

6.5.3.4.3.9 KmlScaissGmresFactorize?I

在求解器内部对矩阵进行赋值，并计算预条件子。无论是否使用预条件子，都需要调用该接口。

接口定义

C Interface：

int KmlScaissGmresFactorizeSI(KmlScasolverTask **handle);

int KmlScaissGmresFactorizeDI(KmlScasolverTask **handle);

参数

参数名	类型	描述	输入/输出
handle	KmlScasolverTask **	求解器句柄，传入之前步骤的变量。	输入/输出

返回值

返回值	类型	描述
KMLSS_NO_ERROR	int	正常执行。
KMLSS_NULL_ARGUMENT	int	handle为空参数。
KMLSS_ZERO_DIAGONAL_ELEMENT	int	矩阵对角线存在零元。

依赖

#include "kml_scaiss.h"

示例

C Interface：

```
MPI_Init(NULL, NULL);
int size, rank;
MPI_Comm_size(MPI_COMM_WORLD, &size);//获取总进程数
MPI_Comm_rank(MPI_COMM_WORLD, &rank);//获取当前进程标识

int mat_size = 8;
int n = mat_size / size;
int n_beg = n * rank;
if (n * size != mat_size && rank == (size - 1)) {
    n = mat_size - n * rank;
}
int ja[26] = { 0, 3, 4, 1, 2, 3, 5, 1, 2, 7, 0, 1, 3, 6, 0, 4, 5, 1, 4, 5, 7, 3, 6, 2, 5, 7 };
double a[26] = { 1.0, 1.0, 2.0, 9.0, 2.0, 1.0, -3.0, 2.0, 3.0, 2.0, 1.0, 1.0, 9.0, -5.0, 2.0, 6.0, 1.0, -3.0, 1.0, 4.0, 1.0,
-5.0, 7.0, 2.0, 1.0, 2.0 };
```

 版权所有 © 华为技术有限公司

```
int ia[9] = { 0, 3, 7, 10, 14, 17, 21, 23, 26 };
int a_beg = ia[n_beg];
for (int i = n_beg; i < (n_beg + n + 1); i++) {
    ia[i] -= a_beg;
}

/* Internal KML_SCAISS structure */
KmlScasolverTask *handle;

/* KML_SCAISS control parameters */
int error;              /* Output error handle */

/* Create data structures */
const double *a_holder = &a[a_beg];
const int *ja_holder = &ja[a_beg];
const int *ia_holder = &ia[n_beg];
error = KmlScaissGmresInitStripesDI(&handle, mat_size, 1, &n, &n_beg, &a_holder, &ja_holder, &ia_holder,
MPI_COMM_WORLD);

double eps = 1e-4;
error = KmlScaissGmresSetDID(&handle, KMLSS_THRESHOLD, &eps, 1);
if (error != 0) {
    printf("\nERROR in KmlScaissGmresSetDID with KMLSS_THRESHOLD: %d", error);
    return 1;
}
int max_iters = 2000;
error = KmlScaissGmresSetDII(&handle, KMLSS_MAX_ITERATION_COUNT, &max_iters, 1);
if (error != 0) {
    printf("\nERROR in KmlScaissGmresSetDII with KMLSS_MAX_ITERATION_COUNT: %d", error);
    return 1;
}
int precond = KMLSS_BJACOBI;
error = KmlScaissGmresSetDII(&handle, KMLSS_PRECONDITIONER_TYPE, &precond, 1);
if (error != 0) {
    printf("\nERROR in KmlScaissGmresSetDII with KMLSS_PRECONDITIONER_TYPE: %d", error);
    return 1;
}
int num_blocks = 1;
error = KmlScaissGmresPcSetDII(&handle, KMLSS_NUM_BLOCKS, &num_blocks, 1);
if (error != 0) {
    printf("ERROR in KmlScaissGmresPcSetDII: %d\n", error);
    return 1;
}
int subprecond = KMLSS_ICC;
error = KmlScaissGmresPcSetDII(&handle, KMLSS_BLOCK_METHOD, &subprecond, 1);
if (error != 0) {
    printf("ERROR in KmlScaissGmresPcSetDII: %d\n", error);
    return 1;
}
error = KmlScaissGmresAnalyzeDI(&handle);
if (error != 0) {
    printf("ERROR in KmlScaissGmresAnalyzeDI: %d\n", error);
    return 1;
}
error = KmlScaissGmresFactorizeDI(&handle);
if (error != 0) {
    printf("ERROR in KmlScaissGmresFactorizeDI: %d\n", error);
    return 1;
}
```

6.5.3.4.3.10 KmlScaissGmresSolve?I

求解线性代数方程组，每个进程读取并存储全局求解结果。

接口定义

C Interface：

int KmlScaissGmresSolveSI(KmlScasolverTask **handle, int nb, float *x, int ldx, const float *b, int ldb);

int KmlScaissGmresSolveDI(KmlScasolverTask **handle, int nb, double *x, int ldx, const double *b, int ldb);

参数

参数名	类型	描述	输入/输出
handle	KmlScasolverTask **	求解器句柄，传入之前步骤的变量。	输入/输出
nb	int	右端项矢量的个数，当前nb只能为1。	输入
x	• 在KmlScaissGmresSolveSI中为float *。 • 在KmlScaissGmresSolveDI中为double *。	读取并存储全局求解结果，其元素个数为全局矩阵大小n。	输入/输出
ldx	int	x的leading dimension大小，当前参数未使用。	输入
b	• 在KmlScaissGmresSolveSI中为const float *。 • 在KmlScaissGmresSolveDI中为const double *。	读取进程对应的局部右端项，其元素个数为当前进程对应的矩阵的行数，即stripeWidth。	输入
ldb	int	b的leading dimension大小，当前参数未使用。	输入

返回值

返回值	类型	描述
KMLSS_NO_ERROR	int	正常执行。
KMLSS_BAD_NB	int	参数nb不等于1。
KMLSS_NO_MEMORY	int	内存不足。
KMLSS_NULL_ARGUMENT	int	handle，x，b中存在空参数。

 版权所有 © 华为技术有限公司

依赖

#include "kml_scaiss.h"

示例

C Interface：

```
MPI_Init(NULL, NULL);
int size, rank;
MPI_Comm_size(MPI_COMM_WORLD, &size);//获取总进程数
MPI_Comm_rank(MPI_COMM_WORLD, &rank);//获取当前进程标识

int mat_size = 8;
int n = mat_size / size;
int n_beg = n * rank;
if (n * size != mat_size && rank == (size - 1)) {
    n = mat_size - n * rank;
}
int ja[26] = { 0, 3, 4, 1, 2, 3, 5, 1, 2, 7, 0, 1, 3, 6, 0, 4, 5, 1, 4, 5, 7, 3, 6, 2, 5, 7 };
double a[26] = { 1.0, 1.0, 2.0, 9.0, 2.0, 1.0, -3.0, 2.0, 3.0, 2.0, 1.0, 1.0, 9.0, -5.0, 2.0, 6.0, 1.0, -3.0, 1.0, 4.0, 1.0,
-5.0, 7.0, 2.0, 1.0, 2.0 };
int ia[9] = { 0, 3, 7, 10, 14, 17, 21, 23, 26 };
int a_beg = ia[n_beg];
for (int i = n_beg; i < (n_beg + n + 1); i++) {
    ia[i] -= a_beg;
}

/* Internal KML_SCAISS structure */
KmlScasolverTask *handle;

/* KML_SCAISS control parameters */
int error;              /* Output error handle */

/* Create data structures */
const double *a_holder = &a[a_beg];
const int *ja_holder = &ja[a_beg];
const int *ia_holder = &ia[n_beg];
error = KmlScaissGmresInitStripesDI(&handle, mat_size, 1, &n, &n_beg, &a_holder, &ja_holder, &ia_holder,
MPI_COMM_WORLD);

double eps = 1e-4;
error = KmlScaissGmresSetDID(&handle, KMLSS_THRESHOLD, &eps, 1);
if (error != 0) {
    printf("\nERROR in KmlScaissGmresSetDID with KMLSS_THRESHOLD: %d", error);
    return 1;
}
int max_iters = 2000;
error = KmlScaissGmresSetDII(&handle, KMLSS_MAX_ITERATION_COUNT, &max_iters, 1);
if (error != 0) {
    printf("\nERROR in KmlScaissGmresSetDII with KMLSS_MAX_ITERATION_COUNT: %d", error);
    return 1;
}
int precond = KMLSS_BJACOBI;
error = KmlScaissGmresSetDII(&handle, KMLSS_PRECONDITIONER_TYPE, &precond, 1);
if (error != 0) {
    printf("\nERROR in KmlScaissGmresSetDII with KMLSS_PRECONDITIONER_TYPE: %d", error);
    return 1;
}
int num_blocks = 1;
error = KmlScaissGmresPcSetDII(&handle, KMLSS_NUM_BLOCKS, &num_blocks, 1);
if (error != 0) {
    printf("ERROR in KmlScaissGmresPcSetDII: %d\n", error);
    return 1;
}
int subprecond = KMLSS_ICC;
error = KmlScaissGmresPcSetDII(&handle, KMLSS_BLOCK_METHOD, &subprecond, 1);
if (error != 0) {
    printf("ERROR in KmlScaissGmresPcSetDII: %d\n", error);
```

 版权所有 © 华为技术有限公司

```
    return 1;
}
error = KmlScaissGmresAnalyzeDI(&handle);
if (error != 0) {
    printf("ERROR in KmlScaissGmresAnalyzeDI: %d\n", error);
    return 1;
}
error = KmlScaissGmresFactorizeDI(&handle);
if (error != 0) {
    printf("ERROR in KmlScaissGmresFactorizeDI: %d\n", error);
    return 1;
}

int nrhs = 1;
int ldx = mat_size, ldb = n;
double b[8] = { 4.0, 9.0, 7.0, 6.0, 9.0, 3.0, 2.0, 5.0 };
double x[8] = { 0.0, 0.0, 0.0, 0.0, 0.0, 0.0, 0.0, 0.0 };
error = KmlScaissGmresSolveDxDI(&handle, nrhs, &x[0], ldx, &b[n_beg], ldb);
if (error != 0) {
    printf("ERROR in KmlScaissGmresSolveDI: %d\n", error);
    return 1;
}
```

6.5.3.4.3.11 KmlScaissGmresSolveDx?I

求解线性代数方程组，每个进程读取并存储当前进程对应的求解结果。

接口定义

C Interface：

int KmlScaissGmresSolveDxSI(KmlScasolverTask **handle, int nb, float *x, int ldx, const float *b, int ldb);

int KmlScaissGmresSolveDxDI(KmlScasolverTask **handle, int nb, double *x, int ldx, const double *b, int ldb);

参数

参数名	类型	描述	输入/输出
handle	KmlScasolverTask **	求解器句柄，传入之前步骤的变量。	输入/输出
nb	int	右端项矢量的个数，当前nb只能为1。	输入
x	• 在KmlScaissGmresSolveDxSI中为float *。 • 在KmlScaissGmresSolveDxDI中为double *。	读取并存储当前进程对应的求解结果，其元素个数为当前进程对应的矩阵的行数，即stripeWidth。	输入/输出
ldx	int	x的leading dimension大小，当前参数未使用。	输入

版权所有 © 华为技术有限公司

参数名	类型	描述	输入/输出
b	• 在KmlScaissGmresSolveDxSI中为const float * • 在KmlScaissGmresSolveDxDI中为const double *。	读取进程对应的局部右端项，其元素个数为当前进程对应的矩阵的行数，即stripeWidth。	输入
ldb	int	b的leading dimension大小，当前参数未使用。	输入

返回值

返回值	类型	描述
KMLSS_NO_ERROR	int	正常执行。
KMLSS_BAD_NB	int	参数nb不等于1。
KMLSS_NO_MEMORY	int	内存不足。
KMLSS_NULL_ARGUMENT	int	handle，x，b中存在空参数。

依赖

#include "kml_scaiss.h"

示例

C Interface：

```
MPI_Init(NULL, NULL);
int size, rank;
MPI_Comm_size(MPI_COMM_WORLD, &size);//获取总进程数
MPI_Comm_rank(MPI_COMM_WORLD, &rank);//获取当前进程标识

int mat_size = 8;
int n = mat_size / size;
int n_beg = n * rank;
if (n * size != mat_size && rank == (size - 1)) {
    n = mat_size - n * rank;
}
int ja[26] = { 0, 3, 4, 1, 2, 3, 5, 1, 2, 7, 0, 1, 3, 6, 0, 4, 5, 1, 4, 5, 7, 3, 6, 2, 5, 7 };
double a[26] = { 1.0, 1.0, 2.0, 9.0, 2.0, 1.0, -3.0, 2.0, 3.0, 2.0, 1.0, 1.0, 9.0, -5.0, 2.0, 6.0, 1.0, -3.0, 1.0, 4.0, 1.0,
-5.0, 7.0, 2.0, 1.0, 2.0 };
int ia[9] = { 0, 3, 7, 10, 14, 17, 21, 23, 26 };
int a_beg = ia[n_beg];
for (int i = n_beg; i < (n_beg + n + 1); i++) {
    ia[i] -= a_beg;
}
```


版权所有 © 华为技术有限公司

```
/* Internal KML_SCAISS structure */
KmlScasolverTask *handle;

/* KML_SCAISS control parameters */
int error;            /* Output error handle */

/* Create data structures */
const double *a_holder = &a[a_beg];
const int *ja_holder = &ja[a_beg];
const int *ia_holder = &ia[n_beg];
error = KmlScaissGmresInitStripesDI(&handle, mat_size, 1, &n, &n_beg, &a_holder, &ja_holder, &ia_holder,
MPI_COMM_WORLD);

double eps = 1e-4;
error = KmlScaissGmresSetDID(&handle, KMLSS_THRESHOLD, &eps, 1);
if (error != 0) {
    printf("\nERROR in KmlScaissGmresSetDID with KMLSS_THRESHOLD: %d", error);
    return 1;
}
int max_iters = 2000;
error = KmlScaissGmresSetDII(&handle, KMLSS_MAX_ITERATION_COUNT, &max_iters, 1);
if (error != 0) {
    printf("\nERROR in KmlScaissGmresSetDII with KMLSS_MAX_ITERATION_COUNT: %d", error);
    return 1;
}
int precond = KMLSS_BJACOBI;
error = KmlScaissGmresSetDII(&handle, KMLSS_PRECONDITIONER_TYPE, &precond, 1);
if (error != 0) {
    printf("\nERROR in KmlScaissGmresSetDII with KMLSS_PRECONDITIONER_TYPE: %d", error);
    return 1;
}
int num_blocks = 1;
error = KmlScaissGmresPcSetDII(&handle, KMLSS_NUM_BLOCKS, &num_blocks, 1);
if (error != 0) {
    printf("ERROR in KmlScaissGmresPcSetDII: %d\n", error);
    return 1;
}
int subprecond = KMLSS_ICC;
error = KmlScaissGmresPcSetDII(&handle, KMLSS_BLOCK_METHOD, &subprecond, 1);
if (error != 0) {
    printf("ERROR in KmlScaissGmresPcSetDII: %d\n", error);
    return 1;
}
error = KmlScaissGmresAnalyzeDI(&handle);
if (error != 0) {
    printf("ERROR in KmlScaissGmresAnalyzeDI: %d\n", error);
    return 1;
}
error = KmlScaissGmresFactorizeDI(&handle);
if (error != 0) {
    printf("ERROR in KmlScaissGmresFactorizeDI: %d\n", error);
    return 1;
}

int nrhs = 1;
int ldx = n, ldb = n;
double b[8] = { 4.0, 9.0, 7.0, 6.0, 9.0, 3.0, 2.0, 5.0 };
double x[8] = { 0.0, 0.0, 0.0, 0.0, 0.0, 0.0, 0.0, 0.0 };
error = KmlScaissGmresSolveDxDI(&handle, nrhs, &x[n_beg], ldx, &b[n_beg], ldb);
if (error != 0) {
    printf("ERROR in KmlScaissGmresSolveDI: %d\n", error);
    return 1;
}
```

6.5.3.4.3.12 KmlScaissGmresGet?I?

获得迭代求解相关参数。

 版权所有 © 华为技术有限公司

接口定义

C Interface：

int KmlScaissGmresGetSII(KmlScasolverTask **handle, enum KmlSolverParam param, int *data, int nd);

int KmlScaissGmresGetSIS(KmlScasolverTask **handle, enum KmlSolverParam param, float *data, int nd);

int KmlScaissGmresGetDII(KmlScasolverTask **handle, enum KmlSolverParam param, int *data, int nd);

int KmlScaissGmresGetDID(KmlScasolverTask **handle, enum KmlSolverParam param, double *data, int nd);

参数

参数名	类型	描述	输入/输出
handle	KmlScasolverTask **	求解器句柄，传入之前步骤的变量。	输入/输出
param	enum KmlSolverParam	• KMLSS_ITERATION_COUNT表示迭代次数。 • KMLSS_MAX_ITERATION_COUNT表示设置的最大迭代次数。 • KMLSS_TOLERANCE表示最终的迭代相对残差。 • KMLSS_THRESHOLD表示设置的相对残差。	输入
data	• 在KmlScaissGmresGetSII/KmlScaissGmresGetDII中为int *。 • 在KmlScaissGmresGetSIS中为float *。 • 在KmlScaissGmresGetDID中为double *。	• 在KmlScaissGmresGetSII/KmlScaissGmresGetDII中为迭代步数。 • 在KmlScaissGmresGetSIS中为残差。 • 在KmlScaissGmresGetDID中为残差。	输出
nd	int	data数组元素个数。	输入

版权所有 © 华为技术有限公司

返回值

返回值	类型	描述
KMLSS_NO_ERROR	int	正常执行。
KMLSS_DATA_SIZE	int	参数nd不等于1。
KMLSS_NULL_ARGUMENT	int	handle，data中存在空参数。
KMLSS_BAD_SELECTOR	int	param为无效参数。

依赖

#include "kml_scaiss.h"

示例

C Interface：

```
MPI_Init(NULL, NULL);
int size, rank;
MPI_Comm_size(MPI_COMM_WORLD, &size);//获取总进程数
MPI_Comm_rank(MPI_COMM_WORLD, &rank);//获取当前进程标识

int mat_size = 8;
int n = mat_size / size;
int n_beg = n * rank;
if (n * size != mat_size && rank == (size - 1)) {
    n = mat_size - n * rank;
}
int ja[26] = { 0, 3, 4, 1, 2, 3, 5, 1, 2, 7, 0, 1, 3, 6, 0, 4, 5, 1, 4, 5, 7, 3, 6, 2, 5, 7 };
double a[26] = { 1.0, 1.0, 2.0, 9.0, 2.0, 1.0, -3.0, 2.0, 3.0, 2.0, 1.0, 1.0, 9.0, -5.0, 2.0, 6.0, 1.0, -3.0, 1.0, 4.0, 1.0,
-5.0, 7.0, 2.0, 1.0, 2.0 };
int ia[9] = { 0, 3, 7, 10, 14, 17, 21, 23, 26 };
int a_beg = ia[n_beg];
for (int i = n_beg; i < (n_beg + n + 1); i++) {
    ia[i] -= a_beg;
}

/* Internal KML_SCAISS structure */
KmlScasolverTask *handle;

/* KML_SCAISS control parameters */
int error;              /* Output error handle */

/* Create data structures */
const double *a_holder = &a[a_beg];
const int *ja_holder = &ja[a_beg];
const int *ia_holder = &ia[n_beg];
error = KmlScaissGmresInitStripesDI(&handle, mat_size, 1, &n, &n_beg, &a_holder, &ja_holder, &ia_holder,
MPI_COMM_WORLD);

double eps = 1e-4;
error = KmlScaissGmresSetDID(&handle, KMLSS_THRESHOLD, &eps, 1);
if (error != 0) {
    printf("\nERROR in KmlScaissGmresSetDID with KMLSS_THRESHOLD: %d", error);
    return 1;
}
int max_iters = 2000;
error = KmlScaissGmresSetDII(&handle, KMLSS_MAX_ITERATION_COUNT, &max_iters, 1);
if (error != 0) {
```

 版权所有 © 华为技术有限公司

```
    printf("\nERROR in KmlScaissGmresSetDII with KMLSS_MAX_ITERATION_COUNT: %d", error);
    return 1;
}
int precond = KMLSS_BJACOBI;
error = KmlScaissGmresSetDII(&handle, KMLSS_PRECONDITIONER_TYPE, &precond, 1);
if (error != 0) {
    printf("\nERROR in KmlScaissGmresSetDII with KMLSS_PRECONDITIONER_TYPE: %d", error);
    return 1;
}
int num_blocks = 1;
error = KmlScaissGmresPcSetDII(&handle, KMLSS_NUM_BLOCKS, &num_blocks, 1);
if (error != 0) {
    printf("ERROR in KmlScaissGmresPcSetDII: %d\n", error);
    return 1;
}
int subprecond = KMLSS_ICC;
error = KmlScaissGmresPcSetDII(&handle, KMLSS_BLOCK_METHOD, &subprecond, 1);
if (error != 0) {
    printf("ERROR in KmlScaissGmresPcSetDII: %d\n", error);
    return 1;
}
error = KmlScaissGmresAnalyzeDI(&handle);
if (error != 0) {
    printf("ERROR in KmlScaissGmresAnalyzeDI: %d\n", error);
    return 1;
}
error = KmlScaissGmresFactorizeDI(&handle);
if (error != 0) {
    printf("ERROR in KmlScaissGmresFactorizeDI: %d\n", error);
    return 1;
}

int nrhs = 1;
int ldx = n, ldb = n;
double b[8] = { 4.0, 9.0, 7.0, 6.0, 9.0, 3.0, 2.0, 5.0 };
double x[8] = { 0.0, 0.0, 0.0, 0.0, 0.0, 0.0, 0.0, 0.0 };
error = KmlScaissGmresSolveDxDI(&handle, nrhs, &x[n_beg], ldx, &b[n_beg], ldb);
if (error != 0) {
    printf("ERROR in KmlScaissGmresSolveDI: %d\n", error);
    return 1;
}

int idata[] = {0};
error = KmlScaissGmresGetDII(&handle, KMLSS_ITERATION_COUNT, idata, 1);
if (error != 0) {
    printf("\nERROR in KmlScaissGmresGetDII: %d", error);
    return 1;
    }
double ddata[] = {0.0};
error = KmlScaissGmresGetDID(&handle, KMLSS_TOLERANCE, ddata, 1);
if (error != 0) {
    printf("\nERROR in KmlScaissGmresGetDID: %d", error);
    return 1;
    }
if (rank == 0) printf("[Solve info] Res= \t%.12g\t, iteration = \t%d\t\n", sqrt(ddata[0]), idata[0]);
```

运行结果（2个进程）：

```
[Solve info] Res=      2.04475505752e-15       , iteration =   8
```

6.5.3.4.3.13 KmlScaissGmresClean?I

释放内部数据结构。

接口定义

C Interface：

int KmlScaissGmresCleanSI(KmlScaSolverTask **handle);

int KmlScaissGmresCleanDI(KmlScaSolverTask **handle);

参数

参数名	类型	描述	输入/输出
handle	KmlScasolverTask **	求解器句柄，传入之前步骤的变量。	输入/输出

返回值

返回值	类型	描述
KMLSS_NO_ERROR	int	正常执行。
KMLSS_NULL_ARGUMENT	int	handle为空参数。

依赖

#include "kml_scaiss.h"

示例

C Interface：

```
MPI_Init(NULL, NULL);
int size, rank;
MPI_Comm_size(MPI_COMM_WORLD, &size);//获取总进程数
MPI_Comm_rank(MPI_COMM_WORLD, &rank);//获取当前进程标识

int mat_size = 8;
int n = mat_size / size;
int n_beg = n * rank;
if (n * size != mat_size && rank == (size - 1)) {
    n = mat_size - n * rank;
}
int ja[26] = { 0, 3, 4, 1, 2, 3, 5, 1, 2, 7, 0, 1, 3, 6, 0, 4, 5, 1, 4, 5, 7, 3, 6, 2, 5, 7 };
double a[26] = { 1.0, 1.0, 2.0, 9.0, 2.0, 1.0, -3.0, 2.0, 3.0, 2.0, 1.0, 1.0, 9.0, -5.0, 2.0, 6.0, 1.0, -3.0, 1.0, 4.0, 1.0,
-5.0, 7.0, 2.0, 1.0, 2.0 };
int ia[9] = { 0, 3, 7, 10, 14, 17, 21, 23, 26 };
int a_beg = ia[n_beg];
for (int i = n_beg; i < (n_beg + n + 1); i++) {
    ia[i] -= a_beg;
}

/* Internal KML_SCAISS structure */
KmlScasolverTask *handle;

/* KML_SCAISS control parameters */
int error;            /* Output error handle */

/* Create data structures */
const double *a_holder = &a[a_beg];
const int *ja_holder = &ja[a_beg];
```

 版权所有 © 华为技术有限公司

```
const int *ia_holder = &ia[n_beg];
error = KmlScaissGmresInitStripesDI(&handle, mat_size, 1, &n, &n_beg, &a_holder, &ja_holder, &ia_holder,
MPI_COMM_WORLD);

double eps = 1e-4;
error = KmlScaissGmresSetDID(&handle, KMLSS_THRESHOLD, &eps, 1);
if (error != 0) {
    printf("\nERROR in KmlScaissGmresSetDID with KMLSS_THRESHOLD: %d", error);
    return 1;
}
int max_iters = 2000;
error = KmlScaissGmresSetDII(&handle, KMLSS_MAX_ITERATION_COUNT, &max_iters, 1);
if (error != 0) {
    printf("\nERROR in KmlScaissGmresSetDII with KMLSS_MAX_ITERATION_COUNT: %d", error);
    return 1;
}
int precond = KMLSS_BJACOBI;
error = KmlScaissGmresSetDII(&handle, KMLSS_PRECONDITIONER_TYPE, &precond, 1);
if (error != 0) {
    printf("\nERROR in KmlScaissGmresSetDII with KMLSS_PRECONDITIONER_TYPE: %d", error);
    return 1;
}
int num_blocks = 1;
error = KmlScaissGmresPcSetDII(&handle, KMLSS_NUM_BLOCKS, &num_blocks, 1);
if (error != 0) {
    printf("ERROR in KmlScaissGmresPcSetDII: %d\n", error);
    return 1;
}
int subprecond = KMLSS_ICC;
error = KmlScaissGmresPcSetDII(&handle, KMLSS_BLOCK_METHOD, &subprecond, 1);
if (error != 0) {
    printf("ERROR in KmlScaissGmresPcSetDII: %d\n", error);
    return 1;
}
error = KmlScaissGmresAnalyzeDI(&handle);
if (error != 0) {
    printf("ERROR in KmlScaissGmresAnalyzeDI: %d\n", error);
    return 1;
}
error = KmlScaissGmresFactorizeDI(&handle);
if (error != 0) {
    printf("ERROR in KmlScaissGmresFactorizeDI: %d\n", error);
    return 1;
}

int nrhs = 1;
int ldx = n, ldb = n;
double b[8] = { 4.0, 9.0, 7.0, 6.0, 9.0, 3.0, 2.0, 5.0 };
double x[8] = { 0.0, 0.0, 0.0, 0.0, 0.0, 0.0, 0.0, 0.0 };
error = KmlScaissGmresSolveDxDI(&handle, nrhs, &x[n_beg], ldx, &b[n_beg], ldb);
if (error != 0) {
    printf("ERROR in KmlScaissGmresSolveDI: %d\n", error);
    return 1;
}

int idata[] = {0};
error = KmlScaissGmresGetDII(&handle, KMLSS_ITERATION_COUNT, idata, 1);
if (error != 0) {
    printf("\nERROR in KmlScaissGmresGetDII: %d", error);
    return 1;
}
double ddata[] = {0.0};
error = KmlScaissGmresGetDID(&handle, KMLSS_TOLERANCE, ddata, 1);
if (error != 0) {
    printf("\nERROR in KmlScaissGmresGetDID: %d", error);
    return 1;
}
if (rank == 0) printf("[Solve info] Res= \t%.12g\t, iteration = \t%d\t\n", sqrt(ddata[0]), idata[0]);
```


版权所有 © 华为技术有限公司

```
error = KmlScaissGmresCleanDI(&handle);
if (error != 0) {
    printf("\nERROR in KmlScaissGmresCleanDI: %d", error);
    return 1;
}
```

6.5.3.4.3.14 示例

依赖

```
#include "kml_scaiss.h"
```

示例

C Interface：

```
#include <stdio.h>
#include <kml_scaiss.h>
#include "mpi.h"
// USER data struct
typedef struct {
    int n;
    const int *ia;
    const int *ja;
    const double *a;
    MPI_Comm comm;
} duser;
int diagonal_preconditioner(void *usr, double *x)
{
    duser *u = (duser *)usr;
    int i, j;
    /* Apply diagonal preconditioner */
    for (i = 0; i < u->n; i++) {
        for (j = u->ia[i]; j < u->ia[i + 1]; j++) {
            if (u->ja[j] == i) {
                x[i] /= u->a[j];
            }
        }
    }
    return 0;
}
int user_spmv(void *usr, const double *x, double *y)
{
    duser *u = (duser *)usr;
    int size;
    int rank;
    MPI_Comm_size(u->comm, &size);
    MPI_Comm_rank(u->comm, &rank);
    int n = 8 / size;
    double fullX[8] = { 0.0 };
    MPI_Allgather(&x[0], n, MPI_DOUBLE, &fullX[0], n, MPI_DOUBLE, u->comm);
    int i, j;
    for (i = 0; i < u->n; i++) {
        double sum = 0.0;
        for (j = u->ia[i]; j < u->ia[i + 1]; j++) {
            int k = u->ja[j];
            double value = u->a[j];
            sum += fullX[k] * value;
        }
        y[i] = sum;
    }
    return 0;
}
int main(void)
{
    /* MPI initialization */
    MPI_Init(NULL, NULL);
    int size, rank;
```

 版权所有 © 华为技术有限公司

```
MPI_Comm_size(MPI_COMM_WORLD, &size);
MPI_Comm_rank(MPI_COMM_WORLD, &rank);
/* Matrix data (CSR, stored full matrix)
   | 1   *   *   1   2   *   *   * |
   | *   9   2   1   *  -3   *   * |
   | *   2   3   *   *   *   *   2 |
   | 1   1   *   9   *   *  -5   * |
   | 2   *   *   *   6   1   *   * |
   | *  -3   *   *   1   4   *   1 |
   | *   *   *  -5   *   *   7   * |
   | *   *   2   *   *   1   *   2 |
*/
/* Initialize separations */
int mat_size = 8;
int n = mat_size / size;
int n_beg = n * rank;
if (n * size != 8 && rank == (size - 1)) {
    n = mat_size - n * rank;
}
int ia[9] = { 0, 3, 7, 10, 14, 17, 21, 23, 26 };
int a_beg = ia[n_beg];
for (int i = n_beg; i < (n_beg + n + 1); i++) {
    ia[i] -= a_beg;
}
/* clang-format off */
int ja[26] = { 0,       3, 4,
                1, 2, 3,    5,
                1, 2,          7,
             0, 1,    3,       6,
             0,          4, 5,
                1,       4, 5,    7,
                     3,       6,
                  2,       5,    7 };
double a[26] = { 1.0,           1.0, 2.0,
                    9.0, 2.0, 1.0,     -3.0,
                    2.0, 3.0,                  2.0,
                 1.0, 1.0,      9.0,          -5.0,
                 2.0,               6.0, 1.0,
                     -3.0,          1.0, 4.0,      1.0,
                              -5.0,          7.0,
                           2.0,          1.0,      2.0 };
/* clang-format on */
/* Right-hand side vector */
double b[8] = { 4.0, 9.0, 7.0, 6.0, 9.0, 3.0, 2.0, 5.0 };
/* Internal KML_SCAISS structure */
KmlScasolverTask *handle;
/* KML_SCAISS control parameters */
int nrhs = 1;         /* Number of right-hand sides */
int ldx = n, ldb = n; /*!Leading dimension of B and X */
int error;            /* Output error handle */
/* Create data structures */
const double *a_holder = &a[a_beg];
const int *ja_holder = &ja[a_beg];
const int *ia_holder = &ia[n_beg];
error = KmlScaissGmresInitStripesDI(&handle, mat_size, 1, &n, &n_beg, &a_holder, &ja_holder, &ia_holder,
    MPI_COMM_WORLD);
if (error != 0) {
    printf("ERROR in KmlScaissGmresInitStripesDI: %d\n", error);
    return 1;
}
error = KmlScaissGmresAnalyzeDI(&handle);
if (error != 0) {
    printf("ERROR in KmlScaissGmresAnalyzeDI: %d\n", error);
    return 1;
}
error = KmlScaissGmresFactorizeDI(&handle);
if (error != 0) {
    printf("ERROR in KmlScaissGmresFactorizeDI: %d\n", error);
    return 1;
```

 版权所有 © 华为技术有限公司

```
}
/* solve */
double x7[8] = { 0.0, 0.0, 0.0, 0.0, 0.0, 0.0, 0.0, 0.0 };
error = KmlScaissGmresSolveDI(&handle, nrhs, &x7[0], ldx, &b[n_beg], ldb);
if (error != 0) {
    printf("ERROR in KmlScaissGmresSolveDI: %d\n", error);
    return 1;
}
/* Print the solution */
if (rank == 0) {
    printf("solve, x:\n");
    for (int i = 0; i < n; i++) {
        printf("%lf\n", x7[i]);
    }
}
/* solveDx */
double x0[8] = { 0.0, 0.0, 0.0, 0.0, 0.0, 0.0, 0.0, 0.0 };
error = KmlScaissGmresSolveDxDI(&handle, nrhs, &x0[n_beg], ldx, &b[n_beg], ldb);
if (error != 0) {
    printf("ERROR in KmlScaissGmresSolveDI: %d\n", error);
    return 1;
}
/* Print the solution */
if (rank == 0) {
    printf("solveDx, x:\n");
    for (int i = 0; i < n; i++) {
        printf("%lf\n", x0[i]);
    }
}
/* L2 norm */
int NormType = KMLSS_L2;
error = KmlScaissGmresSetDII(&handle, KMLSS_VECTOR_NORM_TYPE, &NormType, 1);
if (error != 0) {
    printf("ERROR in KmlScaissGmresSetDII: %d\n", error);
    return 1;
}
/* Solve */
double x2[8] = { 0.0, 0.0, 0.0, 0.0, 0.0, 0.0, 0.0, 0.0 };
error = KmlScaissGmresSolveDI(&handle, nrhs, &x2[0], ldx, &b[n_beg], ldb);
if (error != 0) {
    printf("ERROR in KmlScaissGmresSolveDI: %d\n", error);
    return 1;
}
/* Print the solution */
if (rank == 0) {
    printf("L2 norm, x:\n");
    for (int i = 0; i < mat_size; i++) {
        printf("%lf\n", x2[i]);
    }
}
/* SolveDx */
double x6[8] = { 0.0, 0.0, 0.0, 0.0, 0.0, 0.0, 0.0, 0.0 };
error = KmlScaissGmresSolveDxDI(&handle, nrhs, &x6[n_beg], ldx, &b[n_beg], ldb);
if (error != 0) {
    printf("ERROR in KmlScaissGmresSolveDI: %d\n", error);
    return 1;
}
/* Print the solution */
if (rank == 0) {
    printf("L2 norm, x:\n");
    for (int i = 0; i < n; i++) {
        printf("%lf\n", x6[i]);
    }
}
/* abs residual */
double res = 1e-10;
error = KmlScaissGmresSetDID(&handle, KMLSS_ABS_TOLERANCE, &res, 1);
if (error != 0) {
    printf("ERROR in KmlScaissGmresSetDIA: %d\n", error);
```

 版权所有 © 华为技术有限公司

```
    return 1;
}
/* Solve */
double x3[8] = { 0.0, 0.0, 0.0, 0.0, 0.0, 0.0, 0.0, 0.0 };
error = KmlScaissGmresSolveDI(&handle, nrhs, &x3[0], ldx, &b[n_beg], ldb);
if (error != 0) {
    printf("ERROR in KmlScaissGmresSolveDI: %d\n", error);
    return 1;
}
/* Print the solution */
if (rank == 0) {
    printf("abs residual, x:\n");
    for (int i = 0; i < mat_size; i++) {
        printf("%lf\n", x3[i]);
    }
}
/* relative residual */
res = 1e-10;
error = KmlScaissGmresSetDID(&handle, KMLSS_THRESHOLD, &res, 1);
if (error != 0) {
    printf("ERROR in KmlScaissGmresSetDIA: %d\n", error);
    return 1;
}
/* Solve */
double x4[8] = { 0.0, 0.0, 0.0, 0.0, 0.0, 0.0, 0.0, 0.0 };
error = KmlScaissGmresSolveDI(&handle, nrhs, &x4[0], ldx, &b[n_beg], ldb);
if (error != 0) {
    printf("ERROR in KmlScaissGmresSolveDI: %d\n", error);
    return 1;
}
/* Print the solution */
if (rank == 0) {
    printf("relative residual, x:\n");
    for (int i = 0; i < mat_size; i++) {
        printf("%lf\n", x4[i]);
    }
}
/* Second call with preconditioner */
double x5[8] = { 0.0, 0.0, 0.0, 0.0, 0.0, 0.0, 0.0, 0.0 };
duser prea;
prea.n = n;
prea.ia = &ia[n_beg];
prea.ja = &ja[a_beg];
prea.a = &a[a_beg];
/* Set user preconditioner */
error = KmlScaissGmresSetUserPreconditionerDI(&handle, &prea, &diagonal_preconditioner);
if (error != 0) {
    printf("ERROR in KmlScaissGmresSetUserPreconditionerDI: %d\n", error);
    return 1;
}
/* Solve */
error = KmlScaissGmresSolveDI(&handle, nrhs, &x5[0], ldx, &b[n_beg], ldb);
if (error != 0) {
    printf("ERROR in KmlScaissGmresSolveDI: %d\n", error);
    return 1;
}
/* Finalize and Clean-up */
error = KmlScaissGmresCleanDI(&handle);
if (error != 0) {
    printf("ERROR in KmlScaissGmresCleanDI: %d\n", error);
    return 1;
}
/* Print the solution */
if (rank == 0) {
    printf("user preconditioner, x:\n");
    for (int i = 0; i < mat_size; i++) {
        printf("%lf\n", x5[i]);
    }
}
```

 版权所有 © 华为技术有限公司

```
    /* Second call with user spmv */
    KmlScasolverTask *handle2;
    double x8[8] = { 0.0, 0.0, 0.0, 0.0, 0.0, 0.0, 0.0, 0.0 };
    duser prea2;
    prea2.n = n;
    prea2.ia = &ia[n_beg];
    prea2.ja = &ja[a_beg];
    prea2.a = &a[a_beg];
    prea2.comm = MPI_COMM_WORLD;
    error = KmlScaissGmresInitWithoutMatDI(&handle2, 1, &n, MPI_COMM_WORLD);
    if (error != 0) {
        printf("ERROR in KmlScaissGmresInitWithoutMatDI: %d\n", error);
        return 1;
    }
    /* Set user spmv */
    error = KmlScaissGmresSetUserSpmvDI(&handle2, &prea2, &user_spmv);
    if (error != 0) {
        printf("ERROR in KmlScaissGmresSetUserSpmvDI: %d\n", error);
        return 1;
    }
    /* Solve */
    error = KmlScaissGmresSolveDxDI(&handle2, nrhs, &x8[n_beg], ldx, &b[n_beg], ldb);
    if (error != 0) {
        printf("ERROR in KmlScaissGmresSolveDI: %d\n", error);
        return 1;
    }
    /* Finalize and Clean-up */
    error = KmlScaissGmresCleanDI(&handle2);
    if (error != 0) {
        printf("ERROR in KmlScaissGmresCleanDI: %d\n", error);
        return 1;
    }
    /* Print the solution */
    if (rank == 0) {
        printf("user spmv, x:\n");
        for (int i = 0; i < n; i++) {
            printf("%lf\n", x8[i]);
        }
    }
    MPI_Finalize();
    return 0;
}
```

6.5.3.4.4 SCAISS-Csi

6.5.3.4.4.1 KmlScaissCsiInitStripes?I

初始化数据结构，并将用户提供的系数矩阵关联到求解系统中，初始化集群版的Csi方法时，每个进程使用局部的CSR格式矩阵。

接口定义

C Interface：

int KmlScaissCsiInitStripesSI(KmlScasolverTask **handle, int n, int nstripes, const int *stripeWidth, const int *stripeRow, const float * const *a, const int * const *ja, const int * const *ia, MPI_Comm comm);

int KmlScaissCsiInitStripesDI(KmlScasolverTask **handle, int n, int nstripes, const int *stripeWidth, const int *stripeRow, const double * const *a, const int * const *ja, const int * const *ia, MPI_Comm comm);

 版权所有 © 华为技术有限公司

参数

参数名	类型	描述	输入/输出
handle	KmlScasolverTask **	求解器句柄，handle不可为空指针。	输入/输出
n	int	矩阵A的维度。	输入
nstripes	int	nstripes为存储在当前MPI进程中的矩阵数，目前只能取1。	输入
stripeWidth	int *	stripeWidth为当前MPI进程中的矩阵行数。该变量的生命周期与handle相同。	输入
stripeRow	int *	stripeRow为当前MPI进程中的矩阵的全局行索引，即首行在全局矩阵中对应行数。该数组的生命周期与handle相同。	输入
a	• 在KmlScaissCsiInitStripesSI中为const float * const * • 在KmlScaissCsiInitStripesDI中为const double * const *	a为维度为非零元素的个数的数组，a[i]为第i个非零元的值。该数组的生命周期与handle相同。	输入
ja	const int * const *。	ja为维度为非零元素的个数的数组，ja[i]为第i个非零元的列号。该数组的生命周期与handle相同。	输入
ia	const int * const *。	ia为stripeWidth+1维数组，ia[i]为第i行第一个非零元在所有非零元中的索引，ia[0]=0。该数组的生命周期与handle相同。	输入
comm	MPI_Comm	为MPI进程通信域。	输入

返回值

返回值	类型	描述
KMLSS_NO_ERROR	int	正常执行。
KMLSS_NULL_ARGUMENT	int	handle，stripeWidth，stripeRow，a，ja，ia中存在空参数。
KMLSS_BAD_N	int	参数n不是正整数。

 版权所有 © 华为技术有限公司

返回值	类型	描述
KMLSS_NONZERO_INDEXING	int	参数ia的首元素不是0。
KMLSS_NOT_IMPLEMENTED	int	参数nstripes不等于1。
KMLSS_NO_MEMORY	int	内存不足。

依赖

#include "kml_scaiss.h"

示例代码请参见 **示例**。

6.5.3.4.4.2 KmlScaissCsiInitWithoutMat?I

初始化数据结构，用于无矩阵求解的初始化，需要与用户自定义矩阵向量乘功能一并使用。

接口定义

C Interface：

int KmlScaissCsiInitWithoutMatSI(KmlScasolverTask **handle, int nstripes, const int *stripeWidth, MPI_Comm comm);

int KmlScaissCsiInitWithoutMatDI(KmlScasolverTask **handle, int nstripes, const int *stripeWidth, MPI_Comm comm);

参数

参数名	类型	描述	输入/输出
handle	KmlScasolverTask **	求解器句柄，handle不可为空指针。	输入/输出
nstripes	int	nstripes为存储在当前MPI进程中的矩阵数，目前只能取1。	输入
stripeWidth	int *	stripeWidth为当前MPI进程中的矩阵行数。该变量的生命周期与handle相同。	输入
comm	MPI_Comm	为MPI进程通信域。	输入

 版权所有 © 华为技术有限公司

返回值

返回值	类型	描述
KMLSS_NO_ERROR	int	正常执行。
KMLSS_NULL_ARGUMENT	int	handle，stripeWidth中存在空参数。
KMLSS_NONZERO_INDEXING	int	参数ia的首元素不是0。
KMLSS_NOT_IMPLEMENTED	int	参数nstripes不等于1。
KMLSS_NO_MEMORY	int	内存不足。

依赖

#include "kml_scaiss.h"

示例代码请参见 **示例**。

6.5.3.4.4.3 KmlScaissCsiSetUserPreconditioner?I

关联用户自定义预条件子回调函数，如不使用自定义的预条件子，该接口无需使用。

接口定义

C Interface：

int KmlScaissCsiSetUserPreconditionerSI(KmlScasolverTask **handle, void *ustruct, int (*fptr)(void *ustruct, float *x));

int KmlScaissCsiSetUserPreconditionerDI(KmlScasolverTask **handle, void *ustruct, int (*fptr)(void *ustruct, double *x));

参数

参数名	类型	描述	输入/输出
handle	KmlScasolverTask **	求解器句柄，传入之前步骤的变量。	输入/输出
ustruct	void *	指向用户数据的指针，每次调用预条件子时作为第一个参数传递。	输入
(*fptr)	int	用户自定义的函数指针，指向回调函数，该回调函数进行预条件子与其第二参数间的计算。	输入

 版权所有 © 华为技术有限公司

返回值

返回值	类型	描述
KMLSS_NO_ERROR	int	正常执行。
KMLSS_NULL_ARGUMENT	int	handle或fptr中存在空参数。

依赖

#include "kml_scaiss.h"

示例代码请参见 **示例**。

6.5.3.4.4.4 KmlScaissCsiSet?I?

设置迭代求解的相关参数。

接口定义

C Interface：

int KmlScaissCsiSetSII(KmlScasolverTask **handle, enum KmlSolverParam param, const int *data, int nd);

int KmlScaissCsiSetSIS(KmlScasolverTask **handle, enum KmlSolverParam param, const float *data, int nd);

int KmlScaissCsiSetDII(KmlScasolverTask **handle, enum KmlSolverParam param, const int *data, int nd);

int KmlScaissCsiSetDID(KmlScasolverTask **handle, enum KmlSolverParam param, const double *data, int nd);

参数

参数名	类型	描述	输入/输出
handle	KmlScasolverTask **	求解器句柄，传入之前步骤的变量。	输入/输出

版权所有 © 华为技术有限公司

参数名	类型	描述	输入/输出
param	enum KmlSolverParam	• KMLSS_THRESHOLD表示迭代残差相对阈值。 • KMLSS_ABS_TOLERANCE表示迭代残差绝对阈值。 • KMLSS_MAX_ITERATION_COUNT表示最大迭代次数。 • KMLSS_PRECONDITIONER_TYPE表示预条件子类型。 • KMLSS_VECTOR_NORM_TYPE表示迭代残差的计算范数 • KMLSS_MATRIX_FORMAT表示矩阵对角块稀疏存储格式。 • KMLSS_OFFDIAG_MATRIX_FORMAT表示矩阵非对角块稀疏存储格式。 • KMLSS_SPECTRUM_BOUNDS表示最小和最大特征值的数组。	输入
data	• 在KmlScaissCsiSetSII中为const int *。 • 在KmlScaissCsiSetSIS中为const float *。 • KmlScaissCsiSetDII中为const int *。 • KmlScaissCsiSetDID中为const double *。	迭代求解相关参数数组（如阈值、最大迭代次数等）。 当前可选的计算范数： • 1-NORM：KMLSS_L1 • 2-NORM：KMLSS_L2 当前可选的稀疏矩阵存储格式： • CSR格式：KMLSS_CSR • ELLPACK格式：KMLSS_ELL • COO格式：KMLSS_COO • ELLPACK_COO混合格式：KMLSS_SELLCOO 当前可选择的预条件子类型： • JACOBI预条件子：KMLSS_JACOBI • BLOCK JACOBI预条件子：KMLSS_BJACOBI • AMG预条件子：KMLSS_AMG	输入
nd	int	data数组元素个数。	输入

版权所有 © 华为技术有限公司

返回值

返回值	类型	描述
KMLSS_NO_ERROR	int	正常执行。
KMLSS_DATA_SIZE	int	参数nd不等于1。
KMLSS_NULL_ARGUMENT	int	handle和data中存在空参数。
KMLSS_BAD_SELECTOR	int	param为无效参数。
KMLSS_BAD_PRECONDITIONER	int	选择的预条件子暂未实现。

依赖

#include "kml_scaiss.h"

示例代码请参见 **示例**。

6.5.3.4.4.5 KmlScaissCsiSetUserSpmv?I

关联用户自定义矩阵向量乘函数，需要与无矩阵初始化接口一起使用。

接口定义

C Interface：

int KmlScaissCsiSetUserSpmvSI(KmlScasolverTask **handle, void *ustruct, int (*fptr)(void *ustruct, const float *x, float *x));

int KmlScaissCsiSetUserSpmvDI(KmlScasolverTask **handle, void *ustruct, int (*fptr)(void *ustruct, const double *x, double *y));

参数

参数名	类型	描述	输入/输出
handle	KmlScasolverTask **	求解器句柄，传入之前步骤的变量。	输入/输出
ustruct	void *	指向用户数据的指针，每次调用预条件子时作为第一个参数传递。	输入
(*fptr)	int	用户自定义的函数指针，指向矩阵向量乘函数，该矩阵向量乘函数进行矩阵与其第二参数间的计算。	输入

版权所有 © 华为技术有限公司

返回值

返回值	类型	描述
KMLSS_NO_ERROR	int	正常执行。
KMLSS_DATA_SIZE	int	参数nd不等于1。
KMLSS_NULL_ARGUMENT	int	handle，data中存在空参数。

依赖

#include "kml_scaiss.h"

示例代码请参见 **示例**。

6.5.3.4.4.6 KmlScaissCsiPcSet?I?

设置迭代求解的预条件子相关参数，当前仅支持为block jacobi预条件子设置相关参数。

接口定义

C Interface：

int KmlScaissCsiPcSetSII(KmlScasolverTask **handle, enum KmlSolverPreconditionerParam param, const int *data, int nd);

int KmlScaissCsiPcSetSIS(KmlScasolverTask **handle, enum KmlSolverPreconditionerParam param, const float *data, int nd);

int KmlScaissCsiPcSetDII(KmlScasolverTask **handle, enum KmlSolverPreconditionerParam param, const int *data, int nd);

int KmlScaissCsiPcSetDID(KmlScasolverTask **handle, enum KmlSolverPreconditionerParam param, const double *data, int nd);

参数

参数名	类型	描述	输入/输出
handle	KmlScasolverTask **	求解器句柄，传入之前步骤的变量。	输入/输出
param	enum KmlSolverParam	• KMLSS_BLOCK_METHOD表示每个分块中的预条件子类型。 • KMLSS_NUM_BLOCKS表示每个进程中的分块数量。 • KMLSS_BLOCK_SIZES表示每个进程中的分块大小。	输入

 版权所有 © 华为技术有限公司

参数名	类型	描述	输入/输出
data	• 在KmlScaissCsiPcSetSII、KmlScaissCsiPcSetDII中为const int *。 • 在KmlScaissCsiPcSetSIS中为const float *。 • 在KmlScaissCsiPcSetDID中为const double *。	预条件子相关参数（如预条件子类型、分块数量、分块大小）。 当前可选择的预条件子类型： 1. ILU预条件子：KMLSS_ILU。 2. ICC预条件子：KMLSS_ICC。 3. SOR预条件子：KMLSS_SOR。 4. ILUT预条件子：KMLSS_ILUT。	输入
nd	int	data数组元素个数。	输入

返回值

返回值	类型	描述
KMLSS_NO_ERROR	int	正常执行。
KMLSS_DATA_SIZE	int	参数nd不等于1。
KMLSS_NULL_ARGUMENT	int	handle，data中存在空参数。
KMLSS_BAD_SELECTOR	int	param为无效参数。
KMLSS_BAD_PRECONDITIONER	int	选择的预条件子暂未实现。

依赖

#include "kml_scaiss.h"

示例代码请参见 **示例**。

6.5.3.4.4.7 KmlScaissCsiSubPcSet?I?

设置迭代求解的block jacobi中子预条件子相关参数。

接口定义

C Interface：

 版权所有 © 华为技术有限公司

int KmlScaissCsiSubPcSetSIS(KmlScasolverTask **handle, enum KmlSolverPreconditionerParam param, int subn, const float *data, int nd);

int KmlScaissCsiSubPcSetSII(KmlScasolverTask **handle, enum KmlSolverPreconditionerParam param, int subn, const int *data, int nd);

int KmlScaissCsiSubPcSetDII(KmlScasolverTask **handle, enum KmlSolverPreconditionerParam param, int subn, const int *data, int nd);

int KmlScaissCsiSubPcSetDID(KmlScasolverTask **handle, enum KmlSolverPreconditionerParam param, int subn, const double *data, int nd);

参数

参数名	类型	描述	输入/输出
handle	KmlScasolverTask **	求解器句柄，传入之前步骤的变量。	输入/输出
param	enum KmlSolverParam	• KML_RELAXATION_FACTOR表示SOR预条件子相关参数。 • KMLSS_DROP_FILL表示ILUT预条件子每行最多保留元素个数。 • KMLSS_DROP_TOL表示ILUT预条件子保留元素的阈值。	输入
subn	int	子预条件子索引	输入
data	• 在KmlScaissCsiSubPcSetSII、KmlScaissCsiSubPcSetDII中为const int *。 • 在KmlScaissCsiSubPcSetSIS、KmlScaissCsiSubPcSetSLS中为const float *。 • 在KmlScaissCsiSubPcSetDID、KmlScaissCsiSubPcSetDLD中为const double *。	预条件子相关参数	输入
nd	int	data数组元素个数。	输入

版权所有 © 华为技术有限公司

返回值

返回值	类型	描述
KMLSS_NO_ERROR	int	正常执行。
KMLSS_DATA_SIZE	int	参数nd不等于1。
KMLSS_NULL_ARGUMENT	int	handle和data中存在空参数。
KMLSS_BAD_SELECTOR	int	param为无效参数。
KMLSS_BAD_PRECONDITIONER	int	选择的预条件子暂未实现。

依赖

#include "kml_scaiss.h"

示例代码请参见 **示例**。

6.5.3.4.4.8 KmlScaissCsiAnalyze?I

分析稀疏矩阵，并在需要时更改存储表示。

接口定义

C Interface：

int KmlScaissCsiAnalyzeSI(KmlScasolverTask **handle);

int KmlScaissCsiAnalyzeDI(KmlScasolverTask **handle);

参数

参数名	类型	描述	输入/输出
handle	KmlScasolverTask **	求解器句柄，传入之前步骤的变量。	输入/输出

返回值

返回值	类型	描述
KMLSS_NO_ERROR	int	正常执行。
KMLSS_NULL_ARGUMENT	int	handle为空参数。

 版权所有 © 华为技术有限公司

返回值	类型	描述
KMLSS_NO_MEMORY	int	内存不足。
KMLSS_BAD_MATRIX_FORMAT	int	输入矩阵格式不正确。

依赖

#include "kml_scaiss.h"

示例代码请参见 **示例**。

6.5.3.4.4.9 KmlScaissCsiFactorize?I

在求解器内部对矩阵进行赋值，并计算预条件子。无论是否使用预条件子，都需要调用该接口。

接口定义

C Interface：

int KmlScaissCsiFactorizeSI(KmlScasolverTask **handle);

int KmlScaissCsiFactorizeDI(KmlScasolverTask **handle);

参数

参数名	类型	描述	输入/输出
handle	KmlScasolverTask **	求解器句柄，传入之前步骤的变量。	输入/输出

返回值

返回值	类型	描述
KMLSS_NO_ERROR	int	正常执行。
KMLSS_NULL_ARGUMENT	int	handle为空参数。
KMLSS_ZERO_DIAGONAL_ELEMENT	int	矩阵对角线存在零元。

依赖

#include "kml_scaiss.h"

 版权所有 © 华为技术有限公司

示例代码请参见 **示例**。

6.5.3.4.4.10 KmlScaissCsiSolve?I

求解线性代数方程组，每个进程读取并存储全局求解结果。

接口定义

C Interface：

int KmlScaissCsiSolveSI(KmlScasolverTask **handle, int nb, float *x, int ldx, const float *b, int ldb);

int KmlScaissCsiSolveDI(KmlScasolverTask **handle, int nb, double *x, int ldx, const double *b, int ldb);

参数

参数名	类型	描述	输入/输出
handle	KmlScasolverTask **	求解器句柄，传入之前步骤的变量。	输入/输出
nb	int	右端项矢量的个数，当前nb只能为1。	输入
x	• 在KmlScaissCsiSolveSI中为float * • 在KmlScaissCsiSolveDI中为double *。	读取并存储全局求解结果，其元素个数为全局矩阵大小n。	输入/输出
ldx	int	x的leading dimension大小，当前参数未使用。	输入
b	• 在KmlScaissCsiSolveSI中为float * • 在KmlScaissCsiSolveDI中为double *。	读取进程对应的局部右端项，其元素个数为当前进程对应的矩阵的行数，即stripeWidth。	输入
ldb	int	b的leading dimension大小，当前参数未使用。	输入

返回值

返回值	类型	描述
KMLSS_NO_ERROR	int	正常执行。

版权所有 © 华为技术有限公司

返回值	类型	描述
KMLSS_BAD_NB	int	参数nb不等于1。
KMLSS_NO_MEMORY	int	内存不足。
KMLSS_NULL_ARGUMENT	int	handle，x，b中存在空参数。

依赖

#include "kml_scaiss.h"

示例代码请参见 **示例**。

6.5.3.4.4.11 KmlScaissCsiSolveDx?I

求解线性代数方程组，每个进程读取并存储当前进程对应的求解结果。

接口定义

C Interface：

int KmlScaissCsiSolveDxSI(KmlScasolverTask **handle, int nb, float *x, int ldx, const float *b, int ldb);

int KmlScaissCsiSolveDxDI(KmlScasolverTask **handle, int nb, double *x, int ldx, const double *b, int ldb);

参数

参数名	类型	描述	输入/输出
handle	KmlScasolverTask **	求解器句柄，传入之前步骤的变量。	输入/输出
nb	int	右端项矢量的个数，当前nb只能为1。	输入
x	• 在KmlScaissCsiSolveDxSI中为float * 。 • 在KmlScaissCsiSolveDxDI中为double *。	读取并存储当前进程对应的求解结果，其元素个数为当前进程对应的矩阵的行数，即stripeWidth。	输入/输出
ldx	int	x的leading dimension大小，当前参数未使用。	输入

版权所有 © 华为技术有限公司

参数名	类型	描述	输入/输出
b	• 在KmlScaissCsiSolveDxSI中为const float *。 • 在KmlScaissCsiSolveDxDI中为const double *。	读取进程对应的局部右端项，其元素个数为当前进程对应的矩阵的行数，即stripeWidth。	输入
ldb	int	b的leading dimension大小，当前参数未使用。	输入

返回值

返回值	类型	描述
KMLSS_NO_ERROR	int	正常执行。
KMLSS_BAD_NB	int	参数nb不等于1。
KMLSS_NO_MEMORY	int	内存不足。
KMLSS_NULL_ARGUMENT	int	handle，x，b中存在空参数。

依赖

#include "kml_scaiss.h"

示例代码请参见 **示例**。

6.5.3.4.4.12 KmlScaissCsiGet?I?

获得迭代求解相关参数。

接口定义

C Interface：

int KmlScaissCsiGetSII(KmlScasolverTask **handle, enum KmlSolverParam param, int *data, int nd);

int KmlScaissCsiGetSIS(KmlScasolverTask **handle, enum KmlSolverParam param, float *data, int nd);

int KmlScaissCsiGetDII(KmlScasolverTask **handle, enum KmlSolverParam param, int *data, int nd);

 版权所有 © 华为技术有限公司

```
int KmlScaissCsiGetDID(KmlScasolverTask **handle, enum KmlSolverParam param, double *data, int nd);
```

参数

参数名	类型	描述	输入/输出
handle	KmlScasolverTask **	求解器句柄，传入之前步骤的变量。	输入/输出
param	enum KmlSolverParam	• KMLSS_ITERATION_COUNT表示迭代次数。 • KMLSS_MAX_ITERATION_COUNT表示设置的最大迭代次数。 • KMLSS_TOLERANCE表示最终的迭代相对残差。 • KMLSS_THRESHOLD表示设置的相对残差。	输入
data	• 在KmlScaissCsiGetSII、KmlScaissCsiGetDII中为int *。 • 在KmlScaissCsiGetSIS中为float *。 • 在KmlScaissCsiGetDID中为double *。	-	输出
nd	int	data数组元素个数。	输入

返回值

返回值	类型	描述
KMLSS_NO_ERROR	int	正常执行。
KMLSS_DATA_SIZE	int	参数nd不等于1。
KMLSS_NULL_ARGUMENT	int	handle，data中存在空参数。
KMLSS_BAD_SELECTOR	int	param为无效参数。

版权所有 © 华为技术有限公司

依赖

#include "kml_scaiss.h"

示例代码请参见 **示例**。

6.5.3.4.4.13 KmlScaissCsiClean?I

释放内部数据结构。

接口定义

C Interface：

int KmlScaissCsiCleanSI(KmlScasolverTask **handle);

int KmlScaissCsiCleanDI(KmlScasolverTask **handle);

参数

参数名	类型	描述	输入/输出
handle	KmlScasolverTask **	求解器句柄，传入之前步骤的变量。	输入/输出

返回值

返回值	类型	描述
KMLSS_NO_ERROR	int	正常执行。
KMLSS_NULL_ARGUMENT	int	handle为空参数。

依赖

#include "kml_scaiss.h"

示例代码请参见 **示例**。

6.5.3.4.4.14 示例

示例

C Interface：

```
#include <stdio.h>
#include <kml_scaiss.h>
#include "mpi.h"
// USER data struct
typedef struct {
    int n;
    const int *ia;
```

 版权所有 © 华为技术有限公司

```
    const int *ja;
    const double *a;
    MPI_Comm comm;
} duser;
int user_spmv(void *usr, const double *x, double *y)
{
    duser *u = (duser *)usr;
    int size;
    int rank;
    MPI_Comm_size(u->comm, &size);
    MPI_Comm_rank(u->comm, &rank);
    int n = 8 / size;
    double fullX[8] = { 0.0 };
    MPI_Allgather(&x[0], n, MPI_DOUBLE, &fullX[0], n, MPI_DOUBLE, u->comm);
    int i, j;
    for (i = 0; i < u->n; i++) {
        double sum = 0.0;
        for (j = u->ia[i]; j < u->ia[i + 1]; j++) {
            int k = u->ja[j];
            double value = u->a[j];
            sum += fullX[k] * value;
        }
        y[i] = sum;
    }
    return 0;
}
int main(void)
{
    /* MPI initialization */
    MPI_Init(NULL, NULL);
    int size, rank;
    MPI_Comm_size(MPI_COMM_WORLD, &size);
    MPI_Comm_rank(MPI_COMM_WORLD, &rank);
    /* Matrix data (CSR, stored full matrix)
        | 1   *   *   1   2   *   *   * |
        | *   9   2   1   *  -3   *   * |
        | *   2   3   *   *   *   *   2 |
        | 1   1   *   9   *   *  -5   * |
        | 2   *   *   *   6   1   *   * |
        | *  -3   *   *   1   4   *   1 |
        | *   *   *  -5   *   *   7   * |
        | *   *   2   *   *   1   *   2 |
    */
    /* Initialize separations */
    int mat_size = 8;
    int n = mat_size / size;
    int n_beg = n * rank;
    if (n * size != 8 && rank == (size - 1)) {
        n = mat_size - n * rank;
    }
    int ia[9] = { 0, 3, 7, 10, 14, 17, 21, 23, 26 };
    int a_beg = ia[n_beg];
    for (int i = n_beg; i < (n_beg + n + 1); i++) {
        ia[i] -= a_beg;
    }
    /* clang-format off */
    int ja[26] = { 0,       3, 4,
                   1, 2, 3,    5,
                   1, 2,             7,
                0, 1,    3,       6,
                0,          4, 5,
                   1,       4, 5,    7,
                        3,       6,
                     2,       5,    7 };
    double a[26] = { 1.0,           1.0, 2.0,
                       9.0, 2.0, 1.0,     -3.0,
                       2.0, 3.0,                   2.0,
                    1.0, 1.0,      9.0,          -5.0,
                    2.0,               6.0, 1.0,
```


版权所有 © 华为技术有限公司

```
                        -3.0,          1.0, 4.0,      1.0,
                                -5.0,          7.0,
                             2.0,          1.0,      2.0 };
/* clang-format on */
/* Right-hand side vector */
double b[8] = { 4.0, 9.0, 7.0, 6.0, 9.0, 3.0, 2.0, 5.0 };
/* Solution vector */
double x[8] = { 0.0, 0.0, 0.0, 0.0, 0.0, 0.0, 0.0, 0.0 };
/* Internal KML_SCAISS structure */
KmlScasolverTask *handle;
/* KML_SCAISS control parameters */
int nrhs = 1;         /* Number of right-hand sides */
int ldx = n, ldb = n; /*!Leading dimension of B and X */
int error;            /* Output error handle */
/* Create data structures */
const double *a_holder = &a[a_beg];
const int *ja_holder = &ja[a_beg];
const int *ia_holder = &ia[n_beg];
error = KmlScaissCsiInitStripesDI(&handle, mat_size, 1, &n, &n_beg, &a_holder, &ja_holder, &ia_holder,
    MPI_COMM_WORLD);
if (error != 0) {
    printf("ERROR in KmlScaissCsiInitStripesDI: %d\n", error);
    return 1;
}
double eigs[2] = { 0.061645, 13.350966 };
error = KmlScaissCsiSetDID(&handle, KMLSS_SPECTRUM_BOUNDS, &eigs[0], 2);
if (error != 0) {
    printf("ERROR in KmlScaissCsiSetDID: %d\n", error);
    return 1;
}
error = KmlScaissCsiAnalyzeDI(&handle);
if (error != 0) {
    printf("ERROR in KmlScaissCsiAnalyzeDI: %d\n", error);
    return 1;
}
error = KmlScaissCsiFactorizeDI(&handle);
if (error != 0) {
    printf("ERROR in KmlScaissCsiFactorizeDI: %d\n", error);
    return 1;
}
/* solve */
double x7[8] = { 0.0, 0.0, 0.0, 0.0, 0.0, 0.0, 0.0, 0.0 };
error = KmlScaissCsiSolveDI(&handle, nrhs, &x7[0], ldx, &b[n_beg], ldb);
if (error != 0) {
    printf("ERROR in KmlScaissCsiSolveDI: %d\n", error);
    return 1;
}
/* Print the solution */
if (rank == 0) {
    printf("solve, x:\n");
    for (int i = 0; i < n; i++) {
        printf("%lf\n", x7[i]);
    }
}
/* solveDx */
double x0[8] = { 0.0, 0.0, 0.0, 0.0, 0.0, 0.0, 0.0, 0.0 };
error = KmlScaissCsiSolveDxDI(&handle, nrhs, &x0[n_beg], ldx, &b[n_beg], ldb);
if (error != 0) {
    printf("ERROR in KmlScaissCsiSolveDI: %d\n", error);
    return 1;
}
/* Print the solution */
if (rank == 0) {
    printf("solveDx, x:\n");
    for (int i = 0; i < n; i++) {
        printf("%lf\n", x0[i]);
    }
}
/* L1 norm */
```

 版权所有 © 华为技术有限公司

```
int NormType = KMLSS_L1;
error = KmlScaissCsiSetDII(&handle, KMLSS_VECTOR_NORM_TYPE, &NormType, 1);
if (error != 0) {
    printf("ERROR in KmlScaissCsiSetDII: %d\n", error);
    return 1;
}
/* Solve */
error = KmlScaissCsiSolveDI(&handle, nrhs, &x[0], ldx, &b[n_beg], ldb);
if (error != 0) {
    printf("ERROR in KmlScaissCsiSolveDI: %d\n", error);
    return 1;
}
/* Print the solution */
if (rank == 0) {
    printf("L1 norm, x:\n");
    for (int i = 0; i < mat_size; i++) {
        printf("%lf\n", x[i]);
    }
}
/* SolveDx */
double x6[8] = { 0.0, 0.0, 0.0, 0.0, 0.0, 0.0, 0.0, 0.0 };
error = KmlScaissCsiSolveDxDI(&handle, nrhs, &x6[n_beg], ldx, &b[n_beg], ldb);
if (error != 0) {
    printf("ERROR in KmlScaissCsiSolveDI: %d\n", error);
    return 1;
}
/* Print the solution */
if (rank == 0) {
    printf("L2 norm, x:\n");
    for (int i = 0; i < n; i++) {
        printf("%lf\n", x6[i]);
    }
}
/* L2 norm */
NormType = KMLSS_L2;
error = KmlScaissCsiSetDII(&handle, KMLSS_VECTOR_NORM_TYPE, &NormType, 1);
if (error != 0) {
    printf("ERROR in KmlScaissCsiSetDII: %d\n", error);
    return 1;
}
/* Solve */
double x2[8] = { 0.0, 0.0, 0.0, 0.0, 0.0, 0.0, 0.0, 0.0 };
error = KmlScaissCsiSolveDI(&handle, nrhs, &x2[0], ldx, &b[n_beg], ldb);
if (error != 0) {
    printf("ERROR in KmlScaissCsiSolveDI: %d\n", error);
    return 1;
}
/* Print the solution */
if (rank == 0) {
    printf("L2 norm, x:\n");
    for (int i = 0; i < mat_size; i++) {
        printf("%lf\n", x2[i]);
    }
}
/* abs residual */
double res = 1e-10;
error = KmlScaissCsiSetDID(&handle, KMLSS_ABS_TOLERANCE, &res, 1);
if (error != 0) {
    printf("ERROR in KmlScaissCsiSetDIA: %d\n", error);
    return 1;
}
/* Solve */
double x3[8] = { 0.0, 0.0, 0.0, 0.0, 0.0, 0.0, 0.0, 0.0 };
error = KmlScaissCsiSolveDI(&handle, nrhs, &x3[0], ldx, &b[n_beg], ldb);
if (error != 0) {
    printf("ERROR in KmlScaissCsiSolveDI: %d\n", error);
    return 1;
}
/* Print the solution */
```

 版权所有 © 华为技术有限公司

```
if (rank == 0) {
    printf("abs residual, x:\n");
    for (int i = 0; i < mat_size; i++) {
        printf("%lf\n", x3[i]);
    }
}
/* relative residual */
res = 1e-10;
error = KmlScaissCsiSetDID(&handle, KMLSS_THRESHOLD, &res, 1);
if (error != 0) {
    printf("ERROR in KmlScaissCsiSetDIA: %d\n", error);
    return 1;
}
/* Solve */
double x4[8] = { 0.0, 0.0, 0.0, 0.0, 0.0, 0.0, 0.0, 0.0 };
error = KmlScaissCsiSolveDI(&handle, nrhs, &x4[0], ldx, &b[n_beg], ldb);
if (error != 0) {
    printf("ERROR in KmlScaissCsiSolveDI: %d\n", error);
    return 1;
}
/* Print the solution */
if (rank == 0) {
    printf("relative residual, x:\n");
    for (int i = 0; i < mat_size; i++) {
        printf("%lf\n", x4[i]);
    }
}
/* Second call with user spmv */
double x8[8] = { 0.0, 0.0, 0.0, 0.0, 0.0, 0.0, 0.0, 0.0 };
duser prea;
prea.n = n;
prea.ia = &ia[n_beg];
prea.ja = &ja[a_beg];
prea.a = &a[a_beg];
prea.comm = MPI_COMM_WORLD;
error = KmlScaissCsiInitWithoutMatDI(&handle, 1, &n, MPI_COMM_WORLD);
if (error != 0) {
    printf("ERROR in KmlScaissCsiInitWithoutMatDI: %d\n", error);
    return 1;
}
error = KmlScaissCsiSetDID(&handle, KMLSS_SPECTRUM_BOUNDS, &eigs[0], 2);
if (error != 0) {
    printf("ERROR in KmlScaissCsiSetDID: %d\n", error);
    return 1;
}
/* Set user spmv */
error = KmlScaissCsiSetUserSpmvDI(&handle, &prea, &user_spmv);
if (error != 0) {
    printf("ERROR in KmlScaissCsiSetUserSpmvDI: %d\n", error);
    return 1;
}
/* Solve */
error = KmlScaissCsiSolveDxDI(&handle, nrhs, &x8[n_beg], ldx, &b[n_beg], ldb);
if (error != 0) {
    printf("ERROR in KmlScaissCsiSolveDxDI: %d\n", error);
    return 1;
}
/* Finalize and Clean-up */
error = KmlScaissCsiCleanDI(&handle);
if (error != 0) {
    printf("ERROR in KmlScaissCsiCleanDI: %d\n", error);
    return 1;
}
/* Print the solution */
if (rank == 0) {
    printf("user spmv, x:\n");
    for (int i = 0; i < n; i++) {
        printf("%lf\n", x8[i]);
    }
```

 版权所有 © 华为技术有限公司

```
    }
    MPI_Finalize();
    return 0;
}
```

6.5.3.4.5 HMG

6.5.3.4.5.1 KmlMgMatrixCreate

创建矩阵句柄。

说明

创建矩阵时是浅拷贝，用户需要保证MgMatrixDestroy之前不释放参数store指定的内存。

接口定义

C Interface：

KmlMgStatus KmlMgMatrixCreate(KmlMgMatrixH* pMatrix, const KmlMgMatrixStore* store, const KmlMgMatrixOptions* options);

参数

参数名	类型	描述	输入/输出
pMatrix	KmlMgMatrixH*	矩阵句柄，pMatrix不可为空指针	输入/输出
store	const KmlMgMatrixStore*	矩阵的结构与数据信息	输入
options	const KmlMgMatrixOptions*	矩阵的数据类型等信息	输入

返回值

返回值	类型	描述
KML_MG_OK	int	正常执行
KML_MG_ERR_INVALID_PARAM	int	参数无效
KML_MG_ERR_UNSUPPORTED	int	暂不支持的功能
KML_MG_ERR_NO_MEMORY	int	分配内存失败
KML_MG_ERR_EXCEEDS_LIMIT	int	超过设置限制

版权所有 © 华为技术有限公司

返回值	类型	描述
KML_MG_ERR_INCOMPATIBLE	int	版本不兼容
KML_MG_ERR_NO_REQUIRED_FIELD	int	没设置必填字段
KML_MG_ERR_UNKNOWN_TYPE	int	未知的类型
KML_MG_ERR_THIRD_PART	int	内部调用第三方错误
KML_MG_ERR_INCONSISTENT	int	传递信息不一致
KML_MG_ERR_UNEXPECTED_CALL_ORDER	int	预期之外的调用顺序
KML_MG_ERR_MATRIX_VALUE_IS_NULL	int	矩阵值为空
KML_MG_ERR_NO_RESOURCE	int	资源未就绪

结构体说明

KmlMgMatrixOptions

字段名	类型	描述
fieldMask	uint64_t	配置生效的字段掩码。非0的位代表配置对应字段指定的值，0的位代表使用默认值，可配置的掩码位请参见具体字段的说明。
indexType	KmlMgIndexType	坐标类型，有如下值。 ● KML_MG_INDEX_INT32（目前只支持32位int） ● KML_MG_INDEX_INT64 掩码位：MG_MATRIX_OPTIONS_INDEX_TYPE
valueType	KmlMgValueType	数值类型，有如下值。（目前只支持FP32和FP64作为用户输入的数值类型） ● KML_MG_VALUE_FP16 ● KML_MG_VALUE_FP32 ● KML_MG_VALUE_FP64 掩码位：KML_MG_MATRIX_OPTIONS_VALUE_TYPE

版权所有 © 华为技术有限公司

KmlMgMatrixStore

字段名	类型	描述
comm	MPI_Comm	MPI 通信子
globalx	int64_t	所有进程组成的全局矩阵的x方向维度大小
globaly	int64_t	所有进程组成的全局矩阵的y方向维度大小
globalz	int64_t	所有进程组成的全局矩阵的z方向维度大小
procx	int64_t	x方向上划分的进程数量
procy	int64_t	y方向上划分的进程数量
procz	int64_t	z方向上划分的进程数量
xbeg	int64_t	当前进程的x方向起始坐标（包含该坐标）
xend	int64_t	当前进程的x方向终止坐标（不含该坐标）
ybeg	int64_t	当前进程的y方向起始坐标（包含该坐标）
yend	int64_t	当前进程的y方向终止坐标（不含该坐标）
zbeg	int64_t	当前进程的z方向起始坐标（包含该坐标）
zend	int64_t	当前进程的z方向终止坐标（不含该坐标）
xhalo	int64_t	当前进程的x方向的halo区域大小，当前版本仅支持halo=1
yhalo	int64_t	当前进程的y方向的halo区域大小，当前版本仅支持halo=1
zhalo	int64_t	当前进程的z方向的halo区域大小，当前版本仅支持halo=1
crossPolar	bool	网格是否跨极点通信，即网格边界是否连续

版权所有 © 华为技术有限公司

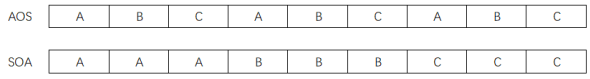
字段名	类型	描述
type	KmlMgMatrixStoreType	矩阵格式，均为结构化网格矩阵，有如下值。 ● KML_MG_MATRIX_STORE_DENSE_ROW_MAJOR（暂不支持） ● KML_MG_MATRIX_STORE_DENSE_COL_MAJOR（暂不支持） ● KML_MG_MATRIX_STORE_DENSE_ROW_MAJOR_HALO（暂不支持） ● KML_MG_MATRIX_STORE_DENSE_COL_MAJOR_HALO ● KML_MG_MATRIX_STORE_SPARSE（暂不支持） ● KML_MG_MATRIX_STORE_SPARSE_HALO 目前只支持向量为KML_MG_MATRIX_STORE_DENSE_COL_MAJOR_HALO，矩阵为KML_MG_MATRIX_STORE_SPARSE_HALO
dense	KmlMgMatrixStoreDense	稠密格式，目前用于存储右端向量和解向量，与sparse字段共同构成联合体
sparse	KmlMgMatrixStoreSparse	稀疏格式，目前用于存储稀疏矩阵，与dense字段共同构成联合体

KmlMgMatrixStoreDense

字段名	类型	描述
values	void*	数值数组，目前实现必须按Y-X-Z顺序（从外到里）排列
numDim	int64_t	次维度大小，表示行（列）主序的列（行）数
is2D	bool	是否为2D网格
needCorner	bool	网格结构是否存在角点，对应KmlMgMatrixStoreSparse中的stencil为KML_MG_STENCIL_2D5或KML_MG_STENCIL_3D7则为false，否则为true。

KmlMgMatrixStoreSparse

版权所有 © 华为技术有限公司

字段名	类型	描述
values	void*	数值数组，目前实现必须按Y-X-Z顺序（从外到里）排列
type	KmlMgSparseType	稀疏矩阵排列顺序，有如下值。 • KML_MG_SPARSE_AOS • KML_MG_SPARSE_SOA（暂不支持） 如下图所示，对于本程序的结构化网格矩阵来说，AOS按网格（行）连续存储，SOA则按每条对角线连续存储 
stencil	KmlMgStencilType	表示网格结构，稀疏矩阵按照行存储，根据网格结构决定矩阵每行的非零元个数，有如下值。 • KML_MG_STENCIL_2D5（暂不支持） • KML_MG_STENCIL_2D9（暂不支持） • KML_MG_STENCIL_3D7 • KML_MG_STENCIL_3D15（暂不支持） • KML_MG_STENCIL_3D19 • KML_MG_STENCIL_3D27

说明

使用暂不支持的参数会导致程序异常退出。

依赖

#include "mg.h"

示例代码请参见 **示例**。

6.5.3.4.5.2 KmlMgMatrixSetValues

为矩阵类型填入数值。

说明

指定的矩阵数值应当与创建该矩阵句柄时指定的非零元结构对应，该接口应用于矩阵结构不变但值变化的场景。通过该接口更新values数组后，原values数组空间可释放。

接口定义

C Interface：

KmlMgStatus KmlMgMatrixSetValues(KmlMgMatrixH matrix, void* values);

 版权所有 © 华为技术有限公司

参数

参数名	类型	描述	输入/输出
matrix	KmlMgMatrixH	由KmlMgMatrixCreate函数创建的矩阵句柄	输入/输出
values	void *	矩阵数值数组	输入

返回值

返回值	类型	描述
KML_MG_OK	int	正常执行
KML_MG_ERR_INVALID_PARAM	int	参数无效
KML_MG_ERR_UNSUPPORTED	int	暂不支持的功能
KML_MG_ERR_NO_MEMORY	int	分配内存失败
KML_MG_ERR_EXCEEDS_LIMIT	int	超过设置限制
KML_MG_ERR_INCOMPATIBLE	int	版本不兼容
KML_MG_ERR_NO_REQUIRED_FIELD	int	没设置必填字段
KML_MG_ERR_UNKNOWN_TYPE	int	未知的类型
KML_MG_ERR_THIRD_PART	int	内部调用第三方错误
KML_MG_ERR_INCONSISTENT	int	传递信息不一致
KML_MG_ERR_UNEXPECTED_CALL_ORDER	int	预期之外的调用顺序
KML_MG_ERR_MATRIX_VALUE_IS_NULL	int	矩阵值为空
KML_MG_ERR_NO_RESOURCE	int	资源未就绪

版权所有 © 华为技术有限公司

依赖

#include "mg.h"

示例代码请参见 **示例**。

6.5.3.4.5.3 KmlMgMatrixDestroy

销毁矩阵句柄并释放资源。

接口定义

C Interface：

void KmlMgMatrixDestroy(KmlMgMatrixH matrix);

参数

参数名	类型	描述	输入/输出
matrix	KmlMgMatrixH	由KmlMgMatrixCreate函数创建的矩阵句柄	输入/输出

依赖

#include "mg.h"

示例代码请参见 **示例**。

6.5.3.4.5.4 KmlMgMatrixQuery

查询矩阵信息。

接口定义

C Interface：

void KmlMgMatrixQuery(KmlMgMatrixH matrix, KmlMgMatrixInfo* info);

参数

参数名	类型	描述	输入/输出
matrix	KmlMgMatrixH	由KmlMgMatrixCreate函数创建的矩阵句柄	输入/输出
info	KmlMgMatrixInfo*	用于保存用户查询的矩阵信息结果	输入

版权所有 © 华为技术有限公司

结构体说明

KmlMgMatrixInfo

字段名	类型	描述
fieldMask	uint64_t	配置生效的字段掩码。非0的位代表配置对应字段指定的值，0的位代表使用默认值，可配置的掩码位请参见具体字段的说明。
store	KmlMgMatrixStore	详见**KmlMgMatrixStore**结构体。 掩码位：KML_MG_MATRIX_INFO_STORE

依赖

#include "mg.h"

示例代码请参见 **示例**。

6.5.3.4.5.5 KmlMgSolverCreate

创建求解器句柄。

接口定义

C Interface：

KmlMgStatus KmlMgSolverCreate(KmlMgSolverH* pSolver, MPI_Comm comm, const KmlMgSolverOptions* options);

参数

参数名	类型	描述	输入/输出
pSolver	KmlMgSolverH*	求解器句柄，solver不可为空指针	输入/输出
comm	MPI_Comm	求解器通信子，须与求解矩阵中的通信子保持一致，否则可能会出现未知问题	输入
options	const KmlMgSolverOptions*	求解器配置信息	输入

返回值

返回值	类型	描述
KML_MG_OK	int	正常执行

版权所有 © 华为技术有限公司

返回值	类型	描述
KML_MG_ERR_INVALID_PARAM	int	参数无效
MG_ERR_UNSUPPORTED	int	暂不支持的功能
KML_MG_ERR_NO_MEMORY	int	分配内存失败
KML_MG_ERR_EXCEEDS_LIMIT	int	超过设置限制
KML_MG_ERR_INCOMPATIBLE	int	版本不兼容
KML_MG_ERR_NO_REQUIRED_FIELD	int	没设置必填字段
KML_MG_ERR_UNKNOWN_TYPE	int	未知的类型
KML_MG_ERR_THIRD_PART	int	内部调用第三方错误
KML_MG_ERR_INCONSISTENT	int	传递信息不一致
KML_MG_ERR_UNEXPECTED_CALL_ORDER	int	预期之外的调用顺序
KML_MG_ERR_MATRIX_VALUE_IS_NULL	int	矩阵值为空
KML_MG_ERR_NO_RESOURCE	int	资源未就绪

结构体说明

KmlMgSolverOptions

字段名	类型	描述
fieldMask	uint64_t	配置生效的字段掩码。非0的位代表配置对应字段指定的值，0的位代表使用默认值，可配置的掩码位请参见具体字段的说明。

 版权所有 © 华为技术有限公司

字段名	类型	描述
indexType	KmlMgIndexType	坐标类型，有如下值。 ● KML_MG_INDEX_INT32（目前只支持32位int） ● KML_MG_INDEX_INT64 掩码位：KML_MG_SOLVER_OPTIONS_INDEX_TYPE
kspValueType	KmlMgValueType	输入矩阵的数值类型，有如下值。（目前实现要求与pcCalcType字段保持一致，支持的精度组合请看下方精度组合表） ● KML_MG_VALUE_FP16 ● KML_MG_VALUE_FP32 ● KML_MG_VALUE_FP64 掩码位：KML_MG_SOLVER_OPTIONS_KSP_VALUE_TYPE
pcValueType	KmlMgValueType	预条件子（多重网格）的数值类型，有如下值。（目前只支持FP16、FP32和FP64，支持的精度组合请看下方精度组合表） ● KML_MG_VALUE_FP16 ● KML_MG_VALUE_FP32 ● KML_MG_VALUE_FP64 掩码位：KML_MG_MATRIX_INFO_STORE
pcCalcType	KmlMgValueType	预条件子（多重网格）的计算数值类型，有如下值。（目前只支持FP32和FP64，支持的精度组合请看下方精度组合表） ● KML_MG_VALUE_FP16 ● KML_MG_VALUE_FP32 ● KML_MG_VALUE_FP64 掩码位：KML_MG_MATRIX_INFO_STORE
ksp	KmlMgSolverKsp	Krylov子空间方法类型，有如下值。 ● KML_MG_SOLVER_NONE ● KML_MG_SOLVER_BICG ● KML_MG_SOLVER_CG ● KML_MG_SOLVER_CR ● KML_MG_SOLVER_GCR ● KML_MG_SOLVER_GMRES 掩码位：KML_MG_MATRIX_INFO_STORE

目前支持的精度组合有三种：

 版权所有 © 华为技术有限公司

组合	kspValueType	pcValueType	pcCalcType
1	KML_MG_VALUE_FP64	KML_MG_VALUE_FP64	KML_MG_VALUE_FP64
2	KML_MG_VALUE_FP32	KML_MG_VALUE_FP32	KML_MG_VALUE_FP32
3	KML_MG_VALUE_FP32	KML_MG_VALUE_FP16	KML_MG_VALUE_FP32

依赖

#include "mg.h"

示例代码请参见 **示例**。

6.5.3.4.5.6 KmlMgSolverSetPreconditioner

设置预条件子。

接口定义

C Interface：

KmlMgStatus KmlMgSolverSetPreconditioner(KmlMgSolverH solver, const KmlMgPreconditionerOptions* options);

参数

参数名	类型	描述	输入/输出
solver	KmlMgSolverH	由KmlMgSolverCreate创建的求解器句柄	输入/输出
options	const KmlMgPreconditionerOptions*	预条件子配置信息，即多重网格配置信息	输入

返回值

返回值	类型	描述
KML_MG_OK	int	正常执行
KML_MG_ERR_INVALID_PARAM	int	参数无效
KML_MG_ERR_UNSUPPORTED	int	暂不支持的功能

版权所有 © 华为技术有限公司

返回值	类型	描述
KML_MG_ERR_NO_MEMORY	int	分配内存失败
KML_MG_ERR_EXCEEDS_LIMIT	int	超过设置限制
KML_MG_ERR_INCOMPATIBLE	int	版本不兼容
KML_MG_ERR_NO_REQUIRED_FIELD	int	未设置必填字段
KML_MG_ERR_UNKNOWN_TYPE	int	未知的类型
KML_MG_ERR_THIRD_PART	int	内部调用第三方错误
KML_MG_ERR_INCONSISTENT	int	传递信息不一致
KML_MG_ERR_UNEXPECTED_CALL_ORDER	int	预期之外的调用顺序
KML_MG_ERR_MATRIX_VALUE_IS_NULL	int	矩阵值为空
KML_MG_ERR_NO_RESOURCE	int	资源未就绪

结构体说明

KmlMgPreconditionerOptions

字段名	字段类型	字段说明
fieldMask	uint64_t	配置生效的字段掩码。非0的位代表配置对应字段指定的值，0的位代表使用默认值，可配置的掩码位请参见具体字段的说明。
indexType	KmlMgIndexType	坐标类型，有如下值。 ● KML_MG_INDEX_INT32（目前只支持32位int） ● KML_MG_INDEX_INT64 掩码位：KML_MG_PRECONDITIONER_OPTIONS_INDEX_TYPE

 版权所有 © 华为技术有限公司

字段名	字段类型	字段说明
kspValueType	KmlMgValueType	输入矩阵的数值类型，有如下值。（目前实现要求与KmlMgSolverOptions中的kspValueType字段保持一致，支持的精度组合请看下方精度组合表） ● KML_MG_VALUE_FP16 ● KML_MG_VALUE_FP32 ● KML_MG_VALUE_FP64 掩码位：KML_MG_PRECONDITIONER_OPTIONS_KSP_VALUE_TYPE
valueType	KmlMgValueType	预条件子（多重网格）的数值类型，有如下值。（目前实现要求与KmlMgSolverOptions中的pcValueType字段保持一致） ● KML_MG_VALUE_FP16 ● KML_MG_VALUE_FP32 ● KML_MG_VALUE_FP64 掩码位：KML_MG_PRECONDITIONER_OPTIONS_VALUE_TYPE
calcType	KmlMgValueType	预条件子（多重网格）的计算数值类型，有如下值。（目前实现要求与KmlMgSolverOptions中的pcCalcType字段保持一致） ● KML_MG_VALUE_FP16 ● KML_MG_VALUE_FP32 ● KML_MG_VALUE_FP64 掩码位：KML_MG_PRECONDITIONER_OPTIONS_CALC_TYPE
numLevels	int64_t	多重网格层数 掩码位：KML_MG_PRECONDITIONER_OPTIONS_NUM_LEVEL
shiftLevelId	int64_t	混合精度向单一精度转换的层数 掩码位：KML_MG_PRECONDITIONER_OPTIONS_SHIFT_LEVEL

版权所有 © 华为技术有限公司

字段名	字段类型	字段说明
cycle	KmlMgCycleType	有如下值 ● KML_MG_CYCLE_V ● KML_MG_CYCLE_W（暂不支持） 掩码位：KML_MG_PRECONDITIONER_OPTIONS_CYCLE_TYPE
gridSweep	void*	多重网格的sweep数组，类型为坐标类型indexType，长度为2 掩码位：KML_MG_PRECONDITIONER_OPTIONS_SWEEP
xpss	void*	XPSS数组为X-Process Shrink Strides，表示X方向进程减少的步长，类型由坐标类型indexType指定，长度为网格层数numLevels 掩码位：KML_MG_PRECONDITIONER_OPTIONS_XPSS
smoother	KmlMgSmootherType*	每层网格的光滑子类型数组，长度为网格层数numLevels，有如下值 ● KML_MG_SMOOTHER_POINT_JACOBI ● KML_MG_SMOOTHER_POINT_GS ● KML_MG_SMOOTHER_LINE_JACOBI ● KML_MG_SMOOTHER_LINE_GS ● KML_MG_SMOOTHER_PLANE_ILU ● KML_MG_SMOOTHER_SHUFFLE_PLANE_ILU ● KML_MG_SMOOTHER_BLOCK_ILU ● KML_MG_SMOOTHER_ADI ● KML_MG_SMOOTHER_BLOCK_LU（暂不支持） ● KML_MG_SMOOTHER_LU（暂不支持） ● KML_MG_SMOOTHER_CUSTOM（暂不支持） 掩码位：KML_MG_PRECONDITIONER_OPTIONS_SMOOTHER

 版权所有 © 华为技术有限公司

字段名	字段类型	字段说明
weight	void*	光滑子权值，类型由计算数值类型calcType指定，长度为网格层数numLevels 掩码位：KML_MG_PRECONDITIONER_OPTIONS_WEIGHT
coarsen	KmlMgCoarsenType*	粗化网格方法，长度为网格层数numLevels，有如下值 ● KML_MG_COARSEN_FULL_XYZ ● KML_MG_COARSEN_SEMI_XY ● KML_MG_COARSEN_SEMI_XZ（暂不支持） ● KML_MG_COARSEN_SEMI_YZ（暂不支持） ● KML_MG_COARSEN_SEMI_Z（暂不支持） ● KML_MG_COARSEN_FULL_XZ（暂不支持） 掩码位：KML_MG_PRECONDITIONER_OPTIONS_COARSEN
rest	KmlMgRestInteType*	限制算子，长度为网格层数numLevels，有如下值 ● KML_MG_REST_INTE_VTX_1D1（暂不支持） ● KML_MG_REST_INTE_VTX_1D3（暂不支持） ● KML_MG_REST_INTE_VTX_2D1（暂不支持） ● KML_MG_REST_INTE_VTX_2D5（暂不支持） ● KML_MG_REST_INTE_VTX_2D9（暂不支持） ● KML_MG_REST_INTE_CELL_2D4 ● KML_MG_REST_INTE_CELL_2D16 ● KML_MG_REST_INTE_CELL_3D8 ● KML_MG_REST_INTE_CELL_3D64 掩码位：KML_MG_PRECONDITIONER_OPTIONS_RESTRICT

版权所有 © 华为技术有限公司

字段名	字段类型	字段说明
interp	KmlMgRestInteType*	插值算子，长度为网格层数numLevels，有如下值 • KML_MG_REST_INTE_VTX_1D1（暂不支持） • KML_MG_REST_INTE_VTX_1D3（暂不支持） • KML_MG_REST_INTE_VTX_2D1（暂不支持） • KML_MG_REST_INTE_VTX_2D5（暂不支持） • KML_MG_REST_INTE_VTX_2D9（暂不支持） • KML_MG_REST_INTE_CELL_2D4 • KML_MG_REST_INTE_CELL_2D16 • KML_MG_REST_INTE_CELL_3D8 • KML_MG_REST_INTE_CELL_3D64 掩码位：KML_MG_PRECONDITIONER_OPTIONS_PROLONG
alpha	void*	Galerkin算子调整系数，类型由计算数值类型calcType指定，长度为网格层数numLevels 掩码位：KML_MG_PRECONDITIONER_OPTIONS_ALPHA
control	bool*	控制粗化后的网格的stencil是否与上一层保持一致，若为true保持一致，若为false则可能膨胀，长度为网格层数numLevels。 掩码位：KML_MG_PRECONDITIONER_OPTIONS_CONTROL

说明

使用暂不支持的参数会导致程序异常退出。

依赖

#include "mg.h"

示例代码请参见 **示例**。

 版权所有 © 华为技术有限公司

6.5.3.4.5.7 KmlMgSolverSetPreconditionerMatrix

设置预条件子矩阵，当前实现预条件子须与ksp阶段的矩阵相同。

接口定义

C Interface：

KmlMgStatus KmlMgSolverSetPreconditionerMatrix(KmlMgSolverH solver, KmlMgMatrixH A);

参数

参数名	类型	描述	输入/输出
solver	KmlMgSolverH	由KmlMgSolverCreate创建的求解器句柄	输入/输出
A	KmlMgMatrixH	由KmlMgMatrixCreate函数创建的矩阵句柄	输入

返回值

返回值	类型	描述
KML_MG_OK	int	正常执行
MG_ERR_INVALID_PARAM	int	参数无效
KML_MG_ERR_UNSUPPORTED	int	暂不支持的功能
KML_MG_ERR_NO_MEMORY	int	分配内存失败
KML_MG_ERR_EXCEEDS_LIMIT	int	超过设置限制
KML_MG_ERR_INCOMPATIBLE	int	版本不兼容
KML_MG_ERR_NO_REQUIRED_FIELD	int	没设置必填字段
KML_MG_ERR_UNKNOWN_TYPE	int	未知的类型
KML_MG_ERR_THIRD_PART	int	内部调用第三方错误
KML_MG_ERR_INCONSISTENT	int	传递信息不一致

版权所有 © 华为技术有限公司

返回值	类型	描述
KML_MG_ERR_UNEXPECTED_CALL_ORDER	int	预期之外的调用顺序
KML_MG_ERR_MATRIX_VALUE_IS_NULL	int	矩阵值为空
KML_MG_ERR_NO_RESOURCE	int	资源未就绪

依赖

#include "mg.h"

示例代码请参见 **示例**。

6.5.3.4.5.8 KmlMgSolverPrepare

传入矩阵数值，调用前必须保证入参矩阵使用KmlMgMatrixSetValues函数填入了有效数值。

接口定义

C Interface：

KmlMgStatus KmlMgSolverPrepare(KmlMgSolverH solver, KmlMgMatrixH A);

参数

参数名	类型	描述	输入/输出
solver	KmlMgSolverH	由KmlMgSolverCreate创建的求解器句柄	输入/输出
A	KmlMgMatrixH	由KmlMgMatrixCreate函数创建的矩阵句柄	输入/输出

返回值

返回值	类型	描述
KML_MG_OK	int	正常执行
KML_MG_ERR_INVALID_PARAM	int	参数无效
KML_MG_ERR_UNSUPPORTED	int	暂不支持的功能

 版权所有 © 华为技术有限公司

返回值	类型	描述
KML_MG_ERR_NO_MEMORY	int	分配内存失败
KML_MG_ERR_EXCEEDS_LIMIT	int	超过设置限制
KML_MG_ERR_INCOMPATIBLE	int	版本不兼容
KML_MG_ERR_NO_REQUIRED_FIELD	int	没设置必填字段
KML_MG_ERR_UNKNOWN_TYPE	int	未知的类型
KML_MG_ERR_THIRD_PART	int	内部调用第三方错误
KML_MG_ERR_INCONSISTENT	int	传递信息不一致
KML_MG_ERR_UNEXPECTED_CALL_ORDER	int	预期之外的调用顺序
KML_MG_ERR_MATRIX_VALUE_IS_NULL	int	矩阵值为空
KML_MG_ERR_NO_RESOURCE	int	资源未就绪

依赖

#include "mg.h"

示例代码请参见 **示例**。

6.5.3.4.5.9 KmlMgSolverApply

进行求解回代。

接口定义

C Interface：

KmlMgStatus KmlMgSolverApply(KmlMgSolverH solver, KmlMgMatrixH B, KmlMgMatrixH X, KmlMgApplyOptions* options);

版权所有 © 华为技术有限公司

参数

参数名	类型	描述	输入/输出
solver	KmlMgSolverH	由KmlMgSolverCreate创建的求解器句柄	输入/输出
B	KmlMgMatrixH	由KmlMgMatrixCreate函数创建的矩阵句柄，表示右端向量	输入
X	KmlMgMatrixH	由KmlMgMatrixCreate函数创建的矩阵句柄，表示解空间，初始输入值可作为初次迭代初值	输入/输出
options	KmlMgApplyOptions&	回代配置信息	输入

返回值

返回值	类型	描述
KML_MG_OK	int	正常执行
KML_MG_ERR_INVALID_PARAM	int	参数无效
KML_MG_ERR_UNSUPPORTED	int	暂不支持的功能
KML_MG_ERR_NO_MEMORY	int	分配内存失败
KML_MG_ERR_EXCEEDS_LIMIT	int	超过设置限制
KML_MG_ERR_INCOMPATIBLE	int	版本不兼容
KML_MG_ERR_NO_REQUIRED_FIELD	int	没设置必填字段
KML_MG_ERR_UNKNOWN_TYPE	int	未知的类型
KML_MG_ERR_THIRD_PART	int	内部调用第三方错误
KML_MG_ERR_INCONSISTENT	int	传递信息不一致
KML_MG_ERR_UNEXPECTED_CALL_ORDER	int	预期之外的调用顺序

 版权所有 © 华为技术有限公司

返回值	类型	描述
KML_MG_ERR_MATRIX_VALUE_IS_NULL	int	矩阵值为空
KML_MG_ERR_NO_RESOURCE	int	资源未就绪

结构体说明

KmlMgApplyOptions

字段名	字段类型	字段说明
fieldMask	uint64_t	配置生效的字段掩码。非0的位代表配置对应字段指定的值，0的位代表使用默认值，可配置的掩码位请参见具体字段的说明。
useZeroGuess	bool	是否以X=0为初值开始迭代，若为true则迭代初值为0，若为false则以参数X传入时的值为初值。 掩码位：KML_MG_APPLY_OPTIONS_USE_ZERO_GUESS
toleranceType	KmlMgToleranceType	迭代收敛的阈值类型，有如下值 • KML_MG_TOLERANCE_ABS • KML_MG_TOLERANCE_REL 掩码位：KML_MG_APPLY_OPTIONS_TOLERANCE_TYPE
tolerance	double	迭代收敛的阈值，小于该阈值则停止迭代 掩码位：KML_MG_APPLY_OPTIONS_TOLERANCE
maxIteration	int64_t	最大迭代次数，迭代次数达到阈值则停止迭代 掩码位：KML_MG_APPLY_OPTIONS_MAX_ITERATION

版权所有 © 华为技术有限公司

字段名	字段类型	字段说明
restartStep	int64_t	重启步数，仅当KmlMgSolverOptions.ksp为KML_MG_SOLVER_GCR或KML_MG_SOLVER_GMRES时生效。 掩码位：KML_MG_APPLY_OPTIONS_RESTART_STEP
solveResi	bool	是否转化成求解残差方程，精度优化选项。在给定初值X0时，若为true，则以将方程A*X=B转化为A*(X-X0)=B-A*X0。当X0非常接近真实解时仅当KmlMgSolverOptions.ksp为KML_MG_SOLVER_G时生效。 掩码位：KML_MG_APPLY_OPTIONS_SOLVE_RESI

依赖

#include "mg.h"

示例代码请参见 **示例**。

6.5.3.4.5.10 KmlMgSolverDestroy

销毁求解器句柄并释放资源。

接口定义

C Interface：

KmlMgStatus KmlMgSolverDestroy(KmlMgSolverH solver);

参数

参数名	类型	描述	输入/输出
solver	KmlMgSolverH	由KmlMgSolverCreate创建的求解器句柄。	输入/输出

版权所有 © 华为技术有限公司

返回值

返回值	类型	描述
KML_MG_OK	int	正常执行
MG_ERR_INVALID_PARAM	int	参数无效
KML_MG_ERR_UNSUPPORTED	int	暂不支持的功能
KML_MG_ERR_NO_MEMORY	int	分配内存失败
KML_MG_ERR_EXCEEDS_LIMIT	int	超过设置限制
KML_MG_ERR_INCOMPATIBLE	int	版本不兼容
KML_MG_ERR_NO_REQUIRED_FIELD	int	没设置必填字段
KML_MG_ERR_UNKNOWN_TYPE	int	未知的类型
KML_MG_ERR_THIRD_PART	int	内部调用第三方错误
KML_MG_ERR_INCONSISTENT	int	传递信息不一致
KML_MG_ERR_UNEXPECTED_CALL_ORDER	int	预期之外的调用顺序
KML_MG_ERR_MATRIX_VALUE_IS_NULL	int	矩阵值为空
KML_MG_ERR_NO_RESOURCE	int	资源未就绪

依赖

#include "mg.h"

示例代码请参见 **示例**。

6.5.3.4.5.11 KmlMgSolverQuery

查询求解器配置信息。

接口定义

C Interface：

 版权所有 © 华为技术有限公司

KmlMgStatus KmlMgSolverQuery(KmlMgSolverH solver, KmlMgSolverInfo* info);

参数

参数名	类型	描述	输入/输出
solver	KmlMgSolverH	由KmlMgSolverCreate创建的求解器句柄	输入/输出
info	KmlMgSolverInfo*	用于保存用户查询的求解器信息结果	输出

返回值

返回值	类型	描述
KML_MG_OK	int	正常执行
KML_MG_ERR_INVALID_PARAM	int	参数无效
KML_MG_ERR_UNSUPPORTED	int	暂不支持的功能
KML_MG_ERR_NO_MEMORY	int	分配内存失败
KML_MG_ERR_EXCEEDS_LIMIT	int	超过设置限制
KML_MG_ERR_INCOMPATIBLE	int	版本不兼容
KML_MG_ERR_NO_REQUIRED_FIELD	int	没设置必填字段
KML_MG_ERR_UNKNOWN_TYPE	int	未知的类型
KML_MG_ERR_THIRD_PART	int	内部调用第三方错误
KML_MG_ERR_INCONSISTENT	int	传递信息不一致
KML_MG_ERR_UNEXPECTED_CALL_ORDER	int	预期之外的调用顺序
KML_MG_ERR_MATRIX_VALUE_IS_NULL	int	矩阵值为空
KML_MG_ERR_NO_RESOURCE	int	资源未就绪

 版权所有 © 华为技术有限公司

结构体说明

KmlMgSolverInfo

字段名	字段类型	字段说明
fieldMask	uint64_t	配置生效的字段掩码。非0的位代表配置对应字段指定的值，0的位代表使用默认值，可配置的掩码位请参见具体字段的说明。
ksp	KmlMgSolverKsp	KmlMgSolverOptions的ksp字段，详细说明见**KmlMgSolverOptions** 掩码位：KML_MG_SOLVER_INFO_KSP_TYPE
useZeroGuess	bool	KmlMgApplyOptions的useZeroGuess字段，详细说明见**KmlMgApplyOptions** 掩码位：KML_MG_SOLVER_INFO_USE_ZERO_GUESS
toleranceType	KmlMgToleranceType	KmlMgApplyOptions的toleranceType字段，详细说明见**KmlMgApplyOptions** 掩码位：KML_MG_SOLVER_INFO_TOLERANCE_TYPE
tolerance	double	KmlMgApplyOptions的tolerance字段，详细说明见**KmlMgApplyOptions** 掩码位：KML_MG_SOLVER_INFO_TOLERANCE
maxIteration	int64_t	KmlMgApplyOptions的maxIteration字段，详细说明见**KmlMgApplyOptions** 掩码位：KML_MG_SOLVER_INFO_MAX_ITERATION

版权所有 © 华为技术有限公司

字段名	字段类型	字段说明
restartStep	int64_t	KmlMgApplyOptions的restartStep字段，详细说明见**KmlMgApplyOptions**，仅当KmlMgSolverOptions.ksp为KML_MG_SOLVER_GCR或KML_MG_SOLVER_GMRES时生效 掩码位：KML_MG_SOLVER_INFO_RESTART_STEP
solveResi	bool	KmlMgApplyOptions的solveResi字段，详细说明见**KmlMgApplyOptions**，仅当KmlMgSolverOptions.ksp为KML_MG_SOLVER_GCR时生效 掩码位：KML_MG_SOLVER_INFO_SOLVE_RESI

依赖

#include "mg.h"

示例代码请参见 **示例**。

6.5.3.4.5.12 KmlMgStatusStr

查询MG接口不同返回值的含义。

接口定义

C Interface：

const char *KmlMgStatusStr(KmlMgStatus status);

参数

参数名	类型	描述	输入/输出
status	KmlMgStatus	MG接口返回的不同状态值	输入

版权所有 © 华为技术有限公司

返回值

返回值	类型	描述
KML_MG_OK	int	正常执行
KML_MG_ERR_INVALID_PARAM	int	参数无效
KML_MG_ERR_UNSUPPORTED	int	暂不支持的功能
KML_MG_ERR_NO_MEMORY	int	分配内存失败
KML_MG_ERR_EXCEEDS_LIMIT	int	超过设置限制
KML_MG_ERR_INCOMPATIBLE	int	版本不兼容
KML_MG_ERR_NO_REQUIRED_FIELD	int	没设置必填字段
KML_MG_ERR_UNKNOWN_TYPE	int	未知的类型
KML_MG_ERR_THIRD_PART	int	内部调用第三方错误
KML_MG_ERR_INCONSISTENT	int	传递信息不一致
KML_MG_ERR_UNEXPECTED_CALL_ORDER	int	预期之外的调用顺序
KML_MG_ERR_MATRIX_VALUE_IS_NULL	int	矩阵值为空
KML_MG_ERR_NO_RESOURCE	int	资源未就绪

依赖

#include "mg_status.h"

示例代码请参见 **示例**。

6.5.3.4.5.13 示例

C Interface:

```
#include "mg.h"
#include <stdio.h>
#include <stdlib.h>
#include <stdbool.h>
#include "mpi.h"
```

 版权所有 © 华为技术有限公司

```
int main(int argc, char **argv)
{
    MPI_Init(&argc, &argv);
    MPI_Comm comm = MPI_COMM_WORLD;
    int rankSize;
    MPI_Comm_size(comm, &rankSize);
    int pId;
    MPI_Comm_rank(comm, &pId);
    // Init proc size
    int px = 1;
    int py = 1;
    int pz = 1;
    // Init matrix size
    int gx = 400;
    int gy = 200;
    int gz = 50;
    int lx = gx / px;
    int ly = gy / py;
    int lz = gz;
    int xbeg = (pId % px) * lx;
    int xend = xbeg + lx;
    int ybeg = (pId / px) * ly;
    int yend = ybeg + ly;
    int zbeg = 0;
    int zend = lz;
    int totLen = (lx + 2) * (ly + 2) * (lz + 2);
    // Init matrix A
    KmlMgMatrixH A = NULL;
    float *aValue = (float *)malloc(19 * totLen * sizeof(float));
    {
        KmlMgMatrixStore store;
        store.comm = comm;
        store.globalx = gx;
        store.globaly = gy;
        store.globalz = gz;
        store.procx = px;
        store.procy = py;
        store.procz = pz;
        store.xbeg = xbeg;
        store.xend = xend;
        store.ybeg = ybeg;
        store.yend = yend;
        store.zbeg = zbeg;
        store.zend = zend;
        store.xhalo = 1;
        store.yhalo = 1;
        store.zhalo = 1;
        store.crossPolar = 1;
        store.type = KML_MG_MATRIX_STORE_SPARSE_HALO;
        store.sparse.values = NULL;
        store.sparse.type = KML_MG_SPARSE_AOS;
        store.sparse.stencil = KML_MG_STENCIL_3D19;
        KmlMgMatrixOptions options;
        options.fieldMask = KML_MG_MATRIX_OPTIONS_INDEX_TYPE |
KML_MG_MATRIX_OPTIONS_VALUE_TYPE;
        options.indexType = KML_MG_INDEX_INT32;
        options.valueType = KML_MG_VALUE_FP32;
        if (A != NULL) {
            KmlMgMatrixDestroy(A);
        }
        if (KmlMgMatrixCreate(&A, &store, &options) != KML_MG_OK) {
            printf("Create A error!\n");
        };
        // Set A values, order is Y-X-Z
        for (int i = 1; i < lx + 1; i++) {
            for (int j = 1; j < ly + 1; j++) {
                for (int k = 1; k < lz + 1; k++) {
                    for (int l = 0; l < 19; l++) {
                        aValue[j * (lx + 2) * (lz + 2) * 19 + i * (lz + 2) * 19 + k * 19 + l] = 0.5;
```

 版权所有 © 华为技术有限公司

```
                }
                aValue[j * (lx + 2) * (lz + 2) * 19 + i * (lz + 2) * 19 + k * 19 + 9] *= 100;
            }
        }
    }
    if (KmlMgMatrixSetValues(A, (void *)aValue) != KML_MG_OK) {
        printf("SetValues of A error!\n");
    };
}
// Init B & X
KmlMgMatrixH B = NULL;
float *bValue = (float *)malloc(totLen * sizeof(float));
KmlMgMatrixH X = NULL;
float *xValue = (float *)malloc(totLen * sizeof(float));
{
    KmlMgMatrixStore store;
    store.comm = comm;
    store.globalx = gx;
    store.globaly = gy;
    store.globalz = gz;
    store.procx = px;
    store.procy = py;
    store.procz = pz;
    store.xbeg = xbeg;
    store.xend = xend;
    store.ybeg = ybeg;
    store.yend = yend;
    store.zbeg = zbeg;
    store.zend = zend;
    store.xhalo = 1;
    store.yhalo = 1;
    store.zhalo = 1;
    store.crossPolar = 1;
    store.type = KML_MG_MATRIX_STORE_DENSE_COL_MAJOR_HALO;
    store.dense.values = NULL;
    store.dense.numDim = 1;
    store.dense.is2D = 0;
    store.dense.needCorner = 1;
    KmlMgMatrixOptions options;
    options.fieldMask = KML_MG_MATRIX_OPTIONS_INDEX_TYPE |
KML_MG_MATRIX_OPTIONS_VALUE_TYPE;
    options.indexType = KML_MG_INDEX_INT32;
    options.valueType = KML_MG_VALUE_FP32;
    // Create B
    if (KmlMgMatrixCreate(&B, &store, &options) != KML_MG_OK) {
        printf("Create B error!\n");
    };
    // Set B values
    for (int i = 0; i < totLen; i++) {
        bValue[i] = 1.1;
    }
    if (KmlMgMatrixSetValues(B, (void *)bValue) != KML_MG_OK) {
        printf("SetValues of B error!\n");
    };
    // Create X
    if (KmlMgMatrixCreate(&X, &store, &options) != KML_MG_OK) {
        printf("Create X error!\n");
    };
    // Set X values
    for (int i = 0; i < totLen; i++) {
        xValue[i] = 0.0;
    }
    if (KmlMgMatrixSetValues(X, (void *)xValue) != KML_MG_OK) {
        printf("SetValues of X error!\n");
    };
}
// Init solver
KmlMgSolverH solver = NULL;
KmlMgPreconditionerOptions preOpt;  // need malloc and free
```

 版权所有 © 华为技术有限公司

```
  {
      KmlMgSolverOptions solOpt;
      solOpt.fieldMask = KML_MG_SOLVER_OPTIONS_INDEX_TYPE |
KML_MG_SOLVER_OPTIONS_KSP_VALUE_TYPE |
                         KML_MG_SOLVER_OPTIONS_PC_VALUE_TYPE |
KML_MG_SOLVER_OPTIONS_PC_CALC_TYPE |
                         KML_MG_SOLVER_OPTIONS_KSP;
      solOpt.indexType = KML_MG_INDEX_INT32;
      solOpt.kspValueType = KML_MG_VALUE_FP32;
      solOpt.pcValueType = KML_MG_VALUE_FP32;
      solOpt.pcCalcType = KML_MG_VALUE_FP32;
      solOpt.ksp = KML_MG_SOLVER_GCR;
      if (solver != NULL) {
          KmlMgSolverDestroy(solver);
      }
      if (KmlMgSolverCreate(&solver, comm, &solOpt) != KML_MG_OK) {
          printf("solver Create error!\n");
      };
      preOpt.fieldMask = KML_MG_PRECONDITIONER_OPTIONS_INDEX_TYPE |
KML_MG_PRECONDITIONER_OPTIONS_KSP_VALUE_TYPE |
                         KML_MG_PRECONDITIONER_OPTIONS_VALUE_TYPE |
KML_MG_PRECONDITIONER_OPTIONS_CALC_TYPE |
                         KML_MG_PRECONDITIONER_OPTIONS_NUM_LEVEL |
KML_MG_PRECONDITIONER_OPTIONS_SHIFT_LEVEL |
                         KML_MG_PRECONDITIONER_OPTIONS_CYCLE_TYPE |
KML_MG_PRECONDITIONER_OPTIONS_SWEEP |
                         KML_MG_PRECONDITIONER_OPTIONS_XPSS |
KML_MG_PRECONDITIONER_OPTIONS_SMOOTHER |
                         KML_MG_PRECONDITIONER_OPTIONS_WEIGHT |
KML_MG_PRECONDITIONER_OPTIONS_COARSEN |
                         KML_MG_PRECONDITIONER_OPTIONS_RESTRICT |
KML_MG_PRECONDITIONER_OPTIONS_PROLONG |
                         KML_MG_PRECONDITIONER_OPTIONS_ALPHA |
KML_MG_PRECONDITIONER_OPTIONS_CONTROL;
      preOpt.indexType = KML_MG_INDEX_INT32;
      preOpt.kspValueType = KML_MG_VALUE_FP32;
      preOpt.valueType = KML_MG_VALUE_FP32;
      preOpt.calcType = KML_MG_VALUE_FP32;
      preOpt.numLevels = 4;
      preOpt.shiftLevelId = 1;
      preOpt.cycle = KML_MG_CYCLE_V;
      int *gridSweep = (int *)malloc(2 * sizeof(int));
      gridSweep[0] = 1;
      gridSweep[1] = 0;
      preOpt.gridSweep = gridSweep;
      int *xpss = (int *)malloc(4 * sizeof(int));
      xpss[0] = 1;
      xpss[1] = 1;
      xpss[2] = 1;
      xpss[3] = 1;
      preOpt.xpss = xpss;
      KmlMgSmootherType *smoother = (KmlMgSmootherType *)malloc(4 * sizeof(KmlMgSmootherType));
      smoother[0] = KML_MG_SMOOTHER_PLANE_ILU;
      smoother[1] = KML_MG_SMOOTHER_LINE_GS;
      smoother[2] = KML_MG_SMOOTHER_LINE_GS;
      smoother[3] = KML_MG_SMOOTHER_LINE_GS;
      preOpt.smoother = smoother;
      float *weight = (float *)malloc(4 * sizeof(float));
      weight[0] = 1.0;
      weight[1] = 1.0;
      weight[2] = 1.0;
      weight[3] = 1.0;
      preOpt.weight = weight;
      KmlMgCoarsenType *coarsen = (KmlMgCoarsenType *)malloc(4 * sizeof(KmlMgCoarsenType));
      coarsen[0] = KML_MG_COARSEN_SEMI_XY;
      coarsen[1] = KML_MG_COARSEN_SEMI_XY;
      coarsen[2] = KML_MG_COARSEN_SEMI_XY;
      coarsen[3] = KML_MG_COARSEN_SEMI_XY;
      preOpt.coarsen = coarsen;
```

 版权所有 © 华为技术有限公司

```
    KmlMgRestInteType *rest = (KmlMgRestInteType *)malloc(4 * sizeof(KmlMgRestInteType));
    rest[0] = KML_MG_REST_INTE_CELL_2D16;
    rest[1] = KML_MG_REST_INTE_CELL_2D4;
    rest[2] = KML_MG_REST_INTE_CELL_2D4;
    rest[3] = KML_MG_REST_INTE_CELL_2D4;
    preOpt.rest = rest;
    KmlMgRestInteType *interp = (KmlMgRestInteType *)malloc(4 * sizeof(KmlMgRestInteType));
    interp[0] = KML_MG_REST_INTE_CELL_2D16;
    interp[1] = KML_MG_REST_INTE_CELL_2D16;
    interp[2] = KML_MG_REST_INTE_CELL_2D16;
    interp[3] = KML_MG_REST_INTE_CELL_2D16;
    preOpt.interp = interp;
    float *alpha = (float *)malloc(4 * sizeof(float));
    alpha[0] = 1.0;
    alpha[1] = 1.0;
    alpha[2] = 1.0;
    alpha[3] = 1.0;
    preOpt.alpha = alpha;
    int *control = (int *)malloc(4 * sizeof(int));
    control[0] = 0;
    control[1] = 1;
    control[2] = 1;
    control[3] = 1;
    preOpt.control = control;
    if (KmlMgSolverSetPreconditioner(solver, &preOpt) != KML_MG_OK) {
        printf("solver SetPreconditioner error!\n");
    }
    printf("SetPreconditioner done!\n");
    if (KmlMgSolverSetPreconditionerMatrix(solver, A) != KML_MG_OK) {
        printf("solver SetPreconditionerMatrix error!\n");
    }
    printf("SetPreconditionerMatrix done!\n");
    if (KmlMgSolverPrepare(solver, A) != KML_MG_OK) {
        printf("solver Prepare error!\n");
    }
    printf("Prepare done!\n");
  }
  // Apply
  {
    KmlMgApplyOptions opt;
    opt.fieldMask = KML_MG_APPLY_OPTIONS_USE_ZERO_GUESS |
KML_MG_APPLY_OPTIONS_TOLERANCE_TYPE |
                    KML_MG_APPLY_OPTIONS_TOLERANCE | KML_MG_APPLY_OPTIONS_MAX_ITERATION |
                    KML_MG_APPLY_OPTIONS_RESTART_STEP;
    opt.useZeroGuess = 1;
    opt.toleranceType = KML_MG_TOLERANCE_ABS;
    opt.tolerance = 1e-6;
    opt.maxIteration = 200;
    opt.restartStep = 10;
    KmlMgStatus sta = KmlMgSolverApply(solver, B, X, &opt);
    if (sta != KML_MG_OK) {
        printf("solver Apply error!\n");
    }
    const char * str = KmlMgStatusStr(sta);
    printf("%s\n", str);
    printf("Apply done\n");
  }
  KmlMgMatrixInfo info;
  info.fieldMask = KML_MG_MATRIX_INFO_STORE;
  KmlMgMatrixQuery(X, &info);
  // Clear
  {
    if (A != NULL) {
        KmlMgMatrixDestroy(A);
    }
    if (aValue != NULL) {
        free((void *)aValue);
    }
    if (B != NULL) {
```

 版权所有 © 华为技术有限公司

```
            KmlMgMatrixDestroy(B);
        }
        if (bValue != NULL) {
            free((void *)bValue);
        }
        if (X != NULL) {
            KmlMgMatrixDestroy(X);
        }
        if (xValue != NULL) {
            free((void *)xValue);
        }
        if (preOpt.gridSweep != NULL) {
            free((void *)preOpt.gridSweep);
        }
        if (preOpt.xpss != NULL) {
            free((void *)preOpt.xpss);
        }
        if (preOpt.smoother != NULL) {
            free((void *)preOpt.smoother);
        }
        if (preOpt.weight != NULL) {
            free((void *)preOpt.weight);
        }
        if (preOpt.coarsen != NULL) {
            free((void *)preOpt.coarsen);
        }
        if (preOpt.rest != NULL) {
            free((void *)preOpt.rest);
        }
        if (preOpt.interp != NULL) {
            free((void *)preOpt.interp);
        }
        if (preOpt.alpha != NULL) {
            free((void *)preOpt.alpha);
        }
        if (preOpt.control != NULL) {
            free((void *)preOpt.control);
        }
    }
    MPI_Finalize();
    return 0;
}
```

6.5.3.4.6 TYAMG

6.5.3.4.6.1 KmlAmgMatrixCreate

创建矩阵句柄。

说明

创建矩阵时是浅拷贝，用户需要保证KmlAmgMatrixCreate创建的矩阵在求解结束之前不释放参数store指定的内存。

接口定义

C Interface：

KmlAmgStatus KmlAmgMatrixCreate(KmlAmgMatrixH* matrix, KmlAmgMatrixStore* store, const KmlAmgMatrixOptions* options);

 版权所有 © 华为技术有限公司

参数

参数名	类型	描述	输入/输出
matrix	KmlAmgMatrixH*	矩阵句柄，matrix不可为空指针	输入/输出
store	KmlAmgMatrixStore*	矩阵的格式与数据信息结构体	输入
options	KmlAmgMatrixOptions*	矩阵配置选项	输入

返回值

返回值	类型	描述
KMLAMG_SUCCESS	int	运行成功
KMLAMG_ERR_INVALID_PARAM	int	参数无效
KMLAMG_ERR_UNSUPPORTED	int	功能暂不支持
KMLAMG_ERR_NULL_POINTER	int	空指针
KMLAMG_ERR_NO_REQUIRED_FIELD	int	没有设置必需字段
KMLAMG_ERR_NO_MEMORY	int	内存分配失败
KMLAMG_ERR_UNKNOWN_TYPE	int	未知的类型
KMLAMG_ERR_UNEXPECTED_CALL_ORDER	int	预期之外的调用顺序
KMLAMG_ERR_NO_RESOURCE	int	资源未就绪
KMLAMG_ERR_INCONSISTENT	int	不同阶段传递的信息不一致
KMLAMG_ERR_UNKNOWN_ERROR	int	未知错误

结构体说明

KmlAmgMatrixStore结构体描述：

 版权所有 © 华为技术有限公司

表 6-57 KmlAmgMatrixStore 结构体

字段名	类型	描述
globalNRows	int64_t	全局矩阵行数
globalNCols	int64_t	全局矩阵列数
localNRows	int64_t	本进程矩阵行数
localNCols	int64_t	本进程矩阵列数
type	KmlAmgMatrixStoreType	矩阵的类型，有如下值： ● KMLAMG_MATRIX_STORE_CSR表示CSR格式的矩阵 ● KMLAMG_MATRIX_STORE_CSC表示CSC格式的矩阵（暂不支持） ● KMLAMG_MATRIX_STORE_COO表示COO格式的矩阵（暂不支持） ● KMLAMG_MATRIX_STORE_DENSE_ROW_MAJOR表示行主序矩阵（目前只支持列数为1，即向量） ● KMLAMG_MATRIX_STORE_DENSE_COL_MAJOR表示列主序矩阵（暂不支持）
partitionType	KmlAmgMatrixPartitionType	矩阵切分的类型，有如下值： KMLAMG_MATRIX_PARTITION_ROW表示按行切分 KMLAMG_MATRIX_PARTITION_COL表示按列切分（暂不支持） KMLAMG_MATRIX_PARTITION_NZ表示按非零元切分（暂不支持）
isAvgPart	int8_t	矩阵是否是平均切分，0表示不平均，非0值表示平均切分
localBegin	int64_t	当前进程矩阵的起始行/列索引
dense	KmlAmgMatrixStoreDense	type为KMLAMG_MATRIX_STORE_DENSE_ROW_MAJOR或KMLAMG_MATRIX_STORE_DENSE_COL_MAJOR时，该字段有效，存储矩阵的值
csr	KmlAmgMatrixStoreCSR	type为KMLAMG_MATRIX_STORE_CSR时该字段生效，存储CSR矩阵的三元组信息
csc	KmlAmgMatrixStoreCSC	type为KMLAMG_MATRIX_STORE_CSC时该字段生效，存储CSC矩阵的三元组信息
coo	KmlAmgMatrixStoreCOO	type为KMLAMG_MATRIX_STORE_COO时该字段生效，存储COO矩阵的三元组信息

 版权所有 © 华为技术有限公司

KmlAmgMatrixOptions结构体描述：

字段名	类型	描述
fieldMask	uint64_t	配置生效的字段掩码。非0的位代表配置对应字段指定的值，0的位代表使用默认值，可配置的掩码位见具体字段的说明。
valueType	KmlAmgValueType	矩阵数值类型，有如下值： ● KMLAMG_VALUE_FP32表示float ● KMLAMG_VALUE_FP64表示double 掩码位：KMLAMG_MATRIX_OPTIONS_VALUE_TYPE
indexType	KmlAmgIndexType	矩阵索引类型： ● KMLAMG_INDEX_INT32表示int32_t ● KMLAMG_INDEX_INT64表示int64_t（暂不支持） 掩码位：KMLAMG_MATRIX_OPTIONS_INDEX_TYPE
comm	MPI_Comm	通信子 掩码位：KMLAMG_MATRIX_OPTIONS_COMM
numThreads	int32_t	线程数 掩码位：KMLAMG_MATRIX_OPTIONS_NUM_THREADS

说明

使用暂不支持的参数会导致程序异常退出。

依赖

#include "amg4c.h"

6.5.3.4.6.2 KmlAmgMatrixDestroy

销毁矩阵句柄并释放资源。

接口定义

C Interface：KmlAmgStatus KmlAmgMatrixDestroy(KmlAmgMatrixH matrix);

版权所有 © 华为技术有限公司

参数

参数名	类型	描述	输入/输出
matrix	KmlAmgMatrixH	矩阵句柄	输入/输出

返回值

返回值	类型	描述
KMLAMG_SUCCESS	int	运行成功
KMLAMG_ERR_INVALID_PARAM	int	参数无效
KMLAMG_ERR_UNSUPPORTED	int	功能暂不支持
KMLAMG_ERR_NULL_POINTER	int	空指针
KMLAMG_ERR_NO_REQUIRED_FIELD	int	没有设置必需字段
KMLAMG_ERR_NO_MEMORY	int	内存分配失败
KMLAMG_ERR_UNKNOWN_TYPE	int	未知的类型
KMLAMG_ERR_UNEXPECTED_CALL_ORDER	int	预期之外的调用顺序
KMLAMG_ERR_NO_RESOURCE	int	资源未就绪
KMLAMG_ERR_INCONSISTENT	int	不同阶段传递的信息不一致
KMLAMG_ERR_UNKNOWN_ERROR	int	未知错误

依赖

#include "amg4c.h"

6.5.3.4.6.3 KmlAmgMatrixSetValues

对矩阵进行赋值。

版权所有 © 华为技术有限公司

接口定义

C Interface： KmlAmgStatus KmlAmgMatrixSetValues(KmlAmgMatrixH matrix, void* values);

参数

参数名	类型	描述	输入/输出
matrix	KmlAmgMatrixH	矩阵句柄	输入/输出
values	void*	矩阵数值数组	输入

返回值

返回值	类型	描述
KMLAMG_SUCCESS	int	运行成功
KMLAMG_ERR_INVALID_PARAM	int	参数无效
KMLAMG_ERR_UNSUPPORTED	int	功能暂不支持
KMLAMG_ERR_NULL_POINTER	int	空指针
KMLAMG_ERR_NO_REQUIRED_FIELD	int	没有设置必需字段
KMLAMG_ERR_NO_MEMORY	int	内存分配失败
KMLAMG_ERR_UNKNOWN_TYPE	int	未知的类型
KMLAMG_ERR_UNEXPECTED_CALL_ORDER	int	预期之外的调用顺序
KMLAMG_ERR_NO_RESOURCE	int	资源未就绪
KMLAMG_ERR_INCONSISTENT	int	不同阶段传递的信息不一致
KMLAMG_ERR_UNKNOWN_ERROR	int	未知错误

依赖

#include "amg4c.h"

版权所有 © 华为技术有限公司

6.5.3.4.6.4 KmlAmgPrecondCreate

创建预条件子句柄。

接口定义

C Interface：KmlAmgStatus KmlAmgPrecondCreate(KmlAmgPrecondH* precond, const KmlAmgPrecondOptions* options);

参数

参数名	类型	描述	输入/输出
precond	KmlAmgPrecondH*	预条件子句柄，precond不可为空指针	输入/输出
options	const KmlAmgPrecondOptions*	预条件子配置选项	输入

返回值

返回值	类型	描述
KMLAMG_SUCCESS	int	运行成功
KMLAMG_ERR_INVALID_PARAM	int	参数无效
KMLAMG_ERR_UNSUPPORTED	int	功能暂不支持
KMLAMG_ERR_NULL_POINTER	int	空指针
KMLAMG_ERR_NO_REQUIRED_FIELD	int	没有设置必需字段
KMLAMG_ERR_NO_MEMORY	int	内存分配失败
KMLAMG_ERR_UNKNOWN_TYPE	int	未知的类型
KMLAMG_ERR_UNEXPECTED_CALL_ORDER	int	预期之外的调用顺序
KMLAMG_ERR_NO_RESOURCE	int	资源未就绪
KMLAMG_ERR_INCONSISTENT	int	不同阶段传递的信息不一致

版权所有 © 华为技术有限公司

返回值	类型	描述
KMLAMG_ERR_UNKNOWN_ERROR	int	未知错误

结构体说明

KmlAmgPrecondOptions结构体描述：

字段名	类型	描述
fieldMask	uint64_t	配置生效的字段掩码。非0的位代表配置对应字段指定的值，0的位代表使用默认值，可配置的掩码位见具体字段的说明。
valueType	KmlAmgValueType	预条件子矩阵数值类型 ● KMLAMG_VALUE_FP32表示float ● KMLAMG_VALUE_FP64表示double 掩码位：KMLAMG_PRECOND_OPTIONS_VALUE_TYPE
indexType	KmlAmgIndexType	预条件子矩阵索引类型 ● KMLAMG_INDEX_INT32表示int32_t ● KMLAMG_INDEX_INT64表示int64_t（暂不支持） 掩码位：KMLAMG_PRECOND_OPTIONS_INDEX_TYPE
precondType	KmlAmgPrecondType	预条件子类型有如下值： KMLAMG_PRECOND_AMG表示amg预条件子 掩码位：KMLAMG_PRECOND_OPTIONS_TYPE
numThreads	int32_t	线程数 掩码位：KMLAMG_PRECOND_OPTIONS_NUM_THREADS

依赖

#include "amg4c.h"

6.5.3.4.6.5 KmlAmgPrecondDestroy

销毁预条件子句柄并释放资源。

接口定义

C Interface：KmlAmgStatus KmlAmgPrecondDestroy(KmlAmgPrecondH precond);

 版权所有 © 华为技术有限公司

参数

参数名	类型	描述	输入/输出
precond	KmlAmgPrecondH	预条件子句柄	输入/输出

返回值

返回值	类型	描述
KMLAMG_SUCCESS	int	运行成功
KMLAMG_ERR_INVALID_PARAM	int	参数无效
KMLAMG_ERR_UNSUPPORTED	int	功能暂不支持
KMLAMG_ERR_NULL_POINTER	int	空指针
KMLAMG_ERR_NO_REQUIRED_FIELD	int	没有设置必需字段
KMLAMG_ERR_NO_MEMORY	int	内存分配失败
KMLAMG_ERR_UNKNOWN_TYPE	int	未知的类型
KMLAMG_ERR_UNEXPECTED_CALL_ORDER	int	预期之外的调用顺序
KMLAMG_ERR_NO_RESOURCE	int	资源未就绪
KMLAMG_ERR_INCONSISTENT	int	不同阶段传递的信息不一致
KMLAMG_ERR_UNKNOWN_ERROR	int	未知错误

依赖

#include "amg4c.h"

6.5.3.4.6.6 KmlAmgPrecondSetSetupOptions

设置构建预条件子所需要的参数。

版权所有 © 华为技术有限公司

接口定义

C Interface：KmlAmgStatus KmlAmgPrecondSetSetupOptions(KmlAmgPrecondH precond, const KmlAmgPrecondSetupOptions* options);

参数

参数名	类型	描述	输入/输出
precond	KmlAmgPrecondH	预条件子句柄	输入/输出
options	KmlAmgPrecondSetupOptions	预条件子设置的选项	输入

返回值

返回值	类型	描述
KMLAMG_SUCCESS	int	运行成功
KMLAMG_ERR_INVALID_PARAM	int	参数无效
KMLAMG_ERR_UNSUPPORTED	int	功能暂不支持
KMLAMG_ERR_NULL_POINTER	int	空指针
KMLAMG_ERR_NO_REQUIRED_FIELD	int	没有设置必需字段
KMLAMG_ERR_NO_MEMORY	int	内存分配失败
KMLAMG_ERR_UNKNOWN_TYPE	int	未知的类型
KMLAMG_ERR_UNEXPECTED_CALL_ORDER	int	预期之外的调用顺序
KMLAMG_ERR_NO_RESOURCE	int	资源未就绪
KMLAMG_ERR_INCONSISTENT	int	不同阶段传递的信息不一致
KMLAMG_ERR_UNKNOWN_ERROR	int	未知错误

结构体说明

KmlAmgPrecondSetupOptions结构体描述：

 版权所有 © 华为技术有限公司

字段名	类型	描述
fieldMask	uint64_t	配置生效的字段掩码。非0的位代表配置对应字段指定的值，0的位代表使用默认值，可配置的掩码位请参见具体字段的说明。
partitionAlgo	KmlAmgPartitionAlgo	对矩阵进行重排，有如下值： ● KMLAMG_PARTITION_NONE表示不进行重排（默认值） ● KMLAMG_PARTITION_KRDR表示使用krdr算法重排 掩码位：KMLAMG_PRECOND_SETUP_OPTIONS_PARTITION
reusePartition	int8_t	重复使用上次setup的重排（暂不支持） 掩码位：KMLAMG_PRECOND_SETUP_OPTIONS_REUSE_PARTITION
strongThreshold	double	AMG做网格粗化时使用的强阈值，默认值为0.25 掩码位：KMLAMG_PRECOND_SETUP_OPTIONS_STRONG_THRESHOLD
maxLevels	int32_t	AMG的最大网格层数，默认值为8 掩码位：KMLAMG_PRECOND_SETUP_OPTIONS_MAX_LEVELS
coarsenType	KmlAmgPrecondAmgCoarsenType	AMG的粗化类型，有如下值： ● KMLAMG_PRECOND_AMG_COARSEN_PMIS（默认值） ● KMLAMG_PRECOND_AMG_COARSEN_HMIS ● KMLAMG_PRECOND_AMG_COARSEN_PAIR_FACE_AREA（暂不支持） 掩码位：KMLAMG_PRECOND_SETUP_OPTIONS_COARSEN_TYPE

版权所有 © 华为技术有限公司

字段名	类型	描述
interpType	KmlAmgPrecondAmgInterpType	AMG的插值类型，有如下值： ● KMLAMG_PRECOND_AMG_INTERP_DIRECT（默认值） ● KMLAMG_PRECOND_AMG_INTERP_FF ● KMLAMG_PRECOND_AMG_INTERP_PAIR（暂不支持） 掩码位：KMLAMG_PRECOND_SETUP_OPTIONS_INTERP_TYPE
downSmoothType	KmlAmgPrecondAmgSmoothType	前平滑算子类型，有如下值： ● KMLAMG_PRECOND_AMG_SMOOTH_SOR_FORWARD ● KMLAMG_PRECOND_AMG_SMOOTH_SOR_BACKWARD ● KMLAMG_PRECOND_AMG_SMOOTH_SOR_SYMMETRIC ● KMLAMG_PRECOND_AMG_SMOOTH_L1_SOR_FORWARD（默认值） ● KMLAMG_PRECOND_AMG_SMOOTH_L1_SOR_BACKWARD ● KMLAMG_PRECOND_AMG_SMOOTH_L1_SOR_SYMMETRIC 掩码位：KMLAMG_PRECOND_SETUP_OPTIONS_DOWN_SMOOTH_TYPE
upSmoothType	KmlAmgPrecondAmgSmoothType	后平滑算子类型，有如下值： ● KMLAMG_PRECOND_AMG_SMOOTH_SOR_FORWARD ● KMLAMG_PRECOND_AMG_SMOOTH_SOR_BACKWARD ● KMLAMG_PRECOND_AMG_SMOOTH_SOR_SYMMETRIC ● KMLAMG_PRECOND_AMG_SMOOTH_L1_SOR_FORWARD ● KMLAMG_PRECOND_AMG_SMOOTH_L1_SOR_BACKWARD（默认值） ● KMLAMG_PRECOND_AMG_SMOOTH_L1_SOR_SYMMETRIC 掩码位：KMLAMG_PRECOND_SETUP_OPTIONS_UP_SMOOTH_TYPE

 版权所有 © 华为技术有限公司

字段名	类型	描述
coarsestSmoothType	KmlAmgPrecondAmgSmoothType	最粗层平滑算子类型，有如下值： • KMLAMG_PRECOND_AMG_SMOOTH_SOR_FORWARD • KMLAMG_PRECOND_AMG_SMOOTH_SOR_BACKWARD • KMLAMG_PRECOND_AMG_SMOOTH_SOR_SYMMETRIC • KMLAMG_PRECOND_AMG_SMOOTH_L1_SOR_FORWARD • KMLAMG_PRECOND_AMG_SMOOTH_L1_SOR_BACKWARD • KMLAMG_PRECOND_AMG_SMOOTH_L1_SOR_SYMMETRIC（默认值） 掩码位：KMLAMG_PRECOND_SETUP_OPTIONS_COARSEST_SMOOTH_TYPE

依赖

#include "amg4c.h"

6.5.3.4.6.7 KmlAmgPrecondSetup

构建预条件子。

接口定义

C Interface：KmlAmgStatus KmlAmgPrecondSetup(KmlAmgPrecondH precond, KmlAmgMatrixH matrix);

参数

参数名	类型	描述	输入/输出
precond	KmlAmgPrecondH	预条件子句柄	输入/输出
matrix	KmlAmgMatrixH	矩阵句柄	输入

返回值

返回值	类型	描述
KMLAMG_SUCCESS	int	运行成功

版权所有 © 华为技术有限公司

返回值	类型	描述
KMLAMG_ERR_INVALID_PARAM	int	参数无效
KMLAMG_ERR_UNSUPPORTED	int	功能暂不支持
KMLAMG_ERR_NULL_POINTER	int	空指针
KMLAMG_ERR_NO_REQUIRED_FIELD	int	没有设置必需字段
KMLAMG_ERR_NO_MEMORY	int	内存分配失败
KMLAMG_ERR_UNKNOWN_TYPE	int	未知的类型
KMLAMG_ERR_UNEXPECTED_CALL_ORDER	int	预期之外的调用顺序
KMLAMG_ERR_NO_RESOURCE	int	资源未就绪
KMLAMG_ERR_INCONSISTENT	int	不同阶段传递的信息不一致
KMLAMG_ERR_UNKNOWN_ERROR	int	未知错误

依赖

#include "amg4c.h"

6.5.3.4.6.8 KmlAmgPrecondSetApplyOptions

进行预条件子参数设置执行。

接口定义

C Interface：KmlAmgStatus KmlAmgPrecondSetApplyOptions(KmlAmgPrecondH precond, const KmlAmgPrecondApplyOptions* options);

参数

参数名	类型	描述	输入/输出
precond	KmlAmgPrecondH	预条件子句柄	输入/输出

版权所有 © 华为技术有限公司

参数名	类型	描述	输入/输出
options	KmlAmgPrecondApplyOptions	预条件子求解配置选项	输入

返回值

返回值	类型	描述
KMLAMG_SUCCESS	int	运行成功
KMLAMG_ERR_INVALID_PARAM	int	参数无效
KMLAMG_ERR_UNSUPPORTED	int	功能暂不支持
KMLAMG_ERR_NULL_POINTER	int	空指针
KMLAMG_ERR_NO_REQUIRED_FIELD	int	没有设置必需字段
KMLAMG_ERR_NO_MEMORY	int	内存分配失败
KMLAMG_ERR_UNKNOWN_TYPE	int	未知的类型
KMLAMG_ERR_UNEXPECTED_CALL_ORDER	int	预期之外的调用顺序
KMLAMG_ERR_NO_RESOURCE	int	资源未就绪
KMLAMG_ERR_INCONSISTENT	int	不同阶段传递的信息不一致
KMLAMG_ERR_UNKNOWN_ERROR	int	未知错误

结构体说明

KmlAmgPrecondApplyOptions结构体描述（暂无配置项）：

字段名	类型	描述
fieldMask	uint64_t	配置生效的字段掩码。非0的位代表配置对应字段指定的值，0的位代表使用默认值，暂无可配置的掩码位。

 版权所有 © 华为技术有限公司

依赖

#include "amg4c.h"

6.5.3.4.6.9 KmlAmgPrecondApply

进行预条件子求解。如果单独使用预条件子，参数x必须是零初值，暂不支持非零初值。

接口定义

C Interface：KmlAmgStatus KmlAmgPrecondApply(KmlAmgPrecondH precond, KmlAmgMatrixH b, KmlAmgMatrixH x);

参数

参数名	类型	描述	输入/输出
precond	KmlAmgPrecondH	预条件子句柄	输入
b	KmlAmgMatrixH	矩阵句柄，表示右端向量	输入
x	KmlAmgMatrixH	矩阵句柄，表示解向量，初始输入值作为初次迭代初值	输入/输出

返回值

返回值	类型	描述
KMLAMG_SUCCESS	int	运行成功
KMLAMG_ERR_INVALID_PARAM	int	参数无效
KMLAMG_ERR_UNSUPPORTED	int	功能暂不支持
KMLAMG_ERR_NULL_POINTER	int	空指针
KMLAMG_ERR_NO_REQUIRED_FIELD	int	没有设置必需字段
KMLAMG_ERR_NO_MEMORY	int	内存分配失败
KMLAMG_ERR_UNKNOWN_TYPE	int	未知的类型
KMLAMG_ERR_UNEXPECTED_CALL_ORDER	int	预期之外的调用顺序

版权所有 © 华为技术有限公司

返回值	类型	描述
KMLAMG_ERR_NO_RESOURCE	int	资源未就绪
KMLAMG_ERR_INCONSISTENT	int	不同阶段传递的信息不一致
KMLAMG_ERR_UNKNOWN_ERROR	int	未知错误

依赖

#include "amg4c.h"

6.5.3.4.6.10 KmlAmgPrecondQuery

进行预条件子信息查询。

接口定义

C Interface：

KmlAmgStatus KmlAmgPrecondQuery(KmlAmgPrecondH precond, KmlAmgPrecondInfo* info);

参数

参数名	类型	描述	输入/输出
precond	KmlAmgPrecondH	预条件子句柄	输入
info	KmlAmgPrecondInfo	用于保存用户查询的预条件子信息结果	输入/输出

返回值

返回值	类型	描述
KMLAMG_SUCCESS	int	运行成功
KMLAMG_ERR_INVALID_PARAM	int	参数无效
KMLAMG_ERR_UNSUPPORTED	int	功能暂不支持
KMLAMG_ERR_NULL_POINTER	int	空指针

版权所有 © 华为技术有限公司

返回值	类型	描述
KMLAMG_ERR_NO_REQUIRED_FIELD	int	没有设置必需字段
KMLAMG_ERR_NO_MEMORY	int	内存分配失败
KMLAMG_ERR_UNKNOWN_TYPE	int	未知的类型
KMLAMG_ERR_UNEXPECTED_CALL_ORDER	int	预期之外的调用顺序
KMLAMG_ERR_NO_RESOURCE	int	资源未就绪
KMLAMG_ERR_INCONSISTENT	int	不同阶段传递的信息不一致
KMLAMG_ERR_UNKNOWN_ERROR	int	未知错误

结构体说明

KmlAmgPrecondInfo结构体描述：

字段名	类型	描述
fieldMask	uint64_t	配置生效的字段掩码。非0的位代表配置对应字段指定的值，0的位代表使用默认值，暂无可配置的掩码位。
perm	void*	应用于矩阵的重新分区排列。排列后矩阵的第i行/列对应于原始矩阵的第perm[i]行/列。用户需要分配足够的空间，即N * sizeof(IndexType)。 - N：矩阵的全局维度； - IndexType：依赖预条件子的TyamgIndexType类型，可以是int32_t或int64_t。
localBegin	int64_t	重排后本进程矩阵的起始行/列索引。

依赖

#include "amg4c.h"

6.5.3.4.6.11 KmlAmgSolverCreate

创建迭代法求解器句柄。

 版权所有 © 华为技术有限公司

接口定义

C Interface： KmlAmgStatus KmlAmgSolverCreate(KmlAmgSolverH* solver, const KmlAmgSolverOptions* options);

参数

参数名	类型	描述	输入/输出
solver	KmlAmgSolverH*	求解器句柄，solver不可为空指针	输入/输出
options	const KmlAmgSolverOptions*	求解器配置信息	输入

返回值

返回值	类型	描述
KMLAMG_SUCCESS	int	运行成功
KMLAMG_ERR_INVALID_PARAM	int	参数无效
KMLAMG_ERR_UNSUPPORTED	int	功能暂不支持
KMLAMG_ERR_NULL_POINTER	int	空指针
KMLAMG_ERR_NO_REQUIRED_FIELD	int	没有设置必需字段
KMLAMG_ERR_NO_MEMORY	int	内存分配失败
KMLAMG_ERR_UNKNOWN_TYPE	int	未知的类型
KMLAMG_ERR_UNEXPECTED_CALL_ORDER	int	预期之外的调用顺序
KMLAMG_ERR_NO_RESOURCE	int	资源未就绪
KMLAMG_ERR_INCONSISTENT	int	不同阶段传递的信息不一致
KMLAMG_ERR_UNKNOWN_ERROR	int	未知错误

 版权所有 © 华为技术有限公司

结构体说明

KmlAmgSolverOptions结构体描述：

字段名	类型	描述
fieldMask	uint64_t	配置生效的字段掩码。非0的位代表配置对应字段指定的值，0的位代表使用默认值，可配置的掩码位请参见具体字段的说明。
valueType	KmlAmgValueType	Solver使用的矩阵数值类型，有如下值： ● KMLAMG_VALUE_FP32表示float ● KMLAMG_VALUE_FP64表示double 掩码位：KMLAMG_SOLVER_OPTIONS_VALUE_TYPE
indexType	KmlAmgIndexType	Solver使用的矩阵索引类型，有如下值： ● KMLAMG_INDEX_INT32表示int32_t ● KMLAMG_INDEX_INT64表示int64_t（暂不支持） 掩码位：KMLAMG_SOLVER_OPTIONS_INDEX_TYPE
solverType	KmlAmgSolverType	Solver使用算法类型，有如下值： ● KMLAMG_SOLVER_NONE表示不使用任何迭代法 ● KMLAMG_SOLVER_CG表示使用CG算法（默认值） ● KMLAMG_SOLVER_BICGSTAB表示使用bicgstab算法 ● KMLAMG_SOLVER_GMRES表示使用gmres算法 掩码位：KMLAMG_SOLVER_OPTIONS_TYPE
numThreads	int32_t	线程数，默认值为1 掩码位：KMLAMG_SOLVER_OPTIONS_NUM_THREADS

依赖

#include "amg4c.h"

6.5.3.4.6.12 KmlAmgSolverDestroy

销毁迭代法求解器句柄并释放资源。

接口定义

C Interface：KmlAmgStatus KmlAmgSolverDestroy(KmlAmgSolverH solver);

 版权所有 © 华为技术有限公司

参数

参数名	类型	描述	输入/输出
solver	KmlAmgSolverH	迭代法求解器句柄	输入/输出

返回值

返回值	类型	描述
KMLAMG_SUCCESS	int	运行成功
KMLAMG_ERR_INVALID_PARAM	int	参数无效
KMLAMG_ERR_UNSUPPORTED	int	功能暂不支持
KMLAMG_ERR_NULL_POINTER	int	空指针
KMLAMG_ERR_NO_REQUIRED_FIELD	int	没有设置必需字段
KMLAMG_ERR_NO_MEMORY	int	内存分配失败
KMLAMG_ERR_UNKNOWN_TYPE	int	未知的类型
KMLAMG_ERR_UNEXPECTED_CALL_ORDER	int	预期之外的调用顺序
KMLAMG_ERR_NO_RESOURCE	int	资源未就绪
KMLAMG_ERR_INCONSISTENT	int	不同阶段传递的信息不一致
KMLAMG_ERR_UNKNOWN_ERROR	int	未知错误

依赖

#include "amg4c.h"

6.5.3.4.6.13 KmlAmgSolverSetSetupOptions

设置迭代法求解器的参数。

版权所有 © 华为技术有限公司

接口定义

C Interface： KmlAmgStatus KmlAmgSolverSetSetupOptions(KmlAmgSolverH solver, const KmlAmgSolverSetupOptions* options);

参数

参数名	类型	描述	输入/输出
solver	KmlAmgSolverH	迭代法求解器句柄	输入/输出
options	const KmlAmgSolverSetupOptions*	迭代法配置选项	输入

返回值

返回值	类型	描述
KMLAMG_SUCCESS	int	运行成功
KMLAMG_ERR_INVALID_PARAM	int	参数无效
KMLAMG_ERR_UNSUPPORTED	int	功能暂不支持
KMLAMG_ERR_NULL_POINTER	int	空指针
KMLAMG_ERR_NO_REQUIRED_FIELD	int	没有设置必需字段
KMLAMG_ERR_NO_MEMORY	int	内存分配失败
KMLAMG_ERR_UNKNOWN_TYPE	int	未知的类型
KMLAMG_ERR_UNEXPECTED_CALL_ORDER	int	预期之外的调用顺序
KMLAMG_ERR_NO_RESOURCE	int	资源未就绪
KMLAMG_ERR_INCONSISTENT	int	不同阶段传递的信息不一致
KMLAMG_ERR_UNKNOWN_ERROR	int	未知错误

版权所有 © 华为技术有限公司

结构体说明

KmlAmgSolverSetupOptions结构体描述：

字段名	类型	描述
fieldMask	uint64_t	配置生效的字段掩码。非0的位代表配置对应字段指定的值，0的位代表使用默认值，可配置的掩码位请参见具体字段的说明
partitionAlgo	KmlAmgPartitionAlgo	对矩阵进行重排，有如下值： • KMLAMG_PARTITION_NONE表示不进行重排（默认值） • KMLAMG_PARTITION_KRDR表示使用krdr算法重排 掩码位：KMLAMG_SOLVER_SETUP_OPTIONS_PARTITION
reusePartition	int8_t	重复使用上一次setup的重排（暂不支持） 掩码位：KMLAMG_SOLVER_SETUP_OPTIONS_REUSE_PARTITION
restartLen	int	表示gmres的重启长度，只有当solverType为KMLAMG_SOLVER_GMRES时生效，默认值为30 掩码位：KMLAMG_SOLVER_SETUP_OPTIONS_RESTART_LEN

依赖

#include "amg4c.h"

6.5.3.4.6.14 KmlAmgSolverSetup

构建迭代法求解器，调用该接口前必须设置预条件子，即调用**KmlAmgSolverSetPrec**接口。

接口定义

C Interface：KmlAmgStatus KmlAmgSolverSetup(KmlAmgSolverH solver, KmlAmgMatrixH matrix);

参数

参数名	类型	描述	输入/输出
solver	KmlAmgSolverH	迭代法求解器句柄	输入/输出
matrix	KmlAmgMatrixH	矩阵句柄	输入

版权所有 © 华为技术有限公司

返回值

返回值	类型	描述
KMLAMG_SUCCESS	int	运行成功
KMLAMG_ERR_INVALID_PARAM	int	参数无效
KMLAMG_ERR_UNSUPPORTED	int	功能暂不支持
KMLAMG_ERR_NULL_POINTER	int	空指针
KMLAMG_ERR_NO_REQUIRED_FIELD	int	没有设置必需字段
KMLAMG_ERR_NO_MEMORY	int	内存分配失败
KMLAMG_ERR_UNKNOWN_TYPE	int	未知的类型
KMLAMG_ERR_UNEXPECTED_CALL_ORDER	int	预期之外的调用顺序
KMLAMG_ERR_NO_RESOURCE	int	资源未就绪
KMLAMG_ERR_INCONSISTENT	int	不同阶段传递的信息不一致
KMLAMG_ERR_UNKNOWN_ERROR	int	未知错误

依赖

#include "amg4c.h"

6.5.3.4.6.15 KmlAmgSolverSetApplyOptions

进行回代求解。

接口定义

C Interface：KmlAmgStatus KmlAmgSolverSetApplyOptions(KmlAmgSolverHsolver, const KmlAmgSolverApplyOptions* options);

版权所有 © 华为技术有限公司

参数

参数名	类型	描述	输入/输出
solver	KmlAmgSolverH	迭代法求解器句柄	输入/输出
options	const KmlAmgSolverApplyOptions*	迭代法求解的配置参数	输入

返回值

返回值	类型	描述
KMLAMG_SUCCESS	int	运行成功
KMLAMG_ERR_INVALID_PARAM	int	参数无效
KMLAMG_ERR_UNSUPPORTED	int	功能暂不支持
KMLAMG_ERR_NULL_POINTER	int	空指针
KMLAMG_ERR_NO_REQUIRED_FIELD	int	没有设置必需字段
KMLAMG_ERR_NO_MEMORY	int	内存分配失败
KMLAMG_ERR_UNKNOWN_TYPE	int	未知的类型
KMLAMG_ERR_UNEXPECTED_CALL_ORDER	int	预期之外的调用顺序
KMLAMG_ERR_NO_RESOURCE	int	资源未就绪
KMLAMG_ERR_INCONSISTENT	int	不同阶段传递的信息不一致
KMLAMG_ERR_UNKNOWN_ERROR	int	未知错误

结构体说明

KmlAmgSolverApplyOptions结构体描述：

 版权所有 © 华为技术有限公司

字段名	类型	描述
fieldMask	uint64_t	配置生效的字段掩码。非0的位代表配置对应字段指定的值，0的位代表使用默认值，可配置的掩码位请参见具体字段的说明
maxIters	int64_t	迭代法允许的最大迭代次数，超过该次数会自动停止迭代，默认值为1000 掩码位：KMLAMG_SOLVER_APPLY_OPTIONS_MAX_ITERS
tolerance	double	迭代法的误差值，默认值为1e-10 掩码位：KMLAMG_SOLVER_APPLY_OPTIONS_TOLERANCE

依赖

#include "amg4c.h"

6.5.3.4.6.16 KmlAmgSolverApply

进行回代求解。

接口定义

C Interface：KmlAmgStatus KmlAmgSolverApply(KmlAmgSolverH solver, KmlAmgMatrixH b, KmlAmgMatrixH x);

参数

参数名	类型	描述	输入/输出
solver	KmlAmgSolverH	迭代法求解器句柄	输入
b	KmlAmgMatrixH	右端向量	输入
x	KmlAmgMatrixH	解向量	输入/输出

依赖

#include "amg4c.h"

6.5.3.4.6.17 KmlAmgSolverSetPrec

为迭代法求解器设置预条件子。

接口定义

C Interface：KmlamgStatus KmlamgSolverSetPrec(KmlAmgSolverH solver, KmlamgKmlAmgPrecondH precond);

版权所有 © 华为技术有限公司

参数

参数名	类型	描述	输入/输出
solver	KmlAmgSolverH	迭代法求解器句柄	输入/输出
precond	KmlAmgPrecondH	预条件子句柄	输入

返回值

返回值	类型	描述
KMLAMG_SUCCESS	int	运行成功
KMLAMG_ERR_INVALID_PARAM	int	参数无效
KMLAMG_ERR_UNSUPPORTED	int	功能暂不支持
KMLAMG_ERR_NULL_POINTER	int	空指针
KMLAMG_ERR_NO_REQUIRED_FIELD	int	没有设置必需字段
KMLAMG_ERR_NO_MEMORY	int	内存分配失败
KMLAMG_ERR_UNKNOWN_TYPE	int	未知的类型
KMLAMG_ERR_UNEXPECTED_CALL_ORDER	int	预期之外的调用顺序
KMLAMG_ERR_NO_RESOURCE	int	资源未就绪
KMLAMG_ERR_INCONSISTENT	int	不同阶段传递的信息不一致
KMLAMG_ERR_UNKNOWN_ERROR	int	未知错误

依赖

#include "amg4c.h"

6.5.3.4.6.18 KmlAmgSolverQuery

查询迭代法求解器信息。

版权所有 © 华为技术有限公司

接口定义

C Interface： KmlAmgStatus KmlAmgSolverQuery(KmlAmgSolverH solver, KmlAmgSolverInfo* info);

参数

参数名	类型	描述	输入/输出
solver	KmlAmgSolverH	迭代法求解器句柄	输入
info	KmlAmgSolverInfo*	用于保存用户查询的迭代法求解器信息结果	输入/输出

返回值

返回值	类型	描述
KMLAMG_SUCCESS	int	运行成功
KMLAMG_ERR_INVALID_PARAM	int	参数无效
KMLAMG_ERR_UNSUPPORTED	int	功能暂不支持
KMLAMG_ERR_NULL_POINTER	int	空指针
KMLAMG_ERR_NO_REQUIRED_FIELD	int	没有设置必需字段
KMLAMG_ERR_NO_MEMORY	int	内存分配失败
KMLAMG_ERR_UNKNOWN_TYPE	int	未知的类型
KMLAMG_ERR_UNEXPECTED_CALL_ORDER	int	预期之外的调用顺序
KMLAMG_ERR_NO_RESOURCE	int	资源未就绪
KMLAMG_ERR_INCONSISTENT	int	不同阶段传递的信息不一致
KMLAMG_ERR_UNKNOWN_ERROR	int	未知错误

 版权所有 © 华为技术有限公司

结构体说明

KmlAmgSolverInfo结构体描述：

字段名	类型	描述
fieldMask	uint64_t	配置生效的字段掩码。非0的位代表配置对应字段指定的值，0的位代表使用默认值，可配置的掩码位请参见具体字段的说明
nIters	int64_t	迭代次数 掩码位：KMLAMG_SOLVER_INFO_NUM_ITERS
perm	void *	对矩阵做重排时使用的perm，只有当partitionAlgo不为KMLAMG_PARTITION_NONE时有效 掩码位：KMLAMG_SOLVER_INFO_PERM
localBegin	int64_t	重排后本进程矩阵的起始行/列索引，只有当partitionAlgo不为KMLAMG_PARTITION_NONE时有效 掩码位：KMLAMG_SOLVER_INFO_LOCAL_BEGIN

依赖

#include "amg4c.h"

6.5.3.4.6.19 示例

C Interface：

```
#include <stdio.h>
#include <math.h>
#include <mpi.h>
#include <omp.h>
#include <assert.h>
#include "amg4c.h"
int test(int isonlyAMG)
{
    int nthreads = 1;
    int rtn = 0;
    int gid;
    MPI_Comm comm = MPI_COMM_WORLD;
    MPI_Comm_rank(comm, &gid);
    int globalN = 8;
    int localOffset = 0;
    int localLength = globalN;
    int offsets[9] = {0, 2, 4, 6, 7, 8, 10, 12, 14};
    int indices[14] = {0, 7, 1, 6, 2, 5, 3, 4, 2, 5, 1, 6, 0, 7};
    double values[14] = {1.0, 2.0, -2.0, 3.0, 3.0, 4.0, -4.0, 5.0, 4.0, -6.0, 3.0, 7.0, 2.0, 8.0};
    double b_[8] = {3.0, 1.0, 7.0, -4.0, 5.0, -2.0, 10.0, 10.0};
    double x_[8] = {0.5, 0.5, 0.5, 0.5, 0.5, 0.5, 0.5, 0.5};
    int isAvg = 0;
    int64_t localNnz = offsets[localLength];
    int *indices_ = NULL;
    double *values_ = NULL;
    indices_ = indices;
    values_ = values;
    KmlAmgMatrixH matrixA = NULL;
    KmlAmgMatrixOptions MatrixAoptions;
    KmlAmgMatrixStore MatrixAstore;
    {
```


版权所有 © 华为技术有限公司

```
        MatrixAoptions.fieldMask = KMLAMG_MATRIX_OPTIONS_VALUE_TYPE |
KMLAMG_MATRIX_OPTIONS_INDEX_TYPE |
                                   KMLAMG_MATRIX_OPTIONS_COMM |
KMLAMG_MATRIX_OPTIONS_NUM_THREADS;
        MatrixAoptions.comm = comm;
        MatrixAoptions.indexType = KMLAMG_INDEX_INT32;
        MatrixAoptions.valueType = KMLAMG_VALUE_FP64;
        MatrixAoptions.numThreads = nthreads;
        MatrixAstore.globalNRows = globalN;
        MatrixAstore.globalNCols = globalN;
        MatrixAstore.isAvgPart = isAvg;
        MatrixAstore.localBegin = localOffset;
        MatrixAstore.localNRows = localLength;
        MatrixAstore.localNCols = localLength;
        MatrixAstore.type = KMLAMG_MATRIX_STORE_CSR;
        MatrixAstore.csr.offsets = offsets;
        MatrixAstore.csr.indices = indices_;
        MatrixAstore.csr.values = NULL;
        if (KmlAmgMatrixCreate(&matrixA, &MatrixAstore, &MatrixAoptions) != KMLAMG_SUCCESS) {
            printf("Create A error!\n");
        };
    }
    if (KmlAmgMatrixSetValues(matrixA, values_) != KMLAMG_SUCCESS) {
        printf("matrixA SetValues error!\n");
    }
    KmlAmgMatrixH VectorB = NULL;
    KmlAmgMatrixOptions VectorBoptions;
    KmlAmgMatrixStore VectorBstore;
    {
        VectorBoptions.fieldMask = KMLAMG_MATRIX_OPTIONS_VALUE_TYPE |
KMLAMG_MATRIX_OPTIONS_INDEX_TYPE |
                                   KMLAMG_MATRIX_OPTIONS_COMM |
KMLAMG_MATRIX_OPTIONS_NUM_THREADS;
        VectorBoptions.comm = comm;
        VectorBoptions.indexType = KMLAMG_INDEX_INT32;
        VectorBoptions.valueType = KMLAMG_VALUE_FP64;
        VectorBoptions.numThreads = nthreads;
        VectorBstore.globalNRows = globalN;
        VectorBstore.globalNCols = 1;
        VectorBstore.isAvgPart = isAvg;
        VectorBstore.localBegin = localOffset;
        VectorBstore.localNRows = localLength;
        VectorBstore.localNCols = 1;
        VectorBstore.type = KMLAMG_MATRIX_STORE_DENSE_ROW_MAJOR;
        VectorBstore.dense.values = NULL;
        if (KmlAmgMatrixCreate(&VectorB, &VectorBstore, &VectorBoptions) != KMLAMG_SUCCESS) {
            printf("Create B error!\n");
        };
    }
    if (KmlAmgMatrixSetValues(VectorB, b_) != KMLAMG_SUCCESS) {
        printf("VectorB SetValues error!\n");
    }
    KmlAmgMatrixH VectorX = NULL;
    KmlAmgMatrixOptions VectorXoptions;
    KmlAmgMatrixStore VectorXstore;
    {
        VectorXoptions.fieldMask = KMLAMG_MATRIX_OPTIONS_VALUE_TYPE |
KMLAMG_MATRIX_OPTIONS_INDEX_TYPE |
                                   KMLAMG_MATRIX_OPTIONS_COMM |
KMLAMG_MATRIX_OPTIONS_NUM_THREADS;
        VectorXoptions.comm = comm;
        VectorXoptions.indexType = KMLAMG_INDEX_INT32;
        VectorXoptions.valueType = KMLAMG_VALUE_FP64;
        VectorXoptions.numThreads = nthreads;
        VectorXstore.globalNRows = globalN;
        VectorXstore.globalNCols = 1;
        VectorXstore.isAvgPart = isAvg;
        VectorXstore.localBegin = localOffset;
        VectorXstore.localNRows = localLength;
```

 版权所有 © 华为技术有限公司

```
        VectorXstore.localNCols = 1;
        VectorXstore.type = KMLAMG_MATRIX_STORE_DENSE_ROW_MAJOR;
        VectorXstore.dense.values = x_;
        if (KmlAmgMatrixCreate(&VectorX, &VectorXstore, &VectorXoptions) != KMLAMG_SUCCESS) {
            printf("Create X error!\n");
        };
    }
    if (KmlAmgMatrixSetValues(VectorX, x_) != KMLAMG_SUCCESS) {
        printf("VectorX SetValues error!\n");
    }
    KmlAmgPrecondH precond = NULL;
    KmlAmgPrecondOptions PreOptions;
    KmlAmgPrecondSetupOptions PreSetOptions;
    {
        PreOptions.fieldMask = KMLAMG_PRECOND_OPTIONS_VALUE_TYPE |
KMLAMG_PRECOND_OPTIONS_INDEX_TYPE |
                               KMLAMG_PRECOND_OPTIONS_TYPE | KMLAMG_PRECOND_OPTIONS_NUM_THREADS;
        PreOptions.valueType = KMLAMG_VALUE_FP64;
        PreOptions.indexType = KMLAMG_INDEX_INT32;
        PreOptions.precondType = KMLAMG_PRECOND_AMG;
        PreOptions.numThreads = nthreads;
        if (KmlAmgPrecondCreate(&precond, &PreOptions) != KMLAMG_SUCCESS) {
            printf("Create Precond error!\n");
        };
        PreSetOptions.fieldMask =
            KMLAMG_PRECOND_SETUP_OPTIONS_PARTITION |
KMLAMG_PRECOND_SETUP_OPTIONS_STRONG_THRESHOLD |
            KMLAMG_PRECOND_SETUP_OPTIONS_MAX_LEVELS |
KMLAMG_PRECOND_SETUP_OPTIONS_COARSEN_TYPE |
            KMLAMG_PRECOND_SETUP_OPTIONS_INTERP_TYPE |
KMLAMG_PRECOND_SETUP_OPTIONS_DOWN_SMOOTH_TYPE |
            KMLAMG_PRECOND_SETUP_OPTIONS_UP_SMOOTH_TYPE |
KMLAMG_PRECOND_SETUP_OPTIONS_COARSEST_SMOOTH_TYPE;
        PreSetOptions.strongThreshold = 0.25;
        PreSetOptions.maxLevels = 1;
        PreSetOptions.coarsenType = KMLAMG_PRECOND_AMG_COARSEN_PMIS;
        PreSetOptions.interpType = KMLAMG_PRECOND_AMG_INTERP_DIRECT;
        PreSetOptions.downSmoothType = KMLAMG_PRECOND_AMG_SMOOTH_L1_SOR_FORWARD;
        PreSetOptions.upSmoothType = KMLAMG_PRECOND_AMG_SMOOTH_L1_SOR_BACKWARD;
        PreSetOptions.coarsestSmoothType = KMLAMG_PRECOND_AMG_SMOOTH_L1_SOR_SYMMETRIC;
        PreSetOptions.partitionAlgo = KMLAMG_PARTITION_KRDR;
        if (KmlAmgPrecondSetSetupOptions(precond, &PreSetOptions) != KMLAMG_SUCCESS) {
            printf("Set Precond setup options error!\n");
        };
        if (KmlAmgPrecondSetup(precond, matrixA) != KMLAMG_SUCCESS) {
            printf("Setup Precond error!\n");
        };
    }
    KmlAmgSolverH Ksp = NULL;
    KmlAmgSolverOptions KspOptions;
    KmlAmgSolverSetupOptions KspSetupOptions;
    {
        KspOptions.fieldMask = KMLAMG_SOLVER_OPTIONS_VALUE_TYPE |
KMLAMG_SOLVER_OPTIONS_INDEX_TYPE |
                               KMLAMG_SOLVER_OPTIONS_TYPE | KMLAMG_SOLVER_OPTIONS_NUM_THREADS;
        KspOptions.valueType = KMLAMG_VALUE_FP64;
        KspOptions.indexType = KMLAMG_INDEX_INT32;
        KspOptions.solverType = KMLAMG_SOLVER_CG;
        KspOptions.numThreads = nthreads;
        if (KmlAmgSolverCreate(&Ksp, &KspOptions) != KMLAMG_SUCCESS) {
            printf("KmlAmgSolverCreate  error!\n");
        };
        KspSetupOptions.fieldMask = KMLAMG_SOLVER_SETUP_OPTIONS_PARTITION;
        KspSetupOptions.partitionAlgo = KMLAMG_PARTITION_KRDR;
    }
    if (isonlyAMG) {
        KmlAmgPrecondApplyOptions preApplyOptions;
        preApplyOptions.fieldMask = 0;
        KmlAmgPrecondSetApplyOptions(precond, &preApplyOptions);
```

 版权所有 © 华为技术有限公司

```
        KmlAmgPrecondApply(precond, VectorB, VectorX);
        KmlAmgPrecondInfo Precondinfo;
        Precondinfo.fieldMask = 0;
        KmlAmgPrecondQuery(precond, &Precondinfo);
        double *xvalue = (double *)VectorXstore.dense.values;
        printf("x is ");
        for (int i = 0; i < globalN; i++) {
            printf("%f ", xvalue[i]);
        }
        printf("\n");
        return 0;
    }
    KmlAmgSolverSetPrec(Ksp, precond);
    KmlAmgSolverApplyOptions KspApplyOptions;
    {
        KspApplyOptions.fieldMask = KMLAMG_SOLVER_APPLY_OPTIONS_MAX_ITERS |
KMLAMG_SOLVER_APPLY_OPTIONS_TOLERANCE;
        KspApplyOptions.maxIters = 1000;
        KspApplyOptions.tolerance = 1e-9;
        MPI_Barrier(comm);
        for (int s = 0; s < 1; s++) {
            KmlAmgMatrixSetValues(matrixA, values_);
            KmlAmgMatrixSetValues(VectorB, b_);
            MPI_Barrier(comm);
            KmlAmgSolverSetSetupOptions(Ksp, &KspSetupOptions);
            KmlAmgSolverSetup(Ksp, matrixA);
            for (int ss = 0; ss < 1; ss++) {
                MPI_Barrier(comm);
                KmlAmgMatrixSetValues(VectorX, x_);
                KmlAmgSolverSetApplyOptions(Ksp, &KspApplyOptions);
                KmlAmgSolverApply(Ksp, VectorB, VectorX);
            }
        }
    }
    KmlAmgSolverInfo SolverInfo;
    SolverInfo.fieldMask = 0;
    KmlAmgSolverQuery(Ksp, &SolverInfo);
    double *xvalue = (double *)VectorXstore.dense.values;
    printf("x is ");
    for (int i = 0; i < globalN; i++) {
        printf("%f ", xvalue[i]);
    }
    printf("\n");
    int *perm = (int *)malloc(sizeof(int)*globalN);
    KmlAmgPrecondInfo precondInfo;
    precondInfo.fieldMask = KMLAMG_PRECOND_INFO_LOCAL_BEGIN | KMLAMG_PRECOND_INFO_PERM;
    precondInfo.perm = (void *)perm;
    KmlAmgPrecondQuery(precond, &precondInfo);
    int64_t permSum = 0;
    for (int64_t i = 0; i < globalN; i++) {
        permSum += perm[i];
    }
    assert(permSum == (globalN * (globalN - 1)) / 2);
    KmlAmgSolverInfo solverInfo;
    solverInfo.fieldMask = KMLAMG_SOLVER_INFO_LOCAL_BEGIN | KMLAMG_SOLVER_INFO_PERM |
KMLAMG_SOLVER_INFO_NUM_ITERS;
    solverInfo.perm = (void *)perm;
    KmlAmgSolverQuery(Ksp, &solverInfo);
    permSum = 0;
    for (int64_t i = 0; i < globalN; i++) {
        permSum += perm[i];
    }
    assert(permSum == (globalN * (globalN - 1)) / 2);
    printf("nIterations = %ld\n", solverInfo.nIters);
    KmlAmgMatrixDestroy(matrixA);
    KmlAmgMatrixDestroy(VectorB);
    KmlAmgMatrixDestroy(VectorX);
    KmlAmgPrecondDestroy(precond);
    KmlAmgSolverDestroy(Ksp);
```

 版权所有 © 华为技术有限公司

```
    return rtn;
}
int main(int argc, char **argv)
{
    MPI_Init(&argc, &argv);
    test(0);
    MPI_Finalize();
    return 0;
}
```

6.5.3.4.7 SCADSS-LLT/LLH (DEPRECATED)

集群版直接法推荐使用**SCADSS**。

6.5.3.4.7.1 KmlScadss(SPD/HPD)Init?I

初始化数据结构，并将用户提供的系数矩阵关联到求解系统中，初始化集群版LLT/LLH方法时，使用上三角的CSR格式。

接口定义

C Interface：

int KmlScadssSpdInitSI(KmlScasolverTask **pHandle, int n, float *a, int *ja, int *ia, MPI_Comm comm);

int KmlScadssSpdInitDI(KmlScasolverTask **pHandle, int n, double *a, int *ja, int *ia, MPI_Comm comm);

int KmlScadssHpdInitCI(KmlScasolverTask **pHandle, int n, kml_complex_float *a, int *ja, int *ia, MPI_Comm comm);

int KmlScadssHpdInitZI(KmlScasolverTask **pHandle, int n, kml_complex_double *a, int *ja, int *ia, MPI_Comm comm);

参数

参数名	类型	描述	输入/输出
pHandle	KmlScasolverTask **	求解器句柄，pHandle不可为空指针。	输出
n	int	n为矩阵A的维度。	输入

版权所有 © 华为技术有限公司

参数名	类型	描述	输入/输出
a	• 在KmlScadssSpdInitSI中为float *。 • 在KmlScadssSpdInitDI中为double *。 • 在KmlScadssHpdInitCI中为kml_complex_float *。 • 在KmlScadssHpdInitZI中为kml_complex_double *。	a为维度为非零元素的个数的数组，a[i]为第i个非零元的值。该数组的生命周期与pHandle相同。	输入
ja	int	ja为维度为非零元素的个数的数组，ja[i]为第i个非零元的列号。该数组的生命周期与pHandle相同。	输入
ia	int	ia为n+1维数组，ia[i]为第i行第一个非零元在所有非零元中的索引。该数组的生命周期与pHandle相同。	输入
comm	MPI_Comm	为MPI进程通信域。	输入

返回值

返回值	类型	描述
KMLSS_NO_ERROR	int	正常执行。
KMLSS_NULL_ARGUMENT	int	pHandle，a，ja，ia中有空指针。
KMLSS_BAD_N	int	矩阵维数输入值小于等于0。
KMLSS_NO_MEMORY	int	内存开辟失败。

依赖

#include "kml_scadss.h"

 版权所有 © 华为技术有限公司

示例

C Interface：

```
MPI_Init(NULL, NULL);
int size, rank;
MPI_Comm_size(MPI_COMM_WORLD, &size);//获取总进程数
MPI_Comm_rank(MPI_COMM_WORLD, &rank);//获取当前进程标识

int n = 8;
int ia[9] = {0, 3, 7, 9, 11, 13, 15, 16, 17};
int ja[17] = {0, 3, 4, 1, 2, 3, 5, 2, 7, 3, 6, 4, 5, 5, 7, 6, 7};
double a[17] = {1.0, 1.0, 2.0, 9.0, 2.0, 1.0, -3.0, 3.0, 2.0, 9.0, -5.0, 6.0, 1.0, 4.0, 1.0, 7.0, 2.0};
KmlScasolverTask* matrix_handle = NULL;
int ierr = KmlScadssSpdInitDI(&matrix_handle, n, a, ja, ia, MPI_COMM_WORLD);
if (ierr != 0) {
    printf("\nERROR in KmlScadssSpdInitDI: %d", ierr);
}
```

6.5.3.4.7.2 KmlScadss(SPD/HPD)Set?I?

设置与handle关联的值。

接口定义

C Interface：

int KmlScadssSpdSetSII(KmlScaSolverTask **pHandle, enum KML_SOLVER_PARAM param, int *value, int nvalue);

int KmlScadssSpdSetDII(KmlScaSolverTask **pHandle, enum KML_SOLVER_PARAM param, int *value, int nvalue);

int KmlScadssHpdSetCII(KmlScaSolverTask **pHandle, enum KML_SOLVER_PARAM param, int *value, int nvalue);

int KmlScadssHpdSetZII(KmlScaSolverTask **pHandle, enum KML_SOLVER_PARAM param, int *value, int nvalue);

int KmlScadssSpdSetDID(KmlScaSolverTask **pHandle, enum KML_SOLVER_PARAM param, double *value, int nvalue);

int KmlScadssHpdSetZID(KmlScaSolverTask **pHandle, enum KML_SOLVER_PARAM param, double *value, int nvalue);

int KmlScadssSpdSetSIS(KmlScaSolverTask **pHandle, enum KML_SOLVER_PARAM param, float *value, int nvalue);

int KmlScadssHpdSetCIS(KmlScaSolverTask **pHandle, enum KML_SOLVER_PARAM param, float *value, int nvalue);

参数

参数名	类型	描述	输入/输出
pHandle	KmlScaSolverTask	求解器句柄，pHandle不可为空指针。	输入

版权所有 © 华为技术有限公司

参数名	类型	描述	输入/输出
param	enum KML_SOLVER_PARAM	• KMLSS_REFINEMENT_MAX_STEPS表示REFINEMENT的最大次数。 • KMLSS_PERM表示矩阵行列顺序，暂不支持。 • KMLSS_ITER_REF_STEPS表示迭代优化次数，暂不支持。	输入
value	int *	直接求解相关参数数组。	输出
nvalue	int	value数组元素个数。	输入

返回值

返回值	类型	描述
KMLSS_NO_ERROR	int	正常执行。
KMLSS_NULL_ARGUMENT	int	pHandle为空指针。
KMLSS_BAD_HANDLE	int	pHandle内部格式转换错误。
KMLSS_BAD_DATA_SIZE	int	KMLSS_FILL_IN数组长度nvalue设置错误。
KMLSS_INTERNAL_ERROR	int	内部数据返回空指针，KMLSS_FILL_IN获取失败。
KMLSS_BAD_SELECTOR	int	用户的KML_SOLVER_PARAM输入当前不支持，或在求解过程中不存在。

依赖

#include "kml_scadss.h"

示例

C Interface：

```
MPI_Init(NULL, NULL);
int size, rank;
MPI_Comm_size(MPI_COMM_WORLD, &size);//获取总进程数
MPI_Comm_rank(MPI_COMM_WORLD, &rank);//获取当前进程标识

int n = 8;
int ia[9] = {0, 3, 7, 9, 11, 13, 15, 16, 17};
int ja[17] = {0, 3, 4, 1, 2, 3, 5, 2, 7, 3, 6, 4, 5, 5, 7, 6, 7};
double a[17] = {1.0, 1.0, 2.0, 9.0, 2.0, 1.0, -3.0, 3.0, 2.0, 9.0, -5.0, 6.0, 1.0, 4.0, 1.0, 7.0, 2.0};
```

 版权所有 © 华为技术有限公司

```
double b[8] = {4.0, 9.0, 7.0, 6.0, 9.0, 3.0, 2.0, 5.0};
double x[8];
int max_refinement_steps = 10;
KmlScasolverTask* matrix_handle = NULL;
int ierr = KmlScadssSpdInitDI(&matrix_handle, n, a, ja, ia, MPI_COMM_WORLD);
ierr = KmlScadssSpdSetDII(&matrix_handle, KMLSS_REFINEMENT_MAX_STEPS, &max_refinement_steps , 1);
ierr = KmlScadssSpdAnalyzeDI(&matrix_handle);
ierr = KmlScadssSpdFactorizeDI(&matrix_handle);
int nrhs = 1;
int ldx=n, ldb=n;
ierr = KmlScadssSpdSolveDI(&matrix_handle, nrhs, x, ldx, b, ldb);
```

6.5.3.4.7.3 KmlScadss(SPD/HPD)Analyze?I

分析数据结构，对稀疏矩阵进行重排和符号分解。

接口定义

C Interface：

int KmlScadssSpdAnalyzeSI(KmlScasolverTask **pHandle);

int KmlScadssSpdAnalyzeDI(KmlScasolverTask **pHandle);

int KmlScadssHpdAnalyzeCI(KmlScasolverTask **pHandle);

int KmlScadssHpdAnalyzeZI(KmlScasolverTask **pHandle);

参数

参数名	类型	描述	输入/输出
pHandle	KmlScasolverTask **	求解器句柄，传入之前步骤的变量。	输入

返回值

返回值	类型	描述
KMLSS_NO_ERROR	int	正常执行。
KMLSS_NULL_ARGUMENT	int	pHandle为空。
KMLSS_BAD_HANDLE	int	pHandle格式存在问题。

依赖

#include "kml_scadss.h"

示例

C Interface：

 版权所有 © 华为技术有限公司

```
MPI_Init(NULL, NULL);
int size, rank;
MPI_Comm_size(MPI_COMM_WORLD, &size);//获取总进程数
MPI_Comm_rank(MPI_COMM_WORLD, &rank);//获取当前进程标识

int n = 8;
int ia[9] = {0, 3, 7, 9, 11, 13, 15, 16, 17};
int ja[17] = {0, 3, 4, 1, 2, 3, 5, 2, 7, 3, 6, 4, 5, 5, 7, 6, 7};
double a[17] = {1.0, 1.0, 2.0, 9.0, 2.0, 1.0, -3.0, 3.0, 2.0, 9.0, -5.0, 6.0, 1.0, 4.0, 1.0, 7.0, 2.0};
KmlScasolverTask* matrix_handle = NULL;
int ierr = KmlScadssSpdInitDI(&matrix_handle, n, a, ja, ia, MPI_COMM_WORLD);
ierr = KmlScadssSpdAnalyzeDI(&matrix_handle);
if (ierr != 0) {
    printf("\nERROR in KmlScadssSpdAnalyzeDI: %d", ierr);
}
```

6.5.3.4.7.4 KmlScadss(SPD/HPD)Factorize?I

实现对称正定矩阵的LLT分解或者Hermitian正定矩阵的LLH分解。

接口定义

C Interface：

int KmlScadssSpdFactorizeSI(KmlScasolverTask **pHandle);

int KmlScadssSpdFactorizeDI(KmlScasolverTask **pHandle);

int KmlScadssHpdFactorizeCI(KmlScasolverTask **pHandle);

int KmlScadssHpdFactorizeZI(KmlScasolverTask **pHandle);

参数

参数名	类型	描述	输入/输出
pHandle	KmlScasolverTask **	求解器句柄，传入之前步骤的变量。	输入

返回值

返回值	类型	描述
KMLSS_NO_ERROR	int	正常执行。
KMLSS_NULL_ARGUMENT	int	参数无效或格式存在问题。

依赖

#include "kml_scadss.h"

示例

C Interface：

 版权所有 © 华为技术有限公司

```
MPI_Init(NULL, NULL);
int size, rank;
MPI_Comm_size(MPI_COMM_WORLD, &size);//获取总进程数
MPI_Comm_rank(MPI_COMM_WORLD, &rank);//获取当前进程标识

int n = 8;
int ia[9] = {0, 3, 7, 9, 11, 13, 15, 16, 17};
int ja[17] = {0, 3, 4, 1, 2, 3, 5, 2, 7, 3, 6, 4, 5, 5, 7, 6, 7};
double a[17] = {1.0, 1.0, 2.0, 9.0, 2.0, 1.0, -3.0, 3.0, 2.0, 9.0, -5.0, 6.0, 1.0, 4.0, 1.0, 7.0, 2.0};
KmlScasolverTask* matrix_handle = NULL;
int ierr = KmlScadssSpdInitDI(&matrix_handle, n, a, ja, ia, MPI_COMM_WORLD);
ierr = KmlScadssSpdAnalyzeDI(&matrix_handle);
ierr = KmlScadssSpdFactorizeDI(&matrix_handle);
if (ierr != 0) {
    printf("\nERROR in KmlScadssSpdFactorizeDI: %d", ierr);
}
```

6.5.3.4.7.5 KmlScadss(SPD/HPD)Solve?I

求解$LL^{T}x=b$或者$LL^{H}x=b$。

接口定义

C Interface：

int KmlScadssSpdSolveSI(KmlScaSolverTask **pHandle, int nb, float *x, int ldx, float *b, int ldb);

int KmlScadssSpdSolveDI(KmlScaSolverTask **pHandle, int nb, double *x, int ldx, double *b, int ldb);

int KmlScadssHpdSolveCI(KmlScaSolverTask **pHandle, int nb, kml_complex_float *x, int ldx, kml_complex_float *b, int ldb);

int KmlScadssHpdSolveZI(KmlScaSolverTask **pHandle, int nb, kml_complex_double *x, int ldx, kml_complex_double *b, int ldb);

参数

参数名	类型	描述	输入/输出
pHandle	KmlScasolverTask **	求解器句柄，传入之前步骤的变量。	输入
nb	int	右端项矢量的个数。	输入

版权所有 © 华为技术有限公司

参数名	类型	描述	输入/输出
x	• 在KmlScadssSpdSolveSI中为float *。 • 在KmlScadssSpdSolveDI中为double *。 • 在KmlScadssHpdSolveCI中为kml_complex_float *。 • KmlScadssHpdSolveZI中为kml_complex_double *。	用于存储求解结果的稠密向量，其元素个数为ldx*nb，第k个矢量的第i个元素位置为x[i + k * ldx]。	输出
ldx	int	x的leading dimension大小，要求ldx ≥ max(1, n)。	输入
b	• 在KmlScadssSpdSolveSI中为float *。 • 在KmlScadssSpdSolveDI中为double *。 • 在KmlScadssHpdSolveCI中为kml_complex_float *。 • KmlScadssHpdSolveZI中为kml_complex_double *。	稠密向量存储右端项，其元素个数为ldb*nb，第k个矢量的第i个元素位置为b[i + k * ldb]。	输入
ldb	int	b的leading dimension大小，要求ldb ≥ max(1, n)。	输入

返回值

返回值	类型	描述
KMLSS_NO_ERROR	int	正常执行。
KMLSS_NULL_ARGUMENT	int	pHandle，x，b为空。

版权所有 © 华为技术有限公司

返回值	类型	描述
KMLSS_BAD_NB	int	输入参数nb小于等于0。
KMLSS_BAD_HANDLE	int	pHandle格式存在问题。
KMLSS_BAD_LDX	int	ldx取值不在有效范围内。
KMLSS_BAD_LDB	int	ldb取值不在有效范围内。
KMLSS_INVALID_CALL_ORDER	int	函数执行顺序错误，上一阶段非Factorize。

依赖

#include "kml_scadss.h"

示例

C Interface：

```
MPI_Init(NULL, NULL);
int size, rank;
MPI_Comm_size(MPI_COMM_WORLD, &size);//获取总进程数
MPI_Comm_rank(MPI_COMM_WORLD, &rank);//获取当前进程标识

int n = 8;
int ia[9] = {0, 3, 7, 9, 11, 13, 15, 16, 17};
int ja[17] = {0, 3, 4, 1, 2, 3, 5, 2, 7, 3, 6, 4, 5, 5, 7, 6, 7};
double a[17] = {1.0, 1.0, 2.0, 9.0, 2.0, 1.0, -3.0, 3.0, 2.0, 9.0, -5.0, 6.0, 1.0, 4.0, 1.0, 7.0, 2.0};
double b[8] = {4.0, 9.0, 7.0, 6.0, 9.0, 3.0, 2.0, 5.0};
double x[8];
KmlScasolverTask* matrix_handle = NULL;
int ierr = KmlScadssSpdInitDI(&matrix_handle, n, a, ja, ia, MPI_COMM_WORLD);
ierr = KmlScadssSpdAnalyzeDI(&matrix_handle);
ierr = KmlScadssSpdFactorizeDI(&matrix_handle);
int nrhs = 1;
int ldx=n, ldb=n;
ierr = KmlScadssSpdSolveDI(&matrix_handle, nrhs, x, ldx, b, ldb);
if (ierr != 0) {
    printf("\nERROR in KmlScadssSpdSolveDI: %d", ierr);
}
if (rank == 0) {
    for (int i = 0; i < n; i++) {
        printf("%lf ", x[i]);
    }
    printf("\n");
}
```

运行结果：

```
1.000000 1.000000 1.000000 1.000000 1.000000 1.000000 1.000000 1.000000
```

6.5.3.4.7.6 KmlScadss(SPD/HPD)Get?I?

获取与handle关联的整型值。

接口定义

C Interface：

 版权所有 © 华为技术有限公司

int KmlScadssSpdGetSIL(KmlScaSolverTask **pHandle, enum KML_SOLVER_PARAM param, int64_t *value, int nvalue);

int KmlScadssSpdGetDIL(KmlScaSolverTask **pHandle, enum KML_SOLVER_PARAM param, int64_t *value, int nvalue);

int KmlScadssHpdGetCIL(KmlScaSolverTask **pHandle, enum KML_SOLVER_PARAM param, int64_t *value, int nvalue);

int KmlScadssHpdGetZIL(KmlScaSolverTask **pHandle, enum KML_SOLVER_PARAM param, int64_t *value, int nvalue);

int KmlScadssSpdGetSII(KmlScaSolverTask **pHandle, enum KML_SOLVER_PARAM param, int *value, int nvalue);

int KmlScadssSpdGetDII(KmlScaSolverTask **pHandle, enum KML_SOLVER_PARAM param, int *value, int nvalue);

int KmlScadssHpdGetCII(KmlScaSolverTask **pHandle, enum KML_SOLVER_PARAM param, int *value, int nvalue);

int KmlScadssHpdGetZII(KmlScaSolverTask **pHandle, enum KML_SOLVER_PARAM param, int *value, int nvalue);

int KmlScadssSpdGetSIS(KmlScaSolverTask **pHandle, enum KML_SOLVER_PARAM param, float *value, int nvalue);

int KmlScadssSpdGetDID(KmlScaSolverTask **pHandle, enum KML_SOLVER_PARAM param, double *value, int nvalue);

int KmlScadssHpdGetCIS(KmlScaSolverTask **pHandle, enum KML_SOLVER_PARAM param, float *value, int nvalue);

int KmlScadssHpdGetZID(KmlScaSolverTask **pHandle, enum KML_SOLVER_PARAM param, double *value, int nvalue);

参数

参数名	类型	描述	输入/输出
pHandle	KmlScaSolverTask	求解器句柄，pHandle不可为空指针。	输入
param	enum KML_SOLVER_PARAM	• KMLSS_FILL_IN表示填入数目。 • KMLSS_PERM表示矩阵行列顺序，暂不支持。 • KMLSS_ITER_REF_STEPS表示迭代优化次数，暂不支持。	输入
value	int64_t	直接求解相关参数数组。	输出
nvalue	int	value数组元素个数。	输入

版权所有 © 华为技术有限公司

返回值

返回值	类型	描述
KMLSS_NO_ERROR	int	正常执行。
KMLSS_NULL_ARGUMENT	int	pHandle为空指针。
KMLSS_BAD_HANDLE	int	pHandle内部格式转换错误。
KMLSS_BAD_DATA_SIZE	int	KMLSS_FILL_IN数组长度nvalue设置错误。
KMLSS_INTERNAL_ERROR	int	内部数据返回空指针，KMLSS_FILL_IN获取失败。
KMLSS_BAD_SELECTOR	int	用户的KML_SOLVER_PARAM输入当前不支持，或在求解过程中不存在。

依赖

#include "kml_scadss.h"

示例

C Interface：

```
MPI_Init(NULL, NULL);
int size, rank;
MPI_Comm_size(MPI_COMM_WORLD, &size);//获取总进程数
MPI_Comm_rank(MPI_COMM_WORLD, &rank);//获取当前进程标识

int n = 8;
int ia[9] = {0, 3, 7, 9, 11, 13, 15, 16, 17};
int ja[17] = {0, 3, 4, 1, 2, 3, 5, 2, 7, 3, 6, 4, 5, 5, 7, 6, 7};
double a[17] = {1.0, 1.0, 2.0, 9.0, 2.0, 1.0, -3.0, 3.0, 2.0, 9.0, -5.0, 6.0, 1.0, 4.0, 1.0, 7.0, 2.0};
double b[8] = {4.0, 9.0, 7.0, 6.0, 9.0, 3.0, 2.0, 5.0};
double x[8];
KmlScasolverTask* matrix_handle = NULL;
int ierr = KmlScadssSpdInitDI(&matrix_handle, n, a, ja, ia, MPI_COMM_WORLD);
ierr = KmlScadssSpdAnalyzeDI(&matrix_handle);
ierr = KmlScadssSpdFactorizeDI(&matrix_handle);
int nrhs = 1;
int ldx=n, ldb=n;
ierr = KmlScadssSpdSolveDI(&matrix_handle, nrhs, x, ldx, b, ldb);
if (rank == 0) {
    int64_t nnz;
    ierr = KmlScadssSpdGetDII(&matrix_handle, KMLSS_FILL_IN, &nnz, 1);
    printf("NNZ = %ld\n", nnz);
}
```

运行结果：

```
NNZ = 40
```

6.5.3.4.7.7 KmlScadss(SPD/HPD)Clean?I

释放内部数据结构。

 版权所有 © 华为技术有限公司

接口定义

C Interface：

int KmlScadssSpdCleanSI(KmlScaSolverTask **pHandle);

int KmlScadssSpdCleanDI(KmlScaSolverTask **pHandle);

int KmlScadssHpdCleanCI(KmlScaSolverTask **pHandle);

int KmlScadssHpdCleanZI(KmlScaSolverTask **pHandle);

参数

参数名	类型	描述	输入/输出
pHandle	KmlScasolverTask **	求解器句柄，传入之前步骤的变量。	输入/输出

返回值

返回值	类型	描述
KMLSS_NO_ERROR	int	正常执行。
KMLSS_NULL_ARGUMENT	int	无效的参数。
KMLSS_BAD_HANDLE	int	pHandle格式存在问题。

依赖

#include "kml_scadss.h"

示例

C Interface：

```
MPI_Init(NULL, NULL);
int size, rank;
MPI_Comm_size(MPI_COMM_WORLD, &size);//获取总进程数
MPI_Comm_rank(MPI_COMM_WORLD, &rank);//获取当前进程标识

int n = 8;
int ia[9] = {0, 3, 7, 9, 11, 13, 15, 16, 17};
int ja[17] = {0, 3, 4, 1, 2, 3, 5, 2, 7, 3, 6, 4, 5, 5, 7, 6, 7};
double a[17] = {1.0, 1.0, 2.0, 9.0, 2.0, 1.0, -3.0, 3.0, 2.0, 9.0, -5.0, 6.0, 1.0, 4.0, 1.0, 7.0, 2.0};
double b[8] = {4.0, 9.0, 7.0, 6.0, 9.0, 3.0, 2.0, 5.0};
double x[8];
KmlScasolverTask* matrix_handle = NULL;
int ierr = KmlScadssSpdInitDI(&matrix_handle, n, a, ja, ia, MPI_COMM_WORLD);
ierr = KmlScadssSpdAnalyzeDI(&matrix_handle);
ierr = KmlScadssSpdFactorizeDI(&matrix_handle);
int nrhs = 1;
```

 版权所有 © 华为技术有限公司

```
int ldx=n, ldb=n;
ierr = KmlScadssSpdSolveDI(&matrix_handle, nrhs, x, ldx, b, ldb);
ierr = KmlScadssSpdCleanDI(&matrix_handle);
if (ierr != 0) {
    printf("\nERROR in KmlScadssSpdCleanDI: %d", ierr);
}
```

6.5.3.4.8 SCADSS

6.5.3.4.8.1 矩阵函数

?.1. KmlScasolverMatrixCreate

创建矩阵操作句柄。

说明

创建矩阵时是浅拷贝，用户需要保证在KmlScasolverMatrixDestroy前不释放通过参数store指定的内存。

接口定义

C Interface：

int KmlScasolverMatrixCreate(KmlScasolverMatrix **pMatrix, KmlSolverMatrixStore *store, const KmlSolverMatrixOption *basicOptions, const KmlScasolverMatrixOption *scaOptions);

参数

参数名	类型	描述	输入/输出
pMatrix	KmlScasolverMatrix **	矩阵句柄，pMatrix不可为空指针。	输入/输出
store	KmlSolverMatrixStore *	矩阵格式信息结构体，仅描述当前进程本地矩阵，不可为空指针，具体内容如4.3.3.10.1.1-表1所示。	输入
basicOptions	KmlSolverMatrixOption *	矩阵基础配置选项，不可为空指针，具体内容如4.3.3.10.1.1-表2所示。	输入
scaOptions	KmlScasolverMatrixOption *	矩阵分布式配置选项，所有进程保存的矩阵需保证能够组成完整矩阵且无重复部分，不可为空指针，具体内容如表1所示。	输入

版权所有 © 华为技术有限公司

表 6-58 KmlScasolverMatrixOption 结构体字段描述

字段名	类型	描述
fieldMask	uint64_t	配置生效的字段掩码。非0的位代表配置对应字段指定的值，0的位代表使用默认值，可配置的掩码位见具体字段的说明。
globalNumRows	int64_t	全局矩阵的行数。如果未指定，则默认值与KmlSolverMatrixStore::nrows相同。 掩码位：KMLSS_MATRIX_OPTIONS_GLOBAL_NROWS
globalNumCols	int64_t	全局矩阵的列数。如果未指定，则默认值与KmlSolverMatrixStore::ncols相同。 掩码位：KMLSS_MATRIX_OPTIONS_GLOBAL_NCOLS
partition	KmlScadssMatrixPartition	全局矩阵在进程间的划分，具体内容如**表2**所示。 掩码位：KMLSS_MATRIX_OPTIONS_PARTITION

表 6-59 KmlScadssMatrixPartition 结构体字段描述

字段名	类型	描述
type	KmlScasolverMatrixPartitionType	矩阵在进程间的划分方式，有如下值。 • KMLSS_MATRIX_PARTITION_ROW表示按行划分。 • KMLSS_MATRIX_PARTITION_COL表示按列划分。 • KMLSS_MATRIX_PARTITION_NZ表示按非零元素划分。
localBegin	int64_t	表示当前进程在全局矩阵中的偏移量，意义取决于划分方式type，具体如下。 • KMLSS_MATRIX_PARTITION_ROW时表示第一个本进程矩阵行的全局行号。 • KMLSS_MATRIX_PARTITION_COL时表示第一个本进程矩阵列的全局列号。 • KMLSS_MATRIX_PARTITION_NZ时则忽略本参数。

版权所有 © 华为技术有限公司

返回值

返回值	类型	描述
KMLSS_NO_ERROR	int	正常执行。
KMLSS_INTERNAL_ERROR	int	内部错误。
KMLSS_NULL_ARGUMENT	int	pMatrix，store，basicOptions，scaOptions存在空参数。

依赖

#include "kml_scadss.h"

?.2. KmlScasolverMatrixSetValue

设置或修改矩阵数值。

说明

指定的矩阵数值应当与创建该矩阵句柄时指定的非零元结构对应，该接口应用于矩阵结构不变但值变化的场景。通过该接口更新value数组后，原value数组空间可释放。

接口定义

C Interface：

int KmlScasolverMatrixSetValue(KmlScasolverMatrix *matrix, void *values);

参数

参数名	类型	描述	输入/输出
matrix	KmlScasolverMatrix *	由KmlScasolverMatrixCreate函数创建的矩阵句柄。	输入
values	void *	矩阵数值数组，不可为空指针。	输入

返回值

返回值	类型	描述
KMLSS_NO_ERROR	int	正常执行。
KMLSS_INTERNAL_ERROR	int	内部错误。
KMLSS_NULL_ARGUMENT	int	matrix，values存在空参数。

 版权所有 © 华为技术有限公司

依赖

#include "kml_scadss.h"

?.3. KmlScasolverMatrixDestroy

销毁由KmlScasolverMatrixCreate函数创建的矩阵句柄。

接口定义

C Interface：

int KmlScasolverMatrixDestroy(KmlScasolverMatrix **pMatrix);

参数

参数名	类型	描述	输入/输出
pMatrix	KmlScasolverMatrix **	由KmlScasolverMatrixCreate函数创建的矩阵句柄。	输入/输出

返回值

返回值	类型	描述
KMLSS_NO_ERROR	int	正常执行。
KMLSS_INTERNAL_ERROR	int	内部错误。
KMLSS_NULL_ARGUMENT	int	pMatrix为空参数。

依赖

#include "kml_scadss.h"

6.5.3.4.8.2 求解器函数

?.1. KmlScadssInit

创建求解器操作句柄。

接口定义

C Interface：

 版权所有 © 华为技术有限公司

```
int KmlScadssInit(KmlScadssSolver **pSolver, const KmlDssInitOption
*basicOptions, const KmlScadssInitOption *scaOptions);
```

参数

参数名	类型	描述	输入/输出
pSolver	KmlScadssSolver **	求解器句柄，传入之前步骤的变量。	输入/输出
basicOptions	KmlDssInitOption *	求解器基础配置选项，不可为空指针，具体内容如4.3.3.10.2.1-表1所示。	输入
scaOptions	KmlScadssInitOption *	求解器分布式配置选项，具体内容如表1所示。	输入

表 6-60 KmlScadssInitOption 结构体字段描述

字段名	类型	描述
fieldMask	uint64_t	配置生效的字段掩码。非0的位代表配置对应字段指定的值，0的位代表使用默认值，可配置的掩码位见具体字段的说明。
comm	MPI_Comm	MPI通信子。 掩码位：KMLSCADSS_OPTIONS_COMM
autotune	int8_t	指示是否允许求解器实例自动调整输入的进程与线程配比以获得更好的性能，但该参数不会调整Fillin Reduction的进程线程数。有如下值。 ● 0表示关闭自动调整，默认选项。 ● 1表示开启自动调整。 掩码位：KMLSCADSS_OPTIONS_AUTOTUNE

返回值

返回值	类型	描述
KMLSS_NO_ERROR	int	正常执行。
KMLSS_INTERNAL_ERROR	int	内部错误。
KMLSS_NULL_ARGUMENT	int	pSolver，basicOptions，scaOptions存在空参数。

版权所有 © 华为技术有限公司

依赖

#include "kml_scadss.h"

?.2. KmlScadssAnalyze

分析输入矩阵，包括Reordering、符号分解等。

接口定义

C Interface：

int KmlScadssAnalyze(KmlScadssSolver *solver, KmlScasolverMatrix *matrix, const KmlDssAnalyzeOption *basicOptions, const KmlScadssAnalyzeOption *scaOptions);

参数

参数名	类型	描述	输入/输出
solver	KmlScadssSolver *	由KmlScadssInit创建的求解器句柄。	输入
matrix	KmlScasolverMatrix *	矩阵句柄，由KmlScasolverMatrixCreate函数创建，不可为空指针。	输入
basicOptions	KmlDssAnalyzeOption *	分析阶段基础配置选项，不可为空指针，具体内容如4.3.3.10.2.2-表1所示。	输入
scaOptions	KmlScadssAnalyzeOption *	分析阶段分布式配置选项，不可为空指针，具体内容如表1所示。	输入

表 6-61 KmlScadssAnalyzeOption 结构体字段描述

字段名	类型	描述
fieldMask	uint64_t	配置生效的字段掩码。非0的位代表配置对应字段指定的值，0的位代表使用默认值，可配置的掩码位见具体字段的说明。
fr	KmlScadssFRAlgo	Fill-in reduction算法类型，若不指定此掩码，则使用分析阶段基础配置项basicOptions的相关配置项。有如下值。 • KMLSCADSS_FR_USER表示使用自定义Fill-in reduction函数。 掩码位：KMLSCADSS_ANALYSE_OPTIONS_FR

版权所有 © 华为技术有限公司

字段名	类型	描述
userFR	KmlScadssUserFRFunc	用户自定义的分布式Fill-in reduction算法，使用KmlScadssUserFRFunc结构体描述，见表2。 ● 如果fr字段掩码KMLSCADSS_FR_USER未指定，则将忽略该参数。 ● 如果fr字段掩码为KMLSCADSS_FR_USER，但userFR.func为NULL，则会出现错误. 掩码位：KMLSCADSS_ANALYSE_OPTIONS_USER_FR

表 6-62 KmlScadssAnalyzeOption 结构体字段描述

字段名	类型	描述
func	int (*)(void *arg, const KmlScadssMatrixPartition *partition, const KmlSolverMatrixStore *store, void *iPerm, MPI_Comm comm);	用户自定义的分布式Fill-in reduction函数，该函数的原型为int (*func)(void *arg, const KmlScadssMatrixPartition *partition, const KmlSolverMatrixStore *store, void *iPerm, MPI_Comm comm);
arg	void *	用户相关的参数，可以用作用户上下文。在调用func函数时将此作为第一个参数。

返回值

返回值	类型	描述
KMLSS_NO_ERROR	int	正常执行。
KMLSS_INTERNAL_ERROR	int	内部错误。
KMLSS_NULL_ARGUMENT	int	solver，matrix，basicOptions，scaOptions存在空参数。

依赖

#include "kml_scadss.h"

 版权所有 © 华为技术有限公司

?.3. KmlScadssFactorize

对矩阵做数值分解，需先使用KmlScadssAnalyze进行分析。

接口定义

C Interface：

int KmlScadssFactorize(KmlScadssSolver *solver, KmlScasolverMatrix *matrix, const KmlDssFactorizeOption *basicOptions, const KmlScadssFactorizeOption *scaOptions);

参数

参数名	类型	描述	输入/输出
solver	KmlScadssSolver *	由KmlScadssInit创建的求解器句柄。	输入
matrix	KmlSolverMatrix *	矩阵句柄，由KmlSolverMatrixCreate函数创建，不可为空指针。	输入
basicOptions	KmlDssFactorizeOption *	分解阶段基础配置选项，不可为空指针，具体内容如4.3.3.10.2.3-表1所示。	输入
scaOptions	KmlScadssFactorizeOption *	分解阶段分布式配置选项，具体内容如表1所示（暂无配置项）。	输入

表 6-63 KmlScadssFactorizeOption 结构体字段描述

字段名	类型	描述
fieldMask	uint64_t	配置生效的字段掩码。非0的位代表配置对应字段指定的值，0的位代表使用默认值，暂无可配置的掩码位。

返回值

返回值	类型	描述
KMLSS_NO_ERROR	int	正常执行。
KMLSS_INTERNAL_ERROR	int	内部错误。
KMLSS_NULL_ARGUMENT	int	solver，matrix，basicOptions，scaOptions存在空参数。

版权所有 © 华为技术有限公司

依赖

#include "kml_scadss.h"

?.4. KmlScadssSolve

对矩阵做回代求解，需先使用KmlScadssFactorize进行分解。

接口定义

C Interface：

int KmlScadssSolve(KmlScadssSolver *solver, KmlScasolverMatrix *b, KmlScasolverMatrix *x, const KmlDssSolveOption *basicOptions, const KmlScadssSolveOption *scaOptions);

参数

参数名	类型	描述	输入/输出
solver	KmlScadssSolver *	由KmlScadssInit创建的求解器句柄。	输入
b	KmlScasolverMatrix *	右端向量结构体指针，本质是矩阵，由KmlScasolverMatrixCreate函数创建，不可为空指针。	输入
x	KmlScasolverMatrix *	解空间结构体指针，本质是矩阵，由KmlScasolverMatrixCreate函数创建，输入不可为空指针，输出为包含求解出的x的数值数组。	输出
basicOptions	KmlDssSolveOption *	分解阶段基础配置选项，不可为空指针，具体内容如4.3.3.10.2.4-表1所示。	输入
scaOptions	KmlScadssSolveOption *	分解阶段分布式配置选项，具体内容如表1所示（暂无配置项）。	输入

表 6-64 KmlScadssFactorizeOption 结构体字段描述

字段名	类型	描述
fieldMask	uint64_t	配置生效的字段掩码。非0的位代表配置对应字段指定的值，0的位代表使用默认值，暂无可配置的掩码位。

版权所有 © 华为技术有限公司

返回值

返回值	类型	描述
KMLSS_NO_ERROR	int	正常执行。
KMLSS_INTERNAL_ERROR	int	内部错误。
KMLSS_NULL_ARGUMENT	int	solver，b，x，basicOptions，scaOptions存在空参数。

依赖

#include "kml_scadss.h"

?.5. KmlScadssClean

销毁由KmlScadssInit创建的求解器句柄。

接口定义

C Interface：

int KmlScadssClean(KmlScadssSolver **pSolver);

参数

参数名	类型	描述	输入/输出
pSolver	KmlScadssSolver **	由KmlScadssInit创建的求解器句柄。	输入/输出

返回值

返回值	类型	描述
KMLSS_NO_ERROR	int	正常执行。
KMLSS_NULL_ARGUMENT	int	pSolver为空参数。

依赖

#include "kml_scadss.h"

?.6. KmlScadssQuery

查询求解器内部信息。

 版权所有 © 华为技术有限公司

接口定义

C Interface：

int KmlScadssQuery(KmlScadssSolver *solver, KmlDssInfo *basicInfo, KmlScadssInfo *scaInfo);

参数

参数名	类型	描述	输入/输出
solver	KmlScadssSolver *	由KmlScadssInit创建的求解器句柄。	输入
basicInfo	KmlDssInfo *	基础信息查询结构体指针，输入不可为空指针，输出为查询后的结果，具体内容如**4.3.3.10.2.6-表1**所示。	输入/输出
scaInfo	KmlScadssInfo *	分布式信息查询结构体指针，输入不可为空指针，输出为查询后的结果，具体内容如**表1**所示（暂无配置项）。	输入/输出

表 6-65 KmlScadssInfo 结构体字段描述

字段名	类型	描述
fieldMask	uint64_t	配置生效的字段掩码。非0的位代表配置对应字段指定的值，0的位代表使用默认值，暂无可配置的掩码位。

返回值

返回值	类型	描述
KMLSS_NO_ERROR	int	正常执行。
KMLSS_INTERNAL_ERROR	int	内部错误。
KMLSS_NULL_ARGUMENT	int	solver，basicInfo，scaInfo存在空参数。

依赖

#include "kml_scadss.h"

?.7. 示例

C Interface：

 版权所有 © 华为技术有限公司

```
#include <stdio.h>
#include <kml_scadss.h>

int Run(MPI_Comm comm)
{
    int ierr;
    int rank;
    MPI_Comm_rank(comm, &rank);

    int n = 8;
    int nrhs = 1;

    // Create matrix A
    int ia[9] = {0, 2, 4, 6, 7, 8, 10, 12, 14};
    int ja[14] = {0, 7, 1, 6, 2, 5, 3, 4, 2, 5, 1, 6, 0, 7};
    double a[14] = {1.0, 2.0, -2.0, 3.0, 3.0, 4.0, -4.0, 5.0, 4.0, -6.0, 3.0, 7.0, 2.0, 8.0};
    KmlSolverMatrixStore storeA;
    storeA.indexType = KMLSS_INDEX_INT32;
    storeA.valueType = KMLSS_VALUE_FP64;
    storeA.format = KMLSS_MATRIX_STORE_CSR;
    if (rank == 0) {
        storeA.nRow = n;
        storeA.nCol = n;
        storeA.csr.rowOffset = ia;
        storeA.csr.colIndex = ja;
        storeA.csr.value = a;
    } else {
        storeA.nRow = 0;
        storeA.nCol = 0;
        storeA.csr.rowOffset = nullptr;
        storeA.csr.colIndex = nullptr;
        storeA.csr.value = nullptr;
    }

    KmlSolverMatrixOption optA;
    optA.fieldMask = KMLSS_MATRIX_OPTION_TYPE;
    optA.type = KMLSS_MATRIX_GEN;

    KmlScasolverMatrixOption scaOptA;
    if (rank == 0) {
        scaOptA.fieldMask = KMLSS_MATRIX_OPTIONS_GLOBAL_NROWS |
                            KMLSS_MATRIX_OPTIONS_GLOBAL_NCOLS |
                            KMLSS_MATRIX_OPTIONS_PARTITION;
        scaOptA.partition.type = KMLSS_MATRIX_PARTITION_ROW;
        scaOptA.globalNumRows = n;
        scaOptA.globalNumCols = n;
        scaOptA.partition.localBegin = 0;
    } else {
        scaOptA.fieldMask = 0;
    }

    KmlScasolverMatrix *A;
    ierr = KmlScasolverMatrixCreate(&A, &storeA, &optA, &scaOptA);
    if (ierr != KMLSS_NO_ERROR) {
        printf("ERROR when create A: %d\n", ierr);
        return 1;
    }

    // Create vector b
    double b[8] = {3.0, 1.0, 7.0, -4.0, 5.0, -2.0, 10.0, 10.0};
    KmlSolverMatrixStore storeB;
    storeB.indexType = KMLSS_INDEX_INT32;
    storeB.valueType = KMLSS_VALUE_FP64;
    storeB.format = KMLSS_MATRIX_STORE_DENSE_COL_MAJOR;
    if (rank == 0) {
        storeB.nRow = n;
        storeB.nCol = nrhs;
        storeB.dense.value = b;
        storeB.dense.ld = n;
```

 版权所有 © 华为技术有限公司

```
} else {
    storeB.nRow = 0;
    storeB.nCol = 0;
    storeB.dense.value = nullptr;
    storeB.dense.ld = 0;
}

KmlSolverMatrixOption optB;
optB.fieldMask = KMLSS_MATRIX_OPTION_TYPE;
optB.type = KMLSS_MATRIX_GEN;

KmlScasolverMatrixOption scaOptB;
if (rank == 0) {
    scaOptB.fieldMask = KMLSS_MATRIX_OPTIONS_GLOBAL_NROWS |
                        KMLSS_MATRIX_OPTIONS_GLOBAL_NCOLS |
                        KMLSS_MATRIX_OPTIONS_PARTITION;
    scaOptB.partition.type = KMLSS_MATRIX_PARTITION_ROW;
    scaOptB.partition.localBegin = 0;
    scaOptB.globalNumRows = n;
    scaOptB.globalNumCols = nrhs;
} else {
    scaOptB.fieldMask = 0;
}

KmlScasolverMatrix *B;
ierr = KmlScasolverMatrixCreate(&B, &storeB, &optB, &scaOptB);
if (ierr != KMLSS_NO_ERROR) {
    printf("ERROR when create b: %d\n", ierr);
    return 1;
}

// Create vector x
double x[8] = {0};
KmlSolverMatrixStore storeX;
storeX.indexType = KMLSS_INDEX_INT32;
storeX.valueType = KMLSS_VALUE_FP64;
storeX.format = KMLSS_MATRIX_STORE_DENSE_COL_MAJOR;
if (rank == 0) {
    storeX.nRow = n;
    storeX.nCol = nrhs;
    storeX.dense.value = x;
    storeX.dense.ld = n;
} else {
    storeX.nRow = 0;
    storeX.nCol = 0;
    storeX.dense.value = nullptr;
    storeX.dense.ld = 0;
}

KmlSolverMatrixOption optX;
optX.fieldMask = KMLSS_MATRIX_OPTION_TYPE;
optX.type = KMLSS_MATRIX_GEN;

KmlScasolverMatrixOption scaOptX;
if (rank == 0) {
    scaOptX.fieldMask = KMLSS_MATRIX_OPTIONS_GLOBAL_NROWS |
                        KMLSS_MATRIX_OPTIONS_GLOBAL_NCOLS |
                        KMLSS_MATRIX_OPTIONS_PARTITION;
    scaOptX.partition.type = KMLSS_MATRIX_PARTITION_ROW;
    scaOptX.partition.localBegin = 0;
    scaOptX.globalNumRows = n;
    scaOptX.globalNumCols = nrhs;
} else {
    scaOptX.fieldMask = 0;
}


KmlScasolverMatrix *X;
ierr = KmlScasolverMatrixCreate(&X, &storeX, &optX, &scaOptX);
```

 版权所有 © 华为技术有限公司

```
    if (ierr != KMLSS_NO_ERROR) {
        printf("ERROR when create x: %d\n", ierr);
        return 1;
    }

    // Init solver
    KmlDssInitOption opt;
    opt.fieldMask = KMLDSS_INIT_OPTION_BWR_MODE | KMLDSS_INIT_OPTION_NTHREADS;
    opt.bwrMode = KMLDSS_BWR_OFF;
    opt.nThreads = 32;

    KmlScadssInitOption scaOpt;
    scaOpt.fieldMask = KMLSCADSS_OPTIONS_COMM;
    scaOpt.comm = comm;

    KmlScadssSolver *solver;
    ierr = KmlScadssInit(&solver, &opt, &scaOpt);
    if (ierr != KMLSS_NO_ERROR) {
        printf("ERROR in KmlDssInit: %d\n", ierr);
        return ierr;
    }

    // Analyze
    KmlDssAnalyzeOption optAnalyze;
    optAnalyze.fieldMask = KMLDSS_ANALYZE_OPTION_MATCHING_TYPE |
KMLDSS_ANALYZE_OPTION_RDR_TYPE |
                           KMLDSS_ANALYZE_OPTION_NTHREADS_RDR;
    optAnalyze.matchingType = KMLDSS_MATCHING_OFF;
    optAnalyze.rdrType = KMLDSS_RDR_KRDR;
    optAnalyze.nThreadsRdr = 1;

    KmlScadssAnalyzeOption scaOptAnalyze;
    scaOptAnalyze.fieldMask = 0;

    ierr = KmlScadssAnalyze(solver, A, &optAnalyze, &scaOptAnalyze);
    if (ierr != KMLSS_NO_ERROR) {
        printf("ERROR in KmlDssAnalyze: %d\n", ierr);
        return ierr;
    }

    // Factorize
    KmlDssFactorizeOption optFact;
    optFact.fieldMask = KMLDSS_FACTORIZE_OPTION_PERTURBATION_THRESHOLD;
    optFact.perturbationThreshold = 1e-8;

    KmlScadssFactorizeOption scaOptFact;
    scaOptFact.fieldMask = 0;

    ierr = KmlScadssFactorize(solver, A, &optFact, &scaOptFact);
    if (ierr != KMLSS_NO_ERROR) {
        printf("ERROR in KmlDssFactorize: %d\n", ierr);
        return ierr;
    }

    // Solve
    KmlDssSolveOption optSolve;
    optSolve.fieldMask = KMLDSS_SOLVE_OPTION_SOLVE_STAGE |
KMLDSS_SOLVE_OPTION_REFINE_METHOD;
    optSolve.stage = KMLDSS_SOLVE_ALL;
    optSolve.refineMethod = KMLDSS_REFINE_OFF;

    KmlScadssSolveOption scaOptSolve;
    scaOptSolve.fieldMask = 0;

    ierr = KmlScadssSolve(solver, B, X, &optSolve, &scaOptSolve);
    if (ierr != KMLSS_NO_ERROR) {
        printf("ERROR in KmlDssSolve: %d\n", ierr);
        return ierr;
    }
```

 版权所有 © 华为技术有限公司

```
    // Output result x
    if (rank == 0) {
        printf("Result of first factorize and solve:\n");
        for (int i = 0; i < n; i++) {
            printf("%lf ", x[i]);
        }
        printf("\n");
    }

    // Set new values of A
    double a1[14] = {2.0, 3.0, -3.0, 4.0, 4.0, 5.0, -5.0, 6.0, 5.0, -7.0, 4.0, 8.0, 3.0, 9.0};
    KmlScasolverMatrixSetValue(A, a1);

    // Factorize with new values
    ierr = KmlScadssFactorize(solver, A, &optFact, &scaOptFact);
    if (ierr != KMLSS_NO_ERROR) {
        printf("ERROR in KmlDssFactorize: %d\n", ierr);
        return ierr;
    }

    // Set new values of B
    double b1[8] = {1.0, 1.0, 1.0, 1.0, 1.0, 1.0, 1.0, 1.0};
    KmlScasolverMatrixSetValue(B, b1);

    // Solve with new values
    ierr = KmlScadssSolve(solver, B, X, &optSolve, &scaOptSolve);
    if (ierr != KMLSS_NO_ERROR) {
        printf("ERROR in KmlDssSolve: %d\n", ierr);
        return ierr;
    }

    // Output new result x
    if (rank == 0) {
        printf("Result of second factorize and solve:\n");
        for (int i = 0; i < n; i++) {
            printf("%lf ", x[i]);
        }
        printf("\n");
    }

    // Query
    KmlDssInfo info;
    info.fieldMask = KMLDSS_INFO_PEAK_MEM;
    KmlScadssInfo scaInfo;
    scaInfo.fieldMask = 0;
    ierr = KmlScadssQuery(solver, &info, &scaInfo);
    if (ierr != KMLSS_NO_ERROR) {
        printf("ERROR in KmlDssQuery: %d\n", ierr);
        return ierr;
    }
    if (rank == 0) {
        printf("Peak memory is %ld Byte\n", info.peakMem);
    }

    // Destroy
    KmlScadssClean(&solver);

    KmlScasolverMatrixDestroy(&A);
    KmlScasolverMatrixDestroy(&B);
    KmlScasolverMatrixDestroy(&X);

    return 0;
}

int main(int argc, char **argv)
{
    MPI_Init(&argc, &argv);
    Run(MPI_COMM_WORLD);
```

 版权所有 © 华为技术有限公司

```
    MPI_Finalize();
    return 0;
}
```

运行结果：

```
Result of first factorize and solve:
1.000000 1.000000 1.000000 1.000000 1.000000 1.000000 1.000000 1.000000
Result of second factorize and solve:
0.666667 -0.100000 0.226415 -0.200000 0.166667 0.018868 0.175000 -0.111111
Peak memory is 102376 Byte
```

6.5.3.4.9 SCADSS-LU (DEPRECATED)

集群版直接法推荐使用**SCADSS**。

6.5.3.4.9.1 KmlScadssGENInit?I

初始化数据结构，并将用户提供的系数矩阵关联到求解系统中，初始化集群版的LU方法时使用上三角的CSR格式。

接口定义

C Interface：

int KmlScadssGenInitSI(KmlScasolverTask **pHandle, int n, float *a, int *ja, int *ia, MPI_Comm comm);

int KmlScadssGenInitDI(KmlScasolverTask **pHandle, int n, double *a, int *ja, int *ia, MPI_Comm comm);

int KmlScadssGenInitCI(KmlScasolverTask **pHandle, int n, kml_complex_float *a, int *ja, int *ia, MPI_Comm comm);

int KmlScadssGenInitZI(KmlScasolverTask **pHandle, int n, kml_complex_double *a, int *ja, int *ia, MPI_Comm comm);

参数

参数名	类型	描述	输入/输出
pHandle	KmlScasolverTask **	求解器句柄，传入之前步骤的变量。	输入/输出
n	int	n为矩阵A的维度	输入

版权所有 © 华为技术有限公司

参数名	类型	描述	输入/输出
a	• 在KmlScadssGenInitSI中为float *。 • 在KmlScadssGenInitDI中为double *。 • 在KmlScadssGenInitCI中为kml_complex_float *。 • 在KmlScadssGenInitZI中为kml_complex_double *。	a为维度为非零元素的个数的数组，a[i]为第i个非零元的值。该数组的生命周期与handle相同。	输入
ja	int *	ja为CSR格式的列索引数组，ja[i]为第i个非零元的列号。该数组的生命周期与handle相同。	输入
ia	int *	ia为CSR格式的行偏移数组，ia[i]为第i行第一个非零元在所有非零元中的索引，ia[0]=0。该数组的生命周期与handle相同。	输入
comm	MPI_Comm	为MPI进程通信域。	输入

返回值

返回值	类型	描述
KMLSS_NO_ERROR	int	正常执行。
KMLSS_NULL_ARGUMENT	int	无效的参数。
KMLSS_BAD_HANDLE	int	pHandle格式存在问题。

依赖

#include "kml_scadss.h"

示例

C Interface：

 版权所有 © 华为技术有限公司

```
MPI_Init(NULL, NULL);
int size, rank;
MPI_Comm_size(MPI_COMM_WORLD, &size);//获取总进程数
MPI_Comm_rank(MPI_COMM_WORLD, &rank);//获取当前进程标识

int n = 8;
int ia[9] = {0, 3, 7, 9, 11, 13, 15, 16, 17};
int ja[17] = {0, 3, 4, 1, 2, 3, 5, 2, 7, 3, 6, 4, 5, 5, 7, 6, 7};
double a[17] = {1.0, 1.0, 2.0, 9.0, 2.0, 1.0, -3.0, 3.0, 2.0, 9.0, -5.0, 6.0, 1.0, 4.0, 1.0, 7.0, 2.0};
double b[8] = {4.0, 9.0, 7.0, 6.0, 9.0, 3.0, 2.0, 5.0};
double x[8];
KmlScasolverTask* matrix_handle = NULL;
int ierr = KmlScadssGenInitDI(&matrix_handle, n, a, ja, ia, MPI_COMM_WORLD);
ierr = KmlScadssGenAnalyzeDI(&matrix_handle);
ierr = KmlScadssGenFactorizeDI(&matrix_handle);
int nrhs = 1;
int ldx=n, ldb=n;
ierr = KmlScadssGenSolveDI(&matrix_handle, nrhs, x, ldx, b, ldb);
ierr = KmlScadssGenCleanDI(&matrix_handle);
if (ierr != 0) {
    printf("\nERROR in KmlScadssGenCleanDI: %d", ierr);
}
```

6.5.3.4.9.2 KmlScadssGENSet?I?

设置与handle关联的值。

接口定义

C Interface：

int KmlScadssGenSetSII(KmlScaSolverTask **pHandle, enum KML_SOLVER_PARAM param, int *value, int nvalue);

int KmlScadssGenSetDII(KmlScaSolverTask **pHandle, enum KML_SOLVER_PARAM param, int *value, int nvalue);

int KmlScadssGenSetCII(KmlScaSolverTask **pHandle, enum KML_SOLVER_PARAM param, int *value, int nvalue);

int KmlScadssGenSetZII(KmlScaSolverTask **pHandle, enum KML_SOLVER_PARAM param, int *value, int nvalue);

int KmlScadssGenSetDID(KmlScaSolverTask **pHandle, enum KML_SOLVER_PARAM param, double *value, int nvalue);

int KmlScadssGenSetZID(KmlScaSolverTask **pHandle, enum KML_SOLVER_PARAM param, double *value, int nvalue);

int KmlScadssGenSetSIS(KmlScaSolverTask **pHandle, enum KML_SOLVER_PARAM param, float *value, int nvalue);

int KmlScadssGenSetCIS(KmlScaSolverTask **pHandle, enum KML_SOLVER_PARAM param, float *value, int nvalue);

版权所有 © 华为技术有限公司

参数

参数名	类型	描述	输入/输出
pHandle	KmlScaSolverTask	求解器句柄，pHandle不可为空指针。	输入
param	enum KML_SOLVER_PARAM	• KMLSS_REFINEMENT_MAX_STEPS表示REFINEMENT的最大次数。 • KMLSS_PERM表示矩阵行列顺序，暂不支持。 • KMLSS_ITER_REF_STEPS表示迭代优化次数，暂不支持。	输入
value	int *	直接求解相关参数数组。	输出
nvalue	int	value数组元素个数。	输入

返回值

返回值	类型	描述
KMLSS_NO_ERROR	int	正常执行。
KMLSS_NULL_ARGUMENT	int	pHandle为空指针。
KMLSS_BAD_HANDLE	int	pHandle内部格式转换错误。
KMLSS_BAD_DATA_SIZE	int	KMLSS_FILL_IN数组长度nvalue设置错误。
KMLSS_INTERNAL_ERROR	int	内部数据返回空指针，KMLSS_FILL_IN获取失败。
KMLSS_BAD_SELECTOR	int	用户的KML_SOLVER_PARAM输入当前不支持，或在求解过程中不存在。

依赖

#include "kml_scadss.h"

示例

C Interface：

```
MPI_Init(NULL, NULL);
int size, rank;
MPI_Comm_size(MPI_COMM_WORLD, &size);//获取总进程数
MPI_Comm_rank(MPI_COMM_WORLD, &rank);//获取当前进程标识
```

 版权所有 © 华为技术有限公司

```
int n = 8;
int ia[9] = {0, 3, 7, 9, 11, 13, 15, 16, 17};
int ja[17] = {0, 3, 4, 1, 2, 3, 5, 2, 7, 3, 6, 4, 5, 5, 7, 6, 7};
double a[17] = {1.0, 1.0, 2.0, 9.0, 2.0, 1.0, -3.0, 3.0, 2.0, 9.0, -5.0, 6.0, 1.0, 4.0, 1.0, 7.0, 2.0};
double b[8] = {4.0, 9.0, 7.0, 6.0, 9.0, 3.0, 2.0, 5.0};
double x[8];
KmlScasolverTask* matrix_handle = NULL;
int ierr = KmlScadssGenInitDI(&matrix_handle, n, a, ja, ia, MPI_COMM_WORLD);
ierr = KmlScadssGenSetDII(&matrix_handle, KMLSS_REFINEMENT_MAX_STEPS, &max_refinement_steps , 1);
ierr = KmlScadssGenAnalyzeDI(&matrix_handle);
ierr = KmlScadssGenFactorizeDI(&matrix_handle);
int nrhs = 1;
int ldx=n, ldb=n;
ierr = KmlScadssGenSolveDI(&matrix_handle, nrhs, x, ldx, b, ldb);
```

6.5.3.4.9.3 KmlScadssGENAnalyze?I

分析数据结构，对稀疏矩阵进行重排和符号分解

接口定义

C Interface：

int KmlScadssGenAnalyzeSI(KmlScasolverTask **pHandle);

int KmlScadssGenAnalyzeDI(KmlScasolverTask **pHandle);

int KmlScadssGenAnalyzeCI(KmlScasolverTask **pHandle);

int KmlScadssGenAnalyzeZI(KmlScasolverTask **pHandle);

参数

参数名	类型	描述	输入/输出
pHandle	KmlScasolverTask **	求解器句柄，传入之前步骤的变量。	输入/输出

返回值

返回值	类型	描述
KMLSS_NO_ERROR	int	正常执行。
KMLSS_NULL_ARGUMENT	int	无效的参数。
KMLSS_BAD_HANDLE	int	pHandle格式存在问题。

依赖

#include "kml_scadss.h"

 版权所有 © 华为技术有限公司

示例

C Interface：

```
MPI_Init(NULL, NULL);
int size, rank;
MPI_Comm_size(MPI_COMM_WORLD, &size);//获取总进程数
MPI_Comm_rank(MPI_COMM_WORLD, &rank);//获取当前进程标识

int n = 8;
int ia[9] = {0, 3, 7, 9, 11, 13, 15, 16, 17};
int ja[17] = {0, 3, 4, 1, 2, 3, 5, 2, 7, 3, 6, 4, 5, 5, 7, 6, 7};
double a[17] = {1.0, 1.0, 2.0, 9.0, 2.0, 1.0, -3.0, 3.0, 2.0, 9.0, -5.0, 6.0, 1.0, 4.0, 1.0, 7.0, 2.0};
double b[8] = {4.0, 9.0, 7.0, 6.0, 9.0, 3.0, 2.0, 5.0};
double x[8];
KmlScasolverTask* matrix_handle = NULL;
int ierr = KmlScadssGenInitDI(&matrix_handle, n, a, ja, ia, MPI_COMM_WORLD);
ierr = KmlScadssGenAnalyzeDI(&matrix_handle);
ierr = KmlScadssGenFactorizeDI(&matrix_handle);
int nrhs = 1;
int ldx=n, ldb=n;
ierr = KmlScadssGenSolveDI(&matrix_handle, nrhs, x, ldx, b, ldb);
ierr = KmlScadssGenCleanDI(&matrix_handle);
if (ierr != 0) {
    printf("\nERROR in KmlScadssGenCleanDI: %d", ierr);
}
```

6.5.3.4.9.4 KmlScadssGENFactorize?I

实现一般矩阵的LU分解。

接口定义

C Interface：

int KmlScadssGenFactorizeSI(KmlScasolverTask **pHandle);

int KmlScadssGenFactorizeDI(KmlScasolverTask **pHandle);

int KmlScadssGenFactorizeCI(KmlScasolverTask **pHandle);

int KmlScadssGenFactorizeZI(KmlScasolverTask **pHandle);

参数

参数名	类型	描述	输入/输出
pHandle	KmlScasolverTask **	求解器句柄，传入之前步骤的变量。	输入/输出

返回值

返回值	类型	描述
KMLSS_NO_ERROR	int	正常执行。
KMLSS_NULL_ARGUMENT	int	无效的参数。

 版权所有 © 华为技术有限公司

返回值	类型	描述
KMLSS_BAD_HANDLE	int	pHandle格式存在问题。

依赖

#include "kml_scadss.h"

示例

C Interface：

```
MPI_Init(NULL, NULL);
int size, rank;
MPI_Comm_size(MPI_COMM_WORLD, &size);//获取总进程数
MPI_Comm_rank(MPI_COMM_WORLD, &rank);//获取当前进程标识

int n = 8;
int ia[9] = {0, 3, 7, 9, 11, 13, 15, 16, 17};
int ja[17] = {0, 3, 4, 1, 2, 3, 5, 2, 7, 3, 6, 4, 5, 5, 7, 6, 7};
double a[17] = {1.0, 1.0, 2.0, 9.0, 2.0, 1.0, -3.0, 3.0, 2.0, 9.0, -5.0, 6.0, 1.0, 4.0, 1.0, 7.0, 2.0};
double b[8] = {4.0, 9.0, 7.0, 6.0, 9.0, 3.0, 2.0, 5.0};
double x[8];
KmlScasolverTask* matrix_handle = NULL;
int ierr = KmlScadssGenInitDI(&matrix_handle, n, a, ja, ia, MPI_COMM_WORLD);
ierr = KmlScadssGenAnalyzeDI(&matrix_handle);
ierr = KmlScadssGenFactorizeDI(&matrix_handle);
int nrhs = 1;
int ldx=n, ldb=n;
ierr = KmlScadssGenSolveDI(&matrix_handle, nrhs, x, ldx, b, ldb);
ierr = KmlScadssGenCleanDI(&matrix_handle);
if (ierr != 0) {
    printf("\nERROR in KmlScadssGenCleanDI: %d", ierr);
}
```

6.5.3.4.9.5 KmlScadssGENSolve?I

求解LUx=b。

接口定义

C Interface：

int KmlScadssGenSolveSI(KmlScaSolverTask **pHandle, int nb, float *x, int ldx, float *b, int ldb);

int KmlScadssGenSolveDI(KmlScaSolverTask **pHandle, int nb, double *x, int ldx, double *b, int ldb);

int KmlScadssGenSolveCI(KmlScaSolverTask **pHandle, int nb, kml_complex_float *x, int ldx, kml_complex_float *b, int ldb);

int KmlScadssGenSolveZI(KmlScaSolverTask **pHandle, int nb, kml_complex_double *x, int ldx, kml_complex_double *b, int ldb);

 版权所有 © 华为技术有限公司

参数

参数名	类型	描述	输入/输出
pHandle	KmlScasolverTask **	求解器句柄，传入之前步骤的变量。	输入/输出

返回值

返回值	类型	描述
KMLSS_NO_ERROR	int	正常执行。
KMLSS_NULL_ARGUMENT	int	无效的参数。
KMLSS_BAD_HANDLE	int	pHandle格式存在问题。

依赖

#include "kml_scadss.h"

示例

C Interface：

```
MPI_Init(NULL, NULL);
int size, rank;
MPI_Comm_size(MPI_COMM_WORLD, &size);//获取总进程数
MPI_Comm_rank(MPI_COMM_WORLD, &rank);//获取当前进程标识

int n = 8;
int ia[9] = {0, 3, 7, 9, 11, 13, 15, 16, 17};
int ja[17] = {0, 3, 4, 1, 2, 3, 5, 2, 7, 3, 6, 4, 5, 5, 7, 6, 7};
double a[17] = {1.0, 1.0, 2.0, 9.0, 2.0, 1.0, -3.0, 3.0, 2.0, 9.0, -5.0, 6.0, 1.0, 4.0, 1.0, 7.0, 2.0};
double b[8] = {4.0, 9.0, 7.0, 6.0, 9.0, 3.0, 2.0, 5.0};
double x[8];
KmlScasolverTask* matrix_handle = NULL;
int ierr = KmlScadssGenInitDI(&matrix_handle, n, a, ja, ia, MPI_COMM_WORLD);
ierr = KmlScadssGenAnalyzeDI(&matrix_handle);
ierr = KmlScadssGenFactorizeDI(&matrix_handle);
int nrhs = 1;
int ldx=n, ldb=n;
ierr = KmlScadssGenSolveDI(&matrix_handle, nrhs, x, ldx, b, ldb);
ierr = KmlScadssGenCleanDI(&matrix_handle);
if (ierr != 0) {
    printf("\nERROR in KmlScadssGenCleanDI: %d", ierr);
}
```

6.5.3.4.9.6 KmlScadssGENGet?I?

获取与handle关联的值。

 版权所有 © 华为技术有限公司

接口定义

C Interface：

int KmlScadssGenGetSIL(KmlScaSolverTask **pHandle, enum KML_SOLVER_PARAM param, int64_t *value, int nvalue);

int KmlScadssGenGetDIL(KmlScaSolverTask **pHandle, enum KML_SOLVER_PARAM param, int64_t *value, int nvalue);

int KmlScadssGenGetCIL(KmlScaSolverTask **pHandle, enum KML_SOLVER_PARAM param, int64_t *value, int nvalue);

int KmlScadssGenGetZIL(KmlScaSolverTask **pHandle, enum KML_SOLVER_PARAM param, int64_t *value, int nvalue);

int KmlScadssGenGetSII(KmlScaSolverTask **pHandle, enum KML_SOLVER_PARAM param, int *value, int nvalue);

int KmlScadssGenGetDII(KmlScaSolverTask **pHandle, enum KML_SOLVER_PARAM param, int *value, int nvalue);

int KmlScadssGenGetCII(KmlScaSolverTask **pHandle, enum KML_SOLVER_PARAM param, int *value, int nvalue);

int KmlScadssGenGetZII(KmlScaSolverTask **pHandle, enum KML_SOLVER_PARAM param, int *value, int nvalue);

int KmlScadssGenGetSIS(KmlScaSolverTask **pHandle, enum KML_SOLVER_PARAM param, float *value, int nvalue);

int KmlScadssGenGetDID(KmlScaSolverTask **pHandle, enum KML_SOLVER_PARAM param, double *value, int nvalue);

int KmlScadssGenGetCIS(KmlScaSolverTask **pHandle, enum KML_SOLVER_PARAM param, float *value, int nvalue);

int KmlScadssGenGetZID(KmlScaSolverTask **pHandle, enum KML_SOLVER_PARAM param, double *value, int nvalue);

参数

参数名	类型	描述	输入/输出
pHandle	KmlScaSolverTask	求解器句柄，pHandle不可为空指针。	输入
param	enum KML_SOLVER_PARAM	• KMLSS_FILL_IN表示填入数目。 • KMLSS_PERM表示矩阵行列顺序，暂不支持。 • KMLSS_ITER_REF_STEPS表示迭代优化次数，暂不支持。	输入
value	int64_t *	直接求解相关参数数组。	输出
nvalue	int	value数组元素个数。	输入

 版权所有 © 华为技术有限公司

返回值

返回值	类型	描述
KMLSS_NO_ERROR	int	正常执行。
KMLSS_NULL_ARGUMENT	int	pHandle为空指针。
KMLSS_BAD_HANDLE	int	pHandle内部格式转换错误。
KMLSS_BAD_DATA_SIZE	int	KMLSS_FILL_IN数组长度nvalue设置错误。
KMLSS_INTERNAL_ERROR	int	内部数据返回空指针，KMLSS_FILL_IN获取失败。
KMLSS_BAD_SELECTOR	int	用户的KML_SOLVER_PARAM输入当前不支持，或在求解过程中不存在。

依赖

#include "kml_scadss.h"

示例

C Interface：

```
MPI_Init(NULL, NULL);
int size, rank;
MPI_Comm_size(MPI_COMM_WORLD, &size);//获取总进程数
MPI_Comm_rank(MPI_COMM_WORLD, &rank);//获取当前进程标识

int n = 8;
int ia[9] = {0, 3, 7, 9, 11, 13, 15, 16, 17};
int ja[17] = {0, 3, 4, 1, 2, 3, 5, 2, 7, 3, 6, 4, 5, 5, 7, 6, 7};
double a[17] = {1.0, 1.0, 2.0, 9.0, 2.0, 1.0, -3.0, 3.0, 2.0, 9.0, -5.0, 6.0, 1.0, 4.0, 1.0, 7.0, 2.0};
double b[8] = {4.0, 9.0, 7.0, 6.0, 9.0, 3.0, 2.0, 5.0};
double x[8];
KmlScasolverTask* matrix_handle = NULL;
int ierr = KmlScadssGenInitDI(&matrix_handle, n, a, ja, ia, MPI_COMM_WORLD);
ierr = KmlScadssGenAnalyzeDI(&matrix_handle);
ierr = KmlScadssGenFactorizeDI(&matrix_handle);
int nrhs = 1;
int ldx=n, ldb=n;
ierr = KmlScadssGenSolveDI(&matrix_handle, nrhs, x, ldx, b, ldb);
if (rank == 0) {
    int64_t nnz;
    ierr = KmlScadssGenGetDII(&matrix_handle, KMLSS_FILL_IN, &nnz, 1);
    printf("NNZ = %ld\n", nnz);
}
```

运行结果：

```
NNZ = 40
```

6.5.3.4.9.7 KmlScadssGENClean?I

释放内部数据结构。

 版权所有 © 华为技术有限公司

接口定义

C Interface：

int KmlScadssGenCleanSI(KmlScasolverTask **pHandle);

int KmlScadssGenCleanDI(KmlScasolverTask **pHandle);

int KmlScadssGenCleanCI(KmlScasolverTask **pHandle);

int KmlScadssGenCleanZI(KmlScasolverTask **pHandle);

参数

参数名	类型	描述	输入/输出
pHandle	KmlScasolverTask **	求解器句柄，传入之前步骤的变量。	输入/输出

返回值

返回值	类型	描述
KMLSS_NO_ERROR	int	正常执行。
KMLSS_NULL_ARGUMENT	int	无效的参数。
KMLSS_BAD_HANDLE	int	pHandle格式存在问题。

依赖

#include "kml_scadss.h"

示例

C Interface：

```
MPI_Init(NULL, NULL);
int size, rank;
MPI_Comm_size(MPI_COMM_WORLD, &size);//获取总进程数
MPI_Comm_rank(MPI_COMM_WORLD, &rank);//获取当前进程标识

int n = 8;
int ia[9] = {0, 3, 7, 9, 11, 13, 15, 16, 17};
int ja[17] = {0, 3, 4, 1, 2, 3, 5, 2, 7, 3, 6, 4, 5, 5, 7, 6, 7};
double a[17] = {1.0, 1.0, 2.0, 9.0, 2.0, 1.0, -3.0, 3.0, 2.0, 9.0, -5.0, 6.0, 1.0, 4.0, 1.0, 7.0, 2.0};
double b[8] = {4.0, 9.0, 7.0, 6.0, 9.0, 3.0, 2.0, 5.0};
double x[8];
KmlScasolverTask* matrix_handle = NULL;
int ierr = KmlScadssGenInitDI(&matrix_handle, n, a, ja, ia, MPI_COMM_WORLD);
ierr = KmlScadssGenAnalyzeDI(&matrix_handle);
ierr = KmlScadssGenFactorizeDI(&matrix_handle);
int nrhs = 1;
```

 版权所有 © 华为技术有限公司

```
int ldx=n, ldb=n;
ierr = KmlScadssGenSolveDI(&matrix_handle, nrhs, x, ldx, b, ldb);
ierr = KmlScadssGenCleanDI(&matrix_handle);
if (ierr != 0) {
    printf("\nERROR in KmlScadssGenCleanDI: %d", ierr);
}
```

6.5.3.4.10 SCADSS-LDLT (DEPRECATED)

集群版直接法推荐使用**SCADSS**。

6.5.3.4.10.1 KmlScadssSYMInit?I

初始化数据结构，并将用户提供的系数矩阵关联到求解系统中，初始化集群版的LDLT方法时使用上三角的CSR格式。

接口定义

C Interface：

int KmlScadssSymInitSI(KmlScasolverTask **pHandle, int n, float *a, int *ja, int *ia, MPI_Comm comm);

int KmlScadssSymInitDI(KmlScasolverTask **pHandle, int n, double *a, int *ja, int *ia, MPI_Comm comm);

int KmlScadssSymInitCI(KmlScasolverTask **pHandle, int n, kml_complex_float *a, int *ja, int *ia, MPI_Comm comm);

int KmlScadssSymInitZI(KmlScasolverTask **pHandle, int n, kml_complex_double *a, int *ja, int *ia, MPI_Comm comm);

参数

参数名	类型	描述	输入/输出
pHandle	KmlScasolverTask **	求解器句柄，传入之前步骤的变量。	输入/输出
n	int	n为矩阵A的维度	输入

版权所有 © 华为技术有限公司

参数名	类型	描述	输入/输出
a	• 在KmlScadssSymInitSI中为float *。 • 在KmlScadssSymInitDI中为double *。 • 在KmlScadssSymInitCI中为kml_complex_float *。 • 在KmlScadssSymInitZI中为kml_complex_double *。	a为维度为非零元素的个数的数组，a[i]为第i个非零元的值。该数组的生命周期与handle相同。	输入
ja	int *	ja为CSR格式的列索引数组，ja[i]为第i个非零元的列号。该数组的生命周期与handle相同。	输入
ia	int *	ia为CSR格式的行偏移数组，ia[i]为第i行第一个非零元在所有非零元中的索引，ia[0]=0。该数组的生命周期与handle相同。	输入
comm	MPI_Comm	为MPI进程通信域。	输入

返回值

返回值	类型	描述
KMLSS_NO_ERROR	int	正常执行。
KMLSS_NULL_ARGUMENT	int	无效的参数。
KMLSS_BAD_HANDLE	int	pHandle格式存在问题。

依赖

#include "kml_scadss.h"

示例

C Interface：

 版权所有 © 华为技术有限公司

```
MPI_Init(NULL, NULL);
int size, rank;
MPI_Comm_size(MPI_COMM_WORLD, &size);//获取总进程数
MPI_Comm_rank(MPI_COMM_WORLD, &rank);//获取当前进程标识

int n = 8;
int ia[9] = {0, 3, 7, 9, 11, 13, 15, 16, 17};
int ja[17] = {0, 3, 4, 1, 2, 3, 5, 2, 7, 3, 6, 4, 5, 5, 7, 6, 7};
double a[17] = {1.0, 1.0, 2.0, 9.0, 2.0, 1.0, -3.0, 3.0, 2.0, 9.0, -5.0, 6.0, 1.0, 4.0, 1.0, 7.0, 2.0};
double b[8] = {4.0, 9.0, 7.0, 6.0, 9.0, 3.0, 2.0, 5.0};
double x[8];
KmlScasolverTask* matrix_handle = NULL;
int ierr = KmlScadssSymInitDI(&matrix_handle, n, a, ja, ia, MPI_COMM_WORLD);
ierr = KmlScadssSymAnalyzeDI(&matrix_handle);
ierr = KmlScadssSymFactorizeDI(&matrix_handle);
int nrhs = 1;
int ldx=n, ldb=n;
ierr = KmlScadssSymSolveDI(&matrix_handle, nrhs, x, ldx, b, ldb);
ierr = KmlScadssSymCleanDI(&matrix_handle);
if (ierr != 0) {
    printf("\nERROR in KmlScadssSymCleanDI: %d", ierr);
}
```

6.5.3.4.10.2 KmlScadssSYMSet?I?

设置与handle关联的值。

接口定义

C Interface：

int KmlScadssSymSetSII(KmlScaSolverTask **pHandle, enum KML_SOLVER_PARAM param, int *value, int nvalue);

int KmlScadssSymSetDII(KmlScaSolverTask **pHandle, enum KML_SOLVER_PARAM param, int *value, int nvalue);

int KmlScadssSymSetCII(KmlScaSolverTask **pHandle, enum KML_SOLVER_PARAM param, int *value, int nvalue);

int KmlScadssSymSetZII(KmlScaSolverTask **pHandle, enum KML_SOLVER_PARAM param, int *value, int nvalue);

int KmlScadssSymSetDID(KmlScaSolverTask **pHandle, enum KML_SOLVER_PARAM param, double *value, int nvalue);

int KmlScadssSymSetZID(KmlScaSolverTask **pHandle, enum KML_SOLVER_PARAM param, double *value, int nvalue);

int KmlScadssSymSetSIS(KmlScaSolverTask **pHandle, enum KML_SOLVER_PARAM param, float *value, int nvalue);

int KmlScadssSymSetCIS(KmlScaSolverTask **pHandle, enum KML_SOLVER_PARAM param, float *value, int nvalue);

 版权所有 © 华为技术有限公司

参数

参数名	类型	描述	输入/输出
pHandle	KmlScaSolverTask	求解器句柄，pHandle不可为空指针。	输入
param	enum KML_SOLVER_PARAM	• KMLSS_REFINEMENT_MAX_STEPS表示REFINEMENT的最大次数。 • KMLSS_PERM表示矩阵行列顺序，暂不支持。 • KMLSS_ITER_REF_STEPS表示迭代优化次数，暂不支持。	输入
value	int *	直接求解相关参数数组。	输出
nvalue	int	value数组元素个数。	输入

返回值

返回值	类型	描述
KMLSS_NO_ERROR	int	正常执行。
KMLSS_NULL_ARGUMENT	int	pHandle为空指针。
KMLSS_BAD_HANDLE	int	pHandle内部格式转换错误。
KMLSS_BAD_DATA_SIZE	int	KMLSS_FILL_IN数组长度nvalue设置错误。
KMLSS_INTERNAL_ERROR	int	内部数据返回空指针，KMLSS_FILL_IN获取失败。
KMLSS_BAD_SELECTOR	int	用户的KML_SOLVER_PARAM输入当前不支持，或在求解过程中不存在。

依赖

#include "kml_scadss.h"

示例

C Interface：

```
MPI_Init(NULL, NULL);
int size, rank;
MPI_Comm_size(MPI_COMM_WORLD, &size);//获取总进程数
MPI_Comm_rank(MPI_COMM_WORLD, &rank);//获取当前进程标识
```

 版权所有 © 华为技术有限公司

```
int n = 8;
int ia[9] = {0, 3, 7, 9, 11, 13, 15, 16, 17};
int ja[17] = {0, 3, 4, 1, 2, 3, 5, 2, 7, 3, 6, 4, 5, 5, 7, 6, 7};
double a[17] = {1.0, 1.0, 2.0, 9.0, 2.0, 1.0, -3.0, 3.0, 2.0, 9.0, -5.0, 6.0, 1.0, 4.0, 1.0, 7.0, 2.0};
double b[8] = {4.0, 9.0, 7.0, 6.0, 9.0, 3.0, 2.0, 5.0};
double x[8];
KmlScasolverTask* matrix_handle = NULL;
int ierr = KmlScadssSymInitDI(&matrix_handle, n, a, ja, ia, MPI_COMM_WORLD);
ierr = KmlScadssSymSetDII(&matrix_handle, KMLSS_REFINEMENT_MAX_STEPS, &max_refinement_steps , 1);
ierr = KmlScadssSymAnalyzeDI(&matrix_handle);
ierr = KmlScadssSymFactorizeDI(&matrix_handle);
int nrhs = 1;
int ldx=n, ldb=n;
ierr = KmlScadssSymSolveDI(&matrix_handle, nrhs, x, ldx, b, ldb);
```

6.5.3.4.10.3 KmlScadssSYMAnalyze?I

分析数据结构，对稀疏矩阵进行重排和符号分解

接口定义

C Interface：

int KmlScadssSymAnalyzeSI(KmlScasolverTask **pHandle);

int KmlScadssSymAnalyzeDI(KmlScasolverTask **pHandle);

int KmlScadssSymAnalyzeCI(KmlScasolverTask **pHandle);

int KmlScadssSymAnalyzeZI(KmlScasolverTask **pHandle);

参数

参数名	类型	描述	输入/输出
pHandle	KmlScasolverTask **	求解器句柄，传入之前步骤的变量。	输入/输出

返回值

返回值	类型	描述
KMLSS_NO_ERROR	int	正常执行。
KMLSS_NULL_ARGUMENT	int	无效的参数。
KMLSS_BAD_HANDLE	int	pHandle格式存在问题。

依赖

#include "kml_scadss.h"

 版权所有 © 华为技术有限公司

示例

C Interface：

```
MPI_Init(NULL, NULL);
int size, rank;
MPI_Comm_size(MPI_COMM_WORLD, &size);//获取总进程数
MPI_Comm_rank(MPI_COMM_WORLD, &rank);//获取当前进程标识

int n = 8;
int ia[9] = {0, 3, 7, 9, 11, 13, 15, 16, 17};
int ja[17] = {0, 3, 4, 1, 2, 3, 5, 2, 7, 3, 6, 4, 5, 5, 7, 6, 7};
double a[17] = {1.0, 1.0, 2.0, 9.0, 2.0, 1.0, -3.0, 3.0, 2.0, 9.0, -5.0, 6.0, 1.0, 4.0, 1.0, 7.0, 2.0};
double b[8] = {4.0, 9.0, 7.0, 6.0, 9.0, 3.0, 2.0, 5.0};
double x[8];
KmlScasolverTask* matrix_handle = NULL;
int ierr = KmlScadssSymInitDI(&matrix_handle, n, a, ja, ia, MPI_COMM_WORLD);
ierr = KmlScadssSymAnalyzeDI(&matrix_handle);
ierr = KmlScadssSymFactorizeDI(&matrix_handle);
int nrhs = 1;
int ldx=n, ldb=n;
ierr = KmlScadssSymSolveDI(&matrix_handle, nrhs, x, ldx, b, ldb);
ierr = KmlScadssSymCleanDI(&matrix_handle);
if (ierr != 0) {
    printf("\nERROR in KmlScadssSymCleanDI: %d", ierr);
}
```

6.5.3.4.10.4 KmlScadssSYMFactorize?I

实现对称正定矩阵的LLT分解或者Hermitian正定矩阵的LLH分解。

接口定义

C Interface：

int KmlScadssSymFactorizeSI(KmlScasolverTask **pHandle);

int KmlScadssSymFactorizeDI(KmlScasolverTask **pHandle);

int KmlScadssSymFactorizeCI(KmlScasolverTask **pHandle);

int KmlScadssSymFactorizeZI(KmlScasolverTask **pHandle);

参数

参数名	类型	描述	输入/输出
pHandle	KmlScasolverTask **	求解器句柄，传入之前步骤的变量。	输入/输出

返回值

返回值	类型	描述
KMLSS_NO_ERROR	int	正常执行。
KMLSS_NULL_ARGUMENT	int	无效的参数。

版权所有 © 华为技术有限公司

返回值	类型	描述
KMLSS_BAD_HANDLE	int	pHandle格式存在问题。

依赖

#include "kml_scadss.h"

示例

C Interface：

```
MPI_Init(NULL, NULL);
int size, rank;
MPI_Comm_size(MPI_COMM_WORLD, &size);//获取总进程数
MPI_Comm_rank(MPI_COMM_WORLD, &rank);//获取当前进程标识

int n = 8;
int ia[9] = {0, 3, 7, 9, 11, 13, 15, 16, 17};
int ja[17] = {0, 3, 4, 1, 2, 3, 5, 2, 7, 3, 6, 4, 5, 5, 7, 6, 7};
double a[17] = {1.0, 1.0, 2.0, 9.0, 2.0, 1.0, -3.0, 3.0, 2.0, 9.0, -5.0, 6.0, 1.0, 4.0, 1.0, 7.0, 2.0};
double b[8] = {4.0, 9.0, 7.0, 6.0, 9.0, 3.0, 2.0, 5.0};
double x[8];
KmlScasolverTask* matrix_handle = NULL;
int ierr = KmlScadssSymInitDI(&matrix_handle, n, a, ja, ia, MPI_COMM_WORLD);
ierr = KmlScadssSymAnalyzeDI(&matrix_handle);
ierr = KmlScadssSymFactorizeDI(&matrix_handle);
int nrhs = 1;
int ldx=n, ldb=n;
ierr = KmlScadssSymSolveDI(&matrix_handle, nrhs, x, ldx, b, ldb);
ierr = KmlScadssSymCleanDI(&matrix_handle);
if (ierr != 0) {
    printf("\nERROR in KmlScadssSymCleanDI: %d", ierr);
}
```

6.5.3.4.10.5 KmlScadssSYMSolve?I

求解LDLTx=b。

接口定义

C Interface：

int KmlScadssSymSolveSI(KmlScaSolverTask **pHandle, int nb, float *x, int ldx, float *b, int ldb);

int KmlScadssSymSolveDI(KmlScaSolverTask **pHandle, int nb, double *x, int ldx, double *b, int ldb);

int KmlScadssSymSolveCI(KmlScaSolverTask **pHandle, int nb, kml_complex_float *x, int ldx, kml_complex_float *b, int ldb);

int KmlScadssSymSolveZI(KmlScaSolverTask **pHandle, int nb, kml_complex_double *x, int ldx, kml_complex_double *b, int ldb);

 版权所有 © 华为技术有限公司

参数

参数名	类型	描述	输入/输出
pHandle	KmlScasolverTask **	求解器句柄，传入之前步骤的变量。	输入/输出

返回值

返回值	类型	描述
KMLSS_NO_ERROR	int	正常执行。
KMLSS_NULL_ARGUMENT	int	无效的参数。
KMLSS_BAD_HANDLE	int	pHandle格式存在问题。

依赖

#include "kml_scadss.h"

示例

C Interface：

```
MPI_Init(NULL, NULL);
int size, rank;
MPI_Comm_size(MPI_COMM_WORLD, &size);//获取总进程数
MPI_Comm_rank(MPI_COMM_WORLD, &rank);//获取当前进程标识

int n = 8;
int ia[9] = {0, 3, 7, 9, 11, 13, 15, 16, 17};
int ja[17] = {0, 3, 4, 1, 2, 3, 5, 2, 7, 3, 6, 4, 5, 5, 7, 6, 7};
double a[17] = {1.0, 1.0, 2.0, 9.0, 2.0, 1.0, -3.0, 3.0, 2.0, 9.0, -5.0, 6.0, 1.0, 4.0, 1.0, 7.0, 2.0};
double b[8] = {4.0, 9.0, 7.0, 6.0, 9.0, 3.0, 2.0, 5.0};
double x[8];
KmlScasolverTask* matrix_handle = NULL;
int ierr = KmlScadssSymInitDI(&matrix_handle, n, a, ja, ia, MPI_COMM_WORLD);
ierr = KmlScadssSymAnalyzeDI(&matrix_handle);
ierr = KmlScadssSymFactorizeDI(&matrix_handle);
int nrhs = 1;
int ldx=n, ldb=n;
ierr = KmlScadssSymSolveDI(&matrix_handle, nrhs, x, ldx, b, ldb);
ierr = KmlScadssSymCleanDI(&matrix_handle);
if (ierr != 0) {
    printf("\nERROR in KmlScadssSymCleanDI: %d", ierr);
}
```

6.5.3.4.10.6 KmlScadssSYMGet?I?

获取与handle关联的值。

 版权所有 © 华为技术有限公司

接口定义

C Interface：

int KmlScadssSymGetSIL(KmlScaSolverTask **pHandle, enum KML_SOLVER_PARAM param, int64_t *value, int nvalue);

int KmlScadssSymGetDIL(KmlScaSolverTask **pHandle, enum KML_SOLVER_PARAM param, int64_t *value, int nvalue);

int KmlScadssSymGetCIL(KmlScaSolverTask **pHandle, enum KML_SOLVER_PARAM param, int64_t *value, int nvalue);

int KmlScadssSymGetZIL(KmlScaSolverTask **pHandle, enum KML_SOLVER_PARAM param, int64_t *value, int nvalue);

int KmlScadssSymGetSII(KmlScaSolverTask **pHandle, enum KML_SOLVER_PARAM param, int *value, int nvalue);

int KmlScadssSymGetDII(KmlScaSolverTask **pHandle, enum KML_SOLVER_PARAM param, int *value, int nvalue);

int KmlScadssSymGetCII(KmlScaSolverTask **pHandle, enum KML_SOLVER_PARAM param, int *value, int nvalue);

int KmlScadssSymGetZII(KmlScaSolverTask **pHandle, enum KML_SOLVER_PARAM param, int *value, int nvalue);

int KmlScadssSymGetSIS(KmlScaSolverTask **pHandle, enum KML_SOLVER_PARAM param, float *value, int nvalue);

int KmlScadssSymGetDID(KmlScaSolverTask **pHandle, enum KML_SOLVER_PARAM param, double *value, int nvalue);

int KmlScadssSymGetCIS(KmlScaSolverTask **pHandle, enum KML_SOLVER_PARAM param, float *value, int nvalue);

int KmlScadssSymGetZID(KmlScaSolverTask **pHandle, enum KML_SOLVER_PARAM param, double *value, int nvalue);

参数

参数名	类型	描述	输入/输出
pHandle	KmlScaSolverTask	求解器句柄，pHandle不可为空指针。	输入
param	enum KML_SOLVER_PARAM	• KMLSS_FILL_IN表示填入数目。 • KMLSS_PERM表示矩阵行列顺序，暂不支持。 • KMLSS_ITER_REF_STEPS表示迭代优化次数，暂不支持。	输入
value	int64_t *	直接求解相关参数数组。	输出
nvalue	int	value数组元素个数。	输入

版权所有 © 华为技术有限公司

返回值

返回值	类型	描述
KMLSS_NO_ERROR	int	正常执行。
KMLSS_NULL_ARGUMENT	int	pHandle为空指针。
KMLSS_BAD_HANDLE	int	pHandle内部格式转换错误。
KMLSS_BAD_DATA_SIZE	int	KMLSS_FILL_IN数组长度nvalue设置错误。
KMLSS_INTERNAL_ERROR	int	内部数据返回空指针，KMLSS_FILL_IN获取失败。
KMLSS_BAD_SELECTOR	int	用户的KML_SOLVER_PARAM输入当前不支持，或在求解过程中不存在。

依赖

#include "kml_scadss.h"

示例

C Interface：

```
MPI_Init(NULL, NULL);
int size, rank;
MPI_Comm_size(MPI_COMM_WORLD, &size);//获取总进程数
MPI_Comm_rank(MPI_COMM_WORLD, &rank);//获取当前进程标识

int n = 8;
int ia[9] = {0, 3, 7, 9, 11, 13, 15, 16, 17};
int ja[17] = {0, 3, 4, 1, 2, 3, 5, 2, 7, 3, 6, 4, 5, 5, 7, 6, 7};
double a[17] = {1.0, 1.0, 2.0, 9.0, 2.0, 1.0, -3.0, 3.0, 2.0, 9.0, -5.0, 6.0, 1.0, 4.0, 1.0, 7.0, 2.0};
double b[8] = {4.0, 9.0, 7.0, 6.0, 9.0, 3.0, 2.0, 5.0};
double x[8];
KmlScasolverTask* matrix_handle = NULL;
int ierr = KmlScadssSymInitDI(&matrix_handle, n, a, ja, ia, MPI_COMM_WORLD);
ierr = KmlScadssSymAnalyzeDI(&matrix_handle);
ierr = KmlScadssSymFactorizeDI(&matrix_handle);
int nrhs = 1;
int ldx=n, ldb=n;
ierr = KmlScadssSymSolveDI(&matrix_handle, nrhs, x, ldx, b, ldb);
if (rank == 0) {
    int64_t nnz;
    ierr = KmlScadssSymGetDII(&matrix_handle, KMLSS_FILL_IN, &nnz, 1);
    printf("NNZ = %ld\n", nnz);
}
```

运行结果：

```
NNZ = 40
```

6.5.3.4.10.7 KmlScadssSYMClean?I

释放内部数据结构。

 版权所有 © 华为技术有限公司

接口定义

C Interface：

int KmlScadssSymCleanSI(KmlScasolverTask **pHandle);

int KmlScadssSymCleanDI(KmlScasolverTask **pHandle);

int KmlScadssSymCleanCI(KmlScasolverTask **pHandle);

int KmlScadssSymCleanZI(KmlScasolverTask **pHandle);

参数

参数名	类型	描述	输入/输出
pHandle	KmlScasolverTask **	求解器句柄，传入之前步骤的变量。	输入/输出

返回值

返回值	类型	描述
KMLSS_NO_ERROR	int	正常执行。
KMLSS_NULL_ARGUMENT	int	无效的参数。
KMLSS_BAD_HANDLE	int	pHandle格式存在问题。

依赖

#include "kml_scadss.h"

示例

C Interface：

```
MPI_Init(NULL, NULL);
int size, rank;
MPI_Comm_size(MPI_COMM_WORLD, &size);//获取总进程数
MPI_Comm_rank(MPI_COMM_WORLD, &rank);//获取当前进程标识

int n = 8;
int ia[9] = {0, 3, 7, 9, 11, 13, 15, 16, 17};
int ja[17] = {0, 3, 4, 1, 2, 3, 5, 2, 7, 3, 6, 4, 5, 5, 7, 6, 7};
double a[17] = {1.0, 1.0, 2.0, 9.0, 2.0, 1.0, -3.0, 3.0, 2.0, 9.0, -5.0, 6.0, 1.0, 4.0, 1.0, 7.0, 2.0};
double b[8] = {4.0, 9.0, 7.0, 6.0, 9.0, 3.0, 2.0, 5.0};
double x[8];
KmlScasolverTask* matrix_handle = NULL;
int ierr = KmlScadssSymInitDI(&matrix_handle, n, a, ja, ia, MPI_COMM_WORLD);
ierr = KmlScadssSymAnalyzeDI(&matrix_handle);
ierr = KmlScadssSymFactorizeDI(&matrix_handle);
int nrhs = 1;
```

 版权所有 © 华为技术有限公司

```
int ldx=n, ldb=n;
ierr = KmlScadssSymSolveDI(&matrix_handle, nrhs, x, ldx, b, ldb);
ierr = KmlScadssSymCleanDI(&matrix_handle);
if (ierr != 0) {
    printf("\nERROR in KmlScadssSymCleanDI: %d", ierr);
}
```

6.5.3.4.11 KSCAKSOLVERGetVersion

获得求解器产品版本信息。

接口定义

C Interface：

int KSCASOLVERGetVersion(KSCASOLVERVersion *ver);

参数

参数名	类型	描述	输入/输出
ver	KSCASOLVERVersion结构体指针。	存储版本信息，非空。	输入/输出

依赖

#include "kml_scasolver.h"

#include "kml_service.h"

示例

C Interface：

```
int main()
{
    KSCASOLVERVersion *ver = (KSCASOLVERVersion*)malloc(sizeof(KSCASOLVERVersion));
    int res = KSCASOLVERGetVersion(ver);
    if (res != 0) {
        printf("Get version error.\n");
        return 0;
    }
    printf("Product Name: %s\nProduct Version: %s\nComponent Name: %s\nComponent Version: %s
\nComponent AbiVersion: %s\nComponent AppendInfo: %s\nSoftware Name: %s\nSoftware Version: %s\n",
            ver->productName, ver->productVersion, ver->componentName,
            ver->componentVersion, ver->componentAbiVersion, ver->componentAppendInfo,
            ver->softwareName, ver->softwareVersion);

    return 0;
}
```

运行结果：

```
Product Name: Kunpeng HPCKit
Product Version: 24.0.RC1
Component Name: HPCKit-kml
Component Version: 2.3.0
Component AbiVersion: 1
Component AppendInfo: gcc
```

 版权所有 © 华为技术有限公司

```
Software Name: hpckit-kml-scasolver
Software Version: 2.3.0
```

说明

版本号以实际运行结果为准，上述结果仅供参考。

6.5.4 KML_EIGENSOLVER 库函数说明

6.5.4.1 函数说明

特征值求解库（EigenSolver）提供大型矩阵的特征值迭代求解器，支持MPI多进程、用户自定义矩阵向量乘及预条件子。

特征值求解器包含的求解方法有：

- LOBPCG（locally optimal block preconditioned conjugate gradient），支持单精度实数，单精度复数，双精度实数，双精度复数；适用场景是所需的特征值个数较少，成本低。
- PPCG（projected preconditioned conjugate gradient），支持单精度实数，单精度复数，双精度实数，双精度复数；适用场景是所需的特征值个数较多，多节点，成本高。

6.5.4.2 数据类型

KML_EIGENSOLVER支持单精度实数、单精度复数、双精度实数、双精度复数求解，支持用户自定义矩阵向量乘。

6.5.4.3 函数定义

6.5.4.3.1 函数返回代码

KesReturnCode	类型	描述
KES_GOOD	int	正常执行。
KES_INVALID_PARAMETER	int	参数范围不正确。
KES_INVALID_TYPE	int	数据类型不正确。
KES_INVALID_CALL	int	输入的向量或者矩阵类型不正确。
KES_NULL_PTR	int	输入参数有空指针。

6.5.4.3.2 创建内部对象

版权所有 © 华为技术有限公司

6.5.4.3.2.1 KesMatrixCreate

接口定义

KesReturnCode KesMatrixCreate(KesMatrix *A, KesScalarType type, KesMatrixFormat format);

接口说明

创建矩阵，并指定数据类型、矩阵形式。

参数

参数名	类型	描述	输入/输出
A	KesMatrix *	矩阵句柄，不能为空指针。	输入/输出
type	KesScalarType	数据类型。	输入
format	KesMatrixFormat	矩阵类型。	输入

KesMatrixFormat	类型描述
KES_EMPTY_MATRIX	空矩阵。
KES_SHELL	用户自定义的矩阵向量乘格式，矩阵的数据以及相应的MatVec通过相关接口指定。

KesScalarType	类型描述	对应C的类型
KES_SCALAR_FP32	单精度实数浮点类型。	float
KES_SCALAR_FP32C	单精度复数浮点类型。	float _Complex
KES_SCALAR_FP64	双精度实数浮点类型。	double
KES_SCALAR_FP64C	双精度复数浮点类型。	double _Complex

6.5.4.3.2.2 KesVectorCreate

接口定义

KesReturnCode KesVectorCreate(KesVector *X, KesScalarType type, KesVectorFormat format);

版权所有 © 华为技术有限公司

接口说明

创建向量，并指定数据类型、向量形式。

参数

参数名	类型	描述	输入/输出
X	KesVector *	向量句柄，不能为空指针。	输入/输出
type	KesScalarType	数据类型。	输入
format	KesVectorFormat	向量类型。	输入

KesVectorFormat	类型描述
KES_EMPTY_VECTOR	空向量
KES_DENSE	采用BLAS数组列优先存储的方式，向量数据由（首元素指针，向量长度，向量个数，leading dimension）指定。

KesScalarType	类型描述	对应C的类型
KES_SCALAR_FP32	单精度实数浮点类型。	float
KES_SCALAR_FP32C	单精度复数浮点类型。	float _Complex
KES_SCALAR_FP64	双精度实数浮点类型。	double
KES_SCALAR_FP64C	双精度复数浮点类型。	double _Complex

6.5.4.3.2.3 KesSolverCreate

接口定义

KesReturnCode KesSolverCreate(KesSolver *solver, KesScalarType type, KesSolverAlgorithm algo);

接口说明

创建求解器，并指定数据类型、求解算法。

版权所有 © 华为技术有限公司

参数

参数名	类型	描述	输入/输出
solver	KesSolver *	求解器句柄，不能为空指针。	输入/输出
type	KesScalarType	数据类型。	输入
algo	KesSolverAlgorithm	求解器算法。	输入

KesSolverAlgorithm	类型描述	适用场景
KES_LOBPCG	LOBPCG算法	所求特征值个数较少；计算节点少。
KES_LOBPCG_BATCH	分块LOBPCG算法	所求特征值个数较多；最大迭代步数多；计算节点少。
KES_PPCG	PPCG算法	所求特征值个数较多；最大迭代步数少；计算节点多。

KesScalarType	类型描述	对应C的类型
KES_SCALAR_FP32	单精度实数浮点类型。	float
KES_SCALAR_FP32C	单精度复数浮点类型。	float _Complex
KES_SCALAR_FP64	双精度实数浮点类型。	double
KES_SCALAR_FP64C	双精度复数浮点类型。	double _Complex

6.5.4.3.2.4 KesMatrixSetDim

接口定义

KesReturnCode KesMatrixSetDim(KesMatrix A, int nRows, int nCols);

接口说明

设置矩阵全局维度。

版权所有 © 华为技术有限公司

参数

参数名	类型	描述	输入/输出
A	KesMatrix	矩阵句柄，不能为空指针。	输入/输出
nRows	int	矩阵全局行数。	输入
nCols	int	矩阵全局列数。	输入

6.5.4.3.2.5 KesVectorSetDim

接口定义

KesReturnCode KesVectorSetDim(KesVector X, int nRows, int nCols);

接口说明

设置向量的局部长度和个数。

参数

参数名	类型	描述	输入/输出
X	KesVector	向量句柄，必须为已经创建过的KES_DENSE格式的向量。	输入/输出
nRows	int	向量的局部长度。	输入
nCols	int	向量个数。	输入

6.5.4.3.2.6 KesMatrixSetShellMatVec

接口定义

KesReturnCode KesMatrixSetShellMatVec(KesMatrix A, UserData data, UserMatVec matvec);

接口说明

设置KES_SHELL矩阵的矩阵向量乘。

版权所有 © 华为技术有限公司

参数

参数名	类型	描述	输入/输出
A	KesMatrix	矩阵句柄，必须为已经创建过的KES_SHELL格式的矩阵。	输入/输出
data	void *	用户自定义数据类型。	输入
matvec	void (*)(void *data, void *X, void *AX, int nRows, int nCols, int ld)	用户自定义矩阵向量乘。	输入

matvec中的参数	类型	描述
data	void *	与KesMatrixSetShellMatVec中的data参数一致。
X	void *	矩阵向量乘AX = A * X中的原向量X的首元素地址，X的实际数据类型必须与KES矩阵A的数据类型一致。
AX	void *	矩阵向量乘AX = A * X中的结果向量AX的首元素地址，AX的实际数据类型必须与KES矩阵A的数据类型一致。
nRows	int	X的局部向量长度，也是AX的局部向量长度。
nCols	int	X的向量个数，也是AX的向量个数。
ld	int	X的leading dimension，也是AX的leading dimension。

6.5.4.3.2.7 KesVectorDenseSetValues

接口定义

KesReturnCode KesVectorDenseSetValues(KesVector X, void *values, int ld);

接口说明

设置KES_DENSE向量的具体数据。

参数

参数名	类型	描述	输入/输出
X	KesVector	向量句柄，不能为空指针。	输入/输出
values	void *	数组元素首地址，values的实际数据类型必须与X的数据类型一致。	输入

版权所有 © 华为技术有限公司

参数名	类型	描述	输入/输出
ld	int	leading dimension。	输入

6.5.4.3.2.8 KesVectorSetComm

接口定义

KesReturnCode KesVectorSetComm(KesVector X, KesCommDirection direction, MPI_Comm comm);

接口说明

设置向量的通信子。

参数

参数名	类型	描述	输入/输出
X	KesVector	向量句柄，不能为空指针。	输入/输出
direction	KesCommDirection	通信子类型，行分割的通信子/列通信子。	输入
comm	MPI_Comm	MPI通信子。	输入

KesCommDirection	类型描述
KES_ROW	行分割的通信子。
KES_COL	列通信子（目前仅支持MPI_COMM_SELF）。

6.5.4.3.2.9 KesVectorReplicate

接口定义

KesReturnCode KesVectorReplicate(const KesVector X, KesVector *replicateX, bool toAllocateMem, bool toCopyData);

接口说明

复制一个向量。

版权所有 © 华为技术有限公司

参数

参数名	类型	描述	输入/输出
X	const KesVector	向量句柄，不能为空指针。	输入
replicateX	KesVector *	即将被创建的向量，其通信子、维度与X一致。	输出
toAllocateMem	bool	指定是否为replicateX分配内存。	输入
toCopyData	bool	指定是否将X的数据复制到replicateX。	输入

6.5.4.3.3 销毁内部对象

6.5.4.3.3.1 KesMatrixDestroy

接口定义

```
void KesMatrixDestroy(KesMatrix A);
```

接口说明

销毁由KesMatrixCreate创建的矩阵。

参数

参数名	类型	描述	输入/输出
A	KesMatrix	矩阵句柄。	输入/输出

6.5.4.3.3.2 KesVectorDestroy

接口定义

```
void KesVectorDestroy(KesVector X);
```

接口说明

销毁由KesVectorCreate创建的向量。

版权所有 © 华为技术有限公司

参数

参数名	类型	描述	输入/输出
X	KesVector	向量句柄。	输入/输出

6.5.4.3.3.3 KesSolverDestroy

接口定义

void KesSolverDestroy(KesSolver solver);

接口说明

销毁由KesSolverCreate创建的求解器。

参数

参数名	类型	描述	输入/输出
solver	KesSolver	求解器句柄。	输入/输出

6.5.4.3.4 设置求解参数

6.5.4.3.4.1 KesSolverSetProblemType

接口定义

KesReturnCode KesSolverSetProblemType(KesSolver solver, KesProblemType type);

接口说明

设置求解问题的类型。

参数

参数名	类型	描述	输入/输出
solver	KesSolver	求解器句柄，不能为空指针。	输入/输出
type	KesProblemType	特征值问题类型，标准特征值问题/广义特征值问题。	输入

版权所有 © 华为技术有限公司

6.5.4.3.4.2 KesSolverSetMaxIter

接口定义

KesReturnCode KesSolverSetMaxIter(KesSolver solver, int maxIter);

接口说明

设置最大迭代步数，默认值为100。

参数

参数名	类型	描述	输入/输出
solver	KesSolver	求解器句柄，不能为空指针。	输入/输出
maxIter	int	最大迭代步数。	输入

6.5.4.3.4.3 KesSolverSetMaxOrthoIter

接口定义

KesReturnCode KesSolverSetMaxOrthoIter(KesSolver solver, int maxOrthoIter);

接口说明

设置正交化操作中的最大重复次数，默认值为1。

参数

参数名	类型	描述	输入/输出
solver	KesSolver	求解器句柄，不能为空指针。	输入/输出
maxOrthoIter	int	正交化操作中的最大重复次数。	输入

版权所有 © 华为技术有限公司

6.5.4.3.4.4 KesSolverSetThreshold

接口定义

KesReturnCode KesSolverSetThreshold(KesSolver solver, double threshold);

接口说明

设置收敛误差，默认值为0.000001。

参数

参数名	类型	描述	输入/输出
solver	KesSolver	求解器句柄，不能为空指针。	输入/输出
threshold	double	收敛误差。	输入

6.5.4.3.4.5 KesSolverSetDenseEigenSolverType

接口定义

KesReturnCode KesSolverSetDenseEigenSolverType(KesSolver solver, KesDenseEigenSolverType type);

接口说明

设置Rayleigh-Ritz子空间稠密特征值求解器类型，默认值为KES_SCALAPACK。

参数

参数名	类型	描述	输入/输出
solver	KesSolver	求解器句柄，不能为空指针。	输入/输出
type	KesDenseEigenSolverType	Rayleigh-Ritz子空间稠密特征值求解器类型。	输入

KesDenseEigenSolverType	描述
KES_LAPACK	采用LAPACK的?syevd/?heevd求解。
KES_SCALAPACK(默认值)	采用SCALAPACK的p?syevd/p?heevd求解。

版权所有 © 华为技术有限公司

6.5.4.3.4.6 KesSolverSetConvergeDetectorType

接口定义

KesReturnCode KesSolverSetConvergeDetectorType(KesSolver solver, KesConvergeDetectorType type);

接口说明

设置收敛判别标准，默认值为KES_RESIDUALS。

参数

参数名	类型	描述	输入/输出
solver	KesSolver	求解器句柄，不能为空指针。	输入/输出
type	KesConvergeDetectorType	收敛判别标准。	输入

KesConvergeDetectorType	描述
KES_RESIDUALS	残差||Ax - kBx||小于收敛阈值。
KES_RELATIVE_RESIDUALS	相对残差||Ax - kBx|| / |k|小于收敛阈值。
KES_EIGENVALUES	与上一迭代步的特征值之差|k - k'|小于收敛阈值。
KES_RESIDUALS_OR_EIGENVALUES	残差|| Ax - kBx||或|k - k'|小于收敛阈值。

6.5.4.3.4.7 KesSolverSetOrthogonalizerType

接口定义

KesReturnCode KesSolverSetOrthogonalizerType(KesSolver solver, KesOrthogonalizerType type);

接口说明

设置求解正交基组向量的算法，默认值为KES_CHOLESKY_QR。

版权所有 © 华为技术有限公司

参数

参数名	类型	描述	输入/输出
solver	KesSolver	求解器句柄，不能为空指针。	输入/输出
type	KesOrthogonalizerType	求解正交基组向量的算法。	输入

KesOrthogonalizerType	描述
KES_CHOLESKY_QR	基于Cholesky的QR分解算法。
KES_SVD_QR	基于SVD的QR分解算法。

6.5.4.3.4.8 KesSolverSetOrthoProjectorType

接口定义

KesReturnCode KesSolverSetOrthoProjectorType(KesSolver solver, KesOrthoProjectorType type);

接口说明

设置正交投影的算法，默认值为KES_GRAM_SCHMIDT。

参数

参数名	类型	描述	输入/输出
solver	KesSolver	求解器句柄，不能为空指针。	输入/输出
type	KesOrthoProjectorType	正交投影的算法。	输入

KesOrthoProjectorType	描述
KES_GRAM_SCHMIDT	Block-Gram-Schmidt正交投影。

6.5.4.3.5 运行求解器

 版权所有 © 华为技术有限公司

6.5.4.3.5.1 KesRunSolver

接口定义

KesReturnCode KesRunSolver(const KesSolver solver, const KesMatrix A, const KesMatrix B, const KesMatrix P, KesVector X, KesVector D);

接口说明

运行求解器。

参数

参数名	类型	描述	输入/输出
solver	const KesSolver	求解器句柄，不能为空指针。	输入
A	const KesMatrix	AX=BXD中的A矩阵。	输入
B	const KesMatrix	AX=BXD中的B矩阵。	输入
P	const KesMatrix	A的预条件矩阵。	输入
X	KesVector	AX=BXD中的特征向量X。	输入
D	KesVector	AX=BXD中的特征值D。	输出

6.5.4.3.6 查询求解信息

6.5.4.3.6.1 KesEigResidual

接口定义

KesReturnCode KesEigResidual(const KesMatrix A, const KesMatrix B, const KesVector X, const KesVector D, KesVector R);

接口说明

计算特征值/特征向量的残差。

参数

参数名	类型	描述	输入/输出
A	const KesMatrix	残差R = AX - BXD中的A矩阵。	输入
B	const KesMatrix	残差R = AX - BXD中的B矩阵。	输入

版权所有 © 华为技术有限公司

参数名	类型	描述	输入/输出
X	const KesVector	残差R = AX - BXD中的特征向量X。	输入
D	const KesVector	残差R = AX - BXD中的特征值D。	输入
R	KesVector	残差向量。	输出

6.5.4.3.6.2 KesGetSolverIter

接口定义

KesReturnCode KesGetSolverIter(KesSolver solver, int *nIter);

接口说明

查询求解器的迭代次数。

参数

参数名	类型	描述	输入/输出
solver	KesSolver	求解器句柄，不能为空指针。	输入
nIter	int *	求解器的迭代次数。	输出

6.5.4.3.6.3 KesGetSolverConv

接口定义

KesReturnCode KesGetSolverConv(KesSolver solver, int *nConv);

接口说明

查询求解器成功求解的特征值个数。

参数

参数名	类型	描述	输入/输出
solver	KesSolver	求解器句柄，不能为空指针。	输入
nConv	int *	成功求解的特征值个数。	输出

 版权所有 © 华为技术有限公司

6.5.4.3.6.4 KesVectorNorm

接口定义

KesReturnCode KesVectorNorm(const KesVector X, KesVector normX);

接口说明

计算向量的2-NORM。

参数

参数名	类型	描述	输入/输出
X	const KesVector	向量句柄，不能为空指针。	输入
normX	KesVector	向量矩阵，normX存储X的2-NORM。	输入/输出

6.5.4.4 示例

```
/*****************************************************************************
   KML_EigenSolver example codes for solving eigenvalues problem AX = BXD
*****************************************************************************/

#include "kes.h"

void MyMatVec4A(void *kesA, void *kesX, void *kesAX, int nRows, int nCols, int ld)
{
   // user-defined mat-vec routine for matrix A ......
}

void MyMatVec4B(void *kesB, void *kesX, void *kesBX, int nRows, int nCols, int ld)
{
   // user-defined mat-vec routine for matrix B ......
}

void MyMatVec4P(void *kesP, void *kesX, void *kesPX, int nRows, int nCols, int ld)
{
   // user-defined mat-vec routine for preconditioner P ......
}

int main (int argc, char **argv) {
   MPI_Init(&argc, &argv);
   int rank, size;
   MPI_Comm_rank(MPI_COMM_WORLD, &rank);
   MPI_Comm_size(MPI_COMM_WORLD, &size);

/*
   set parameters
*/
   int nRows = 1000, nCols = 100;
   MPI_Comm rowComm = MPI_COMM_WORLD;
   MPI_Comm colComm = MPI_COMM_SELF;

/*
   create matrix, vector, and solver
*/
   KesMatrix A, P, B;
   KesVector X, D;
```


版权所有 © 华为技术有限公司

```
    KesSolver solver;

    KesScalarType scalarType = KES_SCALAR_FP64;
    KesSolverAlgorithm solverAlgorithm = KES_LOBPCG;
    KesMatrixFormat matrixFormat = KES_SHELL;
    KesVectorFormat vectorFormat = KES_DENSE;

    KesMatrixCreate(&A, scalarType, matrixFormat);
    KesMatrixCreate(&B, scalarType, matrixFormat);
    KesMatrixCreate(&P, scalarType, matrixFormat);
    KesVectorCreate(&X, scalarType, vectorFormat);
    KesVectorCreate(&D, scalarType, vectorFormat);
    KesSolverCreate(&solver, scalarType, solverAlgorithm);

    KesMatrixSetDim(A, nRows * size, nRows * size);
    KesMatrixSetDim(B, nRows * size, nRows * size);

    KesVectorSetDim(X, nRows, nCols);
    KesVectorSetComm(X, KES_ROW, rowComm);
    KesVectorSetComm(X, KES_COL, colComm);

    KesVectorSetDim(D, nCols, 1);
    KesVectorSetComm(D, KES_ROW, rowComm);
    KesVectorSetComm(D, KES_COL, colComm);

    double *X_vals = new double[nRows * nCols];
    double *D_vals = new double[nCols];
    KesVectorDenseSetValues(X, X_vals, nRows);
    KesVectorDenseSetValues(D, D_vals, nCols);

    double *A_vals = new double[nRows * nRows];
    double *B_vals = new double[nRows * nRows];
    double *P_vals = new double[nRows];
    KesMatrixSetShellMatVec(A, A_vals, MyMatVec4A);
    KesMatrixSetShellMatVec(B, B_vals, MyMatVec4B);
    KesMatrixSetShellMatVec(P, P_vals, MyMatVec4P);

/*
    set solver-related parameters
*/
    int maxIter = 100000;
    int maxOrthoIter = 2;
    double threshold = 1e-6;
    KesProblemType problemType = KES_STANDARD;
    KesDenseEigenSolverType denseEigenSolverType = KES_SCALAPACK;
    KesOrthoProjectorType orthoProjectorType = KES_GRAM_SCHMIDT;
    KesOrthogonalizerType orthogonalizerType = KES_CHOLESKY_QR;
    KesConvergeDetectorType convergeDetectorType = KES_RESIDUALS;

    KesSolverSetThreshold(solver, threshold);
    KesSolverSetMaxIter(solver, maxIter);
    KesSolverSetMaxOrthoIter(solver, maxOrthoIter);
    KesSolverSetProblemType(solver, problemType);
    KesSetDenseEigenSolverType(solver, denseEigenSolverType);
    KesSetConvergeDetectorType(solver, convergeDetectorType);
    KesSetOrthogonalizerType(solver, orthogonalizerType);
    KesSetOrthoProjectorType(solver, orthoProjectorType);

/*
    run the solver
*/
    KesRunSolver(solver, A, B, P, X, D);

/*
    get solver information
*/
    int nIter;
    int nConv;
    KesGetSolverConv(solver, &nConv);
```

 版权所有 © 华为技术有限公司

```
    KesGetSolverIter(solver, &nIter);
    if (rank == 0) {
        printf(" Number of iteration steps = %d\n", nIter);
        printf(" Number of converged eigenvalues = %d\n", nConv);
    }

/*
    compute residuals
*/
    KesVector R;
    KesVectorReplicate(X, &R, true, true);
    KesVector normR;
    KesVectorReplicate(D, &normR, true, true);
    KesEigResidual(A, B, X, D, R);
    KesVectorNorm(R, normR);

/*
    clean up
*/
    delete[] A_vals;
    delete[] B_vals;
    delete[] P_vals;
    delete[] X_vals;
    delete[] D_vals;

/*
    clean up internal object
*/
    KesMatrixDestroy(A);
    KesMatrixDestroy(B);
    KesMatrixDestroy(P);
    KesVectorDestroy(X);
    KesVectorDestroy(D);
    KesVectorDestroy(R);
    KesVectorDestroy(normR);
    KesSolverDestroy(solver);

    MPI_Finalize();
    return 0;
}
```

6.5.5 环境变量配置

以下环境变量会用于应用数学库中。

表 6-66 通用环境变量说明

环境变量名	变量意义	有效值	默认值
HGEMM_F16F16F32_MAX_KB	设置hgemm最大kb数	整数	1'000'000'000'000'000L
KML_SCADSS_VARIANT	scadss调度策略	整数（0|1），默认0	0
KSCALAPACK_REDIST_TO_LOCAL_APPLICABLE	scalapack本地内存适配	O\D\F	D
KML_NUM_THREADS	local线程数设置	整数	0
KML_ALLOCATE_POLICY	alloc规则	整数	0

版权所有 © 华为技术有限公司

环境变量名	变量意义	有效值	默认值
KML_ALLOCATE_HUGEPAGE_SIZE	alloc大页规模	整数	512*1024*1024
KML_PXGETRF_FORCE_CALU	pgetrf函数使用CALU算法	整数	0
KML_FAST_EIGENSOLVER	使用高性能版本特征值求解器	整数	1
KML_TSQR_CHECK	对于高瘦型QR分解进行结果检查	整数	0
KML_MPI_USE_SHARED_MEMORY	MPI使用共享内存	整数	1
OMP_PLACES	OMP内置参数	cores	-
OMP_PROC_BIND	OMP内置参数	TRUE	-
KML_VERBOSE	打印等级	0/1/2/3	0

表 6-67 KML_DSS 环境变量说明

环境变量名	变量意义	有效值	默认值
KML_DSS_PPN	开启autotune后每个节点的进程个数	正整数	2
KML_DSS_SCHE_MODE	DSS调度模式	STATIC \DYNAMIC \KUPL\LAST	LAST
KML_DSS_SCALE_ALGO	根据行列最大值进行放缩，当原始矩阵各行各列最大值差异过大时，可以改善精度。若选择ROWCOL_SCAN，则Analyze阶段需要传入有效的矩阵values数值	NONE \ROWCOL_SCAN	NONE
KML_DSS_SGEMM_ISA	求解器内部SGEMM选择合适的算子优化分支	NEON \SVE256	通过机型判断
KML_DSS_DGEMM_ISA	求解器内部DGEMM选择合适的算子优化分支	NEON \SVE256	通过机型判断

 版权所有 © 华为技术有限公司

环境变量名	变量意义	有效值	默认值
KML_DSS_CGEMM_ISA	求解器内部CGEMM选择合适的算子优化分支	NEON\SVE256	通过机型判断
KML_DSS_ZGEMM_ISA	求解器内部ZGEMM选择合适的算子优化分支	NEON\SVE256	通过机型判断
KML_DSS_DSYRK_ISA	求解器内部DSYRK选择合适的算子优化分支	NEON\SVE256	通过机型判断
KML_DSS_ZSYRK_ISA	求解器内部ZSYRK选择合适的算子优化分支	NEON\SVE256	通过机型判断
KML_DSS_DSYRKMM_ISA	求解器内部DSYRKMM选择合适的算子优化分支	NEON\SVE256	通过机型判断
KML_DSS_ZSYRKMM_ISA	求解器内部ZSYRKMM选择合适的算子优化分支	NEON\SVE256	通过机型判断
KML_DSS_SGETRF_ISA	求解器内部SGETRF选择合适的算子优化分支	NEON\SVE256	通过机型判断
KML_DSS_DGETRF_ISA	求解器内部DGETRF选择合适的算子优化分支	NEON\SVE256	通过机型判断
KML_DSS_CGETRF_ISA	求解器内部CGETRF选择合适的算子优化分支	NEON\SVE256	通过机型判断
KML_DSS_ZGETRF_ISA	求解器内部ZGETRF选择合适的算子优化分支	NEON\SVE256	通过机型判断
KML_DSS_DPOTRF_ISA	求解器内部DPOTRF选择合适的算子优化分支	NEON\SVE256	通过机型判断

说明

通过机型判断指在TaiShan 200 1280、TaiShan 200 2280上默认值为NEON，TaiShan 200 1280 V2；TaiShan 200 2280 V2机器上默认值为SVE256。

6.6 其他

版权所有 © 华为技术有限公司

6.6.1 KML_JAVA 库函数说明

6.6.1.1 函数说明

基于JNI技术，用Java语言封装的KML数学库。

表 6-68 BLAS 库函数族说明

函数族	数据类型	描述
?gemm	s，d，c，z	一般矩阵乘矩阵。
?hemm	c, z	复数埃尔米特矩阵乘矩阵。
?her2k	c，z	复数矩阵乘矩阵秩2更新，矩阵C为复数埃尔米特矩阵。
?symm	s，d，c，z	实数对称矩阵乘矩阵。
?syrk	s，d，c，z	矩阵乘矩阵秩k更新，矩阵C为对称矩阵。
?syr2k	s，d，c，z	矩阵乘矩阵秩2更新，矩阵C为对称矩阵。
?trmm	s，d，c，z	矩阵乘矩阵，其中一个矩阵为三角矩阵。
?trsm	s，d，c，z	三角矩阵方程求解。
?gemm3m	c，z	一般复数矩阵乘矩阵。

表 6-69 VML 库函数族说明

函数族	数据类型	描述
v?Add	s	向量加法函数
v?Sub	s	向量减法函数
v?Mul	s	向量乘法函数
v?Div	s	向量除法函数

6.6.1.2 函数定义

KML_JAVA库的函数定义与C语言库一致，请参考**6.4.8 KML_BLAS库函数说明**~**6.5.3 KML_SOLVER库函数说明**。

6.6.1.2.1 BLAS 库函数

根据Java语言的书写风格，矩阵默认使用行主序。

枚举类型

表6-70描述了KML_JAVA库中BLAS的枚举类型。

版权所有 © 华为技术有限公司

表 6-70 BLAS 枚举类型及定义

枚举类型	说明	定义

Transpose	定义常规矩阵或共轭矩阵是否进行转置。	public enum Transpose {
		NO_TRANS(111),
		TRANS(112),
		CONJ_TRANS(113),
		CONJ_NO_TRANS(114);
		}

Uplo	定义使用矩阵的上三角还是下三角部分。	public enum Uplo {
		UPPER=121,
		LOWER=122;
		}

Diag	定义使用矩阵是否为单元矩阵。	public enum Diag {
		NON_UNIT=131,
		UNIT=132;
		}

Side	定义矩阵为左操作还是右操作元素。	public enum Side {
		LEFT=141,
		RIGHT=142;
		}

依赖

import com.huawei.kml.BLAS;

若用户需使用枚举体，请显式声明：

import com.huawei.kml.BLAS.Diag;

import com.huawei.kml.BLAS.Side;

import com.huawei.kml.BLAS.Transpose;

import com.huawei.kml.BLAS.Uplo;

示例

```
public class DMMTest {
    void testGemm() {
        int sizeM = 4;
        int sizeN = 4;
        int sizeK = 3;
        double[] matrixA = new double[]{ 0.340188, 0.411647, -0.222225,
                                        -0.105617, -0.302449, 0.053970,
                                         0.283099, -0.164777, -0.022603,
                                         0.298440, 0.268230, 0.128871};
        double[] matrixB = new double[]{ -0.135216, 0.416195, -0.358397, -0.257113,
                                         0.013401, 0.135712, 0.106969, -0.362768,
```

 版权所有 © 华为技术有限公司

```
                                    0.452230, 0.217297, -0.483699, 0.304177};
        double[] matrixC = new double[]{ -0.343321, 0.498924, 0.112640, -0.006417,
                                    -0.099056, -0.281743, -0.203968, 0.472775,
                                    -0.370210, 0.012932, 0.137552, -0.207483,
                                    -0.391191, 0.339112, 0.024287, 0.271358};
        double alpha = 1.0;
        double beta = 2.0;
        BLAS.dgemm( Transpose.NO_TRANS, Transpose.NO_TRANS, sizeM, sizeN, sizeK, alpha, matrixA, sizeK,
matrixB, sizeN, beta, matrixC, sizeN);
    }
}
/*
     * matrixC:
     *     -0.827621       1.147010        0.254881        -0.317229
     *     -0.163476       -0.636762       -0.428542       1.098841
     *     -0.791128       0.116416        0.166949        -0.434854
     *     -0.760862       0.866839        -0.092028       0.407877
     *      */
```

6.6.1.2.2 VML 库函数

依赖

import com.huawei.kml.VML;

示例

```
public class VMLTest {
    void testAdd() {
        int len = 5;
        double[] src1 = new double[]{1, 2, 3, 4, 5};
        double[] src2 = new double[]{0.1, 0.2, 0.3, 0.4, 0.5};
        double[] dst = new double[src1.length];
        VML.vsAdd(len, src1, src2, dst);
    }
}
/* dst: 1.1, 2.2, 3.3, 4.4, 5.5 */
```

6.6.2 KML_NUMPY 库函数说明

6.6.2.1 函数说明

当前KML_NUMPY库提供了40个适配KML或自行优化后的接口，优化的数据类型为float和double，涵盖了基础运算、傅里叶变换、统计和线性代数四个部分。

NumPy（Numerical Python）是Python的一种开源的数值计算扩展。这种工具可用来存储和处理大型矩阵，比Python自身的嵌套列表（nested list structure）结构要高效得多（该结构也可以用来表示矩阵），支持大量的维度数组与矩阵运算，此外也针对数组运算提供大量的数学函数库。

6.6.2.2 数据类型和数据存储

提供KML_NUMPY库函数的数据类型和数据存储说明。

 版权所有 © 华为技术有限公司

NumPy 数据类型

类型	描述	备注
bool_	布尔型数据类型（True或False）。	NumPy的数据类型实际上是dtype对象的实例，并可对应唯一的字符，如numpy.bool__，numpy.int32，numpy.float64等。
int_	默认的整数类型（类似于C语言中的long，int32或int64）。	
intc	与C的int类型一样，一般是int32或int64。	
intp	用于索引的整数类型（类似于C的ssize_t，一般情况下仍然是int32或int64）。	
int8	字节（-128~127）。	
int16	整数（-32768~32767）。	
int32	整数（-2147483648~2147483647）。	
int64	整数（-9223372036854775808~9223372036854775807）。	
uint8	无符号整数（0~255）。	
uint16	无符号整数（0~65535）。	
uint32	无符号整数（0~4294967295）。	
uint64	无符号整数（0~18446744073709551615）。	
float_	float64类型的简写。	
float16	半精度浮点数，包括：1个符号位，5个指数位，10个尾数位。	
float32	单精度浮点数，包括：1符号位，8指数位，23个尾数位。	
float64	双精度浮点数，包括：1个符号位，11个指数位，52个尾数位。	
complex_	complex128类型的简写，即128位复数。	
complex64	复数，表示双32位浮点数（实数部分和虚数部分）。	
complex128	复数，表示双64位浮点数（实数部分和虚数部分）。	

NumPy 接口其它参数类型

- array_like：指可以作为第一个参数传递numpy.array()以创建数组的任何内容。
- **可选:** ndarray：存储单一数据类型的多维数组，可以使用索引（从0开始）访问ndarray对象中的元素。任何array_like对象都可以使用函数numpy.array()转换为ndarray对象。

 版权所有 © 华为技术有限公司

- scalar：表示类型为标量，标量可能的类型为**NumPy数据类型**中描述的数据类型。

6.6.2.3 函数定义

6.6.2.3.1 基础运算函数

6.6.2.3.1.1 add

功能描述

将输入参数逐元素相加。

必选输入参数

参数名	类型	说明
x1，x2	array_like	相加的数组或标量，如果x1.shape!=x2.shape，则它们必须能广播至相同的形状。

可选输入参数

参数名	类型	默认值	说明
out	ndarray或ndarray的元组	None	计算结果保存的位置。如果提供，其形状必须与输入数组广播后的形状一致。未指定时，返回一个新的数组。
where	array_like	None	将被广播至输入的条件数组。在条件为真处的元素，其计算结果会存入out中，其余位置保留原有值。
**kwargs	-	-	其他关键字参数，请参见NumPy官方文档**Universal functions (ufunc)**。

返回数据

类型	说明
ndarray/scalar	x1和x2逐元素相加的结果。

版权所有 © 华为技术有限公司

示例

```
>>> import numpy as np
>>> np.add(1.1, 5.5)
6.6
>>> x1 = np.arange(9.0).reshape((3,3))
>>> x2 = np.arange(3.0)
>>> x1
array([[0., 1., 2.],
       [3., 4., 5.],
       [6., 7., 8.]])
>>> x2
array([0., 1., 2.])
>>>
>>> np.add(x1, x2)
array([[ 0.,  2.,  4.],
       [ 3.,  5.,  7.],
       [ 6.,  8., 10.]])
>>

#  若x1和x2均为ndarray，那么还可以使用“+”来替代np.add

>>> x1 = np.arange(0, 4).reshape((2,2))
>>> x1
array([[0, 1],
       [2, 3]])
>>> x2 = np.arange(4, 8).reshape((2,2))
>>> x2
array([[4, 5],
       [6, 7]])
>>>
>>> x1 + x2
array([[ 4,  6],
       [ 8, 10]])
>>>
```

6.6.2.3.1.2 subtract

功能描述

将输入参数逐元素相减。

必选输入参数

参数名	类型	说明
x1，x2	array_like	相减的数组或标量，如果x1.shape!=x2.shape，则它们必须能广播至相同的形状。

可选输入参数

参数名	类型	默认值	说明
out	ndarray或ndarray的元组	None	计算结果保存的位置。如果提供，其形状必须与输入数组广播后的形状一致。未指定时，返回一个新的数组。

版权所有 © 华为技术有限公司

参数名	类型	默认值	说明
where	array_like	None	将被广播至输入的条件数组。在条件为真处的元素，其计算结果会存入out中，其余位置保留原有值。
**kwargs	-	-	其他关键字参数，请参见NumPy官方文档**Universal functions (ufunc)**。

返回数据

类型	说明
ndarray/scalar	x1和x2逐元素相减的结果。

示例

```
>>> import numpy as np
>>> np.subtract(1.1, 5.5)
-4.4
>>> x1 = np.arange(9.0).reshape((3,3))
>>> x2 = np.arange(3.0)
>>> x1
array([[0., 1., 2.],
       [3., 4., 5.],
       [6., 7., 8.]])
>>> x2
array([0., 1., 2.])
>>>
>>> np.subtract(x1, x2)
array([[ 0.,  0.,  0.],
       [ 3.,  3.,  3.],
       [ 6.,  6.,  6.]])
>>

#  若x1和x2均为ndarray，那么还可以使用“-”来替代np.subtract

>>> x1 = np.arange(0, 4).reshape((2,2))
>>> x1
array([[0, 1],
       [2, 3]])
>>> x2 = np.arange(4, 8).reshape((2,2))
>>> x2
array([[4, 5],
       [6, 7]])
>>>
>>> x1 - x2
array([[ -4,  -4],
       [ -4,  -4]])
>>>
```

6.6.2.3.1.3 multiply

功能描述

将输入参数逐元素相乘。

 版权所有 © 华为技术有限公司

必选输入参数

参数名	类型	说明
x1，x2	array_like	相乘的数组或标量，如果x1.shape!=x2.shape，则它们必须能广播至相同的形状。

可选输入参数

参数名	类型	默认值	说明
out	ndarray或ndarray的元组	None	计算结果保存的位置。如果提供，其形状必须与输入数组广播后的形状一致。未指定时，返回一个新的数组。
where	array_like	None	将被广播至输入的条件数组。在条件为真处的元素，其计算结果会存入out中，其余位置保留原有值。
**kwargs	-	-	其他关键字参数，请参见NumPy官方文档**Universal functions (ufunc)**。

返回数据

类型	说明
ndarray/scalar	x1和x2逐元素相乘的结果。

示例

```
>>> import numpy as np
>>> np.multiply(1.1, 5.5)
6.050000000000001
>>> x1 = np.arange(9.0).reshape((3,3))
>>> x2 = np.arange(3.0)
>>> x1
array([[0., 1., 2.],
       [3., 4., 5.],
       [6., 7., 8.]])
>>> x2
array([0., 1., 2.])
>>>
>>> np.multiply(x1, x2)
array([[ 0.,  1.,  4.],
       [ 0.,  4., 10.],
       [ 0.,  7., 16.]])
>>

#  若x1和x2均为ndarray，那么还可以使用“*”来替代np.multiply
```

 版权所有 © 华为技术有限公司

```
>>> x1 = np.arange(0, 4).reshape((2,2))
>>> x1
array([[0, 1],
       [2, 3]])
>>> x2 = np.arange(4, 8).reshape((2,2))
>>> x2
array([[4, 5],
       [6, 7]])
>>>
>>> x1 * x2
array([[ 0,  5],
       [12, 21]])
>>>
```

6.6.2.3.1.4 exp

功能描述

对每个元素计算以自然常数为底的对应次幂，即 e^x 。

必选输入参数

参数名	类型	说明
x	array_like	输入的数组或标量。

可选输入参数

参数名	类型	默认值	说明
out	ndarray或ndarray的元组	None	计算结果保存的位置。如果提供，其形状必须与输入数组广播后的形状一致。未指定时，返回一个新的数组。
where	array_like	None	将被广播至输入的条件数组。在条件为真处的元素，其计算结果会存入out中，其余位置保留原有值。
**kwargs	-	-	其他关键字参数，请参见NumPy官方文档**Universal functions (ufunc)**。

返回数据

类型	说明
ndarray/scalar	自然常数e对应次幂的结果。

版权所有 © 华为技术有限公司

示例

```
>>> import numpy as np
>>> np.exp(1.0)
2.718281828459045
>>>
>>> x = np.arange(4)
>>> x
array([0, 1, 2, 3])
>>> np.exp(x)
array([ 1.        ,  2.71828183,  7.3890561 , 20.08553692])
>>>
```

6.6.2.3.1.5 sqrt

功能描述

计算每个元素（非负）的平方根，即 $\sqrt[2]{x}$ 。

必选输入参数

参数名	类型	说明
x	array_like	输入的数组或标量。

可选输入参数

参数名	类型	默认值	说明
out	ndarray或ndarray的元组	None	计算结果保存的位置。如果提供，其形状必须与输入数组广播后的形状一致。未指定时，返回一个新的数组。
where	array_like	None	将被广播至输入的条件数组。在条件为真处的元素，其计算结果会存入out中，其余位置保留原有值。
**kwargs	-	-	其他关键字参数，请参见NumPy官方文档**Universal functions (ufunc)**。

返回数据

类型	说明
ndarray/scalar	每个元素（非负）的平方根的结果。

版权所有 © 华为技术有限公司

示例

```
>>> import numpy as np
>>> np.sqrt(4)
2.0
>>>
>>> x = np.array([2.25, 6.25, 25])
>>> np.sqrt(x)
array([1.5, 2.5, 5. ])
>>>
```

6.6.2.3.1.6 log

功能描述

计算每个元素（非负）的自然对数，即ln(x)。

必选输入参数

参数名	类型	说明
x	array_like	输入的数组或标量。

可选输入参数

参数名	类型	默认值	说明
out	ndarray/ndarray的元组	None	计算结果保存的位置。如果提供，其形状必须与输入数组广播后的形状一致。未指定时，返回一个新的数组。
where	array_like	None	将被广播至输入的条件数组。在条件为真处的元素，其计算结果会存入out中，其余位置保留原有值。
**kwargs	-	-	其他关键字参数，请参见NumPy官方文档**Universal functions (ufunc)**。

返回数据

类型	说明
ndarray/scalar	每个元素（非负）自然对数的结果。

示例

```
>>> import numpy as np
>>> np.log(1.0)
0.0
```


版权所有 © 华为技术有限公司

```
>>> x = np.array([1.0, np.e, np.e**2, 0])
>>> np.log(x)
array([ 0.,  1.,  2., -inf])
>>>
```

6.6.2.3.1.7 sin

功能描述

计算每个元素的三角正弦值，即sin(x)。

必选输入参数

参数名	类型	说明
x	array_like	输入的数组或标量，单位为弧度。

可选输入参数

参数名	类型	默认值	说明
out	ndarray或ndarray的元组	None	计算结果保存的位置。如果提供，其形状必须与输入数组广播后的形状一致。未指定时，返回一个新的数组。
where	array_like	None	将被广播至输入的条件数组。在条件为真处的元素，其计算结果会存入out中，其余位置保留原有值。
**kwargs	-	-	其他关键字参数，请参见NumPy官方文档**Universal functions (ufunc)**。

返回数据

类型	说明
ndarray/scalar	每个元素的三角正弦计算结果。

示例

```
>>> import numpy as np
>>> np.sin(np.pi / 2)
1.0
>>> x = np.array([0., 30., 45., 60., 90.]) * np.pi / 180.0
>>> np.sin(x)
array([0.        , 0.5       , 0.70710678, 0.8660254 , 1.        ])
>>>
```

 版权所有 © 华为技术有限公司

6.6.2.3.1.8 cos

功能描述

计算每个元素的三角余弦值，即cos(x)。

必选输入参数

参数名	类型	说明
x	array_like	输入的数组或标量，单位为弧度。

可选输入参数

参数名	类型	默认值	说明
out	ndarray或ndarray的元组	None	计算结果保存的位置。如果提供，其形状必须与输入数组广播后的形状一致。未指定时，返回一个新的数组。
where	array_like	None	将被广播至输入的条件数组。在条件为真处的元素，其计算结果会存入out中，其余位置保留原有值。
**kwargs	-	-	其他关键字参数，请参见NumPy官方文档**Universal functions (ufunc)**。

返回数据

类型	说明
ndarray/scalar	每个元素的三角余弦计算结果。

示例

```
>>> import numpy as np
>>> np.cos(np.pi)
-1.0
>>> x = np.array([0., 30., 45., 60., 90.]) * np.pi / 180.0
>>> np.cos(x)
array([1.00000000e+00, 8.66025404e-01, 7.07106781e-01, 5.00000000e-01,
       6.12323400e-17])
>>>
```


版权所有 © 华为技术有限公司

6.6.2.3.1.9 tan

功能描述

计算每个元素的三角正切值，即tan(x)。

必选输入参数

参数名	类型	说明
x	array_like	输入的数组或标量，单位为弧度。

可选输入参数

参数名	类型	默认值	说明
out	ndarray或ndarray的元组	None	计算结果保存的位置。如果提供，其形状必须与输入数组广播后的形状一致。未指定时，返回一个新的数组。
where	array_like	None	将被广播至输入的条件数组。在条件为真处的元素，其计算结果会存入out中，其余位置保留原有值。
**kwargs	-	-	其他关键字参数，请参见NumPy官方文档**Universal functions (ufunc)**。

返回数据

类型	说明
ndarray/scalar	每个元素的三角正切计算结果。

示例

```
>>> import numpy as np
>>> np.tan(np.pi / 4)
0.9999999999999999
>>> x = np.array([0., 30., 45., 60., 90.]) * np.pi / 180.0
>>> np.tan(x)
array([0.00000000e+00, 5.77350269e-01, 1.00000000e+00, 1.73205081e+00,
       1.63312394e+16])
>>>
```


版权所有 © 华为技术有限公司

6.6.2.3.1.10 arctan

功能描述

计算每个元素的反三角正切值，即$\tan^{-1}(x)$。

必选输入参数

参数名	类型	说明
x	array_like	输入的数组或标量。

可选输入参数

参数名	类型	默认值	说明
out	ndarray或ndarray的元组	None	计算结果保存的位置。如果提供，其形状必须与输入数组广播后的形状一致。未指定时，返回一个新的数组。
where	array_like	None	将被广播至输入的条件数组。在条件为真处的元素，其计算结果会存入out中，其余位置保留原有值。
**kwargs	-	-	其他关键字参数，请参见NumPy官方文档**Universal functions (ufunc)**。

返回数据

类型	说明
ndarray/scalar	每个元素的反三角正切计算结果，单位为弧度。实部的取值范围为[-π/2, π/2]。

示例

```
>>> import numpy as np
>>> np.pi / 4
0.7853981633974483
>>> np.tan([0., np.pi / 4])
array([0., 1.])
>>>
>>> np.arctan([0., 1.])
array([0.        , 0.78539816])
>>>
```


版权所有 © 华为技术有限公司

6.6.2.3.1.11 power

功能描述

将第一个输入中的每个元素作为底数，第二个输入中的每个元素作为指数，计算其结果，即 $x1[i]^{x2[i]}$ 。

x1和x2必须能广播至相同的形状。若x1[i]为整数且x2[i]为负整数，则会抛出ValueError异常。若x1[i]<0且x2[i]不是整数，则返回nan。返回复数结果需要将输入的数据类型转换为complex或者指定dtype=complex。

必选输入参数

参数名	类型	说明
x1	array_like	底数，输入的数组或标量。
x2	array_like	指数，输入的数组或标量。若x1.shape!=x2.shape，则它们必须能广播至相同形状。

可选输入参数

参数名	类型	默认值	说明
out	ndarray或ndarray的元组	None	计算结果保存的位置。如果提供，其形状必须与输入数组广播后的形状一致。未指定时，返回一个新的数组。
where	array_like	None	将被广播至输入的条件数组。在条件为真处的元素，其计算结果会存入out中，其余位置保留原有值。
**kwargs	-	-	其他关键字参数，请参见NumPy官方文档**Universal functions (ufunc)**。

返回数据

类型	说明
ndarray/scalar	幂运算的结果。

示例

```
>>> import numpy as np
>>> np.power(3, 2)
```

 版权所有 © 华为技术有限公司

```
9
>>> x1 = np.arange(6)
>>> x1
array([0, 1, 2, 3, 4, 5])
>>> np.power(x1, 3)
array([  0,   1,   8,  27,  64, 125])
>>>
>>> x2 = [1.0, 2.0, 3.0, 3.0, 2.0, 1.0]
>>> np.power(x1, x2)
array([ 0.,  1.,  8., 27., 16.,  5.])
>>>
```

6.6.2.3.1.12 fmod

功能描述

返回按元素划分的除法余数。注意：这是C库函数fmod的NumPy实现，返回的余数与x1具有相同符号，等效于Matlab的rem函数，不应与Python的模数运算x1 % x2混淆。

必选输入参数

参数名	类型	说明
x1	array_like	被除数，输入的数组或标量。
x2	array_like	除数，输入的数组或标量。若x1.shape!=x2.shape，则它们必须能广播至相同形状。

可选输入参数

参数名	类型	默认值	说明
out	ndarray或ndarray的元组	None	计算结果保存的位置。如果提供，其形状必须与输入数组广播后的形状一致。未指定时，返回一个新的数组。
where	array_like	None	将被广播至输入的条件数组。在条件为真处的元素，其计算结果会存入out中，其余位置保留原有值。
**kwargs	-	-	其他关键字参数，请参见NumPy官方文档**Universal functions (ufunc)**。

返回数据

类型	说明
ndarray/scalar	x1 % x2的结果，结果符号与x1保持一致。

 版权所有 © 华为技术有限公司

示例

```
>>> import numpy as np
>>> np.fmod(-3,2)
-1
>>>
>>> x1 = [-1, 2, -3, 4, -5]
>>> np.fmod(x1, 2)
array([-1,  0, -1,  0, -1])
>>>
>>> x2 = [1, -1, 2, -2, 3]
>>> np.fmod(x1, x2)
array([ 0,  0, -1,  0, -2])
>>>
```

6.6.2.3.1.13 exp2

功能描述

对每个元素计算以2为底的对应次幂，即 2^x 。

必选输入参数

参数名	类型	说明
x	array_like	输入的数组或标量。

可选输入参数

参数名	类型	默认值	说明
out	ndarray或ndarray的元组	None	计算结果保存的位置。如果提供，其形状必须与输入数组广播后的形状一致。未指定时，返回一个新的数组。
where	array_like	None	将被广播至输入的条件数组。在条件为真处的元素，其计算结果会存入out中，其余位置保留原有值。
**kwargs	-	-	其他关键字参数，请参见NumPy官方文档**Universal functions (ufunc)**。

返回数据

类型	说明
ndarray/scalar	2对应次幂的结果。

版权所有 © 华为技术有限公司

示例

```
>>> import numpy as np
>>>
>>> np.exp2(4)
16.0
>>>
>>> np.exp2([1, 2, 3])
array([2., 4., 8.])
>>>
```

6.6.2.3.2 傅里叶变换函数

6.6.2.3.2.1 fft.fft

功能描述

计算一维离散傅里叶变换。

必选输入参数

参数名	类型	说明
a	array_like	输入数组，可为复数。

可选输入参数

参数名	类型	默认值	说明
n	int	None	输出的变换轴的长度。 • 如果“n”小于输入的长度，则输入被裁剪。 • 如果“n”大于输入的长度，输入将填充零。 • 如果未给出“n”，则使用沿“轴”指定的轴的输入长度。
axis	int	-1	计算FFT的轴。如果未给出，则使用最后一个轴。
norm	{“backward”, “ortho”, “forward”}	backward	规范化模式（请参见“**numpy.fft**”）。指示对前向或后向做缩放，以及使用何种标准化因子。

版权所有 © 华为技术有限公司

返回数据

类型	说明
ndarray	沿“轴”表示的轴变换的截断或零填充输入，如果未指定“轴”，则为最后一个输入。

示例

```
>>> import numpy as np
>>> t = np.arange(12)
>>> ts = np.sin(t)
>>> ts
array([ 0.        ,  0.84147098,  0.90929743,  0.14112001, -0.7568025 ,
       -0.95892427, -0.2794155 ,  0.6569866 ,  0.98935825,  0.41211849,
       -0.54402111, -0.99999021])
>>>
>>> np.sum(ts)
0.41119816466730696
>>>
>>> np.fft.fft(ts)
array([ 0.41119816+0.00000000e+00j,  0.46997981+4.11832106e-01j,
       -1.36179847-5.76500237e+00j,  0.14669493-4.96548795e-01j,
        0.20513541-2.23341703e-01j,  0.22157176-9.53854700e-02j,
        0.22563497+4.44089210e-16j,  0.22157176+9.53854700e-02j,
        0.20513541+2.23341703e-01j,  0.14669493+4.96548795e-01j,
       -1.36179847+5.76500237e+00j,  0.46997981-4.11832106e-01j])
>>>
```

6.6.2.3.2.2 fft.ifft

功能描述

计算一维逆向离散傅里叶变换。

必选输入参数

参数名	类型	说明
a	array_like	输入数组，可为复数。

可选输入参数

参数名	类型	默认值	说明
n	int	None	输出的变换轴的长度。 ● 如果“n”小于输入的长度，则输入被裁剪。 ● 如果“n”大于输入的长度，输入将填充零。 ● 如果未给出“n”，则使用沿“轴”指定的轴的输入长度。

版权所有 © 华为技术有限公司

参数名	类型	默认值	说明
axis	int	-1	计算FFT的轴。如果未给出，则使用最后一个轴。
norm	{“backward”, “ortho”, “forward”}	backward	规范化模式（请参见“numpy.fft”）。指示对前向或后向做缩放，以及使用何种标准化因子。

返回数据

类型	说明
ndarray	沿“轴”表示的轴变换的截断或零填充输入，如果未指定“轴”，则为最后一个输入。

示例

```
>>> import numpy as np
>>> t = np.arange(12)
>>> ts = np.sin(t)
>>> ts
array([ 0.        ,  0.84147098,  0.90929743,  0.14112001, -0.7568025 ,
       -0.95892427, -0.2794155 ,  0.6569866 ,  0.98935825,  0.41211849,
       -0.54402111, -0.99999021])
>>>
>>> np.sum(ts)
0.41119816466730696
>>>
>>> fts = np.fft.fft(ts)
>>> fts
array([ 0.41119816+0.00000000e+00j,  0.46997981+4.11832106e-01j,
       -1.36179847-5.76500237e+00j,  0.14669493-4.96548795e-01j,
        0.20513541-2.23341703e-01j,  0.22157176-9.53854700e-02j,
        0.22563497+4.44089210e-16j,  0.22157176+9.53854700e-02j,
        0.20513541+2.23341703e-01j,  0.14669493+4.96548795e-01j,
       -1.36179847+5.76500237e+00j,  0.46997981-4.11832106e-01j])
>>>
>>> np.fft.ifft(fts)
array([ 1.85037171e-17+1.06396373e-16j,  8.41470985e-01-3.46944695e-17j,
        9.09297427e-01+7.86407976e-17j,  1.41120008e-01-3.93203988e-17j,
       -7.56802495e-01+3.38641526e-18j, -9.58924275e-01-3.46944695e-17j,
       -2.79415498e-01-4.09691099e-17j,  6.56986599e-01-3.93203988e-17j,
        9.89358247e-01-1.26382738e-17j,  4.12118485e-01-3.46944695e-17j,
       -5.44021111e-01+8.72284026e-17j, -9.99990207e-01-3.93203988e-17j])
>>>
```

6.6.2.3.2.3 fft.fft2

功能描述

计算二维离散傅里叶变换。

版权所有 © 华为技术有限公司

必选输入参数

参数名	类型	说明
a	array_like	输入数组，可为复数。

可选输入参数

参数名	类型	默认值	说明
s	sequence of ints	None	输出的形状（每个变换轴的长度，s[0]表示轴0，s[1]表示轴1，以此类推）。这对应于fft(x,n)的n。沿着每个轴： • 如果给定的形状小于输入的形状，则会裁剪输入。 • 如果给定的形状大于输入的形状，输入将填充零。 • 如果未给定s，则使用沿轴指定的轴的输入形状。
axes	sequence of ints	-1	计算FFT的轴。如果未给出，则使用最后两个轴。
norm	{“backward”, “ortho”, “forward”}, optional	backward	规范化模式（请参见“**numpy.fft**”）。指示对前向或后向做缩放，以及使用何种标准化因子。

返回数据

类型	说明
ndarray	沿“轴”表示的轴变换的截断或零填充输入，如果未指定“轴”，则为最后两个输入。

示例

```
>>> import numpy as np
>>> a = np.mgrid[:5, :5][0]
>>> a
array([[0, 0, 0, 0, 0],
       [1, 1, 1, 1, 1],
       [2, 2, 2, 2, 2],
       [3, 3, 3, 3, 3],
       [4, 4, 4, 4, 4]])
>>>
>>> np.fft.fft2(a)
array([[ 50.  +0.j        ,   0.  +0.j        ,   0.  +0.j        ,
         0.  +0.j        ,   0.  +0.j        ],
       [-12.5+17.20477401j,   0.  +0.j        ,   0.  +0.j        ,
```


版权所有 © 华为技术有限公司

```
       0.  +0.j        ,   0.  +0.j        ],
     [-12.5 +4.0614962j ,   0.  +0.j        ,   0.  +0.j        ,
       0.  +0.j        ,   0.  +0.j        ],
     [-12.5 -4.0614962j ,   0.  +0.j        ,   0.  +0.j        ,
       0.  +0.j        ,   0.  +0.j        ],
     [-12.5-17.20477401j,   0.  +0.j        ,   0.  +0.j        ,
       0.  +0.j        ,   0.  +0.j        ]])
>>>
```

6.6.2.3.2.4 fft.ifft2

功能描述

计算二维逆向离散傅里叶变换。

必选输入参数

参数名	类型	说明
a	array_like	输入数组，可为复数。

可选输入参数

参数名	类型	默认值	说明
s	sequence of ints	None	输出的形状（每个变换轴的长度，s[0]表示轴0，s[1]表示轴1，以此类推）。这对应于fft(x,n)的n。沿着每个轴： • 如果给定的形状小于输入的形状，则会裁剪输入。 • 如果给定的形状大于输入的形状，输入将填充零。 • 如果未给定s，则使用沿轴指定的轴的输入形状。
axes	sequence of ints	-1	计算FFT的轴。如果未给出，则使用最后两个轴。
norm	{“backward”, “ortho”, “forward”}, optional	backward	规范化模式（请参见“**numpy.fft**”）。指示对前向或后向做缩放，以及使用何种标准化因子。

返回数据

类型	说明
ndarray	沿“轴”表示的轴变换的截断或零填充输入，如果未指定“轴”，则为最后两个输入。

版权所有 © 华为技术有限公司

示例

```
>>> import numpy as np
>>> a = np.mgrid[:5, :5][0]
>>> a
array([[0, 0, 0, 0, 0],
       [1, 1, 1, 1, 1],
       [2, 2, 2, 2, 2],
       [3, 3, 3, 3, 3],
       [4, 4, 4, 4, 4]])
>>>
>>> t = np.fft.fft2(a)
>>> t
array([[ 50.  +0.j        ,   0.  +0.j        ,   0.  +0.j        ,
         0.  +0.j        ,   0.  +0.j        ],
       [-12.5+17.20477401j,   0.  +0.j        ,   0.  +0.j        ,
         0.  +0.j        ,   0.  +0.j        ],
       [-12.5 +4.0614962j ,   0.  +0.j        ,   0.  +0.j        ,
         0.  +0.j        ,   0.  +0.j        ],
       [-12.5 -4.0614962j ,   0.  +0.j        ,   0.  +0.j        ,
         0.  +0.j        ,   0.  +0.j        ],
       [-12.5-17.20477401j,   0.  +0.j        ,   0.  +0.j        ,
         0.  +0.j        ,   0.  +0.j        ]])
>>>
>>> np.fft.ifft2(t)
array([[0.+0.j, 0.+0.j, 0.+0.j, 0.+0.j, 0.+0.j],
       [1.+0.j, 1.+0.j, 1.+0.j, 1.+0.j, 1.+0.j],
       [2.+0.j, 2.+0.j, 2.+0.j, 2.+0.j, 2.+0.j],
       [3.+0.j, 3.+0.j, 3.+0.j, 3.+0.j, 3.+0.j],
       [4.+0.j, 4.+0.j, 4.+0.j, 4.+0.j, 4.+0.j]])
>>>
```

6.6.2.3.2.5 fft.fftn

功能描述

计算n维离散傅里叶变换。

必选输入参数

参数名	类型	说明
a	array_like	输入数组，可为复数。

版权所有 © 华为技术有限公司

可选输入参数

参数名	类型	默认值	说明
s	sequence of ints	None	输出的形状（每个变换轴的长度，s[0]表示轴0，s[1]表示轴1，以此类推）。这对应于fft(x,n)的n。沿着每个轴： • 如果给定的形状小于输入的形状，则会裁剪输入。 • 如果给定的形状大于输入的形状，输入将填充零。 • 如果未给定s，则使用沿轴指定的轴的输入形状。
axes	sequence of ints	-1	计算FFT的轴。如果未给出，则使用最后n个轴。
norm	{“backward”, “ortho”, “forward”}, optional	backward	规范化模式（请参见“**numpy.fft**”）。指示对前向或后向做缩放，以及使用何种标准化因子。

返回数据

类型	说明
ndarray	沿“轴”表示的轴变换的截断或零填充输入，如果未指定“轴”，则为最后n个输入。

示例

```
>>> import numpy as np
>>> a = np.mgrid[:3, :3, :3][0]
>>> a
array([[[0, 0, 0],
        [0, 0, 0],
        [0, 0, 0]],

       [[1, 1, 1],
        [1, 1, 1],
        [1, 1, 1]],

       [[2, 2, 2],
        [2, 2, 2],
        [2, 2, 2]]])
>>>
>>> np.fft.fftn(a)
array([[[ 27. +0.j        ,   0. +0.j        ,   0. +0.j        ],
        [  0. +0.j        ,   0. +0.j        ,   0. +0.j        ],
        [  0. +0.j        ,   0. +0.j        ,   0. +0.j        ]],

       [[-13.5+7.79422863j,   0. +0.j        ,   0. +0.j        ],
        [  0. +0.j        ,   0. +0.j        ,   0. +0.j        ],
        [  0. +0.j        ,   0. +0.j        ,   0. +0.j        ]],
```


版权所有 © 华为技术有限公司

```
      [[-13.5-7.79422863j,  0. +0.j        ,  0. +0.j        ],
       [  0. +0.j        ,  0. +0.j        ,  0. +0.j        ],
       [  0. +0.j        ,  0. +0.j        ,  0. +0.j        ]]])
>>>
>>> np.fft.fftn(a, (2,2), axes=(0,1))
array([[[ 2.+0.j,  2.+0.j,  2.+0.j],
        [ 0.+0.j,  0.+0.j,  0.+0.j]],

       [[-2.+0.j, -2.+0.j, -2.+0.j],
        [ 0.+0.j,  0.+0.j,  0.+0.j]]])
>>>
```

6.6.2.3.2.6 fft.ifftn

功能描述

计算n维逆向离散傅里叶变换。

必选输入参数

参数名	类型	说明
a	array_like	输入数组，可为复数。

可选输入参数

参数名	类型	默认值	说明
s	sequence of ints	None	输出的形状（每个变换轴的长度，s[0]表示轴0，s[1]表示轴1，以此类推）。这对应于fft(x,n)的n。沿着每个轴： ● 如果给定的形状小于输入的形状，则会裁剪输入。 ● 如果给定的形状大于输入的形状，输入将填充零。 ● 如果未给定s，则使用沿轴指定的轴的输入形状。
axes	sequence of ints	-1	计算FFT的轴。如果未给出，则使用最后n个轴。
norm	{“backward”, “ortho”, “forward”}, optional	backward	规范化模式（请参见“**numpy.fft**”）。指示对前向或后向做缩放，以及使用何种标准化因子。

版权所有 © 华为技术有限公司

返回数据

类型	说明
ndarray	沿“轴”表示的轴变换的截断或零填充输入，如果未指定“轴”，则为最后n个输入。

示例

```
>>> import numpy as np
>>> a = np.eye(4)
>>> a
array([[1., 0., 0., 0.],
       [0., 1., 0., 0.],
       [0., 0., 1., 0.],
       [0., 0., 0., 1.]])
>>>
>>> t = np.fft.fftn(a, axes=(0,))
>>> t
array([[ 1.+0.j,  1.+0.j,  1.+0.j,  1.+0.j],
       [ 1.+0.j,  0.-1.j, -1.+0.j,  0.+1.j],
       [ 1.+0.j, -1.+0.j,  1.+0.j, -1.+0.j],
       [ 1.+0.j,  0.+1.j, -1.+0.j,  0.-1.j]])
>>>
>>> np.fft.ifftn(t, axes=(1,))
array([[1.+0.j, 0.+0.j, 0.+0.j, 0.+0.j],
       [0.+0.j, 1.+0.j, 0.+0.j, 0.+0.j],
       [0.+0.j, 0.+0.j, 1.+0.j, 0.+0.j],
       [0.+0.j, 0.+0.j, 0.+0.j, 1.+0.j]])
>>>
```

6.6.2.3.3 统计基础函数

6.6.2.3.3.1 dot

功能描述

两个数组的点积。具体来说：

- 如果a和b都是一维数组，它是向量的内积（没有复数的共轭）。
- 如果a和b都是二维数组，则是矩阵乘法，但最好使用matmul或a @ b。
- 如果a或b是0-D（标量），则等价于乘法，最好使用multiply或a * b。
- 如果a是N-D数组，而b是1D数组，那么它是a的最后一个轴和b的乘积之和。
- 如果a是N-D数组，而b是M-D数组（其中M>=2），则它是a最后一个轴和b的倒数第二个轴的乘积之和。

必选输入参数

参数名	类型	说明
a，b	array_like	做点积的数组。

版权所有 © 华为技术有限公司

可选输入参数

参数名	类型	默认值	说明
out	ndarray	None	输出参数。它必须具有正确的类型，必须是C-连续的，其dtype必须是为dot(a,b)返回的dtype。为保证性能，如果不满足这些条件，就会抛出异常，而不是强行进行计算。

返回数据

类型	说明
ndarray	返回a和b的点积。如果a和b都是标量或两个1D数组，则返回标量；否则返回数组。如果参数out非空，则返回out。

示例

```
>>> import numpy as np
>>> np.dot(7, 9)
63
>>>
>>> np.dot([2j,3j], [2j, 3j])
(-13+0j)
>>>
>>> a = [[1,0], [0,1]]
>>> b = [[4,1], [2,2]]
>>> np.dot(a, b)
array([[4, 1],
       [2, 2]])
>>>
>>> a = np.arange(3*4*5*6).reshape(3,4,5,6)
>>> a = np.arange(3*4*5*6)[::-1].reshape((5,4,6,3))
>>>
>>>
>>>
>>> a = np.arange(3*4*5*6).reshape(3,4,5,6)
>>> b = np.arange(3*4*5*6)[::-1].reshape((5,4,6,3))
>>> np.dot(a, b)[2,3,2,1,2,2]
499128
>>>
>>> sum(a[2,3,2,:] * b[1,2,:,2])
499128
>>>
```

6.6.2.3.3.2 vdot

功能描述

返回两个向量的点积。

vdot(a, b)函数处理复数的方式与dot(a, b)不同。如果第一个参数是复数，则使用第一个参数的复共轭来计算点积。

 版权所有 © 华为技术有限公司

请注意，vdot处理多维数组的方式与dot不同：它不执行矩阵乘积，而是首先将输入参数扁平化为一维向量。因此，它只用于向量。

必选输入参数

参数名	类型	说明
a	array_like	第一个实参。若a是复数，则在计算点积之前取复共轭。
b	array_like	第二个实参。

可选输入参数

无。

返回数据

类型	说明
ndarray	返回a和b的点积。可以是int、float或complex，具体取决于a和b的类型。

示例

```
>>> import numpy as np
>>> a = np.array([1+2j, 3+4j])
>>> b = np.array([5+6j, 7+8j])
>>> np.vdot(a, b)
(70-8j)
>>> np.vdot(b, a)
(70+8j)
>>>
>>> a = np.array([[1,4], [5,6]])
>>> b = np.array([[4,1], [2,2]])
>>> np.vdot(a, b)
30
>>> np.vdot(b, a)
30
>>>
```

6.6.2.3.3.3 inner

功能描述

两个阵列的内积。对于一维数组，返回向量的普通内积（没有复数的共轭），对于更高维度的数组，返回最后一个轴上的乘积之和。

必选输入参数

参数名	类型	说明
a	array_like	第一个实参。

版权所有 © 华为技术有限公司

参数名	类型	说明
b	array_like	第二个实参。

注意，如果a和b是非标量的，它们的最后一维的大小必须匹配，否则抛出异常。

可选输入参数

无。

返回数据

类型	说明
ndarray	返回a和b的内积。如果a和b都是标量或两个1D数组，则返回标量；否则返回数组。

示例

```
>>> import numpy as np
>>> a = np.array([1,2,3])
>>> b = np.array([0,1,2])
>>> np.inner(a, b)
8
>>>
>>> a = np.arange(24).reshape((2,3,4))
>>> a
array([[[ 0,  1,  2,  3],
        [ 4,  5,  6,  7],
        [ 8,  9, 10, 11]],

       [[12, 13, 14, 15],
        [16, 17, 18, 19],
        [20, 21, 22, 23]]])
>>> b = np.arange(4)
>>> b
array([0, 1, 2, 3])
>>> np.inner(a, b)
array([[ 14,  38,  62],
       [ 86, 110, 134]])
>>>
>>> a = np.arange(2).reshape((1,1,2))
>>> b = np.arange(6).reshape((3,2))
>>> np.inner(a, b)
array([[[1, 3, 5]]])
>>>
>>> a = np.eye(2)
>>> a
array([[1., 0.],
       [0., 1.]])
>>> np.inner(a, 5)
array([[5., 0.],
       [0., 5.]])
>>>
```

 版权所有 © 华为技术有限公司

6.6.2.3.3.4 matmul

功能描述

两个数组的矩阵乘积。

必选输入参数

参数名	类型	说明
x1	array_like	第一个实参。
x2	array_like	第二个实参。

注意，不允许输入标量，否则抛出异常。

可选输入参数

参数名	类型	默认值	说明
out	ndarray	None	存储结果的位置。如果提供，它必须具有与签名(n,k),(k,m)->(n,m)匹配的形状。如果没有提供或None，则返回新分配的数组。
**kwargs	-	-	其他关键字参数，见NumPy的ufunc相关文档。

返回数据

类型	说明
ndarray	输入的矩阵乘积。只有当x1、x2都是一维向量时，这才是一个标量。

示例

```
>>> import numpy as np
>>> a = np.array([1,2,3])
>>> b = np.array([0,1,2])
>>> np.inner(a, b)
8
>>>
>>> a = np.arange(24).reshape((2,3,4))
>>> a
array([[[ 0,  1,  2,  3],
        [ 4,  5,  6,  7],
        [ 8,  9, 10, 11]],

       [[12, 13, 14, 15],
        [16, 17, 18, 19],
```


版权所有 © 华为技术有限公司

```
       [20, 21, 22, 23]]])
>>> b = np.arange(4)
>>> b
array([0, 1, 2, 3])
>>> np.inner(a, b)
array([[ 14,  38,  62],
       [ 86, 110, 134]])
>>>
>>> a = np.arange(2).reshape((1,1,2))
>>> b = np.arange(6).reshape((3,2))
>>> np.inner(a, b)
array([[[1, 3, 5]]])
>>>
>>> a = np.eye(2)
>>> a
array([[1., 0.],
       [0., 1.]])
>>> np.inner(a, 5)
array([[5., 0.],
       [0., 5.]])
>>>
```

6.6.2.3.3.5 einsum

功能描述

评估操作数的爱因斯坦求和约定。

- 在隐式模式下，einsum使用爱因斯坦求和约定，许多常见的多维线性代数矩阵运算可以用简单的方式表示。
- 在显式模式下，einsum通过禁用或在指定的下标标签上强制求和，为计算其他可能不被视为经典爱因斯坦求和操作的数组操作提供了进一步的灵活性。

必选输入参数

参数名	类型	说明
subscript	str	将求和的下标指定为逗号分隔的下标标签列表。除非包含显式指标“->”以及精确输出形式的下标标签，否则将执行隐式（经典爱因斯坦求和）计算。
operands	list	操作数列表。

说明

不允许输入标量，否则抛出异常。

可选输入参数

参数名	类型	默认值	说明
out	ndarray	None	如果提供，将计算结果存储到这个数组中。

版权所有 © 华为技术有限公司

参数名	类型	默认值	说明
dtype	{data-type, None}	None	如果提供，则强制使用指定的数据类型计算。请注意，您可能还必须给出一个更自由的casting参数。
order	{‘C’，‘F’，‘A’，‘K’}	‘K’	控制输出的内存布局。 ● “C”表示是C连续的。 ● “F”表示是Fortran连续的。 ● “A”表示如果输入都是“F”，则应该是“F”；否则应该是“C”。 ● “K”表示它应该尽可能接近布局和输入，包括任意状态的轴。
casting	{‘no’，‘equiv’，‘safe’，‘same_kind’，‘unsafe’}	‘safe’	控制可能发生哪种数据转换。不建议将其设置为“unsafe”，可能会出现未定义的结果。 ● “no”表示根本不应该转换数据类型。 ● “equiv”表示只允许字节顺序更改。 ● “safe”表示只允许可以保存值的转换。 ● “same_kind”表示只允许安全转换或某种类型的转换，如float64到float32。 ● “unsafe”表示可以进行任何数据转换。
optimize	{False, True, ‘greedy’，‘optimal’}	False	控制是否应该进行中间优化。如果False和True默认为“贪婪”算法，则不会发生优化。还接受np.einsum_path函数的显式收缩列表。

返回数据

类型	说明
ndarray	基于爱因斯坦求和约定的计算结果。

示例

```
>>> import numpy as np
>>> a = np.arange(25).reshape((5,5))
```

 版权所有 © 华为技术有限公司

```
>>> a
array([[ 0,  1,  2,  3,  4],
       [ 5,  6,  7,  8,  9],
       [10, 11, 12, 13, 14],
       [15, 16, 17, 18, 19],
       [20, 21, 22, 23, 24]])
>>> b = np.arange(5)
>>> b
array([0, 1, 2, 3, 4])
>>> c = np.arange(6).reshape(2,3)
>>> c
array([[0, 1, 2],
       [3, 4, 5]])
>>>
>>> # 矩阵的迹(对角线之和)
>>> np.einsum('ii', a)
60
>>> np.einsum(a, [0,0])
60
>>> np.trace(a)
60
>>>
>>> # 矩阵对角线元素
>>> np.einsum('ii->i', a)
array([ 0,  6, 12, 18, 24])
>>> np.einsum(a, [0,0], [0])
array([ 0,  6, 12, 18, 24])
>>> np.diag(a)
array([ 0,  6, 12, 18, 24])
>>>
>>> # 矩阵按行求和
>>> np.einsum('ij->i', a)
array([ 10,  35,  60,  85, 110])
>>> np.einsum(a, [0,1], [0])
array([ 10,  35,  60,  85, 110])
>>> np.sum(a, axis = 1)
array([ 10,  35,  60,  85, 110])
>>> np.einsum(a, [Ellipsis,1], [Ellipsis])
array([ 10,  35,  60,  85, 110])
>>> np.einsum('...j->...', a)
array([ 10,  35,  60,  85, 110])
>>>
>>> c # 矩阵转置
array([[0, 1, 2],
       [3, 4, 5]])
>>> np.einsum('ji', c)
array([[0, 3],
       [1, 4],
       [2, 5]])
>>> np.einsum('ij->ji', c)
array([[0, 3],
       [1, 4],
       [2, 5]])
>>> np.einsum(c, [1,0])
array([[0, 3],
       [1, 4],
       [2, 5]])
>>>
>>> # 向量内积
>>> b
array([0, 1, 2, 3, 4])
>>> np.einsum('i,i', b, b)
30
>>> np.einsum(b, [0], b, [0])
30
>>> np.inner(b, b)
30
>>>
>>> # 向量和矩阵的乘积
```

 版权所有 © 华为技术有限公司

```
>>> np.einsum('ij,j', a, b)
array([ 30,  80, 130, 180, 230])
>>> np.einsum(a, [0,1], b, [1])
array([ 30,  80, 130, 180, 230])
>>> np.dot(a, b)
array([ 30,  80, 130, 180, 230])
>>> np.einsum('...j,j', a, b)
array([ 30,  80, 130, 180, 230])
>>>
>>> # 广播和标量乘法
>>> c
array([[0, 1, 2],
       [3, 4, 5]])
>>> np.einsum('...,...', 3, c)
array([[ 0,  3,  6],
       [ 9, 12, 15]])
>>> np.einsum(',ij', 3, c)
array([[ 0,  3,  6],
       [ 9, 12, 15]])
>>> np.einsum(3, [Ellipsis], c, [Ellipsis])
array([[ 0,  3,  6],
       [ 9, 12, 15]])
>>> np.multiply(3, c)
array([[ 0,  3,  6],
       [ 9, 12, 15]])
>>>
>>> # 向量外积
>>> t = np.arange(2) + 1
>>> t
array([1, 2])
>>> b
array([0, 1, 2, 3, 4])
>>>
>>> np.einsum('i,j', t, b)
array([[0, 1, 2, 3, 4],
       [0, 2, 4, 6, 8]])
>>> np.einsum(t, [0], b, [1])
array([[0, 1, 2, 3, 4],
       [0, 2, 4, 6, 8]])
>>> np.outer(t, b)
array([[0, 1, 2, 3, 4],
       [0, 2, 4, 6, 8]])
>>>
>>> # tensordot
>>> a = np.arange(60).reshape(3,4,5)
>>> b = np.arange(24).reshape(4,3,2)
>>> np.einsum('ijk,jil->kl', a, b)
array([[4400, 4730],
       [4532, 4874],
       [4664, 5018],
       [4796, 5162],
       [4928, 5306]])
>>> np.einsum(a, [0,1,2], b, [1,0,3], [2,3])
array([[4400, 4730],
       [4532, 4874],
       [4664, 5018],
       [4796, 5162],
       [4928, 5306]])
>>> np.tensordot(a, b, axes=([1,0],[0,1]))
array([[4400, 4730],
       [4532, 4874],
       [4664, 5018],
       [4796, 5162],
       [4928, 5306]])
>>>
```

6.6.2.3.4 线性代数函数

6.6.2.3.4.1 linalg.slogdet

功能描述

计算数组行列式的符号和绝对值的（自然）对数。

如果数组具有非常小或非常大的行列式，那么对det的调用可能会下溢或上溢。对于这些问题，这个函数更稳健，因为它计算行列式的对数，而不是行列式本身。

必选输入参数

参数名	类型	说明
a	array_like	输入数组，必须是正方形二维数组。

可选输入参数

无。

返回数据

参数名	类型	说明
sign	ndarray	代表行列式符号的数字。对于实矩阵，这是1、0或-1。对于复矩阵，这是一个绝对值为1的复数（即它在单位圆上），否则为0。
logdet	ndarray	行列式绝对值的自然对数。

如果行列式为零，则sign为0，logdet为-Inf。在所有情况下，行列式等于sign * np.exp(logdet)。

示例

```
>>> import numpy as np
>>> #二维数组[[a,b],[c,d]]，行列式结果 ad - bc
>>> a = np.array([[1,2],[3,4]])
>>> a
array([[1, 2],
       [3, 4]])
>>>
>>> (sign, logdet) = np.linalg.slogdet(a)
>>> (sign, logdet)
(-1.0, 0.6931471805599455)
>>>
>>> sign * np.exp(logdet)
-2.0000000000000004
>>>
>>>
>>> a = np.array([[[1,2],[3,4]], [[1,2],[2,1]], [[1,3],[3,1]]])
>>> a
array([[[1, 2],
        [3, 4]],
```


版权所有 © 华为技术有限公司

```
      [[1, 2],
       [2, 1]],

      [[1, 3],
       [3, 1]]])
>>>
>>> (sign, logdet) = np.linalg.slogdet(a)
>>> (sign, logdet)
(array([-1., -1., -1.]), array([0.69314718, 1.09861229, 2.07944154]))
>>> sign * np.exp(logdet)
array([-2., -3., -8.])
>>>
>>>
>>> np.linalg.det(np.eye(500) * 0.1)
0.0
>>> np.linalg.slogdet(np.eye(500) * 0.1)
(1.0, -1151.2925464970251)
>>>
```

6.6.2.3.4.2 linalg.inv

功能描述

计算矩阵的（乘法）逆。

给定一个正方形矩阵a，返回矩阵ainv满足dot(a, ainv) = dot(ainv, a) = eye(a.shape[0])。

必选输入参数

参数名	类型	说明
a	array_like	求逆的矩阵。

可选输入参数

无。

返回数据

类型	说明
ndarray/matrix	输出矩阵的逆。

示例

```
>>> import numpy as np
>>> a = np.array([[1.,2.], [3.,4.]])
>>> a
array([[1., 2.],
       [3., 4.]])
>>> ainv = np.linalg.inv(a)
>>> ainv
array([[-2. ,  1. ],
       [ 1.5, -0.5]])
>>>
>>> t = np.dot(a, ainv)
```


版权所有 © 华为技术有限公司

```
>>> t
array([[1.0000000e+00, 0.0000000e+00],
       [8.8817842e-16, 1.0000000e+00]])
>>> np.allclose(t, np.eye(2))
True
>>>
```

6.6.2.3.4.3 linalg.det

功能描述

计算数组的行列式。

必选输入参数

参数名	类型	说明
a	array_like	输入数组。

可选输入参数

无。

返回数据

类型	说明
ndarray	a的行列式。

示例

```
>>> import numpy as np
>>> # 二维数组[[a,b],[c,d]]的行列式结果 ad - bc
>>> a = np.array([[1,2],[3,4]])
>>> a
array([[1, 2],
       [3, 4]])
>>>
>>> np.linalg.det(a)
-2.0
>>>
>>> a = np.array([[[1,2],[3,4]], [[1,2],[2,1]], [[1,3],[3,1]]])
>>> a
array([[[1, 2],
        [3, 4]],

       [[1, 2],
        [2, 1]],

       [[1, 3],
        [3, 1]]])
>>>
>>> np.linalg.det(a)
array([-2., -3., -8.])
>>>
```


版权所有 © 华为技术有限公司

6.6.2.3.4.4 linalg.solve

功能描述

求解线性矩阵方程或线性标量方程组。

计算确定的“精确”解x，即满秩线性矩阵方程ax = b

必选输入参数

参数名	类型	说明
a	(…,M,M) array_like	系数矩阵。
b	{(…,M,),(…,M,K)}, array_like	纵坐标或“因变量”值。

可选输入参数

无。

返回数据

类型	说明
{(…, M,), (…, M, K)} ndarray	方程组ax = b的解，返回的大小与b相同。

示例

```
>>> import numpy as np
>>> # 解方程 x0 + 2*x1 = 1, 3*x0 + 5*x1 = 2
>>> a = np.array([[1,2],[3,5]])
>>> a
array([[1, 2],
       [3, 5]])
>>> b = np.array([1,2])
>>> b
array([1, 2])
>>>
>>> x = np.linalg.solve(a, b)
>>> x
array([-1.,  1.])
>>>
>>> np.allclose(np.dot(a,x), b)
True
>>>
```

6.6.2.3.4.5 tensordot

功能描述

沿着指定的轴计算张量点积。

 版权所有 © 华为技术有限公司

给定两个张量a和b，以及一个整数或包含两个array_like的对象（a_axes，b_axes），求a和b在a_axes和b_axes指定的轴上的乘积。

必选输入参数

参数名	类型	说明
a	array_like	第一个张量。
b	array_like	第二个张量。
axes	int or (2,) array_like	如果为整数，则按顺序，对a的最后N个轴和b的前N个轴求和。相应轴的尺寸必须匹配。 如果为两个array，则为要求和的轴列表，第一个序列适用于a，第二个序列适用于b。两个元素array_like必须具有相同的长度。元素array_like里的元素代表序列a、b要删除的轴。

可选输入参数

无。

返回数据

类型	说明
ndarray	输入张量的点积。

示例

```
>>> import numpy as np
>>> a = np.arange(60).reshape(3,4,5)
>>> b = np.arange(24).reshape(4,3,2)
>>> a
array([[[ 0,  1,  2,  3,  4],
        [ 5,  6,  7,  8,  9],
        [10, 11, 12, 13, 14],
        [15, 16, 17, 18, 19]],

       [[20, 21, 22, 23, 24],
        [25, 26, 27, 28, 29],
        [30, 31, 32, 33, 34],
        [35, 36, 37, 38, 39]],

       [[40, 41, 42, 43, 44],
        [45, 46, 47, 48, 49],
        [50, 51, 52, 53, 54],
        [55, 56, 57, 58, 59]]])
>>> b
array([[[ 0,  1],
        [ 2,  3],
        [ 4,  5]],

       [[ 6,  7],
```

 版权所有 © 华为技术有限公司

```
       [ 8,  9],
       [10, 11]],

      [[12, 13],
       [14, 15],
       [16, 17]],

      [[18, 19],
       [20, 21],
       [22, 23]]])
>>> c = np.tensordot(a, b, axes=([1,0],[0,1]))
>>> c
array([[4400, 4730],
       [4532, 4874],
       [4664, 5018],
       [4796, 5162],
       [4928, 5306]])
>>> c.shape
(5, 2)
>>>
>>> d = np.zeros((5,2))
>>> for i in range(5):
...     for j in range(2):
...         for k in range(3):
...             for n in range(4):
...                 d[i,j] += a[k,n,i] * b[n,k,j]
...
>>> c == d
array([[ True,  True],
       [ True,  True],
       [ True,  True],
       [ True,  True],
       [ True,  True]])
>>>
```

6.6.2.3.4.6 linalg.lstsq

功能描述

返回线性矩阵方程的最小二乘解。

该方程可能未确定、良好或过度确定（即a的线性独立行数可以小于、等于或大于其线性独立列数）。如果a是方阵且为满秩，则x（但对于舍入误差）是等式的“精确”解。否则，x最小化欧几里得2范数。如果有多个最小化解，则返回具有最小2范数的解。

必选输入参数

参数名	类型	说明
a	(M, N) array_like	系数矩阵。
b	{(M,), (M, K)} array_like	纵轴或因变量值。如果b是二维的，则为b的每个K列计算最小二乘解。

版权所有 © 华为技术有限公司

可选输入参数

参数名	类型	默认值	说明
rcond	float	None	*a* 的小奇异值的截止比率。为了确定秩，当奇异值小于 *rcond* 乘以 *a* 的最大奇异值时，将它们视为零。

返回数据

名称	类型	说明
x	{(N,),(N,K)} ndarray	最小二乘解。如果b是二维的，则解在x的K列中。
residuals	{(1,), (K,), (0,)} ndarray	平方残差之和：b中每列的平方欧几里得2范数-a @ x。如果a的秩是<N或M <= N，这是一个空数组。如果b是一维的，这是一个（1,）形状数组。否则形状是（K，）。
rank	int	矩阵a的秩。
s	(min(M, N),) ndarray	a的奇异值。

示例

```
>>> import numpy as np
>>> # y = mx + c
>>> x = np.array([0,1,2,3])
>>> y = np.array([-1,0.2,0.9,2.1])
>>>
>>> # 令 y = Ap, 其中 A = [[x1]], p = [[m], [c]]
>>> A = np.vstack([x, np.ones(len(x))]).T
>>> A
array([[0., 1.],
       [1., 1.],
       [2., 1.],
       [3., 1.]])
>>>
>>> m, c = np.linalg.lstsq(A, y, rcond=None)[0]
>>> m, c
(1.0000000000000002, -0.9499999999999996)
>>>
```

6.6.2.3.4.7 linalg.qr

功能描述

计算矩阵的QR分解。

将矩阵a因子分解为q和r矩阵，其中q是正交的，r是上三角形的。

版权所有 © 华为技术有限公司

必选输入参数

参数名	类型	说明
a	array_like, shape (…, M, N)	维度至少为2的类数组对象。

可选输入参数

参数名	类型	默认值	说明
mode	{‘reduced’, ‘complete’, ‘r’, ‘raw’}	‘reduced’	如果K=min（M,N），那么： ● “reduced”返回q，r，尺寸分别为(...,M,K)、(...,K,N)。 ● “complete”：返回q，r，尺寸分别为(...,M,M)、(...,M,N)。 ● “r”：仅返回尺寸为(...,K,N)的r。 ● “raw”：返回h，tau尺寸分别为(...,N,M)、(...,K,)。

返回数据

名称	类型	说明
q	ndarray of float or complex	具有正交列的矩阵。当模式=“完成”时，结果是一个正交/酉矩阵，这取决于a是否为实/复数。在这种情况下，行列式可以是+/- 1。如果输入数组中的维数大于2，则返回具有上述属性的一组矩阵。
r	ndarray of float or complex	如果输入数组中的维数大于2，上三角矩阵或一组上三角矩阵。
(h, tau)	ndarrays of np.double or np.cdouble	数组h包含与r一起生成q的豪斯霍尔德镜射算子。数组tau包含镜射算子的缩放因子。

示例

```
>>> import numpy as np
>>> a = np.random.randn(9, 6)
>>> q, r = np.linalg.qr(a)
>>>
>>> np.allclose(a, np.dot(q,r))
True
>>>
>>> r2 = np.linalg.qr(a, mode='r')
>>> np.allclose(r, r2)
```

 版权所有 © 华为技术有限公司

```
True
>>>
>>> a = np.random.normal(size=(3,2,2))
>>> q, r = np.linalg.qr(a)
>>> q.shape
(3, 2, 2)
>>> r.shape
(3, 2, 2)
>>> np.allclose(a, np.matmul(q,r))
True
>>>
>>> # Ax = b, 若A可分解为A = qr, 则 x = inv(r) * (q.T) * b
>>>
>>> A = np.array([[0,1], [1,1], [1,1], [2,1]])
>>> A
array([[0, 1],
       [1, 1],
       [1, 1],
       [2, 1]])
>>> b = np.array([1, 0, 2, 1])
>>> q, r = np.linalg.qr(A)
>>> p = np.dot(q.T, b)
>>>
>>> np.dot(np.linalg.inv(r), p)
array([2.62287285e-16, 1.00000000e+00])
>>>
```

6.6.2.3.4.8 linalg.svd

功能描述

奇异值分解。

当a是二维数组，并且full_matrices=False时，它被分解为u @ np.diag(s) @ vh = (u * s) @ vh，其中u和vh的埃尔米特转置是具有正交列的二维数组，s是a奇异值的1维数组。当a是更高维度时，SVD（奇异值分解）以堆叠模式应用。

必选输入参数

参数名	类型	说明
a	(…,M,N) array_like	具有a.ndim >= 2的实数组或复数组。
b	array_like	-

可选输入参数

参数名	类型	默认值	说明
full_matrices	bool	True	如果是默认值True，u和vh分别具有形状(...,M,M)和(...,N,N)。否则，形状分别是(...,M,K)和(...,K,N)，其中K=min(M,N)。
compute_uv	bool	True	除了s之外，是否计算u和vh。

版权所有 © 华为技术有限公司

返回数据

名称	类型	说明
u	{(…,M,M), (…,M,K) } array	酉矩阵。前a.ndim-2维度与输入a的尺寸相同，最后两个维度的大小取决于full_matrices的值。仅在compute_uv为True时返回。
s	(…, K) array	具有奇异值的向量，在每个向量中按降序排序。前a.ndim-2维度与输入a的尺寸相同。
vh	{(…,N,N), (…,K,N)} array	酉矩阵。前a.ndim-2维度与输入a的尺寸相同，最后两个维度的大小取决于full_matrices的值。仅在compute_uv为True时返回。

示例

```
>>> import numpy as np
>>> a = np.random.randn(9, 6) + 1j * np.random.randn(9, 6)
>>> b = np.random.randn(2, 7, 8, 3) + 1j * np.random.randn(2, 7, 8, 3)
>>>
>>> u, s, vh = np.linalg.svd(a, full_matrices=True)
>>> u.shape, s.shape, vh.shape
((9, 9), (6,), (6, 6))
>>> np.allclose(a, np.dot(u[:, :6] * s, vh))
True
>>>
>>> smat = np.zeros((9,6), dtype=complex)
>>> smat[:6, :6] = np.diag(s)
>>> np.allclose(a, np.dot(u, np.dot(smat, vh)))
True
>>>
```

6.6.2.3.4.9 linalg.eigh

功能描述

返回复埃尔米特（共轭对称）或实对称矩阵的特征值和特征向量。

返回两个对象，一个包含a特征值的1维数组，以及相应特征向量（以列为单位）的二维正方形数组或矩阵（取决于输入类型）。

必选输入参数

参数名	类型	说明
a	(…, M, M) array	埃尔米特矩阵或实对称矩阵。

版权所有 © 华为技术有限公司

可选输入参数

参数名	类型	默认值	说明
UPLO	{‘L’, ‘U’},	‘L’	指定计算是用a的下三角部分（“L”）还是上三角部分（“U”）完成。无论这个值如何，在计算中只考虑对角线的实际部分，以保留埃尔米特矩阵的概念。因此，对角线的虚部将始终被视为零。

返回数据

名称	类型	说明
w	(…, M) ndarray	特征值按升序排列，每个特征值根据其多重性重复。
v	{(…, M, M) ndarray, (…, M, M) matrix}	列v[:,i]是与特征值w[i]相对应的归一化特征向量。如果a是矩阵对象，将返回矩阵对象。

示例

```
>>> import numpy as np
>>> # 向量输入
>>> a = np.array([[1,-2j], [2j, 5]])
>>> a
array([[ 1.+0.j, -0.-2.j],
       [ 0.+2.j,  5.+0.j]])
>>> w, v = np.linalg.eigh(a)
>>> w, v
(array([0.17157288, 5.82842712]), array([[-0.92387953-0.j        , -0.38268343+0.j        ],
       [ 0.        +0.38268343j,  0.        -0.92387953j]]))
>>>
>>> np.dot(a, v[:, 0]) - w[0] * v[:, 0]
array([5.55111512e-17+0.0000000e+00j, 0.00000000e+00+1.2490009e-16j])
>>> np.dot(a, v[:, 1]) - w[1] * v[:, 1]
array([0.+0.j, 0.+0.j])
>>>
>>> # 矩阵输入
>>> A = np.matrix(a)
>>> A
matrix([[ 1.+0.j, -0.-2.j],
        [ 0.+2.j,  5.+0.j]])
>>> w, v = np.linalg.eigh(A)
>>> w; v
array([0.17157288, 5.82842712])
matrix([[-0.92387953-0.j        , -0.38268343+0.j        ],
        [ 0.        +0.38268343j,  0.        -0.92387953j]])
>>>
>>> # 对角线虚部处理
>>> a = np.array([[5+2j, 9-2j], [0+2j, 2-1j]])
>>> a
array([[5.+2.j, 9.-2.j],
```

 版权所有 © 华为技术有限公司

```
       [0.+2.j, 2.-1.j]])
>>> b = np.array([[5.+0.j, 0.-2.j], [0.+2.j, 2.-0.j]])
>>> b
array([[5.+0.j, 0.-2.j],
       [0.+2.j, 2.+0.j]])
>>>
>>> np.linalg.eigh(a)
(array([1., 6.]), array([[-0.4472136 -0.j         , -0.89442719+0.j         ],
       [ 0.         +0.89442719j,  0.         -0.4472136j ]]))
>>>
>>> np.linalg.eigh(b)
(array([1., 6.]), array([[-0.4472136 -0.j         , -0.89442719+0.j         ],
       [ 0.         +0.89442719j,  0.         -0.4472136j ]]))
>>>
```

6.6.2.3.4.10 linalg.eigvalsh

功能描述

计算复埃尔米特矩阵或实对称矩阵的特征值。

与eigh的主要区别是：特征向量没有计算。

必选输入参数

参数名	类型	说明
a	(…, M, M) array	埃尔米特矩阵或实对称矩阵。

可选输入参数

参数名	类型	默认值	说明
UPLO	{‘L’, ‘U’},	‘L’	指定计算是用a的下三角部分（“L”）还是上三角部分（“U”）完成。无论这个值如何，在计算中只考虑对角线的实部，以保留埃尔米特矩阵的概念。因此，对角线的虚部将始终被视为零。

返回数据

名称	类型	说明
w	(…, M) ndarray	特征值按升序排列，每个特征值根据其多重性重复。

示例

```
>>> import numpy as np
>>> a = np.array([[1, -2j], [2j, 5]])
```

 版权所有 © 华为技术有限公司

```
>>> np.linalg.eigvalsh(a)
array([0.17157288, 5.82842712])
>>>
>>> a = np.array([[5+2j, 9-2j], [0+2j, 2-1j]])
>>> a
array([[5.+2.j, 9.-2.j],
       [0.+2.j, 2.-1.j]])
>>> b = np.array([[5.+0.j, 0.-2.j], [0.+2.j, 2.-0.j]])
>>> b
array([[5.+0.j, 0.-2.j],
       [0.+2.j, 2.+0.j]])
>>>
>>> wa = np.linalg.eigvalsh(a)
>>> wa
array([1., 6.])
>>> wb = np.linalg.eigvalsh(b)
>>> wb
array([1., 6.])
>>> wb = np.linalg.eigvals(b)
>>> wb
array([6.+0.j, 1.+0.j])
>>>
```

6.6.2.3.4.11 linalg.cholesky

功能描述

返回方阵a的Cholesky分解，L * L.H，其中L是下三角矩阵。.H是共轭转置算子（如果a是实矩阵，则是普通转置）。a必须是埃尔米特矩阵（如果是实矩阵，则为对称）和正定矩阵。没有进行检查来验证a是否是埃尔米特矩阵。此外，只使用a的下三角和对角元素。实际上只有L被返回。

必选输入参数

参数名	类型	说明
a	(…,M,M) array_like	埃尔米特（如果所有元素都是实数，则为对称）、正定输入矩阵。

可选输入参数

无。

返回数据

类型	说明
(…, M, M) array_like	a的下三角Cholesky因子。如果a是矩阵对象，则返回矩阵对象。

示例

```
>>> import numpy as np
>>> A = np.array([[1,-2j], [2j,5]])
>>> A
array([[ 1.+0.j, -0.-2.j],
```

 版权所有 © 华为技术有限公司

```
       [ 0.+2.j,  5.+0.j]])
>>> L = np.linalg.cholesky(A)
>>> L
array([[1.+0.j, 0.+0.j],
       [0.+2.j, 1.+0.j]])
>>>
>>> np.allclose(A, np.dot(L, L.T.conj()))
True
>>>
>>> A = [[1,-2j], [2j,5]]
>>> L = np.linalg.cholesky(A)
>>> L
array([[1.+0.j, 0.+0.j],
       [0.+2.j, 1.+0.j]])
>>>
>>> np.linalg.cholesky(np.matrix(A))
matrix([[1.+0.j, 0.+0.j],
        [0.+2.j, 1.+0.j]])
>>>
```

6.6.2.3.4.12 linalg.matrix_power

功能描述

对于正整数n，幂通过重复矩阵平方和矩阵乘法计算。

- 如果n == 0，则返回与M形状相同的单位矩阵。
- 如果n < 0，则计算逆矩阵，然后提高到abs(n)。

必选输入参数

参数名	类型	说明
a	(…,M,M) array_like	需要做幂运算的矩阵。
n	int	指数可以是任何整数或长整数、正数、负数或零。

可选输入参数

无。

返回数据

类型	说明
(…, M, M) ndarray or matrix object	返回值与M的形状和类型相同。 • 如果指数为正或为零，则元素的类型与M的类型相同。 • 如果指数为负数，则元素为浮点。

示例

```
>>> import numpy as np
>>> a = np.array([[0,1], [-1,0]])
```

 版权所有 © 华为技术有限公司

```
>>> np.linalg.matrix_power(a, 3)
array([[ 0, -1],
       [ 1,  0]])
>>>
>>> np.linalg.matrix_power(a, 0)
array([[1, 0],
       [0, 1]])
>>>
>>> np.linalg.matrix_power(a, -3)
array([[ 0.,  1.],
       [-1.,  0.]])
>>>
>>> q = np.zeros((4,4))
>>> q[0:2, 0:2] = -a
>>> q[2:4, 2:4] = a
>>> a
array([[ 0,  1],
       [-1,  0]])
>>>
>>> np.linalg.matrix_power(q, 2)
array([[-1.,  0.,  0.,  0.],
       [ 0., -1.,  0.,  0.],
       [ 0.,  0., -1.,  0.],
       [ 0.,  0.,  0., -1.]])
>>>
```

6.6.2.3.4.13 linalg.tensorsolve

功能描述

为x求解张量方程a x = b。

假设x的所有索引都与a的最右边的索引一起求和，例如tensordot(a，x，axes=x.ndim)中所做的那样。

必选输入参数

参数名	类型	说明
a	array_like	系数张量，形状为b.shape+Q。 Q，元组，等于由其最右边索引的适当数量组成的该子张量的形状，并且必须使prod(Q)== prod(b.shape)（在某种意义上说是“正方形”）。
b	array_like	右手张量，可以是任何形状。

可选输入参数

参数名	类型	默认值	说明
axes	tuple of ints	None	在求逆之前，在a中向右重新排序的轴。如果值为None，则不会进行重新排序。

 版权所有 © 华为技术有限公司

返回数据

类型	说明
ndarray, shape Q	张量方程ax = b的解。

示例

```
>>> import numpy as np
>>> a = np.eye(2*3*4)
>>> a.shape = (2*3, 4, 2, 3, 4)
>>> b = np.random.randn(2*3, 4)
>>> x = np.linalg.tensorsolve(a, b)
>>> x.shape
(2, 3, 4)
>>>
>>> np.allclose(np.tensordot(a, x, axes=3), b)
True
>>>
```

6.6.2.3.4.14 linalg.norm

功能描述

矩阵或向量范数。

该函数能够返回八个不同的矩阵范数之一，或无限数量的向量范数之一（如表6-71所述），具体取决于参数ord的值。

表 6-71 参数 ord 对应的范数

ord	矩阵范数	向量范数
None	Frobenius范数	2-范数
‘fro’	Frobenius范数	-
‘nuc’	核范数	-
inf	max(sum(abs(x), axis=1))	max(abs(x))
-inf	min(sum(abs(x), axis=1))	min(abs(x))
0	-	sum(x != 0)
1	max(sum(abs(x), axis=0))	sum(abs(x)**ord)**(1./ord)
-1	min(sum(abs(x), axis=0))	sum(abs(x)**ord)**(1./ord)
2	2-范数（奇异值的最大值）	sum(abs(x)**ord)**(1./ord)

版权所有 © 华为技术有限公司

ord	矩阵范数	向量范数
-2	奇异值的最小值	sum(abs(x)**ord)**(1./ord)
其他	-	sum(abs(x)**ord)**(1./ord)

必选输入参数

参数名	类型	说明
x	array_like	输入数组。 ● 如果轴为None，x必须是1-D或2-D，除非ord为None。 ● 如果轴和ord均为None，则将返回x.ravel的2范数。

可选输入参数

参数名	类型	默认值	说明
ord	{non-zero int, inf, -inf, ‘fro’, ‘nuc’}	None	请参见表6-71。
axis	{None, int, 2-tuple of ints}	None	● 如果轴是一个整数，它指定了计算向量范数的x轴。 ● 如果轴是2元组，它指定容纳二维矩阵的轴，并计算这些矩阵的矩阵范数。 ● 如果轴为None，则返回向量范数（当x为1-D时）或矩阵范数（当x为2-D时）。
keepdims	bool	False	如果将其设置为True，则在结果中将作为尺寸为1的维度留在结果中。使用此选项，结果将正确地针对原始x广播。

返回数据

类型	说明
float or ndarray	矩阵/向量范数。

 版权所有 © 华为技术有限公司

示例

```
>>> import numpy as np
>>> a = np.arange(9) - 4
>>> a
array([-4, -3, -2, -1,  0,  1,  2,  3,  4])
>>> b = a.reshape((3,3))
>>> b
array([[-4, -3, -2],
       [-1,  0,  1],
       [ 2,  3,  4]])
>>>
>>> np.linalg.norm(a)
7.745966692414834
>>> np.linalg.norm(b)
7.745966692414834
>>> np.linalg.norm(b, 'fro')
7.745966692414834
>>> np.linalg.norm(a, np.inf)
4.0
>>> np.linalg.norm(b, np.inf)
9.0
>>> np.linalg.norm(a, -np.inf)
0.0
>>> np.linalg.norm(b, -np.inf)
2.0
>>>
>>> np.linalg.norm(a, 1)
20.0
>>> np.linalg.norm(b, 1)
7.0
>>> np.linalg.norm(b, -1)
6.0
>>> np.linalg.norm(a, 2)
7.745966692414834
>>> np.linalg.norm(b, 2)
7.348469228349533
>>> np.linalg.norm(a, -2)
0.0
>>> np.linalg.norm(b, -2)
3.757577037301703e-16
>>>
>>> c = np.array([[1,2,3], [-1,1,4]])
>>> np.linalg.norm(c, axis=0)
array([1.41421356, 2.23606798, 5.        ])
>>> np.linalg.norm(c, axis=1)
array([3.74165739, 4.24264069])
>>> np.linalg.norm(c, ord=1, axis=1)
array([6., 6.])
>>>
>>>
>>> m = np.arange(8).reshape(2,2,2)
>>> m
array([[[0, 1],
        [2, 3]],

       [[4, 5],
        [6, 7]]])
>>> np.linalg.norm(m, axis=(1,2))
array([ 3.74165739, 11.22497216])
>>> np.linalg.norm(m[0, :, :])
3.7416573867739413
>>> np.linalg.norm(m[1, :, :])
11.224972160321824
>>>
```

 版权所有 © 华为技术有限公司

6.6.2.3.4.15 linalg.cond

功能描述

计算矩阵的条件数。

该函数能够根据p的值，使用七个不同范数之一返回条件数。范数取值请参见表6-71。

必选输入参数

参数名	类型	说明
x	(…,M,N) array_like	要求条件数的矩阵。

可选输入参数

参数名	类型	默认值	说明
p	{None, 1, -1, 2, -2, inf, -inf, ‘fro’}	None	用矩阵范数计算条件数。

返回数据

类型	说明
ndarray	矩阵的条件数。

示例

```
>>> import numpy as np
>>> a = np.array([[1,0,-1], [0,1,0], [1,0,1]])
>>> a
array([[ 1,  0, -1],
       [ 0,  1,  0],
       [ 1,  0,  1]])
>>> np.linalg.cond(a)
1.4142135623730951
>>> np.linalg.cond(a, 'fro')
3.1622776601683795
>>> np.linalg.cond(a, np.inf)
2.0
>>> np.linalg.cond(a, -np.inf)
1.0
>>> np.linalg.cond(a, 1)
2.0
>>> np.linalg.cond(a, -1)
1.0
>>> np.linalg.cond(a, 2)
1.4142135623730951
>>> np.linalg.cond(a, -2)
0.7071067811865475
>>> min(np.linalg.svd(a, compute_uv=False))*min(np.linalg.svd(np.linalg.inv(a), compute_uv=False))
```

 版权所有 © 华为技术有限公司

```
0.7071067811865475
>>>
```

6.6.2.3.4.16 linalg.tensorinv

功能描述

结果是相对于张量点积运算tensordot(a, b, ind)的逆，即在浮点精度下，tensordot(tensorinv(a), a, ind)是张量运算的“单位”张量。

必选输入参数

参数名	类型	说明
a	array_like	“求逆”的张量。它的形状必须是“正方形”，即prod(a.shape[:ind]) == prod(a.shape[ind:])。

可选输入参数

参数名	类型	默认值	说明
ind	int	2	反向和中涉及的第一个索引的数量。必须是正整数。

返回数据

类型	说明
ndarray	a的张量点积的逆，形状为a.shape[ind:] + a.shape[:ind]。

示例

```
>>> import numpy as np
>>> a = np.eye(4*6)
>>> a.shape = (4, 6, 8, 3)
>>>
>>> ainv = np.linalg.tensorinv(a, ind=2)
>>> ainv.shape
(8, 3, 4, 6)
>>>
>>> b = np.random.randn(4, 6)
>>> np.allclose(np.tensordot(ainv,b), np.linalg.tensorsolve(a,b))
True
>>>
>>> a = np.eye(4*6)
>>> a.shape = (24, 8, 3)
>>> ainv = np.linalg.tensorinv(a, ind=1)
>>> ainv.shape
(8, 3, 24)
>>>
>>> b = np.random.randn(24)
```

 版权所有 © 华为技术有限公司

```
>>> np.allclose(np.tensordot(ainv,b,1), np.linalg.tensorsolve(a,b))
True
>>>
```

6.6.2.3.5 版本查询函数

获取KML_NUMPY产品版本信息。

接口定义

NumpyResult NumpyGetProductVersion(NumpyProVersion *packageInfo);

参数

参数名	描述	取值范围	输入/输出
packageInfo	产品信息结构体。	非空	输出

返回值

- 成功：返回NUMPY_STS_NO_ERR。
- 失败：返回NUMPY_STS_NULL_PTR_ERR。

示例

```
#include <stdio.h>
#include "knpy.h"

int main()
{
    NumpyProVersion packageInfo;
    NumpyResult ret = NumpyGetProductVersion(&packageInfo);
    if (ret == NUMPY_STS_NO_ERR) {
        printf("\nNumpy\n");
        printf("Product Name:         %s\n", packageInfo.productName);
        printf("Product Version:      %s\n", packageInfo.productVersion);
        printf("Component Name:       %s\n", packageInfo.componentName);
        printf("Component Version:    %s\n", packageInfo.componentVersion);
        printf("Component AbiVersion: %s\n", packageInfo.componentAbiVersion);
        printf("Component AppendInfo: %s\n", packageInfo.componentAppendInfo);
        printf("Software Name:        %s\n", packageInfo.softwareName);
        printf("Software Version:     %s\n", packageInfo.softwareVersion);
    }
    return 0;
}
```

运行结果：

```
Numpy
Product Name:         Kunpeng HPCKit
Product Version:      24.0.RC1
Component Name:       HPCKit-kml
Component Version:    2.3.0
Component AbiVersion: 1
Component AppendInfo: gcc
Software Name:        hpckit-kml-numpy
Software Version:     2.3.0
```

 版权所有 © 华为技术有限公司

说明

版本号和编译时间以实际运行结果为准，上述结果仅供参考。

6.6.3 KML_RT 库说明

6.6.3.1 KML_RT 库的作用

KML_RT是鲲鹏数学库（KML）的一个运行时动态库，该库包含了运行时所需的函数，使得应用程序能够直接链接KML_RT库以实现对KML中数学运算函数的调用而无需链接对应的数学库，可避免用户在编译应用程序时需要链接多个KML数学库的繁琐操作，以提升应用程序开发和编译效率。同时KML_RT提供了程序运行时识别硬件环境的能力，以实现加载不同鲲鹏CPU平台对应的KML数学库，减少用户需要根据硬件环境设置不同环境变量的操作。

6.6.3.2 KML_RT 包含的动态库

KML_RT库提供了如下鲲鹏数学库对应动态库中的所有函数接口。

数学库名称	动态库名称
KML_BLAS	libkblas.so
KML_SPBLAS	libkspblas.so
KML_FFT	libkfft.so libkfftf.so libkffth.so libkfft_mpi.so libkfftf_mpi.so libkffth_mpi.so
KML_CONV	libkconv.so
KML_VSL	libkvsl.so
KML_EIGENSOLVER	libkes.so
KML_SOLVER	libksolver.so libkscasolver.so
KML_LAPACK	libklapack.so
KML_SCALAPACK	libkscalapack.so
KML_IPL	libkipl.so libkipl_omp.so

6.6.3.3 使用说明

 版权所有 © 华为技术有限公司

6.6.3.3.1 示例

应用程序需要使用KML_RT运行时动态库前，需要保证libkml_rt.so动态库的位置正确，正确应该为HPCKit或鲲鹏数学库rpm包安装后kml动态库所在的根目录，如果不在该目录下，将无法正常加载和使用。

以KML_BLAS level3中函数sgemm为例，应用程序编写和头文件的引用方式不变，在编译时使用KML_RT动态库即可。

依赖

#include "kblas.h"

示例

main.c：

```
int m = 4, k = 3, n = 4, lda = 4, ldb = 3, ldc = 4;
float alpha = 1.0, beta = 2.0;
 /*
 * A:
 *     0.340188,       0.411647,       -0.222225,
 *     -0.105617,      -0.302449,      0.053970,
 *     0.283099,       -0.164777,      -0.022603,
 *     0.298440,       0.268230,       0.128871,
 * B:
 *     -0.135216,      0.416195,       -0.358397,      -0.257113,
 *     0.013401,       0.135712,       0.106969,       -0.362768,
 *     0.452230,       0.217297,       -0.483699,      0.304177,
 * C:
 *     -0.343321,      0.498924,       0.112640,       -0.006417,
 *     -0.099056,      -0.281743,      -0.203968,      0.472775,
 *     -0.370210,      0.012932,       0.137552,       -0.207483,
 *     -0.391191,      0.339112,       0.024287,       0.271358,
 */
float a[12] = {0.340188, -0.105617, 0.283099,
               0.298440, 0.411647, -0.302449,
               -0.164777, 0.268230, -0.222225,
               0.053970, -0.022603, 0.128871};
float b[12] = {-0.135216, 0.013401, 0.452230, 0.416195,
               0.135712, 0.217297, -0.358397, 0.106969,
               -0.483699, -0.257113, -0.362768, 0.304177};
float c[16] = {-0.343321, -0.099056, -0.370210, -0.391191,
               0.498924, -0.281743, 0.012932, 0.339112,
               0.112640, -0.203968, 0.137552, 0.024287,
               -0.006417, 0.472775, -0.207483, 0.271358};

cblas_sgemm(CblasColMajor,CblasNoTrans,CblasNoTrans, m, n, k, alpha, a, lda, b, ldb, beta, c, ldc);
/*
 * Output C:
 *     -0.827621       1.147010        0.254881        -0.317229
 *     -0.163476       -0.636762       -0.428542       1.098841
 *     -0.791128       0.116416        0.166949        -0.434854
 *     -0.760862       0.866839        -0.092028       0.407877
 *
 */
```

编译

```
gcc main.c -o main -I /usr/local/kml/include -L /usr/local/kml/lib -lkml_rt
```

6.6.3.3.2 环境变量说明

- KML_BLAS_TYPE（可选）

 版权所有 © 华为技术有限公司

KML_BLAS具有nolocking、locking和multi三种版本，链接kml_rt运行时动态库默认使用multi版本的函数。因此如果需要指定使用的KML_BLAS版本，则需要通过环境变量“KML_BLAS_TYPE”来控制。例如要使用nolocking版本的KML_BLAS，则需要添加如下环境变量：

```
export KML_BLAS_TYPE=nolocking
```

- KML_SPBLAS_TYPE（可选）

  KML_SPBLAS具有single和multi两种版本，链接kml_rt运行时动态库默认使用single版本的函数，因此如果需要指定使用的KML_SPBLAS版本，则需要通过环境变量“KML_SPBLAS_TYPE”来控制。例如要使用multi版本的KML_SPBLAS，则需要添加如下环境变量：

  ```
  export KML_SPBLAS_TYPE=multi
  ```

- KML_FFT_TYPE（可选）

  KML_FFT具有常规和mpi两种版本，链接kml_rt运行时动态库默认使用常规版本的函数，因此如果需要指定使用的KML_FFT版本，则需要通过环境变量“KML_FFT_TYPE”来控制。例如要使用mpi版本的KML_FFT，则需要添加如下环境变量：

  ```
  export KML_FFT_TYPE=mpi
  ```

- KML_IPL_TYPE（可选）

  KML_IPL具有非omp和omp两种版本，链接kml_rt运行时动态库默认使用非omp版本的函数，因此如果需要指定使用的KML_IPL版本，则需要通过环境变量“KML_IPL_TYPE”来控制。例如要使用omp版本的KML_IPL，则需要添加如下环境变量：

  ```
  export KML_IPL_TYPE=omp
  ```

6.6.3.3.3 注意事项

- KML_LAPACK及KML_SCALAPACK的使用

  通过KML_RT使用KML_LAPACK及KML_SCALAPACK的（包含所有开源lapack和scalpack）函数功能时，需要保证kml数学库安装路径的目录下（假设kml安装路径为kml_install_path，则对应目录分别是：“${kml_install_path}/lib/neon”、“${kml_install_path}/lib/sve”、“${kml_install_path}/lib/sve512”）包含libklapack.so和libkscalapack.so两个动态库，否则相关功能将无法使用。通过HPCKit的方式安装会默认包含libklapack.so和libkscalapack.so两个动态库。除此之外，使用KML_LAPACK及KML_SCALAPACK函数功能需要指定omp版本的KML_BLAS，即将环境变量KML_BLAS_TYPE设置为omp。

- KML_EIGENSIOLVER的使用

  通过KML_RT使用KML_EIGENSIOLVER的函数功能时，由于其依赖KML_LAPACK及KML_SCALAPACK的函数功能，需要保证kml数学库安装路径的目录下（假设kml安装路径为kml_install_path，则对应目录分别是：“${kml_install_path}/lib/neon”、“${kml_install_path}/lib/sve”、“${kml_install_path}/lib/sve512”）包含libklapack.so和libkscalapack.so两个动态库，否则相关功能将无法使用。通过HPCKit的方式安装会默认包含libklapack.so和libkscalapack.so两个动态库。除此之外，使用KML_LAPACK及KML_SCALAPACK函数功能需要指定omp版本的KML_BLAS，即将环境变量KML_BLAS_TYPE设置为omp。

- KML_IPL的使用

  通过KML_RT使用KML_IPL的函数功能时，由于其依赖KML_LAPACK的函数功能，需要保证kml数学库安装路径的目录下（假设kml安装路径为kml_install_path，则对应目录分别是：“${kml_install_path}/lib/neon”、“${kml_install_path}/lib/sve”、“${kml_install_path}/lib/sve512”）包含libklapack.so动态库，否则相关功能将无法使用。通过HPCKit的方式安装会默认

 版权所有 © 华为技术有限公司

包含libklapack.so动态库。除此之外，使用KML_LAPACK函数功能需要指定omp版本的KML_BLAS，即将环境变量KML_BLAS_TYPE设置为omp。

 版权所有 © 华为技术有限公司

7 Hyper IO 库开发指南

产品描述
使用指南
Hyper IO测试
接口支持列表
术语和缩略语

7.1 产品描述

7.1.1 软件架构

Hyper IO软件架构如**图1**所示。

 版权所有 © 华为技术有限公司

图 7-1 Hyper IO 软件架构图

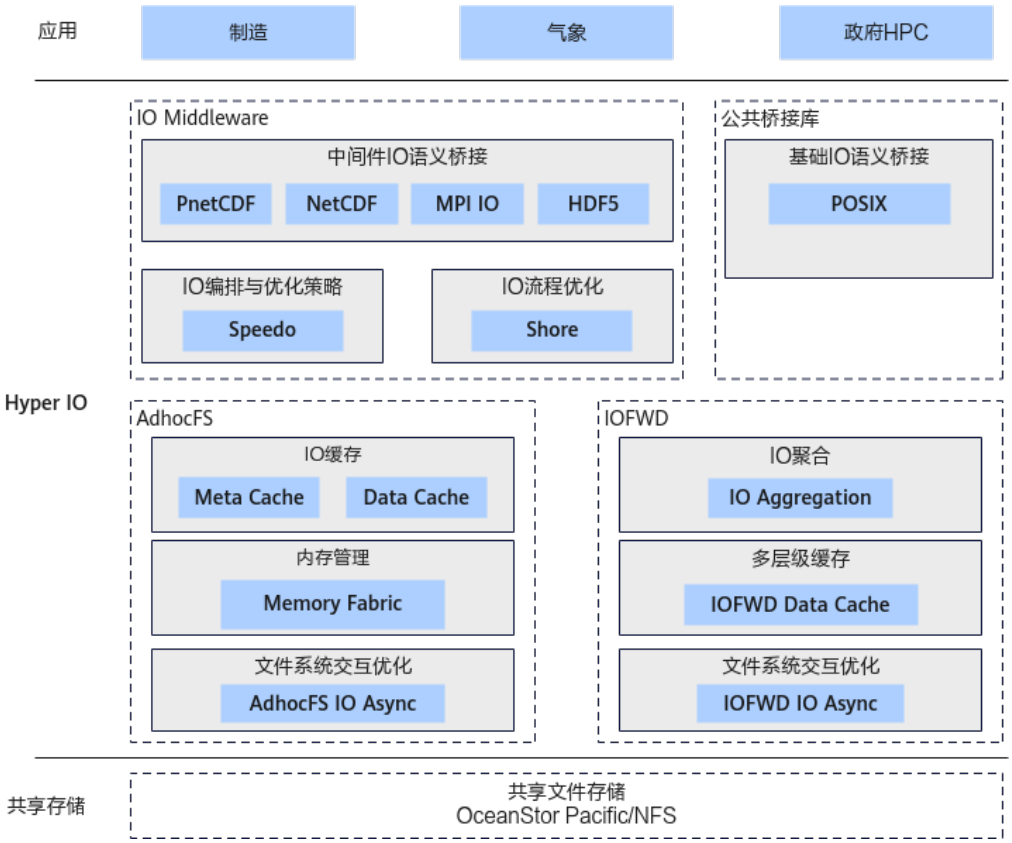


表 7-1 Hyper IO 软件架构中各模块说明

模块名称	用途
PnetCDF桥接器	负责PnetCDF标准接口桥接，用于IO编排、策略优化和IO执行优化。
NetCDF桥接器	负责NetCDF标准接口桥接，用于IO编排、策略优化和IO执行优化。
MPI-IO桥接器	负责MPI-IO标准接口桥接，用于IO编排、策略优化和IO执行优化。
HDF5桥接器	负责HDF5标准接口桥接，用于IO编排、策略优化和IO执行优化。
Speedo	通过配置信息和流程管理完成IO编排和执行策略的制定。
Shore	负责IO流程优化实施，包含序列化、聚集、通信和运输。
POSIX桥接器	负责POSIX标准接口桥接，用于AdhocFS或者IO转发执行优化。
Meta Cache	AdhocFS文件访问元数据缓存。
Data Cache	AdhocFS文件数据缓存。
Memory Fabric	负责节点内多进程间共享内存管理。

版权所有 © 华为技术有限公司

模块名称	用途
AdhocFS IO Async	负责AdhocFS和文件系统交互的IO异步优化。
IO Aggregation	应用计算过程IO聚合。
IOFWD Data Cache	转发过程多层级数据缓存计算过程数据。
IOFWD IO Async	负责IO转发和文件系统交互的IO异步优化。
OceanStor Pacific/NFS	高性能计算解决方案中使用的共享文件存储系统，包含通用的并行文件系统和网络文件系统。

7.1.2 关键特性

Hyper IO具有以下关键特性：

- 理解多场景应用数据的物理意义，在多个维度上进行数据集聚合、拼接、重组，缩减应用元数据量和减少IO访问次数。
- 识别高性能计算典型应用的数据类型信息、数据统计信息，针对不同数据类型（包括浮点、整型类型、稀疏矩阵）引入差异化的数据压缩技术，减少网络传输带宽，提升数据交互效率和稳定性。
- 通过拆分应用数据处理步骤、解耦不同阶段处理单元间的算法和基于数据属性的灵活处理单元调用，完全实现数据驱动。
- 单一数据对象任意数量、任意种类和任意顺序的预处理和后处理；多个数据对象任意形式、任意维度的聚合、拆解和重整操作。
- 感知应用的数据集信息，经过分析、优化决策，并执行上述相关的优化策略。

当前版本的Hyper IO提供针对NetCDF、PnetCDF、HDF5和MPI-IO这几种IO库的数据IO桥接和加速能力，后续版本中，Hyper IO会针对POSIX IO语义提供加速能力。后续章节中，会对Hyper IO支持的IO库相关的接口进行详细的描述。

7.1.3 产品介绍

概念介绍

Hyper IO是一类提供系统软件和应用软件之间连接的软件，提供科学数据的存储描述格式定义和文件操作，在高性能计算应用以及大量后处理、可视化及分析框架中广泛应用。

价值

为了满足高性能计算对应用可扩展性和端到端性能提升的诉求，在华为自研高性能计算解决方案中加入自研Hyper IO，此中间件面向鲲鹏平台优化，兼容多类型开源中间件接口。通过优化进程间IO范式、Step间IO范式将应用进行抽象和分类，并通过数据块聚合、重组、缓存等方式将应用的IO操作重新组合成对后端文件系统更为友好的操作，消除大规模应用的IO瓶颈，从而提升鲲鹏平台上高性能计算应用运行时的性能。

版权所有 © 华为技术有限公司

功能

Hyper IO可提升大型IO密集型高性能计算应用的可扩展性和端到端性能，减少并行文件系统故障，解决IO瓶颈和实践中应用直接针对特定存储进行优化不可持续的问题。

须知

- 请仔细梳理应用中使用的IO库接口，如果存在Hyper IO不支持的IO库接口，使能Hyper IO可能会导致未定义行为，此时请谨慎使用Hyper IO的加速功能。
- 由于开源中间件的底层原生接口缺乏线程安全机制，经中间件拦截与封装后的接口同样不具备线程安全性。
- 调用桥接函数传入的入参合法性需要用户自行保证，所有函数接口行为均与原接口保持一致。
- 环境变量配置给中间件使用的so文件：libock_shore_mpiio.so、libock_shore_pnetcdf.so、libock_shore_netcdf.so、libock_shore_hdf5.so，需要用户自行保证so文件的安全性、可用性及可靠性。
- 请确保客户场景允许使用LD_PRELOAD。
- 遵循标准MPI-IO/PnetCDF/NetCDF/HDF5接口，接口参数合法性、编码安全性、渗透安全性由调用者保证。

应用场景

Hyper IO，在Lammps、Grapes、NEMO和WRF等高性能计算应用，以及大量后处理、可视化及分析框架中广泛应用。

7.2 使用指南

7.2.1 使用 Hyper IO

Hyper IO是一类提供系统软件和应用软件之间连接的软件，提供科学数据的存储描述格式定义和文件操作，在高性能计算应用以及大量后处理、可视化及分析框架中广泛应用，同时考虑兼容其他应用，提供文件格式转换能力。

前提条件

已安装Hyper IO，已安装应用运行所需第三方库（如MPI、PnetCDF、NetCDF、HDF5）。

须知

使用Hyper IO的IO Middleware组件时不支持同时叠加Hyper IO中的AdhocFS或IO转发特性。

版权所有 © 华为技术有限公司

映射关系

表 7-2 加速库和 JSON 配置文件的映射关系

应用名称	加速库	JSON配置文件
WRF	libock_shore_pnetcdf.so	wrf.json
NEMO	libock_shore_netcdf.so	nemo.json
Grapes	libock_shore_mpiio.so	grapes.json
LAMMPS	libock_shore_hdf5.so/ libock_shore_pnetcdf.so	lammps.json
QE	libock_shore_hdf5.so	qe.json

操作步骤

步骤1 使用PuTTY工具，以普通用户登录作业执行节点。

步骤2 执行以下命令，设置JSON配置文件环境变量。

export OCK_IO_CONFIG=*/opt/IO_Middleware***/IO_Middleware_2.6.1_aarch64/conf/***xxx*.json**

以Grapes为例，执行命令为**export OCK_IO_CONFIG=***/opt/IO_Middleware***/IO_Middleware_2.6.1_aarch64/conf/grapes.json**

说明

JSON配置文件属主为当前用户，且要有读权限。

步骤3 （可选）如果想获得应用最优性能提升，请修改JSON配置文件。

说明

- 已提供各自应用不同的JSON样例文件，运行不同的应用使用对应的JSON文件即可，可以根据实际需求修改JSON文件。
- 配置文件中的配置项应避免重复配置，若重复配置则以第一次配置为准。如配置"LOG_FLOW" : "ON" ，"LOG_FLOW" : "OFF"，则生效的为ON。
- 用户如果需要修改样例文件，需要将样例文件的内容复制，新建一个JSON文件并拷贝复制内容，再执行以下命令，按需修改该JSON文件。

1. 执行以下命令，拷贝并配置JSON文件。

   cp */opt/IO_Middleware***/IO_Middleware_2.6.1_aarch64/conf/***xxx*.json** */userpath/to/xxx.json*

   vi */userpath/to/xxx*.json**

2. 按“i”进入编辑模式，根据实际情况修改配置文件，参数说明如表7-3、表7-4和表7-5所示。
   - （可选）需要配置数据集整体信息。

 版权所有 © 华为技术有限公司

表 7-3 数据集整体信息（DATASET）

名称		描述	类型	缺省值
数据集整体通用信息（GENERAL）	LOG_FLOW	打印FLOW模块日志。	String ■ ON：开启 ■ OFF：关闭	OFF
	LOG_PROCESSOR	打印PROCESSOR模块日志。	String ■ ON：开启 ■ OFF：关闭	OFF
	LOG_BRIDGE	打印BRIDGE模块日志。	String ■ ON：开启 ■ OFF：关闭	OFF
	LOG_INTERCEPTOR	打印所有INTERCEPTOR模块日志。	String ■ ON：开启 ■ OFF：关闭	OFF
	LOG_CONFIG	打印CONFIG模块日志。	String ■ ON：开启 ■ OFF：关闭	OFF
	TRANSCACHESIZE	当数据总量达到Cache大小时，聚合成一次写操作。取值范围为[0，128]，单位MB，缺省值为0，表示不聚合。	Integer	0

版权所有 © 华为技术有限公司

名称		描述	类型	缺省值
	COMPRESS METHOD	数据压缩方法。 取值范围为[1，2]，缺省值为1。	Integer ■ 1：表示压缩算法为LZ4 ■ 2：表示压缩算法为Bzip2	1
	COMPRESSL EVEL	数据压缩级别。 取值范围为[0，5]，缺省值为0，表示不压缩。 数据压缩级别越高，数据压缩比越高。	Integer	0
	WITHOUTGR OUP	不配置group信息接入Hyper IO。	String ■ ON：开启 ■ OFF：关闭	OFF
数据集访问模式信息（DATA_ACCESS_PATTERN）	FIXED	应用中所有Entity的写操作在每个step间是否保持不变。	Boolean ■ false ■ true	false
	RANDOM	应对粒子型应用中不要求保证粒子顺序的场景。	Boolean ■ false ■ true	false
	WRITE_AGG REGATE_STE PS	元数据多少个step聚合成一次写，避免IO出现频率高且数据量小的情况。 取值范围为[1，64]。	Integer	1
	WRITE_AGG REGATE_RA NKS	元数据多少个rank聚合成一个写。 取值范围为[1，64]。	Integer	1

- （可选）如果需要使用文件聚合功能，需要配置数据集整体组信息。

版权所有 © 华为技术有限公司

表 7-4 数据集整体组信息（GROUPS）

名称	描述	类型	缺省值
GROUPNAME	必填。聚合模式下聚合后文件名称，需要设置为可以匹配FILEWHITELIST正则的字符串。	String，最大长度64	-
FILEWHITELIST	可选。将符合该正则表达式的文件聚合为上述GROUPNAME文件。	String，最大长度64	-

- （可选）如果需要设置单个文件内容可乱序输出，需要配置数据集个体信息。

表 7-5 数据集个体信息（ENTITIES）

名称	描述	类型	缺省值
NAME	必填。路径表示的数据项名称。	String，最大长度64	-
SHUFFLE	可选。应对粒子型应用中不要求保证粒子顺序的场景。	Boolean ■ false ■ true 不区分大小写。	false

- 包含全量配置项的配置文件样例如下所示，可根据实际需要配置所需配置项的最小集即可。

```
{
  "DATASET":
  {
     "GENERAL": {
        "LOG_FLOW" : "OFF",
        "LOG_PROCESSOR": "OFF",
        "LOG_BRIDGE" : "OFF",
        "LOG_INTERCEPTOR" : "OFF",
        "LOG_CONFIG" : "OFF",
        "TRANSCACHESIZE" : 0,
        "COMPRESSMETHOD" : 1,
        "COMPRESSLEVEL" : 0,
        "WITHOUTGROUP" : OFF,
        "DATA_ACCESS_PATTERN": {
           "FIXED": "false",
           "RANDOM": false,
           "WRITE_AGGREGATE_STEPS": 1,
           "WRITE_AGGREGATE_RANKS": 1
        }
     },
     "GROUPS":
     [
        {
           "GROUPNAME": "xxxxx",
           "FILEWHITELIST" : "xxxxx",
           "ENTITIES":
           [
              {
```


版权所有 © 华为技术有限公司

```
                "NAME": "xxxxx",
                "SHUFFLE": false
            }
        ]
    }
  ]
 }
}
```

3. 按“Esc”键，输入**:wq!**，按“Enter”保存并退出编辑。
4. 重新设置JSON配置文件环境变量。

   export OCK_IO_CONFIG=*/userpath/to/xxx***.json**

步骤4 运行应用程序，使能IO能力。通过LD_PRELOAD环境变量运行应用定向到Hyper IO，应用运行完成之后会生成以shore和format结尾的目录和与原文件同名的文件。

- 以shore结尾的目录用于存储数据信息。
- 以format结尾的目录用于存储描述数据的结构信息。
- 同名文件用于存储IO Middleware配置文件信息。

以Grapes为例，执行命令为**mpirun -N** *80* **-x LD_PRELOAD=***/opt/IO_Middleware/***IO_Middleware_2.6.1_aarch64/lib/libock_shore_mpiio.so -x OMP_NUM_THREADS=1 -mca pml ucx -mca btl ^vader,tcp,openib,uct -x UCX_TLS=self,sm,rc -x UCX_NET_DEVICES=mlx5_0:1 -x UCX_WARN_UNUSED_ENV_VARS=0** *./grapes_global.exe*，执行完成后生成以shore结尾的目录和与原文件同名的文件。

说明

- /opt/IO_Middleware/IO_Middleware_2.6.1_aarch64：表示Hyper IO的安装目录。
- 每个高性能计算应用只能设置一个加速库环境变量。

步骤5 （可选）使用Hyper IO格式转换功能。

须知

- 使用Hyper IO格式转换功能前需确保系统环境中已安装开源MPI-IO，否则会导致功能运行异常。
- 使用Hyper IO格式转换功能时转换的目标路径和转换结果输出路径不能相同。
- 转换失败会输出日志在屏幕上，需手动删除生成文件。
- 设置多个线程运行时需要在mpirun命令中添加-map-by ppr: *m*:node:pe=*n* 参数进行资源映射，m表示每个节点运行的进程数，n表示每个进程可用的CPU核数。
- 应用推荐配置：
  - bcc-esm应用使用netcdf中间件库的情况下，转换工具不需要使用-t与-d参数。
  - wrf应用使用pnetcdf中间件库的情况下，转换工具需要使用-t8与-d参数。

Hyper IO格式转换功能使用方式如下：

- 单节点转换：

  */opt/IO_Middleware/***IO_Middleware_2.6.1_aarch64/bin/**
  ock_shore_convertor *[--format <目标转换格式（当前仅支持netcdf)>] [-t <1~8>（设置线程数）] [-d（无参数，设置是否开启数据集压缩，默认不开*

 版权所有 © 华为技术有限公司

启）] 共享存储目录输入为shore文件或shore文件所在目录 目标格式文件输出目录

- 多节点多进程转换：

  mpirun -hostfile *$HOSTFILE* **-np** *进程数* **/opt/IO_Middleware/IO_Middleware_2.6.1_aarch64/bin/ock_shore_convertor** *[--format <目标转换格式（当前仅netcdf)>] [-t <1~8>（设置线程数）] [-d（无参数，设置是否开启数据集压缩，默认不开启）] 共享存储目录输入为shore文件或shore文件所在目录 目标格式文件输出目录*

步骤6 （可选）如果不使用Hyper IO，就不再需要该环境变量，执行以下命令，删除已生效的环境变量。

unset OCK_IO_CONFIG

----结束

Hyper IO 支持的应用和版本

应用	版本
WRF	4.2.2版本
LAMMPS	版本30Nov20
Grapes	• CMA_GFS_3.3版本3.3 • CMA_MESO版本5.1
NEMO	4.2版本
QE	6.4.1版本

7.2.2 Hyper IO 函数说明

应用场景

Hyper IO提供以下类型的接口供高性能计算应用使用：

名称	说明
MPI-IO接口函数	通过对MPI-IO的API进行拦截，定向到Hyper IO的同名接口函数获得性能加速。
PnetCDF接口函数	通过对PnetCDF的API进行拦截，定向到Hyper IO的同名接口函数获得性能加速。
NetCDF接口函数	通过对NetCDF的API进行拦截，定向到Hyper IO的同名接口函数获得性能加速。
HDF5接口函数	通过对HDF5的API进行拦截，定向到Hyper IO的同名接口函数获得性能加速。

 版权所有 © 华为技术有限公司

7.2.2.1 MPI-IO 桥接函数

7.2.2.1.1 函数说明

该部分包含134个MPI-IO桥接接口。

7.2.2.1.2 数据类型

函数定义

- 创建连续的数据类型。

  MPI_Type_contiguous(int count, MPI_Datatype oldtype, MPI_Datatype *newtype)

  PMPI_Type_contiguous(int count, MPI_Datatype oldtype, MPI_Datatype *newtype)

- 创建矢量数据类型。

  MPI_Type_vector(int count, int blocklength, int stride, MPI_Datatype oldtype, MPI_Datatype *newtype)

  PMPI_Type_vector(int count, int blocklength, int stride, MPI_Datatype oldtype, MPI_Datatype *newtype)

- 创建偏移量位的矢量数据类型。

  MPI_Type_create_hvector(int count, int blocklength, MPI_Aint stride, MPI_Datatype oldtype, MPI_Datatype *newtype)

  PMPI_Type_create_hvector(int count, int blocklength, MPI_Aint stride, MPI_Datatype oldtype, MPI_Datatype *newtype)

- 创建描述n维数组的子数组的数据类型。

  MPI_Type_create_subarray(int ndims, const int sizeArray[], const int subSizeArray[], const int subStartArray[], int order, MPI_Datatype oldtype, MPI_Datatype *newtype)

  PMPI_Type_create_subarray(int ndims, const int sizeArray[], const int subSizeArray[], const int subStartArray[], int order, MPI_Datatype oldtype, MPI_Datatype *newtype)

- 提交数据类型。

  MPI_Type_commit(MPI_Datatype *type)

  PMPI_Type_commit(MPI_Datatype *type)

- 复制具有关联键值的数据类型。

  MPI_Type_dup(MPI_Datatype oldtype, MPI_Datatype *newtype)

  PMPI_Type_dup(MPI_Datatype oldtype, MPI_Datatype *newtype)

- 释放自定义数据类型。

  MPI_Type_free(MPI_Datatype *type)

  PMPI_Type_free(MPI_Datatype *type)

 版权所有 © 华为技术有限公司

参数

参数名	描述	取值范围	输入/输出
count	块数	正整数	输入
blocklength	每个块中的元素个数	正整数	输入
stride	每个块开始之间的元素数	正整数	输入
ndims	数组维度数	正整数[1, 10]	输入
order	数组存储顺序标志	正整数（MPI_ORDER_C、MPI_ORDER_FORTRAN）	输入
sizeArray[]	完整数组的每个维度中oldtype类型的元素数	正整数数组	输入
subSizeArray[]	子数组的每个维度中oldtype类型的元素数	正整数数组	输入
subStartArray[]	每个维度中子数组的起始坐标	非负数数组	输入
oldtype	旧数据类型（句柄）	有效句柄	输入
*newtype	目标地址，存放新的类型（句柄）	非空	输出
*type	释放的自定义数据类型（句柄）	非空	输入

返回值

- 成功：返回MPI_SUCCESS。
- 失败：返回错误码。

错误码

错误码	描述
MPI_ERR_COUNT	无效count参数
MPI_ERR_TYPE	无效数据类型参数
MPI_ERR_ARG	其他类型的无效参数
MPI_ERR_OTHER	其他错误

版权所有 © 华为技术有限公司

注意

- oldtype目前仅支持MPI基本数据类型。
- count *（oldtype字节数）必须<=INT_MAX。
- blocklength * count *（oldtype字节数）必须<=INT_MAX。

7.2.2.1.3 文件操作

7.2.2.1.3.1 以聚合模式打开文件

函数定义

MPI_File_open(MPI_Comm comm, const char* filename, int amode, MPI_Info info, MPI_File *fh)

PMPI_File_open(MPI_Comm comm, const char* filename, int amode, MPI_Info info, MPI_File *fh)

参数

参数名	描述	取值范围	输入/输出
comm	通信器（句柄）	非空	输入
*filename	要打开文件的名称	非空，长度小于256字节	输入
amode	文件访问模式	整数	输入
info	Info对象（句柄）	非空	输入
*fh	目标地址，存放新的文件（句柄）	非空	输出

返回值

- 成功：返回MPI_SUCCESS。
- 失败：返回错误码。

错误码

错误码	描述
MPI_ERR_BAD_FILE	无效的文件名
MPI_ERR_AMODE	不支持的访问模式
MPI_ERR_ARG	其他类型的无效参数
MPI_ERR_ACCESS	权限被拒绝
MPI_ERR_IO	其他I/O错误

 版权所有 © 华为技术有限公司

错误码	描述
MPI_ERR_OTHER	其他错误

注意

Hyper IO支持以以下amode打开文件：

- MPI_MODE_CREATE
- MPI_MODE_DELETE_ON_CLOSE
- MPI_MODE_RDONLY
- MPI_MODE_WRONLY
- MPI_MODE_RDWR（将以只写看待，若以此模式打开文件后执行读操作，将报错）

7.2.2.1.3.2 关闭文件

函数定义

MPI_File_close(MPI_File *fh)

PMPI_File_close(MPI_File *fh)

参数

参数名	描述	取值范围	输入/输出
*fh	文件（句柄）	非空	输入

返回值

- 成功：返回MPI_SUCCESS。
- 失败：返回错误码。

错误码

错误码	描述
MPI_ERR_FILE	无效文件句柄
MPI_ERR_IO	其他I/O错误
MPI_ERR_OTHER	其他错误

版权所有 © 华为技术有限公司

7.2.2.1.3.3 删除文件

函数定义

MPI_File_delete(const char* filename, MPI_Info info)

PMPI_File_delete(const char* filename, MPI_Info info)

参数

参数名	描述	取值范围	输入/输出
*filename	要删除文件的名称	非空，长度小于256字节	输入
info	info对象（句柄）	非空	输入

返回值

- 成功：返回MPI_SUCCESS。
- 失败：返回错误码。

错误码

错误码	描述
MPI_ERR_NO_SUCH_FILE	文件不存在

注意

若使用了配置文件中group字段使能文件聚合功能，MPI_File_delete/PMPI_File_delete将只会删除同名文件，不删除.shore结尾文件夹中的数据。

7.2.2.1.3.4 使数据访问操作的语义保持一致

函数定义

MPI_File_sync(MPI_File fh)

PMPI_File_sync(MPI_File fh)

参数

参数名	描述	取值范围	输入/输出
fh	文件句柄	非空	输入

版权所有 © 华为技术有限公司

返回值

返回MPI_SUCCESS。

注意

中间件现接口返回值恒为MPI_SUCCESS，需要等MPI_File_close / PMPI_File_close接口调用后，数据才会落盘。

7.2.2.1.4 信息设置与访问

7.2.2.1.4.1 以聚合模式设置和访问 info 对象的新值

函数定义

- 设置info对象的新值。
  MPI_File_set_info(MPI_File fh, MPI_Info info)
  PMPI_File_set_info(MPI_File fh, MPI_Info info)
- 返回新的信息对象，包含与文件关联的当前info的值。
  MPI_File_get_info(MPI_File fh, MPI_Info *info_used)
  PMPI_File_get_info(MPI_File fh, MPI_Info *info_used)

参数

参数名	描述	取值范围	输入/输出
fh	要删除文件的句柄	非空	输入
info	info对象（句柄）	非空	输入
*info	目标地址，存放info对象（句柄）	非空	输出

返回值

- 成功：返回MPI_SUCCESS。
- 失败：返回错误码。

错误码

错误码	描述
MPI_ERR_FILE	无效文件句柄

注意

info内部值对中间件无用，仅返回设置的info对象。

版权所有 © 华为技术有限公司

7.2.2.1.4.2 以聚合模式更改和返回进程对文件中数据的视图

函数定义

- 更改进程对文件中数据的视图。

  MPI_File_set_view(MPI_File fh, MPI_Offset disp, MPI_Datatype etype, MPI_Datatype filetype, const char* datarep, MPI_Info info)

  PMPI_File_set_view(MPI_File fh, MPI_Offset disp, MPI_Datatype etype, MPI_Datatype filetype, const char* datarep, MPI_Info info)

- 返回进程对文件中数据的视图。

  MPI_File_get_view(MPI_File fh, MPI_Offset *disp, MPI_Datatype *etype, MPI_Datatype *filetype, char* datarep)

  PMPI_File_get_view(MPI_File fh, MPI_Offset *disp, MPI_Datatype *etype, MPI_Datatype *filetype, char* datarep)

参数

参数名	描述	取值范围	输入/输出
fh	要删除文件的句柄	非空	输入
disp	位移	非负整数	输入
*disp	目标地址，存放位移	非空	输出
etype	基本数据类型（句柄）	非空	输入
*etype	目标地址，存放基本数据类型（句柄）	非空	输出
filetype	文件类型（句柄）	非空	输入
*filetype	目标地址，存放文件类型	非空	输出
const *datarep	数据表示	非空	输入
*datarep	目标地址，存放数据表示	非空	输出
info	info对象（句柄）	非空	输入

返回值

- 成功：返回MPI_SUCCESS。
- 失败：返回错误码。

版权所有 © 华为技术有限公司

错误码

错误码	描述
MPI_ERR_FILE	无效文件句柄
MPI_ERR_DISP	无效disp参数值
MPI_ERR_TYPE	无效数据类型（句柄）参数
MPI_ERR_ARG	其他类型的无效参数

注意

MPI_File_set_view / PMPI_File_set_view入参etype仅支持为MPI基础数据类型，读写时需要视图保持一致，且设置视图后，读写的基本单位为etype。

7.2.2.1.4.3 返回与打开的文件关联的访问模式

函数定义

MPI_File_get_amode(MPI_File fh, int *amode)

PMPI_File_get_amode(MPI_File fh, int *amode)

参数

参数名	描述	取值范围	输入/输出
fh	文件（句柄）	非空	输入
*amode	目标地址，存放访问模式	非空	输出

返回值

- 成功：返回MPI_SUCCESS。
- 失败：返回错误码。

错误码

错误码	描述
MPI_ERR_FILE	无效文件句柄

版权所有 © 华为技术有限公司

7.2.2.1.4.4 返回文件的进程组的副本

函数定义

MPI_File_get_group(MPI_File fh, MPI_Group *group)

PMPI_File_get_group(MPI_File fh, MPI_Group *group)

参数

参数名	描述	取值范围	输入/输出
fh	文件（句柄）	非空	输入
*group	目标地址，存放group句柄	非空	输出

返回值

- 成功：返回MPI_SUCCESS。
- 失败：返回错误码。

错误码

错误码	描述
MPI_ERR_FILE	无效文件句柄
MPI_ERR_GROUP	无效group句柄参数
MPI_ERR_OTHER	其他错误

7.2.2.1.4.5 返回单个文件指针的当前位置

函数定义

MPI_File_get_position(MPI_File fh, MPI_Offset *offset)

PMPI_File_get_position(MPI_File fh, MPI_Offset *offset)

参数

参数名	描述	取值范围	输入/输出
fh	文件（句柄）	非空	输入
*offset	目标地址，存放偏移量	非空	输出

版权所有 © 华为技术有限公司

返回值

- 成功：返回MPI_SUCCESS。
- 失败：返回错误码。

错误码

错误码	描述
MPI_ERR_FILE	无效文件句柄

7.2.2.1.4.6 返回文件中数据类型的范围

函数定义

MPI_File_get_type_extent(MPI_File fh, MPI_Datatype datatype, MPI_Aint *extent)

PMPI_File_get_type_extent(MPI_File fh, MPI_Datatype datatype, MPI_Aint *extent)

参数

参数名	描述	取值范围	输入/输出
fh	文件（句柄）	非空	输入
datatype	数据类型（句柄）	非空	输入
*extent	数据类型范围	非空	输出

返回值

- 成功：返回MPI_SUCCESS。
- 失败：返回错误码。

错误码

错误码	描述
MPI_ERR_FILE	无效文件句柄

注意

仅支持基本数据类型与中间件提供接口产生数据类型的计算。

 版权所有 © 华为技术有限公司

7.2.2.1.4.7 将视图相对偏移量转换为绝对字节位置

函数定义

MPI_File_get_byte_offset(MPI_File fh, MPI_Offset offset, MPI_Offset *disp)

PMPI_File_get_byte_offset(MPI_File fh, MPI_Offset offset, MPI_Offset *disp)

参数

参数名	描述	取值范围	输入/输出
fh	文件（句柄）	非空	输入
offset	偏移量	非负整数	输入
*disp	偏移量的绝对字节位置	非空	输出

返回值

- 成功：返回MPI_SUCCESS。
- 失败：返回错误码。

错误码

错误码	描述
MPI_ERR_FILE	无效文件句柄

7.2.2.1.4.8 以非聚合模式（noncollective）更新单个文件指针

函数定义

MPI_File_seek(MPI_File fh, MPI_Offset offset, int whence)

PMPI_File_seek(MPI_File fh, MPI_Offset offset, int whence)

参数

参数名	描述	取值范围	输入/输出
fh	文件（句柄）	非空	输入
offset	偏移量	非负整数	输入

版权所有 © 华为技术有限公司

参数名	描述	取值范围	输入/输出
whence	更新模式	MPI_SEEK_SET MPI_SEEK_CUR	输入

返回值

- 成功：返回MPI_SUCCESS。
- 失败：返回错误码。

错误码

错误码	描述
MPI_ERR_FILE	无效文件句柄

7.2.2.1.5 句柄转换

函数定义

- 将C句柄转换为Fortran句柄。
  MPI_File_c2f(MPI_File file)
  PMPI_File_c2f(MPI_File file)
- 将Fortran句柄转换为C句柄。
  MPI_File_f2c(MPI_Fint file)
  PMPI_File_f2c(MPI_Fint file)

参数

参数名	描述	取值范围	输入/输出
file	c/fortran格式句柄	非负	输入

返回值

- 成功：返回Fotran/c格式句柄。
- 失败：返回NULL。

注意

MPI_File_c2f与MPI_File_f2c仅转换文件句柄。

版权所有 © 华为技术有限公司

7.2.2.1.6 异常处理程序

函数定义

- 注册用户例程函数以用作MPI异常处理程序。

  MPI_File_create_errhandler(MPI_File_errhandler_function *function, MPI_Errhandler *errhandler)

  PMPI_File_create_errhandler(MPI_File_errhandler_function *function, MPI_Errhandler *errhandler)
- 提供错误码并调用异常处理程序。

  MPI_File_call_errhandler(MPI_File fh, int errorcode)

  PMPI_File_call_errhandler(MPI_File fh, int errorcode)
- 设置文件的异常处理程序。

  MPI_File_set_errhandler(MPI_File fh, MPI_Errhandler errhandler)

  PMPI_File_set_errhandler(MPI_File fh, MPI_Errhandler errhandler)
- 获取文件的异常处理程序。

  MPI_File_get_errhandler(MPI_File fh, MPI_Errhandler *errhandler)

  PMPI_File_get_errhandler(MPI_File fh, MPI_Errhandler *errhandler)

参数

参数名	描述	取值范围	输入/输出
*function	用户定义异常处理程序	非空	输入
*errhandler	目标地址，存放MPI错误处理（句柄）	非空	输出
fh	绑定错误处理句柄的文件（句柄）	非空	输入
errorcode	MPI错误码	非空	输入
errhandler	MPI错误处理（句柄）	非空	输入

返回值

- 成功：返回MPI_SUCCESS。
- 失败：返回错误码。

错误码

错误码	描述
MPI_ERR_FILE	无效文件句柄
MPI_ERR_ARG	其他类型的无效参数
MPI_ERR_OTHER	其他错误

版权所有 © 华为技术有限公司

注意

用户定义的异常处理程序正确性由用户进行保证。

7.2.2.1.7 文件写入

7.2.2.1.7.1 从指定的位置开始写入文件

函数定义

- 以阻塞非聚合模式从单个文件指针指定的位置开始写入文件。

  MPI_File_write(MPI_File fh, const void *buf, int count, MPI_Datatype datatype, MPI_Status *status)

  PMPI_File_write(MPI_File fh, const void *buf, int count, MPI_Datatype datatype, MPI_Status *status)
- 以阻塞聚合模式从单个文件指针指定的位置开始写入文件。

  MPI_File_write_all(MPI_File fh, const void *buf, int count, MPI_Datatype datatype, MPI_Status *status)

  PMPI_File_write_all(MPI_File fh, const void *buf, int count, MPI_Datatype datatype, MPI_Status *status)
- 在一个分离的集合例程的开始部分，从各个文件指针指定的位置开始写入文件。

  MPI_File_write_all_begin(MPI_File fh, const void *buf, int count, MPI_Datatype datatype)

  PMPI_File_write_all_begin(MPI_File fh, const void *buf, int count, MPI_Datatype datatype)
- 在一个分离的集合例程的结束部分，从各个文件指针指定的位置开始写入文件。

  MPI_File_write_all_end(MPI_File fh, const void *buf, MPI_Status *status)

  PMPI_File_write_all_end(MPI_File fh, const void *buf, MPI_Status *status)

参数

参数名	描述	取值范围	输入/输出
fh	文件（句柄）	非空	输入
*buf	缓冲区的初始地址	非空	输入
count	缓冲区中的元素数	正整数	输入
datatype	每个缓冲元素的数据类型（句柄）	非空	输入
*status	Status对象（句柄）	非空	输出

返回值

- 成功：返回MPI_SUCCESS。
- 失败：返回错误码。

版权所有 © 华为技术有限公司

错误码

错误码	描述
MPI_ERR_NO_SUCH_FILE	文件不存在
MPI_ERR_BAD_FILE	无效的文件名
MPI_ERR_FILE	无效文件句柄
MPI_ERR_ARG	其他类型的无效参数
MPI_ERR_AMODE	不支持的访问模式
MPI_ERR_BUFFER	无效buffer参数
MPI_ERR_COUNT	无效count参数
MPI_ERR_TYPE	无效类型参数
MPI_ERR_DIMS	维度不合理
MPI_ERR_NO_SPACE	空间不足
MPI_ERR_NO_MEM	内存错误
MPI_ERR_IO	其他I/O错误
MPI_ERR_ACCESS	权限被拒绝
MPI_ERR_IN_STATUS	已存在分离式操作开始
MPI_ERR_OTHER	其他错误

注意

- 中间件不存在collective与independent的严格划分。
- begin与end以file为核心进行对应，begin函数需要在调用end函数之后，才能再次调用。
- 若文件设置了视图，则读写时需要视图保持一致，且设置视图后，读写的基本单位为MPI_File_set_view / PMPI_File_set_view接口中的etype。

7.2.2.1.7.2 以显式指定的偏移量写入文件

函数定义

- 以阻塞非聚合模式显式指定的偏移量写入文件。

  MPI_File_write_at(MPI_File fh, MPI_Offset offset, const void *buf, int count, MPI_Datatype datatype, MPI_Status *status)

  PMPI_File_write_at(MPI_File fh, MPI_Offset offset, const void *buf, int count, MPI_Datatype datatype, MPI_Status *status)
- 以阻塞聚合模式显式指定的偏移量写入文件。

  MPI_File_write_at_all(MPI_File fh, MPI_Offset offset, const void *buf, int count, MPI_Datatype datatype, MPI_Status *status)

版权所有 © 华为技术有限公司

PMPI_File_write_at_all(MPI_File fh, MPI_Offset offset, const void *buf, int count, MPI_Datatype datatype, MPI_Status *status)

- 在一个分离的集合例程的开始部分，以显式指定的偏移量写入文件。

  MPI_File_write_at_all_begin(MPI_File fh, MPI_Offset offset, const void *buf, int count, MPI_Datatype datatype)

  PMPI_File_write_at_all_begin(MPI_File fh, MPI_Offset offset, const void *buf, int count, MPI_Datatype datatype)

- 在一个分离的集合例程的结束部分，以显式指定的偏移量写入文件。

  MPI_File_write_at_all_end(MPI_File fh, const void *buf, MPI_Status *status)

  PMPI_File_write_at_all_end(MPI_File fh, const void *buf, MPI_Status *status)

参数

参数名	描述	取值范围	输入/输出
fh	文件（句柄）	非空	输入
offset	偏移量	整数	输入
*buf	缓冲区的初始地址	非空	输入
count	缓冲区中的元素数	非负整数	输入
datatype	每个缓冲元素的数据类型（句柄）	非空	输入
*status	Status对象（句柄）	非空	输出

返回值

- 成功：返回MPI_SUCCESS。
- 失败：返回错误码。

错误码

错误码	描述
MPI_ERR_NO_SUCH_FILE	文件不存在
MPI_ERR_BAD_FILE	无效的文件名
MPI_ERR_FILE	无效文件句柄
MPI_ERR_ARG	其他类型的无效参数
MPI_ERR_AMODE	不支持的访问模式
MPI_ERR_BUFFER	无效buffer参数
MPI_ERR_COUNT	无效count参数

 版权所有 © 华为技术有限公司

错误码	描述
MPI_ERR_TYPE	无效类型参数
MPI_ERR_DIMS	维度不合理
MPI_ERR_NO_SPACE	空间不足
MPI_ERR_NO_MEM	内存错误
MPI_ERR_IO	其他I/O错误
MPI_ERR_ACCESS	权限被拒绝
MPI_ERR_IN_STATUS	已存在分离式操作开始
MPI_ERR_OTHER	其他错误

注意

- 中间件不存在collective与independent的严格划分。
- begin与end以file为核心进行对应，begin函数需要在调用end函数之后，才能再次调用。
- 若文件设置了视图，则读写时需要视图保持一致，且设置视图后，读写的基本单位为MPI_File_set_view / PMPI_File_set_view接口中的etype。

7.2.2.1.7.3 以非阻塞方式写入文件

函数定义

- 以非聚合模式从单个文件指针指定的位置开始写入文件。

  MPI_File_iwrite(MPI_File fh, const void *buf, int count, MPI_Datatype datatype, MPI_Request *request)

  PMPI_File_iwrite(MPI_File fh, const void *buf, int count, MPI_Datatype datatype, MPI_Request *request)
- 以聚合模式从单个文件指针指定的位置开始写入文件。

  MPI_File_iwrite_all(MPI_File fh, const void *buf, int count, MPI_Datatype datatype, MPI_Request *request)

  PMPI_File_iwrite_all(MPI_File fh, const void *buf, int count, MPI_Datatype datatype, MPI_Request *request)
- 以非聚合模式显式指定的偏移量写入文件。

  MPI_File_iwrite_at(MPI_File fh, MPI_Offset offset, const void *buf, int count, MPI_Datatype datatype, MPI_Request *request)

  PMPI_File_iwrite_at(MPI_File fh, MPI_Offset offset, const void *buf, int count, MPI_Datatype datatype, MPI_Request *request)
- 以聚合模式显式指定的偏移量写入文件。

  MPI_File_iwrite_at_all(MPI_File fh, MPI_Offset offset, const void *buf, int count, MPI_Datatype datatype, MPI_Request *request)

  PMPI_File_iwrite_at_all(MPI_File fh, MPI_Offset offset, const void *buf, int count, MPI_Datatype datatype, MPI_Request *request)

 版权所有 © 华为技术有限公司

参数

参数名	描述	取值范围	输入/输出
fh	文件（句柄）	非空	输入
offset	偏移量	整数	输入
*buf	缓冲区的初始地址	非空	输入
count	缓冲区中的元素数	非负整数	输入
datatype	每个缓冲元素的数据类型（句柄）	非空	输入
*request	request对象（句柄）	非空	输出

返回值

- 成功：返回MPI_SUCCESS。
- 失败：返回错误码。

错误码

错误码	描述
MPI_ERR_NO_SUCH_FILE	文件不存在
MPI_ERR_BAD_FILE	无效的文件名
MPI_ERR_FILE	无效文件句柄
MPI_ERR_ARG	其他类型的无效参数
MPI_ERR_AMODE	不支持的访问模式
MPI_ERR_BUFFER	无效buffer参数
MPI_ERR_COUNT	无效count参数
MPI_ERR_TYPE	无效类型参数
MPI_ERR_DIMS	维度不合理
MPI_ERR_NO_SPACE	空间不足
MPI_ERR_NO_MEM	内存错误
MPI_ERR_IO	其他I/O错误
MPI_ERR_ACCESS	权限被拒绝
MPI_ERR_OTHER	其他错误

版权所有 © 华为技术有限公司

注意

- 中间件不存在collective与independent的严格划分。
- begin与end以file为核心进行对应，begin函数需要在调用end函数之后，才能再次调用。
- 经IO Middleware拦截后， iwrite与 write两个接口在底层复用了同一套核心实现逻辑。
- 若文件设置了视图，则读写时需要视图保持一致，且设置视图后，读写的基本单位为MPI_File_set_view / PMPI_File_set_view接口中的etype。

7.2.2.1.8 文件读取

7.2.2.1.8.1 从指定的位置开始读取文件

函数定义

- 以阻塞非聚合模式从单个文件指针指定的位置开始读取文件。

  MPI_File_read(MPI_File fh, void *buf, int count, MPI_Datatype datatype, MPI_Status *status)

  PMPI_File_read(MPI_File fh, void *buf, int count, MPI_Datatype datatype, MPI_Status *status)
- 以阻塞聚合模式从单个文件指针指定的位置开始读取文件。

  MPI_File_read_all(MPI_File fh, void *buf, int count, MPI_Datatype datatype, MPI_Status *status)

  PMPI_File_read_all(MPI_File fh, void *buf, int count, MPI_Datatype datatype, MPI_Status *status)
- 在一个分离的集合例程的开始部分，从各个文件指针指定的位置开始读取文件。

  MPI_File_read_all_begin(MPI_File fh, void* buf, int count, MPI_Datatype datatype)

  PMPI_File_read_all_begin(MPI_File fh, void* buf, int count, MPI_Datatype datatype)
- 在一个分离的集合例程的结束部分，从各个文件指针指定的位置开始读取文件。

  MPI_File_read_all_end(MPI_File fh, void* buf, MPI_Status *status)

  PMPI_File_read_all_end(MPI_File fh, void* buf, MPI_Status *status)

参数

参数名	描述	取值范围	输入/输出
fh	文件（句柄）	非空	输入
*buf	缓冲区的初始地址	非空	输出
count	缓冲区中的元素数	非负整数	输入
datatype	每个缓冲元素的数据类型（句柄）	非空	输入
*status	Status对象（句柄）	非空	输出

版权所有 © 华为技术有限公司

返回值

- 成功：返回MPI_SUCCESS。
- 失败：返回错误码。

错误码

错误码	描述
MPI_ERR_NO_SUCH_FILE	文件不存在
MPI_ERR_BAD_FILE	无效的文件名
MPI_ERR_FILE	无效文件句柄
MPI_ERR_ARG	其他类型的无效参数
MPI_ERR_AMODE	不支持的访问模式
MPI_ERR_BUFFER	无效buffer参数
MPI_ERR_COUNT	无效count参数
MPI_ERR_TYPE	无效类型参数
MPI_ERR_DIMS	维度不合理
MPI_ERR_NO_SPACE	空间不足
MPI_ERR_NO_MEM	内存错误
MPI_ERR_IO	其他I/O错误
MPI_ERR_ACCESS	权限被拒绝
MPI_ERR_IN_STATUS	已存在分离式操作开始
MPI_ERR_OTHER	其他错误

注意

- 中间件不存在collective与independent的严格划分。
- begin与end以file为核心进行对应，begin函数需要在调用end函数之后，才能再次调用。
- 若文件设置了视图，则读写时需要视图保持一致，且设置视图后，读写的基本单位为MPI_File_set_view / PMPI_File_set_view接口中的etype。

版权所有 © 华为技术有限公司

7.2.2.1.8.2 以显式指定的偏移量读取文件

函数定义

- 以阻塞非聚合模式显式指定的偏移量读取文件。

  MPI_File_read_at(MPI_File fh, MPI_Offset offset, void *buf, int count, MPI_Datatype datatype, MPI_Status *status)

  PMPI_File_read_at(MPI_File fh, MPI_Offset offset, void *buf, int count, MPI_Datatype datatype, MPI_Status *status)
- 以阻塞聚合模式显式指定的偏移量读取文件。

  MPI_File_read_at_all(MPI_File fh, MPI_Offset offset, void *buf, int count, MPI_Datatype datatype, MPI_Status *status)

  PMPI_File_read_at_all(MPI_File fh, MPI_Offset offset, void *buf, int count, MPI_Datatype datatype, MPI_Status *status)
- 在一个分离的集合例程的开始部分，以显式指定的偏移量读取文件。

  MPI_File_read_at_all_begin(MPI_File fh, MPI_Offset offset, void* buf, int count, MPI_Datatype datatype)

  PMPI_File_read_at_all_begin(MPI_File fh, MPI_Offset offset, void* buf, int count, MPI_Datatype datatype)
- 在一个分离的集合例程的结束部分，以显式指定的偏移量读取文件。

  MPI_File_read_at_all_end(MPI_File fh, void* buf, MPI_Status *status)

  PMPI_File_read_at_all_end(MPI_File fh, void* buf, MPI_Status *status)

参数

参数名	描述	取值范围	输入/输出
fh	文件（句柄）	非空	输入
offset	偏移量	整数	输入
*buf	缓冲区的初始地址	非空	输出
count	缓冲区中的元素数	非负整数	输入
datatype	每个缓冲元素的数据类型	非空	输入
*status	Status对象（句柄）	非空	输出

返回值

- 成功：返回MPI_SUCCESS。
- 失败：返回错误码。

版权所有 © 华为技术有限公司

错误码

错误码	描述
MPI_ERR_NO_SUCH_FILE	文件不存在
MPI_ERR_BAD_FILE	无效的文件名
MPI_ERR_FILE	无效文件句柄
MPI_ERR_ARG	其他类型的无效参数
MPI_ERR_AMODE	不支持的访问模式
MPI_ERR_BUFFER	无效buffer参数
MPI_ERR_COUNT	无效count参数
MPI_ERR_TYPE	无效类型参数
MPI_ERR_DIMS	维度不合理
MPI_ERR_NO_SPACE	空间不足
MPI_ERR_NO_MEM	内存错误
MPI_ERR_IO	其他I/O错误
MPI_ERR_ACCESS	权限被拒绝
MPI_ERR_IN_STATUS	已存在分离式操作开始
MPI_ERR_OTHER	其他错误

注意

- 中间件不存在collective与independent的严格划分。
- begin与end以file为核心进行对应，begin函数需要在调用end函数之后，才能再次调用。
- 若文件设置了视图，则读写时需要视图保持一致，且设置视图后，读写的基本单位为MPI_File_set_view / PMPI_File_set_view接口中的etype。

7.2.2.1.8.3 以非阻塞方式读取文件

函数定义

- 以非聚合模式从单个文件指针指定的位置开始读取文件。

  MPI_File_iread(MPI_File fh, void *buf, int count, MPI_Datatype datatype, MPI_Request *request)

  PMPI_File_iread(MPI_File fh, void *buf, int count, MPI_Datatype datatype, MPI_Request *request)
- 以聚合模式从单个文件指针指定的位置开始读取文件。

  MPI_File_iread_all(MPI_File fh, void *buf, int count, MPI_Datatype datatype, MPI_Request *request)

版权所有 © 华为技术有限公司

PMPI_File_iread_all(MPI_File fh, void *buf, int count, MPI_Datatype datatype, MPI_Request *request)

- 以非聚合模式显式指定的偏移量读取文件。

  MPI_File_iread_at(MPI_File fh, MPI_Offset offset, void *buf, int count, MPI_Datatype datatype, MPI_Request *request)

  PMPI_File_iread_at(MPI_File fh, MPI_Offset offset, void *buf, int count, MPI_Datatype datatype, MPI_Request *request)

- 以聚合模式显式指定的偏移量读取文件。

  MPI_File_iread_at_all(MPI_File fh, MPI_Offset offset, void *buf, int count, MPI_Datatype datatype, MPI_Request *request)

  PMPI_File_iread_at_all(MPI_File fh, MPI_Offset offset, void *buf, int count, MPI_Datatype datatype, MPI_Request *request)

参数

参数名	描述	取值范围	输入/输出
fh	文件（句柄）	非空	输入
offset	偏移量	整数	输入
*buf	缓冲区的初始地址	非空	输出
count	缓冲区中的元素数	非负整数	输入
datatype	每个缓冲元素的数据类型	非空	输入
*request	request对象（句柄）	非空	输出

返回值

- 成功：返回MPI_SUCCESS。
- 失败：返回错误码。

错误码

错误码	描述
MPI_ERR_NO_SUCH_FILE	文件不存在
MPI_ERR_BAD_FILE	无效的文件名
MPI_ERR_FILE	无效文件句柄
MPI_ERR_ARG	其他类型的无效参数
MPI_ERR_AMODE	不支持的访问模式
MPI_ERR_BUFFER	无效buffer参数

版权所有 © 华为技术有限公司

错误码	描述
MPI_ERR_COUNT	无效count参数
MPI_ERR_TYPE	无效类型参数
MPI_ERR_DIMS	维度不合理
MPI_ERR_NO_SPACE	空间不足
MPI_ERR_NO_MEM	内存错误
MPI_ERR_IO	其他I/O错误
MPI_ERR_ACCESS	权限被拒绝
MPI_ERR_OTHER	其他错误

注意

- 中间件不存在collective与independent的严格划分。
- begin与end以file为核心进行对应，begin函数需要在调用end函数之后，才能再次调用。
- iread与read无明显区别。
- 若文件设置了视图，则读写时需要视图保持一致，且设置视图后，读写的基本单位为MPI_File_set_view / PMPI_File_set_view接口中的etype。

7.2.2.1.9 终止函数

函数定义

MPI终止函数：

int **PMPI_Finalize()**

int **MPI_Finalize()**

返回值

- 成功：返回MPI_SUCCESS。
- 失败：返回错误码。

错误码

错误码	描述
MPI_ERR_OTHER	其他类型的错误

7.2.2.1.10 功能不支持函数

针对本小节内所述函数，Hyper IO暂未做功能支持，若文件使用Hyper IO创建后，使用该文件句柄调用以下函数，将会返回错误码或者做特殊处理。若以下函数入参中的

版权所有 © 华为技术有限公司

文件句柄为原MPI-IO打开，则函数将桥接至原函数调用，参数范围及错误码等将与原MPI-IO保持一致。

函数定义

- int **PMPI_File_iwrite_shared**(MPI_File fh, const void *buf, int count, MPI_Datatype datatype,
- *request)
- int **PMPI_File_write_ordered**(MPI_File fh, const void *buf, int count, MPI_Datatype datatype, MPI_Status *status)
- int **PMPI_File_write_ordered_begin**(MPI_File fh, const void *buf, int count, MPI_Datatype datatype)
- int **PMPI_File_write_ordered_end**(MPI_File fh, const void *buf, MPI_Status *status)
- int **PMPI_File_write_shared**(MPI_File fh, const void *buf, int count, MPI_Datatype datatype, MPI_Status *status)
- int **PMPI_File_read_ordered**(MPI_File fh, void *buf, int count, MPI_Datatype datatype, MPI_Status *status)
- int **PMPI_File_read_ordered_begin**(MPI_File fh, void *buf, int count, MPI_Datatype datatype)
- int **PMPI_File_read_ordered_end**(MPI_File fh, void *buf, MPI_Status *status)
- int **PMPI_File_read_shared**(MPI_File fh, void *buf, int count, MPI_Datatype datatype, MPI_Status *status)
- int **PMPI_File_iread_shared**(MPI_File fh, void *buf, int count, MPI_Datatype datatype, MPI_Request *request)
- int **PMPI_File_get_atomicity**(MPI_File fh, int *flag)
- int **PMPI_File_get_size**(MPI_File fh, MPI_Offset *size)
- int **PMPI_File_preallocate**(MPI_File fh, MPI_Offset size)
- int **PMPI_File_set_atomicity**(MPI_File fh, int flag)
- int **PMPI_File_set_size**(MPI_File fh, MPI_Offset size)

  为达成性能加速目的，Hyper IO不支持预分配文件大小功能，若文件使用Hyper IO创建后调用此接口，将恒返回MPI_SUCCESS，但文件大小不会改变。
- int **MPI_File_iwrite_shared**(MPI_File fh, const void *buf, int count, MPI_Datatype datatype, MPI_Request *request)
- int **MPI_File_write_ordered**(MPI_File fh, const void *buf, int count, MPI_Datatype datatype, MPI_Status *status)
- int **MPI_File_write_ordered_begin**(MPI_File fh, const void *buf, int count, MPI_Datatype datatype)
- int **MPI_File_write_ordered_end**(MPI_File fh, const void *buf, MPI_Status *status)
- int **MPI_File_write_shared**(MPI_File fh, const void *buf, int count, MPI_Datatype datatype, MPI_Status *status)
- int **MPI_File_read_ordered**(MPI_File fh, void *buf, int count, MPI_Datatype datatype, MPI_Status *status)

 版权所有 © 华为技术有限公司

- int **MPI_File_read_ordered_begin**(MPI_File fh, void *buf, int count, MPI_Datatype datatype)
- int **MPI_File_read_ordered_end**(MPI_File fh, void *buf, MPI_Status *status)
- int **MPI_File_read_shared**(MPI_File fh, void *buf, int count, MPI_Datatype datatype, MPI_Status *status)
- int **MPI_File_iread_shared**(MPI_File fh, void *buf, int count, MPI_Datatype datatype, MPI_Request *request)
- int **MPI_File_get_atomicity**(MPI_File fh, int *flag)
- int **MPI_File_get_size**(MPI_File fh, MPI_Offset *size)
- int **MPI_File_preallocate**(MPI_File fh, MPI_Offset size)
- int **MPI_File_set_atomicity**(MPI_File fh, int flag)
- int **MPI_File_set_size**(MPI_File fh, MPI_Offset size)

  为达成性能加速目的，Hyper IO不支持预分配文件大小功能，若文件使用Hyper IO创建后调用此接口，将恒返回MPI_SUCCESS，但文件大小不会改变。

7.2.2.2 NetCDF 桥接函数

7.2.2.2.1 函数说明

该部分包含NetCDF桥接接口90个，主要涉及NetCDF经典文件格式相关接口。

7.2.2.2.2 文件操作

函数定义

- 创建文件。

  NC_create(const char *name, int mode, size_t initialsz, int basepe,

  size_t *chunksizehintp, int useparallel, void *parameters, int *ncidp)
- 创建文件进行并行IO。

  nc_create_par(const char *name, int mode, MPI_Comm comm, MPI_Info info, int* ncidp)
- 打开现有文件。

  NC_open(const char *name, int mode, int basepe, size_t *chunksizehintp,

  int useparallel, void *parameters, int *ncidp)
- 打开现有文件进行并行IO。

  nc_open_par(const char *name, int mode, MPI_Comm comm, MPI_Info info, int* ncidp)
- 关闭打开的文件。

  nc_close(int ncid)
- 进入定义模式。

  nc_redef(int ncid)
- 退出定义模式。

  nc_enddef(int ncid)

 版权所有 © 华为技术有限公司

参数

参数名	描述	取值范围	输入/输出
ncid	文件ID	非负整数	输入
*ncidp	返回文件ID的指针	非空	输出
*name	文件名称	非空，长度小于256字节	输入
comm	MPI通信器句柄	非空	输入
info	MPIinfo对象句柄	非空	输入
*chunksizehintp	分块大小指针	非空	输入
useparallel	是否开启并行IO	整数	输入
*parameters	存储MPI信息的结构体指针	非空	输入
mode	创建/打开模式	整数	输入
initialsz	初始化大小	非负整数	输入
basepe	基本处理元素	整数	输入

返回值

- 成功：返回NC_NOERR。
- 失败：返回错误码。

错误码

错误码	描述
NC_EBADID	无效ncid
NC_EINVAL	无效参数
NC_ENOMEM	内存不足

7.2.2.2.3 信息设置

7.2.2.2.3.1 更改填充值模式

函数定义

更改填充值模式以提高写入性能。

nc_set_fill(int ncid, int fillmode, int *old_modep)

版权所有 © 华为技术有限公司

参数

参数名	描述	取值范围	输入/输出
ncid	文件ID	非负整数	输入
fillmode	填充模式	0或0x100	输入
*old_modep	返回之前填充模式的指针	非空	输出

返回值

- 成功：返回NC_NOERR。
- 失败：返回错误码。

错误码

错误码	描述
NC_EBADID	无效ncid
NC_EINVAL	无效参数
NC_ENOMEM	内存不足

7.2.2.2.3.2 定义一个新维度

函数定义

nc_def_dim(int ncid, const char* name, size_t len, int *idp)

参数

参数名	描述	取值范围	输入/输出
ncid	文件ID	非负整数	输入
*name	文件名称	非空，长度小于256字节	输入
len	维度长度	非负整数	输入
*idp	返回维度ID的指针	非空	输出

返回值

- 成功：返回NC_NOERR。

版权所有 © 华为技术有限公司

- 失败：返回错误码。

错误码

错误码	描述
NC_EBADID	无效ncid
NC_EMAXNAME	名称过长
NC_EINVAL	无效参数
NC_EDIMSIZE	无效维度大小
NC_ENOMEM	内存不足

7.2.2.2.3.3 定义新变量相关函数

函数定义

- 定义一个新变量。

  nc_def_var(int ncid, const char *name, nc_type xtype, int ndims, const int *dimidsp, int *varidp)

- 设置新的过滤器。

  nc_def_var_filter(int ncid, int varid, unsigned int id, size_t nparams, const unsigned int *params)

- 设置netCDF/HDF5文件中变量的zlib压缩和随机设置。

  nc_def_var_deflate(int ncid, int varid, int shuffle, int deflate, int deflate_level)

参数

参数名	描述	取值范围	输入/输出
ncid	文件ID	非负整数	输入
*name	变量名称	非空，长度小于256字节	输入
xtype	变量数据类型	非负整数	输入
ndims	维度数量	[1,1024]	输入
*dimidsp	维度ID列表	非空	输入
*varidp	变量ID的指针	非空	输入
id	过滤器ID	整数	输入
nparams	参数个数	非空	输入
*params	指向设置的参数的指针	非空	输入

版权所有 © 华为技术有限公司

参数名	描述	取值范围	输入/输出
varid	变量ID	非负整数	输入
shuffle	随机设置参数	非空	输入
deflate	压缩设置参数	非空	输入
deflate_level	压缩等级	非空	输入

返回值

- 成功：返回NC_NOERR。
- 失败：返回错误码。

错误码

错误码	描述
NC_EBADID	无效ncid
NC_EBADTYPE	无效数据类型
NC_EINVAL	无效参数

注意

- 目前仅支持有限数据类型：NC_BYTE、NC_UBYTE、NC_CHAR、NC_SHORT、NC_USHORT、NC_INT、NC_UINT、NC_INT64、NC_UINT64、NC_FLOAT、NC_DOUBLE。
- 维度ID列表内存大小需要与维度数量对应，不进行检查。

7.2.2.2.3.4 定义变量存储方式

函数定义

nc_def_var_chunking(int ncid, int varid, int storage, const size *chunksizesp)

参数

参数名	描述	取值范围	输入/输出
ncid	文件ID	非负整数	输入
varid	变量ID	非负整数	输入
storage	存储方式	[0,4]	输入
chunksizesp	指向块规模数组的指针	非空	输入

版权所有 © 华为技术有限公司

返回值

- 成功：返回NC_NOERR。
- 失败：返回错误码。

错误码

错误码	描述
NC_EBADID	无效ncid
NC_ENOTVAR	无效变量ID
NC_EINVAL	无效参数

注意

中间件暂时没有存储方式的配置，函数未完成功能上的支持。

7.2.2.2.3.5 设置变量填充值

函数定义

nc_def_var_fill(int ncid, int varid, int no_fill, const void* fill_value)

参数

参数名	描述	取值范围	输入/输出
ncid	文件ID	非负整数	输入
varid	变量ID	非空	输入
no_fill	是否填充	NC_NOFILL或NC_FILL	输入
*fill_value	指向填充值的指针	非空	输入

返回值

- 成功：返回NC_NOERR。
- 失败：返回错误码。

版权所有 © 华为技术有限公司

错误码

错误码	描述
NC_EBADID	无效ncid
NC_ENOTVAR	无效变量ID

注意

中间件暂时没有填充值功能，函数未完成功能上的支持。

7.2.2.2.3.6 设置变量访问方式

函数定义

nc_var_par_access(int ncid, int varid, int par_access)

参数

参数名	描述	取值范围	输入/输出
ncid	文件ID	非负整数	输入
varid	变量ID	非空	输入
par_access	访问方式	NC_COLLECTIVE或NC_INDEPENDENT	输入

返回值

- 成功：返回NC_NOERR。
- 失败：返回错误码。

错误码

错误码	描述
NC_EBADID	无效ncid
NC_ENOTVAR	无效变量ID
NC_EINVAL	无效参数

7.2.2.2.4 信息查询

版权所有 © 华为技术有限公司

7.2.2.2.4.1 查询公共信息

函数定义

- 查询有关文件的信息。
  nc_inq(int ncid, int *ndimsp, int *nvarsp, int *nattsp, int *unlimdimidp)
- 查询文件格式的信息。
  nc_inq_format(int ncid, int *formatp)
- 查询文件详细格式的信息。
  nc_inq_format_extended(int ncid, int *formatp, int *modep)
- 查询variable数量。
  nc_inq_nvars(int ncid, int *nvarsp)
- 查询维度数量。
  nc_inq_unlimdims(int ncid, int *nunlimdimsp, int *unlimdimidsp)
- 查询类型id。
  nc_inq_typeids(int ncid, int *ntypes, int *typeids)
- 查询group数量。
  nc_inq_grps(int ncid, int *numgrps, int *ncids)
- 查询groups名称。
  nc_inq_grpname(int ncid, char *name)
- 查询group所有ncid。
  nc_inq_grp_full_ncid(int ncid, const char* full_name, int *grp_ncid)
- 查询维度ID和数量。
  nc_inq_dimids(int ncid, int *ndims, int *dimids, int include_parents)
- 查询属性类型。
  nc_inq_atttype(int ncid, int varid, const char *name, nc_type *xtypep)
- 查询变量类型。
  nc_inq_vartype(int ncid, int varid, nc_type *typep)
- 查询变量名称。
  nc_inq_varname(int ncid, int varid, char *name)
- 查询全局属性。
  nc_inq_natts(int ncid, int *nattsp)
- 查询变量属性。
  nc_inq_varnatts(int ncid, int varid, int *nattsp)
- 查询变量信息。
  nc_inq_var_deflate(int ncid, int varid, int *shufflep, int *deflatep, int *deflate_levelp)
- 查找与变量关联并具有指定ID的过滤器的参数信息。
  nc_inq_var_filter_info(int ncid, int varid, unsigned int id, size_t *nparamsp, unsigned int *params)
- 查找与变量关联的过滤器集。

 版权所有 © 华为技术有限公司

nc_inq_var_filter_ids(int ncid, int varid, size_t * nfiltersp, unsigned int * ids)
nc_inq_var_filterids(int ncid, int varid, size_t *nfilters, unsigned int *filterids)

- 获取变量的存储和分块大小。
  nc_inq_var_chunking(int ncid, int varid, int *storagep, size_t *chunksizesp)

参数

参数名	描述	取值范围	输入/输出
ncid	文件ID	非负整数	输入
varid	变量ID	非负整数	输入
id	过滤器ID	非负整数	输入
*ndimsp	返回dim数量的指针	非空	输出
*nvarsp	返回variable数量的指针	非空	输出
*nattsp	返回attribute数量的指针	非空	输出
*unlimdimidp	返回第一个无限维度id的指针	非空	输出
*formatp	返回文件格式的指针	非空	输出
*modep	返回文件模式的指针	非空	输出
*nunlimdimsp	返回无限维度数量的指针	非空	输出
*ntypes	返回类型数量的指针	非空	输出
*typeids	返回第一个类型id的指针	非空	输出
*numgrps	返回group数量的指针	非空	输出
*nfilters	返回过滤器数量的指针	非空	输出
*filterids	返回过滤器ID的指针	非空	输出
*storagep	返回存储大小的指针	非空	输出
*chunksizesp	返回分块大小的指针	非空	输出

返回值

- 成功：返回NC_NOERR。
- 失败：返回错误码。

错误码

错误码	描述
NC_EBADID	无效ncid

版权所有 © 华为技术有限公司

注意

- nc_inq_format的默认文件格式为NC_FORMAT_CLASSIC，支持nc_create_par中cmode为NC_64BIT_OFFSET、NC_64BIT_DATA、NC_NETCDF4情况下的格式查询。
- nc_inq_format_extended的默认文件扩展格式为NC_FORMATX_PNETCDF，支持nc_create_par中cmode为NC_64BIT_OFFSET、NC_64BIT_DATA、NC_NETCDF4情况下的格式查询。
- *ndimsp、*nvarsp、*nattsp、*unlimdimidp、*formatp、*modep、*nunlimdimsp、*ntypes、*typeids、*numgrps、*nfilters、*filterids、*storagep、*chunksizesp参数在不需要获取数据时可以为空指针，需要获取数据时必须非空。

7.2.2.2.4.2 查询维度信息

函数定义

- 查询维度名称和长度。

  nc_inq_dim(int ncid, int dimid, char *name, size_t* lenp)
- 查询维度名称。

  nc_inq_dimname(int ncid, int dimid, char *name)
- 查询维度长度。

  nc_inq_dimlen(int ncid, int dimid, size_t *lenp)
- 根据名称查询维度ID。

  nc_inq_dimid(int ncid, const char *name, int* idp)

参数

参数名	描述	取值范围	输入/输出
ncid	文件ID	非负整数	输入
dimid	维度ID	非负整数	输入
const *name	维度名称	非空，长度小于256字节	输入
*name	返回维度名称的指针	非空	输出
*lenp	返回维度长度的指针	非空	输出
*idp	返回维度ID的指针	非空	输出

返回值

- 成功：返回NC_NOERR。
- 失败：返回错误码。

版权所有 © 华为技术有限公司

错误码

错误码	描述
NC_EBADID	无效ncid
NC_EBADDIM	无效dimid

注意

*name、*lenp、*idp参数在不需要获取数据时可以为空指针，需要获取数据时必须非空。

7.2.2.2.4.3 查询变量信息

函数定义

- 查询有关变量信息。

  nc_inq_var(int ncid, int varid, char *name, nc_type *xtypep, int *ndimsp, int *dimidsp, int *nattsp)
- 查询变量的维度数量。

  nc_inq_varndims(int ncid, int varid, int *ndimsp)
- 查询变量的维度ID。

  nc_inq_vardimid(int ncid, int varid, int *dimidsp)
- 根据名称查询变量ID。

  nc_inq_varid(int ncid, const char *name, int *varidp)

参数

参数名	描述	取值范围	输入/输出
ncid	文件ID	非负整数	输入
varid	variable ID	非负整数	输入
const *name	variable名称	非空，长度小于256字节	输入
*name	返回variable名称的指针	非空	输出
*xtypep	返回variable数据类型的指针	非空	输出
*ndimsp	返回variable dim数量的指针	非空	输出
*dimidsp	返回variable dimid数组的指针	非空	输出
*nattsp	返回variable attribute数量的指针	非空	输出
*varidp	返回variable id的指针	非空	输出

版权所有 © 华为技术有限公司

返回值

- 成功：返回NC_NOERR。
- 失败：返回错误码。

错误码

错误码	描述
NC_EBADID	无效ncid
NC_ENOTVAR	无效varid

注意

*name、*xtypep、*ndimsp、*dimidsp、*nattsp、*varidp参数在不需要获取数据时可以为空指针，需要获取数据时必须非空。

7.2.2.2.4.4 查询属性信息

函数定义

- 查询属性相关信息。

  nc_inq_att(int ncid, int varid, const char *name, nc_type *xtypep, size_t *lenp)
- 查询属性名称。

  nc_inq_attname(int ncid, int varid, int attnum, char *name)

参数

参数名	描述	取值范围	输入/输出
ncid	文件ID	非负整数	输入
varid	变量ID	非负整数或-1	输入
attnum	属性编号	非负整数	输入
const *name	属性名称	非空，长度小于256字节	输入
*xtypep	返回数据类型的指针	非空	输出
*lenp	返回属性长度的指针	非空	输出
*name	返回属性名称的指针	非空	输出

返回值

- 成功：返回NC_NOERR。

版权所有 © 华为技术有限公司

- 失败：返回错误码。

错误码

错误码	描述
NC_EBADID	无效ncid
NC_ENOTVAR	无效variableid
NC_ENOTATT	无法找到attribute
NC_ENOMEM	内存不足

注意

*xtypep、*lenp、*name参数在不需要获取数据时可以为空指针，需要获取数据时必须非空。

7.2.2.2.5 属性信息写入

函数定义

- 写入属性。

  nc_put_att(int ncid, int varid, const char* name, int xtype, size_t len, const void* op)
- 写入int类型属性。

  nc_put_att_int(int ncid, int varid, const char* name, nc_type xtype, size_t len, const int *op)
- 写入float类型属性。

  nc_put_att_float(int ncid, int varid, const char* name, nc_type xtype, size_t len, const float *op)
- 写入double类型属性。

  nc_put_att_double(int ncid, int varid, const char* name, nc_type xtype, size_t len, const double *op)
- 写入文本类型属性。

  nc_put_att_text(int ncid, int varid, const char* name, size_t len, const char *op)

参数

参数名	描述	取值范围	输入/输出
ncid	文件ID	非负整数	输入
varid	variable ID	非负整数	输入
xtype	attribute类型	非负整数	输入

版权所有 © 华为技术有限公司

参数名	描述	取值范围	输入/输出
name	attribute名称	非空	输入
len	attribute名称长度	非负整数	输入
op	数据指针	非空	输入

返回值

- 成功：返回NC_NOERR。
- 失败：返回错误码。

错误码

错误码	描述
NC_EBADID	无效ncid
NC_ENOTVAR	无效variableid
NC_EINVAL	无效参数
NC_ENOMEM	内存不足

注意

- 用户需要保证指针内存大小正确性，中间件无法检查指针大小。
- 目前仅支持上述类型att的写入，需要保证类型与函数对应。
- nc_type是int类型的别名。

7.2.2.2.6 属性信息读取

函数定义

- 读取属性。
  nc_get_att(int ncid, int varid, const char* name, void *ip)
- 读取int类型属性。
  nc_get_att_int(int ncid, int varid, const char* name, int *ip)
- 读取float类型属性。
  nc_get_att_float(int ncid, int varid, const char* name, float *ip)
- 读取double类型属性。
  nc_get_att_double(int ncid, int varid, const char* name, double *ip)
- 读取文本类型属性。
  nc_get_att_text(int ncid, int varid, const char* name, char *ip)

 版权所有 © 华为技术有限公司

参数

参数名	描述	取值范围	输入/输出
ncid	文件ID	非负整数	输入
varid	variable ID	非负整数	输入
name	attribute名称	非空	输入
ip	数据指针	非空	输出

返回值

- 成功：返回NC_NOERR。
- 失败：返回错误码。

错误码

错误码	描述
NC_EBADID	无效ncid
NC_ENOTVAR	无效variableid
NC_ENOTATT	无法找到attribute
NC_EINVAL	无效参数
NC_ENOMEM	内存不足

注意

- 用户需要保证指针内存大小正确性，中间件无法检查指针大小。
- 目前仅支持上述类型att的读取。

7.2.2.2.7 单元素模式数据写入

函数定义

- 以单元素方式写入数据。
  nc_put_var1(int ncid, int varid, const size_t* indexp, const void* op)
- 以单元素方式写入int类型数据。
  nc_put_var1_int(int ncid, int varid, const size_t* indexp, const int* op)
- 以单元素方式写入double类型数据。
  nc_put_var1_double(int ncid, int varid, const size_t* indexp, const double* op)

 版权所有 © 华为技术有限公司

参数

参数名	描述	取值范围	输入/输出
ncid	文件ID	非负整数	输入
varid	variable ID	非负整数	输入
*indexp	访问数据的起始向量	非空	输入
*op	数据指针	非空	输入

返回值

- 成功：返回NC_NOERR。
- 失败：返回错误码。

错误码

错误码	描述
NC_EBADID	无效ncid
NC_ENOTVAR	无法找到variable
NC_EINVALCOORDS	无效坐标参数
NC_EEDGE	超出维度界限

注意

用户需要保证指针内存大小正确性，中间件无法检查指针大小。

7.2.2.2.8 单元素模式数据读取

函数定义

- 以单元素方式读取数据。
  nc_get_var1(int ncid, int varid, const size_t* indexp, void* ip)
- 以单元素方式读取int类型数据。
  nc_get_var1_int(int ncid, int varid, const size_t* indexp, int* ip)
- 以单元素方式读取double类型数据。
  nc_get_var1_double(int ncid, int varid, const size_t* indexp, double* ip)

版权所有 © 华为技术有限公司

参数

参数名	描述	取值范围	输入/输出
ncid	文件ID	非负整数	输入
varid	variable ID	非负整数	输入
*indexp	访问数据的起始向量	非空	输入
*ip	数据指针	非空	输出

返回值

- 成功：返回NC_NOERR。
- 失败：返回错误码。

错误码

错误码	描述
NC_EBADID	无效ncid
NC_ENOTVAR	无法找到variable
NC_EINVALCOORDS	无效坐标参数
NC_EEDGE	超出维度界限

注意

用户需要保证指针内存大小正确性，中间件无法检查指针大小。

7.2.2.2.9 array 模式数据写入

函数定义

- 以stride模式写入数据。

  nc_put_vars_text(int ncid, int varid, const size_t* startp, const size_t* countp, const ptrdiff_t* stridep, const char* op)
- 以数组方式写入数据。

  nc_put_vara(int ncid, int varid, const size_t* startp, const size_t* countp, const void* op)
- 以数组方式写入double类型数据。

  nc_put_vara_double(int ncid, int varid, const size_t* startp, const size_t* countp, const double* op)
- 以数组方式写入char类型数据。

  int nc_put_vara_text(int ncid, int varid, const size_t* startp, const size_t* countp, const char* op)

版权所有 © 华为技术有限公司

- 以数组方式写入unsigned char类型数据。

  int nc_put_vara_uchar(int ncid, int varid, const size_t* startp, const size_t* countp, const unsigned char* op)
- 以数组方式写入signed char类型数据。

  int nc_put_vara_schar(int ncid, int varid, const size_t* startp, const size_t* countp, const signed char* op)
- 以数组方式写入short类型数据。

  int nc_put_vara_short(int ncid, int varid, const size_t* startp, const size_t* countp, const short* op)
- 以数组方式写入int类型数据。

  int nc_put_vara_int(int ncid, int varid, const size_t* startp, const size_t* countp, const int* op)
- 以数组方式写入long类型数据。

  int nc_put_vara_long(int ncid, int varid, const size_t* startp, const size_t* countp, const long* op)
- 以数组方式写入float类型数据。

  int nc_put_vara_float(int ncid, int varid, const size_t* startp, const size_t* countp, const float* op)
- 以数组方式写入unsigned short类型数据。

  int nc_put_vara_ushort(int ncid, int varid, const size_t* startp, const size_t* countp, const unsigned short* op)
- 以数组方式写入unsigned int类型数据。

  int nc_put_vara_uint(int ncid, int varid, const size_t* startp, const size_t* countp, const unsigned int* op)
- 以数组方式写入long long类型数据。

  int nc_put_vara_longlong(int ncid, int varid, const size_t* startp, const size_t* countp, const long long* op)
- 以数组方式写入unsigned long long类型数据。

  int nc_put_vara_ulonglong(int ncid, int varid, const size_t* startp, const size_t* countp, const unsigned long long* op)

参数

参数名	描述	取值范围	输入/输出
ncid	文件ID	非负整数	输入
varid	variable ID	非负整数	输入
*startp	访问数据的起始向量	非空	输入
*countp	访问数据的范围向量	非空	输入
*op	数据指针	非空	输入
const* stridep	跨步向量	非空	输入

版权所有 © 华为技术有限公司

返回值

- 成功：返回NC_NOERR。
- 失败：返回错误码。

错误码

错误码	描述
NC_EBADID	无效ncid
NC_ENOTVAR	无法找到variable
NC_EINVALCOORDS	无效坐标参数
NC_EEDGE	超出维度界限

注意

用户需要保证指针内存大小正确性，中间件无法检查指针大小。

7.2.2.2.10 array 模式数据读取

函数定义

- 以数组方式读取数据。

  nc_get_vara(int ncid, int varid, const size_t* startp, const size_t* countp, void* ip)
- 以数组方式读取double类型数据。

  nc_get_vara_double(int ncid, int varid, const size_t *startp, const size_t* countp, double* ip)
- 以数组方式读取char类型数据。

  int nc_get_vara_text(int ncid, int varid, const size_t* startp, const size_t* countp, char* ip)
- 以数组方式读取unsigned char类型数据。

  int nc_get_vara_uchar(int ncid, int varid, const size_t* startp, const size_t* countp, unsigned char* ip)
- 以数组方式读取signed char类型数据。

  int nc_get_vara_schar(int ncid, int varid, const size_t* startp, const size_t* countp, signed char* ip)
- 以数组方式读取short类型数据。

  int nc_get_vara_short(int ncid, int varid, const size_t* startp, const size_t* countp, short* ip)
- 以数组方式读取int类型数据。

  int nc_get_vara_int(int ncid, int varid, const size_t* startp, const size_t* countp, int* ip)
- 以数组方式读取long类型数据。

 版权所有 © 华为技术有限公司

int nc_get_vara_long(int ncid, int varid, const size_t* startp, const size_t* countp, long* ip)

- 以数组方式读取float类型数据。

  int nc_get_vara_float(int ncid, int varid, const size_t* startp, const size_t* countp, float* ip)

- 以数组方式读取unsigned short类型数据。

  int nc_get_vara_ushort(int ncid, int varid, const size_t* startp, const size_t* countp, unsigned short* ip)

- 以数组方式读取unsigned int类型数据。

  int nc_get_vara_uint(int ncid, int varid, const size_t* startp, const size_t* countp, unsigned int* ip)

- 以数组方式读取long long类型数据。

  int nc_get_vara_longlong(int ncid, int varid, const size_t* startp, const size_t* countp, long long* ip)

- 以数组方式读取unsigned long long类型数据。

  int nc_get_vara_ulonglong(int ncid, int varid, const size_t* startp, const size_t* countp, unsigned long long* ip)

参数

参数名	描述	取值范围	输入/输出
ncid	文件ID	非负整数	输入
varid	variable ID	非负整数	输入
*startp	访问数据的起始向量	非空	输入
*countp	访问数据的范围向量	非空	输入
*ip	数据指针	非空	输出

返回值

- 成功：返回NC_NOERR。
- 失败：返回错误码。

错误码

错误码	描述
NC_EBADID	无效ncid
NC_ENOTVAR	未找到variable
NC_EINVALCOORDS	无效坐标参数
NC_EEDGE	超出维度界限

版权所有 © 华为技术有限公司

注意

用户需要保证指针内存大小正确性，中间件无法检查指针大小。

7.2.2.2.11 其他函数

函数定义

MPI终止函数

int **PMPI_Finalize()**

int **MPI_Finalize()**

返回值

- 成功：返回NC_NOERR。
- 失败：返回错误码。

错误码

错误码	描述
MPI_ERR_OTHER	其他类型的错误

7.2.2.3 PnetCDF 桥接函数

7.2.2.3.1 函数说明

该部分包含PnetCDF桥接接口49个，主要涉及PnetCDF经典文件格式相关接口。

7.2.2.3.2 文件操作

函数定义

- 以聚合模式创建文件。

  int ncmpi_create(MPI_Comm comm, const char* name, int mode, MPI_Info info, int* idPoint);
- 读取文件。

  int ncmpi_open(MPI_Comm comm, const char* name, int mode, MPI_Info info, int* idPoint);
- 关闭文件。

  int ncmpi_close(int fileId);
- 退出定义模式。

  int ncmpi_enddef(int fileId);
- 进入独立数据模式。

  int ncmpi_begin_indep_data(int fileId);
- 退出独立数据模式。

 版权所有 © 华为技术有限公司

int ncmpi_end_indep_data(int fileId);

- 依据错误码打印错误信息。

  const char* ncmpi_strerror(int err);

参数

参数名	描述	取值范围	输入/输出
fileId	文件ID	非负整数	输入
*name	路径名称	非空，长度小于256字节	输入
mode	访问模式	非空	输入
info	MPI信息	非空	输入
*idPoint	返回文件ID的指针	非空	输出
err	错误码	整数	输入

返回值

- 成功：返回NC_NOERR。
- 失败：返回错误码。

错误码

错误码	描述
NC_EBADID	无效fileId
NC_EBAD_FILE	无效的文件名
NC_ENOMEM	空指针或无效内存

7.2.2.3.3 信息设置

函数定义

- 定义维度。

  int ncmpi_def_dim(int fileId, const char* name, MPI_Offset length, int* idPoint);

- 定义变量。

  int ncmpi_def_var(int fileId, const char* name, int dataType, int dimNumber, const int* dimsArray, int* idPoint);

 版权所有 © 华为技术有限公司

参数

参数名	描述	取值范围	输入/输出
fileId	文件ID	非负整数	输入
*name	维度名称/variable名称	非空，长度小于256字节	输入
length	维度长度	非负整数	输入
*idPoint	目标地址，存放维度id句柄	非空	输入
dataType	variable的数据类型	非空	输入
*dimsArray	维度ID数组	非空	输入
dimNumber	维度数量	非负整数	输入
*idPoint	返回variable ID的指针	非空	输出

返回值

- 成功：返回NC_NOERR。
- 失败：返回错误码。

错误码

错误码	描述
NC_EBADID	无效ncid
NC_EBAD_FILE	无效的文件名
NC_ENOMEM	空指针或无效内存

7.2.2.3.4 信息查询

7.2.2.3.4.1 查询文件和维度信息

函数定义

- 查询信息。
  int ncmpi_inq(int fileId, int* nDims, int* nVars, int* nGAtts, int* unlimDimId);
  int ncmpi_inq_version(int fileId, int* ncMode);
  const char* ncmpi_inq_libvers(void)
- 查询维度信息。
  int ncmpi_inq_dim(int fileId, int dimId, char* name, MPI_Offset* length);
  int ncmpi_inq_dimlen(int fileId, int dimId, MPI_Offset* lenPoint);

 版权所有 © 华为技术有限公司

```
int ncmpi_inq_dimid(int fileId, const char* name, int* idPoint);
```

参数

参数名	描述	取值范围	输入/输出
fileId	文件ID	非负整数	输入
*nDims	返回dim数量的指针	非空	输出
*nVars	返回variable数量的指针	非空	输出
*nGAtts	返回att数量的指针	非空	输出
*unlimDimId	返回unlimited dim的ID	非空	输出
*ncMode	返回format version的指针	非空	输出
dimId	维度的ID	非负整数	输入
*name	返回维度名称的指针	非空	输出/输入
*length	返回维度长度的指针	非空	输出
*lenPoint	返回维度长度的指针	非空	输出
*idPoint	返回维度ID的指针	非空	输出

返回值

- 成功：返回NC_NOERR。
- 失败：返回错误码。

错误码

错误码	描述
NC_EBADID	无效ncid
NC_EBADDIM	无效dimid
NC_ENOMEM	空指针或无效内存

注意

*nDims、*nVars、*nGAtts、*unlimDimId、*ncMode、*name、*length、*lenPoint、*idPoint参数在不需要获取数据时可以为空指针，需要获取数据时必须非空。

版权所有 © 华为技术有限公司

7.2.2.3.4.2 查询变量和属性信息

函数定义

- 查询变量信息。

  int ncmpi_inq_varid(int fileId, const char* name, int* idPoint);

  int ncmpi_inq_var(int fileId, int variableId, char* name, int* ncDataType, int* dimNumber, int* dimsArray, int* nattsp);

  int ncmpi_inq_varndims(int fileId, int variableId, int* dimNumber);

- 查询属性信息。

  int ncmpi_inq_att(int fileId, int varId, const char* name, int* xtypep, MPI_Offset* lenp);

  int ncmpi_inq_attname(int fileId, int varId, int attNum, char* name);

参数

参数名	描述	取值范围	输入/输出
fileId	文件ID	非负整数	输入
*idPoint	返回varid的指针	非空	输出
variableId/ varId	varid	非负整数	输入
const *name	var名称/attribute名称	非空，长度小于256字节	输入
*name	var名称/attribute名称	非空	输出
*ncDataType	返回variable数据类型的指针	非空	输出
*dimNumber	返回维度数量的指针	非空	输出
*dimsArray	返回维度ID数组的指针	非空	输出
*nattsp	返回属性数量的指针	非空	输出
*xtypep	返回att类型的指针	非空	输出
*lenp	返回att数量的指针	非空	输出
attNum	att ID	非负整数	输入

返回值

- 成功：返回NC_NOERR。
- 失败：返回错误码。

版权所有 © 华为技术有限公司

错误码

错误码	描述
NC_EBADID	无效ncid
NC_EBADDIM	无效dimid
NC_ENOMEM	空指针或无效内存
NC_EBADNAME	无效名称

注意

*idPoint、*name、*ncDataType、*dimNumber、*dimsArray、*nattsp、*xtypep、*lenp参数在不需要获取数据时可以为空指针，在需要获取数据时必须非空。

7.2.2.3.5 属性信息写入

函数定义

- char类型属性信息写入。

  int ncmpi_put_att_text(int fileId, int varId, const char* name, MPI_Offset nelems, const char* buf);
- int类型属性信息写入。

  int ncmpi_put_att_int(int fileId, int varId, const char* name, int xType, MPI_Offset nelems, const int* buf);
- float类型属性信息写入。

  int ncmpi_put_att_float(int fileId, int varId, const char* name, int xType, MPI_Offset nelems, const float* buf);
- double类型属性信息写入。

  int ncmpi_put_att_double(int fileId, int varId, const char* name, int xType, MPI_Offset nelems, const double* buf);

参数

参数名	描述	取值范围	输入/输出
fileId	文件ID	非负整数	输入
varId	variable ID	非负整数	输入
*name	属性名称	非空，长度小于256字节	输入
nelems	写入信息的长度	非负整数	输入
*buf	写入信息的内存的指针	非空	输入
xtype	写入属性的数据类型	非负整数	输入

版权所有 © 华为技术有限公司

返回值

- 成功：返回NC_NOERR。
- 失败：返回错误码。

错误码

错误码	描述
NC_EBADID	无效ncid
NC_EBADNAME	无效名称
NC_ENOMEM	空指针或无效内存
NC_EINVAL	无效的参数

7.2.2.3.6 属性信息读取

函数定义

- 任意类型属性信息读取。
  int ncmpi_get_att(int fileId, int varid, const char* name, void* value);
- char类型属性信息读取。
  int ncmpi_get_att_text(int fileId, int varid, const char* name, char* ip);
- int类型属性信息读取。
  int ncmpi_get_att_int(int fileId, int varid, const char* name, int* ip);
- float类型属性信息读取。
  int ncmpi_get_att_float(int fileId, int varid, const char* name, float* ip);
- double类型属性信息读取。
  int ncmpi_get_att_double(int fileId, int varid, const char* name, double* ip);

参数

参数名	描述	取值范围	输入/输出
fileId	文件ID	非负整数	输入
varid	variable ID	非负整数	输入
*name	att名称	非空，长度小于256字节	输入
*ip	读取属性信息的内存的指针	非空	输出

版权所有 © 华为技术有限公司

返回值

- 成功：返回NC_NOERR。
- 失败：返回错误码。

错误码

错误码	描述
NC_EBADID	无效ncid
NC_EBADNAME	无效名称
NC_ENOMEM	空指针或无效内存
NC_EINVAL	无效的参数

7.2.2.3.7 单元素模式数据写入

函数定义

- 任意类型单元素模式数据写入。

  int ncmpi_put_var1(int fileId, int variableId, const MPI_Offset* start, const void* data, MPI_Offset bufcount, MPI_Datatype buftype);

- double类型单元素模式数据写入。

  int ncmpi_put_var1_double(int fileId, int variableId, const MPI_Offset* start, const double* data);

参数

参数名	描述	取值范围	输入/输出
fileId	文件ID	非负整数	输入
varId	variable ID	非负整数	输入
*start	写入起始位置偏移	非空	输入
*data	指向写入数据的指针	非空	输入
bufcount	写入大小	非负整数	输入
buftype	写入类型	非空	输入

返回值

- 成功：返回NC_NOERR。
- 失败：返回错误码。

 版权所有 © 华为技术有限公司

错误码

错误码	描述
NC_EBADID	无效ncid
NC_ENOTINDEP	在集合数据模式下尝试独立I/O操作
NC_EINVALCOORDS	指定的索引超出了指定变量的秩的范围
NC_ENOTVAR	无效的varid

7.2.2.3.8 stride 模式数据写入

函数定义

- 任意类型集合通信写入函数，stride模式。

  int ncmpi_put_vars_all(int fileId, int variableId, const MPI_Offset* start, const MPI_Offset* count, const MPI_Offset* stride, const void *data, MPI_Offset bufCount, MPI_Datatype bufType)

- double类型集合通信写入函数，stride模式。

  int ncmpi_put_vars_double_all(int fileId, int variableId, const MPI_Offset* start, const MPI_Offset* count, const MPI_Offset* stride, const double *data)

参数

参数名	描述	取值范围	输入/输出
fileId	文件ID	非负整数	输入
variableId	variable ID	非负整数	输入
*start	写入起始位置偏移	非空	输入
*count	写入大小	非空	输入
*stride	数据间隔大小	非空	输入
*data	指向写入数据的指针	非空	输入
bufCount	写入数据大小	非负整数	输入
bufType	写入类型	非空	输入

返回值

- 成功：返回NC_NOERR。
- 失败：返回错误码。

版权所有 © 华为技术有限公司

错误码

错误码	描述
NC_EBADID	无效ncid
NC_ENOTINDEP	在集合数据模式下尝试独立I/O操作
NC_EINVALCOORDS	指定的索引超出了指定变量的秩的范围
NC_ENOTVAR	无效的varid

7.2.2.3.9 array 模式数据写入

函数定义

- 任意类型array模式数据写入。

  int ncmpi_put_vara(int fileId, int variableId, const MPI_Offset* start, const MPI_Offset* count, const void *data, MPI_Offset bufCount, MPI_Datatype buftype)
- char类型array模式数据写入。

  int ncmpi_put_vara_text(int fileId, int variableId, const MPI_Offset* start, const MPI_Offset* count, const char *buf)
- double类型array模式数据写入。

  int ncmpi_put_vara_double(int fileId, int varid, const MPI_Offset* start, const MPI_Offset* count, const double *op)
- int类型array模式数据写入。

  int ncmpi_put_vara_int(int fileId, int varid, const MPI_Offset* start, const MPI_Offset* count, const int *op)
- long类型array模式数据写入。

  int ncmpi_put_vara_long(int fileId, int varid, const MPI_Offset* start, const MPI_Offset* count, const long *op)
- float类型array模式数据写入。

  int ncmpi_put_vara_float(int fileId, int varid, const MPI_Offset* start, const MPI_Offset* count, const float *op)
- array模式集合通信数据写入。

  int ncmpi_put_vara_all(int fileId, int variableId, const MPI_Offset* start, const MPI_Offset* count, const void *data, MPI_Offset bufcount, MPI_Datatype buftype)
- char类型array模式集合通信数据写入。

  int ncmpi_put_vara_text_all(int fileId, int varid, const MPI_Offset* start, const MPI_Offset* count, const char *op)
- int类型array模式集合通信数据写入。

  int ncmpi_put_vara_int_all(int fileId, int variableId, const MPI_Offset* start, const MPI_Offset* count, const int *data)
- long类型array模式集合通信数据写入。

版权所有 © 华为技术有限公司

int ncmpi_put_vara_long_all(int fileId, int variableId, const MPI_Offset* start, const MPI_Offset* count, const long *data)

- float类型array模式集合通信数据写入。

  int ncmpi_put_vara_float_all(int fileId, int varid, const MPI_Offset* start, const MPI_Offset* count, const float *op)

参数

参数名	描述	取值范围	输入/输出
fileId	文件ID	非负整数	输入
varid/ variableId	variable ID	非负整数	输入
*start	写入起始位置偏移	非空	输入
*count	写入大小	非空	输入
*buf	指向写入数据的指针	非空	输入
*data	指向写入数据的指针	非空	输入
*op	指向写入数据的指针	非空	输入
bufcount/ bufCount	写入数据数量	非负整数	输入
buftype	写入类型	非空	输入

返回值

- 成功：返回NC_NOERR。
- 失败：返回错误码。

错误码

错误码	描述
NC_EBADID	无效ncid
NC_ENOTINDEP	在集合数据模式下尝试独立I/O操作
NC_EINVALCOORDS	指定的索引超出了指定变量的秩的范围
NC_ENOTVAR	无效的varid

版权所有 © 华为技术有限公司

7.2.2.3.10 array 模式数据读取

函数定义

任意类型array模式数据读取。

int ncmpi_get_vara_all(int fileId, int variableId, const MPI_Offset* start, const MPI_Offset* count, void *data, MPI_Offset bufcount, MPI_Datatype bufType)

参数

参数名	描述	取值范围	输入/输出
fileId	文件ID	非负整数	输入
variableId	variable ID	非负整数	输入
*start	读取起始位置偏移	非空	输入
*count	读取大小	非空	输入
*data	指向读取数据的指针	非空	输出
bufcount	指向读取数据大小的指针	非负整数	输出
bufType	指向读取数据类型的指针	非空	输出

返回值

- 成功：返回NC_NOERR。
- 失败：返回错误码。

错误码

错误码	描述
NC_EBADID	无效ncid
NC_ENOTINDEP	在集合数据模式下尝试独立I/O操作
NC_EINVALCOORDS	指定的索引超出了指定变量的秩的范围
NC_ENOTVAR	无效的varid

7.2.2.3.11 全变量模式数据写入

函数定义

char类型全变量模式数据写入。

int ncmpi_put_var_text(int fileId, int varId, const char *buf)

版权所有 © 华为技术有限公司

参数

参数名	描述	取值范围	输入/输出
fileId	文件ID	非负整数	输入
varId	variable ID	非负整数	输入
*buf	指向写入数据的指针	非空	输入

返回值

- 成功：返回NC_NOERR。
- 失败：返回错误码。

错误码

错误码	描述
NC_EBADID	无效ncid
NC_ENOTINDEP	在集合数据模式下尝试独立I/O操作
NC_EINVALCOORDS	指定的索引超出了指定变量的秩的范围
NC_ENOTVAR	无效的varid

7.2.2.3.12 全变量模式数据读取

函数定义

char类型全变量模式数据读取。

int ncmpi_get_var_text_all(int fileId, int varId, char *buf)

参数

参数名	描述	取值范围	输入/输出
fileId	文件ID	非负整数	输入
varId	variable ID	非负整数	输入
*buf	指向读取数据的指针	非空	输出

返回值

- 成功：返回NC_NOERR。

版权所有 © 华为技术有限公司

- 失败：返回错误码。

错误码

错误码	描述
NC_EBADID	无效ncid
NC_ENOTINDEP	在集合数据模式下尝试独立I/O操作
NC_EINVALCOORDS	指定的索引超出了指定变量的秩的范围
NC_ENOTVAR	无效的varid

7.2.2.3.13 其他函数

函数定义

MPI终止函数

int MPI_Finalize()

int PMPI_Finalize()

返回值

- 成功：返回NC_NOERR。
- 失败：返回错误码。

错误码

错误码	描述
MPI_ERR_OTHER	其他类型的错误

7.2.2.4 HDF5 桥接函数

7.2.2.4.1 函数说明

该部分包含HDF5桥接接口46个。

7.2.2.4.2 文件接口

7.2.2.4.2.1 创建文件

函数定义

创建HDF5文件。

 版权所有 © 华为技术有限公司

hid_t H5Fcreate(const char *name, unsigned flags, hid_t createPlistId, hid_t accessPlistId)

参数

参数名	描述	取值范围	输入/输出
*name	要创建的文件的名称。	非空，长度小于256字节	输入
flags	文件访问标志。	无符号整数	输入
createPlistId	文件创建属性列表标识符。	整数	输入
accessPlistId	文件访问属性列表标识符。	整数	输入

返回值

- 成功：返回文件标识符。
- 失败：返回-1。

注意

当flags包含H5F_ACC_TRUNC时，IO Middleware不支持Hyper IO文件聚合功能。

7.2.2.4.2.2 打开文件

函数定义

打开现有的HDF5文件。

hid_t H5Fopen(const char *name, unsigned flags, hid_t accessPlistId)

参数

参数名	描述	取值范围	输入/输出
*name	要打开的文件的名称。	非空，长度小于256字节	输入
flags	文件访问标志。	无符号整数	输入
accessPlistId	文件访问属性列表标识符。	整数	输入

返回值

- 成功：返回文件标识符。
- 失败：返回-1。

 版权所有 © 华为技术有限公司

7.2.2.4.2.3 关闭文件

函数定义

终止对HDF5文件的访问。

herr_t H5Fclose(hid_t fileId)

参数

参数名	描述	取值范围	输入/输出
fileId	文件标识符。	整数	输入

返回值

- 成功：返回非负值。
- 失败：返回负值。

7.2.2.4.2.4 刷新缓存区

函数定义

将与文件关联的所有缓冲区刷新到磁盘。

herr_t H5Fflush(hid_t object_id, H5F_scope_t scope)

参数

参数名	描述	取值范围	输入/输出
object_id	用于标识文件的对象标识符。	整数	输入
scope	指定刷新动作的范围。	非空	输入

返回值

返回0。

7.2.2.4.3 属性接口

7.2.2.4.3.1 创建属性

函数定义

创建附加到指定对象的属性。

版权所有 © 华为技术有限公司

hid_t H5Acreate2(hid_t loc, const char *name, hid_t datatypeId, hid_t spaceId, hid_t createPlistId, hid_t accessPlistId)

参数

参数名	描述	取值范围	输入/输出
loc	位置标识符。标识符可以是文件、组、数据集或命名数据类型的标识符。	整数	输入
*name	属性名称。	非空，长度小于256字节	输入
datatypeId	属性数据类型标识符。	整数	输入
spaceId	数据空间标识符。	整数	输入
createPlistId	属性创建属性列表标识符。	整数，参数acpl_id目前未使用，指定H5P_DEFAULT	输入
accessPlistId	属性的访问列表标识符。	整数，参数aapl_id目前未使用，指定H5P_DEFAULT	输入

返回值

- 成功：返回属性标识符。
- 失败：返回负值。

7.2.2.4.3.2 写入属性

函数定义

将数据写入属性。

herr_t H5Awrite(hid_t attributeId, hid_t datatypeId, const void *buffer)

参数

参数名	描述	取值范围	输入/输出
attributeId	要写入的属性的标识符。	整数	输入
datatypeId	属性数据类型的标识符（在内存中）。	整数	输入

版权所有 © 华为技术有限公司

参数名	描述	取值范围	输入/输出
*buffer	要写入的数据。	非空	输入

返回值

- 成功：返回非负值。
- 失败：返回负值。

7.2.2.4.3.3 读取属性

函数定义

读取属性。

herr_t H5Aread(hid_t attributeId, hid_t datatypeId, void *buffer)

参数

参数名	描述	取值范围	输入/输出
attributeId	要读取的属性的标识符。	整数	输入
datatypeId	属性数据类型的标识符（在内存中）。	整数	输入
*buffer	读取数据的缓冲区。	非空	输出

返回值

- 成功：返回非负值。
- 失败：返回负值。

7.2.2.4.3.4 打开指定属性

函数定义

打开由对象标识符和属性名称指定的对象的属性。

hid_t H5Aopen(hid_t loc, const char * name, hid_t accessPlistId)

版权所有 © 华为技术有限公司

参数

参数名	描述	取值范围	输入/输出
loc	位置标识符。标识符可以是文件、组、数据集或命名数据类型的标识符。	整数	输入
*name	要打开的属性的名称。	非空	输入
accessPlistId	属性访问属性列表标识符。	整数	输入

返回值

- 成功：返回属性标识符。
- 失败：返回负值。

7.2.2.4.3.5 关闭指定属性

函数定义

关闭指定属性。

herr_t H5Aclose(hid_t id)

参数

参数名	描述	取值范围	输入/输出
id	要释放的属性标识符。	整数	输入

返回值

返回0。

7.2.2.4.3.6 属性是否存在

函数定义

确定对象上是否存在具有给定名称的属性。

htri_t H5Aexists_by_name(hid_t locId, const char * objName, const char * attrName, hid_t lapl)

版权所有 © 华为技术有限公司

参数

参数名	描述	取值范围	输入/输出
locId	位置标识符。标识符可以是文件、组、数据集或命名数据类型的标识符。	整数	输入
*objName	对象名称。	非空	输入
*attrName	属性名称。	非空	输入
lapl	链接访问属性列表标识符。	整数	输入

返回值

返回0（假）、正（真）或负（失败）值。

7.2.2.4.4 数据集接口

7.2.2.4.4.1 创建新数据集

函数定义

在指定位置创建数据集。

hid_t H5Dcreate2(hid_t parentId, const char *name, hid_t datatypeId, hid_t spaceId, hid_t linkCreatePlistId, hid_t createPlistId, hid_t accessPlistId)

参数

参数名	描述	取值范围	输入/输出
parentId	位置标识符；可以是文件、组、数据集、命名数据类型或属性。	整数	输入
*name	数据集名称。	非空，长度小于256字节	输入
datatypeId	数据类型标识符。	整数	输入
spaceId	数据空间标识符。	整数	输入
linkCreatePlistId	链路创建属性列表。	整数，参数lcpl_id目前未使用，指定H5P_DEFAULT	输入

版权所有 © 华为技术有限公司

参数名	描述	取值范围	输入/输出
createPlistId	数据集创建属性列表标识符。	整数，参数dcpl_id目前未使用，指定H5P_DEFAULT	输入
accessPlistId	数据集访问属性列表。	整数，参数dapl_id目前未使用，指定H5P_DEFAULT	输入

返回值

- 成功：返回数据集标识符。
- 失败：返回负值。

7.2.2.4.4.2 打开数据集

函数定义

打开已存在的数据集。

hid_t H5Dopen2(hid_t locId, const char *name, hid_t accessPlistId)

参数

参数名	描述	取值范围	输入/输出
locId	位置标识符；可以是文件、组、命名数据类型、数据集或属性。	整数	输入
*name	数据集名称。	非空，长度小于256字节	输入
accessPlistId	数据集访问属性列表。	整数，参数dapl_id目前未使用，均指定H5P_DEFAULT	输入

返回值

- 成功：返回数据集标识符。
- 失败：返回负值。

版权所有 © 华为技术有限公司

7.2.2.4.4.3 关闭数据集

函数定义

关闭指定数据集。

herr_t H5Dclose(hid_t datasetId)

参数

参数名	描述	取值范围	输入/输出
datasetId	数据集标识符。	整数	输入

返回值

- 成功：返回非负值。
- 失败：返回负值。

7.2.2.4.4.4 获取数据集的数据空间

函数定义

获取指定数据集的数据空间标识符。

hid_t H5Dget_space(hid_t datasetId)

参数

参数名	描述	取值范围	输入/输出
datasetId	要查询的数据集的标识符。	整数	输入

返回值

- 成功：返回数据空间标识符。
- 失败：返回负值。

7.2.2.4.4.5 写入数据集

函数定义

将原始数据从缓冲区写入数据集。

herr_t H5Dwrite(hid_t datasetId, hid_t datatypeId, hid_t memorySpaceId, hid_t fileSpaceId, hid_t xferPropertylistId, const void * buffer)

 版权所有 © 华为技术有限公司

参数

参数名	描述	取值范围	输入/输出
datasetId	要写入的数据集的标识符。	整数	输入
datatypeId	内存数据类型的标识符。	整数	输入
memorySpaceId	内存数据空间的标识符。	整数	输入
fileSpaceId	文件中数据集数据空间的标识符。	整数	输入
xferPropertylistId	此I/O操作的传输属性列表的标识符。	整数	输入
*buffer	缓冲区，其中包含要写入文件的数据。	非空	输入

返回值

- 成功：返回非负值。
- 失败：返回负值。

7.2.2.4.4.6 读取数据集

函数定义

将原始数据从数据集中读取到缓冲区。

herr_t H5Dread(hid_t datasetId, hid_t datatypeId, hid_t memorySpaceId, hid_t fileSpaceId, hid_t xferPropertylistId, void *buffer)

参数

参数名	描述	取值范围	输入/输出
datasetId	从中读取的数据集的标识符。	整数	输入
datatypeId	内存数据类型的标识符。	整数	输入
memorySpaceId	内存数据空间的标识符。	整数	输入
fileSpaceId	文件中数据集数据空间的标识符。	整数	输入
xferPropertylistId	此I/O操作的传输属性列表的标识符。	整数	输入
*buffer	用于接收从文件读取的数据的缓冲区。	非空	输出

版权所有 © 华为技术有限公司

返回值

- 成功：返回非负值。
- 失败：返回负值。

7.2.2.4.4.7 更改数据集维度

函数定义

更改数据集维度的大小。

herr_t H5Dset_extent(hid_t datasetId, const hsize_t dims[])

参数

参数名	描述	取值范围	输入/输出
datasetId	数据集标识符。	整数	输入
dims[]	包含数据集每个维度的新幅度的数组。	非空	输入

返回值

- 成功：返回非负值。
- 失败：返回负值。

7.2.2.4.5 群组接口

7.2.2.4.5.1 创建群组

函数定义

创建一个新的空组并为其指定名称。

hid_t H5Gcreate2(hid_t parentId, const char *name, hid_t linkCreatePlistId, hid_t createPlistId, hid_t accessPlistId)

参数

参数名	描述	取值范围	输入/输出
parentId	位置标识符；可以是文件、组、数据集、命名数据类型或属性。	整数	输入
*name	指向新组的链接的绝对或相对名称。	非空，长度小于256字节	输入

版权所有 © 华为技术有限公司

参数名	描述	取值范围	输入/输出
linkCreatePlistId	链路创建属性列表标识符。	整数，参数lcpl_id目前未使用，指定H5P_DEFAULT	输入
createPlistId	群组创建属性列表标识符。	整数，参数gcpl_id目前未使用，指定H5P_DEFAULT	输入
accessPlistId	群组访问属性列表标识符。	整数，参数gapl_id目前未使用，指定H5P_DEFAULT	输入

返回值

- 成功：返回群组标识符。
- 失败：返回负值。

7.2.2.4.5.2 打开群组

函数定义

打开已存在的群组。

hid_t H5Gopen2(hid_t parentId, const char * name, hid_t accessPlistId)

参数

参数名	描述	取值范围	输入/输出
parentId	文件、组、数据集、命名数据类型或属性标识符，指定要打开的组的位置。	整数	输入
*name	要打开的群组名称。	非空，长度小于256字节	输入
accessPlistId	组访问属性列表标识符。	整数	输入

版权所有 © 华为技术有限公司

返回值

- 成功：返回群组标识符。
- 失败：返回负值。

7.2.2.4.5.3 关闭群组

函数定义

关闭指定的群组。

herr_t H5Gclose(hid_t groupId)

参数

参数名	描述	取值范围	输入/输出
groupId	释放的群组标识符。	整数	输入

返回值

- 成功：返回非负值。
- 失败：返回负值。

7.2.2.4.6 属性列表接口

7.2.2.4.6.1 创建属性列表

函数定义

创建新属性作为属性列表类的实例。

hid_t H5Pcreate(H5P_class_t classId)

参数

参数名	描述	取值范围	输入/输出
classId	要创建的属性列表的类型。	非空	输入

返回值

- 成功：属性列表标识符。
- 失败：返回负值。

版权所有 © 华为技术有限公司

7.2.2.4.6.2 终止属性列表访问

函数定义

终止对属性列表的访问。

herr_t H5Pclose(hid_t propertylistId)

参数

参数名	描述	取值范围	输入/输出
propertylistId	要终止访问的属性列表的标识符。	整数	输入

返回值

- 成功：返回非负值。
- 失败：返回负值。

7.2.2.4.6.3 通信器存储到属性列表

函数定义

将MPI-IO通信器信息存储到文件访问属性列表中。

herr_t H5Pset_fapl_mpio(hid_t propertylistId, MPI_Comm comm, MPI_Info info)

参数

参数名	描述	取值范围	输入/输出
propertylistId	文件访问属性列表标识符。	整数	输入
comm	MPI-2通信器。	非空	输入
info	MPI-2信息对象。	非空	输入

返回值

- 成功：返回非负值。
- 失败：返回负值。

7.2.2.4.6.4 设置数据传输

函数定义

设置数据传输模式。

herr_t H5Pset_dxpl_mpio(hid_t propertylistId, H5FD_mpio_xfer_t xferMode)

版权所有 © 华为技术有限公司

参数

参数名	描述	取值范围	输入/输出
propertylistId	数据传输属性列表标识符。	整数	输入
xferMode	传输模式。	非空	输入

返回值

- 成功：返回非负值。
- 失败：返回负值。

7.2.2.4.6.5 设置分块大小

函数定义

设置数据分块大小。

herr_t H5Pset_chunk(hid_t propertylistId, int ndims, const hsize_t dim[])

参数

参数名	描述	取值范围	输入/输出
propertylistId	要查询的属性列表的标识符。	整数	输入
ndims	每个块的维数。	整数	输入
dim[]	包含每个块大小的数组。	非空	输入

返回值

返回0。

7.2.2.4.6.6 设置元数据 I/O 模式

函数定义

将读取操作的元数据I/O模式设置为集体或独立（默认）。

herr_t H5Pset_all_coll_metadata_ops(hid_t propertylistId, hbool_t collective)

版权所有 © 华为技术有限公司

参数

参数名	描述	取值范围	输入/输出
propertylistId	文件、组、数据集、数据类型、链接或属性访问属性列表标识符。	整数	输入
collective	布尔值，指示元数据读取是集体（1）还是独立（0）。默认模式：独立（0）。	布尔值	输入

返回值

返回零。

7.2.2.4.6.7 设置元数据写入模式

函数定义

将元数据写入模式设置为集体或独立（默认）。

herr_t H5Pset_coll_metadata_write(hid_t propertylistId, hbool_t collective)

参数

参数名	描述	取值范围	输入/输出
propertylistId	文件访问属性列表标识符。	整数	输入
collective	布尔值，指示元数据写入是集体（>0）还是独立（0）。默认模式：独立（0）。	布尔值	输出

返回值

返回零。

7.2.2.4.6.8 元数据缓存配置

函数定义

将指示的文件访问属性列表中的初始元数据缓存配置设置为提供的值。

herr_t H5Pset_mdc_config(hid_t propertylistId, H5AC_cache_config_t *config)

版权所有 © 华为技术有限公司

参数

参数名	描述	取值范围	输入/输出
propertylistId	文件访问属性列表标识符。	整数	输入
*config	指针，指向包含所需配置的H5AC_cache_config_t实例。	非空	输入

返回值

返回零。

7.2.2.4.6.9 获取元数据缓存配置

函数定义

从提供的文件访问属性列表中获取当前初始元数据缓存配置。

herr_t H5Pget_mdc_config(hid_t propertylistId, H5AC_cache_config_t *config)

参数

参数名	描述	取值范围	输入/输出
propertylistId	要查询的属性列表的标识符。	整数	输入
*config	指针，指向要报告当前元数据缓存配置的H5AC_cache_config_t实例。	整数	输出

返回值

返回零。

7.2.2.4.7 数据空间接口

7.2.2.4.7.1 创建数据空间

函数定义

创建一个新的简单数据空间并打开它以进行访问。

hid_t H5Screate_simple(int rank, const hsize_t dims[], const hsize_t maxDims[])

版权所有 © 华为技术有限公司

参数

参数名	描述	取值范围	输入/输出
rank	数据空间的维度数。	整数	输入
dims[]	每个维度大小的数组。	非空	输入
maxDims[]	每个维度的最大大小的数组。	非空	输入

返回值

- 成功：返回数据空间标识符。
- 失败：返回负值。

7.2.2.4.7.2 关闭数据空间

函数定义

释放和终止对数据空间的访问。

herr_t H5Sclose(hid_t spaceId)

参数

参数名	描述	取值范围	输入/输出
spaceId	要释放的数据空间的标识符。	整数	输入

返回值

- 成功：返回非负值。
- 失败：返回负值。

7.2.2.4.7.3 获取数据元素切片

函数定义

选择要添加到当前选定区域的超级板区域。

herr_t H5Sselect_hyperslab(hid_t spaceId, H5S_seloper_t operate, const hssize_t start[], const hsize_t stride[], const hsize_t count[], const hsize_t block[])

版权所有 © 华为技术有限公司

参数

参数名	描述	取值范围	输入/输出
spaceId	要修改的数据空间选择的标识符。	整数	输入
operate	要对当前选择执行的操作。	非空	输入
start[]	hyperslab开始偏移量。	非空	输入
stride[]	hyperslab步长。	非空	输入
count[]	hyperslab中包含的块数。	非空	输入
block[]	hyperslab中的块大小。	非空	输入

返回值

- 成功：返回非负值。
- 失败：返回负值。

7.2.2.4.7.4 创建指定类型数据空间

函数定义

创建指定类型的新数据空间。

hid_t H5Screate(H5S_class_t type)

参数

参数名	描述	取值范围	输入/输出
type	要创建的数据空间类型。	非空	输入

返回值

- 成功：返回数据空间标识符。
- 失败：返回负值。

7.2.2.4.7.5 确定数据空间维度

函数定义

确定数据空间的维度。

int H5Sget_simple_extent_ndims(hid_t spaceId)

版权所有 © 华为技术有限公司

参数

参数名	描述	取值范围	输入/输出
spaceId	数据空间标识。	整数	输入

返回值

成功：返回数据空间中的维数。

失败：返回负值。

7.2.2.4.7.6 检索数据空间维度

函数定义

检索数据空间维度大小和最大大小。

int H5Sget_simple_extent_dims(hid_t spaceId, hsize_t dims[], hsize_t maxDims[])

参数

参数名	描述	取值范围	输入/输出
spaceId	要查询的数据空间对象的标识符。	非空	输入
dims[]	指向数组的指针，以存储每个维度的大小。	非空	输出
maxDims[]	指向数组的指针，用于存储每个维度的最大大小。	非空	输出

返回值

- 成功：返回数据空间中的维数。
- 失败：返回负值。

注意

dims[]、maxDims[]参数在不需要获取数据时可以为空指针，需要获取数据时必须非空。

7.2.2.4.8 数据类型接口

版权所有 © 华为技术有限公司

7.2.2.4.8.1 创建数组数据类型

函数定义

创建一个新的数组对象。

hid_t H5Tarray_create2(hid_t base, unsigned ndims, const hsize_t dim[])

参数

参数名	描述	取值范围	输入/输出
base	数组基础数据类型的数据类型标识符。	整数	输入
ndims	数组的维度数。	无符号整数	输入
dim[]	每个数组维度的大小。	非空	输入

返回值

- 成功：返回数据类型标识符。
- 失败：返回负值。

7.2.2.4.8.2 复制现有数据类型

函数定义

复制一个已存在的数据类型。

hid_t H5Tcopy(hid_t datatype)

参数

参数名	描述	取值范围	输入/输出
datatype	要复制的数据类型的标识符。可以是数据类型标识符、预定义数据类型（在H5Tpublic.h中定义）或数据集标识符。	整数	输入

返回值

- 成功：返回数据类型标识符。
- 失败：返回负值。

版权所有 © 华为技术有限公司

7.2.2.4.8.3 设置数据类型大小

函数定义

设置原子数据类型的总大小。

herr_t H5Tset_size(hid_t datatypeId, size_t size)

参数

参数名	描述	取值范围	输入/输出
datatypeId	要更改大小的数据类型标识符。	整数	输入
size	要修改数据类型的大小（以字节为单位）。	无符号整数	输入

返回值

- 成功：返回非负值。
- 失败：返回负值。

7.2.2.4.8.4 获取数据类型大小

函数定义

获取原子数据类型的总大小。

size_t H5Tget_size(hid_t datatypeId)

参数

参数名	描述	取值范围	输入/输出
datatypeId	数据类型标识符。	整数	输入

7.2.2.4.8.5 设置字符编码集

函数定义

设置字符编码集。

herr_t H5Tset_cset(hid_t datatype, H5T_cset_t cset)

版权所有 © 华为技术有限公司

参数

参数名	描述	取值范围	输入/输出
datatype	数据类型标识符。	整数	输入
cset	字符集类型。	H5T_CSET_ASCII/ H5T_CSET_UTF8	输入

7.2.2.4.8.6 设置数据类型精度

函数定义

设置数据类型精度。

herr_t H5Tset_precision(hid_t datatype, size_t prec)

参数

参数名	描述	取值范围	输入/输出
datatype	数据类型标识符。	整数	输入
prec	数据类型精度。	整数	输入

7.2.2.4.8.7 定义字符串填充类型

函数定义

定义字符串的填充类型。

herr_t H5Tset_strpad(hid_t datatype, H5T_str_t strpad)

参数

参数名	描述	取值范围	输入/输出
datatype	要修改的数据类型的标识符。	整数	输入
strpad	字符串填充类型。	非空	输入

返回值

- 成功：返回非负值。

 版权所有 © 华为技术有限公司

- 失败：返回负值。

7.2.2.4.9 链路接口

7.2.2.4.9.1 链路是否存在

函数定义

确定组中是否存在具有指定名称的链路。

htri_t H5Lexists(hid_t locId, const char * name, hid_t lapl_id)

参数

参数名	描述	取值范围	输入/输出
locId	位置标识符。标识符可以是文件、组、数据集、命名数据类型或属性的标识符。	整数	输入
*name	字符串填充类型。	非空	输入
lapl_id	链路访问属性列表标识符。	整数	输入

返回值

- 成功：返回非负值。
- 失败：返回负值。

7.2.2.4.9.2 链路创建

函数定义

创建指向对象的硬链接。

herr_t H5Lcreate_hard(hid_t curLoc, const char * curName, hid_t dstLoc, const char * dstName, hid_t lcpl, hid_t lapl)

参数

参数名	描述	取值范围	输入/输出
curLoc	位置标识符。标识符可以是文件、组、数据集、命名数据类型或属性的标识符。	整数	输入
*curName	目标对象的名称，必须已存在。	非空	输入

版权所有 © 华为技术有限公司

参数名	描述	取值范围	输入/输出
dstLoc	位置标识符。标识符可以是文件、组、数据集、命名数据类型或属性的标识符。	整数	输入
*dstName	新链接的名称。	非空	输入
lcpl	链接创建属性列表标识符。	整数	输入
lapl	链接访问属性列表标识符。	整数	输入

返回值

- 成功：返回非负值。
- 失败：返回负值。

7.2.2.4.10 其他函数

函数定义

MPI终止函数

int MPI_Finalize()

int PMPI_Finalize()

返回值

- 成功：返回MPI_SUCCESS。
- 失败：返回错误码。

错误码

错误码	描述
MPI_ERR_OTHER	其他类型的错误

7.2.3 Hyper IO 配置项参考

通过配置“/opt/IO_Middleware/IO_Middleware_2.6.1_aarch64/conf/xxx.json”文件，可为Hyper IO设定打印日志模块、数据压缩等级、数据压缩方法及中间件加速策略等选项。其中，“/opt/IO_Middleware/”为Hyper IO安装包路径，“/xxx.json”为Hyper IO安装包中为各个应用提供的样例配置文件。配置项说明如**表7-6**、**•（可选）如果需要使用文件聚合功能，需要配置数据...**和**•（可选）如果需要设置单个文件内容可乱序输出，需...**所示。

版权所有 © 华为技术有限公司

表 7-6 数据集整体信息（DATASET）

名称		描述	类型	缺省值
数据集整体通用信息（GENERAL）	LOG_FLOW	打印FLOW模块日志。	String ● ON：开启 ● OFF：关闭	OFF
	LOG_PROCESSOR	打印PROCESSOR模块日志。	String ● ON：开启 ● OFF：关闭	OFF
	LOG_BRIDGE	打印BRIDGE模块日志。	String ● ON：开启 ● OFF：关闭	OFF
	LOG_INTERCEPTOR	打印所有INTERCEPTOR模块日志。	String ● ON：开启 ● OFF：关闭	OFF
	LOG_CONFIG	打印CONFIG模块日志。	String ● ON：开启 ● OFF：关闭	OFF
	TRANSCACHESIZE	当数据总量达到Cache大小时，聚合成一次写操作。取值范围为[0，128]，单位MB，缺省值为0，表示不聚合。	Integer	0
	COMPRESSMETHOD	数据压缩方法。 取值范围为[1，2]，缺省值为1。	Integer ● 1：表示压缩算法为LZ4 ● 2：表示压缩算法为Bzip2	1

版权所有 © 华为技术有限公司

名称		描述	类型	缺省值
	COMPRESSLEVEL	数据压缩级别。 取值范围为[0，5]，缺省值为0，表示不压缩。 数据压缩级别越高，数据压缩比越高。	Integer	0
	WITHOUTGROUP	不配置group信息接入Hyper IO。	String ● ON：开启 ● OFF：关闭	OFF
数据集访问模式信息（DATA_ACCESS_PATTERN）	FIXED	应用中所有Entity的写操作在每个step间是否保持不变。	Boolean ● false ● true	false
	RANDOM	应对粒子型应用中不要求保证粒子顺序的场景。	Boolean ● false ● true	false
	WRITE_AGGREGATE_STEPS	元数据多少个step聚合成一次写，避免IO出现频率高且数据量小的情况。 取值范围为[1，64]。	Integer	1
	WRITE_AGGREGATE_RANKS	元数据多少个rank聚合成一个写。 取值范围为[1，64]。	Integer	1

- （可选）如果需要使用文件聚合功能，需要配置数据集整体组信息。

  表 7-7 数据集整体组信息（GROUPS）

名称	描述	类型	缺省值
GROUPNAME	必填。聚合模式下聚合后文件名称，需要设置为可以匹配FILEWHITELIST正则的字符串。	String，最大长度64	-
FILEWHITELIST	可选。将符合该正则表达式的文件聚合为上述GROUPNAME文件。	String，最大长度64	-

- （可选）如果需要设置单个文件内容可乱序输出，需要配置数据集个体信息。

版权所有 © 华为技术有限公司

表 7-8 数据集个体信息（ENTITIES）

名称	描述	类型	缺省值
NAME	必填。路径表示的数据项名称。	String，最大长度64	-
SHUFFLE	可选。应对粒子型应用中不要求保证粒子顺序的场景。	Boolean ● false ● true 不区分大小写。	false

7.3 Hyper IO 测试

测试目的	使用Hyper IO加速应用软件性能测试
用例编号	HyperIO_WRF_1
预置条件	1. 8台Kunpeng 920服务器，共享存储挂载正常，ROCE网络通信正常。 2. OS版本为OpenEuler22.03，已部署并健康运行。 3. 编译器使用系统默认版本。 4. Hyper MPI版本为1.3.1，已部署并正确加载Hyper MPI环境变量。 5. 应用软件WRF版本为4.2.2，应用算例准备完成。 6. Hyper IO已解压部署。 7. 用户已登录服务器控制终端。

版权所有 © 华为技术有限公司

操作步骤	1. 获取算例文件： **mkdir -p /path/to/WRFTEST** **cd /path/to/WRFTEST** **wget https://www2.mmm.ucar.edu/WG2bench/conus_2.5_v3/1-RST/RST/ -r -c -np -nH --cut-dirs 2 --restrict-file-names=nocontrol -e robots=off --reject "index.html*"** **wget https://www2.mmm.ucar.edu/WG2bench/conus_2.5_v3/wrfbdy_d01.gz -r -c -np -nH --cut-dirs 2 --restrict-file-names=nocontrol -e robots=off --reject "index.html*"** **wget https://www2.mmm.ucar.edu/WG2bench/conus_2.5_v3/1-RST/namelist.input -r -c -np -nH --cut-dirs 2 --restrict-file-names=nocontrol -e robots=off --reject "index.html*"** 2. 生成算例： **cat rst_6hr* | gunzip -c > wrfrst_d01_2005-06-04_06_00_00** 3. 拷贝WRF-4.2.2/run/*到目录“/path/to/WRFTEST”下： **cp /path/to/WRF/WRF-4.2.2/run/* ./** **rm -rf wrf.exe** **ln -s /path/to/WRF/WRF-4.2.2/main/wrf.exe wrf.exe** 4. 解压“wrfbdy_d01”文件： **gunzip wrfbdy_d01.gz** 5. 修改namelist.input文件。 6. 运行WRF程序，记录运行时间： **time mpirun --allow-run-as-root -N** *8* **-np** *640* **-x PATH -x LD_LIBRARY_PATH --hostfile** *hf4* **OMP_NUM_THREADS=1 /path/to/WRFTEST/wrf.exe** 7. 使用Hyper IO运行WRF程序，记录运行时间： **time mpirun --allow-run-as-root -N** *8* **-np** *640* **-x PATH -x LD_LIBRARY_PATH -x LD_PRELOAD=***/path/to/IO_Middleware_version_aarch64/lib/libock_shore_pnetcdf.so* **-x OCK_IO_CONFIG=***/path/to/wrf.json* **--hostfile** *hf4* **OMP_NUM_THREADS=1 /path/to/WRFTEST/wrf.exe** 8. 对比**6**和**7**应用运行的时长。
预期输出	WRF应用运行时长提升10%以上。
测试结果	-
备注	-

版权所有 © 华为技术有限公司

7.4 接口支持列表

Hyper IO接口支持NetCDF、PnetCDF、MPI-IO、HDF5，具体接口支持列表可参见7.2.2 Hyper IO函数说明。

名称	说明
MPI-IO接口函数	通过对MPI-IO的API进行桥接，定向到Hyper IO的同名接口函数获得性能加速。
PnetCDF接口函数	通过对PnetCDF的API进行桥接，定向到Hyper IO的同名接口函数获得性能加速。
NetCDF接口函数	通过对NetCDF的API进行桥接，定向到Hyper IO的同名接口函数获得性能加速。
HDF5接口函数	通过对HDF5的API进行桥接，定向到Hyper IO的同名接口函数获得性能加速。

7.5 术语和缩略语

表 7-9 术语&缩略语

名称	说明
AdhocFS	Hyper IO套件中的一个功能组件，是一种近计算的数据缓存系统，用于应对突发IO请求，提升应用IO性能。
Hyper IO	华为高性能IO加速套件，包含Hyper IO、AdhocFS、IO转发等功能组件。
HDF5	HDF5（Hierarchical Data Format）是由美国伊利诺伊大学厄巴纳-香槟分校UIUC（University of Illinois at Urbana-Champaign）开发的一种常见的跨平台数据储存文件，可以存储不同类型的图像和数码数据，并且可以在不同类型的机器上传输，同时还有统一处理这种文件格式的函数库。
IO转发	IOFWD（IO Forward），Hyper IO套件中的一个功能组件，可以实施进程级IO转发代理，降低计算节点上的资源消耗对计算过程的影响并提升IO性能。
IO Middleware	Hyper IO套件中的一个功能组件，以动态库的方式呈现，应用程序可以通过它提升IO性能。
MPI-IO	MPI套件的一个功能组件，作为高性能计算应用可以使用的一种Hyper IO，应用程序可以通过使用它提供的缓存和数据过滤技术，来整体提升数据访问性能。MPI-IO有多种实现方式，其中ROMIO就是MPI-IO标准中比较常见的一种实现。

版权所有 © 华为技术有限公司

名称	说明
NetCDF	Network Common Data Form的缩写，是由美国大学大气研究协会（University Corporation for Atmospheric Research，UCAR）的Unidata项目科学家针对科学数据的特点开发的一种面向数组型并适于网络共享的数据的描述和编码标准。广泛应用于大气科学、水文、海洋学、环境模拟、地球物理等诸多领域。
PnetCDF	PnetCDF是一个并行I/O库，用于并发访问NetCDF格式的数据。该软件开发是西北大学和阿贡国家实验室的合作成果。

版权所有 © 华为技术有限公司

8 领域加速库开发指南

KuTACC开发指南
KuDNN开发指南
KuQCD开发指南

8.1 KuTACC 开发指南

8.1.1 总述

KuTACC是基于鲲鹏平台的Transformer模型融合算子库，采用C/C++实现，针对AI算子各层核心算子进行优化，并提供相应接口用于在应用中直接调用，为AI应用提供高效算力支撑。目前KuTACC基于毕昇编译器和libstdc++标准库实现，对gcc编译器暂未支持。

8.1.2 KuTACC 库函数说明

8.1.2.1 函数说明

由C/C++实现，其中包含功能函数和优化算子函数。功能函数主要用于提供与算子优化无关的功能的接口，例如输出当前KuTACC库的版本号等；而优化算子函数为优化算子的使用接口，用于在AI应用中调用。

8.1.2.2 函数定义

8.1.2.2.1 功能函数

8.1.2.2.1.1 返回值定义

下表定义了功能函数的常用返回值及其含义。

 版权所有 © 华为技术有限公司

表 8-1 返回值定义

ReturnCode	类型	值	描述
KUTACC_OK	int	0	执行成功。
KUTACC_ERROR	int	-1	执行过程出现错误，执行失败。

8.1.2.2.1.2 kutacc_get_version

获取KuTACC产品版本信息。

接口定义

int kutacc_get_version(kutacc_version_t *version)

参数

参数名	描述	取值范围	输入/输出
version	产品信息结构体。	非空	输出

返回值

- 成功：返回KUTACC_OK
- 失败：返回KUTACC_ERROR

示例

```
#include <stdio.h>
#include "kutacc.h"

int main()
{
    kutacc_version_t version;
    int status = kutacc_get_version(&version);
    printf("product_name = %s\n", version.product_name);
    printf("product_version = %s\n", version.product_version);
    printf("component_name = %s\n", version.component_name);
    printf("component_version = %s\n", version.component_version);
    printf("component_abi_version = %s\n", version.component_abi_version);
    printf("component_appendinfo = %s\n", version.component_appendinfo);
    return 0;
}
```

运行结果：

```
product_name = Kunpeng HPCKit
product_version = 26.1.RC1
component_name = KuTACC
component_version = 26.1.RC1
component_abi_version = 1
component_appendinfo = bisheng
```

 版权所有 © 华为技术有限公司

说明

以上版本号和编译时间以实际运行结果为准，上述结果仅供参考。

8.1.2.2.2 共性融合算子函数

8.1.2.2.2.1 通信算子

?.1. kutacc_af2_all_gather

进程间的all_gather操作

接口定义

void kutacc_af2_all_gather(kutacc_tensor_h data, kutacc_tensor_h out);

参数

参数名	类型	描述	输入/输出
data	kutacc_tensor_h	输入张量	输入
out	kutacc_tensor_h	输出张量	输出

示例

C++ interface：

```
#include <vector>
#include <mpi.h>
#include "kutacc.h"

int main()
{
    int world_size = 2, rank = 0;
    int m = 4, n = 5, len = 1;
    int64_t dim = 3, buffer_size = m * n * len;

    MPI_Init(&argc, &argv);
    MPI_Comm_size(MPI_COMM_WORLD, &world_size);
    MPI_Comm_rank(MPI_COMM_WORLD, &rank);
    kutacc_initialize(world_size, rank, buffer_size);

    std::vector<int64_t> sizes = {m, n, len};
    std::vector<int64_t> strides = {n * len, len, 1};
    std::vector<int64_t> data(m * n * len);

    int val;
    if (rank == 0) {
        val = 0;
    } else if (rank == 1) {
        val = 32;
    }
    for (int i = 0; i < m; ++i) {
        for (int j = 0; j < n; ++j) {
            data[i * n * len + j * len] = val++;
        }
    }
```

 版权所有 © 华为技术有限公司

```
    std::vector<int64_t> out_sizes = {world_size * m, n, len};
    std::vector<int64_t> out_strides = {n * len, len, 1};
    std::vector<int64_t> out_data(world_size * m * n * len);

    kutacc::TensorWrapper in(data.data(), sizes, strides, dim, kutacc::kBF16);
    kutacc::TensorWrapper out(out_data.data(), out_sizes, out_strides, dim, kutacc::kBF16);
    kutacc_af2_all_gather(in.get_tensor(), out.get_tensor());

    kutacc_finalize();
    MPI_Finalize();

    return 0;
}

/**
 * rank0:              rank1:
 * 0 1 2 3 4           32 33 34 35 36
 * 5 6 7 8 9           37 38 39 40 41
 * 10 11 12 13 14      42 43 44 45 46
 * 15 16 17 18 19      47 48 49 50 51
 *
 * all_gather()
 *
 * rank0 && rank1:
 * 0 1 2 3 4
 * 5 6 7 8 9
 * 10 11 12 13 14
 * 15 16 17 18 19
 * 32 33 34 35 36
 * 37 38 39 40 41
 * 42 43 44 45 46
 * 47 48 49 50 51
**/
```

?.2. kutacc_af2_transpose

进程间的transpose操作

接口定义

void kutacc_af2_transpose(kutacc_tensor_h data, kutacc_tensor_h out);

参数

参数名	类型	描述	输入/输出
data	kutacc_tensor_h	输入张量	输入
out	kutacc_tensor_h	输出张量	输出

示例

C++ interface：

```
#include <vector>
#include <mpi.h>
#include "kutacc.h"

int main()
{
    int world_size = 2, rank = 0;
    int m = 5, n = 6, len = 1;
```

 版权所有 © 华为技术有限公司

```
    int64_t dim = 3, buffer_size = m * n * len;
    int64_t block_n = (n + world_size - 1) / world_size;

    MPI_Init(&argc, &argv);
    MPI_Comm_size(MPI_COMM_WORLD, &world_size);
    MPI_Comm_rank(MPI_COMM_WORLD, &rank);
    kutacc_initialize(world_size, rank, buffer_size);

    std::vector<int64_t> sizes = {m, n, len};
    std::vector<int64_t> strides = {n * len, len, 1};
    std::vector<int64_t> data(m * n * len);

    int val;
    if (rank == 0) {
        val = 0;
    } else if (rank == 1) {
        val = 32;
    }
    for (int i = 0; i < m; ++i) {
        for (int j = 0; j < n; ++j) {
            data[i * n * len + j * len] = val++;
        }
    }
    std::vector<int64_t> out_sizes = {world_size * m, block_n, len};
    std::vector<int64_t> out_strides = {block_n * len, len, 1};
    std::vector<int64_t> out_data(world_size * m * block_n * len);

    kutacc::TensorWrapper in(data.data(), sizes, strides, dim, kutacc::kBF16);
    kutacc::TensorWrapper out(out_data.data(), out_sizes, out_strides, dim, kutacc::kBF16);

    kutacc_af2_transpose(in.get_tensor(), out.get_tensor());

    kutacc_finalize();
    MPI_Finalize();

    return 0;
}

/**
 * rank0:                rank:1
 * 0 1 2 3 4 5             32 33 34 35 36 37
 * 6 7 8 9 10 11           38 39 40 41 42 43
 * 12 13 14 15 16 17       44 45 46 47 48 49
 * 18 19 20 21 22 23       50 51 52 53 54 55
 * 24 25 26 27 28 29       56 57 58 59 60 61
 *
 * transpose()
 *
 * rank:0                rank:1
 * 0 1 2                 3 4 5
 * 6 7 8                 9 10 11
 * 12 13 14                15 16 17
 * 18 19 20                21 22 23
 * 24 25 26                27 28 29
 * 32 33 34                35 36 37
 * 38 39 40                41 42 43
 * 44 45 46                47 48 49
 * 50 51 52                53 54 55
 * 56 57 58                59 60 61
**/
```

?.3. kutacc_comm_init

通信相关变量的创建和初始化

接口定义

void kutacc_comm_init(int64_t world_size, int64_t rank, int64_t buffer_size);

 版权所有 © 华为技术有限公司

参数

参数名	类型	描述	输入/输出
world_size	int64_t	总进程数	输入
rank	int64_t	当前进程号	输入
buffer_size	int64_t	共享内存空间大小	输入

示例

C++ interface：

```
#include <vector>
#include <mpi.h>
#include "kutacc.h"

int main()
{
    int world_size = 2, rank = 0;
    int m = 4, n = 5, len = 1;
    int64_t dim = 3, buffer_size = m * n * len;

    MPI_Init(&argc, &argv);
    MPI_Comm_size(MPI_COMM_WORLD, &world_size);
    MPI_Comm_rank(MPI_COMM_WORLD, &rank);
    kutacc_comm_init(world_size, rank, buffer_size);

    std::vector<int64_t> sizes = {m, n, len};
    std::vector<int64_t> strides = {n * len, len, 1};
    std::vector<int64_t> data(m * n * len);

    int val;
    if (rank == 0) {
        val = 0;
    } else if (rank == 1) {
        val = 32;
    }
    for (int i = 0; i < m; ++i) {
        for (int j = 0; j < n; ++j) {
            data[i * n * len + j * len] = val++;
        }
    }
    std::vector<int64_t> out_sizes = {world_size * m, n, len};
    std::vector<int64_t> out_strides = {n * len, len, 1};
    std::vector<int64_t> out_data(world_size * m * n * len);

    kutacc::TensorWrapper in(data.data(), sizes, strides, dim, kutacc::kBF16);
    kutacc::TensorWrapper out(out_data.data(), out_sizes, out_strides, dim, kutacc::kBF16);
    kutacc_af2_all_gather(in.get_tensor(), out.get_tensor());

    kutacc_comm_fini();
    MPI_Finalize();

    return 0;
}
```

?.4. kutacc_comm_fini

通信相关变量的释放和销毁

 版权所有 © 华为技术有限公司

接口定义

void kutacc_comm_fini();

参数

参数名	类型	描述	输入/输出
/	/	/	/

示例

C++ interface：

```
#include <vector>
#include <mpi.h>
#include "kutacc.h"

int main()
{
    int world_size = 2, rank = 0;
    int m = 4, n = 5, len = 1;
    int64_t dim = 3, buffer_size = m * n * len;

    MPI_Init(&argc, &argv);
    MPI_Comm_size(MPI_COMM_WORLD, &world_size);
    MPI_Comm_rank(MPI_COMM_WORLD, &rank);
    kutacc_comm_init(world_size, rank, buffer_size);

    std::vector<int64_t> sizes = {m, n, len};
    std::vector<int64_t> strides = {n * len, len, 1};
    std::vector<int64_t> data(m * n * len);

    int val;
    if (rank == 0) {
        val = 0;
    } else if (rank == 1) {
        val = 32;
    }
    for (int i = 0; i < m; ++i) {
        for (int j = 0; j < n; ++j) {
            data[i * n * len + j * len] = val++;
        }
    }
    std::vector<int64_t> out_sizes = {world_size * m, n, len};
    std::vector<int64_t> out_strides = {n * len, len, 1};
    std::vector<int64_t> out_data(world_size * m * n * len);

    kutacc::TensorWrapper in(data.data(), sizes, strides, dim, kutacc::kBF16);
    kutacc::TensorWrapper out(out_data.data(), out_sizes, out_strides, dim, kutacc::kBF16);
    kutacc_af2_all_gather(in.get_tensor(), out.get_tensor());

    kutacc_comm_fini();
    MPI_Finalize();

    return 0;
}
```

8.1.2.2.2.2 GEMM 算子

 版权所有 © 华为技术有限公司

?.1. kutacc_core_bgemm_ex

一般矩阵乘矩阵。

即：$C = \text{alpha} * \text{op}(A) * \text{op}(B) + \text{beta} * C$。

接口定义

void kutacc_core_bgemm_ex(char transA, char transB, const BLASINT m, const BLASINT n, const BLASINT k, const __bf16 alpha, const __bf16 *a, const BLASINT lda, const __bf16 *b, const BLASINT ldb, const __bf16 beta, __bf16 *c, const BLASINT ldc, const BlasExtendParam *blas_extend_param);

参数

参数名	类型	描述	输入/输出
transA	字符型	矩阵A的转置情况。 ● 如果transA= 'N'，矩阵A不转置。 ● 如果transA= 'T'，矩阵A转置。	输入
transB	字符型	矩阵B的转置情况。 ● 如果transB= 'N'，矩阵B不转置。 ● 如果transB= 'T'，矩阵B转置。	输入
m	整数型	矩阵A的行。	输入
n	整数型	矩阵B的列。	输入
k	整数型	矩阵A的列和B的行。	输入
alpha	半精度浮点类型	矩阵A、B乘法系数。	输入
a	半精度浮点类型指针	矩阵A。	输入
lda	整数型	矩阵A的主维度。	输入
b	半精度浮点类型指针	矩阵B。	输入
ldb	整数型	矩阵B的主维度。	输入
beta	半精度浮点类型	矩阵C乘法系数。	输入
c	半精度浮点类型指针	矩阵C。	输入
ldc	半精度浮点类型	矩阵C的主维度。	输入/输出

 版权所有 © 华为技术有限公司

参数名	类型	描述	输入/输出
blas_extend_param	结构体指针类型	设置拓展操作，结构体中type为操作类型，extra为操作数据，next为指向下一个拓展操作结构体的指针。 ● 如果type=BLAS_EXTEND_TYPE_NUM_THREADS，操作为设置线程数。 ● 如果type=BLAS_EXTEND_TYPE_PREPACK，操作为对prepack后的矩阵直接计算。 ● 如果type=BLAS_EXTEND_TYPE_BIAS，操作为对结果矩阵添加bias。 ● 如果type=BLAS_EXTEND_TYPE_ACTIVATION，操作为对结果函数进行激活函数运算。	输入

依赖

#include "kutacc_core.h"

示例

C interface：

```
char transA = 'N', transB = 'N';
BLASINT m = 4, k = 3, n = 4;
BLASINT lda = m;
BLASINT ldb = k;
BLASINT ldc = m;
__bf16 alpha = vcvth_bf16_f32(1.0);
__bf16 beta = vcvth_bf16_f32(3.0);

__bf16 a[12] = {
    3.4019, -1.0562, 2.8310, 2.9844,
    4.1165, -3.0245, -1.6478, 2.6829,
    -2.2223, 0.5397, -0.2260, 1.2887
};

__bf16 b[12] = {
    -1.3522, 0.1340, 4.5223,
    4.1612, 1.3571, 2.1730,
    -3.5840, 1.0697, -4.8370,
    -2.5711, -3.6277, 3.0418
};

__bf16 c[12] = {0};

BlasExtendParam *extend_param = (BlasExtendParam *)malloc(sizeof(BlasExtendParam));
if (extend_param == NULL) {
```

 版权所有 © 华为技术有限公司

```
    return;
}

extend_param->type = BLAS_EXTEND_TYPE_ACTIVATION;
extend_param->extra = BLAS_EXTEND_ACTIVATION_RELU;
extend_param->next = NULL;

kutacc_core_bgemm_ex(transA, transB, m, n, k, alpha, a, lda, b, ldb, beta, c, ldc, extend_param);
free(extend_param);

/*
 * Output c:
 *     0.000000      43.250000      2.968750      0.000000
 *     10.437500      0.000000      0.000000      15.750000
 *     0.000000      32.500000      0.000000      11.625000
 *     6.531250      43.750000      0.000000      0.000000
 */
```

?.2. kutacc_core_bgemm_pack

对矩阵进行pack操作并将其存储到已分配的缓冲区中。

接口定义

void kutacc_core_bgemm_pack(char matrixIdentifier, char transA, char transB, const BLASINT m, const BLASINT n, const BLASINT k, const BLASINT lda, const BLASINT ldb, const __bf16 *src, __bf16 *dst);

参数

参数名	类型	描述	输入/输出
matrixIdentifier	字符型	指定要pack的矩阵，默认为矩阵B。 ● 如果identifier= 'A'，pack 矩阵A。 ● 如果identifier= 'B'，pack 矩阵B。	输入
transA	字符型	矩阵A的转置情况。 ● 如果transA= 'N'，矩阵A不转置。 ● 如果transA= 'T'，矩阵A转置。	输入
transB	字符型	矩阵B的转置情况。 ● 如果transB= 'N'，矩阵B不转置。 ● 如果transB= 'T'，矩阵B转置。	输入
m	整数型	矩阵A的行。	输入
n	整数型	矩阵B的列。	输入

 版权所有 © 华为技术有限公司

参数名	类型	描述	输入/输出
k	整数型	矩阵A的列和B的行。	输入
lda	整数型	矩阵A的主维度。	输入
ldb	整数型	矩阵B的主维度。	输入
src	半精度浮点类型指针	输入矩阵。	输入
dst	半精度浮点类型指针	pack后的矩阵。	输入/输出

依赖

#include "kutacc_core.h"

示例

C interface：

```
char transA = 'N', transB = 'N';
BLASINT m = 4, k = 3, n = 4;
BLASINT lda = m;
BLASINT ldb = n;

__bf16 a[12] = {
    0.1, 0.2, 0.3, 0.4,
    0.5, 0.6, 0.7, 0.8,
    0.9, 1.0, 1.1, 1.2
};
size_t size_a = kutacc_core_bgemm_pack_get_size('A', m, n, k);
__bf16 *sa = (__bf16 *)malloc(size_a * sizeof(__bf16));
kutacc_core_bgemm_pack('A', transA, transB, m, n, k, lda, ldb, a, sa);
free(sa);
/*
 * Output dst:
 *     0.100098      0.601562      0.400391      1.000000
 *     0.500000      0.300781      0.800781      1.101562
 *     0.200195      0.699219      0.898438      1.203125
 */
```

?.3. kutacc_core_bgemm_pack_get_size

获取存储pack后的矩阵所需的字节数。

接口定义

size_t kutacc_core_bgemm_pack_get_size(char matrixIdentifier, const BLASINT m, const BLASINT n, const BLASINT k);

 版权所有 © 华为技术有限公司

参数

参数名	类型	描述	输入/输出
matrixIdentifier	字符型	指定要pack的矩阵，默认为B矩阵。 • 如果identifier= 'A'，返回pack 矩阵A所需字节数。 • 如果identifier= 'B'，返回pack 矩阵B所需字节数。	输入
m	整数型	需要pack的矩阵A的行。	输入
n	整数型	需要pack的矩阵B的列。	输入
k	整数型	需要pack的矩阵A的列和B的行。	输入

依赖

#include "kutacc_core.h"

示例

C interface：

```
int m = 4, k = 3, n = 4;

size_t size = kutacc_core_bgemm_pack_get_size('A', m, n, k);
/*
 * Output size: 12
 */
```

?.4. kutacc_af2_layernorm

layernorm归一化的kutacc实现

接口定义

kutacc_export void kutacc_af2_layernorm(__bf16 * data, float* gamma, float* beta, int64_t size, float eps, __bf16 * out);

参数

参数名	类型	描述	输入/输出
data	__bf16 *	数据源指针	输入
gamma	float *	layernorm归一化权重参数指针	输入
beta	float*	layernorm归一化偏置参数指针	输入

 版权所有 © 华为技术有限公司

参数名	类型	描述	输入/输出
size	float	数据源最后一个维度的大小	输入
eps	float	极小量 用于稳定计算结果	输入
out	__bf16 *	计算结果	输出

参数约束关系：

data源数据的最后一维大小与gamma， beta大小一致，即size = gamma.size, size = beta.size

gamma和beta均只有一个维度，data维度支持三个维度进行运算

?.5. kutacc_af2_linear

linear线性变化的kutacc实现, 对应公式:

$$C=1*op(A)*op(B)+beta*C+bias$$

接口定义

void kutacc_af2_linear(kutacc_tensor_h act, kutacc_tensor_h weight, float* bias_data, kutacc_tensor_h result, int64_t beta);

参数

参数名	类型	描述	输入/输出
act	kutacc_tensor_h	基础数据A	输入
weight	kutacc_tensor_h	权重参数B	输入
bias_data	float *	偏移量参数	输入
result	kutacc_tensor_h	运算结果	输出
beta	int64_t	C的系数	输入

参数约束关系:

bias维度大小等于weight维度大小减1，

act和weight维度均大于等于2

8.1.2.2.3 个性融合算子函数

 版权所有 © 华为技术有限公司

8.1.2.2.3.1 AlphaFold2

?.1. kutacc_af2_gating_attention

gating_attention函数通过在标准多头自注意力（multi-self-attention）输出上施加可学习的门控（gating），来动态地筛选和控制信息流，从而提升模型对关键残基或序列位置的敏感性与表达能力。

接口定义

void kutacc_af2_gating_attention(kutacc_tensor_h input, kutacc_af2_attention_inputs_t *q_based_ptr, kutacc_tensor_h bias, kutacc_tensor_h nonbatched_bias, kutacc_af2_attention_weights_t *weight_ptr, kutacc_tensor_h out, int64_t block_size);

参数

表1 入参定义

参数名	类型	描述	输入/输出
input	kutacc_tensor_h	输入数据	输入
q_based_ptr	kutacc_af2_attention_inputs_t *	kutacc_af2_attention_inputs_t类型的指针，它指向多重输入张量数据的集合，具体见下方kutacc_af2_attention_inputs_t数据结构表	输入
bias	kutacc_tensor_h	神经网络中的attention偏移量数据	输入
nonbatched_bias	kutacc_tensor_h	神经网络中的通用偏移量数据	输入
weight_ptr	kutacc_af2_attention_weights_t *	kutacc_af2_attention_weights_t类型的指针，它指向多重权重数据的结合，具体见下方kutacc_af2_attention_weights_t数据结构表	输入
out	kutacc_tensor_h	输出数据	输出
block_size	int64_t	并行操作中的数据块切分大小	输入

表2 kutacc_af2_attention_inputs_t数据结构表定义

参数名	类型	描述

 版权所有 © 华为技术有限公司

gate	kutacc_tensor_h	输入中间层gate数据
k	kutacc_tensor_h	输入中间层k数据
v	kutacc_tensor_h	输入中间层v数据
q	kutacc_tensor_h	输入中间层q数据
avg	kutacc_tensor_h	权重avg数据
batch	int64_t	输入批次
seq_len	int64_t	输入数据的序列长度

表3 kutacc_af2_attention_weights_t数据结构表

参数名	类型	描述
nchannels	int64_t	输入权重的总数
nheads	int64_t	输入权重的head数
head_size	int64_t	输入权重每个head中的数据数量
query_w	kutacc_tensor_h	权重query_w数据
key_w	kutacc_tensor_h	权重key_w数据
gating_w	kutacc_tensor_h	权重gating_w数据
gating_b	kutacc_tensor_h	门控偏移量数据
output_w	kutacc_tensor_h	权重output_w数据
output_b	kutacc_tensor_h	输出矩阵乘法偏移量数据
value_w	kutacc_tensor_h	权重value_w数据

整数参数的约束关系 应满足:

nchannels = nheads * head_size;

batch, seq_len, nchannels, nheads, head_size, block_size > 0,

nchannels < INT64_MAX

在调用kpex接口时构造tensor应满足以下形状约束

表4 kpex gating_attention入参形状约束

参数	形状
query_w	[nheads, head_size, nchannels]

 版权所有 © 华为技术有限公司

参数	形状
key_w	[nheads, head_size, nchannels]
value_w	[nheads, head_size, nchannels]
gating_w	[nchannels, nheads, head_size]
gating_b	[nheads, head_size]
output_w	[nchannels, nheads, head_size]
output_b	[nchannels]
q_data	[batch, seq_len, nchannels]
m_data	[batch, seq_len, nchannels]
bias	[batch, 1, 1, seq_len]
nonbatched_bias	[nheads, seq_len, seq_len] or [0]

示例

C++ interface：

```
//test_gating_attention.h
#include <ATen/core/Tensor.h>
#include <ATen/ops/empty.h>
#include <ATen/ops/ones.h>
#include <ATen/ops/zeros.h>
#include <ATen/ops/full.h>
#include <ATen/native/cpu/utils.h>
#include <c10/core/ScalarType.h>
#include "common_header.h"

namespace alphafold {
at::Tensor test_global_attention(int64_t batch, int64_t seq_len, int64_t nchannels, int64_t nheads, int64_t
head_size);
}

//bind.h
#include "test_gating_attention.h"
#include <torch/extension.h>

namespace alphafold {
inline void bind(pybind11::module &m)
{
    auto submodule = m.def_submodule("alphafold");
    submodule.("test_gating_attention", &test_gating_attention, py::("batch"), py::("seq_len"), py::
("nchannels"), py::("nheads"), py::("head_size"), py::("block_size"));
}
}


//test_gating_attention.cpp
#include "kutacc.h"
#include <utils/memory.h>
#include <utils/bf16.h>
#include <utils/TensorWrapper.h>
#include "test_gating_attention.h"
namespace alphafold {
at::Tensor test_gating_attention(int64_t batch, int64_t seq_len, int64_t nchannels, int64_t nheads, int64_t
head_size, int64_t block_size) {
```

 版权所有 © 华为技术有限公司

```
    float a = 3.0f;
    float b = 2.5f;
    at::Tensor q_data = at::ones({batch, seq_len, nchannels},
at::TensorOptions().device(kpex::device()).dtype(at::kBFloat16));
    auto q = q_data.new_empty({batch, seq_len, nheads, head_size});
    auto k = q_data.new_empty({batch, seq_len, nheads, head_size});
    auto v = q_data.new_empty({nheads, head_size, batch, seq_len});
    auto gate = q_data.new_empty({batch, seq_len, nheads, head_size});
    auto weighted_avg = q_data.new_empty({batch, seq_len, nheads, head_size});
    at::Tensor input = at::ones({batch * seq_len, nchannels}, q_data.options());
    at::Tensor bias =  at::ones({batch, 1, 1, seq_len}, q_data.options());
    at::Tensor nonbatched_bias = at::zeros({nheads, seq_len, seq_len}, q_data.options());
    at::Tensor query_w = at::full({nheads, head_size, nchannels}, a, q_data.options()).to(at::kBFloat16);
    auto bf16_opt = query_w.options().device(kpex::device()).dtype(at::kBFloat16);
    auto float_opt = query_w.options().device(kpex::device()).dtype(at::kFloat);
    at::Tensor key_w = at::full({nheads, head_size, nchannels}, b, q_data.options()).to(bf16_opt);
    at::Tensor value_w = at::full({nheads, head_size, nchannels}, a, q_data.options()).to(bf16_opt);
    at::Tensor gating_w = at::full({nheads, head_size, nchannels}, a, q_data.options()).to(bf16_opt);
    at::Tensor gating_b = at::full({nheads, head_size}, b, q_data.options()).to(float_opt);
    at::Tensor output_w = at::full({nchannels, nheads, head_size}, b, q_data.options()).to(bf16_opt);
    at::Tensor output_b = at::full({nchannels}, a, q_data.options()).to(float_opt);
    query_w = query_w.to(bf16_opt).contiguous().view({nchannels, nchannels});
    key_w = key_w.to(bf16_opt).contiguous().view({nchannels, nchannels});
    value_w = value_w.to(bf16_opt).contiguous().view({nchannels, nchannels});
    gating_w = gating_w.to(bf16_opt).contiguous().view({nchannels, nchannels});
    output_w = output_w.to(bf16_opt).contiguous().view({nchannels, nchannels});
    auto query_w_res = linear_weight_prepack(query_w);
    auto key_w_res = linear_weight_prepack(key_w);
    auto value_w_res = linear_weight_prepack(value_w);
    auto gating_w_res = linear_weight_prepack(gating_w);
    auto output_w_res = linear_weight_prepack(output_w);
    at::Tensor out = at::empty(q_data.sizes(), q_data.options());
    auto input_tw = convert_to_tensor_wrapper(input);
    auto q_tw = convert_to_tensor_wrapper(q);
    auto k_tw = convert_to_tensor_wrapper(k);
    auto v_tw = convert_to_tensor_wrapper(v);
    auto gate_tw = convert_to_tensor_wrapper(gate);
    auto weighted_avg_tw = convert_to_tensor_wrapper(weighted_avg);
    auto bias_tw = convert_to_tensor_wrapper(bias);
    auto nonbatched_bias_tw = convert_to_tensor_wrapper(nonbatched_bias);
    auto query_w_tw = convert_to_tensor_wrapper(query_w_res);
    auto key_w_tw = convert_to_tensor_wrapper(key_w_res);
    auto value_w_tw = convert_to_tensor_wrapper(value_w_res);
    auto gating_w_tw = convert_to_tensor_wrapper(gating_w);
    auto gating_b_tw = convert_to_tensor_wrapper(gating_w_res);
    auto output_w_tw = convert_to_tensor_wrapper(output_w_res);
    auto output_b_tw = convert_to_tensor_wrapper(output_b);
    auto out_tw = convert_to_tensor_wrapper(out);
    kutacc_af2_attention_weights_t_wrapper *gating_attention_weight_ptr = new
kutacc_af2_attention_weights_t_wrapper(query_w_tw, key_w_tw, value_w_tw, gating_w_tw, gating_b_tw,
        output_w_tw, output_b_tw, nchannels, nheads, head_size);
    kutacc_af2_attention_inputs_t_wrapper *gating_attention_q_ptr = new
kutacc_af2_attention_inputs_t_wrapper(q_tw, k_tw, v_tw, gate_tw, weighted_avg_tw, batch, seq_len);
    kutacc_af2_gating_attention(input_tw.get_tensor(), gating_attention_q_ptr,bias_tw.get_tensor(),
nonbatched_bias_tw.get_tensor(), gating_attention_weight_ptr, out_tw.get_tensor(), block_size);
    delete gating_attention_weight_ptr;
    delete gating_attention_q_ptr;
    return out;
}
}

//test.py
import torch
import kpex._C as kernel
def test_gating_attention(batch, seq_len, nchannels, nheads, head_size, block_size):
    out = kernel.alphafold.test_gating_attention(batch, seq_len, nchannels, nheads, head_size, block_size)
    print(out)
    return out
```

 版权所有 © 华为技术有限公司

```
/**
 * input：2, 2, 8, 4, 2, 2
 * output:
tensor([[[484., 484., 484., 484., 484., 484., 484., 484.],
         [484., 484., 484., 484., 484., 484., 484., 484.]],

        [[484., 484., 484., 484., 484., 484., 484., 484.],
         [484., 484., 484., 484., 484., 484., 484., 484.]]],
       dtype=torch.bfloat16)
 *
 */
```

?.2. kutacc_af2_triangle_multiplication_calc_proj

用于计算triangle_multiplication过程中生成的中间值：左右投影

接口定义

void kutacc_af2_triangle_multiplication_calc_proj(kutacc_af2_tm_act_inputs_t *tm_acts_ptr, kutacc_tensor_h mask, kutacc_af2_tm_proj_weights_t *tm_weights_ptr, bool input_prepack);

参数

表1 入参定义

参数名	类型	描述	输入/输出
tm_acts_ptr	kutacc_af2_tm_act_inputs_t *	kutacc_af2_tm_act_inputs_t类型的指针，具体数据结构定义见下方表2 kutacc_af2_tm_act_inputs_t数据结构表定义	输入
mask	kutacc_tensor_h	掩码张量	输入
tm_weights_ptr	kutacc_af2_tm_proj_weights_t *	kutacc_af2_tm_proj_weights_t类型的指针，具体数据结构定义见下方表3 kutacc_af2_tm_proj_weights_t数据结构表定义	输入
input_prepack	bool	是否对输入张量进行prepack操作	输入

表2 kutacc_af2_tm_act_inputs_t数据结构表定义

参数名	类型	描述	输入/输出
n_res	int64_t	残基数	输入
n_res_gather	int64_t	残基数	输入
proj_act	kutacc_tensor_h	act的左右投影	输出

 版权所有 © 华为技术有限公司

参数名	类型	描述	输入/输出
input_act	kutacc_tensor_h	输入值act经过layernorm归一化后的值	输入
proj_act_gate	kutacc_tensor_h	临时门控张量	输入

表3 kutacc_af2_tm_proj_weights_t数据结构表定义

参数名	类型	描述	输入/输出
c_o	int64_t	输入特征维度	输入
c_i	int64_t	输出特征维度	输入
proj_w	kutacc_tensor_h	生成投影所需的权重	输入
proj_b	kutacc_tensor_h	生成投影所需的偏置	输入
gate_w	kutacc_tensor_h	门控权重	输入
gate_b	kutacc_tensor_h	门控偏置	输入

triangle_multiplication的整数参数应满足的约束关系：

n_res, n_res_gather, c_i, c_o > 0,

n_res * n_res_gather < INT64_MAX，

c_i = c_o，

单进程情况下n_res必须等于n_res_gather，多进程则不需要满足该条件

?.3. kutacc_af2_triangle_multiplication_equation

triangle_multiplication的中间步骤，将左右投影通过矩阵相乘生成中间过程参数center_act

接口定义

void kutacc_af2_triangle_multiplication_equation(kutacc_tensor_h center_act, kutacc_tensor_h left_proj_act, kutacc_tensor_h right_proj_act, int64_t n_res_gather, bool is_incoming);

参数

表1 入参定义

参数名	类型	描述	输入/输出
center_act	kutacc_tensor_h	中间过程参数	输出

 版权所有 © 华为技术有限公司

参数名	类型	描述	输入/输出
left_proj_act	kutacc_tensor_h	左投影	输入
right_proj_act	kutacc_tensor_h	右投影	输入
n_res_gather	int64_t	残基数	输入
is_incoming	bool	是否为传入方向	输入

triangle_multiplication的整数参数应满足约束关系:

n_res_gather > 0

单进程情况下n_res必须等于n_res_gather，多进程则不需要满足该条件

?.4. kutacc_af2_triangle_multiplication_gate_and_out_linear

triangle_multiplication的中间步骤，基于input_act通过线性变化生成gate及基于center_act线性变化生成out

接口定义

void kutacc_af2_triangle_multiplication_gate_and_out_linear(kutacc_tensor_h gate, kutacc_tensor_h out, kutacc_af2_tm_act_inputs_t *tm_acts_ptr, kutacc_tensor_h center_act,

kutacc_af2_tm_linear_weights_t *tm_weights_ptr, bool input_prepack);

参数

表1 入参定义

参数名	类型	描述	输入/输出
gate	kutacc_tensor_h	门控张量	输入
out	kutacc_tensor_h	输出张量	输出
tm_acts_ptr	kutacc_af2_tm_act_inputs_t *	kutacc_af2_tm_act_inputs_t类型的指针，具体数据结构定义见表2 kutacc_af2_tm_act_inputs_t数据结构表定义	输入
center_act	kutacc_tensor_h	equation步骤获得center_act经过permute及layernorm变化后获得的输入	输入

 版权所有 © 华为技术有限公司

参数名	类型	描述	输入/输出
tm_weights_ptr	kutacc_af2_tm_linear_weights_t *	kutacc_af2_tm_linear_weights_t类型的指针，具体数据结构定义见表3 kutacc_af2_tm_linear_weights_t数据结构表定义	输入
input_prepack	bool	是否prepack	输入

表2 kutacc_af2_tm_act_inputs_t数据结构表定义

参数名	类型	描述	输入/输出
n_res	int64_t	残基数	输入
n_res_gather	int64_t	残基数	输入
proj_act	kutacc_tensor_h	act的左右投影	输出
input_act	kutacc_tensor_h	输入值act经过layernorm归一化后的值	输入
proj_act_gate	kutacc_tensor_h	临时门控张量	输入

表3 kutacc_af2_tm_linear_weights_t数据结构表定义

参数名	类型	描述	输入/输出
c_o	int64_t	输出特征维度	输入
c_i	int64_t	输入特征维度	输入
gating_w	kutacc_tensor_h	生成gate用的权重	输出
gating_b	kutacc_tensor_h	生成gate用的偏置	输入
output_proj_w	kutacc_tensor_h	生成out用的权重	输入
output_proj_b	kutacc_tensor_h	生成out用的偏置	输入

triangle_multiplication的整数参数应满足的约束关系：

n_res, n_res_gather, c_o, c_i, c_z > 0，

 版权所有 © 华为技术有限公司

n_res * n_res_gather < INT64_MAX，

c_i = c_o;

单进程情况下n_res必须等于n_res_gather，多进程则不需要满足该条件

?.5. kutacc_af2_triangle_multiplication_last

triangle_multiplication的最后步骤，基于out_linear步骤获得的out与gate相乘生成输出张量。

接口定义

void kutacc_af2_triangle_multiplication_last(kutacc_tensor_h out, kutacc_tensor_h gate, int64_t n_res, int64_t n_res_gather, int64_t c_o);

参数

表1 入参定义

参数名	类型	描述	输入/输出
out	kutacc_tensor_h	输出数据	输出
gate	kutacc_tensor_h	门控张量	输入
n_res	int64_t	残基数	输入
n_res_gather	int64_t	残基数	输入
c_o	int64_t	输出特征维度	输入

triangle_multiplication的整数参数应满足的约束关系:

n_res, n_res_gather, c_o > 0，

n_res * n_res_gather < INT64_MAX，

c_i = c_o，

单进程情况下n_res必须等于n_res_gather，多进程则不需要满足该条件

在构建用例及使用KPEX时应满足以下算子形状约束

表2 kpex triangle_multiplication入参形状约束

tensor	shape	描述
act	[n_res, n_res_gather, c_z]	KPEX层输入，且act最后一维大小等于c_o及c_i
mask	[n_res, n_res_gather]	KPEX层掩码输入

 版权所有 © 华为技术有限公司

tensor	shape	描述
input_ln_w	[c_o]	输入参数act经过layernorm变化过程的权重
input_ln_b	[c_o]	输入参数act经过layernorm变化过程的偏置
left_proj_w	[c_i, c_o]	见8.1.2.2.2.1.1.2表2kutacc_af2_tm_proj_weights_t数据结构表定义参数proj_w
left_proj_b	[c_i]	见8.1.2.2.2.1.1.2表2kutacc_af2_tm_proj_weights_t数据结构表定义参数proj_b
right_proj_w	[c_i, c_o]	见8.1.2.2.2.1.1.2表2kutacc_af2_tm_proj_weights_t数据结构表定义参数proj_w
right_proj_b	[c_i]	见8.1.2.2.2.1.1.2表2kutacc_af2_tm_proj_weights_t数据结构表定义参数proj_b

版权所有 © 华为技术有限公司

tensor	shape	描述
left_gate_w	[c_i, c_o]	见8.1.2.2.2.1.1.2表2kutacc_af2_tm_proj_weights_t数据结构表定义参数gate_w
left_gate_b	[c_i]	见8.1.2.2.2.1.1.2表2kutacc_af2_tm_proj_weights_t数据结构表定义参数gate_b
right_gate_w	[c_i, c_o]	见8.1.2.2.2.1.1.2表2kutacc_af2_tm_proj_weights_t数据结构表定义参数gate_w
right_gate_b	[c_i]	见8.1.2.2.2.1.1.2表2kutacc_af2_tm_proj_weights_t数据结构表定义参数gate_b
gating_w	[c_i, c_o]	见上一节表3kutacc_af2_tm_linear_weights_t数据结构表定义参数gating_w
gating_b	[c_i]	见上一节表3kutacc_af2_tm_linear_weights_t数据结构表定义参数gating_b

版权所有 © 华为技术有限公司

tensor	shape	描述
center_ln_w	[c_i]	center_act layernorm归一化过程中的权重
center_ln_b	[c_i]	center_act layernorm归一化过程中的偏置
output_proj_w	[c_o, c_i]	见上一节表3 kutacc_af2_tm_linear_weights_t数据结构表定义参数output_proj_w
output_proj_b	[c_o]	见上一节表3 kutacc_af2_tm_linear_weights_t数据结构表定义参数output_proj_b

示例

C++ interface：

```
// test_triangle_multiplication.h
#define KPEX_TPP_ALPHAFOLD_TEST_TMP_H
#include <ATen/core/Tensor.h>
#include <ATen/ops/empty.h>
#include <ATen/ops/ones.h>
#include <ATen/ops/zeros.h>
#include <ATen/ops/full.h>
#include <ATen/native/cpu/utils.h>
#include <c10/core/ScalarType.h>
namespace alphafold {
at::Tensor test_triangle_multiplication(int64_t n_res, int64_t n_res_gather, int64_t c_o, int64_t c_i);
}
#endif

// bind.h
#include <torch/extension.h>
#include "test_triangle_multiplication.h"
namespace alphafold {
inline void bind(pybind11::module &m)
{
    auto submodule = m.def_submodule("alphafold");
    submodule.def("test_triangle_multiplication", &test_triangle_multiplication,
        py::arg("n_res"), py::arg("n_res_gather"), py::arg("c_o"), py::arg("c_i"));
}
}

// test.py
import torch
from torch import nn
import numpy as np
import torch.distributed as dist
```

 版权所有 © 华为技术有限公司

```
import kpex._C as kernel
import kpex
import os
def test_triangle_multiplication(n_res, n_res_gather, c_o, c_i):
    out = kernel.alphafold.test_triangle_multiplication(n_res, n_res_gather, c_o, c_i)
    return out

// test_triangle_multiplication.cpp
#include "test_triangle_multiplication.h"
#include "triangle_multiplication.h"
#include "kutacc.h"
#include "utils/memory.h"
#include "utils/layernorm.h"
#include "utils/TensorWrapper.h"
namespace alphafold {
at::Tensor test_triangle_multiplication(int64_t n_res, int64_t n_res_gather, int64_t c_o, int64_t c_i)
{
    float a = 0.5f;
    float b = 0.2f;
    float c = 0.4f;
    float d = 1.25f;
    auto bf16_opt = at::TensorOptions().device(kpex::device()).dtype(at::kBFloat16);
    auto float_opt = at::TensorOptions().device(kpex::device()).dtype(at::kFloat);
    at::Tensor input_ln_w = at::full({c_o}, a, float_opt);
    at::Tensor input_ln_b = at::full({c_o}, b, float_opt);
    at::Tensor left_proj_w = linear_weight_prepack(at::ones({c_i, c_o}, bf16_opt));
    at::Tensor left_proj_b = at::zeros({c_i}, float_opt);
    at::Tensor right_proj_w = linear_weight_prepack(at::full({c_i, c_o}, c, bf16_opt));
    at::Tensor right_proj_b = at::ones({c_i}, float_opt);
    at::Tensor left_gate_w = linear_weight_prepack(at::ones({c_i, c_o}, bf16_opt));
    at::Tensor left_gate_b = at::ones({c_i}, float_opt);
    at::Tensor right_gate_w = linear_weight_prepack(at::zeros({c_i, c_o}, bf16_opt));
    at::Tensor right_gate_b = at::zeros({c_i}, float_opt);
    at::Tensor gating_w = linear_weight_prepack(at::full({c_i, c_o}, d, bf16_opt));
    at::Tensor gating_b = at::zeros({c_i}, float_opt);
    at::Tensor center_ln_w = at::ones({c_i}, float_opt);
    at::Tensor center_ln_b = at::zeros({c_i}, float_opt);
    at::Tensor output_proj_w = linear_weight_prepack(at::zeros({c_o, c_i}, bf16_opt));
    at::Tensor output_proj_b =  at::zeros({c_o}, float_opt);
    float e = 1.2f;
    at::Tensor act = at::full({n_res, n_res_gather, c_o}, e, bf16_opt);
    at::Tensor mask = at::ones({n_res, n_res_gather}, bf16_opt);
    at::Tensor out = at::empty(act.sizes(), act.options());
    at::Tensor input_act = layernorm(act, input_ln_w, input_ln_b);
    at::Tensor center_act;
    at::Tensor left_proj_act = input_act.new_empty({c_i, n_res, n_res_gather});
    at::Tensor right_proj_act = input_act.new_empty({c_i, n_res, n_res_gather});
    at::Tensor gate = act.new_empty({n_res, n_res_gather, c_o});
    auto left_proj_act_tw = convert_to_tensor_wrapper(left_proj_act);
    auto right_proj_act_tw = convert_to_tensor_wrapper(right_proj_act);
    auto gate_tw = convert_to_tensor_wrapper(gate);
    auto input_act_tw = convert_to_tensor_wrapper(input_act);
    auto mask_tw = convert_to_tensor_wrapper(mask);
    auto left_proj_w_tw = convert_to_tensor_wrapper(left_proj_w);
    auto left_proj_b_tw = convert_to_tensor_wrapper(left_proj_b);
    auto left_gate_w_tw = convert_to_tensor_wrapper(left_gate_w);
    auto left_gate_b_tw = convert_to_tensor_wrapper(left_gate_b);
    auto right_proj_w_tw = convert_to_tensor_wrapper(right_proj_w);
    auto right_proj_b_tw = convert_to_tensor_wrapper(right_proj_b);
    auto right_gate_w_tw= convert_to_tensor_wrapper(right_gate_w);
    auto right_gate_b_tw= convert_to_tensor_wrapper(right_gate_b);
    at::Tensor gate_left = input_act.new_empty({c_i, n_res, n_res_gather});
    auto gate_left_tw = convert_to_tensor_wrapper(gate_left);
    kutacc_af2_tm_act_inputs_t_wrapper *left_acts_ptr = new
kutacc_af2_tm_act_inputs_t_wrapper(left_proj_act_tw,
        input_act_tw, gate_left_tw, n_res, n_res_gather);
    kutacc_af2_tm_proj_weights_t_wrapper *left_weights_ptr = new
kutacc_af2_tm_proj_weights_t_wrapper(left_proj_w_tw,
        left_proj_b_tw, left_gate_w_tw, left_gate_b_tw, c_o, c_i);
```

 版权所有 © 华为技术有限公司

```
    kutacc_af2_triangle_multiplication_calc_proj(left_acts_ptr, mask_tw.get_tensor(), left_weights_ptr, false);

    at::Tensor gate_right = input_act.new_empty({c_i, n_res, n_res_gather});
    auto gate_right_tw = convert_to_tensor_wrapper(gate_right);
    kutacc_af2_tm_act_inputs_t_wrapper *right_acts_ptr = new
kutacc_af2_tm_act_inputs_t_wrapper(right_proj_act_tw,
        input_act_tw, gate_right_tw, n_res, n_res_gather);
    kutacc_af2_tm_proj_weights_t_wrapper *right_weights_ptr = new
kutacc_af2_tm_proj_weights_t_wrapper(right_proj_w_tw,
        right_proj_b_tw, right_gate_w_tw, right_gate_b_tw, c_o, c_i);
    kutacc_af2_triangle_multiplication_calc_proj(right_acts_ptr, mask_tw.get_tensor(), right_weights_ptr,
false);
    center_act = act.new_empty({left_proj_act.sizes()[0], n_res_gather, n_res_gather});
    auto center_act_tw = convert_to_tensor_wrapper(center_act);
    kutacc_af2_triangle_multiplication_equation(center_act_tw.get_tensor(), left_proj_act_tw.get_tensor(),
        right_proj_act_tw.get_tensor(), n_res_gather, true);
    center_act = center_act.permute({1, 2, 0}).contiguous();
    center_act = layernorm(center_act, center_ln_w, center_ln_b);
    auto center_act_new_tw = convert_to_tensor_wrapper(center_act);
    auto out_tw = convert_to_tensor_wrapper(out);
    auto gating_w_tw = convert_to_tensor_wrapper(gating_w);
    auto gating_b_tw = convert_to_tensor_wrapper(gating_b);
    auto output_proj_w_tw = convert_to_tensor_wrapper(output_proj_w);
    auto output_proj_b_tw = convert_to_tensor_wrapper(output_proj_b);
    kutacc_af2_tm_linear_weights_t_wrapper *linear_weights_ptr = new
kutacc_af2_tm_linear_weights_t_wrapper(gating_w_tw,
        gating_b_tw, output_proj_w_tw, output_proj_b_tw, c_o, c_i);
    kutacc_af2_triangle_multiplication_gate_and_out_linear(gate_tw.get_tensor(), out_tw.get_tensor(),
left_acts_ptr,
        center_act_new_tw.get_tensor(), linear_weights_ptr, true);
    kutacc_af2_triangle_multiplication_last(out_tw.get_tensor(), gate_tw.get_tensor(), n_res, n_res_gather,
c_o);
    delete left_acts_ptr;
    delete left_weights_ptr;
    delete right_acts_ptr;
    delete right_weights_ptr;
    delete linear_weights_ptr;
    return out;
}
}
```

?.6. kutacc_af2_outer_product_mean_calc_left_and_right_mul

outer_product_mean中用于计算左右投影的计算函数

接口定义

void kutacc_af2_outer_product_mean_calc_left_and_right_mul(kutacc_af2_opm_act_inputs_t *opm_acts_ptr, kutacc_af2_opm_mask_inputs_t *opm_masks_ptr, kutacc_af2_opm_weights_t *opm_weights_ptr);

参数

表1 入参定义

 版权所有 © 华为技术有限公司

参数名	类型	描述	输入/输出
opm_acts_ptr	kutacc_af2_opm_act_inputs_t *	kutacc_af2_opm_act_inputs_t类型的指针，具体数据结构定义见下方表2 kutacc_af2_opm_act_inputs_t 数据结构表定义	输入
opm_masks_ptr	kutacc_af2_opm_mask_inputs_t *	kutacc_af2_opm_mask_inputs_t类型的指针，具体数据结构定义见下方表3 kutacc_af2_opm_mask_inputs_t 数据结构表定义	输入
opm_weights_ptr	kutacc_af2_opm_weights_t *	kutacc_af2_opm_weights_t类型的指针，具体数据结构定义见下方表4 kutacc_af2_opm_weights_t数据结构表定义	输入

表2 kutacc_af2_opm_act_inputs_t 数据结构表定义

参数名	类型	描述	输入/输出
n_seq	int64_t	序列数量	输入
n_res	int64_t	残基数量	输入
input_act	kutacc_tensor_h	输入激活张量	输入
left_proj	kutacc_tensor_h	左投影	输入
right_proj	kutacc_tensor_h	右投影	输入
left_proj_	kutacc_tensor_h	经过掩码处理后的左投影	输入
right_proj_	kutacc_tensor_h	经过掩码处理后的右投影	输入

表3 kutacc_af2_opm_mask_inputs_t 数据结构表定义

参数名	类型	描述	输入/输出
n_res_gather	int64_t	聚合后的残基数量	输入
mask_bias	int64_t	掩码张量地址偏移量	输入

 版权所有 © 华为技术有限公司

参数名	类型	描述	输入/输出
mask	kutacc_tensor_h	掩码张量	输入
norm	kutacc_tensor_h	归一化因子张量	输入

表4 kutacc_af2_opm_weights_t 数据结构表定义

参数名	类型	描述	输入/输出
c_m	int64_t	输入特征维度	输入
c_i	int64_t	投影后的特征维度	输入
c_z	int64_t	输出特征维度	输入
left_proj_w	kutacc_tensor_h	左投影权重	输入
left_proj_b	kutacc_tensor_h	左投影偏移量	输入
right_proj_w	kutacc_tensor_h	右投影权重	输入
right_proj_b	kutacc_tensor_h	右投影偏移量	输入
outer_w	kutacc_tensor_h	输出权重	输入
outer_b	kutacc_tensor_h	输出偏移量	输入

outer_product_mean整数参数应满足的约束关系：

mask_bias >= 0,

c_i, c_m, n_res, n_res_gather, n_seq > 0,

n_seq * n_res <INT64_MAX

单进程时n_res必须等于n_res_gather，多进程下不满足该条件

?.7. kutacc_af2_outer_product_mean_chunk

outer_product_mean中用于分块计算结果的计算函数。

接口定义

```
kutacc_export void
kutacc_af2_outer_product_mean_chunk(kutacc_af2_opm_act_inputs_t
```

 版权所有 © 华为技术有限公司

*opm_acts_ptr, kutacc_af2_opm_mask_inputs_t *opm_masks_ptr, kutacc_af2_opm_weights_t *opm_weights_ptr,kutacc_tensor_h out, int64_t left_block_size, int64_t right_block_size);

参数

表1 入参定义

参数名	类型	描述	输入/输出
opm_acts_ptr	kutacc_af2_opm_act_inputs_t *	kutacc_af2_opm_act_inputs_t类型的指针，具体数据结构定义见下方表2 kutacc_af2_opm_act_inputs_t 数据结构表定义	输入
opm_masks_ptr	kutacc_af2_opm_mask_inputs_t *	kutacc_af2_opm_mask_inputs_t类型的指针，具体数据结构定义见下方表3 kutacc_af2_opm_mask_inputs_t 数据结构表定义	输入
opm_weights_ptr	kutacc_af2_opm_weights_t *	kutacc_af2_opm_weights_t类型的指针，具体数据结构定义见下方表4 kutacc_af2_opm_weights_t数据结构表定义	输入
out	kutacc_tensor_h	输出数据	输出
left_block_size	int64_t	左分块大小	输入
right_block_size	int64_t	右分块大小	输入

表2 kutacc_af2_opm_act_inputs_t 数据结构表定义

参数名	类型	描述	输入/输出
n_seq	int64_t	序列数量	输入
n_res	int64_t	残基数量	输入
input_act	kutacc_tensor_h	输入激活张量	输入
left_proj	kutacc_tensor_h	左投影	输入
right_proj	kutacc_tensor_h	右投影	输入

 版权所有 © 华为技术有限公司

参数名	类型	描述	输入/输出
left_proj_	kutacc_tensor_h	经过掩码处理后的左投影	输入
right_proj_	kutacc_tensor_h	经过掩码处理后的右投影	输入

表3 kutacc_af2_opm_mask_inputs_t 数据结构表定义

参数名	类型	描述	输入/输出
n_res_gather	int64_t	聚合后的残基数量	输入
mask_bias	int64_t	掩码张量地址偏移量	输入
mask	kutacc_tensor_h	掩码张量	输入
norm	kutacc_tensor_h	归一化因子张量	输入

表4 kutacc_af2_opm_weights_t 数据结构表定义

参数名	类型	描述	输入/输出
c_m	int64_t	输入特征维度	输入
c_i	int64_t	投影后的特征维度	输入
c_z	int64_t	输出特征维度	输入
left_proj_w	kutacc_tensor_h	左投影权重	输入
left_proj_b	kutacc_tensor_h	左投影偏移量	输入
right_proj_w	kutacc_tensor_h	右投影权重	输入
right_proj_b	kutacc_tensor_h	右投影偏移量	输入
outer_w	kutacc_tensor_h	输出权重	输入
outer_b	kutacc_tensor_h	输出偏移量	输入

outer_product_mean整数参数应满足的约束关系：

 版权所有 © 华为技术有限公司

n_res, n_res_gather, c_i, c_z, n_res, n_res_gather, left_block_size, right_block_size > 0;

n_seq * n_res <INT64_MAX，

left_block_size * right_block_size * c_i * c_i < INT64_MAX，

left_block_size* right_block_size * c_z < INT64_MAX，

单进程时n_res必须等于n_res_gather 多进程下不满足该条件

在构建用例及使用KPEX时应满足以下算子形状约束

表5 KPEX outer_product_mean入参形状约束

tensor/param	shape/value	描述
input_ln_w	[c_m]	通过layernorm生成input_act所需的权重参数
input_ln_b	[c_m]	通过layernorm生成input_act所需的偏置参数
left_proj_w	[c_i, c_m]	见表4参数left_proj_w
left_proj_b	[c_i]	见表4参数left_proj_b
right_proj_w	[c_i, c_m]	见表4参数right_proj_w
right_proj_b	[c_i]	见表4参数right_proj_b
output_w	[c_z, c_i, c_i]	见表4参数outer_w
output_b	[c_z]	见表4参数outer_b
act	[n_seq, n_res, c_m]	KPEX输入，经过layernorm生成input_act

 版权所有 © 华为技术有限公司

tensor/param	shape/value	描述
mask	[n_seq, n_res_gather]	见表3参数mask
left_block_size	greater than 0 or None	见表1参数left_block_size
right_block_size	greater than 0 or None	见表2参数right_block_size

示例

C++ interface：

```
// test_outer_product_mean.h
#ifndef KPEX_TPP_ALPHAFOLD_TEST_OPM_H
#define KPEX_TPP_ALPHAFOLD_TEST_OPM_H
#include <ATen/core/Tensor.h>
#include <ATen/ops/empty.h>
#include <ATen/ops/ones.h>
#include <ATen/ops/zeros.h>
#include <ATen/ops/full.h>
#include <ATen/native/cpu/utils.h>
#include <c10/core/ScalarType.h>
namespace alphafold {
at::Tensor test_outer_product_mean(int64_t c_i, int64_t c_m, int64_t c_z, int64_t n_seq, int64_t n_res, int64_t
n_res_gather);
}
#endif

// bind.h
#include <torch/extension.h>
#include "test_outer_product_mean.h"
namespace alphafold {
inline void bind(pybind11::module &m)
{
    autosubmodule = m.def_submodule("alphafold");
    submodule.def("test_outer_product_mean", &test_outer_product_mean, py::arg("c_i"), py::arg("c_m"),
py::arg("c_z"), py::arg("n_seq"), py::arg("n_res"), py::arg("n_res_gather"));
}
}

// test.py
import copy
import time
import types
import torch
from torch import nn
import numpy as np
import torch.distributed as dist
import kpex._C as kernel
import kpex
import os

def test_triangle_multiplication(n_res, n_res_gather, c_o, c_i):
    out = kernel.alphafold.test_triangle_multiplication(n_res, n_res_gather, c_o, c_i)
    return out

// test_outer_product_mean.cpp
#include "test_outer_product_mean.h"
```

 版权所有 © 华为技术有限公司

```
#include "kutacc.h"
#include "outer_product_mean.h"
#include "utils/memory.h"
#include "utils/layernorm.h"
#include <utils/TensorWrapper.h>
namespace alphafold {
at::Tensor test_outer_product_mean(int64_t c_i, int64_t c_m, int64_t c_z, int64_t n_seq, int64_t n_res, int64_t
n_res_gather)
{
    float a = 0.2f;
    float b = 0.5f;
    float c = 1.5f;
    float d = 2.0f;
    at::Tensor act = at::full({n_seq, n_res, c_m}, d,
at::TensorOptions().device(kpex::device()).dtype(at::kBFloat16));
    at::Tensor mask = at::ones({n_res_gather, n_seq},
at::TensorOptions().device(kpex::device()).dtype(at::kBFloat16));
    at::Tensor left_proj = act.new_empty({c_i, n_res, n_seq});
    at::Tensor right_proj = act.new_empty({c_i, n_res, n_seq});
    at::Tensor left_proj_ = act.new_empty({n_res, c_i, n_seq});
    at::Tensor right_proj_ = act.new_empty({n_res, c_i, n_seq});
    at::Tensor norm = mask.new_empty({n_res, n_res_gather});
    int64_t mask_bias = 0;
    at::Tensor input_ln_w = at::full({c_m}, a, at::TensorOptions().device(kpex::device()).dtype(at::kFloat));
    at::Tensor input_ln_b = at::full({c_m}, b, at::TensorOptions().device(kpex::device()).dtype(at::kFloat));
    at::Tensor left_proj_w = linear_weight_prepack(at::full({c_i, c_m}, c,
at::TensorOptions().device(kpex::device()).dtype(at::kBFloat16)));
    at::Tensor left_proj_b = at::zeros({c_i}, at::TensorOptions().device(kpex::device()).dtype(at::kFloat));
    at::Tensor right_proj_w = linear_weight_prepack(at::ones({c_i, c_m},
at::TensorOptions().device(kpex::device()).dtype(at::kBFloat16)));
    at::Tensor right_proj_b = at::zeros({c_i}, at::TensorOptions().device(kpex::device()).dtype(at::kFloat));
    at::Tensor output_w = linear_weight_prepack(at::ones({c_z, c_i * c_i},
at::TensorOptions().device(kpex::device()).dtype(at::kBFloat16)));
    at::Tensor output_b = at::zeros({c_z}, at::TensorOptions().device(kpex::device()).dtype(at::kFloat));
    at::Tensor input_act = layernorm(act.transpose(0, 1), input_ln_w, input_ln_b);
    at::Tensor out = act.new_empty({n_res, n_res_gather, c_z});
    kutacc::TensorWrapper input_act_tw = convert_to_tensor_wrapper(input_act);
    kutacc::TensorWrapper mask_tw = convert_to_tensor_wrapper(mask);
    kutacc::TensorWrapper left_proj_w_tw = convert_to_tensor_wrapper(left_proj_w);
    kutacc::TensorWrapper left_proj_b_tw = convert_to_tensor_wrapper(left_proj_b);
    kutacc::TensorWrapper right_proj_w_tw = convert_to_tensor_wrapper(right_proj_w);
    kutacc::TensorWrapper right_proj_b_tw = convert_to_tensor_wrapper(right_proj_b);
    kutacc::TensorWrapper left_proj_tw = convert_to_tensor_wrapper(left_proj);
    kutacc::TensorWrapper right_proj_tw = convert_to_tensor_wrapper(right_proj);
    kutacc::TensorWrapper left_proj_tw_ = convert_to_tensor_wrapper(left_proj_);
    kutacc::TensorWrapper right_proj_tw_ = convert_to_tensor_wrapper(right_proj_);
    kutacc::TensorWrapper norm_tw = convert_to_tensor_wrapper(norm);
    kutacc::TensorWrapper output_w_tw = convert_to_tensor_wrapper(output_w);
    kutacc::TensorWrapper output_b_tw = convert_to_tensor_wrapper(output_b);
    kutacc::TensorWrapper out_tw = convert_to_tensor_wrapper(out);
    int64_t left_block_size = 1024;
    int64_t right_block_size = 1024;
    kutacc_af2_opm_weights_t_wrapper *opm_weights_ptr = new
kutacc_af2_opm_weights_t_wrapper(left_proj_w_tw, left_proj_b_tw, right_proj_w_tw, right_proj_b_tw,
        output_w_tw, output_b_tw, c_m, c_i, c_z);
    kutacc_af2_opm_act_inputs_t_wrapper *opm_inputs_ptr = new
kutacc_af2_opm_act_inputs_t_wrapper(input_act_tw, left_proj_tw, right_proj_tw, left_proj_tw_,
        right_proj_tw_, n_seq, n_res);
    kutacc_af2_opm_mask_inputs_t_wrapper *opm_mask_ptr = new
kutacc_af2_opm_mask_inputs_t_wrapper(mask_tw, norm_tw, n_res_gather, mask_bias);
    kutacc_af2_outer_product_mean_calc_left_and_right_mul(opm_inputs_ptr, opm_mask_ptr,
opm_weights_ptr);
    kutacc_af2_outer_product_mean_chunk(opm_inputs_ptr, opm_mask_ptr, opm_weights_ptr,
out_tw.get_tensor(), left_block_size, right_block_size);
    delete opm_inputs_ptr;
    delete opm_weights_ptr;
    delete opm_mask_ptr;
    return out;
```

 版权所有 © 华为技术有限公司

```
}
}
```

?.8. kutacc_af2_global_attention

在AF2中，global_attention是一种全局的注意力机制，通过对多序列比对过程中列方向的目标残基的信息进行整合，会将来自首个序列的三维结构信息传递给其他序列，从而更好地理解该残基在不同进化背景下所起的作用，以及残基之间的协同进化关系等，进而为蛋白质结构预测提供更丰富的信息。

接口定义

void kutacc_af2_global_attention(kutacc_af2_attention_inputs_t *q_based_ptr, kutacc_tensor_h q_data, kutacc_tensor_h q_mask, kutacc_af2_attention_weights_t *weight_ptr, kutacc_tensor_h out);

参数

表1 入参定义

参数名	类型	描述	输入/输出
q_based_ptr	kutacc_af2_attention_inputs_t *	kutacc_af2_attention_inputs_t类型的指针，数据结构表定义见下方表2 kutacc_af2_attention_inputs_t数据结构表定义	输入
q_data	kutacc_tensor_h	Q矩阵数据	输入
q_mask	kutacc_tensor_h	Q掩码矩阵	输入
weight_ptr	kutacc_af2_attention_weights_t *	kutacc_af2_attention_weights_t类型的指针，数据结构表定义见下方表3 kutacc_af2_attention_weights_t数据结构表	输入
out	kutacc_tensor_h	输出数据	输出

表2 kutacc_af2_attention_inputs_t数据结构表定义

参数名	类型	描述
gate	kutacc_tensor_h	输入中间层门控张量数据
k	kutacc_tensor_h	输入中间层k数据
v	kutacc_tensor_h	输入中间层v数据
q	kutacc_tensor_h	输入中间层q数据
avg	kutacc_tensor_h	权重avg数据

 版权所有 © 华为技术有限公司

batch	int64_t	输入批次
seq_len	int64_t	输入数据的序列长度

表3 kutacc_af2_attention_weights_t数据结构表

参数名	类型	描述
nchannels	int64_t	输入权重的总数
nheads	int64_t	输入权重的head数
head_size	int64_t	输入权重每个head中的数据数量
query_w	kutacc_tensor_h	权重query_w数据
key_w	kutacc_tensor_h	权重key_w数据
gating_w	kutacc_tensor_h	权重gating_w数据
gating_b	kutacc_tensor_h	门控偏移量数据
output_w	kutacc_tensor_h	权重output_w数据
output_b	kutacc_tensor_h	输出矩阵乘法偏移量数据
value_w	kutacc_tensor_h	权重value_w数据

global_attention整数参数应满足的约束关系：

nchannels = nheads * head_size，

batch, seq_len, nchannels, nheads, head_size > 0

在满足上述条件的同时应满足nchannels=64

在构建用例及使用KPEX时应满足以下算子形状约束

表4 global attention入参形状约束

tensor	shape
query_w	[nheads, head_size, nchannels]
key_w	[head_size, nchannels]
value_w	[head_size, nchannels]
gating_w	[nheads, head_size, nchannels]
gating_b	[nheads, head_size]
output_w	[nchannels, nheads, head_size]

 版权所有 © 华为技术有限公司

tensor	shape
output_b	[nchannels]
q_data	[batch, seq_len, nchannels]
m_data	[batch, seq_len, nchannels]
q_mask	[batch, seq_len, 1]

示例

C++ interface：

```
//test_global_attention.h
#ifndef KPEX_TPP_ALPHAFOLD_TEST_GLOBAL_ATTENTION_H
#define KPEX_TPP_ALPHAFOLD_TEST_GLOBAL_ATTENTION_H
#include <ATen/core/Tensor.h>
#include <ATen/ops/empty.h>
#include <ATen/ops/ones.h>
#include <ATen/ops/zeros.h>
#include <ATen/ops/full.h>
#include <ATen/native/cpu/utils.h>
#include <c10/core/ScalarType.h>
#include "common_header.h"
namespace alphafold {
at::Tensor test_global_attention(int64_t batch, int64_t seq_len, int64_t nchannels, int64_t nheads, int64_t
head_size);
}
#endif

//bind.h
#include <torch/extension.h>
#include "test_global_attention.h"
namespace alphafold {
inline void bind(pybind11::module &m)
{
    auto submodule = m.def_submodule("alphafold");
    submodule.def("test_global_attention", &test_global_attention, py::arg("batch"), py::arg("seq_len"),
py::arg("nchannels"), py::arg("nheads"), py::arg("head_size"));
}

//test_global_attention.cpp
#include "kutacc.h"
#include <utils/memory.h>
#include <utils/bf16.h>
#include <utils/TensorWrapper.h>
#include "test_global_attention.h"
namespace alphafold {
at::Tensor test_global_attention(int64_t batch, int64_t seq_len, int64_t nchannels, int64_t nheads, int64_t
head_size) {
    float a = 0.75f;
    float b = 0.25f;
    float c = 0.5f;
    at::Tensor q_data = at::ones({batch, seq_len, nchannels},
at::TensorOptions().device(kpex::device()).dtype(at::kBFloat16));
    at::Tensor out = at::empty(q_data.sizes(), q_data.options());
    at::Tensor q_mask = at::ones({batch, seq_len, 1}, q_data.options());
    q_mask = q_mask.contiguous();
    at::Tensor q_avg = q_data.new_empty({batch, nchannels});
    at::Tensor q = q_data.new_empty({batch, nheads, head_size});
    at::Tensor k = q_data.new_empty({batch, seq_len, head_size});
    at::Tensor v = q_data.new_empty({head_size, batch, seq_len});
    at::Tensor gate = q_data.new_empty({batch, seq_len, nheads, head_size});
    at::Tensor query_w = at::full({nheads, head_size, nchannels}, a, q_data.options()).to(at::kBFloat16);
```

 版权所有 © 华为技术有限公司

```
    auto bf16_opt = query_w.options().device(kpex::device()).dtype(at::kBFloat16);
    auto float_opt = query_w.options().device(kpex::device()).dtype(at::kFloat);
    at::Tensor key_w = at::full({head_size, nchannels}, b,
at::TensorOptions().device(kpex::device()).dtype(at::kBFloat16));
    at::Tensor value_w = at::full({head_size, nchannels}, a,
at::TensorOptions().device(kpex::device()).dtype(at::kBFloat16));
    at::Tensor gating_w = at::full({nheads, head_size, nchannels}, a,
at::TensorOptions().device(kpex::device()).dtype(at::kBFloat16));
    at::Tensor gating_b = at::full({nheads, head_size}, c,
at::TensorOptions().device(kpex::device()).dtype(at::kFloat));
    at::Tensor output_w = at::full({nchannels, nheads, head_size}, b,
at::TensorOptions().device(kpex::device()).dtype(at::kBFloat16));
    at::Tensor output_b = at::full({nchannels}, c, at::TensorOptions().device(kpex::device()).dtype(at::kFloat));
    query_w = query_w.to(bf16_opt).contiguous().view({nchannels, nchannels});
    key_w = key_w.to(bf16_opt).contiguous().view({head_size, nchannels});
    value_w = value_w.to(bf16_opt).contiguous().view({head_size, nchannels});
    gating_w = gating_w.to(bf16_opt).contiguous().view({nchannels, nchannels});
    output_w = output_w.to(bf16_opt).contiguous().view({nchannels, nchannels});
    auto query_w_res = linear_weight_prepack(query_w);
    auto key_w_res = linear_weight_prepack(key_w);
    auto value_w_res = linear_weight_prepack(value_w);
    auto gating_w_res = linear_weight_prepack(gating_w);
    auto output_w_res = linear_weight_prepack(output_w);
    kutacc::TensorWrapper q_avg_tw = convert_to_tensor_wrapper(q_avg);
    kutacc::TensorWrapper q_tw = convert_to_tensor_wrapper(q);
    kutacc::TensorWrapper k_tw = convert_to_tensor_wrapper(k);
    kutacc::TensorWrapper v_tw = convert_to_tensor_wrapper(v);
    kutacc::TensorWrapper gate_tw = convert_to_tensor_wrapper(gate);
    kutacc::TensorWrapper q_data_tw = convert_to_tensor_wrapper(q_data);
    kutacc::TensorWrapper q_mask_tw = convert_to_tensor_wrapper(q_mask);
    kutacc::TensorWrapper out_tw = convert_to_tensor_wrapper(out);
    kutacc::TensorWrapper query_w_tw = convert_to_tensor_wrapper(query_w_res);
    kutacc::TensorWrapper key_w_tw = convert_to_tensor_wrapper(key_w_res);
    kutacc::TensorWrapper value_w_tw = convert_to_tensor_wrapper(value_w_res);
    kutacc::TensorWrapper gating_w_tw = convert_to_tensor_wrapper(gating_w_res);
    kutacc::TensorWrapper gating_b_tw = convert_to_tensor_wrapper(gating_b);
    kutacc::TensorWrapper output_w_tw = convert_to_tensor_wrapper(output_w_res);
    kutacc::TensorWrapper output_b_tw = convert_to_tensor_wrapper(output_b);
    kutacc_af2_attention_weights_t_wrapper *global_attention_weight_ptr = new
kutacc_af2_attention_weights_t_wrapper(query_w_tw, key_w_tw, value_w_tw, gating_w_tw, gating_b_tw,
        output_w_tw, output_b_tw, nchannels, nheads, head_size);
    kutacc_af2_attention_inputs_t_wrapper *global_attention_q_ptr = new
kutacc_af2_attention_inputs_t_wrapper(q_tw, k_tw, v_tw, gate_tw, q_avg_tw, batch, seq_len);
    kutacc_af2_global_attention(global_attention_q_ptr, q_data_tw.get_tensor(), q_mask_tw.get_tensor(),
global_attention_weight_ptr, out_tw.get_tensor());
    delete global_attention_weight_ptr;
    delete global_attention_q_ptr;
    return out;
}
}

//test.py
import torch
import kpex._C as kernel
def test_global_attention(batch, seq_len, nchannels, nheads, head_size):
    out = kernel.alphafold.test_global_attention(batch, seq_len, nchannels, nheads, head_size)
    return out

//output:
tensor([[[768., 768., 768., 768., 768., 768., 768., 768., 768., 768., 768.,
          768., 768., 768., 768., 768., 768., 768., 768., 768., 768., 768.,
          768., 768., 768., 768., 768., 768., 768., 768., 768., 768., 768.,
          768., 768., 768., 768., 768., 768., 768., 768., 768., 768., 768.,
          768., 768., 768., 768., 768., 768., 768., 768., 768., 768., 768.,
          768., 768., 768., 768., 768., 768., 768., 768., 768.],
         [768., 768., 768., 768., 768., 768., 768., 768., 768., 768., 768.,
          768., 768., 768., 768., 768., 768., 768., 768., 768., 768., 768.,
          768., 768., 768., 768., 768., 768., 768., 768., 768., 768., 768.,
          768., 768., 768., 768., 768., 768., 768., 768., 768., 768., 768.,
          768., 768., 768., 768., 768., 768., 768., 768., 768., 768., 768.,
```

 版权所有 © 华为技术有限公司

```
      768., 768., 768., 768., 768., 768., 768., 768., 768., 768.,
      768., 768., 768., 768., 768., 768., 768., 768., 768.]],

    [[768., 768., 768., 768., 768., 768., 768., 768., 768., 768.,
      768., 768., 768., 768., 768., 768., 768., 768., 768., 768.,
      768., 768., 768., 768., 768., 768., 768., 768., 768., 768.,
      768., 768., 768., 768., 768., 768., 768., 768., 768., 768.,
      768., 768., 768., 768., 768., 768., 768., 768., 768., 768.,
      768., 768., 768., 768., 768., 768., 768., 768., 768.],
     [768., 768., 768., 768., 768., 768., 768., 768., 768., 768.,
      768., 768., 768., 768., 768., 768., 768., 768., 768., 768.,
      768., 768., 768., 768., 768., 768., 768., 768., 768., 768.,
      768., 768., 768., 768., 768., 768., 768., 768., 768., 768.,
      768., 768., 768., 768., 768., 768., 768., 768., 768., 768.,
      768., 768., 768., 768., 768., 768., 768., 768., 768.]]],
    dtype=torch.bfloat16)
```

?.9. kutacc_af2_invariant_point

利用注意力机制来保持特征点的不变性，从而提高模型在蛋白质结构预测中的性能。通过对输入特征进行加权聚合，模块能够有效地捕获特征之间的关系，并精确地调整蛋白质原子的坐标，从而提高结构预测的准确性和稳定性。

接口定义

void kutacc_af2_invariant_point(kutacc_af2_ipa_s_inputs_t *ipa_s_ptrs, kutacc_af2_ipa_o_inputs_t *ipa_o_ptrs, kutacc_tensor_h z, kutacc_tensor_h rigid_rot_mats, kutacc_tensor_h rigid_trans, kutacc_tensor_h mask, kutacc_af2_ipa_weights_t *ipa_weight_ptrs);

参数

表1 入参说明

参数名	类型	描述	输入/输出
ipa_s_ptrs	kutacc_af2_ipa_s_inputs_t *	kutacc_af2_ipa_s_inputs_t 类型的指针，具体定义见表2 kutacc_af2_ipa_s_inputs_t数据结构表定义	输入
ipa_o_ptrs	kutacc_af2_ipa_o_inputs_t *	kutacc_af2_ipa_o_inputs_t类型的指针，具体定义见表3 kutacc_af2_ipa_o_inputs_t数据结构表定义	输入
z	kutacc_tensor_h	中间变量	输入
rigid_rot_mats	kutacc_tensor_h	刚体变换：旋转参数	输入
rigid_trans	kutacc_tensor_h	刚体变换：平移参数	输入
mask	kutacc_tensor_h	掩码，用于忽略无效残基	输入

 版权所有 © 华为技术有限公司

参数名	类型	描述	输入/输出
ipa_weight_ptrs	kutacc_af2_ipa_weights_t *	kutacc_af2_ipa_weights_t类型的指针，具体定义见表4 kutacc_af2_ipa_weights_t数据结构表定义	输入

表2 kutacc_af2_ipa_s_inputs_t数据结构表定义

参数名	类型	描述	输入/输出
n_res	int64_t	输入向量s的第0维长度	输入
a	kutacc_tensor_h	用于保存q转置与k的矩阵乘，进行softmax后的临时结果	输入
b	kutacc_tensor_h	用于保存z向量进行线性变换后的临时结果	输入
q	kutacc_tensor_h	输入s经过线性变化生成的Q矩阵	输入
k	kutacc_tensor_h	K矩阵	输入
v	kutacc_tensor_h	V矩阵	输入
q_pts	kutacc_tensor_h	查询矩阵的3D点	输入
k_pts	kutacc_tensor_h	键矩阵的3D点	输入
v_pts	kutacc_tensor_h	值矩阵的3D点	输入

表3 kutacc_af2_ipa_o_inputs_t数据结构表定义

参数名	类型	描述	输入/输出
o	kutacc_tensor_h	用于保存qkv，attention计算结果的向量	输入
o_pt	kutacc_tensor_h	用于保存pts相关向量attention计算结果的向量	输入
o_pt_norm	kutacc_tensor_h	用于保存需要进行layernorm变换的向量	输入

 版权所有 © 华为技术有限公司

参数名	类型	描述	输入/输出
o_pair	kutacc_tensor_h	残基对特征	输入

表4 kutacc_af2_ipa_weights_t数据结构表定义

参数名	类型	描述	输入/输出
c_z	int64_t	单一single的长度	输入
c_hidden	int64_t	隐藏层长度	输入
no_heads	int64_t	attention头数量	输入
no_qk_points	int64_t	需要生成的q/k数量	输入
no_v_points	int64_t	需要生成的v数量	输入
head_weights	kutacc_tensor_h	注意力头的权重	输入
weights_head_weights	kutacc_tensor_h	注意力头的权重	输入
linear_b_w	kutacc_tensor_h	对b进行线性变化时的权重	输入
linear_b_b	kutacc_tensor_h	对b进行线性变化时的偏置	输入

整数参数应满足的约束关系： n_res, c_z, c_hidden, no_heads, no_qk_points, no_v_points > 0;

no_heads * c_hidden < INT64_MAX; no_heads * no_v_points * 3 <INT64_MAX;

no_heads*(c_hidden+no_v_points*3) <INT64_MAX; no_heads*(c_hidden +no_v_points*4) <INT64_MAX; no_heads * c_z < INT64_MAX;

no_heads * c_hidden * 2 * c_s < INT64_MAX; no_heads * 3 *no_qk_points * c_s < INT64_MAX;

3 * no_heads * (no_qk_points+ no_v_points) * c_s <INT64_MAX;

no_heads * 2 * c_hidden = c_s

no_qk_points * 3 < 16

构建KPEX层所需tensor参数的约束如下：

tensor	shape	描述
s	[n_res, c_s]	输入张量

 版权所有 © 华为技术有限公司

tensor	shape	描述
z	[n_res, n_res, c_z]	输入张量，见入参说明参数z
rigid_trans	[n_res, 3]	输入张量，见入参说明参数rigid_trans
rigid_rot_mats	[n_res, 3, 3]	输入张量，见入参说明rigid_rot_mats
mask	[n_res]	输入张量，见入参说明参数mask
linear_q_w	[no_heads * c_hidden, c_s]	将s通过线性变化生成q的权重
linear_q_b	[no_heads * c_hidden]	将s通过线性变化生成q的偏置
linear_kv_w	[no_heads * 2 * c_hidden, c_s]	将s通过线性变化生成k, v的权重
linear_kv_b	[no_heads * 2 * c_hidden]	将s通过线性变化生成k, v的偏置
linear_q_points_w	[no_heads * no_qk_points * 3, c_s]	将s通过线性变化生成q_pts的权重
linear_q_points_b	[no_heads * no_qk_points *3]	将s通过线性变化生成q_pts的偏置
linear_kv_points_w	[3 * no_heads * (no_qk_points + no_v_points), c_s]	将s通过线性变化生成k_pts, v_pts的权重
linear_kv_points_b	[3 * no_heads * (no_qk_points + no_v_points)]	将s通过线性变化生成k_pts, v_pts的偏置
linear_b_w	[no_heads, c_z]	见表4 kutacc_af2_ipa_weights_t数据结构表定义参数linear_b_w

 版权所有 © 华为技术有限公司

tensor	shape	描述
linear_b_b	[no_heads]	见表4 kutacc_af2_ipa_weights_t数据结构表定义参数linear_b_b
head_weights	[no_heads]	见表4 kutacc_af2_ipa_weights_t数据结构表定义参数head_weights
linear_out_w	[c_s, no_heads * (c_hidden + no_v_points * 4 + c_z)]	见表4 kutacc_af2_ipa_weights_t数据结构表定义参数linear_out_w
linear_out_b	[c_s]	见表4 kutacc_af2_ipa_weights_t数据结构表定义参数linear_out_b

示例

C++ interface：

```
//test_invariant.h
#ifndef KPEX_TPP_ALPHAFOLD_TEST_INVARIAN_H

#define KPEX_TPP_ALPHAFOLD_TEST_INVARIAN_H
#include <ATen/core/Tensor.h>
#include <ATen/ops/empty.h>
#include <ATen/ops/ones.h>
#include <ATen/ops/zeros.h>
#include <ATen/ops/full.h>
#include <ATen/native/cpu/utils.h>
#include <c10/core/ScalarType.h>
namespace alphafold {
at::Tensor test_invariant_point_attention(int64_t n_res, int64_t no_heads, int64_t c_hidden, int64_t
no_qk_points, int64_t no_v_points, int64_t c_z, int64_t c_s);
}
#endif

//bind.h
#include "test_invariant_point.h"
namespace alphafold {

inline void bind(pybind11::module &m)
{
    auto submodule = m.def_submodule("alphafold");
    submodule.def("test_invariant_point_attention", &test_invariant_point_attention, py::arg("n_res"),
py::arg("no_heads"), py::arg("c_hidden"), py::arg("no_qk_points"),
        py::arg("no_v_points"), py::arg("c_z"), py::arg("c_s"));
}
}
```

 版权所有 © 华为技术有限公司

```
//test_invariant.cpp
#include "test_invariant_point.h"

#include "utils/linear.h"
#include "rigid.h"
#include "kutacc.h"
#include "utils/memory.h"
#include "invariant_point.h"
namespace alphafold {
at::Tensor test_invariant_point_attention(int64_t n_res, int64_t no_heads, int64_t c_hidden, int64_t
no_qk_points, int64_t no_v_points, int64_t c_z, int64_t c_s)
{
    float a = 0.6f;
    float b = 0.25f;
    float c = 0.1f;
    float d = 0.2f;
    float e = 0.5f;
    at::Tensor s = at::full({n_res, c_s}, a, at::TensorOptions().device(kpex::device()).dtype(at::kBFloat16));
    at::Tensor out = at::empty(s.sizes(), s.options());
    at::Tensor rigid_trans = at::ones({n_res, 1, 1, 3}, s.options()).to(at::kFloat);
    at::Tensor rigid_rot_mats = at::ones({n_res, 1, 1, 3, 3}, s.options()).to(at::kFloat);
    at::Tensor linear_q_w = at::full({no_heads, c_hidden, c_s}, b, s.options());
    at::Tensor linear_q_b = at::full({no_heads, c_hidden}, c, rigid_rot_mats.options());
    at::Tensor linear_k_w = at::ones({no_heads, c_hidden, c_s}, s.options());
    at::Tensor linear_k_b = at::zeros({no_heads, c_hidden}, rigid_rot_mats.options());
    at::Tensor linear_v_w = at::ones({no_heads, c_hidden, c_s}, s.options());
    at::Tensor linear_v_b = at::zeros({no_heads, c_hidden}, rigid_rot_mats.options());
    at::Tensor linear_q_points_w = at::full({no_heads, no_qk_points, 3, c_s}, c, s.options());
    at::Tensor linear_q_points_b = at::ones({no_heads, no_qk_points, 3}, rigid_rot_mats.options());
    at::Tensor linear_k_points_w = at::full({no_heads, no_qk_points, 3, c_s}, d, s.options());
    at::Tensor linear_k_points_b = at::full({no_heads, no_qk_points, 3}, d, rigid_rot_mats.options());
    at::Tensor linear_v_points_w = at::ones({no_heads, no_v_points, 3, c_s}, s.options());
    at::Tensor linear_v_points_b = at::zeros({no_heads, no_v_points, 3}, rigid_rot_mats.options());
    at::Tensor linear_b_w = at::ones({no_heads, c_z}, s.options());
    at::Tensor linear_b_b = at::ones({no_heads}, rigid_rot_mats.options());
    at::Tensor head_weights = at::ones({no_heads}, rigid_rot_mats.options());
    at::Tensor linear_out_w = at::ones({c_s, no_heads * (c_hidden + no_v_points * 4 + c_z)}, s.options());
    at::Tensor linear_out_b = at::full({c_s}, e, rigid_rot_mats.options());
    at::Tensor mask = at::ones({n_res}, s.options());
    at::Tensor z = at::ones({n_res, n_res, c_z}, s.options());
    at::Tensor q = linear(s, linear_q_w, linear_q_b);
    at::Tensor k = linear(s, linear_k_w, linear_k_b);
    at::Tensor v = linear(s, linear_v_w, linear_v_b);
    at::Tensor q_pts = linear(s, linear_q_points_w, linear_q_points_b);
    q_pts = rigid_rot_vec_mul(q_pts, rigid_rot_mats, rigid_trans);
    at::Tensor k_pts = linear(s, linear_k_points_w, linear_k_points_b);
    k_pts = rigid_rot_vec_mul(k_pts, rigid_rot_mats, rigid_trans);
    at::Tensor v_pts = linear(s, linear_v_points_w, linear_v_points_b);
    v_pts = rigid_rot_vec_mul(v_pts, rigid_rot_mats, rigid_trans);
    v_pts = v_pts.permute({1, 2, 3, 0}).contiguous();
    at::Tensor m = at::empty({no_heads, n_res, n_res}, s.options()); // b
    at::Tensor n = at::empty({no_heads, n_res, n_res}, q.options()); // a
    at::Tensor head_weights_2 = at::empty(head_weights.sizes(), head_weights.options());
    at::Tensor collect = at::empty({n_res, no_heads * (c_hidden + no_v_points * 4 + c_z)}, s.options());
    at::Tensor o = collect.narrow(1, 0, no_heads * c_hidden).view({n_res, no_heads, c_hidden});
    at::Tensor o_pt = collect.narrow(1, no_heads * c_hidden, no_heads * no_v_points * 3).view({n_res, 3,
no_heads, no_v_points});
    at::Tensor o_pt_norm = collect.narrow(1, no_heads * (c_hidden + no_v_points * 3), no_heads *
no_v_points).view({n_res, no_heads, no_v_points});
    at::Tensor o_pair = collect.narrow(1, no_heads * (c_hidden + no_v_points * 4), no_heads *
c_z).view({n_res, no_heads, c_z});
    auto q_tw = convert_to_tensor_wrapper(q);
    auto k_tw = convert_to_tensor_wrapper(k);
    auto v_tw = convert_to_tensor_wrapper(v);
    auto q_pts_tw = convert_to_tensor_wrapper(q_pts);
    auto k_pts_tw = convert_to_tensor_wrapper(k_pts);
    auto v_pts_tw = convert_to_tensor_wrapper(v_pts);
    auto m_tw = convert_to_tensor_wrapper(m);
    auto n_tw = convert_to_tensor_wrapper(n);
```

 版权所有 © 华为技术有限公司

```
    auto head_weights_tw = convert_to_tensor_wrapper(head_weights);
    auto weights_head_weights_tw = convert_to_tensor_wrapper(head_weights_2);
    auto o_tw = convert_to_tensor_wrapper(o);
    auto o_pt_tw = convert_to_tensor_wrapper(o_pt);
    auto o_pt_norm_tw = convert_to_tensor_wrapper(o_pt_norm);
    auto o_pair_tw = convert_to_tensor_wrapper(o_pair);
    auto z_tw = convert_to_tensor_wrapper(z);
    auto rigid_rot_mats_tw = convert_to_tensor_wrapper(rigid_rot_mats);
    auto rigid_trans_tw = convert_to_tensor_wrapper(rigid_trans);
    auto mask_tw = convert_to_tensor_wrapper(mask);
    auto linear_b_w_tw = convert_to_tensor_wrapper(linear_b_w);
    auto linear_b_b_tw = convert_to_tensor_wrapper(linear_b_b);
    kutacc_af2_ipa_weights_t_wrapper *ipa_weight_ptr = new
kutacc_af2_ipa_weights_t_wrapper(head_weights_tw, weights_head_weights_tw, linear_b_w_tw,
linear_b_b_tw, c_z, c_hidden, no_heads,
        no_qk_points, no_v_points);
    kutacc_af2_ipa_s_inputs_t_wrapper *ipa_s_ptrs = new kutacc_af2_ipa_s_inputs_t_wrapper(n_tw, m_tw,
q_tw, k_tw, v_tw, q_pts_tw, k_pts_tw, v_pts_tw, n_res);
    kutacc_af2_ipa_o_inputs_t_wrapper *ipa_o_ptrs = new kutacc_af2_ipa_o_inputs_t_wrapper(o_tw, o_pt_tw,
o_pt_norm_tw, o_pair_tw);
    if (unlikely(ipa_s_ptrs == nullptr || ipa_o_ptrs == nullptr || ipa_weight_ptr == nullptr)) {
        return out;
    }
    kutacc_af2_invariant_point(ipa_s_ptrs, ipa_o_ptrs, z_tw.get_tensor(), rigid_rot_mats_tw.get_tensor(),
rigid_trans_tw.get_tensor(), mask_tw.get_tensor(), ipa_weight_ptr);
    out = linear(collect, linear_out_w, linear_out_b);
    delete ipa_weight_ptr;
    delete ipa_s_ptrs;
    delete ipa_o_ptrs;
    return out;
}
}

//test.py
import torch
import kpex._C as kernel
def test_invariant_point_attention(n_res, no_heads, c_hidden, no_qk_points, no_v_points, c_z, c_s):
    out = kernel.alphafold.test_invariant_point_attention(n_res, no_heads, c_hidden, no_qk_points,
no_v_points, c_z, c_s)
    return out

//input : 4, 2, 2, 3, 8, 2, 8
//output:
>>> kpex.tpp.alphafold.alphafold.test_invariant_point_attention(4, 2, 2, 3, 8, 2, 8)
tensor([[[3296., 3296., 3296., 3296., 3296., 3296., 3296., 3296.],
         [3296., 3296., 3296., 3296., 3296., 3296., 3296., 3296.],
         [3296., 3296., 3296., 3296., 3296., 3296., 3296., 3296.],
         [3296., 3296., 3296., 3296., 3296., 3296., 3296., 3296.]],
        dtype=torch.bfloat16)
```

?.10. kutacc_af2_rigid_rot_vec_mul

rigid_rot_vec_mul实现矩阵乘向量，输入为矩阵和向量，输出为向量。

接口定义

void kutacc_af2_rigid_rot_vec_mul(kutacc_tensor_h pts, kutacc_tensor_h rot_mats, kutacc_tensor_h out, kutacc_tensor_h trans);

参数

参数名	类型	描述	输入/输出	tensor shape
pts	kutacc_tensor_h	输入数据	输入	[..., 3]

 版权所有 © 华为技术有限公司

参数名	类型	描述	输入/输出	tensor shape
rot_mats	kutacc_tensor_h	旋转矩阵	输入	[..., 3, 3]
out	kutacc_tensor_h	输出数据	输出	[..., 3]
trans	kutacc_tensor_h	平移向量	输入	[..., 3]

kutacc_af2_rigid_rot_vec_mul参数的约束关系： dim = pts.dim - 1; pts.strides[dim] = 1, rot_mats.strides[dim] = 3, rot_mats.strides[dim + 1] = 1 trans.strides[dim] = 1

?.11. kutacc_af2_rigid_rot_matmul

rigid_rot_matmul实现矩阵乘矩阵，输入为两个矩阵，输出为矩阵。

接口定义

void kutacc_af2_rigid_rot_matmul(kutacc_tensor_h a, kutacc_tensor_h b, kutacc_tensor_h out);

参数

参数名	类型	描述	输入/输出	tensor shape
a	kutacc_tensor_h	第一个旋转矩阵	输入	[..., 3, 3]
b	kutacc_tensor_h	第二个旋转矩阵	输入	[..., 3, 3]
out	kutacc_tensor_h	输出数据	输出	[..., 3, 3]

kutacc_af2_rigid_rot_matmul参数的约束关系：dim = a.dim - 2; a.strides[dim] = 3; a.strides[dim +1] = 1; b.strides[dim] = 3; b.strides[dim + 1] = 1

示例

rigid算子均用在invariant_point_attention算子内部，具体用例应参考8.1.2.2.2.1.1.9环节的invariant_point_attention算子的示例

?.12. kutacc_af2_transition

transition算子：输入act张量，使用layernorm将输入归一化，在进行linear特性扩展，使用relu进行非线性激活，最后再进行linear特征压缩，然后返回输出

接口定义

kutacc_export void kutacc_af2_transition(kutacc_af2_trans_act_inputs_t *trans_inputs_ptr, kutacc_af2_trans_weights_t * trans_weights_ptr, kutacc_tensor_h out);

 版权所有 © 华为技术有限公司

参数

表1 参数定义

参数名	类型	描述	输入/输出
trans_inputs_ptr	kutacc_af2_trans_act_inputs_t *	kutacc_af2_trans_act_inputs_t类型的指针，结构体定义见表2 kutacc_af2_trans_act_inputs_t数据结构表定义	输入
trans_weights_ptr	kutacc_af2_trans_weights_t *	kutacc_af2_trans_weights_t类型的指针，结构体定义见表3 kutacc_af2_trans_weights_t数据结构表定义	输入
out	kutacc_tensor_h	输出数据	输出

表2 kutacc_af2_trans_act_inputs_t数据结构表定义

参数名	类型	描述	输入/输出
batch	int64_t	act的第一维度大小	输入
n_res	int64_t	act的第二维度大小	输入
input_act	kutacc_tensor_h	act经过layernorm后获得的输入矩阵	输入
intermediate_act	kutacc_tensor_h	第一次线性变化生成的中间变量，用于第二次线性变化作为输入	输入

表3 kutacc_af2_trans_weights_t数据结构表定义

参数名	类型	描述	输入/输出
c_o	int64_t	input_ln_w(见表4参数input_ln_w)第一维度大小	输入
c_i	int64_t	linear1_w第一维度大小	输入

 版权所有 © 华为技术有限公司

参数名	类型	描述	输入/输出
linear1_w	kutacc_tensor_h	第一次线性变化权重	输入
linear1_b	kutacc_tensor_h	第一次线性变化偏置	输入
linear2_w	kutacc_tensor_h	第二次线性变化权重	输入
linear2_b	kutacc_tensor_h	第二次线性变化偏置	输入

transition整数参数应满足的约束关系：

batch, n_res, c_o, c_i > 0，

batch, n_res, c_o, c_i, batch * n_res < INT64_MAX

在KPEX层构造参数时各参数的形状约束表（括号之后为全名）：

表4 KPEX transition入参张量形状及描述

参数名	形状	描述
input_ln_w(input_layer_norm_weight)	[c_o]	act layernorm归一化过程中的权重参数
input_ln_b(input_layer_norm_bias)	[c_o]	act layernorm归一化过程中的偏置参数
linear_1_w(transition1_weight)	[c_i, c_o]	见表3参数linear1_w
linear_1_b(transition1_bias)	[c_i]	见表3参数linear1_b
linear_2_w(transition2_weight)	[c_o, c_i]	见表3参数linear2_w
linear_2_b(transition2_bias)	[c_o]	见表3参数linear2_b
act	[batch, n_res, c_o]	输入张量

示例

C++ interface：

```
// test_transition.h
#ifndef KPEX_TPP_ALPHAFOLD_TEST_TRANSITION_H
#define KPEX_TPP_ALPHAFOLD_TEST_TRANSITION_H
#include <ATen/core/Tensor.h>
```

 版权所有 © 华为技术有限公司

```
#include <ATen/ops/empty.h>
#include <ATen/ops/ones.h>
#include <ATen/ops/zeros.h>
#include <ATen/ops/full.h>
#include <ATen/native/cpu/utils.h>
#include <c10/core/ScalarType.h>
namespace alphafold {
at::Tensor test_transition(int64_t batch, int64_t n_res, int64_t c_o, int64_t c_i);
}
#endif

// test_transition.cpp
#include "kutacc.h"
#include "test_transition.h"
#include "transition.h"
#include <utils/memory.h>
#include "utils/TensorWrapper.h"
#include "utils/layernorm.h"
namespace alphafold {
at::Tensor test_transition(int64_t batch, int64_t n_res, int64_t c_o, int64_t c_i)
{
    float a = 0.2f;
    at::Tensor act = at::full({batch, n_res, c_o}, a,
at::TensorOptions().device(kpex::device()).dtype(at::kBFloat16));
    at::Tensor out = act.new_empty(act.sizes());
    float b = 0.1f;
    float c = 0.5f;
    at::Tensor input_ln_w = at::ones({c_o}, at::TensorOptions().device(kpex::device()).dtype(at::kFloat));
    at::Tensor input_ln_b = at::zeros({c_o}, input_ln_w.options());
    at::Tensor linear1_w = at::full({c_i, c_o}, b, act.options());
    at::Tensor linear1_b = at::full({c_i}, c, input_ln_w.options());
    at::Tensor linear1_w_new = linear_weight_prepack(linear1_w);
    at::Tensor linear2_w = at::full({c_o, c_i}, b, act.options());
    at::Tensor linear2_w_new = linear_weight_prepack(linear2_w);
    at::Tensor linear_2_b = at::zeros({c_o}, input_ln_w.options());
    at::Tensor input_act = layernorm(act, input_ln_w, input_ln_b);
    at::Tensor intermediate_act = act.new_empty({batch * n_res, c_i});
    kutacc::TensorWrapper input_act_tw = convert_to_tensor_wrapper(input_act);
    kutacc::TensorWrapper linear1_w_tw = convert_to_tensor_wrapper(linear1_w_new);
    kutacc::TensorWrapper linear1_b_tw = convert_to_tensor_wrapper(linear1_b);
    kutacc::TensorWrapper linear2_w_tw = convert_to_tensor_wrapper(linear2_w_new);
    kutacc::TensorWrapper linear2_b_tw = convert_to_tensor_wrapper(linear2_b);
    kutacc::TensorWrapper intermediate_act_tw = convert_to_tensor_wrapper(intermediate_act);
    kutacc::TensorWrapper out_tw = convert_to_tensor_wrapper(out);

    kutacc_af2_trans_weights_t_wrapper *trans_weights_ptr = new
kutacc_af2_trans_weights_t_wrapper(linear1_w_tw, linear1_b_tw, linear2_w_tw, linear2_b_tw, c_o, c_i);
    kutacc_af2_trans_act_inputs_t_wrapper *trans_inputs_ptr = new
kutacc_af2_trans_act_inputs_t_wrapper(input_act_tw, intermediate_act_tw, batch, n_res);
    kutacc_af2_transition(trans_inputs_ptr, trans_weights_ptr, out_tw.get_tensor());
    delete trans_weights_ptr;
    delete trans_inputs_ptr;
    return out;
}
}

// bind.h
#include <torch/extension.h>
#include "test_transition.h"

namespace alphafold {
inline void bind(pybind11::module &m)
{
    auto submodule = m.def_submodule("alphafold");
    submodule.def("test_transition", &test_transition, py::arg("batch"), py::arg("n_res"), py::arg("c_o"),
py::arg("c_i"));
}
}
```

 版权所有 © 华为技术有限公司

```
// test.py
import torch
from torch import nn
import numpy as np
import torch.distributed as dist
import kpex._C as kernel
import kpex
import os

def test_transition(batch, n_res, c_o, c_i):
    out = kernel.alphafold.test_transition(batch, n_res, c_o, c_i)
    return out
```

8.2 KuDNN 开发指南

8.2.1 总述

大模型鲲鹏KuDNN算子库是用于AI框架的加速算子库，结合鲲鹏CPU微架构特性与软件优化手段，优化AI算子性能。通过插件的形式集成PyTorch等AI框架。

KuDNN代码已开源，请参见**https://gitcode.com/kunpengcompute/kudnn**

8.2.2 环境变量

以下环境变量会被KuDNN使用。

表 8-2 变量名说明

环境变量名	变量意义	有效值	默认值
KUDNN_THREAD_TYPE	线程管理后端方式	OMP\KUPL	OMP
KUDNN_NOT_USE_JIT	是否使用JIT动态代码	1	0

8.2.3 KuDNN 算子库函数说明

8.2.3.1 函数说明

KuDNN目前支持的算子如**表8-3**所示。

表 8-3 KuDNN 支持的算子

算子名称	算子说明
GEMM（单线程）&Linear（多线程）	矩阵乘法算子。
Linear+postops	在Linear算子计算后进行relu、silu、gelu、resext、resmul及residential等操作。

版权所有 © 华为技术有限公司

算子名称	算子说明
Softmax	Softmax归一化算子。
Conv2d/Conv3d	2d/3d卷积算子。
Group Normalization	组归一化算子。
Layer Normalization	层归一化算子。
Embedding	映射。
Root Mean Square Normalization	均方根归一化算子。
Rotary Position Embedding	旋转位置编码。

常用对象如表8-4所示说明。

表 8-4 KuDNN::TensorInfo 初始化需要传入参数列表

参数名称	数据类型	描述
shape	struct KuDNN::Shape	{d1, d2, d3, m, n}最高5d需要和layout对应。
type	struct KuDNN::Element::Type	枚举类型支持以下4种： • KuDNN::Element::TypeT::F16 • KuDNN::Element::TypeT::F32 • KuDNN::Element::TypeT::BF16 • KuDNN::Element::TypeT::S8
layout	enum class KuDNN::Layout	枚举类型，具体支持的排布参考各算子TensorInfo对象初始化时支持的layout类型表格。

8.2.3.2 函数定义

8.2.3.2.1 GEMM（单线程）&Linear（多线程）

8.2.3.2.1.1 功能描述

算子功能

实现对输入数据的线性变换。

算子公式

$C=alpha\times A\times B+beta\times C+bias$，其中B为权重，bias为偏置。

 版权所有 © 华为技术有限公司

8.2.3.2.1.2 使用说明

接口定义

构造Gemm对象，构造时需要传入输入矩阵、权重矩阵、输出矩阵的Tensor信息，其中偏置矩阵可以不输入，会调用第二个构造函数进行初始化。

- Gemm(const TensorInfo &srcTensor, const TensorInfo &weiTensor, const TensorInfo &dstTensor, int numThreads = 0)->void
- Gemm(const TensorInfo &srcTensor, const TensorInfo &weiTensor, const TensorInfo &dstTensor,const TensorInfo &biasTensor, int numThreads = 0)->void

表 8-5 Gemm 函数输入参数

参数名称	数据类型	描述	取值范围
srcTensor	KuDNN::TensorInfo	src张量信息。	{shape，type，layout}
weiTensor	KuDNN::TensorInfo	wei张量信息。	{shape，type，layout}
biasTensor（可选）	KuDNN::TensorInfo	bias张量信息。	{shape，type，layout}
dstTensor	KuDNN::TensorInfo	dst张量信息。	{shape，type，layout}
alpha（可选）	float	gemm参数。	默认1.0f
beta（可选）	float	gemm参数。	默认0.0f
numThreads（可选）	int	线程数。	默认送0时会使用GetMaxNumThreads返回的最大线程数

执行算子运算，要传入存放输入的内存地址和输出的内存地址，src、wei、dst、bias分别是输入矩阵、权重矩阵、输出矩阵及偏置矩阵的内存地址。其中偏置矩阵的地址要根据构造时是否传入bias选择是否传入。alpha和beta的值默认为1.0f和0.0f。

$$Mdst=alpha\times Msrc\times Mwei+beta\times Mdst+Mbias$$

- Run(const void *src, const void *wei, void *dst, float alpha = 1.0f, float beta = 0.0f, int numThreads = 0)->void
- Run(const void *src, const void *wei, void *dst, void *bias, float alpha = 1.0f, float beta = 0.0f, int numThreads = 0)->void

 版权所有 © 华为技术有限公司

表 8-6 Run 函数输入参数

参数名称	数据类型	描述	取值范围
src	void *	src指针。	大小为MxKxtype.GetSize()的指针
wei	void *	wei指针。	大小为KxNxtype.GetSize()的指针
dst	void *	dst指针。	大小为MxNxtype.GetSize()的指针
bias	void *	bias指针。	大小为MxNxtype.GetSize()的指针
numThreads（可选）	int	线程数。	默认送0时会使用GetMaxNumThreads返回的最大线程数

ValidateInput负责验证Gemm输入参数，并在算子构造过程中自动触发执行。

- ValidateInput(const TensorInfo &srcTensor, const TensorInfo &weiTensor, const TensorInfo &dstTensor, int numThreads = 0)->KuDNN::Status
- ValidateInput(const TensorInfo &srcTensor, const TensorInfo &weiTensor, const TensorInfo &dstTensor, const TensorInfo &biasTensor, int numThreads = 0)->KuDNN::Status

表 8-7 ValidateInput 输入参数列表

参数名称	数据类型	描述	取值范围
srcTensor	KuDNN::TensorInfo	src张量信息。	{shape，type，layout}
weiTensor	KuDNN::TensorInfo	wei张量信息。	{shape，type，layout}
biasTensor（可选）	KuDNN::TensorInfo	bias张量信息。	{shape，type，layout}
dstTensor	KuDNN::TensorInfo	dst张量信息。	{shape，type，layout}
alpha（可选）	float	gemm参数。	默认1.0f
beta（可选）	float	gemm参数。	默认0.0f

 版权所有 © 华为技术有限公司

参数名称	数据类型	描述	取值范围
numThreads（可选）	int	线程数。	默认送0时会使用GetMaxNumThreads返回的最大线程数

支持数据类型

- 矩阵计算数据支持以下数据类型组合。（TensorInfo对象初始化时需传入Shape、Type、Layout参数，此处列出为Type支持数据类型。）

  表 8-8 TensorInfo 对象初始化时支持的 Type 类型

srcTensor	weiTensor	dstTensor	biasTensor
KuDNN::Element::TypeT::F16(fp16)	KuDNN::Element::TypeT::F16(fp16)	KuDNN::Element::TypeT::F16(fp16)	KuDNN::Element::TypeT::F16(fp16)
KuDNN::Element::TypeT::F16(fp16)	KuDNN::Element::TypeT::F16(fp16)	KuDNN::Element::TypeT::F32(fp32)	KuDNN::Element::TypeT::F32(fp32)
KuDNN::Element::TypeT::S8(int8)	KuDNN::Element::TypeT::S8(int8)	KuDNN::Element::TypeT::S32(int32)	KuDNN::Element::TypeT::S32(int32)
KuDNN::Element::TypeT::F16(fp16)	KuDNN::Element::TypeT::S8(int8)	KuDNN::Element::TypeT::F16(fp16)	KuDNN::Element::TypeT::F32(fp32)
KuDNN::Element::TypeT::F32(fp32)	KuDNN::Element::TypeT::F32(fp32)	KuDNN::Element::TypeT::F32(fp32)	KuDNN::Element::TypeT::F32(fp32)

- 最高支持5Dtensor，支持顺序数据排布：a、ab、abc、abcd、abcde。

  实际对应KuDNN::Layout::A、KuDNN::Layout::AB、KuDNN::Layout::ABC、KuDNN::Layout::ABCD、KuDNN::Layout::ABCDE。

  表 8-9 TensorInfo 对象初始化时支持的 Layout 类型

dimension	srcTensor	weiTensor	dstTensor	biasTensor
2D	ab/ba	ab/ba	ab/ba	ab/ba
3D	abc/acb	abc/acb	abc/acb	abc/acb
4D	abcd/abdc	abcd/abdc	abcd/abdc	abcd/abdc
5D	abcde/abced	abcde/abced	abcde/abced	abcde/abced

使用示例

二维数据类型均为fp16的矩阵乘法运算示例。

```
// layout为AB M N K 分别为 5 5 5
using SizeType = KuDNN::SizeType;
```

 版权所有 © 华为技术有限公司

```
using Shape = KuDNN::Shape;
using Type KuDNN::Element::TypeT;
Shape srcShape(5, 5);
Shape weiShape(5, 5);
Shape dstShape(5, 5);
Shape biasShape(5, 5);
// Tensor初始化
const KuDNN::TensorInfo srcTensor = {srcShape, Type::F16, KuDNN::Layout::AB};
const KuDNN::TensorInfo weiTensor = {weiShape, Type::F16, KuDNN::Layout::AB};
const KuDNN::TensorInfo dstTensor = {dstShape, Type::F16, KuDNN::Layout::AB};
const KuDNN::TensorInfo biasTensor = {biasShape, Type::F16, KuDNN::Layout::AB};
SizeType srcSize = 5 * 5;
SizeType dstSize = 5 * 5;
SizeType weiSize = 5 * 5;
SizeType biasSize =5 * 5;
// 申请存放入参和结果的内存空间
void * src = malloc(srcSize * Type::F16.GetSize());
void * wei = malloc(weiSize * Type::F16.GetSize());
void * dst = malloc(dstSize * Type::F16.GetSize());
void * ref = malloc(dstSize * Type::F16.GetSize());
// 构造算子
KuDNN::Gemm gemmLayer(srcTensor,
weiTensor, dstTensor, numThreads);
// 执行算子
gemmLayer.Run(src, wei, dst, 1, 0, numThreads);
```

8.2.3.2.2 Linear+postops（relu，silu，gelu）

8.2.3.2.2.1 功能描述

算子功能

Linear+relu、Linear+silu、Linear+gelu：在Linear算子计算后，进行relu、silu以及gelu操作。

算子公式

计算类型	forward
relu	$d=\begin{cases} s \ if \ \ s>0 \\ \alpha s \ if \ s \le 0 \end{cases}$
swish	$d=\frac{s}{1+e^{-\alpha s}}$
gelu_erf	$d=0.5s\left(1+erf\left[\frac{s}{\sqrt{2}}\right]\right)$

版权所有 © 华为技术有限公司

计算类型	forward
gelu_tanh	$d=0.5s\left(1+tanh\left[\sqrt{\frac{2}{\pi}}(s+0.044715s^3)\right]\right)$

8.2.3.2.2.2 使用说明

接口定义

初始化Linear+relu、Linear+silu、Linear+gelu，构造时需要传入输入矩阵、权重矩阵、输出矩阵的tensor信息，其中偏置矩阵可以不传入，会调用第二个构造函数进行初始化。

- LinearActivationLayerFWD(const TensorInfo &aInfo, const TensorInfo &bInfo, const TensorInfo &cInfo, const TensorInfo &biasInfo, float alpha = 1.0f, float beta = 0.0f, ActivationFunction kind = KuDNN::ActivationFunction::RELU, int numthreads = 0)->void
- LinearActivationLayerFWD(const TensorInfo &aInfo, const TensorInfo &bInfo, const TensorInfo &cInfo, float alpha = 1.0f, float beta = 0.0f, ActivationFunction kind = KuDNN::ActivationFunction::RELU, int numthreads = 0)->void

表 8-10 LinearActivationLayerFWD 函数输入参数

参数名称	数据类型	描述	取值范围
aInfo	KuDNN::TensorInfo	src张量信息。	{shape，type，layout}
bInfo	KuDNN::TensorInfo	weight张量信息。	{shape，type，layout}
cInfo	KuDNN::TensorInfo	dst张量信息。	{shape，type，layout}
biasInfo	KuDNN::TensorInfo	bias张量信息。	{shape，type，layout}
alpha	float	激活函数参数。	默认1.0（kind=RELU时填0）
beta	float	激活函数参数。	默认0.0
kind	enum class	激活函数类型。	默认KuDNN::ActivationFunction::RELU 可选SWISH/GELU_TANH/GELU_ERF

版权所有 © 华为技术有限公司

参数名称	数据类型	描述	取值范围
numthreads	int	线程数。	默认0

执行算子运算，要传入存放输入的内存地址和输出的内存地址，a、b、c、bias分别是输入矩阵、权重矩阵、输出矩阵及偏置矩阵的内存地址。其中偏置矩阵的地址要根据构造时是否传入bias选择是否传入。

- void Run(const void *a, const void *b, void *c, const void *bias, int numThreads = 0)->void
- Run(const void *a, const void *b, void *c, int numThreads = 0)->void

表 8-11 Run 函数输入参数

参数名称	数据类型	描述	取值范围
a	void *	输入指针。	大小为MxKxtype.GetSize()的指针
b	void *	权重指针。	大小为KxNxtype.GetSize()的指针
c	void *	输出指针。	大小为MxNxtype.GetSize()的指针
bias	void *	偏置指针。	大小为MxNxtype.GetSize()的指针
numThreads（可选）	int	线程数。	默认设为0时会使用GetMaxNumThreads返回的最大线程数

ValidateInput验证Linear+relu、Linear+silu、Linear+gelu输入参数，并在算子构造过程中自动触发执行。

- ValidateInput(const TensorInfo &srcInfo, const TensorInfo &weiInfo, const TensorInfo &dstInfo, const TensorInfo &biasInfo, float alpha = 1.0f, float beta =0.0f, ActivationFunctionkind=KuDNN::ActivationFunction::RELU, int numthreads = 0)-> KuDNN::Status
- ValidateInput(const TensorInfo &srcInfo, const TensorInfo &weiInfo, const TensorInfo &dstInfo, float alpha = 1.0f, float beta =0.0f,ActivationFunctionkind=KuDNN::ActivationFunction::RELU, int numthreads = 0)-> KuDNN::Status

 版权所有 © 华为技术有限公司

表 8-12 ValidateInput 函数输入参数

参数名称	数据类型	描述	取值范围
srcInfo	KuDNN::TensorInfo	src张量信息。	{shape，type，layout}
weiInfo	KuDNN::TensorInfo	weight张量信息。	{shape，type，layout}
dstInfo	KuDNN::TensorInfo	dst张量信息。	{shape，type，layout}
biasInfo	KuDNN::TensorInfo	bias张量信息。	{shape，type，layout}
alpha	float	激活函数参数。	默认1.0（kind=RELU时为0）
beta	float	激活函数参数。	默认0.0
kind	enum class	激活函数类型。	默认KuDNN::ActivationFunction::RELU 可选SWISH/GELU_TANH/GELU_ERF
numthreads	int	线程数。	默认0

支持数据类型

- postops支持fp16/fp32/bf16数据类型（TensorInfo对象初始化时需传入Shape、Type、Layout参数，此处列出为Type支持数据类型）。

  表 8-13 TensorInfo 对象初始化时支持的 Type 类型

srcInfo	weiInfo	dstInfo	biasInfo
KuDNN::Element::TypeT::F16(fp16)	KuDNN::Element::TypeT::F16(fp16)	KuDNN::Element::TypeT::F16(fp16)	KuDNN::Element::TypeT::F16(fp16)
KuDNN::Element::TypeT::F32(fp32)	KuDNN::Element::TypeT::F32(fp32)	KuDNN::Element::TypeT::F32(fp32)	KuDNN::Element::TypeT::F32(fp32)
KuDNN::Element::TypeT::BF16(bf16)	KuDNN::Element::TypeT::BF16(bf16)	KuDNN::Element::TypeT::BF16(bf16)	KuDNN::Element::TypeT::BF16(bf16)

- 最高支持5Dtensor，支持顺序数据排布：a、ab、abc、abcd、abcde。

  对应KuDNN::Layout::A、KuDNN::Layout::AB、KuDNN::Layout::ABC、KuDNN::Layout::ABCD、KuDNN::Layout::ABCDE。

 版权所有 © 华为技术有限公司

表 8-14 TensorInfo 对象初始化时支持的 Layout 类型

dimension	srcInfo	weiInfo	dstInfo	biasInfo
2D	ab/ba	ab/ba	ab/ba	ab/ba
3D	abc/acb	abc/acb	abc/acb	abc/acb
4D	abcd/abdc	abcd/abdc	abcd/abdc	abcd/abdc
5D	abcde/abced	abcde/abced	abcde/abced	abcde/abced

使用示例

二维数据类型为FP32 ops为RES_IDENTIAL的linear+ops操作。

```
using SizeType = KuDNN::SizeType;
using Shape = KuDNN::Shape;
using Type KuDNN::Element::TypeT
// Tensor初始化
const KuDNN::TensorInfo srcTensor = {{110, 20}, KuDNN::Element::TypeT::F32, KuDNN::Layout::AB};
const KuDNN::TensorInfo weiTensor = {{20, 200}, KuDNN::Element::TypeT::F32, KuDNN::Layout::AB};
const KuDNN::TensorInfo dstTensor = {{110, 200}, KuDNN::Element::TypeT::F32, KuDNN::Layout::AB};
const KuDNN::TensorInfo biasTensor = {{110, 200}, KuDNN::Element::TypeT::F32, KuDNN::Layout::AB};
float alpha = 0.0f;
float beta = 0.0f;
KuDNN::ActivationFunction algKind = KuDNN::ActivationFunction::RELU;
int numThreads = 0;
SizeType srcSize = srcTensor.GetTotalTensorSize();
SizeType dstSize = weiBTensor.GetTotalTensorSize();
SizeType weiSize = dstCTensor.GetTotalTensorSize();
SizeType biasSize = biasTensor.GetTotalTensorSize();
// 申请存放入参和结果的内存空间
float * src = malloc(srcSize * sizeof(float));
float * wei = malloc(weiSize * sizeof(float));
float * dst = malloc(dstSize * sizeof(float));
float * bias = malloc(biasSize * sizeof(float));
// 构造算子KuDNN::LinearActivationLayerFWD
linearActivationLayerFwd(srcTensor, weiTensor, dstTensor,
biasTensor, alpha, beta, algKind, numThreads);
// 算子执行
linearActivationLayerFwd.Run(src, wei, dst, bias, numThreads);
```

8.2.3.2.3 Linear+postops（resext，resmul，residential）

8.2.3.2.3.1 功能描述

算子功能

Linear+resext、Linear+resmul、Linear+residential：在Linear算子计算后，进行resext、resmul、residential操作。

算子公式

- Linear+resext：linear输出加上gamma * res可表示为：

$$z=y+gamma\times res$$

 版权所有 © 华为技术有限公司

- Linear+resmul：linear输出乘以res可表示为：$z=y\times res$
- Linear+residential：linear输出加上res可表示为：$z=y+res$

8.2.3.2.3.2 使用说明

接口定义

初始化Linear+resext、Linear+resmul、Linear+residential。构造时需要传入linear输入矩阵、linear权重矩阵、linear输出矩阵、res输入矩阵、linearRes输出矩阵的tensor信息，其中偏置矩阵可以不送，会调用第二个构造函数进行初始化。

- LinearResFWD(const TensorInfo &srcInfo, const TensorInfo &weiInfo, const TensorInfo &dstInfo, const TensorInfo &biasInfo, const TensorInfo &resInfo, const TensorInfo &linearResInfo, float alpha = 1.0f, float beta = 0.0f, float gamma = 1.0f, ResOpsFunction kind = ResOpsFunction::RES_IDENTIAL, int numthreads)->void
- LinearResFWD(const TensorInfo &srcInfo, const TensorInfo &weiInfo, const TensorInfo &dstInfo, const TensorInfo &resInfo, const TensorInfo &linearResInfo, float alpha = 1.0f, float beta = 0.0f, float gamma = 1.0f, ResOpsFunction kind = ResOpsFunction::RES_IDENTIAL, int numthreads)->void

表 8-15 LinearResFWD 函数输入参数

参数名称	数据类型	描述	取值范围
srcInfo	KuDNN::TensorInfo	src张量信息。	{shape，type，layout}
weiInfo	KuDNN::TensorInfo	weight张量信息。	{shape，type，layout}
dstInfo	KuDNN::TensorInfo	dst张量信息。	{shape，type，layout}
biasInfo	KuDNN::TensorInfo	bias张量信息。	{shape，type，layout}
resInfo	KuDNN::TensorInfo	res张量信息。	{shape，type，layout}
linearResInfo	KuDNN::TensorInfo	linear+res输出张量信息。	{shape，type，layout}
alpha	float	linear参数。	默认1.0
beta	float	linear参数。	默认0.0
gamma	float	res操作参数。	默认1.0（kind为RES_EXT时gamma与res矩阵相乘，其他时候不生效）

 版权所有 © 华为技术有限公司

参数名称	数据类型	描述	取值范围
kind	enum class	激活函数类型。	默认KuDNN::ResOpsFunction::RES_IDENTIAL 可选RES_EXT/RES_MU
numthreads	int	线程数。	默认0

执行算子运算，要传入存放输入的内存地址和输出的内存地址，a、b、c、bias分别是输入矩阵、权重矩阵、输出矩阵及偏置矩阵的内存地址。其中偏置矩阵的地址要根据构造时是否传入bias选择是否传入。

- Run(const void *a, const void *b, void *c, const void *bias, const void *res, void *linearRes, float gamma = 1.0f, int numThreads = 0)->void
- Run(const void *a, const void *b, void *c, const void *res, void *linearRes, float gamma = 1.0f, int numThreads = 0)->void

表 8-16 Run 函数输入参数

参数名称	数据类型	描述	取值范围
a	void *	输入指针。	大小为MxKxtype.GetSize()的指针
b	void *	权重指针。	大小为KxNxtype.GetSize()的指针
c	void *	输出指针。	大小为MxNxtype.GetSize()的指针
bias	void *	偏置指针。	大小为MxNxtype.GetSize()的指针
res	void *	ops操作的输入指针。	• 如果ops操作是相乘，res为NxOxtype.GetSize()的指针 • 如果ops操作是相加，res为MxNxtype.GetSize()的指针

 版权所有 © 华为技术有限公司

参数名称	数据类型	描述	取值范围
linearRes	void *	linear+res操作结果指针。	• 如果ops操作是相乘，res为MxOxtype.GetSize()的指针 • 如果ops操作是相加，res为MxNxtype.GetSize()的指针
numThreads（可选）	int	线程数。	默认送0时会使用GetMaxNumThreads返回的最大线程数

ValidateInput负责验证Linear+resext、Linear+resmul、Linear+residential输入参数，也分为有bias版本和无bias版本。在算子构造过程中自动触发执行。

- ValidateInput(const TensorInfo &srcInfo, const TensorInfo &weiInfo, const TensorInfo &dstInfo, const TensorInfo &biasInfo, const TensorInfo &resInfo, const TensorInfo &linearResInfo, float alpha = 1.0f, float beta = 0.0f, float gamma = 1.0f, ResOpsFunction kind = ResOpsFunction::RES_IDENTIAL, int numthreads = 0)->KuDNN::Status
- ValidateInput(const TensorInfo &srcInfo, const TensorInfo &weiInfo, const TensorInfo &dstInfo, const TensorInfo &resInfo, const TensorInfo &linearResInfo, float alpha = 1.0f, float beta = 0.0f, float gamma = 1.0f, ResOpsFunction kind = ResOpsFunction::RES_IDENTIAL, int numthreads = 0)->KuDNN::Status

表 8-17 ValidateInput 函数输入参数

参数名称	数据类型	描述	取值范围
srcInfo	KuDNN::TensorInfo	src张量信息。	{shape，type，layout}
weiInfo	KuDNN::TensorInfo	weight张量信息。	{shape，type，layout}
dstInfo	KuDNN::TensorInfo	dst张量信息。	{shape，type，layout}
biasInfo	KuDNN::TensorInfo	bias张量信息。	{shape，type，layout}
resInfo	KuDNN::TensorInfo	res张量信息。	{shape，type，layout}
linearResInfo	KuDNN::TensorInfo	linear+res输出张量信息。	{shape，type，layout}
alpha	float	linear参数。	默认1.0

 版权所有 © 华为技术有限公司

参数名称	数据类型	描述	取值范围
beta	float	linear参数。	默认0.0
gamma	float	res操作参数。	默认1.0（kind为RES_EXT时gamma与res矩阵相乘，其他时候不生效）
kind	enum class	激活函数类型。	默认KuDNN::ResOpsFunction::RES_IDENTIAL 可选RES_EXT/RES_MU
numthreads	int	线程数。	默认0

支持数据类型

- postops支持fp16/fp32/bf16数据类型（TensorInfo对象初始化时需传入Shape、Type、Layout参数，此处列出为Type支持数据类型。）

  表 8-18 TensorInfo 对象初始化时支持的 Type 类型

srcInfo	weiInfo	dstInfo	biasInfo	resInfo	linearResInfo
KuDNN::Element::TypeT::F16(fp16)	KuDNN::Element::TypeT::F16(fp16)	KuDNN::Element::TypeT::F16(fp16)	KuDNN::Element::TypeT::F16(fp16)	KuDNN::Element::TypeT::F16(fp16)	KuDNN::Element::TypeT::F16(fp16)
KuDNN::Element::TypeT::F32(fp32)	KuDNN::Element::TypeT::F32(fp32)	KuDNN::Element::TypeT::F32(fp32)	KuDNN::Element::TypeT::F32(fp32)	KuDNN::Element::TypeT::F32(fp32)	KuDNN::Element::TypeT::F32(fp32)
KuDNN::Element::TypeT::BF16(bf16)	KuDNN::Element::TypeT::BF16(bf16)	KuDNN::Element::TypeT::BF16(bf16)	KuDNN::Element::TypeT::BF16(bf16)	KuDNN::Element::TypeT::BF16(bf16)	KuDNN::Element::TypeT::BF16(bf16)

- 最高支持5Dtensor，支持顺序数据排布：a、ab、abc、abcd、abcde。
  对应KuDNN::Layout::A、KuDNN::Layout::AB、KuDNN::Layout::ABC、KuDNN::Layout::ABCD、KuDNN::Layout::ABCDE。

 版权所有 © 华为技术有限公司

表 8-19 TensorInfo 对象初始化时支持的 Layout 类型

dimension	srcInfo	weiInfo	dstInfo	biasInfo
2D	ab/ba	ab/ba	ab/ba	ab/ba
3D	abc/acb	abc/acb	abc/acb	abc/acb
4D	abcd/abdc	abcd/abdc	abcd/abdc	abcd/abdc
5D	abcde/abced	abcde/abced	abcde/abced	abcde/abced

使用示例

二维数据类型为FP32 ops为RES_IDENTIAL的linear+ops操作。

```
// 示例
// using SizeType = KuDNN::SizeType;
using Shape = KuDNN::Shape;
using Type KuDNN::Element::TypeT
// Tensor初始化
const KuDNN::TensorInfo srcTensor= {{3, 2}, KuDNN::Element::TypeT::F32, KuDNN::Layout::AB};
const KuDNN::TensorInfo weiTensor= {{2, 2}, KuDNN::Element::TypeT::F32, KuDNN::Layout::AB};
const KuDNN::TensorInfo dstTensor= {{3, 2}, KuDNN::Element::TypeT::F32, KuDNN::Layout::AB};
const KuDNN::TensorInfo biasTensor = {{3, 2}, KuDNN::Element::TypeT::F32, KuDNN::Layout::AB};
const KuDNN::TensorInfo resTensor = {{3, 2}, KuDNN::Element::TypeT::F32, KuDNN::Layout::AB};
const KuDNN::TensorInfo linearResTensor = {{3, 2}, KuDNN::Element::TypeT::F32, KuDNN::Layout::AB};
float alpha = 1.0f;
float beta = 0.0f;
float gamma = 1.0f;KuDNN::ResOpsFunction algKind = KuDNN::ResOpsFunction::RES_IDENTIAL;
int numThreads = 0;
SizeType srcSize = srcATensor.GetTotalTensorSize();
SizeType dstSize = weiBTensor.GetTotalTensorSize();
SizeType weiSize = dstCTensor.GetTotalTensorSize();
SizeType biasSize = biasTensor.GetTotalTensorSize();
SizeType resSize = resTensor.GetTotalTensorSize();
SizeType linearResSize = linearResTensor.GetTotalTensorSize();
// 申请存放入参和结果的内存空间
float * src = malloc(srcSize* sizeof(float));
float * wei = malloc(weiSize* sizeof(float));
float * dst = malloc(dstSize* sizeof(float));
float * bias = malloc(biasSize* sizeof(float));
float * res = malloc(resSize* sizeof(float));
float * linearRes = malloc(linearResSize* sizeof(float));
// 构造算子
KuDNN::LinearResFWD
linearResFwd(srcTensor, weiTensor, dstTensor,
biasTensor, resTensor, linearResTensor, alpha, beta, gamma, algKind, numThreads);
// 算子执行
linearResFwd.Run(src, wei, dst, bias, res, linearRes, gamma, numThreads);
```

8.2.3.2.4 SoftMax

8.2.3.2.4.1 功能描述

算子功能

SoftMax是一种常用的激活函数，特别在多分类问题中经常被使用。它将一组任意实数转换为概率分布，其输出值的范围在0到1之间，并且所有输出值的总和为1。

主要特点包括：

 版权所有 © 华为技术有限公司

- 归一化输出：SoftMax函数将输入归一化，确保输出是一个有效的概率分布。即使输入是任意数值，SoftMax函数的输出总和仍为1，因此它常用于多分类问题的输出层。
- 非线性特征：SoftMax函数是一个非线性函数，可以对输入进行非线性变换，增加模型的表现能力，从而更好地拟合复杂的数据模式。
- 平移不变性：SoftMax函数常用于输出层，将神经网络的原始输出转换为表示类别概率的向量。在训练过程中，SoftMax输出与真实标签之间的差异可以用作损失函数，通过反向传播算法来更新网络参数，以最小化损失并提高模型性能。

算子公式

$$dst(\overline{ou},c,\overline{in}) = \frac{e^{src(\overline{ou},c,\overline{in}) - v(\overline{ou},\overline{in})}}{\sum_{ic} e^{src(\overline{ou},c,\overline{in}) - v(\overline{ou},\overline{in})}}$$

其中：

- c 是执行运算所沿的轴（axis）。
- $\overline{ou}$ 表示最外层的索引（位于该轴的左侧）。
- $\overline{in}$ 表示最内层索引（位于该轴右侧）。
- v 用于生成数值稳定的结果，其定义为：$v\left(\overline{ou},\overline{in}\right) = \max_{ic} src\left(\overline{ou},ic,\overline{in}\right)$。

8.2.3.2.4.2 使用说明

接口定义

初始化SoftmaxLayerFWD。构造时需要传入输入矩阵、权重矩阵、输出矩阵的tensor信息，其中偏置矩阵可以不送，会调用第二个构造函数进行初始化。

SoftmaxLayerFWD(const TensorInfo &srcInfo, const TensorInfo &dstInfo, SizeType axis, SoftmaxAlgorithmKind algorithm)->void

表 8-20 SoftmaxLayerFWD 函数输入参数

参数名称	数据类型	描述	取值范围
srcInfo	KuDNN::TensorInfo	src张量信息。	{shape，type，layout}
dstInfo	KuDNN::TensorInfo	dst张量信息。	{shape，type，layout}

 版权所有 © 华为技术有限公司

参数名称	数据类型	描述	取值范围
axis	KuDNN::SizeType	维度信息。	0<axis<srcInfo的维度，如srcInfo为三维，则0<axis<3
algorithm	KuDNN::SoftmaxAlgorithmKind	计算类型。	KuDNN::SoftmaxAlgorithmKind::SOFTMAX

执行算子运算，要传入存放输入的内存地址和输出的内存地址。

Run(const void *src, void *dst)->void

表 8-21 Run 函数输入参数

参数名称	数据类型	描述	取值范围
src	void *	src矩阵数据指针。	指向输入的内存地址
dst	void *	dst矩阵数据指针。	指向输出的内存地址

ValidateInput负责验证Softmax输入参数。在算子构造过程中自动触发执行。

ValidateInput(const TensorInfo &srcInfo, const TensorInfo &dstInfo, SizeType axis, SoftmaxAlgorithmKind algorithm)->KuDNN::Status

表 8-22 ValidateInput 函数输入参数

参数名称	数据类型	描述	取值范围
srcInfo	KuDNN::TensorInfo	src张量信息	{shape，type，layout}
dstInfo	KuDNN::TensorInfo	dst张量信息	{shape，type，layout}
axis	KuDNN::SizeType	维度信息	0<axis<srcInfo的维度 如srcInfo为三维， 0<axis<3
algKind	KuDNN::SoftmaxAlgorithmKind	计算类型	KuDNN::SoftmaxAlgorithmKind::SOFTMAX

支持数据类型

- SoftMax支持fp32数据类型（TensorInfo对象初始化时需传入Shape、Type、Layout参数，此处列出为Type支持数据类型。）

 版权所有 © 华为技术有限公司

表 8-23 TensorInfo 对象初始化时支持的 Type 类型

srcInfo	dstInfo
KuDNN::Element::TypeT::F32(fp32)	KuDNN::Element::TypeT::F32(fp32)

- 最高支持5Dtensor，支持顺序数据排布：a、ab、abc、abcd、abcde。

  对应KuDNN::Layout::A、KuDNN::Layout::AB、KuDNN::Layout::ABC、KuDNN::Layout::ABCD、KuDNN::Layout::ABCDE。

表 8-24 TensorInfo 对象初始化时支持的 Layout 类型

dimension	srcInfo	dstInfo
1D	a	a
2D	ab	ab
3D	abc	abc
4D	abcd	abcd
5D	abcde	abcde

使用示例

二维数据类型为F32的SoftMax操作。

```
// 示例
using SizeType = KuDNN::SizeType;
using Shape = KuDNN::Shape;
using Type KuDNN::Element::TypeT
// Tensor初始化
const KuDNN::TensorInfo srcTensor= {{3, 2}, KuDNN::Element::TypeT::F32, KuDNN::Layout::AB};
const KuDNN::TensorInfo dstTensor= {{3, 2}, KuDNN::Element::TypeT::F32, KuDNN::Layout::AB};
SizeType axis = 1;KuDNN::SoftmaxAlgorithmKind algKind = KuDNN::SoftmaxAlgorithmKind::SOFTMAX
SizeType srcSize = srcATensor.GetTotalTensorSize();
SizeType dstSize = weiBTensor.GetTotalTensorSize();
// 申请存放入参和结果的内存空间
float * src = malloc(srcSize* sizeof(float));
float * dst = malloc(dstSize* sizeof(float));
// 构造算子
KuDNN::SoftmaxLayerFWD softMaxLayerFwd(srcTensor, dstTensor, axis, algKind);
// 执行算子运算
softMaxLayerFwd.Run(src, dst);
```

8.2.3.2.5 Conv2d&Conv3d

8.2.3.2.5.1 功能描述

算子功能

在二维、三维空间实现卷积操作。

 版权所有 © 华为技术有限公司

算子公式

$$dst(n, oc, oh, ow) = bias(oc) + \sum_{ic=0}^{IC-1}\sum_{kh=0}^{KH-1}\sum_{kw=0}^{KW-1} src(n, ic, oh \cdot SH + kh - PH_L, ow \cdot SW + kw - PH_L) \cdot weights(oc, ic, kh, kw).$$

其中：

- $$OH = \left\lfloor \frac{IH - KH + PH_L + PH_R}{SH} \right\rfloor + 1,$$
- $$OW = \left\lfloor \frac{IW - KW + PW_L + PW_R}{SW} \right\rfloor + 1.$$

8.2.3.2.5.2 使用说明

接口定义

初始化ConvolutionLayerFWD。构造时需要传入输入矩阵、权重矩阵、输出矩阵的tensor信息，其中dilates可以不送，会默认使用{0, 0, 0}。

ConvolutionLayerFWD(const TensorInfo &src, const TensorInfo &weights, const TensorInfo &dst, const TensorInfo &bias,const Shape &strides, const Shape &dilates, const Shape &paddingL, const Shape &paddingR,ConvolutionAlgorithm alg)->void

表 8-25 ConvolutionLayerFWD 函数输入参数

参数名称	数据类型	conv2d取值	conv3d取值
src	KuDNN::TensorInfo	{shape{N, IC, IH, IW}, type, layout}	{shape{N, IC, ID, IH, IW}, type, layout}
weights	KuDNN::TensorInfo	{shape{OC, IC, KH, KW}, type, layout}	{shape{OC, IC, KD, KH, KW}, type, layout}
dst	KuDNN::TensorInfo	{shape{N, OC, OH, OW}, type, layout}	{shape{N, OC, OD, OH, OW}, type, layout}
bias	KuDNN::TensorInfo	{shape{OC}, type, layout}	{shape{OC},type, layout}
strides	KuDNN::Shape	(SH, SW)	(SD, SH, SW)
dilates	KuDNN::Shape	(SD, SH, SW)	(DD, DH, DW)
paddingL	KuDNN::Shape	(PL_H, PL_W)	(PL_D, PL_H, PL_W)

版权所有 © 华为技术有限公司

参数名称	数据类型	conv2d取值	conv3d取值
paddingR	KuDNN::Shape	(PR_H, PR_W)	(PR_D, PR_H, PR_W)
alg	KuDNN::ConvolutionAlgorithm	UNIMPLEMENTED/AUTO/DIRECT/WINOGRAD	同conv2d取值

表 8-26 conv2d&conv3d 取值含义

取值	描述
N	批大小batchSize。IC、IH和IW分别表示输入的通道数、高度和宽度。ID表示深度。
OC、OD、OH、OW	分别表示输出的通道数、深度、高度和宽度。
KD、KH、KW	分别表示卷积kernel的深度、高度和宽度。
SD、SH、SW	分别表示在卷积核在输入深度、高度和宽度上的移动步长。
DD、DH、DW	分别表示空洞卷积在输入深度、高度和宽度对应点的相邻距离。
PL_D、PL_H、PL_W	分别表示在深度前方、高度上方和宽度左边的填充指定个数。
PR_D、PR_H、PR_W	分别表示在深度后方、高度下方和宽度右边的填充指定个数。

执行算子运算，要传入存放输入的内存地址和输出的内存地址。

Run(const void *src, const void *wei, void *dst, const void *bia)->void

表 8-27 Run 函数输入参数

参数名称	数据类型	描述
src	void *	指向src的内存地址
wei	void *	指向wei的内存地址
dst	void *	指向dst的内存地址
bias	void *	指向bias的内存地址

ValidateInput负责验证ConvolutionLayerFWD输入参数。在算子构造过程中自动触发执行。

 版权所有 © 华为技术有限公司

ValidateInput(const TensorInfo &src, const TensorInfo &weights, const TensorInfo &dst, const TensorInfo &bias, const Shape &strides, const Shape &paddingL, const Shape &paddingR, ConvolutionAlgorithm alg)->KuDNN::Status

表 8-28 ValidateInput 函数输入参数

参数名称	数据类型	描述	取值范围
src	KuDNN::TensorInfo	src张量信息	{shape，type，layout}
weights	KuDNN::TensorInfo	wei张量信息	{shape，type，layout}
dst	KuDNN::TensorInfo	dst张量信息	{shape，type，layout}
bias	KuDNN::TensorInfo	bias张量信息	{shape，type，layout}
strides	KuDNN::Shape	移动步长	(SD, SH, SW)
paddingL	KuDNN::Shape	填充指定个数	(PL_D, PL_H, PL_W)
paddingR	KuDNN::Shape	填充指定个数	(PR_D, PR_H, PR_W)
algKind	KuDNN::SoftmaxAlgorithmKind	计算类型	KuDNN::SoftmaxAlgorithmKind::SOFTMAX

支持数据类型

- Conv2d&3d支持fp16/fp32数据类型（TensorInfo对象初始化时需传入Shape、Type、Layout参数，此处列出为Type支持数据类型）

  表 8-29 TensorInfo 对象初始化时支持的 Type 类型

src	weights	dst	bias
KuDNN::Element::TypeT::F16(fp16)	KuDNN::Element::TypeT::F16(fp16)	KuDNN::Element::TypeT::F16(fp16)	KuDNN::Element::TypeT::F16(fp16)
KuDNN::Element::TypeT::F32(fp32)	KuDNN::Element::TypeT::F32(fp32)	KuDNN::Element::TypeT::F32(fp32)	KuDNN::Element::TypeT::F32(fp32)

- 最高支持5Dtensor，支持顺序数据排布：a、ab、abc、abcd、abcde。

  对应KuDNN::Layout::A、KuDNN::Layout::AB、KuDNN::Layout::ABC、KuDNN::Layout::ABCD、KuDNN::Layout::ABCDE。

  表 8-30 TensorInfo 对象初始化时支持的 Layout 类型

dimension	src	wei	dst	bias
4D	abcd/abdc	abcd/abdc	abcd/abdc	abcd/abdc

 版权所有 © 华为技术有限公司

dimension	src	wei	dst	bias
5D	abcde/abced	abcde/abced	abcde/abced	abcde/abced

使用示例

四维fp32数据类型的卷积操作。

```
using SizeType = KuDNN::SizeType;
using Shape = KuDNN::Shape;
using Type KuDNN::Element::TypeT
SizeType N = 4, IC = 4, IH = 128, IW = 100;
SizeType OC = 5, OH = 0, OW = 0;
SizeType KH = 3, KW = 3;
Shape strides(1, 1), dilates(1, 1), paddingL(1, 1), paddingR(1, 1);
OH = (IH + paddingL[0] + paddingR[0] - 1 - (KH - 1)*(dilates[0] + 1)) / strides[0] + 1;
OW = (IW + paddingL[1] + paddingR[1] - 1 - (KW - 1)*(dilates[1] + 1)) / strides[1] + 1;
Shape srcShape(N, IC, IH, IW), weiShape(OC, IC, KH, KW), dstShape(N, OC, OH,
OW);KuDNN::ConvolutionAlgorithm alg(KuDNN::ConvolutionAlgorithm::AUTO);
// Tensor初始化
const KuDNN::TensorInfo srcTensor = {srcShape, Type::F32, KuDNN::Layout::ABCD};
const KuDNN::TensorInfo weightsTensor = {weiShape, Type::F32, KuDNN::Layout::ABCD};
const KuDNN::TensorInfo dstTensor = {dstShape, Type::F32, KuDNN::Layout::ABCD};
const KuDNN::TensorInfo biasTensor = {{OC}, Type::F32, KuDNN::Layout::A};
// 构造算子KuDNN::ConvolutionLayerFWD convFwdLayer(srcTensor, weightsTensor, dstTensor, biasTensor,
strides, dilates,
paddingL, paddingR, alg);
SizeType srcSize = N * IC * IH * IW;
SizeType dstSize = N * OC * OH * OW;
SizeType weiSize = OC * IC * KH * KW;
SizeType biaSize = OC;
// 申请内存
float *src = (float *)malloc(srcSize * sizeof(float));
float *dst = (float *)malloc(dstSize * sizeof(float));
float *dstRef = (float *)malloc(dstSize * sizeof(float));
float *wei = (float *)malloc(weiSize * sizeof(float));
float *bia = (float *)malloc(biaSize * sizeof(float));
// 执行算子计算
convFwdLayer.Run(src, wei, dst, bia);
```

8.2.3.2.6 GroupNorm

8.2.3.2.6.1 功能描述

算子功能

组归一化算子。

算子公式

$$dst(n, g \cdot C_g + c_g, h, w) = \gamma(g \cdot C_G + c_g) \cdot \frac{src(n, g \cdot C_G + c_g, h, w) - \mu(n, g)}{\sqrt{\sigma^2(n, g) + \varepsilon}} + \beta(g \cdot C_G + c_g)$$

其中：

- $C_G = \frac{C}{G}$

 版权所有 © 华为技术有限公司

- $c_g \in [0, C_G)$
- $\gamma(c), \beta(c)$ 是通道的可选缩放和偏移量。
- $\mu(n, g), \sigma^2(n, g)$ 分别表示一批中一组通道的均值和方差。
- ε是一个用于提高数值稳定性的常量。

8.2.3.2.6.2 使用说明

接口定义

初始化GroupNormalizationLayerFWD。构造时需要传入输入矩阵、缩放矩阵、组大小、输出矩阵的tensor信息以及归一化的flag。

GroupNormalizationLayerFWD(const TensorInfo &srcInfo, const TensorInfo &scaleShiftInfo, SizeType groupSize, const TensorInfo &dstInfo, NormalizationFlags flags)->void

表 8-31 GroupNormalizationLayerFWD 函数输入参数

参数名称	数据类型	描述	取值范围
srcInfo	KuDNN::TensorInfo	输入矩阵信息。	{shape{A, ... , D}, type, layout}
scaleShiftInfo	KuDNN::TensorInfo	缩放和偏移矩阵的信息。	{shape{D}, type, layout}
groupSize	KuDNN::SizeType	组大小。	无符号整型，在第二维度上分组的个数
dstInfo	KuDNN::TensorInfo	输出矩阵信息。	{shape{A, ... , D}, type, layout}
flags	KuDNN::NormalizationFlags	枚举类，用于选择归一化方式。	默认NONE，可选： • USE_GLOBAL_STATS使用传入的mean和var来计算。 • USE_SCALE对归一化结果再缩放。 • USE_SHIFT对归一化结果再偏移。

说明

假设输入数据形状为{A，B，C}则在B维度上进行分组归一化，缩放和偏移操作也在B维度上进行。

 版权所有 © 华为技术有限公司

执行算子运算。src和dst分别为输入和输出的指针，scale和shift为缩放和偏移，mean和variance为均值和方差的指针，saveStats表示是否保存计算的均值和方差。eps用于避免除0异常。

Run(const void *src, void *dst, const void *scale, const void *shift, float *mean, float *variance, bool saveStats, const float eps)->void

表 8-32 Run 函数输入参数

参数名称	数据类型	描述	取值范围
src	void*	输入指针。	-
dst	void*	输出指针。	-
scale	void*	缩放指针。	-
shift	void*	偏移指针。	-
mean	float*	均值指针。	-
variance	float*	方差指针。	-
saveStats	bool	是否保存计算均值和方差。	布尔值
eps	float	避免除0异常。	浮点数

验证GroupNormalizationLayerFWD的输入参数，并在算子构造过程中自动触发执行。

ValidateInput(const TensorInfo &srcInfo, const TensorInfo &scaleshiftInfo, SizeType groupInfo,const TensorInfo &dstInfo, NormalizationFlags flags)->KuDNN::Status

表 8-33 ValidateInput 输入参数

参数名称	数据类型	描述	取值范围
srcInfo	KuDNN::TensorInfo	src张量信息。	{shape{A, ... , D}, type, layout}
scaleshiftInfo	KuDNN::TensorInfo	scale张量信息。	{shape{D}, type, layout}
groupSize	KuDNN::SizeType	分组信息。	无符号整型，在第二维度上分组的个数
dstInfo	KuDNN::TensorInfo	dst张量信息。	{shape{A, ... , D}, type, layout}
flags	KuDNN::NormalizationFlags	枚举类，用于选择归一化方式。	无符号整型

 版权所有 © 华为技术有限公司

支持数据类型

TensorInfo对象初始化时需传入Shape、Type、Layout参数，此处列出为Type支持数据类型。

- GroupNormalizationLayerFWD支持fp16/fp32数据类型，以下是各个tensor初始化时可用的type类型。

  表 8-34 TensorInfo 对象初始化时支持的 Type 类型

srcInfo	scaleShiftInfo	dstInfo
KuDNN::Element::Type::F16(fp16)	KuDNN::Element::Type::F16(fp16)	KuDNN::Element::Type::F16(fp16)
KuDNN::Element::Type::F32(float)	KuDNN::Element::Type::F32(float)	KuDNN::Element::Type::F32(float)

- 最高支持3D到5D 的tensor，支持顺序数据排布：abc、abcd、abcde。
  对应KuDNN::Layout::ABC、KuDNN::Layout::ABCD、KuDNN::Layout::ABCDE。

  表 8-35 TensorInfo 对象初始化时支持的 Layout 类型

dimension	srcInfo数据排布	dstInfo数据排布
3D	abc	abc
4D	abcd	abcd
5D	abcde	abcde

使用示例

使用fp16数据类型组归一化，srcInfo数据排布为KuDNN::Layout::ABCD，scaleShiftInfo数据排布为KuDNN::Layout::A，dstInfo数据排布KuDNN::Layout::ABCD。

```
using SizeType = KuDNN::SizeType;
using Shape = KuDNN::Shape;
using Type KuDNN::Element::TypeT
Shape shape(4, 128, 18, 320);
SizeType groupInfo = 2;
//定义张量信息
TensorInfo srcInfo = {shape, Type::F16, KuDNN::Layout::ABCD};
TensorInfo scaleShiftInfo = {{shape[1]}, Type::F16, KuDNN::Layout::A};
TensorInfo dstInfo = {shape, Type::F16, KuDNN::Layout::ABCD};
KuDNN::NormalizationFlags flags = KuDNN::NormalizationFlags::NONE;
// 构造算子KuDNN::GroupNormalizationLayerFWD gnormLayer(srcInfo, scaleShiftInfo, groupInfo, dstInfo, flags);
// 初始化矩阵数据
SizeType srcSize = srcInfo.GetTotalTensorSize();
SizeType dstSize = dstInfo.GetTotalTensorSize();
SizeType statSize = srcInfo.GetDims()[0] * groupInfo;
SizeType scaleSize = scaleShiftInfo.GetTotalTensorSize();
__fp16 *src = (__fp16*)malloc(srcSize * sizeof(__fp16));
__fp16 *dst = (__fp16*)malloc(srcSize * sizeof(__fp16));
__fp16 *dstRef = (__fp16*)malloc(srcSize * sizeof(__fp16));
float *mean = (float *)malloc(statSize * sizeof(float));
```

 版权所有 © 华为技术有限公司

```
float *variance = (float *)malloc(statSize * sizeof(float));
__fp16 *scale = (__fp16*)malloc(scaleSize * sizeof(__fp16));
__fp16 *shift = (__fp16*)malloc(scaleSize * sizeof(__fp16));
float eps = 1e-5;
// 执行算子
gnormLayer.Run(src, dst, scale, shift, mean, variance, true, eps);
```

8.2.3.2.7 LayerNorm

8.2.3.2.7.1 功能描述

算子功能

层归一化算子。

算子公式

$$dst(t,n,c) = \gamma(c) \cdot \frac{src(t,n,c) - \mu(t,n)}{\sqrt{\sigma^2(t,n) + \varepsilon}} + \beta(c)$$

其中：

- $\gamma(c), \beta(c)$ 是通道的可选缩放和偏移量。
- $\mu(n,g), \sigma^2(n,g)$ 是均值和方差。
- ε是一个用于提高数值稳定性的常量。

8.2.3.2.7.2 使用说明

接口定义

初始化NormalizationLayerFWD。构造时需要传入输入矩阵、缩放矩阵、输出矩阵的tensor信息，其中statsinfo是均值和方差的描述信息。

NormalizationLayerFWD(const TensorInfo &srcInfo, const TensorInfo &statsInfo, const TensorInfo &scaleshiftInfo, const TensorInfo &dstInfo, NormalizationFlags flags)->void

表 8-36 NormalizationLayerFWD 函数输入参数

参数名称	数据类型	描述	取值范围
srcInfo	KuDNN::TensorInfo	输入矩阵信息。	{shape{A, ... , D}, type, layout}
statInfo	KuDNN::TensorInfo	均值和方差的信息。	{shape{A, ...}, type, layout}
scaleshiftInfo	KuDNN::TensorInfo	缩放矩阵信息。	{shape{D}, type, layout}

 版权所有 © 华为技术有限公司

参数名称	数据类型	描述	取值范围
dstInfo	KuDNN::TensorInfo	输出矩阵信息。	{shape{A, ... , D}, type, layout}
flags	KuDNN::NormalizationFlags	枚举类，用于选择归一化方式。	默认NONE，可选以下参数： • USE_GLOBAL_STATS使用传入的mean和var计算。 • USE_SCALE对归一化结果再放缩。 • USE_SHIFT对归一化结果再偏移。

执行算子运算。src和dst分别为输入和输出的指针，scale和shift为放缩和偏移的指针，mean和variance为均值和方差的指针，saveStats表示是否保存计算的均值和方差。eps用于避免除0异常。

Run(const void *src, void *dst, const void *scale, const void *shift, float *mean, float *variance, bool saveStats, const float eps)->void

表 8-37 Run 函数输入参数

参数名称	数据类型	描述	取值范围
src	void*	输入指针。	-
dst	void*	输出指针。	-
scale	void*	放缩指针。	-
shift	void*	偏移指针。	-
mean	float*	均值指针。	-
variance	float*	方差指针。	-
saveStats	bool	是否保存计算方差。	布尔值
eps	float	避免除0异常。	浮点数

说明

假设输入数据形状为{A，B，C，D},则在最后一个维度即D维上进行归一化，放缩和偏移操作也在D维度上进行。

验证NormalizationLayerFWD的输入参数，并在算子构造过程中自动触发执行。

ValidateInput(const TensorInfo &srcInfo, const TensorInfo &statsInfo,const TensorInfo &scaleshiftInfo, const TensorInfo &dstInfo, NormalizationFlags flags)->KuDNN::Status

 版权所有 © 华为技术有限公司

表 8-38 ValidateInput 函数输入参数

参数名称	数据类型	描述	取值范围
srcInfo	KuDNN::TensorInfo	输入矩阵信息。	{shape{A, ... , D}, type, layout}
statsInfo	KuDNN::TensorInfo	均值和方差的信息。	{shape{A, ...}, type, layout}
scaleshiftInfo	KuDNN::TensorInfo	缩放矩阵信息。	{shape{D}, type, layout}
dstInfo	KuDNN::TensorInfo	输出矩阵信息。	{shape{A, ... , D}, type, layout}
flags	KuDNN::NormalizationFlags	枚举类，用于选择归一化方式。	默认NONE，可选 • USE_GLOBAL_STATS • USE_SCALE • USE_SHIFT • FUSE_NORM_RELU

支持数据类型

- NormalizationLayerFWD支持fp16/fp32数据类型。（ TensorInfo对象初始化时需传入Shape、Type、Layout参数，此处列出为Type支持数据类型。）

  表 8-39 TensorInfo 对象初始化时支持的 Type 类型

srcInfo	statInfo	scaleShiftInfo	dstInfo
KuDNN::Element::Type::F16(fp16)	KuDNN::Element::Type::F16(fp16)	KuDNN::Element::Type::F16(fp16)	KuDNN::Element::Type::F16(fp16)
KuDNN::Element::Type::F32(float)	KuDNN::Element::Type::F32(float)	KuDNN::Element::Type::F32(float)	KuDNN::Element::Type::F32(float)

- 最高支持5Dtensor，支持顺序数据排布：a、ab、abc、abcd、abcde。

  对应KuDNN::Layout::A、KuDNN::Layout::AB、KuDNN::Layout::ABC、KuDNN::Layout::ABCD、KuDNN::Layout::ABCDE。

  表 8-40 TensorInfo 对象初始化时支持的 Layout 类型

dimension	srcInfo数据排布	dstInfo数据排布
2D	ab	ab
3D	abc	abc
4D	abcd	abcd

 版权所有 © 华为技术有限公司

dimension	srcInfo数据排布	dstInfo数据排布
5D	abcde	abcde

使用示例

使用float数据类型层归一化，srcInfo数据排布为KuDNN::Layout::ABCDE，statInfo数据排布为KuDNN::Layout::ABCD，scaleShiftInfo数据排布为KuDNN::Layout::A，dstInfo数据排布KuDNN::Layout::ABCDE。

```
using SizeType = KuDNN::SizeType;
using Shape = KuDNN::Shape;
using Type KuDNN::Element::TypeT
Shape shape(5, 5, 5, 5, 5);
// 定义张量信息
TensorInfo srcInfo = {shape, TypeF32, KuDNN::Layout::ABCDE};
TensorInfo statInfo = {{shape[0], shape[1], shape[2], shape[3]}, TypeF32, KuDNN::Layout::ABCD};
TensorInfo scaleShiftInfo = {{shape[4]}, TypeF32, KuDNN::Layout::A};
TensorInfo dstInfo = {shape, TypeF32, KuDNN::Layout::ABCDE};
KuDNN::NormalizationFlags flags = KuDNN::NormalizationFlags::NONE;
// 算子构造KuDNN::NormalizationLayerFWD lnormLayer(srcInfo, statInfo, scaleShiftInfo, dstInfo,
flags);
// 初始化矩阵数据
SizeType srcSize = srcInfo.GetTotalTensorSize();
SizeType dstSize = dstInfo.GetTotalTensorSize();
SizeType statSize = statInfo.GetTotalTensorSize();
SizeType innerSize = scaleShiftInfo.GetTotalTensorSize();
float *src = (float *)malloc(srcSize * sizeof(float));
float *dst = (float *)malloc(dstSize * sizeof(float));
float *dstRef = (float *)malloc(dstSize * sizeof(float));
float *mean = (float *)malloc(statSize * sizeof(float));
float *variance = (float *)malloc(statSize * sizeof(float));
float *scale = (float *)malloc(innerSize * sizeof(float));
float *shift = (float *)malloc(innerSize * sizeof(float));
float eps = 1e-5;
// 执行算子
lnormLayer.Run(src, dst, scale, shift, mean, variance, true, eps);
```

8.2.3.2.8 RMSNorm

8.2.3.2.8.1 功能描述

算子功能

Root Mean Square Normalization，均方根归一化。

算子公式

$$RMSNorm(x) = \frac{x}{\sqrt{\frac{1}{n}\sum_{i=1}^{n} x_i^2 + \varepsilon}}, \quad y_i = \frac{x_i}{RMS(x)} \times \gamma_i$$

其中：

- x是输入张量。

 版权所有 © 华为技术有限公司

- n是输入张量的最后一个维度的大小（即特征维度）。
- x^i是输入张量在最后一个维度上的第i个元素。
- ε是一个很小的正数用于避免除以零。

8.2.3.2.8.2 使用说明

接口定义

初始化RMSNormalizationLayerFWD。构造时需要传入输入矩阵、缩放矩阵、输出矩阵的tensor信息，其中statsinfo是均值和方差的描述信息。

RMSNormalizationLayerFWD(const TensorInfo &srcInfo, const TensorInfo &statsInfo, const TensorInfo &scaleInfo,const TensorInfo &dstInfo, NormalizationFlags flags)->void

表 8-41 RMSNormalizationLayerFWD 函数输入参数

参数名称	数据类型	描述	取值范围
srcInfo	KuDNN::TensorInfo	输入矩阵信息。	{shape{A, ... , D}, type, layout}
statsInfo	KuDNN::TensorInfo	均值和方差的信息。	{shape{A, ...}, type, layout}
scaleShiftInfo	KuDNN::TensorInfo	缩放矩阵信息。	{shape{D}, type, layout}
dstInfo	KuDNN::TensorInfo	输出矩阵信息。	{shape{A, ... , D}, type, layout}
flags	KuDNN::NormalizationFlags	枚举类，用于选择归一化方式。	默认NONE，可选以下参数： • USE_GLOBAL_STATS使用传入的mean和var来计算。 • USE_SCALE对归一化结果再放缩。 • USE_SHIFT对归一化结果再偏移。

说明

假设输入数据形状为{A，B，C，D} 则在最后一个维度即D维上进行归一化，放缩和偏移操作也在D维度上进行。

执行算子运算。src和dst分别为输入和输出的指针，scale和shift为放缩和偏移，mean和variance为均值和方差的指针，saveStats表示是否保存计算的均值和方差。eps用于避免除0异常。

Run(const void *src, void *dst, const void *scale, float *variance,bool saveStats, const float eps) ->void

 版权所有 © 华为技术有限公司

表 8-42 Run 函数输入参数

参数名称	数据类型	描述	取值范围
src	void*	输入指针。	-
dst	void*	输出指针。	-
scale	void*	放缩指针。	-
shift	void*	偏移指针。	-
mean	float*	均值指针。	-
variance	float*	方差指针。	-
saveStats	bool	是否保存计算方差。	布尔值
eps	float	避免除0异常。	浮点数

验证RMSNormalizationLayerFWD的输入参数，并在算子构造过程中自动触发执行。

ValidateInput(const TensorInfo &srcInfo, const TensorInfo &statsInfo,const TensorInfo &scaleInfo, const TensorInfo &dstInfo,NormalizationFlags flags) ->KuDNN::Status

表 8-43 ValidateInput 输入参数

参数名称	数据类型	描述	取值范围
srcInfo	KuDNN::TensorInfo	输入矩阵信息。	{shape{A, ... , D}, type, layout}
statsInfo	KuDNN::TensorInfo	均值和方差的信息。	{shape{A, ...}, type, layout}
scaleShiftInfo	KuDNN::TensorInfo	缩放矩阵信息。	{shape{D}, type, layout}
dstInfo	KuDNN::TensorInfo	输出矩阵信息。	{shape{A, ... , D}, type, layout}
flags	KuDNN::NormalizationFlags	枚举类，用于选择归一化方式。	默认NONE，可选： ● USE_GLOBAL_STATS使用传入的mean和var来计算。 ● USE_SCALE对归一化结果再放缩。 ● USE_SHIFT对归一化结果再偏移。

 版权所有 © 华为技术有限公司

支持数据类型

- RMSNorm支持fp16数据类型。（ TensorInfo对象初始化时需传入Shape、Type、Layout参数，此处列出为Type支持数据类型。）

  表 8-44 TensorInfo 对象初始化时支持的 Type 类型

srcInfo	statInfo	scaleShiftInfo
KuDNN::Element::Type::F16(fp16)	KuDNN::Element::Type::F16(fp16)	KuDNN::Element::Type::F16(fp16)

- 最高支持5Dtensor，支持顺序数据排布：a、ab、abc、abcd、abcde。

  对应KuDNN::Layout::A、KuDNN::Layout::AB、KuDNN::Layout::ABC、KuDNN::Layout::ABCD、KuDNN::Layout::ABCDE。

  表 8-45 TensorInfo 对象初始化时支持的 Layout 类型

dimension	srcInfo数据排布	dstInfo数据排布
2D	ab	ab
3D	abc	abc
4D	abcd	abcd
5D	abcde	abcde

使用示例

使用fp16数据类型均方根归一化，srcInfo数据排布为KuDNN::Layout::AB，statInfo数据排布为KuDNN::Layout::A，scaleShiftInfo数据排布为KuDNN::Layout::A，dstInfo数据排布KuDNN::Layout::AB。

```
using SizeType = KuDNN::SizeType;
using Shape = KuDNN::Shape;
using Type KuDNN::Element::TypeT
Shape shape(100, 100);
//定义张量信息
TensorInfo srcInfo = {shape, Type::F16, KuDNN::Layout::AB};
TensorInfo statInfo = {{shape[0]}, Type::F16, KuDNN::Layout::A};
TensorInfo scaleInfo = {{shape[1]}, Type::F16, KuDNN::Layout::A};
TensorInfo dstInfo = {shape, Type::F16, KuDNN::Layout::AB};
KuDNN::NormalizationFlags flags = KuDNN::NormalizationFlags::NONE;
// 构造算子KuDNN::RMSNormalizationLayerFWD rmsLayer1(srcInfo, statInfo, scaleInfo, dstInfo, flags);
//初始化矩阵数据
SizeType srcSize = srcInfo.GetTotalTensorSize();
SizeType dstSize = dstInfo.GetTotalTensorSize();
SizeType statSize = statInfo.GetTotalTensorSize();
SizeType innerSize = scaleInfo.GetTotalTensorSize();
__fp16 *src = (__fp16 *)malloc(srcSize * sizeof(__fp16));
__fp16 *dst = (__fp16 *)malloc(dstSize * sizeof(__fp16));
__fp16 *dstRef = (__fp16 *)malloc(dstSize * sizeof(__fp16));
float *variance = (float *)malloc(statSize * sizeof(float));
__fp16 *scale = (__fp16 *)malloc(innerSize * sizeof(__fp16));
float eps = 1e-5;
// 执行算子
rmsLayer1.Run(src, dst, scale, variance, true, eps);
```

 版权所有 © 华为技术有限公司

8.2.3.2.9 RoPE

8.2.3.2.9.1 功能描述

算子功能

Rotary Position Embedding，对key和query应用旋转位置编码。

算子公式

RoPE的核心思想是通过旋转操作将位置信息嵌入到向量中，而不是简单地相加。具体来说，对于每个位置i和维度d，RoPE的公式如下：

$$RoPE_{i,2d} = cos\left(\frac{i}{10000^{\frac{2d}{d_{model}}}}\right)$$

$$RoPE_{i,2d} = sin\left(\frac{i}{10000^{\frac{2d}{d_{model}}}}\right)$$

其中：

- i是序列中的位置索引。
- d是维度索引。
- d_{model}是模型的维度（通常是嵌入向量的维度）。
- cos和sin是余弦和正弦函数，用于生成选择角度。

8.2.3.2.9.2 使用说明

接口定义

初始化RopeLayerFWD。构造时需要传入最大支持的文本上下文长度、head的特征维度、计算角度theta的基数。

RopeLayerFWD(const int maxPosition, const int dim, const float base = 10000,const float scalingFactor = 1.0f)->void

表 8-46 RopeLayerFWD 输入参数

参数名称	数据类型	描述	取值范围
maxPosition	int	最大的上下文长度。	[1, INT_MAX]
dim	int	head的特征维度。	[1, INT_MAX]
base	float	计算theta的底数。	计算theta的底数

 版权所有 © 华为技术有限公司

参数名称	数据类型	描述	取值范围
scalingFactor	float	theta的放缩因子。	theta的放缩因子

执行算子运算。

Run(void *query, void *key, const int qStride, const int kStride,const int batchSize, const int seqLen, const int qHeads, const int kHeads,const int *positionIds, KuDNN::RopeKind kind)->void

表 8-47 Run 函数输入参数

参数名称	数据类型	描述	取值范围
query	void*	需要应用旋转位置编码的数据指针。	-
key	void*	需要应用旋转位置编码的数据指针。	-
qStride	int	query的内存步长。	[1, INT_MAX]
kStride	int	Key的内存步长。	[1, INT_MAX]
batchSize	int	批大小。	[1, INT_MAX]
seqLen	int	语句长度。	[1, INT_MAX]
qHeads	int	query头个数。	[1, INT_MAX]
kHeads	int	key头个数。	[1, INT_MAX]
positionlds	int*	token的位置编号列表。	-
kind	KuDNN::RopeKind	rope算法实现类型（可选参数）	KuDNN::RopeKind::OpenSora KuDNN::RopeKind::DeepSeek

验证RopeLayerFWD的输入参数，并在算子构造过程中自动触发执行。

ValidateInput(const int maxPosition, const int dim, const float base, const float scalingFactor)->KuDNN::Status

 版权所有 © 华为技术有限公司

表 8-48 ValidateInput 输入参数

参数名称	数据类型	描述	取值范围
max_position	int	最大的上下文长度。	[1, INT_MAX]
dim	int	head的特征维度。	[1, INT_MAX]
base	float	计算theta的底数。	计算theta的底数
scaling_factor	float	theta的放缩因子。	theta的放缩因子

支持的数据类型

RoPE支持fp16数据类型。（query和key指针指向数据类型。）

表 8-49 query 和 key 参数支持的数据类型

query	key
fp16	fp16

支持的内存排布顺序为（B, S, H, D）。

- B：表示batch_size批大小。
- S：表示seq_len语句长度。
- H：表示head_num头数量。
- D：表示head_dim头特征维度。

使用示例

在最大的上下文长度max_position为10000，head的特征维度dim为128时，对query和key进行旋转位置编码。

```
int max_position = 10000, dim = 128;
int batchSize = 2, seqLen = 128, qHeads = 28, kHeads = 28; //对应内存排布顺序为（B, S, H, D）
// 构造算子
KuDNN::RopeLayerFWD layer(max_position, dim);
int qStride = qHeads * dim, kStride = kHeads * dim;
int qSize = batchSize * seqLen * qHeads * dim;
int kSize = batchSize * seqLen * kHeads * dim;
// 入参出参内存申请
__fp16 *query = (__fp16 *)malloc(qSize * sizeof(__fp16));
__fp16 *key = (__fp16 *)malloc(kSize * sizeof(__fp16));
int *positionIds = (int *)malloc(seqLen * sizeof(int));
// 执行算子
layer.Run(query, key, qStride, kStride, batchSize, seqLen, qHeads, kHeads, positionIds);
```

8.2.3.2.10 Embedding

 版权所有 © 华为技术有限公司

8.2.3.2.10.1 功能描述

算子功能

将整数索引映射到一个固定大小的向量。

算子公式

$$Embedding(x) = W[x]$$

其中：

- x是输入的整数索引。
- W是一个二维张量，其形状为[num_embeddings, embedding_dim]，其中每一行表示一个嵌入向量。
- W[x]表示从权重矩阵W中取出索引为x的行作为嵌入向量。

8.2.3.2.10.2 使用说明

接口定义

初始化EmbeddingLayerFWD。

EmbeddingLayerFWD(const int num_embeddings, const int embedding_dim, __fp16 *_weight,const int padding_idx = -1, const float max_norm = 1.0, const float norm_type = 2.0) ->void

表 8-50 EmbeddingLayerFWD 输入参数

参数名称	数据类型	描述	取值范围
num_embeddings	int	嵌入层中嵌入向量的数量。	[1, INT_MAX]
embedding_dim	int	每个嵌入向量的维度。	[1, INT_MAX]
_weight	_fp16*	权重。	推理阶段要求非空
padding_idx	int	指定填充标记的参数。	[INT_MIN, num_embeddings)
max_norm	float	指定权重向量的最大范数。	推理阶段不涉及
norm_type	float	指定权重向量的范数类型。	推理阶段不涉及

执行算子运算。

Run(int *indice, int dims) ->void

 版权所有 © 华为技术有限公司

表 8-51 Run 函数输入参数

参数名称	数据类型	描述	取值范围
indice	int*	需要取出的向量索引组成的数组。	-
dims	int	目标向量维度数。	[1, INT_MAX]

验证EmbeddingLayerFWD的输入参数，并在算子构造过程中自动触发执行。

ValidateInput(const int num_embeddings, const int embedding_dim, __fp16 *_weight, const int padding_idx, const float max_norm, const float norm_type) ->KuDNN::Status

表 8-52 ValidateInput 函数输入参数

参数名称	数据类型	描述	取值范围
num_embeddings	int	嵌入层中嵌入向量的数量。	[1, INT_MAX]
embedding_dim	int	每个嵌入向量的维度。	[1, INT_MAX]
_weight	_fp16*	权重。	推理阶段要求非空
padding_idx	int	指定填充标记的参数。	[INT_MIN, num_embeddings)
max_norm	float	指定权重向量的最大范数。	推理阶段不涉及
norm_type	float	指定权重向量的范数类型。	推理阶段不涉及

支持数据类型

Embedding支持fp16数据类型。（ g_weight数组数据类型 ）_weight支持数据类型为fp16。

使用示例

将g_weight数组中对应索引数组indice中每个索引对应的向量取出，嵌入到固定大小的结果向量中。

```
__fp16 g_weight[10][3] = {
{0.1, 0.12, 0.15}, {0.21, 0.22, 0.25}, {0.11, 0.21, 0.32}, {0.77, 0.02, 0.05}, {0.54, 0.32, 0.08},
{0.14, 0.42, 0.25}, {0.31, 0.2, 0.07}, {0.22, 0.14, 0.03}, {0.17, 0.28, 0.32}, {0.44, 0.26, 0.23}
}; //使用fp16类型初始化向量数组
int num_embeddings = 10;
int embedding_dim = 3;
int num = 4;
// 构造算子KuDNN::EmbeddingLayerFWD embedding(num_embeddings, embedding_dim,
(__fp16*)g_weight);
int indice[4] = {1, 2, 3, 7}; //索引数组
```

 版权所有 © 华为技术有限公司

```
// 执行算子
__fp16 *out = embedding.Run(indice, num);
```

8.2.4 KuDNN 适配 PyTorch 接口说明

8.2.4.1 算子接口

8.2.4.1.1 接口说明

KuDNN支持的适配后接口如表8-53所示。

表 8-53 接口说明

算子	功能	KuDNN支持数据类型
Linear算子	线性变换。	input weight bias output（fp16 fp16 fp16 fp16） input weight bias output（fp16 fp16 fp32 fp32） input weight bias output（int8 int8 int32 int32） input weight bias output（fp16 int8 int32 fp32） input weight bias output（fp32 fp32 fp32 fp32）
SoftMax算子	概率分布归一化。	fp32
Conv算子	局部特征提取。	fp16、fp32
GroupNorm算子	通道分组归一化。	fp16、fp32
LayerNorm算子	层分组归一化。	fp16、fp32
RMSNorm算子	均方根归一化。	fp16
Embedding算子	将整数索引到固定向量。	fp16

表 8-54 数据类型

数据类型	对应torch数据类型
fp16	torch.float16
fp32	torch.float32

版权所有 © 华为技术有限公司

数据类型	对应torch数据类型
int8	torch.int8
int32	torch.int32

8.2.4.1.2 Linear

接口功能

PyTorch的Linear用于创建全连接层，进行线性变换 $y = xW^T + b$ 。

函数原型

```
torch.nn.Linear(
    in_features: int,
    out_features: int,
    bias: bool = True,
    device=None,
    dtype=None
)
```

参数说明

表 8-55 参数说明

参数	类型	必填	说明
in_features	int	是	输入特征维度。
out_features	int	是	输出特征维度。
bias	bool	否	是否包含偏置（默认True）。
device	torch.device	否	计算设备（如“cpu”，“cuda”）。
dtype	torch.dtype	否	权重数据类型（如torch.float16、torch.float32）。

8.2.4.1.3 SoftMax

接口功能

沿指定维度计算归一化指数概率分布。

 版权所有 © 华为技术有限公司

函数原型

```
torch.nn.Softmax(
    dim: Optional[int] = None
)
# 或函数式
torch.nn.functional.softmax(
    input: Tensor,
    dim: int,
    dtype: Optional[torch.dtype] = None
) -> Tensor
```

参数说明

表 8-56 参数说明

参数	类型	必填	说明
dim	int	是	计算维度（如dim=1表示对每行计算）。
input	Tensor	是	输入张量（函数式专用）。
dtype	torch.dtype	否	输出数据类型（函数式专用）。

8.2.4.1.4 Conv

接口功能

Conv2d卷积操作：提取图像/特征图的局部空间特征。

Conv3d对三维体积数据执行卷积操作，提取时空特征。

函数原型

```
torch.nn.Conv2d(
    in_channels: int,
    out_channels: int,
    kernel_size: Union[int, Tuple[int, int]],
    stride: Union[int, Tuple[int, int]] = 1,
    padding: Union[str, int, Tuple[int, int]] = 0,
    dilation: Union[int, Tuple[int, int]] = 1,
    groups: int = 1,
    bias: bool = True,
    padding_mode: str = 'zeros',
    device=None,
    dtype=None
)
torch.nn.Conv3d(
    in_channels: int,
    out_channels: int,
    kernel_size: Union[int, Tuple[int, int, int]],
    stride: Union[int, Tuple[int, int, int]] = 1,
    padding: Union[str, int, Tuple[int, int, int]] = 0,
    dilation: Union[int, Tuple[int, int, int]] = 1,
    groups: int = 1,
    bias: bool = True,
```

 版权所有 © 华为技术有限公司

```
    padding_mode: str = 'zeros',
    device=None,
    dtype=None
)
```

参数说明

表 8-57 Conv2d 参数说明

参数	类型	必填	说明
in_channels	int	是	输入通道数。
out_channels	int	是	输出通道数（卷积核数量）。
kernel_size	int/tuple	是	卷积核尺寸（如3或 (3,3)）。
stride	int/tuple	否	滑动步长（默认1）。
padding	int/tuple/str	否	边界填充（默认0，支持“same”等）。
dilation	int/tuple	否	空洞卷积率（默认1）。
groups	int	否	分组卷积数（默认1，in_channels需被整除）。
bias	bool	否	是否添加偏置（默认True）。
padding_mode	str	否	填充模式（"zeros", "reflect", "replicate", "circular"）。

表 8-58 Conv3d

参数	类型	必填	说明
in_channels	int	是	输入体积数据的通道数（如RGB视频为3）。
out_channels	int	是	输出通道数（卷积核数量）。

 版权所有 © 华为技术有限公司

参数	类型	必填	说明
kernel_size	int/tuple	是	三维卷积核尺寸（如(3,3,3)或单值5）。
stride	int/tuple	否	三维滑动步长（默认(1,1,1)）。
padding	int/tuple/str	否	三维边界填充（默认0，支持"same"）。
dilation	int/tuple	否	三维空洞卷积率（默认1）。
groups	int	否	分组卷积数（默认1）。
bias	bool	否	是否添加偏置（默认True）。
padding_mode	str	否	填充模式（"zeros", "reflect", "replicate", "circular"）。

8.2.4.1.5 GroupNorm

接口功能

沿通道维度分组归一化（独立于batch大小）。

函数原型

```
torch.nn.GroupNorm(
    num_groups: int,
    num_channels: int,
    eps: float = 1e-5,
    affine: bool = True,
    device=None,
    dtype=None
)
```

参数说明

表 8-59 参数说明

参数	类型	必填	说明
num_groups	int	是	分组数量（需被num_channels整除）。

版权所有 © 华为技术有限公司

参数	类型	必填	说明
num_channels	int	是	输入通道总数。
eps	float	否	数值稳定性常数（默认1e-5）。
affine	bool	否	是否学习缩放因子γ和偏移β（默认True）。

8.2.4.1.6 LayerNorm

接口功能

对单个样本的所有特征进行归一化。

函数原型

```
torch.nn.LayerNorm(
    normalized_shape: Union[int, List[int], torch.Size],
    eps: float = 1e-5,
    elementwise_affine: bool = True,
    device=None,
    dtype=None
)
```

参数说明

表 8-60 参数说明

参数	类型	必填	说明
normalized_shape	int/list/tuple	是	需归一化的维度（如 [C, H, W]或hidden_size）。
eps	float	否	数值稳定性常数（默认1e-5）。
elementwise_affine	bool	否	是否逐元素学习γ和β（默认True）。

8.2.4.1.7 RMSNorm

接口功能

LayerNorm的轻量变体，仅使用均方根值归一化（无均值中心化）。

 版权所有 © 华为技术有限公司

函数原型

```
class RMSNorm(torch.nn.Module):
    def __init__(self, dim: int, eps: float = 1e-6):
        super().__init__()
        self.eps = eps
        self.weight = torch.nn.Parameter(torch.ones(dim))

    def forward(self, x: torch.Tensor):
        # 计算均方根值
        rms = x.norm(2, dim=-1, keepdim=True) * (x.size(-1) ** -0.5)
        return x / (rms + self.eps) * self.weight
```

参数说明

表 8-61 参数说明

参数	类型	必填	说明
dim	int	是	输入特征维度。
eps	float	否	数值稳定性常数（默认1e-6）。
weight	tensor	否	可学习的缩放因子（自动创建）。

8.2.4.1.8 Embedding

接口功能

将离散索引映射为连续向量（查表操作）。

函数原型

```
torch.nn.Embedding(
    num_embeddings: int,
    embedding_dim: int,
    padding_idx: Optional[int] = None,
    max_norm: Optional[float] = None,
    norm_type: float = 2.0,
    scale_grad_by_freq: bool = False,
    sparse: bool = False,
    _weight: Optional[Tensor] = None,
    device=None,
    dtype=None
)
```

 版权所有 © 华为技术有限公司

参数说明

表 8-62 参数说明

参数	类型	必填	说明
num_embeddings	int	是	嵌入字典大小（索引范围[0, num_embeddings-1]）。
embedding_dim	int	是	嵌入向量维度。
padding_idx	int	否	填充索引（该索引对应的向量梯度固定为0）。
max_norm	float	否	最大范数约束（超过时缩放）。
norm_type	float	否	范数计算类型（默认2.0表示L2范数）。
scale_grad_by_freq	bool	否	是否按频率缩放梯度（默认False）。
sparse	bool	否	是否使用稀疏梯度（默认False）。

8.2.4.2 使用示例

使用前配置环境变量KML_BLAS_NOT_USE_HBM=1关闭片上内存。

```
export KML_BLAS_NOT_USE_HBM=1
```

⚠ 注意

目前使用KuDNN库需要关闭片上内存，原因是KuDNN使用kblas库在启用片上内存时依赖开源库libnuma.so。KuDNN启用asan扫描显示该开源库存在内存泄漏问题。(**issue地址**)

```
==305003==ERROR: LeakSanitizer: detected memory leaks

Direct leak of 1024 byte(s) in 1 object(s) allocated from:
    #0 0xffff8206711c in calloc (/hom[redacted]/install/HPCKit/HPCKit/25.1.
_rt.asan.so+0x10711c)
    #1 0xffff76716a14 in numa_node_to_cpus (/usr/lib64/libnuma.so.1+0x6a14)
    #2 0xffff76716ba0 in numa_node_of_cpu (/usr/lib64/libnuma.so.1+0x6ba0) (
```

 版权所有 © 华为技术有限公司

8.2.4.2.1 Linear

场景说明

对输入tensor进行线性变换，目前KuDNN支持torch.int8、torch.float16和torch.float32数据类型，其他数据类型会走开源分支。

示例代码

```
import torch
import torch.nn as nn
#使能KuDNN
torch._C._set_kdnn_enabled(True)

# linear示例，
# 输入数据: (batch, in_features)，默认数据类型torch.float32
input_tensor = torch.randn(128, 20)

# 构造linear层
linear = nn.Linear(20, 30, bias=True) # input_feature为20， output_feature为30

#前向计算
output_tensor = linear(input_tensor) # output_tensor shape 为 [128,30]

# 打印输出的形状和数值
print("linear输出形状", output_tensor.shape)
print(ouput_tensor)
```

8.2.4.2.2 SoftMax

场景说明

多分类任务中沿指定维度计算归一化指数概率分布，目前KuDNN支持torch.float32数据类型，其他数据类型会使用开源分支。

示例代码

```
import torch
import torch.nn as nn

#使能KuDNN
torch._C._set_kdnn_enabled(True)

# 构造Softmax层对列进行归一化
m = nn.Softmax(dim=1)
logits = torch.randn(4, 5)  # 4个样本，5分类
probs = m(logits)

# 每行概率和为1
print(probs)
```

8.2.4.2.3 Conv

场景说明

Conv2d：常用于边缘检测、特征提取。

Conv3d：常用于时空特征提取。

目前KuDNN支持torch.float16和torch.float32数据类型，其他数据类型会使用开源分支。

 版权所有 © 华为技术有限公司

示例代码

```
import torch
import torch.nn as nn

#使能KuDNN
torch._C._set_kdnn_enabled(True)

# Conv2d示例
conv2d = nn.Conv2d(in_channels=3, out_channels=16, kernel_size=3, stride=1)
x = torch.randn(8, 3, 32, 32)  # [batch, channel, H, W] # 默认为fp32类型
y = conv2d(x)  # 输出 [8, 16, 30, 30]
print("Conv2d输出形状", y.shape)
print(y)

# Conv3d示例
conv3d = nn.Conv3d(1, 8, kernel_size=(3,3,3))
x = torch.randn(4, 1, 10, 64, 64)  # [batch, channel, depth, H, W] # 默认为fp32类型
y = conv3d(x)  # 输出 [4, 8, 8, 62, 62]

print("Conv3d输出形状", y.shape)
print(y)
```

8.2.4.2.4 GroupNorm

场景说明

使用均方差对tensor做归一化，目前KuDNN支持torch.float16和torch.float32数据类型，其他数据类型会走开源分支。

示例代码

```
import torch
import torch.nn as nn

#使能KuDNN
torch._C._set_kdnn_enabled(True)

# 输入数据: (batch_size, channels, height, width)
input = torch.randn(2, 6, 32, 32)  # 2张图片，6个通道，32x32分辨率 默认为fp32数据类型

# 定义 GroupNorm (分3组，每组2个通道)
group_norm = nn.GroupNorm(num_groups=3, num_channels=6)  # 6个通道分成3组 默认为fp32数据类型

# 前向计算
output = group_norm(input)

# 验证: 每组内归一化
print("GroupNorm 输出形状:", output.shape)  # 保持原形状
```

8.2.4.2.5 LayerNorm

场景说明

使用均方差对tensor做归一化，目前KuDNN支持torch.float16和torch.float32数据类型，其他数据类型会走开源分支。

示例代码

```
import torch
import torch.nn as nn

#使能KuDNN
torch._C._set_kdnn_enabled(True)
```

 版权所有 © 华为技术有限公司

```
# 输入数据: (batch_size, seq_len, hidden_dim)
input = torch.randn(2, 5, 10)  # 2个样本，5个时间步，特征维度10 默认为fp32数据类型

# 定义 LayerNorm (对最后一维 hidden_dim=10 归一化)
layer_norm = nn.LayerNorm(10)  # 或者 normalized_shape=[5,10] 对最后两维归一化 默认为fp32数据类型

# 前向计算
output = layer_norm(input)

# 验证: 归一化后最后一维的均值接近0，方差接近1
print("LayerNorm 输出均值:", output.mean(dim=-1))  # 应接近0
print("LayerNorm 输出方差:", output.var(dim=-1, unbiased=False))  # 应接近1
```

8.2.4.2.6 RMSNorm

场景说明

使用均方差对tensor做归一化，目前KuDNN支持torch.float16类型，其他数据类型会走开源分支。

示例代码

```
import torch
import torch.nn as nn

# 启用kdnn
torch._C._set_kdnn_enabled(True)

# 输入数据: (batch_size, seq_len, hidden_dim)
input = torch.randn(2, 5, 10)

# 构造RMSNorm层
# 对最后一维归一化指定eps为1e-5并指定数据类型为fp16
rms_norm = nn.RMSNorm(10, eps = 1e-5, dtype = torch.float16)
output = rms_norm(input)

# 验证: 输出的均方根接近缩放值
print("RMSNorm 输出形状:", output.shape)  # 同输入
print("RMSNorm 最后一维RMS:", torch.sqrt(torch.mean(output.pow(2), dim=-1)))  # 接近scale值
```

8.2.4.2.7 Embedding

场景说明

将离散ID映射为稠密向量，目前KuDNN支持torch.float16数据类型，其他数据类型会走开源分支。

示例代码

```
import torch
import torch.nn as nn

# 启用kdnn
torch._C._set_kdnn_enabled(True)

# 构造Embedding层
embed = nn.Embedding(num_embeddings=1000, embedding_dim=128, dtype=torch.float16)

# 输入数据2x2的token索引
input_ids = torch.LongTensor([[1, 2], [3, 4]])
embeddings = embed(input_ids)  # 输出 [2, 2, 128]

print(embeddings)
```

 版权所有 © 华为技术有限公司

8.3 KuQCD 开发指南

8.3.1 总述

KuQCD是基于鲲鹏平台的格点量子色动力学领域加速库，采用C/C++实现，针对求解器等功能进行优化，并提供相应接口，以便在应用中使用。目前KuQCD仅支持在鲲鹏920新型号上使用，依赖KUPL，支持毕昇编译器和gcc编译器。

8.3.2 KuQCD 函数说明

8.3.2.1 数据结构

8.3.2.1.1 Lattice

lattice为格点，结构体里参数如下表所示。

表 8-63 Lattice 结构体

变量名	类型	描述
nx	unsigned short	格点x方向大小
ny	unsigned short	格点y方向大小
nz	unsigned short	格点z方向大小
nt	unsigned short	格点t方向大小
V	size_t	格点整体大小，值为nx*ny*nz*nt

8.3.2.1.2 LattBase

lattbase结构体参数如下表所示。

表 8-64 LattBase 结构体

变量名	类型	描述
latSupBulkFL	Lattice	全进程，无halo，全格点
latSupBulkCB	Lattice	全进程，无halo，奇或偶格点
latSubFullFL	Lattice	子进程，有halo，全格点
latSubFullCB	Lattice	子进程，有halo，奇或偶格点

版权所有 © 华为技术有限公司

变量名	类型	描述
latSubBulkFL	Lattice	子进程，无halo，全格点
latSubBulkCB	Lattice	子进程，无halo，奇或偶格点
latFull	Lattice *	根据site_subset，为子进程有halo格点
latBulk	Lattice *	根据site_subset，为子进程无halo格点
lenFull	size_t	根据site_subset，为子进程有halo总格点大小
lenBulk	size_t	根据site_subset，为子进程无halo总格点大小
gridDim	array<unsigned short, 4>	各维度进程切分策略
haloDepth	unsigned short	halo的深度，向量时设置为0，矩阵时为3
Nbatch	unsigned int	x方向批量大小
myRank	int	当前进程所属进程号
np	int	进程数
prec	KuQCDPrecision	精度选择，有如下值： ● KUQCD_HALF ● KUQCD_SINGLE ● KUQCD_DOUBLE
gauge_order	KuQCDGaugeOrder	是否做x方向批量，有如下值： ● KUQCD_NORMAL_ORDER ● KUQCD_BATCH_ORDER
rankOrder	KuQCDRankOrder	格点排布顺序，有如下值： ● XYZT_ORDER ● TZYX_ORDER
site_subset	KuQCDSiteSubset	子格点，有如下值： ● KUQCD_EVEN ● KUQCD_ODD ● KUQCD_ALL
myIdx	Index	格点所在的index

 版权所有 © 华为技术有限公司

8.3.2.1.3 CommBase

CommBase结构体如表CommBase结构体所示，仅规范场用到

表 8-65 CommBase 结构体

变量名	类型	描述
latBase	LattBase &	所用到的格点
rankHyperPlane	array<array<int, 2>, 4>	一维通信进程号
lenHyperPlane	array<size_t, 4>	一维通信包长度
rankPlane	array<array<int, 4>, 6>	二维通信进程号
lenPlane	array<size_t, 6>	二维通信包长度
rankStripe	array<array<int, 8>, 4>	三维通信进程号
lenStripe	array<size_t, 4>	三维通信包长度
rankCorner	array<int, 16>	四维通信进程号
lenCorner	size_t	四维通信包长度
nbIdx	Index	邻居格点序号
haloIdxSend1	array<array<Index, 2>, 4>	一维通信需要发送的位置序号
haloIdxRecv1	array<array<Index, 2>, 4>	一维通信需要接收的位置序号
haloIdxSend2	array<array<Index, 4>, 6>	二维通信需要发送的位置序号
haloIdxRecv2	array<array<Index, 4>, 6>	二维通信需要接收的位置序号
haloIdxSend3	array<array<Index, 8>, 4>	三维通信需要发送的位置序号
haloIdxRecv3	array<array<Index, 8>, 4>	三维通信需要接收的位置序号
haloIdxSend4	array<Index, 16>	四维通信需要发送的位置序号
haloIdxRecv4	array<Index, 16>	四维通信需要接收的位置序号

8.3.2.1.4 SpinorField

SpinorField为旋量场，属于模板类，模板的具体内容如表SpinorField模板所示

 版权所有 © 华为技术有限公司

表 8-66 SpinorField 模板

模板名	类型	描述
T	typename	旋量场的数据类型，取值可为float,double
Nbatch	int	x方向批量大小，对于float可为1或16，对于double为1
Ns	int	旋量场分量数，对于Wilson型费米子为4，对于Stagger型费米子为1

SpinorField的参数的具体内容如表SpinorField参数所示

表 8-67 SpinorField 参数

变量名	类型	描述
latBase	LattBase &	所用到的格点
data	ColorSpinor<T,Nbatch>	数据
locSize	int	每个时空格点储存浮点数个数
length	size_t	总时空格点数，考虑奇偶格点

8.3.2.1.5 GaugeField

GaugeField为规范场，属于模板类，模板的具体内容如表GaugeField模板所示

表 8-68 GaugeField 模板

模板名	类型	描述
T	typename	规范场的数据类型，取值可为float,double
Nbatch	int	x方向批量大小，对于float可为1或16，对于double为1
Recon	KuQCDGaugeReconstruct	SU(3)矩阵压缩方案，目前仅可取值KUQCD_RECON_NO

GaugeField的参数的具体内容如表GaugeField参数所示

 版权所有 © 华为技术有限公司

表 8-69 GaugeField 参数

变量名	类型	描述
even	SU3<T, Nbatch, Recon>	指向规范场的偶部分
odd	SU3<T, Nbatch, Recon>	指向规范场的奇部分
comm	CommBase &	规范场通信
commHyperPlane	array<array<Communicator<GaugeT>, 2>, 4>	一维通信子
commPlane	array<array<Communicator<GaugeT>, 4>, 6>	二维通信子
commStripe	array<array<Communicator<GaugeT>, 8>, 4>	三维通信子
commCorner	array<Communicator<GaugeT>, 16>	四维通信子
latBase	LattBase &	所用到的时空格点
data	SU3<T, Nbatch, Recon>*	数据
locSize	int	每个时空格点储存浮点数个数
length	size_t	总时空格点数，全格点

8.3.2.2 函数定义

8.3.2.2.1 kuqcd_init

初始化

须知

程序需要在使用其他函数前调用该函数一次，且只允许调用一次，运行时需添加输入文件，若可执行文件为run.exe，输入文件为input，需用以下命令运行：mpirun <xxx mpi参数> ./run.exe < input

接口定义

KuQCDStatus kuqcd_init();

返回值

返回值	类型	描述
KUQCD_SUCCESS	int	执行成功

版权所有 © 华为技术有限公司

返回值	类型	描述
KUQCD_ERR_INVALID_PARAM	int	无效参数
KUQCD_ERR_UNSUPPORTED	int	不支持的选项
KUQCD_ERR_NULL_POINTER	int	空指针

8.3.2.2.2 kuqcd_load_gaugefield

加载规范场对象

接口定义

KuQCDStatus kuqcd_load_gaugefield(void *h_gauge, KuQCDGaugeParam *param);

参数

参数名	类型	描述	输入/输出
h_gauge	void *	规范场的数据	输入
param	KuQCDGaugeParam *	规范场的配置选项，具体内容如表**表8-70**所示	输入

表 8-70 KuQCDGaugeParam 结构体字段描述

字段名	类型	描述
lat	unsigned int [4]	x,y,z,t方向格点数
halo_depth	int	halo区大小，对于旋量场取0；对于规范场，若做rhmc，取3，只做inverter，取0。
od	KuQCDRankOrder	进程排布顺序，可取KUQCD_XYZT_ORDER（x方向快变）或KUQCD_TZYX_ORDER（t方向快变）
site_subset	KuQCDSiteSubset	子格点，可取值KUQCD_EVEN, KUQCD_ODD, KUQCD_ALL

版权所有 © 华为技术有限公司

字段名	类型	描述
gauge_order	KuQCDGaugeOrder	数据排布顺序，可取KUQCD_NORMAL_ORDER（对于旋量场(even-odd)-y-z-y-x-Ns-3-(real-imag)，对于规范场(even-odd)-t-z-y-x-direction-col-row-(real-imag)）和KUQCD_BATCH_ORDER（对于旋量场(even-odd)-y-z-y-(x/Nbatch)-Ns-3-(real*Nbatch-imag*Nbatch)，对于规范场(even-odd)-t-z-y-(x/Nbatch)-direction-col-row-(real*Nbatch-imag*Nbatch)）
prec	KuQCDPrecision	精度，可取KUQCD_SINGLE_PRECISION和KUQCD_DOUBLE_PRECISION
link_type	KuQCDLinkType	链接类型，取值有 • KUQCD_WILSON_LINK（暂不支持） • KUQCD_FAT_LINK • KUQCD_LONG_LINK
recon	KuQCDReconstructType	SU(3)矩阵压缩方式，取值有 • KUQCD_GAUGE_RECON_NO • KUQCD_GAUGE_RECON_U14（暂不支持） • KUQCD_GAUGE_RECON_12（暂不支持） • KUQCD_GAUGE_RECON_8（暂不支持）

返回值

返回值	类型	描述
KUQCD_SUCCESS	int	执行成功

 版权所有 © 华为技术有限公司

返回值	类型	描述
KUQCD_ERR_INVALID_PARAM	int	无效参数
KUQCD_ERR_UNSUPPORTED	int	不支持的选项
KUQCD_ERR_NULL_POINTER	int	空指针

8.3.2.2.3 kuqcd_solve_multishift

进行求解

接口定义

KuQCDStatus kuqcd_solve_multishift(void **_hp_x, void *_hp_b, KuQCDInvertParam *param);

参数

参数名	类型	描述	输入/输出
_hp_x	void **	指向多偏移的解数据	输入/输出
_hp_b	void *	指向源数据	输入
param	KuQCDInvertParam *	求解进行配置选项，具体内容如表**表8-71**所示	输入

表 8-71 KuQCDInvertParam 结构体字段描述

字段名	类型	描述
dslash_type	KuQCDDslashType	狄拉克算子类型，取值有 • KUQCD_WILSON_DIRAC（暂不支持） • KUQCD_WILSONPC_DIRAC（暂不支持） • KUQCD_HISQ_DIRAC • KUQCD_HISQPC_DIRAC

版权所有 © 华为技术有限公司

字段名	类型	描述
inverter_type	KuQCDInverterType	求解器类型，取值有 ● KUQCD_CG（暂不支持） ● KUQCD_CGM ● KUQCD_BICGSTAB（暂不支持） ● KUQCD_BICGSTABM（暂不支持）
matpc_type	KuQCDMatPCType	预条件子类型，取值有 ● KUQCD_EVEN_EVEN ● KUQCD_ODD_ODD
prec	KuQCDPrecision	求解精度，需要和KuQCDGaugeParam参数的值保持一致，取值有 ● KUQCD_HALF（暂不支持） ● KUQCD_SINGLE ● KUQCD_DOUBLE
iter	int	迭代次数，求解完之后读取该值可获取求解迭代的次数
num_shift	int	解的个数
max_iter	int	最大迭代次数
tol	double	收敛精度
shifts	double *	偏移量数组

8.3.2.2.4 kuqcd_gauge_free

释放规范场对象

接口定义

```
void kuqcd_gauge_free();
```

8.3.2.2.5 kuqcd_end

求解结束

接口定义

```
void kuqcd_end();
```

 版权所有 © 华为技术有限公司

8.3.2.2.6 示例

```
#include <omp.h>
#include <mpi.h>
#include <cstdio>
#include "kuqcd.h"

using namespace kuqcd;

template<typename T, int Nbatch, KuQCDReconstructType ReconS, KuQCDReconstructType ReconN>
void TestCGMPC(const KuQCDGaugeOrder gauge_order);


int main (int argc, char** argv)
{
   int provided = 0;
   int required = MPI_THREAD_FUNNELED;
   int flag = MPI_Init_thread(&argc, &argv, required, &provided);

   TestCGMPC<float, 1, KUQCD_GAUGE_RECON_NO,
KUQCD_GAUGE_RECON_NO>(KUQCD_NORMAL_ORDER);
   MPI_Finalize();
   return 0;
}


template<typename T, int Nbatch, KuQCDReconstructType ReconS, KuQCDReconstructType ReconN>
void TestCGMPC(const KuQCDGaugeOrder gauge_order)
{
   int repos = 1;
   int nx = 4, ny = 4, nz = 4, nt = 1, seed = 1;
   const int num_shift = 1;
   std::array<unsigned short, 4> gridDim{1, 1, 1, 1};
   KuQCDRankOrder od = KUQCD_TZYX_ORDER;
   KuQCDSiteSubset site_subset = KUQCD_ODD;
   int haloDepthSpin = 0;
   int haloDepthGauge = 3;

   Lattice latSup(nx, ny, nz, nt);

   KuQCDPrecision prec = (sizeof(T)==sizeof(double)) ? KUQCD_DOUBLE : KUQCD_SINGLE;

   rootLogger.Info("initializing spinor field lattice base");
   LattBase *latBaseOdd = nullptr;
   KuQCDGaugeParam Gagugeoptions;
   Gagugeoptions.lat[0] = nx;
   Gagugeoptions.lat[1] = ny;
   Gagugeoptions.lat[2] = nz;
   Gagugeoptions.lat[3] = nt;
   Gagugeoptions.gauge_order = gauge_order;
   Gagugeoptions.od = od;
   Gagugeoptions.prec = prec;
   site_subset = KUQCD_ALL;
   Gagugeoptions.site_subset = site_subset;
   Gagugeoptions.halo_depth = haloDepthGauge;


   double shifts[num_shift];
   shifts[0] = 3.67167631338816e-05;
   FILE* fp;
   float* origin;
   int np = GetNp();
   int rk = GetRank();

   size_t vol = (size_t)nx *ny * nt *nz / Nbatch / (np);
   size_t volh = vol / 2;
   size_t spinorLen = volh* Nbatch * 6;
   size_t gaugeLen = vol* Nbatch * 4 * ReconS;
   size_t gaugeLenNaik = vol* Nbatch * 4 * ReconN;
```


版权所有 © 华为技术有限公司

```
  printf("vol is %zu, spinorLen %zu, gaugeLen %zu\n",vol,spinorLen,gaugeLen);
  origin = (float*)malloc(((num_shift + 1)*spinorLen+gaugeLen+gaugeLenNaik)*sizeof(float));
  char name[100];

  for(int i = 0; i<(num_shift + 1)*spinorLen+gaugeLen+gaugeLenNaik;i++) {
      origin[i] = 1.0;
  }
  kuqcd_init();
  float *originSrc = origin;
  float *origingaugeSmeared = &(origin[(num_shift+1)*spinorLen]);
  float *origingaugeNaik = &(origin[(num_shift+1)*spinorLen+gaugeLen]);

  rootLogger.Info("SpinorFieldSetValues1 done");
  Gagugeoptions.link_type = KUQCD_FAT_LINK;
  Gagugeoptions.recon = ReconS;
  kuqcd_load_gaugefield((void*)origingaugeSmeared, &Gagugeoptions);
  Gagugeoptions.link_type = KUQCD_LONG_LINK;
  Gagugeoptions.recon = ReconN;
  kuqcd_load_gaugefield((void*)origingaugeNaik, &Gagugeoptions);

  rootLogger.Info("initializing SetValues done");

  KuQCDInvertParam invertOpts;
  invertOpts.dslash_type = KUQCD_HISQPC_DIRAC;
  invertOpts.inverter_type = KUQCD_CGM;
  invertOpts.matpc_type = KUQCD_ODD_ODD;
  int iter = 0;
  invertOpts.num_shift = num_shift;
  invertOpts.max_iter = 6500;
  invertOpts.tol = 1e-12;
  invertOpts.shifts = shifts;
  invertOpts.prec =prec;
  void *dst[num_shift] = {nullptr};
  float *dst_value[num_shift];
  for(int i = 0;i<num_shift;i++) {
      dst_value[i] = (float*)malloc(spinorLen * sizeof(float));
      dst[i] = (void *)(dst_value[i]);
  }

  double tStart = MPI_Wtime();

  kuqcd_solve_multishift(dst, (void *)originSrc, &invertOpts);
  double tEnd = MPI_Wtime();
  Rank0Printf("done\n");
  fflush(0);
  Rank0Printf("total time =  %e [s]\n", tEnd - tStart);
  Rank0Printf("time/iter  =  %e [s]\n", (tEnd - tStart) / invertOpts.iter);
  fflush(stdout);



}
```

 版权所有 © 华为技术有限公司

9 高性能计算开发工具

高性能计算集群检查可对高性能计算物理机或虚拟机集群各节点，进行节点间软件、硬件配置的一致性检查，并给出优化建议。请参见**《高性能计算集群检查》**

使用高性能计算开发工具可以采集系统的PMU事件并配合采集面向OpenMP和MPI应用的关键指标，帮助用户精准获得Parallel region及Barrier-to-Barrier的串行及并行时间、校准的2层微架构指标、指令分布及L3的利用率和内存带宽等相关信息。请参见**《高性能计算应用分析》**

Roofline分析可帮助用户在给定硬件平台上，分析出应用程序的瓶颈点位置，从而有针对性的进行优化。请参见**《Roofline分析》**

 版权所有 © 华为技术有限公司

10 修订记录

文档版本	发布日期	修改说明
01	2026-06-30	第一次正式发布。

版权所有 © 华为技术有限公司